HPX Documentation

2.0.0

The STE||AR Group

September 11, 2026

USER DOCUMENTATION

Welcome to the *HPX* documentation!

If you're new to *HPX* you can get started with the *Quick start* guide. Don't forget to read the *Terminology* section to learn about the most important concepts in *HPX*. The *Examples* give you a feel for how it is to write real *HPX* applications and the *Manual* contains detailed information about everything from building *HPX* to debugging it. There are links to blog posts and videos about *HPX* in *Additional material*.

You can find a comprehensive list of contact options on Support for deploying and using HPX[1]. Do not hesitate to contact us if you can't find what you are looking for in the documentation!

See *Citing HPX* for details on how to cite *HPX* in publications. See *HPX users* for a list of institutions and projects using *HPX*.

There are also available a PDF version of this documentation as well as a Single HTML Page.

[1] https://github.com/TheHPXProject/hpx/blob/master/.github/SUPPORT.md

CHAPTER ONE

WHAT IS *HPX*?

HPX is a C++ Standard Library for Concurrency and Parallelism. It implements all of the corresponding facilities as defined by the C++ Standard. Additionally, in *HPX* we implement functionalities proposed as part of the ongoing C++ standardization process. We also extend the C++ Standard APIs to the distributed case. *HPX* is developed by the STE||AR group (see *People*).

The goal of *HPX* is to create a high quality, freely available, open source implementation of a new programming model for conventional systems, such as classic Linux based Beowulf clusters or multi-socket highly parallel SMP nodes. At the same time, we want to have a very modular and well designed runtime system architecture which would allow us to port our implementation onto new computer system architectures. We want to use real-world applications to drive the development of the runtime system, coining out required functionalities and converging onto a stable API which will provide a smooth migration path for developers.

The API exposed by *HPX* is not only modeled after the interfaces defined by the C++11/14/17/20 ISO standard. It also adheres to the programming guidelines used by the Boost collection of C++ libraries. We aim to improve the scalability of today's applications and to expose new levels of parallelism which are necessary to take advantage of the exascale systems of the future.

HPX is a member of the High-Performance Software Foundation (HPSF[2]) network, which signals its commitment to the open-source, high-performance computing (HPC) community. By joining HPSF[3] - and, concurrently, the Linux Foundation - HPX gains access to a broader collaborative ecosystem that promotes shared standards, interoperable tools, and joint research initiatives, all of which accelerates the development of scalable parallel applications. This dual affiliation enhances HPX's credibility, attracting contributors and industry partners who value the rigorous governance and transparent development practices upheld by both organizations. Consequently, HPX can more effectively influence the direction of future HPC frameworks while benefiting from the collective expertise, resources, and industry backing provided by HPSF[4] and the Linux Foundation.

[2] https://hpsf.io/projects/hpx/
[3] https://hpsf.io/projects/hpx/
[4] https://hpsf.io/projects/hpx/

CHAPTER

TWO

WHAT'S SO SPECIAL ABOUT *HPX*?

- HPX exposes a uniform, standards-oriented API for ease of programming parallel and distributed applications.
- It enables programmers to write fully asynchronous code using hundreds of millions of threads.
- HPX provides unified syntax and semantics for local and remote operations.
- HPX makes concurrency manageable with dataflow and future based synchronization.
- It implements a rich set of runtime services supporting a broad range of use cases.
- HPX exposes a uniform, flexible, and extendable performance counter framework which can enable runtime adaptivity
- It is designed to solve problems conventionally considered to be scaling-impaired.
- HPX has been designed and developed for systems of any scale, from hand-held devices to very large scale systems.
- It is the first fully functional implementation of the ParalleX execution model.
- HPX is published under a liberal open-source license and has an open, active, and thriving developer community.

2.1 Quick start

The following steps will help you get started with *HPX*. Before getting started, make sure you have all the necessary prerequisites, which are listed in _prerequisites. After *Installing HPX*, you can check how to run a simple example *Hello, World!*. *Writing task-based applications* explains how you can get started with *HPX*. You can refer to our *Migration guide* if you use other APIs for parallelism (like OpenMP, MPI or Intel Threading Building Blocks (TBB)) and you would like to convert your code to *HPX* code.

2.1.1 Installing *HPX*

The easiest way to install *HPX* on your system is by choosing one of the steps below:

1. **vcpkg**

   You can download and install *HPX* using the vcpkg[5] dependency manager:

   ```
   $ vcpkg install hpx
   ```

2. **Spack**

   Another way to install *HPX* is using Spack[6]:

[5] https://github.com/Microsoft/vcpkg
[6] https://spack.readthedocs.io/en/latest/

```
$ spack install hpx
```

3. **Conan**

   You can also use the Conan[7] package manager to build and use *HPX* in your projects. First, export the *HPX* recipe from the *HPX* repository to your local Conan cache:

```
$ git clone https://github.com/TheHPXProject/hpx.git
$ cd hpx
$ conan create . --build=missing
```

   This builds *HPX* and makes it available in your local Conan cache. You can then use it in your Conan-based projects by adding it to your `conanfile.txt`:

```
[requires]
hpx/2.0.0

[generators]
CMakeDeps
CMakeToolchain
```

   Then use `find_package(HPX REQUIRED)` in your `CMakeLists.txt` as usual. Conan will automatically handle building *HPX* with your chosen configuration options and provide the necessary CMake integration files.

4. **Fedora**

   Installation can be done with Fedora[8] as well:

```
$ dnf install hpx*
```

5. **Arch Linux**

   HPX is available in the Arch User Repository (AUR)[9] as `hpx` too.

More information or alternatives regarding the installation can be found in the *Building HPX*, a detailed guide with thorough explanation of ways to build and use *HPX*.

2.1.2 Hello, World!

To get started with this minimal example you need to create a new project directory and a file `CMakeLists.txt` with the contents below in order to build an executable using CMake[10] and *HPX*:

```
cmake_minimum_required(VERSION 3.19)
project(my_hpx_project CXX)
find_package(HPX REQUIRED)
add_executable(my_hpx_program main.cpp)
target_link_libraries(my_hpx_program HPX::hpx HPX::wrap_main hpx::iostreams_component)
```

The next step is to create a `main.cpp` with the contents below:

[7] https://conan.io/
[8] https://fedoraproject.org/wiki/DNF
[9] https://wiki.archlinux.org/title/Arch_User_Repository
[10] https://www.cmake.org

```cpp
// Including 'hpx/hpx_main.hpp' instead of the usual 'hpx/hpx_init.hpp' enables
// to use the plain C-main below as the direct main HPX entry point.
#include <hpx/hpx_main.hpp>
#include <hpx/iostream.hpp>

int main()
{
    // Say hello to the world!
    hpx::cout << "Hello World!\n" << std::flush;
    return 0;
}
```

Then, in your project directory run the following:

```
$ mkdir build && cd build
$ cmake -DHPX_DIR=</path/to/hpx/installation> ..
$ make all
$ ./my_hpx_program
```

```
$ ./my_hpx_program
Hello World!
```

The program looks almost like a regular C++ hello world with the exception of the two includes and `hpx::cout`.

- When you include `hpx_main.hpp` *HPX* makes sure that `main` actually gets launched on the *HPX* runtime. So while it looks almost the same you can now use futures, `async`, parallel algorithms and more which make use of the *HPX* runtime with lightweight threads.
- `hpx::cout` is a replacement for `std::cout` to make sure printing never blocks a lightweight thread. You can read more about `hpx::cout` in *The HPX I/O-streams component*.

Note

- You will most likely have more than one `main.cpp` file in your project. See the section on *Using HPX with CMake-based projects* for more details on how to use `add_hpx_executable`.
- `HPX::wrap_main` is required if you are implicitly using `main()` as the runtime entry point. See *Re-use the main() function as the main HPX entry point* for more information.
- `hpx::iostreams_component` is optional for a minimal project but lets us use the *HPX* equivalent of `std::cout`, i.e., the *HPX The HPX I/O-streams component* functionality in our application.
- You do not have to let *HPX* take over your main function like in the example. See *Starting the HPX runtime* for more details on how to initialize and run the *HPX* runtime.

Caution

Ensure that *HPX* is installed with `HPX_WITH_DISTRIBUTED_RUNTIME=ON` to prevent encountering an error indicating that the `HPX::iostreams_component` target is not found.

When including `hpx_main.hpp` the user-defined `main` gets renamed and the real `main` function is defined by *HPX*. This means that the user-defined `main` must include a return statement, unlike the real `main`. If you do not include the return statement, you may end up with confusing compile time errors mentioning `user_main` or even runtime errors.

2.1.3 Writing task-based applications

So far we haven't done anything that can't be done using the C++ standard library. In this section we will give a short overview of what you can do with *HPX* on a single node. The essence is to avoid global synchronization and break up your application into small, composable tasks whose dependencies control the flow of your application. Remember, however, that *HPX* allows you to write distributed applications similarly to how you would write applications for a single node (see *Why HPX?* and *Writing distributed applications*).

If you are already familiar with `async` and `future` from the C++ standard library, the same functionality is available in *HPX*.

The following terminology is essential when talking about task-based C++ programs:

- lightweight thread: Essential for good performance with task-based programs. Lightweight refers to smaller stacks and faster context switching compared to OS threads. Smaller overheads allow the program to be broken up into smaller tasks, which in turns helps the runtime fully utilize all processing units.
- `async`: The most basic way of launching tasks asynchronously. Returns a `future<T>`.
- `future<T>`: Represents a value of type `T` that will be ready in the future. The value can be retrieved with `get` (blocking) and one can check if the value is ready with `is_ready` (non-blocking).
- `shared_future<T>`: Same as `future<T>` but can be copied (similar to `std::unique_ptr` vs `std::shared_ptr`).
- continuation: A function that is to be run after a previous task has run (represented by a future). `then` is a method of `future<T>` that takes a function to run next. Used to build up dataflow DAGs (directed acyclic graphs). `shared_future`s help you split up nodes in the DAG and functions like `when_all` help you join nodes in the DAG.

The following example is a collection of the most commonly used functionality in *HPX*:

```
#include <hpx/algorithm.hpp>
#include <hpx/future.hpp>
#include <hpx/init.hpp>

#include <iostream>
#include <random>
#include <vector>

void final_task(hpx::future<hpx::tuple<hpx::future<double>, hpx::future<void>>>)
{
    std::cout << "in final_task" << std::endl;
}

int hpx_main()
{
    // A function can be launched asynchronously. The program will not block
    // here until the result is available.
    hpx::future<int> f = hpx::async([]() { return 42; });
    std::cout << "Just launched a task!" << std::endl;

    // Use get to retrieve the value from the future. This will block this task
    // until the future is ready, but the HPX runtime will schedule other tasks
    // if there are tasks available.
    std::cout << "f contains " << f.get() << std::endl;
```

(continues on next page)

(continued from previous page)

```
    // Let's launch another task.
    hpx::future<double> g = hpx::async([]() { return 3.14; });

    // Tasks can be chained using the then method. The continuation takes the
    // future as an argument.
    hpx::future<double> result = g.then([](hpx::future<double>&& gg) {
        // This function will be called once g is ready. gg is g moved
        // into the continuation.
        return gg.get() * 42.0 * 42.0;
    });

    // You can check if a future is ready with the is_ready method.
    std::cout << "Result is ready? " << result.is_ready() << std::endl;

    // You can launch other work in the meantime. Let's sort a vector.
    std::vector<int> v(1000000);

    // We fill the vector synchronously and sequentially.
    hpx::generate(hpx::execution::seq, std::begin(v), std::end(v), &std::rand);

    // We can launch the sort in parallel and asynchronously.
    hpx::future<void> done_sorting =
        hpx::sort(hpx::execution::par(          // In parallel.
                      hpx::execution::task),    // Asynchronously.
            std::begin(v), std::end(v));

    // We launch the final task when the vector has been sorted and result is
    // ready using when_all.
    auto all = hpx::when_all(result, done_sorting).then(&final_task);

    // We can wait for all to be ready.
    all.wait();

    // all must be ready at this point because we waited for it to be ready.
    std::cout << (all.is_ready() ? "all is ready!" : "all is not ready...")
              << std::endl;

    return hpx::local::finalize();
}

int main(int argc, char* argv[])
{
    return hpx::local::init(hpx_main, argc, argv);
}
```

Try copying the contents to your `main.cpp` file and look at the output. It can be a good idea to go through the program step by step with a debugger. You can also try changing the types or adding new arguments to functions to make sure you can get the types to match. The type of the `then` method can be especially tricky to get right (the continuation needs to take the future as an argument).

> **Note**
>
> *HPX* programs accept command line arguments. The most important one is `--hpx:threads=N` to set the number of OS threads used by *HPX*. *HPX* uses one thread per core by default. Play around with the example above and see what difference the number of threads makes on the `sort` function. See *Launching and configuring HPX applications* for more details on how and what options you can pass to *HPX*.

> **Tip**
>
> The example above used the construction `hpx::when_all(...).then(...)`. For convenience and performance it is a good idea to replace uses of `hpx::when_all(...).then(...)` with `dataflow`. See *Dataflow* for more details on `dataflow`.

> **Tip**
>
> If possible, try to use the provided parallel algorithms instead of writing your own implementation. This can save you time and the resulting program is often faster.

2.1.4 Next steps

If you haven't done so already, reading the *Terminology* section will help you get familiar with the terms used in *HPX*.

The *Examples* section contains small, self-contained walkthroughs of example *HPX* programs. The *Local to remote* example is a thorough, realistic example starting from a single node implementation and going stepwise to a distributed implementation.

The *Manual* contains detailed information on writing, building and running *HPX* applications.

2.2 Tutorials

This section provides structured, video-based learning resources for *HPX*. The tutorials complement the reference documentation with guided, example-driven explanations of core concepts and advanced features.

Each tutorial introduces the relevant concepts and then applies them step by step through practical code examples. The focus is on demonstrating how to design, implement, and reason about parallel and distributed applications using *HPX*. While the main documentation emphasizes API descriptions and detailed reference material, the tutorial series adopts a practical, code-oriented approach.

The tutorials are intended for a broad audience: from beginners setting up HPX for the first time to advanced users exploring performance tuning, custom executors, and SIMD vectorization. Together, they form a structured learning path that lowers the entry barrier while progressively introducing more advanced features.

The complete video series is available on YouTube:

HPX Tutorial Playlist[11]

The code is available on Github:

HPX Tutorial Code <https://github.com/STEllAR-GROUP/HPX_Tutorials_Code>

[11] https://www.youtube.com/watch?v=OZWTmrlWUF0&list=PL7vEgTL3FalZkvNqssEP9-tXQ5rEwZy2l&index=1

2.2.1 Available Modules

- **Introduction to |hpx|**
  Foundational overview of *HPX*, including its architecture, design principles, and key capabilities in the context of high-performance computing.
- **Building |hpx| on Windows/Unix**
  Step-by-step guidance on installing *HPX* and setting up the environment based on your operating system.
- **Running |hpx| applications**
  Demonstation on how to build and run a simple Hello World example using *HPX*.
- **Algorithms in |hpx|**
  Overview of the parallel algorithms provided by *HPX* and explanation on how execution policies unlock high-performance C++ computing.
- **Vectorization and SIMD in |hpx|**
  Introduction to data-parallel programming, SIMD execution policies, and vectorization techniques relevant to modern C++ and HPC workloads.
- **Futures and Asynchronous Programming**
  Explanation of the asynchronous programming model in *HPX*, including futures, *async*, and dataflow-based execution patterns.
- **Performance Analysis and Optimization**
  Guidance on profiling *HPX* applications, analyzing performance counters, identifying bottlenecks, and applying optimization strategies.
- **Task Scheduling and Custom Executors**
  Detailed discussion of the execution model in *HPX*, built-in executors, and mechanisms for defining custom execution strategies.
- **Migration Guides**
  Best practices and recommendations for transitioning from other HPC frameworks to *HPX*.
- **HPX in Practice**
  Examples of open-source libraries and applications using *HPX*, illustrating real-world usage patterns and ecosystem integration.

2.3 Examples

The following sections analyze some examples to help you get familiar with the *HPX* style of programming. We start off with simple examples that utilize basic *HPX* elements and then begin to expose the reader to the more complex and powerful *HPX* concepts. Section *Building tests and examples* shows how you can build the examples.

2.3.1 Asynchronous execution

The Fibonacci sequence is a sequence of numbers starting with 0 and 1 where every subsequent number is the sum of the previous two numbers. In this example, we will use *HPX* to calculate the value of the n-th element of the Fibonacci sequence. In order to compute this problem in parallel, we will use a facility known as a future.

As shown in the Fig. **??** below, a future encapsulates a delayed computation. It acts as a proxy for a result initially not known, most of the time because the computation of the result has not completed yet. The future synchronizes the access of this value by optionally suspending any *HPX*-threads requesting the result until the value is available. When a future is created, it spawns a new *HPX*-thread (either remotely with a *parcel* or locally by placing it into the thread queue) which, when run, will execute the function associated with the future. The arguments of the function are bound when the future is created.

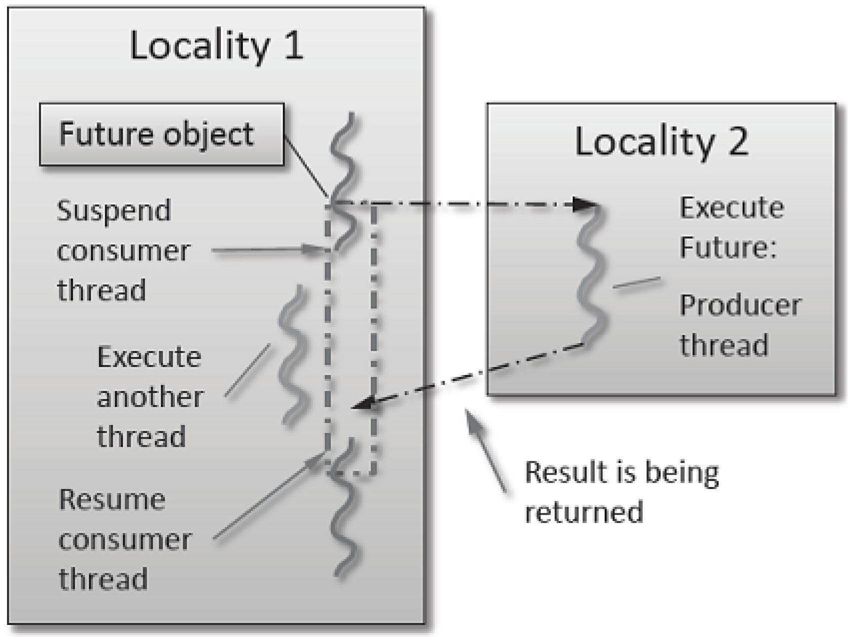


Fig. 2.1: Schematic of a future execution.

Once the function has finished executing, a write operation is performed on the future. The write operation marks the future as completed, and optionally stores data returned by the function. When the result of the delayed computation is needed, a read operation is performed on the future. If the future's function hasn't completed when a read operation is performed on it, the reader *HPX*-thread is suspended until the future is ready. The future facility allows *HPX* to schedule work early in a program so that when the function value is needed it will already be calculated and available. We use this property in our Fibonacci example below to enable its parallel execution.

Setup

The source code for this example can be found here: `fibonacci_local.cpp`.

To compile this program, go to your *HPX* build directory (see *Building HPX* for information on configuring and building *HPX*) and enter:

```
$ make examples.quickstart.fibonacci_local
```

To run the program type:

```
$ ./bin/fibonacci_local
```

This should print (time should be approximate):

```
fibonacci(10) == 55
elapsed time: 0.002430 [s]
```

This run used the default settings, which calculate the tenth element of the Fibonacci sequence. To declare which Fibonacci value you want to calculate, use the `--n-value` option. Additionally you can use the `--hpx:threads` option to declare how many OS-threads you wish to use when running the program. For instance, running:

```
$ ./bin/fibonacci --n-value 20 --hpx:threads 4
```

Will yield:

```
fibonacci(20) == 6765
elapsed time: 0.062854 [s]
```

Walkthrough

Now that you have compiled and run the code, let's look at how the code works. Since this code is written in C++, we will begin with the `main()` function. Here you can see that in *HPX*, `main()` is only used to initialize the runtime system. It is important to note that application-specific command line options are defined here. *HPX* uses Boost.Program_options[12] for command line processing. You can see that our programs `--n-value` option is set by calling the `add_options()` method on an instance of `hpx::program_options::options_description`. The default value of the variable is set to 10. This is why when we ran the program for the first time without using the `--n-value` option the program returned the 10th value of the Fibonacci sequence. The constructor argument of the description is the text that appears when a user uses the `--hpx:help` option to see what command line options are available. `HPX_APPLICATION_STRING` is a macro that expands to a string constant containing the name of the *HPX* application currently being compiled.

In *HPX* `main()` is used to initialize the runtime system and pass the command line arguments to the program. If you wish to add command line options to your program you would add them here using the instance of the Boost class `options_description`, and invoking the public member function `.add_options()` (see Boost Documentation[13] for more details). *hpx::init* calls `hpx_main()` after setting up *HPX*, which is where the logic of our program is encoded.

```
int main(int argc, char* argv[])
{
    // Configure application-specific options
    hpx::program_options::options_description desc_commandline(
        "Usage: " HPX_APPLICATION_STRING " [options]");

    // clang-format off
    desc_commandline.add_options()
        ("n-value",
            hpx::program_options::value<std::uint64_t>()->default_value(10),
            "n value for the Fibonacci function")
        ;
    // clang-format on

    // Initialize and run HPX
    hpx::local::init_params init_args;
    init_args.desc_cmdline = desc_commandline;

    return hpx::local::init(hpx_main, argc, argv, init_args);
}
```

The *hpx::init* function in `main()` starts the runtime system, and invokes `hpx_main()` as the first *HPX*-thread. Below we can see that the basic program is simple. The command line option `--n-value` is read in, a timer (*hpx::chrono::high_resolution_timer*) is set up to record the time it takes to do the computation, the `fibonacci` function is invoked synchronously, and the answer is printed out.

[12] https://www.boost.org/doc/html/program_options.html
[13] https://www.boost.org/doc/

```
int hpx_main(hpx::program_options::variables_map& vm)
{
    hpx::threads::add_scheduler_mode(
        hpx::threads::policies::scheduler_mode::fast_idle_mode);

    // extract command line argument, i.e. fib(N)
    std::uint64_t n = vm["n-value"].as<std::uint64_t>();

    {
        // Keep track of the time required to execute.
        hpx::chrono::high_resolution_timer t;

        std::uint64_t r = fibonacci(n);

        char const* fmt = "fibonacci({1}) == {2}\nelapsed time: {3} [s]\n";
        hpx::util::format_to(std::cout, fmt, n, r, t.elapsed());
    }

    return hpx::local::finalize();    // Handles HPX shutdown
}
```

The `fibonacci` function itself is synchronous as the work done inside is asynchronous. To understand what is happening we have to look inside the `fibonacci` function:

```
std::uint64_t fibonacci(std::uint64_t n)
{
    if (n < 2)
        return n;

    hpx::future<std::uint64_t> n1 = hpx::async(fibonacci, n - 1);
    std::uint64_t n2 = fibonacci(n - 2);

    return n1.get() + n2;    // wait for the Future to return their values
}
```

This block of code looks similar to regular C++ code. First, `if (n < 2)`, meaning n is 0 or 1, then we return 0 or 1 (recall the first element of the Fibonacci sequence is 0 and the second is 1). If n is larger than 1 we spawn two new tasks whose results are contained in `n1` and `n2`. This is done using *hpx::async* which takes as arguments a function (function pointer, object or lambda) and the arguments to the function. Instead of returning a `std::uint64_t` like `fibonacci` does, `hpx::async` returns a future of a `std::uint64_t`, i.e. `hpx::future<std::uint64_t>`. Each of these futures represents an asynchronous, recursive call to `fibonacci`. After we've created the futures, we wait for both of them to finish computing, we add them together, and return that value as our result. We get the values from the futures using the `get` method. The recursive call tree will continue until n is equal to 0 or 1, at which point the value can be returned because it is implicitly known. When this termination condition is reached, the futures can then be added up, producing the n-th value of the Fibonacci sequence.

Note that calling `get` potentially blocks the calling *HPX*-thread, and lets other *HPX*-threads run in the meantime. There are, however, more efficient ways of doing this. `examples/quickstart/fibonacci_futures.cpp` contains many more variations of locally computing the Fibonacci numbers, where each method makes different tradeoffs in where asynchrony and parallelism is applied. To get started, however, the method above is sufficient and optimizations can be applied once you are more familiar with *HPX*. The example *Dataflow* presents dataflow, which is a way to more efficiently chain together multiple tasks.

2.3.2 Parallel algorithms

This program will perform a matrix multiplication in parallel. The output will look something like this:

```
Matrix A is :
4 9 6
1 9 8

Matrix B is :
4 9
6 1
9 8

Resultant Matrix is :
124 93
130 82
```

Setup

The source code for this example can be found here: `matrix_multiplication.cpp`.

To compile this program, go to your *HPX* build directory (see *Building HPX* for information on configuring and building *HPX*) and enter:

```
$ make examples.quickstart.matrix_multiplication
```

To run the program type:

```
$ ./bin/matrix_multiplication
```

or:

```
$ ./bin/matrix_multiplication --n 2 --m 3 --k 2 --s 100 --l 0 --u 10
```

where the first matrix is *n* x *m* and the second *m* x *k*, s is the seed for creating the random values of the matrices and the range of these values is [l,u]

This should print:

```
Matrix A is :
4 9 6
1 9 8

Matrix B is :
4 9
6 1
9 8

Resultant Matrix is :
124 93
130 82
```

Notice that the numbers may be different because of the random initialization of the matrices.

Walkthrough

Now that you have compiled and run the code, let's look at how the code works.

First, `main()` is used to initialize the runtime system and pass the command line arguments to the program. `hpx::init` calls `hpx_main()` after setting up HPX, which is where our program is implemented.

```
int main(int argc, char* argv[])
{
    using namespace hpx::program_options;
    options_description cmdline("usage: " HPX_APPLICATION_STRING " [options]");
    // clang-format off
    cmdline.add_options()
        ("n",
        hpx::program_options::value<std::size_t>()->default_value(2),
        "Number of rows of first matrix")
        ("m",
        hpx::program_options::value<std::size_t>()->default_value(3),
        "Number of columns of first matrix (equal to the number of rows of "
        "second matrix)")
        ("k",
        hpx::program_options::value<std::size_t>()->default_value(2),
        "Number of columns of second matrix")
        ("seed,s",
        hpx::program_options::value<unsigned int>(),
        "The random number generator seed to use for this run")
        ("l",
        hpx::program_options::value<int>()->default_value(0),
        "Lower limit of range of values")
        ("u",
        hpx::program_options::value<int>()->default_value(10),
        "Upper limit of range of values");
    // clang-format on
    hpx::local::init_params init_args;
    init_args.desc_cmdline = cmdline;

    return hpx::local::init(hpx_main, argc, argv, init_args);
}
```

Proceeding to the `hpx_main()` function, we can see that matrix multiplication can be done very easily.

```
int hpx_main(hpx::program_options::variables_map& vm)
{
    using element_type = int;

    // Define matrix sizes
    std::size_t const rowsA = vm["n"].as<std::size_t>();
    std::size_t const colsA = vm["m"].as<std::size_t>();
    std::size_t const rowsB = colsA;
    std::size_t const colsB = vm["k"].as<std::size_t>();
    std::size_t const rowsR = rowsA;
    std::size_t const colsR = colsB;

    // Initialize matrices A and B
```

(continues on next page)

(continued from previous page)

```
    std::vector<int> A(rowsA * colsA);
    std::vector<int> B(rowsB * colsB);
    std::vector<int> R(rowsR * colsR);

    // Define seed
    unsigned int seed = std::random_device{}();
    if (vm.count("seed"))
        seed = vm["seed"].as<unsigned int>();

    gen.seed(seed);
    std::cout << "using seed: " << seed << std::endl;

    // Define range of values
    int const lower = vm["l"].as<int>();
    int const upper = vm["u"].as<int>();

    // Matrices have random values in the range [lower, upper]
    std::uniform_int_distribution<element_type> dis(lower, upper);
    auto generator = std::bind(dis, gen);
    hpx::ranges::generate(A, generator);
    hpx::ranges::generate(B, generator);

    // Perform matrix multiplication
    hpx::experimental::for_loop(hpx::execution::par, 0, rowsA, [&](auto i) {
        hpx::experimental::for_loop(0, colsB, [&](auto j) {
            R[i * colsR + j] = 0;
            hpx::experimental::for_loop(0, rowsB, [&](auto k) {
                R[i * colsR + j] += A[i * colsA + k] * B[k * colsB + j];
            });
        });
    });

    // Print all 3 matrices
    print_matrix(A, rowsA, colsA, "A");
    print_matrix(B, rowsB, colsB, "B");
    print_matrix(R, rowsR, colsR, "R");

    return hpx::local::finalize();
}
```

First, the dimensions of the matrices are defined. If they were not given as command-line arguments, their default values are *2* x *3* for the first matrix and *3* x *2* for the second. We use standard vectors to define the matrices to be multiplied as well as the resultant matrix.

To give some random initial values to our matrices, we use std::uniform_int_distribution[14]. Then, `std::bind()` is used along with `hpx::ranges::generate()` to yield two matrices A and B, which contain values in the range of [0, 10] or in the range defined by the user at the command-line arguments. The seed to generate the values can also be defined by the user.

The next step is to perform the matrix multiplication in parallel. This can be done by just using an *hpx::experimental::for_loop* combined with a parallel execution policy `hpx::execution::par` as the outer loop of the multiplication. Note that the execution of *hpx::experimental::for_loop* without specifying an execution policy is

[14] https://en.cppreference.com/w/cpp/numeric/random/uniform_int_distribution

equivalent to specifying `hpx::execution::seq` as the execution policy.

Finally, the matrices A, B that are multiplied as well as the resultant matrix R are printed using the following function.

```
void print_matrix(std::vector<int> const& M, std::size_t rows, std::size_t cols,
    char const* message)
{
    std::cout << "\nMatrix " << message << " is:" << std::endl;
    for (std::size_t i = 0; i < rows; i++)
    {
        for (std::size_t j = 0; j < cols; j++)
            std::cout << M[i * cols + j] << " ";
        std::cout << "\n";
    }
}
```

2.3.3 Asynchronous execution with actions

This example extends the *previous example* by introducing *actions*: functions that can be run remotely. In this example, however, we will still only run the action locally. The mechanism to execute *actions* stays the same: *hpx::async*. Later examples will demonstrate running actions on remote *localities* (e.g. *Remote execution with actions*).

Setup

The source code for this example can be found here: `fibonacci.cpp`.

To compile this program, go to your *HPX* build directory (see *Building HPX* for information on configuring and building *HPX*) and enter:

```
$ make examples.quickstart.fibonacci
```

To run the program type:

```
$ ./bin/fibonacci
```

This should print (time should be approximate):

```
fibonacci(10) == 55
elapsed time: 0.00186288 [s]
```

This run used the default settings, which calculate the tenth element of the Fibonacci sequence. To declare which Fibonacci value you want to calculate, use the `--n-value` option. Additionally you can use the `--hpx:threads` option to declare how many OS-threads you wish to use when running the program. For instance, running:

```
$ ./bin/fibonacci --n-value 20 --hpx:threads 4
```

Will yield:

```
fibonacci(20) == 6765
elapsed time: 0.233827 [s]
```

Walkthrough

The code needed to initialize the *HPX* runtime is the same as in the *previous example*:

```
int main(int argc, char* argv[])
{
    // Configure application-specific options
    hpx::program_options::options_description desc_commandline(
        "Usage: " HPX_APPLICATION_STRING " [options]");

    desc_commandline.add_options()("n-value",
        hpx::program_options::value<std::uint64_t>()->default_value(10),
        "n value for the Fibonacci function");

    // Initialize and run HPX
    hpx::init_params init_args;
    init_args.desc_cmdline = desc_commandline;

    return hpx::init(argc, argv, init_args);
}
```

The *hpx::init* function in `main()` starts the runtime system, and invokes `hpx_main()` as the first *HPX*-thread. The command line option `--n-value` is read in, a timer (*hpx::chrono::high_resolution_timer*) is set up to record the time it takes to do the computation, the `fibonacci` *action* is invoked synchronously, and the answer is printed out.

```
int hpx_main(hpx::program_options::variables_map& vm)
{
    // extract command line argument, i.e. fib(N)
    std::uint64_t n = vm["n-value"].as<std::uint64_t>();

    {
        // Keep track of the time required to execute.
        hpx::chrono::high_resolution_timer t;

        // Wait for fib() to return the value
        fibonacci_action fib;
        std::uint64_t r = fib(hpx::find_here(), n);

        char const* fmt = "fibonacci({1}) == {2}\nelapsed time: {3} [s]\n";
        hpx::util::format_to(std::cout, fmt, n, r, t.elapsed());
    }

    return hpx::finalize();    // Handles HPX shutdown
}
```

Upon a closer look we see that we've created a `std::uint64_t` to store the result of invoking our `fibonacci_action fib`. This *action* will launch synchronously (as the work done inside of the *action* will be asynchronous itself) and return the result of the Fibonacci sequence. But wait, what is an *action*? And what is this `fibonacci_action`? For starters, an *action* is a wrapper for a function. By wrapping functions, *HPX* can send packets of work to different processing units. These vehicles allow users to calculate work now, later, or on certain nodes. The first argument to our *action* is the location where the *action* should be run. In this case, we just want to run the *action* on the machine that we are currently on, so we use *`hpx::find_here`*. To further understand this we turn to the code to find where `fibonacci_action` was defined:

```
// forward declaration of the Fibonacci function
std::uint64_t fibonacci(std::uint64_t n);

// This is to generate the required boilerplate we need for the remote
// invocation to work.
HPX_PLAIN_ACTION(fibonacci, fibonacci_action)
```

A plain *action* is the most basic form of *action*. Plain *action*s wrap simple global functions which are not associated with any particular object (we will discuss other types of *action*s in *Components and actions*). In this block of code the function `fibonacci()` is declared. After the declaration, the function is wrapped in an *action* in the declaration *HPX_PLAIN_ACTION*. This function takes two arguments: the name of the function that is to be wrapped and the name of the *action* that you are creating.

This picture should now start making sense. The function `fibonacci()` is wrapped in an *action* `fibonacci_action`, which was run synchronously but created asynchronous work, then returns a `std::uint64_t` representing the result of the function `fibonacci()`. Now, let's look at the function `fibonacci()`:

```
std::uint64_t fibonacci(std::uint64_t n)
{
    if (n < 2)
        return n;

    // We restrict ourselves to execute the Fibonacci function locally.
    hpx::id_type const locality_id = hpx::find_here();

    // Invoking the Fibonacci algorithm twice is inefficient.
    // However, we intentionally demonstrate it this way to create some
    // heavy workload.

    fibonacci_action fib;
    hpx::future<std::uint64_t> n1 = hpx::async(fib, locality_id, n - 1);
    hpx::future<std::uint64_t> n2 = hpx::async(fib, locality_id, n - 2);

    return n1.get() +
        n2.get();    // wait for the Futures to return their values
}
```

This block of code is much more straightforward and should look familiar from the *previous example*. First, `if (n < 2)`, meaning n is 0 or 1, then we return 0 or 1 (recall the first element of the Fibonacci sequence is 0 and the second is 1). If n is larger than 1 we spawn two tasks using *hpx::async*. Each of these futures represents an asynchronous, recursive call to `fibonacci`. As previously we wait for both futures to finish computing, get the results, add them together, and return that value as our result. The recursive call tree will continue until n is equal to 0 or 1, at which point the value can be returned because it is implicitly known. When this termination condition is reached, the futures can then be added up, producing the n-th value of the Fibonacci sequence.

2.3.4 Remote execution with actions

This program will print out a hello world message on every OS-thread on every *locality*. The output will look something like this:

```
hello world from OS-thread 1 on locality 0
hello world from OS-thread 1 on locality 1
hello world from OS-thread 0 on locality 0
hello world from OS-thread 0 on locality 1
```

Setup

The source code for this example can be found here: `hello_world_distributed.cpp`.

To compile this program, go to your *HPX* build directory (see *Building HPX* for information on configuring and building *HPX*) and enter:

```
$ make examples.quickstart.hello_world_distributed
```

To run the program type:

```
$ ./bin/hello_world_distributed
```

This should print:

```
hello world from OS-thread 0 on locality 0
```

To use more OS-threads use the command line option `--hpx:threads` and type the number of threads that you wish to use. For example, typing:

```
$ ./bin/hello_world_distributed --hpx:threads 2
```

will yield:

```
hello world from OS-thread 1 on locality 0
hello world from OS-thread 0 on locality 0
```

Notice how the ordering of the two print statements will change with subsequent runs. To run this program on multiple localities please see the section *How to use HPX applications with PBS*.

Walkthrough

Now that you have compiled and run the code, let's look at how the code works, beginning with `main()`:

```
// Here is the main entry point. By using the include 'hpx/hpx_main.hpp' HPX
// will invoke the plain old C-main() as its first HPX thread.
int main()
{
    // Get a list of all available localities.
    std::vector<hpx::id_type> localities = hpx::find_all_localities();

    // Reserve storage space for futures, one for each locality.
    std::vector<hpx::future<void>> futures;
```

(continues on next page)

(continued from previous page)

```
    futures.reserve(localities.size());

    for (hpx::id_type const& node : localities)
    {
        // Asynchronously start a new task. The task is encapsulated in a
        // future, which we can query to determine if the task has
        // completed.
        typedef hello_world_foreman_action action_type;
        futures.push_back(hpx::async<action_type>(node));
    }

    // The non-callback version of hpx::wait_all takes a single parameter,
    // a vector of futures to wait on. hpx::wait_all only returns when
    // all of the futures have finished.
    hpx::wait_all(futures);
    return 0;
}
```

In this excerpt of the code we again see the use of futures. This time the futures are stored in a vector so that they can easily be accessed. *hpx::wait_all* is a family of functions that wait on for an `std::vector<>` of futures to become ready. In this piece of code, we are using the synchronous version of *`hpx::wait_all`*, which takes one argument (the `std::vector<>` of futures to wait on). This function will not return until all the futures in the vector have been executed.

In *Asynchronous execution with actions* we used *`hpx::find_here`* to specify the target of our actions. Here, we instead use *`hpx::find_all_localities`*, which returns an `std::vector<>` containing the identifiers of all the machines in the system, including the one that we are on.

As in *Asynchronous execution with actions* our futures are set using *`hpx::async<>`*. The `hello_world_foreman_action` is declared here:

```
// Define the boilerplate code necessary for the function 'hello_world_foreman'
// to be invoked as an HPX action.
HPX_PLAIN_ACTION(hello_world_foreman, hello_world_foreman_action)
```

Another way of thinking about this wrapping technique is as follows: functions (the work to be done) are wrapped in actions, and actions can be executed locally or remotely (e.g. on another machine participating in the computation).

Now it is time to look at the `hello_world_foreman()` function which was wrapped in the action above:

```
void hello_world_foreman()
{
    // Get the number of worker OS-threads in use by this locality.
    std::size_t const os_threads = hpx::get_os_thread_count();

    // Populate a set with the OS-thread numbers of all OS-threads on this
    // locality. When the hello world message has been printed on a particular
    // OS-thread, we will remove it from the set.
    std::set<std::size_t> attendance;
    for (std::size_t os_thread = 0; os_thread < os_threads; ++os_thread)
        attendance.insert(os_thread);

    // As long as there are still elements in the set, we must keep scheduling
    // HPX-threads. Because HPX features work-stealing task schedulers, we have
```

(continues on next page)

(continued from previous page)

```
    // no way of enforcing which worker OS-thread will actually execute
    // each HPX-thread.
    while (!attendance.empty())
    {
        // Each iteration, we create a task for each element in the set of
        // OS-threads that have not said "Hello world". Each of these tasks
        // is encapsulated in a future.
        std::vector<hpx::future<std::size_t>> futures;
        futures.reserve(attendance.size());

        for (std::size_t worker : attendance)
        {
            // Asynchronously start a new task. The task is encapsulated in a
            // future that we can query to determine if the task has completed.
            //
            // We give the task a hint to run on a particular worker thread
            // (core) and suggest binding the scheduled thread to the given
            // core, but no guarantees are given by the scheduler that the task
            // will actually run on that worker thread. It will however try as
            // hard as possible to place the new task on the given worker
            // thread.
            hpx::execution::parallel_executor exec(
                hpx::threads::thread_priority::bound);

            hpx::threads::thread_schedule_hint hint(
                hpx::threads::thread_schedule_hint_mode::thread,
                static_cast<std::int16_t>(worker));

            // Annotate the created threads with the given thread name
            hpx::execution::experimental::annotating_executor annotating_exec(
                hpx::execution::experimental::with_hint(exec, hint),
                std::string("hello_world_worker#") + std::to_string(worker));

            futures.push_back(
                hpx::async(annotating_exec, hello_world_worker, worker));
        }

        // Wait for all of the futures to finish. The callback version of the
        // hpx::wait_each function takes two arguments: a vector of futures,
        // and a binary callback.  The callback takes two arguments; the first
        // is the index of the future in the vector, and the second is the
        // return value of the future. hpx::wait_each doesn't return until
        // all the futures in the vector have returned.
        hpx::spinlock mtx;
        hpx::wait_each(hpx::unwrapping([&](std::size_t t) {
            if (std::size_t(-1) != t)
            {
                std::lock_guard<hpx::spinlock> lk(mtx);
                attendance.erase(t);
            }
        }),
            futures);
```

(continues on next page)

(continued from previous page)

```
    }
}
```

Now, before we discuss `hello_world_foreman()`, let's talk about the *hpx::wait_each* function. The version of *hpx::wait_each* invokes a callback function provided by the user, supplying the callback function with the result of the future.

In `hello_world_foreman()`, an `std::set<>` called `attendance` keeps track of which OS-threads have printed out the hello world message. When the OS-thread prints out the statement, the future is marked as ready, and *hpx::wait_each* in `hello_world_foreman()`. If it is not executing on the correct OS-thread, it returns a value of -1, which causes `hello_world_foreman()` to leave the OS-thread id in `attendance`.

```
std::size_t hello_world_worker(std::size_t desired)
{
    // Returns the OS-thread number of the worker that is running this
    // HPX-thread.
    std::size_t current = hpx::get_worker_thread_num();
    if (current == desired)
    {
        // The HPX-thread has been run on the desired OS-thread.
        char const* msg = "hello world from OS-thread {1} on locality {2}\n";

        hpx::util::format_to(hpx::cout, msg, desired, hpx::get_locality_id())
            << std::flush;

        return desired;
    }

    // This HPX-thread has been run by the wrong OS-thread, make the foreman
    // try again by rescheduling it.
    return std::size_t(-1);
}
```

Because *HPX* features work stealing task schedulers, there is no way to guarantee that an action will be scheduled on a particular OS-thread. This is why we must use a guess-and-check approach.

2.3.5 Components and actions

The accumulator examples demonstrate the use of components. Components are C++ classes that expose methods as a type of *HPX* action. These actions are called component actions. There are three examples: - accumulator - template accumulator - template function accumulator

Components are globally named, meaning that a component action can be called remotely (e.g., from another machine). There are two accumulator examples in *HPX*.

In the *Asynchronous execution with actions* and the *Remote execution with actions*, we introduced plain actions, which wrapped global functions. The target of a plain action is an identifier which refers to a particular machine involved in the computation. For plain actions, the target is the machine where the action will be executed.

Component actions, however, do not target machines. Instead, they target component instances. The instance may live on the machine that we've invoked the component action from, or it may live on another machine.

The components in these examples expose three different functions:

- `reset()` - Resets the accumulator value to 0.

- `add(arg)` - Adds `arg` to the accumulators value.
- `query()` - Queries the value of the accumulator.

These examples create an instance of the (template or template function) accumulator, and then allow the user to enter commands at a prompt, which subsequently invoke actions on the accumulator instance.

Accumulator

Setup

The source code for this example can be found here: `accumulator_client.cpp`.

To compile this program, go to your *HPX* build directory (see *Building HPX* for information on configuring and building *HPX*) and enter:

```
$ make examples.accumulators.accumulator
```

To run the program type:

```
$ ./bin/accumulator_client
```

Once the program starts running, it will print the following prompt and then wait for input. An example session is given below:

```
commands: reset, add [amount], query, help, quit
> add 5
> add 10
> query
15
> add 2
> query
17
> reset
> add 1
> query
1
> quit
```

Walkthrough

Now, let's take a look at the source code of the accumulator example. This example consists of two parts: an *HPX* component library (a library that exposes an *HPX* component) and a client application which uses the library. This walkthrough will cover the *HPX* component library. The code for the client application can be found here: `accumulator_client.cpp`.

An *HPX* component is represented by two C++ classes:

- **A server class** - The implementation of the component's functionality.
- **A client class** - A high-level interface that acts as a proxy for an instance of the component.

Typically, these two classes both have the same name, but the server class usually lives in different sub-namespaces (`server`). For example, the full names of the two classes in accumulator are:

- `examples::server::accumulator` (server class)

- `examples::accumulator` (client class)

The server class

The following code is from server/accumulator.hpp[15].

All *HPX* component server classes must inherit publicly from the *HPX* component base class: *hpx::components::component_base*

The accumulator component inherits from `hpx::components::locking_hook`. This allows the runtime system to ensure that all action invocations are serialized. That means that the system ensures that no two actions are invoked at the same time on a given component instance. This makes the component thread safe and no additional locking has to be implemented by the user. Moreover, an accumulator component is a component because it also inherits from *hpx::components::component_base* (the template argument passed to locking_hook is used as its base class). The following snippet shows the corresponding code:

```
    class accumulator
      : public hpx::components::locking_hook<
            hpx::components::component_base<accumulator>>
```

Our accumulator class will need a data member to store its value in, so let's declare a data member:

```
        argument_type value_;
```

The constructor for this class simply initializes `value_` to 0:

```
        accumulator()
          : value_(0)
        {
        }
```

Next, let's look at the three methods of this component that we will be exposing as component actions:

Here are the action types. These types wrap the methods we're exposing. The wrapping technique is very similar to the one used in the *Asynchronous execution with actions* and the *Remote execution with actions*:

```
        HPX_DEFINE_COMPONENT_ACTION(accumulator, reset)
        HPX_DEFINE_COMPONENT_ACTION(accumulator, add)
        HPX_DEFINE_COMPONENT_ACTION(accumulator, query)
```

The last piece of code in the server class header is the declaration of the action type registration code:

```
HPX_REGISTER_ACTION_DECLARATION(
    examples::server::accumulator::reset_action, accumulator_reset_action)

HPX_REGISTER_ACTION_DECLARATION(
    examples::server::accumulator::add_action, accumulator_add_action)

HPX_REGISTER_ACTION_DECLARATION(
    examples::server::accumulator::query_action, accumulator_query_action)
```

[15] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/examples/accumulators/server/accumulator.hpp

Note

The code above must be placed in the global namespace.

The rest of the registration code is in accumulator.cpp[16]

```
///////////////////////////////////////////////////////////////////////////////
// Add factory registration functionality.
HPX_REGISTER_COMPONENT_MODULE()

///////////////////////////////////////////////////////////////////////////////
typedef hpx::components::component<examples::server::accumulator>
    accumulator_type;

HPX_REGISTER_COMPONENT(accumulator_type, accumulator)

///////////////////////////////////////////////////////////////////////////////
// Serialization support for accumulator actions.
HPX_REGISTER_ACTION(
    accumulator_type::wrapped_type::reset_action, accumulator_reset_action)
HPX_REGISTER_ACTION(
    accumulator_type::wrapped_type::add_action, accumulator_add_action)
HPX_REGISTER_ACTION(
    accumulator_type::wrapped_type::query_action, accumulator_query_action)
```

Note

The code above must be placed in the global namespace.

The client class

The following code is from accumulator.hpp[17]

The client class is the primary interface to a component instance. Client classes are used to create components:

```
// Create a component on this locality.
examples::accumulator c = hpx::new_<examples::accumulator>(hpx::find_here());
```

and to invoke component actions:

```
c.add(hpx::launch::apply, 4);
```

Clients, like servers, need to inherit from a base class, this time, *hpx::components::client_base*:

```
    class accumulator
      : public hpx::components::client_base<accumulator, server::accumulator>
```

For readability, we typedef the base class like so:

[16] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/examples/accumulators/accumulator.cpp
[17] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/examples/accumulators/accumulator.hpp

```
    typedef hpx::components::client_base<accumulator, server::accumulator>
        base_type;
```

Here are examples of how to expose actions through a client class:

There are a few different ways of invoking actions:

- **Non-blocking**: For actions that don't have return types, or when we do not care about the result of an action, we can invoke the action using fire-and-forget semantics. This means that once we have asked *HPX* to compute the action, we forget about it completely and continue with our computation. We use *hpx::post* to invoke an action in a non-blocking fashion.

```
    void reset(hpx::launch::apply_policy)
    {
        HPX_ASSERT(this->get_id());

        typedef server::accumulator::reset_action action_type;
        hpx::post(action_type(), this->get_id());
    }
```

- **Asynchronous**: Futures, as demonstrated in *Asynchronous execution*, *Asynchronous execution with actions*, and the *Remote execution with actions*, enable asynchronous action invocation. Here's an example from the accumulator client class:

```
    hpx::future<argument_type> query(hpx::launch::async_policy)
    {
        HPX_ASSERT(this->get_id());

        typedef server::accumulator::query_action action_type;
        return hpx::async(action_type(), this->get_id());
    }
```

- **Synchronous**: To invoke an action in a fully synchronous manner, we can simply call *hpx::sync* which is semantically equivalent to *hpx::async*`().get()` (i.e., create a future and immediately wait on it to be ready). Here's an example from the accumulator client class:

```
    void add(argument_type arg)
    {
        HPX_ASSERT(this->get_id());

        typedef server::accumulator::add_action action_type;
        action_type()(this->get_id(), arg);
    }
```

Note that `this->get_id()` references a data member of the *hpx::components::client_base* base class which identifies the server accumulator instance.

`hpx::id_type` is a type which represents a global identifier in *HPX*. This type specifies the target of an action. This is the type that is returned by *`hpx::find_here`* in which case it represents the *locality* the code is running on.

Template accumulator

Walkthrough

The server class

The following code is from server/template_accumulator.hpp[18].

Similarly to the accumulator example, the component server class inherits publicly from *hpx::components::component_base* and from `hpx::components::locking_hook` ensuring thread-safe method invocations.

```
template <typename T>
class template_accumulator
  : public hpx::components::locking_hook<
        hpx::components::component_base<template_accumulator<T>>>
```

The body of the template accumulator class remains mainly the same as the accumulator with the difference that it uses templates in the data types.

```
        typedef T argument_type;

        template_accumulator()
          : value_(0)
        {
        }

        ///////////////////////////////////////////////////////////////////////
        // Exposed functionality of this component.

        /// Reset the components value to 0.
        void reset()
        {
            //  set value_ to 0.
            value_ = 0;
        }

        /// Add the given number to the accumulator.
        void add(argument_type arg)
        {
            //  add value_ to arg, and store the result in value_.
            value_ += arg;
        }

        /// Return the current value to the caller.
        argument_type query() const
        {
            // Get the value of value_.
            return value_;
        }
```

(continues on next page)

18 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/examples/accumulators/server/template_accumulator.hpp

(continued from previous page)

```
    ///////////////////////////////////////////////////////////////////////
    // Each of the exposed functions needs to be encapsulated into an
    // action type, generating all required boilerplate code for threads,
    // serialization, etc.
    HPX_DEFINE_COMPONENT_ACTION(template_accumulator, reset)
    HPX_DEFINE_COMPONENT_ACTION(template_accumulator, add)
    HPX_DEFINE_COMPONENT_ACTION(template_accumulator, query)
```

The last piece of code in the server class header is the declaration of the action type registration code. *REGISTER_TEMPLATE_ACCUMULATOR_DECLARATION(type)* declares actions for the specified type, while *REGISTER_TEMPLATE_ACCUMULATOR(type)* registers the actions and the component for the specified type, using macros to handle boilerplate code.

```
#define REGISTER_TEMPLATE_ACCUMULATOR_DECLARATION(type)                      \
    HPX_REGISTER_ACTION_DECLARATION(                                         \
        examples::server::template_accumulator<type>::reset_action,          \
        HPX_PP_CAT(__template_accumulator_reset_action_, type))              \
                                                                             \
    HPX_REGISTER_ACTION_DECLARATION(                                         \
        examples::server::template_accumulator<type>::add_action,            \
        HPX_PP_CAT(__template_accumulator_add_action_, type))                \
                                                                             \
    HPX_REGISTER_ACTION_DECLARATION(                                         \
        examples::server::template_accumulator<type>::query_action,          \
        HPX_PP_CAT(__template_accumulator_query_action_, type))              \
    /**/

#define REGISTER_TEMPLATE_ACCUMULATOR(type)                                  \
    HPX_REGISTER_ACTION(                                                     \
        examples::server::template_accumulator<type>::reset_action,          \
        HPX_PP_CAT(__template_accumulator_reset_action_, type))              \
                                                                             \
    HPX_REGISTER_ACTION(                                                     \
        examples::server::template_accumulator<type>::add_action,            \
        HPX_PP_CAT(__template_accumulator_add_action_, type))                \
                                                                             \
    HPX_REGISTER_ACTION(                                                     \
        examples::server::template_accumulator<type>::query_action,          \
        HPX_PP_CAT(__template_accumulator_query_action_, type))              \
                                                                             \
    typedef ::hpx::components::component<                                    \
        examples::server::template_accumulator<type>>                        \
        HPX_PP_CAT(__template_accumulator_, type);                           \
    HPX_REGISTER_COMPONENT(HPX_PP_CAT(__template_accumulator_, type))        \
    /**/
```

Note

The code above must be placed in the global namespace.

Finally, *HPX_REGISTER_COMPONENT_MODULE()* in file server/template_accumulator.cpp[19] adds the factory registration functionality.

The client class

The client class of the template accumulator can be found in template_accumulator.hpp[20] and is very similar to the client class of the accumulator with the only difference that it uses templates and hence can work with different types.

Template function accumulator

Walkthrough

The server class

The following code is from server/template_function_accumulator.hpp[21].

The component server class inherits publicly from *hpx::components::component_base*.

```
class template_function_accumulator
  : public hpx::components::component_base<template_function_accumulator>
```

typedef hpx::spinlock mutex_type defines a *mutex_type* as *hpx::spinlock* for thread safety, while the code that follows exposes the functionality of this component.

```
        ///////////////////////////////////////////////////////////////////////
        // Exposed functionality of this component.

        /// Reset the value to 0.
        void reset()
        {
            // Atomically set value_ to 0.
            std::lock_guard<mutex_type> l(mtx_);
            value_ = 0;
        }

        /// Add the given number to the accumulator.
        template <typename T>
        void add(T arg)
        {
            // Atomically add value_ to arg, and store the result in value_.
            std::lock_guard<mutex_type> l(mtx_);
            value_ += static_cast<double>(arg);
        }

        /// Return the current value to the caller.
        double query() const
```

(continues on next page)

[19] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/examples/accumulators/server/template_accumulator.cpp

[20] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/examples/accumulators/template_accumulator.hpp

[21] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/examples/accumulators/server/template_function_accumulator.hpp

(continued from previous page)

```
        {
            // Get the value of value_.
            std::lock_guard<mutex_type> l(mtx_);
            return value_;
        }
```

- *reset()*: Resets the accumulator value to *0* in a thread-safe manner using *std::lock_guard*.
- *add()*: Adds a value to the accumulator, allowing any type *T* that can be cast to double.
- *query()*: Returns the current value of the accumulator in a thread-safe manner.

To define the actions for *reset()* and *query()* we can use the macro *HPX_DEFINE_COMPONENT_ACTION*. However, actions with template arguments require special type definitions. Therefore, we use *make_action()* to define *add()*.

```
        ///////////////////////////////////////////////////////////////////////
        // Each of the exposed functions needs to be encapsulated into an
        // action type, generating all required boilerplate code for threads,
        // serialization, etc.

        HPX_DEFINE_COMPONENT_ACTION(template_function_accumulator, reset)
        HPX_DEFINE_COMPONENT_ACTION(template_function_accumulator, query)

        // Actions with template arguments (see add<>() above) require special
        // type definitions. The simplest way to define such an action type is
        // by deriving from the HPX facility make_action.
        template <typename T>
        struct add_action
          : hpx::actions::make_action<void (template_function_accumulator::*)(
                                          T),
                &template_function_accumulator::template add<T>,
                add_action<T>>::type
        {
        };
```

The last piece of code in the server class header is the action registration:

```
HPX_REGISTER_ACTION_DECLARATION(
    examples::server::template_function_accumulator::reset_action,
    managed_accumulator_reset_action)

HPX_REGISTER_ACTION_DECLARATION(
    examples::server::template_function_accumulator::query_action,
    managed_accumulator_query_action)
```

Note

The code above must be placed in the global namespace.

The rest of the registration code is in accumulator.cpp[22]

[22] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/examples/accumulators/accumulator.cpp

```
///////////////////////////////////////////////////////////////////////////////
// Add factory registration functionality.
HPX_REGISTER_COMPONENT_MODULE()

///////////////////////////////////////////////////////////////////////////////
typedef hpx::components::component<
    examples::server::template_function_accumulator>
    accumulator_type;

HPX_REGISTER_COMPONENT(accumulator_type, template_function_accumulator)

///////////////////////////////////////////////////////////////////////////////
// Serialization support for managed_accumulator actions.
HPX_REGISTER_ACTION(accumulator_type::wrapped_type::reset_action,
    managed_accumulator_reset_action)
HPX_REGISTER_ACTION(accumulator_type::wrapped_type::query_action,
    managed_accumulator_query_action)
```

Note

The code above must be placed in the global namespace.

The client class

The client class of the template accumulator can be found in template_function_accumulator.hpp[23] and is very similar to the client class of the accumulator with the only difference that it uses templates and hence can work with different types.

2.3.6 Dataflow

HPX provides its users with several different tools to simply express parallel concepts. One of these tools is a *local control object* (*LCO*) called dataflow. An *LCO* is a type of component that can spawn a new thread when triggered. They are also distinguished from other components by a standard interface that allow users to understand and use them easily. A Dataflow, being an *LCO*, is triggered when the values it depends on become available. For instance, if you have a calculation X that depends on the results of three other calculations, you could set up a dataflow that would begin the calculation X as soon as the other three calculations have returned their values. Dataflows are set up to depend on other dataflows. It is this property that makes dataflow a powerful parallelization tool. If you understand the dependencies of your calculation, you can devise a simple algorithm that sets up a dependency tree to be executed. In this example, we calculate compound interest. To calculate compound interest, one must calculate the interest made in each compound period, and then add that interest back to the principal before calculating the interest made in the next period. A practical person would, of course, use the formula for compound interest:

$$F = P(1 + i)^n$$

where F is the future value, P is the principal value, i is the interest rate, and n is the number of compound periods.

However, for the sake of this example, we have chosen to manually calculate the future value by iterating:

$$I = Pi$$

[23] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/examples/accumulators/template_function_accumulator.hpp

and

$$P = P + I$$

Setup

The source code for this example can be found here: `interest_calculator.cpp`.

To compile this program, go to your *HPX* build directory (see *Building HPX* for information on configuring and building *HPX*) and enter:

```
$ make examples.quickstart.interest_calculator
```

To run the program type:

```
$ ./bin/interest_calculator --principal 100 --rate 5 --cp 6 --time 36
Final amount: 134.01
Amount made: 34.0096
```

Walkthrough

Let us begin with main. Here we can see that we again are using Boost.Program_options to set our command line variables (see *Asynchronous execution with actions* for more details). These options set the principal, rate, compound period, and time. It is important to note that the units of time for `cp` and `time` must be the same.

```
int main(int argc, char** argv)
{
    options_description cmdline("Usage: " HPX_APPLICATION_STRING " [options]");

    cmdline.add_options()("principal", value<double>()->default_value(1000),
        "The principal [$]")("rate", value<double>()->default_value(7),
        "The interest rate [%]")("cp", value<int>()->default_value(12),
        "The compound period [months]")("time",
        value<int>()->default_value(12 * 30),
        "The time money is invested [months]");

    hpx::init_params init_args;
    init_args.desc_cmdline = cmdline;

    return hpx::init(argc, argv, init_args);
}
```

Next we look at hpx_main.

```
int hpx_main(variables_map& vm)
{
    {
        using hpx::dataflow;
        using hpx::make_ready_future;
        using hpx::shared_future;
        using hpx::unwrapping;
        hpx::id_type here = hpx::find_here();
```

(continues on next page)

(continued from previous page)

```
        double init_principal =
            vm["principal"].as<double>();              //Initial principal
        double init_rate = vm["rate"].as<double>();    //Interest rate
        int cp = vm["cp"].as<int>();      //Length of a compound period
        int t = vm["time"].as<int>();     //Length of time money is invested

        init_rate /= 100;     //Rate is a % and must be converted
        t /= cp;     //Determine how many times to iterate interest calculation:
        //How many full compound periods can fit in the time invested

        // In non-dataflow terms the implemented algorithm would look like:
        //
        // int t = 5;     // number of time periods to use
        // double principal = init_principal;
        // double rate = init_rate;
        //
        // for (int i = 0; i < t; ++i)
        // {
        //     double interest = calc(principal, rate);
        //     principal = add(principal, interest);
        // }
        //
        // Please note the similarity with the code below!

        shared_future<double> principal = make_ready_future(init_principal);
        shared_future<double> rate = make_ready_future(init_rate);

        for (int i = 0; i < t; ++i)
        {
            shared_future<double> interest =
                dataflow(unwrapping(calc), principal, rate);
            principal = dataflow(unwrapping(add), principal, interest);
        }

        // wait for the dataflow execution graph to be finished calculating our
        // overall interest
        double result = principal.get();

        std::cout << "Final amount: " << result << std::endl;
        std::cout << "Amount made: " << result - init_principal << std::endl;
    }

    return hpx::finalize();
}
```

Here we find our command line variables read in, the rate is converted from a percent to a decimal, the number of calculation iterations is determined, and then our shared_futures are set up. Notice that we first place our principal and rate into shares futures by passing the variables `init_principal` and `init_rate` using *hpx::make_ready_future*.

In this way *hpx::shared_future*`<double>` `principal` and `rate` will be initialized to `init_principal` and `init_rate` when *hpx::make_ready_future*`<double>` returns a future containing those initial values. These shared futures then enter the for loop and are passed to `interest`. Next `principal` and `interest` are passed to the reassignment of `principal` using a *hpx::dataflow*. A dataflow will first wait for its arguments to be ready before launching any callbacks, so add in this case will not begin until both principal and interest are ready. This loop continues for each compound period that must be calculated. To see how interest and principal are calculated in the loop, let us look at calc_action and add_action:

```
// Calculate interest for one period
double calc(double principal, double rate)
{
    return principal * rate;
}

///////////////////////////////////////////////////////////////////////////////
// Add the amount made to the principal
double add(double principal, double interest)
{
    return principal + interest;
}
```

After the shared future dependencies have been defined in hpx_main, we see the following statement:

```
double result = principal.get();
```

This statement calls hpx::future::get on the shared future principal which had its value calculated by our for loop. The program will wait here until the entire dataflow tree has been calculated and the value assigned to result. The program then prints out the final value of the investment and the amount of interest made by subtracting the final value of the investment from the initial value of the investment.

2.3.7 Local to remote

When developers write code they typically begin with a simple serial code and build upon it until all of the required functionality is present. The following set of examples were developed to demonstrate this iterative process of evolving a simple serial program to an efficient, fully-distributed *HPX* application. For this demonstration, we implemented a 1D heat distribution problem. This calculation simulates the diffusion of heat across a ring from an initialized state to some user-defined point in the future. It does this by breaking each portion of the ring into discrete segments and using the current segment's temperature and the temperature of the surrounding segments to calculate the temperature of the current segment in the next timestep as shown by Fig. **??** below.

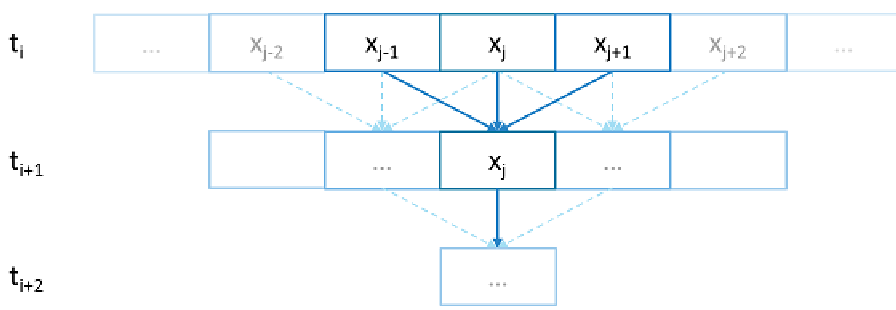


Fig. 2.2: Heat diffusion example program flow.

We parallelize this code over the following eight examples:

- Example 1
- Example 2
- Example 3

- Example 4
- Example 5
- Example 6
- Example 7
- Example 8

The first example is straight serial code. In this code we instantiate a vector U that contains two vectors of doubles as seen in the structure stepper.

```
struct stepper
{
    // Our partition type
    typedef double partition;

    // Our data for one time step
    typedef std::vector<partition> space;

    // Our operator
    static double heat(double left, double middle, double right)
    {
        return middle + (k * dt / (dx * dx)) * (left - 2 * middle + right);
    }

    // do all the work on 'nx' data points for 'nt' time steps
    space do_work(std::size_t nx, std::size_t nt)
    {
        // U[t][i] is the state of position i at time t.
        std::vector<space> U(2);
        for (space& s : U)
            s.resize(nx);

        // Initial conditions: f(0, i) = i
        for (std::size_t i = 0; i != nx; ++i)
            U[0][i] = double(i);

        // Actual time step loop
        for (std::size_t t = 0; t != nt; ++t)
        {
            space const& current = U[t % 2];
            space& next = U[(t + 1) % 2];

            next[0] = heat(current[nx - 1], current[0], current[1]);

            for (std::size_t i = 1; i != nx - 1; ++i)
                next[i] = heat(current[i - 1], current[i], current[i + 1]);

            next[nx - 1] = heat(current[nx - 2], current[nx - 1], current[0]);
        }

        // Return the solution at time-step 'nt'.
        return U[nt % 2];
    }
```

(continues on next page)

(continued from previous page)

```
};
```

Each element in the vector of doubles represents a single grid point. To calculate the change in heat distribution, the temperature of each grid point, along with its neighbors, is passed to the function `heat`. In order to improve readability, references named `current` and `next` are created which, depending on the time step, point to the first and second vector of doubles. The first vector of doubles is initialized with a simple heat ramp. After calling the heat function with the data in the `current` vector, the results are placed into the `next` vector.

In example 2 we employ a technique called futurization. Futurization is a method by which we can easily transform a code that is serially executed into a code that creates asynchronous threads. In the simplest case this involves replacing a variable with a future to a variable, a function with a future to a function, and adding a `.get()` at the point where a value is actually needed. The code below shows how this technique was applied to the `struct stepper`.

```
struct stepper
{
    // Our partition type
    typedef hpx::shared_future<double> partition;

    // Our data for one time step
    typedef std::vector<partition> space;

    // Our operator
    static double heat(double left, double middle, double right)
    {
        return middle + (k * dt / (dx * dx)) * (left - 2 * middle + right);
    }

    // do all the work on 'nx' data points for 'nt' time steps
    hpx::future<space> do_work(std::size_t nx, std::size_t nt)
    {
        using hpx::dataflow;
        using hpx::unwrapping;

        // U[t][i] is the state of position i at time t.
        std::vector<space> U(2);
        for (space& s : U)
            s.resize(nx);

        // Initial conditions: f(0, i) = i
        for (std::size_t i = 0; i != nx; ++i)
            U[0][i] = hpx::make_ready_future(double(i));

        auto Op = unwrapping(&stepper::heat);

        // Actual time step loop
        for (std::size_t t = 0; t != nt; ++t)
        {
            space const& current = U[t % 2];
            space& next = U[(t + 1) % 2];

            // WHEN U[t][i-1], U[t][i], and U[t][i+1] have been computed, THEN we
            // can compute U[t+1][i]
            for (std::size_t i = 0; i != nx; ++i)
```

(continues on next page)

(continued from previous page)

```
            {
                next[i] =
                    dataflow(hpx::launch::async, Op, current[idx(i, -1, nx)],
                        current[i], current[idx(i, +1, nx)]);
            }
        }

        // Now the asynchronous computation is running; the above for-loop does not
        // wait on anything. There is no implicit waiting at the end of each timestep;
        // the computation of each U[t][i] will begin as soon as its dependencies
        // are ready and hardware is available.

        // Return the solution at time-step 'nt'.
        return hpx::when_all(U[nt % 2]);
    }
};
```

In example 2, we redefine our partition type as a `shared_future` and, in `main`, create the object `result`, which is a future to a vector of partitions. We use `result` to represent the last vector in a string of vectors created for each timestep. In order to move to the next timestep, the values of a partition and its neighbors must be passed to `heat` once the futures that contain them are ready. In *HPX*, we have an LCO (Local Control Object) named Dataflow that assists the programmer in expressing this dependency. Dataflow allows us to pass the results of a set of futures to a specified function when the futures are ready. Dataflow takes three types of arguments, one which instructs the dataflow on how to perform the function call (async or sync), the function to call (in this case `Op`), and futures to the arguments that will be passed to the function. When called, dataflow immediately returns a future to the result of the specified function. This allows users to string dataflows together and construct an execution tree.

After the values of the futures in dataflow are ready, the values must be pulled out of the future container to be passed to the function `heat`. In order to do this, we use the HPX facility `unwrapping`, which underneath calls `.get()` on each of the futures so that the function `heat` will be passed doubles and not futures to doubles.

By setting up the algorithm this way, the program will be able to execute as quickly as the dependencies of each future are met. Unfortunately, this example runs terribly slow. This increase in execution time is caused by the overheads needed to create a future for each data point. Because the work done within each call to heat is very small, the overhead of creating and scheduling each of the three futures is greater than that of the actual useful work! In order to amortize the overheads of our synchronization techniques, we need to be able to control the amount of work that will be done with each future. We call this amount of work per overhead grain size.

In example 3, we return to our serial code to figure out how to control the grain size of our program. The strategy that we employ is to create "partitions" of data points. The user can define how many partitions are created and how many data points are contained in each partition. This is accomplished by creating the `struct partition`, which contains a member object `data_`, a vector of doubles that holds the data points assigned to a particular instance of `partition`.

In example 4, we take advantage of the partition setup by redefining `space` to be a vector of shared_futures with each future representing a partition. In this manner, each future represents several data points. Because the user can define how many data points are in each partition, and, therefore, how many data points are represented by one future, a user can control the grainsize of the simulation. The rest of the code is then futurized in the same manner as example 2. It should be noted how strikingly similar example 4 is to example 2.

Example 4 finally shows good results. This code scales equivalently to the OpenMP version. While these results are promising, there are more opportunities to improve the application's scalability. Currently, this code only runs on one *locality*, but to get the full benefit of *HPX*, we need to be able to distribute the work to other machines in a cluster. We begin to add this functionality in example 5.

In order to run on a distributed system, a large amount of boilerplate code must be added. Fortunately, *HPX* provides us with the concept of a *component*, which saves us from having to write quite as much code. A component is an object that can be remotely accessed using its global address. Components are made of two parts: a server and a client class. While the client class is not required, abstracting the server behind a client allows us to ensure type safety instead of having to pass around pointers to global objects. Example 5 renames example 4's `struct partition` to `partition_data` and adds serialization support. Next, we add the server side representation of the data in the structure `partition_server`. `Partition_server` inherits from `hpx::components::component_base`, which contains a server-side component boilerplate. The boilerplate code allows a component's public members to be accessible anywhere on the machine via its Global Identifier (GID). To encapsulate the component, we create a client side helper class. This object allows us to create new instances of our component and access its members without having to know its GID. In addition, we are using the client class to assist us with managing our asynchrony. For example, our client class `partition`'s member function `get_data()` returns a future to `partition_data get_data()`. This struct inherits its boilerplate code from `hpx::components::client_base`.

In the structure `stepper`, we have also had to make some changes to accommodate a distributed environment. In order to get the data from a particular neighboring partition, which could be remote, we must retrieve the data from all of the neighboring partitions. These retrievals are asynchronous and the function `heat_part_data`, which, amongst other things, calls `heat`, should not be called unless the data from the neighboring partitions have arrived. Therefore, it should come as no surprise that we synchronize this operation with another instance of dataflow (found in `heat_part`). This dataflow receives futures to the data in the current and surrounding partitions by calling `get_data()` on each respective partition. When these futures are ready, dataflow passes them to the `unwrapping` function, which extracts the shared_array of doubles and passes them to the lambda. The lambda calls `heat_part_data` on the *locality*, which the middle partition is on.

Although this example could run distributed, it only runs on one *locality*, as it always uses `hpx::find_here()` as the target for the functions to run on.

In example 6, we begin to distribute the partition data on different nodes. This is accomplished in `stepper::do_work()` by passing the GID of the *locality* where we wish to create the partition to the partition constructor.

```
    for (std::size_t i = 0; i != np; ++i)
        U[0][i] = partition(localities[locidx(i, np, nl)], nx, double(i));
```

We distribute the partitions evenly based on the number of localities used, which is described in the function `locidx`. Because some of the data needed to update the partition in `heat_part` could now be on a new *locality*, we must devise a way of moving data to the *locality* of the middle partition. We accomplished this by adding a switch in the function `get_data()` that returns the end element of the `buffer data_` if it is from the left partition or the first element of the buffer if the data is from the right partition. In this way only the necessary elements, not the whole buffer, are exchanged between nodes. The reader should be reminded that this exchange of end elements occurs in the function `get_data()` and, therefore, is executed asynchronously.

Now that we have the code running in distributed, it is time to make some optimizations. The function `heat_part` spends most of its time on two tasks: retrieving remote data and working on the data in the middle partition. Because we know that the data for the middle partition is local, we can overlap the work on the middle partition with that of the possibly remote call of `get_data()`. This algorithmic change, which was implemented in example 7, can be seen below:

```
    // The partitioned operator, it invokes the heat operator above on all elements
    // of a partition.
    static partition heat_part(
        partition const& left, partition const& middle, partition const& right)
    {
        using hpx::dataflow;
        using hpx::unwrapping;

        hpx::shared_future<partition_data> middle_data =
            middle.get_data(partition_server::middle_partition);
```

(continues on next page)

(continued from previous page)

```
        hpx::future<partition_data> next_middle = middle_data.then(
            unwrapping([middle](partition_data const& m) -> partition_data {
                HPX_UNUSED(middle);

                // All local operations are performed once the middle data of
                // the previous time step becomes available.
                std::size_t size = m.size();
                partition_data next(size);
                for (std::size_t i = 1; i != size - 1; ++i)
                    next[i] = heat(m[i - 1], m[i], m[i + 1]);
                return next;
            }));

        return dataflow(hpx::launch::async,
            unwrapping([left, middle, right](partition_data next,
                           partition_data const& l, partition_data const& m,
                           partition_data const& r) -> partition {
                HPX_UNUSED(left);
                HPX_UNUSED(right);

                // Calculate the missing boundary elements once the
                // corresponding data has become available.
                std::size_t size = m.size();
                next[0] = heat(l[size - 1], m[0], m[1]);
                next[size - 1] = heat(m[size - 2], m[size - 1], r[0]);

                // The new partition_data will be allocated on the same locality
                // as 'middle'.
                return partition(middle.get_id(), std::move(next));
            }),
            std::move(next_middle),
            left.get_data(partition_server::left_partition), middle_data,
            right.get_data(partition_server::right_partition));
    }
```

Example 8 completes the futurization process and utilizes the full potential of *HPX* by distributing the program flow to multiple localities, usually defined as nodes in a cluster. It accomplishes this task by running an instance of *HPX* main on each *locality*. In order to coordinate the execution of the program, the `struct stepper` is wrapped into a component. In this way, each *locality* contains an instance of stepper that executes its own instance of the function `do_work()`. This scheme does create an interesting synchronization problem that must be solved. When the program flow was being coordinated on the head node, the GID of each component was known. However, when we distribute the program flow, each partition has no notion of the GID of its neighbor if the next partition is on another *locality*. In order to make the GIDs of neighboring partitions visible to each other, we created two buffers to store the GIDs of the remote neighboring partitions on the left and right respectively. These buffers are filled by sending the GID of newly created edge partitions to the right and left buffers of the neighboring localities.

In order to finish the simulation, the solution vectors named `result` are then gathered together on *locality* 0 and added into a vector of spaces `overall_result` using the *HPX* functions `gather_id` and `gather_here`.

Example 8 completes this example series, which takes the serial code of example 1 and incrementally morphs it into a fully distributed parallel code. This evolution was guided by the simple principles of futurization, the knowledge of grainsize, and utilization of components. Applying these techniques easily facilitates the scalable parallelization of most applications.

2.3.8 Serializing user-defined types

In order to facilitate the sending and receiving of complex datatypes HPX provides a serialization abstraction.

Just like boost, hpx allows users to serialize user-defined types by either providing the serializer as a member function or defining the serialization as a free function.

Unlike Boost HPX doesn't acknowledge second unsigned int parameter, it is solely there to preserve API compatibility with Boost Serialization

This is tutorial was heavily inspired by Boost's serialization concepts[24].

Setup

The source code for this example can be found here: `custom_serialization.cpp`.

To compile this program, go to your *HPX* build directory (see *Building HPX* for information on configuring and building *HPX*) and enter:

```
$ make examples.quickstart.custom_serialization
```

To run the program type:

```
$ ./bin/custom_serialization
```

This should print:

```
Rectangle(Point(x=0,y=0),Point(x=0,y=5))
gravity.g = 9.81%
```

Serialization Requirements

In order to serialize objects in HPX, at least one of the following criteria must be met:

In the case of default constructible objects:

- The object is an empty type.
- Has a serialization function as shown in this tutorial.
- All members are accessible publicly and they can be used in structured binding contexts.

Otherwise:

- They need to have special serialization support.

[24] https://www.boost.org/doc/libs/1_79_0/libs/serialization/doc/serialization.html

Member function serialization

```
struct point_member_serialization
{
    int x{0};
    int y{0};

    // Required when defining the serialization function as private
    // In this case it isn't
    // Provides serialization access to HPX
    friend class hpx::serialization::access;

    // Second argument exists solely for compatibility with boost serialize
    // it is NOT processed by HPX in any way.
    template <typename Archive>
    void serialize(Archive& ar, unsigned int const)
    {
        // clang-format off
        ar & x & y;
        // clang-format on
    }
};

// Allow bitwise serialization
HPX_IS_BITWISE_SERIALIZABLE(point_member_serialization)
```

Notice that `point_member_serialization` is defined as bitwise serializable (see *Bitwise serialization for bitwise copyable data* for more details). HPX is also able to recursively serialize composite classes and structs given that its members are serializable.

```
struct rectangle_member_serialization
{
    point_member_serialization top_left;
    point_member_serialization lower_right;

    template <typename Archive>
    void serialize(Archive& ar, unsigned int const)
    {
        // clang-format off
        ar & top_left & lower_right;
        // clang-format on
    }
};
```

Free function serialization

In order to decouple your models from HPX, HPX also allows for the definition of free function serializers.

```
struct rectangle_free
{
    point_member_serialization top_left;
    point_member_serialization lower_right;
};

template <typename Archive>
void serialize(Archive& ar, rectangle_free& pt, unsigned int const)
{
    // clang-format off
    ar & pt.lower_right & pt.top_left;
    // clang-format on
}
```

Even if you can't modify a class to befriend it, you can still be able to serialize your class provided that your class is default constructable and you are able to reconstruct it yourself.

```
class point_class
{
public:
    point_class(int x, int y)
      : x(x)
      , y(y)
    {
    }

    point_class() = default;

    [[nodiscard]] int get_x() const noexcept
    {
        return x;
    }

    [[nodiscard]] int get_y() const noexcept
    {
        return y;
    }

private:
    int x;
    int y;
};

template <typename Archive>
void load(Archive& ar, point_class& pt, unsigned int const)
{
    int x, y;
    ar >> x >> y;
    pt = point_class(x, y);
```

(continues on next page)

(continued from previous page)

```
}

template <typename Archive>
void save(Archive& ar, point_class const& pt, unsigned int const)
{
    ar << pt.get_x() << pt.get_y();
}

// This tells HPX that you have spilt your serialize function into
// load and save
HPX_SERIALIZATION_SPLIT_FREE(point_class)
```

Serializing non default constructable classes

Some classes don't provide any default constructor.

```
class planet_weight_calculator
{
public:
    explicit planet_weight_calculator(double g)
      : g(g)
    {
    }

    template <class Archive>
    friend void save_construct_data(
        Archive&, planet_weight_calculator const*, unsigned int);

    [[nodiscard]] double get_g() const
    {
        return g;
    }

private:
    // Provides serialization access to HPX
    friend class hpx::serialization::access;
    template <class Archive>
    void serialize(Archive&, unsigned int const)
    {
        // Serialization will be done in the save_construct_data
        // Still needs to be defined
    }

    double g;
};
```

In this case you have to define a `save_construct_data` and `load_construct_data` in which you do the serialization yourself.

```
template <class Archive>
inline void save_construct_data(Archive& ar,
```

(continues on next page)

(continued from previous page)

```
    planet_weight_calculator const* weight_calc, unsigned int const)
{
    ar << weight_calc->g;    // Do all of your serialization here
}

template <class Archive>
inline void load_construct_data(
    Archive& ar, planet_weight_calculator* weight_calc, unsigned int const)
{
    double g;
    ar >> g;

    // ::new(ptr) construct new object at given address
    hpx::construct_at(weight_calc, g);
}
```

Bitwise serialization for bitwise copyable data

When sending non arithmetic types not defined by std::is_arithmetic[25], HPX has to (de)serialize each object separately. However, if the class you are trying to send classes consists only of bitwise copyable datatypes, you may mark your class as such. Then HPX will serialize your object bitwise instead of element wise. This has enormous benefits, especially when sending a vector/array of your class. To define your class as such you need to call `HPX_IS_BITWISE_SERIALIZABLE(T)` with your desired custom class.

```
struct point_member_serialization
{
    int x{0};
    int y{0};

    // Required when defining the serialization function as private
    // In this case it isn't
    // Provides serialization access to HPX
    friend class hpx::serialization::access;

    // Second argument exists solely for compatibility with boost serialize
    // it is NOT processed by HPX in any way.
    template <typename Archive>
    void serialize(Archive& ar, unsigned int const)
    {
        // clang-format off
        ar & x & y;
        // clang-format on
    }
};

// Allow bitwise serialization
HPX_IS_BITWISE_SERIALIZABLE(point_member_serialization)
```

[25] https://en.cppreference.com/w/cpp/types/is_arithmetic

2.3.9 Termination detection

This example demonstrates how to use *HPX*'s termination detection API to ensure all asynchronous work has completed before shutting down the runtime. This is particularly useful when you need to guarantee that all posted tasks have finished execution before the application exits.

Overview

The termination detection API provides a way to wait for all local *HPX*-threads to complete their work. This is essential in scenarios where you spawn asynchronous tasks using *hpx::post* or *hpx::async* and need to ensure they all finish before the runtime shuts down.

The API supports several use cases:

- **Basic usage**: Wait indefinitely for all tasks to complete
- **Timeout**: Wait for a specified duration
- **Deadline**: Wait until a specific time point
- **Cancellation**: Support for cancellation tokens to interrupt the wait

Setup

The source code for this example can be found here: `termination_detection.cpp`.

To compile this program, go to your *HPX* build directory (see *Building HPX* for information on configuring and building *HPX*) and enter:

```
$ make examples.quickstart.termination_detection
```

To run the program type:

```
$ ./bin/termination_detection
```

This will execute all four examples demonstrating different usage patterns of the termination detection API.

Walkthrough

The example demonstrates four different usage patterns of the termination detection API. Let's examine each one:

Basic usage

The simplest form waits indefinitely for all local threads to complete:

This is useful when you want to ensure all work is done before shutting down, and you don't have time constraints.

Timeout usage

You can specify a maximum duration to wait:

The function returns `true` if all threads completed within the timeout, or `false` if the timeout elapsed. This is useful when you want to give tasks a reasonable amount of time to complete but don't want to wait indefinitely.

Deadline usage

Similar to timeout, but you specify an absolute time point:

This is useful when you have a specific deadline by which all work must be completed.

Cancellation support

The most flexible form supports cancellation tokens:

This allows external control over the wait operation. You can request cancellation from another thread, which is useful in scenarios like graceful shutdown with user interruption support.

API Reference

The termination detection API is available in the `hpx::local` namespace:

void *hpx*::*local*::**termination_detection**()

Wait indefinitely for all local *HPX*-threads to complete.

bool *hpx*::*local*::**termination_detection**(*hpx*::chrono::steady_duration const &timeout)

Wait for all local *HPX*-threads to complete, with a timeout.

Parameters

timeout – Maximum duration to wait

Returns

`true` if all threads completed, `false` if timeout elapsed

bool *hpx*::*local*::**termination_detection**(*hpx*::chrono::steady_time_point const &deadline)

Wait for all local *HPX*-threads to complete, until a deadline.

Parameters

deadline – Absolute time point to wait until

Returns

`true` if all threads completed, `false` if deadline passed

bool *hpx*::*local*::**termination_detection**(*hpx*::*stop_token* stop_token, *hpx*::chrono::steady_duration const &timeout)

Wait for all local *HPX*-threads to complete, with cancellation support.

Parameters

- **stop_token** – Token that can be used to request cancellation
- **timeout** – Maximum duration to wait (defaults to maximum duration)

Returns

`true` if all threads completed, `false` if timeout elapsed or stop was requested

Use cases

The termination detection API is particularly useful in the following scenarios:

- **Graceful shutdown**: Ensure all background tasks complete before exiting
- **Testing**: Verify that all spawned tasks have finished in unit tests
- **Batch processing**: Wait for all work items in a batch to complete
- **Resource cleanup**: Ensure all tasks using shared resources have finished before cleanup

See also

- *hpx::post* - Fire-and-forget asynchronous execution
- *hpx::async* - Asynchronous execution with a future
- *hpx::finalize* - Shut down the *HPX* runtime

2.4 Manual

The manual is your comprehensive guide to *HPX*. It contains detailed information on how to build and use *HPX* in different scenarios.

2.4.1 Prerequisites

Supported platforms

At this time, *HPX* supports the following platforms. Other platforms may work, but we do not test *HPX* with other platforms, so please be warned.

Table 2.1: Supported Platforms for *HPX*

Name	Minimum Version	Architectures
Linux	2.6	x86-32, x86-64, k1om
BlueGeneQ	V1R2M0	PowerPC A2
Windows	Any Windows system	x86-32, x86-64
Mac OSX	Any OSX system	x86-64
ARM	Any ARM system	Any architecture
RISC-V	Any RISC-V system	Any architecture

Supported compilers

The table below shows the supported compilers for *HPX*.

Table 2.2: Supported Compilers for *HPX*

Name	Minimum Version	Latest tested
GNU Compiler Collection (g++)[26]	12.0	15.0
clang: a C language family frontend for LLVM[27]	16.0	20.0
Visual C++[28] (x64)	2022	2022

Software and libraries

The table below presents all the necessary prerequisites for building *HPX*.

Table 2.3: Software prerequisites for *HPX*

	Name	Minimum Version	Latest tested
Build System	CMake[29]	3.20	4.1
Required Libraries	Boost[30]	1.71.0	1.88.0
	Portable Hardware Locality (HWLOC)[31]	1.5	2.4

The most important dependencies are Boost[32] and Portable Hardware Locality (HWLOC)[33]. The installation of Boost is described in detail in Boost's Getting Started[34] document. A recent version of hwloc is required in order to support thread pinning and NUMA awareness and can be found in Hwloc Downloads[35].

HPX is written in 99.99% Standard C++ (the remaining 0.01% is platform specific assembly code). As such, *HPX* is compilable with almost any standards compliant C++ compiler. The code base takes advantage of C++ language and standard library features when available.

Note

When building Boost using gcc, please note that it is required to specify a `cxxflags=-std=c++20` command line argument to `b2` (`bjam`).

Note

In most configurations, *HPX* depends only on header-only Boost. Boost.Filesystem is required if the standard library does not support filesystem. The following are not needed by default, but are required in certain configurations: Boost.Chrono, Boost.DateTime, Boost.Log, Boost.LogSetup, Boost.Regex, and Boost.Thread.

Depending on the options you chose while building and installing *HPX*, you will find that *HPX* may depend on several other libraries such as those listed below.

Note

In order to use a high speed parcelport, we currently recommend configuring *HPX* to use MPI so that MPI can be used for communication between different localities. Please set the CMake variable `MPI_CXX_COMPILER` to your MPI C++ compiler wrapper if not detected automatically.

[26] https://gcc.gnu.org
[27] https://clang.llvm.org/
[28] https://msdn.microsoft.com/en-us/visualc/default.aspx
[29] https://www.cmake.org
[30] https://www.boost.org/
[31] https://www.open-mpi.org/projects/hwloc/
[32] https://www.boost.org/
[33] https://www.open-mpi.org/projects/hwloc/
[34] https://www.boost.org/more/getting_started/index.html
[35] https://www.open-mpi.org/software/hwloc/v1.11

Table 2.4: Optional software prerequisites for *HPX*

Name	Minimum version
google-perftools[36]	1.7.1
jemalloc[37]	2.1.0
mi-malloc[38]	1.0.0
Performance Application Programming Interface (PAPI)	

2.4.2 Getting *HPX*

Download a tarball of the latest release from *HPX* Downloads[39] and unpack it or clone the repository directly using `git`:

```
$ git clone https://github.com/TheHPXProject/hpx.git
```

It is also recommended that you check out the latest stable tag:

```
$ cd hpx
```

```
$ git checkout v2.0.0
```

2.4.3 Building *HPX*

Basic information

The build system for *HPX* is based on CMake[40], a cross-platform build-generator tool which is not responsible for building the project but rather generates the files needed by your build tool (GNU make, Visual Studio, etc.) for building *HPX*. If CMake is not already installed in your system, you can download it and install it here: CMake Downloads[41].

Once CMake[42] has been run, the build process can be started. The build process consists of the following parts:

- The *HPX* core libraries (target `core`): This forms the basic set of *HPX* libraries.
- *HPX* Examples (target `examples`): This target is enabled by default and builds all *HPX* examples (disable by setting `HPX_WITH_EXAMPLES:BOOL=Off`). *HPX* examples are part of the `all` target and are included in the installation if enabled.
- *HPX* Tests (target `tests`): This target builds the *HPX* test suite and is enabled by default (disable by setting `HPX_WITH_TESTS:BOOL =Off`). They are not built by the `all` target and have to be built separately.
- *HPX* Documentation (target `docs`): This target builds the documentation, and is not enabled by default (enable by setting `HPX_WITH_DOCUMENTATION:BOOL=On`. For more information see *Documentation*.

The *HPX* build process is highly configurable through CMake[43], and various CMake[44] variables influence the build process. A list with the most important CMake[45] variables can be found in the section that follows, while the complete

[36] https://code.google.com/p/gperftools
[37] http://jemalloc.net
[38] http://microsoft.github.io/mimalloc/
[39] https://hpx.dev/downloads/
[40] https://www.cmake.org
[41] https://www.cmake.org/cmake/resources/software.html
[42] https://www.cmake.org
[43] https://www.cmake.org
[44] https://www.cmake.org
[45] https://www.cmake.org

list of available CMake[46] variables is in *CMake options*. These variables can be used to refine the recipes that can be found at *Platform specific build recipes*, a section that shows some basic steps on how to build *HPX* for a specific platform.

In order to use *HPX*, only the core libraries are required. In order to use the optional libraries, you need to specify them as link dependencies in your build (See *Creating HPX projects*).

Most important CMake options

While building *HPX*, you are provided with multiple CMake options which correspond to different configurations. Below, there is a set of the most important and frequently used CMake options.

HPX_WITH_MALLOC

Use a custom allocator. Using a custom allocator tuned for multithreaded applications is very important for the performance of *HPX* applications. When debugging applications, it's useful to set this to `system`, as custom allocators can hide some memory-related bugs. Note that setting this to something other than `system` requires an external dependency.

HPX_WITH_CUDA

Enable support for CUDA. Use `CMAKE_CUDA_COMPILER` to set the CUDA compiler. This is a standard CMake[47] variable, like `CMAKE_CXX_COMPILER`.

HPX_WITH_PARCELPORT_MPI

Enable the MPI parcelport. This enables the use of MPI for the networking operations in the HPX runtime. The default value is `OFF` because it's not available on all systems and/or requires another dependency. However, it is the recommended parcelport.

HPX_WITH_PARCELPORT_TCP

Enable the TCP parcelport. Enables the use of TCP for networking in the runtime. The default value is `ON`. However, it's only recommended for debugging purposes, as it is slower than the MPI parcelport.

HPX_WITH_PARCELPORT_LCI

Enable the LCI parcelport. This enables the use of LCI for the networking operations in the HPX runtime. The default value is `OFF` because it's not available on all systems and/or requires another dependency. However, this experimental parcelport may provide better performance than the MPI parcelport. Please refer to *Using the LCI parcelport* for more information about the LCI parcelport.

HPX_WITH_APEX

Enable APEX integration. APEX[48] can be used to profile *HPX* applications. In particular, it provides information about individual tasks in the *HPX* runtime.

HPX_WITH_TRACY

Enable Tracy[49] integration. Tracy shows task lifecycle, work stealing, suspension, and (with *HPX_WITH_PARCEL_PROFILING*) distributed parcel events. See *Tracy integration* for build and attach instructions.

HPX_WITH_ITTNOTIFY

Enable Intel ITT API[50] integration. Requires VTune or any other Intel ITT API provider; point `Amplifier_ROOT` at the install (the name is legacy; it accepts any ITTNotify installation).

[46] https://www.cmake.org
[47] https://www.cmake.org
[48] https://uo-oaciss.github.io/apex/quickstarthpx/
[49] https://github.com/wolfpld/tracy
[50] https://github.com/intel/ittapi

HPX_WITH_PARCEL_PROFILING

Enable per-parcel profiling data. Adds per-parcel identifier and timestamp fields (creation time, start time, parcel id) that are carried on the wire and surfaced to whichever tracing backend is enabled. Defaults to `ON` when *HPX_WITH_APEX* is on, `OFF` otherwise.

HPX_WITH_TRACING_LIFECYCLE_EVENTS

Emit per-task lifecycle events (staged, created, executing, yielded, suspended, resumed, completed, deleted). Defaults to `ON`. Turn `OFF` to compile out the per-task hooks and their runtime connection check when profiling a workload where only causal or work-stealing events matter. The saving applies on Tracy builds; the other backends already treat these hooks as no-ops.

HPX_WITH_TRACING_CAUSAL_EVENTS

Emit causal-chain events (`future_fulfilled`, `future_exception_set`, `continuation_run`, `continuation_finished`, `handle_on_completed_fired`). Defaults to `ON`. Turning this `OFF` also elides the active-continuations counter update in `continuation_run` / `continuation_finished`. The saving applies on Tracy builds; the other backends already treat these hooks as no-ops.

HPX_WITH_TRACING_WORK_STEALING_EVENTS

Emit the work-stealing signal when a worker steals a task from another worker. Defaults to `ON`. Turn `OFF` when profiling a workload with heavy work stealing where the signal itself is not being consumed. The saving applies on Tracy builds; the other backends already treat this hook as a no-op.

HPX_WITH_GENERIC_CONTEXT_COROUTINES

Enable Boost. Context for task context switching. It must be enabled for non-x86 architectures such as ARM and Power.

HPX_WITH_MAX_CPU_COUNT

Set the maximum CPU count supported by *HPX*. The default value is 64, and should be set to a number at least as high as the number of cores on a system including virtual cores such as hyperthreads.

HPX_WITH_CXX_STANDARD

Set a specific C++ standard version e.g. `HPX_WITH_CXX_STANDARD=23`. The default and minimum value is `20`. Possible values are `20`, `23`, or `26`.

HPX_WITH_EXAMPLES

Build examples.

HPX_WITH_TESTS

Build tests.

HPX_WITH_DEBUG_POSTFIX

Set the postfix for debug libraries. The default is `d`. This variable is used to set `CMAKE_DEBUG_POSTFIX` and is only relevant for Debug builds or multi-configuration generators. The default rarely needs to be changed. It ensures that generated debug binaries have a different name than release binaries, which is important to avoid ABI problems when both debug and release binaries are installed on the same system.

For a complete list of available CMake[51] variables that influence the build of *HPX*, see *CMake options*.

[51] https://www.cmake.org

Build types

CMake[52] can be configured to generate project files suitable for builds that have enabled debugging support or for an optimized build (without debugging support). The CMake[53] variable used to set the build type is `CMAKE_BUILD_TYPE` (for more information see the CMake Documentation[54]). Available build types are:

- **Debug**: Full debug symbols are available as well as additional assertions to help debugging. To enable the debug build type for the *HPX* API, the C++ Macro `HPX_DEBUG` is defined.
- **RelWithDebInfo**: Release build with debugging symbols. This is most useful for profiling applications
- **Release**: Release build. This disables assertions and enables default compiler optimizations.
- **RelMinSize**: Release build with optimizations for small binary sizes.

Important

We currently don't guarantee ABI compatibility between Debug and Release builds. Please make sure that applications built against *HPX* use the same build type as you used to build *HPX*. For CMake builds, this means that the `CMAKE_BUILD_TYPE` variables have to match and for projects not using CMake[55], the `HPX_DEBUG` macro has to be set in debug mode.

Using CMake Presets

HPX provides a `CMakePresets.json` file which includes a variety of pre-defined build configurations. These presets allow you to easily configure the build for common scenarios without needing to manually specify multiple CMake variables.

To use a preset, you can use the `--preset` option with CMake:

```
$ cmake --preset <preset-name>
$ cmake --build --preset <preset-name>
```

Some of the available presets include:

- `default`: Standard release build with tests and examples enabled.
- `minimal`: Minimal build with only core features (no tests, examples, or tools).
- `full`: Full build with all standard features enabled.
- `debug`: Debug build with symbols and debug-optimized settings.
- `performance`: Build optimized for performance analysis with APEX profiling.
- `cuda`: Build with CUDA support (requires CUDA toolkit).
- `sycl`: Build with SYCL support (requires compatible compiler).
- `sanitizer-address`: Build with AddressSanitizer enabled.

For a full list of available presets, you can run:

```
$ cmake --list-presets
```

52 https://www.cmake.org
53 https://www.cmake.org
54 https://cmake.org/cmake/help/latest/variable/CMAKE_BUILD_TYPE.html
55 https://www.cmake.org

Platform specific build recipes

Unix variants

Once you have the source code and the dependencies and assuming all your dependencies are in paths known to CMake[56], the following gets you started:

1. First, set up a separate build directory to configure the project:

```
$ mkdir build && cd build
```

2. To configure the project you have the following options:

   - To build the core *HPX* libraries and examples, and install them to your chosen location (recommended):

   ```
   $ cmake -DCMAKE_INSTALL_PREFIX=/install/path ..
   ```

   Tip

   If you want to change CMake[57] variables for your build, it is usually a good idea to start with a clean build directory to avoid configuration problems. It is especially important that you use a clean build directory when changing between `Release` and `Debug` modes.

   - To install *HPX* to the default system folders, simply leave out the `CMAKE_INSTALL_PREFIX` option:

   ```
   $ cmake ..
   ```

   - If your dependencies are in custom locations, you may need to tell CMake[58] where to find them by passing one or more options to CMake[59] as shown below:

   ```
   $ cmake -DBoost_ROOT=/path/to/boost
         -DHwloc_ROOT=/path/to/hwloc
         -DTcmalloc_ROOT=/path/to/tcmalloc
         -DJemalloc_ROOT=/path/to/jemalloc
         [other CMake variable definitions]
         /path/to/source/tree
   ```

   For instance:

   ```
   $ cmake -DBoost_ROOT=~/packages/boost -DHwloc_ROOT=/packages/hwloc -DCMAKE_
   ↪INSTALL_PREFIX=~/packages/hpx ~/downloads/hpx_1.5.1
   ```

   - If you want to try *HPX* without using a custom allocator pass `-DHPX_WITH_MALLOC=system` to CMake[60]:

   ```
   $ cmake -DCMAKE_INSTALL_PREFIX=/install/path -DHPX_WITH_MALLOC=system ..
   ```

[56] https://www.cmake.org
[57] https://www.cmake.org
[58] https://www.cmake.org
[59] https://www.cmake.org
[60] https://www.cmake.org

Note

Please pay special attention to the section about *HPX_WITH_MALLOC:STRING* as this is crucial for getting decent performance.

Important

If you are building *HPX* for a system with more than 64 processing units, you must change the CMake[61] variable `HPX_WITH_MAX_CPU_COUNT` (to a value at least as big as the number of (virtual) cores on your system). Note that the default value is 64.

Caution

Compiling and linking *HPX* needs a considerable amount of memory. It is advisable that at least 2 GB of memory per parallel process is available.

3. Once the configuration is complete, to build the project you run:

```
$ cmake --build . --target install
```

Windows

Note

The following build recipes are mostly user-contributed and may be outdated. We always welcome updated and new build recipes.

To build *HPX* under Windows 10 x64 with Visual Studio 2015:

- Download the CMake V3.19 installer (or latest version) from here[62]
- Download the hwloc V1.11.0 (or the latest version) from here[63] and unpack it.
- Download the latest Boost libraries from here[64] and unpack them.
- Build the Boost DLLs and LIBs by using these commands from Command Line (or PowerShell). Open CMD/PowerShell inside the Boost dir and type in:

```
.\bootstrap.bat
```

This batch file will set up everything needed to create a successful build. Now execute:

```
.\b2.exe link=shared variant=release,debug architecture=x86 address-model=64␣
↪threading=multi --build-type=complete install
```

[61] https://www.cmake.org
[62] https://blog.kitware.com/cmake-3-19-0-available-for-download/
[63] https://www.open-mpi.org/software/hwloc/v2.11/
[64] https://www.boost.org/users/download/

This command will start a (very long) build of all available Boost libraries. Please, be patient.

- Open CMake-GUI.exe and set up your source directory (input field 'Where is the source code') to the *base directory* of the source code you downloaded from *HPX*'s GitHub pages. Here's an example of CMake path settings, which point to the `Documents/GitHub/hpx` folder:

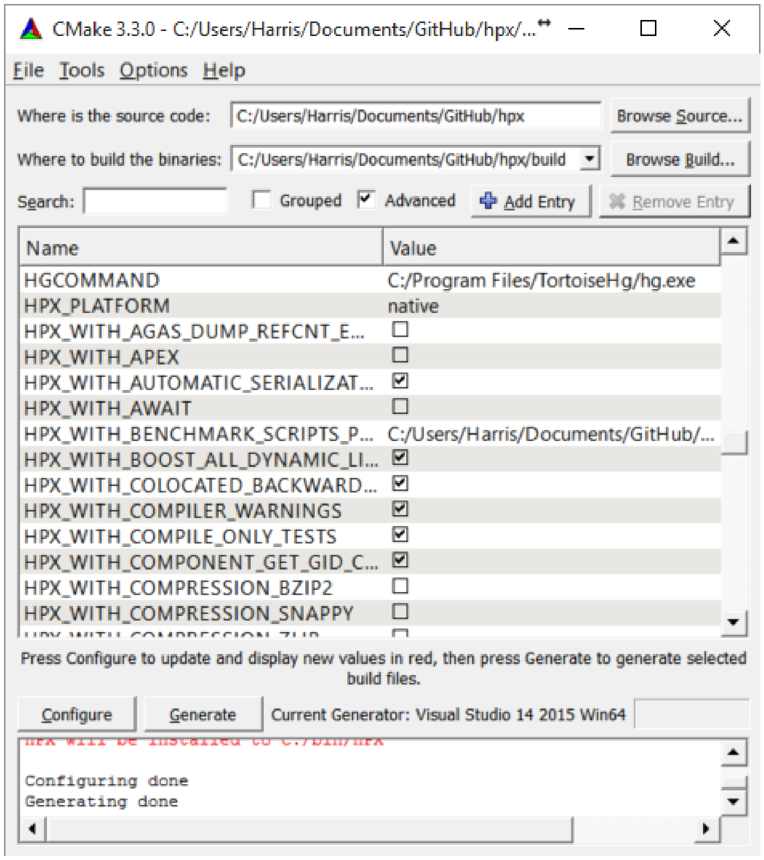


Fig. 2.3: Example CMake path settings.

Inside 'Where is the source-code' enter the base directory of your *HPX* source directory (do not enter the "src" sub-directory!). Inside 'Where to build the binaries' you should put in the path where all the building processes will happen. This is important because the building machinery will do an "out-of-tree" build. CMake will not touch or change the original source files in any way. Instead, it will generate Visual Studio Solution Files, which will build *HPX* packages out of the *HPX* source tree.

- Set new configuration variables (in CMake, not in Windows environment): `Boost_ROOT`, `Hwloc_ROOT`, `Asio_ROOT`, `CMAKE_INSTALL_PREFIX`. The meaning of these variables is as follows:
  - `Boost_ROOT` the *HPX* root directory of the unpacked Boost headers/cpp files.
  - `Hwloc_ROOT` the *HPX* root directory of the unpacked Portable Hardware Locality files.
  - `Asio_ROOT` the *HPX* root directory of the unpacked ASIO files. Alternatively use `HPX_WITH_FETCH_ASIO` with value `True`.
  - `CMAKE_INSTALL_PREFIX` the *HPX* root directory where the future builds of *HPX* should be installed.

Note

> *HPX* is a very large software collection, so it is not recommended to use the default `C:\Program Files\hpx`. Many users may prefer to use simpler paths *without* whitespace, like `C:\bin\hpx` or `D:\bin\hpx` etc.

To insert new env-vars click on "Add Entry" and then insert the name inside "Name", select `PATH` as Type and put the path-name in the "Path" text field. Repeat this for the first three variables.

This is how variable insertion will look:

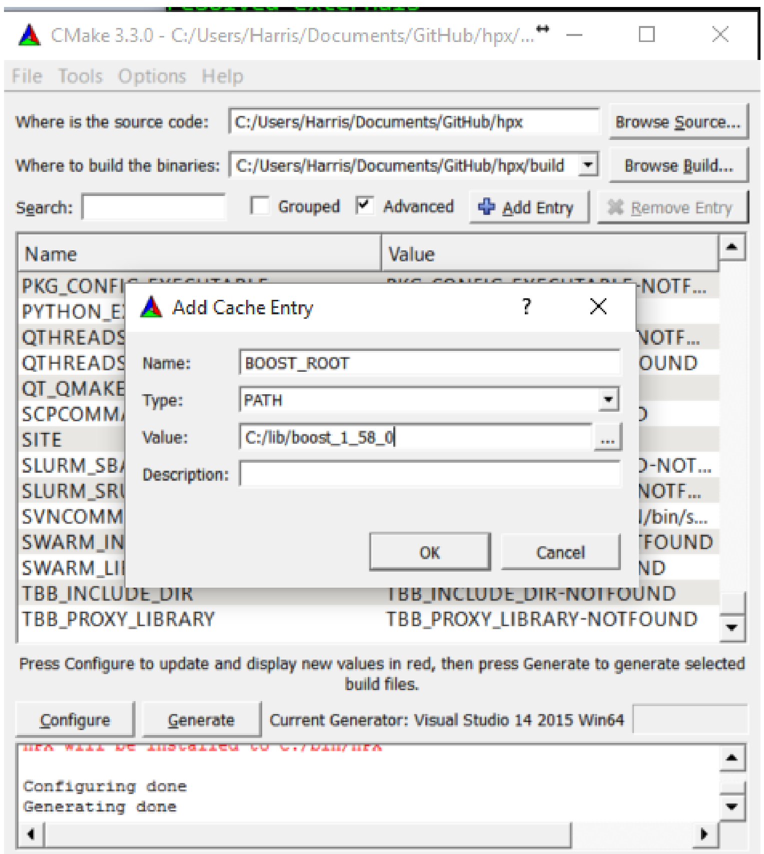


Fig. 2.4: Example CMake adding entry.

Alternatively, users could provide `Boost_LIBRARYDIR` instead of `Boost_ROOT`; the difference is that `Boost_LIBRARYDIR` should point to the subdirectory inside Boost root where all the compiled DLLs/LIBs are. For example, `Boost_LIBRARYDIR` may point to the `bin.v2` subdirectory under the Boost rootdir. It is important to keep the meanings of these two variables separated from each other: `Boost_DIR` points to the ROOT folder of the Boost library. `Boost_LIBRARYDIR` points to the subdir inside the Boost root folder where the compiled binaries are.

- Click the 'Configure' button of CMake-GUI. You will be immediately presented with a small window where you can select the C++ compiler to be used within Visual Studio. This has been tested using the latest v14 (a.k.a C++ 2015) but older versions should be sufficient too. Make sure to select the 64Bit compiler.
- After the generate process has finished successfully, click the 'Generate' button. Now, CMake will put new VS Solution files into the BUILD folder you selected at the beginning.
- Open Visual Studio and load the `HPX.sln` from your build folder.
- Go to `CMakePredefinedTargets` and build the `INSTALL` project:

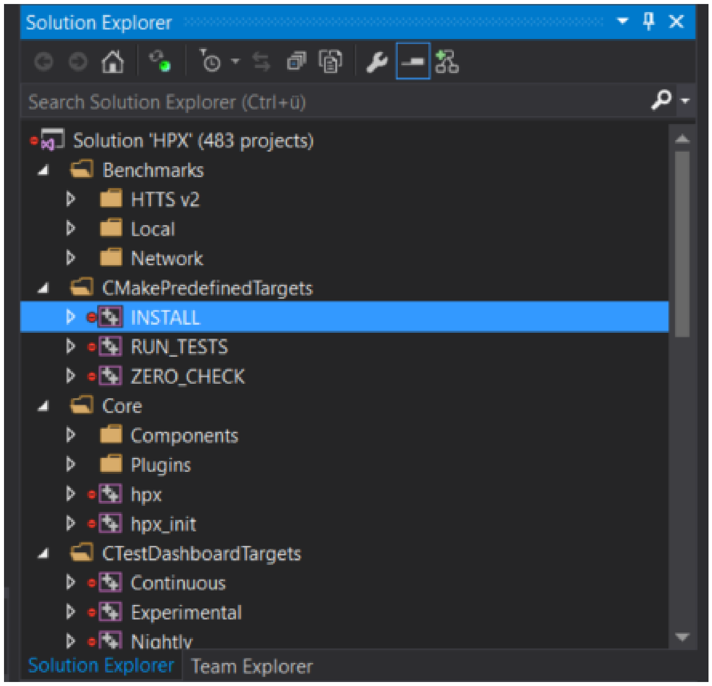


Fig. 2.5: Visual Studio INSTALL target.

It will take some time to compile everything, and in the end you should see an output similar to this one:

2.4.4 CMake options

In order to configure *HPX*, you can set a variety of options to allow CMake to generate your specific makefiles/project files. A list of the most important CMake options can be found in *Most important CMake options*, while this section includes the comprehensive list.

Variables that influence how *HPX* is built

The options are split into these categories:

- *Generic options*
- *Build Targets options*
- *Thread Manager options*
- *AGAS options*
- *Parcelport options*
- *Profiling options*
- *Debugging options*
- *Modules options*

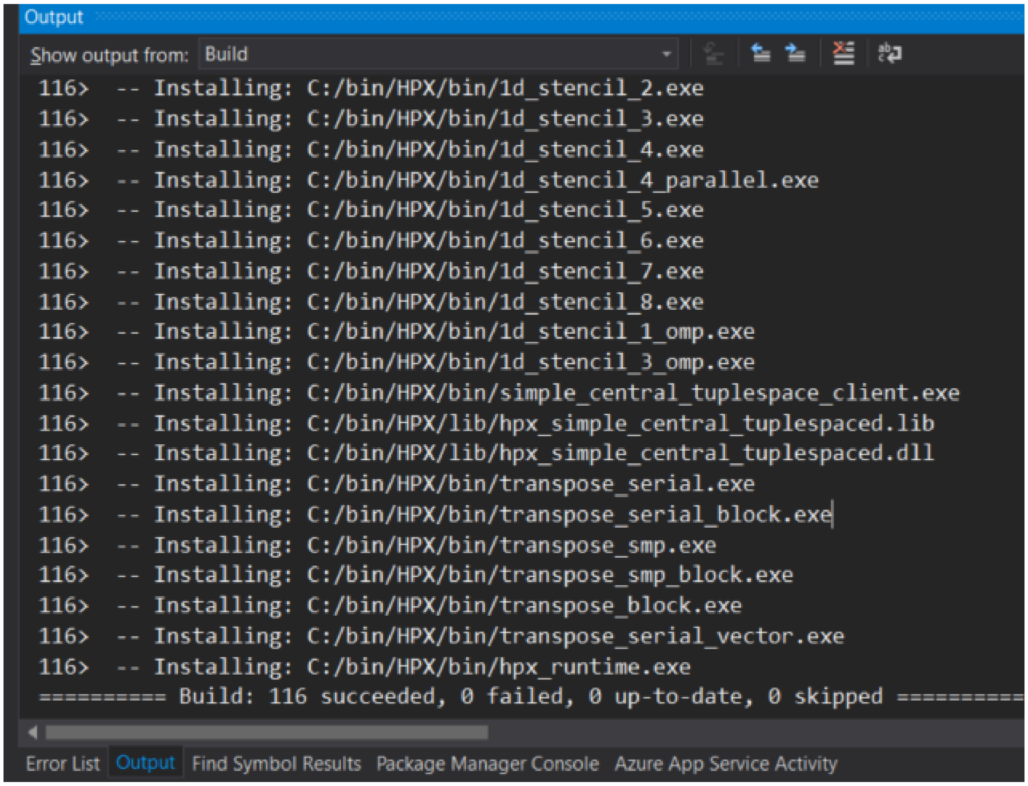


Fig. 2.6: Visual Studio build output.

Generic options

- *HPX_WITH_AUTOMATIC_SERIALIZATION_REGISTRATION:BOOL*
- *HPX_WITH_BUILD_BINARY_PACKAGE:BOOL*
- *HPX_WITH_CHECK_MODULE_DEPENDENCIES:BOOL*
- *HPX_WITH_COMPILER_WARNINGS:BOOL*
- *HPX_WITH_COMPILER_WARNINGS_AS_ERRORS:BOOL*
- *HPX_WITH_COMPRESSION_BZIP2:BOOL*
- *HPX_WITH_COMPRESSION_SNAPPY:BOOL*
- *HPX_WITH_COMPRESSION_ZLIB:BOOL*
- *HPX_WITH_CONTRACTS_MODE:STRING*
- *HPX_WITH_CUDA:BOOL*
- *HPX_WITH_CXX20_STD_EXECUTION_POLICES:INTERNAL*
- *HPX_WITH_CXX_MODULES:BOOL*
- *HPX_WITH_CXX_STANDARD:STRING*
- *HPX_WITH_DATAPAR_BACKEND:STRING*
- *HPX_WITH_DEPRECATION_WARNINGS:BOOL*
- *HPX_WITH_DISABLED_SIGNAL_EXCEPTION_HANDLERS:BOOL*

- *HPX_WITH_DYNAMIC_HPX_MAIN:BOOL*
- *HPX_WITH_FAULT_TOLERANCE:BOOL*
- *HPX_WITH_FORCE_DISCONNECT:BOOL*
- *HPX_WITH_FULL_RPATH:BOOL*
- *HPX_WITH_GCC_VERSION_CHECK:BOOL*
- *HPX_WITH_GENERIC_CONTEXT_COROUTINES:BOOL*
- *HPX_WITH_GIT_HOOKS:BOOL*
- *HPX_WITH_HIDDEN_VISIBILITY:BOOL*
- *HPX_WITH_HIP:BOOL*
- *HPX_WITH_HIPSYCL:BOOL*
- *HPX_WITH_IGNORE_COMPILER_COMPATIBILITY:BOOL*
- *HPX_WITH_LOGGING:BOOL*
- *HPX_WITH_MALLOC:STRING*
- *HPX_WITH_MODULES_AS_STATIC_LIBRARIES:BOOL*
- *HPX_WITH_MODULE_COMPATIBILITY_HEADERS:BOOL*
- *HPX_WITH_NICE_THREADLEVEL:BOOL*
- *HPX_WITH_PARCEL_COALESCING:BOOL*
- *HPX_WITH_PKGCONFIG:BOOL*
- *HPX_WITH_PRECOMPILED_HEADERS:BOOL*
- *HPX_WITH_PSEUDO_DEPENDENCIES:BOOL*
- *HPX_WITH_RUN_MAIN_EVERYWHERE:BOOL*
- *HPX_WITH_STACKOVERFLOW_DETECTION:BOOL*
- *HPX_WITH_STATIC_LINKING:BOOL*
- *HPX_WITH_SUPPORT_NO_UNIQUE_ADDRESS_ATTRIBUTE:BOOL*
- *HPX_WITH_SYCL:BOOL*
- *HPX_WITH_SYCL_FLAGS:STRING*
- *HPX_WITH_UNITY_BUILD:BOOL*
- *HPX_WITH_VIM_YCM:BOOL*
- *HPX_WITH_ZERO_COPY_SERIALIZATION_THRESHOLD:STRING*

HPX_WITH_AUTOMATIC_SERIALIZATION_REGISTRATION:BOOL

Use automatic serialization registration for actions and functions. This affects compatibility between HPX applications compiled with different compilers (default ON)

HPX_WITH_BUILD_BINARY_PACKAGE:BOOL

Build HPX on the build infrastructure on any LINUX distribution (default: OFF).

HPX_WITH_CHECK_MODULE_DEPENDENCIES:BOOL

Verify that no modules are cross-referenced from a different module category (default: OFF)

`HPX_WITH_COMPILER_WARNINGS:BOOL`
Enable compiler warnings (default: ON)

`HPX_WITH_COMPILER_WARNINGS_AS_ERRORS:BOOL`
Turn compiler warnings into errors (default: OFF)

`HPX_WITH_COMPRESSION_BZIP2:BOOL`
Enable bzip2 compression for parcel data (default: OFF).

`HPX_WITH_COMPRESSION_SNAPPY:BOOL`
Enable snappy compression for parcel data (default: OFF).

`HPX_WITH_COMPRESSION_ZLIB:BOOL`
Enable zlib compression for parcel data (default: OFF).

`HPX_WITH_CONTRACTS_MODE:STRING`
Contract evaluation mode for pre-C++26 compilers, options are: ENFORCE, QUICK_ENFORCE, OBSERVE, IGNORE

`HPX_WITH_CUDA:BOOL`
Enable support for CUDA (default: OFF)

`HPX_WITH_CXX20_STD_EXECUTION_POLICES:INTERNAL`

`HPX_WITH_CXX_MODULES:BOOL`
Enable exposing C++20 modules (default: OFF).

`HPX_WITH_CXX_STANDARD:STRING`
Set the C++ standard to use when compiling HPX itself. (default: 20)

`HPX_WITH_DATAPAR_BACKEND:STRING`
Define which vectorization library should be used. Options are: VC, EVE, STD_EXPERIMENTAL_SIMD, SVE, EMULATE, NONE

`HPX_WITH_DEPRECATION_WARNINGS:BOOL`
Enable warnings for deprecated facilities (default: ON).

`HPX_WITH_DISABLED_SIGNAL_EXCEPTION_HANDLERS:BOOL`
Disables the mechanism that produces debug output for caught signals and unhandled exceptions (default: OFF)

`HPX_WITH_DYNAMIC_HPX_MAIN:BOOL`
Enable dynamic overload of system `main()` (Linux and Apple only, default: ON)

`HPX_WITH_FAULT_TOLERANCE:BOOL`
Build HPX to tolerate failures of nodes, i.e. ignore errors in active communication channels (default: OFF)

`HPX_WITH_FORCE_DISCONNECT:BOOL`
Build HPX to support forceful disconnect of localities (default: OFF)

`HPX_WITH_FULL_RPATH:BOOL`
Build and link HPX libraries and executables with full RPATHs (default: ON)

`HPX_WITH_GCC_VERSION_CHECK:BOOL`
Don't ignore version reported by gcc (default: ON)

`HPX_WITH_GENERIC_CONTEXT_COROUTINES:BOOL`
Use Boost.Context as the underlying coroutines context switch implementation.

HPX_WITH_GIT_HOOKS:BOOL
Enable git commit hooks for formatting checks on commit (default: OFF).

HPX_WITH_HIDDEN_VISIBILITY:BOOL
Use -fvisibility=hidden for builds on platforms which support it (default OFF)

HPX_WITH_HIP:BOOL
Enable compilation with HIPCC (default: OFF)

HPX_WITH_HIPSYCL:BOOL
Use hipsycl cmake integration (default: OFF)

HPX_WITH_IGNORE_COMPILER_COMPATIBILITY:BOOL
Ignore compiler incompatibility in dependent projects (default: ON).

HPX_WITH_LOGGING:BOOL
Build HPX with logging enabled (default: ON).

HPX_WITH_MALLOC:STRING
Define which allocator should be linked in. Options are: system, tcmalloc, jemalloc, mimalloc, tbbmalloc, and custom (default is: tcmalloc)

HPX_WITH_MODULES_AS_STATIC_LIBRARIES:BOOL
Compile HPX modules as STATIC (whole-archive) libraries instead of OBJECT libraries (Default: ON)

HPX_WITH_MODULE_COMPATIBILITY_HEADERS:BOOL
Generate backwards-compatibility headers for HPX Modules (default: OFF)

HPX_WITH_NICE_THREADLEVEL:BOOL
Set HPX worker threads to have high NICE level (may impact performance) (default: OFF)

HPX_WITH_PARCEL_COALESCING:BOOL
Enable the parcel coalescing plugin (default: ON).

HPX_WITH_PKGCONFIG:BOOL
Enable generation of pkgconfig files (default: ON on Linux without CUDA/HIP, otherwise OFF)

HPX_WITH_PRECOMPILED_HEADERS:BOOL
Enable precompiled headers for certain build targets (experimental) (default OFF)

HPX_WITH_PSEUDO_DEPENDENCIES:BOOL
Turn compiler warnings into errors (default: ON)

HPX_WITH_RUN_MAIN_EVERYWHERE:BOOL
Run hpx_main by default on all localities (default: OFF, deprecated, will be removed).

HPX_WITH_STACKOVERFLOW_DETECTION:BOOL
Enable stackoverflow detection for HPX threads/coroutines (default: OFF, debug: ON).

HPX_WITH_STATIC_LINKING:BOOL
Compile HPX statically linked libraries (Default: OFF)

HPX_WITH_SUPPORT_NO_UNIQUE_ADDRESS_ATTRIBUTE:BOOL
Enable the use of the [[no_unique_address]] attribute (default: ON)

HPX_WITH_SYCL:BOOL
Enable support for Sycl (default: OFF)

HPX_WITH_SYCL_FLAGS:STRING
Sycl compile flags for selecting specific targets (default: empty)

HPX_WITH_UNITY_BUILD:BOOL
Enable unity build for certain build targets (default OFF)

HPX_WITH_VIM_YCM:BOOL
Generate HPX completion file for VIM YouCompleteMe plugin

HPX_WITH_ZERO_COPY_SERIALIZATION_THRESHOLD:STRING
The threshold in bytes to when perform zero copy optimizations (default: 8192)

Build Targets options

- *HPX_WITH_ASIO_TAG:STRING*
- *HPX_WITH_COMPILE_ONLY_TESTS:BOOL*
- *HPX_WITH_DISTRIBUTED_RUNTIME:BOOL*
- *HPX_WITH_DOCUMENTATION:BOOL*
- *HPX_WITH_DOCUMENTATION_OUTPUT_FORMATS:STRING*
- *HPX_WITH_EXAMPLES:BOOL*
- *HPX_WITH_EXAMPLES_HDF5:BOOL*
- *HPX_WITH_EXAMPLES_OPENMP:BOOL*
- *HPX_WITH_EXAMPLES_QT4:BOOL*
- *HPX_WITH_EXAMPLES_QTHREADS:BOOL*
- *HPX_WITH_EXAMPLES_TASKBENCH:BOOL*
- *HPX_WITH_EXAMPLES_TBB:BOOL*
- *HPX_WITH_EXECUTABLE_PREFIX:STRING*
- *HPX_WITH_FAIL_COMPILE_TESTS:BOOL*
- *HPX_WITH_FETCH_APEX:BOOL*
- *HPX_WITH_FETCH_ASIO:BOOL*
- *HPX_WITH_FETCH_BOOST:BOOL*
- *HPX_WITH_FETCH_GASNET:BOOL*
- *HPX_WITH_FETCH_HWLOC:BOOL*
- *HPX_WITH_FETCH_LCI:BOOL*
- *HPX_WITH_FETCH_LCW:BOOL*
- *HPX_WITH_FETCH_TRACY:BOOL*
- *HPX_WITH_IO_COUNTERS:BOOL*
- *HPX_WITH_LCI_BOOTSTRAP_MPI:BOOL*
- *HPX_WITH_LCI_TAG:STRING*
- *HPX_WITH_LCW_TAG:STRING*
- *HPX_WITH_NANOBENCH:BOOL*

- *HPX_WITH_PARALLEL_LINK_JOBS:STRING*
- *HPX_WITH_TESTS:BOOL*
- *HPX_WITH_TESTS_BENCHMARKS:BOOL*
- *HPX_WITH_TESTS_EXAMPLES:BOOL*
- *HPX_WITH_TESTS_EXTERNAL_BUILD:BOOL*
- *HPX_WITH_TESTS_HEADERS:BOOL*
- *HPX_WITH_TESTS_REGRESSIONS:BOOL*
- *HPX_WITH_TESTS_UNIT:BOOL*
- *HPX_WITH_THRUST:BOOL*
- *HPX_WITH_TOOLS:BOOL*

HPX_WITH_ASIO_TAG:STRING

Asio repository tag or branch

HPX_WITH_COMPILE_ONLY_TESTS:BOOL

Create build system support for compile time only HPX tests (default ON)

HPX_WITH_DISTRIBUTED_RUNTIME:BOOL

Enable the distributed runtime (default: ON). Turning off the distributed runtime completely disallows the creation and use of components and actions. Turning this option off is experimental!

HPX_WITH_DOCUMENTATION:BOOL

Build the HPX documentation (default OFF).

HPX_WITH_DOCUMENTATION_OUTPUT_FORMATS:STRING

List of documentation output formats to generate. Valid options are html;singlehtml;latexpdf;man. Multiple values can be separated with semicolons. (default html).

HPX_WITH_EXAMPLES:BOOL

Build the HPX examples (default ON)

HPX_WITH_EXAMPLES_HDF5:BOOL

Enable examples requiring HDF5 support (default: OFF).

HPX_WITH_EXAMPLES_OPENMP:BOOL

Enable examples requiring OpenMP support (default: OFF).

HPX_WITH_EXAMPLES_QT4:BOOL

Enable examples requiring Qt4 support (default: OFF).

HPX_WITH_EXAMPLES_QTHREADS:BOOL

Enable examples requiring QThreads support (default: OFF).

HPX_WITH_EXAMPLES_TASKBENCH:BOOL

Enable the task-bench example (fetched from upstream StanfordLegion/task-bench, default: OFF).

HPX_WITH_EXAMPLES_TBB:BOOL

Enable examples requiring TBB support (default: OFF).

HPX_WITH_EXECUTABLE_PREFIX:STRING

Executable prefix (default none), '**hpx_**' useful for system install.

HPX_WITH_FAIL_COMPILE_TESTS:BOOL
Create build system support for fail compile HPX tests (default ON)

HPX_WITH_FETCH_APEX:BOOL
Use FetchContent to fetch APEX. By default an installed APEX will be used. (default: OFF)

HPX_WITH_FETCH_ASIO:BOOL
Use FetchContent to fetch Asio. By default an installed Asio will be used. (default: OFF)

HPX_WITH_FETCH_BOOST:BOOL
Use FetchContent to fetch Boost. By default an installed Boost will be used. (default: OFF)

HPX_WITH_FETCH_GASNET:BOOL
Use FetchContent to fetch GASNET. By default an installed GASNET will be used. (default: OFF).

HPX_WITH_FETCH_HWLOC:BOOL
Use FetchContent to fetch Hwloc. By default an installed Hwloc will be used. (default: OFF)

HPX_WITH_FETCH_LCI:BOOL
Use FetchContent to fetch LCI. By default an installed LCI will be used. (default: OFF)

HPX_WITH_FETCH_LCW:BOOL
Use FetchContent to fetch LCW. By default an installed LCW will be used. (default: OFF)

HPX_WITH_FETCH_TRACY:BOOL
Use FetchContent to fetch Tracy. By default an installed Tracy will be used. (default: OFF)

HPX_WITH_IO_COUNTERS:BOOL
Enable IO counters (default: ON)

HPX_WITH_LCI_BOOTSTRAP_MPI:BOOL
Configure the autofetched LCI with mpi bootstrap support (default: OFF)

HPX_WITH_LCI_TAG:STRING
LCI repository tag or branch

HPX_WITH_LCW_TAG:STRING
LCW repository tag or branch

HPX_WITH_NANOBENCH:BOOL
Use Nanobench for performance tests. Nanobench will be fetched using FetchContent (default: OFF)

HPX_WITH_PARALLEL_LINK_JOBS:STRING
Number of Parallel link jobs while building hpx (only for Ninja as generator) (default 2)

HPX_WITH_TESTS:BOOL
Build the HPX tests (default ON)

HPX_WITH_TESTS_BENCHMARKS:BOOL
Build HPX benchmark tests (default: ON)

HPX_WITH_TESTS_EXAMPLES:BOOL
Add HPX examples as tests (default: ON)

HPX_WITH_TESTS_EXTERNAL_BUILD:BOOL
Build external cmake build tests (default: ON)

HPX_WITH_TESTS_HEADERS:BOOL
Build HPX header tests (default: OFF)

HPX_WITH_TESTS_REGRESSIONS:BOOL
Build HPX regression tests (default: ON)

HPX_WITH_TESTS_UNIT:BOOL
Build HPX unit tests (default: ON)

HPX_WITH_THRUST:BOOL
Enable support for NVIDIA Thrust integration (default: ON when CUDA is enabled, OFF otherwise)

HPX_WITH_TOOLS:BOOL
Build HPX tools (default: OFF)

Thread Manager options

- *HPX_COROUTINES_WITH_SWAP_CONTEXT_EMULATION:BOOL*
- *HPX_COROUTINES_WITH_THREAD_SCHEDULE_HINT_RUNS_AS_CHILD:BOOL*
- *HPX_WITH_COROUTINE_COUNTERS:BOOL*
- *HPX_WITH_IO_POOL:BOOL*
- *HPX_WITH_MAX_CPU_COUNT:STRING*
- *HPX_WITH_MAX_NUMA_DOMAIN_COUNT:STRING*
- *HPX_WITH_SCHEDULER_LOCAL_STORAGE:BOOL*
- *HPX_WITH_SPINLOCK_DEADLOCK_DETECTION:BOOL*
- *HPX_WITH_SPINLOCK_POOL_NUM:STRING*
- *HPX_WITH_STACKTRACES:BOOL*
- *HPX_WITH_STACKTRACES_DEMANGLE_SYMBOLS:BOOL*
- *HPX_WITH_STACKTRACES_STATIC_SYMBOLS:BOOL*
- *HPX_WITH_THREAD_BACKTRACE_DEPTH:STRING*
- *HPX_WITH_THREAD_BACKTRACE_ON_SUSPENSION:BOOL*
- *HPX_WITH_THREAD_CREATION_AND_CLEANUP_RATES:BOOL*
- *HPX_WITH_THREAD_CUMULATIVE_COUNTS:BOOL*
- *HPX_WITH_THREAD_IDLE_RATES:BOOL*
- *HPX_WITH_THREAD_LOCAL_STORAGE:BOOL*
- *HPX_WITH_THREAD_MANAGER_IDLE_BACKOFF:BOOL*
- *HPX_WITH_THREAD_QUEUE_WAITTIME:BOOL*
- *HPX_WITH_THREAD_STACK_MMAP:BOOL*
- *HPX_WITH_THREAD_STEALING_COUNTS:BOOL*
- *HPX_WITH_THREAD_TARGET_ADDRESS:BOOL*
- *HPX_WITH_TIMER_POOL:BOOL*
- *HPX_WITH_WORK_REQUESTING_SCHEDULERS:BOOL*

`HPX_COROUTINES_WITH_SWAP_CONTEXT_EMULATION:BOOL`
: Emulate SwapContext API for coroutines (Windows only, default: OFF)

`HPX_COROUTINES_WITH_THREAD_SCHEDULE_HINT_RUNS_AS_CHILD:BOOL`
: Futures attempt to run associated threads directly if those have not been started (default: OFF)

`HPX_WITH_COROUTINE_COUNTERS:BOOL`
: Enable keeping track of coroutine creation and rebind counts (default: OFF)

`HPX_WITH_IO_POOL:BOOL`
: Disable internal IO thread pool, do not change if not absolutely necessary (default: ON)

`HPX_WITH_MAX_CPU_COUNT:STRING`
: HPX applications will not use more that this number of OS-Threads (empty string means dynamic) (default: "")

`HPX_WITH_MAX_NUMA_DOMAIN_COUNT:STRING`
: HPX applications will not run on machines with more NUMA domains (default: 8)

`HPX_WITH_SCHEDULER_LOCAL_STORAGE:BOOL`
: Enable scheduler local storage for all HPX schedulers (default: OFF)

`HPX_WITH_SPINLOCK_DEADLOCK_DETECTION:BOOL`
: Enable spinlock deadlock detection (default: OFF)

`HPX_WITH_SPINLOCK_POOL_NUM:STRING`
: Number of elements a spinlock pool manages (default: 128)

`HPX_WITH_STACKTRACES:BOOL`
: Attach backtraces to HPX exceptions (default: ON)

`HPX_WITH_STACKTRACES_DEMANGLE_SYMBOLS:BOOL`
: Thread stack back trace symbols will be demangled (default: ON)

`HPX_WITH_STACKTRACES_STATIC_SYMBOLS:BOOL`
: Thread stack back trace will resolve static symbols (default: OFF)

`HPX_WITH_THREAD_BACKTRACE_DEPTH:STRING`
: Thread stack back trace depth being captured (default: 20)

`HPX_WITH_THREAD_BACKTRACE_ON_SUSPENSION:BOOL`
: Enable thread stack back trace being captured on suspension (default: OFF)

`HPX_WITH_THREAD_CREATION_AND_CLEANUP_RATES:BOOL`
: Enable measuring thread creation and cleanup times (default: OFF)

`HPX_WITH_THREAD_CUMULATIVE_COUNTS:BOOL`
: Enable keeping track of cumulative thread counts in the schedulers (default: ON)

`HPX_WITH_THREAD_IDLE_RATES:BOOL`
: Enable measuring the percentage of overhead times spent in the scheduler (default: OFF)

`HPX_WITH_THREAD_LOCAL_STORAGE:BOOL`
: Enable thread local storage for all HPX threads (default: OFF)

`HPX_WITH_THREAD_MANAGER_IDLE_BACKOFF:BOOL`
: HPX scheduler threads do exponential backoff on idle queues (default: ON)

HPX_WITH_THREAD_QUEUE_WAITTIME:BOOL
Enable collecting queue wait times for threads (default: OFF)

HPX_WITH_THREAD_STACK_MMAP:BOOL
Use mmap for stack allocation on appropriate platforms

HPX_WITH_THREAD_STEALING_COUNTS:BOOL
Enable keeping track of counts of thread stealing incidents in the schedulers (default: OFF)

HPX_WITH_THREAD_TARGET_ADDRESS:BOOL
Enable storing target address in thread for NUMA awareness (default: OFF)

HPX_WITH_TIMER_POOL:BOOL
Disable internal timer thread pool, do not change if not absolutely necessary (default: ON)

HPX_WITH_WORK_REQUESTING_SCHEDULERS:BOOL
Enable work requesting scheduler (default: ON)

AGAS options

- *HPX_WITH_AGAS_DUMP_REFCNT_ENTRIES:BOOL*

HPX_WITH_AGAS_DUMP_REFCNT_ENTRIES:BOOL
Enable dumps of the AGAS refcnt tables to logs (default: OFF)

Parcelport options

- *HPX_WITH_NETWORKING:BOOL*
- *HPX_WITH_PARCELPORT_ACTION_COUNTERS:BOOL*
- *HPX_WITH_PARCELPORT_COUNTERS:BOOL*
- *HPX_WITH_PARCELPORT_GASNET:BOOL*
- *HPX_WITH_PARCELPORT_LCI:BOOL*
- *HPX_WITH_PARCELPORT_LCI_LOG:STRING*
- *HPX_WITH_PARCELPORT_LCI_PCOUNTER:STRING*
- *HPX_WITH_PARCELPORT_LCW:BOOL*
- *HPX_WITH_PARCELPORT_LCW_LOG:STRING*
- *HPX_WITH_PARCELPORT_LCW_PCOUNTER:STRING*
- *HPX_WITH_PARCELPORT_LIBFABRIC:BOOL*
- *HPX_WITH_PARCELPORT_MPI:BOOL*
- *HPX_WITH_PARCELPORT_TCP:BOOL*
- *HPX_WITH_PARCEL_PROFILING:BOOL*

HPX_WITH_NETWORKING:BOOL
Enable support for networking and multi-node runs (default: ON)

HPX_WITH_PARCELPORT_ACTION_COUNTERS:BOOL
Enable performance counters reporting parcelport statistics on a per-action basis.

HPX_WITH_PARCELPORT_COUNTERS:BOOL
Enable performance counters reporting parcelport statistics.

HPX_WITH_PARCELPORT_GASNET:BOOL
Enable the GASNET based parcelport.

HPX_WITH_PARCELPORT_LCI:BOOL
Enable the LCI based parcelport.

HPX_WITH_PARCELPORT_LCI_LOG:STRING
Enable the LCI-parcelport-specific logger

HPX_WITH_PARCELPORT_LCI_PCOUNTER:STRING
Enable the LCI-parcelport-specific performance counter

HPX_WITH_PARCELPORT_LCW:BOOL
Enable the LCW based parcelport.

HPX_WITH_PARCELPORT_LCW_LOG:STRING
Enable the LCW-parcelport-specific logger

HPX_WITH_PARCELPORT_LCW_PCOUNTER:STRING
Enable the LCW-parcelport-specific performance counter

HPX_WITH_PARCELPORT_LIBFABRIC:BOOL
Enable the libfabric based parcelport. This is currently an experimental feature

HPX_WITH_PARCELPORT_MPI:BOOL
Enable the MPI based parcelport.

HPX_WITH_PARCELPORT_TCP:BOOL
Enable the TCP based parcelport.

HPX_WITH_PARCEL_PROFILING:BOOL
Enable profiling data for parcels

Profiling options

- *HPX_LIKWID_WITH_LIKWID:BOOL*
- *HPX_WITH_APEX:BOOL*
- *HPX_WITH_ITTNOTIFY:BOOL*
- *HPX_WITH_PAPI:BOOL*
- *HPX_WITH_TRACY:BOOL*

HPX_LIKWID_WITH_LIKWID:BOOL
Enable Likwid instrumentation support.

HPX_WITH_APEX:BOOL
Enable APEX instrumentation support.

HPX_WITH_ITTNOTIFY:BOOL
Enable Amplifier (ITT) instrumentation support.

HPX_WITH_PAPI:BOOL
Enable the PAPI based performance counter.

HPX_WITH_TRACY:BOOL
Enable Tracy instrumentation support.

Debugging options

- *HPX_WITH_ASSERTS_AS_CONTRACT_ASSERTS:BOOL*
- *HPX_WITH_ATTACH_DEBUGGER_ON_TEST_FAILURE:BOOL*
- *HPX_WITH_CONTRACTS:BOOL*
- *HPX_WITH_PARALLEL_TESTS_BIND_NONE:BOOL*
- *HPX_WITH_SANITIZERS:BOOL*
- *HPX_WITH_TESTS_COMMAND_LINE:STRING*
- *HPX_WITH_TESTS_DEBUG_LOG:BOOL*
- *HPX_WITH_TESTS_DEBUG_LOG_DESTINATION:STRING*
- *HPX_WITH_TESTS_MAX_THREADS_PER_LOCALITY:STRING*
- *HPX_WITH_THREAD_DEBUG_INFO:BOOL*
- *HPX_WITH_THREAD_DESCRIPTION_FULL:BOOL*
- *HPX_WITH_THREAD_GUARD_PAGE:BOOL*
- *HPX_WITH_VALGRIND:BOOL*
- *HPX_WITH_VERIFY_LOCKS:BOOL*
- *HPX_WITH_VERIFY_LOCKS_BACKTRACE:BOOL*

HPX_WITH_ASSERTS_AS_CONTRACT_ASSERTS:BOOL
Swap hpx_assert with hpx_contract_assert

HPX_WITH_ATTACH_DEBUGGER_ON_TEST_FAILURE:BOOL
Break the debugger if a test has failed (default: OFF)

HPX_WITH_CONTRACTS:BOOL
Enable C++ contracts support in HPX

HPX_WITH_PARALLEL_TESTS_BIND_NONE:BOOL
Pass –hpx:bind=none to tests that may run in parallel (cmake -j flag) (default: OFF)

HPX_WITH_SANITIZERS:BOOL
Configure with sanitizer instrumentation support.

HPX_WITH_TESTS_COMMAND_LINE:STRING
Add given command line options to all tests run

HPX_WITH_TESTS_DEBUG_LOG:BOOL
Turn on debug logs (–hpx:debug-hpx-log) for tests (default: OFF)

HPX_WITH_TESTS_DEBUG_LOG_DESTINATION:STRING
Destination for test debug logs (default: cout)

HPX_WITH_TESTS_MAX_THREADS_PER_LOCALITY:STRING
Maximum number of threads to use for tests (default: 0, use the number of threads specified by the test)

HPX_WITH_THREAD_DEBUG_INFO:BOOL

Enable thread debugging information (default: OFF, implicitly enabled in debug builds)

HPX_WITH_THREAD_DESCRIPTION_FULL:BOOL

Use function address for thread description (default: OFF)

HPX_WITH_THREAD_GUARD_PAGE:BOOL

Enable thread guard page (default: ON)

HPX_WITH_VALGRIND:BOOL

Enable Valgrind instrumentation support.

HPX_WITH_VERIFY_LOCKS:BOOL

Enable lock verification code (default: OFF, enabled in debug builds)

HPX_WITH_VERIFY_LOCKS_BACKTRACE:BOOL

Enable thread stack back trace being captured on lock registration (to be used in combination with HPX_WITH_VERIFY_LOCKS=ON, default: OFF)

Modules options

- *HPX_ALLOCATOR_SUPPORT_WITH_CACHING:BOOL*
- *HPX_COMMAND_LINE_HANDLING_LOCAL_WITH_JSON_CONFIGURATION_FILES:BOOL*
- *HPX_DATASTRUCTURES_WITH_ADAPT_STD_TUPLE:BOOL*
- *HPX_DATASTRUCTURES_WITH_ADAPT_STD_VARIANT:BOOL*
- *HPX_FILESYSTEM_WITH_BOOST_FILESYSTEM_COMPATIBILITY:BOOL*
- *HPX_FUNCTIONAL_WITH_BOOST_PLACEHOLDERS:BOOL*
- *HPX_IOSTREAM_WITH_WIDE_STREAMS:BOOL*
- *HPX_ITERATOR_SUPPORT_WITH_BOOST_ITERATOR_TRAVERSAL_TAG_COMPATIBILITY:BOOL*
- *HPX_LOGGING_WITH_SEPARATE_DESTINATIONS:BOOL*
- *HPX_SERIALIZATION_WITH_ALLOW_CONST_TUPLE_MEMBERS:BOOL*
- *HPX_SERIALIZATION_WITH_ALLOW_RAW_POINTER_SERIALIZATION:BOOL*
- *HPX_SERIALIZATION_WITH_ALL_TYPES_ARE_BITWISE_SERIALIZABLE:BOOL*
- *HPX_SERIALIZATION_WITH_BOOST_TYPES:BOOL*
- *HPX_SERIALIZATION_WITH_SUPPORTS_ENDIANESS:BOOL*
- *HPX_TOPOLOGY_WITH_ADDITIONAL_HWLOC_TESTING:BOOL*
- *HPX_WITH_POWER_COUNTER:BOOL*
- *HPX_WITH_SUPERVISION:BOOL*

HPX_ALLOCATOR_SUPPORT_WITH_CACHING:BOOL

Enable caching allocator. (default: ON)

HPX_COMMAND_LINE_HANDLING_LOCAL_WITH_JSON_CONFIGURATION_FILES:BOOL

Enable reading JSON formatted configuration files on the command line.

(default: OFF)

`HPX_DATASTRUCTURES_WITH_ADAPT_STD_TUPLE:BOOL`
Enable compatibility of hpx::get with std::tuple. (default: ON)

`HPX_DATASTRUCTURES_WITH_ADAPT_STD_VARIANT:BOOL`

Enable compatibility of hpx::get with std::variant.

(default: OFF)

`HPX_FILESYSTEM_WITH_BOOST_FILESYSTEM_COMPATIBILITY:BOOL`
Enable Boost.FileSystem compatibility. (default: OFF)

`HPX_FUNCTIONAL_WITH_BOOST_PLACEHOLDERS:BOOL`
Enable support for Boost placeholder types. (default: OFF)

`HPX_IOSTREAM_WITH_WIDE_STREAMS:BOOL`
Enable wide IO streams. (default: OFF)

`HPX_ITERATOR_SUPPORT_WITH_BOOST_ITERATOR_TRAVERSAL_TAG_COMPATIBILITY:BOOL`
Enable Boost.Iterator traversal tag compatibility. (default: OFF)

`HPX_LOGGING_WITH_SEPARATE_DESTINATIONS:BOOL`
Enable separate logging channels for AGAS, timing, and parcel transport. (default: ON)

`HPX_SERIALIZATION_WITH_ALLOW_CONST_TUPLE_MEMBERS:BOOL`
Enable serializing std::tuple with const members. (default: OFF)

`HPX_SERIALIZATION_WITH_ALLOW_RAW_POINTER_SERIALIZATION:BOOL`
Enable serializing raw pointers. (default: OFF)

`HPX_SERIALIZATION_WITH_ALL_TYPES_ARE_BITWISE_SERIALIZABLE:BOOL`
Assume all types are bitwise serializable. (default: OFF)

`HPX_SERIALIZATION_WITH_BOOST_TYPES:BOOL`
Enable serialization of certain Boost types. (default: OFF)

`HPX_SERIALIZATION_WITH_SUPPORTS_ENDIANESS:BOOL`
Support endian conversion on inout and output archives. (default: OFF)

`HPX_TOPOLOGY_WITH_ADDITIONAL_HWLOC_TESTING:BOOL`

Enable HWLOC filtering that makes it report no cores, this is purely an

option supporting better testing - do not enable under normal circumstances. (default: OFF)

`HPX_WITH_POWER_COUNTER:BOOL`
Enable use of performance counters based on pwr library (default: OFF)

`HPX_WITH_SUPERVISION:BOOL`
Enable supervision functionalities. (default: OFF)

Additional tools and libraries used by *HPX*

Here is a list of additional libraries and tools that are either optionally supported by the build system or are optionally required for certain examples or tests. These libraries and tools can be detected by the *HPX* build system.

Each of the tools or libraries listed here will be automatically detected if they are installed in some standard location. If a tool or library is installed in a different location, you can specify its base directory by appending `_ROOT` to the variable name as listed below. For instance, to configure a custom directory for `Boost`, specify `Boost_ROOT=/custom/boost/root`.

`Boost_ROOT:PATH`

Specifies where to look for the Boost installation to be used for compiling *HPX*. Set this if CMake is not able to locate a suitable version of Boost. The directory specified here can be either the root of an installed Boost distribution or the directory where you unpacked and built Boost without installing it (with staged libraries).

`Hwloc_ROOT:PATH`

Specifies where to look for the hwloc library. Set this if CMake is not able to locate a suitable version of hwloc. Hwloc provides platform- independent support for extracting information about the used hardware architecture (number of cores, number of NUMA domains, hyperthreading, etc.). *HPX* utilizes this information if available.

`Papi_ROOT:PATH`

Specifies where to look for the PAPI library. The PAPI library is needed to compile a special component exposing PAPI hardware events and counters as *HPX* performance counters. This is not available on the Windows platform.

`Amplifier_ROOT:PATH`

Specifies where to look for one of the tools of the Intel Parallel Studio product, either Intel Amplifier or Intel Inspector. This should be set if the CMake variable `HPX_USE_ITT_NOTIFY` is set to `ON`. Enabling ITT support in *HPX* will integrate any application with the mentioned Intel tools, which customizes the generated information for your application and improves the generated diagnostics.

In addition, some of the examples may need the following variables:

`Hdf5_ROOT:PATH`

Specifies where to look for the Hierarchical Data Format V5 (HDF5) include files and libraries.

2.4.5 Using *HPX* on Compiler Explorer

Compiler Explorer[65] (CE) is a browser-based tool that compiles C++ code in real time and shows assembly, program output, and optimization results side by side. It is widely used for quick experiments, conference demos, and sharing reproducible snippets. This page explains how to build *HPX* for CE, how to link against its static libraries from a raw compiler command, and how to use the *sandbox header* that *HPX* ships specifically for constrained environments.

Sandbox environment constraints

CE executes compiled programs inside a Linux container (nsjail) with no network access and limited resources. The constraints that matter most for *HPX* are:

- No network namespace — sockets cannot be opened. Any attempt to initialise the distributed runtime will fail with `EACCES` at startup.
- A small PID budget (typically 32 PIDs) — *HPX* starts cleanly within this limit when the worker thread count is kept low.

[65] https://godbolt.org

- A short execution timeout — CE's public instance allows roughly 10 seconds. An *HPX* fork or self-hosted instance can relax this to 30 seconds, which is enough for benchmarks on problem sizes of one million elements or more.
- Static linking is required — the sandbox does not augment `LD_LIBRARY_PATH`, so shared *HPX* libraries are not found at runtime.

Building *HPX* for Compiler Explorer

HPX ships a `godbolt-minimal` configure preset in `CMakePresets.json` that encodes all the flags needed for a CE-compatible build:

```
$ cmake --preset godbolt-minimal
$ cmake --build --preset godbolt-minimal
```

This preset enables:

- `HPX_WITH_STATIC_LINKING=ON` — bundles everything into the `.a` archives that CE links against.
- `HPX_WITH_NETWORKING=OFF` — disables the parcelset so *HPX* never attempts to open a network socket.
- `HPX_WITH_FETCH_ASIO=ON` — downloads Asio via FetchContent, removing the need for a system-level Asio installation.
- `HPX_WITH_MALLOC=system` — uses the system allocator; avoids a jemalloc or tcmalloc dependency.
- `HPX_WITH_TESTS=OFF`, `HPX_WITH_EXAMPLES=OFF`, `HPX_WITH_DOCUMENTATION=OFF` — skips everything that CE does not need.

The preset produces four static libraries under `build/godbolt-minimal/lib/`:

```
libhpx_wrap.a
libhpx_init.a
libhpx.a
libhpx_core.a
```

These are the only artifacts that CE consumes.

Note

The `HPX_WITH_CXX_STANDARD` variable pins the C++ standard for the build (e.g. `-DHPX_WITH_CXX_STANDARD=20`). This is *HPX*'s own cache variable and is distinct from `CMAKE_CXX_STANDARD`, which *HPX* rejects unless `HPX_USE_CMAKE_CXX_STANDARD` is also set.

Linking without CMake

CE's backend compiles user code with a raw `g++` or `clang++` invocation. The complete set of flags needed is:

```
$ g++ -std=c++20 -O2 -o my_program my_program.cpp          \
    -isystem /path/to/hpx/include                            \
    -isystem /path/to/boost/include                          \
    -L/path/to/hpx/lib                                       \
    -DHPX_APPLICATION_EXPORTS                                \
    -Wl,-wrap=main                                           \
```

(continues on next page)

(continued from previous page)

```
    -lhpx_wrap -lhpx_init -lhpx -lhpx_core                  \
    -lpthread -ldl -lrt
```

Two details here are easy to get wrong:

Library link order. The order `hpx_wrap` → `hpx_init` → `hpx` → `hpx_core` must be preserved. Reversing it produces undefined-reference errors because `hpx_wrap` depends on symbols in `hpx_init`, which depends on the full runtime in `hpx`, which in turn depends on the core library.

The `-Wl,-wrap=main` flag. Including `hpx/hpx_main.hpp` (see *Re-use the main() function as the main HPX entry point*) works by re-routing control through *HPX*'s own entry point before the user's `main` is called. On Linux this is implemented via the linker's `--wrap` option; the flag must therefore appear on the *linker* command line, not merely in the compile flags. Without it, the *HPX* runtime is never initialised and all API calls crash at startup. See *Linux implementation* for a detailed explanation of the mechanism.

Important

`-DHPX_APPLICATION_EXPORTS` must be passed as a preprocessor definition when compiling application code against the static libraries. Omitting it causes link failures related to *HPX*'s symbol visibility macros.

Writing *HPX* code for Compiler Explorer

The simplest way to write an *HPX* program for CE is to include `hpx/hpx_main.hpp`. This header arranges for the *HPX* runtime to be initialised before `main` runs, so the body of `main` can call any *HPX* API function directly:

```cpp
#include <hpx/hpx_main.hpp>
#include <hpx/algorithm.hpp>
#include <hpx/execution.hpp>

#include <iostream>
#include <numeric>
#include <vector>

int main()
{
    std::vector<int> v(1'000'000);
    std::iota(v.begin(), v.end(), 0);

    long long sum = hpx::transform_reduce(
        hpx::execution::par, v.begin(), v.end(), 0LL,
        std::plus<>{}, [](int x) { return static_cast<long long>(x); });

    std::cout << "sum = " << sum << "\n";
}
```

Caution

Include `hpx/hpx_main.hpp` in exactly one translation unit — the file that contains `main`. Including it in more than one file causes a multiple definition error for the `include_libhpx_wrap` variable that controls runtime initialisation.

The hpx/experimental/sandbox.hpp header

HPX ships a header-only toolkit at `hpx/experimental/sandbox.hpp` designed specifically for code running in constrained environments. It provides:

- **Environment introspection** — `hpx::experimental::sandbox::detect_environment()` returns an `environment_info` struct describing the number of physical cores, processing units, NUMA domains, and active *HPX* worker threads. The `is_sandbox` flag is set when the `COMPILER_EXPLORER` or `HPX_SANDBOX` environment variable is present, letting code adapt its behaviour automatically.
- **Timing** — `hpx::experimental::sandbox::measure(fn, iterations)` runs a callable `fn` for `iterations` repetitions (with one warmup pass) and returns the mean execution time in milliseconds.
- **Comparative benchmarking** — `hpx::experimental::sandbox::benchmark(label, seq_fn, par_fn, iterations)` measures both a sequential and a parallel version of the same computation, computes speedup and parallel efficiency, and returns a `benchmark_report` struct. Calling `report.print(std::cout)` produces a fixed-width table that renders cleanly in CE's output pane.

A typical usage pattern looks like this:

```
#include <hpx/hpx_main.hpp>
#include <hpx/algorithm.hpp>
#include <hpx/execution.hpp>
#include <hpx/experimental/sandbox.hpp>

#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>

int main()
{
    namespace sb = hpx::experimental::sandbox;

    sb::describe_environment(std::cout);

    std::vector<int> data(500'000);
    std::iota(data.begin(), data.end(), 0);

    auto report = sb::benchmark(
        "transform_reduce",
        [&]() {
            hpx::transform_reduce(
                hpx::execution::seq,
                data.begin(), data.end(), 0LL,
                std::plus<>{}, [](int x) { return (long long)x; });
        },
        [&]() {
            hpx::transform_reduce(
                hpx::execution::par,
                data.begin(), data.end(), 0LL,
                std::plus<>{}, [](int x) { return (long long)x; });
        });

    report.print(std::cout);
}
```

The output includes sequential and parallel mean times, speedup, parallel efficiency, and a verdict (`Excellent scaling`, `Good scaling`, `Moderate scaling`, `Limited scaling`, or `No speedup`).

> **Note**
>
> All functions in `hpx/experimental/sandbox.hpp` must be called from within a running *HPX* runtime, i.e., from an *HPX* thread. Calling them before *hpx::init* or after *hpx::finalize* is undefined behaviour. When using `hpx/hpx_main.hpp`, the body of `main` satisfies this requirement automatically.

Known limitations in sandboxed environments

- **Single locality only.** The distributed runtime can be compiled in (`HPX_WITH_DISTRIBUTED_RUNTIME=ON`) and actions on locality 0 work normally, but there is no way to launch a second locality from within CE's sandbox. Code that calls `hpx::find_all_localities()` or `hpx::get_num_localities()` will always see exactly one locality.
- **Networking is disabled.** `HPX_WITH_NETWORKING=OFF` means all parcelport-dependent functionality (remote actions, distributed data structures) is unavailable regardless of what the code requests.
- **Thread count is limited by the sandbox's CPU allocation.** CE's public instance typically exposes two cores. Use `--hpx:threads=N` on the command line or `hpx::init_params::cfg` in code to set the worker count explicitly rather than relying on hardware detection. See *Launching and configuring HPX applications* for details.
- **macOS link flag differs.** The `-Wl,-wrap=main` flag is Linux-specific. On macOS the linker uses `-Wl,-e,_initialize_main` instead. CE runs Linux containers, so this only matters when building the CE integration locally on macOS for testing.

2.4.6 Migration guide

The Migration Guide serves as a valuable resource for developers seeking to transition their parallel computing applications from different APIs (i.e. OpenMP, Intel Threading Building Blocks (TBB), MPI) to *HPX*. *HPX*, an advanced C++ library, offers a versatile and high-performance platform for parallel and distributed computing, providing a wide range of features and capabilities. This guide aims to assist developers in understanding the key differences between different APIs and *HPX*, and it provides step-by-step instructions for converting code to *HPX* code effectively.

Some general steps that can be used to migrate code to *HPX* code are the following:

1. Install *HPX* using the *Quick start* guide.
2. Include the *HPX* header files:

   Add the necessary header files for HPX at the beginning of your code, such as:

   ```
   #include <hpx/init.hpp>
   ```

3. Replace your code with *HPX* code using the guide that follows.
4. Use HPX-specific features and APIs:

   HPX provides additional features and APIs that can be used to take advantage of the library's capabilities. For example, you can use the HPX asynchronous execution to express fine-grained tasks and dependencies, or utilize HPX's distributed computing features for distributed memory systems.

5. Compile and run the *HPX* code:

   Compile the converted code with the *HPX* library and run it using the appropriate HPX runtime environment.

OpenMP

The OpenMP API supports multi-platform shared-memory parallel programming in C/C++. Typically it is used for loop-level parallelism, but it also supports function-level parallelism. Below are some examples on how to convert OpenMP to *HPX* code:

OpenMP parallel for loop

Parallel for loop

OpenMP code:

```
#pragma omp parallel for
for (int i = 0; i < n; ++i) {
    // loop body
}
```

HPX equivalent:

```
#include <hpx/algorithm.hpp>

hpx::experimental::for_loop(hpx::execution::par, 0, n, [&](int i) {
    // loop body
});
```

In the above code, the OpenMP *#pragma omp parallel for* directive is replaced with *hpx::experimental::for_loop* from the *HPX* library. The loop body within the lambda function will be executed in parallel for each iteration.

Private variables

OpenMP code:

```
int x = 0;

#pragma omp parallel for private(x)
for (int i = 0; i < n; ++i) {
    // loop body
}
```

HPX equivalent:

```
#include <hpx/algorithm.hpp>

hpx::experimental::for_loop(hpx::execution::par, 0, n, [&](int i) {
        int x = 0; // Declare 'x' as a local variable inside the loop body
        // loop body
});
```

The variable *x* is declared as a local variable inside the loop body, ensuring that it is private to each thread.

Shared variables

OpenMP code:

```
int x = 0;

#pragma omp parallel for shared(x)
for (int i = 0; i < n; ++i) {
    // loop body
}
```

HPX equivalent:

```
#include <hpx/algorithm.hpp>

std::atomic<int> x = 0; // Declare 'x' as a shared variable outside the loop

hpx::experimental::for_loop(hpx::execution::par, 0, n, [&](int i) {
    // loop body
});
```

To ensure variable *x* is shared among all threads, you simply have to declare it as an atomic variable outside the *for_loop*.

Number of threads

OpenMP code:

```
#pragma omp parallel for num_threads(2)
for (int i = 0; i < n; ++i) {
    // loop body
}
```

HPX equivalent:

```
#include <hpx/algorithm.hpp>
#include <hpx/execution.hpp>

hpx::execution::experimental::num_cores nc(2);

hpx::experimental::for_loop(hpx::execution::par.with(nc), 0, n, [&](int i) {
    // loop body
});
```

To declare the number of threads to be used for the parallel region, you can use *hpx::execution::experimental::num_cores* and pass the number of cores (*nc*) to *hpx::experimental::for_loop* using *hpx::execution::par.with(nc)*. This example uses 2 threads for the parallel loop.

Reduction

OpenMP code:

```
int s = 0;

#pragma omp parallel for reduction(+: s)
for (int i = 0; i < n; ++i) {
    s += i;
    // loop body
}
```

HPX equivalent:

```
#include <hpx/algorithm.hpp>
#include <hpx/execution.hpp>

int s = 0;

hpx::experimental::for_loop(hpx::execution::par, 0, n, reduction(s, 0, plus<>()), [&
↪](int i, int& accum) {
    accum += i;
    // loop body
});
```

The reduction clause specifies that the variable *s* should be reduced across iterations using the *plus*<> operation. It initializes *s* to *0* at the beginning of the loop and accumulates the values of *s* from each iteration using the + operator. The lambda function representing the loop body takes two parameters: *i*, which represents the loop index, and *accum*, which is the reduction variable *s*. The lambda function is executed for each iteration of the loop. The reduction ensures that the *accum* value is correctly accumulated across different iterations and threads.

Schedule

OpenMP code:

```
int s = 0;

// static scheduling with chunk size 1000
#pragma omp parallel for schedule(static, 1000)
for (int i = 0; i < n; ++i) {
    // loop body
}
```

HPX equivalent:

```
#include <hpx/algorithm.hpp>
#include <hpx/execution.hpp>

hpx::execution::experimental::static_chunk_size cs(1000);

hpx::experimental::for_loop(hpx::execution::par.with(cs), 0, n, [&](int i) {
    // loop body
});
```

To define the scheduling type, you can use the corresponding execution policy from *hpx::execution::experimental*, define the chunk size (cs, here declared as 1000) and pass it to the to *hpx::experimental::for_loop* using *hpx::execution::par.with(cs)*.

Accordingly, other types of scheduling are available and can be used in a similar manner:

```
#include <hpx/execution.hpp>
hpx::execution::experimental::dynamic_chunk_size cs(1000);
```

```
#include <hpx/execution.hpp>
hpx::execution::experimental::guided_chunk_size cs(1000);
```

```
#include <hpx/execution.hpp>
hpx::execution::experimental::auto_chunk_size cs(1000);
```

OpenMP single thread

OpenMP code:

```
{   // parallel code
    #pragma omp single
    {
        // single-threaded code
    }
    // more parallel code
}
```

HPX equivalent:

```
#include <hpx/mutex.hpp>

hpx::mutex mtx;

{   // parallel code
    {   // single-threaded code
        std::scoped_lock l(mtx);
    }
    // more parallel code
}
```

To make sure that only one thread accesses a specific code within a parallel section you can use *hpx::mutex* and *std::scoped_lock* to take ownership of the given mutex *mtx*. For more information about mutexes please refer to *Mutex*.

OpenMP tasks

Simple tasks

OpenMP code:

```
// executed asynchronously by any available thread
#pragma omp task
{
    // task code
}
```

HPX equivalent:

```
#include <hpx/future.hpp>

auto future = hpx::async([](){
    // task code
});
```

or

```
#include <hpx/future.hpp>

hpx::post([](){
    // task code
}); // fire and forget
```

The tasks in *HPX* can be defined simply by using the `async` function and passing as argument the code you wish to run asynchronously. Another alternative is to use `post` which is a fire-and-forget method.

Tip

If you think you will like to synchronize your tasks later on, we suggest you use *hpx::async* which provides synchronization options, while *hpx::post* explicitly states that there is no return value or way to synchronize with the function execution. Synchronization options are listed below.

Task wait

OpenMP code:

```
#pragma omp task
{
    // task code
}

#pragma omp taskwait
// code after completion of task
```

HPX equivalent:

```
#include <hpx/future.hpp>

hpx::async([](){
    // task code
}).get(); // wait for the task to complete

// code after completion of task
```

The *get()* function can be used to ensure that the task created with *hpx::async* is completed before the code continues executing beyond that point.

Multiple tasks synchronization

OpenMP code:

```
#pragma omp task
{
    // task 1 code
}

#pragma omp task
{
    // task 2 code
}

#pragma omp taskwait
// code after completion of both tasks 1 and 2
```

HPX equivalent:

```
#include <hpx/future.hpp>

auto future1 = hpx::async([](){
    // task 1 code
});

auto future2 = hpx::async([](){
    // task 2 code
});

auto future = hpx::when_all(future1, future2).then([](auto&&){
    // code after completion of both tasks 1 and 2
});
```

If you would like to synchronize multiple tasks, you can use the *hpx::when_all* function to define which futures have to be ready and the *then()* function to declare what should be executed once these futures are ready.

Dependencies

OpenMP code:

```
int a = 10;
int b = 20;
int c = 0;

#pragma omp task depend(in: a, b) depend(out: c)
{
    // task code
    c = 100;
}
```

HPX equivalent:

```
#include <hpx/future.hpp>

int a = 10;
int b = 20;
int c = 0;

// Create a future representing 'a'
auto future_a = hpx::make_ready_future(a);

// Create a future representing 'b'
auto future_b = hpx::make_ready_future(b);

// Create a task that depends on 'a' and 'b' and executes 'task_code'
auto future_c = hpx::dataflow(
    []() {
        // task code
        return 100;
    },
    future_a, future_b);

c = future_c.get();
```

If one of the arguments of *hpx::dataflow* is a future, then it will wait for the future to be ready to launch the thread. Hence, to define the dependencies of tasks you have to create futures representing the variables that create dependencies and pass them as arguments to *hpx::dataflow*. *get()* is used to save the result of the future to the desired variable.

Nested tasks

OpenMP code:

```
#pragma omp task
{
    // Outer task code
    #pragma omp task
    {
        // Inner task code
```

(continues on next page)

(continued from previous page)

```
    }
}
```

HPX equivalent:

```
#include <hpx/future.hpp>

auto future_outer = hpx::async([](){
    // Outer task code

    hpx::async([](){
        // Inner task code
    });
});
```

or

```
#include <hpx/future.hpp>

auto future_outer = hpx::post([](){ // fire and forget
    // Outer task code

    hpx::post([](){ // fire and forget
        // Inner task code
    });
});
```

If you have nested tasks, you can simply use nested *hpx::async* or *hpx::post* calls. The implementation is similar if you want to take care of synchronization:

OpenMP code:

```
#pragma omp taskwait
{
    // Outer task code
    #pragma omp taskwait
    {
        // Inner task code
    }
}
```

HPX equivalent:

```
#include <hpx/future.hpp>

auto future_outer = hpx::async([]() {
    // Outer task code

    hpx::async([]() {
        // Inner task code
    }).get();    // Wait for the inner task to complete
});

future_outer.get();    // Wait for the outer task to complete
```

Task yield

OpenMP code:

```
#pragma omp task
{
    // code before yielding
    #pragma omp taskyield
    // code after yielding
}
```

HPX equivalent:

```
#include <hpx/future.hpp>
#include <hpx/thread.hpp>

auto future = hpx::async([](){
    // code before yielding
});

// yield execution to potentially allow other tasks to run
hpx::this_thread::yield();

// code after yielding
```

After creating a task using *hpx::async*, *hpx::this_thread::yield* can be used to reschedule the execution of threads, allowing other threads to run.

Task group

OpenMP code:

```
#pragma omp taskgroup
{
    #pragma omp task
    {
        // task 1 code
    }

    #pragma omp task
    {
        // task 2 code
    }
}
```

HPX equivalent:

```
#include <hpx/task_group.hpp>

// Declare a task group
hpx::experimental::task_group tg;

// Run the tasks
```

(continues on next page)

(continued from previous page)

```
tg.run([](){
    // task 1 code
});
tg.run(
    // task 2 code
});

// Wait for the task group
tg.wait();
```

To create task groups, you can use *hpx::experimental::task_group*. The function *run()* can be used to run each task within the task group, while *wait()* can be used to achieve synchronization. If you do not care about waiting for the task group to complete its execution, you can simply remove the *wait()* function.

OpenMP sections

OpenMP code:

```
#pragma omp sections
{
    #pragma omp section
    // section 1 code
    #pragma omp section
    // section 2 code
} // implicit synchronization
```

HPX equivalent:

```
#include <hpx/future.hpp>

auto future_section1 = hpx::async([](){
    // section 1 code
});
auto future_section2 = hpx::async([](){
    // section 2 code
);

// synchronization: wait for both sections to complete
hpx::wait_all(future_section1, future_section2);
```

Unlike tasks, there is an implicit synchronization barrier at the end of each *sections* directive in OpenMP. This synchronization is achieved using *hpx::wait_all* function.

Note

If the *nowait* clause is used in the *sections* directive, then you can just remove the *hpx::wait_all* function while keeping the rest of the code as it is.

Intel Threading Building Blocks (TBB)

Intel Threading Building Blocks (TBB) provides a high-level interface for parallelism and concurrent programming using standard ISO C++ code. Below are some examples on how to convert Intel Threading Building Blocks (TBB) to *HPX* code:

parallel_for

Intel Threading Building Blocks (TBB) code:

```
auto values = std::vector<double>(10000);

tbb::parallel_for( tbb::blocked_range<int>(0,values.size()),
                   [&](tbb::blocked_range<int> r)
{
    for (int i=r.begin(); i<r.end(); ++i)
    {
        // loop body
    }
});
```

HPX equivalent:

```
#include <hpx/algorithm.hpp>

auto values = std::vector<double>(10000);

hpx::experimental::for_loop(hpx::execution::par, 0, values.size(), [&](int i) {
    // loop body
});
```

In the above code, *tbb::parallel_for* is replaced with *hpx::experimental::for_loop* from the *HPX* library. The loop body within the lambda function will be executed in parallel for each iteration.

parallel_for_each

Intel Threading Building Blocks (TBB) code:

```
auto values = std::vector<double>(10000);

tbb::parallel_for_each(values.begin(), values.end(), [&](){
    // loop body
});
```

HPX equivalent:

```
#include <hpx/algorithm.hpp>

auto values = std::vector<double>(10000);

hpx::for_each(hpx::execution::par, values.begin(), values.end(), [&](){
```

(continues on next page)

(continued from previous page)

```
    // loop body
});
```

By utilizing *hpx::for_each* and specifying a parallel execution policy with *hpx::execution::par*, it is possible to transform *tbb::parallel_for_each* into its equivalent counterpart in *HPX*.

parallel_invoke

Intel Threading Building Blocks (TBB) code:

```
tbb::parallel_invoke(task1, task2, task3);
```

HPX equivalent:

```
#include <hpx/future.hpp>

hpx::wait_all(hpx::async(task1), hpx::async(task2), hpx::async(task3));
```

To convert *tbb::parallel_invoke* to *HPX*, we use *hpx::async* to asynchronously execute each task, which returns a future representing the result of each task. We then pass these futures to *hpx::when_all*, which waits for all the futures to complete before returning.

parallel_pipeline

Intel Threading Building Blocks (TBB) code:

```
tbb::parallel_pipeline(4,
    tbb::make_filter<void, int>(tbb::filter::serial_in_order,
        [](tbb::flow_control& fc) -> int {
            // Generate numbers from 1 to 10
            static int i = 1;
            if (i <= 10) {
                return i++;
            }
            else {
                fc.stop();
                return 0;
            }
        }) &
    tbb::make_filter<int, int>(tbb::filter::parallel,
        [](int num) -> int {
            // Multiply each number by 2
            return num * 2;
        }) &
    tbb::make_filter<int, void>(tbb::filter::serial_in_order,
        [](int num) {
            // Print the results
            std::cout << num << " ";
        })
);
```

HPX equivalent:

```
#include <iostream>
#include <vector>
#include <ranges>
#include <hpx/algorithm.hpp>

// generate the values
auto range = std::views::iota(1) | std::views::take(10);

// materialize the output vector
std::vector<int> results(10);

// in parallel execution of pipeline and transformation
hpx::ranges::transform(
    hpx::execution::par, range, result.begin(), [](int i) { return 2 * i; });

// print the modified vector
for (int i : result)
{
    std::cout << i << " ";
}
std::cout << std::endl;
```

The line *auto range = std::views::iota(1) | std::views::take(10);* generates a range of values using the *std::views::iota* function. It starts from the value 1 and generates an infinite sequence of incrementing values. The *std::views::take(10)* function is then applied to limit the sequence to the first 10 values. The result is stored in the *range* variable.

Hint

A view is a lightweight object that represents a particular view of a sequence or range. It acts as a read-only interface to the original data, providing a way to query and traverse the elements without making any copies or modifications.

Views can be composed and chained together to form complex pipelines of operations. These operations are evaluated lazily, meaning that the actual computation is performed only when the result is needed or consumed.

Since views perform lazy evaluation, we use *std::vector<int> results(10);* to meterialize the vector that will store the transformed values. The *hpx::ranges::transform* function is then used to perform a parallel transformation on the range. The transformed values will be written to the *results* vector.

Hint

Ranges enable loop fusion by combining multiple operations into a single parallel loop, eliminating waiting time and reducing overhead. Using ranges, you can express these operations as a pipeline of transformations on a sequence of elements. This pipeline is evaluated in a single pass, performing all the desired operations in parallel without the need to wait between them.

In addition, *HPX* enhances the benefits of range fusion by offering parallel execution policies, which can be used to optimize the execution of the fused loop across multiple threads.

parallel_reduce

Reduction

Intel Threading Building Blocks (TBB) code:

```
auto values = std::vector<double>{1,2,3,4,5,6,7,8,9};

auto total = tbb::parallel_reduce(
                tbb::blocked_range<int>(0,values.size()),
                0.0,
                [&](tbb::blocked_range<int> r, double running_total)
                {
                    for (int i=r.begin(); i<r.end(); ++i)
                    {
                        running_total += values[i];
                    }


                    return running_total;
                },
                std::plus<double>());
```

HPX equivalent:

```
#include <hpx/numeric.hpp>

auto values = std::vector<double>{1,2,3,4,5,6,7,8,9};

auto total = hpx::reduce(
    hpx::execution::par, values.begin(), values.end(), 0, std::plus{});
```

By utilizing *hpx::reduce* and specifying a parallel execution policy with *hpx::execution::par*, it is possible to transform *tbb::parallel_reduce* into its equivalent counterpart in *HPX*. As demonstrated in the previous example, the management of intermediate results is seamlessly handled internally by *HPX*, eliminating the need for explicit consideration.

Transformation & Reduction

Intel Threading Building Blocks (TBB) code:

```
auto values = std::vector<double>{1,2,3,4,5,6,7,8,9};

auto transform_function(double current_value){
    // transformation code
}

auto total = tbb::parallel_reduce(
                tbb::blocked_range<int>(0,values.size()),
                0.0,
                [&](tbb::blocked_range<int> r, double transformed_val)
                {
                    for (int i=r.begin(); i<r.end(); ++i)
```

(continues on next page)

(continued from previous page)

```
                    {
                        transformed_val += transform_function(values[i]);
                    }
                    return transformed_val;
                },
                std::plus<double>());
```

HPX equivalent:

```
#include <hpx/numeric.hpp>

auto values = std::vector<double>{1,2,3,4,5,6,7,8,9};

auto transform_function(double current_value)
{
    // transformation code
}

auto total = hpx::transform_reduce(hpx::execution::par, values.begin(),
    values.end(), 0, std::plus{},
    [&](double current_value) { return transform_function(current_value); });
```

In situations where certain values require transformation before the reduction process, *HPX* provides a straightforward solution through *hpx::transform_reduce*. The *transform_function()* allows for the application of the desired transformation to each value.

parallel_scan

Intel Threading Building Blocks (TBB) code:

```
tbb::parallel_scan(tbb::blocked_range<size_t>(0, input.size()),
    0,
    [&input, &output](const tbb::blocked_range<size_t>& range, int& partial_sum, bool is_
↪final_scan) {
        for (size_t i = range.begin(); i != range.end(); ++i) {
            partial_sum += input[i];
            if (is_final_scan) {
                output[i] = partial_sum;
            }
        }
        return partial_sum;
    },
    [](int left_sum, int right_sum) {
        return left_sum + right_sum;
    }
);
```

HPX equivalent:

```
#include <hpx/numeric.hpp>

hpx::inclusive_scan(hpx::execution::par, input.begin(), input.end(),
```

(continues on next page)

(continued from previous page)

```
    output.begin(),
    [](const int& left, const int& right) { return left + right; });
```

hpx::inclusive_scan with *hpx::execution::par* as execution policy can be used to perform a prefix scan in parallel. The management of intermediate results is seamlessly handled internally by *HPX*, eliminating the need for explicit consideration. *input.begin()* and *input.end()* refer to the beginning and end of the sequence of elements the algorithm will be applied to respectively. *output.begin()* refers to the beginning of the destination, while the last argument specifies the function which will be invoked for each of the values of the input sequence.

Apart from *hpx::inclusive_scan*, *HPX* provides its users with *hpx::exclusive_scan*. The key difference between inclusive scan and exclusive scan lies in the treatment of the current element during the scan operation. In an inclusive scan, each element in the output sequence includes the contribution of the corresponding element in the input sequence, while in an exclusive scan, the current element in the input sequence does not contribute to the corresponding element in the output sequence.

parallel_sort

Intel Threading Building Blocks (TBB) code:

```
std::vector<int> numbers = {9, 2, 7, 1, 5, 3};

tbb::parallel_sort(numbers.begin(), numbers.end());
```

HPX equivalent:

```
#include <hpx/algorithm.hpp>

std::vector<int> numbers = {9, 2, 7, 1, 5, 3};

hpx::sort(hpx::execution::par, numbers.begin(), numbers.end());
```

hpx::sort provides an equivalent functionality to *tbb::parallel_sort*. When given a parallel execution policy with *hpx::execution::par*, the algorithm employs parallel execution, allowing for efficient sorting across available threads.

task_group

Intel Threading Building Blocks (TBB) code:

```
// Declare a task group
tbb::task_group tg;

// Run the tasks
tg.run(task1);
tg.run(task2);

// Wait for the task group
tg.wait();
```

HPX equivalent:

```
#include <hpx/task_group.hpp>

// Declare a task group
hpx::experimental::task_group tg;

// Run the tasks
tg.run(task1);
tg.run(task2);

// Wait for the task group
tg.wait();
```

HPX drew inspiration from Intel Threading Building Blocks (TBB) to introduce the *hpx::experimental::task_group* feature. Therefore, utilizing *hpx::experimental::task_group* provides an equivalent functionality to *tbb::task_group*.

MPI

MPI is a standardized communication protocol and library that allows multiple processes or nodes in a parallel computing system to exchange data and coordinate their execution.

List of MPI-*HPX* functions

MPI function	*HPX* equivalent
MPI_Allgather	*hpx::collectives::all_gather*
MPI_Allreduce	*hpx::collectives::all_reduce*
MPI_Alltoall	*hpx::collectives::all_to_all*
MPI_Barrier	*hpx::distributed::barrier*
MPI_Bcast	`hpx::collectives::broadcast_to` and `hpx::collectives::broadcast_from()` used with `get()`
MPI_Comm_siz	*hpx::get_num_localities*
MPI_Comm_ra	`hpx::get_locality_id`
MPI_Exscan	`hpx::collectives::exclusive_scan` used with `get()`
MPI_Gather	`hpx::collectives::gather_here` and `hpx::collectives::gather_there()` used with `get()`
MPI_Irecv	`hpx::collectives::get`
MPI_Isend	`hpx::collectives::set`
MPI_Reduce	*hpx::collectives::reduce_here* and *hpx::collectives::reduce_there* used with `get()`
MPI_Scan	`hpx::collectives::inclusive_scan` used with `get()`
MPI_Scatter	`hpx::collectives::scatter_to` and `hpx::collectives::scatter_from()`
MPI_Wait	`hpx::collectives::get` used with a future i.e. `setf.get()`

MPI_Send & MPI_Recv

Let's assume we have the following simple message passing code where each process sends a message to the next process in a circular manner. The exchanged message is modified and printed to the console.

MPI code:

```
#include <cstddef>
#include <cstdint>
#include <iostream>
#include <mpi.h>
#include <vector>

constexpr int times = 2;

int main(int argc, char *argv[]) {
MPI_Init(&argc, &argv);

int num_localities;
MPI_Comm_size(MPI_COMM_WORLD, &num_localities);

int this_locality;
MPI_Comm_rank(MPI_COMM_WORLD, &this_locality);

int next_locality = (this_locality + 1) % num_localities;
std::vector<int> msg_vec = {0, 1};

int cnt = 0;
int msg = msg_vec[this_locality];

int recv_msg;
MPI_Request request_send, request_recv;
MPI_Status status;

while (cnt < times) {
    cnt += 1;

    MPI_Isend(&msg, 1, MPI_INT, next_locality, cnt, MPI_COMM_WORLD,
            &request_send);
    MPI_Irecv(&recv_msg, 1, MPI_INT, next_locality, cnt, MPI_COMM_WORLD,
            &request_recv);

    MPI_Wait(&request_send, &status);
    MPI_Wait(&request_recv, &status);

    std::cout << "Time: " << cnt << ", Locality " << this_locality
            << " received msg: " << recv_msg << "\n";

    recv_msg += 10;
    msg = recv_msg;
}

MPI_Finalize();
return 0;
```

(continues on next page)

(continued from previous page)

```
}
```

HPX equivalent:

```
#include <hpx/config.hpp>

#if !defined(HPX_COMPUTE_DEVICE_CODE)
#include <hpx/algorithm.hpp>
#include <hpx/hpx_init.hpp>
#include <hpx/modules/collectives.hpp>

#include <cstddef>
#include <cstdint>
#include <iostream>
#include <utility>
#include <vector>

using namespace hpx::collectives;

constexpr char const* channel_communicator_name =
    "/example/channel_communicator/";

// the number of times
constexpr int times = 2;

int hpx_main()
{
    std::uint32_t num_localities = hpx::get_num_localities(hpx::launch::sync);
    std::uint32_t this_locality = hpx::get_locality_id();

    // allocate channel communicator
    // NOTE: The communicator object must be kept alive until all
    // participating sites have successfully connected. Destroying it
    // prematurely may cause other sites to hang. Consider using a barrier
    // after creation if the communicator would otherwise go out of scope.
    auto comm = create_channel_communicator(hpx::launch::sync,
        channel_communicator_name, num_sites_arg(num_localities),
        this_site_arg(this_locality));

    std::uint32_t next_locality = (this_locality + 1) % num_localities;
    std::vector<int> msg_vec = {0, 1};

    int cnt = 0;
    int msg = msg_vec[this_locality];

    // send values to another locality
    auto setf = set(comm, that_site_arg(next_locality), msg, tag_arg(cnt));
    auto got_msg = get<int>(comm, that_site_arg(next_locality), tag_arg(cnt));

    setf.get();

    while (cnt < times)
```

(continues on next page)

(continued from previous page)

```
    {
        cnt += 1;

        auto done_msg = got_msg.then([&](auto&& f) {
            int rec_msg = f.get();
            std::cout << "Time: " << cnt << ", Locality " << this_locality
                      << " received msg: " << rec_msg << "\n";

            // change msg by adding 10
            rec_msg += 10;

            // start next round
            setf =
                set(comm, that_site_arg(next_locality), rec_msg, tag_arg(cnt));
            got_msg =
                get<int>(comm, that_site_arg(next_locality), tag_arg(cnt));
            setf.get();
        });

        done_msg.get();
    }

    return hpx::finalize();
}
#endif

int main(int argc, char* argv[])
{
#if !defined(HPX_COMPUTE_DEVICE_CODE)
    hpx::init_params params;
    params.cfg = {"--hpx:run-hpx-main"};
    return hpx::init(argc, argv, params);
#else
    (void) argc;
    (void) argv;
    return 0;
#endif
}
```

To perform message passing between different processes in *HPX* we can use a channel communicator. To understand this example, let's focus on the *hpx_main()* function:

- *hpx::get_num_localities(hpx::launch::sync)* retrieves the number of localities, while *hpx::get_locality_id()* returns the ID of the current locality.
- *create_channel_communicator* function is used to create a channel to serve the communication. This function takes several arguments, including the launch policy (*hpx::launch::sync*), the name of the communicator (*channel_communicator_name*), the number of localities, and the ID of the current locality.
- The communication follows a ring pattern, where each process (or locality) sends a message to its neighbor in a circular manner. This means that the messages circulate around the localities, ensuring that the communication wraps around when reaching the end of the locality sequence. To achieve this, the *next_locality* variable is calculated as the ID of the next locality in the ring.
- The initial values for the communication are set (*msg_vec*, *cnt*, *msg*).

- The *set()* function is called to send the message to the next locality in the ring. The message is sent asynchronously and is associated with a tag (*cnt*).
- The *get()* function is called to receive a message from the next locality. It is also associated with the same tag as the *set()* operation.
- The *setf.get()* call blocks until the message sending operation is complete.
- A continuation is set up using the function *then()* to handle the received message. Inside the continuation:
  - The received message value (*rec_msg*) is retrieved using *f.get()*.
  - The received message is printed to the console and then modified by adding 10.
  - The *set()* and *get()* operations are repeated to send and receive the modified message to the next locality.
  - The *setf.get()* call blocks until the new message sending operation is complete.
- The *done_msg.get()* call blocks until the continuation is complete for the current loop iteration.

Having said that, we conclude to the following table:

MPI_Gather

The following code gathers data from all processes to the root process and verifies the gathered data in the root process.

MPI code:

```
#include <iostream>
#include <mpi.h>
#include <numeric>
#include <vector>

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int num_localities, this_locality;
    MPI_Comm_size(MPI_COMM_WORLD, &num_localities);
    MPI_Comm_rank(MPI_COMM_WORLD, &this_locality);

    std::vector<int> local_data; // Data to be gathered

    if (this_locality == 0) {
        local_data.resize(num_localities); // Resize the vector on the root process
    }

    // Each process calculates its local data value
    int my_data = 42 + this_locality;

    for (std::uint32_t i = 0; i != 10; ++i) {

        // Gather data from all processes to the root process (process 0)
        MPI_Gather(&my_data, 1, MPI_INT, local_data.data(), 1, MPI_INT, 0,
                   MPI_COMM_WORLD);

        // Only the root process (process 0) will print the gathered data
        if (this_locality == 0) {
```

(continues on next page)

(continued from previous page)

```
        std::cout << "Gathered data on the root: ";
        for (int i = 0; i < num_localities; ++i) {
            std::cout << local_data[i] << " ";
        }
        std::cout << std::endl;
        }
    }
    std::cout << std::endl;

    MPI_Finalize();
    return 0;
}
```

HPX equivalent:

```
std::uint32_t num_localities = hpx::get_num_localities(hpx::launch::sync);
std::uint32_t this_locality = hpx::get_locality_id();

// test functionality based on immediate local result value
auto gather_direct_client = create_communicator(gather_direct_basename,
    num_sites_arg(num_localities), this_site_arg(this_locality));

for (std::uint32_t i = 0; i != 10; ++i)
{
    if (this_locality == 0)
    {
        hpx::future<std::vector<std::uint32_t>> overall_result =
            gather_here(gather_direct_client, std::uint32_t(42));

        std::vector<std::uint32_t> sol = overall_result.get();
        std::cout << "Gathered data on the root:";

        for (std::size_t j = 0; j != sol.size(); ++j)
        {
            HPX_TEST(j + 42 == sol[j]);
            std::cout << " " << sol[j];
        }
        std::cout << std::endl;
    }
    else
    {
        hpx::future<void> overall_result =
            gather_there(gather_direct_client, this_locality + 42);
        overall_result.get();
    }

}
```

This code will print 10 times the following message:

```
Gathered data on the root: 42 43
```

HPX uses two functions to implement the functionality of *MPI_Gather*: *gather_here* and *gather_there*. *gather_here* is gathering data from all localities to the locality with ID 0 (root locality). *gather_there* allows non-root localities to participate in the gather operation by sending data to the root locality. In more detail:

- *hpx::get_num_localities(hpx::launch::sync)* retrieves the number of localities, while *hpx::get_locality_id()* returns the ID of the current locality.
- The function *create_communicator()* is used to create a communicator called *gather_direct_client*.
- If the current locality is the root (its ID is equal to 0):
  - The *gather_here* function is used to perform the gather operation. It collects data from all other localities into the *overall_result* future object. The function arguments provide the necessary information, such as the base name for the gather operation (*gather_direct_basename*), the value to be gathered (*value*), the number of localities (*num_localities*), the current locality ID (*this_locality*), and the generation number (related to the gather operation).
  - The *get()* member function of the *overall_result* future is used to retrieve the gathered data.
  - The next *for* loop is used to verify the correctness of the gathered data (*sol*). *HPX_TEST* is a macro provided by the *HPX* testing utilities to perform similar testing with the Standard C++ macro *assert*.
- If the current locality is not the root:
  - The *gather_there* function is used to participate in the gather operation initiated by the root locality. It sends the data (in this case, the value *this_locality + 42*) to the root locality, indicating that it should be included in the gathering.
  - The *get()* member function of the *overall_result* future is used to wait for the gather operation to complete for this locality.

MPI_Scatter

The following code gathers data from all processes to the root process and verifies the gathered data in the root process.

MPI code:

```
#include <iostream>
#include <mpi.h>
#include <vector>

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int num_localities, this_locality;
    MPI_Comm_size(MPI_COMM_WORLD, &num_localities);
    MPI_Comm_rank(MPI_COMM_WORLD, &this_locality);

    int num_localities = num_localities;
    std::vector<int> data(num_localities);

    if (this_locality == 0) {
        // Fill the data vector on the root locality (locality 0)
        for (int i = 0; i < num_localities; ++i) {
        data[i] = 42 + i;
        }
    }
```

(continues on next page)

(continued from previous page)

```
    int local_data; // Variable to store the received data

    // Scatter data from the root locality to all other localities
    MPI_Scatter(&data[0], 1, MPI_INT, &local_data, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // Now, each locality has its own local_data

    // Print the local_data on each locality
    std::cout << "Locality " << this_locality << " received " << local_data
              << std::endl;

    MPI_Finalize();
    return 0;
}
```

HPX equivalent:

```
std::uint32_t num_localities = hpx::get_num_localities(hpx::launch::sync);
HPX_TEST_LTE(std::uint32_t(2), num_localities);

std::uint32_t this_locality = hpx::get_locality_id();

auto scatter_direct_client =
    hpx::collectives::create_communicator(scatter_direct_basename,
        num_sites_arg(num_localities), this_site_arg(this_locality));

// test functionality based on immediate local result value
for (std::uint32_t i = 0; i != 10; ++i)
{
    if (this_locality == 0)
    {
        std::vector<std::uint32_t> data(num_localities);
        std::iota(data.begin(), data.end(), 42 + i);

        hpx::future<std::uint32_t> result =
            scatter_to(scatter_direct_client, std::move(data));

        HPX_TEST_EQ(i + 42 + this_locality, result.get());
    }
    else
    {
        hpx::future<std::uint32_t> result =
            scatter_from<std::uint32_t>(scatter_direct_client);

        HPX_TEST_EQ(i + 42 + this_locality, result.get());

        std::cout << "Locality " << this_locality << " received "
                  << i + 42 + this_locality << std::endl;
    }
}
```

For num_localities = 2 and since we run for 10 iterations this code will print the following message:

```
Locality 1 received 43
Locality 1 received 44
Locality 1 received 45
Locality 1 received 46
Locality 1 received 47
Locality 1 received 48
Locality 1 received 49
Locality 1 received 50
Locality 1 received 51
Locality 1 received 52
```

HPX uses two functions to implement the functionality of *MPI_Scatter*: *hpx::scatter_to* and *hpx::scatter_from*. *hpx::scatter_to* is distributing the data from the locality with ID 0 (root locality) to all other localities. *hpx::scatter_from* allows non-root localities to receive the data from the root locality. In more detail:

- *hpx::get_num_localities(hpx::launch::sync)* retrieves the number of localities, while *hpx::get_locality_id()* returns the ID of the current locality.
- The function *hpx::collectives::create_communicator()* is used to create a communicator called *scatter_direct_client*.
- If the current locality is the root (its ID is equal to 0):
  - The data vector is filled with values ranging from *42 + i* to *42 + i + num_localities - 1*.
  - The *hpx::scatter_to* function is used to perform the scatter operation using the communicator *scatter_direct_client*. This scatters the data vector to other localities and returns a future representing the result.
  - *HPX_TEST_EQ* is a macro provided by the *HPX* testing utilities to test the distributed values.
- If the current locality is not the root:
  - The *hpx::scatter_from* function is used to collect the data by the root locality.
  - *HPX_TEST_EQ* is a macro provided by the *HPX* testing utilities to test the collected values.

MPI_Allgather

The following code gathers data from all processes and sends the data to all processes.

MPI code:

```
#include <cstdint>
#include <iostream>
#include <mpi.h>
#include <vector>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Get the number of MPI processes
    int num_localities = size;
```

(continues on next page)

(continued from previous page)

```
    // Get the MPI process rank
    int here = rank;

    std::uint32_t value = here;

    std::vector<std::uint32_t> r(num_localities);

    // Perform an all-gather operation to gather values from all processes.
    MPI_Allgather(&value, 1, MPI_UINT32_T, r.data(), 1, MPI_UINT32_T,
                  MPI_COMM_WORLD);

    // Print the result.
    std::cout << "Locality " << here << " has values:";
    for (size_t j = 0; j < r.size(); ++j) {
        std::cout << " " << r[j];
    }
    std::cout << std::endl;

    MPI_Finalize();
    return 0;
}
```

HPX equivalent:

```
std::uint32_t num_localities = hpx::get_num_localities(hpx::launch::sync);
std::uint32_t here = hpx::get_locality_id();

// test functionality based on immediate local result value
auto all_gather_direct_client =
    create_communicator(all_gather_direct_basename,
        num_sites_arg(num_localities), this_site_arg(here));

std::uint32_t value = here;

hpx::future<std::vector<std::uint32_t>> overall_result =
    all_gather(all_gather_direct_client, value);

std::vector<std::uint32_t> r = overall_result.get();

std::cout << "Locality " << here << " has values:";
for (std::size_t j = 0; j != r.size(); ++j)
{
    std::cout << " " << j;
}
std::cout << std::endl;
```

For num_localities = 2 this code will print the following message:

```
Locality 0 has values: 0 1
Locality 1 has values: 0 1
```

HPX uses the function *all_gather* to implement the functionality of *MPI_Allgather*. In more detail:

- *hpx::get_num_localities(hpx::launch::sync)* retrieves the number of localities, while *hpx::get_locality_id()* returns the ID of the current locality.

- The function *hpx::collectives::create_communicator()* is used to create a communicator called *all_gather_direct_client*.
- The values that the localities exchange with each other are equal to each locality's ID.
- The gather operation is performed using *all_gather*. The result is stored in an *hpx::future* object called *overall_result*, which represents a future result that can be retrieved later when needed.
- The *get()* function waits until the result is available and then stores it in the vector called *r*.

MPI_Allreduce

The following code combines values from all processes and distributes the result back to all processes.

MPI code:

```
#include <cstdint>
#include <iostream>
#include <mpi.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Get the number of MPI processes
    int num_localities = size;

    // Get the MPI process rank
    int here = rank;

    // Create a communicator for the all reduce operation.
    MPI_Comm all_reduce_direct_client;
    MPI_Comm_split(MPI_COMM_WORLD, 0, rank, &all_reduce_direct_client);

    // Perform the all reduce operation to calculate the sum of 'here' values.
    std::uint32_t value = here;
    std::uint32_t res = 0;
    MPI_Allreduce(&value, &res, 1, MPI_UINT32_T, MPI_SUM,
                  all_reduce_direct_client);

    std::cout << "Locality " << rank << " has value: " << res << std::endl;

    MPI_Finalize();
    return 0;
}
```

HPX equivalent:

```
std::uint32_t const num_localities =
    hpx::get_num_localities(hpx::launch::sync);
```

(continues on next page)

(continued from previous page)

```
std::uint32_t const here = hpx::get_locality_id();

auto const all_reduce_direct_client =
    create_communicator(all_reduce_direct_basename,
        num_sites_arg(num_localities), this_site_arg(here));

std::uint32_t value = here;

hpx::future<std::uint32_t> overall_result =
    all_reduce(all_reduce_direct_client, value, std::plus<std::uint32_t>{});

std::uint32_t res = overall_result.get();
std::cout << "Locality " << here << " has value: " << res << std::endl;
```

For num_localities = 2 this code will print the following message:

```
Locality 0 has value: 1
Locality 1 has value: 1
```

HPX uses the function *all_reduce* to implement the functionality of *MPI_Allreduce*. In more detail:

- *hpx::get_num_localities(hpx::launch::sync)* retrieves the number of localities, while *hpx::get_locality_id()* returns the ID of the current locality.
- The function *hpx::collectives::create_communicator()* is used to create a communicator called *all_reduce_direct_client*.
- The value of each locality is equal to its ID.
- The reduce operation is performed using *all_reduce*. The result is stored in an *hpx::future* object called *overall_result*, which represents a future result that can be retrieved later when needed.
- The *get()* function waits until the result is available and then stores it in the variable *res*.

MPI_Alltoall

The following code gathers data from and scatters data to all processes.

MPI code:

```
#include <algorithm>
#include <cstdint>
#include <iostream>
#include <mpi.h>
#include <vector>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Get the number of MPI processes
```

(continues on next page)

(continued from previous page)

```
    int num_localities = size;

    // Get the MPI process rank
    int this_locality = rank;

    // Create a communicator for all-to-all operation.
    MPI_Comm all_to_all_direct_client;
    MPI_Comm_split(MPI_COMM_WORLD, 0, rank, &all_to_all_direct_client);

    std::vector<std::uint32_t> values(num_localities);
    std::fill(values.begin(), values.end(), this_locality);

    // Create vectors to store received values.
    std::vector<std::uint32_t> r(num_localities);

    // Perform an all-to-all operation to exchange values with other localities.
    MPI_Alltoall(values.data(), 1, MPI_UINT32_T, r.data(), 1, MPI_UINT32_T,
                 all_to_all_direct_client);

    // Print the results.
    std::cout << "Locality " << this_locality << " has values:";
    for (std::size_t j = 0; j != r.size(); ++j) {
        std::cout << " " << r[j];
    }
    std::cout << std::endl;

    MPI_Finalize();
    return 0;
}
```

HPX equivalent:

```
std::uint32_t num_localities = hpx::get_num_localities(hpx::launch::sync);
std::uint32_t this_locality = hpx::get_locality_id();

auto all_to_all_direct_client =
    create_communicator(all_to_all_direct_basename,
        num_sites_arg(num_localities), this_site_arg(this_locality));

std::vector<std::uint32_t> values(num_localities);
std::fill(values.begin(), values.end(), this_locality);

hpx::future<std::vector<std::uint32_t>> overall_result =
    all_to_all(all_to_all_direct_client, std::move(values));

std::vector<std::uint32_t> r = overall_result.get();
std::cout << "Locality " << this_locality << " has values:";

for (std::size_t j = 0; j != r.size(); ++j)
{
    std::cout << " " << r[j];
}
```

(continues on next page)

(continued from previous page)

```
std::cout << std::endl;
```

For num_localities = 2 this code will print the following message:

```
Locality 0 has values: 0 1
Locality 1 has values: 0 1
```

HPX uses the function *all_to_all* to implement the functionality of *MPI_Alltoall*. In more detail:

- *hpx::get_num_localities(hpx::launch::sync)* retrieves the number of localities, while *hpx::get_locality_id()* returns the ID of the current locality.
- The function *hpx::collectives::create_communicator()* is used to create a communicator called *all_to_all_direct_client*.
- The value each locality sends is equal to its ID.
- The all-to-all operation is performed using *all_to_all*. The result is stored in an *hpx::future* object called *overall_result*, which represents a future result that can be retrieved later when needed.
- The *get()* function waits until the result is available and then stores it in the variable *r*.

MPI_Barrier

The following code shows how barrier is used to synchronize multiple processes.

MPI code:

```
#include <cstdlib>
#include <iostream>
#include <mpi.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    std::size_t iterations = 5;

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    for (std::size_t i = 0; i != iterations; ++i) {
        MPI_Barrier(MPI_COMM_WORLD);
        if (rank == 0) {
        std::cout << "Iteration " << i << " completed." << std::endl;
        }
    }

    MPI_Finalize();
    return 0;
}
```

HPX equivalent:

```
std::size_t iterations = 5;
std::uint32_t this_locality = hpx::get_locality_id();

char const* const barrier_test_name = "/test/barrier/multiple";

hpx::distributed::barrier b(barrier_test_name);
for (std::size_t i = 0; i != iterations; ++i)
{
    b.wait();
    if (this_locality == 0)
    {
        std::cout << "Iteration " << i << " completed." << std::endl;
    }
}
```

This code will print the following message:

```
Iteration 0 completed.
Iteration 1 completed.
Iteration 2 completed.
Iteration 3 completed.
Iteration 4 completed.
```

HPX uses the function *barrier* to implement the functionality of *MPI_Barrier*. In more detail:

- After defining the number of iterations, we use *hpx::get_locality_id()* to get the ID of the current locality.
- *char const* const barrier_test_name = "/test/barrier/multiple"*: This line defines a constant character array as the name of the barrier. This name is used to identify the barrier across different localities. All participating threads that use this name will synchronize at this barrier.
- Using *hpx::distributed::barrier b(barrier_test_name)*, we create an instance of the distributed barrier with the previously defined name. This barrier will be used to synchronize the execution of threads across different localities.
- Running for all the desired iterations, we use *b.wait()* to synchronize the threads. Each thread waits until all other threads also reach this point before any of them can proceed further.

MPI_Bcast

The following code broadcasts data from one process to all other processes.

MPI code:

```
#include <iostream>
#include <mpi.h>

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int num_localities;
    MPI_Comm_size(MPI_COMM_WORLD, &num_localities);

    int here;
```

(continues on next page)

(continued from previous page)

```
    MPI_Comm_rank(MPI_COMM_WORLD, &here);

    int value;

    for (int i = 0; i < 5; ++i) {
        if (here == 0) {
            value = i + 42;
        }

        // Broadcast the value from process 0 to all other processes
        MPI_Bcast(&value, 1, MPI_INT, 0, MPI_COMM_WORLD);

        if (here != 0) {
            std::cout << "Locality " << here << " received " << value << std::endl;
        }

    }

    MPI_Finalize();
    return 0;
}
```

HPX equivalent:

```
std::uint32_t num_localities = hpx::get_num_localities(hpx::launch::sync);

std::uint32_t here = hpx::get_locality_id();

auto broadcast_direct_client =
    create_communicator(broadcast_direct_basename,
        num_sites_arg(num_localities), this_site_arg(here));

// test functionality based on immediate local result value
for (std::uint32_t i = 0; i != 5; ++i)
{
    if (here == 0)
    {
        hpx::future<std::uint32_t> result =
            broadcast_to(broadcast_direct_client, i + 42);

        result.get();
    }
    else
    {
        hpx::future<std::uint32_t> result =
            hpx::collectives::broadcast_from<std::uint32_t>(
                broadcast_direct_client);

        uint32_t r = result.get();

        std::cout << "Locality " << here << " received " << r << std::endl;
    }
```

(continues on next page)

(continued from previous page)

```
}
```

For num_localities = 2 this code will print the following message:

```
Locality 1 received 42
Locality 1 received 43
Locality 1 received 44
Locality 1 received 45
Locality 1 received 46
```

HPX uses two functions to implement the functionality of *MPI_Bcast*: *broadcast_to* and *broadcast_from*. *broadcast_to* is broadcasting the data from the root locality to all other localities. *broadcast_from* allows non-root localities to collect the data sent by the root locality. In more detail:

- *hpx::get_num_localities(hpx::launch::sync)* retrieves the number of localities, while *hpx::get_locality_id()* returns the ID of the current locality.
- The function *create_communicator()* is used to create a communicator called *broadcast_direct_client*.
- If the current locality is the root (its ID is equal to 0):
  - The *broadcast_to* function is used to perform the broadcast operation using the communicator *broadcast_direct_client*. This sends the data to other localities and returns a future representing the result.
  - The *get()* member function of the *result* future is used to wait for and retrieve the result.
- If the current locality is not the root:
  - The *broadcast_from* function is used to collect the data by the root locality.
  - The *get()* member function of the *result* future is used to wait for the result.

MPI_Exscan

The following code computes the exclusive scan (partial reductions) of data on a collection of processes.

MPI code:

```
#include <iostream>
#include <mpi.h>
#include <numeric>
#include <vector>

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int num_localities;
    MPI_Comm_size(MPI_COMM_WORLD, &num_localities);

    int here;
    MPI_Comm_rank(MPI_COMM_WORLD, &here);

    // Calculate the value for this locality (here)
    int value = here;

    // Perform an exclusive scan
```

(continues on next page)

(continued from previous page)

```
    std::vector<int> result(num_localities);
    MPI_Exscan(&value, &result[0], 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);

    if (here != 0) {
        int r = result[here - 1]; // Result is in the previous rank's slot

        std::cout << "Locality " << here << " has value " << r << std::endl;
    }

    MPI_Finalize();
    return 0;
}
```

HPX equivalent:

```
std::uint32_t num_localities = hpx::get_num_localities(hpx::launch::sync);
std::uint32_t here = hpx::get_locality_id();

auto exclusive_scan_client = create_communicator(exclusive_scan_basename,
    num_sites_arg(num_localities), this_site_arg(here));

// test functionality based on immediate local result value
std::uint32_t value = here;

hpx::future<std::uint32_t> overall_result = exclusive_scan(
    exclusive_scan_client, value, std::plus<std::uint32_t>{});

uint32_t r = overall_result.get();

if (here != 0)
{
    std::cout << "Locality " << here << " has value " << r << std::endl;
}
```

For num_localities = 2 this code will print the following message:

```
Locality 1 has value 0
```

HPX uses the function *exclusive_scan* to implement *MPI_Exscan*. In more detail:

- *hpx::get_num_localities(hpx::launch::sync)* retrieves the number of localities, while *hpx::get_locality_id()* returns the ID of the current locality.
- The function *create_communicator()* is used to create a communicator called *exclusive_scan_client*.
- The *exclusive_scan* function is used to perform the exclusive scan operation using the communicator *exclusive_scan_client*. *std::plus<std::uint32_t>{}* specifies the binary associative operator to use for the scan. In this case, it's addition for summing values.
- The *get()* member function of the *overall_result* future is used to wait for the result.

MPI_Scan

The following code Computes the inclusive scan (partial reductions) of data on a collection of processes.

MPI code:

```
#include <iostream>
#include <mpi.h>
#include <numeric>
#include <vector>

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int num_localities;
    MPI_Comm_size(MPI_COMM_WORLD, &num_localities);

    int here;
    MPI_Comm_rank(MPI_COMM_WORLD, &here);

    // Calculate the value for this locality (here)
    int value = here;

    std::vector<int> result(num_localities);

    MPI_Scan(&value, &result[0], 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);

    std::cout << "Locality " << here << " has value " << result[0] << std::endl;

    MPI_Finalize();
    return 0;
}
```

HPX equivalent:

```
std::uint32_t num_localities = hpx::get_num_localities(hpx::launch::sync);
std::uint32_t here = hpx::get_locality_id();

auto inclusive_scan_client = create_communicator(inclusive_scan_basename,
    num_sites_arg(num_localities), this_site_arg(here));

std::uint32_t value = here;

hpx::future<std::uint32_t> overall_result = inclusive_scan(
    inclusive_scan_client, value, std::plus<std::uint32_t>{});

uint32_t r = overall_result.get();

std::cout << "Locality " << here << " has value " << r << std::endl;
```

For num_localities = 2 this code will print the following message:

```
Locality 0 has value 0
Locality 1 has value 1
```

HPX uses the function *inclusive_scan* to implement *MPI_Scan*. In more detail:

- *hpx::get_num_localities(hpx::launch::sync)* retrieves the number of localities, while *hpx::get_locality_id()* returns the ID of the current locality.
- The function *create_communicator()* is used to create a communicator called *inclusive_scan_client*.
- The *inclusive_scan* function is used to perform the exclusive scan operation using the communicator *inclusive_scan_client*. *std::plus<std::uint32_t>{}* specifies the binary associative operator to use for the scan. In this case, it's addition for summing values.
- The *get()* member function of the *overall_result* future is used to wait for the result.

MPI_Reduce

The following code performs a global reduce operation across all processes.

MPI code:

```
#include <iostream>
#include <mpi.h>

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int num_processes;
    MPI_Comm_size(MPI_COMM_WORLD, &num_processes);

    int this_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &this_rank);

    int value = this_rank;

    int result = 0;

    // Perform the reduction operation
    MPI_Reduce(&value, &result, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    // Print the result for the root process (process 0)
    if (this_rank == 0) {
        std::cout << "Locality " << this_rank << " has value " << result
                  << std::endl;
    }

    MPI_Finalize();
    return 0;
}
```

HPX equivalent:

```
std::uint32_t num_localities = hpx::get_num_localities(hpx::launch::sync);
std::uint32_t this_locality = hpx::get_locality_id();

auto reduce_direct_client = create_communicator(reduce_direct_basename,
    num_sites_arg(num_localities), this_site_arg(this_locality));
```

(continues on next page)

(continued from previous page)

```
std::uint32_t value = hpx::get_locality_id();

if (this_locality == 0)
{
    hpx::future<std::uint32_t> overall_result = reduce_here(
        reduce_direct_client, value, std::plus<std::uint32_t>{});

    uint32_t r = overall_result.get();

    std::cout << "Locality " << this_locality << " has value " << r
              << std::endl;
}
else
{
    hpx::future<void> overall_result =
        reduce_there(reduce_direct_client, std::move(value));
    overall_result.get();
}
```

This code will print the following message:

```
Locality 0 has value 1
```

HPX uses two functions to implement the functionality of *MPI_Reduce*: *reduce_here* and *reduce_there*. *reduce_here* is gathering data from all localities to the locality with ID 0 (root locality) and then performs the defined reduction operation. *reduce_there* allows non-root localities to participate in the reduction operation by sending data to the root locality. In more detail:

- *hpx::get_num_localities(hpx::launch::sync)* retrieves the number of localities, while *hpx::get_locality_id()* returns the ID of the current locality.
- The function *create_communicator()* is used to create a communicator called *reduce_direct_client*.
- If the current locality is the root (its ID is equal to 0):
  - The *reduce_here* function initiates a reduction operation with addition (*std::plus*) as the reduction operator. The result is stored in *overall_result*.
  - The *get()* member function of the *overall_result* future is used to wait for the result.
- If the current locality is not the root:
  - The *reduce_there* initiates a remote reduction operation.
  - The *get()* member function of the *overall_result* future is used to wait for the remote reduction operation to complete. This is done to ensure synchronization among localities.

Migrating to C++26 reflection-based actions

Starting with HPX 2.0, C++26 static reflection (P2996) can be used to define distributed actions without the verbose macro boilerplate previously required. This section describes how to migrate existing action definitions to the new reflection-based API when building with `HPX_WITH_CXX26_REFLECTION=ON`.

Build requirements

Reflection-based actions require a C++26-capable compiler with reflection support enabled:

- GCC 16.0.1 trunk or later (`stellargroup/gcc_trunk_build_env:1`) with `-freflection -std=c++26`
- Clang 22 with P2996 support (`stellargroup/clang_p2996_build_env:1`) with `-freflection-latest -stdlib=libc++`

Reflection support is auto-detected by HPX's CMake build system via `hpx_check_for_cxx26_reflection()` in `cmake/HPX_PerformCxxFeatureTests.cmake`. No manual flag is required when using a supported compiler — `HPX_HAVE_CXX26_REFLECTION` is set automatically.

Existing code requires no changes

The most important point: **existing code using** `HPX_PLAIN_ACTION`, `HPX_DEFINE_COMPONENT_ACTION`, and related macros requires zero source changes. When `HPX_WITH_CXX26_REFLECTION=ON`, these macros automatically use the reflection-based implementation internally. No `HPX_REGISTER_ACTION` calls are needed — registration is automatic.

```
// This works unchanged with or without reflection enabled.
// When HPX_WITH_CXX26_REFLECTION=ON, it automatically uses
// reflect_action<^^some_function> internally.
HPX_PLAIN_ACTION(some_function, some_action)
```

Explicit migration: plain actions

For new code or explicit migration, use `HPX_ACTION` or `reflect_action` directly:

```
// Before: three-step macro sequence
HPX_PLAIN_ACTION(app::compute, compute_action)
HPX_REGISTER_ACTION_DECLARATION(compute_action)
HPX_REGISTER_ACTION(compute_action)

// After: single line, no registration required
HPX_ACTION(app::compute, compute_action)

// Or equivalently, using the template directly:
using compute_action = hpx::actions::reflect_action<^^app::compute>;
```

Explicit migration: component actions

```
// Before: three-step macro sequence
HPX_DEFINE_COMPONENT_ACTION(server, compute, compute_action)
HPX_REGISTER_ACTION_DECLARATION(compute_action)
HPX_REGISTER_ACTION(compute_action)

// After: single line
HPX_COMPONENT_ACTION(server, compute, compute_action)

// Or equivalently:
using compute_action =
    hpx::actions::reflect_component_action<^^server::compute>;
```

Explicit migration: direct plain actions

```
// Before: three-step macro sequence
HPX_PLAIN_DIRECT_ACTION(app::compute, compute_action)
HPX_REGISTER_ACTION_DECLARATION(compute_action)
HPX_REGISTER_ACTION(compute_action)

// After: single line
HPX_DIRECT_ACTION(app::compute, compute_action)

// Or equivalently, using the template directly:
using compute_action =
    hpx::actions::reflect_direct_action<^^app::compute>;
```

Explicit migration: direct component actions

Direct actions execute without spawning a new HPX thread — locally they run inline on the calling thread, and remotely on the target locality's existing HPX thread. The reflection-based API preserves this behavior:

```
// Before: three-step macro sequence
HPX_DEFINE_COMPONENT_DIRECT_ACTION(server, compute, compute_action)
HPX_REGISTER_ACTION_DECLARATION(compute_action)
HPX_REGISTER_ACTION(compute_action)

// After: single line
HPX_COMPONENT_DIRECT_ACTION(server, compute, compute_action)

// Or equivalently, using the template directly:
using compute_action =
    hpx::actions::reflect_component_direct_action<^^server::compute>;
```

Overloaded functions

Reflection cannot disambiguate overloaded functions. If your action function is overloaded, keep the existing HPX_PLAIN_ACTION macro:

```
// Not supported with reflection (ambiguous):
// using my_action = hpx::actions::reflect_action<^^overloaded_func>;

// Use the existing macro instead:
HPX_PLAIN_ACTION(static_cast<int(*)(int)>(overloaded_func), my_action)
```

Properties available at compile time

Reflection-based action types expose additional compile-time properties:

```
using my_action = hpx::actions::reflect_action<^^app::compute>;

// Number of parameters
static_assert(my_action::arity == 2);

// Function pointer type and value
using fptr = my_action::func_ptr_type;
constexpr auto fp = my_action::func_ptr;
```

2.4.7 Building tests and examples

Tests

To build the tests:

```
$ cmake --build . --target tests
```

To control which tests to run use `ctest`:

- To run single tests, for example a test for `for_loop`:

```
$ ctest --output-on-failure -R tests.unit.modules.algorithms.algorithms.for_loop
```

- To run a whole group of tests:

```
$ ctest --output-on-failure -R tests.unit
```

Examples

- To build (and install) all examples invoke:

```
$ cmake -DHPX_WITH_EXAMPLES=On .
$ make examples
$ make install
```

- To build the hello_world_1 example run:

```
$ make hello_world_1
```

HPX executables end up in the bin directory in your build directory. You can now run hello_world_1 and should see the following output:

```
$ ./bin/hello_world_1
Hello World!
```

You've just run an example which prints Hello World! from the *HPX* runtime. The source for the example is in examples/quickstart/hello_world_1.cpp. The hello_world_distributed example (also available in the examples/quickstart directory) is a distributed hello world program, which is described in *Remote execution with actions*. It provides a gentle introduction to the distributed aspects of *HPX*.

Tip

Most build targets in *HPX* have two names: a simple name and a hierarchical name corresponding to what type of example or test the target is. If you are developing *HPX* it is often helpful to run make help to get a list of available targets. For example, make help | grep hello_world outputs the following:

```
... examples.quickstart.hello_world_2
... hello_world_2
... examples.quickstart.hello_world_1
... hello_world_1
... examples.quickstart.hello_world_distributed
... hello_world_distributed
```

It is also possible to build, for instance, all quickstart examples using make examples.quickstart.

2.4.8 Creating *HPX* projects

Using HPX with pkg-config

How to build *HPX* applications with pkg-config

After you are done installing *HPX*, you should be able to build the following program. It prints Hello World! on the *locality* you run it on.

```
// Including 'hpx/hpx_main.hpp' instead of the usual 'hpx/hpx_init.hpp' enables
// to use the plain C-main below as the direct main HPX entry point.
#include <hpx/hpx_main.hpp>
#include <hpx/iostream.hpp>
```

(continues on next page)

(continued from previous page)

```
int main()
{
    // Say hello to the world!
    hpx::cout << "Hello World!\n" << std::flush;
    return 0;
}
```

Copy the text of this program into a file called hello_world.cpp.

Now, in the directory where you put hello_world.cpp, issue the following commands (where `$HPX_LOCATION` is the build directory or `CMAKE_INSTALL_PREFIX` you used while building *HPX*):

```
$ export PKG_CONFIG_PATH=$PKG_CONFIG_PATH:$HPX_LOCATION/lib/pkgconfig
$ c++ -o hello_world hello_world.cpp \
   `pkg-config --cflags --libs hpx_application`\
    -lhpx_iostreams -DHPX_APPLICATION_NAME=hello_world
```

Important

When using pkg-config with *HPX*, the pkg-config flags must go after the `-o` flag.

Note

HPX libraries have different names in debug and release mode. If you want to link against a debug *HPX* library, you need to use the `_debug` suffix for the pkg-config name. That means instead of `hpx_application` or `hpx_component`, you will have to use `hpx_application_debug` or `hpx_component_debug` Moreover, all referenced *HPX* components need to have an appended `d` suffix. For example, instead of `-lhpx_iostreams` you will need to specify `-lhpx_iostreamsd`.

Important

If the *HPX* libraries are in a path that is not found by the dynamic linker, you will need to add the path `$HPX_LOCATION/lib` to your linker search path (for example `LD_LIBRARY_PATH` on Linux).

To test the program, type:

```
$ ./hello_world
```

which should print `Hello World!` and exit.

How to build *HPX* components with pkg-config

Let's try a more complex example involving an *HPX* component. An *HPX* component is a class that exposes *HPX* actions. *HPX* components are compiled into dynamically loaded modules called component libraries. Here's the source code:

hello_world_component.cpp

```
#include <hpx/config.hpp>
#if !defined(HPX_COMPUTE_DEVICE_CODE)
#include <hpx/iostream.hpp>
#include "hello_world_component.hpp"

#include <iostream>

namespace examples { namespace server {
    void hello_world::invoke()
    {
        hpx::cout << "Hello HPX World!" << std::endl;
    }
}}    // namespace examples::server

HPX_REGISTER_COMPONENT_MODULE()

typedef hpx::components::component<examples::server::hello_world>
    hello_world_type;

HPX_REGISTER_COMPONENT(hello_world_type, hello_world)

HPX_REGISTER_ACTION(
    examples::server::hello_world::invoke_action, hello_world_invoke_action)
#endif
```

hello_world_component.hpp

```
#pragma once

#include <hpx/config.hpp>
#if !defined(HPX_COMPUTE_DEVICE_CODE)
#include <hpx/hpx.hpp>
#include <hpx/include/actions.hpp>
#include <hpx/include/components.hpp>
#include <hpx/include/lcos.hpp>
#include <hpx/serialization.hpp>

#include <utility>

namespace examples { namespace server {
    struct HPX_COMPONENT_EXPORT hello_world
      : hpx::components::component_base<hello_world>
    {
        void invoke();
        HPX_DEFINE_COMPONENT_ACTION(hello_world, invoke)
    };
```

(continues on next page)

(continued from previous page)

```
}}    // namespace examples::server

HPX_REGISTER_ACTION_DECLARATION(
    examples::server::hello_world::invoke_action, hello_world_invoke_action)

namespace examples {
    struct hello_world
      : hpx::components::client_base<hello_world, server::hello_world>
    {
        typedef hpx::components::client_base<hello_world, server::hello_world>
            base_type;

        hello_world(hpx::future<hpx::id_type>&& f)
          : base_type(std::move(f))
        {
        }

        hello_world(hpx::id_type&& f)
          : base_type(std::move(f))
        {
        }

        void invoke()
        {
            hpx::async<server::hello_world::invoke_action>(this->get_id())
                .get();
        }
    };
}    // namespace examples

#endif
```

hello_world_client.cpp

```
#include <hpx/config.hpp>
#if defined(HPX_COMPUTE_HOST_CODE)
#include <hpx/wrap_main.hpp>

#include "hello_world_component.hpp"

int main()
{
    {
        // Create a single instance of the component on this locality.
        examples::hello_world client =
            hpx::new_<examples::hello_world>(hpx::find_here());

        // Invoke the component's action, which will print "Hello World!".
        client.invoke();
    }

    return 0;
```

(continues on next page)

(continued from previous page)

```
}
#endif
```

Copy the three source files above into three files (called `hello_world_component.cpp`, `hello_world_component.hpp` and `hello_world_client.cpp`, respectively).

Now, in the directory where you put the files, run the following command to build the component library. (where `$HPX_LOCATION` is the build directory or `CMAKE_INSTALL_PREFIX` you used while building *HPX*):

```
$ export PKG_CONFIG_PATH=$PKG_CONFIG_PATH:$HPX_LOCATION/lib/pkgconfig
$ c++ -o libhpx_hello_world.so hello_world_component.cpp \
   `pkg-config --cflags --libs hpx_component` \
    -lhpx_iostreams -DHPX_COMPONENT_NAME=hpx_hello_world
```

Now pick a directory in which to install your *HPX* component libraries. For this example, we'll choose a directory named `my_hpx_libs`:

```
$ mkdir ~/my_hpx_libs
$ mv libhpx_hello_world.so ~/my_hpx_libs
```

Note

HPX libraries have different names in debug and release mode. If you want to link against a debug *HPX* library, you need to use the `_debug` suffix for the pkg-config name. That means instead of `hpx_application` or `hpx_component` you will have to use `hpx_application_debug` or `hpx_component_debug`. Moreover, all referenced *HPX* components need to have a appended `d` suffix, e.g. instead of `-lhpx_iostreams` you will need to specify `-lhpx_iostreamsd`.

Important

If the *HPX* libraries are in a path that is not found by the dynamic linker. You need to add the path `$HPX_LOCATION/lib` to your linker search path (for example `LD_LIBRARY_PATH` on Linux).

Now, to build the application that uses this component (`hello_world_client.cpp`), we do:

```
$ export PKG_CONFIG_PATH=$PKG_CONFIG_PATH:$HPX_LOCATION/lib/pkgconfig
$ c++ -o hello_world_client hello_world_client.cpp \
   ``pkg-config --cflags --libs hpx_application``\
    -L${HOME}/my_hpx_libs -lhpx_hello_world -lhpx_iostreams
```

Important

When using pkg-config with *HPX*, the pkg-config flags must go after the `-o` flag.

Finally, you'll need to set your LD_LIBRARY_PATH before you can run the program. To run the program, type:

```
$ export LD_LIBRARY_PATH="$LD_LIBRARY_PATH:$HOME/my_hpx_libs"
$ ./hello_world_client
```

which should print `Hello HPX World!` and exit.

Using *HPX* with CMake-based projects

In addition to the pkg-config support discussed on the previous pages, *HPX* comes with full CMake support. In order to integrate *HPX* into existing or new CMakeLists.txt, you can leverage the find_package[66] command integrated into CMake. Following, is a Hello World component example using CMake.

Let's revisit what we have. We have three files that compose our example application:

- `hello_world_component.hpp`
- `hello_world_component.cpp`
- `hello_world_client.hpp`

The basic structure to include *HPX* into your CMakeLists.txt is shown here:

```
# Require a recent version of cmake
cmake_minimum_required(VERSION 3.18 FATAL_ERROR)

# This project is C++ based.
project(your_app CXX)

# Instruct cmake to find the HPX settings
find_package(HPX)
```

In order to have CMake find *HPX*, it needs to be told where to look for the `HPXConfig.cmake` file that is generated when *HPX* is built or installed. It is used by `find_package(HPX)` to set up all the necessary macros needed to use *HPX* in your project. The ways to achieve this are:

- Set the `HPX_DIR` CMake variable to point to the directory containing the `HPXConfig.cmake` script on the command line when you invoke CMake:

  ```
  $ cmake -DHPX_DIR=$HPX_LOCATION/lib/cmake/HPX ...
  ```

  where `$HPX_LOCATION` is the build directory or `CMAKE_INSTALL_PREFIX` you used when building/configuring *HPX*.

- Set the `CMAKE_PREFIX_PATH` variable to the root directory of your *HPX* build or install location on the command line when you invoke CMake:

  ```
  $ cmake -DCMAKE_PREFIX_PATH=$HPX_LOCATION ...
  ```

  The difference between `CMAKE_PREFIX_PATH` and `HPX_DIR` is that CMake will add common postfixes, such as `lib/cmake/<project`, to the `CMAKE_PREFIX_PATH` and search in these locations too. Note that if your project uses *HPX* as well as other CMake-managed projects, the paths to the locations of these multiple projects may be concatenated in the `CMAKE_PREFIX_PATH`.

- The variables above may be set in the CMake GUI or curses ccmake interface instead of the command line.

Additionally, if you wish to require *HPX* for your project, replace the `find_package(HPX)` line with `find_package(HPX REQUIRED)`.

You can check if *HPX* was successfully found with the `HPX_FOUND` CMake variable.

66 https://www.cmake.org/cmake/help/latest/command/find_package.html

Using CMake targets

The recommended way of setting up your targets to use *HPX* is to link to the `HPX::hpx` CMake[67] target:

```
target_link_libraries(hello_world_component PUBLIC HPX::hpx)
```

This requires that you have already created the target like this:

```
add_library(hello_world_component SHARED hello_world_component.cpp)
target_include_directories(hello_world_component PUBLIC ${CMAKE_CURRENT_SOURCE_DIR})
```

When you link your library to the `HPX::hpx` CMake[68] target, you will be able use *HPX* functionality in your library. To use `main()` as the implicit entry point in your application you must additionally link your application to the CMake target `HPX::wrap_main`. This target is automatically linked to executables if you are using the macros described below (*Using macros to create new targets*). See *Re-use the main() function as the main HPX entry point* for more information on implicitly using `main()` as the entry point. If you want the same wrapping behavior without including hpx/hpx_main.hpp[69], link to the `HPX::auto_wrap_main` target instead. This enables the runtime initialization around `main()` unconditionally and is useful for codebases where adding the header to `main.cpp` is impractical

Note

The use of `HPX::auto_wrap_main` is not supported when using the native Windows MSVC toolchain.

If you want to use the facilities exposed by `hpx::runtime_manager` in binaries that were not linked as executables (e.g., in shared libraries), you will need make your cmake target explicitly depend on the `HPX::init` target:

```
add_library(hello_world_component SHARED hello_world_component.cpp)
target_link_libraries(hello_world_component PRIVATE HPX::init)
```

Otherwise you may see compilation errors complaining about the header file `hpx/runtime_manager.hpp` not being found.

Creating a component requires setting two additional compile definitions:

```
target_compile_options(hello_world_component
  HPX_COMPONENT_NAME=hello_world
  HPX_COMPONENT_EXPORTS)
```

Instead of setting these definitions manually you may link to the `HPX::component` target, which sets `HPX_COMPONENT_NAME` to `hpx_<target_name>`, where `<target_name>` is the target name of your library. Note that these definitions should be `PRIVATE` to make sure these definitions are not propagated transitively to dependent targets.

In addition to making your library a component you can make it a plugin. To do so link to the `HPX::plugin` target. Similarly to `HPX::component` this will set `HPX_PLUGIN_NAME` to `hpx_<target_name>`. This definition should also be `PRIVATE`. Unlike regular shared libraries, plugins are loaded at runtime from certain directories and will not be found without additional configuration. Plugins should be installed into a directory containing only plugins. For example, the plugins created by *HPX* itself are installed into the `hpx` subdirectory in the library install directory (typically `lib` or `lib64`). When using the `HPX::plugin` target you need to install your plugins into an appropriate directory. You may also want to set the location of your plugin in the build directory with the `*_OUTPUT_DIRECTORY*` CMake target properties to be able to load the plugins in the build directory. Once you've set the install or output directory of your plugin

[67] https://www.cmake.org
[68] https://www.cmake.org
[69] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/wrap/include/hpx/hpx_main.hpp

you need to tell your executable where to find it at runtime. You can do this either by setting the environment variable HPX_COMPONENT_PATHS or the ini setting hpx.component_paths (see *--hpx:ini*) to the directory containing your plugin.

Using macros to create new targets

In addition to the targets described above, *HPX* provides convenience macros to hide optional boilerplate code that may be useful for your project. The link to the targets described above. We recommend that you use the targets directly whenever possible as they tend to compose better with other targets.

The macro for adding an *HPX* component is add_hpx_component. It can be used in your CMakeLists.txt file like this:

```
# build your application using HPX
add_hpx_component(hello_world
    SOURCES hello_world_component.cpp
    HEADERS hello_world_component.hpp
    COMPONENT_DEPENDENCIES iostreams)
```

Note

add_hpx_component adds a _component suffix to the target name. In the example above, a hello_world_component target will be created.

The available options to add_hpx_component are:

- SOURCES: The source files for that component
- HEADERS: The header files for that component
- DEPENDENCIES: Other libraries or targets this component depends on
- COMPONENT_DEPENDENCIES: The components this component depends on
- PLUGIN: Treats this component as a plugin-able library
- COMPILE_FLAGS: Additional compiler flags
- LINK_FLAGS: Additional linker flags
- FOLDER: Adds the headers and source files to this Source Group folder
- EXCLUDE_FROM_ALL: Do not build this component as part of the all target

After adding the component, the way you add the executable is as follows:

```
# build your application using HPX
add_hpx_executable(hello_world
    SOURCES hello_world_client.cpp
    COMPONENT_DEPENDENCIES hello_world)
```

Note

add_hpx_executable automatically adds a _component suffix to dependencies specified in COMPONENT_DEPENDENCIES, meaning you can directly use the name given when adding a component using add_hpx_component.

When you configure your application, all you need to do is set the `HPX_DIR` variable to point to the installation of *HPX*.

Note

All library targets built with *HPX* are exported and readily available to be used as arguments to target_link_libraries[70] in your targets. The *HPX* include directories are available with the `HPX_INCLUDE_DIRS` CMake variable.

Using the *HPX* compiler wrapper `hpxcxx`

The `hpxcxx` compiler wrapper helps to compile a *HPX* component, application, or object file, based on the arguments passed to it.

```
$ hpxcxx [--exe=<APPLICATION_NAME> | --comp=<COMPONENT_NAME> | -c] FLAGS FILES
```

The `hpxcxx` command **requires** that either an application or a component is built or `-c` flag is specified. If the build is against a debug build, the `-g` is to be specified while building.

Optional `FLAGS`

- `-l <LIBRARY> | -l<LIBRARY>`: Links `<LIBRARY>` to the build
- `-g`: Specifies that the application or component build is against a debug build
- `-rd`: Sets `release-with-debug-info` option
- `-mr`: Sets `minsize-release` option

All other flags (like `-o OUTPUT_FILE`) are directly passed to the underlying C++ compiler.

Using macros to set up existing targets to use *HPX*

In addition to the `add_hpx_component` and `add_hpx_executable`, you can use the `hpx_setup_target` macro to have an already existing target to be used with the *HPX* libraries:

```
hpx_setup_target(target)
```

Optional parameters are:

- `EXPORT`: Adds it to the CMake export list HPXTargets
- `INSTALL`: Generates an install rule for the target
- `PLUGIN`: Treats this component as a plugin-able library
- `TYPE`: The type can be: EXECUTABLE, LIBRARY or COMPONENT
- `DEPENDENCIES`: Other libraries or targets this component depends on
- `COMPONENT_DEPENDENCIES`: The components this component depends on
- `COMPILE_FLAGS`: Additional compiler flags
- `LINK_FLAGS`: Additional linker flags

[70] https://www.cmake.org/cmake/help/latest/command/target_link_libraries.html

If you do not use CMake, you can still build against *HPX*, but you should refer to the section on *How to build HPX components with pkg-config*.

Note

Since *HPX* relies on dynamic libraries, the dynamic linker needs to know where to look for them. If *HPX* isn't installed into a path that is configured as a linker search path, external projects need to either set RPATH or adapt LD_LIBRARY_PATH to point to where the *HPX* libraries reside. In order to set RPATHs, you can include HPX_SetFullRPATH in your project after all libraries you want to link against have been added. Please also consult the CMake documentation here[71].

Using *HPX* with Makefile

A basic project building with *HPX* is through creating makefiles. The process of creating one can get complex depending upon the use of cmake parameter HPX_WITH_HPX_MAIN (which defaults to ON).

How to build *HPX* applications with makefile

If *HPX* is installed correctly, you should be able to build and run a simple Hello World program. It prints Hello World! on the *locality* you run it on.

```
// Including 'hpx/hpx_main.hpp' instead of the usual 'hpx/hpx_init.hpp' enables
// to use the plain C-main below as the direct main HPX entry point.
#include <hpx/hpx_main.hpp>
#include <hpx/iostream.hpp>

int main()
{
    // Say hello to the world!
    hpx::cout << "Hello World!\n" << std::flush;
    return 0;
}
```

Copy the content of this program into a file called hello_world.cpp.

Now, in the directory where you put hello_world.cpp, create a Makefile. Add the following code:

```
CXX=(CXX)  # Add your favourite compiler here or let makefile choose default.

CXXFLAGS=-O3 -std=c++17

Boost_ROOT=/path/to/boost
Hwloc_ROOT=/path/to/hwloc
Tcmalloc_ROOT=/path/to/tcmalloc
HPX_ROOT=/path/to/hpx

INCLUDE_DIRECTIVES=$(HPX_ROOT)/include $(Boost_ROOT)/include $(Hwloc_ROOT)/include

LIBRARY_DIRECTIVES=-L$(HPX_ROOT)/lib $(HPX_ROOT)/lib/libhpx_init.a $(HPX_ROOT)/lib/
↪libhpx.so $(Boost_ROOT)/lib/libboost_atomic-mt.so $(Boost_ROOT)/lib/libboost_
```

(continues on next page)

[71] https://gitlab.kitware.com/cmake/community/wikis/doc/cmake/RPATH-handling

(continued from previous page)

```
↪filesystem-mt.so $(Boost_ROOT)/lib/libboost_program_options-mt.so $(Boost_ROOT)/lib/
↪libboost_regex-mt.so $(Boost_ROOT)/lib/libboost_system-mt.so -lpthread $(Tcmalloc_
↪ROOT)/libtcmalloc_minimal.so $(Hwloc_ROOT)/libhwloc.so -ldl -lrt

LINK_FLAGS=$(HPX_ROOT)/lib/libhpx_wrap.a -Wl,-wrap=main  # should be left empty for HPX_
↪WITH_HPX_MAIN=OFF

hello_world: hello_world.o
   $(CXX) $(CXXFLAGS) -o hello_world hello_world.o $(LIBRARY_DIRECTIVES) $(LINK_FLAGS)

hello_world.o:
   $(CXX) $(CXXFLAGS) -c -o hello_world.o hello_world.cpp $(INCLUDE_DIRECTIVES)
```

Important

`LINK_FLAGS` should be left empty if HPX_WITH_HPX_MAIN is set to OFF. Boost in the above example is build with `--layout=tagged`. Actual Boost flags may vary on your build of Boost.

To build the program, type:

```
$ make
```

A successful build should result in hello_world binary. To test, type:

```
$ ./hello_world
```

How to build *HPX* components with makefile

Let's try a more complex example involving an *HPX* component. An *HPX* component is a class that exposes *HPX* actions. *HPX* components are compiled into dynamically-loaded modules called component libraries. Here's the source code:

hello_world_component.cpp

```
#include <hpx/config.hpp>
#if !defined(HPX_COMPUTE_DEVICE_CODE)
#include <hpx/iostream.hpp>
#include "hello_world_component.hpp"

#include <iostream>

namespace examples { namespace server {
    void hello_world::invoke()
    {
        hpx::cout << "Hello HPX World!" << std::endl;
    }
}}    // namespace examples::server

HPX_REGISTER_COMPONENT_MODULE()

```

(continues on next page)

(continued from previous page)

```
typedef hpx::components::component<examples::server::hello_world>
    hello_world_type;

HPX_REGISTER_COMPONENT(hello_world_type, hello_world)

HPX_REGISTER_ACTION(
    examples::server::hello_world::invoke_action, hello_world_invoke_action)
#endif
```

hello_world_component.hpp

```
#pragma once

#include <hpx/config.hpp>
#if !defined(HPX_COMPUTE_DEVICE_CODE)
#include <hpx/hpx.hpp>
#include <hpx/include/actions.hpp>
#include <hpx/include/components.hpp>
#include <hpx/include/lcos.hpp>
#include <hpx/serialization.hpp>

#include <utility>

namespace examples { namespace server {
    struct HPX_COMPONENT_EXPORT hello_world
      : hpx::components::component_base<hello_world>
    {
        void invoke();
        HPX_DEFINE_COMPONENT_ACTION(hello_world, invoke)
    };
}}    // namespace examples::server

HPX_REGISTER_ACTION_DECLARATION(
    examples::server::hello_world::invoke_action, hello_world_invoke_action)

namespace examples {
    struct hello_world
      : hpx::components::client_base<hello_world, server::hello_world>
    {
        typedef hpx::components::client_base<hello_world, server::hello_world>
            base_type;

        hello_world(hpx::future<hpx::id_type>&& f)
          : base_type(std::move(f))
        {
        }

        hello_world(hpx::id_type&& f)
          : base_type(std::move(f))
        {
        }

```

(continues on next page)

(continued from previous page)

```
        void invoke()
        {
            hpx::async<server::hello_world::invoke_action>(this->get_id())
                .get();
        }
    };
}    // namespace examples

#endif
```

hello_world_client.cpp

```
#include <hpx/config.hpp>
#if defined(HPX_COMPUTE_HOST_CODE)
#include <hpx/wrap_main.hpp>

#include "hello_world_component.hpp"

int main()
{
    {
        // Create a single instance of the component on this locality.
        examples::hello_world client =
            hpx::new_<examples::hello_world>(hpx::find_here());

        // Invoke the component's action, which will print "Hello World!".
        client.invoke();
    }

    return 0;
}
#endif
```

Now, in the directory, create a Makefile. Add the following code:

```
CXX=(CXX)  # Add your favourite compiler here or let makefile choose default.

CXXFLAGS=-O3 -std=c++17

Boost_ROOT=/path/to/boost
Hwloc_ROOT=/path/to/hwloc
Tcmalloc_ROOT=/path/to/tcmalloc
HPX_ROOT=/path/to/hpx

INCLUDE_DIRECTIVES=$(HPX_ROOT)/include $(Boost_ROOT)/include $(Hwloc_ROOT)/include

LIBRARY_DIRECTIVES=-L$(HPX_ROOT)/lib $(HPX_ROOT)/lib/libhpx_init.a $(HPX_ROOT)/lib/
↪libhpx.so $(Boost_ROOT)/lib/libboost_atomic-mt.so $(Boost_ROOT)/lib/libboost_
↪filesystem-mt.so $(Boost_ROOT)/lib/libboost_program_options-mt.so $(Boost_ROOT)/lib/
↪libboost_regex-mt.so $(Boost_ROOT)/lib/libboost_system-mt.so -lpthread $(Tcmalloc_
↪ROOT)/libtcmalloc_minimal.so $(Hwloc_ROOT)/libhwloc.so -ldl -lrt
```

(continues on next page)

(continued from previous page)

```
LINK_FLAGS=$(HPX_ROOT)/lib/libhpx_wrap.a -Wl,-wrap=main  # should be left empty for HPX_
↪WITH_HPX_MAIN=OFF

hello_world_client: libhpx_hello_world hello_world_client.o
  $(CXX) $(CXXFLAGS) -o hello_world_client $(LIBRARY_DIRECTIVES) libhpx_hello_world
↪$(LINK_FLAGS)

hello_world_client.o: hello_world_client.cpp
  $(CXX) $(CXXFLAGS) -o hello_world_client.o hello_world_client.cpp $(INCLUDE_DIRECTIVES)

libhpx_hello_world: hello_world_component.o
  $(CXX) $(CXXFLAGS) -o libhpx_hello_world hello_world_component.o $(LIBRARY_DIRECTIVES)

hello_world_component.o: hello_world_component.cpp
  $(CXX) $(CXXFLAGS) -c -o hello_world_component.o hello_world_component.cpp $(INCLUDE_
↪DIRECTIVES)
```

To build the program, type:

```
$ make
```

A successful build should result in hello_world binary. To test, type:

```
$ ./hello_world
```

Note

Due to high variations in CMake flags and library dependencies, it is recommended to build *HPX* applications and components with pkg-config or CMakeLists.txt. Writing Makefile may result in broken builds if due care is not taken. pkg-config files and CMake systems are configured with CMake build of *HPX*. Hence, they are stable when used together and provide better support overall.

2.4.9 Starting the *HPX* runtime

In order to write an application that uses services from the *HPX* runtime system, you need to initialize the *HPX* library by inserting certain calls into the code of your application. Depending on your use case, this can be done in 3 different ways:

- *Minimally invasive*: Re-use the `main()` function as the main *HPX* entry point.
- *Balanced use case*: Supply your own main *HPX* entry point while blocking the main thread.
- *Most flexibility*: Supply your own main *HPX* entry point while avoiding blocking the main thread.
- *Suspend and resume*: As above but suspend and resume the *HPX* runtime to allow for other runtimes to be used.

Re-use the `main()` function as the main *HPX* entry point

This method is the least intrusive to your code. However, it provides you with the smallest flexibility in terms of initializing the *HPX* runtime system. The following code snippet shows what a minimal *HPX* application using this technique looks like:

```
#include <hpx/hpx_main.hpp>

int main(int argc, char* argv[])
{
    return 0;
}
```

The only change to your code you have to make is to include the file `hpx/hpx_main.hpp`. In this case the function `main()` will be invoked as the first *HPX* thread of the application. The runtime system will be initialized behind the scenes before the function `main()` is executed and will automatically stop after `main()` has returned. For this method to work you must link your application to the CMake[72] target `HPX::wrap_main`. This is done automatically if you are using the provided macros (*Using macros to create new targets*) to set up your application, but must be done explicitly if you are using targets directly (*Using CMake targets*). All *HPX* API functions can be used from within the `main()` function now. If you cannot or do not want to include `hpx/hpx_main.hpp` in `main.cpp`, you can instead link against `HPX::auto_wrap_main`. That target enables the same runtime startup path without needing the header-triggered opt-in.

Note

The use of `HPX::auto_wrap_main` is not supported when using the native Windows MSVC toolchain.

Note

The function `main()` does not need to expect receiving `argc` and `argv` as shown above, but could expose the signature `int main()`. This is consistent with the usually allowed prototypes for the function `main()` in C++ applications.

All command line arguments specific to *HPX* will still be processed by the *HPX* runtime system as usual. However, those command line options will be removed from the list of values passed to `argc/argv` of the function `main()`. The list of values passed to `main()` will hold only the commandline options that are not recognized by the *HPX* runtime system (see the section *HPX Command Line Options* for more details on what options are recognized by *HPX*).

Note

In this mode all one-letter shortcuts that are normally available on the *HPX* command line are disabled (such as `-t` or `-l` see *HPX Command Line Options*). This is done to minimize any possible interaction between the command line options recognized by the *HPX* runtime system and any command line options defined by the application.

The value returned from the function `main()` as shown above will be returned to the operating system as usual.

Important

[72] https://www.cmake.org

To achieve this seamless integration, the header file hpx/hpx_main.hpp defines a macro:

```
#define main hpx_startup::user_main
```

which could result in unexpected behavior.

Important

To achieve this seamless integration, we use different implementations for different operating systems. In case of Linux or macOS, the code present in hpx_wrap.cpp is put into action. We hook into the system function in case of Linux and provide alternate entry point in case of macOS. For other operating systems we rely on a macro:

```
#define main hpx_startup::user_main
```

provided in the header file hpx/hpx_main.hpp. This implementation can result in unexpected behavior.

Caution

We make use of an *override* variable include_libhpx_wrap in the header file hpx/hpx_main.hpp to swiftly choose the function call stack at runtime. Therefore, the header file should *only* be included in the main executable. Including it in the components will result in multiple definition of the variable.

Supply your own main *HPX* entry point while blocking the main thread

With this method you need to provide an explicit main-thread function named hpx_main at global scope. This function will be invoked as the main entry point of your *HPX* application on the console *locality* only (this function will be invoked as the first *HPX* thread of your application). All *HPX* API functions can be used from within this function.

The thread executing the function *hpx::init* will block waiting for the runtime system to exit. The value returned from hpx_main will be returned from *hpx::init* after the runtime system has stopped.

The function *hpx::finalize* has to be called on one of the *HPX* localities in order to signal that all work has been scheduled and the runtime system should be stopped after the scheduled work has been executed.

This method of invoking *HPX* has the advantage of the user being able to decide which version of *hpx::init* to call. This allows to pass additional configuration parameters while initializing the *HPX* runtime system.

```
#include <hpx/hpx_init.hpp>

int hpx_main(int argc, char* argv[])
{
    // Any HPX application logic goes here...
    return hpx::finalize();
}

int main(int argc, char* argv[])
{
    // Initialize HPX, run hpx_main as the first HPX thread, and
    // wait for hpx::finalize being called.
    return hpx::init(argc, argv);
}
```

Note

The function `hpx_main` does not need to expect receiving `argc/argv` as shown above, but could expose one of the following signatures:

```
int hpx_main();
int hpx_main(int argc, char* argv[]);
int hpx_main(hpx::program_options::variables_map& vm);
```

This is consistent with (and extends) the usually allowed prototypes for the function `main()` in C++ applications.

The header file to include for this method of using *HPX* is `hpx/hpx_init.hpp`.

There are many additional overloads of *hpx::init* available, such as the ability to provide your own entry-point function instead of `hpx_main`. Please refer to the function documentation for more details (see: `hpx/hpx_init.hpp`).

Supply your own main *HPX* entry point while avoiding blocking the main thread

With this method you need to provide an explicit main thread function named `hpx_main` at global scope. This function will be invoked as the main entry point of your *HPX* application on the console *locality* only (this function will be invoked as the first *HPX* thread of your application). All *HPX* API functions can be used from within this function.

The thread executing the function *hpx::start* will *not* block waiting for the runtime system to exit, but will return immediately. The function *hpx::finalize* has to be called on one of the *HPX* localities in order to signal that all work has been scheduled and the runtime system should be stopped after the scheduled work has been executed.

This method of invoking *HPX* is useful for applications where the main thread is used for special operations, such a GUIs. The function `hpx::stop` can be used to wait for the *HPX* runtime system to exit and should at least be used as the last function called in `main()`. The value returned from `hpx_main` will be returned from `hpx::stop` after the runtime system has stopped.

```
#include <hpx/hpx_start.hpp>

int hpx_main(int argc, char* argv[])
{
    // Any HPX application logic goes here...
    return hpx::finalize();
}

int main(int argc, char* argv[])
{
    // Initialize HPX, run hpx_main.
    hpx::start(argc, argv);

    // ...Execute other code here...

    // Wait for hpx::finalize being called.
    return hpx::stop();
}
```

Note

The function `hpx_main` does not need to expect receiving `argc/argv` as shown above, but could expose one of the following signatures:

```
int hpx_main();
int hpx_main(int argc, char* argv[]);
int hpx_main(hpx::program_options::variables_map& vm);
```

This is consistent with (and extends) the usually allowed prototypes for the function `main()` in C++ applications.

The header file to include for this method of using *HPX* is `hpx/hpx_start.hpp`.

There are many additional overloads of *hpx::start* available, such as the option for users to provide their own entry point function instead of `hpx_main`. Please refer to the function documentation for more details (see: `hpx/hpx_start.hpp`).

Supply your own explicit startup function as the main *HPX* entry point

There is also a way to specify any function (besides `hpx_main`) to be used as the main entry point for your *HPX* application:

```
#include <hpx/hpx_init.hpp>

int application_entry_point(int argc, char* argv[])
{
    // Any HPX application logic goes here...
    return hpx::finalize();
}

int main(int argc, char* argv[])
{
    // Initialize HPX, run application_entry_point as the first HPX thread,
    // and wait for hpx::finalize being called.
    return hpx::init(&application_entry_point, argc, argv);
}
```

Note

The function supplied to *hpx::init* must have one of the following prototypes:

int application_entry_point(int argc, char* argv[]); int application_entry_point(hpx::program_options::variables_map& vm);

Note

If `nullptr` is used as the function argument, *HPX* will not run any startup function on this locality.

Suspending and resuming the *HPX* runtime

In some applications it is required to combine *HPX* with other runtimes. To support this use case, *HPX* provides two functions: *hpx::suspend* and *hpx::resume*. *hpx::suspend* is a blocking call which will wait for all scheduled tasks to finish executing and then put the thread pool OS threads to sleep. *hpx::resume* simply wakes up the sleeping threads so that they are ready to accept new work. *hpx::suspend* and *hpx::resume* can be found in the header `hpx/hpx_suspend.hpp`.

```
#include <hpx/hpx_start.hpp>
#include <hpx/hpx_suspend.hpp>

int main(int argc, char* argv[])
{
   // Initialize HPX, don't run hpx_main
    hpx::start(nullptr, argc, argv);

    // Schedule a function on the HPX runtime
    hpx::post(&my_function, ...);

    // Wait for all tasks to finish, and suspend the HPX runtime
    hpx::suspend();

    // Execute non-HPX code here

    // Resume the HPX runtime
    hpx::resume();

    // Schedule more work on the HPX runtime

    // hpx::finalize has to be called from the HPX runtime before hpx::stop
    hpx::post([]() { hpx::finalize(); });
    return hpx::stop();
}
```

Note

hpx::suspend does not wait for *hpx::finalize* to be called. Only call *hpx::finalize* when you wish to fully stop the *HPX* runtime.

Warning

***hpx::suspend* only waits for local tasks, i.e. tasks on the**
current locality, to finish executing. When using *hpx::suspend* in a multi-locality scenario the user is responsible for ensuring that any work required from other localities has also finished.

HPX also supports suspending individual thread pools and threads. For details on how to do that, see the documentation for *`hpx::threads::thread_pool_base`*.

Automatically suspending worker threads

The previous method guarantees that the worker threads are suspended when you ask for it and that they stay suspended. An alternative way to achieve the same effect is to tweak how quickly *HPX* suspends its worker threads when they run out of work. The following configuration values make sure that *HPX* idles very quickly:

```
hpx.max_idle_backoff_time = 1000
hpx.max_idle_loop_count = 0
```

They can be set on the command line using `--hpx:ini=hpx.max_idle_backoff_time=1000` and `--hpx:ini=hpx.max_idle_loop_count=0`. See *Launching and configuring HPX applications* for more details on how to set configuration parameters.

After setting idling parameters the previous example could now be written like this instead:

```cpp
#include <hpx/hpx_start.hpp>

int main(int argc, char* argv[])
{

   // Initialize HPX, don't run hpx_main
    hpx::start(nullptr, argc, argv);

    // Schedule some functions on the HPX runtime
    // NOTE: run_as_hpx_thread blocks until completion.
    hpx::run_as_hpx_thread(&my_function, ...);
    hpx::run_as_hpx_thread(&my_other_function, ...);

    // hpx::finalize has to be called from the HPX runtime before hpx::stop
    hpx::post([]() { hpx::finalize(); });
    return hpx::stop();
}
```

In this example each call to `hpx::run_as_hpx_thread` acts as a "parallel region".

Working of `hpx_main.hpp`

In order to initialize *HPX* from `main()`, we make use of linker tricks.

It is implemented differently for different operating systems. The method of implementation is as follows:

- *Linux*: Using linker `--wrap` option.
- *Mac OSX*: Using the linker `-e` option.
- *Windows*: Using `#define main hpx_startup::user_main`

Linux implementation

We make use of the Linux linker `ld`‘s `--wrap` option to wrap the `main()` function. This way any calls to `main()` are redirected to our own implementation of main. It is here that we check for the existence of `hpx_main.hpp` by making use of a shadow variable `include_libhpx_wrap`. The value of this variable determines the function stack at runtime.

The implementation can be found in `libhpx_wrap.a`.

> **Important**
>
> It is necessary that `hpx_main.hpp` be not included more than once. Multiple inclusions can result in multiple definition of `include_libhpx_wrap`.

Mac OSX implementation

Here we make use of yet another linker option `-e` to change the entry point to our custom entry function `initialize_main`. We initialize the *HPX* runtime system from this function and call main from the initialized system. We determine the function stack at runtime by making use of the shadow variable `include_libhpx_wrap`.

The implementation can be found in `libhpx_wrap.a`.

> **Important**
>
> It is necessary that `hpx_main.hpp` be not included more than once. Multiple inclusions can result in multiple definition of `include_libhpx_wrap`.

Windows implementation

We make use of a macro `#define main hpx_startup::user_main` to take care of the initializations.

This implementation could result in unexpected behaviors.

2.4.10 Launching and configuring *HPX* applications

Configuring *HPX* applications

All *HPX* applications can be configured using special command line options and/or using special configuration files. This section describes the available options, the configuration file format, and the algorithm used to locate possible predefined configuration files. Additionally, this section describes the defaults assumed if no external configuration information is supplied.

During startup any *HPX* application applies a predefined search pattern to locate one or more configuration files. All found files will be read and merged in the sequence they are found into one single internal database holding all configuration properties. This database is used during the execution of the application to configure different aspects of the runtime system.

In addition to the ini files, any application can supply its own configuration files, which will be merged with the configuration database as well. Moreover, the user can specify additional configuration parameters on the command line when executing an application. The *HPX* runtime system will merge all command line configuration options (see the description of the `--hpx:ini`, `--hpx:config`, and `--hpx:app-config` command line options).

The *HPX* ini file format

All *HPX* applications can be configured using a special file format that is similar to the well-known Windows INI file format[73]. This is a structured text format that allows users to group key/value pairs (properties) into sections. The basic element contained in an ini file is the property. Every property has a name and a value, delimited by an equal sign `'='`. The name appears to the left of the equal sign:

```
name=value
```

The value may contain equal signs as only the first `'='` character is interpreted as the delimiter between `name` and `value`. Whitespace before the name, after the value and immediately before and after the delimiting equal sign is ignored. Whitespace inside the value is retained.

Properties may be grouped into arbitrarily named sections. The section name appears on a line by itself, in square brackets. All properties after the section declaration are associated with that section. There is no explicit "end of section" delimiter; sections end at the next section declaration or the end of the file:

```
[section]
```

In *HPX* sections can be nested. A nested section has a name composed of all section names it is embedded in. The section names are concatenated using a dot `'.'`:

```
[outer_section.inner_section]
```

Here, `inner_section` is logically nested within `outer_section`.

It is possible to use the full section name concatenated with the property name to refer to a particular property. For example, in:

```
[a.b.c]
d = e
```

the property value of `d` can be referred to as `a.b.c.d=e`.

In *HPX* ini files can contain comments. Hash signs `'#'` at the beginning of a line indicate a comment. All characters starting with `'#'` until the end of the line are ignored.

If a property with the same name is reused inside a section, the second occurrence of this property name will override the first occurrence (discard the first value). Duplicate sections simply merge their properties together, as if they occurred contiguously.

In *HPX* ini files a property value `${FOO:default}` will use the environmental variable `FOO` to extract the actual value if it is set and `default` otherwise. No default has to be specified. Therefore, `${FOO}` refers to the environmental variable `FOO`. If `FOO` is not set or empty, the overall expression will evaluate to an empty string. A property value `$[section.key:default]` refers to the value held by the property `section.key` if it exists and `default` otherwise. No default has to be specified. Therefore `$[section.key]` refers to the property `section.key`. If the property `section.key` is not set or empty, the overall expression will evaluate to an empty string.

Note

Any property `$[section.key:default]` is evaluated whenever it is queried and not when the configuration data is initialized. This allows for lazy evaluation and relaxes initialization order of different sections. The only exception are recursive property values, e.g., values referring to the very key they are associated with. Those property values are evaluated at initialization time to avoid infinite recursion.

[73] https://en.wikipedia.org/wiki/INI_file

Built-in default configuration settings

During startup any *HPX* application applies a predefined search pattern to locate one or more configuration files. All found files will be read and merged in the sequence they are found into one single internal data structure holding all configuration properties.

As a first step the internal configuration database is filled with a set of default configuration properties. Those settings are described on a section by section basis below.

> **Note**
>
> You can print the default configuration settings used for an executable by specifying the command line option *--hpx:dump-config*.

The `system` configuration section

```
[system]
pid = <process-id>
prefix = <current prefix path of core HPX library>
executable = <current prefix path of executable>
```

Property	Description
`system.pid`	This is initialized to store the current OS-process id of the application instance.
`system.prefix`	This is initialized to the base directory *HPX* has been loaded from.
`system.executable_prefix`	This is initialized to the base directory the current executable has been loaded from.

The *HPX* configuration section

```
[hpx]
location = ${HPX_LOCATION:$[system.prefix]}
component_path = $[hpx.location]/lib/hpx:$[system.executable_prefix]/lib/hpx:$[system.
↪executable_prefix]/../lib/hpx
master_ini_path = $[hpx.location]/share/hpx-<version>:$[system.executable_prefix]/share/
↪hpx-<version>:$[system.executable_prefix]/../share/hpx-<version>
ini_path = $[hpx.master_ini_path]/ini
os_threads = 1
cores = all
localities = 1
program_name =
cmd_line =
lock_detection = ${HPX_LOCK_DETECTION:0}
throw_on_held_lock = ${HPX_THROW_ON_HELD_LOCK:1}
minimal_deadlock_detection = <debug>
spinlock_deadlock_detection = <debug>
spinlock_deadlock_detection_limit = ${HPX_SPINLOCK_DEADLOCK_DETECTION_LIMIT:1000000}
max_background_threads = ${HPX_MAX_BACKGROUND_THREADS:$[hpx.os_threads]}
max_idle_loop_count = ${HPX_MAX_IDLE_LOOP_COUNT:<hpx_idle_loop_count_max>}
```

(continues on next page)

(continued from previous page)

```
max_busy_loop_count = ${HPX_MAX_BUSY_LOOP_COUNT:<hpx_busy_loop_count_max>}
max_idle_backoff_time = ${HPX_MAX_IDLE_BACKOFF_TIME:<hpx_idle_backoff_time_max>}
exception_verbosity = ${HPX_EXCEPTION_VERBOSITY:2}
trace_depth = ${HPX_TRACE_DEPTH:20}
handle_signals = ${HPX_HANDLE_SIGNALS:1}
handle_failed_new = ${HPX_HANDLE_FAILED_NEW:1}

[hpx.stacks]
small_size = ${HPX_SMALL_STACK_SIZE:<hpx_small_stack_size>}
medium_size = ${HPX_MEDIUM_STACK_SIZE:<hpx_medium_stack_size>}
large_size = ${HPX_LARGE_STACK_SIZE:<hpx_large_stack_size>}
huge_size = ${HPX_HUGE_STACK_SIZE:<hpx_huge_stack_size>}
use_guard_pages = ${HPX_THREAD_GUARD_PAGE:1}
```

Property	Description
hpx. location	This is initialized to the id of the *locality* this application instance is running on.
hpx. component_	Duplicates are discarded. This property can refer to a list of directories separated by `':'` (Linux, Android, and MacOS) or by `';'` (Windows).
hpx. master_in	This is initialized to the list of default paths of the main hpx.ini configuration files. This property can refer to a list of directories separated by `':'` (Linux, Android, and MacOS) or using `';'` (Windows).
hpx. ini_path	This is initialized to the default path where *HPX* will look for more ini configuration files. This property can refer to a list of directories separated by `':'` (Linux, Android, and MacOS) or using `';'` (Windows).
hpx. os_thread	This setting reflects the number of OS threads used for running *HPX* threads. Defaults to number of detected cores (not hyperthreads/PUs).
hpx. cores	This setting reflects the number of cores used for running *HPX* threads. Defaults to number of detected cores (not hyperthreads/PUs).
hpx. localitie	This setting reflects the number of localities the application is running on. Defaults to `1`.
hpx. program_n	This setting reflects the program name of the application instance. Initialized from the command line `argv[0]`.
hpx. cmd_line	This setting reflects the actual command line used to launch this application instance.
hpx. lock_dete	This setting verifies that no locks are being held while a *HPX* thread is suspended. This setting is applicable only if `HPX_WITH_VERIFY_LOCKS` is set during configuration in CMake.
hpx. throw_on_l	This setting causes an exception if during lock detection at least one lock is being held while a *HPX* thread is suspended. This setting is applicable only if `HPX_WITH_VERIFY_LOCKS` is set during configuration in CMake. This setting has no effect if `hpx.lock_detection=0`.
hpx. minimal_d	This setting enables support for minimal deadlock detection for *HPX* threads. By default this is set to `1` (for Debug builds) or to `0` (for Release, RelWithDebInfo, RelMinSize builds). This setting is effective only if `HPX_WITH_THREAD_DEADLOCK_DETECTION` is set during configuration in CMake.
hpx. spinlock_	This setting verifies that spinlocks don't spin longer than specified using the `hpx.spinlock_deadlock_detection_limit`. This setting is applicable only if `HPX_WITH_SPINLOCK_DEADLOCK_DETECTION` is set during configuration in CMake. By default this is set to `1` (for Debug builds) or to `0` (for Release, RelWithDebInfo, RelMinSize builds).
hpx. spinlock_	This setting specifies the upper limit of the allowed number of spins that spinlocks are allowed to perform. This setting is applicable only if `HPX_WITH_SPINLOCK_DEADLOCK_DETECTION` is set during configuration in CMake. By default this is set to `1000000`.
hpx. max_backg	This setting defines the number of threads in the scheduler, which are used to execute background work. By default this is the same as the number of cores used for the scheduler.
hpx. max_idle_	By default this is defined by the preprocessor constant `HPX_IDLE_LOOP_COUNT_MAX`. This is an internal setting that you should change only if you know exactly what you are doing.
hpx. max_busy_	This setting defines the maximum value of the busy-loop counter in the scheduler. By default this is defined by the preprocessor constant `HPX_BUSY_LOOP_COUNT_MAX`. This is an internal setting that you should change only if you know exactly what you are doing.
hpx. max_idle_l	This setting defines the maximum time (in milliseconds) for the scheduler to sleep after being idle for `hpx.max_idle_loop_count` iterations. This setting is applicable only if `HPX_WITH_THREAD_MANAGER_IDLE_BACKOFF` is set during configuration in CMake[74]. By default this is defined by the preprocessor constant `HPX_IDLE_BACKOFF_TIME_MAX`. This is an internal setting that you should change only if you know exactly what you are doing.
hpx. exception_	This setting defines the verbosity of exceptions. Valid values are integers. A setting of `2` or higher prints all available information. A setting of `1` leaves out the build configuration and environment variables. A setting of `0` or lower prints only the description of the thrown exception and the file name, function, and line number where the exception was thrown. The default value is `2` or the value of the environment variable `HPX_EXCEPTION_VERBOSITY`.
hpx. trace_dep	This setting defines the number of stack-levels printed in generated stack backtraces. This defaults to `20`, but can be changed using the cmake `HPX_WITH_THREAD_BACKTRACE_DEPTH` configuration setting.
hpx. handle_si	This setting defines whether HPX will register signal handlers that will print the configuration information (stack backtrace, system information, etc.) whenever a signal is raised. The default is `1`. Setting this value to `0` can be useful in cases when generating a core-dump on segmentation faults or similar signals is desired.

The hpx.threadpools configuration section

```
[hpx.threadpools]
io_pool_size = ${HPX_NUM_IO_POOL_SIZE:2}
parcel_pool_size = ${HPX_NUM_PARCEL_POOL_SIZE:2}
timer_pool_size = ${HPX_NUM_TIMER_POOL_SIZE:2}
```

Property	Description
hpx.threadpools.io_pool_size	The value of this property defines the number of OS threads created for the internal I/O thread pool.
hpx.threadpools.parcel_pool_size	The value of this property defines the number of OS threads created for the internal parcel thread pool.
hpx.threadpools.timer_pool_size	The value of this property defines the number of OS threads created for the internal timer thread pool.

The hpx.thread_queue configuration section

Important

These are the setting control internal values used by the thread scheduling queues in the *HPX* scheduler. You should not modify these settings unless you know exactly what you are doing.

```
[hpx.thread_queue]
min_tasks_to_steal_pending = ${HPX_THREAD_QUEUE_MIN_TASKS_TO_STEAL_PENDING:0}
min_tasks_to_steal_staged = ${HPX_THREAD_QUEUE_MIN_TASKS_TO_STEAL_STAGED:0}
min_add_new_count = ${HPX_THREAD_QUEUE_MIN_ADD_NEW_COUNT:10}
max_add_new_count = ${HPX_THREAD_QUEUE_MAX_ADD_NEW_COUNT:10}
max_delete_count = ${HPX_THREAD_QUEUE_MAX_DELETE_COUNT:1000}
```

Property	Description
hpx.thread_queue.min_tasks_to_steal_pe	The value of this property defines the number of pending *HPX* threads that have to be available before neighboring cores are allowed to steal work. The default is to allow stealing always.
hpx.thread_queue.min_tasks_to_steal_st	The value of this property defines the number of staged *HPX* tasks that need to be available before neighboring cores are allowed to steal work. The default is to allow stealing always.
hpx.thread_queue.min_add_new_count	The value of this property defines the minimal number of tasks to be converted into *HPX* threads whenever the thread queues for a core have run empty.
hpx.thread_queue.max_add_new_count	The value of this property defines the maximal number of tasks to be converted into *HPX* threads whenever the thread queues for a core have run empty.
hpx.thread_queue.max_delete_count	The value of this property defines the number of terminated *HPX* threads to discard during each invocation of the corresponding function.

[74] https://www.cmake.org

The hpx.components configuration section

```
[hpx.components]
load_external = ${HPX_LOAD_EXTERNAL_COMPONENTS:1}
```

Property	Description
hpx.components.load_external	This entry defines whether external components will be loaded on this *locality*. This entry is normally set to 1, and usually there is no need to directly change this value. It is automatically set to 0 for a dedicated *AGAS* server *locality*.

Additionally, the section hpx.components will be populated with the information gathered from all found components. The information loaded for each of the components will contain at least the following properties:

```
[hpx.components.<component_instance_name>]
name = <component_name>
path = <full_path_of_the_component_module>
enabled = $[hpx.components.load_external]
```

Property	Description
hpx.components.<component_instar name	This is the name of a component, usually the same as the second argument to the macro used while registering the component with *HPX_REGISTER_COMPONENT*. Set by the component factory.
hpx.components.<component_instar path	This is either the full path file name of the component module or the directory the component module is located in. In this case, the component module name will be derived from the property hpx.components.<component_instance_name>.name. Set by the component factory.
hpx.components.<component_instar enabled	This setting explicitly enables or disables the component. This is an optional property. *HPX* assumes that the component is enabled if it is not defined.

The value for <component_instance_name> is usually the same as for the corresponding name property. However, generally it can be defined to any arbitrary instance name. It is used to distinguish between different ini sections, one for each component.

The hpx.parcel configuration section

```
[hpx.parcel]
address = ${HPX_PARCEL_SERVER_ADDRESS:<hpx_initial_ip_address>}
port = ${HPX_PARCEL_SERVER_PORT:<hpx_initial_ip_port>}
bootstrap = ${HPX_PARCEL_BOOTSTRAP:<hpx_parcel_bootstrap>}
max_connections = ${HPX_PARCEL_MAX_CONNECTIONS:<hpx_parcel_max_connections>}
max_connections_per_locality = ${HPX_PARCEL_MAX_CONNECTIONS_PER_LOCALITY:<hpx_parcel_max_
→connections_per_locality>}
max_message_size = ${HPX_PARCEL_MAX_MESSAGE_SIZE:<hpx_parcel_max_message_size>}
max_outbound_message_size = ${HPX_PARCEL_MAX_OUTBOUND_MESSAGE_SIZE:<hpx_parcel_max_
→outbound_message_size>}
array_optimization = ${HPX_PARCEL_ARRAY_OPTIMIZATION:1}
zero_copy_optimization = ${HPX_PARCEL_ZERO_COPY_OPTIMIZATION:$[hpx.parcel.array_
→optimization]}
```

(continues on next page)

(continued from previous page)

```
zero_copy_receive_optimization = ${HPX_PARCEL_ZERO_COPY_RECEIVE_OPTIMIZATION:$[hpx.
↪parcel.array_optimization]}
async_serialization = ${HPX_PARCEL_ASYNC_SERIALIZATION:1}
message_handlers = ${HPX_PARCEL_MESSAGE_HANDLERS:0}
```

Property	Description
`hpx.parcel.address`	This property defines the default IP address to be used for the *parcel* layer to listen to. This IP address will be used as long as no other values are specified (for instance, using the `--hpx:hpx` command line option). The expected format is any valid IP address or domain name format that can be resolved into an IP address. The default depends on the compile time preprocessor constant `HPX_INITIAL_IP_ADDRESS` (`"127.0.0.1"`).
`hpx.parcel.port`	This property defines the default IP port to be used for the *parcel* layer to listen to. This IP port will be used as long as no other values are specified (for instance using the `--hpx:hpx` command line option). The default depends on the compile time preprocessor constant `HPX_INITIAL_IP_PORT` (`7910`).
`hpx.parcel.bootstrap`	This property defines which parcelport type should be used during application bootstrap. The default depends on the compile time preprocessor constant `HPX_PARCEL_BOOTSTRAP` (`"tcp"`).
`hpx.parcel.max_connect`	This property defines how many network connections between different localities are overall kept alive by each *locality*. The default depends on the compile time preprocessor constant `HPX_PARCEL_MAX_CONNECTIONS` (`512`).
`hpx.parcel.max_connect`	This property defines the maximum number of network connections that one *locality* will open to another *locality*. The default depends on the compile time preprocessor constant `HPX_PARCEL_MAX_CONNECTIONS_PER_LOCALITY` (`4`).
`hpx.parcel.max_message`	This property defines the maximum allowed message size that will be transferrable through the *parcel* layer. The default depends on the compile time preprocessor constant `HPX_PARCEL_MAX_MESSAGE_SIZE` (`1000000000` bytes).
`hpx.parcel.max_outboun`	This property defines the maximum allowed outbound coalesced message size that will be transferrable through the parcel layer. The default depends on the compile time preprocessor constant `HPX_PARCEL_MAX_OUTBOUND_MESSAGE_SIZE` (`1000000` bytes).
`hpx.parcel.array_optim`	This property defines whether this *locality* is allowed to utilize array optimizations during serialization of *parcel* data. The default is 1.
`hpx.parcel.zero_copy_o`	This property defines whether this *locality* is allowed to utilize zero copy optimizations during serialization of *parcel* data. The default is the same value as set for `hpx.parcel.array_optimization`.
`hpx.parcel.zero_copy_r`	This property defines whether this *locality* is allowed to utilize zero copy optimizations on the receiving end during de-serialization of *parcel* data. The default is the same value as set for `hpx.parcel.zero_copy_optimization`.
`hpx.parcel.zero_copy_s`	This property defines the threshold value (in bytes) starting at which the serialization layer will apply zero-copy optimizations for serialized entities. The default value is defined by the preprocessor constant `HPX_ZERO_COPY_SERIALIZATION_THRESHOLD`.
`hpx.parcel.async_seria`	This property defines whether this *locality* is allowed to spawn a new thread for serialization (this is both for encoding and decoding parcels). The default is 1.
`hpx.parcel.message_han`	This property defines whether message handlers are loaded. The default is `0`.
`hpx.parcel.max_backgro`	This property defines how many cores should be used to perform background operations. The default is -1 (all cores).

The following settings relate to the TCP/IP parcelport.

```
[hpx.parcel.tcp]
enable = ${HPX_HAVE_PARCELPORT_TCP:$[hpx.parcel.enabled]}
array_optimization = ${HPX_PARCEL_TCP_ARRAY_OPTIMIZATION:$[hpx.parcel.array_
↪optimization]}
zero_copy_optimization = ${HPX_PARCEL_TCP_ZERO_COPY_OPTIMIZATION:$[hpx.parcel.zero_copy_
↪optimization]}
zero_copy_receive_optimization = ${HPX_PARCEL_TCP_ZERO_COPY_RECEIVE_OPTIMIZATION:$[hpx.
↪parcel.zero_copy_receive_optimization]}
zero_copy_serialization_threshold =  ${HPX_PARCEL_TCP_ZERO_COPY_SERIALIZATION_THRESHOLD:
↪$[hpx.parcel.zero_copy_serialization_threshold]}
async_serialization = ${HPX_PARCEL_TCP_ASYNC_SERIALIZATION:$[hpx.parcel.async_
↪serialization]}
parcel_pool_size = ${HPX_PARCEL_TCP_PARCEL_POOL_SIZE:$[hpx.threadpools.parcel_pool_size]}
max_connections =  ${HPX_PARCEL_TCP_MAX_CONNECTIONS:$[hpx.parcel.max_connections]}
max_connections_per_locality = ${HPX_PARCEL_TCP_MAX_CONNECTIONS_PER_LOCALITY:$[hpx.
↪parcel.max_connections_per_locality]}
max_message_size =  ${HPX_PARCEL_TCP_MAX_MESSAGE_SIZE:$[hpx.parcel.max_message_size]}
max_outbound_message_size =  ${HPX_PARCEL_TCP_MAX_OUTBOUND_MESSAGE_SIZE:$[hpx.parcel.max_
↪outbound_message_size]}
max_background_threads =  ${HPX_PARCEL_TCP_MAX_BACKGROUND_THREADS:$[hpx.parcel.max_
↪background_threads]}
```

Property	Description
hpx.parcel.tcp.enable	Enables the use of the default TCP parcelport. Note that the initial bootstrap of the overall *HPX* application will be performed using the default TCP connections. This parcelport is enabled by default. This will be disabled only if MPI is enabled (see below).
hpx.parcel.tcp.array_optimization	This property defines whether this *locality* is allowed to utilize array optimizations in the TCP/IP parcelport during serialization of parcel data. The default is the same value as set for `hpx.parcel.array_optimization`.
hpx.parcel.tcp.zero_copy_optimiza	This property defines whether this *locality* is allowed to utilize zero copy optimizations during serialization of parcel data. The default is the same value as set for `hpx.parcel.zero_copy_optimization`.
hpx.parcel.tcp.zero_copy_receive_	This property defines whether this *locality* is allowed to utilize zero copy optimizations on the receiving end in the TCP/IP parcelport during de-serialization of *parcel* data. The default is the same value as set for `hpx.parcel.zero_copy_optimization`.
hpx.parcel.tcp.zero_copy_serializ	This property defines the threshold value (in bytes) starting at which the serialization layer will apply zero-copy optimizations for serialized entities. The default is the same value as set for `hpx.parcel.zero_copy_serialization_threshold`.
hpx.parcel.tcp.async_serializatio	This property defines whether this *locality* is allowed to spawn a new thread for serialization in the TCP/IP parcelport (this is both for encoding and decoding parcels). The default is the same value as set for `hpx.parcel.async_serialization`.
hpx.parcel.tcp.parcel_pool_size	The value of this property defines the number of OS threads created for the internal parcel thread pool of the TCP *parcel* port. The default is taken from `hpx.threadpools.parcel_pool_size`.
hpx.parcel.tcp.max_connections	This property defines how many network connections between different localities are overall kept alive by each *locality*. The default is taken from `hpx.parcel.max_connections`.
hpx.parcel.tcp.max_connections_pe	This property defines the maximum number of network connections that one *locality* will open to another *locality*. The default is taken from `hpx.parcel.max_connections_per_locality`.
hpx.parcel.tcp.max_message_size	This property defines the maximum allowed message size that will be transferrable through the *parcel* layer. The default is taken from `hpx.parcel.max_message_size`.
hpx.parcel.tcp.max_outbound_messa	This property defines the maximum allowed outbound coalesced message size that will be transferrable through the *parcel* layer. The default is taken from `hpx.parcel.max_outbound_connections`.
hpx.parcel.tcp.max_background_thr	This property defines how many cores should be used to perform background operations. The default is taken from `hpx.parcel.max_background_threads`.

The following settings relate to the MPI parcelport. These settings take effect only if the compile time constant `HPX_HAVE_PARCELPORT_MPI` is set (the equivalent CMake variable is `HPX_WITH_PARCELPORT_MPI` and has to be set to `ON`).

```
[hpx.parcel.mpi]
enable = ${HPX_HAVE_PARCELPORT_MPI:$[hpx.parcel.enabled]}
env = ${HPX_HAVE_PARCELPORT_MPI_ENV:MV2_COMM_WORLD_RANK,PMI_RANK,OMPI_COMM_WORLD_SIZE,
↪ALPS_APP_PE,PALS_NODEID}
multithreaded = ${HPX_HAVE_PARCELPORT_MPI_MULTITHREADED:1}
rank = <MPI_rank>
processor_name = <MPI_processor_name>
array_optimization = ${HPX_HAVE_PARCEL_MPI_ARRAY_OPTIMIZATION:$[hpx.parcel.array_
↪optimization]}
zero_copy_optimization = ${HPX_HAVE_PARCEL_MPI_ZERO_COPY_OPTIMIZATION:$[hpx.parcel.zero_
↪copy_optimization]}
zero_copy_receive_optimization = ${HPX_HAVE_PARCEL_MPI_ZERO_COPY_RECEIVE_OPTIMIZATION:
↪$[hpx.parcel.zero_copy_receive_optimization]}
```

(continues on next page)

(continued from previous page)

```
zero_copy_serialization_threshold =  ${HPX_PARCEL_MPI_ZERO_COPY_SERIALIZATION_THRESHOLD:
↪$[hpx.parcel.zero_copy_serialization_threshold]}
use_io_pool = ${HPX_HAVE_PARCEL_MPI_USE_IO_POOL:$1}
async_serialization = ${HPX_HAVE_PARCEL_MPI_ASYNC_SERIALIZATION:$[hpx.parcel.async_
↪serialization]}
parcel_pool_size = ${HPX_HAVE_PARCEL_MPI_PARCEL_POOL_SIZE:$[hpx.threadpools.parcel_pool_
↪size]}
max_connections =  ${HPX_HAVE_PARCEL_MPI_MAX_CONNECTIONS:$[hpx.parcel.max_connections]}
max_connections_per_locality = ${HPX_HAVE_PARCEL_MPI_MAX_CONNECTIONS_PER_LOCALITY:$[hpx.
↪parcel.max_connections_per_locality]}
max_message_size =  ${HPX_HAVE_PARCEL_MPI_MAX_MESSAGE_SIZE:$[hpx.parcel.max_message_
↪size]}
max_outbound_message_size =  ${HPX_HAVE_PARCEL_MPI_MAX_OUTBOUND_MESSAGE_SIZE:$[hpx.
↪parcel.max_outbound_message_size]}
max_background_threads =  ${HPX_PARCEL_MPI_MAX_BACKGROUND_THREADS:$[hpx.parcel.max_
↪background_threads]}
```

Property	Description
`hpx.parcel.mpi.enable`	Enables the use of the MPI parcelport. *HPX* tries to detect if the application was started within a parallel MPI environment. If the detection was successful, the MPI parcelport is enabled by default. To explicitly disable the MPI parcelport, set to 0. Note that the initial bootstrap of the overall *HPX* application will be performed using MPI as well.
`hpx.parcel.mpi.env`	This property influences which environment variables (separated by commas) will be analyzed to find out whether the application was invoked by MPI.
`hpx.parcel.mpi.multithreaded`	This property is used to determine what threading mode to use when initializing MPI. If this setting is `0`, *HPX* will initialize MPI with `MPI_THREAD_SINGLE`. If the value is not equal to `0`, *HPX* will initialize MPI with `MPI_THREAD_MULTI`.
`hpx.parcel.mpi.rank`	This property will be initialized to the MPI rank of the *locality*.
`hpx.parcel.mpi.processor_name`	This property will be initialized to the MPI processor name of the *locality*.
`hpx.parcel.mpi.array_optimiza`	This property defines whether this *locality* is allowed to utilize array optimizations in the MPI parcelport during serialization of *parcel* data. The default is the same value as set for `hpx.parcel.array_optimization`.
`hpx.parcel.mpi.zero_copy_opti`	This property defines whether this *locality* is allowed to utilize zero copy optimizations in the MPI parcelport during serialization of parcel data. The default is the same value as set for `hpx.parcel.zero_copy_optimization`.
`hpx.parcel.mpi.zero_copy_rece`	This property defines whether this *locality* is allowed to utilize zero copy optimizations on the receiving end in the MPI parcelport during de-serialization of *parcel* data. The default is the same value as set for `hpx.parcel.zero_copy_optimization`.
`hpx.parcel.mpi.zero_copy_seri`	This property defines the threshold value (in bytes) starting at which the serialization layer will apply zero-copy optimizations for serialized entities. The default is the same value as set for `hpx.parcel.zero_copy_serialization_threshold`.
`hpx.parcel.mpi.use_io_pool`	This property can be set to run the progress thread inside of *HPX* threads instead of a separate thread pool. The default is 1.
`hpx.parcel.mpi.async_serializ`	This property defines whether this *locality* is allowed to spawn a new thread for serialization in the MPI parcelport (this is both for encoding and decoding parcels). The default is the same value as set for `hpx.parcel.async_serialization`.
`hpx.parcel.mpi.parcel_pool_si`	The value of this property defines the number of OS threads created for the internal parcel thread pool of the MPI *parcel* port. The default is taken from `hpx.threadpools.parcel_pool_size`.
`hpx.parcel.mpi.max_connection`	This property defines how many network connections between different localities are overall kept alive by each *locality*. The default is taken from `hpx.parcel.max_connections`.
`hpx.parcel.mpi.max_connection`	This property defines the maximum number of network connections that one *locality* will open to another *locality*. The default is taken from `hpx.parcel.max_connections_per_locality`.
`hpx.parcel.mpi.max_message_si`	This property defines the maximum allowed message size that will be transferrable through the *parcel* layer. The default is taken from `hpx.parcel.max_message_size`.
`hpx.parcel.mpi.max_outbound_m`	This property defines the maximum allowed outbound coalesced message size that will be transferrable through the *parcel* layer. The default is taken from `hpx.parcel.max_outbound_connections`.
`hpx.parcel.mpi.max_background_`	This property defines how many cores should be used to perform background operations. The default is taken from `hpx.parcel.max_background_threads`.

The hpx.agas configuration section

```
[hpx.agas]
address = ${HPX_AGAS_SERVER_ADDRESS:<hpx_initial_ip_address>}
port = ${HPX_AGAS_SERVER_PORT:<hpx_initial_ip_port>}
service_mode = hosted
dedicated_server = 0
max_pending_refcnt_requests = ${HPX_AGAS_MAX_PENDING_REFCNT_REQUESTS:<hpx_initial_agas_max_pending_refcnt_requests>}
use_caching = ${HPX_AGAS_USE_CACHING:1}
use_range_caching = ${HPX_AGAS_USE_RANGE_CACHING:1}
local_cache_size = ${HPX_AGAS_LOCAL_CACHE_SIZE:<hpx_agas_local_cache_size>}
```

Property	Description
`hpx.agas.address`	This property defines the default IP address to be used for the *AGAS* root server. This IP address will be used as long as no other values are specified (for instance, using the `--hpx:agas` command line option). The expected format is any valid IP address or domain name format that can be resolved into an IP address. The default depends on the compile time preprocessor constant `HPX_INITIAL_IP_ADDRESS` (`"127.0.0.1"`).
`hpx.agas.port`	This property defines the default IP port to be used for the *AGAS* root server. This IP port will be used as long as no other values are specified (for instance, using the `--hpx:agas` command line option). The default depends on the compile time preprocessor constant `HPX_INITIAL_IP_PORT` (`7009`).
`hpx.agas.service_m`	This property specifies what type of *AGAS* service is running on this *locality*. Currently, two modes exist. The *locality* that acts as the *AGAS* server runs in `bootstrap` mode. All other localities are in `hosted` mode.
`hpx.agas.dedicated_`	This property specifies whether the *AGAS* server is exclusively running *AGAS* services and not hosting any application components. It is a boolean value. Set to 1 if `--hpx:run-agas-server-only` is present.
`hpx.agas.max_pendi`	This property defines the number of reference counting requests (increments or decrements) to buffer. The default depends on the compile time preprocessor constant `HPX_INITIAL_AGAS_MAX_PENDING_REFCNT_REQUESTS` (`4096`).
`hpx.agas.use_cachi`	This property specifies whether a software address translation cache is used. It is a boolean value. Defaults to 1.
`hpx.agas.use_range_`	This property specifies whether range-based caching is used by the software address translation cache. This property is ignored if *hpx.agas.use_caching* is false. It is a boolean value. Defaults to 1.
`hpx.agas.local_cac`	This property defines the size of the software address translation cache for *AGAS* services. This property is ignored if `hpx.agas.use_caching` is false. Note that if `hpx.agas.use_range_caching` is true, this size will refer to the maximum number of ranges stored in the cache, not the number of entries spanned by the cache. The default depends on the compile time preprocessor constant `HPX_AGAS_LOCAL_CACHE_SIZE` (`4096`).

The `hpx.commandline` configuration section

The following table lists the definition of all pre-defined command line option shortcuts. For more information about commandline options, see the section *HPX Command Line Options*.

```
[hpx.commandline]
aliasing = ${HPX_COMMANDLINE_ALIASING:1}
allow_unknown = ${HPX_COMMANDLINE_ALLOW_UNKNOWN:0}

[hpx.commandline.aliases]
-a = --hpx:agas
-c = --hpx:console
-h = --hpx:help
-I = --hpx:ini
-l = --hpx:localities
-p = --hpx:app-config
-q = --hpx:queuing
-r = --hpx:run-agas-server
-t = --hpx:threads
-v = --hpx:version
-w = --hpx:worker
-x = --hpx:hpx
-0 = --hpx:node=0
-1 = --hpx:node=1
-2 = --hpx:node=2
-3 = --hpx:node=3
-4 = --hpx:node=4
-5 = --hpx:node=5
-6 = --hpx:node=6
-7 = --hpx:node=7
-8 = --hpx:node=8
-9 = --hpx:node=9
```

Note

The short options listed above are disabled by default if the application is built using `#include <hpx/hpx_main.hpp>`. See *Re-use the main() function as the main HPX entry point* for more information. The rationale behind this is that in this case the user's application may handle its own command line options, since *HPX* passes all unknown options to `main()`. Short options like `-t` are prone to create ambiguities regarding what the application will support. Hence, the user should instead rely on the corresponding long options like `--hpx:threads` in such a case.

Property	Description
hpx.commandline.aliasing	Enable command line aliases as defined in the section hpx.commandline.aliases (see below). Defaults to 1.
hpx.commandline.allow_unknown	Allow for unknown command line options to be passed through to hpx_main() Defaults to 0.
hpx.commandline.aliases.-a	On the commandline -a expands to: *--hpx:agas*.
hpx.commandline.aliases.-c	On the commandline -c expands to: *--hpx:console*.
hpx.commandline.aliases.-h	On the commandline -h expands to: *--hpx:help*.
hpx.commandline.aliases.--help	On the commandline --help expands to: *--hpx:help*.
hpx.commandline.aliases.-I	On the commandline -I expands to: *--hpx:ini*.
hpx.commandline.aliases.-l	On the commandline -l expands to: *--hpx:localities*.
hpx.commandline.aliases.-p	On the commandline -p expands to: *--hpx:app-config*.
hpx.commandline.aliases.-q	On the commandline -q expands to: *--hpx:queuing*.
hpx.commandline.aliases.-r	On the commandline -r expands to: *--hpx:run-agas-server*.
hpx.commandline.aliases.-t	On the commandline -t expands to: *--hpx:threads*.
hpx.commandline.aliases.-v	On the commandline -v expands to: *--hpx:version*.
hpx.commandline.aliases.--version	On the commandline --version expands to: *--hpx:version*.
hpx.commandline.aliases.-w	On the commandline -w expands to: *--hpx:worker*.
hpx.commandline.aliases.-x	On the commandline -x expands to: *--hpx:hpx*.
hpx.commandline.aliases.-0	On the commandline -0 expands to: *--hpx:node*=0.
hpx.commandline.aliases.-1	On the commandline -1 expands to: *--hpx:node*=1.
hpx.commandline.aliases.-2	On the commandline -2 expands to: *--hpx:node*=2.
hpx.commandline.aliases.-3	On the commandline -3 expands to: *--hpx:node*=3.
hpx.commandline.aliases.-4	On the commandline -4 expands to: *--hpx:node*=4.
hpx.commandline.aliases.-5	On the commandline -5 expands to: *--hpx:node*=5.
hpx.commandline.aliases.-6	On the commandline -6 expands to: *--hpx:node*=6.
hpx.commandline.aliases.-7	On the commandline -7 expands to: *--hpx:node*=7.
hpx.commandline.aliases.-8	On the commandline -8 expands to: *--hpx:node*=8.
hpx.commandline.aliases.-9	On the commandline -9 expands to: *--hpx:node*=9.

Loading INI files

During startup and after the internal database has been initialized as described in the section *Built-in default configuration settings*, *HPX* will try to locate and load additional ini files to be used as a source for configuration properties. This allows for a wide spectrum of additional customization possibilities by the user and system administrators. The sequence of locations where *HPX* will try loading the ini files is well defined and documented in this section. All ini files found are merged into the internal configuration database. The merge operation itself conforms to the rules as described in the section *The HPX ini file format*.

1. Load all component shared libraries found in the directories specified by the property `hpx.component_path` and retrieve their default configuration information (see section *Loading components* for more details). This property can refer to a list of directories separated by `':'` (Linux, Android, and MacOS) or by `';'` (Windows).
2. Load all files named `hpx.ini` in the directories referenced by the property `hpx.master_ini_path` This property can refer to a list of directories separated by `':'` (Linux, Android, and MacOS) or by `';'` (Windows).
3. Load a file named `.hpx.ini` in the current working directory, e.g., the directory the application was invoked from.
4. Load a file referenced by the environment variable `HPX_INI`. This variable is expected to provide the full path name of the ini configuration file (if any).
5. Load a file named `/etc/hpx.ini`. This lookup is done on non-Windows systems only.
6. Load a file named `.hpx.ini` in the home directory of the current user, e.g., the directory referenced by the environment variable `HOME`.
7. Load a file named `.hpx.ini` in the directory referenced by the environment variable `PWD`.
8. Load the file specified on the command line using the option `--hpx:config`.
9. Load all properties specified on the command line using the option `--hpx:ini`. The properties will be added to the database in the same sequence as they are specified on the command line. The format for those options is, for instance, `--hpx:ini=hpx.default_stack_size=0x4000`. In addition to the explicit command line options, this will set the following properties as implied from other settings:
   - `hpx.parcel.address` and `hpx.parcel.port` as set by `--hpx:hpx`
   - `hpx.agas.address`, `hpx.agas.port` and `hpx.agas.service_mode` as set by `--hpx:agas`
   - `hpx.program_name` and `hpx.cmd_line` will be derived from the actual command line
   - **`hpx.os_threads` and `hpx.localities` as set by**
     `--hpx:threads` and `--hpx:localities`
   - `hpx.runtime_mode` will be derived from any explicit `--hpx:console`, `--hpx:worker`, or `--hpx:connect`, or it will be derived from other settings, such as `--hpx:node =0`, which implies `--hpx:console`.
10. Load files based on the pattern `*.ini` in all directories listed by the property `hpx.ini_path`. All files found during this search will be merged. The property `hpx.ini_path` can hold a list of directories separated by `':'` (on Linux or Mac) or `';'` (on Windows).
11. Load the file specified on the command line using the option `--hpx:app-config`. Note that this file will be merged as the content for a top level section `[application]`.

Note

Any changes made to the configuration database caused by one of the steps will influence the loading process for all subsequent steps. For instance, if one of the ini files loaded changes the property `hpx.ini_path`, this will influence the directories searched in step 9 as described above.

Important

The *HPX* core library will verify that all configuration settings specified on the command line (using the `--hpx:ini` option) will be checked for validity. That means that the library will accept only *known* configuration settings. This is to protect the user from unintentional typos while specifying those settings. This behavior can be overwritten by appending a `'!'` to the configuration key, thus forcing the setting to be entered into the configuration database. For instance: `--hpx:ini=hpx.foo! = 1`

If any of the environment variables or files listed above are not found, the corresponding loading step will be silently skipped.

Loading components

HPX relies on loading application specific components during the runtime of an application. Moreover, *HPX* comes with a set of preinstalled components supporting basic functionalities useful for almost every application. Any component in *HPX* is loaded from a shared library, where any of the shared libraries can contain more than one component type. During startup, *HPX* tries to locate all available components (e.g., their corresponding shared libraries) and creates an internal component registry for later use. This section describes the algorithm used by *HPX* to locate all relevant shared libraries on a system. As described, this algorithm is customizable by the configuration properties loaded from the ini files (see section *Loading INI files*).

Loading components is a two-stage process. First *HPX* tries to locate all component shared libraries, loads those, and generates a default configuration section in the internal configuration database for each component found. For each found component the following information is generated:

```
[hpx.components.<component_instance_name>]
name = <name_of_shared_library>
path = $[component_path]
enabled = $[hpx.components.load_external]
default = 1
```

The values in this section correspond to the expected configuration information for a component as described in the section *Built-in default configuration settings*.

In order to locate component shared libraries, *HPX* will try loading all shared libraries (files with the platform specific extension of a shared library, Linux: `*.so`, Windows: `*.dll`, MacOS: `*.dylib` found in the directory referenced by the ini property `hpx.component_path`).

This first step corresponds to step 1) during the process of filling the internal configuration database with default information as described in section *Loading INI files*.

After all of the configuration information has been loaded, *HPX* performs the second step in terms of loading components. During this step, *HPX* scans all existing configuration sections `[hpx.component.<some_component_instance_name>]` and instantiates a special factory object for each of the successfully located and loaded components. During the application's life time, these factory objects are responsible for creating new and discarding old instances of the component they are associated with. This step is performed after step 11) of the process of filling the internal configuration database with default information as described in section *Loading INI files*.

Application specific component example

This section assumes there is a simple application component that exposes one member function as a component action. The header file `app_server.hpp` declares the C++ type to be exposed as a component. This type has a member function `print_greeting()`, which is exposed as an action `print_greeting_action`. We assume the source files for this example are located in a directory referenced by `$APP_ROOT`:

```
// file: $APP_ROOT/app_server.hpp
#include <hpx/hpx.hpp>
#include <hpx/include/iostreams.hpp>

namespace app
{
    // Define a simple component exposing one action 'print_greeting'
    class HPX_COMPONENT_EXPORT server
      : public hpx::components::component_base<server>
    {
        void print_greeting ()
        {
            hpx::cout << "Hey, how are you?\n" << std::flush;
        }

        // Component actions need to be declared, this also defines the
        // type 'print_greeting_action' representing the action.
        HPX_DEFINE_COMPONENT_ACTION(server, print_greeting, print_greeting_action);
    };
}

// Declare boilerplate code required for each of the component actions.
HPX_REGISTER_ACTION_DECLARATION(app::server::print_greeting_action);
```

The corresponding source file contains mainly macro invocations that define the boilerplate code needed for *HPX* to function properly:

```
// file: $APP_ROOT/app_server.cpp
#include "app_server.hpp"

// Define boilerplate required once per component module.
HPX_REGISTER_COMPONENT_MODULE();

// Define factory object associated with our component of type 'app::server'.
HPX_REGISTER_COMPONENT(app::server, app_server);

// Define boilerplate code required for each of the component actions. Use the
// same argument as used for HPX_REGISTER_ACTION_DECLARATION above.
HPX_REGISTER_ACTION(app::server::print_greeting_action);
```

The following gives an example of how the component can be used. Here, one instance of the `app::server` component is created on the current *locality* and the exposed action `print_greeting_action` is invoked using the global id of the newly created instance. Note that no special code is required to delete the component instance after it is not needed anymore. It will be deleted automatically when its last reference goes out of scope (shown in the example below at the closing brace of the block surrounding the code):

```cpp
// file: $APP_ROOT/use_app_server_example.cpp
#include <hpx/hpx_init.hpp>
#include "app_server.hpp"

int hpx_main()
{
    {
        // Create an instance of the app_server component on the current locality.
        hpx::naming:id_type app_server_instance =
            hpx::create_component<app::server>(hpx::find_here());

        // Create an instance of the action 'print_greeting_action'.
        app::server::print_greeting_action print_greeting;

        // Invoke the action 'print_greeting' on the newly created component.
        print_greeting(app_server_instance);
    }
    return hpx::finalize();
}

int main(int argc, char* argv[])
{
    return hpx::init(argc, argv);
}
```

In order to make sure that the application will be able to use the component `app::server`, special configuration information must be passed to *HPX*. The simplest way to allow *HPX* to 'find' the component is to provide special ini configuration files that add the necessary information to the internal configuration database. The component should have a special ini file containing the information specific to the component `app_server`.

```ini
# file: $APP_ROOT/app_server.ini
[hpx.components.app_server]
name = app_server
path = $APP_LOCATION/
```

Here, `$APP_LOCATION` is the directory where the (binary) component shared library is located. *HPX* will attempt to load the shared library from there. The section name `hpx.components.app_server` reflects the instance name of the component (`app_server` is an arbitrary, but unique name). The property value for `hpx.components.app_server.name` should be the same as used for the second argument to the macro *HPX_REGISTER_COMPONENT* above.

Additionally, a file `.hpx.ini`, which could be located in the current working directory (see step 3 as described in the section *Loading INI files*), can be used to add to the ini search path for components:

```ini
# file: $PWD/.hpx.ini
[hpx]
ini_path = $[hpx.ini_path]:$APP_ROOT/
```

This assumes that the above ini file specific to the component is located in the directory `$APP_ROOT`.

Note

It is possible to reference the defined property from inside its value. *HPX* will gracefully use the previous value of `hpx.ini_path` for the reference on the right hand side and assign the overall (now expanded) value to the property.

Logging

HPX uses a sophisticated logging framework, allowing users to follow in detail what operations have been performed inside the *HPX* library in what sequence. This information proves to be very useful for diagnosing problems or just for improving the understanding of what is happening in *HPX* as a consequence of invoking *HPX* API functionality.

Default logging

Enabling default logging is a simple process. The detailed description in the remainder of this section explains different ways to customize the defaults. Default logging can be enabled by using one of the following:

- A command line switch `--hpx:debug-hpx-log`, which will enable logging to the console terminal.
- The command line switch `--hpx:debug-hpx-log=<filename>`, which enables logging to a given file `<filename>`.
- Setting an environment variable `HPX_LOGLEVEL=<loglevel>` while running the *HPX* application. In this case `<loglevel>` should be a number between (or equal to) `1` and `5` where `1` means minimal logging and `5` causes all available messages to be logged. When setting the environment variable, the logs will be written to a file named `hpx.<PID>.lo` in the current working directory, where `<PID>` is the process id of the console instance of the application.

Customizing logging

Generally, logging can be customized either using environment variable settings or using by an ini configuration file. Logging is generated in several categories, each of which can be customized independently. All customizable configuration parameters have reasonable defaults, allowing for the use of logging without any additional configuration effort. The following table lists the available categories.

Table 2.5: Logging categories

Category	Category shortcut	Information to be generated	Environment variable
General	None	Logging information generated by different subsystems of *HPX*, such as thread-manager, parcel layer, LCOs, etc.	`HPX_LOGLEVEL`
AGAS	`AGAS`	Logging output generated by the *AGAS* subsystem	`HPX_AGAS_LOGLEVEL`
Application	`APP`	Logging generated by applications.	`HPX_APP_LOGLEVEL`

By default, all logging output is redirected to the console instance of an application, where it is collected and written to a file, one file for each logging category.

Each logging category can be customized at two levels. The parameters for each are stored in the ini configuration sections `hpx.logging.CATEGORY` and `hpx.logging.console.CATEGORY` (where `CATEGORY` is the category shortcut as listed in the table above). The former influences logging at the source *locality* and the latter modifies the logging behaviour for each of the categories at the console instance of an application.

Levels

All *HPX* logging output has seven different logging levels. These levels can be set explicitly or through environment variables in the main *HPX* ini file as shown below. The logging levels and their associated integral values are shown in the table below, ordered from most verbose to least verbose. By default, all *HPX* logs are set to 0, e.g., all logging output is disabled by default.

Table 2.6: Logging levels

Logging level	Integral value
<debug>	5
<info>	4
<warning>	3
<error>	2
<fatal>	1
No logging	0

Tip

The easiest way to enable logging output is to set the environment variable corresponding to the logging category to an integral value as described in the table above. For instance, setting HPX_LOGLEVEL=5 will enable full logging output for the general category. Please note that the syntax and means of setting environment variables varies between operating systems.

Configuration

Logs will be saved to destinations as configured by the user. By default, logging output is saved on the console instance of an application to hpx.<CATEGORY>.<PID>.lo (where CATEGORY and PID> are placeholders for the category shortcut and the OS process id). The output for the general logging category is saved to hpx.<PID>.log. The default settings for the general logging category are shown here (the syntax is described in the section *The HPX ini file format*):

```
[hpx.logging]
level = ${HPX_LOGLEVEL:0}
destination = ${HPX_LOGDESTINATION:console}
format = ${HPX_LOGFORMAT:(T%locality%/%hpxthread%.%hpxphase%/%hpxcomponent%) P%parentloc
↪%/%hpxparent%.%hpxparentphase% %time%($hh:$mm.$ss.$mili) [%idx%]|\\n}
```

The logging level is taken from the environment variable HPX_LOGLEVEL and defaults to zero, e.g., no logging. The default logging destination is read from the environment variable HPX_LOGDESTINATION On any of the localities it defaults to console, which redirects all generated logging output to the console instance of an application. The following table lists the possible destinations for any logging output. It is possible to specify more than one destination separated by whitespace.

Table 2.7: Logging destinations

Logging destination	Description
file(<filename>)	Directs all output to a file with the given <filename>.
cout	Directs all output to the local standard output of the application instance on this *locality*.
cerr	Directs all output to the local standard error output of the application instance on this *locality*.
console	Directs all output to the console instance of the application. The console instance has its logging destinations configured separately.
android_log	Directs all output to the (Android) system log (available on Android systems only).

The logging format is read from the environment variable `HPX_LOGFORMAT`, and it defaults to a complex format description. This format consists of several placeholder fields (for instance `%locality%`), which will be replaced by concrete values when the logging output is generated. All other information is transferred verbatim to the output. The table below describes the available field placeholders. The separator character `|` separates the logging message prefix formatted as shown and the actual log message which will replace the separator.

Table 2.8: Available field placeholders

Name	Description
locality	The id of the *locality* on which the logging message was generated.
hpxthread	The id of the *HPX* thread generating this logging output.
hpxphase	The phase[76] of the *HPX* thread generating this logging output.
hpxcomponent	The local virtual address of the component which the current *HPX* thread is accessing.
parentloc	The id of the *locality* where the *HPX* thread was running that initiated the current *HPX* thread. The current *HPX* thread is generating this logging output.
hpxparent	The id of the *HPX* thread that initiated the current *HPX* thread. The current *HPX* thread is generating this logging output.
hpxparentphase	The phase of the *HPX* thread when it initiated the current *HPX* thread. The current *HPX* thread is generating this logging output.
time	The time stamp for this logging outputline as generated by the source *locality*.
idx	The sequence number of the logging output line as generated on the source *locality*.
osthread	The sequence number of the OS thread that executes the current *HPX* thread.

Note

Not all of the field placeholder may be expanded for all generated logging output. If no value is available for a particular field, it is replaced with a sequence of `'-'` characters.

Here is an example line from a logging output generated by one of the *HPX* examples (please note that this is generated on a single line, without a line break):

```
(T00000000/0000000002d46f90.01/00000000009ebc10) P--------/0000000002d46f80.02 17:49.37.
↪320 [000000000000004d]
    <info>  [RT] successfully created component {0000000100ff0001, 0000000000030002} of␣
↪type: component_barrier[7(3)]
```

The default settings for the general logging category on the console is shown here:

[76] The phase of a *HPX*-thread counts how often this thread has been activated.

```
[hpx.logging.console]
level = ${HPX_LOGLEVEL:$[hpx.logging.level]}
destination = ${HPX_CONSOLE_LOGDESTINATION:file(hpx.$[system.pid].log)}
format = ${HPX_CONSOLE_LOGFORMAT:|}
```

These settings define how the logging is customized once the logging output is received by the console instance of an application. The logging level is read from the environment variable `HPX_LOGLEVEL` (as set for the console instance of the application). The level defaults to the same values as the corresponding settings in the general logging configuration shown before. The destination on the console instance is set to be a file that's name is generated based on its OS process id. Setting the environment variable `HPX_CONSOLE_LOGDESTINATION` allows customization of the naming scheme for the output file. The logging format is set to leave the original logging output unchanged, as received from one of the localities the application runs on.

HPX Command Line Options

The predefined command line options for any application using *hpx::init* are described in the following subsections.

HPX options (allowed on command line only)

`--hpx:help`
: Print out program usage (default: this message). Possible values: `full` (additionally prints options from components).

`--hpx:version`
: Print out *HPX* version and copyright information.

`--hpx:info`
: Print out *HPX* configuration information.

`--hpx:options-file` `arg`
: Specify a file containing command line options (alternatively: @filepath).

HPX options (additionally allowed in an options file)

`--hpx:worker`
: Run this instance in worker mode.

`--hpx:console`
: Run this instance in console mode.

`--hpx:connect`
: Run this instance in worker mode, but connecting late.

`--hpx:run-agas-server`
: Run *AGAS* server as part of this runtime instance.

`--hpx:run-hpx-main`
: Run the hpx_main function, regardless of *locality* mode.

`--hpx:hpx` `arg`
: The IP address the *HPX* parcelport is listening on, expected format: `address:port` (default: `127.0.0.1:7910`).

--hpx:agas arg

The IP address the *AGAS* root server is running on, expected format: `address:port` (default: `127.0.0.1:7910`).

--hpx:run-agas-server-only

Run only the *AGAS* server.

--hpx:nodefile arg

The file name of a node file to use (list of nodes, one node name per line and core).

--hpx:nodes arg

The (space separated) list of the nodes to use (usually this is extracted from a node file).

--hpx:endnodes

This can be used to end the list of nodes specified using the option *--hpx:nodes*.

--hpx:ifsuffix arg

Suffix to append to host names in order to resolve them to the proper network interconnect.

--hpx:ifprefix arg

Prefix to prepend to host names in order to resolve them to the proper network interconnect.

--hpx:iftransform arg

Sed-style search and replace (`s/search/replace/`) used to transform host names to the proper network interconnect.

--hpx:force_ipv4

Network hostnames will be resolved to ipv4 addresses instead of using the first resolved endpoint. This is especially useful on Windows where the local hostname will resolve to an ipv6 address while remote network hostnames are commonly resolved to ipv4 addresses.

--hpx:localities arg

The number of localities to wait for at application startup (default: `1`).

--hpx:node arg

Number of the node this *locality* is run on (must be unique).

--hpx:ignore-batch-env

Ignore batch environment variables.

--hpx:expect-connecting-localities

This *locality* expects other localities to dynamically connect (this is implied if the number of initial localities is larger than 1).

--hpx:pu-offset

The first processing unit this instance of *HPX* should be run on (default: `0`).

--hpx:pu-step

The step between used processing unit numbers for this instance of *HPX* (default: `1`).

--hpx:threads arg

The number of operating system threads to spawn for this *HPX locality*. Possible values are: numeric values `1`, `2`, `3` and so on, `all` (which spawns one thread per processing unit, includes hyperthreads), or `cores` (which spawns one thread per core) (default: `cores`).

--hpx:cores arg

The number of cores to utilize for this *HPX locality* (default: `all`, i.e., the number of cores is based on the number of threads *--hpx:threads* assuming *--hpx:bind*`=compact`.

--hpx:affinity arg

The affinity domain the OS threads will be confined to, possible values: `pu`, `core`, `numa`, `machine` (default: `pu`).

--hpx:bind arg

he detailed affinity description for the OS threads, see *More details about HPX command line options* for a detailed description of possible values. Do not use with *--hpx:pu-step*, *--hpx:pu-offset* or *--hpx:affinity* options. Implies *--hpx:numa-sensitive* (*--hpx:bind*=none) disables defining thread affinities).

--hpx:use-process-mask

Use the process mask to restrict available hardware resources (implies *--hpx:ignore-batch-env*).

--hpx:print-bind

Print to the console the bit masks calculated from the arguments specified to all *--hpx:bind* options.

--hpx:queuing arg

The queue scheduling policy to use. Options are `local`, `local-priority-fifo`, `local-priority-lifo`, `static`, `static-priority`, `abp-priority-fifo`, `local-workrequesting-fifo`, `local-workrequesting-lifo` `local-workrequesting-mc`, and `abp-priority-lifo` (default: `local-priority-fifo`).

--hpx:high-priority-threads arg

The number of operating system threads maintaining a high priority queue (default: number of OS threads), valid for *--hpx:queuing*=abp-priority, *--hpx:queuing*`static-priority` and *--hpx:queuing*`local-priority` only.

--hpx:numa-sensitive

Makes the scheduler NUMA sensitive.

HPX configuration options

--hpx:app-config arg

Load the specified application configuration (ini) file.

--hpx:config arg

Load the specified *HPX* configuration (ini) file.

--hpx:ini arg

Add a configuration definition to the default runtime configuration.

--hpx:exit

Exit after configuring the runtime.

HPX debugging options

--hpx:list-symbolic-names

List all registered symbolic names after startup.

--hpx:list-component-types

List all dynamic component types after startup.

--hpx:dump-config-initial

Print the initial runtime configuration.

`--hpx:dump-config`

Print the final runtime configuration.

`--hpx:debug-hpx-log [arg]`

Enable all messages on the *HPX* log channel and send all *HPX* logs to the target destination (default: `cout`).

`--hpx:debug-agas-log [arg]`

Enable all messages on the *AGAS* log channel and send all *AGAS* logs to the target destination (default: `cout`).

`--hpx:debug-parcel-log [arg]`

Enable all messages on the parcel transport log channel and send all parcel transport logs to the target destination (default: `cout`).

`--hpx:debug-timing-log [arg]`

Enable all messages on the timing log channel and send all timing logs to the target destination (default: `cout`).

`--hpx:debug-app-log [arg]`

Enable all messages on the application log channel and send all application logs to the target destination (default: `cout`).

`--hpx:debug-clp`

Debug command line processing.

`--hpx:attach-debugger arg`

Wait for a debugger to be attached, possible arg values: `startup` or `exception` (default: `startup`)

HPX options related to performance counters

`--hpx:print-counter`

Print the specified performance counter either repeatedly and/or at the times specified by *--hpx:print-counter-at* (see also option *--hpx:print-counter-interval*).

`--hpx:print-counter-reset`

Print the specified performance counter either repeatedly and/or at the times specified by *--hpx:print-counter-at*. Reset the counter after the value is queried (see also option *--hpx:print-counter-interval*).

`--hpx:print-counter-interval`

Print the performance counter(s) specified with *--hpx:print-counter* repeatedly after the time interval (specified in milliseconds), (default: `0`, which means print once at shutdown).

`--hpx:print-counter-destination`

Print the performance counter(s) specified with *--hpx:print-counter* to the given file (default: `console`).

`--hpx:list-counters`

List the names of all registered performance counters, possible values: `minimal` (prints counter name skeletons), `full` (prints all available counter names).

`--hpx:list-counter-infos`

List the description of all registered performance counters, possible values: `minimal` (prints info for counter name skeletons), `full` (prints all available counter infos).

`--hpx:print-counter-format`

Print the performance counter(s) specified with *--hpx:print-counter*. Possible formats in CSV include a format with a header or without any header (see option *--hpx:no-csv-header*). Possible values: `csv` (prints counter values in CSV format with full names as header), `csv-short` (prints counter values in CSV format with short names provided with *--hpx:print-counter* as *--hpx:print-counter* `shortname,full-countername`

`--hpx:no-csv-header`

Print the performance counter(s) specified with *--hpx:print-counter* and `csv` or `csv-short` format specified with *--hpx:print-counter-format* without header.

`--hpx:print-counter-at` `arg`

Print the performance counter(s) specified with *--hpx:print-counter* (or *--hpx:print-counter-reset*) at the given point in time, possible argument values: `startup`, `shutdown` (default), `noshutdown`.

`--hpx:reset-counters`

Reset all performance counter(s) specified with *--hpx:print-counter* after they have been evaluated.

`--hpx:print-counters-locally`

Each *locality* prints only its own local counters. If this is used with *--hpx:print-counter-destination*=`<file>`, the code will append a `".<locality_id>"` to the file name in order to avoid clashes between localities.

Command line argument shortcuts

Additionally, the following shortcuts are available from every *HPX* application.

Table 2.9: Predefined command line option shortcuts

Shortcut option	Equivalent long option
`-a`	*--hpx:agas*
`-c`	*--hpx:console*
`-h`	*--hpx:help*
`-I`	*--hpx:ini*
`-l`	*--hpx:localities*
`-p`	*--hpx:app-config*
`-q`	*--hpx:queuing*
`-r`	*--hpx:run-agas-server*
`-t`	*--hpx:threads*
`-v`	*--hpx:version*
`-w`	*--hpx:worker*
`-x`	*--hpx:hpx*
`-0`	*--hpx:node*=0
`-1`	*--hpx:node*=1
`-2`	*--hpx:node*=2
`-3`	*--hpx:node*=3
`-4`	*--hpx:node*=4
`-5`	*--hpx:node*=5
`-6`	*--hpx:node*=6
`-7`	*--hpx:node*=7
`-8`	*--hpx:node*=8
`-9`	*--hpx:node*=9

Note

The short options listed above are disabled by default if the application is built using `#include <hpx/hpx_main.hpp>`. See *Re-use the main() function as the main HPX entry point* for more information. The rationale behind this is that in this case the user's application may handle its own command line options, since *HPX* passes all unknown options to `main()`. Short options like `-t` are prone to create ambiguities regarding what the application will support. Hence, the user should instead rely on the corresponding long options like `--hpx:threads` in such a case.

It is possible to define your own shortcut options. In fact, all of the shortcuts listed above are pre-defined using the technique described here. Also, it is possible to redefine any of the pre-defined shortcuts to expand differently as well.

Shortcut options are obtained from the internal configuration database. They are stored as key-value properties in a special properties section named `hpx.commandline`. You can define your own shortcuts by adding the corresponding definitions to one of the ini configuration files as described in the section *Configuring HPX applications*. For instance, in order to define a command line shortcut `--p`, which should expand to `-hpx:print-counter`, the following configuration information needs to be added to one of the ini configuration files:

```
[hpx.commandline.aliases]
--pc = --hpx:print-counter
```

Note

Any arguments for shortcut options passed on the command line are retained and passed as arguments to the corresponding expanded option. For instance, given the definition above, the command line option:

```
--pc=/threads{locality#0/total}/count/cumulative
```

would be expanded to:

```
--hpx:print-counter=/threads{locality#0/total}/count/cumulative
```

Important

Any shortcut option should either start with a single `'-'` or with two `'--'` characters. Shortcuts starting with a single `'-'` are interpreted as short options (i.e., everything after the first character following the `'-'` is treated as the argument). Shortcuts starting with `'--'` are interpreted as long options. No other shortcut formats are supported.

Specifying options for single localities only

For runs involving more than one *locality*, it is sometimes desirable to supply specific command line options to single localities only. When the *HPX* application is launched using a scheduler (like PBS; for more details see section *How to use HPX applications with PBS*), specifying dedicated command line options for single localities may be desirable. For this reason all of the command line options that have the general format `--hpx:<some_key>` can be used in a more general form: `--hpx:<N>:<some_key>`, where `<N>` is the number of the *locality* this command line option will be applied to; all other localities will simply ignore the option. For instance, the following PBS script passes the option `--hpx:pu-offset=4` to the *locality* `'1'` only.

```
#!/bin/bash
#
#PBS -l nodes=2:ppn=4
```

(continues on next page)

(continued from previous page)

```
APP_PATH=~/packages/hpx/bin/hello_world_distributed
APP_OPTIONS=

pbsdsh -u $APP_PATH $APP_OPTIONS --hpx:1:pu-offset=4 --hpx:nodes=`cat $PBS_NODEFILE`
```

Caution

If the first application specific argument (inside `$APP_OPTIONS`) is a non-option (i.e., does not start with a `-` or a `--`), then it must be placed before the option *--hpx:nodes*, which, in this case, should be the last option on the command line.

Alternatively, use the option *--hpx:endnodes* to explicitly mark the end of the list of node names:

```
$ pbsdsh -u $APP_PATH --hpx:1:pu-offset=4 --hpx:nodes=`cat $PBS_NODEFILE` --
↪hpx:endnodes $APP_OPTIONS
```

More details about *HPX* command line options

This section documents the following list of the command line options in more detail:

- *The command line option --hpx:bind*

The command line option `--hpx:bind`

This command line option allows one to specify the required affinity of the *HPX* worker threads to the underlying processing units. As a result the worker threads will run only on the processing units identified by the corresponding bind specification. The affinity settings are to be specified using *--hpx:bind*=`<BINDINGS>`, where `<BINDINGS>` have to be formatted as described below.

In addition to the syntax described below, one can use *--hpx:bind*=`none` to disable all binding of any threads to a particular core. This is mostly supported for debugging purposes.

The specified affinities refer to specific regions within a machine hardware topology. In order to understand the hardware topology of a particular machine, it may be useful to run the lstopo tool, which is part of Portable Hardware Locality (HWLOC), to see the reported topology tree. Seeing and understanding a topology tree will definitely help in understanding the concepts that are discussed below.

Affinities can be specified using hwloc tuples. Tuples of hwloc *objects* and associated *indexes* can be specified in the form `object:index`, `object:index-index` or `object:index,...,index`. Hwloc objects represent types of mapped items in a topology tree. Possible values for objects are `socket`, `numanode`, `core` and `pu` (processing unit). Indexes are non-negative integers that specify a unique physical object in a topology tree using its logical sequence number.

Chaining multiple tuples together in the more general form `object1:index1[.object2:index2[...]]` is permissible. While the first tuple's object may appear anywhere in the topology, the Nth tuple's object must have a shallower topology depth than the (N+1)th tuple's object. Put simply: as you move right in a tuple chain, objects must go deeper in the topology tree. Indexes specified in chained tuples are relative to the scope of the parent object. For example, `socket:0.core:1` refers to the second core in the first socket (all indices are zero based).

Multiple affinities can be specified using several *--hpx:bind* command line options or by appending several affinities separated by a `';'`. By default, if multiple affinities are specified, they are added.

`"all"` is a special affinity consisting in the entire current topology.

Note

All "names" in an affinity specification, such as `thread`, `socket`, `numanode`, `pu` or `all`, can be abbreviated. Thus, the affinity specification `threads:0-3=socket:0.core:1.pu:1` is fully equivalent to its shortened form `t:0-3=s:0.c:1.p:1`.

Here is a full grammar describing the possible format of mappings:

```
mappings      ::=  distribution | mapping (";" mapping)*
distribution  ::=  "compact" | "scatter" | "balanced" | "numa-balanced"
mapping       ::=  thread_spec "=" pu_specs
thread_spec   ::=  "thread:" range_specs
pu_specs      ::=  pu_spec ("." pu_spec)*
pu_spec       ::=  type ":" range_specs | "~" pu_spec
range_specs   ::=  range_spec ("," range_spec)*
range_spec    ::=  int | int "-" int | "all"
type          ::=  "socket" | "numanode" | "core" | "pu"
```

The following example assumes a system with at least 4 cores, where each core has more than 1 processing unit (hardware threads). Running `hello_world_distributed` with 4 OS threads (on 4 processing units), where each of those threads is bound to the first processing unit of each of the cores, can be achieved by invoking:

```
$ hello_world_distributed -t4 --hpx:bind=thread:0-3=core:0-3.pu:0
```

Here, `thread:0-3` specifies the OS threads used to define affinity bindings, and `core:0-3.pu:` defines that for each of the cores (`core:0-3`) only their first processing unit `pu:0` should be used.

Note

The command line option *--hpx:print-bind* can be used to print the bitmasks generated from the affinity mappings as specified with *--hpx:bind*. For instance, on a system with hyperthreading enabled (i.e. 2 processing units per core), the command line:

```
$ hello_world_distributed -t4 --hpx:bind=thread:0-3=core:0-3.pu:0 --hpx:print-bind
```

will cause this output to be printed:

```
0: PU L#0(P#0), Core L#0, Socket L#0, Node L#0(P#0)
1: PU L#2(P#2), Core L#1, Socket L#0, Node L#0(P#0)
2: PU L#4(P#4), Core L#2, Socket L#0, Node L#0(P#0)
3: PU L#6(P#6), Core L#3, Socket L#0, Node L#0(P#0)
```

where each bit in the bitmasks corresponds to a processing unit the listed worker thread will be bound to run on.

The difference between the four possible predefined distribution schemes (`compact`, `scatter`, `balanced` and `numa-balanced`) is best explained with an example. Imagine that we have a system with 4 cores and 4 hardware threads per core on 2 sockets. If we place 8 threads the assignments produced by the `compact`, `scatter`, `balanced` and `numa-balanced` types are shown in the figure below. Notice that `compact` does not fully utilize all the cores in the system. For this reason it is recommended that applications are run using the `scatter` or `balanced`/`numa-balanced` options in most cases.

In addition to the predefined distributions it is possible to restrict the resources used by *HPX* to the process CPU

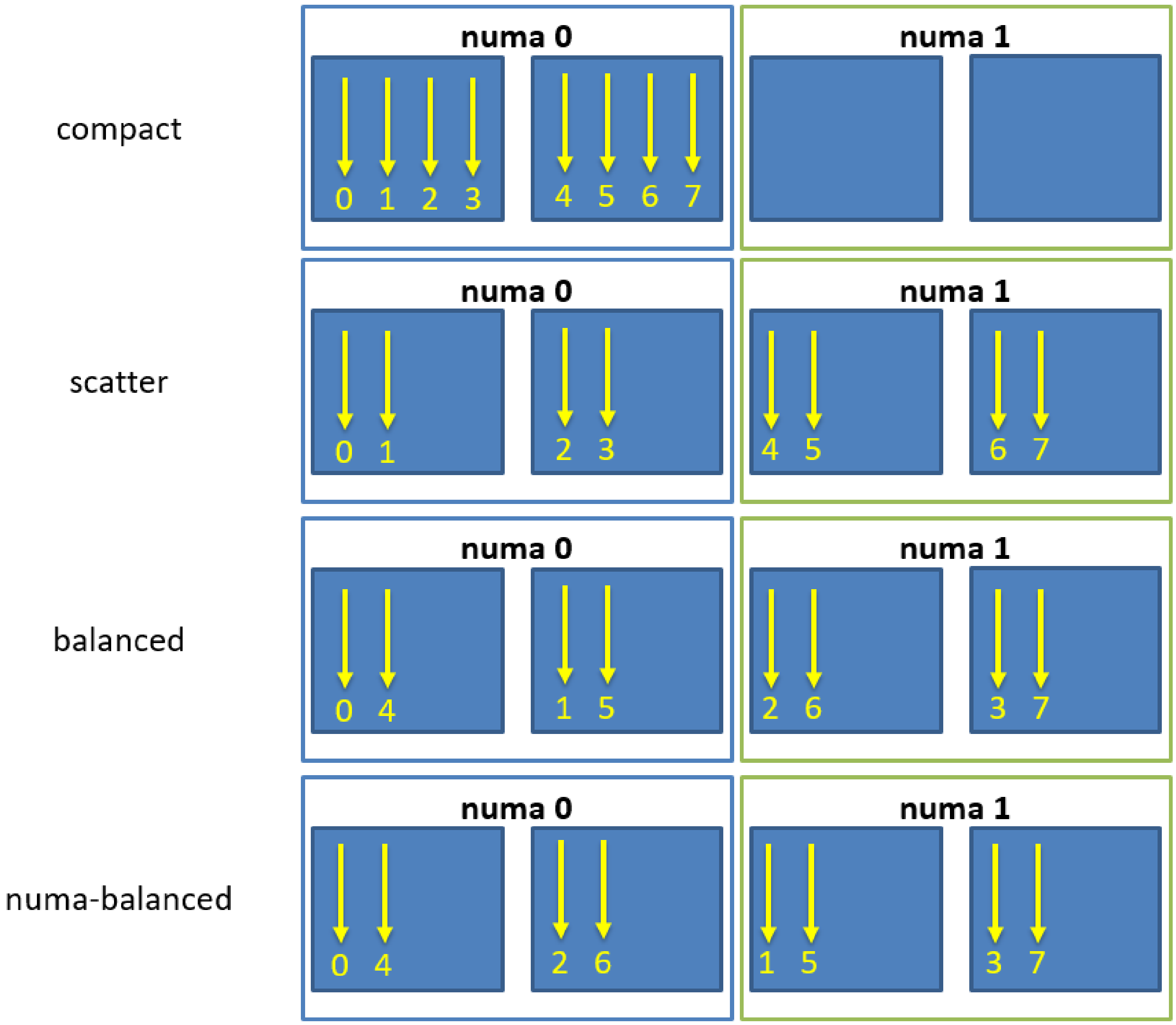


Fig. 2.7: Schematic of thread affinity type distributions.

mask. The CPU mask is typically set by e.g. MPI[75] and batch environments. Using the command line option `--hpx:use-process-mask` makes *HPX* act as if only the processing units in the CPU mask are available for use by *HPX*. The number of threads is automatically determined from the CPU mask. The number of threads can still be changed manually using this option, but only to a number less than or equal to the number of processing units in the CPU mask. The option `--hpx:print-bind` is useful in conjunction with `--hpx:use-process-mask` to make sure threads are placed as expected.

2.4.11 Writing single-node applications

Being a C++ Standard Library for Concurrency and Parallelism, *HPX* implements all of the corresponding facilities as defined by the C++ Standard but also those which are proposed as part of the ongoing C++ standardization process. This section focuses on the features available in *HPX* for parallel and concurrent computation on a single node, although many of the features presented here are also implemented to work in the distributed case.

Synchronization objects

The following objects are providing synchronization for *HPX* applications:

1. *Barrier*
2. *Condition variable*
3. *Latch*
4. *Mutex*
5. *Shared mutex*
6. *Semaphore*
7. *Composable guards*

Barrier

Barriers are used for synchronizing multiple threads. They provide a synchronization point, where all threads must wait until they have all reached the barrier, before they can continue execution. This allows multiple threads to work together to solve a common task, and ensures that no thread starts working on the next task until all threads have completed the current task. This ensures that all threads are in the same state before performing any further operations, leading to a more consistent and accurate computation.

Unlike latches, barriers are reusable: once the participating threads are released from a barrier's synchronization point, they can re-use the same barrier. It is thus useful for managing repeated tasks, or phases of a larger task, that are handled by multiple threads. The code below shows how barriers can be used to synchronize two threads:

```
#include <hpx/barrier.hpp>
#include <hpx/future.hpp>
#include <hpx/init.hpp>

#include <iostream>

int hpx_main()
{
    hpx::barrier b(2);
```

(continues on next page)

[75] https://en.wikipedia.org/wiki/Message_Passing_Interface

(continued from previous page)

```
    hpx::future<void> f1 = hpx::async([&b]() {
        std::cout << "Thread 1 started." << std::endl;
        // Do some computation
        b.arrive_and_wait();
        // Continue with next task
        std::cout << "Thread 1 finished." << std::endl;
    });

    hpx::future<void> f2 = hpx::async([&b]() {
        std::cout << "Thread 2 started." << std::endl;
        // Do some computation
        b.arrive_and_wait();
        // Continue with next task
        std::cout << "Thread 2 finished." << std::endl;
    });

    f1.get();
    f2.get();

    return hpx::local::finalize();
}

int main(int argc, char* argv[])
{
    return hpx::local::init(hpx_main, argc, argv);
}
```

In this example, two `hpx::future` objects are created, each representing a separate thread of execution. The `wait` function of the `hpx::barrier` object is called by each thread. The threads will wait at the barrier until both have reached it. Once both threads have reached the barrier, they can continue with their next task.

Condition variable

A *condition variable* is a synchronization primitive in *HPX* that allows a thread to wait for a specific condition to be satisfied before continuing execution. It is typically used in conjunction with a mutex or a lock to protect shared data that is being modified by multiple threads. Hence, it blocks one or more threads until another thread both modifies a shared variable (the condition) and notifies the `condition_variable`. The code below shows how two threads modifying the shared variable `data` can be synchronized using the `condition_variable`:

```
#include <hpx/condition_variable.hpp>
#include <hpx/init.hpp>
#include <hpx/mutex.hpp>
#include <hpx/thread.hpp>

#include <iostream>
#include <string>

hpx::condition_variable cv;
hpx::mutex m;
std::string data;
```

(continues on next page)

(continued from previous page)

```
bool ready = false;
bool processed = false;

void worker_thread()
{
    // Wait until the main thread signals that data is ready
    std::unique_lock<hpx::mutex> lk(m);
    cv.wait(lk, [] { return ready; });

    // Access the shared resource
    std::cout << "Worker thread: Processing data...\n";
    data = "Test data after";

    // Send data back to the main thread
    processed = true;
    std::cout << "Worker thread: data processing is complete\n";

    // Manual unlocking is done before notifying, to avoid waking up
    // the waiting thread only to block again
    lk.unlock();
    cv.notify_one();
}

int hpx_main()
{
    hpx::thread worker(worker_thread);

    // Do some work
    std::cout << "Main thread: Preparing data...\n";
    data = "Test data before";
    hpx::this_thread::sleep_for(std::chrono::seconds(1));
    std::cout << "Main thread: Data before processing = " << data << '\n';

    // Signal that data is ready and send data to worker thread
    {
        std::lock_guard<hpx::mutex> lk(m);
        ready = true;
        std::cout << "Main thread: Data is ready...\n";
    }
    cv.notify_one();

    // Wait for the worker thread to finish
    {
        std::unique_lock<hpx::mutex> lk(m);
        cv.wait(lk, [] { return processed; });
    }
    std::cout << "Main thread: Data after processing = " << data << '\n';
    worker.join();

    return hpx::local::finalize();
}
```

(continues on next page)

(continued from previous page)

```
int main(int argc, char* argv[])
{
    return hpx::local::init(hpx_main, argc, argv);
}
```

The main thread of the code above starts by creating a worker thread and preparing the shared variable `data`. Once the data is ready, the main thread acquires a lock on the mutex `m` using `std::lock_guard<hpx::mutex> lk(m)` and sets the ready flag to true, then signals the worker thread to start processing by calling `cv.notify_one()`. The `cv.wait()` call in the main thread then blocks until the worker thread signals that processing is complete by setting the `processed` flag.

The worker thread starts by acquiring a lock on the mutex `m` to ensure exclusive access to the shared data. The `cv.wait()` call blocks the thread until the `ready` flag is set by the main thread. Once this is true, the worker thread accesses the shared data resource, processes it, and sets the `processed` flag to indicate completion. The mutex is then unlocked using `lk.unlock()` and the `cv.notify_one()` call signals the main thread to resume execution. Finally, the new `data` is printed by the main thread to the console.

Latch

A *latch* is a downward counter which can be used to synchronize threads. The value of the counter is initialized on creation. Threads may block on the latch until the counter is decremented to zero. There is no possibility to increase or reset the counter, which makes the latch a single-use barrier.

In *HPX*, a latch is implemented as a counting semaphore, which can be initialized with a specific count value and decremented each time a thread reaches the latch. When the count value reaches zero, all waiting threads are unblocked and allowed to continue execution. The code below shows how latch can be used to synchronize 16 threads:

```
std::ptrdiff_t num_threads = 16;

///////////////////////////////////////////////////////////////////////////////
void wait_for_latch(hpx::latch& l)
{
    l.arrive_and_wait();
}

///////////////////////////////////////////////////////////////////////////////
int hpx_main(hpx::program_options::variables_map& vm)
{
    num_threads = vm["num-threads"].as<std::ptrdiff_t>();

    hpx::latch l(num_threads + 1);

    std::vector<hpx::future<void>> results;
    for (std::ptrdiff_t i = 0; i != num_threads; ++i)
        results.push_back(hpx::async(&wait_for_latch, std::ref(l)));

    // Wait for all threads to reach this point.
    l.arrive_and_wait();

    hpx::wait_all(results);

    return hpx::local::finalize();
```

(continues on next page)

(continued from previous page)

```
}
```

In the above code, the `hpx_main` function creates a latch object `l` with a count of `num_threads + 1` and `num_threads` number of threads using `hpx::async`. These threads call the `wait_for_latch` function and pass the reference to the latch object. In the `wait_for_latch` function, the thread calls the `arrive_and_wait` method on the latch, which decrements the count of the latch and causes the thread to wait until the count reaches zero. Finally, the main thread waits for all the threads to arrive at the latch by calling the `arrive_and_wait` method and then waits for all the threads to finish by calling the `hpx::wait_all` method.

Mutex

A *mutex* (short for "mutual exclusion") is a synchronization primitive in *HPX* used to control access to a shared resource, ensuring that only one thread can access it at a time. A mutex is used to protect data structures from race conditions and other synchronization-related issues. When a thread acquires a mutex, other threads that try to access the same resource will be blocked until the mutex is released. The code below shows the basic use of mutexes:

```
#include <hpx/future.hpp>
#include <hpx/init.hpp>
#include <hpx/mutex.hpp>

#include <iostream>

int hpx_main()
{
    hpx::mutex m;

    hpx::future<void> f1 = hpx::async([&m]() {
        std::scoped_lock sl(m);
        std::cout << "Thread 1 acquired the mutex" << std::endl;
    });

    hpx::future<void> f2 = hpx::async([&m]() {
        std::scoped_lock sl(m);
        std::cout << "Thread 2 acquired the mutex" << std::endl;
    });

    hpx::wait_all(f1, f2);

    return hpx::local::finalize();
}

int main(int argc, char* argv[])
{
    return hpx::local::init(hpx_main, argc, argv);
}
```

In this example, two *HPX* threads created using `hpx::async` are acquiring a `hpx::mutex m`. `std::scoped_lock sl(m)` is used to take ownership of the given mutex `m`. When control leaves the scope in which the `scoped_lock` object was created, the `scoped_lock` is destructed and the mutex is released.

Attention

A common way to acquire and release mutexes is by using the function `m.lock()` before accessing the shared resource, and `m.unlock()` called after the access is complete. However, these functions may lead to deadlocks in case of exception(s). That is, if an exception happens when the mutex is locked then the code that unlocks the mutex will never be executed, the lock will remain held by the thread that acquired it, and other threads will be unable to access the shared resource. This can cause a deadlock if the other threads are also waiting to acquire the same lock. For this reason, we suggest you use `std::scoped_lock`, which prevents this issue by releasing the lock when control leaves the scope in which the `scoped_lock` object was created.

Shared mutex

A *shared mutex* is a synchronization primitive that can be used to protect shared data from being simultaneously accessed by multiple threads. In contrast to other mutex types which facilitate exclusive access, a `shared_mutex` has two levels of access:

- *Exclusive access* prevents any other thread from acquiring the mutex, just as with the normal mutex. It does not matter if the other thread tries to acquire shared or exclusive access.
- *Shared access* allows multiple threads to acquire the mutex, but all of them only in shared mode. Exclusive access is not granted until all of the previous shared holders have returned the mutex (typically, as long as an exclusive request is waiting, new shared ones are queued to be granted after the exclusive access).

Shared mutexes are especially useful when shared data can be safely read by any number of threads simultaneously, but a thread may only write the same data when no other thread is reading or writing at the same time. A typical scenario is a database: The data can be read simultaneously by different threads with no problem. However, modification of the database is critical: if some threads read data while another one is writing, the threads reading may receive inconsistent data. Hence, while a thread is writing, reading should not be allowed. After writing is complete, reads can occur simultaneously again. The code below shows how `shared_mutex` can be used to synchronize reads and writes:

```
int const writers = 3;
int const readers = 3;
int const cycles = 10;

using std::chrono::milliseconds;

int hpx_main()
{
    std::vector<hpx::thread> threads;
    std::atomic<bool> ready(false);
    hpx::shared_mutex stm;

    for (int i = 0; i < writers; ++i)
    {
        threads.emplace_back([&ready, &stm, i] {
            std::mt19937 urng(static_cast<std::uint32_t>(std::time(nullptr)));
            std::uniform_int_distribution<int> dist(1, 1000);

            while (!ready)
            { /*** wait... ***/
            }

            for (int j = 0; j < cycles; ++j)
```

(continues on next page)

(continued from previous page)

```
            {
                // scope of unique_lock
                {
                    std::unique_lock<hpx::shared_mutex> ul(stm);

                    std::cout << "^^^ Writer " << i << " starting..."
                              << std::endl;
                    hpx::this_thread::sleep_for(milliseconds(dist(urng)));
                    std::cout << "vvv Writer " << i << " finished."
                              << std::endl;
                }

                hpx::this_thread::sleep_for(milliseconds(dist(urng)));
            }
        });
    }

    for (int i = 0; i < readers; ++i)
    {
        int k = writers + i;
        threads.emplace_back([&ready, &stm, k, i] {
            HPX_UNUSED(k);
            std::mt19937 urng(static_cast<std::uint32_t>(std::time(nullptr)));
            std::uniform_int_distribution<int> dist(1, 1000);

            while (!ready)
            { /*** wait... ***/
            }

            for (int j = 0; j < cycles; ++j)
            {
                // scope of shared_lock
                {
                    std::shared_lock<hpx::shared_mutex> sl(stm);

                    std::cout << "Reader " << i << " starting..." << std::endl;
                    hpx::this_thread::sleep_for(milliseconds(dist(urng)));
                    std::cout << "Reader " << i << " finished." << std::endl;
                }
                hpx::this_thread::sleep_for(milliseconds(dist(urng)));
            }
        });
    }

    ready = true;
    for (auto& t : threads)
        t.join();

    return hpx::local::finalize();
}
```

The above code creates `writers` and `readers` threads, each of which will perform `cycles` of operations. Both the writer and reader threads use the `hpx::shared_mutex` object `stm` to synchronize access to a shared resource.

- For the writer threads, a `unique_lock` on the shared mutex is acquired before each write operation and is released after control leaves the scope in which the `unique_lock` object was created.
- For the reader threads, a `shared_lock` on the shared mutex is acquired before each read operation and is released after control leaves the scope in which the `shared_lock` object was created.

Before each operation, both the reader and writer threads sleep for a random time period, which is generated using a random number generator. The random time period simulates the processing time of the operation.

Semaphore

Semaphores are a synchronization mechanism used to control concurrent access to a shared resource. The two types of semaphores are:

- counting semaphore: it has a counter that is bigger than zero. The counter is initialized in the constructor. Acquiring the semaphore decreases the counter and releasing the semaphore increases the counter. If a thread tries to acquire the semaphore when the counter is zero, the thread will block until another thread increments the counter by releasing the semaphore. Unlike `hpx::mutex`, an `hpx::counting_semaphore` is not bound to a thread, which means that the acquire and release call of a semaphore can happen on different threads.
- binary semaphore: it is an alias for a `hpx::counting_semaphore<1>`. In this case, the least maximal value is 1. `hpx::binary_semaphore` can be used to implement locks.

```cpp
#include <hpx/init.hpp>
#include <hpx/semaphore.hpp>
#include <hpx/thread.hpp>

#include <iostream>

// initialize the semaphore with a count of 3
hpx::counting_semaphore<> semaphore(3);

void worker()
{
    semaphore.acquire();    // decrement the semaphore's count
    std::cout << "Entering critical section" << std::endl;
    hpx::this_thread::sleep_for(std::chrono::seconds(1));
    semaphore.release();    // increment the semaphore's count
    std::cout << "Exiting critical section" << std::endl;
}

int hpx_main()
{
    hpx::thread t1(worker);
    hpx::thread t2(worker);
    hpx::thread t3(worker);
    hpx::thread t4(worker);
    hpx::thread t5(worker);

    t1.join();
    t2.join();
    t3.join();
    t4.join();
    t5.join();
```

(continues on next page)

(continued from previous page)

```
    return hpx::local::finalize();
}

int main(int argc, char* argv[])
{
    return hpx::local::init(hpx_main, argc, argv);
}
```

In this example, the counting semaphore is initialized to the value of 3. This means that up to 3 threads can access the critical section (the section of code inside `the worker()` function) at the same time. When a thread enters the critical section, it acquires the semaphore, which decrements the count, while when it exits the critical section, it releases the semaphore, incrementing thus the count. The `worker()` function simulates a critical section by acquiring the semaphore, sleeping for 1 second and then releasing the semaphore.

In the main function, 5 worker threads are created and started, each trying to enter the critical section. If the count of the semaphore is already 0, a worker will wait until another worker releases the semaphore (increasing its value).

Composable guards

Composable guards operate in a manner similar to locks, but are applied only to asynchronous functions. The guard (or guards) is automatically locked at the beginning of a specified task and automatically unlocked at the end. Because guards are never added to an existing task's execution context, the calling of guards is freely composable and can never deadlock.

To call an application with a single guard, simply declare the guard and call `run_guarded()` with a function `(task)`:

```
hpx::lcos::local::guard gu;
run_guarded(gu,task);
```

If a single method needs to run with multiple guards, use a guard set:

```
std::shared_ptr<hpx::lcos::local::guard> gu1(new hpx::lcos::local::guard());
std::shared_ptr<hpx::lcos::local::guard> gu2(new hpx::lcos::local::guard());
gs.add(*gu1);
gs.add(*gu2);
run_guarded(gs,task);
```

Guards use two atomic operations (which are not called repeatedly) to manage what they do, so overhead should be extremely low.

Execution control

The following objects are providing control of the execution in *HPX* applications:

1. *Futures*
2. *Channels*
3. *Task blocks*
4. *Task groups*
5. *Threads*

Futures

Futures are a mechanism to represent the result of a potentially asynchronous operation. A future is a type that represents a value that will become available at some point in the future, and it can be used to write asynchronous and parallel code. Futures can be returned from functions that perform time-consuming operations, allowing the calling code to continue executing while the function performs its work. The value of the future is set when the operation completes and can be accessed later. Futures are used in *HPX* to write asynchronous and parallel code. Below is an example demonstrating different features of futures:

```
#include <hpx/assert.hpp>
#include <hpx/future.hpp>
#include <hpx/hpx_main.hpp>
#include <hpx/tuple.hpp>

#include <iostream>
#include <utility>

int main()
{
    // Asynchronous execution with futures
    hpx::future<void> f1 = hpx::async(hpx::launch::async, []() {});
    hpx::shared_future<int> f2 =
        hpx::async(hpx::launch::async, []() { return 42; });
    hpx::future<int> f3 =
        f2.then([](hpx::shared_future<int>&& f) { return f.get() * 3; });

    hpx::promise<double> p;
    auto f4 = p.get_future();
    HPX_ASSERT(!f4.is_ready());
    p.set_value(123.45);
    HPX_ASSERT(f4.is_ready());

    hpx::packaged_task<int()> t([]() { return 43; });
    hpx::future<int> f5 = t.get_future();
    HPX_ASSERT(!f5.is_ready());
    t();
    HPX_ASSERT(f5.is_ready());

    // Fire-and-forget
    hpx::post([]() {
        std::cout << "This will be printed later\n" << std::flush;
    });

    // Synchronous execution
    hpx::sync([]() {
        std::cout << "This will be printed immediately\n" << std::flush;
    });

    // Combinators
    hpx::future<double> f6 = hpx::async([]() { return 3.14; });
    hpx::future<double> f7 = hpx::async([]() { return 42.0; });
    std::cout
        << hpx::when_all(f6, f7)
```

(continues on next page)

(continued from previous page)

```
              .then([](hpx::future<
                         hpx::tuple<hpx::future<double>, hpx::future<double>>>
                             f) {
                  hpx::tuple<hpx::future<double>, hpx::future<double>> t =
                      f.get();
                  double pi = hpx::get<0>(t).get();
                  double r = hpx::get<1>(t).get();
                  return pi * r * r;
              })
              .get()
          << std::endl;

    // Easier continuations with dataflow; it waits for all future or
    // shared_future arguments before executing the continuation, and also
    // accepts non-future arguments
    hpx::future<double> f8 = hpx::async([]() { return 3.14; });
    hpx::future<double> f9 = hpx::make_ready_future(42.0);
    hpx::shared_future<double> f10 = hpx::async([]() { return 123.45; });
    hpx::future<hpx::tuple<double, double>> f11 = hpx::dataflow(
        [](hpx::future<double> a, hpx::future<double> b,
            hpx::shared_future<double> c, double d) {
            return hpx::make_tuple<>(a.get() + b.get(), c.get() / d);
        },
        f8, f9, f10, -3.9);

    // split_future gives a tuple of futures from a future of tuple
    hpx::tuple<hpx::future<double>, hpx::future<double>> f12 =
        hpx::split_future(std::move(f11));
    std::cout << hpx::get<1>(f12).get() << std::endl;

    return 0;
}
```

The first section of the main function demonstrates how to use futures for asynchronous execution. The first two lines create two futures, one for void and another for an integer, using the `hpx::async()` function. These futures are executed *asynchronously* in separate threads using the `hpx::launch::async` launch policy. The third future is created by *chaining* the second future using the `then()` member function. This future multiplies the result of the second future by 3.

The next part of the code demonstrates how to use *promises* and *packaged* tasks, which are constructs used for communicating data between threads. The `promise` class is used to store a value that can be retrieved *later* using a future. The `packaged_task` class represents a task that can be executed *asynchronously*, and its result can be obtained using a future. The last three lines create a packaged task that returns an integer, obtain its future, execute the task, and check whether the future is ready or not.

The code then demonstrates how to use the `hpx::post()` and `hpx::sync()` functions for *fire-and-forget* and *synchronous* execution, respectively. The `hpx::post()` function executes a given function *asynchronously* and *returns immediately* without waiting for the result. The `hpx::sync()` function executes a given function *synchronously* and *waits* for the result before returning.

Next the code demonstrates the use of *combinators*, which are higher-order functions that combine two or more futures into a single future. The `hpx::when_all()` function is used to combine two futures, which return double values, into a tuple of futures. The `then()` member function is then used to compute the area of a circle using the values of the two futures. The `get()` member function is used to retrieve the result of the computation.

The last section demonstrates the use of `hpx::dataflow()`, which is a higher-order function that waits for all the future or shared_future arguments to be ready before executing the continuation. The `hpx::make_ready_future()` function is used to create a future with a given value. The `hpx::split_future()` function is used to split a future of a tuple into a tuple of futures. The last line retrieves the value of the second future in the tuple using `hpx::get()` and prints it to the console.

Extended facilities for futures

Concurrency is about both decomposing and composing the program from the parts that work well individually and together. It is in the composition of connected and multicore components where today's C++ libraries are still lacking.

The functionality of std::future[77] offers a partial solution. It allows for the separation of the initiation of an operation and the act of waiting for its result; however, the act of waiting is synchronous. In communication-intensive code this act of waiting can be unpredictable, inefficient and simply frustrating. The example below illustrates a possible synchronous wait using futures:

```
#include <future>
using namespace std;
int main()
{
    future<int> f = async([]() { return 123; });
    int result = f.get(); // might block
}
```

For this reason, *HPX* implements a set of extensions to std::future[78] (as proposed by N4313[79]). This proposal introduces the following key asynchronous operations to *hpx::future*, *hpx::shared_future* and *hpx::async*, which enhance and enrich these facilities.

[77] http://en.cppreference.com/w/cpp/thread/future
[78] http://en.cppreference.com/w/cpp/thread/future
[79] http://wg21.link/n4313

Table 2.11: Facilities extending `std::future`

Facility	Description
`hpx::future::then`	In asynchronous programming, it is very common for one asynchronous operation, on completion, to invoke a second operation and pass data to it. The current C++ standard does not allow one to register a continuation to a future. With `then`, instead of waiting for the result, a continuation is "attached" to the asynchronous operation, which is invoked when the result is ready. Continuations registered using then function will help to avoid blocking waits or wasting threads on polling, greatly improving the responsiveness and scalability of an application.
unwrapping constructor for *hpx::future*	In some scenarios, you might want to create a future that returns another future, resulting in nested futures. Although it is possible to write code to unwrap the outer future and retrieve the nested future and its result, such code is not easy to write because users must handle exceptions and it may cause a blocking call. Unwrapping can allow users to mitigate this problem by doing an asynchronous call to unwrap the outermost future.
`hpx::future::is_r`	There are often situations where a `get()` call on a future may not be a blocking call, or is only a blocking call under certain circumstances. This function gives the ability to test for early completion and allows us to avoid associating a continuation, which needs to be scheduled with some non-trivial overhead and near-certain loss of cache efficiency.
hpx::make_ready_futu	Some functions may know the value at the point of construction. In these cases the value is immediately available, but needs to be returned as a future. By using `hpx::make_ready_future` a future can be created that holds a pre-computed result in its shared state. In the current standard it is non-trivial to create a future directly from a value. First a promise must be created, then the promise is set, and lastly the future is retrieved from the promise. This can now be done with one operation.

The standard also omits the ability to compose multiple futures. This is a common pattern that is ubiquitous in other asynchronous frameworks and is absolutely necessary in order to make C++ a powerful asynchronous programming language. Not including these functions is synonymous to Boolean algebra without AND/OR.

In addition to the extensions proposed by N4313[80], *HPX* adds functions allowing users to compose several futures in a more flexible way.

Table 2.12: Facilities for composing `hpx::futures`

Facility	Description
hpx::when_any, `hpx::when_any_n`	Asynchronously wait for at least one of multiple future or shared_future objects to finish.
hpx::wait_any, *hpx::wait_any_n*	Synchronously wait for at least one of multiple future or shared_future objects to finish.
hpx::when_all, `hpx::when_all_n`	Asynchronously wait for all future and shared_future objects to finish.
hpx::wait_all, *hpx::wait_all_n*	Synchronously wait for all future and shared_future objects to finish.
hpx::when_some, `hpx::when_some_n`	Asynchronously wait for multiple future and shared_future objects to finish.
hpx::wait_some, *hpx::wait_some_n*	Synchronously wait for multiple future and shared_future objects to finish.
hpx::when_each	Asynchronously wait for multiple future and shared_future objects to finish and call a function for each of the future objects as soon as it becomes ready.
hpx::wait_each, *hpx::wait_each_n*	Synchronously wait for multiple future and shared_future objects to finish and call a function for each of the future objects as soon as it becomes ready.

80 http://wg21.link/n4313

Channels

Channels combine communication (the exchange of a value) with synchronization (guaranteeing that two calculations (tasks) are in a known state). A channel can transport any number of values of a given type from a sender to a receiver:

```
    hpx::lcos::local::channel<int> c;
    hpx::future<int> f = c.get();
    HPX_ASSERT(!f.is_ready());
    c.set(42);
    HPX_ASSERT(f.is_ready());
    std::cout << f.get() << std::endl;
```

Channels can be handed to another thread (or in case of channel components, to other localities), thus establishing a communication channel between two independent places in the program:

```
void do_something(hpx::lcos::local::receive_channel<int> c,
    hpx::lcos::local::send_channel<> done)
{
    // prints 43
    std::cout << c.get(hpx::launch::sync) << std::endl;
    // signal back
    done.set();
}

void send_receive_channel()
{
    hpx::lcos::local::channel<int> c;
    hpx::lcos::local::channel<> done;

    hpx::post(&do_something, c, done);

    // send some value
    c.set(43);
    // wait for thread to be done
    done.get().wait();
}
```

Note how `hpx::lcos::local::channel::get` without any arguments returns a future which is ready when a value has been set on the channel. The launch policy `hpx::launch::sync` can be used to make `hpx::lcos::local::channel::get` block until a value is set and return the value directly.

A channel component is created on one *locality* and can be sent to another *locality* using an action. This example also demonstrates how a channel can be used as a range of values:

```
// channel components need to be registered for each used type (not needed
// for hpx::lcos::local::channel)
HPX_REGISTER_CHANNEL(double)

void channel_sender(hpx::lcos::channel<double> c)
{
    for (double d : c)
        hpx::cout << d << std::endl;
}
HPX_PLAIN_ACTION(channel_sender)
```

(continues on next page)

(continued from previous page)

```
void channel()
{
    // create the channel on this locality
    hpx::lcos::channel<double> c(hpx::find_here());

    // pass the channel to a (possibly remote invoked) action
    hpx::post(channel_sender_action(), hpx::find_here(), c);

    // send some values to the receiver
    std::vector<double> v = {1.2, 3.4, 5.0};
    for (double d : v)
        c.set(d);

    // explicitly close the communication channel (implicit at destruction)
    c.close();
}
```

Task blocks

Task blocks in *HPX* provide a way to structure and organize the execution of tasks in a parallel program, making it easier to manage dependencies between tasks. A task block actually is a group of tasks that can be executed in parallel. Tasks in a task block can depend on other tasks in the same task block. The task block allows the runtime to optimize the execution of tasks, by scheduling them in an optimal order based on the dependencies between them.

The `define_task_block`, `run` and the `wait` functions implemented based on N4755[81] are based on the `task_block` concept that is a part of the common subset of the Microsoft Parallel Patterns Library (PPL)[82] and the Intel Threading Building Blocks (TBB)[83] libraries.

These implementations adopt a simpler syntax than exposed by those libraries— one that is influenced by language-based concepts, such as spawn and sync from Cilk++[84] and async and finish from X10[85]. They improve on existing practice in the following ways:

- The exception handling model is simplified and more consistent with normal C++ exceptions.
- Most violations of strict fork-join parallelism can be enforced at compile time (with compiler assistance, in some cases).
- The syntax allows scheduling approaches other than child stealing.

Consider an example of a parallel traversal of a tree, where a user-provided function compute is applied to each node of the tree, returning the sum of the results:

```
template <typename Func>
int traverse(node& n, Func && compute)
{
    int left = 0, right = 0;
    define_task_block(
        [&](task_block<>& tr) {
```

(continues on next page)

[81] http://wg21.link/n4755
[82] https://msdn.microsoft.com/en-us/library/dd492418.aspx
[83] https://www.threadingbuildingblocks.org/
[84] https://software.intel.com/en-us/articles/intel-cilk-plus/
[85] https://x10-lang.org/

(continued from previous page)

```
            if (n.left)
                tr.run([&] { left = traverse(*n.left, compute); });
            if (n.right)
                tr.run([&] { right = traverse(*n.right, compute); });
        });

    return compute(n) + left + right;
}
```

The example above demonstrates the use of two of the functions, *hpx::experimental::define_task_block* and the `hpx::experimental::task_block::run` member function of a *hpx::experimental::task_block*.

The `task_block` function delineates a region in a program code potentially containing invocations of threads spawned by the `run` member function of the `task_block` class. The `run` function spawns an *HPX* thread, a unit of work that is allowed to execute in parallel with respect to the caller. Any parallel tasks spawned by `run` within the task block are joined back to a single thread of execution at the end of the `define_task_block`. `run` takes a user-provided function object `f` and starts it asynchronously—i.e., it may return before the execution of `f` completes. The *HPX* scheduler may choose to run `f` immediately or delay running `f` until compute resources become available.

A `task_block` can be constructed only by `define_task_block` because it has no public constructors. Thus, `run` can be invoked directly or indirectly only from a user-provided function passed to `define_task_block`:

```
void g();

void f(task_block<>& tr)
{
    tr.run(g);          // OK, invoked from within task_block in h
}

void h()
{
    define_task_block(f);
}

int main()
{
    task_block<> tr;    // Error: no public constructor
    tr.run(g);          // No way to call run outside of a define_task_block
    return 0;
}
```

Extensions for task blocks

Using execution policies with task blocks

HPX implements some extensions for `task_block` beyond the actual standards proposal N4755[86]. The main addition is that a `task_block` can be invoked with an execution policy as its first argument, very similar to the parallel algorithms.

An execution policy is an object that expresses the requirements on the ordering of functions invoked as a consequence of the invocation of a task block. Enabling passing an execution policy to `define_task_block` gives the user control

[86] http://wg21.link/n4755

over the amount of parallelism employed by the created `task_block`. In the following example the use of an explicit `par` execution policy makes the user's intent explicit:

```
template <typename Func>
int traverse(node *n, Func&& compute)
{
    int left = 0, right = 0;

    define_task_block(
        execution::par,                // execution::parallel_policy
        [&](task_block<>& tb) {
            if (n->left)
                tb.run([&] { left = traverse(n->left, compute); });
            if (n->right)
                tb.run([&] { right = traverse(n->right, compute); });
        });

    return compute(n) + left + right;
}
```

This also causes the *hpx::experimental::task_block* object to be a template in our implementation. The template argument is the type of the execution policy used to create the task block. The template argument defaults to *hpx::execution::parallel_policy*.

HPX still supports calling *hpx::experimental::define_task_block* without an explicit execution policy. In this case the task block will run using the *hpx::execution::parallel_policy*.

HPX also adds the ability to access the execution policy that was used to create a given `task_block`.

Using executors to run tasks

Often, users want to be able to not only define an execution policy to use by default for all spawned tasks inside the task block, but also to customize the execution context for one of the tasks executed by `task_block::run`. Adding an optionally passed executor instance to that function enables this use case:

```
template <typename Func>
int traverse(node *n, Func&& compute)
{
    int left = 0, right = 0;

    define_task_block(
        execution::par,                // execution::parallel_policy
        [&](auto& tb) {
            if (n->left)
            {
                // use explicitly specified executor to run this task
                tb.run(my_executor(), [&] { left = traverse(n->left, compute); });
            }
            if (n->right)
            {
                // use the executor associated with the par execution policy
                tb.run([&] { right = traverse(n->right, compute); });
            }
        });
```

(continues on next page)

(continued from previous page)

```
    return compute(n) + left + right;
}
```

HPX still supports calling `hpx::experimental::task_block::run` without an explicit executor object. In this case the task will be run using the executor associated with the execution policy that was used to call *hpx::experimental::define_task_block*.

Task groups

A *task group* in *HPX* is a synchronization primitive that allows you to execute a group of tasks concurrently and wait for their completion before continuing. The tasks in an `hpx::experimental::task_group` can be added dynamically. This is the *HPX* implementation of *tbb::task_group* of the Intel Threading Building Blocks (TBB)[87] library.

The example below shows that to use a task group, you simply create an `hpx::task_group` object and add tasks to it using the `run()` method. Once all the tasks have been added, you can call the `wait()` method to synchronize the tasks and wait for them to complete.

```
#include <hpx/experimental/task_group.hpp>
#include <hpx/init.hpp>

#include <iostream>

void task1()
{
    std::cout << "Task 1 executed." << std::endl;
}

void task2()
{
    std::cout << "Task 2 executed." << std::endl;
}

int hpx_main()
{
    hpx::experimental::task_group tg;

    tg.run(task1);
    tg.run(task2);

    tg.wait();

    std::cout << "All tasks finished!" << std::endl;

    return hpx::local::finalize();
}

int main(int argc, char* argv[])
{
    return hpx::local::init(hpx_main, argc, argv);
}
```

[87] https://www.threadingbuildingblocks.org/

Note

task groups and *task blocks* are both ways to group and synchronize parallel tasks, but *task groups* are used to group multiple tasks together as a single unit, while *task blocks* are used to execute a loop in parallel, with each iteration of the loop executing in a separate task. If the difference is not clear yet, continue reading.

A *task group* is a construct that allows multiple parallel tasks to be grouped together as a single unit. The task group provides a way to synchronize all the tasks in the group before continuing with the rest of the program.

A *task block*, on the other hand, is a parallel loop construct that allows you to execute a loop in parallel, with each iteration of the loop executing in a separate task. The loop iterations are executed in a block, meaning that the loop body is executed as a single task.

Threads

A thread in *HPX* refers to a sequence of instructions that can be executed concurrently with other such sequences in multithreading environments, while sharing a same address space. These threads can communicate with each other through various means, such as futures or shared data structures.

The example below demonstrates how to launch multiple threads and synchronize them using a `hpx::latch` object. It also shows how to query the state of threads and wait for futures to complete.

```
#include <hpx/future.hpp>
#include <hpx/init.hpp>
#include <hpx/thread.hpp>

#include <functional>
#include <iostream>
#include <vector>

int const num_threads = 10;

///////////////////////////////////////////////////////////////////////////////
void wait_for_latch(hpx::latch& l)
{
    l.arrive_and_wait();
}

int hpx_main()
{
    // Spawn a couple of threads
    hpx::latch l(num_threads + 1);

    std::vector<hpx::future<void>> results;
    results.reserve(num_threads);

    for (int i = 0; i != num_threads; ++i)
        results.push_back(hpx::async(&wait_for_latch, std::ref(l)));

    // Allow spawned threads to reach latch
    hpx::this_thread::yield();

    // Enumerate all suspended threads
```

(continues on next page)

(continued from previous page)

```
    hpx::threads::enumerate_threads(
        [](hpx::threads::thread_id_type id) -> bool {
            std::cout << "thread " << hpx::thread::id(id) << " is "
                      << hpx::threads::get_thread_state_name(
                             hpx::threads::get_thread_state(id))
                      << std::endl;
            return true;    // always continue enumeration
        },
        hpx::threads::thread_schedule_state::suspended);

    // Wait for all threads to reach this point.
    l.arrive_and_wait();

    hpx::wait_all(results);

    return hpx::local::finalize();
}

int main(int argc, char* argv[])
{
    return hpx::local::init(hpx_main, argc, argv);
}
```

In more detail, the `wait_for_latch()` function is a simple helper function that waits for a `hpx::latch` object to be released. At this point we remind that `hpx::latch` is a synchronization primitive that allows multiple threads to wait for a common event to occur.

In the `hpx_main()` function, an `hpx::latch` object is created with a count of `num_threads + 1`, indicating that `num_threads` threads need to arrive at the latch before the latch is released. The loop that follows launches `num_threads` asynchronous operations, each of which calls the `wait_for_latch` function. The resulting futures are added to the vector.

After the threads have been launched, `hpx::this_thread::yield()` is called to give them a chance to reach the latch before the program proceeds. Then, the `hpx::threads::enumerate_threads` function prints the state of each suspended thread, while the next call of `l.arrive_and_wait()` waits for all the threads to reach the latch. Finally, `hpx::wait_all` is called to wait for all the futures to complete.

Hint

An advantage of using `hpx::thread` over other threading libraries is that it is optimized for high-performance parallelism, with support for lightweight threads and task scheduling to minimize thread overhead and maximize parallelism. Additionally, `hpx::thread` integrates seamlessly with other features of *HPX* such as futures, promises, and task groups, making it a powerful tool for parallel programming.

Checkout the examples of *Shared mutex*, *Condition variable*, *Semaphore* to see how *HPX* threads are used in combination with other features.

High level parallel facilities

In preparation for the upcoming C++ Standards, there are currently several proposals targeting different facilities supporting parallel programming. *HPX* implements (and extends) some of those proposals. This is well aligned with our strategy to align the APIs exposed from *HPX* with current and future C++ Standards.

At this point, *HPX* implements several of the C++ Standardization working papers, most notably N4409[88] (Working Draft, Technical Specification for C++ Extensions for Parallelism), N4755[89] (Task Blocks), and N4406[90] (Parallel Algorithms Need Executors).

Using parallel algorithms

A parallel algorithm is a function template declared in the namespace `hpx::parallel`.

All parallel algorithms are very similar in semantics to their sequential counterparts (as defined in the `namespace std`) with an additional formal template parameter named `ExecutionPolicy`. The execution policy is generally passed as the first argument to any of the parallel algorithms and describes the manner in which the execution of these algorithms may be parallelized and the manner in which they apply user-provided function objects.

The applications of function objects in parallel algorithms invoked with an execution policy object of type *hpx::execution::sequenced_policy* or *hpx::execution::sequenced_task_policy* execute in sequential order. For *hpx::execution::sequenced_policy* the execution happens in the calling thread.

The applications of function objects in parallel algorithms invoked with an execution policy object of type *hpx::execution::parallel_policy* or *hpx::execution::parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and are indeterminately sequenced within each thread.

Important

It is the caller's responsibility to ensure correctness, such as making sure that the invocation does not introduce data races or deadlocks.

The example below demonstrates how to perform a sequential and parallel *hpx::for_each* loop on a vector of integers.

```
#include <hpx/algorithm.hpp>
#include <hpx/execution.hpp>
#include <hpx/init.hpp>

#include <iostream>
#include <vector>

int hpx_main()
{
    std::vector<int> v{1, 2, 3, 4, 5};

    auto print = [](int const& n) { std::cout << n << ' '; };

    std::cout << "Print sequential: ";
    hpx::for_each(v.begin(), v.end(), print);
    std::cout << '\n';
```

(continues on next page)

[88] http://wg21.link/n4409
[89] http://wg21.link/n4755
[90] http://wg21.link/n4406

(continued from previous page)

```
    std::cout << "Print parallel: ";
    hpx::for_each(hpx::execution::par, v.begin(), v.end(), print);
    std::cout << '\n';

    return hpx::local::finalize();
}

int main(int argc, char* argv[])
{
    return hpx::local::init(hpx_main, argc, argv);
}
```

The above code uses `hpx::for_each` to print the elements of the vector `v{1, 2, 3, 4, 5}`. At first, `hpx::for_each()` is called without an execution policy, which means that it applies the lambda function `print` to each element in the vector sequentially. Hence, the elements are printed in order.

Next, `hpx::for_each()` is called with the `hpx::execution::par` execution policy, which applies the lambda function `print` to each element in the vector in parallel. Therefore, the output order of the elements in the vector is not deterministic and may vary from run to run.

Parallel exceptions

During the execution of a standard parallel algorithm, if temporary memory resources are required by any of the algorithms and no memory is available, the algorithm throws a `std::bad_alloc` exception.

During the execution of any of the parallel algorithms, if the application of a function object terminates with an uncaught exception, the behavior of the program is determined by the type of execution policy used to invoke the algorithm:

- If the execution policy object is of type *hpx::execution::parallel_unsequenced_policy*, `hpx::terminate` shall be called.
- If the execution policy object is of type *hpx::execution::sequenced_policy*, *hpx::execution::sequenced_task_policy*, *hpx::execution::parallel_policy*, or *hpx::execution::parallel_task_policy*, the execution of the algorithm terminates with an *`hpx::exception_list`* exception. All uncaught exceptions thrown during the application of user-provided function objects shall be contained in the *`hpx::exception_list`*.

For example, the number of invocations of the user-provided function object in for_each is unspecified. When *hpx::for_each* is executed sequentially, only one exception will be contained in the *`hpx::exception_list`* object.

These guarantees imply that, unless the algorithm has failed to allocate memory and terminated with `std::bad_alloc`, all exceptions thrown during the execution of the algorithm are communicated to the caller. It is unspecified whether an algorithm implementation will "forge ahead" after encountering and capturing a user exception.

The algorithm may terminate with the `std::bad_alloc` exception even if one or more user-provided function objects have terminated with an exception. For example, this can happen when an algorithm fails to allocate memory while creating or adding elements to the *`hpx::exception_list`* object.

Parallel algorithms

HPX provides implementations of the following parallel algorithms:

Table 2.13: Non-modifying parallel algorithms of header hpx/algorithm.hpp

Name	Description	C++ standard
hpx::adjacent_find	Computes the differences between adjacent elements in a range.	adjacent_find[91]
hpx::all_of	Checks if a predicate is `true` for all of the elements in a range.	all_any_none_of[92]
hpx::any_of	Checks if a predicate is `true` for any of the elements in a range.	all_any_none_of[93]
hpx::count	Returns the number of elements equal to a given value.	count[94]
hpx::count_if	Returns the number of elements satisfying a specific criteria.	count_if[95]
hpx::equal	Determines if two sets of elements are the same.	equal[96]
hpx::find	Finds the first element equal to a given value.	find[97]
hpx::find_end	Finds the last sequence of elements in a certain range.	find_end[98]
hpx::find_first_of	Searches for any one of a set of elements.	find_first_of[99]
hpx::find_if	Finds the first element satisfying a specific criteria.	find_if[100]
hpx::find_if_not	Finds the first element not satisfying a specific criteria.	find_if_not[101]
hpx::for_each	Applies a function to a range of elements.	for_each[102]
hpx::for_each_n	Applies a function to a number of elements.	for_each_n[103]
hpx::lexicographical_compa	Checks if a range of values is lexicographically less than another range of values.	lexicographical_compare[104]
hpx::mismatch	Finds the first position where two ranges differ.	mismatch[105]
hpx::none_of	Checks if a predicate is `true` for none of the elements in a range.	all_any_none_of[106]
hpx::search	Searches for a range of elements.	search[107]
hpx::search_n	Searches for a number consecutive copies of an element in a range.	search_n[108]

[91] http://en.cppreference.com/w/cpp/algorithm/adjacent_find
[92] http://en.cppreference.com/w/cpp/algorithm/all_any_none_of
[93] http://en.cppreference.com/w/cpp/algorithm/all_any_none_of
[94] http://en.cppreference.com/w/cpp/algorithm/count
[95] http://en.cppreference.com/w/cpp/algorithm/count_if
[96] http://en.cppreference.com/w/cpp/algorithm/equal
[97] http://en.cppreference.com/w/cpp/algorithm/find
[98] http://en.cppreference.com/w/cpp/algorithm/find_end
[99] http://en.cppreference.com/w/cpp/algorithm/find_first_of
[100] http://en.cppreference.com/w/cpp/algorithm/find_if
[101] http://en.cppreference.com/w/cpp/algorithm/find_if_not
[102] http://en.cppreference.com/w/cpp/algorithm/for_each
[103] http://en.cppreference.com/w/cpp/algorithm/for_each_n
[104] http://en.cppreference.com/w/cpp/algorithm/lexicographical_compare
[105] http://en.cppreference.com/w/cpp/algorithm/mismatch
[106] http://en.cppreference.com/w/cpp/algorithm/all_any_none_of
[107] http://en.cppreference.com/w/cpp/algorithm/search
[108] http://en.cppreference.com/w/cpp/algorithm/search_n

Table 2.14: Modifying parallel algorithms of header hpx/algorithm.hpp

Name	Description	C++ standard
hpx::copy	Copies a range of elements to a new location.	exclusive_scan[109]
hpx::copy_n	Copies a number of elements to a new location.	copy_n[110]
hpx::copy_if	Copies the elements from a range to a new location for which the given predicate is `true`	copy[111]
hpx::move	Moves a range of elements to a new location.	move[112]
hpx::fill	Assigns a range of elements a certain value.	fill[113]
hpx::fill_n	Assigns a value to a number of elements.	fill_n[114]
hpx::generate	Saves the result of a function in a range.	generate[115]
hpx::generate_n	Saves the result of N applications of a function.	generate_n[116]
`hpx::experimental::re`	Performs an inclusive scan on consecutive elements with matching keys, with a reduction to output only the final sum for each key. The key sequence `{1,1,1,2,3,3,3,3,1}` and value sequence `{2,3,4,5,6,7,8,9,10}` would be reduced to `keys={1,2,3,1}`, `values={9,5,30,10}`.	
hpx::remove	Removes the elements from a range that are equal to the given value.	remove[117]
hpx::remove_if	Removes the elements from a range that are equal to the given predicate is `false`	remove[118]
hpx::remove_copy	Copies the elements from a range to a new location that are not equal to the given value.	remove_copy[119]
hpx::remove_copy_if	Copies the elements from a range to a new location for which the given predicate is `false`	remove_copy[120]
hpx::replace	Replaces all values satisfying specific criteria with another value.	replace[121]
hpx::replace_if	Replaces all values satisfying specific criteria with another value.	replace[122]
hpx::replace_copy	Copies a range, replacing elements satisfying specific criteria with another value.	replace_copy[123]
hpx::replace_copy_if	Copies a range, replacing elements satisfying specific criteria with another value.	replace_copy[124]
hpx::reverse	Reverses the order elements in a range.	reverse[125]
hpx::reverse_copy	Creates a copy of a range that is reversed.	reverse_copy[126]
hpx::rotate	Rotates the order of elements in a range.	rotate[127]
hpx::rotate_copy	Copies and rotates a range of elements.	rotate_copy[128]
hpx::shift_left	Shifts the elements in the range left by n positions.	shift_left[129]
hpx::shift_right	Shifts the elements in the range right by n positions.	shift_right[130]
hpx::swap_ranges	Swaps two ranges of elements.	swap_ranges[131]
hpx::transform	Applies a function to a range of elements.	transform[132]
hpx::unique	Eliminates all but the first element from every consecutive group of equivalent elements from a range.	unique[133]
hpx::unique_copy	Copies the elements from one range to another in such a way that there are no consecutive equal elements.	unique_copy[134]

Table 2.15: Set operations on sorted sequences of header hpx/algorithm.hpp

Name	Description	C++ standard
hpx::merge	Merges two sorted ranges.	merge[135]
hpx::inplace_merge	Merges two ordered ranges in-place.	inplace_merge[136]
hpx::includes	Returns true if one set is a subset of another.	includes[137]
hpx::set_difference	Computes the difference between two sets.	set_difference[138]
hpx::set_intersection	Computes the intersection of two sets.	set_intersection[139]
hpx::set_symmetric_differer	Computes the symmetric difference between two sets.	set_symmetric_difference[140]
hpx::set_union	Computes the union of two sets.	set_union[141]

Table 2.16: Heap operations of header hpx/algorithm.hpp

Name	Description	C++ standard
hpx::is_heap	Returns `true` if the range is max heap.	is_heap[142]
hpx::is_heap_until	Returns the first element that breaks a max heap.	is_heap_until[143]
hpx::make_heap	Constructs a max heap in the range [first, last).	make_heap[144]

[109] http://en.cppreference.com/w/cpp/algorithm/exclusive_scan
[110] http://en.cppreference.com/w/cpp/algorithm/copy_n
[111] http://en.cppreference.com/w/cpp/algorithm/copy
[112] http://en.cppreference.com/w/cpp/algorithm/move
[113] http://en.cppreference.com/w/cpp/algorithm/fill
[114] http://en.cppreference.com/w/cpp/algorithm/fill_n
[115] http://en.cppreference.com/w/cpp/algorithm/generate
[116] http://en.cppreference.com/w/cpp/algorithm/generate_n
[117] http://en.cppreference.com/w/cpp/algorithm/remove
[118] http://en.cppreference.com/w/cpp/algorithm/remove
[119] http://en.cppreference.com/w/cpp/algorithm/remove_copy
[120] http://en.cppreference.com/w/cpp/algorithm/remove_copy
[121] http://en.cppreference.com/w/cpp/algorithm/replace
[122] http://en.cppreference.com/w/cpp/algorithm/replace
[123] http://en.cppreference.com/w/cpp/algorithm/replace_copy
[124] http://en.cppreference.com/w/cpp/algorithm/replace_copy
[125] http://en.cppreference.com/w/cpp/algorithm/reverse
[126] http://en.cppreference.com/w/cpp/algorithm/reverse_copy
[127] http://en.cppreference.com/w/cpp/algorithm/rotate
[128] http://en.cppreference.com/w/cpp/algorithm/rotate_copy
[129] http://en.cppreference.com/w/cpp/algorithm/shift_left
[130] http://en.cppreference.com/w/cpp/algorithm/shift_right
[131] http://en.cppreference.com/w/cpp/algorithm/swap_ranges
[132] http://en.cppreference.com/w/cpp/algorithm/transform
[133] http://en.cppreference.com/w/cpp/algorithm/unique
[134] http://en.cppreference.com/w/cpp/algorithm/unique_copy
[135] http://en.cppreference.com/w/cpp/algorithm/merge
[136] http://en.cppreference.com/w/cpp/algorithm/inplace_merge
[137] http://en.cppreference.com/w/cpp/algorithm/includes
[138] http://en.cppreference.com/w/cpp/algorithm/set_difference
[139] http://en.cppreference.com/w/cpp/algorithm/set_intersection
[140] http://en.cppreference.com/w/cpp/algorithm/set_symmetric_difference
[141] http://en.cppreference.com/w/cpp/algorithm/set_union

Table 2.17: Minimum/maximum operations of header hpx/algorithm.hpp

Name	Description	C++ standard
hpx::max_element	Returns the largest element in a range.	max_element[145]
hpx::min_element	Returns the smallest element in a range.	min_element[146]
hpx::minmax_element	Returns the smallest and the largest element in a range.	minmax_element[147]

Table 2.18: Partitioning Operations of header hpx/algorithm.hpp

Name	Description	C++ standard
hpx::nth_element	Partially sorts the given range making sure that it is partitioned by the given element	nth_element[148]
hpx::is_partitioned	Returns `true` if each true element for a predicate precedes the false elements in a range.	is_partitioned[149]
hpx::partition	Divides elements into two groups without preserving their relative order.	partition[150]
hpx::partition_copy	Copies a range dividing the elements into two groups.	partition_copy[151]
hpx::stable_partition	Divides elements into two groups while preserving their relative order.	stable_partition[152]

[142] http://en.cppreference.com/w/cpp/algorithm/is_heap
[143] http://en.cppreference.com/w/cpp/algorithm/is_heap_until
[144] http://en.cppreference.com/w/cpp/algorithm/make_heap
[145] http://en.cppreference.com/w/cpp/algorithm/max_element
[146] http://en.cppreference.com/w/cpp/algorithm/min_element
[147] http://en.cppreference.com/w/cpp/algorithm/minmax_element
[148] http://en.cppreference.com/w/cpp/algorithm/nth_element
[149] http://en.cppreference.com/w/cpp/algorithm/is_partitioned
[150] http://en.cppreference.com/w/cpp/algorithm/partition
[151] http://en.cppreference.com/w/cpp/algorithm/partition_copy
[152] http://en.cppreference.com/w/cpp/algorithm/stable_partition

Table 2.19: Sorting Operations of header hpx/algorithm.hpp

Name	Description	C++ standard
hpx::is_sorted	Returns `true` if each element in a range is sorted.	is_sorted[153]
hpx::is_sorted_until	Returns the first unsorted element.	is_sorted_until[154]
hpx::sort	Sorts the elements in a range.	sort[155]
hpx::stable_sort	Sorts the elements in a range, maintain sequence of equal elements.	stable_sort[156]
hpx::partial_sort	Sorts the first elements in a range.	partial_sort[157]
hpx::partial_sort_copy	Sorts the first elements in a range, storing the result in another range.	partial_sort_copy[158]
*hpx::experimental::sort_by_*	Sorts one range of data using keys supplied in another range.	

Table 2.20: Numeric Parallel Algorithms of header hpx/numeric.hpp

Name	Description	C++ standard
hpx::adjacent_difference	Calculates the difference between each element in an input range and the preceding element.	adjacent_difference[159]
hpx::exclusive_scan	Does an exclusive parallel scan over a range of elements.	exclusive_scan[160]
hpx::inclusive_scan	Does an inclusive parallel scan over a range of elements.	inclusive_scan[161]
hpx::reduce	Sums up a range of elements.	reduce[162]
hpx::transform_exclusive_s	Does an exclusive parallel scan over a range of elements after applying a function.	transform_exclusive_scan[163]
hpx::transform_inclusive_s	Does an inclusive parallel scan over a range of elements after applying a function.	transform_inclusive_scan[164]
hpx::transform_reduce	Sums up a range of elements after applying a function. Also, accumulates the inner products of two input ranges.	transform_reduce[165]

[153] http://en.cppreference.com/w/cpp/algorithm/is_sorted
[154] http://en.cppreference.com/w/cpp/algorithm/is_sorted_until
[155] http://en.cppreference.com/w/cpp/algorithm/sort
[156] http://en.cppreference.com/w/cpp/algorithm/stable_sort
[157] http://en.cppreference.com/w/cpp/algorithm/partial_sort
[158] http://en.cppreference.com/w/cpp/algorithm/partial_sort_copy
[159] http://en.cppreference.com/w/cpp/algorithm/adjacent_difference
[160] http://en.cppreference.com/w/cpp/algorithm/exclusive_scan
[161] http://en.cppreference.com/w/cpp/algorithm/inclusive_scan
[162] http://en.cppreference.com/w/cpp/algorithm/reduce
[163] http://en.cppreference.com/w/cpp/algorithm/transform_exclusive_scan
[164] http://en.cppreference.com/w/cpp/algorithm/transform_inclusive_scan
[165] http://en.cppreference.com/w/cpp/algorithm/transform_reduce

Table 2.21: Dynamic Memory Management of header hpx/memory.hpp

Name	Description	C++ standard
hpx::destroy	Destroys a range of objects.	destroy[166]
hpx::destroy_n	Destroys a range of objects.	destroy_n[167]
hpx::uninitialized_copy	Copies a range of objects to an uninitialized area of memory.	uninitial-ized_copy[168]
hpx::uninitialized_copy_n	Copies a number of objects to an uninitialized area of memory.	uninitial-ized_copy_n[169]
hpx::uninitialized_default_c	Copies a range of objects to an uninitialized area of memory.	uninitial-ized_default_construct[170]
hpx::uninitialized_default_c	Copies a number of objects to an uninitialized area of memory.	uninitial-ized_default_construct_n[171]
hpx::uninitialized_fill	Copies an object to an uninitialized area of memory.	uninitialized_fill[172]
hpx::uninitialized_fill_n	Copies an object to an uninitialized area of memory.	uninitial-ized_fill_n[173]
hpx::uninitialized_move	Moves a range of objects to an uninitialized area of memory.	uninitial-ized_move[174]
hpx::uninitialized_move_n	Moves a number of objects to an uninitialized area of memory.	uninitial-ized_move_n[175]
hpx::uninitialized_value_co	Constructs objects in an uninitialized area of memory.	uninitial-ized_value_construct[176]
hpx::uninitialized_value_co	Constructs objects in an uninitialized area of memory.	uninitial-ized_value_construct_n[177]

Table 2.22: Index-based for-loops of header hpx/algorithm.hpp

Name	Description
hpx::experimental::for_loop	Implements loop functionality over a range specified by integral or iterator bounds.
hpx::experimental::for_loop_strided	Implements loop functionality over a range specified by integral or iterator bounds.
hpx::experimental::for_loop_n	Implements loop functionality over a range specified by integral or iterator bounds.
hpx::experimental::for_loop_n_stride	Implements loop functionality over a range specified by integral or iterator bounds.

[166] http://en.cppreference.com/w/cpp/memory/destroy
[167] http://en.cppreference.com/w/cpp/memory/destroy_n
[168] http://en.cppreference.com/w/cpp/memory/uninitialized_copy
[169] http://en.cppreference.com/w/cpp/memory/uninitialized_copy_n
[170] http://en.cppreference.com/w/cpp/memory/uninitialized_default_construct
[171] http://en.cppreference.com/w/cpp/memory/uninitialized_default_construct_n
[172] http://en.cppreference.com/w/cpp/memory/uninitialized_fill
[173] http://en.cppreference.com/w/cpp/memory/uninitialized_fill_n
[174] http://en.cppreference.com/w/cpp/memory/uninitialized_move
[175] http://en.cppreference.com/w/cpp/memory/uninitialized_move_n
[176] http://en.cppreference.com/w/cpp/memory/uninitialized_value_construct
[177] http://en.cppreference.com/w/cpp/memory/uninitialized_value_construct_n

Executor parameters and executor parameter traits

HPX introduces the notion of execution parameters and execution parameter traits. At this point, the only parameter that can be customized is the size of the chunks of work executed on a single *HPX* thread (such as the number of loop iterations combined to run as a single task).

An executor parameter object is responsible for exposing the calculation of the size of the chunks scheduled. It abstracts the (potentially platform-specific) algorithms of determining those chunk sizes.

The way executor parameters are implemented is aligned with the way executors are implemented. All functionalities of concrete executor parameter types are exposed and accessible through a corresponding customization point, e.g. `get_chunk_size()`.

With `executor_parameter_traits`, clients access all types of executor parameters uniformly, e.g.:

```
std::size_t chunk_size =
    hpx::execution::experimental::get_chunk_size(my_parameter, my_executor,
        num_cores, num_tasks);
```

This call synchronously retrieves the size of a single chunk of loop iterations (or similar) to combine for execution on a single *HPX* thread if the overall number of cores `num_cores` and tasks to schedule is given by `num_tasks`. The lambda function exposes a means of test-probing the execution of a single iteration for performance measurement purposes. The execution parameter type might dynamically determine the execution time of one or more tasks in order to calculate the chunk size; see *hpx::execution::experimental::auto_chunk_size* for an example of this executor parameter type.

Other functions in the interface exist to discover whether an executor parameter type should be invoked once (i.e., it returns a static chunk size; see *hpx::execution::experimental::static_chunk_size*) or whether it should be invoked for each scheduled chunk of work (i.e., it returns a variable chunk size; for an example, see *hpx::execution::experimental::guided_chunk_size*).

Although this interface appears to require executor parameter type authors to implement all different basic operations, none are required. In practice, all operations have sensible defaults. However, some executor parameter types will naturally specialize all operations for maximum efficiency.

HPX implements the following executor parameter types:

- *hpx::execution::experimental::auto_chunk_size*: Loop iterations are divided into pieces and then assigned to threads. The number of loop iterations combined is determined based on measurements of how long the execution of 1% of the overall number of iterations takes. This executor parameter type makes sure that as many loop iterations are combined as necessary to run for the amount of time specified.
- *hpx::execution::experimental::static_chunk_size*: Loop iterations are divided into pieces of a given size and then assigned to threads. If the size is not specified, the iterations are, if possible, evenly divided contiguously among the threads. This executor parameters type is equivalent to OpenMP's STATIC scheduling directive.
- *hpx::execution::experimental::dynamic_chunk_size*: Loop iterations are divided into pieces of a given size and then dynamically scheduled among the cores; when a core finishes one chunk, it is dynamically assigned another. If the size is not specified, the default chunk size is 1. This executor parameter type is equivalent to OpenMP's DYNAMIC scheduling directive.
- *hpx::execution::experimental::guided_chunk_size*: Iterations are dynamically assigned to cores in blocks as cores request them until no blocks remain to be assigned. This is similar to `dynamic_chunk_size` except that the block size decreases each time a number of loop iterations is given to a thread. The size of the initial block is proportional to `number_of_iterations / number_of_cores`. Subsequent blocks are proportional to `number_of_iterations_remaining / number_of_cores`. The optional chunk size parameter defines the minimum block size. The default minimal chunk size is 1. This executor parameter type is equivalent to OpenMP's GUIDED scheduling directive.

2.4.12 Writing distributed applications

This section focuses on the features of *HPX* needed to write distributed applications, namely the *Active Global Address Space* (*AGAS*), remotely executable functions (i.e., *actions*), and distributed objects (i.e., *components*).

Global names

HPX implements an *Active Global Address Space* (*AGAS*) which exposes a single uniform address space spanning all localities an application runs on. AGAS is a fundamental component of the ParalleX execution model. Conceptually, there is no rigid demarcation of local or global memory in AGAS; all available memory is a part of the same address space. AGAS enables named objects to be moved (migrated) across localities without having to change the object's name; i.e., no references to migrated objects have to be ever updated. This feature has significance for dynamic load balancing and in applications where the workflow is highly dynamic, allowing work to be migrated from heavily loaded nodes to less loaded nodes. In addition, immutability of names ensures that AGAS does not have to keep extra indirections ("bread crumbs") when objects move, hence, minimizing complexity of code management for system developers as well as minimizing overheads in maintaining and managing aliases.

The AGAS implementation in *HPX* does not automatically expose every local address to the global address space. It is the responsibility of the programmer to explicitly define which of the objects have to be globally visible and which of the objects are purely local.

In *HPX* global addresses (global names) are represented using the `hpx::id_type` data type. This data type is conceptually very similar to `void*` pointers as it does not expose any type information of the object it is referring to.

The only predefined global addresses are assigned to all localities. The following *HPX* API functions allow one to retrieve the global addresses of localities:

- `hpx::find_here`: retrieves the global address of the *locality* this function is called on.
- `hpx::find_all_localities`: retrieves the global addresses of all localities available to this application (including the *locality* the function is being called on).
- `hpx::find_remote_localities`: retrieves the global addresses of all remote localities available to this application (not including the *locality* the function is being called on).
- `hpx::get_num_localities`: retrieves the number of localities available to this application.
- `hpx::find_locality`: retrieves the global address of any *locality* supporting the given component type.
- `hpx::get_colocation_id`: retrieves the global address of the *locality* currently hosting the object with the given global address.

Additionally, the global addresses of localities can be used to create new instances of components using the following *HPX* API function:

- `hpx::components::new_`: Creates a new instance of the given `Component` type on the specified *locality*.

Note

HPX does not expose any functionality to delete component instances. All global addresses (as represented using `hpx::id_type`) are automatically garbage collected. When the last (global) reference to a particular component instance goes out of scope, the corresponding component instance is automatically deleted.

Posting actions

Action type definition

Actions are special types used to describe possibly remote operations. For every global function and every member function which has to be invoked distantly, a special type must be defined. For any global function the special macro *HPX_PLAIN_ACTION* can be used to define the action type. Here is an example demonstrating this:

```
namespace app
{
    void some_global_function(double d)
    {
        cout << d;
    }
}

// This will define the action type 'some_global_action' which represents
// the function 'app::some_global_function'.
HPX_PLAIN_ACTION(app::some_global_function, some_global_action);
```

Important

The macro *HPX_PLAIN_ACTION* has to be placed in global namespace, even if the wrapped function is located in some other namespace. The newly defined action type is placed in the global namespace as well.

If the action type should be defined somewhere not in global namespace, the action type definition has to be split into two macro invocations (*HPX_DEFINE_PLAIN_ACTION* and *HPX_REGISTER_ACTION*) as shown in the next example:

```
namespace app
{
    void some_global_function(double d)
    {
        cout << d;
    }

    // On conforming compilers the following macro expands to:
    //
    //    typedef hpx::actions::make_action<
    //        decltype(&some_global_function), &some_global_function
    //    >::type some_global_action;
    //
    // This will define the action type 'some_global_action' which represents
    // the function 'some_global_function'.
    HPX_DEFINE_PLAIN_ACTION(some_global_function, some_global_action);
}

// The following macro expands to a series of definitions of global objects
// which are needed for proper serialization and initialization support
// enabling the remote invocation of the function``some_global_function``
HPX_REGISTER_ACTION(app::some_global_action, app_some_global_action);
```

The shown code defines an action type `some_global_action` inside the namespace `app`.

Important

If the action type definition is split between two macros as shown above, the name of the action type to create has to be the same for both macro invocations (here `some_global_action`).

Important

The second argument passed to *HPX_REGISTER_ACTION* (`app_some_global_action`) has to comprise a globally unique C++ identifier representing the action. This is used for serialization purposes.

For member functions of objects which have been registered with AGAS (e.g., 'components'), a different registration macro *HPX_DEFINE_COMPONENT_ACTION* has to be utilized. Any component needs to be declared in a header file and have some special support macros defined in a source file. Here is an example demonstrating this. The first snippet has to go into the header file:

```
namespace app
{
    struct some_component
      : hpx::components::component_base<some_component>
    {
        int some_member_function(std::string s)
        {
            return std::stoi(s);
        }

        // This will define the action type 'some_member_action' which
        // represents the member function 'some_member_function' of the
        // object type 'some_component'.
        HPX_DEFINE_COMPONENT_ACTION(some_component, some_member_function,
            some_member_action);
    };
}

// Note: The second argument to the macro below has to be systemwide-unique
//       C++ identifiers
HPX_REGISTER_ACTION_DECLARATION(app::some_component::some_member_action, some_component_
↪some_action);
```

The next snippet belongs in a source file (e.g., the main application source file) in the simplest case:

```
typedef hpx::components::component<app::some_component> component_type;
typedef app::some_component some_component;

HPX_REGISTER_COMPONENT(component_type, some_component);

// The parameters for this macro have to be the same as used in the corresponding
// HPX_REGISTER_ACTION_DECLARATION() macro invocation above
typedef some_component::some_member_action some_component_some_action;
HPX_REGISTER_ACTION(some_component_some_action);
```

While these macro invocations are a bit more complex than those for simple global functions, they should still be manageable.

The most important macro invocation is the *HPX_DEFINE_COMPONENT_ACTION* in the header file as this defines the action type we need to invoke the member function. For a complete example of a simple component action see `component_in_executable.cpp`.

Reflection-based component actions (experimental)

Note

This feature requires a compiler with C++26 reflection support (`HPX_HAVE_CXX26_REFLECTION`) and is currently experimental.

When reflection support is available, component actions can be defined with a single macro instead of the three-step process described above:

```
namespace app
{
    struct some_component
      : hpx::components::component_base<some_component>
    {
        int some_member_function(std::string s)
        {
            return std::stoi(s);
        }
        // This single line replaces HPX_DEFINE_COMPONENT_ACTION,
        // HPX_REGISTER_ACTION_DECLARATION, and HPX_REGISTER_ACTION.
        HPX_COMPONENT_ACTION(some_component, some_member_function,
            some_member_action);
    };
}
```

The resulting `some_member_action` type can be used exactly like an action type created with *HPX_DEFINE_COMPONENT_ACTION*, with *hpx::post*, *hpx::async*, and *hpx::sync*, or by invoking it directly. No separate registration step is required in the source file.

Action invocation

The process of invoking a global function (or a member function of an object) with the help of the associated action is called ‘posting the action’. Actions can have arguments, which will be supplied while the action is applied. At the minimum, one parameter is required to `post` any action - the id of the *locality* the associated function should be invoked on (for global functions), or the id of the component instance (for member functions). Generally, *HPX* provides several ways to `post` an action, all of which are described in the following sections.

Generally, *HPX* actions are very similar to ‘normal’ C++ functions except that actions can be invoked remotely. Fig. **??** below shows an overview of the main API exposed by HPX. This shows the function invocation syntax as defined by the C++ language (dark gray), the additional invocation syntax as provided through C++ Standard Library features (medium gray), and the extensions added by *HPX* (light gray) where:

- `f` function to invoke,
- `p..`: (optional) arguments,
- R: return type of `f`,

- `action`: action type defined by, *HPX_DEFINE_PLAIN_ACTION* or *HPX_DEFINE_COMPONENT_ACTION* encapsulating `f`,
- `a`: an instance of the type `action`,
- `id`: the global address the action is applied to.

R f(p...)	Synchronous (return R)	Asynchronous (return future<R>)	Fire & Forget (return void)
Functions (direct)	f(p…) C++	async(f, p…)	post(f, p…)
Functions (lazy)	bind(f, p…)(…)	async(bind(f, p…), …) C++ Library	post(bind(f, p…), …)
Actions (direct)	HPX_ACTION(f, a) a(id, p…)	HPX_ACTION(f, a) async(a, id, p…)	HPX_ACTION(f, a) post(a, id, p…)
Actions (lazy)	HPX_ACTION(f, a) bind(a, id, p…)(…)	HPX_ACTION(f, a) async(bind(a, id, p…), …)	HPX_ACTION(f, a) post(bind(a, id, p…), …) HPX

Fig. 2.8: Overview of the main API exposed by *HPX*.

This figure shows that *HPX* allows the user to `post` actions with a syntax similar to the C++ standard. In fact, all action types have an overloaded function operator allowing to synchronously `post` the action. Further, *HPX* implements `hpx::async` which semantically works similar to the way `std::async` works for plain C++ function.

> **Note**
>
> The similarity of posting an action to conventional function invocations extends even further. *HPX* implements `hpx::bind` and `hpx::function` two facilities which are semantically equivalent to the `std::bind` and `std::function` types as defined by the C++11 Standard. While `hpx::async` extends beyond the conventional semantics by supporting actions and conventional C++ functions, the *HPX* facilities `hpx::bind` and `hpx::function` extend beyond the conventional standard facilities too. The *HPX* facilities not only support conventional functions, but can be used for actions as well.

Additionally, *HPX* exposes `hpx::post` and `hpx::async_continue` both of which refine and extend the standard C++ facilities.

The different ways to invoke a function in *HPX* will be explained in more detail in the following sections.

Posting an action asynchronously without any synchronization

This method ('fire and forget') will make sure the function associated with the action is scheduled to run on the target *locality*. Posting the action does not wait for the function to start running, instead it is a fully asynchronous operation. The following example shows how to `post` the action as defined *in the previous section* on the local *locality* (the *locality* this code runs on):

```
some_global_action act;     // define an instance of some_global_action
hpx::post(act, hpx::find_here(), 2.0);
```

(the function `hpx::find_here()` returns the id of the local *locality*, i.e. the *locality* this code executes on).

Any component member function can be invoked using the same syntactic construct. Given that `id` is the global address for a component instance created earlier, this invocation looks like:

```
some_component_action act;     // define an instance of some_component_action
hpx::post(act, id, "42");
```

In this case any value returned from this action (e.g. in this case the integer `42` is ignored. Please look at *Action type definition* for the code defining the component action `some_component_action` used.

Posting an action asynchronously with synchronization

This method will make sure the action is scheduled to run on the target *locality*. Posting the action itself does not wait for the function to start running or to complete, instead this is a fully asynchronous operation similar to using `hpx::post` as described above. The difference is that this method will return an instance of a `hpx::future<>` encapsulating the result of the (possibly remote) execution. The future can be used to synchronize with the asynchronous operation. The following example shows how to `post` the action from above on the local *locality*:

```
some_global_action act;     // define an instance of some_global_action
hpx::future<void> f = hpx::async(act, hpx::find_here(), 2.0);
//
// ... other code can be executed here
//
f.get();    // this will possibly wait for the asynchronous operation to 'return'
```

(as before, the function `hpx::find_here()` returns the id of the local *locality* (the *locality* this code is executed on).

Note

The use of a `hpx::future<void>` allows the current thread to synchronize with any remote operation not returning any value.

Note

Any `std::future<>` returned from `std::async()` is required to block in its destructor if the value has not been set for this future yet. This is not true for `hpx::future<>` which will never block in its destructor, even if the value has not been returned to the future yet. We believe that consistency in the behavior of futures is more important than standards conformance in this case.

Any component member function can be invoked using the same syntactic construct. Given that `id` is the global address for a component instance created earlier, this invocation looks like:

```
some_component_action act;     // define an instance of some_component_action
hpx::future<int> f = hpx::async(act, id, "42");
//
// ... other code can be executed here
//
cout << f.get();    // this will possibly wait for the asynchronous operation to 'return'
↪42
```

Note

The invocation of `f.get()` will return the result immediately (without suspending the calling thread) if the result from the asynchronous operation has already been returned. Otherwise, the invocation of `f.get()` will suspend the execution of the calling thread until the asynchronous operation returns its result.

Posting an action synchronously

This method will schedule the function wrapped in the specified action on the target *locality*. While the invocation appears to be synchronous (as we will see), the calling thread will be suspended while waiting for the function to return. Invoking a plain action (e.g. a global function) synchronously is straightforward:

```
some_global_action act;     // define an instance of some_global_action
act(hpx::find_here(), 2.0);
```

While this call looks just like a normal synchronous function invocation, the function wrapped by the action will be scheduled to run on a new thread and the calling thread will be suspended. After the new thread has executed the wrapped global function, the waiting thread will resume and return from the synchronous call.

Equivalently, any action wrapping a component member function can be invoked synchronously as follows:

```
some_component_action act;     // define an instance of some_component_action
int result = act(id, "42");
```

The action invocation will either schedule a new thread locally to execute the wrapped member function (as before, `id` is the global address of the component instance the member function should be invoked on), or it will send a parcel to the remote *locality* of the component causing a new thread to be scheduled there. The calling thread will be suspended until the function returns its result. This result will be returned from the synchronous action invocation.

It is very important to understand that this 'synchronous' invocation syntax in fact conceals an asynchronous function call. This is beneficial as the calling thread is suspended while waiting for the outcome of a potentially remote operation. The *HPX* thread scheduler will schedule other work in the meantime, allowing the application to make further progress while the remote result is computed. This helps overlapping computation with communication and hiding communication latencies.

Note

The syntax of posting an action is always the same, regardless whether the target *locality* is remote to the invocation *locality* or not. This is a very important feature of *HPX* as it frees the user from the task of keeping track what actions have to be applied locally and which actions are remote. If the target for posting an action is local, a new thread is automatically created and scheduled. Once this thread is scheduled and run, it will execute the function encapsulated by that action. If the target is remote, *HPX* will send a parcel to the remote *locality* which encapsulates the action and its parameters. Once the parcel is received on the remote *locality* *HPX* will create and schedule a new thread there. Once this thread runs on the remote *locality*, it will execute the function encapsulated by the action.

Posting an action with a continuation but without any synchronization

This method is very similar to the method described in section *Posting an action asynchronously without any synchronization*. The difference is that it allows the user to chain a sequence of asynchronous operations, while handing the (intermediate) results from one step to the next step in the chain. Where `hpx::post` invokes a single function using 'fire and forget' semantics, `hpx::post_continue` asynchronously triggers a chain of functions without the need for the execution flow 'to come back' to the invocation site. Each of the asynchronous functions can be executed on a different *locality*.

Posting an action with a continuation and with synchronization

This method is very similar to the method described in section *Posting an action asynchronously with synchronization*. In addition to what `hpx::async` can do, the functions `hpx::async_continue` takes an additional function argument. This function will be called as the continuation of the executed action. It is expected to perform additional operations and to make sure that a result is returned to the original invocation site. This method chains operations asynchronously by providing a continuation operation which is automatically executed once the first action has finished executing.

As an example we chain two actions, where the result of the first action is forwarded to the second action and the result of the second action is sent back to the original invocation site:

```
// first action
std::int32_t action1(std::int32_t i)
{
    return i+1;
}
HPX_PLAIN_ACTION(action1);    // defines action1_type

// second action
std::int32_t action2(std::int32_t i)
{
    return i*2;
}
HPX_PLAIN_ACTION(action2);    // defines action2_type

// this code invokes 'action1' above and passes along a continuation
// function which will forward the result returned from 'action1' to
// 'action2'.
action1_type act1;     // define an instance of 'action1_type'
action2_type act2;     // define an instance of 'action2_type'
hpx::future<int> f =
    hpx::async_continue(act1, hpx::make_continuation(act2),
        hpx::find_here(), 42);
hpx::cout << f.get() << "\n";   // will print: 86 ((42 + 1) * 2)
```

By default, the continuation is executed on the same *locality* as `hpx::async_continue` is invoked from. If you want to specify the *locality* where the continuation should be executed, the code above has to be written as:

```
// this code invokes 'action1' above and passes along a continuation
// function which will forward the result returned from 'action1' to
// 'action2'.
action1_type act1;     // define an instance of 'action1_type'
action2_type act2;     // define an instance of 'action2_type'
hpx::future<int> f =
    hpx::async_continue(act1, hpx::make_continuation(act2, hpx::find_here()),
        hpx::find_here(), 42);
hpx::cout << f.get() << "\n";   // will print: 86 ((42 + 1) * 2)
```

Similarly, it is possible to chain more than 2 operations:

```
action1_type act1;     // define an instance of 'action1_type'
action2_type act2;     // define an instance of 'action2_type'
hpx::future<int> f =
    hpx::async_continue(act1,
        hpx::make_continuation(act2, hpx::make_continuation(act1)),
        hpx::find_here(), 42);
hpx::cout << f.get() << "\n";   // will print: 87 ((42 + 1) * 2 + 1)
```

The function `hpx::make_continuation` creates a special function object which exposes the following prototype:

```
struct continuation
{
    template <typename Result>
    void operator()(hpx::id_type id, Result&& result) const
    {
        ...
    }
};
```

where the parameters passed to the overloaded function operator `operator()()` are:

- the `id` is the global id where the final result of the asynchronous chain of operations should be sent to (in most cases this is the id of the `hpx::future` returned from the initial call to `hpx::async_continue`. Any custom continuation function should make sure this `id` is forwarded to the last operation in the chain.
- the `result` is the result value of the current operation in the asynchronous execution chain. This value needs to be forwarded to the next operation.

Note

All of those operations are implemented by the predefined continuation function object which is returned from `hpx::make_continuation`. Any (custom) function object used as a continuation should conform to the same interface.

Action error handling

Like in any other asynchronous invocation scheme it is important to be able to handle error conditions occurring while the asynchronous (and possibly remote) operation is executed. In *HPX* all error handling is based on standard C++ exception handling. Any exception thrown during the execution of an asynchronous operation will be transferred back to the original invocation *locality*, where it is rethrown during synchronization with the calling thread.

Important

Exceptions thrown during asynchronous execution can be transferred back to the invoking thread only for the synchronous and the asynchronous case with synchronization. Like with any other unhandled exception, any exception thrown during the execution of an asynchronous action *without* synchronization will result in calling `hpx::terminate` causing the running application to exit immediately.

Note

Even if error handling internally relies on exceptions, most of the API functions exposed by *HPX* can be used without throwing an exception. Please see *Working with exceptions* for more information.

As an example, we will assume that the following remote function will be executed:

```
namespace app
{
    void some_function_with_error(int arg)
    {
        if (arg < 0) {
            HPX_THROW_EXCEPTION(hpx::error::bad_parameter,
                "some_function_with_error",
                "some really bad error happened");
        }
        // do something else...
    }
}

// This will define the action type 'some_error_action' which represents
// the function 'app::some_function_with_error'.
HPX_PLAIN_ACTION(app::some_function_with_error, some_error_action);
```

The use of *HPX_THROW_EXCEPTION* to report the error encapsulates the creation of a *hpx::exception* which is initialized with the error code `hpx::error::bad_parameter`. Additionally it carries the passed strings, the information about the file name, line number, and call stack of the point the exception was thrown from.

We invoke this action using the synchronous syntax as described before:

```
// note: wrapped function will throw hpx::exception
some_error_action act;            // define an instance of some_error_action
try {
    act(hpx::find_here(), -3);    // exception will be rethrown from here
}
catch (hpx::exception const& e) {
    // prints: 'some really bad error happened: HPX(bad parameter)'
```

(continues on next page)

(continued from previous page)

```
    cout << e.what();
}
```

If this action is invoked asynchronously with synchronization, the exception is propagated to the waiting thread as well and is re-thrown from the future's function `get()`:

```
// note: wrapped function will throw hpx::exception
some_error_action act;            // define an instance of some_error_action
hpx::future<void> f = hpx::async(act, hpx::find_here(), -3);
try {
    f.get();                      // exception will be rethrown from here
}
catch (hpx::exception const& e) {
    // prints: 'some really bad error happened: HPX(bad parameter)'
    cout << e.what();
}
```

For more information about error handling please refer to the section *Working with exceptions*. There we also explain how to handle error conditions without having to rely on exception.

Writing components

A component in *HPX* is a C++ class which can be created remotely and for which its member functions can be invoked remotely as well. The following sections highlight how components can be defined, created, and used.

Defining components

In order for a C++ class type to be managed remotely in *HPX*, the type must be derived from the `hpx::components::component_base` template type. We call such C++ class types 'components'.

Note that the component type itself is passed as a template argument to the base class:

```
// header file some_component.hpp

#include <hpx/include/components.hpp>

namespace app
{
    // Define a new component type 'some_component'
    struct some_component
      : hpx::components::component_base<some_component>
    {
        // This member function is has to be invoked remotely
        int some_member_function(std::string const& s)
        {
            return std::stoi(s);
        }

        // This will define the action type 'some_member_action' which
        // represents the member function 'some_member_function' of the
        // object type 'some_component'.
```

(continues on next page)

(continued from previous page)

```
        HPX_DEFINE_COMPONENT_ACTION(some_component, some_member_function, some_member_
↪action);
    };
}

// This will generate the necessary boiler-plate code for the action allowing
// it to be invoked remotely. This declaration macro has to be placed in the
// header file defining the component itself.
//
// Note: The second argument to the macro below has to be systemwide-unique
//       C++ identifiers
//
HPX_REGISTER_ACTION_DECLARATION(app::some_component::some_member_action, some_component_
↪some_action);
```

There is more boiler plate code which has to be placed into a source file in order for the component to be usable. Every component type is required to have macros placed into its source file, one for each component type and one macro for each of the actions defined by the component type.

For instance:

```
// source file some_component.cpp

#include "some_component.hpp"

// The following code generates all necessary boiler plate to enable the
// remote creation of 'app::some_component' instances with 'hpx::new_<>()'
//
using some_component = app::some_component;
using some_component_type = hpx::components::component<some_component>;

// Please note that the second argument to this macro must be a
// (system-wide) unique C++-style identifier (without any namespaces)
//
HPX_REGISTER_COMPONENT(some_component_type, some_component);
```

Note

When C++26 reflection is enabled (`HPX_WITH_CXX26_REFLECTION=ON`), the second argument to *HPX_REGISTER_COMPONENT* can be omitted. The component name is then derived automatically from the server type via `std::meta::identifier_of`:

```
// With C++26 reflection -- single argument, name derived automatically
HPX_REGISTER_COMPONENT(some_component_type)
```

The derived name is fully qualified (includes enclosing namespaces) and is equivalent to the explicit two-argument form. Explicit names always take precedence over reflection-derived ones.

> // The parameters for this macro have to be the same as used in the corresponding // HPX_REGISTER_ACTION_DECLARATION() macro invocation in the corresponding // header file. // // Please note that the second argument to this macro must be a // (system-wide) unique C++-style identifier (without any namespaces) // HPX_REGISTER_ACTION(app::some_component::some_member_action,

```
        some_component_some_action);
```

Defining client side representation classes

Often it is very convenient to define a separate type for a component which can be used on the client side (from where the component is instantiated and used). This step might seem as unnecessary duplicating code, however it significantly increases the type safety of the code.

A possible implementation of such a client side representation for the component described in the previous section could look like:

```
#include <hpx/include/components.hpp>

namespace app
{
    // Define a client side representation type for the component type
    // 'some_component' defined in the previous section.
    //
    struct some_component_client
      : hpx::components::client_base<some_component_client, some_component>
    {
        using base_type = hpx::components::client_base<
                some_component_client, some_component>;

        some_component_client(hpx::future<hpx::id_type> && id)
          : base_type(std::move(id))
        {}

        hpx::future<int> some_member_function(std::string const& s)
        {
            some_component::some_member_action act;
            return hpx::async(act, get_id(), s);
        }
    };
}
```

A client side object stores the global id of the component instance it represents. This global id is accessible by calling the function `client_base<>::get_id()`. The special constructor which is provided in the example allows to create this client side object directly using the API function *hpx::new_*.

Creating component instances

Instances of defined component types can be created in two different ways. If the component to create has a defined client side representation type, then this can be used, otherwise use the server type.

The following examples assume that `some_component_type` is the type of the server side implementation of the component to create. All additional arguments (see `, ...` notation below) are passed through to the corresponding constructor calls of those objects:

```
// create one instance on the given locality
hpx::id_type here = hpx::find_here();
```

(continues on next page)

(continued from previous page)

```
hpx::future<hpx::id_type> f =
    hpx::new_<some_component_type>(here, ...);

// create one instance using the given distribution
// policy (here: hpx::colocating_distribution_policy)
hpx::id_type here = hpx::find_here();
hpx::future<hpx::id_type> f =
    hpx::new_<some_component_type>(hpx::colocated(here), ...);

// create multiple instances on the given locality
hpx::id_type here = find_here();
hpx::future<std::vector<hpx::id_type>> f =
    hpx::new_<some_component_type[]>(here, num, ...);

// create multiple instances using the given distribution
// policy (here: hpx::binpacking_distribution_policy)
hpx::future<std::vector<hpx::id_type>> f = hpx::new_<some_component_type[]>(
    hpx::binpacking(hpx::find_all_localities()), num, ...);
```

The examples below demonstrate the use of the same API functions for creating client side representation objects (instead of just plain ids). These examples assume that `client_type` is the type of the client side representation of the component type to create. As above, all additional arguments (see `, ...` notation below) are passed through to the corresponding constructor calls of the server side implementation objects corresponding to the `client_type`:

```
// create one instance on the given locality
hpx::id_type here = hpx::find_here();
client_type c = hpx::new_<client_type>(here, ...);

// create one instance using the given distribution
// policy (here: hpx::colocating_distribution_policy)
hpx::id_type here = hpx::find_here();
client_type c = hpx::new_<client_type>(hpx::colocated(here), ...);

// create multiple instances on the given locality
hpx::id_type here = hpx::find_here();
hpx::future<std::vector<client_type>> f =
    hpx::new_<client_type[]>(here, num, ...);

// create multiple instances using the given distribution
// policy (here: hpx::binpacking_distribution_policy)
hpx::future<std::vector<client_type>> f = hpx::new_<client_type[]>(
    hpx::binpacking(hpx::find_all_localities()), num, ...);
```

Using component instances

After having created the component instances as described above, we can simply use them as indicated below:

```
#include <hpx/include/components.hpp>
#include <iostream>
#include <vector>

// Define a simple component
struct some_component : hpx::components::component_base<some_component>
{
    void print() const
    {
        std::cout << "Hello from component instance!" << std::endl;
    }
    HPX_DEFINE_COMPONENT_ACTION(some_component, print, print_action);
};

typedef some_component::print_action print_action;

// Create one instance on the given locality
hpx::id_type here = hpx::find_here();
hpx::future<hpx::id_type> f1 =
    hpx::new_<some_component>(here);

// Get the future value
hpx::id_type instance_id = f1.get();

// Invoke action on the instance
hpx::async<print_action>(instance_id).get();

// Create multiple instances on the given locality
int num = 3;
hpx::future<std::vector<hpx::id_type>> f2 =
    hpx::new_<some_component[]>(here, num);

// Get the future value
std::vector<hpx::id_type> instance_ids = f2.get();

// Invoke action on each instance
for (const auto& id : instance_ids)
{
    hpx::async<print_action>(id).get();
}
```

We can use the component instances with distribution policies the same way.

Segmented containers

In parallel programming, there is now a plethora of solutions aimed at implementing “partially contiguous” or segmented data structures, whether on shared memory systems or distributed memory systems. *HPX* implements such structures by drawing inspiration from Standard C++ containers.

Using segmented containers

A segmented container is a template class that is described in the namespace `hpx`. All segmented containers are very similar semantically to their sequential counterpart (defined in `namespace std` but with an additional template parameter named `DistPolicy`). The distribution policy is an optional parameter that is passed last to the segmented container constructor (after the container size when no default value is given, after the default value if not). The distribution policy describes the manner in which a container is segmented and the placement of each segment among the available runtime localities.

However, only a part of the `std` container member functions were reimplemented:

- `(constructor)`, `(destructor)`, `operator=`
- `operator[]`
- `begin`, `cbegin`, `end`, `cend`
- `size`

An example of how to use the `partitioned_vector` container would be:

```
#include <hpx/include/partitioned_vector.hpp>

// The following code generates all necessary boiler plate to enable the
// remote creation of 'partitioned_vector' segments
//
HPX_REGISTER_PARTITIONED_VECTOR(double);

// By default, the number of segments is equal to the current number of
// localities
//
hpx::partitioned_vector<double> va(50);
hpx::partitioned_vector<double> vb(50, 0.0);
```

An example of how to use the `partitioned_vector` container with distribution policies would be:

```
#include <hpx/include/partitioned_vector.hpp>
#include <hpx/modules/runtime_distributed.hpp>

// The following code generates all necessary boiler plate to enable the
// remote creation of 'partitioned_vector' segments
//
HPX_REGISTER_PARTITIONED_VECTOR(double);

std::size_t num_segments = 10;
std::vector<hpx::id_type> locs = hpx::find_all_localities();

auto layout =
        hpx::container_layout( num_segments, locs );
```

(continues on next page)

(continued from previous page)

```
// The number of segments is 10 and those segments are spread across the
// localities collected in the variable locs in a Round-Robin manner
//
hpx::partitioned_vector<double> va(50, layout);
hpx::partitioned_vector<double> vb(50, 0.0, layout);
```

By definition, a segmented container must be accessible from any thread although its construction is synchronous only for the thread who has called its constructor. To overcome this problem, it is possible to assign a symbolic name to the segmented container:

```
#include <hpx/include/partitioned_vector.hpp>

// The following code generates all necessary boiler plate to enable the
// remote creation of 'partitioned_vector' segments
//
HPX_REGISTER_PARTITIONED_VECTOR(double);

hpx::future<void> fserver = hpx::async(
  [](){
    hpx::partitioned_vector<double> v(50);

    // Register the 'partitioned_vector' with the name "some_name"
    //
    v.register_as("some_name");

    /* Do some code  */
  });

hpx::future<void> fclient =
  hpx::async(
    [](){
      // Naked 'partitioned_vector'
      //
      hpx::partitioned_vector<double> v;

      // Now the variable v points to the same 'partitioned_vector' that has
      // been registered with the name "some_name"
      //
      v.connect_to("some_name");

      /* Do some code  */
    });
```

Segmented containers

HPX provides the following segmented containers:

Table 2.23: Sequence containers

Name	Description	In header	C++ standard
`hpx::partitioned_vec`	Dynamic segmented contiguous array.	`<hpx/include/partitioned_vector.hpp>`	vector[178]

Table 2.24: Unordered associative containers

Name	Description	In header	C++ standard
`hpx::unordere`	Segmented collection of key-value pairs, hashed by keys, keys are unique.	`<hpx/include/unordered_map.hpp>`	unordered_map[179]

Segmented iterators and segmented iterator traits

The basic iterator used in the STL library is only suitable for one-dimensional structures. The iterators we use in *HPX* must adapt to the segmented format of our containers. Our iterators are then able to know when incrementing themselves if the next element of type `T` is in the same data segment or in another segment. In this second case, the iterator will automatically point to the beginning of the next segment.

Note

Note that the dereference operation `operator *` does not directly return a reference of type `T&` but an intermediate object wrapping this reference. When this object is used as an l-value, a remote write operation is performed; When this object is used as an r-value, implicit conversion to `T` type will take care of performing remote read operation.

It is sometimes useful not only to iterate element by element, but also segment by segment, or simply get a local iterator in order to avoid additional construction costs at each deferencing operations. To mitigate this need, the `hpx::traits::segmented_iterator_traits` are used.

With `segmented_iterator_traits` users can uniformly get the iterators which specifically iterates over segments (by providing a segmented iterator as a parameter), or get the local begin/end iterators of the nearest local segment (by providing a per-segment iterator as a parameter):

```
#include <hpx/include/partitioned_vector.hpp>

// The following code generates all necessary boiler plate to enable the
// remote creation of 'partitioned_vector' segments
//
HPX_REGISTER_PARTITIONED_VECTOR(double);

using iterator = hpx::partitioned_vector<T>::iterator;
using traits   = hpx::traits::segmented_iterator_traits<iterator>;
```

(continues on next page)

[178] http://en.cppreference.com/w/cpp/container/vector
[179] http://en.cppreference.com/w/cpp/container/unordered_map

(continued from previous page)

```
hpx::partitioned_vector<T> v;
std::size_t count = 0;

auto seg_begin = traits::segment(v.begin());
auto seg_end   = traits::segment(v.end());

// Iterate over segments
for (auto seg_it = seg_begin; seg_it != seg_end; ++seg_it)
{
    auto loc_begin = traits::begin(seg_it);
    auto loc_end   = traits::end(seg_it);

    // Iterate over elements inside segments
    for (auto lit = loc_begin; lit != loc_end; ++lit, ++count)
    {
        *lit = count;
    }
}
```

Which is equivalent to:

```
hpx::partitioned_vector<T> v;
std::size_t count = 0;

auto begin = v.begin();
auto end   = v.end();

for (auto it = begin; it != end; ++it, ++count)
{
    *it = count;
}
```

Using views

The use of multidimensional arrays is quite common in the numerical field whether to perform dense matrix operations or to process images. It exist many libraries which implement such object classes overloading their basic operators (e.g. +, -, *, (), etc.). However, such operation becomes more delicate when the underlying data layout is segmented or when it is mandatory to use optimized linear algebra subroutines (i.e. BLAS subroutines).

Our solution is thus to relax the level of abstraction by allowing the user to work not directly on n-dimensionnal data, but on "n-dimensionnal collections of 1-D arrays". The use of well-accepted techniques on contiguous data is thus preserved at the segment level, and the composability of the segments is made possible thanks to multidimensional array-inspired access mode.

Preface: Why SPMD?

Although *HPX* refutes by design this programming model, the *locality* plays a dominant role when it comes to implement vectorized code. To maximize local computations and avoid unneeded data transfers, a parallel section (or Single Programming Multiple Data section) is required. Because the use of global variables is prohibited, this parallel section is created via the RAII idiom.

To define a parallel section, simply write an action taking a `spmd_block` variable as a first parameter:

```
#include <hpx/modules/collectives.hpp>

void bulk_function(hpx::lcos::spmd_block block /* , arg0, arg1, ... */)
{
    // Parallel section

    /* Do some code */
}
HPX_PLAIN_ACTION(bulk_function, bulk_action);
```

Note

In the following paragraphs, we will use the term "image" several times. An image is defined as a lightweight process whose entry point is a function provided by the user. It's an "image of the function".

The `spmd_block` class contains the following methods:

- Team information: `get_num_images`, `this_image`, `images_per_locality`
- Control statements: `sync_all`, `sync_images`

Here is a sample code summarizing the features offered by the `spmd_block` class:

```
#include <hpx/modules/collectives.hpp>

void bulk_function(hpx::lcos::spmd_block block /* , arg0, arg1, ... */)
{
    std::size_t num_images = block.get_num_images();
    std::size_t this_image = block.this_image();
    std::size_t images_per_locality = block.images_per_locality();

    /* Do some code */

    // Synchronize all images in the team
    block.sync_all();

    /* Do some code */

    // Synchronize image 0 and image 1
    block.sync_images(0,1);

    /* Do some code */

    std::vector<std::size_t> vec_images = {2,3,4};
```

(continues on next page)

(continued from previous page)

```
    // Synchronize images 2, 3 and 4
    block.sync_images(vec_images);

    // Alternative call to synchronize images 2, 3 and 4
    block.sync_images(vec_images.begin(), vec_images.end());

    /* Do some code */

    // Non-blocking version of sync_all()
    hpx::future<void> event =
        block.sync_all(hpx::launch::async);

    // Callback waiting for 'event' to be ready before being scheduled
    hpx::future<void> cb =
        event.then(
          [](hpx::future<void>)
          {

            /* Do some code */

          });

    // Finally wait for the execution tree to be finished
    cb.get();
}
HPX_PLAIN_ACTION(bulk_test_function, bulk_test_action);
```

Then, in order to invoke the parallel section, call the function `define_spmd_block` specifying an arbitrary symbolic name and indicating the number of images per *locality* to create:

```
void bulk_function(hpx::lcos::spmd_block block, /* , arg0, arg1, ... */)
{

}
HPX_PLAIN_ACTION(bulk_test_function, bulk_test_action);

int main()
{
    /* std::size_t arg0, arg1, ...; */

    bulk_action act;
    std::size_t images_per_locality = 4;

    // Instantiate the parallel section
    hpx::lcos::define_spmd_block(
        "some_name", images_per_locality, std::move(act) /*, arg0, arg1, ... */);

    return 0;
}
```

> **Note**
>
> In principle, the user should never call the `spmd_block` constructor. The `define_spmd_block` function is responsible of instantiating `spmd_block` objects and broadcasting them to each created image.

SPMD multidimensional views

Some classes are defined as "container views" when the purpose is to observe and/or modify the values of a container using another perspective than the one that characterizes the container. For example, the values of an `std::vector` object can be accessed via the expression `[i]`. Container views can be used, for example, when it is desired for those values to be "viewed" as a 2D matrix that would have been flattened in a `std::vector`. The values would be possibly accessible via the expression `vv(i,j)` which would call internally the expression `v[k]`.

By default, the `partitioned_vector` class integrates 1-D views of its segments:

```
#include <hpx/include/partitioned_vector.hpp>

// The following code generates all necessary boiler plate to enable the
// remote creation of 'partitioned_vector' segments
//
HPX_REGISTER_PARTITIONED_VECTOR(double);

using iterator = hpx::partitioned_vector<double>::iterator;
using traits   = hpx::traits::segmented_iterator_traits<iterator>;

hpx::partitioned_vector<double> v;

// Create a 1-D view of the vector of segments
auto vv = traits::segment(v.begin());

// Access segment i
std::vector<double> v = vv[i];
```

Our views are called "multidimensional" in the sense that they generalize to N dimensions the purpose of `segmented_iterator_traits::segment()` in the 1-D case. Note that in a parallel section, the 2-D expression `a(i,j) = b(i,j)` is quite confusing because without convention, each of the images invoked will race to execute the statement. For this reason, our views are not only multidimensional but also "spmd-aware".

> **Note**
>
> SPMD-awareness: The convention is simple. If an assignment statement contains a view subscript as an l-value, it is only and only the image holding the r-value who is evaluating the statement. (In MPI sense, it is called a Put operation).

Subscript-based operations

Here are some examples of using subscripts in the 2-D view case:

```
#include <hpx/components/containers/partitioned_vector/partitioned_vector_view.hpp>
#include <hpx/include/partitioned_vector.hpp>

// The following code generates all necessary boiler plate to enable the
// remote creation of 'partitioned_vector' segments
//
HPX_REGISTER_PARTITIONED_VECTOR(double);

using Vec = hpx::partitioned_vector<double>;
using View_2D = hpx::partitioned_vector_view<double,2>;

/* Do some code */

Vec v;

// Parallel section (suppose 'block' an spmd_block instance)
{
    std::size_t height, width;

    // Instantiate the view
    View_2D vv(block, v.begin(), v.end(), {height,width});

    // The l-value is a view subscript, the image that owns vv(1,0)
    // evaluates the assignment.
    vv(0,1) = vv(1,0);

    // The l-value is a view subscript, the image that owns the r-value
    // (result of expression 'std::vector<double>(4,1.0)') evaluates the
    // assignment : oops! race between all participating images.
    vv(2,3) = std::vector<double>(4,1.0);
}
```

Iterator-based operations

Here are some examples of using iterators in the 3-D view case:

```
#include <hpx/components/containers/partitioned_vector/partitioned_vector_view.hpp>
#include <hpx/include/partitioned_vector.hpp>

// The following code generates all necessary boiler plate to enable the
// remote creation of 'partitioned_vector' segments
//
HPX_REGISTER_PARTITIONED_VECTOR(int);

using Vec = hpx::partitioned_vector<int>;
using View_3D = hpx::partitioned_vector_view<int,3>;

/* Do some code */
```

(continues on next page)

(continued from previous page)

```
Vec v1, v2;

// Parallel section (suppose 'block' an spmd_block instance)
{
    std::size_t sixe_x, size_y, size_z;

    // Instantiate the views
    View_3D vv1(block, v1.begin(), v1.end(), {sixe_x,size_y,size_z});
    View_3D vv2(block, v2.begin(), v2.end(), {sixe_x,size_y,size_z});

    // Save previous segments covered by vv1 into segments covered by vv2
    auto vv2_it = vv2.begin();
    auto vv1_it = vv1.cbegin();

    for(; vv2_it != vv2.end(); vv2_it++, vv1_it++)
    {
        // It's a Put operation
        *vv2_it = *vv1_it;
    }

    // Ensure that all images have performed their Put operations
    block.sync_all();

    // Ensure that only one image is putting updated data into the different
    // segments covered by vv1
    if(block.this_image() == 0)
    {
        int idx = 0;

        // Update all the segments covered by vv1
        for(auto i = vv1.begin(); i != vv1.end(); i++)
        {
            // It's a Put operation
            *i = std::vector<float>(elt_size,idx++);
        }
    }
}
```

Here is an example that shows how to iterate only over segments owned by the current image:

```
#include <hpx/components/containers/partitioned_vector/partitioned_vector_view.hpp>
#include <hpx/components/containers/partitioned_vector/partitioned_vector_local_view.hpp>
#include <hpx/include/partitioned_vector.hpp>

// The following code generates all necessary boiler plate to enable the
// remote creation of 'partitioned_vector' segments
//
HPX_REGISTER_PARTITIONED_VECTOR(float);

using Vec = hpx::partitioned_vector<float>;
using View_1D = hpx::partitioned_vector_view<float,1>;
```

(continues on next page)

(continued from previous page)

```
/* Do some code */

Vec v;

// Parallel section (suppose 'block' an spmd_block instance)
{
    std::size_t num_segments;

    // Instantiate the view
    View_1D vv(block, v.begin(), v.end(), {num_segments});

    // Instantiate the local view from the view
    auto local_vv = hpx::local_view(vv);

    for ( auto i = local_vv.begin(); i != local_vv.end(); i++ )
    {
        std::vector<float> & segment = *i;

        /* Do some code */
    }

}
```

Instantiating sub-views

It is possible to construct views from other views: we call it sub-views. The constraint nevertheless for the subviews is to retain the dimension and the value type of the input view. Here is an example showing how to create a sub-view:

```
#include <hpx/components/containers/partitioned_vector/partitioned_vector_view.hpp>
#include <hpx/include/partitioned_vector.hpp>

// The following code generates all necessary boiler plate to enable the
// remote creation of 'partitioned_vector' segments
//
HPX_REGISTER_PARTITIONED_VECTOR(float);

using Vec = hpx::partitioned_vector<float>;
using View_2D = hpx::partitioned_vector_view<float,2>;

/* Do some code */

Vec v;

// Parallel section (suppose 'block' an spmd_block instance)
{
    std::size_t N = 20;
    std::size_t tilesize = 5;

    // Instantiate the view
    View_2D vv(block, v.begin(), v.end(), {N,N});
```

(continues on next page)

(continued from previous page)

```
    // Instantiate the subview
    View_2D svv(
        block,&vv(tilesize,0),&vv(2*tilesize-1,tilesize-1),{tilesize,tilesize},{N,N});

    if(block.this_image() == 0)
    {
        // Equivalent to 'vv(tilesize,0) = 2.0f'
        svv(0,0) = 2.0f;

        // Equivalent to 'vv(2*tilesize-1,tilesize-1) = 3.0f'
        svv(tilesize-1,tilesize-1) = 3.0f;
    }

}
```

Note

The last parameter of the subview constructor is the size of the original view. If one would like to create a subview of the subview and so on, this parameter should stay unchanged. `{N,N}` for the above example).

C++ co-arrays

Fortran has extended its scalar element indexing approach to reference each segment of a distributed array. In this extension, a segment is attributed a ?co-index? and lives in a specific *locality*. A co-index provides the application with enough information to retrieve the corresponding data reference. In C++, containers present themselves as a ?smarter? alternative of Fortran arrays but there are still no corresponding standardized features similar to the Fortran co-indexing approach. We present here an implementation of such features in *HPX*.

Preface: co-array, a segmented container tied to a SPMD multidimensional views

As mentioned before, a co-array is a distributed array whose segments are accessible through an array-inspired access mode. We have previously seen that it is possible to reproduce such access mode using the concept of views. Nevertheless, the user must pre-create a segmented container to instantiate this view. We illustrate below how a single constructor call can perform those two operations:

```
#include <hpx/components/containers/coarray/coarray.hpp>
#include <hpx/modules/collectives.hpp>

// The following code generates all necessary boiler plate to enable the
// co-creation of 'coarray'
//
HPX_REGISTER_COARRAY(double);

// Parallel section (suppose 'block' an spmd_block instance)
{
    using hpx::container::placeholders::_;

    std::size_t height=32, width=4, segment_size=10;
```

(continues on next page)

(continued from previous page)

```
    hpx::coarray<double,3> a(block, "a", {height,width,_}, segment_size);

    /* Do some code */
}
```

Unlike segmented containers, a co-array object can only be instantiated within a parallel section. Here is the description of the parameters to provide to the coarray constructor:

Table 2.25: Parameters of coarray constructor

Parameter	Description
`block`	Reference to a `spmd_block` object
`"a"`	Symbolic name of type `std::string`
`{height,width, _}`	Dimensions of the `coarray` object
`segment_size`	Size of a co-indexed element (i.e. size of the object referenced by the expression `a(i,j,k)`)

Note that the "last dimension size" cannot be set by the user. It only accepts the constexpr variable `hpx::container::placeholders::_`. This size, which is considered private, is equal to the number of current images (value returned by `block.get_num_images()`).

Note

An important constraint to remember about coarray objects is that all segments sharing the same "last dimension index" are located in the same image.

Using co-arrays

The member functions owned by the `coarray` objects are exactly the same as those of spmd multidimensional views. These are:

```
* Subscript-based operations
* Iterator-based operations
```

However, one additional functionality is provided. Knowing that the element `a(i,j,k)` is in the memory of the `k`th image, the use of local subscripts is possible.

Note

For spmd multidimensional views, subscripts are only global as it still involves potential remote data transfers.

Here is an example of using local subscripts:

```
#include <hpx/components/containers/coarray/coarray.hpp>
#include <hpx/modules/collectives.hpp>

// The following code generates all necessary boiler plate to enable the
// co-creation of 'coarray'
```

(continues on next page)

(continued from previous page)

```
//
HPX_REGISTER_COARRAY(double);

// Parallel section (suppose 'block' an spmd_block instance)
{
    using hpx::container::placeholders::_;

    std::size_t height=32, width=4, segment_size=10;

    hpx::coarray<double,3> a(block, "a", {height,width,_}, segment_size);

    double idx = block.this_image()*height*width;

    for (std::size_t j = 0; j<width; j++)
    for (std::size_t i = 0; i<height; i++)
    {
        // Local write operation performed via the use of local subscript
        a(i,j,_) = std::vector<double>(elt_size,idx);
        idx++;
    }

    block.sync_all();
}
```

Note

When the "last dimension index" of a subscript is equal to `hpx::container::placeholders::_`, local subscript (and not global subscript) is used. It is equivalent to a global subscript used with a "last dimension index" equal to the value returned by `block.this_image()`.

2.4.13 Running on batch systems

This section walks you through launching *HPX* applications on various batch systems.

How to use *HPX* applications with PBS

Most *HPX* applications are executed on parallel computers. These platforms typically provide integrated job management services that facilitate the allocation of computing resources for each parallel program. *HPX* includes support for one of the most common job management systems, the Portable Batch System (PBS).

All PBS jobs require a script to specify the resource requirements and other parameters associated with a parallel job. The PBS script is basically a shell script with PBS directives placed within commented sections at the beginning of the file. The remaining (not commented-out) portions of the file executes just like any other regular shell script. While the description of all available PBS options is outside the scope of this tutorial (the interested reader may refer to in-depth documentation[180] for more information), below is a minimal example to illustrate the approach. The following test application will use the multithreaded `hello_world_distributed` program, explained in the section *Remote execution with actions*.

[180] http://www.clusterresources.com/torquedocs21/

```
#!/bin/bash
#
#PBS -l nodes=2:ppn=4

APP_PATH=~/packages/hpx/bin/hello_world_distributed
APP_OPTIONS=

pbsdsh -u $APP_PATH $APP_OPTIONS --hpx:nodes=`cat $PBS_NODEFILE`
```

Caution

If the first application specific argument (inside `$APP_OPTIONS`) is a non-option (i.e., does not start with a `-` or a `--`), then the argument has to be placed before the option *--hpx:nodes*, which, in this case, should be the last option on the command line.

Alternatively, use the option *--hpx:endnodes* to explicitly mark the end of the list of node names:

```
$ pbsdsh -u $APP_PATH --hpx:nodes`cat $PBS_NODEFILE` --hpx:endnodes $APP_OPTIONS
```

The `#PBS -l nodes=2:ppn=4` directive will cause two compute nodes to be allocated for the application, as specified in the option `nodes`. Each of the nodes will dedicate four cores to the program, as per the option `ppn`, short for "processors per node" (PBS does not distinguish between processors and cores). Note that requesting more cores per node than physically available is pointless and may prevent PBS from accepting the script.

On newer PBS versions the PBS command syntax might be different. For instance, the PBS script above would look like:

```
#!/bin/bash
#
#PBS -l select=2:ncpus=4

APP_PATH=~/packages/hpx/bin/hello_world_distributed
APP_OPTIONS=

pbsdsh -u $APP_PATH $APP_OPTIONS --hpx:nodes=`cat $PBS_NODEFILE`
```

`APP_PATH` and `APP_OPTIONS` are shell variables that respectively specify the correct path to the executable (`hello_world_distributed` in this case) and the command line options. Since the `hello_world_distributed` application doesn't need any command line options, `APP_OPTIONS` has been left empty. Unlike in other execution environments, there is no need to use the *--hpx:threads* option to indicate the required number of OS threads per node; the *HPX* library will derive this parameter automatically from PBS.

Finally, pbsdsh is a PBS command that starts tasks to the resources allocated to the current job. It is recommended to leave this line as shown and modify only the PBS options and shell variables as needed for a specific application.

Important

A script invoked by pbsdsh starts in a very basic environment: the user's `$HOME` directory is defined and is the current directory, the `LANG` variable is set to `C` and the `PATH` is set to the basic `/usr/local/bin:/usr/bin:/bin` as defined in a system-wide file pbs_environment. Nothing that would normally be set up by a system shell profile or user shell profile is defined, unlike the environment for the main job script.

Another choice is for the pbsdsh command in your main job script to invoke your program via a shell, like `sh` or `bash`, so that it gives an initialized environment for each instance. Users can create a small script `runme.sh`, which is used to invoke the program:

```
#!/bin/bash
# Small script which invokes the program based on what was passed on its
# command line.
#
# This script is executed by the bash shell which will initialize all
# environment variables as usual.
$@
```

Now, the script is invoked using the pbsdsh tool:

```
#!/bin/bash
#
#PBS -l nodes=2:ppn=4

APP_PATH=~/packages/hpx/bin/hello_world_distributed
APP_OPTIONS=

pbsdsh -u runme.sh $APP_PATH $APP_OPTIONS --hpx:nodes=`cat $PBS_NODEFILE`
```

All that remains now is submitting the job to the queuing system. Assuming that the contents of the PBS script were saved in the file `pbs_hello_world.sh` in the current directory, this is accomplished by typing:

```
$ qsub ./pbs_hello_world_pbs.sh
```

If the job is accepted, qsub will print out the assigned job ID, which may look like:

```
$ 42.supercomputer.some.university.edu
```

To check the status of your job, issue the following command:

```
$ qstat 42.supercomputer.some.university.edu
```

and look for a single-letter job status symbol. The common cases include:

- *Q* - signifies that the job is queued and awaiting its turn to be executed.
- *R* - indicates that the job is currently running.
- *C* - means that the job has completed.

The example qstat output below shows a job waiting for execution resources to become available:

```
Job id                    Name             User            Time Use S Queue
------------------------- ---------------- --------------- -------- - -----
42.supercomputer          ...ello_world.sh joe_user               0 Q batch
```

After the job completes, PBS will place two files, `pbs_hello_world.sh.o42` and `pbs_hello_world.sh.e42`, in the directory where the job was submitted. The first contains the standard output and the second contains the standard error from all the nodes on which the application executed. In our example, the error output file should be empty and the standard output file should contain something similar to:

```
hello world from OS-thread 3 on locality 0
hello world from OS-thread 2 on locality 0
hello world from OS-thread 1 on locality 1
```

(continues on next page)

(continued from previous page)

```
hello world from OS-thread 0 on locality 0
hello world from OS-thread 3 on locality 1
hello world from OS-thread 2 on locality 1
hello world from OS-thread 1 on locality 0
hello world from OS-thread 0 on locality 1
```

Congratulations! You have just run your first distributed *HPX* application!

How to use *HPX* applications with SLURM

Just like PBS (described in section *How to use HPX applications with PBS*), SLURM is a job management system which is widely used on large supercomputing systems. Any *HPX* application can easily be run using SLURM. This section describes how this can be done.

The easiest way to run an *HPX* application using SLURM is to utilize the command line tool srun, which interacts with the SLURM batch scheduling system:

```
$ srun -p <partition> -N <number-of-nodes> hpx-application <application-arguments>
```

Here, `<partition>` is one of the node partitions existing on the target machine (consult the machine's documentation to get a list of existing partitions) and `<number-of-nodes>` is the number of compute nodes that should be used. By default, the *HPX* application is started with one *locality* per node and uses all available cores on a node. You can change the number of localities started per node (for example, to account for NUMA effects) by specifying the `-n` option of srun. The number of cores per *locality* can be set by `-c`. The `<application-arguments>` are any application specific arguments that need to be passed on to the application.

Note

There is no need to use any of the *HPX* command line options related to the number of localities, number of threads, or related to networking ports. All of this information is automatically extracted from the SLURM environment by the *HPX* startup code.

Important

The srun documentation explicitly states: "If `-c` is specified without `-n`, as many tasks will be allocated per node as possible while satisfying the `-c` restriction. For instance on a cluster with 8 CPUs per node, a job request for 4 nodes and 3 CPUs per task may be allocated 3 or 6 CPUs per node (1 or 2 tasks per node) depending upon resource consumption by other jobs." For this reason, it's recommended to always specify `-n <number-of-instances>`, even if `<number-of-instances>` is equal to one (1).

Interactive shells

To get an interactive development shell on one of the nodes, users can issue the following command:

```
$ srun -p <node-type> -N <number-of-nodes> --pty /bin/bash -l
```

After the shell has been opened, users can run their *HPX* application. By default, it uses all available cores. Note that if you requested one node, you don't need to do `srun` again. However, if you requested more than one node, and want to run your distributed application, you can use `srun` again to start up the distributed *HPX* application. It will use the resources that have been requested for the interactive shell.

Scheduling batch jobs

The above mentioned method of running *HPX* applications is fine for development purposes. The disadvantage that comes with `srun` is that it only returns once the application is finished. This might not be appropriate for longer-running applications (for example, benchmarks or larger scale simulations). In order to cope with that limitation, users can use the sbatch command.

The `sbatch` command expects a script that it can run once the requested resources are available. In order to request resources, users need to add `#SBATCH` comments in their script or provide the necessary parameters to `sbatch` directly. The parameters are the same as with `run`. The commands you need to execute are the same you would need to start your application as if you were in an interactive shell.

2.4.14 Debugging *HPX* applications

Using a debugger with *HPX* applications

Using a debugger such as `gdb` with *HPX* applications is no problem. However, there are some things to keep in mind to make the experience somewhat more productive.

Call stacks in *HPX* can often be quite unwieldy as the library is heavily templated and the call stacks can be very deep. For this reason it is sometimes a good idea compile *HPX* in `RelWithDebInfo` mode, which applies some optimizations but keeps debugging symbols. This can often compress call stacks significantly. On the other hand, stepping through the code can also be more difficult because of statements being reordered and variables being optimized away. Also, note that because *HPX* implements user-space threads and context switching, call stacks may not always be complete in a debugger.

HPX launches not only worker threads but also a few helper threads. The first thread is the main thread, which typically does no work in an *HPX* application, except at startup and shutdown. If using the default settings, *HPX* will spawn six additional threads (used for service thread pools). The first worker thread is usually the eighth thread, and most user codes will be run on these worker threads. The last thread is a helper thread used for *HPX* shutdown.

Finally, since *HPX* is a multi-threaded runtime, the following `gdb` options can be helpful:

```
set pagination off
set non-stop on
```

Non-stop mode allows users to have a single thread stop on a breakpoint without stopping all other threads as well.

Using sanitizers with *HPX* applications

Warning

Not all parts of *HPX* are sanitizer clean. This means that users may end up with false positives from *HPX* itself when using sanitizers for their applications.

To use sanitizers with *HPX*, turn on `HPX_WITH_SANITIZERS` and turn off `HPX_WITH_STACKOVERFLOW_DETECTION` during CMake[181] configuration. It's recommended to also build Boost with the same sanitizers that will be used for *HPX*. The appropriate sanitizers can then be enabled using CMake by appending `-fsanitize=address -fno-omit-frame-pointer` to `CMAKE_CXX_FLAGS` and `-fsanitize=address` to `CMAKE_EXE_LINKER_FLAGS`. Replace `address` with the sanitizer that you want to use.

Debugging applications using core files

For *HPX* to generate useful core files, *HPX* has to be compiled without signal and exception handlers *`HPX_WITH_DISABLED_SIGNAL_EXCEPTION_HANDLERS:BOOL`*. If this option is not specified, the signal handlers change the application state. For example, after a segmentation fault the stack trace will show the signal handler. Similarly, unhandled exceptions are also caught by these handlers and the stack trace will not point to the location where the unhandled exception was thrown.

In general, core files are a helpful tool to inspect the state of the application at the moment of the crash (post-mortem debugging), without the need of attaching a debugger beforehand. This approach to debugging is especially useful if the error cannot be reliably reproduced, as only a single crashed application run is required to gain potentially helpful information like a stacktrace.

To debug with core files, the operating system first has to be told to actually write them. On most Unix systems this can be done by calling:

```
$ ulimit -c unlimited
```

in the shell. Now the debugger can be started up with:

```
$ gdb <application> <core file name>
```

The debugger should now display the last state of the application. The default file name for core files is `core`.

2.4.15 Optimizing *HPX* applications

Performance counters

Performance counters in *HPX* are used to provide information as to how well the runtime system or an application is performing. The counter data can help determine system bottlenecks, and fine-tune system and application performance. The *HPX* runtime system, its networking, and other layers provide counter data that an application can consume to provide users with information about how well the application is performing.

Applications can also use counter data to determine how much system resources to consume. For example, an application that transfers data over the network could consume counter data from a network switch to determine how much data to transfer without competing for network bandwidth with other network traffic. The application could use the counter data to adjust its transfer rate as the bandwidth usage from other network traffic increases or decreases.

[181] https://www.cmake.org

Performance counters are *HPX* parallel processes that expose a predefined interface. *HPX* exposes special API functions that allow one to create, manage, and read the counter data, and release instances of performance counters. Performance Counter instances are accessed by name, and these names have a predefined structure which is described in the section *Performance counter names*. The advantage of this is that any Performance Counter can be accessed remotely (from a different *locality*) or locally (from the same *locality*). Moreover, since all counters expose their data using the same API, any code consuming counter data can be utilized to access arbitrary system information with minimal effort.

Counter data may be accessed in real time. More information about how to consume counter data can be found in the section *Consuming performance counter data*.

All *HPX* applications provide command line options related to performance counters, such as the ability to list available counter types, or periodically query specific counters to be printed to the screen or save them in a file. For more information, please refer to the section *HPX Command Line Options*.

Performance counter names

All Performance Counter instances have a name uniquely identifying each instance. This name can be used to access the counter, retrieve all related meta data, and to query the counter data (as described in the section *Consuming performance counter data*). Counter names are strings with a predefined structure. The general form of a countername is:

```
/objectname{full_instancename}/countername@parameters
```

where `full_instancename` could be either another (full) counter name or a string formatted as:

```
parentinstancename#parentindex/instancename#instanceindex
```

Each separate part of a countername (e.g., `objectname`, `countername` `parentinstancename`, `instancename`, and `parameters`) should start with a letter (`'a'`… `'z'`, `'A'`… `'Z'`) or an underscore character (`'_'`), optionally followed by letters, digits (`'0'`… `'9'`), hyphen (`'-'`), or underscore characters. Whitespace is not allowed inside a counter name. The characters `'/'`, `'{'`, `'}'`, `'#'` and `'@'` have a special meaning and are used to delimit the different parts of the counter name.

The parts `parentinstanceindex` and `instanceindex` are integers. If an index is not specified, *HPX* will assume a default of `-1`.

Two counter name examples

This section gives examples of both simple counter names and aggregate counter names. For more information on simple and aggregate counter names, please see *Performance counter instances*.

An example of a well-formed (and meaningful) simple counter name would be:

```
/threads{locality#0/total}/count/cumulative
```

This counter returns the current cumulative number of executed (retired) *HPX* threads for the *locality* `0`. The counter type of this counter is `/threads/count/cumulative` and the full instance name is `locality#0/total`. This counter type does not require an `instanceindex` or `parameters` to be specified.

In this case, the `parentindex` (the `'0'`) designates the *locality* for which the counter instance is created. The counter will return the number of *HPX* threads retired on that particular *locality*.

Another example for a well formed (aggregate) counter name is:

```
/statistics{/threads{locality#0/total}/count/cumulative}/average@500
```

This counter takes the simple counter from the first example, samples its values every `500` milliseconds, and returns the average of the value samples whenever it is queried. The counter type of this counter is `/statistics/average` and the instance name is the full name of the counter for which the values have to be averaged. In this case, the `parameters` (the `'500'`) specify the sampling interval for the averaging to take place (in milliseconds).

Performance counter types

Every performance counter belongs to a specific performance counter type which classifies the counters into groups of common semantics. The type of a counter is identified by the `objectname` and the `countername` parts of the name.

```
/objectname/countername
```

When an application starts *HPX* will register all available counter types on each of the localities. These counter types are held in a special performance counter registration database, which can be used to retrieve the meta data related to a counter type and to create counter instances based on a given counter instance name.

Performance counter instances

The `full_instancename` distinguishes different counter instances of the same counter type. The formatting of the `full_instancename` depends on the counter type. There are two types of counters: simple counters, which usually generate the counter values based on direct measurements, and aggregate counters, which take another counter and transform its values before generating their own counter values. An example for a simple counter is given *above*: counting retired *HPX* threads. An aggregate counter is shown as an example *above* as well: calculating the average of the underlying counter values sampled at constant time intervals.

While simple counters use instance names formatted as `parentinstancename#parentindex/instancename#instanceindex`, most aggregate counters have the full counter name of the embedded counter as their instance name.

Not all simple counter types require specifying all four elements of a full counter instance name; some of the parts (`parentinstancename`, `parentindex`, `instancename`, and `instanceindex`) are optional for specific counters. Please refer to the documentation of a particular counter for more information about the formatting requirements for the name of this counter (see *Existing HPX performance counters*).

The `parameters` are used to pass additional information to a counter at creation time. They are optional, and they fully depend on the concrete counter. Even if a specific counter type allows additional parameters to be given, those usually are not required as sensible defaults will be chosen. Please refer to the documentation of a particular counter for more information about what parameters are supported, how to specify them, and what default values are assumed (see also *Existing HPX performance counters*).

Every *locality* of an application exposes its own set of performance counter types and performance counter instances. The set of exposed counters is determined dynamically at application start based on the execution environment of the application. For instance, this set is influenced by the current hardware environment for the *locality* (such as whether the *locality* has access to accelerators), and the software environment of the application (such as the number of OS threads used to execute *HPX* threads).

Using wildcards in performance counter names

It is possible to use wildcard characters when specifying performance counter names. Performance counter names can contain two types of wildcard characters:

- Wildcard characters in the performance counter type
- Wildcard characters in the performance counter instance name

A wildcard character has a meaning which is very close to usual file name wildcard matching rules implemented by common shells (like bash).

Table 2.26: Wildcard characters in the performance counter type

Wildcard	Description
`*`	This wildcard character matches any number (zero or more) of arbitrary characters.
`?`	This wildcard character matches any single arbitrary character.
`[...]`	This wildcard character matches any single character from the list of specified within the square brackets.

Table 2.27: Wildcard characters in the performance counter instance name

Wildcard	Description
`*`	This wildcard character matches any *locality* or any thread, depending on whether it is used for `locality#*` or `worker-thread#*`. No other wildcards are allowed in counter instance names.

Consuming performance counter data

You can consume performance data using either the command line interface, the *HPX* application or the *HPX* API. The command line interface is easier to use, but it is less flexible and does not allow one to adjust the behaviour of your application at runtime. The command line interface provides a convenience abstraction but simplified abstraction for querying and logging performance counter data for a set of performance counters.

Consuming performance counter data from the command line

HPX provides a set of predefined command line options for every application that uses `hpx::init` for its initialization. While there are many more command line options available (see *HPX Command Line Options*), the set of options related to performance counters allows one to list existing counters, and query existing counters once at application termination or repeatedly after a constant time interval.

The following table summarizes the available command line options:

Table 2.28: *HPX* Command Line Options Related to Performance Counters

Command line option	Description
`--hpx:pr`	Prints the specified performance counter either repeatedly and/or at the times specified by `--hpx:print-counter-at` (see also option `--hpx:print-counter-interval`).
`--hpx:pr`	Prints the specified performance counter either repeatedly and/or at the times specified by `--hpx:print-counter-at`. Reset the counter after the value is queried (see also option `--hpx:print-counter-interval`).
`--hpx:pr`	Prints the performance counter(s) specified with `--hpx:print-counter` repeatedly after the time interval (specified in milliseconds) (default:`0` which means print once at shutdown).
`--hpx:pr`	Prints the performance counter(s) specified with `--hpx:print-counter` to the given file (default: console).
`--hpx:li`	Lists the names of all registered performance counters.
`--hpx:li`	Lists the description of all registered performance counters.
`--hpx:pr`	Prints the performance counter(s) specified with `--hpx:print-counter`. Possible formats in CVS format with header or without any header (see option `--hpx:no-csv-header`), possible values: `csv` (prints counter values in CSV format with full names as header) `csv-short` (prints counter values in CSV format with shortnames provided with `--hpx:print-counter` as `--hpx:print-counter shortname,full-countername`).
`--hpx:no`	Prints the performance counter(s) specified with `--hpx:print-counter` and `csv` or `csv-short` format specified with `--hpx:print-counter-format` without header.
`--hpx:pr arg`	Prints the performance counter(s) specified with `--hpx:print-counter` (or `--hpx:print-counter-reset`) at the given point in time. Possible argument values: `startup`, `shutdown` (default), `noshutdown`.
`--hpx:re`	Resets all performance counter(s) specified with `--hpx:print-counter` after they have been evaluated.
`--hpx:pr`	Appends counter type description to generated output.
`--hpx:pr`	Each locality prints only its own local counters.

While the options `--hpx:list-counters` and `--hpx:list-counter-infos` give a short list of all available counters, the full documentation for those can be found in the section *Existing HPX performance counters*.

HPX-Top

HPX-Top is a real-time, terminal-based dashboard designed for monitoring the health and performance of distributed *HPX* applications. Inspired by tools like `htop` and `btop`, it provides a high-level overview of system utilization, network flow, and internal runtime metrics.

Fig. 2.9: HPX-Top Real-time Performance Dashboard

Features

- **Locality-Aware Monitoring**: Automatically detects and displays metrics for all localities in a running distributed application.
- **Thread Pool Utilization**: Visualizes real-time usage of worker threads with instantaneous counts and historical sparkline graphs.
- **Parcel Throughput**: Tracks incoming and outgoing parcels, helping to identify network-bound bottlenecks or communication hotspots.
- **AGAS Health Monitor**: Displays cache hit and miss rates for the Application Global Address Space (AGAS), crucial for diagnosing naming service bottlenecks.

Usage

HPX-Top is implemented as a Python script that acts as a wrapper around your *HPX* application. It automatically configures the necessary performance counters and output formats.

The tool requires the `rich` Python library for rendering the terminal UI:

```
$ pip install rich
```

To monitor an *HPX* application, launch it through `hpx-top.py`:

```
$ python3 tools/hpx-top.py ./your_application --hpx:threads=4 [other HPX flags]
```

You can also run HPX-Top in mock mode to explore the interface without a running *HPX* application:

```
$ python3 tools/hpx-top.py --mock
```

Technical Details

The tool leverages *HPX*'s built-in performance counter framework. It launches the target application with specific flags (`--hpx:print-counter-interval` and `--hpx:print-counter-format=csv-short`) and parses the standard output in a non-blocking background thread to update the TUI.

HPX Smart Telemetry (hpx_stat_viewer)

HPX Smart Telemetry (`hpx_stat_viewer.py`) is an intelligent, lightweight performance dashboard designed to identify performance anomalies in real-time. While `hpx-top` provides a broad system overview, `hpx_stat_viewer` focuses on deep-dive analysis of specific metrics with built-in heuristic alerting.

Fig. 2.10: HPX Smart Telemetry Dashboard with Anomaly Detection

Features

- **Heuristic Anomaly Detection**: Uses an Exponential Moving Average (EMA) to monitor counter trends. It automatically flags sudden spikes (+150%) or drops (-60%) with prominent visual alerts like `[ SPIKE]` and `[ DROP]`.
- **Live History Sparklines**: Displays 10-tick historical trend graphs next to every metric, helping developers distinguish between momentary noise and persistent bottlenecks.
- **Smart Filtering and Grouping**: Supports regex-based counter filtering and automatically groups metrics by locality for large-scale distributed runs.
- **Zero-Dependency Core**: Requires no external TUI libraries, making it highly portable for various terminal environments.

Usage

`hpx_stat_viewer.py` reads HPX counter data from standard input in `csv-short` transient format. This allows it to be used for both live monitoring and post-mortem analysis of log files.

To monitor a live application with smart filtering:

```
$ ./your_hpx_app --hpx:print-counter-interval=500 --hpx:print-counter-format=csv-short ... | \
  python3 tools/hpx_stat_viewer.py --filter "threads|utilization"
```

To replay an offline performance log for analysis:

```
$ python3 tools/hpx_stat_viewer.py --replay your_hpx_counters.csv --speed 0.5
```

Technical Details

The dashboard implements a thread-safe parser that handles HPX's CSV-short format. The anomaly detection engine uses a smoothing factor of 0.4 for its EMA calculations, balancing responsiveness with stability. It is officially integrated into the HPX installation rules under the `tools` component.

A simple example

All of the commandline options mentioned above can be tested using the `hello_world_distributed` example.

Listing all available counters `hello_world_distributed --hpx:list-counters` yields:

```
List of available counter instances (replace * below with the appropriate
sequence number)
-------------------------------------------------------------------------
/agas/count/allocate /agas/count/bind /agas/count/bind_gid
/agas/count/bind_name ... /threads{locality#*/allocator#*}/count/objects
/threads{locality#*/total}/count/stack-recycles
/threads{locality#*/total}/idle-rate
/threads{locality#*/worker-thread#*}/idle-rate
```

Providing more information about all available counters, `hello_world_distributed --hpx:list-counter-infos` yields:

```
Information about available counter instances (replace * below with the
appropriate sequence number)
------------------------------------------------------------------------------
fullname: /agas/count/allocate helptext: returns the number of invocations of
the AGAS service 'allocate' type: counter_type::raw version: 1.0.0
------------------------------------------------------------------------------

------------------------------------------------------------------------------
fullname: /agas/count/bind helptext: returns the number of invocations of the
AGAS service 'bind' type: counter_type::raw version: 1.0.0
------------------------------------------------------------------------------

------------------------------------------------------------------------------
fullname: /agas/count/bind_gid helptext: returns the number of invocations of
the AGAS service 'bind_gid' type: counter_type::raw version: 1.0.0
------------------------------------------------------------------------------

...
```

This command will not only list the counter names but also a short description of the data exposed by this counter.

> **Note**
>
> The list of available counters may differ depending on the concrete execution environment (hardware or software) of your application.

Requesting the counter data for one or more performance counters can be achieved by invoking `hello_world_distributed` with a list of counter names:

```
$ hello_world_distributed \
    --hpx:print-counter=/threads{locality#0/total}/count/cumulative \
    --hpx:print-counter=/agas{locality#0/total}/count/bind
```

which yields for instance:

```
hello world from OS-thread 0 on locality 0
/threads{locality#0/total}/count/cumulative,1,0.212527,[s],33
/agas{locality#0/total}/count/bind,1,0.212790,[s],11
```

The first line is the normal output generated by `hello_world_distributed` and has no relation to the counter data listed. The last two lines contain the counter data as gathered at application shutdown. These lines have six fields, the counter name, the sequence number of the counter invocation, the time stamp at which this information has been sampled, the unit of measure for the time stamp, the actual counter value and an optional unit of measure for the counter value.

Note

The command line option `--hpx:print-counter-types` will append a seventh field to the generated output. This field will hold an abbreviated counter type.

The actual counter value can be represented by a single number (for counters returning singular values) or a list of numbers separated by `':'` (for counters returning an array of values, like for instance a histogram).

Note

The name of the performance counter will be enclosed in double quotes `'"'` if it contains one or more commas `','`.

Requesting to query the counter data once after a constant time interval with this command line:

```
$ hello_world_distributed \
    --hpx:print-counter=/threads{locality#0/total}/count/cumulative \
    --hpx:print-counter=/agas{locality#0/total}/count/bind \
    --hpx:print-counter-interval=20
```

yields for instance (leaving off the actual console output of the `hello_world_distributed` example for brevity):

```
threads{locality#0/total}/count/cumulative,1,0.002409,[s],22
agas{locality#0/total}/count/bind,1,0.002542,[s],9
threads{locality#0/total}/count/cumulative,2,0.023002,[s],41
agas{locality#0/total}/count/bind,2,0.023557,[s],10
threads{locality#0/total}/count/cumulative,3,0.037514,[s],46
agas{locality#0/total}/count/bind,3,0.038679,[s],10
```

The command `--hpx:print-counter-destination=<file>` will redirect all counter data gathered to the specified file name, which avoids cluttering the console output of your application.

The command line option `--hpx:print-counter` supports using a limited set of wildcards for a (very limited) set of use cases. In particular, all occurrences of `#*` as in `locality#*` and in `worker-thread#*` will be automatically expanded to the proper set of performance counter names representing the actual environment for the executed program. For instance, if your program is utilizing four worker threads for the execution of *HPX* threads (see command line option *--hpx:threads*) the following command line

```
$ hello_world_distributed \
    --hpx:threads=4 \
    --hpx:print-counter=/threads{locality#0/worker-thread#*}/count/cumulative
```

will print the value of the performance counters monitoring each of the worker threads:

```
hello world from OS-thread 1 on locality 0
hello world from OS-thread 0 on locality 0
hello world from OS-thread 3 on locality 0
hello world from OS-thread 2 on locality 0
/threads{locality#0/worker-thread#0}/count/cumulative,1,0.0025214,[s],27
/threads{locality#0/worker-thread#1}/count/cumulative,1,0.0025453,[s],33
/threads{locality#0/worker-thread#2}/count/cumulative,1,0.0025683,[s],29
/threads{locality#0/worker-thread#3}/count/cumulative,1,0.0025904,[s],33
```

The command `--hpx:print-counter-format` takes values `csv` and `csv-short` to generate CSV formatted counter values with a header.

With format as csv:

```
$ hello_world_distributed \
    --hpx:threads=2 \
    --hpx:print-counter-format csv \
    --hpx:print-counter /threads{locality#*/total}/count/cumulative \
    --hpx:print-counter /threads{locality#*/total}/count/cumulative-phases
```

will print the values of performance counters in CSV format with the full countername as a header:

```
hello world from OS-thread 1 on locality 0
hello world from OS-thread 0 on locality 0
/threads{locality#*/total}/count/cumulative,/threads{locality#*/total}/count/cumulative-
↪phases
39,93
```

With format csv-short:

```
$ hello_world_distributed \
    --hpx:threads 2 \
    --hpx:print-counter-format csv-short \
    --hpx:print-counter cumulative,/threads{locality#*/total}/count/cumulative \
    --hpx:print-counter phases,/threads{locality#*/total}/count/cumulative-phases
```

will print the values of performance counters in CSV format with the short countername as a header:

```
hello world from OS-thread 1 on locality 0
hello world from OS-thread 0 on locality 0
cumulative,phases
39,93
```

With format csv and csv-short when used with `--hpx:print-counter-interval`:

```
$ hello_world_distributed \
    --hpx:threads 2 \
    --hpx:print-counter-format csv-short \
    --hpx:print-counter cumulative,/threads{locality#*/total}/count/cumulative \
    --hpx:print-counter phases,/threads{locality#*/total}/count/cumulative-phases \
    --hpx:print-counter-interval 5
```

will print the header only once repeating the performance counter value(s) repeatedly:

```
cum,phases
25,42
hello world from OS-thread 1 on locality 0
hello world from OS-thread 0 on locality 0
44,95
```

The command `--hpx:no-csv-header` can be used with `--hpx:print-counter-format` to print performance counter values in CSV format without any header:

```
$ hello_world_distributed \
--hpx:threads 2 \
--hpx:print-counter-format csv-short \
--hpx:print-counter cumulative,/threads{locality#*/total}/count/cumulative \
--hpx:print-counter phases,/threads{locality#*/total}/count/cumulative-phases \
--hpx:no-csv-header
```

will print:

```
hello world from OS-thread 1 on locality 0
hello world from OS-thread 0 on locality 0
37,91
```

Consuming performance counter data using the *HPX* API

HPX provides an API that allows users to discover performance counters and to retrieve the current value of any existing performance counter from any application.

Discover existing performance counters

Retrieve the current value of any performance counter

Performance counters are specialized *HPX* components. In order to retrieve a counter value, the performance counter needs to be instantiated. *HPX* exposes a client component object for this purpose:

```
hpx::performance_counters::performance_counter counter(std::string const& name);
```

Instantiating an instance of this type will create the performance counter identified by the given `name`. Only the first invocation for any given counter name will create a new instance of that counter. All following invocations for a given counter name will reference the initially created instance. This ensures that at any point in time there is never more than one active instance of any of the existing performance counters.

In order to access the counter value (or to invoke any of the other functionality related to a performance counter, like `start`, `stop` or `reset`) member functions of the created client component instance should be called:

```
// print the current number of threads created on locality 0
hpx::performance_counters::performance_counter count(
    "/threads{locality#0/total}/count/cumulative");
hpx::cout << count.get_value<int>().get() << std::endl;
```

For more information about the client component type, see `hpx::performance_counters::performance_counter`

Note

In the above example `count.get_value()` returns a future. In order to print the result we must append `.get()` to retrieve the value. You could write the above example like this for more clarity:

```
// print the current number of threads created on locality 0
hpx::performance_counters::performance_counter count(
    "/threads{locality#0/total}/count/cumulative");
hpx::future<int> result = count.get_value<int>();
hpx::cout << result.get() << std::endl;
```

Providing performance counter data

HPX offers several ways by which you may provide your own data as a performance counter. This has the benefit of exposing additional, possibly application-specific information using the existing Performance Counter framework, unifying the process of gathering data about your application.

An application that wants to provide counter data can implement a performance counter to provide the data. When a consumer queries performance data, the *HPX* runtime system calls the provider to collect the data. The runtime system uses an internal registry to determine which provider to call.

Generally, there are two ways of exposing your own performance counter data: a simple, function-based way and a more complex, but more powerful way of implementing a full performance counter. Both alternatives are described in the following sections.

Exposing performance counter data using a simple function

The simplest way to expose arbitrary numeric data is to write a function which will then be called whenever a consumer queries this counter. Currently, this type of performance counter can only be used to expose integer values. The expected signature of this function is:

```
std::int64_t some_performance_data(bool reset);
```

The argument `bool reset` (which is supplied by the runtime system when the function is invoked) specifies whether the counter value should be reset after evaluating the current value (if applicable).

For instance, here is such a function returning how often it was invoked:

```
// The atomic variable 'counter' ensures the thread safety of the counter.
boost::atomic<std::int64_t> counter(0);

std::int64_t some_performance_data(bool reset)
{
    std::int64_t result = ++counter;
    if (reset)
```

(continues on next page)

(continued from previous page)

```
        counter = 0;
    return result;
}
```

This example function exposes a linearly-increasing value as our performance data. The value is incremented on each invocation, i.e., each time a consumer requests the counter data of this performance counter.

The next step in exposing this counter to the runtime system is to register the function as a new raw counter type using the *HPX* API function `hpx::performance_counters::install_counter_type`. A counter type represents certain common characteristics of counters, like their counter type name and any associated description information. The following snippet shows an example of how to register the function `some_performance_data`, which is shown above, for a counter type named `"/test/data"`. This registration has to be executed before any consumer instantiates, and queries an instance of this counter type:

```
#include <hpx/include/performance_counters.hpp>

void register_counter_type()
{
    // Call the HPX API function to register the counter type.
    hpx::performance_counters::install_counter_type(
        "/test/data",                                      // counter type name
        &some_performance_data,                            // function providing counter␣
↪data
        "returns a linearly increasing counter value"    // description text (optional)
        ""                                                 // unit of measure (optional)
    );
}
```

Now it is possible to instantiate a new counter instance based on the naming scheme `"/test{locality#*/total}/data"` where `*` is a zero-based integer index identifying the *locality* for which the counter instance should be accessed. The function `hpx::performance_counters::install_counter_type` enables users to instantiate exactly one counter instance for each *locality*. Repeated requests to instantiate such a counter will return the same instance, i.e., the instance created for the first request.

If this counter needs to be accessed using the standard *HPX* command line options, the registration has to be performed during application startup, before `hpx_main` is executed. The best way to achieve this is to register an *HPX* startup function using the API function *hpx::register_startup_function* before calling `hpx::init` to initialize the runtime system:

```
int main(int argc, char* argv[])
{
    // By registering the counter type we make it available to any consumer
    // who creates and queries an instance of the type "/test/data".
    //
    // This registration should be performed during startup. The
    // function 'register_counter_type' should be executed as an HPX thread right
    // before hpx_main is executed.
    hpx::register_startup_function(&register_counter_type);

    // Initialize and run HPX.
    return hpx::init(argc, argv);
}
```

Please see the code in `simplest_performance_counter.cpp` for a full example demonstrating this functionality.

Implementing a full performance counter

Sometimes, the simple way of exposing a single value as a performance counter is not sufficient. For that reason, *HPX* provides a means of implementing full performance counters which support:

- Retrieving the descriptive information about the performance counter
- Retrieving the current counter value
- Resetting the performance counter (value)
- Starting the performance counter
- Stopping the performance counter
- Setting the (initial) value of the performance counter

Every full performance counter will implement a predefined interface:

```
//  Copyright (c) 2007-2026 Hartmut Kaiser
//
//  SPDX-License-Identifier: BSL-1.0
//  Distributed under the Boost Software License, Version 1.0. (See accompanying
//  file LICENSE_1_0.txt or copy at http://www.boost.org/LICENSE_1_0.txt)

#pragma once

#include <hpx/config.hpp>
#include <hpx/modules/async_base.hpp>
#include <hpx/modules/components.hpp>
#include <hpx/modules/execution.hpp>
#include <hpx/modules/functional.hpp>
#include <hpx/modules/futures.hpp>

#include <hpx/performance_counters/counters_fwd.hpp>
#include <hpx/performance_counters/server/base_performance_counter.hpp>

#include <string>
#include <utility>
#include <vector>

#include <hpx/config/warnings_prefix.hpp>

///////////////////////////////////////////////////////////////////////////////
namespace hpx::performance_counters {

    ///////////////////////////////////////////////////////////////////////////
    HPX_CXX_EXPORT struct HPX_EXPORT performance_counter
      : components::client_base<performance_counter,
            server::base_performance_counter>
    {
        using base_type = components::client_base<performance_counter,
            server::base_performance_counter>;

        performance_counter() = default;

        explicit performance_counter(std::string const& name);
```

(continues on next page)

(continued from previous page)

```
    performance_counter(
        std::string const& name, hpx::id_type const& locality);

    performance_counter(id_type const& id)
      : base_type(id)
    {
    }

    performance_counter(future<id_type>&& id)
      : base_type(HPX_MOVE(id))
    {
    }

    performance_counter(hpx::future<performance_counter>&& c)
      : base_type(HPX_MOVE(c))
    {
    }

    ///////////////////////////////////////////////////////////////////////
    future<counter_info> get_info() const;
    counter_info get_info(
        launch::sync_policy, error_code& ec = throws) const;

    future<counter_value> get_counter_value(bool reset) const;
    counter_value get_counter_value(
        launch::sync_policy, bool reset, error_code& ec = throws) const;

    future<counter_value> get_counter_value() const;
    counter_value get_counter_value(
        launch::sync_policy, error_code& ec = throws) const;

    future<counter_values_array> get_counter_values_array(bool reset) const;
    counter_values_array get_counter_values_array(
        launch::sync_policy, bool reset, error_code& ec = throws) const;

    future<counter_values_array> get_counter_values_array() const;
    counter_values_array get_counter_values_array(
        launch::sync_policy, error_code& ec = throws) const;

    ///////////////////////////////////////////////////////////////////////
    future<bool> start() const;
    bool start(launch::sync_policy, error_code& ec = throws) const;

    future<bool> stop() const;
    bool stop(launch::sync_policy, error_code& ec = throws) const;

    future<void> reset() const;
    void reset(launch::sync_policy, error_code& ec = throws) const;

    future<void> reinit(bool reset = true) const;
    void reinit(launch::sync_policy, bool reset = true,
```

(continues on next page)

(continued from previous page)

```
            error_code& ec = throws) const;

        ///////////////////////////////////////////////////////////////////////
        future<std::string> get_name() const;
        std::string get_name(
            launch::sync_policy, error_code& ec = throws) const;

    private:
        template <typename T>
        static T extract_value(future<counter_value>&& value)
        {
            return value.get().get_value<T>();
        }

    public:
        template <typename T>
        future<T> get_value(bool reset = false)
        {
            return get_counter_value(reset).then(hpx::launch::sync,
                hpx::bind_front(&performance_counter::extract_value<T>));
        }
        template <typename T>
        T get_value(
            launch::sync_policy, bool reset = false, error_code& ec = throws)
        {
            return get_counter_value(launch::sync, reset).get_value<T>(ec);
        }

        template <typename T>
        future<T> get_value() const
        {
            return get_counter_value(false).then(hpx::launch::sync,
                hpx::bind_front(&performance_counter::extract_value<T>));
        }
        template <typename T>
        T get_value(launch::sync_policy, error_code& ec = throws) const
        {
            return get_counter_value(launch::sync, false).get_value<T>(ec);
        }
    };

    // Return all counters matching the given name (with optional wild cards).
    HPX_CXX_EXPORT HPX_EXPORT std::vector<performance_counter>
    discover_counters(std::string const& name, error_code& ec = throws);
}    // namespace hpx::performance_counters

#include <hpx/config/warnings_suffix.hpp>
```

In order to implement a full performance counter, you have to create an *HPX* component exposing this interface. To simplify this task, *HPX* provides a ready-made base class which handles all the boiler plate of creating a component for you. The remainder of this section will explain the process of creating a full performance counter based on the Sine example, which you can find in the directory `examples/performance_counters/sine/`.

The base class is defined in the header file `base_performance_counter.cpp` as:

```
//  Copyright (c) 2007-2026 Hartmut Kaiser
//
//  SPDX-License-Identifier: BSL-1.0
//  Distributed under the Boost Software License, Version 1.0. (See accompanying
//  file LICENSE_1_0.txt or copy at http://www.boost.org/LICENSE_1_0.txt)

#pragma once

#include <hpx/config.hpp>
#include <hpx/modules/actions_base.hpp>
#include <hpx/modules/components_base.hpp>
#include <hpx/modules/runtime_local.hpp>

#include <hpx/performance_counters/counters.hpp>
#include <hpx/performance_counters/server/base_performance_counter.hpp>

#if defined(DOXYGEN)
///////////////////////////////////////////////////////////////////////////////
//[performance_counter_base_class
namespace hpx::performance_counters {

    template <typename Derived>
    class base_performance_counter;
}    // namespace hpx::performance_counters
//]
#endif

///////////////////////////////////////////////////////////////////////////////
namespace hpx::performance_counters {

    HPX_CXX_EXPORT template <typename Derived>
    class base_performance_counter
      : public hpx::performance_counters::server::base_performance_counter
      , public hpx::components::component_base<Derived>
    {
    private:
        using base_type = hpx::components::component_base<Derived>;

    public:
        using type_holder = Derived;
        using base_type_holder =
            hpx::performance_counters::server::base_performance_counter;

        // NOLINTBEGIN(bugprone-crtp-constructor-accessibility)
        base_performance_counter() = default;

        explicit base_performance_counter(
            hpx::performance_counters::counter_info const& info)
          : base_type_holder(info)
        {
        }
```

(continues on next page)

(continued from previous page)

```
        // NOLINTEND(bugprone-crtp-constructor-accessibility)

        // Disambiguate finalize() which is implemented in both base classes
        void finalize()
        {
            base_type_holder::finalize();
            base_type::finalize();
        }

        hpx::naming::address get_current_address() const
        {
            return hpx::naming::address(
                hpx::naming::get_gid_from_locality_id(hpx::get_locality_id()),
                hpx::components::get_component_type<Derived>(),
                const_cast<Derived*>(static_cast<Derived const*>(this)));
        }
    };
}    // namespace hpx::performance_counters
```

The single template parameter is expected to receive the type of the derived class implementing the performance counter. In the Sine example this looks like:

```
//  Copyright (c) 2007-2012 Hartmut Kaiser
//
//  SPDX-License-Identifier: BSL-1.0
//  Distributed under the Boost Software License, Version 1.0. (See accompanying
//  file LICENSE_1_0.txt or copy at http://www.boost.org/LICENSE_1_0.txt)

#pragma once

#include <hpx/config.hpp>
#if !defined(HPX_COMPUTE_DEVICE_CODE)
#include <hpx/hpx.hpp>
#include <hpx/include/lcos_local.hpp>
#include <hpx/include/performance_counters.hpp>
#include <hpx/include/util.hpp>

#include <cstdint>

namespace performance_counters { namespace sine { namespace server {
    ///////////////////////////////////////////////////////////////////////////
    //[sine_counter_definition
    class sine_counter
      : public hpx::performance_counters::base_performance_counter<sine_counter>
    //]
    {
    public:
        sine_counter()
          : current_value_(0)
          , evaluated_at_(0)
        {
        }
```

(continues on next page)

(continued from previous page)

```
        explicit sine_counter(
            hpx::performance_counters::counter_info const& info);

        /// This function will be called in order to query the current value of
        /// this performance counter
        hpx::performance_counters::counter_value get_counter_value(bool reset);

        /// The functions below will be called to start and stop collecting
        /// counter values from this counter.
        bool start();
        bool stop();

        /// finalize() will be called just before the instance gets destructed
        void finalize();

    protected:
        bool evaluate();

    private:
        typedef hpx::spinlock mutex_type;

        mutable mutex_type mtx_;
        double current_value_;
        std::uint64_t evaluated_at_;

        hpx::util::interval_timer timer_;
    };
}}}    // namespace performance_counters::sine::server
#endif
```

i.e., the type `sine_counter` is derived from the base class passing the type as a template argument (please see `simplest_performance_counter.cpp` for the full source code of the counter definition). For more information about this technique (called Curiously Recurring Template Pattern - CRTP), please see for instance the corresponding Wikipedia article[182]. This base class itself is derived from the `performance_counter` interface described above.

Additionally, a full performance counter implementation not only exposes the actual value but also provides information about:

- The point in time a particular value was retrieved.
- A (sequential) invocation count.
- The actual counter value.
- An optional scaling coefficient.
- Information about the counter status.

[182] http://en.wikipedia.org/wiki/Curiously_recurring_template_pattern

Existing *HPX* performance counters

The *HPX* runtime system exposes a wide variety of predefined performance counters. These counters expose critical information about different modules of the runtime system. They can help determine system bottlenecks and fine-tune system and application performance.

Table 2.29: AGAS performance counter /agas/count/<agas_service>

Counter type	/agas/count/<agas_service> where <agas_service> is one of the following: *primary namespace services*: route, bind_gid, resolve_gid, unbind_gid, increment_credit, decrement_credit, allocate, begin_migration, end_migration *component namespace services*: bind_prefix, bind_name, resolve_id, unbind_name, iterate_types, get_component_typename, num_localities_type *locality namespace services*: free, localities, num_localities, num_threads, resolve_locality, resolved_localities *symbol namespace services*: bind, resolve, unbind, iterate_names, on_symbol_namespace_event
Counter instance formatting	<agas_instance>/total where <agas_instance> is the name of the *AGAS* service to query. Currently, this value will be locality#0 where 0 is the root *locality* (the id of the locality hosting the *AGAS* service). The value for * can be any *locality* id for the following <agas_service>: route, bind_gid, resolve_gid, unbind_gid, increment_credit, decrement_credit, bin, resolve, unbind, and iterate_names (only the primary and symbol *AGAS* service components live on all localities, whereas all other *AGAS* services are available on locality#0 only).
Description	Returns the total number of invocations of the specified *AGAS* service since its creation.

Table 2.30: AGAS performance counter /agas/<agas_service_category>/count

Counter type	/agas/<agas_service_category>/count where <agas_service_category> is one of the following: primary, locality, component or symbol
Counter instance formatting	<agas_instance>/total where <agas_instance> is the name of the *AGAS* service to query. Currently, this value will be locality#0 where 0 is the root *locality* (the id of the *locality* hosting the *AGAS* service). Except for <agas_service_category>, primary or symbol for which the value for * can be any *locality* id (only the primary and symbol *AGAS* service components live on all localities, whereas all other *AGAS* services are available on locality#0 only).
Description	Returns the overall total number of invocations of all *AGAS* services provided by the given *AGAS* service category since its creation.

Table 2.31: AGAS performance counter /agas/<agas_service_category>/count

Counter type	/agas/<agas_service_category>/count where <agas_service_category> is one of the following: primary, locality, component or symbol
Counter instance formatting	<agas_instance>/total where <agas_instance> is the name of the *AGAS* service to query. Currently, this value will be locality#0 where 0 is the root *locality* (the id of the *locality* hosting the *AGAS* service). Except for <agas_service_category>, primary or symbol for which the value for * can be any *locality* id (only the primary and symbol *AGAS* service components live on all localities, whereas all other *AGAS* services are available on locality#0 only).
Description	Returns the overall total number of invocations of all *AGAS* services provided by the given *AGAS* service category since its creation.

Table 2.32: AGAS performance counter agas/time/<agas_service>

Counter type	agas/time/<agas_service> where <agas_service> is one of the following: *primary namespace services*: route, bind_gid, resolve_gid, unbind_gid, increment_credit, decrement_credit, allocate begin_migration, end_migration *component namespace services*: bind_prefix, bind_name, resolve_id, unbind_name, iterate_types, get_component_typename, num_localities_type *locality namespace services*: free, localities, num_localities, num_threads, resolve_locality, resolved_localities *symbol namespace services*: bind, resolve, unbind, iterate_names, on_symbol_namespace_event
Counter instance formatting	<agas_instance>/total where <agas_instance> is the name of the *AGAS* service to query. Currently, this value will be locality#0 where 0 is the root *locality* (the id of the *locality* hosting the *AGAS* service). The value for * can be any *locality* id for the following <agas_service>: route, bind_gid, resolve_gid, unbind_gid, increment_credit, decrement_credit, bin, resolve, unbind, and iterate_names (only the primary and symbol *AGAS* service components live on all localities, whereas all other *AGAS* services are available on locality#0 only).
Description	Returns the overall execution time of the specified *AGAS* service since its creation (in nanoseconds).

Table 2.33: AGAS performance counter */agas/<agas_service_category>/time*`

Counter type	`/agas/<agas_service_category>/time` where `<agas_service_category>` is one of the following: `primary`, `locality`, `component` or `symbol`
Counter instance formatting	`<agas_instance>/total` where `<agas_instance>` is the name of the *AGAS* service to query. Currently, this value will be `locality#0` where `0` is the root *locality* (the id of the *locality* hosting the *AGAS* service). Except for `<agas_service_category` `primary` or `symbol` for which the value for * can be any *locality* id (only the primary and symbol *AGAS* service components live on all localities, whereas all other *AGAS* services are available on `locality#0` only).
Description	Returns the overall execution time of all *AGAS* services provided by the given *AGAS* service category since its creation (in nanoseconds).

Table 2.34: AGAS performance counter `/agas/count/entries`

Counter type	`/agas/count/entries`
Counter instance formatting	`locality#*/total` where * is the *locality* id of the *locality* the *AGAS* cache should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the number of cache entries resident in the *AGAS* cache of the specified *locality* (see `<cache_statistics>`).

Table 2.35: AGAS performance counter `/agas/count/<cache_statistics>`

Counter type	`/agas/count/<cache_statistics>` where `<cache_statistics>` is one of the following: `cache/evictions`, `cache/hits`, `cache/insertions`, `cache/misses`
Counter instance formatting	`locality#*/total` where * is the *locality* id of the *locality* the *AGAS* cache should be queried. The *locality* id is a (zero based) number identifying the *locality*
Description	Returns the number of cache events (evictions, hits, inserts, and misses) in the *AGAS* cache of the specified *locality* (see `<cache_statistics>`).

Table 2.36: AGAS performance counter `/agas/count/<full_cache_statistics>`

Counter type	`/agas/count/<full_cache_statistics>` where `<full_cache_statistics>` is one of the following: `cache/get_entry`, `cache/insert_entry`, `cache/update_entry`, `cache/erase_entry`
Counter instance formatting	`locality#*/total` where * is the *locality* id of the *locality* the *AGAS* cache should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the number of invocations of the specified cache API function of the *AGAS* cache.

Table 2.37: AGAS performance counter `/agas/time/<full_cache_statistics>`

Counter type	`/agas/time/<full_cache_statistics>` where `<full_cache_statistics>` is one of the following: `cache/get_entry`, `cache/insert_entry`, `cache/update_entry`, `cache/erase_entry`
Counter instance formatting	`locality#*/total` where `*` is the *locality* id of the *locality* the *AGAS* cache should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the overall time spent executing of the specified API function of the *AGAS* cache.

Table 2.38: Parcel layer performance counter `/data/count/<connection_type>/<operation>`

Counter type	`/data/count/<connection_type>/<operation>`
	where: `<operation>` is one of the following: `sent`, `received`
	`<connection_type>` is one of the following: `tcp`, `mpi`

Counter instance formatting	`locality#*/total` where `*` is the *locality* id of the *locality* the overall number of transmitted bytes should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the overall number of raw (uncompressed) bytes sent or received (see `<operation>`, e.g. `sent` or `received`) for the specified `<connection_type>`. The performance counters are available only if the compile time constant `HPX_HAVE_PARCELPORT_COUNTERS` was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is `HPX_WITH_PARCELPORT_COUNTERS`. The performance counters for the connection type `mpi` are available only if the compile time constant `HPX_HAVE_PARCELPORT_MPI` was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is `HPX_WITH_PARCELPORT_MPI`. Please see *CMake options* for more details.

Table 2.39: Parcel layer performance counter /data/time/<connection_type>/<operation>

Counter type	/data/time/<connection_type>/<operation> where: <operation> is one of the following: sent, received <connection_type> is one of the following: tcp, mpi
Counter instance formatting	locality#*/total where * is the *locality* id of the *locality* the total transmission time should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the total time (in nanoseconds) between the start of each asynchronous transmission operation and the end of the corresponding operation for the specified <connection_type> the given *locality* (see <operation>, e.g. sent or received). The performance counters are available only if the compile time constant HPX_HAVE_PARCELPORT_COUNTERS was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_COUNTERS. The performance counters for the connection type mpi are available only if the compile time constant HPX_HAVE_PARCELPORT_MPI was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_MPI. Please see *CMake options* for more details.

Table 2.40: Parcel layer performance counter /serialize/count/<connection_type>/<operation>

Counter type	/serialize/count/<connection_type>/<operation> where: <operation> is one of the following: sent, received <connection_type> is one of the following: tcp, mpi
Counter instance formatting	locality#*/total where * is the *locality* id of the *locality* the overall number of transmitted bytes should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the overall number of bytes transferred (see <operation>, e.g. sent or received possibly compressed) for the specified <connection_type> by the given *locality*. The performance counters are available only if the compile time constant HPX_HAVE_PARCELPORT_COUNTERS was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_COUNTERS. The performance counters for the connection type mpi are available only if the compile time constant HPX_HAVE_PARCELPORT_MPI was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_MPI. Please see *CMake options* for more details.
Description	If the configure-time option -DHPX_WITH_PARCELPORT_ACTION_COUNTERS=On was specified, this counter allows one to specify an optional action name as its parameter. In this case the counter will report the number of bytes transmitted for the given action only.

Table 2.41: Parcel layer performance counter /serialize/time/<connection_type>/<operation>

Counter type	/serialize/time/<connection_type>/<operation> where: <operation> is one of the following: sent, received <connection_type> is one of the following: tcp, mpi
Counter instance formatting	locality#*/total where * is the *locality* id of the *locality* the serialization time should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the overall time spent performing outgoing data serialization for the specified <connection_type> on the given *locality* (see <operation>, e.g. sent or received). The performance counters are available only if the compile time constant HPX_HAVE_PARCELPORT_COUNTERS was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_COUNTERS. The performance counters for the connection type mpi are available only if the compile time constant HPX_HAVE_PARCELPORT_MPI was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_MPI. Please see *CMake options* for more details.
Parameters	If the configure-time option -DHPX_WITH_PARCELPORT_ACTION_COUNTERS=On was specified, this counter allows one to specify an optional action name as its parameter. In this case the counter will report the serialization time for the given action only.

Table 2.42: Parcel layer performance counter /parcels/count/routed

Counter type	/parcels/count/routed
Counter instance formatting	locality#*/total where * is the *locality* id of the *locality* the number of routed parcels should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the overall number of routed (outbound) parcels transferred by the given *locality*. Routed parcels are those which cannot directly be delivered to its destination as the local *AGAS* is not able to resolve the destination address. In this case a parcel is sent to the *AGAS* service component which is responsible for creating the destination GID (and is responsible for resolving the destination address). This *AGAS* service component will deliver the parcel to its final target.
Parameters	If the configure-time option -DHPX_WITH_PARCELPORT_ACTION_COUNTERS=On was specified, this counter allows one to specify an optional action name as its parameter. In this case the counter will report the number of parcels for the given action only.

Table 2.43: Parcel layer performance counter `/parcels/count/<connection_type>/<operation>`

Counter type	`/parcels/count/<connection_type>/<operation>`
	where:
	`<operation>` is one of the following: `sent`, `received`
	`<connection_type>` is one of the following: `tcp`, `mpi`

Counter instance formatting	`locality#*/total` where `*` is the *locality* id of the *locality* the number of parcels should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the overall number of parcels transferred using the specified `<connection_type>` by the given *locality* (see `operation>`, e.g. `sent` or `received`. The performance counters are available only if the compile time constant `HPX_HAVE_PARCELPORT_COUNTERS` was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is `HPX_WITH_PARCELPORT_COUNTERS`. The performance counters for the connection type `mpi` are available only if the compile time constant `HPX_HAVE_PARCELPORT_MPI` was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is `HPX_WITH_PARCELPORT_MPI`. Please see *CMake options* for more details.

Table 2.44: Parcel layer performance counter `/messages/count/<connection_type>/<operation>`

Counter type	`/messages/count/<connection_type>/<operation>` where:
	`<operation>` is one of the following: `sent`, `received`
	`<connection_type>` is one of the following: `tcp`, `mpi`

Counter instance formatting	`locality#*/total` where `*` is the *locality* id of the *locality* the number of messages should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the overall number of messages[183] transferred using the specified `<connection_type>` by the given *locality* (see `<operation>`, e.g. `sent` or `received`) The performance counters are available only if the compile time constant `HPX_HAVE_PARCELPORT_COUNTERS` was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is `HPX_WITH_PARCELPORT_COUNTERS`. The performance counters for the connection type `mpi` are available only if the compile time constant `HPX_HAVE_PARCELPORT_MPI` was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is `HPX_WITH_PARCELPORT_MPI`. Please see *CMake options* for more details.

[183] A message can potentially consist of more than one *parcel*.

Table 2.45: Parcel layer performance counter /parcelport/count/<connection_type>/zero_copy_chunks/<operation>

Counter type	/parcelport/count/<connection_type>/zero_copy_chunks/<operation> where: <operation> is one of the following: sent, received <connection_type> is one of the following: tcp, mpi
Counter instance formatting	locality#*/total where * is the *locality* id of the *locality* the overall number of transmitted bytes should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the overall number of zero-copy chunks sent or received (see <operation>, e.g. sent or received) for the specified <connection_type>. The performance counters are available only if the compile time constant HPX_HAVE_PARCELPORT_COUNTERS was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_COUNTERS. The performance counters for the connection type mpi are available only if the compile time constant HPX_HAVE_PARCELPORT_MPI was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_MPI. Please see *CMake options* for more details.

Table 2.46: Parcel layer performance counter /parcelport/count-max/<connection_type>/zero_copy_chunks/<operation>

Counter type	/parcelport/count-max/<connection_type>/zero_copy_chunks/<operation> where: <operation> is one of the following: sent, received <connection_type> is one of the following: tcp, mpi
Counter instance formatting	locality#*/total where * is the *locality* id of the *locality* the overall number of transmitted bytes should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the maximum number of zero-copy chunks sent or received per message (see <operation>, e.g. sent or received) for the specified <connection_type>. The performance counters are available only if the compile time constant HPX_HAVE_PARCELPORT_COUNTERS was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_COUNTERS. The performance counters for the connection type mpi are available only if the compile time constant HPX_HAVE_PARCELPORT_MPI was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_MPI. Please see *CMake options* for more details.

Table 2.47: Parcel layer performance counter /parcelport/size/<connection_type>/zero_copy_chunks/<operation>

Counter type	/parcelport/size/<connection_type>/zero_copy_chunks/<operation> where: <operation> is one of the following: sent, received <connection_type> is one of the following: tcp, mpi
Counter instance formatting	locality#*/total where * is the *locality* id of the *locality* the overall number of transmitted bytes should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the overall size of zero-copy chunks sent or received (see <operation>, e.g. sent or received) for the specified <connection_type>. The performance counters are available only if the compile time constant HPX_HAVE_PARCELPORT_COUNTERS was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_COUNTERS. The performance counters for the connection type mpi are available only if the compile time constant HPX_HAVE_PARCELPORT_MPI was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_MPI. Please see *CMake options* for more details.

Table 2.48: Parcel layer performance counter /parcelport/size-max/<connection_type>/zero_copy_chunks/<operation>

Counter type	/parcelport/size-max/<connection_type>/zero_copy_chunks/<operation> where: <operation> is one of the following: sent, received <connection_type> is one of the following: tcp, mpi
Counter instance formatting	locality#*/total where * is the *locality* id of the *locality* the overall number of transmitted bytes should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the maximum size of zero-copy chunks sent or received (see <operation>, e.g. sent or received) for the specified <connection_type>. The performance counters are available only if the compile time constant HPX_HAVE_PARCELPORT_COUNTERS was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_COUNTERS. The performance counters for the connection type mpi are available only if the compile time constant HPX_HAVE_PARCELPORT_MPI was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_MPI. Please see *CMake options* for more details.

Table 2.49: Parcel layer performance counter /parcelport/count/<connection_type>/<cache_statistics>

Counter type	/parcelport/count/<connection_type>/<cache_statistics> where: <cache_statistics> is one of the following: cache/insertions, cache/evictions, cache/hits, cache/misses, cache/reclaims <connection_type> is one of the following: tcp, mpi
Counter instance formatting	locality#*/total where * is the *locality* id of the *locality* the number of messages should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the overall number cache events (evictions, hits, inserts, misses, and reclaims) for the connection cache of the given connection type on the given *locality* (see <cache_statistics, e.g. ache/insertions, cache/evictions, cache/hits, cache/misses or``cache/reclaims``. The performance counters for the connection type mpi are available only if the compile time constant HPX_HAVE_PARCELPORT_MPI was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_MPI. Please see *CMake options* for more details.

Table 2.50: Parcel layer performance counter /parcelport/count/<connection_type>/cache-reservation-failures

Counter type	/parcelport/count/<connection_type>/cache-reservation-failures where <connection_type> is one of the following: tcp, mpi
Counter instance formatting	locality#*/total where * is the *locality* id of the *locality* the counter should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the cumulative number of times get_or_reserve() returned false because every connection slot in the connection cache was already checked out (pool saturation). This is distinct from ordinary cache misses, which occur whenever a new connection to a *locality* is first created and do not cause parcel deferral. A non-zero and growing value of this counter means that outgoing parcels are being silently queued until a connection is reclaimed; raising hpx.max_connections or hpx.max_connections_per_locality in the ini configuration will relieve the saturation. When pool exhaustion is detected, *HPX* also emits a warning-level log message (throttled to at most once every five seconds) of the form: parcelport connection cache exhausted: N/M connections in use, K deferred reservation(s) since last reset. The performance counters for the connection type mpi are available only if the compile time constant HPX_HAVE_PARCELPORT_MPI was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_MPI. Please see *CMake options* for more details.

Table 2.51: Parcel layer performance counters /parcelport/count/<connection_type>/cache-connections and /parcelport/count/<connection_type>/cache-max-connections

Counter type	/parcelport/count/<connection_type>/cache-connections /parcelport/count/<connection_type>/cache-max-connections where <connection_type> is one of the following: tcp, mpi
Counter instance formatting	locality#*/total where * is the *locality* id of the *locality* the counter should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	cache-connections returns the current number of connections tracked by the connection cache for the given connection type on the given *locality*. This includes both connections that are currently checked out and connections that are idle and available for reuse. cache-max-connections returns the configured upper limit on the total number of connections the cache is allowed to hold. This value is fixed at startup and corresponds to the hpx.max_connections ini key. Together these two counters give the pool utilisation ratio. If cache-connections is consistently equal to cache-max-connections and cache-reservation-failures is increasing, the connection pool is saturated and the limit should be raised. The performance counters for the connection type mpi are available only if the compile time constant HPX_HAVE_PARCELPORT_MPI was defined while compiling the *HPX* core library (which is not defined by default). The corresponding cmake configuration constant is HPX_WITH_PARCELPORT_MPI. Please see *CMake options* for more details.

Table 2.52: Parcel layer performance counter /parcelqueue/length/<operation>

Counter type	/parcelqueue/length/<operation> where <operation> is one of the following: sent, receive
Counter instance formatting	locality#*/total where * is the *locality* id of the *locality* the *parcel* queue should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the current number of parcels stored in the *parcel* queue (see <operation> for which queue to query, e.g. sent or received).

Table 2.53: Thread manager performance counter /threads/count/cumulative

Counter type	/threads/count/cumulative
Counter instance formatting	locality#*/total or locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the overall number of retired *HPX*-threads should be queried for. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. **pool#* is defining the pool for which the current value of the** idle-loop counter should be queried for. **worker-thread#* is defining the worker thread for which the overall** number of retired *HPX*-threads should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the overall number of executed (retired) *HPX*-threads on the given *locality* since application start. If the instance name is total the counter returns the accumulated number of retired *HPX*-threads for all worker threads (cores) on that *locality*. If the instance name is worker-thread#* the counter will return the overall number of retired *HPX*-threads for all worker threads separately. This counter is available only if the configuration time constant HPX_WITH_THREAD_CUMULATIVE_COUNTS is set to ON (default: ON).

Table 2.54: Thread manager performance counter /threads/time/average

Counter type	/threads/time/average
Counter instance formatting	locality#*/total or locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the average time spent executing one *HPX*-thread should be queried for. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. pool#* is defining the pool for which the current value of the idle-loop counter should be queried for. worker-thread#* is defining the worker thread for which the average time spent executing one *HPX*-thread should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the average time spent executing one *HPX*-thread on the given *locality* since application start. If the instance name is total the counter returns the average time spent executing one *HPX*-thread for all worker threads (cores) on that *locality*. If the instance name is worker-thread#* the counter will return the average time spent executing one *HPX*-thread for all worker threads separately. This counter is available only if the configuration time constants HPX_WITH_THREAD_CUMULATIVE_COUNTS (default: ON) and HPX_WITH_THREAD_IDLE_RATES are set to ON (default: OFF). The unit of measure for this counter is nanosecond [ns].

Table 2.55: Thread manager performance counter `/threads/time/average-overhead`

Counter type	`/threads/time/average-overhead`
Counter instance formatting	`locality#*/total` or `locality#*/worker-thread#*` or `locality#*/pool#*/worker-thread#*` where: `locality#*` is defining the *locality* for which the average overhead spent executing one *HPX*-thread should be queried for. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. `pool#*` is defining the pool for which the current value of the idle-loop counter should be queried for. `worker-thread#*` is defining the worker thread for which the average overhead spent executing one *HPX*-thread should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option `--hpx:threads`. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the average time spent on overhead while executing one *HPX*-thread on the given *locality* since application start. If the instance name is `total` the counter returns the average time spent on overhead while executing one *HPX*-thread for all worker threads (cores) on that *locality*. If the instance name is `worker-thread#*` the counter will return the average time spent on overhead executing one *HPX*-thread for all worker threads separately. This counter is available only if the configuration time constants `HPX_WITH_THREAD_CUMULATIVE_COUNTS` (default: `ON`) and `HPX_WITH_THREAD_IDLE_RATES` are set to `ON` (default: `OFF`). The unit of measure for this counter is nanosecond [ns].

Table 2.56: Thread manager performance counter `/threads/count/cumulative-phases`

Counter type	`/threads/count/cumulative-phases`
Counter instance formatting	`locality#*/total` or `locality#*/worker-thread#*` or `locality#*/pool#*/worker-thread#*` where: `locality#*` is defining the *locality* for which the overall number of executed *HPX*-thread phases (invocations) should be queried for. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. `pool#*` is defining the pool for which the current value of the idle-loop counter should be queried for. `worker-thread#*` is defining the worker thread for which the overall number of executed *HPX*-thread phases (invocations) should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option `--hpx:threads`. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the overall number of executed *HPX*-thread phases (invocations) on the given *locality* since application start. If the instance name is `total` the counter returns the accumulated number of executed *HPX*-thread phases (invocations) for all worker threads (cores) on that *locality*. If the instance name is `worker-thread#*` the counter will return the overall number of executed *HPX*-thread phases for all worker threads separately. This counter is available only if the configuration time constant `HPX_WITH_THREAD_CUMULATIVE_COUNTS` is set to `ON` (default: `ON`). The unit of measure for this counter is nanosecond [ns].

Table 2.57: Thread manager performance counter /threads/time/average-phase

Counter type	/threads/time/average-phase
Counter instance formatting	locality#*/total or locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the average time spent executing one *HPX*-thread phase (invocation) should be queried for. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. pool#* is defining the pool for which the current value of the idle-loop counter should be queried for. worker-thread#* is defining the worker thread for which the average time executing one *HPX*-thread phase (invocation) should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option --hpx:threads. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the average time spent executing one *HPX*-thread phase (invocation) on the given *locality* since application start. If the instance name is total the counter returns the average time spent executing one *HPX*-thread phase (invocation) for all worker threads (cores) on that *locality*. If the instance name is worker-thread#* the counter will return the average time spent executing one *HPX*-thread phase for all worker threads separately. This counter is available only if the configuration time constants HPX_WITH_THREAD_CUMULATIVE_COUNTS (default: ON) and HPX_WITH_THREAD_IDLE_RATES are set to ON (default: OFF). The unit of measure for this counter is nanosecond [ns].

Table 2.58: Thread manager performance counter `/threads/time/average-phase-overhead`

Counter type	`/threads/time/average-phase-overhead`
Counter instance formatting	`locality#*/total` or `locality#*/worker-thread#*` or `locality#*/pool#*/worker-thread#*` where: `locality#*` is defining the *locality* for which the average time overhead executing one *HPX*-thread phase (invocation) should be queried for. The *locality* id (given by the `*`) is a (zero based) number identifying the *locality*. `pool#*` is defining the pool for which the current value of the idle-loop counter should be queried for. `worker-thread#*` is defining the worker thread for which the average overhead executing one *HPX*-thread phase (invocation) should be queried for. The worker thread number (given by the `*`) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option `--hpx:threads`. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the average time spent on overhead executing one *HPX*-thread phase (invocation) on the given *locality* since application start. If the instance name is `total` the counter returns the average time spent on overhead while executing one *HPX*-thread phase (invocation) for all worker threads (cores) on that *locality*. If the instance name is `worker-thread#*` the counter will return the average time spent on overhead executing one *HPX*-thread phase for all worker threads separately. This counter is available only if the configuration time constants `HPX_WITH_THREAD_CUMULATIVE_COUNTS` (default: `ON`) and `HPX_WITH_THREAD_IDLE_RATES` are set to `ON` (default: `OFF`). The unit of measure for this counter is nanosecond [ns].

Table 2.59: Thread manager performance counter `/threads/time/overall`

Counter type	`/threads/time/overall`
Counter instance formatting	`locality#*/total` or `locality#*/worker-thread#*` or `locality#*/pool#*/worker-thread#*` where: `locality#*` is defining the *locality* for which the overall time spent running the scheduler should be queried for. The *locality* id (given by the `*`) is a (zero based) number identifying the *locality*. `pool#*` is defining the pool for which the current value of the idle-loop counter should be queried for. `worker-thread#*` is defining the worker thread for which the overall time spent running the scheduler should be queried for. The worker thread number (given by the `*`) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option `--hpx:threads`. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the overall time spent running the scheduler on the given *locality* since application start. If the instance name is `total` the counter returns the overall time spent running the scheduler for all worker threads (cores) on that *locality*. If the instance name is `worker-thread#*` the counter will return the overall time spent running the scheduler for all worker threads separately. This counter is available only if the configuration time constant `HPX_WITH_THREAD_IDLE_RATES` is set to `ON` (default: `OFF`). The unit of measure for this counter is nanosecond [ns].

Table 2.60: Thread manager performance counter /threads/time/cumulative

Counter type	/threads/time/cumulative
Counter instance formatting	locality#*/total or locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the overall time spent executing all *HPX*-threads should be queried for. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. pool#* is defining the pool for which the current value of the idle-loop counter should be queried for. worker-thread#* is defining the worker thread for which the overall time spent executing all *HPX*-threads should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the overall time spent executing all *HPX*-threads on the given *locality* since application start. If the instance name is total the counter returns the overall time spent executing all *HPX*-threads for all worker threads (cores) on that *locality*. If the instance name is worker-thread#* the counter will return the overall time spent executing all *HPX*-threads for all worker threads separately. This counter is available only if the configuration time constants HPX_THREAD_MAINTAIN_CUMULATIVE_COUNTS (default: ON) and HPX_THREAD_MAINTAIN_IDLE_RATES are set to ON (default: OFF).

Table 2.61: Thread manager performance counter /threads/time/cumulative-overheads

Counter type	/threads/time/cumulative-overheads
Counter instance formatting	locality#*/total or locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the overall overhead time incurred by executing all *HPX*-threads should be queried for. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. pool#* is defining the pool for which the current value of the idle-loop counter should be queried for. worker-thread#* is defining the worker thread for which the the overall overhead time incurred by executing all *HPX*-threads should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the overall overhead time incurred executing all *HPX*-threads on the given *locality* since application start. If the instance name is total the counter returns the overall overhead time incurred executing all *HPX*-threads for all worker threads (cores) on that *locality*. If the instance name is worker-thread#* the counter will return the overall overhead time incurred executing all *HPX*-threads for all worker threads separately. This counter is available only if the configuration time constants HPX_THREAD_MAINTAIN_CUMULATIVE_COUNTS (default: ON) and HPX_THREAD_MAINTAIN_IDLE_RATES are set to ON (default: OFF). The unit of measure for this counter is nanosecond [ns].

Table 2.62: Thread manager performance counter threads/count/instantaneous/<thread-state>

Counter type	threads/count/instantaneous/<thread-state> where: <thread-state> is one of the following: all, active, pending, suspended, terminated, staged
Counter instance formatting	locality#*/total or locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the current number of threads with the given state should be queried for. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. pool#* is defining the pool for which the current value of the idle-loop counter should be queried for. worker-thread#* is defining the worker thread for which the current number of threads with the given state should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*. If no pool-name is specified the counter refers to the 'default' pool. The staged thread state refers to registered tasks before they are converted to thread objects.
Description	Returns the current number of *HPX*-threads having the given thread state on the given *locality*. If the instance name is total the counter returns the current number of *HPX*-threads of the given state for all worker threads (cores) on that *locality*. If the instance name is worker-thread#* the counter will return the current number of *HPX*-threads in the given state for all worker threads separately.

Table 2.63: Thread manager performance counter threads/wait-time/<thread-state>

Counter type	threads/wait-time/<thread-state> where: <thread-state> is one of the following: pending staged
Counter instance formatting	locality#*/total or locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the average wait time of *HPX*-threads (pending) or thread descriptions (staged) with the given state should be queried for. The *locality* id (given by * is a (zero based) number identifying the *locality*. pool#* is defining the pool for which the current value of the idle-loop counter should be queried for. worker-thread#* is defining the worker thread for which the average wait time for the given state should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*. If no pool-name is specified the counter refers to the 'default' pool. The staged thread state refers to the wait time of registered tasks before they are converted into thread objects, while the pending thread state refers to the wait time of threads in any of the scheduling queues.
Description	Returns the average wait time of *HPX*-threads (if the thread state is pending or of task descriptions (if the thread state is staged on the given *locality* since application start. If the instance name is total the counter returns the wait time of *HPX*-threads of the given state for all worker threads (cores) on that *locality*. If the instance name is worker-thread#* the counter will return the wait time of *HPX*-threads in the given state for all worker threads separately. These counters are available only if the compile time constant HPX_WITH_THREAD_QUEUE_WAITTIME was defined while compiling the *HPX* core library (default: OFF). The unit of measure for this counter is nanosecond [ns].

Table 2.64: Thread manager performance counter /threads/idle-rate

Counter type	/threads/idle-rate
Counter instance formatting	locality#*/total or locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the average idle rate of all (or one) worker threads should be queried for. The *locality* id (given by the *) is a (zero based) number identifying the *locality* pool#* is defining the pool for which the current value of the idle-loop counter should be queried for. worker-thread#* is defining the worker thread for which the averaged idle rate should be queried for. The worker thread number (given by the * is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the average idle rate for the given worker thread(s) on the given *locality*. The idle rate is defined as the ratio of the time spent on scheduling and management tasks and the overall time spent executing work since the application started. This counter is available only if the configuration time constant HPX_WITH_THREAD_IDLE_RATES is set to ON (default: OFF).

Table 2.65: Thread manager performance counter /threads/creation-idle-rate

Counter type	/threads/creation-idle-rate
Counter instance formatting	locality#*/total or locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the average creation idle rate of all (or one) worker threads should be queried for. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. pool#* is defining the pool for which the current value of the idle-loop counter should be queried for. worker-thread#* is defining the worker thread for which the averaged idle rate should be queried for. The worker thread number (given by the * is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the average idle rate for the given worker thread(s) on the given *locality* which is caused by creating new threads. The creation idle rate is defined as the ratio of the time spent on creating new threads and the overall time spent executing work since the application started. This counter is available only if the configuration time constants HPX_WITH_THREAD_IDLE_RATES (default: OFF) and HPX_WITH_THREAD_CREATION_AND_CLEANUP_RATES are set to ON.

Table 2.66: Thread manager performance counter /threads/cleanup-idle-rate

Counter type	/threads/cleanup-idle-rate
Counter instance formatting	locality#*/total or locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the average cleanup idle rate of all (or one) worker threads should be queried for. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. pool#* is defining the pool for which the current value of the idle-loop counter should be queried for. worker-thread#* is defining the worker thread for which the averaged cleanup idle rate should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the average idle rate for the given worker thread(s) on the given *locality* which is caused by cleaning up terminated threads. The cleanup idle rate is defined as the ratio of the time spent on cleaning up terminated thread objects and the overall time spent executing work since the application started. This counter is available only if the configuration time constants HPX_WITH_THREAD_IDLE_RATES (default: OFF) and HPX_WITH_THREAD_CREATION_AND_CLEANUP_RATES are set to ON.

Table 2.67: Thread manager performance counter /threadqueue/length

Counter type	/threadqueue/length
Counter instance formatting	locality#*/total or locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the current length of all thread queues in the scheduler for all (or one) worker threads should be queried for. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. pool#* is defining the pool for which the current value of the idle-loop counter should be queried for. worker-thread#* is defining the worker thread for which the current length of all thread queues in the scheduler should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the overall length of all queues for the given worker thread(s) on the given *locality*.

Table 2.68: Thread manager performance counter `/threads/count/stack-unbinds`

Counter type	`/threads/count/stack-unbinds`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the unbind (madvise) operations should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the total number of *HPX*-thread unbind (madvise) operations performed for the referenced *locality*. Note that this counter is not available on Windows based platforms.

Table 2.69: Thread manager performance counter `/threads/count/stack-recycles`

Counter type	`/threads/count/stack-recycles`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the recycling operations should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the total number of *HPX*-thread recycling operations performed.

Table 2.70: Thread manager performance counter `/threads/count/stolen-from-pending`

Counter type	`/threads/count/stolen-from-pending`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the number of 'stole' threads should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the total number of *HPX*-threads 'stolen' from the pending thread queue by a neighboring thread worker thread (these threads are executed by a different worker thread than they were initially scheduled on). This counter is available only if the configuration time constant `HPX_WITH_THREAD_STEALING_COUNTS` is set to `ON` (default: `ON`).

Table 2.71: Thread manager performance counter `/threads/count/pending-misses`

Counter type	`/threads/count/pending-misses`
Counter instance formatting	`locality#*/total` or `locality#*/worker-thread#*` or `locality#*/pool#*/worker-thread#*` where: `locality#*` is defining the *locality* for which the number of pending queue misses of all (or one) worker threads should be queried for. The *locality* id (given by the `*`) is a (zero based) number identifying the *locality* `pool#*` is defining the pool for which the current value of the idle-loop counter should be queried for. `worker-thread#*` is defining the worker thread for which the number of pending queue misses should be queried for. The worker thread number (given by the `*`) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option `--hpx:threads`. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the total number of times that the referenced worker-thread on the referenced *locality* failed to find pending *HPX*-threads in its associated queue. This counter is available only if the configuration time constant `HPX_WITH_THREAD_STEALING_COUNTS` is set to `ON` (default: `ON`).

Table 2.72: Thread manager performance counter `/threads/count/pending-accesses`

Counter type	`/threads/count/pending-accesses`
Counter instance formatting	`locality#*/total` or `locality#*/worker-thread#*` or `locality#*/pool#*/worker-thread#*` where: `locality#*` is defining the *locality* for which the number of pending queue accesses of all (or one) worker threads should be queried for. The *locality* id (given by the `*`) is a (zero based) number identifying the *locality* `pool#*` is defining the pool for which the current value of the idle-loop counter should be queried for. `worker-thread#*` is defining the worker thread for which the number of pending queue accesses should be queried for. The worker thread number (given by the `*`) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option `--hpx:threads`. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the total number of times that the referenced worker-thread on the referenced *locality* looked for pending *HPX*-threads in its associated queue. This counter is available only if the configuration time constant `HPX_WITH_THREAD_STEALING_COUNTS` is set to `ON` (default: `ON`).

Table 2.73: Thread manager performance counter /threads/count/stolen-from-staged

Counter type	/threads/count/stolen-from-staged
Counter instance formatting	locality#*/total or locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the number of *HPX*-threads stolen from the staged queue of all (or one) worker threads should be queried for. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. pool#* is defining the pool for which the current value of the idle-loop counter should be queried for. worker-thread#* is defining the worker thread for which the number of *HPX*-threads stolen from the staged queue should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the total number of *HPX*-threads 'stolen' from the staged thread queue by a neighboring worker thread (these threads are executed by a different worker thread than they were initially scheduled on). This counter is available only if the configuration time constant HPX_WITH_THREAD_STEALING_COUNTS is set to ON (default: ON).

Table 2.74: Thread manager performance counter /threads/count/stolen-to-pending

Counter type	/threads/count/stolen-to-pending
Counter instance formatting	locality#*/total or locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the number of *HPX*-threads stolen to the pending queue of all (or one) worker threads should be queried for. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. pool#* is defining the pool for which the current value of the idle-loop counter should be queried for. worker-thread#* is defining the worker thread for which the number of *HPX*-threads stolen to the pending queue should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the total number of *HPX*-threads 'stolen' to the pending thread queue of the worker thread (these threads are executed by a different worker thread than they were initially scheduled on). This counter is available only if the configuration time constant HPX_WITH_THREAD_STEALING_COUNTS is set to ON (default: ON).

Table 2.75: Thread manager performance counter /threads/count/stolen-to-staged

Counter type	/threads/count/stolen-to-staged
Counter instance formatting	locality#*/total or locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the number of *HPX*-threads stolen to the staged queue of all (or one) worker threads should be queried for. The *locality* id (given by *) is a (zero based) number identifying the *locality*. pool#* is defining the pool for which the current value of the idle-loop counter should be queried for. worker-thread#* is defining the worker thread for which the number of *HPX*-threads stolen to the staged queue should be queried for. The worker thread number (given by the *) is a (zero based) worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the total number of *HPX*-threads 'stolen' to the staged thread queue of a neighboring worker thread (these threads are executed by a different worker thread than they were initially scheduled on). This counter is available only if the configuration time constant HPX_WITH_THREAD_STEALING_COUNTS is set to ON (default: ON).

Table 2.76: Thread manager performance counter /threads/count/objects

Counter type	/threads/count/objects
Counter instance formatting	locality#*/total or locality#*/allocator#* where: locality#* is defining the *locality* for which the current (cumulative) number of all created *HPX*-thread objects should be queried for. The *locality* id (given by *) is a (zero based) number identifying the *locality*. allocator#* is defining the number of the allocator instance using which the threads have been created. *HPX* uses a varying number of allocators to create (and recycle) *HPX*-thread objects, most likely these counters are of use for debugging purposes only. The allocator id (given by *) is a (zero based) number identifying the allocator to query.
Description	Returns the total number of *HPX*-thread objects created. Note that thread objects are reused to improve system performance, thus this number does not reflect the number of actually executed (retired) *HPX*-threads.

Table 2.77: Thread manager performance counter `/scheduler/utilization/instantaneous`

Counter type	`/scheduler/utilization/instantaneous`
Counter instance formatting	`locality#*/total` where: `locality#*` is defining the *locality* for which the current (instantaneous) scheduler utilization queried for. The *locality* id (given by *) is a (zero based) number identifying the *locality*.
Description	**Returns the total (instantaneous) scheduler utilization. This is the** current percentage of scheduler threads executing *HPX* threads.
Parameters	Percent

Table 2.78: Thread manager performance counter `/threads/idle-loop-count/instantaneous`

Counter type	`/threads/idle-loop-count/instantaneous`
Counter instance formatting	`locality#*/worker-thread#*` or `locality#*/pool#*/worker-thread#*` where: `locality#*` is defining the *locality* for which the current current accumulated value of all idle-loop counters of all worker threads should be queried. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. `pool#*` is defining the pool for which the current value of the idle-loop counter should be queried for. `worker-thread#*` is defining the worker thread for which the current value of the idle-loop counter should be queried for. The worker thread number (given by the *) is a (zero based) worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option `--hpx:threads`. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the current (instantaneous) idle-loop count for the given *HPX*- worker thread or the accumulated value for all worker threads.

Table 2.79: Thread manager performance counter /threads/busy-loop-count/instantaneous

Counter type	/threads/busy-loop-count/instantaneous
Counter instance formatting	locality#*/worker-thread#* or locality#*/pool#*/worker-thread#* where: locality#* is defining the *locality* for which the current current accumulated value of all busy-loop counters of all worker threads should be queried. The *locality* id (given by the *) is a (zero based) number identifying the *locality*. pool#* is defining the pool for which the current value of the idle-loop counter should be queried for. worker-thread#* is defining the worker thread for which the current value of the busy-loop counter should be queried for. The worker thread number (given by the *) is a (zero based) worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*. If no pool-name is specified the counter refers to the 'default' pool.
Description	Returns the current (instantaneous) busy-loop count for the given *HPX*- worker thread or the accumulated value for all worker threads.

Table 2.80: Thread manager performance counter /threads/time/background-work-duration

Counter type	/threads/time/background-work-duration
Counter instance formatting	locality#*/total or locality#*/worker-thread#* where: locality#* is defining the locality for which the overall time spent performing background work should be queried for. The locality id (given by *) is a (zero based) number identifying the locality. worker-thread#* is defining the worker thread for which the overall time spent performing background work should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*.
Description	Returns the overall time spent performing background work on the given locality since application start. If the instance name is total the counter returns the overall time spent performing background work for all worker threads (cores) on that locality. If the instance name is worker-thread#* the counter will return the overall time spent performing background work for all worker threads separately. This counter is available only if the configuration time constants HPX_WITH_BACKGROUND_THREAD_COUNTERS (default: OFF) and HPX_WITH_THREAD_IDLE_RATES are set to ON (default: OFF). The unit of measure for this counter is nanosecond [ns].

Table 2.81: Thread manager performance counter /threads/background-overhead

Counter type	/threads/background-overhead
Counter instance formatting	locality#*/total or locality#*/worker-thread#* where: locality#* is defining the locality for which the background overhead should be queried for. The locality id (given by *) is a (zero based) number identifying the locality. worker-thread#* is defining the worker thread for which the background overhead should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option --hpx:threads.
Description	Returns the background overhead on the given locality since application start. If the instance name is total the counter returns the background overhead for all worker threads (cores) on that locality. If the instance name is worker-thread#* the counter will return background overhead for all worker threads separately. This counter is available only if the configuration time constants HPX_WITH_BACKGROUND_THREAD_COUNTERS (default: OFF) and HPX_WITH_THREAD_IDLE_RATES are set to ON (default: OFF). The unit of measure displayed for this counter is 0.1%.

Table 2.82: Thread manager performance counter /threads/time/background-send-duration

Counter type	/threads/time/background-send-duration
Counter instance formatting	locality#*/total or locality#*/worker-thread#* where: locality#* is defining the locality for which the overall time spent performing background work related to sending parcels should be queried for. The locality id (given by *) is a (zero based) number identifying the locality. worker-thread#* is defining the worker thread for which the overall time spent performing background work related to sending parcels should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option --hpx:threads.
Description	Returns the overall time spent performing background work related to sending parcels on the given locality since application start. If the instance name is total the counter returns the overall time spent performing background work for all worker threads (cores) on that locality. If the instance name is worker-thread#* the counter will return the overall time spent performing background work for all worker threads separately. This counter is available only if the configuration time constants HPX_WITH_BACKGROUND_THREAD_COUNTERS (default: OFF) and HPX_WITH_THREAD_IDLE_RATES are set to ON (default: OFF). The unit of measure for this counter is nanosecond [ns]. This counter will currently return meaningful values for the MPI parcelport only.

Table 2.83: Thread manager performance counter /threads/background-send-overhead

Counter type	/threads/background-send-overhead
Counter instance formatting	locality#*/total or locality#*/worker-thread#* where: locality#* is defining the locality for which the background overhead related to sending parcels should be queried for. The locality id (given by *) is a (zero based) number identifying the locality. worker-thread#* is defining the worker thread for which the background overhead related to sending parcels should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*.
Description	Returns the background overhead related to sending parcels on the given locality since application start. If the instance name is total the counter returns the background overhead for all worker threads (cores) on that locality. If the instance name is worker-thread#* the counter will return background overhead for all worker threads separately. This counter is available only if the configuration time constants HPX_WITH_BACKGROUND_THREAD_COUNTERS (default: OFF) and HPX_WITH_THREAD_IDLE_RATES are set to ON (default: OFF). The unit of measure displayed for this counter is 0.1%. This counter will currently return meaningful values for the MPI parcelport only.

Table 2.84: Thread manager performance counter /threads/time/background-receive-duration

Counter type	/threads/time/background-receive-duration
Counter instance formatting	locality#*/total or locality#*/worker-thread#* where: locality#* is defining the locality for which the overall time spent performing background work related to receiving parcels should be queried for. The locality id (given by *) is a (zero based) number identifying the locality. worker-thread#* is defining the worker thread for which the overall time spent performing background work related to receiving parcels should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*.
Description	Returns the overall time spent performing background work related to receiving parcels on the given locality since application start. If the instance name is total the counter returns the overall time spent performing background work for all worker threads (cores) on that locality. If the instance name is worker-thread#* the counter will return the overall time spent performing background work for all worker threads separately. This counter is available only if the configuration time constants HPX_WITH_BACKGROUND_THREAD_COUNTERS (default: OFF) and HPX_WITH_THREAD_IDLE_RATES are set to ON (default: OFF). The unit of measure for this counter is nanosecond [ns]. This counter will currently return meaningful values for the MPI parcelport only.

Table 2.85: Thread manager performance counter /threads/background-receive-overhead

Counter type	/threads/background-receive-overhead
Counter instance formatting	locality#*/total or locality#*/worker-thread#* where: locality#* is defining the locality for which the background overhead related to receiving should be queried for. The locality id (given by *) is a (zero based) number identifying the locality. worker-thread#* is defining the worker thread for which the background overhead related to receiving parcels should be queried for. The worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*.
Description	Returns the background overhead related to receiving parcels on the given locality since application start. If the instance name is total the counter returns the background overhead for all worker threads (cores) on that locality. If the instance name is worker-thread#* the counter will return background overhead for all worker threads separately. This counter is available only if the configuration time constants HPX_WITH_BACKGROUND_THREAD_COUNTERS (default: OFF) and HPX_WITH_THREAD_IDLE_RATES are set to ON (default: OFF). The unit of measure displayed for this counter is 0.1%. This counter will currently return meaningful values for the MPI parcelport only.

Table 2.86: General performance counter /runtime/count/component

Counter type	/runtime/count/component
Counter instance formatting	locality#*/total where: * is the *locality* id of the *locality* the number of components should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the overall number of currently active components of the specified type on the given *locality*.
Parameters	The type of the component. This is the string which has been used while registering the component with *HPX*, e.g. which has been passed as the second parameter to the macro *HPX_REGISTER_COMPONENT*.

Table 2.87: General performance counter /runtime/count/action-invocation

Counter type	/runtime/count/action-invocation
Counter instance formatting	locality#*/total where: * is the *locality* id of the locality the number of action invocations should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the overall (local) invocation count of the specified action type on the given *locality*.
Parameters	The action type. This is the string which has been used while registering the action with *HPX*, e.g. which has been passed as the second parameter to the macro *HPX_REGISTER_ACTION* or *HPX_REGISTER_ACTION_ID*.

Table 2.88: General performance counter `/runtime/count/remote-action-invocation`

Counter type	`/runtime/count/remote-action-invocation`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the number of action invocations should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the overall (remote) invocation count of the specified action type on the given *locality*.
Parameters	The action type. This is the string which has been used while registering the action with *HPX*, e.g. which has been passed as the second parameter to the macro *`HPX_REGISTER_ACTION`* or *`HPX_REGISTER_ACTION_ID`*.

Table 2.89: General performance counter `/runtime/uptime`

Counter type	`/runtime/uptime`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the system uptime should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the overall time since application start on the given *locality* in nanoseconds.

Table 2.90: General performance counter `/runtime/memory/virtual`

Counter type	`/runtime/memory/virtual`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the allocated virtual memory should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the amount of virtual memory currently allocated by the referenced *locality* (in bytes).

Table 2.91: General performance counter `/runtime/memory/resident`

Counter type	`/runtime/memory/resident`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the allocated resident memory should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the amount of resident memory currently allocated by the referenced *locality* (in bytes).

Table 2.92: General performance counter `/runtime/memory/total`

Counter type	`/runtime/memory/total`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the total available memory should be queried. The *locality* id is a (zero based) number identifying the *locality*. Note: only supported in Linux.
Description	**Returns the total available memory for use by the referenced** *locality* (in bytes). This counter is available on Linux and Windows systems only.

Table 2.93: General performance counter `/runtime/io/read_bytes_issued`

Counter type	`/runtime/io/read_bytes_issued`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the number of bytes read should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the number of bytes read by the process (aggregate of count arguments passed to read() call or its analogues). This performance counter is available only on systems which expose the related data through the /proc file system.

Table 2.94: General performance counter `/runtime/io/write_bytes_issued`

Counter type	`/runtime/io/write_bytes_issued`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the number of bytes written should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the number of bytes written by the process (aggregate of count arguments passed to write() call or its analogues). This performance counter is available only on systems which expose the related data through the /proc file system.

Table 2.95: General performance counter `/runtime/io/read_syscalls`

Counter type	`/runtime/io/read_syscalls`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the number of system calls should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the number of system calls that perform I/O reads. This performance counter is available only on systems which expose the related data through the /proc file system.

Table 2.96: General performance counter /runtime/io/write_syscalls

Counter type	`/runtime/io/write_syscalls`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the number of system calls should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the number of system calls that perform I/O writes. This performance counter is available only on systems which expose the related data through the /proc file system.

Table 2.97: General performance counter /runtime/io/read_bytes_transferred

Counter type	`/runtime/io/read_bytes_transferred`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the number of bytes transferred should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the number of bytes retrieved from storage by I/O operations. This performance counter is available only on systems which expose the related data through the /proc file system.

Table 2.98: General performance counter /runtime/io/write_bytes_transferred

Counter type	`/runtime/io/write_bytes_transferred`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the number of bytes transferred should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the number of bytes retrieved from storage by I/O operations. This performance counter is available only on systems which expose the related data through the /proc file system.

Table 2.99: General performance counter /runtime/io/write_bytes_cancelled

Counter type	`/runtime/io/write_bytes_cancelled`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the number of bytes not being transferred should be queried. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the number of bytes accounted by write_bytes_transferred that has not been ultimately stored due to truncation or deletion. This performance counter is available only on systems which expose the related data through the /proc file system.

Table 2.100: Performance counter /papi/<papi_event>

Counter type	/papi/<papi_event> where: <papi_event> is the name of the PAPI event to expose as a performance counter (such as PAPI_SR_INS). Note that the list of available PAPI events changes depending on the used architecture. For a full list of available PAPI events and their (short) description use the --hpx:list-counters and --hpx:papi-event-info=all command line options.
Counter instance formatting	locality#*/total or locality#*/worker-thread#* where: locality#* is defining the *locality* for which the current current accumulated value of all busy-loop counters of all worker threads should be queried. The *locality* id (given by *) is a (zero based) number identifying the *locality*. worker-thread#* is defining the worker thread for which the current value of the busy-loop counter should be queried for. The worker thread number (given by the *) is a (zero based) worker thread number (given by the *) is a (zero based) number identifying the worker thread. The number of available worker threads is usually specified on the command line for the application using the option *--hpx:threads*.
Description	Returns the current count of occurrences of the specified PAPI event. This counter is available only if the configuration time constant HPX_WITH_PAPI is set to ON (default: OFF).

Table 2.101: Performance counter /statistics/average

Counter type	/statistics/average
Counter instance formatting	Any full performance counter name. The referenced performance counter is queried at fixed time intervals as specified by the first parameter.
Description	Returns the current average (mean) value calculated based on the values queried from the underlying counter (the one specified as the instance name).
Parameters	Any parameter will be interpreted as a list of up to two comma separated (integer) values, where the first is the time interval (in milliseconds) at which the underlying counter should be queried. If no value is specified, the counter will assume 1000 [ms] as the default. The second value can be either 0 or 1 and specifies whether the underlying counter should be reset during evaluation 1 or not 0. The default value is 0.

Table 2.102: Performance counter /statistics/rolling_average

Counter type	/statistics/rolling_average
Counter instance formatting	Any full performance counter name. The referenced performance counter is queried at fixed time intervals as specified by the first parameter.
Description	Returns the current rolling average (mean) value calculated based on the values queried from the underlying counter (the one specified as the instance name).
Parameters	Any parameter will be interpreted as a list of up to three comma separated (integer) values, where the first is the time interval (in milliseconds) at which the underlying counter should be queried. If no value is specified, the counter will assume 1000 [ms] as the default. The second value will be interpreted as the size of the rolling window (the number of latest values to use to calculate the rolling average). The default value for this is 10. The third value can be either 0 or 1 and specifies whether the underlying counter should be reset during evaluation 1 or not 0. The default value is 0.

Table 2.103: Performance counter `/statistics/stddev`

Counter type	`/statistics/stddev`
Counter instance formatting	Any full performance counter name. The referenced performance counter is queried at fixed time intervals as specified by the first parameter.
Description	Returns the current standard deviation (stddev) value calculated based on the values queried from the underlying counter (the one specified as the instance name).
Parameters	Any parameter will be interpreted as a list of up to two comma separated (integer) values, where the first is the time interval (in milliseconds) at which the underlying counter should be queried. If no value is specified, the counter will assume `1000` [ms] as the default. The second value can be either `0` or `1` and specifies whether the underlying counter should be reset during evaluation `1` or not `0`. The default value is `0`.

Table 2.104: Performance counter `/statistics/rolling_stddev`

Counter type	`/statistics/rolling_stddev`
Counter instance formatting	Any full performance counter name. The referenced performance counter is queried at fixed time intervals as specified by the first parameter.
Description	Returns the current rolling variance (stddev) value calculated based on the values queried from the underlying counter (the one specified as the instance name).
Parameters	Any parameter will be interpreted as a list of up to three comma separated (integer) values, where the first is the time interval (in milliseconds) at which the underlying counter should be queried. If no value is specified, the counter will assume `1000` [ms] as the default. The second value will be interpreted as the size of the rolling window (the number of latest values to use to calculate the rolling average). The default value for this is `10`. The third value can be either `0` or `1` and specifies whether the underlying counter should be reset during evaluation `1` or not `0`. The default value is `0`.

Table 2.105: Performance counter `/statistics/median`

Counter type	`/statistics/median`
Counter instance formatting	Any full performance counter name. The referenced performance counter is queried at fixed time intervals as specified by the first parameter.
Description	Returns the current (statistically estimated) median value calculated based on the values queried from the underlying counter (the one specified as the instance name).
Parameters	Any parameter will be interpreted as a list of up to two comma separated (integer) values, where the first is the time interval (in milliseconds) at which the underlying counter should be queried. If no value is specified, the counter will assume `1000` [ms] as the default. The second value can be either `0` or `1` and specifies whether the underlying counter should be reset during evaluation `1` or not `0`. The default value is `0`.

Table 2.106: Performance counter /statistics/max

Counter type	/statistics/max
Counter instance formatting	Any full performance counter name. The referenced performance counter is queried at fixed time intervals as specified by the first parameter.
Description	Returns the current maximum value calculated based on the values queried from the underlying counter (the one specified as the instance name).
Parameters	Any parameter will be interpreted as a list of up to two comma separated (integer) values, where the first is the time interval (in milliseconds) at which the underlying counter should be queried. If no value is specified, the counter will assume 1000 [ms] as the default. The second value can be either 0 or 1 and specifies whether the underlying counter should be reset during evaluation 1 or not 0. The default value is 0.

Table 2.107: Performance counter /statistics/rolling_max

Counter type	/statistics/rolling_max
Counter instance formatting	Any full performance counter name. The referenced performance counter is queried at fixed time intervals as specified by the first parameter.
Description	Returns the current rolling maximum value calculated based on the values queried from the underlying counter (the one specified as the instance name).
Parameters	Any parameter will be interpreted as a list of up to three comma separated (integer) values, where the first is the time interval (in milliseconds) at which the underlying counter should be queried. If no value is specified, the counter will assume 1000 [ms] as the default. The second value will be interpreted as the size of the rolling window (the number of latest values to use to calculate the rolling average). The default value for this is 10. The third value can be either 0 or 1 and specifies whether the underlying counter should be reset during evaluation 1 or not 0. The default value is 0.

Table 2.108: Performance counter /statistics/min

Counter type	/statistics/min
Counter instance formatting	Any full performance counter name. The referenced performance counter is queried at fixed time intervals as specified by the first parameter.
Description	Returns the current minimum value calculated based on the values queried from the underlying counter (the one specified as the instance name).
Parameters	Any parameter will be interpreted as a list of up to two comma separated (integer) values, where the first is the time interval (in milliseconds) at which the underlying counter should be queried. If no value is specified, the counter will assume 1000 [ms] as the default. The second value can be either 0 or 1 and specifies whether the underlying counter should be reset during evaluation 1 or not 0. The default value is 0.

Table 2.109: Performance counter `/statistics/rolling_min`

Counter type	`/statistics/rolling_min`
Counter instance formatting	Any full performance counter name. The referenced performance counter is queried at fixed time intervals as specified by the first parameter.
Description	Returns the current rolling minimum value calculated based on the values queried from the underlying counter (the one specified as the instance name).
Parameters	Any parameter will be interpreted as a list of up to three comma separated (integer) values, where the first is the time interval (in milliseconds) at which the underlying counter should be queried. If no value is specified, the counter will assume `1000` [ms] as the default. The second value will be interpreted as the size of the rolling window (the number of latest values to use to calculate the rolling average). The default value for this is `10`. The third value can be either `0` or `1` and specifies whether the underlying counter should be reset during evaluation `1` or not `0`. The default value is `0`.

Table 2.110: Performance counter `/arithmetics/add`

Counter type	`/arithmetics/add`
Description	Returns the sum calculated based on the values queried from the underlying counters (the ones specified as the parameters).
Parameters	The parameter will be interpreted as a comma separated list of full performance counter names which are queried whenever this counter is accessed. Any wildcards in the counter names will be expanded.

Table 2.111: Performance counter `/arithmetics/subtract`

Counter type	`/arithmetics/subtract`
Description	Returns the difference calculated based on the values queried from the underlying counters (the ones specified as the parameters).
Parameters	The parameter will be interpreted as a comma separated list of full performance counter names which are queried whenever this counter is accessed. Any wildcards in the counter names will be expanded.

Table 2.112: Performance counter `/arithmetics/multiply`

Counter type	`/arithmetics/multiply`
Description	Returns the product calculated based on the values queried from the underlying counters (the ones specified as the parameters).
Parameters	The parameter will be interpreted as a comma separated list of full performance counter names which are queried whenever this counter is accessed. Any wildcards in the counter names will be expanded.

Table 2.113: Performance counter `/arithmetics/divide`

Counter type	`/arithmetics/divide`
Description	Returns the result of division of the values queried from the underlying counters (the ones specified as the parameters).
Parameters	The parameter will be interpreted as a comma separated list of full performance counter names which are queried whenever this counter is accessed. Any wildcards in the counter names will be expanded.

Table 2.114: Performance counter `/arithmetics/mean`

Counter type	`/arithmetics/mean`
Description	Returns the average value of all values queried from the underlying counters (the ones specified as the parameters).
Parameters	The parameter will be interpreted as a comma separated list of full performance counter names which are queried whenever this counter is accessed. Any wildcards in the counter names will be expanded.

Table 2.115: Performance counter `/arithmetics/variance`

Counter type	`/arithmetics/variance`
Description	Returns the standard deviation of all values queried from the underlying counters (the ones specified as the parameters).
Parameters	The parameter will be interpreted as a comma separated list of full performance counter names which are queried whenever this counter is accessed. Any wildcards in the counter names will be expanded.

Table 2.116: Performance counter `/arithmetics/median`

Counter type	`/arithmetics/median`
Description	Returns the median value of all values queried from the underlying counters (the ones specified as the parameters).
Parameters	The parameter will be interpreted as a comma separated list of full performance counter names which are queried whenever this counter is accessed. Any wildcards in the counter names will be expanded.

Table 2.117: Performance counter `/arithmetics/min`

Counter type	`/arithmetics/min`
Description	Returns the minimum value of all values queried from the underlying counters (the ones specified as the parameters).
Parameters	The parameter will be interpreted as a comma separated list of full performance counter names which are queried whenever this counter is accessed. Any wildcards in the counter names will be expanded.

Table 2.118: Performance counter `/arithmetics/max`

Counter type	`/arithmetics/max`
Description	Returns the maximum value of all values queried from the underlying counters (the ones specified as the parameters).
Parameters	The parameter will be interpreted as a comma separated list of full performance counter names which are queried whenever this counter is accessed. Any wildcards in the counter names will be expanded.

Table 2.119: Performance counter `/arithmetics/count`

Counter type	`/arithmetics/count`
Description	Returns the count value of all values queried from the underlying counters (the ones specified as the parameters).
Parameters	The parameter will be interpreted as a comma separated list of full performance counter names which are queried whenever this counter is accessed. Any wildcards in the counter names will be expanded.

Note

The `/arithmetics` counters can consume an arbitrary number of other counters. For this reason those have to be specified as parameters (a comma separated list of counters appended after a `'@'`). For instance:

```
$ ./bin/hello_world_distributed -t2 \
    --hpx:print-counter=/threads{locality#0/worker-thread#*}/count/cumulative \
    --hpx:print-counter=/arithmetics/add@/threads{locality#0/worker-thread#*}/count/
↪cumulative
hello world from OS-thread 0 on locality 0
hello world from OS-thread 1 on locality 0
/threads{locality#0/worker-thread#0}/count/cumulative,1,0.515640,[s],25
/threads{locality#0/worker-thread#1}/count/cumulative,1,0.515520,[s],36
/arithmetics/add@/threads{locality#0/worker-thread#*}/count/cumulative,1,0.516445,[s],
↪64
```

Since all wildcards in the parameters are expanded, this example is fully equivalent to specifying both counters separately to `/arithmetics/add`:

```
$ ./bin/hello_world_distributed -t2 \
    --hpx:print-counter=/threads{locality#0/worker-thread#*}/count/cumulative \
    --hpx:print-counter=/arithmetics/add@\
        /threads{locality#0/worker-thread#0}/count/cumulative,\
        /threads{locality#0/worker-thread#1}/count/cumulative
```

Table 2.120: Performance counter `/coalescing/count/parcels`

Counter type	`/coalescing/count/parcels`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the number of parcels for the given action should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the number of parcels handled by the message handler associated with the action which is given by the counter parameter.
Parameters	The action type. This is the string which has been used while registering the action with *HPX*, e.g. which has been passed as the second parameter to the macro *`HPX_REGISTER_ACTION`* or *`HPX_REGISTER_ACTION_ID`*.

Table 2.121: Performance counter `/coalescing/count/messages`

Counter type	`/coalescing/count/messages`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the number of messages for the given action should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the number of messages generated by the message handler associated with the action which is given by the counter parameter.
Parameters	The action type. This is the string which has been used while registering the action with *HPX*, e.g. which has been passed as the second parameter to the macro `HPX_REGISTER_ACTION` or `HPX_REGISTER_ACTION_ID`.

Table 2.122: Performance counter `/coalescing/count/average-parcels-per-message`

Counter type	`/coalescing/count/average-parcels-per-message`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the number of messages for the given action should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the average number of parcels sent in a message generated by the message handler associated with the action which is given by the counter parameter.
Parameters	The action type. This is the string which has been used while registering the action with *HPX*, e.g. which has been passed as the second parameter to the macro `HPX_REGISTER_ACTION` or `HPX_REGISTER_ACTION_ID`

Table 2.123: Performance counter `/coalescing/time/average-parcel-arrival`

Counter type	`/coalescing/time/average-parcel-arrival`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the average time between parcels for the given action should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns the average time between arriving parcels for the action which is given by the counter parameter.
Parameters	The action type. This is the string which has been used while registering the action with *HPX*, e.g. which has been passed as the second parameter to the macro `HPX_REGISTER_ACTION` or `HPX_REGISTER_ACTION_ID`

Table 2.124: Performance counter /coalescing/time/parcel-arrival-histogram

Counter type	`/coalescing/time/parcel-arrival-histogram`
Counter instance formatting	`locality#*/total` where: * is the *locality* id of the *locality* the average time between parcels for the given action should be queried for. The *locality* id is a (zero based) number identifying the *locality*.
Description	Returns a histogram representing the times between arriving parcels for the action which is given by the counter parameter. This counter returns an array of values, where the first three values represent the three parameters used for the histogram followed by one value for each of the histogram buckets. The first unit of measure displayed for this counter `[ns]` refers to the lower and upper boundary values in the returned histogram data only. The second unit of measure displayed `[0.1%]` refers to the actual histogram data. For each bucket the counter shows a value between `0` and `1000` which corresponds to a percentage value between `0%` and `100%`.
Parameters	The action type and optional histogram parameters. The action type is the string which has been used while registering the action with *HPX*, e.g. which has been passed as the second parameter to the macro *HPX_REGISTER_ACTION* or *HPX_REGISTER_ACTION_ID*. The action type may be followed by a comma separated list of up-to three numbers: the lower and upper boundaries for the collected histogram, and the number of buckets for the histogram to generate. By default these three numbers will be assumed to be `0` (`[ns]`, lower bound), `1000000` (`[ns]`, upper bound), and `20` (number of buckets to generate).

Note

The performance counters related to *parcel* coalescing are available only if the configuration time constant `HPX_WITH_PARCEL_COALESCING` is set to `ON` (default: `ON`). However, even in this case it will be available only for actions that are enabled for parcel coalescing (see the macros `HPX_ACTION_USES_MESSAGE_COALESCING` and `HPX_ACTION_USES_MESSAGE_COALESCING_NOTHROW`).

APEX integration

HPX provides integration with APEX[184], which is a framework for application profiling using task timers and various performance counters Huck *et al.*[194]. It can be added as a `git` submodule by turning on the option *HPX_WITH_APEX:BOOL* during CMake[185] configuration. TAU[186] is an optional dependency when using APEX.

To build *HPX* with APEX[187], add *HPX_WITH_APEX*=`ON`, and, optionally, `Tau_ROOT=$PATH_TO_TAU` to your CMake[188] configuration. In addition, you can override the tag used for APEX[189] with the `HPX_WITH_APEX_TAG` option. Please see the APEX *HPX* documentation[190] for detailed instructions on using APEX[191] with *HPX*.

[184] http://uo-oaciss.github.io/apex
[194] K. A. Huck, A. Porterfield, N Chaimov, H. Kaiser, A. D. Malony, T. Sterling, and R. Fowler. *An autonomic performance environment for exascale*. Supercomputing Frontiers and Innovations, 2015.
[185] https://www.cmake.org
[186] https://www.cs.uoregon.edu/research/tau/home.php
[187] http://uo-oaciss.github.io/apex
[188] https://www.cmake.org
[189] http://uo-oaciss.github.io/apex
[190] https://uo-oaciss.github.io/apex/usage/#hpx-louisiana-state-university
[191] http://uo-oaciss.github.io/apex

Tracy integration

HPX provides integration with the Tracy[192] profiler, which offers per-thread zone tracking, message logs, and fiber support. Enable it with *HPX_WITH_TRACY*=ON during CMake[193] configuration.

Tracy can be supplied via a system install (point `Tracy_ROOT` at the install tree) or fetched by CMake at configure time by adding `HPX_WITH_FETCH_TRACY=ON`. The version fetched is pinned by `HPX_WITH_TRACY_TAG`, which defaults to `v0.14.1`. When Tracy is fetched, *HPX* forces `TRACY_ENABLE`, `TRACY_ON_DEMAND` and `TRACY_FIBERS` on the built client. A system-supplied Tracy must have been built with the same three options; Tracy 0.14 mangles its exported profiler symbol based on the active define set, so a mismatch fails at link time rather than producing a silent inconsistency at runtime.

To profile a distributed run, additionally enable *HPX_WITH_PARCEL_PROFILING*=ON so per-parcel identifiers are carried on the wire and the `send_parcel` / `recv_parcel` / `parcel_scheduled` events can be correlated across localities by parcel id.

Event classes can be compiled out individually to reduce the tracing cost when only a subset of the timeline is being investigated: *HPX_WITH_TRACING_LIFECYCLE_EVENTS*, *HPX_WITH_TRACING_CAUSAL_EVENTS*, and *HPX_WITH_TRACING_WORK_STEALING_EVENTS*. All three default to `ON`; turning one off compiles the wrapper for that class to a no-op, so the runtime connection check and the event body do not run (argument evaluation at each call site is unchanged). These gates affect the Tracy backend, which is the only backend that emits these classes today; the APEX, ITT-Notify and empty backends already treat all three as no-ops.

Start `tracy-profiler` (or `tracy-capture` for headless capture) before or during the run. Tracy discovers instrumented processes via UDP broadcast on the local network; no port configuration is required for the common case.

References

2.4.16 Using the LCI parcelport

Basic information

The Lightweight Communication Interface[195] (LCI) is an ongoing research project aiming to provide efficient support for applications with irregular and asynchronous communication patterns such as graph analysis, sparse linear algebra, and task-based runtime on modern parallel architectures. Its features include (a) support for more communication primitives such as two-sided send/recv and one-sided (dynamic or direct) remote put/get (b) better multi-threaded performance (c) explicit user control of communication resource (d) flexible signaling mechanisms such as synchronizer, completion queue, and active message handler. It is designed to be a low-level communication library used by high-level libraries and frameworks.

The LCI parcelport is an experimental parcelport. It aims to provide the best possible communication performance on high-performance computation platforms. Compared to the MPI parcelport, it uses much fewer messages and memory copies to transfer an *HPX* parcel over the network. Its message transmission path involves minimum synchronization points and is almost lock-free. It is expected to be much faster than the MPI parcelport.

[192] https://github.com/wolfpld/tracy
[193] https://www.cmake.org
[195] https://github.com/uiuc-hpc/lci

Build *HPX* with the LCI parcelport

While building *HPX*, you can specify a set of CMake[196] variables to enable and configure the LCI parcelport. Below, there is a set of the most important and frequently used CMake variables.

HPX_WITH_PARCELPORT_LCI

Enable the LCI parcelport. This enables the use of LCI for networking operations in the *HPX* runtime. The default value is `OFF` because it's not available on all systems and/or requires another dependency. However, this experimental parcelport may provide better performance than the MPI parcelport. You must set this variable to `ON` in order to use the LCI parcelport. All the following variables only make sense when this variable is set to `ON`.

HPX_WITH_FETCH_LCI

Use FetchContent to fetch LCI. The default value is `OFF`. If this option is set to `OFF`. You need to install your own LCI library and *HPX* will try to find it using CMake[197] `find_package`. You can specify the location of the LCI installation by the environmental variable `LCI_ROOT`. Refer to the LCI README[198] for how to install LCI. If this option is set to `ON`. *HPX* will fetch and build LCI for you. You can use the following CMake[199] variables to configure this behavior for your platform.

HPX_WITH_LCI_TAG

This variable only takes effect when `HPX_WITH_FETCH_LCI` is set to `ON` and `FETCHCONTENT_SOURCE_DIR_LCI` is not set. *HPX* will fetch LCI from its github repository. This variable controls the branch/tag LCI will be fetched.

FETCHCONTENT_SOURCE_DIR_LCI

This variable only takes effect when `HPX_WITH_FETCH_LCI` is set to `ON`. When it is defined, `HPX_WITH_LCI_TAG` will be ignored. It accepts a path to a local version of LCI source code and *HPX* will fetch and build LCI from there. The default value is set conservatively for the stability of *HPX*, but users are welcome to set this variable to `master` for potentially better performance.

Run *HPX* with the LCI parcelport

We use the same mechanisms as MPI to launch LCI, so you can use the same way you run MPI parcelport to run LCI parcelport. Typically, it would be `hpxrun.py`, `mpirun`, or `srun`.

`hpxrun.py` serves as a wrapper for `mpirun` and `srun`. If you are using `hpxrun.py`, pass `-p lci` to the scripts. You also need to pass either `-r mpi` or `-r srun` to select the correct run wrapper according to the platform.

If you are using `mpirun` or `srun`, you can just pass `--hpx:ini=hpx.parcel.lci.priority=1000`, `--hpx:ini=hpx.parcel.lci.enable=1`, and `--hpx:ini=hpx.parcel.bootstrap=lci` to the *HPX* applications.

The `hpxrun.py` argument `-r none` (the default option for the run wrapper) and its corresponding *HPX* arguments `--hpx:hpx` and `--hpx:agas` do not work for the MPI or the LCI parcelport.

196 https://www.cmake.org
197 https://www.cmake.org
198 https://github.com/uiuc-hpc/lci#readme
199 https://www.cmake.org

Performance tuning of the LCI parcelport

We encourage users to set the following environmental variables when using the LCI parcelport to get better performance.

```
$ export LCI_SERVER_MAX_SENDS=1024
$ export LCI_SERVER_MAX_RECVS=4096
$ export LCI_SERVER_NUM_PKTS=65536
$ export LCI_SERVER_MAX_CQES=65536
$ export LCI_PACKET_SIZE=12288
```

This setting needs roughly 800MB memory per process. The memory consumption mainly comes from the packets, which can be calculated using *LCI_SERVER_NUM_PKTS x LCI_PACKET_SIZE*.

In addition, users can tune the following command-line options when using the LCI parcelport to get better performance.

--hpx:ini=hpx.parcel.lci.ndevices=<int>

The number of LCI devices to use. The default value is 2. An LCI device represents a collection of network resources. More devices lead to lower thread contention, but too many devices may lead to load imbalance or hardware overhead.

--hpx:ini=hpx.parcel.lci.progress_type=<worker|rp>

The way to progress the LCI device. The default value is `worker`. The `worker` option uses all worker threads to progress the LCI devices. The `rp` option uses dedicated pinned threads to progress the LCI devices. Normally, the `worker` option gives better performance, but the `rp` option has been observed with better performance on some clusters with prior generation of InfiniBand hardware.

Reference

Yan, Jiakun, Hartmut Kaiser, and Marc Snir. *Understanding the Communication Needs of Asynchronous Many-Task Systems–A Case Study of HPX+ LCI.* arXiv preprint, 2025.

Yan, Jiakun, and Marc Snir. *LCI: a Lightweight Communication Interface for Efficient Asynchronous Multithreaded Communication.* The International Conference for High Performance Computing, Networking, Storage, and Analysis (SC), 2025.

2.4.17 Using the LCW parcelport

Basic information

The LCW parcelport is an advanced MPI parcelport that can utilize the MPICH VCI and Continuation extensions. The exact MPI implementation is wrapped in a thin wrapper layer (the Lightweight Communication Wrapper, or LCW). This wrapper provides an active message, send/recv, completion queue, and device abstraction on top of MPI (with/without extensions), GASNet-EX, and LCI. The GASNet-EX backend of LCW cannot be used in *HPX* for now, as it lacks send/recv support.

Build *HPX* with the LCW parcelport

While building *HPX*, you can specify a set of CMake[200] variables to enable and configure the LCW parcelport. Below, there is a set of the most important and frequently used CMake variables.

HPX_WITH_PARCELPORT_LCW

Enable the LCW parcelport. This enables the use of LCW for networking operations in the *HPX* runtime. The default value is `OFF` because it's not available on all systems and/or requires another dependency. You must set this variable to `ON` in order to use the LCW parcelport. All the following variables only make sense when this variable is set to `ON`.

HPX_WITH_FETCH_LCW

Use FetchContent to fetch LCW. The default value is `OFF`. If this option is set to `OFF`. You need to install your own LCW library and *HPX* will try to find it using CMake[201] `find_package`. You can specify the location of the LCW installation by the environmental variable `LCW_ROOT`. Refer to the LCW README[202] for how to install LCW. If this option is set to `ON`. *HPX* will fetch and build LCW for you. You can use the following CMake[203] variables to configure this behavior for your platform.

HPX_WITH_LCW_TAG

This variable only takes effect when `HPX_WITH_FETCH_LCW` is set to `ON` and `FETCHCONTENT_SOURCE_DIR_LCW` is not set. *HPX* will fetch LCW from its github repository. This variable controls the branch/tag LCW will be fetched.

FETCHCONTENT_SOURCE_DIR_LCW

This variable only takes effect when `HPX_WITH_FETCH_LCW` is set to `ON`. When it is defined, `HPX_WITH_LCW_TAG` will be ignored. It accepts a path to a local version of LCW source code and *HPX* will fetch and build LCW from there.

Run *HPX* with the LCW parcelport

You can configure runtime behavior with the following options.

--hpx:ini=hpx.parcel.lcw.ndevices=<n>

The number of LCW devices to use (default: 2). Each LCW device maps to an MPI communicator or an LCI device. When the MPICH VCI extension is enabled, each MPI communicator will be mapped to a dedicated collection of network resources. More devices lead to lower thread contention, but too many devices may lead to load imbalance or hardware overhead.

Reference

Yan, Jiakun, Marc Snir, and Yanfei Guo. *Examining MPI and its Extensions for Asynchronous Multithreaded Communication.* In *European MPI Users' Group Meeting*, pp. 122-142. Cham: Springer Nature Switzerland, 2025.

[200] https://www.cmake.org
[201] https://www.cmake.org
[202] https://github.com/JiakunYan/lcw#readme
[203] https://www.cmake.org

2.4.18 *HPX* runtime and resources

HPX thread scheduling policies

The *HPX* runtime has six thread scheduling policies: local-priority, static-priority, local, static, local-workrequesting-fifo, and abp-priority. These policies can be specified from the command line using the command line option *--hpx:queuing*. In order to use a particular scheduling policy, the runtime system must be built with the appropriate scheduler flag turned on (e.g. `cmake -DHPX_THREAD_SCHEDULERS=local`, see *CMake options* for more information).

Priority local scheduling policy (default policy)

The priority local scheduling policy maintains one queue per operating system (OS) thread. The OS thread pulls its work from this queue. By default the number of high priority queues is equal to the number of OS threads; the number of high priority queues can be specified on the command line using *--hpx:high-priority-threads*. High priority threads are executed by any of the OS threads before any other work is executed. When a queue is empty, work will be taken from high priority queues first. There is one low priority queue from which threads will be scheduled only when there is no other work.

For this scheduling policy there is an option to turn on NUMA sensitivity using the command line option *--hpx:numa-sensitive*. When NUMA sensitivity is turned on, work stealing is done from queues associated with the same NUMA domain first, only after that work is stolen from other NUMA domains.

This scheduler is enabled at build time by default using the FIFO (first-in-first-out) queueing policy. This policy can be invoked using *--hpx:queuing*`local-priority-fifo`. The scheduler can also be enabled using the LIFO (last-in-first-out) policy. This is not the default policy and must be invoked using the command line option *--hpx:queuing*`local-priority-lifo`.

Static priority scheduling policy

- invoke using: *--hpx:queuing*`static-priority` (or `-qs`)

The static scheduling policy maintains one queue per OS thread from which each OS thread pulls its tasks (user threads). Threads are distributed in a round robin fashion. There is no thread stealing in this policy.

Local scheduling policy

- invoke using: *--hpx:queuing*`local` (or `-ql`)
- flag to turn on for build: `HPX_THREAD_SCHEDULERS=all` or `HPX_THREAD_SCHEDULERS=local`

The local scheduling policy maintains one queue per OS thread from which each OS thread pulls its tasks (user threads).

Static scheduling policy

- invoke using: *--hpx:queuing*`static`
- flag to turn on for build: `HPX_THREAD_SCHEDULERS=all` or `HPX_THREAD_SCHEDULERS=static`

The static scheduling policy maintains one queue per OS thread from which each OS thread pulls its tasks (user threads). Threads are distributed in a round robin fashion. There is no thread stealing in this policy.

Priority ABP scheduling policy

- invoke using: `--hpx:queuingabp-priority-fifo`
- flag to turn on for build: `HPX_THREAD_SCHEDULERS=all` or `HPX_THREAD_SCHEDULERS=abp-priority`

Priority ABP policy maintains a double ended lock free queue for each OS thread. By default the number of high priority queues is equal to the number of OS threads; the number of high priority queues can be specified on the command line using `--hpx:high-priority-threads`. High priority threads are executed by the first OS threads before any other work is executed. When a queue is empty work will be taken from high priority queues first. There is one low priority queue from which threads will be scheduled only when there is no other work. For this scheduling policy there is an option to turn on NUMA sensitivity using the command line option `--hpx:numa-sensitive`. When NUMA sensitivity is turned on work stealing is done from queues associated with the same NUMA domain first, only after that work is stolen from other NUMA domains.

This scheduler can be used with two underlying queuing policies (FIFO: first-in-first-out, and LIFO: last-in-first-out). In order to use the LIFO policy use the command line option `--hpx:queuingabp-priority-lifo`.

Work requesting scheduling policies

- invoke using: `--hpx:queuinglocal-workrequesting-fifo`, using `--hpx:queuinglocal-workrequesting-lifo`, or using `--hpx:queuinglocal-workrequesting-mc`

The work-requesting policies rely on a different mechanism of balancing work between cores (compared to the other policies listed above). Instead of actively trying to steal work from other cores, requesting work relies on a less disruptive mechanism. If a core runs out of work, instead of actively looking at the queues of neighboring cores, in this case a request is posted to another core. This core now (whenever it is not busy with other work) either responds to the original core by sending back work or passes the request on to the next possible core in the system. In general, this scheme avoids contention on the work queues as those are always accessed by their own cores only.

The *HPX* resource partitioner

The *HPX* resource partitioner lets you take the execution resources available on a system—processing units, cores, and numa domains—and assign them to thread pools. By default *HPX* creates a single thread pool name `default`. While this is good for most use cases, the resource partitioner lets you create multiple thread pools with custom resources and options.

Creating custom thread pools is useful for cases where you have tasks which absolutely need to run without interference from other tasks. An example of this is when using MPI[204] for distribution instead of the built-in mechanisms in *HPX* (useful in legacy applications). In this case one can create a thread pool containing a single thread for MPI communication. MPI tasks will then always run on the same thread, instead of potentially being stuck in a queue behind other threads.

Note that *HPX* thread pools are completely independent from each other in the sense that task stealing will never happen between different thread pools. However, tasks running on a particular thread pool can schedule tasks on another thread pool.

> **Note**
>
> It is simpler in some situations to schedule important tasks with high priority instead of using a separate thread pool.

204 https://en.wikipedia.org/wiki/Message_Passing_Interface

Using the resource partitioner

The `hpx::resource::partitioner` is now created during *HPX* runtime initialization without explicit action needed from the user. To specify some of the initialization parameters you can use the *hpx::init_params*.

The resource partitioner callback is the interface to add thread pools to the *HPX* runtime and to assign resources to the thread pools. In order to create custom thread pools you can specify the resource partitioner callback `hpx::init_params::rp_callback` which will be called once the resource partitioner will be created , see the example below. You can also specify other parameters, see *hpx::init_params*.

To add a thread pool use the `hpx::resource::partitioner::create_thread_pool` method. If you simply want to use the default scheduler and scheduler options, it is enough to call `rp.create_thread_pool("my-thread-pool")`.

Then, to add resources to the thread pool you can use the `hpx::resource::partitioner::add_resource` method. The resource partitioner exposes the hardware topology retrieved using Portable Hardware Locality (HWLOC)[205] and lets you iterate through the topology to add the wanted processing units to the thread pool. Below is an example of adding all processing units from the first NUMA domain to a custom thread pool, unless there is only one NUMA domain in which case we leave the first processing unit for the default thread pool:

Note

Whatever processing units are not assigned to a thread pool by the time *hpx::init* is called will be added to the default thread pool. It is also possible to explicitly add processing units to the default thread pool, and to create the default thread pool manually (in order to e.g. set the scheduler type).

Tip

The command line option `--hpx:print-bind` is useful for checking that the thread pools have been set up the way you expect.

Difference between the old and new version

In the old version, you had to create an instance of the `resource_partitioner` with `argc` and `argv`.

```
int main(int argc, char** argv)
{
    hpx::resource::partitioner rp(argc, argv);
    hpx::init();
}
```

From *HPX* 1.5.0 onwards, you just pass `argc` and `argv` to `hpx::init()` or `hpx::start()` for the binding options to be parsed by the resource partitioner.

```
int main(int argc, char** argv)
{
    hpx::init_params init_args;
    hpx::init(argc, argv, init_args);
}
```

[205] https://www.open-mpi.org/projects/hwloc/

In the old version, when creating a custom thread pool, you just called the utilities on the resource partitioner instantiated previously.

```
int main(int argc, char** argv)
{
    hpx::resource::partitioner rp(argc, argv);

    rp.create_thread_pool("my-thread-pool");

    bool one_numa_domain = rp.numa_domains().size() == 1;
    bool skipped_first_pu = false;

    hpx::resource::numa_domain const& d = rp.numa_domains()[0];

    for (const hpx::resource::core& c : d.cores())
    {
        for (const hpx::resource::pu& p : c.pus())
        {
            if (one_numa_domain && !skipped_first_pu)
            {
                skipped_first_pu = true;
                continue;
            }

            rp.add_resource(p, "my-thread-pool");
        }
    }

    hpx::init();
}
```

You now specify the resource partitioner callback which will tie the resources to the resource partitioner created during runtime initialization.

```
void init_resource_partitioner_handler(hpx::resource::partitioner& rp)
{
    rp.create_thread_pool("my-thread-pool");

    bool one_numa_domain = rp.numa_domains().size() == 1;
    bool skipped_first_pu = false;

    hpx::resource::numa_domain const& d = rp.numa_domains()[0];

    for (const hpx::resource::core& c : d.cores())
    {
        for (const hpx::resource::pu& p : c.pus())
        {
            if (one_numa_domain && !skipped_first_pu)
            {
                skipped_first_pu = true;
                continue;
            }

            rp.add_resource(p, "my-thread-pool");
```

(continues on next page)

(continued from previous page)

```
        }
    }
}

int main(int argc, char* argv[])
{
    hpx::init_params init_args;
    init_args.rp_callback = &init_resource_partitioner_handler;

    hpx::init(argc, argv, init_args);
}
```

Advanced usage

It is possible to customize the built in schedulers by passing scheduler options to `hpx::resource::partitioner::create_thread_pool`. It is also possible to create and use custom schedulers.

Note

It is not recommended to create your own scheduler. The *HPX* developers use this to experiment with new scheduler designs before making them available to users via the standard mechanisms of choosing a scheduler (command line options). If you would like to experiment with a custom scheduler the resource partitioner example `shared_priority_queue_scheduler.cpp` contains a fully implemented scheduler with logging, etc. to make exploration easier.

To choose a scheduler and custom mode for a thread pool, pass additional options when creating the thread pool like this:

```
rp.create_thread_pool("my-thread-pool",
    hpx::resource::policies::local_priority_lifo,
    hpx::policies::scheduler_mode(
        hpx::policies::scheduler_mode::default |
        hpx::policies::scheduler_mode::enable_elasticity));
```

The available schedulers are documented here: `hpx::resource::scheduling_policy`, and the available scheduler modes here: `hpx::threads::policies::scheduler_mode`. Also see the examples folder for examples of advanced resource partitioner usage: `simple_resource_partitioner.cpp` and `oversubscribing_resource_partitioner.cpp`.

2.4.19 Executors

Executors in *HPX* provide a flexible way to control **how, when, and where** tasks are executed. Instead of manually creating threads or managing thread pools, you can hand your tasks to an executor, and it takes care of the details of running them.

This page introduces the concept of executors, the main types available in *HPX*, and how to create custom executors.

What is an executor?

An **executor** is an abstraction that separates the *what* from the *how* of task execution:

- **What**: the work to be performed (e.g. a function or task).
- **How**: whether the task runs synchronously, asynchronously, or in parallel across multiple cores.

By using executors, you can switch between execution strategies without rewriting your algorithms. *HPX* provides a rich set of executors with a unified API.

Main Executors in *HPX*

HPX provides multiple executors. Below are three of the most commonly used:

Parallel Executor

- Default in *HPX*.
- Creates a new HPX thread for every scheduled task.
- Works well for large tasks, but frequent small tasks incur overhead due to thread creation/destruction.

```
#include <hpx/execution.hpp>
#include <hpx/algorithm.hpp>
#include <vector>

std::vector<int> data(100, 1);

hpx::for_each(
    hpx::execution::par,
    data.begin(), data.end(),
    [](int &x) {
        x += 1;
    }
);
```

Fork-Join Executor

- Spawns one thread per CPU core when created.
- Threads are reused across tasks, avoiding repeated creation costs.
- Efficient for workloads with many sequential parallel regions.
- May waste resources if parallel work is limited.

```
#include <hpx/execution.hpp>
#include <hpx/algorithm.hpp>
#include <vector>

std::vector<int> data(100, 1);

hpx::execution::experimental::fork_join_executor exec;

hpx::for_each(
    hpx::execution::par.on(exec),
    data.begin(), data.end(),
    [](int &x) {
        x += 1;
    }
);
```

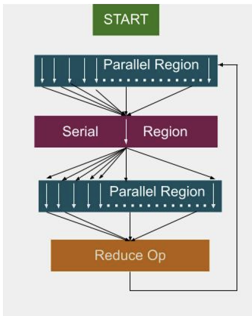
Sequential Executor

- Executes all tasks synchronously on the calling thread.
- Useful for debugging (deterministic execution) or when parallelism brings no benefit.
- Maintains the same executor-based API while running tasks sequentially.

```
#include <hpx/execution.hpp>
#include <hpx/algorithm.hpp>
#include <vector>

std::vector<int> data(100, 1);

hpx::execution::sequenced_executor seq_exec;

hpx::for_each(
    hpx::execution::seq.on(seq_exec),
    data.begin(), data.end(),
    [](int &x) { x += 1; }
);
```

Executors in real-world applications

A practical example is the **LULESH** mini-application (a hydrodynamics benchmark).



- With the **parallel executor**, each parallel loop creates and destroys new threads, introducing overhead.
- With the **fork-join executor**, threads are created once and reused across loops, reducing overhead and improving performance.

In studies, the fork-join executor achieved significant speedups, in some cases more than twice as fast as traditional OpenMP implementations.

Fig. 2.11:
threads for e

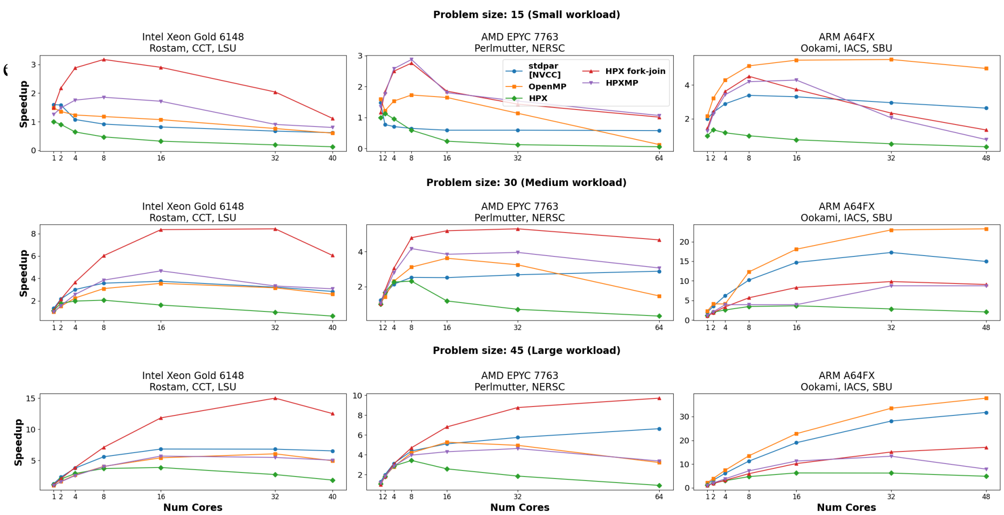
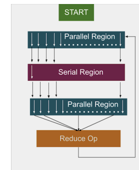


Fig. 2.13: Performance comparison between executors in the LULESH benchmark.

Custom executors

While *HPX* provides a variety of built-in executors, you may sometimes need to adapt task execution to your own requirements. This is where **custom executors** come in. By writing a small wrapper around an existing executor, you can extend its behavior?for example, to add logging, profiling information, or special scheduling rules?while still taking advantage of the *HPX* executor API.

Custom annotating executor



The following example shows how to implement a simple executor that annotates tasks with a string label for easier debugging and profiling.

> **Note**
>
> Annotations do not affect how tasks run or what results they produce. Their main purpose is to give human-readable names to tasks so that they can be identified in profilers, and debuggers.

Full example code

```
#include <hpx/hpx_main.hpp>
#include <hpx/include/parallel_executors.hpp>
#include <hpx/include/async.hpp>
#include <hpx/execution.hpp>
#include <hpx/modules/async_base.hpp>
#include <hpx/modules/threading_base.hpp>

#include <iostream>
#include <string>
#include <utility>

template <typename BaseExecutor>
struct simple_annotating_executor
{
    BaseExecutor base_;
    const char* annotation_;

    simple_annotating_executor(BaseExecutor exec, const char* ann)
        : base_(std::move(exec)), annotation_(ann)
    {}

    // Non-blocking one-way executor
    template <typename F, typename... Ts>
    friend void tag_invoke(hpx::parallel::execution::post_t,
                           simple_annotating_executor const& exec,
                           F&& f, Ts&&... ts)
    {
        hpx::post(
            hpx::annotated_function(std::forward<F>(f), exec.annotation_),
            std::forward<Ts>(ts)...);
    }

    // Synchronous execution
    template <typename F, typename... Ts>
    friend auto tag_invoke(hpx::parallel::execution::sync_execute_t,
```

(continues on next page)

(continued from previous page)

```
                          simple_annotating_executor const& exec,
                          F&& f, Ts&&... ts)
    {
        return hpx::parallel::execution::sync_execute(
            exec.base_,
         ␣
↪   hpx::annotated_function(std::forward<F>(f), exec.annotation_),
            std::forward<Ts>(ts)...);
    }
};

// Example functions
int compute_square(int x)
{
    std::cout << "[sync_execute] Running task with annotation\n";
    return x * x;
}

void say_hello()
{
    std::cout << "[post] Running task with annotation\n";
}

int main()
{
    simple_annotating_executor␣
↪exec(hpx::execution::parallel_executor{}, "my_custom_task");

    // Synchronous execution
    int result = hpx::parallel::execution::sync_
↪execute(exec, &compute_square, 7);
    std::cout << "Result from sync_execute: " << result << "\n";

    // Post a task
    hpx::parallel::execution::post(exec, &say_hello);

    return 0;
}
```

Explanation

The first lines pull in the necessary HPX headers for executors, asynchronous execution, and annotated functions. The key one here is *hpx/threading_base/annotated_function.hpp*, which provides the facility to tag tasks with a string label. We then define a simple_annotating_executor that wraps another executor and associates an annotation string with every task:

```
template <typename BaseExecutor>
struct simple_annotating_executor
{
    BaseExecutor base_;
```

(continues on next page)

(continued from previous page)

```
    const char* annotation_;

    simple_annotating_executor(BaseExecutor exec, const char* ann)
        : base_(std::move(exec)), annotation_(ann)
    {}
};
```

The post customization schedules a task to run asynchronously. We wrap the task in *hpx::annotated_function* so that it carries the annotation. Executors in HPX customize these operations through *tag_invoke* overloads, which are selected by special tag objects like *post_t*` and *sync_execute_t*. This is why the executor interface may look different from a normal member function API.

```
template <typename F, typename... Ts>
friend void tag_invoke(hpx::parallel::execution::post_t,
                       simple_annotating_executor const& exec,
                       F&& f, Ts&&... ts)
{
    hpx::post(
     ␣
↪   hpx::annotated_function(std::forward<F>(f), exec.annotation_),
        std::forward<Ts>(ts)...);
}
```

The *sync_execute* customization runs a task immediately and returns the result. Again, we wrap the function with an annotation before executing. The key difference is that *post* schedules a task in a fire-and-forget style (no result is returned), while *sync_execute* blocks until the task finishes and gives you the result back.

```
template <typename F, typename... Ts>
friend auto tag_invoke(hpx::parallel::execution::sync_execute_t,
                       simple_annotating_executor const& exec,
                       F&& f, Ts&&... ts)
{
    return hpx::parallel::execution::sync_execute(
        exec.base_,
     ␣
↪   hpx::annotated_function(std::forward<F>(f), exec.annotation_),
        std::forward<Ts>(ts)...);
}
```

Note how we delegate the actual execution to *exec.base_*, the underlying executor. This makes the custom executor lightweight: it only adds annotations, while leaving the scheduling strategy to the base executor (here, a *parallel_executor*).

We define two simple functions to demonstrate both synchronous and asynchronous execution: These functions also print to *std::cout*, but this output is not the actual annotation. Annotations are stored internally by HPX and become visible when you use debugging or profiling tools.

```
int compute_square(int x)
{
    std::cout << "[sync_execute] Running task with annotation\n";
```

(continues on next page)

(continued from previous page)

```
    return x * x;
}

void say_hello()
{
    std::cout << "[post] Running task with annotation\n";
}
```

Finally, in main we create the executor with a base executor and annotation string. We then run one task with sync_execute (blocking, returns result) and one with post (asynchronous, fire-and-forget). You can also create multiple annotating executors with different strings, so each task gets its own label. This is especially useful in larger applications with many different kinds of tasks, where annotations make it much easier to trace what is happening.

```
int main()
{
    simple_annotating_executor␣
↪exec(hpx::execution::parallel_executor{}, "my_custom_task");

    // Synchronous execution
    int result = hpx::parallel::execution::sync_
↪execute(exec, &compute_square, 7);
    std::cout << "Result from sync_execute: " << result << "\n";

    // Post a task
    hpx::parallel::execution::post(exec, &say_hello);

    return 0;
}
```

Custom annotating executor with parallel algorithms

The following example demonstrates how to use a custom annotating executor with *HPX* parallel algorithms, such as *for_each*. This allows you to attach annotations to tasks while executing them in parallel.

Full example code

```
#include <hpx/execution.hpp>
#include <hpx/hpx_main.hpp>
#include <hpx/include/parallel_algorithm.hpp>
#include <hpx/include/parallel_executors.hpp>
#include <hpx/modules/threading_base.hpp>

#include <iostream>
#include <utility>
#include <vector>
```

(continues on next page)

(continued from previous page)

```
template <typename BaseExecutor>
struct simple_annotating_executor
{
    BaseExecutor base_;
    char const* annotation_;

    using execution_category =
        hpx::traits::executor_execution_category_t<BaseExecutor>;

    simple_annotating_executor(BaseExecutor exec, char const* ann)
    : base_(std::move(exec))
    , annotation_(ann)
    {
    }

    // Bulk async_execute (used by parallel algorithms)
    template <typename F, typename Shape, typename... Ts>
    friend␣
↪auto tag_invoke(hpx::parallel::execution::bulk_async_execute_t,
        simple_
↪annotating_executor const& exec, F&& f, Shape const& shape,
        Ts&&... ts)
    {
        return hpx::parallel::execution::bulk_async_execute(
            exec.base_,
                ␣
↪   hpx::annotated_function(std::forward<F>(f), exec.annotation_),
            shape, std::forward<Ts>(ts)...);
    }
};

namespace hpx::execution::experimental {

    // The␣
↪annotating executor exposes the same executor categories as its
    // underlying (wrapped) executor.

    template <typename BaseExecutor>
    struct␣
↪is_two_way_executor<simple_annotating_executor<BaseExecutor>>
    : is_two_way_executor<BaseExecutor>
    {
    };
}    // namespace hpx::execution::experimental

int main()
{
    using base_executor = hpx::execution::parallel_executor;
    simple_
↪annotating_executor exec(base_executor{}, "for_each_task");

    std::vector<int> data = {1, 2, 3, 4, 5};
```

(continues on next page)

(continued from previous page)

```
    // Use the custom executor with a parallel algorithm
    hpx::for_
↪each(hpx::execution::par.on(exec),    // attach executor
        data.begin(), data.end(), [](int& x) {
            std::cout << "Processing " << x << " on thread "
                    << hpx::get_worker_thread_num() << "\n";
            x *= x;
        });

    std::cout << "Squared values: ";
    for (int v : data)
        std::cout << v << " ";
    std::cout << "\n";

    return 0;
}
```

Explanation

Similar as before, the first lines pull in the necessary HPX headers for executors, asynchronous execution, and annotated functions. The key one here is *hpx/threading_base/annotated_function.hpp*, which provides the facility to tag tasks with a string label. We then define a simple_annotating_executor that wraps another executor and associates an annotation string with every task:

```
template <typename BaseExecutor>
struct simple_annotating_executor
{
    BaseExecutor base_;
    const char* annotation_;

    using execution_category =
        hpx::traits::executor_execution_category_t<BaseExecutor>;

    simple_annotating_executor(BaseExecutor exec, const char* ann)
        : base_(std::move(exec)), annotation_(ann)
    {}
};
```

Note that we expose the execution category of the custom executor with *using execution_category = hpx::traits::executor_execution_category_t<BaseExecutor>*. We inherit the execution category from the underlying executor (*BaseExecutor`*), which ensures that our *simple_annotating_executor* behaves like the base executor in terms of parallelism and task execution capabilities.

The *bulk_async_execute* customization schedules a set of tasks to run asynchronously. We wrap each task in *hpx::annotated_function* so that it carries the annotation. Executors in *HPX* customize these operations through *tag_invoke* overloads, which are selected by special tag objects like *bulk_async_execute_t*. This is why the executor interface may look different from a normal member function API - it uses tag dispatch to integrate seamlessly with the *HPX* parallel algorithms infrastructure.

```
template <typename F, typename Shape, typename... Ts>
friend␣
↪auto tag_invoke(hpx::parallel::execution::bulk_async_execute_t,
    simple_
↪annotating_executor const& exec, F&& f, Shape const& shape,
    Ts&&... ts)
{
    return hpx::parallel::execution::bulk_async_execute(
        exec.base_,
        ␣
↪   hpx::annotated_function(std::forward<F>(f), exec.annotation_),
        shape, std::forward<Ts>(ts)...);
}
```

Note how we delegate the actual execution to *exec.base_*, the underlying executor. This makes the custom executor lightweight: it only adds annotations, while leaving the scheduling strategy to the base executor (here, a *parallel_executor*).

The *hpx::execution::experimental* namespace contains traits that describe executor capabilities, such as whether an executor can run tasks one-way, two-way, or never-blocking. These traits are used internally by *HPX* to verify that an executor is compatible with a given parallel algorithm or execution policy.

```
namespace hpx::execution::experimental {

    // The␣
↪annotating executor exposes the same executor categories as its
    // underlying (wrapped) executor.

    template <typename BaseExecutor>
    struct is_never_blocking_one_way_executor<
        simple_annotating_executor<BaseExecutor>>
    : is_never_blocking_one_way_executor<BaseExecutor>
    {
    };

    template <typename BaseExecutor>
    struct␣
↪is_one_way_executor<simple_annotating_executor<BaseExecutor>>
    : is_one_way_executor<BaseExecutor>
    {
    };

    template <typename BaseExecutor>
    struct␣
↪is_two_way_executor<simple_annotating_executor<BaseExecutor>>
    : is_two_way_executor<BaseExecutor>
    {
    };
}
```

- *is_never_blocking_one_way_executor* indicates whether the executor can schedule tasks in a fire-and-forget style without blocking.
- *is_one_way_executor* indicates support for one-way execution (tasks can be scheduled

but no result is returned).

- *is_two_way_executor* indicates support for two-way execution (tasks return a result or a future).

In all cases, the custom executor inherits the capabilities of the base executor, so it integrates seamlessly with *HPX* algorithms.

This design ensures that *simple_annotating_executor* can be used anywhere its underlying executor could be used, while still adding the annotation functionality. It keeps the custom executor lightweight and fully compatible with the parallel algorithms infrastructure.

In main, we create the executor with a base executor and an annotation string, and then use it with a parallel algorithm:

```
int main()
{
    using base_executor = hpx::execution::parallel_executor;
    simple_
↪annotating_executor exec(base_executor{}, "for_each_task");

    std::vector<int> data = {1, 2, 3, 4, 5};

    // Use the custom executor with a parallel algorithm
    hpx::for_
↪each(hpx::execution::par.on(exec),    // attach executor
        data.begin(), data.end(), [](int& x) {
            std::cout << "Processing " << x << " on thread "
                    << hpx::get_worker_thread_num() << "\n";
            x *= x;
        });

    std::cout << "Squared values: ";
    for (int v : data)
        std::cout << v << " ";
    std::cout << "\n";

    return 0;
}
```

First, we create a base executor (*parallel_executor*) and wrap it in our *simple_annotating_executor*, providing an annotation string “for_each_task”. This custom executor will attach the annotation to every task it schedules, while delegating actual execution to the base executor.

We then use hpx::for_each with a parallel execution policy and attach our custom executor using *par.on(exec)*: * hpx::execution::par.on(exec) attaches our custom executor to the algorithm. * for_each internally partitions the work across threads and schedules each task using *bulk_async_execute*. * Each task is annotated with “for_each_task”, visible in debuggers and profilers. * The results of the parallel computation are stored in the data vector, demonstrating that the algorithm

executed successfully in parallel.

This pattern is especially useful in larger applications with many tasks, as annotations make it much easier to trace and debug the execution of parallel algorithms.

2.4.20 Error handling

Like in any other asynchronous invocation scheme, it is important to be able to handle error conditions occurring while the asynchronous (and possibly remote) operation is executed. In *HPX* all error handling is based on standard C++ exception handling. Any exception thrown during the execution of an asynchronous operation will be transferred back to the original invocation *locality*, where it will be rethrown during synchronization with the calling thread.

The source code for this example can be found here: `error_handling.cpp`.

Working with exceptions

For the following description assume that the function `raise_exception()` is executed by invoking the plain action `raise_exception_type`.

```
void raise_exception()
{
    HPX_THROW_EXCEPTION(
        ␣
↪   hpx::error::no_success, "raise_exception", "simulated error");
}
HPX_PLAIN_ACTION(raise_exception, raise_exception_action)
```

The exception is thrown using the macro *HPX_THROW_EXCEPTION*. The type of the thrown exception is *hpx::exception*. This associates additional diagnostic information with the exception, such as file name and line number, *locality* id and thread id, and stack backtrace from the point where the exception was thrown.

Any exception thrown during the execution of an action is transferred back to the (asynchronous) invocation site. It will be rethrown in this context when the calling thread tries to wait for the result of the action by invoking either `future<>::get()` or the synchronous action invocation wrapper as shown here:

> **Note**
>
> The exception is transferred back to the invocation site even if it is executed on a different *locality*.

Additionally, this example demonstrates how an exception thrown by an (possibly remote) action can be handled. It shows the use of *hpx::diagnostic_information*, which retrieves all available diagnostic information from the exception as a formatted string. This includes, for instance, the name of the source file and line number, the sequence number of the OS thread and the *HPX* thread id, the *locality* id and the stack backtrace of the point where the original exception was thrown.

Under certain circumstances it is desirable to output only some of the diagnostics, or to output those using different formatting. For this case, *HPX* exposes a set of lower-level functions as demonstrated in the following code snippet:

Working with error codes

Most of the API functions exposed by *HPX* can be invoked in two different modes. By default those will throw an exception on error as described above. However, sometimes it is desirable not to throw an exception in case of an error condition. In this case an object instance of the *hpx::error_code* type can be passed as the last argument to the API function. In case of an error, the error condition will be returned in that *hpx::error_code* instance. The following example demonstrates extracting the full diagnostic information without exception handling:

```
            hpx::cout << "Error reporting using error code\n";

            // Create a new error_code instance.
            hpx::error_code ec;

            // If an␣
↪instance of an error_code is passed as the last argument while
            // invoking␣
↪the action, the function will not throw in case of an error
            // but␣
↪store the error information in this error_code instance instead.
            raise_exception_action do_it;
            do_it(hpx::find_here(), ec);

            if (ec)
            {
                // Print just the essential error information.
                ␣
↪    hpx::cout << "returned error: " << ec.get_message() << "\n";

                //␣
↪Print all of the available diagnostic information as stored with
                // the exception.
                hpx::cout << "diagnostic information:"
   ␣
↪                      << hpx::diagnostic_information(ec) << "\n";
            }

            hpx::cout << std::flush;
```

Note

The error information is transferred back to the invocation site even if it is executed on a different *locality*.

This example show how an error can be handled without having to resolve to exceptions and that the returned *hpx::error_code* instance can be used in a very similar way as the *hpx::exception* type above. Simply pass it to the `hpx::diagnostic_information`, which retrieves all available diagnostic information from the error code instance as a formatted string.

As for handling exceptions, when working with error codes, under certain circumstances it is desirable to output only some of the diagnostics, or to output those using different formatting. For this case, *HPX* exposes a set of lower-level functions usable with error codes as demonstrated in the following code snippet:

```
␣
↪     hpx::cout << "Detailed error reporting using error code\n";

          // Create a new error_code instance.
          hpx::error_code ec;

          // If an␣
↪instance of an error_code is passed as the last argument while
          // invoking␣
↪the action, the function will not throw in case of an error
          // but␣
↪store the error information in this error_code instance instead.
          raise_exception_action do_it;
          do_it(hpx::find_here(), ec);

          if (ec)
          {
                ␣
↪ // Print the elements of the diagnostic information separately.
            ␣
↪     hpx::cout << "{what}: " << hpx::get_error_what(ec) << "\n";
                ␣
↪hpx::cout << "{locality-id}: " << hpx::get_error_locality_id(ec)
                          << "\n";
            ␣
↪     hpx::cout << "{hostname}: " << hpx::get_error_host_name(ec)
                          << "\n";
                ␣
↪hpx::cout << "{pid}: " << hpx::get_error_process_id(ec) << "\n";
                ␣
↪ hpx::cout << "{function}: " << hpx::get_error_function_name(ec)
                          << "\n";
                ␣
↪hpx::cout << "{file}: " << hpx::get_error_file_name(ec) << "\n";
          ␣
↪       hpx::cout << "{line}: " << hpx::get_error_line_number(ec)
                          << "\n";
             ␣
↪    hpx::cout << "{os-thread}: " << hpx::get_error_os_thread(ec)
                          << "\n";
              hpx::cout << "{thread-id}: " << std::hex
                          << hpx::get_error_thread_id(ec) << "\n";
              hpx::cout << "{thread-description}: "
              ␣
↪             << hpx::get_error_thread_description(ec) << "\n\n";
                ␣
↪hpx::cout << "{state}: " << std::hex << hpx::get_error_state(ec)
                          << "\n";
               ␣
↪  hpx::cout << "{stack-trace}: " << hpx::get_error_backtrace(ec)
```

(continues on next page)

(continued from previous page)

```
                          << "\n";
↪       hpx::cout << "{env}: " << hpx::get_error_env(ec) << "\n";
            }

            hpx::cout << std::flush;
```

For more information please refer to the documentation of hpx::get_error_what, *hpx::get_error_locality_id*, *hpx::get_error_host_name*, *hpx::get_error_process_id*, hpx::get_error_function_name, hpx::get_error_file_name, hpx::get_error_line_number, *hpx::get_error_os_thread*, *hpx::get_error_thread_id*, *hpx::get_error_thread_description*, *hpx::get_error_backtrace*, *hpx::get_error_env*, and *hpx::get_error_state*.

Lightweight error codes

Sometimes it is not desirable to collect all the ambient information about the error at the point where it happened as this might impose too much overhead for simple scenarios. In this case, *HPX* provides a lightweight error code facility that will hold the error code only. The following snippet demonstrates its use:

```
            hpx::cout␣
↪<< "Error reporting using an lightweight error code\n";

            // Create a new error_code instance.
            hpx::error_code ec(hpx::throwmode::lightweight);

            // If an␣
↪instance of an error_code is passed as the last argument while
            // invoking␣
↪the action, the function will not throw in case of an error
            // but␣
↪store the error information in this error_code instance instead.
            raise_exception_action do_it;
            do_it(hpx::find_here(), ec);

            if (ec)
            {
                // Print just the essential error information.
↪     hpx::cout << "returned error: " << ec.get_message() << "\n";

                //␣
↪Print all of the available diagnostic information as stored with
                // the exception.
                hpx::cout << "error code:" << ec.value() << "\n";
            }

            hpx::cout << std::flush;
```

All functions that retrieve other diagnostic elements from the *hpx::error_code* will fail if called with a lightweight error_code instance.

2.4.21 The *HPX* I/O-streams component

The *HPX* I/O-streams subsystem extends the standard C++ output streams `std::cout` and `std::cerr` to work in the distributed setting of an *HPX* application. All of the output streamed to `hpx::cout` will be dispatched to `std::cout` on the console *locality*. Likewise, all output generated from `hpx::cerr` will be dispatched to `std::cerr` on the console *locality*.

Note

All existing standard manipulators can be used in conjunction with `hpx::cout` and `hpx::cerr`.

In order to use either `hpx::cout` or `hpx::cerr`, application codes need to `#include <hpx/include/iostreams.hpp>`. For an example, please see the following 'Hello world' program:

```
// Including␣
↪'hpx/hpx_main.hpp' instead of the usual 'hpx/hpx_init.hpp' enables
// to␣
↪use the plain C-main below as the direct main HPX entry point.
#include <hpx/hpx_main.hpp>
#include <hpx/iostream.hpp>

int main()
{
    // Say hello to the world!
    hpx::cout << "Hello World!\n" << std::flush;
    return 0;
}
```

Additionally, those applications need to link with the iostreams component. When using CMake this can be achieved by using the `COMPONENT_DEPENDENCIES` parameter; for instance:

```
include(HPX_AddExecutable)

add_hpx_executable(
    hello_world
    SOURCES hello_world.cpp
    COMPONENT_DEPENDENCIES iostreams
)
```

Note

The `hpx::cout` and `hpx::cerr` streams buffer all output locally until a `std::endl` or `std::flush` is encountered. That means that no output will appear on the console as long as either of these is explicitly used.

2.4.22 Supervision dispatch

The supervision dispatch component (`hpx::supervision`, built on top of the *Supervision module* module) lets a locality discover its peers, detect when one of them has gone silent, and safely stop dispatching work to a peer that has latched a terminal lifecycle event - all without every caller needing to make an authoritative, synchronous round-trip for every dispatch.

This page describes the initialization and shutdown ordering applications must follow, the one-shot nature of peer discovery, the distinction between the locally cached peer state and the registry's authoritative state, and how that distinction shows up in the two-tier admission check used by fenced dispatch. For the detailed semantics of individual lifecycle events, `check_admission`, and `dispatch_outcome` values, see *Supervision module*; this page does not repeat that reference material. For the API reference of the dispatch-specific types and functions this component introduces - `registry`, `discovery::discover_and_join`, and `dispatch_work` - see *Supervision dispatch module*.

Initialization and shutdown ordering

Supervision dispatch is brought up with a single call to *hpx::supervision::init* and torn down with a single call to *hpx::supervision::finalize*:

```
#include <hpx/supervision_dispatch.hpp>

// Blocking, one-shot startup for this locality.
hpx::supervision::init(hpx::launch::sync);

// ... application runs, dispatching fenced work to peers ...

// Cooperative, one-shot shutdown for this locality.
hpx::supervision::finalize();
```

`init()` performs, in order:

1. Creation of a local `registry` component for this locality.
2. Publication of `event::started` on the locality - *before* the component's symbol name is registered with AGAS, so that no peer can discover and join a not-yet-started locality.
3. Registration of the symbol name.
4. A single *hpx::supervision::discover_and_join* pass that joins every peer registry reachable within `discovery_timeout`. Its result is a list of `joined_discovery_result` (peer paired with the shadow id its `join()` call was assigned); peers that time out mid-pass are omitted from that list entirely rather than leaving a gap in it.

   `init()` is idempotent: calling it while already active is a no-op, and concurrent callers attach to a single in-flight initialization rather than racing to create duplicate registries. `finalize()` mirrors this ordering in reverse - it publishes `event::completed` for the local locality, unregisters the symbol name, and releases the `registry` component - and is always safe to call speculatively (e.g. during shutdown) even if `init()` was never called, since it is a no-op unless the runtime is currently active.

Applications should treat `init()`/`finalize()` as a matched pair around the portion of the program that performs supervised, fenced dispatch, and should not assume any lifecycle event is visible to peers before `init()`'s future (or synchronous call) has completed.

Calling `init()` again after it has already succeeded is a documented no-op: it returns an already-ready future without repeating any of the four steps above, and in particular does **not** re-run `discover_and_join()`. Peers that started, or otherwise became discoverable, after the first `init()` call completed are *not* retroactively added to the registry by a second call – the one-shot discovery behavior described below applies equally to repeated `init()` calls, not just the first one. To pick up such a peer, join it explicitly via `registry::join()`. Calling `init()` while a previous call is still in progress attaches to that same in-flight initialization rather than starting a second one; calling it while `finalize()` is concurrently in progress fails immediately with `hpx::error::invalid_status` rather than queuing behind it.

What is a target, and how is it introduced

A "target" is nothing more than an `hpx::id_type` - the id of any locality or component whose lifecycle should be tracked. It is not a distinct kind of object, and there is no separate handle or wrapper type: the same id you'd use to invoke an action on a component is what you pass as `target` to `publish_event()`, `check_admission()`, `dispatch_work()`, and the rest of the supervision API.

Because `dispatch_work()` resolves `target` via `hpx::colocated()` to find the locality to dispatch to, a target must also be independently resolvable by AGAS to some locality - this is a separate requirement from having recorded supervision state, and it is not something supervision dispatch itself grants. Any id obtained the ordinary way (creating a component, `hpx::find_here()`, a symbolic-name lookup, a peer's joined registry id) already satisfies this. If a target id can no longer be resolved to a locality - for instance, because the underlying component was already destroyed and unregistered - `dispatch_work()` fails when `hpx::colocated()` cannot resolve it, *before* either admission check runs; this is a distinct failure mode from `hpx::error::target_fenced` and is not something publishing `event::started` can paper over.

In practice, two kinds of ids show up as targets in this component:

- The registry id of peer localities, once joined via *`hpx::supervision::registry::join`* - these are the targets whose shadow state feeds `registry::snapshot_peers()` and local admission filtering.
- The ids of individual localities/components that fenced work is dispatched against via *hpx::supervision::dispatch_work* (`dispatch_work(target, epoch, ...)`) - these need not be peers' localities at all; any component id that has had lifecycle events published for it can be fenced.

Unlike joining a peer registry, there is no explicit "register this target" call for the underlying supervision manager. A target is introduced implicitly, purely as a side effect of publishing its first event: the supervision manager tracks targets in a map keyed by id, and a lookup miss is treated as "this target has never been seen," with an implicit current epoch of `0` and a default event of `unknown`. The lifecycle-transition validation then requires that the *first* event ever published for a target open with `started` - publishing anything else for a target with no prior recorded state is rejected as an invalid transition. Put simply:

```
// Introduces␣
↪`some_actor` as a tracked target under epoch 0. This is the
// only step required - no prior registration call exists.
```

(continues on next page)

(continued from previous page)

```
hpx::supervision::publish_
→event(some_actor, hpx::supervision::event::started, 0);

// From␣
→this point on, `some_actor` has recorded state: query_state(),
// check_
→admission(), and dispatch_work() against it now reflect that
// state rather than "unknown."
```

Before a target's first `started` publication, it has no recorded state at all: `query_state()` reports it as `unknown` rather than as any specific lifecycle stage, and `check_admission()` for an epoch-0, never-published target behaves as admitting (there is nothing latched to fence against yet). Once introduced, a target's tracked state persists on the supervision manager that owns it until either it reaches a terminal event that is superseded by a higher epoch (see "Epochs and when to change them" below), or it is explicitly forgotten via `remove_target()` - typically done by callers that know a given id will never be queried or observed again locally, e.g. after a failed registration attempt or once a peer has been evicted from the registry, to avoid accumulating unbounded per-target state.

Removing a target

hpx::supervision::remove_target is the counterpart to introducing a target: it unconditionally erases whatever supervision state is locally recorded for a target id, on a single locality.

```
// Synchronous version, target on a possibly remote locality:
hpx::supervision::remove_
→target(hpx::launch::sync, locality, target);
```

See *Supervision module* for the full semantics of `remove_target`, and *Supervision dispatch module* for this component's own API reference.

A few points worth calling out:

- **It is a forget operation, not a lifecycle event.** It erases the target's tracked state (epoch, current event, sequence number) and any observers still registered for it, without publishing an event of any kind. This is different from a target reaching a terminal event – removal discards history rather than recording an outcome.
- **It is local only.** Removal only clears state held on the given `locality` (or the calling locality, for the local-only overload). It does not touch `target`'s state on any other locality, and it does not affect AGAS registration or symbolic names for that id.
- **It is safe to call on a target with no recorded state.** This is a documented no-op rather than an error, so callers do not need to first check whether a target was ever introduced before removing it.
- **Republishing afterward reintroduces the target from scratch.** After removal, publishing `event::started` for the same id behaves exactly as if the id had never been seen before - epoch resets to whatever the new `started` publication specifies.
- **Safe to call from within the target's own observer callback.** Calling `remove_target()` on `target` from inside one of `target`'s own observer callbacks is supported and does not deadlock.

Removing a target is the natural cleanup step once it is known to be permanently done with - for example, once a peer has been evicted from the registry, or once a fenced target has reached a terminal event and will not be reincarnated under a new epoch - so that per-target state does not accumulate indefinitely over the lifetime of a long-running application.

Application-visible state machines

Applications observe two distinct, overlapping state machines.

Per-locality dispatcher lifecycle

Tracked once per locality (`dispatch_api.cpp`), driven by `init()`/ `finalize()`:

```
uninitialized␣
↪-> initializing -> active -> finalizing -> uninitialized
```

- `uninitialized`: no local registry exist.
- `initializing`: registry creation, `event::started` publication, symbol registration, and the one-shot `discover_and_join()` pass are in progress. Concurrent callers attach to the same in-flight initialization rather than racing.
- `active`: heartbeat and failure-detection background tasks are running.
- `finalizing`: background tasks are being stopped, `event::completed` is published, and the symbol name is being unregistered before component release.

  `init()/finalize()` are idempotent no-ops when already in the target state. A failure during `initializing` unregisters any names already registered and returns to `uninitialized` rather than leaving the locality stuck mid-transition.

Per-target lifecycle event

Reported by an actor about itself (`hpx::supervision::event`), tracked per epoch and mirrored into peer shadows:

```
unknown -> started -> running -> suspending -> completed
                   \-> failed
                   \-> losing_locality
```

`completed` and `failed` are terminal: once published for a target/epoch, later terminal publications for that same target/epoch are rejected as already-terminal, and publications carrying a lower epoch are rejected as stale. `losing_locality` is a non-terminal warning that the hosting locality may become unavailable.

How they interact

The dispatcher-lifecycle state machine governs whether *this locality* can participate at all. The per-target event state machine governs whether *a specific peer* remains eligible for fenced dispatch. Failure detection moves a peer's locally cached shadow toward `failed` only when a timeout on `await_terminal` coincides with an unchanged event sequence number - a timeout alone is not sufficient, since the peer may simply still be `running`. This local `failed` marking never touches the peer's own authoritative state; it only affects what this locality's client-side filtering sees, per the shadow/authoritative distinction above.

Heartbeats: distinguishing idle from dead

While a locality is `active` (see "Application-visible state machines" above), a background heartbeat task runs alongside the failure-detection loop. Its job is narrow but essential: periodically republish `event::running` against the locality, purely to keep that localty's event sequence number and timestamp advancing even when the application itself has nothing new to report.

Why this is needed

Failure detection works by calling `await_terminal()` with a timeout against each joined peer's shadow. On its own, a bare timeout on that call is ambiguous: it fires identically whether the peer has crashed, or the peer is simply alive and quietly idle – `running` is a stable state that can legitimately persist for a long time with no events published at all. Without some independent signal of liveness, failure detection would have no way to tell these two situations apart, and would either fence peers that are perfectly healthy but quiet, or fail to fence peers that have genuinely died.

The heartbeat resolves this by giving every alive peer a steady, low-cost source of forward progress – its event sequence number – independent of whatever the application is actually doing. This lets `peer_is_alive_but_silent()` compare a peer's sequence number immediately before and after an `await_terminal()` timeout:

- If the sequence number **advanced** during the timeout window, the peer's heartbeat kept running, so the peer is alive but simply not reporting a terminal event - no action is taken.
- If the sequence number **did not change**, the peer's own heartbeat task has itself gone silent, which is treated as genuine evidence of a stall, and the failure-detection loop fences the peer locally by publishing `event::failed` against its shadow (never against the peer's own, remote, authoritative state - see "Shadow state versus the authoritative registry" above).

Operational properties

- **Best-effort.** A failed heartbeat publication attempt does not stop or crash the loop; it is simply retried on the next interval.
- **Fixed interval, tied to detection timing.** The heartbeat republishes at roughly a third of the failure-detection poll timeout, so that under normal conditions several heartbeats land within any single detection window – making a missed heartbeat a meaningful signal rather than noise from timing jitter.
- **Started and stopped in lockstep with failure detection.** Both background tasks are started together, immediately after the one-shot `discover_and_join()` pass inside `init()`, and both are signaled to stop and joined together during `finalize()`, before `event::completed` is published and either symbol name is unregistered.
- **Entirely internal.** Applications do not start, stop, or configure the heartbeat directly, and do not need to publish any events themselves to keep it running – it is a side effect of calling `init()`, and requires no ongoing application action beyond staying inside the `init()/finalize()` bracket.

  In short, the heartbeat is what makes failure detection's timeout-based stall check meaningful at all: without it, a timeout by itself would not be able to distinguish "no news" from "no pulse."

  This is also why fenced dispatch depends on the heartbeat working correctly, not just on failure detection running. `check_admission()` - whether called as the caller's local, non-authoritative early-out or as the target's own authoritative re-check inside `invoke_fenced_action()` - ultimately answers "is this target still available" by consulting whatever lifecycle state has been latched for it, which for peers is only ever updated to `failed` once failure detection has genuine evidence of a stall. Without the heartbeat, that evidence would not exist: a silent-but-alive peer and a genuinely dead one would look identical to `await_terminal()`, and admission would have no reliable basis for deciding whether it is safe to keep dispatching work to that locality. In short, the heartbeat is not just an internal detail of failure detection - it is what makes the fencing decisions applications rely on throughout this page trustworthy in the first place.

One-shot discovery and staleness

The `discover_and_join()` pass performed by `init()` runs exactly once. It finds whichever peer registries are reachable *at that moment* and joins them; it does not re-run periodically, and peers that join the system later are not automatically discovered by localities that have already initialized. This has two practical implications:

- A locality's set of known peers is fixed at `init()` time (plus any peers joined explicitly afterwards via `registry::join()`). It does not grow on its own as new peers appear.
- Because failure detection (see below) only tracks peers that have actually been joined, a peer that starts too late to be discovered is invisible to both admission filtering and fencing until it is joined explicitly.

Applications that need a stable, complete peer set at some later point should join peers explicitly (`registry::join()`) rather than relying on rediscovery after `init()`.

Shadow state versus the authoritative registry

Once a peer is joined, its lifecycle state is mirrored locally into a *shadow* target owned by the joining registry - a lightweight, locally readable copy of that peer's most recently observed state, updated as lifecycle and heartbeat events arrive. This shadow state is what `registry::snapshot_peers()` and the local admission check consult.

It is important to keep two things distinct:

- **Shadow state** (local): a locally cached view of a peer's lifecycle, updated asynchronously as events are received. It can lag reality by however long it takes an event to propagate, and can go stale entirely if the peer's heartbeat has stopped but failure detection has not yet timed out.
- **Registry state** (remote, authoritative): the actual lifecycle/fencing state tracked by the peer's own supervision manager, on the peer's own locality. This is the only state that is authoritative for admission decisions.

Failure detection narrows - but does not eliminate - the gap between the two: a background loop periodically checks each joined peer's shadow for silence and marks it accordingly, but there is always a window between a peer actually going silent and that silence being locally observed.

How registries stay synchronized

The shadow-mirroring described above is not maintained by polling. It is driven entirely by event-based observer callbacks, registered once at `registry::join()` time and then firing automatically for the lifetime of the joined peer.

Registration, at join time

When `registry::join(peer_locality)` is called, the local registry does two things before returning a shadow id to the caller. A peer's identity *is* its locality since exactly one registry exists per locality.

1. **Seeds the shadow with the peer's current state.** It performs a single, synchronous `query_state()` call against `peer_locality` to read whatever epoch the peer is currently in, then publishes `event::started` for that epoch onto a freshly minted shadow id. This seed step exists so the shadow reflects the peer's state as of joining, rather than starting from `unknown` and waiting for the first subsequent event to arrive.
2. **Registers a lifecycle-event observer** on `peer_locality` (via `hpx::supervision::register_observer()`), along with an activity-transition observer (via `register_activity_observer()`, currently a no-op placeholder reserved for future use). These registrations live on the peer's own locality and are what make the subsequent synchronization automatic: no further polling or explicit refresh call from the local registry is needed.

Both registrations are made atomically with respect to concurrent `join()` calls for the same peer (a reservation is taken in `peers_` before either remote call is issued), and either observer failing to register cleanly rolls back the other, so a peer is never left half-joined.

Ongoing synchronization

From then on, every event the peer publishes for itself - including the heartbeat's periodic `event::running` republication (see "Heartbeats: distinguishing idle from dead" above) – invokes the registered lifecycle observer callback on the *peer's* locality, which in turn calls back into the joining registry to mirror that event onto the local shadow, at the same epoch it actually occurred in. No round trip is initiated by the joining side to "check in" on a peer; synchronization is entirely push-driven by whichever locality actually observes the peer's own state changing.

If the mirrored event is terminal (`completed` or `failed`), the callback additionally schedules - via `hpx::post()`, deferred rather than inline, since the callback itself does not hold the registry's lock - the peer's eviction from `peers_`. Once evicted, the peer no longer appears in `registry::snapshot_peers()`, and its observer registrations are unregistered on the peer's locality so nothing is left listening for further events from a peer that has already reached a terminal state.

What this does and does not cover

This automatic synchronization keeps the shadow up to date with whatever the peer *chooses to publish* - it is not a substitute for failure detection. A peer that stops publishing anything at all (e.g. because it crashed) generates no event, so no observer callback fires, and the shadow simply stops advancing. Detecting that silence, and deciding whether it means the peer is dead rather than merely idle, is exactly the job of the heartbeat/failure-detection pairing described earlier in this page – the observer-based synchronization described here and the polling-with-timeout failure detection described earlier are complementary, not overlapping, mechanisms.

Because `observers_` is keyed per target and holds a list rather than a single callback, this fan-out is genuinely one-to-many: if several localities have each independently joined the same peer, a single event published by that peer notifies every one of their registries in the same pass, each mirroring the event onto its own local shadow. This reach is scoped to the publishing target, however – only localities that have actually called `join()` against that specific peer are registered as observers and receive its events. A locality that has not joined a given peer is not notified when that peer publishes, regardless of whether it participates in supervision dispatch generally.

Client-side filtering versus fenced-dispatch admission

This shadow/authoritative distinction is exactly why fenced dispatch performs *two* admission checks rather than one:

- **Client-side filtering** (`check_admission()` called directly by the caller, or peer filtering based on `registry::snapshot_peers()`) consults only the locally cached shadow state. It is fast and involves no remote round-trip, but its answer can be stale: it may say a peer is admitted when the peer has, in fact, already latched a terminal event that just hasn't propagated back yet. This check is an optimization - a cheap way to avoid dispatching work that is *already known locally* to be pointless - and must never be treated as authoritative.
- **Fenced-dispatch admission** (the re-check performed inside `invoke_fenced_action()`, once the dispatched action has actually reached the target's own locality via `hpx::colocated`) runs on the same locality/thread that owns the target's authoritative supervision state, with no suspension point between the check

and the action invocation that follows it. This is the check that actually decides whether the wrapped action runs, and it is what closes the race between a stale client-side "admitted" result and a peer that has since gone terminal.

In short: use client-side filtering to avoid *obviously* wasted dispatches; rely only on the fenced-dispatch outcome to know whether work actually ran.

Epochs and when to change them

Every lifecycle event and every fenced dispatch is scoped to an epoch (`std::uint64_t`, defaulting to `0`). The epoch is what lets a target be "un-fenced" and reused after it has latched a terminal event, instead of being permanently unusable once it completes or fails once.

A target's epoch only ever increases, and publications are compared against whatever epoch is currently in effect for that target:

- A publication for the **same** epoch as the target's current one is normal progression (`started` -> `running` -> ... -> a terminal event), subject to the terminal/already-terminal rule described above.
- A publication for a **lower** epoch than the target's current one is rejected as stale (`publish_result::stale_epoch`) and has no effect - this guards against a delayed or reordered event from an earlier generation of the target clobbering newer state.
- A publication for a **higher** epoch than the target's current one resets the target: it becomes the new current epoch, the event sequence number resets, and any previously latched terminal event (`completed`/ `failed`) under the old epoch no longer fences new dispatches. This is the only way to make a target eligible for fenced dispatch again after it has gone terminal.

The same epoch value is consulted end-to-end: `check_admission(target, epoch)` compares the *epoch you pass in* against the target's current epoch, not just whether the target is terminal in some absolute sense. This is why both `dispatch_work()`'s local check and `invoke_fenced_action()`'s remote re-check take an explicit `epoch` argument rather than operating on "the" state of the target.

When to keep the epoch the same

Use the same epoch value across all fenced dispatches that belong to the same logical run/incarnation of the target - e.g. repeated or retried calls against a target that is still expected to be `started`/`running` under the epoch it was initialized with. Fencing is only meaningful when related dispatches share an epoch: if a target has already latched `failed` for epoch `N`, every further dispatch under epoch `N` is correctly rejected, which is the intended behavior - it stops work from continuing to target something that has already terminated in this generation.

When to bump the epoch

Bump to a new, higher epoch when you are intentionally starting a new generation of the target and want prior terminal state to stop fencing new work - typically when a target is restarted/reincarnated after failing or completing, and you want dispatch to resume against the restarted instance rather than remain permanently fenced by the old generation's terminal event. Concretely: publish the target's own `event::started` under the new epoch as part of bringing the restarted instance up, then use that same, higher epoch value for subsequent dispatches and admission checks against it.

Do not reuse an old epoch value after bumping, and do not bump the epoch merely to "clear" a fence for the *same* running instance - if the target has not actually restarted, admission is correctly rejecting dispatch because the target is genuinely terminal, and forcing an epoch bump only defeats that protection rather than fixing the underlying condition.

How fenced work is invoked

`hpx::supervision::dispatch_work<action_type>(target, epoch, action, args...)` is the entry point callers use. It performs two distinct steps, on two different localities:

1. **Local, non-authoritative early-out (caller's locality).** `dispatch_work()` first calls `check_admission(target, epoch)` against the caller's own locally cached shadow state. If this reports `dispatch_outcome::rejected_fenced`, `dispatch_work()` returns an already-exceptional future carrying `hpx::error::target_fenced` immediately - no parcel is ever sent, and the wrapped action is never invoked. This is purely an optimization to avoid a wasted round trip; it is not authoritative, since the shadow state it consults can be stale (see the sections above).
2. **Colocated dispatch and authoritative re-check (target's locality).** If the local check does not reject, `dispatch_work()` wraps the caller's action in a generated `fenced_action` and dispatches it via `hpx::async` to `hpx::colocated(target)` - i.e. it runs on the same locality that hosts `target` and owns its authoritative supervision state. The action body that actually runs there is `invoke_fenced_action()`, which performs a *second* `check_admission(target, epoch)` call. Because this call executes on the same locality/thread that owns the target's supervision state, with no suspension point before the wrapped action runs immediately afterward (via `hpx::sync`, invoked directly/locally with no further parcel hop), this second check is authoritative. If it also reports `rejected_fenced` - meaning the target latched a terminal event for this epoch sometime between the caller's stale check and this point - `invoke_fenced_action()` throws `hpx::error::target_fenced` instead of invoking the action.

Note that "dispatched to the target's locality" does not imply a network round-trip: `hpx::colocated(target)` resolves to whichever locality actually hosts `target`, including the invoking locality itself when `target` happens to be local. In that case no parcel is sent, but `invoke_fenced_action()` - and, if admitted, the wrapped action inside it - still each run on a newly spawned HPX thread rather than inline on the calling thread, exactly as an unfenced `hpx::async<Action>(target, args...)` call behaves when `target` is local: it schedules a new thread rather than invoking the action synchronously in place. The one exception, on both sides, is an action explicitly declared as a `direct_action`, which HPX is permitted to invoke inline when the target is local; `fenced_action` itself is an ordinary action, and the wrapped user `Action` follows whatever declaration the caller gave it. Either way, fenced dispatch adds only the two-tier admission check described above - it does not change whether or where the action is scheduled to run relative to unfenced dispatch.

Both the local early-out and the remote re-check surface failure as the same exception type/message shape (`hpx::error::target_fenced`), so callers handle a single error regardless of which side detected the fence:

```
caller locality                         target locality
----------------                        ---------------
dispatch_work(action, target, epoch, args...)
  check_admission(target, epoch)   <-- reads local shadow (non-authoritative)
    rejected -> exceptional future, done (no dispatch sent)
    admitted -> hpx::async(fenced_action, hpx::colocated(target), ...)
                                    |
                                    v
                                        invoke_fenced_action(action, target, epoch)
                                          check_admission(target, epoch)  <-- authoritative,
                                                                              same locality/thread
                                            rejected -> throw target_fenced
                                            admitted -> hpx::sync(action, target, ...)
```

Only `target` (not `epoch`) is forwarded into the wrapped action's own argument list; the epoch is fencing metadata consumed entirely by the dispatch machinery. A convenience overload of `dispatch_work()` accepts an action instance directly rather than only an action type, but follows the identical two-check flow described above.

Relationship to unfenced dispatch

From the caller's point of view, converting an unfenced dispatch into a fenced one requires exactly two changes to the call site:

```
// Unfenced:
hpx::future<result_type> f = hpx::async<Action>(target, args...);

// Fenced:
hpx::future<result_type> f =
    hpx::supervision::dispatch_work<Action>(target, epoch, args...);
```

1. Replace the call to `hpx::async<Action>` with `hpx::supervision::dispatch_work<Action>`.
2. Insert the fencing `epoch` as the second argument, ahead of the action's own arguments (`args...` is otherwise unchanged and is forwarded to `Action` verbatim).

Both a template-argument form and an instance-argument form are available for `dispatch_work()`, mirroring the two ways `hpx::async` can be called:

```
// Action specified as an explicit template argument (as above):
hpx::future<result_type> f =
    hpx::supervision::dispatch_work<Action>(target, epoch, args...);

// Convenience overload: action passed as an instance, deduced instead of
// spelled out explicitly. Note the action instance comes *first*, ahead
// of `target` and `epoch`.
hpx::future<result_type> f =
    hpx::supervision::dispatch_work(Action(), target, epoch, args...);
```

The instance-argument overload is a thin wrapper that simply forwards to the template-argument overload above; the two are otherwise identical in behavior (same two-check admission flow, same exception on fencing, same argument forwarding to `Action`). It exists purely so that call sites already holding an action instance - rather than naming the action type directly – don't need an extra step to obtain one, the same convenience `hpx::async` itself offers by accepting either an action type or an action instance.

Everything else - `target`, the returned `hpx::future<result_type>`, and how the caller consumes it (`.get()`, continuations, etc.) - stays the same. The only additional caller-side responsibility is handling `hpx::error::target_fenced` on that future (see below), since a fenced dispatch never invokes `Action` at all, whereas an unfenced `hpx::async<Action>` call has no equivalent rejection path.

Observing whether an action was fenced

From the caller's point of view, `dispatch_work()` always returns an `hpx::future`. Whether the fence was detected locally (early-out, no dispatch sent) or remotely (authoritative re-check on the target's own locality), the future becomes exceptional in exactly the same way, so callers do not need to special-case which side caught it:

```
hpx::future<result_type> f =
    hpx::supervision::dispatch_work(action, target, epoch, args...);

try
{
    result_type result = f.get();
    // Admitted on both checks; action ran and produced `result`.
}
catch (hpx::exception const& e)
{
    if (e.get_error() == hpx::error::target_fenced)
    {
        // Fenced - either the caller's local admission check rejected
        // it before any dispatch was sent, or the target's own
        // authoritative re-check rejected it just before invocation.
```

(continues on next page)

(continued from previous page)

```
        // The wrapped action was NOT invoked in either case.
    }
    else
    {
↪   throw;    // some other failure (e.g. transport, target action
                  // itself threw)
    }
}
```

Two points worth calling out explicitly:

- `e.get_error() == hpx::error::target_fenced` is the single, reliable signal that an action was fenced. Matching on the exception's `what()` text is unnecessary and discouraged, since the message wording is not part of the stable contract; only the error code is.
- A fenced result and a "the action itself threw" result are both surfaced through the same future/exception mechanism, so callers must check `get_error()` to distinguish "not invoked because fenced" from "invoked, but failed." There is no separate return value, flag, or callback that reports fencing independently of the future's exceptional state.

  Because `f.get()` throws synchronously the moment either check rejects, a caller waiting with a timeout (see below) sees ready-with-exception rather than ready-with-result if either side fenced the dispatch before the timeout elapsed.

Worked examples

Fencing a silent peer

```
#include <hpx/supervision_dispatch.hpp>

hpx::supervision::registry local_registry(hpx::find_here());

hpx::supervision::joined_peer const peer = local_registry.join(
    hpx::launch::sync, hpx::find_here());

// ... time passes; the peer's heartbeat stops ...

// Client-
↪side filtering: cheap, but only as fresh as the last observed
// heartbeat/lifecycle event mirrored onto `shadow`.
for (auto
↪const& peer : local_registry.snapshot_peers(hpx::launch::sync))
{
    if (peer.state == hpx::supervision::lifecycle_state::failed)
    {
↪    // Skip obviously-dead peers before even attempting dispatch.
        continue;
    }
    //
```

(continues on next page)

(continued from previous page)

```
↪ Attempt fenced dispatch anyway; the target-side re-check inside
    // invoke_
↪fenced_action() is authoritative and will reject the action
    // with␣
↪hpx::error::target_fenced if the peer has latched a terminal
    // event we haven't observed locally yet.
}
```

Waiting on a dispatch outcome with a timeout

```
#include <hpx/async_combinators/wait_all_for.hpp>
#include <hpx/supervision_dispatch.hpp>

hpx::future<result_type> f =
   ␣
↪hpx::supervision::dispatch_work(target, epoch, action, args...);

if (hpx::wait_all_for(std::chrono::seconds(5), f) ==
    hpx::future_status::ready)
{
    try
    {
        result_type result = f.get();
    }
    catch (hpx::exception const& e)
    {
        if (e.get_error() == hpx::error::target_fenced)
        {
    ␣
↪           // Authoritative rejection: the target latched a terminal
            // event for this epoch before the action could run.
        }
    }
}
else
{
    //
↪ Dispatch did not complete within the timeout; the target may be
    // slow, unreachable, or the shadow state used for client-side
    // filtering may be stale.
}
```

Querying state and publishing events through a handle

The handle returned by `init()` also doubles as the argument to a small family of convenience overloads of `query_state()`/`publish_event()` that resolve locality and target automatically, instead of requiring the caller to extract `handle.get_id()` and `hpx::find_here()` manually:

```
#include <hpx/supervision_dispatch.hpp>

hpx::supervision::registry const handle =
    hpx::supervision::init(hpx::launch::sync);

// Recover the epoch init() started this handle's locality at, for use
// with dispatch_work()/publish_event() later on.
std::uint64_t const epoch =
    hpx::supervision::query_state(hpx::launch::sync, handle).epoch;

// ... later, e.g. once this locality's own supervised work completes ...
hpx::supervision::publish_event(
    hpx::launch::sync,
    handle, hpx::supervision::event::completed, epoch);
```

Querying a peer's state after joining it follows the same pattern, using a `discovered_peer` in place of an explicit locality/target pair:

```
for (auto const& peer :
    hpx::supervision::discover_and_join(handle))
{
    hpx::supervision::lifecycle_state const peer_state =
        hpx::supervision::query_state(hpx::launch::sync, handle, peer);
    // `handle` is only needed here for overload resolution; `peer`
    // alone carries the locality/target this call actually queries.
}
```

There is no peer-publishing overload: a locality only ever publishes lifecycle events it self, never on behalf of a peer's.

A worked example combining this with a supervised worker loop - covering when to publish `started`/`running`/a terminal event for a worker's own locality, and how a supervisor queries peer worker state through the handle above - is available in `components/supervision_dispatch/examples/plain_worker.cpp`.

See *Supervision module* for the full semantics of `check_admission`, `dispatch_outcome`, and the individual lifecycle events referenced above, and *Supervision dispatch module* for the API reference of this component's own types and functions (`registry`, `discovery`, and `dispatch_work`).

2.4.23 Troubleshooting

Common issues

This section contains commonly encountered problems when compiling or using HPX.

See also the closed issues on GitHub[206] to find out how other people resolved a similar problem. If nothing of that works, you can also open a new issue on GitHub[207] or contact us using one the options found in Support for deploying and using HPX[208].

`hpx::iostreams_component" target not found`

You may see a CMake[209] error message that looks a bit like this:

```
error: `hpx::iostreams_component`` target not found
```

Simply ensure that *HPX* is installed with `HPX_WITH_DISTRIBUTED_RUNTIME=ON` to prevent encountering such error(s). This is required if you want to use `hpx::cout`.

`Undefined reference to hpx::cout`

You may see a linker error message that looks a bit like this:

```
hello_world.cpp:(.text+0x5aa): undefined reference to `hpx::cout'
```

This usually happens if you are trying to use *HPX* iostreams functionality such as `hpx::cout` but are not linking against it. The iostreams functionality is not part of the core *HPX* library, and must be linked to explicitly. Typically this can be solved by adding `COMPONENT_DEPENDENCIES iostreams` to a call to `add_hpx_library/add_hpx_executable/hpx_setup_target` if using CMake[210]. See *Creating HPX projects* for more details.

`Build fails with ASIO error`

You may see an error message that looks a bit like this:

```
Cannot open include file asio/io_context.hpp
```

This can be resolved by using `-DHPX_WITH_FETCH_ASIO=ON` to the cmake command line.

See also the corresponding closed Issue #5404[211] for more information.

[206] https://github.com/TheHPXProject/hpx/issues?q=is%3Aissue+is%3Aclosed
[207] https://github.com/TheHPXProject/hpx/issues
[208] https://github.com/TheHPXProject/hpx/blob/master/.github/SUPPORT.md
[209] https://www.cmake.org
[210] https://www.cmake.org
[211] https://github.com/TheHPXProject/hpx/issues/5404

Build fails with TCMalloc error

You may see an error message that looks a bit like this:

```
Could NOT␣
↪find TCMalloc (missing: Tcmalloc_LIBRARY Tcmalloc_INCLUDE_DIR)
ERROR:␣
↪HPX_WITH_MALLOC was set to tcmalloc, but tcmalloc could not be
found.  Valid␣
↪options for HPX_WITH_MALLOC are: system, tcmalloc, jemalloc,
mimalloc, tbbmalloc, and custom
```

This can be resolved either by defining `HPX_WITH_MALLOC=system` or by installing TC-Malloc. This error occurs when users don't specify an option for `HPX_WITH_MALLOC`; in that case, *HPX* will be looking `tcmalloc`, which is the default value.

Useful suggestions

Reducing compilation time

If you want to significantly reduce compilation time, you can just use the local part of *HPX* for parallelism by disabling the distributed functionality. Moreover, you can avoid compiling examples. These can be done with the following flags:

```
-DHPX_WITH_NETWORKING=OFF
-DHPX_WITH_DISTRIBUTED_RUNTIME=OFF
-DHPX_WITH_EXAMPLES=OFF
-DHPX_WITH_TESTS=OFF
```

Linking *HPX* to your application

If you want to avoid installing and linking *HPX*, you can just build *HPX* and then use the following flag on your *HPX* application CMake configuration:

```
-DHPX_DIR=<build_dir>/lib/cmake/HPX
```

Note

For this to work you need not to specify `-DCMAKE_INSTALL_PREFIX` when building *HPX*.

HPX-application build type conformance

Your application's build type should align with the HPX build type. For example, if you specified `-DCMAKE_BUILD_TYPE=Debug` during the *HPX* compilation, then your application needs to be compiled with the same flag. We recommend keeping a separate build folder for different build types and just point accordingly to the type you want by using `-DHPX_DIR=<build_dir>/lib/cmake/HPX`.

2.5 Terminology

This section gives definitions for some of the terms used throughout the *HPX* documentation and source code.

Locality
A locality in *HPX* describes a synchronous domain of execution, or the domain of bounded upper response time. This normally is just a single node in a cluster or a NUMA domain in a SMP machine.

Active Global Address Space
AGAS
HPX incorporates a global address space. Any executing thread can access any object within the domain of the parallel application with the caveat that it must have appropriate access privileges. The model does not assume that global addresses are cache coherent; all loads and stores will deal directly with the site of the target object. All global addresses within a Synchronous Domain are assumed to be cache coherent for those processor cores that incorporate transparent caches. The Active Global Address Space used by *HPX* differs from research PGAS[212] models. Partitioned Global Address Space is passive in their means of address translation. Copy semantics, distributed compound operations, and affinity relationships are some of the global functionality supported by AGAS.

Process
The concept of the "process" in *HPX* is extended beyond that of either sequential execution or communicating sequential processes. While the notion of process suggests action (as do "function" or "subroutine") it has a further responsibility of context, that is, the logical container of program state. It is this aspect of operation that process is employed in *HPX*. Furthermore, referring to "parallel processes" in *HPX* designates the presence of parallelism within the context of a given process, as well as the coarse grained parallelism achieved through concurrency of multiple processes of an executing user job. *HPX* processes provide a hierarchical name space within the framework of the active global address space and support multiple means of internal state access from external sources.

Parcel
The Parcel is a component in *HPX* that communicates data, invokes an action at a distance, and distributes flow-control through the migration of continuations. Parcels bridge the gap of asynchrony between synchronous domains while maintaining symmetry of semantics between local and global execution. Parcels enable message-driven computation and may be seen as a form of "active messages". Other important forms of message-driven computation predating active messages include dataflow tokens[213], the J-machine's[214] support for remote method instantiation, and at the coarse grained

[212] https://www.pgas.org/
[213] http://en.wikipedia.org/wiki/Dataflow_architecture
[214] http://en.wikipedia.org/wiki/J%E2%80%93Machine

variations of Unix remote procedure calls, among others. This enables work to be moved to the data as well as performing the more common action of bringing data to the work. A parcel can cause actions to occur remotely and asynchronously, among which are the creation of threads at different system nodes or synchronous domains.

Local Control Object
Lightweight Control Object
LCO

A local control object (sometimes called a lightweight control object) is a general term for the synchronization mechanisms used in *HPX*. Any object implementing a certain concept can be seen as an LCO. This concepts encapsulates the ability to be triggered by one or more events which when taking the object into a predefined state will cause a thread to be executed. This could either create a new thread or resume an existing thread.

The LCO is a family of synchronization functions potentially representing many classes of synchronization constructs, each with many possible variations and multiple instances. The LCO is sufficiently general that it can subsume the functionality of conventional synchronization primitives such as spinlocks, mutexes, semaphores, and global barriers. However due to the rich concept an LCO can represent powerful synchronization and control functionality not widely employed, such as dataflow and futures (among others), which open up enormous opportunities for rich diversity of distributed control and operation.

See lcos for more details on how to use LCOs in *HPX*.

Action

An action is a function that can be invoked remotely. In *HPX* a plain function can be made into an action using a macro. See applying_actions for details on how to use actions in *HPX*.

Component

A component is a C++ object which can be accessed remotely. A component can also contain member functions which can be invoked remotely. These are referred to as component actions. See *Writing components* for details on how to use components in *HPX*.

2.6 Why *HPX*?

Current advances in high performance computing (HPC) continue to suffer from the issues plaguing parallel computation. These issues include, but are not limited to, ease of programming, inability to handle dynamically changing workloads, scalability, and efficient utilization of system resources. Emerging technological trends such as multi-core processors further highlight limitations of existing parallel computation models. To mitigate the aforementioned problems, it is necessary to rethink the approach to parallelization models. ParalleX contains mechanisms such as multi-threading, *parcels*, *global name space* support, percolation and *local control object*s (*LCO*). By design, ParalleX overcomes limitations of current models of parallelism by alleviating contention, latency, overhead and starvation. With ParalleX, it is further possible to increase performance by at least an order of magnitude on challenging parallel algorithms, e.g., dynamic directed graph algorithms and adaptive mesh refinement methods for astrophysics. An additional benefit of ParalleX is fine-grained control of power usage, enabling reductions in power consumption.

2.6.1 ParalleX—a new execution model for future architectures

ParalleX is a new parallel execution model that offers an alternative to the conventional computation models, such as message passing. ParalleX distinguishes itself by:

- Split-phase transaction model
- Message-driven
- Distributed shared memory (not cache coherent)
- Multi-threaded
- Futures synchronization
- *Local Control Object*s (*LCO*s)
- Synchronization for anonymous producer-consumer scenarios
- Percolation (pre-staging of task data)

The ParalleX model is intrinsically latency hiding, delivering an abundance of variable-grained parallelism within a hierarchical namespace environment. The goal of this innovative strategy is to enable future systems delivering very high efficiency, increased scalability and ease of programming. ParalleX can contribute to significant improvements in the design of all levels of computing systems and their usage from application algorithms and their programming languages to system architecture and hardware design together with their supporting compilers and operating system software.

2.6.2 What is *HPX*?

High Performance ParalleX (*HPX*) is the first runtime system implementation of the ParalleX execution model. The *HPX* runtime software package is a modular, feature-complete, and performance-oriented representation of the ParalleX execution model targeted at conventional parallel computing architectures, such as SMP nodes and commodity clusters. It is academically developed and freely available under an open source license. We provide *HPX* to the community for experimentation and application to achieve high efficiency and scalability for dynamic adaptive and irregular computational problems. *HPX* is a C++ library that supports a set of critical mechanisms for dynamic adaptive resource management and lightweight task scheduling within the context of a global address space. It is solidly based on many years of experience in writing highly parallel applications for HPC systems.

The two-decade success of the communicating sequential processes (CSP) execution model and its message passing interface (MPI) programming model have been seriously eroded by challenges of power, processor core complexity, multi-core sockets, and heterogeneous structures of GPUs. Both efficiency and scalability for some current (strong scaled) applications and future Exascale applications demand new techniques to expose new sources of algorithm parallelism and exploit unused resources through adaptive use of runtime information.

The ParalleX execution model replaces CSP to provide a new computing paradigm embodying the governing principles for organizing and conducting highly efficient scalable computations greatly exceeding the capabilities of today's problems. *HPX* is the first practical, reliable, and performance-oriented runtime system incorporating the principal concepts of the ParalleX model publicly provided in open source release form.

HPX is designed by the STE||AR[215] Group (**S**ystems **T**echnology, **E**mergent Para**ll**elism, and **A**lgorithm **R**esearch) at Louisiana State University (LSU)[216]'s Center for Computation and Technology (CCT)[217] to enable developers to exploit the full processing power of many-core systems with an unprecedented degree of parallelism. STE||AR[218] is a research group focusing on system software solutions and scientific application development for hybrid and many-core hardware architectures.

For more information about the STE||AR[219] Group, see *People*.

2.6.3 What makes our systems slow?

Estimates say that we currently run our computers at well below 100% efficiency. The theoretical peak performance (usually measured in FLOPS[220]—floating point operations per second) is much higher than any practical peak performance reached by any application. This is particularly true for highly parallel hardware. The more hardware parallelism we provide to an application, the better the application must scale in order to efficiently use all the resources of the machine. Roughly speaking, we distinguish two forms of scalability: strong scaling (see Amdahl's Law[221]) and weak scaling (see Gustafson's Law[222]). Strong scaling is defined as how the solution time varies with the number of processors for a fixed **total** problem size. It gives an estimate of how much faster we can solve a particular problem by throwing more resources at it. Weak scaling is defined as how the solution time varies with the number of processors for a fixed problem size **per processor**. In other words, it defines how much more data can we process by using more hardware resources.

In order to utilize as much hardware parallelism as possible an application must exhibit excellent strong and weak scaling characteristics, which requires a high percentage of work executed in parallel, i.e., using multiple threads of execution. Optimally, if you execute an application on a hardware resource with N processors it either runs N times faster or it can handle N times more data. Both cases imply 100% of the work is executed on all available processors in parallel. However, this is just a theoretical limit. Unfortunately, there are more things that limit scalability, mostly inherent to the hardware architectures and the programming models we use. We break these limitations into four fundamental factors that make our systems *SLOW*:

- **S**tarvation occurs when there is insufficient concurrent work available to maintain high utilization of all resources.
- **L**atencies are imposed by the time-distance delay intrinsic to accessing remote resources and services.
- **O**verhead is work required for the management of parallel actions and resources on the critical execution path, which is not necessary in a sequential variant.
- **W**aiting for contention resolution is the delay due to the lack of availability of oversubscribed shared resources.

Each of those four factors manifests itself in multiple and different ways; each of the hardware architectures and programming models expose specific forms. However, the

[215] https://stellar-group.org
[216] https://www.lsu.edu
[217] https://www.cct.lsu.edu
[218] https://stellar-group.org
[219] https://stellar-group.org
[220] http://en.wikipedia.org/wiki/FLOPS
[221] http://en.wikipedia.org/wiki/Amdahl%27s_law
[222] http://en.wikipedia.org/wiki/Gustafson%27s_law

interesting part is that all of them are limiting the scalability of applications no matter what part of the hardware jungle we look at. Hand-helds, PCs, supercomputers, or the cloud, all suffer from the reign of the 4 horsemen: **S**tarvation, **L**atency, **O**verhead, and **C**ontention. This realization is very important as it allows us to derive the criteria for solutions to the scalability problem from first principles, and it allows us to focus our analysis on very concrete patterns and measurable metrics. Moreover, any derived results will be applicable to a wide variety of targets.

2.6.4 Technology demands new response

Today's computer systems are designed based on the initial ideas of John von Neumann[223], as published back in 1945, and later extended by the Harvard architecture[224]. These ideas form the foundation, the execution model, of computer systems we use currently. However, a new response is required in the light of the demands created by today's technology.

So, what are the overarching objectives for designing systems allowing for applications to scale as they should? In our opinion, the main objectives are:

- Performance: as previously mentioned, scalability and efficiency are the main criteria people are interested in.
- Fault tolerance: the low expected mean time between failures (MTBF[225]) of future systems requires embracing faults, not trying to avoid them.
- Power: minimizing energy consumption is a must as it is one of the major cost factors today, and will continue to rise in the future.
- Generality: any system should be usable for a broad set of use cases.
- Programmability: for programmer this is a very important objective, ensuring long term platform stability and portability.

What needs to be done to meet those objectives, to make applications scale better on tomorrow's architectures? Well, the answer is almost obvious: we need to devise a new execution model—a set of governing principles for the holistic design of future systems—targeted at minimizing the effect of the outlined **SLOW** factors. Everything we create for future systems, every design decision we make, every criteria we apply, have to be validated against this single, uniform metric. This includes changes in the hardware architecture we prevalently use today, and it certainly involves new ways of writing software, starting from the operating system, runtime system, compilers, and at the application level. However, the key point is that all those layers have to be co-designed; they are interdependent and cannot be seen as separate facets. The systems we have today have been evolving for over 50 years now. All layers function in a certain way, relying on the other layers to do so. But we do not have the time to wait another 50 years for a new coherent system to evolve. The new paradigms are needed now—therefore, co-design is the key.

[223] http://qss.stanford.edu/~godfrey/vonNeumann/vnedvac.pdf
[224] http://en.wikipedia.org/wiki/Harvard_architecture
[225] http://en.wikipedia.org/wiki/Mean_time_between_failures

2.6.5 Governing principles applied while developing *HPX*

As it turn out, we do not have to start from scratch. Not everything has to be invented and designed anew. Many of the ideas needed to combat the 4 horsemen already exist, many for more than 30 years. All it takes is to gather them into a coherent approach. We'll highlight some of the derived principles we think to be crucial for defeating **SLOW**. Some of those are focused on high-performance computing, others are more general.

Focus on latency hiding instead of latency avoidance

It is impossible to design a system exposing zero latencies. In an effort to come as close as possible to this goal many optimizations are mainly targeted towards minimizing latencies. Examples for this can be seen everywhere, such as low latency network technologies like InfiniBand[226], caching memory hierarchies in all modern processors, the constant optimization of existing MPI[227] implementations to reduce related latencies, or the data transfer latencies intrinsic to the way we use GPGPUs[228] today. It is important to note that existing latencies are often tightly related to some resource having to wait for the operation to be completed. At the same time it would be perfectly fine to do some other, unrelated work in the meantime, allowing the system to hide the latencies by filling the idle-time with useful work. Modern systems already employ similar techniques (pipelined instruction execution in the processor cores, asynchronous input/output operations, and many more). What we propose is to go beyond anything we know today and to make latency hiding an intrinsic concept of the operation of the whole system stack.

Embrace fine-grained parallelism instead of heavyweight threads

If we plan to hide latencies even for very short operations, such as fetching the contents of a memory cell from main memory (if it is not already cached), we need to have very lightweight threads with extremely short context switching times, optimally executable within one cycle. Granted, for mainstream architectures, this is not possible today (even if we already have special machines supporting this mode of operation, such as the Cray XMT[229]). For conventional systems, however, the smaller the overhead of a context switch and the finer the granularity of the threading system, the better will be the overall system utilization and its efficiency. For today's architectures we already see a flurry of libraries providing exactly this type of functionality: non-pre-emptive, task-queue based parallelization solutions, such as Intel Threading Building Blocks (TBB)[230], Microsoft Parallel Patterns Library (PPL)[231], Cilk++[232], and many others. The possibility to suspend a current task if some preconditions for its execution are not met (such as waiting for I/O or the result of a different task), seamlessly switching to any other task which can continue, and to reschedule the initial task after the required result has been calculated, which makes the implementation of latency hiding almost trivial.

226 http://en.wikipedia.org/wiki/InfiniBand
227 https://en.wikipedia.org/wiki/Message_Passing_Interface
228 http://en.wikipedia.org/wiki/GPGPU
229 http://en.wikipedia.org/wiki/Cray_XMT
230 https://www.threadingbuildingblocks.org/
231 https://msdn.microsoft.com/en-us/library/dd492418.aspx
232 https://software.intel.com/en-us/articles/intel-cilk-plus/

Rediscover constraint-based synchronization to replace global barriers

The code we write today is riddled with implicit (and explicit) global barriers. By "global barriers," we mean the synchronization of the control flow between several (very often all) threads (when using OpenMP[233]) or processes (MPI[234]). For instance, an implicit global barrier is inserted after each loop parallelized using OpenMP[235] as the system synchronizes the threads used to execute the different iterations in parallel. In MPI[236] each of the communication steps imposes an explicit barrier onto the execution flow as (often all) nodes have to be synchronized. Each of those barriers is like the eye of a needle the overall execution is forced to be squeezed through. Even minimal fluctuations in the execution times of the parallel threads (jobs) causes them to wait. Additionally, it is often only one of the executing threads that performs the actual reduce operation, which further impedes parallelism. A closer analysis of a couple of key algorithms used in science applications reveals that these global barriers are not always necessary. In many cases it is sufficient to synchronize a small subset of the threads. Any operation should proceed whenever the preconditions for its execution are met, and only those. Usually there is no need to wait for iterations of a loop to finish before you can continue calculating other things; all you need is to complete the iterations that produce the required results for the next operation. Good bye global barriers, hello constraint based synchronization! People have been trying to build this type of computing (and even computers) since the 1970s. The theory behind what they did is based on ideas around static and dynamic dataflow. There are certain attempts today to get back to those ideas and to incorporate them with modern architectures. For instance, a lot of work is being done in the area of constructing dataflow-oriented execution trees. Our results show that employing dataflow techniques in combination with the other ideas, as outlined herein, considerably improves scalability for many problems.

Adaptive locality control instead of static data distribution

While this principle seems to be a given for single desktop or laptop computers (the operating system is your friend), it is everything but ubiquitous on modern supercomputers, which are usually built from a large number of separate nodes (i.e., Beowulf clusters), tightly interconnected by a high-bandwidth, low-latency network. Today's prevalent programming model for those is MPI, which does not directly help with proper data distribution, leaving it to the programmer to decompose the data to all of the nodes the application is running on. There are a couple of specialized languages and programming environments based on PGAS[237] (Partitioned Global Address Space) designed to overcome this limitation, such as Chapel[238], X10[239], UPC[240], or Fortress[241]. However, all systems based on PGAS rely on static data distribution. This works fine as long as this static data distribution does not result in heterogeneous workload distributions or other resource utilization imbalances. In a distributed system these imbalances can be mitigated by migrating part of the application data to different localities (nodes). The only framework supporting (limited) migration today is Charm++[242]. The first attempts towards solving related problem go back decades as well, a good example is the Linda coordination language[243]. Nevertheless, none of the other mentioned systems support data migration today, which forces the users to either rely on static data distribution and live with the related performance hits or to implement everything themselves, which is very tedious and difficult. We believe that the only viable way to flexibly support dynamic and adaptive *locality* control is to provide a global, uniform address space to the applications, even on distributed systems.

233 https://openmp.org/wp/
234 https://en.wikipedia.org/wiki/Message_Passing_Interface
235 https://openmp.org/wp/
236 https://en.wikipedia.org/wiki/Message_Passing_Interface
237 https://www.pgas.org/
238 https://chapel.cray.com/
239 https://x10-lang.org/
240 https://upc.lbl.gov/
241 https://labs.oracle.com/projects/plrg/Publications/index.html
242 https://charm.cs.uiuc.edu/
243 http://en.wikipedia.org/wiki/Linda_(coordination_language)

Prefer moving work to the data over moving data to the work

For the best performance it seems obvious to minimize the amount of bytes transferred from one part of the system to another. This is true on all levels. At the lowest level we try to take advantage of processor memory caches, thus, minimizing memory latencies. Similarly, we try to amortize the data transfer time to and from GPGPUs[244] as much as possible. At high levels we try to minimize data transfer between different nodes of a cluster or between different virtual machines on the cloud. Our experience (well, it's almost common wisdom) shows that the amount of bytes necessary to encode a certain operation is very often much smaller than the amount of bytes encoding the data the operation is performed upon. Nevertheless, we still often transfer the data to a particular place where we execute the operation just to bring the data back to where it came from afterwards. As an example let's look at the way we usually write our applications for clusters using MPI. This programming model is all about data transfer between nodes. MPI is the prevalent programming model for clusters, and it is fairly straightforward to understand and to use. Therefore, we often write applications in a way that accommodates this model, centered around data transfer. These applications usually work well for smaller problem sizes and for regular data structures. The larger the amount of data we have to churn and the more irregular the problem domain becomes, the worse the overall machine utilization and the (strong) scaling characteristics become. While it is not impossible to implement more dynamic, data driven, and asynchronous applications using MPI, it is somewhat difficult to do so. At the same time, if we look at applications that prefer to execute the code close to the *locality* where the data was placed, i.e., utilizing active messages (for instance based on Charm++[245]), we see better asynchrony, simpler application codes, and improved scaling.

Favor message driven computation over message passing

Today's prevalently used programming model on parallel (multi-node) systems is MPI. It is based on message passing, as the name implies, which means that the receiver has to be aware of a message about to come in. Both codes, the sender and the receiver, have to synchronize in order to perform the communication step. Even the newer, asynchronous interfaces require explicitly coding the algorithms around the required communication scheme. As a result, everything but the most trivial MPI applications spends a considerable amount of time waiting for incoming messages, thus, causing starvation and latencies to impede full resource utilization. The more complex and more dynamic the data structures and algorithms become, the larger the adverse effects. The community discovered message-driven and data-driven methods of implementing algorithms a long time ago, and systems such as Charm++[246] have already integrated active messages demonstrating the validity of the concept. Message-driven computation allows for sending messages without requiring the receiver to actively wait for them. Any incoming message is handled asynchronously and triggers the encoded action by passing along arguments and—possibly—continuations. *HPX* combines this scheme with work-queue based scheduling as described above, which allows the system to almost completely overlap any communication with useful work, thereby minimizing latencies.

[244] http://en.wikipedia.org/wiki/GPGPU
[245] https://charm.cs.uiuc.edu/
[246] https://charm.cs.uiuc.edu/

2.7 Additional material

- 2-day workshop held at CSCS in 2016
- – Recorded lectures[247]
- – Slides[248]
- Tutorials repository[249]
- *C++ Lectures <https://www.youtube.com/playlist?list=PL7vEgTL3FalY2eBxud1wsfz8OKvE9sd_z>*
- *Parallel C++ for Scientific Applications Lectures <https://www.youtube.com/watch?v=RdmXiIilArM&list=PL7vEgTL3Falab59uJoOb7AtFQKVuL0MV->*
- HPX blog posts[250]
- Basic *HPX* recipes
- – Exporting a free function from a shared library which lives in a namespace, to use as Action[251]
- – Turning a struct or class into a component and use it's methods[252]
- – Creating and referencing components in hpx[253]

2.8 Overview

HPX is organized into different sub-libraries and those in turn into modules. The libraries and modules are independent, with clear dependencies and no cycles. As an end-user, the use of these libraries is completely transparent. If you use e.g. `add_hpx_executable` to create a target in your project you will automatically get all modules as dependencies. See below for a list of the available libraries and modules. Currently these are nothing more than an internal grouping and do not affect usage. They cannot be consumed individually at the moment.

> **Note**
>
> There is a dependency report that displays useful information about the structure of the code. It is available for each commit at *HPX* Dependency report.

[247] https://www.youtube.com/playlist?list=PL1tk5lGm7zvSXfS-sqOOmIJ0lFNjKze18
[248] https://github.com/STEllAR-GROUP/tutorials/tree/master/cscs2016
[249] https://github.com/STEllAR-GROUP/tutorials
[250] http://hpx.dev/blog/
[251] https://gitlab.com/-/snippets/1821389
[252] https://gitlab.com/-/snippets/1822983
[253] https://gitlab.com/-/snippets/1828131

2.8.1 Core modules

affinity

The affinity module contains helper functionality for mapping worker threads to hardware resources.

See the API reference of the module for more details.

algorithms

The algorithms module exposes the full set of algorithms defined by the C++ standard. There is also partial support for C++ ranges.

See the *API reference* of the module for more details.

allocator_support

This module provides utilities for allocators. It contains `hpx::util::internal_allocator` which directly forwards allocation calls to `jemalloc`. This utility is is mainly useful on Windows.

See the API reference of the module for more details.

asio

The asio module is a thin wrapper around the Boost.Asio[254] library, providing a few additional helper functions.

See the *API reference* of the module for more details.

assertion

The assertion library implements the macros `HPX_ASSERT` and `HPX_ASSERT_MSG`. Those two macros can be used to implement assertions which are turned of during a release build.

By default, the location and function where the assert has been called from are displayed when the assertion fires. This behavior can be modified by using *`hpx::assertion::set_assertion_handler`*. When HPX initializes, it uses this function to specify a more elaborate assertion handler. If your application needs to customize this, it needs to do so before calling *hpx::init*, *hpx_main* or using the C-main wrappers.

See the *API reference* of the module for more details.

254 https://www.boost.org/doc/libs/release/doc/html/boost_asio.html

async_base

The async_base module defines the basic functionality for spawning tasks on thread pools. This module does not implement any functionality on its own, but is extended by *async_local* and *async_distributed* with implementations for the local and distributed cases.

See the *API reference* of this module for more details.

async_combinators

This module contains combinators for futures. The `when_*` functions allow you to turn multiple futures into a single future which is ready when all, any, some, or each of the given futures are ready. The `wait_*` combinators are equivalent to the `when_*` functions except that they do not return a future. Those wait for all futures to become ready before returning to the user. Note that the `wait_*` functions will rethrow one of the exceptions from exceptional futures. The `wait_*_nothrow` combinators are equivalent to the `wait_*` functions exception that they do not throw if one of the futures has become exceptional.

The `split_future` combinator takes a single future of a container (e.g. `tuple`) and turns it into a container of futures.

See *lcos_local*, *synchronization*, and *async_distributed* for other synchronization facilities.

See the *API reference* of this module for more details.

async_cuda

This library adds a simple API that enables the user to retrieve a future from a CUDA[255] stream. Typically, a user may launch one or more kernels and then get a future from the stream that will become ready when those kernels have completed. It is important to note that multiple kernels may be launched without fetching a future, and multiple futures may be obtained from the helper. Please refer to the unit tests and examples for further examples.

See the *API reference* of this module for more details.

async_local

This module extends *async_base* to provide local implementations of *hpx::async*, *hpx::post*, *hpx::sync*, and *hpx::dataflow*. The *async_distributed* module extends the functionality in this module to work with *action*s.

See the API reference of this module for more details.

[255] https://www.nvidia.com/object/cuda_home_new.html

async_mpi

The MPI library is intended to simplify the process of integrating MPI[256] based codes with the *HPX* runtime. Any MPI function that is asynchronous and uses an `MPI_Request` may be converted into an `hpx::future`. The syntax is designed to allow a simple replacement of the MPI call with a futurized async version that accepts an executor instead of a communicator, and returns a future instead of assigning a request. Typically, an MPI call of the form

```
int MPI_Isend(buf, count, datatype, rank, tag, comm, request);
```

becomes

```
hpx::future<int> f = hpx::async(executor,
↪ MPI_Isend, buf, count, datatype, rank, tag);
```

When the MPI operation is complete, the future will become ready. This allows communication to integrated cleanly with the rest of *HPX*, in particular the continuation style of programming may be used to build up more complex code. Consider the following example, that chains user processing, sends and receives using continuations…

```
// create an executor for MPI dispatch
hpx::mpi::experimental::executor exec(MPI_COMM_WORLD);

// post an asynchronous receive using MPI_Irecv
hpx::future<int> f_recv = hpx::async(
    exec, MPI_Irecv, &data, rank, MPI_INT, rank_from, i);

// attach a continuation to run when the recv completes,
f_recv.then([=, &tokens, &counter](auto&&)
{
    // call an application specific function
    msg_recv(rank, size, rank_to, rank_from, tokens[i], i);

    // send a new message
    hpx::future<int> f_send = hpx::async(
        exec, MPI_Isend, &tokens[i], 1, MPI_INT, rank_to, i);

    // when that send completes
    f_send.then([=, &tokens, &counter](auto&&)
    {
        // call an application specific function
        msg_send(rank, size, rank_to, rank_from, tokens[i], i);
    });
}
```

The example above makes use of `MPI_Isend` and `MPI_Irecv`, but *any* MPI function that uses requests may be futurized in this manner. The following is a (non exhaustive) list of MPI functions that *should* be supported, though not all have been tested at the time of writing (please report any problems to the issue tracker).

```
int MPI_Isend(...);
int MPI_Ibsend(...);
```

(continues on next page)

256 https://en.wikipedia.org/wiki/Message_Passing_Interface

(continued from previous page)

```
int MPI_Issend(...);
int MPI_Irsend(...);
int MPI_Irecv(...);
int MPI_Imrecv(...);
int MPI_Ibarrier(...);
int MPI_Ibcast(...);
int MPI_Igather(...);
int MPI_Igatherv(...);
int MPI_Iscatter(...);
int MPI_Iscatterv(...);
int MPI_Iallgather(...);
int MPI_Iallgatherv(...);
int MPI_Ialltoall(...);
int MPI_Ialltoallv(...);
int MPI_Ialltoallw(...);
int MPI_Ireduce(...);
int MPI_Iallreduce(...);
int MPI_Ireduce_scatter(...);
int MPI_Ireduce_scatter_block(...);
int MPI_Iscan(...);
int MPI_Iexscan(...);
int MPI_Ineighbor_allgather(...);
int MPI_Ineighbor_allgatherv(...);
int MPI_Ineighbor_alltoall(...);
int MPI_Ineighbor_alltoallv(...);
int MPI_Ineighbor_alltoallw(...);
```

Note that the *HPX* mpi futurization wrapper should work with *any* asynchronous `MPI` call, as long as the function signature has the last two arguments `MPI_xxx(..., MPI_Comm comm, MPI_Request *request)` - internally these two parameters will be substituted by the executor and future data parameters that are supplied by template instantiations inside the `hpx::mpi` code.

See the API reference of this module for more details.

async_sycl

This module allows creating HPX futures using SYCL[257] events, effectively integrating asynchronous SYCL kernels and memory transfers with HPX. Building on this integration, this module also contains a SYCL executor. This executor encapsulates a SYCL queue. When SYCL queue member functions are launched with this executor, the user can automatically obtain the HPX futures associated with them.

The creation of the HPX futures using SYCL events is based on the same event polling mechanism that the CUDA HPX integration uses. Each registered event gets an associated callback and gets inserted into a callback vector to be polled by the scheduler in between tasks. Once the polling reveals the event is complete, the callback will be called, which in turn sets the future to ready (see sycl_event_callback.cpp). There are multiple adaptions for HipSYCL for this: To keep the runtime alive (avoiding the repeated on-the-fly creation of the runtime during the polling), we keep a default queue. Furthermore, we flush the internal SYCL DAG to ensure that the launched SYCL function is actually being executed.

[257] https://en.wikipedia.org/wiki/SYCL

The SYCL executor offers the usual post and async_execute functions. Additionally, it contains two get_future functions. One expects a pre-existing SYCL event to return a future, the other one does not but will launch an empty SYCL kernel instead, to obtain an event (causing higher overhead for the sake of being more convenient). The post and async_execute implementations here are actually different for HipSYCL and OneAPI, since the sycl::queue in OneAPI uses a different interface (using a code_location parameter) which requires some adaptations here.

To make this module compile, we use the -fno-sycl and -fsycl compiler parameters for the OneAPI use-case (requiring HPX to be compiled with dpcpp). For HipSYCL we use its cmake integration instead (requiring HPX to be compiled with clang++ and including HipSYCL as a library).

To build with OneAPI, use the CMake Variable HPX_WITH_SYCL=ON. To build with HipSYCL, use HPX_WITH_SYCL=ON and HPX_WITH_HIPSYCL=ON (and make sure find_package will find HipSYCL).

Lastly, the module contains three tests/examples. All three implement a simple vector add example. The first one obtains a future using the free method get_future, the second one uses a single SYCL executor and the last one is using multiple executors called from multiple host threads.

To build the tests, use " make tests.unit.modules.async_sycl " To run the tests, use "ctest -R sycl".

NOTE: Theoretically, this module could work with other SYCL implementations, but was only tested using OneAPI and HipSYCL so far.

See the *API reference* of this module for more details.

batch_environments

This module allows for the detection of execution as batch jobs, a series of programs executed without user intervention. All data is preselected and will be executed according to preset parameters, such as date or completion of another task. Batch environments are especially useful for executing repetitive tasks.

HPX supports the creation of batch jobs through the Portable Batch System (PBS) and SLURM.

For more information on batch environments, see *Running on batch systems* and the API reference for the module.

cache

This module provides two cache data structures:

- `hpx::util::cache::local_cache`
- `hpx::util::cache::lru_cache`

See the *API reference* of the module for more details.

checkpoint_base

The checkpoint_base module contains lower level facilities that wrap simple checkpointing capabilities. This module does not implement special handling for futures or components, but simply serializes all arguments to or from a given container.

This module exposes the `hpx::util::save_checkpoint_data`, `hpx::util::restore_checkpoint_data`, and `hpx::util::prepare_checkpoint_data` APIs. These functions encapsulate the basic serialization functionalities necessary to save/restore a variadic list of arguments to/from a given data container.

See the *API reference* of this module for more details.

command_line_handling_local

TODO: High-level description of the module.

See the API reference of this module for more details.

compute_local

TODO: High-level description of the module.

See the *API reference* of this module for more details.

concepts

This module provides helpers for emulating concepts. It provides the following macros:

- `HPX_CONCEPT_REQUIRES`
- `HPX_HAS_MEMBER_XXX_TRAIT_DEF`
- `HPX_HAS_XXX_TRAIT_DEF`

See the API reference of the module for more details.

concurrency

This module provides concurrency primitives useful for multi-threaded programming such as:

- *hpx::barrier*
- `hpx::util::cache_line_data` and `hpx::util::cache_aligned_data`: wrappers for aligning and padding data to cache lines.
- various lockfree queue data structures

See the API reference of the module for more details.

config

The config module contains various configuration options, typically hidden behind macros that choose the correct implementation based on the compiler and other available options. It also contains platform independent macros to control inlinining, export sets and more.

See the *API reference* of the module for more details.

config_registry

The config_registry module is a low level module providing helper functionality for registering configuration entries to a global registry from other modules. The `hpx::config_registry::add_module_config` function is used to add configuration options, and `hpx::config_registry::get_module_configs` can be used to retrieve configuration entries registered so far. `add_module_config_helper` can be used to register configuration entries through static global options.

See the API reference of this module for more details.

contracts

The contracts module provides C++ contracts support for HPX with intelligent fallback to assertions when native contracts are not available. This module implements a forward-compatible API that works across different C++ standards and compiler capabilities.

The module provides three primary macros: `HPX_PRE`, `HPX_POST`, and `HPX_CONTRACT_ASSERT`. These macros automatically adapt their behavior based on compiler capabilities:

- When C++26 native contracts are available (`__cpp_contracts` defined), they map to standard contract syntax
- When `HPX_WITH_CX26_CONTRACTS=OFF`, preconditions and postconditions become no-ops while contract assertions remain available as enhanced assertions

Configuration

Enable contracts in CMake:

```
cmake -DHPX_WITH_CXX26_CONTRACTS=ON -DCMAKE_CXX_STANDARD=26
```

Enable contract-enhanced assertions (optional):

```
cmake -DHPX_WITH_CXX26_
↪CONTRACTS=ON -DHPX_CONTRACTS_WITH_ASSERTS_AS_CONTRACT_ASSERTS=ON
```

Contract assertions work even when contracts are disabled:

```
cmake -DHPX_WITH_CXX26_
↪CONTRACTS=OFF  # HPX_CONTRACT_ASSERT still maps to HPX_ASSERT
```

Advanced Features

Contract-Enhanced Assertions

When HPX_CONTRACTS_WITH_ASSERTS_AS_CONTRACT_ASSERTS=ON is enabled, regular HPX_ASSERT calls are automatically upgraded to use contract assertions in C++26 mode:

```
void example_function(int value)
{
    HPX_ASSERT(value␣
↪> 0);  // Becomes contract_assert(value > 0) in C++26 mode
    // ... rest of function
}
```

Warning

Enhanced assertions only provide benefits when native C++26 contracts are supported by the compiler. Without native contract support, HPX_ASSERT -> HPX_CONTRACT_ASSERT -> HPX_ASSERT (no enhancement). CMake will issue a warning if you enable this option without native contract support.

This provides enhanced contract semantics throughout your existing codebase without requiring changes to assertion code. The transformation occurs in the contracts module (contracts.hpp) where the HPX_ASSERT macro is overridden to use HPX_CONTRACT_ASSERT when HPX_WITH_ASSERTS_AS_CONTRACT_ASSERTS=ON:

- HPX_WITH_CXX26_CONTRACTS=ON - Contracts module is enabled
- HPX_CONTRACTS_WITH_ASSERTS_AS_CONTRACT_ASSERTS=ON - Assertion enhancement is enabled

The implementation works by redefining HPX_ASSERT in contracts.hpp to use HPX_CONTRACT_ASSERT, which automatically adapts to the current contract mode (native C++26 contracts or assertion fallback). This ensures all existing HPX_ASSERT calls throughout the HPX codebase automatically gain contract semantics when available.

Usage Examples

Preconditions and postconditions using declaration syntax (C++26):

```
int divide(int a, int b) HPX_PRE(b != 0)
{
    return a / b;
}

int factorial(int n) HPX_PRE(n >= 0) HPX_POST(r; r > 0)
{
    return n <= 1 ? 1 : n * factorial(n - 1);
}
```

Contract assertions (available in all modes):

```
void process_array(std::vector<int>& arr, size_t index)
{
    HPX_CONTRACT_ASSERT(index < arr.size());
    arr[index] *= 2;
}
```

Design Philosophy

HPX_CONTRACT_ASSERT: Enhanced assertion mechanism
Available even when `HPX_WITH_CONTRACTS=OFF` because it provides value as an enhanced assertion. Maps to `HPX_ASSERT` in all configurations.

HPX_PRE/HPX_POST: True contract syntax
Represent language-level contract semantics. When contracts are enabled but native C++26 contracts are not available, these become no-ops to maintain forward compatibility. When `HPX_WITH_CONTRACTS=OFF`, they are also no-ops. This prepares for C++26 migration where they will be attached to function declarations rather than used in function bodies.

Migration Strategy

The module is designed for smooth migration to C++26 native contracts:

Current (transition mode):

```
int func(int x)
{
    HPX_
↪PRE(x > 0);     // No-op in fallback mode, active in native mode
    return x;
}
```

Target (C++26 native):

```
int func(int x) HPX_PRE(x > 0)
{
    return x;
}
```

Note: In fallback mode, `HPX_PRE` and `HPX_POST` become no-ops to maintain forward compatibility and avoid performance overhead. Use `HPX_CONTRACT_ASSERT` when you need runtime validation in all modes.

Testing

The module includes comprehensive testing with automatic compiler capability detection. Tests are organized into three categories:

- **Declaration tests**: Test C++26 native contract syntax when `__cpp_contracts` is available
- **Fallback tests**: Test assertion fallback behavior when contracts are not natively supported
- **Disabled tests**: Test no-op behavior when contracts are disabled

The test suite automatically detects compiler capabilities at configure time and builds only the appropriate tests for the current configuration.

See the *API reference* of the module for more details.

coroutines

The coroutines module provides coroutine (user-space thread) implementations for different platforms.

See the *API reference* of the module for more details.

datastructures

The datastructures module provides basic data structures (typically provided for compatibility with older C++ standards):

- `hpx::detail::small_vector`
- `hpx::util::basic_any`
- `hpx::util::member_pack`
- `hpx::optional`
- *hpx::tuple*
- `hpx::variant`

See the *API reference* of the module for more details.

debugging

This module provides helpers for demangling symbol names.

See the *API reference* of the module for more details.

errors

This module provides support for exceptions and error codes:

- *hpx::exception*
- *hpx::error_code*
- `hpx::error`

See the *API reference* of the module for more details.

execution

This library implements executors and execution policies for use with parallel algorithms and other facilities related to managing the execution of tasks.

See the *API reference* of the module for more details.

execution_base

The basic execution module is the main entry point to implement parallel and concurrent operations. It is modeled after P0443[258] with some additions and implementations for the described concepts. Most notably, it provides an abstraction for execution resources, execution contexts and execution agents in such a way, that it provides customization points that those aforementioned concepts can be replaced and combined with ease.

For that purpose, three virtual base classes are provided to be able to provide implementations with different properties:

- **`resource_base`: This is the abstraction for execution resources, that is** for example CPU cores or an accelerator.
- **`context_base`: An execution context uses execution resources and is able** to spawn new execution agents, as new threads of executions on the available resources.
- **`agent_base`: The execution agent represents the thread of execution, and** can be used to yield, suspend, resume or abort a thread of execution.

executors

The executors module exposes executors and execution policies. Most importantly, it exposes the following classes and constants:

- `hpx::execution::sequenced_executor`
- `hpx::execution::parallel_executor`
- *hpx::execution::sequenced_policy*
- *hpx::execution::parallel_policy*
- *hpx::execution::parallel_unsequenced_policy*
- *hpx::execution::sequenced_task_policy*
- *hpx::execution::parallel_task_policy*
- *hpx::execution::seq*

[258] http://wg21.link/p0443

- *hpx::execution::par*
- *hpx::execution::par_unseq*
- *hpx::execution::task*

See the *API reference* of this module for more details.

filesystem

This module provides a compatibility layer for the C++17 filesystem library. If the filesystem library is available this module will simply forward its contents into the `hpx::filesystem` namespace. If the library is not available it will fall back to Boost.Filesystem instead.

See the *API reference* of the module for more details.

format

The format module exposes the `format` and `format_to` functions for formatting strings.

See the API reference of the module for more details.

functional

This module provides function wrappers and helpers for managing functions and their arguments.

- *hpx::function*
- *hpx::function_ref*
- *hpx::move_only_function*
- *hpx::bind*
- *hpx::bind_back*
- *hpx::bind_front*
- `hpx::util::deferred_call`
- *hpx::invoke*
- `hpx::invoke_r`
- *hpx::invoke_fused*
- *hpx::invoke_fused_r*
- *hpx::mem_fn*
- `hpx::util::one_shot`
- `hpx::util::protect`
- `hpx::util::result_of`
- *hpx::placeholders::_1*
- *hpx::placeholders::_2*
- …

- *hpx::placeholders::_9*

See the *API reference* of the module for more details.

futures

This module defines the *hpx::future* and *hpx::shared_future* classes corresponding to the C++ standard library classes std::future[259] and std::shared_future[260]. Note that the specializations of `hpx::future::then` for executors and execution policies are defined in the *execution* module.

See the *API reference* of this module for more details.

hardware

The hardware module abstracts away hardware specific details of timestamps and CPU features.

See the API reference of the module for more details.

hashing

The hashing module provides two hashing implementations:

- `hpx::util::fibhash`
- `hpx::util::jenkins_hash`

See the API reference of the module for more details.

include_local

This module provides no functionality in itself. Instead it provides headers that group together other headers that often appear together. This module provides local-only headers.

See the API reference of this module for more details.

ini

TODO: High-level description of the module.

See the API reference of this module for more details.

[259] http://en.cppreference.com/w/cpp/thread/future
[260] http://en.cppreference.com/w/cpp/thread/shared_future

init_runtime_local

TODO: High-level description of the module.

See the API reference of this module for more details.

io_service

This module provides an abstraction over Boost.ASIO, combining multiple `asio::io_contexts` into a single pool. *`hpx::util::io_service_pool`* provides a simple pool of `asio::io_contexts` with an API similar to `asio::io_context`. `hpx::threads::detail::io_service_thread_pool` wraps *`hpx::util::io_service_pool`* into an interface derived from `hpx::threads::detail::thread_pool_base`.

See the *API reference* of this module for more details.

iterator_support

This module provides helpers for iterators. It provides `hpx::util::iterator_facade` and `hpx::util::iterator_adaptor` for creating new iterators, and the trait `hpx::util::is_iterator` along with more specific iterator traits.

See the API reference of the module for more details.

itt_notify

This module provides support for profiling with Intel VTune[261].

See the API reference of this module for more details.

lci_base

This module provides helper functionality for detecting LCI environments.

See the API reference of this module for more details.

lcos_local

This module provides the following local *LCO*s:

- *hpx::lcos::local::and_gate*
- `hpx::lcos::local::channel`
- `hpx::lcos::local::one_element_channel`
- `hpx::lcos::local::receive_channel`
- `hpx::lcos::local::send_channel`
- `hpx::lcos::local::guard`
- `hpx::lcos::local::guard_set`

[261] https://software.intel.com/content/www/us/en/develop/tools/vtune-profiler.html

- `hpx::lcos::local::run_guarded`
- `hpx::lcos::local::conditional_trigger`
- *hpx::packaged_task*
- *hpx::promise*
- `hpx::lcos::local::receive_buffer`
- *`hpx::lcos::local::trigger`*

See *lcos_distributed* for distributed LCOs. Basic synchronization primitives for use in *HPX* threads can be found in *synchronization*. *async_combinators* contains useful utility functions for combining futures.

See the *API reference* of this module for more details.

likwid

TODO: High-level description of the module.

See the API reference of this module for more details.

lock_registration

This module contains fucntionality for registering locks to detect when they are locked and unlocked on different threads.

See the API reference of this module for more details.

logging

This module provides useful macros for logging information.

See the API reference of the module for more details.

memory

Part of this module is a forked version of `boost::intrusive_ptr` from Boost.SmartPtr[262].

See the API reference of the module for more details.

mpi_base

This module provides helper functionality for detecting MPI[263] environments.

See the API reference of this module for more details.

[262] https://www.boost.org/doc/libs/release/libs/smart_ptr/doc/html/smart_ptr.html
[263] https://en.wikipedia.org/wiki/Message_Passing_Interface

pack_traversal

This module exposes the basic functionality for traversing various packs, both synchronously and asynchronously: `hpx::util::traverse_pack` and *hpx::util::traverse_pack_async*. It also exposes the higher level functionality of unwrapping nested futures: `hpx::util::unwrap` and its function object form `hpx::util::functional::unwrap`.

See the *API reference* of this module for more details.

plugin

This module provides base utilities for creating plugins.

See the API reference of the module for more details.

prefix

This module provides utilities for handling the prefix of an *HPX* application, i.e. the paths used for searching components and plugins.

See the API reference of this module for more details.

preprocessor

This library contains useful preprocessor macros:

- *HPX_PP_CAT*: Concatenate two tokens
- *HPX_PP_EXPAND*: Expands a preprocessor token
- *HPX_PP_NARGS*: Determines the number of arguments passed to a variadic macro
- *HPX_PP_STRINGIZE*: Turns a token into a string
- *HPX_PP_STRIP_PARENS*: Strips parenthesis from a token

See the *API reference* of the module for more details.

program_options

The module program_options is a direct fork of the Boost.Program_options[264] library (Boost V1.70.0[265]). In order to be included as an *HPX* module, the Boost.Program_options library has been moved to the `namespace hpx::program_options`. We have also replaced all Boost facilities the library depends on with either the equivalent facilities from the standard library or from *HPX*. As a result, the *HPX* program_options module is fully interface compatible with Boost.Program_options (sans the `hpx` namespace and the `#include <hpx/modules/program_options.hpp>` changes that need to be applied to all code relying on this library).

All credit goes to Vladimir Prus, the author of the excellent Boost.Program_options library. All bugs have been introduced by us.

See the API reference of the module for more details.

[264] https://www.boost.org/doc/html/program_options.html
[265] https://www.boost.org/doc/libs/1_70_0/doc/html/program_options.html

properties

This module implements the `prefer` customization point for properties in terms of P2220[266]. This differs from P1393[267] in that it relies fully on `tag_invoke` overloads and fewer base customization points. Actual properties are defined in modules. All functionality is experimental and can be accessed through the `hpx::experimental` namespace.

See the API reference of this module for more details.

resiliency

In *HPX*, a program failure is a manifestation of a failing task. This module exposes several APIs that allow users to manage failing tasks in a convenient way by either replaying a failed task or by replicating a specific task.

Task replay is analogous to the Checkpoint/Restart mechanism found in conventional execution models. The key difference being localized fault detection. When the runtime detects an error, it replays the failing task as opposed to completely rolling back the entire program to the previous checkpoint.

Task replication is designed to provide reliability enhancements by replicating a set of tasks and evaluating their results to determine a consensus among them. This technique is most effective in situations where there are few tasks in the critical path of the DAG which leaves the system underutilized or where hardware or software failures may result in an incorrect result instead of an error. However, the drawback of this method is the additional computational cost incurred by repeating a task multiple times.

The following API functions are exposed:

- *`hpx::resiliency::experimental::async_replay`*: This version of task replay will catch user-defined exceptions and automatically reschedule the task N times before throwing an `hpx::resiliency::experimental::abort_replay_exception` if no task is able to complete execution without an exception.
- *`hpx::resiliency::experimental::async_replay_validate`*: This version of replay adds an argument to async replay which receives a user-provided validation function to test the result of the task against. If the task's output is validated, the result is returned. If the output fails the check or an exception is thrown, the task is replayed until no errors are encountered or the number of specified retries has been exceeded.
- *`hpx::resiliency::experimental::async_replicate`*: This is the most basic implementation of the task replication. The API returns the first result that runs without detecting any errors.
- *`hpx::resiliency::experimental::async_replicate_validate`*: This API additionally takes a validation function which evaluates the return values produced by the threads. The first task to compute a valid result is returned.
- *`hpx::resiliency::experimental::async_replicate_vote`*: This API adds a vote function to the basic replicate function. Many hardware or software failures are silent errors which do not interrupt program flow. In order to detect errors of this kind, it is necessary to run the task several times and compare the values returned by every

[266] https://wg21.link/p2220
[267] http://wg21.link/p1393

version of the task. In order to determine which return value is "correct", the API allows the user to provide a custom consensus function to properly form a consensus. This voting function then returns the "correct"" answer.

- *hpx::resiliency::experimental::async_replicate_vote_validate*: This combines the features of the previously discussed replicate set. Replicate vote validate allows a user to provide a validation function to filter results. Additionally, as described in replicate vote, the user can provide a "voting function" which returns the consensus formed by the voting logic.
- *hpx::resiliency::experimental::dataflow_replay*: This version of dataflow replay will catch user-defined exceptions and automatically reschedules the task N times before throwing an `hpx::resiliency::experimental::abort_replay_exception` if no task is able to complete execution without an exception. Any arguments for the executed task that are futures will cause the task invocation to be delayed until all of those futures have become ready.
- *hpx::resiliency::experimental::dataflow_replay_validate* : This version of replay adds an argument to dataflow replay which receives a user-provided validation function to test the result of the task against. If the task's output is validated, the result is returned. If the output fails the check or an exception is thrown, the task is replayed until no errors are encountered or the number of specified retries have been exceeded. Any arguments for the executed task that are futures will cause the task invocation to be delayed until all of those futures have become ready.
- *hpx::resiliency::experimental::dataflow_replicate*: This is the most basic implementation of the task replication. The API returns the first result that runs without detecting any errors. Any arguments for the executed task that are futures will cause the task invocation to be delayed until all of those futures have become ready.
- *hpx::resiliency::experimental::dataflow_replicate_validate*: This API additionally takes a validation function which evaluates the return values produced by the threads. The first task to compute a valid result is returned. Any arguments for the executed task that are futures will cause the task invocation to be delayed until all of those futures have become ready.
- *hpx::resiliency::experimental::dataflow_replicate_vote*: This API adds a vote function to the basic replicate function. Many hardware or software failures are silent errors which do not interrupt program flow. In order to detect errors of this kind, it is necessary to run the task several times and compare the values returned by every version of the task. In order to determine which return value is "correct", the API allows the user to provide a custom consensus function to properly form a consensus. This voting function then returns the "correct" answer. Any arguments for the executed task that are futures will cause the task invocation to be delayed until all of those futures have become ready.
- *hpx::resiliency::experimental::dataflow_replicate_vote_validate*: This combines the features of the previously discussed replicate set. Replicate vote validate allows a user to provide a validation function to filter results. Additionally, as described in replicate vote, the user can provide a "voting function" which returns the consensus formed by the voting logic. Any arguments for the executed task that are futures will cause the task invocation to be delayed until all of those futures have become ready.

See the *API reference* of the module for more details.

resource_partitioner

The resource_partitioner module defines `hpx::resource::partitioner`, the class used by the runtime and users to partition available hardware resources into thread pools. See *Using the resource partitioner* for more details on using the resource partitioner in applications.

See the API reference of this module for more details.

runtime_configuration

This module handles the configuration options required by the runtime.

See the *API reference* of this module for more details.

runtime_local

TODO: High-level description of the library.

See the *API reference* of this module for more details.

schedulers

This module provides schedulers used by thread pools in the *thread_pools* module. There are currently three main schedulers:

- `hpx::threads::policies::local_priority_queue_scheduler`
- `hpx::threads::policies::static_priority_queue_scheduler`
- `hpx::threads::policies::shared_priority_queue_scheduler`

Other schedulers are specializations or variations of the above schedulers. See the examples of the *resource_partitioner* module for examples of specifying a custom scheduler for a thread pool.

See the API reference of this module for more details.

serialization

This module provides serialization primitives and support for all built-in types as well as all C++ Standard Library collection and utility types. This list is extended by *HPX* vocabulary types with proper support for global reference counting. *HPX*'s mode of serialization is derived from Boost's serialization model[268] and, as such, is mostly interface compatible with its Boost counterpart.

The purest form of serializing data is to copy the content of the payload bit by bit; however, this method is impractical for generic C++ types, which might be composed of more than just regular built-in types. Instead, *HPX*'s approach to serialization is derived from the Boost Serialization library, and is geared towards allowing the programmer of a given class explicit control and syntax of what to serialize. It is based on operator overloading of two special archive types that hold a buffer or stream to store the serialized data and is responsible for dispatching the serialization mechanism to the intrusive or non-intrusive version. The serialization process is recursive. Each member

[268] https://www.boost.org/doc/libs/1_72_0/libs/serialization/doc/index.html

that needs to be serialized must be specified explicitly. The advantage of this approach is that the serialization code is written in C++ and leverages all necessary programming techniques. The generic, user-facing interface allows for effective application of the serialization process without obstructing the algorithms that need special code for packing and unpacking. It also allows for optimizations in the implementation of the archives.

See the *API reference* of the module for more details.

static_reinit

This module provides a simple wrapper around static variables that can be reinitialized.

See the *API reference* of this module for more details.

string_util

This module contains string utilities inspired by the Boost String Algorithms Library.

See the API reference of this module for more details.

synchronization

This module provides synchronization primitives that should be used rather than the C++ standard ones in *HPX* threads:

- *hpx::barrier*
- *hpx::binary_semaphore*
- *hpx::call_once*
- *hpx::condition_variable*
- *hpx::condition_variable_any*
- *hpx::counting_semaphore*
- *`hpx::lcos::local::event`*
- *hpx::latch*
- *hpx::mutex*
- *hpx::no_mutex*
- *hpx::once_flag*
- *hpx::recursive_mutex*
- *hpx::shared_mutex*
- *`hpx::sliding_semaphore`*
- *hpx::spinlock* (*std::mutex* compatible spinlock)
- `hpx::spinlock_no_backoff` (*boost::mutex* compatible spinlock)
- `hpx::spinlock_pool`
- *hpx::stop_callback*
- *hpx::stop_source*

- *hpx::stop_token*
- `hpx::in_place_stop_token`
- *hpx::timed_mutex*
- `hpx::upgrade_to_unique_lock`
- `hpx::upgrade_lock`

See *lcos_local*, *async_combinators*, and *async_distributed* for higher level synchronization facilities.

See the *API reference* of this module for more details.

tag_invoke

TODO: High-level description of the module.

See the *API reference* of this module for more details.

testing

The testing module contains useful macros for testing. The results of tests can be printed with `hpx::util::report_errors`. The following macros are provided:

- `HPX_TEST`
- `HPX_TEST_MSG`
- `HPX_TEST_EQ`
- `HPX_TEST_NEQ`
- `HPX_TEST_LT`
- `HPX_TEST_LTE`
- `HPX_TEST_RANGE`
- `HPX_TEST_EQ_MSG`
- `HPX_TEST_NEQ_MSG`
- `HPX_SANITY`
- `HPX_SANITY_MSG`
- `HPX_SANITY_EQ`
- `HPX_SANITY_NEQ`
- `HPX_SANITY_LT`
- `HPX_SANITY_LTE`
- `HPX_SANITY_RANGE`
- `HPX_SANITY_EQ_MSG`

See the API reference of the module for more details.

thread_pool_util

This module contains helper functions for asynchronously suspending and resuming thread pools and their worker threads.

See the *API reference* of this module for more details.

thread_pools

This module defines the thread pools and utilities used by the *HPX* runtime. The only thread pool implementation provided by this module is `hpx::threads::detail::scheduled_thread_pool`, which is derived from `hpx::threads::detail::thread_pool_base` defined in the *threading_base* module.

See the API reference of this module for more details.

thread_support

This module provides miscellaneous utilities for threading and concurrency.

See the *API reference* of the module for more details.

threading

This module provides the equivalents of `std::thread` and `std::jthread` for lightweight *HPX* threads:

- *hpx::thread*
- *hpx::jthread*

See the *API reference* of this module for more details.

threading_base

This module contains the base class definition required for threads. The base class *`hpx::threads::thread_data`* is inherited by two specializations for stackful and stackless threads: `hpx::threads::thread_data_stackful` and `hpx::threads::thread_data_stackless`. In addition, the module defines the base classes for schedulers and thread pools: `hpx::threads::policies::scheduler_base` and *`hpx::threads::thread_pool_base`*.

See the API reference of this module for more details.

thread_manager

This module defines the *hpx::threads::threadmanager* class. This is used by the runtime to manage the creation and destruction of thread pools. The *resource_partitioner* module handles the partitioning of resources into thread pools, but not the creation of thread pools.

See the API reference of this module for more details.

thrust

The thrust module integrates *HPX* parallel algorithms with NVIDIA Thrust, enabling GPU acceleration using familiar *HPX* algorithm syntax.

Execution Policies

hpx::thrust::thrust_host_policy
CPU execution using optimized parallel implementations

hpx::thrust::thrust_device_policy
Synchronous GPU execution

hpx::thrust::thrust_task_policy
Asynchronous GPU execution returning *HPX* futures

Policy Mappings

The *HPX* thrust policies map directly to Thrust execution policies:

```
// Policy mappings
hpx::thrust::thrust_host_policy    -> thrust::host
hpx::thrust::thrust_device_policy  -> thrust::device
hpx::thrust::thrust_task_policy    -> thrust::par_nosync
```

Usage

Synchronous Device Execution

```
#include <hpx/modules/thrust.hpp>
#include <thrust/device_vector.h>

hpx::thrust::thrust_device_policy device{};
thrust::device_vector<int> d_vec(1000, 0);

hpx::fill(device, d_vec.begin(), d_vec.end(), 42);
int sum = hpx::reduce(device, d_vec.begin(), d_vec.end(), 0);
```

Asynchronous Execution with Futures

```
#include <hpx/modules/async_cuda.hpp>
#include <hpx/modules/thrust.hpp>

int hpx_main() {
    // Required for async operations
    hpx::cuda::experimental::enable_
↪user_polling polling_guard("default");

    hpx::thrust::thrust_task_policy task{};
    thrust::device_vector<int> d_vec(1000, 1);

    auto␣
↪fill_future = hpx::fill(task, d_vec.begin(), d_vec.end(), 42);
    fill_future.get();  // Standard HPX future operations

}
```

Build Requirements

- CUDA Toolkit 12.4.0+
- *HPX* with `HPX_WITH_CUDA=ON`
- Enable with: `cmake -DHPX_WITH_CUDA=ON ...`

See Also

- *async_cuda* - CUDA integration
- *execution* - *HPX* execution policies

timed_execution

This module provides extensions to the executor interfaces defined in the *execution* module that allow timed submission of tasks on thread pools (at or after a specified time).

See the *API reference* of this module for more details.

timing

This module provides the timing utilities (clocks and timers).

See the *API reference* of the module for more details.

topology

This module provides the class `hpx::threads::topology` which represents the hardware resources available on a node. The class is a light wrapper around the Portable Hardware Locality (HWLOC)[269] library. The `hpx::threads::cpu_mask` is a small companion class that represents a set of resources on a node.

See the *API reference* of the module for more details.

tracing

This module provides the *HPX* tracing abstraction: a compile-time-dispatched API that annotates *HPX* runtime events (task lifecycle, futures, parcels, work stealing, worker sleep, and more) and routes those annotations to a single profiler backend selected at build time.

Three profiler backends are supported, with a no-op fallback when none is enabled. `HPX_SetupTracing.cmake` errors out at configure time if more than one of *`HPX_WITH_APEX`*, *`HPX_WITH_ITTNOTIFY`* or *`HPX_WITH_TRACY`* is enabled.

- *tracy* — the Tracy[270] profiler, enabled with *`HPX_WITH_TRACY`*.
- Intel ITT API[271], enabled with *`HPX_WITH_ITTNOTIFY`*.
- APEX[272], enabled with *`HPX_WITH_APEX`*.
- an empty backend when none of the above is enabled — the C++ hooks are `constexpr` no-ops and the tracing macros expand to nothing, so the hooks compile away when tracing is disabled.

Include `hpx/modules/tracing.hpp` to get the public API; it dispatches at compile time to one of the per-backend headers under `hpx/tracing/backends/` . Entries are declared in the `hpx::tracing` namespace and the dispatch macros (`HPX_TRACING_MARK_EVENT`, `HPX_TRACING_ZONE`, `HPX_TRACING_PAUSE`, `HPX_TRACING_RESUME`) live in `hpx/tracing/macros.hpp`.

See *Tracy integration* for how to build *HPX* against Tracy and attach the profiler.

tracy

This module enables the integration of *HPX* applications with the Tracy[273] profiler. See *tracing* for the backend abstraction that dispatches to Tracy at compile time, and *Tracy integration* for how to build *HPX* against Tracy and attach the profiler.

[269] https://www.open-mpi.org/projects/hwloc/
[270] https://github.com/wolfpld/tracy
[271] https://github.com/intel/ittapi
[272] http://uo-oaciss.github.io/apex
[273] https://github.com/wolfpld/tracy

type_support

This module provides helper facilities related to types.

See the API reference of the module for more details.

util

The util module provides miscellaneous standalone utilities.

See the *API reference* of the module for more details.

version

This module macros and functions for accessing version information about *HPX* and its dependencies.

See the *API reference* of this module for more details.

2.8.2 Main *HPX* modules

actions

TODO: High-level description of the library.

See the *API reference* of this module for more details.

actions_base

TODO: High-level description of the library.

See the *API reference* of this module for more details.

agas

TODO: High-level description of the module.

See the *API reference* of this module for more details.

agas_base

This module holds the implementation of the four AGAS services: primary namespace, locality namespace, component namespace, and symbol namespace.

See the *API reference* of this module for more details.

async_colocated

TODO: High-level description of the module.

See the *API reference* of this module for more details.

async_distributed

This module contains functionality for asynchronously launching work on remote localities: *hpx::async*, *hpx::post*. This module extends the local-only functions in libs_async_local.

See the API reference of this module for more details.

checkpoint

A common need of users is to periodically backup an application. This practice provides resiliency and potential restart points in code. *HPX* utilizes the concept of a `checkpoint` to support this use case.

Found in `hpx/util/checkpoint.hpp`, `checkpoint`s are defined as objects that hold a serialized version of an object or set of objects at a particular moment in time. This representation can be stored in memory for later use or it can be written to disk for storage and/or recovery at a later point. In order to create and fill this object with data, users must use a function called `save_checkpoint`. In code the function looks like this:

```
hpx::future<hpx::util::checkpoint>
↪ hpx::util::save_checkpoint(a, b, c, ...);
```

`save_checkpoint` takes arbitrary data containers, such as `int`, `double`, `float`, `vector`, and `future`, and serializes them into a newly created `checkpoint` object. This function returns a `future` to a `checkpoint` containing the data. Here's an example of a simple use case:

```
using hpx::util::checkpoint;
using hpx::util::save_checkpoint;

std::vector<int> vec{1,2,3,4,5};
hpx::future<checkpoint> save_checkpoint(vec);
```

Once the future is ready, the checkpoint object will contain the `vector` `vec` and its five elements.

`prepare_checkpoint` takes arbitrary data containers (same as for `save_checkpoint`), , such as `int`, `double`, `float`, `vector`, and `future`, and calculates the necessary buffer space for the checkpoint that would be created if `save_checkpoint` was called with the same arguments. This function returns a `future` to a `checkpoint` that is appropriately initialized. Here's an example of a simple use case:

```
using hpx::util::checkpoint;
using hpx::util::prepare_checkpoint;

std::vector<int> vec{1,2,3,4,5};
hpx::future<checkpoint> prepare_checkpoint(vec);
```

Once the future is ready, the checkpoint object will be initialized with an appropriately sized internal buffer.

It is also possible to modify the launch policy used by `save_checkpoint`. This is accomplished by passing a launch policy as the first argument. It is important to note that passing `hpx::launch::sync` will cause `save_checkpoint` to return a `checkpoint` instead of a `future` to a `checkpoint`. All other policies passed to `save_checkpoint` will return a `future` to a `checkpoint`.

Sometimes `checkpoint` s must be declared before they are used. `save_checkpoint` allows users to move pre-created `checkpoint` s into the function as long as they are the first container passing into the function (In the case where a launch policy is used, the `checkpoint` will immediately follow the launch policy). An example of these features can be found below:

```
    char character = 'd';
    int integer = 10;
    float flt = 10.01f;
    bool boolean = true;
    std::string str = "I am a string of characters";
    std::vector<char> vec(str.begin(), str.end());
    checkpoint archive;

    // Test 1
    //  test basic functionality
    hpx::shared_future<checkpoint> f_archive = save_checkpoint(
        std::move(archive), character, integer, flt, boolean, str, vec);
```

Once users can create `checkpoints` they must now be able to restore the objects they contain into memory. This is accomplished by the function `restore_checkpoint`. This function takes a `checkpoint` and fills its data into the containers it is provided. It is important to remember that the containers must be ordered in the same way they were placed into the `checkpoint`. For clarity see the example below:

```
    char character2;
    int integer2;
    float flt2;
    bool boolean2;
    std::string str2;
    std::vector<char> vec2;

    restore_checkpoint(data,
 character2, integer2, flt2, boolean2, str2, vec2);
```

The core utility of `checkpoint` is in its ability to make certain data persistent. Often, this means that the data needs to be stored in an object, such as a file, for later use. *HPX* has two solutions for these issues: stream operator overloads and access iterators.

HPX contains two stream overloads, `operator<<` and `operator>>`, to stream data out of and into `checkpoint`. Here is an example of the overloads in use below:

```
    double a9 = 1.0, b9 = 1.1, c9 = 1.2;
    std::ofstream test_file_9("test_file_9.txt");
    hpx::future<checkpoint> f_9 = save_checkpoint(a9, b9, c9);
```

(continues on next page)

(continued from previous page)

```
    test_file_9 << f_9.get();
    test_file_9.close();

    double a9_1, b9_1, c9_1;
    std::ifstream test_file_9_1("test_file_9.txt");
    checkpoint archive9;
    test_file_9_1 >> archive9;
    restore_checkpoint(archive9, a9_1, b9_1, c9_1);
```

This is the primary way to move data into and out of a `checkpoint`. It is important to note, however, that users should be cautious when using a stream operator to load data and another function to remove it (or vice versa). Both `operator<<` and `operator>>` rely on a `.write()` and a `.read()` function respectively. In order to know how much data to read from the `std::istream`, the `operator<<` will write the size of the `checkpoint` before writing the `checkpoint` data. Correspondingly, the `operator>>` will read the size of the stored data before reading the data into a new instance of `checkpoint`. As long as the user employs the `operator<<` and `operator>>` to stream the data, this detail can be ignored.

Important

Be careful when mixing `operator<<` and `operator>>` with other facilities to read and write to a `checkpoint`. `operator<<` writes an extra variable, and `operator>>` reads this variable back separately. Used together the user will not encounter any issues and can safely ignore this detail.

Users may also move the data into and out of a `checkpoint` using the exposed `.begin()` and `.end()` iterators. An example of this use case is illustrated below.

```
    std::ofstream test_file_7("checkpoint_test_file.txt");
    std::vector<float> vec7{1.02f, 1.03f, 1.04f, 1.05f};
    hpx::future<checkpoint> fut_7 = save_checkpoint(vec7);
    checkpoint archive7 = fut_7.get();
    std::copy(archive7.begin(),    // Write data to ofstream
        archive7.end(),            // ie. the file
        std::ostream_iterator<char>(test_file_7));
    test_file_7.close();

    std::vector<float> vec7_1;
    std::vector<char> char_vec;
    std::ifstream test_file_7_1("checkpoint_test_file.txt");
    if (test_file_7_1)
    {
        test_file_7_1.seekg(0, test_file_7_1.end);
        auto length = test_file_7_1.tellg();
        test_file_7_1.seekg(0, test_file_7_1.beg);
        char_vec.resize(length);
        test_file_7_1.read(char_vec.data(), length);
    }
    checkpoint␣
↪archive7_1(std::move(char_vec));    // Write data to checkpoint
```

(continues on next page)

(continued from previous page)

```
restore_checkpoint(archive7_1, vec7_1);
```

Checkpointing components

save_checkpoint and restore_checkpoint are also able to store components inside checkpoints. This can be done in one of two ways. First a client of the component can be passed to save_checkpoint. When the user wishes to resurrect the component she can pass a client instance to restore_checkpoint.

This technique is demonstrated below:

```
// Try to checkpoint and restore a component with a client
std::vector<int> vec3{10, 10, 10, 10, 10};

// Create a component instance through client constructor
data_client D(hpx::find_here(), std::move(vec3));
hpx::future<checkpoint> f3 = save_checkpoint(D);

// Create a new client
data_client E;

// Restore server inside client instance
restore_checkpoint(f3.get(), E);
```

The second way a user can save a component is by passing a shared_ptr to the component to save_checkpoint. This component can be resurrected by creating a new instance of the component type and passing a shared_ptr to the new instance to restore_checkpoint.

This technique is demonstrated below:

```
// test checkpoint a component using a shared_ptr
std::vector<int> vec{1, 2, 3, 4, 5};
data_client A(hpx::find_here(), std::move(vec));

// Checkpoint Server
hpx::id_type old_id = A.get_id();

hpx::future<std::shared_ptr<data_server>> f_a_ptr =
    hpx::get_ptr<data_server>(A.get_id());
std::shared_ptr<data_server> a_ptr = f_a_ptr.get();
hpx::future<checkpoint> f = save_checkpoint(a_ptr);
auto&& data = f.get();

// test prepare_checkpoint API
checkpoint c = prepare_checkpoint(hpx::launch::sync, a_ptr);
HPX_TEST(c.size() == data.size());

// Restore Server
// Create a new server instance
std::shared_ptr<data_server> b_server;
restore_checkpoint(data, b_server);
```

collectives

The collectives module exposes a set of distributed collective operations. Those can be used to exchange data between participating sites in a coordinated way. At this point the module exposes the following collective primitives:

- *hpx::collectives::all_gather*: receives a set of values from all participating sites.
- *hpx::collectives::all_reduce*: performs a reduction on data from each participating site to each participating site.
- *hpx::collectives::all_to_all*: each participating site provides its element of the data to collect while all participating sites receive the data from every other site.
- *hpx::collectives::broadcast_to* and *hpx::collectives::broadcast_from*: performs a broadcast operation from a root site to all participating sites.
- *hpx::collectives::exclusive_scan*: performs an exclusive scan operation on a set of values received from all call sites operating on the given base name.
- *hpx::collectives::gather_here* and *hpx::collectives::gather_there*: gathers values from all participating sites.
- *hpx::collectives::inclusive_scan*: performs an inclusive scan operation on a set of values received from all call sites operating on the given base name.
- *hpx::collectives::reduce_here* and *hpx::collectives::reduce_there*: performs a reduction on data from each participating site to a root site.
- *hpx::collectives::scatter_to* and *hpx::collectives::scatter_from*: receives an element of a set of values operating on the given base name.
- *hpx::lcos::broadcast*: performs a given action on all given global identifiers.
- *hpx::distributed::barrier*: distributed barrier.
- *hpx::lcos::fold*: performs a fold with a given action on all given global identifiers.
- *hpx::distributed::latch*: distributed latch.
- *hpx::lcos::reduce*: performs a reduction on data from each given global identifiers.
- *hpx::lcos::spmd_block*: performs the same operation on a local image while providing handles to the other images.

See the *API reference* of the module for more details.

Hierarchical collectives

Hierarchical communicators split a collective into sub-communicators arranged as a tree. This reduces the number of participants in each individual operation, but it also means that one user-visible hierarchical call can touch several internal communicators.

Generation model

Hierarchical collectives use two internal generations for each user generation. For a user generation `k`, the first internal phase uses `2k - 1` and the second uses `2k`. Single-pass hierarchical operations and the inter-group exchange of *hpx::collectives::all_to_all* advance the communicator by two generations in one step so that shared hierarchical communicators stay aligned with two-phase collectives.

When a hierarchical communicator instance is shared between collective operations, callers must pass explicit, strictly consecutive positive generation numbers. Two-phase hierarchical collectives, including *hpx::collectives::all_gather*, *hpx::collectives::all_reduce*, *hpx::collectives::all_to_all*, *hpx::collectives::inclusive_scan*, *hpx::collectives::exclusive_scan*, and *hpx::distributed::barrier*, reject the default generation because their phase generations are derived from the explicit user generation.

command_line_handling

The command_line_handling module defines and handles the command-line options required by the *HPX* runtime, combining them with configuration options defined by the *runtime_configuration* module. The actual parsing of command line options is handled by the *program_options* module.

See the API reference of the module for more details.

components

TODO: High-level description of the module.

See the *API reference* of this module for more details.

components_base

TODO: High-level description of the library.

See the *API reference* of this module for more details.

compute

The compute module provides utilities for handling task and memory affinity on host systems.

See the *API reference* of the module for more details.

distribution_policies

TODO: High-level description of the module.

See the *API reference* of this module for more details.

execution_distributed

Provides distributed sender and receiver facilities for HPX execution.

See the *API reference* of this module for more details.

executors_distributed

This module provides the executor `hpx::execution::experimental::distribution_policy_executor`. It allows one to create work that is implicitly distributed over multiple localities.

See the *API reference* of this module for more details.

include

This module provides no functionality in itself. Instead it provides headers that group together other headers that often appear together.

See the *API reference* of this module for more details.

init_runtime

TODO: High-level description of the library.

See the *API reference* of this module for more details.

lcos_distributed

This module contains distributed *LCO*s. Currently the only LCO provided is `hpx::lcos::channel`, a construct for sending values from one *locality* to another. See libs_lcos_local for local LCOs.

See the API reference of this module for more details.

naming

TODO: High-level description of the module.

See the API reference of this module for more details.

naming_base

This module provides a forward declaration of *address_type*, *component_type* and *invalid_locality_id*.

See the *API reference* of this module for more details.

parcelport_lci

TODO: High-level description of the module.

See the API reference of this module for more details.

parcelport_mpi

TODO: High-level description of the module.

See the API reference of this module for more details.

parcelport_tcp

TODO: High-level description of the module.

See the API reference of this module for more details.

parcelset

TODO: High-level description of the module.

See the *API reference* of this module for more details.

parcelset_base

TODO: High-level description of the module.

See the *API reference* of this module for more details.

performance_counters

This module provides the basic functionality required for defining performance counters. See *Performance counters* for more information about performance counters.

See the API reference of this module for more details.

plugin_factories

TODO: High-level description of the module.

See the *API reference* of this module for more details.

resiliency_distributed

Software resiliency features of *HPX* were introduced in the *resiliency module*. This module extends the APIs to run on distributed-memory systems allowing the user to invoke the failing task on other localities at runtime. This is useful in cases where a node is identified to fail more often (e.g., for certain ALU computes) as the task can now be replayed or replicated among different localities. The API exposed allows for an easy integration with the local only resiliency APIs as well.

Distributed software resilience APIs have a similar function signature and lives under the same namespace of `hpx::resiliency::experimental`. The difference arises in the formal parameters where distributed APIs takes the localities as the first argument, and an action as opposed to a function or a function object. The localities signify the order in which the API will either schedule (in case of Task Replay) tasks in a round robin fashion or replicate the tasks onto the list of localities.

The list of APIs exposed by distributed resiliency modules is the same as those defined in *local resiliency module*.

See the *API reference* of this module for more details.

runtime_components

TODO: High-level description of the module.

See the *API reference* of this module for more details.

runtime_distributed

TODO: High-level description of the module.

See the *API reference* of this module for more details.

segmented_algorithms

Segmented algorithms extend the usual parallel *algorithms* by providing overloads that work with distributed containers, such as partitioned vectors.

See the *API reference* of the module for more details.

statistics

This module provide some statistics utilities like rolling min/max and histogram.

See the API reference of the module for more details.

Supervision module

The supervision module provides lifecycle event publication, state querying, and observer registration for actors/components running on local or remote localities.

Overview

Several core operations are exposed, each with a local (synchronous) call and a remote (locality-qualified, future-returning or `launch::sync_policy`) call:

Functions

Table 2.125: *hpx::supervision* functions

Function	Description
hpx::supervision::publish_event	*Publish a lifecycle event for a target.*
hpx::supervision::query_state	*Query the most recently observed lifecycle state.*
hpx::supervision::register_observer	*Register an observer for a target's lifecycle events.*
hpx::supervision::unregister_observer	*Unregister a previously registered lifecycle observer.*
hpx::supervision::remove_target	*Clear all locally tracked state for a target.*
hpx::supervision::await_terminal	*Wait for a target to reach a terminal lifecycle event.*
hpx::supervision::check_admission	*Check whether a target currently admits new dispatch.*
hpx::supervision::register_activity_observer	*Register an observer for activity-state transitions of all targets.*
hpx::supervision::unregister_activity_observer	*Unregister a previously registered activity observer.*

Publishing events

hpx::*future*<*hpx*::*supervision*::publish_result> `hpx::supervision::publish_event`(*hpx*::id_type const &locality, *hpx*::id_type const &target, *hpx*::*supervision*::event ev, *std*::uint64_t epoch = 0)

hpx::*supervision*::publish_result `hpx::supervision::publish_event`(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::id_type const &target, *hpx*::*supervision*::event ev, *std*::uint64_t epoch = 0, *hpx*::*error_code* &ec = *hpx*::*throws*) *hpx*::*supervision*::publish_result `hpx::supervision::publish_event`(*hpx*::id_type const &target, *hpx*::*supervision*::event ev, *std*::uint64_t epoch = 0, *hpx*::*error_code* &ec = *hpx*::*throws*)

Publish a lifecycle event for `target`. Events are visible immediately to local observers; remote observers are notified within roughly one to two parcel round-trips. Publishing non-terminal events is not idempotent - each call creates a distinct, timestamped record. `event::completed` and `event::failed` are latched: the first terminal publication for a target returns `publish_result::applied`; every later terminal publication for that target is a no-op that returns `publish_result::already_terminal`.

Querying lifecycle state

hpx::*future*<*hpx*::*supervision*::lifecycle_state> `hpx::supervision::query_state`(*hpx*::id_type const &locality, *hpx*::id_type const &target)

hpx::*supervision*::lifecycle_state `hpx::supervision::query_state`(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::id_type const &target, *hpx*::*error_code* &ec = *hpx*::*throws*)

hpx::*supervision*::lifecycle_state `hpx::supervision::query_state`(*hpx*::id_type const &target, *hpx*::*error_code* &ec = *hpx*::*throws*)

Query the most recently observed lifecycle state for `target`. Includes a sequence number for gap detection and a staleness error code for remote queries whose result may lag the latest event.

Registering observers

hpx::*future*<*hpx*::id_type> `hpx::supervision::register_observer`(*hpx*::id_type const &locality, *hpx*::id_type const &target, *hpx*::*supervision*::lifecycle_callback const &callback, *std*::optional<*std*::uint64_t> epoch_filter = *std*::nullopt)

hpx::id_type `hpx::supervision::register_observer`(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::id_type const &target, *hpx*::*supervision*::lifecycle_callback const &callback, *std*::optional<*std*::uint64_t> epoch_filter = *std*::nullopt, *hpx*::*error_code* &ec = *hpx*::*throws*)

hpx::id_type `hpx::supervision::register_observer`(*hpx*::id_type const &target, *hpx*::*supervision*::lifecycle_callback const &callback, *std*::optional<*std*::uint64_t> epoch_filter = *std*::nullopt, *hpx*::*error_code* &ec = *hpx*::*throws*)

Register `callback` to be invoked on lifecycle events of `target`, returning an observer handle usable with `unregister_observer`. Local callbacks fire synchronously within the publish call; remote callbacks fire via a retried parcel. If `epoch_filter` is set, the observer only receives notifications (including the initial state snapshot delivered at registration time) whose epoch matches; by default the observer receives notifications for every epoch.

Unregistering observers

hpx::*future*<void> `hpx::supervision::unregister_observer`(*hpx*::id_type const &locality, *hpx*::id_type const &observer_handle)

void `hpx::supervision::unregister_observer`(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::id_type const &observer_handle, *hpx*::*error_code* &ec = *hpx*::*throws*)

void `hpx::supervision::unregister_observer`(*hpx*::id_type const &observer_handle, *hpx*::*error_code* &ec = *hpx*::*throws*)

Unregister a previously registered observer. `observer_handle` must have been obtained from `register_observer`; a handle obtained from `register_activity_observer`, or any handle never returned by either registration function, is rejected.

Removing target state

hpx::*future*<void> `hpx::supervision::remove_target`(*hpx*::id_type const &locality, *hpx*::id_type const &target)

void `hpx::supervision::remove_target`(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::id_type const &target, *hpx*::*error_code* &ec = *hpx*::*throws*)

void `hpx::supervision::remove_target`(*hpx*::id_type const &target, *hpx*::*error_code* &ec = *hpx*::*throws*)

Clear all locally tracked state for `target`. Unlike `unregister_observer`, which removes a single previously registered observer handle, `remove_target` unconditionally forgets every piece of local state held for `target` - its recorded lifecycle state and current epoch, and any observers still registered for it - regardless of any specific observer handle. Intended for callers that know `target` will never be queried or observed again locally (e.g. after a failed registration that seeded some state for it, or once a peer has been evicted), so that local state does not accumulate indefinitely.

Waiting for terminal events

hpx::*future*<*hpx*::*supervision*::lifecycle_state> `hpx::supervision::await_terminal`(*hpx*::id_type const &locality, *hpx*::id_type const &target, *std*::uint64_t epoch = 0, *std*::chrono::steady_clock::duration timeout = *std*::chrono::steady_clock::duration::max())

hpx::*supervision*::lifecycle_state `hpx::supervision::await_terminal`(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::id_type const &target, *std*::uint64_t epoch = 0, *std*::chrono::steady_clock::duration timeout = *std*::chrono::steady_clock::duration::max(), *hpx*::*error_code* &ec = *hpx*::*throws*)

hpx::*future*<*hpx*::*supervision*::lifecycle_state> `hpx::supervision::await_terminal`(*hpx*::id_type const &target, *std*::uint64_t epoch = 0, *std*::chrono::steady_clock::duration timeout = *std*::chrono::steady_clock::duration::max())

Asynchronously wait, one-shot and without blocking a worker thread, and the `launch::sync_policy` overload waits synchronously until `target` reaches a terminal event (`event::completed` or `event::failed`) within `epoch`. If a terminal event has already been recorded for `target` in `epoch`, the returned future is immediately satisfied with that state. Waits are scoped to a single epoch: if `target`'s epoch advances past `epoch` before a terminal event occurs, any outstanding waiter for the stale epoch is invalidated. A waiter that reaches neither of these outcomes is bounded by `timeout` (or a built-in default if `timeout` is left at its sentinel value `std::chrono::steady_clock::duration::max()`), after which it is swept and invalidated. Dispatch is routed locally when `target`'s supervision manager lives on the calling locality, and remotely (via a registered distributed action) otherwise.

Checking dispatch admission

hpx::*supervision*::dispatch_outcome `hpx::supervision::check_admission`(*hpx*::id_type const &target, *std*::uint64_t epoch = 0)

Check whether `target` currently admits new dispatch under `epoch`, i.e. whether it is safe to schedule or route work to it right now. This is a pure, `noexcept` local read of the same terminal latch state consulted by `await_terminal`: it returns `dispatch_outcome::rejected_fenced` if `target` has already latched a terminal event (`event::completed` or `event::failed`) under `epoch`, and `dispatch_outcome::admitted` otherwise. Unlike `publish_result`, which reports how a publication was resolved, `dispatch_outcome` answers the separate, consumer-side question of whether dispatch should proceed; it must only be called on the locality `target` lives on.

This admission fence is scoped to a single locality's latch state: it reflects only terminal events published to the supervision manager hosting `target`, and it fails open (returns `admitted`) when `target` is unknown locally, including when its terminal event was only ever published elsewhere. There is no built-in mechanism that propagates terminal latch state across localities - observer registration delivers notifications, not admission state. Any additional locality that needs to fence its own dispatch decisions on `target` must independently mirror or route to the owning locality's state; this module does not provide that for you.

Registering activity observers

hpx::*future*<*hpx*::id_type> `hpx::supervision::register_activity_observer`(*hpx*::id_type const &locality, *hpx*::*supervision*::activity_callback const &callback, *std*::optional<*std*::uint64_t> epoch_filter = *std*::nullopt)

hpx::id_type `hpx::supervision::register_activity_observer`(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::*supervision*::activity_callback const &callback, *std*::optional<*std*::uint64_t> epoch_filter = *std*::nullopt, *hpx*::*error_code* &ec = *hpx*::*throws*)

hpx::id_type `hpx::supervision::register_activity_observer`(*hpx*::*supervision*::activity_callback const &callback, *std*::optional<*std*::uint64_t> epoch_filter = *std*::nullopt, *hpx*::*error_code* &ec = *hpx*::*throws*)

Register `callback` to observe activity-state transitions (between `hpx::supervision::activity_state::inactive` and `active`) of *all* targets tracked by a locality's supervision manager. Unlike `register_observer`, none of these overloads take a `target` parameter: the feature reports on every target the addressed manager tracks rather than on a single one.

Registering an activity observer for an already-active target triggers a replay of its current state. The snapshot of tracked state and the insertion of the observer are performed atomically under the manager's internal lock, which guarantees exactly-once delivery: the observer receives either the replay or a live transition that raced with registration, but never both and never neither.

Note that the notification itself (replay or live) is delivered after the lock is released. Its delivery order relative to any other concurrent live notification for the same target is *not* guaranteed, only the exactly-once property is guaranteed, not relative ordering.

Activity notifications are a pure discovery/notification signal: unlike `publish_event`, registering or unregistering an activity observer never feeds the terminal latch consulted by `check_admission`.

If `epoch_filter` is set, `callback` only receives notifications - including the registration-time replay - whose epoch matches; by default it receives notifications regardless of epoch.

Unregistering activity observers

hpx::*future*<void> `hpx::supervision::unregister_activity_observer`(*hpx*::id_type const &locality, *hpx*::id_type const &observer_handle)

void `hpx::supervision::unregister_activity_observer`(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::id_type const &observer_handle, *hpx*::*error_code* &ec = *hpx*::*throws*)

void `hpx::supervision::unregister_activity_observer`(*hpx*::id_type const &observer_handle, *hpx*::*error_code* &ec = *hpx*::*throws*)

Unregister a previously registered activity observer. `observer_handle` must have been obtained from `register_activity_observer`; a handle obtained from `register_observer`, or any handle never returned by either registration function, is rejected.

See the *API reference* of this module for more details.

2.9 API reference

HPX follows a versioning scheme with three numbers: `major.minor.patch`. We guarantee no breaking changes in the API for patch releases. Minor releases may remove or break existing APIs, but only after a deprecation period of at least two minor releases. In rare cases do we outright remove old and unused functionality without a deprecation period.

We do not provide any ABI compatibility guarantees between any versions, debug and release builds, and builds with different C++ standards.

The public API of *HPX* is presented below. Clicking on a name brings you to the full documentation for the class or function. Including the header specified in a heading brings in the features listed under that heading.

Note

Names listed here are guaranteed stable with respect to semantic versioning. However, at the moment the list is incomplete and certain unlisted features are intended to be in the public API. While we work on completing the list, if you're unsure about whether a particular unlisted name is part of the public API you can get into contact with us or open an issue and we'll clarify the situation.

2.9.1 Public API

Our API is semantically conforming; hence, the reader is highly encouraged to refer to the corresponding facility in the C++ Standard[274] if needed. All names below are also available in the top-level `hpx` namespace unless otherwise noted. The names in `hpx` should be preferred. The names in sub-namespaces will eventually be removed.

hpx/algorithm.hpp

The header hpx/algorithm.hpp[275] corresponds to the C++ standard library header algorithm[276]. See *Using parallel algorithms* for more information about the parallel algorithms.

Classes

Table 2.126: Classes of header `hpx/algorithm.hpp`

Class	C++ standard
hpx::experimental::reduction	N4808[277]
hpx::experimental::induction	N4808[278]

Functions

Table 2.127: *hpx* functions of header `hpx/algorithm.hpp`

hpx function	C++ standard
hpx::adjacent_find	std::adjacent_find[279]
hpx::all_of	std::all_of[280]
hpx::any_of	std::any_of[281]
hpx::copy	std::copy[282]
hpx::copy_if	std::copy_if[283]
hpx::copy_n	std::copy_n[284]
hpx::count	std::count[285]
hpx::count_if	std::count_if[286]
hpx::ends_with	std::ends_with[287]
hpx::equal	std::equal[288]
hpx::fill	std::fill[289]
hpx::fill_n	std::fill_n[290]
hpx::find	std::find[291]
hpx::find_end	std::find_end[292]
hpx::find_first_of	std::find_first_of[293]
hpx::find_if	std::find_if[294]
hpx::find_if_not	std::find_if_not[295]

continues on next page

[274] https://en.cppreference.com/w/cpp/header
[275] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp
[276] http://en.cppreference.com/w/cpp/header/algorithm
[277] http://wg21.link/n4808
[278] http://wg21.link/n4808

Table 2.127 – continued from previous page

hpx function	C++ standard
hpx::for_each	std::for_each[296]
hpx::for_each_n	std::for_each_n[297]
hpx::generate	std::generate[298]
hpx::generate_n	std::generate_n[299]
hpx::includes	std::includes[300]
hpx::inplace_merge	std::inplace_merge[301]
hpx::is_heap	std::is_heap[302]
hpx::is_heap_until	std::is_heap_until[303]
hpx::is_partitioned	std::is_partitioned[304]
hpx::is_sorted	std::is_sorted[305]
hpx::is_sorted_until	std::is_sorted_until[306]
hpx::lexicographical_compare	std::lexicographical_compare[307]
hpx::make_heap	std::make_heap[308]
hpx::max_element	std::max_element[309]
hpx::merge	std::merge[310]
hpx::min_element	std::min_element[311]
hpx::minmax_element	std::minmax_element[312]
hpx::mismatch	std::mismatch[313]
hpx::move	std::move[314]
hpx::none_of	std::none_of[315]
hpx::nth_element	std::nth_element[316]
hpx::partial_sort	std::partial_sort[317]
hpx::partial_sort_copy	std::partial_sort_copy[318]
hpx::partition	std::partition[319]
hpx::partition_copy	std::partition_copy[320]
`hpx::experimental::reduce_by_key`	reduce_by_key[321]
hpx::remove	std::remove[322]
hpx::remove_copy	std::remove_copy[323]
hpx::remove_copy_if	std::remove_copy_if[324]
hpx::remove_if	std::remove_if[325]
hpx::replace	std::replace[326]
hpx::replace_copy	std::replace_copy[327]
hpx::replace_copy_if	std::replace_copy_if[328]
hpx::replace_if	std::replace_if[329]
hpx::reverse	std::reverse[330]
hpx::reverse_copy	std::reverse_copy[331]
hpx::rotate	std::rotate[332]
hpx::rotate_copy	std::rotate_copy[333]
hpx::search	std::search[334]
hpx::search_n	std::search_n[335]
hpx::set_difference	std::set_difference[336]
hpx::set_intersection	std::set_intersection[337]
hpx::set_symmetric_difference	std::set_symmetric_difference[338]
hpx::set_union	std::set_union[339]
hpx::shift_left	std::shift_left[340]
hpx::shift_right	std::shift_right[341]
hpx::sort	std::sort[342]
hpx::experimental::sort_by_key	sort_by_key[343]
hpx::stable_partition	std::stable_partition[344]
hpx::stable_sort	std::stable_sort[345]

continues on next page

Table 2.127 – continued from previous page

hpx function	C++ standard
hpx::starts_with	std::starts_with[346]
hpx::swap_ranges	std::swap_ranges[347]
hpx::transform	std::transform[348]
hpx::unique	std::unique[349]
hpx::unique_copy	std::unique_copy[350]
hpx::experimental::for_loop	N4808[351]
hpx::experimental::for_loop_strided	N4808[352]
hpx::experimental::for_loop_n	N4808[353]
hpx::experimental::for_loop_n_strided	N4808[354]

[279] http://en.cppreference.com/w/cpp/algorithm/adjacent_find
[280] http://en.cppreference.com/w/cpp/algorithm/all_any_none_of
[281] http://en.cppreference.com/w/cpp/algorithm/all_any_none_of
[282] http://en.cppreference.com/w/cpp/algorithm/copy
[283] http://en.cppreference.com/w/cpp/algorithm/copy
[284] http://en.cppreference.com/w/cpp/algorithm/copy_n
[285] http://en.cppreference.com/w/cpp/algorithm/count
[286] http://en.cppreference.com/w/cpp/algorithm/count
[287] http://en.cppreference.com/w/cpp/algorithm/ranges/ends_with
[288] http://en.cppreference.com/w/cpp/algorithm/equal
[289] http://en.cppreference.com/w/cpp/algorithm/fill
[290] http://en.cppreference.com/w/cpp/algorithm/fill_n
[291] http://en.cppreference.com/w/cpp/algorithm/find
[292] http://en.cppreference.com/w/cpp/algorithm/find_end
[293] http://en.cppreference.com/w/cpp/algorithm/find_first_of
[294] http://en.cppreference.com/w/cpp/algorithm/find
[295] http://en.cppreference.com/w/cpp/algorithm/find
[296] http://en.cppreference.com/w/cpp/algorithm/for_each
[297] http://en.cppreference.com/w/cpp/algorithm/for_each_n
[298] http://en.cppreference.com/w/cpp/algorithm/generate
[299] http://en.cppreference.com/w/cpp/algorithm/generate_n
[300] http://en.cppreference.com/w/cpp/algorithm/includes
[301] http://en.cppreference.com/w/cpp/algorithm/inplace_merge
[302] http://en.cppreference.com/w/cpp/algorithm/is_heap
[303] http://en.cppreference.com/w/cpp/algorithm/is_heap_until
[304] http://en.cppreference.com/w/cpp/algorithm/is_partitioned
[305] http://en.cppreference.com/w/cpp/algorithm/is_sorted
[306] http://en.cppreference.com/w/cpp/algorithm/is_sorted_until
[307] http://en.cppreference.com/w/cpp/algorithm/lexicographical_compare
[308] http://en.cppreference.com/w/cpp/algorithm/make_heap
[309] http://en.cppreference.com/w/cpp/algorithm/max_element
[310] http://en.cppreference.com/w/cpp/algorithm/merge
[311] http://en.cppreference.com/w/cpp/algorithm/min_element
[312] http://en.cppreference.com/w/cpp/algorithm/minmax_element
[313] http://en.cppreference.com/w/cpp/algorithm/mismatch
[314] http://en.cppreference.com/w/cpp/algorithm/move
[315] http://en.cppreference.com/w/cpp/algorithm/all_any_none_of
[316] http://en.cppreference.com/w/cpp/algorithm/nth_element
[317] http://en.cppreference.com/w/cpp/algorithm/partial_sort
[318] http://en.cppreference.com/w/cpp/algorithm/partial_sort_copy
[319] http://en.cppreference.com/w/cpp/algorithm/partition
[320] http://en.cppreference.com/w/cpp/algorithm/partition_copy
[321] https://thrust.github.io/doc/group__reductions_gad5623f203f9b3fdcab72481c3913f0e0.html
[322] http://en.cppreference.com/w/cpp/algorithm/remove
[323] http://en.cppreference.com/w/cpp/algorithm/remove_copy
[324] http://en.cppreference.com/w/cpp/algorithm/remove_copy
[325] http://en.cppreference.com/w/cpp/algorithm/remove
[326] http://en.cppreference.com/w/cpp/algorithm/replace
[327] http://en.cppreference.com/w/cpp/algorithm/replace_copy
[328] http://en.cppreference.com/w/cpp/algorithm/replace_copy
[329] http://en.cppreference.com/w/cpp/algorithm/replace
[330] http://en.cppreference.com/w/cpp/algorithm/reverse
[331] http://en.cppreference.com/w/cpp/algorithm/reverse_copy
[332] http://en.cppreference.com/w/cpp/algorithm/rotate
[333] http://en.cppreference.com/w/cpp/algorithm/rotate_copy
[334] http://en.cppreference.com/w/cpp/algorithm/search
[335] http://en.cppreference.com/w/cpp/algorithm/search_n
[336] http://en.cppreference.com/w/cpp/algorithm/set_difference
[337] http://en.cppreference.com/w/cpp/algorithm/set_intersection
[338] http://en.cppreference.com/w/cpp/algorithm/set_symmetric_difference
[339] http://en.cppreference.com/w/cpp/algorithm/set_union
[340] http://en.cppreference.com/w/cpp/algorithm/shift
[341] http://en.cppreference.com/w/cpp/algorithm/shift
[342] http://en.cppreference.com/w/cpp/algorithm/sort
[343] https://thrust.github.io/doc/group__sorting_gabe038d6107f7c824cf74120500ef45ea.html
[344] http://en.cppreference.com/w/cpp/algorithm/stable_partition
[345] http://en.cppreference.com/w/cpp/algorithm/stable_sort
[346] http://en.cppreference.com/w/cpp/algorithm/ranges/starts_with
[347] http://en.cppreference.com/w/cpp/algorithm/swap_ranges
[348] http://en.cppreference.com/w/cpp/algorithm/transform
[349] http://en.cppreference.com/w/cpp/algorithm/unique
[350] http://en.cppreference.com/w/cpp/algorithm/unique_copy
[351] http://wg21.link/n4808
[352] http://wg21.link/n4808

Table 2.128: *hpx::ranges* functions of header `hpx/algorithm.hpp`

hpx::ranges function	C++ standard
hpx::ranges::adjacent_find	std::adjacent_find[355]
hpx::ranges::all_of	std::all_of[356]
hpx::ranges::any_of	std::any_of[357]
hpx::ranges::copy	std::copy[358]
hpx::ranges::copy_if	std::copy_if[359]
hpx::ranges::copy_n	std::copy_n[360]
hpx::ranges::count	std::count[361]
hpx::ranges::count_if	std::count_if[362]
hpx::ranges::ends_with	std::ends_with[363]
hpx::ranges::equal	std::equal[364]
hpx::ranges::fill	std::fill[365]
hpx::ranges::fill_n	std::fill_n[366]
hpx::ranges::find	std::find[367]
hpx::ranges::find_end	std::find_end[368]
hpx::ranges::find_first_of	std::find_first_of[369]
hpx::ranges::find_if	std::find_if[370]
hpx::ranges::find_if_not	std::find_if_not[371]
hpx::ranges::for_each	std::for_each[372]
hpx::ranges::for_each_n	std::for_each_n[373]
hpx::ranges::generate	std::generate[374]
hpx::ranges::generate_n	std::generate_n[375]
hpx::ranges::includes	std::includes[376]
hpx::ranges::inplace_merge	std::inplace_merge[377]
hpx::ranges::is_heap	std::is_heap[378]
hpx::ranges::is_heap_until	std::is_heap_until[379]
hpx::ranges::is_partitioned	std::is_partitioned[380]
hpx::ranges::is_sorted	std::is_sorted[381]
hpx::ranges::is_sorted_until	std::is_sorted_until[382]
hpx::ranges::make_heap	std::make_heap[383]
hpx::ranges::max_element	std::max_element[384]
hpx::ranges::merge	std::merge[385]
hpx::ranges::min_element	std::min_element[386]
hpx::ranges::minmax_element	std::minmax_element[387]
hpx::ranges::mismatch	std::mismatch[388]
hpx::ranges::move	std::move[389]
hpx::ranges::none_of	std::none_of[390]
hpx::ranges::nth_element	std::nth_element[391]
hpx::ranges::partial_sort	std::partial_sort[392]
hpx::ranges::partial_sort_copy	std::partial_sort_copy[393]
hpx::ranges::partition	std::partition[394]
hpx::ranges::partition_copy	std::partition_copy[395]
hpx::ranges::set_difference	std::set_difference[396]
hpx::ranges::set_intersection	std::set_intersection[397]
hpx::ranges::set_symmetric_difference	std::set_symmetric_difference[398]
hpx::ranges::set_union	std::set_union[399]
hpx::ranges::shift_left	P2440[400]
hpx::ranges::shift_right	P2440[401]
hpx::ranges::sort	std::sort[402]
hpx::ranges::stable_partition	std::stable_partition[403]

continues on next page

Table 2.128 – continued from previous page

hpx::ranges function	C++ standard
hpx::ranges::stable_sort	std::stable_sort[404]
hpx::ranges::starts_with	std::starts_with[405]
hpx::ranges::swap_ranges	std::swap_ranges[406]
hpx::ranges::transform	std::transform[407]
hpx::ranges::unique	std::unique[408]
hpx::ranges::unique_copy	std::unique_copy[409]
hpx::ranges::experimental::for_loop	N4808[410]
hpx::ranges::experimental::for_loop_strided	N4808[411]

hpx/any.hpp

The header hpx/any.hpp[412] corresponds to the C++ standard library header any[413].

hpx::any is compatible with `std::any`.

[355] http://en.cppreference.com/w/cpp/algorithm/ranges/adjacent_find
[356] http://en.cppreference.com/w/cpp/algorithm/ranges/all_any_none_of
[357] http://en.cppreference.com/w/cpp/algorithm/ranges/all_any_none_of
[358] http://en.cppreference.com/w/cpp/algorithm/ranges/copy
[359] http://en.cppreference.com/w/cpp/algorithm/ranges/copy
[360] http://en.cppreference.com/w/cpp/algorithm/ranges/copy_n
[361] http://en.cppreference.com/w/cpp/algorithm/ranges/count
[362] http://en.cppreference.com/w/cpp/algorithm/ranges/count
[363] http://en.cppreference.com/w/cpp/algorithm/ranges/ends_with
[364] http://en.cppreference.com/w/cpp/algorithm/ranges/equal
[365] http://en.cppreference.com/w/cpp/algorithm/ranges/fill
[366] http://en.cppreference.com/w/cpp/algorithm/ranges/fill_n
[367] http://en.cppreference.com/w/cpp/algorithm/ranges/find
[368] http://en.cppreference.com/w/cpp/algorithm/ranges/find_end
[369] http://en.cppreference.com/w/cpp/algorithm/ranges/find_first_of
[370] http://en.cppreference.com/w/cpp/algorithm/ranges/find
[371] http://en.cppreference.com/w/cpp/algorithm/ranges/find
[372] http://en.cppreference.com/w/cpp/algorithm/ranges/for_each
[373] http://en.cppreference.com/w/cpp/algorithm/ranges/for_each_n
[374] http://en.cppreference.com/w/cpp/algorithm/ranges/generate
[375] http://en.cppreference.com/w/cpp/algorithm/ranges/generate_n
[376] http://en.cppreference.com/w/cpp/algorithm/ranges/includes
[377] http://en.cppreference.com/w/cpp/algorithm/ranges/inplace_merge
[378] http://en.cppreference.com/w/cpp/algorithm/ranges/is_heap
[379] http://en.cppreference.com/w/cpp/algorithm/ranges/is_heap_until
[380] http://en.cppreference.com/w/cpp/algorithm/ranges/is_partitioned
[381] http://en.cppreference.com/w/cpp/algorithm/ranges/is_sorted
[382] http://en.cppreference.com/w/cpp/algorithm/ranges/is_sorted_until
[383] http://en.cppreference.com/w/cpp/algorithm/ranges/make_heap
[384] http://en.cppreference.com/w/cpp/algorithm/ranges/max_element
[385] http://en.cppreference.com/w/cpp/algorithm/ranges/merge
[386] http://en.cppreference.com/w/cpp/algorithm/ranges/min_element
[387] http://en.cppreference.com/w/cpp/algorithm/ranges/minmax_element
[388] http://en.cppreference.com/w/cpp/algorithm/ranges/mismatch
[389] http://en.cppreference.com/w/cpp/algorithm/ranges/move
[390] http://en.cppreference.com/w/cpp/algorithm/ranges/all_any_none_of
[391] http://en.cppreference.com/w/cpp/algorithm/ranges/nth_element
[392] http://en.cppreference.com/w/cpp/algorithm/ranges/partial_sort
[393] http://en.cppreference.com/w/cpp/algorithm/ranges/partial_sort_copy
[394] http://en.cppreference.com/w/cpp/algorithm/ranges/partition
[395] http://en.cppreference.com/w/cpp/algorithm/ranges/partition_copy
[396] http://en.cppreference.com/w/cpp/algorithm/ranges/set_difference
[397] http://en.cppreference.com/w/cpp/algorithm/ranges/set_intersection
[398] http://en.cppreference.com/w/cpp/algorithm/ranges/set_symmetric_difference
[399] http://en.cppreference.com/w/cpp/algorithm/ranges/set_union
[400] https://wg21.link/p2440
[401] https://wg21.link/p2440
[402] http://en.cppreference.com/w/cpp/algorithm/ranges/sort
[403] http://en.cppreference.com/w/cpp/algorithm/ranges/stable_partition
[404] http://en.cppreference.com/w/cpp/algorithm/ranges/stable_sort
[405] http://en.cppreference.com/w/cpp/algorithm/ranges/starts_with
[406] http://en.cppreference.com/w/cpp/algorithm/ranges/swap_ranges
[407] http://en.cppreference.com/w/cpp/algorithm/ranges/transform
[408] http://en.cppreference.com/w/cpp/algorithm/ranges/unique
[409] http://en.cppreference.com/w/cpp/algorithm/ranges/unique_copy
[410] http://wg21.link/n4808
[411] http://wg21.link/n4808
[412] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/any.hpp
[413] http://en.cppreference.com/w/cpp/header/any

hpx::any_nonser, *hpx::unique_any_nonser*, and their corresponding factory functions *hpx::make_any_nonser* and *hpx::make_unique_any_nonser* are *HPX* extensions that do not support serialization and have no counterparts in the C++ standard library.

Classes

Table 2.129: Classes of header hpx/any.hpp

Class	C++ standard
hpx::any	std::any[414]
hpx::any_nonser	
hpx::bad_any_cast	std::bad_any_cast[415]
hpx::unique_any_nonser	

Functions

Table 2.130: Functions of header hpx/any.hpp

Function	C++ standard
hpx::any_cast	std::any_cast[416]
hpx::make_any	std::make_any[417]
hpx::make_any_nonser	
hpx::make_unique_any_nonser	

hpx/assert.hpp

The header hpx/assert.hpp[418] corresponds to the C++ standard library header cassert[419].

HPX_ASSERT is the *HPX* equivalent to assert in cassert. HPX_ASSERT can also be used in CUDA device code.

Macros

Table 2.131: Macros of header hpx/assert.hpp

Macro
HPX_ASSERT
HPX_ASSERT_MSG

414 http://en.cppreference.com/w/cpp/utility/any
415 http://en.cppreference.com/w/cpp/utility/any/bad_any_cast
416 http://en.cppreference.com/w/cpp/utility/any/any_cast
417 http://en.cppreference.com/w/cpp/utility/any/make_any
418 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/assertion/include/hpx/assert.hpp
419 http://en.cppreference.com/w/cpp/header/cassert

hpx/barrier.hpp

The header hpx/barrier.hpp[420] corresponds to the C++ standard library header barrier[421] and contains a distributed barrier implementation. This functionality is also exposed through the `hpx::distributed` namespace. The name in `hpx::distributed` should be preferred.

Classes

Table 2.132: Classes of header `hpx/barrier.hpp`

Class	C++ standard
hpx::barrier	std::barrier[422]

Table 2.133: Distributed implementation of classes of header `hpx/barrier.hpp`

Class
hpx::distributed::barrier

hpx/channel.hpp

The header hpx/channel.hpp[423] contains a local and a distributed channel implementation. This functionality is also exposed through the `hpx::distributed` namespace. The name in `hpx::distributed` should be preferred.

The channel functionality is specific to *HPX* and has no counterpart in the C++ standard library.

Classes

Table 2.134: Classes of header `hpx/channel.hpp`

Class
`hpx::channel`

Table 2.135: Distributed implementation of classes of header `hpx/channel.hpp`

Class
`hpx::distributed::channel`

420 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/barrier.hpp
421 http://en.cppreference.com/w/cpp/header/barrier
422 http://en.cppreference.com/w/cpp/thread/barrier
423 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/channel.hpp

hpx/chrono.hpp

The header hpx/chrono.hpp[424] corresponds to the C++ standard library header chrono[425]. The following replacements and extensions are provided compared to chrono[426].

Classes

Table 2.136: Classes of header hpx/chrono.hpp

Class	C++ standard
hpx::chrono::high_resolution_clock	std::high_resolution_clock[427]
hpx::chrono::high_resolution_timer	
hpx::chrono::steady_time_point	std::time_point[428]

hpx/condition_variable.hpp

The header hpx/condition_variable.hpp[429] corresponds to the C++ standard library header condition_variable[430].

Classes

Table 2.137: Classes of header hpx/condition_variable.hpp

Class	C++ standard
hpx::condition_variable	std::condition_variable[431]
hpx::condition_variable_any	std::condition_variable_any[432]
hpx::cv_status	std::cv_status[433]

hpx/exception.hpp

The header hpx/exception.hpp[434] corresponds to the C++ standard library header exception[435]. *hpx::exception* extends `std::exception` and is the base class for all exceptions thrown in *HPX*. *HPX_THROW_EXCEPTION* can be used to throw *HPX* exceptions with file and line information attached to the exception.

[424] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/chrono.hpp
[425] http://en.cppreference.com/w/cpp/header/chrono
[426] http://en.cppreference.com/w/cpp/header/chrono
[427] http://en.cppreference.com/w/cpp/chrono/high_resolution_clock
[428] http://en.cppreference.com/w/cpp/chrono/time_point
[429] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/condition_variable.hpp
[430] http://en.cppreference.com/w/cpp/header/condition_variable
[431] http://en.cppreference.com/w/cpp/thread/condition_variable
[432] http://en.cppreference.com/w/cpp/thread/condition_variable_any
[433] http://en.cppreference.com/w/cpp/thread/cv_status
[434] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/exception.hpp
[435] http://en.cppreference.com/w/cpp/header/exception

Macros

- *HPX_THROW_EXCEPTION*

Classes

Table 2.138: Classes of header hpx/exception.hpp

Class	C++ standard
hpx::exception	std::exception[436]

hpx/execution.hpp

The header hpx/execution.hpp[437] corresponds to the C++ standard library header execution[438]. See *High level parallel facilities*, *Using parallel algorithms* and *Executor parameters and executor parameter traits* for more information about execution policies and executor parameters.

Note

These names are only available in the `hpx::execution` namespace, not in the top-level `hpx` namespace.

Constants

Table 2.139: Constants of header hpx/execution.hpp

Constant	C++ standard
hpx::execution::seq	std::execution_policy_tag[439]
hpx::execution::par	std::execution_policy_tag[440]
hpx::execution::par_unseq	std::execution_policy_tag[441]
hpx::execution::task	

436 http://en.cppreference.com/w/cpp/error/exception
437 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp
438 http://en.cppreference.com/w/cpp/header/execution
439 http://en.cppreference.com/w/cpp/algorithm/execution_policy_tag
440 http://en.cppreference.com/w/cpp/algorithm/execution_policy_tag
441 http://en.cppreference.com/w/cpp/algorithm/execution_policy_tag

Classes

Table 2.140: Classes of header `hpx/execution.hpp`

Class	C++ standard
hpx::execution::sequenced_policy	std::execution_policy_tag_t[442]
hpx::execution::parallel_policy	std::execution_policy_tag_t[443]
hpx::execution::parallel_unsequenced_policy	std::execution_policy_tag_t[444]
hpx::execution::sequenced_task_policy	
hpx::execution::parallel_task_policy	
hpx::execution::experimental::auto_chunk_size	
hpx::execution::experimental::dynamic_chunk_size	
hpx::execution::experimental::guided_chunk_size	
hpx::execution::experimental::persistent_auto_chunk_size	
hpx::execution::experimental::static_chunk_size	
hpx::execution::experimental::num_cores	

`hpx/functional.hpp`

The header hpx/functional.hpp[445] corresponds to the C++ standard library header functional[446]. *hpx::function* is a more efficient and serializable replacement for `std::function`.

Constants

The following constants correspond to the C++ standard std::placeholders[447]

Table 2.141: Constants of header `hpx/functional.hpp`

Constant
hpx::placeholders::_1
hpx::placeholders::_2
...
hpx::placeholders::_9

[442] http://en.cppreference.com/w/cpp/algorithm/execution_policy_tag_t
[443] http://en.cppreference.com/w/cpp/algorithm/execution_policy_tag_t
[444] http://en.cppreference.com/w/cpp/algorithm/execution_policy_tag_t
[445] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp
[446] http://en.cppreference.com/w/cpp/header/functional
[447] http://en.cppreference.com/w/cpp/utility/functional/placeholders

Classes

Table 2.142: Classes of header `hpx/functional.hpp`

Class	C++ standard
hpx::function	std::function[448]
hpx::function_ref	P0792[449]
hpx::move_only_function	std::move_only_function[450]
hpx::is_bind_expression	std::is_bind_expression[451]
hpx::is_placeholder	std::is_placeholder[452]
hpx::scoped_annotation	

Functions

Table 2.143: Functions of header `hpx/functional.hpp`

Function	C++ standard
hpx::annotated_function	
hpx::bind	std::bind[453]
hpx::bind_back	std::bind_front[454]
hpx::bind_front	std::bind_front[455]
hpx::invoke	std::invoke[456]
hpx::invoke_fused	std::apply[457]
hpx::invoke_fused_r	
hpx::mem_fn	std::mem_fn[458]

`hpx/future.hpp`

The header hpx/future.hpp[459] corresponds to the C++ standard library header future[460]. See *Extended facilities for futures* for more information about extensions to futures compared to the C++ standard library.

This header file also contains overloads of *hpx::async*, *hpx::post*, *hpx::sync*, and *hpx::dataflow* that can be used with actions. See *Action invocation* for more information about invoking actions.

[448] http://en.cppreference.com/w/cpp/utility/functional/function
[449] http://wg21.link/p0792
[450] http://en.cppreference.com/w/cpp/utility/functional/move_only_function
[451] http://en.cppreference.com/w/cpp/utility/functional/is_bind_expression
[452] http://en.cppreference.com/w/cpp/utility/functional/is_placeholder
[453] http://en.cppreference.com/w/cpp/utility/functional/bind
[454] http://en.cppreference.com/w/cpp/utility/functional/bind_front
[455] http://en.cppreference.com/w/cpp/utility/functional/bind_front
[456] http://en.cppreference.com/w/cpp/utility/functional/invoke
[457] http://en.cppreference.com/w/cpp/utility/apply
[458] http://en.cppreference.com/w/cpp/utility/functional/mem_fn
[459] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp
[460] http://en.cppreference.com/w/cpp/header/future

Classes

Table 2.144: Classes of header `hpx/future.hpp`

Class	C++ standard
hpx::future	std::future[461]
hpx::shared_future	std::shared_future[462]
hpx::promise	std::promise[463]
hpx::launch	std::launch[464]
hpx::packaged_task	std::packaged_task[465]

Note

All names except *hpx::promise* are also available in the top-level `hpx` namespace. `hpx::promise` refers to *hpx::distributed::promise*, a distributed variant of *hpx::promise*, but will eventually refer to *hpx::promise* after a deprecation period.

Table 2.145: Distributed implementation of classes of header `hpx/future.hpp`

Class
hpx::distributed::promise

[461] http://en.cppreference.com/w/cpp/thread/future
[462] http://en.cppreference.com/w/cpp/thread/shared_future
[463] http://en.cppreference.com/w/cpp/thread/promise
[464] http://en.cppreference.com/w/cpp/thread/launch
[465] http://en.cppreference.com/w/cpp/thread/packaged_task

Functions

Table 2.146: Functions of header `hpx/future.hpp`

Function	C++ standard
hpx::async	std::async[466]
hpx::post	
hpx::sync	
hpx::dataflow	
hpx::make_future	
hpx::make_shared_future	
hpx::make_ready_future	P0159[467]
hpx::make_ready_future_alloc	
hpx::make_ready_future_at	
hpx::make_ready_future_after	
hpx::make_exceptional_future	P0159[468]
hpx::when_all	P0159[469]
hpx::when_any	P0159[470]
hpx::when_some	
hpx::when_each	
hpx::wait_all	
hpx::wait_any	
hpx::wait_some	
hpx::wait_each	

`hpx/init.hpp`

The header hpx/init.hpp[471] contains functionality for starting, stopping, suspending, and resuming the *HPX* runtime. This is the main way to explicitly start the *HPX* runtime. See *Starting the HPX runtime* for more details on starting the *HPX* runtime.

Classes

Table 2.147: Classes of header `hpx/init.hpp`

Class
hpx::init_params
hpx::runtime_mode

466 http://en.cppreference.com/w/cpp/thread/async
467 http://wg21.link/p0159
468 http://wg21.link/p0159
469 http://wg21.link/p0159
470 http://wg21.link/p0159
471 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/init_runtime/include/hpx/init.hpp

Functions

Table 2.148: Functions of header hpx/init.hpp

Function
hpx::init
hpx::start
hpx::finalize
hpx::disconnect
hpx::suspend
hpx::resume

hpx/latch.hpp

The header hpx/latch.hpp[472] corresponds to the C++ standard library header latch[473]. It contains a local and a distributed latch implementation. This functionality is also exposed through the `hpx::distributed` namespace. The name in `hpx::distributed` should be preferred.

Classes

Table 2.149: Classes of header hpx/latch.hpp

Class	C++ standard
hpx::latch	std::latch[474]

Table 2.150: Distributed implementation of classes of header hpx/latch.hpp

Class
hpx::distributed::latch

hpx/mutex.hpp

The header hpx/mutex.hpp[475] corresponds to the C++ standard library header mutex[476].

[472] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/latch.hpp
[473] http://en.cppreference.com/w/cpp/header/latch
[474] http://en.cppreference.com/w/cpp/thread/latch
[475] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/mutex.hpp
[476] http://en.cppreference.com/w/cpp/header/mutex

Classes

Table 2.151: Classes of header `hpx/mutex.hpp`

Class	C++ standard
hpx::mutex	std::mutex[477]
hpx::no_mutex	
hpx::once_flag	std::once_flag[478]
hpx::recursive_mutex	std::recursive_mutex[479]
hpx::spinlock	
hpx::timed_mutex	std::timed_mutex[480]
hpx::unlock_guard	

Functions

Table 2.152: Functions of header `hpx/mutex.hpp`

Class	C++ standard
hpx::call_once	std::call_once[481]

`hpx/memory.hpp`

The header hpx/memory.hpp[482] corresponds to the C++ standard library header memory[483]. It contains parallel versions of the copy, fill, move, and construct helper functions in memory[484]. See *Using parallel algorithms* for more information about the parallel algorithms.

477 http://en.cppreference.com/w/cpp/thread/mutex
478 http://en.cppreference.com/w/cpp/thread/once_flag
479 http://en.cppreference.com/w/cpp/thread/recursive_mutex
480 http://en.cppreference.com/w/cpp/thread/timed_mutex
481 http://en.cppreference.com/w/cpp/thread/call_once
482 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/memory.hpp
483 http://en.cppreference.com/w/cpp/header/memory
484 http://en.cppreference.com/w/cpp/header/memory

Functions

Table 2.153: *hpx* functions of header `hpx/memory.hpp`

hpx function	C++ standard
hpx::uninitialized_copy	std::uninitialized_copy[485]
hpx::uninitialized_copy_n	std::uninitialized_copy_n[486]
hpx::uninitialized_default_construct	std::uninitialized_default_construct[487]
hpx::uninitialized_default_construct_n	std::uninitialized_default_construct_n[488]
hpx::uninitialized_fill	std::uninitialized_fill[489]
hpx::uninitialized_fill_n	std::uninitialized_fill_n[490]
hpx::uninitialized_move	std::uninitialized_move[491]
hpx::uninitialized_move_n	std::uninitialized_move_n[492]
hpx::uninitialized_value_construct	std::uninitialized_value_construct[493]
hpx::uninitialized_value_construct_n	std::uninitialized_value_construct_n[494]

Table 2.154: *hpx::ranges* functions of header `hpx/memory.hpp`

hpx::ranges function	C++ standard
hpx::ranges::uninitialized_copy	std::uninitialized_copy[495]
hpx::ranges::uninitialized_copy_n	std::uninitialized_copy_n[496]
hpx::ranges::uninitialized_default_construct	std::uninitialized_default_construct[497]
hpx::ranges::uninitialized_default_construct_n	std::uninitialized_default_construct_n[498]
hpx::ranges::uninitialized_fill	std::uninitialized_fill[499]
hpx::ranges::uninitialized_fill_n	std::uninitialized_fill_n[500]
hpx::ranges::uninitialized_move	std::uninitialized_move[501]
hpx::ranges::uninitialized_move_n	std::uninitialized_move_n[502]
hpx::ranges::uninitialized_value_construct	std::uninitialized_value_construct[503]
hpx::ranges::uninitialized_value_construct_n	std::uninitialized_value_construct_n[504]

[485] http://en.cppreference.com/w/cpp/memory/uninitialized_copy
[486] http://en.cppreference.com/w/cpp/memory/uninitialized_copy_n
[487] http://en.cppreference.com/w/cpp/memory/uninitialized_default_construct
[488] http://en.cppreference.com/w/cpp/memory/uninitialized_default_construct_n
[489] http://en.cppreference.com/w/cpp/memory/uninitialized_fill
[490] http://en.cppreference.com/w/cpp/memory/uninitialized_fill_n
[491] http://en.cppreference.com/w/cpp/memory/uninitialized_move
[492] http://en.cppreference.com/w/cpp/memory/uninitialized_move_n
[493] http://en.cppreference.com/w/cpp/memory/uninitialized_value_construct
[494] http://en.cppreference.com/w/cpp/memory/uninitialized_value_construct_n
[495] http://en.cppreference.com/w/cpp/memory/ranges/uninitialized_copy
[496] http://en.cppreference.com/w/cpp/memory/ranges/uninitialized_copy_n
[497] http://en.cppreference.com/w/cpp/memory/ranges/uninitialized_default_construct
[498] http://en.cppreference.com/w/cpp/memory/ranges/uninitialized_default_construct_n
[499] http://en.cppreference.com/w/cpp/memory/ranges/uninitialized_fill
[500] http://en.cppreference.com/w/cpp/memory/ranges/uninitialized_fill_n
[501] http://en.cppreference.com/w/cpp/memory/ranges/uninitialized_move
[502] http://en.cppreference.com/w/cpp/memory/ranges/uninitialized_move_n
[503] http://en.cppreference.com/w/cpp/memory/ranges/uninitialized_value_construct
[504] http://en.cppreference.com/w/cpp/memory/ranges/uninitialized_value_construct_n

hpx/numeric.hpp

The header hpx/numeric.hpp[505] corresponds to the C++ standard library header numeric[506]. See *Using parallel algorithms* for more information about the parallel algorithms.

Functions

Table 2.155: *hpx* functions of header `hpx/numeric.hpp`

hpx function	C++ standard
hpx::adjacent_difference	std::adjacent_difference[507]
hpx::exclusive_scan	std::exclusive_scan[508]
hpx::inclusive_scan	std::inclusive_scan[509]
hpx::reduce	std::reduce[510]
hpx::transform_exclusive_scan	std::transform_exclusive_scan[511]
hpx::transform_inclusive_scan	std::transform_inclusive_scan[512]
hpx::transform_reduce	std::transform_reduce[513]

Table 2.156: *hpx::ranges* functions of header `hpx/numeric.hpp`

hpx::ranges function
hpx::ranges::adjacent_difference
hpx::ranges::exclusive_scan
hpx::ranges::inclusive_scan
hpx::ranges::reduce
hpx::ranges::transform_exclusive_scan
hpx::ranges::transform_inclusive_scan
hpx::ranges::transform_reduce

hpx/optional.hpp

The header hpx/optional.hpp[514] corresponds to the C++ standard library header optional[515]. `hpx::optional` is compatible with `std::optional`.

505 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/numeric.hpp
506 http://en.cppreference.com/w/cpp/header/numeric
507 http://en.cppreference.com/w/cpp/algorithm/adjacent_difference
508 http://en.cppreference.com/w/cpp/algorithm/exclusive_scan
509 http://en.cppreference.com/w/cpp/algorithm/inclusive_scan
510 http://en.cppreference.com/w/cpp/algorithm/reduce
511 http://en.cppreference.com/w/cpp/algorithm/transform_exclusive_scan
512 http://en.cppreference.com/w/cpp/algorithm/transform_inclusive_scan
513 http://en.cppreference.com/w/cpp/algorithm/transform_reduce
514 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/optional.hpp
515 http://en.cppreference.com/w/cpp/header/optional

Constants

- `hpx::nullopt`

Classes

Table 2.157: Classes of header `hpx/optional.hpp`

Class	C++ standard
`hpx::optional`	std::optional[516]
`hpx::nullopt_t`	std::nullopt_t[517]
`hpx::bad_optional_access`	std::bad_optional_access[518]

`hpx/runtime.hpp`

The header hpx/runtime.hpp[519] contains functions for accessing local and distributed runtime information.

Typedefs

Table 2.158: Typedefs of header `hpx/runtime.hpp`

Typedef
hpx::startup_function_type
hpx::shutdown_function_type

[516] http://en.cppreference.com/w/cpp/utility/optional
[517] http://en.cppreference.com/w/cpp/utility/nullopt_t
[518] http://en.cppreference.com/w/cpp/utility/optional/bad_optional_access
[519] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

Functions

Table 2.159: Functions of header `hpx/runtime.hpp`

Function
hpx::find_root_locality
hpx::find_all_localities
hpx::find_remote_localities
hpx::find_locality
hpx::get_colocation_id
hpx::get_locality_id
hpx::get_num_worker_threads
hpx::get_worker_thread_num
hpx::get_thread_name
hpx::register_pre_startup_function
hpx::register_startup_function
hpx::register_pre_shutdown_function
hpx::register_shutdown_function
hpx::get_num_localities
hpx::get_locality_name

`hpx/experimental/scope.hpp`

The header hpx/experimental/scope.hpp[520] corresponds to the C++ standard library header experimental/scope[521].

Classes

Table 2.160: Classes of header `hpx/scope.hpp`

Class	C++ standard
hpx::experimental::scope_exit	std::scope_exit[522]
hpx::experimental::scope_fail	std::scope_fail[523]
hpx::experimental::scope_success	std::scope_success[524]

520 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/experimental/scope.hpp
521 http://en.cppreference.com/w/cpp/header/experimental/scope
522 http://en.cppreference.com/w/cpp/experimental/scope_exit
523 http://en.cppreference.com/w/cpp/experimental/scope_fail
524 http://en.cppreference.com/w/cpp/experimental/scope_success

hpx/semaphore.hpp

The header hpx/semaphore.hpp[525] corresponds to the C++ standard library header semaphore[526].

Classes

Table 2.161: Classes of header hpx/semaphore.hpp

Class	C++ standard
hpx::binary_semaphore	std::counting_semaphore[527]
hpx::counting_semaphore	std::counting_semaphore[528]

hpx/shared_mutex.hpp

The header hpx/shared_mutex.hpp[529] corresponds to the C++ standard library header shared_mutex[530].

Classes

Table 2.162: Classes of header hpx/shared_mutex.hpp

Class	C++ standard
hpx::shared_mutex	std::shared_mutex[531]

hpx/source_location.hpp

The header hpx/source_location.hpp[532] corresponds to the C++ standard library header source_location[533].

525 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/semaphore.hpp
526 http://en.cppreference.com/w/cpp/header/semaphore
527 http://en.cppreference.com/w/cpp/thread/counting_semaphore
528 http://en.cppreference.com/w/cpp/thread/counting_semaphore
529 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/shared_mutex.hpp
530 http://en.cppreference.com/w/cpp/header/shared_mutex
531 http://en.cppreference.com/w/cpp/thread/shared_mutex
532 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/source_location.hpp
533 http://en.cppreference.com/w/cpp/header/source_location

Classes

Table 2.163: Classes of header `hpx/system_error.hpp`

Class	C++ standard
hpx::source_location	std::source_location[534]

`hpx/stop_token.hpp`

The header hpx/stop_token.hpp[535] corresponds to the C++ standard library header stop_token[536].

Constants

Table 2.164: Constants of header `hpx/stop_token.hpp`

Constant	C++ standard
hpx::nostopstate	std::nostopstate[537]

Classes

Table 2.165: Classes of header `hpx/stop_token.hpp`

Class	C++ standard
hpx::stop_callback	std::stop_callback[538]
hpx::stop_source	std::stop_source[539]
hpx::stop_token	std::stop_token[540]
hpx::nostopstate_t	std::nostopstate_t[541]

534 http://en.cppreference.com/w/cpp/utility/source_location
535 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/stop_token.hpp
536 http://en.cppreference.com/w/cpp/header/stop_token
537 http://en.cppreference.com/w/cpp/thread/stop_source/nostopstate
538 http://en.cppreference.com/w/cpp/thread/stop_callback
539 http://en.cppreference.com/w/cpp/thread/stop_source
540 http://en.cppreference.com/w/cpp/thread/stop_token
541 http://en.cppreference.com/w/cpp/thread/stop_source/nostopstate_t

hpx/system_error.hpp

The header hpx/system_error.hpp[542] corresponds to the C++ standard library header system_error[543].

Classes

Table 2.166: Classes of header hpx/system_error.hpp

Class	C++ standard
hpx::error_code	std::error_code[544]

hpx/task_block.hpp

The header hpx/task_block.hpp[545] corresponds to the task_block feature in N4755[546]. See using_task_block for more details on using task blocks.

Classes

Table 2.167: Classes of header hpx/task_block.hpp

Class
hpx::experimental::task_canceled_exception
hpx::experimental::task_block

Functions

Table 2.168: Functions of header hpx/task_block.hpp

Function
hpx::experimental::define_task_block
hpx::experimental::define_task_block_restore_thread

[542] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/system_error.hpp
[543] http://en.cppreference.com/w/cpp/header/system_error
[544] http://en.cppreference.com/w/cpp/error/error_code
[545] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/task_block.hpp
[546] http://wg21.link/n4755

hpx/experimental/task_group.hpp

The header hpx/experimental/task_group.hpp[547] corresponds to the `task_group` feature in oneAPI Threading Building Blocks (oneTBB)[548].

Classes

Table 2.169: Classes of header `hpx/experimental/task_group.hpp`

Class
hpx::experimental::task_group

hpx/thread.hpp

The header hpx/thread.hpp[549] corresponds to the C++ standard library header thread[550]. The functionality in this header is equivalent to the standard library thread functionality, with the exception that the *HPX* equivalents are implemented on top of lightweight threads and the *HPX* runtime.

Classes

Table 2.170: Classes of header `hpx/thread.hpp`

Class	C++ standard
hpx::thread	std::thread[551]
hpx::jthread	std::jthread[552]

Functions

Table 2.171: Functions of header `hpx/thread.hpp`

Function	C++ standard
hpx::this_thread::yield	std::yield[553]
hpx::this_thread::get_id	std::get_id[554]
hpx::this_thread::sleep_for	std::sleep_for[555]
hpx::this_thread::sleep_until	std::sleep_until[556]

[547] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/experimental/task_group.hpp
[548] https://spec.oneapi.io/versions/1.0-rev-3/elements/oneTBB/source/task_scheduler/task_group/task_group_cls.html
[549] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/thread.hpp
[550] http://en.cppreference.com/w/cpp/header/thread
[551] http://en.cppreference.com/w/cpp/thread/thread
[552] http://en.cppreference.com/w/cpp/thread/jthread
[553] http://en.cppreference.com/w/cpp/thread/yield
[554] http://en.cppreference.com/w/cpp/thread/get_id
[555] http://en.cppreference.com/w/cpp/thread/sleep_for
[556] http://en.cppreference.com/w/cpp/thread/sleep_until

hpx/tuple.hpp

The header hpx/tuple.hpp[557] corresponds to the C++ standard library header tuple[558].
hpx::tuple can be used in CUDA device code, unlike `std::tuple`.

Constants

Table 2.172: Constants of header `hpx/tuple.hpp`

Constant	C++ standard
hpx::ignore	std::ignore[559]

Classes

Table 2.173: Classes of header `hpx/tuple.hpp`

Class	C++ standard
hpx::tuple	std::tuple[560]
hpx::tuple_size	std::tuple_size[561]
hpx::tuple_element	std::tuple_element[562]

Functions

Table 2.174: Functions of header `hpx/tuple.hpp`

Function	C++ standard
hpx::make_tuple	std::tuple_element[563]
hpx::tie	std::tie[564]
hpx::forward_as_tuple	std::forward_as_tuple[565]
hpx::tuple_cat	std::tuple_cat[566]
hpx::get	std::get[567]

557 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/tuple.hpp
558 http://en.cppreference.com/w/cpp/header/tuple
559 http://en.cppreference.com/w/cpp/utility/tuple/ignore
560 http://en.cppreference.com/w/cpp/utility/tuple
561 http://en.cppreference.com/w/cpp/utility/tuple_size
562 http://en.cppreference.com/w/cpp/utility/tuple_element
563 http://en.cppreference.com/w/cpp/utility/tuple/tuple_element
564 http://en.cppreference.com/w/cpp/utility/tuple/tie
565 http://en.cppreference.com/w/cpp/utility/tuple/forward_as_tuple
566 http://en.cppreference.com/w/cpp/utility/tuple/tuple_cat
567 http://en.cppreference.com/w/cpp/utility/tuple/get

hpx/type_traits.hpp

The header hpx/type_traits.hpp[568] corresponds to the C++ standard library header type_traits[569].

Classes

Table 2.175: Classes of header hpx/type_traits.hpp

Class	C++ standard
hpx::is_invocable	std::is_invocable[570]
hpx::is_invocable_r	std::is_invocable[571]

hpx/unwrap.hpp

The header hpx/unwrap.hpp[572] contains utilities for unwrapping futures.

Classes

Table 2.176: Classes of header hpx/unwrap.hpp

Class
hpx::functional::unwrap
hpx::functional::unwrap_n
hpx::functional::unwrap_all

Functions

Table 2.177: Functions of header hpx/unwrap.hpp

Function
hpx::unwrap
hpx::unwrap_n
hpx::unwrap_all
hpx::unwrapping
hpx::unwrapping_n
hpx::unwrapping_all

568 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/type_traits.hpp
569 http://en.cppreference.com/w/cpp/header/type_traits
570 http://en.cppreference.com/w/cpp/types/is_invocable
571 http://en.cppreference.com/w/cpp/types/is_invocable
572 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/unwrap.hpp

hpx/version.hpp

The header hpx/version.hpp[573] provides version information about *HPX*.

The macros and functions provided by this header are specific to *HPX* and have no counterparts in the C++ standard library.

Macros

Table 2.178: Macros of header hpx/version.hpp

Macro
HPX_VERSION_MAJOR
HPX_VERSION_MINOR
HPX_VERSION_SUBMINOR
HPX_VERSION_FULL
HPX_VERSION_DATE
HPX_VERSION_TAG
HPX_AGAS_VERSION

Functions

Table 2.179: Functions of header hpx/version.hpp

Function
hpx::major_version
hpx::minor_version
hpx::subminor_version
hpx::full_version
hpx::full_version_as_string
hpx::tag
hpx::agas_version
hpx::build_type
hpx::build_date_time

hpx/wrap_main.hpp

The header hpx/wrap_main.hpp[574] does not provide any direct functionality but is used for implicitly using `main` as the runtime entry point. See *Re-use the main() function as the main HPX entry point* for more details on implicitly starting the *HPX* runtime.

573 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/version/include/hpx/version.hpp
574 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/wrap/include/hpx/wrap_main.hpp

2.9.2 Public distributed API

Our Public Distributed API offers a rich set of tools and functions that enable developers to harness the full potential of distributed computing. Here, you'll find a comprehensive list of header files, classes and functions for various distributed computing features provided by *HPX*.

hpx/barrier.hpp

The header hpx/barrier.hpp[575] includes a distributed barrier implementation. For information regarding the C++ standard library header barrier[576], see *Public API*.

Classes

Table 2.180: Distributed implementation of classes of header `hpx/barrier.hpp`

Class
hpx::distributed::barrier

Functions

Table 2.181: *hpx* functions of header `hpx/barrier.hpp`

Function
`hpx::distributed::wait`
`hpx::distributed::synchronize`

hpx/collectives.hpp

The header hpx/collectives.hpp[577] contains definitions and implementations related to the collectives operations.

575 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/barrier.hpp
576 http://en.cppreference.com/w/cpp/header/barrier
577 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/collectives.hpp

Classes

Table 2.182: *hpx* classes of header `hpx/collectives.hpp`

Class
hpx::collectives::num_sites_arg
hpx::collectives::this_site_arg
hpx::collectives::that_site_arg
hpx::collectives::generation_arg
hpx::collectives::root_site_arg
hpx::collectives::tag_arg
hpx::collectives::arity_arg
hpx::collectives::communicator
hpx::collectives::channel_communicator

Functions

Table 2.183: *hpx* functions of header `hpx/collectives.hpp`

Function
hpx::collectives::all_gather
hpx::collectives::all_reduce
hpx::collectives::all_to_all
hpx::collectives::broadcast_to
hpx::collectives::broadcast_from
hpx::collectives::create_channel_communicator
hpx::collectives::set
hpx::collectives::get
hpx::collectives::create_communicator
hpx::collectives::create_hierarchical_communicator
hpx::collectives::create_local_communicator
hpx::collectives::communicator::set_info
hpx::collectives::communicator::get_info
hpx::collectives::communicator::is_root
hpx::collectives::exclusive_scan
hpx::collectives::gather_here
hpx::collectives::gather_there
hpx::collectives::inclusive_scan
hpx::collectives::reduce_here
hpx::collectives::reduce_there
hpx::collectives::scatter_from
hpx::collectives::scatter_to

`hpx/latch.hpp`

The header hpx/latch.hpp[578] includes a distributed latch implementation. For information regarding the C++ standard library header latch[579], see *Public API*.

Classes

Table 2.184: Distributed implementation of classes of header `hpx/latch.hpp`

Class
hpx::distributed::latch

Member functions

Table 2.185: *hpx* functions of class hpx::distributed::latch from header `hpx/latch.hpp`

Function
`hpx::distributed::latch::count_down_and_wait`
`hpx::distributed::latch::arrive_and_wait`
`hpx::distributed::latch::count_down`
`hpx::distributed::latch::is_ready`
`hpx::distributed::latch::try_wait`
`hpx::distributed::latch::wait`

`hpx/async.hpp`

The header hpx/async.hpp[580] includes distributed implementations of *hpx::async*, *hpx::post*, *hpx::sync*, and *hpx::dataflow*. For information regarding the C++ standard library header, see *Public API*.

Functions

Table 2.186: Distributed implementation of functions of header `hpx/async.hpp`

Functions
modules_hpx/async_distributed/async.hpp_api
modules_hpx/async_distributed/sync.hpp_api
modules_hpx/async_distributed/post.hpp_api
modules_hpx/async_distributed/dataflow.hpp_api

[578] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/latch.hpp
[579] http://en.cppreference.com/w/cpp/header/latch
[580] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/async_distributed/include/hpx/async.hpp

hpx/components.hpp

The header hpx/include/components.hpp[581] includes the components implementation. A component in *hpx* is a C++ class which can be created remotely and for which its member functions can be invoked remotely as well. More information about how components can be defined, created, and used can be found in *Writing components*. *Components and actions* includes examples on the accumulator, template accumulator and template function accumulator.

Macros

Table 2.187: *hpx* macros of header `hpx/components.hpp`

Macro
HPX_DEFINE_COMPONENT_ACTION
HPX_REGISTER_ACTION_DECLARATION
HPX_REGISTER_ACTION
HPX_REGISTER_COMMANDLINE_MODULE
HPX_REGISTER_COMPONENT
HPX_REGISTER_COMPONENT_MODULE
HPX_REGISTER_STARTUP_MODULE

Classes

Table 2.188: *hpx* classes of header `hpx/components.hpp`

Class
hpx::components::client
hpx::components::client_base
hpx::components::component
hpx::components::component_base
hpx::components::component_commandline_base

Functions

Table 2.189: *hpx* functions of header `hpx/components.hpp`

Function
*hpx::new_*

[581] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/include/components.hpp

hpx/supervision_dispatch.hpp

The header hpx/supervision_dispatch.hpp[582] provides peer discovery, cooperative lifecycle management, failure detection, and fenced action dispatch across localities. See *Supervision dispatch module* for the full API reference and *Supervision dispatch* for the narrative documentation.

Classes

Table 2.190: *hpx* classes of header `hpx/supervision_dispatch.hpp`

Class
hpx::supervision::registry

Functions

Table 2.191: *hpx* functions of header `hpx/supervision_dispatch.hpp`

Function
hpx::supervision::discover_and_join
hpx::supervision::dispatch_work

2.9.3 Supervision dispatch module

The supervision dispatch module provides peer discovery, cooperative lifecycle management, failure detection, and fenced action dispatch across localities, built on top of the *supervision* module's lifecycle events and admission checks. See that module's reference for the semantics of *hpx::supervision::check_admission* and `hpx::supervision::dispatch_outcome`; this page documents only the dispatch-specific surface layered on top of them. For the narrative on initialization ordering, one-shot discovery, shadow-state semantics, and client-side filtering versus fenced-dispatch admission, see *Supervision dispatch*.

Overview

[582] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/components/supervision_dispatch/include/hpx/supervision_dispatch.hpp

Functions

Table 2.192: *hpx::supervision* dispatch functions

Function	Description
hpx::supervision::init	*One-shot, idempotent runtime initialization.*
hpx::supervision::finalize	*One-shot, idempotent runtime teardown.*
hpx::supervision::is_initialized	*Whether the runtime is currently active.*
hpx::supervision::discover_peers	*One-time discovery pull across localities.*
hpx::supervision::fan_out_join	*Fan out join() calls to discovered peers.*
hpx::supervision::discover_and_join	*Composed discovery-and-join pass.*
hpx::supervision::dispatch_work	*Dispatch an action under supervision fencing.*

Types

Table 2.193: *hpx::supervision* dispatch types

Type	Description
hpx::supervision::registry	*Client handle mirroring a peer's lifecycle state.*
`hpx::supervision::server::peer_snapshot`	*Point-in-time view of a joined peer.*
`hpx::supervision::discovered_peer`	*A discovered peer with locality and join epoch.*

Lifecycle initialization and shutdown

hpx::*shared_future*<*hpx*::*supervision*::*registry*> `hpx::supervision::init`(*hpx*::chrono::steady_duration const &discovery_timeout = *hpx*::*supervision*::default_discovery_timeout)

hpx::*supervision*::*registry* `hpx::supervision::init`(*hpx*::*launch*::sync_policy, *hpx*::chrono::steady_duration const &discovery_timeout = *hpx*::*supervision*::default_discovery_timeout)

Performs one-shot, idempotent initialization of the supervision-dispatch runtime for this locality: creates a local registry, publishes `event::started` before its symbol name is registered, registers that name, and performs a single `discover_and_join()` pass. Calling `init()` while already active is a no-op; concurrent callers during `initializing` attach to the in-flight operation instead of racing it.

void `hpx::supervision::finalize`()

Performs one-shot, idempotent teardown of the runtime previously started by `init()`: publishes `event::completed` at the current/unchanged epoch, unregisters the registry's symbol name, and releases the registry. A no-op unless the runtime is currently `active`.

bool `hpx::supervision::is_initialized`() noexcept

Returns whether the runtime is currently `active`. Never blocks; safe to call from any thread, including while `init()`/`finalize()` is in flight.

Peer discovery

struct *hpx*::*supervision*::**discovered_peer**

A peer whose registry symbol name resolved during a `discover_peers()` call. Holds `locality`, `registry_client`, and `join_epoch`. `discover_peers()` leaves `join_epoch` at `hpx::supervision::unjoined_epoch` (never `0`, which is a legitimate join epoch); `fan_out_join()` fills it in with the epoch recorded by `registry::join()`. `locality` is cached for convenience but is never independently resolved or validated: it is always derived from `registry_client` itself (see `registry::get_locality()`), since exactly one registry exists per locality.

std::vector<*hpx*::*supervision*::*discovered_peer*> *hpx*::*supervision*::**discover_peers**(*hpx*::chrono::steady_duration const &timeout = *hpx*::*supervision*::default_discovery_timeout)

Performs a one-time discovery pull: concurrently resolves the pinned registry name of every remote locality, bounded by a single wait for `timeout`. Localities that have not yet called `init()` are silently excluded rather than causing a hang or failure.

std::vector<*hpx*::*supervision*::*discovered_peer*> *hpx*::*supervision*::**fan_out_join**(...)

Fans out `join()` calls from `local_registry` to every entry in `peers`. Reuses `registry::join()`'s reservation/idempotency machinery, so repeated or overlapping calls never create more than one shadow per peer locality.

Returns a `discovered_peer` for each peer whose `join()` call settled successfully within `timeout`, in the same relative order as `peers`. Peers whose `join()` call did not settle in time are omitted from the result (rather than left as gaps or defaulted entries), so the returned vector is a same-order *subset* of `peers`, not index-aligned with it.

std::vector<*hpx*::*supervision*::*discovered_peer*> *hpx*::*supervision*::**discover_and_join**(*hpx*::*supervision*::*registry* const &local_registry, *hpx*::chrono::steady_duration const &timeout = *hpx*::*supervision*::default_discovery_timeout)

Composes a single reactive discovery-and-join pass: one `discover_peers()` pull followed by one `fan_out_join()` call. Introduces no polling, timer, or repeated broadcast of its own.

Returns the same `discovered_peer` vector produced by `fan_out_join()` for the peers it discovered.

State query and event publication

hpx::*future*<*hpx*::*supervision*::lifecycle_state> `hpx::supervision::query_state`(*hpx*::*supervision*::*registry* const &handle)

hpx::*supervision*::lifecycle_state `hpx::supervision::query_state`(*hpx*::*launch*::sync_policy, *hpx*::*supervision*::*registry* const &handle, *hpx*::*error_code* &ec = *hpx*::*throws*)

hpx::*future*<*hpx*::*supervision*::lifecycle_state> `hpx::supervision::query_state`(*hpx*::*supervision*::*registry* const &handle, *hpx*::*supervision*::*discovered_peer* const &peer)

hpx::*supervision*::lifecycle_state `hpx::supervision::query_state`(*hpx*::*launch*::sync_policy, *hpx*::*supervision*::*registry* const &handle, *hpx*::*supervision*::*discovered_peer* const &peer, *hpx*::*error_code* &ec = *hpx*::*throws*)

The self variants resolve both locality and target to `hpx::find_here()` and query it on the local locality; the peer variants resolve both locality and target from `peer.locality` instead - `handle` is accepted only for overload resolution and symmetry with the self variants, and is otherwise unused in that case. Each overload is a thin forwarder over the corresponding raw-id `query_state()` overload in *Supervision module*; see that module's reference for the fields of the returned `lifecycle_state`, including the `epoch` value useful for recovering the epoch a handle's locality was started at.

hpx::*future*<*hpx*::*supervision*::publish_result> `hpx::supervision::publish_event`(*hpx*::*supervision*::*registry* const &handle, *hpx*::*supervision*::event ev, *std*::uint64_t epoch = 0)

hpx::*supervision*::publish_result `hpx::supervision::publish_event`(*hpx*::*launch*::sync_policy, *hpx*::*supervision*::*registry* const &handle, *hpx*::*supervision*::event ev, *std*::uint64_t epoch = 0, *hpx*::*error_code* &ec = *hpx*::*throws*)

Handle-based convenience overloads of `hpx::supervision::publish_event` that resolve both locality and target to `hpx::find_here()` instead of requiring the caller to name them explicitly. Thin forwarders over the raw-id `publish_event()` overloads in *Supervision module*. There is no peer variant: a locality publishes lifecycle events only for itself, never on behalf of a peer. The default epoch value of 0 is only correct for a locality's very first publication after init(); once the target's real epoch has advanced, 0 is stale. Callers publishing events after the initial one should first obtain the current epoch - e.g. via query_state(hpx::launch::sync, handle).epoch - and pass it explicitly."

Registry client

class *hpx*::*supervision*::**registry**

A lightweight, self-supervising client handle for pairing with peer localities; mirrors a peer's lifecycle state locally via `join()` and can itself be discovered by name in AGAS.

hpx::*future*<*hpx*::*supervision*::*joined_peer*> **join**(*hpx*::id_type const &peer_locality) const

hpx::*supervision*::*joined_peer* **join**(*hpx*::*launch*::sync_policy, *hpx*::id_type const &peer_locality, *hpx*::*error_code* &ec = *hpx*::*throws*) const

Joins a peer locality: creates (or reuses) local supervision state mirroring the peer's lifecycle. The returned `joined_peer::target` is `peer_locality` itself (not a generated local id) and is what `dispatch_work()` colocates against.

hpx::*future*<*std*::vector<*hpx*::*supervision*::*server*::*peer_snapshot*>> **snapshot_peers**() const

std::vector<*hpx*::*supervision*::*server*::*peer_snapshot*> **snapshot_peers**(*hpx*::*launch*::sync_policy, *hpx*::*error_code* &ec = *hpx*::*throws*) const

Returns a point-in-time snapshot of all fully joined, non-evicting peers.

hpx::*future*<bool> **register_name**()

bool **register_name**(*hpx*::*launch*::sync_policy, *hpx*::*error_code* &ec = *hpx*::*throws*)

hpx::*future*<*hpx*::id_type> **unregister_name**() const

hpx::id_type **unregister_name**(*hpx*::*launch*::sync_policy, *hpx*::*error_code* &ec = *hpx*::*throws*) const

struct *hpx*::*supervision*::*server*::**peer_snapshot**

Plain-data view of a single joined peer: `peer_locality` and `join_epoch`.

Fenced dispatch

template<typename **Action**, typename ...**Ts**>

decltype(auto) *hpx*::*supervision*::**dispatch_work**(*hpx*::id_type const &target, *std*::uint64_t epoch, *Ts*&&... ts)

Dispatches `Action` to `target` under supervision fencing. Performs a cheap, non-authoritative admission check on the caller's locality to short-circuit an already-known-fenced target, then dispatches to the target's own locality (via `hpx::colocated`) where an authoritative re-check against the same latch consulted by *hpx::supervision::check_admission* runs immediately before the wrapped action executes, on the same thread, closing the admission/invocation race. See *Supervision dispatch* for the full client-side-filtering versus fenced-admission narrative and worked examples.

Throws

`hpx::exception` with `hpx::error::target_fenced` if the target has latched a terminal event for `epoch` since the client-side check; the wrapped action is not invoked in that case.

Testing support

The following are declared in `hpx/supervision_dispatch/testing.hpp` and are **not** part of the stable public dispatch API; they exist solely to make unit tests for this module deterministic:

std::vector<*hpx*::*supervision*::*server*::*peer_snapshot*> *hpx*::*supervision*::testing::**local_snapshot_peers**()

void *hpx*::*supervision*::testing::**set_failure_detection_poll_timeout_for_testing**(*hpx*::chrono::steady_duration const &timeout)

hpx::id_type *hpx*::*supervision*::testing::**last_join_locality**()

void *hpx*::*supervision*::testing::**suspend_heartbeat_for_testing**()

bool *hpx*::*supervision*::testing::**failure_detection_sweep_in_flight_for_testing**()

See the *API reference* of this module for more details, and *Supervision module* for the underlying lifecycle event, admission, and dispatch-outcome semantics this module builds on.

2.9.4 Full API

The full API of *HPX* is presented below. The listings for the public API above refer to the full documentation below.

Note

Most names listed in the full API reference are implementation details or considered unstable. They are listed mostly for completeness. If there is a particular feature you think deserves being in the public API we may consider promoting it. In general we prioritize making sure features corresponding to C++ standard library features are stable and complete.

actions

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/actions/transfer_action.hpp

Defined in header hpx/actions/transfer_action.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/actions/transfer_base_action.hpp

Defined in header hpx/actions/transfer_base_action.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/actions/base_action.hpp

Defined in header hpx/actions/base_action.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/actions/actions_fwd.hpp

Defined in header hpx/actions/actions_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/actions/action_support.hpp

Defined in header hpx/actions/action_support.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

actions_base

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/actions_base/macros.hpp

Defined in header hpx/actions_base/macros.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Defines

`HPX_REGISTER_ACTION_INVOCATION_COUNT`(Action)

`HPX_REGISTER_PER_ACTION_DATA_COUNTER_TYPES`(Action)

`HPX_MAKE_ACTION`(func)

`HPX_MAKE_DIRECT_ACTION`(func)

`HPX_DECLARE_ACTION`(...)

Declares an action type.

`HPX_REGISTER_ACTION_DECLARATION`(...)

Declare the necessary component action boilerplate code.

The macro *HPX_REGISTER_ACTION_DECLARATION* can be used to declare all the boilerplate code which is required for proper functioning of component actions in the context of HPX.

The parameter *action* is the type of the action to declare the boilerplate for.

This macro can be invoked with an optional second parameter. This parameter specifies a unique name of the action to be used for serialization purposes. The second parameter has to be specified if the first parameter is not usable as a plain (nonqualified) C++ identifier, i.e. the first parameter contains special characters which cannot be part of a C++ identifier, such as '<', '>', or ':'.

```
namespace app
{
    ␣
↪// Define a simple component exposing one action 'print_greeting'
    class HPX_COMPONENT_EXPORT server
      : public hpx::components::component_base<server>
    {
        void print_greeting ()
        {
            hpx::cout << "Hey, how are you?\n" << std::flush;
        }

        ␣
↪ // Component actions need to be declared, this also defines the
        // type 'print_greeting_action' representing the action.
        HPX_DEFINE_COMPONENT_ACTION(server,
            print_greeting, print_greeting_action);
    };
}

// Declare␣
↪boilerplate code required for each of the component actions.
HPX_
↪REGISTER_ACTION_DECLARATION(app::server::print_greeting_action)
```

Example:

> **Note**
>
> This macro has to be used once for each of the component actions defined using one of the *HPX_DEFINE_COMPONENT_ACTION* macros. It has to be visible in all translation units using the action, thus it is recommended to place it into the header file defining the component.

HPX_REGISTER_ACTION_DECLARATION_(...)

HPX_REGISTER_ACTION_DECLARATION_1(action)

HPX_REGISTER_ACTION(...)

Define the necessary component action boilerplate code.

The macro *HPX_REGISTER_ACTION* can be used to define all the boilerplate code which is required for proper functioning of component actions in the context of HPX.

The parameter *action* is the type of the action to define the boilerplate for.

This macro can be invoked with an optional second parameter. This parameter specifies a unique name of the action to be used for serialization purposes. The second parameter has to be specified if the first parameter is not usable as a plain (nonqualified) C++ identifier, i.e. the first parameter contains special characters which cannot be part of a C++ identifier, such as ‘<’, ‘>’, or ‘:’.

Note

This macro has to be used once for each of the component actions defined using one of the *HPX_DEFINE_COMPONENT_ACTION* or *HPX_DEFINE_PLAIN_ACTION* macros. It has to occur exactly once for each of the actions, thus it is recommended to place it into the source file defining the component.

Note

Only one of the forms of this macro *HPX_REGISTER_ACTION* or *HPX_REGISTER_ACTION_ID* should be used for a particular action, never both.

HPX_REGISTER_ACTION_ID(action, actionname, actionid)

Define the necessary component action boilerplate code and assign a predefined unique id to the action.

The macro *HPX_REGISTER_ACTION* can be used to define all the boilerplate code which is required for proper functioning of component actions in the context of HPX.

The parameter *action* is the type of the action to define the boilerplate for.

The parameter *actionname* specifies a unique name of the action to be used for serialization purposes. The second parameter has to be usable as a plain (nonqualified) C++ identifier, it should not contain special characters which cannot be part of a C++ identifier, such as ‘<’, ‘>’, or ‘:’.

The parameter *actionid* specifies a unique integer value which will be used to represent the action during serialization.

Note

This macro has to be used once for each of the component actions defined using one of the *HPX_DEFINE_COMPONENT_ACTION* or global actions *HPX_DEFINE_PLAIN_ACTION* macros. It has to occur exactly once for each of the actions, thus it is recommended to place it into the source file defining the component.

Note

Only one of the forms of this macro *HPX_REGISTER_ACTION* or *HPX_REGISTER_ACTION_ID* should be used for a particular action, never both.

HPX_DEFINE_COMPONENT_ACTION(...)

Registers a member function of a component as an action type with HPX.

The macro *HPX_DEFINE_COMPONENT_ACTION* can be used to register a member function of a component as an action type named *action_type*.

The parameter *component* is the type of the component exposing the member function *func* which should be associated with the newly defined action type. The parameter `action_type` is the name of the action type to register with HPX.

```
namespace app
{
    // Define a simple component exposing one action 'print_greeting'
    class HPX_COMPONENT_EXPORT server
      : public hpx::components::component_base<server>
    {
        void print_greeting() const
        {
            hpx::cout << "Hey, how are you?\n" << std::flush;
        }

        // Component actions need to be declared, this also defines the
        // type 'print_greeting_action' representing the action.
        HPX_DEFINE_COMPONENT_ACTION(server, print_greeting,
            print_greeting_action);
    };
}
```

Example:

The first argument must provide the type name of the component the action is defined for.

The second argument must provide the member function name the action should wrap.

The default value for the third argument (the typename of the defined action) is derived from the name of the function (as passed as the second argument) by appending '_action'. The third argument can be omitted only if the second argument with an appended suffix '_action' resolves to a valid, unqualified C++ type name.

> **Note**
>
> The macro *HPX_DEFINE_COMPONENT_ACTION* can be used with 2 or 3 arguments. The third argument is optional.

`HPX_DEFINE_PLAIN_ACTION`(...)

Defines a plain action type.

```
namespace app
{
    void some_global_function(double d)
    {
        cout << d;
    }

    //
↪ This will define the action type 'app::some_global_action' which
    // represents the function 'app::some_global_function'.
    HPX_
↪DEFINE_PLAIN_ACTION(some_global_function, some_global_action);
}
```

Example:

Note

Usually this macro will not be used in user code unless the intent is to avoid defining the action_type in global namespace. Normally, the use of the macro *HPX_PLAIN_ACTION* is recommended.

Note

The macro *HPX_DEFINE_PLAIN_ACTION* can be used with 1 or 2 arguments. The second argument is optional. The default value for the second argument (the typename of the defined action) is derived from the name of the function (as passed as the first argument) by appending '_action'. The second argument can be omitted only if the first argument with an appended suffix '_action' resolves to a valid, unqualified C++ type name.

HPX_DECLARE_PLAIN_ACTION(...)

Declares a plain action type.

HPX_PLAIN_ACTION(...)

Defines a plain action type based on the given function *func* and registers it with HPX.

The macro *HPX_PLAIN_ACTION* can be used to define a plain action (e.g. an action encapsulating a global or free function) based on the given function *func*. It defines the action type *name* representing the given function. This macro additionally registers the newly define action type with HPX.

The parameter `func` is a global or free (non-member) function which should be encapsulated into a plain action. The parameter `name` is the name of the action type defined by this macro.

```
namespace app {
    void some_global_function(double d) {
        cout << d;
    }
}

// This will define the action type 'some_global_action' which
// represents the function 'app::some_global_function'.
HPX_PLAIN_ACTION(app::some_global_function, some_global_action)
```

Example:

Note

The macro *HPX_PLAIN_ACTION* has to be used at global namespace even if the wrapped function is located in some other namespace. The newly defined action type is placed into the global namespace as well.

Note

The macro *HPX_PLAIN_ACTION_ID* can be used with 1, 2, or 3 arguments. The second and third arguments are optional. The default value for the second argument (the typename of the defined action) is derived from the name of the function (as passed as the first argument) by appending '_action'. The second argument can be omitted only if the first argument with an appended suffix '_action' resolves to a valid, unqualified C++ type name. The default value for the third argument is *hpx::components::factory_state::check*.

Note

Only one of the forms of this macro *HPX_PLAIN_ACTION* or *HPX_PLAIN_ACTION_ID* should be used for a particular action, never both.

HPX_PLAIN_ACTION_ID(func, name, id)

Defines a plain action type based on the given function *func* and registers it with HPX.

The macro *HPX_PLAIN_ACTION_ID* can be used to define a plain action (e.g. an action encapsulating a global or free function) based on the given function *func*. It defines the action type *actionname* representing the given function.

The parameter *actionid* specifies a unique integer value which will be used to represent the action during serialization.

The parameter `func` is a global or free (non-member) function which should be encapsulated into a plain action. The parameter `name` is the name of the action type defined by this macro.

The second parameter has to be usable as a plain (nonqualified) C++ identifier, it should not contain special characters which cannot be part of a C++ identifier, such as ‘<’, ‘>’, or ‘:’.

```
namespace app {
    void some_global_function(double d) {
        cout << d;
    }
}

// This will define the action type 'some_global_action' which
// represents the function 'app::some_global_function'.
HPX_PLAIN_ACTION_ID(app::some_global_function, some_global_action,
    some_unique_id);
```

Example:

> **Note**
>
> The macro *HPX_PLAIN_ACTION_ID* has to be used at global namespace even if the wrapped function is located in some other namespace. The newly defined action type is placed into the global namespace as well.

> **Note**
>
> Only one of the forms of this macro *HPX_PLAIN_ACTION* or *HPX_PLAIN_ACTION_ID* should be used for a particular action, never both.

HPX_REGISTER_ACTION_FACTORY_ID(Name, Id)

hpx/actions_base/basic_action_fwd.hpp

Defined in header hpx/actions_base/basic_action_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **actions**

hpx/actions_base/reflect_annotated_actions.hpp

Defined in header hpx/actions_base/reflect_annotated_actions.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Annotation-driven HPX action registration via C++26 reflection.

Functions decorated with [[=hpx::actions::detail::remote_function{}]] are automatically discovered and registered as HPX plain actions.

Usage:

```
namespace my_app {
    [[=hpx::actions::detail::remote_function{}]]
    int compute(int x) { return x + 1; }
}
HPX_REGISTER_ANNOTATED_ACTIONS(my_app)

\warning This macro must be invoked after all annotated function
definitions in \a Ns are complete. Since C++ namespaces are open,
functions added to \a Ns after this macro is expanded will NOT be
discovered or registered.

// Then dispatch via:
hpx::async
↪<hpx::actions::reflect_action<^^my_app::compute>>(id, 42);
```

HPX_REGISTER_ACTION_DECLARATION

Defined in header hpx/components.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *HPX_REGISTER_ACTION*

Warning

doxygenfunction: Cannot find function "HPX_REGISTER_ACTION_DECLARATION" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

HPX_REGISTER_ACTION

Defined in header hpx/components.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *HPX_REGISTER_ACTION_DECLARATION*

Warning

doxygenfunction: Cannot find function "HPX_REGISTER_ACTION" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx/actions_base/preassigned_action_id.hpp

Defined in header hpx/actions_base/preassigned_action_id.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **actions**

hpx/actions_base/plain_action.hpp

Defined in header hpx/actions_base/plain_action.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **actions**

namespace **traits**

HPX_DEFINE_COMPONENT_ACTION

Defined in header hpx/components.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Warning

doxygenfunction: Unable to resolve function "HPX_DEFINE_COMPONENT_ACTION" with arguments "None". Candidate function could not be parsed. Parsing error is Error when parsing function declaration. If the function has no return type: Invalid C++ declaration: Expected end of definition or ;. [error at 87] HPX_DEFINE_COMPONENT_ACTION (action_move_semantics, test_movable, test_movable_action) HPX_DEFINE_COMPONENT_ACTION(action_move_semantics

————————————————————————————^ If the function has a return type: Error in declarator If declarator-id with parameters-and-qualifiers: Invalid C++ declaration: Expected identifier in nested name. [error at 28] HPX_DEFINE_COMPONENT_ACTION (action_move_semantics, test_movable, test_movable_action) HPX_DEFINE_COMPONENT_ACTION(action_move_semantics ——————————-^ If parenthesis in noptr-declarator: Error in declarator or parameters-and-qualifiers If pointer to member declarator: Invalid C++ declaration: Expected '::' in pointer to member (function). [error at 50] HPX_DEFINE_COMPONENT_ACTION (action_move_semantics, test_movable, test_movable_action) HPX_DEFINE_COMPONENT_ACTION(action_move_semantics ——————————————————^ If declarator-id: Invalid C++ declaration: Expecting "(" in parameters-and-qualifiers. [error at 50] HPX_DEFINE_COMPONENT_ACTION (action_move_semantics, test_movable, test_movable_action) HPX_DEFINE_COMPONENT_ACTION(action_move_semantics ——————————————————^

hpx/actions_base/lambda_to_action.hpp

Defined in header hpx/actions_base/lambda_to_action.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/actions_base/reflect_action.hpp

Defined in header hpx/actions_base/reflect_action.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Reflection-based action definition for HPX remote operations.

hpx/actions_base/actions_base_support.hpp

Defined in header hpx/actions_base/actions_base_support.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **actions**

hpx/actions_base/actions_base_fwd.hpp

Defined in header hpx/actions_base/actions_base_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **actions**

hpx/actions_base/traits/action_remote_result.hpp

Defined in header hpx/actions_base/traits/action_remote_result.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **traits**

Typedefs

template<typename **Result**>

using **action_remote_result_t** = typename *action_remote_result*<*Result*>::type

template<typename **Result**>

struct **action_remote_result** : public detail::action_remote_result_customization_point<*Result*>

agas

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/agas/addressing_service.hpp

Defined in header hpx/agas/addressing_service.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **agas**

struct **addressing_service**

Public Types

enum class **resolved_locality_state** : *std*::uint8_t

How a locality became known to this instance, which decides whether hpx::force_disconnect may remove it.

Values:

enumerator **connected**

Started with the application, or learned about by resolving its address. hpx::force_disconnect rejects it.

enumerator **connecting**

Connected after the application started and registered as such here, which only the console does. hpx::force_disconnect accepts it.

enumerator **disconnecting**

Claimed by an hpx::force_disconnect call that is still removing it. Further claims are rejected until the entry is erased.

using **component_id_type** = *components*::component_type

using **iterate_names_return_type** = *std*::map<*std*::string, *hpx*::id_type>

using **iterate_types_function_type** = *hpx*::*function*<void(*std*::string const&, *components*::component_type), true>

using **mutex_type** = *hpx*::*spinlock*

using **gva_cache_type** = *hpx*::*util*::*cache*::*lru_cache*<gva_cache_key, gva, *hpx*::*util*::*cache*::*statistics*::local_full_statistics>

using **migrated_objects_table_type** = *std*::set<*naming*::gid_type>

using **refcnt_requests_type** = *std*::map<*naming*::gid_type, *std*::int64_t>

using **resolved_localities_type** = *std*::map<*naming*::gid_type, *resolved_locality*>

Public Functions

HPX_NON_COPYABLE(*addressing_service*)

explicit **addressing_service**(*util*::*runtime_configuration* const &ini_)

~addressing_service() = default

void **bootstrap**(*parcelset*::endpoints_type const &endpoints, *util*::*runtime_configuration* &rtcfg)

void **initialize**(*std*::uint64_t rts_lva)

void **adjust_local_cache_size**(*std*::size_t) const

Adjust the size of the local AGAS Address resolution cache.

inline state **get_status**() const

inline void **set_status**(state new_state)

inline *naming*::gid_type const &**get_local_locality**(*error_code*& = *throws*) const

void **set_local_locality**(*naming*::gid_type const &g)

void **register_console**(*parcelset*::endpoints_type const &eps)

inline bool **is_bootstrap**() const

inline bool **is_console**() const

Returns whether this *addressing_service* represents the console locality.

inline bool **is_connecting**() const

Returns whether this *addressing_service* is connecting to a running application.

bool **is_connecting**(*hpx*::*naming*::gid_type const &locality) const

Return whether a locality joined while connecting.

Parameters
`locality` – The locality GID to inspect. If locality is invalid, return the connecting status of the calling locality.

Returns
`true` if the locality connected late, also while it is being removed and after it was disconnected, and `false` for a locality that started with the application.

bool **mark_connecting_locality_as_disconnecting**(*hpx*::*naming*::gid_type const &locality)

Atomically claim a connecting locality for disconnection.

Parameters
`locality` – The locality GID to claim.

Returns
`true` if this call changed the locality state from connecting to disconnecting, and `false` otherwise.

bool **mark_disconnecting_locality_as_connecting**(*hpx*::*naming*::gid_type const &locality)

Release a claim made by mark_connecting_locality_as_disconnecting.

Parameters
`locality` – The locality GID to release.

Returns
`true` if this call changed the locality state from disconnecting back to connecting, and `false` otherwise.

bool **resolve_locally_known_addresses**(*naming*::gid_type const &id, *naming*::address &addr) const

void **register_server_instances**()

void **unregister_server_instances**(*hpx*::*error_code* &ec = *hpx*::*throws*)

void **garbage_collect_non_blocking**(*error_code* &ec = *throws*)

void **garbage_collect**(*error_code* &ec = *throws*)

inline *server*::*primary_namespace* &**get_local_primary_namespace_service**()

inline *naming*::address::address_type **get_primary_ns_lva**() const inline *naming*::address::address_type **get_symbol_ns_lva**() const

inline *server*::component_namespace ***get_local_component_namespace_service**() const

inline *server*::locality_namespace ***get_local_locality_namespace_service**() const

inline *server*::symbol_namespace &**get_local_symbol_namespace_service**() const

inline *naming*::address::address_type **get_runtime_support_lva**() const

std::uint64_t **get_cache_entries**(bool) const

std::uint64_t **get_cache_hits**(bool) const

std::uint64_t **get_cache_misses**(bool) const

std::uint64_t **get_cache_evictions**(bool) const

std::uint64_t **get_cache_insertions**(bool) const

std::uint64_t **get_cache_get_entry_count**(bool reset) const

std::uint64_t **get_cache_insertion_entry_count**(bool reset) const

std::uint64_t **get_cache_update_entry_count**(bool reset) const

std::uint64_t **get_cache_erase_entry_count**(bool reset) const *std*::uint64_t **get_cache_get_entry_time**(bool reset) const

std::uint64_t **get_cache_insertion_entry_time**(bool reset) const

std::uint64_t **get_cache_update_entry_time**(bool reset) const

std::uint64_t **get_cache_erase_entry_time**(bool reset) const

bool **register_locality**(*parcelset*::endpoints_type const &endpoints, *naming*::gid_type &prefix, *std*::uint32_t num_threads, bool is_connecting, *error_code* &ec = *throws*)

Add a locality to the runtime.

parcelset::endpoints_type const &**resolve_locality**(*naming*::gid_type const &gid, *error_code* &ec = *throws*)

Resolve a locality to its prefix.

Returns

Returns an empty vector if the locality is not registered.

bool **has_resolved_locality**(*naming*::gid_type const &gid) const

bool **unregister_locality**(*naming*::gid_type const &gid, *error_code* &ec = *throws*)

Remove a locality from the runtime.

void **remove_resolved_locality**(*naming*::gid_type const &gid)

remove given locality from locality cache

bool **get_console_locality**(*naming*::gid_type &locality, *error_code* &ec = *throws*)

Get locality locality_id of the console locality.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Parameters

- **locality** – [out] The locality_id value uniquely identifying the console locality. This is valid only, if the return value of this function is true.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns *true* if a console locality_id exists and returns *false* otherwise.

bool **get_localities**(*std*::vector<*naming*::gid_type> &locality_ids, *components*::component_type type, *error_code* &ec = *throws*) const

Query for the locality_ids of all known localities.

This function returns the locality_ids of all localities known to the AGAS server or all localities having a registered factory for a given component type.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- `locality_ids` – [out] The vector will contain the prefixes of all localities registered with the AGAS server. The returned vector holds the prefixes representing the runtime_support components of these localities.
- `type` – [in] The component type will be used to determine the set of prefixes having a registered factory for this component. The default value for this parameter is *components::component_enum_type::invalid*, which will return prefixes of all localities.
- `ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

inline bool **get_localities**(*std*::vector<*naming*::gid_type> &locality_ids, *error_code* &ec = *throws*) const

hpx::*future*<*std*::uint32_t> **get_num_localities_async**(*components*::component_type type = to_int(*hpx*::*components*::*component_enum_type*::*invalid*)) const

Query for the number of all known localities.

This function returns the number of localities known to the AGAS server or the number of localities having a registered factory for a given component type.

Parameters

`type` – [in] The component type will be used to determine the set of prefixes having a registered factory for this component. The default value for this parameter is *components::component_type::invalid*, which will return prefixes of all localities.

Returns

A future that becomes ready with the number of matching localities.

std::uint32_t **get_num_localities**(*components*::component_type type, *error_code* &ec = *throws*) const

Query for the number of all known localities.

This function returns the number of localities known to the AGAS server or the number of localities having a registered factory for a given component type.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- `type` – [in] The component type will be used to determine the set of prefixes having a registered factory for this component. The default value for this parameter is *components::component_type::invalid*, which will return prefixes of all localities.
- `ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

inline *std*::uint32_t **`get_num_localities`**(*error_code* &ec = *throws*) const

Query for the number of all known localities.

This function returns the number of localities known to the AGAS server or the number of localities having a registered factory for a given component type.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

`ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

hpx::*future*<*std*::uint32_t> **`get_num_overall_threads_async`**() const

std::uint32_t **`get_num_overall_threads`**(*error_code* &ec = *throws*) const

hpx::*future*<*std*::vector<*std*::uint32_t>> **`get_num_threads_async`**() const

std::vector<*std*::uint32_t> **`get_num_threads`**(*error_code* &ec = *throws*) const

components::component_type **`get_component_id`**(*std*::string const &name, *error_code* &ec = *throws*) const

Return a unique id usable as a component type.

This function returns the component type id associated with the given component name. If this is the first request for this component name a new unique id will be created.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Parameters

- `name` – [in] The component name (string) to get the component type for.
- `ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

The function returns the currently associated component type. Any error results in an exception thrown from this function.

void **iterate_types**(*iterate_types_function_type* const &f, *error_code* &ec = *throws*) const

std::string **get_component_type_name**(*components*::component_type id, *error_code* &ec = *throws*) const

inline *components*::component_type **register_factory**(*naming*::gid_type const &locality_id, *std*::string const &name, *error_code* &ec = *throws*) const

Register a factory for a specific component type.

This function allows to register a component factory for a given locality and component type.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Parameters

- `locality_id` – [in] The locality value uniquely identifying the given locality the factory needs to be registered for.
- `name` – [in] The component name (string) to register a factory for the given component type for.
- `ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

The function returns the currently associated component type. Any error results in an exception thrown from this function. The returned component type is the same as if the function *get_component_id* was called using the same component name.

components::component_type **register_factory**(*std*::uint32_t locality_id, *std*::string const &name, *error_code* &ec = *throws*) const

bool **get_id_range**(*std*::uint64_t count, *naming*::gid_type &lower_bound, *naming*::gid_type &upper_bound, *error_code* &ec = *throws*)

Get unique range of freely assignable global ids.

Every locality needs to be able to assign global ids to different components without having to consult the AGAS server for every id to generate. This function can be called to preallocate a range of ids usable for this purpose.

> **Note**
>
> This function assigns a range of global ids usable by the given locality for newly created components. Any of the returned global ids still has to be bound to a local address, either by calling *bind* or *bind_range*.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Parameters

- `count` – [in] The number of global ids to be generated.
- `lower_bound` – [out] The lower bound of the assigned id range. The returned value can be used as the first id to assign. This is valid only, if the return value of this function is true.
- `upper_bound` – [out] The upper bound of the assigned id range. The returned value can be used as the last id to assign. This is valid only, if the return value of this function is true.
- `ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns *true* if a new range has been generated (it has been called for the first time for the given locality) and returns *false* if this locality already got a range assigned in an earlier call. Any error results in an exception thrown from this function.

inline bool **bind_local**(*naming*::gid_type const &id, *naming*::address const &addr, *error_code* &ec = *throws*)

Bind a global address to a local address.

Every element in the HPX namespace has a unique global address (global id). This global address has to be associated with a concrete local address to be able to address an instance of a component using its global address.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

> **Note**
>
> Binding a gid to a local address sets its global reference count to one.

Parameters

- **id** – [in] The global address which has to be bound to the local address.
- **addr** – [in] The local address to be bound to the global address.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns *true*, if this global id got associated with an local address. It returns *false* otherwise.

inline *hpx*::*future*<bool> **bind_async**(*naming*::gid_type const &id, *naming*::address const &addr, *std*::uint32_t locality_id)

inline *hpx*::*future*<bool> **bind_async**(*naming*::gid_type const &id, *naming*::address const &addr, *naming*::gid_type const &locality)

bool **bind_range_local**(*naming*::gid_type const &lower_id, *std*::uint64_t count, *naming*::address const &baseaddr, *std*::uint64_t offset, *error_code* &ec = *throws*)

Bind unique range of global ids to given base address.

Every locality needs to be able to bind global ids to different components without having to consult the AGAS server for every id to bind. This function can be called to bind a range of consecutive global ids to a range of consecutive local addresses (separated by a given *offset*).

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

> **Note**
>
> Binding a gid to a local address sets its global reference count to one.

Parameters

- **lower_id** – [in] The lower bound of the assigned id range. The value can be used as the first id to assign.
- **count** – [in] The number of consecutive global ids to bind starting at *lower_id*.
- **baseaddr** – [in] The local address to bind to the global id given by *lower_id*. This is the base address for all additional local addresses to bind to the remaining global ids.
- **offset** – [in] The offset to use to calculate the local addresses to be bound to the range of global ids.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns *true*, if the given range was successfully bound. It returns *false* otherwise.

hpx::*future*<bool> **bind_range_async**(*naming*::gid_type const &lower_id, *std*::uint64_t count, *naming*::address const &baseaddr, *std*::uint64_t offset, *naming*::gid_type const &locality)

inline *hpx*::*future*<bool> **bind_range_async**(*naming*::gid_type const &lower_id, *std*::uint64_t count, *naming*::address const &baseaddr, *std*::uint64_t offset, *std*::uint32_t locality_id)

inline bool **unbind_local**(*naming*::gid_type const &id, *error_code* &ec = *throws*)

Unbind a global address.

Remove the association of the given global address with any local address, which was bound to this global address. Additionally it returns the local address which was bound at the time of this call.

> **Note**
>
> You can unbind only global ids bound using the function *bind*. Do not use this function to unbind any of the global ids bound using *bind_range*.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

> **Note**
>
> This function will raise an error if the global reference count of the given gid is not zero!

Parameters

- **id** – [in] The global address (id) for which the association has to be removed.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

The function returns *true* if the association has been removed, and it returns *false* if no association existed. Any error results in an exception thrown from this function.

inline bool **unbind_local**(*naming*::gid_type const &id, *naming*::address &addr, *error_code* &ec = *throws*)

Unbind a global address.

Remove the association of the given global address with any local address, which was bound to this global address. Additionally it returns the local address which was bound at the time of this call.

Note

You can unbind only global ids bound using the function *bind*. Do not use this function to unbind any of the global ids bound using *bind_range*.

Note

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Note

This function will raise an error if the global reference count of the given gid is not zero!

Parameters

- **id** – [in] The global address (id) for which the association has to be removed.
- **addr** – [out] The local address which was associated with the given global address (id). This is valid only if the return value of this function is true.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

The function returns *true* if the association has been removed, and it returns *false* if no association existed. Any error results in an exception thrown from this function.

inline bool **unbind_range_local**(*naming*::gid_type const &lower_id, *std*::uint64_t count, *error_code* &ec = *throws*)

Unbind the given range of global ids.

Note

You can unbind only global ids bound using the function *bind_range*. Do not use this function to unbind any of the global ids bound using *bind*.

Note

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Note

This function will raise an error if the global reference count of the given gid is not zero!

Parameters

- **lower_id** – [in] The lower bound of the assigned id range. The value must the first id of the range as specified to the corresponding call to *bind_range*.

- **count** – [in] The number of consecutive global ids to unbind starting at *lower_id*. This number must be identical to the number of global ids bound by the corresponding call to *bind_range*
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns *true* if a new range has been generated (it has been called for the first time for the given locality) and returns *false* if this locality already got a range assigned in an earlier call. Any error results in an exception thrown from this function.

bool **unbind_range_local**(*naming*::gid_type const &lower_id, *std*::uint64_t count, *naming*::address &addr, *error_code* &ec = *throws*)

Unbind the given range of global ids.

> **Note**
>
> You can unbind only global ids bound using the function *bind_range*. Do not use this function to unbind any of the global ids bound using *bind*.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

> **Note**
>
> This function will raise an error if the global reference count of the given gid is not zero!

Parameters

- **lower_id** – [in] The lower bound of the assigned id range. The value must the first id of the range as specified to the corresponding call to *bind_range*.
- **count** – [in] The number of consecutive global ids to unbind starting at *lower_id*. This number must be identical to the number of global ids bound by the corresponding call to *bind_range*
- **addr** – [out] The local address which was associated with the given global address (id). This is valid only if the return value of this function is true.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns *true* if a new range has been generated (it has been called for the first time for the given locality) and returns *false* if this locality already got a range assigned in an earlier call.

hpx::*future*<*naming*::address> **unbind_range_async**(*naming*::gid_type const &lower_id, *std*::uint64_t count = 1)

inline bool **is_local_address_cached**(*naming*::gid_type const &id, *error_code* &ec = *throws*)

Test whether the given address refers to a local object.

This function will test whether the given address refers to an object living on the locality of the caller.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Parameters

- **id** – [in] The address to test.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns *true* if the passed address refers to an object which lives on the locality of the caller.

bool **is_local_address_cached**(*naming*::gid_type const &id, *naming*::address &addr, *error_code* &ec = *throws*)

bool **is_local_address_cached**(*naming*::gid_type const &id, *naming*::address &addr, *std*::pair<bool, *components*::pinned_ptr> &r, *hpx*::*move_only_function*<*std*::pair<bool, *components*::pinned_ptr>(*naming*::address const&)> &&f, *error_code* &ec = *throws*)

bool **is_local_lva_encoded_address**(*std*::uint64_t msb) const

inline bool **resolve_local**(*naming*::gid_type const &id, *naming*::address &addr, *error_code* &ec = *throws*)

Resolve a given global address (*id*) to its associated local address.

This function returns the local address which is currently associated with the given global address (*id*).

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Parameters

- **id** – [in] The global address (*id*) for which the associated local address should be returned.
- **addr** – [out] The local address which currently is associated with the given global address (id), this is valid only if the return value of this function is true.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns *true* if the global address has been resolved successfully (there exists an association to a local address) and the associated local address has been returned. The function returns *false* if no association exists for the given global address. Any error results in an exception thrown from this function.

inline bool **resolve_local**(*hpx*::id_type const &id, *naming*::address &addr, *error_code* &ec = *throws*)

inline *naming*::address **resolve_local**(*naming*::gid_type const &id, *error_code* &ec = *throws*)

inline *naming*::address **resolve_local**(*hpx*::id_type const &id, *error_code* &ec = *throws*)

hpx::future_or_value<*naming*::address> **resolve_async**(*naming*::gid_type const &id)

inline *hpx*::future_or_value<*naming*::address> **resolve_async**(*hpx*::id_type const &id)

hpx::future_or_value<id_type> **get_colocation_id_async**(*hpx*::id_type const &id)

bool **resolve_full_local**(*naming*::gid_type const &id, *naming*::address &addr, *error_code* &ec = *throws*)

inline bool **resolve_full_local**(*hpx*::id_type const &id, *naming*::address &addr, *error_code* &ec = *throws*)

inline *naming*::address **resolve_full_local**(*naming*::gid_type const &id, *error_code* &ec = *throws*)

inline *naming*::address **resolve_full_local**(*hpx*::id_type const &id, *error_code* &ec = *throws*)

hpx::future_or_value<*naming*::address> **resolve_full_async**(*naming*::gid_type const &id)

inline *hpx*::future_or_value<*naming*::address> **resolve_full_async**(*hpx*::id_type const &id)

bool **resolve_cached**(*naming*::gid_type const &id, *naming*::address &addr, *error_code* &ec = *throws*) const

inline bool **resolve_cached**(*hpx*::id_type const &id, *naming*::address &addr, *error_code* &ec = *throws*)

inline bool **resolve_local**(*naming*::gid_type const *gids, *naming*::address *addrs, *std*::size_t size, *hpx*::detail::dynamic_bitset<> &locals, *error_code* &ec = *throws*)

bool **resolve_full_local**(*naming*::gid_type const *gids, *naming*::address *addrs, *std*::size_t size, *hpx*::detail::dynamic_bitset<> &locals, *error_code* &ec = *throws*)

bool **resolve_cached**(*naming*::gid_type const *gids, *naming*::address *addrs, *std*::size_t size, *hpx*::detail::dynamic_bitset<> &locals, *error_code* &ec = *throws*) const

hpx::future_or_value<*std*::int64_t> **incref_async**(*naming*::gid_type const &gid, *std*::int64_t credits = 1, *hpx*::id_type const &keep_alive = *hpx*::invalid_id)

Increment the global reference count for the given id.

Note

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Parameters

- **gid** – [in] The global address (id) for which the global reference count has to be incremented.
- **credits** – [in] The number of reference counts to add for the given id.
- **keep_alive** – [in] Id to keep alive (if valid)

Returns

Whether the operation was successful.

inline *std*::int64_t **incref**(*naming*::gid_type const &gid, *std*::int64_t credits = 1, *error_code* &ec = *throws*)

void **decref**(*naming*::gid_type const &id, *std*::int64_t credits = 1, *error_code* &ec = *throws*)

Decrement the global reference count for the given id.

Note

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Parameters

- **id** – [in] The global address (id) for which the global reference count has to be decremented.
- **credits** – [in] The number of reference counts to add for the given id.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns
The global reference count after the decrement.

hpx::*future*<*iterate_names_return_type*> **iterate_ids**(*std*::string const &pattern) const

Invoke the supplied *hpx::function* for every registered global name.

This function iterates over all registered global ids and returns every found entry matching the given name pattern. Any error results in an exception thrown (or reported) from this function.

Parameters
pattern – [in] pattern (possibly using wildcards) to match all existing entries against

bool **register_name**(*std*::string const &name, *naming*::gid_type const &id, *error_code* &ec = *throws*) const

Register a global name with a global address (id)

This function registers an association between a global name (string) and a global address (id) usable with one of the functions above (bind, unbind, and resolve).

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Parameters

- **name** – [in] The global name (string) to be associated with the global address.
- **id** – [in] The global address (id) to be associated with the global address.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns
The function returns *true* if the global name was registered. It returns false if the global name is not registered.

hpx::*future*<bool> **register_name_async**(*std*::string const &name, *hpx*::id_type const &id) const

bool **register_name**(*std*::string const &name, *hpx*::id_type const &id, *error_code* &ec = *throws*) const

hpx::*future*<*hpx*::id_type> **unregister_name_async**(*std*::string const &name) const

Unregister a global name (release any existing association).

This function releases any existing association of the given global name with a global address (id).

Parameters

name – [in] The global name (string) for which any association with a global address (id) has to be released.

Returns

A future that becomes ready with the global id that was associated with the given name.

hpx::id_type **unregister_name**(*std*::string const &name, *error_code* &ec = *throws*) const

Unregister a global name (release any existing association)

This function releases any existing association of the given global name with a global address (id).

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Parameters

- **name** – [in] The global name (string) for which any association with a global address (id) has to be released.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

The global id that was associated with the given name. Any error results in an exception thrown from this function.

hpx::*future*<*hpx*::id_type> **resolve_name_async**(*std*::string const &name) const

Query for the global address associated with a given global name.

This function returns the global address associated with the given global name.

Parameters

name – [in] The global name (string) for which the currently associated global address has to be retrieved.

Returns

A future holding the global id currently associated with the given global name.

hpx::id_type **resolve_name**(*std*::string const &name, *error_code* &ec = *throws*) const

Query for the global address associated with a given global name.

This function returns the global address associated with the given global name.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Parameters

- **name** – [in] The global name (string) for which the currently associated global address has to be retrieved.

- **ec** – [in,out] This represents the error status on exit. If this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

The global id currently associated with the given global name.

future<*hpx*::id_type> **on_symbol_namespace_event**(*std*::string const &name, bool call_for_past_events = false) const

Install a listener for a given symbol namespace event.

This function installs a listener for a given symbol namespace event. It returns a future which becomes ready as a result of the listener being triggered.

> **Note**
>
> The only event type which is currently supported is *symbol_ns_bind*, i.e. the listener is triggered whenever a global id is registered with the given name.

Parameters

- **name** – [in] The global name (string) for which the given event should be triggered.
- **call_for_past_events** – [in, optional] Trigger the listener even if the given event has already happened in the past. The default for this parameter is *false*.

Returns

A future instance encapsulating the global id which is causing the registered listener to be triggered.

void **update_cache_entry**(*naming*::gid_type const &gid, *gva* const &gva, *error_code* &ec = *throws*)

> **Warning**
>
> This function is for internal use only. It is dangerous and may break your code if you use it.

inline void **update_cache_entry**(*naming*::gid_type const &gid, *naming*::address const &addr, *std*::uint64_t count = 0, *std*::uint64_t offset = 0, *error_code* &ec = *throws*)

> **Warning**
>
> This function is for internal use only. It is dangerous and may break your code if you use it.

bool **get_cache_entry**(*naming*::gid_type const &gid, *gva* &gva, *naming*::gid_type &idbase, *error_code* &ec = *throws*) const

Warning

This function is for internal use only. It is dangerous and may break your code if you use it.

void **remove_cache_entry**(*naming*::gid_type const &id, *error_code* &ec = *throws*) const

Warning

This function is for internal use only. It is dangerous and may break your code if you use it.

void **clear_cache**(*error_code* &ec = *throws*) const

Warning

This function is for internal use only. It is dangerous and may break your code if you use it.

void **start_shutdown**(*error_code* &ec = *throws*)

hpx::*future*<*std*::pair<*hpx*::id_type, *naming*::address>> **begin_migration**(*hpx*::id_type const &id)

start/stop migration of an object

Returns

Current locality and address of the object to migrate

bool **end_migration**(*hpx*::id_type const &id)

std::pair<bool, *components*::pinned_ptr> **was_object_migrated**(*naming*::gid_type const &gid, *hpx*::*move_only_function*<*components*::pinned_ptr()> &&f) const

Maintain list of migrated objects.

hpx::*future*<void> **mark_as_migrated**(*naming*::gid_type const &gid, *hpx*::*move_only_function*<*std*::pair<bool, *hpx*::*future*<void>>()> &&f, bool expect_to_be_marked_as_migrating)

Mark the given object as being migrated (if the object is unpinned). Delay migration until the object is unpinned otherwise.

void **unmark_as_migrated**(*naming*::gid_type const &gid_, *hpx*::*move_only_function*<void()> &&f)

Remove the given object from the table of migrated objects.

void **pre_cache_endpoints**(*std*::vector<*parcelset*::endpoints_type> const&)

Public Members

mutable *hpx*::*shared_mutex* **gva_cache_mtx_**

std::shared_ptr<*gva_cache_type*> **gva_cache_**

mutable *mutex_type* **migrated_objects_mtx_** = *mutex_type*("addressing_service::migrated_objects_mtx")

migrated_objects_table_type **migrated_objects_table_**

mutable *mutex_type* **console_cache_mtx_** = *mutex_type*("addressing_service::console_cache_mtx")

std::uint32_t **console_cache_**

std::size_t const **max_refcnt_requests_**

mutex_type **refcnt_requests_mtx_** = *mutex_type*("addressing_service::refcnt_requests_mtx")

std::size_t **refcnt_requests_count_**

bool **enable_refcnt_caching_**

std::shared_ptr<*refcnt_requests_type*> **refcnt_requests_** service_mode const **service_type**

runtime_mode const **runtime_type**

bool const **caching_**

bool const **range_caching_**

threads::*thread_priority* const **action_priority_**

std::uint64_t **rts_lva_**

std::unique_ptr<component_namespace> **component_ns_**

std::unique_ptr<locality_namespace> **locality_ns_**

symbol_namespace **symbol_ns_**

primary_namespace **primary_ns_**

std::atomic<*hpx*::state> **state_**

naming::gid_type **locality_**

mutable *hpx*::*shared_mutex* **resolved_localities_mtx_**

resolved_localities_type **resolved_localities_**

Public Static Functions

static *std*::int64_t **synchronize_with_async_incref**(*std*::int64_t old_credit, *hpx*::id_type const &id, *std*::int64_t compensated_credit)

Protected Functions

void **launch_bootstrap**(*parcelset*::endpoints_type const &endpoints, *util*::*runtime_configuration* &rtcfg)

naming::address **resolve_full_postproc**(*naming*::gid_type const &id, primary_namespace::resolved_type const&)

bool **bind_postproc**(*naming*::gid_type const &id, gva const &g, *future*<bool> f)

bool **was_object_migrated_locked**(*naming*::gid_type const &id) const

Maintain list of migrated objects.

Private Functions

bool **transition_resolved_locality**(*hpx*::*naming*::gid_type const &locality, *resolved_locality_state* from, *resolved_locality_state* to)

Move a locality from one state to another under the resolved localities lock. Returns `false` if it is unknown or not in *from*.

void **send_refcnt_requests**(*std*::unique_lock<*mutex_type*> &l, *error_code* &ec = *throws*)

Assumes that *refcnt_requests_mtx_* is locked.

void **send_refcnt_requests_non_blocking**(*std*::unique_lock<*mutex_type*> &l, *error_code* &ec)

Assumes that *refcnt_requests_mtx_* is locked.

std::vector<*hpx*::*future*<*std*::vector<*std*::int64_t>>> **send_refcnt_requests_async**(*std*::unique_lock<*mutex_type*> &l)

Assumes that *refcnt_requests_mtx_* is locked.

void **send_refcnt_requests_sync**(*std*::unique_lock<*mutex_type*> &l, *error_code* &ec)

Assumes that *refcnt_requests_mtx_* is locked.

struct **resolved_locality**

#include <addressing_service.hpp> What this instance knows about one locality.

Public Members

parcelset::endpoints_type **endpoints**

resolved_locality_state **state**

agas_base

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/agas_base/server/primary_namespace.hpp

Defined in header hpx/agas_base/server/primary_namespace.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Variables

HPX_ACTION_USES_MEDIUM_STACK(hpx::agas::server::primary_namespace::allocate_action) HPX_REGISTER_ACTION_DECL hpx::naming::address > std_pair_address_id_type

namespace **hpx**

namespace **agas**

Functions

naming::gid_type **bootstrap_primary_namespace_gid**()

hpx::id_type **bootstrap_primary_namespace_id**()

namespace **server**

AGAS's primary namespace maps 128-bit global identifiers (GIDs) to resolved addresses.

The following is the canonical description of the partitioning of AGAS's primary namespace.

```
|-----MSB------||------LSB-----|
BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB
|prefix||RC||----identifier----|

MSB        - Most significant bits (bit 64 to bit 127)
LSB        - Least significant bits (bit 0 to bit 63)
prefix     - Highest 32 bits (bit 96 to bit 127) of the MSB. Each
             locality is assigned a prefix. This creates a 96-bit
             address space for each locality.
RC      ␣
↪   - Bit 88 to bit 92 of the MSB. This is the log2 of the number
             of reference counting credits on the GID.
             Bit 93 is used by the locking scheme for gid_types.
        ␣
↪     Bit 94 is a flag which is set if the credit value is valid.
    ␣
↪        Bit 95 is a flag that is set if a GID's credit count is
             ever split (e.g. if the GID is ever passed to another
             locality).
    ␣
↪       - Bit 87 marks the gid such that it will not be stored in
             any of the AGAS caches. This is used mainly for ids
             which represent 'one-shot' objects (like promises).
identifier - Bit 64 to bit 86 of the MSB, and the entire LSB. The
   ␣
↪         content of these bits depends on the component type of
    ␣
↪        the underlying object. For all user-defined components,
             these bits contain a unique 88-bit number which is
             assigned sequentially for each locality. For
             \a hpx#components#component_enum_type#runtime_support
     ␣
↪       the high 24 bits are zeroed and the low 64 bits hold the
             LVA of the component.
```

The following address ranges are reserved. Some are either explicitly or implicitly protected by AGAS. The letter x represents a single-byte wild card.

```
00000000xxxxxxxxxxxxxxxxxxxxxxxx
    Historically unused address space reserved for future use.
xxxxxxxxxxxx0000xxxxxxxxxxxxxxxx
    Address space for LVA-encoded GIDs.
00000001xxxxxxxxxxxxxxxxxxxxxxxx
    Prefix of the bootstrap AGAS locality.
00000001000000010000000000000001
    ␣
↪Address of the primary_namespace component on the bootstrap AGAS
    locality.
00000001000000010000000000000002
    Address␣
↪of the component_namespace component on the bootstrap AGAS
    locality.
00000001000000010000000000000003
```

(continues on next page)

(continued from previous page)

```
 ␣
↪ Address of the symbol_namespace component on the bootstrap AGAS
   locality.
0000000100000001000000000000004
   Address␣
↪of the locality_namespace component on the bootstrap AGAS
   locality.
```

Note

The layout of the address space is implementation defined, and subject to change. Never write application code that relies on the internal layout of GIDs. AGAS only guarantees that all assigned GIDs will be unique.

Variables

constexpr char const *const **primary_namespace_service_name** = "primary/"

struct **primary_namespace** : public *components*::*fixed_component_base*<*primary_namespace*>

Public Types

using **mutex_type** = *hpx*::*spinlock*

using **base_type** = *components*::*fixed_component_base*<*primary_namespace*>

using **component_type** = *std*::int32_t

using **gva_table_data_type** = *std*::pair<gva, *naming*::gid_type>

using **gva_table_type** = *std*::map<*naming*::gid_type, *gva_table_data_type*>

using **refcnt_table_type** = *std*::map<*naming*::gid_type, *std*::int64_t>

using **resolved_type** = *hpx*::*tuple*<*naming*::gid_type, gva, *naming*::gid_type>

Public Functions

inline *mutex_type* &**mutex**()

void **wait_for_migration_locked**(*std*::unique_lock<*mutex_type*> &l, *naming*::gid_type const &id, *error_code* &ec)

inline **primary_namespace**()

void **finalize**() const

inline void **set_local_locality**(*naming*::gid_type const &g)

void **register_server_instance**(char const *servicename, *std*::uint32_t locality_id = *naming*::invalid_locality_id, *error_code* &ec = *throws*)

void **unregister_server_instance**(*error_code* &ec = *throws*) const

bool **bind_gid**(gva const &g, *naming*::gid_type id, *naming*::gid_type const &locality)

std::pair<*hpx*::id_type, *naming*::address> **begin_migration**(*naming*::gid_type id)

bool **end_migration**(*naming*::gid_type const &id)

resolved_type **resolve_gid**(*naming*::gid_type const &id)

hpx::id_type **colocate**(*naming*::gid_type const &id)

naming::address **unbind_gid**(*std*::uint64_t count, *naming*::gid_type id)

std::int64_t **increment_credit**(*std*::int64_t credits, *naming*::gid_type lower, *naming*::gid_type upper)

std::vector<*std*::int64_t> **decrement_credit**(*std*::vector<*hpx*::*tuple*<*std*::int64_t, *naming*::gid_type, *naming*::gid_type>> const &requests)

std::pair<*naming*::gid_type, *naming*::gid_type> **allocate**(*std*::uint64_t count)

resolved_type **resolve_gid_locked**(*std*::unique_lock<*mutex_type*> &l, *naming*::gid_type const &gid, *error_code* &ec)

Public Members

counter_data **counter_data_**

Private Types

using **migration_table_type** = *std*::map<*naming*::gid_type, *hpx*::*tuple*<bool, *std*::size_t, *lcos*::*local*::detail::condition_variable>>

using **free_entry_allocator_type** = *util*::internal_allocator<*free_entry*>

using **free_entry_list_type** = *std*::list<*free_entry*, *free_entry_allocator_type*>

Private Functions

resolved_type **resolve_gid_locked_non_local**(*std*::unique_lock<*mutex_type*> &l, *naming*::gid_type const &gid, *error_code* &ec)

void **increment**(*naming*::gid_type const &lower, *naming*::gid_type const &upper, *std*::int64_t const &credits, *error_code* &ec)

void **resolve_free_list**(*std*::unique_lock<*mutex_type*> &l, *std*::list<*refcnt_table_type*::iterator> const &free_list, *free_entry_list_type* &free_entry_list, *naming*::gid_type const &lower, *naming*::gid_type const &upper, *error_code* &ec)

void **decrement_sweep**(*free_entry_list_type* &free_list, *naming*::gid_type const &lower, *naming*::gid_type const &upper, *std*::int64_t credits, *error_code* &ec)

void **free_components_sync**(*free_entry_list_type* const &free_list, *naming*::gid_type const &lower, *naming*::gid_type const &upper, *error_code* &ec) const

Private Members

mutex_type **mutex_** = *mutex_type*("primary_namespace")

gva_table_type **gvas_**

refcnt_table_type **refcnts_**

std::string **instance_name_**

naming::gid_type **next_id_**

naming::gid_type **locality_**

migration_table_type **migrating_objects_**

struct **counter_data**

Public Functions

counter_data(*counter_data* const&) = delete

counter_data(*counter_data*&&) = delete

counter_data &**operator=**(*counter_data* const&) = delete

counter_data &**operator=**(*counter_data*&&) = delete

counter_data() = default

std::int64_t **get_bind_gid_count**(bool)

std::int64_t **get_resolve_gid_count**(bool)

std::int64_t **get_unbind_gid_count**(bool)

std::int64_t **get_increment_credit_count**(bool)

std::int64_t **get_decrement_credit_count**(bool)

std::int64_t **get_allocate_count**(bool)

std::int64_t **get_begin_migration_count**(bool)

std::int64_t **get_end_migration_count**(bool)

std::int64_t **get_overall_count**(bool) *std*::int64_t **get_bind_gid_time**(bool)

std::int64_t **get_resolve_gid_time**(bool)

std::int64_t **get_unbind_gid_time**(bool)

std::int64_t **get_increment_credit_time**(bool)

std::int64_t **get_decrement_credit_time**(bool)

std::int64_t **get_allocate_time**(bool)

std::int64_t **get_begin_migration_time**(bool)

std::int64_t **get_end_migration_time**(bool)

std::int64_t **get_overall_time**(bool)

void **increment_bind_gid_count**()

void **increment_resolve_gid_count**()

void **increment_unbind_gid_count**()

void **increment_increment_credit_count**()

void **increment_decrement_credit_count**()

void **increment_allocate_count**()

void **increment_begin_migration_count**()

void **increment_end_migration_count**()

void **enable_all**()

Public Members

api_counter_data **bind_gid_**

api_counter_data **resolve_gid_**

api_counter_data **unbind_gid_**

api_counter_data **increment_credit_**

api_counter_data **decrement_credit_**

api_counter_data **allocate_**

api_counter_data **begin_migration_**

api_counter_data **end_migration_**

struct **api_counter_data**

Public Members

std::atomic<*std*::int64_t> **count_** = 0

std::atomic<*std*::int64_t> **time_** = 0

bool **enabled_** = false

struct **free_entry**

Public Functions

inline **free_entry**(*agas*::gva const &gva, *naming*::gid_type const &gid, *naming*::gid_type const &loc)

Public Members

agas::gva **gva_**

naming::gid_type **gid_**

naming::gid_type **locality_**

async_colocated

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::get_colocation_id

Defined in header hpx/runtime.hpp[583].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

[583] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

hpx::id_type **hpx::get_colocation_id**(*launch*::sync_policy, *hpx*::id_type const &id, *error_code* &ec = *throws*)

Return the id of the locality where the object referenced by the given id is currently located on.

The function *hpx::get_colocation_id()* returns the id of the locality where the given object is currently located.

See also

hpx::get_colocation_id()

Note

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of *hpx::exception*.

Parameters

- **id** – [in] The id of the object to locate.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

hpx::*future*<*hpx*::id_type> **hpx::get_colocation_id**(*hpx*::id_type const &id)

Asynchronously return the id of the locality where the object referenced by the given id is currently located on.

See also

hpx::get_colocation_id(launch::sync_policy)

Parameters

id – [in] The id of the object to locate.

async_distributed

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/async_distributed/trigger.hpp

Defined in header hpx/async_distributed/trigger.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **actions**

Functions

template<typename **Result**, typename **RemoteResult**, typename **F**, typename ...**Ts**>

void **trigger**(*typed_continuation*<*Result*, *RemoteResult*> &&cont, *F* &&f, *Ts*&&... vs) noexcept

Invoke *f* with the given arguments and forward its result to *cont*.

On success, the result of invoking *f* is passed to *cont.trigger_value()*. If invoking *f* throws, the exception is forwarded to the destination instead by calling *cont.trigger_error()* so that hpx::exceptions are propagated back to the client.

Parameters

- **cont** – The continuation that either the result or an exception is delivered to.
- **f** – The callable to invoke.
- **vs** – The arguments to invoke *f* with.

template<typename **Result**, typename **F**, typename ...**Ts**>

void **trigger**(*typed_continuation*<*Result*, *util*::unused_type> &&cont, *F* &&f, *Ts*&&... vs) noexcept

Invoke *f* with the given arguments and notify *cont* of completion. Overload for when the return type is "void" aka util::unused_type.

On success, *cont.trigger()* is called to signal completion. If invoking *f* throws, the exception is forwarded to the destination instead by calling *cont.trigger_error()* so that hpx::exceptions are propagated back to the client.

Parameters

- **cont** – The continuation that completion or an exception is delivered to.
- **f** – The callable to invoke.
- **vs** – The arguments to invoke *f* with.

hpx::sync (distributed)

Defined in header hpx/async.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also

hpx::sync

template<typename **Action**, typename **F**, typename ...**Ts**>

auto *hpx*::**sync**(*F* &&f, *Ts*&&... ts) -> decltype(detail::sync_action_dispatch<*Action*, *std*::decay_t<*F*>>::call(*std*::forward<*F*>(*f*), *std*::forward<*Ts*>(*ts*)...))

The function template *sync* runs the function *f* synchronously and returns a hpx::future that will eventually hold the result of that function call.

template<typename **Action**, typename **F**, typename ...**Ts**>

auto *hpx*::**sync**(*F* &&f, *Ts*&&... ts) -> decltype(detail::sync_action_dispatch<*Action*, *std*::decay_t<*F*>>::call(*std*::forward<*F*>(*f*), *std*::forward<*Ts*>(*ts*)...))

The function template *sync* runs the function *f* synchronously and returns a hpx::future that will eventually hold the result of that function call.

Warning

doxygenfunction: Unable to resolve function "hpx::sync" with arguments (Action&&, Target&&, Ts&&…) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename
↪Action, typename F, typename ...Ts> auto sync(F &&f, Ts&&...
↪ts) -> decltype(detail::sync_action_dispatch<Action, std::decay_
↪t<F>>::call(std::forward<F>(f), std::forward<Ts>(ts)...))
- template<typename
↪Action, typename Target, typename ...Ts> decltype(auto)
↪sync(Action &&action, Target &&target, Ts&&... ts)
- template<typename
↪F, typename ...Ts> decltype(auto) sync(F &&f, Ts&&... ts)
```

hpx/async_distributed/promise.hpp

Defined in header hpx/async_distributed/promise.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **distributed**

Functions

template<typename **Result**, typename **RemoteResult**>

void **swap**(*promise*<*Result*, *RemoteResult*> &x, *promise*<*Result*, *RemoteResult*> &y) noexcept

template<typename **Result**, typename **RemoteResult**>

class **promise** : public *lcos*::detail::promise_base<*Result*, *RemoteResult*, *lcos*::detail::promise_data<*Result*>>

#include <promise.hpp> A promise can be used by a single *thread* to invoke a (remote) action and wait for the result. The result is expected to be sent back to the promise using the LCO's set_event action

A promise is one of the simplest synchronization primitives provided by HPX. It allows to synchronize on a eager evaluated remote operation returning a result of the type *Result*. The *promise* allows to synchronize exactly one *thread* (the one passed during construction time).

```
// Create the promise (the expected result is a id_type)
hpx::distributed::promise<hpx::id_type> p;

// Get the associated future
future<hpx::id_type> f = p.get_future();

// initiate the action supplying the promise as a
// continuation
apply<some_action>(new continuation(p.get_id()), ...);

// Wait for the result to be returned, yielding control
// in the meantime.
hpx::id_type result = f.get();
// ...
```

> **Note**
>
> The action executed by the promise must return a value of a type convertible to the type as specified by the template parameter *RemoteResult*

Template Parameters

- **Result** – The template parameter *Result* defines the type this promise is expected to return from *promise::get*.
- **RemoteResult** – The template parameter *RemoteResult* defines the type this promise is expected to receive from the remote action.

Public Functions

promise() = default

constructs a promise object and a shared state.

template<typename **Allocator**>

inline **promise**(*std*::allocator_arg_t, *Allocator* const &a)

constructs a promise object and a shared state. The constructor uses the allocator *to* allocate the memory for the shared state.

promise(*promise* &&other) noexcept = default

constructs a new promise object and transfers ownership of the shared state of other (if any) to the newly- constructed object.

Post

other has no shared state.

~promise() = default

Abandons any shared state.

promise &**operator=**(*promise* &&other) noexcept = default

Abandons any shared state (30.6.4) and then as if promise(HPX_MOVE(other)).swap(*this).

Returns

*this.

inline void **swap**(*promise* &other) noexcept

Exchanges the shared state of *this and other.

Post

*this has the shared state (if any) that other had prior to the call to swap. other has the shared state (if any) that *this had prior to the call to swap.

Private Types

using **base_type** = *lcos*::detail::promise_base<*Result*, *RemoteResult*, *lcos*::detail::promise_data<*Result*>>

template<>

class **promise**<void, *hpx*::*util*::unused_type> : public *lcos*::detail::promise_base<void, *hpx*::*util*::unused_type, *lcos*::detail::promise_data<void>>

Public Functions

promise() = default

constructs a promise object and a shared state.

template<typename **Allocator**>

inline **promise**(*std*::allocator_arg_t, *Allocator* const &a)

constructs a promise object and a shared state. The constructor uses the allocator *to* allocate the memory for the shared state.

promise(*promise* &&other) noexcept = default

constructs a new promise object and transfers ownership of the shared state of other (if any) to the newly- constructed object.

Post

other has no shared state.

~promise() = default

Abandons any shared state.

promise &**operator=**(*promise* &&other) noexcept = default

Abandons any shared state (30.6.4) and then as if promise(HPX_MOVE(other)).swap(*this).

Returns

*this.

inline void **swap**(*promise* &other) noexcept

Exchanges the shared state of *this and other.

Post

*this has the shared state (if any) that other had prior to the call to swap. other has the shared state (if any) that *this had prior to the call to swap.

inline void **set_value**()

atomically stores the value r in the shared state and makes that state ready (30.6.4).

Throws

`future_error` – if its shared state already has a stored value. if shared state has no stored value exception is raised. promise_already_satisfied if its shared state already has a stored value or exception. no_state if *this has no shared state.

Private Types

using **base_type** = *lcos*::detail::promise_base<void, *hpx*::*util*::unused_type, *lcos*::detail::promise_data<void>>

namespace **std**

template<typename **R**, typename **Allocator**>

struct **uses_allocator**<*hpx*::*distributed*::*promise*<*R*>, *Allocator*> : public *std*::true_type

#include <promise.hpp> Requires: Allocator shall be an allocator (17.6.3.5)

hpx/async_distributed/bind_action.hpp

Defined in header hpx/async_distributed/bind_action.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

template<typename **Action**, typename ...**Ts**>

detail::bound_action<*std*::decay_t<*Action*>, *util*::make_index_pack_t<sizeof...(*Ts*)>, *hpx*::*util*::decay_unwrap_t<*Ts*>...> **bind**(*Ts*&&... vs)

template<typename **Component**, typename **Signature**, typename **Derived**, typename ...**Ts**>

detail::bound_action<*Derived*, *util*::make_index_pack_t<sizeof...(*Ts*)>, *std*::decay_t<*Ts*>...> **bind**(*hpx*::*actions*::basic_action<*Component*, *Signature*, *Derived*> action, *Ts*&&... vs)

namespace **serialization**

Functions

template<typename **Archive**, typename **F**, typename ...**Ts**>

void **serialize**(*Archive* &ar, ::*hpx*::detail::bound_action<*F*, *Ts*...> &bound, unsigned int const version = 0)

hpx::post (distributed)

Defined in header hpx/async.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also

hpx::post

template<typename **F**, typename ...**Ts**>

bool *hpx*::**post**(*F* &&f, *Ts*&&... ts)

Runs the function `f` asynchronously (potentially in a separate thread which might be a part of a thread pool). This is done in a fire-and-forget manner, meaning there is no return value or way to synchronize with the function execution (it does not return an `hpx::future` that would hold the result of that function call).

`hpx::post` is particularly useful when synchronization mechanisms as heavyweight as futures are not desired, and instead, more lightweight mechanisms like latches or atomic variables are preferred. Essentially, the post function enables the launch of a new thread without the overhead of creating a future.

Note

`hpx::post` is similar to `hpx::async` but does not return a future. This is why there is no way of finding out the result/failure of the execution of this function.

template<typename **Action**, typename ...**Ts**>

bool *hpx*::**post**(*hpx*::id_type const &id, *Ts*&&... vs)

template<typename **Action**, typename **Client**, typename **Stub**, typename **Data**, typename ...**Ts**>

bool *hpx*::**post**(*components*::*client_base*<*Client*, *Stub*, *Data*> const &c, *Ts*&&... vs)

template<typename **Action**, typename **DistPolicy**, typename ...**Ts**>

bool *hpx*::**post**(*DistPolicy* const &policy, *Ts*&&... vs)

template<typename **Action**, typename **Continuation**, typename ...**Ts**>

bool *hpx*::**post**(*Continuation* &&c, *hpx*::id_type const &gid, *Ts*&&... vs)

template<typename **Action**, typename **Continuation**, typename **Client**, typename **Stub**, typename **Data**, typename ...**Ts**>

bool *hpx*::**post**(*Continuation* &&cont, *components*::*client_base*<*Client*, *Stub*, *Data*> const &c, *Ts*&&... vs)

template<typename **Action**, typename **Continuation**, typename **DistPolicy**, typename ...**Ts**>

bool *hpx*::**post**(*Continuation* &&c, *DistPolicy* const &policy, *Ts*&&... vs)

template<typename **Action**, typename **Target**, typename ...**Ts**>

bool *hpx*::**post**(*Action* &&action, *Target* &&target, *Ts*&&... ts)

The distributed implementation of `hpx::post` can be used by giving an action instance as argument instead of a function, and also by providing another argument with the locality ID or the target ID.

Template Parameters

- **Action** – The type of action instance
- **Target** – The type of target where the action should be executed
- **Ts** – The type of any additional arguments

Parameters

- **action** – The action instance to be executed
- **target** – The target where the action should be executed
- **ts** – Additional arguments

Returns

`true` if the action was successfully posted, `false` otherwise.

hpx/async_distributed/async_continue_callback.hpp

Defined in header hpx/async_distributed/async_continue_callback.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

template<typename **Action**, typename **Cont**, typename **Callback**, typename ...**Ts**>

hpx::*future*<typename *traits*::promise_local_result<typename detail::result_of_async_continue<*Action*, *Cont*>::type>::type> **async_contin**

template<typename **Component**, typename **Signature**, typename **Derived**, typename **Cont**, typename **Callback**, typename ...**Ts**>

hpx::*future*<typename *traits*::promise_local_result<typename detail::result_of_async_continue<*Derived*, *Cont*>::type>::type> **async_conti**

template<typename **Action**, typename **Cont**, typename **DistPolicy**, typename **Callback**, typename ...**Ts**>

hpx::*future*<typename *traits*::promise_local_result<typename detail::result_of_async_continue<*Action*, *Cont*>::type>::type> **async_contin**

template<typename **Component**, typename **Signature**, typename **Derived**, typename **Cont**, typename **DistPolicy**, typename **Callback**, typename ...**Ts**>

hpx::*future*<typename *traits*::promise_local_result<typename detail::result_of_async_continue<*Derived*, *Cont*>::type>::type> **async_conti**

hpx/async_distributed/transfer_continuation_action.hpp

Defined in header hpx/async_distributed/transfer_continuation_action.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::dataflow (distributed)

Defined in header hpx/async.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also

hpx::dataflow

Warning

doxygenfunction: Unable to resolve function "hpx::dataflow" with arguments (F&&, Ts&&...) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename␣
↪Action, typename Target, typename ...Ts> decltype(auto)␣
↪dataflow(Action &&action, Target &&target, Ts&&... ts)
- template<typename␣
↪F, typename ...Ts> decltype(auto) dataflow(F &&f, Ts&&... ts)
```

Warning

doxygenfunction: Unable to resolve function "hpx::dataflow" with arguments (Action&&, Target&&, Ts&&...) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename␣
↪Action, typename Target, typename ...Ts> decltype(auto)␣
↪dataflow(Action &&action, Target &&target, Ts&&... ts)
- template<typename␣
↪F, typename ...Ts> decltype(auto) dataflow(F &&f, Ts&&... ts)
```

hpx/async_distributed/async_callback.hpp

Defined in header hpx/async_distributed/async_callback.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

template<typename **Action**, typename **F**, typename ...**Ts**>

auto **async_cb**(*F* &&f, *Ts*&&... ts) -> decltype(detail::async_cb_action_dispatch<*Action*, *std*::decay_t<*F*>>::call(*std*::forward<*F*>(*f*), *std*::forward<*Ts*>(*ts*)...))

template<typename **F**, typename ...**Ts**>

auto **async_cb**(*F* &&f, *Ts*&&... ts) -> decltype(detail::async_cb_dispatch<*std*::decay_t<*F*>>::call(*std*::forward<*F*>(*f*), *std*::forward<*Ts*>(*ts*)...))

hpx/async_distributed/async_continue.hpp

Defined in header hpx/async_distributed/async_continue.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

template<typename **Action**, typename **Cont**, typename ...**Ts**>

hpx::*future*<typename *traits*::promise_local_result<typename detail::result_of_async_continue<*Action*, *Cont*>::type>::type> **async_contin**

template<typename **Component**, typename **Signature**, typename **Derived**, typename **Cont**, typename ...**Ts**>

hpx::*future*<typename *traits*::promise_local_result<typename detail::result_of_async_continue<*Derived*, *Cont*>::type>::type> **async_conti** template<typename **Action**, typename **Cont**, typename **DistPolicy**, typename ...**Ts**>

hpx::*future*<typename *traits*::promise_local_result<typename detail::result_of_async_continue<*Action*, *Cont*>::type>::type> **async_contin**

template<typename **Component**, typename **Signature**, typename **Derived**, typename **Cont**, typename **DistPolicy**, typename ...**Ts**>

hpx::*future*<typename *traits*::promise_local_result<typename detail::result_of_async_continue<*Derived*, *Cont*>::type>::type> **async_conti**

hpx/async_distributed/base_lco_with_value.hpp

Defined in header hpx/async_distributed/base_lco_with_value.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

namespace **lcos**

template<typename **Result**, typename **RemoteResult**, typename **ComponentTag**>

class **base_lco_with_value** : public *hpx*::*lcos*::*base_lco*, public *ComponentTag*

#include <base_lco_with_value.hpp> The base_lco_with_value class is the common base class for all LCO's synchronizing on a value. The *RemoteResult* template argument should be set to the type of the argument expected for the set_value action.

Template Parameters

- **RemoteResult** – The type of the result value to be carried back to the LCO instance.
- **ComponentTag** – The tag type representing the type of the component (either component_tag or managed_component_tag).

Public Types

using **wrapping_type** = detail::base_lco_wrapping_type<*ComponentTag*, *base_lco_with_value*>::type

using **base_type_holder** = *base_lco_with_value*

Public Functions

inline void **set_value_nonvirt**(*RemoteResult* &&result)

The *function* set_value_nonvirt is called whenever a *set_value_action* is applied on this LCO instance. This function just forwards to the virtual function *set_value*, which is overloaded by the derived concrete LCO.

Parameters

result – [in] The result value to be transferred from the remote operation back to this LCO instance.

inline *Result* **get_value_nonvirt**()

The *function* get_result_nonvirt is called whenever a *get_result_action* is applied on this LCO instance. This function just forwards to the virtual function *get_result*, which is overloaded by the derived concrete LCO.

HPX_DEFINE_COMPONENT_DIRECT_ACTION (base_lco_with_value, set_value_nonvirt, set_value_action) HPX_DEFINE_COMPONENT_DIRECT_ACTION(base_lco_with_value

The *set_value_action* may be used to trigger any LCO instances while carrying an additional parameter of any type.

RemoteResult is taken by rvalue ref. This allows for perfect forwarding. When the action thread function is created, the values are moved into the called function. If we took it by const lvalue reference, we would disable the possibility to further move the result to the designated destination.

Parameters

RemoteResult – [in] The type of the result to be transferred back to this LCO instance.

The *get_value_action* may be used to query the value this LCO instance exposes as its 'result' value.

Public Members

get_value_nonvirt

Public Static Functions

static inline *components*::component_type **get_component_type**() noexcept

static inline void **set_component_type**(*components*::component_type type)

Protected Types

using **result_type** = *std*::conditional_t<*std*::is_void_v<*Result*>, *util*::unused_type, *Result*>

Protected Functions

~base_lco_with_value() override = default

Destructor, needs to be virtual to allow for clean destruction of derived objects

inline virtual void **set_event**() override

virtual void **set_value**(*RemoteResult* &&result) = 0

virtual *result_type* **get_value**() = 0

inline virtual *result_type* **get_value**(*error_code*&)

template<typename **ComponentTag**>

class **base_lco_with_value**<void, void, *ComponentTag*> : public *hpx*::*lcos*::*base_lco*, public *ComponentTag*

#include <base_lco_with_value.hpp> The base_lco<void> specialization is used whenever the set_event action for a particular LCO doesn't carry any argument.

Template Parameters

void – This specialization expects no result value and is almost completely equivalent to the plain base_lco.

Public Types

using **wrapping_type** = detail::base_lco_wrapping_type<*ComponentTag*, base_lco_with_value>::type

using **base_type_holder** = base_lco_with_value

using **set_value_action** = *base_lco*::set_event_action

Public Functions

inline void **get_value**()

Protected Functions

~base_lco_with_value() override = default

Destructor, needs to be virtual to allow for clean destruction of derived objects

namespace **traits**

hpx/async_distributed/continuation.hpp

Defined in header hpx/async_distributed/continuation.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **actions**

class **continuation**

Subclassed by *hpx::actions::typed_continuation< Result, Result >*, *hpx::actions::typed_continuation< void, util::unused_type >*

Public Types

using **continuation_tag** = void

Public Functions

continuation()

explicit **continuation**(*hpx*::id_type const &id)

explicit **continuation**(*hpx*::id_type &&id)

continuation(*hpx*::id_type const &id, *naming*::address &&addr)

continuation(*hpx*::id_type &&id, *naming*::address &&addr) noexcept

continuation(*continuation* &&o) noexcept

continuation &**operator=**(*continuation* &&o) noexcept

void **trigger_error**(*std*::exception_ptr const &e) const

Deliver an exception to this continuation's destination instead of a value.

Parameters

e – The exception to forward to the destination.

void **trigger_error**(*std*::exception_ptr &&e) const

Deliver an exception to this continuation's destination instead of a value.

Parameters

e – The exception to forward to the destination.

void **serialize**(*hpx*::*serialization*::input_archive &ar, unsigned)

void **serialize**(*hpx*::*serialization*::output_archive &ar, unsigned)

inline constexpr *hpx*::id_type const &**get_id**() const noexcept

inline constexpr *naming*::address **get_addr**() const noexcept

Protected Attributes

hpx::id_type **id_**

naming::address **addr_**

template<typename **Result**>

struct **typed_continuation**<*Result*, *Result*> : public *hpx*::*actions*::*continuation*

Public Types

using **result_type** = *Result*

Public Functions

typed_continuation() = default

inline explicit **typed_continuation**(*hpx*::id_type const &id)

inline explicit **typed_continuation**(*hpx*::id_type &&id) noexcept

template<typename **F**>

inline **typed_continuation**(*hpx*::id_type const &id, *F* &&f)

template<typename **F**>

inline **typed_continuation**(*hpx*::id_type &&id, *F* &&f)

inline **typed_continuation**(*hpx*::id_type const &id, *naming*::address &&addr)

inline **typed_continuation**(*hpx*::id_type &&id, *naming*::address &&addr) noexcept

template<typename **F**>

inline **typed_continuation**(*hpx*::id_type const &id, *naming*::address &&addr, *F* &&f)

template<typename **F**>

inline **typed_continuation**(*hpx*::id_type &&id, *naming*::address &&addr, *F* &&f)

template<typename **F**>

inline explicit **typed_continuation**(*F* &&f)

typed_continuation(*typed_continuation*&&) noexcept = default

typed_continuation &**operator=**(*typed_continuation*&&) noexcept = default

inline void **trigger_value**(*Result* &&result)

Protected Attributes

function_type **f_**

Private Types

using **function_type** = *hpx*::*distributed*::move_only_function<void(*hpx*::id_type, *Result*)>

Private Functions

template<typename **Archive**>

inline void **serialize**(*Archive* &ar, unsigned)

Friends

friend class hpx::serialization::access

template<typename **Result**, typename **RemoteResult**>

struct **typed_continuation**

Public Functions

typed_continuation() = default

inline explicit **typed_continuation**(*hpx*::id_type const &id)

inline explicit **typed_continuation**(*hpx*::id_type &&id) noexcept

template<typename **F**>

inline **typed_continuation**(*hpx*::id_type const &id, *F* &&f)

template<typename **F**>

inline **typed_continuation**(*hpx*::id_type &&id, *F* &&f)

inline **typed_continuation**(*hpx*::id_type const &id, *naming*::address &&addr)

inline **typed_continuation**(*hpx*::id_type &&id, *naming*::address &&addr) noexcept

template<typename **F**>

inline **typed_continuation**(*hpx*::id_type const &id, *naming*::address &&addr, *F* &&f)

template<typename **F**>

inline **typed_continuation**(*hpx*::id_type &&id, *naming*::address &&addr, *F* &&f)

template<typename **F**>

inline explicit **typed_continuation**(*F* &&f)

typed_continuation(*typed_continuation*&&) noexcept = default

typed_continuation &**operator=**(*typed_continuation*&&) noexcept = default

inline void **trigger_value**(*RemoteResult* &&result)

Private Types

using **base_type** = *typed_continuation*<*RemoteResult*>

using **function_type** = *hpx*::*distributed*::move_only_function<void(*hpx*::id_type, *RemoteResult*)>

Private Functions

template<typename **Archive**>

inline void **serialize**(*Archive* &ar, unsigned)

Friends

friend class hpx::serialization::access

template<>

struct **typed_continuation**<void, *util*::unused_type> : public *hpx*::*actions*::*continuation*

Public Types

using **result_type** = void

Public Functions

typed_continuation() = default

explicit **typed_continuation**(*hpx*::id_type const &id)

explicit **typed_continuation**(*hpx*::id_type &&id) noexcept

template<typename **F**>

inline **typed_continuation**(*hpx*::id_type const &id, *F* &&f)

template<typename **F**>

inline **typed_continuation**(*hpx*::id_type &&id, *F* &&f)

typed_continuation(*hpx*::id_type const &id, *naming*::address &&addr)

typed_continuation(*hpx*::id_type &&id, *naming*::address &&addr) noexcept

template<typename **F**>

inline **typed_continuation**(*hpx*::id_type const &id, *naming*::address &&addr, *F* &&f)

template<typename **F**>

inline **typed_continuation**(*hpx*::id_type &&id, *naming*::address &&addr, *F* &&f)

template<typename **F**>

inline explicit **typed_continuation**(*F* &&f)

typed_continuation(*typed_continuation*&&) noexcept

typed_continuation &**operator=**(*typed_continuation*&&) noexcept

void **trigger**() const

void **trigger_value**(*util*::unused_type&&) const

void **trigger_value**(*util*::unused_type const&) const

Private Types

using **function_type** = *hpx*::*distributed*::move_only_function<void(*hpx*::id_type)>

Private Functions

void **serialize**(*hpx*::*serialization*::input_archive &ar, unsigned)

void **serialize**(*hpx*::*serialization*::output_archive &ar, unsigned)

Private Members

function_type **f_**

Friends

friend class hpx::serialization::access

hpx/async_distributed/base_lco.hpp

Defined in header hpx/async_distributed/base_lco.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

template<>

struct **get_lva**<*hpx*::*lcos*::*base_lco*>

Public Static Functions

static inline constexpr *hpx*::*lcos*::*base_lco* ***call**(*hpx*::*naming*::address_type lva) noexcept

template<>

struct **get_lva**<*hpx*::*lcos*::*base_lco* const>

Public Static Functions

static inline constexpr *hpx*::*lcos*::*base_lco* const ***call**(*hpx*::*naming*::address_type lva) noexcept

namespace **lcos**

class **base_lco**

#include <base_lco.hpp> The base_lco class is the common base class for all LCO's implementing a simple set_event action

Subclassed by *hpx::lcos::base_lco_with_value< Result, RemoteResult, ComponentTag >*, *hpx::lcos::base_lco_with_value< void, void, ComponentTag >*

Public Types

typedef *components*::*managed_component*<*base_lco*> **wrapping_type**

typedef *base_lco* **base_type_holder**

Public Functions

virtual void **set_event**() = 0

virtual void **set_exception**(*std*::exception_ptr const &e)

virtual void **connect**(*hpx*::id_type const&)

virtual void **disconnect**(*hpx*::id_type const&)

virtual **~base_lco**()

Destructor, needs to be virtual to allow for clean destruction of derived objects

void **set_event_nonvirt**()

The *function* set_event_nonvirt is called whenever a *set_event_action* is applied on a instance of a LCO. This function just forwards to the virtual function *set_event*, which is overloaded by the derived concrete LCO.

void **set_exception_nonvirt**(*std*::exception_ptr const &e)

The *function* set_exception is called whenever a *set_exception_action* is applied on a instance of a LCO. This function just forwards to the virtual function *set_exception*, which is overloaded by the derived concrete LCO.

Parameters

e – [in] The exception encapsulating the error to report to this LCO instance.

void **connect_nonvirt**(*hpx*::id_type const &id)

The *function* connect_nonvirt is called whenever a *connect_action* is applied on a instance of a LCO. This function just forwards to the virtual function *connect*, which is overloaded by the derived concrete LCO.

Parameters

id – [in] target id

void **disconnect_nonvirt**(*hpx*::id_type const &id)

The *function* disconnect_nonvirt is called whenever a *disconnect_action* is applied on a instance of a LCO. This function just forwards to the virtual function *disconnect*, which is overloaded by the derived concrete LCO.

Parameters

id – [in] target id

HPX_DEFINE_COMPONENT_DIRECT_ACTION (base_lco, set_event_nonvirt, set_event_action) HPX_DEFINE_COMPONENT_DIRECT_ACTION(base_lco

Each of the exposed functions needs to be encapsulated into an action type, allowing to generate all required boilerplate code for threads, serialization, etc.

The *set_event_action* may be used to unconditionally trigger any LCO instances, it carries no additional parameters. The *set_exception_action* may be used to transfer arbitrary error information from the remote site to the LCO instance specified as a continuation. This action carries 2 parameters:

Parameters

std::exception_ptr – [in] The exception encapsulating the error to report to this LCO instance.

set_exception_action HPX_DEFINE_COMPONENT_DIRECT_ACTION (base_lco, connect_nonvirt, connect_action) HPX_DEFINE_COMPONENT_DIRECT_ACTION(base_lco

The *connect_action* may be used to.

The *set_exception_action* may be used to

Public Members

set_exception_nonvirt

set_exception_action **disconnect_nonvirt**

Public Static Functions

static *components*::component_type **get_component_type**() noexcept

static void **set_component_type**(*components*::component_type type)

hpx::distributed::promise

Defined in header hpx/future.hpp[584].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **Result**, typename **RemoteResult**>

class **promise** : public *hpx*::*lcos*::detail::promise_base<*Result*, *RemoteResult*, *lcos*::detail::promise_data<*Result*>>

A promise can be used by a single *thread* to invoke a (remote) action and wait for the result. The result is expected to be sent back to the promise using the LCO's set_event action

A promise is one of the simplest synchronization primitives provided by HPX. It allows to synchronize on a eager evaluated remote operation returning a result of the type *Result*. The *promise* allows to synchronize exactly one *thread* (the one passed during construction time).

```
// Create the promise (the expected result is a id_type)
hpx::distributed::promise<hpx::id_type> p;

// Get the associated future
future<hpx::id_type> f = p.get_future();

// initiate the action supplying the promise as a
// continuation
apply<some_action>(new continuation(p.get_id()), ...);
```

(continues on next page)

[584] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

(continued from previous page)

```
// Wait for the result to be returned, yielding control
// in the meantime.
hpx::id_type result = f.get();
// ...
```

Note

The action executed by the promise must return a value of a type convertible to the type as specified by the template parameter *RemoteResult*

Template Parameters

- **Result** – The template parameter *Result* defines the type this promise is expected to return from *promise::get*.
- **RemoteResult** – The template parameter *RemoteResult* defines the type this promise is expected to receive from the remote action.

hpx/async_distributed/trigger_lco.hpp

Defined in header hpx/async_distributed/trigger_lco.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

hpx::async (distributed)

Defined in header hpx/async.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also

hpx::async

template<typename **Action**, typename **F**, typename ...**Ts**>

auto *hpx*::**async**(*F* &&f, *Ts*&&... ts) -> decltype(detail::async_action_dispatch<*Action*, *std*::decay_t<*F*>>::call(*std*::forward<*F*>(*f*), *std*::forward<*Ts*>(*ts*)...))

The function template *async* runs the function *f* asynchronously (potentially in a separate thread which might be a part of a thread pool) and returns a hpx::future that will eventually hold the result of that function call. If no policy is defined, *async* behaves as if it is called with policy being *hpx::launch::async* | hpx::launch::deferred. Otherwise, it calls a function *f* with arguments *ts* according to a specific launch policy.

- If the async flag is set (i.e. (policy & hpx::launch::async) != 0), then async executes the callable object f on a new thread of execution (with all thread-locals initialized) as if spawned by *hpx::thread*(std::forward<F>(f), std::forward<Ts>(ts)…), except that if the function f returns a value or throws an exception, it is stored in the shared state accessible through the *hpx::future* that async returns to the caller.
- If the deferred flag is set (i.e. (policy & hpx::launch::deferred) != 0), then async converts f and ts… the same way as by *hpx::thread* constructor, but does not spawn a new thread of execution. Instead, lazy evaluation is performed: the first call to a non-timed wait function on the *hpx::future* that async returned to the caller will cause the copy of f to be invoked (as a rvalue) with the copies of ts… (also passed as rvalues) in the current thread (which does not have to be the thread that originally called hpx::async). The result or exception is placed in the shared state associated with the future and only then it is made ready. All further accesses to the same *hpx::future* will return the result immediately.
- If neither hpx::launch::async nor hpx::launch::deferred, nor any implementation-defined policy flag is set in policy, the behavior is undefined.

  If more than one flag is set, it is implementation-defined which policy is selected. For the default (both the hpx::launch::async and hpx::launch::deferred flags are set in policy), standard recommends (but doesn't require) utilizing available concurrency, and deferring any additional tasks.

  In any case, the call to hpx::async synchronizes-with (as defined in std::memory_order) the call to f, and the completion of f is sequenced-before making the shared state ready. If the async policy is chosen, the associated thread completion synchronizes-with the successful return from the first function that is waiting on the shared state, or with the return of the last function that releases the shared state, whichever comes first. If std::decay<Function>::type or each type in std::decay<Ts>::type is not constructible from its corresponding argument, the program is ill-formed.

Parameters

- `f` – Callable object to call
- `ts` – parameters to pass to f

Returns

hpx::future referring to the shared state created by this call to *hpx::async*.

template<typename **Action**, typename **F**, typename ...**Ts**>

auto `hpx::async`(*F* &&f, *Ts*&&... ts) -> decltype(detail::async_action_dispatch<*Action*, *std*::decay_t<*F*>>::call(*std*::forward<*F*>(*f*), *std*::forward<*Ts*>(*ts*)...))

The function template *async* runs the function *f* asynchronously (potentially in a separate thread which might be a part of a thread pool) and returns a hpx::future that will eventually hold the result of that function call. If no policy is defined, *async* behaves as if it is called with policy being *hpx::launch::async* | hpx::launch::deferred. Otherwise, it calls a function *f* with arguments *ts* according to a specific launch policy.

- If the async flag is set (i.e. (policy & hpx::launch::async) != 0), then async executes the callable object f on a new thread of execution (with all thread-locals initialized) as if spawned by *hpx::thread*(std::forward<F>(f), std::forward<Ts>(ts)…), except that if the function f returns a value or throws an exception, it is stored in the shared state accessible through the *hpx::future* that async returns to the caller.
- If the deferred flag is set (i.e. (policy & hpx::launch::deferred) != 0), then async converts f and ts… the same way as by *hpx::thread* constructor, but does not spawn a new thread

of execution. Instead, lazy evaluation is performed: the first call to a non-timed wait function on the *hpx::future* that async returned to the caller will cause the copy of f to be invoked (as a rvalue) with the copies of ts… (also passed as rvalues) in the current thread (which does not have to be the thread that originally called hpx::async). The result or exception is placed in the shared state associated with the future and only then it is made ready. All further accesses to the same *hpx::future* will return the result immediately.

- If neither hpx::launch::async nor hpx::launch::deferred, nor any implementation-defined policy flag is set in policy, the behavior is undefined.

  If more than one flag is set, it is implementation-defined which policy is selected. For the default (both the hpx::launch::async and hpx::launch::deferred flags are set in policy), standard recommends (but doesn't require) utilizing available concurrency, and deferring any additional tasks.

  In any case, the call to hpx::async synchronizes-with (as defined in std::memory_order) the call to f, and the completion of f is sequenced-before making the shared state ready. If the async policy is chosen, the associated thread completion synchronizes-with the successful return from the first function that is waiting on the shared state, or with the return of the last function that releases the shared state, whichever comes first. If std::decay<Function>::type or each type in std::decay<Ts>::type is not constructible from its corresponding argument, the program is ill-formed.

Parameters

- **f** – Callable object to call
- **ts** – parameters to pass to f

Returns

hpx::future referring to the shared state created by this call to *hpx::async*.

Warning

doxygenfunction: Unable to resolve function "hpx::async" with arguments (Executor&&, hpx::sycl::experimental::sycl_executor::queue_function_ptr_t<Ts…>&&, Ts&&…) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename Action,
↪ typename F, typename ...Ts> auto async(F &&f, Ts&&... ts)␣
↪-> decltype(detail::async_action_dispatch<Action, std::decay_
↪t<F>>::call(std::forward<F>(f), std::forward<Ts>(ts)...))
- template<typename Action, typename Target, typename␣
↪...Ts> auto async(Action &&action, Target &&target, Ts&&... ts)
- template<typename Executor, typename ...Ts> decltype(auto)␣
↪async(Executor &&exec, hpx::sycl::experimental::sycl_
↪executor::queue_function_ptr_t<Ts...> &&f, Ts&&... ts)
- template<typename␣
↪F, typename ...Ts> decltype(auto) async(F &&f, Ts&&... ts)
```

template<typename **Action**, typename **Target**, typename ...**Ts**>

auto *hpx*::**async**(*Action* &&action, *Target* &&target, *Ts*&&... ts)

The distributed implementation of `hpx::async` can be used by giving an action instance as argument instead of a function, and also by providing another argument with the locality ID or the target ID. The action executes asynchronously.

Template Parameters

- **`Action`** – The type of action instance
- **`Target`** – The type of target where the action should be executed
- **`Ts`** – The type of any additional arguments

Parameters

- **`action`** – The action instance to be executed
- **`target`** – The target where the action should be executed
- **`ts`** – Additional arguments

Returns

`hpx::future` referring to the shared state created by this call to `hpx::async`

hpx/async_distributed/trigger_lco_fwd.hpp

Defined in header hpx/async_distributed/trigger_lco_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

void **trigger_lco_event**(*hpx*::id_type id, *naming*::address &&addr, bool move_credits = true)

Trigger the LCO referenced by the given id.

Parameters

- **`id`** – [in] This represents the id of the LCO which should be triggered.
- **`addr`** – [in] This represents the addr of the LCO which should be triggered.
- **`move_credits`** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

inline void **trigger_lco_event**(*hpx*::id_type const &id, bool move_credits = true)

Trigger the LCO referenced by the given id.

Parameters

- **`id`** – [in] This represents the id of the LCO which should be triggered.

- **move_credits** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

void **trigger_lco_event**(*hpx*::id_type id, *naming*::address &&addr, *hpx*::id_type const &cont, bool move_credits = true)

Trigger the LCO referenced by the given id.

Parameters

- **id** – [in] This represents the id of the LCO which should be triggered.
- **addr** – [in] This represents the addr of the LCO which should be triggered.
- **cont** – [in] This represents the LCO to trigger after completion.
- **move_credits** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

inline void **trigger_lco_event**(*hpx*::id_type const &id, *hpx*::id_type const &cont, bool move_credits = true)

Trigger the LCO referenced by the given id.

Parameters

- **id** – [in] This represents the id of the LCO which should be triggered.
- **cont** – [in] This represents the LCO to trigger after completion.
- **move_credits** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

template<typename **Result**>

void **set_lco_value**(*hpx*::id_type id, *naming*::address &&addr, *Result* &&t, bool move_credits = true)

Set the result value for the LCO referenced by the given id.

Parameters

- **id** – [in] This represents the id of the LCO which should receive the given value.
- **addr** – [in] This represents the addr of the LCO which should be triggered.
- **t** – [in] This is the value which should be sent to the LCO.
- **move_credits** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

template<typename **Result**>

void **set_lco_value**(*hpx*::id_type const &id, *Result* &&t, bool move_credits = true)

Set the result value for the (managed) LCO referenced by the given id.

Parameters

- **id** – [in] This represents the id of the LCO which should receive the given value.
- **t** – [in] This is the value which should be sent to the LCO.

- **move_credits** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

template<typename **Result**>

void **set_lco_value_unmanaged**(*hpx*::id_type const &id, *Result* &&t, bool move_credits = true)

Set the result value for the (unmanaged) LCO referenced by the given id.

Parameters

- **id** – [in] This represents the id of the LCO which should receive the given value.
- **t** – [in] This is the value which should be sent to the LCO.
- **move_credits** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

template<typename **Result**>

void **set_lco_value**(*hpx*::id_type id, *naming*::address &&addr, *Result* &&t, *hpx*::id_type const &cont, bool move_credits = true)

Set the result value for the LCO referenced by the given id.

Parameters

- **id** – [in] This represents the id of the LCO which should receive the given value.
- **addr** – [in] This represents the addr of the LCO which should be triggered.
- **t** – [in] This is the value which should be sent to the LCO.
- **cont** – [in] This represents the LCO to trigger after completion.
- **move_credits** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

template<typename **Result**>

void **set_lco_value**(*hpx*::id_type const &id, *Result* &&t, *hpx*::id_type const &cont, bool move_credits = true)

Set the result value for the (managed) LCO referenced by the given id.

Parameters

- **id** – [in] This represents the id of the LCO which should receive the given value.
- **t** – [in] This is the value which should be sent to the LCO.
- **cont** – [in] This represents the LCO to trigger after completion.
- **move_credits** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

template<typename **Result**>

void **set_lco_value_unmanaged**(*hpx*::id_type const &id, *Result* &&t, *hpx*::id_type const &cont, bool move_credits = true)

Set the result value for the (unmanaged) LCO referenced by the given id.

Parameters

- **id** – [in] This represents the id of the LCO which should receive the given value.
- **t** – [in] This is the value which should be sent to the LCO.
- **cont** – [in] This represents the LCO to trigger after completion.
- **move_credits** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

void **set_lco_error**(*hpx*::id_type id, *naming*::address &&addr, *std*::exception_ptr const &e, bool move_credits = true)

Set the error state for the LCO referenced by the given id.

Parameters

- **id** – [in] This represents the id of the LCO which should receive the error value.
- **addr** – [in] This represents the addr of the LCO which should be triggered.
- **e** – [in] This is the error value which should be sent to the LCO.
- **move_credits** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

void **set_lco_error**(*hpx*::id_type id, *naming*::address &&addr, *std*::exception_ptr &&e, bool move_credits = true)

Set the error state for the LCO referenced by the given id.

Parameters

- **id** – [in] This represents the id of the LCO which should receive the error value.
- **addr** – [in] This represents the addr of the LCO which should be triggered.
- **e** – [in] This is the error value which should be sent to the LCO.
- **move_credits** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

inline void **set_lco_error**(*hpx*::id_type const &id, *std*::exception_ptr const &e, bool move_credits = true)

Set the error state for the LCO referenced by the given id.

Parameters

- **id** – [in] This represents the id of the LCO which should receive the error value.
- **e** – [in] This is the error value which should be sent to the LCO.
- **move_credits** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

inline void **`set_lco_error`**(*hpx*::id_type const &id, *std*::exception_ptr &&e, bool move_credits = true)

Set the error state for the LCO referenced by the given id.

Parameters

- **`id`** – [in] This represents the id of the LCO which should receive the error value.
- **`e`** – [in] This is the error value which should be sent to the LCO.
- **`move_credits`** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

void **`set_lco_error`**(*hpx*::id_type id, *naming*::address &&addr, *std*::exception_ptr const &e, *hpx*::id_type const &cont, bool move_credits = true)

Set the error state for the LCO referenced by the given id.

Parameters

- **`id`** – [in] This represents the id of the LCO which should receive the error value.
- **`addr`** – [in] This represents the addr of the LCO which should be triggered.
- **`e`** – [in] This is the error value which should be sent to the LCO.
- **`cont`** – [in] This represents the LCO to trigger after completion.
- **`move_credits`** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

void **`set_lco_error`**(*hpx*::id_type id, *naming*::address &&addr, *std*::exception_ptr &&e, *hpx*::id_type const &cont, bool move_credits = true)

Set the error state for the LCO referenced by the given id.

Parameters

- **`id`** – [in] This represents the id of the LCO which should receive the error value.
- **`addr`** – [in] This represents the addr of the LCO which should be triggered.
- **`e`** – [in] This is the error value which should be sent to the LCO.
- **`cont`** – [in] This represents the LCO to trigger after completion.
- **`move_credits`** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

inline void **`set_lco_error`**(*hpx*::id_type const &id, *std*::exception_ptr const &e, *hpx*::id_type const &cont, bool move_credits = true)

Set the error state for the LCO referenced by the given id.

Parameters

- **`id`** – [in] This represents the id of the LCO which should receive the error value.
- **`e`** – [in] This is the error value which should be sent to the LCO.
- **`cont`** – [in] This represents the LCO to trigger after completion.

- **move_credits** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

inline void **set_lco_error**(*hpx*::id_type const &id, *std*::exception_ptr &&e, *hpx*::id_type const &cont, bool move_credits = true)

Set the error state for the LCO referenced by the given id.

Parameters

- **id** – [in] This represents the id of the LCO which should receive the error value.
- **e** – [in] This is the error value which should be sent to the LCO.
- **cont** – [in] This represents the LCO to trigger after completion.
- **move_credits** – [in] If this is set to *true* then it is ok to send all credits in *id* along with the generated message. The default value is *true*.

hpx/async_distributed/packaged_action.hpp

Defined in header hpx/async_distributed/packaged_action.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **lcos**

template<typename **Action**, typename **Result**>

class **packaged_action**<*Action*, *Result*, false> : public *hpx*::*distributed*::*promise*<*Result*, *hpx*::*traits*::extract_action<*Action*>::remote_result_type>

Subclassed by *hpx::lcos::packaged_action< Action, Result, true >*

Public Functions

inline **packaged_action**()

template<typename **Allocator**>

inline **packaged_action**(*std*::allocator_arg_t, *Allocator* const &alloc)

template<typename ...**Ts**>

inline void **post**(*hpx*::id_type const &id, *Ts*&&... vs)

template<typename ...**Ts**>

inline void **post**(*naming*::address &&addr, *hpx*::id_type const &id, *Ts*&&... vs)

template<typename **Callback**, typename ...**Ts**>

inline void **post_cb**(*hpx*::id_type const &id, *Callback* &&cb, *Ts*&&... vs)

template<typename **Callback**, typename ...**Ts**>

inline void **post_cb**(*naming*::address &&addr, *hpx*::id_type const &id, *Callback* &&cb, *Ts*&&... vs)

template<typename ...**Ts**>

inline void **post_p**(*hpx*::id_type const &id, *hpx*::*launch* policy, *Ts*&&... vs)

template<typename ...**Ts**>

inline void **post_p**(*naming*::address &&addr, *hpx*::id_type const &id, *hpx*::*launch* policy, *Ts*&&... vs)

template<typename **Callback**, typename ...**Ts**>

inline void **post_p_cb**(*hpx*::id_type const &id, *hpx*::*launch* policy, *Callback* &&cb, *Ts*&&... vs)

template<typename **Callback**, typename ...**Ts**>

inline void **post_p_cb**(*naming*::address &&addr, *hpx*::id_type const &id, *hpx*::*launch* policy, *Callback* &&cb, *Ts*&&... vs)

template<typename ...**Ts**>

inline void **post_deferred**(*naming*::address &&addr, *hpx*::id_type const &id, *hpx*::*launch* policy, *Ts*&&... vs)

template<typename **Callback**, typename ...**Ts**>

inline void **post_deferred_cb**(*naming*::address &&addr, *hpx*::id_type const &id, *hpx*::*launch* policy, *Callback* &&cb, *Ts*&&... vs)

Protected Types

using **action_type** = *hpx*::*traits*::extract_action<*Action*>::type

using **remote_result_type** = *action_type*::remote_result_type

using **base_type** = *hpx*::*distributed*::*promise*<*Result*, *remote_result_type*>

Protected Functions

template<typename ...**Ts**>

inline void **do_post**(*naming*::address &&addr, *hpx*::id_type const &id, *hpx*::*launch* policy, *Ts*&&... vs)

template<typename ...**Ts**>

inline void **do_post**(*hpx*::id_type const &id, *hpx*::*launch* policy, *Ts*&&... vs)

template<typename **Callback**, typename ...**Ts**>

inline void **do_post_cb**(*naming*::address &&addr, *hpx*::id_type const &id, *hpx*::*launch* policy, *Callback* &&cb, *Ts*&&... vs)

template<typename **Callback**, typename ...**Ts**>

inline void **do_post_cb**(*hpx*::id_type const &id, *hpx*::*launch* policy, *Callback* &&cb, *Ts*&&... vs)

template<typename **Action**, typename **Result**>

class **packaged_action**<*Action*, *Result*, true> : public *hpx*::*lcos*::packaged_action<*Action*, *Result*, false>

Public Functions

inline **packaged_action**()

Construct a (non-functional) instance of a *packaged_action*. To use this instance its member function *post* needs to be directly called.

template<typename **Allocator**>

inline **packaged_action**(*std*::allocator_arg_t, *Allocator* const &alloc)

template<typename ...**Ts**>

inline void **post**(*hpx*::id_type const &id, *Ts*&&... vs)

template<typename ...**Ts**>

inline void **post**(*naming*::address &&addr, *hpx*::id_type const &id, *Ts*&&... vs)

template<typename **Callback**, typename ...**Ts**>

inline void **post_cb**(*hpx*::id_type const &id, *Callback* &&cb, *Ts*&&... vs)

template<typename **Callback**, typename ...**Ts**>

inline void **post_cb**(*naming*::address &&addr, *hpx*::id_type const &id, *Callback* &&cb, *Ts*&&... vs)

Private Types

using **action_type** = *packaged_action*<*Action*, *Result*, false>::action_type

checkpoint

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/checkpoint/checkpoint.hpp

Defined in header hpx/checkpoint/checkpoint.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

This header defines the save_checkpoint and restore_checkpoint functions. These functions are designed to help HPX application developer's checkpoint their applications. Save_checkpoint serializes one or more objects and saves them as a byte stream. Restore_checkpoint converts the byte stream back into instances of the objects.

namespace **hpx**

namespace **util**

Functions

std::ostream &**operator<<**(*std*::ostream &ost, *checkpoint* const &ckp)

Operator<< Overload

This overload is the main way to write data from a checkpoint to an object such as a file. Inside the function, the size of the checkpoint will be written to the stream before the checkpoint's data. The operator>> overload uses this to read the correct number of bytes. Be mindful of this additional write and read when you use different facilities to write out or read in data to a checkpoint!

Parameters

- **ost** – Output stream to write to.
- **ckp** – Checkpoint to copy from.

Returns

Operator<< returns the ostream object.

std::istream &**operator>>**(*std*::istream &ist, *checkpoint* &ckp)

Operator>> Overload

This overload is the main way to read in data from an object such as a file to a checkpoint. It is important to note that inside the function, the first variable to be read is the size of the checkpoint. This size variable is written to the stream before the checkpoint's data in the operator<< overload. Be mindful of this additional read and write when you use different facilities to read in or write out data from a checkpoint!

Parameters

- **ist** – Input stream to write from.
- **ckp** – Checkpoint to write to.

Returns

Operator>> returns the ostream object.

template<typename **T**, typename ...**Ts**>

hpx::*future*<*checkpoint*> **save_checkpoint**(*T* &&t, *Ts*&&... ts)

Save_checkpoint

Save_checkpoint takes any number of objects which a user may wish to store and returns a future to a checkpoint object. This function can also store a component either by passing a shared_ptr to the component or by passing a component's client instance to save_checkpoint. Additionally, the function can take a policy as a first object which changes its behavior depending on the policy passed to it. Most notably, if a sync policy is used save_checkpoint will simply return a checkpoint object.

Template Parameters

- **T** – Containers passed to save_checkpoint to be serialized and placed into a checkpoint object.
- **Ts** – More containers passed to save_checkpoint to be serialized and placed into a checkpoint object.
- **U** – This parameter is used to make sure that T is not a launch policy or a checkpoint. This forces the compiler to choose the correct overload.

Parameters

- **t** – A container to restore.
- **ts** – Other containers to restore Containers must be in the same order that they were inserted into the checkpoint.

Returns

Save_checkpoint returns a future to a checkpoint with one exception: if you pass hpx::launch::sync as the first argument. In this case save_checkpoint will simply return a checkpoint.

template<typename **T**, typename ...**Ts**>

hpx::*future*<*checkpoint*> **save_checkpoint**(*checkpoint* &&c, *T* &&t, *Ts*&&... ts)

Save_checkpoint - Take a pre-initialized checkpoint

Save_checkpoint takes any number of objects which a user may wish to store and returns a future to a checkpoint object. This function can also store a component either by passing a shared_ptr to the component or by passing a component's client instance to save_checkpoint. Additionally, the function can take a policy as a first object which changes its behavior depending on the policy passed to it. Most notably, if a sync policy is used save_checkpoint will simply return a checkpoint object.

Template Parameters

- **T** – Containers passed to save_checkpoint to be serialized and placed into a checkpoint object.
- **Ts** – More containers passed to save_checkpoint to be serialized and placed into a checkpoint object.

Parameters

- **c** – Takes a pre-initialized checkpoint to copy data into.
- **t** – A container to restore.

- **ts** – Other containers to restore Containers must be in the same order that they were inserted into the checkpoint.

Returns

Save_checkpoint returns a future to a checkpoint with one exception: if you pass hpx::launch::sync as the first argument. In this case save_checkpoint will simply return a checkpoint.

template<typename **T**, typename ...**Ts**>

hpx::*future*<*checkpoint*> **save_checkpoint**(*hpx*::*launch* p, *T* &&t, *Ts*&&... ts)

Save_checkpoint - Policy overload

Save_checkpoint takes any number of objects which a user may wish to store and returns a future to a checkpoint object. This function can also store a component either by passing a shared_ptr to the component or by passing a component's client instance to save_checkpoint. Additionally, the function can take a policy as a first object which changes its behavior depending on the policy passed to it. Most notably, if a sync policy is used save_checkpoint will simply return a checkpoint object.

Template Parameters

- **T** – Containers passed to save_checkpoint to be serialized and placed into a checkpoint object.
- **Ts** – More containers passed to save_checkpoint to be serialized and placed into a checkpoint object.

Parameters

- **p** – Takes an HPX launch policy. Allows the user to change the way the function is launched i.e. async, sync, etc.
- **t** – A container to restore.
- **ts** – Other containers to restore Containers must be in the same order that they were inserted into the checkpoint.

Returns

Save_checkpoint returns a future to a checkpoint with one exception: if you pass hpx::launch::sync as the first argument. In this case save_checkpoint will simply return a checkpoint.

template<typename **T**, typename ...**Ts**>

hpx::*future*<*checkpoint*> **save_checkpoint**(*hpx*::*launch* p, *checkpoint* &&c, *T* &&t, *Ts*&&... ts)

Save_checkpoint - Policy overload & pre-initialized checkpoint

Save_checkpoint takes any number of objects which a user may wish to store and returns a future to a checkpoint object. This function can also store a component either by passing a shared_ptr to the component or by passing a component's client instance to save_checkpoint. Additionally, the function can take a policy as a first object which changes its behavior depending on the policy passed to it. Most notably, if a sync policy is used save_checkpoint will simply return a checkpoint object.

Template Parameters

- **T** – Containers passed to save_checkpoint to be serialized and placed into a checkpoint object.

- **Ts** – More containers passed to save_checkpoint to be serialized and placed into a checkpoint object.

Parameters

- **p** – Takes an HPX launch policy. Allows the user to change the way the function is launched i.e. async, sync, etc.
- **c** – Takes a pre-initialized checkpoint to copy data into.
- **t** – A container to restore.
- **ts** – Other containers to restore Containers must be in the same order that they were inserted into the checkpoint.

Returns

Save_checkpoint returns a future to a checkpoint with one exception: if you pass hpx::launch::sync as the first argument. In this case save_checkpoint will simply return a checkpoint.

template<typename **T**, typename ...**Ts**>

checkpoint **save_checkpoint**(*hpx*::*launch*::sync_policy sync_p, *T* &&t, *Ts*&&... ts)

Save_checkpoint - Sync_policy overload

Save_checkpoint takes any number of objects which a user may wish to store and returns a future to a checkpoint object. This function can also store a component either by passing a shared_ptr to the component or by passing a component's client instance to save_checkpoint. Additionally, the function can take a policy as a first object which changes its behavior depending on the policy passed to it. Most notably, if a sync policy is used save_checkpoint will simply return a checkpoint object.

Template Parameters

- **T** – Containers passed to save_checkpoint to be serialized and placed into a checkpoint object.
- **Ts** – More containers passed to save_checkpoint to be serialized and placed into a checkpoint object.
- **U** – This parameter is used to make sure that T is not a checkpoint. This forces the compiler to choose the correct overload.

Parameters

- **sync_p** – hpx::launch::sync_policy
- **t** – A container to restore.
- **ts** – Other containers to restore Containers must be in the same order that they were inserted into the checkpoint.

Returns

Save_checkpoint which is passed hpx::launch::sync_policy will return a checkpoint which contains the serialized values checkpoint.

template<typename **T**, typename ...**Ts**>

checkpoint **save_checkpoint**(*hpx*::*launch*::sync_policy sync_p, *checkpoint* &&c, *T* &&t, *Ts*&&... ts)

Save_checkpoint - Sync_policy overload & pre-init. checkpoint

Save_checkpoint takes any number of objects which a user may wish to store and returns a future to a checkpoint object. This function can also store a component either by passing a shared_ptr to the component or by passing a component's client instance to save_checkpoint. Additionally, the function can take a policy as a first object which changes its behavior depending on the policy passed to it. Most notably, if a sync policy is used save_checkpoint will simply return a checkpoint object.

Template Parameters

- `T` – Containers passed to save_checkpoint to be serialized and placed into a checkpoint object.
- `Ts` – More containers passed to save_checkpoint to be serialized and placed into a checkpoint object.

Parameters

- `sync_p` – hpx::launch::sync_policy
- `c` – Takes a pre-initialized checkpoint to copy data into.
- `t` – A container to restore.
- `ts` – Other containers to restore Containers must be in the same order that they were inserted into the checkpoint.

Returns

Save_checkpoint which is passed hpx::launch::sync_policy will return a checkpoint which contains the serialized values checkpoint.

template<typename **T**, typename ...**Ts**>

hpx::*future*<*checkpoint*> **prepare_checkpoint**(*T* const &t, *Ts* const&... ts)

prepare_checkpoint

prepare_checkpoint takes the containers which have to be filled from the byte stream by a subsequent restore_checkpoint invocation. prepare_checkpoint will calculate the necessary buffer size and will return an appropriately sized checkpoint object.

Template Parameters

- `T` – A container to restore.
- `Ts` – Other containers to restore. Containers must be in the same order that they were inserted into the checkpoint.

Parameters

- `t` – A container to restore.
- `ts` – Other containers to restore Containers must be in the same order that they were inserted into the checkpoint.

Returns

prepare_checkpoint returns a properly resized checkpoint object that can be used for a subsequent restore_checkpoint operation.

template<typename **T**, typename ...**Ts**>

hpx::*future*<*checkpoint*> **prepare_checkpoint**(*checkpoint* &&c, *T* const &t, *Ts* const&... ts)

prepare_checkpoint

prepare_checkpoint takes the containers which have to be filled from the byte stream by a subsequent restore_checkpoint invocation. prepare_checkpoint will calculate the necessary buffer size and will return an appropriately sized checkpoint object.

Template Parameters

- `T` – A container to restore.

- **Ts** – Other containers to restore. Containers must be in the same order that they were inserted into the checkpoint.

Parameters

- **c** – Takes a pre-initialized checkpoint to prepare
- **t** – A container to restore.
- **ts** – Other containers to restore Containers must be in the same order that they were inserted into the checkpoint.

Returns

prepare_checkpoint returns a properly resized checkpoint object that can be used for a subsequent restore_checkpoint operation.

template<typename **T**, typename ...**Ts**>

hpx::*future*<*checkpoint*> **prepare_checkpoint**(*hpx*::*launch* p, *T* const &t, *Ts* const&... ts)

prepare_checkpoint

prepare_checkpoint takes the containers which have to be filled from the byte stream by a subsequent restore_checkpoint invocation. prepare_checkpoint will calculate the necessary buffer size and will return an appropriately sized checkpoint object.

Template Parameters

- **T** – A container to restore.
- **Ts** – Other containers to restore. Containers must be in the same order that they were inserted into the checkpoint.

Parameters

- **p** – Takes an HPX launch policy. Allows the user to change the way the function is launched i.e. async, sync, etc.
- **t** – A container to restore.
- **ts** – Other containers to restore Containers must be in the same order that they were inserted into the checkpoint.

Returns

prepare_checkpoint returns a properly resized checkpoint object that can be used for a subsequent restore_checkpoint operation.

template<typename **T**, typename ...**Ts**>

hpx::*future*<*checkpoint*> **prepare_checkpoint**(*hpx*::*launch* p, *checkpoint* &&c, *T* const &t, *Ts* const&... ts)

prepare_checkpoint

prepare_checkpoint takes the containers which have to be filled from the byte stream by a subsequent restore_checkpoint invocation. prepare_checkpoint will calculate the necessary buffer size and will return an appropriately sized checkpoint object.

Template Parameters

- **T** – A container to restore.
- **Ts** – Other containers to restore. Containers must be in the same order that they were inserted into the checkpoint.

Parameters

- **p** – Takes an HPX launch policy. Allows the user to change the way the function is launched i.e. async, sync, etc.

- **c** – Takes a pre-initialized checkpoint to prepare
- **t** – A container to restore.
- **ts** – Other containers to restore Containers must be in the same order that they were inserted into the checkpoint.

Returns

prepare_checkpoint returns a properly resized checkpoint object that can be used for a subsequent restore_checkpoint operation.

template<typename **T**, typename ...**Ts**>

void **restore_checkpoint**(*checkpoint* const &c, *T* &t, *Ts*&... ts)

Restore_checkpoint

Restore_checkpoint takes a checkpoint object as a first argument and the containers which will be filled from the byte stream (in the same order as they were placed in save_checkpoint). Restore_checkpoint can resurrect a stored component in two ways: by passing in an instance of a component's shared_ptr or by passing in an instance of the component's client.

Template Parameters

- **T** – A container to restore.
- **Ts** – Other containers to restore. Containers must be in the same order that they were inserted into the checkpoint.

Parameters

- **c** – The checkpoint to restore.
- **t** – A container to restore.
- **ts** – Other containers to restore Containers must be in the same order that they were inserted into the checkpoint.

Returns

Restore_checkpoint returns void.

class **checkpoint**

#include <checkpoint.hpp> Checkpoint Object

Checkpoint is the container object which is produced by save_checkpoint and is consumed by a restore_checkpoint. A checkpoint may be moved into the save_checkpoint object to write the byte stream to the pre-created checkpoint object.

Checkpoints are able to store all containers which are serializable including components.

Public Types

using **const_iterator** = *std*::vector<char>::const_iterator

Public Functions

checkpoint() = default

~**checkpoint**() = default

checkpoint(*checkpoint* const &c) = default

checkpoint(*checkpoint* &&c) noexcept = default

inline explicit **checkpoint**(*std*::vector<char> const &vec)

inline explicit **checkpoint**(*std*::vector<char> &&vec) noexcept

checkpoint &**operator=**(*checkpoint* const &c) = default

checkpoint &**operator=**(*checkpoint* &&c) noexcept = default

inline *const_iterator* **begin**() const noexcept

inline *const_iterator* **end**() const noexcept

inline *std*::size_t **size**() const noexcept

inline char ***data**() noexcept

inline char const ***data**() const noexcept

Private Functions

template<typename **Archive**>

inline void **serialize**(*Archive* &arch, unsigned int const)

Private Members

std::vector<char> **data_**

Friends

friend class hpx::serialization::access

friend *std*::ostream &**operator<<**(*std*::ostream &ost, *checkpoint* const &ckp)

Operator<< Overload

This overload is the main way to write data from a checkpoint to an object such as a file. Inside the function, the size of the checkpoint will be written to the stream before the checkpoint's data. The operator>> overload uses this to read the correct number of bytes. Be mindful of this additional write and read when you use different facilities to write out or read in data to a checkpoint!

Parameters

- **ost** – Output stream to write to.
- **ckp** – Checkpoint to copy from.

Returns

Operator<< returns the ostream object.

friend *std*::istream &**operator>>**(*std*::istream &ist, *checkpoint* &ckp)

Operator>> Overload

This overload is the main way to read in data from an object such as a file to a checkpoint. It is important to note that inside the function, the first variable to be read is the size of the checkpoint. This size variable is written to the stream before the checkpoint's data in the operator<< overload. Be mindful of this additional read and write when you use different facilities to read in or write out data from a checkpoint!

Parameters

- **ist** – Input stream to write from.
- **ckp** – Checkpoint to write to.

Returns

Operator>> returns the ostream object.

template<typename **T**, typename ...**Ts**>

friend void **restore_checkpoint**(*checkpoint* const &c, *T* &t, *Ts*&... ts)

Restore_checkpoint

Restore_checkpoint takes a checkpoint object as a first argument and the containers which will be filled from the byte stream (in the same order as they were placed in save_checkpoint). Restore_checkpoint can resurrect a stored component in two ways: by passing in an instance of a component's shared_ptr or by passing in an instance of the component's client.

Template Parameters

- **T** – A container to restore.
- **Ts** – Other containers to restore. Containers must be in the same order that they were inserted into the checkpoint.

Parameters

- **c** – The checkpoint to restore.
- **t** – A container to restore.
- **ts** – Other containers to restore Containers must be in the same order that they were inserted into the checkpoint.

Returns

Restore_checkpoint returns void.

inline friend bool **operator==**(*checkpoint* const &lhs, *checkpoint* const &rhs) noexcept

inline friend bool **operator!=**(*checkpoint* const &lhs, *checkpoint* const &rhs) noexcept

collectives

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::collectives::exclusive_scan

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**, typename **F**>

hpx::*future*<*std*::decay_t<*T*>> *hpx*::collectives::**exclusive_scan**(char const *basename, *T* &&result, *F* &&op, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

Exclusive scan a set of values from different call sites

This function performs an exclusive scan operation on a set of values received from all call sites operating on the given base name.

> **Note**
>
> This no-init overload has MPI_Exscan-like semantics, with a deterministic HPX first result: participating site 0 returns a value-initialized result (`std::decay_t<T>{}`, or `false` for `bool`). Use the overload taking an explicit `init` argument for C++ exclusive-scan semantics where site 0 returns `init`.

Parameters

- **basename** – The base name identifying the exclusive_scan operation
- **result** – The value to transmit to all participating sites from this call site.
- **op** – Reduction operation to apply to all values supplied from all participating sites
- **num_sites** – The number of participating sites (default: all localities).
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the exclusive_scan operation performed on the given base name. This is optional and needs to be supplied only if the exclusive_scan operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **root_site** – The site that is responsible for creating the exclusive_scan support object. This value is optional and defaults to '0' (zero).

Returns

For the participating site i this function returns the reduction (calculated according to the function op) of the values passed in by the participating sites 0, …, i-1. Participating site 0 returns a value-initialized result. The returned future will become ready once the exclusive_scan operation has been completed.

template<typename **T**, typename **F**>

hpx::*future*<*std*::decay_t<*T*>> *hpx*::collectives::**exclusive_scan**(*communicator* comm, *T* &&result, *F* &&op, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Exclusive scan a set of values from different call sites

This function performs an exclusive scan operation on a set of values received from all call sites operating on the given base name.

Exclusive scan a set of values from different call sites

This function performs an exclusive scan operation on a set of values received from all call sites operating on the given base name.

Note

This no-init overload has MPI_Exscan-like semantics, with a deterministic HPX first result: participating site 0 returns a value-initialized result (`std::decay_t<T>{}`, or `false` for `bool`). Use the overload taking an explicit `init` argument for C++ exclusive-scan semantics where site 0 returns `init`.

Note

This no-init overload has MPI_Exscan-like semantics, with a deterministic HPX first result: participating site 0 returns a value-initialized result (`std::decay_t<T>{}`, or `false` for `bool`). Use the overload taking an explicit `init` argument for C++ exclusive-scan semantics where site 0 returns `init`.

Parameters

- **`comm`** – A communicator object returned from *create_communicator*
- **`result`** – The value to transmit to all participating sites from this call site.
- **`op`** – Reduction operation to apply to all values supplied from all participating sites
- **`this_site`** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **`generation`** – The generational counter identifying the sequence number of the exclusive_scan operation performed on the given base name. This is optional and needs to be supplied only if the exclusive_scan operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **`comm`** – A communicator object returned from *create_communicator*
- **`result`** – The value to transmit to all participating sites from this call site.
- **`op`** – Reduction operation to apply to all values supplied from all participating sites
- **`generation`** – The generational counter identifying the sequence number of the exclusive_scan operation performed on the given base name. This is optional and needs to be supplied only if the exclusive_scan operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **`this_site`** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

For the participating site i this function returns the reduction (calculated according to the function op) of the values passed in by the participating sites 0, …, i-1. Participating site 0 returns a value-initialized result. The returned future will become ready once the exclusive_scan operation has been completed.

Returns

For the participating site i this function returns a future the reduction (calculated according to the function op) of the values passed in by the participating sites 0, …, i-1. Participating site 0 returns a value-initialized result. The returned future will become ready once the exclusive_scan operation has been completed.

Warning

doxygenfunction: Unable to resolve function “hpx::collectives::exclusive_scan” with arguments (hpx::launch::sync_policy, char const*, T&&, F&&, num_sites_arg, this_site_arg, generation_arg, root_site_arg) in doxygen xml output for project “hpx” from directory: ../hpx_autodoc. Potential matches:

```
- template<typename T, typename F> decltype(auto) exclusive_
↪scan(hpx::launch::sync_policy, char const *basename, T &&
↪result, F &&op, num_sites_arg num_sites = num_sites_arg(), this_
↪site_arg this_site = this_site_arg(), generation_arg generation␣
↪= generation_arg(), root_site_arg root_site = root_site_arg())
- template<typename T, typename␣
↪F> decltype(auto) exclusive_scan(hpx::launch::sync_policy,
↪ communicator comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T, typename F> hpx::future
↪<std::decay_t<T>> exclusive_scan(char const *basename, T &&
↪result, F &&op, num_sites_arg num_sites = num_sites_arg(), this_
↪site_arg this_site = this_site_arg(), generation_arg generation␣
↪= generation_arg(), root_site_arg root_site = root_site_arg())
- template<typename T, typename␣
↪F> hpx::future<std::decay_t<T>> exclusive_scan(communicator␣
↪comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
```

Warning

doxygenfunction: Unable to resolve function “hpx::collectives::exclusive_scan” with arguments (hpx::launch::sync_policy, communicator, T&&, F&&, this_site_arg, generation_arg) in doxygen xml output for project “hpx” from directory: ../hpx_autodoc. Potential matches:

```
- template<typename T, typename F> decltype(auto) exclusive_
↪scan(hpx::launch::sync_policy, char const *basename, T &&
↪result, F &&op, num_sites_arg num_sites = num_sites_arg(), this_
↪site_arg this_site = this_site_arg(), generation_arg generation␣
↪= generation_arg(), root_site_arg root_site = root_site_arg())
- template<typename T, typename␣
↪F> decltype(auto) exclusive_scan(hpx::launch::sync_policy,
↪ communicator comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T, typename F> hpx::future
↪<std::decay_t<T>> exclusive_scan(char const *basename, T &&
↪result, F &&op, num_sites_arg num_sites = num_sites_arg(), this_
↪site_arg this_site = this_site_arg(), generation_arg generation␣
↪= generation_arg(), root_site_arg root_site = root_site_arg())
- template<typename T, typename␣
↪F> hpx::future<std::decay_t<T>> exclusive_scan(communicator␣
↪comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
```

hpx::collectives::scatter_from

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::scatter_to*

template<typename **T**>

hpx::*future*<*T*> `hpx::collectives::scatter_from`(char const *basename, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

Scatter (receive) a set of values to different call sites

This function receives an element of a set of values operating on the given base name.

Parameters

- **basename** – The base name identifying the scatter operation
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the scatter operation performed on the given base name. This is optional and needs to be supplied only if the scatter operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **root_site** – The sequence number of the central scatter point (usually the locality id). This value is optional and defaults to 0.

Returns

This function returns a future holding the scattered value. It will become ready once the scatter operation has been completed.

template<typename **T**>

hpx::*future*<*T*> `hpx::collectives::scatter_from`(*communicator* comm, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Scatter (receive) a set of values to different call sites

This function receives an element of a set of values operating on the given base name.

Scatter (receive) a set of values to different call sites

This function receives an element of a set of values operating on the given base name.

> **Note**
>
> The generation values from corresponding *scatter_to* and *scatter_from* have to match.

> **Note**
>
> The generation values from corresponding *scatter_to* and *scatter_from* have to match.

Parameters

- **comm** – A communicator object returned from *create_communicator*
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the scatter operation performed on the given base name. This is optional and needs to be supplied only if the scatter operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **comm** – A communicator object returned from *create_communicator*
- **generation** – The generational counter identifying the sequence number of the scatter operation performed on the given base name. This is optional and needs to be supplied only if the scatter operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a future holding the scattered value. It will become ready once the scatter operation has been completed.

Returns

This function returns a future holding the scattered value. It will become ready once the scatter operation has been completed.

template<typename **T**>

T `hpx::collectives::scatter_from`(*hpx*::*launch*::sync_policy, char const *basename, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

Scatter (receive) a set of values to different call sites

This function receives an element of a set of values operating on the given base name.

Parameters

- **basename** – The base name identifying the scatter operation
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the scatter operation performed on the given base name. This is optional and needs to be supplied only if the scatter operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **root_site** – The sequence number of the central scatter point (usually the locality id). This value is optional and defaults to 0.

Returns

This function returns the scattered value. It executes synchronously and directly returns the result.

template<typename **T**>

T `hpx::collectives::scatter_from`(*hpx*::*launch*::sync_policy, *communicator* comm, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Scatter (receive) a set of values to different call sites

This function receives an element of a set of values operating on the given base name.

Scatter (receive) a set of values to different call sites

This function receives an element of a set of values operating on the given base name.

> **Note**
>
> The generation values from corresponding *scatter_to* and *scatter_from* have to match.

> **Note**
>
> The generation values from corresponding *scatter_to* and *scatter_from* have to match.

Parameters

- **comm** – A communicator object returned from *create_communicator*
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the scatter operation performed on the given base name. This is optional and needs to be supplied only if the scatter operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **comm** – A communicator object returned from *create_communicator*
- **generation** – The generational counter identifying the sequence number of the scatter operation performed on the given base name. This is optional and needs to be supplied only if the scatter operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns the scattered value. It executes synchronously and directly returns the result.

Returns

This function returns the scattered value. It executes synchronously and directly returns the result.

hpx::collectives::scatter_to

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::scatter_from*

template<typename **T**>

hpx::*future*<*T*> `hpx::collectives::scatter_to`(char const *basename, *std*::vector<*T*> &&result, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Scatter (send) a part of the value set at the given call site

This function transmits the value given by *result* to a central scatter site (where the corresponding *scatter_from* is executed)

Parameters

- **basename** – The base name identifying the scatter operation
- **result** – The value to transmit to the central scatter point from this call site.
- **num_sites** – The number of participating sites (default: all localities).
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the scatter operation performed on the given base name. This is optional and needs to be supplied only if the scatter operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.

Returns

This function returns a future holding the scattered value. It will become ready once the scatter operation has been completed.

template<typename **T**>

hpx::*future*<*T*> `hpx::collectives::scatter_to`(*communicator* comm, *std*::vector<*T*> &&result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Scatter (send) a part of the value set at the given call site

This function transmits the value given by *result* to a central scatter site (where the corresponding *scatter_from* is executed)

Scatter (send) a part of the value set at the given call site

This function transmits the value given by *result* to a central scatter site (where the corresponding *scatter_from* is executed)

> **Note**
>
> The generation values from corresponding *scatter_to* and *scatter_from* have to match.

> **Note**
>
> The generation values from corresponding *scatter_to* and *scatter_from* have to match.

Parameters

- **`comm`** – A communicator object returned from *create_communicator*
- **`result`** – The value to transmit to the central scatter point from this call site.
- **`this_site`** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **`generation`** – The generational counter identifying the sequence number of the scatter operation performed on the given base name. This is optional and needs to be supplied only if the scatter operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **`comm`** – A communicator object returned from *create_communicator*
- **`result`** – The value to transmit to the central scatter point from this call site.
- **`generation`** – The generational counter identifying the sequence number of the scatter operation performed on the given base name. This is optional and needs to be supplied only if the scatter operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **`this_site`** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a future holding the scattered value. It will become ready once the scatter operation has been completed.

Returns

This function returns a future holding the scattered value. It will become ready once the scatter operation has been completed.

template<typename **T**>

T `hpx::collectives::`**`scatter_to`**(*hpx*::*launch*::sync_policy, char const *basename, *std*::vector<*T*> &&result, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Scatter (send) a part of the value set at the given call site

This function transmits the value given by *result* to a central scatter site (where the corresponding *scatter_from* is executed)

Parameters

- **`basename`** – The base name identifying the scatter operation

- **result** – The value to transmit to the central scatter point from this call site.
- **num_sites** – The number of participating sites (default: all localities).
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the scatter operation performed on the given base name. This is optional and needs to be supplied only if the scatter operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.

Returns

This function returns the scattered value. It executes synchronously and directly returns the result.

template<typename **T**>

T hpx::collectives::**scatter_to**(*hpx*::*launch*::sync_policy, *communicator* comm, *std*::vector<*T*> &&result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Scatter (send) a part of the value set at the given call site

This function transmits the value given by *result* to a central scatter site (where the corresponding *scatter_from* is executed)

Scatter (send) a part of the value set at the given call site

This function transmits the value given by *result* to a central scatter site (where the corresponding *scatter_from* is executed)

> **Note**
>
> The generation values from corresponding *scatter_to* and *scatter_from* have to match.

> **Note**
>
> The generation values from corresponding *scatter_to* and *scatter_from* have to match.

Parameters

- **comm** – A communicator object returned from *create_communicator*
- **result** – The value to transmit to the central scatter point from this call site.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the scatter operation performed on the given base name. This is optional and needs to be supplied only if the scatter operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **comm** – A communicator object returned from *create_communicator*

- **result** – The value to transmit to the central scatter point from this call site.
- **generation** – The generational counter identifying the sequence number of the scatter operation performed on the given base name. This is optional and needs to be supplied only if the scatter operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns the scattered value. It executes synchronously and directly returns the result.

Returns

This function returns the scattered value. It executes synchronously and directly returns the result.

hpx/collectives/macros.hpp

Defined in header hpx/collectives/macros.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Defines

`HPX_REGISTER_BROADCAST_POST_ACTION_DECLARATION(...)`

`HPX_REGISTER_BROADCAST_POST_ACTION_DECLARATION_(...)`

`HPX_REGISTER_BROADCAST_POST_ACTION_DECLARATION_1(Action)`

`HPX_REGISTER_BROADCAST_POST_ACTION_DECLARATION_2(Action, Name)`

`HPX_REGISTER_BROADCAST_POST_ACTION(...)`

`HPX_REGISTER_BROADCAST_POST_ACTION_(...)`

`HPX_REGISTER_BROADCAST_POST_ACTION_1(Action)`

`HPX_REGISTER_BROADCAST_POST_ACTION_2`(Action, Name)

`HPX_REGISTER_BROADCAST_POST_WITH_INDEX_ACTION_DECLARATION`(...)

`HPX_REGISTER_BROADCAST_POST_WITH_INDEX_ACTION_DECLARATION_`(...)

`HPX_REGISTER_BROADCAST_POST_WITH_INDEX_ACTION_DECLARATION_1`(Action)

`HPX_REGISTER_BROADCAST_POST_WITH_INDEX_ACTION_DECLARATION_2`(Action, Name)

`HPX_REGISTER_BROADCAST_POST_WITH_INDEX_ACTION`(...)

`HPX_REGISTER_BROADCAST_POST_WITH_INDEX_ACTION_`(...)

`HPX_REGISTER_BROADCAST_POST_WITH_INDEX_ACTION_1`(Action)

`HPX_REGISTER_BROADCAST_POST_WITH_INDEX_ACTION_2`(Action, Name)

`HPX_REGISTER_BROADCAST_ACTION_DECLARATION`(...)

`HPX_REGISTER_BROADCAST_ACTION_DECLARATION_`(...)

`HPX_REGISTER_BROADCAST_ACTION_DECLARATION_1`(Action)

`HPX_REGISTER_BROADCAST_ACTION_DECLARATION_2`(Action, Name)

`HPX_REGISTER_BROADCAST_ACTION`(...)

`HPX_REGISTER_BROADCAST_ACTION_`(...)

`HPX_REGISTER_BROADCAST_ACTION_1`(Action)

`HPX_REGISTER_BROADCAST_ACTION_2`(Action, Name)

`HPX_REGISTER_BROADCAST_ACTION_ID`(Action, Name, Id)

`HPX_REGISTER_BROADCAST_WITH_INDEX_ACTION_DECLARATION`(...)

`HPX_REGISTER_BROADCAST_WITH_INDEX_ACTION_DECLARATION_`(...)

`HPX_REGISTER_BROADCAST_WITH_INDEX_ACTION_DECLARATION_1`(Action)

`HPX_REGISTER_BROADCAST_WITH_INDEX_ACTION_DECLARATION_2`(Action, Name)

`HPX_REGISTER_BROADCAST_WITH_INDEX_ACTION`(...)

`HPX_REGISTER_BROADCAST_WITH_INDEX_ACTION_`(...)

`HPX_REGISTER_BROADCAST_WITH_INDEX_ACTION_1`(Action)

`HPX_REGISTER_BROADCAST_WITH_INDEX_ACTION_2`(Action, Name)

`HPX_REGISTER_BROADCAST_WITH_INDEX_ACTION_ID`(Action, Name, Id)

`HPX_REGISTER_FOLD_ACTION_DECLARATION`(...)

`HPX_REGISTER_FOLD_ACTION_DECLARATION_`(...)

`HPX_REGISTER_FOLD_ACTION_DECLARATION_2`(Action, FoldOp)

`HPX_REGISTER_FOLD_ACTION_DECLARATION_3`(Action, FoldOp, Name)

`HPX_REGISTER_FOLD_ACTION`(...)

`HPX_REGISTER_FOLD_ACTION_`(...)

`HPX_REGISTER_FOLD_ACTION_2`(Action, FoldOp)

`HPX_REGISTER_FOLD_ACTION_3`(Action, FoldOp, Name)

`HPX_REGISTER_FOLD_WITH_INDEX_ACTION_DECLARATION`(...)

`HPX_REGISTER_FOLD_WITH_INDEX_ACTION_DECLARATION_`(...)

`HPX_REGISTER_FOLD_WITH_INDEX_ACTION_DECLARATION_2`(Action, FoldOp)

`HPX_REGISTER_FOLD_WITH_INDEX_ACTION_DECLARATION_3`(Action, FoldOp, Name)

`HPX_REGISTER_FOLD_WITH_INDEX_ACTION`(...)

`HPX_REGISTER_FOLD_WITH_INDEX_ACTION_`(...)

`HPX_REGISTER_FOLD_WITH_INDEX_ACTION_2`(Action, FoldOp)

`HPX_REGISTER_FOLD_WITH_INDEX_ACTION_3`(Action, FoldOp, Name)

`HPX_REGISTER_REDUCE_ACTION_DECLARATION`(...)

`HPX_REGISTER_REDUCE_ACTION_DECLARATION_`(...)

`HPX_REGISTER_REDUCE_ACTION_DECLARATION_2`(Action, ReduceOp)

`HPX_REGISTER_REDUCE_ACTION_DECLARATION_3`(Action, ReduceOp, Name)

`HPX_REGISTER_REDUCE_ACTION`(...)

`HPX_REGISTER_REDUCE_ACTION_`(...)

`HPX_REGISTER_REDUCE_ACTION_2`(Action, ReduceOp)

`HPX_REGISTER_REDUCE_ACTION_3`(Action, ReduceOp, Name)

`HPX_REGISTER_REDUCE_WITH_INDEX_ACTION_DECLARATION`(...)

`HPX_REGISTER_REDUCE_WITH_INDEX_ACTION_DECLARATION_`(...)

`HPX_REGISTER_REDUCE_WITH_INDEX_ACTION_DECLARATION_2`(Action, ReduceOp)

`HPX_REGISTER_REDUCE_WITH_INDEX_ACTION_DECLARATION_3`(Action, ReduceOp, Name)

`HPX_REGISTER_REDUCE_WITH_INDEX_ACTION`(...)

`HPX_REGISTER_REDUCE_WITH_INDEX_ACTION_`(...)

`HPX_REGISTER_REDUCE_WITH_INDEX_ACTION_2`(Action, ReduceOp)

`HPX_REGISTER_REDUCE_WITH_INDEX_ACTION_3`(Action, ReduceOp, Name)

hpx::collectives::all_to_all

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**>

hpx::*future*<*std*::vector<*T*>> `hpx::collectives::all_to_all`(char const *basename, *std*::vector<*T*> &&local_result, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*(), pairwise_threshold_arg threshold = pairwise_threshold_arg())

AllToAll a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

Parameters

- **basename** – The base name identifying the all_to_all operation
- **local_result** – A vector of values to transmit to all participating sites from this call site.
- **num_sites** – The number of participating sites (default: all localities).
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the all_to_all operation performed on the given base name. This is optional and needs to be supplied only if the all_to_all operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **root_site** – The site that is responsible for creating the all_to_all support object. This value is optional and defaults to '0' (zero).
- **threshold** – The fixed row-size threshold, in bytes, at or above which rows are exchanged directly between sites instead of being routed through one communicator site. Automatic selection uses sizeof(T) only for a trivially copyable T; other row types stay routed unless every site passes zero. See *pairwise_threshold_arg* for the default and the

remaining direct-exchange requirements. Successive generations using one basename may select different paths. Which path runs affects performance only, never the result.

Returns

This function returns a future holding a vector with all values send by all participating sites. It will become ready once the all_to_all operation has been completed.

template<typename **T**>

hpx::*future*<*std*::vector<*T*>> `hpx::collectives::all_to_all`(*communicator* fid, *std*::vector<*T*> &&local_result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

AllToAll a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

AllToAll a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

Parameters

- `fid` – A communicator object returned from *create_communicator*
- `local_result` – A vector of values to transmit to all participating sites from this call site.
- `this_site` – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- `generation` – The generational counter identifying the sequence number of the all_to_all operation performed on the given base name. This is optional and needs to be supplied only if the all_to_all operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- `fid` – A communicator object returned from *create_communicator*
- `local_result` – A vector of values to transmit to all participating sites from this call site.
- `generation` – The generational counter identifying the sequence number of the all_to_all operation performed on the given base name. This is optional and needs to be supplied only if the all_to_all operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- `this_site` – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a future holding a vector with all values send by all participating sites. It will become ready once the all_to_all operation has been completed.

Returns

This function returns a future holding a vector with all values send by all participating sites. It will become ready once the all_to_all operation has been completed.

template<typename **T**>

std::vector<*T*> `hpx::collectives::all_to_all`(*hpx*::*launch*::sync_policy policy, char const *basename, *std*::vector<*T*> &&local_result, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*(), pairwise_threshold_arg threshold = pairwise_threshold_arg())

AllToAll a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

Parameters

- **policy** – The execution policy specifying synchronous execution.
- **basename** – The base name identifying the all_to_all operation
- **local_result** – A vector of values to transmit to all participating sites from this call site.
- **num_sites** – The number of participating sites (default: all localities).
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the all_to_all operation performed on the given base name. This is optional and needs to be supplied only if the all_to_all operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **root_site** – The site that is responsible for creating the all_to_all support object. This value is optional and defaults to '0' (zero).
- **threshold** – The fixed row-size threshold, in bytes, at or above which rows are exchanged directly between sites instead of being routed through one communicator site. Automatic selection uses sizeof(T) only for a trivially copyable T; other row types stay routed unless every site passes zero. See *pairwise_threshold_arg* for the default and the remaining direct-exchange requirements. Successive generations using one basename may select different paths. Which path runs affects performance only, never the result.

Returns

This function returns a vector with all values send by all participating sites. This function executes synchronously and directly returns the result.

template<typename **T**>

std::vector<*T*> `hpx::collectives::all_to_all`(*hpx*::*launch*::sync_policy policy, *communicator* fid, *std*::vector<*T*> &&local_result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

AllToAll a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

AllToAll a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

Parameters

- `policy` – The execution policy specifying synchronous execution.
- `fid` – A communicator object returned from *create_communicator*
- `local_result` – A vector of values to transmit to all participating sites from this call site.
- `this_site` – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- `generation` – The generational counter identifying the sequence number of the all_to_all operation performed on the given base name. This is optional and needs to be supplied only if the all_to_all operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- `policy` – The execution policy specifying synchronous execution.
- `fid` – A communicator object returned from *create_communicator*
- `local_result` – A vector of values to transmit to all participating sites from this call site.
- `generation` – The generational counter identifying the sequence number of the all_to_all operation performed on the given base name. This is optional and needs to be supplied only if the all_to_all operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- `this_site` – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a vector with all values send by all participating sites. This function executes synchronously and directly returns the result.

Returns

This function returns a vector with all values send by all participating sites. This function executes synchronously and directly returns the result.

hpx::collectives::inclusive_scan

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**, typename **F**>

hpx::*future*<*std*::decay_t<*T*>> *hpx*::collectives::**inclusive_scan**(char const *basename, *T* &&result, *F* &&op, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

Inclusive inclusive_scan a set of values from different call sites

This function performs an inclusive scan operation on a set of values received from all call sites operating on the given base name.

Parameters

- `basename` – The base name identifying the inclusive_scan operation
- `result` – The value to transmit to all participating sites from this call site.
- `op` – Reduction operation to apply to all values supplied from all participating sites
- `num_sites` – The number of participating sites (default: all localities).
- `this_site` – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- `generation` – The generational counter identifying the sequence number of the inclusive_scan operation performed on the given base name. This is optional and needs to be supplied only if the inclusive_scan operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- `root_site` – The site that is responsible for creating the inclusive_scan support object. This value is optional and defaults to '0' (zero).

Returns

This function returns a future holding a vector with all values send by all participating sites. It will become ready once the inclusive_scan operation has been completed.

template<typename **T**, typename **F**>

hpx::*future*<*std*::decay_t<*T*>> `hpx::collectives::inclusive_scan`(*communicator* comm, *T* &&result, *F* &&op, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Inclusive inclusive_scan a set of values from different call sites

This function performs an inclusive scan operation on a set of values received from all call sites operating on the given base name.

Inclusive inclusive_scan a set of values from different call sites

This function performs an inclusive scan operation on a set of values received from all call sites operating on the given base name.

Parameters

- `comm` – A communicator object returned from *create_communicator*
- `result` – The value to transmit to all participating sites from this call site.
- `op` – Reduction operation to apply to all values supplied from all participating sites
- `this_site` – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- `generation` – The generational counter identifying the sequence number of the inclusive_scan operation performed on the given base name. This is optional and needs to

be supplied only if the inclusive_scan operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.

- **comm** – A communicator object returned from *create_communicator*
- **result** – The value to transmit to all participating sites from this call site.
- **op** – Reduction operation to apply to all values supplied from all participating sites
- **generation** – The generational counter identifying the sequence number of the inclusive_scan operation performed on the given base name. This is optional and needs to be supplied only if the inclusive_scan operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a future holding a vector with all values send by all participating sites. It will become ready once the inclusive_scan operation has been completed.

Returns

This function returns a future holding a vector with all values send by all participating sites. It will become ready once the inclusive_scan operation has been completed.

Warning

doxygenfunction: Unable to resolve function "hpx::collectives::inclusive_scan" with arguments (hpx::launch::sync_policy, char const*, T&&, F&&, num_sites_arg, this_site_arg, generation_arg, root_site_arg) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename T, typename F> decltype(auto) inclusive_
↪scan(hpx::launch::sync_policy, char const *basename, T &&
↪result, F &&op, num_sites_arg num_sites = num_sites_arg(), this_
↪site_arg this_site = this_site_arg(), generation_arg generation␣
↪= generation_arg(), root_site_arg root_site = root_site_arg())
- template<typename T, typename␣
↪F> decltype(auto) inclusive_scan(hpx::launch::sync_policy,
↪ communicator comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T, typename F> hpx::future
↪<std::decay_t<T>> inclusive_scan(char const *basename, T &&
↪result, F &&op, num_sites_arg num_sites = num_sites_arg(), this_
↪site_arg this_site = this_site_arg(), generation_arg generation␣
↪= generation_arg(), root_site_arg root_site = root_site_arg())
- template<typename T, typename␣
↪F> hpx::future<std::decay_t<T>> inclusive_scan(communicator␣
↪comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
```

Warning

doxygenfunction: Unable to resolve function “hpx::collectives::inclusive_scan” with arguments (hpx::launch::sync_policy, communicator, T&&, F&&, this_site_arg, generation_arg) in doxygen xml output for project “hpx” from directory: ../hpx_autodoc. Potential matches:

```
- template<typename T, typename F> decltype(auto) inclusive_
↪scan(hpx::launch::sync_policy, char const *basename, T &&
↪result, F &&op, num_sites_arg num_sites = num_sites_arg(), this_
↪site_arg this_site = this_site_arg(), generation_arg generation␣
↪= generation_arg(), root_site_arg root_site = root_site_arg())
- template<typename T, typename␣
↪F> decltype(auto) inclusive_scan(hpx::launch::sync_policy,
↪ communicator comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T, typename F> hpx::future
↪<std::decay_t<T>> inclusive_scan(char const *basename, T &&
↪result, F &&op, num_sites_arg num_sites = num_sites_arg(), this_
↪site_arg this_site = this_site_arg(), generation_arg generation␣
↪= generation_arg(), root_site_arg root_site = root_site_arg())
- template<typename T, typename␣
↪F> hpx::future<std::decay_t<T>> inclusive_scan(communicator␣
↪comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
```

hpx::collectives::broadcast_to

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::broadcast_from*

template<typename **T**>

hpx::future<T> `hpx::collectives::`**`broadcast_to`**(char const *basename, *T* &&local_result, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Broadcast a value to different call sites

This function sends a set of values to all call sites operating on the given base name.

Parameters

- **basename** – The base name identifying the broadcast operation
- **local_result** – A value to transmit to all participating sites from this call site.
- **num_sites** – The number of participating sites (default: all localities).
- **generation** – The generational counter identifying the sequence number of the broadcast operation performed on the given base name. This is optional and needs to be supplied only if the broadcast operation on the given base name has to be performed

more than once. The generation number (if given) must be a positive number greater than zero.

- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a future holding the value that was sent to all participating sites. It will become ready once the broadcast operation has been completed.

template<typename **T**>

hpx::*future*<*T*> `hpx::collectives::broadcast_to`(*communicator* comm, *T* &&local_result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Broadcast a value to different call sites

This function sends a set of values to all call sites operating on the given base name.

> **Note**
>
> The generation values from corresponding *broadcast_to* and *broadcast_from* have to match.

Parameters

- **comm** – A communicator object returned from *create_communicator*
- **local_result** – A value to transmit to all participating sites from this call site.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the broadcast operation performed on the given base name. This is optional and needs to be supplied only if the broadcast operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.

Returns

This function returns a future holding the value that was sent to all participating sites. It will become ready once the broadcast operation has been completed.

template<typename **T**>

hpx::*future*<*T*> `hpx::collectives::broadcast_to`(*communicator* comm, *generation_arg* generation, *T* &&local_result, *this_site_arg* this_site = *this_site_arg*())

Broadcast a value to different call sites

This function sends a set of values to all call sites operating on the given base name.

> **Note**
>
> The generation values from corresponding *broadcast_to* and *broadcast_from* have to match.

Parameters

- **comm** – A communicator object returned from *create_communicator*
- **local_result** – A value to transmit to all participating sites from this call site.
- **generation** – The generational counter identifying the sequence number of the broadcast operation performed on the given base name. This is optional and needs to be supplied only if the broadcast operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a future holding the value that was sent to all participating sites. It will become ready once the broadcast operation has been completed.

Warning

doxygenfunction: Unable to resolve function "hpx::collectives::broadcast_to" with arguments (hpx::launch::sync_policy, char const*, T&&, num_sites_arg, this_site_arg, generation_arg) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template
↪<typename T> decltype(auto) broadcast_to(hpx::launch::sync_
↪policy policy, char const *basename, T &&local_result, num_
↪sites_arg num_sites = num_sites_arg(), this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename␣
↪T> decltype(auto) broadcast_to(hpx::launch::sync_policy policy,
↪ communicator comm, T &&local_result, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T> hpx::future
↪<T> broadcast_to(char const *basename, T &&local_result, num_
↪sites_arg num_sites = num_sites_arg(), this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T> hpx::future<T> broadcast_
↪to(communicator comm, T &&local_result, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T> hpx::future
↪<T> broadcast_to(communicator comm, generation_arg generation,
↪ T &&local_result, this_site_arg this_site = this_site_arg())
```

Warning

doxygenfunction: Unable to resolve function "hpx::collectives::broadcast_to" with arguments (hpx::launch::sync_policy, communicator, T&&, this_site_arg, generation_arg) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template
↪<typename T> decltype(auto) broadcast_to(hpx::launch::sync_
↪policy policy, char const *basename, T &&local_result, num_
↪sites_arg num_sites = num_sites_arg(), this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename␣
↪T> decltype(auto) broadcast_to(hpx::launch::sync_policy policy,
↪ communicator comm, T &&local_result, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T> hpx::future
↪<T> broadcast_to(char const *basename, T &&local_result, num_
↪sites_arg num_sites = num_sites_arg(), this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T> hpx::future<T> broadcast_
↪to(communicator comm, T &&local_result, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T> hpx::future
↪<T> broadcast_to(communicator comm, generation_arg generation,
↪ T &&local_result, this_site_arg this_site = this_site_arg())
```

hpx::collectives::broadcast_from

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::broadcast_to*

template<typename **T**>

hpx::future<T> `hpx::collectives::`**`broadcast_from`**(char const *basename, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

Receive a value that was broadcast to different call sites

This function sends a set of values to all call sites operating on the given base name.

Parameters

- **`basename`** – The base name identifying the broadcast operation
- **`generation`** – The generational counter identifying the sequence number of the broadcast operation performed on the given base name. This is optional and needs to be supplied only if the broadcast operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **`this_site`** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **`root_site`** – The site responsible for broadcasting the value. This is optional and defaults to site `0` (zero).

Returns

This function returns a future holding the value that was sent to all participating sites. It will become ready once the broadcast operation has been completed.

template<typename **T**>

hpx::future<T> `hpx::collectives::broadcast_from`(*communicator* comm, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Receive a value that was broadcast to different call sites

This function sends a set of values to all call sites operating on the given base name.

Receive a value that was broadcast to different call sites

This function sends a set of values to all call sites operating on the given base name.

Note

The generation values from corresponding *broadcast_to* and *broadcast_from* have to match.

Note

The generation values from corresponding *broadcast_to* and *broadcast_from* have to match.

Parameters

- `comm` – A communicator object returned from *create_communicator*
- `this_site` – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- `generation` – The generational counter identifying the sequence number of the broadcast operation performed on the given base name. This is optional and needs to be supplied only if the broadcast operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- `comm` – A communicator object returned from *create_communicator*
- `generation` – The generational counter identifying the sequence number of the broadcast operation performed on the given base name. This is optional and needs to be supplied only if the broadcast operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- `this_site` – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a future holding the value that was sent to all participating sites. It will become ready once the broadcast operation has been completed.

Returns

This function returns a future holding the value that was sent to all participating sites. It will become ready once the broadcast operation has been completed.

template<typename **T**>

T hpx::collectives::**broadcast_from**(*hpx*::*launch*::sync_policy policy, char const *basename, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

Receive a value that was broadcast to different call sites

This function sends a set of values to all call sites operating on the given base name.

Parameters

- **policy** – The execution policy specifying synchronous execution.
- **basename** – The base name identifying the broadcast operation
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the broadcast operation performed on the given base name. This is optional and needs to be supplied only if the broadcast operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **root_site** – The site responsible for broadcasting the value. This is optional and defaults to site `0` (zero).

Returns

This function returns a vector with all values send by all participating sites. This function executes synchronously and directly returns the result.

template<typename **T**>

T hpx::collectives::**broadcast_from**(*hpx*::*launch*::sync_policy policy, *communicator* comm, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Receive a value that was broadcast to different call sites

This function sends a set of values to all call sites operating on the given base name.

Receive a value that was broadcast to different call sites

This function sends a set of values to all call sites operating on the given base name.

> **Note**
>
> The generation values from corresponding *broadcast_to* and *broadcast_from* have to match.

Note

The generation values from corresponding *broadcast_to* and *broadcast_from* have to match.

Parameters

- `policy` – The execution policy specifying synchronous execution.
- `comm` – A communicator object returned from *create_communicator*
- `this_site` – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- `generation` – The generational counter identifying the sequence number of the broadcast operation performed on the given base name. This is optional and needs to be supplied only if the broadcast operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- `policy` – The execution policy specifying synchronous execution.
- `comm` – A communicator object returned from *create_communicator*
- `generation` – The generational counter identifying the sequence number of the broadcast operation performed on the given base name. This is optional and needs to be supplied only if the broadcast operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- `this_site` – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a vector with all values send by all participating sites. This function executes synchronously and directly returns the result.

Returns

This function returns a vector with all values send by all participating sites. This function executes synchronously and directly returns the result.

hpx::distributed::latch

Defined in header hpx/latch.hpp[585].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

class **latch** : public *hpx*::*components*::*client_base*<*latch*, *hpx*::*lcos*::server::latch>

Latch is an implementation of a synchronization primitive that allows multiple threads to wait for a shared event to occur before proceeding. This latch can be invoked in a distributed application.

For a local only latch

585 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/latch.hpp

See also

hpx::latch.

hpx::collectives::all_gather

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**>

hpx::*future*<*std*::vector<*std*::decay_t<*T*>>> `hpx::collectives::all_gather`(char const *basename, *T* &&result, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

AllGather a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

Parameters

- **basename** – The base name identifying the all_gather operation
- **result** – The value to transmit to all participating sites from this call site.
- **num_sites** – The number of participating sites (default: all localities).
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the all_gather operation performed on the given base name. This is optional and needs to be supplied only if the all_gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **root_site** – The site that is responsible for creating the all_gather support object. This value is optional and defaults to '0' (zero).

Returns

This function returns a future holding a vector with all values send by all participating sites. It will become ready once the all_gather operation has been completed.

template<typename **T**>

hpx::*future*<*std*::vector<*std*::decay_t<*T*>>> `hpx::collectives::all_gather`(*communicator* comm, *T* &&result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

AllGather a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

AllGather a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

Parameters

- **comm** – A communicator object returned from *create_communicator*
- **result** – The value to transmit to all participating sites from this call site.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the all_gather operation performed on the given base name. This is optional and needs to be supplied only if the all_gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **comm** – A communicator object returned from *create_communicator*
- **result** – The value to transmit to all participating sites from this call site.
- **generation** – The generational counter identifying the sequence number of the all_gather operation performed on the given base name. This is optional and needs to be supplied only if the all_gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a future holding a vector with all values send by all participating sites. It will become ready once the all_gather operation has been completed.

Returns

This function returns a future holding a vector with all values send by all participating sites. It will become ready once the all_gather operation has been completed.

template<typename **T**>

std::vector<*std*::decay_t<*T*>> `hpx::collectives::all_gather`(*hpx*::*launch*::sync_policy, char const *basename, *T* &&result, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

AllGather a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

Parameters

- **basename** – The base name identifying the all_gather operation
- **result** – The value to transmit to all participating sites from this call site.

- **num_sites** – The number of participating sites (default: all localities).
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the all_gather operation performed on the given base name. This is optional and needs to be supplied only if the all_gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **root_site** – The site that is responsible for creating the all_gather support object. This value is optional and defaults to '0' (zero).

Returns

This function returns a vector with all values send by all participating sites. This function executes synchronously and directly returns the result.

template<typename **T**>

std::vector<*std*::decay_t<*T*>> `hpx::collectives::all_gather`(*hpx*::*launch*::sync_policy, *communicator* comm, *T* &&result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

AllGather a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

AllGather a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

Parameters

- **comm** – A communicator object returned from *create_communicator*
- **result** – The value to transmit to all participating sites from this call site.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the all_gather operation performed on the given base name. This is optional and needs to be supplied only if the all_gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **comm** – A communicator object returned from *create_communicator*
- **result** – The value to transmit to all participating sites from this call site.
- **generation** – The generational counter identifying the sequence number of the all_gather operation performed on the given base name. This is optional and needs to be supplied only if the all_gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a vector with all values send by all participating sites. This function executes synchronously and directly returns the result.

Returns

This function returns a vector with all values send by all participating sites. This function executes synchronously and directly returns the result.

hpx::lcos::broadcast

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::lcos::broadcast_post*
- *hpx::lcos::broadcast_with_index*
- *hpx::lcos::broadcast_post_with_index*

template<typename **Action**>

struct **broadcast**

hpx::lcos::broadcast_post

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::lcos::broadcast*
- *hpx::lcos::broadcast_with_index*
- *hpx::lcos::broadcast_post_with_index*

Warning

doxygenfunction: Unable to resolve function "hpx::lcos::broadcast_post" with arguments "None". Candidate function could not be parsed. Parsing error is Error in template parameter list. If parameter: Error when parsing template parameter. If unconstrained type parameter or template type parameter: Invalid C++ declaration: Expected 'typename' or 'class' in the beginning of template type parameter. [error at 41] template<typename Action, typename ArgN, …> void broadcast_post (std::vector< hpx::id_type > const &ids, ArgN argN,…) ——————————————^ If constrained type parameter or non-type parameter: Invalid C++ declaration: Expected identifier in nested name. [error at 41] template<typename Action, typename ArgN, …> void broadcast_post (std::vector< hpx::id_type > const &ids, ArgN argN,…) ——————————————^ If no parameter: Invalid C++ declaration: Expected "," or ">". [error at 41] template<typename Action, typename ArgN, …> void broadcast_post (std::vector< hpx::id_type > const &ids, ArgN argN,…) ——————————————^

hpx::lcos::broadcast_with_index

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::lcos::broadcast*
- *hpx::lcos::broadcast_post*
- *hpx::lcos::broadcast_post_with_index*

Warning

doxygenfunction: Unable to resolve function “hpx::lcos::broadcast_with_index” with arguments “None”. Candidate function could not be parsed. Parsing error is Error in template parameter list. If parameter: Error when parsing template parameter. If unconstrained type parameter or template type parameter: Invalid C++ declaration: Expected ‘typename’ or ‘class’ in the beginning of template type parameter. [error at 41] template<typename Action, typename ArgN, …> hpx::future< std::vector< decltype(Action(hpx::id_type, ArgN,…, std::size_t))> > broadcast_with_index (std::vector< hpx::id_type > const &ids, ArgN argN,…) ————————————^ If constrained type parameter or non-type parameter: Invalid C++ declaration: Expected identifier in nested name. [error at 41] template<typename Action, typename ArgN, …> hpx::future< std::vector< decltype(Action(hpx::id_type, ArgN,…, std::size_t))> > broadcast_with_index (std::vector< hpx::id_type > const &ids, ArgN argN,…) ————————————^ If no parameter: Invalid C++ declaration: Expected “,” or “>”. [error at 41] template<typename Action, typename ArgN, …> hpx::future< std::vector< decltype(Action(hpx::id_type, ArgN,…, std::size_t))> > broadcast_with_index (std::vector< hpx::id_type > const &ids, ArgN argN,…) ————————————^

hpx::lcos::broadcast_post_with_index

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::lcos::broadcast*
- *hpx::lcos::broadcast_post*
- *hpx::lcos::broadcast_with_index*

Warning

doxygenfunction: Unable to resolve function “hpx::lcos::broadcast_post_with_index” with arguments “None”. Candidate function could not be parsed. Parsing error is Error in template parameter list. If parameter: Error when parsing template parameter. If unconstrained type parameter or template type parameter: Invalid C++ declaration: Expected ‘typename’ or ‘class’ in the beginning of template type parameter. [error at 41] template<typename Action, typename ArgN, …> void broadcast_post_with_index (std::vector< hpx::id_type > const

&ids, ArgN argN,…) ——————————————^ If constrained type parameter or non-type parameter: Invalid C++ declaration: Expected identifier in nested name. [error at 41] template<typename Action, typename ArgN, …> void broadcast_post_with_index (std::vector< hpx::id_type > const &ids, ArgN argN,…) ——————————————^ If no parameter: Invalid C++ declaration: Expected "," or ">". [error at 41] template<typename Action, typename ArgN, …> void broadcast_post_with_index (std::vector< hpx::id_type > const &ids, ArgN argN,…) ——————————————^

hpx::collectives::gather_here

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::gather_there*

template<typename **T**>

hpx::*future*<*std*::vector<decay_t<*T*>>> `hpx::collectives::gather_here`(char const *basename, *T* &&result, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Gather a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

Parameters

- **basename** – The base name identifying the gather operation
- **result** – The value to transmit to the central gather point from this call site.
- **num_sites** – The number of participating sites (default: all localities).
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the gather operation performed on the given base name. This is optional and needs to be supplied only if the gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.

Returns

This function returns a future holding a vector with all gathered values. It will become ready once the gather operation has been completed.

template<typename **T**>

hpx::*future*<*std*::vector<decay_t<*T*>>> `hpx::collectives::gather_here`(*communicator* comm, *T* &&result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Gather a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

Gather a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

Note

The generation values from corresponding *gather_here* and *gather_there* have to match.

Note

The generation values from corresponding *gather_here* and *gather_there* have to match.

Parameters

- **`comm`** – A communicator object returned from *create_communicator*
- **`result`** – The value to transmit to the central gather point from this call site.
- **`this_site`** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **`generation`** – The generational counter identifying the sequence number of the gather operation performed on the given base name. This is optional and needs to be supplied only if the gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **`comm`** – A communicator object returned from *create_communicator*
- **`result`** – The value to transmit to the central gather point from this call site.
- **`generation`** – The generational counter identifying the sequence number of the gather operation performed on the given base name. This is optional and needs to be supplied only if the gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **`this_site`** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a future holding a vector with all gathered values. It will become ready once the gather operation has been completed.

Returns

This function returns a future holding a vector with all gathered values. It will become ready once the gather operation has been completed.

Warning

doxygenfunction: Unable to resolve function “hpx::collectives::gather_here” with arguments (hpx::launch::sync_policy, char const*, T&&, num_sites_arg, this_site_arg, generation_arg) in doxygen xml output for project “hpx” from directory: ../hpx_autodoc. Potential matches:

```
- template
↪<typename T> decltype(auto) gather_here(hpx::launch::sync_
↪policy, char const *basename, T &&result, num_
↪sites_arg num_sites = num_sites_arg(), this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template
↪<typename T> decltype(auto) gather_here(hpx::launch::sync_
↪policy, communicator comm, T &&result, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T> hpx::future<std::vector
↪<decay_t<T>>> gather_here(char const *basename, T &&result, num_
↪sites_arg num_sites = num_sites_arg(), this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template
↪<typename T> hpx::future<std::vector<decay_t<T>>> gather_
↪here(communicator comm, T &&result, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
```

Warning

doxygenfunction: Unable to resolve function “hpx::collectives::gather_here” with arguments (hpx::launch::sync_policy, communicator, T&&, this_site_arg, generation_arg) in doxygen xml output for project “hpx” from directory: ../hpx_autodoc. Potential matches:

```
- template
↪<typename T> decltype(auto) gather_here(hpx::launch::sync_
↪policy, char const *basename, T &&result, num_
↪sites_arg num_sites = num_sites_arg(), this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template
↪<typename T> decltype(auto) gather_here(hpx::launch::sync_
↪policy, communicator comm, T &&result, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T> hpx::future<std::vector
↪<decay_t<T>>> gather_here(char const *basename, T &&result, num_
↪sites_arg num_sites = num_sites_arg(), this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template
↪<typename T> hpx::future<std::vector<decay_t<T>>> gather_
↪here(communicator comm, T &&result, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
```

hpx::collectives::gather_there

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::gather_here*

template<typename **T**>

hpx::*future*<void> `hpx::collectives::gather_there`(char const *basename, *T* &&result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

Gather a given value at the given call site

This function transmits the value given by *result* to a central gather site (where the corresponding *gather_here* is executed)

Parameters

- **basename** – The base name identifying the gather operation
- **result** – The value to transmit to the central gather point from this call site.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the gather operation performed on the given base name. This is optional and needs to be supplied only if the gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **root_site** – The sequence number of the central gather point (usually the locality id). This value is optional and defaults to 0.

Returns

This function returns a future holding a vector with all gathered values. It will become ready once the gather operation has been completed.

template<typename **T**>

hpx::*future*<void> `hpx::collectives::gather_there`(*communicator* comm, *T* &&result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Gather a given value at the given call site

This function transmits the value given by *result* to a central gather site (where the corresponding *gather_here* is executed)

Gather a given value at the given call site

This function transmits the value given by *result* to a central gather site (where the corresponding *gather_here* is executed)

> **Note**
>
> The generation values from corresponding *gather_here* and *gather_there* have to match.

> **Note**
>
> The generation values from corresponding *gather_here* and *gather_there* have to match.

Parameters

- **comm** – A communicator object returned from *create_communicator*
- **result** – The value to transmit to the central gather point from this call site.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the gather operation performed on the given base name. This is optional and needs to be supplied only if the gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **comm** – A communicator object returned from *create_communicator*
- **result** – The value to transmit to the central gather point from this call site.
- **generation** – The generational counter identifying the sequence number of the gather operation performed on the given base name. This is optional and needs to be supplied only if the gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a future holding a vector with all gathered values. It will become ready once the gather operation has been completed.

Returns

This function returns a future holding a vector with all gathered values. It will become ready once the gather operation has been completed.

template<typename **T**>

void *hpx*::collectives::**gather_there**(*hpx*::*launch*::sync_policy, char const *basename, *T* &&result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

Gather a given value at the given call site

This function transmits the value given by *result* to a central gather site (where the corresponding *gather_here* is executed)

Parameters

- **basename** – The base name identifying the gather operation

- **result** – The value to transmit to the central gather point from this call site.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the gather operation performed on the given base name. This is optional and needs to be supplied only if the gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **root_site** – The sequence number of the central gather point (usually the locality id). This value is optional and defaults to 0.

Returns

This function returns a vector with all values send by all participating sites. This function executes synchronously and directly returns the result.

template<typename **T**>

void *hpx*::collectives::**gather_there**(*hpx*::*launch*::sync_policy, *communicator* comm, *T* &&result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Gather a given value at the given call site

This function transmits the value given by *result* to a central gather site (where the corresponding *gather_here* is executed)

Gather a given value at the given call site

This function transmits the value given by *result* to a central gather site (where the corresponding *gather_here* is executed)

> **Note**
>
> The generation values from corresponding *gather_here* and *gather_there* have to match.

> **Note**
>
> The generation values from corresponding *gather_here* and *gather_there* have to match.

Parameters

- **comm** – A communicator object returned from *create_communicator*
- **result** – The value to transmit to the central gather point from this call site.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the gather operation performed on the given base name. This is optional and needs to be supplied only if the gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **comm** – A communicator object returned from *create_communicator*

- **result** – The value to transmit to the central gather point from this call site.
- **generation** – The generational counter identifying the sequence number of the gather operation performed on the given base name. This is optional and needs to be supplied only if the gather operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a vector with all values send by all participating sites. This function executes synchronously and directly returns the result.

Returns

This function returns a vector with all values send by all participating sites. This function executes synchronously and directly returns the result.

hpx::lcos::spmd_block

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::lcos::define_spmd_block*

struct **spmd_block**

The class *spmd_block* defines an interface for launching multiple images while giving handles to each image to interact with the remaining images. The *define_spmd_block* function templates create multiple images of a user-defined action and launches them in a possibly separate thread. A temporary spmd block object is created and diffused to each image. The constraint for the action given to the define_spmd_block function is to accept a *spmd_block* as first parameter.

hpx::lcos::define_spmd_block

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::lcos::spmd_block*

template<typename **F**, typename ...**Args**>

hpx::*future*<void> hpx::lcos::**define_spmd_block**(*std*::string &&name, *std*::size_t images_per_locality, *F*&&, *Args*&&... args)

hpx::collectives::all_reduce

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**, typename **F**>

hpx::*future*<*std*::decay_t<*T*>> `hpx::collectives::all_reduce`(char const *basename, *T* &&result, *F* &&op, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

AllReduce a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

Parameters

- `basename` – The base name identifying the all_reduce operation
- `result` – The value to transmit to all participating sites from this call site.
- `op` – Reduction operation to apply to all values supplied from all participating sites
- `num_sites` – The number of participating sites (default: all localities).
- `generation` – The generational counter identifying the sequence number of the all_reduce operation performed on the given base name. This is optional and needs to be supplied only if the all_reduce operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- `this_site` – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- `root_site` – The site that is responsible for creating the all_reduce support object. This value is optional and defaults to '0' (zero).

Returns

This function returns a future holding a vector with all values send by all participating sites. It will become ready once the all_reduce operation has been completed.

template<typename **T**, typename **F**>

hpx::*future*<*std*::decay_t<*T*>> `hpx::collectives::all_reduce`(*communicator* comm, *T* &&result, *F* &&op, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

AllReduce a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

AllReduce a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

Parameters

- **comm** – A communicator object returned from *create_communicator*
- **result** – The value to transmit to all participating sites from this call site.
- **op** – Reduction operation to apply to all values supplied from all participating sites
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the all_reduce operation performed on the given base name. This is optional and needs to be supplied only if the all_reduce operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **comm** – A communicator object returned from *create_communicator*
- **result** – The value to transmit to all participating sites from this call site.
- **op** – Reduction operation to apply to all values supplied from all participating sites
- **generation** – The generational counter identifying the sequence number of the all_reduce operation performed on the given base name. This is optional and needs to be supplied only if the all_reduce operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a future holding a vector with all values send by all participating sites. It will become ready once the all_reduce operation has been completed.

Returns

This function returns a future holding a vector with all values send by all participating sites. It will become ready once the all_reduce operation has been completed.

Warning

doxygenfunction: Unable to resolve function "hpx::collectives::all_reduce" with arguments (hpx::launch::sync_policy, char const*, T&&, F&&, num_sites_arg, this_site_arg, generation_arg, root_site_arg) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename
↪T, typename F> decltype(auto) all_reduce(hpx::launch::sync_
↪policy policy, char const *basename, T &&
↪result, F &&op, num_sites_arg num_sites = num_sites_arg(), this_
↪site_arg this_site = this_site_arg(), generation_arg generation
↪= generation_arg(), root_site_arg root_site = root_site_arg())
- template<typename T, typename
↪F> decltype(auto) all_reduce(hpx::launch::sync_policy policy,
↪ communicator comm, T &&result, F &&op, this_site_arg this_site
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T, typename F> hpx::future
↪<std::decay_t<T>> all_reduce(char const *basename, T &&
↪result, F &&op, num_sites_arg num_sites = num_sites_arg(), this_
↪site_arg this_site = this_site_arg(), generation_arg generation
```

```
↪= generation_arg(), root_site_arg root_site = root_site_arg())
- template<typename T, typename␣
↪F> hpx::future<std::decay_t<T>> all_reduce(communicator␣
↪comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
```

Warning

doxygenfunction: Unable to resolve function "hpx::collectives::all_reduce" with arguments (hpx::launch::sync_policy, communicator, T&&, F&&, this_site_arg, generation_arg) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename␣
↪T, typename F> decltype(auto) all_reduce(hpx::launch::sync_
↪policy policy, char const *basename, T &&
↪result, F &&op, num_sites_arg num_sites = num_sites_arg(), this_
↪site_arg this_site = this_site_arg(), generation_arg generation␣
↪= generation_arg(), root_site_arg root_site = root_site_arg())
- template<typename T, typename␣
↪F> decltype(auto) all_reduce(hpx::launch::sync_policy policy,
↪ communicator comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T, typename F> hpx::future
↪<std::decay_t<T>> all_reduce(char const *basename, T &&
↪result, F &&op, num_sites_arg num_sites = num_sites_arg(), this_
↪site_arg this_site = this_site_arg(), generation_arg generation␣
↪= generation_arg(), root_site_arg root_site = root_site_arg())
- template<typename T, typename␣
↪F> hpx::future<std::decay_t<T>> all_reduce(communicator␣
↪comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
```

hpx::collectives::reduce_here

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::reduce_there*

template<typename **T**, typename **F**>

hpx::*future*<*std*::decay_t<*T*>> `hpx::collectives::`**`reduce_here`**(char const *basename, *T* &&result, *F* &&op, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Reduce a set of values from different call sites

This function receives a set of values from all call sites operating on the given base name.

Parameters

- **basename** – The base name identifying the reduce operation
- **result** – A value to reduce on the central reduction point from this call site.
- **op** – Reduction operation to apply to all values supplied from all participating sites
- **num_sites** – The number of participating sites (default: all localities).
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the reduce operation performed on the given base name. This is optional and needs to be supplied only if the reduce operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.

Returns

This function returns a future holding a vector with all values send by all participating sites. It will become ready once the reduce operation has been completed.

template<typename **T**, typename **F**>

hpx::*future*<decay_t<*T*>> `hpx::collectives::`**`reduce_here`**(*communicator* comm, *T* &&result, *F* &&op, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Reduce a set of values from different call sites

This function receives a set of values that are the result of applying a given operator on values supplied from all call sites operating on the given base name.

Reduce a set of values from different call sites

This function receives a set of values that are the result of applying a given operator on values supplied from all call sites operating on the given base name.

> **Note**
>
> The generation values from corresponding *reduce_here* and *reduce_there* have to match.

> **Note**
>
> The generation values from corresponding *reduce_here* and *reduce_there* have to match.

Parameters

- **comm** – A communicator object returned from *create_communicator*
- **result** – A value to reduce on the root_site from this call site.
- **op** – Reduction operation to apply to all values supplied from all participating sites

- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the reduce operation performed on the given base name. This is optional and needs to be supplied only if the reduce operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **comm** – A communicator object returned from *create_communicator*
- **result** – A value to reduce on the root_site from this call site.
- **op** – Reduction operation to apply to all values supplied from all participating sites
- **generation** – The generational counter identifying the sequence number of the reduce operation performed on the given base name. This is optional and needs to be supplied only if the reduce operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a future holding a value calculated based on the values send by all participating sites. It will become ready once the reduce operation has been completed.

Returns

This function returns a future holding a value calculated based on the values send by all participating sites. It will become ready once the reduce operation has been completed.

Warning

doxygenfunction: Unable to resolve function "hpx::collectives::reduce_here" with arguments (hpx::launch::sync_policy, char const*, T&&, F&&, num_sites_arg, this_site_arg, generation_arg) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename␣
↪T, typename F> decltype(auto) reduce_here(hpx::launch::sync_
↪policy, char const *basename, T &&result, F &&op, num_
↪sites_arg num_sites = num_sites_arg(), this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T,␣
↪typename F> decltype(auto) reduce_here(hpx::launch::sync_policy,
↪ communicator comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename␣
↪T, typename F> hpx::future<decay_t<T>> reduce_here(communicator␣
↪comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T, typename F> hpx::future<std::decay_
↪t<T>> reduce_here(char const *basename, T &&result, F &&op, num_
↪sites_arg num_sites = num_sites_arg(), this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
```

Warning

doxygenfunction: Unable to resolve function “hpx::collectives::reduce_here” with arguments (hpx::launch::sync_policy, communicator, T&&, F&&, this_site_arg, generation_arg) in doxygen xml output for project “hpx” from directory: ../hpx_autodoc. Potential matches:

```
- template<typename␣
↪T, typename F> decltype(auto) reduce_here(hpx::launch::sync_
↪policy, char const *basename, T &&result, F &&op, num_
↪sites_arg num_sites = num_sites_arg(), this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T,␣
↪typename F> decltype(auto) reduce_here(hpx::launch::sync_policy,
↪ communicator comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename␣
↪T, typename F> hpx::future<decay_t<T>> reduce_here(communicator␣
↪comm, T &&result, F &&op, this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
- template<typename T, typename F> hpx::future<std::decay_
↪t<T>> reduce_here(char const *basename, T &&result, F &&op, num_
↪sites_arg num_sites = num_sites_arg(), this_site_arg this_site␣
↪= this_site_arg(), generation_arg generation = generation_arg())
```

hpx::collectives::reduce_there

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::reduce_here*

template<typename **T**, typename **F**>

hpx::*future*<void> `hpx::collectives::`**`reduce_there`**(char const *basename, *T* &&result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

Reduce a given value at the given call site

This function transmits the value given by *result* to a central reduce site (where the corresponding *reduce_here* is executed)

Parameters

- **basename** – The base name identifying the reduction operation
- **result** – A future referring to the value to transmit to the central reduction point from this call site.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

- **generation** – The generational counter identifying the sequence number of the reduce operation performed on the given base name. This is optional and needs to be supplied only if the reduce operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **root_site** – The sequence number of the central reduction point (usually the locality id). This value is optional and defaults to 0.

Returns

This function returns a *future<void>*. It will become ready once the reduction operation has been completed.

template<typename **T**>

hpx::*future*<void> `hpx::collectives::reduce_there`(*communicator* comm, *T* &&result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Reduce a given value at the given call site

This function transmits the value given by *result* to a central reduce site (where the corresponding *reduce_here* is executed)

Reduce a given value at the given call site

This function transmits the value given by *result* to a central reduce site (where the corresponding *reduce_here* is executed)

Note

The generation values from corresponding *reduce_here* and *reduce_there* have to match.

Note

The generation values from corresponding *reduce_here* and *reduce_there* have to match.

Parameters

- **comm** – A communicator object returned from *create_communicator*
- **result** – A value to reduce on the central reduction point from this call site.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the reduce operation performed on the given base name. This is optional and needs to be supplied only if the reduce operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **comm** – A communicator object returned from *create_communicator*
- **result** – A value to reduce on the central reduction point from this call site.
- **generation** – The generational counter identifying the sequence number of the reduce operation performed on the given base name. This is optional and needs to be supplied

only if the reduce operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.

- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a future holding a value calculated based on the values send by all participating sites. It will become ready once the reduce operation has been completed.

Returns

This function returns a future holding a value calculated based on the values send by all participating sites. It will become ready once the reduce operation has been completed.

template<typename **T**>

void *hpx*::collectives::**reduce_there**(*hpx*::*launch*::sync_policy, char const *basename, *T* &&result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

Reduce a given value at the given call site

This function transmits the value given by *result* to a central reduce site (where the corresponding *reduce_here* is executed)

Parameters

- **basename** – The base name identifying the reduction operation
- **result** – A future referring to the value to transmit to the central reduction point from this call site.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the reduce operation performed on the given base name. This is optional and needs to be supplied only if the reduce operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **root_site** – The sequence number of the central reduction point (usually the locality id). This value is optional and defaults to 0.

template<typename **T**>

void *hpx*::collectives::**reduce_there**(*hpx*::*launch*::sync_policy, *communicator* comm, *T* &&result, *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*())

Reduce a given value at the given call site

This function transmits the value given by *result* to a central reduce site (where the corresponding *reduce_here* is executed)

Reduce a given value at the given call site

This function transmits the value given by *result* to a central reduce site (where the corresponding *reduce_here* is executed)

Note

The generation values from corresponding *reduce_here* and *reduce_there* have to match.

Note

The generation values from corresponding *reduce_here* and *reduce_there* have to match.

Parameters

- **comm** – A communicator object returned from *create_communicator*
- **result** – A value to reduce on the central reduction point from this call site.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **generation** – The generational counter identifying the sequence number of the reduce operation performed on the given base name. This is optional and needs to be supplied only if the reduce operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **comm** – A communicator object returned from *create_communicator*
- **result** – A value to reduce on the central reduction point from this call site.
- **generation** – The generational counter identifying the sequence number of the reduce operation performed on the given base name. This is optional and needs to be supplied only if the reduce operation on the given base name has to be performed more than once. The generation number (if given) must be a positive number greater than zero.
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

hpx::collectives::channel_communicator

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::create_channel_communicator*
- *hpx::collectives::get*
- *hpx::collectives::set*

class **channel_communicator**

A handle identifying the communication channel to use for get/set operations

hpx::collectives::create_channel_communicator

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::channel_communicator*
- *hpx::collectives::get*
- *hpx::collectives::set*

hpx::*future*<*channel_communicator*> `hpx::collectives::create_channel_communicator`(char const *basename, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*())

Create a new communicator object usable with peer-to-peer channel-based operations

This functions creates a new communicator object that can be called in order to pre-allocate a communicator object usable with multiple invocations of channel-based peer-to-peer operations.

> **Note**
>
> The caller must ensure that the communicator object returned by this function is kept alive (i.e., does not go out of scope) until all participating sites have successfully connected. Destroying the communicator before all sites have reached the creation point may cause the remaining sites to hang indefinitely. Consider using a barrier or similar synchronization mechanism after creating the communicator to ensure all sites are connected before allowing it to be destroyed.

Parameters

- **`basename`** – The base name identifying the collective operation
- **`num_sites`** – The number of participating sites (default: all localities).
- **`this_site`** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a future to a new communicator object usable with the collective operation.

channel_communicator `hpx::collectives::create_channel_communicator`(*hpx*::*launch*::sync_policy, char const *basename, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*())

Create a new communicator object usable with peer-to-peer channel-based operations

This functions creates a new communicator object that can be called in order to pre-allocate a communicator object usable with multiple invocations of channel-based peer-to-peer operations.

> **Note**
>
> The caller must ensure that the communicator object returned by this function is kept alive (i.e., does not go out of scope) until all participating sites have successfully connected. Destroying the communicator before all sites have reached the creation point may cause the remaining sites to hang indefinitely. Consider using a barrier or similar synchronization mechanism after creating the communicator to ensure all sites are connected before allowing it to be destroyed.

Parameters

- **basename** – The base name identifying the collective operation
- **num_sites** – The number of participating sites (default: all localities).
- **this_site** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.

Returns

This function returns a new communicator object usable with the collective operation.

hpx::collectives::get

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::channel_communicator*
- *hpx::collectives::create_channel_communicator*
- *hpx::collectives::set*

template<typename **T**>

hpx::future<T> `hpx::collectives::get`(*channel_communicator* comm, *that_site_arg* site, *tag_arg* tag = *tag_arg*())

Send a value to the given site

This function receives a value from the given site based on the given communicator.

Parameters

- **comm** – The channel communicator object to use for the data transfer
- **site** – The source site

Returns

This function returns a future<T> that becomes ready once the data transfer operation has finished. The future will hold the received value.

hpx::collectives::set

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::channel_communicator*
- *hpx::collectives::create_channel_communicator*
- *hpx::collectives::get*

template<typename **T**>

hpx::*future*<void> `hpx::collectives::set`(*channel_communicator* comm, *that_site_arg* site, *T* &&value, *tag_arg* tag = *tag_arg*())

Send a value to the given site

This function sends a value to the given site based on the given communicator.

Parameters

- **comm** – The channel communicator object to use for the data transfer
- **site** – The destination site
- **value** – The value to send
- **tag** – The (optional) tag identifying the concrete operation

Returns

This function returns a *future<void>* that becomes ready once the data transfer operation has finished.

hpx::collectives::num_sites_arg

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::this_site_arg*
- *hpx::collectives::that_site_arg*
- *hpx::collectives::generation_arg*
- *hpx::collectives::root_site_arg*
- *hpx::collectives::tag_arg*
- *hpx::collectives::arity_arg*
- *hpx::collectives::flat_fallback_threshold_arg*

using `hpx::collectives::num_sites_arg` = detail::argument_type<detail::num_sites_tag>

The number of participating sites (default: all localities)

hpx::collectives::this_site_arg

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::num_sites_arg*
- *hpx::collectives::that_site_arg*
- *hpx::collectives::generation_arg*
- *hpx::collectives::root_site_arg*
- *hpx::collectives::tag_arg*
- *hpx::collectives::arity_arg*
- *hpx::collectives::flat_fallback_threshold_arg*

using `hpx::collectives::this_site_arg` = detail::argument_type<detail::this_site_tag>

The local end of the communication channel.

hpx::collectives::that_site_arg

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::num_sites_arg*
- *hpx::collectives::this_site_arg*
- *hpx::collectives::generation_arg*
- *hpx::collectives::root_site_arg*
- *hpx::collectives::tag_arg*
- *hpx::collectives::arity_arg*
- *hpx::collectives::flat_fallback_threshold_arg*

using `hpx::collectives::that_site_arg` = detail::argument_type<detail::that_site_tag>

The opposite end of the communication channel.

hpx::collectives::generation_arg

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::num_sites_arg*
- *hpx::collectives::this_site_arg*
- *hpx::collectives::that_site_arg*
- *hpx::collectives::root_site_arg*
- *hpx::collectives::tag_arg*
- *hpx::collectives::arity_arg*
- *hpx::collectives::flat_fallback_threshold_arg*

using `hpx::collectives::generation_arg` = detail::argument_type<detail::generation_tag>

The generational counter identifying the sequence number of the operation performed on the given base name. It needs to be supplied only if the operation on the given base name has to be performed more than once. It must be a positive number greater than zero. On any given communicator instance, operations that pass an explicit generation may not follow operations that used the default: the internal counter's position is no longer reliably known to the caller. The reverse transition, from explicit numbering back to the default, remains valid.

hpx::collectives::root_site_arg

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::num_sites_arg*
- *hpx::collectives::this_site_arg*
- *hpx::collectives::that_site_arg*
- *hpx::collectives::generation_arg*
- *hpx::collectives::tag_arg*
- *hpx::collectives::arity_arg*
- *hpx::collectives::flat_fallback_threshold_arg*

using `hpx::collectives::root_site_arg` = detail::argument_type<detail::root_site_tag, 0>

The site that is responsible for creating the support object of the operation. It defaults to '0' (zero).

hpx::collectives::tag_arg

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::num_sites_arg*
- *hpx::collectives::this_site_arg*
- *hpx::collectives::that_site_arg*
- *hpx::collectives::generation_arg*
- *hpx::collectives::root_site_arg*
- *hpx::collectives::arity_arg*
- *hpx::collectives::flat_fallback_threshold_arg*

using `hpx::collectives::tag_arg` = detail::argument_type<detail::tag_tag, 0>

The tag identifying the concrete operation.

hpx::collectives::arity_arg

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::num_sites_arg*
- *hpx::collectives::this_site_arg*
- *hpx::collectives::that_site_arg*
- *hpx::collectives::generation_arg*
- *hpx::collectives::root_site_arg*
- *hpx::collectives::tag_arg*
- *hpx::collectives::flat_fallback_threshold_arg*

using `hpx::collectives::arity_arg` = detail::argument_type<detail::arity_tag, 2>

The number of children each of the communication nodes is connected to (default: 2).

hpx::collectives::flat_fallback_threshold_arg

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::num_sites_arg*
- *hpx::collectives::this_site_arg*
- *hpx::collectives::that_site_arg*
- *hpx::collectives::generation_arg*
- *hpx::collectives::root_site_arg*
- *hpx::collectives::tag_arg*
- *hpx::collectives::arity_arg*

using `hpx::collectives::flat_fallback_threshold_arg` = detail::argument_type<detail::flat_fallback_threshold_tag, 16>

The site-count threshold below which a hierarchical_communicator collapses to a single flat communicator spanning all sites. At small site counts, flat collectives outperform hierarchical composition because tree-walking overhead exceeds the synchronization-depth benefit. Pass 0 to disable the fallback and always build a tree.

hpx::collectives::communicator

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::create_communicator*
- *hpx::collectives::create_local_communicator*
- *hpx::collectives::create_hierarchical_communicator*
- *hpx::collectives::communicator::set_info*
- *hpx::collectives::communicator::get_info*
- *hpx::collectives::communicator::is_root*

struct **communicator**

A communicator instance represents the list of sites that participate in a particular collective operation.

hpx::collectives::create_communicator

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::communicator*
- *hpx::collectives::create_local_communicator*
- *hpx::collectives::create_hierarchical_communicator*
- *hpx::collectives::communicator::set_info*
- *hpx::collectives::communicator::get_info*
- *hpx::collectives::communicator::is_root*

communicator `hpx::collectives::create_communicator`(char const *basename, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

Create a new communicator object usable with any collective operation

This functions creates a new communicator object that can be called in order to pre-allocate a communicator object usable with multiple invocations of the collective operations (such as *all_gather*, *all_reduce*, *all_to_all*, *broadcast*, etc.).

Parameters

- `basename` – The base name identifying the collective operation
- `num_sites` – The number of participating sites (default: all localities).
- `this_site` – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- `generation` – The generational counter identifying the sequence number of the collective operation performed on the given base name. This is optional and needs to be supplied only if the collective operation on the given base name has to be performed more than once.
- `root_site` – The site that is responsible for creating the collective support object. This value is optional and defaults to '0' (zero).

Throws

`hpx::exception` – with hpx::error::bad_parameter if *basename* is null or empty, if the (resolved) *num_sites* is zero, if the (resolved) *this_site* is not smaller than *num_sites*, or if the (resolved) *root_site* does not designate a participating site.

Returns

This function returns a new communicator object usable with the collective operation.

communicator `hpx::collectives::create_communicator`(*hpx*::*launch*::sync_policy policy, char const *basename, *num_sites_arg* num_sites, *this_site_arg* this_site, *generation_arg* generation, *root_site_arg* root_site)

hpx::collectives::create_local_communicator

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::communicator*
- *hpx::collectives::create_communicator*
- *hpx::collectives::create_hierarchical_communicator*
- *hpx::collectives::communicator::set_info*
- *hpx::collectives::communicator::get_info*
- *hpx::collectives::communicator::is_root*

communicator `hpx::collectives::create_local_communicator`(char const *basename, *num_sites_arg* num_sites, *this_site_arg* this_site, *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*())

Create a new communicator object usable with any local collective operation

This functions creates a new communicator object that can be called in order to pre-allocate a communicator object usable with multiple invocations of the collective operations (such as *all_gather*, *all_reduce*, *all_to_all*, *broadcast*, etc.).

Parameters

- **basename** – The base name identifying the collective operation
- **num_sites** – The number of participating sites
- **this_site** – The sequence number of this invocation (usually the sequence number of the object participating in the collective operation). This value must be in the range [0, num_sites).
- **generation** – The generational counter identifying the sequence number of the collective operation performed on the given base name. This is optional and needs to be supplied only if the collective operation on the given base name has to be performed more than once.
- **root_site** – The site that is responsible for creating the collective support object. This value is optional and defaults to '0' (zero).

Throws

`hpx::exception` – with hpx::error::bad_parameter if *basename* is null or empty, if *num_sites* is unspecified or zero, if *this_site* is unspecified or not smaller than *num_sites*, or if the (resolved) *root_site* does not designate a participating site.

Returns

This function returns a new communicator object usable for all local collective operations.

communicator `hpx::collectives::create_local_communicator`(*hpx*::*launch*::sync_policy policy, char const *basename, *num_sites_arg* num_sites, *this_site_arg* this_site, *generation_arg* generation, *root_site_arg* root_site)

hpx::collectives::create_hierarchical_communicator

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::communicator*
- *hpx::collectives::create_communicator*
- *hpx::collectives::create_local_communicator*
- *hpx::collectives::communicator::set_info*
- *hpx::collectives::communicator::get_info*
- *hpx::collectives::communicator::is_root*

hierarchical_communicator *hpx*::collectives::**create_hierarchical_communicator**(char const *basename, *num_sites_arg* num_sites = *num_sites_arg*(), *this_site_arg* this_site = *this_site_arg*(), *arity_arg* arity = *arity_arg*(), *generation_arg* generation = *generation_arg*(), *root_site_arg* root_site = *root_site_arg*(), *flat_fallback_threshold_arg* threshold = *flat_fallback_threshold_arg*())

Create a new hierarchical communicator object usable with collective operations

This functions creates a new hierarchical_communicator object that can be called in order to pre-allocate a communicator object usable with multiple invocations of a collective operation (such as *all_gather*, *all_reduce*, *all_to_all*, *broadcast*, etc.).

> **Note**
>
> The sub-communicators of a hierarchical_communicator share one generation sequence per registered name. Every hierarchical collective advances each sub-communicator it touches by exactly two internal generations per call (a single-pass collective such as *broadcast*, *gather*, *scatter* or *reduce*, and the inter-group exchange of *all_to_all*, skip the second generation in one step rather than performing a second round-trip). A single hierarchical_communicator instance may therefore be shared freely across collective operations, provided every call uses an explicit, strictly consecutive generation number

so the shared sequence stays gap-free. Because the two-phase hierarchical collectives (*all_gather*, *all_reduce*, *all_to_all*, *inclusive_scan*, *exclusive_scan* and the hierarchical *barrier*) derive their phase generations from that explicit number, they require it and reject an auto (default) generation. The non-hierarchical (flat) collectives keep supporting the default generation, falling back to the internal per-communicator counter maintained in the and_gate. Once any operation on a communicator has used that default, later operations on the same instance may no longer pass an explicit generation number: the counter's position is no longer reliably known to the caller. The reverse transition, from explicit numbering back to the default, remains valid. All participants of a single operation must use the same generation mode.

Note

Hierarchical collective overloads that return *hpx::future* may still perform internal waits while walking the communicator tree. They should not be treated as fully non-blocking: validation failures are reported through the returned exceptional future, but internal phase hand-offs can throw or block before that future is delivered.

Parameters

- **`basename`** – The base name identifying the collective operation
- **`num_sites`** – The number of participating sites (default: all localities).
- **`this_site`** – The sequence number of this invocation (usually the locality id). This value is optional and defaults to whatever *hpx::get_locality_id()* returns.
- **`arity`** – The arity of the hierarchical tree of communicators to create for the given endpoint. The default arity is 2 and the value must be at least 2.
- **`generation`** – The generational counter identifying the sequence number of the collective operation performed on the given base name. This is optional and needs to be supplied only if the collective operation on the given base name has to be performed more than once.
- **`root_site`** – The site that is responsible for creating the collective support object. Hierarchical communicators currently support only site 0 as the root, which is also the default.
- **`threshold`** – The site-count threshold below which the communicator collapses to a single flat communicator spanning all sites (strict comparison: num_sites < threshold). The default is 16; pass 0 to disable the fallback and always build a tree.

Throws

`hpx::exception` – with hpx::error::bad_parameter if *basename* is null or empty, if the (resolved) *num_sites* is zero, if the (resolved) *this_site* is not smaller than *num_sites*, if the (resolved) *root_site* is not zero, or if *arity* is smaller than 2.

Returns

This function returns a new communicator object usable with the collective operation.

hpx::collectives::communicator::set_info

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::communicator*
- *hpx::collectives::create_communicator*
- *hpx::collectives::create_local_communicator*
- *hpx::collectives::create_hierarchical_communicator*
- *hpx::collectives::communicator::get_info*
- *hpx::collectives::communicator::is_root*

void *hpx*::collectives::*communicator*::**set_info**(*num_sites_arg* num_sites, *this_site_arg* this_site) noexcept

Store the number of used sites and the index of the current site for this communicator instance.

Parameters

- **num_sites** – The number of participating sites (default: all localities).
- **this_site** – The sequence number of this site (usually the locality id).

hpx::collectives::communicator::get_info

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::communicator*
- *hpx::collectives::create_communicator*
- *hpx::collectives::create_local_communicator*
- *hpx::collectives::create_hierarchical_communicator*
- *hpx::collectives::communicator::set_info*
- *hpx::collectives::communicator::is_root*

std::pair<*num_sites_arg*, *this_site_arg*> *hpx*::collectives::*communicator*::**get_info**() const noexcept

Retrieve the number of used sites and the index of the current site for this communicator instance.

hpx::collectives::communicator::is_root

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::collectives::communicator*
- *hpx::collectives::create_communicator*
- *hpx::collectives::create_local_communicator*
- *hpx::collectives::create_hierarchical_communicator*
- *hpx::collectives::communicator::set_info*
- *hpx::collectives::communicator::get_info*

bool `hpx::collectives::communicator::is_root()` const

Return whether this communicator instance represents the root site of the communication operation.

hpx::distributed::barrier

Defined in header hpx/barrier.hpp[586].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

explicit `hpx::distributed::barrier(`*std*::string const &base_name)

Creates a barrier, rank is locality id, size is number of localities

A barrier *base_name* is created. It expects that *hpx::get_num_localities()* participate and the local rank is *hpx::get_locality_id()*.

Parameters

base_name – The name of the barrier

`hpx::distributed::barrier(`*std*::string const &base_name, *std*::size_t num)

Creates a barrier with a given size, rank is locality id

A barrier *base_name* is created. It expects that *num* participate and the local rank is *hpx::get_locality_id()*.

Parameters

- **base_name** – The name of the barrier
- **num** – The number of participating threads

`hpx::distributed::barrier(`*std*::string const &base_name, *std*::size_t num, *std*::size_t rank)

Creates a barrier with a given size and rank

[586] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/barrier.hpp

A barrier *base_name* is created. It expects that *num* participate and the local rank is *rank*.

Parameters

- **base_name** – The name of the barrier
- **num** – The number of participating threads
- **rank** – The rank of the calling site for this invocation

hpx::*distributed*::**barrier**(*std*::string const &base_name, *std*::vector<*std*::size_t> const &ranks, *std*::size_t rank)

Creates a barrier with a vector of ranks

A barrier *base_name* is created. It expects that ranks.size() and the local rank is *rank* (must be contained in *ranks*).

Parameters

- **base_name** – The name of the barrier
- **ranks** – Gives a list of participating ranks (this could be derived from a list of locality ids
- **rank** – The rank of the calling site for this invocation

hpx::lcos::fold

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::lcos::fold_with_index*
- *hpx::lcos::inverse_fold*
- *hpx::lcos::inverse_fold_with_index*

Warning

doxygenfunction: Unable to resolve function “hpx::lcos::fold” with arguments “None”. Candidate function could not be parsed. Parsing error is Error in template parameter list. If parameter: Error when parsing template parameter. If unconstrained type parameter or template type parameter: Invalid C++ declaration: Expected ‘typename’ or ‘class’ in the beginning of template type parameter. [error at 73] template<typename Action, typename FoldOp, typename Init, typename ArgN, …> hpx::future< decltype(Action(hpx::id_type, ArgN,…))> fold (std::vector< hpx::id_type > const &ids, FoldOp &&fold_op, Init &&init, ArgN argN,…) ————————————————————————-
^ If constrained type parameter or non-type parameter: Invalid C++ declaration: Expected identifier in nested name. [error at 73] template<typename Action, typename FoldOp, typename Init, typename ArgN, …> hpx::future< decltype(Action(hpx::id_type, ArgN,…))> fold (std::vector< hpx::id_type > const &ids, FoldOp &&fold_op, Init &&init, ArgN argN,…) ————————————————————————-^ If no parameter: Invalid C++ declaration: Expected

"," or ">". [error at 73] template<typename Action, typename FoldOp, typename Init, typename ArgN, ...> hpx::future< decltype(Action(hpx::id_type, ArgN,...))> fold (std::vector< hpx::id_type > const &ids, FoldOp &&fold_op, Init &&init, ArgN argN,...) ————————————————————————-^

hpx::lcos::fold_with_index

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::lcos::fold*
- *hpx::lcos::inverse_fold*
- *hpx::lcos::inverse_fold_with_index*

Warning

doxygenfunction: Unable to resolve function "hpx::lcos::fold_with_index" with arguments "None". Candidate function could not be parsed. Parsing error is Error in template parameter list. If parameter: Error when parsing template parameter. If unconstrained type parameter or template type parameter: Invalid C++ declaration: Expected 'typename' or 'class' in the beginning of template type parameter. [error at 73] template<typename Action, typename FoldOp, typename Init, typename ArgN, ...> hpx::future< decltype(Action(hpx::id_type, ArgN,..., std::size_t))> fold_with_index (std::vector< hpx::id_type > const &ids, FoldOp &&fold_op, Init &&init, ArgN argN,...) ————————————————————————-^ If constrained type parameter or non-type parameter: Invalid C++ declaration: Expected identifier in nested name. [error at 73] template<typename Action, typename FoldOp, typename Init, typename ArgN, ...> hpx::future< decltype(Action(hpx::id_type, ArgN,..., std::size_t))> fold_with_index (std::vector< hpx::id_type > const &ids, FoldOp &&fold_op, Init &&init, ArgN argN,...) ————————————————————————-^ If no parameter: Invalid C++ declaration: Expected "," or ">". [error at 73] template<typename Action, typename FoldOp, typename Init, typename ArgN, ...> hpx::future< decltype(Action(hpx::id_type, ArgN,..., std::size_t))> fold_with_index (std::vector< hpx::id_type > const &ids, FoldOp &&fold_op, Init &&init, ArgN argN,...) ————————————————————————-^

hpx::lcos::inverse_fold

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::lcos::fold*
- *hpx::lcos::fold_with_index*
- *hpx::lcos::inverse_fold_with_index*

Warning

doxygenfunction: Unable to resolve function "hpx::lcos::inverse_fold" with arguments "None". Candidate function could not be parsed. Parsing error is Error in template parameter list. If parameter: Error when parsing template parameter. If unconstrained type parameter or template type parameter: Invalid C++ declaration: Expected 'typename' or 'class' in the beginning of template type parameter. [error at 73] template<typename Action, typename FoldOp, typename Init, typename ArgN, ...> hpx::future< decltype(Action(hpx::id_type, ArgN,...))> inverse_fold (std::vector< hpx::id_type > const &ids, FoldOp &&fold_op, Init &&init, ArgN argN,...) ————————————————————-^ If constrained type parameter or non-type parameter: Invalid C++ declaration: Expected identifier in nested name. [error at 73] template<typename Action, typename FoldOp, typename Init, typename ArgN, ...> hpx::future< decltype(Action(hpx::id_type, ArgN,...))> inverse_fold (std::vector< hpx::id_type > const &ids, FoldOp &&fold_op, Init &&init, ArgN argN,...) ————————————————————-^ If no parameter: Invalid C++ declaration: Expected "," or ">". [error at 73] template<typename Action, typename FoldOp, typename Init, typename ArgN, ...> hpx::future< decltype(Action(hpx::id_type, ArgN,...))> inverse_fold (std::vector< hpx::id_type > const &ids, FoldOp &&fold_op, Init &&init, ArgN argN,...) ————————————————————-^

hpx::lcos::inverse_fold_with_index

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::lcos::fold*
- *hpx::lcos::fold_with_index*
- *hpx::lcos::inverse_fold*

Warning

doxygenfunction: Unable to resolve function "hpx::lcos::inverse_fold_with_index" with arguments "None". Candidate function could not be parsed. Parsing error is Error in template parameter list. If parameter: Error when parsing template parameter. If unconstrained type parameter or template type parameter: Invalid C++ declaration: Expected 'typename' or 'class' in the beginning of template type parameter. [error at 73] template<typename Action, typename FoldOp, typename Init, typename ArgN, ...> hpx::future< decltype(Action(hpx::id_type, ArgN,..., std::size_t))> inverse_fold_with_index (std::vector< hpx::id_type > const &ids, FoldOp &&fold_op, Init &&init, ArgN argN,...) ————————————————————- ^ If constrained type parameter or non-type parameter: Invalid C++ declaration: Expected identifier in nested name. [error at 73] template<typename Action, typename FoldOp, typename Init, typename ArgN, ...> hpx::future< decltype(Action(hpx::id_type, ArgN,..., std::size_t))> inverse_fold_with_index (std::vector< hpx::id_type > const &ids, FoldOp &&fold_op, Init &&init, ArgN argN,...) ————————————————————-^ If no parameter: Invalid C++ declaration: Expected "," or ">". [error at 73] template<typename Action, typename FoldOp, typename Init, typename ArgN, ...> hpx::future< decltype(Action(hpx::id_type, ArgN,..., std::size_t))> inverse_fold_with_index (std::vector< hpx::id_type > const &ids, FoldOp &&fold_op, Init &&init, ArgN argN,...)

————————————————————————————————-^

hpx::lcos::reduce

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::lcos::reduce_with_index*

Warning

doxygenfunction: Unable to resolve function "hpx::lcos::reduce" with arguments "None". Candidate function could not be parsed. Parsing error is Error in template parameter list. If parameter: Error when parsing template parameter. If unconstrained type parameter or template type parameter: Invalid C++ declaration: Expected 'typename' or 'class' in the beginning of template type parameter. [error at 60] template<typename Action, typename ReduceOp, typename ArgN, …> hpx::future< decltype(Action(hpx::id_type, ArgN,…))> reduce (std::vector< hpx::id_type > const &ids, ReduceOp &&reduce_op, ArgN argN,…) ————————————————————————^ If constrained type parameter or non-type parameter: Invalid C++ declaration: Expected identifier in nested name. [error at 60] template<typename Action, typename ReduceOp, typename ArgN, …> hpx::future< decltype(Action(hpx::id_type, ArgN,…))> reduce (std::vector< hpx::id_type > const &ids, ReduceOp &&reduce_op, ArgN argN,…) ————————————————————————^ If no parameter: Invalid C++ declaration: Expected "," or ">". [error at 60] template<typename Action, typename ReduceOp, typename ArgN, …> hpx::future< decltype(Action(hpx::id_type, ArgN,…))> reduce (std::vector< hpx::id_type > const &ids, ReduceOp &&reduce_op, ArgN argN,…) ————————————————————————^

hpx::lcos::reduce_with_index

Defined in header hpx/collectives.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::lcos::reduce*

Warning

doxygenfunction: Unable to resolve function "hpx::lcos::reduce_with_index" with arguments "None". Candidate function could not be parsed. Parsing error is Error in template parameter list. If parameter: Error when parsing template parameter. If unconstrained type parameter or template type parameter: Invalid C++ declaration: Expected 'typename' or 'class' in the beginning of template type parameter. [error at 60] template<typename Action, typename ReduceOp, typename ArgN, …> hpx::future< decltype(Action(hpx::id_type, ArgN,…, std::size_t))> reduce_with_index (std::vector< hpx::id_type > const &ids, ReduceOp &&reduce_op, ArgN argN,…) ————————————————————————^ If constrained type parameter or

non-type parameter: Invalid C++ declaration: Expected identifier in nested name. [error at 60] template<typename Action, typename ReduceOp, typename ArgN, . . . > hpx::future< decltype(Action(hpx::id_type, ArgN,. . . , std::size_t))> reduce_with_index (std::vector< hpx::id_type > const &ids, ReduceOp &&reduce_op, ArgN argN,. . .) ————————————————————^
If no parameter: Invalid C++ declaration: Expected "," or ">". [error at 60] template<typename Action, typename ReduceOp, typename ArgN, . . . > hpx::future< decltype(Action(hpx::id_type, ArgN,. . . , std::size_t))> reduce_with_index (std::vector< hpx::id_type > const &ids, ReduceOp &&reduce_op, ArgN argN,. . .) ————————————————————^

components

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/components/basename_registration.hpp

Defined in header hpx/components/basename_registration.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

template<typename **Client**>

std::vector<*Client*> **find_all_from_basename**(*std*::string base_name, *std*::size_t num_ids)

Return all registered clients from all localities from the given base name.

This function locates all ids which were registered with the given base name. It returns a list of futures representing those ids.

Return all registered ids from all localities from the given base name.

This function locates all ids which were registered with the given base name. It returns a list of futures representing those ids.

> **Note**
>
> The futures embedded in the returned client objects will become ready even if the event (for instance, binding the name to an id) has already happened in the past. This is important in order to reliably retrieve ids from a name, even if the name was already registered.

> **Note**
>
> The futures will become ready even if the event (for instance, binding the name to an id) has already happened in the past. This is important in order to reliably retrieve ids from a name, even if the name was already registered.

Template Parameters

`Client` – The client type to return

Parameters

- `base_name` – [in] The base name for which to retrieve the registered ids.
- `num_ids` – [in] The number of registered ids to expect.
- `base_name` – [in] The base name for which to retrieve the registered ids.
- `num_ids` – [in] The number of registered ids to expect.

Returns

A list of futures representing the ids which were registered using the given base name.

Returns

A list of futures representing the ids which were registered using the given base name.

template<typename **Client**>

std::vector<*Client*> `find_all_from_basename`(*hpx*::*launch*::sync_policy policy, *std*::string base_name, *std*::size_t num_ids)

template<typename **Client**>

std::vector<*Client*> `find_from_basename`(*std*::string base_name, *std*::vector<*std*::size_t> const &ids)

Return registered clients from the given base name and sequence numbers.

This function locates the ids which were registered with the given base name and the given sequence numbers. It returns a list of futures representing those ids.

Return registered ids from the given base name and sequence numbers.

This function locates the ids which were registered with the given base name and the given sequence numbers. It returns a list of futures representing those ids.

> **Note**
>
> The futures embedded in the returned client objects will become ready even if the event (for instance, binding the name to an id) has already happened in the past. This is important in order to reliably retrieve ids from a name, even if the name was already registered.

> **Note**
>
> The futures will become ready even if the event (for instance, binding the name to an id) has already happened in the past. This is important in order to reliably retrieve ids from a name, even if the name was already registered.

Template Parameters

`Client` – The client type to return

Parameters

- `base_name` – [in] The base name for which to retrieve the registered ids.
- `ids` – [in] The sequence numbers of the registered ids.
- `base_name` – [in] The base name for which to retrieve the registered ids.
- `ids` – [in] The sequence numbers of the registered ids.

Returns

A list of futures representing the ids which were registered using the given base name and sequence numbers.

Returns

A list of futures representing the ids which were registered using the given base name and sequence numbers.

template<typename `Client`>

std::vector<*Client*> `find_from_basename`(*hpx*::*launch*::sync_policy policy, *std*::string base_name, *std*::vector<*std*::size_t> const &ids)

template<typename `Client`>

Client `find_from_basename`(*std*::string base_name, *std*::size_t sequence_nr)

Return registered id from the given base name and sequence number.

This function locates the id which was registered with the given base name and the given sequence number. It returns a future representing those id.

This function locates the id which was registered with the given base name and the given sequence number. It returns a future representing those id.

> **Note**
>
> The future embedded in the returned client object will become ready even if the event (for instance, binding the name to an id) has already happened in the past. This is important in order to reliably retrieve ids from a name, even if the name was already registered.

> **Note**
>
> The future will become ready even if the event (for instance, binding the name to an id) has already happened in the past. This is important in order to reliably retrieve ids from a name, even if the name was already registered.

Template Parameters
`Client` – The client type to return

Parameters

- **`base_name`** – [in] The base name for which to retrieve the registered ids.
- **`sequence_nr`** – [in] The sequence number of the registered id.
- **`base_name`** – [in] The base name for which to retrieve the registered ids.
- **`sequence_nr`** – [in] The sequence number of the registered id.

Returns
A representing the id which was registered using the given base name and sequence numbers.

Returns
A representing the id which was registered using the given base name and sequence numbers.

template<typename **Client**>

Client **find_from_basename**(*hpx*::*launch*::sync_policy policy, *std*::string base_name, *std*::size_t sequence_nr)

template<typename **Client**, typename **Stub**, typename **Data**>

hpx::*future*<bool> **register_with_basename**(*std*::string base_name, *components*::*client_base*<*Client*, *Stub*, *Data*> &client, *std*::size_t sequence_nr)

Register the id wrapped in the given client using the given base name.

The function registers the object the given client refers to using the provided base name.

> **Note**
>
> The operation will fail if the given sequence number is not unique.

Template Parameters
`Client` – The client type to register

Parameters

- **`base_name`** – [in] The base name for which to retrieve the registered ids.
- **`client`** – [in] The client which should be registered using the given base name.

- **sequence_nr** – [in, optional] The sequential number to use for the registration of the id. This number has to be unique system-wide for each registration using the same base name. The default is the current locality identifier. Also, the sequence numbers have to be consecutive starting from zero.

Returns
A future representing the result of the registration operation itself.

template<typename **Client**>

Client **unregister_with_basename**(*std*::string base_name, *std*::size_t sequence_nr = ~static_cast<*std*::size_t>(0))

Unregister the given id using the given base name.

Unregister the given base name.

The function unregisters the given ids using the provided base name.

The function unregisters the given ids using the provided base name.

Template Parameters
Client – The client type to return

Parameters

- **base_name** – [in] The base name for which to retrieve the registered ids.
- **sequence_nr** – [in, optional] The sequential number to use for the un-registration. This number has to be the same as has been used with *register_with_basename* before.
- **base_name** – [in] The base name for which to retrieve the registered ids.
- **sequence_nr** – [in, optional] The sequential number to use for the un-registration. This number has to be the same as has been used with *register_with_basename* before.

Returns
A future representing the result of the un-registration operation itself.

Returns
A future representing the result of the un-registration operation itself.

hpx::components::client

Defined in header hpx/components.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **Component**, typename **Data** = void>

class **client** : public *hpx*::*components*::*client_base*<*client*<*Component*, void>, *Component*, void>

The client class is a wrapper that manages a distributed component. It extends *client_base* with specific `Component` and `Data` types.

Template Parameters

- **Component** – The type of the component.

- **Data** – The type of the data associated with the client (default is void).

hpx/components/get_ptr.hpp

Defined in header hpx/components/get_ptr.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

template<typename **Component**>

hpx::*future*<*std*::shared_ptr<*Component*>> **get_ptr**(*hpx*::id_type const &id)

Returns a future referring to the pointer to the underlying memory of a component.

The function hpx::get_ptr can be used to extract a future referring to the pointer to the underlying memory of a given component.

> **Note**
>
> This function will successfully return the requested result only if the given component is currently located on the calling locality. Otherwise, the function will raise an error.

> **Note**
>
> The component instance the returned pointer refers to can not be migrated as long as there is at least one copy of the returned shared_ptr alive.

Parameters

id – [in] The global id of the component for which the pointer to the underlying memory should be retrieved.

Template Parameters

Component – The type of the server side component.

Returns

This function returns a future representing the pointer to the underlying memory for the component instance with the given *id*.

template<typename **Derived**, typename **Stub**, typename **Data**>

hpx::*future*<*std*::shared_ptr<typename *components*::*client_base*<*Derived*, *Stub*, *Data*>::server_component_type>> **get_ptr**(*components*::*cl* *Stub*, *Data*> const &c)

Returns a future referring to the pointer to the underlying memory of a component.

The function hpx::get_ptr can be used to extract a future referring to the pointer to the underlying memory of a given component.

> **Note**
>
> This function will successfully return the requested result only if the given component is currently located on the calling locality. Otherwise, the function will raise an error.

> **Note**
>
> The component instance the returned pointer refers to can not be migrated as long as there is at least one copy of the returned shared_ptr alive.

Parameters

c – [in] A client side representation of the component for which the pointer to the underlying memory should be retrieved.

Returns

This function returns a future representing the pointer to the underlying memory for the component instance with the given *id*.

template<typename **Component**>

std::shared_ptr<*Component*> **get_ptr**(*launch*::sync_policy p, *hpx*::id_type const &id, *error_code* &ec = *throws*)

Returns the pointer to the underlying memory of a component.

The function hpx::get_ptr_sync can be used to extract the pointer to the underlying memory of a given component.

> **Note**
>
> This function will successfully return the requested result only if the given component is currently located on the requesting locality. Otherwise, the function will raise and error.

> **Note**
>
> The component instance the returned pointer refers to can not be migrated as long as there is at least one copy of the returned shared_ptr alive.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **p** – [in] The parameter *p* represents a placeholder type to turn make the call synchronous.
- **id** – [in] The global id of the component for which the pointer to the underlying memory should be retrieved.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Template Parameters

Component – The only template parameter has to be the type of the server side component.

Returns

This function returns the pointer to the underlying memory for the component instance with the given *id*.

template<typename **Derived**, typename **Stub**, typename **Data**>

std::shared_ptr<typename *components*::*client_base*<*Derived*, *Stub*, *Data*>::server_component_type> **get_ptr**(*launch*::sync_policy p, *components*::*client_base*<*Derived*, *Stub*, *Data*> const &c, *error_code* &ec = *throws*)

Returns the pointer to the underlying memory of a component.

The function hpx::get_ptr_sync can be used to extract the pointer to the underlying memory of a given component.

> **Note**
>
> This function will successfully return the requested result only if the given component is currently located on the requesting locality. Otherwise, the function will raise and error.

> **Note**
>
> The component instance the returned pointer refers to can not be migrated as long as there is at least one copy of the returned shared_ptr alive.

> **Note**

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **p** – [in] The parameter *p* represents a placeholder type to turn make the call synchronous.
- **c** – [in] A client side representation of the component for which the pointer to the underlying memory should be retrieved.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns the pointer to the underlying memory for the component instance with the given *id*.

hpx/components/basename_registration_fwd.hpp

Defined in header hpx/components/basename_registration_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

hpx::*future*<bool> **register_with_basename**(*std*::string base_name, *hpx*::id_type const &id, *std*::size_t sequence_nr = ~static_cast<*std*::size_t>(0))

Register the given id using the given base name.

The function registers the given ids using the provided base name.

Note

The operation will fail if the given sequence number is not unique.

Parameters

- **base_name** – [in] The base name for which to retrieve the registered ids.
- **id** – [in] The id to register using the given base name.
- **sequence_nr** – [in, optional] The sequential number to use for the registration of the id. This number has to be unique system-wide for each registration using the same base name. The default is the current locality identifier. Also, the sequence numbers have to be consecutive starting from zero.

Returns

A future representing the result of the registration operation itself.

bool **register_with_basename**(*hpx*::*launch*::sync_policy, *std*::string base_name, *hpx*::id_type const &id, *std*::size_t sequence_nr = ~static_cast<*std*::size_t>(0), *error_code* &ec = *throws*)

hpx::*future*<bool> **register_with_basename**(*std*::string base_name, *hpx*::*future*<*hpx*::id_type> f, *std*::size_t sequence_nr = ~static_cast<*std*::size_t>(0))

Register the id wrapped in the given future using the given base name.

The function registers the object the given future refers to using the provided base name.

> **Note**
>
> The operation will fail if the given sequence number is not unique.

Parameters

- **base_name** – [in] The base name for which to retrieve the registered ids.
- **f** – [in] The future which should be registered using the given base name.
- **sequence_nr** – [in, optional] The sequential number to use for the registration of the id. This number has to be unique system-wide for each registration using the same base name. The default is the current locality identifier. Also, the sequence numbers have to be consecutive starting from zero.

Returns

A future representing the result of the registration operation itself.

hpx::components::client_base

Defined in header hpx/components.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **Derived**, typename **Stub**, typename **Data**>

class **client_base** : public *hpx*::*components*::stub_base<ServerComponent>

This class template serves as a base class for client components, providing common functionality such as managing shared state, ID retrieval, and asynchronous operations.

Template Parameters

- **Derived** – The derived client component type.
- **Stub** – The stub type used for communication.
- **Data** – The extra data type used for additional information.

components_base

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::components::component

Defined in header hpx/components.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::components::component_base*

template<typename **Component**>

class **component** : public *Component*

The *component* class wraps around a given component type, adding additional type aliases and constructors. It inherits from the specified component type.

hpx::components::component_base

Defined in header hpx/components.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::components::component*

template<typename **Component**>

class **component_base** : public *hpx*::*components*::detail::base_component

component_base serves as a base class for components. It provides common functionality needed by components, such as address and ID retrieval. The template parameter *Component* specifies the derived component type.

Subclassed by hpx::performance_counters::base_performance_counter< test_counter >, Component, Component, Component

HPX_REGISTER_COMMANDLINE_MODULE

Defined in header hpx/components.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

> **Warning**
>
> doxygenfunction: Cannot find function "HPX_REGISTER_COMMANDLINE_MODULE" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx/components_base/component_type.hpp

Defined in header hpx/components_base/component_type.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Defines

HPX_COMPONENT_ENUM_TYPE_ENUM_DEPRECATION_MSG

HPX_FACTORY_STATE_ENUM_DEPRECATION_MSG

namespace **hpx**

namespace **components**

Typedefs

using **component_deleter_type** = void (*)(*hpx*::*naming*::gid_type const&, *hpx*::*naming*::address const&)

Enums

enum class **component_enum_type** : *naming*::component_type

Values:

enumerator **invalid**

enumerator **runtime_support**

enumerator **plain_function**

enumerator **base_lco**

enumerator **base_lco_with_value_unmanaged**

enumerator **base_lco_with_value**

enumerator **latch**

enumerator **barrier**

enumerator **promise**

enumerator **agas_locality_namespace**

enumerator **agas_primary_namespace**

enumerator **agas_component_namespace**

enumerator **agas_symbol_namespace**

enumerator **supervision_manager**

enumerator **last**

enumerator **first_dynamic**

enum class **factory_state** : *std*::uint8_t

Values:

enumerator **enabled**

enumerator **disabled**

enumerator **check**

Functions

constexpr *naming*::component_type **to_int**(*component_enum_type* t) noexcept

constexpr int **to_int**(*factory_state* t) noexcept

bool &**enabled**(component_type type)

util::atomic_count &**instance_count**(component_type type)

component_deleter_type &**deleter**(component_type type)

bool **enumerate_instance_counts**(*hpx*::*move_only_function*<bool(component_type)> const &f)

std::string **get_component_type_name**(component_type type)

Return the string representation for a given component type id.

constexpr component_type **get_base_type**(component_type t) noexcept

The lower short word of the component type is the type of the component exposing the actions.

constexpr component_type **get_derived_type**(component_type t) noexcept

The upper short word of the component is the actual component type.

constexpr component_type **derived_component_type**(component_type derived, component_type base) noexcept

A component derived from a base component exposing the actions needs to have a specially formatted component type.

constexpr bool **types_are_compatible**(component_type lhs, component_type rhs) noexcept

Verify the two given component types are matching (compatible) template<typename **Component**, typename **Enable** = void>

char const ***get_component_name**() noexcept

template<typename **Component**, typename **Enable** = void>

char const ***get_component_base_name**() noexcept

template<typename **Component**>

component_type **get_component_type**() noexcept

template<typename **Component**>

void **set_component_type**(component_type type)

Variables

constexpr *component_enum_type* **component_invalid** = *component_enum_type*::*invalid*

constexpr *component_enum_type* **component_runtime_support** = *component_enum_type*::*runtime_support*

constexpr *component_enum_type* **component_plain_function** = *component_enum_type*::*plain_function*

constexpr *component_enum_type* **component_base_lco** = *component_enum_type*::*base_lco*

constexpr *component_enum_type* **component_base_lco_with_value_unmanaged** = *component_enum_type*::*base_lco_with_value_unmanaged*

constexpr *component_enum_type* **component_base_lco_with_value** = *component_enum_type*::*base_lco_with_value* constexpr *component_enum_type* **component_latch** = *component_enum_type*::*latch*

constexpr *component_enum_type* **component_barrier** = *component_enum_type*::*barrier*

constexpr *component_enum_type* **component_promise** = *component_enum_type*::*promise*

constexpr *component_enum_type* **component_agas_locality_namespace** = *component_enum_type*::*agas_locality_namespace*

constexpr *component_enum_type* **component_agas_primary_namespace** = *component_enum_type*::*agas_primary_namespace*

constexpr *component_enum_type* **component_agas_component_namespace** = *component_enum_type*::*agas_component_namespace*

constexpr *component_enum_type* **component_agas_symbol_namespace** = *component_enum_type*::*agas_symbol_namespace*

constexpr *component_enum_type* **component_last** = *component_enum_type*::*last*

constexpr *component_enum_type* **component_first_dynamic** = *component_enum_type*::*first_dynamic*

constexpr *factory_state* **factory_enabled** = *factory_state*::*enabled*

constexpr *factory_state* **factory_disabled** = *factory_state*::*disabled*

constexpr *factory_state* **factory_check** = *factory_state*::*check*

constexpr int **component_type_mask** = 0x3FF constexpr int **component_type_shift** = 10

hpx/components_base/get_lva.hpp

Defined in header hpx/components_base/get_lva.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

template<typename **Component**, typename **Enable** = void>

struct **get_lva**

#include <get_lva.hpp> The get_lva template is a helper structure allowing to convert a local virtual address as stored in a local address (returned from the function *agas::addressing_service::resolve*) to the address of the component implementing the action.

The default implementation uses the template argument *Component* to deduce the type wrapping the component implementing the action. This is used to get the needed address.

Template Parameters

Component – This is the type of the component implementing the action to execute.

Public Static Functions

static inline constexpr *Component* ***call**(*naming*::address_type lva) noexcept

hpx/components_base/agas_interface.hpp

Defined in header hpx/components_base/agas_interface.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **agas**

Functions

bool **is_console**()

bool **is_connecting**(*naming*::gid_type const &locality = *naming*::invalid_gid)

bool **register_name**(*launch*::sync_policy, *std*::string const &name, *naming*::gid_type const &gid, *error_code* &ec = *throws*)

bool **register_name**(*launch*::sync_policy, *std*::string const &name, *hpx*::id_type const &id, *error_code* &ec = *throws*)

hpx::*future*<bool> **register_name**(*std*::string const &name, *hpx*::id_type const &id)

hpx::id_type **unregister_name**(*launch*::sync_policy, *std*::string const &name, *error_code* &ec = *throws*)

hpx::*future*<*hpx*::id_type> **unregister_name**(*std*::string const &name)

hpx::id_type **resolve_name**(*launch*::sync_policy, *std*::string const &name, *error_code* &ec = *throws*)

hpx::*future*<*hpx*::id_type> **resolve_name**(*std*::string const &name)

hpx::*future*<*std*::uint32_t> **get_num_localities**(*naming*::component_type type = *naming*::component_invalid)

std::uint32_t **get_num_localities**(*launch*::sync_policy, *naming*::component_type type, *error_code* &ec = *throws*)

inline *std*::uint32_t **get_num_localities**(*launch*::sync_policy, *error_code* &ec = *throws*)

std::string **get_component_type_name**(*naming*::component_type type, *error_code* &ec = *throws*) *hpx*::*future*<*std*::vector<*std*::uint32_t>> **get_num_threads**()

std::vector<*std*::uint32_t> **get_num_threads**(*launch*::sync_policy, *error_code* &ec = *throws*)

hpx::*future*<*std*::uint32_t> **get_num_overall_threads**()

std::uint32_t **get_num_overall_threads**(*launch*::sync_policy, *error_code* &ec = *throws*)

std::uint32_t **get_locality_id**(*error_code* &ec = *throws*)

inline *hpx*::*naming*::gid_type **get_locality**()

std::vector<*std*::uint32_t> **get_all_locality_ids**(*naming*::component_type type, *error_code* &ec = *throws*)

inline *std*::vector<*std*::uint32_t> **get_all_locality_ids**(*error_code* &ec = *throws*)

bool **is_local_address_cached**(*naming*::gid_type const &gid, *error_code* &ec = *throws*)

bool **is_local_address_cached**(*naming*::gid_type const &gid, *naming*::address &addr, *error_code* &ec = *throws*)

bool **is_local_address_cached**(*naming*::gid_type const &gid, *naming*::address &addr, *std*::pair<bool, *components*::pinned_ptr> &r, *hpx*::*move_only_function*<*std*::pair<bool, *components*::pinned_ptr>(*naming*::address const&)> &&f, *error_code* &ec = *throws*)

inline bool **is_local_address_cached**(*hpx*::id_type const &id, *error_code* &ec = *throws*)

inline bool **is_local_address_cached**(*hpx*::id_type const &id, *naming*::address &addr, *error_code* &ec = *throws*) inline bool **is_local_address_cached**(*hpx*::id_type const &id, *naming*::address &addr, *std*::pair<bool, *components*::pinned_ptr> &r, *hpx*::*move_only_function*<*std*::pair<bool, *components*::pinned_ptr>(*naming*::address const&)> &&f, *error_code* &ec = *throws*)

void **update_cache_entry**(*naming*::gid_type const &gid, *naming*::address const &addr, *std*::uint64_t count = 0, *std*::uint64_t offset = 0, *error_code* &ec = *throws*)

bool **is_local_lva_encoded_address**(*naming*::gid_type const &gid)

inline bool **is_local_lva_encoded_address**(*hpx*::id_type const &id)

hpx::future_or_value<*naming*::address> **resolve_async**(*hpx*::id_type const &id)

hpx::*future*<*naming*::address> **resolve**(*hpx*::id_type const &id)

naming::address **resolve**(*launch*::sync_policy, *hpx*::id_type const &id, *error_code* &ec = *throws*)

bool **resolve_local**(*naming*::gid_type const &gid, *naming*::address &addr, *error_code* &ec = *throws*)

bool **resolve_cached**(*naming*::gid_type const &gid, *naming*::address &addr)

hpx::*future*<bool> **bind**(*naming*::gid_type const &gid, *naming*::address const &addr, *std*::uint32_t locality_id)

bool **bind**(*launch*::sync_policy, *naming*::gid_type const &gid, *naming*::address const &addr, *std*::uint32_t locality_id, *error_code* &ec = *throws*)

hpx::*future*<bool> **bind**(*naming*::gid_type const &gid, *naming*::address const &addr, *naming*::gid_type const &locality_)

bool **bind**(*launch*::sync_policy, *naming*::gid_type const &gid, *naming*::address const &addr, *naming*::gid_type const &locality_, *error_code* &ec = *throws*) *hpx*::*future*<*naming*::address> **unbind**(*naming*::gid_type const &gid, *std*::uint64_t count = 1)

naming::address **unbind**(*launch*::sync_policy, *naming*::gid_type const &gid, *std*::uint64_t count = 1, *error_code* &ec = *throws*)

bool **bind_gid_local**(*naming*::gid_type const &gid, *naming*::address const &addr, *error_code* &ec = *throws*)

void **unbind_gid_local**(*naming*::gid_type const &gid, *error_code* &ec = *throws*)

bool **bind_range_local**(*naming*::gid_type const &gid, *std*::size_t count, *naming*::address const &addr, *std*::size_t offset, *error_code* &ec = *throws*)

void **unbind_range_local**(*naming*::gid_type const &gid, *std*::size_t count, *error_code* &ec = *throws*)

void **garbage_collect_non_blocking**(*error_code* &ec = *throws*)

void **garbage_collect**(*error_code* &ec = *throws*)

void **garbage_collect_non_blocking**(*hpx*::id_type const &id, *error_code* &ec = *throws*)

Invoke an asynchronous garbage collection step on the given target locality.

void **garbage_collect**(*hpx*::id_type const &id, *error_code* &ec = *throws*)

Invoke a synchronous garbage collection step on the given target locality.

hpx::id_type **get_console_locality**(*error_code* &ec = *throws*)

Return an id_type referring to the console locality.

naming::gid_type **get_next_id**(*std*::size_t count, *error_code* &ec = *throws*)

void **decref**(*naming*::gid_type const &id, *std*::int64_t credits, *error_code* &ec = *throws*)

hpx::future_or_value<*std*::int64_t> **incref**(*naming*::gid_type const &gid, *std*::int64_t credits, *hpx*::id_type const &keep_alive = *hpx*::invalid_id) *std*::int64_t **incref**(*launch*::sync_policy, *naming*::gid_type const &gid, *std*::int64_t credits = 1, *hpx*::id_type const &keep_alive = *hpx*::invalid_id, *error_code* &ec = *throws*)

std::int64_t **replenish_credits**(*naming*::gid_type &gid)

hpx::future_or_value<id_type> **get_colocation_id**(*hpx*::id_type const &id)

hpx::id_type **get_colocation_id**(*launch*::sync_policy, *hpx*::id_type const &id, *error_code* &ec = *throws*)

hpx::*future*<*hpx*::id_type> **on_symbol_namespace_event**(*std*::string const &name, bool call_for_past_events)

hpx::*future*<*std*::pair<*hpx*::id_type, *naming*::address>> **begin_migration**(*hpx*::id_type const &id)

bool **end_migration**(*hpx*::id_type const &id)

hpx::*future*<void> **mark_as_migrated**(*naming*::gid_type const &gid, *hpx*::*move_only_function*<*std*::pair<bool, *hpx*::*future*<void>>()> &&f, bool expect_to_be_marked_as_migrating)

std::pair<bool, *components*::pinned_ptr> **was_object_migrated**(*naming*::gid_type const &gid, *hpx*::*move_only_function*<*components*::pinned_ptr()> &&f)

void **unmark_as_migrated**(*naming*::gid_type const &gid, *hpx*::*move_only_function*<void()> &&f)

hpx::*future*<*std*::map<*std*::string, *hpx*::id_type>> **find_symbols**(*std*::string const &pattern = "*")

std::map<*std*::string, *hpx*::id_type> **find_symbols**(*hpx*::*launch*::sync_policy, *std*::string const &pattern = "*")

naming::component_type **register_factory**(*std*::uint32_t prefix, *std*::string const &name, *error_code* &ec = *throws*) *naming*::component_type **get_component_id**(*std*::string const &name, *error_code* &ec = *throws*)

void **destroy_component**(*naming*::gid_type const &gid, *naming*::address const &addr)

naming::address_type **get_primary_ns_lva**()

naming::address_type **get_symbol_ns_lva**()

naming::address_type **get_runtime_support_lva**()

struct agas_interface_functions &**agas_init**()

HPX_REGISTER_STARTUP_MODULE

Defined in header hpx/components.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Warning

doxygenfunction: Cannot find function "HPX_REGISTER_STARTUP_MODULE" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx/components_base/server/fixed_component_base.hpp

Defined in header hpx/components_base/server/fixed_component_base.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

template<typename **Component**>

class **fixed_component** : public *Component*

Public Types

using **type_holder** = *Component*

using **component_type** = *fixed_component*<*Component*>

using **derived_type** = *component_type*

using **heap_type** = detail::fixed_heap

Public Static Functions

static inline *Component* ***create**(*std*::size_t) noexcept

The function *create* is used for allocation and initialization of instances of the derived components.

static inline void **destroy**(*Component**, *std*::size_t = 1) noexcept

The function *destroy* is used for destruction and de-allocation of instances of the derived components.

template<typename **Component**>

class **fixed_component_base** : public *traits*::detail::fixed_component_tag

Public Types

using **wrapped_type** = *this_component_type*

using **base_type_holder** = *this_component_type*

using **wrapping_type** = *fixed_component*<*this_component_type*>

Public Functions

inline constexpr **fixed_component_base**(*std*::uint64_t msb, *std*::uint64_t lsb) noexcept

~fixed_component_base() = default

inline void **finalize**() const

finalize() will be called just before the instance gets destructed

inline *naming*::gid_type **get_base_gid**(*naming*::gid_type const &assign_gid = *naming*::invalid_gid) const

inline *hpx*::id_type **get_id**() const

inline *hpx*::id_type **get_unmanaged_id**() const

inline void **set_locality_id**(*std*::uint32_t locality_id, *error_code* &ec = *throws*)

Public Static Functions

static inline void **mark_as_migrated**() noexcept

static inline void **on_migrated**() noexcept

Private Types

using **this_component_type** = *std*::conditional_t<*std*::is_void_v<*Component*>, *fixed_component_base*, *Component*>

Private Functions

inline constexpr *Component* &**derived**() noexcept

inline constexpr *Component* const &**derived**() const noexcept

Private Members

mutable *naming*::gid_type **gid_**

std::uint64_t **msb_**

std::uint64_t **lsb_**

hpx/components_base/server/managed_component_base.hpp

Defined in header hpx/components_base/server/managed_component_base.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

Functions

template<typename **Component**, typename **Derived**>

void **intrusive_ptr_add_ref**(*managed_component*<*Component*, *Derived*> *p) noexcept

template<typename **Component**, typename **Derived**>

void **intrusive_ptr_release**(*managed_component*<*Component*, *Derived*> *p) noexcept

template<typename **Component**, typename **Derived**>

class **managed_component**

#include <managed_component_base.hpp> The *managed_component* template is used as an indirection layer for components allowing to gracefully handle the access to non-existing components.

Additionally, it provides memory management capabilities for the wrapping instances, and it integrates the memory management with the AGAS service. Every instance of a *managed_component* gets assigned a global id. The provided memory management allocates the *managed_component* instances from a special heap, ensuring fast allocation and avoids a full network round trip to the AGAS service for each of the allocated instances.

Template Parameters

- **Component** – Component type
- **Derived** – Most derived component type

Public Types

using **derived_type** = *std*::conditional_t<*std*::is_void_v<*Derived*>, *managed_component*, *Derived*>

using **wrapped_type** = *Component*

using **type_holder** = *Component*

using **base_type_holder** = typename *Component*::base_type_holder

using **heap_type** = detail::wrapper_heap_list<detail::fixed_wrapper_heap<*derived_type*>>

using **value_type** = *derived_type*

Public Functions

managed_component(*managed_component* const&) = delete

managed_component(*managed_component*&&) = delete *managed_component* &**operator=**(*managed_component* const&) = delete

managed_component &**operator=**(*managed_component*&&) = delete

inline explicit **managed_component**(*Component* *comp)

Construct a *managed_component* instance holding a wrapped instance. This constructor takes ownership of the passed pointer.

Parameters

comp – [in] The pointer to the wrapped instance. The *managed_component* takes ownership of this pointer.

inline **managed_component**()

template<typename **T**, typename ...**Ts**, typename **Enable** = *std*::enable_if_t<!*std*::is_same_v<*std*::decay_t<*T*>, *managed_component*>>>

inline explicit **managed_component**(*T* &&t, *Ts*&&... ts)

inline **~managed_component**()

inline constexpr *Component* ***get**() noexcept

Return a pointer to the wrapped instance.

> **Note**
>
> Caller must check validity of returned pointer

inline constexpr *Component* const ***get**() const noexcept

inline *Component* ***get_checked**()

inline *Component* const ***get_checked**() const

inline *Component* ***operator->**()

inline *Component* const ***operator->**() const

inline *Component* &**operator***()

inline *Component* const &**operator***() const

inline *hpx*::id_type **get_unmanaged_id**() const

Return the global id of this *future* instance.

inline *naming*::gid_type **get_base_gid**(*naming*::gid_type const &assign_gid = *naming*::invalid_gid) const

Public Static Functions

static inline constexpr void **finalize**() noexcept

Protected Attributes

Component ***component_** = nullptr

Friends

template<typename **C**, typename **D**>

friend void **intrusive_ptr_add_ref**(*managed_component*<*C*, *D*> *p) noexcept

template<typename **C**, typename **D**>

friend void **intrusive_ptr_release**(*managed_component*<*C*, *D*> *p) noexcept

template<typename **Component**, typename **Wrapper**, typename **CtorPolicy**, typename **DtorPolicy**>

class **managed_component_base** : public *hpx*::*components*::detail::base_managed_component

Public Types

using **this_component_type** = *std*::conditional_t<*std*::is_void_v<*Component*>, *managed_component_base*, *Component*>

using **wrapped_type** = *this_component_type*

using **has_managed_component_base** = void

using **ctor_policy** = *CtorPolicy*

using **dtor_policy** = *DtorPolicy*

using **wrapping_type** = *managed_component*<*Component*, *Wrapper*>

using **base_type_holder** = *Component*

Public Functions

managed_component_base(*managed_component_base* const&) = delete

managed_component_base(*managed_component_base*&&) = delete

managed_component_base &**operator=**(*managed_component_base* const&) = delete

managed_component_base &**operator=**(*managed_component_base*&&) = delete

constexpr **managed_component_base**() noexcept = default inline explicit **managed_component_base**(*managed_component*<*Component*, *Wrapper*> *back_ptr) noexcept

inline **~managed_component_base**()

hpx::id_type **get_unmanaged_id**() const

hpx::id_type **get_id**() const

Protected Functions

naming::gid_type **get_base_gid**() const

inline void **set_back_ptr**(*components*::*managed_component*<*Component*, *Wrapper*> *bp) noexcept

Private Members

managed_component<*Component*, *Wrapper*> ***back_ptr_** = nullptr

Friends

friend struct detail_adl_barrier::init

namespace **detail_adl_barrier**

template<>

struct **init**<*traits*::construct_with_back_ptr>

Public Static Functions

template<typename **Component**, typename **Managed**>

static inline constexpr void **call**(*Component**, *Managed**) noexcept

template<typename **Component**, typename **Managed**, typename ...**Ts**>

static inline void **call_new**(*Component* *&component, *Managed* *this_, *Ts*&&... vs)

template<>

struct **init**<*traits*::construct_without_back_ptr>

Public Static Functions

template<typename **Component**, typename **Managed**>

static inline void **call**(*Component* *component, *Managed* *this_)

template<typename **Component**, typename **Managed**, typename ...**Ts**>

static inline void **call_new**(*Component* *&component, *Managed* *this_, *Ts*&&... vs)

template<>

struct **destroy_backptr**<*traits*::managed_object_is_lifetime_controlled>

Public Static Functions

template<typename **BackPtr**>

static inline void **call**(*BackPtr* *back_ptr)

template<>

struct **destroy_backptr**<*traits*::managed_object_controls_lifetime>

Public Static Functions

template<typename **BackPtr**>

static inline constexpr void **call**(*BackPtr**) noexcept

template<>

struct **manage_lifetime**<*traits*::managed_object_is_lifetime_controlled>

Public Static Functions

template<typename **Component**>

static inline constexpr void **call**(*Component**) noexcept

template<typename **Component**>

static inline void **addref**(*Component* *component) noexcept

template<typename **Component**>

static inline void **release**(*Component* *component) noexcept

template<>

struct **manage_lifetime**<*traits*::managed_object_controls_lifetime>

Public Static Functions

template<typename **Component**>

static inline void **call**(*Component* *component) noexcept(noexcept(*component*->finalize()))

template<typename **Component**>

static inline constexpr void **addref**(*Component**) noexcept template<typename **Component**>

static inline constexpr void **release**(*Component**) noexcept

hpx/components_base/server/migration_support.hpp

Defined in header hpx/components_base/server/migration_support.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

template<typename **BaseComponent**, typename **Mutex** = *hpx*::*spinlock*>

struct **migration_support** : public *BaseComponent*

#include <migration_support.hpp> This hook has to be inserted into the derivation chain of any component for it to support migration.

Public Types

using **decorates_action** = void

Public Functions

inline **migration_support**()

template<typename **T**, typename ...**Ts**, typename = *std*::enable_if_t<!*std*::is_same_v<*std*::decay_t<*T*>, *migration_support*>>>

inline explicit **migration_support**(*T* &&t, *Ts*&&... ts)

migration_support(*migration_support* const&) = default

migration_support(*migration_support*&&) = default *migration_support* &**operator=**(*migration_support* const&) = default

migration_support &**operator=**(*migration_support*&&) = default

~migration_support() = default

inline *naming*::gid_type **get_base_gid**(*naming*::gid_type const &assign_gid = *naming*::invalid_gid) const

inline bool **pin**() noexcept

inline bool **unpin**()

inline *std*::uint32_t **pin_count**() const noexcept

inline void **mark_as_migrated**()

inline *hpx*::*future*<void> **mark_as_migrated**(*hpx*::id_type const &to_migrate)

inline void **unmark_as_migrated**(*hpx*::id_type const &to_migrate)

Public Static Functions

static inline constexpr bool **supports_migration**() noexcept

static inline constexpr void **on_migrated**() noexcept

template<typename **F**>

static inline *threads*::thread_function_type **decorate_action**(*naming*::address_type lva, *F* &&f)

static inline *std*::pair<bool, *components*::pinned_ptr> **was_object_migrated**(*hpx*::*naming*::gid_type const &id, *naming*::address_type lva)

Protected Functions

inline *threads*::thread_result_type **thread_function**(*threads*::thread_function_type &&f, *components*::pinned_ptr, *threads*::*thread_restart_state* state)

Private Types

using **base_type** = *BaseComponent*

using **this_component_type** = typename *base_type*::this_component_type

Private Members

hpx::intrusive_ptr<detail::migration_support_data<*Mutex*>> **data_**

hpx::*promise*<void> **trigger_migration_**

bool **started_migration_** = false

bool **was_marked_for_migration_** = false

compute

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/compute/host/target_distribution_policy.hpp

Defined in header hpx/compute/host/target_distribution_policy.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

distribution_policies

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/distribution_policies/unwrapping_result_policy.hpp

Defined in header hpx/distribution_policies/unwrapping_result_policy.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

struct **unwrapping_result_policy**

#include <unwrapping_result_policy.hpp> This class is a distribution policy that can be used with actions that return futures. For those actions it is possible to apply certain optimizations if the action is invoked synchronously.

Public Functions

inline explicit **unwrapping_result_policy**(id_type const &id)

template<typename **Client**, typename **Stub**, typename **Data**>

inline explicit **unwrapping_result_policy**(*client_base*<*Client*, *Stub*, *Data*> const &client)

template<typename **Action**, typename ...**Ts**>

inline *async_result*<*Action*>::type **async**(*launch* policy, *Ts*&&... vs) const

template<typename **Action**, typename ...**Ts**>

inline *async_result*<*Action*>::type **async**(*launch*::sync_policy, *Ts*&&... vs) const template<typename **Action**, typename **Callback**, typename ...**Ts**>

inline *async_result*<*Action*>::type **async_cb**(*launch* policy, *Callback* &&cb, *Ts*&&... vs) const

template<typename **Action**, typename **Continuation**, typename ...**Ts**>

inline bool **apply**(*Continuation* &&c, *launch* policy, *Ts*&&... vs) const

> **Note**
>
> This function is part of the invocation policy implemented by this class

template<typename **Action**, typename ...**Ts**>

inline bool **apply**(*launch* policy, *Ts*&&... vs) const

template<typename **Action**, typename **Continuation**, typename **Callback**, typename ...**Ts**>

inline bool **apply_cb**(*Continuation* &&c, *launch* policy, *Callback* &&cb, *Ts*&&... vs) const

> **Note**
>
> This function is part of the invocation policy implemented by this class

template<typename **Action**, typename **Callback**, typename ...**Ts**>

inline bool **apply_cb**(*launch* policy, *Callback* &&cb, *Ts*&&... vs) const

inline *hpx*::id_type const &**get_next_target**() const

template<typename **Action**>

struct **async_result**

Public Types

using **type** = typename *traits*::promise_local_result<typename *hpx*::*traits*::extract_action<*Action*>::remote_result_type>::type

hpx/distribution_policies/default_distribution_policy.hpp

Defined in header hpx/distribution_policies/default_distribution_policy.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

Variables

default_distribution_policy const **default_layout**

A predefined instance of the default *distribution_policy*. It will represent the local locality and will place all items to create here.

struct **default_distribution_policy**

#include <default_distribution_policy.hpp> This class specifies the parameters for a simple distribution policy to use for creating (and evenly distributing) a given number of items on a given set of localities.

Public Functions

constexpr **default_distribution_policy**() = default

Default-construct a new instance of a default_distribution_policy. This policy will represent one locality (the local locality).

inline *default_distribution_policy* **operator()**(*std*::vector<id_type> locs) const

Create a new *default_distribution* policy representing the given set of localities.

Parameters

locs – [in] The list of localities the new instance should represent

inline *default_distribution_policy* **operator()**(id_type loc) const

Create a new *default_distribution* policy representing the given locality

Parameters

loc – [in] The locality the new instance should represent template<typename **Component**, typename ...**Ts**>

inline *hpx*::*future*<*hpx*::id_type> **create**(*Ts*&&... vs) const

Create one object on one of the localities associated by this policy instance

Note

This function is part of the placement policy implemented by this class

Parameters

vs – [in] The arguments which will be forwarded to the constructor of the new object.

Returns

A future holding the global address which represents the newly created object

template<bool **WithCount**, typename **Component**, typename ...**Ts**>

inline *hpx*::*future*<*std*::vector<bulk_locality_result>> **bulk_create**(*std*::size_t count, *Ts*&&... vs) const

Create multiple objects on the localities associated by this policy instance

Note

This function is part of the placement policy implemented by this class

Parameters

- **count** – [in] The number of objects to create
- **vs** – [in] The arguments which will be forwarded to the constructors of the new objects.

Returns

A future holding the list of global addresses that represent the newly created objects

template<typename **Action**, typename ...**Ts**>

inline *async_result*<*Action*>::type **async**(*launch* policy, *Ts*&&... vs) const

template<typename **Action**, typename **Callback**, typename ...**Ts**>

inline *async_result*<*Action*>::type **async_cb**(*launch* policy, *Callback* &&cb, *Ts*&&... vs) const

Note

This function is part of the invocation policy implemented by this class

template<typename **Action**, typename **Continuation**, typename ...**Ts**>

inline bool **apply**(*Continuation* &&c, *launch* policy, *Ts*&&... vs) const

> **Note**
>
> This function is part of the invocation policy implemented by this class

template<typename **Action**, typename ...**Ts**>

inline bool **apply**(*threads*::*thread_priority* priority, *Ts*&&... vs) const

template<typename **Action**, typename **Continuation**, typename **Callback**, typename ...**Ts**>

inline bool **apply_cb**(*Continuation* &&c, *launch* policy, *Callback* &&cb, *Ts*&&... vs) const

> **Note**
>
> This function is part of the invocation policy implemented by this class

template<typename **Action**, typename **Callback**, typename ...**Ts**>

inline bool **apply_cb**(*launch* policy, *Callback* &&cb, *Ts*&&... vs) const

inline *std*::size_t **get_num_localities**() const

Returns the number of associated localities for this distribution policy

> **Note**
>
> This function is part of the creation policy implemented by this class

inline *hpx*::id_type **get_next_target**() const

Returns the locality which is anticipated to be used for the next async operation

template<typename **Action**>

struct **async_result**

#include <default_distribution_policy.hpp>

> **Note**
>
> This function is part of the invocation policy implemented by this class

Public Types

using **type** = *hpx*::*future*<typename *traits*::promise_local_result<typename *hpx*::*traits*::extract_action<*Action*>::remote_result_type>::type>

hpx/distribution_policies/binpacking_distribution_policy.hpp

Defined in header hpx/distribution_policies/binpacking_distribution_policy.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

Variables

constexpr char const *const **default_binpacking_counter_name** = "/runtime{locality/total}/count/component@"

binpacking_distribution_policy const **binpacked**

A predefined instance of the binpacking *distribution_policy*. It will represent the local locality and will place all items to create here.

struct **binpacking_distribution_policy**

#include <binpacking_distribution_policy.hpp> This class specifies the parameters for a binpacking distribution policy to use for creating a given number of items on a given set of localities. The binpacking policy will distribute the new objects in a way such that each of the localities will equalize the number of overall objects of this type based on a given criteria (by default this criteria is the overall number of objects of this type).

Public Functions

inline **binpacking_distribution_policy**()

Default-construct a new instance of a binpacking_distribution_policy. This policy will represent one locality (the local locality).

inline *binpacking_distribution_policy* **operator()**(*std*::vector<id_type> const &locs, char const *perf_counter_name = *default_binpacking_counter_name*) const

Create a new *default_distribution* policy representing the given set of localities.

Parameters

- **locs** – [in] The list of localities the new instance should represent
- **perf_counter_name** – [in] The name of the performance counter which should be used as the distribution criteria (by default the overall number of existing instances of the given component type will be used).

inline *binpacking_distribution_policy* **operator()**(*std*::vector<id_type> &&locs, char const *perf_counter_name = *default_binpacking_counter_name*) const

Create a new *default_distribution* policy representing the given set of localities.

Parameters

- **locs** – [in] The list of localities the new instance should represent
- **perf_counter_name** – [in] The name of the performance counter which should be used as the distribution criteria (by default the overall number of existing instances of the given component type will be used).

inline *binpacking_distribution_policy* **operator()**(id_type const &loc, char const *perf_counter_name = *default_binpacking_counter_name*) const

Create a new *default_distribution* policy representing the given locality

Parameters

- **loc** – [in] The locality the new instance should represent
- **perf_counter_name** – [in] The name of the performance counter that should be used as the distribution criteria (by default the overall number of existing instances of the given component type will be used).

template<typename **Component**, typename ...**Ts**>

inline *hpx*::*future*<*hpx*::id_type> **create**(*Ts*&&... vs) const

Create one object on one of the localities associated by this policy instance

Parameters

vs – [in] The arguments which will be forwarded to the constructor of the new object.

Returns

A future holding the global address which represents the newly created object

template<bool **WithCount**, typename **Component**, typename ...**Ts**>

inline *hpx*::*future*<*std*::vector<bulk_locality_result>> **bulk_create**(*std*::size_t count, *Ts*&&... vs) const

Create multiple objects on the localities associated by this policy instance

Parameters

- **count** – [in] The number of objects to create
- **vs** – [in] The arguments which will be forwarded to the constructors of the new objects.

Returns

A future holding the list of global addresses which represent the newly created objects

inline *std*::string const &**get_counter_name**() const

Returns the name of the performance counter associated with this policy instance.

inline *std*::size_t **get_num_localities**() const

Returns the number of associated localities for this distribution policy

> **Note**
>
> This function is part of the creation policy implemented by this class

hpx/distribution_policies/colocating_distribution_policy.hpp

Defined in header hpx/distribution_policies/colocating_distribution_policy.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

Variables

colocating_distribution_policy const **colocated**

A predefined instance of the co-locating *distribution_policy*. It will represent the local locality and will place all items to create here.

struct **colocating_distribution_policy**

#include <colocating_distribution_policy.hpp> This class specifies the parameters for a distribution policy to use for creating a given number of items on the locality where a given object is currently placed.

Public Functions

constexpr **colocating_distribution_policy**() = default

Default-construct a new instance of a colocating_distribution_policy. This policy will represent the local locality.

inline *colocating_distribution_policy* **operator()**(id_type const &id) const

Create a new colocating_distribution_policy representing the locality where the given object is current located

Parameters

id – [in] The global address of the object with which the new instances should be colocated on

template<typename **Client**, typename **Stub**, typename **Data**>

inline *colocating_distribution_policy* **operator()**(*client_base*<*Client*, *Stub*, *Data*> const &client) const

Create a new colocating_distribution_policy representing the locality where the given object is current located

Parameters

`client` – [in] The client side representation of the object with which the new instances should be colocated on

template<typename **Component**, typename ...**Ts**>

inline *hpx*::*future*<*hpx*::id_type> **create**(*Ts*&&... vs) const

Create one object on the locality of the object this distribution policy instance is associated with

> **Note**
>
> This function is part of the placement policy implemented by this class

Parameters

`vs` – [in] The arguments which will be forwarded to the constructor of the new object.

Returns

A future holding the global address which represents the newly created object

template<bool **WithCount**, typename **Component**, typename ...**Ts**>

inline *hpx*::*future*<*std*::vector<bulk_locality_result>> **bulk_create**(*std*::size_t count, *Ts*&&... vs) const

Create multiple objects colocated with the object represented by this policy instance

> **Note**
>
> This function is part of the placement policy implemented by this class

Parameters

- `count` – [in] The number of objects to create
- `vs` – [in] The arguments which will be forwarded to the constructors of the new objects.

Returns

A future holding the list of global addresses which represent the newly created objects

template<typename **Action**, typename ...**Ts**>

inline *async_result*<*Action*>::type **async**(*launch* policy, *Ts*&&... vs) const

template<typename **Action**, typename **Callback**, typename ...**Ts**>

inline *async_result*<*Action*>::type **async_cb**(*launch* policy, *Callback* &&cb, *Ts*&&... vs) const

> **Note**
>
> This function is part of the invocation policy implemented by this class

template<typename **Action**, typename **Continuation**, typename ...**Ts**>

inline bool **apply**(*Continuation* &&c, *launch* policy, *Ts*&&... vs) const

> **Note**
>
> This function is part of the invocation policy implemented by this class

template<typename **Action**, typename ...**Ts**>

inline bool **apply**(*launch* policy, *Ts*&&... vs) const

template<typename **Action**, typename **Continuation**, typename **Callback**, typename ...**Ts**>

inline bool **apply_cb**(*Continuation* &&c, *launch* policy, *Callback* &&cb, *Ts*&&... vs) const

> **Note**
>
> This function is part of the invocation policy implemented by this class

template<typename **Action**, typename **Callback**, typename ...**Ts**>

inline bool **apply_cb**(*launch* policy, *Callback* &&cb, *Ts*&&... vs) const

inline *hpx*::id_type **get_next_target**() const

Returns the locality which is anticipated to be used for the next async operation

Public Static Functions

static inline *std*::size_t **get_num_localities**()

Returns the number of associated localities for this distribution policy

> **Note**
>
> This function is part of the creation policy implemented by this class

Private Functions

inline *hpx*::id_type **resolve_direct_target**() const

template<typename **Action**>

struct **async_result**

#include <colocating_distribution_policy.hpp>

> **Note**
>
> This function is part of the invocation policy implemented by this class

Public Types

using **type** = *hpx*::*future*<typename *traits*::promise_local_result<typename *hpx*::*traits*::extract_action<*Action*>::remote_result_type>::type>

hpx/distribution_policies/target_distribution_policy.hpp

Defined in header hpx/distribution_policies/target_distribution_policy.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

Variables

target_distribution_policy const **target**

A predefined instance of the target_distribution_policy. It will represent the local locality and will place all items to create here.

struct **target_distribution_policy**

#include <target_distribution_policy.hpp> This class specifies the parameters for a simple distribution policy to use for creating (and evenly distributing) a given number of items on a given set of localities.

Public Functions

target_distribution_policy() = default

Default-construct a new instance of a target_distribution_policy. This policy will represent one locality (the local locality).

inline *target_distribution_policy* **operator()**(id_type const &id) const

Create a new target_distribution_policy representing the given locality

Parameters

loc – [in] The locality the new instance should represent

template<typename **Component**, typename ...**Ts**>

inline *hpx*::*future*<*hpx*::id_type> **create**(*Ts*&&... vs) const

Create one object on one of the localities associated by this policy instance

> **Note**
>
> This function is part of the placement policy implemented by this class

Parameters

vs – [in] The arguments which will be forwarded to the constructor of the new object.

Returns

A future holding the global address which represents the newly created object

template<bool **WithCount**, typename **Component**, typename ...**Ts**>

inline *hpx*::*future*<*std*::vector<bulk_locality_result>> **bulk_create**(*std*::size_t count, *Ts*&&... vs) const

Create multiple objects on the localities associated by this policy instance

> **Note**
>
> This function is part of the placement policy implemented by this class

Parameters

- **count** – [in] The number of objects to create
- **vs** – [in] The arguments which will be forwarded to the constructors of the new objects.

Returns

A future holding the list of global addresses which represent the newly created objects

template<typename **Action**, typename ...**Ts**>

inline *async_result*<*Action*>::type **async**(*launch* policy, *Ts*&&... vs) const

template<typename **Action**, typename **Callback**, typename ...**Ts**>

inline *async_result*<*Action*>::type **async_cb**(*launch* policy, *Callback* &&cb, *Ts*&&... vs) const

> **Note**
>
> This function is part of the invocation policy implemented by this class

template<typename **Action**, typename **Continuation**, typename ...**Ts**>

inline bool **apply**(*Continuation* &&c, *launch* policy, *Ts*&&... vs) const

> **Note**
>
> This function is part of the invocation policy implemented by this class

template<typename **Action**, typename ...**Ts**>

inline bool **apply**(*launch* policy, *Ts*&&... vs) const

template<typename **Action**, typename **Continuation**, typename **Callback**, typename ...**Ts**>

inline bool **apply_cb**(*Continuation* &&c, *launch* policy, *Callback* &&cb, *Ts*&&... vs) const

> **Note**
>
> This function is part of the invocation policy implemented by this class

template<typename **Action**, typename **Callback**, typename ...**Ts**>

inline bool **apply_cb**(*launch* policy, *Callback* &&cb, *Ts*&&... vs) const

inline *std*::size_t **get_num_localities**() const

Returns the number of associated localities for this distribution policy

> **Note**
>
> This function is part of the creation policy implemented by this class

inline *hpx*::id_type **get_next_target**() const

Returns the locality which is anticipated to be used for the next async operation

template<typename **Action**>

struct **async_result**

#include <target_distribution_policy.hpp>

> **Note**
>
> This function is part of the invocation policy implemented by this class

Public Types

using **type** = *hpx*::*future*<typename *traits*::promise_local_result<typename *hpx*::*traits*::extract_action<*Action*>::remote_result_type>::type>

execution_distributed

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/execution_distributed/distributed_continues_on_sender.hpp

Defined in header hpx/execution_distributed/distributed_continues_on_sender.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Placeholder for the distributed continues_on sender adaptor.

A proper component-based receiver marshalling system (nvexec-style) will be implemented here in a future phase.

hpx/execution_distributed/distributed_scheduler.hpp

Defined in header hpx/execution_distributed/distributed_scheduler.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

P2300-compliant scheduler for cross-locality execution.

Provides a scheduler that dispatches work to a remote HPX locality via the parcelport, bridging the hpx::future completion back into the P2300 set_value / set_error receiver protocol.

Usage:

```
hpx::id_type remote = hpx::find_all_localities()[1];
auto sched = hpx::distributed::distributed_scheduler{remote};
auto result = ex::just(42)
```

(continues on next page)

(continued from previous page)

```
    | ex::continues_on(sched)
    | ex::then([](int x){ return x * 2; })
    | tt::sync_wait();
```

hpx/execution_distributed/distributed_bulk_sender.hpp

Defined in header hpx/execution_distributed/distributed_bulk_sender.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

P2300-compliant distributed bulk sender adaptor.

Provides a bulk sender that executes data-parallel work on a remote HPX locality via the distributed_scheduler. The sender intercepts ex::bulk() when the upstream sender's completion scheduler is a distributed_scheduler, and dispatches a shape-indexed invocation across the parcelport.

Architecture: upstream_sender | v ex::bulk(shape, f) | (completion scheduler == distributed_scheduler) v distributed_bulk_sender<Sender, Shape, F> | v (connect + start) local_receiver wraps downstream_receiver | set_value(Ts…) | => for each index in shape: invoke f(index, ts…) | => forward set_value(ts…) to downstream v downstream_receiver

> **Note**
>
> **Current status: local-only stub.** The bulk loop executes sequentially on the calling locality. Remote parcelport dispatch will be added once the component-based receiver marshalling infrastructure (nvexec-style) is complete.

hpx/execution_distributed/distributed_then_sender.hpp

Defined in header hpx/execution_distributed/distributed_then_sender.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Placeholder for the distributed then sender adaptor.

A proper component-based receiver marshalling system (nvexec-style) will be implemented here in a future phase.

executors_distributed

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/executors_distributed/distribution_policy_executor.hpp

Defined in header hpx/executors_distributed/distribution_policy_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

Functions

template<typename **DistPolicy**>

distribution_policy_executor(*DistPolicy*&&) -> *distribution_policy_executor*<*std*::decay_t<*DistPolicy*>>

```
template<typename DistPolicy>  HPX_DEPRECATED_V (1, 9,
"hpx::parallel::execution::make_distribution_policy_executor is " "deprecated,
use " "hpx::parallel::execution::distribution_policy_executor instead") distribution_policy_executor< std
```

Create a new *distribution_policy_executor* from the given distribution_policy.

Parameters

`policy` – The distribution_policy to create an executor from

template<typename **DistPolicy**>

class **distribution_policy_executor**

#include <distribution_policy_executor.hpp> A distribution_policy_executor creates groups of parallel execution agents that execute in threads implicitly created by the executor and placed on any of the associated localities.

Template Parameters

`DistPolicy` – The distribution policy type for which an executor should be created. The expression *hpx::traits::is_distribution_policy_v<DistPolicy>* must evaluate to true.

include

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/experimental/sandbox.hpp

Defined in header hpx/experimental/sandbox.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **experimental**

namespace **sandbox**

Functions

environment_info **detect_environment**()

Detect and return information about the current environment.

inline void **describe_environment**(*std*::ostream &os)

Print a formatted environment report.

template<typename **F**>

double **measure**(*F* &&fn, *std*::size_t iterations = 5)

Measure the mean execution time of a callable.

template<typename **SeqFn**, typename **ParFn**>

benchmark_report **benchmark**(*std*::string label, *SeqFn* &&seq_fn, *ParFn* &&par_fn, *std*::size_t iterations = 5)

Compare sequential and parallel execution of the same work.

struct **environment_info**

#include <sandbox.hpp> Describes the hardware and runtime environment.

Public Functions

void **print**(*std*::ostream &os) const

Print a formatted environment report.

Parameters

os – Output stream.

Public Members

std::size_t **cores** = 0

Physical CPU cores.

std::size_t **pus** = 0

Processing units (hardware threads)

std::size_t **numa_nodes** = 0

NUMA domains.

std::size_t **hpx_workers** = 0

Active HPX worker threads.

bool **is_sandbox** = false

Likely constrained environment.

struct **benchmark_report**

#include <*sandbox.hpp*> Holds the results of a comparative benchmark.

Public Functions

void **print**(*std*::ostream &os) const

Print a formatted benchmark report.

Parameters

os – Output stream.

Public Members

std::string **label**

User-provided description.

std::size_t **iterations** = 0

Measurement iterations.

double **seq_mean_ms** = 0.0

Sequential mean time (ms)

double **par_mean_ms** = 0.0

Parallel mean time (ms)

double **speedup** = 0.0

seq_time / par_time

double **efficiency_pct** = 0.0

(speedup / workers) * 100

std::size_t **num_workers** = 0

HPX workers during measurement.

init_runtime

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/hpx_start.hpp

Defined in header hpx/hpx_start.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

hpx::init_params

Defined in header hpx/init.hpp[587].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

struct **init_params**

Parameters used to initialize the HPX runtime through hpx::init and hpx::start.

[587] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/init_runtime/include/hpx/init.hpp

hpx/hpx_init.hpp

Defined in header hpx/hpx_init.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

hpx::suspend

Defined in header hpx/init.hpp[588].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::resume*

int *hpx*::**suspend**(*error_code* &ec = *throws*)

Suspend the runtime system.

The function hpx::suspend is used to suspend the HPX runtime system. It can only be used when running HPX on a single locality. It will block waiting for all thread pools to be empty. This function only be called when the runtime is running, or already suspended in which case this function will do nothing.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of *hpx::exception*.

Parameters

ec – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function will always return zero.

hpx::resume

Defined in header hpx/init.hpp[589].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::suspend*

[588] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/init_runtime/include/hpx/init.hpp
[589] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/init_runtime/include/hpx/init.hpp

int `hpx::resume`(*error_code* &ec = *throws*)

Resume the HPX runtime system.

The function hpx::resume is used to resume the HPX runtime system. It can only be used when running HPX on a single locality. It will block waiting for all thread pools to be resumed. This function only be called when the runtime suspended, or already running in which case this function will do nothing.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of *hpx::exception*.

Parameters

`ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function will always return zero.

hpx::finalize

Defined in header hpx/init.hpp[590].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::disconnect*

int `hpx::finalize`(double shutdown_timeout, double localwait = -1.0, *hpx*::*error_code* &ec = *throws*)

Main function to gracefully terminate the HPX runtime system.

The function hpx::finalize is the main way to (gracefully) exit any HPX application. It must be called at least once, but can be called multiple times as well. However, only the first invocation will have effect. It will notify all connected localities to finish execution. Only after all other localities have exited this function will return, allowing to exit the console locality as well.

During the execution of this function the runtime system will invoke all registered shutdown functions (see hpx::init) on all localities.

The default value (`-1.0`) will try to find a globally set timeout value (can be set as the configuration parameter `hpx.shutdown_timeout`), and if that is not set or `-1.0` as well, it will disable any timeout, each connected locality will wait for all existing HPX-threads to terminate.

The default value (`-1.0`) will try to find a globally set wait time value (can be set as the configuration parameter "hpx.finalize_wait_time"), and if this is not set or `-1.0` as well, it will disable any addition local wait time before proceeding.

[590] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/init_runtime/include/hpx/init.hpp

This function will block and wait for all connected localities to exit before returning to the caller. It should be the last HPX-function called by any application.

Using this function is an alternative to hpx::disconnect, these functions do not need to be called both.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of *hpx::exception*.

Parameters

- **shutdown_timeout** – This parameter allows to specify a timeout (in microseconds), specifying how long any of the connected localities should wait for pending tasks to be executed. After this timeout, all suspended HPX-threads will be aborted. Note, that this function will not abort any running HPX-threads. In any case the shutdown will not proceed as long as there is at least one pending/running HPX-thread.
- **localwait** – This parameter allows to specify a local wait time (in microseconds) before the connected localities will be notified and the overall shutdown process starts.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function will always return zero.

inline int `hpx::finalize`(*hpx*::*error_code* &ec = *throws*)

Main function to gracefully terminate the HPX runtime system.

The function hpx::finalize is the main way to (gracefully) exit any HPX application. It must be called at least once, but can be called multiple times as well. However, only the first invocation will have effect. It will notify all connected localities to finish execution. Only after all other localities have exited this function will return, allowing to exit the console locality as well.

During the execution of this function the runtime system will invoke all registered shutdown functions (see hpx::init) on all localities.

This function will block and wait for all connected localities to exit before returning to the caller. It should be the last HPX-function called by any application.

Using this function is an alternative to hpx::disconnect, these functions do not need to be called both.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of

hpx::exception.

Parameters

ec – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function will always return zero.

hpx::disconnect

Defined in header hpx/init.hpp[591].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::finalize*

int *hpx*::**disconnect**(double shutdown_timeout, double localwait = -1.0, *hpx*::*error_code* &ec = *throws*)

Disconnect this locality from the application.

The function hpx::disconnect can be used to disconnect a locality from a running HPX application.

During the execution of this function the runtime system will invoke all registered shutdown functions (see hpx::init) on this locality.

The default value (`-1.0`) will try to find a globally set timeout value (can be set as the configuration parameter "hpx.shutdown_timeout"), and if that is not set or `-1.0` as well, it will disable any timeout, each connected locality will wait for all existing HPX-threads to terminate.

The default value (`-1.0`) will try to find a globally set wait time value (can be set as the configuration parameter `hpx.finalize_wait_time`), and if this is not set or `-1.0` as well, it will disable any addition local wait time before proceeding.

This function will block and wait for this locality to finish executing before returning to the caller. It should be the last HPX-function called by any locality being disconnected.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of *hpx::exception*.

Parameters

[591] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/init_runtime/include/hpx/init.hpp

- **`shutdown_timeout`** – This parameter allows to specify a timeout (in microseconds), specifying how long this locality should wait for pending tasks to be executed. After this timeout, all suspended HPX-threads will be aborted. Note, that this function will not abort any running HPX-threads. In any case the shutdown will not proceed as long as there is at least one pending/running HPX-thread.
- **`localwait`** – This parameter allows to specify a local wait time (in microseconds) before the connected localities will be notified and the overall shutdown process starts.
- **`ec`** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function will always return zero.

inline int *hpx*::**disconnect**(*hpx*::*error_code* &ec = *throws*)

Disconnect this locality from the application.

The function hpx::disconnect can be used to disconnect a locality from a running HPX application.

During the execution of this function the runtime system will invoke all registered shutdown functions (see hpx::init) on this locality.

This function will block and wait for this locality to finish executing before returning to the caller. It should be the last HPX-function called by any locality being disconnected.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of *hpx::exception*.

Parameters

`ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function will always return zero if successful, -1 otherwise.

int *hpx*::**disconnect**([[maybe_unused]] double shutdown_timeout, [[maybe_unused]] double localwait, [[maybe_unused]] *error_code* &ec)

hpx::init

Defined in header hpx/init.hpp[592].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

inline int *hpx*::**init**(*std*::function<int(*hpx*::program_options::variables_map&)> f, int argc, char **argv, *init_params* const &params = *init_params*())

Main entry point for launching the HPX runtime system.

This is the main entry point for any HPX application. This function (or one of its overloads) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as a HPX thread. This overload will not call `hpx_main`.

This is the main entry point for any HPX application. This function (or one of its overloads below) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as a HPX thread.

> **Note**
>
> If the parameter `mode` is not given (defaulted), the created runtime system instance will be executed in console or worker mode depending on the command line arguments passed in `argc`/`argv`. Otherwise, it will be executed as specified by the parameter`mode`.

Parameters

- **f** – [in] The function to be scheduled as an HPX thread. Usually this function represents the main entry point of any HPX application. If `f` is `nullptr` the HPX runtime environment will be started without invoking `f`.
- **argc** – [in] The number of command line arguments passed in `argv`. This is usually the unchanged value as passed by the operating system (to `main()`).
- **argv** – [in] The command line arguments for this application, usually that is the value as passed by the operating system (to `main()`).
- **params** – [in] The parameters to the hpx::init function (See documentation of hpx::init_params)

Returns

The function returns the value, which has been returned from the user supplied `f`.

inline int *hpx*::**init**(*std*::function<int(int, char**)> f, int argc, char **argv, *init_params* const &params = *init_params*())

Main entry point for launching the HPX runtime system.

This is the main entry point for any HPX application. This function (or one of its overloads) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as a HPX thread. This overload will not call `hpx_main`.

[592] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/init_runtime/include/hpx/init.hpp

This is the main entry point for any HPX application. This function (or one of its overloads below) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as a HPX thread.

> **Note**
>
> If the parameter `mode` is not given (defaulted), the created runtime system instance will be executed in console or worker mode depending on the command line arguments passed in `argc`/`argv`. Otherwise, it will be executed as specified by the parameter`mode`.

Parameters

- **`f`** – [in] The function to be scheduled as an HPX thread. Usually this function represents the main entry point of any HPX application. If `f` is `nullptr` the HPX runtime environment will be started without invoking `f`.
- **`argc`** – [in] The number of command line arguments passed in `argv`. This is usually the unchanged value as passed by the operating system (to `main()`).
- **`argv`** – [in] The command line arguments for this application, usually that is the value as passed by the operating system (to `main()`).
- **`params`** – [in] The parameters to the hpx::init function (See documentation of hpx::init_params)

Returns

The function returns the value, which has been returned from the user supplied `f`.

inline int *hpx*::**init**(int argc, char **argv, *init_params* const &params = *init_params*())

Main entry point for launching the HPX runtime system.

This is the main entry point for any HPX application. This function (or one of its overloads) should be called from the users `main()` function. This overload expects a user-defined function named `hpx_main` at global scope, which will be used as the entry point for the HPX application.

This is the main entry point for any HPX application. This function (or one of its overloads below) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as a HPX thread.

> **Note**
>
> If the parameter `mode` is not given (defaulted), the created runtime system instance will be executed in console or worker mode depending on the command line arguments passed in `argc`/`argv`. Otherwise, it will be executed as specified by the parameter`mode`.

Parameters

- **`argc`** – [in] The number of command line arguments passed in `argv`. This is usually the unchanged value as passed by the operating system (to `main()`).

- **argv** – [in] The command line arguments for this application, usually that is the value as passed by the operating system (to `main()`).
- **params** – [in] The parameters to the hpx::init function (See documentation of hpx::init_params)

Returns
The function returns the value, which has been returned from `hpx_main` (or 0 when executed in worker mode).

inline int *hpx*::**init**(*std*::nullptr_t f, int argc, char **argv, *init_params* const &params = *init_params*())

Main entry point for launching the HPX runtime system.

This is the main entry point for any HPX application. This function (or one of its overloads below) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as a HPX thread. This overload will not call `hpx_main`.

This is the main entry point for any HPX application. This function (or one of its overloads below) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as a HPX thread.

> **Note**
>
> If the parameter `mode` is not given (defaulted), the created runtime system instance will be executed in console or worker mode depending on the command line arguments passed in `argc`/`argv`. Otherwise, it will be executed as specified by the parameter`mode`.

Parameters

- **f** – [in] The function to be scheduled as an HPX thread. Usually this function represents the main entry point of any HPX application. If `f` is `nullptr` the HPX runtime environment will be started without invoking `f`.
- **argc** – [in] The number of command line arguments passed in `argv`. This is usually the unchanged value as passed by the operating system (to `main()`).
- **argv** – [in] The command line arguments for this application, usually that is the value as passed by the operating system (to `main()`).
- **params** – [in] The parameters to the hpx::init function (See documentation of hpx::init_params)

Returns
The function returns the value, which has been returned from the user supplied `f`.

inline int *hpx*::**init**(*init_params* const &params = *init_params*())

Main entry point for launching the HPX runtime system.

This is a simplified main entry point, which can be used to set up the runtime for an HPX application (the runtime system will be set up in console mode or worker mode depending on the command line settings). This overload expects a user-defined function named `hpx_main` at global scope, which will be used as the entry point for the HPX application.

This is a simplified main entry point, which can be used to set up the runtime for an HPX application (the runtime system will be set up in console mode or worker mode depending on the command line settings).

> **Note**
>
> The created runtime system instance will be executed in console or worker mode depending on the command line arguments passed in `argc/argv`. If not command line arguments are passed, console mode is assumed.

> **Note**
>
> If no command line arguments are passed the HPX runtime system will not support any of the default command line options as described in the section 'HPX Command Line Options'.

Parameters

params – [in] The parameters to the hpx::init function (See documentation of hpx::init_params)

Returns

The function returns the value, which has been returned from `hpx_main` (or 0 when executed in worker mode).

hpx::start

Defined in header hpx/init.hpp[593].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

inline bool ***hpx*::start**(*std*::function<int(*hpx*::program_options::variables_map&)> f, int argc, char **argv, *init_params* const &params = *init_params*())

Main non-blocking entry point for launching the HPX runtime system.

This is the main, non-blocking entry point for any HPX application. This function (or one of its overloads below) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as a HPX thread. It will return immediately after that. Use `hpx::wait` and `hpx::stop` to synchronize with the runtime system's execution. This overload will not call `hpx_main`.

This is the main, non-blocking entry point for any HPX application. This function (or one of its overloads below) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as an HPX thread. It will return immediately after that. Use `hpx::wait` and `hpx::stop` to synchronize with the runtime system's execution.

[593] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/init_runtime/include/hpx/init.hpp

> **Note**
>
> If the parameter `mode` is not given (defaulted), the created runtime system instance will be executed in console or worker mode depending on the command line arguments passed in `argc`/`argv`. Otherwise, it will be executed as specified by the parameter`mode`.

Parameters

- **f** – [in] The function to be scheduled as an HPX thread. Usually this function represents the main entry point of any HPX application. If `f` is `nullptr` the HPX runtime environment will be started without invoking `f`.
- **argc** – [in] The number of command line arguments passed in `argv`. This is usually the unchanged value as passed by the operating system (to `main()`).
- **argv** – [in] The command line arguments for this application, usually that is the value as passed by the operating system (to `main()`).
- **params** – [in] The parameters to the hpx::start function (See documentation of hpx::init_params)

Returns

The function returns *true* if command line processing succeeded and the runtime system was started successfully. It will return *false* otherwise.

inline bool `hpx::start`(*std*::function<int(int, char**)> f, int argc, char **argv, *init_params* const &params = *init_params*())

Main non-blocking entry point for launching the HPX runtime system.

This is the main, non-blocking entry point for any HPX application. This function (or one of its overloads below) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as a HPX thread. It will return immediately after that. Use `hpx::wait` and `hpx::stop` to synchronize with the runtime system's execution. This overload will not call `hpx_main`.

This is the main, non-blocking entry point for any HPX application. This function (or one of its overloads below) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as an HPX thread. It will return immediately after that. Use `hpx::wait` and `hpx::stop` to synchronize with the runtime system's execution.

> **Note**
>
> If the parameter `mode` is not given (defaulted), the created runtime system instance will be executed in console or worker mode depending on the command line arguments passed in `argc`/`argv`. Otherwise, it will be executed as specified by the parameter`mode`.

Parameters

- **f** – [in] The function to be scheduled as an HPX thread. Usually this function represents the main entry point of any HPX application. If `f` is `nullptr` the HPX runtime environment will be started without invoking `f`.

- **argc** – [in] The number of command line arguments passed in `argv`. This is usually the unchanged value as passed by the operating system (to `main()`).
- **argv** – [in] The command line arguments for this application, usually that is the value as passed by the operating system (to `main()`).
- **params** – [in] The parameters to the hpx::start function (See documentation of hpx::init_params)

Returns

The function returns *true* if command line processing succeeded and the runtime system was started successfully. It will return *false* otherwise.

inline bool `hpx::start`(int argc, char **argv, *init_params* const &params = *init_params*())

Main non-blocking entry point for launching the HPX runtime system.

This is the main, non-blocking entry point for any HPX application. This function (or one of its overloads below) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as a HPX thread. It will return immediately after that. Use `hpx::wait` and `hpx::stop` to synchronize with the runtime system's execution. This overload will not call `hpx_main`.

This is the main, non-blocking entry point for any HPX application. This function (or one of its overloads below) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as an HPX thread. It will return immediately after that. Use `hpx::wait` and `hpx::stop` to synchronize with the runtime system's execution.

> **Note**
>
> If the parameter `mode` is not given (defaulted), the created runtime system instance will be executed in console or worker mode depending on the command line arguments passed in `argc`/`argv`. Otherwise, it will be executed as specified by the parameter`mode`.

Parameters

- **argc** – [in] The number of command line arguments passed in `argv`. This is usually the unchanged value as passed by the operating system (to `main()`).
- **argv** – [in] The command line arguments for this application, usually that is the value as passed by the operating system (to `main()`).
- **params** – [in] The parameters to the hpx::start function (See documentation of hpx::init_params)

Returns

The function returns *true* if command line processing succeeded and the runtime system was started successfully. It will return *false* otherwise.

inline bool `hpx::start`(*std*::nullptr_t f, int argc, char **argv, *init_params* const &params = *init_params*())

Main non-blocking entry point for launching the HPX runtime system.

This is the main, non-blocking entry point for any HPX application. This function (or one of its overloads below) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as a HPX thread. It will return immediately after that. Use `hpx::wait` and `hpx::stop` to synchronize with the runtime system's execution. This overload will not call `hpx_main`.

This is the main, non-blocking entry point for any HPX application. This function (or one of its overloads below) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as an HPX thread. It will return immediately after that. Use `hpx::wait` and `hpx::stop` to synchronize with the runtime system's execution.

> **Note**
>
> If the parameter `mode` is not given (defaulted), the created runtime system instance will be executed in console or worker mode depending on the command line arguments passed in `argc`/`argv`. Otherwise, it will be executed as specified by the parameter`mode`.

Parameters

- **`f`** – [in] The function to be scheduled as an HPX thread. Usually this function represents the main entry point of any HPX application. If `f` is `nullptr` the HPX runtime environment will be started without invoking `f`.
- **`argc`** – [in] The number of command line arguments passed in `argv`. This is usually the unchanged value as passed by the operating system (to `main()`).
- **`argv`** – [in] The command line arguments for this application, usually that is the value as passed by the operating system (to `main()`).
- **`params`** – [in] The parameters to the hpx::start function (See documentation of hpx::init_params)

Returns

The function returns *true* if command line processing succeeded and the runtime system was started successfully. It will return *false* otherwise.

inline bool *hpx*::**start**(*init_params* const &params = *init_params*())

Main non-blocking entry point for launching the HPX runtime system.

This is a simplified main, non-blocking entry point, which can be used to set up the runtime for an HPX application (the runtime system will be set up in console mode or worker mode depending on the command line settings). It will return immediately after that. Use `hpx::wait` and `hpx::stop` to synchronize with the runtime system's execution.

This is the main, non-blocking entry point for any HPX application. This function (or one of its overloads below) should be called from the users `main()` function. It will set up the HPX runtime environment and schedule the function given by `f` as an HPX thread. It will return immediately after that. Use `hpx::wait` and `hpx::stop` to synchronize with the runtime system's execution.

Note

The created runtime system instance will be executed in console or worker mode depending on the command line arguments passed in `argc/argv`. If not command line arguments are passed, console mode is assumed.

Note

If no command line arguments are passed the HPX runtime system will not support any of the default command line options as described in the section 'HPX Command Line Options'.

Parameters

params – [in] The parameters to the hpx::start function (See documentation of hpx::init_params)

Returns

The function returns *true* if command line processing succeeded and the runtime system was started successfully. It will return *false* otherwise.

naming_base

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/naming_base/unmanaged.hpp

Defined in header hpx/naming_base/unmanaged.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

hpx::id_type **unmanaged**(*hpx*::id_type const &id)

The helper function *hpx::unmanaged* can be used to generate a global identifier which does not participate in the automatic garbage collection.

Note

This function allows to apply certain optimizations to the process of memory management in HPX. It however requires the user to take full responsibility for keeping the referenced objects alive long enough.

Parameters

`id` – [in] The id to generated the unmanaged global id from This parameter can be itself a managed or a unmanaged global id.

Returns

This function returns a new global id referencing the same object as the parameter *id*. The only difference is that the returned global identifier does not participate in the automatic garbage collection.

namespace **naming**

parcelset

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/parcelset/message_handler_fwd.hpp

Defined in header hpx/parcelset/message_handler_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/parcelset/parcelhandler.hpp

Defined in header hpx/parcelset/parcelhandler.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/parcelset/connection_cache.hpp

Defined in header hpx/parcelset/connection_cache.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/parcelset/parcelset_fwd.hpp

Defined in header hpx/parcelset/parcelset_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

parcelset_base

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/parcelset_base/set_parcel_write_handler.hpp

Defined in header hpx/parcelset_base/set_parcel_write_handler.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/parcelset_base/parcelport.hpp

Defined in header hpx/parcelset_base/parcelport.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/parcelset_base/parcelset_base_fwd.hpp

Defined in header hpx/parcelset_base/parcelset_base_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Defines

HPX_PARCELPORT_BACKGROUND_MODE_ENUM_DEPRECATION_MSG

namespace **hpx**

namespace **parcelset**

Typedefs

using **parcel_write_handler_type** = *hpx*::*function*<bool(*std*::error_code const&, *parcelset*::parcel const&)>

The type of the function that can be registered as a parcel write handler using the function *hpx::set_parcel_write_handler*.

> **Note**
>
> A parcel write handler is a function which is called by the parcel layer whenever a parcel has been sent by the underlying networking library and if no explicit parcel handler function was specified for the parcel.

using **write_handler_type** = *hpx*::*function*<void(*std*::error_code const&, *parcelset*::parcel const&)>

Enums

enum class **parcelport_background_mode** : *std*::uint8_t

Type of background work to perform.

Values:

enumerator **flush_buffers**

perform buffer flush operations

enumerator **send**

perform send operations (includes buffer flush)

enumerator **receive**

perform receive operations

enumerator **all**

perform all operations

Functions

inline bool **operator&**(*parcelport_background_mode* lhs, *parcelport_background_mode* rhs)

char const ***get_parcelport_background_mode_name**(*parcelport_background_mode* mode)

Variables

parcel **empty_parcel**

constexpr *parcelport_background_mode* **parcelport_background_mode_flush_buffers** = *parcelport_background_mode*::*flush_buffers*

constexpr *parcelport_background_mode* **parcelport_background_mode_send** = *parcelport_background_mode*::*send*

constexpr *parcelport_background_mode* **parcelport_background_mode_receive** = *parcelport_background_mode*::*receive* constexpr *parcelport_background_mode* **parcelport_background_mode_all** = *parcelport_background_mode*::*all*

plugin_factories

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/plugin_factories/plugin_registry.hpp

Defined in header hpx/plugin_factories/plugin_registry.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **plugins**

template<typename **Plugin**, char const *const **Name**, char const *const **Section**, char const *const **Suffix**>

struct **plugin_registry** : public *plugin_registry_base*

#include <plugin_registry.hpp> The plugin_registry provides a minimal implementation of a plugin's registry. If no additional functionality is required this type can be used to implement the full set of minimally required functions to be exposed by a plugin's registry instance.

Template Parameters

Plugin – The plugin type this registry should be responsible for.

Public Functions

inline bool **get_plugin_info**(*std*::vector<*std*::string> &fillini) override

Return the ini-information for all contained components.

Parameters

fillini – [in] The module is expected to fill this vector with the ini-information (one line per vector element) for all components implemented in this module.

Returns

Returns *true* if the parameter *fillini* has been successfully initialized with the registry data of all implemented in this module.

hpx/plugin_factories/parcelport_factory.hpp

Defined in header hpx/plugin_factories/parcelport_factory.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/plugin_factories/message_handler_factory.hpp

Defined in header hpx/plugin_factories/message_handler_factory.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/plugin_factories/binary_filter_factory.hpp

Defined in header hpx/plugin_factories/binary_filter_factory.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **plugins**

template<typename **BinaryFilter**>

struct **binary_filter_factory** : public binary_filter_factory_base

#include <binary_filter_factory.hpp> The *message_handler_factory* provides a minimal implementation of a message handler's factory. If no additional functionality is required this type can be used to implement the full set of minimally required functions to be exposed by a message handler's factory instance.

Template Parameters

BinaryFilter – The message handler type this factory should be responsible for.

Public Functions

inline **binary_filter_factory**(*util*::section const *global, *util*::section const *local, bool isenabled)

Construct a new factory instance.

> **Note**
>
> The contents of both sections has to be cloned in order to save the configuration setting for later use.

Parameters

- **global** – [in] The pointer to a *hpx::util::section* instance referencing the settings read from the [settings] section of the global configuration file (hpx.ini) This pointer may be nullptr if no such section has been found.
- **local** – [in] The pointer to a *hpx::util::section* instance referencing the settings read from the section describing this component type: [hpx.components.<name>], where <name> is the instance name of the component as given in the configuration files.
- **isenabled** –

~binary_filter_factory() override = default

inline *serialization*::binary_filter ***create**(bool compress, *serialization*::binary_filter *next_filter = nullptr) override

Create a new instance of a message handler

return Returns the newly created instance of the message handler supported by this factory

Protected Attributes

util::section **global_settings_**

util::section **local_settings_**

bool **isenabled_**

resiliency_distributed

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/resiliency_distributed/async_replay_distributed.hpp

Defined in header hpx/resiliency_distributed/async_replay_distributed.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **resiliency**

namespace **experimental**

Functions

template<typename **Action**, typename ...**Ts**>

hpx::*future*<*hpx*::*util*::detail::invoke_deferred_result_t<*Action*, *hpx*::id_type, *Ts*...>> **hpx_invoke**(*async_replay_t*, *std*::vector<*hpx*::id_type> const &ids, *Action* &&action, *Ts*&&... ts)

ADL hook for distributed async_replay CPO.

Asynchronously launches *action* on localities in *ids*, replaying on the next locality on error (aborts on abort_replay_exception).

Parameters

- **tag** – CPO tag (async_replay_t).
- **ids** – Target localities to try in order.
- **action** – Action to invoke on each locality.
- **ts** – Arguments forwarded to *action*.

Returns

future with the first successful result.

template<typename **Pred**, typename **Action**, typename ...**Ts**>

hpx::*future*<*hpx*::*util*::detail::invoke_deferred_result_t<*Action*, *hpx*::id_type, *Ts*...>> **hpx_invoke**(*async_replay_validate_t*, *std*::vector<*hpx*::id_type> const &ids, *Pred* &&pred, *Action* &&action, *Ts*&&... ts)

ADL hook for distributed async_replay_validate CPO.

Asynchronously launches *action* on localities in *ids*, validating results with *pred*. Replays on the next locality on error (aborts on abort_replay_exception).

Parameters

- **tag** – CPO tag (async_replay_validate_t).
- **ids** – Target localities to try in order.
- **pred** – Predicate validating each invocation result.
- **action** – Action to invoke on each locality.
- **ts** – Arguments forwarded to *action*.

Returns

future with the first valid result.

hpx/resiliency_distributed/async_replicate_distributed.hpp

Defined in header hpx/resiliency_distributed/async_replicate_distributed.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **resiliency**

namespace **experimental**

Functions

template<typename **Vote**, typename **Pred**, typename **Action**, typename ...**Ts**>

decltype(auto) **hpx_invoke**(*async_replicate_vote_validate_t*, *std*::vector<*hpx*::id_type> const &ids, *Vote* &&vote, *Pred* &&pred, *Action* &&action, *Ts*&&... ts)

ADL hook for distributed async_replicate_vote_validate CPO.

Asynchronously launches *action* on each locality in *ids*, validates results with *pred*, then runs valid results through *vote*.

Parameters

- **tag** – CPO tag (async_replicate_vote_validate_t).
- **ids** – Target localities (one invocation per locality).
- **vote** – Voting function selecting among valid results.
- **pred** – Predicate validating each invocation result.
- **action** – Action to invoke on each locality.
- **ts** – Arguments forwarded to *action*.

Returns

future with the voted-upon result.

template<typename **Vote**, typename **Action**, typename ...**Ts**>

decltype(auto) **hpx_invoke**(*async_replicate_vote_t*, *std*::vector<*hpx*::id_type> const &ids, *Vote* &&vote, *Action* &&action, *Ts*&&... ts)

ADL hook for distributed async_replicate_vote CPO.

Asynchronously launches *action* on each locality in *ids*, runs valid results through *vote*. Validation uses the default exception check.

Parameters

- **tag** – CPO tag (async_replicate_vote_t).
- **ids** – Target localities (one invocation per locality).
- **vote** – Voting function selecting among valid results.
- **action** – Action to invoke on each locality.
- **ts** – Arguments forwarded to *action*.

Returns

future with the voted-upon result.

template<typename **Pred**, typename **Action**, typename ...**Ts**>

decltype(auto) **hpx_invoke**(*async_replicate_validate_t*, *std*::vector<*hpx*::id_type> const &ids, *Pred* &&pred, *Action* &&action, *Ts*&&... ts)

ADL hook for distributed async_replicate_validate CPO.

Asynchronously launches *action* on each locality in *ids*, validates results with *pred*. Returns the first valid result.

Parameters

- **tag** – CPO tag (async_replicate_validate_t).
- **ids** – Target localities (one invocation per locality).
- **pred** – Predicate validating each invocation result.
- **action** – Action to invoke on each locality.
- **ts** – Arguments forwarded to *action*.

Returns

future with the first valid result.

template<typename **Action**, typename ...**Ts**>

decltype(auto) **hpx_invoke**(*async_replicate_t*, *std*::vector<*hpx*::id_type> const &ids, *Action* &&action, *Ts*&&... ts)

ADL hook for distributed async_replicate CPO.

Asynchronously launches *action* on each locality in *ids*. Validates by checking for exceptions. Returns the first valid result.

Parameters

- **tag** – CPO tag (async_replicate_t).
- **ids** – Target localities (one invocation per locality).
- **action** – Action to invoke on each locality.
- **ts** – Arguments forwarded to *action*.

Returns

future with the first non-throwing result.

runtime_components

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/runtime_components/macros.hpp

Defined in header hpx/runtime_components/macros.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Defines

`HPX_REGISTER_MINIMAL_COMPONENT_FACTORY`(...)

`HPX_REGISTER_COMPONENT`(...)

`HPX_REGISTER_ENABLED_COMPONENT_FACTORY`(ComponentType, componentname)

`HPX_REGISTER_DISABLED_COMPONENT_FACTORY`(ComponentType, componentname)

`HPX_REGISTER_MINIMAL_COMPONENT_FACTORY_`(...)

`HPX_REGISTER_MINIMAL_COMPONENT_FACTORY_1`(ComponentType)

`HPX_REGISTER_MINIMAL_COMPONENT_FACTORY_2`(ComponentType, componentname)

`HPX_REGISTER_MINIMAL_COMPONENT_FACTORY_3`(ComponentType, componentname, state)

`HPX_REGISTER_MINIMAL_COMPONENT_FACTORY_DYNAMIC`(...)

`HPX_REGISTER_COMPONENT_DYNAMIC`(...)

This macro is used create and to register a minimal component factory for a component type which allows it to be remotely created using the hpx::new_<> function. This macro can be invoked with one, two or three arguments

`HPX_REGISTER_ENABLED_COMPONENT_FACTORY_DYNAMIC`(ComponentType, componentname)

`HPX_REGISTER_DISABLED_COMPONENT_FACTORY_DYNAMIC`(ComponentType, componentname)

`HPX_REGISTER_MINIMAL_COMPONENT_FACTORY_DYNAMIC_`(...)

`HPX_REGISTER_MINIMAL_COMPONENT_FACTORY_DYNAMIC_1`(ComponentType)

`HPX_REGISTER_MINIMAL_COMPONENT_FACTORY_DYNAMIC_2`(ComponentType, componentname)

`HPX_REGISTER_MINIMAL_COMPONENT_FACTORY_DYNAMIC_3`(ComponentType, componentname, state)

`HPX_REGISTER_MINIMAL_COMPONENT_REGISTRY`(...)

This macro is used create and to register a minimal component registry with Hpx.Plugin.

`HPX_REGISTER_MINIMAL_COMPONENT_REGISTRY_`(...)

`HPX_REGISTER_MINIMAL_COMPONENT_REGISTRY_2`(ComponentType, componentname)

`HPX_REGISTER_MINIMAL_COMPONENT_REGISTRY_3`(ComponentType, componentname, state)

`HPX_REGISTER_MINIMAL_COMPONENT_REGISTRY_DYNAMIC`(...)

`HPX_REGISTER_MINIMAL_COMPONENT_REGISTRY_DYNAMIC_`(...)

`HPX_REGISTER_MINIMAL_COMPONENT_REGISTRY_DYNAMIC_2`(ComponentType, componentname)

`HPX_REGISTER_MINIMAL_COMPONENT_REGISTRY_DYNAMIC_3`(ComponentType, componentname, state)

`HPX_REGISTER_DERIVED_COMPONENT_FACTORY`(...)

This macro is used create and to register a minimal component factory with Hpx.Plugin. This macro may be used if the registered component factory is the only factory to be exposed from a particular module. If more than one factory needs to be exposed the *HPX_REGISTER_COMPONENT_FACTORY* and *HPX_REGISTER_COMPONENT_MODULE* macros should be used instead.

`HPX_REGISTER_DERIVED_COMPONENT_FACTORY_`(...)

`HPX_REGISTER_DERIVED_COMPONENT_FACTORY_3`(ComponentType, componentname, basecomponentname)

`HPX_REGISTER_DERIVED_COMPONENT_FACTORY_4`(ComponentType, componentname, basecomponentname, state)

`HPX_REGISTER_DERIVED_COMPONENT_FACTORY_DYNAMIC`(...)

`HPX_REGISTER_DERIVED_COMPONENT_FACTORY_DYNAMIC_`(...)

`HPX_REGISTER_DERIVED_COMPONENT_FACTORY_DYNAMIC_3`(ComponentType, componentname, basecomponentname)

`HPX_REGISTER_DERIVED_COMPONENT_FACTORY_DYNAMIC_4`(ComponentType, componentname, basecomponentname, state)

`HPX_DISTRIBUTED_METADATA_DECLARATION`(...)

`HPX_DISTRIBUTED_METADATA_DECLARATION_`(...)

`HPX_DISTRIBUTED_METADATA_DECLARATION_1`(config)

`HPX_DISTRIBUTED_METADATA_DECLARATION_2`(config, name)

`HPX_DISTRIBUTED_METADATA`(...)

`HPX_DISTRIBUTED_METADATA_`(...)

`HPX_DISTRIBUTED_METADATA_1`(config)

`HPX_DISTRIBUTED_METADATA_2`(config, name)

HPX_REGISTER_COMPONENT

Defined in header hpx/components.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

> **Warning**
>
> doxygenfunction: Cannot find function "HPX_REGISTER_COMPONENT" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx/runtime_components/reflect_client_macros.hpp

Defined in header hpx/runtime_components/reflect_client_macros.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/runtime_components/reflect_client.hpp

Defined in header hpx/runtime_components/reflect_client.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/runtime_components/components_fwd.hpp

Defined in header hpx/runtime_components/components_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

components::*server*::*runtime_support* *`get_runtime_support_ptr`()

namespace **components**

namespace **server**

namespace **stubs**

namespace **components**

hpx::new_

Defined in header hpx/components.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **Component**, typename ...**Ts**>

auto *hpx*::**new_**(id_type const &locality, *Ts*&&... vs)

Create one or more new instances of the given Component type on the specified locality.

This function creates one or more new instances of the given Component type on the specified locality and returns a future object for the global address which can be used to reference the new component instance.

> **Note**
>
> This function requires to specify an explicit template argument which will define what type of component(s) to create, for instance:
>
> ```
> hpx::future<hpx::id_type> f =
>    hpx::new_<some_component>(hpx::find_here(), ...);
> hpx::id_type id = f.get();
> ```

Parameters

- **locality** – [in] The global address of the locality where the new instance should be created on.
- **vs** – [in] Any number of arbitrary arguments (passed by value, by const reference or by rvalue reference) which will be forwarded to the constructor of the created component instance.

Returns

The function returns different types depending on its use:

- If the explicit template argument *Component* represents a component type (`traits::is_component<Component>::value` evaluates to true), the function will return an hpx::future object instance which can be used to retrieve the global address of the newly created component.
- If the explicit template argument *Component* represents a client side object (`traits::is_client<Component>::value` evaluates to true), the function will return a new instance of that type which can be used to refer to the newly created component instance.

template<typename **Component**, typename ...**Ts**>

auto *hpx*::**new_**(id_type const &locality, *std*::size_t count, *Ts*&&... vs)

Create multiple new instances of the given Component type on the specified locality.

This function creates multiple new instances of the given Component type on the specified locality and returns a future object for the global address which can be used to reference the new component instance.

Note

This function requires to specify an explicit template argument which will define what type of component(s) to create, for instance:

```
hpx::future<std::vector<hpx::id_type> > f =
   hpx::new_<some_component[]>(hpx::find_here(), 10, ...);
hpx::id_type id = f.get();
```

Parameters

- **`locality`** – [in] The global address of the locality where the new instance should be created on.
- **`count`** – [in] The number of component instances to create
- **`vs`** – [in] Any number of arbitrary arguments (passed by value, by const reference or by rvalue reference) which will be forwarded to the constructor of the created component instance.

Returns

The function returns different types depending on its use:

- If the explicit template argument *Component* represents an array of a component type (i.e. *Component*[], where `traits::is_component<Component>::value` evaluates to true), the function will return an hpx::future object instance which holds a std::vector<hpx::id_type>, where each of the items in this vector is a global address of one of the newly created components.
- If the explicit template argument *Component* represents an array of a client side object type (i.e. *Component*[], where `traits::is_client<Component>::value` evaluates to true), the function will return an hpx::future object instance which holds a std::vector<hpx::id_type>, where each of the items in this vector is a client side instance of the given type, each representing one of the newly created components.

template<typename **Component**, typename **DistPolicy**, typename ...**Ts**>

auto *hpx*::**new_**(*DistPolicy* const &policy, *Ts*&&... vs)

Create one or more new instances of the given Component type based on the given distribution policy.

This function creates one or more new instances of the given Component type on the localities defined by the given distribution policy and returns a future object for global address which can be used to reference the new component instance(s).

Note

This function requires to specify an explicit template argument which will define what type of component(s) to create, for instance:

```
hpx::future<hpx::id_type> f =
   hpx::new_<some_component>(hpx::default_layout, ...);
hpx::id_type id = f.get();
```

Parameters

- **policy** – [in] The distribution policy used to decide where to place the newly created.
- **vs** – [in] Any number of arbitrary arguments (passed by value, by const reference or by rvalue reference) which will be forwarded to the constructor of the created component instance.

Returns

The function returns different types depending on its use:

- If the explicit template argument *Component* represents a component type (`traits::is_component<Component>::value` evaluates to true), the function will return an hpx::future object instance which can be used to retrieve the global address of the newly created component.
- If the explicit template argument *Component* represents a client side object (`traits::is_client<Component>::value` evaluates to true), the function will return a new instance of that type which can be used to refer to the newly created component instance.

template<typename **Component**, typename **DistPolicy**, typename ...**Ts**>

auto *hpx*::**new_**(*DistPolicy* const &policy, *std*::size_t count, *Ts*&&... vs)

Create multiple new instances of the given Component type on the localities as defined by the given distribution policy.

This function creates multiple new instances of the given Component type on the localities defined by the given distribution policy and returns a future object for the global address which can be used to reference the new component instance.

Note

This function requires to specify an explicit template argument which will define what type of component(s) to create, for instance:

```
hpx::future<std::vector<hpx::id_type> > f =
   hpx::new_<some_component[]>(hpx::default_layout, 10, ...);
hpx::id_type id = f.get();
```

Parameters

- **policy** – [in] The distribution policy used to decide where to place the newly created.

- **count** – [in] The number of component instances to create
- **vs** – [in] Any number of arbitrary arguments (passed by value, by const reference or by rvalue reference) which will be forwarded to the constructor of the created component instance.

Returns

The function returns different types depending on its use:

- If the explicit template argument *Component* represents an array of a component type (i.e. *Component*[], where `traits::is_component<Component>::value` evaluates to true), the function will return an hpx::future object instance which holds a std::vector<hpx::id_type>, where each of the items in this vector is a global address of one of the newly created components.
- If the explicit template argument *Component* represents an array of a client side object type (i.e. *Component*[], where `traits::is_client<Component>::value` evaluates to true), the function will return an hpx::future object instance which holds a std::vector<hpx::id_type>, where each of the items in this vector is a client side instance of the given type, each representing one of the newly created components.

hpx/runtime_components/component_registry.hpp

Defined in header hpx/runtime_components/component_registry.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

template<typename **Component**, *factory_state* **state**>

struct **component_registry** : public *component_registry_base*

#include <component_registry.hpp> The component_registry provides a minimal implementation of a component's registry. If no additional functionality is required this type can be used to implement the full set of minimally required functions to be exposed by a component's registry instance.

Template Parameters

Component – The component type this registry should be responsible for.

Public Functions

inline bool **get_component_info**(*std*::vector<*std*::string> &fillini, *std*::string const &filepath, bool is_static = false) override

Return the ini-information for all contained components.

Parameters

- **fillini** – [in] The module is expected to fill this vector with the ini-information (one line per vector element) for all components implemented in this module.
- **filepath** –
- **is_static** –

Returns

Returns *true* if the parameter *fillini* has been successfully initialized with the registry data of all implemented in this module.

inline void **register_component_type**() override

Enables this type of registry and sets its destroy mechanism.

runtime_distributed

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/runtime_distributed.hpp

Defined in header hpx/runtime_distributed.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

class **runtime_distributed** : public *runtime*

#include <runtime_distributed.hpp> The *runtime* class encapsulates the HPX runtime system in a simple to use way. It makes sure all required parts of the HPX runtime system are properly initialized.

Public Functions

explicit **runtime_distributed**(*util*::*runtime_configuration* &rtcfg, int (*pre_main)(*runtime_mode*) = nullptr, void (*post_main)() = nullptr)

Construct a new HPX runtime instance

Parameters

- **rtcfg** – Runtime configuration for this instance
- **pre_main** – Function to be called before running the main action of this instance
- **post_main** – Function to be called after running the main action of this instance

~runtime_distributed()

The destructor makes sure all HPX runtime services are properly shut down before exiting.

int **start**(*hpx*::*function*<hpx_main_function_type> const &func, bool blocking = false) override

Start the runtime system.

Parameters

- **func** – [in] This is the main function of an HPX application. It will be scheduled for execution by the thread manager as soon as the runtime has been initialized. This function is expected to expose an interface as defined by the typedef *hpx_main_function_type*.
- **blocking** – [in] This allows to control whether this call blocks until the runtime system has been stopped. If this parameter is *true* the function *runtime::start* will call *runtime::wait* internally.

Returns

If a blocking is a true, this function will return the value as returned as the result of the invocation of the function object given by the parameter `func`. Otherwise it will return zero.

int **start**(bool blocking = false) override

Start the runtime system.

Parameters

blocking – [in] This allows to control whether this call blocks until the runtime system has been stopped. If this parameter is *true* the function *runtime::start* will call *runtime::wait* internally .

Returns

If a blocking is a true, this function will return the value as returned as the result of the invocation of the function object given by the parameter `func`. Otherwise it will return zero.

int **wait**() override

Wait for the shutdown action to be executed.

Returns

This function will return the value as returned as the result of the invocation of the function object given by the parameter `func`.

void **stop**(bool blocking = true) override

Initiate termination of the runtime system.

Parameters

blocking – [in] This allows to control whether this call blocks until the runtime system has been fully stopped. If this parameter is *false* then this call will initiate the stop action but will return immediately. Use a second call to stop with this parameter set to *true* to wait for all internal work to be completed.

int **finalize**(double shutdown_timeout) override

void **stop_helper**(bool blocking, *std*::condition_variable &cond, *std*::mutex &mtx)

Stop the runtime system, wait for termination.

Parameters

- `blocking` – [in] This allows to control whether this call blocks until the runtime system has been fully stopped. If this parameter is *false* then this call will initiate the stop action but will return immediately. Use a second call to stop with this parameter set to *true* to wait for all internal work to be completed.
- `cond` – Condition used to update all thread when done
- `mtx` – Mutex used by this function to sync all threads

int **suspend**() override

Suspend the runtime system.

int **resume**() override

Resume the runtime system.

bool **report_error**(*std*::size_t num_thread, *std*::exception_ptr const &e, bool terminate_all = true) override

Report a non-recoverable error to the runtime system.

Parameters

- `num_thread` – [in] The number of the operating system thread the error has been detected in.
- `e` – [in] This is an instance encapsulating an exception which lead to this function call.
- `terminate_all` – [in] Kill all localities attached to the currently running application (default: true)

bool **report_error**(*std*::exception_ptr const &e, bool terminate_all = true) override

Report a non-recoverable error to the runtime system.

> **Note**
>
> This function will retrieve the number of the current shepherd thread and forward to the report_error function above.

Parameters

- `e` – [in] This is an instance encapsulating an exception which lead to this function call.
- `terminate_all` – [in] Kill all localities attached to the currently running application (default: true)

int **run**(*hpx*::*function*<hpx_main_function_type> const &func) override

Run the HPX runtime system, use the given function for the main *thread* and block waiting for all threads to finish.

> **Note**
>
> The parameter *func* is optional. If no function is supplied, the runtime system will simply wait for the shutdown action without explicitly executing any main thread.

Parameters

`func` – [in] This is the main function of an HPX application. It will be scheduled for execution by the thread manager as soon as the runtime has been initialized. This function is expected to expose an interface as defined by the typedef *hpx_main_function_type*. This parameter is optional and defaults to none main thread function, in which case all threads have to be scheduled explicitly.

Returns

This function will return the value as returned as the result of the invocation of the function object given by the parameter `func`.

int **run**() override

Run the HPX runtime system, initially use the given number of (OS) threads in the thread-manager and block waiting for all threads to finish.

Returns

This function will always return 0 (zero).

bool **is_networking_enabled**() override

template<typename **F**>

inline *components*::*server*::console_error_dispatcher::sink_type **set_error_sink**(*F* &&sink)

performance_counters::registry &**get_counter_registry**()

Allow access to the registry counter registry instance used by the HPX runtime.

performance_counters::registry const &**get_counter_registry**() const

Allow access to the registry counter registry instance used by the HPX runtime.

void **register_query_counters**(*std*::shared_ptr<*util*::query_counters> const &active_counters)

void **start_active_counters**(*error_code* &ec = *throws*) const

void **stop_active_counters**(*error_code* &ec = *throws*) const

void **reset_active_counters**(*error_code* &ec = *throws*) const

void **reinit_active_counters**(bool reset = true, *error_code* &ec = *throws*) const

void **evaluate_active_counters**(bool reset = false, char const *description = nullptr, *error_code* &ec = *throws*) const void **stop_evaluating_counters**(bool terminate = false) const

agas::*addressing_service* &**get_agas_client**()

Allow access to the AGAS client instance used by the HPX runtime.

hpx::*threads*::*threadmanager* &**get_thread_manager**() override

Allow access to the thread manager instance used by the HPX runtime.

applier::*applier* &**get_applier**()

Allow access to the applier instance used by the HPX runtime.

std::string **here**() const override

Returns a string of the locality endpoints (usable in debug output)

naming::address_type **get_runtime_support_lva**() const

naming::gid_type **get_next_id**(*std*::size_t count = 1)

void **init_id_pool_range**()

util::unique_id_ranges &**get_id_pool**()

void **initialize_agas**()

Initialize AGAS operation.

void **add_pre_startup_function**(*startup_function_type* f) override

Add a function to be executed inside a HPX thread before hpx_main but guaranteed to be executed before any startup function registered with *add_startup_function*.

> **Note**
>
> The difference to a startup function is that all pre-startup functions will be (system-wide) executed before any startup function.

Parameters

f – The function 'f' will be called from inside a HPX thread before hpx_main is executed. This is very useful to setup the runtime environment of the application (install performance counters, etc.)

void **add_startup_function**(*startup_function_type* f) override

Add a function to be executed inside a HPX thread before hpx_main

Parameters

f – The function 'f' will be called from inside a HPX thread before hpx_main is executed. This is very useful to setup the runtime environment of the application (install performance counters, etc.)

void **add_pre_shutdown_function**(*shutdown_function_type* f) override

Add a function to be executed inside a HPX thread during hpx::finalize, but guaranteed before any of the shutdown functions is executed.

> **Note**
>
> The difference to a shutdown function is that all pre-shutdown functions will be (system-wide) executed before any shutdown function.

Parameters

f – The function 'f' will be called from inside a HPX thread while hpx::finalize is executed. This is very useful to tear down the runtime environment of the application (uninstall performance counters, etc.)

void **add_shutdown_function**(*shutdown_function_type* f) override

Add a function to be executed inside a HPX thread during hpx::finalize

Parameters

f – The function 'f' will be called from inside a HPX thread while hpx::finalize is executed. This is very useful to tear down the runtime environment of the application (uninstall performance counters, etc.)

hpx::*util*::*io_service_pool* ***get_thread_pool**(char const *name) override

Access one of the internal thread pools (io_service instances) HPX is using to perform specific tasks. The three possible values for the argument `name` are "main_pool", "io_pool", "parcel_pool", and "timer_pool". For any other argument value the function will return zero.

bool **register_thread**(char const *name, *std*::size_t num = 0, bool service_thread = true, *error_code* &ec = *throws*) override

Register an external OS-thread with HPX.

notification_policy_type **get_notification_policy**(char const *prefix, runtime_local::os_thread_type type) override

Generate a new notification policy instance for the given thread name prefix

std::uint32_t **get_locality_id**(*error_code* &ec) const override

std::size_t **get_num_worker_threads**() const override *std*::uint32_t **get_num_localities**(*hpx*::*launch*::sync_policy, *error_code* &ec) const override

std::uint32_t **get_initial_num_localities**() const override

hpx::*future*<*std*::uint32_t> **get_num_localities**() const override

std::string **get_locality_name**() const override

std::uint32_t **get_num_localities**(*hpx*::*launch*::sync_policy, *components*::component_type type, *error_code* &ec) const

hpx::*future*<*std*::uint32_t> **get_num_localities**(*components*::component_type type) const

std::uint32_t **assign_cores**(*std*::string const &locality_basename, *std*::uint32_t num_threads) override

std::uint32_t **assign_cores**() override

Public Static Functions

static void **register_counter_types**()

Install all performance counters related to this runtime instance.

Private Types

using **used_cores_map_type** = *std*::map<*std*::string, *std*::uint32_t>

Private Functions

threads::thread_result_type **run_helper**(*hpx*::*function*<*runtime*::*hpx_main_function_type*> const &func, int &result)

void **init_global_data**()

void **deinit_global_data**()

void **wait_helper**(*std*::mutex &mtx, *std*::condition_variable &cond, bool &running)

void **init_tss_helper**(char const *context, runtime_local::os_thread_type type, *std*::size_t local_thread_num, *std*::size_t global_thread_num, char const *pool_name, char const *postfix, bool service_thread)

void **deinit_tss_helper**(char const *context, *std*::size_t num) const

void **init_tss_ex**(*std*::string const &locality, char const *context, runtime_local::os_thread_type type, *std*::size_t local_thread_num, *std*::size_t global_thread_num, char const *pool_name, char const *postfix, bool service_thread, *error_code* &ec) const

Private Members

runtime_mode **mode_**

util::unique_id_ranges **id_pool_**

agas::*addressing_service* **agas_client_**

applier::*applier* **applier_**

used_cores_map_type **used_cores_map_** *std*::unique_ptr<*components*::*server*::*runtime_support*> **runtime_support_**

std::shared_ptr<*util*::query_counters> **active_counters_**

int (***pre_main_**)(*runtime_mode*)

void (***post_main_**)()

Private Static Functions

static void **default_errorsink**(*std*::string const&)

hpx::find_locality

Defined in header hpx/runtime.hpp[594].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::id_type *hpx*::**find_locality**(*components*::component_type type, *error_code* &ec = *throws*)

Return the global id representing an arbitrary locality which supports the given component type.

The function find_locality() can be used to retrieve the global id of an arbitrary locality currently available to this application which supports the creation of instances of the given component type.

See also

hpx::find_here(), hpx::find_all_localities()

Note

Generally, the id of a locality can be used for instance to create new instances of components and to invoke plain actions (global functions).

[594] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

Note

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of *hpx::exception*.

Note

This function will return meaningful results only if called from an HPX-thread. It will return *hpx::invalid_id* otherwise.

Parameters

- **type** – [in] The type of the components for which the function should return any available locality.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

The global id representing an arbitrary locality currently available to this application which supports the creation of instances of the given component type. If no locality supporting the given component type is currently available, this function will return *hpx::invalid_id*.

hpx/runtime_distributed/applier.hpp

Defined in header hpx/runtime_distributed/applier.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **applier**

class **applier**

#include <applier.hpp> The *applier* class is used to decide whether a particular action has to be issued on a local or a remote resource. If the target component is local a new *thread* will be created, if the target is remote a parcel will be sent.

Public Functions

applier(*applier* const&) = delete

applier(*applier*&&) = delete

applier &**operator=**(*applier* const&) = delete

applier &**operator=**(*applier*&&) = delete

applier()

void **init**(*threads*::*threadmanager* &tm)

~applier() = default

void **initialize**(*std*::uint64_t rts)

threads::*threadmanager* &**get_thread_manager**()

Access the *thread-manager* instance associated with this *applier*.

This function returns a reference to the thread manager this applier instance has been created with.

naming::gid_type const &**get_raw_locality**(*error_code* &ec = *throws*) const

Allow access to the locality of the locality this applier instance is associated with.

This function returns a reference to the locality this applier instance is associated with.

std::uint32_t **get_locality_id**(*error_code* &ec = *throws*) const

Allow access to the id of the locality this applier instance is associated with.

This function returns a reference to the id of the locality this applier instance is associated with.

bool **get_raw_remote_localities**(*std*::vector<*naming*::gid_type> &locality_ids, *components*::component_type type = to_int(*hpx*::*components*::*component_enum_type*::*invalid*), *error_code* &ec = *throws*) const

Return list of localities of all remote localities registered with the AGAS service for a specific component type.

This function returns a list of all remote localities (all localities known to AGAS except the local one) supporting the given component type.

Parameters

- `locality_ids` – [out] The reference to a vector of id_types filled by the function.
- `type` – [in] The type of the component which needs to exist on the returned localities.

Returns

The function returns *true* if there is at least one remote locality known to the AGAS service (!prefixes.empty()).

bool `get_remote_localities`(*std*::vector<*hpx*::id_type> &locality_ids, *components*::component_type type = to_int(*hpx*::*components*::*component_enum_type*::*invalid*), *error_code* &ec = *throws*) const

bool `get_raw_localities`(*std*::vector<*naming*::gid_type> &locality_ids, *components*::component_type type = to_int(*hpx*::*components*::*component_enum_type*::*invalid*)) const

Return list of locality_ids of all localities registered with the AGAS service for a specific component type.

This function returns a list of all localities (all localities known to AGAS except the local one) supporting the given component type.

Parameters

- `locality_ids` – [out] The reference to a vector of id_types filled by the function.
- `type` – [in] The type of the component which needs to exist on the returned localities.

Returns

The function returns *true* if there is at least one remote locality known to the AGAS service (!prefixes.empty()).

bool `get_localities`(*std*::vector<*hpx*::id_type> &locality_ids, *error_code* &ec = *throws*) const

bool `get_localities`(*std*::vector<*hpx*::id_type> &locality_ids, *components*::component_type type, *error_code* &ec = *throws*) const

inline *naming*::gid_type const &`get_runtime_support_raw_gid`() const

By convention the runtime_support has a gid identical to the prefix of the locality the runtime_support is responsible for

inline *hpx*::id_type const &`get_runtime_support_gid`() const

By convention the runtime_support has a gid identical to the prefix of the locality the runtime_support is responsible for

Private Members

threads::*threadmanager* ***thread_manager_**

hpx::id_type **runtime_support_id_**

hpx/runtime_distributed/get_num_localities.hpp

Defined in header hpx/runtime_distributed/get_num_localities.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

hpx::*future*<*std*::uint32_t> **get_num_localities**(*components*::component_type t)

Asynchronously return the number of localities which are currently registered for the running application.

The function *get_num_localities* asynchronously returns the number of localities currently connected to the console which support the creation of the given component type. The returned future represents the actual result.

> **See also**
>
> *hpx::find_all_localities*, *hpx::get_num_localities*

> **Note**
>
> This function will return meaningful results only if called from an HPX-thread. It will return 0 otherwise.

Parameters

t – The component type for which the number of connected localities should be retrieved.

std::uint32_t **get_num_localities**(*launch*::sync_policy, *components*::component_type t, *error_code* &ec = *throws*)

Synchronously return the number of localities which are currently registered for the running application.

The function *get_num_localities* returns the number of localities currently connected to the console which support the creation of the given component type. The returned future represents the actual result.

> **See also**
>
> *hpx::find_all_localities*, *hpx::get_num_localities*

> **Note**
>
> This function will return meaningful results only if called from an HPX-thread. It will return 0 otherwise.

Parameters

- `t` – The component type for which the number of connected localities should be retrieved.
- `ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

hpx/runtime_distributed/get_locality_name.hpp

Defined in header hpx/runtime_distributed/get_locality_name.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

future<*std*::string> **get_locality_name**(*hpx*::id_type const &id)

Return the name of the referenced locality.

This function returns a future referring to the name for the locality of the given id.

> **See also**
>
> *std::string get_locality_name()*

Parameters

`id` – [in] The global id of the locality for which the name should be retrieved

Returns

This function returns the name for the locality of the given id. The name is retrieved from the underlying networking layer and may be different for different parcel ports.

hpx::find_root_locality

Defined in header hpx/runtime.hpp[595].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::find_all_localities*
- *hpx::find_remote_localities*

hpx::id_type **hpx::find_root_locality**(*error_code* &ec = *throws*)

Return the global id representing the root locality.

The function find_root_locality() can be used to retrieve the global id usable to refer to the root locality. The root locality is the locality where the main AGAS service is hosted.

> **See also**
>
> hpx::find_all_localities(), hpx::find_locality()

> **Note**
>
> Generally, the id of a locality can be used for instance to create new instances of components and to invoke plain actions (global functions).

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of *hpx::exception*.

> **Note**
>
> This function will return meaningful results only if called from an HPX-thread. It will return *hpx::invalid_id* otherwise.

Parameters

`ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

The global id representing the root locality for this application.

[595] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

hpx::find_all_localities

Defined in header hpx/runtime.hpp[596].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::find_root_locality*
- *hpx::find_remote_localities*

std::vector<*hpx*::id_type> **hpx::find_all_localities**(*error_code* &ec = *throws*)

Return the list of global ids representing all localities available to this application.

The function find_all_localities() can be used to retrieve the global ids of all localities currently available to this application.

> **See also**
>
> *hpx::find_here()*, hpx::find_locality()

> **Note**
>
> Generally, the id of a locality can be used for instance to create new instances of components and to invoke plain actions (global functions).

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of *hpx::exception*.

> **Note**
>
> This function will return meaningful results only if called from an HPX-thread. It will return an empty vector otherwise.

Parameters

`ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

The global ids representing the localities currently available to this application.

std::vector<*hpx*::id_type> **hpx::find_all_localities**(*components*::component_type type, *error_code* &ec = *throws*)

[596] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

Return the list of global ids representing all localities available to this application which support the given component type.

The function find_all_localities() can be used to retrieve the global ids of all localities currently available to this application which support the creation of instances of the given component type.

> **See also**
>
> *hpx::find_here()*, hpx::find_locality()

> **Note**
>
> Generally, the id of a locality can be used for instance to create new instances of components and to invoke plain actions (global functions).

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of *hpx::exception*.

> **Note**
>
> This function will return meaningful results only if called from an HPX-thread. It will return an empty vector otherwise.

Parameters

- **`type`** – [in] The type of the components for which the function should return the available localities.
- **`ec`** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

The global ids representing the localities currently available to this application which support the creation of instances of the given component type. If no localities supporting the given component type are currently available, this function will return an empty vector.

hpx::find_remote_localities

Defined in header hpx/runtime.hpp[597].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::find_root_locality*
- *hpx::find_all_localities*

std::vector<*hpx*::id_type> **hpx::find_remote_localities**(*error_code* &ec = *throws*)

Return the list of locality ids of remote localities supporting the given component type. By default this function will return the list of all remote localities (all but the current locality).

The function find_remote_localities() can be used to retrieve the global ids of all remote localities currently available to this application (i.e. all localities except the current one).

See also

hpx::find_here(), hpx::find_locality()

Note

Generally, the id of a locality can be used for instance to create new instances of components and to invoke plain actions (global functions).

Note

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of *hpx::exception*.

Note

This function will return meaningful results only if called from an HPX-thread. It will return an empty vector otherwise.

Parameters

ec – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

The global ids representing the remote localities currently available to this application.

[597] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

std::vector<*hpx*::id_type> `hpx::find_remote_localities`(*components*::component_type type, *error_code* &ec = *throws*)

Return the list of locality ids of remote localities supporting the given component type. By default this function will return the list of all remote localities (all but the current locality).

The function find_remote_localities() can be used to retrieve the global ids of all remote localities currently available to this application (i.e. all localities except the current one) which support the creation of instances of the given component type.

See also

hpx::find_here(), hpx::find_locality()

Note

Generally, the id of a locality can be used for instance to create new instances of components and to invoke plain actions (global functions).

Note

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of *hpx::exception*.

Note

This function will return meaningful results only if called from an HPX-thread. It will return an empty vector otherwise.

Parameters

- **type** – [in] The type of the components for which the function should return the available remote localities.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

The global ids representing the remote localities currently available to this application.

hpx/runtime_distributed/big_boot_barrier.hpp

Defined in header hpx/runtime_distributed/big_boot_barrier.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/runtime_distributed/migrate_component.hpp

Defined in header hpx/runtime_distributed/migrate_component.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

Functions

template<typename **Component**, typename **DistPolicy**>

future<*hpx*::id_type> **migrate**(*hpx*::id_type const &to_migrate, [[maybe_unused]] *DistPolicy* const &policy)

Migrate the given component to the specified target locality

The function *migrate<Component>* will migrate the component referenced by *to_migrate* to the locality specified with *target_locality*. It returns a future referring to the migrated component instance.

Parameters

- **to_migrate** – [in] The client side representation of the component to migrate.
- **policy** – [in] A distribution policy which will be used to determine the locality to migrate this object to.

Template Parameters

- **Component** – Specifies the component type of the component to migrate.
- **DistPolicy** – Specifies the distribution policy to use to determine the destination locality.

Returns

A future representing the global id of the migrated component instance. This should be the same as *migrate_to*.

template<typename **Derived**, typename **Stub**, typename **Data**, typename **DistPolicy**>

Derived **migrate**(*client_base*<*Derived*, *Stub*, *Data*> const &to_migrate, *DistPolicy* const &policy)

Migrate the given component to the specified target locality

The function *migrate<Component>* will migrate the component referenced by *to_migrate* to the locality specified with *target_locality*. It returns a future referring to the migrated component instance.

Parameters

- **to_migrate** – [in] The client side representation of the component to migrate.
- **policy** – [in] A distribution policy which will be used to determine the locality to migrate this object to.

Template Parameters

- **Derived** – Specifies the component type of the component to migrate.
- **DistPolicy** – Specifies the distribution policy to use to determine the destination locality.

Returns

A future representing the global id of the migrated component instance. This should be the same as *migrate_to*.

template<typename **Component**>

future<*hpx*::id_type> **migrate**(*hpx*::id_type const &to_migrate, *hpx*::id_type const &target_locality)

Migrate the component with the given id to the specified target locality

The function *migrate*<*Component*> will migrate the component referenced by *to_migrate* to the locality specified with *target_locality*. It returns a future referring to the migrated component instance.

Parameters

- **to_migrate** – [in] The global id of the component to migrate.
- **target_locality** – [in] The locality where the component should be migrated to.

Template Parameters

Component – Specifies the component type of the component to migrate.

Returns

A future representing the global id of the migrated component instance. This should be the same as *migrate_to*.

template<typename **Derived**, typename **Stub**, typename **Data**>

Derived **migrate**(*client_base*<*Derived*, *Stub*, *Data*> const &to_migrate, *hpx*::id_type const &target_locality)

Migrate the given component to the specified target locality

The function *migrate*<*Component*> will migrate the component referenced by *to_migrate* to the locality specified with *target_locality*. It returns a future referring to the migrated component instance.

Parameters

- **to_migrate** – [in] The client side representation of the component to migrate.
- **target_locality** – [in] The id of the locality to migrate this object to.

Template Parameters

Derived – Specifies the component type of the component to migrate.

Returns

A client side representation of representing of the migrated component instance. This should be the same as *migrate_to*.

hpx/runtime_distributed/copy_component.hpp

Defined in header hpx/runtime_distributed/copy_component.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

Functions

template<typename **Component**>

future<*hpx*::id_type> **copy**(*hpx*::id_type const &to_copy)

Copy given component to the specified target locality.

The function *copy*<*Component*> will create a copy of the component referenced by *to_copy* on the locality specified with *target_locality*. It returns a future referring to the newly created component instance.

> **Note**
>
> The new component instance is created on the locality of the component instance which is to be copied.

Parameters
to_copy – [in] The global id of the component to copy

Template Parameters
The – only template argument specifies the component type to create.

Returns
A future representing the global id of the newly (copied) component instance.

template<typename **Component**>

future<*hpx*::id_type> **copy**(*hpx*::id_type const &to_copy, *hpx*::id_type const &target_locality)

Copy given component to the specified target locality.

The function *copy*<*Component*> will create a copy of the component referenced by *to_copy* on the locality specified with *target_locality*. It returns a future referring to the newly created component instance.

Parameters

- **to_copy** – [in] The global id of the component to copy
- **target_locality** – [in] The locality where the copy should be created.

Template Parameters
The – only template argument specifies the component type to create.

Returns
A future representing the global id of the newly (copied) component instance.

template<typename **Derived**, typename **Stub**, typename **Data**>

Derived **copy**(*client_base*<*Derived*, *Stub*, *Data*> const &to_copy, *hpx*::id_type const &target_locality = *hpx*::invalid_id)

Copy given component to the specified target locality.

The function *copy* will create a copy of the component referenced by the client side object *to_copy* on the locality specified with *target_locality*. It returns a new client side object future referring to the newly created component instance.

Note

If the second argument is omitted (or is invalid_id) the new component instance is created on the locality of the component instance which is to be copied.

Parameters

- **to_copy** – [in] The client side object representing the component to copy
- **target_locality** – [in, optional] The locality where the copy should be created (default is same locality as source).

Template Parameters

The – only template argument specifies the component type to create.

Returns

A future representing the global id of the newly (copied) component instance.

hpx/runtime_distributed/runtime_support.hpp

Defined in header hpx/runtime_distributed/runtime_support.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **agas**

Functions

struct runtime_components_init_interface_functions &**runtime_components_init**()

namespace **components**

Functions

struct counter_interface_functions &**counter_init**()

class **runtime_support** : public *hpx*::*components*::*stubs*::*runtime_support*

#include <runtime_support.hpp> The runtime_support class is the client side representation of a server::runtime_support component

Public Functions

inline **runtime_support**(*hpx*::id_type const &gid = *hpx*::invalid_id)

Create a client side representation for the existing server::runtime_support instance with the given global id *gid*.

template<typename **Component**, typename ...**Ts**>

inline *hpx*::id_type **create_component**(*Ts*&&... vs)

Create a new component type using the *runtime_support*.

template<typename **Component**, typename ...**Ts**>

inline *hpx*::*future*<*hpx*::id_type> **create_component_async**(*Ts*&&... vs)

Asynchronously create a new component using the *runtime_support*.

template<bool **WithCount**, typename **Component**, typename ...**Ts**>

inline *std*::vector<*hpx*::id_type> **bulk_create_component**(*std*::size_t count, *Ts*&&... vs)

Asynchronously create N new default constructed components using the *runtime_support*

template<bool **WithCount**, typename **Component**, typename ...**Ts**>

inline *hpx*::*future*<*std*::vector<*hpx*::id_type>> **bulk_create_components_async**(*std*::size_t count, *Ts*&&... vs)

Asynchronously create a new component using the *runtime_support*.

inline *hpx*::*future*<int> **load_components_async**() const

inline int **load_components**() const

inline *hpx*::*future*<void> **call_startup_functions_async**(bool pre_startup) const inline void **call_startup_functions**(bool pre_startup) const

inline *hpx*::*future*<void> **shutdown_async**(double timeout = -1) const

Shutdown the given runtime system.

inline void **shutdown**(double timeout = -1) const

inline void **shutdown_all**(double timeout = -1) const

Shutdown the runtime systems of all localities.

inline *hpx*::*future*<void> **terminate_async**() const

Terminate the given runtime system.

inline void **terminate**() const

inline void **terminate_all**() const

Terminate the runtime systems of all localities.

inline void **get_config**(*util*::section &ini) const

Retrieve configuration information.

inline *hpx*::id_type const &**get_id**() const

inline *naming*::gid_type const &**get_raw_gid**() const

Private Types

using **base_type** = *stubs*::*runtime_support*

Private Members

hpx::id_type **gid_**

hpx/runtime_distributed/find_here.hpp

Defined in header hpx/runtime_distributed/find_here.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

hpx::id_type **find_here**(*error_code* &ec = *throws*)

Return the global id representing this locality.

The function find_here() can be used to retrieve the global id usable to refer to the current locality.

See also

hpx::find_all_localities(), *hpx::find_locality()*

Note

Generally, the id of a locality can be used for instance to create new instances of components and to invoke plain actions (global functions).

Note

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Note

This function will return meaningful results only if called from an HPX-thread. It will return *hpx::invalid_id* otherwise.

Parameters
`ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns
The global id representing the locality this function has been called on.

hpx/runtime_distributed/runtime_fwd.hpp

Defined in header hpx/runtime_distributed/runtime_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

runtime_distributed &**get_runtime_distributed**()

runtime_distributed *&**get_runtime_distributed_ptr**()

naming::gid_type const &**get_locality**()

The function *get_locality* returns a reference to the locality prefix.

void **start_active_counters**(*error_code* &ec = *throws*)

Start all active performance counters, optionally naming the section of code.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

> **Note**
>
> The active counters are those which have been specified on the command line while executing the application (see command line option –hpx:print-counter)

Parameters
`ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

void **reset_active_counters**(*error_code* &ec = *throws*)

Resets all active performance counters.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

> **Note**
>
> The active counters are those which have been specified on the command line while executing the application (see command line option –hpx:print-counter)

Parameters

`ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

void **reinit_active_counters**(bool reset = true, *error_code* &ec = *throws*)

Re-initialize all active performance counters.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

> **Note**
>
> The active counters are those which have been specified on the command line while executing the application (see command line option –hpx:print-counter)

Parameters

- `reset` – [in] Reset the current values before re-initializing counters (default: true)
- `ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

void **stop_active_counters**(*error_code* &ec = *throws*)

Stop all active performance counters.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

> **Note**
>
> The active counters are those which have been specified on the command line while executing the application (see command line option –hpx:print-counter)

Parameters

ec – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

void **evaluate_active_counters**(bool reset = false, char const *description = nullptr, *error_code* &ec = *throws*)

Evaluate and output all active performance counters, optionally naming the point in code marked by this function.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

> **Note**
>
> The output generated by this function is redirected to the destination specified by the corresponding command line options (see –hpx:print-counter-destination).

> **Note**
>
> The active counters are those which have been specified on the command line while executing the application (see command line option –hpx:print-counter)

Parameters

- **reset** – [in] this is an optional flag allowing to reset the counter value after it has been evaluated.
- **description** – [in] this is an optional value naming the point in the code marked by the call to this function.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

serialization::binary_filter ***create_binary_filter**(char const *binary_filter_type, bool compress, *serialization*::binary_filter *next_filter = nullptr, *error_code* &ec = *throws*)

Create an instance of a binary filter plugin.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Parameters

- **binary_filter_type** – [in] The type of the binary filter to create
- **compress** – [in] The created filter should support compression
- **next_filter** – [in] Use this as the filter to dispatch the invocation into.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

hpx/runtime_distributed/applier_fwd.hpp

Defined in header hpx/runtime_distributed/applier_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **applier**

Functions

applier &**get_applier**()

The function *get_applier* returns a reference to the (thread specific) applier instance.

applier ***get_applier_ptr**()

The function *get_applier* returns a pointer to the (thread specific) applier instance. The returned pointer is NULL if the current thread is not known to HPX or if the runtime system is not active.

namespace **applier**

The namespace *applier* contains all definitions needed for the class hpx::applier::applier and its related functionality. This namespace is part of the HPX core module.

hpx/runtime_distributed/server/copy_component.hpp

Defined in header hpx/runtime_distributed/server/copy_component.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

namespace **server**

Functions

template<typename **Component**>

hpx::id_type **copy_component_here**(*hpx*::id_type const &to_copy)

template<typename **Component**>

future<*hpx*::id_type> **copy_component**(*hpx*::id_type const &to_copy, *hpx*::id_type const &target_locality)

template<typename **Component**>

struct **copy_component_action_here** : public *hpx*::*actions*::action<*hpx*::id_type (*)(*hpx*::id_type const&), &*copy_component_here*<*Component*>, *copy_component_action_here*<*Component*>>

template<typename **Component**>

struct **copy_component_action** : public *hpx*::*actions*::action<*future*<*hpx*::id_type> (*)(*hpx*::id_type const&, *hpx*::id_type const&), &*copy_component*<*Component*>, *copy_component_action*<*Component*>>

hpx/runtime_distributed/server/runtime_support.hpp

Defined in header hpx/runtime_distributed/server/runtime_support.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

namespace **server**

class **runtime_support**

Public Types

using **type_holder** = *runtime_support*

Public Functions

explicit **runtime_support**(*hpx*::*util*::*runtime_configuration* &cfg)

inline **~runtime_support**()

void **delete_function_lists**()

void **tidy**()

template<typename **Component**>

naming::gid_type **create_component**()

Actions to create new objects.

template<typename **Component**, typename **T**, typename ...**Ts**>

naming::gid_type **create_component**(*T* v, *Ts*... vs)

template<typename **Component**, typename ...**Ts**>

std::vector<*naming*::gid_type> **bulk_create_component**(*std*::size_t count, *Ts*... vs)

template<typename **Component**, typename ...**Ts**>

std::vector<*naming*::gid_type> **bulk_create_component_with_count**(*std*::size_t count, *Ts*... vs)

template<typename **Component**>

naming::gid_type **copy_create_component**(*std*::shared_ptr<*Component*> const &p, bool)

template<typename **Component**>

naming::gid_type **migrate_component_to_here**(*std*::shared_ptr<*Component*> const &p, *hpx*::id_type)

void **shutdown**(double timeout, *hpx*::id_type const &respond_to, bool force_disconnect)

Gracefully shutdown this runtime system instance.

void **shutdown_all**(double timeout)

Gracefully shutdown runtime system instances on all localities.

void **terminate**(*hpx*::id_type const &respond_to)

Shutdown this runtime system instance.

inline void **terminate_act**(*hpx*::id_type const &id)

void **terminate_all**()

Shutdown runtime system instances on all localities.

inline void **terminate_all_act**()

util::section **get_config**()

Retrieve configuration information.

int **load_components**()

Load all components on this locality.

void **call_startup_functions**(bool pre_startup)

void **call_shutdown_functions**(bool pre_shutdown)

void **garbage_collect**()

Force a garbage collection operation in the AGAS layer.

naming::gid_type **create_performance_counter**(performance_counters::counter_info const &info)

Create the given performance counter instance.

void **remove_from_connection_cache**(*naming*::gid_type const &gid, *parcelset*::endpoints_type const &eps)

Remove the given locality from our connection cache.

HPX_DEFINE_COMPONENT_ACTION (runtime_support, terminate_act, terminate_action) HPX_DEFINE_COMPONENT_ACTION(runtime_support

termination detection

terminate_all_action HPX_DEFINE_COMPONENT_ACTION (runtime_support, remove_from_connection_cache) void run()

Start the *runtime_support* component.

void **wait**()

Wait for the *runtime_support* component to notify the calling thread.

This function will be called from the main thread, causing it to block while the HPX functionality is executed. The main thread will block until the shutdown_action is executed, which in turn notifies all waiting threads.

void **stop**(double timeout, *hpx*::id_type const &respond_to, bool remove_from_remote_caches, bool force_disconnect)

Notify all waiting (blocking) threads allowing the system to be properly stopped.

> **Note**
>
> This function can be called from any thread.

bool **remove_locality**(*hpx*::id_type const &locality, *error_code* &ec = *hpx*::*throws*)

Remove the given locality from this locality's runtime support component.

> **Note**
>
> This function must be called locally only; it is not exposed as a component action.

Parameters

- `locality` – The locality to be removed.
- `ec` – Used to hold error code value originating from the AGAS operations invoked by this function.

void **stopped**()

void **notify_waiting_main**()

inline bool **was_stopped**() const

void **add_pre_startup_function**(*startup_function_type* f)

void **add_startup_function**(*startup_function_type* f)

void **add_pre_shutdown_function**(*shutdown_function_type* f)

void **add_shutdown_function**(*shutdown_function_type* f)

Public Members

terminate_all_act

Public Static Functions

static inline component_type **get_component_type**()

static inline void **set_component_type**(component_type t) static inline constexpr void **finalize**()

finalize() will be called just before the instance gets destructed

static inline bool **is_target_valid**(*hpx*::id_type const &id)

static void **remove_locality_from_connection_cache**(*hpx*::*naming*::gid_type const &locality, bool skip_current = false)

Broadcast a request to remove the given locality from the connection caches of all remote, non-console localities.

Parameters

- **locality** – The locality to remove from the remote connection caches.
- **skip_current** – If true, omits the locality identified by *locality* from this broadcast (that locality will not receive a removal request for itself).

static void **remove_locality_from_console_connection_cache**(*hpx*::*naming*::gid_type const &locality)

Remove the given locality from the console's connection cache.

Parameters

locality – The locality to remove from the console's connection cache.

Protected Functions

int **load_components**(*util*::section &ini, *naming*::gid_type const &prefix, *agas*::*addressing_service* &agas_client, *hpx*::program_options::options_description &options, *std*::set<*std*::string> &startup_handled)

bool **load_component**(*hpx*::*util*::plugin::dll &d, *util*::section &ini, *std*::string const &instance, *std*::string const &component, *filesystem*::path const &lib, *naming*::gid_type const &prefix, *agas*::*addressing_service* &agas_client, bool isdefault, bool isenabled, *hpx*::program_options::options_description &options, *std*::set<*std*::string> &startup_handled)

bool **load_component_dynamic**(*util*::section &ini, *std*::string const &instance, *std*::string const &component, *filesystem*::path lib, *naming*::gid_type const &prefix, *agas*::*addressing_service* &agas_client, bool isdefault, bool isenabled, *hpx*::program_options::options_description &options, *std*::set<*std*::string> &startup_handled)

bool **load_startup_shutdown_functions**(*hpx*::*util*::plugin::dll &d, *error_code* &ec)

bool **load_commandline_options**(*hpx*::*util*::plugin::dll &d, *hpx*::program_options::options_description &options, *error_code* &ec)

bool **load_component_static**(*util*::section &ini, *std*::string const &instance, *std*::string const &component, *filesystem*::path const &lib, *naming*::gid_type const &prefix, *agas*::*addressing_service* &agas_client, bool isdefault, bool isenabled, *hpx*::program_options::options_description &options, *std*::set<*std*::string> &startup_handled)

bool **load_startup_shutdown_functions_static**(*std*::string const &mod, *error_code* &ec)

bool **load_commandline_options_static**(*std*::string const &mod, *hpx*::program_options::options_description &options, *error_code* &ec)

bool **load_plugins**(*util*::section &ini, *hpx*::program_options::options_description &options, *std*::set<*std*::string> &startup_handled)

bool **load_plugin**(*hpx*::*util*::plugin::dll &d, *util*::section &ini, *std*::string const &instance, *std*::string const &component, *filesystem*::path const &lib, bool isenabled, *hpx*::program_options::options_description &options, *std*::set<*std*::string> &startup_handled)

bool **load_plugin_dynamic**(*util*::section &ini, *std*::string const &instance, *std*::string const &component, *filesystem*::path lib, bool isenabled, *hpx*::program_options::options_description &options, *std*::set<*std*::string> &startup_handled)

bool **load_plugin_static**(*util*::section &ini, *std*::string const &instance, *std*::string const &plugin, bool isenabled, *hpx*::program_options::options_description &options, *std*::set<*std*::string> &startup_handled)

std::size_t **dijkstra_termination_detection**(*std*::vector<*hpx*::id_type> const &locality_ids)

Private Types

using **plugin_map_mutex_type** = *hpx*::*spinlock*

using **plugin_factory_type** = *plugin_factory*

using **plugin_map_type** = *std*::map<*std*::string, *plugin_factory_type*>

using **modules_map_type** = *std*::map<*std*::string, *hpx*::*util*::plugin::dll>

using **static_modules_type** = *std*::vector<static_factory_load_data_type>

Private Members

std::mutex **mtx_**

std::condition_variable **wait_condition_**

std::condition_variable **stop_condition_**

bool **stop_called_**

bool **stop_done_**

bool **terminated_**

std::thread::id **main_thread_id_**

std::atomic<bool> **shutdown_all_invoked_** *plugin_map_mutex_type* **p_mtx_**

plugin_map_type **plugins_**

modules_map_type &**modules_**

static_modules_type **static_modules_**

hpx::*spinlock* **globals_mtx_**

std::list<*startup_function_type*> **pre_startup_functions_**

std::list<*startup_function_type*> **startup_functions_**

std::list<*shutdown_function_type*> **pre_shutdown_functions_**

std::list<*shutdown_function_type*> **shutdown_functions_**

struct **plugin_factory**

Public Functions

inline **plugin_factory**(*std*::shared_ptr<*plugins*::plugin_factory_base> const &f, bool enabled)

Public Members

std::shared_ptr<*plugins*::plugin_factory_base> **first**

bool **isenabled**

hpx/runtime_distributed/stubs/runtime_support.hpp

Defined in header hpx/runtime_distributed/stubs/runtime_support.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

namespace **stubs**

struct **runtime_support**

Subclassed by *hpx::components::runtime_support*

Public Static Functions

template<typename **Component**, typename ...**Ts**>

static inline *hpx*::*future*<*hpx*::id_type> **create_component_async**(*hpx*::id_type const &gid, *Ts*&&... vs)

Create a new component *type* using the *runtime_support* with the given *targetgid*. This is a non-blocking call. The caller needs to call *future::get* on the result of this function to obtain the global id of the newly created object.

template<typename **Component**, typename ...**Ts**>

static inline *hpx*::id_type **create_component**(*hpx*::id_type const &gid, *Ts*&&... vs)

Create a new component *type* using the *runtime_support* with the given *targetgid*. Block for the creation to finish.

template<bool **WithCount**, typename **Component**, typename ...**Ts**>

static inline *hpx*::*future*<*std*::vector<*hpx*::id_type>> **bulk_create_component_colocated_async**(*hpx*::id_type const &gid, *std*::size_t count, *Ts*&&... vs)

Create multiple new components *type* using the *runtime_support* colocated with the given *targetgid*. This is a non-blocking call.

template<bool **WithCount**, typename **Component**, typename ...**Ts**>

static inline *std*::vector<*hpx*::id_type> **bulk_create_component_colocated**(*hpx*::id_type const &gid, *std*::size_t count, *Ts*&&... vs)

Create multiple new components *type* using the *runtime_support* colocated with the given *targetgid*. Block for the creation to finish.

template<bool **WithCount**, typename **Component**, typename ...**Ts**>

static inline *hpx*::*future*<*std*::vector<*hpx*::id_type>> **bulk_create_component_async**(*hpx*::id_type const &gid, *std*::size_t count, *Ts*&&... vs)

Create multiple new components *type* using the *runtime_support* on the given locality. This is a non-blocking call.

template<bool **WithCount**, typename **Component**, typename ...**Ts**>

static inline *std*::vector<*hpx*::id_type> **bulk_create_component**(*hpx*::id_type const &gid, *std*::size_t count, *Ts*&&... vs)

Create multiple new components *type* using the *runtime_support* on the given locality. Block for the creation to finish.

template<typename **Component**, typename ...**Ts**>

static inline *hpx*::*future*<*hpx*::id_type> **create_component_colocated_async**(*hpx*::id_type const &gid, *Ts*&&... vs)

Create a new component *type* using the *runtime_support* with the given *targetgid*. This is a non-blocking call. The caller needs to call *future::get* on the result of this function to obtain the global id of the newly created object.

template<typename **Component**, typename ...**Ts**>

static inline *hpx*::id_type **create_component_colocated**(*hpx*::id_type const &gid, *Ts*&&... vs)

Create a new component *type* using the *runtime_support* with the given *targetgid*. Block for the creation to finish.

template<typename **Component**>

static inline *hpx*::*future*<*hpx*::id_type> **copy_create_component_async**(*hpx*::id_type const &gid, *std*::shared_ptr<*Component*> const &p, bool local_op)

template<typename **Component**>

static inline *hpx*::id_type **copy_create_component**(*hpx*::id_type const &gid, *std*::shared_ptr<*Component*> const &p, bool local_op)

template<typename **Component**>

static inline *hpx*::*future*<*hpx*::id_type> **migrate_component_async**(*hpx*::id_type const &target_locality, *std*::shared_ptr<*Component*> const &p, *hpx*::id_type const &to_migrate)

template<typename **Component**, typename **DistPolicy**>

static inline *hpx*::*future*<*hpx*::id_type> **migrate_component_async**(*DistPolicy* const &policy, *std*::shared_ptr<*Component*> const &p, *hpx*::id_type const &to_migrate)

template<typename **Component**, typename **Target**>

static inline *hpx*::id_type **migrate_component**(*Target* const &target, *hpx*::id_type const &to_migrate, *std*::shared_ptr<*Component*> const &p)

static *hpx*::*future*<int> **load_components_async**(*hpx*::id_type const &gid)

static int **load_components**(*hpx*::id_type const &gid)

static *hpx*::*future*<void> **call_startup_functions_async**(*hpx*::id_type const &gid, bool pre_startup) static void **call_startup_functions**(*hpx*::id_type const &gid, bool pre_startup)

static *hpx*::*future*<void> **shutdown_async**(*hpx*::id_type const &targetid, double timeout = -1, bool force_disconnect = false)

Shutdown the given runtime system.

static void **shutdown**(*hpx*::id_type const &targetgid, double timeout = -1, bool force_disconnect = false)

static void **shutdown_all**(*hpx*::id_type const &targetgid, double timeout = -1)

Shutdown the runtime systems of all localities.

static void **shutdown_all**(double timeout = -1)

static *hpx*::*future*<void> **terminate_async**(*hpx*::id_type const &targetgid)

Retrieve configuration information.

Terminate the given runtime system

static void **terminate**(*hpx*::id_type const &targetgid)

static void **terminate_all**(*hpx*::id_type const &targetgid)

Terminate the runtime systems of all localities.

static void **terminate_all**()

static void **garbage_collect_non_blocking**(*hpx*::id_type const &targetgid)

static *hpx*::*future*<void> **garbage_collect_async**(*hpx*::id_type const &targetgid)

static void **garbage_collect**(*hpx*::id_type const &targetgid)

static *hpx*::*future*<*hpx*::id_type> **create_performance_counter_async**(*hpx*::id_type const &targetgid, performance_counters::counter_info const &info)

static *hpx*::id_type **create_performance_counter**(*hpx*::id_type const &targetgid, performance_counters::counter_info const &info, *error_code* &ec = *throws*)

static *hpx*::*future*<*util*::section> **get_config_async**(*hpx*::id_type const &targetgid)

Retrieve configuration information.

static void **get_config**(*hpx*::id_type const &targetgid, *util*::section &ini)

static void **remove_from_connection_cache_async**(*hpx*::id_type const &target, *naming*::gid_type const &gid, *parcelset*::endpoints_type const &endpoints)

segmented_algorithms

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/parallel/segmented_algorithms/exclusive_scan.hpp

Defined in header hpx/parallel/segmented_algorithms/exclusive_scan.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **InIter**, typename **OutIter**, typename **T**, typename **Op** = *std*::plus<*T*>>

OutIter **hpx_invoke**(*hpx*::exclusive_scan_t, *InIter* first, *InIter* last, *OutIter* dest, *T* init, *Op* &&op = *Op*())

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **T**, typename **Op** = *std*::plus<*T*>>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> **hpx_invoke**(*hpx*::exclusive_scan_t, *ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *T* init, *Op* &&op = *Op*())

hpx/parallel/segmented_algorithms/is_partitioned.hpp

Defined in header hpx/parallel/segmented_algorithms/is_partitioned.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **InIter**, typename **Pred**>

bool **hpx_invoke**(*hpx*::is_partitioned_t, *InIter* first, *InIter* last, *Pred* &&pred)

template<typename **ExPolicy**, typename **SegIter**, typename **Pred**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> **hpx_invoke**(*hpx*::is_partitioned_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *Pred* &&pred)

hpx/parallel/segmented_algorithms/inclusive_scan.hpp

Defined in header hpx/parallel/segmented_algorithms/inclusive_scan.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **InIter**, typename **OutIter**, typename **Op** = *std*::plus<>>

OutIter **hpx_invoke**(*hpx*::inclusive_scan_t, *InIter* first, *InIter* last, *OutIter* dest, *Op* &&op = *Op*())

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Op** = *std*::plus<>>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> **hpx_invoke**(*hpx*::inclusive_scan_t, *ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *Op* &&op = *Op*())

template<typename **InIter**, typename **OutIter**, typename **Op**, typename **T**>

OutIter **hpx_invoke**(*hpx*::inclusive_scan_t, *InIter* first, *InIter* last, *OutIter* dest, *Op* &&op, *T* &&init)

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Op**, typename **T**>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> **hpx_invoke**(*hpx*::inclusive_scan_t, *ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *Op* &&op, *T* &&init)

hpx/parallel/segmented_algorithms/equal.hpp

Defined in header hpx/parallel/segmented_algorithms/equal.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **SegIter1**, typename **SegIter2**, typename **Pred** = *hpx*::*parallel*::detail::equal_to>

bool **hpx_invoke**(*hpx*::equal_t, *SegIter1* first1, *SegIter1* last1, *SegIter2* first2, *Pred* &&pred = *Pred*())

template<typename **ExPolicy**, typename **SegIter1**, typename **SegIter2**, typename **Pred** = *hpx*::*parallel*::detail::equal_to>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> **hpx_invoke**(*hpx*::equal_t, *ExPolicy* &&policy, *SegIter1* first1, *SegIter1* last1, *SegIter2* first2, *Pred* &&pred = *Pred*())

hpx/parallel/segmented_algorithms/mismatch.hpp

Defined in header hpx/parallel/segmented_algorithms/mismatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **SegIter1**, typename **SegIter2**, typename **Pred** = *hpx*::*parallel*::detail::equal_to>

std::pair<*SegIter1*, *SegIter2*> **hpx_invoke**(*hpx*::mismatch_t, *SegIter1* first1, *SegIter1* last1, *SegIter2* first2, *Pred* &&pred = *Pred*())

template<typename **ExPolicy**, typename **SegIter1**, typename **SegIter2**, typename **Pred** = *hpx*::*parallel*::detail::equal_to>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *std*::pair<*SegIter1*, *SegIter2*>> **hpx_invoke**(*hpx*::mismatch_t, *ExPolicy* &&policy, *SegIter1* first1, *SegIter1* last1, *SegIter2* first2, *Pred* &&pred = *Pred*())

template<typename **SegIter1**, typename **SegIter2**, typename **Pred** = *hpx*::*parallel*::detail::equal_to>

std::pair<*SegIter1*, *SegIter2*> **hpx_invoke**(*hpx*::mismatch_t, *SegIter1* first1, *SegIter1* last1, *SegIter2* first2, *SegIter2* last2, *Pred* &&pred = *Pred*())

template<typename **ExPolicy**, typename **SegIter1**, typename **SegIter2**, typename **Pred** = *hpx*::*parallel*::detail::equal_to>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *std*::pair<*SegIter1*, *SegIter2*>> **hpx_invoke**(*hpx*::mismatch_t, *ExPolicy* &&policy, *SegIter1* first1, *SegIter1* last1, *SegIter2* first2, *SegIter2* last2, *Pred* &&pred = *Pred*())

hpx/parallel/segmented_algorithms/minmax.hpp

Defined in header hpx/parallel/segmented_algorithms/minmax.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

Typedefs

template<typename **T**>

using **minmax_element_result** = *hpx*::*parallel*::*util*::min_max_result<*T*>

namespace **segmented**

Typedefs

template<typename **T**>

using **minmax_element_result** = *hpx*::*parallel*::*util*::min_max_result<*T*>

Functions

template<typename **SegIter**, typename **F**>

SegIter **hpx_invoke**(*hpx*::min_element_t, *SegIter* first, *SegIter* last, *F* &&f)

template<typename **ExPolicy**, typename **SegIter**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *SegIter*> **hpx_invoke**(*hpx*::min_element_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *F* &&f)

template<typename **SegIter**, typename **F**>

SegIter **hpx_invoke**(*hpx*::max_element_t, *SegIter* first, *SegIter* last, *F* &&f)

template<typename **ExPolicy**, typename **SegIter**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *SegIter*> **hpx_invoke**(*hpx*::max_element_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *F* &&f)

template<typename **SegIter**, typename **F**>

minmax_element_result<*SegIter*> **hpx_invoke**(*hpx*::minmax_element_t, *SegIter* first, *SegIter* last, *F* &&f)

template<typename **ExPolicy**, typename **SegIter**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *minmax_element_result*<*SegIter*>> **hpx_invoke**(*hpx*::minmax_element_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *F* &&f)

hpx/parallel/segmented_algorithms/adjacent_find.hpp

Defined in header hpx/parallel/segmented_algorithms/adjacent_find.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **InIter**, typename **Pred**>

InIter **hpx_invoke**(*hpx*::adjacent_find_t, *InIter* first, *InIter* last, *Pred* &&pred = *Pred*())

template<typename **ExPolicy**, typename **SegIter**, typename **Pred**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *SegIter*> **hpx_invoke**(*hpx*::adjacent_find_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *Pred* &&pred)

hpx/parallel/segmented_algorithms/generate.hpp

Defined in header hpx/parallel/segmented_algorithms/generate.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **SegIter**, typename **F**>

SegIter **hpx_invoke**(*hpx*::generate_t, *SegIter* first, *SegIter* last, *F* &&f)

template<typename **SegIter**, typename **Size**, typename **F**>

SegIter **hpx_invoke**(*hpx*::generate_n_t, *SegIter* first, *Size* count, *F* &&f)

template<typename **ExPolicy**, typename **SegIter**, typename **F**>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *SegIter*> **hpx_invoke**(*hpx*::generate_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *F* &&f)

template<typename **ExPolicy**, typename **SegIter**, typename **Size**, typename **F**>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *SegIter*> **hpx_invoke**(*hpx*::generate_n_t, *ExPolicy* &&policy, *SegIter* first, *Size* count, *F* &&f)

hpx/parallel/segmented_algorithms/transform_inclusive_scan.hpp

Defined in header hpx/parallel/segmented_algorithms/transform_inclusive_scan.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **segmented**

Functions

template<typename **InIter**, typename **OutIter**, typename **Op**, typename **Conv**>

OutIter **hpx_invoke**(*hpx*::transform_inclusive_scan_t, *InIter* first, *InIter* last, *OutIter* dest, *Op* &&op, *Conv* &&conv)

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Op**, typename **Conv**>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> **hpx_invoke**(*hpx*::transform_inclusive_scan_t, *ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *Op* &&op, *Conv* &&conv)

template<typename **InIter**, typename **OutIter**, typename **T**, typename **Op**, typename **Conv**>

OutIter **hpx_invoke**(*hpx*::transform_inclusive_scan_t, *InIter* first, *InIter* last, *OutIter* dest, *Op* &&op, *Conv* &&conv, *T* init)

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **T**, typename **Op**, typename **Conv**>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> **hpx_invoke**(*hpx*::transform_inclusive_scan_t, *ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *Op* &&op, *Conv* &&conv, *T* init)

hpx/parallel/segmented_algorithms/adjacent_difference.hpp

Defined in header hpx/parallel/segmented_algorithms/adjacent_difference.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Op**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> **hpx_invoke**(*hpx*::adjacent_difference_t, *ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *Op* &&op)

template<typename **InIter1**, typename **InIter2**, typename **Op**>

InIter2 **hpx_invoke**(*hpx*::adjacent_difference_t, *InIter1* first, *InIter1* last, *InIter2* dest, *Op* &&op)

hpx/parallel/segmented_algorithms/copy.hpp

Defined in header hpx/parallel/segmented_algorithms/copy.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **SegIter**, typename **OutIter**>

OutIter **hpx_invoke**(*hpx*::copy_t, *SegIter* first, *SegIter* last, *OutIter* dest)

template<typename **ExPolicy**, typename **SegIter**, typename **OutIter**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *OutIter*> **hpx_invoke**(*hpx*::copy_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *OutIter* dest)

hpx/parallel/segmented_algorithms/is_sorted.hpp

Defined in header hpx/parallel/segmented_algorithms/is_sorted.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **InIter**, typename **Pred** = *hpx*::*parallel*::detail::less>

InIter **hpx_invoke**(*hpx*::is_sorted_until_t, *InIter* first, *InIter* last, *Pred* &&pred = *Pred*())

template<typename **ExPolicy**, typename **SegIter**, typename **Pred** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *SegIter*> **hpx_invoke**(*hpx*::is_sorted_until_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *Pred* &&pred = *Pred*())

template<typename **InIter**, typename **Pred** = *hpx*::*parallel*::detail::less>

bool **hpx_invoke**(*hpx*::is_sorted_t, *InIter* first, *InIter* last, *Pred* &&pred = *Pred*())

template<typename **ExPolicy**, typename **SegIter**, typename **Pred** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> **hpx_invoke**(*hpx*::is_sorted_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *Pred* &&pred = *Pred*())

hpx/parallel/segmented_algorithms/transform_exclusive_scan.hpp

Defined in header hpx/parallel/segmented_algorithms/transform_exclusive_scan.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **segmented**

Functions

template<typename **InIter**, typename **OutIter**, typename **T**, typename **Op**, typename **Conv**>

OutIter **hpx_invoke**(*hpx*::transform_exclusive_scan_t, *InIter* first, *InIter* last, *OutIter* dest, *T* init, *Op* &&op, *Conv* &&conv)

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **T**, typename **Op**, typename **Conv**>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> **hpx_invoke**(*hpx*::transform_exclusive_scan_t, *ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *T* init, *Op* &&op, *Conv* &&conv)

hpx/parallel/segmented_algorithms/replace_copy.hpp

Defined in header hpx/parallel/segmented_algorithms/replace_copy.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **SegIter**, typename **OutIter**, typename **T**>

OutIter **hpx_invoke**(*hpx*::replace_copy_t, *SegIter* first, *SegIter* last, *OutIter* dest, *T* const &old_value, *T* const &new_value)

template<typename **ExPolicy**, typename **SegIter**, typename **OutIter**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *OutIter*> **hpx_invoke**(*hpx*::replace_copy_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *OutIter* dest, *T* const &old_value, *T* const &new_value)

template<typename **SegIter**, typename **OutIter**, typename **Pred**, typename **T**>

OutIter **hpx_invoke**(*hpx*::replace_copy_if_t, *SegIter* first, *SegIter* last, *OutIter* dest, *Pred* &&pred, *T* const &new_value)

template<typename **ExPolicy**, typename **SegIter**, typename **OutIter**, typename **Pred**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *OutIter*> **hpx_invoke**(*hpx*::replace_copy_if_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *OutIter* dest, *Pred* &&pred, *T* const &new_value)

hpx/parallel/segmented_algorithms/fill.hpp

Defined in header hpx/parallel/segmented_algorithms/fill.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **SegIter**, typename **T**>

SegIter **hpx_invoke**(*hpx*::fill_t, *SegIter* first, *SegIter* last, *T* const &value)

template<typename **SegIter**, typename **Size**, typename **T**>

SegIter **hpx_invoke**(*hpx*::fill_n_t, *SegIter* first, *Size* count, *T* const &value)

template<typename **ExPolicy**, typename **SegIter**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *SegIter*> **hpx_invoke**(*hpx*::fill_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *T* const &value)

template<typename **ExPolicy**, typename **SegIter**, typename **Size**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *SegIter*> **hpx_invoke**(*hpx*::fill_n_t, *ExPolicy* &&policy, *SegIter* first, *Size* count, *T* const &value)

hpx/parallel/segmented_algorithms/count.hpp

Defined in header hpx/parallel/segmented_algorithms/count.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **InIter**, typename **T**>

std::iterator_traits<*InIter*>::difference_type **hpx_invoke**(*hpx*::count_t, *InIter* first, *InIter* last, *T* const &value)

template<typename **ExPolicy**, typename **SegIter**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, typename *std*::iterator_traits<*SegIter*>::difference_type> **hpx_invoke**(*hpx*::count_t *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *T* const &value)

template<typename **InIter**, typename **F**>

std::iterator_traits<*InIter*>::difference_type **hpx_invoke**(*hpx*::count_if_t, *InIter* first, *InIter* last, *F* &&f)

template<typename **ExPolicy**, typename **SegIter**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, typename *std*::iterator_traits<*SegIter*>::difference_type> **hpx_invoke**(*hpx*::count_i *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *F* &&f)

hpx/parallel/segmented_algorithms/reduce.hpp

Defined in header hpx/parallel/segmented_algorithms/reduce.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **hpx**

namespace **segmented**

Functions

template<typename **InIterB**, typename **InIterE**, typename **T**, typename **F**>

T **hpx_invoke**(*hpx*::reduce_t, *InIterB* first, *InIterE* last, *T* init, *F* &&f)

template<typename **ExPolicy**, typename **InIterB**, typename **InIterE**, typename **T**, typename **F**>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> **hpx_invoke**(*hpx*::reduce_t, *ExPolicy* &&policy, *InIterB* first, *InIterE* last, *T* init, *F* &&f)

hpx/parallel/segmented_algorithms/replace.hpp

Defined in header hpx/parallel/segmented_algorithms/replace.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **SegIter**, typename **T**>

SegIter **hpx_invoke**(*hpx*::replace_t, *SegIter* first, *SegIter* last, *T* const &old_value, *T* const &new_value)

template<typename **ExPolicy**, typename **SegIter**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *SegIter*> **hpx_invoke**(*hpx*::replace_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *T* const &old_value, *T* const &new_value)

template<typename **SegIter**, typename **Pred**, typename **T**>

SegIter **hpx_invoke**(*hpx*::replace_if_t, *SegIter* first, *SegIter* last, *Pred* &&pred, *T* const &new_value)

template<typename **ExPolicy**, typename **SegIter**, typename **Pred**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *SegIter*> **hpx_invoke**(*hpx*::replace_if_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *Pred* &&pred, *T* const &new_value)

hpx/parallel/segmented_algorithms/transform_reduce.hpp

Defined in header hpx/parallel/segmented_algorithms/transform_reduce.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **SegIter**, typename **T**, typename **Reduce**, typename **Convert**>

std::decay_t<*T*> **hpx_invoke**(*hpx*::transform_reduce_t, *SegIter* first, *SegIter* last, *T* &&init, *Reduce* &&red_op, *Convert* &&conv_op)

template<typename **ExPolicy**, typename **SegIter**, typename **T**, typename **Reduce**, typename **Convert**>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *std*::decay_t<*T*>> **hpx_invoke**(*hpx*::transform_reduce_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *T* &&init, *Reduce* &&red_op, *Convert* &&conv_op)

template<typename **FwdIter1**, typename **FwdIter2**, typename **T**, typename **Reduce**, typename **Convert**>

T **hpx_invoke**(*hpx*::transform_reduce_t, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *T* init, *Reduce* &&red_op, *Convert* &&conv_op)

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **T**, typename **Reduce**, typename **Convert**>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> **hpx_invoke**(*hpx*::transform_reduce_t, *ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *T* init, *Reduce* &&red_op, *Convert* &&conv_op)

hpx/parallel/segmented_algorithms/for_each.hpp

Defined in header hpx/parallel/segmented_algorithms/for_each.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **InIter**, typename **F**>

InIter **hpx_invoke**(*hpx*::for_each_t, *InIter* first, *InIter* last, *F* &&f)

template<typename **ExPolicy**, typename **SegIter**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *SegIter*> **hpx_invoke**(*hpx*::for_each_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *F* &&f)

template<typename **InIter**, typename **Size**, typename **F**>

InIter **hpx_invoke**(*hpx*::for_each_n_t, *InIter* first, *Size* count, *F* &&f)

template<typename **ExPolicy**, typename **SegIter**, typename **Size**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *SegIter*> **hpx_invoke**(*hpx*::for_each_n_t, *ExPolicy* &&policy, *SegIter* first, *Size* count, *F* &&f)

hpx/parallel/segmented_algorithms/all_any_none.hpp

Defined in header hpx/parallel/segmented_algorithms/all_any_none.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **InIter**, typename **F**>

bool **hpx_invoke**(*hpx*::none_of_t, *InIter* first, *InIter* last, *F* &&f)

template<typename **ExPolicy**, typename **SegIter**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> **hpx_invoke**(*hpx*::none_of_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *F* &&f)

template<typename **InIter**, typename **F**>

bool **hpx_invoke**(*hpx*::any_of_t, *InIter* first, *InIter* last, *F* &&f)

template<typename **ExPolicy**, typename **SegIter**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> **hpx_invoke**(*hpx*::any_of_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *F* &&f)

template<typename **InIter**, typename **F**>

bool **hpx_invoke**(*hpx*::all_of_t, *InIter* first, *InIter* last, *F* &&f)

template<typename **ExPolicy**, typename **SegIter**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> **hpx_invoke**(*hpx*::all_of_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *F* &&f)

hpx/parallel/segmented_algorithms/transform.hpp

Defined in header hpx/parallel/segmented_algorithms/transform.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **segmented**

Functions

template<typename **SegIter**, typename **OutIter**, typename **F**>

hpx::*parallel*::*util*::in_out_result<*SegIter*, *OutIter*> **hpx_invoke**(*hpx*::transform_t, *SegIter* first, *SegIter* last, *OutIter* dest, *F* &&f)

template<typename **ExPolicy**, typename **SegIter**, typename **OutIter**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *hpx*::*parallel*::*util*::in_out_result<*SegIter*, *OutIter*>> **hpx_invoke**(*hpx*::transform_t, *ExPolicy* &&policy, *SegIter* first, *SegIter* last, *OutIter* dest, *F* &&f)

template<typename **InIter1**, typename **InIter2**, typename **OutIter**, typename **F**>

hpx::*parallel*::*util*::in_in_out_result<*InIter1*, *InIter2*, *OutIter*> **hpx_invoke**(*hpx*::transform_t, *InIter1* first1, *InIter1* last1, *InIter2* first2, *OutIter* dest, *F* &&f)

template<typename **ExPolicy**, typename **InIter1**, typename **InIter2**, typename **OutIter**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *hpx*::*parallel*::*util*::in_in_out_result<*InIter1*, *InIter2*, *OutIter*>> **hpx_invoke**(*hpx*::t *ExPolicy* &&p icy, *InIter* first1, *InIter* last1, *InIter* first2, *OutIter* dest, *F* &&f)

template<typename **InIter1**, typename **InIter2**, typename **OutIter**, typename **F**>

hpx::*parallel*::*util*::in_in_out_result<*InIter1*, *InIter2*, *OutIter*> **hpx_invoke**(*hpx*::transform_t, *InIter1* first1, *InIter1* last1, *InIter2* first2, *InIter2* last2, *OutIter* dest, *F* &&f)

template<typename **ExPolicy**, typename **InIter1**, typename **InIter2**, typename **OutIter**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *hpx*::*parallel*::*util*::in_in_out_result<*InIter1*, *InIter2*, *OutIter*>> **hpx_invoke**(*hpx*::t *ExPolicy* &&policy, *InIter* first1, *InIter* last1, *InIter* first2, *InIter* last2, *OutIter* dest, *F* &&f)

supervision

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::supervision::publish_event

Defined in header hpx/supervision.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::query_state*
- *hpx::supervision::register_observer*
- *hpx::supervision::unregister_observer*
- *hpx::supervision::remove_target*
- *hpx::supervision::register_activity_observer*
- *hpx::supervision::unregister_activity_observer*
- *hpx::supervision::check_admission*
- *hpx::supervision::await_terminal*
- *hpx::supervision::is_valid_transition*

hpx::*future*<publish_result> `hpx::supervision::`**`publish_event`**(*hpx*::id_type const &locality, *hpx*::id_type const &target, *hpx*::*supervision*::event ev, *std*::uint64_t epoch = 0)

Publish a lifecycle event for a target actor on a possibly remote locality.

The function publish_event() asynchronously notifies the supervision manager running on *locality* that *target* has reached the lifecycle state *ev*. Registered observers of *target* are notified as a result.

> **Note**
>
> Publishing is not idempotent: publishing the same event twice creates two distinct records with different timestamps.

> **Note**
>
> Events are immediately visible to observers on the same locality as *target*; remote observers become aware of the event within about 1-2 parcel round-trips to the AGAS authority managing the actor's registration.

Parameters

- **`locality`** – [in] The locality on which the supervision manager responsible for *target* is running.
- **`target`** – [in] The actor (or component) for which the event is published.
- **`ev`** – [in] The lifecycle event to publish for *target*.
- **`epoch`** – [in] The epoch this publication belongs to. Publications are compared against the target's current epoch: publications for a lower epoch are rejected as stale no-ops, publications for a higher epoch reset the target's sequence number and become the new current epoch, and publications for the current epoch are applied normally (subject to terminal-state latching).

Throws

`hpx::exception` – if *locality* does not represent a locality, or if *target* does not represent a valid target. The returned future becomes exceptional if the operation fails.

Returns

A future that becomes ready once the event has been recorded by the supervision manager on *locality*, holding `publish_result::applied`, `publish_result::already_terminal` if *target* had already reached a terminal event (`completed` or `failed`) within *epoch*, or `publish_result::stale_epoch` if *epoch* is lower than the target's current epoch.

publish_result *`hpx::supervision::`***`publish_event`**(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::id_type const &target, *hpx*::*supervision*::event ev, *std*::uint64_t epoch = 0, *hpx*::*error_code* &ec = *hpx*::*throws*)

Publish a lifecycle event for a target actor on a possibly remote locality, blocking until the operation has completed.

This is the synchronous equivalent of *publish_event(hpx::id_type const&, hpx::id_type const&, event)*.

Note

Publishing non-terminal events is not idempotent: publishing the same event twice creates two distinct records with different timestamps. `event::completed` and `event::failed` are latched: the first terminal publication for a target wins, and every later terminal publication for that target is a no-op that returns `publish_result::already_terminal`.

Parameters

- **locality** – [in] The locality on which the supervision manager responsible for *target* is running.
- **target** – [in] The actor (or component) for which the event is published.
- **ev** – [in] The lifecycle event to publish for *target*.
- **epoch** – [in] The epoch this publication belongs to. See the asynchronous remote overload for the epoch-scoped semantics.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

`hpx::exception` – if *locality* does not represent a locality, or if *target* does not represent a valid target, unless *ec* was not pre-initialized to *hpx::throws*. As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*.

Returns

`publish_result::applied`, `publish_result::already_terminal` if *target* had already reached a terminal event (`completed` or `failed`) within *epoch*, or `publish_result::stale_epoch` if *epoch* is lower than the target's current epoch.

publish_result `hpx::supervision::`**`publish_event`**(*hpx*::id_type const &target, *hpx*::*supervision*::event ev, *std*::uint64_t epoch = 0, *hpx*::*error_code* &ec = *throws*)

Publish a lifecycle event for a target actor on the local locality.

This overload publishes *ev* directly through the local supervision manager without going through AGAS or a remote parcel. It is intended to be called from within an actor or action running on the same locality as *target*.

Note

Publishing non-terminal events is not idempotent: publishing the same event twice creates two distinct records with different timestamps. `event::completed` and `event::failed` are latched: the first terminal publication for a target wins, and every later terminal publication for that target is a no-op that returns `publish_result::already_terminal`.

Note

Local observers of *target* are notified synchronously as part of this call.

Parameters

- `target` – [in] The actor (or component) for which the event is published. Must be local to the calling locality.
- `ev` – [in] The lifecycle event to publish for *target*.
- `epoch` – [in] The epoch this publication belongs to. See the asynchronous remote overload for the epoch-scoped semantics.
- `ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

hpx::exception – if *target* does not represent a valid target, unless *ec* was not pre-initialized to *hpx::throws*.

Returns

`publish_result::applied`, `publish_result::already_terminal` if *target* had already reached a terminal event (`completed` or `failed`) within *epoch*, or `publish_result::stale_epoch` if *epoch* is lower than the target's current epoch.

hpx::*future*<publish_result> *hpx*::*supervision*::**publish_event**(*registry* const &handle, event ev, *std*::uint64_t epoch = 0)

Publish a lifecycle event for the local locality tracked by *handle*.

Convenience overload of hpx::supervision::publish_event(hpx::id_type const&, hpx::id_type const&, event, std::uint64_t) that resolves both locality and target to *hpx::find_here()* instead of requiring the caller to name them explicitly. Forwards to *hpx::supervision::publish_event*(`hpx::find_here(), hpx::find_here(), ev, epoch`).

Parameters

- `handle` – [in] The registry (typically returned by *init()*) whose locality the event is published for.
- `ev` – [in] The lifecycle event to publish.
- `epoch` – [in] The epoch this publication belongs to. Typically obtained from a prior `query_state(hpx::launch::sync, handle).epoch` call. See the raw-id overload for the epoch-scoped semantics.

Returns

A future that becomes ready once the event has been recorded, holding the same *publish_result* values as the raw-id overload.

publish_result *hpx*::*supervision*::**publish_event**(*hpx*::*launch*::sync_policy policy, *registry* const &handle, event ev, *std*::uint64_t epoch = 0, *hpx*::*error_code* &ec = *hpx*::*throws*)

Publish a lifecycle event for the local locality tracked by *handle*, blocking until the operation has completed.

This is the synchronous equivalent of publish_event(registry const&, event, std::uint64_t).

Parameters

- **policy** – [in] Tag selecting the blocking overload of *publish_event()*.
- **handle** – [in] The registry whose locality the event is published for.
- **ev** – [in] The lifecycle event to publish.
- **epoch** – [in] The epoch this publication belongs to. See the asynchronous overload.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws
`hpx::exception` – if *handle* does not hold a valid registry client, unless *ec* was pre-initialized to something other than *hpx::throws*.

Returns
The same *publish_result* values as the raw-id overload.

hpx::supervision::query_state

Defined in header hpx/supervision.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::publish_event*
- *hpx::supervision::register_observer*
- *hpx::supervision::unregister_observer*
- *hpx::supervision::remove_target*
- *hpx::supervision::register_activity_observer*
- *hpx::supervision::unregister_activity_observer*
- *hpx::supervision::check_admission*
- *hpx::supervision::await_terminal*
- *hpx::supervision::is_valid_transition*

hpx::*future*<lifecycle_state> `hpx::supervision::`**`query_state`**(*hpx*::id_type const &locality, *hpx*::id_type const &target)

Query the last known lifecycle state of a target actor on a possibly remote locality.

The function query_state() asynchronously retrieves the most recently published lifecycle event for *target*, as observed by the supervision manager running on *locality*.

> **Note**
>
> If *target* is not local to *locality*, the query is forwarded to the actor's home locality, adding about one parcel round-trip of latency.

> **Note**
>
> The returned *lifecycle_state::ec* may carry a staleness error code, indicating that the returned state may be stale if a preceding observer parcel has not yet been delivered.

> **Note**
>
> The *lifecycle_state::event_sequence_number* allows clients to detect gaps, e.g. if an observer callback was skipped due to a network failure.

Parameters

- **locality** – [in] The locality on which the supervision manager responsible for *target* is running.
- **target** – [in] The actor (or component) whose state is queried.

Throws

hpx::exception – if *locality* does not represent a locality, or if *target* does not represent a valid target. The returned future becomes exceptional if the operation fails.

Returns

A future holding the last event recorded for *target* (i.e., the most recent value in the runtime's event log), together with its timestamp and sequence number.

lifecycle_state *hpx::supervision::***query_state**(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::id_type const &target, *hpx*::*error_code* &ec = *hpx*::*throws*)

Query the last known lifecycle state of a target actor on a possibly remote locality, blocking until the result is available.

This is the synchronous equivalent of query_state(hpx::id_type const&, hpx::id_type const&).

> **Note**
>
> The returned *lifecycle_state::ec* may carry a staleness error code, indicating that the returned state may be stale if a preceding observer parcel has not yet been delivered.

Parameters

- **locality** – [in] The locality on which the supervision manager responsible for *target* is running.
- **target** – [in] The actor (or component) whose state is queried.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

`hpx::exception` – if *locality* does not represent a locality, or if *target* does not represent a valid target, unless *ec* was not pre-initialized to *hpx::throws*.

Returns

The last event recorded for *target*, together with its timestamp and sequence number.

lifecycle_state `hpx::supervision::query_state`(*hpx*::id_type const &target, *hpx*::*error_code* &ec = *hpx*::*throws*)

Query the last known lifecycle state of a target actor on the local locality.

Parameters

- `target` – [in] The actor (or component) whose state is queried. Must be local to the calling locality.
- `ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

`hpx::exception` – if *target* does not represent a valid target, unless *ec* was not pre-initialized to *hpx::throws*.

Returns

The last event recorded locally for *target*, together with its timestamp and sequence number.

hpx::*future*<lifecycle_state> `hpx::supervision::query_state`(*registry* const &handle)

Query the last known lifecycle state of the local locality, as tracked by *handle*.

Convenience overload of hpx::supervision::query_state(hpx::id_type const&, hpx::id_type const&) that resolves both the locality and the target to *hpx::find_here()* instead of requiring the caller to name them explicitly. Forwards to `hpx::supervision::query_state(hpx::find_here(), hpx::find_here())`.

Parameters

`handle` – [in] The registry (typically returned by *init()*) whose locality's state is queried.

Returns

A future holding the last event recorded for the local locality, together with its timestamp, sequence number, and epoch (see *lifecycle_state::epoch* — in particular, useful for recovering the epoch init() started this locality's lifecycle tracking at, for use with *publish_event()*).

lifecycle_state `hpx::supervision::query_state`(*hpx*::*launch*::sync_policy policy, *registry* const &handle, *hpx*::*error_code* &ec = *hpx*::*throws*)

Query the last known lifecycle state of the local locality, as tracked by *handle*, blocking until the result is available.

This is the synchronous equivalent of query_state(registry const&).

Parameters

- `policy` – Tag selecting the blocking overload of *query_state()*.
- `handle` – [in] The registry whose locality's state is queried.

- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws
hpx::exception – if *handle* does not hold a valid registry client, unless *ec* was pre-initialized to something other than *hpx::throws*.

Returns
The last event recorded for the local locality.

hpx::*future*<lifecycle_state> *hpx::supervision*::**query_state**(*registry* const &handle, *discovered_peer* const &peer)

Query the last known lifecycle state of a discovered peer's locality.

Convenience overload of hpx::supervision::query_state(hpx::id_type const&, hpx::id_type const&) that resolves both the locality and the target to *peer.locality* instead of requiring the caller to name them explicitly — mirroring the self-query overload above, but issued against *peer.locality's* own supervision manager instead of the local one. Forwards to `hpx::supervision::query_state(peer.locality, peer.locality)`.

Parameters

- **handle** – [in] Unused beyond overload resolution and symmetry with the self-query overload above; *peer* alone carries everything this call needs.
- **peer** – [in] The peer (typically returned by *discover_peers()*/ *discover_and_join()*) whose state is queried.

Returns
A future holding the last event recorded for *peer.locality*, as observed by the supervision manager running on *peer.locality*.

lifecycle_state *hpx::supervision*::**query_state**(*hpx*::*launch*::sync_policy policy, *registry* const &handle, *discovered_peer* const &peer, *hpx*::*error_code* &ec = *hpx*::*throws*)

Query the last known lifecycle state of a discovered peer's locality, blocking until the result is available.

This is the synchronous equivalent of query_state(registry const&, discovered_peer const&).

Parameters

- **policy** – Tag selecting the blocking overload of *query_state()*.
- **handle** – [in] Unused beyond overload resolution; see the asynchronous overload.
- **peer** – [in] The peer whose state is queried.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws
hpx::exception – if *peer.locality* does not hold a valid registry client, unless *ec* was pre-initialized to something other than *hpx::throws*.

Returns
The last event recorded for *peer.locality*.

hpx::supervision::register_observer

Defined in header hpx/supervision.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::publish_event*
- *hpx::supervision::query_state*
- *hpx::supervision::unregister_observer*
- *hpx::supervision::remove_target*
- *hpx::supervision::register_activity_observer*
- *hpx::supervision::unregister_activity_observer*
- *hpx::supervision::check_admission*
- *hpx::supervision::await_terminal*
- *hpx::supervision::is_valid_transition*

hpx::*future*<*hpx*::id_type> `hpx::supervision::register_observer`(*hpx*::id_type const &locality, *hpx*::id_type const &target, lifecycle_callback const &callback, *std*::optional<*std*::uint64_t> epoch_filter = *std*::nullopt)

Register a callback to observe lifecycle events published by a target actor running on a possibly remote locality.

Note

If *locality* is the locality *target* is running on, the callback is invoked synchronously from within the publish call (best effort; the callback must not block indefinitely).

Note

If *locality* differs from the locality *target* is running on, the callback is invoked asynchronously via a dedicated parcel, with retry semantics (e.g., 3 attempts over 500 ms before logging failure).

Note

The callback must not throw; exceptions thrown from it are logged and do not affect the observer registration.

Parameters

- **locality** – [in] The locality on which the callback should be registered.
- **target** – [in] The actor (or component) to observe.

- **callback** – [in] The callback invoked whenever *target* publishes a lifecycle event.
- **epoch_filter** – [in] If set, restricts notifications to events published under this epoch; events published under any other epoch are silently skipped for this observer. If `std::nullopt` (the default), the observer is notified regardless of epoch.

Throws

hpx::exception – if *locality* does not represent a locality, or if *target* does not represent a valid target. The returned future becomes exceptional if the operation fails.

Returns

A future holding the global id of the observer handle. This handle must be passed to unregister_observer() in order to stop receiving notifications.

hpx::id_type **hpx::supervision::register_observer**(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::id_type const &target, lifecycle_callback const &callback, *std*::optional<*std*::uint64_t> epoch_filter = *std*::nullopt, *hpx*::*error_code* &ec = *hpx*::*throws*)

Register a callback to observe lifecycle events published by a target actor running on a possibly remote locality, blocking until the registration has completed.

This is the synchronous equivalent of *register_observer*(hpx::id_type const&, hpx::id_type const&, lifecycle_callback const&).

Parameters

- **locality** – [in] The locality on which the callback should be registered.
- **target** – [in] The actor (or component) to observe.
- **callback** – [in] The callback invoked whenever *target* publishes a lifecycle event.
- **epoch_filter** – [in] If set, restricts notifications to events published under this epoch; see the asynchronous overload for details.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

hpx::exception – if *locality* does not represent a locality, or if *target* does not represent a valid target, unless *ec* was not pre-initialized to *hpx::throws*.

Returns

The global id of the observer handle. This handle must be passed to unregister_observer() in order to stop receiving notifications.

hpx::id_type **hpx::supervision::register_observer**(*hpx*::id_type const &target, lifecycle_callback const &callback, *std*::optional<*std*::uint64_t> epoch_filter = *std*::nullopt, *hpx*::*error_code* &ec = *hpx*::*throws*)

Register a callback to observe lifecycle events published by a target actor on the local locality.

> **Note**
>
> The callback is invoked synchronously from within the publish call (best effort; the callback must not block indefinitely) and must not throw; exceptions thrown from it are

logged and do not affect the observer registration.

Parameters

- **target** – [in] The actor (or component) to observe. Must be local to the calling locality.
- **callback** – [in] The callback invoked whenever *target* publishes a lifecycle event.
- **epoch_filter** – [in] If set, restricts notifications to events published under this epoch; events published under any other epoch are silently skipped for this observer. If `std::nullopt` (the default), the observer is notified regardless of epoch.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

`hpx::exception` – if *target* does not represent a valid target, unless *ec* was initialized to *hpx::throws*.

Returns

The global id of the observer handle. This handle must be passed to unregister_observer() in order to stop receiving notifications.

hpx::supervision::unregister_observer

Defined in header hpx/supervision.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::publish_event*
- *hpx::supervision::query_state*
- *hpx::supervision::register_observer*
- *hpx::supervision::remove_target*
- *hpx::supervision::register_activity_observer*
- *hpx::supervision::unregister_activity_observer*
- *hpx::supervision::check_admission*
- *hpx::supervision::await_terminal*
- *hpx::supervision::is_valid_transition*

hpx::*future*<void> `hpx::supervision::`**`unregister_observer`**(*hpx*::id_type const &locality, *hpx*::id_type const &observer_handle)

Unregister a previously registered observer on a possibly remote locality.

Note

Once the returned future has become ready, no orphaned callbacks fire after unregistration completes.

Parameters

- **locality** – [in] The locality on which the observer was registered.
- **observer_handle** – [in] The handle returned by a prior call to register_observer(). Only handles returned from that function are accepted; passing a handle obtained from register_activity_observer() (or any foreign handle) is rejected.

Throws

`hpx::exception` – if *locality* does not represent a locality, or if *observer_handle* does not represent a valid observer handle. The returned future becomes exceptional if the operation fails.

Returns

A future that becomes ready once the observer has been unregistered.

void `hpx::supervision::unregister_observer`(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::id_type const &observer_handle, *hpx*::*error_code* &ec = *hpx*::*throws*)

Unregister a previously registered observer on a possibly remote locality, blocking until the operation has completed.

This is the synchronous equivalent of unregister_observer(hpx::id_type const&, hpx::id_type const&).

> **Note**
>
> No orphaned callbacks fire after unregistration completes.

Parameters

- **locality** – [in] The locality on which the observer was registered.
- **observer_handle** – [in] The handle returned by a prior call to register_observer(). See the asynchronous overload regarding foreign handles.
- **ec** – [in,out] this represents the error status on exit, if this pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

`hpx::exception` – if *locality* does not represent a locality, or if *observer_handle* does not represent a valid observer handle, unless *ec* was initialized not to *hpx::throws*.

void `hpx::supervision::unregister_observer`(*hpx*::id_type const &observer_handle, *hpx*::*error_code* &ec = *hpx*::*throws*)

Unregister a previously registered observer on the local locality.

> **Note**
>
> No orphaned callbacks fire after unregistration completes.

Parameters

- **observer_handle** – [in] The handle returned by a prior call to register_observer(). Must be local to the calling locality. See the remote overload regarding foreign handles.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

`hpx::exception` – if *observer_handle* does not represent a valid observer handle, unless *ec* was not pre-initialized to *hpx::throws*.

hpx::supervision::remove_target

Defined in header hpx/supervision.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::publish_event*
- *hpx::supervision::query_state*
- *hpx::supervision::register_observer*
- *hpx::supervision::unregister_observer*
- *hpx::supervision::register_activity_observer*
- *hpx::supervision::unregister_activity_observer*
- *hpx::supervision::check_admission*
- *hpx::supervision::await_terminal*
- *hpx::supervision::is_valid_transition*

hpx::*future*<void> `hpx::supervision::`**`remove_target`**(*hpx*::id_type const &locality, *hpx*::id_type const &target)

Clear all locally tracked state for a target on a possibly remote locality.

Unlike unregister_observer(), which removes a single previously registered observer handle, remove_target() unconditionally forgets every piece of local state held for *target* - its recorded lifecycle state (see publish_event() / query_state()) and any observers still registered for it (see register_observer()) - regardless of any specific observer handle.

> **Note**
>
> This is intended for callers that know *target* will never be queried or observed again locally (e.g. after a failed registration that seeded some state for it, or once a peer has been evicted), so that local state does not accumulate indefinitely. It is not itself a lifecycle event and does not affect *target's* state on any other locality.

Parameters

- **locality** – [in] The locality on which *target's* state is tracked.
- **target** – [in] The actor (or component) whose locally tracked state should be removed.

Throws

`hpx::exception` – if *locality* does not represent a locality, or if *target* does not represent a valid target. The returned future becomes exceptional if the operation fails.

Returns

A future that becomes ready once *target's* state has been removed.

void `hpx::supervision::remove_target`(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::id_type const &target, *hpx*::*error_code* &ec = *hpx*::*throws*)

Clear all locally tracked state for a target on a possibly remote locality, blocking until the operation has completed.

This is the synchronous equivalent of remove_target(hpx::id_type const&, hpx::id_type const&).

Parameters

- `locality` – [in] The locality on which *target's* state is tracked.
- `target` – [in] The actor (or component) whose locally tracked state should be removed.
- `ec` – [in,out] this represents the error status on exit, if this pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

`hpx::exception` – if *locality* does not represent a locality, or if *target* does not represent a valid target, unless *ec* was initialized not to *hpx::throws*.

void `hpx::supervision::remove_target`(*hpx*::id_type const &target, *hpx*::*error_code* &ec = *hpx*::*throws*)

Clear all locally tracked state for a target on the local locality.

Parameters

- `target` – [in] The actor (or component) whose locally tracked state should be removed. Must be local to the calling locality.
- `ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

`hpx::exception` – if *target* does not represent a valid target, unless *ec* was not pre-initialized to *hpx::throws*.

hpx::supervision::register_activity_observer

Defined in header hpx/supervision.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::publish_event*
- *hpx::supervision::query_state*
- *hpx::supervision::register_observer*

- *hpx::supervision::unregister_observer*
- *hpx::supervision::remove_target*
- *hpx::supervision::unregister_activity_observer*
- *hpx::supervision::check_admission*
- *hpx::supervision::await_terminal*
- *hpx::supervision::is_valid_transition*

hpx::*future*<*hpx*::id_type> `hpx::supervision::register_activity_observer`(*hpx*::id_type const &locality, activity_callback const &callback, *std*::optional<*std*::uint64_t> epoch_filter = *std*::nullopt)

Register a callback to observe activity-state transitions of all targets tracked on a possibly remote locality.

> **Note**
>
> For every target tracked on *locality* that is already `activity_state::active` at registration time, the new observer synchronously receives one replay notification (carrying `activity_transition::already_active`) as part of registration, taken under the same lock that serializes activity-state transitions. This guarantees the observer sees exactly one notification for each such target's current activity state: either the replay, or a live transition that raced with registration, but never both and never neither.

> **Note**
>
> If *locality* is the local locality, callbacks (including the registration-time replay) are invoked synchronously from within the call that triggered the transition (best effort; the callback must not block indefinitely).

> **Note**
>
> If *locality* is remote, callbacks are invoked asynchronously via a dedicated parcel, with retry semantics (e.g., 3 attempts over 500 ms before logging failure).

> **Note**
>
> The callback must not throw; exceptions thrown from it are logged and do not affect the observer registration.

Parameters

- **`locality`** – [in] The locality on which the callback should be registered.
- **`callback`** – [in] The callback invoked whenever any target tracked on *locality* transitions between *activity_state::inactive* and *activity_state::active*.

- **epoch_filter** – [in] If set, restricts notifications to transitions recorded under this epoch; transitions recorded under any other epoch are silently skipped for this observer. This also applies to the registration-time replay described below. If `std::nullopt` (the default), the observer is notified regardless of epoch.

Throws

hpx::exception – if *locality* does not represent a locality. The returned future becomes exceptional if the operation fails.

Returns

A future holding the global id of the observer handle. This handle must be passed to unregister_activity_observer() in order to stop receiving notifications.

hpx::id_type *hpx*::*supervision*::**register_activity_observer**(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, activity_callback const &callback, *std*::optional<*std*::uint64_t> epoch_filter = *std*::nullopt, *hpx*::*error_code* &ec = *hpx*::*throws*)

Register a callback to observe activity-state transitions of all targets tracked on a possibly remote locality, blocking until the registration has completed.

This is the synchronous equivalent of *register_activity_observer*(hpx::id_type const&, activity_callback const&, std::optional<std::uint64_t>).

Parameters

- **locality** – [in] The locality on which the callback should be registered.
- **callback** – [in] The callback invoked whenever any target tracked on *locality* transitions between *activity_state::inactive* and *activity_state::active*.
- **epoch_filter** – [in] If set, restricts notifications to transitions recorded under this epoch; see the asynchronous overload for details, including how this applies to the registration-time replay.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

hpx::exception – if *locality* does not represent a locality, unless *ec* was not pre-initialized to *hpx::throws*.

Returns

The global id of the observer handle. This handle must be passed to unregister_activity_observer() in order to stop receiving notifications.

hpx::id_type *hpx*::*supervision*::**register_activity_observer**(activity_callback const &callback, *std*::optional<*std*::uint64_t> epoch_filter = *std*::nullopt, *hpx*::*error_code* &ec = *hpx*::*throws*)

Register a callback to observe activity-state transitions of all targets tracked on the local locality.

Note

> As for the remote overloads, for every target on the local locality that is already `activity_state::active` at registration time, one `activity_transition::` already_active replay notification is delivered synchronously as part of this call, taken under the same lock that serializes activity-state transitions, so the observer can neither miss nor double-receive any active target's current activity state.

Parameters

- **`callback`** – [in] The callback invoked whenever any target tracked on the local locality transitions between *activity_state::inactive* and *activity_state::active*.
- **`epoch_filter`** – [in] If set, restricts notifications to transitions recorded under this epoch, including the registration-time replay; see the remote overload for details. If `std::nullopt` (the default), the observer is notified regardless of epoch.
- **`ec`** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

`hpx::exception` – if the operation fails, unless *ec* was initialized to *hpx::throws*.

Returns

The global id of the observer handle. This handle must be passed to unregister_activity_observer() in order to stop receiving notifications.

hpx::supervision::unregister_activity_observer

Defined in header hpx/supervision.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::publish_event*
- *hpx::supervision::query_state*
- *hpx::supervision::register_observer*
- *hpx::supervision::unregister_observer*
- *hpx::supervision::remove_target*
- *hpx::supervision::register_activity_observer*
- *hpx::supervision::check_admission*
- *hpx::supervision::await_terminal*
- *hpx::supervision::is_valid_transition*

hpx::*future*<void> `hpx::supervision::unregister_activity_observer`(*hpx*::id_type const &locality, *hpx*::id_type const &observer_handle)

Unregister a previously registered activity observer on a possibly remote locality.

> **Note**
>
> Once the returned future has become ready, no orphaned callbacks fire after unregistration completes.

Parameters

- **locality** – [in] The locality on which the observer was registered.
- **observer_handle** – [in] The handle returned by a prior call to register_activity_observer(). Only handles returned from that function are accepted; passing a handle obtained from register_observer() (or any foreign handle) is rejected.

Throws

`hpx::exception` – if *locality* does not represent a locality, or if *observer_handle* does not represent a valid activity-observer handle. The returned future becomes exceptional if the operation fails.

Returns

A future that becomes ready once the observer has been unregistered.

void `hpx::supervision::unregister_activity_observer`(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::id_type const &observer_handle, *hpx*::*error_code* &ec = *hpx*::*throws*)

Unregister a previously registered activity observer on a possibly remote locality, blocking until the operation has completed.

This is the synchronous equivalent of *unregister_activity_observer*(hpx::id_type const&, hpx::id_type const&).

> **Note**
>
> No orphaned callbacks fire after unregistration completes.

Parameters

- **locality** – [in] The locality on which the observer was registered.
- **observer_handle** – [in] The handle returned by a prior call to register_activity_observer(). See the asynchronous overload regarding foreign handles.
- **ec** – [in,out] this represents the error status on exit, if this pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

`hpx::exception` – if *locality* does not represent a locality, or if *observer_handle* does not represent a valid activity-observer handle, unless *ec* was initialized not to *hpx::throws*.

void `hpx::supervision::unregister_activity_observer`(*hpx*::id_type const &observer_handle, *hpx*::*error_code* &ec = *hpx*::*throws*)

Unregister a previously registered activity observer on the local locality.

> **Note**
>
> No orphaned callbacks fire after unregistration completes.

Parameters

- **observer_handle** – [in] The handle returned by a prior call to register_activity_observer(). Must be local to the calling locality. See the remote overload regarding foreign handles.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

`hpx::exception` – if *observer_handle* does not represent a valid activity-observer handle, unless *ec* was not pre-initialized to *hpx::throws*.

hpx::supervision::check_admission

Defined in header hpx/supervision.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::publish_event*
- *hpx::supervision::query_state*
- *hpx::supervision::register_observer*
- *hpx::supervision::unregister_observer*
- *hpx::supervision::remove_target*
- *hpx::supervision::register_activity_observer*
- *hpx::supervision::unregister_activity_observer*
- *hpx::supervision::await_terminal*
- *hpx::supervision::is_valid_transition*

dispatch_outcome `hpx::supervision::`**`check_admission`**(*hpx*::id_type const &target, *std*::uint64_t epoch = 0)

Check whether *target* currently admits new dispatch under *epoch*.

Queries the local supervision manager's terminal-latch state for *target* within *epoch*, i.e. the same state maintained by publish_event() and consulted by await_terminal(). Does not publish, mutate state, or notify observers.

> **Note**
>
> This is a pure local read of already-resident latch state, not a query that can fail the way a remote publish_event() call can; it never throws and carries no hpx::error_code out-parameter.

Note

The terminal latch consulted here is populated only by publish_event() calls delivered to *this* locality's supervision manager for *target*. It is not synchronized across localities: a terminal event published to a different locality's manager for the same logical target/epoch is invisible here and does not affect this call's outcome. `register_observer` / `unregister_observer` notifications do not feed this state either. Any locality other than the one hosting *target's* supervision manager that needs to gate its own dispatch decisions on *target's* terminal state must build that cross-locality propagation itself (e.g. registering an observer with the owning locality and re-publishing locally, or routing the admission decision to the owning locality); this API provides no such mechanism.

Parameters

- **`target`** – [in] The actor (or component) to check. Must be local to the calling locality.
- **`epoch`** – [in] The epoch dispatch would occur under.

Returns

`dispatch_outcome::rejected_fenced` if *target* has already latched a terminal event (`completed` or `failed`) under *epoch*, `dispatch_outcome::admitted` otherwise (including when *target* is unknown locally or invalid).

hpx::supervision::await_terminal

Defined in header hpx/supervision.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::publish_event*
- *hpx::supervision::query_state*
- *hpx::supervision::register_observer*
- *hpx::supervision::unregister_observer*
- *hpx::supervision::remove_target*
- *hpx::supervision::register_activity_observer*
- *hpx::supervision::unregister_activity_observer*
- *hpx::supervision::check_admission*
- *hpx::supervision::is_valid_transition*

```
hpx::future<lifecycle_state> hpx::supervision::await_terminal(hpx::id_type const &locality, hpx::id_type const &target, std::uint64_t epoch = 0, std::optional<std::chrono::steady_clock::duration> timeout = std::nullopt)
```

Asynchronously wait for a target actor on a possibly remote locality to reach a terminal lifecycle event (`completed` or `failed`) within a given epoch.

The function await_terminal() is a one-shot, blocking-free counterpart to register_observer() for the specific case of waiting on the terminal transition: unlike an observer, it does not need to be explicitly unregistered, but it also only ever resolves once, for the exact (*target*, *epoch*) pair passed in.

> **Note**
>
> below for this call only. If `std::nullopt` (the default), *locality's* current default timeout is used instead.

> **Note**
>
> below).

> **Note**
>
> *locality* bounds the lifetime of every waiter it registers on behalf of an outstanding call, so dropping the returned future without *target* ever reaching a terminal event under *epoch* does not leak the corresponding waiter entry indefinitely: it is swept and invalidated at or after *timeout* elapses, enforced by a background timer on *locality* independently of any further activity for *target*, so an idle target's waiter is not left to accumulate until some later, unrelated call happens to sweep it. Callers that need a tighter bound than *timeout* should still race the returned future against an external timeout.

Parameters

- **`locality`** – [in] The locality on which the supervision manager responsible for *target* is running.
- **`target`** – [in] The actor (or component) to wait on.
- **`epoch`** – [in] The epoch *target* must reach a terminal event under for the returned future to resolve. If *target's* epoch is later superseded by a higher epoch (see the epoch semantics of *publish_event*) before it reaches a terminal event under *epoch*, the returned future becomes exceptional (see
- **`timeout`** – [in] Overrides the server-enforced timeout described in

Throws

- `below)` – instead of ever resolving with a value.
- *`hpx::exception`* – if *locality* does not represent a locality, or if *target* does not represent a valid target. The returned future becomes exceptional with a *hpx::error::stale_state* exception, naming *target*, *epoch*, and the epoch that superseded it, if *target's* epoch advances past *epoch* before reaching a terminal event under it. It also becomes exceptional with a *hpx::error::future_cancelled* exception if neither of the above happens before *timeout* elapses (see

Returns

A future that becomes ready, holding the *lifecycle_state* recorded for *target*, as soon as *target* reaches a terminal event under *epoch*. If *target* has already reached a terminal event under *epoch* at the time of the call, the returned future is ready immediately.

lifecycle_state `hpx::supervision::await_terminal`(*hpx*::*launch*::sync_policy, *hpx*::id_type const &locality, *hpx*::id_type const &target, *std*::uint64_t epoch = 0, *std*::optional<*std*::chrono::steady_clock::duration> timeout = *std*::nullopt, *hpx*::*error_code* &ec = *hpx*::*throws*)

Wait for a target actor on a possibly remote locality to reach a terminal lifecycle event, blocking until it does.

This is the synchronous equivalent of *await_terminal*(hpx::id_type const&, hpx::id_type const&, std::uint64_t).

> **Note**
>
> This call blocks until *target* reaches a terminal event, its epoch is superseded, or *timeout* elapses.

Parameters

- **`locality`** – [in] The locality on which the supervision manager responsible for *target* is running.
- **`target`** – [in] The actor (or component) to wait on.
- **`epoch`** – [in] The epoch *target* must reach a terminal event under. See the asynchronous overload for details.
- **`timeout`** – [in] Overrides the server-enforced timeout described in the asynchronous overload for this call only. If `std::nullopt` (the default), *locality's* current default timeout is used instead.
- **`ec`** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

`hpx::exception` – if *locality* does not represent a locality, or if *target* does not represent a valid target, unless *ec* was not pre-initialized to *hpx::throws*. Also throws (or sets *ec* to) a *hpx::error::stale_state* exception, naming *target*, *epoch*, and the epoch that superseded it, if *target's* epoch advances past *epoch* before reaching a terminal event under it. Also throws (or sets *ec* to) a *hpx::error::future_cancelled* exception if neither of the above happens before *timeout* elapses.

Returns

The *lifecycle_state* recorded for *target* once it reaches a terminal event under *epoch*.

hpx::*future*<lifecycle_state> `hpx::supervision::await_terminal`(*hpx*::id_type const &target, *std*::uint64_t epoch = 0, *std*::optional<*std*::chrono::steady_clock::duration> timeout = *std*::nullopt)

Asynchronously wait for a target actor on the local locality to reach a terminal lifecycle event within a given epoch.

> **Note**
>
> As for the remote overloads, the returned future becomes exceptional with *hpx::error::stale_state* if *epoch* is superseded, or with *hpx::error::future_cancelled* if *timeout* elapses first.

Parameters

- **`target`** – [in] The actor (or component) to wait on. Must be local to the calling locality.
- **`epoch`** – [in] The epoch *target* must reach a terminal event under. See the remote overload for details.
- **`timeout`** – [in] Overrides the server-enforced timeout described in the asynchronous overload for this call only. If `std::nullopt` (the default), the local locality's default timeout is used instead.

Returns

A future that becomes ready, holding the *lifecycle_state* recorded for *target*, as soon as *target* reaches a terminal event under *epoch*. See the remote overload for how the returned future behaves if *target's* epoch is superseded before reaching a terminal event under *epoch*.

lifecycle_state *hpx*::*supervision*::**await_terminal**(*hpx*::*launch*::sync_policy, *hpx*::id_type const &target, *std*::uint64_t epoch = 0, *std*::optional<*std*::chrono::steady_clock::duration> timeout = *std*::nullopt, *hpx*::*error_code* &ec = *hpx*::*throws*)

Synchronously wait for a local supervised target to reach a terminal lifecycle state (`completed` or `failed`).

This overload is restricted to targets that are guaranteed to be local to the calling locality. Because no dispatch/AGAS round-trip is required, it is cheaper than the remote `hpx::launch::sync_policy` overload and simply blocks the calling thread on the underlying local future.

Parameters

- **`target`** – The id of the supervised entity to wait on. Must refer to a component that is local to this locality.
- **`epoch`** – The join/generation epoch the caller expects `target` to still be part of. Passing an epoch that no longer matches the target's current epoch may cause the wait to resolve immediately with a mismatch-related error.
- **`timeout`** – Optional upper bound on how long to wait before giving up. If not supplied, uses the local supervision manager's default timeout.
- **`ec`** – Used to report errors instead of throwing an exception. When `target` does not represent a valid id, `hpx::error::bad_parameter` is reported. If the wait is cancelled due to `timeout` elapsing without a terminal state being reached, `hpx::error::future_cancelled` is reported.

Throws

hpx::exception – if `ec` is `hpx::throws` (the default) and an error is encountered; otherwise sets `ec` to reflect the error and returns.

Returns

The terminal `lifecycle_state` observed for `target`, or a default-constructed `lifecycle_state` if an error occurred and `ec` was used to report it.

hpx::supervision::is_valid_transition

Defined in header hpx/supervision.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::publish_event*
- *hpx::supervision::query_state*
- *hpx::supervision::register_observer*
- *hpx::supervision::unregister_observer*
- *hpx::supervision::remove_target*
- *hpx::supervision::register_activity_observer*
- *hpx::supervision::unregister_activity_observer*
- *hpx::supervision::check_admission*
- *hpx::supervision::await_terminal*

bool *hpx*::*supervision*::**is_valid_transition**(event from, event to) noexcept

Determine whether transitioning from lifecycle event *from* to lifecycle event *to* is permitted by the supervision lifecycle state machine.

event::completed and *event::failed* are terminal states and have no outgoing transitions. *event::losing_locality* is only reachable from *event::started*, *event::running*, or *event::suspending*, and may only transition to *failed*. A missing prior event is represented as *event::unknown*, from which the only valid transition is to *event::started*.

hpx::supervision::testing::set_register_observer_snapshot_hook

Defined in header hpx/supervision.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

void *hpx*::*supervision*::testing::**set_register_observer_snapshot_hook**(*std*::function<void()> hook)

Set the hook invoked after an observer snapshot is captured.

> **Note**
>
> Used for testing purposes only.

Parameters

hook – Hook to invoke. An empty function clears the hook.

hpx::supervision::supervision_manager

Defined in header hpx/supervision.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

class **supervision_manager**

hpx/supervision/server/supervision_manager.hpp

Defined in header hpx/supervision/server/supervision_manager.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **supervision**

namespace **server**

Variables

constexpr char const *const **supervision_manager_name** = "supervision_manager/"

struct **observer_entry**

Public Members

hpx::id_type **agent**

std::uint64_t **epoch_filter**

struct **waiter_key**

Public Members

hpx::id_type **target**

std::uint64_t **epoch**

Friends

inline friend bool **operator<**(*waiter_key* const &lhs, *waiter_key* const &rhs)

inline friend bool **operator==**(*waiter_key* const &lhs, *waiter_key* const &rhs)

struct **waiter_entry**

Public Members

hpx::*promise*<lifecycle_state> **promise**

std::chrono::steady_clock::time_point **deadline**

struct **supervision_manager** : public *hpx*::*components*::*fixed_component_base*<*supervision_manager*>

Public Types

using **base_type** = *components*::*fixed_component_base*<*supervision_manager*>

Public Functions

supervision_manager()

~supervision_manager()

void **register_server_instance**(char const *service_name, *std*::uint32_t locality_id, *error_code* &ec = *throws*)

void **unregister_server_instance**(*error_code* &ec = *throws*) const

publish_result **publish_event**(*hpx*::id_type const &target, event ev, *std*::uint64_t epoch)

Applies epoch/terminal-latch state-mutation rules, resolves any await_terminal() waiters affected by the transition, and invokes registered per-target lifecycle observers and activity observers for this call.

Parameters

- **target** – Target whose supervision state is updated. Must live on this locality.
- **ev** – Lifecycle event to apply.
- **epoch** – Epoch associated with `ev`.

Returns

The result of applying `ev` to target's state.

lifecycle_state **query_state**(*hpx*::id_type const &target)

hpx::id_type **register_observer**(*hpx*::id_type const &target, *hpx*::id_type const &agent, *std*::uint64_t epoch_filter = static_cast<*std*::uint64_t>(-1))

void **unregister_observer**(*hpx*::id_type const &observer_handle)

void **remove_target**(*hpx*::id_type const &target)

Clears all locally tracked state for `target`.

Unlike unregister_observer(), which removes a single previously registered observer handle (and leaves any recorded lifecycle state for its target(s) intact), *remove_target()* unconditionally forgets every piece of local bookkeeping this supervision manager holds for `target` - its recorded lifecycle state and current epoch (see *publish_event()*), and any per-target observers still registered for it (see register_observer()) - regardless of any specific observer handle. Intended for callers that know `target` will never be queried or observed again locally (e.g. after a failed registration that seeded some state for it, or once a peer has been evicted) and want to reclaim that local state instead of letting it accumulate indefinitely.

void **tidy**()

Unconditionally clears all locally tracked state.

Snapshots every target currently present in states_ or observers_ under mtx_, releases the lock, then calls *remove_target()* for each one in turn, reusing its existing per-target teardown (waiter invalidation, activity-observer notifications, agent/state cleanup) instead of duplicating that logic here. Targets added concurrently with (or after) the snapshot are left untouched. If states_ and observers_ are empty at snapshot time, this is a no-op: *tidy()* never reports an error and never fires any notification of its own for the "nothing to do" case. Local-only: unlike *remove_target()*, this is not exposed as a remote action.

hpx::id_type **register_activity_observer**(*hpx*::id_type const &agent, *std*::uint64_t epoch_filter = static_cast<*std*::uint64_t>(-1))

Registers a locality-scoped activity observer.

Replays targets that are already active when registration takes its snapshot.

The snapshot of the target's tracked state and the insertion of the observer into the tracked set happen atomically under `mtx_`, which guarantees the observer receives exactly one notification for the target's current state: either the replay (if already active at registration time) or a live transition that raced with registration, but never both and never neither.

Delivery of that notification (replay or live) happens after `mtx_` is released and is therefore not ordered relative to any other concurrent live notification for the same target; callers must not assume replay is delivered before or after a racing live event.

void **unregister_activity_observer**(*hpx*::id_type const &observer_handle)

Unregisters an activity observer.

The handle must have been returned by *register_activity_observer()*. As with unregister_observer(), no orphaned callbacks fire after this call completes. `observer_handle` must have been returned by *register_activity_observer()*; a handle returned by register_observer() instead (i.e. found in agents_ rather than activity_observers_) is rejected.

hpx::*future*<lifecycle_state> **await_terminal**(*hpx*::id_type const &target, *std*::uint64_t epoch = 0, *std*::chrono::steady_clock::duration timeout = (*std*::chrono::steady_clock::duration::max)())

dispatch_outcome **check_admission**(*hpx*::id_type const &target, *std*::uint64_t epoch = 0) const noexcept

Protected Types

using **waiters_t** = *std*::vector<*waiter_entry*>

using **stale_waiters_t** = *std*::vector<*std*::pair<*std*::uint64_t, *waiters_t*>>

using **expired_waiters_t** = *std*::vector<*expired_waiter*>

Protected Functions

hpx::*future*<void> **fire_events**(*hpx*::id_type const &target, lifecycle_event_notification const &notification)

hpx::*future*<void> **fire_event**(*hpx*::id_type const &target, *hpx*::id_type const &agent, lifecycle_event_notification notification)

hpx::*future*<void> **deliver_activity_notification**(activity_notification const &notification, *std*::vector<*observer_entry*> const &observers)

hpx::*future*<void> **fire_activity_event**(*hpx*::id_type const &agent, activity_notification notification)

void **record_error**(*hpx*::id_type const &target, *std*::uint64_t expected_sequence_number, *hpx*::*error_code* const &ec)

bool **unregister_observer_target**(*hpx*::id_type const &target, *hpx*::id_type const &observer_handle)

std::uint64_t **current_epoch_for**(*hpx*::id_type const &target) const

apply_event_outcome **apply_event_and_resolve**(*hpx*::id_type const &target, event ev, *std*::uint64_t epoch)

waiters_t **drain_terminal_waiters_locked**(*std*::unique_lock<*hpx*::*spinlock*> &l, *hpx*::id_type const &target, *std*::uint64_t epoch, event last_event)

stale_waiters_t **drain_stale_waiters_locked**(*std*::unique_lock<*hpx*::*spinlock*> &l, *hpx*::id_type const &target, *std*::uint64_t epoch) *apply_result* **apply_new_epoch_locked**(*std*::unique_lock<*hpx*::*spinlock*> &l, *hpx*::id_type const &target, *supervision*::event ev, *std*::uint64_t epoch)

std::optional<*apply_result*> **apply_current_epoch_locked**(*std*::unique_lock<*hpx*::*spinlock*> &l, *hpx*::id_type const &target, *supervision*::event ev, *std*::uint64_t epoch)

expired_waiters_t **drain_expired_waiters_locked**(*std*::unique_lock<*hpx*::*spinlock*> &l, *std*::chrono::steady_clock::time_point now)

stale_waiters_t **drain_all_waiters_for_target_locked**(*std*::unique_lock<*hpx*::*spinlock*> &l, *hpx*::id_type const &target)

void **sweep_expired_waiters**()

void **arm_sweep_timer_locked**(*std*::unique_lock<*hpx*::*spinlock*> &&l)

bool **sweep_timer_callback**()

void **recompute_earliest_deadline_locked**(*std*::unique_lock<*hpx*::*spinlock*> &l)

void **remove_target_from_agents_locked**(*std*::unique_lock<*hpx*::*spinlock*> &l, *hpx*::id_type const &agent, *hpx*::id_type const &target)

Protected Static Functions

static void **invalidate_stale_waiters**(*hpx*::id_type const &target, *std*::uint64_t epoch, *stale_waiters_t* &stale)

static void **resolve_terminal_waiters**(lifecycle_event_notification const &notification, *waiters_t* &to_resolve)

static void **invalidate_expired_waiters**(*expired_waiters_t* &expired)

Private Members

mutable *hpx*::*spinlock* **mtx_**

hpx::*spinlock* **sweep_timer_mtx_**

mutable *std*::string **instance_name_**

std::map<*hpx*::id_type, lifecycle_state> **states_**

std::map<*hpx*::id_type, *std*::vector<*observer_entry*>> **observers_**

std::map<*hpx*::id_type, *std*::vector<*hpx*::id_type>> **agents_**

std::vector<*observer_entry*> **activity_observers_**

std::map<*hpx*::id_type, *std*::chrono::steady_clock::time_point> **activated_at_**

std::map<*waiter_key*, *waiters_t*> **waiters_**

std::chrono::steady_clock::time_point **earliest_deadline_** = (*std*::chrono::steady_clock::time_point::max)()

hpx::*util*::pool_timer **sweep_timer_**

std::chrono::steady_clock::time_point **armed_deadline_** = (*std*::chrono::steady_clock::time_point::max)()

bool **shutting_down_** = false

std::size_t **callbacks_in_flight_** = 0

hpx::*lcos*::*local*::detail::condition_variable **shutdown_cv_**

struct **apply_event_outcome**

Public Members

publish_result **result**

lifecycle_event_notification **notification**

bool **had_state_before** = false

std::vector<*observer_entry*> **activity_observers_snapshot**

struct **apply_result**

Public Members

lifecycle_event_notification **notification**

waiters_t **to_resolve**

struct **expired_waiter**

Public Members

hpx::id_type **target**

std::uint64_t **epoch**

hpx::*promise*<lifecycle_state> **promise**

algorithms

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::experimental::run_on_all

Defined in header hpx/task_block.hpp[598].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Warning

doxygenfunction: Unable to resolve function "hpx::experimental::run_on_all" with arguments (ExPolicy&&, T&&, Ts&&…) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename ExPolicy, typename T, typename ...Ts>
↪ decltype(auto) run_on_all(ExPolicy &&policy, T &&t, Ts&&... ts)
- template<typename␣
↪T, typename ...Ts> decltype(auto) run_on_all(T &&t, Ts&&... ts)
```

Warning

doxygenfunction: Unable to resolve function "hpx::experimental::run_on_all" with arguments (T&&, Ts&&…) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename ExPolicy, typename T, typename ...Ts>
↪ decltype(auto) run_on_all(ExPolicy &&policy, T &&t, Ts&&... ts)
- template<typename␣
↪T, typename ...Ts> decltype(auto) run_on_all(T &&t, Ts&&... ts)
```

[598] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/task_block.hpp

hpx::experimental::task_group

Defined in header hpx/experimental/task_group.hpp[599].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

class **task_group**

A *task_group* represents concurrent execution of a group of tasks. Tasks can be dynamically added to the group while it is executing.

hpx::experimental::task_canceled_exception

Defined in header hpx/task_block.hpp[600].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::experimental::task_block*
- *hpx::experimental::define_task_block*
- *hpx::experimental::define_task_block_restore_thread*

class **task_canceled_exception** : public *hpx::exception*

The class task_canceled_exception defines the type of objects thrown by task_block::run or task_block::wait if they detect that an exception is pending within the current parallel region.

hpx::experimental::task_block

Defined in header hpx/task_block.hpp[601].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::experimental::task_canceled_exception*
- *hpx::experimental::define_task_block*
- *hpx::experimental::define_task_block_restore_thread*

template<typename **ExPolicy** = *hpx::execution::parallel_policy*>

class **task_block**

The class *task_block* defines an interface for forking and joining parallel tasks. The *define_task_block* and *define_task_block_restore_thread* function templates create an

[599] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/experimental/task_group.hpp

[600] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/task_block.hpp

[601] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/task_block.hpp

object of type *task_block* and pass a reference to that object to a user-provided callable object.

An object of class task_block cannot be constructed, destroyed, copied, or moved except by the implementation of the task region library. Taking the address of a *task_block* object via operator& or addressof is ill formed. The result of obtaining its address by any other means is unspecified.

A task_block is active if it was created by the nearest enclosing task block, where "task block" refers to an invocation of define_task_block or define_task_block_restore_thread and "nearest

enclosing" means the most recent invocation that has not yet completed. Code designated for execution in another thread by means other than the facilities in this section (e.g., using thread or async) are not enclosed in the task region and a *task_block* passed to (or captured by) such code is not active within that code. Performing any operation on a *task_block* that is not active results in undefined behavior.

The task_block that is active before a specific call to the run member function is not active within the asynchronous function that invoked run. (The invoked function should not, therefore, capture the task_block from the surrounding block.)

```
Example:
   define_task_block([&](auto& tr) {
       tr.run([&] {
      ␣
↪    tr.run([] { f(); });              // Error: tr is not active
           define_task_block([&](auto& tr) { // Nested task block
     ␣
↪          tr.run(f);                   // OK: inner tr is active
               /// ...
           });
       }); /// ...
   });
```

Template Parameters

ExPolicy – The execution policy an instance of a task_block was created with. This defaults to *parallel_policy*.

hpx::experimental::define_task_block

Defined in header hpx/task_block.hpp[602].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::experimental::task_canceled_exception*
- *hpx::experimental::task_block*
- *hpx::experimental::define_task_block_restore_thread*

[602] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/task_block.hpp

Warning

doxygenfunction: Unable to resolve function “hpx::experimental::define_task_block” with arguments (ExPolicy&&, F&&) in doxygen xml output for project “hpx” from directory: ../hpx_autodoc. Potential matches:

```
- template<typename ExPolicy, typename␣
↪F> decltype(auto) define_task_block(ExPolicy &&policy, F &&f)
- template<typename F> void define_task_block(F &&f)
```

template<typename **F**>

void *hpx*::*experimental*::**define_task_block**(*F* &&f)

Constructs a task_block, tr, and invokes the expression *f(tr)* on the user-provided object, *f*. This version uses *parallel_policy* for task scheduling.

Postcondition: All tasks spawned from *f* have finished execution. A call to define_task_block may return on a different thread than that on which it was called.

Note

It is expected (but not mandated) that f will (directly or indirectly) call tr.run(*callable_object*).

Template Parameters

F – The type of the user defined function to invoke inside the define_task_block (deduced). *F* shall be MoveConstructible.

Parameters

f – The user defined function to invoke inside the task block. Given a lvalue *tr* of type task_block, the expression, (void)f(tr), shall be well-formed.

Throws

exception_list – specified in Exception Handling.

hpx::experimental::define_task_block_restore_thread

Defined in header hpx/task_block.hpp[603].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::experimental::task_canceled_exception*
- *hpx::experimental::task_block*
- *hpx::experimental::define_task_block*

template<typename **ExPolicy**, typename **F**>

[603] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/task_block.hpp

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*> `hpx::experimental::define_task_block_restore_thread`(*ExPolicy* &&policy, *F* &&f)

Constructs a task_block, tr, and invokes the expression *f(tr)* on the user-provided object, *f*.

Postcondition: All tasks spawned from *f* have finished execution. A call to *define_task_block_restore_thread* always returns on the same thread as that on which it was called.

> **Note**
>
> It is expected (but not mandated) that f will (directly or indirectly) call tr.run(*callable_object*).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the task block may be parallelized.
- **F** – The type of the user defined function to invoke inside the define_task_block (deduced). *F* shall be MoveConstructible.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **f** – The user defined function to invoke inside the define_task_block. Given an lvalue *tr* of type task_block, the expression, (void)f(tr), shall be well-formed.

Throws

exception_list – specified in Exception Handling.

template<typename **F**>

void `hpx::experimental::define_task_block_restore_thread`(*F* &&f)

Constructs a task_block, tr, and invokes the expression *f(tr)* on the user-provided object, *f*. This version uses *parallel_policy* for task scheduling.

Postcondition: All tasks spawned from *f* have finished execution. A call to *define_task_block_restore_thread* always returns on the same thread as that on which it was called.

> **Note**
>
> It is expected (but not mandated) that f will (directly or indirectly) call tr.run(*callable_object*).

Template Parameters

F – The type of the user defined function to invoke inside the define_task_block (deduced). *F* shall be MoveConstructible.

Parameters

f – The user defined function to invoke inside the define_task_block. Given an lvalue *tr* of type task_block, the expression, (void)f(tr), shall be well-formed.

Throws

`exception_list` – specified in Exception Handling.

hpx/parallel/util/range.hpp

Defined in header hpx/parallel/util/range.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **util**

Typedefs

template<typename **Iterator**, typename **Sentinel** = *Iterator*>

using **range** = *hpx*::*util*::iterator_range<*Iterator*, *Sentinel*>

Functions

template<typename **Iter**, typename **Sent**>

range<*Iter*, *Sent*> **concat**(*range*<*Iter*, *Sent*> const &it1, *range*<*Iter*, *Sent*> const &it2)

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**>

range<*Iter2*, *Iter2*> **init_move**(*range*<*Iter2*, *Sent2*> const &dest, *range*<*Iter1*, *Sent1*> const &src)

Move objects from the range src to dest.

Parameters

- **dest** – **[in]** : range where move the objects
- **src** – **[in]** : range from where move the objects

Returns
range with the objects moved and the size adjusted

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**>

range<*Iter2*, *Sent2*> **uninit_move**(*range*<*Iter2*, *Sent2*> const &dest, *range*<*Iter1*, *Sent1*> const &src)

Move objects from the range src creating them in dest.

Parameters

- **dest** – **[in]** : range where move and create the objects
- **src** – **[in]** : range from where move the objects

Returns
range with the objects moved and the size adjusted

template<typename **Iter**, typename **Sent**>

void **destroy_range**(*range*<*Iter*, *Sent*> r)

destroy a range of objects

Parameters
r – **[in]** : range to destroy

template<typename **Iter**, typename **Sent**>

range<*Iter*, *Sent*> **init**(*range*<*Iter*, *Sent*> const &r, typename *std*::iterator_traits<*Iter*>::value_type &val)

initialize a range of objects with the object val moving across them

Parameters

- **r** – **[in]** : range of elements not initialized
- **val** – **[in]** : object used for the initialization

Returns
range initialized

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Compare**>

bool **is_mergeable**(*range*<*Iter1*, *Sent1*> const &src1, *range*<*Iter2*, *Sent2*> const &src2, *Compare* comp)

: indicate if two ranges have a possible merge

Remark

Parameters

- **src1** – **[in]** : first range
- **src2** – **[in]** : second range
- **comp** – **[in]** : object for to compare elements

Returns
true : they can be merged false : they can’t be merged

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Iter3**, typename **Sent3**, typename **Compare**>

range<*Iter3*, *Sent3*> **full_merge**(*range*<*Iter3*, *Sent3*> const &dest, *range*<*Iter1*, *Sent1*> const &src1, *range*<*Iter2*, *Sent2*> const &src2, *Compare* comp)

Merge two contiguous ranges src1 and src2 , and put the result in the range dest, returning the range merged.

Parameters

- **dest** – **[in]** : range where locate the elements merged. the size of dest must be greater or equal than the sum of the sizes of src1 and src2
- **src1** – **[in]** : first range to merge
- **src2** – **[in]** : second range to merge
- **comp** – **[in]** : comparison object

Returns

range with the elements merged and the size adjusted

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Value**, typename **Compare**>

range<*Value**> **uninit_full_merge**(*range*<*Value**> const &dest, *range*<*Iter1*, *Sent1*> const &src1, *range*<*Iter2*, *Sent2*> const &src2, *Compare* comp)

Merge two contiguous ranges src1 and src2 , and create and move the result in the uninitialized range dest, returning the range merged.

Parameters

- **dest** – **[in]** : range where locate the elements merged. the size of dest must be greater or equal than the sum of the sizes of src1 and src2. Initially is un-initialize memory
- **src1** – **[in]** : first range to merge
- **src2** – **[in]** : second range to merge
- **comp** – **[in]** : comparison object

Returns

range with the elements merged and the size adjusted

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Compare**>

range<*Iter2*, *Sent2*> **half_merge**(*range*<*Iter2*, *Sent2*> const &dest, *range*<*Iter1*, *Sent1*> const &src1, *range*<*Iter2*, *Sent2*> const &src2, *Compare* comp)

: Merge two buffers. The first buffer is in a separate memory

Parameters

- **dest** – **[in]** : range where finish the two buffers merged
- **src1** – **[in]** : first range to merge in a separate memory
- **src2** – **[in]** : second range to merge, in the final part of the range where deposit the final results
- **comp** – **[in]** : object for compare two elements of the type pointed by the Iter1 and Iter2

Returns

: range with the two buffers merged

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Iter3**, typename **Sent3**, typename **Compare**>

bool **in_place_merge_uncontiguous**(*range*<*Iter1*, *Sent1*> const &src1, *range*<*Iter2*, *Sent2*> const &src2, *range*<*Iter3*, *Sent3*> &aux, *Compare* comp)

: merge two non contiguous buffers src1 , src2, using the range aux as auxiliary memory

Remark

Parameters

- **src1** – **[in]** : first range to merge
- **src2** – **[in]** : second range to merge
- **aux** – **[in]** : auxiliary range used in the merge
- **comp** – **[in]** : object for to compare elements

Returns

true : not changes done false : changes in the buffers

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Compare**>

range<Iter1, Sent1> **in_place_merge**(*range<Iter1, Sent1>* const &src1, *range<Iter1, Sent1>* const &src2, *range<Iter2, Sent2>* &buf, *Compare* comp)

: merge two contiguous buffers (src1, src2) using buf as auxiliary memory

Remark

Parameters

- **src1** – **[in]** : first range to merge
- **src2** – **[in]** : second range to merge
- **buf** – **[in]** : auxiliary memory used in the merge
- **comp** – **[in]** : object for to compare elements

Returns

true : not changes done false : changes in the buffers

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Compare**>

void **merge_flow**(*range<Iter1, Sent1>* rng1, *range<Iter2, Sent2>* rbuf, *range<Iter1, Sent1>* rng2, *Compare* cmp)

hpx::shift_left

Defined in header hpx/algorithm.hpp[604].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **FwdIter**, typename **Size**>

[604] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

FwdIter `hpx::shift_left`(*FwdIter* first, *FwdIter* last, *Size* n)

Shifts the elements in the range [first, last) by n positions towards the beginning of the range. For every integer i in [0, last - first

- n), moves the element originally at position first + n + i to position first + i.

The assignment operations in the parallel *shift_left* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: At most (last - first) - n assignments.

> **Note**
>
> The type of dereferenced *FwdIter* must meet the requirements of *MoveAssignable*.

Template Parameters

- **`FwdIter`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **`Size`** – The type of the argument specifying the number of positions to shift by.

Parameters

- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`n`** – Refers to the number of positions to shift.

Returns

The *shift_left* algorithm returns *FwdIter*. The *shift_left* algorithm returns an iterator to the end of the resulting range. Specifically: If *n* is 0 or *n* >= (last - first), does nothing and returns *last* and *first* respectively. Otherwise returns *first* + ((last - first) - n).

template<typename **`ExPolicy`**, typename **`FwdIter`**, typename **`Size`**>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter*> `hpx::shift_left`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *Size* n)

Shifts the elements in the range [first, last) by n positions towards the beginning of the range. For every integer i in [0, last - first

- n), moves the element originally at position first + n + i to position first + i. Executed according to the policy.

The assignment operations in the parallel *shift_left* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignment operations in the parallel *shift_left* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most (last - first) - n assignments.

> **Note**
>
> The type of dereferenced *FwdIter* must meet the requirements of *MoveAssignable*.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **`Size`** – The type of the argument specifying the number of positions to shift by.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`n`** – Refers to the number of positions to shift.

Returns

The *shift_left* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *shift_left* algorithm returns an iterator to the end of the resulting range. Specifically: If *n* is 0, does nothing and returns *last*. If *n* >= (last - first), does nothing and returns *first*. Otherwise returns *first* + ((last - first) - n).

hpx::reduce_deterministic

Defined in header hpx/algorithm.hpp[605].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **`ExPolicy`**, typename **`FwdIter`**, typename **`F`**, typename **`T`** = typename *std*::iterator_traits<*FwdIter*>::value_type>

[605] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> `hpx::reduce_deterministic`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *T* init, *F* &&f)

Returns GENERALIZED_SUM(f, init, *first, …, *(first + (last - first) - 1)). Executed according to the policy.

The reduce operations in the parallel *reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *f*.

> **Note**
>
> GENERALIZED_SUM(op, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter`** – The type of the source begin and end iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **`F`** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *reduce* requires *F* to meet the requirements of *CopyConstructible*.
- **`T`** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **init** – The initial value for the generalized sum.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&. The types *Type1 Ret* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to any of those types.

Returns

The *reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *T* otherwise. The *reduce* algorithm returns the result of the generalized sum over the elements given by the input range [first, last).

template<typename **ExPolicy**, typename **FwdIter**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>

util::detail::algorithm_result_t<*ExPolicy*, *T*> hpx::**reduce_deterministic**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *T* init)

Returns GENERALIZED_SUM(+, init, *first, …, *(first + (last - first) - 1)). Executed according to the policy.

The reduce operations in the parallel *reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

Note

Complexity: O(*last* - *first*) applications of the operator+().

Note

GENERALIZED_SUM(+, a1, …, aN) is defined as follows:

- a1 when N is 1

- op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
  - b1, …, bN may be any permutation of a1, …, aN and
  - 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source begin and end iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **init** – The initial value for the generalized sum.

Returns

The *reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *T* otherwise. The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

template<typename **ExPolicy**, typename **FwdIter**>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, typename *std*::iterator_traits<*FwdIter*>::value_type>::type `hpx::reduce_determinis`

Returns GENERALIZED_SUM(+, T(), *first, …, *(first + (last - first) - 1)). Executed according to the policy.

The reduce operations in the parallel *reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *reduce* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the operator+().

> **Note**
>
> The type of the initial value (and the result type) *T* is determined from the value_type of the used *FwdIter*.

> **Note**
>
> GENERALIZED_SUM(+, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source begin and end iterators used (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns T otherwise (where T is the value_type of *FwdIter*). The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

template<typename **FwdIter**, typename **F**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>

T *hpx*::**reduce_deterministic**(*FwdIter* first, *FwdIter* last, *T* init, *F* &&f)

Returns GENERALIZED_SUM(f, init, *first, …, *(first + (last - first) - 1)). Executed according to the policy.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *f*.

> **Note**
>
> GENERALIZED_SUM(op, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **`FwdIter`** – The type of the source begin and end iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **`F`** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *reduce* requires *F* to meet the requirements of *CopyConstructible*.
- **`T`** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`init`** – The initial value for the generalized sum.
- **`f`** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&. The types *Type1 Ret* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to any of those types.

Returns

The *reduce* algorithm returns *T*. The *reduce* algorithm returns the result of the generalized sum over the elements given by the input range [first, last).

template<typename **FwdIter**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>

T *hpx*::**reduce_deterministic**(*FwdIter* first, *FwdIter* last, *T* init)

Returns GENERALIZED_SUM(+, init, *first, …, *(first + (last - first) - 1)). Executed according to the policy.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the operator+().

> **Note**
>
> GENERALIZED_SUM(+, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **FwdIter** – The type of the source begin and end iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **init** – The initial value for the generalized sum.

Returns

The *reduce* algorithm returns a *T*. The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

template<typename **FwdIter**>

std::iterator_traits<*FwdIter*>::value_type `hpx::reduce_deterministic`(*FwdIter* first, *FwdIter* last)

Returns GENERALIZED_SUM(+, T(), *first, …, *(first + (last - first) - 1)). Executed according to the policy.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

Note

Complexity: O(*last* - *first*) applications of the operator+().

Note

The type of the initial value (and the result type) *T* is determined from the value_type of the used *FwdIter*.

Note

GENERALIZED_SUM(+, a1, …, aN) is defined as follows:

- a1 when N is 1
- op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
  - b1, …, bN may be any permutation of a1, …, aN and
  - 1 < K+1 = M <= N.

Template Parameters

`FwdIter` – The type of the source begin and end iterators used (deduced). This iterator type must meet the requirements of an input iterator.

Parameters

- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `last` – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *reduce* algorithm returns *T* (where T is the value_type of *FwdIter*). The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

hpx::exclusive_scan

Defined in header hpx/algorithm.hpp[606].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter**, typename **OutIter**, typename **T**>

OutIter `hpx::exclusive_scan`(*InIter* first, *InIter* last, *OutIter* dest, *T* init)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(+, init, *first, …, *(first + (i - result) - 1))

The reduce operations in the parallel *exclusive_scan* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum.

Note

Complexity: O(*last* - *first*) applications of the predicate *std::plus<T>*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, …, aN) is defined as:

- a1 when N is 1
- GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, …, aK)
- GENERALIZED_NONCOMMUTATIVE_SUM(+, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

[606] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **dest** – Refers to the beginning of the destination range.
- **init** – The initial value for the generalized sum.

Returns

The *exclusive_scan* algorithm returns *OutIter*. The *exclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **T**>

util::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> *hpx*::**exclusive_scan**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *T* init)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(+, init, *first, …, *(first + (i - result) - 1))

The reduce operations in the parallel *exclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *exclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *std::plus*<*T*>.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, …, aN) is defined as:
>
> - a1 when N is 1
> - GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, …, aK)
> - GENERALIZED_NONCOMMUTATIVE_SUM(+, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.

- `FwdIter2` – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- `T` – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `last` – Refers to the end of the sequence of elements the algorithm will be applied to.
- `dest` – Refers to the beginning of the destination range.
- `init` – The initial value for the generalized sum.

Returns

The *exclusive_scan* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *exclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **InIter**, typename **OutIter**, typename **T**, typename **Op**>

OutIter `hpx::exclusive_scan`(*InIter* first, *InIter* last, *OutIter* dest, *T* init, *Op* &&op)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, *first, …, *(first + (i - result) - 1)).

The reduce operations in the parallel *exclusive_scan* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum. If *op* is not mathematically associative, the behavior of *inclusive_scan* may be non-deterministic.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:
>
> - a1 when N is 1
> - op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN)) where 1 < K+1 = M <= N.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **Op** – The type of the binary function object used for the reduction operation.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **init** – The initial value for the generalized sum.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

Returns

The *exclusive_scan* algorithm returns *OutIter*. The *exclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Op**, typename **T**>

util::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> hpx::**exclusive_scan**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *T* init, *Op* &&op)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, *first, …, *(first + (i - result) - 1)).

The reduce operations in the parallel *exclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *exclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum. If *op* is not mathematically associative, the behavior of *inclusive_scan* may be non-deterministic.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:
>
> - a1 when N is 1
> - op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN)) where 1 < K+1 = M <= N.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter1`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **`FwdIter2`** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Op`** – The type of the binary function object used for the reduction operation.
- **`T`** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`dest`** – Refers to the beginning of the destination range.
- **`init`** – The initial value for the generalized sum.
- **`op`** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

Returns

The *exclusive_scan* algorithm returns a *hpx::future<OutIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *OutIter* otherwise. The *exclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::swap_ranges

Defined in header hpx/algorithm.hpp[607].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **FwdIter1**, typename **FwdIter2**>

FwdIter2 `hpx::swap_ranges`(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2)

Exchanges elements between range [first1, last1) and another range starting at *first2*.

The swap operations in the parallel *swap_ranges* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear in the distance between *first1* and *last1*.

Template Parameters

- **FwdIter1** – The type of the first range of iterators to swap (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the second range of iterators to swap (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **first1** – Refers to the beginning of the first sequence of elements the algorithm will be applied to.
- **last1** – Refers to the end of the first sequence of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second sequence of elements the algorithm will be applied to.

Returns

The *swap_ranges* algorithm returns *FwdIter2*. The *swap_ranges* algorithm returns iterator to the element past the last element exchanged in the range beginning with *first2*.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**>

[607] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> `hpx::swap_ranges`(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2)

Exchanges elements between range [first1, last1) and another range starting at *first2*. Executed according to the policy.

The swap operations in the parallel *swap_ranges* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The swap operations in the parallel *swap_ranges* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Linear in the distance between *first1* and *last1*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the swap operations.
- **FwdIter1** – The type of the first range of iterators to swap (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the second range of iterators to swap (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the first sequence of elements the algorithm will be applied to.
- **last1** – Refers to the end of the first sequence of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second sequence of elements the algorithm will be applied to.

Returns

The *swap_ranges* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *parallel_task_policy* and returns *FwdIter2* otherwise. The *swap_ranges* algorithm returns iterator to the element past the last element exchanged in the range beginning with *first2*.

hpx::stable_sort

Defined in header hpx/algorithm.hpp[608].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **RandomIt**, typename **Comp** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

void *hpx*::**stable_sort**(*RandomIt* first, *RandomIt* last, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Sorts the elements in the range [first, last) in ascending order. The relative order of equal elements is preserved. The function uses the given comparison function object comp (defaults to using operator<()).

A sequence is sorted with respect to a comparator *comp* and a projection *proj* if for every iterator i pointing to the sequence and every non-negative integer n such that i + n is a valid iterator pointing to an element of the sequence, and INVOKE(comp, INVOKE(proj, *(i + n)), INVOKE(proj, *i)) == false.

comp has to induce a strict weak ordering on the values.

The assignments in the parallel *stable_sort* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Note

Complexity: O(N log(N)), where N = std::distance(first, last) comparisons.

Template Parameters

- **RandomIt** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *stable_sort* algorithm returns *void*.

[608] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

template<typename **ExPolicy**, typename **RandomIt**, typename **Comp** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*> **hpx::stable_sort**(*ExPolicy* &&policy, *RandomIt* first, *RandomIt* last, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Sorts the elements in the range [first, last) in ascending order. The relative order of equal elements is preserved. The function uses the given comparison function object comp (defaults to using operator<()). Executed according to the policy.

A sequence is sorted with respect to a comparator *comp* and a projection *proj* if for every iterator i pointing to the sequence and every non-negative integer n such that i + n is a valid iterator pointing to an element of the sequence, and INVOKE(comp, INVOKE(proj, *(i + n)), INVOKE(proj, *i)) == false.

comp has to induce a strict weak ordering on the values.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(N log(N)), where N = std::distance(first, last) comparisons.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **RandomIt** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *stable_sort* algorithm returns a *hpx::future<void>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *void* otherwise.

hpx::unique

Defined in header hpx/algorithm.hpp[609].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::unique_copy*

template<typename **FwdIter**, typename **Pred** = *hpx*::*parallel*::detail::equal_to, typename **Proj** = *hpx*::identity>

FwdIter `hpx::unique`(*FwdIter* first, *FwdIter* last, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Eliminates all but the first element from every consecutive group of equivalent elements from the range [first, last) and returns a past-the-end iterator for the new logical end of the range.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* - 1 applications of the predicate *pred* and no more than twice as many applications of the projection *proj*.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *unique* requires *Pred* to meet the requirements of *Copy-Constructible*. This defaults to std::equal_to<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an binary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

```
bool pred(const Type1 &a, const Type2 &b);
```

[609] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to both *Type1* and *Type2*

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *pred* is invoked.

Returns

The *unique* algorithm returns *FwdIter*. The *unique* algorithm returns the iterator to the new end of the range.

template<typename **ExPolicy**, typename **FwdIter**, typename **Pred** = *hpx*::*parallel*::detail::equal_to, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter*>::type **hpx::unique**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Eliminates all but the first element from every consecutive group of equivalent elements from the range [first, last) and returns a past-the-end iterator for the new logical end of the range. Executed according to the policy.

The assignments in the parallel *unique* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *unique* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* - 1 applications of the predicate *pred* and no more than twice as many applications of the projection *proj*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *unique* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.

- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an binary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

```
bool pred(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to both *Type1* and *Type2*
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *pred* is invoked.

Returns

The *unique* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *unique* algorithm returns the iterator to the new end of the range.

hpx::unique_copy

Defined in header hpx/algorithm.hpp[610].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::unique*

template<typename **InIter**, typename **OutIter**, typename **Pred** = *hpx*::*parallel*::detail::equal_to, typename **Proj** = *hpx*::identity>

OutIter `hpx::unique_copy`(*InIter* first, *InIter* last, *OutIter* dest, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Copies the elements from the range [first, last), to another range beginning at *dest* in such a way that there are no consecutive equal elements. Only the first element of each group of equal elements is copied.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* - 1 applications of the predicate *pred* and no more than twice as many applications of the projection *proj*

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.

[610] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *unique_copy* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an binary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a, const Type &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *pred* is invoked.

Returns

The *unique_copy* algorithm returns a returns *OutIter*. The *unique_copy* algorithm returns the destination iterator to the end of the *dest* range.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Pred** = *hpx*::*parallel*::detail::equal_to, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter2*>::type **hpx::unique_copy**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Copies the elements from the range [first, last), to another range beginning at *dest* in such a way that there are no consecutive equal elements. Only the first element of each group of equal elements is copied. Executed according to the policy.

The assignments in the parallel *unique_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *unique_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* - 1 applications of the predicate *pred* and no more than twice as many applications of the projection *proj*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *unique_copy* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an binary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a, const Type &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to *Type*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *pred* is invoked.

Returns

The *unique_copy* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *unique_copy* algorithm returns the pair of the source iterator to *last*, and the destination iterator to the end of the *dest* range.

hpx::merge

Defined in header hpx/algorithm.hpp[611].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::inplace_merge*

template<typename **ExPolicy**, typename **RandIter1**, typename **RandIter2**, typename **RandIter3**, typename **Comp** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *RandIter3*> `hpx::merge`(*ExPolicy* &&policy, *RandIter1* first1, *RandIter1* last1, *RandIter2* first2, *RandIter2* last2, *RandIter3* dest, *Comp* &&comp = *Comp*())

Merges two sorted ranges [first1, last1) and [first2, last2) into one sorted range beginning at *dest*. The order of equivalent elements in each of the original two ranges is preserved. For equivalent elements in the original two ranges, the elements from the first range precede the elements from the second range. The destination range cannot overlap with either of the input ranges. Executed according to the policy.

The assignments in the parallel *merge* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *merge* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs O(std::distance(first1, last1) + std::distance(first2, last2)) applications of the comparison *comp* and each projection.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **RandIter1** – The type of the source iterators used (deduced) representing the first sorted range. This iterator type must meet the requirements of a random access iterator.
- **RandIter2** – The type of the source iterators used (deduced) representing the second sorted range. This iterator type must meet the requirements of a random access iterator.
- **RandIter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a random access iterator.
- **Comp** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *merge* requires *Comp* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

[611] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the first range of elements the algorithm will be applied to.
- **last1** – Refers to the end of the first range of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second range of elements the algorithm will be applied to.
- **last2** – Refers to the end of the second range of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **comp** – *comp* is a callable object which returns true if the first argument is less than the second, and false otherwise. The signature of this comparison should be equivalent to:

```
bool comp(const Type1 &a, const Type2 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *RandIter1* and *RandIter2* can be dereferenced and then implicitly converted to both *Type1* and *Type2*

Returns

The *merge* algorithm returns a *hpx::future<RandIter3> >* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *RandIter3* otherwise. The *merge* algorithm returns the destination iterator to the end of the *dest* range.

template<typename **RandIter1**, typename **RandIter2**, typename **RandIter3**, typename **Comp** = *hpx*::*parallel*::detail::less>

RandIter3 **hpx::merge**(*RandIter1* first1, *RandIter1* last1, *RandIter2* first2, *RandIter2* last2, *RandIter3* dest, *Comp* &&comp = *Comp*())

Merges two sorted ranges [first1, last1) and [first2, last2) into one sorted range beginning at *dest*. The order of equivalent elements in each of the original two ranges is preserved. For equivalent elements in the original two ranges, the elements from the first range precede the elements from the second range. The destination range cannot overlap with either of the input ranges.

Note

Complexity: Performs O(std::distance(first1, last1) + std::distance(first2, last2)) applications of the comparison *comp* and each projection.

Template Parameters

- **RandIter1** – The type of the source iterators used (deduced) representing the first sorted range. This iterator type must meet the requirements of a random access iterator.
- **RandIter2** – The type of the source iterators used (deduced) representing the second sorted range. This iterator type must meet the requirements of a random access iterator.
- **RandIter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a random access iterator.

- **Comp** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *merge* requires *Comp* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **first1** – Refers to the beginning of the first range of elements the algorithm will be applied to.
- **last1** – Refers to the end of the first range of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second range of elements the algorithm will be applied to.
- **last2** – Refers to the end of the second range of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **comp** – *comp* is a callable object which returns true if the first argument is less than the second, and false otherwise. The signature of this comparison should be equivalent to:

```
bool comp(const Type1 &a, const Type2 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *RandIter1* and *RandIter2* can be dereferenced and then implicitly converted to both *Type1* and *Type2*

Returns

The *merge* algorithm returns a *RandIter3*. The *merge* algorithm returns the destination iterator to the end of the *dest* range.

hpx::inplace_merge

Defined in header hpx/algorithm.hpp[612].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::merge*

template<typename **ExPolicy**, typename **RandIter**, typename **Comp** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*> **hpx::inplace_merge**(*ExPolicy* &&policy, *RandIter* first, *RandIter* middle, *RandIter* last, *Comp* &&comp = *Comp*())

Merges two consecutive sorted ranges [first, middle) and [middle, last) into one sorted range [first, last). The order of equivalent elements in each of the original two ranges is preserved. For equivalent elements in the original two ranges, the elements from the first range precede the elements from the second range. Executed according to the policy.

The assignments in the parallel *inplace_merge* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

[612] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The assignments in the parallel *inplace_merge* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs O(std::distance(first, last)) applications of the comparison *comp* and each projection.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`RandIter`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **`Comp`** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *inplace_merge* requires *Comp* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the first sorted range the algorithm will be applied to.
- **`middle`** – Refers to the end of the first sorted range and the beginning of the second sorted range the algorithm will be applied to.
- **`last`** – Refers to the end of the second sorted range the algorithm will be applied to.
- **`comp`** – *comp* is a callable object which returns true if the first argument is less than the second, and false otherwise. The signature of this comparison should be equivalent to:

  ```
  bool comp(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *RandIter* can be dereferenced and then implicitly converted to both *Type1* and *Type2*

Returns

The *inplace_merge* algorithm returns a *hpx::future<void>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns void otherwise. The *inplace_merge* algorithm returns the source iterator *last*.

template<typename **`RandIter`**, typename **`Comp`** = *hpx*::*parallel*::detail::less>

void *hpx*::**`inplace_merge`**(*RandIter* first, *RandIter* middle, *RandIter* last, *Comp* &&comp = *Comp*())

Merges two consecutive sorted ranges [first, middle) and [middle, last) into one sorted range [first, last). The order of equivalent elements in each of the original two ranges is preserved. For equivalent elements in the original two ranges, the elements from the first range precede the elements from the second range.

> **Note**
>
> Complexity: Performs O(std::distance(first, last)) applications of the comparison *comp* and each projection.

Template Parameters

- **RandIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **Comp** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *inplace_merge* requires *Comp* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **first** – Refers to the beginning of the first sorted range the algorithm will be applied to.
- **middle** – Refers to the end of the first sorted range and the beginning of the second sorted range the algorithm will be applied to.
- **last** – Refers to the end of the second sorted range the algorithm will be applied to.
- **comp** – *comp* is a callable object which returns true if the first argument is less than the second, and false otherwise. The signature of this comparison should be equivalent to:

```
bool comp(const Type1 &a, const Type2 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *RandIter* can be dereferenced and then implicitly converted to both *Type1* and *Type2*

Returns

The *inplace_merge* algorithm returns a *void*. The *inplace_merge* algorithm returns the source iterator *last*.

hpx::search

Defined in header hpx/algorithm.hpp[613].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::search_n*

template<typename **FwdIter**, typename **FwdIter2**, typename **Pred** = *parallel*::detail::equal_to>

[613] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

FwdIter `hpx::search`(*FwdIter* first, *FwdIter* last, *FwdIter2* s_first, *FwdIter2* s_last, *Pred* &&op = *Pred*())

Searches the range [first, last) for any elements in the range [s_first, s_last). Uses a provided predicate to compare elements.

The comparison operations in the parallel *search* algorithm execute in sequential order in the calling thread.

Note

Complexity: at most (S*N) comparisons where S = distance(s_first, s_last) and N = distance(first, last).

Template Parameters

- **FwdIter** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *search* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to *std::equal_to*<>

Parameters

- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **s_first** – Refers to the beginning of the sequence of elements the algorithm will be searching for.
- **s_last** – Refers to the end of the sequence of elements of the algorithm will be searching for.
- **op** – Refers to the binary predicate which returns true if the elements should be treated as equal. the signature of the function should be equivalent to

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

Returns

The *search* algorithm returns a *hpx::future*<*FwdIter*> if the execution policy is of type *task_execution_policy* and returns *FwdIter* otherwise. The *search* algorithm returns an iterator to the beginning of the first subsequence [s_first, s_last) in range [first, last). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, last), *last* is returned. Additionally if the size of the subsequence is empty *first* is returned. If no subsequence is found, *last* is returned.

template<typename **ExPolicy**, typename **FwdIter**, typename **FwdIter2**, typename **Pred** = *parallel*::detail::equal_to>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> **hpx::search**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *FwdIter2* s_first, *FwdIter2* s_last, *Pred* &&op = *Pred*())

Searches the range [first, last) for any elements in the range [s_first, s_last). Uses a provided predicate to compare elements. Executed according to the policy.

The comparison operations in the parallel *search* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *search* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: at most (S*N) comparisons where *S* = distance(s_first, s_last) and *N* = distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *search* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to *std::equal_to<>*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **s_first** – Refers to the beginning of the sequence of elements the algorithm will be searching for.

- **s_last** – Refers to the end of the sequence of elements of the algorithm will be searching for.
- **op** – Refers to the binary predicate which returns true if the elements should be treated as equal. the signature of the function should be equivalent to

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

Returns

The *search* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *task_execution_policy* and returns *FwdIter* otherwise. The *search* algorithm returns an iterator to the beginning of the first subsequence [s_first, s_last) in range [first, last). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, last), *last* is returned. Additionally if the size of the subsequence is empty *first* is returned. If no subsequence is found, *last* is returned.

template<typename **FwdIter**, typename **Searcher**>

FwdIter hpx::**search**(*FwdIter* first, *FwdIter* last, *Searcher* &&searcher)

Searches the range [first, last) using a searcher object.

Template Parameters

- **FwdIter** – The type of the source iterators (deduced). This iterator type must meet the requirements of a forward iterator.
- **Searcher** – A callable searcher object meeting the C++ Searcher requirements (e.g. std::boyer_moore_searcher).

Parameters

- **first** – Refers to the beginning of the sequence to search.
- **last** – Refers to the end of the sequence to search.
- **searcher** – A callable object performing the search.

Returns

An iterator to the beginning of the found subsequence, or *last* if no subsequence is found.

template<typename **ExPolicy**, typename **FwdIter**, typename **Searcher**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> hpx::**search**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *Searcher* &&searcher)

Searches the range [first, last) using a searcher object. Executed according to the policy.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced).

- **FwdIter** – The type of the source iterators (deduced).
- **Searcher** – A callable searcher object meeting the C++ Searcher requirements.

Parameters

- **policy** – The execution policy to use.
- **first** – Refers to the beginning of the sequence to search.
- **last** – Refers to the end of the sequence to search.
- **searcher** – A callable object performing the search.

Returns

The algorithm returns a *hpx::future<FwdIter>* if the execution policy is a task policy, or *FwdIter* otherwise.

hpx::search_n

Defined in header hpx/algorithm.hpp[614].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::search*

template<typename **FwdIter**, typename **FwdIter2**, typename **Pred** = *parallel*::detail::equal_to>

FwdIter **hpx::search_n**(*FwdIter* first, *std*::size_t count, *FwdIter2* s_first, *FwdIter2* s_last, *Pred* &&op = *Pred*())

Searches the range [first, last) for any elements in the range [s_first, s_last). Uses a provided predicate to compare elements.

The comparison operations in the parallel *search_n* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: at most (S*N) comparisons where *S* = distance(s_first, s_last) and *N* = count.

Template Parameters

- **FwdIter** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *search_n* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to *std::equal_to<>*

[614] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Parameters

- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **count** – Refers to the range of elements of the first range the algorithm will be applied to.
- **s_first** – Refers to the beginning of the sequence of elements the algorithm will be searching for.
- **s_last** – Refers to the end of the sequence of elements of the algorithm will be searching for.
- **op** – Refers to the binary predicate which returns true if the elements should be treated as equal. the signature of the function should be equivalent to

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

Returns

The *search_n* algorithm returns *FwdIter*. The *search_n* algorithm returns an iterator to the beginning of the last subsequence [s_first, s_last) in range [first, first+count). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, first+count), *first* is returned. Additionally if the size of the subsequence is empty or no subsequence is found, *first* is also returned.

template<typename **ExPolicy**, typename **FwdIter**, typename **FwdIter2**, typename **Pred** = *parallel*::detail::equal_to>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> **hpx::search_n**(*ExPolicy* &&policy, *FwdIter* first, *std*::size_t count, *FwdIter2* s_first, *FwdIter2* s_last, *Pred* &&op = *Pred*())

Searches the range [first, last) for any elements in the range [s_first, s_last). Uses a provided predicate to compare elements. Executed according to the policy.

The comparison operations in the parallel *search_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *search_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (S*N) comparisons where *S* = distance(s_first, s_last) and *N* = count.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *search_n* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to *std::equal_to*<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **count** – Refers to the range of elements of the first range the algorithm will be applied to.
- **s_first** – Refers to the beginning of the sequence of elements the algorithm will be searching for.
- **s_last** – Refers to the end of the sequence of elements of the algorithm will be searching for.
- **op** – Refers to the binary predicate which returns true if the elements should be treated as equal. the signature of the function should be equivalent to

```
bool pred(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

Returns

The *search_n* algorithm returns a *hpx::future*<*FwdIter*> if the execution policy is of type *task_execution_policy* and returns *FwdIter* otherwise. The *search_n* algorithm returns an iterator to the beginning of the last subsequence [s_first, s_last) in range [first, first+count). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, first+count), *first* is returned. Additionally if the size of the subsequence is empty or no subsequence is found, *first* is also returned.

hpx::experimental::reduction_bit_or

Defined in header hpx/algorithm.hpp[615].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *std*::bit_or<*T*>> `hpx::experimental::`**`reduction_bit_or`**(*T* &var)

The function template *reduction_bit_or* returns a reduction object of unspecified type having a value type and encapsulating an identity value for the reduction, it uses std::bit_or{} as its combiner function, and a live-out object from which the initial value is obtained and into which the final value is stored.

A parallel algorithm uses the *reduction_bit_or* object by allocating an unspecified number of instances, called views, of the reduction's value type. Each view is initialized with the reduction object's identity value, except that the live-out object (which was initialized by the caller) comprises one of the views. The algorithm passes a reference to a view to each application of an element-access function, ensuring that no two concurrently-executing invocations share the same view. A view can be shared between two applications that do not execute concurrently, but initialization is performed only once per view.

Modifications to the view by the application of element access functions accumulate as partial results. At some point before the algorithm returns, the partial results are combined, two at a time, using the std::bit_or{} operation until a single value remains, which is then assigned back to the live-out object.

T shall meet the requirements of *CopyConstructible* and *MoveAssignable*.

> **Note**
>
> In order to produce useful results, modifications to the view should be limited to commutative operations closely related to the combiner operation.

Template Parameters

- **T** – The value type to be used by the induction object.
- **Op** – The type of the binary function (object) used to perform the reduction operation.

Parameters

- **var** – [in,out] The life-out value to use for the reduction object. This will hold the reduced value after the algorithm is finished executing.
- **identity** – [in] The identity value to use for the reduction operation. This argument is optional and defaults to a copy of *var*.

Returns

This returns a reduction object of unspecified type having a value type of *T*. When the

[615] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

return value is used by an algorithm, the reference to *var* is used as the live-out object, new views are initialized to a copy of identity, and views are combined by invoking the copy of combiner, passing it the two views to be combined.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *std*::bit_or<*T*>> *hpx*::*experimental*::**reduction_bit_or**(*T* &var, *T* const &identity)

hpx::is_partitioned

Defined in header hpx/algorithm.hpp[616].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **FwdIter**, typename **Pred**>

bool *hpx*::**is_partitioned**(*FwdIter* first, *FwdIter* last, *Pred* &&pred)

Determines if the range [first, last) is partitioned.

> **Note**
>
> Complexity: at most (N) predicate evaluations where *N* = distance(first, last).

Template Parameters

- **FwdIter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of the function/function object to use (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of that the algorithm will be applied to.
- **pred** – Refers to the unary predicate which returns true for elements expected to be found in the beginning of the range. The signature of the function should be equivalent to

```
bool pred(const Type &a);
```

[616] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.

Returns

The *is_partitioned* algorithm returns *bool*. The *is_partitioned* algorithm returns true if each element in the sequence for which pred returns true precedes those for which pred returns false. Otherwise is_partitioned returns false. If the range [first, last) contains less than two elements, the function is always true.

template<typename **ExPolicy**, typename **FwdIter**, typename **Pred**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::is_partitioned`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *Pred* &&pred)

Determines if the range [first, last) is partitioned. Executed according to the policy.

The predicate operations in the parallel *is_partitioned* algorithm invoked with an execution policy object of type *sequenced_policy* executes in sequential order in the calling thread.

The comparison operations in the parallel *is_partitioned* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: at most (N) predicate evaluations where *N* = distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of the function/function object to use (deduced). *Pred* must be *Copy-Constructible* when using a parallel policy.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of that the algorithm will be applied to.

- **pred** – Refers to the unary predicate which returns true for elements expected to be found in the beginning of the range. The signature of the function should be equivalent to

```
bool pred(const Type &a);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.

Returns

The *is_partitioned* algorithm returns a *hpx::future<bool>* if the execution policy is of type *task_execution_policy* and returns *bool* otherwise. The *is_partitioned* algorithm returns true if each element in the sequence for which pred returns true precedes those for which pred returns false. Otherwise is_partitioned returns false. If the range [first, last) contains less than two elements, the function is always true.

hpx::starts_with

Defined in header hpx/algorithm.hpp[617].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter1**, typename **InIter2**, typename **Pred** = *hpx*::*parallel*::detail::equal_to>

bool *hpx*::**starts_with**(*InIter1* first1, *InIter1* last1, *InIter2* first2, *InIter2* last2, *Pred* &&pred = *Pred*())

Checks whether the second range defined by [first1, last1) matches the prefix of the first range defined by [first2, last2)

The assignments in the parallel *starts_with* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear: at most min(N1, N2) applications of the predicate.

Template Parameters

- **InIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **InIter2** – The type of the destination iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Pred** – The binary predicate that compares the elements. This defaults to *hpx::parallel::detail::equal_to*.

Parameters

- **first1** – Refers to the beginning of the source range.

[617] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **last1** – Sentinel value referring to the end of the source range.
- **first2** – Refers to the beginning of the destination range.
- **last2** – Sentinel value referring to the end of the destination range.
- **pred** – Specifies the binary predicate function (or function object) which will be invoked for comparison of the elements in the two ranges.

Returns

The *starts_with* algorithm returns *bool*. The *starts_with* algorithm returns a boolean with the value true if the second range matches the prefix of the first range, false otherwise.

template<typename **ExPolicy**, typename **InIter1**, typename **InIter2**, typename **Pred** = *ranges*::equal_to>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> *hpx*::**starts_with**(*ExPolicy* &&policy, *InIter1* first1, *InIter1* last1, *InIter2* first2, *InIter2* last2, *Pred* &&pred = *Pred*())

Checks whether the second range defined by [first1, last1) matches the prefix of the first range defined by [first2, last2). Executed according to the policy.

The assignments in the parallel *starts_with* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear: at most min(N1, N2) applications of the predicate.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **InIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **InIter2** – The type of the destination iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Pred** – The binary predicate that compares the elements. This defaults to *ranges::equal_to*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the source range.
- **last1** – Sentinel value referring to the end of the source range.
- **first2** – Refers to the beginning of the destination range.
- **last2** – Sentinel value referring to the end of the destination range.

- **pred** – Specifies the binary predicate function (or function object) which will be invoked for comparison of the elements in the two ranges.

Returns

The *starts_with* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *starts_with* algorithm returns a boolean with the value true if the second range matches the prefix of the first range, false otherwise.

hpx::uninitialized_copy

Defined in header hpx/algorithm.hpp[618].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::uninitialized_copy_n*

template<typename **InIter**, typename **FwdIter**>

FwdIter hpx::**uninitialized_copy**(*InIter* first, *InIter* last, *FwdIter* dest)

Copies the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the copy operation, the function has no effects.

The assignments in the parallel *uninitialized_copy* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

[618] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Returns

The *uninitialized_copy* algorithm returns *FwdIter*. The *uninitialized_copy* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> `hpx::`**`uninitialized_copy`**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest)

Copies the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the copy operation, the function has no effects. Executed according to the policy.

The assignments in the parallel *uninitialized_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *uninitialized_copy* algorithm returns a *hpx::future<FwdIter2>*, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *uninitialized_copy* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::uninitialized_copy_n

Defined in header hpx/algorithm.hpp[619].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::uninitialized_copy*

template<typename **InIter**, typename **Size**, typename **FwdIter**>

FwdIter `hpx::uninitialized_copy_n`(*InIter* first, *Size* count, *FwdIter* dest)

Copies the elements in the range [first, first + count), starting from first and proceeding to first + count - 1., to another range beginning at dest. If an exception is thrown during the copy operation, the function has no effects.

The assignments in the parallel *uninitialized_copy_n* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *uninitialized_copy_n* algorithm returns a returns *FwdIter*. The *uninitialized_copy_n* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

[619] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

template<typename **ExPolicy**, typename **FwdIter1**, typename **Size**, typename **FwdIter2**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> `hpx::uninitialized_copy_n`(*ExPolicy* &&policy, *FwdIter1* first, *Size* count, *FwdIter2* dest)

Copies the elements in the range [first, first + count), starting from first and proceeding to first + count - 1., to another range beginning at dest. If an exception is thrown during the copy operation, the function has no effects.

The assignments in the parallel *uninitialized_copy_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_copy_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *uninitialized_copy_n* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *uninitialized_copy_n* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::partial_sort_copy

Defined in header hpx/algorithm.hpp[620].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter**, typename **RandIter**, typename **Comp** = *hpx*::*parallel*::detail::less>

RandIter `hpx::partial_sort_copy`(*InIter* first, *InIter* last, *RandIter* d_first, *RandIter* d_last, *Comp* &&comp = *Comp*())

Sorts some of the elements in the range [first, last) in ascending order, storing the result in the range [d_first, d_last). At most d_last - d_first of the elements are placed sorted to the range [d_first, d_first + n) where n is the number of elements to sort (n = min(last - first, d_last - d_first)).

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(N log(min(D,N))), where N = std::distance(first, last) and D = std::distance(d_first, d_last) comparisons.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **RandIter** – The type of the destination iterators used(deduced) This iterator type must meet the requirements of an random iterator.
- **Comp** – The type of the function/function object to use (deduced). Comp defaults to detail::less.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **d_first** – Refers to the beginning of the destination range.
- **d_last** – Refers to the end of the destination range.

[620] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator. This defaults to detail::less.

Returns

The *partial_sort_copy* algorithm returns a *RandomIt*. The algorithm returns an iterator to the element defining the upper boundary of the sorted range i.e. d_first + min(last - first, d_last - d_first)

template<typename **ExPolicy**, typename **FwdIter**, typename **RandIter**, typename **Comp** = *hpx*::*parallel*::detail::less>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *RandIter*> `hpx::partial_sort_copy`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *RandIter* d_first, *RandIter* d_last, *Comp* &&comp = *Comp*())

Sorts some of the elements in the range [first, last) in ascending order, storing the result in the range [d_first, d_last). At most d_last - d_first of the elements are placed sorted to the range [d_first, d_first + n) where n is the number of elements to sort (n = min(last - first, d_last - d_first)). Executed according to the policy.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(N log(min(D,N))), where N = std::distance(first, last) and D = std::distance(d_first, d_last) comparisons.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **RandIter** – The type of the destination iterators used(deduced) This iterator type must meet the requirements of an random iterator.
- **Comp** – The type of the function/function object to use (deduced). Comp defaults to detail::less.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **d_first** – Refers to the beginning of the destination range.
- **d_last** – Refers to the end of the destination range.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator. This defaults to detail::less.

Returns

The *partial_sort_copy* algorithm returns a *hpx::future<RandomIt>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *RandomIt* otherwise. The algorithm returns an iterator to the element defining the upper boundary of the sorted range i.e. d_first + min(last - first, d_last - d_first)

hpx::inclusive_scan

Defined in header hpx/algorithm.hpp[621].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter**, typename **OutIter**>

OutIter `hpx::`**`inclusive_scan`**(*InIter* first, *InIter* last, *OutIter* dest)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(+, *first, …, *(first + (i - result))).

The reduce operations in the parallel *inclusive_scan* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum.

Note

Complexity: O(*last* - *first*) applications of the predicate *op*, here std::plus<>().

Note

GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, …, aN) is defined as:

- a1 when N is 1

[621] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, …, aK)
- GENERALIZED_NONCOMMUTATIVE_SUM(+, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *inclusive_scan* algorithm returns *OutIter*. The *inclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

```
template<typename ExPolicy, typename FwdIter1, typename FwdIter2>
hpx::parallel::util::detail::algorithm_result_t<ExPolicy, FwdIter2> hpx::inclusive_scan(ExPolicy &&policy,
                                                                                        FwdIter1 first,
                                                                                        FwdIter1 last,
                                                                                        FwdIter2 dest)
```

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(+, *first, …, *(first + (i - result))). Executed according to the policy.

The reduce operations in the parallel *inclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *inclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op*, here std::plus<>().

Note

GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, ..., aN) is defined as:

- a1 when N is 1
- GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, ..., aK)
- GENERALIZED_NONCOMMUTATIVE_SUM(+, aM, ..., aN) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *inclusive_scan* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *inclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **InIter**, typename **OutIter**, typename **Op**>

OutIter hpx::**inclusive_scan**(*InIter* first, *InIter* last, *OutIter* dest, *Op* &&op)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, *first, ..., *(first + (i - result))).

The reduce operations in the parallel *inclusive_scan* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, ..., aN) is defined as:
>
> - a1 when N is 1
> - GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, ..., aK)
> - GENERALIZED_NONCOMMUTATIVE_SUM(+, aM, ..., aN) where 1 < K+1 = M <= N.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Op** – The type of the binary function object used for the reduction operation.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

Returns

The *inclusive_scan* algorithm returns *OutIter*. The *inclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Op**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> `hpx::inclusive_scan`(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *Op* &&op)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, *first, …, *(first + (i - result))). Executed according to the policy.

The reduce operations in the parallel *inclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *inclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum.

Note

Complexity: O(*last* - *first*) applications of the predicate *op*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, …, aN) is defined as:

- a1 when N is 1
- GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK)
- GENERALIZED_NONCOMMUTATIVE_SUM(+, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Op** – The type of the binary function object used for the reduction operation.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

- **dest** – Refers to the beginning of the destination range.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

Returns

The *inclusive_scan* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *inclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **InIter**, typename **OutIter**, typename **Op**, typename **T**>

OutIter `hpx::`**`inclusive_scan`**(*InIter* first, *InIter* last, *OutIter* dest, *Op* &&op, *T* init)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, init, *first, …, *(first + (i - result))).

The reduce operations in the parallel *inclusive_scan* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum. If *op* is not mathematically associative, the behavior of *inclusive_scan* may be non-deterministic.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:
>
> - a1 when N is 1
> - op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN)) where 1 < K+1 = M <= N.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.

- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Op** – The type of the binary function object used for the reduction operation.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.
- **init** – The initial value for the generalized sum.

Returns

The *inclusive_scan* algorithm returns *OutIter*. The *inclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Op**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> hpx::**inclusive_scan**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *Op* &&op, *T* init)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, init, *first, …, *(first + (i - result))). Executed according to the policy.

The reduce operations in the parallel *inclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *inclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum. If *op* is not mathematically associative, the behavior of *inclusive_scan* may be non-deterministic.

Note

Complexity: O(*last* - *first*) applications of the predicate *op*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, . . . , aN) is defined as:

- a1 when N is 1
- op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, . . . , aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, . . . , aN)) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Op** – The type of the binary function object used for the reduction operation.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.
- **init** – The initial value for the generalized sum.

Returns

The *inclusive_scan* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *inclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::uninitialized_value_construct

Defined in header hpx/algorithm.hpp[622].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::uninitialized_value_construct_n*

template<typename **FwdIter**>

void *hpx*::**uninitialized_value_construct**(*FwdIter* first, *FwdIter* last)

Constructs objects of type typename iterator_traits<ForwardIt> ::value_type in the uninitialized storage designated by the range by value-initialization. If an exception is thrown during the initialization, the function has no effects.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

FwdIter – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *uninitialized_value_construct* algorithm returns nothing

template<typename **ExPolicy**, typename **FwdIter**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*> *hpx*::**uninitialized_value_construct**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last)

Constructs objects of type typename iterator_traits<ForwardIt> ::value_type in the uninitialized storage designated by the range by value-initialization. If an exception is thrown during the initialization, the function has no effects. Executed according to the policy.

The assignments in the parallel *uninitialized_value_construct* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_value_construct* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to

[622] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *uninitialized_value_construct* algorithm returns a *hpx::future<void>*, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns nothing otherwise.

hpx::uninitialized_value_construct_n

Defined in header hpx/algorithm.hpp[623].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::uninitialized_value_construct*

template<typename **FwdIter**, typename **Size**>

FwdIter hpx::**uninitialized_value_construct_n**(*FwdIter* first, *Size* count)

Constructs objects of type typename iterator_traits<ForwardIt> ::value_type in the uninitialized storage designated by the range [first, first + count) by value-initialization. If an exception is thrown during the initialization, the function has no effects.

> **Note**
>
> Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

[623] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.

Returns

The *uninitialized_value_construct_n* algorithm returns a returns *FwdIter*. The *uninitialized_value_construct_n* algorithm returns the iterator to the element in the source range, one past the last element constructed.

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::uninitialized_value_construct_n`(*ExPolicy* &&policy, *FwdIter* first, *Size* count)

Constructs objects of type typename iterator_traits<ForwardIt> ::value_type in the uninitialized storage designated by the range [first, first + count) by value-initialization. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_value_construct_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_value_construct_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.

Returns

The *uninitialized_value_construct_n* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *uninitialized_value_construct_n* algorithm returns the iterator to the element in the source range, one past the last element constructed.

hpx::equal

Defined in header hpx/algorithm.hpp[624].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Pred** = detail::equal_to>

util::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::equal`(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *Pred* &&op = *Pred*())

Returns true if the range [first1, last1) is equal to the range [first2, last2), and false otherwise. Executed according to the policy.

The comparison operations in the parallel *equal* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *equal* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(min(*last1* - *first1*, *last2* - *first2*)) applications of the predicate *op*.

> **Note**
>
> The two ranges are considered equal if, for every iterator i in the range [first1,last1), *i equals *(first2 + (i - first1)). This overload of equal uses operator== to determine if two elements are equal.

[624] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

Returns

The *equal* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *equal* algorithm returns true if the elements in the two ranges are equal, otherwise it returns false. If the length of the range [first1, last1) does not equal the length of the range [first2, last2), it returns false.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**>

util::detail::algorithm_result_t<*ExPolicy*, bool> **hpx::equal**(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2)

Returns true if the range [first1, last1) is equal to the range [first2, last2), and false otherwise. Executed according to policy.

The comparison operations in the parallel *equal* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *equal* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(min(*last1* - *first1*, *last2* - *first2*)) applications of the predicate *std::equal_to*.

> **Note**
>
> The two ranges are considered equal if, for every iterator i in the range [first1,last1), *i equals *(first2 + (i - first1)). This overload of equal uses operator== to determine if two elements are equal.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.

Returns

The *equal* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *equal* algorithm returns true if the elements in the two ranges are equal, otherwise it returns false. If the length of the range [first1, last1) does not equal the length of the range [first2, last2), it returns false.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Pred** = detail::equal_to>

util::detail::algorithm_result_t<*ExPolicy*, bool> *hpx*::**equal**(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *Pred* &&op = *Pred*())

Returns true if the range [first1, last1) is equal to the range starting at first2, and false otherwise. Executed according to policy.

The comparison operations in the parallel *equal* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *equal* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: O(min(*last1* - *first1*, *last2* - *first2*)) applications of the predicate *op*.

Note

The two ranges are considered equal if, for every iterator i in the range [first1,last1), *i equals *(first2 + (i - first1)). This overload of equal uses operator== to determine if two elements are equal.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *Copy-Constructible*. This defaults to std::equal_to<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.

- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

Returns

The *equal* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *equal* algorithm returns true if the elements in the two ranges are equal, otherwise it returns false.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**>

util::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::equal`(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2)

Returns true if the range [first1, last1) is equal to the range [first2, last2), and false otherwise. Executed according to policy.

The comparison operations in the parallel *equal* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *equal* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most *last1 - first1* applications of the predicate *op*.

> **Note**
>
> The two ranges are considered equal if, for every iterator i in the range [first1,last1), *i equals *(first2 + (i - first1)). This overload of equal uses operator== to determine if two elements are equal.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.

Returns

The *equal* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *equal* algorithm returns true if the elements in the two ranges are equal, otherwise it returns false. If the length of the range [first1, last1) does not equal the length of the range [first2, last2), it returns false.

template<typename **FwdIter1**, typename **FwdIter2**, typename **Pred** = detail::equal_to>

bool *hpx*::**equal**(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *Pred* &&op = *Pred*())

Returns true if the range [first1, last1) is equal to the range [first2, last2), and false otherwise.

> **Note**
>
> Complexity: At most min(last1 - first1, last2 - first2) applications of the predicate *op*.

> **Note**
>
> The two ranges are considered equal if, for every iterator i in the range [first1,last1), *i equals *(first2 + (i - first1)). This overload of equal uses operator== to determine if two elements are equal.

Template Parameters

- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *Copy-Constructible*. This defaults to std::equal_to<>

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.

- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

Returns

The *equal* algorithm returns a *bool* . The *equal* algorithm returns true if the elements in the two ranges are equal, otherwise it returns false. If the length of the range [first1, last1) does not equal the length of the range [first2, last2), it returns false.

template<typename **FwdIter1**, typename **FwdIter2**>

bool *hpx*::**equal**(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2)

Returns true if the range [first1, last1) is equal to the range [first2, last2), and false otherwise.

> **Note**
>
> Complexity: At most min(last1 - first1, last2 - first2) applications of the predicate *std::equal_to*.

> **Note**
>
> The two ranges are considered equal if, for every iterator i in the range [first1,last1), *i equals *(first2 + (i - first1)). This overload of equal uses operator== to determine if two elements are equal.

Template Parameters

- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.

Returns

The *equal* algorithm returns a *bool* . The *equal* algorithm returns true if the elements in the two ranges are equal, otherwise it returns false. If the length of the range [first1, last1) does not equal the length of the range [first2, last2), it returns false.

template<typename **FwdIter1**, typename **FwdIter2**, typename **Pred** = detail::equal_to>

bool *hpx*::**equal**(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *Pred* &&op = *Pred*())

Returns true if the range [first1, last1) is equal to the range [first2, first2 + (last1 - first1)), and false otherwise.

Note

Complexity: At most *last1* - *first1* applications of the predicate *op*.

Note

The two ranges are considered equal if, for every iterator i in the range [first1,last1), *i equals *(first2 + (i - first1)). This overload of equal uses operator== to determine if two elements are equal.

Template Parameters

- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.

- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

Returns

The *equal* algorithm returns a *bool* . The *equal* algorithm returns true if the elements in the two ranges are equal, otherwise it returns false. If the length of the range [first1, last1) does not equal the length of the range [first2, last2), it returns false.

hpx::experimental::induction

Defined in header hpx/algorithm.hpp[625].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::induction_stride_helper<*T*> `hpx::experimental::induction`(*T* &&value, *std*::size_t stride)

The function template returns an induction object of unspecified type having a value type and encapsulating an initial *value* of that type and, optionally, a stride.

For each element in the input range, a looping algorithm over input sequence *S* computes an induction value from an induction variable and ordinal position *p* within *S* by the formula i + p * stride if a stride was specified or i + p otherwise. This induction value is passed to the element access function.

If the *value* argument to *induction* is a non-const lvalue, then that lvalue becomes the live-out object for the returned induction object. For each induction object that has a live-out object, the looping algorithm assigns the value of i + n * stride to the live-out object upon return, where *n* is the number of elements in the input range.

Template Parameters

T – The value type to be used by the induction object.

Parameters

- **value** – [in] The initial value to use for the induction object
- **stride** – [in] The (optional) stride to use for the induction object (default: 1)

Returns

This returns an induction object with value type *T*, initial *value*, and (if specified) *stride*. If *T* is a lvalue of non-const type, *value* is used as the live-out object for the induction object; otherwise there is no live-out object.

[625] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

template<typename **T**>

constexpr *hpx*::*parallel*::detail::induction_helper<*T*> `hpx::experimental::induction`(*T* &&value)

hpx::experimental::reduction

Defined in header hpx/algorithm.hpp[626].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**, typename **Op**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *std*::decay_t<*Op*>> `hpx::experimental::reduction`(*T* &var, *T* const &identity, *Op* &&combiner)

The function template returns a reduction object of unspecified type having a value type and encapsulating an identity value for the reduction, a combiner function object, and a live-out object from which the initial value is obtained and into which the final value is stored.

A parallel algorithm uses reduction objects by allocating an unspecified number of instances, called views, of the reduction's value type. Each view is initialized with the reduction object's identity value, except that the live-out object (which was initialized by the caller) comprises one of the views. The algorithm passes a reference to a view to each application of an element-access function, ensuring that no two concurrently-executing invocations share the same view. A view can be shared between two applications that do not execute concurrently, but initialization is performed only once per view.

Modifications to the view by the application of element access functions accumulate as partial results. At some point before the algorithm returns, the partial results are combined, two at a time, using the reduction object's combiner operation until a single value remains, which is then assigned back to the live-out object.

T shall meet the requirements of *CopyConstructible* and *MoveAssignable*. The expression

```
var = combiner(var, var)
```

shall be well-formed.

Note

[626] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

In order to produce useful results, modifications to the view should be limited to commutative operations closely related to the combiner operation. For example if the combiner is plus<T>, incrementing the view would be consistent with the combiner but doubling it or assigning to it would not.

Template Parameters

- **T** – The value type to be used by the induction object.
- **Op** – The type of the binary function (object) used to perform the reduction operation.

Parameters

- **var** – [in,out] The life-out value to use for the reduction object. This will hold the reduced value after the algorithm is finished executing.
- **identity** – [in] The identity value to use for the reduction operation. This argument is optional and defaults to a copy of *var*.
- **combiner** – [in] The binary function (object) used to perform a pairwise reduction on the elements.

Returns

This returns a reduction object of unspecified type having a value type of *T*. When the return value is used by an algorithm, the reference to *var* is used as the live-out object, new views are initialized to a copy of identity, and views are combined by invoking the copy of combiner, passing it the two views to be combined.

template<typename **T**, typename **Op**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *std*::decay_t<*Op*>> *hpx*::*experimental*::**reduction**(*T* &var, *Op* &&combiner)

hpx::mismatch

Defined in header hpx/algorithm.hpp[627].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Pred**>

[627] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *std*::pair<*FwdIter1*, *FwdIter2*>> `hpx::mismatch`(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *Pred* &&op)

Returns the first mismatching pair of elements from two ranges: one defined by [first1, last1) and another defined by [first2,last2). If last2 is not provided, it denotes first2 + (last1 - first1). Executed according to the policy.

The comparison operations in the parallel *mismatch* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *mismatch* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most min(last1 - first1, last2 - first2) applications of the predicate *op* or *operator==*. If *FwdIter1* and *FwdIter2* meet the requirements of *RandomAccessIterator* and (last1 - first1) != (last2 - first2) then no applications of the predicate *op* or *operator==* are made.

Note

The two ranges are considered mismatch if, for every iterator i in the range [first1,last1), *i mismatches *(first2 + (i - first1)). This overload of mismatch uses operator== to determine if two elements are mismatch.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential

form, the parallel overload of *mismatch* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as mismatch. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

Returns

The *mismatch* algorithm returns a hpx::future<*std::pair<FwdIter1,FwdIter2>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *std::pair<FwdIter1,FwdIter2>* otherwise. If no mismatches are found when the comparison reaches last1 or last2, whichever happens first, the pair holds the end iterator and the corresponding iterator from the other range.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *std*::pair<*FwdIter1*, *FwdIter2*>> **hpx::mismatch**(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2)

Returns the first mismatching pair of elements from two ranges: one defined by [first1, last1) and another defined by [first2,last2). If last2 is not provided, it denotes first2 + (last1 - first1). Executed according to the policy.

The comparison operations in the parallel *mismatch* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *mismatch* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most min(last1 - first1, last2 - first2) applications of *operator==*. If *FwdIter1* and *FwdIter2* meet the requirements of *RandomAccessIterator* and (last1 - first1) != (last2 - first2) then no applications of *operator==* are made.

> **Note**
>
> The two ranges are considered mismatch if, for every iterator i in the range [first1,last1), *i mismatches *(first2 + (i - first1)). This overload of mismatch uses operator== to determine if two elements are mismatch.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter1`** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **`FwdIter2`** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first1`** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **`last1`** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **`first2`** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **`last2`** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.

Returns

The *mismatch* algorithm returns a hpx::future<*std::pair<FwdIter1,FwdIter2*>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *std::pair<FwdIter1,FwdIter2>* otherwise. If no mismatches are found when the comparison reaches last1 or last2, whichever happens first, the pair holds the end iterator and the corresponding iterator from the other range.

template<typename **`ExPolicy`**, typename **`FwdIter1`**, typename **`FwdIter2`**, typename **`Pred`**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *std*::pair<*FwdIter1*, *FwdIter2*>> `hpx::mismatch`(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *Pred* &&op)

Returns the first mismatching pair of elements from two ranges: one defined by [first1, last1) and another defined by [first2,last2). If last2 is not provided, it denotes first2 + (last1 - first1). Executed according to the policy.

The comparison operations in the parallel *mismatch* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *mismatch* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most last1 - first1 applications of the predicate *op* or *operator*==. If *FwdIter1* and *FwdIter2* meet the requirements of *RandomAccessIterator* and (last1 - first1) != (last2 - first2) then no applications of the predicate *op* or *operator*== are made.

Note

The two ranges are considered mismatch if, for every iterator i in the range [first1,last1), *i mismatches *(first2 + (i - first1)). This overload of mismatch uses operator== to determine if two elements are mismatch.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter1`** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **`FwdIter2`** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Pred`** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *mismatch* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as mismatch. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

Returns

The *mismatch* algorithm returns a hpx::future<*std::pair<FwdIter1,FwdIter2>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *std::pair<FwdIter1,FwdIter2>* otherwise. If no mismatches are found when the comparison reaches last1 or last2, whichever happens first, the pair holds the end iterator and the corresponding iterator from the other range.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *std*::pair<*FwdIter1*, *FwdIter2*>> hpx::**mismatch**(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2)

Returns the first mismatching pair of elements from two ranges: one defined by [first1, last1) and another defined by [first2,last2). If last2 is not provided, it denotes first2 + (last1 - first1). Executed according to the policy.

The comparison operations in the parallel *mismatch* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *mismatch* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most last1 - first1 applications of *operator*==. If *FwdIter1* and *FwdIter2* meet the requirements of *RandomAccessIterator* and (last1 - first1) != (last2 - first2) then no applications of *operator*== are made.

> **Note**
>
> The two ranges are considered mismatch if, for every iterator i in the range [first1,last1), *i mismatches *(first2 + (i - first1)). This overload of mismatch uses operator== to determine if two elements are mismatch.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *mismatch* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.

Returns

The *mismatch* algorithm returns a hpx::future<*std::pair<FwdIter1,FwdIter2>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *std::pair<FwdIter1,FwdIter2>* otherwise. If no mismatches are found when the comparison reaches last1 or last2, whichever happens first, the pair holds the end iterator and the corresponding iterator from the other range.

template<typename **FwdIter1**, typename **FwdIter2**, typename **Pred**>

std::pair<*FwdIter1*, *FwdIter2*> `hpx::`**`mismatch`**(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *Pred* &&op)

Returns the first mismatching pair of elements from two ranges: one defined by [first1, last1) and another defined by [first2,last2). If last2 is not provided, it denotes first2 + (last1 - first1).

Note

Complexity: At most min(last1 - first1, last2 - first2) applications of the predicate *op* or *operator==*. If *FwdIter1* and *FwdIter2* meet the requirements of *RandomAccessIterator* and (last1 - first1) != (last2 - first2) then no applications of the predicate *op* or *operator==* are made.

Note

The two ranges are considered mismatch if, for every iterator i in the range [first1,last1), *i mismatches *(first2 + (i - first1)). This overload of mismatch uses operator== to determine if two elements are mismatch.

Template Parameters

- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *mismatch* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as mismatch. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

Returns

The *mismatch* algorithm returns a *std::pair<FwdIter1,FwdIter2>*. If no mismatches are found when the comparison reaches last1 or last2, whichever happens first, the pair holds the end iterator and the corresponding iterator from the other range.

template<typename **FwdIter1**, typename **FwdIter2**>

std::pair<*FwdIter1*, *FwdIter2*> `hpx::mismatch`(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2)

Returns the first mismatching pair of elements from two ranges: one defined by [first1, last1) and another defined by [first2,last2). If last2 is not provided, it denotes first2 + (last1 - first1).

Note

Complexity: At most min(last1 - first1, last2 - first2) applications of *operator*==. If *FwdIter1* and *FwdIter2* meet the requirements of *RandomAccessIterator* and (last1 - first1) != (last2 - first2) then no applications of *operator*== are made.

Note

The two ranges are considered mismatch if, for every iterator i in the range [first1,last1), *i mismatches *(first2 + (i - first1)). This overload of mismatch uses operator== to determine if two elements are mismatch.

Template Parameters

- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.

Returns

The *mismatch* algorithm returns a *std::pair<FwdIter1,FwdIter2>*. If no mismatches are found when the comparison reaches last1 or last2, whichever happens first, the pair holds the end iterator and the corresponding iterator from the other range.

template<typename **FwdIter1**, typename **FwdIter2**, typename **Pred**>

std::pair<*FwdIter1*, *FwdIter2*> `hpx::mismatch`(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *Pred* &&op)

Returns the first mismatching pair of elements from two ranges: one defined by [first1, last1) and another defined by [first2,last2). If last2 is not provided, it denotes first2 + (last1 - first1).

Note

Complexity: At most last1 - first1 applications of the predicate *op* or *operator*==. If *FwdIter1* and *FwdIter2* meet the requirements of *RandomAccessIterator* and (last1 - first1) != (last2 - first2) then no applications of the predicate *op* or *operator*== are made.

Note

The two ranges are considered mismatch if, for every iterator i in the range [first1,last1), *i mismatches *(first2 + (i - first1)). This overload of mismatch uses operator== to determine if two elements are mismatch.

Template Parameters

- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *mismatch* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as mismatch. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

Returns

The *mismatch* algorithm returns a *std::pair<FwdIter1,FwdIter2>*. If no mismatches are found when the comparison reaches last1 or last2, whichever happens first, the pair holds the end iterator and the corresponding iterator from the other range.

template<typename **FwdIter1**, typename **FwdIter2**>

std::pair<*FwdIter1*, *FwdIter2*> hpx::**mismatch**(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2)

Returns the first mismatching pair of elements from two ranges: one defined by [first1, last1) and another defined by [first2,last2). If last2 is not provided, it denotes first2 + (last1 - first1).

> **Note**
>
> Complexity: At most last1 - first1 applications of *operator*==. If *FwdIter1* and *FwdIter2* meet the requirements of *RandomAccessIterator* and (last1 - first1) != (last2 - first2) then no applications of *operator*== are made.

> **Note**
>
> The two ranges are considered mismatch if, for every iterator i in the range [first1,last1), *i mismatches *(first2 + (i - first1)). This overload of mismatch uses operator== to determine if two elements are mismatch.

Template Parameters

- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *mismatch* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.

Returns

The *mismatch* algorithm returns a *std::pair<FwdIter1,FwdIter2>*. If no mismatches are found when the comparison reaches last1 or last2, whichever happens first, the pair holds the end iterator and the corresponding iterator from the other range.

hpx::uninitialized_default_construct

Defined in header hpx/algorithm.hpp[628].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::uninitialized_default_construct_n*

template<typename **FwdIter**>

void *hpx*::**uninitialized_default_construct**(*FwdIter* first, *FwdIter* last)

Constructs objects of type typename iterator_traits<ForwardIt> ::value_type in the uninitialized storage designated by the range by default-initialization. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_default_construct* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

FwdIter – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *uninitialized_default_construct* algorithm returns nothing

template<typename **ExPolicy**, typename **FwdIter**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*> *hpx*::**uninitialized_default_construct**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last)

Constructs objects of type typename iterator_traits<ForwardIt> ::value_type in the uninitialized storage designated by the range by default-initialization. If an exception is thrown during the initialization, the function has no effects. Executed according to the policy.

[628] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The assignments in the parallel *uninitialized_default_construct* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_default_construct* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *uninitialized_default_construct* algorithm returns a *hpx::future<void>*, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns nothing otherwise.

hpx::uninitialized_default_construct_n

Defined in header hpx/algorithm.hpp[629].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::uninitialized_default_construct*

template<typename **FwdIter**, typename **Size**>

FwdIter hpx::**uninitialized_default_construct_n**(*FwdIter* first, *Size* count)

Constructs objects of type typename iterator_traits<ForwardIt> ::value_type in the uninitialized storage designated by the range [first, first + count) by default-initialization. If an exception is thrown during the initialization, the function has no effects.

[629] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Note

Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.

Returns

The *uninitialized_default_construct_n* algorithm returns a returns *FwdIter*. The *uninitialized_default_construct_n* algorithm returns the iterator to the element in the source range, one past the last element constructed.

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::`**`uninitialized_default_construct_n`**(*ExPolicy* &&policy, *FwdIter* first, *Size* count)

Constructs objects of type typename iterator_traits<ForwardIt> ::value_type in the uninitialized storage designated by the range [first, first + count) by default-initialization. If an exception is thrown during the initialization, the function has no effects. Executed according to the policy.

The assignments in the parallel *uninitialized_default_construct_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_default_construct_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.

Returns

The *uninitialized_default_construct_n* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *uninitialized_default_construct_n* algorithm returns the iterator to the element in the source range, one past the last element constructed.

hpx::ends_with

Defined in header hpx/algorithm.hpp[630].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter1**, typename **InIter2**, typename **Pred** = *hpx*::*parallel*::detail::equal_to>

bool *hpx*::**ends_with**(*InIter1* first1, *InIter1* last1, *InIter2* first2, *InIter2* last2, *Pred* &&pred = *Pred*())

Checks whether the second range defined by [first1, last1) matches the suffix of the first range defined by [first2, last2)

The assignments in the parallel *ends_with* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear: at most min(N1, N2) applications of the predicate.

Template Parameters

- **InIter1** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an input iterator.

[630] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **InIter2** – The type of the begin destination iterators used (deduced). This iterator type must meet the requirements of a input iterator.
- **Pred** – The binary predicate that compares the elements. This defaults to *hpx::parallel::detail::equal_to*.

Parameters

- **first1** – Refers to the beginning of the source range.
- **last1** – Refers to the end of the source range.
- **first2** – Refers to the beginning of the destination range.
- **last2** – Refers to the end of the destination range.
- **pred** – Specifies the binary predicate function (or function object) which will be invoked for comparison of the elements in the two ranges.

Returns

The *ends_with* algorithm returns *bool*. The *ends_with* algorithm returns a boolean with the value true if the second range matches the suffix of the first range, false otherwise.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Pred** = *ranges*::equal_to>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, bool>::type **hpx::ends_with**(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *Pred* &&pred = *Pred*())

Checks whether the second range defined by [first1, last1) matches the suffix of the first range defined by [first2, last2). Executed according to the policy.

The assignments in the parallel *ends_with* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *ends_with* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Linear: at most min(N1, N2) applications of the predicate.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the begin destination iterators used (deduced). This iterator type must meet the requirements of a forward iterator.

- **Pred** – The binary predicate that compares the elements. This defaults to *ranges::equal_to*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the source range.
- **last1** – Refers to the end of the source range.
- **first2** – Refers to the beginning of the destination range.
- **last2** – Refers to the end of the destination range.
- **pred** – Specifies the binary predicate function (or function object) which will be invoked for comparison of the elements in the two ranges.

Returns

The *ends_with* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *ends_with* algorithm returns a boolean with the value true if the second range matches the suffix of the first range, false otherwise.

hpx::experimental::reduction_multiplies

Defined in header hpx/algorithm.hpp[631].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *std*::multiplies<*T*>> `hpx::experimental::`**`reduction_multiplies`**`(`*T* &var)

The function template *reduction_multiplies* returns a reduction object of unspecified type having a value type and encapsulating an identity value for the reduction, it uses std::multiplies{} as its combiner function, and a live-out object from which the initial value is obtained and into which the final value is stored.

A parallel algorithm uses the *reduction_multiplies* object by allocating an unspecified number of instances, called views, of the reduction's value type. Each view is initialized with the reduction object's identity value, except that the live-out object (which was initialized by the caller) comprises one of the views. The algorithm passes a reference to a view to each application of an element-access function, ensuring that no two concurrently-executing invocations share the same view. A view can be shared between two applications that do not execute concurrently, but initialization is performed only once per view.

Modifications to the view by the application of element access functions accumulate as partial results. At some point before the algorithm returns, the partial results are combined, two at a time, using the std::multiplies{} operation until a single value remains, which is then assigned back to the live-out object.

T shall meet the requirements of *CopyConstructible* and *MoveAssignable*.

[631] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Note

In order to produce useful results, modifications to the view should be limited to commutative operations closely related to the combiner operation.

Template Parameters

- **T** – The value type to be used by the induction object.
- **Op** – The type of the binary function (object) used to perform the reduction operation.

Parameters

- **var** – [in,out] The life-out value to use for the reduction object. This will hold the reduced value after the algorithm is finished executing.
- **identity** – [in] The identity value to use for the reduction operation. This argument is optional and defaults to a copy of *var*.

Returns

This returns a reduction object of unspecified type having a value type of *T*. When the return value is used by an algorithm, the reference to *var* is used as the live-out object, new views are initialized to a copy of identity, and views are combined by invoking the copy of combiner, passing it the two views to be combined.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *std*::multiplies<*T*>> `hpx::experimental::`**`reduction_multiplies`**(*T* &var, *T* const &identity)

hpx::min_element

Defined in header hpx/algorithm.hpp[632].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::max_element*
- *hpx::minmax_element*

template<typename **FwdIter**, typename **F** = *hpx*::*parallel*::detail::less>

FwdIter `hpx::`**`min_element`**(*FwdIter* first, *FwdIter* last, *F* &&f)

Finds the smallest element in the range [first, last) using the given comparison function *f*.

[632] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The comparisons in the parallel *min_element* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Exactly *max*(N-1, 0) comparisons, where N = std::distance(first, last).

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – The binary predicate which returns true if the left argument is less than the right element. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.

Returns

The *min_element* algorithm returns *FwdIter*. The *min_element* algorithm returns the iterator to the smallest element in the range [first, last). If several elements in the range are equivalent to the smallest element, returns the iterator to the first such element. Returns last if the range is empty.

template<typename **ExPolicy**, typename **FwdIter**, typename **F** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::min_element`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *F* &&f)

Finds the smallest element in the range [first, last) using the given comparison function *f*. Executed according to the policy.

The comparisons in the parallel *min_element* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *min_element* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Exactly *max*(N-1, 0) comparisons, where N = std::distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *min_element* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – The binary predicate which returns true if the left argument is less than the right element. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.

Returns

The *min_element* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *min_element* algorithm returns the iterator to the smallest element in the range [first, last). If several elements in the range are equivalent to the smallest element, returns the iterator to the first such element. Returns last if the range is empty.

hpx::max_element

Defined in header hpx/algorithm.hpp[633].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::min_element*
- *hpx::minmax_element*

template<typename **FwdIter**, typename **F** = *hpx*::*parallel*::detail::less>

[633] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

FwdIter `hpx::max_element`(*FwdIter* first, *FwdIter* last, *F* &&f)

Finds the largest element in the range [first, last) using the given comparison function *f*.

The comparisons in the parallel *min_element* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Exactly *max*(N-1, 0) comparisons, where N = std::distance(first, last).

Template Parameters

- **`FwdIter`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **`F`** – The type of the function/function object to use (deduced).

Parameters

- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`f`** – The binary predicate which returns true if left argument is less than the right element. This argument is optional and defaults to std::less. the left argument is less than the right element. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.

Returns

The *max_element* algorithm returns *FwdIter*. The *max_element* algorithm returns the iterator to the smallest element in the range [first, last). If several elements in the range are equivalent to the smallest element, returns the iterator to the first such element. Returns last if the range is empty.

template<typename **ExPolicy**, typename **FwdIter**, typename **F** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter*>::type `hpx::max_element`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *F* &&f)

Removes all elements satisfying specific criteria from the range Finds the largest element in the range [first, last) using the given comparison function *f*. Executed according to the policy.

The comparisons in the parallel *max_element* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *max_element* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *max*(N-1, 0) comparisons, where N = std::distance(first, last).

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **`F`** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *max_element* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`f`** – The binary predicate which returns true if left argument is less than the right element. This argument is optional and defaults to std::less. the left argument is less than the right element. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.

Returns

The *max_element* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *max_element* algorithm returns the iterator to the smallest element in the range [first, last). If several elements in the range are equivalent to the smallest element, returns the iterator to the first such element. Returns last if the range is empty.

hpx::minmax_element

Defined in header hpx/algorithm.hpp[634].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::min_element*
- *hpx::max_element*

template<typename **FwdIter**, typename **F** = *hpx*::*parallel*::detail::less>

minmax_element_result<*FwdIter*> *hpx*::**minmax_element**(*FwdIter* first, *FwdIter* last, *F* &&f)

Finds the largest element in the range [first, last) using the given comparison function *f*.

The comparisons in the parallel *minmax_element* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: At most *max*(floor(3/2*(N-1)), 0) applications of the predicate, where N = std::distance(first, last).

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – The binary predicate which returns true if the left argument is less than the right element. This argument is optional and defaults to std::less. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.

Returns

The *minmax_element* algorithm returns a *minmax_element_result<FwdIter>* The *minmax_element* algorithm returns a pair consisting of an iterator to the smallest element as the min element and an iterator to the largest element as the max element. Returns

[634] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

minmax_element_result<FwdIter>{first,first} if the range is empty. If several elements are equivalent to the smallest element, the iterator to the first such element is returned. If several elements are equivalent to the largest element, the iterator to the last such element is returned.

template<typename **ExPolicy**, typename **FwdIter**, typename **F** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, minmax_element_result<*FwdIter*>> `hpx::minmax_element`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *F* &&f)

Finds the largest element in the range [first, last) using the given comparison function *f*.

The comparisons in the parallel *minmax_element* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *minmax_element* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most *max*(floor(3/2*(N-1)), 0) applications of the predicate, where N = std::distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *minmax_element* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

- **f** – The binary predicate which returns true if the left argument is less than the right element. This argument is optional and defaults to std::less. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.

Returns

The *minmax_element* algorithm returns a hpx::future<*minmax_element_result*<*FwdIter*>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *minmax_element_result*<*FwdIter*> otherwise. The *minmax_element* algorithm returns a pair consisting of an iterator to the smallest element as the min element and an iterator to the largest element as the max element. Returns *std::make_pair(first,first)* if the range is empty. If several elements are equivalent to the smallest element, the iterator to the first such element is returned. If several elements are equivalent to the largest element, the iterator to the last such element is returned.

hpx::experimental::reduction_plus

Defined in header hpx/algorithm.hpp[635].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *std*::plus<*T*>> `hpx::experimental::reduction_plus`(*T* &var)

The function template *reduction_plus* returns a reduction object of unspecified type having a value type and encapsulating an identity value for the reduction, it uses std::plus{} as its combiner function, and a live-out object from which the initial value is obtained and into which the final value is stored.

A parallel algorithm uses the *reduction_plus* object by allocating an unspecified number of instances, called views, of the reduction's value type. Each view is initialized with the reduction object's identity value, except that the live-out object (which was initialized by the caller) comprises one of the views. The algorithm passes a reference to a view to each application of an element-access function, ensuring that no two concurrently-executing invocations share the same view. A view can be shared between two applications that do not execute concurrently, but initialization is performed only once per view.

Modifications to the view by the application of element access functions accumulate as partial results. At some point before the algorithm returns, the partial results are combined, two at a time, using the std::plus{} operation until a single value remains, which is then assigned back to the live-out object.

T shall meet the requirements of *CopyConstructible* and *MoveAssignable*.

[635] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

> **Note**
>
> In order to produce useful results, modifications to the view should be limited to commutative operations closely related to the combiner operation.

Template Parameters

- **T** – The value type to be used by the induction object.
- **Op** – The type of the binary function (object) used to perform the reduction operation.

Parameters

- **var** – [in,out] The life-out value to use for the reduction object. This will hold the reduced value after the algorithm is finished executing.
- **identity** – [in] The identity value to use for the reduction operation. This argument is optional and defaults to a copy of *var*.

Returns

This returns a reduction object of unspecified type having a value type of *T*. When the return value is used by an algorithm, the reference to *var* is used as the live-out object, new views are initialized to a copy of identity, and views are combined by invoking the copy of combiner, passing it the two views to be combined.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *std*::plus<*T*>> *hpx*::*experimental*::**reduction_plus**(*T* &var, *T* const &identity)

hpx::experimental::reduction_bit_xor

Defined in header hpx/algorithm.hpp[636].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *std*::bit_xor<*T*>> *hpx*::*experimental*::**reduction_bit_xor**(*T* &var)

The function template *reduction_bit_xor* returns a reduction object of unspecified type having a value type and encapsulating an identity value for the reduction, it uses std::bit_xor{} as its combiner function, and a live-out object from which the initial value is obtained and into which the final value is stored.

[636] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

A parallel algorithm uses the *reduction_bit_xor* object by allocating an unspecified number of instances, called views, of the reduction's value type. Each view is initialized with the reduction object's identity value, except that the live-out object (which was initialized by the caller) comprises one of the views. The algorithm passes a reference to a view to each application of an element-access function, ensuring that no two concurrently-executing invocations share the same view. A view can be shared between two applications that do not execute concurrently, but initialization is performed only once per view.

Modifications to the view by the application of element access functions accumulate as partial results. At some point before the algorithm returns, the partial results are combined, two at a time, using the std::bit_xor{} operation until a single value remains, which is then assigned back to the live-out object.

T shall meet the requirements of *CopyConstructible* and *MoveAssignable*.

> **Note**
>
> In order to produce useful results, modifications to the view should be limited to commutative operations closely related to the combiner operation.

Template Parameters

- **T** – The value type to be used by the induction object.
- **Op** – The type of the binary function (object) used to perform the reduction operation.

Parameters

- **var** – [in,out] The life-out value to use for the reduction object. This will hold the reduced value after the algorithm is finished executing.
- **identity** – [in] The identity value to use for the reduction operation. This argument is optional and defaults to a copy of *var*.

Returns

This returns a reduction object of unspecified type having a value type of *T*. When the return value is used by an algorithm, the reference to *var* is used as the live-out object, new views are initialized to a copy of identity, and views are combined by invoking the copy of combiner, passing it the two views to be combined.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *std*::bit_xor<*T*>> *hpx*::*experimental*::**reduction_bit_xor**(*T* &var, *T* const &identity)

hpx::adjacent_find

Defined in header hpx/algorithm.hpp[637].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter**, typename **Pred** = *hpx*::*parallel*::detail::equal_to>

InIter `hpx::adjacent_find`(*InIter* first, *InIter* last, *Pred* &&pred = *Pred*())

Searches the range [first, last) for two consecutive identical elements.

> **Note**
>
> Complexity: Exactly the smaller of (*result* - *first*) + 1 and (*last* - *first*) - 1 application of the predicate where *result* is the value returned

Template Parameters

- **InIter** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an input iterator.
- **Pred** – The type of an optional function/function object to use.

Parameters

- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **pred** – The binary predicate which returns *true* if the elements should be treated as equal. The signature should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1* .

Returns

The *adjacent_find* algorithm returns an iterator to the first of the identical elements. If no such elements are found, *last* is returned.

template<typename **ExPolicy**, typename **FwdIter**, typename **Pred** = *hpx*::*parallel*::detail::equal_to>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::adjacent_find`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *Pred* &&pred = *Pred*())

Searches the range [first, last) for two consecutive identical elements. This version uses the given binary predicate pred

637 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The comparison operations in the parallel *adjacent_find* invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *adjacent_find* invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

This overload of *adjacent_find* is available if the user decides to provide their algorithm their own binary predicate *pred*.

> **Note**
>
> Complexity: Exactly the smaller of (*result* - *first*) + 1 and (*last* - *first*) - 1 application of the predicate where *result* is the value returned

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter`** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of a forward iterator.
- **`Pred`** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *adjacent_find* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **`pred`** – The binary predicate which returns *true* if the elements should be treated as equal. The signature should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1* .

Returns

The *adjacent_find* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *adjacent_find* algorithm returns an iterator to the first of the identical elements. If no such elements are found, *last* is returned.

hpx::shift_right

Defined in header hpx/algorithm.hpp[638].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **FwdIter**, typename **Size**>

FwdIter `hpx::shift_right`(*FwdIter* first, *FwdIter* last, *Size* n)

Shifts the elements in the range [first, last) by n positions towards the end of the range. For every integer i in [0, last - first - n), moves the element originally at position first + i to position first

- n + i.

The assignment operations in the parallel *shift_right* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: At most (last - first) - n assignments.

> **Note**
>
> The type of dereferenced *FwdIter* must meet the requirements of *MoveAssignable*.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of positions to shift by.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **n** – Refers to the number of positions to shift.

Returns

The *shift_right* algorithm returns *FwdIter*. The *shift_right* algorithm returns an iterator to the beginning of the resulting range. Specifically: If *n* is 0 or *n* >= (last - first), does nothing and returns *first* and *last* respectively. Otherwise returns *first* + n.

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**>

[638] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter*> **hpx::shift_right**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *Size* n)

Shifts the elements in the range [first, last) by n positions towards the end of the range. For every integer i in [0, last - first - n), moves the element originally at position first + i to position first

- n + i. Executed according to the policy.

The assignment operations in the parallel *shift_right* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignment operations in the parallel *shift_right* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most (last - first) - n assignments.

Note

The type of dereferenced *FwdIter* must meet the requirements of *MoveAssignable*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of positions to shift by.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **n** – Refers to the number of positions to shift.

Returns

The *shift_right* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *shift_right* algorithm returns an iterator to the beginning of the resulting range. Specifically: If *n* is 0, does nothing and returns *first*. If *n* >= (last - first), does nothing and returns *last*. Otherwise returns *first* + n.

hpx::generate

Defined in header hpx/algorithm.hpp[639].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::generate_n*

template<typename **ExPolicy**, typename **FwdIter**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::generate`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *F* &&f)

Assign each element in range [*first*, *last*) a value generated by the given function object *f*. Executed according to the policy.

The assignments in the parallel *generate* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *generate* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *distance(first, last)* invocations of *f* and assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – generator function that will be called. signature of function should be equivalent to the following:

[639] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

```
Ret fun();
```

The type *Ret* must be such that an object of type *FwdIter* can be dereferenced and assigned a value of type *Ret*.

Returns

The *generate* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise.

template<typename **FwdIter**, typename **F**>

void *hpx*::**generate**(*FwdIter* first, *FwdIter* last, *F* &&f)

Assign each element in range [*first*, *last*) a value generated by the given function object *f*.

> **Note**
>
> Complexity: Exactly *distance(first, last)* invocations of *f* and assignments.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – generator function that will be called. signature of function should be equivalent to the following:

```
Ret fun();
```

The type *Ret* must be such that an object of type *FwdIter* can be dereferenced and assigned a value of type *Ret*.

Returns

The *generate* algorithm returns void.

hpx::generate_n

Defined in header hpx/algorithm.hpp[640].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::generate*

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::generate_n`(*ExPolicy* &&policy, *FwdIter* first, *Size* count, *F* &&f)

Assigns each element in range [*first*, *first+)* a value generated by the given function object *f*. Executed according to the policy.

The assignments in the parallel *generate_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *generate_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *count* invocations of *f* and assignments, for count > 0.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Size** – The type of a non-negative integral value specifying the number of items to iterate over.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements in the sequence the algorithm will be applied to.

[640] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **f** – Refers to the generator function object that will be called. The signature of the function should be equivalent to

  ```
  Ret fun();
  ```

  The type *Ret* must be such that an object of type *OutputIt* can be dereferenced and assigned a value of type *Ret*.

Returns
The *generate_n* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. *generate_n* algorithm returns iterator one past the last element assigned if count>0, first otherwise.

template<typename **FwdIter**, typename **Size**, typename **F**>

FwdIter hpx::**generate_n**(*FwdIter* first, *Size* count, *F* &&f)

Assigns each element in range [*first*, *first+)* a value generated by the given function object *f*.

> **Note**
>
> Complexity: Exactly *count* invocations of *f* and assignments, for count > 0.

Template Parameters

- **Size** – The type of a non-negative integral value specifying the number of items to iterate over.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements in the sequence the algorithm will be applied to.
- **f** – Refers to the generator function object that will be called. The signature of the function should be equivalent to

  ```
  Ret fun();
  ```

  The type *Ret* must be such that an object of type *OutputIt* can be dereferenced and assigned a value of type *Ret*.

Returns
The *generate_n* algorithm returns a *FwdIter*. *generate_n* algorithm returns iterator one past the last element assigned if count>0, first otherwise.

hpx/parallel/algorithms/iota.hpp

Defined in header hpx/parallel/algorithms/iota.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

template<typename **FwdIter**, typename **Sent**, typename **T**>

void **iota**(*FwdIter* first, *Sent* last, *T* value)

Fills the range [first, last) with sequentially increasing values, starting with *value* and repetitively evaluating ++`value`.

The assignments in the *iota* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Exactly *std::distance(first, last)* assignments and increments.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input or output iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **T** – The type of the value to be assigned (deduced). This type must be weakly incrementable.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **value** – The value to be stored in the first element.

Returns

The *iota* algorithm returns *void*.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*> **iota**(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *T* value)

Fills the range [first, last) with sequentially increasing values, starting with *value* and repetitively evaluating ++`value`.

The assignments in the parallel *iota* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *iota* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *std::distance(first, last)* assignments and increments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **T** – The type of the value to be assigned (deduced). This type must be weakly incrementable.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **value** – The value to be stored in the first element.

Returns

The *iota* algorithm returns a *hpx::future<void>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *void* otherwise.

hpx::transform_inclusive_scan

Defined in header hpx/algorithm.hpp[641].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter**, typename **OutIter**, typename **BinOp**, typename **UnOp**>

OutIter hpx::**transform_inclusive_scan**(*InIter* first, *InIter* last, *OutIter* dest, *BinOp* &&binary_op, *UnOp* &&unary_op)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, conv(*first), …, conv(*(first + (i - result)))).

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Neither *binary_op* nor *unary_op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The difference between *inclusive_scan* and *transform_inclusive_scan* is that *transform_inclusive_scan* includes the ith input element in the ith sum.

> **Note**
>
> Complexity: O(last - first) applications of each of *binary_op* and *unary_op*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:
>
> - a1 when N is 1
> - op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN)) where 1 < K+1 = M <= N.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **BinOp** – The type of *binary_op*.
- **UnOp** – The type of *unary_op*.

Parameters

[641] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `last` – Refers to the end of the sequence of elements the algorithm will be applied to.
- `dest` – Refers to the beginning of the destination range.
- `binary_op` – Binary *FunctionObject* that will be applied in to the result of *unary_op*, the results of other *binary_op*, and *init* if provided.
- `unary_op` – Unary *FunctionObject* that will be applied to each element of the input range. The return type must be acceptable as input to *binary_op*.

Returns

The *transform_inclusive_scan* algorithm returns a returns *OutIter*. The *transform_inclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **BinOp**, typename **UnOp**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter2*>::type `hpx::transform_inclusive_scan`(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *BinOp* &&binary_op, *UnOp* &&unary_op)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, conv(*first), …, conv(*(first + (i - result)))). Executed according to the policy.

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Neither *binary_op* nor *unary_op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The difference between *inclusive_scan* and *transform_inclusive_scan* is that *transform_inclusive_scan* includes the ith input element in the ith sum.

> **Note**
>
> Complexity: O(last - first) applications of each of *binary_op* and *unary_op*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:

- a1 when N is 1
- op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN)) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **BinOp** – The type of *binary_op*.
- **UnOp** – The type of *unary_op*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **binary_op** – Binary *FunctionObject* that will be applied in to the result of *unary_op*, the results of other *binary_op*, and *init* if provided.
- **unnary_op** – Unary *FunctionObject* that will be applied to each element of the input range. The return type must be acceptable as input to *binary_op*.

Returns

The *transform_inclusive_scan* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *transform_inclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **InIter**, typename **OutIter**, typename **BinOp**, typename **UnOp**, typename **T** = typename *std*::iterator_traits<*InIter*>::value_type>

OutIter hpx::**transform_inclusive_scan**(*InIter* first, *InIter* last, *OutIter* dest, *BinOp* &&binary_op, *UnOp* &&unary_op, *T* init)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, init, conv(*first), …, conv(*(first + (i - result)))).

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Neither *binary_op* nor *unary_op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The difference between *inclusive_scan* and *transform_inclusive_scan* is that *transform_inclusive_scan* includes the ith input element in the ith sum. If *binary_op* is not mathematically associative, the behavior of *transform_inclusive_scan* may be non-deterministic.

Note

Complexity: O(last - first) applications of each of *binary_op* and *unary_op*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:

- a1 when N is 1
- op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN)) where 1 < K+1 = M <= N.

Template Parameters

- **`InIter`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **`OutIter`** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **`BinOp`** – The type of *binary_op*.
- **`UnOp`** – The type of *unary_op*.
- **`T`** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`dest`** – Refers to the beginning of the destination range.
- **`binary_op`** – Binary *FunctionObject* that will be applied in to the result of *unary_op*, the results of other *binary_op*, and *init* if provided.
- **`unnary_op`** – Unary *FunctionObject* that will be applied to each element of the input range. The return type must be acceptable as input to *binary_op*.
- **`init`** – The initial value for the generalized sum.

Returns

The *transform_inclusive_scan* algorithm returns a returns *OutIter*. The *transform_inclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **BinOp**, typename **UnOp**, typename **T** = typename *std*::iterator_traits<*FwdIter1*>::value_type>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter2*>::type `hpx::transform_inclusive_scan`(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *BinOp* &&binary_op, *UnOp* &&unary_op, *T* init)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, init, conv(*first), …, conv(*(first + (i - result)))). Executed according to the policy.

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Neither *binary_op* nor *unary_op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The difference between *inclusive_scan* and *transform_inclusive_scan* is that *transform_inclusive_scan* includes the ith input element in the ith sum. If *binary_op* is not mathematically associative, the behavior of *transform_inclusive_scan* may be non-deterministic.

Note

Complexity: O(last - first) applications of each of *binary_op* and *unary_op*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:

- a1 when N is 1

- op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, ..., aN)) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **BinOp** – The type of *binary_op*.
- **UnOp** – The type of *unary_op*.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **binary_op** – Binary *FunctionObject* that will be applied in to the result of *unary_op*, the results of other *binary_op*, and *init* if provided.
- **unnary_op** – Unary *FunctionObject* that will be applied to each element of the input range. The return type must be acceptable as input to *binary_op*.
- **init** – The initial value for the generalized sum.

Returns

The *transform_inclusive_scan* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *transform_inclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::includes

Defined in header hpx/algorithm.hpp[642].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Pred** = *hpx*::*parallel*::detail::less>

[642] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool>::type `hpx::includes`(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *Pred* &&op = *Pred*())

Returns true if every element from the sorted range [*first2*, *last2*) is found within the sorted range [*first1*, *last1*). Also returns true if [*first2*, *last2*) is empty. The version expects both ranges to be sorted with the user supplied binary predicate *f*. Executed according to the policy.

The comparison operations in the parallel *includes* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *includes* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> At most 2*(N1+N2-1) comparisons, where N1 = std::distance(first1, last1) and N2 = std::distance(first2, last2).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *includes* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.

- **op** – The binary predicate which returns true if the elements should be treated as includes. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

Returns

The *includes* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *includes* algorithm returns true every element from the sorted range [*first2*, *last2*) is found within the sorted range [*first1*, *last1*). Also returns true if [*first2*, *last2*) is empty.

template<typename **FwdIter1**, typename **FwdIter2**, typename **Pred** = *hpx*::*parallel*::detail::less>

bool *hpx*::**includes**(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *Pred* &&op = *Pred*())

Returns true if every element from the sorted range [*first2*, *last2*) is found within the sorted range [*first1*, *last1*). Also returns true if [*first2*, *last2*) is empty. The version expects both ranges to be sorted with the user supplied binary predicate *f*.

> **Note**
>
> At most 2*(N1+N2-1) comparisons, where N1 = std::distance(first1, last1) and N2 = std::distance(first2, last2).

Template Parameters

- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *includes* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.

- **op** – The binary predicate which returns true if the elements should be treated as includes. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

Returns

The *includes* algorithm returns a *bool*. The *includes* algorithm returns true every element from the sorted range [*first2*, *last2*) is found within the sorted range [*first1*, *last1*). Also returns true if [*first2*, *last2*) is empty.

hpx::partial_sort

Defined in header hpx/algorithm.hpp[643].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **RandIter**, typename **Comp** = *hpx*::*parallel*::detail::less>

RandIter `hpx::partial_sort`(*RandIter* first, *RandIter* middle, *RandIter* last, *Comp* &&comp = *Comp*())

Places the first middle - first elements from the range [first, last) as sorted with respect to comp into the range [first, middle). The rest of the elements in the range [middle, last) are placed in an unspecified order.

> **Note**
>
> Complexity: Approximately (last - first) * log(middle - first) comparisons.

Template Parameters

- **RandIter** – The type of the source begin, middle, and end iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **Comp** – The type of the function/function object to use (deduced). Comp defaults to detail::less.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **middle** – Refers to the middle of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first

[643] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator. It defaults to detail::less.

Returns

The *partial_sort* algorithm returns a *RandIter* that refers to *last*.

template<typename **ExPolicy**, typename **RandIter**, typename **Comp** = *hpx*::*parallel*::detail::less>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *RandIter*> **hpx::partial_sort**(*ExPolicy* &&policy, *RandIter* first, *RandIter* middle, *RandIter* last, *Comp* &&comp = *Comp*())

Places the first middle - first elements from the range [first, last) as sorted with respect to comp into the range [first, middle). The rest of the elements in the range [middle, last) are placed in an unspecified order.

> **Note**
>
> Complexity: Approximately (last - first) * log(middle - first) comparisons.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **RandIter** – The type of the source begin, middle, and end iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **Comp** – The type of the function/function object to use (deduced). Comp defaults to detail::less.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **middle** – Refers to the middle of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator. It defaults to detail::less.

Returns

The *partial_sort* algorithm returns a *hpx::future<RandIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *RandIter* otherwise. The iterator returned refers to *last*.

hpx::adjacent_difference

Defined in header hpx/algorithm.hpp[644].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **FwdIter1**, typename **FwdIter2**>

FwdIter2 `hpx::adjacent_difference`(*FwdIter1* first, *FwdIter1* last, *FwdIter2* dest)

Assigns each value in the range given by result its corresponding element in the range [first, last] and the one preceding it except *result, which is assigned *first.

> **Note**
>
> Complexity: Exactly (last - first) - 1 application of the binary operator and (last - first) assignments.

Template Parameters

- **FwdIter1** – The type of the source iterators used for the input range (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators used for the output range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **dest** – Refers to the beginning of the sequence of elements the results will be assigned to.

Returns

The *adjacent_difference* algorithm returns a *FwdIter2*. The *adjacent_difference* algorithm returns an iterator to the element past the last element written.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> `hpx::adjacent_difference`(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest)

Assigns each value in the range given by result its corresponding element in the range [first, last] and the one preceding it except *result, which is assigned *first. Executed according to the policy.

[644] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The difference operations in the parallel *adjacent_difference* invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The difference operations in the parallel *adjacent_difference* invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly (last - first) - 1 application of the binary operator and (last - first) assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used for the input range (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators used for the output range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **dest** – Refers to the beginning of the sequence of elements the results will be assigned to.

Returns

The *adjacent_difference* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *adjacent_difference* algorithm returns an iterator to the element past the last element written.

template<typename **FwdIter1**, typename **FwdIter2**, typename **Op**>

FwdIter2 hpx::**adjacent_difference**(*FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *Op* &&op)

Assigns each value in the range given by result its corresponding element in the range [first, last] and the one preceding it except *result, which is assigned *first

> **Note**
>
> Complexity: Exactly (last - first) - 1 application of the binary operator and (last - first) assignments.

Template Parameters

- **FwdIter1** – The type of the source iterators used for the input range (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators used for the output range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Op** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *adjacent_difference* requires *Op* to meet the requirements of *CopyConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **dest** – Refers to the beginning of the sequence of elements the results will be assigned to.
- **op** – The binary operator which returns the difference of elements. The signature should be equivalent to the following:

  ```
  bool op(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* must be such that objects of type *FwdIter1* can be dereferenced and then implicitly converted to the dereferenced type of *dest*.

Returns

The *adjacent_difference* algorithm returns *FwdIter2*. The *adjacent_difference* algorithm returns an iterator to the element past the last element written.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Op**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> **hpx::adjacent_difference**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *Op* &&op)

Assigns each value in the range given by result its corresponding element in the range [first, last] and the one preceding it except *result, which is assigned *first

The difference operations in the parallel *adjacent_difference* invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The difference operations in the parallel *adjacent_difference* invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Exactly (last - first) - 1 application of the binary operator and (last - first) assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used for the input range (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators used for the output range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Op** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *adjacent_difference* requires *Op* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **dest** – Refers to the beginning of the sequence of elements the results will be assigned to.
- **op** – The binary operator which returns the difference of elements. The signature should be equivalent to the following:

```
bool op(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* must be such that objects of type *FwdIter1* can be dereferenced and then implicitly converted to the dereferenced type of *dest*.

Returns

The *adjacent_difference* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *adjacent_difference* algorithm returns an iterator to the element past the last element written.

hpx::copy

Defined in header hpx/algorithm.hpp[645].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::copy_n*
- *hpx::copy_if*

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> **hpx::copy**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest)

Copies the elements in the range, defined by [first, last), to another range beginning at *dest*. Executed according to the policy.

The assignments in the parallel *copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Performs exactly *last - first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

[645] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Returns

The *copy* algorithm returns a *hpx::future<FwdIter2> >* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2>* otherwise. The *copy* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

template<typename **FwdIter1**, typename **FwdIter2**>

FwdIter2 `hpx::copy`(*FwdIter1* first, *FwdIter1* last, *FwdIter2* dest)

Copies the elements in the range, defined by [first, last), to another range beginning at *dest*.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- `FwdIter1` – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- `FwdIter2` – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `last` – Refers to the end of the sequence of elements the algorithm will be applied to.
- `dest` – Refers to the beginning of the destination range.

Returns

The *copy* algorithm returns a *FwdIter2* . The *copy* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

hpx::copy_n

Defined in header hpx/algorithm.hpp[646].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::copy*
- *hpx::copy_if*

template<typename **ExPolicy**, typename **FwdIter1**, typename **Size**, typename **FwdIter2**>

[646] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> `hpx::copy_n`(*ExPolicy* &&policy, *FwdIter1* first, *Size* count, *FwdIter2* dest)

Copies the elements in the range [first, first + count), starting from first and proceeding to first + count - 1., to another range beginning at dest. Executed according to the policy.

The assignments in the parallel *copy_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *copy_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *copy_n* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *copy_n* algorithm returns Iterator in the destination range, pointing past the last element copied if count>0 or result otherwise.

template<typename **FwdIter1**, typename **Size**, typename **FwdIter2**>

FwdIter2 `hpx::copy_n`(*FwdIter1* first, *Size* count, *FwdIter2* dest)

Copies the elements in the range [first, first + count), starting from first and proceeding to first + count - 1., to another range beginning at dest.

> **Note**
>
> Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *copy_n* algorithm returns a *FwdIter2* . The *copy_n* algorithm returns Iterator in the destination range, pointing past the last element copied if count>0 or result otherwise.

hpx::copy_if

Defined in header hpx/algorithm.hpp[647].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::copy*
- *hpx::copy_n*

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Pred**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> **hpx::copy_if**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *Pred* &&pred)

Copies the elements in the range, defined by [first, last), to another range beginning at *dest*. Copies only the elements for which the predicate *f* returns true. The order of the elements that are not removed is preserved. Executed according to the policy.

[647] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The assignments in the parallel *copy_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the predicate *f*.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter1`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **`FwdIter2`** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Pred`** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *copy_if* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`dest`** – Refers to the beginning of the destination range.
- **`pred`** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type.

Returns

The *copy_if* algorithm returns a *hpx::future<FwdIter2> >* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *copy_if* algorithm returns output iterator to the element in the destination range, one past the last element copied.

template<typename **FwdIter1**, typename **FwdIter2**, typename **Pred**>

FwdIter2 `hpx::copy_if`(*FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *Pred* &&pred)

Copies the elements in the range, defined by [first, last), to another range beginning at *dest*. Copies only the elements for which the predicate *f* returns true. The order of the elements that are not removed is preserved.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the predicate *f*.

Template Parameters

- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *copy_if* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type.

Returns

The *copy_if* algorithm returns a *FwdIter2* . The *copy_if* algorithm returns output iterator to the element in the destination range, one past the last element copied.

hpx::experimental::reduction_bit_and

Defined in header hpx/algorithm.hpp[648].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *std*::bit_and<*T*>> `hpx::experimental::`**`reduction_bit_and`**(*T* &var)

The function template *reduction_bit_and* returns a reduction object of unspecified type having a value type and encapsulating an identity value for the reduction, it uses std::bit_and{} as its combiner function, and a live-out object from which the initial value is obtained and into which the final value is stored.

A parallel algorithm uses the *reduction_bit_and* object by allocating an unspecified number of instances, called views, of the reduction's value type. Each view is initialized with the reduction object's identity value, except that the live-out object (which was initialized by the caller) comprises one of the views. The algorithm passes a reference to a view to each application of an element-access function, ensuring that no two concurrently-executing invocations share the same view. A view can be shared between two applications that do not execute concurrently, but initialization is performed only once per view.

Modifications to the view by the application of element access functions accumulate as partial results. At some point before the algorithm returns, the partial results are combined, two at a time, using the std::bit_and{} operation until a single value remains, which is then assigned back to the live-out object.

T shall meet the requirements of *CopyConstructible* and *MoveAssignable*.

> **Note**
>
> In order to produce useful results, modifications to the view should be limited to commutative operations closely related to the combiner operation.

Template Parameters

- **T** – The value type to be used by the induction object.
- **Op** – The type of the binary function (object) used to perform the reduction operation.

Parameters

- **var** – [in,out] The life-out value to use for the reduction object. This will hold the reduced value after the algorithm is finished executing.
- **identity** – [in] The identity value to use for the reduction operation. This argument is optional and defaults to a copy of *var*.

Returns

This returns a reduction object of unspecified type having a value type of *T*. When the

[648] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

return value is used by an algorithm, the reference to *var* is used as the live-out object, new views are initialized to a copy of identity, and views are combined by invoking the copy of combiner, passing it the two views to be combined.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *std*::bit_and<*T*>> `hpx::experimental::reduction_bit_and`(*T* &var, *T* const &identity)

hpx::is_sorted

Defined in header hpx/algorithm.hpp[649].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::is_sorted_until*

template<typename **FwdIter**, typename **Pred** = *hpx*::*parallel*::detail::less>

bool `hpx::is_sorted`(*FwdIter* first, *FwdIter* last, *Pred* &&pred = *Pred*())

Determines if the range [first, last) is sorted. Uses pred to compare elements.

The comparison operations in the parallel *is_sorted* algorithm executes in sequential order in the calling thread.

> **Note**
>
> Complexity: at most (N+S-1) comparisons where *N* = distance(first, last). *S* = number of partitions

Template Parameters

- **FwdIter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of an optional function/function object to use.

Parameters

- **first** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.

[649] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **`last`** – Refers to the end of the sequence of elements of that the algorithm will be applied to.
- **`pred`** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second argument. The signature of the function should be equivalent to

  ```
  bool pred(const Type &a, const Type &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.

Returns

The *is_sorted* algorithm returns a *bool*. The *is_sorted* algorithm returns true if each element in the sequence [first, last) satisfies the predicate passed. If the range [first, last) contains less than two elements, the function always returns true.

template<typename **ExPolicy**, typename **FwdIter**, typename **Pred** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::`**`is_sorted`**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *Pred* &&pred = *Pred*())

Determines if the range [first, last) is sorted. Uses pred to compare elements. Executed according to the policy.

The comparison operations in the parallel *is_sorted* algorithm invoked with an execution policy object of type *sequenced_policy* executes in sequential order in the calling thread.

The comparison operations in the parallel *is_sorted* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (N+S-1) comparisons where *N* = distance(first, last). *S* = number of partitions

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter`** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.
- **`Pred`** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *is_sorted* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of that the algorithm will be applied to.
- **pred** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second argument. The signature of the function should be equivalent to

```
bool pred(const Type &a, const Type &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.

Returns

The *is_sorted* algorithm returns a *hpx::future<bool>* if the execution policy is of type *task_execution_policy* and returns *bool* otherwise. The *is_sorted* algorithm returns a bool if each element in the sequence [first, last) satisfies the predicate passed. If the range [first, last) contains less than two elements, the function always returns true.

hpx::is_sorted_until

Defined in header hpx/algorithm.hpp[650].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::is_sorted*

template<typename **FwdIter**, typename **Pred** = *hpx*::*parallel*::detail::less>

FwdIter hpx::**is_sorted_until**(*FwdIter* first, *FwdIter* last, *Pred* &&pred = *Pred*())

Returns the first element in the range [first, last) that is not sorted. Uses a predicate to compare elements or the less than operator.

The comparison operations in the parallel *is_sorted_until* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: at most (N+S-1) comparisons where N = distance(first, last). S = number of partitions

Template Parameters

- **FwdIter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.

[650] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **Pred** – The type of an optional function/function object to use.

Parameters

- **first** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of that the algorithm will be applied to.
- **pred** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second argument. The signature of the function should be equivalent to

```
bool pred(const Type &a, const Type &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.

Returns

The *is_sorted_until* algorithm returns a *FwdIter*. The *is_sorted_until* algorithm returns the first unsorted element. If the sequence has less than two elements or the sequence is sorted, last is returned.

template<typename **ExPolicy**, typename **FwdIter**, typename **Pred** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter*>::type `hpx::is_sorted_until`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *Pred* &&pred = *Pred*())

Returns the first element in the range [first, last) that is not sorted. Uses a predicate to compare elements or the less than operator. Executed according to the policy.

The comparison operations in the parallel *is_sorted_until* algorithm invoked with an execution policy object of type *sequenced_policy* executes in sequential order in the calling thread.

The comparison operations in the parallel *is_sorted_until* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: at most (N+S-1) comparisons where N = distance(first, last). S = number of partitions

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *is_sorted_until* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of that the algorithm will be applied to.
- **pred** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second argument. The signature of the function should be equivalent to

```
bool pred(const Type &a, const Type &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.

Returns

The *is_sorted_until* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *task_execution_policy* and returns *FwdIter* otherwise. The *is_sorted_until* algorithm returns the first unsorted element. If the sequence has less than two elements or the sequence is sorted, last is returned.

hpx::make_heap

Defined in header hpx/algorithm.hpp[651].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **RndIter**, typename **Comp**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*> **hpx::make_heap**(*ExPolicy* &&policy, *RndIter* first, *RndIter* last, *Comp* &&comp)

Constructs a *max heap* in the range [first, last). Executed according to the policy.

The predicate operations in the parallel *make_heap* algorithm invoked with an execution policy object of type *sequential_execution_policy* executes in sequential order in the calling thread.

The comparison operations in the parallel *make_heap* algorithm invoked with an execution policy object of type *parallel_execution_policy* or *parallel_task_execution_policy*

[651] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (3*N) comparisons where *N* = distance(first, last).

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`RndIter`** – The type of the source iterators used for algorithm. This iterator must meet the requirements for a random access iterator.
- **`Comp`** – Comparison function object (i.e. an object that satisfies the requirements of Compare) which returns true if the first argument is less than the second. The signature of the comparison function should be equivalent to the following:

  ```
  bool cmp(const Type1 &a, const Type2 &b);
  ```

  While the signature does not need to have const &, the function must not modify the objects passed to it and must be able to accept all values of type (possibly *const*) *Type1* and *Type2* regardless of value category (thus, *Type1* & is not allowed, nor is *Type1* unless for *Type1* a move is equivalent to a copy. The types *Type1* and *Type2* must be such that an object of type *RandomIt* can be dereferenced and then implicitly converted to both of them.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements of that the algorithm will be applied to.
- **`comp`** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second. The signature of the function should be equivalent to

  ```
  bool comp(const Type &a, const Type &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *RndIter* can be dereferenced and then implicitly converted to Type.

Returns

The *make_heap* algorithm returns a *hpx::future<void>* if the execution policy is of type *task_execution_policy* and returns *void* otherwise.

template<typename **`ExPolicy`**, typename **`RndIter`**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*> `hpx::make_heap`(*ExPolicy* &&policy, *RndIter* first, *RndIter* last)

Constructs a *max heap* in the range [first, last). Uses the operator < for comparisons. Executed according to the policy.

The predicate operations in the parallel *make_heap* algorithm invoked with an execution policy object of type *sequential_execution_policy* executes in sequential order in the calling thread.

The comparison operations in the parallel *make_heap* algorithm invoked with an execution policy object of type *parallel_execution_policy* or *parallel_task_execution_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (3*N) comparisons where *N* = distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **RndIter** – The type of the source iterators used for algorithm. This iterator must meet the requirements for a random access iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of that the algorithm will be applied to.

Returns

The *make_heap* algorithm returns a *hpx::future<void>* if the execution policy is of type *task_execution_policy* and returns *void* otherwise.

template<typename **RndIter**, typename **Comp**>

void `hpx::make_heap`(*RndIter* first, *RndIter* last, *Comp* &&comp)

Constructs a *max heap* in the range [first, last).

> **Note**
>
> Complexity: at most (3*N) comparisons where *N* = distance(first, last).

Template Parameters

- `RndIter` – The type of the source iterators used for algorithm. This iterator must meet the requirements for a random access iterator.
- `Comp` – Comparison function object (i.e. an object that satisfies the requirements of Compare) which returns true if the first argument is less than the second. The signature of the comparison function should be equivalent to the following:

  ```
  bool cmp(const Type1 &a, const Type2 &b);
  ```

  While the signature does not need to have const &, the function must not modify the objects passed to it and must be able to accept all values of type (possibly *const*) *Type1* and *Type2* regardless of value category (thus, *Type1* & is not allowed, nor is *Type1* unless for *Type1* a move is equivalent to a copy. The types *Type1* and *Type2* must be such that an object of type *RandomIt* can be dereferenced and then implicitly converted to both of them.

Parameters

- `first` – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- `last` – Refers to the end of the sequence of elements of that the algorithm will be applied to.
- `comp` – Refers to the binary predicate which returns true if the first argument should be treated as less than the second. The signature of the function should be equivalent to

  ```
  bool comp(const Type &a, const Type &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *RndIter* can be dereferenced and then implicitly converted to Type.

Returns

The *make_heap* algorithm returns a *void*.

template<typename **RndIter**>

void *hpx*::**make_heap**(*RndIter* first, *RndIter* last)

Constructs a *max heap* in the range [first, last).

> **Note**
>
> Complexity: at most (3*N) comparisons where *N* = distance(first, last).

Template Parameters

`RndIter` – The type of the source iterators used for algorithm. This iterator must meet the requirements for a random access iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of that the algorithm will be applied to.

Returns

The *make_heap* algorithm returns a *void*.

hpx/parallel/algorithms/transform_reduce_binary.hpp

Defined in header hpx/parallel/algorithms/transform_reduce_binary.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::set_union

Defined in header hpx/algorithm.hpp[652].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **FwdIter3**, typename **Pred** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter3*> **hpx::set_union**(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *FwdIter3* dest, *Pred* &&op = *Pred*())

Constructs a sorted range beginning at dest consisting of all elements present in one or both sorted ranges [first1, last1) and [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *pred*. Executed according to the policy.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), then all *m* elements will be copied from [first1, last1) to dest, preserving order, and then exactly std::max(n-m, 0) elements will be copied from [first2, last2) to dest, also preserving order.

The resulting range cannot overlap with either of the input ranges.

The application of function objects in parallel algorithm invoked with a sequential execution policy object execute in sequential order in the calling thread (*sequenced_policy*) or in a single new thread spawned from the current thread (for *sequenced_task_policy*).

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

[652] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Note

Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **FwdIter1** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **FwdIter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator or output iterator and sequential execution.
- **Op** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_union* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*

Returns

The *set_union* algorithm returns a *hpx::future<FwdIter3>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter3* otherwise. The *set_union* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **FwdIter1**, typename **FwdIter2**, typename **FwdIter3**, typename **Pred** = *hpx*::*parallel*::detail::less>

FwdIter3 `hpx::set_union`(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *FwdIter3* dest, *Pred* &&op = *Pred*())

Constructs a sorted range beginning at dest consisting of all elements present in one or both sorted ranges [first1, last1) and [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *pred*. Executed according to the policy.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), then all *m* elements will be copied from [first1, last1) to dest, preserving order, and then exactly std::max(n-m, 0) elements will be copied from [first2, last2) to dest, also preserving order.

The resulting range cannot overlap with either of the input ranges.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **FwdIter1** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **FwdIter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator or output iterator and sequential execution.
- **Op** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_union* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*

Returns

The *set_union* algorithm returns a *FwdIter3*. The *set_union* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::lexicographical_compare

Defined in header hpx/algorithm.hpp[653].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter1**, typename **InIter2**, typename **Pred** = *hpx*::*parallel*::detail::less>

bool *hpx*::**lexicographical_compare**(*InIter1* first1, *InIter1* last1, *InIter2* first2, *InIter2* last2, *Pred* &&pred)

Checks if the first range [first1, last1) is lexicographically less than the second range [first2, last2). uses a provided predicate to compare elements.

The comparison operations in the parallel *lexicographical_compare* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: At most 2 * min(N1, N2) applications of the comparison operation, where N1 = std::distance(first1, last) and N2 = std::distance(first2, last2).

> **Note**
>
> Lexicographical comparison is an operation with the following properties
>
> - Two ranges are compared element by element
> - The first mismatching element defines which range is lexicographically *less* or *greater* than the other
> - If one range is a prefix of another, the shorter range is lexicographically *less* than the other
> - If two ranges have equivalent elements and are of the same length, then the ranges are lexicographically *equal*
> - An empty range is lexicographically *less* than any non-empty range
> - Two empty ranges are lexicographically *equal*

Template Parameters

653 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **InIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an input iterator.
- **InIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an input iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *lexicographical_compare* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **pred** – Refers to the comparison function that the first and second ranges will be applied to

Returns

The *lexicographically_compare* algorithm returns a returns *bool* if the execution policy object is not passed in. The *lexicographically_compare* algorithm returns true if the first range is lexicographically less, otherwise it returns false. range [first2, last2), it returns false.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Pred** = *hpx*::*parallel*::detail::less>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::`**`lexicographical_compare`**(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *Pred* &&pred)

Checks if the first range [first1, last1) is lexicographically less than the second range [first2, last2). uses a provided predicate to compare elements.

The comparison operations in the parallel *lexicographical_compare* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *lexicographical_compare* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most 2 * min(N1, N2) applications of the comparison operation, where N1 = std::distance(first1, last) and N2 = std::distance(first2, last2).

Note

Lexicographical comparison is an operation with the following properties

- Two ranges are compared element by element
- The first mismatching element defines which range is lexicographically *less* or *greater* than the other
- If one range is a prefix of another, the shorter range is lexicographically *less* than the other
- If two ranges have equivalent elements and are of the same length, then the ranges are lexicographically *equal*
- An empty range is lexicographically *less* than any non-empty range
- Two empty ranges are lexicographically *equal*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *lexicographical_compare* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **pred** – Refers to the comparison function that the first and second ranges will be applied to

Returns

The *lexicographically_compare* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *lexicographically_compare* algorithm returns true if the first range is lexicographically less, otherwise it returns false. range [first2, last2), it returns false.

hpx::set_intersection

Defined in header hpx/algorithm.hpp[654].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **FwdIter3**, typename **Pred** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter3*> `hpx::set_intersection`(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *FwdIter3* dest, *Pred* &&op = *Pred*())

Constructs a sorted range beginning at dest consisting of all elements present in both sorted ranges [first1, last1) and [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *pred*. Executed according to the policy.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), the first std::min(m, n) elements will be copied from the first range to the destination range. The order of equivalent elements is preserved. The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

The application of function objects in parallel algorithm invoked with a sequential execution policy object execute in sequential order in the calling thread (*sequenced_policy*) or in a single new thread spawned from the current thread (for *sequenced_task_policy*).

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

[654] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **FwdIter1** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **FwdIter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator or output iterator with sequential execution.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_intersection* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*

Returns

The *set_intersection* algorithm returns a *hpx::future<FwdIter3>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter3* otherwise. The *set_intersection* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **FwdIter1**, typename **FwdIter2**, typename **FwdIter3**, typename **Pred** = *hpx*::*parallel*::detail::less>

FwdIter3 **hpx::set_intersection**(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *FwdIter3* dest, *Pred* &&op = *Pred*())

Constructs a sorted range beginning at dest consisting of all elements present in both sorted ranges [first1, last1) and [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *pred*.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), the first std::min(m, n) elements will be copied from the first range to the destination range. The order of equivalent elements is preserved. The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **`FwdIter1`** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **`FwdIter2`** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **`FwdIter3`** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator or output iterator with sequential execution.
- **`Pred`** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_intersection* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **`first1`** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **`last1`** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **`first2`** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **`last2`** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **`dest`** – Refers to the beginning of the destination range.
- **`op`** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*

Returns

The *set_intersection* algorithm returns a *FwdIter3*. The *set_intersection* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::transform_exclusive_scan

Defined in header hpx/algorithm.hpp[655].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter**, typename **OutIter**, typename **BinOp**, typename **UnOp**, typename **T** = typename *std*::iterator_traits<*InIter*>::value_type>

OutIter hpx::**transform_exclusive_scan**(*InIter* first, *InIter* last, *OutIter* dest, *T* init, *BinOp* &&binary_op, *UnOp* &&unary_op)

Transforms each element in the range [first, last) with *unary_op*, then computes an exclusive prefix sum operation using *binary_op* over the resulting range, with *init* as the initial value, and writes the results to the range beginning at *dest*. "exclusive" means that the i-th input element is not included in the i-th sum. Formally, assigns through each iterator i in [dest, d_first + (last - first)) the value of the generalized noncommutative sum of init, unary_op(*j)… for every j in [first, first + (i - d_first)) over binary_op, where generalized noncommutative sum GNSUM(op, a1, …, a N) is defined as follows:

- if N=1, a1
- if N > 1, op(GNSUM(op, a1, …, aK), GNSUM(op, aM, …, aN)) for any K where 1 < K+1 = M <= N In other words, the summation operations may be performed in arbitrary order, and the behavior is nondeterministic if *binary_op* is not associative.

The reduce operations in the parallel *transform_exclusive_scan* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Neither *unary_op* nor *binary_op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The behavior of transform_exclusive_scan may be non-deterministic for a non-associative predicate.

Note

Complexity: O(last - first) applications of each of *binary_op* and *unary_op*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:

- a1 when N is 1
- op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN) where 1 < K+1 = M <= N.

[655] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **BinOp** – The type of *binary_op*.
- **UnOp** – The type of *unary_op*.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **init** – The initial value for the generalized sum.
- **binary_op** – Binary *FunctionObject* that will be applied to the result of *unary_op*, the results of other *binary_op*, and *init*.
- **unary_op** – Unary *FunctionObject* that will be applied to each element of the input range. The return type must be acceptable as input to *binary_op*.

Returns

The *transform_exclusive_scan* algorithm returns a returns *OutIter*. The *transform_exclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **BinOp**, typename **UnOp**, typename **T** = typename *std*::iterator_traits<*FwdIter1*>::value_type>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter2*>::type *hpx*::**transform_exclusive_scan**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *T* init, *BinOp* &&binary_op, *UnOp* &&unary_op)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, conv(*first), …, conv(*(first + (i - result) - 1))). Executed according to the policy.

The reduce operations in the parallel *transform_exclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *transform_exclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Neither *unary_op* nor *binary_op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The behavior of transform_exclusive_scan may be non-deterministic for a non-associative predicate.

> **Note**
>
> Complexity: O(last - first) applications of each of *binary_op* and *unary_op*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:
>
> - a1 when N is 1
> - op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **BinOp** – The type of *binary_op*.
- **UnOp** – The type of *unary_op*.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

- **init** – The initial value for the generalized sum.
- **binary_op** – Binary *FunctionObject* that will be applied in to the result of *unary_op*, the results of other *binary_op*, and *init*.
- **unary_op** – Unary *FunctionObject* that will be applied to each element of the input range. The return type must be acceptable as input to *binary_op*.

Returns

The *transform_exclusive_scan* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *transform_exclusive_scan* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::uninitialized_fill

Defined in header hpx/algorithm.hpp[656].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::uninitialized_fill_n*

template<typename **FwdIter**, typename **T**>

void *hpx*::**uninitialized_fill**(*FwdIter* first, *FwdIter* last, *T* const &value)

Copies the given *value* to an uninitialized memory area, defined by the range [first, last). If an exception is thrown during the initialization, the function has no effects.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *uninitialized_fill* algorithm returns nothing

[656] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

template<typename **ExPolicy**, typename **FwdIter**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*> `hpx::uninitialized_fill`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *T* const &value)

Copies the given *value* to an uninitialized memory area, defined by the range [first, last). If an exception is thrown during the initialization, the function has no effects. Executed according to the policy.

The initializations in the parallel *uninitialized_fill* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The initializations in the parallel *uninitialized_fill* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *uninitialized_fill* algorithm returns a *hpx::future<void>*, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns nothing otherwise.

hpx::uninitialized_fill_n

Defined in header hpx/algorithm.hpp[657].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::uninitialized_fill*

template<typename **FwdIter**, typename **Size**, typename **T**>

FwdIter `hpx::uninitialized_fill_n`(*FwdIter* first, *Size* count, *T* const &value)

Copies the given *value* value to the first count elements in an uninitialized memory area beginning at first. If an exception is thrown during the initialization, the function has no effects.

> **Note**
>
> Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *uninitialized_fill_n* algorithm returns a returns *FwdIter*. The *uninitialized_fill_n* algorithm returns the output iterator to the element in the range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::uninitialized_fill_n`(*ExPolicy* &&policy, *FwdIter* first, *Size* count, *T* const &value)

[657] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Copies the given *value* value to the first count elements in an uninitialized memory area beginning at first. If an exception is thrown during the initialization, the function has no effects. Executed according to the policy.

The initializations in the parallel *uninitialized_fill_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The initializations in the parallel *uninitialized_fill_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *uninitialized_fill_n* algorithm returns a *hpx::future<FwdIter>*, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns FwdIter otherwise. The *uninitialized_fill_n* algorithm returns the output iterator to the element in the range, one past the last element copied.

hpx::move

Defined in header hpx/algorithm.hpp[658].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> **hpx::move**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest)

Moves the elements in the range [first, last), to another range beginning at *dest*. After this operation the elements in the moved-from range will still contain valid values of the appropriate type, but not necessarily the same values as before the move. Executed according to the policy.

The move assignments in the parallel *move* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The move assignments in the parallel *move* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* move assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the move assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *move* algorithm returns a *hpx::future<FwdIter2>>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise.

[658] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The *move* algorithm returns the output iterator to the element in the destination range, one past the last element moved.

template<typename **FwdIter1**, typename **FwdIter2**>

FwdIter2 **hpx::move**(*FwdIter1* first, *FwdIter1* last, *FwdIter2* dest)

Moves the elements in the range [first, last), to another range beginning at *dest*. After this operation the elements in the moved-from range will still contain valid values of the appropriate type, but not necessarily the same values as before the move.

> **Note**
>
> Complexity: Performs exactly *last - first* move assignments.

Template Parameters

- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *move* algorithm returns a *FwdIter2*. The *move* algorithm returns the output iterator to the element in the destination range, one past the last element moved.

hpx::destroy

Defined in header hpx/algorithm.hpp[659].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::destroy_n*

template<typename **ExPolicy**, typename **FwdIter**>

util::detail::algorithm_result_t<*ExPolicy*> **hpx::destroy**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last)

Destroys objects of type typename iterator_traits<ForwardIt>::value_type in the range [first, last). Executed according to the policy.

[659] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The operations in the parallel *destroy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The operations in the parallel *destroy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* operations.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *destroy* algorithm returns a *hpx::future<void>*, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *void* otherwise.

template<typename **`FwdIter`**>

void *`hpx::`***`destroy`**(*FwdIter* first, *FwdIter* last)

Destroys objects of type typename iterator_traits<ForwardIt>::value_type in the range [first, last).

> **Note**
>
> Complexity: Performs exactly *last* - *first* operations.

Template Parameters

`FwdIter` – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *destroy* algorithm returns a *void*

hpx::destroy_n

Defined in header hpx/algorithm.hpp[660].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::destroy*

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**>

util::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::destroy_n`(*ExPolicy* &&policy, *FwdIter* first, *Size* count)

Destroys objects of type typename iterator_traits<ForwardIt>::value_type in the range [first, first + count). Executed according to the policy.

The operations in the parallel *destroy_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The operations in the parallel *destroy_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *count* operations, if count > 0, no assignments otherwise.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply this algorithm to.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.

[660] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Returns

The *destroy_n* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *destroy_n* algorithm returns the iterator to the element in the source range, one past the last element constructed.

template<typename **FwdIter**, typename **Size**>

FwdIter `hpx::destroy_n`(*FwdIter* first, *Size* count)

Destroys objects of type typename iterator_traits<ForwardIt>::value_type in the range [first, first + count).

> **Note**
>
> Complexity: Performs exactly *count* operations, if count > 0, no assignments otherwise.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply this algorithm to.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.

Returns

The *destroy_n* algorithm returns a *FwdIter* . The *destroy_n* algorithm returns the iterator to the element in the source range, one past the last element constructed.

hpx::experimental::reduction_min

Defined in header hpx/algorithm.hpp[661].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *hpx*::*parallel*::detail::min_of<*T*>> `hpx::experimental::reduction_min`(*T* &var)

The function template *reduction_min* returns a reduction object of unspecified type having a value type and encapsulating an identity value for the reduction, it uses std::min_of{} as its combiner function, and a live-out object from which the initial value is obtained and into which the final value is stored.

[661] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

A parallel algorithm uses the *reduction_min* object by allocating an unspecified number of instances, called views, of the reduction's value type. Each view is initialized with the reduction object's identity value, except that the live-out object (which was initialized by the caller) comprises one of the views. The algorithm passes a reference to a view to each application of an element-access function, ensuring that no two concurrently-executing invocations share the same view. A view can be shared between two applications that do not execute concurrently, but initialization is performed only once per view.

Modifications to the view by the application of element access functions accumulate as partial results. At some point before the algorithm returns, the partial results are combined, two at a time, using the std::min_of{} operation until a single value remains, which is then assigned back to the live-out object.

T shall meet the requirements of *CopyConstructible* and *MoveAssignable*.

> **Note**
>
> In order to produce useful results, modifications to the view should be limited to commutative operations closely related to the combiner operation.

Template Parameters

- **T** – The value type to be used by the induction object.
- **Op** – The type of the binary function (object) used to perform the reduction operation.

Parameters

- **var** – [in,out] The life-out value to use for the reduction object. This will hold the reduced value after the algorithm is finished executing.
- **identity** – [in] The identity value to use for the reduction operation. This argument is optional and defaults to a copy of *var*.

Returns

This returns a reduction object of unspecified type having a value type of *T*. When the return value is used by an algorithm, the reference to *var* is used as the live-out object, new views are initialized to a copy of identity, and views are combined by invoking the copy of combiner, passing it the two views to be combined.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *hpx*::*parallel*::detail::min_of<*T*>> `hpx::experimental::`**`reduction_min`**(*T* &var, *T* const &identity)

hpx::rotate

Defined in header hpx/algorithm.hpp[662].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::rotate_copy*

template<typename **FwdIter**>

FwdIter `hpx::rotate`(*FwdIter* first, *FwdIter* new_first, *FwdIter* last)

Performs a left rotation on a range of elements. Specifically, *rotate* swaps the elements in the range [first, last) in such a way that the element new_first becomes the first element of the new range and new_first - 1 becomes the last element.

The assignments in the parallel *rotate* algorithm execute in sequential order in the calling thread.

Note

Complexity: Linear in the distance between *first* and *last*.

Note

The type of dereferenced *FwdIter* must meet the requirements of *MoveAssignable* and *MoveConstructible*.

Template Parameters

FwdIter – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **new_first** – Refers to the element that should appear at the beginning of the rotated range.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *rotate* algorithm returns a FwdIter. The *rotate* algorithm returns the iterator to the new location of the element pointed by first,equal to first + (last - new_first).

template<typename **ExPolicy**, typename **FwdIter**>

[662] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::rotate`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* new_first, *FwdIter* last)

Performs a left rotation on a range of elements. Specifically, *rotate* swaps the elements in the range [first, last) in such a way that the element new_first becomes the first element of the new range and new_first - 1 becomes the last element. Executed according to the policy.

The assignments in the parallel *rotate* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *rotate* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*.

> **Note**
>
> The type of dereferenced *FwdIter* must meet the requirements of *MoveAssignable* and *MoveConstructible*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **new_first** – Refers to the element that should appear at the beginning of the rotated range.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *rotate* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *rotate* algorithm returns the iterator equal to first + (last - new_first).

hpx::rotate_copy

Defined in header hpx/algorithm.hpp[663].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::rotate*

template<typename **FwdIter**, typename **OutIter**>

OutIter `hpx::rotate_copy`(*FwdIter* first, *FwdIter* new_first, *FwdIter* last, *OutIter* dest_first)

Copies the elements from the range [first, last), to another range beginning at *dest_first* in such a way, that the element *new_first* becomes the first element of the new range and *new_first* - 1 becomes the last element.

The assignments in the parallel *rotate_copy* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **OutIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a output iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **new_first** – Refers to the element that should appear at the beginning of the rotated range.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest_first** – Refers to the begin of the destination range.

Returns

The *rotate_copy* algorithm returns a output iterator, The *rotate_copy* algorithm returns the output iterator to the element past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**>

[663] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> `hpx::rotate_copy`(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* new_first, *FwdIter1* last, *FwdIter2* dest_first)

Copies the elements from the range [first, last), to another range beginning at *dest_first* in such a way, that the element *new_first* becomes the first element of the new range and *new_first* - 1 becomes the last element. Executed according to the policy.

The assignments in the parallel *rotate_copy* algorithm execute in sequential order in the calling thread.

The assignments in the parallel *rotate_copy* algorithm execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **new_first** – Refers to the element that should appear at the beginning of the rotated range.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest_first** – Refers to the begin of the destination range.

Returns

The *rotate_copy* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *parallel_task_policy* and returns FwdIter2 otherwise. The *rotate_copy* algorithm returns the output iterator to the element past the last element copied.

hpx::uninitialized_move

Defined in header hpx/algorithm.hpp[664].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::uninitialized_move_n*

template<typename **InIter**, typename **FwdIter**>

FwdIter hpx::**uninitialized_move**(*InIter* first, *InIter* last, *FwdIter* dest)

Moves the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the initialization, some objects in [first, last) are left in a valid but unspecified state.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *uninitialized_move* algorithm returns *FwdIter*. The *uninitialized_move* algorithm returns the output iterator to the element in the destination range, one past the last element moved.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> hpx::**uninitialized_move**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest)

Moves the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the initialization, some objects in [first, last) are left in a valid but unspecified state. Executed according to the policy.

[664] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The assignments in the parallel *uninitialized_move* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_move* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter1`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **`FwdIter2`** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`dest`** – Refers to the beginning of the destination range.

Returns

The *uninitialized_move* algorithm returns a *hpx::future<FwdIter2>*, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *uninitialized_move* algorithm returns the output iterator to the element in the destination range, one past the last element moved.

hpx::uninitialized_move_n

Defined in header hpx/algorithm.hpp[665].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::uninitialized_move*

template<typename **`InIter`**, typename **`Size`**, typename **`FwdIter`**>

[665] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

std::pair<*InIter*, *FwdIter*> hpx::**uninitialized_move_n**(*InIter* first, *Size* count, *FwdIter* dest)

Moves the elements in the range [first, first + count), starting from first and proceeding to first + count - 1., to another range beginning at dest. If an exception is thrown during the initialization, some objects in [first, first + count) are left in a valid but unspecified state.

> **Note**
>
> Complexity: Performs exactly *count* movements, if count > 0, no move operations otherwise.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *uninitialized_move_n* algorithm returns a returns *std::pair<InIter,FwdIter>*. The *uninitialized_move_n* algorithm returns A pair whose first element is an iterator to the element past the last element moved in the source range, and whose second element is an iterator to the element past the last element moved in the destination range.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Size**, typename **FwdIter2**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *std*::pair<*FwdIter1*, *FwdIter2*>>::type hpx::**uninitialized_move_n**(*ExPolicy* &&policy, *FwdIter1* first, *Size* count, *FwdIter2* dest)

Moves the elements in the range [first, first + count), starting from first and proceeding to first + count - 1., to another range beginning at dest. If an exception is thrown during the initialization, some objects in [first, first + count) are left in a valid but unspecified state. Executed according to the policy.

The assignments in the parallel *uninitialized_move_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_move_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *count* movements, if count > 0, no move operations otherwise.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *uninitialized_move_n* algorithm returns a hpx::future<*std::pair<FwdIter1,FwdIter2>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *std::pair<FwdIter1,FwdIter2>* otherwise. The *uninitialized_move_n* algorithm returns A pair whose first element is an iterator to the element past the last element moved in the source range, and whose second element is an iterator to the element past the last element moved in the destination range.

hpx::fill

Defined in header hpx/algorithm.hpp[666].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::fill_n*

template<typename **ExPolicy**, typename **FwdIter**, typename **T**>

util::detail::algorithm_result_t<*ExPolicy*> **hpx::fill**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *T* value)

Assigns the given value to the elements in the range [first, last). Executed according to the policy.

The comparisons in the parallel *fill* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *fill* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last - first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *fill* algorithm returns a *hpx::future<void>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *difference_type* otherwise (where *difference_type* is defined by *void*.

[666] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

template<typename **FwdIter**, typename **T**>

void *hpx*::**fill**(*FwdIter* first, *FwdIter* last, *T* value)

Assigns the given value to the elements in the range [first, last).

> **Note**
>
> Complexity: Performs exactly *last - first* assignments.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *fill* algorithm returns a *void*.

hpx::fill_n

Defined in header hpx/algorithm.hpp[667].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::fill*

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**, typename **T**>

util::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> *hpx*::**fill_n**(*ExPolicy* &&policy, *FwdIter* first, *Size* count, *T* value)

Assigns the given value value to the first count elements in the range beginning at first if count > 0. Does nothing otherwise. Executed according to the policy.

The comparisons in the parallel *fill_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *fill_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

[667] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

> **Note**
>
> Complexity: Performs exactly *count* assignments, for count > 0.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *fill_n* algorithm returns a *hpx::future<void>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *difference_type* otherwise (where *difference_type* is defined by *void*.

template<typename **FwdIter**, typename **Size**, typename **T**>

FwdIter hpx::**fill_n**(*FwdIter* first, *Size* count, *T* value)

Assigns the given value value to the first count elements in the range beginning at first if count > 0. Does nothing otherwise.

> **Note**
>
> Complexity: Performs exactly *count* assignments, for count > 0.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *fill_n* algorithm returns a *FwdIter*.

hpx::count

Defined in header hpx/algorithm.hpp[668].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::count_if*

template<typename **ExPolicy**, typename **FwdIter**, typename **T**>

util::detail::algorithm_result<*ExPolicy*, typename *std*::iterator_traits<*FwdIter*>::difference_type>::type *hpx*::**count**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *T* const &value)

Returns the number of elements in the range [first, last) satisfying a specific criteria. This version counts the elements that are equal to the given *value*. Executed according to the policy.

The comparisons in the parallel *count* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

Note

Complexity: Performs exactly *last* - *first* comparisons.

Note

The comparisons in the parallel *count* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

[668] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the comparisons.
- **FwdIter** – The type of the source iterator used (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to search for (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **value** – The value to search for.

Returns

The *count* algorithm returns a *hpx::future<difference_type>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *difference_type* otherwise (where *difference_type* is defined by *std::iterator_traits<FwdIterB>::difference_type*. The *count* algorithm returns the number of elements satisfying the given criteria.

template<typename **InIter**, typename **T**>

std::iterator_traits<*InIter*>::difference_type `hpx::count`(*InIter* first, *InIter* last, *T* const &value)

Returns the number of elements in the range [first, last) satisfying a specific criteria. This version counts the elements that are equal to the given *value*.

> **Note**
>
> Complexity: Performs exactly *last* - *first* comparisons.

Template Parameters

- **InIter** – The type of the source iterator used (deduced). This iterator type must meet the requirements of an input iterator.
- **T** – The type of the value to search for (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **value** – The value to search for.

Returns

The *count* algorithm returns a *difference_type* (where *difference_type* is defined by *std::iterator_traits<InIter>::difference_type*. The *count* algorithm returns the number of elements satisfying the given criteria.

hpx::count_if

Defined in header hpx/algorithm.hpp[669].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::count*

template<typename **ExPolicy**, typename **FwdIter**, typename **F**>

util::detail::algorithm_result<*ExPolicy*, typename *std*::iterator_traits<*FwdIter*>::difference_type>::type **hpx::count_if**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *F* &&f)

Returns the number of elements in the range [first, last) satisfying a specific criteria. This version counts elements for which predicate *f* returns true. Executed according to the policy.

> **Note**
>
> Complexity: Performs exactly *last* - *first* applications of the predicate.

> **Note**
>
> The assignments in the parallel *count_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

> **Note**
>
> The assignments in the parallel *count_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Template Parameters

[669] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the comparisons.
- **FwdIter** – The type of the source begin iterator used (deduced). This iterator type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *count_if* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

Returns

The *count_if* algorithm returns *hpx::future<difference_type>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *difference_type* otherwise (where *difference_type* is defined by *std::iterator_traits<FwdIter>::difference_type*. The *count* algorithm returns the number of elements satisfying the given criteria.

template<typename **InIter**, typename **F**>

std::iterator_traits<*InIter*>::difference_type **hpx::count_if**(*InIter* first, *InIter* last, *F* &&f)

Returns the number of elements in the range [first, last) satisfying a specific criteria. This version counts elements for which predicate *f* returns true.

> **Note**
>
> Complexity: Performs exactly *last* - *first* applications of the predicate.

Template Parameters

- **InIter** – The type of the source begin iterator used (deduced). This iterator type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *count_if* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.

Returns

The *count_if* algorithm returns *difference_type* (where a difference_type is defined by *std::iterator_traits<InIter>::difference_type*. The *count* algorithm returns the number of elements satisfying the given criteria.

hpx::experimental::for_each_index

Defined in header hpx/algorithm.hpp[670].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **Mapping**, typename **Fun**>

constexpr void `hpx::experimental::for_each_index`(*Mapping* const &mapping, *Fun* fun)

Calls *fun* with every multidimensional index in the domain of *mapping* (i.e., its extents), in sequential order.

If *fun* returns a result, the result is ignored.

The layout mapping is a hint; implementations are encouraged to choose an iteration order that performs well for the given mapping.

> **Note**
>
> Complexity: Applies *fun* exactly the product of all extents of *mapping* times.

Template Parameters

- **Mapping** – A type meeting the layout mapping requirements. Must expose *index_type*, *extents_type*, `static` *rank()*, and *extents()*.
- **Fun** – The type of the function/function object to use (deduced). *Fun* must meet *Copy-Constructible* and be invocable with exactly *Mapping::rank()* arguments of type *Mapping::index_type*.

[670] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Parameters

- **mapping** – The layout mapping whose extents define the iteration domain.
- **fun** – Callable invoked as `fun(i0, i1, ...)` for every multidimensional index in the domain.

template<typename **ExPolicy**, typename **Mapping**, typename **Fun**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*> *hpx*::*experimental*::**for_each_index**(*ExPolicy* &&policy, *Mapping* const &mapping, *Fun* fun)

Calls *fun* with every multidimensional index in the domain of *mapping* (i.e., its extents), executed according to *policy*.

If *fun* returns a result, the result is ignored.

> **Note**
>
> Complexity: Applies *fun* exactly the product of all extents of *mapping* times.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced).
- **Mapping** – A type meeting the layout mapping requirements. Must meet *CopyConstructible*.
- **Fun** – The type of the function/function object to use (deduced). Must meet *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for scheduling.
- **mapping** – The layout mapping whose extents define the iteration domain.
- **fun** – Callable invoked as `fun(i0, i1, ...)` for every multidimensional index in the domain.

Returns

hpx::future<void> if the execution policy is *sequenced_task_policy* or *parallel_task_policy*, *void* otherwise.

hpx::reduce

Defined in header hpx/algorithm.hpp[671].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **FwdIter**, typename **F**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> **hpx::reduce**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *T* init, *F* &&f)

Returns GENERALIZED_SUM(f, init, *first, …, *(first + (last - first) - 1)). Executed according to the policy.

The reduce operations in the parallel *reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *f*.

> **Note**
>
> GENERALIZED_SUM(op, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source begin and end iterators used (deduced). This iterator type must meet the requirements of a forward iterator.

[671] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *reduce* requires *F* to meet the requirements of *CopyConstructible*.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **init** – The initial value for the generalized sum.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const&. The types *Type1 Ret* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to any of those types.

Returns

The *reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *T* otherwise. The *reduce* algorithm returns the result of the generalized sum over the elements given by the input range [first, last).

template<typename **ExPolicy**, typename **FwdIter**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>
util::detail::algorithm_result_t<*ExPolicy*, *T*> **hpx::reduce**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *T* init)

Returns GENERALIZED_SUM(+, init, *first, …, *(first + (last - first) - 1)). Executed according to the policy.

The reduce operations in the parallel *reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the operator+().

Note

GENERALIZED_SUM(+, a1, …, aN) is defined as follows:

- a1 when N is 1
- op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
  - b1, …, bN may be any permutation of a1, …, aN and
  - 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source begin and end iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **init** – The initial value for the generalized sum.

Returns

The *reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *T* otherwise. The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

template<typename **ExPolicy**, typename **FwdIter**>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, typename *std*::iterator_traits<*FwdIter*>::value_type>::type **hpx::reduce**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last)

Returns GENERALIZED_SUM(+, T(), *first, …, *(first + (last - first) - 1)). Executed according to the policy.

The reduce operations in the parallel *reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *reduce* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the operator+().

> **Note**
>
> The type of the initial value (and the result type) *T* is determined from the value_type of the used *FwdIter*.

> **Note**
>
> GENERALIZED_SUM(+, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter`** – The type of the source begin and end iterators used (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns T otherwise (where T is the value_type of *FwdIter*). The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

template<typename **FwdIter**, typename **F**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>

T `hpx::`**`reduce`**(*FwdIter* first, *FwdIter* last, *T* init, *F* &&f)

Returns GENERALIZED_SUM(f, init, *first, . . . , *(first + (last - first) - 1)). Executed according to the policy.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

Note

Complexity: O(*last* - *first*) applications of the predicate *f*.

Note

GENERALIZED_SUM(op, a1, . . . , aN) is defined as follows:

- a1 when N is 1
- op(GENERALIZED_SUM(op, b1, . . . , bK), GENERALIZED_SUM(op, bM, . . . , bN)), where:
  - b1, . . . , bN may be any permutation of a1, . . . , aN and
  - 1 < K+1 = M <= N.

Template Parameters

- **`FwdIter`** – The type of the source begin and end iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **`F`** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *reduce* requires *F* to meet the requirements of *CopyConstructible*.
- **`T`** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`init`** – The initial value for the generalized sum.
- **`f`** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&. The types *Type1 Ret* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to any of those types.

Returns

The *reduce* algorithm returns *T*. The *reduce* algorithm returns the result of the generalized sum over the elements given by the input range [first, last).

template<typename **FwdIter**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>

T `hpx::reduce`(*FwdIter* first, *FwdIter* last, *T* init)

Returns GENERALIZED_SUM(+, init, *first, …, *(first + (last - first) - 1)). Executed according to the policy.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the operator+().

> **Note**
>
> GENERALIZED_SUM(+, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **FwdIter** – The type of the source begin and end iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **init** – The initial value for the generalized sum.

Returns

The *reduce* algorithm returns a *T*. The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

template<typename **FwdIter**>

std::iterator_traits<*FwdIter*>::value_type *hpx*::**reduce**(*FwdIter* first, *FwdIter* last)

Returns GENERALIZED_SUM(+, T(), *first, …, *(first + (last - first) - 1)). Executed according to the policy.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the operator+().

> **Note**
>
> The type of the initial value (and the result type) *T* is determined from the value_type of the used *FwdIter*.

> **Note**
>
> GENERALIZED_SUM(+, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

FwdIter – The type of the source begin and end iterators used (deduced). This iterator type must meet the requirements of an input iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *reduce* algorithm returns *T* (where T is the value_type of *FwdIter*). The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

hpx::find

Defined in header hpx/algorithm.hpp[672].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::find_if*
- *hpx::find_if_not*
- *hpx::find_end*
- *hpx::find_first_of*

template<typename **ExPolicy**, typename **FwdIter**, typename **T**>

util::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::find`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *T* const &val)

Returns the first element in the range [first, last) that is equal to value. Executed according to the policy.

The comparison operations in the parallel *find* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *find* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most last - first applications of the operator==().

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to find (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.

[672] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **val** – the value to compare the elements to

Returns

The *find* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *find* algorithm returns the first element in the range [first,last) that is equal to *val*. If no such element in the range of [first,last) is equal to *val*, then the algorithm returns *last*.

template<typename **InIter**, typename **T**>

InIter `hpx::`**find**(*InIter* first, *InIter* last, *T* const &val)

Returns the first element in the range [first, last) that is equal to value. Executed according to the policy.

> **Note**
>
> Complexity: At most last - first applications of the operator==().

Template Parameters

- **InIter** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an input iterator.
- **T** – The type of the value to find (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **val** – the value to compare the elements to

Returns

The *find* algorithm returns a *InIter*. The *find* algorithm returns the first element in the range [first,last) that is equal to *val*. If no such element in the range of [first,last) is equal to *val*, then the algorithm returns *last*.

hpx::find_if

Defined in header hpx/algorithm.hpp[673].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::find*
- *hpx::find_if_not*
- *hpx::find_end*

[673] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- *hpx::find_first_of*

template<typename **ExPolicy**, typename **FwdIter**, typename **F**>

util::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::find_if`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *F* &&f)

Returns the first element in the range [first, last) for which predicate *f* returns true. Executed according to the policy.

The comparison operations in the parallel *find_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *find_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most last - first applications of the predicate.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **f** – The unary predicate which returns true for the required element. The signature of the predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.

Returns

The *find_if* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise.

The *find_if* algorithm returns the first element in the range [first,last) that satisfies the predicate *f*. If no such element exists that satisfies the predicate f, the algorithm returns *last*.

template<typename **InIter**, typename **F**>

InIter `hpx::`**`find_if`**(*InIter* first, *InIter* last, *F* &&f)

Returns the first element in the range [first, last) for which predicate *f* returns true.

> **Note**
>
> Complexity: At most last - first applications of the predicate.

Template Parameters

- **`InIter`** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an input iterator.
- **`F`** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **`first`** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **`f`** – The unary predicate which returns true for the required element. The signature of the predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type*.

Returns

The *find_if* algorithm returns a *InIter*. The *find_if* algorithm returns the first element in the range [first,last) that satisfies the predicate *f*. If no such element exists that satisfies the predicate f, the algorithm returns *last*.

hpx::find_if_not

Defined in header hpx/algorithm.hpp[674].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::find*
- *hpx::find_if*
- *hpx::find_end*
- *hpx::find_first_of*

template<typename **ExPolicy**, typename **FwdIter**, typename **F**>

util::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> hpx::**find_if_not**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *F* &&f)

Returns the first element in the range [first, last) for which predicate *f* returns false. Executed according to the policy.

The comparison operations in the parallel *find_if_not* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *find_if_not* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most last - first applications of the predicate.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.

[674] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **last** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **f** – The unary predicate which returns false for the required element. The signature of the predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.

Returns

The *find_if_not* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *find_if_not* algorithm returns the first element in the range [first, last) that does **not** satisfy the predicate *f*. If no such element exists that does not satisfy the predicate f, the algorithm returns *last*.

template<typename **FwdIter**, typename **F**>

FwdIter hpx::**find_if_not**(*FwdIter* first, *FwdIter* last, *F* &&f)

Returns the first element in the range [first, last) for which predicate *f* returns false.

> **Note**
>
> Complexity: At most last - first applications of the predicate.

Template Parameters

- **FwdIter** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **f** – The unary predicate which returns false for the required element. The signature of the predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.

Returns

The *find_if_not* algorithm returns a *FwdIter*. The *find_if_not* algorithm returns the first element in the range [first, last) that does **not** satisfy the predicate *f*. If no such element exists that does not satisfy the predicate f, the algorithm returns *last*.

hpx::find_end

Defined in header hpx/algorithm.hpp[675].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::find*
- *hpx::find_if*
- *hpx::find_if_not*
- *hpx::find_first_of*

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Pred** = detail::equal_to>

util::detail::algorithm_result_t<*ExPolicy*, *FwdIter1*> hpx::**find_end**(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *Pred* &&op = *Pred*())

Returns the last subsequence of elements [first2, last2) found in the range [first, last) using the given predicate *op* to compare elements. Executed according to the policy.

The comparison operations in the parallel *find_end* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *find_end* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

This overload of *find_end* is available if the user decides to provide the algorithm their own predicate *op*.

> **Note**
>
> Complexity: at most S*(N-S+1) comparisons where *S* = distance(first2, last2) and *N* = distance(first1, last1).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.

[675] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *replace* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements the algorithm will be searching for.
- **last2** – Refers to the end of the sequence of elements of the algorithm will be searching for.
- **op** – The binary predicate which returns *true* if the elements should be treated as equal. The signature should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively.

Returns

The *find_end* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *find_end* algorithm returns an iterator to the beginning of the last subsequence [first2, last2) in range [first, last). If the length of the subsequence [first2, last2) is greater than the length of the range [first1, last1), *last1* is returned. Additionally if the size of the subsequence is empty or no subsequence is found, *last1* is also returned.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**>

util::detail::algorithm_result_t<*ExPolicy*, *FwdIter1*> hpx::**find_end**(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2)

Returns the last subsequence of elements [first2, last2) found in the range [first, last). Elements are compared using *operator*==. Executed according to the policy.

The comparison operations in the parallel *find_end* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *find_end* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in

an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most S*(N-S+1) comparisons where *S* = distance(first2, last2) and *N* = distance(first1, last1).

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter1`** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **`FwdIter2`** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first1`** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **`last1`** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **`first2`** – Refers to the beginning of the sequence of elements the algorithm will be searching for.
- **`last2`** – Refers to the end of the sequence of elements of the algorithm will be searching for.

Returns

The *find_end* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *find_end* algorithm returns an iterator to the beginning of the last subsequence [first2, last2) in range [first, last). If the length of the subsequence [first2, last2) is greater than the length of the range [first1, last1), *last1* is returned. Additionally if the size of the subsequence is empty or no subsequence is found, *last1* is also returned.

template<typename **`FwdIter1`**, typename **`FwdIter2`**, typename **`Pred`** = detail::equal_to>

FwdIter1 `hpx::`**`find_end`**(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *Pred* &&op = *Pred*())

Returns the last subsequence of elements [first2, last2) found in the range [first, last) using the given predicate *op* to compare elements.

This overload of *find_end* is available if the user decides to provide the algorithm their own predicate *op*.

> **Note**
>
> Complexity: at most S*(N-S+1) comparisons where *S* = distance(first2, last2) and *N* = distance(first1, last1).

Template Parameters

- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *replace* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements the algorithm will be searching for.
- **last2** – Refers to the end of the sequence of elements of the algorithm will be searching for.
- **op** – The binary predicate which returns *true* if the elements should be treated as equal. The signature should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively.

Returns

The *find_end* algorithm returns a *FwdIter1*. The *find_end* algorithm returns an iterator to the beginning of the last subsequence [first2, last2) in range [first, last). If the length of the subsequence [first2, last2) is greater than the length of the range [first1, last1), *last1* is returned. Additionally if the size of the subsequence is empty or no subsequence is found, *last1* is also returned.

template<typename **FwdIter1**, typename **FwdIter2**>

FwdIter1 hpx::**find_end**(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2)

Returns the last subsequence of elements [first2, last2) found in the range [first, last). Elements are compared using *operator==*.

> **Note**
>
> Complexity: at most S*(N-S+1) comparisons where *S* = distance(first2, last2) and *N* = distance(first1, last1).

Template Parameters

- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements the algorithm will be searching for.
- **last2** – Refers to the end of the sequence of elements of the algorithm will be searching for.

Returns

The *find_end* algorithm returns a *FwdIter1*. The *find_end* algorithm returns an iterator to the beginning of the last subsequence [first2, last2) in range [first, last). If the length of the subsequence [first2, last2) is greater than the length of the range [first1, last1), *last1* is returned. Additionally if the size of the subsequence is empty or no subsequence is found, *last1* is also returned.

hpx::find_first_of

Defined in header hpx/algorithm.hpp[676].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::find*
- *hpx::find_if*
- *hpx::find_if_not*
- *hpx::find_end*

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Pred** = detail::equal_to>

util::detail::algorithm_result_t<*ExPolicy*, *FwdIter1*> hpx::**find_first_of**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* s_first, *FwdIter2* s_last, *Pred* &&op = *Pred*())

[676] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Searches the range [first, last) for any elements in the range [s_first, s_last). Uses binary predicate *op* to compare elements. Executed according to the policy.

The comparison operations in the parallel *find_first_of* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *find_first_of* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

This overload of *find_first_of* is available if the user decides to provide the algorithm their own predicate *op*.

> **Note**
>
> Complexity: at most (S*N) comparisons where *S* = distance(s_first, s_last) and *N* = distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **s_first** – Refers to the beginning of the sequence of elements the algorithm will be searching for.
- **s_last** – Refers to the end of the sequence of elements of the algorithm will be searching for.
- **op** – The binary predicate which returns *true* if the elements should be treated as equal. The signature should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively.

Returns

The *find_first_of* algorithm returns a *hpx::future<FwdIter1>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter1* otherwise. The *find_first_of* algorithm returns an iterator to the first element in the range [first, last) that is equal to an element from the range [s_first, s_last). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, last), *last* is returned. Additionally if the size of the subsequence is empty or no subsequence is found, *last* is also returned.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**>

util::detail::algorithm_result_t<*ExPolicy*, *FwdIter1*> `hpx::`**`find_first_of`**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* s_first, *FwdIter2* s_last)

Searches the range [first, last) for any elements in the range [s_first, s_last). Elements are compared using *operator*==. Executed according to the policy.

The comparison operations in the parallel *find_first_of* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *find_first_of* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (S*N) comparisons where *S* = distance(s_first, s_last) and *N* = distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **s_first** – Refers to the beginning of the sequence of elements the algorithm will be searching for.
- **s_last** – Refers to the end of the sequence of elements of the algorithm will be searching for.

Returns

The *find_first_of* algorithm returns a *hpx::future<FwdIter1>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter1* otherwise. The *find_first_of* algorithm returns an iterator to the first element in the range [first, last) that is equal to an element from the range [s_first, s_last). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, last), *last* is returned. Additionally if the size of the subsequence is empty or no subsequence is found, *last* is also returned.

template<typename **FwdIter1**, typename **FwdIter2**, typename **Pred** = detail::equal_to>

FwdIter1 hpx::**find_first_of**(*FwdIter1* first, *FwdIter1* last, *FwdIter2* s_first, *FwdIter2* s_last, *Pred* &&op = *Pred*())

Searches the range [first, last) for any elements in the range [s_first, s_last). Uses binary predicate *op* to compare elements.

This overload of *find_first_of* is available if the user decides to provide the algorithm their own predicate *op*.

> **Note**
>
> Complexity: at most (S*N) comparisons where *S* = distance(s_first, s_last) and *N* = distance(first, last).

Template Parameters

- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.

- **last** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **s_first** – Refers to the beginning of the sequence of elements the algorithm will be searching for.
- **s_last** – Refers to the end of the sequence of elements of the algorithm will be searching for.
- **op** – The binary predicate which returns *true* if the elements should be treated as equal. The signature should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively.

Returns

The *find_first_of* algorithm returns a *FwdIter1*. The *find_first_of* algorithm returns an iterator to the first element in the range [first, last) that is equal to an element from the range [s_first, s_last). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, last), *last* is returned. Additionally if the size of the subsequence is empty or no subsequence is found, *last* is also returned.

template<typename **FwdIter1**, typename **FwdIter2**>

FwdIter1 hpx::**find_first_of**(*FwdIter1* first, *FwdIter1* last, *FwdIter2* s_first, *FwdIter2* s_last)

Searches the range [first, last) for any elements in the range [s_first, s_last). Elements are compared using *operator==*.

> **Note**
>
> Complexity: at most (S*N) comparisons where S = distance(s_first, s_last) and N = distance(first, last).

Template Parameters

- **FwdIter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **s_first** – Refers to the beginning of the sequence of elements the algorithm will be searching for.

- **s_last** – Refers to the end of the sequence of elements of the algorithm will be searching for.

Returns

The *find_first_of* algorithm returns a *FwdIter1*. The *find_first_of* algorithm returns an iterator to the first element in the range [first, last) that is equal to an element from the range [s_first, s_last). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, last), *last* is returned. Additionally if the size of the subsequence is empty or no subsequence is found, *last* is also returned.

hpx::uninitialized_relocate

Defined in header hpx/algorithm.hpp[677].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::uninitialized_relocate_n*

template<typename **InIter1**, typename **InIter2**, typename **FwdIter**>

FwdIter hpx::**uninitialized_relocate**(*InIter1* first, *InIter2* last, *FwdIter* dest)

Relocates the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the move-construction of an element, all elements left in the input range, as well as all objects already constructed in the destination range are destroyed. After this algorithm completes, the source range should be freed or reused without destroying the objects.

The assignments in the parallel *uninitialized_relocate* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: time: O(n), space: O(1) 1) For "trivially relocatable" underlying types (T) and a contiguous iterator range [first, last): std::distance(first, last)*sizeof(T) bytes are copied. 2) For "trivially relocatable" underlying types (T) and a non-contiguous iterator range [first, last): std::distance(first, last) memory copies of sizeof(T) bytes each are performed. 3) For "non-trivially relocatable" underlying types (T): std::distance(first, last) move assignments and destructions are performed.

> **Note**
>
> Declare a type as "trivially relocatable" using the `HPX_DECLARE_TRIVIALLY_RELOCATABLE` macros found in <hpx/type_support/is_trivially_relocatable.hpp>.

Template Parameters

677 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **InIter1** – The type of the source iterator first (deduced). This iterator type must meet the requirements of an input iterator.
- **InIter2** – The type of the source iterator last (deduced). This iterator type must meet the requirements of an input iterator.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *uninitialized_relocate* algorithm returns *FwdIter*. The *uninitialized_relocate* algorithm returns the output iterator to the element in the destination range, one past the last element relocated.

template<typename **ExPolicy**, typename **InIter1**, typename **InIter2**, typename **FwdIter**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> hpx::**uninitialized_relocate**(*ExPolicy* &&policy, *InIter1* first, *InIter2* last, *FwdIter* dest)

Relocates the elements in the range defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the move-construction of an element, all elements left in the input range, as well as all objects already constructed in the destination range are destroyed. After this algorithm completes, the source range should be freed or reused without destroying the objects.

The assignments in the parallel *uninitialized_relocate* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: time: O(n), space: O(1) 1) For “trivially relocatable” underlying types (T) and a contiguous iterator range [first, last): std::distance(first, last)*sizeof(T) bytes are copied. 2) For “trivially relocatable” underlying types (T) and a non-contiguous iterator range [first, last): std::distance(first, last) memory copies of sizeof(T) bytes each are performed. 3) For “non-trivially relocatable” underlying types (T): std::distance(first, last) move assignments and destructions are performed.

Note

Declare a type as "trivially relocatable" using the `HPX_DECLARE_TRIVIALLY_RELOCATABLE` macros found in <hpx/type_support/is_trivially_relocatable.hpp>.

Note

Parallel execution requires both *first* and *dest* to be contiguous iterators. If either iterator is non-contiguous, the algorithm cannot determine whether the source and destination ranges overlap, and will silently fall back to sequential execution regardless of the execution policy provided.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **InIter1** – The type of the source iterator first (deduced). This iterator type must meet the requirements of an input iterator.
- **InIter2** – The type of the source iterator last (deduced). This iterator type must meet the requirements of an input iterator.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range. The assignments in the parallel *uninitialized_relocate_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

Returns

The *uninitialized_relocate* algorithm returns a *hpx::future<FwdIter>*, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *uninitialized_relocate* algorithm returns the output iterator to the element in the destination range, one past the last element relocated.

hpx::uninitialized_relocate_n

Defined in header hpx/algorithm.hpp[678].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::uninitialized_relocate*

template<typename **InIter**, typename **Size**, typename **FwdIter**>

FwdIter `hpx::`**`uninitialized_relocate_n`**(*InIter* first, *Size* count, *FwdIter* dest)

Relocates the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the move-construction of an element, all elements left in the input range, as well as all objects already constructed in the destination range are destroyed. After this algorithm completes, the source range should be freed or reused without destroying the objects.

The assignments in the parallel *uninitialized_relocate_n* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

Note

Complexity: time: O(n), space: O(1) 1) For "trivially relocatable" underlying types (T) and a contiguous iterator range [first, first+count): `count*sizeof(T)` bytes are copied. 2) For "trivially relocatable" underlying types (T) and a non-contiguous iterator range [first, first+count): `count` memory copies of sizeof(T) bytes each are performed. 3) For "non-trivially relocatable" underlying types (T): `count` move assignments and destructions are performed.

Note

Declare a type as "trivially relocatable" using the `HPX_DECLARE_TRIVIALLY_RELOCATABLE` macros found in <hpx/type_support/is_trivially_relocatable.hpp>.

Template Parameters

- **`InIter`** – The type of the source iterator first (deduced). This iterator type must meet the requirements of an input iterator.
- **`Size`** – The type of the argument specifying the number of elements to relocate.
- **`FwdIter`** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

[678] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `count` – Refers to the number of elements starting at *first* the algorithm will be applied to.
- `dest` – Refers to the beginning of the destination range.

Returns

The *uninitialized_relocate_n* algorithm returns *FwdIter*. The *uninitialized_relocate_n* algorithm returns the output iterator to the element in the destination range, one past the last element relocated.

template<typename **ExPolicy**, typename **InIter**, typename **Size**, typename **FwdIter**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> *hpx*::**uninitialized_relocate_n**(*ExPolicy* &&policy, *InIter* first, *Size* count, *FwdIter* dest)

Relocates the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the move-construction of an element, all elements left in the input range, as well as all objects already constructed in the destination range are destroyed. After this algorithm completes, the source range should be freed or reused without destroying the objects.

The assignments in the parallel *uninitialized_relocate_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_relocate_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: time: O(n), space: O(1) 1) For “trivially relocatable” underlying types (T) and a contiguous iterator range [first, first+count): `count*sizeof(T)` bytes are copied. 2) For “trivially relocatable” underlying types (T) and a non-contiguous iterator range [first, first+count): `count` memory copies of sizeof(T) bytes each are performed. 3) For “non-trivially relocatable” underlying types (T): `count` move assignments and destructions are performed.

> **Note**
>
> Declare a type as “trivially relocatable” using the `HPX_DECLARE_TRIVIALLY_RELOCATABLE` macros found in <hpx/type_support/is_trivially_relocatable.hpp>.

Note

Parallel execution requires both *first* and *dest* to be contiguous iterators. If either iterator is non-contiguous, the algorithm cannot determine whether the source and destination ranges overlap, and will silently fall back to sequential execution regardless of the execution policy provided.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **InIter** – The type of the source iterator first (deduced). This iterator type must meet the requirements of an input iterator.
- **Size** – The type of the argument specifying the number of elements to relocate.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *uninitialized_relocate_n* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *uninitialized_relocate_n* algorithm returns the output iterator to the element in the destination range, one past the last element relocated.

hpx::remove_copy

Defined in header hpx/algorithm.hpp[679].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::remove_copy_if*

template<typename **InIter**, typename **OutIter**, typename **T** = typename *std*::iterator_traits<*InIter*>::value_type>

OutIter `hpx::`**`remove_copy`**(*InIter* first, *InIter* last, *OutIter* dest, *T* const &value)

Copies the elements in the range, defined by [first, last), to another range beginning at

[679] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

dest. Copies only the elements for which the comparison operator returns false when compare to value. The order of the elements that are not removed is preserved.

Effects: Copies all the elements referred to by the iterator it in the range [first,last) for which the following corresponding conditions do not hold: *it == value

The assignments in the parallel *remove_copy* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs not more than *last - first* assignments, exactly *last - first* applications of the predicate *pred*, here comparison operator.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **T** – The type that the result of dereferencing FwdIter1 is compared to.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **value** – Value to be removed.

Returns

The *remove_copy* algorithm returns an *OutIter*. The *remove_copy* algorithm returns the iterator to the element past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **T** = typename *std*::iterator_traits<InIter>::value_type>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> hpx::**remove_copy**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *T* const &value)

Copies the elements in the range, defined by [first, last), to another range beginning at *dest*. Copies only the elements for which the comparison operator returns false when compare to value. The order of the elements that are not removed is preserved. Executed according to the policy.

Effects: Copies all the elements referred to by the iterator it in the range [first,last) for which the following corresponding conditions do not hold: *it == value

The assignments in the parallel *remove_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *remove_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the predicate *pred*, here comparison operator.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type that the result of dereferencing FwdIter1 is compared to.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **value** – Value to be removed.

Returns

The *remove_copy* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *remove_copy* algorithm returns the iterator to the element past the last element copied.

hpx::remove_copy_if

Defined in header hpx/algorithm.hpp[680].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::remove_copy*

[680] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

template<typename **InIter**, typename **OutIter**, typename **Pred**>

OutIter hpx::**remove_copy_if**(*InIter* first, *InIter* last, *OutIter* dest, *Pred* &&pred)

Copies the elements in the range, defined by [first, last), to another range beginning at *dest*. Copies only the elements for which the predicate *pred* returns false. The order of the elements that are not removed is preserved.

Effects: Copies all the elements referred to by the iterator it in the range [first,last) for which the following corresponding conditions do not hold: INVOKE(pred, *it) != false.

The assignments in the parallel *remove_copy_if* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the predicate *pred*.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of the function/function object to use (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the elements to be removed. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.

Returns

The *remove_copy_if* algorithm returns an *OutIter* The *remove_copy_if* algorithm returns the iterator to the element past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Pred**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> `hpx::remove_copy_if`(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *Pred* &&pred)

Copies the elements in the range, defined by [first, last), to another range beginning at *dest*. Copies only the elements for which the predicate *pred* returns false. The order of the elements that are not removed is preserved. Executed according to the policy.

Effects: Copies all the elements referred to by the iterator it in the range [first,last) for which the following corresponding conditions do not hold: INVOKE(pred, *it) != false.

The assignments in the parallel *remove_copy_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *remove_copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the predicate *pred*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *remove_copy_if* requires *Pred* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns

true for the elements to be removed. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type.

Returns

The *remove_copy_if* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *remove_copy_if* algorithm returns the iterator to the element past the last element copied.

hpx::set_difference

Defined in header hpx/algorithm.hpp[681].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **FwdIter3**, typename **Pred** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter3*> `hpx::set_difference`(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *FwdIter3* dest, *Pred* &&op = *Pred*())

Constructs a sorted range beginning at dest consisting of all elements present in the range [first1, last1) and not present in the range [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *pred*. Executed according to the policy.

Equivalent elements are treated individually, that is, if some element is found *m* times in [first1, last1) and *n* times in [first2, last2), it will be copied to *dest* exactly std::max(m-n, 0) times. The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

The application of function objects in parallel algorithm invoked with a sequential execution policy object execute in sequential order in the calling thread (*sequenced_policy*) or in a single new thread spawned from the current thread (for *sequenced_task_policy*).

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

[681] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Note

Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **FwdIter1** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **FwdIter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a output iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_difference* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*

Returns

The *set_difference* algorithm returns a *hpx::future<FwdIter3>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter3* otherwise. The *set_difference* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **FwdIter1**, typename **FwdIter2**, typename **FwdIter3**, typename **Pred** = *hpx*::*parallel*::detail::less>

FwdIter3 **hpx::set_difference**(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *FwdIter3* dest, *Pred* &&op = *Pred*())

Constructs a sorted range beginning at dest consisting of all elements present in the range [first1, last1) and not present in the range [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *pred*.

Equivalent elements are treated individually, that is, if some element is found *m* times in [first1, last1) and *n* times in [first2, last2), it will be copied to *dest* exactly std::max(m-n, 0) times. The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

Note

Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **FwdIter1** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **FwdIter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a output iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_difference* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*

Returns

The *set_difference* algorithm returns a *FwdIter3*. The *set_difference* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::reduce_by_key

Defined in header hpx/algorithm.hpp[682].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

> **Warning**
>
> doxygenfunction: Cannot find function "hpx::reduce_by_key" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::nth_element

Defined in header hpx/algorithm.hpp[683].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **RandomIt**, typename **Pred** = *hpx*::*parallel*::detail::less>

void *hpx*::**nth_element**(*RandomIt* first, *RandomIt* nth, *RandomIt* last, *Pred* &&pred = *Pred*())

nth_element is a partial sorting algorithm that rearranges elements in [first, last) such that the element pointed at by nth is changed to whatever element would occur in that position if [first, last) were sorted and all of the elements before this new nth element are less than or equal to the elements after the new nth element. Executed according to the policy.

The comparison operations in the parallel *nth_element* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear in std::distance(first, last) on average. O(N) applications of the predicate, and O(N log N) swaps, where N = last - first.

Template Parameters

- **RandomIt** – The type of the source begin, nth, and end iterators used (deduced). This iterator type must meet the requirements of a random access iterator.

[682] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp
[683] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **Pred** – Comparison function object which returns true if the first argument is less than the second. This defaults to std::less<>.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **nth** – Refers to the iterator defining the sort partition point
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the comparison function object which returns true if the first argument is less than (i.e. is ordered before) the second. The signature of this comparison function should be equivalent to:

```
bool cmp(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type must be such that an object of type *randomIt* can be dereferenced and then implicitly converted to Type. This defaults to std::less<>.

Returns

The *nth_element* algorithms returns nothing.

template<typename **ExPolicy**, typename **RandomIt**, typename **Pred** = *hpx*::*parallel*::detail::less>

void *hpx*::**nth_element**(*ExPolicy* &&policy, *RandomIt* first, *RandomIt* nth, *RandomIt* last, *Pred* &&pred = *Pred*())

nth_element is a partial sorting algorithm that rearranges elements in [first, last) such that the element pointed at by nth is changed to whatever element would occur in that position if [first, last) were sorted and all of the elements before this new nth element are less than or equal to the elements after the new nth element.

The comparison operations in the parallel *nth_element* invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *nth_element* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Linear in std::distance(first, last) on average. O(N) applications of the predicate, and O(N log N) swaps, where N = last - first.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.

- **RandomIt** – The type of the source begin, nth, and end iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **Pred** – Comparison function object which returns true if the first argument is less than the second. This defaults to std::less<>.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **nth** – Refers to the iterator defining the sort partition point
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the comparison function object which returns true if the first argument is less than (i.e. is ordered before) the second. The signature of this comparison function should be equivalent to:

```
bool cmp(const Type1 &a, const Type2 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type must be such that an object of type *randomIt* can be dereferenced and then implicitly converted to Type. This defaults to std::less<>.

Returns

The *nth_element* algorithms returns nothing.

hpx::remove

Defined in header hpx/algorithm.hpp[684].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::remove_if*

template<typename **FwdIter**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>

FwdIter hpx::**remove**(*FwdIter* first, *FwdIter* last, *T* const &value)

Removes all elements satisfying specific criteria from the range [first, last) and returns a past-the-end iterator for the new end of the range. This version removes all elements that are equal to *value*.

The assignments in the parallel *remove* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the operator==().

[684] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to remove (deduced). This value type must meet the requirements of *CopyConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **value** – Specifies the value of elements to remove.

Returns

The *remove* algorithm returns a *FwdIter*. The *remove* algorithm returns the iterator to the new end of the range.

template<typename **ExPolicy**, typename **FwdIter**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> **hpx::remove**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *T* const &value)

Removes all elements satisfying specific criteria from the range [first, last) and returns a past-the-end iterator for the new end of the range. This version removes all elements that are equal to *value*. Executed according to the policy.

The assignments in the parallel *remove* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *remove* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the operator==().

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to remove (deduced). This value type must meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **value** – Specifies the value of elements to remove.

Returns

The *remove* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *remove* algorithm returns the iterator to the new end of the range.

hpx::remove_if

Defined in header hpx/algorithm.hpp[685].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::remove*

template<typename **FwdIter**, typename **Pred**>

FwdIter `hpx::remove_if`(*FwdIter* first, *FwdIter* last, *Pred* &&pred)

Removes all elements satisfying specific criteria from the range [first, last) and returns a past-the-end iterator for the new end of the range. This version removes all elements for which predicate *pred* returns true.

The assignments in the parallel *remove_if* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the predicate *pred*.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *remove_if* requires *Pred* to meet the requirements of *CopyConstructible*.

Parameters

[685] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

Returns

The *remove_if* algorithm returns a *FwdIter*. The *remove_if* algorithm returns the iterator to the new end of the range.

template<typename **ExPolicy**, typename **FwdIter**, typename **Pred**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::remove_if`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *Pred* &&pred)

Removes all elements satisfying specific criteria from the range [first, last) and returns a past-the-end iterator for the new end of the range. This version removes all elements for which predicate *pred* returns true. Executed according to the policy.

The assignments in the parallel *remove_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *remove_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the predicate *pred*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *remove_if* requires *Pred* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

Returns

The *remove_if* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *remove_if* algorithm returns the iterator to the new end of the range.

hpx::reverse

Defined in header hpx/algorithm.hpp[686].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::reverse_copy*

template<typename **BidirIter**>

void *hpx*::**reverse**(*BidirIter* first, *BidirIter* last)

Reverses the order of the elements in the range [first, last). Behaves as if applying *std::iter_swap* to every pair of iterators first+i, (*last-i*) - 1 for each non-negative i < (last-first)/2.

The assignments in the parallel *reverse* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*.

Template Parameters

BidirIter – The type of the source iterators used (deduced). This iterator type must meet the requirements of a bidirectional iterator.

[686] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *reverse* algorithm returns *void*.

template<typename **ExPolicy**, typename **BidirIter**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, void> `hpx::reverse`(*ExPolicy* &&policy, *BidirIter* first, *BidirIter* last)

Reverses the order of the elements in the range [first, last). Behaves as if applying *std::iter_swap* to every pair of iterators first+i, (last-i) - 1 for each non-negative i < (last-first)/2. Executed according to the policy.

The assignments in the parallel *reverse* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *reverse* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **BidirIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a bidirectional iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *reverse* algorithm returns a *hpx::future<void>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *void* otherwise.

hpx::reverse_copy

Defined in header hpx/algorithm.hpp[687].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::reverse*

template<typename **BidirIter**, typename **OutIter**>

OutIter `hpx::reverse_copy`(*BidirIter* first, *BidirIter* last, *OutIter* dest)

Copies the elements from the range [first, last) to another range beginning at dest in such a way that the elements in the new range are in reverse order. Behaves as if by executing the assignment *(dest + (last - first) - 1 - i) = *(first + i) once for each non-negative i < (last - first) If the source and destination ranges (that is, [first, last) and [dest, dest+(last-first)) respectively) overlap, the behavior is undefined.

The assignments in the parallel *reverse_copy* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **BidirIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a bidirectional iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the begin of the destination range.

Returns

The *reverse_copy* algorithm returns an *OutIter*. The *reverse_copy* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **BidirIter**, typename **FwdIter**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::reverse_copy`(*ExPolicy* &&policy, *BidirIter* first, *BidirIter* last, *FwdIter* dest)

[687] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Copies the elements from the range [first, last) to another range beginning at dest in such a way that the elements in the new range are in reverse order. Behaves as if by executing the assignment *(dest + (last - first) - 1 - i) = *(first + i) once for each non-negative i < (last - first) If the source and destination ranges (that is, [first, last) and [dest, dest+(last-first)) respectively) overlap, the behavior is undefined. Executed according to the policy.

The assignments in the parallel *reverse_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *reverse_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **BidirIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a bidirectional iterator.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the begin of the destination range.

Returns

The *reverse_copy* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *reverse_copy* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::experimental::sort_by_key

Defined in header hpx/algorithm.hpp[688].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **KeyIter**, typename **ValueIter**, typename **Compare** = detail::less>

util::detail::algorithm_result_t<*ExPolicy*, sort_by_key_result<*KeyIter*, *ValueIter*>> `hpx::experimental::sort_by_key`(*ExPolicy* &&policy, *KeyIter* key_first, *KeyIter* key_last, *ValueIter* value_first, *Compare* &&comp = *Compare*())

Sorts one range of data using keys supplied in another range. The key elements in the range [key_first, key_last) are sorted in ascending order with the corresponding elements in the value range moved to follow the sorted order. The algorithm is not stable, the order of equal elements is not guaranteed to be preserved. The function uses the given comparison function object comp (defaults to using operator<()). Executed according to the policy.

A sequence is sorted with respect to a comparator *comp* if for every iterator i pointing to the sequence and every non-negative integer n such that i + n is a valid iterator pointing to an element of the sequence, and INVOKE(comp, *(i + n), *i) == false.

comp has to induce a strict weak ordering on the values.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(N log(N)), where N = std::distance(first, last) comparisons.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it

[688] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

applies user-provided function objects.

- **KeyIter** – The type of the key iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **ValueIter** – The type of the value iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **Compare** – The type of the function/function object to use (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **key_first** – Refers to the beginning of the sequence of key elements the algorithm will be applied to.
- **key_last** – Refers to the end of the sequence of key elements the algorithm will be applied to.
- **value_first** – Refers to the beginning of the sequence of value elements the algorithm will be applied to, the range of elements must match [key_first, key_last)
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.

Returns

The *sort_by_key* algorithm returns a hpx::future<*sort_by_key_result<KeyIter,ValueIter>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *otherwise*. The algorithm returns a pair holding an iterator pointing to the first element after the last element in the input key sequence and an iterator pointing to the first element after the last element in the input value sequence.

hpx::replace

Defined in header hpx/algorithm.hpp[689].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::replace_if*
- *hpx::replace_copy*
- *hpx::replace_copy_if*

template<typename **InIter**, typename **T** = typename *std*::iterator_traits<*InIter*>::value_type>

void *hpx*::**replace**(*InIter* first, *InIter* last, *T* const &old_value, *T* const &new_value)

Replaces all elements satisfying specific criteria with *new_value* in the range [first, last).

Effects: Substitutes elements referred by the iterator it in the range [first, last) with new_value, when the following corresponding conditions hold: *it == old_value

[689] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The assignments in the parallel *replace* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **T** – The type of the old and new values to replace (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **old_value** – Refers to the old value of the elements to replace.
- **new_value** – Refers to the new value to use as the replacement.

Returns

The *replace* algorithm returns a *void*.

template<typename **ExPolicy**, typename **FwdIter**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, void> *hpx*::**replace**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *T* const &old_value, *T* const &new_value)

Replaces all elements satisfying specific criteria with *new_value* in the range [first, last). Executed according to the policy.

Effects: Substitutes elements referred by the iterator it in the range [first, last) with new_value, when the following corresponding conditions hold: *it == old_value

The assignments in the parallel *replace* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *replace* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **T** – The type of the old and new values to replace (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **old_value** – Refers to the old value of the elements to replace.
- **new_value** – Refers to the new value to use as the replacement.

Returns

The *replace* algorithm returns a *hpx::future<void>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *void* otherwise.

hpx::replace_if

Defined in header hpx/algorithm.hpp[690].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::replace*
- *hpx::replace_copy*
- *hpx::replace_copy_if*

template<typename **Iter**, typename **Pred**, typename **T** = typename *std*::iterator_traits<*Iter*>::value_type>

void `hpx::replace_if`(*Iter* first, *Iter* last, *Pred* &&pred, *T* const &new_value)

Replaces all elements satisfying specific criteria (for which predicate *pred* returns true) with *new_value* in the range [first, last).

Effects: Substitutes elements referred by the iterator it in the range [first, last) with new_value, when the following corresponding conditions hold: INVOKE(f, *it) != false

The assignments in the parallel *replace_if* algorithm execute in sequential order in the calling thread.

[690] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

> **Note**
>
> Complexity: Performs exactly *last* - *first* applications of the predicate.

Template Parameters

- **Iter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. (deduced).
- **T** – The type of the new values to replace (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the elements which need to replaced. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to *Type*.
- **new_value** – Refers to the new value to use as the replacement.

Returns

The *replace_if* algorithm returns *void*.

template<typename **ExPolicy**, typename **FwdIter**, typename **Pred**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, void> hpx::**replace_if**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *Pred* &&pred, *T* const &new_value)

Replaces all elements satisfying specific criteria (for which predicate *f* returns true) with *new_value* in the range [first, last). Executed according to the policy.

Effects: Substitutes elements referred by the iterator it in the range [first, last) with new_value, when the following corresponding conditions hold: INVOKE(f, *it) != false

The assignments in the parallel *replace_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *replace_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an

unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* applications of the predicate.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. (deduced).
- **T** – The type of the new values to replace (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the elements which need to replaced. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.
- **new_value** – Refers to the new value to use as the replacement.

Returns

The *replace_if* algorithm returns a *hpx::future<void>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *void* otherwise.

hpx::replace_copy

Defined in header hpx/algorithm.hpp[691].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::replace*
- *hpx::replace_if*
- *hpx::replace_copy_if*

template<typename **InIter**, typename **OutIter**, typename **T** = typename *std*::iterator_traits<*OutIter*>::value_type>

OutIter `hpx::replace_copy`(*InIter* first, *InIter* last, *OutIter* dest, *T* const &old_value, *T* const &new_value)

Copies the all elements from the range [first, last) to another range beginning at *dest* replacing all elements satisfying a specific criteria with *new_value*.

Effects: Assigns to every iterator it in the range [result, result + (last - first)) either new_value or *(first + (it - result)) depending on whether the following corresponding condition holds: *(first + (i - result)) == old_value

The assignments in the parallel *replace_copy* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* applications of the predicate.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **T** – The type of the old and new values (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **old_value** – Refers to the old value of the elements to replace.
- **new_value** – Refers to the new value to use as the replacement.

[691] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Returns

The *replace_copy* algorithm returns an *OutIter* The *replace_copy* algorithm returns the Iterator to the element past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **T** = typename *std*::iterator_traits<*FwdIter2*>::value_type>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> `hpx::replace_copy`(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *T* const &old_value, *T* const &new_value)

Copies the all elements from the range [first, last) to another range beginning at *dest* replacing all elements satisfying a specific criteria with *new_value*. Executed according to the policy.

Effects: Assigns to every iterator it in the range [result, result + (last - first)) either new_value or *(first + (it - result)) depending on whether the following corresponding condition holds: *(first + (i - result)) == old_value

The assignments in the parallel *replace_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *replace_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last - first* applications of the predicate.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the old and new values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

- **dest** – Refers to the beginning of the destination range.
- **old_value** – Refers to the old value of the elements to replace.
- **new_value** – Refers to the new value to use as the replacement.

Returns

The *replace_copy* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *replace_copy* algorithm returns the Iterator to the element past the last element copied.

hpx::replace_copy_if

Defined in header hpx/algorithm.hpp[692].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::replace*
- *hpx::replace_if*
- *hpx::replace_copy*

template<typename **InIter**, typename **OutIter**, typename **Pred**, typename **T** = typename *std*::iterator_traits<*OutIter*>::value_type>

OutIter hpx::**replace_copy_if**(*InIter* first, *InIter* last, *OutIter* dest, *Pred* &&pred, *T* const &new_value)

Copies the all elements from the range [first, last) to another range beginning at *dest* replacing all elements satisfying a specific criteria with *new_value*.

Effects: Assigns to every iterator it in the range [result, result + (last - first)) either new_value or *(first + (it - result)) depending on whether the following corresponding condition holds: INVOKE(f, *(first + (i - result))) != false

The assignments in the parallel *replace_copy_if* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last - first* applications of the predicate.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.

692 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. (deduced).
- **T** – The type of the new values to replace (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the elements which need to replaced. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to *Type*.
- **new_value** – Refers to the new value to use as the replacement.

Returns

The *replace_copy_if* algorithm returns an *OutIter*. The *replace_copy_if* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **Pred**, typename **T** = typename *std*::iterator_traits<*FwdIter2*>::value_type>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> hpx::**replace_copy_if**(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *Pred* &&pred, *T* const &new_value)

Copies the all elements from the range [first, last) to another range beginning at *dest* replacing all elements satisfying a specific criteria with *new_value*.

Effects: Assigns to every iterator it in the range [result, result + (last - first)) either new_value or *(first + (it - result)) depending on whether the following corresponding condition holds: INVOKE(f, *(first + (i - result))) != false

The assignments in the parallel *replace_copy_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *replace_copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* applications of the predicate.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *replace_copy_if* requires *Pred* to meet the requirements of *CopyConstructible*. (deduced).
- **T** – The type of the new values to replace (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the elements which need to replaced. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to *Type*.
- **new_value** – Refers to the new value to use as the replacement.

Returns

The *replace_copy_if* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *replace_copy_if* algorithm returns the iterator to the element in the destination range, one past the last element copied.

hpx::sort

Defined in header hpx/algorithm.hpp[693].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **RandomIt**, typename **Comp** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

void *hpx*::**sort**(*RandomIt* first, *RandomIt* last, *Comp* &&comp, *Proj* &&proj = *Proj*())

Sorts the elements in the range [first, last) in ascending order. The order of equal elements is not guaranteed to be preserved. The function uses the given comparison function object comp (defaults to using operator<()).

A sequence is sorted with respect to a comparator *comp* and a projection *proj* if for every iterator i pointing to the sequence and every non-negative integer n such that i + n is a valid iterator pointing to an element of the sequence, and INVOKE(comp, INVOKE(proj, *(i + n)), INVOKE(proj, *i)) == false.

comp has to induce a strict weak ordering on the values.

The assignments in the parallel *sort* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Note

Complexity: O(N log(N)), where N = std::distance(first, last) comparisons.

Template Parameters

- **RandomIt** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type *Comp*, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *sort* algorithm returns *void*.

[693] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

template<typename **ExPolicy**, typename **RandomIt**, typename **Comp** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*> **hpx::sort**(*ExPolicy* &&policy, *RandomIt* first, *RandomIt* last, *Comp* &&comp, *Proj* &&proj)

Sorts the elements in the range [first, last) in ascending order. The order of equal elements is not guaranteed to be preserved. The function uses the given comparison function object comp (defaults to using operator<()). Executed according to the policy.

A sequence is sorted with respect to a comparator *comp* and a projection *proj* if for every iterator i pointing to the sequence and every non-negative integer n such that i + n is a valid iterator pointing to an element of the sequence, and INVOKE(comp, INVOKE(proj, *(i + n)), INVOKE(proj, *i)) == false.

comp has to induce a strict weak ordering on the values.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(N log(N)), where N = std::distance(first, last) comparisons.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **RandomIt** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type *Comp*, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *sort* algorithm returns a *hpx::future<void>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *void* otherwise.

hpx::partition

Defined in header hpx/algorithm.hpp[694].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::stable_partition*
- *hpx::partition_copy*

template<typename **FwdIter**, typename **Pred**, typename **Proj** = *hpx*::identity>

FwdIter **hpx::partition**(*FwdIter* first, *FwdIter* last, *Pred* &&pred, *Proj* &&proj = *Proj*())

Reorders the elements in the range [first, last) in such a way that all elements for which the predicate *pred* returns true precede the elements for which the predicate *pred* returns false. Relative order of the elements is not preserved.

The assignments in the parallel *partition* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Note

Complexity: At most 2 * (last - first) swaps. Exactly *last* - *first* applications of the predicate and projection.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an unary predicate for partitioning the source iterators. The signature of this predicate should be equivalent to:

[694] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *partition* algorithm returns returns *FwdIter*. The *partition* algorithm returns the iterator to the first element of the second group.

template<typename **ExPolicy**, typename **FwdIter**, typename **Pred**, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> hpx::**partition**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *Pred* &&pred, *Proj* &&proj = *Proj*())

Reorders the elements in the range [first, last) in such a way that all elements for which the predicate *pred* returns true precede the elements for which the predicate *pred* returns false. Relative order of the elements is not preserved.

The assignments in the parallel *partition* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *partition* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most 2 * (last - first) swaps. Exactly *last* - *first* applications of the predicate and projection.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an unary predicate for partitioning the source iterators. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *partition* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *parallel_task_policy* and returns *FwdIter* otherwise. The *partition* algorithm returns the iterator to the first element of the second group.

hpx::stable_partition

Defined in header hpx/algorithm.hpp[695].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::partition*
- *hpx::partition_copy*

template<typename **BidirIter**, typename **F**, typename **Proj** = *hpx*::identity>

BidirIter hpx::**stable_partition**(*BidirIter* first, *BidirIter* last, *F* &&f, *Proj* &&proj = *Proj*())

Permutes the elements in the range [first, last) such that there exists an iterator i such that for every iterator j in the range [first, i) INVOKE(f, INVOKE (proj, *j)) != false, and for every iterator k in the range [i, last), INVOKE(f, INVOKE (proj, *k)) == false

The invocations of *f* in the parallel *stable_partition* algorithm invoked without an execution policy object executes in sequential order in the calling thread.

> **Note**
>
> Complexity: At most (last - first) * log(last - first) swaps, but only linear number of swaps if there is enough extra memory. Exactly *last* - *first* applications of the predicate and projection.

Template Parameters

[695] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **BidirIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a bidirectional iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *transform* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Unary predicate which returns true if the element should be ordered before other elements. Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

  ```
  bool fun(const Type &a);
  ```

  The signature does not need to have const&. The type *Type* must be such that an object of type *BidirIter* can be dereferenced and then implicitly converted to *Type*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *f* is invoked.

Returns

The *stable_partition* algorithm returns an iterator i such that for every iterator j in the range [first, i), f(*j) != false INVOKE(f, INVOKE(proj, *j)) != false, and for every iterator k in the range [i, last), f(*k) == false INVOKE(f, INVOKE (proj, *k)) == false. The relative order of the elements in both groups is preserved.

template<typename **ExPolicy**, typename **BidirIter**, typename **F**, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *BidirIter*> **hpx::stable_partition**(*ExPolicy* &&policy, *BidirIter* first, *BidirIter* last, *F* &&f, *Proj* &&proj = *Proj*())

Permutes the elements in the range [first, last) such that there exists an iterator i such that for every iterator j in the range [first, i) INVOKE(f, INVOKE (proj, *j)) != false, and for every iterator k in the range [i, last), INVOKE(f, INVOKE (proj, *k)) == false

The invocations of *f* in the parallel *stable_partition* algorithm invoked with an execution policy object of type *sequenced_policy* executes in sequential order in the calling thread.

The invocations of *f* in the parallel *stable_partition* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most (last - first) * log(last - first) swaps, but only linear number of swaps if there is enough extra memory. Exactly *last* - *first* applications of the predicate and projection.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the invocations of *f*.
- **BidirIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a bidirectional iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *transform* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Unary predicate which returns true if the element should be ordered before other elements. Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

  ```
  bool fun(const Type &a);
  ```

  The signature does not need to have const&. The type *Type* must be such that an object of type *BidirIter* can be dereferenced and then implicitly converted to *Type*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *f* is invoked.

Returns

The *stable_partition* algorithm returns an iterator i such that for every iterator j in the range [first, i), f(*j) != false INVOKE(f, INVOKE(proj, *j)) != false, and for every iterator k in the range [i, last), f(*k) == false INVOKE(f, INVOKE (proj, *k)) == false. The relative order of the elements in both groups is preserved. If the execution policy is of type *parallel_task_policy* the algorithm returns a *future*<> referring to this iterator.

hpx::partition_copy

Defined in header hpx/algorithm.hpp[696].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::partition*
- *hpx::stable_partition*

template<typename **FwdIter1**, typename **FwdIter2**, typename **FwdIter3**, typename **Pred**, typename **Proj** = *hpx*::identity>

std::pair<*FwdIter2*, *FwdIter3*> **hpx::partition_copy**(*FwdIter1* first, *FwdIter1* last, *FwdIter2* dest_true, *FwdIter3* dest_false, *Pred* &&pred, *Proj* &&proj = *Proj*())

Copies the elements in the range, defined by [first, last), to two different ranges depending on the value returned by the predicate *pred*. The elements, that satisfy the predicate *pred* are copied to the range beginning at *dest_true*. The rest of the elements are copied to the range beginning at *dest_false*. The order of the elements is preserved.

The assignments in the parallel *partition_copy* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the predicate *pred*.

Template Parameters

- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range for the elements that satisfy the predicate *pred* (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter3** – The type of the iterator representing the destination range for the elements that don't satisfy the predicate *pred* (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition_copy* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.

[696] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest_true** – Refers to the beginning of the destination range for the elements that satisfy the predicate *pred*
- **dest_false** – Refers to the beginning of the destination range for the elements that don't satisfy the predicate *pred*.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an unary predicate for partitioning the source iterators. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *partition_copy* algorithm returns *std::pair<OutIter1, OutIter2>*. The *partition_copy* algorithm returns the pair of the destination iterator to the end of the *dest_true* range, and the destination iterator to the end of the *dest_false* range.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **FwdIter3**, typename **Pred**, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *std*::pair<*FwdIter2*, *FwdIter3*>> `hpx::partition_copy`(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest_true, *FwdIter3* dest_false, *Pred* &&pred, *Proj* &&proj = *Proj*())

Copies the elements in the range, defined by [first, last), to two different ranges depending on the value returned by the predicate *pred*. The elements, that satisfy the predicate *pred*, are copied to the range beginning at *dest_true*. The rest of the elements are copied to the range beginning at *dest_false*. The order of the elements is preserved.

The assignments in the parallel *partition_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *partition_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an

unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the predicate *pred*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range for the elements that satisfy the predicate *pred* (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter3** – The type of the iterator representing the destination range for the elements that don't satisfy the predicate *pred* (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition_copy* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest_true** – Refers to the beginning of the destination range for the elements that satisfy the predicate *pred*
- **dest_false** – Refers to the beginning of the destination range for the elements that don't satisfy the predicate *pred*.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an unary predicate for partitioning the source iterators. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *partition_copy* algorithm returns a hpx::future<*std::pair<OutIter1, OutIter2*>> if the execution policy is of type *parallel_task_policy* and returns *std::pair<OutIter1, OutIter2*> otherwise. The *partition_copy* algorithm returns the pair of the destination iterator to the end of the *dest_true* range, and the destination iterator to the end of the *dest_false* range.

hpx::transform_reduce

Defined in header hpx/algorithm.hpp[697].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **FwdIter**, typename **T**, typename **Reduce**, typename **Convert**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> `hpx::transform_reduce`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *T* init, *Reduce* &&red_op, *Convert* &&conv_op)

Returns GENERALIZED_SUM(red_op, init, conv_op(*first), …, conv_op(*(first + (last - first) - 1))). Executed according to the policy.

The reduce operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *transform_reduce* and *accumulate* is that the behavior of transform_reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicates *red_op* and *conv_op*.

> **Note**
>
> GENERALIZED_SUM(op, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and

[697] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **Reduce** – The type of the binary function object used for the reduction operation.
- **Convert** – The type of the unary function object used to transform the elements of the input sequence before invoking the reduce function.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **init** – The initial value for the generalized sum.
- **red_op** – Specifies the function (or function object) which will be invoked for each of the values returned from the invocation of *conv_op*. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1*, *Type2*, and *Ret* must be such that an object of a type as returned from *conv_op* can be implicitly converted to any of those types.
- **conv_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

  ```
  R fun(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *parallel_task_policy* and returns *T* otherwise. The *transform_reduce* algorithm returns the result of the generalized sum over the values returned from *conv_op* when applied to the elements given by the input range [first, last).

template<typename **InIter**, typename **T**, typename **Reduce**, typename **Convert**>

T *hpx*::**transform_reduce**(*InIter* first, *InIter* last, *T* init, *Reduce* &&red_op, *Convert* &&conv_op)

Returns GENERALIZED_SUM(red_op, init, conv_op(*first), ..., conv_op(*(first + (last - first) - 1))).

The difference between *transform_reduce* and *accumulate* is that the behavior of *transform_reduce* may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicates *red_op* and *conv_op*.

> **Note**
>
> GENERALIZED_SUM(op, a1, ..., aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(op, b1, ..., bK), GENERALIZED_SUM(op, bM, ..., bN)), where:
>   - b1, ..., bN may be any permutation of a1, ..., aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **Reduce** – The type of the binary function object used for the reduction operation.
- **Convert** – The type of the unary function object used to transform the elements of the input sequence before invoking the reduce function.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **init** – The initial value for the generalized sum.
- **red_op** – Specifies the function (or function object) which will be invoked for each of the values returned from the invocation of *conv_op*. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type2 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1*, *Type2*, and *Ret* must be such that an object of a type as returned from *conv_op* can be implicitly converted to any of those types.

- **conv_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

```
R fun(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_reduce* algorithm returns a *T*. The *transform_reduce* algorithm returns the result of the generalized sum over the values returned from *conv_op* when applied to the elements given by the input range [first, last).

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> **hpx::transform_reduce**(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *T* init)

Returns the result of accumulating init with the inner products of the pairs formed by the elements of two ranges starting at first1 and first2. Executed according to the policy.

The operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(*last* - *first*) applications each of *reduce* and *transform*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the first source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the second source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **T** – The type of the value to be used as return) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the first sequence of elements the result will be calculated with.
- **last1** – Refers to the end of the first sequence of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second sequence of elements the result will be calculated with.
- **init** – The initial value for the sum.

Returns

The *transform_reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *T* otherwise.

template<typename **InIter1**, typename **InIter2**, typename **T**>

T hpx::**transform_reduce**(*InIter1* first1, *InIter1* last1, *InIter2* first2, *T* init)

Returns the result of accumulating init with the inner products of the pairs formed by the elements of two ranges starting at first1 and first2.

> **Note**
>
> Complexity: O(*last* - *first*) applications each of *reduce* and *transform*.

Template Parameters

- **InIter1** – The type of the first source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **InIter2** – The type of the second source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **T** – The type of the value to be used as return) values (deduced).

Parameters

- **first1** – Refers to the beginning of the first sequence of elements the result will be calculated with.
- **last1** – Refers to the end of the first sequence of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second sequence of elements the result will be calculated with.
- **init** – The initial value for the sum.

Returns

The *transform_reduce* algorithm returns a *T*.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **T**, typename **Reduce**, typename **Convert**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> `hpx::transform_reduce`(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *T* init, *Reduce* &&red_op, *Convert* &&conv_op)

Returns the result of accumulating init with the inner products of the pairs formed by the elements of two ranges starting at first1 and first2. Executed according to the policy.

The operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(*last* - *first*) applications each of *reduce* and *transform*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the first source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the second source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **T** – The type of the value to be used as return) values (deduced).
- **Reduce** – The type of the binary function object used for the multiplication operation.
- **Convert** – The type of the unary function object used to transform the elements of the input sequence before invoking the reduce function.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the first sequence of elements the result will be calculated with.
- **last1** – Refers to the end of the first sequence of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second sequence of elements the result will be calculated with.
- **init** – The initial value for the sum.

- **red_op** – Specifies the function (or function object) which will be invoked for the initial value and each of the return values of *conv_op*. This is a binary predicate. The signature of this predicate should be equivalent to should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Ret* must be such that it can be implicitly converted to a type of *T*.

- **conv_op** – Specifies the function (or function object) which will be invoked for each of the input values of the sequence. This is a binary predicate. The signature of this predicate should be equivalent to

  ```
  Ret fun(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Ret* must be such that it can be implicitly converted to an object for the second argument type of *red_op*.

Returns

The *transform_reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *T* otherwise.

template<typename **InIter1**, typename **InIter2**, typename **T**, typename **Reduce**, typename **Convert**>

T *hpx*::**transform_reduce**(ExPolicy &&policy, *InIter1* first1, *InIter1* last1, *InIter2* first2, *T* init, *Reduce* &&red_op, *Convert* &&conv_op)

Returns the result of accumulating init with the inner products of the pairs formed by the elements of two ranges starting at first1 and first2.

> **Note**
>
> Complexity: O(*last* - *first*) applications each of *reduce* and *transform*.

Template Parameters

- **InIter1** – The type of the first source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **InIter2** – The type of the second source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **T** – The type of the value to be used as return) values (deduced).
- **Reduce** – The type of the binary function object used for the multiplication operation.
- **Convert** – The type of the unary function object used to transform the elements of the input sequence before invoking the reduce function.

Parameters

- **first1** – Refers to the beginning of the first sequence of elements the result will be calculated with.
- **last1** – Refers to the end of the first sequence of elements the algorithm will be applied to.

- **first2** – Refers to the beginning of the second sequence of elements the result will be calculated with.
- **init** – The initial value for the sum.
- **red_op** – Specifies the function (or function object) which will be invoked for the initial value and each of the return values of *conv_op*. This is a binary predicate. The signature of this predicate should be equivalent to should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Ret* must be such that it can be implicitly converted to a type of *T*.
- **conv_op** – Specifies the function (or function object) which will be invoked for each of the input values of the sequence. This is a binary predicate. The signature of this predicate should be equivalent to

```
Ret fun(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Ret* must be such that it can be implicitly converted to an object for the second argument type of *red_op*.

Returns

The *transform_reduce* algorithm returns a *T*.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **T**, typename **Reduce**, typename **Convert**>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> `hpx::`**`transform_reduce`**(*ExPolicy* &&policy, *Iter* first, *Sent* last, *T* init, *Reduce* &&red_op, *Convert* &&conv_op)

Returns GENERALIZED_SUM(red_op, init, conv_op(*first), …, conv_op(*(first + (last - first) - 1))).

The reduce operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *transform_reduce* and *accumulate* is that the behavior of transform_reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicates *red_op* and *conv_op*.

Note

GENERALIZED_SUM(op, a1, …, aN) is defined as follows:

- a1 when N is 1
- op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
  - b1, …, bN may be any permutation of a1, …, aN and
  - 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Iter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random access iterator.
- **Sent** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **Reduce** – The type of the binary function object used for the reduction operation.
- **Convert** – The type of the unary function object used to transform the elements of the input sequence before invoking the reduce function.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the first sorted range the algorithm will be applied to.
- **last** – Refers to the end of the second sorted range the algorithm will be applied to.
- **init** – The initial value for the generalized sum.
- **red_op** – Specifies the function (or function object) which will be invoked for each of the values returned from the invocation of *conv_op*. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1*, *Type2*, and *Ret* must be such that an object of a type as returned from *conv_op* can be implicitly converted to any of those types.
- **conv_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

  ```
  R fun(const Type &a);
  ```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *Iter* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *parallel_task_policy* and returns *T* otherwise. The *transform_reduce* algorithm returns the result of the generalized sum over the values returned from *conv_op* when applied to the elements given by the input range [first, last).

template<typename **Iter**, typename **Sent**, typename **T**, typename **Reduce**, typename **Convert**>

T `hpx::transform_reduce`(*Iter* first, *Sent* last, *T* init, *Reduce* &&red_op, *Convert* &&conv_op)

Returns GENERALIZED_SUM(red_op, init, conv_op(*first), …, conv_op(*(first + (last - first) - 1))).

The difference between *transform_reduce* and *accumulate* is that the behavior of transform_reduce may be non-deterministic for non-associative or non-commutative binary predicate.

Note

Complexity: O(*last* - *first*) applications of the predicates *red_op* and *conv_op*.

Note

GENERALIZED_SUM(op, a1, …, aN) is defined as follows:

- a1 when N is 1
- op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
  - b1, …, bN may be any permutation of a1, …, aN and
  - 1 < K+1 = M <= N.

Template Parameters

- **Iter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random access iterator.
- **Sent** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **Reduce** – The type of the binary function object used for the reduction operation.
- **Convert** – The type of the unary function object used to transform the elements of the input sequence before invoking the reduce function.

Parameters

- **first** – Refers to the beginning of the first sorted range the algorithm will be applied to.
- **last** – Refers to the end of the second sorted range the algorithm will be applied to.
- **init** – The initial value for the generalized sum.
- **red_op** – Specifies the function (or function object) which will be invoked for each of the values returned from the invocation of *conv_op*. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1*, *Type2*, and *Ret* must be such that an object of a type as returned from *conv_op* can be implicitly converted to any of those types.
- **conv_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

  ```
  R fun(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *Iter* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_reduce* algorithm returns *T*. The *transform_reduce* algorithm returns the result of the generalized sum over the values returned from *conv_op* when applied to the elements given by the input range [first, last).

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **Iter2**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> `hpx::transform_reduce`(*ExPolicy* &&policy, *Iter* first, *Sent* last, *Iter2* first2, *T* init)

Returns GENERALIZED_SUM(red_op, init, conv_op(*first), …, conv_op(*(first + (last - first) - 1))).

The reduce operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *transform_reduce* and *accumulate* is that the behavior of transform_reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicates *red_op* and *conv_op*.

> **Note**
>
> GENERALIZED_SUM(op, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`Iter`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random access iterator.
- **`Sent`** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter.
- **`Iter2`** – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an random access iterator.
- **`T`** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the first sorted range the algorithm will be applied to.
- **`last`** – Refers to the end of the second sorted range the algorithm will be applied to.
- **`first2`** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **`init`** – The initial value for the generalized sum.

Returns

The *transform_reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *parallel_task_policy* and returns *T* otherwise. The *transform_reduce* algorithm returns the result of the generalized sum over the values returned from *conv_op* when applied to the elements given by the input range [first, last).

template<typename **Iter**, typename **Sent**, typename **Iter2**, typename **T**>

T `hpx::transform_reduce`(*Iter* first, *Sent* last, *Iter2* first2, *T* init)

Returns GENERALIZED_SUM(red_op, init, conv_op(*first), …, conv_op(*(first + (last - first) - 1))).

The difference between *transform_reduce* and *accumulate* is that the behavior of transform_reduce may be non-deterministic for non-associative or non-commutative binary predicate.

Note

Complexity: O(*last* - *first*) applications of the predicates *red_op* and *conv_op*.

Note

GENERALIZED_SUM(op, a1, …, aN) is defined as follows:

- a1 when N is 1
- op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
  - b1, …, bN may be any permutation of a1, …, aN and
  - 1 < K+1 = M <= N.

Template Parameters

- **Iter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random access iterator.
- **Sent** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter.
- **Iter2** – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an random access iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **first** – Refers to the beginning of the first sorted range the algorithm will be applied to.
- **last** – Refers to the end of the second sorted range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **init** – The initial value for the generalized sum.

Returns

The *transform_reduce* algorithm returns *T*. The *transform_reduce* algorithm returns the result of the generalized sum over the values returned from *conv_op* when applied to the elements given by the input range [first, last).

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **Iter2**, typename **T**, typename **Reduce**, typename **Convert**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> `hpx::transform_reduce`(*ExPolicy* &&policy, *Iter* first, *Sent* last, *Iter2* first2, *T* init, *Reduce* &&red_op, *Convert* &&conv_op)

Returns GENERALIZED_SUM(red_op, init, conv_op(*first), …, conv_op(*(first + (last - first) - 1))).

The reduce operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *transform_reduce* and *accumulate* is that the behavior of transform_reduce may be non-deterministic for non-associative or non-commutative binary predicate.

Note

Complexity: O(*last* - *first*) applications of the predicates *red_op* and *conv_op*.

Note

GENERALIZED_SUM(op, a1, …, aN) is defined as follows:

- a1 when N is 1
- op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
  - b1, …, bN may be any permutation of a1, …, aN and
  - 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Iter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random access iterator.

- **Sent** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter.
- **Iter2** – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an random access iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **Reduce** – The type of the binary function object used for the reduction operation.
- **Convert** – The type of the unary function object used to transform the elements of the input sequence before invoking the reduce function.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the first sorted range the algorithm will be applied to.
- **last** – Refers to the end of the second sorted range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **init** – The initial value for the generalized sum.
- **red_op** – Specifies the function (or function object) which will be invoked for each of the values returned from the invocation of *conv_op*. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1*, *Type2*, and *Ret* must be such that an object of a type as returned from *conv_op* can be implicitly converted to any of those types.
- **conv_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

  ```
  R fun(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *Iter* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *parallel_task_policy* and returns *T* otherwise. The *transform_reduce* algorithm returns the result of the generalized sum over the values returned from *conv_op* when applied to the elements given by the input range [first, last).

template<typename **Iter**, typename **Sent**, typename **Iter2**, typename **T**, typename **Reduce**, typename **Convert**>

T *hpx*::**transform_reduce**(*Iter* first, *Sent* last, *Iter2* first2, *T* init, *Reduce* &&red_op, *Convert* &&conv_op)

Returns GENERALIZED_SUM(red_op, init, conv_op(*first), …, conv_op(*(first + (last - first) - 1))).

The difference between *transform_reduce* and *accumulate* is that the behavior of transform_reduce may be non-deterministic for non-associative or non-commutative binary predicate.

Note

Complexity: O(*last* - *first*) applications of the predicates *red_op* and *conv_op*.

Note

GENERALIZED_SUM(op, a1, …, aN) is defined as follows:

- a1 when N is 1
- op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
  - b1, …, bN may be any permutation of a1, …, aN and
  - 1 < K+1 = M <= N.

Template Parameters

- **`Iter`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random access iterator.
- **`Sent`** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter.
- **`Iter2`** – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an random access iterator.
- **`T`** – The type of the value to be used as initial (and intermediate) values (deduced).
- **`Reduce`** – The type of the binary function object used for the reduction operation.
- **`Convert`** – The type of the unary function object used to transform the elements of the input sequence before invoking the reduce function.

Parameters

- **`first`** – Refers to the beginning of the first sorted range the algorithm will be applied to.
- **`last`** – Refers to the end of the second sorted range the algorithm will be applied to.
- **`first2`** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **`init`** – The initial value for the generalized sum.
- **`red_op`** – Specifies the function (or function object) which will be invoked for each of the values returned from the invocation of *conv_op*. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type2 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1*, *Type2*, and *Ret* must be such that an object of a type as returned from *conv_op* can be implicitly converted to any of those types.

- **conv_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

```
R fun(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *Iter* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_reduce* algorithm returns *T*. The *transform_reduce* algorithm returns the result of the generalized sum over the values returned from *conv_op* when applied to the elements given by the input range [first, last).

template<typename **ExPolicy**, typename **Rng**, typename **T**, typename **Reduce**, typename **Convert**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> `hpx::`**`transform_reduce`**(*ExPolicy* &&policy, *Rng* &&rng, *T* init, *Reduce* &&red_op, *Convert* &&conv_op)

Returns GENERALIZED_SUM(red_op, init, conv_op(*first), …, conv_op(*(first + (last - first) - 1))).

The reduce operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *transform_reduce* and *accumulate* is that the behavior of transform_reduce may be non-deterministic for non-associative or non-commutative binary predicate.

Note

Complexity: O(*last* - *first*) applications of the predicates *red_op* and *conv_op*.

Note

GENERALIZED_SUM(op, a1, …, aN) is defined as follows:

- a1 when N is 1
- op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
  - b1, …, bN may be any permutation of a1, …, aN and
  - 1 < K+1 = M <= N.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`Rng`** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **`T`** – The type of the value to be used as initial (and intermediate) values (deduced).
- **`Reduce`** – The type of the binary function object used for the reduction operation.
- **`Convert`** – The type of the unary function object used to transform the elements of the input sequence before invoking the reduce function.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`rng`** – Refers to the sequence of elements the algorithm will be applied to.
- **`init`** – The initial value for the generalized sum.
- **`red_op`** – Specifies the function (or function object) which will be invoked for each of the values returned from the invocation of *conv_op*. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1*, *Type2*, and *Ret* must be such that an object of a type as returned from *conv_op* can be implicitly converted to any of those types.
- **`conv_op`** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

  ```
  R fun(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *Iter* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *parallel_task_policy* and returns *T* otherwise. The *transform_reduce* algorithm returns the result of the generalized sum over the values returned from *conv_op* when applied to the elements given by the input range [first, last).

template<typename **Rng**, typename **T**, typename **Reduce**, typename **Convert**>

T `hpx::transform_reduce`(*Rng* &&rng, *T* init, *Reduce* &&red_op, *Convert* &&conv_op)

Returns GENERALIZED_SUM(red_op, init, conv_op(*first), …, conv_op(*(first + (last - first) - 1))).

The difference between *transform_reduce* and *accumulate* is that the behavior of transform_reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicates *red_op* and *conv_op*.

> **Note**
>
> GENERALIZED_SUM(op, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **Reduce** – The type of the binary function object used for the reduction operation.
- **Convert** – The type of the unary function object used to transform the elements of the input sequence before invoking the reduce function.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **init** – The initial value for the generalized sum.
- **red_op** – Specifies the function (or function object) which will be invoked for each of the values returned from the invocation of *conv_op*. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type2 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1*, *Type2*, and *Ret* must be such that an object of a type as returned from *conv_op* can be implicitly converted to any of those types.

- **conv_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

```
R fun(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *Iter* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_reduce* algorithm returns *T*. The *transform_reduce* algorithm returns the result of the generalized sum over the values returned from *conv_op* when applied to the elements given by the input range [first, last).

template<typename **ExPolicy**, typename **Rng**, typename **Iter2**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> `hpx::transform_reduce`(*ExPolicy* &&policy, *Rng* &&rng, *Iter2* first2, *T* init)

Returns the result of accumulating init with the inner products of the pairs formed by the elements of two ranges starting at first1 and first2.

The operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op2*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Iter2** – The type of the second source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.

- **T** – The type of the value to be used as return) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second sequence of elements the result will be calculated with.
- **init** – The initial value for the sum.

Returns
The *transform_reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *T* otherwise.

template<typename **Rng**, typename **Iter2**, typename **T**>

T hpx::**transform_reduce**(*Rng* &&rng, *Iter2* first2, *T* init)

Returns the result of accumulating init with the inner products of the pairs formed by the elements of two ranges starting at first1 and first2.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op2*.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Iter2** – The type of the second source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to be used as return) values (deduced).

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second sequence of elements the result will be calculated with.
- **init** – The initial value for the sum.

Returns
The *transform_reduce* algorithm returns *T*.

template<typename **ExPolicy**, typename **Rng**, typename **Iter2**, typename **T**, typename **Reduce**, typename **Convert**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> hpx::**transform_reduce**(*ExPolicy* &&policy, *Rng* &&rng, *Iter2* first2, *T* init, *Reduce* &&red_op, *Convert* &&conv_op)

Returns the result of accumulating init with the inner products of the pairs formed by the elements of two ranges starting at first1 and first2.

The operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The operations in the parallel *transform_reduce* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op2*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Iter2** – The type of the second source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to be used as return) values (deduced).
- **Reduce** – The type of the binary function object used for the multiplication operation.
- **Convert** – The type of the unary function object used to transform the elements of the input sequence before invoking the reduce function.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second sequence of elements the result will be calculated with.
- **init** – The initial value for the sum.
- **red_op** – Specifies the function (or function object) which will be invoked for the initial value and each of the return values of *op2*. This is a binary predicate. The signature of this predicate should be equivalent to should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Ret* must be such that it can be implicitly converted to a type of *T*.

- **conv_op** – Specifies the function (or function object) which will be invoked for each of the input values of the sequence. This is a binary predicate. The signature of this predicate should be equivalent to

  ```
  Ret fun(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Ret* must be such that it can be implicitly converted to an object for the second argument type of *op1*.

Returns

The *transform_reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *T* otherwise.

template<typename **Rng**, typename **Iter2**, typename **T**, typename **Reduce**, typename **Convert**>

T *hpx*::**transform_reduce**(*Rng* &&rng, *Iter2* first2, *T* init, *Reduce* &&red_op, *Convert* &&conv_op)

Returns the result of accumulating init with the inner products of the pairs formed by the elements of two ranges starting at first1 and first2.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op2*.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Iter2** – The type of the second source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to be used as return) values (deduced).
- **Reduce** – The type of the binary function object used for the multiplication operation.
- **Convert** – The type of the unary function object used to transform the elements of the input sequence before invoking the reduce function.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second sequence of elements the result will be calculated with.
- **init** – The initial value for the sum.
- **red_op** – Specifies the function (or function object) which will be invoked for the initial value and each of the return values of *op2*. This is a binary predicate. The signature of this predicate should be equivalent to should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Ret* must be such that it can be implicitly converted to a type of *T*.

- **conv_op** – Specifies the function (or function object) which will be invoked for each of the input values of the sequence. This is a binary predicate. The signature of this predicate should be equivalent to

```
Ret fun(const Type1 &a, const Type2 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Ret* must be such that it can be implicitly converted to an object for the second argument type of *op1*.

Returns

The *transform_reduce* algorithm returns *T*.

hpx::is_heap

Defined in header hpx/algorithm.hpp[698].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::is_heap_until*

template<typename **ExPolicy**, typename **RandIter**, typename **Comp** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> **hpx::is_heap**(*ExPolicy* &&policy, *RandIter* first, *RandIter* last, *Comp* &&comp = *Comp*())

Returns whether the range is max heap. That is, true if the range is max heap, false otherwise. The function uses the given comparison function object *comp* (defaults to using operator<()). Executed according to the policy.

comp has to induce a strict weak ordering on the values.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **RandIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a random access iterator.

[698] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **Comp** – The type of the function/function object to use (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – *comp* is a callable object. The return value of the INVOKE operation applied to an object of type *Comp*, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.

Returns

The *is_heap* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *is_heap* algorithm returns whether the range is max heap. That is, true if the range is max heap, false otherwise.

template<typename **RandIter**, typename **Comp** = *hpx*::*parallel*::detail::less>

bool *hpx*::**is_heap**(*RandIter* first, *RandIter* last, *Comp* &&comp = *Comp*())

Returns whether the range is max heap. That is, true if the range is max heap, false otherwise. The function uses the given comparison function object *comp* (defaults to using operator<()).

comp has to induce a strict weak ordering on the values.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*.

Template Parameters

- **RandIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **Comp** – The type of the function/function object to use (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – *comp* is a callable object. The return value of the INVOKE operation applied to an object of type *Comp*, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.

Returns

The *is_heap* a *bool*. The *is_heap* algorithm returns whether the range is max heap. That is, true if the range is max heap, false otherwise.

hpx::is_heap_until

Defined in header hpx/algorithm.hpp[699].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::is_heap*

template<typename **ExPolicy**, typename **RandIter**, typename **Comp** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *RandIter*> **hpx::is_heap_until**(*ExPolicy* &&policy, *RandIter* first, *RandIter* last, *Comp* &&comp = *Comp*())

Returns the upper bound of the largest range beginning at *first* which is a max heap. That is, the last iterator *it* for which range [first, it) is a max heap. The function uses the given comparison function object *comp* (defaults to using operator<()). Executed according to the policy.

comp has to induce a strict weak ordering on the values.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **RandIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **Comp** – The type of the function/function object to use (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.

[699] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – *comp* is a callable object. The return value of the INVOKE operation applied to an object of type *Comp*, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.

Returns

The *is_heap_until* algorithm returns a *hpx::future<RandIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *RandIter* otherwise. The *is_heap_until* algorithm returns the upper bound of the largest range beginning at first which is a max heap. That is, the last iterator *it* for which range [first, it) is a max heap.

template<typename **RandIter**, typename **Comp** = *hpx*::*parallel*::detail::less>

RandIter `hpx::is_heap_until`(*RandIter* first, *RandIter* last, *Comp* &&comp = *Comp*())

Returns the upper bound of the largest range beginning at *first* which is a max heap. That is, the last iterator *it* for which range [first, it) is a max heap. The function uses the given comparison function object *comp* (defaults to using operator<()).

comp has to induce a strict weak ordering on the values.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*.

Template Parameters

- **RandIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **Comp** – The type of the function/function object to use (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – *comp* is a callable object. The return value of the INVOKE operation applied to an object of type *Comp*, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.

Returns

The *is_heap_until* algorithm returns a *RandIter*. The *is_heap_until* algorithm returns the upper bound of the largest range beginning at first which is a max heap. That is, the last iterator *it* for which range [first, it) is a max heap.

hpx::experimental::for_loop

Defined in header hpx/algorithm.hpp[700].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::experimental::for_loop_strided*
- *hpx::experimental::for_loop_n*
- *hpx::experimental::for_loop_n_strided*

template<typename **I**, typename ...**Args**>

void *hpx*::*experimental*::**for_loop**(*std*::decay_t<*I*> first, *I* last, *Args*&&... args)

The for_loop implements loop functionality over a range specified by integral or iterator bounds. For the iterator case, these algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator.

The execution of for_loop without specifying an execution policy is equivalent to specifying *hpx::execution::seq* as the execution policy.

Requires: *I* shall be an integral type or meet the requirements of an input iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of *MoveConstructible*.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is *last* - *first*.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

Note

As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

[700] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Note

The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **I** – The type of the iteration variable. This could be an (forward) iterator type or an integral type.
- **Args** – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **args** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

```
<ignored> pred(I const& a, ...);
```

The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

template<typename **ExPolicy**, typename **I**, typename ...**Args**>

auto *hpx*::*experimental*::**for_loop**(*ExPolicy* &&policy, *std*::decay_t<*I*> first, *I* last, *Args*&&... args)

The for_loop implements loop functionality over a range specified by integral or iterator bounds. For the iterator case, these algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator. Executed according to the policy.

Requires: *I* shall be an integral type or meet the requirements of an input iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of *MoveConstructible*.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is *last - first*.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

> **Note**
>
> As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

> **Note**
>
> The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **I** – The type of the iteration variable. This could be an (forward) iterator type or an integral type.
- **Args** – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **args** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

```
<ignored> pred(I const& a, ...);
```

The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

Returns

The *for_loop* algorithm returns a *hpx::future<void>* if the execution policy is of type *hpx::execution::sequenced_task_policy* or *hpx::execution::parallel_task_policy* and returns *void* otherwise.

hpx::experimental::for_loop_strided

Defined in header hpx/algorithm.hpp[701].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::experimental::for_loop*
- *hpx::experimental::for_loop_n*
- *hpx::experimental::for_loop_n_strided*

template<typename **I**, typename **S**, typename ...**Args**>

void *hpx*::*experimental*::**for_loop_strided**(*std*::decay_t<*I*> first, *I* last, *S* stride, *Args*&&... args)

The for_loop_strided implements loop functionality over a range specified by integral or iterator bounds. For the iterator case, these algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator.

The execution of for_loop without specifying an execution policy is equivalent to specifying *hpx::execution::seq* as the execution policy.

Requires: *I* shall be an integral type or meet the requirements of an input iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of *MoveConstructible*.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is *last* - *first*.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

[701] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

> **Note**
>
> As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

> **Note**
>
> The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **`I`** – The type of the iteration variable. This could be an (forward) iterator type or an integral type.
- **`S`** – The type of the stride variable. This should be an integral type.
- **`Args`** – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`stride`** – Refers to the stride of the iteration steps. This shall have non-zero value and shall be negative only if I has integral type or meets the requirements of a bidirectional iterator.
- **`args`** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

  ```
  <ignored> pred(I const& a, ...);
  ```

  The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

template<typename **`ExPolicy`**, typename **`I`**, typename **`S`**, typename ...**`Args`**>

auto `hpx::experimental::`**`for_loop_strided`**(*ExPolicy* &&policy, *std*::decay_t<*I*> first, *I* last, *S* stride, *Args*&&... args)

The for_loop_strided implements loop functionality over a range specified by integral or iterator bounds. For the iterator case, these algorithms resemble for_each from the

Parallelism TS, but leave to the programmer when and if to dereference the iterator. Executed according to the policy.

Requires: *I* shall be an integral type or meet the requirements of an input iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of *MoveConstructible*.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is *last* - *first*.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

> **Note**
>
> As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

> **Note**
>
> The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **I** – The type of the iteration variable. This could be an (forward) iterator type or an integral type.
- **S** – The type of the stride variable. This should be an integral type.
- **Args** – A parameter pack, it’s last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **stride** – Refers to the stride of the iteration steps. This shall have non-zero value and shall be negative only if I has integral type or meets the requirements of a bidirectional iterator.
- **args** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

  ```
  <ignored> pred(I const& a, ...);
  ```

  The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

Returns

The *for_loop_strided* algorithm returns a *hpx::future<void>* if the execution policy is of type *hpx::execution::sequenced_task_policy* or *hpx::execution::parallel_task_policy* and returns *void* otherwise.

hpx::experimental::for_loop_n

Defined in header hpx/algorithm.hpp[702].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::experimental::for_loop*
- *hpx::experimental::for_loop_strided*
- *hpx::experimental::for_loop_n_strided*

template<typename **I**, typename **Size**, typename ...**Args**>

void *hpx::experimental*::**for_loop_n**(*I* first, *Size* size, *Args*&&... args)

The for_loop_n implements loop functionality over a range specified by integral or iterator bounds. For the iterator case, these algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator.

The execution of for_loop_n without specifying an execution policy is equivalent to specifying *hpx::execution::seq* as the execution policy.

[702] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Requires: *I* shall be an integral type or meet the requirements of an input iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of *MoveConstructible*.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is *last* - *first*.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

> **Note**
>
> As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

> **Note**
>
> The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **I** – The type of the iteration variable. This could be an (forward) iterator type or an integral type.
- **Size** – The type of a non-negative integral value specifying the number of items to iterate over.
- **Args** – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **size** – Refers to the number of items the algorithm will be applied to.

- **args** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

  ```
  <ignored> pred(I const& a, ...);
  ```

  The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

template<typename **ExPolicy**, typename **I**, typename **Size**, typename ...**Args**>

auto *hpx::experimental::***for_loop_n**(*ExPolicy* &&policy, *I* first, *Size* size, *Args*&&... args)

The for_loop_n implements loop functionality over a range specified by integral or iterator bounds. For the iterator case, these algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator. Executed according to the policy.

Requires: *I* shall be an integral type or meet the requirements of an input iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of *MoveConstructible*.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is *last* - *first*.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

> **Note**
>
> As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

> **Note**
>
> The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **`I`** – The type of the iteration variable. This could be an (forward) iterator type or an integral type.
- **`Size`** – The type of a non-negative integral value specifying the number of items to iterate over.
- **`Args`** – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`size`** – Refers to the number of items the algorithm will be applied to.
- **`args`** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

```
<ignored> pred(I const& a, ...);
```

  The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

Returns

The *for_loop_n* algorithm returns a *hpx::future<void>* if the execution policy is of type *hpx::execution::sequenced_task_policy* or *hpx::execution::parallel_task_policy* and returns *void* otherwise.

hpx::experimental::for_loop_n_strided

Defined in header hpx/algorithm.hpp[703].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::experimental::for_loop*
- *hpx::experimental::for_loop_strided*
- *hpx::experimental::for_loop_n*

template<typename **I**, typename **Size**, typename **S**, typename ...**Args**>

void *hpx::experimental*::**for_loop_n_strided**(*I* first, *Size* size, *S* stride, *Args*&&... args)

The for_loop_n_strided implements loop functionality over a range specified by integral or iterator bounds. For the iterator case, these algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator.

The execution of for_loop without specifying an execution policy is equivalent to specifying *hpx::execution::seq* as the execution policy.

Requires: *I* shall be an integral type or meet the requirements of an input iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of *MoveConstructible*.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is *last* - *first*.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

> **Note**
>
> As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

[703] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

> **Note**
>
> The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- `I` – The type of the iteration variable. This could be an (forward) iterator type or an integral type.
- `Size` – The type of a non-negative integral value specifying the number of items to iterate over.
- `S` – The type of the stride variable. This should be an integral type.
- `Args` – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `size` – Refers to the number of items the algorithm will be applied to.
- `stride` – Refers to the stride of the iteration steps. This shall have non-zero value and shall be negative only if I has integral type or meets the requirements of a bidirectional iterator.
- `args` – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

  ```
  <ignored> pred(I const& a, ...);
  ```

  The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

template<typename **ExPolicy**, typename **I**, typename **Size**, typename **S**, typename ...**Args**>

auto *hpx::experimental::***for_loop_n_strided**(*ExPolicy* &&policy, *I* first, *Size* size, *S* stride, *Args*&&... args)

The for_loop_n_strided implements loop functionality over a range specified by integral or iterator bounds. For the iterator case, these algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator. Executed according to the policy.

Requires: *I* shall be an integral type or meet the requirements of an input iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of *MoveConstructible*.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is *last* - *first*.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

> **Note**
>
> As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

> **Note**
>
> The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **I** – The type of the iteration variable. This could be an (forward) iterator type or an integral type.
- **Size** – The type of a non-negative integral value specifying the number of items to iterate over.
- **S** – The type of the stride variable. This should be an integral type.
- **Args** – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **size** – Refers to the number of items the algorithm will be applied to.
- **stride** – Refers to the stride of the iteration steps. This shall have non-zero value and shall be negative only if I has integral type or meets the requirements of a bidirectional iterator.
- **args** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

```
<ignored> pred(I const& a, ...);
```

  The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

Returns

The *for_loop_n_strided* algorithm returns a *hpx::future<void>* if the execution policy is of type *hpx::execution::sequenced_task_policy* or *hpx::execution::parallel_task_policy* and returns *void* otherwise.

hpx::set_symmetric_difference

Defined in header hpx/algorithm.hpp[704].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **FwdIter3**, typename **Pred** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter3*>::type **hpx::set_symmetric_difference**(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *FwdIter3* dest, *Pred* &&op = *Pred*())

Constructs a sorted range beginning at dest consisting of all elements present in either of the sorted ranges [first1, last1) and [first2, last2), but not in both of them are copied to the range beginning at *dest*. The resulting range is also sorted. This algorithm expects

[704] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

both input ranges to be sorted with the given binary predicate *pred*. Executed according to the policy.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), it will be copied to *dest* exactly std::abs(m-n) times. If m>n, then the last m-n of those elements are copied from [first1,last1), otherwise the last n-m elements are copied from [first2,last2). The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

The application of function objects in parallel algorithm invoked with a sequential execution policy object execute in sequential order in the calling thread (*sequenced_policy*) or in a single new thread spawned from the current thread (for *sequenced_task_policy*).

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **`FwdIter1`** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **`FwdIter2`** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **`FwdIter3`** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator or output iterator and sequential execution.
- **`Pred`** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_symmetric_difference* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first1`** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **`last1`** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.

- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*

Returns

The *set_symmetric_difference* algorithm returns a *hpx::future<FwdIter3>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter3* otherwise. The *set_symmetric_difference* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **FwdIter1**, typename **FwdIter2**, typename **FwdIter3**, typename **Pred** = *hpx*::*parallel*::detail::less>

FwdIter3 `hpx::set_symmetric_difference`(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter2* last2, *FwdIter3* dest, *Pred* &&op = *Pred*())

Constructs a sorted range beginning at dest consisting of all elements present in either of the sorted ranges [first1, last1) and [first2, last2), but not in both of them are copied to the range beginning at *dest*. The resulting range is also sorted. This algorithm expects both input ranges to be sorted with the given binary predicate *pred*.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), it will be copied to *dest* exactly std::abs(m-n) times. If m>n, then the last m-n of those elements are copied from [first1,last1), otherwise the last n-m elements are copied from [first2,last2). The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **FwdIter1** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of a forward iterator.

- **FwdIter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator or output iterator and sequential execution.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_symmetric_difference* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*

Returns

The *set_symmetric_difference* algorithm returns a *FwdIter3*. The *set_symmetric_difference* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::for_each

Defined in header hpx/algorithm.hpp[705].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::for_each_n*

template<typename **InIter**, typename **F**>

F `hpx::for_each`(*InIter* first, *InIter* last, *F* &&f)

Applies *f* to the result of dereferencing every iterator in the range [first, last).

If *f* returns a result, the result is ignored.

[705] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

If the type of *first* satisfies the requirements of a mutable iterator, *f* may apply non-constant functions through the dereferenced iterator.

> **Note**
>
> Complexity: Applies *f* exactly *last* - *first* times.

Template Parameters

- **InIter** – The type of the source begin and end iterator used (deduced). This iterator type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). F must meet requirements of *MoveConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
<ignored> pred(const Type &a);
```

  The signature does not need to have const&. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.

Returns

f.

template<typename **ExPolicy**, typename **FwdIter**, typename **F**>

util::detail::algorithm_result_t<*ExPolicy*, void> *hpx*::**for_each**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *F* &&f)

Applies *f* to the result of dereferencing every iterator in the range [first, last). Executed according to the policy.

If *f* returns a result, the result is ignored.

If the type of *first* satisfies the requirements of a mutable iterator, *f* may apply non-constant functions through the dereferenced iterator.

Unlike its sequential form, the parallel overload of *for_each* does not return a copy of its *Function* parameter, since parallelization may not permit efficient state accumulation.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Applies *f* exactly *last* - *first* times.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **FwdIter** – The type of the source begin and end iterator used (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *for_each* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
<ignored> pred(const Type &a);
```

The signature does not need to have const&. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

Returns

The *for_each* algorithm returns a *hpx::future<void>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns void otherwise.

hpx::for_each_n

Defined in header hpx/algorithm.hpp[706].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::for_each*

[706] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

template<typename **InIter**, typename **Size**, typename **F**>

InIter hpx::**for_each_n**(*InIter* first, *Size* count, *F* &&f)

Applies *f* to the result of dereferencing every iterator in the range [first, first + count), starting from first and proceeding to first + count - 1.

If *f* returns a result, the result is ignored.

If the type of *first* satisfies the requirements of a mutable iterator, *f* may apply non-constant functions through the dereferenced iterator.

> **Note**
>
> Complexity: Applies *f* exactly *count* times.

Template Parameters

- **InIter** – The type of the source begin and end iterator used (deduced). This iterator type must meet the requirements of an input iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **F** – The type of the function/function object to use (deduced). F must meet requirements of *MoveConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
<ignored> pred(const Type &a);
```

The signature does not need to have const&. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.

Returns

first + *count* for non-negative values of *count* and *first* for negative values.

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**, typename **F**>

util::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> hpx::**for_each_n**(*ExPolicy* &&policy, *FwdIter* first, *Size* count, *F* &&f)

Applies *f* to the result of dereferencing every iterator in the range [first, first + count), starting from first and proceeding to first + count - 1. Executed according to the policy.

If *f* returns a result, the result is ignored.

If the type of *first* satisfies the requirements of a mutable iterator, *f* may apply non-constant functions through the dereferenced iterator.

Unlike its sequential form, the parallel overload of *for_each_n* does not return a copy of its *Function* parameter, since parallelization may not permit efficient state accumulation.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Applies *f* exactly *count* times.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *for_each_n* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
<ignored> pred(const Type &a);
```

The signature does not need to have const&. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

Returns

The *for_each_n* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. It returns *first* + *count* for non-negative values of *count* and *first* for negative values.

hpx::experimental::reduction_max

Defined in header hpx/algorithm.hpp[707].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *hpx*::*parallel*::detail::max_of<*T*>> `hpx::experimental::reduction_max`(*T* &var)

The function template *reduction_max* returns a reduction object of unspecified type having a value type and encapsulating an identity value for the reduction, it uses std::max_of{} as its combiner function, and a live-out object from which the initial value is obtained and into which the final value is stored.

A parallel algorithm uses the *reduction_max* object by allocating an unspecified number of instances, called views, of the reduction's value type. Each view is initialized with the reduction object's identity value, except that the live-out object (which was initialized by the caller) comprises one of the views. The algorithm passes a reference to a view to each application of an element-access function, ensuring that no two concurrently-executing invocations share the same view. A view can be shared between two applications that do not execute concurrently, but initialization is performed only once per view.

Modifications to the view by the application of element access functions accumulate as partial results. At some point before the algorithm returns, the partial results are combined, two at a time, using the std::max_of{} operation until a single value remains, which is then assigned back to the live-out object.

T shall meet the requirements of *CopyConstructible* and *MoveAssignable*.

> **Note**
>
> In order to produce useful results, modifications to the view should be limited to commutative operations closely related to the combiner operation.

Template Parameters

- **T** – The value type to be used by the induction object.
- **Op** – The type of the binary function (object) used to perform the reduction operation.

Parameters

- **var** – [in,out] The life-out value to use for the reduction object. This will hold the reduced value after the algorithm is finished executing.
- **identity** – [in] The identity value to use for the reduction operation. This argument is optional and defaults to a copy of *var*.

[707] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Returns

This returns a reduction object of unspecified type having a value type of *T*. When the return value is used by an algorithm, the reference to *var* is used as the live-out object, new views are initialized to a copy of identity, and views are combined by invoking the copy of combiner, passing it the two views to be combined.

template<typename **T**>

constexpr *hpx*::*parallel*::detail::reduction_helper<*T*, *hpx*::*parallel*::detail::max_of<*T*>> `hpx::experimental::`**`reduction_max`**`(`*T* &var, *T* const &identity`)`

hpx::all_of

Defined in header hpx/algorithm.hpp[708].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::any_of*
- *hpx::none_of*

template<typename **ExPolicy**, typename **FwdIter**, typename **F**>

util::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::`**`all_of`**`(`*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *F* &&f`)`

Checks if unary predicate *f* returns true for all elements in the range [first, last).

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most *last* - *first* applications of the predicate *f*

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.

[708] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *all_of* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

Returns

The *all_of* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *all_of* algorithm returns true if the unary predicate *f* returns true for all elements in the range, false otherwise. It returns true if the range is empty.

template<typename **ExPolicy**, typename **InIter**, typename **F**>

bool *hpx*::**all_of**(*InIter* first, *InIter* last, *F* &&f)

Checks if unary predicate *f* returns true for all elements in the range [first, last).

> **Note**
>
> Complexity: At most *last* - *first* applications of the predicate *f*

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *all_of* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.

Returns

The *all_of* algorithm returns a *bool* . The *all_of* algorithm returns true if the unary predicate *f* returns true for all elements in the range, false otherwise. It returns true if the range is empty.

hpx::any_of

Defined in header hpx/algorithm.hpp[709].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::all_of*
- *hpx::none_of*

template<typename **ExPolicy**, typename **FwdIter**, typename **F**>

util::detail::algorithm_result_t<*ExPolicy*, bool> **hpx::any_of**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *F* &&f)

Checks if unary predicate *f* returns true for at least one element in the range [first, last).

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most *last* - *first* applications of the predicate *f*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.

[709] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *any_of* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

Returns

The *any_of* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *any_of* algorithm returns true if the unary predicate *f* returns true for at least one element in the range, false otherwise. It returns false if the range is empty.

template<typename **InIter**, typename **F**>

bool *hpx*::**any_of**(*InIter* first, *InIter* last, *F* &&f)

Checks if unary predicate *f* returns true for at least one element in the range [first, last).

> **Note**
>
> Complexity: At most *last* - *first* applications of the predicate *f*

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *any_of* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.

Returns

The *any_of* algorithm returns a *bool* . The *any_of* algorithm returns true if the unary predicate *f* returns true for at least one element in the range, false otherwise. It returns false if the range is empty.

hpx::none_of

Defined in header hpx/algorithm.hpp[710].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::all_of*
- *hpx::any_of*

template<typename **ExPolicy**, typename **FwdIter**, typename **F**>

util::detail::algorithm_result_t<*ExPolicy*, bool> **hpx::none_of**(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* last, *F* &&f)

Checks if unary predicate *f* returns true for no elements in the range [first, last).

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most *last* - *first* applications of the predicate *f*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.

[710] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *none_of* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

Returns

The *none_of* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *none_of* algorithm returns true if the unary predicate *f* returns true for no elements in the range, false otherwise. It returns true if the range is empty.

template<typename **InIter**, typename **F**>

bool *hpx*::**none_of**(*InIter* first, *InIter* last, *F* &&f)

Checks if unary predicate *f* returns true for no elements in the range [first, last).

> **Note**
>
> Complexity: At most *last* - *first* applications of the predicate *f*

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *none_of* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.

Returns

The *none_of* algorithm returns a *bool* . The *none_of* algorithm returns true if the unary predicate *f* returns true for no elements in the range, false otherwise. It returns true if the range is empty.

hpx::transform

Defined in header hpx/algorithm.hpp[711].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **FwdIter1**, typename **FwdIter2**, typename **F**>

FwdIter2 **hpx::transform**(*FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *F* &&f)

Applies the given function *f* to the range [first, last) and stores the result in another range, beginning at dest.

> **Note**
>
> Complexity: Exactly *last* - *first* applications of *f*

Template Parameters

- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *transform* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

[711] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type &a);
```

The signature does not need to have const&. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to *Type*. The type *Ret* must be such that an object of type *FwdIter2* can be dereferenced and assigned a value of type *Ret*.

Returns

The *transform* algorithm returns a *FwdIter2*. The *transform* algorithm returns a tuple holding an iterator referring to the first element after the input sequence and the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **F**>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> `hpx::transform`(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* last, *FwdIter2* dest, *F* &&f)

Applies the given function *f* to the range [first, last) and stores the result in another range, beginning at dest. Executed according to the policy.

The invocations of *f* in the parallel *transform* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The invocations of *f* in the parallel *transform* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *last* - *first* applications of *f*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the invocations of *f*.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *transform* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type &a);
  ```

  The signature does not need to have const&. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to *Type*. The type *Ret* must be such that an object of type *FwdIter2* can be dereferenced and assigned a value of type *Ret*.

Returns

The *transform* algorithm returns a *hpx::future<FwdIter2>* if the execution policy is of type *parallel_task_policy* and returns *FwdIter2* otherwise. The *transform* algorithm returns a tuple holding an iterator referring to the first element after the input sequence and the output iterator to the element in the destination range, one past the last element copied.

template<typename **FwdIter1**, typename **FwdIter2**, typename **FwdIter3**, typename **F**>

FwdIter3 **hpx::transform**(*FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter3* dest, *F* &&f)

Applies the given function *f* to pairs of elements from two ranges: one defined by [first1, last1) and the other beginning at first2, and stores the result in another range, beginning at dest.

> **Note**
>
> Complexity: Exactly *last* - *first* applications of *f*

Template Parameters

- **FwdIter1** – The type of the source iterators for the first range used (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators for the second range used (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *transform* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **first1** – Refers to the beginning of the first sequence of elements the algorithm will be applied to.
- **last1** – Refers to the end of the first sequence of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const&. The types *Type1* and *Type2* must be such that objects of types FwdIter1 and FwdIter2 can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively. The type *Ret* must be such that an object of type *FwdIter3* can be dereferenced and assigned a value of type *Ret*.

Returns

The *transform* algorithm returns a *FwdIter3*. The *transform* algorithm returns a tuple holding an iterator referring to the first element after the first input sequence, an iterator referring to the first element after the second input sequence, and the output iterator referring to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **FwdIter2**, typename **FwdIter3**, typename **F**>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter3*> `hpx::transform`(*ExPolicy* &&policy, *FwdIter1* first1, *FwdIter1* last1, *FwdIter2* first2, *FwdIter3* dest, *F* &&f)

Applies the given function *f* to pairs of elements from two ranges: one defined by [first1, last1) and the other beginning at first2, and stores the result in another range, beginning at dest. Executed according to the policy.

The invocations of *f* in the parallel *transform* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The invocations of *f* in the parallel *transform* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *last* - *first* applications of *f*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the invocations of *f*.

- **FwdIter1** – The type of the source iterators for the first range used (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter2** – The type of the source iterators for the second range used (deduced). This iterator type must meet the requirements of a forward iterator.
- **FwdIter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *transform* requires *F* to meet the requirements of *Copy-Constructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the first sequence of elements the algorithm will be applied to.
- **last1** – Refers to the end of the first sequence of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type2 &b);
```

The signature does not need to have const&. The types *Type1* and *Type2* must be such that objects of types FwdIter1 and FwdIter2 can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively. The type *Ret* must be such that an object of type *FwdIter3* can be dereferenced and assigned a value of type *Ret*.

Returns

The *transform* algorithm returns a *hpx::future<FwdIter3>* if the execution policy is of type *parallel_task_policy* and returns *FwdIter3* otherwise. The *transform* algorithm returns a tuple holding an iterator referring to the first element after the first input sequence, an iterator referring to the first element after the second input sequence, and the output iterator referring to the element in the destination range, one past the last element copied.

hpx::ranges::shift_left

Defined in header hpx/algorithm.hpp[712].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **FwdIter**, typename **Sent**, typename **Size**>

[712] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

FwdIter `hpx::ranges::shift_left`(*FwdIter* first, *Sent* last, *Size* n)

Shifts the elements in the range [first, last) by n positions towards the beginning of the range. For every integer i in [0, last - first

- n), moves the element originally at position first + n + i to position first + i.

The assignment operations in the parallel *shift_left* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: At most (last - first) - n assignments.

> **Note**
>
> The type of dereferenced *FwdIter* must meet the requirements of *MoveAssignable*.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **Size** – The type of the argument specifying the number of positions to shift by.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **n** – Refers to the number of positions to shift.

Returns

The *shift_left* algorithm returns *FwdIter*. The *shift_left* algorithm returns an iterator to the end of the resulting range.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **Size**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::ranges::shift_left`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *Size* n)

Shifts the elements in the range [first, last) by n positions towards the beginning of the range. For every integer i in [0, last - first

- n), moves the element originally at position first + n + i to position first + i.

The assignment operations in the parallel *shift_left* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignment operations in the parallel *shift_left* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most (last - first) - n assignments.

> **Note**
>
> The type of dereferenced *FwdIter* must meet the requirements of *MoveAssignable*.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Sent`** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **`Size`** – The type of the argument specifying the number of positions to shift by.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **`n`** – Refers to the number of positions to shift.

Returns

The *shift_left* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *shift_left* algorithm returns an iterator to the end of the resulting range.

template<typename **`Rng`**, typename **`Size`**>

std::ranges::iterator_t<*Rng*> `hpx::ranges::`**`shift_left`**(*Rng* &&rng, *Size* n)

Shifts the elements in the range [first, last) by n positions towards the beginning of the range. For every integer i in [0, last - first

- n), moves the element originally at position first + n + i to position first + i.

The assignment operations in the parallel *shift_left* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: At most (last - first) - n assignments.

> **Note**
>
> The type of dereferenced *std::ranges::iterator_t<Rng>* must meet the requirements of *MoveAssignable*.

Template Parameters

- **`Rng`** – The type of the range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **`Size`** – The type of the argument specifying the number of positions to shift by.

Parameters

- **`rng`** – Refers to the range in which the elements will be shifted.
- **`n`** – Refers to the number of positions to shift.

Returns

The *shift_left* algorithm returns *std::ranges::iterator_t<Rng>*. The *shift_left* algorithm returns an iterator to the end of the resulting range.

template<typename **ExPolicy**, typename **Rng**, typename **Size**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::shift_left`(*ExPolicy* &&policy, *Rng* &&rng, *Size* n)

Shifts the elements in the range [first, last) by n positions towards the beginning of the range. For every integer i in [0, last - first

- n), moves the element originally at position first + n + i to position first + i.

The assignment operations in the parallel *shift_left* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignment operations in the parallel *shift_left* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in

an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most (last - first) - n assignments.

> **Note**
>
> The type of dereferenced *std::ranges::iterator_t<Rng>* must meet the requirements of *MoveAssignable*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of positions to shift by.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the range in which the elements will be shifted.
- **n** – Refers to the number of positions to shift.

Returns

The *shift_left* algorithm returns a hpx::future<*std::ranges::iterator_t<Rng>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *std::ranges::iterator_t<Rng>* otherwise. The *shift_left* algorithm returns an iterator to the end of the resulting range.

hpx::ranges::exclusive_scan

Defined in header hpx/algorithm.hpp[713].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter**, typename **Sent**, typename **OutIter**, typename **T** = typename *std*::iterator_traits<*InIter*>::value_type, typename **Op** = *std*::plus<*T*>>

exclusive_scan_result<*InIter*, *OutIter*> `hpx::ranges::exclusive_scan`(*InIter* first, *Sent* last, *OutIter* dest, *T* init, *Op* &&op = *Op*())

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, *first, …, *(first + (i - result) - 1)).

[713] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum. If *op* is not mathematically associative, the behavior of *inclusive_scan* may be non-deterministic.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, . . . , aN) is defined as:
>
> - a1 when N is 1
> - op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, . . . , aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, . . . , aN)) where 1 < K+1 = M <= N.

Template Parameters

- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter1.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **Op** – The type of the binary function object used for the reduction operation.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest** – Refers to the beginning of the destination range.
- **init** – The initial value for the generalized sum.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

Returns

The *exclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent**, typename **FwdIter2**, typename **T** = typename *std*::iterator_traits<*FwdIter1*>::value_type, typename **Op** = *std*::plus<*T*>>

parallel::*util*::detail::algorithm_result<*ExPolicy*, exclusive_scan_result<*FwdIter1*, *FwdIter2*>>::type `hpx::ranges::exclusive_scan`(*ExP* && icy, *Fwd* first *Sen* last, *Fwd* dest *T* init, *Op* && = *Op*(

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, *first, …, *(first + (i - result) - 1)).

The reduce operations in the parallel *exclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *exclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum. If *op* is not mathematically associative, the behavior of *inclusive_scan* may be non-deterministic.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:
>
> - a1 when N is 1
> - op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN)) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter1.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **Op** – The type of the binary function object used for the reduction operation.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest** – Refers to the beginning of the destination range.
- **init** – The initial value for the generalized sum.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

Returns

The *exclusive_scan* algorithm returns a hpx::future<*util::in_out_result<FwdIter1, FwdIter2>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *util::in_out_result<FwdIter1, FwdIter2>* otherwise. The *exclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng**, typename **O**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type, typename **Op** = *std*::plus<*T*>>
exclusive_scan_result<*std*::ranges::iterator_t<*Rng*>, *O*> `hpx::ranges::exclusive_scan`(*Rng* &&rng, *O* dest, *T* init, *Op* &&op = *Op*())

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(+, init, *first, …, *(first + (i - result) - 1))

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *std::plus<T>*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, ..., aN) is defined as:
>
> - a1 when N is 1
> - GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, ..., aK)
> - GENERALIZED_NONCOMMUTATIVE_SUM(+, aM, ..., aN) where 1 < K+1 = M <= N.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **O** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **Op** – The type of the binary function object used for the reduction operation.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **init** – The initial value for the generalized sum.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

Returns

The *exclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng**, typename **O**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type, typename **Op** = *std*::plus<*T*>>

parallel::*util*::detail::algorithm_result<*ExPolicy*, exclusive_scan_result<*std*::ranges::iterator_t<*Rng*>, *O*>> `hpx::ranges::exclusive_scan`

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(+, init, *first, …, *(first + (i - result) - 1))

The reduce operations in the parallel *exclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *exclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *std::plus<T>*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, …, aN) is defined as:
>
> - a1 when N is 1
> - GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, …, aK)
>   - GENERALIZED_NONCOMMUTATIVE_SUM(+, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **O** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **Op** – The type of the binary function object used for the reduction operation.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **init** – The initial value for the generalized sum.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

Returns

The *exclusive_scan* algorithm returns a hpx::future<*util::in_out_result* <std::ranges::iterator_t<Rng>, O>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *util::in_out_result* <std::ranges::iterator_t<Rng>, O> otherwise. The *exclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

hpx::ranges::swap_ranges

Defined in header hpx/algorithm.hpp[714].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter1**, typename **Sent1**, typename **InIter2**, typename **Sent2**>

swap_ranges_result<*InIter1*, *InIter2*> `hpx::ranges::`**`swap_ranges`**(*InIter1* first1, *Sent1* last1, *InIter2* first2, *Sent2* last2)

Exchanges elements between range [first1, last1) and another range starting at *first2*.

The swap operations in the parallel *swap_ranges* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

[714] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

> **Note**
>
> Complexity: Linear in the distance between *first1* and *last1*

Template Parameters

- **InIter1** – The type of the first range of iterators to swap (deduced).
- **Sent1** – The type of the first sentinel (deduced). This sentinel type must be a sentinel for InIter1.
- **InIter2** – The type of the second range of iterators to swap (deduced).
- **Sent2** – The type of the second sentinel (deduced). This sentinel type must be a sentinel for InIter2.

Parameters

- **first1** – Refers to the beginning of the sequence of elements for the first range.
- **last1** – Refers to sentinel value denoting the end of the sequence of elements for the first range.
- **first2** – Refers to the beginning of the sequence of elements for the second range.
- **last2** – Refers to sentinel value denoting the end of the sequence of elements for the second range.

Returns

The *swap_ranges* algorithm returns *swap_ranges_result<InIter1, InIter2>*. The *swap_ranges* algorithm returns in_in_result with the first element as the iterator to the element past the last element exchanged in range beginning with *first1* and the second element as the iterator to the element past the last element exchanged in the range beginning with *first2*.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent1**, typename **FwdIter2**, typename **Sent2**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, swap_ranges_result<*FwdIter1*, *FwdIter2*>>::type hpx::ranges::**swap_ranges**(*ExPolicy* &&policy, *FwdIter1* first1, *Sent1* last1, *FwdIter2* first2, *Sent2* last2)

Exchanges elements between range [first1, last1) and another range starting at *first2*.

The swap operations in the parallel *swap_ranges* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The swap operations in the parallel *swap_ranges* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in

an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Linear in the distance between *first1* and *last1*

Template Parameters

- `ExPolicy` – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- `FwdIter1` – The type of the first range of iterators to swap (deduced).
- `Sent1` – The type of the first sentinel (deduced). This sentinel type must be a sentinel for FwdIter1.
- `FwdIter2` – The type of the second range of iterators to swap (deduced).
- `Sent2` – The type of the second sentinel (deduced). This sentinel type must be a sentinel for FwdIter2.

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `first1` – Refers to the beginning of the sequence of elements for the first range.
- `last1` – Refers to sentinel value denoting the end of the sequence of elements for the first range.
- `first2` – Refers to the beginning of the sequence of elements for the second range.
- `last2` – Refers to sentinel value denoting the end of the sequence of elements for the second range.

Returns

The *swap_ranges* algorithm returns a hpx::future<*swap_ranges_result*<*FwdIter1, FwdIter2*>> if the execution policy is of type *parallel_task_policy* and returns *FwdIter2* otherwise. The *swap_ranges* algorithm returns in_in_result with the first element as the iterator to the element past the last element exchanged in range beginning with *first1* and the second element as the iterator to the element past the last element exchanged in the range beginning with *first2*.

template<typename **Rng1**, typename **Rng2**>

swap_ranges_result<*std*::ranges::iterator_t<*Rng1*>, *std*::ranges::iterator_t<*Rng2*>> `hpx::ranges::swap_ranges`(*Rng1* &&rng1, *Rng2* &&rng2)

Exchanges elements between range [first1, last1) and another range starting at *first2*.

The swap operations in the parallel *swap_ranges* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear in the distance between *first1* and *last1*

Template Parameters

- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the destination range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

Parameters

- **rng1** – Refers to the sequence of elements of the first range.
- **rng2** – Refers to the sequence of elements of the second range.

Returns

The *swap_ranges* algorithm returns *swap_ranges_result*< std::ranges::iterator_t<Rng1>, std::ranges::iterator_t<Rng1>>. The *swap_ranges* algorithm returns in_in_result with the first element as the iterator to the element past the last element exchanged in range beginning with *first1* and the second element as the iterator to the element past the last element exchanged in the range beginning with *first2*.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, swap_ranges_result<*std*::ranges::iterator_t<*Rng1*>, *std*::ranges::iterator_t<*Rng2*>>> `hpx::r`

Exchanges elements between range [first1, last1) and another range starting at *first2*.

The swap operations in the parallel *swap_ranges* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The swap operations in the parallel *swap_ranges* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Linear in the distance between *first1* and *last1*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the destination range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng1** – Refers to the sequence of elements of the first range.
- **rng2** – Refers to the sequence of elements of the second range.

Returns

The *swap_ranges* algorithm returns a hpx::future<*swap_ranges_result*< std::ranges::iterator_t<Rng1>, std::ranges::iterator_t<Rng1>>> if the execution policy is of type *parallel_task_policy* and returns *swap_ranges_result*< std::ranges::iterator_t<Rng1>, std::ranges::iterator_t<Rng1>>. otherwise. The *swap_ranges* algorithm returns in_in_result with the first element as the iterator to the element past the last element exchanged in range beginning with *first1* and the second element as the iterator to the element past the last element exchanged in the range beginning with *first2*.

hpx::ranges::stable_sort

Defined in header hpx/algorithm.hpp[715].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **RandomIt**, typename **Sent**, typename **Comp** = *ranges*::less, typename **Proj** = *hpx*::identity>

RandomIt `hpx::ranges::stable_sort`(*RandomIt* first, *Sent* last, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Sorts the elements in the range [first, last) in ascending order. The relative order of equal elements is preserved. The function uses the given comparison function object comp (defaults to using operator<()).

A sequence is sorted with respect to a comparator *comp* and a projection *proj* if for every iterator i pointing to the sequence and every non-negative integer n such that i + n is a valid iterator pointing to an element of the sequence, and INVOKE(comp, INVOKE(proj, *(i + n)), INVOKE(proj, *i)) == false.

comp has to induce a strict weak ordering on the values.

The assignments in the parallel *stable_sort* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: O(N log(N)), where N = std::distance(first, last) comparisons.

[715] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Template Parameters

- `RandomIt` – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random iterator.
- `Sent` – The type of the source sentinel (deduced). This sentinel type must be a sentinel for RandomIt.
- `Comp` – The type of the function/function object to use (deduced).
- `Proj` – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `last` – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- `comp` – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- `proj` – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *stable_sort* algorithm returns *RandomIt*. The algorithm returns an iterator pointing to the first element after the last element in the input sequence.

template<typename **ExPolicy**, typename **RandomIt**, typename **Sent**, typename **Comp** = *ranges*::less, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *RandomIt*>::type `hpx::ranges::stable_sort`(*ExPolicy* &&policy, *RandomIt* first, *Sent* last, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Sorts the elements in the range [first, last) in ascending order. The relative order of equal elements is preserved. The function uses the given comparison function object comp (defaults to using operator<()).

A sequence is sorted with respect to a comparator *comp* and a projection *proj* if for every iterator i pointing to the sequence and every non-negative integer n such that i + n is a valid iterator pointing to an element of the sequence, and INVOKE(comp, INVOKE(proj, *(i + n)), INVOKE(proj, *i)) == false.

comp has to induce a strict weak ordering on the values.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an

unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(N log(N)), where N = std::distance(first, last) comparisons.

Template Parameters

- `ExPolicy` – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- `RandomIt` – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random iterator.
- `Sent` – The type of the source sentinel (deduced). This sentinel type must be a sentinel for RandomIt.
- `Comp` – The type of the function/function object to use (deduced).
- `Proj` – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `last` – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- `comp` – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- `proj` – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *stable_sort* algorithm returns a *hpx::future<RandomIt>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *RandomIt* otherwise. The algorithm returns an iterator pointing to the first element after the last element in the input sequence.

template<typename **Rng**, typename **Comp** = *ranges*::less, typename **Proj** = *hpx*::identity>

std::ranges::iterator_t<*Rng*> `hpx::ranges::stable_sort`(*Rng* &&rng, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Sorts the elements in the range [first, last) in ascending order. The relative order of equal elements is preserved. The function uses the given comparison function object comp (defaults to using operator<()).

A sequence is sorted with respect to a comparator *comp* and a projection *proj* if for every iterator i pointing to the sequence and every non-negative integer n such that i +

n is a valid iterator pointing to an element of the sequence, and INVOKE(comp, INVOKE(proj, *(i + n)), INVOKE(proj, *i)) == false.

comp has to induce a strict weak ordering on the values.

The assignments in the parallel *stable_sort* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: O(N log(N)), where N = std::distance(first, last) comparisons.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *stable_sort* algorithm returns *std::ranges::iterator_t<Rng>*. It returns *last*.

template<typename **ExPolicy**, typename **Rng**, typename **Comp** = *ranges*::less, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::stable_sort`(*ExPolicy* &&policy, *Rng* &&rng, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Sorts the elements in the range [first, last) in ascending order. The relative order of equal elements is preserved. The function uses the given comparison function object comp (defaults to using operator<()).

A sequence is sorted with respect to a comparator *comp* and a projection *proj* if for every iterator i pointing to the sequence and every non-negative integer n such that i + n is a valid iterator pointing to an element of the sequence, and INVOKE(comp, INVOKE(proj, *(i + n)), INVOKE(proj, *i)) == false.

comp has to induce a strict weak ordering on the values.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(N log(N)), where N = std::distance(first, last) comparisons.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **`Rng`** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **`Comp`** – The type of the function/function object to use (deduced).
- **`Proj`** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`rng`** – Refers to the sequence of elements the algorithm will be applied to.
- **`comp`** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **`proj`** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *stable_sort* algorithm returns a hpx::future<*std::ranges::iterator_t*<*Rng*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *std::ranges::iterator_t*<*Rng*> otherwise. It returns *last*.

hpx::ranges::unique

Defined in header hpx/algorithm.hpp[716].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::unique_copy*

template<typename **FwdIter**, typename **Sent**, typename **Pred** = *ranges*::equal_to, typename **Proj** = *hpx*::identity>

subrange_t<*FwdIter*, *Sent*> `hpx::ranges::unique`(*FwdIter* first, *Sent* last, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Eliminates all but the first element from every consecutive group of equivalent elements from the range [first, last) and returns a past-the-end iterator for the new logical end of the range.

The assignments in the parallel *unique* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs not more than *last - first* assignments, exactly *last - first* - 1 applications of the predicate *pred* and no more than twice as many applications of the projection *proj*.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *unique* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an binary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

[716] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to both *Type1* and *Type2*

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *unique* algorithm returns *subrange_t<FwdIter, Sent>*. The *unique* algorithm returns an object {ret, last}, where ret is a past-the-end iterator for a new subrange.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **Pred** = *ranges*::equal_to, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, subrange_t<*FwdIter*, *Sent*>>::type `hpx::ranges::unique`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Eliminates all but the first element from every consecutive group of equivalent elements from the range [first, last) and returns a past-the-end iterator for the new logical end of the range.

The assignments in the parallel *unique* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *unique* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* - 1 applications of the predicate *pred* and no more than twice as many applications of the projection *proj*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *unique* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an binary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to both *Type1* and *Type2*
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *unique* algorithm returns *subrange_t<FwdIter, Sent>*. The *unique* algorithm returns an object {ret, last}, where ret is a past-the-end iterator for a new subrange.

template<typename **Rng**, typename **Pred** = *ranges*::equal_to, typename **Proj** = *hpx*::identity>

subrange_t<*std*::ranges::iterator_t<*Rng*>, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::`**`unique`**(*Rng* &&rng, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Eliminates all but the first element from every consecutive group of equivalent elements from the range *rng* and returns a past-the-end iterator for the new logical end of the range.

The assignments in the parallel *unique* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Note

Complexity: Performs not more than N assignments, exactly N - 1 applications of the predicate *pred* and no more than twice as many applications of the projection *proj*, where N = std::distance(begin(rng), end(rng)).

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *unique* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an binary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a, const Type &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to *Type*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *unique* algorithm returns *subrange_t<std::ranges::iterator_t<Rng>*, std::ranges::iterator_t<Rng>>. The *unique* algorithm returns an object {ret, last}, where ret is a past-the-end iterator for a new subrange.

template<typename **ExPolicy**, typename **Rng**, typename **Pred** = *ranges*::equal_to, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, subrange_t<*std*::ranges::iterator_t<*Rng*>, *std*::ranges::iterator_t<*Rng*>>> `hpx::ranges::u`

Eliminates all but the first element from every consecutive group of equivalent elements from the range *rng* and returns a past-the-end iterator for the new logical end of the range.

The assignments in the parallel *unique* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *unique* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than N assignments, exactly N - 1 applications of the predicate *pred* and no more than twice as many applications of the projection *proj*, where N = std::distance(begin(rng), end(rng)).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *unique* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an binary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a, const Type &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to *Type*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *unique* algorithm returns a hpx::future <subrange_t<std::ranges::iterator_t<Rng>, std::ranges::iterator_t<Rng>>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *subrange_t<std::ranges::iterator_t<Rng>*, std::ranges::iterator_t<Rng>> otherwise. The *unique* algorithm returns an object {ret, last}, where ret is a past-the-end iterator for a new subrange.

hpx::ranges::unique_copy

Defined in header hpx/algorithm.hpp[717].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::unique*

template<typename **InIter**, typename **Sent**, typename **O**, typename **Pred** = *ranges*::equal_to, typename **Proj** = *hpx*::identity>

unique_copy_result<*InIter*, *O*> `hpx::ranges::unique_copy`(*InIter* first, *Sent* last, *O* dest, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Copies the elements from the range [first, last), to another range beginning at *dest* in such a way that there are no consecutive equal elements. Only the first element of each group of equal elements is copied.

The assignments in the parallel *unique_copy* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* - 1 applications of the predicate *pred* and no more than twice as many applications of the projection *proj*

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **O** – The type of the iterator representing the destination range (deduced).
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *unique_copy* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.

[717] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an binary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a, const Type &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *InIter1* can be dereferenced and then implicitly converted to *Type*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *unique_copy* algorithm returns a returns unique_copy_result<InIter, O>. The *unique_copy* algorithm returns an in_out_result with the source iterator to one past the last element and out containing the destination iterator to the end of the *dest* range.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **O**, typename **Pred** = *ranges*::equal_to, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, unique_copy_result<*FwdIter*, *O*>>::type `hpx::ranges::`**`unique_copy`**(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *O* dest, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Copies the elements from the range [first, last), to another range beginning at *dest* in such a way that there are no consecutive equal elements. Only the first element of each group of equal elements is copied.

The assignments in the parallel *unique_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *unique_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* - 1 applications of the predicate *pred* and no more than twice as many applications of the projection *proj*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter1.
- **O** – The type of the iterator representing the destination range (deduced).
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *unique_copy* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an binary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a, const Type &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *unique_copy* algorithm returns returns a *hpx::future*< unique_copy_result<FwdIter, O>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *unique_copy_result<FwdIter, O>* otherwise. The *unique_copy* algorithm returns an in_out_result with the source iterator to one past the last element and out containing the destination iterator to the end of the *dest* range.

template<typename **Rng**, typename **O**, typename **Pred** = *ranges*::equal_to, typename **Proj** = *hpx*::identity>

unique_copy_result<*std*::ranges::iterator_t<*Rng*>, *O*> `hpx::ranges::unique_copy`(*Rng* &&rng, *O* dest, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Copies the elements from the range *rng*, to another range beginning at *dest* in such a way that there are no consecutive equal elements. Only the first element of each group of equal elements is copied.

The assignments in the parallel *unique_copy* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

Note

Complexity: Performs not more than N assignments, exactly N - 1 applications of the predicate *pred*, where N = std::distance(begin(rng), end(rng)).

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **O** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *unique_copy* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by the range *rng*. This is an binary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a, const Type &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to *Type*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *unique_copy* algorithm returns *unique_copy_result*< std::ranges::iterator_t<Rng>, O>. The *unique_copy* algorithm returns the pair of the source iterator to *last*, and the destination iterator to the end of the *dest* range.

template<typename **ExPolicy**, typename **Rng**, typename **O**, typename **Pred** = *ranges*::equal_to, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, unique_copy_result<*std*::ranges::iterator_t<*Rng*>, *O*>> `hpx::ranges::unique_copy`(*ExP* && icy, *Rng* && *O* dest *Pre* && = *Pre* *Pro* && = *Pro*

Copies the elements from the range *rng*, to another range beginning at *dest* in such a way that there are no consecutive equal elements. Only the first element of each group of equal elements is copied.

The assignments in the parallel *unique_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *unique_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than N assignments, exactly N - 1 applications of the predicate *pred*, where N = std::distance(begin(rng), end(rng)).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **O** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *unique_copy* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by the range *rng*. This is an binary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a, const Type &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to *Type*.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *unique_copy* algorithm returns a hpx::future<*unique_copy_result*< std::ranges::iterator_t<Rng>, O>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *unique_copy_result*< std::ranges::iterator_t<Rng>, O> otherwise. The *unique_copy* algorithm returns the pair of the source iterator to *last*, and the destination iterator to the end of the *dest* range.

hpx::ranges::merge

Defined in header hpx/algorithm.hpp[718].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::inplace_merge*

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**, typename **Iter3**, typename **Comp** = *hpx*::*ranges*::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

[718] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *hpx*::*ranges*::merge_result<*std*::ranges::iterator_t<*Rng1*>, *std*::ranges::iterator_t<*Rng2*>

Merges two sorted ranges [first1, last1) and [first2, last2) into one sorted range beginning at *dest*. The order of equivalent elements in the each of original two ranges is preserved. For equivalent elements in the original two ranges, the elements from the first range precede the elements from the second range. The destination range cannot overlap with either of the input ranges.

The assignments in the parallel *merge* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *merge* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs O(std::distance(first1, last1) + std::distance(first2, last2)) applications of the comparison *comp* and the each projection.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng1** – The type of the first source range used (deduced). The iterators extracted from this range type must meet the requirements of an random access iterator.
- **Rng2** – The type of the second source range used (deduced). The iterators extracted from this range type must meet the requirements of an random access iterator.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an random access iterator.

- **Comp** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *merge* requires *Comp* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function to be used for elements of the first range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function to be used for elements of the second range. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng1** – Refers to the first range of elements the algorithm will be applied to.
- **rng2** – Refers to the second range of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **comp** – *comp* is a callable object which returns true if the first argument is less than the second, and false otherwise. The signature of this comparison should be equivalent to:

  ```
  bool comp(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *Iter1* and *Iter2* can be dereferenced and then implicitly converted to both *Type1* and *Type2*
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual comparison *comp* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual comparison *comp* is invoked.

Returns

The *merge* algorithm returns a hpx::future<*merge_result*<*Iter1, Iter2, Iter3*>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *merge_result*<*Iter1, Iter2, Iter3*> otherwise. The *merge* algorithm returns the tuple of the source iterator *last1*, the source iterator *last2*, the destination iterator to the end of the *dest* range.

template<typename **ExPolicy**, typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Iter3**, typename **Comp** = *hpx*::*ranges*::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *hpx*::*ranges*::merge_result<*Iter1*, *Iter2*, *Iter3*>>::type `hpx::ranges::merge`(*ExPolicy* &&policy, *Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Iter3* dest, *Comp* &&comp = *Comp*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Merges two sorted ranges [first1, last1) and [first2, last2) into one sorted range beginning at *dest*. The order of equivalent elements in the each of original two ranges is preserved. For equivalent elements in the original two ranges, the elements from the first range precede the elements from the second range. The destination range cannot overlap with either of the input ranges.

The assignments in the parallel *merge* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *merge* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Performs O(std::distance(first1, last1) + std::distance(first2, last2)) applications of the comparison *comp* and the each projection.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`Iter1`** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of an random access iterator.

- **Sent1** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Iter2** – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an random access iterator.
- **Sent2** – The type of the end source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an sentinel for Iter2.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an random access iterator.
- **Comp** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *merge* requires *Comp* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function to be used for elements of the first range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function to be used for elements of the second range. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **comp** – *comp* is a callable object which returns true if the first argument is less than the second, and false otherwise. The signature of this comparison should be equivalent to:

```
bool comp(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *Iter1* and *Iter2* can be dereferenced and then implicitly converted to both *Type1* and *Type2*
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual comparison *comp* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual comparison *comp* is invoked.

Returns

The *merge* algorithm returns a hpx::future<*merge_result*<*Iter1, Iter2, Iter3*>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *merge_result*<*Iter1, Iter2, Iter3*> otherwise. The *merge* algorithm returns the tuple of the source iterator *last1*, the source iterator *last2*, the destination iterator to the end of the *dest* range.

template<typename **Rng1**, typename **Rng2**, typename **Iter3**, typename **Comp** = *hpx*::*ranges*::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*ranges*::merge_result<*std*::ranges::iterator_t<*Rng1*>, *std*::ranges::iterator_t<*Rng2*>, *Iter3*> `hpx::ranges::merge`(*Rng1* &&rng1, *Rng2* &&rng2, *Iter3* dest, *Comp* &&comp = *Comp*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Merges two sorted ranges [first1, last1) and [first2, last2) into one sorted range beginning at *dest*. The order of equivalent elements in the each of original two ranges is preserved. For equivalent elements in the original two ranges, the elements from the first range precede the elements from the second range. The destination range cannot overlap with either of the input ranges.

> **Note**
>
> Complexity: Performs O(std::distance(first1, last1) + std::distance(first2, last2)) applications of the comparison *comp* and the each projection.

Template Parameters

- **Rng1** – The type of the first source range used (deduced). The iterators extracted from this range type must meet the requirements of an random access iterator.
- **Rng2** – The type of the second source range used (deduced). The iterators extracted from this range type must meet the requirements of an random access iterator.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an random access iterator.
- **Comp** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *merge* requires *Comp* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function to be used for elements of the first range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function to be used for elements of the second range. This defaults to *hpx::identity*

Parameters

- **rng1** – Refers to the first range of elements the algorithm will be applied to.
- **rng2** – Refers to the second range of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **comp** – *comp* is a callable object which returns true if the first argument is less than the second, and false otherwise. The signature of this comparison should be equivalent to:

  ```
  bool comp(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *Iter1* and *Iter2* can be dereferenced and then implicitly converted to both *Type1* and *Type2*
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual comparison *comp* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual comparison *comp* is invoked.

Returns

The *merge* algorithm returns *merge_result<Iter1, Iter2, Iter3>*. The *merge* algorithm returns the tuple of the source iterator *last1*, the source iterator *last2*, the destination iterator to the end of the *dest* range.

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Iter3**, typename **Comp** = *hpx*::*ranges*::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*ranges*::merge_result<*Iter1*, *Iter2*, *Iter3*> **hpx::ranges::merge**(*Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Iter3* dest, *Comp* &&comp = *Comp*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Merges two sorted ranges [first1, last1) and [first2, last2) into one sorted range beginning at *dest*. The order of equivalent elements in the each of original two ranges is preserved. For equivalent elements in the original two ranges, the elements from the first range precede the elements from the second range. The destination range cannot overlap with either of the input ranges.

> **Note**
>
> Complexity: Performs O(std::distance(first1, last1) + std::distance(first2, last2)) applications of the comparison *comp* and the each projection.

Template Parameters

- **Iter1** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of an random access iterator.
- **Sent1** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.

- **Iter2** – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an random access iterator.
- **Sent2** – The type of the end source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an sentinel for Iter2.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an random access iterator.
- **Comp** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *merge* requires *Comp* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function to be used for elements of the first range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function to be used for elements of the second range. This defaults to *hpx::identity*

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **comp** – *comp* is a callable object which returns true if the first argument is less than the second, and false otherwise. The signature of this comparison should be equivalent to:

  ```
  bool comp(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *Iter1* and *Iter2* can be dereferenced and then implicitly converted to both *Type1* and *Type2*
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual comparison *comp* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual comparison *comp* is invoked.

Returns

The *merge* algorithm returns *merge_result<Iter1, Iter2, Iter3>*. The *merge* algorithm returns the tuple of the source iterator *last1*, the source iterator *last2*, the destination iterator to the end of the *dest* range.

hpx::ranges::inplace_merge

Defined in header hpx/algorithm.hpp[719].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::merge*

template<typename **ExPolicy**, typename **Rng**, typename **Iter**, typename **Comp** = *hpx*::*ranges*::less, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *Iter*> `hpx::ranges::inplace_merge`(*ExPolicy* &&policy, *Rng* &&rng, *Iter* middle, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Merges two consecutive sorted ranges [first, middle) and [middle, last) into one sorted range [first, last). The order of equivalent elements in the each of original two ranges is preserved. For equivalent elements in the original two ranges, the elements from the first range precede the elements from the second range.

The assignments in the parallel *inplace_merge* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *inplace_merge* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs O(std::distance(first, last)) applications of the comparison *comp* and the each projection.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an random access iterator.
- **Iter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random access iterator.
- **Comp** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *inplace_merge* requires *Comp* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

[719] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the range of elements the algorithm will be applied to.
- **middle** – Refers to the end of the first sorted range and the beginning of the second sorted range the algorithm will be applied to.
- **comp** – *comp* is a callable object which returns true if the first argument is less than the second, and false otherwise. The signature of this comparison should be equivalent to:

  ```
  bool comp(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *Iter* can be dereferenced and then implicitly converted to both *Type1* and *Type2*
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *inplace_merge* algorithm returns a *hpx::future<Iter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *Iter* otherwise. The *inplace_merge* algorithm returns the source iterator *last*

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **Comp** = *hpx*::*ranges*::less, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *Iter*> `hpx::ranges::inplace_merge`(*ExPolicy* &&policy, *Iter* first, *Iter* middle, *Sent* last, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Merges two consecutive sorted ranges [first, middle) and [middle, last) into one sorted range [first, last). The order of equivalent elements in the each of original two ranges is preserved. For equivalent elements in the original two ranges, the elements from the first range precede the elements from the second range.

The assignments in the parallel *inplace_merge* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *inplace_merge* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Performs O(std::distance(first, last)) applications of the comparison *comp* and the each projection.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Iter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random access iterator.
- **Sent** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Comp** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *inplace_merge* requires *Comp* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the first sorted range the algorithm will be applied to.
- **middle** – Refers to the end of the first sorted range and the beginning of the second sorted range the algorithm will be applied to.
- **last** – Refers to the end of the second sorted range the algorithm will be applied to.
- **comp** – *comp* is a callable object which returns true if the first argument is less than the second, and false otherwise. The signature of this comparison should be equivalent to:

  ```
  bool comp(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *Iter* can be dereferenced and then implicitly converted to both *Type1* and *Type2*
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *inplace_merge* algorithm returns a *hpx::future<Iter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *Iter* otherwise. The *inplace_merge* algorithm returns the source iterator *last*

template<typename **Rng**, typename **Iter**, typename **Comp** = *hpx*::*ranges*::less, typename **Proj** = *hpx*::identity>

Iter hpx::ranges::**inplace_merge**(*Rng* &&rng, *Iter* middle, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Merges two consecutive sorted ranges [first, middle) and [middle, last) into one sorted range [first, last). The order of equivalent elements in the each of original two ranges is preserved. For equivalent elements in the original two ranges, the elements from the first range precede the elements from the second range.

> **Note**
>
> Complexity: Performs O(std::distance(first, last)) applications of the comparison *comp* and the each projection.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an random access iterator.
- **Iter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random access iterator.
- **Comp** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *inplace_merge* requires *Comp* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the range of elements the algorithm will be applied to.
- **middle** – Refers to the end of the first sorted range and the beginning of the second sorted range the algorithm will be applied to.
- **comp** – *comp* is a callable object which returns true if the first argument is less than the second, and false otherwise. The signature of this comparison should be equivalent to:

  ```
  bool comp(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *Iter* can be dereferenced and then implicitly converted to both *Type1* and *Type2*
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *inplace_merge* algorithm returns *Iter*. The *inplace_merge* algorithm returns the source iterator *last*

template<typename **Iter**, typename **Sent**, typename **Comp** = *hpx*::*ranges*::less, typename **Proj** = *hpx*::identity>

Iter hpx::ranges::**inplace_merge**(*Iter* first, *Iter* middle, *Sent* last, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Merges two consecutive sorted ranges [first, middle) and [middle, last) into one sorted range [first, last). The order of equivalent elements in the each of original two ranges is preserved. For equivalent elements in the original two ranges, the elements from the first range precede the elements from the second range.

> **Note**

Complexity: Performs O(std::distance(first, last)) applications of the comparison *comp* and the each projection.

Template Parameters

- **Iter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random access iterator.
- **Sent** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Comp** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *inplace_merge* requires *Comp* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the first sorted range the algorithm will be applied to.
- **middle** – Refers to the end of the first sorted range and the beginning of the second sorted range the algorithm will be applied to.
- **last** – Refers to the end of the second sorted range the algorithm will be applied to.
- **comp** – *comp* is a callable object which returns true if the first argument is less than the second, and false otherwise. The signature of this comparison should be equivalent to:

  ```
  bool comp(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *Iter* can be dereferenced and then implicitly converted to both *Type1* and *Type2*
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *inplace_merge* algorithm *Iter*. The *inplace_merge* algorithm returns the source iterator *last*

hpx::ranges::search

Defined in header hpx/algorithm.hpp[720].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::search_n*

template<typename **FwdIter**, typename **Sent**, typename **FwdIter2**, typename **Sent2**, typename **Pred** = *hpx*::*ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

[720] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

FwdIter `hpx::ranges::search`(*FwdIter* first, *Sent* last, *FwdIter2* s_first, *Sent2* s_last, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Searches the range [first, last) for any elements in the range [s_first, s_last). Uses a provided predicate to compare elements.

The comparison operations in the parallel *search* algorithm execute in sequential order in the calling thread.

Note

Complexity: at most (S*N) comparisons where *S* = distance(s_first, s_last) and *N* = distance(first, last).

Template Parameters

- **FwdIter** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel used for the first range (deduced). This iterator type must meet the requirements of an sentinel.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent2** – The type of the source sentinel used for the second range (deduced). This iterator type must meet the requirements of an sentinel.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *adjacent_find* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj1** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of type dereferenced *FwdIter*.
- **Proj2** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of type dereferenced *FwdIter2*.

Parameters

- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **s_first** – Refers to the beginning of the sequence of elements the algorithm will be searching for.
- **s_last** – Refers to the end of the sequence of elements of the algorithm will be searching for.
- **op** – Refers to the binary predicate which returns true if the elements should be treated as equal. the signature of the function should be equivalent to

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of type dereferenced *FwdIter1* as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of type dereferenced *FwdIter2* as a projection operation before the actual predicate *is* invoked.

Returns

The *search* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *task_execution_policy* and returns *FwdIter* otherwise. The *search* algorithm returns an iterator to the beginning of the first subsequence [s_first, s_last) in range [first, last). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, last), *last* is returned. Additionally if the size of the subsequence is empty *first* is returned. If no subsequence is found, *last* is returned.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **FwdIter2**, typename **Sent2**, typename **Pred** = *hpx*::*ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter*>::type `hpx::ranges::search`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *FwdIter2* s_first, *Sent2* s_last, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Searches the range [first, last) for any elements in the range [s_first, s_last). Uses a provided predicate to compare elements.

The comparison operations in the parallel *search* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *search* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (S*N) comparisons where *S* = distance(s_first, s_last) and *N* = distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.

- **FwdIter** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel used for the first range (deduced). This iterator type must meet the requirements of an sentinel.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent2** – The type of the source sentinel used for the second range (deduced). This iterator type must meet the requirements of an sentinel.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *adjacent_find* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj1** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of type dereferenced *FwdIter*.
- **Proj2** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of type dereferenced *FwdIter2*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **s_first** – Refers to the beginning of the sequence of elements the algorithm will be searching for.
- **s_last** – Refers to the end of the sequence of elements of the algorithm will be searching for.
- **op** – Refers to the binary predicate which returns true if the elements should be treated as equal. the signature of the function should be equivalent to

```
bool pred(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of type dereferenced *FwdIter1* as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of type dereferenced *FwdIter2* as a projection operation before the actual predicate *is* invoked.

Returns

The *search* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *task_execution_policy* and returns *FwdIter* otherwise. The *search* algorithm returns an iterator to the beginning of the first subsequence [s_first, s_last) in range [first, last). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, last), *last* is returned. Additionally if the size of the subsequence is empty *first* is returned. If no subsequence is found, *last* is returned.

template<typename **Rng1**, typename **Rng2**, typename **Pred** = *hpx*::*ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

std::ranges::iterator_t<*Rng1*> `hpx::ranges::search`(*Rng1* &&rng1, *Rng2* &&rng2, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Searches the range [first, last) for any elements in the range [s_first, s_last). Uses a provided predicate to compare elements.

The comparison operations in the parallel *search* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: at most (S*N) comparisons where S = distance(s_first, s_last) and N = distance(first, last).

Template Parameters

- **Rng1** – The type of the examine range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the search range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *adjacent_find* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj1** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of *Rng1*.
- **Proj2** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of *Rng2*.

Parameters

- **rng1** – Refers to the sequence of elements the algorithm will be examining.
- **rng2** – Refers to the sequence of elements the algorithm will be searching for.
- **op** – Refers to the binary predicate which returns true if the elements should be treated as equal. the signature of the function should be equivalent to

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of *rng1* as a projection operation before the actual predicate *is* invoked.

- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of *rng2* as a projection operation before the actual predicate *is* invoked.

Returns

The *search* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *task_execution_policy* and returns *FwdIter* otherwise. The *search* algorithm returns an iterator to the beginning of the first subsequence [s_first, s_last) in range [first, last). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, last), *last* is returned. Additionally if the size of the subsequence is empty *first* is returned. If no subsequence is found, *last* is returned.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**, typename **Pred** = *hpx*::*ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng1*>> `hpx::ranges::search`(*ExPolicy* &&policy, *Rng1* &&rng1, *Rng2* &&rng2, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Searches the range [first, last) for any elements in the range [s_first, s_last). Uses a provided predicate to compare elements.

The comparison operations in the parallel *search* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *search* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (S*N) comparisons where *S* = distance(s_first, s_last) and *N* = distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.

- **Rng1** – The type of the examine range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the search range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *adjacent_find* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj1** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of *Rng1*.
- **Proj2** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of *Rng2*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng1** – Refers to the sequence of elements the algorithm will be examining.
- **rng2** – Refers to the sequence of elements the algorithm will be searching for.
- **op** – Refers to the binary predicate which returns true if the elements should be treated as equal. the signature of the function should be equivalent to

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of *rng1* as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of *rng2* as a projection operation before the actual predicate *is* invoked.

Returns

The *search* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *task_execution_policy* and returns *FwdIter* otherwise. The *search* algorithm returns an iterator to the beginning of the first subsequence [s_first, s_last) in range [first, last). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, last), *last* is returned. Additionally if the size of the subsequence is empty *first* is returned. If no subsequence is found, *last* is returned.

hpx::ranges::search_n

Defined in header hpx/algorithm.hpp[721].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::search*

template<typename **FwdIter**, typename **FwdIter2**, typename **Sent2**, typename **Pred** = *hpx*::*ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

FwdIter `hpx::ranges::search_n`(*FwdIter* first, *std*::size_t count, *FwdIter2* s_first, Sent s_last, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Searches the range [first, last) for any elements in the range [s_first, s_last). Uses a provided predicate to compare elements.

The comparison operations in the parallel *search_n* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: at most (S*N) comparisons where *S* = distance(s_first, s_last) and *N* = count.

Template Parameters

- **FwdIter** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent2** – The type of the source sentinel used for the second range (deduced). This iterator type must meet the requirements of an sentinel.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *adjacent_find* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj1** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of type dereferenced *FwdIter*.
- **Proj2** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of type dereferenced *FwdIter2*.

Parameters

- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **count** – Refers to the range of elements of the first range the algorithm will be applied to.

[721] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **s_first** – Refers to the beginning of the sequence of elements the algorithm will be searching for.
- **s_last** – Refers to the end of the sequence of elements of the algorithm will be searching for.
- **op** – Refers to the binary predicate which returns true if the elements should be treated as equal. the signature of the function should be equivalent to

```
bool pred(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of type dereferenced *FwdIter1* as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of type dereferenced *FwdIter2* as a projection operation before the actual predicate *is* invoked.

Returns

The *search_n* algorithm returns *FwdIter*. The *search_n* algorithm returns an iterator to the beginning of the last subsequence [s_first, s_last) in range [first, first+count). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, first+count), *first* is returned. Additionally, if the size of the subsequence is empty or no subsequence is found, *first* is also returned.

template<typename **ExPolicy**, typename **FwdIter**, typename **FwdIter2**, typename **Sent2**, typename **Pred** = *hpx*::*ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter*>::type `hpx::ranges::search_n`(*ExPolicy* &&policy, *FwdIter* first, *std*::size_t count, *FwdIter2* s_first, *Sent2* s_last, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Searches the range [first, last) for any elements in the range [s_first, s_last). Uses a provided predicate to compare elements.

The comparison operations in the parallel *search_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *search_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (S*N) comparisons where *S* = distance(s_first, s_last) and *N* = count.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent2** – The type of the source sentinel used for the second range (deduced). This iterator type must meet the requirements of an sentinel.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *adjacent_find* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj1** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of type dereferenced *FwdIter*.
- **Proj2** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of type dereferenced *FwdIter2*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **count** – Refers to the range of elements of the first range the algorithm will be applied to.
- **s_first** – Refers to the beginning of the sequence of elements the algorithm will be searching for.
- **s_last** – Refers to the end of the sequence of elements of the algorithm will be searching for.
- **op** – Refers to the binary predicate which returns true if the elements should be treated as equal. the signature of the function should be equivalent to

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of type dereferenced *FwdIter1* as a projection operation before the actual predicate *is* invoked.

- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of type dereferenced *FwdIter2* as a projection operation before the actual predicate *is* invoked.

Returns

The *search_n* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *task_execution_policy* and returns *FwdIter* otherwise. The *search_n* algorithm returns an iterator to the beginning of the last subsequence [s_first, s_last) in range [first, first+count). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, first+count), *first* is returned. Additionally if the size of the subsequence is empty or no subsequence is found, *first* is also returned.

template<typename **Rng1**, typename **Rng2**, typename **Pred** = *hpx*::*ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

std::ranges::iterator_t<*Rng1*> `hpx::ranges::search_n`(*Rng1* &&rng1, *std*::size_t count, *Rng2* &&rng2, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Searches the range [first, last) for any elements in the range [s_first, s_last). Uses a provided predicate to compare elements.

The comparison operations in the parallel *search* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: at most (S*N) comparisons where *S* = distance(s_first, s_last) and *N* = distance(first, last).

Template Parameters

- **Rng1** – The type of the examine range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the search range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *adjacent_find* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj1** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of *Rng1*.
- **Proj2** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of *Rng2*.

Parameters

- **rng1** – Refers to the sequence of elements the algorithm will be examining.
- **count** – The number of elements to apply the algorithm on.
- **rng2** – Refers to the sequence of elements the algorithm will be searching for.

- **op** – Refers to the binary predicate which returns true if the elements should be treated as equal. the signature of the function should be equivalent to

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of *rng1* as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of *rng2* as a projection operation before the actual predicate *is* invoked.

Returns

The *search* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *task_execution_policy* and returns *FwdIter* otherwise. The *search* algorithm returns an iterator to the beginning of the first subsequence [s_first, s_last) in range [first, last). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, last), *last* is returned. Additionally if the size of the subsequence is empty *first* is returned. If no subsequence is found, *last* is returned.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**, typename **Pred** = *hpx*::*ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng1*>> `hpx::ranges::search_n`(*ExPolicy* &&policy, *Rng1* &&rng1, *std*::size_t count, *Rng2* &&rng2, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Searches the range [first, last) for any elements in the range [s_first, s_last). Uses a provided predicate to compare elements.

The comparison operations in the parallel *search* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *search* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (S*N) comparisons where S = distance(s_first, s_last) and N = distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng1** – The type of the examine range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the search range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *adjacent_find* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj1** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of *Rng1*.
- **Proj2** – The type of an optional projection function. This defaults to *hpx::identity* and is applied to the elements of *Rng2*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng1** – Refers to the sequence of elements the algorithm will be examining.
- **count** – The number of elements to apply the algorithm on.
- **rng2** – Refers to the sequence of elements the algorithm will be searching for.
- **op** – Refers to the binary predicate which returns true if the elements should be treated as equal. the signature of the function should be equivalent to

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of *rng1* as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of *rng2* as a projection operation before the actual predicate *is* invoked.

Returns

The *search* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *task_execution_policy* and returns *FwdIter* otherwise. The *search* algorithm returns an iterator to the beginning of the first subsequence [s_first, s_last) in range [first, last). If the length of the subsequence [s_first, s_last) is greater than the length of the range [first, last), *last* is returned. Additionally if the size of the subsequence is empty *first* is returned. If no subsequence is found, *last* is returned.

hpx::ranges::is_partitioned

Defined in header hpx/algorithm.hpp[722].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **FwdIter**, typename **Sent**, typename **Pred**, typename **Proj** = *hpx*::identity>

bool *hpx*::*ranges*::**is_partitioned**(*FwdIter* first, *Sent* last, *Pred* &&pred, *Proj* &&proj = *Proj*())

Determines if the range [first, last) is partitioned.

> **Note**
>
> Complexity: at most (N) predicate evaluations where *N* = distance(first, last).

Template Parameters

- **FwdIter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **Pred** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*.

Parameters

- **first** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of that the algorithm will be applied to.
- **pred** – Refers to the unary predicate which returns true for elements expected to be found in the beginning of the range. The signature of the function should be equivalent to

```
bool pred(const Type &a);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.

[722] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_partitioned* algorithm returns *bool*. The *is_partitioned* algorithm returns true if each element in the sequence for which pred returns true precedes those for which pred returns false. Otherwise is_partitioned returns false. If the range [first, last) contains less than two elements, the function is always true.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **Pred**, typename **Proj** = *hpx*::identity>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::is_partitioned`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *Pred* &&pred, *Proj* &&proj = *Proj*())

Determines if the range [first, last) is partitioned.

The predicate operations in the parallel *is_partitioned* algorithm invoked with an execution policy object of type *sequenced_policy* executes in sequential order in the calling thread.

The comparison operations in the parallel *is_partitioned* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (N) predicate evaluations where *N* = distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **Pred** – The type of the function/function object to use (deduced). *Pred* must be *CopyConstructible* when using a parallel policy.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.

- **first** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of that the algorithm will be applied to.
- **pred** – Refers to the unary predicate which returns true for elements expected to be found in the beginning of the range. The signature of the function should be equivalent to

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_partitioned* algorithm returns a *hpx::future<bool>* if the execution policy is of type *task_execution_policy* and returns *bool* otherwise. The *is_partitioned* algorithm returns true if each element in the sequence for which pred returns true precedes those for which pred returns false. Otherwise is_partitioned returns false. If the range [first, last) contains less than two elements, the function is always true.

template<typename **Rng**, typename **Pred**, typename **Proj** = *hpx*::identity>

bool *hpx::ranges::***is_partitioned**(*Rng* &&rng, *Pred* &&pred, *Proj* &&proj = *Proj*())

Determines if the range rng is partitioned.

> **Note**
>
> Complexity: at most (N) predicate evaluations where *N* = std::size(rng).

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Refers to the unary predicate which returns true for elements expected to be found in the beginning of the range. The signature of the function should be equivalent to

  ```
  bool pred(const Type &a);
  ```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_partitioned* algorithm returns *bool*. The *is_partitioned* algorithm returns true if each element in the sequence for which pred returns true precedes those for which pred returns false. Otherwise is_partitioned returns false. If the range rng contains less than two elements, the function is always true.

template<typename **ExPolicy**, typename **Rng**, typename **Pred**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::is_partitioned`(*ExPolicy* &&policy, *Rng* &&rng, *Pred* &&pred, *Proj* &&proj = *Proj*())

Determines if the range [first, last) is partitioned.

The predicate operations in the parallel *is_partitioned* algorithm invoked with an execution policy object of type *sequenced_policy* executes in sequential order in the calling thread.

The comparison operations in the parallel *is_partitioned* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (N) predicate evaluations where *N* = std::size(rng).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). *Pred* must be *CopyConstructible* when using a parallel policy.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Refers to the unary predicate which returns true for elements expected to be found in the beginning of the range. The signature of the function should be equivalent to

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_partitioned* algorithm returns a *hpx::future<bool>* if the execution policy is of type *task_execution_policy* and returns *bool* otherwise. The *is_partitioned* algorithm returns true if each element in the sequence for which pred returns true precedes those for which pred returns false. Otherwise is_partitioned returns false. If the range rng contains less than two elements, the function is always true.

hpx::ranges::starts_with

Defined in header hpx/algorithm.hpp[723].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Pred** = *ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

bool *hpx*::*ranges*::**starts_with**(*Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Pred* &&pred = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Checks whether the second range defined by [first1, last1) matches the prefix of the first range defined by [first2, last2)

The assignments in the parallel *starts_with* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear: at most min(N1, N2) applications of the predicate and both projections.

Template Parameters

- **Iter1** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent1** – The type of the end source iterators used(deduced). This iterator type must meet the requirements of an sentinel for Iter1.

[723] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **Iter2** – The type of the begin destination iterators used deduced). This iterator type must meet the requirements of a input iterator.
- **Sent2** – The type of the end destination iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter2.
- **Pred** – The binary predicate that compares the projected elements.
- **Proj1** – The type of an optional projection function for the source range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function for the destination range. This defaults to *hpx::identity*

Parameters

- **first1** – Refers to the beginning of the source range.
- **last1** – Sentinel value referring to the end of the source range.
- **first2** – Refers to the beginning of the destination range.
- **last2** – Sentinel value referring to the end of the destination range.
- **pred** – Specifies the binary predicate function (or function object) which will be invoked for comparison of the elements in the in two ranges projected by proj1 and proj2 respectively.
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements in the source range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements in the destination range as a projection operation before the actual predicate *is* invoked.

Returns

The *starts_with* algorithm returns *bool*. The *starts_with* algorithm returns a boolean with the value true if the second range matches the prefix of the first range, false otherwise.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent1**, typename **FwdIter2**, typename **Sent2**, typename **Pred** = *ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::starts_with`(*ExPolicy* &&policy, *FwdIter1* first1, *Sent1* last1, *FwdIter2* first2, *Sent2* last2, *Pred* &&pred = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Checks whether the second range defined by [first1, last1) matches the prefix of the first range defined by [first2, last2)

The assignments in the parallel *starts_with* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *starts_with* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Linear: at most min(N1, N2) applications of the predicate and both projections.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent1** – The type of the end source iterators used(deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **FwdIter2** – The type of the begin destination iterators used deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent2** – The type of the end destination iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter2.
- **Pred** – The binary predicate that compares the projected elements.
- **Proj1** – The type of an optional projection function for the source range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function for the destination range. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the source range.
- **last1** – Sentinel value referring to the end of the source range.
- **first2** – Refers to the beginning of the destination range.
- **last2** – Sentinel value referring to the end of the destination range.
- **pred** – Specifies the binary predicate function (or function object) which will be invoked for comparison of the elements in the in two ranges projected by proj1 and proj2 respectively.
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements in the source range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements in the destination range as a projection operation before the actual predicate *is* invoked.

Returns

The *starts_with* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *starts_with* algorithm returns a boolean with the value true if the second range matches the prefix of the first range, false otherwise.

template<typename **Rng1**, typename **Rng2**, typename **Pred** = *ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

bool `hpx::ranges::starts_with`(*Rng1* &&rng1, *Rng2* &&rng2, *Pred* &&pred = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Checks whether the second range *rng2* matches the prefix of the first range *rng1*.

The assignments in the parallel *starts_with* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Note

Complexity: Linear: at most min(N1, N2) applications of the predicate and both projections.

Template Parameters

- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Rng2** – The type of the destination range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The binary predicate that compares the projected elements.
- **Proj1** – The type of an optional projection function for the source range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function for the destination range. This defaults to *hpx::identity*

Parameters

- **rng1** – Refers to the source range.
- **rng2** – Refers to the destination range.
- **pred** – Specifies the binary predicate function (or function object) which will be invoked for comparison of the elements in the in two ranges projected by proj1 and proj2 respectively.
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements in the source range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements in the destination range as a projection operation before the actual predicate *is* invoked.

Returns

The *starts_with* algorithm returns *bool*. The *starts_with* algorithm returns a boolean with the value true if the second range matches the prefix of the first range, false otherwise.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**, typename **Pred** = *ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, bool>::type `hpx::ranges::starts_with`(*ExPolicy* &&policy, *Rng1* &&rng1, *Rng2* &&rng2, *Pred* &&pred = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Checks whether the second range *rng2* matches the prefix of the first range *rng1*.

The assignments in the parallel *starts_with* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *starts_with* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Linear: at most min(N1, N2) applications of the predicate and both projections.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Rng2** – The type of the destination range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The binary predicate that compares the projected elements.
- **Proj1** – The type of an optional projection function for the source range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function for the destination range. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng1** – Refers to the source range.
- **rng2** – Refers to the destination range.
- **pred** – Specifies the binary predicate function (or function object) which will be invoked for comparison of the elements in the in two ranges projected by proj1 and proj2 respectively.
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements in the source range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements in the destination range as a projection operation before the actual predicate *is* invoked.

Returns

The *starts_with* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *starts_with* algorithm returns a boolean with the value true if the second range matches the prefix of the first range, false otherwise.

hpx::ranges::uninitialized_copy

Defined in header hpx/algorithm.hpp[724].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::uninitialized_copy_n*

template<typename **InIter**, typename **Sent1**, typename **FwdIter**, typename **Sent2**>

hpx::*parallel*::*util*::in_out_result<*InIter*, *FwdIter*> `hpx::ranges::uninitialized_copy`(*InIter* first1, *Sent1* last1, *FwdIter* first2, *Sent2* last2)

Copies the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the copy operation, the function has no effects.

The assignments in the parallel *uninitialized_copy* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

[724] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent1** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent2** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter2.

Parameters

- **first1** – Refers to the beginning of the sequence of elements that will be copied from
- **last1** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied
- **first2** – Refers to the beginning of the destination range.
- **last2** – Refers to sentinel value denoting the end of the second range the algorithm will be applied to.

Returns

The *uninitialized_copy* algorithm returns an *in_out_result<InIter, FwdIter>*. The *uninitialized_copy* algorithm returns an input iterator to one past the last element copied from and the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent1**, typename **FwdIter2**, typename **Sent2**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *parallel*::*util*::in_out_result<*FwdIter1*, *FwdIter2*>>::type *hpx*::*ranges*::**uninitialized_**

Copies the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the copy operation, the function has no effects.

The assignments in the parallel *uninitialized_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- `ExPolicy` – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- `FwdIter1` – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- `Sent1` – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- `FwdIter2` – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- `Sent2` – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter2.

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `first1` – Refers to the beginning of the sequence of elements that will be copied from
- `last1` – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- `first2` – Refers to the beginning of the destination range.
- `last2` – Refers to sentinel value denoting the end of the second range the algorithm will be applied to.

Returns

The *uninitialized_copy* algorithm returns a hpx::future<*in_out_result<InIter, FwdIter>*>, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *in_out_result<InIter, FwdIter>* otherwise. The *uninitialized_copy* algorithm returns an input iterator to one past the last element copied from and the output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng1**, typename **Rng2**>

hpx::*parallel*::*util*::in_out_result<typename *hpx*::*traits*::range_traits<*Rng1*>::iterator_type, typename *hpx*::*traits*::range_traits<*Rng2*>::iterator

Copies the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the copy operation, the function has no effects.

The assignments in the parallel *uninitialized_copy* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the destination range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.

Parameters

- **rng1** – Refers to the range from which the elements will be copied from
- **rng2** – Refers to the range to which the elements will be copied to

Returns

The *uninitialized_copy* algorithm returns an *in_out_result<typename* hpx::traits::range_traits<Rng1>::iterator_type, typename hpx::traits::range_traits<Rng2>::iterator_type>. The *uninitialized_copy* algorithm returns an input iterator to one past the last element copied from and the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *hpx*::*parallel*::*util*::in_out_result<typename *hpx*::*traits*::range_traits<*Rng1*>::iterator_type, ty

Copies the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the copy operation, the function has no effects.

The assignments in the parallel *uninitialized_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the destination range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng1** – Refers to the range from which the elements will be copied from
- **rng2** – Refers to the range to which the elements will be copied to

Returns

The *uninitialized_copy* algorithm returns a hpx::future<*in_out_result*<*InIter, FwdIter*>>, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *in_out_result*< typename hpx::traits::range_traits<Rng1>::iterator_type , typename hpx::traits::range_traits<Rng2>::iterator_type> otherwise. The *uninitialized_copy* algorithm returns the input iterator to one past the last element copied from and the output iterator to the element in the destination range, one past the last element copied.

hpx::ranges::uninitialized_copy_n

Defined in header hpx/algorithm.hpp[725].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::uninitialized_copy*

template<typename **InIter**, typename **Size**, typename **FwdIter**, typename **Sent2**>

hpx::*parallel*::*util*::in_out_result<*InIter*, *FwdIter*> `hpx::ranges::`**`uninitialized_copy_n`**(*InIter* first1, *Size* count, *FwdIter* first2, *Sent2* last2)

Copies the elements in the range [first, first + count), starting from first and proceeding to first + count - 1., to another range beginning at dest. If an exception is thrown during the copy operation, the function has no effects.

The assignments in the parallel *uninitialized_copy_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

[725] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Note

Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- `InIter` – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- `Size` – The type of the argument specifying the number of elements to apply *f* to.
- `FwdIter` – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- `Sent2` – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.

Parameters

- `first1` – Refers to the beginning of the sequence of elements that will be copied from
- `count` – Refers to the number of elements starting at *first* the algorithm will be applied to.
- `first2` – Refers to the beginning of the destination range.
- `last2` – Refers to sentinel value denoting the end of the second range the algorithm will be applied to.

Returns

The *uninitialized_copy_n* algorithm returns *in_out_result<InIter, FwdIter>*. The *uninitialized_copy_n* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Size**, typename **FwdIter2**, typename **Sent2**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *parallel*::*util*::in_out_result<*FwdIter1*, *FwdIter2*>>::type `hpx::ranges::uninitialized_`

Copies the elements in the range [first, first + count), starting from first and proceeding to first + count - 1., to another range beginning at dest. If an exception is thrown during the copy operation, the function has no effects.

The assignments in the parallel *uninitialized_copy_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_copy_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- `ExPolicy` – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- `FwdIter1` – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- `Size` – The type of the argument specifying the number of elements to apply *f* to.
- `FwdIter2` – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- `Sent2` – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter2.

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `first1` – Refers to the beginning of the sequence of elements that will be copied from
- `count` – Refers to the number of elements starting at *first* the algorithm will be applied to.
- `first2` – Refers to the beginning of the destination range.
- `last2` – Refers to sentinel value denoting the end of the second range the algorithm will be applied to.

Returns

The *uninitialized_copy_n* algorithm returns a hpx::future<*in_out_result*<*FwdIter1, FwdIter2*>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *uninitialized_copy_n* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::ranges::partial_sort_copy

Defined in header hpx/algorithm.hpp[726].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter**, typename **Sent1**, typename **RandIter**, typename **Sent2**, typename **Comp** = *ranges*::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

partial_sort_copy_result<*InIter*, *RandIter*> `hpx::ranges::partial_sort_copy`(*InIter* first, *Sent1* last, *RandIter* r_first, *Sent2* r_last, *Comp* &&comp = *Comp*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Sorts some of the elements in the range [first, last) in ascending order, storing the result in the range [r_first, r_last). At most r_last - r_first of the elements are placed sorted to the range [r_first, r_first + n) where n is the number of elements to sort (n = min(last - first, r_last - r_first)).

The assignments in the parallel *partial_sort_copy* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Note

Complexity: O(N log(min(D,N))), where N = std::distance(first, last) and D = std::distance(r_first, r_last) comparisons.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent1** – The type of the source sentinel (deduced).This sentinel type must be a sentinel for InIter.
- **RandIter** – The type of the destination iterators used(deduced) This iterator type must meet the requirements of an random iterator.
- **Sent2** – The type of the destination sentinel (deduced).This sentinel type must be a sentinel for RandIter.
- **Comp** – The type of the function/function object to use (deduced). Comp defaults to detail::less.
- **Proj1** – The type of an optional projection function for the input range. This defaults to *hpx::identity*.
- **Proj1** – The type of an optional projection function for the output range. This defaults to *hpx::identity*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.

[726] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **last** – Refers to the sentinel value denoting the end of the sequence of elements the algorithm will be applied to.
- **r_first** – Refers to the beginning of the destination range.
- **r_last** – Refers to the sentinel denoting the end of the destination range.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator. This defaults to detail::less.
- **proj1** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation after the actual predicate *comp* is invoked.

Returns

The *partial_sort_copy* algorithm returns a returns *partial_sort_copy_result<InIter, RandIter>*. The algorithm returns {last, result_first + N}.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent1**, typename **RandIter**, typename **Sent2**, typename **Comp** = *ranges*::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, partial_sort_copy_result<*FwdIter*, *RandIter*>> `hpx::ranges::`**`partial_sort_copy`**(*Ex*
&
icy
Fv
fir
Se
las
Ra
dI
r_
Se
r_
Co
&
=
Co
Pr
&
=
Pr
Pr
&
=
Pr

Sorts some of the elements in the range [first, last) in ascending order, storing the result in the range [r_first, r_last). At most r_last - r_first of the elements are placed sorted to the range [r_first, r_first + n) where n is the number of elements to sort (n = min(last - first, r_last - r_first)).

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(N log(min(D,N))), where N = std::distance(first, last) and D = std::distance(r_first, r_last) comparisons.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent1** – The type of the source sentinel (deduced).This sentinel type must be a sentinel for FwdIter.
- **RandIter** – The type of the destination iterators used(deduced) This iterator type must meet the requirements of an random iterator.
- **Sent2** – The type of the destination sentinel (deduced).This sentinel type must be a sentinel for RandIter.
- **Comp** – The type of the function/function object to use (deduced). Comp defaults to detail::less.
- **Proj1** – The type of an optional projection function for the input range. This defaults to *hpx::identity*.
- **Proj1** – The type of an optional projection function for the output range. This defaults to *hpx::identity*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the sentinel value denoting the end of the sequence of elements the algorithm will be applied to.
- **r_first** – Refers to the beginning of the destination range.
- **r_last** – Refers to the sentinel denoting the end of the destination range.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator. This defaults to detail::less.

- **proj1** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation after the actual predicate *comp* is invoked.

Returns

The *partial_sort_copy* algorithm returns a hpx::future<*partial_sort_copy_result*<*FwdIter, RandIter*>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *partial_sort_copy_result*<*FwdIter, RandIter*> otherwise. The algorithm returns {last, result_first + N}.

template<typename **Rng1**, typename **Rng2**, typename **Comp** = *ranges*::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

partial_sort_copy_result<*std*::ranges::iterator_t<*Rng1*>, *std*::ranges::iterator_t<*Rng2*>> `hpx::ranges::partial_sort_copy`(*Rng1* &&rng1, *Rng2* &&rng2, *Comp* &&comp = *Comp*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Sorts some of the elements in the range [first, last) in ascending order, storing the result in the range [r_first, r_last). At most r_last - r_first of the elements are placed sorted to the range [r_first, r_first + n) where n is the number of elements to sort (n = min(last - first, r_last - r_first)).

The assignments in the parallel *partial_sort_copy* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: O(N log(min(D,N))), where N = std::distance(first, last) and D = std::distance(r_first, r_last) comparisons.

Template Parameters

- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a input iterator.
- **Rng2** – The type of the destination range used (deduced). The iterators extracted from this range type must meet the requirements of a random iterator.

- **Comp** – The type of the function/function object to use (deduced). Comp defaults to detail::less.
- **Proj1** – The type of an optional projection function for the input range. This defaults to *hpx::identity*.
- **Proj2** – The type of an optional projection function for the output range. This defaults to *hpx::identity*.

Parameters

- **rng1** – Refers to the source range.
- **rng2** – Refers to the destination range.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator. This defaults to detail::less.
- **proj1** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation after the actual predicate *comp* is invoked.

Returns

The *partial_sort_copy* algorithm returns *partial_sort_copy_result<range_iterator_t<Rng1>*, range_iterator_t<Rng2>>. The algorithm returns {last, result_first + N}.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**, typename **Comp** = *ranges*::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, partial_sort_copy_result<*std*::ranges::iterator_t<*Rng1*>, *std*::ranges::iterator_t<*Rng2*>>> h

Sorts some of the elements in the range [first, last) in ascending order, storing the result in the range [r_first, r_last). At most r_last - r_first of the elements are placed sorted to the range [r_first, r_first + n) where n is the number of elements to sort (n = min(last - first, r_last - r_first)).

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(N log(min(D,N))), where N = std::distance(first, last) and D = std::distance(r_first, r_last) comparisons.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a forward iterator.
- **Rng2** – The type of the destination range used (deduced). The iterators extracted from this range type must meet the requirements of a random iterator.
- **Comp** – The type of the function/function object to use (deduced). Comp defaults to detail::less.
- **Proj1** – The type of an optional projection function for the input range. This defaults to *hpx::identity*.
- **Proj2** – The type of an optional projection function for the output range. This defaults to *hpx::identity*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng1** – Refers to the source range.
- **rng2** – Refers to the destination range.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator. This defaults to detail::less.
- **proj1** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation after the actual predicate *comp* is invoked.

Returns

The *partial_sort_copy* algorithm returns a hpx::future<*partial_sort_copy_result*< range_iterator_t<Rng1>, range_iterator_t<Rng2>>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *partial_sort_copy_result<range_iterator_t<Rng1>*, range_iterator_t<Rng2>> otherwise. The algorithm returns {last, result_first + N}.

hpx::ranges::inclusive_scan

Defined in header hpx/algorithm.hpp[727].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter**, typename **Sent**, typename **OutIter**, typename **Op** = *std*::plus<typename *std*::iterator_traits<*InIter*>::value_type>>
inclusive_scan_result<*InIter*, *OutIter*> `hpx::ranges::inclusive_scan`(*InIter* first, *Sent* last, *OutIter* dest, *Op* &&op)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, *first, …, *(first + (i - result))).

The reduce operations in the parallel *inclusive_scan* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum.

Note

Complexity: O(*last* - *first*) applications of the predicate *op*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, …, aN) is defined as:

- a1 when N is 1
- GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, …, aK)
- GENERALIZED_NONCOMMUTATIVE_SUM(+, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Op** – The type of the binary function object used for the reduction operation.

[727] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest** – Refers to the beginning of the destination range.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

Returns

The *inclusive_scan* algorithm returns *util::in_out_result<InIter, OutIter>*. The *inclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent**, typename **FwdIter2**, typename **Op** = *std*::plus<typename *std*::iterator_traits<*FwdIter1*>::value_type>>

parallel::*util*::detail::algorithm_result<*ExPolicy*, inclusive_scan_result<*FwdIter1*, *FwdIter2*>>::type `hpx::ranges::inclusive_scan`(*ExP* && icy, *Fwd* first *Sent* last, *Fwd* dest *Op* &&

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, *first, …, *(first + (i - result))).

The reduce operations in the parallel *inclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *inclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum.

Note

Complexity: O(*last* - *first*) applications of the predicate *op*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, …, aN) is defined as:

- a1 when N is 1
- GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK)
- GENERALIZED_NONCOMMUTATIVE_SUM(+, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Op** – The type of the binary function object used for the reduction operation.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest** – Refers to the beginning of the destination range.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

Returns

The *inclusive_scan* algorithm returns a hpx::future<*util::in_out_result<FwdIter1,*

FwdIter2>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *util::in_out_result<FwdIter1, FwdIter2>* otherwise. The *inclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng**, typename **O**, typename **Op** = *std*::plus<typename *hpx*::*traits*::range_traits<*Rng*>::value_type>>

inclusive_scan_result<*std*::ranges::iterator_t<*Rng*>, *O*> `hpx::ranges::inclusive_scan`(*Rng* &&rng, *O* dest, *Op* &&op)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, *first, ..., *(first + (i - result))).

The reduce operations in the parallel *inclusive_scan* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, ..., aN) is defined as:
>
> - a1 when N is 1
> - GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, ..., aK)
>   - GENERALIZED_NONCOMMUTATIVE_SUM(+, aM, ..., aN) where 1 < K+1 = M <= N.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **O** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Op** – The type of the binary function object used for the reduction operation.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

Returns

The *inclusive_scan* algorithm returns *util::in_out_result<std::ranges::iterator_t<Rng>*, O> The *inclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng**, typename **O**, typename **Op** = *std*::plus<typename *hpx*::*traits*::range_traits<*Rng*>::value_type>>

parallel::*util*::detail::algorithm_result<*ExPolicy*, inclusive_scan_result<*std*::ranges::iterator_t<*Rng*>, *O*>> `hpx::ranges::inclusive_scan`

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, *first, …, *(first + (i - result))).

The reduce operations in the parallel *inclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *inclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(+, a1, …, aN) is defined as:
>
> - a1 when N is 1

- GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK)
- GENERALIZED_NONCOMMUTATIVE_SUM(+, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **O** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Op** – The type of the binary function object used for the reduction operation.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

Returns

The *inclusive_scan* algorithm returns a hpx::future<*util::in_out_result* <std::ranges::iterator_t<Rng>, O>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *util::in_out_result* <std::ranges::iterator_t<Rng>, O> otherwise. The *inclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied

template<typename **InIter**, typename **Sent**, typename **OutIter**, typename **Op**, typename **T** = typename *std*::iterator_traits<*InIter*>::value_type>

inclusive_scan_result<*InIter*, *OutIter*> `hpx::ranges::inclusive_scan`(*InIter* first, *Sent* last, *OutIter* dest, *Op* &&op, *T* init)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, init, *first, …, *(first + (i - result))).

The reduce operations in the parallel *inclusive_scan* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum. If *op* is not mathematically associative, the behavior of *inclusive_scan* may be non-deterministic.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aN) is defined as:
>
> - a1 when N is 1
> - op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, ..., aN)) where 1 < K+1 = M <= N.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Op** – The type of the binary function object used for the reduction operation.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest** – Refers to the beginning of the destination range.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.
- **init** – The initial value for the generalized sum.

Returns

The *inclusive_scan* algorithm returns *util::in_out_result<InIter, OutIter>*. The *inclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent**, typename **FwdIter2**, typename **Op**, typename **T** = typename *std*::iterator_traits<*FwdIter1*>::value_type>

parallel::*util*::detail::algorithm_result<*ExPolicy*, inclusive_scan_result<*FwdIter1*, *FwdIter2*>>::type `hpx::ranges::inclusive_scan`(*ExP* && icy, InIt first *Sent* last, Out Iter dest *Op* && *T* init)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, init, *first, …, *(first + (i - result))).

The reduce operations in the parallel *inclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *inclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum. If *op* is not mathematically associative, the behavior of *inclusive_scan* may be non-deterministic.

Note

Complexity: O(*last* - *first*) applications of the predicate *op*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:

- a1 when N is 1
- op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN)) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Op** – The type of the binary function object used for the reduction operation.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest** – Refers to the beginning of the destination range.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

- **init** – The initial value for the generalized sum.

Returns

The *inclusive_scan* algorithm returns a hpx::future<*util::in_out_result*<*InIter, OutIter*>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *util::in_out_result*<*InIter, OutIter*> otherwise. The *inclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng**, typename **O**, typename **Op**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type>
inclusive_scan_result<*std*::ranges::iterator_t<*Rng*>, *O*> `hpx::ranges::inclusive_scan`(*Rng* &&rng, *O* dest, *Op* &&op, *T* init)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, init, *first, …, *(first + (i - result))).

The reduce operations in the parallel *inclusive_scan* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum. If *op* is not mathematically associative, the behavior of *inclusive_scan* may be non-deterministic.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *op*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:
>
> - a1 when N is 1
> - op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN)) where 1 < K+1 = M <= N.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **O** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Op** – The type of the binary function object used for the reduction operation.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.
- **init** – The initial value for the generalized sum.

Returns

The *inclusive_scan* algorithm returns *util::in_out_result<std::ranges::iterator_t<Rng>*, O> The *inclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng**, typename **O**, typename **Op**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type>

parallel::*util*::detail::algorithm_result<*ExPolicy*, inclusive_scan_result<*std*::ranges::iterator_t<*Rng*>, *O*>> `hpx::ranges::inclusive_scan`

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, init, *first, ..., *(first + (i - result))).

The reduce operations in the parallel *inclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *inclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *exclusive_scan* and *inclusive_scan* is that *inclusive_scan* includes the ith input element in the ith sum. If *op* is not mathematically associative, the behavior of *inclusive_scan* may be non-deterministic.

Note

Complexity: O(*last* - *first*) applications of the predicate *op*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aN) is defined as:

- a1 when N is 1
- op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, ..., aN)) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.

- **O** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Op** – The type of the binary function object used for the reduction operation.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

- **init** – The initial value for the generalized sum.

Returns

The *inclusive_scan* algorithm returns a hpx::future<*util::in_out_result* <std::ranges::iterator_t<Rng>, O>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *util::in_out_result* <std::ranges::iterator_t<Rng>, O> otherwise. The *inclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied

hpx::ranges::uninitialized_value_construct

Defined in header hpx/algorithm.hpp[728].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::uninitialized_value_construct_n*

template<typename **FwdIter**, typename **Sent**>

FwdIter `hpx::ranges::`**`uninitialized_value_construct`**(*FwdIter* first, *Sent* last)

Constructs objects of type typename iterator_traits<ForwardIt>::value_type in the uninitialized storage designated by the range by value-initialization. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_value_construct* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

[728] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- `FwdIter` – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- `Sent` – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.

Parameters

- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `last` – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.

Returns

The *uninitialized_value_construct* algorithm returns a returns *FwdIter*. The *uninitialized_value_construct* algorithm returns the output iterator to the element in the range, one past the last element constructed.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::ranges::uninitialized_value_construct`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last)

Constructs objects of type typename iterator_traits<ForwardIt>::value_type in the uninitialized storage designated by the range by value-initialization. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_value_construct* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_value_construct* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.

Returns

The *uninitialized_value_construct* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *uninitialized_value_construct* algorithm returns the iterator to the element in the source range, one past the last element constructed.

template<typename **Rng**>

hpx::*traits*::range_traits<*Rng*>::iterator_type `hpx::ranges::uninitialized_value_construct`(*Rng* &&rng)

Constructs objects of type typename iterator_traits<ForwardIt>::value_type in the uninitialized storage designated by the range by value-initialization. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_value_construct* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last - first* assignments.

Template Parameters

Rng – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

Parameters

rng – Refers to the range to which will be value constructed.

Returns

The *uninitialized_value_construct* algorithm returns a returns *hpx::traits::range_traits<Rng>::iterator_type*. The *uninitialized_value_construct* algorithm returns the output iterator to the element in the range, one past the last element constructed.

template<typename **ExPolicy**, typename **Rng**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, typename *hpx*::*traits*::range_traits<*Rng*>::iterator_type>::type `hpx::ranges::uninitiali`

Constructs objects of type typename iterator_traits<ForwardIt>::value_type in the uninitialized storage designated by the range by value-initialization. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_value_construct* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_value_construct* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the range to which the value will be value consutrcted

Returns

The *uninitialized_value_construct* algorithm returns a hpx::future<*typename* hpx::traits::range_traits<Rng>::iterator_type>, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *typename* hpx::traits::range_traits<Rng>::iterator_type otherwise. The *uninitialized_value_construct* algorithm returns the output iterator to the element in the range, one past the last element constructed.

hpx::ranges::uninitialized_value_construct_n

Defined in header hpx/algorithm.hpp[729].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::uninitialized_value_construct*

template<typename **FwdIter**, typename **Size**>

FwdIter `hpx::ranges::uninitialized_value_construct_n`(*FwdIter* first, *Size* count)

Constructs objects of type typename iterator_traits<ForwardIt>::value_type in the uninitialized storage designated by the range [first, first + count) by value-initialization. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_value_construct_n* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.

Returns

The *uninitialized_value_construct_n* algorithm returns a returns *FwdIter*. The *uninitialized_value_construct_n* algorithm returns the iterator to the element in the source range, one past the last element constructed.

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**>

[729] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::ranges::uninitialized_value_construct_n`(*ExPolicy* &&policy, *FwdIter* first, *Size* count)

Constructs objects of type typename iterator_traits<ForwardIt>::value_type in the uninitialized storage designated by the range [first, first + count) by value-initialization. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_value_construct_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_value_construct_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.

Returns

The *uninitialized_value_construct_n* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *uninitialized_value_construct_n* algorithm returns the iterator to the element in the source range, one past the last element constructed.

hpx::ranges::equal

Defined in header hpx/algorithm.hpp[730].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Pred** = equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::equal`(*ExPolicy* &&policy, *Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Returns true if the range [first1, last1) is equal to the range [first2, last2), and false otherwise.

The comparison operations in the parallel *equal* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *equal* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most min(last1 - first1, last2 - first2) applications of the predicate *f*.

Note

The two ranges are considered equal if, for every iterator i in the range [first1,last1), *i equals *(first2 + (i - first1)). This overload of equal uses operator== to determine if two elements are equal.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Iter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent1** – The type of the source iterators used for the end of the first range (deduced).
- **Iter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent2** – The type of the source iterators used for the end of the second range (deduced).

[730] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj1** – The type of an optional projection function applied to the first range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second range. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual predicate *is* invoked.

Returns

The *equal* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *equal* algorithm returns true if the elements in the two ranges are equal, otherwise it returns false. If the length of the range [first1, last1) does not equal the length of the range [first2, last2), it returns false.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**, typename **Pred** = equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::equal`(*ExPolicy* &&policy, *Rng1* &&rng1, *Rng2* &&rng2, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Returns true if the range [first1, last1) is equal to the range starting at first2, and false otherwise.

The comparison operations in the parallel *equal* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *equal* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most *last1 - first1* applications of the predicate *f*.

> **Note**
>
> The two ranges are considered equal if, for every iterator i in the range [first1,last1), *i equals *(first2 + (i - first1)). This overload of equal uses operator== to determine if two elements are equal.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`Rng1`** – The type of the first source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **`Rng2`** – The type of the second source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **`Pred`** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **`Proj1`** – The type of an optional projection function applied to the first range. This defaults to *hpx::identity*
- **`Proj2`** – The type of an optional projection function applied to the second range. This defaults to *hpx::identity*

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`rng1`** – Refers to the first sequence of elements the algorithm will be applied to.
- **`rng2`** – Refers to the second sequence of elements the algorithm will be applied to.
- **`op`** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual predicate *is* invoked.

Returns

The *equal* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *equal* algorithm returns true if the elements in the two ranges are equal, otherwise it returns false.

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Pred** = equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

bool *hpx::ranges::***equal**(*Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Returns true if the range [first1, last1) is equal to the range [first2, last2), and false otherwise.

Note

Complexity: At most min(last1 - first1, last2 - first2) applications of the predicate *f*.

Note

The two ranges are considered equal if, for every iterator i in the range [first1,last1), *i equals *(first2 + (i - first1)). This overload of equal uses operator== to determine if two elements are equal.

Template Parameters

- **Iter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent1** – The type of the source iterators used for the end of the first range (deduced).
- **Iter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent2** – The type of the source iterators used for the end of the second range (deduced).
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

- **Proj1** – The type of an optional projection function applied to the first range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second range. This defaults to *hpx::identity*

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual predicate *is* invoked.

Returns

The *equal* algorithm returns true if the elements in the two ranges are equal, otherwise it returns false. If the length of the range [first1, last1) does not equal the length of the range [first2, last2), it returns false.

template<typename **Rng1**, typename **Rng2**, typename **Pred** = equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

bool *hpx::ranges::***equal**(*Rng1* &&rng1, *Rng2* &&rng2, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Returns true if the range [first1, last1) is equal to the range starting at first2, and false otherwise.

> **Note**
>
> Complexity: At most *last1* - *first1* applications of the predicate *f*.

Note

The two ranges are considered equal if, for every iterator i in the range [first1,last1), *i equals *(first2 + (i - first1)). This overload of equal uses operator== to determine if two elements are equal.

Template Parameters

- **Rng1** – The type of the first source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Rng2** – The type of the second source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj1** – The type of an optional projection function applied to the first range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second range. This defaults to *hpx::identity*

Parameters

- **rng1** – Refers to the first sequence of elements the algorithm will be applied to.
- **rng2** – Refers to the second sequence of elements the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual predicate *is* invoked.

Returns

The *equal* algorithm returns true if the elements in the two ranges are equal, otherwise it returns false.

hpx::ranges::mismatch

Defined in header hpx/algorithm.hpp[731].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Pred** = equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, mismatch_result<*Iter1*, *Iter2*>>::type `hpx::ranges::mismatch`(*ExPolicy* &&policy, *Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Returns true if the range [first1, last1) is mismatch to the range [first2, last2), and false otherwise.

The comparison operations in the parallel *mismatch* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *mismatch* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most min(last1 - first1, last2 - first2) applications of the predicate *f*. If *FwdIter1* and *FwdIter2* meet the requirements of *RandomAccessIterator* and (last1 - first1) != (last2 - first2) then no applications of the predicate *f* are made.

[731] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

> **Note**
>
> The two ranges are considered mismatch if, for every iterator i in the range [first1,last1), *i mismatchs *(first2 + (i - first1)). This overload of mismatch uses operator== to determine if two elements are mismatch.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Iter1** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent1** – The type of the source iterators used for the end of the first range (deduced).
- **Iter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent2** – The type of the source iterators used for the end of the second range (deduced).
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *mismatch* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj1** – The type of an optional projection function applied to the first range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second range. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as mismatch. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual predicate *is* invoked.

Returns

The *mismatch* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *mismatch* algorithm returns true if the elements in the two ranges are mismatch, otherwise it returns false. If the length of the range [first1, last1) does not mismatch the length of the range [first2, last2), it returns false.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**, typename **Pred** = equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, mismatch_result<typename *hpx*::*traits*::range_traits<*Rng1*>::iterator_type, typename *hp*

Returns std::pair with iterators to the first two non-equivalent elements.

The comparison operations in the parallel *mismatch* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *mismatch* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most *last1* - *first1* applications of the predicate *f*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng1** – The type of the first source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Rng2** – The type of the second source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *mismatch* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj1** – The type of an optional projection function applied to the first range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second range. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng1** – Refers to the first sequence of elements the algorithm will be applied to.
- **rng2** – Refers to the second sequence of elements the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as mismatch. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual predicate *is* invoked.

Returns

The *mismatch* algorithm returns a hpx::future<*std::pair<FwdIter1, FwdIter2>* > if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *std::pair<FwdIter1, FwdIter2>* otherwise. The *mismatch* algorithm returns the first mismatching pair of elements from two ranges: one defined by [first1, last1) and another defined by [first2, last2).

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Pred** = equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

mismatch_result<*Iter1*, *Iter2*> hpx::ranges::**mismatch**(*Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Returns true if the range [first1, last1) is mismatch to the range [first2, last2), and false otherwise.

> **Note**
>
> Complexity: At most min(last1 - first1, last2 - first2) applications of the predicate *f*. If *FwdIter1* and *FwdIter2* meet the requirements of *RandomAccessIterator* and (last1 - first1) != (last2 - first2) then no applications of the predicate *f* are made.

> **Note**
>
> The two ranges are considered mismatch if, for every iterator i in the range [first1,last1), *i mismatchs *(first2 + (i - first1)). This overload of mismatch uses operator== to determine if two elements are mismatch.

Template Parameters

- `Iter1` – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- `Sent1` – The type of the source iterators used for the end of the first range (deduced).
- `Iter2` – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- `Sent2` – The type of the source iterators used for the end of the second range (deduced).
- `Pred` – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *mismatch* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- `Proj1` – The type of an optional projection function applied to the first range. This defaults to *hpx::identity*
- `Proj2` – The type of an optional projection function applied to the second range. This defaults to *hpx::identity*

Parameters

- `first1` – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- `last1` – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- `first2` – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- `last2` – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- `op` – The binary predicate which returns true if the elements should be treated as mismatch. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual predicate *is* invoked.

Returns

The *mismatch* algorithm returns *bool*. The *mismatch* algorithm returns true if the elements in the two ranges are mismatch, otherwise it returns false. If the length of the range [first1, last1) does not mismatch the length of the range [first2, last2), it returns false.

template<typename **Rng1**, typename **Rng2**, typename **Pred** = equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

mismatch_result<typename *hpx*::*traits*::range_traits<*Rng1*>::iterator_type, typename *hpx*::*traits*::range_traits<*Rng2*>::iterator_type> `hpx::ra`

Returns std::pair with iterators to the first two non-equivalent elements.

> **Note**
>
> Complexity: At most *last1 - first1* applications of the predicate *f*.

Template Parameters

- **Rng1** – The type of the first source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Rng2** – The type of the second source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.

- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *mismatch* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>
- **Proj1** – The type of an optional projection function applied to the first range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second range. This defaults to *hpx::identity*

Parameters

- **rng1** – Refers to the first sequence of elements the algorithm will be applied to.
- **rng2** – Refers to the second sequence of elements the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as mismatch. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual predicate *is* invoked.

Returns

The *mismatch* algorithm returns *std::pair<FwdIter1, FwdIter2>*. The *mismatch* algorithm returns the first mismatching pair of elements from two ranges: one defined by [first1, last1) and another defined by [first2, last2).

hpx::ranges::uninitialized_default_construct

Defined in header hpx/algorithm.hpp[732].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::uninitialized_default_construct_n*

template<typename **FwdIter**, typename **Sent**>

FwdIter `hpx::ranges::`**`uninitialized_default_construct`**(*FwdIter* first, *Sent* last)

Constructs objects of type typename iterator_traits<ForwardIt>::value_type in the uninitialized storage designated by the range by default-initialization. If an exception is thrown during the initialization, the function has no effects.

[732] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The assignments in the parallel *uninitialized_default_construct* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- `FwdIter` – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- `Sent` – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.

Parameters

- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `last` – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.

Returns

The *uninitialized_default_construct* algorithm returns a returns *FwdIter*. The *uninitialized_default_construct* algorithm returns the output iterator to the element in the range, one past the last element constructed.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::ranges::uninitialized_default_construct`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last)

Constructs objects of type typename iterator_traits<ForwardIt>::value_type in the uninitialized storage designated by the range by default-initialization. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_default_construct* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_default_construct* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.

Returns

The *uninitialized_default_construct* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *uninitialized_default_construct* algorithm returns the iterator to the element in the source range, one past the last element constructed.

template<typename **Rng**>

hpx::*traits*::range_traits<*Rng*>::iterator_type `hpx::ranges::uninitialized_default_construct`(*Rng* &&rng)

Constructs objects of type typename iterator_traits<ForwardIt>::value_type in the uninitialized storage designated by the range by default-initialization. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_default_construct* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

Rng – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

Parameters

rng – Refers to the range to which will be default constructed.

Returns

The *uninitialized_default_construct* algorithm returns a returns *hpx::traits::range_traits<Rng>::iterator_type*. The *uninitialized_default_construct* algorithm returns the output iterator to the element in the range, one past the last element constructed.

template<typename **ExPolicy**, typename **Rng**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, typename *hpx*::*traits*::range_traits<*Rng*>::iterator_type>::type `hpx::ranges::uninitiali`

Constructs objects of type typename iterator_traits<ForwardIt>::value_type in the uninitialized storage designated by the range by default-initialization. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_default_construct* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_default_construct* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the range to which the value will be default constructed

Returns

The *uninitialized_default_construct* algorithm returns a hpx::future<*typename* hpx::traits::range_traits<Rng>::iterator_type>, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *typename* hpx::traits::range_traits<Rng>::iterator_type otherwise. The *uninitialized_default_construct* algorithm returns the output iterator to the element in the range, one past the last element constructed.

hpx::ranges::uninitialized_default_construct_n

Defined in header hpx/algorithm.hpp[733].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::uninitialized_default_construct*

template<typename **FwdIter**, typename **Size**>

FwdIter `hpx::ranges::uninitialized_default_construct_n`(*FwdIter* first, *Size* count)

Constructs objects of type typename iterator_traits<ForwardIt>::value_type in the uninitialized storage designated by the range [first, first + count) by default-initialization. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_default_construct_n* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.

Returns

The *uninitialized_default_construct_n* algorithm returns a returns *FwdIter*. The *uninitialized_default_construct_n* algorithm returns the iterator to the element in the source range, one past the last element constructed.

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**>

[733] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

parallel::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter*>::type `hpx::ranges::uninitialized_default_construct_n`(*ExPolicy* &&policy, *FwdIter* first, *Size* count)

Constructs objects of type typename iterator_traits<ForwardIt>::value_type in the uninitialized storage designated by the range [first, first + count) by default-initialization. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_default_construct_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_default_construct_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.

Returns

The *uninitialized_default_construct_n* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *uninitialized_default_construct_n* algorithm returns the iterator to the element in the source range, one past the last element constructed.

hpx::ranges::ends_with

Defined in header hpx/algorithm.hpp[734].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Pred** = *ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

bool *hpx::ranges::***ends_with**(*Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Pred* &&pred = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Checks whether the second range defined by [first1, last1) matches the suffix of the first range defined by [first2, last2)

The assignments in the parallel *ends_with* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear: at most min(N1, N2) applications of the predicate and both projections.

Template Parameters

- **Iter1** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent1** – The type of the end source iterators used(deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Iter2** – The type of the begin destination iterators used deduced). This iterator type must meet the requirements of a input iterator.
- **Sent2** – The type of the end destination iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter2.
- **Pred** – The binary predicate that compares the projected elements.
- **Proj1** – The type of an optional projection function for the source range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function for the destination range. This defaults to *hpx::identity*

Parameters

- **first1** – Refers to the beginning of the source range.
- **last1** – Sentinel value referring to the end of the source range.
- **first2** – Refers to the beginning of the destination range.
- **last2** – Sentinel value referring to the end of the destination range.

[734] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **pred** – Specifies the binary predicate function (or function object) which will be invoked for comparison of the elements in the in two ranges projected by proj1 and proj2 respectively.
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements in the source range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements in the destination range as a projection operation before the actual predicate *is* invoked.

Returns

The *ends_with* algorithm returns *bool*. The *ends_with* algorithm returns a boolean with the value true if the second range matches the suffix of the first range, false otherwise.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent1**, typename **FwdIter2**, typename **Sent2**, typename **Pred** = *ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, bool>::type `hpx::ranges::`**`ends_with`**(*ExPolicy* &&policy, *FwdIter1* first1, *Sent1* last1, *FwdIter2* first2, *Sent2* last2, *Pred* &&pred = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Checks whether the second range defined by [first1, last1) matches the suffix of the first range defined by [first2, last2)

The assignments in the parallel *ends_with* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *ends_with* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Linear: at most min(N1, N2) applications of the predicate and both projections.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent1** – The type of the end source iterators used(deduced). This iterator type must meet the requirements of an sentinel for Iter1.

- `FwdIter2` – The type of the begin destination iterators used deduced). This iterator type must meet the requirements of a forward iterator.
- `Sent2` – The type of the end destination iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter2.
- `Pred` – The binary predicate that compares the projected elements.
- `Proj1` – The type of an optional projection function for the source range. This defaults to *hpx::identity*
- `Proj2` – The type of an optional projection function for the destination range. This defaults to *hpx::identity*

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `first1` – Refers to the beginning of the source range.
- `last1` – Sentinel value referring to the end of the source range.
- `first2` – Refers to the beginning of the destination range.
- `last2` – Sentinel value referring to the end of the destination range.
- `pred` – Specifies the binary predicate function (or function object) which will be invoked for comparison of the elements in the in two ranges projected by proj1 and proj2 respectively.
- `proj1` – Specifies the function (or function object) which will be invoked for each of the elements in the source range as a projection operation before the actual predicate *is* invoked.
- `proj2` – Specifies the function (or function object) which will be invoked for each of the elements in the destination range as a projection operation before the actual predicate *is* invoked.

Returns

The *ends_with* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *ends_with* algorithm returns a boolean with the value true if the second range matches the suffix of the first range, false otherwise.

template<typename **Rng1**, typename **Rng2**, typename **Pred** = *ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

bool `hpx::ranges::ends_with`(*Rng1* &&rng1, *Rng2* &&rng2, *Pred* &&pred = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Checks whether the second range *rng2* matches the suffix of the first range *rng1*.

The assignments in the parallel *ends_with* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear: at most min(N1, N2) applications of the predicate and both projections.

Template Parameters

- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Rng2** – The type of the destination range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The binary predicate that compares the projected elements.
- **Proj1** – The type of an optional projection function for the source range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function for the destination range. This defaults to *hpx::identity*

Parameters

- **rng1** – Refers to the source range.
- **rng2** – Refers to the destination range.
- **pred** – Specifies the binary predicate function (or function object) which will be invoked for comparison of the elements in the in two ranges projected by proj1 and proj2 respectively.
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements in the source range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements in the destination range as a projection operation before the actual predicate *is* invoked.

Returns

The *ends_with* algorithm returns *bool*. The *ends_with* algorithm returns a boolean with the value true if the second range matches the suffix of the first range, false otherwise.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**, typename **Pred** = *ranges*::equal_to, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, bool>::type `hpx::ranges::ends_with`(*ExPolicy* &&policy, *Rng1* &&rng1, *Rng2* &&rng2, *Pred* &&pred = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Checks whether the second range *rng2* matches the suffix of the first range *rng1*.

The assignments in the parallel *ends_with* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *ends_with* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Linear: at most min(N1, N2) applications of the predicate and both projections.

Template Parameters

- `ExPolicy` – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- `Rng1` – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- `Rng2` – The type of the destination range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- `Pred` – The binary predicate that compares the projected elements.
- `Proj1` – The type of an optional projection function for the source range. This defaults to *hpx::identity*
- `Proj2` – The type of an optional projection function for the destination range. This defaults to *hpx::identity*

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `rng1` – Refers to the source range.
- `rng2` – Refers to the destination range.
- `pred` – Specifies the binary predicate function (or function object) which will be invoked for comparison of the elements in the in two ranges projected by proj1 and proj2 respectively.
- `proj1` – Specifies the function (or function object) which will be invoked for each of the elements in the source range as a projection operation before the actual predicate *is* invoked.
- `proj2` – Specifies the function (or function object) which will be invoked for each of the elements in the destination range as a projection operation before the actual predicate *is* invoked.

Returns

The *ends_with* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *ends_with* algorithm returns a boolean with the value true if the second range matches the suffix of the first range, false otherwise.

hpx::ranges::min_element

Defined in header hpx/algorithm.hpp[735].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::max_element*
- *hpx::ranges::minmax_element*

template<typename **FwdIter**, typename **Sent**, typename **F** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

FwdIter `hpx::ranges::min_element`(*FwdIter* first, *Sent* last, *F* &&f = *F*(), *Proj* &&proj = *Proj*())

Finds the smallest element in the range [first, last) using the given comparison function *f*.

The comparisons in the parallel *min_element* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Exactly *max*(N-1, 0) comparisons, where N = std::distance(first, last).

Template Parameters

- **FwdIter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for FwdIter.
- **F** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – The binary predicate which returns true if the the left argument is less than the right element. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.

[735] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *min_element* algorithm returns *FwdIter*. The *min_element* algorithm returns the iterator to the smallest element in the range [first, last). If several elements in the range are equivalent to the smallest element, returns the iterator to the first such element. Returns last if the range is empty.

template<typename **Rng**, typename **F** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

std::ranges::iterator_t<*Rng*> `hpx::ranges::min_element`(*Rng* &&rng, *F* &&f = *F*(), *Proj* &&proj = *Proj*())

Finds the smallest element in the range [first, last) using the given comparison function *f*.

The comparisons in the parallel *min_element* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Exactly *max*(N-1, 0) comparisons, where N = std::distance(first, last).

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – The binary predicate which returns true if the the left argument is less than the right element. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *min_element* algorithm returns a *hpx::traits::range_iterator<Rng>::type* otherwise. The *min_element* algorithm returns the iterator to the smallest element in the range [first, last). If several elements in the range are equivalent to the smallest element, returns the iterator to the first such element. Returns last if the range is empty.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **F** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::ranges::min_element`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *F* &&f = *F*(), *Proj* &&proj = *Proj*())

Finds the smallest element in the range [first, last) using the given comparison function *f*.

The comparisons in the parallel *min_element* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *min_element* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *max*(N-1, 0) comparisons, where N = std::distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for FwdIter.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *min_element* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – The binary predicate which returns true if the the left argument is less than the right element. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *min_element* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *min_element* algorithm returns the iterator to the smallest element in the range [first, last). If several elements in the range are equivalent to the smallest element, returns the iterator to the first such element. Returns last if the range is empty.

template<typename **ExPolicy**, typename **Rng**, typename **F** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::min_element`(*ExPolicy* &&policy, *Rng* &&rng, *F* &&f = *F*(), *Proj* &&proj = *Proj*())

Finds the smallest element in the range [first, last) using the given comparison function *f*.

The comparisons in the parallel *min_element* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *min_element* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *max*(N-1, 0) comparisons, where N = std::distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *min_element* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – The binary predicate which returns true if the the left argument is less than the right element. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *min_element* algorithm returns a hpx::future<*hpx::traits::range_iterator*<*Rng*>*::type*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *min_element* algorithm returns the iterator to the smallest element in the range [first, last). If several elements in the range are equivalent to the smallest element, returns the iterator to the first such element. Returns last if the range is empty.

hpx::ranges::max_element

Defined in header hpx/algorithm.hpp[736].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::min_element*
- *hpx::ranges::minmax_element*

template<typename **FwdIter**, typename **Sent**, typename **F** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

FwdIter `hpx::ranges::max_element`(*FwdIter* first, *Sent* last, *F* &&f = *F*(), *Proj* &&proj = *Proj*())

Finds the greatest element in the range [first, last) using the given comparison function *f*.

The comparisons in the parallel *max_element* algorithm execute in sequential order in the calling thread.

[736] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

> **Note**
>
> Complexity: Exactly *max*(N-1, 0) comparisons, where N = std::distance(first, last).

Template Parameters

- **FwdIter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for FwdIter.
- **F** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – The binary predicate which returns true if the This argument is optional and defaults to std::less. the left argument is less than the right element. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *max_element* algorithm returns a *FwdIter*. The *max_element* algorithm returns the iterator to the smallest element in the range [first, last). If several elements in the range are equivalent to the smallest element, returns the iterator to the first such element. Returns last if the range is empty.

template<typename **Rng**, typename **F** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

std::ranges::iterator_t<*Rng*> `hpx::ranges::`**`max_element`**(*Rng* &&rng, *F* &&f = *F*(), *Proj* &&proj = *Proj*())

Finds the greatest element in the range [first, last) using the given comparison function *f*.

The comparisons in the parallel *max_element* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Exactly *max*(N-1, 0) comparisons, where N = std::distance(first, last).

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – The binary predicate which returns true if the This argument is optional and defaults to std::less. the left argument is less than the right element. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *max_element* algorithm returns a *hpx::traits::range_iterator<Rng>::type* otherwise. The *max_element* algorithm returns the iterator to the smallest element in the range [first, last). If several elements in the range are equivalent to the smallest element, returns the iterator to the first such element. Returns last if the range is empty.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **F** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::ranges::`**`max_element`**(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *F* &&f = *F*(), *Proj* &&proj = *Proj*())

Finds the greatest element in the range [first, last) using the given comparison function *f*.

The comparisons in the parallel *max_element* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *max_element* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an

unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *max*(N-1, 0) comparisons, where N = std::distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for FwdIter.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *max_element* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – The binary predicate which returns true if the This argument is optional and defaults to std::less. the left argument is less than the right element. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *max_element* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *max_element* algorithm returns the iterator to the smallest element in the range [first, last). If several elements in the range are equivalent to the smallest element, returns the iterator to the first such element. Returns last if the range is empty.

template<typename **ExPolicy**, typename **Rng**, typename **F** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::max_element`(*ExPolicy* &&policy, *Rng* &&rng, *F* &&f = *F*(), *Proj* &&proj = *Proj*())

Finds the greatest element in the range [first, last) using the given comparison function *f*.

The comparisons in the parallel *max_element* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *max_element* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *max*(N-1, 0) comparisons, where N = std::distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *max_element* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – The binary predicate which returns true if the This argument is optional and defaults to std::less. the left argument is less than the right element. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *max_element* algorithm returns a hpx::future<*hpx::traits::range_iterator*<*Rng*>*::type*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *max_element* algorithm returns the iterator to the smallest element in the range [first, last). If several elements in the range are equivalent to the smallest element, returns the iterator to the first such element. Returns last if the range is empty.

hpx::ranges::minmax_element

Defined in header hpx/algorithm.hpp[737].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::min_element*
- *hpx::ranges::max_element*

template<typename **FwdIter**, typename **Sent**, typename **F** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

minmax_element_result<*FwdIter*> *hpx*::*ranges*::**minmax_element**(*FwdIter* first, *Sent* last, *F* &&f = *F*(), *Proj* &&proj = *Proj*())

Finds the greatest element in the range [first, last) using the given comparison function *f*.

The assignments in the parallel *minmax_element* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: At most *max*(floor(3/2*(N-1)), 0) applications of the predicate, where N = std::distance(first, last).

Template Parameters

- **FwdIter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for FwdIter.

[737] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **F** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – The binary predicate which returns true if the the left argument is less than the right element. This argument is optional and defaults to std::less. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *minmax_element* algorithm returns a *minmax_element_result<FwdIter, FwdIter>* The *minmax_element* algorithm returns a min_max_result consisting of an iterator to the smallest element as the min element and an iterator to the greatest element as the max element. Returns minmax_element_result{first, first} if the range is empty. If several elements are equivalent to the smallest element, the iterator to the first such element is returned. If several elements are equivalent to the largest element, the iterator to the last such element is returned.

template<typename **Rng**, typename **F** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

minmax_element_result<*std*::ranges::iterator_t<*Rng*>> `hpx::ranges::`**`minmax_element`**(*Rng* &&rng, *F* &&f = *F*(), *Proj* &&proj = *Proj*())

Finds the greatest element in the range [first, last) using the given comparison function *f*.

The assignments in the parallel *minmax_element* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: At most *max*(floor(3/2*(N-1)), 0) applications of the predicate, where N = std::distance(first, last).

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.

- **F** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – The binary predicate which returns true if the the left argument is less than the right element. This argument is optional and defaults to std::less. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *minmax_element* algorithm returns a *minmax_element_result<hpx::traits::range_iterator<Rng>::type* , hpx::traits::range_iterator<Rng>::type> The *minmax_element* algorithm returns a min_max_result consisting of an range iterator to the smallest element as the min element and an range iterator to the greatest element as the max element. If several elements are equivalent to the smallest element, the iterator to the first such element is returned. If several elements are equivalent to the largest element, the iterator to the last such element is returned.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **F** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, minmax_element_result<*FwdIter*>> `hpx::ranges::minmax_element`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *F* &&f = *F*(), *Proj* &&proj = *Proj*())

Finds the greatest element in the range [first, last) using the given comparison function *f*.

The comparisons in the parallel *minmax_element* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *minmax_element* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in

an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most *max*(floor(3/2*(N-1)), 0) applications of the predicate, where N = std::distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for FwdIter.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *minmax_element* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – The binary predicate which returns true if the the left argument is less than the right element. This argument is optional and defaults to std::less. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *minmax_element* algorithm returns a *minmax_element_result<FwdIter, FwdIter>* The *minmax_element* algorithm returns a min_max_result consisting of an iterator to the smallest element as the min element and an iterator to the greatest element as the max element. Returns minmax_element_result{first, first} if the range is empty. If several elements are equivalent to the smallest element, the iterator to the first such element is returned. If several elements are equivalent to the largest element, the iterator to the last such element is returned.

template<typename **ExPolicy**, typename **Rng**, typename **F** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, minmax_element_result<*std*::ranges::iterator_t<*Rng*>>> `hpx::ranges::minmax_el`

Finds the greatest element in the range [first, last) using the given comparison function *f*.

The comparisons in the parallel *minmax_element* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *minmax_element* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most *max*(floor(3/2*(N-1)), 0) applications of the predicate, where N = std::distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *minmax_element* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.

- **f** – The binary predicate which returns true if the the left argument is less than the right element. This argument is optional and defaults to std::less. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1*.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *minmax_element* algorithm returns a *minmax_element_result<hpx::traits::range_iterator<Rng>::type*, hpx::traits::range_iterator<Rng>::type> The *minmax_element* algorithm returns a min_max_result consisting of an range iterator to the smallest element as the min element and an range iterator to the greatest element as the max element. If several elements are equivalent to the smallest element, the iterator to the first such element is returned. If several elements are equivalent to the largest element, the iterator to the last such element is returned.

hpx::ranges::adjacent_find

Defined in header hpx/algorithm.hpp[738].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **FwdIter**, typename **Sent**, typename **Proj** = *hpx*::identity, typename **Pred** = detail::equal_to>

FwdIter `hpx::ranges::adjacent_find`(*FwdIter* first, *Sent* last, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Searches the range [first, last) for two consecutive identical elements.

> **Note**
>
> Complexity: Exactly the smaller of (result - first) + 1 and (last - first) - 1 application of the predicate where *result* is the value returned

Template Parameters

- **FwdIter** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*
- **Pred** – The type of an optional function/function object to use.

Parameters

[738] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **pred** – The binary predicate which returns *true* if the elements should be treated as equal. The signature should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1* .
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *adjacent_find* algorithm returns an iterator to the first of the identical elements. If no such elements are found, *last* is returned.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **Proj** = *hpx*::identity, typename **Pred** = detail::equal_to>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter*>::type `hpx::ranges::adjacent_find`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Searches the range [first, last) for two consecutive identical elements. This version uses the given binary predicate pred

The comparison operations in the parallel *adjacent_find* invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *adjacent_find* invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

This overload of *adjacent_find* is available if the user decides to provide their algorithm their own binary predicate *pred*.

> **Note**
>
> Complexity: Exactly the smaller of (result - first) + 1 and (last - first) - 1 application of the predicate where *result* is the value returned

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *adjacent_find* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **pred** – The binary predicate which returns *true* if the elements should be treated as equal. The signature should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1* .
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *adjacent_find* algorithm returns a *hpx::future<InIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *InIter* otherwise. The *adjacent_find* algorithm returns an iterator to the first of the identical elements. If no such elements are found, *last* is returned.

template<typename **Rng**, typename **Proj** = *hpx*::identity, typename **Pred** = detail::equal_to>

hpx::*traits*::range_traits<*Rng*>::iterator_type **hpx::ranges::adjacent_find**(*Rng* &&rng, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Searches the range rng for two consecutive identical elements.

Note

Complexity: Exactly the smaller of (result - std::begin(rng)) + 1 and (std::begin(rng) - std::end(rng)) - 1 applications of the predicate where *result* is the value returned

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*
- **Pred** – The type of an optional function/function object to use.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – The binary predicate which returns *true* if the elements should be treated as equal. The signature should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1* .
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *adjacent_find* algorithm returns an iterator to the first of the identical elements. If no such elements are found, *last* is returned.

template<typename **ExPolicy**, typename **Rng**, typename **Proj** = *hpx*::identity, typename **Pred** = detail::equal_to>

parallel::*util*::detail::algorithm_result<*ExPolicy*, typename *hpx*::*traits*::range_traits<*Rng*>::iterator_type>::type `hpx::ranges::adjacent_f`

Searches the range rng for two consecutive identical elements.

The comparison operations in the parallel *adjacent_find* invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *adjacent_find* invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

This overload of *adjacent_find* is available if the user decides to provide their algorithm their own binary predicate *pred*.

Note

Complexity: Exactly the smaller of (result - std::begin(rng)) + 1 and (std::begin(rng) - std::end(rng)) - 1 applications of the predicate where *result* is the value returned

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *adjacent_find* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::equal_to<>

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – The binary predicate which returns *true* if the elements should be treated as equal. The signature should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1* .
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *adjacent_find* algorithm returns a *hpx::future<InIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *InIter* otherwise. The *adjacent_find* algorithm returns an iterator to the first of the identical elements. If no such elements are found, *last* is returned.

hpx::ranges::shift_right

Defined in header hpx/algorithm.hpp[739].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **FwdIter**, typename **Sent**, typename **Size**>

[739] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

FwdIter `hpx::ranges::shift_right`(*FwdIter* first, *Sent* last, *Size* n)

Shifts the elements in the range [first, last) by n positions towards the end of the range. For every integer i in [0, last - first - n), moves the element originally at position first + i to position first

- n + i.

The assignment operations in the parallel *shift_right* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: At most (last - first) - n assignments.

> **Note**
>
> The type of dereferenced *FwdIter* must meet the requirements of *MoveAssignable*.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **Size** – The type of the argument specifying the number of positions to shift by.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **n** – Refers to the number of positions to shift.

Returns

The *shift_right* algorithm returns *FwdIter*. The *shift_right* algorithm returns an iterator to the end of the resulting range.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **Size**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::ranges::shift_right`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *Size* n)

Shifts the elements in the range [first, last) by n positions towards the end of the range. For every integer i in [0, last - first - n), moves the element originally at position first + i to position first

- n + i.

The assignment operations in the parallel *shift_right* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignment operations in the parallel *shift_right* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most (last - first) - n assignments.

> **Note**
>
> The type of dereferenced *FwdIter* must meet the requirements of *MoveAssignable*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **Size** – The type of the argument specifying the number of positions to shift by.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **n** – Refers to the number of positions to shift.

Returns

The *shift_right* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *shift_right* algorithm returns an iterator to the end of the resulting range.

template<typename **Rng**, typename **Size**>

std::ranges::iterator_t<*Rng*> `hpx::ranges::shift_right`(*Rng* &&rng, *Size* n)

Shifts the elements in the range [first, last) by n positions towards the end of the range. For every integer i in [0, last - first - n), moves the element originally at position first + i to position first

- n + i.

The assignment operations in the parallel *shift_right* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: At most (last - first) - n assignments.

> **Note**
>
> The type of dereferenced *std::ranges::iterator_t<Rng>* must meet the requirements of *MoveAssignable*.

Template Parameters

- **Rng** – The type of the range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of positions to shift by.

Parameters

- **rng** – Refers to the range in which the elements will be shifted.
- **n** – Refers to the number of positions to shift.

Returns

The *shift_right* algorithm returns *std::ranges::iterator_t<Rng>*. The *shift_right* algorithm returns an iterator to the end of the resulting range.

template<typename **ExPolicy**, typename **Rng**, typename **Size**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::shift_right`(*ExPolicy* &&policy, *Rng* &&rng, *Size* n)

Shifts the elements in the range [first, last) by n positions towards the end of the range. For every integer i in [0, last - first - n), moves the element originally at position first + i to position first

- n + i.

The assignment operations in the parallel *shift_right* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignment operations in the parallel *shift_right* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most (last - first) - n assignments.

> **Note**
>
> The type of dereferenced *std::ranges::iterator_t<Rng>* must meet the requirements of *MoveAssignable*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of positions to shift by.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the range in which the elements will be shifted.
- **n** – Refers to the number of positions to shift.

Returns

The *shift_right* algorithm returns a hpx::future<*std::ranges::iterator_t<Rng>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *std::ranges::iterator_t<Rng>* otherwise. The *shift_right* algorithm returns an iterator to the end of the resulting range.

hpx::ranges::generate

Defined in header hpx/algorithm.hpp[740].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::generate_n*

template<typename **ExPolicy**, typename **Rng**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::generate`(*ExPolicy* &&policy, *Rng* &&rng, *F* &&f)

Assign each element in range [first, last) a value generated by the given function object f

The assignments in the parallel *generate* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *generate* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *distance(first, last)* invocations of *f* and assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.

[740] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **f** – generator function that will be called. signature of function should be equivalent to the following:

```
Ret fun();
```

  The type *Ret* must be such that an object of type *FwdIter* can be dereferenced and assigned a value of type *Ret*.

Returns
The *replace_if* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. It returns *last*.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *Iter*> `hpx::ranges::generate`(*ExPolicy* &&policy, *Iter* first, *Sent* last, *F* &&f)

Assign each element in range [first, last) a value generated by the given function object f

The assignments in the parallel *generate* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *generate* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *distance(first, last)* invocations of *f* and assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Iter** – The type of the source begin iterator used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source end iterator used (deduced). This iterator type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.

- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – generator function that will be called. signature of function should be equivalent to the following:

```
Ret fun();
```

  The type *Ret* must be such that an object of type *FwdIter* can be dereferenced and assigned a value of type *Ret*.

Returns
The *replace_if* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. It returns *last*.

template<typename **Rng**, typename **F**>

std::ranges::iterator_t<*Rng*> `hpx::ranges::generate`(*Rng* &&rng, *F* &&f)

Assign each element in range [first, last) a value generated by the given function object f

> **Note**
>
> Complexity: Exactly *distance(first, last)* invocations of *f* and assignments.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – generator function that will be called. signature of function should be equivalent to the following:

```
Ret fun();
```

  The type *Ret* must be such that an object of type *FwdIter* can be dereferenced and assigned a value of type *Ret*.

Returns
The *replace_if* algorithm returns *last*.

template<typename **Iter**, typename **Sent**, typename **F**>

Iter `hpx::ranges::generate`(*Iter* first, *Sent* last, *F* &&f)

Assign each element in range [first, last) a value generated by the given function object f

> **Note**
>
> Complexity: Exactly *distance(first, last)* invocations of *f* and assignments.

Template Parameters

- **Iter** – The type of the source begin iterator used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source end iterator used (deduced). This iterator type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – generator function that will be called. signature of function should be equivalent to the following:

```
Ret fun();
```

  The type *Ret* must be such that an object of type *FwdIter* can be dereferenced and assigned a value of type *Ret*.

Returns

The *replace_if* algorithm returns *last*.

hpx::ranges::generate_n

Defined in header hpx/algorithm.hpp[741].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::generate*

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**, typename **F**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> **hpx::ranges::generate_n**(*ExPolicy* &&policy, *FwdIter* first, *Size* count, *F* &&f)

Assigns each element in range [first, first+count) a value generated by the given function object g.

[741] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The assignments in the parallel *generate_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *generate_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *count* invocations of *f* and assignments, for count > 0.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements in the sequence the algorithm will be applied to.
- **f** – Refers to the generator function object that will be called. The signature of the function should be equivalent to

```
Ret fun();
```

The type *Ret* must be such that an object of type *OutputIt* can be dereferenced and assigned a value of type *Ret*.

Returns

The *replace_if* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. It returns *last*.

template<typename **FwdIter**, typename **Size**, typename **F**>

FwdIter `hpx::ranges::`**generate_n**(*FwdIter* first, *Size* count, *F* &&f)

Assigns each element in range [first, first+count) a value generated by the given function object g.

> **Note**
>
> Complexity: Exactly *count* invocations of *f* and assignments, for count > 0.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements in the sequence the algorithm will be applied to.
- **f** – Refers to the generator function object that will be called. The signature of the function should be equivalent to

  ```
  Ret fun();
  ```

  The type *Ret* must be such that an object of type *OutputIt* can be dereferenced and assigned a value of type *Ret*.

Returns

The *replace_if* algorithm returns *last*.

hpx/parallel/container_algorithms/iota.hpp

Defined in header hpx/parallel/container_algorithms/iota.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **ranges**

Functions

template<typename **FwdIter**, typename **Sent**, typename **T**>

iota_result<*FwdIter*, *T*> **iota**(*FwdIter* first, *Sent* last, *T* value)

Fills the range [first, last) with sequentially increasing values, starting with *value* and repetitively evaluating ++`value`.

The assignments in the *iota* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Exactly *std::distance(first, last)* assignments and increments.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input or output iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **T** – The type of the target value. This type must meet the requirements of weakly incrementable.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **value** – The value to be stored in the first element.

Returns

The *iota* algorithm returns a *hpx::ranges::iota_result* containing the final iterator and the final value.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, iota_result<*FwdIter*, *T*>> **iota**(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *T* value)

Fills the range [first, last) with sequentially increasing values, starting with *value* and repetitively evaluating ++`value`.

The assignments in the parallel *iota* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *iota* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *std::distance(first, last)* assignments and increments.

Template Parameters

- `ExPolicy` – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- `FwdIter` – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- `Sent` – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- `T` – The type of the target value. This type must meet the requirements of weakly incrementable.

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `last` – Refers to the end of the sequence of elements the algorithm will be applied to.
- `value` – The value to be stored in the first element.

Returns

The *iota* algorithm returns a *hpx::future<iota_result>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *iota_result* otherwise.

template<typename **Range**, typename **T**>

iota_result<typename *hpx*::*traits*::range_traits<*Range*>::iterator_type, *T*> **iota**(*Range* &&r, *T* value)

Fills the range *r* with sequentially increasing values, starting with *value* and repetitively evaluating `++value`.

The assignments in the *iota* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Exactly *std::ranges::distance(r)* assignments and increments.

Template Parameters

- `Range` – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input or output iterator
- `T` – The type of the target value. This type must meet the requirements of weakly incrementable.

Parameters

- `r` – Refers to the sequence of elements the algorithm will be applied to.
- `value` – The value to be stored in the first element.

Returns

The *iota* algorithm returns a *hpx::ranges::iota_result* containing the final iterator and the final value.

template<typename **ExPolicy**, typename **Range**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, iota_result<typename *hpx*::*traits*::range_traits<*Range*>::iterator_type, *T*>> **iota**(*Ex*
&
icy
Ra
&
T
va

Fills the range *r* with sequentially increasing values, starting with *value* and repetitively evaluating `++value`.

The assignments in the parallel *iota* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *iota* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly *std::ranges::distance(r)* assignments and increments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Range** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a forward iterator.
- **T** – The type of the target value. This type must meet the requirements of weakly incrementable.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **r** – Refers to the sequence of elements the algorithm will be applied to.
- **value** – The value to be stored in the first element.

Returns

The *iota* algorithm returns a *hpx::future<iota_result>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *iota_result* otherwise.

hpx::ranges::transform_inclusive_scan

Defined in header hpx/algorithm.hpp[742].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter**, typename **Sent**, typename **OutIter**, typename **BinOp**, typename **UnOp**>

transform_inclusive_scan_result<*InIter*, *OutIter*> `hpx::ranges::transform_inclusive_scan`(*InIter* first, *Sent* last, *OutIter* dest, *BinOp* &&binary_op, *UnOp* &&unary_op)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, conv(*first), ..., conv(*(first + (i - result)))).

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Neither *conv* nor *op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The behavior of transform_inclusive_scan may be non-deterministic for a non-associative predicate.

Note

Complexity: O(*last* - *first*) applications of the predicates *op* and *conv*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aN) is defined as:

- a1 when N is 1
- op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, ..., aN) where 1 < K+1 = M <= N.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.

[742] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **BinOp** – The type of the binary function object used for the reduction operation.
- **UnOp** – The type of the unary function object used for the conversion operation.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest** – Refers to the beginning of the destination range.
- **binary_op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.
- **unary_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

  ```
  R fun(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_inclusive_scan* algorithm returns *transform_inclusive_scan_result<InIter, OutIter>*. The *transform_inclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent**, typename **FwdIter2**, typename **BinOp**, typename **UnOp**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, transform_inclusive_result<*FwdIter1*, *FwdIter2*>>::type *hpx*::*ranges*::**transform_incl**

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, conv(*first), …, conv(*(first + (i - result)))).

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Neither *conv* nor *op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The behavior of transform_inclusive_scan may be non-deterministic for a non-associative predicate.

Note

Complexity: O(*last* - *first*) applications of the predicates *op* and *conv*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:

- a1 when N is 1
- op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **BinOp** – The type of the binary function object used for the reduction operation.
- **UnOp** – The type of the unary function object used for the conversion operation.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest** – Refers to the beginning of the destination range.
- **binary_op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.
- **unary_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

  ```
  R fun(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_inclusive_scan* algorithm returns a hpx::future<*transform_inclusive_result*<*FwdIter1*, FwdIter2>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *transform_inclusive_result*<*FwdIter1*, FwdIter2> otherwise. The *transform_inclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng**, typename **O**, typename **BinOp**, typename **UnOp**>

transform_inclusive_scan_result<*std*::ranges::iterator_t<*Rng*>, *O*> *hpx*::*ranges*::**transform_inclusive_scan**(*Rng* &&rng, *O* dest, *BinOp* &&binary_op, *UnOp* &&unary_op)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, conv(*first), …, conv(*(first + (i - result)))).

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Neither *conv* nor *op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The behavior of transform_inclusive_scan may be non-deterministic for a non-associative predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicates *op* and *conv*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, . . . , aN) is defined as:
>
> - a1 when N is 1
> - op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, . . . , aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, . . . , aN) where 1 < K+1 = M <= N.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **O** – The type of the iterator representing the destination range (deduced).
- **BinOp** – The type of the binary function object used for the reduction operation.
- **UnOp** – The type of the unary function object used for the conversion operation.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **binary_op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

- **unary_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

```
R fun(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_inclusive_scan* algorithm returns a returns *transform_inclusive_scan_result*< std::ranges::iterator_t<Rng>, O>. The *transform_inclusive_scan* algorithm returns an input iterator to one past the end of the range and an output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng**, typename **O**, typename **BinOp**, typename **UnOp**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, transform_inclusive_scan_result<*std*::ranges::iterator_t<*Rng*>, *O*>>::type `hpx::ranges::t`

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(op, conv(*first), ..., conv(*(first + (i - result)))).

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Neither *conv* nor *op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The behavior of transform_inclusive_scan may be non-deterministic for a non-associative predicate.

Note

Complexity: O(*last* - *first*) applications of the predicates *op* and *conv*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aN) is defined as:

- a1 when N is 1
- op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **O** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator. This iterator type must meet the requirements of an forward iterator.
- **BinOp** – The type of the binary function object used for the reduction operation.
- **UnOp** – The type of the unary function object used for the conversion operation.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **binary_op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

- **unary_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

```
R fun(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_inclusive_scan* algorithm returns a hpx::future<*transform_inclusive_scan_result*< std::ranges::iterator_t<Rng>, O>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *transform_inclusive_scan_result*< std::ranges::iterator_t<Rng>, O> otherwise. The *transform_inclusive_scan* algorithm returns an input iterator to one past the end of the range and an output iterator to the element in the destination range, one past the last element copied.

template<typename **InIter**, typename **Sent**, typename **OutIter**, typename **BinOp**, typename **UnOp**, typename **T** = typename *std*::iterator_traits<*InIter*>::value_type>

transform_inclusive_scan_result<*InIter*, *OutIter*> *hpx*::*ranges*::**transform_inclusive_scan**(*InIter* first, *Sent* last, *OutIter* dest, *BinOp* &&binary_op, *UnOp* &&unary_op, *T* init)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, conv(*first), …, conv(*(first + (i - result)))).

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Neither *conv* nor *op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The behavior of transform_inclusive_scan may be non-deterministic for a non-associative predicate.

Note

Complexity: O(*last* - *first*) applications of the predicates *op* and *conv*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:

- a1 when N is 1
- op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **BinOp** – The type of the binary function object used for the reduction operation.

- **UnOp** – The type of the unary function object used for the conversion operation.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest** – Refers to the beginning of the destination range.
- **binary_op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.
- **unary_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

  ```
  R fun(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.
- **init** – The initial value for the generalized sum.

Returns

The *transform_inclusive_scan* algorithm returns *transform_inclusive_scan_result<InIter, OutIter>*. The *transform_inclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent**, typename **FwdIter2**, typename **BinOp**, typename **UnOp**, typename **T** = typename *std*::iterator_traits<*FwdIter1*>::value_type>

parallel::*util*::detail::algorithm_result<*ExPolicy*, transform_inclusive_result<*FwdIter1*, *FwdIter2*>>::type `hpx::ranges::transform_incl`

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, conv(*first), …, conv(*(first + (i - result)))).

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Neither *conv* nor *op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The behavior of transform_inclusive_scan may be non-deterministic for a non-associative predicate.

Note

Complexity: O(*last* - *first*) applications of the predicates *op* and *conv*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:

- a1 when N is 1
- op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **BinOp** – The type of the binary function object used for the reduction operation.
- **UnOp** – The type of the unary function object used for the conversion operation.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest** – Refers to the beginning of the destination range.
- **binary_op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.
- **unary_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

  ```
  R fun(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.
- **init** – The initial value for the generalized sum.

Returns

The *transform_inclusive_scan* algorithm returns a hpx::future<*transform_inclusive_result*<*FwdIter1*, FwdIter2>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *transform_inclusive_result*<*FwdIter1*, FwdIter2> otherwise. The *transform_inclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng**, typename **O**, typename **BinOp**, typename **UnOp**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type>

transform_inclusive_scan_result<*std*::ranges::iterator_t<*Rng*>, *O*> *hpx*::*ranges*::**transform_inclusive_scan**(*Rng* &&rng, *O* dest, *BinOp* &&binary_op, *UnOp* &&unary_op, *T* init)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, conv(*first), …, conv(*(first + (i - result)))).

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Neither *conv* nor *op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The behavior of transform_inclusive_scan may be non-deterministic for a non-associative predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicates *op* and *conv*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:
>
> - a1 when N is 1
> - op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **O** – The type of the iterator representing the destination range (deduced).
- **BinOp** – The type of the binary function object used for the reduction operation.
- **UnOp** – The type of the unary function object used for the conversion operation.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **binary_op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.
- **unary_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

  ```
  R fun(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.
- **init** – The initial value for the generalized sum.

Returns

The *transform_inclusive_scan* algorithm returns a returns *transform_inclusive_scan_result*< std::ranges::iterator_t<Rng>, O>. The *transform_inclusive_scan* algorithm returns an input iterator to one past the end of the range and an output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng**, typename **O**, typename **BinOp**, typename **UnOp**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type>

parallel::*util*::detail::algorithm_result<*ExPolicy*, transform_inclusive_scan_result<*std*::ranges::iterator_t<*Rng*>, *O*>>::type `hpx::ranges::t`

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, conv(*first), …, conv(*(first + (i - result)))).

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *transform_inclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Neither *conv* nor *op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The behavior of transform_inclusive_scan may be non-deterministic for a non-associative predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicates *op* and *conv*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aN) is defined as:
>
> - a1 when N is 1
> - op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, ..., aN) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **O** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator. This iterator type must meet the requirements of an forward iterator.
- **BinOp** – The type of the binary function object used for the reduction operation.
- **UnOp** – The type of the unary function object used for the conversion operation.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

- **binary_op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

- **unary_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

```
R fun(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

- **init** – The initial value for the generalized sum.

Returns

The *transform_inclusive_scan* algorithm returns a hpx::future<*transform_inclusive_scan_result*< std::ranges::iterator_t<Rng>, O>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *transform_inclusive_scan_result*< std::ranges::iterator_t<Rng>, O> otherwise. The *transform_inclusive_scan* algorithm returns an input iterator to one past the end of the range and an output iterator to the element in the destination range, one past the last element copied.

hpx::ranges::includes

Defined in header hpx/algorithm.hpp[743].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::includes`(*ExPolicy* &&policy, *Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Returns true if every element from the sorted range [first2, last2) is found within the sorted range [first1, last1). Also returns true if [first2, last2) is empty. The version expects both ranges to be sorted with the user supplied binary predicate *f*.

The comparison operations in the parallel *includes* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

[743] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The comparison operations in the parallel *includes* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

At most 2*(N1+N2-1) comparisons, where N1 = std::distance(first1, last1) and N2 = std::distance(first2, last2).

Template Parameters

- `ExPolicy` – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- `Iter1` – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of an forward iterator.
- `Sent1` – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- `Iter2` – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an forward iterator.
- `Sent2` – The type of the end source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an sentinel for Iter2.
- `Pred` – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *includes* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- `Proj1` – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- `Proj2` – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `first1` – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- `last1` – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- `first2` – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- `last2` – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- `op` – The binary predicate which returns true if the elements should be treated as includes. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively

- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *includes* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *includes* algorithm returns true every element from the sorted range [first2, last2) is found within the sorted range [first1, last1). Also returns true if [first2, last2) is empty.

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

bool *hpx*::*ranges*::**includes**(*Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Returns true if every element from the sorted range [first2, last2) is found within the sorted range [first1, last1). Also returns true if [first2, last2) is empty. The version expects both ranges to be sorted with the user supplied binary predicate *f*.

> **Note**
>
> At most 2*(N1+N2-1) comparisons, where N1 = std::distance(first1, last1) and N2 = std::distance(first2, last2).

Template Parameters

- **Iter1** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of an forward iterator.
- **Sent1** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Iter2** – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an forward iterator.
- **Sent2** – The type of the end source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an sentinel for Iter2.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *includes* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*

- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as includes. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *includes* algorithm returns true every element from the sorted range [first2, last2) is found within the sorted range [first1, last1). Also returns true if [first2, last2) is empty.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::`**`includes`**(*ExPolicy* &&policy, *Rng1* &&rng1, *Rng2* &&rng2, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Returns true if every element from the sorted range [first2, last2) is found within the sorted range [first1, last1). Also returns true if [first2, last2) is empty. The version expects both ranges to be sorted with the user supplied binary predicate *f*.

The comparison operations in the parallel *includes* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *includes* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in

an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> At most 2*(N1+N2-1) comparisons, where N1 = std::distance(first1, last1) and N2 = std::distance(first2, last2).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *includes* requires *Pred* to meet the requirements of *Copy-Constructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng1** – Refers to the first sequence of elements the algorithm will be applied to.
- **rng2** – Refers to the second sequence of elements the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as includes. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *includes* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *includes* algorithm returns true every element from the sorted range [first2, last2) is found within the sorted range [first1, last1). Also returns true if [first2, last2) is empty.

template<typename **Rng1**, typename **Rng2**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

bool *hpx*::*ranges*::**includes**(*Rng1* &&rng1, *Rng2* &&rng2, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Returns true if every element from the sorted range [first2, last2) is found within the sorted range [first1, last1). Also returns true if [first2, last2) is empty. The version expects both ranges to be sorted with the user supplied binary predicate *f*.

> **Note**
>
> At most 2*(N1+N2-1) comparisons, where N1 = std::distance(first1, last1) and N2 = std::distance(first2, last2).

Template Parameters

- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *includes* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **rng1** – Refers to the first sequence of elements the algorithm will be applied to.
- **rng2** – Refers to the second sequence of elements the algorithm will be applied to.
- **op** – The binary predicate which returns true if the elements should be treated as includes. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* and *Type2* must be such that objects of types *FwdIter1* and *FwdIter2* can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.

- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *includes* algorithm returns true every element from the sorted range [first2, last2) is found within the sorted range [first1, last1). Also returns true if [first2, last2) is empty.

hpx::ranges::partial_sort

Defined in header hpx/algorithm.hpp[744].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **RandomIt**, typename **Sent**, typename **Comp** = *ranges*::less, typename **Proj** = *hpx*::identity>

RandomIt `hpx::ranges::partial_sort`(*RandomIt* first, *RandomIt* middle, *Sent* last, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Places the first middle - first elements from the range [first, last) as sorted with respect to comp into the range [first, middle). The rest of the elements in the range [middle, last) are placed in an unspecified order.

The assignments in the parallel *partial_sort* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Approximately (last - first) * log(middle - first) comparisons.

Template Parameters

- **RandomIt** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for RandomIt.
- **Comp** – The type of the function/function object to use (deduced). Comp defaults to detail::less.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **middle** – Refers to the middle of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.

[744] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator. Comp defaults to detail::less.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *partial_sort* algorithm returns *RandomIt*. The algorithm returns an iterator pointing to the first element after the last element in the input sequence.

template<typename **ExPolicy**, typename **RandomIt**, typename **Sent**, typename **Comp** = *ranges*::less, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *RandomIt*>::type `hpx::ranges::partial_sort`(*ExPolicy* &&policy, *RandomIt* first, *RandomIt* middle, *Sent* last, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Places the first middle - first elements from the range [first, last) as sorted with respect to comp into the range [first, middle). The rest of the elements in the range [middle, last) are placed in an unspecified order.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Approximately (last - first) * log(middle - first) comparisons.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **RandomIt** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for RandomIt.

- **Comp** – The type of the function/function object to use (deduced). Comp defaults to detail::less.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **middle** – Refers to the middle of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator. Comp defaults to detail::less.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *partial_sort* algorithm returns a *hpx::future<RandomIt>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *RandomIt* otherwise. The algorithm returns an iterator pointing to the first element after the last element in the input sequence.

template<typename **Rng**, typename **Comp** = *ranges*::less, typename **Proj** = *hpx*::identity>

std::ranges::iterator_t<*Rng*> `hpx::ranges::partial_sort`(*Rng* &&rng, *std*::ranges::iterator_t<*Rng*> middle, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Places the first middle - first elements from the range [first, last) as sorted with respect to comp into the range [first, middle). The rest of the elements in the range [middle, last) are placed in an unspecified order.

The assignments in the parallel *partial_sort* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Approximately (last - first) * log(middle - first) comparisons.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Comp** – The type of the function/function object to use (deduced). Comp defaults to detail::less.

- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **middle** – Refers to the middle of the sequence of elements the algorithm will be applied to.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator. Comp defaults to detail::less.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *partial_sort* algorithm returns *std::ranges::iterator_t<Rng>*. It returns *last*.

template<typename **ExPolicy**, typename **Rng**, typename **Comp** = *ranges*::less, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::partial_sort`(*ExPolicy* &&policy, *Rng* &&rng, *std*::ranges::iterator_t<*Rn* middle, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Sorts the elements in the range [first, last) in ascending order. The relative order of equal elements is preserved. The function uses the given comparison function object comp (defaults to using operator<()).

A sequence is sorted with respect to a comparator *comp* and a projection *proj* if for every iterator i pointing to the sequence and every non-negative integer n such that i + n is a valid iterator pointing to an element of the sequence, and INVOKE(comp, INVOKE(proj, *(i + n)), INVOKE(proj, *i)) == false.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: O(N log(N)), where N = std::distance(first, last) comparisons.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Comp** – The type of the function/function object to use (deduced). Comp defaults to detail::less;
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **middle** – Refers to the middle of the sequence of elements the algorithm will be applied to.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator. Comp defaults to detail::less.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *partial_sort* algorithm returns a hpx::future<*std::ranges::iterator_t<Rng>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *std::ranges::iterator_t<Rng>* otherwise. It returns *last*.

hpx::ranges::adjacent_difference

Defined in header hpx/algorithm.hpp[745].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **FwdIter1**, typename **FwdIter2**, typename **Sent**>

FwdIter2 `hpx::ranges::adjacent_difference`(*FwdIter1* first, *Sent* last, *FwdIter2* dest)

Searches the range [first, last) for two consecutive identical elements.

[745] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Note

Complexity: Exactly the smaller of (result - first) + 1 and (last - first) - 1 application of the predicate where *result* is the value returned

Template Parameters

- **FwdIter1** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.

Parameters

- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *adjacent_difference* algorithm returns an iterator to the first of the identical elements. If no such elements are found, *last* is returned.

template<typename **Rng**, typename **FwdIter2**>

FwdIter2 `hpx::ranges::adjacent_difference`(*Rng* &&rng, *FwdIter2* dest)

Searches the *rng* for two consecutive identical elements.

Note

Complexity: Exactly the smaller of (result - first) + 1 and (last - first) - 1 application of the predicate where *result* is the value returned

Template Parameters

- **FwdIter2** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *adjacent_difference* algorithm returns an iterator to the first of the identical elements.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent**, typename **FwdIter2**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> `hpx::ranges::adjacent_difference`(*ExPolicy* &&policy, *FwdIter1* first, *Sent* last, *FwdIter2* dest)

Searches the range [first, last) for two consecutive identical elements.

> **Note**
>
> Complexity: Exactly the smaller of (result - first) + 1 and (last - first) - 1 application of the predicate where *result* is the value returned

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *adjacent_difference* algorithm returns an iterator to the first of the identical elements. If no such elements are found, *last* is returned.

template<typename **ExPolicy**, typename **Rng**, typename **FwdIter2**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> `hpx::ranges::adjacent_difference`(*ExPolicy* &&policy, *Rng* &&rng, *FwdIter2* dest)

Searches the *rng* for two consecutive identical elements.

> **Note**
>
> Complexity: Exactly the smaller of (result - first) + 1 and (last - first) - 1 application of the predicate where *result* is the value returned

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter2** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *adjacent_difference* algorithm returns an iterator to the first of the identical elements.

template<typename **FwdIter1**, typename **Sent**, typename **FwdIter2**, typename **Op**>

FwdIter2 `hpx::ranges::adjacent_difference`(*FwdIter1* first, *Sent* last, *FwdIter2* dest, *Op* &&op)

Searches the range [first, last) for two consecutive identical elements.

> **Note**
>
> Complexity: Exactly the smaller of (result - first) + 1 and (last - first) - 1 application of the predicate where *result* is the value returned

Template Parameters

- **FwdIter1** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **Op** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *adjacent_difference* requires *Op* to meet the requirements of *CopyConstructible*.

Parameters

- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – Binary operation function object that will be applied. The signature of the function should be equivalent to the following:

```
Ret fun(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const &. The types *Type1* and *Type2* must be such that an object of type *iterator_traits<InputIt>::value_type* can be implicitly converted to both of them. The type *Ret* must be such that an object of type *OutputIt* can be dereferenced and assigned a value of type *Ret*.

Returns

The *adjacent_difference* algorithm returns an iterator to the first of the identical elements. If no such elements are found, *last* is returned.

template<typename **Rng**, typename **FwdIter2**, typename **Op**>

FwdIter2 `hpx::ranges::adjacent_difference`(*Rng* &&rng, *FwdIter2* dest, *Op* &&op)

Searches the *rng* for two consecutive identical elements.

Template Parameters

- **FwdIter2** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Op** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *adjacent_difference* requires *Op* to meet the requirements of *CopyConstructible*.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.

- **dest** – Refers to the beginning of the destination range.
- **op** – Binary operation function object that will be applied. The signature of the function should be equivalent to the following:

```
Ret fun(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const &. The types *Type1* and *Type2* must be such that an object of type *iterator_traits<InputIt>::value_type* can be implicitly converted to both of them. The type *Ret* must be such that an object of type *OutputIt* can be dereferenced and assigned a value of type *Ret*.?

Returns

The *adjacent_difference* algorithm returns an iterator to the first of the identical elements.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent**, typename **FwdIter2**, typename **Op**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> `hpx::ranges::adjacent_difference`(*ExPolicy* &&policy, *FwdIter1* first, *Sent* last, *FwdIter2* dest, *Op* &&op)

Searches the range [first, last) for two consecutive identical elements.

Note

Complexity: Exactly the smaller of (result - first) + 1 and (last - first) - 1 application of the predicate where *result* is the value returned

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **FwdIter2** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **Op** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *adjacent_difference* requires *Op* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – Binary operation function object that will be applied. The signature of the function should be equivalent to the following:

```
Ret fun(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &. The types *Type1* and *Type2* must be such that an object of type *iterator_traits<InputIt>::value_type* can be implicitly converted to both of them. The type *Ret* must be such that an object of type *OutputIt* can be dereferenced and assigned a value of type *Ret*.?

Returns

The *adjacent_difference* algorithm returns an iterator to the first of the identical elements. If no such elements are found, *last* is returned.

template<typename **ExPolicy**, typename **Rng**, typename **FwdIter2**, typename **Op**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter2*> `hpx::ranges::adjacent_difference`(*ExPolicy* &&policy, *Rng* &&rng, *FwdIter2* dest, *Op* &&op)

Searches the *rng* for two consecutive identical elements.

> **Note**
>
> Complexity: Exactly the smaller of (result - first) + 1 and (last - first) - 1 application of the predicate where *result* is the value returned

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter2** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

- `Op` – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *adjacent_difference* requires *Op* to meet the requirements of *CopyConstructible*.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – Binary operation function object that will be applied. The signature of the function should be equivalent to the following:

```
Ret fun(const Type1 &a, const Type2 &b);
```

The signature does not need to have const &. The types *Type1* and *Type2* must be such that an object of type *iterator_traits<InputIt>::value_type* can be implicitly converted to both of them. The type *Ret* must be such that an object of type *OutputIt* can be dereferenced and assigned a value of type *Ret*.

Returns

The *adjacent_difference* algorithm returns an iterator to the first of the identical elements.

hpx::ranges::copy

Defined in header hpx/algorithm.hpp[746].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::copy_n*
- *hpx::ranges::copy_if*

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent1**, typename **FwdIter**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *ranges*::copy_result<*FwdIter1*, *FwdIter*>>::type `hpx::ranges::copy`(*ExPolicy* &&policy, *FwdIter1* iter, *Sent1* sent, *FwdIter* dest)

Copies the elements in the range, defined by [first, last), to another range beginning at *dest*.

The assignments in the parallel *copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

[746] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The assignments in the parallel *copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter1`** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Sent1`** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **`FwdIter`** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`iter`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`sent`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`dest`** – Refers to the beginning of the destination range.

Returns

The *copy* algorithm returns a hpx::future<*ranges::copy_result<FwdIter1, FwdIter>* > if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::copy_result<FwdIter1, FwdIter>* otherwise. The *copy* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

template<typename **`ExPolicy`**, typename **`Rng`**, typename **`FwdIter`**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *ranges*::copy_result<typename *hpx*::*traits*::range_traits<*Rng*>::iterator_type, *FwdIter*>>::typ

Copies the elements in the range *rng* to another range beginning at *dest*.

The assignments in the parallel *copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Performs exactly std::distance(begin(rng), end(rng)) assignments.

Template Parameters

- `ExPolicy` – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- `Rng` – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- `FwdIter` – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `rng` – Refers to the sequence of elements the algorithm will be applied to.
- `dest` – Refers to the beginning of the destination range.

Returns

The *copy* algorithm returns a hpx::future<*ranges::copy_result*<*iterator_t*<*Rng*>, FwdIter2>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::copy_result*<*iterator_t*<*Rng*>, FwdIter2> otherwise. The *copy* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

template<typename **FwdIter1**, typename **Sent1**, typename **FwdIter**>

ranges::copy_result<*FwdIter1*, *FwdIter*> `hpx::ranges::copy`(*FwdIter1* iter, *Sent1* sent, *FwdIter* dest)

Copies the elements in the range, defined by [first, last), to another range beginning at *dest*.

Note

Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- `FwdIter1` – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- `Sent1` – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.

- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **iter** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **sent** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *copy* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng**, typename **FwdIter**>

ranges::copy_result<typename *hpx*::*traits*::range_traits<*Rng*>::iterator_type, *FwdIter*> `hpx::ranges::copy`(*Rng* &&rng, *FwdIter* dest)

Copies the elements in the range *rng* to another range beginning at *dest*.

> **Note**
>
> Complexity: Performs exactly std::distance(begin(rng), end(rng)) assignments.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *copy* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

hpx::ranges::copy_n

Defined in header hpx/algorithm.hpp[747].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::copy*
- *hpx::ranges::copy_if*

template<typename **ExPolicy**, typename **FwdIter1**, typename **Size**, typename **FwdIter2**>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *ranges*::copy_n_result<*FwdIter1*, *FwdIter2*>>::type `hpx::ranges::copy_n`(*ExPolicy* &&policy, *FwdIter1* first, *Size* count, *FwdIter2* dest)

Copies the elements in the range [first, first + count), starting from first and proceeding to first + count - 1., to another range beginning at dest.

The assignments in the parallel *copy_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *copy_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

[747] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `count` – Refers to the number of elements starting at *first* the algorithm will be applied to.
- `dest` – Refers to the beginning of the destination range.

Returns

The *copy_n* algorithm returns a hpx::future<*ranges::copy_n_result<FwdIter1, FwdIter2>* > if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::copy_n_result<FwdIter1, FwdIter2>* otherwise. The *copy* algorithm returns the pair of the input iterator forwarded to the first element after the last in the input sequence and the output iterator to the element in the destination range, one past the last element copied.

template<typename **FwdIter1**, typename **Size**, typename **FwdIter2**>

ranges::copy_n_result<*FwdIter1*, *FwdIter2*> `hpx::ranges::copy_n`(*FwdIter1* first, *Size* count, *FwdIter2* dest)

Copies the elements in the range [first, first + count), starting from first and proceeding to first + count - 1., to another range beginning at dest.

> **Note**
>
> Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- `FwdIter1` – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- `Size` – The type of the argument specifying the number of elements to apply *f* to.
- `FwdIter2` – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `count` – Refers to the number of elements starting at *first* the algorithm will be applied to.
- `dest` – Refers to the beginning of the destination range.

Returns

The *copy* algorithm returns the pair of the input iterator forwarded to the first element after the last in the input sequence and the output iterator to the element in the destination range, one past the last element copied.

hpx::ranges::copy_if

Defined in header hpx/algorithm.hpp[748].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::copy*
- *hpx::ranges::copy_n*

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent1**, typename **FwdIter**, typename **Pred**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *ranges*::copy_if_result<*FwdIter1*, *FwdIter*>>::type `hpx::ranges::copy_if`(*ExPolicy* &&policy, *FwdIter1* iter, *Sent1* sent, *FwdIter* dest, *Pred* &&pred, *Proj* &&proj = *Proj*())

Copies the elements in the range, defined by [first, last) to another range beginning at *dest*. The order of the elements that are not removed is preserved.

The assignments in the parallel *copy_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent1** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for FwdIter1.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.

[748] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **Pred** – The type of an optional function/function object to use.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **iter** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **sent** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – The binary predicate which returns *true* if the elements should be treated as equal. The signature should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1* .
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *copy_if* algorithm returns a hpx::future<*ranges::copy_if_result<iterator_t<Rng>*, FwdIter2>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::copy_if_result<iterator_t<Rng>*, FwdIter2> otherwise. The *copy_if* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng**, typename **FwdIter**, typename **Pred**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *ranges*::copy_if_result<typename *hpx*::*traits*::range_traits<*Rng*>::iterator_type, *FwdIte*

Copies the elements in the range, defined by *rng* to another range beginning at *dest*. The order of the elements that are not removed is preserved.

The assignments in the parallel *copy_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an

unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of an optional function/function object to use.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – The binary predicate which returns *true* if the elements should be treated as equal. The signature should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1* .
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *copy_if* algorithm returns a hpx::future<*ranges::copy_if_result<iterator_t<Rng>*, FwdIter2>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::copy_if_result<iterator_t<Rng>*, FwdIter2> otherwise. The *copy_if* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

template<typename **FwdIter1**, typename **Sent1**, typename **FwdIter**, typename **Pred**, typename **Proj** = *hpx*::identity>

ranges::copy_if_result<*FwdIter1*, *FwdIter*> `hpx::ranges::copy_if`(*FwdIter1* iter, *Sent1* sent, *FwdIter* dest, *Pred* &&pred, *Proj* &&proj = *Proj*())

Copies the elements in the range, defined by [first, last) to another range beginning at *dest*. The order of the elements that are not removed is preserved.

Template Parameters

- **FwdIter1** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.

- **Sent1** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for FwdIter1.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of an optional function/function object to use.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **iter** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **sent** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – The binary predicate which returns *true* if the elements should be treated as equal. The signature should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1* .
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *copy_if* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng**, typename **FwdIter**, typename **Pred**, typename **Proj** = *hpx*::identity>

ranges::copy_if_result<typename *hpx*::*traits*::range_traits<*Rng*>::iterator_type, *FwdIter*> **hpx::ranges::copy_if**(*Rng* &&rng, *FwdIter* dest, *Pred* &&pred, *Proj* &&proj = *Proj*())

Copies the elements in the range, defined by *rng* to another range beginning at *dest*. The order of the elements that are not removed is preserved.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of an optional function/function object to use.

- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – The binary predicate which returns *true* if the elements should be treated as equal. The signature should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The types *Type1* must be such that objects of type *FwdIter* can be dereferenced and then implicitly converted to *Type1* .

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *copy_if* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

hpx::ranges::is_sorted

Defined in header hpx/algorithm.hpp[749].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::is_sorted_until*

template<typename **FwdIter**, typename **Sent**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

bool `hpx::ranges::is_sorted`(*FwdIter* first, *Sent* last, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Determines if the range [first, last) is sorted. Uses pred to compare elements.

The comparison operations in the parallel *is_sorted* algorithm executes in sequential order in the calling thread.

> **Note**
>
> Complexity: at most (N+S-1) comparisons where *N* = distance(first, last). *S* = number of partitions

Template Parameters

- **FwdIter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.

[749] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **Pred** – The type of an optional function/function object to use.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of that the algorithm will be applied to.
- **pred** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second argument. The signature of the function should be equivalent to

```
bool pred(const Type &a, const Type &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_sorted* algorithm returns a *bool*. The *is_sorted* algorithm returns true if each element in the sequence [first, last) satisfies the predicate passed. If the range [first, last) contains less than two elements, the function always returns true.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::is_sorted`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Determines if the range [first, last) is sorted. Uses pred to compare elements.

The comparison operations in the parallel *is_sorted* algorithm invoked with an execution policy object of type *sequenced_policy* executes in sequential order in the calling thread.

The comparison operations in the parallel *is_sorted* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (N+S-1) comparisons where *N* = distance(first, last). *S* = number of partitions

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *is_sorted* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of that the algorithm will be applied to.
- **pred** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second argument. The signature of the function should be equivalent to

  ```
  bool pred(const Type &a, const Type &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_sorted* algorithm returns a *hpx::future<bool>* if the execution policy is of type *task_execution_policy* and returns *bool* otherwise. The *is_sorted* algorithm returns a bool if each element in the sequence [first, last) satisfies the predicate passed. If the range [first, last) contains less than two elements, the function always returns true.

template<typename **Rng**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

bool *hpx::ranges::***is_sorted**(*Rng* &&rng, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Determines if the range rng is sorted. Uses pred to compare elements.

The comparison operations in the parallel *is_sorted* algorithm executes in sequential order in the calling thread.

> **Note**
>
> Complexity: at most (N+S-1) comparisons where N = size(rng). S = number of partitions

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second argument. The signature of the function should be equivalent to

  ```
  bool pred(const Type &a, const Type &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_sorted* algorithm returns a *bool*. The *is_sorted* algorithm returns true if each element in the rng satisfies the predicate passed. If the range rng contains less than two elements, the function always returns true.

template<typename **ExPolicy**, typename **Rng**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::is_sorted`(*ExPolicy* &&policy, *Rng* &&rng, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Determines if the range rng is sorted. Uses pred to compare elements.

The comparison operations in the parallel *is_sorted* algorithm invoked with an execution policy object of type *sequenced_policy* executes in sequential order in the calling thread.

The comparison operations in the parallel *is_sorted* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (N+S-1) comparisons where N = size(rng). S = number of partitions

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *is_sorted* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second argument. The signature of the function should be equivalent to

  ```
  bool pred(const Type &a, const Type &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_sorted* algorithm returns a *hpx::future<bool>* if the execution policy is of type *task_execution_policy* and returns *bool* otherwise. The *is_sorted* algorithm returns a bool if each element in the range rng satisfies the predicate passed. If the range rng contains less than two elements, the function always returns true.

hpx::ranges::is_sorted_until

Defined in header hpx/algorithm.hpp[750].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::is_sorted*

[750] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

template<typename **FwdIter**, typename **Sent**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

FwdIter `hpx::ranges::is_sorted_until`(*FwdIter* first, *Sent* last, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Returns the first element in the range [first, last) that is not sorted. Uses a predicate to compare elements or the less than operator.

The comparison operations in the parallel *is_sorted_until* algorithm execute in sequential order in the calling thread.

Note

Complexity: at most (N+S-1) comparisons where N = distance(first, last). S = number of partitions

Template Parameters

- **FwdIter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **Pred** – The type of an optional function/function object to use.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of that the algorithm will be applied to.
- **pred** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second argument. The signature of the function should be equivalent to

```
bool pred(const Type &a, const Type &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_sorted_until* algorithm returns a *FwdIter*. The *is_sorted_until* algorithm returns the first unsorted element. If the sequence has less than two elements or the sequence is sorted, last is returned.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter*>::type `hpx::ranges::is_sorted_until`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Returns the first element in the range [first, last) that is not sorted. Uses a predicate to compare elements or the less than operator.

The comparison operations in the parallel *is_sorted_until* algorithm invoked with an execution policy object of type *sequenced_policy* executes in sequential order in the calling thread.

The comparison operations in the parallel *is_sorted_until* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: at most (N+S-1) comparisons where *N* = distance(first, last). *S* = number of partitions

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *is_sorted_until* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of that the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of that the algorithm will be applied to.
- **pred** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second argument. The signature of the function should be equivalent to

  ```
  bool pred(const Type &a, const Type &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_sorted_until* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *task_execution_policy* and returns *FwdIter* otherwise. The *is_sorted_until* algorithm returns the first unsorted element. If the sequence has less than two elements or the sequence is sorted, last is returned.

template<typename **Rng**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

std::ranges::iterator_t<*Rng*> *hpx*::*ranges*::**is_sorted_until**(*Rng* &&rng, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Returns the first element in the range rng that is not sorted. Uses a predicate to compare elements or the less than operator.

> **Note**
>
> Complexity: at most (N+S-1) comparisons where *N* = size(rng). *S* = number of partitions

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *is_sorted_until* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.

- **pred** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second argument. The signature of the function should be equivalent to

```
bool pred(const Type &a, const Type &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_sorted_until* returns *FwdIter*. The *is_sorted_until* algorithm returns the first unsorted element. If the sequence has less than two elements or the sequence is sorted, last is returned.

template<typename **ExPolicy**, typename **Rng**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>
hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::is_sorted_until`(*ExPolicy* &&policy, *Rng* &&rng, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

Returns the first element in the range rng that is not sorted. Uses a predicate to compare elements or the less than operator.

The comparison operations in the parallel *is_sorted_until* algorithm invoked with an execution policy object of type *sequenced_policy* executes in sequential order in the calling thread.

The comparison operations in the parallel *is_sorted_until* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (N+S-1) comparisons where N = size(rng). S = number of partitions

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *is_sorted_until* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second argument. The signature of the function should be equivalent to

```
bool pred(const Type &a, const Type &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_sorted_until* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *task_execution_policy* and returns *FwdIter* otherwise. The *is_sorted_until* algorithm returns the first unsorted element. If the sequence has less than two elements or the sequence is sorted, last is returned.

hpx::ranges::make_heap

Defined in header hpx/algorithm.hpp[751].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **Comp**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *Iter*> `hpx::ranges::make_heap`(*ExPolicy* &&policy, *Iter* first, *Sent* last, *Comp* &&comp, *Proj* &&proj = *Proj*{})

Constructs a *max heap* in the range [first, last).

The predicate operations in the parallel *make_heap* algorithm invoked with an execution policy object of type *sequential_execution_policy* executes in sequential order in the calling thread.

[751] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The comparison operations in the parallel *make_heap* algorithm invoked with an execution policy object of type *parallel_execution_policy* or *parallel_task_execution_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (3*N) comparisons where *N* = distance(first, last).

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`Iter`** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Sent`** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **`Comp`** – The type of the function/function object to use (deduced).
- **`Proj`** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`comp`** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second. The signature of the function should be equivalent to

  ```
  bool comp(const Type &a, const Type &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *RndIter* can be dereferenced and then implicitly converted to Type.
- **`proj`** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *make_heap* algorithm returns a *hpx::future<Iter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *Iter* otherwise. It returns *last*.

template<typename **ExPolicy**, typename **Rng**, typename **Comp**, typename **Proj** = *hpx*::identity>

hpx::parallel::util::detail::algorithm_result<ExPolicy, std::ranges::iterator_t<Rng>> `hpx::ranges::make_heap`(*ExPolicy* &&policy, *Rng* &&rng, *Comp* &&comp, *Proj* &&proj = *Proj*{})

Constructs a *max heap* in the range [first, last).

The predicate operations in the parallel *make_heap* algorithm invoked with an execution policy object of type *sequential_execution_policy* executes in sequential order in the calling thread.

The comparison operations in the parallel *make_heap* algorithm invoked with an execution policy object of type *parallel_execution_policy* or *parallel_task_execution_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (3*N) comparisons where *N* = distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **comp** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second. The signature of the function should be equivalent to

```
bool comp(const Type &a, const Type &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *RndIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *make_heap* algorithm returns a *hpx::future<Iter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *Iter* otherwise. It returns *last*.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *Iter*> `hpx::ranges::make_heap`(*ExPolicy* &&policy, *Iter* first, *Sent* last, *Proj* &&proj = *Proj*{})

Constructs a *max heap* in the range [first, last).

The predicate operations in the parallel *make_heap* algorithm invoked with an execution policy object of type *sequential_execution_policy* executes in sequential order in the calling thread.

The comparison operations in the parallel *make_heap* algorithm invoked with an execution policy object of type *parallel_execution_policy* or *parallel_task_execution_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (3*N) comparisons where *N* = distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Iter** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *make_heap* algorithm returns a *hpx::future<Iter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *Iter* otherwise. It returns *last*.

template<typename **ExPolicy**, typename **Rng**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::make_heap`(*ExPolicy* &&policy, *Rng* &&rng, *Proj* &&proj = *Proj*{})

Constructs a *max heap* in the range [first, last).

The predicate operations in the parallel *make_heap* algorithm invoked with an execution policy object of type *sequential_execution_policy* executes in sequential order in the calling thread.

The comparison operations in the parallel *make_heap* algorithm invoked with an execution policy object of type *parallel_execution_policy* or *parallel_task_execution_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: at most (3*N) comparisons where *N* = distance(first, last).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *make_heap* algorithm returns a *hpx::future<Iter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *Iter* otherwise. It returns *last*.

template<typename **Iter**, typename **Sent**, typename **Comp**, typename **Proj** = *hpx*::identity>

Iter `hpx::ranges::make_heap`(*Iter* first, *Sent* last, *Comp* &&comp, *Proj* &&proj = *Proj*{})

Constructs a *max heap* in the range [first, last).

> **Note**
>
> Complexity: at most (3*N) comparisons where *N* = distance(first, last).

Template Parameters

- **Iter** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second. The signature of the function should be equivalent to

  ```
  bool comp(const Type &a, const Type &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *RndIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *make_heap* algorithm returns *Iter*. It returns *last*.

template<typename **Rng**, typename **Comp**, typename **Proj** = *hpx*::identity>

std::ranges::iterator_t<*Rng*> `hpx::ranges::make_heap`(*Rng* &&rng, *Comp* &&comp, *Proj* &&proj = *Proj*{})

Constructs a *max heap* in the range [first, last).

> **Note**
>
> Complexity: at most (3*N) comparisons where *N* = distance(first, last).

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **comp** – Refers to the binary predicate which returns true if the first argument should be treated as less than the second. The signature of the function should be equivalent to

```
bool comp(const Type &a, const Type &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type* must be such that objects of types *RndIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *make_heap* algorithm returns *Iter*. It returns *last*.

template<typename **Iter**, typename **Sent**, typename **Proj** = *hpx*::identity>

Iter `hpx::ranges::make_heap`(*Iter* first, *Sent* last, *Proj* &&proj = *Proj*{})

Constructs a *max heap* in the range [first, last).

> **Note**
>
> Complexity: at most (3*N) comparisons where *N* = distance(first, last).

Template Parameters

- **Iter** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *make_heap* algorithm returns *Iter*. It returns *last*.

template<typename **Rng**, typename **Proj** = *hpx*::identity>

std::ranges::iterator_t<*Rng*> `hpx::ranges::make_heap`(*Rng* &&rng, *Proj* &&proj = *Proj*{})

Constructs a *max heap* in the range [first, last).

> **Note**
>
> Complexity: at most (3*N) comparisons where *N* = distance(first, last).

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *make_heap* algorithm returns *Iter*. It returns *last*.

hpx::ranges::set_union

Defined in header hpx/algorithm.hpp[752].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

[752] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, set_union_result<*Iter1*, *Iter2*, *Iter3*>>::type `hpx::ranges::set_union`(*ExPolicy* &&policy, *Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Iter3* dest, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Constructs a sorted range beginning at dest consisting of all elements present in one or both sorted ranges [first1, last1) and [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), then all *m* elements will be copied from [first1, last1) to dest, preserving order, and then exactly std::max(n-m, 0) elements will be copied from [first2, last2) to dest, also preserving order.

The resulting range cannot overlap with either of the input ranges.

The application of function objects in parallel algorithm invoked with a sequential execution policy object execute in sequential order in the calling thread (*sequenced_policy*) or in a single new thread spawned from the current thread (for *sequenced_task_policy*).

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Iter1** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of an forward iterator.
- **Sent1** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Iter2** – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an forward iterator.
- **Sent2** – The type of the end source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an sentinel for Iter2.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_union* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.

- `proj2` – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_union* algorithm returns a hpx::future<*ranges::set_union_result*<*Iter1, Iter2, Iter3*>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::set_union_result*<*Iter1, Iter2, Iter3*> otherwise. The *set_union* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, set_union_result<*std*::ranges::iterator_t<*Rng1*>, *std*::ranges::iterator_t<*Rng2*>, *Iter3*>>

Constructs a sorted range beginning at dest consisting of all elements present in one or both sorted ranges [first1, last1) and [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), then all *m* elements will be copied from [first1, last1) to dest, preserving order, and then exactly std::max(n-m, 0) elements will be copied from [first2, last2) to dest, also preserving order.

The resulting range cannot overlap with either of the input ranges.

The application of function objects in parallel algorithm invoked with a sequential execution policy object execute in sequential order in the calling thread (*sequenced_policy*) or in a single new thread spawned from the current thread (for *sequenced_task_policy*).

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an

unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_union* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng1** – Refers to the first sequence of elements the algorithm will be applied to.
- **rng2** – Refers to the second sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_union* algorithm returns a hpx::future<*ranges::set_union_result<Iter1, Iter2, Iter3>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::set_union_result<Iter1, Iter2, Iter3>* otherwise. where Iter1 is range_iterator_t<Rng1> and Iter2 is range_iterator_t<Rng2> The *set_union* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng1**, typename **Rng2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

set_union_result<*std*::ranges::iterator_t<*Rng1*>, *std*::ranges::iterator_t<*Rng2*>, *Iter3*> `hpx::ranges::set_union`(*Rng1* &&rng1, *Rng2* &&rng2, *Iter3* dest, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Constructs a sorted range beginning at dest consisting of all elements present in one or both sorted ranges [first1, last1) and [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), then all *m* elements will be copied from [first1, last1) to dest, preserving order, and then exactly std::max(n-m, 0) elements will be copied from [first2, last2) to dest, also preserving order.

The resulting range cannot overlap with either of the input ranges.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_union* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **rng1** – Refers to the first sequence of elements the algorithm will be applied to.
- **rng2** – Refers to the second sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*

- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_union* algorithm returns *ranges::set_union_result<Iter1, Iter2, Iter3>*. where Iter1 is range_iterator_t<Rng1> and Iter2 is range_iterator_t<Rng2> The *set_union* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::ranges::lexicographical_compare

Defined in header hpx/algorithm.hpp[753].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter1**, typename **Sent1**, typename **InIter2**, typename **Sent2**, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity, typename **Pred** = *hpx*::*parallel*::detail::less>

bool `hpx::ranges::lexicographical_compare`(*InIter1* first1, *Sent1* last1, *InIter2* first2, *Sent2* last2, *Pred* &&pred = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

[753] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Checks if the first range [first1, last1) is lexicographically less than the second range [first2, last2). uses a provided predicate to compare elements.

The comparison operations in the parallel *lexicographical_compare* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: At most 2 * min(N1, N2) applications of the comparison operation, where N1 = std::distance(first1, last) and N2 = std::distance(first2, last2).

> **Note**
>
> Lexicographical comparison is an operation with the following properties
>
> - Two ranges are compared element by element
> - The first mismatching element defines which range is lexicographically *less* or *greater* than the other
> - If one range is a prefix of another, the shorter range is lexicographically *less* than the other
> - If two ranges have equivalent elements and are of the same length, then the ranges are lexicographically *equal*
> - An empty range is lexicographically *less* than any non-empty range
> - Two empty ranges are lexicographically *equal*

Template Parameters

- **`InIter1`** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an input iterator.
- **`Sent1`** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter1.
- **`InIter2`** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an input iterator.
- **`Sent2`** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter2.
- **`Pred`** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *lexicographical_compare* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **`Proj1`** – The type of an optional projection function for FwdIter1. This defaults to *hpx::identity*
- **`Proj2`** – The type of an optional projection function for FwdIter2. This defaults to *hpx::identity*

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **pred** – Refers to the comparison function that the first and second ranges will be applied to
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual predicate *is* invoked.

Returns

The *lexicographically_compare* algorithm returns *bool*. The *lexicographically_compare* algorithm returns true if the first range is lexicographically less, otherwise it returns false. range [first2, last2), it returns false.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent1**, typename **FwdIter2**, typename **Sent2**, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity, typename **Pred** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::lexicographical_compare`(*ExPolicy* &&policy, *FwdIter1* first1, *Sent1* last1, *FwdIter2* first2, *Sent2* last2, *Pred* &&pred = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Checks if the first range [first1, last1) is lexicographically less than the second range [first2, last2). uses a provided predicate to compare elements.

The comparison operations in the parallel *lexicographical_compare* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *lexicographical_compare* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most 2 * min(N1, N2) applications of the comparison operation, where N1 = std::distance(first1, last) and N2 = std::distance(first2, last2).

> **Note**
>
> Lexicographical comparison is an operation with the following properties
>
> - Two ranges are compared element by element
> - The first mismatching element defines which range is lexicographically *less* or *greater* than the other
> - If one range is a prefix of another, the shorter range is lexicographically *less* than the other
> - If two ranges have equivalent elements and are of the same length, then the ranges are lexicographically *equal*
> - An empty range is lexicographically *less* than any non-empty range
> - Two empty ranges are lexicographically *equal*

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter1`** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Sent1`** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter1.
- **`FwdIter2`** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Sent2`** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter2.
- **`Pred`** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *lexicographical_compare* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>

- **Proj1** – The type of an optional projection function for FwdIter1. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function for FwdIter2. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **pred** – Refers to the comparison function that the first and second ranges will be applied to
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual predicate *is* invoked.

Returns

The *lexicographically_compare* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *lexicographically_compare* algorithm returns true if the first range is lexicographically less, otherwise it returns false. range [first2, last2), it returns false.

template<typename **Rng1**, typename **Rng2**, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity, typename **Pred** = *hpx*::*parallel*::detail::less>

bool *hpx::ranges::***lexicographical_compare**(*Rng1* &&rng1, *Rng2* &&rng2, *Pred* &&pred = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Checks if the first range rng1 is lexicographically less than the second range rng2. uses a provided predicate to compare elements.

The comparison operations in the parallel *lexicographical_compare* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: At most 2 * min(N1, N2) applications of the comparison operation, where N1 = std::distance(std::begin(rng1), std::end(rng1)) and N2 = std::distance(std::begin(rng2), std::end(rng2)).

Note

Lexicographical comparison is an operation with the following properties

- Two ranges are compared element by element
- The first mismatching element defines which range is lexicographically *less* or *greater* than the other
- If one range is a prefix of another, the shorter range is lexicographically *less* than the other
- If two ranges have equivalent elements and are of the same length, then the ranges are lexicographically *equal*
- An empty range is lexicographically *less* than any non-empty range
- Two empty ranges are lexicographically *equal*

Template Parameters

- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *lexicographical_compare* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function for elements of the first range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function for elements of the second range. This defaults to *hpx::identity*

Parameters

- **rng1** – Refers to the sequence of elements the algorithm will be applied to.
- **rng2** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Refers to the comparison function that the first and second ranges will be applied to
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual predicate *is* invoked.

Returns

The *lexicographically_compare* algorithm returns *bool*. The *lexicographically_compare* algorithm returns true if the first range is lexicographically less, otherwise it returns false. range [first2, last2), it returns false.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity, typename **Pred** = *hpx*::*parallel*::detail::less>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::lexicographical_compare`(*ExPolicy* &&policy, *Rng1* &&rng1, *Rng2* &&rng2, *Pred* &&pred = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Checks if the first range rng1 is lexicographically less than the second range rng2. uses a provided predicate to compare elements.

The comparison operations in the parallel *lexicographical_compare* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparison operations in the parallel *lexicographical_compare* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most 2 * min(N1, N2) applications of the comparison operation, where N1 = std::distance(std::begin(rng1), std::end(rng1)) and N2 = std::distance(std::begin(rng2), std::end(rng2)).

Note

Lexicographical comparison is an operation with the following properties

- Two ranges are compared element by element
- The first mismatching element defines which range is lexicographically *less* or *greater* than the other
- If one range is a prefix of another, the shorter range is lexicographically *less* than the other

- If two ranges have equivalent elements and are of the same length, then the ranges are lexicographically *equal*
- An empty range is lexicographically *less* than any non-empty range
- Two empty ranges are lexicographically *equal*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *lexicographical_compare* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function for elements of the first range. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function for elements of the second range. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng1** – Refers to the sequence of elements the algorithm will be applied to.
- **rng2** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Refers to the comparison function that the first and second ranges will be applied to
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first range as a projection operation before the actual predicate *is* invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second range as a projection operation before the actual predicate *is* invoked.

Returns

The *lexicographically_compare* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *lexicographically_compare* algorithm returns true if the first range is lexicographically less, otherwise it returns false. range [first2, last2), it returns false.

hpx::ranges::set_intersection

Defined in header hpx/algorithm.hpp[754].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, set_intersection_result<*Iter1*, *Iter2*, *Iter3*>>::type `hpx::ranges::set_intersection`

Constructs a sorted range beginning at dest consisting of all elements present in both sorted ranges [first1, last1) and [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), the first std::min(m, n) elements will be copied from the first range to the destination range. The order of equivalent elements is preserved. The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

The application of function objects in parallel algorithm invoked with a sequential execution policy object execute in sequential order in the calling thread (*sequenced_policy*) or in a single new thread spawned from the current thread (for *sequenced_task_policy*).

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an

[754] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **`Iter1`** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of an forward iterator.
- **`Sent1`** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **`Iter2`** – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an forward iterator.
- **`Sent2`** – The type of the end source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an sentinel for Iter2.
- **`Iter3`** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **`Pred`** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_intersection* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **`Proj1`** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **`Proj2`** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first1`** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **`last1`** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **`first2`** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **`last2`** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **`dest`** – Refers to the beginning of the destination range.
- **`op`** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*

- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_intersection* algorithm returns a hpx::future<*ranges::set_intersection_result*<*Iter1, Iter2, Iter3*>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::set_intersection_result*<*Iter1, Iter2, Iter3*> otherwise. The *set_intersection* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, set_intersection_result<*std*::ranges::iterator_t<*Rng1*>, *std*::ranges::iterator_t<*Rng2*>, *Ite*

Constructs a sorted range beginning at dest consisting of all elements present in both sorted ranges [first1, last1) and [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), the first std::min(m, n) elements will be copied from the first range to the destination range. The order of equivalent elements is preserved. The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

The application of function objects in parallel algorithm invoked with a sequential execution policy object execute in sequential order in the calling thread (*sequenced_policy*) or in a single new thread spawned from the current thread (for *sequenced_task_policy*).

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_intersection* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng1** – Refers to the first sequence of elements the algorithm will be applied to.
- **rng2** – Refers to the second sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*

- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_intersection* algorithm returns a hpx::future<*ranges::set_intersection_result*<*Iter1, Iter2, Iter3*>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::set_intersection_result*<*Iter1, Iter2, Iter3*> otherwise. where Iter1 is range_iterator_t<Rng1> and Iter2 is range_iterator_t<Rng2> The *set_intersection* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

set_intersection_result<*Iter1*, *Iter2*, *Iter3*> **hpx::ranges::set_intersection**(*Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Iter3* dest, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Constructs a sorted range beginning at dest consisting of all elements present in both sorted ranges [first1, last1) and [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), the first std::min(m, n) elements will be copied from the first range to the destination range. The order of equivalent elements is preserved. The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **Iter1** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of an forward iterator.
- **Sent1** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Iter2** – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an forward iterator.

- **Sent2** – The type of the end source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an sentinel for Iter2.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_intersection* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_intersection* algorithm returns *ranges::set_intersection_result<Iter1, Iter2, Iter3>*. The *set_intersection* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng1**, typename **Rng2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

set_intersection_result<*std*::ranges::iterator_t<*Rng1*>, *std*::ranges::iterator_t<*Rng2*>, *Iter3*> `hpx::ranges::set_intersection`(*Rng1* &&rng1, *Rng2* &&rng2, *Iter3* dest, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Constructs a sorted range beginning at dest consisting of all elements present in both sorted ranges [first1, last1) and [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), the first std::min(m, n) elements will be copied from the first range to the destination range. The order of equivalent elements is preserved. The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_intersection* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **rng1** – Refers to the first sequence of elements the algorithm will be applied to.
- **rng2** – Refers to the second sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_intersection* algorithm returns *ranges::set_intersection_result<Iter1, Iter2, Iter3>*. where Iter1 is range_iterator_t<Rng1> and Iter2 is range_iterator_t<Rng2> The *set_intersection* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::ranges::transform_exclusive_scan

Defined in header hpx/algorithm.hpp[755].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InIter**, typename **Sent**, typename **OutIter**, typename **BinOp**, typename **UnOp**, typename **T** = typename *std*::iterator_traits<*InIter*>::value_type>

transform_exclusive_scan_result<*InIter*, *OutIter*> `hpx::ranges::transform_exclusive_scan`(*InIter* first, *Sent* last, *OutIter* dest, *T* init, *BinOp* &&binary_op, *UnOp* &&unary_op)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, conv(*first), …, conv(*(first + (i - result) - 1))).

The reduce operations in the parallel *transform_exclusive_scan* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

[755] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Neither *conv* nor *op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The behavior of transform_exclusive_scan may be non-deterministic for a non-associative predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicates *op* and *conv*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:
>
> - a1 when N is 1
> - op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **BinOp** – The type of the binary function object used for the reduction operation.
- **UnOp** – The type of the unary function object used for the conversion operation.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest** – Refers to the beginning of the destination range.
- **init** – The initial value for the generalized sum.
- **binary_op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

- **unary_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

```
R fun(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_exclusive_scan* algorithm returns *transform_exclusive_scan_result<InIter, OutIter>*. The *transform_exclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent**, typename **FwdIter2**, typename **BinOp**, typename **UnOp**, typename **T** = typename *std*::iterator_traits<*FwdIter1*>::value_type>

parallel::*util*::detail::algorithm_result<*ExPolicy*, transform_exclusive_result<*FwdIter1*, *FwdIter2*>>::type `hpx::ranges::transform_excl`

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, conv(*first), …, conv(*(first + (i - result) - 1))).

The reduce operations in the parallel *transform_exclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *transform_exclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Neither *conv* nor *op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The behavior of transform_exclusive_scan may be non-deterministic for a non-associative predicate.

Note

Complexity: O(*last* - *first*) applications of the predicates *op* and *conv*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:

- a1 when N is 1
- op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **BinOp** – The type of the binary function object used for the reduction operation.
- **UnOp** – The type of the unary function object used for the conversion operation.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest** – Refers to the beginning of the destination range.
- **init** – The initial value for the generalized sum.
- **binary_op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

- **unary_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

```
R fun(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_exclusive_scan* algorithm returns a hpx::future<*transform_exclusive_result*<*FwdIter1*, FwdIter2>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *transform_exclusive_result*<*FwdIter1*, FwdIter2> otherwise. The *transform_exclusive_scan* algorithm returns an input iterator to the point denoted by the sentinel and an output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng**, typename **O**, typename **BinOp**, typename **UnOp**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type>

transform_exclusive_scan_result<*std*::ranges::iterator_t<*Rng*>, *O*> *hpx::ranges::***transform_exclusive_scan**(*Rng* &&rng, *O* dest, *T* init, *BinOp* &&binary_op, *UnOp* &&unary_op)

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, conv(*first), …, conv(*(first + (i - result) - 1))).

The reduce operations in the parallel *transform_exclusive_scan* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Neither *conv* nor *op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The behavior of transform_exclusive_scan may be non-deterministic for a non-associative predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicates *op* and *conv*.

Note

GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:

- a1 when N is 1
- op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **O** – The type of the iterator representing the destination range (deduced).
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **BinOp** – The type of the binary function object used for the reduction operation.
- **UnOp** – The type of the unary function object used for the conversion operation.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **init** – The initial value for the generalized sum.
- **binary_op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.

- **unary_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

  ```
  R fun(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_exclusive_scan* algorithm returns a returns *transform_exclusive_scan_result*< std::ranges::iterator_t<Rng>, O>. The *transform_exclusive_scan* algorithm returns an input iterator to one past the end of the range and an output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng**, typename **O**, typename **BinOp**, typename **UnOp**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type>

parallel::*util*::detail::algorithm_result<*ExPolicy*, transform_exclusive_scan_result<*std*::ranges::iterator_t<*Rng*>, *O*>>::type `hpx::ranges::t`

Assigns through each iterator *i* in [result, result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, conv(*first), …, conv(*(first + (i - result) - 1))).

The reduce operations in the parallel *transform_exclusive_scan* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *transform_exclusive_scan* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Neither *conv* nor *op* shall invalidate iterators or sub-ranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).

The behavior of transform_exclusive_scan may be non-deterministic for a non-associative predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicates *op* and *conv*.

> **Note**
>
> GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aN) is defined as:
>
> - a1 when N is 1
> - op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, …, aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, …, aN) where 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **O** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator. This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).
- **BinOp** – The type of the binary function object used for the reduction operation.
- **UnOp** – The type of the unary function object used for the conversion operation.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **init** – The initial value for the generalized sum.
- **binary_op** – Specifies the function (or function object) which will be invoked for each of the values of the input sequence. This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The types *Type1* and *Ret* must be such that an object of a type as given by the input sequence can be implicitly converted to any of those types.
- **unary_op** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a unary predicate. The signature of this predicate should be equivalent to:

  ```
  R fun(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type. The type *R* must be such that an object of this type can be implicitly converted to *T*.

Returns

The *transform_exclusive_scan* algorithm returns a hpx::future<*transform_exclusive_scan_result*< std::ranges::iterator_t<Rng>, O>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *transform_exclusive_scan_result*< std::ranges::iterator_t<Rng>, O> otherwise. The *transform_exclusive_scan* algorithm returns an input iterator to one past the end of the range and an output iterator to the element in the destination range, one past the last element copied.

hpx::ranges::uninitialized_fill

Defined in header hpx/algorithm.hpp[756].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::uninitialized_fill_n*

template<typename **FwdIter**, typename **Sent**, typename **T**>

FwdIter `hpx::ranges::uninitialized_fill`(*FwdIter* first, *Sent* last, *T* const &value)

Copies the given *value* to an uninitialized memory area, defined by the range [first, last). If an exception is thrown during the initialization, the function has no effects.

The assignments in the ranges *uninitialized_fill* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **value** – The value to be assigned.

Returns

The *uninitialized_fill* algorithm returns a returns *FwdIter*. The *uninitialized_fill* algorithm returns the output iterator to the element in the range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **T**>

[756] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::ranges::uninitialized_fill`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *T* const &value)

Copies the given *value* to an uninitialized memory area, defined by the range [first, last). If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_fill* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_fill* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Linear in the distance between *first* and *last*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **value** – The value to be assigned.

Returns

The *uninitialized_fill* algorithm returns a returns *FwdIter*. The *uninitialized_fill* algorithm returns the output iterator to the element in the range, one past the last element copied.

template<typename **Rng**, typename **T**>

hpx::*traits*::range_traits<*Rng*>::iterator_type `hpx::ranges::`**`uninitialized_fill`**(*Rng* &&rng, *T* const &value)

Copies the given *value* to an uninitialized memory area, defined by the range [first, last). If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_fill* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **rng** – Refers to the range to which the value will be filled
- **value** – The value to be assigned.

Returns

The *uninitialized_fill* algorithm returns a returns *hpx::traits::range_traits<Rng>::iterator_type*. The *uninitialized_fill* algorithm returns the output iterator to the element in the range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng**, typename **T**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, typename *hpx*::*traits*::range_traits<Rng1>::iterator_type>::type `hpx::ranges::`**`uninitial`**

Copies the given *value* to an uninitialized memory area, defined by the range [first, last). If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_fill* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_fill* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the range to which the value will be filled
- **value** – The value to be assigned.

Returns

The *uninitialized_fill* algorithm returns a hpx::future<*typename* hpx::traits::range_traits<Rng>::iterator_type>, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *typename* hpx::traits::range_traits<Rng>::iterator_type otherwise. The *uninitialized_fill* algorithm returns the iterator to one past the last element filled in the range.

hpx::ranges::uninitialized_fill_n

Defined in header hpx/algorithm.hpp[757].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::uninitialized_fill*

template<typename **FwdIter**, typename **Size**, typename **T**>

FwdIter `hpx::ranges::`**`uninitialized_fill_n`**(*FwdIter* first, *Size* count, *T* const &value)

Copies the given *value* value to the first count elements in an uninitialized memory area beginning at first. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_fill_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

[757] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Note

Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *uninitialized_fill_n* algorithm returns a returns *FwdIter*. The *uninitialized_fill_n* algorithm returns the output iterator to the element in the range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**, typename **T**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::ranges::uninitialized_fill_n`(*ExPolicy* &&policy, *FwdIter* first, *Size* count, *T* const &value)

Copies the given *value* value to the first count elements in an uninitialized memory area beginning at first. If an exception is thrown during the initialization, the function has no effects.

The assignments in the parallel *uninitialized_fill_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_fill_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Performs exactly *count* assignments, if count > 0, no assignments otherwise.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *uninitialized_fill_n* algorithm returns a *hpx::future<FwdIter>*, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns FwdIter otherwise. The *uninitialized_fill_n* algorithm returns the output iterator to the element in the range, one past the last element copied.

hpx::ranges::move

Defined in header hpx/algorithm.hpp[758].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **Iter1**, typename **Sent1**, typename **Iter2**>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, move_result<*Iter1*, *Iter2*>>::type `hpx::ranges::move`(*ExPolicy* &&policy, *Iter1* first, *Sent1* last, *Iter2* dest)

[758] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Moves the elements in the range *rng* to another range beginning at *dest*. After this operation the elements in the moved-from range will still contain valid values of the appropriate type, but not necessarily the same values as before the move.

The assignments in the parallel *copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly std::distance(begin(rng), end(rng)) assignments.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`Iter1`** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Sent1`** – The type of the source iterators used for the end of the first range (deduced).
- **`Iter2`** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`dest`** – Refers to the beginning of the destination range.

Returns

The *move* algorithm returns a hpx::future<*ranges::move_result*<*iterator_t*<*Rng*>, FwdIter2>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::move_result*<*iterator_t*<*Rng*>, FwdIter2> otherwise. The *move* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element moved.

template<typename **`ExPolicy`**, typename **`Rng`**, typename **`Iter2`**>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, move_result<*std*::ranges::iterator_t<*Rng*>, *Iter2*>>::type `hpx::ranges::move`(*ExPolicy* &&policy, *Rng* &&rng, *Iter2* dest)

Moves the elements in the range *rng* to another range beginning at *dest*. After this operation the elements in the moved-from range will still contain valid values of the appropriate type, but not necessarily the same values as before the move.

The assignments in the parallel *copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly std::distance(begin(rng), end(rng)) assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Iter2** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *move* algorithm returns a hpx::future<*ranges::move_result<iterator_t<Rng>*, FwdIter2>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::move_result<iterator_t<Rng>*, FwdIter2> otherwise. The *move* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element moved.

template<typename **Iter1**, typename **Sent1**, typename **Iter2**>

move_result<*Iter1*, *Iter2*> `hpx::ranges::move`(*Iter1* first, *Sent1* last, *Iter2* dest)

Moves the elements in the range *rng* to another range beginning at *dest*. After this operation the elements in the moved-from range will still contain valid values of the appropriate type, but not necessarily the same values as before the move.

> **Note**
>
> Complexity: Performs exactly std::distance(begin(rng), end(rng)) assignments.

Template Parameters

- **`Iter1`** – The type of the source iterators used for the first range (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Sent1`** – The type of the source iterators used for the end of the first range (deduced).
- **`Iter2`** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`dest`** – Refers to the beginning of the destination range.

Returns

The *move* algorithm returns *ranges::move_result<iterator_t<Rng>*, FwdIter2>. The *move* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element moved.

template<typename **`Rng`**, typename **`Iter2`**>

move_result<*std*::ranges::iterator_t<*Rng*>, *Iter2*> `hpx::ranges::`**`move`**(*Rng* &&rng, *Iter2* dest)

Moves the elements in the range *rng* to another range beginning at *dest*. After this operation the elements in the moved-from range will still contain valid values of the appropriate type, but not necessarily the same values as before the move.

> **Note**
>
> Complexity: Performs exactly std::distance(begin(rng), end(rng)) assignments.

Template Parameters

- **`Rng`** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **`Iter2`** – The type of the source iterators used for the second range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.

Returns

The *move* algorithm returns a *ranges::move_result<iterator_t<Rng>*, FwdIter2>. The *move* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element moved.

hpx::ranges::destroy

Defined in header hpx/algorithm.hpp[759].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::destroy_n*

template<typename **ExPolicy**, typename **Rng**>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::destroy`(*ExPolicy* &&policy, *Rng* &&rng)

Destroys objects of type typename iterator_traits<ForwardIt>::value_type in the range [first, last).

The operations in the parallel *destroy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The operations in the parallel *destroy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* operations.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

[759] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.

Returns

The *destroy* algorithm returns a *hpx::future<void>*, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *void* otherwise.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *Iter*> `hpx::ranges::destroy`(*ExPolicy* &&policy, *Iter* first, *Sent* last)

Destroys objects of type typename iterator_traits<ForwardIt>::value_type in the range [first, last).

The operations in the parallel *destroy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The operations in the parallel *destroy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* operations.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Iter** – The type of the source iterators used for the range (deduced).
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *destroy* algorithm returns a *hpx::future<void>*, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *void* otherwise.

template<typename **Rng**>

hpx::*traits*::range_iterator<*Rng*>::type `hpx::ranges::destroy`(*Rng* &&rng)

Destroys objects of type typename iterator_traits<ForwardIt>::value_type in the range [first, last).

> **Note**
>
> Complexity: Performs exactly *last* - *first* operations.

Template Parameters

`Rng` – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

Parameters

`rng` – Refers to the sequence of elements the algorithm will be applied to.

Returns

The *destroy* algorithm returns *void*.

template<typename `Iter`, typename `Sent`>

Iter `hpx::ranges::destroy`(*Iter* first, *Sent* last)

Destroys objects of type typename iterator_traits<ForwardIt>::value_type in the range [first, last).

> **Note**
>
> Complexity: Performs exactly *last* - *first* operations.

Template Parameters

- `Iter` – The type of the source iterators used for the range (deduced).
- `Sent` – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.

Parameters

- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `last` – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *destroy* algorithm returns *void*.

hpx::ranges::destroy_n

Defined in header hpx/algorithm.hpp[760].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::destroy*

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter*>::type `hpx::ranges::destroy_n`(*ExPolicy* &&policy, *FwdIter* first, *Size* count)

Destroys objects of type typename iterator_traits<ForwardIt>::value_type in the range [first, first + count).

The operations in the parallel *destroy_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The operations in the parallel *destroy_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *count* operations, if count > 0, no assignments otherwise.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply this algorithm to.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.

[760] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Returns

The *destroy_n* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *destroy_n* algorithm returns the iterator to the element in the source range, one past the last element constructed.

template<typename **FwdIter**, typename **Size**>

FwdIter `hpx::ranges::destroy_n`(*FwdIter* first, *Size* count)

Destroys objects of type typename iterator_traits<ForwardIt>::value_type in the range [first, first + count).

Note

Complexity: Performs exactly *count* operations, if count > 0, no assignments otherwise.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply this algorithm to.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.

Returns

The *destroy_n* algorithm returns the iterator to the element in the source range, one past the last element constructed.

hpx::ranges::rotate

Defined in header hpx/algorithm.hpp[761].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::rotate_copy*

template<typename **FwdIter**, typename **Sent**>

subrange_t<*FwdIter*, *Sent*> `hpx::ranges::rotate`(*FwdIter* first, *FwdIter* middle, *Sent* last)

Performs a left rotation on a range of elements. Specifically, *rotate* swaps the elements

[761] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

in the range [first, last) in such a way that the element middle becomes the first element of the new range and middle - 1 becomes the last element.

The assignments in the parallel *rotate* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*.

> **Note**
>
> The type of dereferenced *FwdIter* must meet the requirements of *MoveAssignable* and *MoveConstructible*.

Template Parameters

- **`FwdIter`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Sent`** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for FwdIter.

Parameters

- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`middle`** – Refers to the element that should appear at the beginning of the rotated range.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *rotate* algorithm returns a *subrange_t<FwdIter, Sent>*. The *rotate* algorithm returns the iterator equal to pair(first + (last - middle), last).

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, subrange_t<*FwdIter*, *Sent*>>::type `hpx::ranges::rotate`(*ExPolicy* &&policy, *FwdIter* first, *FwdIter* middle, *Sent* last)

Performs a left rotation on a range of elements. Specifically, *rotate* swaps the elements in the range [first, last) in such a way that the element middle becomes the first element of the new range and middle - 1 becomes the last element.

The assignments in the parallel *rotate* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *rotate* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Linear in the distance between *first* and *last*.

Note

The type of dereferenced *FwdIter* must meet the requirements of *MoveAssignable* and *MoveConstructible*.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`FwdIter`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Sent`** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for FwdIter.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`middle`** – Refers to the element that should appear at the beginning of the rotated range.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *rotate* algorithm returns a hpx::future<*subrange_t<FwdIter, Sent>*> if the execution policy is of type *parallel_task_policy* and returns a *subrange_t<FwdIter, Sent>* otherwise. The *rotate* algorithm returns the iterator equal to pair(first + (last - middle), last).

template<typename **Rng**>

subrange_t<*std*::ranges::iterator_t<*Rng*>, *std*::ranges::iterator_t<*Rng*>> **`hpx::ranges::rotate`**(*Rng* &&rng, *std*::ranges::iterator_t<*Rng*> middle)

Uses *rng* as the source range, as if using *util::begin(rng)* as *first* and *ranges::end(rng)* as *last*. Performs a left rotation on a range of elements. Specifically, *rotate* swaps the elements in the range [first, last) in such a way that the element middle becomes the first element of the new range and middle - 1 becomes the last element.

The assignments in the parallel *rotate* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*.

> **Note**
>
> The type of dereferenced *FwdIter* must meet the requirements of *MoveAssignable* and *MoveConstructible*.

Template Parameters

Rng – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a forward iterator.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **middle** – Refers to the element that should appear at the beginning of the rotated range.

Returns

The *rotate* algorithm returns a *subrange_t<std::ranges::iterator_t<Rng>*, std::ranges::iterator_t<Rng>>. The *rotate* algorithm returns the iterator equal to pair(first + (last - middle), last).

template<typename **ExPolicy**, typename **Rng**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, subrange_t<*std*::ranges::iterator_t<*Rng*>, *std*::ranges::iterator_t<*Rng*>>> `hpx::ranges::ro`

Uses *rng* as the source range, as if using *util::begin(rng)* as *first* and *ranges::end(rng)* as *last*. Performs a left rotation on a range of elements. Specifically, *rotate* swaps the elements in the range [first, last) in such a way that the element middle becomes the first element of the new range and middle - 1 becomes the last element.

The assignments in the parallel *rotate* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *rotate* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*.

> **Note**
>
> The type of dereferenced *FwdIter* must meet the requirements of *MoveAssignable* and *MoveConstructible*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **middle** – Refers to the element that should appear at the beginning of the rotated range.

Returns

The *rotate* algorithm returns a hpx::future <subrange_t<std::ranges::iterator_t<Rng>, std::ranges::iterator_t<Rng>>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *subrange_t<std::ranges::iterator_t<Rng>*, std::ranges::iterator_t<Rng>>. otherwise. The *rotate* algorithm returns the iterator equal to pair(first + (last - middle), last).

hpx::ranges::rotate_copy

Defined in header hpx/algorithm.hpp[762].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::rotate*

template<typename **FwdIter**, typename **Sent**, typename **OutIter**>

rotate_copy_result<*FwdIter*, *OutIter*> `hpx::ranges::rotate_copy`(*FwdIter* first, *FwdIter* middle, *Sent* last, *OutIter* dest_first)

Copies the elements from the range [first, last), to another range beginning at *dest_first* in such a way, that the element *middle* becomes the first element of the new range and *middle* - 1 becomes the last element.

[762] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

The assignments in the parallel *rotate_copy* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for FwdIter.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **middle** – Refers to the element that should appear at the beginning of the rotated range.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest_first** – Output iterator to the initial position of the range where the reversed range is stored. The pointed type shall support being assigned the value of an element in the range [first,last).

Returns

The *rotate_copy* algorithm returns a *rotate_copy_result<FwdIter, OutIter>*. The *rotate_copy* algorithm returns the output iterator to the element past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent**, typename **FwdIter2**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, rotate_copy_result<*FwdIter1*, *FwdIter2*>>::type `hpx::ranges::rotate_copy`(*ExPolicy* &&policy, *FwdIter1* first, *FwdIter1* middle, *Sent* last, *FwdIter2* dest_first)

Copies the elements from the range [first, last), to another range beginning at *dest_first* in such a way, that the element *middle* becomes the first element of the new range and *middle* - 1 becomes the last element.

The assignments in the parallel *rotate_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *rotate_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for FwdIter.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **middle** – Refers to the element that should appear at the beginning of the rotated range.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest_first** – Output iterator to the initial position of the range where the reversed range is stored. The pointed type shall support being assigned the value of an element in the range [first,last).

Returns

The *rotate_copy* algorithm returns areturns *hpx::future*< rotate_copy_result<FwdIter1, FwdIter2>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *rotate_copy_result<FwdIter1, FwdIter2>* otherwise. The *rotate_copy* algorithm returns the output iterator to the element past the last element copied.

template<typename **Rng**, typename **OutIter**>

rotate_copy_result<*std*::ranges::iterator_t<*Rng*>, *OutIter*> `hpx::ranges::rotate_copy`(*Rng* &&rng, *std*::ranges::iterator_t<*Rng*> middle, *OutIter* dest_first)

Uses *rng* as the source range, as if using *util::begin(rng)* as *first* and *ranges::end(rng)* as *last*. Copies the elements from the range [first, last), to another range beginning at *dest_first* in such a way, that the element *middle* becomes the first element of the new range and *middle* - 1 becomes the last element.

The assignments in the parallel *rotate_copy* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a forward iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **middle** – Refers to the element that should appear at the beginning of the rotated range.
- **dest_first** – Output iterator to the initial position of the range where the reversed range is stored. The pointed type shall support being assigned the value of an element in the range [first,last).

Returns

The *rotate* algorithm returns a *rotate_copy_result<std::ranges::iterator_t<Rng>*, OutIter>. The *rotate_copy* algorithm returns the output iterator to the element past the last element copied.

template<typename **ExPolicy**, typename **Rng**, typename **OutIter**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, rotate_copy_result<*std*::ranges::iterator_t<*Rng*>, *OutIter*>> `hpx::ranges::rotate_copy(`

Uses *rng* as the source range, as if using *util::begin(rng)* as *first* and *ranges::end(rng)* as *last*. Copies the elements from the range [first, last), to another range beginning at *dest_first* in such a way, that the element *new_first* becomes the first element of the new range and *new_first* - 1 becomes the last element.

The assignments in the parallel *rotate_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *rotate_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`Rng`** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a forward iterator.
- **`OutIter`** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`rng`** – Refers to the sequence of elements the algorithm will be applied to.
- **`middle`** – Refers to the element that should appear at the beginning of the rotated range.
- **`dest_first`** – Output iterator to the initial position of the range where the reversed range is stored. The pointed type shall support being assigned the value of an element in the range [first,last).

Returns

The *rotate_copy* algorithm returns a hpx::future<*otate_copy_result*< std::ranges::iterator_t<Rng>, OutIter>> if the execution policy is of type *parallel_task_policy* and returns *rotate_copy_result*< std::ranges::iterator_t<Rng>, OutIter> otherwise. The *rotate_copy* algorithm returns the output iterator to the element past the last element copied.

hpx::ranges::uninitialized_move

Defined in header hpx/algorithm.hpp[763].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::uninitialized_move_n*

[763] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

template<typename **InIter**, typename **Sent1**, typename **FwdIter**, typename **Sent2**>

hpx::*parallel*::*util*::in_out_result<*InIter*, *FwdIter*> `hpx::ranges::`**`uninitialized_move`**(*InIter* first1, *Sent1* last1, *FwdIter* first2, *Sent2* last2)

Moves the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the initialization, some objects in [first, last) are left in a valid but unspecified state.

The assignments in the parallel *uninitialized_move* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent1** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent2** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter2.

Parameters

- **first1** – Refers to the beginning of the sequence of elements that will be moved from
- **last1** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied
- **first2** – Refers to the beginning of the destination range.
- **last2** – Refers to sentinel value denoting the end of the second range the algorithm will be applied to.

Returns

The *uninitialized_move* algorithm returns an *in_out_result<InIter, FwdIter>*. The *uninitialized_move* algorithm returns an input iterator to one past the last element moved from and the output iterator to the element in the destination range, one past the last element moved.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent1**, typename **FwdIter2**, typename **Sent2**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *parallel*::*util*::in_out_result<*FwdIter1*, *FwdIter2*>>::type `hpx::ranges::uninitialized_`

Moves the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the initialization, some objects in [first, last) are left in a valid but unspecified state.

The assignments in the parallel *uninitialized_move* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_move* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent1** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent2** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter2.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements that will be moved from
- **last1** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **first2** – Refers to the beginning of the destination range.

- `last2` – Refers to sentinel value denoting the end of the second range the algorithm will be applied to.

Returns

The *uninitialized_move* algorithm returns a hpx::future<*in_out_result<InIter, FwdIter>*>, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *in_out_result<InIter, FwdIter>* otherwise. The *uninitialized_move* algorithm returns an input iterator to one past the last element moved from and the output iterator to the element in the destination range, one past the last element moved.

template<typename **Rng1**, typename **Rng2**>

hpx::*parallel*::*util*::in_out_result<typename *hpx*::*traits*::range_traits<*Rng1*>::iterator_type, typename *hpx*::*traits*::range_traits<*Rng2*>::iterator_

Moves the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the initialization, some objects in [first, last) are left in a valid but unspecified state.

The assignments in the parallel *uninitialized_move* algorithm invoked without an execution policy object will execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- `Rng1` – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- `Rng2` – The type of the destination range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.

Parameters

- `rng1` – Refers to the range from which the elements will be moved from
- `rng2` – Refers to the range to which the elements will be moved to

Returns

The *uninitialized_move* algorithm returns an *in_out_result<typename* hpx::traits::range_traits<Rng1>::iterator_type, typename hpx::traits::range_traits<Rng2>::iterator_type>. The *uninitialized_move* algorithm returns an input iterator to one past the last element moved from and the output iterator to the element in the destination range, one past the last element moved.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *hpx*::*parallel*::*util*::in_out_result<typename *hpx*::*traits*::range_traits<*Rng1*>::iterator_type, ty

Moves the elements in the range, defined by [first, last), to an uninitialized memory area beginning at *dest*. If an exception is thrown during the initialization, some objects in [first, last) are left in a valid but unspecified state.

The assignments in the parallel *uninitialized_move* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_move* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the destination range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng1** – Refers to the range from which the elements will be moved from
- **rng2** – Refers to the range to which the elements will be moved to

Returns

The *uninitialized_move* algorithm returns a hpx::future<*in_out_result<InIter, FwdIter>*>, if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *in_out_result<* typename hpx::traits::range_traits<Rng1>::iterator_type , typename hpx::traits::range_traits<Rng2>::iterator_type> otherwise. The *uninitialized_move* algorithm returns the input iterator to one past the last element moved from and the output iterator to the element in the destination range, one past the last element moved.

hpx::ranges::uninitialized_move_n

Defined in header hpx/algorithm.hpp[764].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::uninitialized_move*

template<typename **InIter**, typename **Size**, typename **FwdIter**, typename **Sent2**>

hpx::*parallel*::*util*::in_out_result<*InIter*, *FwdIter*> `hpx::ranges::uninitialized_move_n`(*InIter* first1, *Size* count, *FwdIter* first2, *Sent2* last2)

Moves the elements in the range [first, first + count), starting from first and proceeding to first + count - 1., to another range beginning at dest. If an exception is thrown during the initialization, some objects in [first, first + count) are left in a valid but unspecified state.

The assignments in the parallel *uninitialized_move_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *count* movements, if count > 0, no move operations otherwise.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent2** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.

Parameters

- **first1** – Refers to the beginning of the sequence of elements that will be moved from
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **first2** – Refers to the beginning of the destination range.
- **last2** – Refers to sentinel value denoting the end of the second range the algorithm will be applied to.

[764] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Returns

The *uninitialized_move_n* algorithm returns *in_out_result<InIter, FwdIter>*. The *uninitialized_move_n* algorithm returns the output iterator to the element in the destination range, one past the last element moved.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Size**, typename **FwdIter2**, typename **Sent2**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *parallel*::*util*::in_out_result<*FwdIter1*, *FwdIter2*>>::type `hpx::ranges::uninitialized_`

Moves the elements in the range [first, first + count), starting from first and proceeding to first + count - 1., to another range beginning at dest. If an exception is thrown during the initialization, some objects in [first, first + count) are left in a valid but unspecified state.

The assignments in the parallel *uninitialized_move_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *uninitialized_move_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *count* movements, if count > 0, no move operations otherwise.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent2** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter2.

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `first1` – Refers to the beginning of the sequence of elements that will be moved from
- `count` – Refers to the number of elements starting at *first* the algorithm will be applied to.
- `first2` – Refers to the beginning of the destination range.
- `last2` – Refers to sentinel value denoting the end of the second range the algorithm will be applied to.

Returns

The *uninitialized_move_n* algorithm returns a hpx::future<*in_out_result*<*FwdIter1, FwdIter2*>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter2* otherwise. The *uninitialized_move_n* algorithm returns the output iterator to the element in the destination range, one past the last element moved.

hpx::ranges::fill

Defined in header hpx/algorithm.hpp[765].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::fill_n*

template<typename **ExPolicy**, typename **Rng**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, typename *hpx*::*traits*::range_traits<*Rng*>::iterator_type>::type `hpx::ranges::fill`(*E*
&
ic
R
&
T
c
&

Assigns the given value to the elements in the range [first, last).

The comparisons in the parallel *fill* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *fill* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

[765] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *fill* algorithm returns a *hpx::future<void>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *difference_type* otherwise (where *difference_type* is defined by *void*.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **T** = typename *std*::iterator_traits<*Iter*>::value_type>
hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *Iter*>::type `hpx::ranges::fill`(*ExPolicy* &&policy, *Iter* first, *Sent* last, *T* const &value)

Assigns the given value to the elements in the range [first, last).

The comparisons in the parallel *fill* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *fill* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last - first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Iter** – The type of the source iterators used for the range (deduced).
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *fill* algorithm returns a *hpx::future<void>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *difference_type* otherwise (where *difference_type* is defined by *void*.

template<typename **Rng**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type>

std::ranges::iterator_t<*Rng*> `hpx::ranges::fill`(*Rng* &&rng, *T* const &value)

Assigns the given value to the elements in the range [first, last).

> **Note**
>
> Complexity: Performs exactly *last - first* assignments.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *fill* algorithm returns *void*.

template<typename **Iter**, typename **Sent**, typename **T** = typename *std*::iterator_traits<*Iter*>::value_type>

Iter `hpx::ranges::fill`(*Iter* first, *Sent* last, *T* const &value)

Assigns the given value to the elements in the range [first, last).

> **Note**
>
> Complexity: Performs exactly *last - first* assignments.

Template Parameters

- **Iter** – The type of the source iterators used for the range (deduced).
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **value** – The value to be assigned.

Returns
The *fill* algorithm returns *void*.

hpx::ranges::fill_n

Defined in header hpx/algorithm.hpp[766].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::fill*

template<typename **ExPolicy**, typename **Rng**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::fill_n`(*ExPolicy* &&policy, *Rng* &&rng, *T* const &value)

Assigns the given value value to the first count elements in the range beginning at first if count > 0. Does nothing otherwise.

The comparisons in the parallel *fill_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *fill_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Template Parameters

[766] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *fill_n* algorithm returns a *hpx::future<void>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *difference_type* otherwise (where *difference_type* is defined by *void*.

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter*>::type `hpx::ranges::fill_n`(*ExPolicy* &&policy, *FwdIter* first, *Size* count, *T* const &value)

Assigns the given value value to the first count elements in the range beginning at first if count > 0. Does nothing otherwise.

The comparisons in the parallel *fill_n* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The comparisons in the parallel *fill_n* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *count* assignments, for count > 0.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *fill_n* algorithm returns a *hpx::future<void>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *difference_type* otherwise (where *difference_type* is defined by *void*.

template<typename **Rng**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type>

hpx::*traits*::range_traits<*Rng*>::iterator_type `hpx::ranges::fill_n`(*Rng* &&rng, *T* const &value)

Assigns the given value value to the first count elements in the range beginning at first if count > 0. Does nothing otherwise.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *fill_n* algorithm returns an output iterator that compares equal to last.

template<typename **FwdIter**, typename **Size**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>

FwdIter `hpx::ranges::fill_n`(Iterator first, *Size* count, *T* const &value)

Assigns the given value value to the first count elements in the range beginning at first if count > 0. Does nothing otherwise.

> **Note**
>
> Complexity: Performs exactly *count* assignments, for count > 0.

Template Parameters

- **Iterator** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.

- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **T** – The type of the value to be assigned (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **value** – The value to be assigned.

Returns

The *fill_n* algorithm returns an output iterator that compares equal to last.

hpx::ranges::count

Defined in header hpx/algorithm.hpp[767].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::count_if*

template<typename **ExPolicy**, typename **Rng**, typename **Proj** = *hpx*::identity, typename **T** = typename *hpx*::*parallel*::traits::projected<*std*::ranges::iterator_t<*Rng*>, *Proj*>::value_type>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, typename *std*::iterator_traits<typename *hpx*::*traits*::range_traits<*Rng*>::iterator_type>::

Returns the number of elements in the range [first, last) satisfying a specific criteria. This version counts the elements that are equal to the given *value*.

The comparisons in the parallel *count* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* comparisons.

[767] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

> **Note**
>
> The comparisons in the parallel *count* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the comparisons.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **T** – The type of the value to search for (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **value** – The value to search for.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *count* algorithm returns a *hpx::future<difference_type>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *difference_type* otherwise (where *difference_type* is defined by *std::iterator_traits<FwdIter>::difference_type*. The *count* algorithm returns the number of elements satisfying the given criteria.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **Proj** = *hpx*::identity, typename **T** = typename *hpx*::*parallel*::traits::projected<*Iter*, *Proj*>::value_type>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, typename *std*::iterator_traits<*Iter*>::difference_type>::type `hpx::ranges::count`(*Ex*
&&
icy
Ite
firs
Se
las
T
co
&v
Pr
&&
=
Pr

Returns the number of elements in the range [first, last) satisfying a specific criteria. This version counts the elements that are equal to the given *value*.

The comparisons in the parallel *count* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* comparisons.

> **Note**
>
> The comparisons in the parallel *count* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Template Parameters

- `ExPolicy` – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the comparisons.
- `Iter` – The type of the source iterators used for the range (deduced).
- `Sent` – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- `T` – The type of the value to search for (deduced).
- `Proj` – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `last` – Refers to the end of the sequence of elements the algorithm will be applied to.
- `value` – The value to search for.
- `proj` – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *count* algorithm returns a *hpx::future<difference_type>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *difference_type* otherwise (where *difference_type* is defined by *std::iterator_traits<FwdIter>::difference_type*. The *count* algorithm returns the number of elements satisfying the given criteria.

template<typename **Rng**, typename **Proj** = *hpx*::identity, typename **T** = typename *hpx*::*parallel*::traits::projected<*std*::ranges::iterator_t<*Rng*>, *Proj*>::value_type>

std::iterator_traits<typename *hpx*::*traits*::range_traits<*Rng*>::iterator_type>::difference_type `hpx::ranges::count`(*Rng* &&rng, *T* const &value, *Proj* &&proj = *Proj*())

Returns the number of elements in the range [first, last) satisfying a specific criteria. This version counts the elements that are equal to the given *value*.

> **Note**
>
> Complexity: Performs exactly *last* - *first* comparisons.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **T** – The type of the value to search for (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **value** – The value to search for.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *count* algorithm returns the number of elements satisfying the given criteria.

template<typename **Iter**, typename **Sent**, typename **Proj** = *hpx*::identity, typename **T** = typename *hpx*::*parallel*::traits::projected<*Iter*, *Proj*>::value_type>

std::iterator_traits<*Iter*>::difference_type `hpx::ranges::count`(*Iter* first, *Sent* last, *T* const &value, *Proj* &&proj = *Proj*())

Returns the number of elements in the range [first, last) satisfying a specific criteria. This version counts the elements that are equal to the given *value*.

> **Note**
>
> Complexity: Performs exactly *last* - *first* comparisons.

Template Parameters

- **Iter** – The type of the source iterators used for the range (deduced).

- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **T** – The type of the value to search for (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **value** – The value to search for.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *count* algorithm returns the number of elements satisfying the given criteria.

hpx::ranges::count_if

Defined in header hpx/algorithm.hpp[768].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::count*

template<typename **ExPolicy**, typename **Rng**, typename **F**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, typename *std*::iterator_traits<typename *hpx*::*traits*::range_traits<*Rng*>::iterator_type>::

Returns the number of elements in the range [first, last) satisfying a specific criteria. This version counts elements for which predicate *f* returns true.

> **Note**
>
> Complexity: Performs exactly *last* - *first* applications of the predicate.

[768] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Note

The assignments in the parallel *count_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

Note

The assignments in the parallel *count_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the comparisons.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *count_if* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *count_if* algorithm returns *hpx::future<difference_type>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *difference_type* otherwise (where *difference_type* is defined by *std::iterator_traits<FwdIter>::difference_type*. The *count* algorithm returns the number of elements satisfying the given criteria.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **F**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, typename *std*::iterator_traits<*Iter*>::difference_type>::type `hpx::ranges::count_if`

Returns the number of elements in the range [first, last) satisfying a specific criteria. This version counts elements for which predicate *f* returns true.

Note

Complexity: Performs exactly *last* - *first* applications of the predicate.

Note

The assignments in the parallel *count_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

Note

The assignments in the parallel *count_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the comparisons.
- **Iter** – The type of the source iterators used for the range (deduced).
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *count_if* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *count_if* algorithm returns *hpx::future<difference_type>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *difference_type* otherwise (where *difference_type* is defined by *std::iterator_traits<FwdIter>::difference_type*. The *count* algorithm returns the number of elements satisfying the given criteria.

template<typename **Rng**, typename **F**, typename **Proj** = *hpx*::identity>

std::iterator_traits<typename *hpx*::*traits*::range_traits<*Rng*>::iterator_type>::difference_type `hpx::ranges::count_if`(*Rng* &&rng, *F* &&f, *Proj* &&proj = *Proj*())

Returns the number of elements in the range [first, last) satisfying a specific criteria. This version counts elements for which predicate *f* returns true.

> **Note**
>
> Complexity: Performs exactly *last* - *first* applications of the predicate.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *count_if* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *count* algorithm returns the number of elements satisfying the given criteria.

template<typename **Iter**, typename **Sent**, typename **F**, typename **Proj** = *hpx*::identity>

std::iterator_traits<*Iter*>::difference_type *hpx*::*ranges*::**count_if**(*Iter* first, *Sent* last, *F* &&f, *Proj* &&proj = *Proj*())

Returns the number of elements in the range [first, last) satisfying a specific criteria. This version counts elements for which predicate *f* returns true.

> **Note**
>
> Complexity: Performs exactly *last* - *first* applications of the predicate.

Template Parameters

- **Iter** – The type of the source iterators used for the range (deduced).
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *count_if* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns
The *count* algorithm returns the number of elements satisfying the given criteria.

hpx::ranges::reduce

Defined in header hpx/algorithm.hpp[769].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **F**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> `hpx::ranges::reduce`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *T* init, *F* &&f)

Returns GENERALIZED_SUM(f, init, *first, ..., *(first + (last - first) - 1)).

The reduce operations in the parallel *reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *f*.

> **Note**
>
> GENERALIZED_SUM(op, a1, ..., aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(op, b1, ..., bK), GENERALIZED_SUM(op, bM, ..., bN)), where:
>   - b1, ..., bN may be any permutation of a1, ..., aN and

[769] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

```
1 < K+1 = M <= N.
```

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source begin iterator used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel used (deduced). This iterator type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *copy_if* requires *F* to meet the requirements of *CopyConstructible*.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const&. The types *Type1 Ret* must be such that an object of type *FwdIterB* can be dereferenced and then implicitly converted to any of those types.
- **init** – The initial value for the generalized sum.

Returns

The *reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *T* otherwise. The *reduce* algorithm returns the result of the generalized sum over the elements given by the input range [first, last).

template<typename **ExPolicy**, typename **Rng**, typename **F**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> hpx::ranges::**reduce**(*ExPolicy* &&policy, *Rng* &&rng, *T* init, *F* &&f)

Returns GENERALIZED_SUM(f, init, *first, …, *(first + (last - first) - 1)).

The reduce operations in the parallel *reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *f*.

> **Note**
>
> GENERALIZED_SUM(op, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *copy_if* requires *F* to meet the requirements of *CopyConstructible*.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const&. The types *Type1 Ret* must be such that an object of type *FwdIterB* can be dereferenced and then implicitly converted to any of those types.

- **init** – The initial value for the generalized sum.

Returns

The *reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *T* otherwise. The *reduce* algorithm returns the result of the generalized sum over the elements given by the input range [first, last).

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> `hpx::ranges::reduce`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *T* init)

Returns GENERALIZED_SUM(+, init, *first, …, *(first + (last - first) - 1)).

The reduce operations in the parallel *reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

Note

Complexity: O(*last* - *first*) applications of the operator+().

Note

GENERALIZED_SUM(+, a1, …, aN) is defined as follows:

- a1 when N is 1
- op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
  - b1, …, bN may be any permutation of a1, …, aN and
  - 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source begin iterator used (deduced). This iterator type must meet the requirements of an forward iterator.

- `Sent` – The type of the source sentinel used (deduced). This iterator type must meet the requirements of an forward iterator.
- `T` – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `last` – Refers to the end of the sequence of elements the algorithm will be applied to.
- `init` – The initial value for the generalized sum.

Returns

The *reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *T* otherwise. The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

template<typename **ExPolicy**, typename **Rng**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type>
hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *T*> `hpx::ranges::reduce`(*ExPolicy* &&policy, *Rng* &&rng, *T* init)

Returns GENERALIZED_SUM(+, init, *first, …, *(first + (last - first) - 1)).

The reduce operations in the parallel *reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the operator+().

> **Note**
>
> GENERALIZED_SUM(+, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and

- 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **init** – The initial value for the generalized sum.

Returns

The *reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *T* otherwise. The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, typename *std*::iterator_traits<*FwdIter*>::value_type>::type `hpx::ranges::reduce`(*E*
&
ic
F
fi
Se
la

Returns GENERALIZED_SUM(+, T(), *first, …, *(first + (last - first) - 1)).

The reduce operations in the parallel *reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the operator+().

Note

The type of the initial value (and the result type) *T* is determined from the value_type of the used *FwdIterB*.

Note

GENERALIZED_SUM(+, a1, …, aN) is defined as follows:

- a1 when N is 1
- op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
  - b1, …, bN may be any permutation of a1, …, aN and
  - 1 < K+1 = M <= N.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source begin iterator used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel used (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns T otherwise (where T is the value_type of *FwdIterB*). The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

template<typename **ExPolicy**, typename **Rng**>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, typename *std*::iterator_traits<typename *hpx*::*traits*::range_traits<*Rng*>::iterator_type>::

Returns GENERALIZED_SUM(+, T(), *first, …, *(first + (last - first) - 1)).

The reduce operations in the parallel *reduce* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The reduce operations in the parallel *copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the operator+().

> **Note**
>
> The type of the initial value (and the result type) *T* is determined from the value_type of the used *FwdIterB*.

> **Note**
>
> GENERALIZED_SUM(+, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`Rng`** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`rng`** – Refers to the sequence of elements the algorithm will be applied to.

Returns

The *reduce* algorithm returns a *hpx::future<T>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns T otherwise (where T is the value_type of *FwdIterB*). The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

template<typename **FwdIter**, typename **Sent**, typename **F**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>

T `hpx::ranges::reduce`(*FwdIter* first, *Sent* last, *T* init, *F* &&f)

Returns GENERALIZED_SUM(f, init, *first, …, *(first + (last - first) - 1)).

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *f*.

> **Note**
>
> GENERALIZED_SUM(op, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **FwdIter** – The type of the source begin iterator used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel used (deduced). This iterator type must meet the requirements of an forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *copy_if* requires *F* to meet the requirements of *CopyConstructible*.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&. The types *Type1 Ret* must be such that an object of type *FwdIterB* can be dereferenced and then implicitly converted to any of those types.

- **init** – The initial value for the generalized sum.

Returns

The *reduce* algorithm returns *T*. The *reduce* algorithm returns the result of the generalized sum over the elements given by the input range [first, last).

template<typename **Rng**, typename **F**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type>

T `hpx::ranges::reduce`(*Rng* &&rng, *T* init, *F* &&f)

Returns GENERALIZED_SUM(f, init, *first, …, *(first + (last - first) - 1)).

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the predicate *f*.

> **Note**
>
> GENERALIZED_SUM(op, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(op, b1, …, bK), GENERALIZED_SUM(op, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *copy_if* requires *F* to meet the requirements of *CopyConstructible*.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type1 &b);
```

The signature does not need to have const&. The types *Type1 Ret* must be such that an object of type *FwdIterB* can be dereferenced and then implicitly converted to any of those types.

- **init** – The initial value for the generalized sum.

Returns

The *reduce* algorithm returns *T*. The *reduce* algorithm returns the result of the generalized sum over the elements given by the input range [first, last).

template<typename **FwdIter**, typename **Sent**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type>

T *hpx*::*ranges*::**reduce**(*FwdIter* first, *Sent* last, *T* init)

Returns GENERALIZED_SUM(+, init, *first, …, *(first + (last - first) - 1)).

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the operator+().

> **Note**
>
> GENERALIZED_SUM(+, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **FwdIter** – The type of the source begin iterator used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel used (deduced). This iterator type must meet the requirements of an forward iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

- **init** – The initial value for the generalized sum.

Returns

The *reduce* algorithm returns *T*. The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

template<typename **Rng**, typename **T** = typename *std*::iterator_traits<*std*::ranges::iterator_t<*Rng*>>::value_type>

T *hpx*::*ranges*::**reduce**(*Rng* &&rng, *T* init)

Returns GENERALIZED_SUM(+, init, *first, …, *(first + (last - first) - 1)).

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

Note

Complexity: O(*last* - *first*) applications of the operator+().

Note

GENERALIZED_SUM(+, a1, …, aN) is defined as follows:

- a1 when N is 1
- op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
  - b1, …, bN may be any permutation of a1, …, aN and
  - 1 < K+1 = M <= N.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **T** – The type of the value to be used as initial (and intermediate) values (deduced).

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **init** – The initial value for the generalized sum.

Returns

The *reduce* algorithm returns *T*. The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

template<typename **FwdIter**, typename **Sent**>

std::iterator_traits<*FwdIter*>::value_type `hpx::ranges::reduce`(*FwdIter* first, *Sent* last)

Returns GENERALIZED_SUM(+, T(), *first, …, *(first + (last - first) - 1)).

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the operator+().

> **Note**
>
> The type of the initial value (and the result type) *T* is determined from the value_type of the used *FwdIterB*.

> **Note**
>
> GENERALIZED_SUM(+, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

- **FwdIter** – The type of the source begin iterator used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel used (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *reduce* algorithm returns *T* (where T is the value_type of *FwdIterB*). The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

template<typename **Rng**>

std::iterator_traits<typename *hpx*::*traits*::range_traits<*Rng*>::iterator_type>::value_type `hpx::ranges::reduce`(*Rng* &&rng)

Returns GENERALIZED_SUM(+, T(), *first, …, *(first + (last - first) - 1)).

The difference between *reduce* and *accumulate* is that the behavior of reduce may be non-deterministic for non-associative or non-commutative binary predicate.

> **Note**
>
> Complexity: O(*last* - *first*) applications of the operator+().

> **Note**
>
> The type of the initial value (and the result type) *T* is determined from the value_type of the used *FwdIterB*.

> **Note**
>
> GENERALIZED_SUM(+, a1, …, aN) is defined as follows:
>
> - a1 when N is 1
> - op(GENERALIZED_SUM(+, b1, …, bK), GENERALIZED_SUM(+, bM, …, bN)), where:
>   - b1, …, bN may be any permutation of a1, …, aN and
>   - 1 < K+1 = M <= N.

Template Parameters

Rng – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

Parameters

rng – Refers to the sequence of elements the algorithm will be applied to.

Returns

The *reduce* algorithm returns *T* (where T is the value_type of *FwdIterB*). The *reduce* algorithm returns the result of the generalized sum (applying operator+()) over the elements given by the input range [first, last).

hpx::ranges::find

Defined in header hpx/algorithm.hpp[770].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::find_if*

[770] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- *hpx::ranges::find_if_not*
- *hpx::ranges::find_end*
- *hpx::ranges::find_first_of*
- *hpx::ranges::find_last*
- *hpx::ranges::find_last_if*
- *hpx::ranges::find_last_if_not*

Warning

doxygenfunction: Cannot find function "hpx::ranges::find" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::ranges::find_if

Defined in header hpx/algorithm.hpp[771].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::find*
- *hpx::ranges::find_if_not*
- *hpx::ranges::find_end*
- *hpx::ranges::find_first_of*
- *hpx::ranges::find_last*
- *hpx::ranges::find_last_if*
- *hpx::ranges::find_last_if_not*

Warning

doxygenfunction: Cannot find function "hpx::ranges::find_if" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::ranges::find_if_not

Defined in header hpx/algorithm.hpp[772].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::find*
- *hpx::ranges::find_if*
- *hpx::ranges::find_end*

[771] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp
[772] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- *hpx::ranges::find_first_of*
- *hpx::ranges::find_last*
- *hpx::ranges::find_last_if*
- *hpx::ranges::find_last_if_not*

Warning

doxygenfunction: Cannot find function "hpx::ranges::find_if_not" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::ranges::find_end

Defined in header hpx/algorithm.hpp[773].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::find*
- *hpx::ranges::find_if*
- *hpx::ranges::find_if_not*
- *hpx::ranges::find_first_of*
- *hpx::ranges::find_last*
- *hpx::ranges::find_last_if*
- *hpx::ranges::find_last_if_not*

Warning

doxygenfunction: Cannot find function "hpx::ranges::find_end" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::ranges::find_first_of

Defined in header hpx/algorithm.hpp[774].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::find*
- *hpx::ranges::find_if*
- *hpx::ranges::find_if_not*
- *hpx::ranges::find_end*
- *hpx::ranges::find_last*

[773] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp
[774] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- *hpx::ranges::find_last_if*
- *hpx::ranges::find_last_if_not*

> **Warning**
>
> doxygenfunction: Cannot find function "hpx::ranges::find_first_of" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::ranges::find_last

Defined in header hpx/algorithm.hpp[775].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::find*
- *hpx::ranges::find_if*
- *hpx::ranges::find_if_not*
- *hpx::ranges::find_end*
- *hpx::ranges::find_first_of*
- *hpx::ranges::find_last_if*
- *hpx::ranges::find_last_if_not*

> **Warning**
>
> doxygenfunction: Cannot find function "hpx::ranges::find_last" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::ranges::find_last_if

Defined in header hpx/algorithm.hpp[776].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::find*
- *hpx::ranges::find_if*
- *hpx::ranges::find_if_not*
- *hpx::ranges::find_end*
- *hpx::ranges::find_first_of*
- *hpx::ranges::find_last*
- *hpx::ranges::find_last_if_not*

[775] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp
[776] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Warning

doxygenfunction: Cannot find function "hpx::ranges::find_last_if" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::ranges::find_last_if_not

Defined in header hpx/algorithm.hpp[777].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::find*
- *hpx::ranges::find_if*
- *hpx::ranges::find_if_not*
- *hpx::ranges::find_end*
- *hpx::ranges::find_first_of*
- *hpx::ranges::find_last*
- *hpx::ranges::find_last_if*

Warning

doxygenfunction: Cannot find function "hpx::ranges::find_last_if_not" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::ranges::remove_copy

Defined in header hpx/algorithm.hpp[778].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::remove_copy_if*

Warning

doxygenfunction: Cannot find function "hpx::ranges::remove_copy" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

[777] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp
[778] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::ranges::remove_copy_if

Defined in header hpx/algorithm.hpp[779].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::remove_copy*

> **Warning**
>
> doxygenfunction: Cannot find function "hpx::ranges::remove_copy_if" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::ranges::fold_left

Defined in header hpx/algorithm.hpp[780].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::fold_left_first*
- *hpx::ranges::fold_right*
- *hpx::ranges::fold_right_last*
- *hpx::ranges::fold_left_with_iter*
- *hpx::ranges::fold_left_first_with_iter*

template<typename **InIter**, typename **Sent**, typename **T**, typename **F**>

auto `hpx::ranges::fold_left`(*InIter* first, *Sent* last, *T* init, *F* f)

Left-folds a range of elements using a binary operator.

> **Note**
>
> Complexity: Exactly N applications of the binary operator, where N = std::distance(first, last).

Template Parameters

- **InIter** – The type of the source begin iterator (deduced). This iterator must meet the requirements of an input iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **T** – The type of the initial value.

[779] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp
[780] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **F** – The type of the binary function object.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **init** – The initial value for the fold operation.
- **f** – Binary function object that will be applied.

Returns
The result of left-folding the range.

template<typename **Rng**, typename **T**, typename **F**>

auto *hpx*::*ranges*::**fold_left**(*Rng* &&rng, *T* init, *F* f)

Left-folds a range of elements using a binary operator.

> **Note**
>
> Complexity: Exactly N applications of the binary operator, where N = std::distance(begin(rng), end(rng)).

Template Parameters

- **Rng** – The type of the source range (deduced).
- **T** – The type of the initial value.
- **F** – The type of the binary function object.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **init** – The initial value for the fold operation.
- **f** – Binary function object that will be applied.

Returns
The result of left-folding the range.

hpx::ranges::fold_left_first

Defined in header hpx/algorithm.hpp[781].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::fold_left*

[781] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- *hpx::ranges::fold_right*
- *hpx::ranges::fold_right_last*
- *hpx::ranges::fold_left_with_iter*
- *hpx::ranges::fold_left_first_with_iter*

template<typename **InIter**, typename **Sent**, typename **F**>

auto *hpx*::*ranges*::**fold_left_first**(*InIter* first, *Sent* last, *F* f)

Left-folds a range using the first element as the initial value.

> **Note**
>
> Complexity: Exactly N-1 applications of the binary operator, where N = std::distance(first, last).

Template Parameters

- **InIter** – The type of the source begin iterator (deduced).
- **Sent** – The type of the source sentinel (deduced).
- **F** – The type of the binary function object.

Parameters

- **first** – Refers to the beginning of the sequence.
- **last** – Refers to the end of the sequence.
- **f** – Binary function object that will be applied.

Returns

An optional containing the fold result, or std::nullopt if the range is empty.

template<typename **Rng**, typename **F**>

auto *hpx*::*ranges*::**fold_left_first**(*Rng* &&rng, *F* f)

Left-folds a range using the first element as the initial value.

Template Parameters

- **Rng** – The type of the source range (deduced).
- **F** – The type of the binary function object.

Parameters

- **rng** – Refers to the sequence of elements.
- **f** – Binary function object that will be applied.

Returns

An optional containing the fold result, or std::nullopt if the range is empty.

hpx::ranges::fold_right

Defined in header hpx/algorithm.hpp[782].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::fold_left*
- *hpx::ranges::fold_left_first*
- *hpx::ranges::fold_right_last*
- *hpx::ranges::fold_left_with_iter*
- *hpx::ranges::fold_left_first_with_iter*

template<typename **BidIter**, typename **Sent**, typename **T**, typename **F**>

auto *hpx*::*ranges*::**fold_right**(*BidIter* first, *Sent* last, *T* init, *F* f)

Right-folds a range of elements using a binary operator.

> **Note**
>
> Complexity: Exactly N applications of the binary operator.

Template Parameters

- **BidIter** – The type of the source begin iterator (deduced). This iterator must meet the requirements of a bidirectional iterator.
- **Sent** – The type of the source sentinel (deduced).
- **T** – The type of the initial value.
- **F** – The type of the binary function object.

Parameters

- **first** – Refers to the beginning of the sequence.
- **last** – Refers to the end of the sequence.
- **init** – The initial value for the fold operation.
- **f** – Binary function object that will be applied.

Returns

The result of right-folding the range.

template<typename **Rng**, typename **T**, typename **F**>

auto *hpx*::*ranges*::**fold_right**(*Rng* &&rng, *T* init, *F* f)

Right-folds a range of elements using a binary operator.

[782] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Template Parameters

- **Rng** – The type of the source range (deduced).
- **T** – The type of the initial value.
- **F** – The type of the binary function object.

Parameters

- **rng** – Refers to the sequence of elements.
- **init** – The initial value for the fold operation.
- **f** – Binary function object that will be applied.

Returns

The result of right-folding the range.

hpx::ranges::fold_right_last

Defined in header hpx/algorithm.hpp[783].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::fold_left*
- *hpx::ranges::fold_left_first*
- *hpx::ranges::fold_right*
- *hpx::ranges::fold_left_with_iter*
- *hpx::ranges::fold_left_first_with_iter*

template<typename **BidIter**, typename **Sent**, typename **F**>

auto *hpx::ranges::***fold_right_last**(*BidIter* first, *Sent* last, *F* f)

Right-folds a range using the last element as the initial value.

Template Parameters

- **BidIter** – The type of the source begin iterator (deduced).
- **Sent** – The type of the source sentinel (deduced).
- **F** – The type of the binary function object.

Parameters

- **first** – Refers to the beginning of the sequence.
- **last** – Refers to the end of the sequence.
- **f** – Binary function object that will be applied.

[783] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Returns

An optional containing the fold result, or std::nullopt if the range is empty.

template<typename **Rng**, typename **F**>

auto *hpx*::*ranges*::**fold_right_last**(*Rng* &&rng, *F* f)

Right-folds a range using the last element as the initial value.

Template Parameters

- **Rng** – The type of the source range (deduced).
- **F** – The type of the binary function object.

Parameters

- **rng** – Refers to the sequence of elements.
- **f** – Binary function object that will be applied.

Returns

An optional containing the fold result, or std::nullopt if the range is empty.

hpx::ranges::fold_left_with_iter

Defined in header hpx/algorithm.hpp[784].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::fold_left*
- *hpx::ranges::fold_left_first*
- *hpx::ranges::fold_right*
- *hpx::ranges::fold_right_last*
- *hpx::ranges::fold_left_first_with_iter*

template<typename **InIter**, typename **Sent**, typename **T**, typename **F**>

auto *hpx*::*ranges*::**fold_left_with_iter**(*InIter* first, *Sent* last, *T* init, *F* f) -> fold_left_with_iter_result<*InIter*, *std*::decay_t<*hpx*::*util*::invoke_result_t<*F*&, *T*, *hpx*::*traits*::iter_reference_t<*InIter*>>>>

Left-folds a range and returns both the result and the end iterator.

Template Parameters

- **InIter** – The type of the source begin iterator (deduced).
- **Sent** – The type of the source sentinel (deduced).
- **T** – The type of the initial value.

784 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **F** – The type of the binary function object.

Parameters

- **first** – Refers to the beginning of the sequence.
- **last** – Refers to the end of the sequence.
- **init** – The initial value for the fold operation.
- **f** – Binary function object that will be applied.

Returns
A fold_left_with_iter_result containing the end iterator and the fold result.

template<typename **Rng**, typename **T**, typename **F**>

auto *hpx*::*ranges*::**fold_left_with_iter**(*Rng* &&rng, *T* init, *F* f) -> fold_left_with_iter_result<*std*::ranges::iterator_t<*Rng*>, *std*::decay_t<*hpx*::*util*::invoke_result_t<*F*&, *T*, *hpx*::*traits*::range_reference_t<*Rng*>>>>

Left-folds a range and returns both the result and the end iterator.

Template Parameters

- **Rng** – The type of the source range (deduced).
- **T** – The type of the initial value.
- **F** – The type of the binary function object.

Parameters

- **rng** – Refers to the sequence of elements.
- **init** – The initial value for the fold operation.
- **f** – Binary function object that will be applied.

Returns
A fold_left_with_iter_result containing the end iterator and the fold result.

hpx::ranges::fold_left_first_with_iter

Defined in header hpx/algorithm.hpp[785].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::fold_left*
- *hpx::ranges::fold_left_first*
- *hpx::ranges::fold_right*
- *hpx::ranges::fold_right_last*
- *hpx::ranges::fold_left_with_iter*

[785] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

template<typename **InIter**, typename **Sent**, typename **F**>

auto *hpx*::*ranges*::**fold_left_first_with_iter**(*InIter* first, *Sent* last, *F* f) -> fold_left_first_with_iter_result<*InIter*, *hpx*::optional<*std*::decay_t<*hpx*::*util*::invoke_result_t<*F*&, *hpx*::*traits*::iter_reference_t<*InIter*>, *hpx*::*traits*::iter_reference_t<*InIter*>>>>>

Left-folds a range using the first element and returns both an optional result and the end iterator.

Template Parameters

- **InIter** – The type of the source begin iterator (deduced).
- **Sent** – The type of the source sentinel (deduced).
- **F** – The type of the binary function object.

Parameters

- **first** – Refers to the beginning of the sequence.
- **last** – Refers to the end of the sequence.
- **f** – Binary function object that will be applied.

Returns

A fold_left_first_with_iter_result containing the end iterator and an optional with the fold result.

template<typename **Rng**, typename **F**>

auto *hpx*::*ranges*::**fold_left_first_with_iter**(*Rng* &&rng, *F* f) -> fold_left_first_with_iter_result<*std*::ranges::iterator_t<*Rng*>, *hpx*::optional<*std*::decay_t<*hpx*::*util*::invoke_result_t<*F*&, *hpx*::*traits*::range_reference_t<*Rng*>, *hpx*::*traits*::range_reference_t<*Rng*>>>>>

Left-folds a range using the first element and returns both an optional result and the end iterator.

Template Parameters

- **Rng** – The type of the source range (deduced).
- **F** – The type of the binary function object.

Parameters

- **rng** – Refers to the sequence of elements.
- **f** – Binary function object that will be applied.

Returns

A fold_left_first_with_iter_result containing the end iterator and an optional with the fold result.

hpx::ranges::set_difference

Defined in header hpx/algorithm.hpp[786].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, set_difference_result<*Iter1*, *Iter3*>>::type `hpx::ranges::set_difference`(*ExPolicy* &&policy, *Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Iter3* dest, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Constructs a sorted range beginning at dest consisting of all elements present in the range [first1, last1) and not present in the range [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

Equivalent elements are treated individually, that is, if some element is found *m* times in [first1, last1) and *n* times in [first2, last2), it will be copied to *dest* exactly std::max(m-n, 0) times. The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

The application of function objects in parallel algorithm invoked with a sequential execution policy object execute in sequential order in the calling thread (*sequenced_policy*) or in a single new thread spawned from the current thread (for *sequenced_task_policy*).

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an

[786] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Iter1** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of an forward iterator.
- **Sent1** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Iter2** – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an forward iterator.
- **Sent2** – The type of the end source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an sentinel for Iter2.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_difference* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*

- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_difference* algorithm returns a hpx::future<*ranges::set_difference_result<Iter1, Iter3>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::set_difference_result<Iter1, Iter3>* otherwise. The *set_difference* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, set_difference_result<*std*::ranges::iterator_t<*Rng1*>, *Iter3*>> **hpx::ranges::set_dif**

Constructs a sorted range beginning at dest consisting of all elements present in the range [first1, last1) and not present in the range [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

Equivalent elements are treated individually, that is, if some element is found *m* times in [first1, last1) and *n* times in [first2, last2), it will be copied to *dest* exactly std::max(m-n, 0) times. The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

The application of function objects in parallel algorithm invoked with a sequential execution policy object execute in sequential order in the calling thread (*sequenced_policy*) or in a single new thread spawned from the current thread (for *sequenced_task_policy*).

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **`Rng1`** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **`Rng2`** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **`Iter3`** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **`Pred`** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_difference* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **`Proj1`** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **`Proj2`** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`rng1`** – Refers to the first sequence of elements the algorithm will be applied to.
- **`rng2`** – Refers to the second sequence of elements the algorithm will be applied to.
- **`dest`** – Refers to the beginning of the destination range.
- **`op`** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*

- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_difference* algorithm returns a hpx::future<*ranges::set_difference_result<Iter1, Iter3>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::set_difference_result<Iter1, Iter3>* otherwise. where Iter1 is range_iterator_t<Rng1> and Iter2 is range_iterator_t<Rng2> The *set_difference* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

set_difference_result<*Iter1*, *Iter3*> `hpx::ranges::set_difference`(*Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Iter3* dest, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Constructs a sorted range beginning at dest consisting of all elements present in the range [first1, last1) and not present in the range [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

Equivalent elements are treated individually, that is, if some element is found *m* times in [first1, last1) and *n* times in [first2, last2), it will be copied to *dest* exactly std::max(m-n, 0) times. The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **Iter1** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of an forward iterator.
- **Sent1** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Iter2** – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an forward iterator.
- **Sent2** – The type of the end source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an sentinel for Iter2.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.

- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_difference* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_difference* algorithm returns *ranges::set_difference_result<Iter1, Iter3>*. The *set_difference* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng1**, typename **Rng2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

set_difference_result<*std*::ranges::iterator_t<*Rng1*>, *Iter3*> **hpx::ranges::set_difference**(*Rng1* &&rng1, *Rng2* &&rng2, *Iter3* dest, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Constructs a sorted range beginning at dest consisting of all elements present in the range [first1, last1) and not present in the range [first2, last2). This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

Equivalent elements are treated individually, that is, if some element is found *m* times in [first1, last1) and *n* times in [first2, last2), it will be copied to *dest* exactly std::max(m-n, 0) times. The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_difference* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **rng1** – Refers to the first sequence of elements the algorithm will be applied to.
- **rng2** – Refers to the second sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.

- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_difference* algorithm returns *ranges::set_difference_result<Iter1, Iter3>*. where Iter1 is range_iterator_t<Rng1> and Iter2 is range_iterator_t<Rng2> The *set_difference* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::ranges::nth_element

Defined in header hpx/algorithm.hpp[787].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **RandomIt**, typename **Sent**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

RandomIt `hpx::ranges::nth_element`(*RandomIt* first, *RandomIt* nth, *Sent* last, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

nth_element is a partial sorting algorithm that rearranges elements in [first, last) such that the element pointed at by nth is changed to whatever element would occur in that position if [first, last) were sorted and all of the elements before this new nth element are less than or equal to the elements after the new nth element.

The comparison operations in the parallel *nth_element* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear in std::distance(first, last) on average. O(N) applications of the predicate, and O(N log N) swaps, where N = last - first.

Template Parameters

- **RandomIt** – The type of the source begin, nth, and end iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for RandomIt.
- **Pred** – Comparison function object which returns true if the first argument is less than the second.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.

[787] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **nth** – Refers to the iterator defining the sort partition point
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **pred** – Specifies the comparison function object which returns true if the first argument is less than (i.e. is ordered before) the second. The signature of this comparison function should be equivalent to:

  ```
  bool cmp(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type must be such that an object of type *randomIt* can be dereferenced and then implicitly converted to Type. This defaults to std::less<>.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked. This defaults to hpx::identity.

Returns

The *nth_element* algorithm returns returns *RandomIt*. The *nth_element* algorithm returns an iterator equal to last.

template<typename **ExPolicy**, typename **RandomIt**, typename **Sent**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *RandomIt*> `hpx::ranges::nth_element`(*ExPolicy* &&policy, *RandomIt* first, *RandomIt* nth, *Sent* last, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

nth_element is a partial sorting algorithm that rearranges elements in [first, last) such that the element pointed at by nth is changed to whatever element would occur in that position if [first, last) were sorted and all of the elements before this new nth element are less than or equal to the elements after the new nth element.

The comparison operations in the parallel *nth_element* invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *nth_element* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Linear in std::distance(first, last) on average. O(N) applications of the predicate, and O(N log N) swaps, where N = last - first.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.

- **RandomIt** – The type of the source begin, nth, and end iterators used (deduced). This iterator type must meet the requirements of a random access iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for RandomIt.
- **Pred** – Comparison function object which returns true if the first argument is less than the second.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **nth** – Refers to the iterator defining the sort partition point
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **pred** – Specifies the comparison function object which returns true if the first argument is less than (i.e. is ordered before) the second. The signature of this comparison function should be equivalent to:

```
bool cmp(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type must be such that an object of type *randomIt* can be dereferenced and then implicitly converted to Type. This defaults to std::less<>.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked. This defaults to hpx::identity.

Returns

The *partition* algorithm returns a *hpx::future<RandomIt>* if the execution policy is of type *parallel_task_policy* and returns *RandomIt* otherwise. The *nth_element* algorithm returns an iterator equal to last.

template<typename **Rng**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

std::ranges::iterator_t<*Rng*> `hpx::ranges::nth_element`(*Rng* &&rng, *std*::ranges::iterator_t<*Rng*> nth, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

nth_element is a partial sorting algorithm that rearranges elements in [first, last) such that the element pointed at by nth is changed to whatever element would occur in that position if [first, last) were sorted and all of the elements before this new nth element are less than or equal to the elements after the new nth element.

The comparison operations in the parallel *nth_element* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear in std::distance(first, last) on average. O(N) applications of the predicate, and O(N log N) swaps, where N = last - first.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an random access iterator.
- **Pred** – Comparison function object which returns true if the first argument is less than the second.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **nth** – Refers to the iterator defining the sort partition point
- **pred** – Specifies the comparison function object which returns true if the first argument is less than (i.e. is ordered before) the second. The signature of this comparison function should be equivalent to:

  ```
  bool cmp(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type must be such that an object of type *randomIt* can be dereferenced and then implicitly converted to Type. This defaults to std::less<>.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked. This defaults to hpx::identity.

Returns

The *nth_element* algorithm returns returns *std::ranges::iterator_t<Rng>*. The *nth_element* algorithm returns an iterator equal to last.

template<typename **ExPolicy**, typename **Rng**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result_t<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::nth_element`(*ExPolicy* &&policy, *Rng* &&rng, *std*::ranges::iterator_t<*Rng*> nth, *Pred* &&pred = *Pred*(), *Proj* &&proj = *Proj*())

nth_element is a partial sorting algorithm that rearranges elements in [first, last) such that the element pointed at by nth is changed to whatever element would occur in that position if [first, last) were sorted and all of the elements before this new nth element are less than or equal to the elements after the new nth element.

The comparison operations in the parallel *nth_element* invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *nth_element* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Linear in std::distance(first, last) on average. O(N) applications of the predicate, and O(N log N) swaps, where N = last - first.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an random access iterator.
- **Pred** – Comparison function object which returns true if the first argument is less than the second.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **nth** – Refers to the iterator defining the sort partition point

- **pred** – Specifies the comparison function object which returns true if the first argument is less than (i.e. is ordered before) the second. The signature of this comparison function should be equivalent to:

  ```
  bool cmp(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type must be such that an object of type *randomIt* can be dereferenced and then implicitly converted to Type. This defaults to std::less<>.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked. This defaults to hpx::identity.

Returns

The *partition* algorithm returns a hpx::future<*std::ranges::iterator_t*<*Rng*>> if the execution policy is of type *parallel_task_policy* and returns *std::ranges::iterator_t*<*Rng*> otherwise. The *nth_element* algorithm returns an iterator equal to last.

hpx::ranges::remove

Defined in header hpx/algorithm.hpp[788].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::remove_if*

template<typename **Iter**, typename **Sent**, typename **Proj** = *hpx*::identity, typename **T** = typename *hpx*::*parallel*::traits::projected<*Iter*, *Proj*>::value_type>

subrange_t<*Iter*, *Sent*> `hpx::ranges::remove`(*Iter* first, *Sent* last, *T* const &value, *Proj* &&proj = *Proj*())

Removes all elements that are equal to *value* from the range [first, last) and and returns a subrange [ret, last), where ret is a past-the-end iterator for the new end of the range.

The assignments in the parallel *remove* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the operator==() and the projection *proj*.

Template Parameters

- **Iter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for FwdIter.

[788] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **T** – The type of the value to remove (deduced). This value type must meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **value** – Specifies the value of elements to remove.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *remove* algorithm returns a *subrange_t<FwdIter, Sent>*. The *remove* algorithm returns an object {ret, last}, where ret is a past-the-end iterator for a new subrange of the values all in valid but unspecified state.

template<typename **Rng**, typename **Proj** = *hpx*::identity, typename **T** = typename *hpx*::*parallel*::traits::projected<*std*::ranges::iterator_t<*Rng*>, *Proj*>::value_type>

subrange_t<*std*::ranges::iterator_t<*Rng*>> `hpx::ranges::remove`(*Rng* &&rng, *T* const &value, *Proj* &&proj = *Proj*())

Removes all elements that are equal to *value* from the range *rng* and and returns a subrange [ret, util::end(rng)), where ret is a past-the-end iterator for the new end of the range.

The assignments in the parallel *remove* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs not more than *util::end(rng)*
>
> - *util::begin(rng)* assignments, exactly *util::end(rng) - util::begin(rng)* applications of the operator==() and the projection *proj*.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **T** – The type of the value to remove (deduced). This value type must meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **value** – Specifies the value of elements to remove.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *remove* algorithm returns a *subrange_t<std::ranges::iterator_t<Rng>>*. The *remove* algorithm returns an object {ret, last}, where ret is a past-the-end iterator for a new subrange of the values all in valid but unspecified state.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **Proj** = *hpx*::identity, typename **T** = typename *hpx*::*parallel*::traits::projected<*FwdIter*, *Proj*>::value_type>

parallel::*util*::detail::algorithm_result<*ExPolicy*, subrange_t<*FwdIter*, *Sent*>>::type `hpx::ranges::remove`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *T* const &value, *Proj* &&proj = *Proj*())

Removes all elements that are equal to *value* from the range [first, last) and and returns a subrange [ret, last), where ret is a past-the-end iterator for the new end of the range.

The assignments in the parallel *remove* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *remove* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than *last - first* assignments, exactly *last - first* applications of the operator==() and the projection *proj*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for FwdIter.
- **T** – The type of the value to remove (deduced). This value type must meet the requirements of *CopyConstructible*.

- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **value** – Specifies the value of elements to remove.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *remove* algorithm returns a hpx::future<*subrange_t<FwdIter, Sent*>>. The *remove* algorithm returns an object {ret, last}, where ret is a past-the-end iterator for a new subrange of the values all in valid but unspecified state.

template<typename **ExPolicy**, typename **Rng**, typename **Proj** = *hpx*::identity, typename **T** = typename *hpx*::*parallel*::traits::projected<*std*::ranges::iterator_t<*Rng*>, *Proj*>::value_type>
parallel::*util*::detail::algorithm_result<*ExPolicy*, subrange_t<*std*::ranges::iterator_t<*Rng*>>> `hpx::ranges::remove`(*ExPolicy* &&policy, *Rng* &&rng, *T* const &value, *Proj* &&proj = *Proj*())

Removes all elements that are equal to *value* from the range *rng* and and returns a subrange [ret, util::end(rng)), where ret is a past-the-end iterator for the new end of the range.

The assignments in the parallel *remove* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *remove* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than *util::end(rng)*
>
> - *util::begin(rng)* assignments, exactly *util::end(rng) - util::begin(rng)* applications of the operator==() and the projection *proj*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **T** – The type of the value to remove (deduced). This value type must meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **value** – Specifies the value of elements to remove.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *remove* algorithm returns a hpx::future< subrange_t<std::ranges::iterator_t<Rng>>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *remove* algorithm returns the iterator to the new end of the range.

hpx::ranges::remove_if

Defined in header hpx/algorithm.hpp[789].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::remove*

template<typename **Iter**, typename **Sent**, typename **Pred**, typename **Proj** = *hpx*::identity>

subrange_t<*Iter*, *Sent*> `hpx::ranges::remove_if`(*Iter* first, *Sent* sent, *Pred* &&pred, *Proj* &&proj = *Proj*())

Removes all elements for which predicate *pred* returns true from the range [first, last) and returns a subrange [ret, last), where ret is a past-the-end iterator for the new end of the range.

The assignments in the parallel *remove_if* algorithm execute in sequential order in the calling thread.

[789] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Note

Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the predicate *pred* and the projection *proj*.

Template Parameters

- **Iter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for FwdIter.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *remove_if* requires *Pred* to meet the requirements of *CopyConstructible*..
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **sent** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *remove_if* algorithm returns a *subrange_t<FwdIter, Sent>*. The *remove_if* algorithm returns an object {ret, last}, where ret is a past-the-end iterator for a new subrange of the values all in valid but unspecified state.

template<typename **Rng**, typename **Pred**, typename **Proj** = *hpx*::identity>

subrange_t<*std*::ranges::iterator_t<*Rng*>> *hpx*::*ranges*::**remove_if**(*Rng* &&rng, *Pred* &&pred, *Proj* &&proj = *Proj*())

Removes all elements that are equal to *value* from the range *rng* and and returns a subrange [ret, util::end(rng)), where ret is a past-the-end iterator for the new end of the range.

The assignments in the parallel *remove_if* algorithm execute in sequential order in the calling thread.

Note

Complexity: Performs not more than *util::end(rng)*

- *util::begin(rng)* assignments, exactly *util::end(rng) - util::begin(rng)* applications of the operator==() and the projection *proj*.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *remove_if* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *remove_if* algorithm returns a *subrange_t<std::ranges::iterator_t<Rng>>*. The *remove_if* algorithm returns an object {ret, last}, where ret is a past-the-end iterator for a new subrange of the values all in valid but unspecified state.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **Pred**, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, subrange_t<*FwdIter*, *Sent*>>::type **hpx::ranges::remove_if**(*ExPolicy* &&policy, *FwdIter* first, *Sent* sent, *Pred* &&pred, *Proj* &&proj = *Proj*())

Removes all elements for which predicate *pred* returns true from the range [first, last) and returns a subrange [ret, last), where ret is a past-the-end iterator for the new end of the range.

The assignments in the parallel *remove_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *remove_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the predicate *pred* and the projection *proj*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used for the This iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for FwdIter.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *remove_if* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **sent** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *remove_if* algorithm returns a hpx::future<*subrange_t*<*FwdIter, Sent*>>. The *remove_if* algorithm returns an object {ret, last}, where ret is a past-the-end iterator for a new subrange of the values all in valid but unspecified state.

template<typename **ExPolicy**, typename **Rng**, typename **Pred**, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, subrange_t<*std*::ranges::iterator_t<*Rng*>>> `hpx::ranges::`**`remove_if`**(*ExPolicy* &&policy, *Rng* &&rng, *Pred* &&pred, *Proj* &&proj = *Proj*())

Removes all elements that are equal to *value* from the range *rng* and and returns a subrange [ret, util::end(rng)), where ret is a past-the-end iterator for the new end of the range.

The assignments in the parallel *remove_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *remove_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than *util::end(rng)*
>
> - *util::begin(rng)* assignments, exactly *util::end(rng) - util::begin(rng)* applications of the operator==() and the projection *proj*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *remove_if* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the required elements. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *remove_if* algorithm returns a hpx::future<*subrange_t*< std::ranges::iterator_t<Rng>>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. The *remove_if* algorithm returns an object {ret, last}, where ret is a past-the-end iterator for a new subrange of the values all in valid but unspecified state.

hpx::ranges::reverse

Defined in header hpx/algorithm.hpp[790].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::reverse_copy*

template<typename **Iter**, typename **Sent**>

Iter `hpx::ranges::reverse`(*Iter* first, *Sent* sent)

Reverses the order of the elements in the range [first, last). Behaves as if applying std::iter_swap to every pair of iterators first+i, (last-i) - 1 for each non-negative i < (last-first)/2.

The assignments in the parallel *reverse* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*.

Template Parameters

[790] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **Iter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for Iter.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **sent** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *reverse* algorithm returns a *Iter*. It returns *last*.

template<typename **Rng**>

std::ranges::iterator_t<*Rng*> `hpx::ranges::`**`reverse`**(*Rng* &&rng)

Uses *rng* as the source range, as if using *util::begin(rng)* as *first* and *ranges::end(rng)* as *last*. Reverses the order of the elements in the range [first, last). Behaves as if applying std::iter_swap to every pair of iterators first+i, (last-i) - 1 for each non-negative i < (last-first)/2.

The assignments in the parallel *reverse* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*.

Template Parameters

Rng – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a bidirectional iterator.

Parameters

rng – Refers to the sequence of elements the algorithm will be applied to.

Returns

The *reverse* algorithm returns a *hpx::traits::range_iterator<Rng>::type*. It returns *last*.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *Iter*> `hpx::ranges::`**`reverse`**(*ExPolicy* &&policy, *Iter* first, *Sent* sent)

Reverses the order of the elements in the range [first, last). Behaves as if applying std::iter_swap to every pair of iterators first+i, (last-i) - 1 for each non-negative i < (last-first)/2.

The assignments in the parallel *reverse* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *reverse* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Linear in the distance between *first* and *last*.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`Iter`** – The type of the source iterator used (deduced). The iterator type must meet the requirements of an input iterator.
- **`Sent`** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for Iter.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`sent`** – Refers to the end of the sequence of elements the algorithm will be applied to.

Returns

The *reverse* algorithm returns a *hpx::future<Iter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *Iter* otherwise. It returns *last*.

template<typename **ExPolicy**, typename **Rng**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::`**`reverse`**(*ExPolicy* &&policy, *Rng* &&rng)

Uses *rng* as the source range, as if using *util::begin(rng)* as *first* and *ranges::end(rng)* as *last*. Reverses the order of the elements in the range [first, last). Behaves as if applying std::iter_swap to every pair of iterators first+i, (last-i) - 1 for each non-negative i < (last-first)/2.

The assignments in the parallel *reverse* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *reverse* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Linear in the distance between *first* and *last*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a bidirectional iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.

Returns

The *reverse* algorithm returns a hpx::future<*std::ranges::iterator_t<Rng>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns hpx::future< std::ranges::iterator_t<Rng>> otherwise. It returns *last*.

hpx::ranges::reverse_copy

Defined in header hpx/algorithm.hpp[791].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::reverse*

template<typename **Iter**, typename **Sent**, typename **OutIter**>

reverse_copy_result<*Iter*, *OutIter*> `hpx::ranges::reverse_copy`(*Iter* first, *Sent* last, *OutIter* result)

Copies the elements from the range [first, last) to another range beginning at result in such a way that the elements in the new range are in reverse order. Behaves as if by executing the assignment *(result + (last - first) - 1 - i) = *(first + i) once for each non-negative i < (last - first) If the source and destination ranges (that is, [first, last) and [result, result+(last-first)) respectively) overlap, the behavior is undefined.

The assignments in the parallel *reverse_copy* algorithm execute in sequential order in the calling thread.

Note

Complexity: Performs exactly *last* - *first* assignments.

[791] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Template Parameters

- **Iter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for Iter.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **result** – Refers to the begin of the destination range.

Returns

The *reverse_copy* algorithm returns a *reverse_copy_result<Iter, OutIter>*. The *reverse_copy* algorithm returns the pair of the input iterator forwarded to the first element after the last in the input sequence and the output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng**, typename **OutIter**>

reverse_copy_result<*std*::ranges::iterator_t<*Rng*>, *OutIter*> `hpx::ranges::reverse_copy`(*Rng* &&rng, *OutIter* result)

Uses *rng* as the source range, as if using *util::begin(rng)* as *first* and *ranges::end(rng)* as *last*. Copies the elements from the range [first, last) to another range beginning at result in such a way that the elements in the new range are in reverse order. Behaves as if by executing the assignment *(result + (last - first) - 1 - i) = *(first + i) once for each non-negative i < (last - first) If the source and destination ranges (that is, [first, last) and [result, result+(last-first)) respectively) overlap, the behavior is undefined.

The assignments in the parallel *reverse_copy* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a bidirectional iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **result** – Refers to the begin of the destination range.

Returns

The *reverse_copy* algorithm returns a *ranges::reverse_copy_result*< std::ranges::iterator_t<Rng>, OutIter>. The *reverse_copy* algorithm returns an object equal to {last, result + N} where N = last - first

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **FwdIter**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, reverse_copy_result<*Iter*, *FwdIter*>>::type `hpx::ranges::reverse_copy`(*ExPolicy* &&policy, *Iter* first, *Sent* last, *FwdIter* result)

Copies the elements from the range [first, last) to another range beginning at result in such a way that the elements in the new range are in reverse order. Behaves as if by executing the assignment *(result + (last - first) - 1 - i) = *(first + i) once for each non-negative i < (last - first) If the source and destination ranges (that is, [first, last) and [result, result+(last-first)) respectively) overlap, the behavior is undefined.

The assignments in the parallel *reverse_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *reverse_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last - first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Iter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for Iter.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **result** – Refers to the begin of the destination range.

Returns

The *reverse_copy* algorithm returns a hpx::future<*reverse_copy_result<Iter, FwdIter>* > if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *reverse_copy_result<Iter, FwdIter>* otherwise. The *reverse_copy* algorithm returns the pair of the input iterator forwarded to the first element after the last in the input sequence and the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng**, typename **OutIter**>

parallel::*util*::detail::algorithm_result<*ExPolicy*, reverse_copy_result<*std*::ranges::iterator_t<*Rng*>, *OutIter*>>::type `hpx::ranges::revers`

Uses *rng* as the source range, as if using *util::begin(rng)* as *first* and *ranges::end(rng)* as *last*. Copies the elements from the range [first, last) to another range beginning at result in such a way that the elements in the new range are in reverse order. Behaves as if by executing the assignment *(result + (last - first) - 1 - i) = *(first + i) once for each non-negative i < (last - first) If the source and destination ranges (that is, [first, last) and [result, result+(last-first)) respectively) overlap, the behavior is undefined.

The assignments in the parallel *reverse_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *reverse_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a bidirectional iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **result** – Refers to the begin of the destination range.

Returns

The *reverse_copy* algorithm returns a hpx::future<*ranges::reverse_copy_result*< std::ranges::iterator_t<Rng>, OutIter>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::reverse_copy_result*< std::ranges::iterator_t<Rng>, OutIter> otherwise. The *reverse_copy* algorithm returns an object equal to {last, result + N} where N = last - first

hpx::ranges::replace

Defined in header hpx/algorithm.hpp[792].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::replace_if*
- *hpx::ranges::replace_copy*
- *hpx::ranges::replace_copy_if*

template<typename **Iter**, typename **Sent**, typename **Proj** = *hpx*::identity, typename **T1** = typename *hpx*::*parallel*::traits::projected<*Iter*, *Proj*>::value_type, typename **T2** = *T1*>

Iter `hpx::ranges::replace`(*Iter* first, *Sent* sent, *T1* const &old_value, *T2* const &new_value, *Proj* &&proj = *Proj*())

Replaces all elements satisfying specific criteria with *new_value* in the range [first, last).

Effects: Substitutes elements referred by the iterator it in the range [first,last) with new_value, when the following corresponding conditions hold: INVOKE(proj, *i) == old_value

The assignments in the parallel *replace* algorithm execute in sequential order in the calling thread.

[792] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **Iter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for Iter.
- **T1** – The type of the old value to replace (deduced).
- **T2** – The type of the new values to replace (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **sent** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **old_value** – Refers to the old value of the elements to replace.
- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace* algorithm returns an *Iter*.

```
template<typename Rng, typename Proj = hpx::identity, typename T1 = typename hpx::parallel::traits::projected<std::ranges::iterator_t<Rng>, Proj>::value_type, typename T2 = T1>
std::ranges::iterator_t<Rng> hpx::ranges::replace(Rng &&rng, T1 const &old_value, T2 const &new_value, Proj &&proj = Proj())
```

Replaces all elements satisfying specific criteria with *new_value* in the range *rng*.

Effects: Substitutes elements referred by the iterator it in the range rng with new_value, when the following corresponding conditions hold: INVOKE(proj, *i) == old_value

The assignments in the parallel *replace* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *util::end(rng)* - *util::begin(rng)* assignments.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a forward iterator.
- **T1** – The type of the old value to replace (deduced).
- **T2** – The type of the new values to replace (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **old_value** – Refers to the old value of the elements to replace.
- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace* algorithm returns an *hpx::traits::range_iterator<Rng>::type*.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **Proj** = *hpx*::identity, typename **T1** = typename *hpx*::*parallel*::traits::projected<*Iter*, *Proj*>::value_type, typename **T2** = *T1*>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *Iter*> `hpx::ranges::replace`(*ExPolicy* &&policy, *Iter* first, *Sent* sent, *T1* const &old_value, *T2* const &new_value, *Proj* &&proj = *Proj*())

Replaces all elements satisfying specific criteria with *new_value* in the range [first, last).

Effects: Substitutes elements referred by the iterator it in the range [first,last) with new_value, when the following corresponding conditions hold: INVOKE(proj, *i) == old_value

The assignments in the parallel *replace* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *replace* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *last* - *first* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Iter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for Iter.
- **T1** – The type of the old value to replace (deduced).
- **T2** – The type of the new values to replace (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **sent** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **old_value** – Refers to the old value of the elements to replace.
- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace* algorithm returns a *hpx::future<Iter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *Iter* otherwise.

template<typename **ExPolicy**, typename **Rng**, typename **Proj** = *hpx*::identity, typename **T1** = typename *hpx*::*parallel*::traits::projected<*std*::ranges::iterator_t<*Rng*>, *Proj*>::value_type, typename **T2** = *T1*>
parallel::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::replace`(*ExPolicy* &&policy, *Rng* &&rng, *T1* const &old_value, *T2* const &new_value, *Proj* &&proj = *Proj*())

Replaces all elements satisfying specific criteria with *new_value* in the range *rng*.

Effects: Substitutes elements referred by the iterator it in the range rng with new_value, when the following corresponding conditions hold: INVOKE(proj, *i) == old_value

The assignments in the parallel *replace* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an

unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *util::end(rng) - util::begin(rng)* assignments.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a forward iterator.
- **T1** – The type of the old value to replace (deduced).
- **T2** – The type of the new values to replace (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **old_value** – Refers to the old value of the elements to replace.
- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked. The assignments in the parallel *replace* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

Returns

The *replace* algorithm returns an hpx::future<*hpx::traits::range_iterator*<*Rng*>*::type*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *hpx::traits::range_iterator*<*Rng*>*::type* otherwise.

hpx::ranges::replace_if

Defined in header hpx/algorithm.hpp[793].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::replace*
- *hpx::ranges::replace_copy*
- *hpx::ranges::replace_copy_if*

[793] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

template<typename **Iter**, typename **Sent**, typename **Pred**, typename **Proj** = *hpx*::identity, typename **T** = typename *hpx*::*parallel*::traits::projected<*Iter*, *Proj*>::value_type>

Iter **hpx::ranges::replace_if**(*Iter* first, *Sent* sent, *Pred* &&pred, *T* const &new_value, *Proj* &&proj = *Proj*())

Replaces all elements satisfying specific criteria (for which predicate *f* returns true) with *new_value* in the range [first, sent).

Effects: Substitutes elements referred by the iterator it in the range [first, sent) with new_value, when the following corresponding conditions hold: INVOKE(f, INVOKE(proj, *it)) != false

The assignments in the parallel *replace_if* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *sent - first* applications of the predicate.

Template Parameters

- **Iter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for Iter.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*. (deduced).
- **T** – The type of the new values to replace (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **sent** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the elements which need to replaced. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *Iter* can be dereferenced and then implicitly converted to *Type*.

- **new_value** – Refers to the new value to use as the replacement.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace_if* algorithm returns *Iter*. It returns *last*.

template<typename **Rng**, typename **Pred**, typename **Proj** = *hpx*::identity, typename **T** = typename *hpx*::*parallel*::traits::projected<*std*::ranges::iterator_t<*Rng*>, *Proj*>::value_type>
std::ranges::iterator_t<*Rng*> `hpx::ranges::replace_if`(*Rng* &&rng, *Pred* &&pred, *T* const &new_value, *Proj* &&proj = *Proj*())

Replaces all elements satisfying specific criteria (for which predicate *pred* returns true) with *new_value* in the range rng.

Effects: Substitutes elements referred by the iterator it in the range rng with new_value, when the following corresponding conditions hold: INVOKE(f, INVOKE(proj, *it)) != false

> **Note**
>
> Complexity: Performs exactly *util::end(rng) - util::begin(rng)* applications of the predicate.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*. (deduced).
- **T** – The type of the new values to replace (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by rng.This is an unary predicate which returns *true* for the elements which need to replaced. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.
- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace_if* algorithm returns an *hpx::traits::range_iterator<Rng>::type*. It returns *last*.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **Pred**, typename **Proj** = *hpx*::identity, typename **T** = typename *hpx*::*parallel*::traits::projected<*Iter*, *Proj*>::value_type>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *Iter*> `hpx::ranges::replace_if`(*ExPolicy* &&policy, *Iter* first, *Sent* sent, *Pred* &&pred, *T* const &new_value, *Proj* &&proj = *Proj*())

Replaces all elements satisfying specific criteria (for which predicate *pred* returns true) with *new_value* in the range rng.

Effects: Substitutes elements referred by the iterator it in the range rng with new_value, when the following corresponding conditions hold: INVOKE(f, INVOKE(proj, *it)) != false

The assignments in the parallel *replace_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *replace_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *util::end(rng) - util::begin(rng)* applications of the predicate.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Iter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for Iter.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. (deduced).
- **T** – The type of the new values to replace (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **sent** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the elements which need to replaced. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.
- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace_if* algorithm returns a *hpx::future<Iter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy*. It returns *last*.

template<typename **ExPolicy**, typename **Rng**, typename **Pred**, typename **Proj** = *hpx*::identity, typename **T** = typename *hpx*::*parallel*::traits::projected<*std*::ranges::iterator_t<*Rng*>, *Proj*>::value_type>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::replace_if`(*ExPolicy* &&policy, *Rng* &&rng, *Pred* &&pred, *T* const &new_value, *Proj* &&proj = *Proj*())

Replaces all elements satisfying specific criteria (for which predicate *pred* returns true) with *new_value* in the range rng.

Effects: Substitutes elements referred by the iterator it in the range rng with new_value, when the following corresponding conditions hold: INVOKE(f, INVOKE(proj, *it)) != false

The assignments in the parallel *replace* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *replace* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an

unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *util::end(rng) - util::begin(rng)* applications of the predicate.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of a forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *F* to meet the requirements of *CopyConstructible*. (deduced).
- **T** – The type of the new values to replace (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by rng.This is an unary predicate which returns *true* for the elements which need to replaced. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.
- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace_if* algorithm returns a hpx::future<*std::ranges::iterator_t*<*Rng*>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy*. It returns *last*.

hpx::ranges::replace_copy

Defined in header hpx/algorithm.hpp[794].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::replace*
- *hpx::ranges::replace_if*
- *hpx::ranges::replace_copy_if*

template<typename **InIter**, typename **Sent**, typename **OutIter**, typename **Proj** = *hpx*::identity, typename **T1** = typename *hpx*::*parallel*::traits::projected<*InIter*, *Proj*>::value_type, typename **T2** = *T1*>

replace_copy_result<*InIter*, *OutIter*> `hpx::ranges::replace_copy`(*InIter* first, *Sent* sent, *OutIter* dest, *T1* const &old_value, *T2* const &new_value, *Proj* &&proj = *Proj*())

Copies the all elements from the range [first, sent) to another range beginning at *dest* replacing all elements satisfying a specific criteria with *new_value*.

Effects: Assigns to every iterator it in the range [result, result + (sent - first)) either new_value or *(first + (it - result)) depending on whether the following corresponding condition holds: INVOKE(proj, *(first + (i - result))) == old_value

The assignments in the parallel *replace_copy* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *sent* - *first* applications of the predicate.

Template Parameters

- **InIter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for Iter.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **T1** – The type of the old value to replace (deduced).
- **T2** – The type of the new values to replace (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.

[794] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **sent** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **old_value** – Refers to the old value of the elements to replace.
- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace_copy* algorithm returns an *in_out_result<InIter, OutIter>*. The *copy* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng**, typename **OutIter**, typename **Proj** = *hpx*::identity, typename **T1** = typename *hpx*::*parallel*::traits::projected<*std*::ranges::iterator_t<*Rng*>, *Proj*>::value_type, typename **T2** = *T1*>

replace_copy_result<*std*::ranges::iterator_t<*Rng*>, *OutIter*> `hpx::ranges::replace_copy`(*Rng* &&rng, *OutIter* dest, *T1* const &old_value, *T2* const &new_value, *Proj* &&proj = *Proj*())

Copies the all elements from the range rbg to another range beginning at *dest* replacing all elements satisfying a specific criteria with *new_value*.

Effects: Assigns to every iterator it in the range [result, result + (util::end(rng) - util::begin(rng))) either new_value or *(first + (it - result)) depending on whether the following corresponding condition holds: INVOKE(proj, *(first + (i - result))) == old_value

The assignments in the parallel *replace_copy* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *util::end(rng) - util::begin(rng)* applications of the predicate.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **T1** – The type of the old value to replace (deduced).
- **T2** – The type of the new values to replace (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **old_value** – Refers to the old value of the elements to replace.
- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace_copy* algorithm returns an *in_out_result<std::ranges::iterator_t<Rng>*, OutIter>. The *copy* algorithm returns the pair of the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent**, typename **FwdIter2**, typename **Proj** = *hpx*::identity, typename **T1** = typename *hpx*::*parallel*::traits::projected<*FwdIter1*, *Proj*>::value_type, typename **T2** = *T1*>

parallel::*util*::detail::algorithm_result<*ExPolicy*, replace_copy_result<*FwdIter1*, *FwdIter2*>>::type `hpx::ranges::replace_copy`(*ExPolic* &&pol- icy, *FwdIter* first, *Sent* sent, *FwdIter* dest, *T1* const &old_v *T2* const &new_v *Proj* &&proj = *Proj*())

Copies the all elements from the range [first, sent) to another range beginning at *dest* replacing all elements satisfying a specific criteria with *new_value*.

Effects: Assigns to every iterator it in the range [result, result + (sent - first)) either new_value or *(first + (it - result)) depending on whether the following corresponding condition holds: INVOKE(proj, *(first + (i - result))) == old_value

The assignments in the parallel *replace_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *replace_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Performs exactly *sent* - *first* applications of the predicate.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterator used (deduced). The iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for Iter.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **T1** – The type of the old value to replace (deduced).
- **T2** – The type of the new values to replace (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **sent** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **old_value** – Refers to the old value of the elements to replace.
- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace_copy* algorithm returns a hpx::future<*in_out_result*<*FwdIter1, FwdIter2*>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *in_out_result*<*FwdIter1, FwdIter2*> otherwise. The *copy* algorithm returns the pair of the forward iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng**, typename **FwdIter**, typename **Proj** = *hpx*::identity, typename **T1** = typename *hpx*::*parallel*::traits::projected<*std*::ranges::iterator_t<*Rng*>, *Proj*>::value_type, typename **T2** = *T1*>

parallel::*util*::detail::algorithm_result<*ExPolicy*, replace_copy_result<*std*::ranges::iterator_t<*Rng*>, *FwdIter*>>::type `hpx::ranges::replac`

Copies the all elements from the range rbg to another range beginning at *dest* replacing all elements satisfying a specific criteria with *new_value*.

Effects: Assigns to every iterator it in the range [result, result + (util::end(rng) - util::begin(rng))) either new_value or *(first + (it - result)) depending on whether the following corresponding condition holds: INVOKE(proj, *(first + (i - result))) == old_value

The assignments in the parallel *replace_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *replace_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *util::end(rng) - util::begin(rng)* applications of the predicate.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **T1** – The type of the old value to replace (deduced).
- **T2** – The type of the new values to replace (deduced).

- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **old_value** – Refers to the old value of the elements to replace.
- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace_copy* algorithm returns a hpx::future<*in_out_result*< std::ranges::iterator_t<Rng>, FwdIter>> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *in_out_result*< std::ranges::iterator_t<Rng>, FwdIter>> The *copy* algorithm returns the pair of the input iterator *last* and the forward iterator to the element in the destination range, one past the last element copied.

hpx::ranges::replace_copy_if

Defined in header hpx/algorithm.hpp[795].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::replace*
- *hpx::ranges::replace_if*
- *hpx::ranges::replace_copy*

template<typename **InIter**, typename **Sent**, typename **OutIter**, typename **Pred**, typename **T** = typename *std*::iterator_traits<*OutIter*>::value_type, typename **Proj** = *hpx*::identity>

replace_copy_if_result<*InIter*, *OutIter*> `hpx::ranges::replace_copy_if`(*InIter* first, *Sent* sent, *OutIter* dest, *Pred* &&pred, *T* const &new_value, *Proj* &&proj = *Proj*())

Copies the all elements from the range [first, sent) to another range beginning at *dest* replacing all elements satisfying a specific criteria with *new_value*.

Effects: Assigns to every iterator it in the range [result, result + (sent - first)) either new_value or *(first + (it - result)) depending on whether the following corresponding condition holds: INVOKE(f, INVOKE(proj, *(first + (i - result)))) != false

The assignments in the parallel *replace_copy_if* algorithm execute in sequential order in the calling thread.

[795] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Note

Complexity: Performs exactly *sent* - *first* applications of the predicate.

Template Parameters

- **InIter** – The type of the source iterator used (deduced). The iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for InIter.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. (deduced).
- **T** – The type of the new values to replace (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **sent** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the elements which need to replaced. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.
- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace_copy_if* algorithm returns a *in_out_result<InIter, OutIter>*. The *replace_copy_if* algorithm returns the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng**, typename **OutIter**, typename **Pred**, typename **T** = typename *std*::iterator_traits<*OutIter*>::value_type, typename **Proj** = *hpx*::identity>

replace_copy_if_result<*std*::ranges::iterator_t<*Rng*>, *OutIter*> `hpx::ranges::replace_copy_if`(*Rng* &&rng, *OutIter* dest, *Pred* &&pred, *T* const &new_value, *Proj* &&proj = *Proj*())

Copies the all elements from the range rng to another range beginning at *dest* replacing all elements satisfying a specific criteria with *new_value*.

Effects: Assigns to every iterator it in the range [result, result + (util::end(rng) - util::begin(rng))) either new_value or *(first + (it - result)) depending on whether the following corresponding condition holds: INVOKE(f, INVOKE(proj, *(first + (i - result)))) != false

The assignments in the parallel *replace_copy_if* algorithm execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs exactly *util::end(rng) - util::begin(rng)* applications of the predicate.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **OutIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. (deduced).
- **T** – The type of the new values to replace (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the elements which need to replaced. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.

- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace_copy_if* algorithm returns an *in_out_result<std::ranges::iterator_t<Rng>*, OutIter>. The *replace_copy_if* algorithm returns the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent**, typename **FwdIter2**, typename **Pred**, typename **T** = typename *std*::iterator_traits<*FwdIter2*>::value_type, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, replace_copy_if_result<*FwdIter1*, *FwdIter2*>>::type `hpx::ranges::replace_copy_if`(*E*
&
ic
F
fi
S
s
F
d
F
&
T
c
&
F
&
=
F

Copies the all elements from the range [first, sent) to another range beginning at *dest* replacing all elements satisfying a specific criteria with *new_value*.

Effects: Assigns to every iterator it in the range [result, result + (sent - first)) either new_value or *(first + (it - result)) depending on whether the following corresponding condition holds: INVOKE(f, INVOKE(proj, *(first + (i - result)))) != false

The assignments in the parallel *replace_copy_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *replace_copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *sent* - *first* applications of the predicate.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter1** – The type of the source iterator used (deduced). The iterator type must meet the requirements of a forward iterator.
- **Sent** – The type of the end iterators used (deduced). This sentinel type must be a sentinel for InIter.
- **FwdIter2** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. (deduced).
- **T** – The type of the new values to replace (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **sent** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the elements which need to replaced. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.
- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace_copy_if* algorithm returns an *hpx::future<FwdIter1, FwdIter2>*. The *replace_copy_if* algorithm returns the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng**, typename **FwdIter**, typename **Pred**, typename **T** = typename *std*::iterator_traits<*FwdIter*>::value_type, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, replace_copy_if_result<*std*::ranges::iterator_t<*Rng*>, *FwdIter*>>::type `hpx::ranges::rep`

Copies the all elements from the range rng to another range beginning at *dest* replacing all elements satisfying a specific criteria with *new_value*.

Effects: Assigns to every iterator it in the range [result, result + (util::end(rng) - util::begin(rng))) either new_value or *(first + (it - result)) depending on whether the following corresponding condition holds: INVOKE(f, INVOKE(proj, *(first + (i - result)))) != false

The assignments in the parallel *replace_copy_if* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *replace_copy_if* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs exactly *util::end(rng) - util::begin(rng)* applications of the predicate.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`Rng`** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **`FwdIter`** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Pred`** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *equal* requires *Pred* to meet the requirements of *CopyConstructible*. (deduced).
- **`T`** – The type of the new values to replace (deduced).

- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate which returns *true* for the elements which need to replaced. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.
- **new_value** – Refers to the new value to use as the replacement.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *replace_copy_if* algorithm returns an hpx::future<*in_out_result*<*std::ranges::iterator_t*<*Rng*>, OutIter>>. The *replace_copy_if* algorithm returns the input iterator *last* and the output iterator to the element in the destination range, one past the last element copied.

hpx::ranges::sort

Defined in header hpx/algorithm.hpp[796].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **RandomIt**, typename **Sent**, typename **Comp** = *ranges*::less, typename **Proj** = *hpx*::identity>

RandomIt `hpx::ranges::sort`(*RandomIt* first, *Sent* last, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Sorts the elements in the range [first, last) in ascending order. The order of equal elements is not guaranteed to be preserved. The function uses the given comparison function object comp (defaults to using operator<()).

A sequence is sorted with respect to a comparator *comp* and a projection *proj* if for every iterator i pointing to the sequence and every non-negative integer n such that i + n is a valid iterator pointing to an element of the sequence, and INVOKE(comp, INVOKE(proj, *(i + n)), INVOKE(proj, *i)) == false.

comp has to induce a strict weak ordering on the values.

The assignments in the parallel *sort* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

[796] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Note

Complexity: O(N log(N)), where N = detail::distance(first, last) comparisons.

Template Parameters

- **RandomIt** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for RandomIt.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *sort* algorithm returns *RandomIt*. The algorithm returns an iterator pointing to the first element after the last element in the input sequence.

template<typename **ExPolicy**, typename **RandomIt**, typename **Sent**, typename **Comp** = *ranges*::less, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *RandomIt*>::type `hpx::ranges::sort`(*ExPolicy* &&policy, *RandomIt* first, *Sent* last, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Sorts the elements in the range [first, last) in ascending order. The order of equal elements is not guaranteed to be preserved. The function uses the given comparison function object comp (defaults to using operator<()).

A sequence is sorted with respect to a comparator *comp* and a projection *proj* if for every iterator i pointing to the sequence and every non-negative integer n such that i + n is a valid iterator pointing to an element of the sequence, and INVOKE(comp, INVOKE(proj, *(i + n)), INVOKE(proj, *i)) == false.

comp has to induce a strict weak ordering on the values.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(N log(N)), where N = detail::distance(first, last) comparisons.

Template Parameters

- `ExPolicy` – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- `RandomIt` – The type of the source iterators used (deduced). This iterator type must meet the requirements of an random iterator.
- `Sent` – The type of the source sentinel (deduced). This sentinel type must be a sentinel for RandomIt.
- `Comp` – The type of the function/function object to use (deduced).
- `Proj` – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `first` – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- `last` – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- `comp` – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- `proj` – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *sort* algorithm returns a *hpx::future<RandomIt>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *RandomIt* otherwise. The algorithm returns an iterator pointing to the first element after the last element in the input sequence.

template<typename **Rng**, typename **Comp**, typename **Proj**>

std::ranges::iterator_t<*Rng*> `hpx::ranges::sort`(*Rng* &&rng, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Sorts the elements in the range *rng* in ascending order. The order of equal elements is not guaranteed to be preserved. The function uses the given comparison function object comp (defaults to using operator<()).

A sequence is sorted with respect to a comparator *comp* and a projection *proj* if for every iterator i pointing to the sequence and every non-negative integer n such that i + n is a valid iterator pointing to an element of the sequence, and INVOKE(comp, INVOKE(proj, *(i + n)), INVOKE(proj, *i)) == false.

comp has to induce a strict weak ordering on the values.

The assignments in the parallel *sort* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: O(N log(N)), where N = std::distance(begin(rng), end(rng)) comparisons.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *sort* algorithm returns *std::ranges::iterator_t<Rng>*. It returns *last*.

template<typename **ExPolicy**, typename **Rng**, typename **Comp** = *ranges*::less, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::sort`(*ExPolicy* &&policy, *Rng* &&rng, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Sorts the elements in the range *rng* in ascending order. The order of equal elements is not guaranteed to be preserved. The function uses the given comparison function object comp (defaults to using operator<()).

A sequence is sorted with respect to a comparator *comp* and a projection *proj* if for every iterator i pointing to the sequence and every non-negative integer n such that i + n is a valid iterator pointing to an element of the sequence, and INVOKE(comp, INVOKE(proj, *(i + n)), INVOKE(proj, *i)) == false.

comp has to induce a strict weak ordering on the values.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: O(N log(N)), where N = std::distance(begin(rng), end(rng)) comparisons.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **comp** – comp is a callable object. The return value of the INVOKE operation applied to an object of type Comp, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **proj** – Specifies the function (or function object) which will be invoked for each pair of elements as a projection operation before the actual predicate *comp* is invoked.

Returns

The *sort* algorithm returns a hpx::future<*std::ranges::iterator_t<Rng>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *std::ranges::iterator_t<Rng>* otherwise. It returns *last*.

hpx::ranges::partition

Defined in header hpx/algorithm.hpp[797].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::stable_partition*

[797] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- *hpx::ranges::partition_copy*

template<typename **Rng**, typename **Pred**, typename **Proj** = *hpx*::identity>

subrange_t<*std*::ranges::iterator_t<*Rng*>> `hpx::ranges::partition`(*Rng* &&rng, *Pred* &&pred, *Proj* &&proj = *Proj*())

Reorders the elements in the range *rng* in such a way that all elements for which the predicate *pred* returns true precede the elements for which the predicate *pred* returns false. Relative order of the elements is not preserved.

The assignments in the parallel *partition* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs at most 2 * N swaps, exactly N applications of the predicate and projection, where N = std::distance(begin(rng), end(rng)).

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by the range *rng*. This is an unary predicate for partitioning the source iterators. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *partition* algorithm returns *subrange_t<std::ranges::iterator_t<Rng>>* The *partition* algorithm returns a subrange starting with an iterator to the first element of the second group and finishing with an iterator equal to last.

template<typename **ExPolicy**, typename **Rng**, typename **Pred**, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, subrange_t<*std*::ranges::iterator_t<*Rng*>>> `hpx::ranges::partition`(*ExPolicy* &&policy, *Rng* &&rng, *Pred* &&pred, *Proj* &&proj = *Proj*())

Reorders the elements in the range *rng* in such a way that all elements for which the predicate *pred* returns true precede the elements for which the predicate *pred* returns false. Relative order of the elements is not preserved.

The assignments in the parallel *partition* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *partition* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs at most 2 * N swaps, exactly N applications of the predicate and projection, where N = std::distance(begin(rng), end(rng)).

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by the range *rng*. This is an unary predicate for partitioning the source iterators. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to *Type*.

- **`proj`** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *partition* algorithm returns a hpx::future<*subrange_t<std::ranges::iterator_t<Rng>>*> if the execution policy is of type *parallel_task_policy* and returns *subrange_t<std::ranges::iterator_t<Rng>>* The *partition* algorithm returns a subrange starting with an iterator to the first element of the second group and finishing with an iterator equal to last.

template<typename **FwdIter**, typename **Sent**, typename **Pred**, typename **Proj** = *hpx*::identity>

subrange_t<*FwdIter*> `hpx::ranges::partition`(*FwdIter* first, *Sent* last, *Pred* &&pred, *Proj* &&proj = *Proj*())

Reorders the elements in the range [first, last) in such a way that all elements for which the predicate *pred* returns true precede the elements for which the predicate *pred* returns false. Relative order of the elements is not preserved.

The assignments in the parallel *partition* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: At most 2 * (last - first) swaps. Exactly *last* - *first* applications of the predicate and projection.

Template Parameters

- **`FwdIter`** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Sent`** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **`Pred`** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition* requires *Pred* to meet the requirements of *CopyConstructible*.
- **`Proj`** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.

- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an unary predicate for partitioning the source iterators. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *partition* algorithm returns returns *subrange_t<FwdIter>*. The *partition* algorithm returns a subrange starting with an iterator to the first element of the second group and finishing with an iterator equal to last.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **Pred**, typename **Proj** = *hpx*::identity>
parallel::*util*::detail::algorithm_result<*ExPolicy*, subrange_t<*FwdIter*>>::type `hpx::ranges::partition`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *Pred* &&pred, *Proj* &&proj = *Proj*())

Reorders the elements in the range [first, last) in such a way that all elements for which the predicate *pred* returns true precede the elements for which the predicate *pred* returns false. Relative order of the elements is not preserved.

The assignments in the parallel *partition* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *partition* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most 2 * (last - first) swaps. Exactly *last* - *first* applications of the predicate and projection.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an unary predicate for partitioning the source iterators. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *partition* algorithm returns a hpx::future<*subrange_t*<*FwdIter*>> if the execution policy is of type *parallel_task_policy* and returns *subrange_t*<*FwdIter*> otherwise. The *partition* algorithm returns a subrange starting with an iterator to the first element of the second group and finishing with an iterator equal to last.

hpx::ranges::stable_partition

Defined in header hpx/algorithm.hpp[798].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::partition*
- *hpx::ranges::partition_copy*

template<typename **Rng**, typename **Pred**, typename **Proj** = *hpx*::identity>

[798] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

subrange_t<*std*::ranges::iterator_t<*Rng*>> `hpx::ranges::stable_partition`(*Rng* &&rng, *Pred* &&pred, *Proj* &&proj = *Proj*())

Permutes the elements in the range [first, last) such that there exists an iterator i such that for every iterator j in the range [first, i) INVOKE(f, INVOKE (proj, *j)) != false, and for every iterator k in the range [i, last), INVOKE(f, INVOKE (proj, *k)) == false

The invocations of *f* in the parallel *stable_partition* algorithm invoked without an execution policy object executes in sequential order in the calling thread.

> **Note**
>
> Complexity: At most (last - first) * log(last - first) swaps, but only linear number of swaps if there is enough extra memory Exactly *last - first* applications of the predicate and projection.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an birdirectional iterator
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Unary predicate which returns true if the element should be ordered before other elements. Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

  ```
  bool fun(const Type &a);
  ```

  The signature does not need to have const&. The type *Type* must be such that an object of type *BidirIter* can be dereferenced and then implicitly converted to *Type*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *f* is invoked.

Returns

The *stable_partition* algorithm returns an iterator i such that for every iterator j in the range [first, i), f(*j) != false INVOKE(f, INVOKE(proj, *j)) != false, and for every iterator k in the range [i, last), f(*k) == false INVOKE(f, INVOKE (proj, *k)) == false. The relative order of the elements in both groups is preserved.

template<typename **ExPolicy**, typename **Rng**, typename **Pred**, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, subrange_t<*std*::ranges::iterator_t<*Rng*>>> `hpx::ranges::stable_partition`(*ExPolicy* &&policy, *Rng* &&rng, *Pred* &&pred, *Proj* &&proj = *Proj*())

Permutes the elements in the range [first, last) such that there exists an iterator i such that for every iterator j in the range [first, i) INVOKE(f, INVOKE (proj, *j)) != false, and for every iterator k in the range [i, last), INVOKE(f, INVOKE (proj, *k)) == false

The invocations of *f* in the parallel *stable_partition* algorithm invoked with an execution policy object of type *sequenced_policy* executes in sequential order in the calling thread.

The invocations of *f* in the parallel *stable_partition* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most (last - first) * log(last - first) swaps, but only linear number of swaps if there is enough extra memory. Exactly *last* - *first* applications of the predicate and projection.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the invocations of *f*.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an birdirectional iterator
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **pred** – Unary predicate which returns true if the element should be ordered before other elements. Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
bool fun(const Type &a);
```

The signature does not need to have const&. The type *Type* must be such that an object of type *BidirIter* can be dereferenced and then implicitly converted to *Type*.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *f* is invoked.

Returns

The *stable_partition* algorithm returns an iterator i such that for every iterator j in the range [first, i), f(*j) != false INVOKE(f, INVOKE(proj, *j)) != false, and for every iterator k in the range [i, last), f(*k) == false INVOKE(f, INVOKE (proj, *k)) == false. The relative order of the elements in both groups is preserved. If the execution policy is of type *parallel_task_policy* the algorithm returns a future<> referring to this iterator.

template<typename **BidirIter**, typename **Sent**, typename **Pred**, typename **Proj** = *hpx*::identity>

subrange_t<*BidirIter*> *hpx*::*ranges*::**stable_partition**(*BidirIter* first, *Sent* last, *Pred* &&pred, *Proj* &&proj = *Proj*())

Permutes the elements in the range [first, last) such that there exists an iterator i such that for every iterator j in the range [first, i) INVOKE(f, INVOKE (proj, *j)) != false, and for every iterator k in the range [i, last), INVOKE(f, INVOKE (proj, *k)) == false

The invocations of *f* in the parallel *stable_partition* algorithm invoked without an execution policy object executes in sequential order in the calling thread.

> **Note**
>
> Complexity: At most (last - first) * log(last - first) swaps, but only linear number of swaps if there is enough extra memory Exactly *last* - *first* applications of the predicate and projection.

Template Parameters

- **BidirIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for BidirIter.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.

- **pred** – Unary predicate which returns true if the element should be ordered before other elements. Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
bool fun(const Type &a);
```

  The signature does not need to have const&. The type *Type* must be such that an object of type *BidirIter* can be dereferenced and then implicitly converted to *Type*.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *f* is invoked.

Returns

The *stable_partition* algorithm returns an iterator i such that for every iterator j in the range [first, i), f(*j) != false INVOKE(f, INVOKE(proj, *j)) != false, and for every iterator k in the range [i, last), f(*k) == false INVOKE(f, INVOKE (proj, *k)) == false. The relative order of the elements in both groups is preserved.

template<typename **ExPolicy**, typename **BidirIter**, typename **Sent**, typename **Pred**, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, subrange_t<*BidirIter*>>::type `hpx::ranges::stable_partition`(*ExPolicy* &&policy, *BidirIter* first, *Sent* last, *Pred* &&pred, *Proj* &&proj = *Proj*())

Permutes the elements in the range [first, last) such that there exists an iterator i such that for every iterator j in the range [first, i) INVOKE(f, INVOKE (proj, *j)) != false, and for every iterator k in the range [i, last), INVOKE(f, INVOKE (proj, *k)) == false

The invocations of *f* in the parallel *stable_partition* algorithm invoked with an execution policy object of type *sequenced_policy* executes in sequential order in the calling thread.

The invocations of *f* in the parallel *stable_partition* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most (last - first) * log(last - first) swaps, but only linear number of swaps if there is enough extra memory Exactly *last* - *first* applications of the predicate and projection.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the invocations of *f*.
- **BidirIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for BidirIter.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **pred** – Unary predicate which returns true if the element should be ordered before other elements. Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

  ```
  bool fun(const Type &a);
  ```

  The signature does not need to have const&. The type *Type* must be such that an object of type *BidirIter* can be dereferenced and then implicitly converted to *Type*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *f* is invoked.

Returns

The *stable_partition* algorithm returns an iterator i such that for every iterator j in the range [first, i), f(*j) != false INVOKE(f, INVOKE(proj, *j)) != false, and for every iterator k in the range [i, last), f(*k) == false INVOKE(f, INVOKE (proj, *k)) == false. The relative order of the elements in both groups is preserved. If the execution policy is of type *parallel_task_policy* the algorithm returns a future<> referring to this iterator.

hpx::ranges::partition_copy

Defined in header hpx/algorithm.hpp[799].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::partition*
- *hpx::ranges::stable_partition*

template<typename **Rng**, typename **OutIter2**, typename **OutIter3**, typename **Pred**, typename **Proj** = *hpx*::identity>

partition_copy_result<*std*::ranges::iterator_t<*Rng*>, *OutIter2*, *OutIter3*> `hpx::ranges::partition_copy`(*Rng* &&rng, *OutIter2* dest_true, *OutIter3* dest_false, *Pred* &&pred, *Proj* &&proj = *Proj*())

Copies the elements in the range *rng*, to two different ranges depending on the value returned by the predicate *pred*. The elements, that satisfy the predicate *pred* are copied to the range beginning at *dest_true*. The rest of the elements are copied to the range beginning at *dest_false*. The order of the elements is preserved.

The assignments in the parallel *partition_copy* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

> **Note**
>
> Complexity: Performs not more than N assignments, exactly N applications of the predicate *pred*, where N = std::distance(begin(rng), end(rng)).

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **OutIter2** – The type of the iterator representing the destination range for the elements that satisfy the predicate *pred* (deduced). This iterator type must meet the requirements of an forward iterator.

[799] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **OutIter3** – The type of the iterator representing the destination range for the elements that don't satisfy the predicate *pred* (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition_copy* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest_true** – Refers to the beginning of the destination range for the elements that satisfy the predicate *pred*
- **dest_false** – Refers to the beginning of the destination range for the elements that don't satisfy the predicate *pred*.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an unary predicate for partitioning the source iterators. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *partition_copy* algorithm returns a partition_copy_result<std::ranges::iterator_t<Rng>, FwdIter2, FwdIter3>>. The *partition_copy* algorithm returns the tuple of the source iterator *last*, the destination iterator to the end of the *dest_true* range, and the destination iterator to the end of the *dest_false* range.

template<typename **ExPolicy**, typename **Rng**, typename **FwdIter2**, typename **FwdIter3**, typename **Pred**, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, partition_copy_result<*std*::ranges::iterator_t<*Rng*>, *FwdIter2*, *FwdIter3*>>::type `hpx::rang`

Copies the elements in the range *rng*, to two different ranges depending on the value returned by the predicate *pred*. The elements, that satisfy the predicate *pred* are copied to the range beginning at *dest_true*. The rest of the elements are copied to the range beginning at *dest_false*. The order of the elements is preserved.

The assignments in the parallel *partition_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *partition_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs not more than N assignments, exactly N applications of the predicate *pred*, where N = std::distance(begin(rng), end(rng)).

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **`Rng`** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an forward iterator.
- **`FwdIter2`** – The type of the iterator representing the destination range for the elements that satisfy the predicate *pred* (deduced). This iterator type must meet the requirements of an forward iterator.
- **`FwdIter3`** – The type of the iterator representing the destination range for the elements that don't satisfy the predicate *pred* (deduced). This iterator type must meet the requirements of an forward iterator.
- **`Pred`** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition_copy* requires *Pred* to meet the requirements of *CopyConstructible*.
- **`Proj`** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`rng`** – Refers to the sequence of elements the algorithm will be applied to.
- **`dest_true`** – Refers to the beginning of the destination range for the elements that satisfy the predicate *pred*
- **`dest_false`** – Refers to the beginning of the destination range for the elements that don't satisfy the predicate *pred*.
- **`pred`** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an unary predicate for partitioning the source iterators. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *partition_copy* algorithm returns a hpx::future<*partition_copy_result* <std::ranges::iterator_t<Rng>, FwdIter2, FwdIter3>> if the execution policy is of type *parallel_task_policy* and returns partition_copy_result<std::ranges::iterator_t<Rng>, FwdIter2, FwdIter3> otherwise. The *partition_copy* algorithm returns the tuple of the source iterator *last*, the destination iterator to the end of the *dest_true* range, and the destination iterator to the end of the *dest_false* range.

template<typename **InIter**, typename **Sent**, typename **OutIter2**, typename **OutIter3**, typename **Pred**, typename **Proj** = *hpx*::identity>

partition_copy_result<*InIter*, *OutIter2*, *OutIter3*> `hpx::ranges::`**`partition_copy`**(*InIter* first, *Sent* last, *OutIter2* dest_true, *OutIter3* dest_false, *Pred* &&pred, *Proj* &&proj = *Proj*())

Copies the elements in the range, defined by [first, last), to two different ranges depending on the value returned by the predicate *pred*. The elements, that satisfy the predicate *pred* are copied to the range beginning at *dest_true*. The rest of the elements are copied to the range beginning at *dest_false*. The order of the elements is preserved.

The assignments in the parallel *partition_copy* algorithm invoked without an execution policy object execute in sequential order in the calling thread.

Note

Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the predicate *f*.

Template Parameters

- **InIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **OutIter2** – The type of the iterator representing the destination range for the elements that satisfy the predicate *pred* (deduced). This iterator type must meet the requirements of an forward iterator.
- **OutIter3** – The type of the iterator representing the destination range for the elements that don't satisfy the predicate *pred* (deduced). This iterator type must meet the requirements of an forward iterator.

- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition_copy* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest_true** – Refers to the beginning of the destination range for the elements that satisfy the predicate *pred*
- **dest_false** – Refers to the beginning of the destination range for the elements that don't satisfy the predicate *pred*.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an unary predicate for partitioning the source iterators. The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *partition_copy* algorithm returns a *partition_copy_result<FwdIter, OutIter2, OutIter3>*. The *partition_copy* algorithm returns the tuple of the source iterator *last*, the destination iterator to the end of the *dest_true* range, and the destination iterator to the end of the *dest_false* range.

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **OutIter2**, typename **OutIter3**, typename **Pred**, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, partition_copy_result<*FwdIter*, *OutIter2*, *OutIter3*>>::type `hpx::ranges::partition_co`

Copies the elements in the range, defined by [first, last), to two different ranges depending on the value returned by the predicate *pred*. The elements, that satisfy the predicate *pred* are copied to the range beginning at *dest_true*. The rest of the elements are copied to the range beginning at *dest_false*. The order of the elements is preserved.

The assignments in the parallel *partition_copy* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The assignments in the parallel *partition_copy* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Performs not more than *last* - *first* assignments, exactly *last* - *first* applications of the predicate *f*.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **FwdIter** – The type of the source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for FwdIter.
- **OutIter2** – The type of the iterator representing the destination range for the elements that satisfy the predicate *pred* (deduced). This iterator type must meet the requirements of an forward iterator.

- **OutIter3** – The type of the iterator representing the destination range for the elements that don't satisfy the predicate *pred* (deduced). This iterator type must meet the requirements of an forward iterator.
- **Pred** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *partition_copy* requires *Pred* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to sentinel value denoting the end of the sequence of elements the algorithm will be applied.
- **dest_true** – Refers to the beginning of the destination range for the elements that satisfy the predicate *pred*
- **dest_false** – Refers to the beginning of the destination range for the elements that don't satisfy the predicate *pred*.
- **pred** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). This is an unary predicate for partitioning the source iterators. The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *partition_copy* algorithm returns a *hpx::future*<partition_copy_result<FwdIter, OutIter2, OutIter3>> if the execution policy is of type *parallel_task_policy* and returns *partition_copy_result<FwdIter, OutIter2, OutIter3>* otherwise. The *partition_copy* algorithm returns the tuple of the source iterator *last*, the destination iterator to the end of the *dest_true* range, and the destination iterator to the end of the *dest_false* range.

hpx::ranges::transform_reduce

Defined in header hpx/algorithm.hpp[800].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

> **Warning**
>
> doxygenfunction: Cannot find function "hpx::ranges::transform_reduce" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

[800] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::ranges::is_heap

Defined in header hpx/algorithm.hpp[801].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::is_heap_until*

template<typename **ExPolicy**, typename **Rng**, typename **Comp** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::is_heap`(*ExPolicy* &&policy, *Rng* &&rng, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Returns whether the range is max heap. That is, true if the range is max heap, false otherwise. The function uses the given comparison function object *comp* (defaults to using operator<()).

comp has to induce a strict weak ordering on the values.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs at most N applications of the comparison *comp*, at most 2 * N applications of the projection *proj*, where N = last - first.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an random access iterator.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.

801 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **comp** – *comp* is a callable object. The return value of the INVOKE operation applied to an object of type *Comp*, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_heap* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *is_heap* algorithm returns whether the range is max heap. That is, true if the range is max heap, false otherwise.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **Comp** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::is_heap`(*ExPolicy* &&policy, *Iter* first, *Sent* last, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Returns whether the range is max heap. That is, true if the range is max heap, false otherwise. The function uses the given comparison function object *comp* (defaults to using operator<()).

comp has to induce a strict weak ordering on the values.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs at most N applications of the comparison *comp*, at most 2 * N applications of the projection *proj*, where N = last - first.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Iter** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – *comp* is a callable object. The return value of the INVOKE operation applied to an object of type *Comp*, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_heap* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *is_heap* algorithm returns whether the range is max heap. That is, true if the range is max heap, false otherwise.

template<typename **Rng**, typename **Comp** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

bool *hpx::ranges::***is_heap**(*Rng* &&rng, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Returns whether the range is max heap. That is, true if the range is max heap, false otherwise. The function uses the given comparison function object *comp* (defaults to using operator<()).

comp has to induce a strict weak ordering on the values.

> **Note**
>
> Complexity: Performs at most N applications of the comparison *comp*, at most 2 * N applications of the projection *proj*, where N = last - first.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an random access iterator.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **comp** – *comp* is a callable object. The return value of the INVOKE operation applied to an object of type *Comp*, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_heap* algorithm returns *bool*. The *is_heap* algorithm returns whether the range is max heap. That is, true if the range is max heap, false otherwise.

template<typename **Iter**, typename **Sent**, typename **Comp** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

bool *hpx*::*ranges*::**is_heap**(*Iter* first, *Sent* last, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Returns whether the range is max heap. That is, true if the range is max heap, false otherwise. The function uses the given comparison function object *comp* (defaults to using operator<()).

comp has to induce a strict weak ordering on the values.

> **Note**
>
> Complexity: Performs at most N applications of the comparison *comp*, at most 2 * N applications of the projection *proj*, where N = last - first.

Template Parameters

- **Iter** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – *comp* is a callable object. The return value of the INVOKE operation applied to an object of type *Comp*, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_heap* algorithm returns *bool*. The *is_heap* algorithm returns whether the range is max heap. That is, true if the range is max heap, false otherwise.

hpx::ranges::is_heap_until

Defined in header hpx/algorithm.hpp[802].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::is_heap*

template<typename **ExPolicy**, typename **Rng**, typename **Comp** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>
hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::is_heap_until`(*ExPolicy* &&policy, *Rng* &&rng, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Returns the upper bound of the largest range beginning at *first* which is a max heap. That is, the last iterator *it* for which range [first, it) is a max heap. The function uses the given comparison function object *comp* (defaults to using operator<()).

comp has to induce a strict weak ordering on the values.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs at most N applications of the comparison *comp*, at most 2 * N applications of the projection *proj*, where N = last - first.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an random access iterator.
- **Comp** – The type of the function/function object to use (deduced).

[802] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **comp** – *comp* is a callable object. The return value of the INVOKE operation applied to an object of type *Comp*, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_heap_until* algorithm returns a *hpx::future<RandIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *RandIter* otherwise. The *is_heap_until* algorithm returns the upper bound of the largest range beginning at first which is a max heap. That is, the last iterator *it* for which range [first, it) is a max heap.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **Comp** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *Iter*> `hpx::ranges::is_heap_until`(*ExPolicy* &&policy, *Iter* first, *Sent* last, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Returns the upper bound of the largest range beginning at *first* which is a max heap. That is, the last iterator *it* for which range [first, it) is a max heap. The function uses the given comparison function object *comp* (defaults to using operator<()).

comp has to induce a strict weak ordering on the values.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Performs at most N applications of the comparison *comp*, at most 2 * N applications of the projection *proj*, where N = last - first.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the assignments.

- **Iter** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – *comp* is a callable object. The return value of the INVOKE operation applied to an object of type *Comp*, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_heap_until* algorithm returns a *hpx::future<RandIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *RandIter* otherwise. The *is_heap_until* algorithm returns the upper bound of the largest range beginning at first which is a max heap. That is, the last iterator *it* for which range [first, it) is a max heap.

template<typename **Rng**, typename **Comp** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

std::ranges::iterator_t<*Rng*> `hpx::ranges::is_heap_until`(*Rng* &&rng, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Returns the upper bound of the largest range beginning at *first* which is a max heap. That is, the last iterator *it* for which range [first, it) is a max heap. The function uses the given comparison function object *comp* (defaults to using operator<()).

comp has to induce a strict weak ordering on the values.

> **Note**
>
> Complexity: Performs at most N applications of the comparison *comp*, at most 2 * N applications of the projection *proj*, where N = last - first.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an random access iterator.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **comp** – *comp* is a callable object. The return value of the INVOKE operation applied to an object of type *Comp*, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_heap_until* algorithm returns *RandIter*. The *is_heap_until* algorithm returns the upper bound of the largest range beginning at first which is a max heap. That is, the last iterator *it* for which range [first, it) is a max heap.

template<typename **Iter**, typename **Sent**, typename **Comp** = *hpx*::*parallel*::detail::less, typename **Proj** = *hpx*::identity>

Iter `hpx::ranges::is_heap_until`(*Iter* first, *Sent* last, *Comp* &&comp = *Comp*(), *Proj* &&proj = *Proj*())

Returns the upper bound of the largest range beginning at *first* which is a max heap. That is, the last iterator *it* for which range [first, it) is a max heap. The function uses the given comparison function object *comp* (defaults to using operator<()).

comp has to induce a strict weak ordering on the values.

Note

Complexity: Performs at most N applications of the comparison *comp*, at most 2 * N applications of the projection *proj*, where N = last - first.

Template Parameters

- **Iter** – The type of the begin source iterators used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Comp** – The type of the function/function object to use (deduced).
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **comp** – *comp* is a callable object. The return value of the INVOKE operation applied to an object of type *Comp*, when contextually converted to bool, yields true if the first argument of the call is less than the second, and false otherwise. It is assumed that comp will not apply any non-constant function through the dereferenced iterator.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *is_heap_until* algorithm returns *RandIter*. The *is_heap_until* algorithm returns the upper bound of the largest range beginning at first which is a max heap. That is, the last iterator *it* for which range [first, it) is a max heap.

hpx::ranges::experimental::for_loop

Defined in header hpx/algorithm.hpp[803].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::experimental::for_loop_strided*
- *hpx::ranges::experimental::for_loop_n*
- *hpx::ranges::experimental::for_loop_n_strided*

```
template<typename ExPolicy, typename Iter, typename Sent, typename ...Args>
hpx::parallel::util::detail::algorithm_result<ExPolicy>::type hpx::ranges::experimental::for_loop(ExPolicy &&policy, Iter first, Sent last, Args&&... args)
```

The for_loop implements loop functionality over a range specified by iterator bounds. These algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator.

Requires: *Iter* shall meet the requirements of a forward iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of MoveConstructible.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is last - first.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the

[803] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

> **Note**
>
> As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

> **Note**
>
> The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Iter** – The type of the iteration variable (forward iterator).
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for Iter.
- **Args** – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **args** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

  ```
  <ignored> pred(Iter const& a, ...);
  ```

  The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

Returns

The *for_loop* algorithm returns a *hpx::future<void>* if the execution policy is of type *hpx::execution::sequenced_task_policy* or *hpx::execution::parallel_task_policy* and returns *void* otherwise.

template<typename **Iter**, typename **Sent**, typename ...**Args**>

void *hpx*::*ranges*::experimental::**for_loop**(*Iter* first, *Sent* last, *Args*&&... args)

The for_loop implements loop functionality over a range specified by iterator bounds. These algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator.

The execution of for_loop without specifying an execution policy is equivalent to specifying *hpx::execution::seq* as the execution policy.

Requires: *Iter* shall meet the requirements of an input iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of MoveConstructible.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is last - first.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

> **Note**
>
> As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

> **Note**
>
> The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **Iter** – The type of the iteration variable (input iterator).
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for Iter.

- **Args** – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **args** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

```
<ignored> pred(Iter const& a, ...);
```

  The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

template<typename **ExPolicy**, typename **R**, typename ...**Args**>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*>::type `hpx::ranges::experimental::`**for_loop**(*ExPolicy* &&policy, *R* &&rng, *Args*&&... args)

The for_loop implements loop functionality over a range specified by a range. These algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator.

Requires: *Rng::iterator* shall meet the requirements of a forward iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of MoveConstructible.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is last - first.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

> **Note**
>
> As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

> **Note**
>
> The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **R** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Args** – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to theof the sequence of elements the algorithm will be applied to.
- **args** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

  ```
  <ignored> pred(Rng::iterator const& a, ...);
  ```

  The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

Returns

The *for_loop* algorithm returns a *hpx::future<void>* if the execution policy is of type *hpx::execution::sequenced_task_policy* or *hpx::execution::parallel_task_policy* and returns *void* otherwise.

template<typename **Rng**, typename ...**Args**>

void *hpx*::*ranges*::experimental::**for_loop**(*Rng* &&rng, *Args*&&... args)

The for_loop implements loop functionality over a range specified by a range. These algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator.

The execution of for_loop without specifying an execution policy is equivalent to specifying *hpx::execution::seq* as the execution policy.

Requires: *Rng::iterator* shall meet the requirements of an input iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of MoveConstructible.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is last - first.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

> **Note**
>
> As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

> **Note**
>
> The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Args** – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **rng** – Refers to theof the sequence of elements the algorithm will be applied to.
- **args** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

```
<ignored> pred(Rng::iterator const& a, ...);
```

  The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

hpx::ranges::experimental::for_loop_strided

Defined in header hpx/algorithm.hpp[804].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::experimental::for_loop*
- *hpx::ranges::experimental::for_loop_n*
- *hpx::ranges::experimental::for_loop_n_strided*

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **S**, typename ...**Args**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*> `hpx::ranges::experimental::`**`for_loop_strided`**(*ExPolicy* &&policy, *Iter* first, *Sent* last, *S* stride, *Args*&&... args)

The for_loop_strided implements loop functionality over a range specified by iterator bounds. These algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator.

Requires: *Iter* shall meet the requirements of a forward iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of MoveConstructible.

[804] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is last - first.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

> **Note**
>
> As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

> **Note**
>
> The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **`Iter`** – The type of the iteration variable (forward iterator).
- **`Sent`** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for Iter.
- **`S`** – The type of the stride variable. This should be an integral type.
- **`Args`** – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.

- **stride** – Refers to the stride of the iteration steps. This shall have non-zero value and shall be negative only if Iter meets the requirements a bidirectional iterator.
- **args** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

```
<ignored> pred(Iter const& a, ...);
```

  The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

Returns

The *for_loop_strided* algorithm returns a *hpx::future<void>* if the execution policy is of type *hpx::execution::sequenced_task_policy* or *hpx::execution::parallel_task_policy* and returns *void* otherwise.

template<typename **Iter**, typename **Sent**, typename **S**, typename ...**Args**>

void *hpx*::*ranges*::experimental::**for_loop_strided**(*Iter* first, *Sent* last, *S* stride, *Args*&&... args)

The for_loop_strided implements loop functionality over a range specified by iterator bounds. These algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator.

The execution of for_loop_strided without specifying an execution policy is equivalent to specifying *hpx::execution::seq* as the execution policy.

Requires: *Iter* shall meet the requirements of an input iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of MoveConstructible.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is last - first.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

Note

As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

Note

The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **Iter** – The type of the iteration variable (input iterator).
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for Iter.
- **S** – The type of the stride variable. This should be an integral type.
- **Args** – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **stride** – Refers to the stride of the iteration steps. This shall have non-zero value and shall be negative only if Iter meets the requirements a bidirectional iterator.
- **args** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

```
<ignored> pred(Iter const& a, ...);
```

  The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

template<typename **ExPolicy**, typename **Rng**, typename **S**, typename ...**Args**>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*> `hpx::ranges::experimental::for_loop_strided`(*ExPolicy* &&policy, *Rng* &&rng, *S* stride, *Args*&&... args)

The for_loop_strided implements loop functionality over a range specified by a range. These algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator.

Requires: *Rng::iterator* shall meet the requirements of a forward iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of MoveConstructible.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is last - first.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

> **Note**
>
> As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

> **Note**
>
> The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **S** – The type of the stride variable. This should be an integral type.
- **Args** – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to theof the sequence of elements the algorithm will be applied to.
- **stride** – Refers to the stride of the iteration steps. This shall have non-zero value and shall be negative only if Rng::iterator meets the requirements a bidirectional iterator.
- **args** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

```
<ignored> pred(Rng::iterator const& a, ...);
```

The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

Returns

The *for_loop_strided* algorithm returns a *hpx::future<void>* if the execution policy is of type *hpx::execution::sequenced_task_policy* or *hpx::execution::parallel_task_policy* and returns *void* otherwise.

template<typename **Rng**, typename **S**, typename ...**Args**>

void `hpx::ranges::experimental::`**`for_loop_strided`**(*Rng* &&rng, *S* stride, *Args*&&... args)

The for_loop_strided implements loop functionality over a range specified by a range. These algorithms resemble for_each from the Parallelism TS, but leave to the programmer when and if to dereference the iterator.

The execution of for_loop_strided without specifying an execution policy is equivalent to specifying *hpx::execution::seq* as the execution policy.

Requires: *Rng::iterator* shall meet the requirements of an input iterator type. The *args* parameter pack shall have at least one element, comprising objects returned by invocations of *reduction* and/or *induction* function templates followed by exactly one element invocable element-access function, *f*. *f* shall meet the requirements of MoveConstructible.

Effects: Applies *f* to each element in the input sequence, with additional arguments corresponding to the reductions and inductions in the *args* parameter pack. The length of the input sequence is last - first.

The first element in the input sequence is specified by *first*. Each subsequent element is generated by incrementing the previous element.

Along with an element from the input sequence, for each member of the *args* parameter pack excluding *f*, an additional argument is passed to each application of *f* as follows:

If the pack member is an object returned by a call to a reduction function listed in section, then the additional argument is a reference to a view of that reduction object. If the pack member is an object returned by a call to induction, then the additional argument is the induction value for that induction object corresponding to the position of the application of *f* in the input sequence.

Complexity: Applies *f* exactly once for each element of the input sequence.

Remarks: If *f* returns a result, the result is ignored.

> **Note**
>
> As described in the C++ standard, arithmetic on non-random-access iterators is performed using advance and distance.

> **Note**
>
> The order of the elements of the input sequence is important for determining ordinal position of an application of *f*, even though the applications themselves may be unordered.

Template Parameters

- **`Rng`** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **`S`** – The type of the stride variable. This should be an integral type.
- **`Args`** – A parameter pack, it's last element is a function object to be invoked for each iteration, the others have to be either conforming to the induction or reduction concept.

Parameters

- **`rng`** – Refers to theof the sequence of elements the algorithm will be applied to.
- **`stride`** – Refers to the stride of the iteration steps. This shall have non-zero value and shall be negative only if Rng::iterator meets the requirements a bidirectional iterator.
- **`args`** – The last element of this parameter pack is the function (object) to invoke, while the remaining elements of the parameter pack are instances of either induction or reduction objects. The function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last) should expose a signature equivalent to:

```
<ignored> pred(Rng::iterator const& a, ...);
```

The signature does not need to have const&. It will receive the current value of the iteration variable and one argument for each of the induction or reduction objects passed to the algorithms, representing their current values.

hpx::ranges::experimental::for_loop_n

Defined in header hpx/algorithm.hpp[805].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::experimental::for_loop*
- *hpx::ranges::experimental::for_loop_strided*
- *hpx::ranges::experimental::for_loop_n_strided*

Warning

doxygenfunction: Cannot find function "hpx::ranges::experimental::for_loop_n" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::ranges::experimental::for_loop_n_strided

Defined in header hpx/algorithm.hpp[806].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::experimental::for_loop*
- *hpx::ranges::experimental::for_loop_strided*
- *hpx::ranges::experimental::for_loop_n*

Warning

doxygenfunction: Cannot find function "hpx::ranges::experimental::for_loop_n_strided" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::ranges::set_symmetric_difference

Defined in header hpx/algorithm.hpp[807].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

[805] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp
[806] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp
[807] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, set_symmetric_difference_result<*Iter1*, *Iter2*, *Iter3*>>::type `hpx::ranges::set_symm`

Constructs a sorted range beginning at dest consisting of all elements present in either of the sorted ranges [first1, last1) and [first2, last2), but not in both of them are copied to the range beginning at *dest*. The resulting range is also sorted. This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), it will be copied to *dest* exactly std::abs(m-n) times. If m>n, then the last m-n of those elements are copied from [first1,last1), otherwise the last n-m elements are copied from [first2,last2). The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

The application of function objects in parallel algorithm invoked with a sequential execution policy object execute in sequential order in the calling thread (*sequenced_policy*) or in a single new thread spawned from the current thread (for *sequenced_task_policy*).

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Iter1** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of an forward iterator.
- **Sent1** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.
- **Iter2** – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an forward iterator.
- **Sent2** – The type of the end source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an sentinel for Iter2.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_symmetric_difference* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.

- `proj2` – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_symmetric_difference* algorithm returns a hpx::future<*ranges::set_symmetric_difference_result<Iter1, Iter2, Iter3>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::set_symmetric_difference_result<Iter1, Iter2, Iter3>* otherwise. The *set_symmetric_difference* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng1**, typename **Rng2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, set_symmetric_difference_result<*std*::ranges::iterator_t<*Rng1*>, *std*::ranges::iterator_t<

Constructs a sorted range beginning at dest consisting of all elements present in either of the sorted ranges [first1, last1) and [first2, last2), but not in both of them are copied to the range beginning at *dest*. The resulting range is also sorted. This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), it will be copied to *dest* exactly std::abs(m-n) times. If m>n, then the last m-n of those elements are copied from [first1,last1), otherwise the last n-m elements are copied from [first2,last2). The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

The application of function objects in parallel algorithm invoked with a sequential execution policy object execute in sequential order in the calling thread (*sequenced_policy*) or in a single new thread spawned from the current thread (for *sequenced_task_policy*).

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Rng1** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Rng2** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_symmetric_difference* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng1** – Refers to the first sequence of elements the algorithm will be applied to.
- **rng2** – Refers to the second sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.

- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_symmetric_difference* algorithm returns a hpx::future<*ranges::set_symmetric_difference_result<Iter1, Iter2, Iter3>*> if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *ranges::set_symmetric_difference_result<Iter1, Iter2, Iter3>* otherwise. where Iter1 is range_iterator_t<Rng1> and Iter2 is range_iterator_t<Rng2> The *set_symmetric_difference* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **Iter1**, typename **Sent1**, typename **Iter2**, typename **Sent2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

set_symmetric_difference_result<*Iter1*, *Iter2*, *Iter3*> `hpx::ranges::set_symmetric_difference`(*Iter1* first1, *Sent1* last1, *Iter2* first2, *Sent2* last2, *Iter3* dest, *Pred* &&op = *Pred*(), *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Constructs a sorted range beginning at dest consisting of all elements present in either of the sorted ranges [first1, last1) and [first2, last2), but not in both of them are copied to the range beginning at *dest*. The resulting range is also sorted. This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), it will be copied to *dest* exactly std::abs(m-n) times. If m>n, then the last m-n of those elements are copied from [first1,last1), otherwise the last n-m elements are copied from [first2,last2). The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **Iter1** – The type of the source iterators used (deduced) representing the first sequence. This iterator type must meet the requirements of an forward iterator.
- **Sent1** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of an sentinel for Iter1.

- **Iter2** – The type of the source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an forward iterator.
- **Sent2** – The type of the end source iterators used (deduced) representing the second sequence. This iterator type must meet the requirements of an sentinel for Iter2.
- **Iter3** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **Pred** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_symmetric_difference* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **Proj1** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **first1** – Refers to the beginning of the sequence of elements of the first range the algorithm will be applied to.
- **last1** – Refers to the end of the sequence of elements of the first range the algorithm will be applied to.
- **first2** – Refers to the beginning of the sequence of elements of the second range the algorithm will be applied to.
- **last2** – Refers to the end of the sequence of elements of the second range the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

  ```
  bool pred(const Type1 &a, const Type1 &b);
  ```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_symmetric_difference* algorithm returns *ranges::set_symmetric_difference_result<Iter1, Iter2, Iter3>*. The *set_symmetric_difference* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng1**, typename **Rng2**, typename **Iter3**, typename **Pred** = *hpx*::*parallel*::detail::less, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

set_symmetric_difference_result<*std*::ranges::iterator_t<*Rng1*>, *std*::ranges::iterator_t<*Rng2*>, *Iter3*> `hpx::ranges::set_symmetric_dif`

Constructs a sorted range beginning at dest consisting of all elements present in either of the sorted ranges [first1, last1) and [first2, last2), but not in both of them are copied to the range beginning at *dest*. The resulting range is also sorted. This algorithm expects both input ranges to be sorted with the given binary predicate *f*.

If some element is found *m* times in [first1, last1) and *n* times in [first2, last2), it will be copied to *dest* exactly std::abs(m-n) times. If m>n, then the last m-n of those elements are copied from [first1,last1), otherwise the last n-m elements are copied from [first2,last2). The resulting range cannot overlap with either of the input ranges.

The resulting range cannot overlap with either of the input ranges.

> **Note**
>
> Complexity: At most 2*(N1 + N2 - 1) comparisons, where *N1* is the length of the first sequence and *N2* is the length of the second sequence.

Template Parameters

- **`Rng1`** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **`Rng2`** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **`Iter3`** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **`Pred`** – The type of an optional function/function object to use. Unlike its sequential form, the parallel overload of *set_symmetric_difference* requires *Pred* to meet the requirements of *CopyConstructible*. This defaults to std::less<>
- **`Proj1`** – The type of an optional projection function applied to the first sequence. This defaults to *hpx::identity*

- **Proj2** – The type of an optional projection function applied to the second sequence. This defaults to *hpx::identity*

Parameters

- **rng1** – Refers to the first sequence of elements the algorithm will be applied to.
- **rng2** – Refers to the second sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **op** – The binary predicate which returns true if the elements should be treated as equal. The signature of the predicate function should be equivalent to the following:

```
bool pred(const Type1 &a, const Type1 &b);
```

  The signature does not need to have const &, but the function must not modify the objects passed to it. The type *Type1* must be such that objects of type *InIter* can be dereferenced and then implicitly converted to *Type1*

- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *op* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *op* is invoked.

Returns

The *set_symmetric_difference* algorithm returns *ranges::set_symmetric_difference_result<Iter1, Iter2, Iter3>*. where Iter1 is range_iterator_t<Rng1> and Iter2 is range_iterator_t<Rng2> The *set_symmetric_difference* algorithm returns the output iterator to the element in the destination range, one past the last element copied.

hpx::ranges::for_each

Defined in header hpx/algorithm.hpp[808].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::for_each_n*

template<typename **InIter**, typename **Sent**, typename **F**, typename **Proj** = *hpx*::identity>

for_each_result<*InIter*, *F*> `hpx::ranges::for_each`(*InIter* first, *Sent* last, *F* &&f, *Proj* &&proj = *Proj*())

Applies *f* to the result of dereferencing every iterator in the range [first, last).

If *f* returns a result, the result is ignored.

If the type of *first* satisfies the requirements of a mutable iterator, *f* may apply non-constant functions through the dereferenced iterator.

[808] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Note

Complexity: Applies *f* exactly *last* - *first* times.

Template Parameters

- **InIter** – The type of the source begin iterator used (deduced). This iterator type must meet the requirements of an input iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *for_each* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
<ignored> pred(const Type &a);
```

  The signature does not need to have const&. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

{last, HPX_MOVE(f)} where last is the iterator corresponding to the input sentinel last.

template<typename **Rng**, typename **F**, typename **Proj** = *hpx*::identity>

for_each_result<*std*::ranges::iterator_t<*Rng*>, *F*> `hpx::ranges::for_each`(*Rng* &&rng, *F* &&f, *Proj* &&proj = *Proj*())

Applies *f* to the result of dereferencing every iterator in the given range *rng*.

If *f* returns a result, the result is ignored.

If the type of *first* satisfies the requirements of a mutable iterator, *f* may apply non-constant functions through the dereferenced iterator.

Note

Complexity: Applies *f* exactly *size(rng)* times.

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *for_each* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
<ignored> pred(const Type &a);
```

  The signature does not need to have const&. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

{std::end(rng), HPX_MOVE(f)}

template<typename **ExPolicy**, typename **FwdIter**, typename **Sent**, typename **F**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, *FwdIter*> `hpx::ranges::for_each`(*ExPolicy* &&policy, *FwdIter* first, *Sent* last, *F* &&f, *Proj* &&proj = *Proj*())

Applies *f* to the result of dereferencing every iterator in the range [first, last).

If *f* returns a result, the result is ignored.

If the type of *first* satisfies the requirements of a mutable iterator, *f* may apply non-constant functions through the dereferenced iterator.

Unlike its sequential form, the parallel overload of *for_each* does not return a copy of its *Function* parameter, since parallelization may not permit efficient state accumulation.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: Applies *f* exactly *last* - *first* times.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **FwdIter** – The type of the source begin iterator used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *for_each* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

  ```
  <ignored> pred(const Type &a);
  ```

  The signature does not need to have const&. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *for_each* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. It returns *last*.

template<typename **ExPolicy**, typename **Rng**, typename **F**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *std*::ranges::iterator_t<*Rng*>> `hpx::ranges::for_each`(*ExPolicy* &&policy, *Rng* &&rng, *F* &&f, *Proj* &&proj = *Proj*())

Applies *f* to the result of dereferencing every iterator in the given range *rng*.

If *f* returns a result, the result is ignored.

If the type of *first* satisfies the requirements of a mutable iterator, *f* may apply non-constant functions through the dereferenced iterator.

Unlike its sequential form, the parallel overload of *for_each* does not return a copy of its *Function* parameter, since parallelization may not permit efficient state accumulation.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Applies *f* exactly *size(rng)* times.

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **`Rng`** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **`F`** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *for_each* requires *F* to meet the requirements of *CopyConstructible*.
- **`Proj`** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`rng`** – Refers to the sequence of elements the algorithm will be applied to.

- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
<ignored> pred(const Type &a);
```

The signature does not need to have const&. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *for_each* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. It returns *last*.

hpx::ranges::for_each_n

Defined in header hpx/algorithm.hpp[809].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::for_each*

template<typename **InIter**, typename **Size**, typename **F**, typename **Proj** = *hpx*::identity>

for_each_n_result<*InIter*, *F*> **hpx::ranges::for_each_n**(*InIter* first, *Size* count, *F* &&f, *Proj* &&proj = *Proj*())

Applies *f* to the result of dereferencing every iterator in the range [first, first + count), starting from first and proceeding to first + count - 1.

If *f* returns a result, the result is ignored.

If the type of *first* satisfies the requirements of a mutable iterator, *f* may apply non-constant functions through the dereferenced iterator.

Unlike its sequential form, the parallel overload of *for_each* does not return a copy of its *Function* parameter, since parallelization may not permit efficient state accumulation.

> **Note**
>
> Complexity: Applies *f* exactly *count* times.

Template Parameters

- **InIter** – The type of the source begin iterator used (deduced). This iterator type must meet the requirements of an input iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.

[809] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *for_each* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
<ignored> pred(const Type &a);
```

  The signature does not need to have const&. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

It returns *last*.

template<typename **ExPolicy**, typename **FwdIter**, typename **Size**, typename **F**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result<*ExPolicy*, *FwdIter*>::type `hpx::ranges::for_each_n`(*ExPolicy* &&policy, *FwdIter* first, *Size* count, *F* &&f, *Proj* &&proj = *Proj*())

Applies *f* to the result of dereferencing every iterator in the range [first, first + count), starting from first and proceeding to first + count - 1.

If *f* returns a result, the result is ignored.

If the type of *first* satisfies the requirements of a mutable iterator, *f* may apply non-constant functions through the dereferenced iterator.

Unlike its sequential form, the parallel overload of *for_each* does not return a copy of its *Function* parameter, since parallelization may not permit efficient state accumulation.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Applies *f* exactly *count* times.

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **FwdIter** – The type of the source begin iterator used (deduced). This iterator type must meet the requirements of an forward iterator.
- **Size** – The type of the argument specifying the number of elements to apply *f* to.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *for_each* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements the algorithm will be applied to.
- **count** – Refers to the number of elements starting at *first* the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

  ```
  <ignored> pred(const Type &a);
  ```

  The signature does not need to have const&. The type *Type* must be such that an object of type *InIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *for_each* algorithm returns a *hpx::future<FwdIter>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *FwdIter* otherwise. It returns *last*.

hpx::ranges::all_of

Defined in header hpx/algorithm.hpp[810].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::any_of*
- *hpx::ranges::none_of*

template<typename **ExPolicy**, typename **Rng**, typename **F**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::all_of`(*ExPolicy* &&policy, *Rng* &&rng, *F* &&f, *Proj* &&proj = *Proj*())

Checks if unary predicate *f* returns true for all elements in the range *rng*.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most std::distance(begin(rng), end(rng)) applications of the predicate *f*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *none_of* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.

[810] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *all_of* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *all_of* algorithm returns true if the unary predicate *f* returns true for all elements in the range, false otherwise. It returns true if the range is empty.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **F**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::all_of`(*ExPolicy* &&policy, *Iter* first, *Sent* last, *F* &&f, *Proj* &&proj = *Proj*())

Checks if unary predicate *f* returns true for all elements in the range *rng*.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most std::distance(begin(rng), end(rng)) applications of the predicate *f*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Iter** – The type of the source iterators used for the range (deduced).
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *none_of* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *all_of* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *all_of* algorithm returns true if the unary predicate *f* returns true for all elements in the range, false otherwise. It returns true if the range is empty.

template<typename **Rng**, typename **F**, typename **Proj** = *hpx*::identity>

bool *hpx*::*ranges*::**all_of**(*Rng* &&rng, *F* &&f, *Proj* &&proj = *Proj*())

Checks if unary predicate *f* returns true for all elements in the range *rng*.

> **Note**
>
> Complexity: At most std::distance(begin(rng), end(rng)) applications of the predicate *f*

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *none_of* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.

- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *all_of* algorithm returns true if the unary predicate *f* returns true for all elements in the range, false otherwise. It returns true if the range is empty.

template<typename **Iter**, typename **Sent**, typename **F**, typename **Proj** = *hpx*::identity>

bool *hpx*::*ranges*::**all_of**(*Iter* first, *Sent* last, *F* &&f, *Proj* &&proj = *Proj*())

Checks if unary predicate *f* returns true for all elements in the range *rng*.

> **Note**
>
> Complexity: At most std::distance(begin(rng), end(rng)) applications of the predicate *f*

Template Parameters

- **Iter** – The type of the source iterators used for the range (deduced).
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *none_of* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *all_of* algorithm returns true if the unary predicate *f* returns true for all elements in the range, false otherwise. It returns true if the range is empty.

hpx::ranges::any_of

Defined in header hpx/algorithm.hpp[811].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::all_of*
- *hpx::ranges::none_of*

template<typename **ExPolicy**, typename **Rng**, typename **F**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::`**`any_of`**(*ExPolicy* &&policy, *Rng* &&rng, *F* &&f, *Proj* &&proj = *Proj*())

Checks if unary predicate *f* returns true for at least one element in the range *rng*.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most std::distance(begin(rng), end(rng)) applications of the predicate *f*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

[811] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *none_of* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *any_of* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *any_of* algorithm returns true if the unary predicate *f* returns true for at least one element in the range, false otherwise. It returns false if the range is empty.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **F**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::`**`any_of`**(*ExPolicy* &&policy, *Iter* first, *Sent* last, *F* &&f, *Proj* &&proj = *Proj*())

Checks if unary predicate *f* returns true for at least one element in the range *rng*.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most std::distance(begin(rng), end(rng)) applications of the predicate *f*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Iter** – The type of the source iterators used for the range (deduced).
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *none_of* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *any_of* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *any_of* algorithm returns true if the unary predicate *f* returns true for at least one element in the range, false otherwise. It returns false if the range is empty.

template<typename **Rng**, typename **F**, typename **Proj** = *hpx*::identity>

bool *hpx::ranges::***any_of**(*Rng* &&rng, *F* &&f, *Proj* &&proj = *Proj*())

Checks if unary predicate *f* returns true for at least one element in the range *rng*.

> **Note**
>
> Complexity: At most std::distance(begin(rng), end(rng)) applications of the predicate *f*

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *none_of* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *any_of* algorithm returns true if the unary predicate *f* returns true for at least one element in the range, false otherwise. It returns false if the range is empty.

template<typename **Iter**, typename **Sent**, typename **F**, typename **Proj** = *hpx*::identity>

bool *hpx*::*ranges*::**any_of**(*Iter* first, *Sent* last, *F* &&f, *Proj* &&proj = *Proj*())

Checks if unary predicate *f* returns true for at least one element in the range *rng*.

> **Note**
>
> Complexity: At most std::distance(begin(rng), end(rng)) applications of the predicate *f*

Template Parameters

- **Iter** – The type of the source iterators used for the range (deduced).
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *none_of* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *any_of* algorithm returns true if the unary predicate *f* returns true for at least one element in the range, false otherwise. It returns false if the range is empty.

hpx::ranges::none_of

Defined in header hpx/algorithm.hpp[812].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ranges::all_of*
- *hpx::ranges::any_of*

template<typename **ExPolicy**, typename **Rng**, typename **F**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::none_of`(*ExPolicy* &&policy, *Rng* &&rng, *F* &&f, *Proj* &&proj = *Proj*())

Checks if unary predicate *f* returns true for no elements in the range *rng*.

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: At most std::distance(begin(rng), end(rng)) applications of the predicate *f*

[812] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *none_of* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *none_of* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *none_of* algorithm returns true if the unary predicate *f* returns true for no elements in the range, false otherwise. It returns true if the range is empty.

template<typename **ExPolicy**, typename **Iter**, typename **Sent**, typename **F**, typename **Proj** = *hpx*::identity>

hpx::*parallel*::*util*::detail::algorithm_result_t<*ExPolicy*, bool> `hpx::ranges::none_of`(*ExPolicy* &&policy, *Iter* first, *Sent* last, *F* &&f, *Proj* &&proj = *Proj*())

Checks if unary predicate *f* returns true for no elements in the range [first, last).

The application of function objects in parallel algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The application of function objects in parallel algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

Note

Complexity: At most *last - first* applications of the predicate *f*

Template Parameters

- `ExPolicy` – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it applies user-provided function objects.
- `Iter` – The type of the source iterators used for the range (deduced).
- `Sent` – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.
- `F` – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *none_of* requires *F* to meet the requirements of *CopyConstructible*.
- `Proj` – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- `policy` – The execution policy to use for the scheduling of the iterations.
- `first` – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- `last` – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- `f` – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.
- `proj` – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *none_of* algorithm returns a *hpx::future<bool>* if the execution policy is of type *sequenced_task_policy* or *parallel_task_policy* and returns *bool* otherwise. The *none_of* algorithm returns true if the unary predicate *f* returns true for no elements in the range, false otherwise. It returns true if the range is empty.

template<typename **Rng**, typename **F**, typename **Proj** = *hpx*::identity>

bool *hpx::ranges::***none_of**(*Rng* &&rng, *F* &&f, *Proj* &&proj = *Proj*())

Checks if unary predicate *f* returns true for no elements in the range *rng*.

> **Note**
>
> Complexity: At most std::distance(begin(rng), end(rng)) applications of the predicate *f*

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *none_of* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

  ```
  bool pred(const Type &a);
  ```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *none_of* algorithm returns true if the unary predicate *f* returns true for no elements in the range, false otherwise. It returns true if the range is empty.

template<typename **Iter**, typename **Sent**, typename **F**, typename **Proj** = *hpx*::identity>

bool `hpx::ranges::none_of`(*Iter* first, *Sent* last, *F* &&f, *Proj* &&proj = *Proj*())

Checks if unary predicate *f* returns true for no elements in the range [first, last).

> **Note**
>
> Complexity: At most *last - first* applications of the predicate *f*

Template Parameters

- **Iter** – The type of the source iterators used for the range (deduced).
- **Sent** – The type of the source sentinel (deduced). This sentinel type must be a sentinel for InIter.

- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *none_of* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the sequence of elements of the range the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements of the range the algorithm will be applied to.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last). The signature of this predicate should be equivalent to:

```
bool pred(const Type &a);
```

  The signature does not need to have const&, but the function must not modify the objects passed to it. The type *Type* must be such that an object of type *FwdIter* can be dereferenced and then implicitly converted to Type.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *is* invoked.

Returns

The *none_of* algorithm returns true if the unary predicate *f* returns true for no elements in the range, false otherwise. It returns true if the range is empty.

hpx::ranges::transform

Defined in header hpx/algorithm.hpp[813].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent1**, typename **FwdIter2**, typename **F**, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *ranges*::unary_transform_result<*FwdIter1*, *FwdIter2*>>::type `hpx::ranges::transform(`

[813] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/algorithm.hpp

Applies the given function *f* to the given range *rng* and stores the result in another range, beginning at dest.

The invocations of *f* in the parallel *transform* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The invocations of *f* in the parallel *transform* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly size(rng) applications of *f*

Template Parameters

- **`ExPolicy`** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the invocations of *f*.
- **`FwdIter1`** – The type of the source iterators for the first range used (deduced). This iterator type must meet the requirements of a forward iterator.
- **`Sent1`** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of a sentinel for FwdIter1.
- **`FwdIter2`** – The type of the source iterators for the first range used (deduced). This iterator type must meet the requirements of a forward iterator.
- **`F`** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *transform* requires *F* to meet the requirements of *Copy-Constructible*.
- **`Proj`** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **`policy`** – The execution policy to use for the scheduling of the iterations.
- **`first`** – Refers to the beginning of the first sequence of elements the algorithm will be applied to.
- **`last`** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **`dest`** – Refers to the beginning of the destination range.
- **`f`** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type &a);
  ```

  The signature does not need to have const&. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to *Type*. The type

Ret must be such that an object of type *FwdIter2* can be dereferenced and assigned a value of type *Ret*.

- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *f* is invoked.

Returns

The *transform* algorithm returns a hpx::future<*ranges::unary_transform_result<FwdIter1, FwdIter2>* > if the execution policy is of type *parallel_task_policy* and returns *ranges::unary_transform_result<FwdIter1, FwdIter2>* otherwise. The *transform* algorithm returns a tuple holding an iterator referring to the first element after the input sequence and the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **Rng**, typename **FwdIter**, typename **F**, typename **Proj** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *ranges*::unary_transform_result<*std*::ranges::iterator_t<*Rng*>, *FwdIter*>> `hpx::ranges::t`

Applies the given function *f* to the given range *rng* and stores the result in another range, beginning at dest.

The invocations of *f* in the parallel *transform* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The invocations of *f* in the parallel *transform* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly size(rng) applications of *f*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the invocations of *f*.
- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.

- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *transform* requires *F* to meet the requirements of *Copy-Constructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type &a);
```

  The signature does not need to have const&. The type *Type* must be such that an object of type *range_iterator<Rng>::type* can be dereferenced and then implicitly converted to *Type*. The type *Ret* must be such that an object of type *OutIter* can be dereferenced and assigned a value of type *Ret*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *f* is invoked.

Returns

The *transform* algorithm returns a hpx::future<*ranges::unary_transform_result<range_iterator<Rng>::type*, FwdIter>> if the execution policy is of type *parallel_task_policy* and returns *ranges::unary_transform_result<range_iterator<Rng>::type*, FwdIter> otherwise. The *transform* algorithm returns a tuple holding an iterator referring to the first element after the input sequence and the output iterator to the element in the destination range, one past the last element copied.

template<typename **ExPolicy**, typename **FwdIter1**, typename **Sent1**, typename **FwdIter2**, typename **Sent2**, typename **FwdIter3**, typename **F**, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

parallel::*util*::detail::algorithm_result<*ExPolicy*, *ranges*::binary_transform_result<*FwdIter1*, *FwdIter2*, *FwdIter3*>>::type `hpx::ranges::tr`

Applies the given function *f* to pairs of elements from two ranges: one defined by *rng* and the other beginning at first2, and stores the result in another range, beginning at dest.

The invocations of *f* in the parallel *transform* algorithm invoked with an execution policy object of type *sequenced_policy* execute in sequential order in the calling thread.

The invocations of *f* in the parallel *transform* algorithm invoked with an execution policy object of type *parallel_policy* or *parallel_task_policy* are permitted to execute in an unordered fashion in unspecified threads, and indeterminately sequenced within each thread.

> **Note**
>
> Complexity: Exactly size(rng) applications of *f*

Template Parameters

- **ExPolicy** – The type of the execution policy to use (deduced). It describes the manner in which the execution of the algorithm may be parallelized and the manner in which it executes the invocations of *f*.
- **FwdIter1** – The type of the source iterators for the first range used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent1** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of a sentinel for FwdIter1.
- **FwdIter2** – The type of the source iterators for the first range used (deduced). This iterator type must meet the requirements of a forward iterator.

- **Sent2** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of a sentinel for FwdIter2.
- **FwdIter3** – The type of the source iterators for the first range used (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *transform* requires *F* to meet the requirements of *Copy-Constructible*.
- **Proj1** – The type of an optional projection function to be used for elements of the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function to be used for elements of the second sequence. This defaults to *hpx::identity*

Parameters

- **policy** – The execution policy to use for the scheduling of the iterations.
- **first1** – Refers to the beginning of the first sequence of elements the algorithm will be applied to.
- **last1** – Refers to the end of the first sequence of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second sequence of elements the algorithm will be applied to.
- **last2** – Refers to the end of the second sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is a binary predicate. The signature of this predicate should be equivalent to:

```
Ret fun(const Type1 &a, const Type2 &b);
```

  The signature does not need to have const&. The types *Type1* and *Type2* must be such that objects of types FwdIter1 and FwdIter2 can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively. The type *Ret* must be such that an object of type *FwdIter3* can be dereferenced and assigned a value of type *Ret*.
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *f* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *f* is invoked.

Returns

The *transform* algorithm returns A hpx::future<*ranges::binary_transform_result*<*FwdIter1, FwdIter2, FwdIter3*>> if the execution policy is of type *parallel_task_policy* and returns *ranges::binary_transform_result*<*FwdIter1, FwdIter2, FwdIter3*> otherwise. The *transform* algorithm returns a tuple holding an iterator referring to the first element after the first input sequence, an iterator referring to the first element after the second input sequence, and the output iterator referring to the element in the destination range, one past the last element copied.

template<typename **FwdIter1**, typename **Sent1**, typename **FwdIter2**, typename **F**, typename **Proj** = *hpx*::identity>

ranges::unary_transform_result<*FwdIter1*, *FwdIter2*> **hpx::ranges::transform**(*FwdIter1* first, *Sent1* last, *FwdIter2* dest, *F* &&f, *Proj* &&proj = *Proj*())

Applies the given function *f* to the given range *rng* and stores the result in another range, beginning at dest.

> **Note**
>
> Complexity: Exactly size(rng) applications of *f*

Template Parameters

- **FwdIter1** – The type of the source iterators for the first range used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent1** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of a sentinel for FwdIter1.
- **FwdIter2** – The type of the source iterators for the first range used (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *transform* requires *F* to meet the requirements of *Copy-Constructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **first** – Refers to the beginning of the first sequence of elements the algorithm will be applied to.
- **last** – Refers to the end of the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type &a);
  ```

  The signature does not need to have const&. The type *Type* must be such that an object of type *FwdIter1* can be dereferenced and then implicitly converted to *Type*. The type *Ret* must be such that an object of type *FwdIter2* can be dereferenced and assigned a value of type *Ret*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *f* is invoked.

Returns

The *transform* algorithm returns *ranges::unary_transform_result<FwdIter1, FwdIter2>*. The *transform* algorithm returns a tuple holding an iterator referring to the first element after the input sequence and the output iterator to the element in the destination range, one past the last element copied.

template<typename **Rng**, typename **FwdIter**, typename **F**, typename **Proj** = *hpx*::identity>

ranges::unary_transform_result<*std*::ranges::iterator_t<*Rng*>, *FwdIter*> **hpx::ranges::transform**(*Rng* &&rng, *FwdIter* dest, *F* &&f, *Proj* &&proj = *Proj*())

Applies the given function *f* to the given range *rng* and stores the result in another range, beginning at dest.

> **Note**
>
> Complexity: Exactly size(rng) applications of *f*

Template Parameters

- **Rng** – The type of the source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **FwdIter** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *transform* requires *F* to meet the requirements of *CopyConstructible*.
- **Proj** – The type of an optional projection function. This defaults to *hpx::identity*

Parameters

- **rng** – Refers to the sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is an unary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type &a);
  ```

  The signature does not need to have const&. The type *Type* must be such that an object of type *range_iterator<Rng>::type* can be dereferenced and then implicitly converted to *Type*. The type *Ret* must be such that an object of type *OutIter* can be dereferenced and assigned a value of type *Ret*.
- **proj** – Specifies the function (or function object) which will be invoked for each of the elements as a projection operation before the actual predicate *f* is invoked.

Returns

The *transform* algorithm returns *ranges::unary_transform_result<range_iterator<Rng>::type*, FwdIter>. The *transform* algorithm returns a tuple holding an iterator referring to the first element after the input sequence and the output iterator to the element in the destination range, one past the last element copied.

template<typename **FwdIter1**, typename **Sent1**, typename **FwdIter2**, typename **Sent2**, typename **FwdIter3**, typename **F**, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

ranges::binary_transform_result<*FwdIter1*, *FwdIter2*, *FwdIter3*> `hpx::ranges::transform`(*FwdIter1* first1, *Sent1* last1, *FwdIter2* first2, *Sent2* last2, *FwdIter3* dest, *F* &&f, *Proj1* &&proj1 = *Proj1*(), *Proj2* &&proj2 = *Proj2*())

Applies the given function *f* to pairs of elements from two ranges: one defined by *rng* and the other beginning at first2, and stores the result in another range, beginning at dest.

> **Note**
>
> Complexity: Exactly size(rng) applications of *f*

Template Parameters

- **FwdIter1** – The type of the source iterators for the first range used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent1** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of a sentinel for FwdIter1.
- **FwdIter2** – The type of the source iterators for the first range used (deduced). This iterator type must meet the requirements of a forward iterator.
- **Sent2** – The type of the end source iterators used (deduced). This iterator type must meet the requirements of a sentinel for FwdIter2.
- **FwdIter3** – The type of the source iterators for the first range used (deduced). This iterator type must meet the requirements of a forward iterator.
- **F** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *transform* requires *F* to meet the requirements of *Copy-Constructible*.
- **Proj1** – The type of an optional projection function to be used for elements of the first sequence. This defaults to *hpx::identity*
- **Proj2** – The type of an optional projection function to be used for elements of the second sequence. This defaults to *hpx::identity*

Parameters

- **first1** – Refers to the beginning of the first sequence of elements the algorithm will be applied to.

- **last1** – Refers to the end of the first sequence of elements the algorithm will be applied to.
- **first2** – Refers to the beginning of the second sequence of elements the algorithm will be applied to.
- **last2** – Refers to the end of the second sequence of elements the algorithm will be applied to.
- **dest** – Refers to the beginning of the destination range.
- **f** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&. The types *Type1* and *Type2* must be such that objects of types FwdIter1 and FwdIter2 can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively. The type *Ret* must be such that an object of type *FwdIter3* can be dereferenced and assigned a value of type *Ret*.
- **proj1** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *f* is invoked.
- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *f* is invoked.

Returns

The *transform* algorithm returns *ranges::binary_transform_result<FwdIter1, FwdIter2, FwdIter3>*. The *transform* algorithm returns a tuple holding an iterator referring to the first element after the first input sequence, an iterator referring to the first element after the second input sequence, and the output iterator referring to the element in the destination range, one past the last element copied.

template<typename **Rng1**, typename **Rng2**, typename **FwdIter**, typename **F**, typename **Proj1** = *hpx*::identity, typename **Proj2** = *hpx*::identity>

ranges::binary_transform_result<*std*::ranges::iterator_t<*Rng1*>, *std*::ranges::iterator_t<*Rng2*>, *FwdIter*> **hpx::ranges::transform**(*Rng1* &&rn *Rng2* &&rn *FwdIt* dest, *F* &&f, *Proj1* &&pr = *Proj1*(*Proj2* &&pr = *Proj2*(

Applies the given function *f* to pairs of elements from two ranges: one defined by [first1, last1) and the other beginning at first2, and stores the result in another range, beginning at dest.

> **Note**
>
> Complexity: Exactly min(last2-first2, last1-first1) applications of *f*

> **Note**
>
> The algorithm will invoke the binary predicate until it reaches the end of the shorter of the two given input sequences

Template Parameters

- **`Rng1`** – The type of the first source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **`Rng2`** – The type of the second source range used (deduced). The iterators extracted from this range type must meet the requirements of an input iterator.
- **`FwdIter`** – The type of the iterator representing the destination range (deduced). This iterator type must meet the requirements of an output iterator.
- **`F`** – The type of the function/function object to use (deduced). Unlike its sequential form, the parallel overload of *transform* requires *F* to meet the requirements of *Copy-Constructible*.
- **`Proj1`** – The type of an optional projection function to be used for elements of the first sequence. This defaults to *hpx::identity*
- **`Proj2`** – The type of an optional projection function to be used for elements of the second sequence. This defaults to *hpx::identity*

Parameters

- **`rng1`** – Refers to the first sequence of elements the algorithm will be applied to.
- **`rng2`** – Refers to the second sequence of elements the algorithm will be applied to.
- **`dest`** – Refers to the beginning of the destination range.
- **`f`** – Specifies the function (or function object) which will be invoked for each of the elements in the sequence specified by [first, last).This is a binary predicate. The signature of this predicate should be equivalent to:

  ```
  Ret fun(const Type1 &a, const Type2 &b);
  ```

  The signature does not need to have const&. The types *Type1* and *Type2* must be such that objects of types range_iterator<Rng1>::type and range_iterator<Rng2>::type can be dereferenced and then implicitly converted to *Type1* and *Type2* respectively. The type *Ret* must be such that an object of type *FwdIter* can be dereferenced and assigned a value of type *Ret*.
- **`proj1`** – Specifies the function (or function object) which will be invoked for each of the elements of the first sequence as a projection operation before the actual predicate *f* is invoked.

- **proj2** – Specifies the function (or function object) which will be invoked for each of the elements of the second sequence as a projection operation before the actual predicate *f* is invoked.

Returns

The *transform* algorithm returns *ranges::binary_transform_result*< std::ranges::iterator_t<Rng1>, std::ranges::iterator_t<Rng2>, FwdIter>. The *transform* algorithm returns a tuple holding an iterator referring to the first element after the first input sequence, an iterator referring to the first element after the second input sequence, and the output iterator referring to the element in the destination range, one past the last element copied.

asio

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/asio/asio_util.hpp

Defined in header hpx/asio/asio_util.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

Functions

bool **get_endpoint**(*std*::string const &addr, *std*::uint16_t port, ::asio::ip::tcp::endpoint &ep, bool force_ipv4 = false)

std::string **get_endpoint_name**(::asio::ip::tcp::endpoint const &ep)

::asio::ip::tcp::endpoint **resolve_hostname**(*std*::string const &hostname, *std*::uint16_t port, ::asio::io_context &io_service, bool force_ipv4 = false)

detail::endpoint_iterator_type **connect_begin**(*std*::string const &address, *std*::uint16_t port, ::asio::io_context &io_service)

template<typename **Locality**>

detail::endpoint_iterator_type **connect_begin**(*Locality* const &loc, ::asio::io_context &io_service)

Returns an iterator which when dereferenced will give an endpoint suitable for a call to connect() related to this locality.

inline detail::endpoint_iterator_type **connect_end**()

detail::endpoint_iterator_type **accept_begin**(*std*::string const &address, *std*::uint16_t port, ::asio::io_context &io_service)

template<typename **Locality**>

detail::endpoint_iterator_type **accept_begin**(*Locality* const &loc, ::asio::io_context &io_service)

Returns an iterator which when dereferenced will give an endpoint suitable for a call to accept() related to this locality.

inline detail::endpoint_iterator_type **accept_end**()

assertion

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/assertion/evaluate_assert.hpp

Defined in header hpx/assertion/evaluate_assert.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **assertion**

hpx/assertion/api.hpp

Defined in header hpx/assertion/api.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **assertion**

Typedefs

using **assertion_handler** = void (*)(*hpx*::source_location const &loc, char const *expr, *std*::string const &msg)

The signature for an assertion handler.

Functions

assertion_handler **set_assertion_handler**(*assertion_handler* handler)

Set the assertion handler to be used within a program. If the handler has been set already once, the call to this function will be ignored.

> **Note**
>
> This function is not thread safe

HPX_ASSERT

Defined in header hpx/assert.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *HPX_ASSERT_MSG*

> **Warning**
>
> doxygenfunction: Cannot find function "HPX_ASSERT" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

HPX_ASSERT_MSG

Defined in header hpx/assert.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *HPX_ASSERT*

Warning

doxygenfunction: Cannot find function "HPX_ASSERT_MSG" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

HPX_CURRENT_SOURCE_LOCATION

Defined in header hpx/source_location.hpp[814].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::source_location*

Warning

doxygenfunction: Cannot find function "HPX_CURRENT_SOURCE_LOCATION" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::source_location

Defined in header hpx/source_location.hpp[815].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *HPX_CURRENT_SOURCE_LOCATION*

Warning

doxygenfunction: Cannot find function "hpx::source_location" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

async_base

See *Public API* for a list of names and headers that are part of the public *HPX* API.

[814] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/source_location.hpp

[815] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/source_location.hpp

hpx::sync

Defined in header hpx/future.hpp[816].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also

hpx::sync (distributed)

template<typename **Action**, typename **F**, typename ...**Ts**>

auto *hpx*::**sync**(*F* &&f, *Ts*&&... ts) -> decltype(detail::sync_action_dispatch<*Action*, *std*::decay_t<*F*>>::call(*std*::forward<*F*>(*f*), *std*::forward<*Ts*>(*ts*)...))

The function template *sync* runs the function *f* synchronously and returns a hpx::future that will eventually hold the result of that function call.

template<typename **Action**, typename **F**, typename ...**Ts**>

auto *hpx*::**sync**(*F* &&f, *Ts*&&... ts) -> decltype(detail::sync_action_dispatch<*Action*, *std*::decay_t<*F*>>::call(*std*::forward<*F*>(*f*), *std*::forward<*Ts*>(*ts*)...))

The function template *sync* runs the function *f* synchronously and returns a hpx::future that will eventually hold the result of that function call.

Warning

doxygenfunction: Unable to resolve function "hpx::sync" with arguments (Action&&, Target&&, Ts&&…) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename
↪Action, typename F, typename ...Ts> auto sync(F &&f, Ts&&...
↪ts) -> decltype(detail::sync_action_dispatch<Action, std::decay_
↪t<F>>::call(std::forward<F>(f), std::forward<Ts>(ts)...))
- template<typename
↪Action, typename Target, typename ...Ts> decltype(auto)
↪sync(Action &&action, Target &&target, Ts&&... ts)
- template<typename
↪F, typename ...Ts> decltype(auto) sync(F &&f, Ts&&... ts)
```

[816] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

hpx::launch

Defined in header hpx/future.hpp[817].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

struct **launch** : public *hpx*::detail::policy_holder<>

Launch policies for *hpx::async* etc.

hpx::post

Defined in header hpx/future.hpp[818].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

> **See also**
>
> *hpx::post (distributed)*

template<typename **F**, typename ...**Ts**>

bool *hpx*::**post**(*F* &&f, *Ts*&&... ts)

Runs the function `f` asynchronously (potentially in a separate thread which might be a part of a thread pool). This is done in a fire-and-forget manner, meaning there is no return value or way to synchronize with the function execution (it does not return an *`hpx::future`* that would hold the result of that function call).

`hpx::post` is particularly useful when synchronization mechanisms as heavyweight as futures are not desired, and instead, more lightweight mechanisms like latches or atomic variables are preferred. Essentially, the post function enables the launch of a new thread without the overhead of creating a future.

> **Note**
>
> *`hpx::post`* is similar to `hpx::async` but does not return a future. This is why there is no way of finding out the result/failure of the execution of this function.

template<typename **Action**, typename ...**Ts**>

bool *hpx*::**post**(*hpx*::id_type const &id, *Ts*&&... vs)

template<typename **Action**, typename **Client**, typename **Stub**, typename **Data**, typename ...**Ts**>

bool *hpx*::**post**(*components*::*client_base*<*Client*, *Stub*, *Data*> const &c, *Ts*&&... vs)

[817] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp
[818] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

template<typename **Action**, typename **DistPolicy**, typename ...**Ts**>

bool *hpx*::**post**(*DistPolicy* const &policy, *Ts*&&... vs)

template<typename **Action**, typename **Continuation**, typename ...**Ts**>

bool *hpx*::**post**(*Continuation* &&c, *hpx*::id_type const &gid, *Ts*&&... vs)

template<typename **Action**, typename **Continuation**, typename **Client**, typename **Stub**, typename **Data**, typename ...**Ts**>

bool *hpx*::**post**(*Continuation* &&cont, *components*::*client_base*<*Client*, *Stub*, *Data*> const &c, *Ts*&&... vs)

template<typename **Action**, typename **Continuation**, typename **DistPolicy**, typename ...**Ts**>

bool *hpx*::**post**(*Continuation* &&c, *DistPolicy* const &policy, *Ts*&&... vs)

template<typename **Action**, typename **Target**, typename ...**Ts**>

bool *hpx*::**post**(*Action* &&action, *Target* &&target, *Ts*&&... ts)

The distributed implementation of `hpx::post` can be used by giving an action instance as argument instead of a function, and also by providing another argument with the locality ID or the target ID.

Template Parameters

- **Action** – The type of action instance
- **Target** – The type of target where the action should be executed
- **Ts** – The type of any additional arguments

Parameters

- **action** – The action instance to be executed
- **target** – The target where the action should be executed
- **ts** – Additional arguments

Returns

`true` if the action was successfully posted, `false` otherwise.

hpx::dataflow

Defined in header hpx/future.hpp[819].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also

hpx::dataflow (distributed)

Warning

doxygenfunction: Unable to resolve function "hpx::dataflow" with arguments (F&&, Ts&&…) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename Action, typename Target, typename ...Ts> decltype(auto) dataflow(Action &&action, Target &&target, Ts&&... ts)
- template<typename F, typename ...Ts> decltype(auto) dataflow(F &&f, Ts&&... ts)
```

Warning

doxygenfunction: Unable to resolve function "hpx::dataflow" with arguments (Action&&, Target&&, Ts&&…) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename Action, typename Target, typename ...Ts> decltype(auto) dataflow(Action &&action, Target &&target, Ts&&... ts)
- template<typename F, typename ...Ts> decltype(auto) dataflow(F &&f, Ts&&... ts)
```

hpx::async

Defined in header hpx/future.hpp[820].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also

hpx::async (distributed)

template<typename **Action**, typename **F**, typename ...**Ts**>

[819] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp
[820] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

auto **hpx::async**(*F* &&f, *Ts*&&... ts) -> decltype(detail::async_action_dispatch<*Action*, *std*::decay_t<*F*>>::call(*std*::forward<*F*>(*f*), *std*::forward<*Ts*>(*ts*)...))

The function template *async* runs the function *f* asynchronously (potentially in a separate thread which might be a part of a thread pool) and returns a hpx::future that will eventually hold the result of that function call. If no policy is defined, *async* behaves as if it is called with policy being *hpx::launch::async* | hpx::launch::deferred. Otherwise, it calls a function *f* with arguments *ts* according to a specific launch policy.

- If the async flag is set (i.e. (policy & hpx::launch::async) != 0), then async executes the callable object f on a new thread of execution (with all thread-locals initialized) as if spawned by *hpx::thread*(std::forward<F>(f), std::forward<Ts>(ts)…), except that if the function f returns a value or throws an exception, it is stored in the shared state accessible through the *hpx::future* that async returns to the caller.
- If the deferred flag is set (i.e. (policy & hpx::launch::deferred) != 0), then async converts f and ts… the same way as by *hpx::thread* constructor, but does not spawn a new thread of execution. Instead, lazy evaluation is performed: the first call to a non-timed wait function on the *hpx::future* that async returned to the caller will cause the copy of f to be invoked (as a rvalue) with the copies of ts… (also passed as rvalues) in the current thread (which does not have to be the thread that originally called hpx::async). The result or exception is placed in the shared state associated with the future and only then it is made ready. All further accesses to the same *hpx::future* will return the result immediately.
- If neither hpx::launch::async nor hpx::launch::deferred, nor any implementation-defined policy flag is set in policy, the behavior is undefined.

  If more than one flag is set, it is implementation-defined which policy is selected. For the default (both the hpx::launch::async and hpx::launch::deferred flags are set in policy), standard recommends (but doesn't require) utilizing available concurrency, and deferring any additional tasks.

  In any case, the call to hpx::async synchronizes-with (as defined in std::memory_order) the call to f, and the completion of f is sequenced-before making the shared state ready. If the async policy is chosen, the associated thread completion synchronizes-with the successful return from the first function that is waiting on the shared state, or with the return of the last function that releases the shared state, whichever comes first. If std::decay<Function>::type or each type in std::decay<Ts>::type is not constructible from its corresponding argument, the program is ill-formed.

Parameters

- **f** – Callable object to call
- **ts** – parameters to pass to f

Returns

hpx::future referring to the shared state created by this call to *hpx::async*.

template<typename **Action**, typename **F**, typename ...**Ts**>

auto **hpx::async**(*F* &&f, *Ts*&&... ts) -> decltype(detail::async_action_dispatch<*Action*, *std*::decay_t<*F*>>::call(*std*::forward<*F*>(*f*), *std*::forward<*Ts*>(*ts*)...))

The function template *async* runs the function *f* asynchronously (potentially in a separate thread which might be a part of a thread pool) and returns a hpx::future that will eventually hold the result of that function call. If no policy is defined, *async* behaves as if it is called with policy being *hpx::launch::async* | hpx::launch::deferred. Otherwise, it calls a function *f* with arguments *ts* according to a specific launch policy.

- If the async flag is set (i.e. (policy & hpx::launch::async) != 0), then async executes the callable object f on a new thread of execution (with all thread-locals initialized) as if spawned by *hpx::thread*(std::forward<F>(f), std::forward<Ts>(ts)…), except that if the function f returns a value or throws an exception, it is stored in the shared state accessible through the *hpx::future* that async returns to the caller.
- If the deferred flag is set (i.e. (policy & hpx::launch::deferred) != 0), then async converts f and ts… the same way as by *hpx::thread* constructor, but does not spawn a new thread of execution. Instead, lazy evaluation is performed: the first call to a non-timed wait function on the *hpx::future* that async returned to the caller will cause the copy of f to be invoked (as a rvalue) with the copies of ts… (also passed as rvalues) in the current thread (which does not have to be the thread that originally called hpx::async). The result or exception is placed in the shared state associated with the future and only then it is made ready. All further accesses to the same *hpx::future* will return the result immediately.
- If neither hpx::launch::async nor hpx::launch::deferred, nor any implementation-defined policy flag is set in policy, the behavior is undefined.

  If more than one flag is set, it is implementation-defined which policy is selected. For the default (both the hpx::launch::async and hpx::launch::deferred flags are set in policy), standard recommends (but doesn't require) utilizing available concurrency, and deferring any additional tasks.

  In any case, the call to hpx::async synchronizes-with (as defined in std::memory_order) the call to f, and the completion of f is sequenced-before making the shared state ready. If the async policy is chosen, the associated thread completion synchronizes-with the successful return from the first function that is waiting on the shared state, or with the return of the last function that releases the shared state, whichever comes first. If std::decay<Function>::type or each type in std::decay<Ts>::type is not constructible from its corresponding argument, the program is ill-formed.

Parameters

- **f** – Callable object to call
- **ts** – parameters to pass to f

Returns

hpx::future referring to the shared state created by this call to *hpx::async*.

Warning

doxygenfunction: Unable to resolve function "hpx::async" with arguments (Executor&&, hpx::sycl::experimental::sycl_executor::queue_function_ptr_t<Ts…>&&, Ts&&…) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename Action,
↪ typename F, typename ...Ts> auto async(F &&f, Ts&&... ts)␣
↪-> decltype(detail::async_action_dispatch<Action, std::decay_
↪t<F>>::call(std::forward<F>(f), std::forward<Ts>(ts)...))
- template<typename Action, typename Target, typename␣
↪...Ts> auto async(Action &&action, Target &&target, Ts&&... ts)
- template<typename Executor, typename ...Ts> decltype(auto)␣
↪async(Executor &&exec, hpx::sycl::experimental::sycl_
↪executor::queue_function_ptr_t<Ts...> &&f, Ts&&... ts)
- template<typename␣
↪F, typename ...Ts> decltype(auto) async(F &&f, Ts&&... ts)
```

template<typename **Action**, typename **Target**, typename ...**Ts**>

auto *hpx*::**async**(*Action* &&action, *Target* &&target, *Ts*&&... ts)

The distributed implementation of `hpx::async` can be used by giving an action instance as argument instead of a function, and also by providing another argument with the locality ID or the target ID. The action executes asynchronously.

Template Parameters

- **Action** – The type of action instance
- **Target** – The type of target where the action should be executed
- **Ts** – The type of any additional arguments

Parameters

- **action** – The action instance to be executed
- **target** – The target where the action should be executed
- **ts** – Additional arguments

Returns

hpx::future referring to the shared state created by this call to `hpx::async`

async_combinators

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::wait_some

Defined in header hpx/future.hpp[821].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_some_nothrow*
- *hpx::wait_some_n*
- *hpx::wait_some_n_nothrow*

template<typename **InputIter**>

void *hpx*::**wait_some**(*std*::size_t n, *InputIter* first, *InputIter* last)

The function *wait_some* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns a new future object representing the same list of futures after n of them finished executing.

[821] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

> **Note**
>
> The function *wait_some* returns after *n* futures have become ready. All input futures are still valid after *wait_some* returns.

> **Note**
>
> The function wait_some will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_some_nothrow* instead.

Parameters

- **n** – [in] The number of futures out of the arguments which have to become ready in order for the function to return.
- **first** – [in] The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *when_all* should wait.
- **last** – [in] The iterator pointing to the last element of a sequence of *future* or shared_future objects for which *when_all* should wait.

template<typename **R**>

void *hpx*::**wait_some**(*std*::size_t n, *std*::vector<*future*<*R*>> &&futures)

The function *wait_some* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns a new future object representing the same list of futures after n of them finished executing.

> **Note**
>
> The function *wait_some* returns after *n* futures have become ready. All input futures are still valid after *wait_some* returns.

> **Note**
>
> The function wait_some will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_some_nothrow* instead.

Parameters

- **n** – [in] The number of futures out of the arguments which have to become ready in order for the returned future to get ready.
- **futures** – [in] A vector holding an arbitrary amount of *future* or shared_future objects for which *wait_some* should wait.

template<typename **R**, *std*::size_t **N**>

void *hpx*::**wait_some**(*std*::size_t n, *std*::array<*future*<*R*>, *N*> &&futures)

The function *wait_some* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns a new future object representing the same list of futures after n of them finished executing.

> **Note**
>
> The function *wait_some* returns after *n* futures have become ready. All input futures are still valid after *wait_some* returns.

> **Note**
>
> The function wait_some will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_some_nothrow* instead.

Parameters

- **n** – [in] The number of futures out of the arguments which have to become ready in order for the returned future to get ready.
- **futures** – [in] An array holding an arbitrary amount of *future* or shared_future objects for which *wait_some* should wait.

template<typename ...**T**>

void *hpx*::**wait_some**(*std*::size_t n, *T*&&... futures)

The function *wait_some* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns a new future object representing the same list of futures after n of them finished executing.

> **Note**
>
> The function *wait_all* returns after *n* futures have become ready. All input futures are still valid after *wait_some* returns.

> **Note**
>
> The function wait_some will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_some_nothrow* instead.

Parameters

- **n** – [in] The number of futures out of the arguments which have to become ready in order for the returned future to get ready.
- **futures** – [in] An arbitrary number of *future* or shared_future objects, possibly holding different types for which *wait_some* should wait.

hpx::wait_some_nothrow

Defined in header hpx/future.hpp[822].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_some*
- *hpx::wait_some_n*
- *hpx::wait_some_n_nothrow*

> **Warning**
>
> doxygenfunction: Cannot find function "hpx::wait_some_nothrow" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::wait_some_n

Defined in header hpx/future.hpp[823].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_some*
- *hpx::wait_some_nothrow*
- *hpx::wait_some_n_nothrow*

template<typename **InputIter**>

void *hpx*::**wait_some_n**(*std*::size_t n, *InputIter* first, *std*::size_t count)

The function *wait_some_n* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns a new future object representing the same list of futures after n of them finished executing.

> **Note**
>
> The function *wait_some_n* returns after *n* futures have become ready. All input futures are still valid after *wait_some_n* returns.

> **Note**
>
> The function wait_some_n will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_some_n_nothrow* instead.

822 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp
823 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

Parameters

- **n** – [in] The number of futures out of the arguments which have to become ready in order for the returned future to get ready.
- **first** – [in] The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *when_all* should wait.
- **count** – [in] The number of elements in the sequence starting at *first*.

hpx::wait_some_n_nothrow

Defined in header hpx/future.hpp[824].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_some*
- *hpx::wait_some_nothrow*
- *hpx::wait_some_n*

Warning

doxygenfunction: Cannot find function "hpx::wait_some_n_nothrow" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::wait_all

Defined in header hpx/future.hpp[825].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_all_nothrow*
- *hpx::wait_all_n*
- *hpx::wait_all_n_nothrow*

template<typename **InputIter**>

void hpx::**wait_all**(*InputIter* first, *InputIter* last)

The function *wait_all* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns after they finished executing.

[824] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp
[825] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

> **Note**
>
> The function *wait_all* returns after all futures have become ready. All input futures are still valid after *wait_all* returns.

> **Note**
>
> The function wait_all will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_all_nothrow* instead.

> **Note**
>
> The caller is responsible for keeping the *futures* container alive, unmoved, and unmodified until wait_all returns and all inspection of the futures is complete.

Parameters

- **first** – The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *wait_all* should wait.
- **last** – The iterator pointing to the element after the last one of a sequence of *future* or shared_future objects for which *wait_all* should wait.

template<typename **R**>

void *hpx*::**wait_all**(*std*::vector<*future*<*R*>> &&futures)

The function *wait_all* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns after they finished executing.

> **Note**
>
> The function *wait_all* returns after all futures have become ready. All input futures are still valid after *wait_all* returns.

> **Note**
>
> The function wait_all will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_all_nothrow* instead.

> **Note**
>
> The caller is responsible for keeping the *futures* container alive, unmoved, and unmodified until wait_all returns and all inspection of the futures is complete.

Parameters

futures – A vector or array holding an arbitrary amount of *future* or shared_future objects for which *wait_all* should wait.

template<typename **R**, *std*::size_t **N**>

void *hpx*::**wait_all**(*std*::array<*future*<*R*>, *N*> &&futures)

The function *wait_all* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns after they finished executing.

> **Note**
>
> The function *wait_all* returns after all futures have become ready. All input futures are still valid after *wait_all* returns.

> **Note**
>
> The function wait_all will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_all_nothrow* instead.

> **Note**
>
> The caller is responsible for keeping the *futures* container alive, unmoved, and unmodified until wait_all returns and all inspection of the futures is complete.

Parameters

futures – A vector or array holding an arbitrary amount of *future* or shared_future objects for which *wait_all* should wait.

template<typename **T**>

void *hpx*::**wait_all**(*hpx*::*future*<*T*> const &f)

The function *wait_all* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns after they finished executing.

> **Note**
>
> The function *wait_all* returns after the future has become ready. The input future is still valid after *wait_all* returns.

> **Note**
>
> The function wait_all will rethrow any exceptions captured by the future while becoming ready. If this behavior is undesirable, use *wait_all_nothrow* instead.

Parameters

f – A *future* or shared_future for which *wait_all* should wait.

template<typename ...**T**>

void *hpx*::**wait_all**(*T*&&... futures)

The function *wait_all* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns after they finished executing.

> **Note**
>
> The function *wait_all* returns after all futures have become ready. All input futures are still valid after *wait_all* returns.

> **Note**
>
> The function wait_all will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_all_nothrow* instead.

Parameters

futures – An arbitrary number of *future* or shared_future objects, possibly holding different types for which *wait_all* should wait.

hpx::wait_all_nothrow

Defined in header hpx/future.hpp[826].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_all*
- *hpx::wait_all_n*
- *hpx::wait_all_n_nothrow*

template<typename **InputIter**>

bool *hpx*::**wait_all_nothrow**(*InputIter* first, *InputIter* last)

The function *wait_all_nothrow* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns after they finished executing, similar to *wait_all*. It differs from *wait_all* in that it will not rethrow any exceptions captured by the given futures while they became ready.

> **Note**
>
> The function *wait_all_nothrow* returns after all futures have become ready. All input futures are still valid after *wait_all_nothrow* returns.

[826] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

> **Note**
>
> Unlike *wait_all*, this function will not rethrow any exceptions captured by the futures while becoming ready. Any such exceptions are not rethrown by this call; the caller can use the returned *bool* to detect their presence.

> **Note**
>
> The caller is responsible for keeping the *futures* container alive, unmoved, and unmodified until wait_all returns and all inspection of the futures is complete.

Parameters

- **`first`** – The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *wait_all_nothrow* should wait.
- **`last`** – The iterator pointing to the element after the last one of a sequence of *future* or shared_future objects for which *wait_all_nothrow* should wait.

Returns

Returns *true* if any of the given futures held an exception once it became ready, and *false* otherwise.

template<typename **R**>

bool *hpx*::**`wait_all_nothrow`**(*std*::vector<*future*<*R*>> &&futures)

The function *wait_all_nothrow* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns after they finished executing, similar to *wait_all*. It differs from *wait_all* in that it will not rethrow any exceptions captured by the given futures while they became ready.

> **Note**
>
> The function *wait_all_nothrow* returns after all futures have become ready. All input futures are still valid after *wait_all_nothrow* returns.

> **Note**
>
> Unlike *wait_all*, this function will not rethrow any exceptions captured by the futures while becoming ready. Any such exceptions are not rethrown by this call; the caller can use the returned *bool* to detect their presence.

Parameters

`futures` – A vector or array holding an arbitrary amount of *future* or shared_future objects for which *wait_all_nothrow* should wait.

Returns

Returns *true* if any of the given futures held an exception once it became ready, and *false* otherwise.

template<typename **R**, *std*::size_t **N**>

bool *hpx*::**wait_all_nothrow**(*std*::array<*future*<*R*>, *N*> &&futures)

The function *wait_all_nothrow* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns after they finished executing, similar to *wait_all*. It differs from *wait_all* in that it will not rethrow any exceptions captured by the given futures while they became ready.

> **Note**
>
> The function *wait_all_nothrow* returns after all futures have become ready. All input futures are still valid after *wait_all_nothrow* returns.

> **Note**
>
> Unlike *wait_all*, this function will not rethrow any exceptions captured by the futures while becoming ready. Any such exceptions are not rethrown by this call; the caller can use the returned *bool* to detect their presence.

Parameters

`futures` – A vector or array holding an arbitrary amount of *future* or shared_future objects for which *wait_all_nothrow* should wait.

Returns

Returns *true* if any of the given futures held an exception once it became ready, and *false* otherwise.

template<typename **T**>

bool *hpx*::**wait_all_nothrow**(*hpx*::*future*<*T*> const &f)

The function *wait_all_nothrow* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns after they finished executing, similar to *wait_all*. It differs from *wait_all* in that it will not rethrow any exceptions captured by the given future while it became ready.

> **Note**
>
> The function *wait_all_nothrow* returns after the future has become ready. The input future is still valid after *wait_all_nothrow* returns.

> **Note**
>
> Unlike *wait_all*, this function will not rethrow any exception captured by the future while becoming ready. Any such exception is silently discarded; the caller can use the returned *bool* to detect its presence.

Parameters

`f` – A *future* or shared_future for which *wait_all_nothrow* should wait.

Returns

Returns *true* if the given future held an exception once it became ready, and *false* otherwise.

template<typename ...T>

bool *hpx*::**wait_all_nothrow**(*T*&&... futures)

The function *wait_all_nothrow* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns after they finished executing, similar to *wait_all*. It differs from *wait_all* in that it will not rethrow any exceptions captured by the given futures while they became ready.

> **Note**
>
> The function *wait_all_nothrow* returns after all futures have become ready. All input futures are still valid after *wait_all_nothrow* returns.

> **Note**
>
> Unlike *wait_all*, this function will not rethrow any exceptions captured by the futures while becoming ready. Any such exceptions are not rethrown by this call; the caller can use the returned *bool* to detect their presence.

Parameters

`futures` – An arbitrary number of *future* or shared_future objects, possibly holding different types for which *wait_all_nothrow* should wait.

Returns

Returns *true* if any of the given futures held an exception once it became ready, and *false* otherwise.

hpx::wait_all_n

Defined in header hpx/future.hpp[827].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_all*
- *hpx::wait_all_nothrow*
- *hpx::wait_all_n_nothrow*

template<typename **InputIter**>

[827] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

void *hpx*::**wait_all_n**(*InputIter* begin, *std*::size_t count)

The function *wait_all_n* is an operator allowing to join on the result of all given futures.
It AND-composes all future objects given and returns after they finished executing.

> **Note**
>
> The function *wait_all_n* returns after all futures have become ready. All input futures are still valid after *wait_all_n* returns.

> **Note**
>
> The function wait_all_n will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_all_n_nothrow* instead.

> **Note**
>
> The caller is responsible for keeping the *futures* container alive, unmoved, and unmodified until wait_all returns and all inspection of the futures is complete.

Parameters

- **begin** – The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *wait_all_n* should wait.
- **count** – The number of elements in the sequence starting at *first*.

Returns

The function *wait_all_n* will return an iterator referring to the first element in the input sequence after the last processed element.

hpx::wait_all_n_nothrow

Defined in header hpx/future.hpp[828].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_all*
- *hpx::wait_all_nothrow*
- *hpx::wait_all_n*

template<typename **InputIter**>

std::pair<*InputIter*, bool> *hpx*::**wait_all_n_nothrow**(*InputIter* begin, *std*::size_t count)

The function *wait_all_n_nothrow* is an operator allowing to join on the result of all

[828] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

given futures. It AND-composes all future objects given and returns after they finished executing, similar to *wait_all_n*. It differs from *wait_all_n* in that it will not rethrow any exceptions captured by the given futures while they became ready.

Note

The function *wait_all_n_nothrow* returns after all futures have become ready. All input futures are still valid after *wait_all_n_nothrow* returns.

Note

Unlike *wait_all_n*, this function will not rethrow any exceptions captured by the futures while becoming ready. Any such exceptions are not rethrown by this call; the caller can use the returned *bool* to detect their presence.

Note

The caller is responsible for keeping the *futures* container alive, unmoved, and unmodified until wait_all returns and all inspection of the futures is complete.

Parameters

- **`begin`** – The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *wait_all_n_nothrow* should wait.
- **`count`** – The number of elements in the sequence starting at *first*.

Returns

The function *wait_all_n_nothrow* returns a pair consisting of an iterator referring to the first element in the input sequence after the last processed element, and a *bool* that is *true* if any of the given futures held an exception once it became ready, and *false* otherwise.

hpx::when_any

Defined in header hpx/future.hpp[829].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InputIter**, typename **Container** = vector<*future*<typename *std*::iterator_traits<*InputIter*>::value_type>>>

future<when_any_result<*Container*>> `hpx::`**`when_any`**(*InputIter* first, *InputIter* last)

function *when_any* creates a future object that becomes when at least one element in a set of *future* and shared_future objects becomes ready. It is a non-deterministic choice operator. It OR-composes all given future objects and returns a new future object representing the same list of futures after one future of that list finishes execution.

[829] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

Parameters

- **first** – [in] The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *when_any* should wait.
- **last** – [in] The iterator pointing to the last element of a sequence of *future* or shared_future objects for which *when_any* should wait.

Returns

Returns a when_any_result holding the same list of futures as has been passed to when_any and an index pointing to a ready future.

- future<when_any_result<Container<future<R>>>>: If the input cardinality is unknown at compile time and the futures are all have the same type. The order of the futures in the output container will be the same as given by the input iterator.

template<typename **Range**>

future<when_any_result<*Range*>> `hpx::`**`when_any`**(*Range* &values)

function *when_any* creates a future object that becomes when at least one element in a set of *future* and shared_future objects becomes ready. It is a non-deterministic choice operator. It OR-composes all given future objects and returns a new future object representing the same list of futures after one future of that list finishes execution.

Parameters

values – [in] A range holding an arbitrary amount of *futures* or shared_future objects for which *when_any* should wait.

Returns

Returns a when_any_result holding the same list of futures as has been passed to when_any and an index pointing to a ready future.

- future<when_any_result<Container<future<R>>>>: If the input cardinality is unknown at compile time and the futures are all have the same type. The order of the futures in the output container will be the same as given by the input iterator.

template<typename ...**T**>

future<when_any_result<*tuple*<*future*<*T*>...>>> `hpx::`**`when_any`**(*T*&&... futures)

function *when_any* creates a future object that becomes when at least one element in a set of *future* and shared_future objects becomes ready. It is a non-deterministic choice operator. It OR-composes all given future objects and returns a new future object representing the same list of futures after one future of that list finishes execution.

Parameters

futures – [in] An arbitrary number of *future* or shared_future objects, possibly holding different types for which *when_any* should wait.

Returns

Returns a when_any_result holding the same list of futures as has been passed to when_any and an index pointing to a ready future.

- future<when_any_result<tuple<future<T0>, future<T1>…>>>: If inputs are fixed in number and are of heterogeneous types. The inputs can be any arbitrary number of future objects.

- future<when_any_result<tuple<>>> if *when_any* is called with zero arguments. The returned future will be initially ready.

hpx/async_combinators/split_future.hpp

Defined in header hpx/async_combinators/split_future.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Top level HPX namespace.

Functions

template<typename ...**Ts**>

inline *tuple<future<Ts>...>* **split_future**(*future<tuple<Ts...>>* &&f)

The function *split_future* is an operator allowing to split a given future of a sequence of values (any tuple, std::pair, or std::array) into an equivalent container of futures where each future represents one of the values from the original future. In some sense this function provides the inverse operation of *when_all*.

Note

The following cases are special:

```
tuple<future<void> > split_future(future<tuple<> > && f);
array<future<void>, 1> split_future(future<array<T, 0> > && f);
```

here the returned futures are directly representing the futures which were passed to the function.

Parameters

f – [in] A future holding an arbitrary sequence of values stored in a tuple-like container. This facility supports *hpx::tuple<>*, *std::pair<T1, T2>*, and *std::array<T, N>*

Returns

Returns an equivalent container (same container type as passed as the argument) of futures, where each future refers to the corresponding value in the input parameter. All the returned futures become ready once the input future has become ready. If the input future is exceptional, all output futures will be exceptional as well.

template<typename **T**>

inline *std*::vector<*future*<*T*>> **split_future**(*future*<*std*::vector<*T*>> &&f, *std*::size_t size)

The function *split_future* is an operator allowing to split a given future of a sequence of values (any std::vector) into a std::vector of futures where each future represents one of the values from the original std::vector. In some sense this function provides the inverse operation of *when_all*.

Parameters

- **f** – [in] A future holding an arbitrary sequence of values stored in a std::vector.
- **size** – [in] The number of elements the vector will hold once the input future has become ready

Returns
Returns a std::vector of futures, where each future refers to the corresponding value in the input parameter. All the returned futures become ready once the input future has become ready. If the input future is exceptional, all output futures will be exceptional as well.

hpx::when_all

Defined in header hpx/future.hpp[830].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InputIter**, typename **Container** = vector<*future*<typename *std*::iterator_traits<*InputIter*>::value_type>>>

hpx::*future*<*Container*> **hpx::when_all**(*InputIter* first, *InputIter* last)

function *when_all* creates a future object that becomes ready when all elements in a set of *future* and shared_future objects become ready. It is an operator allowing to join on the result of all given futures. It AND-composes all given future objects and returns a new future object representing the same list of futures after they finished executing.

> **Note**
>
> Calling this version of *when_all* where first == last, returns a future with an empty container that is immediately ready. Each future and *shared_future* is waited upon and then copied into the collection of the output (returned) future, maintaining the order of the futures in the input collection. The future returned by *when_all* will not throw an exception, but the futures held in the output collection may.

Parameters

- **first** – [in] The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *when_all* should wait.
- **last** – [in] The iterator pointing to the last element of a sequence of *future* or shared_future objects for which *when_all* should wait.

Returns
Returns a future holding the same list of futures as has been passed to *when_all*.

- future<Container<future<R>>>: If the input cardinality is unknown at compile time and the futures are all have the same type. The order of the futures in the output container will be the same as given by the input iterator.

[830] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

template<typename **Range**>

hpx::future<Range> `hpx::when_all`(*Range* &&values)

function *when_all* creates a future object that becomes ready when all elements in a set of *future* and shared_future objects become ready. It is an operator allowing to join on the result of all given futures. It AND-composes all given future objects and returns a new future object representing the same list of futures after they finished executing.

> **Note**
>
> Calling this version of *when_all* where the input container is empty, returns a future with an empty container that is immediately ready. Each future and *shared_future* is waited upon and then copied into the collection of the output (returned) future, maintaining the order of the futures in the input collection. The future returned by *when_all* will not throw an exception, but the futures held in the output collection may.

Parameters

`values` – [in] A range holding an arbitrary amount of *future* or shared_future objects for which *when_all* should wait.

Returns

Returns a future holding the same list of futures as has been passed to when_all.

- future<Container<future<R>>>: If the input cardinality is unknown at compile time and the futures are all have the same type.

template<typename ...**T**>

hpx::future<hpx::tuple<hpx::future<T>...>> `hpx::when_all`(*T*&&... futures)

function *when_all* creates a future object that becomes ready when all elements in a set of *future* and shared_future objects become ready. It is an operator allowing to join on the result of all given futures. It AND-composes all given future objects and returns a new future object representing the same list of futures after they finished executing.

> **Note**
>
> Each future and *shared_future* is waited upon and then copied into the collection of the output (returned) future, maintaining the order of the futures in the input collection. The future returned by *when_all* will not throw an exception, but the futures held in the output collection may.

Parameters

`futures` – [in] An arbitrary number of *future* or shared_future objects, possibly holding different types for which *when_all* should wait.

Returns

Returns a future holding the same list of futures as has been passed to *when_all*.

- future<tuple<future<T0>, future<T1>, future<T2>…>>: If inputs are fixed in number and are of heterogeneous types. The inputs can be any arbitrary number of future objects.

- future<tuple<>> if *when_all* is called with zero arguments. The returned future will be initially ready.

hpx::when_some

Defined in header hpx/future.hpp[831].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **InputIter**, typename **Container** = vector<*future*<typename *std*::iterator_traits<*InputIter*>::value_type>>>

future<when_some_result<*Container*>> `hpx::when_some`(*std*::size_t n, Iterator first, Iterator last)

The function *when_some* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns a new future object representing the same list of futures after n of them finished executing.

> **Note**
>
> The future returned by the function *when_some* becomes ready when at least *n* argument futures have become ready.

> **Note**
>
> Calling this version of *when_some* where first == last, returns a future with an empty container that is immediately ready. Each future and *shared_future* is waited upon and then copied into the collection of the output (returned) future, maintaining the order of the futures in the input collection. The future returned by *when_some* will not throw an exception, but the futures held in the output collection may.

Parameters

- **n** – [in] The number of futures out of the arguments which have to become ready in order for the returned future to get ready.
- **first** – [in] The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *when_all* should wait.
- **last** – [in] The iterator pointing to the last element of a sequence of *future* or shared_future objects for which *when_all* should wait.

Returns

Returns a when_some_result holding the same list of futures as has been passed to when_some and indices pointing to ready futures.

- future<when_some_result<Container<future<R>>>>: If the input cardinality is unknown at compile time and the futures are all of the same type. The order of the futures in the output container will be the same as given by the input iterator.

template<typename **Range**>

[831] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

future<when_some_result<*Range*>> `hpx::when_some`(*std*::size_t n, *Range* &&futures)

The function *when_some* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns a new future object representing the same list of futures after n of them finished executing.

Note

The future returned by the function *when_some* becomes ready when at least *n* argument futures have become ready.

Note

Each future and *shared_future* is waited upon and then copied into the collection of the output (returned) future, maintaining the order of the futures in the input collection. The future returned by *when_some* will not throw an exception, but the futures held in the output collection may.

Parameters

- **n** – [in] The number of futures out of the arguments which have to become ready in order for the returned future to get ready.
- **futures** – [in] A container holding an arbitrary amount of *future* or shared_future objects for which *when_some* should wait.

Returns

Returns a when_some_result holding the same list of futures as has been passed to when_some and indices pointing to ready futures.

- future<when_some_result<Container<future<R>>>>: If the input cardinality is unknown at compile time and the futures are all of the same type. The order of the futures in the output container will be the same as given by the input iterator.

template<typename ...**Ts**>

future<when_some_result<*tuple*<*future*<T>...>>> `hpx::when_some`(*std*::size_t n, *Ts*&&... futures)

The function *when_some* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns a new future object representing the same list of futures after n of them finished executing.

Note

The future returned by the function *when_some* becomes ready when at least *n* argument futures have become ready.

Note

Each future and *shared_future* is waited upon and then copied into the collection of the output (returned) future, maintaining the order of the futures in the input collection. The future returned by *when_some* will not throw an exception, but the futures held in the output collection may.

Parameters

- **n** – [in] The number of futures out of the arguments which have to become ready in order for the returned future to get ready.
- **futures** – [in] An arbitrary number of *future* or shared_future objects, possibly holding different types for which *when_some* should wait.

Returns

Returns a when_some_result holding the same list of futures as has been passed to when_some and an index pointing to a ready future..

- future<when_some_result<tuple<future<T0>, future<T1>…>>>: If inputs are fixed in number and are of heterogeneous types. The inputs can be any arbitrary number of future objects.
- future<when_some_result<tuple<>>> if *when_some* is called with zero arguments. The returned future will be initially ready.

hpx::when_each

Defined in header hpx/future.hpp[832].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **F**, typename **Future**>

future<void> *hpx*::**when_each**(*F* &&f, *std*::vector<*Future*> &&futures)

The function *when_each* is an operator allowing to join on the results of all given futures. It AND-composes all future objects given and returns a new future object representing the event of all those futures having finished executing. It also calls the supplied callback for each of the futures which becomes ready.

Note

This function consumes the futures as they are passed on to the supplied function. The callback should take one or two parameters, namely either a *future* to be processed or a type that *std::size_t* is implicitly convertible to as the first parameter and the *future* as the second parameter. The first parameter will correspond to the index of the current *future* in the collection.

Parameters

- **f** – The function which will be called for each of the input futures once the future has become ready.

[832] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

- **futures** – A vector holding an arbitrary amount of *future* or shared_future objects for which *wait_each* should wait.

Returns
Returns a future representing the event of all input futures being ready.

template<typename **F**, typename **Iterator**>

future<*Iterator*> *hpx*::**when_each**(*F* &&f, *Iterator* begin, *Iterator* end)

The function *when_each* is an operator allowing to join on the results of all given futures. It AND-composes all future objects given and returns a new future object representing the event of all those futures having finished executing. It also calls the supplied callback for each of the futures which becomes ready.

> **Note**
>
> This function consumes the futures as they are passed on to the supplied function. The callback should take one or two parameters, namely either a *future* to be processed or a type that *std::size_t* is implicitly convertible to as the first parameter and the *future* as the second parameter. The first parameter will correspond to the index of the current *future* in the collection.

Parameters

- **f** – The function which will be called for each of the input futures once the future has become ready.
- **begin** – The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *wait_each* should wait.
- **end** – The iterator pointing to the last element of a sequence of *future* or shared_future objects for which *wait_each* should wait.

Returns
Returns a future representing the event of all input futures being ready.

template<typename **F**, typename ...**Ts**>

future<void> *hpx*::**when_each**(*F* &&f, *Ts*&&... futures)

The function *when_each* is an operator allowing to join on the results of all given futures. It AND-composes all future objects given and returns a new future object representing the event of all those futures having finished executing. It also calls the supplied callback for each of the futures which becomes ready.

> **Note**
>
> This function consumes the futures as they are passed on to the supplied function. The callback should take one or two parameters, namely either a *future* to be processed or a type that *std::size_t* is implicitly convertible to as the first parameter and the *future* as the second parameter. The first parameter will correspond to the index of the current *future* in the collection.

Parameters

- **f** – The function which will be called for each of the input futures once the future has become ready.
- **futures** – An arbitrary number of *future* or shared_future objects, possibly holding different types for which *wait_each* should wait.

Returns

Returns a future representing the event of all input futures being ready.

hpx::wait_all_for

Defined in header hpx/future.hpp[833].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_all_for_nothrow*
- *hpx::wait_all_for_n*
- *hpx::wait_all_for_n_nothrow*

template<typename **InputIter**>

hpx::future_status **hpx::wait_all_for**(*hpx*::chrono::steady_duration const &timeout, *InputIter* first, *InputIter* last)

The function *wait_all_for* is an operator allowing to join on the result of all given futures, similar to *wait_all*. It differs from *wait_all* in that it will not suspend indefinitely if one or more of the given futures never become ready; instead it returns as soon as either all futures have become ready, or the given timeout has elapsed, whichever happens first.

> **Note**
>
> The function *wait_all_for* returns after all futures have become ready, or after the given timeout has expired, whichever comes first. All input futures are still valid after *wait_all_for* returns, independently of the returned status, and can be inspected (e.g. using *is_ready()*) to determine which of the given futures have become ready.

> **Note**
>
> The function wait_all_for will rethrow any exceptions captured by the futures while becoming ready, as long as all the given futures became ready before the timeout elapsed. If this behavior is undesirable, use *wait_all_for_nothrow* instead.

Parameters

- **timeout** – The maximum duration to wait for all the given futures to become ready.

[833] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

- **first** – The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *wait_all_for* should wait.
- **last** – The iterator pointing to the element after the last one of a sequence of *future* or shared_future objects for which *wait_all_for* should wait.

Returns

Returns *hpx::future_status::ready* if all the given futures have become ready before *timeout* has elapsed, and *hpx::future_status::timeout* otherwise.

template<typename **R**>

hpx::future_status **hpx::wait_all_for**(*hpx*::chrono::steady_duration const &timeout, *std*::vector<*future*<*R*>> const &futures)

The function *wait_all_for* is an operator allowing to join on the result of all given futures, similar to *wait_all*. It differs from *wait_all* in that it will not suspend indefinitely if one or more of the given futures never become ready; instead it returns as soon as either all futures have become ready, or the given timeout has elapsed, whichever happens first.

Note

The function *wait_all_for* returns after all futures have become ready, or after the given timeout has expired, whichever comes first. All input futures are still valid after *wait_all_for* returns, independently of the returned status, and can be inspected (e.g. using *is_ready()*) to determine which of the given futures have become ready.

Note

The function wait_all_for will rethrow any exceptions captured by the futures while becoming ready, as long as all the given futures became ready before the timeout elapsed. If this behavior is undesirable, use *wait_all_for_nothrow* instead.

Note

The function wait_all_for returns after all futures have become ready, or after the given timeout has expired, whichever comes first. All input futures are still valid after wait_all_for returns, independently of the returned status, and can be inspected (e.g. using is_ready()) to determine which of the given futures have become ready.

Note

The caller is responsible for keeping the *futures* container itself (and, for the iterator-range overload, the underlying sequence) alive, unmoved, and unmodified until it is done inspecting the futures' state. If the timeout expires before all futures become ready, wait_all_for may leave asynchronous continuations attached to the not-yet-ready futures; destroying, moving, or reallocating the container before those futures settle results in undefined behavior.

Parameters

- **timeout** – The maximum duration to wait for all the given futures to become ready.
- **futures** – A vector or array holding an arbitrary amount of *future* or shared_future objects for which *wait_all_for* should wait.

Returns

Returns *hpx::future_status::ready* if all the given futures have become ready before *timeout* has elapsed, and *hpx::future_status::timeout* otherwise.

template<typename **R**, *std*::size_t **N**>

hpx::future_status **hpx::wait_all_for**(*hpx*::chrono::steady_duration const &timeout, *std*::array<*future*<*R*>, *N*> const &futures)

The function *wait_all_for* is an operator allowing to join on the result of all given futures, similar to *wait_all*. It differs from *wait_all* in that it will not suspend indefinitely if one or more of the given futures never become ready; instead it returns as soon as either all futures have become ready, or the given timeout has elapsed, whichever happens first.

> **Note**
>
> The function *wait_all_for* returns after all futures have become ready, or after the given timeout has expired, whichever comes first. All input futures are still valid after *wait_all_for* returns, independently of the returned status, and can be inspected (e.g. using *is_ready()*) to determine which of the given futures have become ready.

> **Note**
>
> The function wait_all_for will rethrow any exceptions captured by the futures while becoming ready, as long as all the given futures became ready before the timeout elapsed. If this behavior is undesirable, use *wait_all_for_nothrow* instead.

> **Note**
>
> The function wait_all_for returns after all futures have become ready, or after the given timeout has expired, whichever comes first. All input futures are still valid after wait_all_for returns, independently of the returned status, and can be inspected (e.g. using is_ready()) to determine which of the given futures have become ready.

> **Note**
>
> The caller is responsible for keeping the *futures* container itself (and, for the iterator-range overload, the underlying sequence) alive, unmoved, and unmodified until it is done inspecting the futures' state. If the timeout expires before all futures become ready, wait_all_for may leave asynchronous continuations attached to the not-yet-ready futures; destroying, moving, or reallocating the container before those futures settle results in undefined behavior.

Parameters

- **`timeout`** – The maximum duration to wait for all the given futures to become ready.
- **`futures`** – A vector or array holding an arbitrary amount of *future* or shared_future objects for which *wait_all_for* should wait.

Returns

Returns *hpx::future_status::ready* if all the given futures have become ready before *timeout* has elapsed, and *hpx::future_status::timeout* otherwise.

template<typename **T**>

hpx::future_status **`hpx::wait_all_for`**(*hpx*::chrono::steady_duration const &timeout, *hpx*::*future*<*T*> const &f)

The function *wait_all_for* is an operator allowing to join on the result of the given future, similar to *wait_all*. It differs from *wait_all* in that it will not suspend indefinitely if the given future never becomes ready; instead it returns as soon as either the future has become ready, or the given timeout has elapsed, whichever happens first.

> **Note**
>
> The function *wait_all_for* returns after the future has become ready, or after the given timeout has expired, whichever comes first. The input future is still valid after *wait_all_for* returns, independently of the returned status.

> **Note**
>
> The function wait_all_for will rethrow any exceptions captured by the future while becoming ready, as long as the given future became ready before the timeout elapsed. If this behavior is undesirable, use *wait_all_for_nothrow* instead.

Parameters

- **`timeout`** – The maximum duration to wait for the given future to become ready.
- **`f`** – A *future* or shared_future for which *wait_all_for* should wait.

Returns

Returns *hpx::future_status::ready* if the given future has become ready before *timeout* has elapsed, and *hpx::future_status::timeout* otherwise.

template<typename ...**T**>

hpx::future_status **hpx::wait_all_for**(*hpx*::chrono::steady_duration const &timeout, *T* const&... futures)

The function *wait_all_for* is an operator allowing to join on the result of all given futures, similar to *wait_all*. It differs from *wait_all* in that it will not suspend indefinitely if one or more of the given futures never become ready; instead it returns as soon as either all futures have become ready, or the given timeout has elapsed, whichever happens first.

Note

The function *wait_all_for* returns after all futures have become ready, or after the given timeout has expired, whichever comes first. All input futures are still valid after *wait_all_for* returns, independently of the returned status, and can be inspected (e.g. using *is_ready()*) to determine which of the given futures have become ready.

Note

The function wait_all_for will rethrow any exceptions captured by the futures while becoming ready, as long as all the given futures became ready before the timeout elapsed. If this behavior is undesirable, use *wait_all_for_nothrow* instead.

Parameters

- **timeout** – The maximum duration to wait for all the given futures to become ready.
- **futures** – An arbitrary number of *future* or shared_future objects, possibly holding different types for which *wait_all_for* should wait.

Returns

Returns *hpx::future_status::ready* if all the given futures have become ready before *timeout* has elapsed, and *hpx::future_status::timeout* otherwise.

hpx::wait_all_for_nothrow

Defined in header hpx/future.hpp[834].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_all_for*
- *hpx::wait_all_for_n*
- *hpx::wait_all_for_n_nothrow*

template<typename **InputIter**>

[834] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

hpx::wait_all_for_nothrow_result **hpx::wait_all_for_nothrow**(*hpx*::chrono::steady_duration const &timeout, *InputIter* first, *InputIter* last)

The function *wait_all_for_nothrow* is an operator allowing to join on the result of all given futures, similar to *wait_all_for*. It differs from *wait_all_for* in that it will not rethrow any exceptions captured by the given futures while they became ready; instead it returns as soon as either all futures have become ready, or the given timeout has elapsed, whichever happens first.

> **Note**
>
> The function *wait_all_for_nothrow* returns after all futures have become ready, or after the given timeout has expired, whichever comes first. All input futures are still valid after *wait_all_for_nothrow* returns, independently of the returned status, and can be inspected (e.g. using *is_ready()*) to determine which of the given futures have become ready.

> **Note**
>
> Unlike *wait_all_for_nothrow*, this function will not rethrow any exceptions captured by the futures while becoming ready. Any such exceptions are not rethrown by this call.

Parameters

- **timeout** – The maximum duration to wait for all the given futures to become ready.
- **first** – The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *wait_all_for_nothrow* should wait.
- **last** – The iterator pointing to the element after the last one of a sequence of *future* or shared_future objects for which *wait_all_for_nothrow* should wait.

Returns

Returns a *wait_all_for_nothrow_result* object whose *status* member is *hpx::future_status::ready* if all the given futures have become ready before *timeout* has elapsed, and *hpx::future_status::timeout* otherwise; and whose *has_exceptional_results* member indicates whether any of the futures that became ready had captured an exception.

template<typename **R**>

hpx::wait_all_for_nothrow_result **hpx::wait_all_for_nothrow**(*hpx*::chrono::steady_duration const &timeout, *std*::vector<*future*<*R*>> const &futures)

The function *wait_all_for_nothrow* is an operator allowing to join on the result of all given futures, similar to *wait_all_for*. It differs from *wait_all_for* in that it will not rethrow any exceptions captured by the given futures while they became ready; instead it returns as soon as either all futures have become ready, or the given timeout has elapsed, whichever happens first.

> **Note**
>
> The function *wait_all_for_nothrow* returns after all futures have become ready, or after

the given timeout has expired, whichever comes first. All input futures are still valid after *wait_all_for_nothrow* returns, independently of the returned status, and can be inspected (e.g. using *is_ready()*) to determine which of the given futures have become ready.

Note

Unlike *wait_all_for*, this function will not rethrow any exceptions captured by the futures while becoming ready. Any such exceptions are not rethrown by this call.

Note

The function wait_all_for_nothrow returns after all futures have become ready, or after the given timeout has expired, whichever comes first. All input futures are still valid after wait_all_for returns, independently of the returned status, and can be inspected (e.g. using is_ready()) to determine which of the given futures have become ready.

Note

The caller is responsible for keeping the *futures* container itself (and, for the iterator-range overload, the underlying sequence) alive, unmoved, and unmodified until it is done inspecting the futures' state. If the timeout expires before all futures become ready, wait_all_for may leave asynchronous continuations attached to the not-yet-ready futures; destroying, moving, or reallocating the container before those futures settle results in undefined behavior.

Parameters

- **`timeout`** – The maximum duration to wait for all the given futures to become ready.
- **`futures`** – A vector or array holding an arbitrary amount of *future* or shared_future objects for which *wait_all_for_nothrow* should wait.

Returns

Returns a *wait_all_for_nothrow_result* object whose *status* member is *hpx::future_status::ready* if all the given futures have become ready before *timeout* has elapsed, and *hpx::future_status::timeout* otherwise; and whose *has_exceptional_results* member indicates whether any of the futures that became ready had captured an exception.

template<typename **R**, *std*::size_t **N**>

hpx::wait_all_for_nothrow_result **`hpx::wait_all_for_nothrow`**(*hpx*::chrono::steady_duration const &timeout, *std*::array<*future*<*R*>, *N*> const &futures)

The function *wait_all_for_nothrow* is an operator allowing to join on the result of all given futures, similar to *wait_all_for*. It differs from *wait_all_for* in that it will not rethrow any exceptions captured by the given futures while they became ready; instead it returns as soon as either all futures have become ready, or the given timeout has elapsed, whichever happens first.

> **Note**
>
> The function *wait_all_for_nothrow* returns after all futures have become ready, or after the given timeout has expired, whichever comes first. All input futures are still valid after *wait_all_for_nothrow* returns, independently of the returned status, and can be inspected (e.g. using *is_ready()*) to determine which of the given futures have become ready.

> **Note**
>
> Unlike *wait_all_for*, this function will not rethrow any exceptions captured by the futures while becoming ready. Any such exceptions are not rethrown by this call.

> **Note**
>
> The function wait_all_for_nothrow returns after all futures have become ready, or after the given timeout has expired, whichever comes first. All input futures are still valid after wait_all_for returns, independently of the returned status, and can be inspected (e.g. using is_ready()) to determine which of the given futures have become ready.

> **Note**
>
> The caller is responsible for keeping the *futures* container itself (and, for the iterator-range overload, the underlying sequence) alive, unmoved, and unmodified until it is done inspecting the futures' state. If the timeout expires before all futures become ready, wait_all_for may leave asynchronous continuations attached to the not-yet-ready futures; destroying, moving, or reallocating the container before those futures settle results in undefined behavior.

Parameters

- **`timeout`** – The maximum duration to wait for all the given futures to become ready.
- **`futures`** – A vector or array holding an arbitrary amount of *future* or shared_future objects for which *wait_all_for_nothrow* should wait.

Returns

Returns a *wait_all_for_nothrow_result* object whose *status* member is *hpx::future_status::ready* if all the given futures have become ready before *timeout* has elapsed, and *hpx::future_status::timeout* otherwise; and whose *has_exceptional_results* member indicates whether any of the futures that became ready had captured an exception.

template<typename **T**>

hpx::wait_all_for_nothrow_result **`hpx::wait_all_for_nothrow`**(*hpx*::chrono::steady_duration const &timeout, *hpx*::*future*<*T*> const &f)

The function *wait_all_for_nothrow* is an operator allowing to join on the result of the given future, similar to *wait_all_for*. It differs from *wait_all_for* in that it will not rethrow any exceptions captured by the given future while it became ready; instead it returns as soon as either the future has become ready, or the given timeout has elapsed, whichever happens first.

> **Note**
>
> The function *wait_all_for_nothrow* returns after the future has become ready, or after the given timeout has expired, whichever comes first. The input future is still valid after *wait_all_for_nothrow* returns, independently of the returned status.

> **Note**
>
> Unlike *wait_all_for*, this function will not rethrow any exceptions captured by the future while becoming ready. Any such exception is silently discarded.

Parameters

- **timeout** – The maximum duration to wait for the given future to become ready.
- **f** – A *future* or shared_future for which *wait_all_for_nothrow* should wait.

Returns

Returns a *wait_all_for_nothrow_result* object whose *status* member is *hpx::future_status::ready* if all the given futures have become ready before *timeout* has elapsed, and *hpx::future_status::timeout* otherwise; and whose *has_exceptional_results* member indicates whether any of the futures that became ready had captured an exception.

template<typename ...**T**>

hpx::wait_all_for_nothrow_result **hpx::wait_all_for_nothrow**(*hpx*::chrono::steady_duration const &timeout, *T* const&... futures)

The function *wait_all_for_nothrow* is an operator allowing to join on the result of all given futures, similar to *wait_all_for*. It differs from *wait_all_for* in that it will not rethrow any exceptions captured by the given futures while they became ready; instead it returns as soon as either all futures have become ready, or the given timeout has elapsed, whichever happens first.

> **Note**
>
> The function *wait_all_for_nothrow* returns after all futures have become ready, or after the given timeout has expired, whichever comes first. All input futures are still valid after *wait_all_for_nothrow* returns, independently of the returned status, and can be inspected (e.g. using *is_ready()*) to determine which of the given futures have become ready.

> **Note**
>
> Unlike *wait_all_for*, this function will not rethrow any exceptions captured by the futures while becoming ready. Any such exceptions are not rethrown by this call.

Parameters

- **timeout** – The maximum duration to wait for all the given futures to become ready.
- **futures** – An arbitrary number of *future* or shared_future objects, possibly holding different types for which *wait_all_for_nothrow* should wait.

Returns

Returns a *wait_all_for_nothrow_result* object whose *status* member is *hpx::future_status::ready* if all the given futures have become ready before *timeout* has elapsed, and *hpx::future_status::timeout* otherwise; and whose *has_exceptional_results* member indicates whether any of the futures that became ready had captured an exception.

hpx::wait_all_for_n

Defined in header hpx/future.hpp[835].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_all_for*
- *hpx::wait_all_for_nothrow*
- *hpx::wait_all_for_n_nothrow*

template<typename **InputIter**>

hpx::future_status **hpx::wait_all_for_n**(*hpx*::chrono::steady_duration const &timeout, *InputIter* begin, *std*::size_t count)

The function *wait_all_for_n* is an operator allowing to join on the result of all given futures, similar to *wait_all_n*. It differs from *wait_all_n* in that it will not suspend indefinitely if one or more of the given futures never become ready; instead it returns as soon as either all futures have become ready, or the given timeout has elapsed, whichever happens first.

> **Note**
>
> The function *wait_all_for_n* returns after all futures have become ready, or after the given timeout has expired, whichever comes first. All input futures are still valid after *wait_all_for_n* returns, independently of the returned status, and can be inspected (e.g. using *is_ready()*) to determine which of the given futures have become ready.

> **Note**
>
> The function wait_all_for_n will rethrow any exceptions captured by the futures while becoming ready, as long as all the given futures became ready before the timeout elapsed. If this behavior is undesirable, use *wait_all_for_n_nothrow* instead.

[835] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

Parameters

- **timeout** – The maximum duration to wait for all the given futures to become ready.
- **begin** – The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *wait_all_for_n* should wait.
- **count** – The number of elements in the sequence starting at *first*.

Returns

Returns *hpx::future_status::ready* if all the given futures have become ready before *timeout* has elapsed, and *hpx::future_status::timeout* otherwise.

hpx::wait_all_for_n_nothrow

Defined in header hpx/future.hpp[836].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_all_for*
- *hpx::wait_all_for_nothrow*
- *hpx::wait_all_for_n*

template<typename **InputIter**>

hpx::wait_all_for_nothrow_result **hpx::wait_all_for_n_nothrow**(*hpx*::chrono::steady_duration const &timeout, *InputIter* begin, *std*::size_t count)

The function *wait_all_for_n_nothrow* is an operator allowing to join on the result of all given futures, similar to *wait_all_n*. It differs from *wait_all_for_n* in that it will not rethrow any exceptions captured by the given futures while they became ready; instead it returns as soon as either all futures have become ready, or the given timeout has elapsed, whichever happens first.

> **Note**
>
> The function *wait_all_for_n_nothrow* returns after all futures have become ready, or after the given timeout has expired, whichever comes first. All input futures are still valid after *wait_all_for_n_nothrow* returns, independently of the returned status, and can be inspected (e.g. using *is_ready()*) to determine which of the given futures have become ready.

> **Note**
>
> Unlike *wait_all_for_n*, this function will not rethrow any exceptions captured by the futures while becoming ready. Any such exceptions are not rethrown by this call.

Parameters

[836] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

- **timeout** – The maximum duration to wait for all the given futures to become ready.
- **begin** – The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *wait_all_for_n_nothrow* should wait.
- **count** – The number of elements in the sequence starting at *first*.

Returns

Returns a *wait_all_for_nothrow_result* object whose *status* member is *hpx::future_status::ready* if all the given futures have become ready before *timeout* has elapsed, and *hpx::future_status::timeout* otherwise; and whose *has_exceptional_results* member indicates whether any of the futures that became ready had captured an exception.

hpx::wait_each

Defined in header hpx/future.hpp[837].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_each_nothrow*
- *hpx::wait_each_n*
- *hpx::wait_each_n_nothrow*

template<typename **F**, typename **Future**>

void *hpx*::**wait_each**(*F* &&f, *std*::vector<*Future*> &&futures)

The function *wait_each* is an operator allowing to join on the results of all given futures. It AND-composes all future objects given and returns after they finished executing. Additionally, the supplied function is called for each of the passed futures as soon as the future has become ready. *wait_each* returns after all futures have been become ready.

> **Note**
>
> This function consumes the futures as they are passed on to the supplied function. The callback should take one or two parameters, namely either a *future* to be processed or a type that *std::size_t* is implicitly convertible to as the first parameter and the *future* as the second parameter. The first parameter will correspond to the index of the current *future* in the collection.

Parameters

- **f** – The function which will be called for each of the input futures once the future has become ready.
- **futures** – A vector holding an arbitrary amount of *future* or shared_future objects for which *wait_each* should wait.

template<typename **F**, typename **Iterator**>

[837] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

void *hpx*::**wait_each**(*F* &&f, *Iterator* begin, *Iterator* end)

The function *wait_each* is an operator allowing to join on the results of all given futures. It AND-composes all future objects given and returns after they finished executing. Additionally, the supplied function is called for each of the passed futures as soon as the future has become ready. *wait_each* returns after all futures have been become ready.

> **Note**
>
> This function consumes the futures as they are passed on to the supplied function. The callback should take one or two parameters, namely either a *future* to be processed or a type that *std::size_t* is implicitly convertible to as the first parameter and the *future* as the second parameter. The first parameter will correspond to the index of the current *future* in the collection.

Parameters

- **f** – The function which will be called for each of the input futures once the future has become ready.
- **begin** – The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *wait_each* should wait.
- **end** – The iterator pointing to the last element of a sequence of *future* or shared_future objects for which *wait_each* should wait.

template<typename **F**, typename ...**T**>

void *hpx*::**wait_each**(*F* &&f, *T*&&... futures)

The function *wait_each* is an operator allowing to join on the results of all given futures. It AND-composes all future objects given and returns after they finished executing. Additionally, the supplied function is called for each of the passed futures as soon as the future has become ready. *wait_each* returns after all futures have been become ready.

> **Note**
>
> This function consumes the futures as they are passed on to the supplied function. The callback should take one or two parameters, namely either a *future* to be processed or a type that *std::size_t* is implicitly convertible to as the first parameter and the *future* as the second parameter. The first parameter will correspond to the index of the current *future* in the collection.

Parameters

- **f** – The function which will be called for each of the input futures once the future has become ready.
- **futures** – An arbitrary number of *future* or shared_future objects, possibly holding different types for which *wait_each* should wait.

hpx::wait_each_nothrow

Defined in header hpx/future.hpp[838].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_each*
- *hpx::wait_each_n*
- *hpx::wait_each_n_nothrow*

Warning

doxygenfunction: Cannot find function “hpx::wait_each_nothrow” in doxygen xml output for project “hpx” from directory: ../hpx_autodoc

hpx::wait_each_n

Defined in header hpx/future.hpp[839].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_each*
- *hpx::wait_each_nothrow*
- *hpx::wait_each_n_nothrow*

template<typename **F**, typename **Iterator**>

void *hpx*::**wait_each_n**(*F* &&f, *Iterator* begin, *std*::size_t count)

The function *wait_each* is an operator allowing to join on the result of all given futures. It AND-composes all future objects given and returns after they finished executing. Additionally, the supplied function is called for each of the passed futures as soon as the future has become ready.

Note

This function consumes the futures as they are passed on to the supplied function. The callback should take one or two parameters, namely either a *future* to be processed or a type that *std::size_t* is implicitly convertible to as the first parameter and the *future* as the second parameter. The first parameter will correspond to the index of the current *future* in the collection.

Parameters

[838] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp
[839] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

- **f** – The function which will be called for each of the input futures once the future has become ready.
- **begin** – The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *wait_each_n* should wait.
- **count** – The number of elements in the sequence starting at *first*.

hpx::wait_each_n_nothrow

Defined in header hpx/future.hpp[840].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_each*
- *hpx::wait_each_nothrow*
- *hpx::wait_each_n*

Warning

doxygenfunction: Cannot find function "hpx::wait_each_n_nothrow" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::wait_any

Defined in header hpx/future.hpp[841].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_any_nothrow*
- *hpx::wait_any_n*
- *hpx::wait_any_n_nothrow*

```
template<typename InputIter>
void hpx::wait_any(InputIter first, InputIter last)
```

The function *wait_any* is a non-deterministic choice operator. It OR-composes all future objects given and returns after one future of that list finishes execution.

Note

The function *wait_any* returns after at least one future has become ready. All input futures are still valid after *wait_any* returns.

[840] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp
[841] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

> **Note**
>
> The function wait_any will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_any_nothrow* instead.

Parameters

- **first** – [in] The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *wait_any* should wait.
- **last** – [in] The iterator pointing to the last element of a sequence of *future* or shared_future objects for which *wait_any* should wait.

template<typename **R**>

void *hpx*::**wait_any**(*std*::vector<*future*<*R*>> &futures)

The function *wait_any* is a non-deterministic choice operator. It OR-composes all future objects given and returns after one future of that list finishes execution.

> **Note**
>
> The function *wait_any* returns after at least one future has become ready. All input futures are still valid after *wait_any* returns.

> **Note**
>
> The function wait_any will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_any_nothrow* instead.

Parameters

futures – [in] A vector holding an arbitrary amount of *future* or shared_future objects for which *wait_any* should wait.

template<typename **R**, *std*::size_t **N**>

void *hpx*::**wait_any**(*std*::array<*future*<*R*>, *N*> &futures)

The function *wait_any* is a non-deterministic choice operator. It OR-composes all future objects given and returns after one future of that list finishes execution.

> **Note**
>
> The function *wait_any* returns after at least one future has become ready. All input futures are still valid after *wait_any* returns.

> **Note**
>
> The function wait_any will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_any_nothrow* instead.

Parameters

futures – [in] Amn array holding an arbitrary amount of *future* or shared_future objects for which *wait_any* should wait.

template<typename ...**T**>

void *hpx*::**wait_any**(*T*&&... futures)

The function *wait_any* is a non-deterministic choice operator. It OR-composes all future objects given and returns after one future of that list finishes execution.

> **Note**
>
> The function *wait_any* returns after at least one future has become ready. All input futures are still valid after *wait_any* returns.

> **Note**
>
> The function wait_any will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_any_nothrow* instead.

Parameters

futures – [in] An arbitrary number of *future* or shared_future objects, possibly holding different types for which *wait_any* should wait.

hpx::wait_any_nothrow

Defined in header hpx/future.hpp[842].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_any*
- *hpx::wait_any_n*
- *hpx::wait_any_n_nothrow*

> **Warning**
>
> doxygenfunction: Cannot find function "hpx::wait_any_nothrow" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

[842] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

hpx::wait_any_n

Defined in header hpx/future.hpp[843].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_any*
- *hpx::wait_any_nothrow*
- *hpx::wait_any_n_nothrow*

template<typename **InputIter**>

void hpx::**wait_any_n**(*InputIter* first, *std*::size_t count)

The function *wait_any_n* is a non-deterministic choice operator. It OR-composes all future objects given and returns after one future of that list finishes execution.

> **Note**
>
> The function *wait_any_n* returns after at least one future has become ready. All input futures are still valid after *wait_any_n* returns.

> **Note**
>
> The function wait_any_n will rethrow any exceptions captured by the futures while becoming ready. If this behavior is undesirable, use *wait_any_n_nothrow* instead.

Parameters

- **first** – [in] The iterator pointing to the first element of a sequence of *future* or shared_future objects for which *wait_any_n* should wait.
- **count** – [in] The number of elements in the sequence starting at *first*.

hpx::wait_any_n_nothrow

Defined in header hpx/future.hpp[844].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::wait_any*
- *hpx::wait_any_nothrow*
- *hpx::wait_any_n*

[843] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp
[844] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

Warning

doxygenfunction: Cannot find function "hpx::wait_any_n_nothrow" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

async_cuda

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/async_cuda/cublas_executor.hpp

Defined in header hpx/async_cuda/cublas_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/async_cuda/cuda_executor.hpp

Defined in header hpx/async_cuda/cuda_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **cuda**

namespace **experimental**

struct **cuda_executor_base**

Subclassed by *hpx::cuda::experimental::cuda_executor*

Public Types

using **future_type** = *hpx*::*future*<void>

Public Functions

inline **cuda_executor_base**(*std*::size_t device, bool const event_mode)

inline *future_type* **get_future**() const

Protected Attributes

int **device_**

bool **event_mode_**

cudaStream_t **stream_**

std::shared_ptr<*hpx*::*cuda*::*experimental*::target> **target_**

struct **cuda_executor** : public *hpx*::*cuda*::*experimental*::*cuda_executor_base*

Public Functions

inline explicit **cuda_executor**(*std*::size_t const device, bool const event_mode = true)

inline **~cuda_executor**()

template<typename **F**, typename ...**Ts**>

inline void **post**(*F* &&f, *Ts*&&... ts) const

template<typename **F**, typename ...**Ts**>

inline decltype(auto) **async_execute**(*F* &&f, *Ts*&&... ts) const

Protected Functions

template<typename **R**, typename ...**Params**, typename ...**Args**>

inline void **post_impl**(*R* (*cuda_function)(*Params*...), *Args*&&... args) const

template<typename **R**, typename ...**Params**, typename ...**Args**>

inline *hpx*::*future*<void> **async_impl**(*R* (*cuda_kernel)(*Params*...), *Args*&&... args) const

namespace **execution**

namespace **experimental**

async_mpi

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/async_mpi/transform_mpi.hpp

Defined in header hpx/async_mpi/transform_mpi.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **mpi**

namespace **experimental**

Variables

constexpr struct *hpx*::*mpi*::*experimental*::*transform_mpi_t* **transform_mpi**

struct **transform_mpi_t**

Public Functions

template<typename ...**Args**>

inline constexpr decltype(auto) **operator()**(*Args*&&... args) const

template<typename **Sender**, typename **F**>

inline constexpr auto **operator()**(*Sender* &&s, *F* &&f) const

template<typename **F**>

inline constexpr auto **operator()**(*F* &&f) const

hpx/async_mpi/mpi_executor.hpp

Defined in header hpx/async_mpi/mpi_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **mpi**

namespace **experimental**

struct **executor**

Public Types

using **execution_category** = *hpx*::*execution*::*parallel_execution_tag*

using **executor_parameters_type** = *hpx*::*execution*::*experimental*::*default_parameters*

Public Functions

inline explicit constexpr **executor**(MPI_Comm communicator = MPI_COMM_WORLD)

template<typename **F**, typename ...**Ts**>

inline decltype(auto) **async_execute**(*F* &&f, *Ts*&&... ts) const

inline *std*::size_t **in_flight_estimate**() const

Private Members

MPI_Comm **communicator_**

async_sycl

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/async_sycl/sycl_executor.hpp

Defined in header hpx/async_sycl/sycl_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

template<typename **Executor**, typename ...**Ts**>

decltype(auto) **async**(*Executor* &&exec, *hpx*::*sycl*::*experimental*::*sycl_executor*::*queue_function_ptr_t*<*Ts*...> &&f, *Ts*&&... ts)

hpx::async overload for launching sycl queue member functions with an sycl executor

template<typename **Executor**, typename ...**Ts**>

bool **apply**(*Executor* &&exec, *hpx*::*sycl*::*experimental*::*sycl_executor*::*queue_function_ptr_t*<*Ts*...> &&f, *Ts*&&... ts)

hpx::apply overload for launching sycl queue member functions with an sycl executor

namespace **execution**

namespace **experimental**

namespace **sycl**

namespace **experimental**

struct **sycl_executor**

Public Types

using **future_type** = *hpx*::*future*<void>

template<typename ...**Params**>

using **queue_function_ptr_t** = ::sycl::event (::sycl::queue::*)(*std*::conditional_t<*std*::is_trivial_v<*std*::remove_reference_t<*Params*>>, *std*::decay_t<*Params*>, *Params*>...)

Default Implementation without the extra intel code_location parameter. Removes the reference for trivial types to make the function matching easier (see sycl_stream.cpp test)

Public Functions

template<typename **sycl_device_selector**>

inline explicit **sycl_executor**(*sycl_device_selector* const &selector)

Create a SYCL executor (based on a sycl queue)

~sycl_executor() = default

inline *future_type* **get_future**()

Get future for this command_queue (NOTE will be more efficient if an event is provided — otherwise a dummy kernel must be submitted to get an event) inline *future_type* **get_future**(::sycl::event event)

Get future for that becomes ready when the given event completes.

template<typename ...**Params**>

inline void **post**(*queue_function_ptr_t<Params...>* &&queue_member_function, *Params*&&... args)

Invoke queue member function given queue and parameters — do not use event to return a hpx::future (One way)

template<typename ...**Params**>

inline *hpx*::*future*<void> **async_execute**(*queue_function_ptr_t<Params...>* &&queue_member_function, *Params*&&... args)

Invoke queue member function given queue and parameters — hpx::future tied to the sycl event / (two way)

inline ::sycl::device **get_device**() const

Return the device used by the underlying SYCL queue.

inline ::sycl::context **get_context**() const

Return the context used by the underlying SYCL queue.

inline ::sycl::queue &**get_queue**()

Return the underlying queue for direct access.

Protected Attributes

::sycl::queue **command_queue**

cache

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/cache/local_cache.hpp

Defined in header hpx/cache/local_cache.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

namespace **cache**

template<typename **Key**, typename **Entry**, typename **UpdatePolicy** = *std*::less<*Entry*>, typename **InsertPolicy** = policies::always<*Entry*>, typename **CacheStorage** = *std*::map<*Key*, *Entry*>, typename **Statistics** = *statistics*::*no_statistics*>

class **local_cache**

#include <hpx/cache/local_cache.hpp> The local_cache implements the basic functionality needed for a local (non-distributed) cache.

Template Parameters

- `Key` – The type of the keys to use to identify the entries stored in the cache
- `Entry` – The type of the items to be held in the cache, must model the CacheEntry concept
- `UpdatePolicy` – A (optional) type specifying a (binary) function object used to sort the cache entries based on their 'age'. The 'oldest' entries (according to this sorting criteria) will be discarded first if the maximum capacity of the cache is reached. The default is std::less<Entry>. The function object will be invoked using 2 entry instances of the type *Entry*. This type must model the UpdatePolicy model.
- `InsertPolicy` – A (optional) type specifying a (unary) function object used to allow global decisions whether a particular entry should be added to the cache or not. The default is *policies::always*, imposing no global insert related criteria on the cache. The function object will be invoked using the entry instance to be inserted into the cache. This type must model the InsertPolicy model.
- `CacheStorage` – A (optional) container type used to store the cache items. The container must be an associative and STL compatible container.The default is a std::map<Key, Entry>.
- `Statistics` – A (optional) type allowing to collect some basic statistics about the operation of the cache instance. The type must conform to the CacheStatistics concept. The default value is the type statistics::no_statistics which does not collect any numbers, but provides empty stubs allowing the code to compile.

Public Types

using **key_type** = *Key*

using **entry_type** = *Entry*

using **update_policy_type** = *UpdatePolicy*

using **insert_policy_type** = *InsertPolicy*

using **storage_type** = *CacheStorage*

using **statistics_type** = *Statistics*

using **value_type** = typename *entry_type*::value_type

using **size_type** = typename *storage_type*::size_type

using **storage_value_type** = typename *storage_type*::value_type

Public Functions

inline explicit **local_cache**(*size_type* max_size = 0, *update_policy_type* const &up = *update_policy_type*(), *insert_policy_type* const &ip = *insert_policy_type*())

Construct an instance of a *local_cache*.

Parameters

- **max_size** – [in] The maximal size this cache is allowed to reach any time. The default is zero (no size limitation). The unit of this value is usually determined by the unit of the values returned by the entry's *get_size* function.
- **up** – [in] An instance of the *UpdatePolicy* to use for this cache. The default is to use a default constructed instance of the type as defined by the *UpdatePolicy* template parameter.
- **ip** – [in] An instance of the *InsertPolicy* to use for this cache. The default is to use a default constructed instance of the type as defined by the *InsertPolicy* template parameter.

local_cache(*local_cache* const &other) = default

local_cache(*local_cache* &&other) = default

local_cache &**operator=**(*local_cache* const &other) = default

local_cache &**operator=**(*local_cache* &&other) = default

~local_cache() = default

inline constexpr *size_type* **size**() const noexcept

Return current size of the cache.

Returns

The current size of this cache instance.

inline constexpr *size_type* **capacity**() const noexcept

Access the maximum size the cache is allowed to grow to.

> **Note**
>
> The unit of this value is usually determined by the unit of the return values of the entry's function *entry::get_size*.

Returns

The maximum size this cache instance is currently allowed to reach. If this number is zero the cache has no limitation with regard to a maximum size.

inline bool **reserve**(*size_type* max_size)

Change the maximum size this cache can grow to.

Parameters

max_size – [in] The new maximum size this cache will be allowed to grow to.

Returns

This function returns *true* if successful. It returns *false* if the new *max_size* is smaller than the current limit and the cache could not be shrunk to the new maximum size.

inline bool **holds_key**(*key_type* const &k) const

Check whether the cache currently holds an entry identified by the given key.

> **Note**
>
> This function does not call the entry's function *entry::touch*. It just checks if the cache contains an entry corresponding to the given key.

Parameters

k – [in] The key for the entry which should be looked up in the cache.

Returns

This function returns *true* if the cache holds the referenced entry, otherwise it returns *false*.

inline bool **get_entry**(*key_type* const &k, *key_type* &realkey, *entry_type* &val)

Get a specific entry identified by the given key.

> **Note**
>
> The function will call the entry's *entry::touch* function if the value corresponding to the provided key is found in the cache.

Parameters

- **k** – [in] The key for the entry which should be retrieved from the cache.
- **realkey[out]** – Return the full real key found in the cache
- **val** – [out] If the entry indexed by the key is found in the cache this value on successful return will be a copy of the corresponding entry.

Returns
This function returns *true* if the cache holds the referenced entry, otherwise it returns *false*.

inline bool **get_entry**(*key_type* const &k, *entry_type* &val)

Get a specific entry identified by the given key.

> **Note**
>
> The function will call the entry's *entry::touch* function if the value corresponding to the provided key is found in the cache.

Parameters
- **k** – [in] The key for the entry which should be retrieved from the cache.
- **val** – [out] If the entry indexed by the key is found in the cache this value on successful return will be a copy of the corresponding entry.

Returns
This function returns *true* if the cache holds the referenced entry, otherwise it returns *false*.

inline bool **get_entry**(*key_type* const &k, *value_type* &val)

Get a specific entry identified by the given key.

> **Note**
>
> The function will call the entry's *entry::touch* function if the value corresponding to the provided is found in the cache.

Parameters
- **k** – [in] The key for the entry which should be retrieved from the cache
- **val** – [out] If the entry indexed by the key is found in the cache this value on successful return will be a copy of the corresponding value.

Returns
This function returns *true* if the cache holds the referenced entry, otherwise it returns *false*.

inline bool **insert**(*key_type* const &k, *value_type* const &val)

Insert a new element into this cache.

> **Note**
>
> This function invokes both, the insert policy as provided to the constructor and the function *entry::insert* of the newly constructed entry instance. If either of these functions returns false the key/value pair doesn't get inserted into the cache and the *insert* function will return *false*. Other reasons for this function to fail (return *false*) are a) the key/value

pair is already held in the cache or b) inserting the new value into the cache maxed out its capacity and it was not possible to free any of the existing entries.

Parameters

- **k** – [in] The key for the entry which should be added to the cache.
- **val** – [in] The value which should be added to the cache.

Returns

This function returns *true* if the entry has been successfully added to the cache, otherwise it returns *false*.

inline bool **insert**(*key_type* const &k, *value_type* &&val)

template<typename **Entry_**, *std*::enable_if_t<*std*::is_convertible_v<*std*::decay_t<*Entry_*>, *entry_type*>, int> = 0>

inline bool **insert**(*key_type* const &k, *Entry_* &&e)

Insert a new entry into this cache.

> **Note**
>
> This function invokes both, the insert policy as provided to the constructor and the function *entry::insert* of the provided entry instance. If either of these functions returns false the key/value pair doesn't get inserted into the cache and the *insert* function will return *false*. Other reasons for this function to fail (return *false*) are a) the key/value pair is already held in the cache or b) inserting the new value into the cache maxed out its capacity and it was not possible to free any of the existing entries.

Parameters

- **k** – [in] The key for the entry which should be added to the cache.
- **e** – [in] The entry which should be added to the cache.

Returns

This function returns *true* if the entry has been successfully added to the cache, otherwise it returns *false*.

template<typename **Value**, *std*::enable_if_t<*std*::is_convertible_v<*std*::decay_t<*Value*>, *value_type*>, int> = 0>

inline bool **update**(*key_type* const &k, *Value* &&val)

Update an existing element in this cache.

> **Note**
>
> The function will call the entry's *entry::touch* function if the indexed value is found in the cache.

> **Note**
>
> The difference to the other overload of the *insert* function is that this overload replaces

the cached value only, while the other overload replaces the whole cache entry, updating the cache entry properties.

Parameters

- **k** – [in] The key for the value which should be updated in the cache.
- **val** – [in] The value which should be used as a replacement for the existing value in the cache. Any existing cache entry is not changed except for its value.

Returns

This function returns *true* if the entry has been successfully updated, otherwise it returns *false*. If the entry currently is not held by the cache it is added and the return value reflects the outcome of the corresponding insert operation.

template<typename **F**, typename **Value**, typename = *std*::enable_if_t<*std*::is_convertible_v<*std*::decay_t<*Value*>, *value_type*>>>

inline bool **update_if**(*key_type* const &k, *Value* &&val, *F* &&f)

Update an existing element in this cache.

> **Note**
>
> The function will call the entry's *entry::touch* function if the indexed value is found in the cache.

> **Note**
>
> The difference to the other overload of the *insert* function is that this overload replaces the cached value only, while the other overload replaces the whole cache entry, updating the cache entry properties.

Parameters

- **k** – [in] The key for the value which should be updated in the cache.
- **val** – [in] The value which should be used as a replacement for the existing value in the cache. Any existing cache entry is not changed except for its value.
- **f** – [in] A callable taking two arguments, *k* and the key found in the cache (in that order). If *f* returns true, then the update will continue. If *f* returns false, then the update will not succeed.

Returns

This function returns *true* if the entry has been successfully updated, otherwise it returns *false*. If the entry currently is not held by the cache it is added and the return value reflects the outcome of the corresponding insert operation.

template<typename **Entry_**, *std*::enable_if_t<*std*::is_convertible_v<*std*::decay_t<*Entry_*>, *entry_type*>, int> = 0>

inline bool **update**(*key_type* const &k, *Entry_* &&e)

Update an existing entry in this cache.

> **Note**
>
> The function will call the entry's *entry::touch* function if the indexed value is found in

the cache.

> **Note**
>
> The difference to the other overload of the *insert* function is that this overload replaces the whole cache entry, while the other overload replaces the cached value only, leaving the cache entry properties untouched.

Parameters

- **k** – [in] The key for the entry which should be updated in the cache.
- **e** – [in] The entry which should be used as a replacement for the existing entry in the cache. Any existing entry is first removed and then this entry is added.

Returns

This function returns *true* if the entry has been successfully updated, otherwise it returns *false*. If the entry currently is not held by the cache it is added and the return value reflects the outcome of the corresponding insert operation.

template<typename **Func** = policies::always<*storage_value_type*>>

inline *size_type* **erase**(*Func* &&ep = *Func*())

Remove stored entries from the cache for which the supplied function object returns true.

Parameters

ep – [in] This parameter has to be a (unary) function object. It is invoked for each of the entries currently held in the cache. An entry is considered for removal from the cache whenever the value returned from this invocation is *true*. Even then the entry might not be removed from the cache as its *entry::remove* function might return false.

Returns

This function returns the overall size of the removed entries (which is the sum of the values returned by the *entry::get_size* functions of the removed entries).

inline *size_type* **erase**()

Remove all stored entries from the cache.

> **Note**
>
> All entries are considered for removal, but in the end an entry might not be removed from the cache as its *entry::remove* function might return false. This function is very useful for instance in conjunction with an entry's *entry::remove* function enforcing additional criteria like entry expiration, etc.

Returns

This function returns the overall size of the removed entries (which is the sum of the values returned by the *entry::get_size* functions of the removed entries).

inline void **clear**()

Clear the cache.

Unconditionally removes all stored entries from the cache.

inline constexpr *statistics_type* const &**get_statistics**() const noexcept

Allow to access the embedded statistics instance.

Returns

This function returns a reference to the statistics instance embedded inside this cache

inline *statistics_type* &**get_statistics**() noexcept

Protected Functions

inline bool **free_space**(long num_free)

Private Types

using **iterator** = typename *storage_type*::iterator

using **const_iterator** = typename *storage_type*::const_iterator

using **heap_type** = *std*::deque<*iterator*>

using **heap_iterator** = typename *heap_type*::iterator

using **adapted_update_policy_type** = *adapt*<*UpdatePolicy*, *iterator*>

using **update_on_exit** = typename *statistics_type*::update_on_exit

Private Members

size_type **max_size_**

size_type **current_size_**

storage_type **store_**

heap_type **entry_heap_**

adapted_update_policy_type **update_policy_**

insert_policy_type **insert_policy_**

statistics_type **statistics_**

template<typename **Func**, typename **Iterator**>

struct **adapt**

Public Functions

inline explicit **adapt**(*Func* const &f)

inline explicit **adapt**(*Func* &&f) noexcept

inline bool **operator()**(*Iterator* const &lhs, *Iterator* const &rhs) const

Public Members

Func **f_**

hpx/cache/lru_cache.hpp

Defined in header hpx/cache/lru_cache.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

namespace **cache**

template<typename **Key**, typename **Entry**, typename **Statistics** = *statistics*::*no_statistics*>

class **lru_cache**

#include <hpx/cache/lru_cache.hpp> The lru_cache implements the basic functionality needed for a local (non-distributed) LRU cache.

Template Parameters

- **Key** – The type of the keys to use to identify the entries stored in the cache
- **Entry** – The type of the items to be held in the cache.
- **Statistics** – A (optional) type allowing to collect some basic statistics about the operation of the cache instance. The type must conform to the CacheStatistics concept. The default value is the type statistics::no_statistics which does not collect any numbers, but provides empty stubs allowing the code to compile.

Public Types

using **key_type** = *Key*

using **entry_type** = *Entry*

using **statistics_type** = *Statistics*

using **entry_pair** = *std*::pair<*key_type*, *entry_type*>

using **storage_type** = *std*::list<*entry_pair*>

using **map_type** = *std*::map<*Key*, typename *storage_type*::iterator>

using **size_type** = *std*::size_t

Public Functions

inline explicit `lru_cache`(*size_type* max_size = 0)

Construct an instance of a *lru_cache*.

Parameters

`max_size` – [in] The maximal size this cache is allowed to reach any time. The default is zero (no size limitation). The unit of this value is usually determined by the unit of the values returned by the entry's *get_size* function.

`lru_cache`(*lru_cache* const &other) = default

`lru_cache`(*lru_cache* &&other) = default

lru_cache &`operator=`(*lru_cache* const &other) = default

lru_cache &`operator=`(*lru_cache* &&other) = default

`~lru_cache`() = default

inline constexpr *size_type* `size`() const noexcept

Return current size of the cache.

Returns

The current size of this cache instance.

inline constexpr *size_type* `capacity`() const noexcept

Access the maximum size the cache is allowed to grow to.

> **Note**
>
> The unit of this value is usually determined by the unit of the return values of the entry's function *entry::get_size*.

Returns

The maximum size this cache instance is currently allowed to reach. If this number is zero the cache has no limitation with regard to a maximum size.

inline void **`reserve`**(*size_type* max_size)

Change the maximum size this cache can grow to.

Parameters

`max_size` – [in] The new maximum size this cache will be allowed to grow to.

inline bool **`holds_key`**(*key_type* const &key) const

Check whether the cache currently holds an entry identified by the given key.

> **Note**
>
> This function does not call the entry's function *entry::touch*. It just checks if the cache contains an entry corresponding to the given key.

Parameters

`key` – [in] The key for the entry which should be looked up in the cache.

Returns

This function returns *true* if the cache holds the referenced entry, otherwise it returns *false*.

inline bool **`get_entry`**(*key_type* const &key, *key_type* &realkey, *entry_type* &entry)

Get a specific entry identified by the given key.

> **Note**
>
> The function will "touch" the entry and mark it as recently used if the key was found in the cache.

Parameters

- **`key`** – [in] The key for the entry which should be retrieved from the cache.
- **`realkey[out]`** – Return the full real key found in the cache
- **`entry`** – [out] If the entry indexed by the key is found in the cache this value on successful return will be a copy of the corresponding entry.

Returns

This function returns *true* if the cache holds the referenced entry, otherwise it returns *false*.

inline bool **`get_entry`**(*key_type* const &key, *entry_type* const &entry)

Get a specific entry identified by the given key.

> **Note**
>
> The function will "touch" the entry and mark it as recently used if the key was found in the cache.

Parameters

- **`key`** – [in] The key for the entry which should be retrieved from the cache.

- **entry** – [out] If the entry indexed by the key is found in the cache this value on successful return will be a copy of the corresponding entry.

Returns

This function returns *true* if the cache holds the referenced entry, otherwise it returns *false*.

template<typename **Entry_**, typename = *std*::enable_if_t<*std*::is_convertible_v<*std*::decay_t<*Entry_*>, *entry_type*>>>

inline bool **insert**(*key_type* const &key, *Entry_* &&entry)

Insert a new entry into this cache.

> **Note**
>
> This function assumes that the entry is not in the cache already. Inserting an already existing entry is considered undefined behavior

Parameters

- **key** – [in] The key for the entry which should be added to the cache.
- **entry** – [in] The entry which should be added to the cache.

template<typename **Entry_**, typename = *std*::enable_if_t<*std*::is_convertible_v<*std*::decay_t<*Entry_*>, *entry_type*>>>

inline void **update**(*key_type* const &key, *Entry_* &&entry)

Update an existing element in this cache.

> **Note**
>
> The function will "touch" the entry and mark it as recently used if the key was found in the cache.

> **Note**
>
> The difference to the other overload of the *insert* function is that this overload replaces the cached value only, while the other overload replaces the whole cache entry, updating the cache entry properties.

Parameters

- **key** – [in] The key for the value which should be updated in the cache.
- **entry** – [in] The entry which should be used as a replacement for the existing value in the cache. Any existing cache entry is not changed except for its value.

template<typename **F**, typename **Entry_**, *std*::enable_if_t<*std*::is_convertible_v<*std*::decay_t<*Entry_*>, *entry_type*>, int> = 0>

inline bool **update_if**(*key_type* const &key, *Entry_* &&entry, *F* &&f)

Update an existing element in this cache.

> **Note**
>
> The function will "touch" the entry and mark it as recently used if the key was found in the cache.

> **Note**
>
> The difference to the other overload of the *insert* function is that this overload replaces the cached value only, while the other overload replaces the whole cache entry, updating the cache entry properties.

Parameters

- **key** – [in] The key for the value which should be updated in the cache.
- **entry** – [in] The value which should be used as a replacement for the existing value in the cache. Any existing cache entry is not changed except for its value.
- **f** – [in] A callable taking two arguments, *k* and the key found in the cache (in that order). If *f* returns true, then the update will continue. If *f* returns false, then the update will not succeed.

Returns

This function returns *true* if the entry has been successfully updated, otherwise it returns *false*. If the entry currently is not held by the cache it is added and the return value reflects the outcome of the corresponding insert operation.

template<typename **Func**>

inline *size_type* **erase**(*Func* const &ep)

Remove stored entries from the cache for which the supplied function object returns true.

Parameters

ep – [in] This parameter has to be a (unary) function object. It is invoked for each of the entries currently held in the cache. An entry is considered for removal from the cache whenever the value returned from this invocation is *true*.

Returns

This function returns the overall size of the removed entries (which is the sum of the values returned by the *entry::get_size* functions of the removed entries).

inline *size_type* **erase**()

Remove all stored entries from the cache.

Returns

This function returns the overall size of the removed entries (which is the sum of the values returned by the *entry::get_size* functions of the removed entries).

inline *size_type* **clear**()

Clear the cache.

Unconditionally removes all stored entries from the cache.

inline constexpr *statistics_type* const &**get_statistics**() const noexcept

Allow to access the embedded statistics instance.

Returns

This function returns a reference to the statistics instance embedded inside this cache

inline *statistics_type* &**get_statistics**() noexcept

Private Types

using **update_on_exit** = typename *statistics_type*::update_on_exit

Private Functions

template<typename **Entry_**, typename = *std*::enable_if_t<*std*::is_convertible_v<*std*::decay_t<*Entry_*>, *entry_type*>>>

inline void **insert_nonexist**(*key_type* const &key, *Entry_* &&entry)

inline void **touch**(typename *storage_type*::iterator it)

inline void **evict**()

Private Members

size_type **max_size_**

size_type **current_size_** = 0

storage_type **storage_**

map_type **map_**

statistics_type **statistics_**

hpx/cache/entries/size_entry.hpp

Defined in header hpx/cache/entries/size_entry.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

namespace **cache**

namespace **entries**

template<typename **Value**>

class **size_entry** : public *hpx*::*util*::*cache*::*entries*::*entry*<*Value*>

#include <hpx/cache/entries/size_entry.hpp> The size_entry type can be used to store values in a cache which have a size associated (such as files, etc.). Using this type as the cache's entry type makes sure that the entries with the biggest size are discarded from the cache first.

> **Note**
>
> The size_entry conforms to the CacheEntry concept.

> **Note**
>
> This type can be used to model a 'discard smallest first' cache policy if it is used with a std::greater as the caches' UpdatePolicy (instead of the default std::less).

Template Parameters

Value – The data type to be stored in a cache. It has to be default constructible, copy constructible and less_than_comparable.

Public Functions

size_entry() = default

Any cache entry has to be default constructible.

inline explicit **size_entry**(*Value* const &val, *std*::size_t size = 0) noexcept(*std*::is_nothrow_constructible_v<*base_type*, *Value* const&>)

Construct a new instance of a cache entry holding the given value.

inline explicit **size_entry**(*Value* &&val, *std*::size_t size = 0) noexcept

Construct a new instance of a cache entry holding the given value.

inline constexpr *std*::size_t **get_size**() const noexcept

Return the 'size' of this entry.

Private Types

using **base_type** = *entry*<*Value*>

Private Members

std::size_t **size_** = 0

Friends

inline friend auto **operator<**(*size_entry* const &lhs, *size_entry* const &rhs) noexcept

Compare the 'age' of two entries. An entry is 'older' than another entry if it has a bigger size.

inline friend auto **operator>**(*size_entry* const &lhs, *size_entry* const &rhs) noexcept

inline friend auto **operator<=**(*size_entry* const &lhs, *size_entry* const &rhs) noexcept

inline friend auto **operator>=**(*size_entry* const &lhs, *size_entry* const &rhs) noexcept inline friend auto **operator==**(*size_entry* const &lhs, *size_entry* const &rhs) noexcept

inline friend auto **operator!=**(*size_entry* const &lhs, *size_entry* const &rhs) noexcept

hpx/cache/entries/lru_entry.hpp

Defined in header hpx/cache/entries/lru_entry.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

namespace **cache**

namespace **entries**

template<typename **Value**>

class **lru_entry** : public *hpx*::*util*::*cache*::*entries*::*entry*<*Value*>

#include <hpx/cache/entries/lru_entry.hpp> The lru_entry type can be used to store arbitrary values in a cache. Using this type as the cache's entry type makes sure that the least recently used entries are discarded from the cache first.

> **Note**
>
> The lru_entry conforms to the CacheEntry concept.

> **Note**
>
> This type can be used to model a 'most recently used' cache policy if it is used with a std::greater as the caches' UpdatePolicy (instead of the default std::less).

Template Parameters

Value – The data type to be stored in a cache. It has to be default constructible, copy constructible and less_than_comparable.

Public Functions

inline **lru_entry**()

Any cache entry has to be default constructible.

inline explicit **lru_entry**(*Value* const &val) noexcept(*std*::is_nothrow_constructible_v<*base_type*, *Value* const&>)

Construct a new instance of a cache entry holding the given value.

inline explicit **lru_entry**(*Value* &&val) noexcept

Construct a new instance of a cache entry holding the given value.

inline bool **touch**()

The function *touch* is called by a cache holding this instance whenever it has been requested (touched).

In the case of the LRU entry we store the time of the last access which will be used to compare the age of an entry during the invocation of the *operator<()*.

Returns

This function should return true if the cache needs to update its internal heap. Usually this is needed if the entry has been changed by *touch()* in a way influencing the sort order as mandated by the cache's UpdatePolicy

inline constexpr *time_point* const &**get_access_time**() const noexcept

Returns the last access time of the entry.

Private Types

using **base_type** = *entry*<*Value*>

using **time_point** = *std*::chrono::steady_clock::time_point

Private Members

time_point **access_time_**

Friends

inline friend auto **operator<**(*lru_entry* const &lhs, *lru_entry* const &rhs)

Compare the 'age' of two entries. An entry is 'older' than another entry if it has been accessed less recently (LRU).

inline friend auto **operator>**(*lru_entry* const &lhs, *lru_entry* const &rhs)

inline friend auto **operator<=**(*lru_entry* const &lhs, *lru_entry* const &rhs)

inline friend auto **operator>=**(*lru_entry* const &lhs, *lru_entry* const &rhs)

inline friend auto **operator==**(*lru_entry* const &lhs, *lru_entry* const &rhs)

inline friend auto **operator!=**(*lru_entry* const &lhs, *lru_entry* const &rhs)

hpx/cache/entries/fifo_entry.hpp

Defined in header hpx/cache/entries/fifo_entry.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

namespace **cache**

namespace **entries**

template<typename **Value**>

class **fifo_entry** : public *hpx*::*util*::*cache*::*entries*::*entry*<*Value*>

#include <hpx/cache/entries/fifo_entry.hpp> The fifo_entry type can be used to store arbitrary values in a cache. Using this type as the cache's entry type makes sure that the least recently inserted entries are discarded from the cache first.

Note

The fifo_entry conforms to the CacheEntry concept.

Note

This type can be used to model a 'last in first out' cache policy if it is used with a std::greater as the caches' UpdatePolicy (instead of the default std::less).

Template Parameters

Value – The data type to be stored in a cache. It has to be default constructible, copy constructible and less_than_comparable.

Public Functions

fifo_entry() = default

Any cache entry has to be default constructible.

inline explicit **fifo_entry**(*Value* const &val) noexcept(*std*::is_nothrow_constructible_v<*base_type*, *Value* const&>)

Construct a new instance of a cache entry holding the given value.

inline explicit **fifo_entry**(*Value* &&val) noexcept

Construct a new instance of a cache entry holding the given value.

inline constexpr bool **insert**()

The function *insert* is called by a cache whenever it is about to be inserted into the cache.

Note

This function is part of the CacheEntry concept

Returns

This function should return *true* if the entry should be added to the cache, otherwise it should return *false*.

inline constexpr *time_point* const &**get_creation_time**() const noexcept

Private Types

using **base_type** = *entry*<*Value*>

using **time_point** = *std*::chrono::steady_clock::time_point

Private Members

time_point **insertion_time_**

Friends

inline friend auto **operator<**(*fifo_entry* const &lhs, *fifo_entry* const &rhs)

Compare the 'age' of two entries. An entry is 'older' than another entry if it has been created earlier (FIFO).

inline friend auto **operator>**(*fifo_entry* const &lhs, *fifo_entry* const &rhs)

inline friend auto **operator<=**(*fifo_entry* const &lhs, *fifo_entry* const &rhs)

inline friend auto **operator>=**(*fifo_entry* const &lhs, *fifo_entry* const &rhs)

inline friend auto **operator==**(*fifo_entry* const &lhs, *fifo_entry* const &rhs)

inline friend auto **operator!=**(*fifo_entry* const &lhs, *fifo_entry* const &rhs)

hpx/cache/entries/lfu_entry.hpp

Defined in header hpx/cache/entries/lfu_entry.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

namespace **cache**

namespace **entries**

template<typename **Value**>

class **lfu_entry** : public *hpx*::*util*::*cache*::*entries*::*entry*<*Value*>

#include <hpx/cache/entries/lfu_entry.hpp> The lfu_entry type can be used to store arbitrary values in a cache. Using this type as the cache's entry type makes sure that the least frequently used entries are discarded from the cache first.

> **Note**
>
> The lfu_entry conforms to the CacheEntry concept.

> **Note**
>
> This type can be used to model a 'most frequently used' cache policy if it is used with a std::greater as the caches' UpdatePolicy (instead of the default std::less).

Template Parameters

Value – The data type to be stored in a cache. It has to be default constructible, copy constructible and less_than_comparable.

Public Functions

lfu_entry() = default

Any cache entry has to be default constructible.

inline explicit **lfu_entry**(*Value* const &val) noexcept(*std*::is_nothrow_constructible_v<*base_type*, *Value* const&>)

Construct a new instance of a cache entry holding the given value.

inline explicit **lfu_entry**(*Value* &&val) noexcept

Construct a new instance of a cache entry holding the given value.

inline bool **touch**() noexcept

The function *touch* is called by a cache holding this instance whenever it has been requested (touched).

In the case of the LFU entry we store the reference count tracking the number of times this entry has been requested. This which will be used to compare the age of an entry during the invocation of the *operator<()*.

Returns

This function should return true if the cache needs to update its internal heap. Usually this is needed if the entry has been changed by *touch()* in a way influencing the sort order as mandated by the cache's UpdatePolicy

inline constexpr unsigned long const &**`get_access_count`**`()` const noexcept

Private Types

using **`base_type`** = *entry*<*Value*>

Private Members

unsigned long **`ref_count_`** = 0

Friends

inline friend auto **`operator<`**`(`*lfu_entry* const &lhs, *lfu_entry* const &rhs`)`

Compare the 'age' of two entries. An entry is 'older' than another entry if it has been accessed less frequently (LFU).

inline friend auto **`operator>`**`(`*lfu_entry* const &lhs, *lfu_entry* const &rhs`)`

inline friend auto **`operator<=`**`(`*lfu_entry* const &lhs, *lfu_entry* const &rhs`)`

inline friend auto **`operator>=`**`(`*lfu_entry* const &lhs, *lfu_entry* const &rhs`)`

inline friend auto **`operator==`**`(`*lfu_entry* const &lhs, *lfu_entry* const &rhs`)`

inline friend auto **`operator!=`**`(`*lfu_entry* const &lhs, *lfu_entry* const &rhs`)`

hpx/cache/entries/entry.hpp

Defined in header hpx/cache/entries/entry.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

namespace **cache**

namespace **entries**

template<typename **Value**>

class **entry**

#include <hpx/cache/entries/entry.hpp>

Template Parameters

Value – The data type to be stored in a cache. It has to be default constructible, copy constructible and less_than_comparable.

Subclassed by *hpx::util::cache::entries::fifo_entry< Value >*, *hpx::util::cache::entries::lfu_entry< Value >*, *hpx::util::cache::entries::lru_entry< Value >*, *hpx::util::cache::entries::size_entry< Value >*

Public Types

using **value_type** = *Value*

Public Functions

entry() = default

Any cache entry has to be default constructible.

inline explicit **entry**(*value_type* const &val) noexcept(*std*::is_nothrow_copy_constructible_v<*value_type*>)

Construct a new instance of a cache entry holding the given value.

inline explicit **entry**(*value_type* &&val) noexcept

Construct a new instance of a cache entry holding the given value.

inline *value_type* &**get**() noexcept

Get a reference to the stored data value.

> **Note**
>
> This function is part of the CacheEntry concept

inline constexpr *value_type* const &**get**() const noexcept

Public Static Functions

static inline constexpr bool **touch**() noexcept

The function *touch* is called by a cache holding this instance whenever it has been requested (touched).

> **Note**
>
> It is possible to change the entry in a way influencing the sort criteria mandated by the UpdatePolicy. In this case the function should return *true* to indicate this to the cache, forcing to reorder the cache entries.

> **Note**
>
> This function is part of the CacheEntry concept

Returns

This function should return true if the cache needs to update its internal heap. Usually this is needed if the entry has been changed by *touch()* in a way influencing the sort order as mandated by the cache's UpdatePolicy

static inline constexpr bool **insert**() noexcept

The function *insert* is called by a cache whenever it is about to be inserted into the cache.

> **Note**
>
> This function is part of the CacheEntry concept

Returns

This function should return *true* if the entry should be added to the cache, otherwise it should return *false*.

static inline constexpr bool **remove**() noexcept

The function *remove* is called by a cache holding this instance whenever it is about to be removed from the cache.

> **Note**
>
> This function is part of the CacheEntry concept

Returns

The return value can be used to avoid removing this instance from the cache. If the value is *true* it is ok to remove the entry, otherwise it will stay in the cache.

static inline constexpr *std*::size_t **get_size**() noexcept

Return the 'size' of this entry. By default, the size of each entry is just one (1), which is sensible if the cache has a limit (capacity) measured in number of entries.

Private Members

value_type **value_**

Friends

friend auto **operator<=>**(*entry* const&, *entry* const&) = default

Forwarding operator<=> allowing to compare entries instead of the values.

hpx/cache/statistics/no_statistics.hpp

Defined in header hpx/cache/statistics/no_statistics.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

namespace **cache**

namespace **statistics**

Enums

enum class **method**

Values:

enumerator **get_entry**

enumerator **insert_entry**

enumerator **update_entry**

enumerator **erase_entry**

class **no_statistics**

Subclassed by *hpx::util::cache::statistics::local_statistics*

Public Static Functions

static inline constexpr void **got_hit**() noexcept

The function *got_hit* will be called by a cache instance whenever an entry got touched.

static inline constexpr void **got_miss**() noexcept

The function *got_miss* will be called by a cache instance whenever a requested entry has not been found in the cache.

static inline constexpr void **got_insertion**() noexcept

The function *got_insertion* will be called by a cache instance whenever a new entry has been inserted.

static inline constexpr void **got_eviction**() noexcept

The function *got_eviction* will be called by a cache instance whenever an entry has been removed from the cache because a new inserted entry let the cache grow beyond its capacity.

static inline constexpr void **clear**() noexcept

Reset all statistics.

static inline constexpr *std*::int64_t **get_get_entry_count**(bool) noexcept

The function *get_get_entry_count* returns the number of invocations of the get_entry() API function of the cache.

static inline constexpr *std*::int64_t **get_insert_entry_count**(bool) noexcept

The function *get_insert_entry_count* returns the number of invocations of the insert_entry() API function of the cache.

static inline constexpr *std*::int64_t **get_update_entry_count**(bool) noexcept

The function *get_update_entry_count* returns the number of invocations of the update_entry() API function of the cache.

static inline constexpr *std*::int64_t **get_erase_entry_count**(bool) noexcept

The function *get_erase_entry_count* returns the number of invocations of the erase() API function of the cache.

static inline constexpr *std*::int64_t **get_get_entry_time**(bool) noexcept

The function *get_get_entry_time* returns the overall time spent executing of the get_entry() API function of the cache.

static inline constexpr *std*::int64_t **get_insert_entry_time**(bool) noexcept

The function *get_insert_entry_time* returns the overall time spent executing of the insert_entry() API function of the cache.

static inline constexpr *std*::int64_t **get_update_entry_time**(bool) noexcept

The function *get_update_entry_time* returns the overall time spent executing of the update_entry() API function of the cache.

static inline constexpr *std*::int64_t **get_erase_entry_time**(bool) noexcept

The function *get_erase_entry_time* returns the overall time spent executing of the erase() API function of the cache.

struct **update_on_exit**

#include <no_statistics.hpp> Helper class to update timings and counts on function exit.

Public Functions

inline constexpr **update_on_exit**(*no_statistics* const&, *method*) noexcept

~update_on_exit() = default

update_on_exit(*update_on_exit* const&) = delete

update_on_exit(*update_on_exit*&&) = delete

update_on_exit &**operator=**(*update_on_exit* const&) = delete

update_on_exit &**operator=**(*update_on_exit*&&) = delete

hpx/cache/statistics/local_statistics.hpp

Defined in header hpx/cache/statistics/local_statistics.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

namespace **cache**

namespace **statistics**

class **local_statistics** : public *hpx*::*util*::*cache*::*statistics*::*no_statistics*

Public Functions

local_statistics() = default

inline constexpr *std*::size_t **hits**() const noexcept

inline constexpr *std*::size_t **misses**() const noexcept

inline constexpr *std*::size_t **insertions**() const noexcept inline constexpr *std*::size_t **evictions**() const noexcept

inline *std*::size_t **hits**(bool const reset) noexcept

inline *std*::size_t **misses**(bool const reset) noexcept

inline *std*::size_t **insertions**(bool const reset) noexcept

inline *std*::size_t **evictions**(bool const reset) noexcept

inline void **got_hit**() noexcept

The function *got_hit* will be called by a cache instance whenever an entry got touched.

inline void **got_miss**() noexcept

The function *got_miss* will be called by a cache instance whenever a requested entry has not been found in the cache.

inline void **got_insertion**() noexcept

The function *got_insertion* will be called by a cache instance whenever a new entry has been inserted.

inline void **got_eviction**() noexcept

The function *got_eviction* will be called by a cache instance whenever an entry has been removed from the cache because a new inserted entry let the cache grow beyond its capacity.

inline void **clear**() noexcept

Reset all statistics.

Private Members

std::size_t **hits_** = 0

std::size_t **misses_** = 0

std::size_t **insertions_** = 0

std::size_t **evictions_** = 0

Private Static Functions

static inline *std*::size_t **get_and_reset**(*std*::size_t &value, bool const reset) noexcept

checkpoint_base

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/checkpoint_base/checkpoint_data.hpp

Defined in header hpx/checkpoint_base/checkpoint_data.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

Functions

template<typename **Container**, typename ...**Ts**>

void **save_checkpoint_data**(*Container* &data, *Ts*&&... ts)

save_checkpoint_data

Save_checkpoint_data takes any number of objects which a user may wish to store in the given container.

Template Parameters

- **Container** – Container used to store the check-pointed data.
- **Ts** – Types of variables to checkpoint

Parameters

- **data** – Container instance used to store the checkpoint data
- **ts** – Variable instances to be inserted into the checkpoint.

template<typename ...**Ts**>

std::size_t **prepare_checkpoint_data**(*Ts* const&... ts)

prepare_checkpoint_data

prepare_checkpoint_data takes any number of objects which a user may wish to store in a subsequent save_checkpoint_data operation. The function will return the number of bytes necessary to store the data that will be produced.

Template Parameters

`Ts` – Types of variables to checkpoint

Parameters

`ts` – Variable instances to be inserted into the checkpoint.

template<typename **Container**, typename ...**Ts**>

void **restore_checkpoint_data**(*Container* const &cont, *Ts*&... ts)

restore_checkpoint_data

restore_checkpoint_data takes any number of objects which a user may wish to restore from the given container. The sequence of objects has to correspond to the sequence of objects for the corresponding call to save_checkpoint_data that had used the given container instance.

Template Parameters

- `Container` – Container used to restore the check-pointed data.
- `Ts` – Types of variables to restore

Parameters

- `cont` – Container instance used to restore the checkpoint data
- `ts` – Variable instances to be restored from the container

struct **checkpointing_tag**

template<>

struct **extra_data_helper**<*checkpointing_tag*>

Public Static Functions

static extra_data_id_type **id**() noexcept

static inline constexpr void **reset**(*checkpointing_tag**) noexcept

compute_local

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/compute_local/vector.hpp

Defined in header hpx/compute_local/vector.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **compute**

Functions

template<typename **T**, typename **Allocator**>

void **swap**(*vector*<*T*, *Allocator*> &x, *vector*<*T*, *Allocator*> &y) noexcept

Effects: x.swap(y);.

template<typename **T**, typename **Allocator** = *std*::allocator<*T*>>

class **vector**

Public Types

using **value_type** = *T*

Member types (FIXME: add reference to std.

using **allocator_type** = *Allocator*

using **access_target** = typename *alloc_traits*::access_target

using **size_type** = *std*::size_t

using **difference_type** = *std*::ptrdiff_t

using **reference** = typename *alloc_traits*::reference

using **const_reference** = typename *alloc_traits*::const_reference

using **pointer** = typename *alloc_traits*::pointer

using **const_pointer** = typename *alloc_traits*::const_pointer

using **iterator** = detail::iterator<*T*, *Allocator*>

using **const_iterator** = detail::iterator<*T* const, *Allocator*>

using **reverse_iterator** = *std*::reverse_iterator<*iterator*>

using **const_reverse_iterator** = *std*::reverse_iterator<*const_iterator*>

Public Functions

inline explicit **vector**(*Allocator* const &alloc = *Allocator*())

inline **vector**(*size_type* count, *T* const &value, *Allocator* const &alloc = *Allocator*())

inline explicit **vector**(*size_type* count, *Allocator* const &alloc = *Allocator*())

template<typename **InIter**>

inline **vector**(*InIter* first, *InIter* last, *Allocator* const &alloc = *Allocator*())

inline **vector**(*vector* const &other)

inline **vector**(*vector* const &other, *Allocator* const &alloc)

inline **vector**(*vector* &&other) noexcept

inline **vector**(*vector* &&other, *Allocator* const &alloc)

inline **vector**(*std*::initializer_list<*T*> init, *Allocator* const &alloc)

inline **~vector**()

inline *vector* &**operator=**(*vector* const &other)

inline *vector* &**operator=**(*vector* &&other) noexcept

inline void **assign**(*size_type* count, *T* const &value)

template<typename **InIter**>

inline void **assign**(*InIter* first, *InIter* last)

inline void **assign**(*std*::initializer_list<*T*> ilist) inline *allocator_type* **get_allocator**() const noexcept

Returns the allocator associated with the container.

inline *reference* **at**(*size_type* pos)

inline *const_reference* **at**(*size_type* pos) const

inline *reference* **operator[]**(*size_type* pos)

inline *const_reference* **operator[]**(*size_type* pos) const

inline *reference* **front**()

Returns a reference to the first element in the container. Calling front on an empty container is undefined.

inline *const_reference* **front**() const

Returns a reference to the first element in the container. Calling front on an empty container is undefined.

inline *reference* **back**()

Returns a reference to the last element in the container. Calling back on an empty container is undefined.

inline *const_reference* **back**() const

Returns a reference to the last element in the container. Calling back on an empty container is undefined.

inline *pointer* **data**() noexcept

Returns pointer to the underlying array serving as element storage. The pointer is such that range [*data()*; *data()* + size()) is always a valid range, even if the container is empty (*data()* is not dereference-able in that case).

inline *const_pointer* **data**() const noexcept

Returns pointer to the underlying array serving as element storage. The pointer is such that range [*data()*; *data()* + size()) is always a valid range, even if the container is empty (*data()* is not dereference-able in that case).

inline *T* ***device_data**() const noexcept

Returns a raw pointer corresponding to the address of the data allocated on the device.

inline *std*::size_t **size**() const noexcept

inline *std*::size_t **capacity**() const noexcept

inline bool **empty**() const noexcept

Returns: size() == 0.

inline void **resize**(*size_type*)

Effects: If size <= size(), equivalent to calling pop_back() size()

- size times. If size() < size, appends size - size() default-inserted elements to the sequence.

Requires: T shall be MoveInsertable and DefaultInsertable into *this.

Remarks: If an exception is thrown other than by the move constructor of a non-CopyInsertable T there are no effects.

inline void **resize**(*size_type*, *T* const&)

Effects: If size <= size(), equivalent to calling pop_back() size() - size times. If size() < size, appends size - size() copies of val to the sequence.

Requires: T shall be CopyInsertable into *this.

Remarks: If an exception is thrown there are no effects.

inline *iterator* **begin**() noexcept

inline *iterator* **end**() noexcept

inline *const_iterator* **cbegin**() const noexcept

inline *const_iterator* **cend**() const noexcept

inline *const_iterator* **begin**() const noexcept

inline *const_iterator* **end**() const noexcept

inline *reverse_iterator* **rbegin**() noexcept inline *reverse_iterator* **rend**() noexcept

inline *const_reverse_iterator* **crbegin**() const noexcept

inline *const_reverse_iterator* **crend**() const noexcept

inline *const_reverse_iterator* **rbegin**() const noexcept

inline *const_reverse_iterator* **rend**() const noexcept

inline void **swap**(*vector* &other) noexcept

Effects: Exchanges the contents and capacity() of *this with that of x.

Complexity: Constant time.

inline void **clear**() noexcept

Effects: Erases all elements in the range [begin(),end()). Destroys all elements in ‘a’. Invalidates all references, pointers, and iterators referring to the elements of a and may invalidate the past-the-end iterator.

Post: a.empty() returns true.

Complexity: Linear.

Private Types

using **alloc_traits** = *traits*::allocator_traits<*Allocator*>

Private Members

size_type **size_**

size_type **capacity_**

allocator_type **alloc_**

pointer **data_**

namespace **traits**

hpx/compute_local/host/block_fork_join_executor.hpp

Defined in header hpx/compute_local/host/block_fork_join_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

class **block_fork_join_executor**

#include <block_fork_join_executor.hpp> An executor with fork-join (blocking) semantics.

The *block_fork_join_executor* creates on construction a set of worker threads that are kept alive for the duration of the executor. Copying the executor has reference semantics, i.e. copies of a fork_join_executor hold a reference to the worker threads of the original instance. Scheduling work through the executor concurrently from different threads is undefined behaviour.

The executor keeps a set of worker threads alive for the lifetime of the executor, meaning other work will not be executed while the executor is busy or waiting for work. The executor has a customizable delay after which it will yield to other work. Since starting and resuming the worker threads is a slow operation the executor should be reused whenever possible for multiple adjacent parallel algorithms or invocations of bulk_(a)sync_execute.

This behaviour is similar to the plain *fork_join_executor* except that the *block_fork_join_executor* creates a hierarchy of fork_join_executors, one for each target used to initialize it.

Public Functions

inline explicit **block_fork_join_executor**(*threads*::*thread_priority* const priority = *threads*::*thread_priority*::*bound*, *threads*::*thread_stacksize* const stacksize = *threads*::*thread_stacksize*::*small_*, fork_join_executor::loop_schedule const schedule = fork_join_executor::loop_schedule::static_, *std*::chrono::nanoseconds const yield_delay = *std*::chrono::milliseconds(1))

Construct a *block_fork_join_executor*.

> **Note**
>
> This constructor will create one fork_join_executor for each numa domain

Parameters

- **priority** – The priority of the worker threads.
- **stacksize** – The stacksize of the worker threads. Must not be nostack.
- **schedule** – The loop schedule of the parallel regions.
- **yield_delay** – The time after which the executor yields to other work if it has not received any new work for execution.

inline explicit **block_fork_join_executor**(*std*::vector<*compute*::*host*::target> const &targets, *threads*::*thread_priority* const priority = *threads*::*thread_priority*::*bound*, *threads*::*thread_stacksize* const stacksize = *threads*::*thread_stacksize*::*small_*, fork_join_executor::loop_schedule const schedule = fork_join_executor::loop_schedule::static_, *std*::chrono::nanoseconds const yield_delay = *std*::chrono::milliseconds(1))

Construct a *block_fork_join_executor*.

> **Note**
>
> This constructor will create one fork_join_executor for each given target

Parameters

- **targets** – The list of targets to use for thread placement
- **priority** – The priority of the worker threads.
- **stacksize** – The stacksize of the worker threads. Must not be nostack.
- **schedule** – The loop schedule of the parallel regions.
- **yield_delay** – The time after which the executor yields to other work if it has not received any new work for execution.

Private Members

fork_join_executor **exec_**

std::vector<fork_join_executor> **block_execs_**

Private Static Functions

static inline *hpx*::*threads*::mask_type **cores_for_targets**(*std*::vector<*compute*::*host*::target> const &targets)

hpx/compute_local/host/block_executor.hpp

Defined in header hpx/compute_local/host/block_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **compute**

namespace **host**

template<typename **Executor** = *hpx*::*execution*::*experimental*::*restricted_thread_pool_executor*>

struct **block_executor**

#include <block_executor.hpp> The block executor can be used to build NUMA aware programs. It will distribute work evenly across the passed targets

Template Parameters

Executor – The underlying executor to use

Public Types

using **executor_parameters_type** = *hpx*::*execution*::*experimental*::*default_parameters*

Public Functions

inline explicit **block_executor**(*std*::vector<*host*::target> const &targets, *threads*::*thread_priority* priority = *threads*::*thread_priority*::*high*, *threads*::*thread_stacksize* stacksize = *threads*::*thread_stacksize*::*default_*, *threads*::*thread_schedule_hint* schedulehint = {})

inline explicit **block_executor**(*std*::vector<*host*::target> &&targets)

inline **block_executor**(*block_executor* const &other)

inline **block_executor**(*block_executor* &&other) noexcept

inline *block_executor* &**operator=**(*block_executor* const &other)

inline *block_executor* &**operator=**(*block_executor* &&other) noexcept

template<typename **F**, typename ...**Ts**>

inline decltype(auto) **post**(*F* &&f, *Ts*&&... ts) const

template<typename **F**, typename ...**Ts**>

inline decltype(auto) **async_execute**(*F* &&f, *Ts*&&... ts) const

template<typename **F**, typename ...**Ts**>

inline decltype(auto) **sync_execute**(*F* &&f, *Ts*&&... ts) const

template<typename **F**, typename **Shape**, typename ...**Ts**>

inline decltype(auto) **bulk_async_execute_impl**(*F* &&f, *Shape* const &shape, *Ts*&&... ts) const

template<typename **F**, typename **Shape**, typename ...**Ts**>

inline decltype(auto) **bulk_async_execute**(*F* &&f, *Shape* const &shape, *Ts*&&... ts) const

template<typename **F**, typename **Shape**, typename ...**Ts**>

inline decltype(auto) **bulk_sync_execute_impl**(*F* &&f, *Shape* const &shape, *Ts*&&... ts) const

template<typename **F**, typename **Shape**, typename ...**Ts**>

inline decltype(auto) **bulk_sync_execute**(*F* &&f, *Shape* const &shape, *Ts*&&... ts) const

inline *std*::vector<*host*::target> const &**targets**() const noexcept

Private Functions

inline auto **get_next_executor**() const

inline void **init_executors**()

Private Members

std::vector<*host*::target> **targets_**

mutable *std*::atomic<*std*::size_t> **current_**

std::vector<*Executor*> **executors_**

threads::*thread_priority* **priority_** = *threads*::*thread_priority*::*high*

threads::*thread_stacksize* **stacksize_** = *threads*::*thread_stacksize*::*default_* *threads*::*thread_schedule_hint* **schedulehint_** = {}

namespace **execution**

namespace **experimental**

template<typename **Executor**>

struct **executor_execution_category**<*compute*::*host*::*block_executor*<*Executor*>>

Public Types

using **type** = *hpx*::*execution*::*parallel_execution_tag*

template<typename **Executor**>

struct **is_never_blocking_one_way_executor**<*compute*::*host*::*block_executor*<*Executor*>> : public is_never_blocking_one_way_executor<*Executor*>

template<typename **Executor**>

struct **is_one_way_executor**<*compute*::*host*::*block_executor*<*Executor*>> : public is_one_way_executor<*Executor*>

template<typename **Executor**>

struct **is_two_way_executor**<*compute*::*host*::*block_executor*<*Executor*>> : public is_two_way_executor<*Executor*>

template<typename **Executor**>

struct **is_bulk_one_way_executor**<*compute*::*host*::*block_executor*<*Executor*>> : public is_bulk_one_way_executor<*Executor*>

template<typename **Executor**>

struct **is_bulk_two_way_executor**<*compute*::*host*::*block_executor*<*Executor*>> : public is_bulk_two_way_executor<*Executor*>

config

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/config/endian.hpp

Defined in header hpx/config/endian.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

contracts

See *Public API* for a list of names and headers that are part of the public *HPX* API.

HPX_PRE

Defined in header hpx/contracts.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *HPX_POST*
- *HPX_CONTRACT_ASSERT*

Warning

doxygenfunction: Cannot find function "HPX_PRE" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

HPX_POST

Defined in header hpx/contracts.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *HPX_PRE*
- *HPX_CONTRACT_ASSERT*

Warning

doxygenfunction: Cannot find function "HPX_POST" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

HPX_CONTRACT_ASSERT

Defined in header hpx/contracts.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *HPX_PRE*
- *HPX_POST*

> **Warning**
>
> doxygenfunction: Cannot find function "HPX_CONTRACT_ASSERT" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

coroutines

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/coroutines/thread_id_type.hpp

Defined in header hpx/coroutines/thread_id_type.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **threads**

Enums

enum class **thread_id_addref** : *std*::uint8_t

Values:

enumerator **yes**

enumerator **no**

Variables

constexpr *thread_id* const **invalid_thread_id**

struct **thread_id**

Public Functions

constexpr **thread_id**() noexcept = default

thread_id(*thread_id* const&) noexcept = default

thread_id &**operator=**(*thread_id* const&) noexcept = default

~thread_id() = default

inline constexpr **thread_id**(*thread_id* &&rhs) noexcept

inline constexpr *thread_id* &**operator=**(*thread_id* &&rhs) noexcept

inline explicit constexpr **thread_id**(*thread_id_repr* const thrd) noexcept

inline constexpr *thread_id* &**operator=**(*thread_id_repr* const rhs) noexcept

inline explicit constexpr **operator bool**() const noexcept

inline constexpr *thread_id_repr* **get**() const noexcept

inline constexpr void **reset**() noexcept

Private Types

using **thread_id_repr** = void*

Private Members

thread_id_repr **thrd_** = nullptr

Friends

inline friend constexpr friend bool operator== (std::nullptr_t, thread_id const &rhs) noexcept

inline friend constexpr friend bool operator!= (std::nullptr_t, thread_id const &rhs) noexcept

inline friend constexpr friend bool operator== (thread_id const &lhs, std::nullptr_t) noexcept

inline friend constexpr friend bool operator!= (thread_id const &lhs, std::nullptr_t) noexcept

inline friend constexpr friend bool operator== (thread_id const &lhs, thread_id const &rhs) noexcept

inline friend constexpr friend bool operator!= (thread_id const &lhs, thread_id const &rhs) noexcept

inline friend constexpr friend bool operator< (thread_id const &lhs, thread_id const &rhs) noexcept

inline friend constexpr friend bool operator> (thread_id const &lhs, thread_id const &rhs) noexcept

inline friend constexpr friend bool operator<= (thread_id const &lhs, thread_id const &rhs) noexcept

inline friend constexpr friend bool operator>= (thread_id const &lhs, thread_id const &rhs) noexcept

friend *std*::ostream &**operator<<**(*std*::ostream &os, *thread_id* const &id)

friend void **format_value**(*std*::ostream &os, *std*::string_view spec, *thread_id* const &id)

struct **thread_id_ref**

Public Types

using **thread_repr** = detail::thread_data_reference_counting

Public Functions

thread_id_ref() noexcept = default

thread_id_ref(*thread_id_ref* const&) = default

thread_id_ref &**operator=**(*thread_id_ref* const&) = default

thread_id_ref(*thread_id_ref* &&rhs) noexcept = default

thread_id_ref &**operator=**(*thread_id_ref* &&rhs) noexcept = default

~thread_id_ref() = default

inline explicit **thread_id_ref**(*thread_id_repr* const &thrd) noexcept

inline explicit **thread_id_ref**(*thread_id_repr* &&thrd) noexcept

inline *thread_id_ref* &**operator=**(*thread_id_repr* const &rhs) noexcept

inline *thread_id_ref* &**operator=**(*thread_id_repr* &&rhs) noexcept

inline explicit **thread_id_ref**(*thread_repr* *thrd, *thread_id_addref* const addref = *thread_id_addref*::*yes*) noexcept

inline *thread_id_ref* &**operator=**(*thread_repr* *rhs) noexcept

inline **thread_id_ref**(*thread_id* const &noref)

inline **thread_id_ref**(*thread_id* &&noref, bool const add_ref = true) noexcept

inline *thread_id_ref* &**operator=**(*thread_id* const &noref)

inline *thread_id_ref* &**operator=**(*thread_id* &&noref) noexcept

inline explicit constexpr **operator bool**() const noexcept

inline constexpr *thread_id* **noref**() const noexcept

inline constexpr *thread_id_repr* &**get**() & noexcept

inline *thread_id_repr* &&**get**() && noexcept inline constexpr *thread_id_repr* const &**get**() const & noexcept

inline void **reset**() noexcept

inline void **reset**(*thread_repr* *thrd, bool const add_ref = true) noexcept

inline constexpr *thread_repr* ***detach**() noexcept

Private Types

using **thread_id_repr** = *hpx*::intrusive_ptr<detail::thread_data_reference_counting>

Private Members

thread_id_repr **thrd_**

Friends

inline friend constexpr friend bool operator== (std::nullptr_t, thread_id_ref const &rhs) noexcept

inline friend constexpr friend bool operator!= (std::nullptr_t, thread_id_ref const &rhs) noexcept

inline friend constexpr friend bool operator== (thread_id_ref const &lhs, std::nullptr_t) noexcept

inline friend constexpr friend bool operator!= (thread_id_ref const &lhs, std::nullptr_t) noexcept

`inline friend constexpr friend bool operator== (thread_id_ref const &lhs, thread_id_ref const &rhs) noexcept`

`inline friend constexpr friend bool operator!= (thread_id_ref const &lhs, thread_id_ref const &rhs) noexcept`

`inline friend constexpr friend bool operator< (thread_id_ref const &lhs, thread_id_ref const &rhs) noexcept`

`inline friend constexpr friend bool operator> (thread_id_ref const &lhs, thread_id_ref const &rhs) noexcept`

`inline friend constexpr friend bool operator<= (thread_id_ref const &lhs, thread_id_ref const &rhs) noexcept`

`inline friend constexpr friend bool operator>= (thread_id_ref const &lhs, thread_id_ref const &rhs) noexcept`

friend *std*::ostream &**operator<<**(*std*::ostream &os, *thread_id_ref* const &id)

friend void **format_value**(*std*::ostream &os, *std*::string_view spec, *thread_id_ref* const &id)

struct **keep_alive_thread_id**

Public Functions

inline explicit **keep_alive_thread_id**(*hpx*::*threads*::*thread_id* id, bool const add_ref) noexcept

keep_alive_thread_id(*keep_alive_thread_id* const &other) = delete

keep_alive_thread_id(*keep_alive_thread_id* &&other) = default *keep_alive_thread_id* &**operator=**(*keep_alive_thread_id* const &other) = delete

inline *keep_alive_thread_id* &**operator=**(*keep_alive_thread_id* &&other) noexcept

inline **~keep_alive_thread_id**()

inline void **detach_if_needed**() noexcept

Public Members

bool **add_ref**

hpx::*threads*::*thread_id_ref* **id**

namespace **std**

template<>

struct **hash**<::*hpx*::*threads*::*thread_id*>

Public Functions

inline *std*::size_t **operator()**(::*hpx*::*threads*::*thread_id* const &v) const noexcept

template<>

struct **hash**<::*hpx*::*threads*::*thread_id_ref*>

Public Functions

inline *std*::size_t **operator()**(::*hpx*::*threads*::*thread_id_ref* const &v) const noexcept

hpx/coroutines/thread_enums.hpp

Defined in header hpx/coroutines/thread_enums.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **threads**

Enums

enum class **thread_schedule_state** : *std*::int8_t

The *thread_schedule_state* enumerator encodes the current state of a *thread* instance

Task Lifecycle State Transitions:

```
stateDiagram-v2
    %% Creation Phase
    [*] --> Staged : task_staged
    [*] --> Pending : task_created
    [*] --> Suspended : task_created
    Staged --> Pending : task_created

    %% Execution Loop
    Pending --> Active : task_executing
    Active --> Pending : task_yielded

    %% Sleep / Wake Cycle
    Active --> Suspended : task_suspended
    Suspended --> Pending : task_resumed

    %% Termination Paths
    Active --> Terminated : task_completed
    Active --> Deleted : task_completed (Inline fast-path)
    Pending --> Terminated : task_completed (Aborted)
    Suspended --> Terminated : task_completed (Aborted)

    %% Cleanup (Destruction / Recycling)
    Terminated --> Deleted : task_deleted
    Deleted --> [*]
```

Values:

enumerator **unknown**

enumerator **active**

thread is currently active (running, has resources)

enumerator **pending**

thread is pending (ready to run, but no hardware resource available)

enumerator **suspended**

thread has been suspended (waiting for synchronization event, but still known and under control of the thread-manager)

enumerator **depleted**

thread has been depleted (deeply suspended, it is not known to the thread-manager)

enumerator **terminated**

thread has been stopped and may be garbage collected

enumerator **staged**

this is not a real thread state, but allows to reference staged task descriptions, which eventually will be converted into thread objects

enumerator **pending_do_not_schedule**

this is not a real thread state, but allows to create a thread in pending state without scheduling it (internal, do not use)

enumerator **pending_boost**

this is not a real thread state, but allows to suspend a thread in pending state without high priority rescheduling

enumerator **deleted**

thread has been stopped and was deleted

enum class **thread_priority** : *std*::int8_t

This enumeration lists all possible thread-priorities for HPX threads.

Values:

enumerator **unknown**

enumerator **default_**

Will assign the priority of the task to the default (normal) priority.

enumerator **low**

Task goes onto a special low priority queue and will not be executed until all high/normal priority tasks are done, even if they are added after the low priority task.

enumerator **normal**

Task will be executed when it is taken from the normal priority queue, this is usually a first in-first-out ordering of tasks (depending on scheduler choice). This is the default priority.

enumerator **high_recursive**

The task is a high priority task and any child tasks spawned by this task will be made high priority as well - unless they are specifically flagged as non default priority.

enumerator **boost**

Same as *thread_priority_high* except that the thread will fall back to *thread_priority_normal* if resumed after being suspended.

enumerator **high**

Task goes onto a special high priority queue and will be executed before normal/low priority tasks are taken (some schedulers modify the behavior slightly and the documentation for those should be consulted).

enumerator **bound**

Task goes onto a special high priority queue and will never be stolen by another thread after initial assignment. This should be used for thread placement tasks such as OpenMP type for loops.

enumerator **initially_bound**

Task initially goes onto a special high priority queue and will never be stolen by another thread after initial assignment. If the thread is rescheduled later, it will use normal priority.

enum class **thread_restart_state** : *std*::int8_t

The *thread_restart_state* enumerator encodes the reason why a thread is being restarted

Values:

enumerator **unknown**

enumerator **signaled**

The thread has been signaled.

enumerator **timeout**

The thread has been reactivated after a timeout.

enumerator **terminate**

The thread needs to be terminated.

enumerator **abort**

The thread needs to be aborted.

enum class **thread_stacksize** : *std*::int8_t

A *thread_stacksize* references any of the possible stack-sizes for HPX threads.

Values:

enumerator **unknown**

enumerator **small_**

use small stack size (the underscore is to work around 'small' being defined to char on Windows)

enumerator **medium**

use medium sized stack size

enumerator **large**

use large stack size

enumerator **huge**

use very large stack size

enumerator **nostack**

this thread does not suspend (does not need a stack)

enumerator **current**

use size of current thread's stack

enumerator **default_**

use default stack size

enumerator **minimal**

use minimally stack size

enumerator **maximal**

use maximally stack size

enum class **thread_schedule_hint_mode** : *std*::int8_t

The type of hint given when creating new tasks.

Values:

enumerator **none**

A hint that leaves the choice of scheduling entirely up to the scheduler.

enumerator **thread**

A hint that tells the scheduler to prefer scheduling a task on the local thread number associated with this hint. Local thread numbers are indexed from zero. It is up to the scheduler to decide how to interpret thread numbers that are larger than the number of threads available to the scheduler. Typically, thread numbers will wrap around when too large.

enumerator **numa**

A hint that tells the scheduler to prefer scheduling a task on the NUMA domain associated with this hint. NUMA domains are indexed from zero. It is up to the scheduler to decide how to interpret NUMA domain indices that are larger than the number of available NUMA domains to the scheduler. Typically, indices will wrap around when too large.

enum class **thread_placement_hint** : *std*::int8_t

The type of hint given to the scheduler related to thread placement

The type of hint given to the scheduler related running a thread as a child directly in the context of the parent thread

Values:

enumerator **none**

No hint is specified. The implementation is free to chose what placement methods to use.

enumerator **depth_first**

A hint that tells the scheduler to prefer spreading thread placement on a depth-first basis (i.e. consecutively scheduled threads are placed on the same core).

enumerator **breadth_first**

A hint that tells the scheduler to prefer spreading thread placement on a breadth-first basis (i.e. consecutively scheduled threads are placed on the neighboring cores).

enumerator **depth_first_reverse**

A hint that tells the scheduler to prefer spreading thread placement on a depth-first basis (i.e. consecutively scheduled threads are placed on the same core). Threads are being scheduled in reverse order.

enumerator **breadth_first_reverse**

A hint that tells the scheduler to prefer spreading thread placement on a breadth-first basis (i.e. consecutively scheduled threads are placed on the neighboring cores). Threads are being scheduled in reverse order.

enum class **thread_sharing_hint** : *std*::int8_t

The type of hint given to the scheduler related to whether it is ok to share the invoked function object between threads

Values:

enumerator **none**

No hint is specified. The implementation is free to chose what sharing methods to use.

enumerator **do_not_share_function**

A hint that tells the scheduler to avoid sharing the given function (object) between threads.

enumerator **do_not_combine_tasks**

A hint that tells the scheduler to avoid combining tasks on the same thread. This is important for tasks that may synchronize between each other, which could lead to deadlocks if those tasks are combined running by the same thread.

enum class **thread_execution_hint** : *std*::int8_t

Values:

enumerator **none**

No hint is specified. Always run the thread in its own execution environment.

enumerator **run_as_child**

Attempt to run the thread in the execution context of the parent thread.

Functions

std::ostream &**operator<<**(*std*::ostream &os, *thread_schedule_state* t)

char const ***get_thread_state_name**(*thread_schedule_state* state) noexcept

Returns the name of the given state.

Get the readable string representing the name of the given thread_state constant.

Parameters

state – this represents the thread state.

std::ostream &**operator<<**(*std*::ostream &os, *thread_priority* t)

char const ***get_thread_priority_name**(*thread_priority* priority) noexcept

Return the thread priority name.

Get the readable string representing the name of the given thread_priority constant.

Parameters

`priority` – this represents the thread priority.

std::ostream &**operator<<**(*std*::ostream &os, *thread_restart_state* t)

char const ***get_thread_state_ex_name**(*thread_restart_state* state) noexcept

Get the readable string representing the name of the given thread_restart_state constant.

char const ***get_thread_state_name**(thread_state state) noexcept

Get the readable string representing the name of the given thread_state constant.

std::ostream &**operator<<**(*std*::ostream &os, *thread_stacksize* t)

char const ***get_stack_size_enum_name**(*thread_stacksize* size) noexcept

Returns the stack size name.

Get the readable string representing the given stack size constant.

Parameters

`size` – this represents the stack size

constexpr bool **do_not_share_function**(*thread_sharing_hint* hint) noexcept

constexpr bool **do_not_combine_tasks**(*thread_sharing_hint* hint) noexcept

constexpr *thread_sharing_hint* **operator|**(*thread_sharing_hint* lhs, *thread_sharing_hint* rhs) noexcept

constexpr bool **run_as_child**(*thread_execution_hint* hint) noexcept

Variables

constexpr *thread_execution_hint* **default_runs_as_child_hint** = *thread_execution_hint*::*none*

Default value to use for runs-as-child mode (if *true*, then futures will attempt to execute associated threads directly if they have not started running).

struct **thread_schedule_hint**

#include <thread_enums.hpp> A hint given to a scheduler to guide where a task should be scheduled.

A scheduler is free to ignore the hint, or modify the hint to suit the resources available to the scheduler.

Public Functions

inline constexpr **thread_schedule_hint**() noexcept

Construct a default hint with mode *thread_schedule_hint_mode::none*.

inline explicit constexpr **thread_schedule_hint**(*std*::int16_t const thread_hint, *thread_placement_hint* placement = *thread_placement_hint*::*none*, *thread_execution_hint* runs_as_child = *default_runs_as_child_hint*, *thread_sharing_hint* sharing = *thread_sharing_hint*::*none*) noexcept

Construct a hint with mode *thread_schedule_hint_mode::thread* and the given hint as the local thread number.

inline constexpr **thread_schedule_hint**(*thread_schedule_hint_mode* const mode, *std*::int16_t const hint, *thread_placement_hint* placement = *thread_placement_hint*::*none*, *thread_execution_hint* runs_as_child = *default_runs_as_child_hint*, *thread_sharing_hint* sharing = *thread_sharing_hint*::*none*) noexcept

Construct a hint with the given mode and hint. The numerical hint is unused when the mode is *thread_schedule_hint_mode::none*.

inline explicit constexpr **thread_schedule_hint**(*thread_placement_hint* placement) noexcept

Construct a hint with the given placement hint.

inline explicit constexpr **thread_schedule_hint**(*thread_execution_hint* runs_as_child) noexcept

Construct a hint with the given execution hint.

inline explicit constexpr **thread_schedule_hint**(*thread_sharing_hint* sharing) noexcept

Construct a hint with the given sharing hint.

inline constexpr *thread_placement_hint* **placement_mode**() const noexcept inline constexpr void **placement_mode**(*thread_placement_hint* bits) noexcept

inline constexpr *thread_sharing_hint* **sharing_mode**() const noexcept

inline constexpr void **sharing_mode**(*thread_sharing_hint* bits) noexcept

inline constexpr *thread_execution_hint* **runs_as_child_mode**() const noexcept

inline constexpr void **runs_as_child_mode**(*thread_execution_hint* bits) noexcept

inline constexpr void **schedule_hint**(*std*::int16_t core) noexcept

Public Members

std::int16_t **hint** = -1

The hint associated with the mode. The interpretation of this hint depends on the given mode.

thread_schedule_hint_mode **mode** = *thread_schedule_hint_mode*::*none*

The mode of the scheduling hint.

std::uint8_t **placement_mode_bits**

The mode of the desired thread placement.

std::uint8_t **sharing_mode_bits**

The mode of the desired sharing hint.

std::uint8_t **runs_as_child_mode_bits**

The thread will run as a child directly in the context of the current thread

datastructures

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::ignore

Defined in header hpx/tuple.hpp[845].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::tuple*
- *hpx::tuple_size*
- *hpx::tuple_element*
- *hpx::make_tuple*
- *hpx::tie*
- *hpx::forward_as_tuple*
- *hpx::tuple_cat*
- *hpx::get*

constexpr *hpx*::detail::ignore_type `hpx::ignore` = {}

An object of unspecified type such that any value can be assigned to it with no effect. Intended for use with *hpx::tie* when unpacking a *hpx::tuple*, as a placeholder for the arguments that are not used. While the behavior of *hpx::ignore* outside of *hpx::tie* is not formally specified, some code guides recommend using *hpx::ignore* to avoid warnings from unused return values of [[nodiscard]] functions.

hpx::tuple

Defined in header hpx/tuple.hpp[846].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ignore*
- *hpx::tuple_size*
- *hpx::tuple_element*
- *hpx::make_tuple*
- *hpx::tie*
- *hpx::forward_as_tuple*
- *hpx::tuple_cat*
- *hpx::get*

[845] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/tuple.hpp
[846] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/tuple.hpp

template<typename ...**Ts**>

class **tuple**

Class template *hpx::tuple* is a fixed-size collection of heterogeneous values. It is a generalization of *hpx::pair*. If *std::is_trivially_destructible<Ti>::value* is true for every *Ti* in *Ts*, the destructor of tuple is trivial.

Param Ts...
the types of the elements that the tuple stores.

hpx::tuple_size

Defined in header hpx/tuple.hpp[847].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ignore*
- *hpx::tuple*
- *hpx::tuple_element*
- *hpx::make_tuple*
- *hpx::tie*
- *hpx::forward_as_tuple*
- *hpx::tuple_cat*
- *hpx::get*

template<typename **T**>

struct **tuple_size**

Provides access to the number of elements in a tuple-like type as a compile-time constant expression.

The primary template is not defined. An explicit (full) or partial specialization is required to make a type tuple-like.

hpx::tuple_element

Defined in header hpx/tuple.hpp[848].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ignore*
- *hpx::tuple*
- *hpx::tuple_size*

[847] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/tuple.hpp
[848] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/tuple.hpp

- *hpx::make_tuple*
- *hpx::tie*
- *hpx::forward_as_tuple*
- *hpx::tuple_cat*
- *hpx::get*

template<*std*::size_t **I**, typename **T**, typename **Enable** = void>

struct **tuple_element**

Provides compile-time indexed access to the types of the elements of a tuple-like type.

The primary template is not defined. An explicit (full) or partial specialization is required to make a type tuple-like.

hpx::make_tuple

Defined in header hpx/tuple.hpp[849].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ignore*
- *hpx::tuple*
- *hpx::tuple_size*
- *hpx::tuple_element*
- *hpx::tie*
- *hpx::forward_as_tuple*
- *hpx::tuple_cat*
- *hpx::get*

template<typename ...**Ts**>

constexpr *tuple*<*util*::decay_unwrap_t<*Ts*>...> *hpx*::**make_tuple**(*Ts*&&... ts)

Provides compile-time indexed access to the types of the elements of the tuple.

hpx::tie

Defined in header hpx/tuple.hpp[850].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ignore*
- *hpx::tuple*

[849] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/tuple.hpp
[850] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/tuple.hpp

- *hpx::tuple_size*
- *hpx::tuple_element*
- *hpx::make_tuple*
- *hpx::forward_as_tuple*
- *hpx::tuple_cat*
- *hpx::get*

template<typename ...**Ts**>

constexpr *tuple*<*Ts*&...> `hpx::tie`(*Ts*&... ts)

Creates a tuple of lvalue references to its arguments or instances of *hpx::ignore*.

Parameters
`ts` – zero or more lvalue arguments to construct the tuple from.

Returns
hpx::tuple object containing lvalue references.

hpx::forward_as_tuple

Defined in header hpx/tuple.hpp[851].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ignore*
- *hpx::tuple*
- *hpx::tuple_size*
- *hpx::tuple_element*
- *hpx::make_tuple*
- *hpx::tie*
- *hpx::tuple_cat*
- *hpx::get*

template<typename ...**Ts**>

constexpr *tuple*<*Ts*&&...> `hpx::forward_as_tuple`(*Ts*&&... ts)

Constructs a tuple of references to the arguments in args suitable for forwarding as an argument to a function. The tuple has rvalue reference data members when rvalues are used as arguments, and otherwise has lvalue reference data members.

Parameters
`ts` – zero or more arguments to construct the tuple from

Returns
hpx::tuple object created as if by

[851] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/tuple.hpp

```
hpx::tuple<Ts&&...>(HPX_FORWARD(Ts, ts)...)
```

hpx::tuple_cat

Defined in header hpx/tuple.hpp[852].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ignore*
- *hpx::tuple*
- *hpx::tuple_size*
- *hpx::tuple_element*
- *hpx::make_tuple*
- *hpx::tie*
- *hpx::forward_as_tuple*
- *hpx::get*

template<typename ...**Tuples**>

constexpr auto *hpx*::**tuple_cat**(*Tuples*&&... tuples)

Constructs a tuple that is a concatenation of all tuples in `tuples`. The behavior is undefined if any type in `std::decay_t<Tuples>...` is not a specialization of *`hpx::tuple`*. However, an implementation may choose to support types (such as `std::array` and `std::pair`) that follow the tuple-like protocol.

Parameters

`tuples` – - zero or more tuples to concatenate

Returns

hpx::tuple object composed of all elements of all argument tuples constructed from *`hpx::get<Is>`*(HPX_FORWARD(UTuple,t) for each individual element.

hpx::get

Defined in header hpx/tuple.hpp[853].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ignore*
- *hpx::tuple*
- *hpx::tuple_size*
- *hpx::tuple_element*

[852] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/tuple.hpp
[853] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/tuple.hpp

- *hpx::make_tuple*
- *hpx::tie*
- *hpx::forward_as_tuple*
- *hpx::tuple_cat*

template<*std*::size_t **I**>

util::at_index<*I*, Ts...>::type &*hpx*::**get**() noexcept

Extracts the Ith element from the tuple. I must be an integer value in [0, sizeof…(Ts)).

template<*std*::size_t **I**>

util::at_index<*I*, Ts...>::type const &*hpx*::**get**() const noexcept

Extracts the Ith element from the tuple. I must be an integer value in [0, sizeof…(Ts)).

hpx::any_nonser. hpx::bad_any_cast

Defined in header hpx/any.hpp[854].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::unique_any_nonser*
- *hpx::any_cast*
- *hpx::make_any_nonser*
- *hpx::make_unique_any_nonser*

Warning

doxygenfunction: Cannot find function "hpx::any_nonser. hpx::bad_any_cast" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::unique_any_nonser

Defined in header hpx/any.hpp[855].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::any_nonser. hpx::bad_any_cast*
- *hpx::any_cast*
- *hpx::make_any_nonser*
- *hpx::make_unique_any_nonser*

[854] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/any.hpp

[855] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/any.hpp

using *hpx*::**unique_any_nonser** = *util*::basic_any<void, void, void, *std*::false_type>

hpx::any_cast

Defined in header hpx/any.hpp[856].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::any_nonser. hpx::bad_any_cast*
- *hpx::unique_any_nonser*
- *hpx::make_any_nonser*
- *hpx::make_unique_any_nonser*

template<typename **T**, typename **IArch**, typename **OArch**, typename **Char**, typename **Copyable**>

T **hpx*::**any_cast**(*util*::basic_any<*IArch*, *OArch*, *Char*, *Copyable*> *operand) noexcept

Performs type-safe access to the contained object.

Parameters
operand – target any object

Returns
If operand is not a null pointer, and the *typeid* of the requested *T* matches that of the contents of *operand*, a pointer to the value contained by *operand*, otherwise a null pointer.

template<typename **T**, typename **IArch**, typename **OArch**, typename **Char**, typename **Copyable**>

T const **hpx*::**any_cast**(*util*::basic_any<*IArch*, *OArch*, *Char*, *Copyable*> const *operand) noexcept

Performs type-safe access to the contained object.

Parameters
operand – target any object

Returns
If operand is not a null pointer, and the *typeid* of the requested *T* matches that of the contents of *operand*, a pointer to the value contained by *operand*, otherwise a null pointer.

template<typename **T**, typename **IArch**, typename **OArch**, typename **Char**, typename **Copyable**>

T *hpx*::**any_cast**(*util*::basic_any<*IArch*, *OArch*, *Char*, *Copyable*> &operand)

Performs type-safe access to the contained object. Let *U* be *std::remove_cv_t<std::remove_reference_t<T>>* The program is ill-formed if *std::is_constructible_v<T, U&>* is false.

Parameters
operand – target any object

[856] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/any.hpp

Returns

static_cast<T>(*std::any_cast<U>(&operand))

template<typename **T**, typename **IArch**, typename **OArch**, typename **Char**, typename **Copyable**>

T const &*hpx*::**any_cast**(*util*::basic_any<*IArch*, *OArch*, *Char*, *Copyable*> const &operand)

Performs type-safe access to the contained object. Let *U* be *std::remove_cv_t<std::remove_reference_t<T>>* The program is ill-formed if *std::is_constructible_v<T, const U&>* is false.

Parameters

operand – target any object

Returns

static_cast<T>(*std::any_cast<U>(&operand))

hpx::make_any_nonser

Defined in header hpx/any.hpp[857].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::any_nonser. hpx::bad_any_cast*
- *hpx::unique_any_nonser*
- *hpx::any_cast*
- *hpx::make_unique_any_nonser*

template<typename **T**, typename ...**Ts**>

util::basic_any<void, void, void, *std*::true_type> *hpx*::**make_any_nonser**(*Ts*&&... ts)

template<typename **T**, typename **U**, typename ...**Ts**>

util::basic_any<void, void, void, *std*::true_type> *hpx*::**make_any_nonser**(*std*::initializer_list<*U*> il, *Ts*&&... ts)

template<typename **T**>

util::basic_any<void, void, void, *std*::true_type> *hpx*::**make_any_nonser**(*T* &&t)

[857] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/any.hpp

hpx::make_unique_any_nonser

Defined in header hpx/any.hpp[858].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::any_nonser. hpx::bad_any_cast*
- *hpx::unique_any_nonser*
- *hpx::any_cast*
- *hpx::make_any_nonser*

template<typename **T**, typename ...**Ts**>

util::basic_any<void, void, void, *std*::false_type> *hpx*::**make_unique_any_nonser**(*Ts*&&... ts)

template<typename **T**, typename **U**, typename ...**Ts**>

util::basic_any<void, void, void, *std*::false_type> *hpx*::**make_unique_any_nonser**(*std*::initializer_list<*U*> il, *Ts*&&... ts)

template<typename **T**>

util::basic_any<void, void, void, *std*::false_type> *hpx*::**make_unique_any_nonser**(*T* &&t)

hpx::any

Defined in header hpx/any.hpp[859].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::make_any*

using *hpx*::**any** = *util*::basic_any<*serialization*::input_archive, *serialization*::output_archive, char, *std*::true_type>

[858] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/any.hpp
[859] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/any.hpp

hpx::make_any

Defined in header hpx/any.hpp[860].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::any*

template<typename **T**, typename **Char**>

util::basic_any<*serialization*::input_archive, *serialization*::output_archive, *Char*> `hpx::make_any`(*T* &&t)

Constructs an any object containing an object of type *T*, passing the provided arguments to T's constructor. Equivalent to:

```
return std::any(std::in_place_type<T>, std::forward<Args>(args)...);
```

debugging

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/debugging/print.hpp

Defined in header hpx/debugging/print.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

errors

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::error_code

Defined in header hpx/system_error.hpp[861].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

class **error_code** : public *std*::error_code

A *hpx::error_code* represents an arbitrary error condition.

The class *hpx::error_code* describes an object used to hold error code values, such as those originating from the operating system or other low-level application program interfaces.

[860] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/any.hpp
[861] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/system_error.hpp

> **Note**
>
> Class *hpx::error_code* is an adjunct to error reporting by exception

hpx::exception

Defined in header hpx/exception.hpp[862].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

class **exception** : public *std*::system_error

A *hpx::exception* is the main exception type used by HPX to report errors.

The *hpx::exception* type is the main exception type used by HPX to report errors. Any exceptions thrown by functions in the HPX library are either of this type or of a type derived from it. This implies that it is always safe to use this type only in catch statements guarding HPX library calls.

Subclassed by hpx::bad_alloc_exception, hpx::cuda::experimental::cuda_exception, *hpx::exception_list*, *hpx::experimental::task_canceled_exception*, hpx::mpi::experimental::mpi_exception

hpx/errors/define_error_info.hpp

Defined in header hpx/errors/define_error_info.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/errors/macros.hpp

Defined in header hpx/errors/macros.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Defines

HPX_THROW_EXCEPTION(errcode, f, ...)

Throw a *hpx::exception* initialized from the given parameters.

The macro *HPX_THROW_EXCEPTION* can be used to throw a *hpx::exception*. The purpose of this macro is to prepend the source file name and line number of the position where the exception is thrown to the error message. Moreover, this associates additional diagnostic information with the exception, such as file name and line number, locality id and thread id, and stack backtrace from the point where the exception was thrown.

The parameter `errcode` holds the hpx::error code the new exception should encapsulate. The parameter `f` is expected to hold the name of the function exception is thrown from and the parameter `msg` holds the error message the new exception should encapsulate.

[862] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/exception.hpp

```
void raise_exception()
{
    ↪// Throw a hpx::exception initialized from the given parameters.
    // Additionally associate with this exception some detailed
    // diagnostic information about the throw-site.
    HPX_THROW_EXCEPTION(hpx::error::no_success, "raise_exception",
        "simulated error");
}
```

Example:

HPX_THROW_EXCEPTION_MODE(errcode, mode, f, ...)

Throw a *hpx::exception* initialized from the given parameters.

The macro *HPX_THROW_EXCEPTION_MODE* can be used to throw a *hpx::exception*. The purpose of this macro is to prepend the source file name and line number of the position where the exception is thrown to the error message. Moreover, this associates additional diagnostic information with the exception, such as file name and line number, locality id and thread id, and stack backtrace from the point where the exception was thrown.

The parameter `errcode` holds the hpx::error code the new exception should encapsulate. The parameter `f` is expected to hold the name of the function exception is thrown from and the parameter `msg` holds the error message the new exception should encapsulate. The `throwmode` parameter allows to specify the type of the thrown exception.

```
void raise_exception() {
    ↪// Throw a hpx::exception initialized from the given parameters.
    HPX_THROW_EXCEPTION_MODE(hpx::error::no_success,
        hpx::throwmode::lighweight, "raise_exception",
        "simulated error");
}
```

Example:

HPX_THROW_BAD_ALLOC(f)

Throw a hpx::bad_alloc_exception initialized from the given parameters.

The macro *HPX_THROW_BAD_ALLOC* can be used to throw a *hpx::exception*. The purpose of this macro is to prepend the source file name and line number of the position where the exception is thrown to the error message. Moreover, this associates additional diagnostic information with the exception, such as file name and line number, locality id and thread id, and stack backtrace from the point where the exception was thrown.

The parameter `errcode` holds the hpx::error code the new exception should encapsulate. The parameter `f` is expected to hold the name of the function exception is thrown from and the parameter `msg` holds the error message the new exception should encapsulate.

```
void raise_exception()
{
    ␣
↪// Throw a hpx::exception initialized from the given parameters.
    // Additionally associate with this exception some detailed
    // diagnostic information about the throw-site.
    HPX_THROW_BAD_ALLOC("raise_exception", "simulated error");
}
```

Example:

`HPX_THROWS_IF`(ec, errcode, f, ...)

Either throw a *hpx::exception* or initialize hpx::error_code from the given parameters.

The macro *HPX_THROWS_IF* can be used to either throw a hpx::exception or to initialize a hpx::error_code from the given parameters. If &ec == &hpx::throws, the semantics of this macro are equivalent to *HPX_THROW_EXCEPTION*. If &ec != &hpx::throws, the hpx::error_code instance `ec` is initialized instead.

The parameter `errcode` holds the hpx::error code from which the new exception should be initialized. The parameter `f` is expected to hold the name of the function exception is thrown from and the parameter `msg` holds the error message the new exception should encapsulate.

`HPX_THROWS_BAD_ALLOC_IF`(ec, f)

Either throw a hpx::bad_alloc_exception or hpx::error_code to out_of_memory.

The macro *HPX_THROWS_BAD_ALLOC_IF* can be used to either throw a *hpx::bad_alloc_exception* or to initialize a hpx::error_code to hpx::error::out_of_memory. If &ec == &hpx::throws, the semantics of this macro are equivalent to *HPX_THROW_BAD_ALLOC*. If &ec != &hpx::throws, the hpx::error_code instance `ec` is initialized instead.

`HPX_DEFINE_ERROR_INFO`(NAME, TYPE)

hpx/errors/try_catch_exception_ptr.hpp

Defined in header hpx/errors/try_catch_exception_ptr.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

HPX_THROW_EXCEPTION

Defined in header hpx/exception.hpp[863].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *HPX_THROW_BAD_ALLOC*
- *HPX_THROWS_IF*

> **Warning**
>
> doxygenfunction: Cannot find function "HPX_THROW_EXCEPTION" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

HPX_THROW_BAD_ALLOC

Defined in header hpx/exception.hpp[864].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *HPX_THROW_EXCEPTION*
- *HPX_THROWS_IF*

> **Warning**
>
> doxygenfunction: Cannot find function "HPX_THROW_BAD_ALLOC" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

[863] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/exception.hpp

[864] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/exception.hpp

HPX_THROWS_IF

Defined in header hpx/exception.hpp[865].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *HPX_THROW_EXCEPTION*
- *HPX_THROW_BAD_ALLOC*

> **Warning**
>
> doxygenfunction: Cannot find function "HPX_THROWS_IF" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx/errors/exception_list.hpp

Defined in header hpx/errors/exception_list.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

class **exception_list** : public *hpx::exception*

#include <exception_list.hpp> The class *exception_list* is a container of exception_ptr objects parallel algorithms may use to communicate uncaught exceptions encountered during parallel execution to the caller of the algorithm

The type exception_list::const_iterator fulfills the requirements of a forward iterator.

Public Types

using **iterator** = exception_list_type::const_iterator

bidirectional iterator

Public Functions

inline *std*::size_t **size**() const noexcept

The number of exception_ptr objects contained within the *exception_list*.

> **Note**
>
> Complexity: Constant time.

[865] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/exception.hpp

inline exception_list_type::const_iterator **begin()** const noexcept

An iterator referring to the first exception_ptr object contained within the *exception_list*.

inline exception_list_type::const_iterator **end()** const noexcept

An iterator which is the past-the-end value for the *exception_list*.

hpx/errors/error.hpp

Defined in header hpx/errors/error.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Defines

HPX_ERROR_UNSCOPED_ENUM_DEPRECATION_MSG

namespace **hpx**

Enums

enum class **error** : *std*::int16_t

Possible error conditions.

This enumeration lists all possible error conditions which can be reported from any of the API functions.

Values:

enumerator **success**

The operation was successful.

enumerator **no_success**

The operation did failed, but not in an unexpected manner.

enumerator **not_implemented**

The operation is not implemented.

enumerator **out_of_memory**

The operation caused an out of memory condition.

enumerator **bad_action_code**

enumerator **bad_component_type**

The specified component type is not known or otherwise invalid.

enumerator **network_error**

A generic network error occurred.

enumerator **version_too_new**

The version of the network representation for this object is too new.

enumerator **version_too_old**

The version of the network representation for this object is too old.

enumerator **version_unknown**

The version of the network representation for this object is unknown.

enumerator **unknown_component_address**

enumerator **duplicate_component_address**

The given global id has already been registered.

enumerator **invalid_status**

The operation was executed in an invalid status.

enumerator **bad_parameter**

One of the supplied parameters is invalid.

enumerator **internal_server_error**

enumerator **service_unavailable**

enumerator **bad_request**

enumerator **repeated_request**

enumerator **lock_error**

enumerator **duplicate_console**

There is more than one console locality.

enumerator **no_registered_console**

There is no registered console locality available.

enumerator **startup_timed_out**

enumerator **uninitialized_value**

enumerator **bad_response_type**

enumerator **deadlock**

enumerator **assertion_failure**

enumerator **null_thread_id**

Attempt to invoke an API function from a non-HPX thread.

enumerator **invalid_data**

enumerator **yield_aborted**

The yield operation was aborted.

enumerator **dynamic_link_failure**

enumerator **commandline_option_error**

One of the options given on the command line is erroneous.

enumerator **serialization_error**

There was an error during serialization of this object.

enumerator **unhandled_exception**

An unhandled exception has been caught.

enumerator **kernel_error**

The OS kernel reported an error.

enumerator **broken_task**

The task associated with this future object is not available anymore.

enumerator **task_moved**

The task associated with this future object has been moved.

enumerator **task_already_started**

The task associated with this future object has already been started.

enumerator **future_already_retrieved**

The future object has already been retrieved.

enumerator **promise_already_satisfied**

The value for this future object has already been set.

enumerator **future_does_not_support_cancellation**

The future object does not support cancellation.

enumerator **future_can_not_be_cancelled**

The future can't be canceled at this time.

enumerator **no_state**

The future object has no valid shared state.

enumerator **broken_promise**

The promise has been deleted.

enumerator **thread_resource_error**

enumerator **future_cancelled**

enumerator **thread_cancelled**

enumerator **thread_not_interruptable**

enumerator **duplicate_component_id**

The component type has already been registered.

enumerator **unknown_error**

An unknown error occurred.

enumerator **bad_plugin_type**

The specified plugin type is not known or otherwise invalid.

enumerator **filesystem_error**

The specified file does not exist or other filesystem related error.

enumerator **bad_function_call**

equivalent of std::bad_function_call

enumerator **task_canceled_exception**

parallel::task_canceled_exception

enumerator **task_block_not_active**

task_region is not active

enumerator **out_of_range**

Equivalent to std::out_of_range.

enumerator **length_error**

Equivalent to std::length_error.

enumerator **migration_needs_retry**

migration failed because of global race, retry

enumerator **stale_state**

The queried state may be stale, e.g. because no state has been recorded yet or a preceding notification has not yet been delivered

enumerator **target_fenced**

A dispatch was deliberately and correctly refused because the target had already latched a terminal lifecycle event; this is a final rejection, not a staleness signal, and should not be retried

enumerator **future_wait_timed_out**

future wait timed out

enumerator **locality_was_disconnected**

the requested target locality was disconnected

enumerator **future_uncompleted**

future was resumed while being empty

Functions

constexpr bool **operator==**(int lhs, *error* rhs) noexcept

constexpr bool **operator==**(*error* lhs, int rhs) noexcept

constexpr bool **operator!=**(int lhs, *error* rhs) noexcept

constexpr bool **operator!=**(*error* lhs, int rhs) noexcept

constexpr bool **operator<**(int lhs, *error* rhs) noexcept

constexpr bool **operator>=**(int lhs, *error* rhs) noexcept

constexpr int **operator&**(*error* lhs, *error* rhs) noexcept

constexpr int **operator&**(int lhs, *error* rhs) noexcept

constexpr int **operator|=**(int &lhs, *error* rhs) noexcept

char const ***get_error_name**(*error* e) noexcept

Variables

constexpr *error* **success** = *error*::*success*

constexpr *error* **no_success** = *error*::*no_success*

constexpr *error* **not_implemented** = *error*::*not_implemented*

constexpr *error* **out_of_memory** = *error*::*out_of_memory*

constexpr *error* **bad_action_code** = *error*::*bad_action_code*

constexpr *error* **bad_component_type** = *error*::*bad_component_type*

constexpr *error* **network_error** = *error*::*network_error*

constexpr *error* **version_too_new** = *error*::*version_too_new*

constexpr *error* **version_too_old** = *error*::*version_too_old*

constexpr *error* **version_unknown** = *error*::*version_unknown*

constexpr *error* **unknown_component_address** = *error*::*unknown_component_address*

constexpr *error* **duplicate_component_address** = *error*::*duplicate_component_address*

constexpr *error* **invalid_status** = *error*::*invalid_status*

constexpr *error* **bad_parameter** = *error*::*bad_parameter* constexpr *error* `internal_server_error` = *error*::*internal_server_error*

constexpr *error* `service_unavailable` = *error*::*service_unavailable*

constexpr *error* `bad_request` = *error*::*bad_request*

constexpr *error* `repeated_request` = *error*::*repeated_request*

constexpr *error* `lock_error` = *error*::*lock_error*

constexpr *error* `duplicate_console` = *error*::*duplicate_console*

constexpr *error* `no_registered_console` = *error*::*no_registered_console*

constexpr *error* `startup_timed_out` = *error*::*startup_timed_out*

constexpr *error* `uninitialized_value` = *error*::*uninitialized_value*

constexpr *error* `bad_response_type` = *error*::*bad_response_type*

constexpr *error* `deadlock` = *error*::*deadlock*

constexpr *error* `assertion_failure` = *error*::*assertion_failure*

constexpr *error* `null_thread_id` = *error*::*null_thread_id*

constexpr *error* `invalid_data` = *error*::*invalid_data* constexpr *error* `yield_aborted` = *error*::*yield_aborted*

constexpr *error* `dynamic_link_failure` = *error*::*dynamic_link_failure*

constexpr *error* `commandline_option_error` = *error*::*commandline_option_error*

constexpr *error* `serialization_error` = *error*::*serialization_error*

constexpr *error* `unhandled_exception` = *error*::*unhandled_exception*

constexpr *error* `kernel_error` = *error*::*kernel_error*

constexpr *error* `broken_task` = *error*::*broken_task*

constexpr *error* `task_moved` = *error*::*task_moved*

constexpr *error* `task_already_started` = *error*::*task_already_started*

constexpr *error* `future_already_retrieved` = *error*::*future_already_retrieved*

constexpr *error* `promise_already_satisfied` = *error*::*promise_already_satisfied*

constexpr *error* `future_does_not_support_cancellation` = *error*::*future_does_not_support_cancellation*

constexpr *error* `future_can_not_be_cancelled` = *error*::*future_can_not_be_cancelled*

constexpr *error* `no_state` = *error*::*no_state* constexpr *error* `broken_promise` = *error*::*broken_promise*

constexpr *error* `thread_resource_error` = *error*::*thread_resource_error*

constexpr *error* `future_cancelled` = *error*::*future_cancelled*

constexpr *error* `thread_cancelled` = *error*::*thread_cancelled*

constexpr *error* `thread_not_interruptable` = *error*::*thread_not_interruptable*

constexpr *error* `duplicate_component_id` = *error*::*duplicate_component_id*

constexpr *error* `unknown_error` = *error*::*unknown_error*

constexpr *error* `bad_plugin_type` = *error*::*bad_plugin_type*

constexpr *error* `filesystem_error` = *error*::*filesystem_error*

constexpr *error* `bad_function_call` = *error*::*bad_function_call*

constexpr *error* `task_canceled_exception` = *error*::*task_canceled_exception*

constexpr *error* `task_block_not_active` = *error*::*task_block_not_active*

constexpr *error* `out_of_range` = *error*::*out_of_range*

constexpr *error* `length_error` = *error*::*length_error* constexpr *error* **migration_needs_retry** = *error*::*migration_needs_retry*

hpx/errors/exception_fwd.hpp

Defined in header hpx/errors/exception_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Enums

enum class **throwmode** : *std*::uint8_t

Encode error category for new *error_code*.

Values:

enumerator **plain**

enumerator **rethrow**

enumerator **lightweight**

Functions

constexpr bool **operator&**(*throwmode* lhs, *throwmode* rhs) noexcept

Variables

error_code **throws**

Predefined *error_code* object used as "throw on error" tag.

The predefined *hpx::error_code* object *hpx::throws* is supplied for use as a "throw on error" tag.

Functions that specify an argument in the form '*error_code*& ec=throws' (with appropriate namespace qualifiers), have the following error handling semantics:

If &ec != &throws and an error occurred: ec.value() returns the implementation specific error number for the particular error that occurred and ec.category() returns the error_category for ec.value().

If &ec != &throws and an error did not occur, ec.clear().

If an error occurs and &ec == &throws, the function throws an exception of type hpx::exception or of a type derived from it. The exception's *get_errorcode()* member function returns a reference to a hpx::error_code object with the behavior as specified above.

execution

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/execution/algorithms/future_sender.hpp

Defined in header hpx/execution/algorithms/future_sender.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Implementation details for adapting HPX futures as P2300 senders.

Usage:

```
hpx::future<int> f = hpx::async([]{ return 42; });
auto
  result = hpx::execution::experimental::as_sender(std::move(f))

  | hpx::execution::experimental::then([](int x){ return x * 2; })
    | hpx::execution::experimental::sync_wait();
```

namespace **hpx**

namespace **execution**

namespace **experimental**

hpx/execution/algorithms/sender_future.hpp

Defined in header hpx/execution/algorithms/sender_future.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Converts any P2300 stdexec sender into an hpx::future<T>.

Usage:

```
// Explicit T:
auto sender␣
↪= stdexec::just(42) | stdexec::then([](int x){ return x; });
hpx::future<int> f =
   ␣
↪hpx::execution::experimental::as_future<int>(std::move(sender));
int val = f.get();   // == 42
```

namespace **hpx**

namespace **execution**

namespace **experimental**

Variables

template<typename **T**>

constexpr *as_future_t*<*T*> **as_future** = {}

template<typename **T**>

struct **as_future_t**

Public Functions

template<typename **Sender**, typename = *std*::enable_if_t<*hpx*::*execution*::*experimental*::is_sender_v<*std*::decay_t<*Sender*>>>>

inline constexpr *hpx*::*future*<*T*> **operator()**(*Sender* &&sender) const

hpx/execution/traits/is_execution_policy.hpp

Defined in header hpx/execution/traits/is_execution_policy.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Variables

template<typename **T**>

constexpr bool **is_execution_policy_v** = *is_execution_policy*<*T*>::value

template<typename T> concept execution_policy = is_execution_policy<T>::value

template<typename **T**>

constexpr bool **is_parallel_execution_policy_v** = *is_parallel_execution_policy*<*T*>::value

template<typename **T**>

constexpr bool **is_sequenced_execution_policy_v** = *is_sequenced_execution_policy*<*T*>::value

template<typename **T**>

constexpr bool **is_async_execution_policy_v** = *is_async_execution_policy*<*T*>::value

template<typename **T**>

struct **is_execution_policy** : public *hpx*::detail::is_execution_policy<*std*::decay_t<*T*>>

#include <is_execution_policy.hpp>

i. The type *is_execution_policy* can be used to detect execution policies for the purpose of excluding function signatures from otherwise ambiguous overload resolution participation.

ii. If T is the type of the standard or implementation-defined execution policy, is_execution_policy<T> shall be publicly derived from integral_constant<bool, true>, otherwise from integral_constant<bool, false>.

iii. The behavior of a program that adds specializations for *is_execution_policy* is undefined.

template<typename **T**>

struct **is_parallel_execution_policy** : public *hpx*::detail::is_parallel_execution_policy<*std*::decay_t<*T*>>

#include <is_execution_policy.hpp> Extension: Detect whether given execution policy enables parallelization

i. The type *is_parallel_execution_policy* can be used to detect parallel execution policies for the purpose of excluding function signatures from otherwise ambiguous overload resolution participation.

ii. If T is the type of the standard or implementation-defined execution policy, is_parallel_execution_policy<T> shall be publicly derived from integral_constant<bool, true>, otherwise from integral_constant<bool, false>.

iii. The behavior of a program that adds specializations for *is_parallel_execution_policy* is undefined.

template<typename **T**>

struct **is_sequenced_execution_policy** : public *hpx*::detail::is_sequenced_execution_policy<*std*::decay_t<*T*>>

#include <is_execution_policy.hpp> Extension: Detect whether given execution policy does not enable parallelization

i. The type *is_sequenced_execution_policy* can be used to detect non-parallel execution policies for the purpose of excluding function signatures from otherwise ambiguous overload resolution participation.

ii. If T is the type of the standard or implementation-defined execution policy, is_sequenced_execution_policy<T> shall be publicly derived from integral_constant<bool, true>, otherwise from integral_constant<bool, false>.

iii. The behavior of a program that adds specializations for *is_sequenced_execution_policy* is undefined.

template<typename **T**>

struct **is_async_execution_policy** : public *hpx*::detail::is_async_execution_policy<*std*::decay_t<*T*>>

#include <is_execution_policy.hpp> Extension: Detect whether given execution policy makes algorithms asynchronous

i. The type *is_async_execution_policy* can be used to detect asynchronous execution policies for the purpose of excluding function signatures from otherwise ambiguous overload resolution participation.

ii. If T is the type of the standard or implementation-defined execution policy, is_async_execution_policy<T> shall be publicly derived from integral_constant<bool, true>, otherwise from integral_constant<bool, false>.

iii. The behavior of a program that adds specializations for *is_async_execution_policy* is undefined.

hpx::execution::experimental::static_chunk_size

Defined in header hpx/execution.hpp[866].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

struct **static_chunk_size**

Loop iterations are divided into pieces of size *chunk_size* and then assigned to threads. If *chunk_size* is not specified, the iterations are evenly (if possible) divided contiguously among the threads.

> **Note**
>
> This executor parameters type is equivalent to OpenMP's STATIC scheduling directive.

hpx/execution/executors/adaptive_static_chunk_size.hpp

Defined in header hpx/execution/executors/adaptive_static_chunk_size.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

struct **adaptive_static_chunk_size**

#include <adaptive_static_chunk_size.hpp> Loop iterations are divided into pieces of size *chunk_size* and then assigned to threads. If *chunk_size* is not specified, the iterations are evenly (if possible) divided contiguously among the threads.

> **Note**
>
> This executor parameters type is equivalent to OpenMP's STATIC scheduling directive.

[866] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

Public Functions

adaptive_static_chunk_size() = default

Construct a adaptive_static_chunk_size executor parameters object

> **Note**
>
> By default the number of loop iterations is determined from the number of available cores and the overall number of loop iterations to schedule.

inline explicit constexpr **adaptive_static_chunk_size**(*std*::size_t const chunk_size) noexcept

Construct a adaptive_static_chunk_size executor parameters object

Parameters

chunk_size – [in] The optional chunk size to use as the number of loop iterations to run on a single thread.

hpx/execution/executors/execution_information.hpp

Defined in header hpx/execution/executors/execution_information.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

Variables

constexpr struct *hpx*::*execution*::*experimental*::*has_pending_closures_t* **has_pending_closures**

constexpr struct *hpx*::*execution*::*experimental*::*get_pu_mask_t* **get_pu_mask**

constexpr struct *hpx*::*execution*::*experimental*::*set_scheduler_mode_t* **set_scheduler_mode** struct **has_pending_closures_t**

#include <execution_information.hpp> Retrieve whether this executor has operations pending or not.

> **Note**
>
> If the executor does not expose this information, this call will always return *false*

Param exec
[in] The executor object to use to extract the requested information for.

Public Functions

template<typename **Executor**>

inline decltype(auto) **operator()**(*Executor* &&exec) const

template<typename **Executor**>

inline decltype(auto) **operator()**(*Executor*&&) const

struct **get_pu_mask_t**

#include <execution_information.hpp> Retrieve the bitmask describing the processing units the given thread is allowed to run on

All threads::executors invoke sched.get_pu_mask().

> **Note**
>
> If the executor does not support this operation, this call will always invoke hpx::threads::get_pu_mask()

Param exec
[in] The executor object to use for querying the number of pending tasks.
Param topo
[in] The topology object to use to extract the requested information.
Param thream_num
[in] The sequence number of the thread to retrieve information for.

Public Functions

template<typename **Executor**>

inline decltype(auto) **operator()**(*Executor* &&exec, *threads*::*topology* &topo, *std*::size_t thread_num) const

template<typename **Executor**>

inline decltype(auto) **operator()**(*Executor*&&, *threads*::*topology* &topo, *std*::size_t thread_num) const

struct **set_scheduler_mode_t**

#include <execution_information.hpp> Set various modes of operation on the scheduler underneath the given executor.

> **Note**
>
> This calls exec.set_scheduler_mode(mode) if it exists; otherwise it does nothing.

Param exec
[in] The executor object to use.

Param mode
[in] The new mode for the scheduler to pick up

Public Functions

template<typename **Executor**, typename **Mode**>

inline void **operator()**(*Executor* &&exec, *Mode* const &mode) const

template<typename **Executor**, typename **Mode**>

inline void **operator()**(*Executor*&&, *Mode* const&) const

hpx::execution::experimental::guided_chunk_size

Defined in header hpx/execution.hpp[867].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

[867] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

struct **guided_chunk_size**

Iterations are dynamically assigned to threads in blocks as threads request those until no blocks remain to be assigned. Similar to dynamic_chunk_size except that the block size decreases each time a number of loop iterations is given to a thread. The size of the initial block is proportional to *number_of_iterations* / *number_of_cores*. Subsequent blocks are proportional to *number_of_iterations_remaining* / *number_of_cores*. The optional chunk size parameter defines the minimum block size. The default chunk size is 1.

> **Note**
>
> This executor parameters type is equivalent to OpenMP's GUIDED scheduling directive.

hpx::execution::experimental::auto_chunk_size

Defined in header hpx/execution.hpp[868].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

struct **auto_chunk_size**

Loop iterations are divided into pieces and then assigned to threads. The number of loop iterations combined is determined based on measurements of how long the execution of 1% of the overall number of iterations takes. This executor parameters type makes sure that as many loop iterations are combined as necessary to run for the amount of time specified.

hpx/execution/executors/execution_parameters.hpp

Defined in header hpx/execution/executors/execution_parameters.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

[868] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

Functions

template<typename ...**Params**>

constexpr *executor_parameters_join*<*Params*...>::type **join_executor_parameters**(*Params*&&... params)

template<typename **Param**>

constexpr *Param* &&**join_executor_parameters**(*Param* &&param) noexcept

template<typename ...**Params**>

struct **executor_parameters_join**

Public Types

using **type** = detail::executor_parameters<*std*::decay_t<*Params*>...>

template<typename **Param**>

struct **executor_parameters_join**<*Param*>

Public Types

using **type** = *Param*

hpx/execution/executors/rebind_executor.hpp

Defined in header hpx/execution/executors/rebind_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

Typedefs

template<typename **ExPolicy**, typename **Executor**, typename **Parameters**>

using **rebind_executor_t** = typename *rebind_executor*<*ExPolicy*, *Executor*, *Parameters*>::type

Variables

constexpr struct *hpx*::*execution*::*experimental*::*create_rebound_policy_t* **create_rebound_policy**

template<typename **ExPolicy**, typename **Executor**, typename **Parameters**>

struct **rebind_executor**

#include <rebind_executor.hpp> Rebind the type of executor used by an execution policy. The execution category of Executor shall not be weaker than that of ExecutionPolicy.

Public Types

using **type** = typename policy_type::template rebind<executor_type, parameters_type>::type

The type of the rebound execution policy.

struct **create_rebound_policy_t**

Public Functions

template<typename **ExPolicy**, typename **Executor**, typename **Parameters**>

inline constexpr decltype(auto) **operator()**(*ExPolicy*&&, *Executor* &&exec, *Parameters* &&parameters) const

template<typename **ExPolicy**, typename **Executor**>

inline constexpr decltype(auto) **operator()**(*ExPolicy* &&policy, *Executor* &&exec) const template<typename **ExPolicy**, typename **Parameters**>

inline constexpr decltype(auto) **operator()**(*ExPolicy* &&policy, *Parameters* &&parameters) const

hpx::execution::experimental::max_num_chunks

Defined in header hpx/execution.hpp[869].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

struct **max_num_chunks**

Loop iterations are divided into not more than *num_chunks* partitions that are assigned to threads. If *num_chunks* is not specified, the number of chunks is determined based on the number of available cores.

hpx::execution::experimental::persistent_auto_chunk_size

Defined in header hpx/execution.hpp[870].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

struct **persistent_auto_chunk_size**

Loop iterations are divided into pieces and then assigned to threads. The number of loop iterations combined is determined based on measurements of how long the execution of 1% of the overall number of iterations takes. This executor parameters type makes sure that as many loop iterations are combined as necessary to run for the amount of time specified.

hpx/execution/executors/default_parameters.hpp

Defined in header hpx/execution/executors/default_parameters.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

[869] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

[870] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

struct **default_parameters**

#include <default_parameters.hpp> Loop iterations are divided into pieces of size *chunk_size* and then assigned to threads. If *chunk_size* is not specified, the iterations are evenly (if possible) divided contiguously among the threads.

Note

This executor parameters type is equivalent to OpenMP's STATIC scheduling directive.

Public Functions

default_parameters() = default

Construct a default_parameters executor parameters object

Note

By default, the number of loop iterations is determined from the number of available cores and the overall number of loop iterations to schedule.

hpx/execution/executors/polymorphic_executor.hpp

Defined in header hpx/execution/executors/polymorphic_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

namespace **parallel**

namespace **execution**

template<typename **R**, typename ...**Ts**>

class **polymorphic_executor**<*R*(*Ts*...)> : private *hpx*::*parallel*::*execution*::detail::polymorphic_executor_base

Public Types

template<typename>

using **future_type** = *hpx*::*future*<*R*>

Public Functions

inline constexpr **polymorphic_executor**() noexcept

inline **polymorphic_executor**(*polymorphic_executor* const &other)

inline **polymorphic_executor**(*polymorphic_executor* &&other) noexcept

inline *polymorphic_executor* &**operator=**(*polymorphic_executor* const &other)

inline *polymorphic_executor* &**operator=**(*polymorphic_executor* &&other) noexcept

template<typename **Exec**, typename **PE** = *std*::decay_t<*Exec*>, typename **Enable** = *std*::enable_if_t<!*std*::is_same_v<*PE*, *polymorphic_executor*>>>

inline **polymorphic_executor**(*Exec* &&exec)

template<typename **Exec**, typename **PE** = *std*::decay_t<*Exec*>, typename **Enable** = *std*::enable_if_t<!*std*::is_same_v<*PE*, *polymorphic_executor*>>>

inline *polymorphic_executor* &**operator=**(*Exec* &&exec)

inline void **reset**() noexcept

template<typename **F**>

inline void **post**(*F* &&f, *Ts*... ts) const

template<typename **F**>

inline *R* **sync_execute**(*F* &&f, *Ts*... ts) const

template<typename **F**>

inline *hpx*::*future*<*R*> **async_execute**(*F* &&f, *Ts*... ts) const

template<typename **F**, typename **Future**>

inline *hpx*::*future*<*R*> **then_execute**(*F* &&f, *Future* &&predecessor, *Ts*&&... ts) const

template<typename **F**, typename **Shape**>

inline *std*::vector<*R*> **bulk_sync_execute**(*F* &&f, *Shape* const &s, *Ts*&&... ts) const

template<typename **F**, typename **Shape**>

inline *hpx*::*future*<*std*::vector<*R*>> **bulk_async_execute**(*F* &&f, *Shape* const &s, *Ts*&&... ts) const

template<typename **F**, typename **Shape**>

inline *hpx*::*future*<*std*::vector<*R*>> **bulk_then_execute**(*F* &&f, *Shape* const &s, *hpx*::*shared_future*<void> const &predecessor, *Ts*&&... ts) const

Private Types

using **base_type** = detail::polymorphic_executor_base

using **vtable** = detail::polymorphic_executor_vtable<*R*(*Ts*...)>

Private Functions

inline void **assign**(*std*::nullptr_t) noexcept

template<typename **Exec**>

inline void **assign**(*Exec* &&exec)

Private Static Functions

static inline constexpr *vtable* const ***get_empty_vtable**() noexcept

template<typename **T**>

static inline constexpr *vtable* const ***get_vtable**() noexcept

hpx/execution/executors/execution_parameters_fwd.hpp

Defined in header hpx/execution/executors/execution_parameters_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

Variables

constexpr struct *hpx*::*execution*::*experimental*::*null_parameters_t* **null_parameters**

constexpr struct *hpx*::*execution*::*experimental*::*get_chunk_size_t* **get_chunk_size** constexpr struct *hpx*::*execution*::*experimental*::*measure_iteration_t* `measure_iteration`

constexpr struct *hpx*::*execution*::*experimental*::*maximal_number_of_chunks_t*
`maximal_number_of_chunks`

constexpr struct *hpx*::*execution*::*experimental*::*reset_thread_distribution_t*
`reset_thread_distribution`

constexpr struct *hpx*::*execution*::*experimental*::*processing_units_count_t*
`processing_units_count`

constexpr struct *hpx*::*execution*::*experimental*::*with_processing_units_count_t*
`with_processing_units_count`

constexpr struct *hpx*::*execution*::*experimental*::*mark_begin_execution_t* `mark_begin_execution`

constexpr struct *hpx*::*execution*::*experimental*::*mark_end_of_scheduling_t*
`mark_end_of_scheduling`

constexpr struct *hpx*::*execution*::*experimental*::*mark_end_execution_t* `mark_end_execution`

constexpr struct *hpx*::*execution*::*experimental*::*mark_partition_t* `mark_partition`

constexpr struct *hpx*::*execution*::*experimental*::*collect_execution_parameters_t*
`collect_execution_parameters`

struct `null_parameters_t`

struct `get_chunk_size_t`

#include <execution_parameters_fwd.hpp> Return the number of invocations of the given function *f* which should be combined into a single task

Param params

[in] The executor parameters object to use for determining the chunk size for the given number of tasks *num_tasks*.

Param exec
[in] The executor object which will be used for scheduling of the loop iterations.
Param iteration_duration
[in] The time one of the tasks require to be executed.
Param cores
[in] The number of cores the number of chunks should be determined for.
Param num_tasks
[in] The number of tasks the chunk size should be determined for
Return
The size of the chunks (number of iterations per chunk) that should be used for parallel execution.

Public Functions

template<typename **Parameters**, typename **Executor**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec, *hpx*::chrono::steady_duration const &iteration_duration, *std*::size_t cores, *std*::size_t num_tasks) const

template<typename **Parameters**, typename **Executor**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec, *std*::size_t cores, *std*::size_t num_tasks) const

struct **measure_iteration_t**

#include <execution_parameters_fwd.hpp> Return the measured execution time for one iteration based on running the given function.

> **Note**
>
> The parameter *f* is expected to be a nullary function returning a `std::size_t` representing the number of iteration the function has already executed (i.e. which don't have to be scheduled anymore).

Param params
[in] The executor parameters object to use for determining the chunk size for the given number of tasks *num_tasks*.
Param exec
[in] The executor object which will be used for scheduling of the loop iterations.
Param f
[in] The function which will be optionally scheduled using the given executor.
Param num_tasks
[in] The number of tasks the chunk size should be determined for
Return
The execution time for one of the tasks.

Public Functions

template<typename **Parameters**, typename **Executor**, typename **F**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec, *F* &&f, *std*::size_t num_tasks) const

struct **maximal_number_of_chunks_t**

#include <execution_parameters_fwd.hpp> Return the largest reasonable number of chunks to create for a single algorithm invocation.

Param params
[in] The executor parameters object to use for determining the number of chunks for the given number of *cores*.

Param exec
[in] The executor object which will be used for scheduling of the loop iterations.

Param cores
[in] The number of cores the number of chunks should be determined for.

Param num_tasks
[in] The number of tasks the chunk size should be determined for

Public Functions

template<typename **Parameters**, typename **Executor**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec, *std*::size_t cores, *std*::size_t num_tasks) const

struct **reset_thread_distribution_t**

#include <execution_parameters_fwd.hpp> Reset the internal round robin thread distribution scheme for the given executor.

> **Note**
>
> This calls params.reset_thread_distribution(exec) if it exists; otherwise it does nothing.

Param params
[in] The executor parameters object to use for resetting the thread distribution scheme.

Param exec
[in] The executor object to use.

Public Functions

template<typename **Parameters**, typename **Executor**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec) const

struct **processing_units_count_t**

#include <execution_parameters_fwd.hpp> Retrieve the number of (kernel-)threads used by the associated executor.

> **Note**
>
> This calls params.processing_units_count(Executor&&) if it exists; otherwise it forwards the request to the executor parameters object.

Param params
[in] The executor parameters object to use as a fallback if the executor does not expose

Param exec
[in] The executor object which will be used for scheduling of the loop iterations.

Param iteration_duration
[in] The time one of the tasks require to be executed.

Param num_tasks
[in] The number of tasks the number of cores should be determined for

Return
The number of cores to use

Public Functions

template<typename **Parameters**, typename **Executor**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec, *hpx*::chrono::steady_duration const &iteration_duration, *std*::size_t num_tasks) const

template<typename **Parameters**, typename **Executor**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec, *std*::size_t num_tasks) const

template<typename **Executor**>

inline decltype(auto) **operator()**(*Executor* &&exec, *hpx*::chrono::steady_duration const &iteration_duration, *std*::size_t num_tasks) const

template<typename **Executor**>

inline decltype(auto) **operator()**(*Executor* &&exec, *std*::size_t num_tasks) const

template<typename **Parameters**, typename **Executor**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec) const

template<typename **Executor**>

inline decltype(auto) **operator()**(*Executor* &&exec) const

template<typename **Parameters**, typename **Executor**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec, *hpx*::chrono::steady_duration const &iteration_duration, *std*::size_t num_tasks) const

template<typename **Parameters**, typename **Executor**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec, *std*::size_t num_tasks = 0) const

template<typename **Executor**>

inline decltype(auto) **operator()**(*Executor* &&exec, *hpx*::chrono::steady_duration const &iteration_duration, *std*::size_t num_tasks) const

template<typename **Executor**>

inline decltype(auto) **operator()**(*Executor* &&exec, *std*::size_t num_tasks = 0) const

template<typename **ExPolicy**>

inline decltype(auto) **operator()**(*ExPolicy* &&policy) const

struct **with_processing_units_count_t**

#include <execution_parameters_fwd.hpp> Generate a policy that supports setting the number of cores for execution.

Public Functions

template<typename **Executor**>

inline decltype(auto) **operator()**(*Executor* &&exec, *std*::size_t num_cores) const

template<typename **Policy**, typename **Property**>

inline decltype(auto) **operator()**(*Policy* &&policy, *Property* &&property) const

struct **mark_begin_execution_t**

#include <execution_parameters_fwd.hpp> Mark the begin of a parallel algorithm execution

> **Note**
>
> This calls params.mark_begin_execution(exec) if it exists; otherwise it does nothing.

Param params
[in] The executor parameters object to use as a fallback if the executor does not expose

Param exec
[in] The executor object which will be used for scheduling of the loop iterations.

Public Functions

template<typename **Parameters**, typename **Executor**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec) const

struct **mark_end_of_scheduling_t**

#include <execution_parameters_fwd.hpp> Mark the end of scheduling tasks during parallel algorithm execution

> **Note**
>
> This calls params.mark_begin_execution(exec) if it exists; otherwise it does nothing.

Param params
[in] The executor parameters object to use as a fallback if the executor does not expose

Param exec
[in] The executor object which will be used for scheduling of the loop iterations.

Public Functions

template<typename **Parameters**, typename **Executor**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec) const

struct **mark_end_execution_t**

#include <execution_parameters_fwd.hpp> Mark the end of a parallel algorithm execution

> **Note**
>
> This calls params.mark_end_execution(exec) if it exists; otherwise it does nothing.

Param params
[in] The executor parameters object to use as a fallback if the executor does not expose

Param exec
[in] The executor object which will be used for scheduling of the loop iterations.

Public Functions

template<typename **Parameters**, typename **Executor**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec) const

struct **mark_partition_t**

#include <execution_parameters_fwd.hpp> Mark custom point in parallel algorithm execution

> **Note**
>
> This calls params.mark_partition(exec, partition, args…) if it exists; otherwise it does nothing.

Param params
[in] The executor parameters object to use as a fallback if the executor does not expose

Param exec
[in] The executor object which will be used for scheduling of the loop iterations.

Public Functions

template<typename **Parameters**, typename **Executor**, typename ...**Args**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec, *std*::size_t partition, *Args*&&... args) const

struct **collect_execution_parameters_t**

#include <execution_parameters_fwd.hpp> Collect various parameters of the chunking for this parallel algorithm execution

> **Note**
>
> This calls params.mark_begin_execution(exec) if it exists; otherwise it does nothing.

Param params
[in] The executor parameters object to use as a fallback if the executor does not expose

Param exec
[in] The executor object which will be used for scheduling of the loop iterations.

Param num_elements
[in] The overall number of elements for the algorithm.

Param num_cores
[in] The overall number of cores to utilize for the algorithm.

Param num_chunks
[in] The overall number of chunks for the algorithm.

Param chunk_size
[in] The size of the chunks created for the algorithm.

Public Functions

template<typename **Parameters**, typename **Executor**>

inline decltype(auto) **operator()**(*Parameters* &&params, *Executor* &&exec, *std*::size_t num_elements, *std*::size_t num_cores, *std*::size_t num_chunks, *std*::size_t chunk_size) const

template<>

struct **is_scheduling_property**<*with_processing_units_count_t*> : public *std*::true_type

hpx::execution::experimental::num_cores

Defined in header hpx/execution.hpp[871].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

struct **num_cores**

Control number of cores in executors which need a functionality for setting the number of cores to be used by an algorithm directly

hpx/execution/executors/execution.hpp

Defined in header hpx/execution/executors/execution.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **execution**

hpx::execution::experimental::collect_chunking_parameters

Defined in header hpx/execution.hpp[872].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

struct **collect_chunking_parameters**

Collect various parameters used for running a parallel algorithm.

hpx::execution::experimental::dynamic_chunk_size

Defined in header hpx/execution.hpp[873].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

[871] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

[872] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

[873] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

struct **dynamic_chunk_size**

Loop iterations are divided into pieces of size *chunk_size* and then dynamically scheduled among the threads; when a thread finishes one chunk, it is dynamically assigned another If *chunk_size* is not specified, the default chunk size is 1.

> **Note**
>
> This executor parameters type is equivalent to OpenMP's DYNAMIC scheduling directive.

execution_base

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/execution_base/execution.hpp

Defined in header hpx/execution_base/execution.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

struct **sequenced_execution_tag**

#include <execution.hpp> Function invocations executed by a group of sequential execution agents execute in sequential order.

struct **parallel_execution_tag**

#include <execution.hpp> Function invocations executed by a group of parallel execution agents execute in unordered fashion. Any such invocations executing in the same thread are indeterminately sequenced with respect to each other.

> **Note**
>
> parallel_execution_tag is weaker than sequenced_execution_tag.

struct **unsequenced_execution_tag**

#include <execution.hpp> Function invocations executed by a group of vector execution agents are permitted to execute in unordered fashion when executed in different threads, and un-sequenced with respect to one another when executed in the same thread.

Note

unsequenced_execution_tag is weaker than parallel_execution_tag.

namespace **parallel**

namespace **execution**

Variables

constexpr struct *hpx*::*parallel*::*execution*::*sync_execute_t* **sync_execute**

constexpr struct *hpx*::*parallel*::*execution*::*async_execute_t* **async_execute**

constexpr struct *hpx*::*parallel*::*execution*::*then_execute_t* **then_execute**

constexpr struct *hpx*::*parallel*::*execution*::*post_t* **post**

constexpr struct *hpx*::*parallel*::*execution*::*bulk_sync_execute_t* **bulk_sync_execute**

constexpr struct *hpx*::*parallel*::*execution*::*bulk_async_execute_t* **bulk_async_execute**

constexpr struct *hpx*::*parallel*::*execution*::*bulk_then_execute_t* **bulk_then_execute**

constexpr struct *hpx*::*parallel*::*execution*::*async_invoke_t* **async_invoke**

constexpr struct *hpx*::*parallel*::*execution*::*sync_invoke_t* **sync_invoke**

struct **sync_execute_t**

#include <*execution.hpp*> Customization point for synchronous execution agent creation.

This synchronously creates a single function invocation f() using the associated executor. The execution of the supplied function synchronizes with the caller

> **Note**
>
> It will call exec.sync_execute(f, ts…) if the executor exposes a corresponding member function. For two-way executors it will invoke async_execute and wait for the task's completion before returning.

Param exec
[in] The executor object to use for scheduling of the function *f*.

Param f
[in] The function which will be scheduled using the given executor.

Param ts
[in] Additional arguments to use to invoke *f*.

Return
f(ts…)'s result

Public Functions

template<typename **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Ts*&&... ts) const

```
template<typename Executor, typename F, typename...
Ts>  HPX_DEPRECATED_V (2, 0,
"tag_invoke overloads for sync_execute are deprecated. " "Define a .
sync_execute() member function instead.
") decltype(auto) operator()(Executor &&exec
```

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Ts*&&... ts) const

Public Members

F &&**f**

```
F Ts &&ts const  {return hpx::functional::tag_invoke_ns::tag_invoke(sync_execute_t{},
std::forward<  Executor  >(  exec  ), std::forward<  F  >(  f  ),
std::forward<  Ts  >(  ts  )...)
```

struct **async_execute_t**

#include <execution.hpp> Customization point for asynchronous execution agent creation.

This asynchronously creates a single function invocation f() using the associated executor.

> **Note**
>
> Executors have to implement only `async_execute()`. All other functions will be emulated by this or other customization points in terms of this single basic primitive. However, some executors will naturally specialize all operations for maximum efficiency.

> **Note**
>
> This is valid for one way executors (calls make_ready_future(exec.sync_execute(f, ts…) if it exists) and for two-way executors (calls exec.async_execute(f, ts…) if it exists).

Param exec
[in] The executor object to use for scheduling of the function *f*.

Param f
[in] The function which will be scheduled using the given executor.

Param ts
[in] Additional arguments to use to invoke *f*.

Return
f(ts…)'s result through a future

Public Functions

template<typename **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Ts*&&... ts) const

```
template<typename Executor, typename F, typename...
Ts>  HPX_DEPRECATED_V (2, 0,
"tag_invoke overloads for async_execute are deprecated. " "Define an .
async_execute() member function instead.
") decltype(auto) operator()(Executor &&exec
```

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Ts*&&... ts) const

Public Members

F &&**f**

F Ts &&ts const {return hpx::functional::tag_invoke_ns::tag_invoke(async_execute_t{}, std::forward< Executor >(exec), std::forward< F >(f), std::forward< Ts >(ts)...)

struct **then_execute_t**

#include <execution.hpp> Customization point for execution agent creation depending on a given future.

This creates a single function invocation f() using the associated executor after the given future object has become ready.

> **Note**
>
> This is valid for two-way executors (calls exec.then_execute(f, predecessor, ts...) if it exists) and for one way executors (calls predecessor.then(bind(f, ts...))).

Param exec
[in] The executor object to use for scheduling of the function *f*.

Param f
[in] The function which will be scheduled using the given executor.

Param predecessor
[in] The future object the execution of the given function depends on.

Param ts
[in] Additional arguments to use to invoke *f*.

Return
f(ts...)'s result through a future

Public Functions

template<typename **Executor**, typename **F**, typename **Future**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Future* &&predecessor, *Ts*&&... ts) const

template<typename Executor, typename F, typename Future, typename... Ts> HPX_DEPRECATED_V (2, 0, "tag_invoke overloads for then_execute are deprecated. " "Define a . then_execute() member function instead. ") decltype(auto) operator()(Executor &&exec

template<executor_any **Executor**, typename **F**, typename **Future**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Future* &&predecessor, *Ts*&&... ts) const

Public Members

F &&**f**

F Future && predecessor

```
F Future Ts &&ts const {return hpx::functional::tag_invoke_ns::tag_invoke(then_execute_t{},
std::forward< Executor >( exec ), std::forward< F >( f ),
std::forward< Future >( predecessor ), std::forward< Ts >( ts )..
.)
```

struct **post_t**

#include <execution.hpp> Customization point for asynchronous fire & forget execution agent creation.

This asynchronously (fire & forget) creates a single function invocation f() using the associated executor.

> **Note**
>
> This is valid for two-way executors (calls exec.post(f, ts. . .), if available, otherwise it calls exec.async_execute(f, ts. . .) while discarding the returned future), and for non-blocking two-way executors (calls exec.post(f, ts. . .) if it exists).

Param exec
[in] The executor object to use for scheduling of the function *f*.

Param f
[in] The function which will be scheduled using the given executor.

Param ts
[in] Additional arguments to use to invoke *f*.

Public Functions

template<typename **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Ts*&&... ts) const

```
template<typename Executor, typename F, typename...
Ts> HPX_DEPRECATED_V (2, 0,
"tag_invoke overloads for post are deprecated. " "Define a .
post() member function instead.
") decltype(auto) operator()(Executor &&exec
```

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Ts*&&... ts) const

Public Members

F &&**f**

```
F Ts &&ts const {return hpx::functional::tag_invoke_ns::tag_invoke(post_t{},
std::forward< Executor >( exec ), std::forward< F >( f ),
std::forward< Ts >( ts )...)
```

struct **bulk_sync_execute_t**

#include <execution.hpp> Bulk form of synchronous execution agent creation.

This synchronously creates a group of function invocations f(i) whose ordering is given by the execution_category associated with the executor. The function synchronizes the execution of all scheduled functions with the caller.

Here *i* takes on all values in the index space implied by shape. All exceptions thrown by invocations of f(i) are reported in a manner consistent with parallel algorithm execution through the returned future.

> **Note**
>
> This is deliberately different from the bulk_sync_execute customization points specified in P0443.The bulk_sync_execute customization point defined here is more generic and is used as the workhorse for implementing the specified APIs.

> **Note**
>
> This calls exec.bulk_sync_execute(f, shape, ts...) if it exists; otherwise it executes sync_execute(f, shape, ts...) as often as needed.

Param exec
[in] The executor object to use for scheduling of the function *f*.

Param f
[in] The function which will be scheduled using the given executor.

Param shape
[in] The shape objects which defines the iteration boundaries for the arguments to be passed to *f*.

Param ts
[in] Additional arguments to use to invoke *f*.

Return
The return type of *executor_type::bulk_sync_execute* if defined by *executor_type*. Otherwise, a vector holding the returned values of each invocation of *f* except when *f* returns void, which case void is returned.

Public Functions

template<typename **Executor**, typename **F**, typename **Shape**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Shape* const &shape, *Ts*&&... ts) const

```
template<typename Executor, typename F, typename Shape, typename...
Ts>  HPX_DEPRECATED_V (2, 0,
"tag_invoke overloads for bulk_sync_execute are deprecated. " "Define a .
bulk_sync_execute() member function instead.
") decltype(auto) operator()(Executor &&exec
```

template<executor_any **Executor**, typename **F**, typename **Shape**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Shape* const &shape, *Ts*&&... ts) const

template<executor_any **Executor**, typename **F**, typename **Shape**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Shape* const &shape, *Ts*&&... ts) const

Public Members

F &&**f**

```
F Shape const  & shape
```

```
F Shape const Ts &&ts const  {return hpx::functional::tag_invoke_ns::tag_invoke(bulk_sync_execute_t{},
std::forward<  Executor  >(  exec  ),std::forward<  F  >(  f  ), shape,
std::forward<  Ts  >(  ts  )...)
```

struct **bulk_async_execute_t**

#include <execution.hpp> Bulk form of asynchronous execution agent creation.

This asynchronously creates a group of function invocations f(i) whose ordering is given by the execution_category associated with the executor.

Here *i* takes on all values in the index space implied by shape. All exceptions thrown by invocations of f(i) are reported in a manner consistent with parallel algorithm execution through the returned future.

> **Note**
>
> This is deliberately different from the bulk_async_execute customization points specified in P0443.The bulk_async_execute customization point defined here is more generic and is used as the workhorse for implementing the specified APIs.

> **Note**
>
> This calls exec.bulk_async_execute(f, shape, ts…) if it exists; otherwise it executes async_execute(f, shape, ts…) as often as needed.

Param exec
[in] The executor object to use for scheduling of the function *f*.

Param f
[in] The function which will be scheduled using the given executor.

Param shape
[in] The shape objects which defines the iteration boundaries for the arguments to be passed to *f*.

Param ts
[in] Additional arguments to use to invoke *f*.

Return
The return type of *executor_type::bulk_async_execute* if defined by *executor_type*. Otherwise, a vector of futures holding the returned values of each invocation of *f*.

Public Functions

template<typename **Executor**, typename **F**, typename **Shape**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Shape* const &shape, *Ts*&&... ts) const

template<typename **Executor**, typename **F**, typename **Shape**, typename ...**Ts**>

inline auto **operator()**(*Executor* &&exec, *F* &&f, *Shape* const &shape, *Ts*&&... ts) const -> decltype(*std*::forward<*Executor*>(*exec*).bulk_async_execute(*std*::forward<*F*>(*f*), *shape*, *std*::forward<*Ts*>(*ts*)...))

template<typename Executor, typename F, typename Shape, typename... Ts> HPX_DEPRECATED_V (2, 0, "tag_invoke overloads for bulk_async_execute are deprecated. " "Define a .bulk_async_execute() member function instead. ") decltype(auto) operator()(Executor &&exec

template<executor_any **Executor**, typename **F**, typename **Shape**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Shape* const &shape, *Ts*&&... ts) const

template<executor_any **Executor**, typename **F**, typename **Shape**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Shape* const &shape, *Ts*&&... ts) const

Public Members

F &&**f**

F Shape const & shape

F Shape const Ts &&ts const {return hpx::functional::tag_invoke_ns::tag_invoke(bulk_async_execute_t{}, std::forward< Executor >(exec),std::forward< F >(f), shape, std::forward< Ts >(ts)...)

struct **bulk_then_execute_t**

#include <execution.hpp> Bulk form of execution agent creation depending on a given future.

This creates a group of function invocations f(i) whose ordering is given by the execution_category associated with the executor.

Here *i* takes on all values in the index space implied by shape. All exceptions thrown by invocations of f(i) are reported in a manner consistent with parallel algorithm execution through the returned future.

Note

This is deliberately different from the then_sync_execute customization points specified in P0443.The bulk_then_execute customization point defined here is more generic and is used as the workhorse for implementing the specified APIs.

Note

This calls exec.bulk_then_execute(f, shape, pred, ts…) if it exists; otherwise it executes sync_execute(f, shape, pred.share(), ts…) (if this executor is also an OneWayExecutor), or async_execute(f, shape, pred.share(), ts…) (if this executor is also a TwoWayExecutor) - as often as needed.

Param exec
: [in] The executor object to use for scheduling of the function *f*.

Param f
: [in] The function which will be scheduled using the given executor.

Param shape
: [in] The shape objects which defines the iteration boundaries for the arguments to be passed to *f*.

Param predecessor
: [in] The future object the execution of the given function depends on.

Param ts
: [in] Additional arguments to use to invoke *f*.

Return
: The return type of *executor_type::bulk_then_execute* if defined by *executor_type*. Otherwise, a vector holding the returned values of each invocation of *f*.

Public Functions

template<typename **Executor**, typename **F**, typename **Shape**, typename **Future**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Shape* const &shape, *Future* &&predecessor, *Ts*&&... ts) const

```
template<typename Executor, typename F, typename Shape, typename Future,
typename... Ts>  HPX_DEPRECATED_V (2, 0,
"tag_invoke overloads for bulk_then_execute are deprecated. " "Define a .
bulk_then_execute() member function instead.
") decltype(auto) operator()(Executor &&exec
```

template<executor_any **Executor**, typename **F**, typename **Shape**, typename **Future**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Shape* const &shape, *Future* &&predecessor, *Ts*&&... ts) const template<executor_any **Executor**, typename **F**, typename **Shape**, typename **Future**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Shape* const &shape, *Future* &&predecessor, *Ts*&&... ts) const

Public Members

F &&**f**

```
F Shape const  & shape
```

```
F Shape const Future && predecessor
```

```
F Shape const Future Ts &&ts const  {return hpx::functional::tag_invoke_ns::tag_invoke(bulk_then_execute_
std::forward<  Executor  >(  exec  ),std::forward<  F  >(  f  ), shape,
std::forward<  Future  >(  predecessor  ),std::forward<  Ts  >(  ts  )...
)
```

struct **async_invoke_t**

#include <execution.hpp> Asynchronously invoke the given set of nullary functions, each on its own execution agent

This creates a group of function invocations whose ordering is given by the execution_category associated with the executor.

All exceptions thrown by invocations of the functions are reported in a manner consistent with parallel algorithm execution through the returned future.

> **Note**
>
> This calls exec.async_invoke(fs…) if it exists; otherwise it executes async_execute(fs) for each fs.

Param exec
[in] The executor object to use for scheduling of the functions *fs*.

Param fs
[in] The functions which will be scheduled using the given executor.

Return
The return type of *executor_type::async_invoke* if defined by *executor_type*. Otherwise, a future<void> representing finishing the execution of all functions *fs*.

Public Functions

template<typename **Executor**, typename **F**, typename ...**Fs**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Fs*&&... fs) const

```
template<typename Executor, typename F, typename...
Fs> HPX_DEPRECATED_V (2, 0,
"tag_invoke overloads for async_invoke are deprecated. " "Define an .
async_invoke() member function instead.
") decltype(auto) operator()(Executor &&exec
```

template<executor_any **Executor**, typename **F**, typename ...**Fs**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Fs*&&... fs) const

Public Members

F &&**f**

```
F Fs &&fs const {return hpx::functional::tag_invoke_ns::tag_invoke(async_invoke_t{},
std::forward< Executor >( exec ), std::forward< F >( f ),
std::forward< Fs >( fs )...)
```

struct **sync_invoke_t**

#include <execution.hpp> Synchronously invoke the given set of nullary functions, each on its own execution agent

This creates a group of function invocations whose ordering is given by the execution_category associated with the executor.

All exceptions thrown by invocations of the functions are reported in a manner consistent with parallel algorithm execution through the returned future.

> **Note**
>
> This calls exec.sync_invoke(fs. . .) if it exists; otherwise it executes sync_execute(fs) for each fs.

Param exec
[in] The executor object to use for scheduling of the functions *fs*.

Param fs
[in] The functions which will be scheduled using the given executor.

Return

The return type of *executor_type::async_invoke* if defined by *executor_type*.

Public Functions

template<typename **Executor**, typename **F**, typename ...**Fs**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Fs*&&... fs) const

```
template<typename Executor, typename F, typename...
Fs>  HPX_DEPRECATED_V (2, 0,
"tag_invoke overloads for sync_invoke are deprecated. " "Define a .
sync_invoke() member function instead.
") decltype(auto) operator()(Executor &&exec
```

template<executor_any **Executor**, typename **F**, typename ...**Fs**>

inline decltype(auto) **operator()**(*Executor* &&exec, *F* &&f, *Fs*&&... fs) const

Public Members

F &&**f**

```
F Fs &&fs const  {return hpx::functional::tag_invoke_ns::tag_invoke(sync_invoke_t{},
std::forward<  Executor  >(  exec  ), std::forward<  F  >(  f  ),
std::forward<  Fs  >(  fs  )...)
```

hpx/execution_base/traits/is_executor_parameters.hpp

Defined in header hpx/execution_base/traits/is_executor_parameters.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Variables

template<typename Parameters> concept **executor_parameters** =hpx::traits::is_executor_parameters_v<Parameters

namespace **execution**

namespace **experimental**

Typedefs

template<typename **Executor**>

using **extract_executor_parameters_t** = typename *extract_executor_parameters*<*Executor*>::type

Variables

template<typename **Parameters**>

constexpr bool **extract_has_variable_chunk_size_v** = *extract_has_variable_chunk_size*<*Parameters*>::value

template<typename **Parameters**>

constexpr bool **extract_invokes_testing_function_v** = *extract_invokes_testing_function*<*Parameters*>::value

template<typename **T**>

constexpr bool **is_executor_parameters_v** = *is_executor_parameters*<*T*>::value

struct **sequential_executor_parameters**

template<typename **Executor**, typename **Enable** = void>

struct **extract_executor_parameters**

Public Types

using **type** = *sequential_executor_parameters*

template<typename **Executor**>

struct **extract_executor_parameters**<*Executor*, *std*::void_t<typename *Executor*::executor_parameters_type>>

Public Types

using **type** = typename *Executor*::executor_parameters_type

template<typename **Parameters**, typename **Enable** = void>

struct **extract_has_variable_chunk_size** : public *std*::false_type

Subclassed by *hpx::execution::experimental::extract_has_variable_chunk_size<::std::reference_wrapper< Parameters > >*

template<typename **Parameters**>

struct **extract_has_variable_chunk_size**<*Parameters*, *std*::void_t<typename *Parameters*::has_variable_chunk_size>> : public *std*::true_type

template<typename **Parameters**>

struct **extract_has_variable_chunk_size**<::*std*::reference_wrapper<*Parameters*>> : public *hpx*::*execution*::*experimental*::*extract_has_variable_chunk_size*<*Parameters*>

template<typename **Parameters**, typename **Enable** = void>

struct **extract_invokes_testing_function** : public *std*::false_type

Subclassed by *hpx::execution::experimental::extract_invokes_testing_function< ::std::reference_wrapper< Parameters > >*

template<typename **Parameters**>

struct **extract_invokes_testing_function**<::*std*::reference_wrapper<*Parameters*>> : public *hpx*::*execution*::*experimental*::*extract_invokes_testing_function*<*Parameters*>

template<typename **T**>

struct **is_executor_parameters** : public detail::is_executor_parameters<*std*::decay_t<*T*>>

namespace **traits**

Variables

template<typename **T**>

constexpr bool **is_executor_parameters_v** = *is_executor_parameters*<*T*>::value

template<typename **Parameters**, typename **Enable**>

struct **is_executor_parameters** : public *hpx*::*execution*::*experimental*::*is_executor_parameters*<*std*::decay_t<*Parameters*>>

executors

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/executors/sequenced_executor.hpp

Defined in header hpx/executors/sequenced_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

struct **sequenced_executor**

#include <sequenced_executor.hpp> A *sequential_executor* creates groups of sequential execution agents which execute in the calling thread. The sequential order is given by the lexicographical order of indices in the index space.

Private Static Functions

template<typename **F**, typename ...**Ts**>

static inline decltype(auto) **sync_execute_impl**(*F* &&f, *Ts*&&... ts)

template<typename **F**, typename ...**Ts**>

static inline decltype(auto) **async_execute_impl**(*hpx*::*threads*::*thread_description* const &desc, *F* &&f, *Ts*&&... ts)

template<typename **Archive**>

static inline constexpr void **serialize**(*Archive*&, unsigned int const) noexcept

Friends

friend class hpx::serialization::access

namespace **experimental**

hpx/executors/restricted_thread_pool_executor.hpp

Defined in header hpx/executors/restricted_thread_pool_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

Typedefs

using **restricted_thread_pool_executor** = *restricted_policy_executor*<*hpx*::*launch*>

template<typename **Policy**>

class **restricted_policy_executor**

Public Types

using **execution_category** = typename *embedded_executor*::execution_category

Associate the parallel_execution_tag executor tag type as a default with this executor.

using **executor_parameters_type** = typename *embedded_executor*::executor_parameters_type

Public Functions

inline explicit **restricted_policy_executor**(*std*::size_t first_thread = 0, *std*::size_t num_threads = 1, *threads*::*thread_priority* priority = *threads*::*thread_priority*::*default_*, *threads*::*thread_stacksize* stacksize = *threads*::*thread_stacksize*::*default_*, *threads*::*thread_schedule_hint* schedulehint = {}, [[maybe_unused]] *std*::size_t hierarchical_threshold = *hierarchical_threshold_default_*)

Create a new parallel executor.

inline **restricted_policy_executor**(*restricted_policy_executor* const &other) noexcept(*std*::is_nothrow_copy_constructible_v<*embedded_executor*>)

inline **restricted_policy_executor**(*restricted_policy_executor* &&other) noexcept

inline *restricted_policy_executor* &**operator=**(*restricted_policy_executor* const &rhs) noexcept(*std*::is_nothrow_copy_assignable_v<*embedded_executor*>)

inline *restricted_policy_executor* &**operator=**(*restricted_policy_executor* &&rhs) noexcept

inline constexpr *hpx*::*execution*::*experimental*::*executor_scheduler*<*restricted_policy_executor*> **query**(*hpx*::*execution*::*experimental*::get
const
noex-
cept(*std*::is_nothrow_copy_const

Private Types

using **embedded_executor** = *hpx*::*execution*::*parallel_policy_executor*<*Policy*>

Private Members

std::uint16_t **first_thread_**

mutable *std*::atomic<*std*::size_t> **os_thread_**

embedded_executor **exec_**

Private Static Attributes

static constexpr *std*::size_t **hierarchical_threshold_default_** = 6

hpx/executors/explicit_scheduler_executor.hpp

Defined in header hpx/executors/explicit_scheduler_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

Functions

template<typename **BaseScheduler**>

explicit **explicit_scheduler_executor**(*BaseScheduler* &&sched) -> *explicit_scheduler_executor*<*std*::decay_t<*BaseScheduler*>>

template<typename **BaseScheduler**>

struct **explicit_scheduler_executor**

hpx/executors/scheduler_executor.hpp

Defined in header hpx/executors/scheduler_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

Functions

template<typename **BaseScheduler**>

explicit **scheduler_executor**(*BaseScheduler* &&sched) -> *scheduler_executor*<*std*::decay_t<*BaseScheduler*>>

template<typename **BaseScheduler**>

struct **scheduler_executor**

hpx/executors/thread_pool_continues_on_sender.hpp

Defined in header hpx/executors/thread_pool_continues_on_sender.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Domain-optimized continues_on sender for HPX thread pool schedulers.

When stdexec::continues_on is applied with an HPX thread_pool_scheduler, the generic stdexec implementation creates two operation states: one for the source sender and one for the scheduler hop (schedule_from). This double-state overhead adds ~1 us per task in microbenchmarks.

This header provides a custom sender that replaces the generic path by:

1. Connecting the source sender to a lightweight receiver that captures the downstream receiver and the scheduler.
2. On set_value: if we are already on the same HPX thread pool, forwarding inline; otherwise, posting a single HPX task to forward the values.

   This eliminates the intermediate __storage_for_t, the second operation state, and the run_loop overhead.

namespace **hpx**

namespace **execution**

namespace **experimental**

hpx/executors/annotating_executor.hpp

Defined in header hpx/executors/annotating_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

template<executor_any **BaseExecutor**>

struct **annotating_executor**

#include <annotating_executor.hpp> An annotating_executor wraps any other executor and adds the capability to add annotations to the launched threads.

Public Functions

template<executor_any **Executor**>

inline explicit constexpr **annotating_executor**(*Executor* &&exec, char const *annotation = nullptr)

template<executor_any **Executor**>

inline explicit **annotating_executor**(*Executor* &&exec, *std*::string annotation)

hpx::execution::seq

Defined in header hpx/execution.hpp[874].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::execution::par*
- *hpx::execution::par_unseq*
- *hpx::execution::task*
- *hpx::execution::sequenced_policy*
- *hpx::execution::parallel_policy*
- *hpx::execution::parallel_unsequenced_policy*
- *hpx::execution::sequenced_task_policy*
- *hpx::execution::parallel_task_policy*

constexpr *sequenced_policy* `hpx::execution::seq` = {}

Default sequential execution policy object.

[874] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

hpx::execution::par

Defined in header hpx/execution.hpp[875].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::execution::seq*
- *hpx::execution::par_unseq*
- *hpx::execution::task*
- *hpx::execution::sequenced_policy*
- *hpx::execution::parallel_policy*
- *hpx::execution::parallel_unsequenced_policy*
- *hpx::execution::sequenced_task_policy*
- *hpx::execution::parallel_task_policy*

constexpr *parallel_policy* `hpx::execution::par` = {}

Default parallel execution policy object.

hpx::execution::par_unseq

Defined in header hpx/execution.hpp[876].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::execution::seq*
- *hpx::execution::par*
- *hpx::execution::task*
- *hpx::execution::sequenced_policy*
- *hpx::execution::parallel_policy*
- *hpx::execution::parallel_unsequenced_policy*
- *hpx::execution::sequenced_task_policy*
- *hpx::execution::parallel_task_policy*

constexpr *parallel_unsequenced_policy* `hpx::execution::par_unseq` = {}

Default vector execution policy object.

[875] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

[876] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

hpx::execution::task

Defined in header hpx/execution.hpp[877].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::execution::seq*
- *hpx::execution::par*
- *hpx::execution::par_unseq*
- *hpx::execution::sequenced_policy*
- *hpx::execution::parallel_policy*
- *hpx::execution::parallel_unsequenced_policy*
- *hpx::execution::sequenced_task_policy*
- *hpx::execution::parallel_task_policy*

constexpr task_policy_tag `hpx::execution::task` = { }

hpx::execution::sequenced_policy

Defined in header hpx/execution.hpp[878].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::execution::seq*
- *hpx::execution::par*
- *hpx::execution::par_unseq*
- *hpx::execution::task*
- *hpx::execution::parallel_policy*
- *hpx::execution::parallel_unsequenced_policy*
- *hpx::execution::sequenced_task_policy*
- *hpx::execution::parallel_task_policy*

using `hpx::execution::sequenced_policy` = detail::sequenced_policy_shim<*sequenced_executor*, *hpx*::*traits*::executor_parameters_type_t<*sequenced_executor*>>

The class sequenced_policy is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and require that a parallel algorithm's execution may not be parallelized.

[877] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

[878] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

hpx::execution::parallel_policy

Defined in header hpx/execution.hpp[879].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::execution::seq*
- *hpx::execution::par*
- *hpx::execution::par_unseq*
- *hpx::execution::task*
- *hpx::execution::sequenced_policy*
- *hpx::execution::parallel_unsequenced_policy*
- *hpx::execution::sequenced_task_policy*
- *hpx::execution::parallel_task_policy*

using `hpx::execution::parallel_policy` = detail::parallel_policy_shim<*parallel_executor*, *hpx*::*traits*::executor_parameters_type_t<*parallel_executor*>>

The class parallel_policy is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may be parallelized.

hpx::execution::parallel_unsequenced_policy

Defined in header hpx/execution.hpp[880].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::execution::seq*
- *hpx::execution::par*
- *hpx::execution::par_unseq*
- *hpx::execution::task*
- *hpx::execution::sequenced_policy*
- *hpx::execution::parallel_policy*
- *hpx::execution::sequenced_task_policy*
- *hpx::execution::parallel_task_policy*

using `hpx::execution::parallel_unsequenced_policy` = detail::parallel_unsequenced_policy_shim<*parallel_executor*, *hpx*::*traits*::executor_parameters_type_t<*parallel_executor*>>

[879] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

[880] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

The class parallel_unsequenced_policy is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may be parallelized and vectorized.

hpx::execution::sequenced_task_policy

Defined in header hpx/execution.hpp[881].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::execution::seq*
- *hpx::execution::par*
- *hpx::execution::par_unseq*
- *hpx::execution::task*
- *hpx::execution::sequenced_policy*
- *hpx::execution::parallel_policy*
- *hpx::execution::parallel_unsequenced_policy*
- *hpx::execution::parallel_task_policy*

using *hpx*::*execution*::**sequenced_task_policy** = detail::sequenced_task_policy_shim<*sequenced_executor*, *hpx*::*traits*::executor_parameters_type_t<*sequenced_executor*>>

Extension: The class sequenced_task_policy is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may not be parallelized (has to run sequentially).

The algorithm returns a future representing the result of the corresponding algorithm when invoked with the sequenced_policy.

hpx::execution::parallel_task_policy

Defined in header hpx/execution.hpp[882].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::execution::seq*
- *hpx::execution::par*
- *hpx::execution::par_unseq*
- *hpx::execution::task*
- *hpx::execution::sequenced_policy*
- *hpx::execution::parallel_policy*

[881] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

[882] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/execution.hpp

- *hpx::execution::parallel_unsequenced_policy*
- *hpx::execution::sequenced_task_policy*

using *hpx*::*execution*::**parallel_task_policy** = detail::parallel_task_policy_shim<*parallel_executor*, *hpx*::*traits*::executor_parameters_type_t<*parallel_executor*>>

Extension: The class parallel_task_policy is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may be parallelized.

The algorithm returns a future representing the result of the corresponding algorithm when invoked with the parallel_policy.

hpx/executors/execution_policy_parameters.hpp

Defined in header hpx/executors/execution_policy_parameters.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/executors/execution_policy_mappings.hpp

Defined in header hpx/executors/execution_policy_mappings.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

Variables

template<typename **Tag**>

constexpr bool **is_execution_policy_mapping_v** = *is_execution_policy_mapping*<*Tag*>::value

constexpr struct *hpx*::*execution*::*experimental*::*to_non_par_t* **to_non_par**

constexpr struct *hpx*::*execution*::*experimental*::*to_par_t* **to_par** constexpr struct *hpx*::*execution*::*experimental*::*to_non_task_t* **to_non_task**

constexpr struct *hpx*::*execution*::*experimental*::*to_task_t* **to_task**

constexpr struct *hpx*::*execution*::*experimental*::*to_non_unseq_t* **to_non_unseq**

constexpr struct *hpx*::*execution*::*experimental*::*to_unseq_t* **to_unseq**

template<typename **Tag**>

struct **is_execution_policy_mapping** : public *std*::false_type

struct **to_non_par_t**

Public Functions

inline constexpr decltype(auto) **operator()** (Target const &target) const

template<execution_policy **ExPolicy**>

inline constexpr decltype(auto) **operator()** (*ExPolicy* &&policy) const noexcept

Public Members

template<typename Target> requires = { t.to_non_par()

template<>

struct **is_execution_policy_mapping**<*to_non_par_t*> : public *std*::true_type

struct **to_par_t**

Public Functions

inline constexpr decltype(auto) **operator()**(Target const &target) const

template<execution_policy **ExPolicy**>

inline constexpr decltype(auto) **operator()**(*ExPolicy* &&policy) const noexcept

Public Members

template<typename Target> requires = { t.to_par()

template<>

struct **is_execution_policy_mapping**<*to_par_t*> : public *std*::true_type

struct **to_non_task_t**

Subclassed by hpx::execution::non_task_policy_tag

Public Functions

inline constexpr decltype(auto) **operator()**(Target const &target) const

template<execution_policy **ExPolicy**>

inline constexpr decltype(auto) **operator()**(*ExPolicy* &&policy) const noexcept

Public Members

template<typename Target> requires = { t.to_non_task()

template<>

struct **is_execution_policy_mapping**<*to_non_task_t*> : public *std*::true_type struct **to_task_t**

Subclassed by hpx::execution::task_policy_tag

Public Functions

inline constexpr decltype(auto) **operator()**(Target const &target) const

template<execution_policy **ExPolicy**>

inline constexpr decltype(auto) **operator()**(*ExPolicy* &&policy) const noexcept

Public Members

template<typename Target> requires = { t.to_task()

template<>

struct **is_execution_policy_mapping**<*to_task_t*> : public *std*::true_type

struct **to_non_unseq_t**

Public Functions

inline constexpr decltype(auto) **operator()**(Target const &target) const

template<execution_policy **ExPolicy**>

inline constexpr decltype(auto) **operator()**(*ExPolicy* &&policy) const noexcept

Public Members

`template<typename Target> requires = { t.to_non_unseq()`

template<>

struct **is_execution_policy_mapping**<*to_non_unseq_t*> : public *std*::true_type

struct **to_unseq_t**

Public Functions

inline constexpr decltype(auto) **operator()** (Target const &target) const

template<execution_policy **ExPolicy**>

inline constexpr decltype(auto) **operator()** (*ExPolicy* &&policy) const noexcept

Public Members

`template<typename Target> requires = { t.to_unseq()`

template<>

struct **is_execution_policy_mapping**<*to_unseq_t*> : public *std*::true_type

hpx/executors/exception_list.hpp

Defined in header hpx/executors/exception_list.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

hpx/executors/execution_policy_scheduling_property.hpp

Defined in header hpx/executors/execution_policy_scheduling_property.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

hpx/executors/fork_join_executor.hpp

Defined in header hpx/executors/fork_join_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/executors/parallel_executor.hpp

Defined in header hpx/executors/parallel_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

Typedefs

using **parallel_executor** = *parallel_policy_executor*<*hpx*::*launch*>

using **parallel_executor_spawn_hierarchically** = *parallel_policy_executor*<*hpx*::*launch*, true>

Functions

template<typename **Policy**>

constexpr *parallel_policy_executor*<*Policy*, true> **to_hierarchical_spawning**(*parallel_policy_executor*<*Policy*> &exec) noexcept

template<typename **Policy**>

constexpr *parallel_policy_executor*<*Policy*, true> **to_hierarchical_spawning**(*parallel_policy_executor*<*Policy*> &&exec) noexcept

template<typename **Policy**>

constexpr *parallel_policy_executor*<*Policy*, true> **to_hierarchical_spawning**(*parallel_policy_executor*<*Policy*> const &exec) noexcept

template<typename **Executor**>

constexpr *Executor* **to_hierarchical_spawning**(*Executor* &&exec) noexcept

template<typename **Policy**>

constexpr *parallel_policy_executor*<*Policy*> **to_non_hierarchical_spawning**(*parallel_policy_executor*<*Policy*, true> &exec) noexcept

template<typename **Policy**>

constexpr *parallel_policy_executor*<*Policy*> **to_non_hierarchical_spawning**(*parallel_policy_executor*<*Policy*, true> &&exec) noexcept

template<typename **Policy**>

constexpr *parallel_policy_executor*<*Policy*> **to_non_hierarchical_spawning**(*parallel_policy_executor*<*Policy*, true> const &exec) noexcept

template<typename **Executor**>

constexpr *Executor* **to_non_hierarchical_spawning**(*Executor* &&exec) noexcept template<typename **Policy**>

struct **parallel_policy_executor_base**

#include <parallel_executor.hpp> A *parallel_executor* creates groups of parallel execution agents which execute in threads implicitly created by the executor. This executor prefers continuing with the creating thread first before executing newly created threads.

This executor conforms to the concepts of a TwoWayExecutor, and a BulkTwoWayExecutor

Subclassed by *hpx::execution::parallel_policy_executor< Policy, HierarchicalSpawning >*, *hpx::execution::parallel_policy_executor< Policy, true >*

Public Types

using **execution_category** = *std*::conditional_t<*std*::is_same_v<*Policy*, *launch*::sync_policy>, *sequenced_execution_tag*, *parallel_execution_tag*>

Associate the parallel_execution_tag executor tag type as a default with this executor, except if the given launch policy is sync.

using **executor_parameters_type** = *experimental*::*default_parameters*

Associate the default_parameters executor parameters type as a default with this executor.

Public Functions

inline **parallel_policy_executor_base**(*parallel_policy_executor_base* const &rhs) noexcept

inline *parallel_policy_executor_base* &**operator=**(*parallel_policy_executor_base* const &rhs) noexcept

constexpr **~parallel_policy_executor_base**() = default

template<typename **Parameters**>

inline *std*::size_t **processing_units_count**(*Parameters*&&, *hpx*::chrono::steady_duration const& = *hpx*::chrono::null_duration, *std*::size_t = 0) const

Public Static Functions

template<typename **Self**>

static inline *Self* **with_num_cores**(*Self* self, *std*::size_t num_cores)

template<typename **Self**>

static inline *Self* **with_first_core**(*Self* self, *std*::size_t first_core) noexcept

Protected Functions

inline explicit constexpr **parallel_policy_executor_base**(*threads*::*thread_priority* priority, *threads*::*thread_stacksize* stacksize = *threads*::*thread_stacksize*::*default_*, *threads*::*thread_schedule_hint* schedulehint = { }, *Policy* l = *parallel*::*execution*::detail::get_default_policy<*Policy*>::call()) noexcept

Create a new parallel executor.

inline explicit constexpr **parallel_policy_executor_base**(*threads*::*thread_stacksize* stacksize, *threads*::*thread_schedule_hint* schedulehint = { }, *Policy* l = *parallel*::*execution*::detail::get_default_policy<*Policy*>::call()) noexcept

inline explicit constexpr **parallel_policy_executor_base**(*threads*::*thread_schedule_hint* schedulehint, *Policy* l = *parallel*::*execution*::detail::get_default_policy<*Policy*>::call()) noexcept

inline explicit constexpr **parallel_policy_executor_base**(*Policy* l) noexcept

inline constexpr **parallel_policy_executor_base**() noexcept

inline explicit constexpr **parallel_policy_executor_base**(*threads*::*thread_pool_base* *pool, *Policy* l) noexcept inline explicit constexpr **parallel_policy_executor_base**(*threads*::*thread_pool_base* *pool, *threads*::*thread_priority* priority = *threads*::*thread_priority*::*default_*, *threads*::*thread_stacksize* stacksize = *threads*::*thread_stacksize*::*default_*, *threads*::*thread_schedule_hint* schedulehint = {}, *Policy* l = *parallel*::*execution*::detail::get_default_policy<*Policy*>::call())

template<typename **Policy**, bool **HierarchicalSpawning** = false>

struct **parallel_policy_executor** : public *hpx*::*execution*::*parallel_policy_executor_base*<*Policy*>

Public Types

using **base_type** = *parallel_policy_executor_base*<*Policy*>

Public Functions

inline explicit constexpr **parallel_policy_executor**(*threads*::*thread_priority* priority, *threads*::*thread_stacksize* stacksize = *threads*::*thread_stacksize*::*default_*, *threads*::*thread_schedule_hint* schedulehint = {}, *Policy* l = *parallel*::*execution*::detail::get_default_policy<*Policy*>::call()) noexcept

inline explicit constexpr **parallel_policy_executor**(*threads*::*thread_stacksize* stacksize, *threads*::*thread_schedule_hint* schedulehint = {}, *Policy* l = *parallel*::*execution*::detail::get_default_policy<*Policy*>::call()) noexcept

inline explicit constexpr **parallel_policy_executor**(*threads*::*thread_schedule_hint* schedulehint, *Policy* l = *parallel*::*execution*::detail::get_default_policy<*Policy*>::call()) noexcept

inline explicit constexpr **parallel_policy_executor**(*Policy* l) noexcept constexpr **parallel_policy_executor**() noexcept = default

inline explicit constexpr **parallel_policy_executor**(*threads*::*thread_pool_base* *pool, *Policy* l) noexcept

inline explicit constexpr **parallel_policy_executor**(*threads*::*thread_pool_base* *pool, *threads*::*thread_priority* priority = *threads*::*thread_priority*::*default_*, *threads*::*thread_stacksize* stacksize = *threads*::*thread_stacksize*::*default_*, *threads*::*thread_schedule_hint* schedulehint = {}, *Policy* l = *parallel*::*execution*::detail::get_default_policy<*Policy*>::call()) noexcept

parallel_policy_executor(*parallel_policy_executor* const&) = default

parallel_policy_executor(*parallel_policy_executor*&&) = default

parallel_policy_executor &**operator=**(*parallel_policy_executor* const&) = default

parallel_policy_executor &**operator=**(*parallel_policy_executor*&&) = default

inline auto **query**(*experimental*::*with_processing_units_count_t*, *std*::size_t num_cores) const

template<typename **Parameters**>

inline *std*::size_t **query**(*experimental*::*processing_units_count_t*, *Parameters* &&params, *hpx*::chrono::steady_duration const &iter_dur = *hpx*::chrono::null_duration, *std*::size_t num_tasks = 0) const

inline auto **query**(*experimental*::with_first_core_t, *std*::size_t first_core) const noexcept

inline constexpr *std*::size_t **query**(*experimental*::get_first_core_t) const noexcept inline auto **query**(*experimental*::get_processing_units_mask_t) const

inline auto **query**(*experimental*::get_cores_mask_t) const

template<typename **Tag**, typename **Property**>

inline auto **query**(*Tag* tag, *Property* &&prop) const

template<typename **Tag**>

inline auto **query**(*Tag* tag) const

template<typename **F**, typename ...**Ts**>

inline decltype(auto) **sync_execute**(*F* &&f, *Ts*&&... ts) const

template<typename **F**, typename ...**Ts**>

inline decltype(auto) **async_execute**(*F* &&f, *Ts*&&... ts) const

template<typename **F**, typename **Future**, typename ...**Ts**>

inline decltype(auto) **then_execute**(*F* &&f, *Future* &&predecessor, *Ts*&&... ts) const

template<typename **F**, typename ...**Ts**>

inline void **post**(*F* &&f, *Ts*&&... ts) const

template<typename **F**, typename **S**, typename **Future**, typename ...**Ts**>

inline decltype(auto) **bulk_then_execute**(*F* &&f, *S* const &shape, *Future* &&predecessor, *Ts*&&... ts) const

template<typename **F**, typename **S**, typename ...**Ts**>

inline decltype(auto) **bulk_sync_execute**(*F* &&f, *S* const &shape, *Ts*&&... ts) const template<typename **F**, typename **S**, typename ...**Ts**>

inline decltype(auto) **bulk_async_execute**(*F* &&f, *S* const &shape, *Ts*&&... ts) const

inline decltype(auto) **to_non_par**() const

template<typename **Policy**, bool **HierarchicalSpawning**>

constexpr *hpx*::*execution*::*experimental*::*executor_scheduler*<*parallel_policy_executor*<*Policy*, *HierarchicalSpawning*>> **query**(*hpx*::*execu* const noexcept

template<typename **Policy**>

struct **parallel_policy_executor**<*Policy*, true> : public *hpx*::*execution*::*parallel_policy_executor_base*<*Policy*>

Public Types

using **base_type** = *parallel_policy_executor_base*<*Policy*>

Public Functions

inline explicit constexpr **parallel_policy_executor**(*threads*::*thread_priority* priority, *threads*::*thread_stacksize* stacksize = *threads*::*thread_stacksize*::*default_*, *threads*::*thread_schedule_hint* schedulehint = {}, *Policy* l = *parallel*::*execution*::detail::get_default_policy<*Policy*>::call(), *std*::size_t const hierarchical_threshold = hierarchical_threshold_default_) noexcept

inline explicit constexpr **parallel_policy_executor**(*threads*::*thread_stacksize* stacksize, *threads*::*thread_schedule_hint* schedulehint = {}, *Policy* l = *parallel*::*execution*::detail::get_default_policy<*Policy*>::call()) noexcept

inline explicit constexpr **parallel_policy_executor**(*threads*::*thread_schedule_hint* schedulehint, *Policy* l = *parallel*::*execution*::detail::get_default_policy<*Policy*>::call()) noexcept inline explicit constexpr **parallel_policy_executor**(*Policy* l) noexcept

constexpr **parallel_policy_executor**() noexcept = default

inline explicit constexpr **parallel_policy_executor**(*threads*::*thread_pool_base* *pool, *Policy* l, *std*::size_t const hierarchical_threshold = hierarchical_threshold_default_) noexcept

inline explicit constexpr **parallel_policy_executor**(*threads*::*thread_pool_base* *pool, *threads*::*thread_priority* priority = *threads*::*thread_priority*::*default_*, *threads*::*thread_stacksize* stacksize = *threads*::*thread_stacksize*::*default_*, *threads*::*thread_schedule_hint* schedulehint = {}, *Policy* l = *parallel*::*execution*::detail::get_default_policy<*Policy*>::call(), *std*::size_t const hierarchical_threshold = hierarchical_threshold_default_) noexcept

inline constexpr void **set_hierarchical_threshold**(*std*::size_t const threshold) noexcept

parallel_policy_executor(*parallel_policy_executor* const&) = default

parallel_policy_executor(*parallel_policy_executor*&&) = default

parallel_policy_executor &**operator=**(*parallel_policy_executor* const&) = default

parallel_policy_executor &**operator=**(*parallel_policy_executor*&&) = default

~parallel_policy_executor() = default

inline auto **query**(*experimental*::*with_processing_units_count_t*, *std*::size_t num_cores) const template<typename **Parameters**>

inline *std*::size_t **query**(*experimental*::*processing_units_count_t*, *Parameters* &&params, *hpx*::chrono::steady_duration const &iter_dur = *hpx*::chrono::null_duration, *std*::size_t num_tasks = 0) const

inline auto **query**(*experimental*::with_first_core_t, *std*::size_t first_core) const noexcept

inline constexpr *std*::size_t **query**(*experimental*::get_first_core_t) const noexcept

inline auto **query**(*experimental*::get_processing_units_mask_t) const

inline auto **query**(*experimental*::get_cores_mask_t) const

template<typename **Tag**, typename **Property**>

inline auto **query**(*Tag* tag, *Property* &&prop) const

template<typename **Tag**>

inline auto **query**(*Tag* tag) const

template<typename **F**, typename ...**Ts**>

inline decltype(auto) **sync_execute**(*F* &&f, *Ts*&&... ts) const

template<typename **F**, typename ...**Ts**>

inline decltype(auto) **async_execute**(*F* &&f, *Ts*&&... ts) const

template<typename **F**, typename **Future**, typename ...**Ts**>

inline decltype(auto) **then_execute**(*F* &&f, *Future* &&predecessor, *Ts*&&... ts) const

template<typename **F**, typename ...**Ts**>

inline void **post**(*F* &&f, *Ts*&&... ts) const

template<typename **F**, typename **S**, typename **Future**, typename ...**Ts**>

inline decltype(auto) **bulk_then_execute**(*F* &&f, *S* const &shape, *Future* &&predecessor, *Ts*&&... ts) const

template<typename **F**, typename **S**, typename ...**Ts**>

inline decltype(auto) **bulk_sync_execute**(*F* &&f, *S* const &shape, *Ts*&&... ts) const

template<typename **F**, typename **S**, typename ...**Ts**>

inline decltype(auto) **bulk_async_execute**(*F* &&f, *S* const &shape, *Ts*&&... ts) const

inline decltype(auto) **to_non_par**() const

namespace **experimental**

namespace **parallel**

namespace **execution**

hpx/executors/std_execution_policy.hpp

Defined in header hpx/executors/std_execution_policy.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/executors/execution_policy_annotation.hpp

Defined in header hpx/executors/execution_policy_annotation.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

hpx/executors/current_executor.hpp

Defined in header hpx/executors/current_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **execution**

Typedefs

typedef *hpx*::*execution*::*parallel_executor* **instead**

namespace **this_thread**

Functions

hpx::*execution*::*parallel_executor* **get_executor**(*error_code* &ec = *throws*)

Returns a reference to the executor that was used to create the current thread.

Throws

`If – &ec != &throws`, never throws, but will set *ec* to an appropriate value when an error occurs. Otherwise, this function will throw an *hpx::exception* with an error code of *hpx::error::yield_aborted* if it is signaled with *wait_aborted*. If called outside a HPX-thread, this function will throw a *hpx::exception* with an error code of *hpx::error::null_thread_id*. If this function is called while the thread-manager is not running, it will throw an *hpx::exception* with an error code of *hpx::error::invalid_status*.

namespace **threads**

Functions

hpx::*execution*::*parallel_executor* **get_executor**(thread_id_type const &id, *error_code* &ec = *throws*)

Returns a reference to the executor that was used to create the given thread.

Throws

If – `&ec != &throws`, never throws, but will set *ec* to an appropriate value when an error occurs. Otherwise, this function will throw an *hpx::exception* with an error code of *hpx::error::yield_aborted* if it is signaled with *wait_aborted*. If called outside a HPX-thread, this function will throw a *hpx::exception* with an error code of *hpx::error::null_thread_id*. If this function is called while the thread-manager is not running, it will throw an *hpx::exception* with an error code of *hpx::error::invalid_status*.

hpx/executors/executor_scheduler.hpp

Defined in header hpx/executors/executor_scheduler.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

template<typename **Executor**>

struct **executor_scheduler**

Public Types

using **executor_type** = *std*::decay_t<*Executor*>

Public Functions

constexpr **executor_scheduler**() = default

constexpr **executor_scheduler**(*executor_scheduler* const&) = default constexpr *executor_scheduler* &**operator=**(*executor_scheduler* const&) = default

inline constexpr **executor_scheduler**(*executor_scheduler* &&other) noexcept

inline constexpr *executor_scheduler* &**operator=**(*executor_scheduler* &&other) noexcept

template<typename **Exec**>

inline explicit constexpr **executor_scheduler**(*Exec* &&exec) noexcept(*std*::is_nothrow_constructible_v<*executor_type*, *Exec*>)

inline constexpr bool **operator==**(*executor_scheduler* const &rhs) const noexcept

inline constexpr bool **operator!=**(*executor_scheduler* const &rhs) const noexcept

inline constexpr detail::executor_sender<*Executor*> **schedule**() const & noexcept(*std*::is_nothrow_copy_constructible_v<*executor_type*>)

inline constexpr detail::executor_sender<*Executor*> **schedule**() && noexcept(*std*::is_nothrow_move_constructible_v<*executor_type*>)

template<typename **Sender**, typename **Shape**, typename **F**>

auto **bulk**(*Sender* &&sender, *Shape* const &shape, *F* &&f) const

Public Members

HPX_NO_UNIQUE_ADDRESS executor_type exec_

hpx/executors/thread_pool_scheduler.hpp

Defined in header hpx/executors/thread_pool_scheduler.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

Typedefs

using **thread_pool_scheduler** = *thread_pool_policy_scheduler*<*hpx*::*launch*>

Variables

template<typename Sender> concept bulk_chunked_or_unchunked_sender =sender_invokes_algorithm_v<Sender, hpx::execution::experimental::bulk_t> ||sender_invokes_algorithm_v<Sender, hpx::execution::experimental::bulk_chunked_t> ||sender_invokes_algorithm_v<Sender, hpx::execution::experimental::bulk_unchunked_t>

template<typename **Policy**>

constexpr bool **is_sequenced_policy_v** = false

template<>

constexpr bool **is_sequenced_policy_v**<*sequenced_policy*> = true

template<>

constexpr bool **is_sequenced_policy_v**<unsequenced_policy> = true

template<typename **Policy**>

constexpr bool **is_unsequenced_bulk_policy_v** = false

template<>

constexpr bool **is_unsequenced_bulk_policy_v**<unsequenced_policy> = true

template<>

constexpr bool **is_unsequenced_bulk_policy_v**<*parallel_unsequenced_policy*> = true

template<typename **Policy**>

struct **thread_pool_policy_scheduler**

Public Types

using **policy_type** = *Policy*

using **execution_category** = *std*::conditional_t<*std*::is_same_v<*Policy*, *launch*::sync_policy>, *sequenced_execution_tag*, *parallel_execution_tag*>

Public Functions

inline explicit constexpr **thread_pool_policy_scheduler**(*Policy* l = *experimental*::detail::get_default_scheduler_policy<*Policy*>::call())

inline explicit **thread_pool_policy_scheduler**(*hpx*::*threads*::*thread_pool_base* *pool, *Policy* l = *experimental*::detail::get_default_scheduler_policy<*Policy*>::call()) noexcept

template<typename **Policy**>

struct **thread_pool_domain** : public *hpx*::*execution*::*experimental*::detail::sync_wait_domain

Public Functions

template<bulk_chunked_or_unchunked_sender **Sender**, typename **Env**>

inline constexpr auto **transform_sender**(*hpx*::*execution*::*experimental*::set_value_t, *Sender* &&sndr, *Env* const &env) const noexcept

template<typename Sender,
typename Env> inline requires constexpr sender_invokes_algorithm_v< Sender,
hpx::execution::experimental::continues_on_t > auto transform_sender (hpx::execution::experimental::set_v
Sender &&sndr, Env const &) const noexcept

hpx/executors/service_executors.hpp

Defined in header hpx/executors/service_executors.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **execution**

hpx/executors/parallel_executor_aggregated.hpp

Defined in header hpx/executors/parallel_executor_aggregated.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **execution**

hpx/executors/datapar/execution_policy.hpp

Defined in header hpx/executors/datapar/execution_policy.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/executors/datapar/execution_policy_mappings.hpp

Defined in header hpx/executors/datapar/execution_policy_mappings.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

filesystem

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/filesystem/api.hpp

Defined in header hpx/filesystem/api.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

This file provides a compatibility layer using Boost.Filesystem for the C++17 filesystem library. It is *not* intended to be a complete compatibility layer. It only contains functions required by the HPX codebase. It also provides some functions only available in Boost.Filesystem when using C++17 filesystem.

namespace **hpx**

namespace **filesystem**

Functions

inline path **initial_path**()

inline *std*::string **to_string**(path const &p)

inline *std*::string **basename**(path const &p)

inline path **canonical**(path const &p, path const &base)

inline path **canonical**(path const &p, path const &base, *std*::error_code &ec)

namespace **filesystem**

functional

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::mem_fn

Defined in header hpx/functional.hpp[883].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Warning

doxygenfunction: Unable to resolve function "hpx::mem_fn" with arguments "(M C::*)". Candidate function could not be parsed. Parsing error is Invalid C++ declaration: Expected identifier in nested name. [error at 4] (C::*)(Ps…)> mem_fn(R (C::* pm)(Ps…)) noexcept —-^

Warning

doxygenfunction: Unable to resolve function "hpx::mem_fn" with arguments "(R (C::*)(Ps…))". Candidate function could not be parsed. Parsing error is Invalid C++ declaration: Expected identifier in nested name. [error at 4] (C::*)(Ps…)> mem_fn(R (C::* pm)(Ps…)) noexcept —-^

Warning

doxygenfunction: Unable to resolve function "hpx::mem_fn" with arguments "(R (C::*)(Ps…) const)". Candidate function could not be parsed. Parsing error is Invalid C++ declaration: Expected identifier in nested name. [error at 4] (C::*)(Ps…)> mem_fn(R (C::* pm)(Ps…)) noexcept —-^

[883] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

hpx::reference_wrapper

Defined in header hpx/functional.hpp[884].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::ref*
- *hpx::cref*

template<typename **T**>

hpx::**reference_wrapper**(*T*&) -> *reference_wrapper*<*T*>

hpx::ref

Defined in header hpx/functional.hpp[885].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::reference_wrapper*
- *hpx::cref*

template<typename **T**>

constexpr *reference_wrapper*<*T*> *hpx*::**ref**(*T* &val) noexcept

template<typename **T**>

void *hpx*::**ref**(*T* const&&) = delete

template<typename **T**>

constexpr *reference_wrapper*<*T*> *hpx*::**ref**(*reference_wrapper*<*T*> val) noexcept

template<typename **T**>

reference_wrapper<*T*> *hpx*::**ref**(*T* &&val) noexcept

[884] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

[885] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

hpx::cref

Defined in header hpx/functional.hpp[886].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::reference_wrapper*
- *hpx::ref*

template<typename **T**>

constexpr *reference_wrapper*<*T* const> *hpx*::**cref**(*T* const &val) noexcept

template<typename **T**>

void *hpx*::**cref**(*T* const&&) = delete

template<typename **T**>

constexpr *reference_wrapper*<*T* const> *hpx*::**cref**(*reference_wrapper*<*T*> val) noexcept

hpx::bind_back

Defined in header hpx/functional.hpp[887].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Warning

doxygenfunction: Unable to resolve function "hpx::bind_back" with arguments (F&&, Ts&&...) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template
↪<typename F, typename ...Ts> constexpr hpx::detail::bound_
↪back<std::decay_t<F>, util::make_index_pack_t<sizeof...
↪(Ts)>, util::decay_unwrap_t<Ts>...> bind_back(F &&f, Ts&&... vs)
- template<typename F> constexpr std::decay_t<F> bind_back(F &&f)
```

template<typename **F**>

[886] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

[887] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

constexpr *std*::decay_t<*F*> `hpx::bind_back`(*F* &&f)

hpx::invoke_fused

Defined in header hpx/functional.hpp[888].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::invoke_fused_r*

template<typename **F**, typename **Tuple**>

constexpr detail::invoke_fused_result_t<*F*, *Tuple*> `hpx::invoke_fused`(*F* &&f, *Tuple* &&t) noexcept(noexcept(detail::invoke_fused_impl(detail::fused_index_pack_
std::forward<*F*>(*f*),
std::forward<*Tuple*>(*t*))))

Invokes the given callable object f with the content of the sequenced type t (tuples, pairs).

> **Note**
>
> This function is similar to `std::apply` (C++17). The difference between *`hpx::invoke`* and *`hpx::invoke_fused`* is that the later unpacks the tuples while the former cannot. Turning a tuple into a parameter pack is not a trivial operation which makes *`hpx::invoke_fused`* rather useful.

Parameters

- `f` – Must be a callable object. If f is a member function pointer, the first argument in the sequenced type will be treated as the callee (this object).
- `t` – A type whose contents are accessible through a call to *hpx::get*.

Throws

`std::exception` – like objects thrown by call to object f with the arguments contained in the sequenceable type t.

Returns

The result of the callable object when it's called with the content of the given sequenced type.

[888] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

hpx::invoke_fused_r

Defined in header hpx/functional.hpp[889].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::invoke_fused*

template<typename **R**, typename **F**, typename **Tuple**>

constexpr *R* hpx::**invoke_fused_r**(*F* &&f, *Tuple* &&t) noexcept(noexcept(detail::invoke_fused_impl(detail::fused_index_pack_t<*Tuple*>{}, *std*::forward<*F*>(*f*), *std*::forward<*Tuple*>(*t*))))

Invokes the given callable object f with the content of the sequenced type t (tuples, pairs).

> **Note**
>
> This function is similar to `std::apply` (C++17). The difference between *hpx::invoke* and *hpx::invoke_fused* is that the later unpacks the tuples while the former cannot. Turning a tuple into a parameter pack is not a trivial operation which makes *hpx::invoke_fused* rather useful.

> **Note**
>
> The difference between *hpx::invoke_fused* and *hpx::invoke_fused_r* is that the later allows to specify the return type as well.

Parameters

- **f** – Must be a callable object. If f is a member function pointer, the first argument in the sequenced type will be treated as the callee (this object).
- **t** – A type whose contents are accessible through a call to *hpx::get*.

Throws

`std::exception` – like objects thrown by call to object f with the arguments contained in the sequenceable type t.

Template Parameters

R – The result type of the function when it's called with the content of the given sequenced type.

Returns

The result of the callable object when it's called with the content of the given sequenced type.

889 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

hpx::move_only_function

Defined in header hpx/functional.hpp[890].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **Sig**, bool **Serializable** = false>

class **move_only_function**

Class template *hpx::move_only_function* is a general-purpose polymorphic function wrapper. *hpx::move_only_function* objects can store and invoke any constructible (not required to be move constructible) Callable target — functions, lambda expressions, bind expressions, or other function objects, as well as pointers to member functions and pointers to member objects.

The stored callable object is called the target of *hpx::move_only_function*. If an *hpx::move_only_function* contains no target, it is called empty. Unlike *hpx::function*, invoking an empty *hpx::move_only_function* results in undefined behavior.

hpx::move_only_functions supports every possible combination of cv-qualifiers, ref-qualifiers, and noexcept-specifiers not including volatile provided in its template parameter. These qualifiers and specifier (if any) are added to its operator(). *hpx::move_only_function* satisfies the requirements of MoveConstructible and MoveAssignable, but does not satisfy CopyConstructible or CopyAssignable.

hpx::bind

Defined in header hpx/functional.hpp[891].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::placeholders::_1*
- *hpx::placeholders::_2*
- …
- *hpx::placeholders::_9*

Warning

doxygenfunction: Unable to resolve function "hpx::bind" with arguments (F&&, Ts&&…) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename Action, typename ...Ts> detail::bound_
↪action<std::decay_t<Action>, util::make_index_pack_t<sizeof.
↪..(Ts)>, hpx::util::decay_unwrap_t<Ts>...> bind(Ts&&... vs)
- template<typename␣
↪Component, typename Signature, typename Derived, typename␣
↪...Ts> detail::bound_action<Derived, util::make_index_pack_
```

[890] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

[891] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

```
↪t<sizeof...(Ts)>, std::decay_t<Ts>...> bind(hpx::actions::basic_
↪action<Component, Signature, Derived> action, Ts&&... vs)
- template<typename F, typename ...Ts> constexpr␣
↪detail::bound<std::decay_t<F>, util::make_index_pack_t<sizeof.
↪..(Ts)>, util::decay_unwrap_t<Ts>...> bind(F &&f, Ts&&... vs)
```

Warning

doxygenfunction: Unable to resolve function "hpx::bind" with arguments (Ts&&...) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename Action, typename ...Ts> detail::bound_
↪action<std::decay_t<Action>, util::make_index_pack_t<sizeof.
↪..(Ts)>, hpx::util::decay_unwrap_t<Ts>...> bind(Ts&&... vs)
- template<typename␣
↪Component, typename Signature, typename Derived, typename␣
↪...Ts> detail::bound_action<Derived, util::make_index_pack_
↪t<sizeof...(Ts)>, std::decay_t<Ts>...> bind(hpx::actions::basic_
↪action<Component, Signature, Derived> action, Ts&&... vs)
- template<typename F, typename ...Ts> constexpr␣
↪detail::bound<std::decay_t<F>, util::make_index_pack_t<sizeof.
↪..(Ts)>, util::decay_unwrap_t<Ts>...> bind(F &&f, Ts&&... vs)
```

Warning

doxygenfunction: Unable to resolve function "hpx::bind" with arguments (hpx::actions::basic_action<Component, Signature, Derived>, Ts&&...) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename Action, typename ...Ts> detail::bound_
↪action<std::decay_t<Action>, util::make_index_pack_t<sizeof.
↪..(Ts)>, hpx::util::decay_unwrap_t<Ts>...> bind(Ts&&... vs)
- template<typename␣
↪Component, typename Signature, typename Derived, typename␣
↪...Ts> detail::bound_action<Derived, util::make_index_pack_
↪t<sizeof...(Ts)>, std::decay_t<Ts>...> bind(hpx::actions::basic_
↪action<Component, Signature, Derived> action, Ts&&... vs)
- template<typename F, typename ...Ts> constexpr␣
↪detail::bound<std::decay_t<F>, util::make_index_pack_t<sizeof.
↪..(Ts)>, util::decay_unwrap_t<Ts>...> bind(F &&f, Ts&&... vs)
```

hpx::placeholders::_1

Defined in header hpx/functional.hpp[892].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::bind*
- *hpx::placeholders::_2*
- …
- *hpx::placeholders::_9*

constexpr detail::placeholder<1> `hpx::placeholders::_1` = {}

hpx::placeholders::_2

Defined in header hpx/functional.hpp[893].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::bind*
- *hpx::placeholders::_1*
- …
- *hpx::placeholders::_9*

constexpr detail::placeholder<2> `hpx::placeholders::_2` = {}

_

_

Defined in header hpx/functional.hpp[894].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::bind*
- *hpx::placeholders::_1*
- *hpx::placeholders::_2*

[892] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

[893] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

[894] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

- *hpx::placeholders::_9*

Warning

doxygenfunction: Cannot find function "…" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx::placeholders::_9

Defined in header hpx/functional.hpp[895].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::bind*
- *hpx::placeholders::_1*
- *hpx::placeholders::_2*
- …

constexpr detail::placeholder<9> *hpx*::placeholders::**_9** = {}

hpx::function_ref

Defined in header hpx/functional.hpp[896].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **Sig**>

class **function_ref**

function_ref class is a vocabulary type with reference semantics for passing entities to call.

An example use case that benefits from higher-order functions is `retry(n,f)` which attempts to call `f` up to `n` times synchronously until success. This example might model the real-world scenario of repeatedly querying a flaky web service.

```
using payload = std::optional< /* ... */ >;
// Repeatedly invokes `action` up to `times` repetitions.
// Immediately returns if `action` returns a valid `payload`.
// Returns `std::nullopt` otherwise.
payload retry(size_t times, /* ????? */ action);
```

[895] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

[896] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

The passed-in action should be a callable entity that takes no arguments and returns a payload. This can be done with function pointers, *hpx::function* or a template but it is much simpler with *function_ref* as seen below:

```
payload retry(size_t times, function_ref<payload()> action);
```

hpx::function

Defined in header hpx/functional.hpp[897].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **Sig**, bool **Serializable** = false>

class **function**

Class template *hpx::function* is a general-purpose polymorphic function wrapper. Instances of *hpx::function* can store, copy, and invoke any CopyConstructible Callable target — functions, lambda expressions, bind expressions, or other function objects, as well as pointers to member functions and pointers to data members. The stored callable object is called the target of *hpx::function*. If an *hpx::function* contains no target, it is called empty. Invoking the target of an empty *hpx::function* results in *hpx::error::bad_function_call* exception being thrown. *hpx::function* satisfies the requirements of CopyConstructible and CopyAssignable.

hpx::bind_front

Defined in header hpx/functional.hpp[898].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Warning

doxygenfunction: Unable to resolve function "hpx::bind_front" with arguments (F&&, Ts&&…) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename F, typename ...Ts> constexpr detail::bound_
↪front<std::decay_t<F>, util::make_index_pack_t<sizeof...(Ts)>
↪, util::decay_unwrap_t<Ts>...> bind_front(F &&f, Ts&&... vs)
- template<typename F> constexpr std::decay_t<F> bind_front(F &&f)
```

template<typename **F**>

constexpr *std*::decay_t<*F*> *hpx*::**bind_front**(*F* &&f)

[897] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

[898] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

hpx::experimental::scope_success

Defined in header hpx/experimental/scope.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **F**>

auto *hpx*::*experimental*::**scope_success**(*F* &&f)

The class template scope_success is a general-purpose scope guard intended to call its exit function when a scope is normally exited.

Template Parameters
F – type of stored exit function

Parameters
f – stored exit function

hpx::experimental::scope_fail

Defined in header hpx/experimental/scope.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **F**>

auto *hpx*::*experimental*::**scope_fail**(*F* &&f)

The class template scope_fail is a general-purpose scope guard intended to call its exit function when a scope is exited via an exception.

Template Parameters
F – type of stored exit function

Parameters
f – stored exit function

hpx::experimental::scope_exit

Defined in header hpx/experimental/scope.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **F**>

auto *hpx*::*experimental*::**scope_exit**(*F* &&f)

The class template scope_exit is a general-purpose scope guard intended to call its exit function when a scope is exited.

Template Parameters
F – type of stored exit function

Parameters
f – stored exit function

hpx::is_bind_expression

Defined in header hpx/functional.hpp[899].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**>

struct **is_bind_expression** : public *std*::is_bind_expression<*T*>

If `T` is the type produced by a call to `hpx::bind`, this template is derived from `std::true_type`. For any other type, this template is derived from `std::false_type`.

This template may be specialized for a user-defined type `T` to implement *UnaryTypeTrait* with base characteristic of `std::true_type` to indicate that `T` should be treated by `hpx::bind` as if it were the type of a bind subexpression: when a bind-generated function object is invoked, a bound argument of this type will be invoked as a function object and will be given all the unbound arguments passed to the bind-generated object.

Subclassed by hpx::is_bind_expression< T const >

hpx::is_placeholder

Defined in header hpx/functional.hpp[900].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **T**>

struct **is_placeholder**

If `T` is a standard, Boost, or HPX placeholder (_1, _2, _3, …) then this template is derived from `std::integral_constant<int,1>`, `std::integral_constant<int, 2>`, `std::integral_constant<int,3>`, respectively. Otherwise, it is derived from `std::integral_constant<int,0>`.

The template may be specialized for any user-defined `T` type: the specialization must satisfy *UnaryTypeTrait* with base characteristic of `std::integral_constant<int, N>` with N>0 to indicate that `T` should be treated as N'th placeholder type. `hpx::bind` uses *`hpx::is_placeholder`* to detect placeholders for unbound arguments.

[899] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

[900] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

futures

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::packaged_task

Defined in header hpx/future.hpp[901].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **Sig**>

class **packaged_task**

The class template *hpx::packaged_task* wraps any Callable` target (function, lambda expression, bind expression, or another function object) so that it can be invoked asynchronously. Its return value or exception thrown is stored in a shared state which can be accessed through *hpx::future* objects. Just like *hpx::function*, *hpx::packaged_task* is a polymorphic, allocator-aware container: the stored callable target may be allocated on heap or with a provided allocator.

hpx::promise

Defined in header hpx/future.hpp[902].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **R**>

class **promise** : public *hpx*::detail::promise_base<*R*>

The class template *hpx::promise* provides a facility to store a value or an exception that is later acquired asynchronously via a *hpx::future* object created by the *hpx::promise* object. Note that the *hpx::promise* object is meant to be used only once. Each promise is associated with a shared state, which contains some state information and a result which may be not yet evaluated, evaluated to a value (possibly void) or evaluated to an exception. A promise may do three things with the shared state:

- make ready: the promise stores the result or the exception in the shared state. Marks the state ready and unblocks any thread waiting on a future associated with the shared state.
- release: the promise gives up its reference to the shared state. If this was the last such reference, the shared state is destroyed. Unless this was a shared state created by hpx::async which is not yet ready, this operation does not block.
- abandon: the promise stores the exception of type hpx::future_error with error code hpx::error::broken_promise, makes the shared state ready, and then releases it. The promise is the "push" end of the promise-future communication channel: the operation that stores a value in the shared state synchronizes-with (as defined in hpx::memory_order) the successful return from any function that is waiting on the shared state (such as hpx::future::get). Concurrent access to the same shared state may conflict otherwise: for example multiple callers of hpx::shared_future::get must either all be read-only or provide external synchronization.

[901] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp
[902] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

hpx::future

Defined in header hpx/future.hpp[903].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::shared_future*
- *hpx::make_future*
- *hpx::make_shared_future*
- *hpx::make_ready_future*
- *hpx::make_ready_future_alloc*
- *hpx::make_ready_future_at*
- *hpx::make_ready_future_after*
- *hpx::make_exceptional_future*

template<typename **R**>

class **future** : public *hpx*::*lcos*::detail::future_base<*future*<*R*>, *R*>

The class template *hpx::future* provides a mechanism to access the result of asynchronous operations:

- An asynchronous operation (created via hpx::async, *hpx::packaged_task*, or *hpx::promise*) can provide a *hpx::future* object to the creator of that asynchronous operation.
- The creator of the asynchronous operation can then use a variety of methods to query, wait for, or extract a value from the *hpx::future*. These methods may block if the asynchronous operation has not yet provided a value.
- When the asynchronous operation is ready to send a result to the creator, it can do so by modifying shared state (e.g. hpx::promise::set_value) that is linked to the creator's *hpx::future*. Note that *hpx::future* references shared state that is not shared with any other asynchronous return objects (as opposed to *hpx::shared_future*).

hpx::shared_future

Defined in header hpx/future.hpp[904].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::future*
- *hpx::make_future*
- *hpx::make_shared_future*
- *hpx::make_ready_future*
- *hpx::make_ready_future_alloc*

[903] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp
[904] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

- *hpx::make_ready_future_at*
- *hpx::make_ready_future_after*
- *hpx::make_exceptional_future*

template<typename **R**>

class **shared_future** : public *hpx*::*lcos*::detail::future_base<*shared_future*<*R*>, *R*>

The class template *hpx::shared_future* provides a mechanism to access the result of asynchronous operations, similar to *hpx::future*, except that multiple threads are allowed to wait for the same shared state. Unlike *hpx::future*, which is only moveable (so only one instance can refer to any particular asynchronous result), *hpx::shared_future* is copyable and multiple shared future objects may refer to the same shared state. Access to the same shared state from multiple threads is safe if each thread does it through its own copy of a *shared_future* object.

hpx::make_future

Defined in header hpx/future.hpp[905].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::future*
- *hpx::shared_future*
- *hpx::make_shared_future*
- *hpx::make_ready_future*
- *hpx::make_ready_future_alloc*
- *hpx::make_ready_future_at*
- *hpx::make_ready_future_after*
- *hpx::make_exceptional_future*

template<typename **R**, typename **U**>

hpx::*future*<*R*> `hpx::make_future`(*hpx*::*future*<*U*> &&f)

Converts any future of type U to any other future of type R based on an existing conversion path from U to R.

template<typename **R**, typename **U**, typename **Conv**>

hpx::*future*<*R*> `hpx::make_future`(*hpx*::*future*<*U*> &&f, *Conv* &&conv)

Converts any future of type U to any other future of type R based on a given conversion function: R conv(U).

template<typename **R**, typename **U**>

[905] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

hpx::*future*<*R*> **hpx::make_future**(*hpx*::*shared_future*<*U*> f)

Converts any *shared_future* of type U to any other future of type R based on an existing conversion path from U to R.

template<typename **R**, typename **U**, typename **Conv**>

hpx::*future*<*R*> **hpx::make_future**(*hpx*::*shared_future*<*U*> f, *Conv* &&conv)

Converts any future of type U to any other future of type R based on an existing conversion path from U to R.

hpx::make_shared_future

Defined in header hpx/future.hpp[906].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::future*
- *hpx::shared_future*
- *hpx::make_future*
- *hpx::make_ready_future*
- *hpx::make_ready_future_alloc*
- *hpx::make_ready_future_at*
- *hpx::make_ready_future_after*
- *hpx::make_exceptional_future*

template<typename **R**>

hpx::*shared_future*<*R*> **hpx::make_shared_future**(*hpx*::*future*<*R*> &&f) noexcept

Converts any future or *shared_future* of type T to a corresponding *shared_future* of type T.

template<typename **R**>

hpx::*shared_future*<*R*> &**hpx::make_shared_future**(*hpx*::*shared_future*<*R*> &f) noexcept

Converts any future or *shared_future* of type T to a corresponding *shared_future* of type T.

template<typename **R**>

hpx::*shared_future*<*R*> &&**hpx::make_shared_future**(*hpx*::*shared_future*<*R*> &&f) noexcept

Converts any future or *shared_future* of type T to a corresponding *shared_future* of type T.

[906] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

template<typename **R**>

hpx::*shared_future*<*R*> const &*hpx*::**make_shared_future**(*hpx*::*shared_future*<*R*> const &f) noexcept

Converts any future or *shared_future* of type T to a corresponding *shared_future* of type T.

hpx::make_ready_future

Defined in header hpx/future.hpp[907].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::future*
- *hpx::shared_future*
- *hpx::make_future*
- *hpx::make_shared_future*
- *hpx::make_ready_future_alloc*
- *hpx::make_ready_future_at*
- *hpx::make_ready_future_after*
- *hpx::make_exceptional_future*

template<typename **T**, typename ...**Ts**>

std::enable_if_t<*std*::is_constructible_v<*T*, *Ts*&&...> || *std*::is_void_v<*T*>, *future*<*T*>> *hpx*::**make_ready_future**(*Ts*&&... ts)

The function creates a shared state that is immediately ready and returns a future associated with that shared state. For the returned future, valid() == true and is_ready() == true.

template<int **DeductionGuard** = 0, typename **T**>

future<*hpx*::*util*::decay_unwrap_t<*T*>> *hpx*::**make_ready_future**(*T* &&init)

The function creates a shared state that is immediately ready and returns a future associated with that shared state. For the returned future, valid() == true and is_ready() == true.

future<void> *hpx*::**make_ready_future**()

The function creates a shared state that is immediately ready and returns a future associated with that shared state. For the returned future, valid() == true and is_ready() == true.

[907] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

hpx::make_ready_future_alloc

Defined in header hpx/future.hpp[908].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::future*
- *hpx::shared_future*
- *hpx::make_future*
- *hpx::make_shared_future*
- *hpx::make_ready_future*
- *hpx::make_ready_future_at*
- *hpx::make_ready_future_after*
- *hpx::make_exceptional_future*

template<typename **T**, typename **Allocator**, typename ...**Ts**>

std::enable_if_t<*std*::is_constructible_v<*T*, *Ts*&&...> || *std*::is_void_v<*T*>, *future*<*T*>> `hpx::make_ready_future_alloc`(*Allocator* const &a, *Ts*&&... ts)

Creates a pre-initialized future object with allocator (extension)

template<int **DeductionGuard** = 0, typename **Allocator**, typename **T**>

future<*hpx*::*util*::decay_unwrap_t<*T*>> `hpx::make_ready_future_alloc`(*Allocator* const &a, *T* &&init)

template<typename **Allocator**>

inline *future*<void> `hpx::make_ready_future_alloc`(*Allocator* const &a)

hpx::make_ready_future_at

Defined in header hpx/future.hpp[909].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::future*
- *hpx::shared_future*
- *hpx::make_future*

[908] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp
[909] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

- *hpx::make_shared_future*
- *hpx::make_ready_future*
- *hpx::make_ready_future_alloc*
- *hpx::make_ready_future_after*
- *hpx::make_exceptional_future*

template<int **DeductionGuard** = 0, typename **T**>

future<*hpx*::*util*::decay_unwrap_t<*T*>> `hpx::make_ready_future_at`(*hpx*::chrono::steady_time_point const &abs_time, *T* &&init)

Creates a pre-initialized future object which gets ready at a given point in time (extension)

inline *future*<void> `hpx::make_ready_future_at`(*hpx*::chrono::steady_time_point const &abs_time)

Creates a pre-initialized future object which gets ready at a given point in time (extension)

hpx::make_ready_future_after

Defined in header hpx/future.hpp[910].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::future*
- *hpx::shared_future*
- *hpx::make_future*
- *hpx::make_shared_future*
- *hpx::make_ready_future*
- *hpx::make_ready_future_alloc*
- *hpx::make_ready_future_at*
- *hpx::make_exceptional_future*

template<int **DeductionGuard** = 0, typename **T**>

future<*hpx*::*util*::decay_unwrap_t<*T*>> `hpx::make_ready_future_after`(*hpx*::chrono::steady_duration const &rel_time, *T* &&init)

Creates a pre-initialized future object which gets ready after a given point in time (extension)

inline *future*<void> `hpx::make_ready_future_after`(*hpx*::chrono::steady_duration const &rel_time)

Creates a pre-initialized future object which gets ready after a given point in time (extension)

[910] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

hpx::make_exceptional_future

Defined in header hpx/future.hpp[911].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::future*
- *hpx::shared_future*
- *hpx::make_future*
- *hpx::make_shared_future*
- *hpx::make_ready_future*
- *hpx::make_ready_future_alloc*
- *hpx::make_ready_future_at*
- *hpx::make_ready_future_after*

template<typename **T**>

future<*T*> `hpx::`**`make_exceptional_future`**(*std*::exception_ptr const &e)

Creates a pre-initialized future object which holds the given error (extension)

template<typename **T**, typename **E**>

future<*T*> `hpx::`**`make_exceptional_future`**(*E* e)

Creates a pre-initialized future object which holds the given error (extension)

hpx/futures/future_fwd.hpp

Defined in header hpx/futures/future_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Top level HPX namespace.

namespace **lcos**

namespace **lcos**

[911] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/future.hpp

io_service

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/io_service/io_service_pool.hpp

Defined in header hpx/io_service/io_service_pool.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

class **io_service_pool**

#include <io_service_pool.hpp> A pool of io_service objects.

Public Functions

explicit **io_service_pool**(*std*::size_t pool_size = 2, *threads*::*policies*::callback_notifier const &notifier = *threads*::*policies*::callback_notifier(), char const *pool_name = "", char const *name_postfix = "")

Construct the io_service pool.

Parameters

- **pool_size** – [in] The number of threads to run to serve incoming requests
- **notifier** – [in]
- **pool_name** – [in]
- **name_postfix** – [in]

explicit **io_service_pool**(*threads*::*policies*::callback_notifier const &notifier, char const *pool_name = "", char const *name_postfix = "")

Construct the io_service pool.

Parameters

- **notifier** – [in]
- **pool_name** – [in]
- **name_postfix** – [in]

io_service_pool(*io_service_pool* const&) = delete

io_service_pool(*io_service_pool*&&) = delete *io_service_pool* &**operator=**(*io_service_pool* const&) = delete

io_service_pool &**operator=**(*io_service_pool*&&) = delete

~io_service_pool()

bool **run**(bool join_threads = true, *std*::shared_ptr<*barrier*> startup = {}) const

Run all io_service objects in the pool. If join_threads is true this will also wait for all threads to complete.

Parameters

- **join_threads** – [in] If true, wait for all pool threads to join before returning.
- **startup** – [in] Optional startup barrier used to synchronize pool worker thread startup with the caller. Its participant count must equal the number of pool worker threads plus one for the caller. The caller must call startup->*wait()* after invoking *run()*.

bool **run**(*std*::size_t num_threads, bool join_threads = true, *std*::shared_ptr<*barrier*> startup = {}) const

Run num_threads io_service objects in the pool. If join_threads is true this will also wait for all threads to complete.

Parameters

- **num_threads** – [in] The number of worker threads to start.
- **join_threads** – [in] If true, wait for all pool threads to join before returning.
- **startup** – [in] Optional startup barrier used to synchronize pool worker thread startup with the caller. Its participant count must equal num_threads plus one for the caller. The caller must call startup->*wait()* after invoking *run()*.

void **stop**() const

Stop all io_service objects in the pool.

void **join**() const

Join all io_service threads in the pool.

void **clear**() const

Clear all internal data structures.

void **wait**() const

Wait for all work to be done.

bool **stopped**() const

::asio::io_context &**get_io_service**(int index = -1) const

Get an io_service to use.

std::thread &**get_os_thread_handle**(*std*::size_t thread_num) const

access underlying thread handle

std::size_t **size**() const noexcept

Get number of threads associated with this I/O service.

void **thread_run**(*std*::size_t index, *std*::shared_ptr<*barrier*> startup = {}) const

Activate the thread *index* for this thread pool.

char const ***get_name**() const noexcept

Return name of this pool.

void **init**(*std*::size_t pool_size) const

Private Members

detail::io_service_pool_base ***pool_**

lcos_local

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/lcos_local/trigger.hpp

Defined in header hpx/lcos_local/trigger.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **lcos**

namespace **local**

template<typename **Mutex** = *hpx*::*spinlock*>

struct **base_trigger**

Public Functions

inline **base_trigger**() noexcept

inline **base_trigger**(*base_trigger* &&rhs) noexcept

inline *base_trigger* &**operator=**(*base_trigger* &&rhs) noexcept

inline *hpx*::*future*<void> **get_future**(*std*::size_t *generation_value = nullptr, *error_code* &ec = *hpx*::*throws*)

get a future allowing to wait for the trigger to fire

inline bool **set**(*error_code* &ec = *throws*)

Trigger this object.

inline void **synchronize**(*std*::size_t generation_value, char const *function_name = "trigger::synchronize", *error_code* &ec = *throws*)

Wait for the generational counter to reach the requested stage.

inline *std*::size_t **next_generation**()

inline *std*::size_t **generation**() const

Protected Types

using **mutex_type** = *Mutex*

Protected Functions

inline bool **trigger_conditions**(*error_code* &ec = *throws*)

template<typename **Lock**>

inline void **synchronize**(*std*::size_t generation_value, *Lock* &l, char const *function_name = "trigger::synchronize", *error_code* &ec = *throws*)

Private Types

using **condition_list_type** = *hpx*::detail::intrusive_list<*condition_list_entry*>

Private Functions

inline bool **test_condition**(*std*::size_t const generation_value) const noexcept

Private Members

mutable *mutex_type* **mtx_**

hpx::*promise*<void> **promise_**

std::size_t **generation_**

condition_list_type **conditions_**

struct **condition_list_entry** : public conditional_trigger

Public Functions

condition_list_entry() = default

Public Members

condition_list_entry ***prev** = nullptr

condition_list_entry ***next** = nullptr

struct **manage_condition**

Public Functions

inline **manage_condition**(*base_trigger* &gate, *condition_list_entry* &cond) noexcept

inline **~manage_condition**()

template<typename **Condition**>

inline *hpx*::*future*<void> **get_future**(*Condition* &&func, *error_code* &ec = *hpx*::*throws*)

Public Members

base_trigger &**this_**

condition_list_entry &**e_**

struct **trigger** : public *hpx*::*lcos*::*local*::*base_trigger*<*hpx*::*no_mutex*>

Public Functions

trigger() = default

inline **trigger**(*trigger* &&rhs) noexcept

inline *trigger* &**operator=**(*trigger* &&rhs) noexcept

template<typename **Lock**>

inline void **synchronize**(*std*::size_t generation_value, *Lock* &l, char const *function_name = "trigger::synchronize", *error_code* &ec = *throws*)

Private Types

using **base_type** = *base_trigger*<*hpx*::*no_mutex*>

hpx::lcos::local::and_gate

Defined in header hpx/modules/lcos_local.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

struct **and_gate** : public *hpx*::*lcos*::*local*::base_and_gate<*hpx*::*no_mutex*>

An AND-gate synchronization primitive that fires once all of its expected inputs have been received.

> **Note**
>
> This type is not thread-safe. It has to be protected from concurrent access by different threads by the code using instances of this type.

pack_traversal

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/pack_traversal/pack_traversal.hpp

Defined in header hpx/pack_traversal/pack_traversal.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

Functions

template<typename Mapper, typename... T> < unspecified > map_pack (Mapper &&mapper, T &&... pack)

Maps the pack with the given mapper.

This function tries to visit all plain elements which may be wrapped in:

- homogeneous containers (`std::vector`, `std::list`)
- heterogeneous containers (`hpx::tuple`, `std::pair`, `std::array`) and re-assembles the pack with the result of the mapper. Mapping from one type to a different one is supported.

Elements that aren't accepted by the mapper are routed through and preserved through the hierarchy.

```
// Maps all integers to floats
map_pack([](int value) {
    return float(value);
},
1, hpx::make_tuple(2, std::vector<int>{3, 4}), 5);
```

Throws

`std::exception` – like objects which are thrown by an invocation to the mapper.

Parameters

- **mapper** – A callable object, which accept an arbitrary type and maps it to another type or the same one.
- **pack** – An arbitrary variadic pack which may contain any type.

Returns

The mapped element or in case the pack contains multiple elements, the pack is wrapped into a `hpx::tuple`.

hpx::functional::unwrap

Defined in header hpx/unwrap.hpp[912].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::functional::unwrap_n*
- *hpx::functional::unwrap_all*
- *hpx::unwrap*
- *hpx::unwrap_n*
- *hpx::unwrap_all*
- *hpx::unwrapping*
- *hpx::unwrapping_n*
- *hpx::unwrapping_all*

struct **unwrap**

A helper function object for functionally invoking `hpx::unwrap`. For more information please refer to its documentation.

hpx::functional::unwrap_n

Defined in header hpx/unwrap.hpp[913].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::functional::unwrap*
- *hpx::functional::unwrap_all*
- *hpx::unwrap*
- *hpx::unwrap_n*
- *hpx::unwrap_all*
- *hpx::unwrapping*
- *hpx::unwrapping_n*
- *hpx::unwrapping_all*

template<*std*::size_t **Depth**>

struct **unwrap_n**

A helper function object for functionally invoking `hpx::unwrap_n`. For more information please refer to its documentation.

[912] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/unwrap.hpp
[913] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/unwrap.hpp

hpx::functional::unwrap_all

Defined in header hpx/unwrap.hpp[914].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::functional::unwrap*
- *hpx::functional::unwrap_n*
- *hpx::unwrap*
- *hpx::unwrap_n*
- *hpx::unwrap_all*
- *hpx::unwrapping*
- *hpx::unwrapping_n*
- *hpx::unwrapping_all*

struct **unwrap_all**

A helper function object for functionally invoking `hpx::unwrap_all`. For more information please refer to its documentation.

hpx::unwrap

Defined in header hpx/unwrap.hpp[915].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::functional::unwrap*
- *hpx::functional::unwrap_n*
- *hpx::functional::unwrap_all*
- *hpx::unwrap_n*
- *hpx::unwrap_all*
- *hpx::unwrapping*
- *hpx::unwrapping_n*
- *hpx::unwrapping_all*

template<typename ...**Args**>

auto `hpx::unwrap`(*Args*&&... args) -> decltype(*util*::detail::unwrap_depth_impl<1U>(*std*::forward<*Args*>(*args*)...))

A helper function for retrieving the actual result of any *hpx::future* like type which is wrapped in an arbitrary way.

914 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/unwrap.hpp
915 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/unwrap.hpp

Unwraps the given pack of arguments, so that any *hpx::future* object is replaced by its future result type in the argument pack:

- `hpx::future<int>` -> `int`
- `hpx::future<std::vector<float>>` -> `std::vector<float>`
- `std::vector<future<float>>` -> `std::vector<float>`

The function is capable of unwrapping *hpx::future* like objects that are wrapped inside any container or tuple like type, see hpx::util::map_pack() for a detailed description about which surrounding types are supported. Non *hpx::future* like types are permitted as arguments and passed through.

```
hpx:unwrap(hpx::make_ready_future(0));

// Multiple arguments hpx::tuple<int, int> i2 =
    hpx:unwrap(hpx::make_ready_future(1),
               hpx::make_ready_future(2));
```

> **Note**
>
> This function unwraps the given arguments until the first traversed nested *hpx::future* which corresponds to an unwrapping depth of one. See *hpx::unwrap_n()* for a function which unwraps the given arguments to a particular depth or *hpx::unwrap_all()* that unwraps all future like objects recursively which are contained in the arguments.

Parameters

args – the arguments that are unwrapped which may contain any arbitrary future or non-future type.

Throws

`std::exception` – like objects in case any of the given wrapped *hpx::future* objects were resolved through an exception. See hpx::future::get() for details.

Returns

Depending on the count of arguments this function returns a *hpx::tuple* containing the unwrapped arguments if multiple arguments are given. In case the function is called with a single argument, the argument is unwrapped and returned.

hpx::unwrap_n

Defined in header hpx/unwrap.hpp[916].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::functional::unwrap*
- *hpx::functional::unwrap_n*
- *hpx::functional::unwrap_all*
- *hpx::unwrap*
- *hpx::unwrap_all*

[916] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/unwrap.hpp

- *hpx::unwrapping*
- *hpx::unwrapping_n*
- *hpx::unwrapping_all*

template<*std*::size_t **Depth**, typename ...**Args**>

auto *hpx*::**unwrap_n**(*Args*&&... args) -> decltype(*util*::detail::unwrap_depth_impl<*Depth*>(*std*::forward<*Args*>(*args*)...))

An alternative version of *hpx::unwrap()*, which unwraps the given arguments to a certain depth of *hpx::future* like objects.

See unwrap for a detailed description.

Template Parameters

Depth – The count of *hpx::future* like objects which are unwrapped maximally.

hpx::unwrap_all

Defined in header hpx/unwrap.hpp[917].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::functional::unwrap*
- *hpx::functional::unwrap_n*
- *hpx::functional::unwrap_all*
- *hpx::unwrap*
- *hpx::unwrap_n*
- *hpx::unwrapping*
- *hpx::unwrapping_n*
- *hpx::unwrapping_all*

template<typename ...**Args**>

auto *hpx*::**unwrap_all**(*Args*&&... args) -> decltype(*util*::detail::unwrap_depth_impl<0U>(*std*::forward<*Args*>(*args*)...))

An alternative version of *hpx::unwrap()*, which unwraps the given arguments recursively so that all contained *hpx::future* like objects are replaced by their actual value.

See *hpx::unwrap()* for a detailed description.

[917] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/unwrap.hpp

hpx::unwrapping

Defined in header hpx/unwrap.hpp[918].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::functional::unwrap*
- *hpx::functional::unwrap_n*
- *hpx::functional::unwrap_all*
- *hpx::unwrap*
- *hpx::unwrap_n*
- *hpx::unwrap_all*
- *hpx::unwrapping_n*
- *hpx::unwrapping_all*

template<typename **T**>

auto *hpx*::**unwrapping**(*T* &&callable) -> decltype(*util*::detail::functional_unwrap_depth_impl<1U>(*std*::forward<*T*>(*callable*)))

Returns a callable object which unwraps its arguments upon invocation using the *hpx::unwrap()* function and then passes the result to the given callable object.

```
    return left + right;
});

int i1 = callable(hpx::make_ready_future(1),
                  hpx::make_ready_future(2));
```

See *hpx::unwrap()* for a detailed description.

Parameters

callable – the callable object which is called with the result of the corresponding unwrap function.

hpx::unwrapping_n

Defined in header hpx/unwrap.hpp[919].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::functional::unwrap*
- *hpx::functional::unwrap_n*
- *hpx::functional::unwrap_all*
- *hpx::unwrap*

[918] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/unwrap.hpp
[919] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/unwrap.hpp

- *hpx::unwrap_n*
- *hpx::unwrap_all*
- *hpx::unwrapping*
- *hpx::unwrapping_all*

template<*std*::size_t **Depth**, typename **T**>

auto *hpx*::**unwrapping_n**(*T* &&callable) -> decltype(*util*::detail::functional_unwrap_depth_impl<*Depth*>(*std*::forward<*T*>(*callable*)))

Returns a callable object which unwraps its arguments upon invocation using the *hpx::unwrap_n()* function and then passes the result to the given callable object.

See *hpx::unwrapping()* for a detailed description.

hpx::unwrapping_all

Defined in header hpx/unwrap.hpp[920].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::functional::unwrap*
- *hpx::functional::unwrap_n*
- *hpx::functional::unwrap_all*
- *hpx::unwrap*
- *hpx::unwrap_n*
- *hpx::unwrap_all*
- *hpx::unwrapping*
- *hpx::unwrapping_n*

template<typename **T**>

auto *hpx*::**unwrapping_all**(*T* &&callable) -> decltype(*util*::detail::functional_unwrap_depth_impl<0U>(*std*::forward<*T*>(*callable*)))

Returns a callable object which unwraps its arguments upon invocation using the *hpx::unwrap_all()* function and then passes the result to the given callable object.

See *hpx::unwrapping()* for a detailed description.

[920] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/unwrap.hpp

hpx/pack_traversal/pack_traversal_async.hpp

Defined in header hpx/pack_traversal/pack_traversal_async.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

Functions

template<typename **Visitor**, typename ...**T**>

auto **traverse_pack_async**(*Visitor* &&visitor, *T*&&... pack) -> decltype(detail::apply_pack_transform_async(*std*::forward<*Visitor*>(*visitor*), *std*::forward<*T*>(*pack*)...))

Traverses the pack with the given visitor in an asynchronous way.

This function works in the same way as `traverse_pack`, however, we are able to suspend and continue the traversal at later time. Thus, we require a visitor callable object which provides three `operator()` overloads as depicted by the code sample below:

```
struct my_async_visitor {
    // The synchronous overload is called for each object. It may
    // return false to suspend the current control. In that case the
    // overload below is called.
    template <typename T>
    bool operator()(async_traverse_visit_tag, T&& element)
    {
        return true;
    }

    // The asynchronous overload is called when the synchronous
    // overload returned false. In addition to the current visited
    // element, the overload is called with a continuation callable
    // object which resumes the traversal when called later.
    template <typename T, typename N>
    void operator()(async_traverse_detach_tag, T&& element, N&& next)
    {
    }

    // The overload is called when the traversal was finished. The
    // whole pack which was traversed asynchronously is passed.
    template <typename T>
    void operator()(async_traverse_complete_tag, T&& pack)
```

(continues on next page)

(continued from previous page)

```
    {
    }
};
```

See `traverse_pack` for a detailed description about the traversal behavior and capabilities.

Parameters

- **`visitor`** – A visitor object which provides the three `operator()` overloads that were described above. Additionally, the visitor must be compatible for referencing it from a `hpx::intrusive_ptr`. The visitor should have a virtual destructor!
- **`pack`** – The arbitrary parameter pack which is traversed asynchronously. Nested objects inside containers and tuple like types are traversed recursively.

Returns

A hpx::intrusive_ptr that references an instance of the given visitor object.

template<typename **Allocator**, typename **Visitor**, typename ...**T**>

auto **`traverse_pack_async_allocator`**(*Allocator* const &alloc, *Visitor* &&visitor, *T*&&... pack) -> decltype(detail::apply_pack_transform_async_allocator(*alloc*, *std*::forward<*Visitor*>(*visitor*), *std*::forward<*T*>(*pack*)...))

Traverses the pack with the given visitor in an asynchronous way.

This function works in the same way as `traverse_pack`, however, we are able to suspend and continue the traversal at later time. Thus, we require a visitor callable object which provides three `operator()` overloads as depicted by the code sample below:

```
struct my_async_visitor {
    // The synchronous overload is called for each object. It may
    // return false to suspend the current control. In that case the
    // overload below is called.
    template <typename T>
    bool operator()(async_traverse_visit_tag, T&& element)
    {
        return true;
    }

    // The asynchronous overload is called when the synchronous
    // overload returned false. In addition to the current visited
    // element, this overload receives a continuation callable object
    // which resumes the traversal when called later. The continuation
    // may be stored and called later or dropped completely to abort
    // the traversal early.
    template <typename T, typename N>
    void operator()(async_traverse_detach_tag, T&& element, N&& next)
```

(continues on next page)

(continued from previous page)

```
    {
    }

    // This overload is called when the traversal is finished. The
    // complete pack that was traversed asynchronously is passed.
    template <typename T>
    void operator()(async_traverse_complete_tag, T&& pack)
    {
    }
};
```

See `traverse_pack` for a detailed description about the traversal behavior and capabilities.

Parameters

- **visitor** – A visitor object which provides the three `operator()` overloads that were described above. Additionally, the visitor must be compatible for referencing it from a `hpx::intrusive_ptr`. The visitor should have a virtual destructor!
- **pack** – The arbitrary parameter pack which is traversed asynchronously. Nested objects inside containers and tuple like types are traversed recursively.
- **alloc** – Allocator instance to use to create the traversal frame.

Returns

A hpx::intrusive_ptr that references an instance of the given visitor object.

preprocessor

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/preprocessor/strip_parens.hpp

Defined in header hpx/preprocessor/strip_parens.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Defines

HPX_PP_STRIP_PARENS(X)

For any symbol `X`, this macro returns the same symbol from which potential outer parens have been removed. If no outer parens are found, this macros evaluates to `X` itself without error.

The original implementation of this macro is from Steven Watanbe as shown in http://boost.2283326.n4.nabble.com/preprocessor-removing-parentheses-td2591973.html#a2591976

```
HPX_PP_STRIP_PARENS(no_parens)
HPX_PP_STRIP_PARENS((with_parens))
```

Example Usage:

This produces the following output

```
no_parens
with_parens
```

Parameters

- **X** – Symbol to strip parens from

hpx/preprocessor/cat.hpp

Defined in header hpx/preprocessor/cat.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Defines

HPX_PP_CAT(A, B)

Concatenates the tokens `A` and `B` into a single token. Evaluates to `AB`

Parameters

- **A** – First token
- **B** – Second token

hpx/preprocessor/nargs.hpp

Defined in header hpx/preprocessor/nargs.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Defines

HPX_PP_NARGS(...)

Expands to the number of arguments passed in

Example Usage:

```
HPX_PP_NARGS(hpx, pp, nargs)
HPX_PP_NARGS(hpx, pp)
HPX_PP_NARGS(hpx)
```

Expands to:

```
3
2
1
```

Parameters

- **...** – The variadic number of arguments

hpx/preprocessor/stringize.hpp

Defined in header hpx/preprocessor/stringize.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Defines

HPX_PP_STRINGIZE(X)

The *HPX_PP_STRINGIZE* macro stringizes its argument after it has been expanded.

The passed argument `X` will expand to `"X"`. Note that the stringizing operator (#) prevents arguments from expanding. This macro circumvents this shortcoming.

Parameters

- **X** – The text to be converted to a string literal

hpx/preprocessor/expand.hpp

Defined in header hpx/preprocessor/expand.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Defines

HPX_PP_EXPAND(X)

The HPX_PP_EXPAND macro performs a double macro-expansion on its argument.

This macro can be used to produce a delayed preprocessor expansion.

Example:

```
#define MACRO(a, b, c) (a)(b)(c)
#define ARGS() (1, 2, 3)

HPX_PP_EXPAND(MACRO ARGS()) // expands to (1)(2)(3)
```

Parameters

- **X** – Token to be expanded twice

resiliency

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/resiliency/replay_executor.hpp

Defined in header hpx/resiliency/replay_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

template<typename **BaseExecutor**, typename **Validator**>

struct **is_two_way_executor**<*hpx*::*resiliency*::*experimental*::*replay_executor*<*BaseExecutor*, *Validator*>> : public *std*::true_type

template<typename **BaseExecutor**, typename **Validator**>

struct **is_bulk_two_way_executor**<*hpx*::*resiliency*::*experimental*::*replay_executor*<*BaseExecutor*, *Validator*>> : public *std*::true_type

namespace **resiliency**

namespace **experimental**

Functions

template<executor_any **BaseExecutor**, typename **Validate**>

replay_executor<*BaseExecutor*, *std*::decay_t<*Validate*>> **make_replay_executor**(*BaseExecutor* &exec, *std*::size_t n, *Validate* &&validate)

template<executor_any **BaseExecutor**>

replay_executor<*BaseExecutor*, detail::replay_validator> **make_replay_executor**(*BaseExecutor* &exec, *std*::size_t n)

template<typename **BaseExecutor**, typename **Validate**>

class **replay_executor**

Public Types

using **execution_category** = *hpx*::*traits*::executor_execution_category_t<*BaseExecutor*>

using **executor_parameters_type** = *hpx*::*traits*::executor_parameters_type_t<*BaseExecutor*>

template<typename **Result**>

using **future_type** = *hpx*::*traits*::executor_future_t<*BaseExecutor*, *Result*>

Public Functions

template<typename **BaseExecutor_**, typename **F**>

inline explicit **replay_executor**(*BaseExecutor_* &&exec, *std*::size_t n, *F* &&f)

inline bool **operator==**(*replay_executor* const &rhs) const noexcept inline bool **operator!=**(*replay_executor* const &rhs) const noexcept

inline constexpr *replay_executor* const &**context**() const noexcept

template<typename **F**, typename ...**Ts**>

inline decltype(auto) **async_execute**(*F* &&f, *Ts*&&... ts) const

template<typename **F**, typename **S**, typename ...**Ts**>

inline decltype(auto) **bulk_async_execute**(*F* &&f, *S* const &shape, *Ts*&&... ts) const

inline *BaseExecutor* const &**get_executor**() const

inline *std*::size_t **get_replay_count**() const

inline *Validate* const &**get_validator**() const

template<typename **Tag**, typename ...**Args**>

inline auto **query**(*Tag* tag, *Args*&&... args) const -> decltype(*replay_executor*<*BaseExecutor*, *Validate*>(*std*::declval<*Tag*>()(*std*::declval<*BaseExecutor*>(), *std*::forward<*Args*>(*args*)...), *std*::declval<*std*::size_t>(), *std*::declval<*Validate*>()))

Public Static Attributes

static constexpr int **num_spread** = 4

static constexpr int **num_tasks** = 128

Private Members

BaseExecutor **exec_**

std::size_t **replay_count_**

Validate **validator_**

hpx/resiliency/async_replicate_executor.hpp

Defined in header hpx/resiliency/async_replicate_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **resiliency**

namespace **experimental**

Functions

template<typename **Executor**, typename **Vote**, typename **Pred**, typename **F**, typename ...**Ts**>

decltype(auto) **hpx_invoke**(*async_replicate_vote_validate_t*, *Executor* &&exec, *std*::size_t n, *Vote* &&vote, *Pred* &&pred, *F* &&f, *Ts*&&... ts)

ADL hook for async_replicate_vote_validate CPO with executor.

Asynchronously launches *f* exactly *n* times on *exec*, validates results with *pred*, then runs valid results through *vote*.

Parameters

- **tag** – CPO tag (*async_replicate_vote_validate_t*).
- **exec** – Executor to run *f* on.
- **n** – Number of replicated invocations.
- **vote** – Voting function selecting among valid results.
- **pred** – Predicate validating each invocation result.
- **f** – Callable to invoke asynchronously.
- **ts** – Arguments forwarded to *f*.

Returns

future with the voted-upon result.

template<typename **Executor**, typename **Vote**, typename **F**, typename ...**Ts**>

decltype(auto) **hpx_invoke**(*async_replicate_vote_t*, *Executor* &&exec, *std*::size_t n, *Vote* &&vote, *F* &&f, *Ts*&&... ts)

ADL hook for async_replicate_vote CPO with executor.

Asynchronously launches *f* exactly *n* times on *exec*, runs valid results through *vote*. Validation uses the default exception check.

Parameters

- **tag** – CPO tag (*async_replicate_vote_t*).
- **exec** – Executor to run *f* on.
- **n** – Number of replicated invocations.
- **vote** – Voting function selecting among valid results.
- **f** – Callable to invoke asynchronously.
- **ts** – Arguments forwarded to *f*.

Returns

future with the voted-upon result.

template<typename **Executor**, typename **Pred**, typename **F**, typename ...**Ts**>

decltype(auto) **hpx_invoke**(*async_replicate_validate_t*, *Executor* &&exec, *std*::size_t n, *Pred* &&pred, *F* &&f, *Ts*&&... ts)

ADL hook for async_replicate_validate CPO with executor.

Asynchronously launches *f* exactly *n* times on *exec*, validates results with *pred*. Returns the first valid result.

Parameters

- **tag** – CPO tag (*async_replicate_validate_t*).
- **exec** – Executor to run *f* on.
- **n** – Number of replicated invocations.
- **pred** – Predicate validating each invocation result.
- **f** – Callable to invoke asynchronously.
- **ts** – Arguments forwarded to *f*.

Returns

future with the first valid result.

template<typename **Executor**, typename **F**, typename ...**Ts**>

decltype(auto) **hpx_invoke**(*async_replicate_t*, *Executor* &&exec, *std*::size_t n, *F* &&f, *Ts*&&... ts)

ADL hook for async_replicate CPO with executor.

Asynchronously launches *f* exactly *n* times on *exec*. Validates by checking for exceptions. Returns the first valid result.

Parameters

- **tag** – CPO tag (*async_replicate_t*).
- **exec** – Executor to run *f* on.
- **n** – Number of replicated invocations.
- **f** – Callable to invoke asynchronously.
- **ts** – Arguments forwarded to *f*.

Returns

future with the first non-throwing result.

hpx/resiliency/async_replicate.hpp

Defined in header hpx/resiliency/async_replicate.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **resiliency**

namespace **experimental**

Functions

template<typename **Vote**, typename **Pred**, typename **F**, typename ...**Ts**>

hpx::*future*<*hpx*::*util*::detail::invoke_deferred_result_t<*F*, *Ts*...>> **hpx_invoke**(*async_replicate_vote_validate_t*, *std*::size_t n, *Vote* &&vote, *Pred* &&pred, *F* &&f, *Ts*&&... ts)

ADL hook for async_replicate_vote_validate CPO.

Asynchronously launches *f* exactly *n* times, validates results with *pred*, then runs valid results through *vote*.

Parameters

- **tag** – CPO tag (*async_replicate_vote_validate_t*).
- **n** – Number of replicated invocations.
- **vote** – Voting function selecting among valid results.
- **pred** – Predicate validating each invocation result.
- **f** – Callable to invoke asynchronously.
- **ts** – Arguments forwarded to *f*.

Returns

future with the voted-upon result.

template<typename **Vote**, typename **F**, typename ...**Ts**>

hpx::*future*<*hpx*::*util*::detail::invoke_deferred_result_t<*F*, *Ts*...>> **hpx_invoke**(*async_replicate_vote_t*, *std*::size_t n, *Vote* &&vote, *F* &&f, *Ts*&&... ts)

ADL hook for async_replicate_vote CPO.

Asynchronously launches *f* exactly *n* times, runs valid results through *vote*. Validation uses the default exception check.

Parameters

- **tag** – CPO tag (*async_replicate_vote_t*).
- **n** – Number of replicated invocations.
- **vote** – Voting function selecting among valid results.
- **f** – Callable to invoke asynchronously.
- **ts** – Arguments forwarded to *f*.

Returns

future with the voted-upon result.

template<typename **Pred**, typename **F**, typename ...**Ts**>

hpx::*future*<*hpx*::*util*::detail::invoke_deferred_result_t<*F*, *Ts*...>> **hpx_invoke**(*async_replicate_validate_t*, *std*::size_t n, *Pred* &&pred, *F* &&f, *Ts*&&... ts)

ADL hook for async_replicate_validate CPO.

Asynchronously launches *f* exactly *n* times, validates results with *pred*. Returns the first valid result.

Parameters

- **tag** – CPO tag (*async_replicate_validate_t*).
- **n** – Number of replicated invocations.
- **pred** – Predicate validating each invocation result.
- **f** – Callable to invoke asynchronously.
- **ts** – Arguments forwarded to *f*.

Returns

future with the first valid result.

template<typename **F**, typename ...**Ts**>

hpx::*future*<*hpx*::*util*::detail::invoke_deferred_result_t<*F*, *Ts*...>> **hpx_invoke**(*async_replicate_t*, *std*::size_t n, *F* &&f, *Ts*&&... ts)

ADL hook for async_replicate CPO.

Asynchronously launches *f* exactly *n* times. Validates by checking for exceptions. Returns the first valid result.

Parameters

- **tag** – CPO tag (*async_replicate_t*).
- **n** – Number of replicated invocations.
- **f** – Callable to invoke asynchronously.
- **ts** – Arguments forwarded to *f*.

Returns

future with the first non-throwing result.

hpx/resiliency/replicate_executor.hpp

Defined in header hpx/resiliency/replicate_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

template<typename **BaseExecutor**, typename **Voter**, typename **Validator**>

struct **is_two_way_executor**<*hpx*::*resiliency*::*experimental*::*replicate_executor*<*BaseExecutor*, *Voter*, *Validator*>> : public *std*::true_type

template<typename **BaseExecutor**, typename **Voter**, typename **Validator**>

struct **is_bulk_two_way_executor**<*hpx*::*resiliency*::*experimental*::*replicate_executor*<*BaseExecutor*, *Voter*, *Validator*>> : public *std*::true_type

namespace **resiliency**

namespace **experimental**

Functions

template<executor_any **BaseExecutor**, typename **Voter**, typename **Validate**>

replicate_executor<*BaseExecutor*, *std*::decay_t<*Voter*>, *std*::decay_t<*Validate*>> **make_replicate_executor**(*BaseExecutor* &exec, *std*::size_t n, *Voter* &&voter, *Validate* &&validate)

template<executor_any **BaseExecutor**, typename **Validate**>

replicate_executor<*BaseExecutor*, detail::replicate_voter, *std*::decay_t<*Validate*>> **make_replicate_executor**(*BaseExecutor* &exec, *std*::size_t n, *Validate* &&validate)

template<executor_any **BaseExecutor**>

replicate_executor<*BaseExecutor*, detail::replicate_voter, detail::replicate_validator> **make_replicate_executor**(*BaseExecutor* &exec, *std*::size_t n)

template<typename **BaseExecutor**, typename **Vote**, typename **Validate**>

class **replicate_executor**

Public Types

using **execution_category** = *hpx*::*traits*::executor_execution_category_t<*BaseExecutor*>

using **executor_parameters_type** = *hpx*::*traits*::executor_parameters_type_t<*BaseExecutor*>

template<typename **Result**>

using **future_type** = *hpx*::*traits*::executor_future_t<*BaseExecutor*, *Result*>

Public Functions

template<typename **BaseExecutor_**, typename **V**, typename **F**>

inline explicit **replicate_executor**(*BaseExecutor_* &&exec, *std*::size_t n, *V* &&v, *F* &&f)

inline bool **operator==**(*replicate_executor* const &rhs) const noexcept

inline bool **operator!=**(*replicate_executor* const &rhs) const noexcept

inline constexpr *replicate_executor* const &**context**() const noexcept

template<typename **F**, typename ...**Ts**>

inline decltype(auto) **async_execute**(*F* &&f, *Ts*&&... ts) const

template<typename **F**, typename **S**, typename ...**Ts**>

inline decltype(auto) **bulk_async_execute**(*F* &&f, *S* const &shape, *Ts*&&... ts) const

inline *BaseExecutor* const &**get_executor**() const

inline *std*::size_t **get_replicate_count**() const

inline *Vote* const &**get_voter**() const

inline *Validate* const &**get_validator**() const

template<typename **Tag**, typename ...**Args**>

inline auto **query**(*Tag* tag, *Args*&&... args) const -> decltype(*replicate_executor*<*BaseExecutor*, *Vote*, *Validate*>(*std*::declval<*Tag*>()(*std*::declval<*BaseExecutor*>(), *std*::forward<*Args*>(*args*)...), *std*::declval<*std*::size_t>(), *std*::declval<*Vote*>(), *std*::declval<*Validate*>()))

Public Static Attributes

static constexpr int **num_spread** = 4

static constexpr int **num_tasks** = 128

Private Members

BaseExecutor **exec_**

std::size_t **replicate_count_**

Vote **voter_**

Validate **validator_**

hpx/resiliency/async_replay_executor.hpp

Defined in header hpx/resiliency/async_replay_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **resiliency**

namespace **experimental**

Functions

template<typename **Executor**, typename **Pred**, typename **F**, typename ...**Ts**>

decltype(auto) **hpx_invoke**(*async_replay_validate_t*, *Executor* &&exec, *std*::size_t n, *Pred* &&pred, *F* &&f, *Ts*&&... ts)

ADL hook for async_replay_validate CPO with executor.

Asynchronously launches *f* on *exec*, validating results with *pred*. Repeats on error up to *n* times (aborts on abort_replay_exception).

Parameters

- **tag** – CPO tag (*async_replay_validate_t*).
- **exec** – Executor to run *f* on.
- **n** – Maximum number of retry attempts.
- **pred** – Predicate validating each invocation result.
- **f** – Callable to invoke asynchronously.
- **ts** – Arguments forwarded to *f*.

Returns

future with the first valid result of *f*.

template<typename **Executor**, typename **F**, typename ...**Ts**>

decltype(auto) **hpx_invoke**(*async_replay_t*, *Executor* &&exec, *std*::size_t n, *F* &&f, *Ts*&&... ts)

ADL hook for async_replay CPO with executor.

Asynchronously launches *f* on *exec*, repeating on error up to *n* times (aborts on abort_replay_exception).

Parameters

- **tag** – CPO tag (*async_replay_t*).
- **exec** – Executor to run *f* on.
- **n** – Maximum number of retry attempts.
- **f** – Callable to invoke asynchronously.
- **ts** – Arguments forwarded to *f*.

Returns

future with the first successful result of *f*.

hpx/resiliency/async_replay.hpp

Defined in header hpx/resiliency/async_replay.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **resiliency**

namespace **experimental**

Functions

template<typename **Pred**, typename **F**, typename ...**Ts**>

hpx::*future*<*hpx*::*util*::detail::invoke_deferred_result_t<*F*, *Ts*...>> **hpx_invoke**(*async_replay_validate_t*, *std*::size_t n, *Pred* &&pred, *F* &&f, *Ts*&&... ts)

ADL hook for async_replay_validate CPO.

Asynchronously launches *f*, validating results with *pred*. Repeats on error up to *n* times (aborts on abort_replay_exception).

Parameters

- **tag** – CPO tag (*async_replay_validate_t*).
- **n** – Maximum number of retry attempts.
- **pred** – Predicate validating each invocation result.
- **f** – Callable to invoke asynchronously.
- **ts** – Arguments forwarded to *f*.

Returns

future with the first valid result of *f*.

template<typename **F**, typename ...**Ts**>

hpx::*future*<*hpx*::*util*::detail::invoke_deferred_result_t<*F*, *Ts*...>> **hpx_invoke**(*async_replay_t*, *std*::size_t n, *F* &&f, *Ts*&&... ts)

ADL hook for async_replay CPO.

Asynchronously launches *f*, repeating on error up to *n* times (aborts on abort_replay_exception).

Parameters

- **tag** – CPO tag (*async_replay_t*).
- **n** – Maximum number of retry attempts.
- **f** – Callable to invoke asynchronously.
- **ts** – Arguments forwarded to *f*.

Returns

future with the first successful result of *f*.

hpx/resiliency/resiliency_cpos.hpp

Defined in header hpx/resiliency/resiliency_cpos.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **resiliency**

namespace **experimental**

Variables

hpx::*resiliency*::*experimental*::*async_replay_validate_t* **async_replay_validate**

hpx::*resiliency*::*experimental*::*async_replay_t* **async_replay**

hpx::*resiliency*::*experimental*::*dataflow_replay_validate_t* **dataflow_replay_validate**

hpx::*resiliency*::*experimental*::*dataflow_replay_t* **dataflow_replay**

hpx::*resiliency*::*experimental*::*async_replicate_vote_validate_t* **async_replicate_vote_validate**

hpx::*resiliency*::*experimental*::*async_replicate_vote_t* **async_replicate_vote**

hpx::*resiliency*::*experimental*::*async_replicate_validate_t* **async_replicate_validate**

hpx::*resiliency*::*experimental*::*async_replicate_t* **async_replicate**

hpx::*resiliency*::*experimental*::*dataflow_replicate_vote_validate_t* **dataflow_replicate_vote_validate**

hpx::*resiliency*::*experimental*::*dataflow_replicate_vote_t* **dataflow_replicate_vote**

hpx::*resiliency*::*experimental*::*dataflow_replicate_validate_t* **dataflow_replicate_validate** *hpx*::*resiliency*::*experimental*::*dataflow_replicate_t* **dataflow_replicate**

struct **async_replay_validate_t** : public *hpx*::*resiliency*::*experimental*::detail::dispatch_cpo<*async_replay_validate_t*>

#include <resiliency_cpos.hpp> Customization point for asynchronously launching the given function *f*. repeatedly. Verify the result of those invocations using the given predicate *pred*. Repeat launching on error exactly *n* times (except if abort_replay_exception is thrown).

struct **async_replay_t** : public *hpx*::*resiliency*::*experimental*::detail::dispatch_cpo<*async_replay_t*>

#include <resiliency_cpos.hpp> Customization point for asynchronously launching given function *f* repeatedly. Repeat launching on error exactly *n* times (except if abort_replay_exception is thrown).

struct **dataflow_replay_validate_t** : public *hpx*::*resiliency*::*experimental*::detail::tag_deferred<*dataflow_replay_validate_t*, *async_replay_validate_t*>

#include <resiliency_cpos.hpp> Customization point for asynchronously launching the given function *f*. repeatedly. Verify the result of those invocations using the given predicate *pred*. Repeat launching on error exactly *n* times.

Delay the invocation of *f* if any of the arguments to *f* are futures.

struct **dataflow_replay_t** : public *hpx*::*resiliency*::*experimental*::detail::tag_deferred<*dataflow_replay_t*, *async_replay_t*>

#include <resiliency_cpos.hpp> Customization point for asynchronously launching the given function *f*. repeatedly. Repeat launching on error exactly *n* times.

Delay the invocation of *f* if any of the arguments to *f* are futures.

struct **async_replicate_vote_validate_t** : public *hpx*::*resiliency*::*experimental*::detail::dispatch_cpo<*async_replicate_vote_validate_t*>

#include <resiliency_cpos.hpp> Customization point for asynchronously launching the given function *f* exactly *n* times concurrently. Verify the result of those invocations using the given predicate *pred*. Run all the valid results against a user provided voting function. Return the valid output.

struct **async_replicate_vote_t** : public *hpx*::*resiliency*::*experimental*::detail::dispatch_cpo<*async_replicate_vote_t*>

#include <resiliency_cpos.hpp> Customization point for asynchronously launching the given function *f* exactly *n* times concurrently. Verify the result of those invocations using the given predicate *pred*. Run all the valid results against a user provided voting function. Return the valid output.

struct **async_replicate_validate_t** : public *hpx*::*resiliency*::*experimental*::detail::dispatch_cpo<*async_replicate_validate_t*>

#include <resiliency_cpos.hpp> Customization point for asynchronously launching the

given function *f* exactly *n* times concurrently. Verify the result of those invocations using the given predicate *pred*. Return the first valid result.

struct **async_replicate_t** : public *hpx*::*resiliency*::*experimental*::detail::dispatch_cpo<*async_replicate_t*>

#include <resiliency_cpos.hpp> Customization point for asynchronously launching the given function *f* exactly *n* times concurrently. Verify the result of those invocations by checking for exception. Return the first valid result.

struct **dataflow_replicate_vote_validate_t** : public *hpx*::*resiliency*::*experimental*::detail::tag_deferred<*dataflow_replicate_vote_validate_t*, *async_replicate_vote_validate_t*>

#include <resiliency_cpos.hpp> Customization point for asynchronously launching the given function *f* exactly *n* times concurrently. Run all the valid results against a user provided voting function. Return the valid output.

Delay the invocation of *f* if any of the arguments to *f* are futures.

struct **dataflow_replicate_vote_t** : public *hpx*::*resiliency*::*experimental*::detail::tag_deferred<*dataflow_replicate_vote_t*, *async_replicate_vote_t*>

#include <resiliency_cpos.hpp> Customization point for asynchronously launching the given function *f* exactly *n* times concurrently. Run all the valid results against a user provided voting function. Return the valid output.

Delay the invocation of *f* if any of the arguments to *f* are futures.

struct **dataflow_replicate_validate_t** : public *hpx*::*resiliency*::*experimental*::detail::tag_deferred<*dataflow_replicate_validate_t*, *async_replicate_validate_t*>

#include <resiliency_cpos.hpp> Customization point for asynchronously launching the given function *f* exactly *n* times concurrently. Verify the result of those invocations using the given predicate *pred*. Return the first valid result.

Delay the invocation of *f* if any of the arguments to *f* are futures.

struct **dataflow_replicate_t** : public *hpx*::*resiliency*::*experimental*::detail::tag_deferred<*dataflow_replicate_t*, *async_replicate_t*>

#include <resiliency_cpos.hpp> Customization point for asynchronously launching the given function *f* exactly *n* times concurrently. Return the first valid result.

Delay the invocation of *f* if any of the arguments to *f* are futures.

runtime_configuration

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/runtime_configuration/runtime_configuration.hpp

Defined in header hpx/runtime_configuration/runtime_configuration.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

class **runtime_configuration** : public section

Public Functions

runtime_configuration(char const *argv0, *runtime_mode* mode, *std*::vector<*std*::string> extra_static_ini_defs = { })

void **reconfigure**(*std*::string ini_file)

void **reconfigure**(*std*::vector<*std*::string> ini_defs)

std::vector<*std*::shared_ptr<*plugins*::*plugin_registry_base*>> **load_modules**(*std*::vector<*std*::shared_ptr<*components*::*component_registr* &component_registries)

void **load_components_static**(*std*::vector<*components*::static_factory_load_data_type> const &static_modules)

agas::service_mode **get_agas_service_mode**() const

std::uint32_t **get_num_localities**() const void **set_num_localities**(*std*::uint32_t)

bool **enable_networking**() const

std::uint32_t **get_first_used_core**() const

void **set_first_used_core**(*std*::uint32_t)

std::size_t **get_ipc_data_buffer_cache_size**() const

std::size_t **get_agas_local_cache_size**(*std*::size_t dflt = HPX_AGAS_LOCAL_CACHE_SIZE) const

bool **get_agas_caching_mode**() const

bool **get_agas_range_caching_mode**() const

std::size_t **get_agas_max_pending_refcnt_requests**() const

bool **load_application_configuration**(char const *filename, *error_code* &ec = *throws*)

bool **get_itt_notify_mode**() const

bool **enable_lock_detection**() const

bool **enable_minimal_deadlock_detection**() const

bool **enable_spinlock_deadlock_detection**() const *std*::size_t **get_spinlock_deadlock_detection_limit**() const

std::size_t **trace_depth**() const

std::size_t **get_os_thread_count**() const

std::string **get_cmd_line**() const

inline *std*::ptrdiff_t **get_default_stack_size**() const

std::ptrdiff_t **get_stack_size**(*threads*::*thread_stacksize* stacksize) const

std::size_t **get_thread_pool_size**(char const *poolname) const

std::string **get_endian_out**() const

std::uint64_t **get_max_inbound_message_size**(*std*::string const &type) const

std::uint64_t **get_max_outbound_message_size**(*std*::string const &type) const

inline *std*::map<*std*::string, *hpx*::*util*::plugin::dll> &**modules**()

bool **tolerate_node_faults**()

Check whether fault tolerance for node/locality faults is enabled via the "enable_fault_tolerance" configuration entry in the "hpx" section.

Returns

true if fault tolerance is enabled, false otherwise.

inline void **tolerate_node_faults**(bool const value) const

Set whether this runtime instance should tolerate node faults.

This controls whether the runtime is configured to continue operating in the presence of locality failures (e.g. forced or unexpected disconnection of remote localities) rather than treating such failures as fatal errors.

Parameters

value – Set to `true` to enable fault tolerance for node failures, or `false` to require all localities to remain connected for normal operation.

Public Members

runtime_mode **mode_**

Private Functions

std::ptrdiff_t **init_stack_size**(char const *entryname, char const *defaultvaluestr, *std*::ptrdiff_t defaultvalue) const

std::ptrdiff_t **init_small_stack_size**() const

std::ptrdiff_t **init_medium_stack_size**() const

std::ptrdiff_t **init_large_stack_size**() const

std::ptrdiff_t **init_huge_stack_size**() const

void **pre_initialize_ini**()

void **post_initialize_ini**(*std*::string &hpx_ini_file, *std*::vector<*std*::string> const &cmdline_ini_defs)

void **pre_initialize_logging_ini**()

void **reconfigure**()

void **load_component_paths**(*std*::vector<*std*::shared_ptr<*plugins*::*plugin_registry_base*>> &plugin_registries, *std*::vector<*std*::shared_ptr<*components*::*component_registry_base*>> &component_registries, *std*::string const &component_base_paths, *std*::string const &component_path_suffixes, *std*::set<*std*::string> &component_paths, *std*::map<*std*::string, *filesystem*::path> &basenames)

void **load_component_path**(*std*::vector<*std*::shared_ptr<*plugins*::*plugin_registry_base*>> &plugin_registries, *std*::vector<*std*::shared_ptr<*components*::*component_registry_base*>> &component_registries, *std*::string const &path, *std*::set<*std*::string> &component_paths, *std*::map<*std*::string, *filesystem*::path> &basenames)

Private Members

std::string **hpx_ini_file**

std::vector<*std*::string> **cmdline_ini_defs**

std::vector<*std*::string> **extra_static_ini_defs**

mutable *std*::uint32_t **num_localities**

mutable *std*::uint32_t **num_os_threads**

std::ptrdiff_t **small_stacksize**

std::ptrdiff_t **medium_stacksize**

std::ptrdiff_t **large_stacksize**

std::ptrdiff_t **huge_stacksize**

bool **need_to_call_pre_initialize**

mutable bool **need_to_initialize_tolerate_node_faults** mutable bool **tolerate_node_faults_value**

std::map<*std*::string, *hpx*::*util*::plugin::dll> **modules_**

hpx::runtime_mode

Defined in header hpx/init.hpp[921].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

enum class *hpx*::**runtime_mode** : *std*::int8_t

A HPX runtime can be executed in two different modes: console mode and worker mode.

Values:

enumerator **invalid**

enumerator **console**

The runtime is the console locality.

enumerator **worker**

The runtime is a worker locality.

enumerator **connect**

The runtime is a worker locality connecting late

enumerator **local**

The runtime is fully local.

enumerator **default_**

The runtime mode will be determined based on the command line arguments

enumerator **last**

[921] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/init_runtime/include/hpx/init.hpp

hpx::components::component_commandline_base

Defined in header hpx/components.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

struct **component_commandline_base**

The component_commandline_base has to be used as a base class for all component command-line line handling registries.

Subclassed by hpx::components::component_commandline

HPX_REGISTER_COMPONENT_MODULE

Defined in header hpx/components.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

Warning

doxygenfunction: Cannot find function "HPX_REGISTER_COMPONENT_MODULE" in doxygen xml output for project "hpx" from directory: ../hpx_autodoc

hpx/runtime_configuration/plugin_registry_base.hpp

Defined in header hpx/runtime_configuration/plugin_registry_base.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **plugins**

struct **plugin_registry_base**

#include <plugin_registry_base.hpp> The plugin_registry_base has to be used as a base class for all plugin registries.

Public Functions

virtual **~plugin_registry_base**() = default

virtual bool **get_plugin_info**(*std*::vector<*std*::string> &fillini) = 0

Return the configuration information for any plugin implemented by this module

Parameters

fillini – [in, out] The module is expected to fill this vector with the ini-information (one line per vector element) for all plugins implemented in this module.

Returns

Returns *true* if the parameter *fillini* has been successfully initialized with the registry data of all implemented in this module.

inline virtual void **init**(int*, char***, *util*::*runtime_configuration*&)

hpx/runtime_configuration/component_registry_base.hpp

Defined in header hpx/runtime_configuration/component_registry_base.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

struct **component_registry_base**

#include <component_registry_base.hpp> The component_registry_base has to be used as a base class for all component registries.

Public Functions

virtual **~component_registry_base**() = default

virtual bool **get_component_info**(*std*::vector<*std*::string> &fillini, *std*::string const &filepath, bool is_static = false) = 0

Return the ini-information for all contained components.

Parameters

- **fillini** – [in, out] The module is expected to fill this vector with the ini-information (one line per vector element) for all components implemented in this module.
- **filepath** – [in]
- **is_static** – [in]

Returns

Returns *true* if the parameter *fillini* has been successfully initialized with the registry data of all implemented in this module.

virtual void **register_component_type**() = 0

Register the component type represented by this component.

runtime_local

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::get_locality_id

Defined in header hpx/runtime.hpp[922].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

std::uint32_t **hpx::get_locality_id**(*error_code* &ec = *throws*)

Return the number of the locality this function is being called from.

This function returns the id of the current locality.

> **Note**
>
> The returned value is zero based and its maximum value is smaller than the overall number of localities the current application is running on (as returned by get_num_localities()).

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of *hpx::exception*.

> **Note**
>
> This function needs to be executed on a HPX-thread. It will fail otherwise (it will return -1).

Parameters

ec – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

hpx/runtime_local/get_os_thread_count.hpp

Defined in header hpx/runtime_local/get_os_thread_count.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

[922] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

Functions

std::size_t **get_os_thread_count**()

Return the number of OS-threads running in the runtime instance the current HPX-thread is associated with.

std::size_t **get_os_thread_count**(*threads*::executor const &exec)

Return the number of worker OS- threads used by the given executor to execute HPX threads.

This function returns the number of cores used to execute HPX threads for the given executor. If the function is called while no HPX runtime system is active, it will return zero. If the executor is not valid, this function will fall back to retrieving the number of OS threads used by HPX.

Parameters

exec – [in] The executor to be used.

namespace **threads**

hpx::get_locality_name

Defined in header hpx/runtime.hpp[923].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

std::string *hpx*::**get_locality_name**()

Return the name of the locality this function is called on.

This function returns the name for the locality on which this function is called.

> **See also**
>
> *future<std::string> get_locality_name(hpx::id_type const& id)*

Returns

This function returns the name for the locality on which the function is called. The name is retrieved from the underlying networking layer and may be different for different parcelports.

future<*std*::string> *hpx*::**get_locality_name**(*hpx*::id_type const &id)

Return the name of the referenced locality.

This function returns a future referring to the name for the locality of the given id.

[923] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

See also

std::string get_locality_name()

Parameters
: **id** – [in] The global id of the locality for which the name should be retrieved

Returns
: This function returns the name for the locality of the given id. The name is retrieved from the underlying networking layer and may be different for different parcel ports.

hpx/runtime_local/thread_mapper.hpp

Defined in header hpx/runtime_local/thread_mapper.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

class **thread_mapper**

Public Types

using **callback_type** = detail::thread_mapper_callback_type

Public Functions

thread_mapper(*thread_mapper* const&) = delete

thread_mapper(*thread_mapper*&&) = delete

thread_mapper &**operator=**(*thread_mapper* const&) = delete

thread_mapper &**operator=**(*thread_mapper*&&) = delete

thread_mapper()

~thread_mapper()

std::uint32_t **register_thread**(char const *name, runtime_local::os_thread_type type)

bool **unregister_thread**()

std::uint32_t **get_thread_index**(*std*::string const &label) const

std::uint32_t **get_thread_count**() const

bool **register_callback**(*std*::uint32_t tix, *callback_type* const&)

bool **revoke_callback**(*std*::uint32_t tix)

std::thread::id **get_thread_id**(*std*::uint32_t tix) const

std::uint64_t **get_thread_native_handle**(*std*::uint32_t tix) const

std::string const &**get_thread_label**(*std*::uint32_t tix) const

runtime_local::os_thread_type **get_thread_type**(*std*::uint32_t tix) const

bool **enumerate_os_threads**(*hpx*::*function*<bool(os_thread_data const&)> const &f) const

os_thread_data **get_os_thread_data**(*std*::string const &label) const

Public Static Attributes

static constexpr *std*::uint32_t **invalid_index** = static_cast<*std*::uint32_t>(-1)

static constexpr *std*::uint64_t **invalid_tid** = static_cast<*std*::uint64_t>(-1)

Private Types

using **mutex_type** = *hpx*::*spinlock*

using **thread_map_type** = *std*::vector<detail::os_thread_data>

using **label_map_type** = *std*::map<*std*::string, *std*::size_t>

Private Members

mutable *mutex_type* **mtx_** = *mutex_type*("thread_mapper")

thread_map_type **thread_map_**

label_map_type **label_map_**

hpx/runtime_local/custom_exception_info.hpp

Defined in header hpx/runtime_local/custom_exception_info.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

std::string **diagnostic_information**(exception_info const &xi)

Extract the diagnostic information embedded in the given exception and return a string holding a formatted message.

The function *hpx::diagnostic_information* can be used to extract all diagnostic information stored in the given exception instance as a formatted string. This simplifies debug output as it composes the diagnostics into one, easy to use function call. This includes the name of the source file and line number, the sequence number of the OS-thread and the HPX-thread id, the locality id and the stack backtrace of the point where the original exception was thrown.

> **See also**
>
> *hpx::get_error_locality_id()*, *hpx::get_error_host_name()*, *hpx::get_error_process_id()*, *hpx::get_error_function_name()*, *hpx::get_error_file_name()*, *hpx::get_error_line_number()*, *hpx::get_error_os_thread()*, *hpx::get_error_thread_id()*, *hpx::get_error_thread_description()*, *hpx::get_error()*, *hpx::get_error_backtrace()*, *hpx::get_error_env()*, *hpx::get_error_what()*, *hpx::get_error_config()*, *hpx::get_error_state()*

Parameters

`xi` – The parameter `e` will be inspected for all diagnostic information elements which have been stored at the point where the exception was thrown. This parameter can be one of the following types: *hpx::exception_info*, *hpx::error_code*, *std::exception*, or *std::exception_ptr*.

Throws

`std::bad_alloc` – (if any of the required allocation operations fail)

Returns

The formatted string holding all the available diagnostic information stored in the given exception instance.

std::string **default_diagnostic_information**(*std*::exception_ptr const &e)

std::uint32_t **get_error_locality_id**(*hpx*::exception_info const &xi) noexcept

Return the locality id where the exception was thrown.

The function *hpx::get_error_locality_id* can be used to extract the diagnostic information element representing the locality id as stored in the given exception instance.

> **See also**

hpx::diagnostic_information(), *hpx::get_error_host_name()*, *hpx::get_error_process_id()*, *hpx::get_error_function_name()*, *hpx::get_error_file_name()*, *hpx::get_error_line_number()*, *hpx::get_error_os_thread()*, *hpx::get_error_thread_id()*, *hpx::get_error_thread_description()*, *hpx::get_error()*, *hpx::get_error_backtrace()*, *hpx::get_error_env()*, *hpx::get_error_what()*, *hpx::get_error_config()*, *hpx::get_error_state()*

Parameters

`xi` – The parameter `e` will be inspected for the requested diagnostic information elements which have been stored at the point where the exception was thrown. This parameter can be one of the following types: *hpx::exception_info*, *hpx::error_code*, *std::exception*, or *std::exception_ptr*.

Throws

`nothing` –

Returns

The locality id of the locality where the exception was thrown. If the exception instance does not hold this information, the function will return *hpx::naming::invalid_locality_id*.

std::string **`get_error_host_name`**(*hpx*::exception_info const &xi)

Return the hostname of the locality where the exception was thrown.

The function *hpx::get_error_host_name* can be used to extract the diagnostic information element representing the host name as stored in the given exception instance.

See also

hpx::diagnostic_information() *hpx::get_error_process_id()*, *hpx::get_error_function_name()*, *hpx::get_error_file_name()*, *hpx::get_error_line_number()*, *hpx::get_error_os_thread()*, *hpx::get_error_thread_id()*, *hpx::get_error_thread_description()*, *hpx::get_error()* *hpx::get_error_backtrace()*, *hpx::get_error_env()*, *hpx::get_error_what()*, *hpx::get_error_config()*, *hpx::get_error_state()*

Parameters

`xi` – The parameter `e` will be inspected for the requested diagnostic information elements which have been stored at the point where the exception was thrown. This parameter can be one of the following types: *hpx::exception_info*, *hpx::error_code*, *std::exception*, or *std::exception_ptr*.

Throws

`std::bad_alloc` – (if one of the required allocations fails)

Returns

The hostname of the locality where the exception was thrown. If the exception instance does not hold this information, the function will return and empty string.

std::int64_t **get_error_process_id**(*hpx*::exception_info const &xi) noexcept

Return the (operating system) process id of the locality where the exception was thrown.

The function *hpx::get_error_process_id* can be used to extract the diagnostic information element representing the process id as stored in the given exception instance.

See also

hpx::diagnostic_information(), *hpx::get_error_host_name()*, *hpx::get_error_function_name()*, *hpx::get_error_file_name()*, *hpx::get_error_line_number()*, *hpx::get_error_os_thread()*, *hpx::get_error_thread_id()*, *hpx::get_error_thread_description()*, *hpx::get_error()*, *hpx::get_error_backtrace()*, *hpx::get_error_env()*, *hpx::get_error_what()*, *hpx::get_error_config()*, *hpx::get_error_state()*

Parameters

`xi` – The parameter `e` will be inspected for the requested diagnostic information elements which have been stored at the point where the exception was thrown. This parameter can be one of the following types: *hpx::exception_info*, *hpx::error_code*, *std::exception*, or *std::exception_ptr*.

Throws

`nothing` –

Returns

The process id of the OS-process which threw the exception If the exception instance does not hold this information, the function will return 0.

std::string **get_error_env**(*hpx*::exception_info const &xi)

Return the environment of the OS-process at the point the exception was thrown.

The function *hpx::get_error_env* can be used to extract the diagnostic information element representing the environment of the OS-process collected at the point the exception was thrown.

See also

hpx::diagnostic_information(), *hpx::get_error_host_name()*, *hpx::get_error_process_id()*, *hpx::get_error_function_name()*, *hpx::get_error_file_name()*, *hpx::get_error_line_number()*, *hpx::get_error_os_thread()*, *hpx::get_error_thread_id()*, *hpx::get_error_thread_description()*, *hpx::get_error()*, *hpx::get_error_backtrace()*, *hpx::get_error_what()*, *hpx::get_error_config()*, *hpx::get_error_state()*

Parameters

`xi` – The parameter `e` will be inspected for the requested diagnostic information elements which have been stored at the point where the exception was thrown. This parameter can be one of the following types: *hpx::exception_info*, *hpx::error_code*, *std::exception*, or *std::exception_ptr*.

Throws

`std::bad_alloc` – (if one of the required allocations fails)

Returns

The environment from the point the exception was thrown. If the exception instance does not hold this information, the function will return an empty string.

std::string **`get_error_backtrace`**(*hpx*::exception_info const &xi)

Return the stack backtrace from the point the exception was thrown.

The function *hpx::get_error_backtrace* can be used to extract the diagnostic information element representing the stack backtrace collected at the point the exception was thrown.

> **See also**
>
> *hpx::diagnostic_information()*, *hpx::get_error_host_name()*, *hpx::get_error_process_id()*, *hpx::get_error_function_name()*, *hpx::get_error_file_name()*, *hpx::get_error_line_number()*, *hpx::get_error_os_thread()*, *hpx::get_error_thread_id()*, *hpx::get_error_thread_description()*, *hpx::get_error()*, *hpx::get_error_env()*, *hpx::get_error_what()*, *hpx::get_error_config()*, *hpx::get_error_state()*

Parameters

`xi` – The parameter `e` will be inspected for the requested diagnostic information elements which have been stored at the point where the exception was thrown. This parameter can be one of the following types: *hpx::exception_info*, *hpx::error_code*, *std::exception*, or *std::exception_ptr*.

Throws

`std::bad_alloc` – (if one of the required allocations fails)

Returns

The stack back trace from the point the exception was thrown. If the exception instance does not hold this information, the function will return an empty string.

std::size_t **`get_error_os_thread`**(*hpx*::exception_info const &xi) noexcept

Return the sequence number of the OS-thread used to execute HPX-threads from which the exception was thrown.

The function *hpx::get_error_os_thread* can be used to extract the diagnostic information element representing the sequence number of the OS-thread as stored in the given exception instance.

> **See also**
>
> *hpx::diagnostic_information()*, *hpx::get_error_host_name()*, *hpx::get_error_process_id()*, *hpx::get_error_function_name()*,

hpx::get_error_file_name(), *hpx::get_error_line_number()*, *hpx::get_error_thread_id()*, *hpx::get_error_thread_description()*, *hpx::get_error()*, *hpx::get_error_backtrace()*, *hpx::get_error_env()*, *hpx::get_error_what()*, *hpx::get_error_config()*, *hpx::get_error_state()*

Parameters

xi – The parameter e will be inspected for the requested diagnostic information elements which have been stored at the point where the exception was thrown. This parameter can be one of the following types: *hpx::exception_info*, *hpx::error_code*, *std::exception*, or *std::exception_ptr*.

Throws

nothing –

Returns

The sequence number of the OS-thread used to execute the HPX-thread from which the exception was thrown. If the exception instance does not hold this information, the function will return std::size(-1).

std::size_t **get_error_thread_id**(*hpx*::exception_info const &xi) noexcept

Return the unique thread id of the HPX-thread from which the exception was thrown.

The function *hpx::get_error_thread_id* can be used to extract the diagnostic information element representing the HPX-thread id as stored in the given exception instance.

See also

hpx::diagnostic_information(), *hpx::get_error_host_name()*, *hpx::get_error_process_id()*, *hpx::get_error_function_name()*, *hpx::get_error_file_name()*, *hpx::get_error_line_number()*, *hpx::get_error_os_thread()* *hpx::get_error_thread_description()*, *hpx::get_error()*, *hpx::get_error_backtrace()*, *hpx::get_error_env()*, *hpx::get_error_what()*, *hpx::get_error_config()*, *hpx::get_error_state()*

Parameters

xi – The parameter e will be inspected for the requested diagnostic information elements which have been stored at the point where the exception was thrown. This parameter can be one of the following types: *hpx::exception_info*, *hpx::error_code*, *std::exception*, or *std::exception_ptr*.

Throws

nothing –

Returns

The unique thread id of the HPX-thread from which the exception was thrown. If the exception instance does not hold this information, the function will return std::size_t(0).

std::string **get_error_thread_description**(*hpx*::exception_info const &xi)

Return any additionally available thread description of the HPX-thread from which the exception was thrown.

The function *hpx::get_error_thread_description* can be used to extract the diagnostic information element representing the additional thread description as stored in the given exception instance.

> **See also**
>
> *hpx::diagnostic_information()*, *hpx::get_error_host_name()*, *hpx::get_error_process_id()*, *hpx::get_error_function_name()*, *hpx::get_error_file_name()*, *hpx::get_error_line_number()*, *hpx::get_error_os_thread()*, *hpx::get_error_thread_id()*, *hpx::get_error_backtrace()*, *hpx::get_error_env()*, *hpx::get_error()*, *hpx::get_error_state()*, *hpx::get_error_what()*, *hpx::get_error_config()*

Parameters
: `xi` – The parameter `e` will be inspected for the requested diagnostic information elements which have been stored at the point where the exception was thrown. This parameter can be one of the following types: *hpx::exception_info*, *hpx::error_code*, *std::exception*, or *std::exception_ptr*.

Throws
: `std::bad_alloc` – (if one of the required allocations fails)

Returns
: Any additionally available thread description of the HPX-thread from which the exception was thrown. If the exception instance does not hold this information, the function will return an empty string.

std::string **`get_error_config`**(*hpx*::exception_info const &xi)

Return the HPX configuration information point from which the exception was thrown.

The function *hpx::get_error_config* can be used to extract the HPX configuration information element representing the full HPX configuration information as stored in the given exception instance.

> **See also**
>
> *hpx::diagnostic_information()*, *hpx::get_error_host_name()*, *hpx::get_error_process_id()*, *hpx::get_error_function_name()*, *hpx::get_error_file_name()*, *hpx::get_error_line_number()*, *hpx::get_error_os_thread()*, *hpx::get_error_thread_id()*, *hpx::get_error_backtrace()*, *hpx::get_error_env()*, *hpx::get_error()*, *hpx::get_error_state()* *hpx::get_error_what()*, *hpx::get_error_thread_description()*

Parameters
: `xi` – The parameter `e` will be inspected for the requested diagnostic information elements which have been stored at the point where the exception was thrown. This parameter can be one of the following types: *hpx::exception_info*, *hpx::error_code*, *std::exception*, or *std::exception_ptr*.

Throws

`std::bad_alloc` – (if one of the required allocations fails)

Returns

Any additionally available HPX configuration information the point from which the exception was thrown. If the exception instance does not hold this information, the function will return an empty string.

std::string **`get_error_state`**(*hpx*::exception_info const &xi)

Return the HPX runtime state information at which the exception was thrown.

The function *hpx::get_error_state* can be used to extract the HPX runtime state information element representing the state the runtime system is currently in as stored in the given exception instance.

See also

hpx::diagnostic_information(), *hpx::get_error_host_name()*, *hpx::get_error_process_id()*, *hpx::get_error_function_name()*, *hpx::get_error_file_name()*, *hpx::get_error_line_number()*, *hpx::get_error_os_thread()*, *hpx::get_error_thread_id()*, *hpx::get_error_backtrace()*, *hpx::get_error_env()*, *hpx::get_error()*, *hpx::get_error_what()*, *hpx::get_error_thread_description()*

Parameters

`xi` – The parameter `e` will be inspected for the requested diagnostic information elements which have been stored at the point where the exception was thrown. This parameter can be one of the following types: *hpx::exception_info*, *hpx::error_code*, *std::exception*, or *std::exception_ptr*.

Throws

`std::bad_alloc` – (if one of the required allocations fails)

Returns

The point runtime state at the point at which the exception was thrown. If the exception instance does not hold this information, the function will return an empty string.

hpx/runtime_local/runtime_local.hpp

Defined in header hpx/runtime_local/runtime_local.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **`hpx`**

Functions

void **set_error_handlers**(*hpx*::*util*::*runtime_configuration* const &cfg)

class **runtime**

Public Types

using **notification_policy_type** = *threads*::*policies*::callback_notifier

Generate a new notification policy instance for the given thread name prefix

using **hpx_main_function_type** = int()

The *hpx_main_function_type* is the default function type usable as the main HPX thread function.

using **hpx_errorsink_function_type** = void(*std*::uint32_t, *std*::string const&)

Public Functions

virtual *notification_policy_type* **get_notification_policy**(char const *prefix, runtime_local::os_thread_type type)

state **get_state**() const

void **set_state**(state s)

explicit **runtime**(*hpx*::*util*::*runtime_configuration* rtcfg, bool initialize)

Construct a new HPX runtime instance.

virtual **~runtime**()

The destructor makes sure all HPX runtime services are properly shut down before exiting.

void **on_exit**(*hpx*::*function*<void()> const &f)

Manage list of functions to call on exit.

void **starting**()

Manage runtime 'stopped' state.

void **stopping**()

Call all registered on_exit functions.

bool **stopped**() const

This accessor returns whether the runtime instance has been stopped.

hpx::*util*::*runtime_configuration* &**get_config**()

access configuration information

hpx::*util*::*runtime_configuration* const &**get_config**() const

std::size_t **get_instance_number**() const

util::*thread_mapper* &**get_thread_mapper**() const

Return a reference to the internal PAPI thread manager.

threads::*topology* const &**get_topology**() const

virtual int **run**(*hpx*::*function*<*hpx_main_function_type*> const &func)

Run the HPX runtime system, use the given function for the main *thread* and block waiting for all threads to finish.

> **Note**
>
> The parameter *func* is optional. If no function is supplied, the runtime system will simply wait for the shutdown action without explicitly executing any main thread.

Parameters

`func` – [in] This is the main function of an HPX application. It will be scheduled for execution by the thread manager as soon as the runtime has been initialized. This function is expected to expose an interface as defined by the typedef *hpx_main_function_type*. This parameter is optional and defaults to none main thread function, in which case all threads have to be scheduled explicitly.

Returns

This function will return the value as returned as the result of the invocation of the function object given by the parameter `func`.

virtual int **run**()

Run the HPX runtime system, initially use the given number of (OS) threads in the thread-manager and block waiting for all threads to finish.

Returns

This function will always return 0 (zero).

virtual void **rethrow_exception**()

Rethrow any stored exception (to be called after *stop()*)

virtual int **start**(*hpx*::*function*<*hpx_main_function_type*> const &func, bool blocking = false)

Start the runtime system.

Parameters

- **func** – [in] This is the main function of an HPX application. It will be scheduled for execution by the thread manager as soon as the runtime has been initialized. This function is expected to expose an interface as defined by the typedef *hpx_main_function_type*.
- **blocking** – [in] This allows to control whether this call blocks until the runtime system has been stopped. If this parameter is *true* the function runtime::start will call runtime::wait internally.

Returns

If a blocking is a true, this function will return the value as returned as the result of the invocation of the function object given by the parameter `func`. Otherwise, it will return zero.

virtual int **start**(bool blocking = false)

Start the runtime system.

Parameters

blocking – [in] This allows to control whether this call blocks until the runtime system has been stopped. If this parameter is *true* the function runtime::start will call runtime::wait internally .

Returns

If a blocking is a true, this function will return the value as returned as the result of the invocation of the function object given by the parameter `func`. Otherwise, it will return zero.

virtual int **wait**()

Wait for the shutdown action to be executed.

Returns

This function will return the value as returned as the result of the invocation of the function object given by the parameter `func`.

virtual void **stop**(bool blocking = true)

Initiate termination of the runtime system.

Parameters

blocking – [in] This allows to control whether this call blocks until the runtime system has been fully stopped. If this parameter is *false* then this call will initiate the stop action but will return immediately. Use a second call to stop with this parameter set to *true* to wait for all internal work to be completed.

virtual int **suspend**()

Suspend the runtime system.

virtual int **resume**()

Resume the runtime system.

virtual int **finalize**(double)

virtual bool **is_networking_enabled**()

Return true if networking is enabled.

virtual *hpx*::*threads*::*threadmanager* &**get_thread_manager**()

Allow access to the thread manager instance used by the HPX runtime.

virtual *std*::string **here**() const

Returns a string of the locality endpoints (usable in debug output)

virtual bool **report_error**(*std*::size_t num_thread, *std*::exception_ptr const &e, bool terminate_all = true)

Report a non-recoverable error to the runtime system.

Parameters

- **num_thread** – [in] The number of the operating system thread the error has been detected in.
- **e** – [in] This is an instance encapsulating an exception which lead to this function call.
- **terminate_all** – [in] signal whether all localities should be terminated

virtual bool **report_error**(*std*::exception_ptr const &e, bool terminate_all = true)

Report a non-recoverable error to the runtime system.

> **Note**
>
> This function will retrieve the number of the current shepherd thread and forward to the report_error function above.

Parameters

- **e** – [in] This is an instance encapsulating an exception which lead to this function call.
- **terminate_all** – [in] signal whether all localities should be terminated

virtual void **add_pre_startup_function**(*startup_function_type* f)

Add a function to be executed inside a HPX thread before hpx_main but guaranteed to be executed before any startup function registered with *add_startup_function*.

> **Note**
>
> The difference to a startup function is that all pre-startup functions will be (system-wide) executed before any startup function.

Parameters

f – The function 'f' will be called from inside a HPX thread before hpx_main is executed. This is very useful to set up the runtime environment of the application (install performance counters, etc.)

virtual void **add_startup_function**(*startup_function_type* f)

Add a function to be executed inside a HPX thread before hpx_main

Parameters

f – The function 'f' will be called from inside a HPX thread before hpx_main is executed. This is very useful to set up the runtime environment of the application (install performance counters, etc.)

virtual void **add_pre_shutdown_function**(*shutdown_function_type* f)

Add a function to be executed inside a HPX thread during hpx::finalize, but guaranteed before any of the shutdown functions is executed.

> **Note**
>
> The difference to a shutdown function is that all pre-shutdown functions will be (system-wide) executed before any shutdown function.

Parameters

f – The function 'f' will be called from inside a HPX thread while hpx::finalize is executed. This is very useful to tear down the runtime environment of the application (uninstall performance counters, etc.)

virtual void **add_shutdown_function**(*shutdown_function_type* f)

Add a function to be executed inside a HPX thread during hpx::finalize

Parameters

f – The function 'f' will be called from inside a HPX thread while hpx::finalize is executed. This is very useful to tear down the runtime environment of the application (uninstall performance counters, etc.)

virtual *hpx*::*util*::*io_service_pool* ***get_thread_pool**(char const *name)

Access one of the internal thread pools (io_service instances) HPX is using to perform specific tasks. The three possible values for the argument `name` are "main_pool", "io_pool", "parcel_pool", and "timer_pool". For any other argument value the function will return zero.

virtual bool **register_thread**(char const *name, *std*::size_t num = 0, bool service_thread = true, *error_code* &ec = *throws*)

Register an external OS-thread with HPX.

This function should be called from any OS-thread which is external to HPX (not created by HPX), but which needs to access HPX functionality, such as setting a value on a promise or similar.

'main', 'io', 'timer', 'parcel', 'worker'

> **Note**
>
> The function will compose a thread name of the form '<name>-thread#<num>' which is used to register the thread. It is the user's responsibility to ensure that each (composed) thread name is unique. HPX internally uses the following names for the threads it creates, do not reuse those:

> **Note**
>
> This function should be called for each thread exactly once. It will fail if it is called more than once.

Parameters

- **name** – [in] The name to use for thread registration.
- **num** – [in] The sequence number to use for thread registration. The default for this parameter is zero.
- **service_thread** – [in] The thread should be registered as a service thread. The default for this parameter is 'true'. Any service threads will be pinned to cores not currently used by any of the HPX worker threads.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function will return whether the requested operation succeeded or not.

virtual bool **unregister_thread**()

Unregister an external OS-thread with HPX.

This function will unregister any external OS-thread from HPX.

> **Note**
>
> This function should be called for each thread exactly once. It will fail if it is called more than once. It will fail as well if the thread has not been registered before (see *register_thread*).

Returns

This function will return whether the requested operation succeeded or not.

virtual runtime_local::os_thread_data **get_os_thread_data**(*std*::string const &label) const

Access data for a given OS thread that was previously registered by *register_thread*.

virtual bool **enumerate_os_threads**(*hpx*::*function*<bool(runtime_local::os_thread_data const&)> const &f) const

Enumerate all OS threads that have registered with the runtime.

notification_policy_type::on_startstop_type **on_start_func**() const

notification_policy_type::on_startstop_type **on_stop_func**() const *notification_policy_type*::on_error_type **on_error_func**() const

notification_policy_type::on_startstop_type **on_start_func**(*notification_policy_type*::on_startstop_type&&)

notification_policy_type::on_startstop_type **on_stop_func**(*notification_policy_type*::on_startstop_type&&)

notification_policy_type::on_error_type **on_error_func**(*notification_policy_type*::on_error_type&&)

virtual *std*::uint32_t **get_locality_id**(*error_code* &ec) const

virtual *std*::size_t **get_num_worker_threads**() const

virtual *std*::uint32_t **get_num_localities**(*hpx*::*launch*::sync_policy, *error_code* &ec) const

virtual *std*::uint32_t **get_initial_num_localities**() const

virtual *hpx*::*future*<*std*::uint32_t> **get_num_localities**() const

virtual *std*::string **get_locality_name**() const

virtual *std*::uint32_t **assign_cores**(*std*::string const&, *std*::uint32_t)

virtual *std*::uint32_t **assign_cores**()

inline *hpx*::program_options::options_description const &**get_app_options**() const

inline void **set_app_options**(*hpx*::program_options::options_description const &app_options)

Public Static Functions

static *std*::uint64_t **get_system_uptime**()

Return the system uptime measure on the thread executing this call.

Protected Types

using **on_exit_type** = *std*::vector<*hpx*::*function*<void()>>

Protected Functions

explicit **runtime**(*hpx*::*util*::*runtime_configuration* rtcfg)

void **set_notification_policies**(*notification_policy_type* &&notifier, *threads*::detail::network_background_callback_type const &network_background_callback)

void **init**()

Common initialization for different constructors.

void **init_global_data**()

threads::thread_result_type **run_helper**(*hpx*::*function*<*runtime*::*hpx_main_function_type*> const &func, int &result, bool call_startup_functions, void (*handle_print_bind)(*std*::size_t) = nullptr)

void **wait_helper**(*std*::mutex &mtx, *std*::condition_variable &cond, bool &running)

Protected Attributes

on_exit_type **on_exit_functions_**

mutable *std*::mutex **mtx_**

hpx::*util*::*runtime_configuration* **rtcfg_**

long **instance_number_**

std::unique_ptr<*util*::*thread_mapper*> **thread_support_**

threads::*topology* &**topology_**

std::atomic<state> **state_**

notification_policy_type::on_startstop_type **on_start_func_**

notification_policy_type::on_startstop_type **on_stop_func_**

notification_policy_type::on_error_type **on_error_func_**

int **result_**

std::exception_ptr **exception_**

notification_policy_type **main_pool_notifier_**

std::unique_ptr<*util*::*io_service_pool*> **main_pool_**

notification_policy_type **notifier_**

std::unique_ptr<*hpx*::*threads*::*threadmanager*> **thread_manager_**

Protected Static Functions

static void **deinit_global_data**()

Protected Static Attributes

static *std*::atomic<int> **instance_number_counter_**

Private Functions

void **stop_helper**(bool blocking, *std*::condition_variable &cond, *std*::mutex &mtx) const

Helper function to stop the runtime.

Parameters

- **blocking** – [in] This allows to control whether this call blocks until the runtime system has been fully stopped. If this parameter is *false* then this call will initiate the stop action but will return immediately. Use a second call to stop with this parameter set to *true* to wait for all internal work to be completed.
- **cond** –
- **mtx** –

void **deinit_tss_helper**(char const *context, *std*::size_t num) const

void **init_tss_ex**(char const *context, runtime_local::os_thread_type type, *std*::size_t local_thread_num, *std*::size_t global_thread_num, char const *pool_name, char const *postfix, bool service_thread, *error_code* &ec) const

void **init_tss_helper**(char const *context, runtime_local::os_thread_type type, *std*::size_t local_thread_num, *std*::size_t global_thread_num, char const *pool_name, char const *postfix, bool service_thread) const

void **notify_finalize**()

void **wait_finalize**()

void **call_startup_functions**(bool pre_startup)

Private Members

std::list<*startup_function_type*> **pre_startup_functions_**

std::list<*startup_function_type*> **startup_functions_**

std::list<*shutdown_function_type*> **pre_shutdown_functions_**

std::list<*shutdown_function_type*> **shutdown_functions_**

bool **stop_called_**

bool **stop_done_**

std::condition_variable **wait_condition_**

hpx::program_options::options_description **app_options_**

namespace **threads**

Functions

char const ***get_stack_size_name**(*std*::ptrdiff_t size)

Returns the stack size name.

Get the readable string representing the given stack size constant.

Parameters

size – this represents the stack size

std::ptrdiff_t **get_default_stack_size**()

Returns the default stack size.

Get the default stack size in bytes.

std::ptrdiff_t **get_stack_size**(*thread_stacksize* size)

Returns the stack size corresponding to the given stack size enumeration.

Get the stack size corresponding to the given stack size enumeration.

Parameters

size – this represents the stack size

namespace **util**

Functions

bool **retrieve_commandline_arguments**(*hpx*::program_options::options_description const &app_options, *hpx*::program_options::variables_map &vm)

bool **retrieve_commandline_arguments**(*std*::string const &app_name, *hpx*::program_options::variables_map &vm)

hpx::get_initial_num_localities

Defined in header hpx/runtime.hpp[924].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::get_num_localities*

std::uint32_t *hpx*::**get_initial_num_localities**()

Return the number of localities which were registered at startup for the running application.

The function *get_initial_num_localities* returns the number of localities which were connected to the console at application startup.

See also

hpx::find_all_localities, hpx::get_num_localities

Note

[924] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of *hpx::exception*.

hpx::get_num_localities

Defined in header hpx/runtime.hpp[925].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::get_initial_num_localities*

hpx::*future*<*std*::uint32_t> `hpx::get_num_localities()`

Asynchronously return the number of localities which are currently registered for the running application.

The function *get_num_localities* asynchronously returns the number of localities currently connected to the console. The returned future represents the actual result.

See also

hpx::find_all_localities, hpx::get_num_localities

Note

This function will return meaningful results only if called from an HPX-thread. It will return 0 otherwise.

std::uint32_t `hpx::get_num_localities`(*launch*::sync_policy, *error_code* &ec = *throws*)

Return the number of localities which are currently registered for the running application.

The function *get_num_localities* returns the number of localities currently connected to the console.

See also

hpx::find_all_localities, hpx::get_num_localities

[925] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

Note

This function will return meaningful results only if called from an HPX-thread. It will return 0 otherwise.

Note

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of *hpx::exception*.

Parameters

`ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

std::uint32_t `hpx::get_num_localities`(*hpx*::*launch*::sync_policy, *error_code* &ec)

Return the number of localities which are currently registered for the running application.

hpx::*future*<*std*::uint32_t> `hpx::get_num_localities`(*components*::component_type t)

Asynchronously return the number of localities which are currently registered for the running application.

The function *get_num_localities* asynchronously returns the number of localities currently connected to the console which support the creation of the given component type. The returned future represents the actual result.

See also

hpx::find_all_localities, hpx::get_num_localities

Note

This function will return meaningful results only if called from an HPX-thread. It will return 0 otherwise.

Parameters

`t` – The component type for which the number of connected localities should be retrieved.

std::uint32_t `hpx::get_num_localities`(*launch*::sync_policy, *components*::component_type t, *error_code* &ec = *throws*)

Synchronously return the number of localities which are currently registered for the running application.

The function *get_num_localities* returns the number of localities currently connected to the console which support the creation of the given component type. The returned future represents the actual result.

See also

hpx::find_all_localities, hpx::get_num_localities

Note

This function will return meaningful results only if called from an HPX-thread. It will return 0 otherwise.

Parameters

- **t** – The component type for which the number of connected localities should be retrieved.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

std::uint32_t *hpx*::**get_num_localities**(*hpx*::*launch*::sync_policy, *components*::component_type type, *error_code* &ec)

hpx/runtime_local/report_error.hpp

Defined in header hpx/runtime_local/report_error.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

void **report_error**(*std*::size_t num_thread, *std*::exception_ptr const &e)

The function report_error reports the given exception to the console.

void **report_error**(*std*::exception_ptr const &e)

The function report_error reports the given exception to the console.

hpx::get_thread_name

Defined in header hpx/runtime.hpp[926].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

std::string *hpx*::**get_thread_name**()

Return the name of the calling thread.

This function returns the name of the calling thread. This name uniquely identifies the thread in the context of HPX. If the function is called while no HPX runtime system is active, the result will be "<unknown>".

hpx::startup_function_type

Defined in header hpx/runtime.hpp[927].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::register_pre_startup_function*
- *hpx::register_startup_function*

using *hpx*::**startup_function_type** = *hpx*::*move_only_function*<void()>

The type of the function which is registered to be executed as a startup or pre-startup function.

hpx::register_pre_startup_function

Defined in header hpx/runtime.hpp[928].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::startup_function_type*
- *hpx::register_startup_function*

void *hpx*::**register_pre_startup_function**(*startup_function_type* f)

Add a function to be executed by a HPX thread before hpx_main but guaranteed before any startup function is executed (system-wide).

Any of the functions registered with *register_pre_startup_function* are guaranteed to be executed by an HPX thread before any of the registered startup functions are executed (see hpx::register_startup_function()).

[926] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp
[927] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp
[928] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

This function is one of the few API functions which can be called before the runtime system has been fully initialized. It will automatically stage the provided startup function to the runtime system during its initialization (if necessary).

See also

hpx::register_startup_function()

Note

If this function is called while the pre-startup functions are being executed or after that point, it will raise aninvalid_status exception.

Parameters

`f` – [in] The function to be registered to run by an HPX thread as a pre-startup function.

hpx::register_startup_function

Defined in header hpx/runtime.hpp[929].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::startup_function_type*
- *hpx::register_pre_startup_function*

void `hpx::register_startup_function`(*startup_function_type* f)

Add a function to be executed by a HPX thread before hpx_main but guaranteed after any pre-startup function is executed (system-wide).

Any of the functions registered with *register_startup_function* are guaranteed to be executed by an HPX thread after any of the registered pre-startup functions are executed (see: hpx::register_pre_startup_function()), but before *hpx_main* is being called.

This function is one of the few API functions which can be called before the runtime system has been fully initialized. It will automatically stage the provided startup function to the runtime system during its initialization (if necessary).

See also

hpx::register_pre_startup_function()

[929] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

> **Note**
>
> If this function is called while the startup functions are being executed or after that point, it will raise an invalid_status exception.

Parameters

`f` – [in] The function to be registered to run by an HPX thread as a startup function.

hpx::shutdown_function_type

Defined in header hpx/runtime.hpp[930].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::register_pre_shutdown_function*
- *hpx::register_shutdown_function*

using *hpx*::**shutdown_function_type** = *hpx*::*move_only_function*<void()>

The type of the function which is registered to be executed as a shutdown or pre-shutdown function.

hpx::register_pre_shutdown_function

Defined in header hpx/runtime.hpp[931].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::shutdown_function_type*
- *hpx::register_shutdown_function*

void *hpx*::**register_pre_shutdown_function**(*shutdown_function_type* f)

Add a function to be executed by a HPX thread during hpx::finalize() but guaranteed before any shutdown function is executed (system-wide)

Any of the functions registered with *register_pre_shutdown_function* are guaranteed to be executed by an HPX thread during the execution of hpx::finalize() before any of the registered shutdown functions are executed (see: hpx::register_shutdown_function()).

> **See also**
>
> hpx::register_shutdown_function()

[930] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp
[931] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

Note

If this function is called while the pre-shutdown functions are being executed, or after that point, it will raise an invalid_status exception.

Parameters

`f` – [in] The function to be registered to run by an HPX thread as a pre-shutdown function.

hpx::register_shutdown_function

Defined in header hpx/runtime.hpp[932].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::shutdown_function_type*
- *hpx::register_pre_shutdown_function*

void *hpx*::**register_shutdown_function**(*shutdown_function_type* f)

Add a function to be executed by a HPX thread during hpx::finalize() but guaranteed after any pre-shutdown function is executed (system-wide)

Any of the functions registered with *register_shutdown_function* are guaranteed to be executed by an HPX thread during the execution of hpx::finalize() after any of the registered pre-shutdown functions are executed (see: hpx::register_pre_shutdown_function()).

See also

hpx::register_pre_shutdown_function()

Note

If this function is called while the shutdown functions are being executed, or after that point, it will raise an invalid_status exception.

Parameters

`f` – [in] The function to be registered to run by an HPX thread as a shutdown function.

[932] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

hpx::register_thread

Defined in header hpx/runtime.hpp[933].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::unregister_thread*
- *hpx::get_os_thread_data*
- *hpx::enumerate_os_threads*
- *hpx::get_runtime_instance_number*
- *hpx::register_on_exit*
- *hpx::is_starting*
- *hpx::tolerate_node_faults*
- *hpx::is_running*
- *hpx::is_stopped*
- *hpx::is_stopped_or_shutting_down*
- *hpx::get_num_worker_threads*
- *hpx::get_system_uptime*

bool *hpx*::**register_thread**(*runtime* *rt, char const *name, *error_code* &ec = *throws*)

Register the current kernel thread with HPX, this should be done once for each external OS-thread intended to invoke HPX functionality. Calling this function more than once will return false.

hpx::unregister_thread

Defined in header hpx/runtime.hpp[934].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::register_thread*
- *hpx::get_os_thread_data*
- *hpx::enumerate_os_threads*
- *hpx::get_runtime_instance_number*
- *hpx::register_on_exit*
- *hpx::is_starting*
- *hpx::tolerate_node_faults*
- *hpx::is_running*
- *hpx::is_stopped*

[933] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp
[934] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

- *hpx::is_stopped_or_shutting_down*
- *hpx::get_num_worker_threads*
- *hpx::get_system_uptime*

void `hpx::unregister_thread`(*runtime* *rt)

Unregister the thread from HPX, this should be done once in the end before the external thread exists.

hpx::get_os_thread_data

Defined in header hpx/runtime.hpp[935].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::register_thread*
- *hpx::unregister_thread*
- *hpx::enumerate_os_threads*
- *hpx::get_runtime_instance_number*
- *hpx::register_on_exit*
- *hpx::is_starting*
- *hpx::tolerate_node_faults*
- *hpx::is_running*
- *hpx::is_stopped*
- *hpx::is_stopped_or_shutting_down*
- *hpx::get_num_worker_threads*
- *hpx::get_system_uptime*

runtime_local::os_thread_data `hpx::get_os_thread_data`(*std*::string const &label)

Access data for a given OS thread that was previously registered by *register_thread*. This function must be called from a thread that was previously registered with the runtime.

hpx::enumerate_os_threads

Defined in header hpx/runtime.hpp[936].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::register_thread*
- *hpx::unregister_thread*

[935] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp
[936] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

- *hpx::get_os_thread_data*
- *hpx::get_runtime_instance_number*
- *hpx::register_on_exit*
- *hpx::is_starting*
- *hpx::tolerate_node_faults*
- *hpx::is_running*
- *hpx::is_stopped*
- *hpx::is_stopped_or_shutting_down*
- *hpx::get_num_worker_threads*
- *hpx::get_system_uptime*

bool `hpx::enumerate_os_threads`(*hpx*::*function*<bool(os_thread_data const&)> const &f)

Enumerate all OS threads that have registered with the runtime.

hpx::get_runtime_instance_number

Defined in header hpx/runtime.hpp[937].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::register_thread*
- *hpx::unregister_thread*
- *hpx::get_os_thread_data*
- *hpx::enumerate_os_threads*
- *hpx::register_on_exit*
- *hpx::is_starting*
- *hpx::tolerate_node_faults*
- *hpx::is_running*
- *hpx::is_stopped*
- *hpx::is_stopped_or_shutting_down*
- *hpx::get_num_worker_threads*
- *hpx::get_system_uptime*

std::size_t `hpx::get_runtime_instance_number`()

Return the runtime instance number associated with the runtime instance the current thread is running in.

[937] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

hpx::register_on_exit

Defined in header hpx/runtime.hpp[938].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::register_thread*
- *hpx::unregister_thread*
- *hpx::get_os_thread_data*
- *hpx::enumerate_os_threads*
- *hpx::get_runtime_instance_number*
- *hpx::is_starting*
- *hpx::tolerate_node_faults*
- *hpx::is_running*
- *hpx::is_stopped*
- *hpx::is_stopped_or_shutting_down*
- *hpx::get_num_worker_threads*
- *hpx::get_system_uptime*

bool `hpx::register_on_exit`(*hpx*::*function*<void()> const&)

Register a function to be called during system shutdown.

hpx::is_starting

Defined in header hpx/runtime.hpp[939].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::register_thread*
- *hpx::unregister_thread*
- *hpx::get_os_thread_data*
- *hpx::enumerate_os_threads*
- *hpx::get_runtime_instance_number*
- *hpx::register_on_exit*
- *hpx::tolerate_node_faults*
- *hpx::is_running*
- *hpx::is_stopped*
- *hpx::is_stopped_or_shutting_down*

[938] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp
[939] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

- *hpx::get_num_worker_threads*
- *hpx::get_system_uptime*

bool *hpx*::**is_starting**()

Test whether the runtime system is currently being started.

This function returns whether the runtime system is currently being started or not, e.g. whether the current state of the runtime system is *hpx::state::startup*

> **Note**
>
> This function needs to be executed on a HPX-thread. It will return false otherwise.

hpx::tolerate_node_faults

Defined in header hpx/runtime.hpp[940].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::register_thread*
- *hpx::unregister_thread*
- *hpx::get_os_thread_data*
- *hpx::enumerate_os_threads*
- *hpx::get_runtime_instance_number*
- *hpx::register_on_exit*
- *hpx::is_starting*
- *hpx::is_running*
- *hpx::is_stopped*
- *hpx::is_stopped_or_shutting_down*
- *hpx::get_num_worker_threads*
- *hpx::get_system_uptime*

bool *hpx*::**tolerate_node_faults**()

Test if HPX runs in fault-tolerant mode.

This function returns whether the runtime system is running in fault-tolerant mode

[940] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

hpx::is_running

Defined in header hpx/runtime.hpp[941].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::register_thread*
- *hpx::unregister_thread*
- *hpx::get_os_thread_data*
- *hpx::enumerate_os_threads*
- *hpx::get_runtime_instance_number*
- *hpx::register_on_exit*
- *hpx::is_starting*
- *hpx::tolerate_node_faults*
- *hpx::is_stopped*
- *hpx::is_stopped_or_shutting_down*
- *hpx::get_num_worker_threads*
- *hpx::get_system_uptime*

bool `hpx::is_running()`

Test whether the runtime system is currently running.

This function returns whether the runtime system is currently running or not, e.g. whether the current state of the runtime system is *hpx::state::running*

> **Note**
>
> This function needs to be executed on a HPX-thread. It will return false otherwise.

hpx::is_stopped

Defined in header hpx/runtime.hpp[942].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::register_thread*
- *hpx::unregister_thread*
- *hpx::get_os_thread_data*
- *hpx::enumerate_os_threads*
- *hpx::get_runtime_instance_number*

[941] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp
[942] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

- *hpx::register_on_exit*
- *hpx::is_starting*
- *hpx::tolerate_node_faults*
- *hpx::is_running*
- *hpx::is_stopped_or_shutting_down*
- *hpx::get_num_worker_threads*
- *hpx::get_system_uptime*

bool *hpx*::**is_stopped**()

Test whether the runtime system is currently stopped.

This function returns whether the runtime system is currently stopped or not, e.g. whether the current state of the runtime system is *hpx::state::stopped*

> **Note**
>
> This function needs to be executed on a HPX-thread. It will return false otherwise.

hpx::is_stopped_or_shutting_down

Defined in header hpx/runtime.hpp[943].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::register_thread*
- *hpx::unregister_thread*
- *hpx::get_os_thread_data*
- *hpx::enumerate_os_threads*
- *hpx::get_runtime_instance_number*
- *hpx::register_on_exit*
- *hpx::is_starting*
- *hpx::tolerate_node_faults*
- *hpx::is_running*
- *hpx::is_stopped*
- *hpx::get_num_worker_threads*
- *hpx::get_system_uptime*

[943] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

bool *hpx*::**is_stopped_or_shutting_down**()

Test whether the runtime system is currently being shut down.

This function returns whether the runtime system is currently being shut down or not, e.g. whether the current state of the runtime system is *hpx::state::stopped* or *hpx::state::shutdown*

Note

This function needs to be executed on a HPX-thread. It will return false otherwise.

hpx::get_num_worker_threads

Defined in header hpx/runtime.hpp[944].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::register_thread*
- *hpx::unregister_thread*
- *hpx::get_os_thread_data*
- *hpx::enumerate_os_threads*
- *hpx::get_runtime_instance_number*
- *hpx::register_on_exit*
- *hpx::is_starting*
- *hpx::tolerate_node_faults*
- *hpx::is_running*
- *hpx::is_stopped*
- *hpx::is_stopped_or_shutting_down*
- *hpx::get_system_uptime*

std::size_t *hpx*::**get_num_worker_threads**()

Return the number of worker OS- threads used to execute HPX threads.

This function returns the number of OS-threads used to execute HPX threads. If the function is called while no HPX runtime system is active, it will return zero.

[944] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

hpx::get_system_uptime

Defined in header hpx/runtime.hpp[945].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::register_thread*
- *hpx::unregister_thread*
- *hpx::get_os_thread_data*
- *hpx::enumerate_os_threads*
- *hpx::get_runtime_instance_number*
- *hpx::register_on_exit*
- *hpx::is_starting*
- *hpx::tolerate_node_faults*
- *hpx::is_running*
- *hpx::is_stopped*
- *hpx::is_stopped_or_shutting_down*
- *hpx::get_num_worker_threads*

std::uint64_t *hpx*::**get_system_uptime**()

Return the system uptime measure on the thread executing this call.

This function returns the system uptime measured in nanoseconds for the thread executing this call. If the function is called while no HPX runtime system is active, it will return zero.

hpx/runtime_local/thread_pool_helpers.hpp

Defined in header hpx/runtime_local/thread_pool_helpers.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **resource**

[945] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

Functions

std::size_t **get_num_thread_pools**()

Return the number of thread pools currently managed by the *resource_partitioner*

std::size_t **get_num_threads**()

Return the number of threads in all thread pools currently managed by the *resource_partitioner*

std::size_t **get_num_threads**(*std*::string const &pool_name)

Return the number of threads in the given thread pool currently managed by the *resource_partitioner*

std::size_t **get_num_threads**(*std*::size_t pool_index)

Return the number of threads in the given thread pool currently managed by the *resource_partitioner*

std::size_t **get_pool_index**(*std*::string const &pool_name)

Return the internal index of the pool given its name.

std::string const &**get_pool_name**(*std*::size_t pool_index)

Return the name of the pool given its internal index.

threads::*thread_pool_base* &**get_thread_pool**(*std*::string const &pool_name)

Return the name of the pool given its name.

threads::*thread_pool_base* &**get_thread_pool**(*std*::size_t pool_index)

Return the thread pool given its internal index.

bool **pool_exists**(*std*::string const &pool_name)

Return true if the pool with the given name exists.

bool **pool_exists**(*std*::size_t pool_index)

Return true if the pool with the given index exists.

namespace **threads**

Functions

std::int64_t **get_thread_count**(*thread_schedule_state* state = *thread_schedule_state*::*unknown*)

The function *get_thread_count* returns the number of currently known threads.

> **Note**
>
> If state == unknown this function will not only return the number of currently existing threads, but will add the number of registered task descriptions (which have not been converted into threads yet).

Parameters

state – [in] This specifies the thread-state for which the number of threads should be retrieved.

std::int64_t **get_thread_count**(*thread_priority* priority, *thread_schedule_state* state = *thread_schedule_state*::*unknown*)

The function *get_thread_count* returns the number of currently known threads.

> **Note**
>
> If state == unknown this function will not only return the number of currently existing threads, but will add the number of registered task descriptions (which have not been converted into threads yet).

Parameters

- **priority** – [in] This specifies the thread-priority for which the number of threads should be retrieved.
- **state** – [in] This specifies the thread-state for which the number of threads should be retrieved.

std::int64_t **get_idle_core_count**()

The function *get_idle_core_count* returns the number of currently idling threads (cores).

mask_type **get_idle_core_mask**()

The function *get_idle_core_mask* returns a bit-mask representing the currently idling threads (cores).

bool **enumerate_threads**(*hpx*::*function*<bool(thread_id_type)> const &f, *thread_schedule_state* state = *thread_schedule_state*::*unknown*)

The function *enumerate_threads* will invoke the given function *f* for each thread with a matching thread state.

Parameters

- **f** – [in] The function which should be called for each matching thread. Returning 'false' from this function will stop the enumeration process.
- **state** – [in] This specifies the thread-state for which the threads should be enumerated.

hpx/runtime_local/termination_detection.hpp

Defined in header hpx/runtime_local/termination_detection.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **local**

Functions

void **termination_detection**()

Wait for all local HPX threads to complete execution.

This function blocks the calling thread until all HPX worker threads on the current locality have completed their work and become idle. It performs local termination detection by waiting for the thread manager to drain all pending tasks.

Example:

```
#include <hpx/hpx_init.hpp>
#include <hpx/runtime_local/termination_detection.hpp>

int hpx_main()
{
    // Launch some asynchronous work
    for (int i = 0; i < 100; ++i)
    {
        hpx::post([]{ /* do work */ });
    }

    // Wait for all local threads to complete
    hpx::local::termination_detection();

    return hpx::finalize();
}
```

> **Note**
>
> This function should be called from an HPX thread or after the HPX runtime has been initialized.

Throws

hpx::exception – if called when the runtime is not in a valid state (e.g., before initialization or after shutdown).

bool **termination_detection**(*hpx*::chrono::steady_duration const &timeout)

Wait for all local HPX threads to complete with a timeout.

This function blocks the calling thread until either all HPX worker threads have completed their work, or the specified timeout duration has elapsed.

> **See also**
>
> termination_detection()

Example:

```
#include <hpx/hpx_init.hpp>
#include <hpx/runtime_local/termination_detection.hpp>
#include <chrono>

int hpx_main()
{
    // Launch some work
    hpx::post([]{ /* long running task */ });

    // Wait up to 5 seconds
    if (!hpx::local::termination_detection(std::chrono::seconds(5)))
    {
        std::cerr << "Warning: Work did not complete within timeout\n";
        // Implement fallback strategy
    }

    return hpx::finalize();
}
```

Parameters
: **timeout** – Maximum duration to wait for termination. The function will return false if this duration elapses before all threads complete.

Throws
: *hpx::exception* – if called when the runtime is not in a valid state.

Returns
: true if all threads completed before the timeout, false if the timeout elapsed.

bool **termination_detection**(*hpx*::chrono::steady_time_point const &deadline)

Wait for all local HPX threads to complete until a deadline.

This function blocks the calling thread until either all HPX worker threads have completed their work, or the specified deadline time point is reached.

See also

termination_detection(std::chrono::duration<double>)

Example:

```
#include <hpx/hpx_init.hpp>
#include <hpx/runtime_local/termination_detection.hpp>
#include <chrono>

int hpx_main()
{
    auto deadline = std::chrono::steady_clock::now() +
                    std::chrono::seconds(10);

    hpx::post([]{ /* work */ });

    if (!hpx::local::termination_detection(deadline))
    {
        std::cerr << "Deadline reached\n";
    }

    return hpx::finalize();
}
```

Parameters

deadline – Absolute time point when waiting should stop. The function will return false if this time is reached before all threads complete.

Throws

hpx::exception – if called when the runtime is not in a valid state.

Returns

true if all threads completed before the deadline, false if the deadline was reached.

bool **termination_detection**(*hpx*::*stop_token* stop_token, *hpx*::chrono::steady_duration const &timeout = *hpx*::chrono::steady_duration((*hpx*::chrono::steady_clock::duration::max)()))

Wait for all local HPX threads with cancellation support.

This function blocks the calling thread until all HPX worker threads have completed, the timeout elapses, or a stop is requested via the stop_token. This enables cooperative cancellation of the wait operation.

See also

termination_detection(std::chrono::duration<double>)

Example:

```cpp
#include <hpx/hpx_init.hpp>
#include <hpx/runtime_local/termination_detection.hpp>
#include <hpx/stop_token.hpp>
#include <chrono>

int hpx_main()
{
    hpx::stop_source stop_src;

    // In shutdown handler
    hpx::thread shutdown_thread([&stop_src]() {
        hpx::this_thread::sleep_for(std::chrono::seconds(30));
        stop_src.request_stop();
    });

    hpx::post([]{ /* work */ });

    // Wait with cancellation support
    bool completed = hpx::local::termination_detection(
        stop_src.get_token(),
        std::chrono::minutes(1)
    );

    if (!completed)
    {
        std::cerr << "Cancelled or timed out\n";
    }

    shutdown_thread.join();
    return hpx::finalize();
}
```

Parameters

- **`stop_token`** – Token that can be used to request cancellation of the wait operation.
- **`timeout`** – Maximum duration to wait. Defaults to maximum duration (effectively infinite).

Throws

`hpx::exception` – if called when the runtime is not in a valid state.

Returns

true if all threads completed, false if timeout elapsed or stop was requested.

hpx/runtime_local/thread_hooks.hpp

Defined in header hpx/runtime_local/thread_hooks.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Functions

threads::*policies*::callback_notifier::on_startstop_type **get_thread_on_start_func**()

Retrieve the currently installed start handler function. This is a function that will be called by HPX for each newly created thread that is made known to the runtime. HPX stores exactly one such function reference, thus the caller needs to make sure any newly registered start function chains into the previous one (see *register_thread_on_start_func*).

> **Note**
>
> This function can be called before the HPX runtime is initialized.

Returns
The currently installed error handler function.

threads::*policies*::callback_notifier::on_startstop_type **get_thread_on_stop_func**()

Retrieve the currently installed stop handler function. This is a function that will be called by HPX for each newly created thread that is made known to the runtime. HPX stores exactly one such function reference, thus the caller needs to make sure any newly registered stop function chains into the previous one (see *register_thread_on_stop_func*).

> **Note**
>
> This function can be called before the HPX runtime is initialized.

Returns
The currently installed error handler function.

threads::*policies*::callback_notifier::on_error_type **get_thread_on_error_func**()

Retrieve the currently installed error handler function. This is a function that will be called by HPX for each newly created thread that is made known to the runtime. HPX stores exactly one such function reference, thus the caller needs to make sure any newly registered error function chains into the previous one (see *register_thread_on_error_func*).

> **Note**
>
> This function can be called before the HPX runtime is initialized.

Returns
The currently installed error handler function.

threads::*policies*::callback_notifier::on_startstop_type **register_thread_on_start_func**(*threads*::*policies*::callback_notifier::on_startsto
&&f)

Set the currently installed start handler function. This is a function that will be called by HPX for each newly created thread that is made known to the runtime. HPX stores exactly one such function reference, thus the caller needs to make sure any newly registered start function chains into the previous one (see *get_thread_on_start_func*).

> **Note**
>
> This function can be called before the HPX runtime is initialized.

Parameters
`f` – The function to install as the new start handler.

Returns
The previously registered function of this category. It is the user's responsibility to call that function if the callback is invoked by HPX.

threads::*policies*::callback_notifier::on_startstop_type **`register_thread_on_stop_func`**(*threads*::*policies*::callback_notifier::on_startstop_type &&f)

Set the currently installed stop handler function. This is a function that will be called by HPX for each newly created thread that is made known to the runtime. HPX stores exactly one such function reference, thus the caller needs to make sure any newly registered stop function chains into the previous one (see *get_thread_on_stop_func*).

> **Note**
>
> This function can be called before the HPX runtime is initialized.

Parameters
`f` – The function to install as the new stop handler.

Returns
The previously registered function of this category. It is the user's responsibility to call that function if the callback is invoked by HPX.

threads::*policies*::callback_notifier::on_error_type **`register_thread_on_error_func`**(*threads*::*policies*::callback_notifier::on_error_type &&f)

Set the currently installed error handler function. This is a function that will be called by HPX for each newly created thread that is made known to the runtime. HPX stores exactly one such function reference, thus the caller needs to make sure any newly registered error function chains into the previous one (see *get_thread_on_error_func*).

> **Note**
>
> This function can be called before the HPX runtime is initialized.

Parameters
`f` – The function to install as the new error handler.

Returns
The previously registered function of this category. It is the user's responsibility to call that function if the callback is invoked by HPX.

hpx/runtime_local/get_worker_thread_num.hpp

Defined in header hpx/runtime_local/get_worker_thread_num.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/runtime_local/component_startup_shutdown_base.hpp

Defined in header hpx/runtime_local/component_startup_shutdown_base.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

struct **component_startup_shutdown_base**

#include <component_startup_shutdown_base.hpp> The component_startup_shutdown_base has to be used as a base class for all component startup/shutdown registries.

Public Functions

virtual **~component_startup_shutdown_base**() = default

virtual bool **get_startup_function**(*startup_function_type* &startup, bool &pre_startup) = 0

Return any startup function for this component.

Parameters

- **startup** – [in, out] The module is expected to fill this function object with a reference to a startup function. This function will be executed by the runtime system during system startup.
- **pre_startup** – [in, out] Will be set to true if the returned startup function is executed during the first round of calls.

Returns

Returns *true* if the parameter *startup* has been successfully initialized with the startup function.

virtual bool **get_shutdown_function**(*shutdown_function_type* &shutdown, bool &pre_shutdown) = 0

Return any startup function for this component.

Parameters

- **shutdown** – [in, out] The module is expected to fill this function object with a reference to a startup function. This function will be executed by the runtime system during system startup.

- **pre_shutdown** – [in, out] Will be set to true if the returned shutdown function is executed during the first round of calls.

Returns

Returns *true* if the parameter *shutdown* has been successfully initialized with the shutdown function.

hpx/runtime_local/service_executors.hpp

Defined in header hpx/runtime_local/service_executors.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

Enums

enum class **service_executor_type** : *std*::uint8_t

Values:

enumerator **io_thread_pool**

Selects creating a service executor using the I/O pool of threads

enumerator **parcel_thread_pool**

Selects creating a service executor using the parcel pool of threads

enumerator **timer_thread_pool**

Selects creating a service executor using the timer pool of threads

enumerator **main_thread**

Selects creating a service executor using the main thread

struct **service_executor** : public *parallel*::*execution*::detail::service_executor

Subclassed by *hpx::execution::experimental::io_pool_executor*, *hpx::execution::experimental::main_pool_executor*, *hpx::execution::experimental::parcel_pool_executor*, *hpx::execution::experimental::timer_pool_executor*

Public Functions

explicit **service_executor**(*service_executor_type* t, char const *name_suffix = "")

struct **io_pool_executor** : public *hpx::execution::experimental::service_executor*

Public Functions

io_pool_executor()

struct **parcel_pool_executor** : public *hpx::execution::experimental::service_executor*

Public Functions

explicit **parcel_pool_executor**(char const *name_suffix = "-tcp")

struct **timer_pool_executor** : public *hpx::execution::experimental::service_executor*

Public Functions

timer_pool_executor()

struct **main_pool_executor** : public *hpx::execution::experimental::service_executor*

Public Functions

main_pool_executor()

serialization

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/serialization/base_object.hpp

Defined in header hpx/serialization/base_object.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **serialization**

Functions

template<typename **Base**, typename **Derived**>

constexpr *base_object_type*<*Derived*, *Base*> **base_object**(*Derived* &d) noexcept

template<typename **D**, typename **B**>

output_archive &**operator<<**(output_archive &ar, *base_object_type*<*D*, *B*> t)

template<typename **D**, typename **B**>

input_archive &**operator>>**(input_archive &ar, *base_object_type*<*D*, *B*> t)

template<typename **D**, typename **B**>

output_archive &**operator&**(output_archive &ar, *base_object_type*<*D*, *B*> t)

template<typename **D**, typename **B**>

input_archive &**operator&**(input_archive &ar, *base_object_type*<*D*, *B*> t)

template<typename **Derived**, typename **Base**, typename **Enable** = void>

struct **base_object_type**

Public Functions

inline explicit constexpr **base_object_type**(*Derived* &d) noexcept

template<typename **Archive**>

inline void **serialize**(*Archive* &ar, unsigned)

Public Members

Derived &**d_**

template<typename **Derived**, typename **Base**>

struct **base_object_type**<*Derived*, *Base*, *std*::enable_if_t<*hpx*::*traits*::is_intrusive_polymorphic_v<*Derived*>>>

Public Functions

inline explicit constexpr **base_object_type**(*Derived* &d) noexcept

template<typename **Archive**>

inline void **save**(*Archive* &ar, unsigned) const

template<typename **Archive**>

inline void **load**(*Archive* &ar, unsigned)

HPX_SERIALIZATION_SPLIT_MEMBER()

Public Members

Derived &**d_**

synchronization

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::mutex

Defined in header hpx/mutex.hpp[946].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::timed_mutex*

class **mutex**

mutex class is a synchronization primitive that can be used to protect shared data from being simultaneously accessed by multiple threads. *mutex* offers exclusive, non-recursive ownership semantics:

- A calling thread owns a mutex from the time that it successfully calls either *lock* or *try_lock* until it calls *unlock*.
- When a thread owns a *mutex*, all other threads will block (for calls to *lock*) or receive a *false* return value (for *try_lock*) if they attempt to claim ownership of the *mutex*.
- A calling thread must not own the *mutex* prior to calling *lock* or *try_lock*.

The behavior of a program is undefined if a *mutex* is destroyed while still owned by any threads, or a thread terminates while owning a *mutex*. The mutex class satisfies all requirements of and hpx::mutex is neither copyable nor movable.

Subclassed by *hpx::timed_mutex*

hpx::timed_mutex

Defined in header hpx/mutex.hpp[947].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::mutex*

[946] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/mutex.hpp
[947] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/mutex.hpp

class **timed_mutex** : private *hpx*::*mutex*

The timed_mutex class is a synchronization primitive that can be used to protect shared data from being simultaneously accessed by multiple threads. In a manner similar to *mutex*, timed_mutex offers exclusive, non-recursive ownership semantics. In addition, timed_mutex provides the ability to attempt to claim ownership of a timed_mutex with a timeout via the member functions *try_lock_for()* and *try_lock_until()*. The timed_mutex class satisfies all requirements of and

hpx::timed_mutex is neither copyable nor movable.

hpx::latch

Defined in header hpx/latch.hpp[948].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

class **latch**

Latches are a thread coordination mechanism that allow one or more threads to block until an operation is completed. An individual latch is a single-use object; once the operation has been completed, the latch cannot be reused.

Subclassed by hpx::lcos::local::latch

hpx::no_mutex

Defined in header hpx/mutex.hpp[949].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

struct **no_mutex**

`no_mutex` class can be used in cases where the shared data between multiple threads can be accessed simultaneously without causing inconsistencies.

hpx::nostopstate

Defined in header hpx/stop_token.hpp[950].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::stop_callback*
- *hpx::stop_source*
- *hpx::stop_token*
- *hpx::nostopstate_t*

948 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/latch.hpp
949 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/mutex.hpp
950 http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/stop_token.hpp

constexpr *nostopstate_t* `hpx::nostopstate` = {}

This is a constant object instance of *hpx::nostopstate_t* for use in constructing an empty *hpx::stop_source*, as a placeholder value in the non-default constructor.

hpx::stop_callback

Defined in header hpx/stop_token.hpp[951].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::nostopstate*
- *hpx::stop_source*
- *hpx::stop_token*
- *hpx::nostopstate_t*

template<typename **Callback**>

`hpx::stop_callback`(*stop_token*, *Callback*) -> *stop_callback*<*Callback*>

The *stop_callback* class template provides an RAII object type that registers a callback function for an associated hpx::stop_token object, such that the callback function will be invoked when the hpx::stop_token*'s* associated hpx::stop_source is requested to stop. Callback functions registered via *stop_callback's* constructor are invoked either in the same thread that successfully invokes *request_stop()* for a hpx::stop_source of the *stop_callback's* associated hpx::stop_token; or if stop has already been requested prior to the constructor's registration, then the callback is invoked in the thread constructing the *stop_callback*. More than one *stop_callback* can be created for the same hpx::stop_token, from the same or different threads concurrently. No guarantee is provided for the order in which they will be executed, but they will be invoked synchronously; except for *stop_callback(s)* constructed after stop has already been requested for the hpx::stop_token, as described previously. If an invocation of a callback exits via an exception then hpx::terminate is called. *hpx::stop_callback* is not *CopyConstructible*, *CopyAssignable*, *MoveConstructible*, nor *MoveAssignable*. The template param Callback type must be both *invocable* and *destructible*. Any return value is ignored.

hpx::stop_source

Defined in header hpx/stop_token.hpp[952].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::nostopstate*
- *hpx::stop_callback*
- *hpx::stop_token*

[951] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/stop_token.hpp

[952] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/stop_token.hpp

- *hpx::nostopstate_t*

class **stop_source**

The stop_source class provides the means to issue a stop request, such as for hpx::jthread cancellation. A stop request made for one stop_source object is visible to all *stop_sources* and *hpx::stop_tokens* of the same associated stop-state; any *hpx::stop_callback(s)* registered for associated hpx::stop_token(s) will be invoked, and any *hpx::condition_variable_any* objects waiting on associated hpx::stop_token(s) will be awoken. Once a stop is requested, it cannot be withdrawn. Additional stop requests have no effect.

> **Note**
>
> For the purposes of hpx::jthread cancellation the stop_source object should be retrieved from the *hpx::jthread* object using *get_stop_source()*; or stop should be requested directly from the hpx::jthread object using *request_stop()*. This will then use the same associated stop-state as that passed into the hpx::jthread*'s* invoked function argument (i.e., the function being executed on its thread). For other uses, however, a stop_source can be constructed separately using the default constructor, which creates new stop-state.

hpx::stop_token

Defined in header hpx/stop_token.hpp[953].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::nostopstate*
- *hpx::stop_callback*
- *hpx::stop_source*
- *hpx::nostopstate_t*

class **stop_token**

The stop_token class provides the means to check if a stop request has been made or can be made, for its associated hpx::stop_source object. It is essentially a thread-safe "view" of the associated stop-state. The *stop_token* can also be passed to the constructor of *hpx::stop_callback*, such that the callback will be invoked if the stop_token*'s* associated hpx::stop_source is requested to stop. And stop_token can be passed to the interruptible waiting functions of hpx::condition_variable_any, to interrupt the condition variable's wait if stop is requested.

> **Note**
>
> A stop_token object is not generally constructed independently, but rather retrieved from a hpx::jthread or hpx::stop_source. This makes it share the same associated stop-state as the hpx::jthread or hpx::stop_source.

[953] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/stop_token.hpp

hpx::nostopstate_t

Defined in header hpx/stop_token.hpp[954].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::nostopstate*
- *hpx::stop_callback*
- *hpx::stop_source*
- *hpx::stop_token*

struct **nostopstate_t**

Unit type intended for use as a placeholder in *hpx::stop_source* non-default constructor, that makes the constructed *hpx::stop_source* empty with no associated stop-state.

hpx::shared_mutex

Defined in header hpx/shared_mutex.hpp[955].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

using *hpx*::**shared_mutex** = detail::shared_mutex<>

The *shared_mutex* class is a synchronization primitive that can be used to protect shared data from being simultaneously accessed by multiple threads. In contrast to other mutex types which facilitate exclusive access, a *shared_mutex* has two levels of access:

- *shared* - several threads can share ownership of the same mutex.
- *exclusive* - only one thread can own the mutex.

If one thread has acquired the exclusive lock (through *lock*, *try_lock*), no other threads can acquire the lock (including the shared). If one thread has acquired the shared lock (through *lock_shared*, *try_lock_shared*), no other thread can acquire the exclusive lock, but can acquire the shared lock. Only when the exclusive lock has not been acquired by any thread, the shared lock can be acquired by multiple threads. Within one thread, only one lock (shared or exclusive) can be acquired at the same time. Shared mutexes are especially useful when shared data can be safely read by any number of threads simultaneously, but a thread may only write the same data when no other thread is reading or writing at the same time. The *shared_mutex* class satisfies all requirements of *SharedMutex* and *StandardLayoutType*.

[954] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/stop_token.hpp

[955] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/shared_mutex.hpp

hpx::binary_semaphore

Defined in header hpx/semaphore.hpp[956].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

class **binary_semaphore**

A binary semaphore is a semaphore object that has only two states. binary_semaphore is an alias for specialization of hpx::counting_semaphore with *LeastMaxValue* being 1. HPX's implementation of binary_semaphore is more efficient than the default implementation of a counting semaphore with a unit resource count (hpx::counting_semaphore).

hpx::condition_variable

Defined in header hpx/condition_variable.hpp[957].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::condition_variable_any*
- *hpx::cv_status*

class **condition_variable**

The condition_variable class is a synchronization primitive that can be used to block a thread, or multiple threads at the same time, until another thread both modifies a shared variable (the condition), and notifies the condition_variable.

The thread that intends to modify the shared variable has to

a. acquire a hpx::mutex (typically via *std::lock_guard*)
b. perform the modification while the lock is held
c. execute *notify_one* or *notify_all* on the *condition_variable* (the lock does not need to be held for notification)

Even if the shared variable is atomic, it must be modified under the mutex in order to correctly publish the modification to the waiting thread. Any thread that intends to wait on condition_variable has to

a. acquire a *std::unique_lock<hpx::mutex>*, on the same mutex as used to protect the shared variable
b. either

i. check the condition, in case it was already updated and notified
ii. execute *wait*, *await_for*, or *wait_until*. The wait operations atomically release the mutex and suspend the execution of the thread.

[956] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/semaphore.hpp

[957] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/condition_variable.hpp

iii. When the condition variable is notified, a timeout expires, or a spurious wakeup occurs, the thread is awakened, and the mutex is atomically reacquired. The thread should then check the condition and resume waiting if the wake-up was spurious. or

i. use the predicated overload of *wait*, *wait_for*, and *wait_until*, which takes care of the three steps above.

hpx::condition_variable works only with *std::unique_lock<hpx::mutex>*. This restriction allows for maximal efficiency on some platforms. hpx::condition_variable_any provides a condition variable that works with any object, such as *std::shared_lock*.

Condition variables permit concurrent invocation of the *wait*, *wait_for*, *wait_until*, *notify_one* and *notify_all* member functions.

The class hpx::condition_variable is aIt is not , , , or

hpx::condition_variable_any

Defined in header hpx/condition_variable.hpp[958].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::condition_variable*
- *hpx::cv_status*

class **condition_variable_any**

The condition_variable_any class is a generalization of hpx::condition_variable. Whereas hpx::condition_variable works only on *std::unique_lock<std::mutex>*, *a condition_variable_any* can operate on any lock that meets the requirements.

See hpx::condition_variable for the description of the semantics of condition variables. It is not , , , or

hpx::cv_status

Defined in header hpx/condition_variable.hpp[959].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::condition_variable*
- *hpx::condition_variable_any*

enum class *hpx*::**cv_status**

The scoped enumeration hpx::cv_status describes whether a timed wait returned because of timeout or not. hpx::cv_status is used by the *wait_for* and *wait_until* member functions of hpx::condition_variable and hpx::condition_variable_any.

[958] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/condition_variable.hpp

[959] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/condition_variable.hpp

Values:

enumerator **no_timeout**

The condition variable was awakened with *notify_all*, *notify_one*, or spuriously

enumerator **timeout**

the condition variable was awakened by timeout expiration

enumerator **error**

there was an error

hpx::recursive_mutex

Defined in header hpx/mutex.hpp[960].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

using *hpx*::**recursive_mutex** = detail::recursive_mutex_impl<>

hpx::counting_semaphore

Defined in header hpx/semaphore.hpp[961].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<*std*::ptrdiff_t **LeastMaxValue** = PTRDIFF_MAX>

class **counting_semaphore**

A semaphore is a protected variable (an entity storing a value) or abstract data type (an entity grouping several variables that may or may not be numerical) which constitutes the classic method for restricting access to shared resources, such as shared memory, in a multiprogramming environment. Semaphores exist in many variants, though usually the term refers to a counting semaphore, since a binary semaphore is better known as a mutex. A counting semaphore is a counter for a set of available resources, rather than a locked/unlocked flag of a single resource. It was invented by Edsger Dijkstra. Semaphores are the classic solution to preventing race conditions in the dining philosophers problem, although they do not prevent resource deadlocks.

Counting semaphores can be used for synchronizing multiple threads as well: one thread waiting for several other threads to touch (signal) the semaphore, or several threads waiting for one other thread to touch this semaphore. Unlike hpx::mutex a counting_semaphore is not tied to threads of execution — acquiring a semaphore can occur on a different thread than releasing the semaphore, for example. All operations on counting_semaphore can be performed concurrently and without any relation to specific threads of execution, with the exception of the destructor which cannot be performed concurrently but can be performed on a different thread.

[960] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/mutex.hpp
[961] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/semaphore.hpp

Semaphores are lightweight synchronization primitives used to constrain concurrent access to a shared resource. They are widely used to implement other synchronization primitives and, whenever both are applicable, can be more efficient than condition variables.

A counting semaphore is a semaphore object that models a non-negative resource count.

Class template counting_semaphore maintains an internal counter that is initialized when the semaphore is created. The counter is decremented when a thread acquires the semaphore, and is incremented when a thread releases the semaphore. If a thread tries to acquire the semaphore when the counter is zero, the thread will block until another thread increments the counter by releasing the semaphore.

Specializations of hpx::counting_semaphore are not , , , , or .

> **Note**
>
> counting_semaphore*'s try_acquire()* can spuriously fail.

Template Parameters

LeastMaxValue – counting_semaphore allows more than one concurrent access to the same resource, for at least *LeastMaxValue* concurrent accessors. As its name indicates, the *LeastMaxValue* is the minimum max value, not the actual max value. Thus *max()* can yield a number larger than *LeastMaxValue*.

hpx::once_flag

Defined in header hpx/mutex.hpp[962].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::call_once*

struct **once_flag**

The class *hpx::once_flag* is a helper structure for *hpx::call_once*. An object of type *hpx::once_flag* that is passed to multiple calls to *hpx::call_once* allows those calls to coordinate with each other such that only one of the calls will actually run to completion. *hpx::once_flag* is neither copyable nor movable.

hpx::call_once

Defined in header hpx/mutex.hpp[963].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::once_flag*

template<typename **F**, typename ...**Args**>

[962] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/mutex.hpp
[963] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/mutex.hpp

void *hpx*::**call_once**(*once_flag* &flag, *F* &&f, *Args*&&... args)

Executes the Callable object *f* exactly once, even if called concurrently, from several threads.

In detail:

- If, by the time *call_once* is called, flag indicates that *f* was already called, *call_once* returns right away (such a call to *call_once* is known as passive).
- Otherwise, *call_once* invokes `std::forward<Callable>(f)` with the arguments `std::forward<Args>(args)...` (as if by `hpx::invoke`). Unlike the `hpx::thread` constructor or `hpx::async`, the arguments are not moved or copied because they don't need to be transferred to another thread of execution. (such a call to *call_once* is known as active).

– If that invocation throws an exception, it is propagated to the caller of *call_once*, and the flag is not flipped so that another call will be attempted (such a call to *call_once* is known as exceptional).

– If that invocation returns normally (such a call to *call_once* is known as returning), the flag is flipped, and all other calls to *call_once* with the same flag are guaranteed to be passive. All active calls on the same flag form a single total order consisting of zero or more exceptional calls, followed by one returning call. The end of each active call synchronizes-with the next active call in that order. The return from the returning call synchronizes-with the returns from all passive calls on the same flag: this means that all concurrent calls to *call_once* are guaranteed to observe any side-effects made by the active call, with no additional synchronization.

Note

If concurrent calls to *call_once* pass different functions *f*, it is unspecified which f will be called. The selected function runs in the same thread as the *call_once* invocation it was passed to. Initialization of function-local statics is guaranteed to occur only once even when called from multiple threads, and may be more efficient than the equivalent code using `hpx::call_once`. The POSIX equivalent of this function is *pthread_once*.

Parameters

- **flag** – an object, for which exactly one function gets executed
- **f** – Callable object to invoke
- **args** – arguments to pass to the function

Throws

`std::system_error` – if any condition prevents calls to *call_once* from executing as specified or any exception thrown by *f*

hpx/synchronization/sliding_semaphore.hpp

Defined in header hpx/synchronization/sliding_semaphore.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Typedefs

using **sliding_semaphore** = *sliding_semaphore_var*<>

template<typename **Mutex** = *hpx*::*spinlock*>

class **sliding_semaphore_var**

#include <sliding_semaphore.hpp> A semaphore is a protected variable (an entity storing a value) or abstract data type (an entity grouping several variables that may or may not be numerical) which constitutes the classic method for restricting access to shared resources, such as shared memory, in a multiprogramming environment. Semaphores exist in many variants, though usually the term refers to a counting semaphore, since a binary semaphore is better known as a mutex. A counting semaphore is a counter for a set of available resources, rather than a locked/unlocked flag of a single resource. It was invented by Edsger Dijkstra. Semaphores are the classic solution to preventing race conditions in the dining philosophers problem, although they do not prevent resource deadlocks.

Sliding semaphores can be used for synchronizing multiple threads as well: one thread waiting for several other threads to touch (signal) the semaphore, or several threads waiting for one other thread to touch this semaphore. The difference to a counting semaphore is that a sliding semaphore will not limit the number of threads which are allowed to proceed, but will make sure that the difference between the (arbitrary) number passed to set and wait does not exceed a given threshold.

Public Functions

sliding_semaphore_var(*sliding_semaphore_var* const&) = delete

sliding_semaphore_var &**operator=**(*sliding_semaphore_var* const&) = delete

sliding_semaphore_var(*sliding_semaphore_var*&&) = delete *sliding_semaphore_var* &**operator=**(*sliding_semaphore_var*&&) = delete

inline explicit **sliding_semaphore_var**(*std*::int64_t max_difference, *std*::int64_t lower_limit = 0) noexcept

Construct a new sliding semaphore.

Parameters

- **max_difference** – [in] The max difference between the upper limit (as set by *wait()*) and the lower limit (as set by *signal()*) which is allowed without suspending any thread calling *wait()*.
- **lower_limit** – [in] The initial lower limit.

inline void **set_max_difference**(*std*::int64_t max_difference, *std*::int64_t lower_limit = 0) noexcept

Set/Change the difference that will cause the semaphore to trigger

Parameters

- **max_difference** – [in] The max difference between the upper limit (as set by *wait()*) and the lower limit (as set by *signal()*) which is allowed without suspending any thread calling *wait()*.
- **lower_limit** – [in] The initial lower limit.

inline void **wait**(*std*::int64_t upper_limit)

Wait for the semaphore to be signaled.

Parameters

upper_limit – [in] The new upper limit. The calling thread will be suspended if the difference between this value and the largest lower_limit which was set by *signal()* is larger than the max_difference.

inline bool **try_wait**(*std*::int64_t upper_limit = 1)

Try to wait for the semaphore to be signaled.

Parameters

upper_limit – [in] The new upper limit. The calling thread will be suspended if the difference between this value and the largest lower_limit which was set by *signal()* is larger than the max_difference.

Returns

The function returns true if the calling thread would not block if it was calling *wait()*.

inline void **signal**(*std*::int64_t lower_limit)

Signal the semaphore.

Parameters

lower_limit – [in] The new lower limit. This will update the current lower limit of this semaphore. It will also re-schedule all suspended threads for which their associated upper limit is not larger than the lower limit plus the max_difference.

inline *std*::int64_t **signal_all**()

Private Types

using **mutex_type** = *Mutex*

using **data_type** = *lcos*::*local*::detail::sliding_semaphore_data<*mutex_type*>

Private Members

hpx::intrusive_ptr<*data_type*> **data_**

hpx::barrier

Defined in header hpx/barrier.hpp[964].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **OnCompletion** = detail::empty_oncompletion>

class **barrier**

A barrier is a thread coordination mechanism whose lifetime consists of a sequence of barrier phases, where each phase allows at most an expected number of threads to block until the expected number of threads arrive at the barrier. [Note: A barrier is useful for managing repeated tasks that are handled by multiple threads. - end note] Each barrier phase consists of the following steps:

- The expected count is decremented by each call to arrive or arrive_and_drop.
- When the expected count reaches zero, the phase completion step is run. For the specialization with the default value of the CompletionFunction template parameter, the completion step is run as part of the call to arrive or arrive_and_drop that caused the expected count to reach zero. For other specializations, the completion step is run on one of the threads that arrived at the barrier during the phase.
- When the completion step finishes, the expected count is reset to what was specified by the expected argument to the constructor, possibly adjusted by calls to arrive_and_drop, and the next phase starts.

Each phase defines a phase synchronization point. Threads that arrive at the barrier during the phase can block on the phase synchronization point by calling wait, and will remain blocked until the phase completion step is run. The phase completion step that is executed at the end of each phase has the following effects:

- Invokes the completion function, equivalent to completion().

[964] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/barrier.hpp

- Unblocks all threads that are blocked on the phase synchronization point.

  The end of the completion step strongly happens before the returns from all calls that were unblocked by the completion step. For specializations that do not have the default value of the CompletionFunction template parameter, the behavior is undefined if any of the barrier object's member functions other than wait are called while the completion step is in progress.

  Concurrent invocations of the member functions of barrier, other than its destructor, do not introduce data races. The member functions arrive and arrive_and_drop execute atomically.

  CompletionFunction shall meet the Cpp17MoveConstructible (Table 28) and Cpp17Destructible (Table 32) requirements. std::is_nothrow_invocable_v<CompletionFunction&> shall be true.

  The default value of the CompletionFunction template parameter is an unspecified type, such that, in addition to satisfying the requirements of CompletionFunction, it meets the Cpp17DefaultConstructible requirements (Table 27) and completion() has no effects.

  barrier::arrival_token is an unspecified type, such that it meets the Cpp17MoveConstructible (Table 28), Cpp17MoveAssignable (Table 30), and Cpp17Destructible (Table 32) requirements.

hpx/synchronization/event.hpp

Defined in header hpx/synchronization/event.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **lcos**

namespace **local**

class **event**

#include <event.hpp> Event semaphores can be used for synchronizing multiple threads that need to wait for an event to occur. When the event occurs, all threads waiting for the event are woken up.

Public Functions

inline **event**() noexcept

Construct a new event semaphore.

inline bool **occurred**() const noexcept

Check if the event has occurred.

inline void **wait**()

Wait for the event to occur.

inline void **set**()

Release all threads waiting on this semaphore.

inline void **reset**() noexcept

Reset the event.

Private Types

using **mutex_type** = *hpx*::*spinlock*

Private Functions

inline void **wait_locked**(*std*::unique_lock<*mutex_type*> &l)

inline void **set_locked**(*std*::unique_lock<*mutex_type*> l)

Private Members

mutex_type **mtx_**

This mutex protects the queue.

local::detail::condition_variable **cond_**

std::atomic<bool> **event_**

hpx::spinlock

Defined in header hpx/mutex.hpp[965].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

using *hpx*::**spinlock** = detail::spinlock<true>

`spinlock` is a type of lock that causes a thread attempting to obtain it to check for its availability while waiting in a loop continuously.

tag_invoke

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::invoke

Defined in header hpx/functional.hpp[966].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **F**, typename ...**Ts**>

constexpr *util*::invoke_result_t<*F*, *Ts*&&...> *hpx*::**invoke**(*F* &&f, *Ts*&&... vs) noexcept(noexcept(HPX_INVOKE(*std*::forward<*F*>(*f*), *std*::forward<*Ts*>(*vs*)...)))

Invokes the given callable object f with the content of the argument pack vs

> **Note**
>
> This function is similar to `std::invoke` (C++17)

Parameters

- **f** – Requires to be a callable object. If f is a member function pointer, the first argument in the pack will be treated as the callee (this object).
- **vs** – An arbitrary pack of arguments

Throws

`std::exception` – like objects thrown by call to object f with the argument types vs.

Returns

The result of the callable object when it's called with the given argument types.

[965] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/mutex.hpp

[966] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

hpx::is_invocable

Defined in header hpx/type_traits.hpp[967].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::is_invocable_r*

template<typename **F**, typename ...**Ts**>

struct **is_invocable** : public *hpx*::detail::is_invocable_impl<*F*&&(*Ts*&&...)>

Determines whether *F* can be invoked with the arguments *Ts*.... Formally, determines whether

```
INVOKE(std::declval<F>(), std::declval<Ts>()...)
```

is well-formed when treated as an unevaluated operand, where *INVOKE* is the operation defined in *Callable*.

F, R and all types in the parameter pack Ts shall each be a complete type, (possibly cv-qualified) void, or an array of unknown bound. Otherwise, the behavior is undefined. If an instantiation of a template above depends, directly or indirectly, on an incomplete type, and that instantiation could yield a different result if that type were hypothetically completed, the behavior is undefined.

hpx::is_invocable_r

Defined in header hpx/type_traits.hpp[968].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::is_invocable*

template<typename **R**, typename **F**, typename ...**Ts**>

struct **is_invocable_r** : public *hpx*::detail::is_invocable_r_impl<*F*&&(*Ts*&&...), *R*>

Determines whether *F* can be invoked with the arguments *Ts*... to yield a result that is convertible to *R* and the implicit conversion does not bind a reference to a temporary object (since C++23). If *R* is *cv* void, the result can be any type. Formally, determines whether

```
INVOKE<R>(std::declval<F>(), std::declval<Ts>()...)
```

is well-formed when treated as an unevaluated operand, where *INVOKE* is the operation defined in *Callable*. Determines whether *F* can be invoked with the arguments *Ts*.... Formally, determines whether

[967] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/type_traits.hpp

[968] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/type_traits.hpp

```
INVOKE(std::declval<F>(), std::declval<Ts>()...)
```

is well-formed when treated as an unevaluated operand, where *INVOKE* is the operation defined in *Callable*.

F, R and all types in the parameter pack Ts shall each be a complete type, (possibly cv-qualified) void, or an array of unknown bound. Otherwise, the behavior is undefined. If an instantiation of a template above depends, directly or indirectly, on an incomplete type, and that instantiation could yield a different result if that type were hypothetically completed, the behavior is undefined.

thread_pool_util

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/thread_pool_util/thread_pool_suspension_helpers.hpp

Defined in header hpx/thread_pool_util/thread_pool_suspension_helpers.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **threads**

Functions

hpx::*future*<void> **resume_processing_unit**(*thread_pool_base* &pool, *std*::size_t virt_core)

Resumes the given processing unit. When the processing unit has been resumed the returned future will be ready.

> **Note**
>
> Can only be called from an HPX thread. Use resume_processing_unit_cb or to resume the processing unit from outside HPX. Requires that the pool has threads::policies::enable_elasticity set.

Parameters

- **pool** – [in] The thread pool to resume a processing unit on.
- **virt_core** – [in] The processing unit on the pool to be resumed. The processing units are indexed starting from 0.

Returns

A future<void> which is ready when the given processing unit has been resumed.

void **resume_processing_unit_cb**(*thread_pool_base* &pool, *hpx*::*function*<void()> callback, *std*::size_t virt_core, *error_code* &ec = *throws*)

Resumes the given processing unit. Takes a callback as a parameter which will be called when the processing unit has been resumed.

> **Note**
>
> Requires that the pool has threads::policies::enable_elasticity set.

Parameters

- **pool** – [in] The thread pool to resume a processing unit on.
- **callback** – [in] Callback which is called when the processing unit has been suspended.
- **virt_core** – [in] The processing unit to resume.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

hpx::*future*<void> **suspend_processing_unit**(*thread_pool_base* &pool, *std*::size_t virt_core)

Suspends the given processing unit. When the processing unit has been suspended the returned future will be ready.

> **Note**
>
> Can only be called from an HPX thread. Use suspend_processing_unit_cb or to suspend the processing unit from outside HPX. Requires that the pool has threads::policies::enable_elasticity set.

Parameters

- **pool** – [in] The thread pool to suspend a processing unit from.
- **virt_core** – [in] The processing unit on the pool to be suspended. The processing units are indexed starting from 0.

Throws

hpx::exception – if called from outside the HPX runtime.

Returns

A `future<void>` which is ready when the given processing unit has been suspended.

void **suspend_processing_unit_cb**(*hpx*::*function*<void()> callback, *thread_pool_base* &pool, *std*::size_t virt_core, *error_code* &ec = *throws*)

Suspends the given processing unit. Takes a callback as a parameter which will be called when the processing unit has been suspended.

> **Note**
>
> Requires that the pool has threads::policies::enable_elasticity set.

Parameters

- **pool** – [in] The thread pool to suspend a processing unit from.
- **callback** – [in] Callback which is called when the processing unit has been suspended.
- **virt_core** – [in] The processing unit to suspend.

- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

hpx::*future*<void> **resume_pool**(*thread_pool_base* &pool)

Resumes the thread pool. When the all OS threads on the thread pool have been resumed the returned future will be ready.

> **Note**
>
> Can only be called from an HPX thread. Use resume_cb or resume_direct to suspend the pool from outside HPX.

Parameters
pool – [in] The thread pool to resume.

Throws
hpx::exception – if called from outside the HPX runtime.

Returns
A `future<void>` which is ready when the thread pool has been resumed.

void **resume_pool_cb**(*thread_pool_base* &pool, *hpx*::*function*<void()> callback, *error_code* &ec = *throws*)

Resumes the thread pool. Takes a callback as a parameter which will be called when all OS threads on the thread pool have been resumed.

Parameters

- **pool** – [in] The thread pool to resume.
- **callback** – [in] called when the thread pool has been resumed.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

hpx::*future*<void> **suspend_pool**(*thread_pool_base* &pool)

Suspends the thread pool. When the all OS threads on the thread pool have been suspended the returned future will be ready.

> **Note**
>
> Can only be called from an HPX thread. Use suspend_cb or suspend_direct to suspend the pool from outside HPX. A thread pool cannot be suspended from an HPX thread running on the pool itself.

Parameters
pool – [in] The thread pool to suspend.

Throws
hpx::exception – if called from outside the HPX runtime.

Returns
A `future<void>` which is ready when the thread pool has been suspended.

void **suspend_pool_cb**(*thread_pool_base* &pool, *hpx*::*function*<void()> callback, *error_code* &ec = *throws*)

Suspends the thread pool. Takes a callback as a parameter which will be called when all OS threads on the thread pool have been suspended.

> **Note**
>
> A thread pool cannot be suspended from an HPX thread running on the pool itself.

Parameters

- **pool** – [in] The thread pool to suspend.
- **callback** – [in] called when the thread pool has been suspended.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

hpx::exception – if called from an HPX thread which is running on the pool itself.

thread_support

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/thread_support/spinlock.hpp

Defined in header hpx/thread_support/spinlock.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

hpx::unlock_guard

Defined in header hpx/mutex.hpp[969].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **Lock**>

class **unlock_guard**

The class *unlock_guard* is a mutex wrapper that provides a convenient mechanism for releasing a mutex for the duration of a scoped block.

unlock_guard performs the opposite functionality of `lock_guard`.

When a `lock_guard` object is created, it attempts to take ownership of the mutex it is given. When control leaves the scope in which the `lock_guard` object was created, the `lock_guard` is destructed and the mutex is released. Accordingly, when an *unlock_guard* object is created, it attempts to release the ownership of the mutex it is given. So, when control leaves the scope in which the *unlock_guard* object was created, the *unlock_guard* is destructed and the mutex is owned again. In this way, the

[969] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/mutex.hpp

mutex is unlocked in the constructor and locked in the destructor, so that one can have an unlocked section within a locked one.

> **Note**
>
> *unlock_guard* is non-copyable and non-movable.

Template Parameters

`Lock` – The type of the lockable object managed by the *unlock_guard*.

threading

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::thread

Defined in header hpx/thread.hpp[970].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::this_thread::yield*
- *hpx::this_thread::get_id*
- *hpx::this_thread::sleep_for*
- *hpx::this_thread::sleep_until*

class **thread**

The class thread represents a single thread of execution. Threads allow multiple functions to execute concurrently. Threads begin execution immediately upon construction of the associated thread object (pending any OS scheduling delays), starting at the top-level function provided as a constructor argument. The return value of the top-level function is ignored and if it terminates by throwing an exception, *hpx::terminate* is called. The top-level function may communicate its return value or an exception to the caller via hpx::promise or by modifying shared variables (which may require synchronization, see *hpx::mutex* and hpx::atomic) *hpx::thread* objects may also be in the state that does not represent any thread (after default construction, move from, detach, or join), and a thread of execution may not be associated with any thread objects (after detach). No two *hpx::thread* objects may represent the same thread of execution; hpx::thread is not *CopyConstructible* or *CopyAssignable*, although it is *MoveConstructible* and *MoveAssignable*.

[970] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/thread.hpp

hpx::this_thread::yield

Defined in header hpx/thread.hpp[971].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::thread*
- *hpx::this_thread::get_id*
- *hpx::this_thread::sleep_for*
- *hpx::this_thread::sleep_until*

void `hpx::this_thread::yield() noexcept`

Provides a hint to the implementation to reschedule the execution of threads, allowing other threads to run.

> **Note**
>
> The exact behavior of this function depends on the implementation, in particular on the mechanics of the OS scheduler in use and the state of the system. For example, a first-in-first-out realtime scheduler (SCHED_FIFO in Linux) would suspend the current thread and put it on the back of the queue of the same-priority threads that are ready to run (and if there are no other threads at the same priority, yield has no effect).

hpx::this_thread::get_id

Defined in header hpx/thread.hpp[972].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::thread*
- *hpx::this_thread::yield*
- *hpx::this_thread::sleep_for*
- *hpx::this_thread::sleep_until*

thread::id `hpx::this_thread::get_id() noexcept`

Returns the id of the current thread.

[971] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/thread.hpp
[972] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/thread.hpp

hpx::this_thread::sleep_for

Defined in header hpx/thread.hpp[973].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::thread*
- *hpx::this_thread::yield*
- *hpx::this_thread::get_id*
- *hpx::this_thread::sleep_until*

inline void `hpx::this_thread::sleep_for`(*hpx*::chrono::steady_duration const &rel_time)

Blocks the execution of the current thread for at least the specified *rel_time*. This function may block for longer than *rel_time* due to scheduling or resource contention delays.

It is recommended to use a steady clock to measure the duration. If an implementation uses a system clock instead, the wait time may also be sensitive to clock adjustments.

Parameters

rel_time – time duration to sleep

hpx::this_thread::sleep_until

Defined in header hpx/thread.hpp[974].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::thread*
- *hpx::this_thread::yield*
- *hpx::this_thread::get_id*
- *hpx::this_thread::sleep_for*

void `hpx::this_thread::sleep_until`(*hpx*::chrono::steady_time_point const &abs_time)

Blocks the execution of the current thread until specified *abs_time* has been reached.

It is recommended to use the clock tied to *abs_time*, in which case adjustments of the clock may be taken into account. Thus, the duration of the block might be more or less than `abs_time-Clock::now()` at the time of the call, depending on the direction of the adjustment and whether it is honored by the implementation. The function also may block until after *abs_time* has been reached due to process scheduling or resource contention delays.

Parameters

abs_time – absolute time to block until

[973] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/thread.hpp
[974] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/thread.hpp

hpx::jthread

Defined in header hpx/jthread.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

class **jthread**

The class *jthread* represents a single thread of execution. It has the same general behavior as hpx::thread, except that *jthread* automatically rejoins on destruction, and can be cancelled/stopped in certain situations. Threads begin execution immediately upon construction of the associated thread object (pending any OS scheduling delays), starting at the top-level function provided as a constructor argument. The return value of the top-level function is ignored and if it terminates by throwing an exception, *hpx::terminate* is called. The top-level function may communicate its return value or an exception to the caller via hpx::promise or by modifying shared variables (which may require synchronization, see hpx::mutex and *hpx::atomic*) Unlike hpx::thread, the jthread logically holds an internal private member of type hpx::stop_source, which maintains a shared stop-state. The *jthread* constructor accepts a function that takes a hpx::stop_token as its first argument, which will be passed in by the jthread from its internal stop_source. This allows the function to check if stop has been requested during its execution, and return if it has. hpx::jthread objects may also be in the state that does not represent any thread (after default construction, move from, detach, or join), and a thread of execution may be not associated with any *jthread* objects (after detach). No two hpx::jthread objects may represent the same thread of execution; hpx::jthread is not *CopyConstructible* or *CopyAssignable*, although it is *MoveConstructible* and *MoveAssignable*.

threading_base

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/threading_base/thread_helpers.hpp

Defined in header hpx/threading_base/thread_helpers.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **this_thread**

Functions

threads::*thread_restart_state* **suspend**(*threads*::*thread_schedule_state* state, *threads*::thread_id_type nextid, *threads*::*thread_description* const &description = *threads*::*thread_description*("this_thread::suspend"), *hpx*::*error_code* &ec = *hpx*::*throws*)

The function *suspend* will return control to the thread manager (suspends the current thread). It sets the new state of this thread to the thread state passed as the parameter.

> **Note**
>
> Must be called from within a HPX-thread.

Throws

`If` – `&ec != &throws`, never throws, but will set *ec* to an appropriate value when an error occurs. Otherwise, this function will throw an *hpx::exception* with an error code of *hpx::error::yield_aborted* if it is signaled with *wait_aborted*. If called outside a HPX-thread, this function will throw an *hpx::exception* with an error code of *hpx::error::null_thread_id*. If this function is called while the thread-manager is not running, it will throw an *hpx::exception* with an error code of *hpx::error::invalid_status*.

inline *threads*::*thread_restart_state* **suspend**(*threads*::*thread_schedule_state* state = *threads*::*thread_schedule_state*::*pending*, *threads*::*thread_description* const &description = *threads*::*thread_description*("this_thread::suspend"), *hpx*::*error_code* &ec = *hpx*::*throws*)

The function *suspend* will return control to the thread manager (suspends the current thread). It sets the new state of this thread to the thread state passed as the parameter.

> **Note**
>
> Must be called from within a HPX-thread.

Throws

`If` – `&ec != &throws`, never throws, but will set *ec* to an appropriate value when an error occurs. Otherwise, this function will throw an *hpx::exception* with an error code of *hpx::error::yield_aborted* if it is signaled with *wait_aborted*. If called outside a HPX-thread, this function will throw an *hpx::exception* with an error code of *hpx::error::null_thread_id*. If this function is called while the thread-manager is not running, it will throw an *hpx::exception* with an error code of *hpx::error::invalid_status*.

threads::*thread_restart_state* **suspend**(*hpx*::chrono::steady_time_point const &abs_time, *threads*::thread_id_type id, *threads*::*thread_description* const &description = *threads*::*thread_description*("this_thread::suspend"), *hpx*::*error_code* &ec = *hpx*::*throws*)

The function *suspend* will return control to the thread manager (suspends the current thread). It sets the new state of this thread to *suspended* and schedules a wakeup for this threads at the given time.

> **Note**
>
> Must be called from within a HPX-thread.

Throws

`If – &ec != &throws`, never throws, but will set *ec* to an appropriate value when an error occurs. Otherwise, this function will throw an *hpx::exception* with an error code of *hpx::error::yield_aborted* if it is signaled with *wait_aborted*. If called outside a HPX-thread, this function will throw an *hpx::exception* with an error code of *hpx::error::null_thread_id*. If this function is called while the thread-manager is not running, it will throw an *hpx::exception* with an error code of *hpx::error::invalid_status*.

inline *threads*::*thread_restart_state* **suspend**(*hpx*::chrono::steady_time_point const &abs_time, *threads*::*thread_description* const &description = *threads*::*thread_description*("this_thread::suspend"), *hpx*::*error_code* &ec = *hpx*::*throws*)

The function *suspend* will return control to the thread manager (suspends the current thread). It sets the new state of this thread to *suspended* and schedules a wakeup for this threads at the given time.

> **Note**
>
> Must be called from within a HPX-thread.

Throws

`If – &ec != &throws`, never throws, but will set *ec* to an appropriate value when an error occurs. Otherwise, this function will throw an *hpx::exception* with an error code of *hpx::error::yield_aborted* if it is signaled with *wait_aborted*. If called outside a HPX-thread, this function will throw an *hpx::exception* with an error code of *hpx::error::null_thread_id*. If this function is called while the thread-manager is not running, it will throw an *hpx::exception* with an error code of *hpx::error::invalid_status*.

inline *threads*::*thread_restart_state* **suspend**(*hpx*::chrono::steady_duration const &rel_time, *threads*::*thread_description* const &description = *threads*::*thread_description*("this_thread::suspend"), *hpx*::*error_code* &ec = *hpx*::*throws*)

The function *suspend* will return control to the thread manager (suspends the current thread). It sets the new state of this thread to *suspended* and schedules a wakeup for this threads after the given duration.

> **Note**
>
> Must be called from within a HPX-thread.

Throws

`If – &ec != &throws`, never throws, but will set *ec* to an appropriate value when an error occurs. Otherwise, this function will throw an *hpx::exception* with an error code of *hpx::error::yield_aborted* if it is signaled with *wait_aborted*. If called outside a HPX-thread, this function will throw an *hpx::exception* with an error code of *hpx::error::null_thread_id*. If this function is called while the thread-manager is not running, it will throw an *hpx::exception* with an error code of *hpx::error::invalid_status*.

inline *threads*::*thread_restart_state* **suspend**(*hpx*::chrono::steady_duration const &rel_time, *threads*::thread_id_type const &id, *threads*::*thread_description* const &description = *threads*::*thread_description*("this_thread::suspend"), *hpx*::*error_code* &ec = *hpx*::*throws*)

The function *suspend* will return control to the thread manager (suspends the current thread). It sets the new state of this thread to *suspended* and schedules a wakeup for this threads after the given duration.

> **Note**
>
> Must be called from within a HPX-thread.

Throws

`If` – `&ec != &throws`, never throws, but will set *ec* to an appropriate value when an error occurs. Otherwise, this function will throw an *hpx::exception* with an error code of *hpx::error::yield_aborted* if it is signaled with *wait_aborted*. If called outside a HPX-thread, this function will throw an *hpx::exception* with an error code of *hpx::error::null_thread_id*. If this function is called while the thread-manager is not running, it will throw an *hpx::exception* with an error code of *hpx::error::invalid_status*.

inline *threads*::*thread_restart_state* **suspend**(*std*::uint64_t ms, *threads*::*thread_description* const &description = *threads*::*thread_description*("this_thread::suspend"), *hpx*::*error_code* &ec = *hpx*::*throws*)

The function *suspend* will return control to the thread manager (suspends the current thread). It sets the new state of this thread to *suspended* and schedules a wakeup for this threads after the given time (specified in milliseconds).

> **Note**
>
> Must be called from within a HPX-thread.

Throws

`If` – `&ec != &throws`, never throws, but will set *ec* to an appropriate value when an error occurs. Otherwise, this function will throw an *hpx::exception* with an error code of *hpx::error::yield_aborted* if it is signaled with *wait_aborted*. If called outside a HPX-thread, this function will throw an *hpx::exception* with an error code of *hpx::error::null_thread_id*. If this function is called while the thread-manager is not running, it will throw an *hpx::exception* with an error code of *hpx::error::invalid_status*.

threads::*thread_pool_base* ***get_pool**(*hpx*::*error_code* &ec = *hpx*::*throws*)

Returns a pointer to the pool that was used to run the current thread

Throws

`If` – `&ec != &throws`, never throws, but will set *ec* to an appropriate value when an error occurs. Otherwise, this function will throw an *hpx::exception* with an error code of *hpx::error::yield_aborted* if it is signaled with *wait_aborted*. If called outside a HPX-thread, this function will throw an *hpx::exception* with an error code of *hpx::error::null_thread_id*. If this function is called while the thread-manager is not running, it will throw an *hpx::exception* with an error code of *hpx::error::invalid_status*.

namespace **threads**

Functions

thread_state **set_thread_state**(thread_id_type const &id, *thread_schedule_state* state = *thread_schedule_state*::*pending*, *thread_restart_state* stateex = *thread_restart_state*::*signaled*, *thread_priority* priority = *thread_priority*::*normal*, bool retry_on_active = true, *hpx*::*error_code* &ec = *hpx*::*throws*)

Set the thread state of the *thread* referenced by the thread_id *id*.

> **Note**
>
> If the thread referenced by the parameter *id* is in *thread_state::active* state this function schedules a new thread which will set the state of the thread as soon as it is not active anymore. The function returns *thread_state::active* in this case.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **id** – [in] The thread id of the thread the state should be modified for.
- **state** – [in] The new state to be set for the thread referenced by the *id* parameter.
- **stateex** – [in] The new extended state to be set for the thread referenced by the *id* parameter.
- **priority** – [in]
- **retry_on_active** – [in]
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns the previous state of the thread referenced by the *id* parameter. It will return one of the values as defined by the *thread_state* enumeration. If the thread is not known to the thread-manager the return value will be *thread_state::unknown*.

thread_id_ref_type **set_thread_state**(thread_id_type const &id, *hpx*::chrono::steady_time_point const &abs_time, *std*::atomic<bool> *started, *thread_schedule_state* state = *thread_schedule_state*::*pending*, *thread_restart_state* stateex = *thread_restart_state*::*timeout*, *thread_priority* priority = *thread_priority*::*normal*, bool retry_on_active = true, *hpx*::*error_code* &ec = *hpx*::*throws*)

Set the thread state of the *thread* referenced by the thread_id *id*.

Set a timer to set the state of the given *thread* to the given new value after it expired (at the given time)

Note

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- `id` – [in] The thread id of the thread the state should be modified for.
- `abs_time` – [in] Absolute point in time for the new thread to be run
- `started` – [in,out] A helper variable allowing to track the state of the timer helper thread
- `state` – [in] The new state to be set for the thread referenced by the *id* parameter.
- `stateex` – [in] The new extended state to be set for the thread referenced by the *id* parameter.
- `priority` – [in]
- `retry_on_active` – [in]
- `ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

inline thread_id_ref_type **set_thread_state**(thread_id_type const &id, *hpx*::chrono::steady_time_point const &abs_time, *thread_schedule_state* state = *thread_schedule_state*::*pending*, *thread_restart_state* stateex = *thread_restart_state*::*timeout*, *thread_priority* priority = *thread_priority*::*normal*, bool retry_on_active = true, *hpx*::*error_code*& = *throws*)

inline thread_id_ref_type **set_thread_state**(thread_id_type const &id, *hpx*::chrono::steady_duration const &rel_time, *thread_schedule_state* state = *thread_schedule_state*::*pending*, *thread_restart_state* stateex = *thread_restart_state*::*timeout*, *thread_priority* priority = *thread_priority*::*normal*, bool retry_on_active = true, *hpx*::*error_code* &ec = *hpx*::*throws*)

Set the thread state of the *thread* referenced by the thread_id *id*.

Set a timer to set the state of the given *thread* to the given new value after it expired (after the given duration)

Note

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- `id` – [in] The thread id of the thread the state should be modified for.
- `rel_time` – [in] Time duration after which the new thread should be run
- `state` – [in] The new state to be set for the thread referenced by the *id* parameter.

- **stateex** – [in] The new extended state to be set for the thread referenced by the *id* parameter.
- **priority** – [in]
- **retry_on_active** – [in]
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

thread_state **get_thread_state**(thread_id_type const &id, *hpx*::*error_code* &ec = *hpx*::*throws*) noexcept

The function get_thread_backtrace is part of the thread related API allows to query the currently stored thread back trace (which is captured during thread suspension).

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception. The function get_thread_state is part of the thread related API. It queries the state of one of the threads known to the thread-manager.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **id** – [in] The thread id of the thread being queried.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.
- **id** – [in] The thread id of the thread the state should be modified for.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns the currently captured stack back trace of the thread referenced by the *id* parameter. If the thread is not known to the thread-manager the return value will be the zero.

Returns

This function returns the thread state of the thread referenced by the *id* parameter. If the thread is not known to the thread-manager the return value will be *terminated*.

std::size_t **get_thread_phase**(thread_id_type const &id, *hpx*::*error_code* &ec = *hpx*::*throws*) noexcept

The function get_thread_phase is part of the thread related API. It queries the phase of one of the threads known to the thread-manager.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but re-

turns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **id** – [in] The thread id of the thread the phase should be modified for.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns the thread phase of the thread referenced by the *id* parameter. If the thread is not known to the thread-manager the return value will be ~0.

bool **get_thread_interruption_enabled**(thread_id_type const &id, *hpx*::*error_code* &ec = *hpx*::*throws*)

Returns whether the given thread can be interrupted at this point.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **id** – [in] The thread id of the thread which should be queried.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns *true* if the given thread can be interrupted at this point in time. It will return *false* otherwise.

bool **set_thread_interruption_enabled**(thread_id_type const &id, bool enable, *hpx*::*error_code* &ec = *hpx*::*throws*)

Set whether the given thread can be interrupted at this point.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **id** – [in] The thread id of the thread which should receive the new value.
- **enable** – [in] This value will determine the new interruption enabled status for the given thread.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns the previous value of whether the given thread could have been interrupted.

bool **get_thread_interruption_requested**(thread_id_type const &id, *hpx*::*error_code* &ec = *hpx*::*throws*)

Returns whether the given thread has been flagged for interruption.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **id** – [in] The thread id of the thread which should be queried.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns *true* if the given thread was flagged for interruption. It will return *false* otherwise.

void **interrupt_thread**(thread_id_type const &id, bool flag, *hpx*::*error_code* &ec = *hpx*::*throws*)

Flag the given thread for interruption.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **id** – [in] The thread id of the thread which should be interrupted.
- **flag** – [in] The flag encodes whether the thread should be interrupted (if it is *true*), or 'uninterrupted' (if it is *false*).
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

inline void **interrupt_thread**(thread_id_type const &id, *hpx*::*error_code* &ec = *hpx*::*throws*)

void **interruption_point**(thread_id_type const &id, *hpx*::*error_code* &ec = *hpx*::*throws*)

Interrupt the current thread at this point if it was canceled. This will throw a thread_interrupted exception, which will cancel the thread.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **id** – [in] The thread id of the thread which should be interrupted.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

threads::*thread_priority* **get_thread_priority**(thread_id_type const &id, *hpx*::*error_code* &ec = *hpx*::*throws*) noexcept

Return priority of the given thread

Note

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **id** – [in] The thread id of the thread whose priority is queried.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

std::ptrdiff_t **get_stack_size**(thread_id_type const &id, *hpx*::*error_code* &ec = *hpx*::*throws*) noexcept

Return stack size of the given thread

Note

As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **id** – [in] The thread id of the thread whose priority is queried.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

threads::*thread_pool_base* ***get_pool**(thread_id_type const &id, *hpx*::*error_code* &ec = *hpx*::*throws*)

Returns a pointer to the pool that was used to run the current thread

Throws

`If` – `&ec != &throws`, never throws, but will set *ec* to an appropriate value when an error occurs. Otherwise, this function will throw an *hpx::exception* with an error code of *hpx::error::yield_aborted* if it is signaled with *wait_aborted*. If called outside a HPX-thread, this function will throw an *hpx::exception* with an error code of *hpx::error::null_thread_id*. If this function is called while the thread-manager is not running, it will throw an *hpx::exception* with an error code of *hpx::error::invalid_status*.

hpx/threading_base/thread_pool_base.hpp

Defined in header hpx/threading_base/thread_pool_base.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **threads**

Functions

std::ostream &**operator<<**(*std*::ostream &os, *thread_pool_base* const &thread_pool)

struct **thread_pool_init_parameters**

Public Functions

inline **thread_pool_init_parameters**(*std*::string const &name, *std*::size_t index, *policies*::scheduler_mode mode, *std*::size_t num_threads, *std*::size_t thread_offset, *hpx*::*threads*::*policies*::callback_notifier &notifier, *hpx*::*threads*::*policies*::detail::affinity_data const &affinity_data, *hpx*::*threads*::detail::network_background_callback_type const &network_background_callback = *hpx*::*threads*::detail::network_background_callback_type(), *std*::size_t max_background_threads = static_cast<*std*::size_t>(-1), *std*::size_t max_idle_loop_count = HPX_IDLE_LOOP_COUNT_MAX, *std*::size_t max_busy_loop_count = HPX_BUSY_LOOP_COUNT_MAX, *std*::size_t shutdown_check_count = 10)

Public Members

std::string const &**name_**

std::size_t **index_**

policies::scheduler_mode **mode_**

std::size_t **num_threads_**

std::size_t **thread_offset_**

hpx::*threads*::*policies*::callback_notifier &**notifier_**

hpx::*threads*::*policies*::detail::affinity_data const &**affinity_data_**

hpx::*threads*::detail::network_background_callback_type const &**network_background_callback_**

std::size_t **max_background_threads_**

std::size_t **max_idle_loop_count_**

std::size_t **max_busy_loop_count_**

std::size_t **shutdown_check_count_**

class **thread_pool_base**

#include <thread_pool_base.hpp> The base class used to manage a pool of OS threads.

Public Functions

virtual void **`suspend_processing_unit_direct`**(*std*::size_t virt_core, *error_code* &ec = *throws*) = 0

Suspends the given processing unit. Blocks until the processing unit has been suspended.

Parameters

- **`virt_core`** – [in] The processing unit on the pool to be suspended. The processing units are indexed starting from 0.
- **`ec`** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

virtual void **`resume_processing_unit_direct`**(*std*::size_t virt_core, *error_code* &ec = *throws*) = 0

Resumes the given processing unit. Blocks until the processing unit has been resumed.

Parameters

- **`virt_core`** – [in] The processing unit on the pool to be resumed. The processing units are indexed starting from 0.
- **`ec`** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

virtual void **`resume_direct`**(*error_code* &ec = *throws*) = 0

Resumes the thread pool. Blocks until all OS threads on the thread pool have been resumed.

Parameters

`ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

virtual void **`suspend_direct`**(*error_code* &ec = *throws*) = 0

Suspends the thread pool. Blocks until all OS threads on the thread pool have been suspended.

> **Note**
>
> A thread pool cannot be suspended from an HPX thread running on the pool itself.

Parameters

`ec` – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

`hpx::exception` – if called from an HPX thread which is running on the pool itself.

hpx/threading_base/print.hpp

Defined in header hpx/threading_base/print.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::annotated_function

Defined in header hpx/functional.hpp[975].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

template<typename **F**>

constexpr *F* &&*hpx*::**annotated_function**(*F* &&f, char const* = nullptr) noexcept

Returns a function annotated with the given annotation.

Annotating includes setting the thread description per thread id.

Parameters

f – Function to annotate

template<typename **F**>

constexpr *F* &&*hpx*::**annotated_function**(*F* &&f, *std*::string const&) noexcept

hpx/threading_base/register_thread.hpp

Defined in header hpx/threading_base/register_thread.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **threads**

[975] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

Functions

template<typename **F**>

thread_function_type **make_thread_function**(*F* &&f)

template<typename **F**, typename ...**Ts**>

thread_function_type **make_thread_function_nullary**(*F* &&f, *Ts*&&... ts)

void **register_thread**(*threads*::thread_init_data &data, *threads*::*thread_pool_base* *pool, *threads*::thread_id_ref_type &id, *error_code* &ec = *hpx*::*throws*)

Create a new *thread* using the given data.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- `data` – [in] The data to use for creating the thread.
- `pool` – [in] The thread pool to use for launching the work.
- `id` – [out] The id of the newly created thread (if applicable)
- `ec` – [in,out] This represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws

`invalid_status` – if the runtime system has not been started yet.

Returns

This function will return the internal id of the newly created HPX-thread.

threads::thread_id_ref_type **register_thread**(*threads*::thread_init_data &data, *threads*::*thread_pool_base* *pool, *error_code* &ec = *hpx*::*throws*)

void **register_thread**(*threads*::thread_init_data &data, *threads*::thread_id_ref_type &id, *error_code* &ec = *throws*)

Create a new *thread* using the given data on the same thread pool as the calling thread, or on the default thread pool if not on an HPX thread.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **data** – [in] The data to use for creating the thread.
- **id** – [out] The id of the newly created thread (if applicable)
- **ec** – [in,out] This represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws
invalid_status – if the runtime system has not been started yet.

Returns
This function will return the internal id of the newly created HPX-thread.

threads::thread_id_ref_type **register_thread**(*threads*::thread_init_data &data, *error_code* &ec = *throws*)

thread_id_ref_type **register_work**(*threads*::thread_init_data &data, *threads*::*thread_pool_base* *pool, *error_code* &ec = *hpx*::*throws*)

Create a new work item using the given data.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **data** – [in] The data to use for creating the thread.
- **pool** – [in] The thread pool to use for launching the work.
- **ec** – [in,out] This represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws
invalid_status – if the runtime system has not been started yet.

thread_id_ref_type **register_work**(*threads*::thread_init_data &data, *error_code* &ec = *throws*)

Create a new work item using the given data on the same thread pool as the calling thread, or on the default thread pool if not on an HPX thread.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise, it throws an instance of hpx::exception.

Parameters

- **data** – [in] The data to use for creating the thread.
- **ec** – [in,out] This represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Throws
invalid_status – if the runtime system has not been started yet.

hpx/threading_base/thread_description.hpp

Defined in header hpx/threading_base/thread_description.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **threads**

Functions

std::ostream &**operator<<**(*std*::ostream&, *thread_description* const&)

std::string **as_string**(*thread_description* const &desc)

threads::*thread_description* **get_thread_description**(thread_id_type const &id, *error_code* &ec = *throws*)

The function get_thread_description is part of the thread related API allows to query the description of one of the threads known to the thread-manager.

> **Note**
>
> As long as *ec* is not pre-initialized to *hpx::throws* this function doesn't throw but returns the result code using the parameter *ec*. Otherwise it throws an instance of hpx::exception.

Parameters

- **id** – [in] The thread id of the thread being queried.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

Returns

This function returns the description of the thread referenced by the *id* parameter. If the thread is not known to the thread-manager the return value will be the string "<unknown>".

threads::*thread_description* **set_thread_description**(thread_id_type const &id, *threads*::*thread_description* const &desc = *threads*::*thread_description*(), *error_code* &ec = *throws*)

threads::*thread_description* **get_thread_lco_description**(thread_id_type const &id, *error_code* &ec = *throws*) *threads*::*thread_description* **set_thread_lco_description**(thread_id_type const &id, *threads*::*thread_description* const &desc = *threads*::*thread_description*(), *error_code* &ec = *throws*)

struct **thread_description**

Public Types

enum class **data_type** : *std*::uint8_t

Values:

enumerator **description**

enumerator **address**

Public Functions

thread_description() noexcept = default

inline constexpr **thread_description**(char const*) noexcept

inline explicit constexpr **thread_description**(*std*::string const&) noexcept

template<typename **F**, typename = *std*::enable_if_t<!*std*::is_same_v<*F*, *thread_description*> && !*traits*::is_action_v<*F*>>>

inline explicit constexpr **thread_description**(*F* const&, char const* = nullptr) noexcept

template<typename **Action**, typename = *std*::enable_if_t<*traits*::is_action_v<*Action*>>>

inline explicit constexpr **thread_description**(*Action*, char const* = nullptr) noexcept

inline explicit constexpr **operator bool**() const noexcept

Public Static Functions

static inline constexpr *data_type* **kind**() noexcept

static inline constexpr char const ***get_description**() noexcept

static inline constexpr *std*::size_t **get_address**() noexcept

static inline constexpr bool **valid**() noexcept

Private Functions

void **init_from_alternative_name**(char const *altname)

namespace **util**

Typedefs

using **instead** = *hpx*::*threads*::*thread_description*

hpx/threading_base/threading_base_fwd.hpp

Defined in header hpx/threading_base/threading_base_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **threads**

Functions

thread_data ***get_self_id_data**() noexcept

The function *get_self_id_data* returns the data of the HPX thread id associated with the current thread (or nullptr if the current thread is not a HPX thread).

thread_self &**get_self**()

The function *get_self* returns a reference to the (OS thread specific) self reference to the current HPX thread.

thread_self ***get_self_ptr**() noexcept

The function *get_self_ptr* returns a pointer to the (OS thread specific) self reference to the current HPX thread.

thread_self_impl_type ***get_ctx_ptr**()

The function *get_ctx_ptr* returns a pointer to the internal data associated with each coroutine.

thread_self ***get_self_ptr_checked**(*error_code* &ec = *throws*)

The function *get_self_ptr_checked* returns a pointer to the (OS thread specific) self reference to the current HPX thread.

thread_id_type **get_self_id**() noexcept

The function *get_self_id* returns the HPX thread id of the current thread (or zero if the current thread is not a HPX thread).

HPX_DEPRECATED_V (2, 0, "hpx::threads::get_outer_self_id is deprecated, use " "hpx::threads::get_self_id instead") inline thread_id_type get_outer_self_id() noexcept

The function *get_outer_self_id* returns the HPX thread id of the current outer thread (or zero if the current thread is not a HPX thread). This now always returns the same as *get_self_id*, even for directly executed threads.

thread_id_type **get_parent_id**() noexcept

The function *get_parent_id* returns the HPX thread id of the current thread's parent (or zero if the current thread is not a HPX thread).

> **Note**
>
> This function will return a meaningful value only if the code was compiled with HPX_HAVE_THREAD_PARENT_REFERENCE being defined.

std::size_t **get_parent_phase**() noexcept

The function *get_parent_phase* returns the HPX phase of the current thread's parent (or zero if the current thread is not a HPX thread).

> **Note**
>
> This function will return a meaningful value only if the code was compiled with HPX_HAVE_THREAD_PARENT_REFERENCE being defined.

std::ptrdiff_t **get_self_stacksize**() noexcept

The function *get_self_stacksize* returns the stack size of the current thread (or zero if the current thread is not a HPX thread).

thread_stacksize **get_self_stacksize_enum**() noexcept

The function *get_self_stacksize_enum* returns the stack size of the /.

std::uint32_t **get_parent_locality_id**() noexcept

The function *get_parent_locality_id* returns the id of the locality of the current thread's parent (or zero if the current thread is not a HPX thread).

> **Note**
>
> This function will return a meaningful value only if the code was compiled with HPX_HAVE_THREAD_PARENT_REFERENCE being defined.

std::uint64_t **get_self_component_id**() noexcept

The function *get_self_component_id* returns the lva of the component the current thread is acting on

> **Note**
>
> This function will return a meaningful value only if the code was compiled with HPX_HAVE_THREAD_TARGET_ADDRESS being defined.

namespace **policies**

namespace **tracing**

hpx::get_worker_thread_num

Defined in header hpx/runtime.hpp[976].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::get_local_worker_thread_num*
- *hpx::get_thread_pool_num*

std::size_t *hpx*::**get_worker_thread_num**() noexcept

Return the number of the current OS-thread running in the runtime instance the current HPX-thread is executed with.

This function returns the zero based index of the OS-thread which executes the current HPX-thread.

> **Note**
>
> The returned value is zero based and its maximum value is smaller than the overall number of OS-threads executed (as returned by *get_os_thread_count()*).

> **Note**
>
> This function needs to be executed on a HPX-thread. It will fail otherwise (it will return -1).

std::size_t *hpx*::**get_worker_thread_num**(*error_code* &ec) noexcept

Return the number of the current OS-thread running in the runtime instance the current HPX-thread is executed with.

This function returns the zero based index of the OS-thread which executes the current HPX-thread.

> **Note**
>
> The returned value is zero based and its maximum value is smaller than the overall number of OS-threads executed (as returned by *get_os_thread_count()*). It will return -1 if the current thread is not a known thread or if the runtime is not in running state.

> **Note**
>
> This function needs to be executed on a HPX-thread. It will fail otherwise (it will return -1).

[976] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

Parameters

ec – [in,out] this represents the error status on exit (obsolete, ignored).

hpx::get_local_worker_thread_num

Defined in header hpx/runtime.hpp[977].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::get_worker_thread_num*
- *hpx::get_thread_pool_num*

std::size_t *hpx*::**get_local_worker_thread_num**() noexcept

Return the number of the current OS-thread running in the current thread pool the current HPX-thread is executed with.

This function returns the zero based index of the OS-thread on the current thread pool which executes the current HPX-thread.

> **Note**
>
> The returned value is zero based and its maximum value is smaller than the number of OS-threads executed on the current thread pool. It will return -1 if the current thread is not a known thread or if the runtime is not in running state.

> **Note**
>
> This function needs to be executed on a HPX-thread. It will fail otherwise (it will return -1).

std::size_t *hpx*::**get_local_worker_thread_num**(*error_code* &ec) noexcept

Return the number of the current OS-thread running in the current thread pool the current HPX-thread is executed with.

This function returns the zero based index of the OS-thread on the current thread pool which executes the current HPX-thread.

> **Note**
>
> The returned value is zero based and its maximum value is smaller than the number of OS-threads executed on the current thread pool. It will return -1 if the current thread is not a known thread or if the runtime is not in running state.

[977] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

> **Note**
>
> This function needs to be executed on a HPX-thread. It will fail otherwise (it will return -1).

Parameters

ec – [in,out] this represents the error status on exit (obsolete, ignored).

hpx::get_thread_pool_num

Defined in header hpx/runtime.hpp[978].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::get_worker_thread_num*
- *hpx::get_local_worker_thread_num*

std::size_t *hpx*::**get_thread_pool_num**() noexcept

Return the number of the current thread pool the current HPX-thread is executed with.

This function returns the zero based index of the thread pool which executes the current HPX-thread.

> **Note**
>
> The returned value is zero based and its maximum value is smaller than the number of thread pools started by the runtime. It will return -1 if the current thread pool is not a known thread pool or if the runtime is not in running state.

> **Note**
>
> This function needs to be executed on a HPX-thread. It will fail otherwise (it will return -1).

std::size_t *hpx*::**get_thread_pool_num**(*error_code* &ec) noexcept

Return the number of the current thread pool the current HPX-thread is executed with.

This function returns the zero based index of the thread pool which executes the current HPX-thread.

> **Note**
>
> The returned value is zero based and its maximum value is smaller than the number of thread pools started by the runtime. It will return -1 if the current thread pool is not a known thread pool or if the runtime is not in running state.

[978] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/full/include/include/hpx/runtime.hpp

Note

This function needs to be executed on a HPX-thread. It will fail otherwise (it will return -1).

Parameters

`ec` – [in,out] this represents the error status on exit (obsolete, ignored).

hpx::scoped_annotation

Defined in header hpx/functional.hpp[979].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

struct **scoped_annotation**

scoped_annotation associates a `name` with a section of code (scope). It can be used to visualize code execution in profiling tools like *Intel VTune*, *Apex Profiler*, etc. That allows analyzing performance to figure out which part(s) of code is (are) responsible for performance degradation, etc.

hpx/threading_base/thread_data.hpp

Defined in header hpx/threading_base/thread_data.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **threads**

Functions

constexpr *thread_data* ***get_thread_id_data**(thread_id_ref_type const &tid) noexcept

constexpr *thread_data* ***get_thread_id_data**(thread_id_type const &tid) noexcept

constexpr *tracing*::*region_init_data* **get_region_init_data**(*thread_data* const*) noexcept

[979] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/functional.hpp

constexpr *tracing*::*fiber_region_init_data* **get_fiber_region_init_data**(*thread_data* const*) noexcept

class **thread_data** : public detail::thread_data_reference_counting

#include <thread_data.hpp> A *thread* is the representation of a HPX thread. It's a first class object in HPX. In our implementation this is a user level thread running on top of one of the OS threads spawned by the *thread-manager*.

A *thread* encapsulates:

- A thread status word (see the functions *thread::get_state* and *thread::set_state*)
- A function to execute (the thread function)
- A frame (in this implementation this is a block of memory used as the threads stack)
- A block of registers (not implemented yet)

Generally, *threads* are not created or executed directly. All functionality related to the management of *threads* is implemented by the thread-manager.

Public Types

using **mutex_type** = *util*::detail::spinlock_no_backoff

Public Functions

thread_data(*thread_data* const&) = delete

thread_data(*thread_data*&&) = delete

thread_data &**operator=**(*thread_data* const&) = delete

thread_data &**operator=**(*thread_data*&&) = delete

inline thread_state **get_state**(*std*::memory_order const order = *std*::memory_order_acquire) const noexcept

The get_state function queries the state of this thread instance.

Note

This function will seldom be used directly. Most of the time the state of a thread will be retrieved by using the function *threadmanager::get_state*.

Returns

This function returns the current state of this thread. It will return one of the values as defined by the *thread_state* enumeration.

inline thread_state **`set_state`**(*thread_schedule_state* const state, *thread_restart_state* state_ex = *thread_restart_state*::*unknown*, *std*::memory_order const load_order = *std*::memory_order_acquire, *std*::memory_order const exchange_order = *std*::memory_order_acq_rel) const noexcept

The set_state function changes the state of this thread instance.

> **Note**
>
> This function will seldom be used directly. Most of the time the state of a thread will have to be changed using the thread-manager. Moreover, changing the thread state using this function does not change its scheduling status. It only sets the thread's status word. To change the thread's scheduling status *threadmanager::set_state* should be used.

Parameters

- **`state`** – [in] The new state to be set for the thread.
- **`state_ex`** – [in]
- **`load_order`** – [in]
- **`exchange_order`** – [in]

inline bool **`set_state_tagged`**(*thread_schedule_state* const newstate, thread_state const &prev_state, thread_state &new_tagged_state, *std*::memory_order const exchange_order = *std*::memory_order_acq_rel) const noexcept

inline bool **`restore_state`**(thread_state const new_state, thread_state const old_state, *std*::memory_order const load_order = *std*::memory_order_relaxed, *std*::memory_order const load_exchange = *std*::memory_order_acq_rel) const noexcept

The restore_state function changes the state of this thread instance depending on its current state. It will change the state atomically only if the current state is still the same as passed as the second parameter. Otherwise, it won't touch the thread state of this instance.

> **Note**
>
> This function will seldom be used directly. Most of the time the state of a thread will have to be changed using the threadmanager. Moreover, changing the thread state using this function does not change its scheduling status. It only sets the thread's status word. To change the thread's scheduling status *threadmanager::set_state* should be used.

Parameters

- **`new_state`** – [in] The new state to be set for the thread.
- **`old_state`** – [in] The old state of the thread which still has to be the current state.
- **`load_order`** – [in]
- **`load_exchange`** – [in]

Returns

This function returns *true* if the state has been changed successfully

inline bool **restore_state**(*thread_schedule_state* const new_state, *thread_restart_state* const state_ex, thread_state old_state, *std*::memory_order const load_exchange = *std*::memory_order_acq_rel) const noexcept

char const ***get_fiber_name**() const noexcept

inline constexpr *thread_priority* **get_priority**() const noexcept

inline void **set_priority**(*thread_priority* const priority) noexcept

inline bool **is_background**() const noexcept

inline void **set_is_background**() noexcept

inline bool **interruption_requested**() const noexcept

inline bool **interruption_enabled**() const noexcept

inline bool **set_interruption_enabled**(bool const enable) noexcept

inline void **interrupt**(bool const flag = true)

bool **interruption_point**(bool throw_on_interrupt = true)

bool **add_thread_exit_callback**(*function*<void()> const &f)

void **run_thread_exit_callbacks**()

void **free_thread_exit_callbacks**()

inline bool **runs_as_child**(*std*::memory_order const mo = *std*::memory_order_acquire) const noexcept

inline bool **is_stackless**() const noexcept

void **destroy_thread**() override

inline constexpr *policies*::scheduler_base ***get_scheduler_base**() const noexcept

inline constexpr *std*::uint16_t **get_last_worker_thread_num**() const noexcept

inline void **set_last_worker_thread_num**(*std*::uint16_t const last_worker_thread_num) noexcept

inline constexpr *std*::ptrdiff_t **get_stack_size**() const noexcept

inline *thread_stacksize* **get_stack_size_enum**() const noexcept

template<typename **ThreadQueue**>

inline constexpr *ThreadQueue* &**get_queue**() noexcept

inline coroutine_type::result_type **operator()**(*hpx*::execution_base::this_thread::detail::agent_storage *agent_storage)

Execute the thread function.

Returns

This function returns the thread state the thread should be scheduled from this point on. The thread manager will use the returned value to set the thread's scheduling status.

inline coroutine_type::result_type **invoke_directly**()

Directly execute the thread function (inline)

Returns

This function returns the thread state the thread should be scheduled from this point on. The thread manager will use the returned value to set the thread's scheduling status.

inline virtual thread_id_type **get_thread_id**() const

inline virtual *std*::size_t **get_thread_phase**() const noexcept

virtual *std*::size_t **get_thread_data**() const = 0

virtual *std*::size_t **set_thread_data**(*std*::size_t data) = 0

virtual void **init**() = 0

virtual void **rebind**(thread_init_data &init_data) = 0

inline *hpx*::*tracing*::*task_timer_data* **get_timer_data**() const noexcept

inline void **set_timer_data**(*hpx*::*tracing*::*task_timer_data* data) noexcept

thread_data(thread_init_data &init_data, void *queue, *std*::ptrdiff_t stacksize, bool is_stackless = false, *thread_id_addref* addref = *thread_id_addref*::*yes*)

~thread_data() override

virtual void **destroy**() noexcept = 0

Public Static Functions

static inline constexpr *std*::uint64_t **get_component_id**() noexcept

Return the id of the component this thread is running in.

static inline constexpr *threads*::*thread_description* **get_description**() noexcept static inline constexpr *threads*::*thread_description* **set_description**(*threads*::*thread_description*) noexcept

static inline constexpr *threads*::*thread_description* **get_lco_description**() noexcept

static inline constexpr *threads*::*thread_description* **set_lco_description**(*threads*::*thread_description*) noexcept

static char const ***get_safe_description**(*threads*::*thread_description* const &description, char const *fallback = "<unknown>") noexcept

static inline constexpr *std*::uint32_t **get_parent_locality_id**() noexcept

Return the locality of the parent thread.

static inline constexpr thread_id_type **get_parent_thread_id**() noexcept

Return the thread id of the parent thread.

static inline constexpr *std*::size_t **get_parent_thread_phase**() noexcept

Return the phase of the parent thread.

static inline constexpr *util*::backtrace const ***get_backtrace**() noexcept

static inline constexpr *util*::backtrace const ***set_backtrace**(*util*::backtrace const*) noexcept

Protected Functions

inline *thread_restart_state* **set_state_ex**(*thread_restart_state* const new_state, *std*::memory_order const load_order = *std*::memory_order_acquire, *std*::memory_order const load_exchange = *std*::memory_order_acq_rel) const noexcept

The set_state function changes the extended state of this thread instance.

> **Note**
>
> This function will seldom be used directly. Most of the time the state of a thread will have to be changed using the threadmanager.

Parameters

- **new_state** – [in] The new extended state to be set for the thread.

- **load_order** – [in]
- **load_exchange** – [in]

void **rebind_base**(thread_init_data &init_data)

Private Types

using **state** = detail::thread_data_state

Private Members

mutable *mutex_type* **mtx_**

std::atomic<bool> **runs_as_child_**

state **state_**

thread_stacksize **stacksize_enum_**

std::int32_t **stacksize_**

std::uint16_t **last_worker_thread_num_**

thread_priority **priority_**

HPX_NO_UNIQUE_ADDRESS hpx::tracing::task_timer_data timer_data_

mutable *std*::atomic<thread_state> **current_state_**

hpx::detail::forward_list<*hpx*::*function*<void()>> **exit_funcs_** *policies*::scheduler_base ***scheduler_base_**

void ***queue_**

threadmanager

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/threadmanager/threadmanager.hpp

Defined in header hpx/threadmanager/threadmanager.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **threads**

class **threadmanager**

#include <threadmanager.hpp> The *thread-manager* class is the central instance of management for all (non-depleted) threads

Public Types

using **notification_policy_type** = *threads*::*policies*::callback_notifier

using **pool_type** = *std*::unique_ptr<*thread_pool_base*>

using **pool_vector** = *std*::vector<*pool_type*>

Public Functions

threadmanager(*hpx*::*util*::*runtime_configuration* &rtcfg_, *notification_policy_type* &notifier, detail::network_background_callback_type network_background_callback = detail::network_background_callback_type())

threadmanager(*threadmanager* const&) = delete

threadmanager(*threadmanager*&&) = delete

threadmanager &**operator=**(*threadmanager* const&) = delete

threadmanager &**operator=**(*threadmanager*&&) = delete

~threadmanager()

void **init**() const

void **create_pools**()

void **print_pools**(*std*::ostream&) const

FIXME move to private and add —hpx:printpools cmd line option.

thread_pool_base &**default_pool**() const

thread_pool_base &**get_pool**(*std*::string const &pool_name) const

thread_pool_base &**get_pool**(pool_id_type const &pool_id) const

thread_pool_base &**get_pool**(*std*::size_t thread_index) const bool **pool_exists**(*std*::string const &pool_name) const

bool **pool_exists**(*std*::size_t pool_index) const

thread_id_ref_type **register_work**(thread_init_data &data, *error_code* &ec = *throws*) const

The function *register_work* adds a new work item to the thread manager. It doesn't immediately create a new *thread*, it just adds the task parameters (function, initial state and description) to the internal management data structures. The thread itself will be created when the number of existing threads drops below the number of threads specified by the constructors max_count parameter.

Parameters

- `data` – [in] The value of this parameter allows to specify a description of the thread to create. This information is used for logging purposes mainly, but might be useful for debugging as well. This parameter is optional and defaults to an empty string.
- `ec` –

void **register_thread**(thread_init_data &data, thread_id_ref_type &id, *error_code* &ec = *throws*) const

The function *register_thread* adds a new work item to the thread manager. It creates a new *thread*, adds it to the internal management data structures, and schedules the new thread, if appropriate.

Parameters

- `data` – [in] The value of this parameter allows to specify a description of the thread to create. This information is used for logging purposes mainly, but might be useful for debugging as well. This parameter is optional and defaults to an empty string.
- `id` – [out] This parameter will hold the id of the created thread. This id is guaranteed to be validly initialized before the thread function is executed.
- `ec` –

bool **run**() const

Run the thread manager's work queue. This function instantiates the specified number of OS threads in each pool. All OS threads are started to execute the function *tfunc*.

Returns

The function returns *true* if the thread manager has been started successfully, otherwise it returns *false*.

void **stop**(bool blocking = true) const

Forcefully stop the thread-manager.

Parameters

`blocking` –

bool **is_busy**() const

bool **is_idle**() const

void **wait**() const

bool **wait_for**(*hpx*::chrono::steady_duration const &rel_time) const

template<typename **StopToken**>

inline bool **wait_for**(*StopToken* &&stop_token, *hpx*::chrono::steady_duration const &rel_time) const

void **suspend**() const

void **resume**() const

hpx::state **status**() const

Return whether the thread manager is still running This returns the "minimal state", i.e. the state of the least advanced thread pool

std::int64_t **get_thread_count**(*thread_schedule_state* state = *thread_schedule_state*::*unknown*, *thread_priority* priority = *thread_priority*::*default_*, *std*::size_t num_thread = static_cast<*std*::size_t>(-1), bool reset = false) const

return the number of HPX-threads with the given state

> **Note**
>
> This function locks the internal OS lock in the thread manager

std::int64_t **get_idle_core_count**() const

mask_type **get_idle_core_mask**() const

std::int64_t **get_background_thread_count**() const

bool **enumerate_threads**(*hpx*::*function*<bool(thread_id_type)> const &f, *thread_schedule_state* state = *thread_schedule_state*::*unknown*) const void **abort_all_suspended_threads**() const

bool **cleanup_terminated**(bool delete_all) const

std::size_t **get_os_thread_count**() const

Return the number of OS threads running in this thread-manager.

This function will return correct results only if the thread-manager is running.

std::thread &**get_os_thread_handle**(*std*::size_t num_thread) const

void **report_error**(*std*::size_t num_thread, *std*::exception_ptr const &e) const

API functions forwarding to notification policy.

This notifies the thread manager that the passed exception has been raised. The exception will be routed through the notifier and the scheduler (which will result in it being passed to the runtime object, which in turn will report it to the console, etc.).

mask_type **get_used_processing_units**() const

Returns the mask identifying all processing units used by this thread manager.

hwloc_bitmap_ptr **get_pool_numa_bitmap**(*std*::string const &pool_name) const

void **set_scheduler_mode**(*threads*::*policies*::scheduler_mode mode, *hpx*::*threads*::mask_cref_type pu_mask) const noexcept

void **add_scheduler_mode**(*threads*::*policies*::scheduler_mode mode, *hpx*::*threads*::mask_cref_type pu_mask) const noexcept

void **add_remove_scheduler_mode**(*threads*::*policies*::scheduler_mode to_add_mode, *threads*::*policies*::scheduler_mode to_remove_mode, *hpx*::*threads*::mask_cref_type pu_mask) const noexcept

void **remove_scheduler_mode**(*threads*::*policies*::scheduler_mode mode, *hpx*::*threads*::mask_cref_type pu_mask) const noexcept

void **reset_thread_distribution**() const noexcept *std*::int64_t **get_queue_length**(bool reset) const

std::int64_t **get_cumulative_duration**(bool reset) const

std::int64_t **get_thread_count_unknown**(bool reset) const

std::int64_t **get_thread_count_active**(bool reset) const

std::int64_t **get_thread_count_pending**(bool reset) const

std::int64_t **get_thread_count_suspended**(bool reset) const

std::int64_t **get_thread_count_terminated**(bool reset) const

std::int64_t **get_thread_count_staged**(bool reset) const

Public Static Functions

static void **init_tss**(*std*::size_t global_thread_num)

static void **deinit_tss**()

Private Types

using **mutex_type** = *std*::mutex

Private Functions

policies::thread_queue_init_parameters **get_init_parameters**() const

void **create_scheduler_user_defined**(*hpx*::*resource*::scheduler_function const&, *thread_pool_init_parameters* const&, *policies*::thread_queue_init_parameters const&)

void **create_scheduler_local**(*thread_pool_init_parameters* const&, *policies*::thread_queue_init_parameters const&, *std*::size_t)

void **create_scheduler_local_priority_fifo**(*thread_pool_init_parameters* const&, *policies*::thread_queue_init_parameters const&, *std*::size_t)

void **create_scheduler_local_priority_fifo_double**(*thread_pool_init_parameters* const&, *policies*::thread_queue_init_parameters const&, *std*::size_t)

void **create_scheduler_local_priority_lifo**(*thread_pool_init_parameters* const&, *policies*::thread_queue_init_parameters const&, *std*::size_t)

void **create_scheduler_static**(*thread_pool_init_parameters* const&, *policies*::thread_queue_init_parameters const&, *std*::size_t)

void **create_scheduler_static_priority**(*thread_pool_init_parameters* const&, *policies*::thread_queue_init_parameters const&, *std*::size_t)

void **create_scheduler_abp_priority_fifo**(*thread_pool_init_parameters* const&, *policies*::thread_queue_init_parameters const&, *std*::size_t)

void **create_scheduler_abp_priority_lifo**(*thread_pool_init_parameters* const&, *policies*::thread_queue_init_parameters const&, *std*::size_t)

void **create_scheduler_shared_priority**(*thread_pool_init_parameters* const&, *policies*::thread_queue_init_parameters const&, *std*::size_t) void **create_scheduler_local_workrequesting_fifo**(*thread_pool_init_parameters* const&, *policies*::thread_queue_init_parameters const&, *std*::size_t)

void **create_scheduler_local_workrequesting_lifo**(*thread_pool_init_parameters* const&, *policies*::thread_queue_init_parameters const&, *std*::size_t)

void **create_scheduler_local_workrequesting_mc**(*thread_pool_init_parameters* const&, *policies*::thread_queue_init_parameters const&, *std*::size_t)

Private Members

mutable *mutex_type* **mtx_**

hpx::*util*::*runtime_configuration* &**rtcfg_**

std::vector<pool_id_type> **threads_lookup_**

pool_vector **pools_**

notification_policy_type &**notifier_**

detail::network_background_callback_type **network_background_callback_**

thrust

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/thrust/algorithms.hpp

Defined in header hpx/thrust/algorithms.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **thrust**

Functions

template<typename **Tag**, typename **ThrustPolicy**, typename ...**Args**>

decltype(auto) **hpx_invoke**(*Tag* tag, *ThrustPolicy* &&policy, *Args*&&... args)

ADL hook dispatching HPX algorithm CPOs to Thrust backends.

When a Thrust execution policy is passed to an HPX algorithm CPO, this hook maps the call to the corresponding Thrust implementation. Async policies wrap the result in a future.

Template Parameters

- **Tag** – The algorithm CPO tag type.
- **ThrustPolicy** – A Thrust execution policy satisfying is_thrust_execution_policy.
- **Args** – Arguments forwarded to the Thrust algorithm.

Parameters

- **tag** – The algorithm CPO tag instance.
- **policy** – Thrust execution policy controlling dispatch.
- **args** – Arguments forwarded to the mapped algorithm.

Returns

The algorithm result, or a future thereof for async policies.

timed_execution

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/timed_execution/timed_execution_fwd.hpp

Defined in header hpx/timed_execution/timed_execution_fwd.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **execution**

Variables

constexpr struct *hpx*::*parallel*::*execution*::*post_at_t* **post_at**

constexpr struct *hpx*::*parallel*::*execution*::*post_after_t* **post_after**

constexpr struct *hpx*::*parallel*::*execution*::*async_execute_at_t* **async_execute_at**

constexpr struct *hpx*::*parallel*::*execution*::*async_execute_after_t* **async_execute_after**

constexpr struct *hpx*::*parallel*::*execution*::*sync_execute_at_t* **sync_execute_at**

constexpr struct *hpx*::*parallel*::*execution*::*sync_execute_after_t* **sync_execute_after**

struct **post_at_t**

#include <timed_execution_fwd.hpp> Customization point of asynchronous fire & forget execution agent creation supporting timed execution.

This asynchronously (fire & forget) creates a single function invocation f() using the associated executor at the given point in time.

> **Note**
>
> This calls exec.post_at(abs_time, f, ts…), if available, otherwise it emulates timed scheduling by delaying calling execution::post() on the underlying non-time-scheduled execution agent.

Param exec
[in] The executor object to use for scheduling of the function *f*.

Param abs_time
[in] The point in time the given function should be scheduled at to run.

Param f
[in] The function which will be scheduled using the given executor.

Param ts...
[in] Additional arguments to use to invoke *f*.

Public Functions

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *hpx*::chrono::steady_time_point const &abs_time, *F* &&f, *Ts*&&... ts) const

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *hpx*::chrono::steady_time_point const &abs_time, *F* &&f, *Ts*&&... ts) const

struct **post_after_t**

#include <timed_execution_fwd.hpp> Customization point of asynchronous fire & forget execution agent creation supporting timed execution.

This asynchronously (fire & forget) creates a single function invocation f() using the associated executor at the given point in time.

> **Note**
>
> This calls exec.post_after(rel_time, f, ts...), if available, otherwise it emulates timed scheduling by delaying calling execution::post() on the underlying non-time-scheduled execution agent.

Param exec
[in] The executor object to use for scheduling of the function *f*.

Param rel_time
[in] The duration of time after which the given function should be scheduled to run.

Param f
[in] The function which will be scheduled using the given executor.

Param ts...
[in] Additional arguments to use to invoke *f*.

Public Functions

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *hpx*::chrono::steady_duration const &rel_time, *F* &&f, *Ts*&&... ts) const

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *hpx*::chrono::steady_duration const &rel_time, *F* &&f, *Ts*&&... ts) const

struct **async_execute_at_t**

#include <timed_execution_fwd.hpp> Customization point of asynchronous execution agent creation supporting timed execution.

This asynchronously creates a single function invocation f() using the associated executor at the given point in time.

> **Note**
>
> This calls exec.async_execute_at(abs_time, f, ts…), if available, otherwise it emulates timed scheduling by delaying calling execution::async_execute() on the underlying non-time-scheduled execution agent.

Param exec
[in] The executor object to use for scheduling of the function *f*.

Param abs_time
[in] The point in time the given function should be scheduled at to run.

Param f
[in] The function which will be scheduled using the given executor.

Param ts…
[in] Additional arguments to use to invoke *f*.

Return
f(ts…)'s result through a future

Public Functions

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *hpx*::chrono::steady_time_point const &abs_time, *F* &&f, *Ts*&&... ts) const

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *hpx*::chrono::steady_time_point const &abs_time, *F* &&f, *Ts*&&... ts) const

struct **async_execute_after_t**

#include <timed_execution_fwd.hpp> Customization point of asynchronous execution agent creation supporting timed execution.

This asynchronously creates a single function invocation f() using the associated executor at the given point in time.

> **Note**
>
> This calls exec.async_execute_after(rel_time, f, ts…), if available, otherwise it emulates timed scheduling by delaying calling execution::async_execute() on the underlying non-time-scheduled execution agent.

Param exec
[in] The executor object to use for scheduling of the function *f*.

Param rel_time
[in] The duration of time after which the given function should be scheduled to run.

Param f
[in] The function which will be scheduled using the given executor.

Param ts…
[in] Additional arguments to use to invoke *f*.

Return
f(ts…)'s result through a future

Public Functions

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *hpx*::chrono::steady_duration const &rel_time, *F* &&f, *Ts*&&... ts) const

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *hpx*::chrono::steady_duration const &rel_time, *F* &&f, *Ts*&&... ts) const

struct **sync_execute_at_t**

#include <timed_execution_fwd.hpp> Customization point of synchronous execution agent creation supporting timed execution.

This synchronously creates a single function invocation f() using the associated executor at the given point in time.

> **Note**
>
> This calls exec.sync_execute_at(abs_time, f, ts…), if available, otherwise it emulates timed scheduling by delaying calling execution::sync_execute() on the underlying non-time-scheduled execution agent.

Param exec
[in] The executor object to use for scheduling of the function *f*.

Param abs_time
[in] The point in time the given function should be scheduled at to run.

Param f
[in] The function which will be scheduled using the given executor.

Param ts…
[in] Additional arguments to use to invoke *f*.

Return
f(ts…)'s result

Public Functions

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *hpx*::chrono::steady_time_point const &abs_time, *F* &&f, *Ts*&&... ts) const

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *hpx*::chrono::steady_time_point const &abs_time, *F* &&f, *Ts*&&... ts) const

struct **sync_execute_after_t**

#include <timed_execution_fwd.hpp> Customization point of synchronous execution agent creation supporting timed execution.

This synchronously creates a single function invocation f() using the associated executor at the given point in time.

> **Note**
>
> This calls exec.sync_execute_after(rel_time, f, ts…), if available, otherwise it emulates timed scheduling by delaying calling execution::sync_execute() on the underlying non-time-scheduled execution agent.

Param exec
[in] The executor object to use for scheduling of the function *f*.

Param rel_time
[in] The duration of time after which the given function should be scheduled to run.

Param f
[in] The function which will be scheduled using the given executor.

Param ts…
[in] Additional arguments to use to invoke *f*.

Return
f(ts…)'s result

Public Functions

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *hpx*::chrono::steady_duration const &rel_time, *F* &&f, *Ts*&&... ts) const

template<executor_any **Executor**, typename **F**, typename ...**Ts**>

inline decltype(auto) **operator()**(*Executor* &&exec, *hpx*::chrono::steady_duration const &rel_time, *F* &&f, *Ts*&&... ts) const

hpx/timed_execution/timed_executors.hpp

Defined in header hpx/timed_execution/timed_executors.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **execution**

namespace **experimental**

namespace **parallel**

namespace **execution**

Typedefs

using **sequenced_timed_executor** = *timed_executor*<*hpx*::*execution*::*sequenced_executor*>

using **parallel_timed_executor** = *timed_executor*<*hpx*::*execution*::*parallel_executor*>

template<executor_any **BaseExecutor**>

struct **timed_executor**

Public Types

using **base_executor_type** = *std*::decay_t<*BaseExecutor*>

using **execution_category** = *hpx*::*traits*::executor_execution_category_t<*base_executor_type*>

using **parameters_type** = *hpx*::*traits*::executor_parameters_type_t<*base_executor_type*>

Public Functions

inline explicit **timed_executor**(*hpx*::chrono::steady_time_point const &abs_time)

inline explicit **timed_executor**(*hpx*::chrono::steady_duration const &rel_time)

template<typename **Executor**>

inline **timed_executor**(*Executor* &&exec, *hpx*::chrono::steady_time_point const &abs_time)

template<typename **Executor**>

inline **timed_executor**(*Executor* &&exec, *hpx*::chrono::steady_duration const &rel_time)

template<typename **F**, typename ...**Ts**>

inline decltype(auto) **sync_execute**(*F* &&f, *Ts*&&... ts) const

template<typename **F**, typename ...**Ts**>

inline decltype(auto) **async_execute**(*F* &&f, *Ts*&&... ts) const

template<typename **F**, typename ...**Ts**>

inline decltype(auto) **post**(*F* &&f, *Ts*&&... ts) const

template<typename **Tag**, typename **Property**>

inline *timed_executor* **query**(*Tag*, *Property* &&prop) const

template<typename **Tag**>

inline decltype(auto) **query**(*Tag* tag) const

Private Members

BaseExecutor **exec_**

std::chrono::steady_clock::time_point **execute_at_**

hpx/timed_execution/timed_execution.hpp

Defined in header hpx/timed_execution/timed_execution.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **parallel**

namespace **execution**

hpx/timed_execution/traits/is_timed_executor.hpp

Defined in header hpx/timed_execution/traits/is_timed_executor.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

Variables

template<typename Executor> concept **timed_executor** = hpx::traits::is_timed_executor_v<Executor>

namespace **parallel**

namespace **execution**

Typedefs

template<typename **T**>

using **is_timed_executor_t** = typename *is_timed_executor*<*T*>::type

Variables

template<typename **T**>

constexpr bool **is_timed_executor_v** = *is_timed_executor*<*T*>::value

template<typename **T**>

struct **is_timed_executor** : public detail::is_timed_executor<*std*::decay_t<*T*>>

namespace **traits**

Variables

template<typename **T**>

constexpr bool **is_timed_executor_v** = *is_timed_executor*<*T*>::value

template<typename **Executor**, typename **Enable** = void>

struct **is_timed_executor** : public *hpx*::*parallel*::*execution*::*is_timed_executor*<*Executor*>

timing

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::chrono::high_resolution_clock

Defined in header hpx/chrono.hpp[980].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

struct **high_resolution_clock**

Class *hpx::chrono::high_resolution_clock* represents the clock with the smallest tick period provided by the implementation. It may be an alias of `std::chrono::system_clock` or `std::chrono::steady_clock`, or a third, independent clock. *hpx::chrono::high_resolution_clock* meets the requirements of *TrivialClock*.

hpx::chrono::high_resolution_timer

Defined in header hpx/chrono.hpp[981].

See *Public API* for a list of names and headers that are part of the public *HPX* API.

class **high_resolution_timer**

high_resolution_timer is a timer object which measures the elapsed time

topology

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/topology/cpu_mask.hpp

Defined in header hpx/topology/cpu_mask.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **threads**

[980] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/chrono.hpp
[981] http://github.com/TheHPXProject/hpx/blob/19fe7fe3cad79cf1ed28b4a7ede713343209d8d7/libs/core/include_local/include/hpx/chrono.hpp

hpx/topology/topology.hpp

Defined in header hpx/topology/topology.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **threads**

Typedefs

using **hwloc_bitmap_ptr** = *std*::shared_ptr<*hpx_hwloc_bitmap_wrapper*>

Enums

enum **hpx_hwloc_membind_policy**

Please see hwloc documentation for the corresponding enums HWLOC_MEMBIND_XXX.

Values:

enumerator **membind_default**

enumerator **membind_firsttouch**

enumerator **membind_bind**

enumerator **membind_interleave**

enumerator **membind_replicate**

enumerator **membind_nexttouch**

enumerator **membind_mixed**

enumerator **membind_user**

Functions

topology &**create_topology**()

inline *topology* const &**get_topology**()

Get the global topology instance.

inline *std*::size_t **get_memory_page_size**()

struct **hpx_hwloc_bitmap_wrapper**

Public Functions

HPX_NON_COPYABLE(*hpx_hwloc_bitmap_wrapper*)

inline **hpx_hwloc_bitmap_wrapper**() noexcept

inline explicit **hpx_hwloc_bitmap_wrapper**(void *bmp) noexcept

inline **~hpx_hwloc_bitmap_wrapper**()

inline void **reset**(hwloc_bitmap_t bmp) noexcept

inline explicit constexpr **operator bool**() const noexcept

inline hwloc_bitmap_t **get_bmp**() const noexcept

Private Members

hwloc_bitmap_t **bmp_**

Friends

friend *std*::ostream &**operator<<**(*std*::ostream &os, *hpx_hwloc_bitmap_wrapper* const *bmp)

struct **topology**

Public Functions

topology()

topology(*topology* const&) = delete

topology(*topology*&&) = delete

topology &**operator=**(*topology* const&) = delete

topology &**operator=**(*topology*&&) = delete

~topology()

inline *std*::size_t **get_socket_number**(*std*::size_t num_thread, [[maybe_unused]] *error_code* &ec = *throws*) const noexcept

Return the Socket number of the processing unit the given thread is running on.

Parameters

- **num_thread** – [in]
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

inline *std*::size_t **get_numa_node_number**(*std*::size_t num_thread, [[maybe_unused]] *error_code* &ec = *throws*) const noexcept

Return the NUMA node number of the processing unit the given thread is running on.

Parameters

- **num_thread** – [in]
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

inline *std*::size_t **get_numa_distance**(*std*::size_t node_a, *std*::size_t node_b) const noexcept

Return the hardware latency distance between two NUMA nodes. Values are sourced from hwloc's distance matrix when available; otherwise a standard default (10 = local, 20 = remote) is returned.

Parameters

- **node_a** – [in] logical index of the first NUMA node
- **node_b** – [in] logical index of the second NUMA node

mask_cref_type **get_machine_affinity_mask**(*error_code* &ec = *throws*) const noexcept

Return a bit mask where each set bit corresponds to a processing unit available to the application.

Parameters

ec – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

mask_type **get_service_affinity_mask**(mask_cref_type used_processing_units, *error_code* &ec = *throws*) const

Return a bit mask where each set bit corresponds to a processing unit available to the service threads in the application.

Parameters

- **used_processing_units** – [in] This is the mask of processing units which are not available for service threads.
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

mask_cref_type **get_socket_affinity_mask**(*std*::size_t num_thread, *error_code* &ec = *throws*) const

Return a bit mask where each set bit corresponds to a processing unit available to the given thread inside the socket it is running on.

Parameters

- **num_thread** – [in]
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

mask_cref_type **get_numa_node_affinity_mask**(*std*::size_t num_thread, *error_code* &ec = *throws*) const

Return a bit mask where each set bit corresponds to a processing unit available to the given thread inside the NUMA domain it is running on.

Parameters

- **num_thread** – [in]
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

mask_cref_type **get_core_affinity_mask**(*std*::size_t num_thread, *error_code* &ec = *throws*) const

Return a bit mask where each set bit corresponds to a processing unit available to the given thread inside the core it is running on.

Parameters

- **num_thread** – [in]
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

mask_cref_type **get_thread_affinity_mask**(*std*::size_t num_thread, *error_code* &ec = *throws*) const

Return a bit mask where each set bit corresponds to a processing unit available to the given thread.

Parameters

- **num_thread** – [in]
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

void **set_thread_affinity_mask**(mask_cref_type mask, *error_code* &ec = *throws*) const

Use the given bit mask to set the affinity of the given thread. Each set bit corresponds to a processing unit the thread will be allowed to run on.

> **Note**
>
> Use this function on systems where the affinity must be set from inside the thread itself.

Parameters

- **mask** – [in]
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

mask_type **get_thread_affinity_mask_from_lva**(void const *lva, *error_code* &ec = *throws*) const

Return a bit mask where each set bit corresponds to a processing unit co-located with the memory the given address is currently allocated on.

Parameters

- **lva** – [in]
- **ec** – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

void **print_affinity_mask**(*std*::ostream &os, *std*::size_t num_thread, mask_cref_type m, *std*::string const &pool_name) const

Prints the given mask *m* to os in a human-readable form.

bool **reduce_thread_priority**(*error_code* &ec = *throws*) const

Reduce thread priority of the current thread.

Parameters

ec – [in,out] this represents the error status on exit, if this is pre-initialized to *hpx::throws* the function will throw on error instead.

std::size_t **get_number_of_sockets**() const

Return the number of available NUMA domains.

std::size_t **get_number_of_numa_nodes**() const

Return the number of available NUMA domains.

std::size_t **get_number_of_cores**() const

Return the number of available cores.

std::size_t **get_number_of_pus**() const noexcept

Return the number of available hardware processing units.

std::size_t **get_number_of_numa_node_cores**(*std*::size_t numa) const

Return number of cores in given numa domain.

std::size_t **get_number_of_numa_node_pus**(*std*::size_t numa) const

Return number of processing units in a given numa domain.

std::size_t **get_number_of_socket_pus**(*std*::size_t socket) const

Return number of processing units in a given socket.

std::size_t **get_number_of_core_pus**(*std*::size_t core) const

Return number of processing units in given core.

std::size_t **get_number_of_socket_cores**(*std*::size_t socket) const

Return number of cores units in given socket.

inline *std*::size_t **get_core_number**(*std*::size_t num_thread, *error_code*& = *throws*) const

std::size_t **get_pu_number**(*std*::size_t num_core, *std*::size_t num_pu, *error_code* &ec = *throws*) const

std::size_t **get_cache_size**(mask_cref_type mask, int level) const

Return the size of the cache associated with the given mask.

mask_type **get_cpubind_mask**(*error_code* &ec = *throws*) const

mask_type **get_cpubind_mask**(*std*::thread &handle, *error_code* &ec = *throws*) const

hwloc_bitmap_ptr **cpuset_to_nodeset**(mask_cref_type mask) const

convert a cpu mask into a numa node mask in hwloc bitmap form

void **write_to_log**() const

void *__allocate__(*std*::size_t len) const

This is equivalent to malloc(), except that it tries to allocate page-aligned memory from the OS.

void *__allocate_membind__(*std*::size_t len, *hwloc_bitmap_ptr* const &bitmap, *hpx_hwloc_membind_policy* policy, int flags) const

allocate memory with binding to a numa node set as specified by the policy and flags (see hwloc docs)

threads::mask_type **get_area_membind_nodeset**(void const *addr, *std*::size_t len) const

bool **set_area_membind_nodeset**(void const *addr, *std*::size_t len, void *nodeset) const

int **get_numa_domain**(void const *addr) const

void **deallocate**(void *addr, *std*::size_t len) const noexcept

Free memory that was previously allocated by allocate.

void **print_hwloc**(*std*::ostream&) const

mask_type **init_socket_affinity_mask_from_socket**(*std*::size_t num_socket) const

mask_type **init_numa_node_affinity_mask_from_numa_node**(*std*::size_t num_numa_node) const

mask_type **init_core_affinity_mask_from_core**(*std*::size_t num_core, mask_cref_type default_mask = *empty_mask*) const mask_type **init_thread_affinity_mask**(*std*::size_t num_thread) const

mask_type **init_thread_affinity_mask**(*std*::size_t num_core, *std*::size_t num_pu) const

hwloc_bitmap_t **mask_to_bitmap**(mask_cref_type mask, hwloc_obj_type_t htype, unsigned *count = nullptr) const

mask_type **bitmap_to_mask**(hwloc_bitmap_t bitmap, hwloc_obj_type_t htype) const

Public Static Functions

static void **print_vector**(*std*::ostream &os, *std*::vector<*std*::size_t> const &v)

static void **print_mask_vector**(*std*::ostream &os, *std*::vector<mask_type> const &v)

Private Types

using **mutex_type** = *hpx*::*util*::spinlock

Private Functions

std::size_t **init_node_number**(*std*::size_t num_thread, hwloc_obj_type_t type) const

inline *std*::size_t **init_socket_number**(*std*::size_t num_thread) const

std::size_t **init_numa_node_number**(*std*::size_t num_thread) const

inline *std*::size_t **init_core_number**(*std*::size_t num_thread) const void **extract_node_mask**(hwloc_obj_t parent, mask_type &mask) const

std::size_t **get_number_of_core_pus_locked**(*std*::size_t core) const

std::size_t **extract_node_count**(hwloc_obj_t parent, hwloc_obj_type_t type, *std*::size_t count) const

std::size_t **extract_node_count_locked**(hwloc_obj_t parent, hwloc_obj_type_t type, *std*::size_t count) const

mask_type **init_machine_affinity_mask**() const

inline mask_type **init_socket_affinity_mask**(*std*::size_t num_thread) const

inline mask_type **init_numa_node_affinity_mask**(*std*::size_t num_thread) const

inline mask_type **init_core_affinity_mask**(*std*::size_t num_thread) const

void **init_num_of_pus**()

hwloc_obj_t **get_pu_obj**(*std*::size_t num_pu) const

void **init_numa_node_distances**()

Private Members

hwloc_topology_t **topo** = nullptr

std::size_t **num_of_pus_** = 0

bool **use_pus_as_cores_** = false

mutable *mutex_type* **topo_mtx**

std::vector<*std*::size_t> **socket_numbers_**

std::vector<*std*::size_t> **numa_node_numbers_**

std::vector<*std*::size_t> **core_numbers_**

mask_type **machine_affinity_mask_** = mask_type()

std::vector<mask_type> **socket_affinity_masks_**

std::vector<mask_type> **numa_node_affinity_masks_**

std::vector<mask_type> **core_affinity_masks_**

std::vector<mask_type> **thread_affinity_masks_**

std::vector<*std*::vector<*std*::size_t>> **numa_node_distances_**

Private Static Attributes

static mask_type **empty_mask**

static *std*::size_t **memory_page_size_** static constexpr *std*::size_t **pu_offset** = 0

static constexpr *std*::size_t **core_offset** = 0

Friends

friend *std*::size_t **get_memory_page_size**()

tracing

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/tracing/tracing.hpp

Defined in header hpx/tracing/tracing.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **threads**

namespace **tracing**

Functions

template<typename **ThreadData**>

constexpr void **task_created**([[maybe_unused]] *ThreadData* *thrdptr, [[maybe_unused]] void const *parent_task_id = nullptr) noexcept

template<typename **ThreadData**>

constexpr void **task_executing**([[maybe_unused]] *ThreadData* *thrdptr) noexcept

template<typename **ThreadId**>

constexpr void **`task_yielded`**([[maybe_unused]] *ThreadId* const &id) noexcept

template<typename **`ThreadId`**, typename **`String`**>

constexpr void **`task_suspended`**([[maybe_unused]] *ThreadId* const &id, [[maybe_unused]] *String* const &reason) noexcept

template<typename **`ThreadId`**, typename **`StateX`**>

constexpr void **`task_resumed`**([[maybe_unused]] *ThreadId* const &id, [[maybe_unused]] *StateX* statex) noexcept

template<typename **`ThreadData`**>

constexpr void **`task_completed`**([[maybe_unused]] *ThreadData* *thrdptr) noexcept

template<typename **`ThreadId`**>

constexpr void **`work_stolen`**([[maybe_unused]] *std*::size_t thief_id, [[maybe_unused]] *std*::size_t victim_id, [[maybe_unused]] *ThreadId* const &thrd) noexcept

template<typename **`Range`**>

constexpr void **`work_stolen_range`**([[maybe_unused]] *std*::size_t thief_id, [[maybe_unused]] *std*::size_t victim_id, [[maybe_unused]] *Range* const &tasks) noexcept

struct **`lock_context`**

#include <tracing.hpp> Context for annotating lock acquisitions and releases.

This struct is used to provide tracing annotations for synchronization primitives such as spinlocks and mutexes. It registers the creation, acquisition, and release of locks to the active tracing backend.

Public Functions

explicit **`lock_context`**(char const *name = nullptr, void const *addr = nullptr) noexcept

Constructs a lock context.

Parameters

- **`name`** – The name or description of the lock.
- **`addr`** – The memory address of the lock, used to track it uniquely.

explicit **lock_context**(char const *prefix, char const *suffix, void const *addr = nullptr) noexcept

Constructs a lock context with a dynamic name.

Parameters

- **prefix** – The prefix for the lock name.
- **suffix** – The suffix for the lock name.
- **addr** – The memory address of the lock, used to track it uniquely.

bool **before_lock**() const noexcept

Called immediately before attempting to acquire the lock.

Returns

true if the prepare event was recorded and after_lock should be called.

void **after_lock**() const noexcept

Called immediately after the lock is successfully acquired.

void **after_try_lock**(bool success) const noexcept

Called after a try-lock operation completes.

Parameters

success – true if the lock was acquired, false if it failed.

void **before_unlock**() const noexcept

Called immediately before releasing the lock.

void **after_unlock**() const noexcept

Called immediately after releasing the lock.

namespace **util**

namespace **external_timer**

hpx/tracing/backends/empty.hpp

Defined in header hpx/tracing/backends/empty.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **tracing**

Typedefs

`bool(*)() enable_parent_task_handler_type`

Functions

inline constexpr *annotation_handle* **create_annotation_handle**(char const*) noexcept

inline constexpr void **set_thread_name**(char const *name) noexcept

Set the OS thread name in the profiler timeline.

inline constexpr char const ***rename_region**(char const *name) noexcept

Rename the currently active region on this thread; returns the previous name, or the caller's input if no rename occurred.

constexpr *task_timer_data* **create_task_timer**(*threads*::*thread_description* const&, *std*::uint32_t, *threads*::*thread_id* const&) noexcept

constexpr void **update_task_timer**(*task_timer_data*&, char const*) noexcept

constexpr void **task_staged**(char const*, void const* = nullptr) noexcept

constexpr void **task_created**(char const*, void const*, void const* = nullptr) noexcept

constexpr void **task_executing**(void const*, char const*, *std*::size_t) noexcept

constexpr void **task_yielded**(void const*, char const*) noexcept

constexpr void **task_suspended**(void const*, char const*, char const* = nullptr) noexcept

constexpr void **task_resumed**(void const*, char const*, char const* = nullptr) noexcept constexpr void **`task_completed`**(void const*, char const*) noexcept

constexpr void **`task_deleted`**(void const*) noexcept

constexpr void **`future_fulfilled`**(void const*, char const* = nullptr) noexcept

Producer-side signal: future state fulfilled (no-op stub).

Producer-side signal: future state fulfilled (no-op stub for APEX).

Producer-side signal: future state fulfilled (no-op stub for ITTNotify).

constexpr void **`future_exception_set`**(void const*, char const* = nullptr) noexcept

Producer-side signal: future exception set (no-op stub).

Producer-side signal: future exception set (no-op stub for APEX).

Producer-side signal: future exception set (no-op stub for ITTNotify).

constexpr void **`continuation_run`**(void const* = nullptr) noexcept

Consumer-side signal: continuation started (no-op stub).

Consumer-side signal: continuation started (no-op stub for APEX).

Consumer-side signal: continuation started (no-op stub for ITTNotify).

constexpr void **`continuation_finished`**(void const* = nullptr) noexcept

Consumer-side signal: continuation finished (no-op stub).

Consumer-side signal: continuation finished (no-op stub for APEX).

Consumer-side signal: continuation finished (no-op stub for ITTNotify).

constexpr void **`handle_on_completed_fired`**(void const* = nullptr) noexcept

Consumer-side signal: handle_on_completed fired (no-op stub).

Consumer-side signal: handle_on_completed fired (no-op stub for APEX).

Consumer-side signal: handle_on_completed fired (no-op stub for ITTNotify).

constexpr void **`work_stolen`**(*std*::size_t, *std*::size_t, void const*, char const* = nullptr) noexcept

Signal emitted when a worker thread steals a task from another worker.

inline constexpr void **`frame_mark`**(char const* = nullptr) noexcept

Frame boundary marker (no-op stub).

Emit a frame/stage boundary marker in Tracy timeline.

Frame boundary marker (no-op stub for APEX).

Frame boundary marker (no-op stub for ITTNotify).

Parameters

`name` – Optional name for the frame/stage.

inline constexpr void **os_thread_sleep**(*std*::size_t num_thread) noexcept

Point message emitted just before an OS worker thread suspends on its condition variable.

Parameters

`num_thread` – Index of the worker thread entering sleep.

constexpr void **tracing_init**(char const*, int, char**, *std*::uint32_t = 0, *std*::uint32_t = 1) noexcept

constexpr void **tracing_finalize**() noexcept

constexpr void **register_thread**(char const*) noexcept

constexpr void **create_counter**(*std*::string const&, *std*::string const&) noexcept

constexpr void **sample_counter**(*std*::string const&, *std*::string const&, double) noexcept

inline constexpr void **send_parcel**(*std*::uint64_t, *std*::uint64_t, *std*::uint64_t, *std*::uint64_t) noexcept

Parcel-send signal: parcel handed to parcelport (no-op stub).

Parcel-send signal: a parcel has been handed to the parcelport for transmission.

Parcel-send signal: parcel handed to parcelport (APEX shim).

Parcel-send signal: parcel handed to parcelport (no-op stub for ITTNotify).

Parameters

- `tag_msb` – High 64 bits of the parcel id.
- `tag_lsb` – Low 64 bits of the parcel id. Together with tag_msb they form the full 128-bit id used to pair events across localities; lsb alone restarts per locality and would produce false matches.
- `size` – Wire size of the parcel in bytes.
- `target_locality_id` – Destination locality id, or `naming::invalid_locality_id` if unknown.

inline constexpr void **recv_parcel**(*std*::uint64_t, *std*::uint64_t, *std*::uint64_t) noexcept

Parcel-receive signal: parcel arrived (no-op stub).

Parcel-receive signal: a parcel has arrived and its header has been deserialized.

Parcel-receive signal: parcel arrived (APEX shim).

Parcel-receive signal: parcel arrived (no-op stub for ITTNotify).

Parameters

- `tag_msb` – High 64 bits of the parcel id.
- `tag_lsb` – Low 64 bits of the parcel id.
- `source_locality_id` – Sending locality id, or `naming::invalid_locality_id` when the parcel has no source id yet (pre-AGAS handshake parcels).

inline constexpr void **parcel_scheduled**(*std*::uint64_t, *std*::uint64_t, *std*::uint64_t, *std*::uint64_t) noexcept

Parcel-scheduled signal: action entered local scheduler (no-op stub).

Parcel-scheduled signal: an action carried by a parcel has been placed onto the local scheduler. Fires for deferred receives (zero-copy or multi-parcel message), for AGAS-routed parcels, and for parcels the AGAS re-route chain delivered after a migration detection. Does not fire for the non-deferred path where load_schedule schedules inline.

Parcel-scheduled signal: action entered local scheduler (no-op stub for APEX).

Parcel-scheduled signal: action entered local scheduler (no-op stub for ITTNotify).

Parameters

- `tag_msb` – High 64 bits of the parcel id.
- `tag_lsb` – Low 64 bits of the parcel id.
- `source_locality_id` – Sending locality id (see recv_parcel).
- `source_thread_id` – Parent HPX thread id captured on the sender; may be 0 when unavailable.

constexpr void **set_enable_parent_task_handler**(enable_parent_task_handler_type) noexcept

struct **lock_context**

#include <tracing.hpp>

struct **annotation_handle**

struct **region_init_data**

struct **loop_context**

Public Functions

inline explicit constexpr **loop_context**() noexcept

~loop_context() = default

loop_context(*loop_context* const&) = delete

loop_context &**operator=**(*loop_context* const&) = delete

struct **region**

Public Functions

inline explicit constexpr **region**(*loop_context*&, *region_init_data* const&, *std*::size_t) noexcept

struct **mark_event**

Public Functions

inline explicit constexpr **mark_event**(char const*) noexcept

struct **fiber_region_init_data**

struct **fiber_region**

Public Functions

inline explicit constexpr **fiber_region**(*fiber_region_init_data* const&, *std*::size_t) noexcept

struct **fiber_suspend_region**

Public Functions

inline explicit constexpr **fiber_suspend_region**(char const*) noexcept

struct **background_work_region**

Public Functions

inline explicit constexpr **background_work_region**(*std*::size_t = 0) noexcept

struct **task_timer_data**

Public Static Functions

static inline constexpr bool **valid**() noexcept

struct **scoped_task_timer**

Public Functions

inline explicit constexpr **scoped_task_timer**(*task_timer_data*) noexcept

inline constexpr void **stop**() noexcept inline constexpr void **yield**() noexcept

template<typename **T**, typename **State**>

inline constexpr void **handle_post_execution**(*T* *thrdptr, *State* s) noexcept

hpx/tracing/backends/ittnotify.hpp

Defined in header hpx/tracing/backends/ittnotify.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **tracing**

Typedefs

typedef *util*::itt::string_handle **annotation_handle**

struct **lock_context**

#include <tracing.hpp>

struct **region_init_data**

struct **loop_context**

struct **region**

struct **mark_event**

struct **fiber_region_init_data**

struct **fiber_region**

struct **fiber_suspend_region**

struct **background_work_region**

struct **task_timer_data**

struct **scoped_task_timer**

hpx/tracing/backends/apex.hpp

Defined in header hpx/tracing/backends/apex.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **tracing**

struct **lock_context**

#include <tracing.hpp>

struct **region_init_data**

struct **loop_context**

struct **region**

struct **mark_event**

struct **fiber_region_init_data**

struct **fiber_region**

struct **fiber_suspend_region**

struct **background_work_region**

struct **task_timer_data**

struct **scoped_task_timer**

hpx/tracing/backends/tracy.hpp

Defined in header hpx/tracing/backends/tracy.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **tracing**

struct **lock_context**

#include <tracing.hpp>

struct **region_init_data**

struct **loop_context**

struct **region**

struct `mark_event`

struct `fiber_region_init_data`

struct `fiber_region`

struct `fiber_suspend_region`

struct `background_work_region`

struct `task_timer_data`

struct `scoped_task_timer`

tracy

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/tracy/tracy_tls.hpp

Defined in header hpx/tracy/tracy_tls.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/tracy/tracy.hpp

Defined in header hpx/tracy/tracy.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

util

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/util/sed_transform.hpp

Defined in header hpx/util/sed_transform.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

Functions

bool **parse_sed_expression**(*std*::string const &input, *std*::string &search, *std*::string &replace)

Parse a sed command.

> **Note**
>
> Currently, only supports search and replace syntax (s/search/replace/)

Parameters

- **input** – [in] The content to parse.
- **search** – [out] If the parsing is successful, this string is set to the search expression.
- **replace** – [out] If the parsing is successful, this string is set to the replace expression.

Returns

true if the parsing was successful, false otherwise.

struct **sed_transform**

#include <sed_transform.hpp> An unary function object which applies a sed command to its subject and returns the resulting string.

> **Note**
>
> Currently, only supports search and replace syntax (s/search/replace/)

Public Functions

sed_transform(*std*::string const &search, *std*::string replace)

explicit **sed_transform**(*std*::string const &expression)

std::string **operator()**(*std*::string const &input) const

inline explicit **operator bool**() const noexcept

inline bool **operator!**() const noexcept

Private Members

std::shared_ptr<command> **command_**

hpx/util/insert_checked.hpp

Defined in header hpx/util/insert_checked.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **util**

Functions

template<typename **Iterator**>

constexpr bool **insert_checked**(*std*::pair<*Iterator*, bool> const &r) noexcept

Helper function for writing predicates that test whether an std::map insertion succeeded. This inline template function negates the need to explicitly write the sometimes lengthy std::pair<Iterator, bool> type.

Parameters

r – [in] The return value of a std::map insert operation.

Returns

This function returns **r.second**.

template<typename **Iterator**>

bool **insert_checked**(*std*::pair<*Iterator*, bool> const &r, *Iterator* &it)

Helper function for writing predicates that test whether an std::map insertion succeeded. This inline template function negates the need to explicitly write the sometimes lengthy std::pair<Iterator, bool> type.

Parameters

- **r** – [in] The return value of a std::map insert operation.
- **r** – [out] A reference to an Iterator, which is set to **r.first**.
- **it** – [out] on exit, will hold the iterator referring to the inserted element

Returns

This function returns **r.second**.

component_storage

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/components/component_storage/migrate_to_storage.hpp

Defined in header hpx/components/component_storage/migrate_to_storage.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

Functions

template<typename **Component**>

future<*hpx*::id_type> **migrate_to_storage**(*hpx*::id_type const &to_migrate, *hpx*::id_type const &target_storage)

Migrate the component with the given id to the specified target storage

The function *migrate_to_storage<Component>* will migrate the component referenced by *to_migrate* to the storage facility specified with *target_storage*. It returns a future referring to the migrated component instance.

Parameters

- **to_migrate** – [in] The global id of the component to migrate.
- **target_storage** – [in] The id of the storage facility to migrate this object to.

Template Parameters

The – only template argument specifies the component type of the component to migrate to the given storage facility.

Returns

A future representing the global id of the migrated component instance. This should be the same as *migrate_to*.

template<typename **Derived**, typename **Stub**, typename **Data**>

inline *Derived* **migrate_to_storage**(*client_base*<*Derived*, *Stub*, *Data*> const &to_migrate, *hpx*::*components*::component_storage const &target_storage)

Migrate the given component to the specified target storage

The function *migrate_to_storage* will migrate the component referenced by *to_migrate* to the storage facility specified with *target_storage*. It returns a future referring to the migrated component instance.

Parameters

- **to_migrate** – [in] The client side representation of the component to migrate.
- **target_storage** – [in] The id of the storage facility to migrate this object to.

Returns

A client side representation of representing of the migrated component instance. This should be the same as *migrate_to*.

hpx/components/component_storage/migrate_from_storage.hpp

Defined in header hpx/components/component_storage/migrate_from_storage.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

Functions

template<typename **Component**>

future<*hpx*::id_type> **migrate_from_storage**(*hpx*::id_type const &to_resurrect, *hpx*::id_type const &target = *hpx*::invalid_id)

Migrate the component with the given id from the specified target storage (resurrect the object)

The function *migrate_from_storage*<*Component*> will migrate the component referenced by *to_resurrect* from the storage facility specified where the object is currently stored on. It returns a future referring to the migrated component instance. The component instance is resurrected on the locality specified by *target_locality*.

Parameters

- **to_resurrect** – [in] The global id of the component to migrate.

- `target` – [in] The optional locality to resurrect the object on. By default the object is resurrected on the locality it was located on last.

Template Parameters

The – only template argument specifies the component type of the component to migrate from the given storage facility.

Returns

A future representing the global id of the migrated component instance. This should be the same as *to_resurrect*.

process

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx/components/process/process.hpp

Defined in header hpx/components/process/process.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **components**

namespace **process**

Functions

template<typename ...**Ts**>

child **execute**(*hpx*::id_type const &id, *Ts*&&... ts)

```
HPX_PROCESS_EXPORT child launch_connecting_locality (std::string const &to_launch,
std::vector< std::string > const &outer_args={},
std::vector< std::string > const &outer_env={},
std::optional< std::uint64_t > spawn_id=nullopt)
```

Launch an executable as a new HPX locality that connects to the currently running (bootstrap) locality via TCP.

This function spawns `to_launch` as a child process configured to join the calling HPX runtime as an additional, connecting locality. It forces the TCP parcelport for bootstrapping, forwards AGAS server address/port information from the current runtime's configuration, lets the operating system pick an ephemeral parcel port for the new locality, and blocks the caller until the spawned locality signals that it has finished starting up (via a named latch derived from the executable name and, optionally, `spawn_id`).

> **Note**
>
> The following environment variables are set (overriding any inherited values) before launching:
>
> - `HPX_AGAS_SERVER_ADDRESS` / `HPX_AGAS_SERVER_PORT` ? taken from the current runtime's `hpx.agas.address` / `hpx.agas.port` configuration entries.
> - `HPX_PARCEL_SERVER_ADDRESS` ? same address as the AGAS server.
> - `HPX_PARCEL_SERVER_PORT` ? set to `"0"` so the OS assigns an ephemeral port.
> - `HPX_ON_STARTUP_WAIT_ON_LATCH` ? the startup latch name described above, used to synchronize with the caller.

Parameters

- **`to_launch`** – Path to the executable to launch as the connecting locality.
- **`outer_args`** – Additional command-line arguments appended after the mandatory HPX bootstrap arguments (`--hpx:ignore-batch-env`, `--hpx:ini=hpx.parcel.tcp.priority=1000`, `--hpx:ini=hpx.parcel.bootstrap=tcp`). Defaults to an empty list.
- **`outer_env`** – Additional environment variable entries in `"NAME=value"` form (or just `"NAME"` to set an empty value) that are merged into (and override) the inherited environment before launching the child process. Defaults to an empty list.
- **`spawn_id`** – Optional identifier used to disambiguate the startup latch name when multiple instances of the same executable are launched concurrently. When present, the latch name is `"<exe-stem>/<spawn_id>"`; otherwise it is just `"<exe-stem>"`.

Throws

`hpx::exception` – (or a derived type) if the child process could not be started (`process::throw_on_error()` is used).

Returns

A `child` object representing the launched connecting-locality process, once it has signaled startup completion.

supervision_dispatch

See *Public API* for a list of names and headers that are part of the public *HPX* API.

hpx::supervision::init

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::finalize*
- *hpx::supervision::is_initialized*

hpx::shared_future<registry> `hpx::supervision::init`(*hpx*::chrono::steady_duration const &discovery_timeout = default_discovery_timeout)

Performs one-shot, idempotent initialization of the supervision-dispatch runtime for this locality. On the first successful call, this function atomically transitions the internal lifecycle state from *uninitialized* to *initializing*, after incrementing the internal epoch, then owns the entire startup sequence: it creates a local *registry* for hpx::find_here(), publishes *event::started* for this locality *before* the registry's symbol name is registered (so that no peer can discover and join a not-yet-started locality), registers that symbol name, and finally performs a single *discover_and_join()* pass to join every currently reachable peer registry. On success the state transitions to *active*; if any step throws, all partial state is rolled back and the lifecycle reverts to *uninitialized* so that a subsequent call can retry from scratch.

Calling init() while already *active* is a no-op that resolves immediately without repeating any side effect. Calling it while another call is still *initializing* attaches the caller to the in-flight operation's outcome rather than racing it, so concurrent callers never create more than one registry. Calling it while *finalizing* is in progress is rejected; callers should wait for *finalize()* to complete before re-initializing.

> **Note**
>
> This function has no effect on any state that has not yet been created by a successful call - in particular, no symbol name is ever registered and no event is ever published unless this function (or a concurrent call it is attached to) actually runs the initialization sequence.

Parameters

`discovery_timeout` – The maximum duration to wait, across all discovery candidates combined, for peer registry symbol names to resolve during the *discover_and_join()* step (see *discover_peers()*).

Returns

A shared future that becomes ready, holding the *registry* owned by this *init()* cycle, once the supervision runtime has reached the *active* state, or that becomes exceptional if initialization fails.

registry `hpx::supervision::init`(*hpx*::*launch*::sync_policy policy, *hpx*::chrono::steady_duration const &discovery_timeout = default_discovery_timeout)

Performs one-shot, idempotent initialization of the supervision-dispatch runtime for this locality, blocking the calling thread until the operation completes. This is the synchronous counterpart of init(hpx::chrono::steady_duration const&): it carries out the exact same lifecycle transition and startup sequence (registry creation, *event::started* publication, symbol name registration, and a single *discover_and_join()* pass), but waits for the resulting future to become ready (or exceptional) before returning, rather than handing the future back to the caller.

> **Note**
>
> As with the asynchronous overload, no symbol name is registered and no event is published unless this call (or a concurrent call it attaches to) actually performs the initialization sequence.

Parameters

- **`policy`** – Tag selecting the blocking overload of init().

- **discovery_timeout** – The maximum duration to wait, across all discovery candidates combined, for peer registry symbol names to resolve during the *discover_and_join()* step (see *discover_peers()*).

Throws

hpx::exception – if initialization could not complete (e.g. due to a concurrent *finalize()* still in progress). Calling this while already *active* is a no-op that returns immediately.

Returns

The *registry* owned by this *init()* cycle.

hpx::supervision::finalize

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::init*
- *hpx::supervision::is_initialized*

void *hpx::supervision::***finalize**()

Performs one-shot, idempotent teardown of the supervision-dispatch runtime previously started by *init()*. On the first call observing the *active* state, this function atomically transitions the lifecycle to *finalizing*, publishes *event::completed* for the local locality at the current epoch, unregisters the registry's symbol name, and then releases ownership of the registry (relying on *client_base's* default lifetime management to destroy it once no other references remain) before resetting the lifecycle back to *uninitialized*, allowing a subsequent *init()* call to re-arm the runtime from scratch.

Calling finalize() when the runtime is *uninitialized*, still *initializing*, or already *finalizing* is a documented no-op: it resolves immediately without publishing any event, unregistering any symbol name, or otherwise touching component state.

> **Note**
>
> Because finalize() is a no-op unless the runtime is currently *active*, it is always safe to call speculatively, e.g. during shutdown, without first checking *is_initialized()*.

hpx::supervision::is_initialized

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::init*
- *hpx::supervision::finalize*

bool *hpx*::*supervision*::**is_initialized**() noexcept

Returns whether the supervision-dispatch runtime for this locality is currently *active*, i.e. has completed *init()* and has not yet been torn down by *finalize()*.

> **Note**
>
> This function never blocks and performs a single atomic load; it is safe to call from any thread, including while *init()* or *finalize()* is concurrently in flight.

Returns

true if the lifecycle state is *active*, *false* if it is *uninitialized*, *initializing*, or *finalizing*.

hpx::supervision::dispatch_work

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::invoke_fenced_action*

> **Warning**
>
> doxygenfunction: Unable to resolve function "hpx::supervision::dispatch_work" with arguments (hpx::id_type const&, std::uint64_t const, Ts&&…) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename Action, typename ...Ts> decltype(auto)␣
↪dispatch_work(Action, discovered_peer const &peer, Ts&&... ts)
- template<typename Action,
↪ typename ...Ts> decltype(auto) dispatch_work(Action, hpx::id_
↪type const &target, std::uint64_t const epoch, Ts&&... ts)
- template<typename␣
↪Action, typename ...Ts> decltype(auto) dispatch_work(hpx::id_
↪type const &target, std::uint64_t const epoch, Ts&&... ts)
```

> **Warning**
>
> doxygenfunction: Unable to resolve function "hpx::supervision::dispatch_work" with arguments (Action, hpx::id_type const&, std::uint64_t const, Ts&&…) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename Action, typename ...Ts> decltype(auto)␣
↪dispatch_work(Action, discovered_peer const &peer, Ts&&... ts)
- template<typename Action,
↪ typename ...Ts> decltype(auto) dispatch_work(Action, hpx::id_
↪type const &target, std::uint64_t const epoch, Ts&&... ts)
- template<typename␣
↪Action, typename ...Ts> decltype(auto) dispatch_work(hpx::id_
↪type const &target, std::uint64_t const epoch, Ts&&... ts)
```

Warning

doxygenfunction: Unable to resolve function "hpx::supervision::dispatch_work" with arguments (Action, discovered_peer const&, Ts&&…) in doxygen xml output for project "hpx" from directory: ../hpx_autodoc. Potential matches:

```
- template<typename Action, typename ...Ts> decltype(auto)␣
↪dispatch_work(Action, discovered_peer const &peer, Ts&&... ts)
- template<typename Action,
↪ typename ...Ts> decltype(auto) dispatch_work(Action, hpx::id_
↪type const &target, std::uint64_t const epoch, Ts&&... ts)
- template<typename␣
↪Action, typename ...Ts> decltype(auto) dispatch_work(hpx::id_
↪type const &target, std::uint64_t const epoch, Ts&&... ts)
```

hpx::supervision::invoke_fenced_action

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::dispatch_work*

template<typename **Action**, typename **IdType**, typename **Epoch**, typename ...**Ts**>

decltype(auto) *hpx*::*supervision*::**invoke_fenced_action**(*Action* act, *IdType* target, *Epoch* const epoch, *Ts*... ts)

Remote dispatch point that performs the authoritative admission re-check and invokes the wrapped action.

This is the function body that actually runs as the action on the *destination* locality (the target's own locality, reached via hpx::colocated in dispatch_work()). It is invoked by fenced_action once the action has been dispatched.

The admission check performed here runs on the same locality/thread that owns the target's *supervision_manager* state, so there is no suspension point between this check and the actual invocation below. This is what closes the client-side/dispatch race: even if the client's own (non-authoritative) *check_admission()* in dispatch_work() returned `admitted`, the target may have latched a terminal event for this epoch in the meantime, and this re-check will catch it.

This re-check is authoritative only because a joining registry (see components/supervision_dispatch/src/server/registry_server.cpp) dual-publishes lifecycle mirrors for a joined peer onto that peer's own locality, in addition to the joining registry's local copy: `target` must resolve to that same peer locality for this guarantee to hold. Dispatching `target` against an unrelated third locality - one that never received the dual-published mirror for this target - remains out of contract; the re-check performed there consults a *supervision_manager* state that was never seeded for `target` and so cannot detect fencing.

Template Parameters

- `Action` – The wrapped HPX action type to invoke on admission.
- `IdType` – Type of the target identifier forwarded to `act` (this type is always `hpx::id_type`).
- `Epoch` – Type of the fencing epoch value (this type is always `std::uint64_t`).
- `Ts` – Types of the additional arguments forwarded to `act`.

Parameters

- `act` – The action instance to invoke once admission succeeds.
- `target` – Identifier of the component instance being dispatched to; forwarded to `act` (the epoch is fencing metadata only and is not part of `act's` argument list).
- `epoch` – The fencing epoch to check admission for.
- `ts` – Additional arguments forwarded to `act`.

Throws

`hpx::exception` – with hpx::error::target_fenced if the target has latched a terminal event for `epoch` since the client's own admission check, in which case the wrapped action is not invoked.

Returns

The result of invoking `act`, obtained via hpx::sync (run directly/locally, without an extra parcel hop, since this function already executes on the target's locality).

hpx::supervision::discover_peers

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::fan_out_join*
- *hpx::supervision::discover_and_join*

std::vector<*discovered_peer*> `hpx::supervision::discover_peers`(chrono::steady_duration const &timeout = default_discovery_timeout)

Performs a one-time discovery pull for supervision_dispatch peers: concurrently resolves the pinned registry name (see register_name()) of every remote locality (*hpx::find_remote_localities()*, which already excludes this locality), bounded by a single wait for *timeout* across all candidates.

Because constructing a registry client from a symbolic name (see client_base's symbolic-name constructor) waits/resolves once the symbol is bound rather than failing fast on one that is not (yet) registered, localities that never called register_name() (e.g. because they have not run init_supervision() yet) would otherwise hang this call indefinitely. Instead, once *timeout* elapses, only the candidates whose registry client was resolved by then (checked via the non-blocking is_ready()) are returned; every other candidate is simply excluded as not currently participating in supervision - *discover_peers()* itself never fails or throws on their account.

Parameters

timeout – The maximum duration to wait, across all candidates combined, for their registry names to resolve.

Returns

The peers whose registry client was resolved within *timeout*, as discovered_peer values carrying the hpx::id_type locality it is pinned to.

hpx::supervision::fan_out_join

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::discover_peers*
- *hpx::supervision::discover_and_join*

std::vector<*discovered_peer*> `hpx::supervision::fan_out_join`(*registry* const &local_registry, *std*::vector<*discovered_peer*> const &peers, chrono::steady_duration const &timeout = default_discovery_timeout)

Reactively fans out join() calls from *local_registry* to every peer in *peers* (typically the result of a prior *discover_peers()* call): for each entry, this calls `local_registry.join(peer.locality)`.

This reuses registry::join()'s existing reservation/idempotency machinery unchanged (see server::registry::reserve_ownership()) rather than duplicating any of it, so calling *fan_out_join()* more than once with (partially) overlapping peer lists - or racing with an inbound join() call from one of those same peers, e.g. because that peer is concurrently running its own *fan_out_join()* against this locality - is safe and never creates more than one registry entry per peer locality.

fan_out_join() itself is reactive only: it performs exactly one round of join() calls for the given *peers* and starts no background/polling activity of its own.

Parameters

- **local_registry** – The registry to join every peer in *peers* to.
- **peers** – The peers to join, typically returned by a prior *discover_peers()* call.
- **timeout** – The maximum duration to wait, across all peers combined, for their join() calls to settle. Peers whose join() has not settled (or failed) within this bound are silently dropped from the result, rather than hanging or throwing.

Returns

The discovered_peer values (locality + join_epoch) for peers whose join() calls settled successfully within *timeout*, in the same relative order as *peers* (with unsuccessful/ timed-out peers omitted, rather than left as gaps). Each returned discovered_peer::join_epoch is populated from the corresponding join() call, so callers can dispatch against `{peer.locality, peer.join_epoch}` directly, without needing a separate join() call to obtain it.

hpx::supervision::discover_and_join

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::discover_peers*
- *hpx::supervision::fan_out_join*

std::vector<*discovered_peer*> `hpx::supervision::discover_and_join`(*registry* const &local_registry, chrono::steady_duration const &timeout = default_discovery_timeout)

Composes a single reactive discovery-and-join pass for supervision_dispatch peers: performs exactly one *discover_peers()* pull to resolve the pinned registry name of every remote locality (*hpx::find_remote_localities()* already excludes this locality, so no separate self-exclusion is needed here), then fans the resulting peer list out to *local_registry* via a single *fan_out_join()* pass.

discover_and_join() introduces no state, polling thread, timer, or repeated broadcast of its own - it is a pure composition of *discover_peers()* and *fan_out_join()*, each of which is itself bounded to a single round of resolution/join calls. Because *fan_out_join()* reuses registry::join()'s existing reservation/idempotency machinery unchanged (see server::registry::reserve_ownership()), calling *discover_and_join()* more than once - whether to pick up newly started peers or purely by accident - is safe and never creates more than one registry entry per peer locality; repeated invocations simply return the (stable) join epochs already owned for peers that were previously joined.

Parameters

- `local_registry` – The registry to join every discovered peer to.
- `timeout` – The maximum duration to wait, across all discovery candidates combined, for their registry name to resolve (see *discover_peers()*).

Returns

The discovered_peer value that *local_registry* created (or already had) for each peer discovered and successfully joined within *timeout*, in the same relative order as returned by *discover_peers()*.

hpx::supervision::testing::local_snapshot_peers

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::testing::set_failure_detection_poll_timeout_for_testing*
- *hpx::supervision::testing::last_join_locality*
- *hpx::supervision::testing::suspend_heartbeat_for_testing*
- *hpx::supervision::testing::failure_detection_sweep_in_flight_for_testing*

- *hpx::supervision::testing::stop_background_loops*

std::vector<*server*::*peer_snapshot*> `hpx::supervision::testing::local_snapshot_peers()`

Returns a snapshot of this locality's currently-joined, ready peers, as tracked by the local registry. Not part of the public dispatch API - exists so tests can inspect registry state directly.

hpx::supervision::testing::set_failure_detection_poll_timeout_for_testing

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::testing::local_snapshot_peers*
- *hpx::supervision::testing::last_join_locality*
- *hpx::supervision::testing::suspend_heartbeat_for_testing*
- *hpx::supervision::testing::failure_detection_sweep_in_flight_for_testing*
- *hpx::supervision::testing::stop_background_loops*

void `hpx::supervision::testing::set_failure_detection_poll_timeout_for_testing(`*hpx*::chrono::steady_duration const &timeout`)`

Overrides the timeout failure_detection_loop() passes to *await_terminal()* on each sweep. Must be called before *init()* starts the loop to take effect for that lifecycle; has no effect on a sweep already in flight. Not part of the public dispatch API - exists only to make failure-detection tests deterministic and fast instead of bound by the real 60s default_discovery_timeout.

Parameters

`timeout` – The new poll timeout.

hpx::supervision::testing::last_join_locality

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::testing::local_snapshot_peers*
- *hpx::supervision::testing::set_failure_detection_poll_timeout_for_testing*
- *hpx::supervision::testing::suspend_heartbeat_for_testing*
- *hpx::supervision::testing::failure_detection_sweep_in_flight_for_testing*
- *hpx::supervision::testing::stop_background_loops*

hpx::id_type `hpx::supervision::testing::last_join_locality()`

Returns the locality most recently returned by join(), regardless of whether the register_observers() call that followed it went on to succeed or fail. Lets tests verify that a failed join() does not leak that locality's local supervision state (see registry::register_observers()'s catch block).

hpx::supervision::testing::suspend_heartbeat_for_testing

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::testing::local_snapshot_peers*
- *hpx::supervision::testing::set_failure_detection_poll_timeout_for_testing*
- *hpx::supervision::testing::last_join_locality*
- *hpx::supervision::testing::failure_detection_sweep_in_flight_for_testing*
- *hpx::supervision::testing::stop_background_loops*

void `hpx::supervision::testing::suspend_heartbeat_for_testing()`

Stops this locality's own heartbeat_loop() without finalizing the dispatcher, simulating a hard crash (all lifecycle activity ceases, including the periodic event::running pulse) while staying joined in peers' registries. Idempotent; no-op if *init()* was never called or the loop is already stopped. Not part of the public dispatch API - exists only so failure-detection tests can deterministically simulate silence instead of relying on the absence of explicit calls, which no longer implies silence now that heartbeats are mirrored onto observers' local supervision state.

hpx::supervision::testing::failure_detection_sweep_in_flight_for_testing

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::testing::local_snapshot_peers*
- *hpx::supervision::testing::set_failure_detection_poll_timeout_for_testing*
- *hpx::supervision::testing::last_join_locality*
- *hpx::supervision::testing::suspend_heartbeat_for_testing*
- *hpx::supervision::testing::stop_background_loops*

bool `hpx::supervision::testing::failure_detection_sweep_in_flight_for_testing()`

Reports whether failure_detection_loop() currently has one or more *await_terminal()* calls outstanding against unresponsive peers (i.e. a sweep is mid-flight). Returns false before *init()* is called, between sweeps, and after *finalize()*. Not part of the public dispatch API - exists only so tests can deterministically synchronize on a real in-flight sweep instead of sleeping and hoping the loop got there first.

hpx::supervision::testing::stop_background_loops

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

See also:

- *hpx::supervision::testing::local_snapshot_peers*
- *hpx::supervision::testing::set_failure_detection_poll_timeout_for_testing*
- *hpx::supervision::testing::last_join_locality*
- *hpx::supervision::testing::suspend_heartbeat_for_testing*
- *hpx::supervision::testing::failure_detection_sweep_in_flight_for_testing*

void *hpx*::*supervision*::testing::**stop_background_loops**()

Stops this locality's failure_detection_loop() and heartbeat_loop() background tasks (joining both) without otherwise finalizing the dispatcher: unlike *finalize()*, this never publishes event::completed, never unregisters the registry symbol name, and never resets the lifecycle state back to uninitialized, so the sentinel's last published event stays stale and peers' failure_detection_loop() still eventually times out and observes target_fenced/rejected_fenced against this locality. Lets a caller depart the HPX runtime cleanly (e.g. via *hpx::disconnect()*) afterwards without leaving either background hpx::async task still running - simulating a worker that dies mid-epoch without corrupting AGAS or hanging runtime shutdown, as opposed to a true crash (no call at all) or a fully graceful departure (*finalize()*). Idempotent; no-op if *init()* was never called or the loops are already stopped. Not part of the public dispatch API - exists only for examples/tests that need to simulate this exact scenario deterministically.

hpx::supervision::registry

Defined in header hpx/supervision_dispatch.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

class **registry** : public *hpx*::*components*::*client_base*<*registry*, *server*::*registry*>

A registry is a lightweight, self-supervising handle for pairing with peer localities. It mirrors a peer's lifecycle state locally via join(), and can itself be discovered by name in AGAS like other supervision_dispatch components.

hpx/supervision_dispatch/server/registry.hpp

Defined in header hpx/supervision_dispatch/server/registry.hpp.

See *Public API* for a list of names and headers that are part of the public *HPX* API.

namespace **hpx**

namespace **supervision**

Fenced action dispatch for the supervision subsystem.

This header provides the machinery to dispatch an HPX action to a target component while enforcing supervision "fencing" semantics: once a target has latched a terminal event for a given epoch, no further actions for that epoch may execute against it. Dispatch performs a cheap, non-authoritative admission check on the caller's locality and an authoritative re-check on the target's own locality (via hpx::colocated), immediately before invoking the wrapped action, closing the race between admission and invocation.

Types and functions implementing HPX's supervision and fenced dispatch facilities.

Functions

inline bool **operator==**(*joined_peer* const &lhs, *joined_peer* const &rhs) noexcept

inline bool **operator!=**(*joined_peer* const &lhs, *joined_peer* const &rhs) noexcept

std::ostream &**operator<<**(*std*::ostream &strm, *joined_peer* const &peer)

struct **joined_peer**

#include <registry.hpp> Pairs the two values a caller obtains from a successful `registry::join()` and needs to perform a fenced dispatch.

See also

dispatch_work()

See also

server::registry::join()

Public Members

hpx::id_type **`target`**

The real, colocatable destination for the wrapped action.

The peer's locality id (i.e. `peer_locality`, the value originally passed into `registry::join()`). Forwarded to `hpx::colocated()` and used as the destination of `hpx::sync(act, target, ts...)` inside `dispatch_work()`.

std::uint64_t **`join_epoch`** = 0

The epoch at which this peer was joined.

Recorded by `registry::join()` and used to identify which registry entry a later `registry::leave()` call refers to - so that a racing re-join of the same peer locality (which mints a fresh `join_epoch` of its own) cannot be mistaken for the join this *`joined_peer`* was returned from.

namespace **`server`**

struct **`peer_snapshot`**

#include <registry.hpp> Plain-data view of a single joined peer, safe to hand out to callers outside the registry (unlike peer_entry, does not expose the ready/evict_pending bookkeeping used internally to coordinate concurrent join()/evict_peer() calls).

Public Members

hpx::id_type **`peer_locality`**

The peer locality this entry was joined against.

std::uint64_t **`join_epoch`**

The epoch at which `peer_locality` was joined.

class **`registry`** : public *hpx*::*components*::*component_base*<*registry*>

Public Functions

`registry``()`

joined_peer **`join`**(*hpx*::id_type const &peer_locality)

Joins a peer locality and creates or reuses its local supervision state.

Parameters

`peer_locality` – The peer locality to observe.

std::vector<*peer_snapshot*> **snapshot_peers**() const

Returns a point-in-time snapshot of all fully joined, non-evicting peers.

Intended for a future failure-detection poller (or other local consumers) that needs a stable, lock-safe list of peers to iterate over without reaching into `peers_` directly. Entries mid-join (ready == false) or already scheduled for eviction (evict_pending == true) are excluded, so a poller never observes a half-registered or torn-down peer.

Returns

A snapshot of every peer entry with `ready` == true and `evict_pending` == false at the time of the call.

void **leave**(*hpx*::id_type const &peer_locality, *std*::uint64_t join_epoch)

Removes a previously joined peer.

Parameters

- **peer_locality** – The peer locality to leave.
- **join_epoch** – Must match the join epoch currently recorded for `peer_locality` (i.e. the epoch returned by the corresponding *join()*). If `peer_locality` has since been re-joined (fresh epoch) or already evicted, this call is a no-op.

Protected Functions

std::tuple<*hpx*::id_type, *std*::uint64_t, bool> **reserve_ownership**(*hpx*::id_type const &peer_locality)

std::pair<*hpx*::id_type, *hpx*::id_type> **register_observers**(*hpx*::id_type const &peer_locality, *std*::uint64_t join_epoch)

void **evict_peer**(*hpx*::id_type const &peer_locality, *std*::uint64_t join_epoch, *hpx*::id_type keep_alive, bool preserve_terminal_state)

Protected Static Functions

static void **cleanup_peer**(*hpx*::id_type const &peer_locality, *hpx*::id_type const &lifecycle_observer, *hpx*::id_type const &activity_observer, bool preserve_terminal_state)

Unregisters a peer's observers and, unless `preserve_terminal_state` is set, removes its locally tracked supervision state.

Performs the actual peer teardown deferred by evict_peer(): unregisters `lifecycle_observer` and `activity_observer` on `peer_locality`, and removes `peer_locality's` locally tracked supervision state unless `preserve_terminal_state` is true, in which case that state - which was last set to the peer's terminal event by register_observers()'s lifecycle callback - is left in place as a tombstone rather than reset to "unknown". This lets any admission check still in flight against `peer_locality` (e.g. check_admission()) keep observing the fenced/terminal outcome after eviction has run, instead of racing against it. A subsequent *join()* re-seeding this locality with a fresh epoch naturally overwrites the tombstone. Kept separate from evict_peer() because evict_peer() only posts this work as a task (via hpx::post()) rather than performing it inline.

Parameters

- **peer_locality** – The locality that owns the observers, and whose local supervision state is removed.
- **lifecycle_observer** – The lifecycle observer id to unregister.
- **activity_observer** – The activity observer id to unregister.
- **preserve_terminal_state** – If true, leave peer_locality's local supervision state in place instead of removing it.

Private Members

mutable *hpx*::*spinlock* **mtx_**

hpx::*lcos*::*local*::detail::condition_variable **cond_**

std::map<*hpx*::id_type, *peer_entry*> **peers_**

struct **peer_entry**

Internal bookkeeping for a single joined (or joining) peer.

Not exposed outside the registry; see *peer_snapshot* for the plain-data subset safe to hand to external callers.

Public Functions

inline explicit **peer_entry**(*hpx*::id_type const &peer_locality)

Public Members

hpx::id_type const **peer_locality**

The locality that owns this peer's. Surfaced read-only via *peer_snapshot::peer_locality*.

hpx::id_type **lifecycle_observer**

The observer id registered on peer_locality to receive this peer's terminal lifecycle notification (drives evict_peer() via hpx::post()).

hpx::id_type **activity_observer**

The observer id registered on peer_locality to receive this peer's activity notifications (used to keep peer_locality's local state current between terminal events).

std::uint64_t **join_epoch** = 0

The epoch at which this peer was joined, recorded by *join()* and used by evict_peer() (instead of shadow) to identify and used by evict_peer() to identify which entry a deferred eviction refers to.

bool **ready** = false

False while a *join()* call has reserved ownership of this peer's locality (i.e. is in the process of performing the remote observer registrations) but has not yet filled in the rest of the entry. Concurrent *join()* calls for the same peer wait on cond_ instead of racing to register duplicate observers.

bool **evict_pending** = false

True once a terminal notification has been observed for this peer but eviction has not yet completed (deferred via hpx::post() to evict_peer()). Ensures a terminal notification racing ahead of *join()* completion is deferred instead of dropped, and lets *snapshot_peers()* exclude a peer that is already tearing down.

bool **evict_preserve_terminal_state** = false

The preserve_terminal_state argument evict_peer() was called with when it set evict_pending above, remembered so that *join()*'s deferred *cleanup_peer()* call (once the join in progress completes) preserves or drops peer_locality's local supervision state consistently with what the racing terminal notification requested.

2.10 Contributing to *HPX*

HPX development happens on Github. The following sections are a collection of useful information related to *HPX* development.

2.10.1 Contributing to *HPX*

The main source of information to understand the process of how to contribute to HPX can be found in this document[982]. This is a living document that is constantly updated with relevant information.

2.10.2 *HPX* governance model

The *HPX* project is a meritocratic, consensus-based community project. Anyone with an interest in the project can join the community, contribute to the project design and participate in the decision making process. This document[983] describes how that participation takes place and how to set about earning merit within the project community.

[982] https://github.com/TheHPXProject/hpx/blob/master/.github/CONTRIBUTING.md
[983] http://hpx.dev/documents/governance/

2.10.3 Release procedure for *HPX*

Below is a step by step procedure for making an *HPX* release. We aim to produce two releases per year: one in March-April, and one in September-October.

This is a living document and may not be totally current or accurate. It is an attempt to capture current practices in making an *HPX* release. Please update it as appropriate.

One way to use this procedure is to print a copy and check off the lines as they are completed to avoid confusion.

1. Notify developers that a release is imminent.
2. For minor and major releases: create and check out a new branch at an appropriate point on `master` with the name `release-major.minor.X`. `major` and `minor` should be the major and minor versions of the release. For patch releases: check out the corresponding `release-major.minor.X` branch.
3. Write release notes in `docs/sphinx/releases/whats_new_$VERSION.rst`. Keep adding merged PRs and closed issues to this until just before the release is made. Use `tools/generate_pr_issue_list.sh` to generate the lists. Add the new release notes to the table of contents in `docs/sphinx/releases.rst`.
4. Build the docs, and proof-read them. Update any documentation that may have changed, and correct any typos. Pay special attention to:

- `$HPX_SOURCE/README.rst`

– Update grant information

- `docs/sphinx/releases/whats_new_$VERSION.rst`
- `docs/sphinx/about_hpx/people.rst`

– Update collaborators

– Update grant information

5. This step does not apply to patch releases. For APEX:

- Change the release branch to be the most current release tag available in the APEX `git_external` section in the main `CMakeLists.txt`. Please contact the maintainers of the respective packages to generate a new release to synchronize with the *HPX* release (APEX[984]).

6. Make sure `HPX_VERSION_MAJOR/MINOR/SUBMINOR` in `CMakeLists.txt` contain the correct values. Change them if needed.
7. Change version references in CITATION.cff. There are two occurrences. Change year in the copyright file under `/libs/core/version/src/version.cpp`.
8. This step does not apply to patch releases. Remove features which have been deprecated for at least 2 releases. This involves removing build options which enable those features from the main CMakeLists.txt and also deleting all related code and tests from the main source tree.

   The general deprecation policy involves a three-step process we have to go through in order to introduce a breaking change:

a. First release cycle: add a build option that allows for explicitly disabling any old (now deprecated) code.

[984] http://github.com/UO-OACISS/xpress-apex

b. Second release cycle: turn this build option OFF by default.

c. Third release cycle: completely remove the old code.

The main CMakeLists.txt contains a comment indicating for which version the breaking change was introduced first. In the case of deprecated features which don't have a replacement yet, we keep them around in case (like Vc for example).

9. Update the minimum required versions if necessary (compilers, dependencies, etc.) in `prerequisites.rst`. Update latest tested versions.

10. Verify that the Jenkins setups for the release branch on Rostam are running and do not display any errors.

11. Repeat the following steps until satisfied with the release.

1. Change `HPX_VERSION_TAG` in `CMakeLists.txt` to `-rcN`, where `N` is the current iteration of this step. Start with `-rc1`.

2. Create a pre-release on GitHub using the script `tools/roll_release.sh`. This script automatically tag with the corresponding release number. The script requires that you have the STE||AR Group signing key.

3. This step is not necessary for patch releases. Notify `hpx-users@stellar-group.org` of the availability of the release candidate. Ask users to test the candidate by checking out the release candidate tag.

4. Allow at least a week for testing of the release candidate.

- Use `git merge` when possible, and fall back to `git cherry-pick` when needed. For patch releases `git cherry-pick` is most likely your only choice if there have been significant unrelated changes on master since the previous release.
- Go back to the first step when enough patches have been added.
- If there are no more patches, continue to make the final release.

12. Update any occurrences of the latest stable release to refer to the version about to be released. For example, `quickstart.rst` contains instructions to check out the latest stable tag. Make sure that refers to the new version.

13. Add a new entry to the RPM changelog (`cmake/packaging/rpm/Changelog.txt`) with the new version number and a link to the corresponding changelog.

14. Change `HPX_VERSION_TAG` in `CMakeLists.txt` to an empty string.

15. Add the release date to the caption of the current "What's New" section in the docs, and change the value of `HPX_VERSION_DATE` in `CMakeLists.txt`.

16. Create a release on GitHub using the script `tools/roll_release.sh`. This script automatically tag the with the corresponding release number. The script requires that you have the STE||AR Group signing key.

17. Update the website (hpx.dev[985]). You can login on wordpress through *this page <https://hpx.dev/wp-login.php>*. You can update the pages with the following:

- Update links on the downloads page. Link to the release on GitHub.
- Documentation links on the docs page (link to generated documentation on GitHub Pages). Follow the style of previous releases.
- A new blog post announcing the release, which links to downloads and the "What's New" section in the documentation (see previous releases for examples).

[985] https://hpx.dev

18. Merge release branch into master.
19. Post-release cleanup. Create a new pull request against master with the following changes:
    1. Modify the release procedure if necessary.
    2. Change `HPX_VERSION_TAG` in `CMakeLists.txt` back to `-trunk`.
    3. Increment `HPX_VERSION_MINOR` in `CMakeLists.txt`.
20. Update Vcpkg (`https://github.com/Microsoft/vcpkg`) to pull from latest release.
    - Update version number in CONTROL
    - Update tag and SHA512 to that of the new release
21. Update spack (`https://github.com/spack/spack`) with the latest HPX package.
    - Update version number in `hpx/package.py` and SHA256 to that of the new release
22. Announce the release on hpx-users@stellar-group.org, stellar@cct.lsu.edu, all-cct@cct.lsu.edu, faculty@csc.lsu.edu, faculty@ece.lsu.edu, the *HPX* Slack channel, the STEllAR-GROUP Discord channel, our list of external collaborators, isocpp.org, reddit.com, HPC Wire, Inside HPC, Heise Online, and a CCT press release.
23. Beer and pizza.

2.10.4 Testing *HPX*

To ensure correctness of *HPX*, we ship a large variety of unit and regression tests. The tests are driven by the CTest[986] tool and are executed automatically on each commit to the *HPX* Github[987] repository. In addition, it is encouraged to run the test suite manually to ensure proper operation on your target system. If a test fails for your platform, we highly recommend submitting an issue on our *HPX* Issues[988] tracker with detailed information about the target system.

Running tests manually

Running the tests manually is as easy as typing `make tests && make test`. This will build all tests and run them once the tests are built successfully. After the tests have been built, you can invoke separate tests with the help of the `ctest` command. You can list all available test targets using `make help | grep tests`. Please see the CTest Documentation[989] for further details.

[986] https://gitlab.kitware.com/cmake/community/wikis/doc/ctest/Testing-With-CTest
[987] https://github.com/TheHPXProject/hpx/
[988] https://github.com/TheHPXProject/hpx/issues
[989] https://www.cmake.org/cmake/help/latest/manual/ctest.1.html

Running performance tests

We run performance tests on Piz Daint for each pull request using Jenkins. To run those performance tests locally or on Piz Daint, a script is provided under `tools/perftests_ci/local_run.sh` (to be run in the build directory specifying the *HPX* source directory as the argument to the script, default is `$HOME/projects/hpx_perftests_ci`.

Adding new performance tests

To add a new performance test, you need to wrap the portion of code to benchmark with `hpx::util::perftests_report`, passing the test name, the executor name and the function to time (can be a lambda). This facility is used to output the time results in a json format (format needed to compare the results and plot them). To effectively print them at the end of your test, call `hpx::util::perftests_print_times`. To see an example of use, see `future_overhead_report.cpp`. Finally, you can add the test to the CI report editing the `hpx_targets` variable for the executable name and the `hpx_test_options` variable for the corresponding options to use for the run in the performance test script `.jenkins/cscs-perftests/launch_perftests.sh`. And then run the `tools/perftests_ci/local_run.sh` script to get a reference json run (use the name of the test) to be added in the `tools/perftests_ci/perftest/references/daint_default` directory.

Issue tracker

If you stumble over a bug or missing feature in *HPX*, please submit an issue to our *HPX* Issues[990] page. For more information on how to submit support requests or other means of getting in contact with the developers, please see the Support Website[991] page.

Continuous testing

In addition to manual testing, we run automated tests on various platforms. We also run tests on all pull requests using both **|github|_**. You can see the dashboards CDash HPX dashboard[992] .

2.10.5 Using docker for development

Although it can often be useful to set up a local development environment with system-provided or self-built dependencies, Docker[993] provides a convenient alternative to quickly get all the dependencies needed to start development of *HPX*. Our testing setup on **|github|_** uses a docker image to run all tests.

To get started you need to install Docker[994] using whatever means is most convenient on your system. Once you have Docker[995] installed, you can pull or directly run the docker

990 https://github.com/TheHPXProject/hpx/issues
991 https://stellar.cct.lsu.edu/support/
992 https://cdash.rostam.cct.lsu.edu/index.php?project=HPX
993 https://www.docker.com
994 https://www.docker.com
995 https://www.docker.com

image. The image is based on Debian and Clang, and can be found on Docker Hub[996].
To start a container using the *HPX* build environment, run:

```
$ docker␣
↪run --interactive --tty stellargroup/build_env:latest bash
```

You are now in an environment where all the *HPX* build and runtime dependencies are present. You can install additional packages according to your own needs. Please see the Docker Documentation[997] for more information on using Docker[998].

Warning

All changes made within the container are lost when the container is closed. If you want files to persist (e.g., the *HPX* source tree) after closing the container, you can bind directories from the host system into the container (see Docker Documentation (Bind mounts)[999]).

2.10.6 Documentation

This documentation is built using Sphinx[1000], and an automatically generated API reference using Doxygen[1001] and Breathe[1002].

We always welcome suggestions on how to improve our documentation, as well as pull requests with corrections and additions.

Prerequisites

To build the *HPX* documentation, you need recent versions of the following packages:

- `python3` (3.9 or later)
- `sphinx` 7.2 or later (Python package)
- `sphinx-book-theme` (Python package)
- `breathe` 4.36.0 or later (Python package)
- `doxygen`
- `sphinxcontrib-bibtex`
- `sphinx-copybutton`

If the Python[1003] dependencies are not available through your system package manager, you can install them using the Python package manager `pip`. We recommend using a virtual environment:

[996] https://hub.docker.com/r/stellargroup/build_env/
[997] https://docs.docker.com/
[998] https://www.docker.com
[999] https://docs.docker.com/storage/bind-mounts/
[1000] http://www.sphinx-doc.org
[1001] https://www.doxygen.org
[1002] https://breathe.readthedocs.io/en/latest
[1003] https://www.python.org

```
mkdir -p build
python3 -m venv build/.venv
source build/.venv/bin/activate
pip install \
   "sphinx>=7.2" "breathe>=4.36.0" \
   sphinx-book-theme sphinxcontrib-bibtex \
   sphinx-copybutton
```

You may need to set the following CMake variables to make sure CMake can find the required dependencies.

Doxygen_ROOT:PATH

Specifies where to look for the installation of the Doxygen[1004] tool.

Sphinx_ROOT:PATH

Specifies where to look for the installation of the Sphinx[1005] tool.

Breathe_APIDOC_ROOT:PATH

Specifies where to look for the installation of the Breathe[1006] tool.

Building documentation

Enable building of the documentation by setting `HPX_WITH_DOCUMENTATION=ON` during CMake[1007] configuration. To build the documentation, build the `docs` target using your build tool. The default output format is HTML documentation. You can choose alternative output formats (single-page HTML, PDF, and man) with the `HPX_WITH_DOCUMENTATION_OUTPUT_FORMATS` CMake option.

> **Note**
>
> If you add new source files to the Sphinx documentation, you have to run CMake again to have the files included in the build.

Style guide

The documentation is written using reStructuredText. These are the conventions used for formatting the documentation:

- Use, at most, 80 characters per line.
- Top-level headings use over- and underlines with `=`.
- Sub-headings use only underlines with characters in decreasing level of importance: `=`, `-` and `.`.
- Use sentence case in headings.

[1004] https://www.doxygen.org
[1005] http://www.sphinx-doc.org
[1006] https://breathe.readthedocs.io/en/latest
[1007] https://www.cmake.org

- Refer to common terminology using `:term:`Component``.
- Indent content of directives (`.. directive::`) by three spaces.
- For C++ code samples at the end of paragraphs, use `::` and indent the code sample by 4 spaces.

– For other languages (or if you don't want a colon at the end of the paragraph), use `.. code-block:: language` and indent by three spaces as with other directives.

- Use `.. list-table::` to wrap tables with a lot of text in cells.

API documentation

The source code is documented using Doxygen. If you add new API documentation either to existing or new source files, make sure that you add the documented source files to the `doxygen_dependencies` variable in `docs/CMakeLists.txt`.

Adding new API documentation

When you add new public APIs to *HPX*, follow these guidelines to ensure the documentation is correctly generated and easy to use.

Documenting headers

At the top of a new header file, include the standard Doxygen documentation block. For example, if the new feature is called *my_new_feature*, you should:

- List the name of the new file under `\file`.
- Specify the page name with `\page` (e.g., `hpx::my_new_feature`).
- Reference the existing header that will include your new file under `\headerfile`.

```
/// \file my_new_feature.hpp
/// \page hpx::my_new_feature
/// \headerfile hpx/existing_header_of_new_feature.hpp
```

Adding to the public API reference

For the documentation to appear in the generated API reference, add your new API to `docs/sphinx/api/public_api.rst` under the appropriate table (usually "Functions") for the relevant header (for example, `hpx/existing_header_of_new_feature.hpp`).

Including examples

Do **not** inline examples in the API reference. If you want to provide an example of how to use the new API, there are two recommended options:

1. **Documentation page option** Edit an existing relevant manual page to explain the new functionality and include a code example. Use this option if you want to provide additional context about how the API works.
2. **Standalone examples option** Create a separate example under `docs/sphinx/examples/`. Examples in this directory are typically more comprehensive and executable, making them suitable for richer or more complex scenarios.

2.10.7 Module structure

This section explains the structure of an *HPX* module.

The tool create_library_skeleton.py[1008] can be used to generate a basic skeleton. To create a library skeleton, run the tool in the `libs` subdirectory with the module name as an argument:

```
$ ./create_library_skeleton <lib_name>
```

This creates a skeleton with the necessary files for an *HPX* module. It will not create any actual source files. The structure of this skeleton is as follows:

- `<lib_name>/`
- `README.rst`
- `CMakeLists.txt`
- `cmake`
- `docs/`
- `index.rst`
- `examples/`
- `CMakeLists.txt`
- `include/`
- `hpx/`
- `<lib_name>`
- `src/`
- `CMakeLists.txt`
- `tests/`
- `CMakeLists.txt`
- `unit/`
- `CMakeLists.txt`
- `regressions/`

[1008] https://github.com/TheHPXProject/hpx/blob/master/libs/create_module_skeleton.py

· CMakeLists.txt

∗ performance/

· CMakeLists.txt

A README.rst should be always included which explains the basic purpose of the library and a link to the generated documentation.

A main CMakeLists.txt is created in the root directory of the module. By default it contains a call to add_hpx_module which takes care of most of the boilerplate required for a module. You only need to fill in the source and header files in most cases.

add_hpx_module requires a module name. Optional flags are:

Optional single-value arguments are:

- INSTALL_BINARIES: Install the resulting library.

Optional multi-value arguments-are:

- SOURCES: List of source files.
- HEADERS: List of header files.
- COMPAT_HEADERS: List of compatibility header files.
- DEPENDENCIES: Libraries that this module depends on, such as other modules.
- CMAKE_SUBDIRS: List of subdirectories to add to the module.

The include directory should contain only headers that other libraries need. For each of those headers, an automatic header test to check for self containment will be generated. Private headers should be placed under the src directory. This allows for clear separation. The cmake subdirectory may include additional CMake[1009] scripts needed to generate the respective build configurations.

Compatibility headers (forwarding headers for headers whose location is changed when creating a module, if moving them from the main library) should be placed in an include_compatibility directory. This directory is not created by default.

Documentation is placed in the docs folder. A empty skeleton for the index is created, which is picked up by the main build system and will be part of the generated documentation. Each header inside the include directory will automatically be processed by Doxygen and included into the documentation.

Tests are placed in suitable subdirectories of tests.

When in doubt, consult existing modules for examples on how to structure the module.

Finding circular dependencies

Our CI will perform a check to see if there are circular dependencies between modules. In cases where it's not clear what is causing the circular dependency, running the cpp-dependencies[1010] tool manually can be helpful. It can give you detailed information on exactly which files are causing the circular dependency. If you do not have the cpp-dependencies tool already installed, one way of obtaining it is by using our docker image. This way you will have exactly the same environment as on the CI. See *Using docker for development* for details on how to use the docker image.

[1009] https://www.cmake.org

[1010] https://github.com/tomtom-international/cpp-dependencies

To produce the graph produced by CI run the following command (`HPX_SOURCE` is assumed to hold the path to the *HPX* source directory):

```
$ cpp-dependencies␣
↪--dir $HPX_SOURCE/libs --graph-cycles circular_dependencies.dot
```

This will produce a `dot` file in the current directory. You can inspect this manually with a text editor. You can also convert this to an image if you have `graphviz` installed:

```
$ dot circular_dependencies.dot -Tsvg -o circular_dependencies.svg
```

This produces an `svg` file in the current directory which shows the circular dependencies. Note that if there are no cycles the image will be empty.

You can use `cpp-dependencies` to print the include paths between two modules.

```
$ cpp-dependencies --dir $HPX_SOURCE/libs --shortest <from> <to>
```

prints all possible paths from the module `<from>` to the module `<to>`. For example, as most modules depend on `config`, the following should give you a long list of paths from `algorithms` to `config`:

```
$ cpp-
↪dependencies --dir $HPX_SOURCE/libs --shortest algorithms config
```

The following should report that it can't find a path between the two modules:

```
$ cpp-
↪dependencies --dir $HPX_SOURCE/libs --shortest config algorithms
```

2.11 Releases

2.11.1 List of supported releases

HPX V2.0.0 (TBD)

General changes

Breaking changes

Closed issues

Closed pull requests

HPX V1.11.0 (Jun 30, 2025)

This release is the last version of HPX that supports C++17. Future versions of HPX will require compilation using C++20 or above.

General changes

- Added synchronous versions of all collective operations. Added global predefined communicator objects that are accessible through new APIs: `hpx::collectives::get_world_communicator()` refers to all localities and `hpx::collectives::get_local_communicator()` refers to all threads on the calling locality. We unified the interfaces of the different communicator objects.
- Added `hpx::experimental::run_on_all` allowing to run a given function (possibly concurrently) using a given execution policy.
- Added a helper object `hpx::runtime_manager` that simplifies the initialization of HPX without needing to modify the `main` function of the application.
- We improved the compatibility with various accelerator frameworks (SYCL, OneAPI).
- Applied build system changes that allow building HPX without any prerequisites. This requires to pass `-DHPX_WITH_FETCH_HWLOC=On` and `-DHPX_WITH_FETCH_BOOST=On` to the CMake[1011] configuration.
- We have performed a lot of code cleanup and refactoring to improve the overall code quality and decrease compile times.
- Added the `hpx::contains` and `hpx::contains_subrange` parallel algorithms.
- Adapted many of HPX' parallel algorithms to be usable with senders/receivers.

Breaking changes

- We have moved most of the APIs that were defined in the namespace `hpx::parallel::execution` to the namespace `hpx::execution::experimental`. It was not possible to add compatibility facilities that will allow to continue using the old APIs, applications will have to be changed in order to continue functioning correctly.
- The CMake configuration parameter `HPX_WITH_RUN_MAIN_EVERYWHERE` is now deprecated and will be removed in the future. Use the preprocessor macro `HPX_HAVE_RUN_MAIN_EVERYWHERE` on a target-by-target case instead.
- Removed the dysfunctional libfabric parcelport.
- Removed features that were long deprecated (starting V1.8): - `hpx::flush`, `hpx::endl`, `hpx::async_flush`, `hpx::async_endl` - Various enumerator types are now only available as `class enum` requiring explicit scoped use of the enumerator values - Various non-conforming overloads of parallel algorithms - `hpx::for_loop` and friends (now only available as

  `hpx::experimental::for_loop`)

  - `hpx::parallel::induction` is now only available as `hpx::experimental::induction`
  - `hpx::parallel::reduction` and friends are now only available as `hpx::experimental::reduction`
  - `hpx::assertion::source_location` is now only available as `hpx::source_location`
  - `hpx::lcos::split_future` is now only available as `hpx::split_future`

[1011] https://www.cmake.org

- hpx::lcos::wait and friends have been removed altogether
- hpx::lcos::wait_any and friends are now only available as hpx::wait_any
- hpx::lcos::wait_some and friends are now only available as hpx::wait_some
- hpx::lcos::wait_each and friends are now only available as hpx::wait_each
- hpx::lcos::wait_all and friends are now only available as hpx::wait_all
- hpx::lcos::when_all and friends are now only available as hpx::when_all
- hpx::lcos::when_any and friends are now only available as hpx::when_any
- hpx::lcos::when_each and friends are now only available as hpx::when_each
- hpx::lcos::when_some and friends are now only available as hpx::when_some
- hpx::util::optional and related facilities are now only available as hpx::optional
- hpx::util::bind and related facilities are now only available as hpx::bind
- hpx::util::function and friends are now only available as hpx::function
- hpx::traits::is_bound_action and related facilities are now only available as hpx::is_bound_action
- hpx::traits::is_bind_expression and related facilities are now only available as hpx::is_bind_expression
- hpx::traits::is_placeholder and related facilities are now only available as hpx::is_placeholder
- hpx::lcos::future and related facilities are now only available as hpx::future
- hpx::memory::intrusive_ptr is now only available as hpx::intrusive_ptr
- hpx::lcos::local::barrier is now only available as hpx::barrier
- hpx::lcos::barrier is now only available as hpx::distributed::barrier
- hpx::lcos::local::cpp20_binary_semaphore is now only available as hpx::detail::binary_semaphore
- hpx::lcos::local::condition_variable and friends are now only available as hpx::condition_variable
- hpx::lcos::local::counting_semaphore and friends are now only available as hpx::counting_semaphore
- hpx::lcos::local::cpp20_latch and is now only available as hpx::latch
- hpx::lcos::latch and is now only available as hpx::distributed::latch
- hpx::lcos::local::upgrade_lock and friends are now only available as hpx::upgrade_lock
- hpx::lcos::local::mutex and friends are now only available as hpx::mutex
- hpx::lcos::local::spinlock and friends are now only available as hpx::spinlock
- hpx::lcos::local::call_once and friends are now only available as hpx::call_once
- hpx::util::annotated_function and is now only available as hpx::annotated_function

- `hpx::components::abstract_simple_component_base` and is now only available as `hpx::components::abstract_component_base`
- `hpx::naming::id_type` and is now only available as `hpx::id_type`

Closed issues

- Issue #6699[1012] - Catch lower-level runtime error
- Issue #6696[1013] - HPX master breaks with Kokkos
- Issue #6691[1014] - minimum_category doesn't work with custom iterator categories
- Issue #6681[1015] - build break - missing ';'
- Issue #6658[1016] - CMake error upon building HPX manually
- Issue #6648[1017] - Asio V1.34 deprecates io_context::work
- Issue #6640[1018] - iterator_facade doesn't work with custom iterator categories
- Issue #6636[1019] - problem with hpx::collectives::exclusive_scan
- Issue #6623[1020] - HPX serialization error with std::vector<std::vector<std::vector<float>>>
- Issue #6616[1021] - Add flux support to HPX to run on El Cap
- Issue #6615[1022] - Too many fails test after installed hpx
- Issue #6605[1023] - Partitionend vector copy constructor is broken
- Issue #6586[1024] - Bullet points in quick start/installing HPX section in documentation incorrectly rendered
- Issue #6563[1025] - Compilation issues on Grace Hopper
- Issue #6544[1026] - Errors in Public Distributed Api for all_to_all and gather_there
- Issue #6519[1027] - Option –hpx:queuing=local-priority-lifo is not configured
- Issue #6501[1028] - HPX 1.10 Failed Linking CXX executable for arm64-osx
- Issue #5728[1029] - Add optional fetch_content support for needed Boost libraries

[1012] https://github.com/TheHPXProject/hpx/issues/6699
[1013] https://github.com/TheHPXProject/hpx/issues/6696
[1014] https://github.com/TheHPXProject/hpx/issues/6691
[1015] https://github.com/TheHPXProject/hpx/issues/6681
[1016] https://github.com/TheHPXProject/hpx/issues/6658
[1017] https://github.com/TheHPXProject/hpx/issues/6648
[1018] https://github.com/TheHPXProject/hpx/issues/6640
[1019] https://github.com/TheHPXProject/hpx/issues/6636
[1020] https://github.com/TheHPXProject/hpx/issues/6623
[1021] https://github.com/TheHPXProject/hpx/issues/6616
[1022] https://github.com/TheHPXProject/hpx/issues/6615
[1023] https://github.com/TheHPXProject/hpx/issues/6605
[1024] https://github.com/TheHPXProject/hpx/issues/6586
[1025] https://github.com/TheHPXProject/hpx/issues/6563
[1026] https://github.com/TheHPXProject/hpx/issues/6544
[1027] https://github.com/TheHPXProject/hpx/issues/6519
[1028] https://github.com/TheHPXProject/hpx/issues/6501
[1029] https://github.com/TheHPXProject/hpx/issues/5728

Closed pull requests

- PR #6716[1030] - Fixing some of the reported linker warnings
- PR #6705[1031] - Adding gcc/15 to jenkins
- PR #6701[1032] - Attempting to fix shutdown hang on exception_info
- PR #6698[1033] - Making sure .hpp.in files are not being installed
- PR #6697[1034] - Minor docs fix
- PR #6695[1035] - Adding missing ';'
- PR #6693[1036] - Adding llvm/19 and 20 and cmake/4 Jenkins
- PR #6692[1037] - Better implementation of minimal_category
- PR #6690[1038] - Fixing bad #include in example
- PR #6689[1039] - Fix unreachable code warning in wait_all
- PR #6687[1040] - lci pp: change default ndevices=2 and progress_type=worker; improve document
- PR #6686[1041] - lci pp: upgrade LCI autofetch target to 1.7.9
- PR #6685[1042] - Improve run_on_all implementation and tests
- PR #6683[1043] - Fix bad element comparison for reduce_by_key
- PR #6682[1044] - Add C++23 std::generator equivalence test and fix missing semicolon
- PR #6680[1045] - Add oneapi device init workaround
- PR #6679[1046] - Fix sycl deprecations
- PR #6678[1047] - Fix oneapi overloads
- PR #6677[1048] - Offer a runtime manager object
- PR #6676[1049] - Mention the HPX book
- PR #6675[1050] - Bump required version of JSON library
- PR #6674[1051] - Issue 6631

[1030] https://github.com/TheHPXProject/hpx/pull/6716
[1031] https://github.com/TheHPXProject/hpx/pull/6705
[1032] https://github.com/TheHPXProject/hpx/pull/6701
[1033] https://github.com/TheHPXProject/hpx/pull/6698
[1034] https://github.com/TheHPXProject/hpx/pull/6697
[1035] https://github.com/TheHPXProject/hpx/pull/6695
[1036] https://github.com/TheHPXProject/hpx/pull/6693
[1037] https://github.com/TheHPXProject/hpx/pull/6692
[1038] https://github.com/TheHPXProject/hpx/pull/6690
[1039] https://github.com/TheHPXProject/hpx/pull/6689
[1040] https://github.com/TheHPXProject/hpx/pull/6687
[1041] https://github.com/TheHPXProject/hpx/pull/6686
[1042] https://github.com/TheHPXProject/hpx/pull/6685
[1043] https://github.com/TheHPXProject/hpx/pull/6683
[1044] https://github.com/TheHPXProject/hpx/pull/6682
[1045] https://github.com/TheHPXProject/hpx/pull/6680
[1046] https://github.com/TheHPXProject/hpx/pull/6679
[1047] https://github.com/TheHPXProject/hpx/pull/6678
[1048] https://github.com/TheHPXProject/hpx/pull/6677
[1049] https://github.com/TheHPXProject/hpx/pull/6676
[1050] https://github.com/TheHPXProject/hpx/pull/6675
[1051] https://github.com/TheHPXProject/hpx/pull/6674

- PR #6673[1052] - Fix: FindTBB.cmake cannot find correct TBB library. #6504
- PR #6672[1053] - Update modules.rst
- PR #6670[1054] - Add base template template param to execution_policy
- PR #6669[1055] - Add execution policy support to run_on_all
- PR #6667[1056] - Making sure bound threads are rescheduled on their original core
- PR #6666[1057] - Improve documentation for reduction operations
- PR #6664[1058] - Fix CMake template when fetching Boost
- PR #6663[1059] - More run_on_all overloads
- PR #6662[1060] - Fix "unary minus operator applied to unsigned type" warning
- PR #6661[1061] - Adding simple experimental::run_on_all
- PR #6659[1062] - fix(reduce): Initialize accumulator with init instead of first element
- PR #6656[1063] - Add missing channel_communicator::get_info
- PR #6652[1064] - Adding channel-based ping-pong example
- PR #6650[1065] - Adding constructor overloads to partitioned_vector
- PR #6649[1066] - Remove the use of deprecated asio::io_context::work
- PR #6645[1067] - Fixing collectives::exclusive_scan
- PR #6644[1068] - Update result_type in set_union.hpp
- PR #6643[1069] - Update result_type in set_union.hpp
- PR #6642[1070] - Allowing to use custom iterator tags with iterator_facade
- PR #6641[1071] - Allowing for zip-iterator to refer to sequences of different length
- PR #6639[1072] - docs: Fix spelling in example dictionary
- PR #6638[1073] - Update set_union.hpp
- PR #6637[1074] - lci/mpi pp: fix the case when non-zero-copy data is larger than INT_MAX

[1052] https://github.com/TheHPXProject/hpx/pull/6673
[1053] https://github.com/TheHPXProject/hpx/pull/6672
[1054] https://github.com/TheHPXProject/hpx/pull/6670
[1055] https://github.com/TheHPXProject/hpx/pull/6669
[1056] https://github.com/TheHPXProject/hpx/pull/6667
[1057] https://github.com/TheHPXProject/hpx/pull/6666
[1058] https://github.com/TheHPXProject/hpx/pull/6664
[1059] https://github.com/TheHPXProject/hpx/pull/6663
[1060] https://github.com/TheHPXProject/hpx/pull/6662
[1061] https://github.com/TheHPXProject/hpx/pull/6661
[1062] https://github.com/TheHPXProject/hpx/pull/6659
[1063] https://github.com/TheHPXProject/hpx/pull/6656
[1064] https://github.com/TheHPXProject/hpx/pull/6652
[1065] https://github.com/TheHPXProject/hpx/pull/6650
[1066] https://github.com/TheHPXProject/hpx/pull/6649
[1067] https://github.com/TheHPXProject/hpx/pull/6645
[1068] https://github.com/TheHPXProject/hpx/pull/6644
[1069] https://github.com/TheHPXProject/hpx/pull/6643
[1070] https://github.com/TheHPXProject/hpx/pull/6642
[1071] https://github.com/TheHPXProject/hpx/pull/6641
[1072] https://github.com/TheHPXProject/hpx/pull/6639
[1073] https://github.com/TheHPXProject/hpx/pull/6638
[1074] https://github.com/TheHPXProject/hpx/pull/6637

- PR #6635[1075] - Adding simplified reduction overload
- PR #6634[1076] - Fixed issue 6634: Unqualified calls to insertion_sort
- PR #6633[1077] - Increase timeouts for CircleCI tests
- PR #6630[1078] - Fix CPUId test
- PR #6628[1079] - Link aclocal with aclocal-1.16 as hwloc asks for it
- PR #6626[1080] - Fixing MPI parcel port issue exposed by #6623
- PR #6622[1081] - Newbranch:HPX-Based Task Scheduler with CUDA-Quantum Integration & Benchmarking
- PR #6621[1082] - HPX-Based Task Scheduler with CUDA-Quantum Integration & Benchmarking
- PR #6620[1083] - new test: very big tchunk
- PR #6619[1084] - mpi pp: fix transmission chunk send
- PR #6617[1085] - Adding support for the Flux job scheduling environment
- PR #6614[1086] - Fix fallback to module mode for CMake finding Boost
- PR #6613[1087] - Fix partitioned_vector_handle_values test
- PR #6612[1088] - Fixing naming convention for pp constant
- PR #6611[1089] - Fix Hwloc fetch content
- PR #6610[1090] - Add docs for synchronous collective operations
- PR #6609[1091] - Update perftest CI reference measurements
- PR #6608[1092] - Partially support data parallel for_loop
- PR #6607[1093] - Cleaning up copy_component facility
- PR #6606[1094] - Making sure copy_component creates a new gid
- PR #6600[1095] - Fixing sync collectives
- PR #6599[1096] - Make HPX_HAVE_RUN_MAIN_EVERYWHERE application specific

[1075] https://github.com/TheHPXProject/hpx/pull/6635
[1076] https://github.com/TheHPXProject/hpx/pull/6634
[1077] https://github.com/TheHPXProject/hpx/pull/6633
[1078] https://github.com/TheHPXProject/hpx/pull/6630
[1079] https://github.com/TheHPXProject/hpx/pull/6628
[1080] https://github.com/TheHPXProject/hpx/pull/6626
[1081] https://github.com/TheHPXProject/hpx/pull/6622
[1082] https://github.com/TheHPXProject/hpx/pull/6621
[1083] https://github.com/TheHPXProject/hpx/pull/6620
[1084] https://github.com/TheHPXProject/hpx/pull/6619
[1085] https://github.com/TheHPXProject/hpx/pull/6617
[1086] https://github.com/TheHPXProject/hpx/pull/6614
[1087] https://github.com/TheHPXProject/hpx/pull/6613
[1088] https://github.com/TheHPXProject/hpx/pull/6612
[1089] https://github.com/TheHPXProject/hpx/pull/6611
[1090] https://github.com/TheHPXProject/hpx/pull/6610
[1091] https://github.com/TheHPXProject/hpx/pull/6609
[1092] https://github.com/TheHPXProject/hpx/pull/6608
[1093] https://github.com/TheHPXProject/hpx/pull/6607
[1094] https://github.com/TheHPXProject/hpx/pull/6606
[1095] https://github.com/TheHPXProject/hpx/pull/6600
[1096] https://github.com/TheHPXProject/hpx/pull/6599

- PR #6598[1097] - Adding synchronous collective operations
- PR #6596[1098] - Minor fixes and optimizations
- PR #6595[1099] - Rfa parallel
- PR #6594[1100] - Move get_stack_ptr to source
- PR #6593[1101] - Fix outdated documentation and missing flags
- PR #6592[1102] - HPX_HAVE_THREADS_GET_STACK_POINTER to match builtin_frame_address feature test
- PR #6591[1103] - Feature test for __builtin_frame_address
- PR #6590[1104] - Add device guard for noexcept
- PR #6587[1105] - Fix bullet points in Quickstart
- PR #6585[1106] - Fixed escape characters format to handle warning due to misinterpretation of syntax
- PR #6583[1107] - Execute feature test for at_quick_exit
- PR #6582[1108] - Accommodate for CircleCI reduce available number of cores to two
- PR #6581[1109] - Attempting to work around a Boost.Spirit problem
- PR #6580[1110] - mpi pp: fix messages larger than INT_MAX
- PR #6578[1111] - Remove leftovers from libfabric parcelport
- PR #6577[1112] - Download Boost from their own archives, not from Sourceforge
- PR #6576[1113] - Fix CMake warning issued since CMake V3.30
- PR #6575[1114] - Replace previously downloaded CDash conv.xsl with local version
- PR #6570[1115] - Update exception_list.hpp
- PR #6569[1116] - Update exception_list.hpp
- PR #6567[1117] - Fix vectorization error on copy algorithm
- PR #6566[1118] - lci pp: fix messages larger than INT_MAX

[1097] https://github.com/TheHPXProject/hpx/pull/6598
[1098] https://github.com/TheHPXProject/hpx/pull/6596
[1099] https://github.com/TheHPXProject/hpx/pull/6595
[1100] https://github.com/TheHPXProject/hpx/pull/6594
[1101] https://github.com/TheHPXProject/hpx/pull/6593
[1102] https://github.com/TheHPXProject/hpx/pull/6592
[1103] https://github.com/TheHPXProject/hpx/pull/6591
[1104] https://github.com/TheHPXProject/hpx/pull/6590
[1105] https://github.com/TheHPXProject/hpx/pull/6587
[1106] https://github.com/TheHPXProject/hpx/pull/6585
[1107] https://github.com/TheHPXProject/hpx/pull/6583
[1108] https://github.com/TheHPXProject/hpx/pull/6582
[1109] https://github.com/TheHPXProject/hpx/pull/6581
[1110] https://github.com/TheHPXProject/hpx/pull/6580
[1111] https://github.com/TheHPXProject/hpx/pull/6578
[1112] https://github.com/TheHPXProject/hpx/pull/6577
[1113] https://github.com/TheHPXProject/hpx/pull/6576
[1114] https://github.com/TheHPXProject/hpx/pull/6575
[1115] https://github.com/TheHPXProject/hpx/pull/6570
[1116] https://github.com/TheHPXProject/hpx/pull/6569
[1117] https://github.com/TheHPXProject/hpx/pull/6567
[1118] https://github.com/TheHPXProject/hpx/pull/6566

- PR #6565[1119] - Moving most of APIs from hpx::parallel::execution to hpx::execution::experimental
- PR #6564[1120] - Remove superfluous HPX_MOVE()
- PR #6562[1121] - Fix doc return type of broadcast_to
- PR #6560[1122] - Fixes for bit_cast on 32bit systems
- PR #6559[1123] - Making sure that all parcelport counters are unavailable if no networking is needed or configured
- PR #6558[1124] - Remove CSCS CI's
- PR #6556[1125] - Set copyright year in generated files
- PR #6553[1126] - Fix omp vectorization pragma errors
- PR #6551[1127] - Update building_hpx.rst
- PR #6550[1128] - Partitioned vector updates
- PR #6549[1129] - Fix CMake conditionals checking ENV variables
- PR #6548[1130] - Update CONTRIBUTING.md
- PR #6546[1131] - Fix incorrect signature of distributed API functions
- PR #6543[1132] - Throwing an exception derived from std::bad_alloc on OOM conditions
- PR #6539[1133] - Use thread-safe cache in thread_local_caching_allocator
- PR #6537[1134] - Update README.rst
- PR #6531[1135] - More fixes for the Boost package
- PR #6527[1136] - Improve the LCI parcelport documentation
- PR #6525[1137] - Addressing cmake warnings issued starting V3.30
- PR #6522[1138] - Fixing distance test
- PR #6520[1139] - Adding optional handshakes to acknowledge the received data
- PR #6518[1140] - Make sure that –hpx:ini log settings take effect

[1119] https://github.com/TheHPXProject/hpx/pull/6565
[1120] https://github.com/TheHPXProject/hpx/pull/6564
[1121] https://github.com/TheHPXProject/hpx/pull/6562
[1122] https://github.com/TheHPXProject/hpx/pull/6560
[1123] https://github.com/TheHPXProject/hpx/pull/6559
[1124] https://github.com/TheHPXProject/hpx/pull/6558
[1125] https://github.com/TheHPXProject/hpx/pull/6556
[1126] https://github.com/TheHPXProject/hpx/pull/6553
[1127] https://github.com/TheHPXProject/hpx/pull/6551
[1128] https://github.com/TheHPXProject/hpx/pull/6550
[1129] https://github.com/TheHPXProject/hpx/pull/6549
[1130] https://github.com/TheHPXProject/hpx/pull/6548
[1131] https://github.com/TheHPXProject/hpx/pull/6546
[1132] https://github.com/TheHPXProject/hpx/pull/6543
[1133] https://github.com/TheHPXProject/hpx/pull/6539
[1134] https://github.com/TheHPXProject/hpx/pull/6537
[1135] https://github.com/TheHPXProject/hpx/pull/6531
[1136] https://github.com/TheHPXProject/hpx/pull/6527
[1137] https://github.com/TheHPXProject/hpx/pull/6525
[1138] https://github.com/TheHPXProject/hpx/pull/6522
[1139] https://github.com/TheHPXProject/hpx/pull/6520
[1140] https://github.com/TheHPXProject/hpx/pull/6518

- PR #6512[1141] - Minor cleanup of future_data
- PR #6510[1142] - Include Boost as CMake subproject
- PR #6509[1143] - Add components documentation
- PR #6508[1144] - Fix typo: s/unititiallized/uninitialized/
- PR #6507[1145] - Update LSU Jenkins libraries to match Rostam 3.0 with RHEL9
- PR #6503[1146] - Fix 2 tests on FreeBSD by initializing freebsd_environ
- PR #6499[1147] - Fix crash in get_executable_filename on FreeBSD
- PR #6498[1148] - Avoid rewriting defines.hpp
- PR #6497[1149] - Contains and contains_subrange parallel algorithm implementation GSOC 2024
- PR #6496[1150] - Prevent usage of CMake try_run on crosscompiling
- PR #6494[1151] - Add unit test cases and fixes for the S/R versions of the parallel algorithms
- PR #6487[1152] - Fixing security vulnerabilities reported by MSVC security checks
- PR #6486[1153] - Create codeql.yml
- PR #6474[1154] - Remove remnants of libfabric parcelport
- PR #6473[1155] - Add documentation for distributed implementations of post, async, sync and dataflow
- PR #6471[1156] - Add distance.cpp test in CMake
- PR #6468[1157] - Small vector relocation
- PR #6448[1158] - Standardising Benchmarks, with support for nanobench as an option for its backend
- PR #6365[1159] - Release V1.10.0
- PR #6089[1160] - Implementing p2079

[1141] https://github.com/TheHPXProject/hpx/pull/6512
[1142] https://github.com/TheHPXProject/hpx/pull/6510
[1143] https://github.com/TheHPXProject/hpx/pull/6509
[1144] https://github.com/TheHPXProject/hpx/pull/6508
[1145] https://github.com/TheHPXProject/hpx/pull/6507
[1146] https://github.com/TheHPXProject/hpx/pull/6503
[1147] https://github.com/TheHPXProject/hpx/pull/6499
[1148] https://github.com/TheHPXProject/hpx/pull/6498
[1149] https://github.com/TheHPXProject/hpx/pull/6497
[1150] https://github.com/TheHPXProject/hpx/pull/6496
[1151] https://github.com/TheHPXProject/hpx/pull/6494
[1152] https://github.com/TheHPXProject/hpx/pull/6487
[1153] https://github.com/TheHPXProject/hpx/pull/6486
[1154] https://github.com/TheHPXProject/hpx/pull/6474
[1155] https://github.com/TheHPXProject/hpx/pull/6473
[1156] https://github.com/TheHPXProject/hpx/pull/6471
[1157] https://github.com/TheHPXProject/hpx/pull/6468
[1158] https://github.com/TheHPXProject/hpx/pull/6448
[1159] https://github.com/TheHPXProject/hpx/pull/6365
[1160] https://github.com/TheHPXProject/hpx/pull/6089

2.11.2 List of older releases

HPX V1.10.0 (May 29, 2024)

General changes

- The HPX documentation has seen a major overhaul for this release. We finished documenting the public local HPX API, we have added migration guides from widely used parallelization platforms to HPX (OpenMP, TBB, and MPI).
- We have added facilities enabling optimizations for trivially-relocatable types (see P1144[1161] for more details).
- We have added (and use) the `scope_xxx` helper facilities as specified by the C++ library fundamentals TS v3 (see: N4948[1162]).
- We have added configuration options that allow to build HPX without pre-installing any prerequisites. Use `HPX_WITH_FETCH_HWLOC=On` to have Portable Hardware Locality (HWLOC)[1163] installed for you. Similarly, setting `HPX_WITH_FETCH_BOOST=On` during configuration time will install the necessary Boost[1164] libraries (currently V1.84.0).
- We have performed a lot of code cleanup and refactoring to improve the overall code quality and decrease compile times.
- The collective operations APIs have seen an unification, we have fixed issues and performance problems for the collectives.
- The HPX executors have seen a streamlining and some consistency changes. We have applied many performance improvements to the executor implementations that directly positively impact the performance of our parallel algorithms.
- We have added a new parcelport allowing to use Gasnet as a communication platform.
- We have added optimizations to various parcelports improving overall communication performance. This includes - amongst other things - send immediate optimizations and receiver-side zero-copy optimizations.
- Futures will now execute the associated task eagerly and inline on any wait operation if the task has not started running yet. This feature can be enabled using the `HPX_COROUTINES_WITH_THREAD_SCHEDULE_HINT_RUNS_AS_CHILD=On` configuration setting (which is *Off* by default).
- We have enabled using json files to supply configuration information through the command line. This feature can be enabled with the configuration option `HPX_COMMAND_LINE_HANDLING_WITH_JSON_CONFIGURATION_FILES=On`. This functionality depends on the external JSon library[1165], which can be built at configuration time by supplying `HPX_WITH_FETCH_JSON=On` to CMake[1166].
- We have applied many fixes to our CUDA, ROCm, and SYCL build environments.

[1161] https://wg21.link/p1144
[1162] http://wg21.link/n4948
[1163] https://www.open-mpi.org/projects/hwloc/
[1164] https://www.boost.org/
[1165] https://github.com/nlohmann/json
[1166] https://www.cmake.org

Breaking changes

- The CMake[1167] configuration keys SOMELIB_ROOT (e.g., BOOST_ROOT) have been renamed to Somelib_ROOT (e.g., Boost_ROOT) to avoid warnings when using newer versions of CMake[1168]. Please update your scripts accordingly. For now, the old variable names are re-assigned to the new names and unset in the CMake[1169] cache.

Closed issues

- Issue #6466[1170] - No access limitations to Wiki
- Issue #6461[1171] - handle_received_parcels may never return
- Issue #6459[1172] - Building HPX
- Issue #6451[1173] - HPX hangs at the very end
- Issue #6446[1174] - Issue on page /manual/getting_hpx.html
- Issue #6443[1175] - PR #6435 (parcel_layer_tweaks) broke Octo-Tiger
- Issue #6440[1176] - HPX does not compile with MSVC of Visual Studio 2022 17.9+
- Issue #6437[1177] - HPX 1.9.1 does not compile on Fedora with '#pragma message: [Parallel STL message]: "Vectorized algorithm unimplemented, redirected to serial
- Issue #6419[1178] - Enhancement of the macro functionalities within hpx
- Issue #6417[1179] - The current HPX master branch is still not compatible with Kokkos 4.0.1
- Issue #6414[1180] - Current HPX master causes segfaults within Octo-Tiger
- Issue #6412[1181] - Clangd (Language Server) throws error for __integer_pack at pack.hpp
- Issue #6407[1182] - Cannot build Kokkos 4.0.01 with current HPX master
- Issue #6405[1183] - Spack Build Error with ROCm 5.7.0
- Issue #6398[1184] - HPX sets affinity wrong with multiple processes per node and LCI parcelport enabled
- Issue #6392[1185] - [Feature] Install dependencies using CMake

[1167] https://www.cmake.org
[1168] https://www.cmake.org
[1169] https://www.cmake.org
[1170] https://github.com/TheHPXProject/hpx/issues/6466
[1171] https://github.com/TheHPXProject/hpx/issues/6461
[1172] https://github.com/TheHPXProject/hpx/issues/6459
[1173] https://github.com/TheHPXProject/hpx/issues/6451
[1174] https://github.com/TheHPXProject/hpx/issues/6446
[1175] https://github.com/TheHPXProject/hpx/issues/6443
[1176] https://github.com/TheHPXProject/hpx/issues/6440
[1177] https://github.com/TheHPXProject/hpx/issues/6437
[1178] https://github.com/TheHPXProject/hpx/issues/6419
[1179] https://github.com/TheHPXProject/hpx/issues/6417
[1180] https://github.com/TheHPXProject/hpx/issues/6414
[1181] https://github.com/TheHPXProject/hpx/issues/6412
[1182] https://github.com/TheHPXProject/hpx/issues/6407
[1183] https://github.com/TheHPXProject/hpx/issues/6405
[1184] https://github.com/TheHPXProject/hpx/issues/6398
[1185] https://github.com/TheHPXProject/hpx/issues/6392

- Issue #6388[1186] - HPX error: "Host not found" when running on Expanse with 128 nodes
- Issue #6366[1187] - serialize_buffer allocator support needs adjustments
- Issue #6361[1188] - HPX 1.9.1 does not compile on Fedora 40
- Issue #6355[1189] - Single page documentation is broken
- Issue #6334[1190] - Segmentation fault after adding a padding in one_size_heap_list
- Issue #6329[1191] - Log hpx threads on forced shutdown
- Issue #6316[1192] - Build breaks on FreeBSD
- Issue #6299[1193] - HPX does not use distributed localities on Fugaku
- Issue #6298[1194] - Update config for coroutines on ARM
- Issue #6291[1195] - Zero-copy receive optimization disabled the invocation of direct actions
- Issue #6261[1196] - Add optional reading of json files for command line options
- Issue #6087[1197] - Support for vcpkg on Linux is broken
- Issue #5921[1198] - hpx::info claims that async_mpi was not built, while cmake assures its existence
- Issue #5893[1199] - Tests fail on FreeBSD: Executable copyn_test does not exist
- Issue #5833[1200] - barrier lockup
- Issue #5799[1201] - Investigate CUDA compilation problems
- Issue #5340[1202] - Examples do not run on Mac OSX using the M1 chip

Closed pull requests

- PR #6493[1203] - Fix distributed latch documentation
- PR #6492[1204] - Fix kokkos hpx nvcc compilation
- PR #6491[1205] - More fixes to handling bool arguments for collective operations
- PR #6490[1206] - Remove the default max cpu count

[1186] https://github.com/TheHPXProject/hpx/issues/6388
[1187] https://github.com/TheHPXProject/hpx/issues/6366
[1188] https://github.com/TheHPXProject/hpx/issues/6361
[1189] https://github.com/TheHPXProject/hpx/issues/6355
[1190] https://github.com/TheHPXProject/hpx/issues/6334
[1191] https://github.com/TheHPXProject/hpx/issues/6329
[1192] https://github.com/TheHPXProject/hpx/issues/6316
[1193] https://github.com/TheHPXProject/hpx/issues/6299
[1194] https://github.com/TheHPXProject/hpx/issues/6298
[1195] https://github.com/TheHPXProject/hpx/issues/6291
[1196] https://github.com/TheHPXProject/hpx/issues/6261
[1197] https://github.com/TheHPXProject/hpx/issues/6087
[1198] https://github.com/TheHPXProject/hpx/issues/5921
[1199] https://github.com/TheHPXProject/hpx/issues/5893
[1200] https://github.com/TheHPXProject/hpx/issues/5833
[1201] https://github.com/TheHPXProject/hpx/issues/5799
[1202] https://github.com/TheHPXProject/hpx/issues/5340
[1203] https://github.com/TheHPXProject/hpx/pull/6493
[1204] https://github.com/TheHPXProject/hpx/pull/6492
[1205] https://github.com/TheHPXProject/hpx/pull/6491
[1206] https://github.com/TheHPXProject/hpx/pull/6490

- PR #6489[1207] - Ensure TCP parcelport is deactivated if not needed
- PR #6488[1208] - Fixing handling of bool value type for collective operations
- PR #6485[1209] - Destructive interference size
- PR #6484[1210] - Improve performance counter error handling
- PR #6482[1211] - Generalize the notion of bitwise serialization
- PR #6481[1212] - Fixing use of HPX_WITH_CXX_STANDARD
- PR #6480[1213] - Remove equal_to from hpx::any
- PR #6479[1214] - Remove optimizations for certain built-in compiler intrinsics
- PR #6478[1215] - Fixing issues on MacOS
- PR #6477[1216] - lci pp: lci's github repo name changed from LC to lci
- PR #6476[1217] - Fixing binary filter test target names
- PR #6475[1218] - Fix mac os github actions
- PR #6472[1219] - Troubleshoot CI hangs
- PR #6469[1220] - improve(lci pp): more options to control the LCI parcelport
- PR #6467[1221] - Bump jwlawson/actions-setup-cmake from 1.14 to 2.0
- PR #6464[1222] - Update docs of "Writing distributed applications" page
- PR #6463[1223] - Revert "Always return outermost thread id"
- PR #6458[1224] - Reduce test workload to fix CI/CD time-out
- PR #6457[1225] - replace boost::array with std::array and update file name
- PR #6456[1226] - Move APEX CI to rostam
- PR #6455[1227] - Fixing compilation if HPX_HAVE_THREAD_QUEUE_WAITTIME is defined
- PR #6454[1228] - Update perftests reference measurements
- PR #6453[1229] - Update supported platforms of Manual/Prerequisites page

[1207] https://github.com/TheHPXProject/hpx/pull/6489
[1208] https://github.com/TheHPXProject/hpx/pull/6488
[1209] https://github.com/TheHPXProject/hpx/pull/6485
[1210] https://github.com/TheHPXProject/hpx/pull/6484
[1211] https://github.com/TheHPXProject/hpx/pull/6482
[1212] https://github.com/TheHPXProject/hpx/pull/6481
[1213] https://github.com/TheHPXProject/hpx/pull/6480
[1214] https://github.com/TheHPXProject/hpx/pull/6479
[1215] https://github.com/TheHPXProject/hpx/pull/6478
[1216] https://github.com/TheHPXProject/hpx/pull/6477
[1217] https://github.com/TheHPXProject/hpx/pull/6476
[1218] https://github.com/TheHPXProject/hpx/pull/6475
[1219] https://github.com/TheHPXProject/hpx/pull/6472
[1220] https://github.com/TheHPXProject/hpx/pull/6469
[1221] https://github.com/TheHPXProject/hpx/pull/6467
[1222] https://github.com/TheHPXProject/hpx/pull/6464
[1223] https://github.com/TheHPXProject/hpx/pull/6463
[1224] https://github.com/TheHPXProject/hpx/pull/6458
[1225] https://github.com/TheHPXProject/hpx/pull/6457
[1226] https://github.com/TheHPXProject/hpx/pull/6456
[1227] https://github.com/TheHPXProject/hpx/pull/6455
[1228] https://github.com/TheHPXProject/hpx/pull/6454
[1229] https://github.com/TheHPXProject/hpx/pull/6453

- PR #6452[1230] - Fix nvcc crashes in transform_stream.cu and synchronize.cu
- PR #6450[1231] - Fix git tag name in Getting HPX page
- PR #6449[1232] - LCI parcelport: add yield to potentially infinite retry loop
- PR #6447[1233] - Use compressed ptr in schedulers when 128 atomics are not lockfree
- PR #6445[1234] - Fix agas addressing cache
- PR #6444[1235] - Update CTestConfig.cmake
- PR #6442[1236] - Update CMakeLists.txt
- PR #6441[1237] - Minor documentation fixes
- PR #6439[1238] - Optimizing use of certain #includes
- PR #6438[1239] - Bump jwlawson/actions-setup-cmake from 1.14 to 2.0
- PR #6436[1240] - Update docs
- PR #6435[1241] - Parcel layer tweaks
- PR #6434[1242] - improve termination detection: removing lock from critical path
- PR #6433[1243] - Use shared mutex for resolve_locality procedure
- PR #6432[1244] - Module cleanup up to level 30
- PR #6429[1245] - Making sure HPX_WITH_ASYNC_MPI is reported properly
- PR #6427[1246] - Modifying CMakeLists to copy libhwloc-15.dll to the binary folder in Windows, independently
- PR #6425[1247] - Fix macOS failing test
- PR #6424[1248] - Adding option for downloading Boost using CMake FetchContent
- PR #6423[1249] - Move adjacent_difference to numeric header file
- PR #6422[1250] - Adding steal-half functionalities to work-requesting scheduler
- PR #6421[1251] - Bump actions/checkout from 2 to 4
- PR #6418[1252] - Working around nvcc problems to use CTAD

[1230] https://github.com/TheHPXProject/hpx/pull/6452
[1231] https://github.com/TheHPXProject/hpx/pull/6450
[1232] https://github.com/TheHPXProject/hpx/pull/6449
[1233] https://github.com/TheHPXProject/hpx/pull/6447
[1234] https://github.com/TheHPXProject/hpx/pull/6445
[1235] https://github.com/TheHPXProject/hpx/pull/6444
[1236] https://github.com/TheHPXProject/hpx/pull/6442
[1237] https://github.com/TheHPXProject/hpx/pull/6441
[1238] https://github.com/TheHPXProject/hpx/pull/6439
[1239] https://github.com/TheHPXProject/hpx/pull/6438
[1240] https://github.com/TheHPXProject/hpx/pull/6436
[1241] https://github.com/TheHPXProject/hpx/pull/6435
[1242] https://github.com/TheHPXProject/hpx/pull/6434
[1243] https://github.com/TheHPXProject/hpx/pull/6433
[1244] https://github.com/TheHPXProject/hpx/pull/6432
[1245] https://github.com/TheHPXProject/hpx/pull/6429
[1246] https://github.com/TheHPXProject/hpx/pull/6427
[1247] https://github.com/TheHPXProject/hpx/pull/6425
[1248] https://github.com/TheHPXProject/hpx/pull/6424
[1249] https://github.com/TheHPXProject/hpx/pull/6423
[1250] https://github.com/TheHPXProject/hpx/pull/6422
[1251] https://github.com/TheHPXProject/hpx/pull/6421
[1252] https://github.com/TheHPXProject/hpx/pull/6418

- PR #6416[1253] - Change run_as_os_thread deprecation forwarding due to hipcc compilation issue
- PR #6415[1254] - Attempting to avoid segfault in OctoTiger during initialization
- PR #6413[1255] - Always return outermost thread id
- PR #6411[1256] - Minor refactoring and fixes to the LCI parcelport and ping-pong_performance2 benchmark
- PR #6410[1257] - Adding scope_xxx from library fundamentals TS v3
- PR #6409[1258] - Working around CUDA issue
- PR #6408[1259] - Tightening up collective operation semantics
- PR #6406[1260] - Working around ROCm compiler issue
- PR #6404[1261] - Allow to disable use of [[no_unique_address]] attribute
- PR #6403[1262] - Fixing copyright year
- PR #6402[1263] - fix(lci pp): fix deadlocks with too many failed sends
- PR #6401[1264] - fix(lci pp): fix the null_thread_id bug in the LCI parcelport
- PR #6400[1265] - Fix the affinity setting bug when using LCI pp and multiple localities per node
- PR #6397[1266] - Change API header titles and info
- PR #6396[1267] - Making is_bitwise_serializable SFINAE-friendly
- PR #6395[1268] - Adapt amount of collective testing
- PR #6394[1269] - Adding option for installing Hwloc using CMake FetchContent
- PR #6393[1270] - Optionally disable caching allocator
- PR #6391[1271] - Cleaning up collective operations
- PR #6390[1272] - Making function local constexpr variables non-static
- PR #6389[1273] - Disable resolving hostnames if TCP is disabled
- PR #6387[1274] - Need to break out of the loop when searching the suffixes.

[1253] https://github.com/TheHPXProject/hpx/pull/6416
[1254] https://github.com/TheHPXProject/hpx/pull/6415
[1255] https://github.com/TheHPXProject/hpx/pull/6413
[1256] https://github.com/TheHPXProject/hpx/pull/6411
[1257] https://github.com/TheHPXProject/hpx/pull/6410
[1258] https://github.com/TheHPXProject/hpx/pull/6409
[1259] https://github.com/TheHPXProject/hpx/pull/6408
[1260] https://github.com/TheHPXProject/hpx/pull/6406
[1261] https://github.com/TheHPXProject/hpx/pull/6404
[1262] https://github.com/TheHPXProject/hpx/pull/6403
[1263] https://github.com/TheHPXProject/hpx/pull/6402
[1264] https://github.com/TheHPXProject/hpx/pull/6401
[1265] https://github.com/TheHPXProject/hpx/pull/6400
[1266] https://github.com/TheHPXProject/hpx/pull/6397
[1267] https://github.com/TheHPXProject/hpx/pull/6396
[1268] https://github.com/TheHPXProject/hpx/pull/6395
[1269] https://github.com/TheHPXProject/hpx/pull/6394
[1270] https://github.com/TheHPXProject/hpx/pull/6393
[1271] https://github.com/TheHPXProject/hpx/pull/6391
[1272] https://github.com/TheHPXProject/hpx/pull/6390
[1273] https://github.com/TheHPXProject/hpx/pull/6389
[1274] https://github.com/TheHPXProject/hpx/pull/6387

- PR #6384[1275] - Fixing allocation/deallocation mismatch in serialize_buffer
- PR #6383[1276] - Enable fork_join_executor to handle return values from scheduled functions
- PR #6381[1277] - Consistently treat conflicting parameters provided by executors and parameter objects
- PR #6380[1278] - Fixing setting an annotation for an execution policy
- PR #6378[1279] - Allowing to disable signal handlers
- PR #6377[1280] - Fix gasnet-related test failures
- PR #6375[1281] - Update LSU Jenkins with 2023-10 libraries
- PR #6374[1282] - Investigate builder gasnet failure
- PR #6373[1283] - Fixing communicator API, adding docs
- PR #6372[1284] - Fix resource partitioner tests for small thread count
- PR #6371[1285] - Fix jacobi omp examples.
- PR #6370[1286] - improve one_size_heap_list: use rwlock to speedup the allocation/free
- PR #6369[1287] - working issue with MPI_CC / CC conflict in automake
- PR #6368[1288] - Making sure serialize_buffer properly destroys buffer, if needed.
- PR #6367[1289] - Fix parallel relocation test
- PR #6364[1290] - Relocation variants
- PR #6363[1291] - Update the lci parcelport to use LCI v1.7.6
- PR #6362[1292] - Fixing compilation problems on 32 Linux systems
- PR #6360[1293] - Fix broken links in docs: PDF, Single HTML page, Dependency report
- PR #6359[1294] - Fix header file links in Public API page
- PR #6358[1295] - Fix CMake find_library for HWLOC
- PR #6357[1296] - Replace Custom Benchmarking Code with Nanobench

[1275] https://github.com/TheHPXProject/hpx/pull/6384
[1276] https://github.com/TheHPXProject/hpx/pull/6383
[1277] https://github.com/TheHPXProject/hpx/pull/6381
[1278] https://github.com/TheHPXProject/hpx/pull/6380
[1279] https://github.com/TheHPXProject/hpx/pull/6378
[1280] https://github.com/TheHPXProject/hpx/pull/6377
[1281] https://github.com/TheHPXProject/hpx/pull/6375
[1282] https://github.com/TheHPXProject/hpx/pull/6374
[1283] https://github.com/TheHPXProject/hpx/pull/6373
[1284] https://github.com/TheHPXProject/hpx/pull/6372
[1285] https://github.com/TheHPXProject/hpx/pull/6371
[1286] https://github.com/TheHPXProject/hpx/pull/6370
[1287] https://github.com/TheHPXProject/hpx/pull/6369
[1288] https://github.com/TheHPXProject/hpx/pull/6368
[1289] https://github.com/TheHPXProject/hpx/pull/6367
[1290] https://github.com/TheHPXProject/hpx/pull/6364
[1291] https://github.com/TheHPXProject/hpx/pull/6363
[1292] https://github.com/TheHPXProject/hpx/pull/6362
[1293] https://github.com/TheHPXProject/hpx/pull/6360
[1294] https://github.com/TheHPXProject/hpx/pull/6359
[1295] https://github.com/TheHPXProject/hpx/pull/6358
[1296] https://github.com/TheHPXProject/hpx/pull/6357

- PR #6356[1297] - Fixed matrix multiplication example output
- PR #6354[1298] - Fix broken links for header files in Public API page
- PR #6353[1299] - Enable using std::reference_wrapper with executor parameters
- PR #6352[1300] - Add Public distributed API documentation
- PR #6350[1301] - Make coverage work with Jenkins Github Branch Source plugin
- PR #6349[1302] - Moving hpx::threads::run_as_xxx to namespace hpx
- PR #6348[1303] - Adding –exclusive to launching tests on rostam
- PR #6346[1304] - changed chat link to discord
- PR #6344[1305] - uninitialized_relocate w/ type_support primitive
- PR #6343[1306] - Bump actions/checkout from 3 to 4
- PR #6342[1307] - Fix HPX-APEX cmake integration
- PR #6341[1308] - Fix shared_future_continuation_order regression test
- PR #6340[1309] - Log alive hpx threads on exit
- PR #6339[1310] - Add coverage testing on Jenkins
- PR #6338[1311] - Fixing HPX_CURRENT_SOURCE_LOCATION when std::source_location exists
- PR #6337[1312] - Remove aurianer, biddisco, and msimberg from codeowners
- PR #6336[1313] - More cleaning up for module levels 19-20
- PR #6335[1314] - Finalize the MPI docs of the Migration Guide
- PR #6332[1315] - More fixes for CMake V3.27
- PR #6330[1316] - Adding basic logging to collective operations
- PR #6328[1317] - Cleanup previous patch adapting to CMake V3.27
- PR #6327[1318] - Modernize modules in level 17 and 18
- PR #6324[1319] - P1144 Relocation primitives

[1297] https://github.com/TheHPXProject/hpx/pull/6356
[1298] https://github.com/TheHPXProject/hpx/pull/6354
[1299] https://github.com/TheHPXProject/hpx/pull/6353
[1300] https://github.com/TheHPXProject/hpx/pull/6352
[1301] https://github.com/TheHPXProject/hpx/pull/6350
[1302] https://github.com/TheHPXProject/hpx/pull/6349
[1303] https://github.com/TheHPXProject/hpx/pull/6348
[1304] https://github.com/TheHPXProject/hpx/pull/6346
[1305] https://github.com/TheHPXProject/hpx/pull/6344
[1306] https://github.com/TheHPXProject/hpx/pull/6343
[1307] https://github.com/TheHPXProject/hpx/pull/6342
[1308] https://github.com/TheHPXProject/hpx/pull/6341
[1309] https://github.com/TheHPXProject/hpx/pull/6340
[1310] https://github.com/TheHPXProject/hpx/pull/6339
[1311] https://github.com/TheHPXProject/hpx/pull/6338
[1312] https://github.com/TheHPXProject/hpx/pull/6337
[1313] https://github.com/TheHPXProject/hpx/pull/6336
[1314] https://github.com/TheHPXProject/hpx/pull/6335
[1315] https://github.com/TheHPXProject/hpx/pull/6332
[1316] https://github.com/TheHPXProject/hpx/pull/6330
[1317] https://github.com/TheHPXProject/hpx/pull/6328
[1318] https://github.com/TheHPXProject/hpx/pull/6327
[1319] https://github.com/TheHPXProject/hpx/pull/6324

- PR #6321[1320] - Ensure hpx_main is a proper thread_function
- PR #6320[1321] - Fixing cyclic dependencies in naming and agas modules
- PR #6319[1322] - Generate git tag if needed but it is not available
- PR #6317[1323] - Fixing linker problem on FreeBSD
- PR #6315[1324] - acknowledge triv-rel and nothrow-rel types
- PR #6314[1325] - Relocation algorithms Clean
- PR #6313[1326] - Trivial relocation of c-v-ref-array types
- PR #6312[1327] - Fixing warning/error
- PR #6311[1328] - Adding executor parallel invoke CPOs
- PR #6310[1329] - Define HPX_COMPUTE_CODE in builds with SYCL
- PR #6309[1330] - Making sure changed number of cores is propagated to executor
- PR #6308[1331] - openshmem-parcelport initial import
- PR #6306[1332] - The hpxcxx script was broken such that it could only compile for _release
- PR #6305[1333] - Adapting build system for CMake V3.27
- PR #6304[1334] - Fixing an integral type mismatch warning
- PR #6303[1335] - omp for default vectorization
- PR #6301[1336] - Add MPI migration guide
- PR #6294[1337] - Add internal reference counting to semaphores
- PR #6286[1338] - Simd helpers
- PR #6280[1339] - Add TBB to HPX documentation in Migration Guide
- PR #6276[1340] - Add dependabot.yml
- PR #6275[1341] - Revert "Move dependabot.yml into correct directory"
- PR #6272[1342] - set thread name for linux

[1320] https://github.com/TheHPXProject/hpx/pull/6321
[1321] https://github.com/TheHPXProject/hpx/pull/6320
[1322] https://github.com/TheHPXProject/hpx/pull/6319
[1323] https://github.com/TheHPXProject/hpx/pull/6317
[1324] https://github.com/TheHPXProject/hpx/pull/6315
[1325] https://github.com/TheHPXProject/hpx/pull/6314
[1326] https://github.com/TheHPXProject/hpx/pull/6313
[1327] https://github.com/TheHPXProject/hpx/pull/6312
[1328] https://github.com/TheHPXProject/hpx/pull/6311
[1329] https://github.com/TheHPXProject/hpx/pull/6310
[1330] https://github.com/TheHPXProject/hpx/pull/6309
[1331] https://github.com/TheHPXProject/hpx/pull/6308
[1332] https://github.com/TheHPXProject/hpx/pull/6306
[1333] https://github.com/TheHPXProject/hpx/pull/6305
[1334] https://github.com/TheHPXProject/hpx/pull/6304
[1335] https://github.com/TheHPXProject/hpx/pull/6303
[1336] https://github.com/TheHPXProject/hpx/pull/6301
[1337] https://github.com/TheHPXProject/hpx/pull/6294
[1338] https://github.com/TheHPXProject/hpx/pull/6286
[1339] https://github.com/TheHPXProject/hpx/pull/6280
[1340] https://github.com/TheHPXProject/hpx/pull/6276
[1341] https://github.com/TheHPXProject/hpx/pull/6275
[1342] https://github.com/TheHPXProject/hpx/pull/6272

- PR #6271[1343] - Uninitialised algorithms, move using std::memcpy
- PR #6270[1344] - Bump jwlawson/actions-setup-cmake from 1.9 to 1.14
- PR #6269[1345] - Bump actions/checkout from 2 to 3
- PR #6268[1346] - Move dependabot.yml into correct directory
- PR #6265[1347] - Create dependabot.yml
- PR #6264[1348] - hpx::is_trivially_relocatable trait implementation
- PR #6263[1349] - Adding support for reading json configuration files for command line options
- PR #6249[1350] - Implement the send immediate optimization for the MPI parcelport.
- PR #6237[1351] - Improve compilation performance
- PR #6234[1352] - Adding release notes page for next release
- PR #6233[1353] - Moving is_relocatable to namespace hpx
- PR #6230[1354] - gasnet based parcelport
- PR #6226[1355] - Re-enable dependency on segmented algorithms on CircleCI
- PR #6220[1356] - Add execution on
- PR #6212[1357] - Initial trait definition for *relocatable*
- PR #6199[1358] - added support for unseq, par_unseq for hpx::make_heap algorithm
- PR #6173[1359] - C++ modules
- PR #6122[1360] - Add Module support
- PR #6099[1361] - Futures attempt to execute threads directly if those have not started executing
- PR #6050[1362] - Investigating partitioned_vector problems
- PR #5988[1363] - Adding CI configuration for DGX-A100 at LSU
- PR #5910[1364] - Improve MPI initialization

[1343] https://github.com/TheHPXProject/hpx/pull/6271
[1344] https://github.com/TheHPXProject/hpx/pull/6270
[1345] https://github.com/TheHPXProject/hpx/pull/6269
[1346] https://github.com/TheHPXProject/hpx/pull/6268
[1347] https://github.com/TheHPXProject/hpx/pull/6265
[1348] https://github.com/TheHPXProject/hpx/pull/6264
[1349] https://github.com/TheHPXProject/hpx/pull/6263
[1350] https://github.com/TheHPXProject/hpx/pull/6249
[1351] https://github.com/TheHPXProject/hpx/pull/6237
[1352] https://github.com/TheHPXProject/hpx/pull/6234
[1353] https://github.com/TheHPXProject/hpx/pull/6233
[1354] https://github.com/TheHPXProject/hpx/pull/6230
[1355] https://github.com/TheHPXProject/hpx/pull/6226
[1356] https://github.com/TheHPXProject/hpx/pull/6220
[1357] https://github.com/TheHPXProject/hpx/pull/6212
[1358] https://github.com/TheHPXProject/hpx/pull/6199
[1359] https://github.com/TheHPXProject/hpx/pull/6173
[1360] https://github.com/TheHPXProject/hpx/pull/6122
[1361] https://github.com/TheHPXProject/hpx/pull/6099
[1362] https://github.com/TheHPXProject/hpx/pull/6050
[1363] https://github.com/TheHPXProject/hpx/pull/5988
[1364] https://github.com/TheHPXProject/hpx/pull/5910

- PR #5845[1365] - Adding local work requesting scheduler that is based on message passing internally

HPX V1.9.1 (August 4, 2023)

General changes

This point release fixes a couple of problems reported for the V1.9.0 release. Most importantly, we fixed various occasional hanging during startup and shutdown in distributed scenarios. We also added support for zero-copy serialization on the receiving side to the TCP, MPI, and LCI parcelports. Last but not least, we have added support for Visual Studio 2019 and gcc using MINGW on Windows, and also support for gcc V13 and clang V15.

HPX headers are now made consistently named the same as their standard library counterparts, e.g. *#include <thread>* now corresponds to *#include <hpx/thread.hpp>*. This significantly simplifies porting existing standards conforming codes to HPX.

A lot of work has been done to improve and optimize our network communication layers. Primary focus of this work was on the LCI parcelport, but we have also cleaned up and improved the MPI parcelport.

Additionally, we have continued working on our documentation. The main focus here was on completing the API documentation of the most important API functions. We have started adding migration guides for people interested in moving their codes away from other, commonplace parallelization frameworks like OpenMP.

Breaking changes

None

Closed issues

- Issue #6155[1366] - hpxcxx and hpxrun.py do not work if HPX_WITH_TESTS=OFF
- Issue #6164[1367] - HPX_WITH_DATAPAR_BACKEND=EVE causes compile errors with C++17
- Issue #6175[1368] - Make sure all our parallel algorithms accept the predicates by value
- Issue #6194[1369] - tests.regressions.threads.threads_all_1422 failed at Perlmutter
- Issue #6198[1370] - set_intersection/set_difference fails when run with execution::par
- Issue #6214[1371] - Broken Links to the Documentation page in readme.rst
- Issue #6217[1372] - hpx::make_heap does not terminate when exPolicy is par (or par_unseq) and size of vector is 4

[1365] https://github.com/TheHPXProject/hpx/pull/5845
[1366] https://github.com/TheHPXProject/hpx/issues/6155
[1367] https://github.com/TheHPXProject/hpx/issues/6164
[1368] https://github.com/TheHPXProject/hpx/issues/6175
[1369] https://github.com/TheHPXProject/hpx/issues/6194
[1370] https://github.com/TheHPXProject/hpx/issues/6198
[1371] https://github.com/TheHPXProject/hpx/issues/6214
[1372] https://github.com/TheHPXProject/hpx/issues/6217

- Issue #6246[1373] - HPX fails to compile under cxx 20 (fresh system)
- Issue #6247[1374] - HPX 1.9.0 does not compile with GCC on Windows
- Issue #6282[1375] - The "attach-debugger" option is broken on the current master branch.

Closed pull requests

- PR #6219[1376] - Cleaning up #includes in hpx/ folder
- PR #6223[1377] - Move documentation from README.rst to index.rst files under libs directory
- PR #6229[1378] - Adding zero-copy support on the receiving end of the TCP and MPI parcel ports
- PR #6231[1379] - Remove deprecated email from release procedure
- PR #6235[1380] - Modernize more modules (levels 12-16)
- PR #6236[1381] - Attempt to resolve occasional shutdown hangs in distributed operation
- PR #6239[1382] - Fix Optimizing HPX applications page of Manual
- PR #6241[1383] - LCI parcelport: Refactor, add more variants, zero copy receives.
- PR #6242[1384] - updated deprecated headers
- PR #6243[1385] - Adding github action builders using VS2019
- PR #6248[1386] - Fix CUDA/HIP Jenkins pipelines
- PR #6250[1387] - Resolve gcc problems on Windows
- PR #6251[1388] - Attempting to fix problems in barrier causing hangs
- PR #6253[1389] - Modernize set_thread_name on Windows
- PR #6256[1390] - Fix nvcc/gcc-10 (Octo-Tiger) compilation issue
- PR #6257[1391] - Cmake Tests: Delete operator check for size_t arg
- PR #6258[1392] - Rewriting wait_some to circumvent data races causing hangs
- PR #6260[1393] - Add migration guide to manual

[1373] https://github.com/TheHPXProject/hpx/issues/6246
[1374] https://github.com/TheHPXProject/hpx/issues/6247
[1375] https://github.com/TheHPXProject/hpx/issues/6282
[1376] https://github.com/TheHPXProject/hpx/pull/6219
[1377] https://github.com/TheHPXProject/hpx/pull/6223
[1378] https://github.com/TheHPXProject/hpx/pull/6229
[1379] https://github.com/TheHPXProject/hpx/pull/6231
[1380] https://github.com/TheHPXProject/hpx/pull/6235
[1381] https://github.com/TheHPXProject/hpx/pull/6236
[1382] https://github.com/TheHPXProject/hpx/pull/6239
[1383] https://github.com/TheHPXProject/hpx/pull/6241
[1384] https://github.com/TheHPXProject/hpx/pull/6242
[1385] https://github.com/TheHPXProject/hpx/pull/6243
[1386] https://github.com/TheHPXProject/hpx/pull/6248
[1387] https://github.com/TheHPXProject/hpx/pull/6250
[1388] https://github.com/TheHPXProject/hpx/pull/6251
[1389] https://github.com/TheHPXProject/hpx/pull/6253
[1390] https://github.com/TheHPXProject/hpx/pull/6256
[1391] https://github.com/TheHPXProject/hpx/pull/6257
[1392] https://github.com/TheHPXProject/hpx/pull/6258
[1393] https://github.com/TheHPXProject/hpx/pull/6260

- PR #6262[1394] - Fixing wrong command line options in local command line handling
- PR #6266[1395] - Attempt to resolve occasional hang in run_loop
- PR #6267[1396] - Attempting to fix migration tests
- PR #6278[1397] - Making sure the future's shared state doesn't go out of scope prematurely
- PR #6279[1398] - Re-expose error names
- PR #6281[1399] - Creating directory for file copy
- PR #6283[1400] - Consistently #include unistd.h for _POSIX_VERSION

HPX V1.9.0 (May 2, 2023)

General changes

- Added RISC-V 64bit support. HPX is now compatible with RISC-V architectures which have revolutionized the HPC world.
- LCI parcelport has been optimized to transfer parcels with fewer messages and use the HPX resource partitioner for its progress thread allocation. It should generally provide better performance than before. It also removes its dependency on the MPI library.
- HPX dependency on Boost was further relaxed by replacing headers from Boost.Range, Boost.Tokenizer and Boost.Lockfree.
- Improvements took place on our parallel algorithms implementation.
- Our Senders/Receivers (P2300) integration was extended:

– Coroutines were integrated with senders/receivers.

`get_completion_signatures` now works with awaitable senders. - `with_awaitable_senders` allows the passed senders to retrieve the value i.e. senders are transparently awaitable from within a coroutine. - `when_all_vector` was added.

- `sync_wait` and `sync_wait_with_variant` sender consumers were added. The user can now initiate the execution of their asynchronous pipeline by blocking the current thread that executes the main() function until the result is retrieved.
- The combinators for futures (a.k.a. async_combinators) `when_*`, `wait_*`, `wait_*_nothrow` were turned into CPOs allowing for end-user customization. For more information on the async_combinators refer to the documentation, https://hpx-docs.stellar-group.org/latest/html/libs/core/async_combinators/docs/index.html?highlight=combinators.
- The new datapar backend SVE allows simd and par_simd execution policies to exploit dataparalleism in the processors that have SVE vector registers like A64FX and Neoverse V1.
- The documentation for parallel algorithms, container algorithms was further improved. The Public API page was vastly enriched.

1394 https://github.com/TheHPXProject/hpx/pull/6262
1395 https://github.com/TheHPXProject/hpx/pull/6266
1396 https://github.com/TheHPXProject/hpx/pull/6267
1397 https://github.com/TheHPXProject/hpx/pull/6278
1398 https://github.com/TheHPXProject/hpx/pull/6279
1399 https://github.com/TheHPXProject/hpx/pull/6281
1400 https://github.com/TheHPXProject/hpx/pull/6283

- Copy button shortkey was added at the top-right of code-blocks.
- Pragma directive that reports warnings as errors on MSVC was fixed.
- Command line argument `--hpx:loopback_network` was added to facilitate debugging with networks.
- We added an HPX-SYCL integration, allowing users to obtain HPX futures for SYCL events. This effectively enables the integration of arbitrary asynchronous SYCL operations into the HPX task graph. Bolted on top of this integration, we further added an HPX-SYCL executor for ease of use.

Breaking changes

- Stopped supporting Clang V8, the minimal version supported is now Clang V10.
- Stopped supporting gcc V8, the minimal version supported is now gcc V9.
- Stopped supporting Visual Studio 2015, the minimal version supported is now Visual Studio 2019.
- `tag_policy_tag` et.al. were re-added after HPX V1.8.1 depracation.
- `get_chunk_size` and `processing_units_count` API is now expecting the time for one iteration as an argument.
- The list of all the namespace changes can be found here: *HPX V1.9.0 Namespace changes*.

Closed issues

- Issue #6203[1401] - Compilation error with *-mcpu=a64fx* on Ookami
- Issue #6196[1402] - Incorrect log destination
- Issue #6191[1403] - installing HPX
- Issue #6184[1404] - Wrong processing_units_count of restricted_thread_pool_executor
- Issue #6171[1405] - Release Tag Name Request
- Issue #6162[1406] - Current master does not compile on ROSTAM
- Issue #6156[1407] - hpxcxx does not work if HPX_WITH_PKGCONFIG=OFF
- Issue #6108[1408] - cxx17_aligned_new.cpp on msvc fails due to wrong pragma directive
- Issue #6045[1409] - Can't call nullary callables wrapped with *hpx::unwrapping*
- Issue #6013[1410] - Unable to build subprojects hpx_collectives/hpx_compute with MSVC
- Issue #6008[1411] - Missing *constexpr* default constructor for *hpx::mutex*

[1401] https://github.com/TheHPXProject/hpx/issues/6203
[1402] https://github.com/TheHPXProject/hpx/issues/6196
[1403] https://github.com/TheHPXProject/hpx/issues/6191
[1404] https://github.com/TheHPXProject/hpx/issues/6184
[1405] https://github.com/TheHPXProject/hpx/issues/6171
[1406] https://github.com/TheHPXProject/hpx/issues/6162
[1407] https://github.com/TheHPXProject/hpx/issues/6156
[1408] https://github.com/TheHPXProject/hpx/issues/6108
[1409] https://github.com/TheHPXProject/hpx/issues/6045
[1410] https://github.com/TheHPXProject/hpx/issues/6013
[1411] https://github.com/TheHPXProject/hpx/issues/6008

- Issue #5999[1412] - Add HPX Conda package to conda-forge
- Issue #5998[1413] - Serializing multiple arguments when applying distributed action results in segfault
- Issue #5958[1414] - HPX 1.8.0 and Blaze issues
- Issue #5908[1415] - Windows: duplicated symbols in static builds
- Issue #5802[1416] - Lost status is_ready from future
- Issue #5767[1417] - Performance drop on Piz Daint
- Issue #5752[1418] - Implement stride_view from P1899 (experimental)
- Issue #5744[1419] - HPX_WITH_FETCH_ASIO not working on Ookami
- Issue #5561[1420] - Possible race condition in helper thread / hpx::cout

Closed pull requests

- PR #6228[1421] - Fixing algorithms for zero length sequences when run with s/r scheduler
- PR #6227[1422] - Reliably disable background work when no networking is enabled
- PR #6225[1423] - Make heap fails in par for small sized heaps #6217
- PR #6222[1424] - Add documentation for *hpx::post*
- PR #6221[1425] - Fix segmented algorithms tests
- PR #6218[1426] - Creating INSTALL component 'runtime' to enable installing binaries only
- PR #6216[1427] - added tests for set_difference, updated set_operation.hpp to fix #6198
- PR #6213[1428] - Modernize and streamline MPI parcelport
- PR #6211[1429] - Modernize modules of level 11, 12, and 13
- PR #6210[1430] - Fixing MPI parcelport initialization if MPI is initialized outside of HPX
- PR #6209[1431] - Prevent thread stealing during scheduler shutdown
- PR #6208[1432] - Fix the compilation warning in the MPI parcelport with gcc 11.2

[1412] https://github.com/TheHPXProject/hpx/issues/5999
[1413] https://github.com/TheHPXProject/hpx/issues/5998
[1414] https://github.com/TheHPXProject/hpx/issues/5958
[1415] https://github.com/TheHPXProject/hpx/issues/5908
[1416] https://github.com/TheHPXProject/hpx/issues/5802
[1417] https://github.com/TheHPXProject/hpx/issues/5767
[1418] https://github.com/TheHPXProject/hpx/issues/5752
[1419] https://github.com/TheHPXProject/hpx/issues/5744
[1420] https://github.com/TheHPXProject/hpx/issues/5561
[1421] https://github.com/TheHPXProject/hpx/pull/6228
[1422] https://github.com/TheHPXProject/hpx/pull/6227
[1423] https://github.com/TheHPXProject/hpx/pull/6225
[1424] https://github.com/TheHPXProject/hpx/pull/6222
[1425] https://github.com/TheHPXProject/hpx/pull/6221
[1426] https://github.com/TheHPXProject/hpx/pull/6218
[1427] https://github.com/TheHPXProject/hpx/pull/6216
[1428] https://github.com/TheHPXProject/hpx/pull/6213
[1429] https://github.com/TheHPXProject/hpx/pull/6211
[1430] https://github.com/TheHPXProject/hpx/pull/6210
[1431] https://github.com/TheHPXProject/hpx/pull/6209
[1432] https://github.com/TheHPXProject/hpx/pull/6208

- PR #6207[1433] - Automatically enable Boost.Context when compiling for arm64.
- PR #6206[1434] - Update CMakeLists.txt
- PR #6205[1435] - Do not generate hpxcxx if support for pkgconfig was disabled
- PR #6204[1436] - Use **LRT_** instead of **LAPP_** logging in barrier implementation
- PR #6202[1437] - Fixing Fedora build errors on Power systems
- PR #6201[1438] - Update the LCI parcelport documents
- PR #6200[1439] - Par link jobs
- PR #6197[1440] - LCI parcelport: add doc, upgrade to v1.7.4, refactor cmake autofetch.
- PR #6195[1441] - Change the default tag of autofetch LCI to v1.7.3.
- PR #6192[1442] - Fix page *Writing single-node applications*
- PR #6189[1443] - Making sure restricted_thread_pool_executor properly reports used number of cores
- PR #6187[1444] - Enable using for_loop with range generators
- PR #6186[1445] - thread_support/CMakeLists: Fix build issue
- PR #6185[1446] - Fix EVE datapar with cxx_standard less than 20
- PR #6183[1447] - Update CI integration for EVE
- PR #6182[1448] - Fixing performance regressions
- PR #6181[1449] - LCI parcelport: backlog queue, aggregation, separate devices, and more
- PR #6180[1450] - Fixing use of for_loop with rebound execution policy (using *.with()*)
- PR #6179[1451] - Taking predicates for algorithms by value
- PR #6178[1452] - Changes needed to make chapel_hpx examples work
- PR #6176[1453] - Fixing warnings that were generated by PVS Studio
- PR #6174[1454] - Replace boost::integer::gcd with std::gcd
- PR #6172[1455] - [Docs] Fix example of how to run single/specific test(s)

[1433] https://github.com/TheHPXProject/hpx/pull/6207
[1434] https://github.com/TheHPXProject/hpx/pull/6206
[1435] https://github.com/TheHPXProject/hpx/pull/6205
[1436] https://github.com/TheHPXProject/hpx/pull/6204
[1437] https://github.com/TheHPXProject/hpx/pull/6202
[1438] https://github.com/TheHPXProject/hpx/pull/6201
[1439] https://github.com/TheHPXProject/hpx/pull/6200
[1440] https://github.com/TheHPXProject/hpx/pull/6197
[1441] https://github.com/TheHPXProject/hpx/pull/6195
[1442] https://github.com/TheHPXProject/hpx/pull/6192
[1443] https://github.com/TheHPXProject/hpx/pull/6189
[1444] https://github.com/TheHPXProject/hpx/pull/6187
[1445] https://github.com/TheHPXProject/hpx/pull/6186
[1446] https://github.com/TheHPXProject/hpx/pull/6185
[1447] https://github.com/TheHPXProject/hpx/pull/6183
[1448] https://github.com/TheHPXProject/hpx/pull/6182
[1449] https://github.com/TheHPXProject/hpx/pull/6181
[1450] https://github.com/TheHPXProject/hpx/pull/6180
[1451] https://github.com/TheHPXProject/hpx/pull/6179
[1452] https://github.com/TheHPXProject/hpx/pull/6178
[1453] https://github.com/TheHPXProject/hpx/pull/6176
[1454] https://github.com/TheHPXProject/hpx/pull/6174
[1455] https://github.com/TheHPXProject/hpx/pull/6172

- PR #6170[1456] - Adding missing fallback for processing_units_count customization point
- PR #6169[1457] - LCI parcelport: bypass the parcel queue and connection cache.
- PR #6167[1458] - Add create_local_communicator API function
- PR #6166[1459] - Add missing header for std::intmax_t
- PR #6165[1460] - Attempt to work around MSVC problem
- PR #6161[1461] - Update EVE integration
- PR #6160[1462] - More cleanup for module levels 0 to 10
- PR #6159[1463] - Fix minor spelling mistake in generate_issue_pr_list.sh
- PR #6158[1464] - Update documentation in *writing single-node applications* page
- PR #6157[1465] - Improve index_queue_spawning
- PR #6154[1466] - Avoid performing late command line handling twice in distributed runtime
- PR #6152[1467] - The -rd and -mr options didn't work, and they should have been –rd and –mr
- PR #6151[1468] - Refactoring the Manual page in documentation
- PR #6148[1469] - Investigate the failure of the LCI parcelport.
- PR #6147[1470] - Make posix co-routine stacks non-executable
- PR #6146[1471] - Avoid ambiguities wrt tag_invoke
- PR #6144[1472] - General improvements to scheduling and related fixes
- PR #6143[1473] - Add list of new namespaces for new release
- PR #6140[1474] - Fixing background scheduler to properly exit in the end
- PR #6139[1475] - [P2300] execution: Cleanup coroutines integration and improve ADL isolation
- PR #6137[1476] - Adding example of a simple master/slave distributed application

[1456] https://github.com/TheHPXProject/hpx/pull/6170
[1457] https://github.com/TheHPXProject/hpx/pull/6169
[1458] https://github.com/TheHPXProject/hpx/pull/6167
[1459] https://github.com/TheHPXProject/hpx/pull/6166
[1460] https://github.com/TheHPXProject/hpx/pull/6165
[1461] https://github.com/TheHPXProject/hpx/pull/6161
[1462] https://github.com/TheHPXProject/hpx/pull/6160
[1463] https://github.com/TheHPXProject/hpx/pull/6159
[1464] https://github.com/TheHPXProject/hpx/pull/6158
[1465] https://github.com/TheHPXProject/hpx/pull/6157
[1466] https://github.com/TheHPXProject/hpx/pull/6154
[1467] https://github.com/TheHPXProject/hpx/pull/6152
[1468] https://github.com/TheHPXProject/hpx/pull/6151
[1469] https://github.com/TheHPXProject/hpx/pull/6148
[1470] https://github.com/TheHPXProject/hpx/pull/6147
[1471] https://github.com/TheHPXProject/hpx/pull/6146
[1472] https://github.com/TheHPXProject/hpx/pull/6144
[1473] https://github.com/TheHPXProject/hpx/pull/6143
[1474] https://github.com/TheHPXProject/hpx/pull/6140
[1475] https://github.com/TheHPXProject/hpx/pull/6139
[1476] https://github.com/TheHPXProject/hpx/pull/6137

- PR #6136[1477] - Deprecate *execution::experimental::task_group* in favor of *experimental::task_group*
- PR #6135[1478] - Fixing warnings reported by MSVC analysis
- PR #6134[1479] - Adding notification function for parcelports to be called after early parcel handling
- PR #6132[1480] - Fixing to_non_par() for parallel simd policies
- PR #6131[1481] - modernize modules from level 25
- PR #6130[1482] - Remove the mutex lock in the critical path of get_partitioner.
- PR #6129[1483] - Modernize module from levels 22, 23
- PR #6127[1484] - Working around gccV9 problem that prevent us from storing enum classes in bit fields
- PR #6126[1485] - Deprecate hpx::parallel::task_block in favor of hpx::experimental::task_block
- PR #6125[1486] - Making sure sync_wait compiles when used with an lvalue sender involving bulk
- PR #6124[1487] - Fixing use of any_sender in combination with when_all
- PR #6123[1488] - Fixed issues found by PVS-Studio
- PR #6121[1489] - Modernize modules of level 21, 22
- PR #6120[1490] - Use index_queue for parallel executors bulk_async_execute
- PR #6119[1491] - Update CMakeLists.txt
- PR #6118[1492] - Modernize modules from level 17, 18, 19, and 20
- PR #6117[1493] - Initialize **buffer_allocate_time_** to 0
- PR #6116[1494] - Add new command line argument –hpx:loopback_network
- PR #6115[1495] - Modernize modules of levels 14, 15, and 16
- PR #6114[1496] - Enhance the formatting of the documentation
- PR #6113[1497] - Modernize modules in module level 11, 12, and 13

[1477] https://github.com/TheHPXProject/hpx/pull/6136
[1478] https://github.com/TheHPXProject/hpx/pull/6135
[1479] https://github.com/TheHPXProject/hpx/pull/6134
[1480] https://github.com/TheHPXProject/hpx/pull/6132
[1481] https://github.com/TheHPXProject/hpx/pull/6131
[1482] https://github.com/TheHPXProject/hpx/pull/6130
[1483] https://github.com/TheHPXProject/hpx/pull/6129
[1484] https://github.com/TheHPXProject/hpx/pull/6127
[1485] https://github.com/TheHPXProject/hpx/pull/6126
[1486] https://github.com/TheHPXProject/hpx/pull/6125
[1487] https://github.com/TheHPXProject/hpx/pull/6124
[1488] https://github.com/TheHPXProject/hpx/pull/6123
[1489] https://github.com/TheHPXProject/hpx/pull/6121
[1490] https://github.com/TheHPXProject/hpx/pull/6120
[1491] https://github.com/TheHPXProject/hpx/pull/6119
[1492] https://github.com/TheHPXProject/hpx/pull/6118
[1493] https://github.com/TheHPXProject/hpx/pull/6117
[1494] https://github.com/TheHPXProject/hpx/pull/6116
[1495] https://github.com/TheHPXProject/hpx/pull/6115
[1496] https://github.com/TheHPXProject/hpx/pull/6114
[1497] https://github.com/TheHPXProject/hpx/pull/6113

- PR #6112[1498] - Modernize modules from levels 9 and 10
- PR #6111[1499] - Modernize all modules from module level 8
- PR #6110[1500] - Use pragma error directive to report warnings as errors on msvc
- PR #6109[1501] - Modernize serialization module
- PR #6107[1502] - Modernize error module
- PR #6106[1503] - Modernizing modules of levels 0 to 5
- PR #6105[1504] - Optimizations on LCI parcelport: merge small messages; remove sender mutex lock.
- PR #6104[1505] - Adding parameters API: measure_iteration
- PR #6103[1506] - Document *task_group* and include in Public API
- PR #6102[1507] - Prevent warnings generated by clang-cl
- PR #6101[1508] - Using more fold expressions
- PR #6100[1509] - Deprecate *hpx::parallel::reduce_by_key* in favor of *hpx::experimental::reduce_by_key*
- PR #6098[1510] - Forking Boost.Lockfree
- PR #6096[1511] - Forking Boost.Tokenizer
- PR #6095[1512] - Replacing facilities from Boost.Range
- PR #6094[1513] - Removing object_semaphore
- PR #6093[1514] - Replace boost::string_ref with std::string_view
- PR #6092[1515] - Use C++17 static_assert where possible
- PR #6091[1516] - Replace artificial sequencing with fold expressions
- PR #6090[1517] - Fixing use of get_chunk_size customization point
- PR #6088[1518] - Add/fix Public API documentation
- PR #6086[1519] - Deprecate *hpx::util::unlock_guard* in favor of *hpx::unlock_guard*

[1498] https://github.com/TheHPXProject/hpx/pull/6112
[1499] https://github.com/TheHPXProject/hpx/pull/6111
[1500] https://github.com/TheHPXProject/hpx/pull/6110
[1501] https://github.com/TheHPXProject/hpx/pull/6109
[1502] https://github.com/TheHPXProject/hpx/pull/6107
[1503] https://github.com/TheHPXProject/hpx/pull/6106
[1504] https://github.com/TheHPXProject/hpx/pull/6105
[1505] https://github.com/TheHPXProject/hpx/pull/6104
[1506] https://github.com/TheHPXProject/hpx/pull/6103
[1507] https://github.com/TheHPXProject/hpx/pull/6102
[1508] https://github.com/TheHPXProject/hpx/pull/6101
[1509] https://github.com/TheHPXProject/hpx/pull/6100
[1510] https://github.com/TheHPXProject/hpx/pull/6098
[1511] https://github.com/TheHPXProject/hpx/pull/6096
[1512] https://github.com/TheHPXProject/hpx/pull/6095
[1513] https://github.com/TheHPXProject/hpx/pull/6094
[1514] https://github.com/TheHPXProject/hpx/pull/6093
[1515] https://github.com/TheHPXProject/hpx/pull/6092
[1516] https://github.com/TheHPXProject/hpx/pull/6091
[1517] https://github.com/TheHPXProject/hpx/pull/6090
[1518] https://github.com/TheHPXProject/hpx/pull/6088
[1519] https://github.com/TheHPXProject/hpx/pull/6086

- PR #6085[1520] - Add experimental sycl integration/executor
- PR #6084[1521] - Renaming hpx::apply and friends to hpx::post
- PR #6083[1522] - Using if constexpr instead of tag-dispatching, where possible
- PR #6082[1523] - Replace util::always_void_t with std::void_t
- PR #6081[1524] - Update github actions to avoid warnings
- PR #6080[1525] - Disable some tests that fail on LCI
- PR #6079[1526] - Adding more natvis files, correct existing
- PR #6078[1527] - Changing target name of memory_counters component
- PR #6077[1528] - Making default constructor of hpx::mutex constexpr
- PR #6076[1529] - Cleaning up functionality that was deprecated in V1.7
- PR #6075[1530] - Remove conditional code for gcc V7 and below
- PR #6074[1531] - Fixing compilation issues on gcc V8
- PR #6073[1532] - Fixing PAPI counter component compilation
- PR #6072[1533] - Adding ex::when_all_vector
- PR #6071[1534] - Making get_forward_progress_guarantee_t specializations constexpr
- PR #6070[1535] - Implement P2690 for our algorithms
- PR #6069[1536] - Do not check for cancellation during each iteration but only once per partition
- PR #6068[1537] - Prevent using task and non_task as a CPO
- PR #6067[1538] - Deprecated hpx::util::mem_fn in favor of hpx::mem_fn
- PR #6066[1539] - Create codeql.yml
- PR #6064[1540] - Adapting adjacent_difference for S/R execution
- PR #6063[1541] - Modernize iterator_support module
- PR #6062[1542] - Make sure wrapping executor does not go out of scope prematurely

[1520] https://github.com/TheHPXProject/hpx/pull/6085
[1521] https://github.com/TheHPXProject/hpx/pull/6084
[1522] https://github.com/TheHPXProject/hpx/pull/6083
[1523] https://github.com/TheHPXProject/hpx/pull/6082
[1524] https://github.com/TheHPXProject/hpx/pull/6081
[1525] https://github.com/TheHPXProject/hpx/pull/6080
[1526] https://github.com/TheHPXProject/hpx/pull/6079
[1527] https://github.com/TheHPXProject/hpx/pull/6078
[1528] https://github.com/TheHPXProject/hpx/pull/6077
[1529] https://github.com/TheHPXProject/hpx/pull/6076
[1530] https://github.com/TheHPXProject/hpx/pull/6075
[1531] https://github.com/TheHPXProject/hpx/pull/6074
[1532] https://github.com/TheHPXProject/hpx/pull/6073
[1533] https://github.com/TheHPXProject/hpx/pull/6072
[1534] https://github.com/TheHPXProject/hpx/pull/6071
[1535] https://github.com/TheHPXProject/hpx/pull/6070
[1536] https://github.com/TheHPXProject/hpx/pull/6069
[1537] https://github.com/TheHPXProject/hpx/pull/6068
[1538] https://github.com/TheHPXProject/hpx/pull/6067
[1539] https://github.com/TheHPXProject/hpx/pull/6066
[1540] https://github.com/TheHPXProject/hpx/pull/6064
[1541] https://github.com/TheHPXProject/hpx/pull/6063
[1542] https://github.com/TheHPXProject/hpx/pull/6062

- PR #6061[1543] - Minor fix in small_vector (from upstream)
- PR #6060[1544] - Allow to disable registering signal handlers
- PR #6059[1545] - [P2300] Fix: declval cannot be ODR used
- PR #6058[1546] - Avoid ambiguity for hpx::get used with std::variant
- PR #6057[1547] - Create a dedicated thread pool to run LCI_progress.
- PR #6056[1548] - Fix coroutine test for clang
- PR #6055[1549] - Patches needed to be able to build HPX 1.8.1 on various platforms
- PR #6054[1550] - Use MSVC specific attribute [[msvc::no_unique_address]]
- PR #6052[1551] - Deprecated hpx::util::invoke_fused in favor of hpx::invoke_fused
- PR #6051[1552] - Add non-contiguous index queue and use it in thread_pool_bulk_scheduler
- PR #6049[1553] - Crosscompile arm sve
- PR #6048[1554] - Deprecated hpx::util::invoke in favor of hpx::invoke
- PR #6047[1555] - Separating binary_semaphore into its own file
- PR #6046[1556] - Support using unwrapping with nullary function objects
- PR #6044[1557] - Generalize the use of then() and dataflow
- PR #6043[1558] - Clean up scan_partitioner
- PR #6042[1559] - Modernize dataflow API
- PR #6041[1560] - docs: document semaphores
- PR #6040[1561] - Add/Fix documentation of Public API page
- PR #6039[1562] - remove MPI dependency when only using LCI parcelport
- PR #6038[1563] - Clean up command line handling
- PR #6037[1564] - Avoid performing parcel related background work if networking is disabled

1543 https://github.com/TheHPXProject/hpx/pull/6061
1544 https://github.com/TheHPXProject/hpx/pull/6060
1545 https://github.com/TheHPXProject/hpx/pull/6059
1546 https://github.com/TheHPXProject/hpx/pull/6058
1547 https://github.com/TheHPXProject/hpx/pull/6057
1548 https://github.com/TheHPXProject/hpx/pull/6056
1549 https://github.com/TheHPXProject/hpx/pull/6055
1550 https://github.com/TheHPXProject/hpx/pull/6054
1551 https://github.com/TheHPXProject/hpx/pull/6052
1552 https://github.com/TheHPXProject/hpx/pull/6051
1553 https://github.com/TheHPXProject/hpx/pull/6049
1554 https://github.com/TheHPXProject/hpx/pull/6048
1555 https://github.com/TheHPXProject/hpx/pull/6047
1556 https://github.com/TheHPXProject/hpx/pull/6046
1557 https://github.com/TheHPXProject/hpx/pull/6044
1558 https://github.com/TheHPXProject/hpx/pull/6043
1559 https://github.com/TheHPXProject/hpx/pull/6042
1560 https://github.com/TheHPXProject/hpx/pull/6041
1561 https://github.com/TheHPXProject/hpx/pull/6040
1562 https://github.com/TheHPXProject/hpx/pull/6039
1563 https://github.com/TheHPXProject/hpx/pull/6038
1564 https://github.com/TheHPXProject/hpx/pull/6037

- PR #6036[1565] - Support new datapar backend : SVE
- PR #6035[1566] - Simplify datapar replace copy if
- PR #6034[1567] - Add/Fix documentation of Public API
- PR #6033[1568] - Support for data-parallelism for replace, replace_if, replace_copy, replace_copy_if algorithms
- PR #6032[1569] - Add documentation in public API
- PR #6031[1570] - Expose available cache sizes from topology object
- PR #6030[1571] - Adding parcelport initialization hook for resource partitioner operation
- PR #6029[1572] - Simplify startup code
- PR #6027[1573] - Add/Fix documentation in Public API page
- PR #6026[1574] - add option hpx:force_ipv4 to force resolving hostnames to ipv4 addresses
- PR #6025[1575] - build(docs): remove leftover sections
- PR #6023[1576] - Minor fixes on "How to build on Windows"
- PR #6022[1577] - build(doxy): don't extract private members
- PR #6021[1578] - Adding pu_mask to thread_pool_bulk_scheduler
- PR #6020[1579] - docs: add cppref NamedRequirements support
- PR #6018[1580] - Unseq adaptation for for_each, transform, reduce, transform_reduce, etc.
- PR #6017[1581] - loop and transform_loop unseq adaptation
- PR #6016[1582] - Config and structural updates to support unseq implementation
- PR #6015[1583] - Integrating sync_wait & sync_wait_with_variant
- PR #6012[1584] - docs: add missing links to public api
- PR #6009[1585] - Fixing sender&receiver integration with for_each and for_loop
- PR #6007[1586] - docs: add docs for mutex.hpp
- PR #6006[1587] - Relax future::is_ready where possible

[1565] https://github.com/TheHPXProject/hpx/pull/6036
[1566] https://github.com/TheHPXProject/hpx/pull/6035
[1567] https://github.com/TheHPXProject/hpx/pull/6034
[1568] https://github.com/TheHPXProject/hpx/pull/6033
[1569] https://github.com/TheHPXProject/hpx/pull/6032
[1570] https://github.com/TheHPXProject/hpx/pull/6031
[1571] https://github.com/TheHPXProject/hpx/pull/6030
[1572] https://github.com/TheHPXProject/hpx/pull/6029
[1573] https://github.com/TheHPXProject/hpx/pull/6027
[1574] https://github.com/TheHPXProject/hpx/pull/6026
[1575] https://github.com/TheHPXProject/hpx/pull/6025
[1576] https://github.com/TheHPXProject/hpx/pull/6023
[1577] https://github.com/TheHPXProject/hpx/pull/6022
[1578] https://github.com/TheHPXProject/hpx/pull/6021
[1579] https://github.com/TheHPXProject/hpx/pull/6020
[1580] https://github.com/TheHPXProject/hpx/pull/6018
[1581] https://github.com/TheHPXProject/hpx/pull/6017
[1582] https://github.com/TheHPXProject/hpx/pull/6016
[1583] https://github.com/TheHPXProject/hpx/pull/6015
[1584] https://github.com/TheHPXProject/hpx/pull/6012
[1585] https://github.com/TheHPXProject/hpx/pull/6009
[1586] https://github.com/TheHPXProject/hpx/pull/6007
[1587] https://github.com/TheHPXProject/hpx/pull/6006

- PR #6005[1588] - reshuffle header tests to different instances
- PR #6004[1589] - Add documentation Public API
- PR #6003[1590] - Always exporting get_component_name implementations
- PR #6002[1591] - Making sure that default constructble arguments are properly constructed during deserialization
- PR #5996[1592] - Add back explicit template parameters to lock_guards for nvcc
- PR #5994[1593] - Fix CTRL+C on windows
- PR #5993[1594] - Using EVE requires C++20
- PR #5992[1595] - This properly terminates an application on Ctrl-C on Windows
- PR #5991[1596] - Support IPV6 on command line for explicit network initialization
- PR #5990[1597] - P2300 enhancements
- PR #5989[1598] - Fix missing documentation in Public API page
- PR #5987[1599] - Attempting to fix timed executor API
- PR #5986[1600] - Fix warnings when building docs
- PR #5985[1601] - Re-add deprecated tag_policy_tag et.al. types that were removed in V1.8.1
- PR #5981[1602] - docs: add docs for condition_variable.hpp
- PR #5980[1603] - More work on execution::read
- PR #5979[1604] - Remove support for clang-v8 and clang-v9, switch LSU clang-v13 to C++17
- PR #5977[1605] - fix: Compilation errors for -std=c++17 builders
- PR #5975[1606] - docs: fix & improve parallel algorithms documentation 5
- PR #5974[1607] - [P2300] Adapt get completion signatures for awaitable senders
- PR #5973[1608] - defaults boost.context on riscv64
- PR #5972[1609] - Fix documentation for container algorithms

[1588] https://github.com/TheHPXProject/hpx/pull/6005
[1589] https://github.com/TheHPXProject/hpx/pull/6004
[1590] https://github.com/TheHPXProject/hpx/pull/6003
[1591] https://github.com/TheHPXProject/hpx/pull/6002
[1592] https://github.com/TheHPXProject/hpx/pull/5996
[1593] https://github.com/TheHPXProject/hpx/pull/5994
[1594] https://github.com/TheHPXProject/hpx/pull/5993
[1595] https://github.com/TheHPXProject/hpx/pull/5992
[1596] https://github.com/TheHPXProject/hpx/pull/5991
[1597] https://github.com/TheHPXProject/hpx/pull/5990
[1598] https://github.com/TheHPXProject/hpx/pull/5989
[1599] https://github.com/TheHPXProject/hpx/pull/5987
[1600] https://github.com/TheHPXProject/hpx/pull/5986
[1601] https://github.com/TheHPXProject/hpx/pull/5985
[1602] https://github.com/TheHPXProject/hpx/pull/5981
[1603] https://github.com/TheHPXProject/hpx/pull/5980
[1604] https://github.com/TheHPXProject/hpx/pull/5979
[1605] https://github.com/TheHPXProject/hpx/pull/5977
[1606] https://github.com/TheHPXProject/hpx/pull/5975
[1607] https://github.com/TheHPXProject/hpx/pull/5974
[1608] https://github.com/TheHPXProject/hpx/pull/5973
[1609] https://github.com/TheHPXProject/hpx/pull/5972

- PR #5971[1610] - added logic to detect riscv compiler configured for 64 bit target
- PR #5968[1611] - adds risc-v 64 bit support
- PR #5967[1612] - Adding missing pieces to sync_wait, adding run_loop
- PR #5966[1613] - docs: fix & improve parallel algorithms documentation 4
- PR #5965[1614] - Fixing inspect problems, adding missing header file
- PR #5962[1615] - Changes in html page of documentation
- PR #5961[1616] - Prevent stalling during shutdown when running hello_world_distributed
- PR #5955[1617] - Fix documentation for container algorithms
- PR #5952[1618] - docs: fix & improve parallel algorithms documentation 3
- PR #5950[1619] - Change executors to directly implement the executor CPOs
- PR #5949[1620] - Converting async combinators into CPOs
- PR #5948[1621] - Adding support for pure sender/receiver based executors to parallel algorithms
- PR #5945[1622] - [P2300] Added fundamental coroutine_traits for S/R
- PR #5883[1623] - Optimization on LCI parcelport: uses LCI_putva
- PR #5872[1624] - Block fork join executor
- PR #5855[1625] - Adding performance test Jenkins builder at LSU

HPX V1.8.1 (Aug 5, 2022)

This is a bugfix release with a few minor additions and resolved problems.

[1610] https://github.com/TheHPXProject/hpx/pull/5971
[1611] https://github.com/TheHPXProject/hpx/pull/5968
[1612] https://github.com/TheHPXProject/hpx/pull/5967
[1613] https://github.com/TheHPXProject/hpx/pull/5966
[1614] https://github.com/TheHPXProject/hpx/pull/5965
[1615] https://github.com/TheHPXProject/hpx/pull/5962
[1616] https://github.com/TheHPXProject/hpx/pull/5961
[1617] https://github.com/TheHPXProject/hpx/pull/5955
[1618] https://github.com/TheHPXProject/hpx/pull/5952
[1619] https://github.com/TheHPXProject/hpx/pull/5950
[1620] https://github.com/TheHPXProject/hpx/pull/5949
[1621] https://github.com/TheHPXProject/hpx/pull/5948
[1622] https://github.com/TheHPXProject/hpx/pull/5945
[1623] https://github.com/TheHPXProject/hpx/pull/5883
[1624] https://github.com/TheHPXProject/hpx/pull/5872
[1625] https://github.com/TheHPXProject/hpx/pull/5855

General changes

This patch release adds a number of small new features and fixes a handful of problems discovered since the last release, in particular:

- A lot of work has been done to improve vectorization support for our parallel algorithms. HPX now supports using EVE - the Expressive Vector Engine as a vectorization backend.
- Added a simple average power consumption performance counter.
- Added performance counters related to the use of zero-copy chunks in the networking layer.
- More work was done towards full compatibility with the sender/receivers proposal P2300.
- Fixing sync_wait to decay the result types
- Fixed collective operations to properly avoid overalapping consecutive operations on the same communicator.
- Simplified the implementation of our execution policies and added mapping functions between those.
- Fixed performance issues with our implementation of *small_vector*.
- Serialization now works with buffers of unsigned characters.
- Fixing dangling reference in serialization of non-default constructible types
- Fixed static linking on Windows.
- Fixed support for M1/MacOS based architectures.
- Fixed support for gentoo/musl.
- Fixed *hpx::counting_semaphore_var*.
- Properly check start and end bounds for *hpx::for_loop*
- A lot of changes and fixes to the documentation (see https://hpx-docs.stellar-group.org).

Breaking changes

- No breaking changes have been introduced.

Closed issues

- Issue #5964[1626] - component with multiple inheritance
- Issue #5946[1627] - dll_dlopen.hpp: error: RTLD_DI_ORIGIN was not declared in this scope with musl libc
- Issue #5925[1628] - Simplify implementation of execution policies
- Issue #5924[1629] - {what}: mmap() failed to allocate thread stack: HPX(unhandled_exception)

[1626] https://github.com/TheHPXProject/hpx/issues/5964
[1627] https://github.com/TheHPXProject/hpx/issues/5946
[1628] https://github.com/TheHPXProject/hpx/issues/5925
[1629] https://github.com/TheHPXProject/hpx/issues/5924

- Issue #5912[1630] - collectives all gather hangs if rank 0 is not involved
- Issue #5902[1631] - MPI parcelport issue on Fugaku
- Issue #5900[1632] - Unable to build hello_world_distributed.cpp.
- Issue #5892[1633] - Problems with HPX serialization as a standalone feature. Testcase provided.
- Issue #5886[1634] - Segfault when serializing non default constructible class with stl containers data members
- Issue #5832[1635] - Distributed execution crash
- Issue #5768[1636] - HPX hangs on Perlmutter
- Issue #5735[1637] - hpx::for_loop executes without checking start and end bounds
- Issue #5700[1638] - HPX(serialization_error)

Closed pull requests

- PR #5970[1639] - Fixing component multiple inheritance
- PR #5969[1640] - Fixing sync_wait to avoid dangling references
- PR #5963[1641] - Fixing sync_wait to decay the result types
- PR #5960[1642] - docs: added name to documentation contributors list
- PR #5959[1643] - Fixing sync_wait to decay the result types
- PR #5954[1644] - refactor: rename itr to correct type (*reduce*)
- PR #5954[1645] - refactor: rename itr to correct type (*reduce*)
- PR #5953[1646] - Fixed property handling in hierarchical_spawning
- PR #5951[1647] - Fixing static linking (for Windows)
- PR #5947[1648] - Fix building on musl.
- PR #5944[1649] - added adaptive_static_chunk_size
- PR #5943[1650] - Fix sync_wait

1630 https://github.com/TheHPXProject/hpx/issues/5912
1631 https://github.com/TheHPXProject/hpx/issues/5902
1632 https://github.com/TheHPXProject/hpx/issues/5900
1633 https://github.com/TheHPXProject/hpx/issues/5892
1634 https://github.com/TheHPXProject/hpx/issues/5886
1635 https://github.com/TheHPXProject/hpx/issues/5832
1636 https://github.com/TheHPXProject/hpx/issues/5768
1637 https://github.com/TheHPXProject/hpx/issues/5735
1638 https://github.com/TheHPXProject/hpx/issues/5700
1639 https://github.com/TheHPXProject/hpx/pull/5970
1640 https://github.com/TheHPXProject/hpx/pull/5969
1641 https://github.com/TheHPXProject/hpx/pull/5963
1642 https://github.com/TheHPXProject/hpx/pull/5960
1643 https://github.com/TheHPXProject/hpx/pull/5959
1644 https://github.com/TheHPXProject/hpx/pull/5954
1645 https://github.com/TheHPXProject/hpx/pull/5954
1646 https://github.com/TheHPXProject/hpx/pull/5953
1647 https://github.com/TheHPXProject/hpx/pull/5951
1648 https://github.com/TheHPXProject/hpx/pull/5947
1649 https://github.com/TheHPXProject/hpx/pull/5944
1650 https://github.com/TheHPXProject/hpx/pull/5943

- PR #5942[1651] - Fix doc warnings
- PR #5941[1652] - Fix sync_wait
- PR #5940[1653] - Protect collective operations against std::vector<bool> idiosyncrasies
- PR #5939[1654] - docs: fix & improve parallel algorithms documentation 2
- PR #5938[1655] - Properly implement generation support for collective operations
- PR #5937[1656] - Remove leftover files from PMR based small_vector
- PR #5936[1657] - Adding mapping functions between execution policies
- PR #5935[1658] - Fixing serialization to work with buffers of unsigned chars
- PR #5934[1659] - Attempting to fix datapar issues on CircleCI
- PR #5933[1660] - Fix documentation for ranges algorithms
- PR #5932[1661] - Remove mimalloc version constraint
- PR #5931[1662] - docs: fix & improve parallel algorithms documentation
- PR #5930[1663] - Add boost to hip builder
- PR #5929[1664] - Apply fixes to M1/MacOS related stack allocation to all relevant spots
- PR #5928[1665] - updated context_generic_context to accommodate arm64_arch_8/Apple architecture
- PR #5927[1666] - Public derivation for counting_semaphore_var
- PR #5926[1667] - Fix doxygen warnings when building documentation
- PR #5923[1668] - Fixing git checkout to reflect latest version tag
- PR #5922[1669] - A couple of unrelated changes in support of implementing P1673
- PR #5920[1670] - [P2300] enhancements: receiver_of, sender_of improvements
- PR #5917[1671] - Fixing various 'held lock while suspending' problems
- PR #5916[1672] - Fix minor doxygen parsing typo
- PR #5915[1673] - docs: fix broken api algo links

[1651] https://github.com/TheHPXProject/hpx/pull/5942
[1652] https://github.com/TheHPXProject/hpx/pull/5941
[1653] https://github.com/TheHPXProject/hpx/pull/5940
[1654] https://github.com/TheHPXProject/hpx/pull/5939
[1655] https://github.com/TheHPXProject/hpx/pull/5938
[1656] https://github.com/TheHPXProject/hpx/pull/5937
[1657] https://github.com/TheHPXProject/hpx/pull/5936
[1658] https://github.com/TheHPXProject/hpx/pull/5935
[1659] https://github.com/TheHPXProject/hpx/pull/5934
[1660] https://github.com/TheHPXProject/hpx/pull/5933
[1661] https://github.com/TheHPXProject/hpx/pull/5932
[1662] https://github.com/TheHPXProject/hpx/pull/5931
[1663] https://github.com/TheHPXProject/hpx/pull/5930
[1664] https://github.com/TheHPXProject/hpx/pull/5929
[1665] https://github.com/TheHPXProject/hpx/pull/5928
[1666] https://github.com/TheHPXProject/hpx/pull/5927
[1667] https://github.com/TheHPXProject/hpx/pull/5926
[1668] https://github.com/TheHPXProject/hpx/pull/5923
[1669] https://github.com/TheHPXProject/hpx/pull/5922
[1670] https://github.com/TheHPXProject/hpx/pull/5920
[1671] https://github.com/TheHPXProject/hpx/pull/5917
[1672] https://github.com/TheHPXProject/hpx/pull/5916
[1673] https://github.com/TheHPXProject/hpx/pull/5915

- PR #5914[1674] - Remove CSS rules - update sphinx version
- PR #5911[1675] - Removed references to hpx::vector in comments
- PR #5909[1676] - Remove stuff which is defined in the header
- PR #5906[1677] - Use BUILD_SHARED_LIBS correctly
- PR #5905[1678] - Fix incorrect usage of generator expressions
- PR #5904[1679] - Delete FindBZip2.cmake
- PR #5901[1680] - Fix #5900
- PR #5899[1681] - Replace PMR based version of small_vector
- PR #5897[1682] - Add missing “”
- PR #5896[1683] - Docs: Add serialization tutorial.
- PR #5895[1684] - Update to V1.9.0 on master
- PR #5894[1685] - Fix executor_with_thread_hooks example
- PR #5890[1686] - Adding simple average power consumption performance counter
- PR #5889[1687] - Par unseq/unseq adding
- PR #5888[1688] - Support for data-parallelism for reduce, transform reduce, transform_binary_reduce algorithms
- PR #5887[1689] - Fixing dangling reference in serialization of non-default constructible types
- PR #5879[1690] - New performance counters related to zero-copy chunks.

HPX V1.8.0 (May 18, 2022)

With HPX parallel algorithms been fully adapted to C++20 the new release achieves full conformance with C++20 concurrency and parallelism facilities. HPX now supports all of the algorithms as specified by C++20. We have added support for vectorization to more of our algorithms. Much work has been done towards implementing P2300 (“std::execution”) and implementing the underlying senders/receivers facilities. Finally, The new release comes with a brand new documentation interface!

1674 https://github.com/TheHPXProject/hpx/pull/5914
1675 https://github.com/TheHPXProject/hpx/pull/5911
1676 https://github.com/TheHPXProject/hpx/pull/5909
1677 https://github.com/TheHPXProject/hpx/pull/5906
1678 https://github.com/TheHPXProject/hpx/pull/5905
1679 https://github.com/TheHPXProject/hpx/pull/5904
1680 https://github.com/TheHPXProject/hpx/pull/5901
1681 https://github.com/TheHPXProject/hpx/pull/5899
1682 https://github.com/TheHPXProject/hpx/pull/5897
1683 https://github.com/TheHPXProject/hpx/pull/5896
1684 https://github.com/TheHPXProject/hpx/pull/5895
1685 https://github.com/TheHPXProject/hpx/pull/5894
1686 https://github.com/TheHPXProject/hpx/pull/5890
1687 https://github.com/TheHPXProject/hpx/pull/5889
1688 https://github.com/TheHPXProject/hpx/pull/5888
1689 https://github.com/TheHPXProject/hpx/pull/5887
1690 https://github.com/TheHPXProject/hpx/pull/5879

General changes

- The new documentation can now be found on our webpage: https://hpx-docs.stellar-group.org. This includes a completely new and user-friendly interface environment along with restructuring of certain components. The content in the "Quick start", "Manual" and "Examples" was improved, while the "Build system" page was adapted to include necessary information for newcommers.
- With the vectorization support available in modern hardware architectures HPX now provides new data-parallel vector execution policies `hpx::execution::simd` and `hpx::execution::par_simd` that enable significant speed-up of our parallel algorithm implementations. The following algorithms now support SIMD execution:
  - `copy`, `copy_n`
  - `generate`
  - `adjacent_difference`, `adjacent_find`
  - `all_of`, `any_of`, `none_of`
  - `equal`, `mismatch`,
  - `inner_product`
  - `count`, `count_if`
  - `fill`, `fill_n`
  - `find`, `find_end`, `find_first_of`, `find_if`, `find_if_not`
  - `for_each`, `for_each_n`
  - `generate`, `generate_n`.
- Based on top of P2300 the HPX parallel algorithms now support the pipeline syntax towards an effort to unify their usage along with senders/receivers. The HPX parallel algorithms can now bind with senders/receivers using the pipeline operator.
- Several changes took place on the executors provided by HPX:
- The executors now support the `num_cores` options in order for the user to be able to specify the desired number of cores to be used in the correspodning execution.
- The `scheduler` executor was implemented on top of senders/receivers and can be used with all HPX facilities that schedule new work, such as parallel algorithms, `hpx::async`, `hpx::dataflow`, etc.
- The performance of `fork_join_executor` was improved.
- The following algorithms have been added/adapted to be C++20 conformant:
  - `min_element`
  - `max_element`
  - `minmax_element`
  - `starts_with`
  - `ends_with`
  - `swap_ranges`
  - `unique`
  - `unique_copy`

- rotate
- rotate_copy
- sort
- shift_left
- shift_right
- stable_sort
- partition
- partition_copy
- stable_partition
- adjacent_difference
- nth_element
- partial_sort
- partial_sort_copy.

* HPX_FORWARD/HPX_MOVE macros were introduced that replaced the std::move and std::forward facilities that in the library code.
* Hangs on distributed barrier were fixed.
* The performance of scan_partitioner was improved.
* Support was added for thread_priority to the parallel_execution_policy
* Regarding senders/receivers and the P2300 proposal various actions took place. stop_token was adapted to the recent proposal version (in_place_stop_token was introduced). Also hint, annotation, priority and stacksize properties were added to the scheduler executor. Stop support was added to when_all. Support for completion signatures was added. The following schedulers and algorithms were added:

- get_completion_scheduler
- any_sender and unique_any_sender
- split sender
- transform_mpi sender
- transfer sender
- let_error, let_stopped
- get_env and related environment queries
- schedule, set_value, set_error, set_done, start and connect are now proper customization points as defined in P2300.

* Several namespaces were altered towards conformance with C++20. Compatibility layers have been added and the old versions will be removed in next releases. The namespace changes are the following:

- hpx::parallel::induction/reduction were movied into namespace hpx::experimental
- for_loop and friends were moved into namespace hpx::experimental.
- hpx::util::optional and friends were moved into namespace hpx.

- hpx::lcos::barrier has been moved into the hpx::distributed namespace and hpx::lcos::local::cpp20_barrier has been renamed to barrier and moved into the hpx namespace.
- hpx::lcos::latch has been moved into the hpx::distributed namespace and lcos::local::latch has been moved into the hpx namespace. The count_down_and_wait() functionality of latch has been renamed to arrive_and_wait().
- hpx::util::unique_function_nonser has been renamed to hpx::move_only_function.
- hpx::util::unique_function has been renamed to hpx::distributed::move_only_function.
- hpx::util::function has been renamed to hpx::distributed::function.
- hpx::util::function_nonser has been renamed to hpx::function.
- hpx::util::function_ref have been moved to namespace hpx.
- hpx::lcos::split_future changed namespace and is now used as hpx::split_future.
- hpx::lcos::local::counting_semaphore has been deprecated and hpx::lcos::local::cpp20_counting_semaphore has been renamed to hpx::counting_semaphore.
- hpx::lcos::local::cpp20_binary_semaphore has been renamed to hpx::binary_semaphore.
- hpx::lcos::local::sliding_semaphore has been renamed to hpx::sliding_semaphore and
- hpx::lcos::local::sliding_semaphore_var has been renamed to hpx::sliding_semaphore_var.
- hpx::lcos::local::spinlock has been renamed to hpx::spinlock.
- hpx::lcos::local::mutex has been renamed to hpx::mutex.
- hpx::lcos::local::timed_mutex has been renamed to hpx::timed_mutex.
- hpx::lcos::local::no_mutex has been renamed to hpx::no_mutex.
- hpx::lcos::local::recursive_mutex has been renamed to hpx::recursive_mutex.
- hpx::lcos::local::shared_mutex has been renamed to hpx::shared_mutex.
- hpx::lcos::local::upgrade_lock has been renamed to hpx::upgrade_lock.
- hpx::lcos::local::upgrade_to_unique_lock has been renamed to hpx::upgrade_to_unique_lock.
- hpx::lcos::local::condition_variable has been renamed to hpx::condition_variable. hpx::lcos::local::condition_variable_var has been renamed to hpx::condition_variable_var.
- hpx::lcos::local::once_flag has been renamed to hpx::once_flag, and . hpx::lcos::local::call_once has been renamed to hpx::call_once.

• The new LCI (Lightweight Communication Interface) parcelport was added that supports irregular and asynchronous applications like graph analysis, sparce linear algebra, modern parallel architectures etc. Major features include:

- Support for advanced communication primitives like two sided send/recv and one sided remote put.
- Better multi-threaded performance.
- Explicit user control of communication resource.
- Flexible signaling mechanisms (synchronizer, completion queue, active message handler).

- The following CMake flags were added, mostly to support using HPX as a backend for SHAD (https://github.com/pnnl/SHAD). Please note that these options enable questionable functionalities, partially they even enable undefined behavior. Please only use any of them if you know what you're doing:

- `HPX_SERIALIZATION_WITH_ALLOW_RAW_POINTER_SERIALIZATION`
- `HPX_SERIALIZATION_WITH_ALL_TYPES_ARE_BITWISE_SERIALIZABLE`
- `HPX_SERIALIZATION_WITH_ALLOW_CONST_TUPLE_MEMBERS`

Breaking changes

- Minimum required C++ standard library is C++17.
- Support for GCC 7 and Clang 8.0.0 and below has been removed.
- CUDA version required updated to 11.4.
- CMake version required updated to 3.18.
- The default version of Asio used was updated to 1.20.0.
- The default version of APEX used was updated to 2.5.1.
- APEX version was updated to 2.5.1.
- `tagged_pair` and `tagged_tuple` were removed.
- `tag_dispatch` was renamed to `tag_invoke`.
- `hpx.max_backgroud_threads` was renamed to `hpx.parcel.max_background_threads`.
- The following CMake flags were removed after being deprecated for at least two releases:

- `HPX_SCHEDULER_MAX_TERMINATED_THREADS`
- `HPX_WITH_GOOGLE_PERFTOOLS`
- `HPX_WITH_INIT_START_OVERLOADS_COMPATIBILITY`
- `HPX_HAVE_{COROUTINE,PLUGIN}_GCC_HIDDEN_VISIBILITY`
- `HPX_TOP_LEVEL`
- `HPX_WITH_COMPUTE_CUDA`
- `HPX_WITH_ASYNC_CUDA`

- `annotate_function` was renamed to `scoped_annotation`.
- `execution::transform` was renamed to `execution::then`.
- `execution::detach` was renamed to `execution::start_detached`.
- `execution::on_sender` was renamed to `execution::schedule_on`.

- `execution::just_on` was renamed to `execution::just_transfer`.
- `execution::set_done` was renamed to `execution::set_stopped`.

Closed issues

- Issue #5871[1691] - distributed::channel.regsiter_as terminates the active task.
- Issue #5856[1692] - Performance counters do not compile
- Issue #5828[1693] - hpx::distributed:barrier errors
- Issue #5812[1694] - OctoTiger does not compile with HPX master and CUDA 11.5
- Issue #5784[1695] - HPX failing with co_await and hpx::when_all(futures)
- Issue #5774[1696] - CMake can't find HPXCacheVariables.cmake
- Issue #5764[1697] - Fix HIP problem
- Issue #5724[1698] - Missing binary filter compression header
- Issue #5721[1699] - Cleanup after repository split
- Issue #5701[1700] - It seems that the tcp parcelport is running, and the MPI parcelport is ignored
- Issue #5692[1701] - Kokkos compilation fails when using both HPX and CUDA execution spaces with gcc 9.3.0
- Issue #5686[1702] - Rename *annotate_function*
- Issue #5668[1703] - HPX does not detect the C++ 20 standard using gcc 11.2
- Issue #5666[1704] - Compilation error using boost 1.76 and gcc 11.2.1
- Issue #5653[1705] - Implement P2248 for our algorithms
- Issue #5647[1706] - [User input needed] Remove (CUDA) compute functionality?
- Issue #5590[1707] - hello_world_distributed fails on startup with HPX stable, MPICH 3.3.2, on Deep Bayou
- Issue #5570[1708] - Rename tag_dispatch to tag_invoke

[1691] https://github.com/TheHPXProject/hpx/issues/5871
[1692] https://github.com/TheHPXProject/hpx/issues/5856
[1693] https://github.com/TheHPXProject/hpx/issues/5828
[1694] https://github.com/TheHPXProject/hpx/issues/5812
[1695] https://github.com/TheHPXProject/hpx/issues/5784
[1696] https://github.com/TheHPXProject/hpx/issues/5774
[1697] https://github.com/TheHPXProject/hpx/issues/5764
[1698] https://github.com/TheHPXProject/hpx/issues/5724
[1699] https://github.com/TheHPXProject/hpx/issues/5721
[1700] https://github.com/TheHPXProject/hpx/issues/5701
[1701] https://github.com/TheHPXProject/hpx/issues/5692
[1702] https://github.com/TheHPXProject/hpx/issues/5686
[1703] https://github.com/TheHPXProject/hpx/issues/5668
[1704] https://github.com/TheHPXProject/hpx/issues/5666
[1705] https://github.com/TheHPXProject/hpx/issues/5653
[1706] https://github.com/TheHPXProject/hpx/issues/5647
[1707] https://github.com/TheHPXProject/hpx/issues/5590
[1708] https://github.com/TheHPXProject/hpx/issues/5570

- Issue #5566[1709] - can't build simple example: "Cannot use the dummy implementation of future_then_dispatch"
- Issue #5565[1710] - build failure: hpx::string_util::trim()
- Issue #5553[1711] - Github action to validate the cff file refs #5471
- Issue #5504[1712] - CMake does not work for HPX 1.7.0 on Piz Daint
- Issue #5503[1713] - Use contiguous index queue in bulk execution to reduce number of spawned tasks
- Issue #5502[1714] - C++20 std::coroutine cmake detection
- Issue #5478[1715] - hpx.dll built with vcpkg got functions pointing to the same location
- Issue #5472[1716] - Compilation error with cuda/11.3
- Issue #5469[1717] - Compiler warning about HPX_NODISCARD when building with APEX
- Issue #5463[1718] - Address minor comments of the C++17 PR bump
- Issue #5456[1719] - Use *std::ranges::iter_swap* where available
- Issue #5404[1720] - Build fails with error "Cannot open include file asio/io_context.hpp"
- Issue #5381[1721] - Add starts_with and ends_with algorithms
- Issue #5344[1722] - Further simplify tag_invoke helpers
- Issue #5269[1723] - Allow setting a label on executors/policies
- Issue #5219[1724] - (Re-)Implement executor API on top of sender/receiver infrastructure
- Issue #5216[1725] - Performance counter module not loading
- Issue #5162[1726] - Require C++17 support
- Issue #5156[1727] - Disentangle segmented algorithms
- Issue #5118[1728] - Lock held while suspending
- Issue #5111[1729] - Tests fail to build with binary_filter plugins enabled
- Issue #5110[1730] - Tests don't get built

[1709] https://github.com/TheHPXProject/hpx/issues/5566
[1710] https://github.com/TheHPXProject/hpx/issues/5565
[1711] https://github.com/TheHPXProject/hpx/issues/5553
[1712] https://github.com/TheHPXProject/hpx/issues/5504
[1713] https://github.com/TheHPXProject/hpx/issues/5503
[1714] https://github.com/TheHPXProject/hpx/issues/5502
[1715] https://github.com/TheHPXProject/hpx/issues/5478
[1716] https://github.com/TheHPXProject/hpx/issues/5472
[1717] https://github.com/TheHPXProject/hpx/issues/5469
[1718] https://github.com/TheHPXProject/hpx/issues/5463
[1719] https://github.com/TheHPXProject/hpx/issues/5456
[1720] https://github.com/TheHPXProject/hpx/issues/5404
[1721] https://github.com/TheHPXProject/hpx/issues/5381
[1722] https://github.com/TheHPXProject/hpx/issues/5344
[1723] https://github.com/TheHPXProject/hpx/issues/5269
[1724] https://github.com/TheHPXProject/hpx/issues/5219
[1725] https://github.com/TheHPXProject/hpx/issues/5216
[1726] https://github.com/TheHPXProject/hpx/issues/5162
[1727] https://github.com/TheHPXProject/hpx/issues/5156
[1728] https://github.com/TheHPXProject/hpx/issues/5118
[1729] https://github.com/TheHPXProject/hpx/issues/5111
[1730] https://github.com/TheHPXProject/hpx/issues/5110

- Issue #5105[1731] - PAPI performance counters not available
- Issue #5002[1732] - hpx::lcos::barrier() results in deadlock
- Issue #4992[1733] - Clang-format the rest of the files
- Issue #4987[1734] - Use std::function in public APIs
- Issue #4871[1735] - HEP: conformance to C++20
- Issue #4822[1736] - Adapt parallel algorithms to C++20
- Issue #4736[1737] - Deprecate hpx::flush and hpx::endl
- Issue #4558[1738] - Prevent work-stealing from stalling
- Issue #4495[1739] - Add anchor links to table rows in documentation
- Issue #4469[1740] - New thread state: *pending_low*
- Issue #4321[1741] - After the modularization the libfabric parcelport does not compile
- Issue #4308[1742] - Using APEX on multinode jobs when HPX_WITH_NETWORKING = OFF
- Issue #3995[1743] - Use C++20 std::source_location where available, adapt ours to conform
- Issue #3861[1744] - Selected processor does not support 'yield' in ARM mode
- Issue #3706[1745] - Add shift_left and shift_right algorithms
- Issue #3646[1746] - Parallel algorithms should accept iterator/sentinel pairs
- Issue #3636[1747] - HPX Modularization
- Issue #3546[1748] - Modularization of HPX
- Issue #3474[1749] - Modernize CMake used in HPX
- Issue #1836[1750] - hpx::parallel does not have a sort implementation
- Issue #1668[1751] - Adapt all parallel algorithms to Ranges TS
- Issue #1141[1752] - Implement N4409 on top of HPX

[1731] https://github.com/TheHPXProject/hpx/issues/5105
[1732] https://github.com/TheHPXProject/hpx/issues/5002
[1733] https://github.com/TheHPXProject/hpx/issues/4992
[1734] https://github.com/TheHPXProject/hpx/issues/4987
[1735] https://github.com/TheHPXProject/hpx/issues/4871
[1736] https://github.com/TheHPXProject/hpx/issues/4822
[1737] https://github.com/TheHPXProject/hpx/issues/4736
[1738] https://github.com/TheHPXProject/hpx/issues/4558
[1739] https://github.com/TheHPXProject/hpx/issues/4495
[1740] https://github.com/TheHPXProject/hpx/issues/4469
[1741] https://github.com/TheHPXProject/hpx/issues/4321
[1742] https://github.com/TheHPXProject/hpx/issues/4308
[1743] https://github.com/TheHPXProject/hpx/issues/3995
[1744] https://github.com/TheHPXProject/hpx/issues/3861
[1745] https://github.com/TheHPXProject/hpx/issues/3706
[1746] https://github.com/TheHPXProject/hpx/issues/3646
[1747] https://github.com/TheHPXProject/hpx/issues/3636
[1748] https://github.com/TheHPXProject/hpx/issues/3546
[1749] https://github.com/TheHPXProject/hpx/issues/3474
[1750] https://github.com/TheHPXProject/hpx/issues/1836
[1751] https://github.com/TheHPXProject/hpx/issues/1668
[1752] https://github.com/TheHPXProject/hpx/issues/1141

Closed pull requests

- PR #5885[1753] - Testing newer ASIO version
- PR #5884[1754] - Fix miscellaneous doc sections
- PR #5882[1755] - Fixing OctoTiger incompatibility introduced recently
- PR #5881[1756] - Fixing recent patch that disables ATOMIC_FLAG_INIT for C++20 and up
- PR #5880[1757] - refactor: convert *counter_status* enum to enum class
- PR #5878[1758] - Docs: Replaced non-existent create_reducer function with create_communicator
- PR #5877[1759] - Doc updates hpx runtime and resources
- PR #5876[1760] - Updates to documentation; grammar edits.
- PR #5875[1761] - Doc updates starting the hpx runtime
- PR #5874[1762] - Doc updates launching configuring
- PR #5873[1763] - Prevent certain generated files from being deleted on reconfigure
- PR #5870[1764] - Adding support for the PJM batch environment
- PR #5867[1765] - Update CMakeLists.txt
- PR #5866[1766] - add cmake option HPX_WITH_PARCELPORT_COUNTERS
- PR #5864[1767] - ATOMIC_INIT_FLAG is deprecated starting C++20
- PR #5863[1768] - Adding llvm 14.0.0 with boost 1.79.0 to Jenkins
- PR #5861[1769] - Let install step proceed on CircleCI even if the segmented algorithms fail
- PR #5860[1770] - Updating APEX tag
- PR #5859[1771] - Splitting documentation generation steps on CircleCI
- PR #5854[1772] - Fixing left-overs from changing counter_type to enum class
- PR #5853[1773] - Adding HPX dependency tool (adapted from Boostdep tool)

[1753] https://github.com/TheHPXProject/hpx/pull/5885
[1754] https://github.com/TheHPXProject/hpx/pull/5884
[1755] https://github.com/TheHPXProject/hpx/pull/5882
[1756] https://github.com/TheHPXProject/hpx/pull/5881
[1757] https://github.com/TheHPXProject/hpx/pull/5880
[1758] https://github.com/TheHPXProject/hpx/pull/5878
[1759] https://github.com/TheHPXProject/hpx/pull/5877
[1760] https://github.com/TheHPXProject/hpx/pull/5876
[1761] https://github.com/TheHPXProject/hpx/pull/5875
[1762] https://github.com/TheHPXProject/hpx/pull/5874
[1763] https://github.com/TheHPXProject/hpx/pull/5873
[1764] https://github.com/TheHPXProject/hpx/pull/5870
[1765] https://github.com/TheHPXProject/hpx/pull/5867
[1766] https://github.com/TheHPXProject/hpx/pull/5866
[1767] https://github.com/TheHPXProject/hpx/pull/5864
[1768] https://github.com/TheHPXProject/hpx/pull/5863
[1769] https://github.com/TheHPXProject/hpx/pull/5861
[1770] https://github.com/TheHPXProject/hpx/pull/5860
[1771] https://github.com/TheHPXProject/hpx/pull/5859
[1772] https://github.com/TheHPXProject/hpx/pull/5854
[1773] https://github.com/TheHPXProject/hpx/pull/5853

- PR #5852[1774] - Optimize LCI parcelport
- PR #5851[1775] - Forking dynamic_bitset from Boost
- PR #5850[1776] - Convert perf_counters::counter_type enum to enum class.
- PR #5849[1777] - Update LCI parcelport to LCI v1.7.1
- PR #5848[1778] - Fedora related fixes
- PR #5847[1779] - Fix API, troubleshooting & people
- PR #5844[1780] - Attempting to fix timeouts of segmented iterator tests
- PR #5842[1781] - change the default value of HPX_WITH_LCI_TAG to v1.7
- PR #5841[1782] - Move the split_future facilities into the namespace hpx
- PR #5840[1783] - wait_xxx_nothrow functions return whether one of the futures is exceptional
- PR #5839[1784] - Moving a list of synchronization primitives into namespace hpx
- PR #5837[1785] - Moving latch types to hpx and hpx::distributed namespaces
- PR #5835[1786] - Add missing compatibility layer for id_type::management_type values
- PR #5834[1787] - API docs changes
- PR #5831[1788] - Further improvement actions to rotate
- PR #5830[1789] - Exposing zero-copy serialization threshold through configuration option
- PR #5829[1790] - Attempting to fix failing barrier test
- PR #5827[1791] - Add back explicit template parameter to *ignore_while_checking* to compile with nvcc
- PR #5826[1792] - Reduce number of allocations while calling async_bulk_execute
- PR #5825[1793] - Steal from neighboring NUMA domain only
- PR #5823[1794] - Remove obsolete directories and adjust build system
- PR #5822[1795] - Clang-format remaining files

[1774] https://github.com/TheHPXProject/hpx/pull/5852
[1775] https://github.com/TheHPXProject/hpx/pull/5851
[1776] https://github.com/TheHPXProject/hpx/pull/5850
[1777] https://github.com/TheHPXProject/hpx/pull/5849
[1778] https://github.com/TheHPXProject/hpx/pull/5848
[1779] https://github.com/TheHPXProject/hpx/pull/5847
[1780] https://github.com/TheHPXProject/hpx/pull/5844
[1781] https://github.com/TheHPXProject/hpx/pull/5842
[1782] https://github.com/TheHPXProject/hpx/pull/5841
[1783] https://github.com/TheHPXProject/hpx/pull/5840
[1784] https://github.com/TheHPXProject/hpx/pull/5839
[1785] https://github.com/TheHPXProject/hpx/pull/5837
[1786] https://github.com/TheHPXProject/hpx/pull/5835
[1787] https://github.com/TheHPXProject/hpx/pull/5834
[1788] https://github.com/TheHPXProject/hpx/pull/5831
[1789] https://github.com/TheHPXProject/hpx/pull/5830
[1790] https://github.com/TheHPXProject/hpx/pull/5829
[1791] https://github.com/TheHPXProject/hpx/pull/5827
[1792] https://github.com/TheHPXProject/hpx/pull/5826
[1793] https://github.com/TheHPXProject/hpx/pull/5825
[1794] https://github.com/TheHPXProject/hpx/pull/5823
[1795] https://github.com/TheHPXProject/hpx/pull/5822

- PR #5821[1796] - Enable permissive- flag on Windows GitHub actions builders
- PR #5820[1797] - Convert throwmode enum to enum class
- PR #5819[1798] - Marking customization points for intrusive_ptr as noexcept
- PR #5818[1799] - Unconditionally use C++17 attributes
- PR #5817[1800] - Modernize naming modules
- PR #5816[1801] - Modernize cache module
- PR #5815[1802] - Reapply flyby changes from #5467
- PR #5814[1803] - Avoid test timeouts by reducing test sizes
- PR #5813[1804] - The CUDA problem is not fixed in V11.5 yet…
- PR #5811[1805] - Make sure reduction value is properly moved, when possible
- PR #5810[1806] - Improve error reporting during device initialization in HIP environments
- PR #5809[1807] - Converting scheduler enums into enum class
- PR #5808[1808] - Deprecate hpx::flush and friends
- PR #5807[1809] - Use C++20 std::source_location, if available
- PR #5806[1810] - Moving promise and packaged_task to new namespaces
- PR #5805[1811] - Attempting to fix a test failure when using the LCI parcelpor
- PR #5803[1812] - Attempt to fix CUDA related OctoTiger problems
- PR #5800[1813] - Add option to restrict MPI background work to subset of cores
- PR #5798[1814] - Adding MPI as a dependency to APEX
- PR #5797[1815] - Extend Sphinx role to support arbitrary text to display on a link
- PR #5796[1816] - Disable CUDA tests that cause NVCC to silently fail without error messages
- PR #5795[1817] - Avoid writing path and directories into HPXCacheVariables.cmake
- PR #5793[1818] - Remove features that are deprecated since V1.6

[1796] https://github.com/TheHPXProject/hpx/pull/5821
[1797] https://github.com/TheHPXProject/hpx/pull/5820
[1798] https://github.com/TheHPXProject/hpx/pull/5819
[1799] https://github.com/TheHPXProject/hpx/pull/5818
[1800] https://github.com/TheHPXProject/hpx/pull/5817
[1801] https://github.com/TheHPXProject/hpx/pull/5816
[1802] https://github.com/TheHPXProject/hpx/pull/5815
[1803] https://github.com/TheHPXProject/hpx/pull/5814
[1804] https://github.com/TheHPXProject/hpx/pull/5813
[1805] https://github.com/TheHPXProject/hpx/pull/5811
[1806] https://github.com/TheHPXProject/hpx/pull/5810
[1807] https://github.com/TheHPXProject/hpx/pull/5809
[1808] https://github.com/TheHPXProject/hpx/pull/5808
[1809] https://github.com/TheHPXProject/hpx/pull/5807
[1810] https://github.com/TheHPXProject/hpx/pull/5806
[1811] https://github.com/TheHPXProject/hpx/pull/5805
[1812] https://github.com/TheHPXProject/hpx/pull/5803
[1813] https://github.com/TheHPXProject/hpx/pull/5800
[1814] https://github.com/TheHPXProject/hpx/pull/5798
[1815] https://github.com/TheHPXProject/hpx/pull/5797
[1816] https://github.com/TheHPXProject/hpx/pull/5796
[1817] https://github.com/TheHPXProject/hpx/pull/5795
[1818] https://github.com/TheHPXProject/hpx/pull/5793

- PR #5792[1819] - Making sure num_cores is properly handled by parallel_executor
- PR #5791[1820] - Moving bind, bind_front, bind_back to namespace hpx
- PR #5790[1821] - Moving serializable function/move_only_function into namespace hpx::distributed
- PR #5787[1822] - Remove unneeded (and commented) tests
- PR #5786[1823] - Attempting to fix hangs in distributed barrier
- PR #5785[1824] - add cmake code to detect arm64 on macOS
- PR #5783[1825] - Moving function and function_ref into namespace hpx
- PR #5781[1826] - Updating used version of Visual Studio
- PR #5780[1827] - Update Piz Daint Jenkins configurations from gcc/clang 7 to 8
- PR #5778[1828] - Updated for_loop.hpp
- PR #5777[1829] - Update reference for foreach benchmark
- PR #5775[1830] - Move optional into namespace hpx
- PR #5773[1831] - Moving barrier to consolidated namespaces
- PR #5772[1832] - Adding missing docs for ranges::find_if and find_if_not algorithms
- PR #5771[1833] - Moving for_loop into namespace hpx::experimental
- PR #5770[1834] - Fixing HIP issues
- PR #5769[1835] - Slight improvement of small_vector performance
- PR #5766[1836] - Fixing a integral conversion warning
- PR #5765[1837] - Adding a sphinx role allowing to link to a file directly in github
- PR #5763[1838] - add num_cores facility
- PR #5762[1839] - Fix Public API main page
- PR #5761[1840] - Add missing inline to mpi_exception.hpp error_message function
- PR #5760[1841] - Update cdash build url

[1819] https://github.com/TheHPXProject/hpx/pull/5792
[1820] https://github.com/TheHPXProject/hpx/pull/5791
[1821] https://github.com/TheHPXProject/hpx/pull/5790
[1822] https://github.com/TheHPXProject/hpx/pull/5787
[1823] https://github.com/TheHPXProject/hpx/pull/5786
[1824] https://github.com/TheHPXProject/hpx/pull/5785
[1825] https://github.com/TheHPXProject/hpx/pull/5783
[1826] https://github.com/TheHPXProject/hpx/pull/5781
[1827] https://github.com/TheHPXProject/hpx/pull/5780
[1828] https://github.com/TheHPXProject/hpx/pull/5778
[1829] https://github.com/TheHPXProject/hpx/pull/5777
[1830] https://github.com/TheHPXProject/hpx/pull/5775
[1831] https://github.com/TheHPXProject/hpx/pull/5773
[1832] https://github.com/TheHPXProject/hpx/pull/5772
[1833] https://github.com/TheHPXProject/hpx/pull/5771
[1834] https://github.com/TheHPXProject/hpx/pull/5770
[1835] https://github.com/TheHPXProject/hpx/pull/5769
[1836] https://github.com/TheHPXProject/hpx/pull/5766
[1837] https://github.com/TheHPXProject/hpx/pull/5765
[1838] https://github.com/TheHPXProject/hpx/pull/5763
[1839] https://github.com/TheHPXProject/hpx/pull/5762
[1840] https://github.com/TheHPXProject/hpx/pull/5761
[1841] https://github.com/TheHPXProject/hpx/pull/5760

- PR #5759[1842] - Switch to use generic rostam SLURM partitions
- PR #5758[1843] - Adding support for P2300 completion signatures
- PR #5757[1844] - Fix missing links in Public API
- PR #5756[1845] - Add stop support to when_all
- PR #5755[1846] - Support for data-parallelism for mismatch algorithm
- PR #5754[1847] - Support for data-parallelism for equal algorithm
- PR #5751[1848] - Propagate MPI dependencies to command line handling
- PR #5750[1849] - Make sure required MPI initialization flags are properly applied and supported
- PR #5749[1850] - P2300 stop token
- PR #5748[1851] - Adding environmental query CPOs
- PR #5747[1852] - Renaming set_done to set_stopped (as per P2300)
- PR #5745[1853] - Modernize serialization module
- PR #5743[1854] - Add check for MPICH and set the correct env to support multi-threaded
- PR #5742[1855] - Remove obsolete files related to cpuid, etc.
- PR #5741[1856] - Support for data-parallelism for adjacent find
- PR #5740[1857] - Support for data-parallelism for find algorithms
- PR #5739[1858] - Enable the option to attach a debugger on a segmentation fault (linux)
- PR #5738[1859] - Fixing spell-checking errors
- PR #5737[1860] - Attempt to fix migrate_component issue
- PR #5736[1861] - Set commit status from Jenkins also for special branches
- PR #5734[1862] - Revert #5586
- PR #5732[1863] - Attempt to improve build-id reporting to cdash
- PR #5731[1864] - Randomly delay execution of bash scripts launched by Jenkins

[1842] https://github.com/TheHPXProject/hpx/pull/5759
[1843] https://github.com/TheHPXProject/hpx/pull/5758
[1844] https://github.com/TheHPXProject/hpx/pull/5757
[1845] https://github.com/TheHPXProject/hpx/pull/5756
[1846] https://github.com/TheHPXProject/hpx/pull/5755
[1847] https://github.com/TheHPXProject/hpx/pull/5754
[1848] https://github.com/TheHPXProject/hpx/pull/5751
[1849] https://github.com/TheHPXProject/hpx/pull/5750
[1850] https://github.com/TheHPXProject/hpx/pull/5749
[1851] https://github.com/TheHPXProject/hpx/pull/5748
[1852] https://github.com/TheHPXProject/hpx/pull/5747
[1853] https://github.com/TheHPXProject/hpx/pull/5745
[1854] https://github.com/TheHPXProject/hpx/pull/5743
[1855] https://github.com/TheHPXProject/hpx/pull/5742
[1856] https://github.com/TheHPXProject/hpx/pull/5741
[1857] https://github.com/TheHPXProject/hpx/pull/5740
[1858] https://github.com/TheHPXProject/hpx/pull/5739
[1859] https://github.com/TheHPXProject/hpx/pull/5738
[1860] https://github.com/TheHPXProject/hpx/pull/5737
[1861] https://github.com/TheHPXProject/hpx/pull/5736
[1862] https://github.com/TheHPXProject/hpx/pull/5734
[1863] https://github.com/TheHPXProject/hpx/pull/5732
[1864] https://github.com/TheHPXProject/hpx/pull/5731

- PR #5729[1865] - Workaround for CMake/Ninja generator OOM problem
- PR #5727[1866] - Moving compression plugins to components directory
- PR #5726[1867] - Moving/consolidating parcel coalescing plugin sources
- PR #5725[1868] - Making sure headers for serialization filters are being installed
- PR #5723[1869] - Moving more tests to modules
- PR #5722[1870] - Removing superfluous semicolons
- PR #5720[1871] - Moving parcelports into modules
- PR #5719[1872] - Moving more files to parcelset module
- PR #5718[1873] - build: refactor sphinx config file
- PR #5717[1874] - Creating parcelset modules
- PR #5716[1875] - Avoid duplicate definition error
- PR #5715[1876] - The new LCI parcelport for HPX
- PR #5714[1877] - Refine propagation of **HPX_WITH_**... options
- PR #5713[1878] - Significantly reduce CI jobs run on Piz Daint
- PR #5712[1879] - Updating jenkins configuration for Rostam2.2
- PR #5711[1880] - Refactor manual sections
- PR #5710[1881] - Making task_group serializable
- PR #5709[1882] - Update the MPI cmake setup
- PR #5707[1883] - Better diagnose parcel bootstrap problems
- PR #5704[1884] - Test with hwloc 2.7.0 with GCC 11
- PR #5703[1885] - Fix *counting_iterator* container tests
- PR #5702[1886] - Attempting to fix CircleCI timeouts
- PR #5699[1887] - Update CI to use Boost 1.78.0

[1865] https://github.com/TheHPXProject/hpx/pull/5729
[1866] https://github.com/TheHPXProject/hpx/pull/5727
[1867] https://github.com/TheHPXProject/hpx/pull/5726
[1868] https://github.com/TheHPXProject/hpx/pull/5725
[1869] https://github.com/TheHPXProject/hpx/pull/5723
[1870] https://github.com/TheHPXProject/hpx/pull/5722
[1871] https://github.com/TheHPXProject/hpx/pull/5720
[1872] https://github.com/TheHPXProject/hpx/pull/5719
[1873] https://github.com/TheHPXProject/hpx/pull/5718
[1874] https://github.com/TheHPXProject/hpx/pull/5717
[1875] https://github.com/TheHPXProject/hpx/pull/5716
[1876] https://github.com/TheHPXProject/hpx/pull/5715
[1877] https://github.com/TheHPXProject/hpx/pull/5714
[1878] https://github.com/TheHPXProject/hpx/pull/5713
[1879] https://github.com/TheHPXProject/hpx/pull/5712
[1880] https://github.com/TheHPXProject/hpx/pull/5711
[1881] https://github.com/TheHPXProject/hpx/pull/5710
[1882] https://github.com/TheHPXProject/hpx/pull/5709
[1883] https://github.com/TheHPXProject/hpx/pull/5707
[1884] https://github.com/TheHPXProject/hpx/pull/5704
[1885] https://github.com/TheHPXProject/hpx/pull/5703
[1886] https://github.com/TheHPXProject/hpx/pull/5702
[1887] https://github.com/TheHPXProject/hpx/pull/5699

- PR #5697[1888] - Adding fork_join_executor to foreach_benchmark
- PR #5696[1889] - Modernize when_all and friends (when_any, when_some, when_each)
- PR #5693[1890] - Fix test errors with *_GLIBCXX_DEBUG* defined
- PR #5691[1891] - Rename *annotate_function* to *scoped_annotation*
- PR #5690[1892] - Replace tag_dispatch with tag_invoke in minmax segmented
- PR #5688[1893] - Remove more deprecated macros
- PR #5687[1894] - Add most important CMake options
- PR #5685[1895] - Fix future API
- PR #5684[1896] - Move lock registration to separate module and remove global lock registration
- PR #5683[1897] - Make hpx::wait_all etc. throw exceptions when waited futures hold exceptions and deprecate hpx::lcos::wait_all[_n] in favor of hpx::wait_all[_n]
- PR #5682[1898] - Fix macOS test exceptions
- PR #5681[1899] - docs: add links to hpx recepies
- PR #5680[1900] - Embed base execution policies to datapar execution policies
- PR #5679[1901] - Fix *fork_join_executor* with dynamic schedule
- PR #5678[1902] - Fix compilation of service executors with nvcc
- PR #5677[1903] - Remove compute_cuda module
- PR #5676[1904] - Don't require up-to-date approvals for bors
- PR #5675[1905] - Add default template type parameters for algorithms
- PR #5674[1906] - Allow using *any_sender* in global variables
- PR #5671[1907] - Making sure task_group can be reused
- PR #5670[1908] - Relax constraints on *execution::when_all*
- PR #5669[1909] - Use HPX_WITH_CXX_STANDARD for controlling C++ version

[1888] https://github.com/TheHPXProject/hpx/pull/5697
[1889] https://github.com/TheHPXProject/hpx/pull/5696
[1890] https://github.com/TheHPXProject/hpx/pull/5693
[1891] https://github.com/TheHPXProject/hpx/pull/5691
[1892] https://github.com/TheHPXProject/hpx/pull/5690
[1893] https://github.com/TheHPXProject/hpx/pull/5688
[1894] https://github.com/TheHPXProject/hpx/pull/5687
[1895] https://github.com/TheHPXProject/hpx/pull/5685
[1896] https://github.com/TheHPXProject/hpx/pull/5684
[1897] https://github.com/TheHPXProject/hpx/pull/5683
[1898] https://github.com/TheHPXProject/hpx/pull/5682
[1899] https://github.com/TheHPXProject/hpx/pull/5681
[1900] https://github.com/TheHPXProject/hpx/pull/5680
[1901] https://github.com/TheHPXProject/hpx/pull/5679
[1902] https://github.com/TheHPXProject/hpx/pull/5678
[1903] https://github.com/TheHPXProject/hpx/pull/5677
[1904] https://github.com/TheHPXProject/hpx/pull/5676
[1905] https://github.com/TheHPXProject/hpx/pull/5675
[1906] https://github.com/TheHPXProject/hpx/pull/5674
[1907] https://github.com/TheHPXProject/hpx/pull/5671
[1908] https://github.com/TheHPXProject/hpx/pull/5670
[1909] https://github.com/TheHPXProject/hpx/pull/5669

- PR #5667[1910] - Attempt to fix compilation issues with Boost V1.76
- PR #5664[1911] - Change logging errors to warnings in schedulers
- PR #5663[1912] - Use dynamic bitsets by default for CPU masks
- PR #5662[1913] - Disambiguate namespace for MSVC
- PR #5660[1914] - Replacing remaining std::forward and std::move with HPX_FORWARD and HPX_MOVE
- PR #5659[1915] - Modernize hpx::future and related facilities
- PR #5658[1916] - Replace HPX_INLINE_CONSTEXPR_VARIABLE with inline constexpr
- PR #5657[1917] - Remove tagged, tagged_pair and tagged_tuple, remove tuple/pair specializations
- PR #5656[1918] - Rename on execution::schedule_from, rename just_on to just_transfer, and add transfer
- PR #5655[1919] - Avoid for module lists to grow indefinitely in cmake cache
- PR #5649[1920] - build: replace usage of Python's reserved words and functions as variable names
- PR #5648[1921] - Modernize action modules and related code
- PR #5646[1922] - Fix ends_with test
- PR #5645[1923] - Add matrix multiplication example
- PR #5644[1924] - Rename execution::transform to execution::then and execution::detach to execution::start_detached
- PR #5643[1925] - Update performance test references
- PR #5642[1926] - Adapting adjacent_difference to work with proxy iterators
- PR #5641[1927] - Factorize perftests scripts
- PR #5640[1928] - Fixed links to sources in Sphinx documentation
- PR #5639[1929] - Fix generate datapar tests for Vc

[1910] https://github.com/TheHPXProject/hpx/pull/5667
[1911] https://github.com/TheHPXProject/hpx/pull/5664
[1912] https://github.com/TheHPXProject/hpx/pull/5663
[1913] https://github.com/TheHPXProject/hpx/pull/5662
[1914] https://github.com/TheHPXProject/hpx/pull/5660
[1915] https://github.com/TheHPXProject/hpx/pull/5659
[1916] https://github.com/TheHPXProject/hpx/pull/5658
[1917] https://github.com/TheHPXProject/hpx/pull/5657
[1918] https://github.com/TheHPXProject/hpx/pull/5656
[1919] https://github.com/TheHPXProject/hpx/pull/5655
[1920] https://github.com/TheHPXProject/hpx/pull/5649
[1921] https://github.com/TheHPXProject/hpx/pull/5648
[1922] https://github.com/TheHPXProject/hpx/pull/5646
[1923] https://github.com/TheHPXProject/hpx/pull/5645
[1924] https://github.com/TheHPXProject/hpx/pull/5644
[1925] https://github.com/TheHPXProject/hpx/pull/5643
[1926] https://github.com/TheHPXProject/hpx/pull/5642
[1927] https://github.com/TheHPXProject/hpx/pull/5641
[1928] https://github.com/TheHPXProject/hpx/pull/5640
[1929] https://github.com/TheHPXProject/hpx/pull/5639

- PR #5638[1930] - Simd all any none
- PR #5637[1931] - Use bors for merging pull requests
- PR #5636[1932] - Fix leftover std::holds_alternative usage
- PR #5635[1933] - Update container image tag in GitHub actions HIP configuration
- PR #5633[1934] - Moving packaged_task to module futures
- PR #5632[1935] - Tell Asio to use std::aligned_new only if available
- PR #5631[1936] - Adding tag parameter to channel communicator get/set
- PR #5630[1937] - Add partial_sort_copy and adapt partial sort to c++ 20
- PR #5629[1938] - Set HPX_WITH_FETCH_ASIO to OFF as available in the docker image
- PR #5628[1939] - Add Clang 13 CI configuration
- PR #5627[1940] - Replace alternative keyword
- PR #5626[1941] - docs: add support for BibTeX references in Sphinx docs
- PR #5624[1942] - Fix pkgconfig replacements involving CMAKE_INSTALL_PREFIX
- PR #5623[1943] - build: remove unused import from conf.py.in
- PR #5622[1944] - Remove HPX_WITH_VCPKG CMake option
- PR #5621[1945] - Replacing boost::container::small_vector
- PR #5620[1946] - Update Asio tag from 1.18.2 to 1.20.0
- PR #5619[1947] - Fix block_os_threads_1036 test
- PR #5618[1948] - Make sure condition variables are notified under a lock in the thread_pool_scheduler test
- PR #5617[1949] - Use advance_and_get_distance where required
- PR #5616[1950] - Remove separately building segmented algorithms on CircleCI
- PR #5613[1951] - Fix Vc datapar adjacent_difference
- PR #5609[1952] - docs: add anchor links to performance counter tables

[1930] https://github.com/TheHPXProject/hpx/pull/5638
[1931] https://github.com/TheHPXProject/hpx/pull/5637
[1932] https://github.com/TheHPXProject/hpx/pull/5636
[1933] https://github.com/TheHPXProject/hpx/pull/5635
[1934] https://github.com/TheHPXProject/hpx/pull/5633
[1935] https://github.com/TheHPXProject/hpx/pull/5632
[1936] https://github.com/TheHPXProject/hpx/pull/5631
[1937] https://github.com/TheHPXProject/hpx/pull/5630
[1938] https://github.com/TheHPXProject/hpx/pull/5629
[1939] https://github.com/TheHPXProject/hpx/pull/5628
[1940] https://github.com/TheHPXProject/hpx/pull/5627
[1941] https://github.com/TheHPXProject/hpx/pull/5626
[1942] https://github.com/TheHPXProject/hpx/pull/5624
[1943] https://github.com/TheHPXProject/hpx/pull/5623
[1944] https://github.com/TheHPXProject/hpx/pull/5622
[1945] https://github.com/TheHPXProject/hpx/pull/5621
[1946] https://github.com/TheHPXProject/hpx/pull/5620
[1947] https://github.com/TheHPXProject/hpx/pull/5619
[1948] https://github.com/TheHPXProject/hpx/pull/5618
[1949] https://github.com/TheHPXProject/hpx/pull/5617
[1950] https://github.com/TheHPXProject/hpx/pull/5616
[1951] https://github.com/TheHPXProject/hpx/pull/5613
[1952] https://github.com/TheHPXProject/hpx/pull/5609

- PR #5608[1953] - Fix header test error by adding missing numeric
- PR #5607[1954] - Fix simd adj diff
- PR #5605[1955] - Fix usage of HPX_INVOKE macro
- PR #5604[1956] - Make use of shell-session to allow non-copyable $
- PR #5603[1957] - Suppress some MSVC warnings in C++20 mode
- PR #5602[1958] - Test HPX_DATASTRUCTURES_WITH_ADAPT_STD_TUPLE=OFF to one CI configuration
- PR #5601[1959] - Test case for any_sender should use hpx::tuple
- PR #5600[1960] - Rename tag_dispatch back to tag_invoke
- PR #5599[1961] - Change theme, fix Quickstart & Examples
- PR #5596[1962] - Use precompiled headers in tests
- PR #5595[1963] - Drop semicolons for macro calls
- PR #5594[1964] - Adapt datapar generate
- PR #5593[1965] - Update any_sender to use tag_dispatch for execution customizations
- PR #5592[1966] - Add nth_element
- PR #5591[1967] - Remove unnecessary checks for C++17 for tests
- PR #5589[1968] - Add HPX_FORWARD/HPX_MOVE macros
- PR #5588[1969] - Fixing the output formatting for id_types
- PR #5586[1970] - Remove local functionality
- PR #5585[1971] - Delete GitExternal.cmake
- PR #5584[1972] - Serialization of hpx::tuple must use hpx::get
- PR #5583[1973] - fix coroutine_traits allocate calls, add unhandled_exception() implementation.
- PR #5582[1974] - Make more examples work with local runtime

[1953] https://github.com/TheHPXProject/hpx/pull/5608
[1954] https://github.com/TheHPXProject/hpx/pull/5607
[1955] https://github.com/TheHPXProject/hpx/pull/5605
[1956] https://github.com/TheHPXProject/hpx/pull/5604
[1957] https://github.com/TheHPXProject/hpx/pull/5603
[1958] https://github.com/TheHPXProject/hpx/pull/5602
[1959] https://github.com/TheHPXProject/hpx/pull/5601
[1960] https://github.com/TheHPXProject/hpx/pull/5600
[1961] https://github.com/TheHPXProject/hpx/pull/5599
[1962] https://github.com/TheHPXProject/hpx/pull/5596
[1963] https://github.com/TheHPXProject/hpx/pull/5595
[1964] https://github.com/TheHPXProject/hpx/pull/5594
[1965] https://github.com/TheHPXProject/hpx/pull/5593
[1966] https://github.com/TheHPXProject/hpx/pull/5592
[1967] https://github.com/TheHPXProject/hpx/pull/5591
[1968] https://github.com/TheHPXProject/hpx/pull/5589
[1969] https://github.com/TheHPXProject/hpx/pull/5588
[1970] https://github.com/TheHPXProject/hpx/pull/5586
[1971] https://github.com/TheHPXProject/hpx/pull/5585
[1972] https://github.com/TheHPXProject/hpx/pull/5584
[1973] https://github.com/TheHPXProject/hpx/pull/5583
[1974] https://github.com/TheHPXProject/hpx/pull/5582

- PR #5581[1975] - Add support for several performance tests in CI
- PR #5580[1976] - Adapt simd adj diff
- PR #5579[1977] - Split absolute paths for generated pkg-config files into -L/-l parts
- PR #5577[1978] - fix unit fill test for datapar with Vc
- PR #5576[1979] - Update forgotten "Full" names
- PR #5575[1980] - Change scan partitioner implementation
- PR #5574[1981] - Remove a few deprecated and unused CMake options
- PR #5572[1982] - Remove more guards for the distributed runtime
- PR #5571[1983] - Add workaround for libstc++ in string_util trim
- PR #5569[1984] - Use no_unique_address in sender adaptors
- PR #5568[1985] - Change try catch block to try_catch_exception_ptr
- PR #5567[1986] - Make default_agent::yield actually yield
- PR #5564[1987] - Adjacent
- PR #5562[1988] - More changes to overcome build problems on Windows after recent module rearrangements
- PR #5560[1989] - Update tests and examples
- PR #5559[1990] - Fixing cmake folder names after module restructuring
- PR #5558[1991] - Fixing wrong module dependencies
- PR #5557[1992] - Adding an example for the new channel_communicator API
- PR #5556[1993] - Remove leftover thread pool os executor tests
- PR #5555[1994] - Add option enabling serializing raw pointers
- PR #5554[1995] - Make sure command line aliasing is properly handled
- PR #5552[1996] - Modernizing some of the async facilities
- PR #5551[1997] - Fixing for local executions of actions to properly set task names

1975 https://github.com/TheHPXProject/hpx/pull/5581
1976 https://github.com/TheHPXProject/hpx/pull/5580
1977 https://github.com/TheHPXProject/hpx/pull/5579
1978 https://github.com/TheHPXProject/hpx/pull/5577
1979 https://github.com/TheHPXProject/hpx/pull/5576
1980 https://github.com/TheHPXProject/hpx/pull/5575
1981 https://github.com/TheHPXProject/hpx/pull/5574
1982 https://github.com/TheHPXProject/hpx/pull/5572
1983 https://github.com/TheHPXProject/hpx/pull/5571
1984 https://github.com/TheHPXProject/hpx/pull/5569
1985 https://github.com/TheHPXProject/hpx/pull/5568
1986 https://github.com/TheHPXProject/hpx/pull/5567
1987 https://github.com/TheHPXProject/hpx/pull/5564
1988 https://github.com/TheHPXProject/hpx/pull/5562
1989 https://github.com/TheHPXProject/hpx/pull/5560
1990 https://github.com/TheHPXProject/hpx/pull/5559
1991 https://github.com/TheHPXProject/hpx/pull/5558
1992 https://github.com/TheHPXProject/hpx/pull/5557
1993 https://github.com/TheHPXProject/hpx/pull/5556
1994 https://github.com/TheHPXProject/hpx/pull/5555
1995 https://github.com/TheHPXProject/hpx/pull/5554
1996 https://github.com/TheHPXProject/hpx/pull/5552
1997 https://github.com/TheHPXProject/hpx/pull/5551

- PR #5550[1998] - Update CUDA module in clang-cuda configuration
- PR #5549[1999] - Fixing agent_ref::yield_k to actually call yield_k
- PR #5548[2000] - Making get_action_name() noexcept
- PR #5547[2001] - Fixing communication set
- PR #5546[2002] - Fixing shutdown problems caused by missing ref-counting
- PR #5545[2003] - Remove wrong move in thread_pool_scheduler_bulk.hpp
- PR #5543[2004] - Extend launch policy to carry stack size and scheduling hint in addition to priority
- PR #5542[2005] - Simplify execution CPOs
- PR #5540[2006] - Adapt partition, partition_copy and stable_partition to C++ 20
- PR #5539[2007] - Adapt mismatch to support sentinels
- PR #5538[2008] - Document specific sphinx version required for the documentation
- PR #5537[2009] - Test release and debug builds on Piz Daint
- PR #5536[2010] - This fixes referencing stale iterators during the execution of binary mismatch
- PR #5535[2011] - Rename simdpar to par_simd
- PR #5534[2012] - Fix Quick start & Manual Docs
- PR #5533[2013] - Fix *annotate_function* for *std::string*
- PR #5532[2014] - Update two remaining apex links from khuck to UO-OACISS
- PR #5531[2015] - Use contiguous_index_queue in thread_pool_scheduler
- PR #5530[2016] - Eagerly initialize a configurable number of threads on scheduler/thread queue init
- PR #5529[2017] - Update benchmarks and add support for scheduler_executor
- PR #5528[2018] - Add missing properties to executors/schedulers
- PR #5527[2019] - Set local thread/pool number in local/static_queue_scheduler

[1998] https://github.com/TheHPXProject/hpx/pull/5550
[1999] https://github.com/TheHPXProject/hpx/pull/5549
[2000] https://github.com/TheHPXProject/hpx/pull/5548
[2001] https://github.com/TheHPXProject/hpx/pull/5547
[2002] https://github.com/TheHPXProject/hpx/pull/5546
[2003] https://github.com/TheHPXProject/hpx/pull/5545
[2004] https://github.com/TheHPXProject/hpx/pull/5543
[2005] https://github.com/TheHPXProject/hpx/pull/5542
[2006] https://github.com/TheHPXProject/hpx/pull/5540
[2007] https://github.com/TheHPXProject/hpx/pull/5539
[2008] https://github.com/TheHPXProject/hpx/pull/5538
[2009] https://github.com/TheHPXProject/hpx/pull/5537
[2010] https://github.com/TheHPXProject/hpx/pull/5536
[2011] https://github.com/TheHPXProject/hpx/pull/5535
[2012] https://github.com/TheHPXProject/hpx/pull/5534
[2013] https://github.com/TheHPXProject/hpx/pull/5533
[2014] https://github.com/TheHPXProject/hpx/pull/5532
[2015] https://github.com/TheHPXProject/hpx/pull/5531
[2016] https://github.com/TheHPXProject/hpx/pull/5530
[2017] https://github.com/TheHPXProject/hpx/pull/5529
[2018] https://github.com/TheHPXProject/hpx/pull/5528
[2019] https://github.com/TheHPXProject/hpx/pull/5527

- PR #5526[2020] - Update Rostam HIP configuration to use 4.3.0
- PR #5525[2021] - Fix Building HPX in Quick start
- PR #5524[2022] - Upload image on cdash
- PR #5523[2023] - Modernize facilities related to hpx::sync
- PR #5522[2024] - Add sender overloads for remaining algorithms
- PR #5521[2025] - Minor changes that improve performance
- PR #5520[2026] - Update reference as perftests failing regularly
- PR #5519[2027] - Add transform_mpi sender adapter
- PR #5518[2028] - Add sender overloads to rotate/rotate_copy
- PR #5517[2029] - Fix coroutine integration
- PR #5515[2030] - Avoid deadlock in ignore_while_locked_1485 test
- PR #5514[2031] - Add split sender adapter
- PR #5512[2032] - Update Rostam HIP configuration
- PR #5511[2033] - Fix Asio target name for precompiled headers
- PR #5510[2034] - Add any_sender and unique_any_sender
- PR #5509[2035] - Test with Boost 1.77 on gcc/clang-newest configurations
- PR #5508[2036] - Minor release changes from 1.7.1
- PR #5507[2037] - Add missing commits from scheduler_executor PR
- PR #5506[2038] - Fix condition for checking if we should use our own variant
- PR #5501[2039] - Attempt to fix thread_pool_scheduler test
- PR #5493[2040] - Update Jenkins GitHub token to use StellarBot GitHub account
- PR #5490[2041] - Fix clang-format error on master
- PR #5487[2042] - Add get_completion_scheduler CPO and customize bulk for thread_pool_scheduler

[2020] https://github.com/TheHPXProject/hpx/pull/5526
[2021] https://github.com/TheHPXProject/hpx/pull/5525
[2022] https://github.com/TheHPXProject/hpx/pull/5524
[2023] https://github.com/TheHPXProject/hpx/pull/5523
[2024] https://github.com/TheHPXProject/hpx/pull/5522
[2025] https://github.com/TheHPXProject/hpx/pull/5521
[2026] https://github.com/TheHPXProject/hpx/pull/5520
[2027] https://github.com/TheHPXProject/hpx/pull/5519
[2028] https://github.com/TheHPXProject/hpx/pull/5518
[2029] https://github.com/TheHPXProject/hpx/pull/5517
[2030] https://github.com/TheHPXProject/hpx/pull/5515
[2031] https://github.com/TheHPXProject/hpx/pull/5514
[2032] https://github.com/TheHPXProject/hpx/pull/5512
[2033] https://github.com/TheHPXProject/hpx/pull/5511
[2034] https://github.com/TheHPXProject/hpx/pull/5510
[2035] https://github.com/TheHPXProject/hpx/pull/5509
[2036] https://github.com/TheHPXProject/hpx/pull/5508
[2037] https://github.com/TheHPXProject/hpx/pull/5507
[2038] https://github.com/TheHPXProject/hpx/pull/5506
[2039] https://github.com/TheHPXProject/hpx/pull/5501
[2040] https://github.com/TheHPXProject/hpx/pull/5493
[2041] https://github.com/TheHPXProject/hpx/pull/5490
[2042] https://github.com/TheHPXProject/hpx/pull/5487

- PR #5484[2043] - Add missing header to jacobi_component/server/solver.hpp
- PR #5481[2044] - Changing the APEX repository to the new location
- PR #5479[2045] - Fix version check for CUDA noexcept/result_of bug
- PR #5477[2046] - Require cxx17 minor comments
- PR #5476[2047] - Fix cmake format error
- PR #5475[2048] - Require CMake 3.18 as it is already a requirement for CUDA
- PR #5474[2049] - Make the cuda parameters of try_compile optional
- PR #5473[2050] - Update cuda arch and change cuda version
- PR #5471[2051] - Add corrected citation.cff
- PR #5470[2052] - Adapt stable_sort to C++ 20
- PR #5468[2053] - Experimentation to make the perftest report public
- PR #5466[2054] - Add shift_left and shift_right algorithms
- PR #5465[2055] - Adapt datapar fill
- PR #5464[2056] - Moving tag_dispatch to separate module
- PR #5461[2057] - Rename HPX_WITH_CUDA_COMPUTE with HPX_WITH_COMPUTE_CUDA
- PR #5460[2058] - Adapt sort to C++ 20
- PR #5459[2059] - Adapt rotate/rotate_copy to C++20
- PR #5458[2060] - Adapt unique and unique_copy to C++ 20
- PR #5455[2061] - Remove and clean up fallback sender implementations
- PR #5454[2062] - Make performance plot show even if similar performance
- PR #5453[2063] - Post 1.7.0 version bump
- PR #5452[2064] - Fix find_end parallel overload
- PR #5450[2065] - Change the print-bind output to be more precise

[2043] https://github.com/TheHPXProject/hpx/pull/5484
[2044] https://github.com/TheHPXProject/hpx/pull/5481
[2045] https://github.com/TheHPXProject/hpx/pull/5479
[2046] https://github.com/TheHPXProject/hpx/pull/5477
[2047] https://github.com/TheHPXProject/hpx/pull/5476
[2048] https://github.com/TheHPXProject/hpx/pull/5475
[2049] https://github.com/TheHPXProject/hpx/pull/5474
[2050] https://github.com/TheHPXProject/hpx/pull/5473
[2051] https://github.com/TheHPXProject/hpx/pull/5471
[2052] https://github.com/TheHPXProject/hpx/pull/5470
[2053] https://github.com/TheHPXProject/hpx/pull/5468
[2054] https://github.com/TheHPXProject/hpx/pull/5466
[2055] https://github.com/TheHPXProject/hpx/pull/5465
[2056] https://github.com/TheHPXProject/hpx/pull/5464
[2057] https://github.com/TheHPXProject/hpx/pull/5461
[2058] https://github.com/TheHPXProject/hpx/pull/5460
[2059] https://github.com/TheHPXProject/hpx/pull/5459
[2060] https://github.com/TheHPXProject/hpx/pull/5458
[2061] https://github.com/TheHPXProject/hpx/pull/5455
[2062] https://github.com/TheHPXProject/hpx/pull/5454
[2063] https://github.com/TheHPXProject/hpx/pull/5453
[2064] https://github.com/TheHPXProject/hpx/pull/5452
[2065] https://github.com/TheHPXProject/hpx/pull/5450

- PR #5449[2066] - Adapt swap_ranges to C++ 20
- PR #5446[2067] - Use more verbose names in sender algorithms
- PR #5443[2068] - Properly support ASAN with MSVC
- PR #5441[2069] - Adding reference counting to thread_data
- PR #5429[2070] - Scheduler executor
- PR #5428[2071] - Adapt datapar copy
- PR #5421[2072] - Update CI base image to use clang-format 11
- PR #5410[2073] - Add ranges starts_with and ends_with algorithms
- PR #5383[2074] - Tentatively remove runtime_registration_wrapper from cuda futures
- PR #5377[2075] - Fewer Asio includes and more precompiled headers
- PR #5329[2076] - Sender overloads for parallel algorithms
- PR #5313[2077] - Rearrange modules between libraries
- PR #5283[2078] - Require minimum C++17 and change CUDA handling
- PR #5241[2079] - Adapt min_element, max_element and minmax_element to C++20

HPX V1.7.1 (Aug 12, 2021)

This is a bugfix release with a few minor fixes.

General changes

- Added a CMake option to assume that all types are bitwise serializable by default: `HPX_SERIALIZATION_WITH_ALL_TYPES_ARE_BITWISE_SERIALIZABLE`. The default value `OFF` corresponds to the old behaviour.
- Added a version check for Asio. The minimum Asio version supported by *HPX* is 1.12.0.
- Fixed a bug affecting usage of actions, where the internals of *HPX* relied on function addresses being unique. This was fixed by relying on variable addresses being unique instead.
- Made `hpx::util::bind` more strict in checking the validity of placeholders.
- Small performance improvement to spinlocks.

[2066] https://github.com/TheHPXProject/hpx/pull/5449
[2067] https://github.com/TheHPXProject/hpx/pull/5446
[2068] https://github.com/TheHPXProject/hpx/pull/5443
[2069] https://github.com/TheHPXProject/hpx/pull/5441
[2070] https://github.com/TheHPXProject/hpx/pull/5429
[2071] https://github.com/TheHPXProject/hpx/pull/5428
[2072] https://github.com/TheHPXProject/hpx/pull/5421
[2073] https://github.com/TheHPXProject/hpx/pull/5410
[2074] https://github.com/TheHPXProject/hpx/pull/5383
[2075] https://github.com/TheHPXProject/hpx/pull/5377
[2076] https://github.com/TheHPXProject/hpx/pull/5329
[2077] https://github.com/TheHPXProject/hpx/pull/5313
[2078] https://github.com/TheHPXProject/hpx/pull/5283
[2079] https://github.com/TheHPXProject/hpx/pull/5241

- Adapted the following parallel algorithms to C++20: `inclusive_scan`, `exclusive_scan`, `transform_inclusive_scan`, `transform_exclusive_scan`.

Breaking changes

- The experimental `hpx::execution::simdpar` execution policy (introduced in 1.7.0) was renamed to `hpx::execution::par_simd` for consistency with the other parallel policies.

Closed issues

- Issue #5494[2080] - Rename *simdpar* execution policy to *par_simd*
- Issue #5488[2081] - *hpx::util::bind* doesn't bounds-check placeholders
- Issue #5486[2082] - Possible V1.7.1 release

Closed pull requests

- PR #5500[2083] - Minor bug fix in transform exclusive and inclusive scan tests
- PR #5499[2084] - Rename simdpar to par_simd
- PR #5489[2085] - Adding bound-checking for bind placeholders
- PR #5485[2086] - Add Asio version check
- PR #5482[2087] - Change extra archive data to rely on uniqueness of a variable address, not a function address
- PR #5448[2088] - More fixes to enable for all types to be assumed to be bitwise copyable
- PR #5445[2089] - Improve performance of Spinlocks
- PR #5444[2090] - Adapt transform_inclusive_scan to C++ 20
- PR #5440[2091] - Adapt transform_exclusive_scan to C++ 20
- PR #5439[2092] - Adapt inclusive_scan to C++ 20
- PR #5436[2093] - Adapt exclusive_scan to C++20

[2080] https://github.com/TheHPXProject/hpx/issues/5494
[2081] https://github.com/TheHPXProject/hpx/issues/5488
[2082] https://github.com/TheHPXProject/hpx/issues/5486
[2083] https://github.com/TheHPXProject/hpx/pull/5500
[2084] https://github.com/TheHPXProject/hpx/pull/5499
[2085] https://github.com/TheHPXProject/hpx/pull/5489
[2086] https://github.com/TheHPXProject/hpx/pull/5485
[2087] https://github.com/TheHPXProject/hpx/pull/5482
[2088] https://github.com/TheHPXProject/hpx/pull/5448
[2089] https://github.com/TheHPXProject/hpx/pull/5445
[2090] https://github.com/TheHPXProject/hpx/pull/5444
[2091] https://github.com/TheHPXProject/hpx/pull/5440
[2092] https://github.com/TheHPXProject/hpx/pull/5439
[2093] https://github.com/TheHPXProject/hpx/pull/5436

HPX V1.7.0 (Jul 14, 2021)

This release is again focused on C++20 conformance of algorithms. Additionally, many new experimental sender-based algorithms have been added based on the latest proposals.

General changes

- The following algorithms have been adapted to be C++20 conformant:
  - `remove`,
  - `remove_if`,
  - `remove_copy`,
  - `remove_copy_if`,
  - `replace`,
  - `replace_if`,
  - `reverse`, and
  - `lexicographical_compare`.
- When the compiler and standard library support the standard execution policies `std::execution::seq`, `std::execution::par`, and `std::execution::par_unseq` they can now be used in all *HPX* parallel algorithms with equivalent behaviour to the non-task policies `hpx::execution::seq`, `hpx::execution::par`, and `hpx::execution::par_unseq`.
- Vc support has been fixed, after being broken in 1.6.0. In addition, *HPX* now experimentally supports GCC's SIMD implementation, when available. The implementation can be used through the `hpx::execution::simd` and `hpx::execution::simdpar` execution policies.
- The customization points `sync_execute`, `async_execute`, `then_execute`, `post`, `bulk_sync_execute`, `bulk_async_execute`, and `bulk_then_execute` are now implemented using `tag_dispatch` (previously `tag_invoke`). Executors can still be implemented by providing the aforementioned functions as member functions of an executor.
- New functionality, enhancements, and fixes based on P0443r14 (executors proposal) and P1897 (sender-based algorithms) have been added to the `hpx::execution::experimental` namespace. These can be accessed through the `hpx/execution.hpp` and `hpx/local/execution.hpp` headers. In particular, the following sender-based algorithms have been added:
  - `detach`,
  - `ensure_started`,
  - `just`,
  - `just_on`,
  - `let_error`,
  - `let_value`,
  - on,

  - `transform`, and
  - `when_all`.

  Additionally, futures now implement the sender concept. `make_future` can be used to turn a sender into a future. All functionality is experimental and can change without notice.

- All `hpx::init` and `hpx::start` overloads now take `std::functions` instead of `hpx::util::function_nonser`. No changes should be required in user code to accommodate this change.
- `hpx::util::unwrapping` and other related unwrapping functionality has been moved up into the `hpx` namespace. Names in `hpx::util` are still usable with a deprecation warning. This functionality can now be accessed through the `hpx/unwrap.hpp` and `hpx/local/unwrap.hpp` headers.
- The default tag for APEX has been update from 2.3.1 to 2.4.0. In particular, this fixes a bug which could lead to hangs in distributed runs.
- The dependency on Boost.Asio has been replaced with the standalone Asio available at https://github.com/chriskohlhoff/asio. By default, a system-installed Asio will be used. `ASIO_ROOT` can be given as a hint to tell CMake where to find Asio. Alternatively, Asio can be fetched automatically using CMake's fetchcontent by setting `HPX_WITH_FETCH_ASIO=ON`. In general, dependencies on Boost have again been reduced.
- Modularization of the library has continued. In this release almost all functionality has been moved into modules. These changes do not generally affect user code. Warnings are still issued for headers that have moved.
- hipBLAS is now optional when compiling with `hipcc`. A warning instead of an error will be printed if hipBLAS is not found during configuration.
- Previously `HPX_COMPUTE_HOST_CODE` was defined in host code only if HPX was configured with CUDA or HIP. In this release `HPX_COMPUTE_HOST_CODE` is always defined in host code.
- An experimental `HPX_WITH_PRECOMPILED_HEADERS` CMake option has been added to use precompiled headers when building *HPX*. This option should not be used on Windows.
- Numerous bug fixes.

Breaking changes

- The minimum required CMake version is now 3.17.
- The minimum required Boost version is now 1.71.0.
- The customization mechanism used to implement and extend sender functionality and algorithms has been renamed from `tag_invoke` to `tag_dispatch`. All customization of sender functionality should be done by overloading `tag_dispatch`.
- The following compatibility options have been removed, along with their compatibility implementations: - `HPX_PROGRAM_OPTIONS_WITH_BOOST_PROGRAM_OPTIONS_COMPATIBILITY` - `HPX_WITH_ACTION_BASE_COMPATIBILITY` - `HPX_WITH_EMBEDDED_THREAD_POOLS_COMPATIBILITY` - `HPX_WITH_POOL_EXECUTOR_COMPATIBILITY`. - `HPX_WITH_PROMISE_ALIAS_COMPATIBILITY` - `HPX_WITH_REGISTER_THREAD_COMPATIBILITY` -

HPX_WITH_REGISTER_THREAD_OVERLOADS_COMPATIBILITY - HPX_WITH_THREAD_AWARE_TIMER_COMPATIBILITY - HPX_WITH_THREAD_EXECUTORS_COMPATIBILITY - HPX_WITH_THREAD_POOL_OS_EXECUTOR_COMPATIBILITY

- The HPX_WITH_THREAD_SCHEDULERS CMake option has been removed. All schedulers are now enabled when possible.
- HPX_WITH_INIT_START_OVERLOADS_COMPATIBILITY has been turned off by default.

Closed issues

- Issue #5423[2094] - Fix lvalue-ref qualified connect for when_all-sender
- Issue #5412[2095] - Link error
- Issue #5397[2096] - Performance regression in thread annotations
- Issue #5395[2097] - HPX 1.7.0-rc1 fails to build icw APEX + OTF2
- Issue #5385[2098] - HPX 1.7 crashes on Piz Daint > 64 nodes
- Issue #5380[2099] - CMake should search for asio package installed on the system
- Issue #5378[2100] - HPX 1.7.0 stopped building on Fedora
- Issue #5369[2101] - HPX 1.6 and master hangs on Summit for > 64 nodes
- Issue #5358[2102] - HPX init fails for single-core environments
- Issue #5345[2103] - Rename P2220 property CPOs?
- Issue #5333[2104] - HPX does not compile on the new Mac OSX using the M1 chip
- Issue #5317[2105] - Consider making hipblas optional
- Issue #5306[2106] - asio fails to build with CUDA 10.0
- Issue #5294[2107] - execution::on should be based on execution::schedule
- Issue #5275[2108] - HPX V1.6.0 fails on Fedora release
- Issue #5270[2109] - HPX-1.6.0 fails to build on Windows 10
- Issue #5257[2110] - Allow triggering the output of OS thread affinity from configuration settings

[2094] https://github.com/TheHPXProject/hpx/issues/5423
[2095] https://github.com/TheHPXProject/hpx/issues/5412
[2096] https://github.com/TheHPXProject/hpx/issues/5397
[2097] https://github.com/TheHPXProject/hpx/issues/5395
[2098] https://github.com/TheHPXProject/hpx/issues/5385
[2099] https://github.com/TheHPXProject/hpx/issues/5380
[2100] https://github.com/TheHPXProject/hpx/issues/5378
[2101] https://github.com/TheHPXProject/hpx/issues/5369
[2102] https://github.com/TheHPXProject/hpx/issues/5358
[2103] https://github.com/TheHPXProject/hpx/issues/5345
[2104] https://github.com/TheHPXProject/hpx/issues/5333
[2105] https://github.com/TheHPXProject/hpx/issues/5317
[2106] https://github.com/TheHPXProject/hpx/issues/5306
[2107] https://github.com/TheHPXProject/hpx/issues/5294
[2108] https://github.com/TheHPXProject/hpx/issues/5275
[2109] https://github.com/TheHPXProject/hpx/issues/5270
[2110] https://github.com/TheHPXProject/hpx/issues/5257

- Issue #5246[2111] - HPX fails to build on ppc64le
- Issue #5232[2112] - Annotation using `hpx::util::annotated_function` not working
- Issue #5222[2113] - Build and link errors with ittnotify enabled
- Issue #5204[2114] - Move algorithms to tag_fallback_dispatch
- Issue #5163[2115] - Remove module-specific compatibility and deprecation options
- Issue #5161[2116] - Bump required CMake version to 3.17
- Issue #5143[2117] - Searching for HPX-Application to generate work on multiple Nodes

Closed pull requests

- PR #5438[2118] - Delete datapar/foreach_tests.hpp
- PR #5437[2119] - Add back explicit -pthread flags when available
- PR #5435[2120] - This adds support for systems that assume all types are bitwise serializable by default
- PR #5434[2121] - Update CUDA polling logging to be more verbose
- PR #5433[2122] - Fix `when_all_sender` connect for references
- PR #5432[2123] - Add deprecation warnings for v1.8
- PR #5431[2124] - Rename the new P0443/P2300 executor to `thread_pool_scheduler`
- PR #5430[2125] - Revert "Adding the missing defined for `HPX_HAVE_DEPRECATION_WARNINGS`"
- PR #5427[2126] - Removing unneeded typedef
- PR #5426[2127] - Adding more concept checks for sender/receiver algorithms
- PR #5425[2128] - Adding the missing defined for `HPX_HAVE_DEPRECATION_WARNINGS`
- PR #5424[2129] - Disable Vc in final docker image created in CI
- PR #5422[2130] - Adding `execution::experimental::bulk` algorithm
- PR #5420[2131] - Update logic to find threading library

2111 https://github.com/TheHPXProject/hpx/issues/5246
2112 https://github.com/TheHPXProject/hpx/issues/5232
2113 https://github.com/TheHPXProject/hpx/issues/5222
2114 https://github.com/TheHPXProject/hpx/issues/5204
2115 https://github.com/TheHPXProject/hpx/issues/5163
2116 https://github.com/TheHPXProject/hpx/issues/5161
2117 https://github.com/TheHPXProject/hpx/issues/5143
2118 https://github.com/TheHPXProject/hpx/pull/5438
2119 https://github.com/TheHPXProject/hpx/pull/5437
2120 https://github.com/TheHPXProject/hpx/pull/5435
2121 https://github.com/TheHPXProject/hpx/pull/5434
2122 https://github.com/TheHPXProject/hpx/pull/5433
2123 https://github.com/TheHPXProject/hpx/pull/5432
2124 https://github.com/TheHPXProject/hpx/pull/5431
2125 https://github.com/TheHPXProject/hpx/pull/5430
2126 https://github.com/TheHPXProject/hpx/pull/5427
2127 https://github.com/TheHPXProject/hpx/pull/5426
2128 https://github.com/TheHPXProject/hpx/pull/5425
2129 https://github.com/TheHPXProject/hpx/pull/5424
2130 https://github.com/TheHPXProject/hpx/pull/5422
2131 https://github.com/TheHPXProject/hpx/pull/5420

- PR #5418[2132] - Reduce max size and number of files in ccache cache
- PR #5417[2133] - Final release notes for 1.7.0
- PR #5416[2134] - Adapt `uninitialized_value_construct` and `uninitialized_value_construct_n` to C++ 20
- PR #5415[2135] - Adapt `uninitialized_default_construct` and `uninitialized_default_construct_n` to C++ 20
- PR #5414[2136] - Improve integration of futures and senders
- PR #5413[2137] - Fixing sender/receiver code base to compile with MSVC
- PR #5407[2138] - Handle exceptions thrown during initialization of parcel handler
- PR #5406[2139] - Simplify dispatching to annotation handlers
- PR #5405[2140] - Fetch Asio automatically in perftests CI
- PR #5403[2141] - Create generic executor that adds annotations to any other executor
- PR #5402[2142] - Adapt `uninitialized_fill` and `uninitialized_fill_n` to C++ 20
- PR #5401[2143] - Modernize a variety of facilities related to parallel algorithms
- PR #5400[2144] - Fix sliding semaphore test
- PR #5399[2145] - Rename leftover `tag_fallback_invoke` to `tag_fallback_dispatch`
- PR #5398[2146] - Improve logging in AGAS symbol namespace
- PR #5396[2147] - Introduce compatibility layer for collective operations
- PR #5394[2148] - Enable OTF2 in APEX CI configuration
- PR #5393[2149] - Update APEX tag
- PR #5392[2150] - Fixing wrong usage of `std::forward`
- PR #5391[2151] - Fix forwarding in transform_receiver constructor
- PR #5390[2152] - Make sure shared priority scheduler steals tasks on the current NUMA domain when (core) stealing is enabled
- PR #5389[2153] - Adapt `uninitialized_move` and `uninitialized_move_n` to C++ 20

[2132] https://github.com/TheHPXProject/hpx/pull/5418
[2133] https://github.com/TheHPXProject/hpx/pull/5417
[2134] https://github.com/TheHPXProject/hpx/pull/5416
[2135] https://github.com/TheHPXProject/hpx/pull/5415
[2136] https://github.com/TheHPXProject/hpx/pull/5414
[2137] https://github.com/TheHPXProject/hpx/pull/5413
[2138] https://github.com/TheHPXProject/hpx/pull/5407
[2139] https://github.com/TheHPXProject/hpx/pull/5406
[2140] https://github.com/TheHPXProject/hpx/pull/5405
[2141] https://github.com/TheHPXProject/hpx/pull/5403
[2142] https://github.com/TheHPXProject/hpx/pull/5402
[2143] https://github.com/TheHPXProject/hpx/pull/5401
[2144] https://github.com/TheHPXProject/hpx/pull/5400
[2145] https://github.com/TheHPXProject/hpx/pull/5399
[2146] https://github.com/TheHPXProject/hpx/pull/5398
[2147] https://github.com/TheHPXProject/hpx/pull/5396
[2148] https://github.com/TheHPXProject/hpx/pull/5394
[2149] https://github.com/TheHPXProject/hpx/pull/5393
[2150] https://github.com/TheHPXProject/hpx/pull/5392
[2151] https://github.com/TheHPXProject/hpx/pull/5391
[2152] https://github.com/TheHPXProject/hpx/pull/5390
[2153] https://github.com/TheHPXProject/hpx/pull/5389

- PR #5388[2154] - Fixing `gather_there` for used with lvalue reference argument
- PR #5387[2155] - Extend thread state logging and change default stealing parameters
- PR #5386[2156] - Attempt to fix the startup hang with nodes > 32
- PR #5384[2157] - Remove HPX 1.5.0 deprecations
- PR #5382[2158] - Prefer installed Asio before considering FetchContent
- PR #5379[2159] - Allow using pre-downloaded (not installed) versions of Asio and/or Apex
- PR #5376[2160] - Remove unnecessary explicit listing of library modules.rst files in CMakeLists.txt
- PR #5375[2161] - Slight performance improvement for `hpx::copy` and `hpx::move` et.al.
- PR #5374[2162] - Remove unnecessary moves from future sender implementations
- PR #5373[2163] - More changes to clang-cuda Jenkins configuration
- PR #5372[2164] - Slight improvements to `min/max/minmax_element` algorithms
- PR #5371[2165] - Adapt `uninitialized_copy` and `uninitialized_copy_n` to C++ 20
- PR #5370[2166] - Decay types in `just_sender value_types` to match stored types
- PR #5367[2167] - Disable pkgconfig by default again on macOS
- PR #5365[2168] - Use ccache for Jenkins builds on Piz Daint
- PR #5363[2169] - Update cudatoolkit module name in clang-cuda Jenkins configuration
- PR #5362[2170] - Adding `channel_communicator`
- PR #5361[2171] - Fix compilation with MPI enabled
- PR #5360[2172] - Update APEX and asio tags
- PR #5359[2173] - Fix check for pu-step in single-core case
- PR #5357[2174] - Making sure collective operations can be reused by preallocating communicator
- PR #5356[2175] - Update API documentation

[2154] https://github.com/TheHPXProject/hpx/pull/5388
[2155] https://github.com/TheHPXProject/hpx/pull/5387
[2156] https://github.com/TheHPXProject/hpx/pull/5386
[2157] https://github.com/TheHPXProject/hpx/pull/5384
[2158] https://github.com/TheHPXProject/hpx/pull/5382
[2159] https://github.com/TheHPXProject/hpx/pull/5379
[2160] https://github.com/TheHPXProject/hpx/pull/5376
[2161] https://github.com/TheHPXProject/hpx/pull/5375
[2162] https://github.com/TheHPXProject/hpx/pull/5374
[2163] https://github.com/TheHPXProject/hpx/pull/5373
[2164] https://github.com/TheHPXProject/hpx/pull/5372
[2165] https://github.com/TheHPXProject/hpx/pull/5371
[2166] https://github.com/TheHPXProject/hpx/pull/5370
[2167] https://github.com/TheHPXProject/hpx/pull/5367
[2168] https://github.com/TheHPXProject/hpx/pull/5365
[2169] https://github.com/TheHPXProject/hpx/pull/5363
[2170] https://github.com/TheHPXProject/hpx/pull/5362
[2171] https://github.com/TheHPXProject/hpx/pull/5361
[2172] https://github.com/TheHPXProject/hpx/pull/5360
[2173] https://github.com/TheHPXProject/hpx/pull/5359
[2174] https://github.com/TheHPXProject/hpx/pull/5357
[2175] https://github.com/TheHPXProject/hpx/pull/5356

- PR #5355[2176] - Make the `sequenced_executor` `processing_units_count` member function const
- PR #5354[2177] - Making sure `default_stack_size` is defined whenever declared
- PR #5353[2178] - Add CUDA timestamp support to HPX Hardware Clock
- PR #5352[2179] - Adding missing includes
- PR #5351[2180] - Adding `enable_logging/disable_logging` API functions
- PR #5350[2181] - Adapt lexicographical_compare to C++20
- PR #5349[2182] - Update minimum boost version needed on the docs
- PR #5348[2183] - Rename `tag_invoke` and related facilities to `tag_dispatch`
- PR #5347[2184] - Remove `make_` prefix for executor properties
- PR #5346[2185] - Remove and disable compatibility options for 1.7.0
- PR #5343[2186] - Fix timed_executor static cast conversion
- PR #5342[2187] - Refactor CUDA event polling
- PR #5341[2188] - Adding `make_with_annotation` and `get_annotation` properties
- PR #5339[2189] - Making sure `hpx::util::hardware::timestamp()` is always defined
- PR #5338[2190] - Fixing `timed_executor` specializations of customization points
- PR #5335[2191] - Make `partial_algorithm` work with any number of arguments
- PR #5334[2192] - Follow up `iter_sent` include on #5225
- PR #5332[2193] - Simplify `tag_invoke` and friends
- PR #5331[2194] - More work on cleaning up executor CPOs
- PR #5330[2195] - Add option to disable pkgconfig generation
- PR #5328[2196] - Adapt data parallel support using std-simd
- PR #5327[2197] - Fix missing `ifdef HPX_SMT_PAUSE`
- PR #5326[2198] - Adding `resize()` to `serialize_buffer` allowing to shrink its size

[2176] https://github.com/TheHPXProject/hpx/pull/5355
[2177] https://github.com/TheHPXProject/hpx/pull/5354
[2178] https://github.com/TheHPXProject/hpx/pull/5353
[2179] https://github.com/TheHPXProject/hpx/pull/5352
[2180] https://github.com/TheHPXProject/hpx/pull/5351
[2181] https://github.com/TheHPXProject/hpx/pull/5350
[2182] https://github.com/TheHPXProject/hpx/pull/5349
[2183] https://github.com/TheHPXProject/hpx/pull/5348
[2184] https://github.com/TheHPXProject/hpx/pull/5347
[2185] https://github.com/TheHPXProject/hpx/pull/5346
[2186] https://github.com/TheHPXProject/hpx/pull/5343
[2187] https://github.com/TheHPXProject/hpx/pull/5342
[2188] https://github.com/TheHPXProject/hpx/pull/5341
[2189] https://github.com/TheHPXProject/hpx/pull/5339
[2190] https://github.com/TheHPXProject/hpx/pull/5338
[2191] https://github.com/TheHPXProject/hpx/pull/5335
[2192] https://github.com/TheHPXProject/hpx/pull/5334
[2193] https://github.com/TheHPXProject/hpx/pull/5332
[2194] https://github.com/TheHPXProject/hpx/pull/5331
[2195] https://github.com/TheHPXProject/hpx/pull/5330
[2196] https://github.com/TheHPXProject/hpx/pull/5328
[2197] https://github.com/TheHPXProject/hpx/pull/5327
[2198] https://github.com/TheHPXProject/hpx/pull/5326

- PR #5324[2199] - Add get member functions to `async_rw_mutex` proxy objects for explicitly getting the wrapped value
- PR #5323[2200] - Add `keep_future` algorithm
- PR #5322[2201] - Replace executor customization point implementations with `tag_invoke`
- PR #5321[2202] - Seperate segmented algorithms for reduce
- PR #5320[2203] - Fix `is_sender` trait and other small fixes to p0443 traits
- PR #5319[2204] - gcc 11.1 c++20 build fixes
- PR #5318[2205] - Make hipblas dependency optional as not always available
- PR #5316[2206] - Attempt to fix checking for libatomic
- PR #5315[2207] - Add explicit keyword to fixture constructor
- PR #5314[2208] - Fix a race condition in async mpi affecting limiting executor
- PR #5312[2209] - Use local runtime and local headers in local-only modules and tests
- PR #5311[2210] - Add GCC 11 builder to jenkins
- PR #5310[2211] - Adding `hpx::execution::experimental::task_group`
- PR #5309[2212] - Seperate datapar
- PR #5308[2213] - Seperate segmented algorithms for `find`, `find_if`, `find_if_not`
- PR #5307[2214] - Seperate segmented algorithms for `fill` and `generate`
- PR #5304[2215] - Fix compilation of sender CPOs with nvcc
- PR #5300[2216] - Remove `PRIVATE` flag that was propagated into the `LANGUAGES`
- PR #5298[2217] - Seperate datapar
- PR #5297[2218] - Specify exact cmake and ninja versions when loading them in jenkins jobs
- PR #5295[2219] - Update clang-newest configuration to use clang 12 and Boost 1.76.0
- PR #5293[2220] - Fix Clang 11 cuda_future test bug

[2199] https://github.com/TheHPXProject/hpx/pull/5324
[2200] https://github.com/TheHPXProject/hpx/pull/5323
[2201] https://github.com/TheHPXProject/hpx/pull/5322
[2202] https://github.com/TheHPXProject/hpx/pull/5321
[2203] https://github.com/TheHPXProject/hpx/pull/5320
[2204] https://github.com/TheHPXProject/hpx/pull/5319
[2205] https://github.com/TheHPXProject/hpx/pull/5318
[2206] https://github.com/TheHPXProject/hpx/pull/5316
[2207] https://github.com/TheHPXProject/hpx/pull/5315
[2208] https://github.com/TheHPXProject/hpx/pull/5314
[2209] https://github.com/TheHPXProject/hpx/pull/5312
[2210] https://github.com/TheHPXProject/hpx/pull/5311
[2211] https://github.com/TheHPXProject/hpx/pull/5310
[2212] https://github.com/TheHPXProject/hpx/pull/5309
[2213] https://github.com/TheHPXProject/hpx/pull/5308
[2214] https://github.com/TheHPXProject/hpx/pull/5307
[2215] https://github.com/TheHPXProject/hpx/pull/5304
[2216] https://github.com/TheHPXProject/hpx/pull/5300
[2217] https://github.com/TheHPXProject/hpx/pull/5298
[2218] https://github.com/TheHPXProject/hpx/pull/5297
[2219] https://github.com/TheHPXProject/hpx/pull/5295
[2220] https://github.com/TheHPXProject/hpx/pull/5293

- PR #5292[2221] - Add `async_rw_mutex` based on senders
- PR #5291[2222] - "Fix" termination detection
- PR #5290[2223] - Fixed source file line statements in examples documentation
- PR #5289[2224] - Allow splitting of futures holding `std::tuple`
- PR #5288[2225] - Move algorithms to `tag_fallback_invoke`
- PR #5287[2226] - Move algorithms to `tag_fallback_invoke`
- PR #5285[2227] - Fix clang-format failure on master
- PR #5284[2228] - Replacing `util::function_nonser` on std::function in `hpx_init`
- PR #5282[2229] - Update Boost for daint 20.11 after update
- PR #5281[2230] - Fix Segmentation fault on `foreach_datapar_zipiter`
- PR #5280[2231] - Avoid modulo by zero in `counting_iterator` test
- PR #5279[2232] - Fix more GCC 10 deprecation warnings
- PR #5277[2233] - Small fixes and improvements to CUDA/MPI polling
- PR #5276[2234] - Fix typo in docs
- PR #5274[2235] - More P1897 algorithms
- PR #5273[2236] - Retry CDash submissions on failure
- PR #5272[2237] - Fix bogus deprecation warnings with GCC 10
- PR #5271[2238] - Correcting target ids for `symbol_namespace::iterate`
- PR #5268[2239] - Adding generic `require`, `require_concept`, and `query` properties
- PR #5267[2240] - Support annotations in `hpx::transform_reduce`
- PR #5266[2241] - Making late command line options available for local runtime
- PR #5265[2242] - Leverage `no_unique_address` for `member_pack`
- PR #5264[2243] - Adopt format in more places

[2221] https://github.com/TheHPXProject/hpx/pull/5292
[2222] https://github.com/TheHPXProject/hpx/pull/5291
[2223] https://github.com/TheHPXProject/hpx/pull/5290
[2224] https://github.com/TheHPXProject/hpx/pull/5289
[2225] https://github.com/TheHPXProject/hpx/pull/5288
[2226] https://github.com/TheHPXProject/hpx/pull/5287
[2227] https://github.com/TheHPXProject/hpx/pull/5285
[2228] https://github.com/TheHPXProject/hpx/pull/5284
[2229] https://github.com/TheHPXProject/hpx/pull/5282
[2230] https://github.com/TheHPXProject/hpx/pull/5281
[2231] https://github.com/TheHPXProject/hpx/pull/5280
[2232] https://github.com/TheHPXProject/hpx/pull/5279
[2233] https://github.com/TheHPXProject/hpx/pull/5277
[2234] https://github.com/TheHPXProject/hpx/pull/5276
[2235] https://github.com/TheHPXProject/hpx/pull/5274
[2236] https://github.com/TheHPXProject/hpx/pull/5273
[2237] https://github.com/TheHPXProject/hpx/pull/5272
[2238] https://github.com/TheHPXProject/hpx/pull/5271
[2239] https://github.com/TheHPXProject/hpx/pull/5268
[2240] https://github.com/TheHPXProject/hpx/pull/5267
[2241] https://github.com/TheHPXProject/hpx/pull/5266
[2242] https://github.com/TheHPXProject/hpx/pull/5265
[2243] https://github.com/TheHPXProject/hpx/pull/5264

- PR #5262[2244] - Install HPX in Rostam Jenkins jobs
- PR #5261[2245] - Limit Rostam Jenkins jobs to marvin partition temporarily
- PR #5260[2246] - Separate segmented algorithms for transform_reduce
- PR #5259[2247] - Making sure late command line options are recognized as configuration options
- PR #5258[2248] - Allow for HPX algorithms being invoked with std execution policies
- PR #5256[2249] - Separate segmented algorithms for transform
- PR #5255[2250] - Future/sender adapters
- PR #5254[2251] - Fixing datapar
- PR #5253[2252] - Add utility to format ranges
- PR #5252[2253] - Remove uses of Boost.Bimap
- PR #5251[2254] - Banish `<iostream>` from library headers
- PR #5250[2255] - Try fixing vc circle ci
- PR #5249[2256] - Adding missing header
- PR #5248[2257] - Use old Piz Daint modules after upgrade
- PR #5247[2258] - Significantly speedup simple `for_each`, `for_loop`, and `transform`
- PR #5245[2259] - P1897 `operator|` overloads
- PR #5244[2260] - P1897 `when_all`
- PR #5243[2261] - Make sure `HPX_DEBUG` is set based on HPX's build type, not consuming project's build type
- PR #5242[2262] - Moving last files unrelated to parcel layer to modules
- PR #5240[2263] - change namespace for `transform_loop.hpp`
- PR #5238[2264] - Make sure annotations are used in the binary transform
- PR #5237[2265] - Add P1897 `just`, `just_on`, and `on` algorithms

2244 https://github.com/TheHPXProject/hpx/pull/5262
2245 https://github.com/TheHPXProject/hpx/pull/5261
2246 https://github.com/TheHPXProject/hpx/pull/5260
2247 https://github.com/TheHPXProject/hpx/pull/5259
2248 https://github.com/TheHPXProject/hpx/pull/5258
2249 https://github.com/TheHPXProject/hpx/pull/5256
2250 https://github.com/TheHPXProject/hpx/pull/5255
2251 https://github.com/TheHPXProject/hpx/pull/5254
2252 https://github.com/TheHPXProject/hpx/pull/5253
2253 https://github.com/TheHPXProject/hpx/pull/5252
2254 https://github.com/TheHPXProject/hpx/pull/5251
2255 https://github.com/TheHPXProject/hpx/pull/5250
2256 https://github.com/TheHPXProject/hpx/pull/5249
2257 https://github.com/TheHPXProject/hpx/pull/5248
2258 https://github.com/TheHPXProject/hpx/pull/5247
2259 https://github.com/TheHPXProject/hpx/pull/5245
2260 https://github.com/TheHPXProject/hpx/pull/5244
2261 https://github.com/TheHPXProject/hpx/pull/5243
2262 https://github.com/TheHPXProject/hpx/pull/5242
2263 https://github.com/TheHPXProject/hpx/pull/5240
2264 https://github.com/TheHPXProject/hpx/pull/5238
2265 https://github.com/TheHPXProject/hpx/pull/5237

- PR #5236[2266] - Add an example demonstrating the use of the `invoke_function_action` facility
- PR #5235[2267] - Attempting to fix datapar compilation issues
- PR #5234[2268] - Fix small typo in `--hpx:local` option description
- PR #5233[2269] - Only find Boost.Iostreams if required for plugins
- PR #5231[2270] - Sort printed config options
- PR #5230[2271] - Fix C++20 replace algo adaptation misses
- PR #5229[2272] - Remove leftover Boost include from `sync_wait.hpp`
- PR #5228[2273] - Print module name only if it has custom configuration settings
- PR #5227[2274] - Update .codespell_whitelist
- PR #5226[2275] - Use new docker image in all CircleCI steps
- PR #5225[2276] - Adapt reverse to C++20
- PR #5224[2277] - Separate segmented algorithms for `none_of`, `any_of` and `all_of`
- PR #5223[2278] - Fixing build system for ittnotify
- PR #5221[2279] - Moving LCO related files to modules
- PR #5220[2280] - Seperate segmented algorithms for `count` and `count_if`
- PR #5218[2281] - Seperate segmented algorithms for `adjacent_find`
- PR #5217[2282] - Add a HIP github action
- PR #5215[2283] - Update ROCm to 4.0.1 on Rostam
- PR #5214[2284] - Fix clang-format error in sender.hpp
- PR #5213[2285] - Removing ESSENTIAL option to the doc example
- PR #5212[2286] - Seperate segmented algorithms for `for_each_n`
- PR #5211[2287] - Minor adapted algos fixes
- PR #5210[2288] - Fixing `is_invocable` deprecation warnings

[2266] https://github.com/TheHPXProject/hpx/pull/5236
[2267] https://github.com/TheHPXProject/hpx/pull/5235
[2268] https://github.com/TheHPXProject/hpx/pull/5234
[2269] https://github.com/TheHPXProject/hpx/pull/5233
[2270] https://github.com/TheHPXProject/hpx/pull/5231
[2271] https://github.com/TheHPXProject/hpx/pull/5230
[2272] https://github.com/TheHPXProject/hpx/pull/5229
[2273] https://github.com/TheHPXProject/hpx/pull/5228
[2274] https://github.com/TheHPXProject/hpx/pull/5227
[2275] https://github.com/TheHPXProject/hpx/pull/5226
[2276] https://github.com/TheHPXProject/hpx/pull/5225
[2277] https://github.com/TheHPXProject/hpx/pull/5224
[2278] https://github.com/TheHPXProject/hpx/pull/5223
[2279] https://github.com/TheHPXProject/hpx/pull/5221
[2280] https://github.com/TheHPXProject/hpx/pull/5220
[2281] https://github.com/TheHPXProject/hpx/pull/5218
[2282] https://github.com/TheHPXProject/hpx/pull/5217
[2283] https://github.com/TheHPXProject/hpx/pull/5215
[2284] https://github.com/TheHPXProject/hpx/pull/5214
[2285] https://github.com/TheHPXProject/hpx/pull/5213
[2286] https://github.com/TheHPXProject/hpx/pull/5212
[2287] https://github.com/TheHPXProject/hpx/pull/5211
[2288] https://github.com/TheHPXProject/hpx/pull/5210

- PR #5209[2289] - Moving more files into modules (actions, components, init_runtime, etc.)
- PR #5208[2290] - Add examples and explanation on when `tag_fallback/priority` are useful
- PR #5207[2291] - Always define `HPX_COMPUTE_HOST_CODE` for host code
- PR #5206[2292] - Add formatting exceptions for libhpx to create_module_skeleton.py
- PR #5205[2293] - Moving all distribution policies into modules
- PR #5203[2294] - Move copy algorithms to `tag_fallback_invoke`
- PR #5202[2295] - Make `HPX_WITH_PSEUDO_DEPENDENCIES` a cache variable
- PR #5201[2296] - Replaced `tag_invoke` with `tag_fallback_invoke` for `adjacent_find` algorithm
- PR #5200[2297] - Moving files to (distributed) runtime module
- PR #5199[2298] - Update ICC module name on Piz Daint Jenkins configuration
- PR #5198[2299] - Add doxygen documentation for thread_schedule_hint
- PR #5197[2300] - Attempt to fix compilation of context implementations with unity build enabled
- PR #5196[2301] - Re-enable component tests
- PR #5195[2302] - Moving files related to colocation logic
- PR #5194[2303] - Another attempt at fixing the Fedora 35 problem
- PR #5193[2304] - Components module
- PR #5192[2305] - Adapt `replace(_if)` to C++20
- PR #5190[2306] - Set compatibility headers by default to on
- PR #5188[2307] - Bump Boost minimum version to 1.71.0
- PR #5187[2308] - Force CMake to set the `-std=c++XX` flag
- PR #5186[2309] - Remove message to print .cu extension whenever .cu files are encountered
- PR #5185[2310] - Remove some minor unnecessary CMake options

[2289] https://github.com/TheHPXProject/hpx/pull/5209
[2290] https://github.com/TheHPXProject/hpx/pull/5208
[2291] https://github.com/TheHPXProject/hpx/pull/5207
[2292] https://github.com/TheHPXProject/hpx/pull/5206
[2293] https://github.com/TheHPXProject/hpx/pull/5205
[2294] https://github.com/TheHPXProject/hpx/pull/5203
[2295] https://github.com/TheHPXProject/hpx/pull/5202
[2296] https://github.com/TheHPXProject/hpx/pull/5201
[2297] https://github.com/TheHPXProject/hpx/pull/5200
[2298] https://github.com/TheHPXProject/hpx/pull/5199
[2299] https://github.com/TheHPXProject/hpx/pull/5198
[2300] https://github.com/TheHPXProject/hpx/pull/5197
[2301] https://github.com/TheHPXProject/hpx/pull/5196
[2302] https://github.com/TheHPXProject/hpx/pull/5195
[2303] https://github.com/TheHPXProject/hpx/pull/5194
[2304] https://github.com/TheHPXProject/hpx/pull/5193
[2305] https://github.com/TheHPXProject/hpx/pull/5192
[2306] https://github.com/TheHPXProject/hpx/pull/5190
[2307] https://github.com/TheHPXProject/hpx/pull/5188
[2308] https://github.com/TheHPXProject/hpx/pull/5187
[2309] https://github.com/TheHPXProject/hpx/pull/5186
[2310] https://github.com/TheHPXProject/hpx/pull/5185

- PR #5184[2311] - Remove some leftover `HPX_WITH_*_SCHEDULER` uses
- PR #5183[2312] - Remove dependency on boost/iterators/iterator_categories.hpp
- PR #5182[2313] - Fixing Fedora 35 for Power architectures
- PR #5181[2314] - Bump version number and tag post 1.6.0 release
- PR #5180[2315] - Fix htts_v2 tests linking
- PR #5179[2316] - Make sure `--hpx:local` command line option is respected with networking is off but distributed runtime is on
- PR #5177[2317] - Remove module cmake options
- PR #5176[2318] - Starting to separate segmented algorithms: `for_each`
- PR #5174[2319] - Don't run segmented algorithms twice on CircleCI
- PR #5173[2320] - Fetching APEX using cmake FetchContent
- PR #5172[2321] - Add separate local-only entry point
- PR #5171[2322] - Remove `HPX_WITH_THREAD_SCHEDULERS` CMake option
- PR #5170[2323] - Add `HPX_WITH_PRECOMPILED_HEADERS` option
- PR #5166[2324] - Moving some action tests to modules
- PR #5165[2325] - Require cmake 3.17
- PR #5164[2326] - Move `thread_pool_suspension_helper` files to small utility module
- PR #5160[2327] - Adding checks ensuring modules are not cross-referenced from other module categories
- PR #5158[2328] - Replace boost::asio with standalone asio
- PR #5155[2329] - Allow logging when distributed runtime is off
- PR #5153[2330] - Components module
- PR #5152[2331] - Move more files to performance counter module
- PR #5150[2332] - Adapt `remove_copy(_if)` to C++20

[2311] https://github.com/TheHPXProject/hpx/pull/5184
[2312] https://github.com/TheHPXProject/hpx/pull/5183
[2313] https://github.com/TheHPXProject/hpx/pull/5182
[2314] https://github.com/TheHPXProject/hpx/pull/5181
[2315] https://github.com/TheHPXProject/hpx/pull/5180
[2316] https://github.com/TheHPXProject/hpx/pull/5179
[2317] https://github.com/TheHPXProject/hpx/pull/5177
[2318] https://github.com/TheHPXProject/hpx/pull/5176
[2319] https://github.com/TheHPXProject/hpx/pull/5174
[2320] https://github.com/TheHPXProject/hpx/pull/5173
[2321] https://github.com/TheHPXProject/hpx/pull/5172
[2322] https://github.com/TheHPXProject/hpx/pull/5171
[2323] https://github.com/TheHPXProject/hpx/pull/5170
[2324] https://github.com/TheHPXProject/hpx/pull/5166
[2325] https://github.com/TheHPXProject/hpx/pull/5165
[2326] https://github.com/TheHPXProject/hpx/pull/5164
[2327] https://github.com/TheHPXProject/hpx/pull/5160
[2328] https://github.com/TheHPXProject/hpx/pull/5158
[2329] https://github.com/TheHPXProject/hpx/pull/5155
[2330] https://github.com/TheHPXProject/hpx/pull/5153
[2331] https://github.com/TheHPXProject/hpx/pull/5152
[2332] https://github.com/TheHPXProject/hpx/pull/5150

- PR #5144[2333] - AGAS module
- PR #5125[2334] - Adapt `remove` and `remove_if` to C++20
- PR #5117[2335] - Attempt to fix segfaults assumed to be caused by `future_data` instances going out of scope.
- PR #5099[2336] - Allow mixing debug and release builds
- PR #5092[2337] - Replace spirit.qi with x3
- PR #5053[2338] - Add P0443r14 executor and a a few P1897 algorithms
- PR #5044[2339] - Add performance test in jenkins and reports

HPX V1.6.0 (Feb 17, 2021)

General changes

This release continues the focus on C++20 conformance with multiple new algorithms adapted to be C++20 conformant and becoming customization point objects (CPOs). We have also added experimental support for HIP, allowing previous CUDA features to now be compiled with hipcc and run on AMD GPUs.

- The following algorithms have been adapted to be C++20 conformant: `adjacent_find`, `includes`, `inplace_merge`, `is_heap`, `is_heap_until`, `is_partitioned`, `is_sorted`, `is_sorted_until`, `merge`, `set_difference`, `set_intersection`, `set_symmetric_difference`, `set_union`.
- Experimental HIP support can be enabled by compiling *HPX* with `hipcc`. All CUDA functionality in *HPX* can now be used with HIP. The HIP functionality is for the time being exposed through the same API as the CUDA functionality, i.e. no changes are required in user code. The CUDA, and now HIP, functionality is in the `hpx::cuda` namespace.
- We have added `partial_sort` based on Francisco Tapia's implementation.
- `hpx::init` and `hpx::start` gained new overloads taking an `hpx::init_params` struct in 1.5.0. All overloads not taking an `hpx::init_params` are now deprecated.
- We have added an experimental `fork_join_executor`. This executor can be used for OpenMP-style fork-join parallelism, where the latency of a parallel region is important for performance.
- The `parallel_executor` now uses a hierarchical spawning scheme for bulk execution, which improves data locality and performance.
- `hpx::dataflow` can now be used with executors that inject additional parameters into the call of the user-provided function.
- We have added experimental support for properties as proposed in P2220[2340]. Currently the only supported property is the scheduling hint on `parallel_executor`.

[2333] https://github.com/TheHPXProject/hpx/pull/5144
[2334] https://github.com/TheHPXProject/hpx/pull/5125
[2335] https://github.com/TheHPXProject/hpx/pull/5117
[2336] https://github.com/TheHPXProject/hpx/pull/5099
[2337] https://github.com/TheHPXProject/hpx/pull/5092
[2338] https://github.com/TheHPXProject/hpx/pull/5053
[2339] https://github.com/TheHPXProject/hpx/pull/5044
[2340] https://wg21.link/p2220

- `hpx::util::annotated_function` can now be passed a dynamically generated `std::string`.
- In moving functionality to new namespaces, old names have been deprecated. A deprecation warning will be issued if you are using deprecated functionality, with instructions on how to correct or ignore the warning.
- We have removed all support for C and Fortran from our build system.
- We have further reduced the use of Boost types within *HPX* (`boost::system::error_code` and `boost::detail::spinlock`).
- We have enabled more warnings in our CI builds (unused variables and unused typedefs).

Breaking changes

- hpxMP support has been completely removed.
- The `verbs` parcelport has been removed.
- The following compatibility options have been disabled by default: `HPX_WITH_ACTION_BASE_COMPATIBILITY`, `HPX_WITH_REGISTER_THREAD_COMPATIBILITY`, `HPX_WITH_PROMISE_ALIAS_COMPATIBILITY`, `HPX_WITH_UNSCOPED_ENUM_COMPATIBILITY`, `HPX_PROGRAM_OPTIONS_WITH_BOOST_PROGRAM_OPTIONS_COMPATIBILITY`, `HPX_WITH_EMBEDDED_THREAD_POOLS_COMPATIBILITY`, `HPX_WITH_THREAD_POOL_OS_EXECUTOR_COMPATIBILITY`, `HPX_WITH_THREAD_EXECUTORS_COMPATIBILITY`, `HPX_THREAD_AWARE_TIMER_COMPATIBILITY`, `HPX_WITH_POOL_EXECUTOR_COMPATIBILITY`. Unless noted here, the above functionalities do not come with replacements. Unscoped enumerations have been replaced by scoped enumerations. Previously deprecated unscoped enumerations are disabled by `HPX_WITH_UNSCOPED_ENUM_COMPATIBILITY`. Newly deprecated unscoped enumerations have been given deprecation warnings and replaced by scoped enumerations. `hpx::promise` has been replaced with `hpx::distributed::promise`. `hpx::program_options` is a drop-in replacement for `boost::program_options`. `hpx::execution::parallel_executor` now has constructors which take a thread pool, covering the use case of `hpx::threads::executors::pool_executor`. A pool can be supplied with `hpx::resource::get_thread_pool`.

Closed issues

- Issue #5148[2341] - `runtime_support.hpp` does not work with newer cling
- Issue #5147[2342] - Wrong results with parallel reduce
- Issue #5129[2343] - Missing specialization for `std::hash<hpx::thread::id>`
- Issue #5126[2344] - Use `std::string` for task annotations
- Issue #5115[2345] - Don't expect hwloc to always report Cores
- Issue #5113[2346] - Handle threadmanager exceptions during startup
- Issue #5112[2347] - libatomic problems causing unexpected fails

[2341] https://github.com/TheHPXProject/hpx/issues/5148
[2342] https://github.com/TheHPXProject/hpx/issues/5147
[2343] https://github.com/TheHPXProject/hpx/issues/5129
[2344] https://github.com/TheHPXProject/hpx/issues/5126
[2345] https://github.com/TheHPXProject/hpx/issues/5115
[2346] https://github.com/TheHPXProject/hpx/issues/5113
[2347] https://github.com/TheHPXProject/hpx/issues/5112

- Issue #5089[2348] - Remove non-BSL files
- Issue #5088[2349] - Unwrapping problem
- Issue #5087[2350] - Remove hpxMP support
- Issue #5077[2351] - PAPI counters are not accessible when HPX is installed
- Issue #5075[2352] - Make the structs in all `iter_sent.hpp` lower case
- Issue #5067[2353] - Bug `string_util/split.hpp`
- Issue #5049[2354] - Change back the hipcc jenkins config to the fury partition on rostam
- Issue #5038[2355] - Not all examples link in the latest HPX master
- Issue #5035[2356] - Build with `HPX_WITH_EXAMPLES` fails
- Issue #5019[2357] - Broken help string for hpx
- Issue #5016[2358] - `hpx::parallel::fill` fails compiling
- Issue #5014[2359] - Rename all `.cc` to `.cpp` and `.hh` to `.hpp`
- Issue #4988[2360] - MPI is not finalized if running with only one locality
- Issue #4978[2361] - Change feature test macros to expand to zero/one
- Issue #4949[2362] - Crash when not enabling TCP parcelport
- Issue #4933[2363] - Improve test coverage for unused variable warnings etc.
- Issue #4878[2364] - HPX mpi async might call `MPI_FINALIZE` before app calls it
- Issue #4127[2365] - Local runtime entry-points

Closed pull requests

- PR #5178[2366] - Fix parallel `remove/remove_copy/transform` namespace references in docs
- PR #5169[2367] - Attempt to get Piz Daint jenkins setup running after maintenance
- PR #5168[2368] - Remove include of itself

2348 https://github.com/TheHPXProject/hpx/issues/5089
2349 https://github.com/TheHPXProject/hpx/issues/5088
2350 https://github.com/TheHPXProject/hpx/issues/5087
2351 https://github.com/TheHPXProject/hpx/issues/5077
2352 https://github.com/TheHPXProject/hpx/issues/5075
2353 https://github.com/TheHPXProject/hpx/issues/5067
2354 https://github.com/TheHPXProject/hpx/issues/5049
2355 https://github.com/TheHPXProject/hpx/issues/5038
2356 https://github.com/TheHPXProject/hpx/issues/5035
2357 https://github.com/TheHPXProject/hpx/issues/5019
2358 https://github.com/TheHPXProject/hpx/issues/5016
2359 https://github.com/TheHPXProject/hpx/issues/5014
2360 https://github.com/TheHPXProject/hpx/issues/4988
2361 https://github.com/TheHPXProject/hpx/issues/4978
2362 https://github.com/TheHPXProject/hpx/issues/4949
2363 https://github.com/TheHPXProject/hpx/issues/4933
2364 https://github.com/TheHPXProject/hpx/issues/4878
2365 https://github.com/TheHPXProject/hpx/issues/4127
2366 https://github.com/TheHPXProject/hpx/pull/5178
2367 https://github.com/TheHPXProject/hpx/pull/5169
2368 https://github.com/TheHPXProject/hpx/pull/5168

- PR #5167[2369] - Fixing deprecation warnings that slipped through the net
- PR #5159[2370] - Update APEX tag to 2.3.1
- PR #5154[2371] - Splitting unit tests on circleci to avoid timeouts
- PR #5151[2372] - Use C++20 on `clang-newest` Jenkins CI configuration
- PR #5149[2373] - Rename `'module'` symbols to avoid keyword conflict
- PR #5145[2374] - Adjust handling of CUDA/HIP options in CMake
- PR #5142[2375] - Store annotated_function annotations as `std::strings`
- PR #5140[2376] - Scheduler mode
- PR #5139[2377] - Fix path problem in pre-commit hook, add summary commit line
- PR #5138[2378] - Add program options variable map to resource partitioner init
- PR #5137[2379] - Remove the use of `boost::throw_exception`
- PR #5136[2380] - Make sure codespell checks run on CircleCI
- PR #5132[2381] - Fixing spelling errors
- PR #5131[2382] - Mark `counting_iterator` member functions as `HPX_HOST_DEVICE`
- PR #5130[2383] - Adding specialization for `std::hash<hpx::thread::id>`
- PR #5128[2384] - Fixing environment handling for FreeBSD
- PR #5127[2385] - Fix typo in fibonacci documentation
- PR #5123[2386] - Reduce vector sizes in partial sort benchmarks when running in debug mode
- PR #5122[2387] - Making sure exceptions during runtime initialization are correctly reported
- PR #5121[2388] - Working around hwloc limitation on certain platforms
- PR #5120[2389] - Fixing compatibility warnings in `hpx::transform` implementation
- PR #5119[2390] - Use `sequential_find` and friends from separate detail header

[2369] https://github.com/TheHPXProject/hpx/pull/5167
[2370] https://github.com/TheHPXProject/hpx/pull/5159
[2371] https://github.com/TheHPXProject/hpx/pull/5154
[2372] https://github.com/TheHPXProject/hpx/pull/5151
[2373] https://github.com/TheHPXProject/hpx/pull/5149
[2374] https://github.com/TheHPXProject/hpx/pull/5145
[2375] https://github.com/TheHPXProject/hpx/pull/5142
[2376] https://github.com/TheHPXProject/hpx/pull/5140
[2377] https://github.com/TheHPXProject/hpx/pull/5139
[2378] https://github.com/TheHPXProject/hpx/pull/5138
[2379] https://github.com/TheHPXProject/hpx/pull/5137
[2380] https://github.com/TheHPXProject/hpx/pull/5136
[2381] https://github.com/TheHPXProject/hpx/pull/5132
[2382] https://github.com/TheHPXProject/hpx/pull/5131
[2383] https://github.com/TheHPXProject/hpx/pull/5130
[2384] https://github.com/TheHPXProject/hpx/pull/5128
[2385] https://github.com/TheHPXProject/hpx/pull/5127
[2386] https://github.com/TheHPXProject/hpx/pull/5123
[2387] https://github.com/TheHPXProject/hpx/pull/5122
[2388] https://github.com/TheHPXProject/hpx/pull/5121
[2389] https://github.com/TheHPXProject/hpx/pull/5120
[2390] https://github.com/TheHPXProject/hpx/pull/5119

- PR #5116[2391] - Fix compilation with timer pool off
- PR #5114[2392] - Fix 5112 - make sure libatomic is used when needed
- PR #5109[2393] - Remove default runtime mode argument from init overload, again
- PR #5108[2394] - Refactor `iter_sent.hpp` to make structs lowercase
- PR #5107[2395] - Relax `dataflow` internals
- PR #5106[2396] - Change initialization of property CPOs to satisfy older nvcc versions
- PR #5104[2397] - Fix regeneration of two files that trigger unnecessary rebuilds
- PR #5103[2398] - Remove default runtime mode argument from start/init overloads
- PR #5102[2399] - Untie deprecated thread enums from the CMake option
- PR #5101[2400] - Update APEX tag for 1.6.0
- PR #5100[2401] - Bump minimum required Boost version to 1.66 and update CI configurations
- PR #5098[2402] - Minor fixes to public API listing
- PR #5097[2403] - Remove hpxMP support
- PR #5096[2404] - Remove fractals examples
- PR #5095[2405] - Use all AMD nodes again on rostam
- PR #5094[2406] - Attempt to remove macOS workaround for GH actions environment
- PR #5093[2407] - Remove verbs parcelport
- PR #5091[2408] - Avoid moving from lvalues
- PR #5090[2409] - Adopt C++20 `std::endian`
- PR #5085[2410] - Update daint CI to use Boost 1.75.0
- PR #5084[2411] - Disable compatibility options for 1.6.0 release
- PR #5083[2412] - Remove duplicated call to the `limiting_executor` in `future_overhead` test

[2391] https://github.com/TheHPXProject/hpx/pull/5116
[2392] https://github.com/TheHPXProject/hpx/pull/5114
[2393] https://github.com/TheHPXProject/hpx/pull/5109
[2394] https://github.com/TheHPXProject/hpx/pull/5108
[2395] https://github.com/TheHPXProject/hpx/pull/5107
[2396] https://github.com/TheHPXProject/hpx/pull/5106
[2397] https://github.com/TheHPXProject/hpx/pull/5104
[2398] https://github.com/TheHPXProject/hpx/pull/5103
[2399] https://github.com/TheHPXProject/hpx/pull/5102
[2400] https://github.com/TheHPXProject/hpx/pull/5101
[2401] https://github.com/TheHPXProject/hpx/pull/5100
[2402] https://github.com/TheHPXProject/hpx/pull/5098
[2403] https://github.com/TheHPXProject/hpx/pull/5097
[2404] https://github.com/TheHPXProject/hpx/pull/5096
[2405] https://github.com/TheHPXProject/hpx/pull/5095
[2406] https://github.com/TheHPXProject/hpx/pull/5094
[2407] https://github.com/TheHPXProject/hpx/pull/5093
[2408] https://github.com/TheHPXProject/hpx/pull/5091
[2409] https://github.com/TheHPXProject/hpx/pull/5090
[2410] https://github.com/TheHPXProject/hpx/pull/5085
[2411] https://github.com/TheHPXProject/hpx/pull/5084
[2412] https://github.com/TheHPXProject/hpx/pull/5083

- PR #5079[2413] - Add checks to make sure that MPI/CUDA polling is enabled/not disabled too early
- PR #5078[2414] - Add install lib directory to list of component search paths
- PR #5076[2415] - Fix a typo in the jenkins `clang-newest` cmake config
- PR #5074[2416] - Fixing warnings generated by MSVC
- PR #5073[2417] - Allow using noncopyable types with unwrapping
- PR #5072[2418] - Fix `is_convertible` args in `result_types`
- PR #5071[2419] - Fix unused parameters
- PR #5070[2420] - Fix unused variables warnings in hipcc
- PR #5069[2421] - Add support for sentinels to `adjacent_find`
- PR #5068[2422] - Fix string split function
- PR #5066[2423] - Adapt `search` to C++20 and Range TS
- PR #5065[2424] - Fix `hpx::range::adjacent_find` doxygen function signatures
- PR #5064[2425] - Refactor runtime configuration, command line handling, and resource partitioner
- PR #5063[2426] - Limit the device code guards to the distributed parts of the `future_overhead` bench
- PR #5061[2427] - Remove hipcc guards in examples and tests
- PR #5060[2428] - Fix deprecation warnings generated by msvc
- PR #5059[2429] - Add warning about suspending/resuming the runtime in multi-locality scenarios
- PR #5057[2430] - Fix unused variable warnings
- PR #5056[2431] - Fix `hpx::util::get`
- PR #5055[2432] - Remove hipcc guards
- PR #5054[2433] - Fix typo

[2413] https://github.com/TheHPXProject/hpx/pull/5079
[2414] https://github.com/TheHPXProject/hpx/pull/5078
[2415] https://github.com/TheHPXProject/hpx/pull/5076
[2416] https://github.com/TheHPXProject/hpx/pull/5074
[2417] https://github.com/TheHPXProject/hpx/pull/5073
[2418] https://github.com/TheHPXProject/hpx/pull/5072
[2419] https://github.com/TheHPXProject/hpx/pull/5071
[2420] https://github.com/TheHPXProject/hpx/pull/5070
[2421] https://github.com/TheHPXProject/hpx/pull/5069
[2422] https://github.com/TheHPXProject/hpx/pull/5068
[2423] https://github.com/TheHPXProject/hpx/pull/5066
[2424] https://github.com/TheHPXProject/hpx/pull/5065
[2425] https://github.com/TheHPXProject/hpx/pull/5064
[2426] https://github.com/TheHPXProject/hpx/pull/5063
[2427] https://github.com/TheHPXProject/hpx/pull/5061
[2428] https://github.com/TheHPXProject/hpx/pull/5060
[2429] https://github.com/TheHPXProject/hpx/pull/5059
[2430] https://github.com/TheHPXProject/hpx/pull/5057
[2431] https://github.com/TheHPXProject/hpx/pull/5056
[2432] https://github.com/TheHPXProject/hpx/pull/5055
[2433] https://github.com/TheHPXProject/hpx/pull/5054

- PR #5051[2434] - Adapt transform to C++20
- PR #5050[2435] - Replace old init overloads in tests and examples
- PR #5048[2436] - Limit jenkins hipcc to the reno node
- PR #5047[2437] - Limit cuda jenkins run to nodes with exclusively Nvidia GPUs
- PR #5046[2438] - Convert thread and future enums to class enums
- PR #5043[2439] - Improve `hpxrun.py` for Phylanx
- PR #5042[2440] - Add missing header to partial sort test
- PR #5041[2441] - Adding Francisco Tapia's implementation of `partial_sort`
- PR #5040[2442] - Remove generated headers left behind from a previous configuration
- PR #5039[2443] - Fix GCC 10 release builds
- PR #5037[2444] - Add `is_invocable` typedefs to top-level `hpx` namespace and public API list
- PR #5036[2445] - Deprecate `hpx::util::decay` in favor of `std::decay`
- PR #5034[2446] - Use versioned container image on CircleCI
- PR #5033[2447] - Implement P2220 properties module
- PR #5032[2448] - Do codespell comparison only on files changed from common ancestor
- PR #5031[2449] - Moving traits files to `actions_base`
- PR #5030[2450] - Add codespell version print in circleci
- PR #5029[2451] - Work around problems in GitHub actions macOS builder
- PR #5028[2452] - Moving move files to naming and naming_base
- PR #5027[2453] - Lessen constraints on certain algorithm arguments
- PR #5025[2454] - Adapt `is_sorted` and `is_sorted_until` to C++20
- PR #5024[2455] - Moving `naming_base` to full modules
- PR #5022[2456] - Remove C language from `CMakeLists.txt`

[2434] https://github.com/TheHPXProject/hpx/pull/5051
[2435] https://github.com/TheHPXProject/hpx/pull/5050
[2436] https://github.com/TheHPXProject/hpx/pull/5048
[2437] https://github.com/TheHPXProject/hpx/pull/5047
[2438] https://github.com/TheHPXProject/hpx/pull/5046
[2439] https://github.com/TheHPXProject/hpx/pull/5043
[2440] https://github.com/TheHPXProject/hpx/pull/5042
[2441] https://github.com/TheHPXProject/hpx/pull/5041
[2442] https://github.com/TheHPXProject/hpx/pull/5040
[2443] https://github.com/TheHPXProject/hpx/pull/5039
[2444] https://github.com/TheHPXProject/hpx/pull/5037
[2445] https://github.com/TheHPXProject/hpx/pull/5036
[2446] https://github.com/TheHPXProject/hpx/pull/5034
[2447] https://github.com/TheHPXProject/hpx/pull/5033
[2448] https://github.com/TheHPXProject/hpx/pull/5032
[2449] https://github.com/TheHPXProject/hpx/pull/5031
[2450] https://github.com/TheHPXProject/hpx/pull/5030
[2451] https://github.com/TheHPXProject/hpx/pull/5029
[2452] https://github.com/TheHPXProject/hpx/pull/5028
[2453] https://github.com/TheHPXProject/hpx/pull/5027
[2454] https://github.com/TheHPXProject/hpx/pull/5025
[2455] https://github.com/TheHPXProject/hpx/pull/5024
[2456] https://github.com/TheHPXProject/hpx/pull/5022

- PR #5021[2457] - Warn about unused arguments given to `add_hpx_module`
- PR #5020[2458] - Fixing help string
- PR #5018[2459] - Update CSCS jenkins configuration to clang 11
- PR #5017[2460] - Fixing broken backwards compatibility for `hpx::parallel::fill`
- PR #5015[2461] - Detect if generated global header conflicts with explicitly listed module headers
- PR #5012[2462] - Properly reset pointer tracking data in `output_archive`
- PR #5011[2463] - Inspect command line tweaks
- PR #5010[2464] - Creating AGAS module
- PR #5009[2465] - Replace `boost::system::error_code` with `std::error_code`
- PR #5008[2466] - Replace uses of `boost::detail::spinlock`
- PR #5007[2467] - Bump minimal Boost version to 1.65.0
- PR #5006[2468] - Adapt is_partitioned to C++20
- PR #5005[2469] - Making sure `reduce_by_key` compiles again
- PR #5004[2470] - Fixing template specializations that make extra archive data types unique across module boundaries
- PR #5003[2471] - Relax `dataflow` argument constraints
- PR #5001[2472] - Add <`random`> inspect check
- PR #4999[2473] - Attempt to fix MacOS Github action error
- PR #4997[2474] - Fix unused variable and typedef warnings
- PR #4996[2475] - Adapt `adjacent_find` to C++20
- PR #4995[2476] - Test all schedulers in `cross_pool_injection` test except `shared_priority_queue_scheduler`
- PR #4993[2477] - Fix deprecation warnings
- PR #4991[2478] - Avoid unnecessarily including entire modules

[2457] https://github.com/TheHPXProject/hpx/pull/5021
[2458] https://github.com/TheHPXProject/hpx/pull/5020
[2459] https://github.com/TheHPXProject/hpx/pull/5018
[2460] https://github.com/TheHPXProject/hpx/pull/5017
[2461] https://github.com/TheHPXProject/hpx/pull/5015
[2462] https://github.com/TheHPXProject/hpx/pull/5012
[2463] https://github.com/TheHPXProject/hpx/pull/5011
[2464] https://github.com/TheHPXProject/hpx/pull/5010
[2465] https://github.com/TheHPXProject/hpx/pull/5009
[2466] https://github.com/TheHPXProject/hpx/pull/5008
[2467] https://github.com/TheHPXProject/hpx/pull/5007
[2468] https://github.com/TheHPXProject/hpx/pull/5006
[2469] https://github.com/TheHPXProject/hpx/pull/5005
[2470] https://github.com/TheHPXProject/hpx/pull/5004
[2471] https://github.com/TheHPXProject/hpx/pull/5003
[2472] https://github.com/TheHPXProject/hpx/pull/5001
[2473] https://github.com/TheHPXProject/hpx/pull/4999
[2474] https://github.com/TheHPXProject/hpx/pull/4997
[2475] https://github.com/TheHPXProject/hpx/pull/4996
[2476] https://github.com/TheHPXProject/hpx/pull/4995
[2477] https://github.com/TheHPXProject/hpx/pull/4993
[2478] https://github.com/TheHPXProject/hpx/pull/4991

- PR #4990[2479] - Fixing some warnings from HPX complaining about use of obsolete types
- PR #4989[2480] - add a *destroy* trait for ParcelPort plugins
- PR #4986[2481] - Remove serialization to functional module dependency
- PR #4985[2482] - Compatibility header generation
- PR #4980[2483] - Add ranges overloads to `for_loop` (and variants)
- PR #4979[2484] - Actually enable unity builds on Jenkins
- PR #4977[2485] - Cleaning up `debug::print` functionalities
- PR #4976[2486] - Remove indirection layer in `at_index_impl`
- PR #4975[2487] - Remove indirection layer in `at_index_impl`
- PR #4973[2488] - Avoid warnings/errors for older gcc complaining about multi-line comments
- PR #4970[2489] - Making set algorithms conform to C++20
- PR #4969[2490] - Moving `is_execution_policy` and friends into namespace `hpx`
- PR #4968[2491] - Enable deprecation warnings for 1.6.0 and move `any` functionality to hpx namespace
- PR #4967[2492] - Define deprecation macros conditionally
- PR #4966[2493] - Add `clang-format` and `cmake-format` version prints
- PR #4965[2494] - Making `is_heap` and `is_heap_until` conforming to C++20
- PR #4964[2495] - Adding parallel `make_heap`
- PR #4962[2496] - Fix external timer function pointer exports
- PR #4960[2497] - Fixing folder names for module tests and examples
- PR #4959[2498] - Adding communications set
- PR #4958[2499] - Deprecate tuple and timing functionality `in hpx::util`
- PR #4957[2500] - Fixing unity build option for parcelports

[2479] https://github.com/TheHPXProject/hpx/pull/4990
[2480] https://github.com/TheHPXProject/hpx/pull/4989
[2481] https://github.com/TheHPXProject/hpx/pull/4986
[2482] https://github.com/TheHPXProject/hpx/pull/4985
[2483] https://github.com/TheHPXProject/hpx/pull/4980
[2484] https://github.com/TheHPXProject/hpx/pull/4979
[2485] https://github.com/TheHPXProject/hpx/pull/4977
[2486] https://github.com/TheHPXProject/hpx/pull/4976
[2487] https://github.com/TheHPXProject/hpx/pull/4975
[2488] https://github.com/TheHPXProject/hpx/pull/4973
[2489] https://github.com/TheHPXProject/hpx/pull/4970
[2490] https://github.com/TheHPXProject/hpx/pull/4969
[2491] https://github.com/TheHPXProject/hpx/pull/4968
[2492] https://github.com/TheHPXProject/hpx/pull/4967
[2493] https://github.com/TheHPXProject/hpx/pull/4966
[2494] https://github.com/TheHPXProject/hpx/pull/4965
[2495] https://github.com/TheHPXProject/hpx/pull/4964
[2496] https://github.com/TheHPXProject/hpx/pull/4962
[2497] https://github.com/TheHPXProject/hpx/pull/4960
[2498] https://github.com/TheHPXProject/hpx/pull/4959
[2499] https://github.com/TheHPXProject/hpx/pull/4958
[2500] https://github.com/TheHPXProject/hpx/pull/4957

- PR #4953[2501] - Fixing MSVC problems after recent restructurings
- PR #4952[2502] - Make `parallel_executor` use `thread_pool_executor` spawning mechanism
- PR #4948[2503] - Clean up old artifacts better and more aggressively on Jenkins
- PR #4947[2504] - Add HIP support for AMD GPUs
- PR #4945[2505] - Enable `HPX_WITH_UNITY_BUILD` option on one of the Jenkins configurations
- PR #4943[2506] - Move public `hpx::parallel::execution` functionality to hpx::execution
- PR #4938[2507] - Post release cleanup
- PR #4858[2508] - Extending resilience APIs to support distributed invocations
- PR #4744[2509] - Fork-join executor
- PR #4665[2510] - Implementing sender, receiver, and `operation_state` concepts in terms of P0443r13
- PR #4649[2511] - Split libhpx into multiple libraries
- PR #4642[2512] - Implementing `operation_state` concept in terms of P0443r13
- PR #4640[2513] - Implementing receiver concept in terms of P0443r13
- PR #4622[2514] - Sanitizer fixes

HPX V1.5.1 (Sep 30, 2020)

General changes

This is a patch release. It contains the following changes:

- Remove restriction on suspending runtime with multiple localities, users are now responsible for synchronizing work between localities before suspending.
- Fixes several compilation problems and warnings.
- Adds notes in the documentation explaining how to cite HPX.

2501 https://github.com/TheHPXProject/hpx/pull/4953
2502 https://github.com/TheHPXProject/hpx/pull/4952
2503 https://github.com/TheHPXProject/hpx/pull/4948
2504 https://github.com/TheHPXProject/hpx/pull/4947
2505 https://github.com/TheHPXProject/hpx/pull/4945
2506 https://github.com/TheHPXProject/hpx/pull/4943
2507 https://github.com/TheHPXProject/hpx/pull/4938
2508 https://github.com/TheHPXProject/hpx/pull/4858
2509 https://github.com/TheHPXProject/hpx/pull/4744
2510 https://github.com/TheHPXProject/hpx/pull/4665
2511 https://github.com/TheHPXProject/hpx/pull/4649
2512 https://github.com/TheHPXProject/hpx/pull/4642
2513 https://github.com/TheHPXProject/hpx/pull/4640
2514 https://github.com/TheHPXProject/hpx/pull/4622

Closed issues

- Issue #4971[2515] - Parallel sort fails to compile with C++20
- Issue #4950[2516] - Build with *HPX_WITH_PARCELPORT_ACTION_COUNTERS ON* fails
- Issue #4940[2517] - Codespell report for "HPX" (on fossies.org)
- Issue #4937[2518] - Allow suspension of runtime for multiple localities

Closed pull requests

- PR #4982[2519] - Add page about citing HPX to documentation
- PR #4981[2520] - Adding the missing include
- PR #4974[2521] - Remove leftover format export hack
- PR #4972[2522] - Removing use of get_temporary_buffer and return_temporary_buffer
- PR #4963[2523] - Renaming files to avoid warnings from the vs build system
- PR #4951[2524] - Fixing build if HPX_WITH_PARCELPORT_ACTION_COUNTERS=On
- PR #4946[2525] - Allow suspension on multiple localities
- PR #4944[2526] - Fix typos reported by fossies codespell report
- PR #4941[2527] - Adding some explanation to README about how to cite HPX
- PR #4939[2528] - Small changes

HPX V1.5.0 (Sep 02, 2020)

General changes

The main focus of this release is on APIs and C++20 conformance. We have added many new C++20 features and adapted multiple algorithms to be fully C++20 conformant. As part of the modularization we have begun specifying the public API of *HPX* in terms of headers and functionality, and aligning it more closely to the C++ standard. All non-distributed modules are now in place, along with an experimental option to completely disable distributed features in *HPX*. We have also added experimental asynchronous MPI and CUDA executors. Lastly this release introduces CMake targets for depending projects, performance improvements, and many bug fixes.

[2515] https://github.com/TheHPXProject/hpx/issues/4971
[2516] https://github.com/TheHPXProject/hpx/issues/4950
[2517] https://github.com/TheHPXProject/hpx/issues/4940
[2518] https://github.com/TheHPXProject/hpx/issues/4937
[2519] https://github.com/TheHPXProject/hpx/pull/4982
[2520] https://github.com/TheHPXProject/hpx/pull/4981
[2521] https://github.com/TheHPXProject/hpx/pull/4974
[2522] https://github.com/TheHPXProject/hpx/pull/4972
[2523] https://github.com/TheHPXProject/hpx/pull/4963
[2524] https://github.com/TheHPXProject/hpx/pull/4951
[2525] https://github.com/TheHPXProject/hpx/pull/4946
[2526] https://github.com/TheHPXProject/hpx/pull/4944
[2527] https://github.com/TheHPXProject/hpx/pull/4941
[2528] https://github.com/TheHPXProject/hpx/pull/4939

- We have added the C++20 features `hpx::jthread` and `hpx::stop_token`. `hpx::condition_variable_any` now exposes new functions supporting `hpx::stop_token`.
- We have added `hpx::stable_sort` based on Francisco Tapia's implementation.
- We have adapted existing synchronization primitives to be fully conformant C++20: `hpx::barrier`, `hpx::latch`, `hpx::counting_semaphore`, and `hpx::binary_semaphore`.
- We have started using customization point objects (CPOs) to make the corresponding algorithms fully conformant to C++20 as well as to make algorithm extension easier for the user. `all_of`/`any_of`/`none_of`, `copy`, `count`, `destroy`, `equal`, `fill`, `find`, `for_each`, `generate`, `mismatch`, `move`, `reduce`, `transform_reduce` are using those CPOs (all in namespace `hpx`). We also have adapted their corresponding `hpx::ranges` versions to be conforming to C++20 in this release.
- We have adapted support for `co_await` to C++20, in addition to `hpx::future` it now also supports `hpx::shared_future`. We have also added allocator support for futures returned by `co_return`. It is no longer in the `experimental` namespace.
- We added serialization support for `std::variant` and `std::tuple`.
- `result_of` and `is_callable` are now deprecated and replaced by `invoke_result` and `is_invocable` to conform to C++20.
- We continued with the modularization, making it easier for us to add the new experimental `HPX_WITH_DISTRIBUTED_RUNTIME` CMake option (see below) . An significant amount of headers have been deprecated. We adapted the namespaces and headers we could to be closer to the standard ones (*Public API*). Depending code should still compile, however warnings are now generated instructing to change the include statements accordingly.
- It is now possible to have a basic CUDA support including a helper function to get a future from a CUDA stream and target handling. They are available under the `hpx::cuda::experimental` namespace and they can be enabled with the `-DHPX_WITH_ASYNC_CUDA=ON` CMake option.
- We added a new `hpx::mpi::experimental` namespace for getting futures from an asynchronous MPI call and a new minimal MPI executor `hpx::mpi::experimental::executor`. These can be enabled with the `-DHPX_WITH_ASYNC_MPI=On` CMake option.
- A polymorphic executor has been implemented to reduce compile times as a function accepting executors can potentially be instantiated only once instead of multiple times with different executors. It accepts the function signature as a template argument. It needs to be constructed from any other executor. Please note, that the function signatures that can be scheduled using `then_execute`, `bulk_sync_execute`, `bulk_async_execute` and `bulk_then_execute` are slightly different (See the comment in PR #4514[2529] for more details).
- The underlying executor of `block_executor` has been updated to a newer one.
- We have added a parameter to `auto_chunk_size` to control the amount of iterations to measure.
- All executor parameter hooks can now be exposed through the executor itself. This will allow to deprecate the `.with()` functionality on execution policies in the future. This is

[2529] https://github.com/TheHPXProject/hpx/pull/4514

also a first step towards simplifying our executor APIs in preparation for the upcoming C++23 executors (senders/receivers).

- We have moved all of the existing APIs related to resiliency into the namespace `hpx::resiliency::experimental`. Please note this is a breaking change without backwards-compatibility option. We have converted all of those APIs to be based on customization point objects. Two new executors have been added to enable easy integration of the existing resiliency features with other facilities (like the parallel algorithms): `replay_executor` and `replicate_executor`.
- We have added performance counters type information (`aggregating`, `monotonically increasing`, `average count`, `average timer`, etc.).
- HPX threads are now re-scheduled on the same worker thread they were suspended on to avoid cache misses from moving from one thread to the other. This behavior doesn't prevent the thread from being stolen, however.
- We have added a new configuration option `hpx.exception_verbosity` to allow to control the level of verbosity of the exceptions (3 levels available).
- `broadcast_to`, `broadcast_from`, `scatter_to` and `scatter_from` have been added to the collectives, modernization of `gather_here` and `gather_there` with futures taken by rvalue references. See the breaking change on `all_to_all` in the next section. None of the collectives need supporting macros anymore (e.g. specifying the data types used for a collective operation using `HPX_REGISTER_ALLGATHER` and similar is not needed anymore).
- New API functions have been added: a) to get the number of cores which are idle (`hpx::get_idle_core_count`) and b) returning a bitmask representing the currently idle cores (`hpx::get_idle_core_mask`).
- We have added an experimental option to only enable the local runtime, you can disable the distributed runtime with `HPX_WITH_DISTRIBUTED_RUNTIME=OFF`. You can also enable the local runtime by using the `--hpx:local` runtime option.
- We fixed task annotations for actions.
- The alias `hpx::promise` to `hpx::lcos::promise` is now deprecated. You can use `hpx::lcos::promise` directly instead. `hpx::promise` will refer to the local-only promise in the future.
- We have added a `prepare_checkpoint` API function that calculates the amount of necessary buffer space for a particular set of arguments checkpointed.
- We have added `hpx::upgrade_lock` and `hpx::upgrade_to_unique_lock`, which make `hpx::shared_mutex` (and similar) usable in more flexible ways.
- We have changed the CMake targets exposed to the user, it now includes `HPX::hpx`, `HPX::wrap_main` (`int main` as the first *HPX* thread of the application, see *Starting the HPX runtime*), `HPX::plugin`, `HPX::component`. The CMake variables `HPX_INCLUDE_DIRS` and `HPX_LIBRARIES` are deprecated and will be removed in a future release, you should now link directly to the `HPX::hpx` CMake target.
- A new example is demonstrating how to create and use a wrapping executor (`quickstart/executor_with_thread_hooks.cpp`)
- A new example is demonstrating how to disable thread stealing during the execution of parallel algorithms (`quickstart/disable_thread_stealing_executor.cpp`)
- We now require for our CMake build system configuration files to be formatted using cmake-format.

- We have removed more dependencies on various Boost libraries.
- We have added an experimental option enabling unity builds of HPX using the `-DHPX_WITH_UNITY_BUILD=On` CMake option.
- Many bug fixes.

Breaking changes

- *HPX* now requires a C++14 capable compiler. We have set the *HPX* C++ standard automatically to C++14 and if it needs to be set explicitly, it should be specified through the `CMAKE_CXX_STANDARD` setting as mandated by CMake. The `HPX_WITH_CXX*` variables are now deprecated and will be removed in the future.
- Building and using HPX is now supported only when using CMake V3.13 or later, Boost V1.64 or newer, and when compiling with clang V5, gcc V7, or VS2019, or later. Other compilers might still work but have not been tested thoroughly.
- We have added a `hpx::init_params` struct to pass parameters for *HPX* initialization e.g. the resource partitioner callback to initialize thread pools (*Using the resource partitioner*).
- The `all_to_all` algorithm is renamed to `all_gather`, and the new `all_to_all` algorithm is not compatible with the old one.
- We have moved all of the existing APIs related to resiliency into the namespace `hpx::resiliency::experimental`.

Closed issues

- Issue #4918[2530] - Rename distributed_executors module
- Issue #4900[2531] - Adding JOSS status badge to README
- Issue #4897[2532] - Compiler warning, deprecated header used by HPX itself
- Issue #4886[2533] - A future bound to an action executing on a different locality doesn't capture exception state
- Issue #4880[2534] - Undefined reference to main build error when HPX_WITH_DYNAMIC_HPX_MAIN=OFF
- Issue #4877[2535] - hpx_main might not able to start hpx runtime properly
- Issue #4850[2536] - Issues creating templated component
- Issue #4829[2537] - Spack package & HPX_WITH_GENERIC_CONTEXT_COROUTINES
- Issue #4820[2538] - PAPI counters don't work
- Issue #4818[2539] - HPX can't be used with IO pool turned off

[2530] https://github.com/TheHPXProject/hpx/issues/4918
[2531] https://github.com/TheHPXProject/hpx/issues/4900
[2532] https://github.com/TheHPXProject/hpx/issues/4897
[2533] https://github.com/TheHPXProject/hpx/issues/4886
[2534] https://github.com/TheHPXProject/hpx/issues/4880
[2535] https://github.com/TheHPXProject/hpx/issues/4877
[2536] https://github.com/TheHPXProject/hpx/issues/4850
[2537] https://github.com/TheHPXProject/hpx/issues/4829
[2538] https://github.com/TheHPXProject/hpx/issues/4820
[2539] https://github.com/TheHPXProject/hpx/issues/4818

- Issue #4816[2540] - Build of HPX fails when find_package(Boost) is called before FetchContent_MakeAvailable(hpx)
- Issue #4813[2541] - HPX MPI Future failed
- Issue #4811[2542] - Remove HPX::hpx_no_wrap_main target before 1.5.0 release
- Issue #4810[2543] - In hpx::for_each::invoke_projected the hpx::util::decay is misguided
- Issue #4787[2544] - *transform_inclusive_scan* gives incorrect results for non-commutative operator
- Issue #4786[2545] - transform_inclusive_scan tries to implicitly convert between types, instead of using the provided *conv* function
- Issue #4779[2546] - HPX build error with GCC 10.1
- Issue #4766[2547] - Move HPX.Compute functionality to experimental namespace
- Issue #4763[2548] - License file name
- Issue #4758[2549] - CMake profiling results
- Issue #4755[2550] - Building HPX with support for PAPI fails
- Issue #4754[2551] - CMake cache creation breaks when using HPX with mimalloc
- Issue #4752[2552] - HPX MPI Future build failed
- Issue #4746[2553] - Memory leak when using dataflow icw components
- Issue #4731[2554] - Bug in stencil example, calculation of locality IDs
- Issue #4723[2555] - Build fail with NETWORKING OFF
- Issue #4720[2556] - Add compatibility headers for modules that had their module headers implicitly generated in 1.4.1
- Issue #4719[2557] - Undeprecate some module headers
- Issue #4712[2558] - Rename HPX_MPI_WITH_FUTURES option
- Issue #4709[2559] - Make deprecation warnings overridable in dependent projects
- Issue #4691[2560] - Suggestion to fix and enhance the thread_mapper API

[2540] https://github.com/TheHPXProject/hpx/issues/4816
[2541] https://github.com/TheHPXProject/hpx/issues/4813
[2542] https://github.com/TheHPXProject/hpx/issues/4811
[2543] https://github.com/TheHPXProject/hpx/issues/4810
[2544] https://github.com/TheHPXProject/hpx/issues/4787
[2545] https://github.com/TheHPXProject/hpx/issues/4786
[2546] https://github.com/TheHPXProject/hpx/issues/4779
[2547] https://github.com/TheHPXProject/hpx/issues/4766
[2548] https://github.com/TheHPXProject/hpx/issues/4763
[2549] https://github.com/TheHPXProject/hpx/issues/4758
[2550] https://github.com/TheHPXProject/hpx/issues/4755
[2551] https://github.com/TheHPXProject/hpx/issues/4754
[2552] https://github.com/TheHPXProject/hpx/issues/4752
[2553] https://github.com/TheHPXProject/hpx/issues/4746
[2554] https://github.com/TheHPXProject/hpx/issues/4731
[2555] https://github.com/TheHPXProject/hpx/issues/4723
[2556] https://github.com/TheHPXProject/hpx/issues/4720
[2557] https://github.com/TheHPXProject/hpx/issues/4719
[2558] https://github.com/TheHPXProject/hpx/issues/4712
[2559] https://github.com/TheHPXProject/hpx/issues/4709
[2560] https://github.com/TheHPXProject/hpx/issues/4691

- Issue #4686[2561] - Fix tutorials examples
- Issue #4685[2562] - HPX distributed map fails to compile
- Issue #4680[2563] - Build error with HPX_WITH_DYNAMIC_HPX_MAIN=OFF
- Issue #4679[2564] - Build error for hpx w/ Apex on Summit
- Issue #4675[2565] - build error with HPX_WITH_NETWORKING=OFF
- Issue #4674[2566] - Error running Quickstart tests on OS X
- Issue #4662[2567] - MPI initialization broken when networking off
- Issue #4652[2568] - How to fix distributed action annotation
- Issue #4650[2569] - thread descriptions are broken…again
- Issue #4648[2570] - Thread stacksize not properly set
- Issue #4647[2571] - Rename generated collective headers in modules
- Issue #4639[2572] - Update deprecation warnings in compatibility headers to point to collective headers
- Issue #4628[2573] - mpi parcelport totally broken
- Issue #4619[2574] - Fully document hpx_wrap behaviour and targets
- Issue #4612[2575] - Compilation issue with HPX 1.4.1 and 1.4.0
- Issue #4594[2576] - Rename modules
- Issue #4578[2577] - Default value for HPX_WITH_THREAD_BACKTRACE_DEPTH
- Issue #4572[2578] - Thread manager should be given a runtime_configuration
- Issue #4571[2579] - Add high-level documentation to new modules
- Issue #4569[2580] - Annoying warning when compiling - pls suppress or fix it.
- Issue #4555[2581] - HPX_HAVE_THREAD_BACKTRACE_ON_SUSPENSION compilation error
- Issue #4543[2582] - Segfaults in Release builds using *sleep_for*

[2561] https://github.com/TheHPXProject/hpx/issues/4686
[2562] https://github.com/TheHPXProject/hpx/issues/4685
[2563] https://github.com/TheHPXProject/hpx/issues/4680
[2564] https://github.com/TheHPXProject/hpx/issues/4679
[2565] https://github.com/TheHPXProject/hpx/issues/4675
[2566] https://github.com/TheHPXProject/hpx/issues/4674
[2567] https://github.com/TheHPXProject/hpx/issues/4662
[2568] https://github.com/TheHPXProject/hpx/issues/4652
[2569] https://github.com/TheHPXProject/hpx/issues/4650
[2570] https://github.com/TheHPXProject/hpx/issues/4648
[2571] https://github.com/TheHPXProject/hpx/issues/4647
[2572] https://github.com/TheHPXProject/hpx/issues/4639
[2573] https://github.com/TheHPXProject/hpx/issues/4628
[2574] https://github.com/TheHPXProject/hpx/issues/4619
[2575] https://github.com/TheHPXProject/hpx/issues/4612
[2576] https://github.com/TheHPXProject/hpx/issues/4594
[2577] https://github.com/TheHPXProject/hpx/issues/4578
[2578] https://github.com/TheHPXProject/hpx/issues/4572
[2579] https://github.com/TheHPXProject/hpx/issues/4571
[2580] https://github.com/TheHPXProject/hpx/issues/4569
[2581] https://github.com/TheHPXProject/hpx/issues/4555
[2582] https://github.com/TheHPXProject/hpx/issues/4543

- Issue #4539[2583] - Compilation Error when HPX_MPI_WITH_FUTURES=ON
- Issue #4537[2584] - Linking issue with libhpx_initd.a
- Issue #4535[2585] - API for checking if pool with a given name exists
- Issue #4523[2586] - Build of PR #4311 (git tag 9955e8e) fails
- Issue #4519[2587] - Documentation problem
- Issue #4513[2588] - HPXConfig.cmake contains ill-formed paths when library paths use backslashes
- Issue #4507[2589] - User-polling introduced by MPI futures module should be more generally usable
- Issue #4506[2590] - Make sure force_linking.hpp is not included in main module header
- Issue #4501[2591] - Fix compilation of PAPI tests
- Issue #4497[2592] - Add modules CI checks
- Issue #4489[2593] - Polymorphic executor
- Issue #4476[2594] - Use CMake targets defined by FindBoost
- Issue #4473[2595] - Add vcpkg installation instructions
- Issue #4470[2596] - Adapt hpx::future to C++20 co_await
- Issue #4468[2597] - Compile error on Raspberry Pi 4
- Issue #4466[2598] - Compile error on Windows, current stable:
- Issue #4453[2599] - Installing HPX on fedora with dnf is not adding cmake files
- Issue #4448[2600] - New std::variant serialization broken
- Issue #4438[2601] - Add performance counter flag is monotically increasing
- Issue #4436[2602] - Build problem: same code build and works with 1.4.0 but it doesn't with 1.4.1
- Issue #4429[2603] - Function descriptions not supported in distributed
- Issue #4423[2604] - –hpx:ini=hpx.lock_detection=0 has no effect

[2583] https://github.com/TheHPXProject/hpx/issues/4539
[2584] https://github.com/TheHPXProject/hpx/issues/4537
[2585] https://github.com/TheHPXProject/hpx/issues/4535
[2586] https://github.com/TheHPXProject/hpx/issues/4523
[2587] https://github.com/TheHPXProject/hpx/issues/4519
[2588] https://github.com/TheHPXProject/hpx/issues/4513
[2589] https://github.com/TheHPXProject/hpx/issues/4507
[2590] https://github.com/TheHPXProject/hpx/issues/4506
[2591] https://github.com/TheHPXProject/hpx/issues/4501
[2592] https://github.com/TheHPXProject/hpx/issues/4497
[2593] https://github.com/TheHPXProject/hpx/issues/4489
[2594] https://github.com/TheHPXProject/hpx/issues/4476
[2595] https://github.com/TheHPXProject/hpx/issues/4473
[2596] https://github.com/TheHPXProject/hpx/issues/4470
[2597] https://github.com/TheHPXProject/hpx/issues/4468
[2598] https://github.com/TheHPXProject/hpx/issues/4466
[2599] https://github.com/TheHPXProject/hpx/issues/4453
[2600] https://github.com/TheHPXProject/hpx/issues/4448
[2601] https://github.com/TheHPXProject/hpx/issues/4438
[2602] https://github.com/TheHPXProject/hpx/issues/4436
[2603] https://github.com/TheHPXProject/hpx/issues/4429
[2604] https://github.com/TheHPXProject/hpx/issues/4423

- Issue #4422[2605] - Add performance counter metadata
- Issue #4419[2606] - Weird behavior for –hpx:print-counter-interval with large numbers
- Issue #4401[2607] - Create module repository
- Issue #4400[2608] - Command line options conflict related to performance counters
- Issue #4349[2609] - *–hpx:use-process-mask* option throw an exception on OS X
- Issue #4345[2610] - Move gh-pages branch out of hpx repo
- Issue #4323[2611] - Const-correctness error in assignment operator of compute::vector
- Issue #4318[2612] - ASIO breaks with C++2a concepts
- Issue #4317[2613] - Application runs even if *–hpx:help* is specified
- Issue #4063[2614] - Document hpxcxx compiler wrapper
- Issue #3983[2615] - Implement the C++20 Synchronization Library
- Issue #3696[2616] - C++11 *constexpr* support is now required
- Issue #3623[2617] - Modular HPX branch and an alternative project layout
- Issue #2836[2618] - The worst-case time complexity of parallel::sort seems to be O(N^2).

Closed pull requests

- PR #4936[2619] - Minor documentation fixes part 2
- PR #4935[2620] - Add copyright and license to joss paper file
- PR #4934[2621] - Adding Semicolon in Documentation
- PR #4932[2622] - Fixing compiler warnings
- PR #4931[2623] - Small documentation formatting fixes
- PR #4930[2624] - Documentation Distributed HPX applications localvv with local_vv
- PR #4929[2625] - Add final version of the JOSS paper
- PR #4928[2626] - Add HPX_NODISCARD to enable_user_polling structs

[2605] https://github.com/TheHPXProject/hpx/issues/4422
[2606] https://github.com/TheHPXProject/hpx/issues/4419
[2607] https://github.com/TheHPXProject/hpx/issues/4401
[2608] https://github.com/TheHPXProject/hpx/issues/4400
[2609] https://github.com/TheHPXProject/hpx/issues/4349
[2610] https://github.com/TheHPXProject/hpx/issues/4345
[2611] https://github.com/TheHPXProject/hpx/issues/4323
[2612] https://github.com/TheHPXProject/hpx/issues/4318
[2613] https://github.com/TheHPXProject/hpx/issues/4317
[2614] https://github.com/TheHPXProject/hpx/issues/4063
[2615] https://github.com/TheHPXProject/hpx/issues/3983
[2616] https://github.com/TheHPXProject/hpx/issues/3696
[2617] https://github.com/TheHPXProject/hpx/issues/3623
[2618] https://github.com/TheHPXProject/hpx/issues/2836
[2619] https://github.com/TheHPXProject/hpx/pull/4936
[2620] https://github.com/TheHPXProject/hpx/pull/4935
[2621] https://github.com/TheHPXProject/hpx/pull/4934
[2622] https://github.com/TheHPXProject/hpx/pull/4932
[2623] https://github.com/TheHPXProject/hpx/pull/4931
[2624] https://github.com/TheHPXProject/hpx/pull/4930
[2625] https://github.com/TheHPXProject/hpx/pull/4929
[2626] https://github.com/TheHPXProject/hpx/pull/4928

- PR #4926[2627] - Rename distributed_executors module to executors_distributed
- PR #4925[2628] - Making transform_reduce conforming to C++20
- PR #4923[2629] - Don't acquire lock if not needed
- PR #4921[2630] - Update the release notes for the release candidate 3
- PR #4920[2631] - Disable libcds release
- PR #4919[2632] - Make cuda event pool dynamic instead of fixed size
- PR #4917[2633] - Move chrono functionality to hpx::chrono namespace
- PR #4916[2634] - HPX_HAVE_DEPRECATION_WARNINGS needs to be set even when disabled
- PR #4915[2635] - Moving more action related files to actions modules
- PR #4914[2636] - Add alias targets with namespaces used for exporting
- PR #4912[2637] - Aggregate initialize CPOs
- PR #4910[2638] - Explicitly specify hwloc root on Jenkins CSCS builds
- PR #4908[2639] - Fix algorithms documentation
- PR #4907[2640] - Remove HPX::hpx_no_wrap_main target
- PR #4906[2641] - Fixing unused variable warning
- PR #4905[2642] - Adding specializations for simple for_loops
- PR #4904[2643] - Update boost to 1.74.0 for the newest jenkins configs
- PR #4903[2644] - Hide GITHUB_TOKEN environment variables from environment variable output
- PR #4902[2645] - Cancel previous pull requests builds before starting a new one with Jenkins
- PR #4901[2646] - Update public API list with updated algorithms
- PR #4899[2647] - Suggested changes for HPX V1.5 release notes
- PR #4898[2648] - Minor tweak to hpx::equal implementation

[2627] https://github.com/TheHPXProject/hpx/pull/4926
[2628] https://github.com/TheHPXProject/hpx/pull/4925
[2629] https://github.com/TheHPXProject/hpx/pull/4923
[2630] https://github.com/TheHPXProject/hpx/pull/4921
[2631] https://github.com/TheHPXProject/hpx/pull/4920
[2632] https://github.com/TheHPXProject/hpx/pull/4919
[2633] https://github.com/TheHPXProject/hpx/pull/4917
[2634] https://github.com/TheHPXProject/hpx/pull/4916
[2635] https://github.com/TheHPXProject/hpx/pull/4915
[2636] https://github.com/TheHPXProject/hpx/pull/4914
[2637] https://github.com/TheHPXProject/hpx/pull/4912
[2638] https://github.com/TheHPXProject/hpx/pull/4910
[2639] https://github.com/TheHPXProject/hpx/pull/4908
[2640] https://github.com/TheHPXProject/hpx/pull/4907
[2641] https://github.com/TheHPXProject/hpx/pull/4906
[2642] https://github.com/TheHPXProject/hpx/pull/4905
[2643] https://github.com/TheHPXProject/hpx/pull/4904
[2644] https://github.com/TheHPXProject/hpx/pull/4903
[2645] https://github.com/TheHPXProject/hpx/pull/4902
[2646] https://github.com/TheHPXProject/hpx/pull/4901
[2647] https://github.com/TheHPXProject/hpx/pull/4899
[2648] https://github.com/TheHPXProject/hpx/pull/4898

- PR #4896[2649] - Making generate() and generate_n conforming to C++20
- PR #4895[2650] - Update apex tag
- PR #4894[2651] - Fix exception handling for tasks
- PR #4893[2652] - Remove last use of std::result_of, removed in C++20
- PR #4892[2653] - Adding replay_executor and replicate_executor
- PR #4889[2654] - Restore old behaviour of not requiring linking to hpx_wrap when HPX_WITH_DYNAMIC_HPX_MAIN=OFF
- PR #4887[2655] - Making sure remotely thrown (non-hpx) exceptions are properly marshaled back to invocation site
- PR #4885[2656] - Adapting hpx::find and friends to C++20
- PR #4884[2657] - Adapting mismatch to C++20
- PR #4883[2658] - Adapting hpx::equal to be conforming to C++20
- PR #4882[2659] - Fixing exception handling for hpx::copy and adding missing tests
- PR #4881[2660] - Adds different runtime exception when registering thread with the HPX runtime
- PR #4876[2661] - Adding example demonstrating how to disable thread stealing during the execution of parallel algorithms
- PR #4874[2662] - Adding non-policy tests to all_of, any_of, and none_of
- PR #4873[2663] - Set CUDA compute capability on rostam Jenkins builds
- PR #4872[2664] - Force partitioned vector scan tests to run serially
- PR #4870[2665] - Making move conforming with C++20
- PR #4869[2666] - Making destroy and destroy_n conforming to C++20
- PR #4868[2667] - Fix miscellaneous header problems
- PR #4867[2668] - Add CPOs for for_each
- PR #4865[2669] - Adapting count and count_if to be conforming to C++20

[2649] https://github.com/TheHPXProject/hpx/pull/4896
[2650] https://github.com/TheHPXProject/hpx/pull/4895
[2651] https://github.com/TheHPXProject/hpx/pull/4894
[2652] https://github.com/TheHPXProject/hpx/pull/4893
[2653] https://github.com/TheHPXProject/hpx/pull/4892
[2654] https://github.com/TheHPXProject/hpx/pull/4889
[2655] https://github.com/TheHPXProject/hpx/pull/4887
[2656] https://github.com/TheHPXProject/hpx/pull/4885
[2657] https://github.com/TheHPXProject/hpx/pull/4884
[2658] https://github.com/TheHPXProject/hpx/pull/4883
[2659] https://github.com/TheHPXProject/hpx/pull/4882
[2660] https://github.com/TheHPXProject/hpx/pull/4881
[2661] https://github.com/TheHPXProject/hpx/pull/4876
[2662] https://github.com/TheHPXProject/hpx/pull/4874
[2663] https://github.com/TheHPXProject/hpx/pull/4873
[2664] https://github.com/TheHPXProject/hpx/pull/4872
[2665] https://github.com/TheHPXProject/hpx/pull/4870
[2666] https://github.com/TheHPXProject/hpx/pull/4869
[2667] https://github.com/TheHPXProject/hpx/pull/4868
[2668] https://github.com/TheHPXProject/hpx/pull/4867
[2669] https://github.com/TheHPXProject/hpx/pull/4865

- PR #4864[2670] - Release notes 1.5.0
- PR #4863[2671] - adding libcds-hpx tag to prepare for hpx1.5 release
- PR #4862[2672] - Adding version specific deprecation options
- PR #4861[2673] - Limiting executor improvements
- PR #4860[2674] - Making fill and fill_n compatible with C++20
- PR #4859[2675] - Adapting all_of, any_of, and none_of to C++20
- PR #4857[2676] - Improve libCDS integration
- PR #4856[2677] - Correct typos in the documentation of the hpx performance counters
- PR #4854[2678] - Removing obsolete code
- PR #4853[2679] - Adding test that derives component from two other components
- PR #4852[2680] - Fix mpi_ring test in distributed mode by ensuring all ranks run hpx_main
- PR #4851[2681] - Converting resiliency APIs to tag_invoke based CPOs
- PR #4849[2682] - Enable use of future_overhead test when DISTRIBUTED_RUNTIME is OFF
- PR #4847[2683] - Fixing 'error prone' constructs as reported by Codacy
- PR #4846[2684] - Disable Boost.Asio concepts support
- PR #4845[2685] - Fix PAPI counters
- PR #4843[2686] - Remove dependency on various Boost headers
- PR #4841[2687] - Rearrange public API headers
- PR #4840[2688] - Fixing TSS problems during thread termination
- PR #4839[2689] - Fix async_cuda build problems when distributed runtime is disabled
- PR #4837[2690] - Restore compatibility for old (now deprecated) copy algorithms
- PR #4836[2691] - Adding CPOs for hpx::reduce
- PR #4835[2692] - Remove *using util::result_of* from namespace hpx

[2670] https://github.com/TheHPXProject/hpx/pull/4864
[2671] https://github.com/TheHPXProject/hpx/pull/4863
[2672] https://github.com/TheHPXProject/hpx/pull/4862
[2673] https://github.com/TheHPXProject/hpx/pull/4861
[2674] https://github.com/TheHPXProject/hpx/pull/4860
[2675] https://github.com/TheHPXProject/hpx/pull/4859
[2676] https://github.com/TheHPXProject/hpx/pull/4857
[2677] https://github.com/TheHPXProject/hpx/pull/4856
[2678] https://github.com/TheHPXProject/hpx/pull/4854
[2679] https://github.com/TheHPXProject/hpx/pull/4853
[2680] https://github.com/TheHPXProject/hpx/pull/4852
[2681] https://github.com/TheHPXProject/hpx/pull/4851
[2682] https://github.com/TheHPXProject/hpx/pull/4849
[2683] https://github.com/TheHPXProject/hpx/pull/4847
[2684] https://github.com/TheHPXProject/hpx/pull/4846
[2685] https://github.com/TheHPXProject/hpx/pull/4845
[2686] https://github.com/TheHPXProject/hpx/pull/4843
[2687] https://github.com/TheHPXProject/hpx/pull/4841
[2688] https://github.com/TheHPXProject/hpx/pull/4840
[2689] https://github.com/TheHPXProject/hpx/pull/4839
[2690] https://github.com/TheHPXProject/hpx/pull/4837
[2691] https://github.com/TheHPXProject/hpx/pull/4836
[2692] https://github.com/TheHPXProject/hpx/pull/4835

- PR #4834[2693] - Fixing the calculation of the number of idle cores and the corresponding idle masks
- PR #4833[2694] - Allow thread function destructors to yield
- PR #4832[2695] - Fixing assertion in split_gids and memory leaks in 1d_stencil_7
- PR #4831[2696] - Making sure MPI_CXX_COMPILE_FLAGS is interpreted as a sequence of options
- PR #4830[2697] - Update documentation on using HPX::wrap_main
- PR #4827[2698] - Update clang-newest configuration to use clang 10
- PR #4826[2699] - Add Jenkins configuration for rostam
- PR #4825[2700] - Move all CUDA functionality to hpx::cuda::experimental namespace
- PR #4824[2701] - Add support for building master/release branches to Jenkins configuration
- PR #4821[2702] - Implement customization point for hpx::copy and hpx::ranges::copy
- PR #4819[2703] - Allow finding Boost components before finding HPX
- PR #4817[2704] - Adding range version of stable sort
- PR #4815[2705] - Fix a wrong #ifdef for IO/TIMER pools causing build errors
- PR #4814[2706] - Replace hpx::function_nonser with std::function in error module
- PR #4809[2707] - Foreach adapt
- PR #4808[2708] - Make internal algorithms functions const
- PR #4807[2709] - Add Jenkins configuration for running on Piz Daint
- PR #4806[2710] - Update documentation links to new domain name
- PR #4805[2711] - Applying changes that resolve time complexity issues in sort
- PR #4803[2712] - Adding implementation of stable_sort
- PR #4802[2713] - Fix datapar header paths
- PR #4801[2714] - Replace boost::shared_array<T> with std::shared_ptr<T[]> if supported

[2693] https://github.com/TheHPXProject/hpx/pull/4834
[2694] https://github.com/TheHPXProject/hpx/pull/4833
[2695] https://github.com/TheHPXProject/hpx/pull/4832
[2696] https://github.com/TheHPXProject/hpx/pull/4831
[2697] https://github.com/TheHPXProject/hpx/pull/4830
[2698] https://github.com/TheHPXProject/hpx/pull/4827
[2699] https://github.com/TheHPXProject/hpx/pull/4826
[2700] https://github.com/TheHPXProject/hpx/pull/4825
[2701] https://github.com/TheHPXProject/hpx/pull/4824
[2702] https://github.com/TheHPXProject/hpx/pull/4821
[2703] https://github.com/TheHPXProject/hpx/pull/4819
[2704] https://github.com/TheHPXProject/hpx/pull/4817
[2705] https://github.com/TheHPXProject/hpx/pull/4815
[2706] https://github.com/TheHPXProject/hpx/pull/4814
[2707] https://github.com/TheHPXProject/hpx/pull/4809
[2708] https://github.com/TheHPXProject/hpx/pull/4808
[2709] https://github.com/TheHPXProject/hpx/pull/4807
[2710] https://github.com/TheHPXProject/hpx/pull/4806
[2711] https://github.com/TheHPXProject/hpx/pull/4805
[2712] https://github.com/TheHPXProject/hpx/pull/4803
[2713] https://github.com/TheHPXProject/hpx/pull/4802
[2714] https://github.com/TheHPXProject/hpx/pull/4801

- PR #4799[2715] - Fixing #include paths in compatibility headers
- PR #4798[2716] - Include the main module header (fixes partially #4488)
- PR #4797[2717] - Change cmake targets
- PR #4794[2718] - Removing 128bit integer emulation
- PR #4793[2719] - Make sure global variable is handled properly
- PR #4792[2720] - Replace enable_if with **HPX_CONCEPT_REQUIRES_** and add is_sentinel_for constraint
- PR #4790[2721] - Move deprecation warnings from base template to template specializations for result_of etc. structs
- PR #4789[2722] - Fix hangs during assertion handling and distributed runtime construction
- PR #4788[2723] - Fixing inclusive transform scan algorithm to properly handle initial value
- PR #4785[2724] - Fixing barrier test
- PR #4784[2725] - Fixing deleter argument bindings in serialize_buffer
- PR #4783[2726] - Add coveralls badge
- PR #4782[2727] - Make header tests parallel again
- PR #4780[2728] - Remove outdated comment about hpx::stop in documentation
- PR #4776[2729] - debug print improvements
- PR #4775[2730] - Checkpoint cleanup
- PR #4771[2731] - Fix compilation with HPX_WITH_NETWORKING=OFF
- PR #4767[2732] - Remove all force linking leftovers
- PR #4765[2733] - Fix 1d stencil index calculation
- PR #4764[2734] - Force some tests to run serially
- PR #4762[2735] - Update pointees in compatibility headers
- PR #4761[2736] - Fix running and building of execution module tests on CircleCI

[2715] https://github.com/TheHPXProject/hpx/pull/4799
[2716] https://github.com/TheHPXProject/hpx/pull/4798
[2717] https://github.com/TheHPXProject/hpx/pull/4797
[2718] https://github.com/TheHPXProject/hpx/pull/4794
[2719] https://github.com/TheHPXProject/hpx/pull/4793
[2720] https://github.com/TheHPXProject/hpx/pull/4792
[2721] https://github.com/TheHPXProject/hpx/pull/4790
[2722] https://github.com/TheHPXProject/hpx/pull/4789
[2723] https://github.com/TheHPXProject/hpx/pull/4788
[2724] https://github.com/TheHPXProject/hpx/pull/4785
[2725] https://github.com/TheHPXProject/hpx/pull/4784
[2726] https://github.com/TheHPXProject/hpx/pull/4783
[2727] https://github.com/TheHPXProject/hpx/pull/4782
[2728] https://github.com/TheHPXProject/hpx/pull/4780
[2729] https://github.com/TheHPXProject/hpx/pull/4776
[2730] https://github.com/TheHPXProject/hpx/pull/4775
[2731] https://github.com/TheHPXProject/hpx/pull/4771
[2732] https://github.com/TheHPXProject/hpx/pull/4767
[2733] https://github.com/TheHPXProject/hpx/pull/4765
[2734] https://github.com/TheHPXProject/hpx/pull/4764
[2735] https://github.com/TheHPXProject/hpx/pull/4762
[2736] https://github.com/TheHPXProject/hpx/pull/4761

- PR #4760[2737] - Storing hpx_options in global property to speed up summary report
- PR #4759[2738] - Reduce memory requirements for our main shared state
- PR #4757[2739] - Fix mimalloc linking on Windows
- PR #4756[2740] - Fix compilation issues
- PR #4753[2741] - Re-adding API functions that were lost during merges
- PR #4751[2742] - Revert "Create coverage reports and upload them to codecov.io"
- PR #4750[2743] - Fixing possible race condition during termination detection
- PR #4749[2744] - Deprecate result_of and friends
- PR #4748[2745] - Create coverage reports and upload them to codecov.io
- PR #4747[2746] - Changing #include for MPI parcelport
- PR #4745[2747] - Add *is_sentinel_for* trait implementation and test
- PR #4743[2748] - Fix init_globally example after runtime mode changes
- PR #4742[2749] - Update SUPPORT.md
- PR #4741[2750] - Fixing a warning generated for unity builds with msvc
- PR #4740[2751] - Rename local_lcos and basic_execution modules
- PR #4739[2752] - Undeprecate a couple of hpx/modulename.hpp headers
- PR #4738[2753] - Conditionally test schedulers in thread_stacksize_current test
- PR #4734[2754] - Fixing a bunch of codacy warnings
- PR #4733[2755] - Add experimental unity build option to CMake configuration
- PR #4730[2756] - Fixing compilation problems with unordered map
- PR #4729[2757] - Fix APEX build
- PR #4727[2758] - Fix missing runtime includes for distributed runtime
- PR #4726[2759] - Add more API headers

[2737] https://github.com/TheHPXProject/hpx/pull/4760
[2738] https://github.com/TheHPXProject/hpx/pull/4759
[2739] https://github.com/TheHPXProject/hpx/pull/4757
[2740] https://github.com/TheHPXProject/hpx/pull/4756
[2741] https://github.com/TheHPXProject/hpx/pull/4753
[2742] https://github.com/TheHPXProject/hpx/pull/4751
[2743] https://github.com/TheHPXProject/hpx/pull/4750
[2744] https://github.com/TheHPXProject/hpx/pull/4749
[2745] https://github.com/TheHPXProject/hpx/pull/4748
[2746] https://github.com/TheHPXProject/hpx/pull/4747
[2747] https://github.com/TheHPXProject/hpx/pull/4745
[2748] https://github.com/TheHPXProject/hpx/pull/4743
[2749] https://github.com/TheHPXProject/hpx/pull/4742
[2750] https://github.com/TheHPXProject/hpx/pull/4741
[2751] https://github.com/TheHPXProject/hpx/pull/4740
[2752] https://github.com/TheHPXProject/hpx/pull/4739
[2753] https://github.com/TheHPXProject/hpx/pull/4738
[2754] https://github.com/TheHPXProject/hpx/pull/4734
[2755] https://github.com/TheHPXProject/hpx/pull/4733
[2756] https://github.com/TheHPXProject/hpx/pull/4730
[2757] https://github.com/TheHPXProject/hpx/pull/4729
[2758] https://github.com/TheHPXProject/hpx/pull/4727
[2759] https://github.com/TheHPXProject/hpx/pull/4726

- PR #4725[2760] - Add more compatibility headers for deprecated module headers
- PR #4724[2761] - Fix 4723
- PR #4721[2762] - Attempt to fixing migration tests
- PR #4717[2763] - Make the compatilibility headers macro conditional
- PR #4716[2764] - Add hpx/runtime.hpp and hpx/distributed/runtime.hpp API headers
- PR #4714[2765] - Add hpx/future.hpp header
- PR #4713[2766] - Remove hpx/runtime/threads_fwd.hpp and hpx/util_fwd.hpp
- PR #4711[2767] - Make module deprecation warnings overridable
- PR #4710[2768] - Add compatibility headers and other fixes after module header renaming
- PR #4708[2769] - Add termination handler for parallel algorithms
- PR #4707[2770] - Use hpx::function_nonser instead of std::function internally
- PR #4706[2771] - Move header file to module
- PR #4705[2772] - Fix incorrect behaviour of cmake-format check
- PR #4704[2773] - Fix resource tests
- PR #4701[2774] - Fix missing includes for future::then specializations
- PR #4700[2775] - Removing obsolete memory component
- PR #4699[2776] - Add short descriptions to modules missing documentation
- PR #4696[2777] - Rename generated modules headers
- PR #4693[2778] - Overhauling thread_mapper for public consumption
- PR #4688[2779] - Fix thread stack size handling
- PR #4687[2780] - Adding all_gather and fixing all_to_all
- PR #4684[2781] - Miscellaneous compilation fixes
- PR #4683[2782] - Fix HPX_WITH_DYNAMIC_HPX_MAIN=OFF

[2760] https://github.com/TheHPXProject/hpx/pull/4725
[2761] https://github.com/TheHPXProject/hpx/pull/4724
[2762] https://github.com/TheHPXProject/hpx/pull/4721
[2763] https://github.com/TheHPXProject/hpx/pull/4717
[2764] https://github.com/TheHPXProject/hpx/pull/4716
[2765] https://github.com/TheHPXProject/hpx/pull/4714
[2766] https://github.com/TheHPXProject/hpx/pull/4713
[2767] https://github.com/TheHPXProject/hpx/pull/4711
[2768] https://github.com/TheHPXProject/hpx/pull/4710
[2769] https://github.com/TheHPXProject/hpx/pull/4708
[2770] https://github.com/TheHPXProject/hpx/pull/4707
[2771] https://github.com/TheHPXProject/hpx/pull/4706
[2772] https://github.com/TheHPXProject/hpx/pull/4705
[2773] https://github.com/TheHPXProject/hpx/pull/4704
[2774] https://github.com/TheHPXProject/hpx/pull/4701
[2775] https://github.com/TheHPXProject/hpx/pull/4700
[2776] https://github.com/TheHPXProject/hpx/pull/4699
[2777] https://github.com/TheHPXProject/hpx/pull/4696
[2778] https://github.com/TheHPXProject/hpx/pull/4693
[2779] https://github.com/TheHPXProject/hpx/pull/4688
[2780] https://github.com/TheHPXProject/hpx/pull/4687
[2781] https://github.com/TheHPXProject/hpx/pull/4684
[2782] https://github.com/TheHPXProject/hpx/pull/4683

- PR #4682[2783] - Fix compilation of pack_traversal_rebind_container.hpp
- PR #4681[2784] - Add missing hpx/execution.hpp includes for future::then
- PR #4678[2785] - Typeless communicator
- PR #4677[2786] - Forcing registry option to be accepted without checks.
- PR #4676[2787] - Adding scatter_to/scatter_from collective operations
- PR #4673[2788] - Fix PAPI counters compilation
- PR #4671[2789] - Deprecate hpx::promise alias to hpx::lcos::promise
- PR #4670[2790] - Explicitly instantiate get_exception
- PR #4667[2791] - Add *stopValue* in *Sentinel* struct instead of *Iterator*
- PR #4666[2792] - Add release build on Windows to GitHub actions
- PR #4664[2793] - Creating itt_notify module.
- PR #4663[2794] - Mpi fixes
- PR #4659[2795] - Making sure declarations match definitions in register_locks implementation
- PR #4655[2796] - Fixing task annotations for actions
- PR #4653[2797] - Making sure APEX is linked into every application, if needed
- PR #4651[2798] - Update get_function_annotation.hpp
- PR #4646[2799] - Runtime type
- PR #4645[2800] - Add a few more API headers
- PR #4644[2801] - Fixing support for mpirun (and similar)
- PR #4643[2802] - Fixing the fix for get_idle_core_count() API
- PR #4638[2803] - Remove HPX_API_EXPORT missed in previous cleanup
- PR #4636[2804] - Adding C++20 barrier
- PR #4635[2805] - Adding C++20 latch API

[2783] https://github.com/TheHPXProject/hpx/pull/4682
[2784] https://github.com/TheHPXProject/hpx/pull/4681
[2785] https://github.com/TheHPXProject/hpx/pull/4678
[2786] https://github.com/TheHPXProject/hpx/pull/4677
[2787] https://github.com/TheHPXProject/hpx/pull/4676
[2788] https://github.com/TheHPXProject/hpx/pull/4673
[2789] https://github.com/TheHPXProject/hpx/pull/4671
[2790] https://github.com/TheHPXProject/hpx/pull/4670
[2791] https://github.com/TheHPXProject/hpx/pull/4667
[2792] https://github.com/TheHPXProject/hpx/pull/4666
[2793] https://github.com/TheHPXProject/hpx/pull/4664
[2794] https://github.com/TheHPXProject/hpx/pull/4663
[2795] https://github.com/TheHPXProject/hpx/pull/4659
[2796] https://github.com/TheHPXProject/hpx/pull/4655
[2797] https://github.com/TheHPXProject/hpx/pull/4653
[2798] https://github.com/TheHPXProject/hpx/pull/4651
[2799] https://github.com/TheHPXProject/hpx/pull/4646
[2800] https://github.com/TheHPXProject/hpx/pull/4645
[2801] https://github.com/TheHPXProject/hpx/pull/4644
[2802] https://github.com/TheHPXProject/hpx/pull/4643
[2803] https://github.com/TheHPXProject/hpx/pull/4638
[2804] https://github.com/TheHPXProject/hpx/pull/4636
[2805] https://github.com/TheHPXProject/hpx/pull/4635

- PR #4634[2806] - Adding C++20 counting semaphore API
- PR #4633[2807] - Unify execution parameters customization points
- PR #4632[2808] - Adding missing bulk_sync_execute wrapper to example executor
- PR #4631[2809] - Updates to documentation; grammar edits.
- PR #4630[2810] - Updates to documentation; moved hyperlink
- PR #4624[2811] - Export set_self_ptr in thread_data.hpp instead of with forward declarations where used
- PR #4623[2812] - Clean up export macros
- PR #4621[2813] - Trigger an error for older boost versions on power architectures
- PR #4617[2814] - Ignore user-set compatibility header options if the module does not have compatibility headers
- PR #4616[2815] - Fix cmake-format warning
- PR #4615[2816] - Add handler for serializing custom exceptions
- PR #4614[2817] - Fix error message when HPX_IGNORE_CMAKE_BUILD_TYPE_COMPATIBILITY=OFF
- PR #4613[2818] - Make partitioner constructor private
- PR #4611[2819] - Making auto_chunk_size execute the given function using the given executor
- PR #4610[2820] - Making sure the thread-local lock registration data is moving to the core the suspended HPX thread is resumed on
- PR #4609[2821] - Adding an API function that exposes the number of idle cores
- PR #4608[2822] - Fixing moodycamel namespace
- PR #4607[2823] - Moving winsocket initialization to core library
- PR #4606[2824] - Local runtime module etc.
- PR #4604[2825] - Add config_registry module
- PR #4603[2826] - Deal with distributed modules in their respective CMakeLists.txt

[2806] https://github.com/TheHPXProject/hpx/pull/4634
[2807] https://github.com/TheHPXProject/hpx/pull/4633
[2808] https://github.com/TheHPXProject/hpx/pull/4632
[2809] https://github.com/TheHPXProject/hpx/pull/4631
[2810] https://github.com/TheHPXProject/hpx/pull/4630
[2811] https://github.com/TheHPXProject/hpx/pull/4624
[2812] https://github.com/TheHPXProject/hpx/pull/4623
[2813] https://github.com/TheHPXProject/hpx/pull/4621
[2814] https://github.com/TheHPXProject/hpx/pull/4617
[2815] https://github.com/TheHPXProject/hpx/pull/4616
[2816] https://github.com/TheHPXProject/hpx/pull/4615
[2817] https://github.com/TheHPXProject/hpx/pull/4614
[2818] https://github.com/TheHPXProject/hpx/pull/4613
[2819] https://github.com/TheHPXProject/hpx/pull/4611
[2820] https://github.com/TheHPXProject/hpx/pull/4610
[2821] https://github.com/TheHPXProject/hpx/pull/4609
[2822] https://github.com/TheHPXProject/hpx/pull/4608
[2823] https://github.com/TheHPXProject/hpx/pull/4607
[2824] https://github.com/TheHPXProject/hpx/pull/4606
[2825] https://github.com/TheHPXProject/hpx/pull/4604
[2826] https://github.com/TheHPXProject/hpx/pull/4603

- PR #4602[2827] - Small module fixes
- PR #4598[2828] - Making sure current_executor and service_executor functions are linked into the core library
- PR #4597[2829] - Adding broadcast_to/broadcast_from to collectives module
- PR #4596[2830] - Fix performance regression in block_executor
- PR #4595[2831] - Making sure main.cpp is built as a library if HPX_WITH_DYNAMIC_MAIN=OFF
- PR #4592[2832] - Futures module
- PR #4591[2833] - Adapting co_await support for C++20
- PR #4590[2834] - Adding missing exception test for for_loop()
- PR #4587[2835] - Move traits headers to hpx/modulename/traits directory
- PR #4586[2836] - Remove Travis CI config
- PR #4585[2837] - Update macOS test blacklist
- PR #4584[2838] - Attempting to fix missing symbols in stack trace
- PR #4583[2839] - Fixing bad static_cast
- PR #4582[2840] - Changing download url for Windows prerequisites to circumvent bandwidth limitations
- PR #4581[2841] - Adding missing using placeholder::_X
- PR #4579[2842] - Move get_stack_size_name and related functions
- PR #4575[2843] - Excluding unconditional definition of class backtrace from global header
- PR #4574[2844] - Changing return type of hardware_concurrency() to unsigned int
- PR #4570[2845] - Move tests to modules
- PR #4564[2846] - Reshuffle internal targets and add HPX::hpx_no_wrap_main target
- PR #4563[2847] - fix CMake option typo
- PR #4562[2848] - Unregister lock earlier to avoid holding it while suspending

[2827] https://github.com/TheHPXProject/hpx/pull/4602
[2828] https://github.com/TheHPXProject/hpx/pull/4598
[2829] https://github.com/TheHPXProject/hpx/pull/4597
[2830] https://github.com/TheHPXProject/hpx/pull/4596
[2831] https://github.com/TheHPXProject/hpx/pull/4595
[2832] https://github.com/TheHPXProject/hpx/pull/4592
[2833] https://github.com/TheHPXProject/hpx/pull/4591
[2834] https://github.com/TheHPXProject/hpx/pull/4590
[2835] https://github.com/TheHPXProject/hpx/pull/4587
[2836] https://github.com/TheHPXProject/hpx/pull/4586
[2837] https://github.com/TheHPXProject/hpx/pull/4585
[2838] https://github.com/TheHPXProject/hpx/pull/4584
[2839] https://github.com/TheHPXProject/hpx/pull/4583
[2840] https://github.com/TheHPXProject/hpx/pull/4582
[2841] https://github.com/TheHPXProject/hpx/pull/4581
[2842] https://github.com/TheHPXProject/hpx/pull/4579
[2843] https://github.com/TheHPXProject/hpx/pull/4575
[2844] https://github.com/TheHPXProject/hpx/pull/4574
[2845] https://github.com/TheHPXProject/hpx/pull/4570
[2846] https://github.com/TheHPXProject/hpx/pull/4564
[2847] https://github.com/TheHPXProject/hpx/pull/4563
[2848] https://github.com/TheHPXProject/hpx/pull/4562

- PR #4561[2849] - Adding test macros supporting custom output stream
- PR #4560[2850] - Making sure hash_any::operator()() is linked into core library
- PR #4559[2851] - Fixing compilation if HPX_WITH_THREAD_BACKTRACE_ON_SUSPENSION=On
- PR #4557[2852] - Improve spinlock implementation to perform better in high-contention situations
- PR #4553[2853] - Fix a runtime_ptr problem at shutdown when apex is enabled
- PR #4552[2854] - Add configuration option for making exceptions less noisy
- PR #4551[2855] - Clean up thread creation parameters
- PR #4549[2856] - Test FetchContent build on GitHub actions
- PR #4548[2857] - Fix stack size
- PR #4545[2858] - Fix header tests
- PR #4544[2859] - Fix a typo in sanitizer build
- PR #4541[2860] - Add API to check if a thread pool exists
- PR #4540[2861] - Making sure MPI support is enabled if MPI futures are used but networking is disabled
- PR #4538[2862] - Move channel documentation examples to examples directory
- PR #4536[2863] - Add generic allocator for execution policies
- PR #4534[2864] - Enable compatibility headers for thread_executors module
- PR #4532[2865] - Fixing broken url in README.rst
- PR #4531[2866] - Update scripts
- PR #4530[2867] - Make sure module API docs show up in correct order
- PR #4529[2868] - Adding missing template code to module creation script
- PR #4528[2869] - Make sure version module uses HPX's binary dir, not the parent's
- PR #4527[2870] - Creating actions_base and actions module

[2849] https://github.com/TheHPXProject/hpx/pull/4561
[2850] https://github.com/TheHPXProject/hpx/pull/4560
[2851] https://github.com/TheHPXProject/hpx/pull/4559
[2852] https://github.com/TheHPXProject/hpx/pull/4557
[2853] https://github.com/TheHPXProject/hpx/pull/4553
[2854] https://github.com/TheHPXProject/hpx/pull/4552
[2855] https://github.com/TheHPXProject/hpx/pull/4551
[2856] https://github.com/TheHPXProject/hpx/pull/4549
[2857] https://github.com/TheHPXProject/hpx/pull/4548
[2858] https://github.com/TheHPXProject/hpx/pull/4545
[2859] https://github.com/TheHPXProject/hpx/pull/4544
[2860] https://github.com/TheHPXProject/hpx/pull/4541
[2861] https://github.com/TheHPXProject/hpx/pull/4540
[2862] https://github.com/TheHPXProject/hpx/pull/4538
[2863] https://github.com/TheHPXProject/hpx/pull/4536
[2864] https://github.com/TheHPXProject/hpx/pull/4534
[2865] https://github.com/TheHPXProject/hpx/pull/4532
[2866] https://github.com/TheHPXProject/hpx/pull/4531
[2867] https://github.com/TheHPXProject/hpx/pull/4530
[2868] https://github.com/TheHPXProject/hpx/pull/4529
[2869] https://github.com/TheHPXProject/hpx/pull/4528
[2870] https://github.com/TheHPXProject/hpx/pull/4527

- PR #4526[2871] - Shared state for cv
- PR #4525[2872] - Changing sub-name sequencing for experimental namespace
- PR #4524[2873] - Add API guarantee notes to API reference documentation
- PR #4522[2874] - Enable and fix deprecation warnings in execution module
- PR #4521[2875] - Moves more miscellaneous files to modules
- PR #4520[2876] - Skip execution customization points when executor is known
- PR #4518[2877] - Module distributed lcos
- PR #4516[2878] - Fix various builds
- PR #4515[2879] - Replace backslashes by slashes in windows paths
- PR #4514[2880] - Adding polymorphic_executor
- PR #4512[2881] - Adding C++20 jthread and stop_token
- PR #4510[2882] - Attempt to fix APEX linking in external packages again
- PR #4508[2883] - Only test pull requests (not all branches) with GitHub actions
- PR #4505[2884] - Fix duplicate linking in tests (ODR violations)
- PR #4504[2885] - Fix C++ standard handling
- PR #4503[2886] - Add CMakelists file check
- PR #4500[2887] - Fix .clang-format version requirement comment
- PR #4499[2888] - Attempting to fix hpx_init linking on macOS
- PR #4498[2889] - Fix compatibility of *pool_executor*
- PR #4496[2890] - Removing superfluous SPDX tags
- PR #4494[2891] - Module executors
- PR #4493[2892] - Pack traversal module
- PR #4492[2893] - Update copyright year in documentation

[2871] https://github.com/TheHPXProject/hpx/pull/4526
[2872] https://github.com/TheHPXProject/hpx/pull/4525
[2873] https://github.com/TheHPXProject/hpx/pull/4524
[2874] https://github.com/TheHPXProject/hpx/pull/4522
[2875] https://github.com/TheHPXProject/hpx/pull/4521
[2876] https://github.com/TheHPXProject/hpx/pull/4520
[2877] https://github.com/TheHPXProject/hpx/pull/4518
[2878] https://github.com/TheHPXProject/hpx/pull/4516
[2879] https://github.com/TheHPXProject/hpx/pull/4515
[2880] https://github.com/TheHPXProject/hpx/pull/4514
[2881] https://github.com/TheHPXProject/hpx/pull/4512
[2882] https://github.com/TheHPXProject/hpx/pull/4510
[2883] https://github.com/TheHPXProject/hpx/pull/4508
[2884] https://github.com/TheHPXProject/hpx/pull/4505
[2885] https://github.com/TheHPXProject/hpx/pull/4504
[2886] https://github.com/TheHPXProject/hpx/pull/4503
[2887] https://github.com/TheHPXProject/hpx/pull/4500
[2888] https://github.com/TheHPXProject/hpx/pull/4499
[2889] https://github.com/TheHPXProject/hpx/pull/4498
[2890] https://github.com/TheHPXProject/hpx/pull/4496
[2891] https://github.com/TheHPXProject/hpx/pull/4494
[2892] https://github.com/TheHPXProject/hpx/pull/4493
[2893] https://github.com/TheHPXProject/hpx/pull/4492

- PR #4491[2894] - Add missing current_executor header
- PR #4490[2895] - Update GitHub actions configs
- PR #4487[2896] - Properly dispatch exceptions thrown from hpx_main to be rethrown from hpx::init/hpx::stop
- PR #4486[2897] - Fixing an initialization order problem
- PR #4485[2898] - Move miscellaneous files to their rightful modules
- PR #4483[2899] - Clean up imported CMake target naming
- PR #4481[2900] - Add vcpkg installation instructions
- PR #4479[2901] - Add hints to allow to specify MIMALLOC_ROOT
- PR #4478[2902] - Async modules
- PR #4475[2903] - Fix rp init changes
- PR #4474[2904] - Use #pragma once in headers
- PR #4472[2905] - Add more descriptive error message when using x86 coroutines on non-x86 platforms
- PR #4467[2906] - Add mimalloc find cmake script
- PR #4465[2907] - Add thread_executors module
- PR #4464[2908] - Include module
- PR #4462[2909] - Merge hpx_init and hpx_wrap into one static library
- PR #4461[2910] - Making thread_data test more realistic
- PR #4460[2911] - Suppress MPI warnings in version.cpp
- PR #4459[2912] - Make sure pkgconfig applications link with hpx_init
- PR #4458[2913] - Added example demonstrating how to create and use a wrapping executor
- PR #4457[2914] - Fixing execution of thread exit functions
- PR #4456[2915] - Move backtrace files to debugging module

[2894] https://github.com/TheHPXProject/hpx/pull/4491
[2895] https://github.com/TheHPXProject/hpx/pull/4490
[2896] https://github.com/TheHPXProject/hpx/pull/4487
[2897] https://github.com/TheHPXProject/hpx/pull/4486
[2898] https://github.com/TheHPXProject/hpx/pull/4485
[2899] https://github.com/TheHPXProject/hpx/pull/4483
[2900] https://github.com/TheHPXProject/hpx/pull/4481
[2901] https://github.com/TheHPXProject/hpx/pull/4479
[2902] https://github.com/TheHPXProject/hpx/pull/4478
[2903] https://github.com/TheHPXProject/hpx/pull/4475
[2904] https://github.com/TheHPXProject/hpx/pull/4474
[2905] https://github.com/TheHPXProject/hpx/pull/4472
[2906] https://github.com/TheHPXProject/hpx/pull/4467
[2907] https://github.com/TheHPXProject/hpx/pull/4465
[2908] https://github.com/TheHPXProject/hpx/pull/4464
[2909] https://github.com/TheHPXProject/hpx/pull/4462
[2910] https://github.com/TheHPXProject/hpx/pull/4461
[2911] https://github.com/TheHPXProject/hpx/pull/4460
[2912] https://github.com/TheHPXProject/hpx/pull/4459
[2913] https://github.com/TheHPXProject/hpx/pull/4458
[2914] https://github.com/TheHPXProject/hpx/pull/4457
[2915] https://github.com/TheHPXProject/hpx/pull/4456

- PR #4455[2916] - Move deadlock_detection and maintain_queue_wait_times source files into schedulers module
- PR #4450[2917] - Fixing compilation with std::filesystem enabled
- PR #4449[2918] - Fixing build system to actually build variant test
- PR #4447[2919] - This fixes an obsolete #include
- PR #4446[2920] - Resume tasks where they were suspended
- PR #4444[2921] - Minor CUDA fixes
- PR #4443[2922] - Add missing tests to CircleCI config
- PR #4442[2923] - Adding a tag to all auto-generated files allowing for tools to visually distinguish those
- PR #4441[2924] - Adding performance counter type information
- PR #4440[2925] - Fixing MSVC build
- PR #4439[2926] - Link HPX::plugin and component privately in hpx_setup_target
- PR #4437[2927] - Adding a test that verifies the problem can be solved using a trait specialization
- PR #4434[2928] - Clean up Boost dependencies and copy string algorithms to new module
- PR #4433[2929] - Fixing compilation issues (!) if MPI parcelport is enabled
- PR #4431[2930] - Ignore warnings about name mangling changing
- PR #4430[2931] - Add performance_counters module
- PR #4428[2932] - Don't add compatibility headers to module API reference
- PR #4426[2933] - Add currently failing tests on GitHub actions to blacklist
- PR #4425[2934] - Clean up and correct minimum required versions
- PR #4424[2935] - Making sure hpx.lock_detection=0 works as advertized
- PR #4421[2936] - Making sure interval time stops underlying timer thread on termination
- PR #4417[2937] - Adding serialization support for std::variant (if available) and std::tuple

[2916] https://github.com/TheHPXProject/hpx/pull/4455
[2917] https://github.com/TheHPXProject/hpx/pull/4450
[2918] https://github.com/TheHPXProject/hpx/pull/4449
[2919] https://github.com/TheHPXProject/hpx/pull/4447
[2920] https://github.com/TheHPXProject/hpx/pull/4446
[2921] https://github.com/TheHPXProject/hpx/pull/4444
[2922] https://github.com/TheHPXProject/hpx/pull/4443
[2923] https://github.com/TheHPXProject/hpx/pull/4442
[2924] https://github.com/TheHPXProject/hpx/pull/4441
[2925] https://github.com/TheHPXProject/hpx/pull/4440
[2926] https://github.com/TheHPXProject/hpx/pull/4439
[2927] https://github.com/TheHPXProject/hpx/pull/4437
[2928] https://github.com/TheHPXProject/hpx/pull/4434
[2929] https://github.com/TheHPXProject/hpx/pull/4433
[2930] https://github.com/TheHPXProject/hpx/pull/4431
[2931] https://github.com/TheHPXProject/hpx/pull/4430
[2932] https://github.com/TheHPXProject/hpx/pull/4428
[2933] https://github.com/TheHPXProject/hpx/pull/4426
[2934] https://github.com/TheHPXProject/hpx/pull/4425
[2935] https://github.com/TheHPXProject/hpx/pull/4424
[2936] https://github.com/TheHPXProject/hpx/pull/4421
[2937] https://github.com/TheHPXProject/hpx/pull/4417

- PR #4415[2938] - Partially reverting changes applied by PR 4373
- PR #4414[2939] - Added documentation for the compiler-wrapper script hpxcxx.in in creating_hpx_projects.rst
- PR #4413[2940] - Merging from V1.4.1 release
- PR #4412[2941] - Making sure to issue a warning if a file specified using –hpx:options-file is not found
- PR #4411[2942] - Make test specific to HPX_WITH_SHARED_PRIORITY_SCHEDULER
- PR #4407[2943] - Adding minimal MPI executor
- PR #4405[2944] - Fix cross pool injection test, use default scheduler as falback
- PR #4404[2945] - Fix a race condition and clean-up usage of scheduler mode
- PR #4399[2946] - Add more threading modules
- PR #4398[2947] - Add CODEOWNERS file
- PR #4395[2948] - Adding a parameter to auto_chunk_size allowing to control the amount of iterations to measure
- PR #4393[2949] - Use appropriate cache-line size defaults for different platforms
- PR #4391[2950] - Fixing use of allocator for C++20
- PR #4390[2951] - Making –hpx:help behavior consistent
- PR #4388[2952] - Change the resource partitioner initialization
- PR #4387[2953] - Fix roll_release.sh
- PR #4386[2954] - Add warning messages for using thread binding options on macOS
- PR #4385[2955] - Cuda futures
- PR #4384[2956] - Make enabling dynamic hpx_main on non-Linux systems a configuration error
- PR #4383[2957] - Use configure_file for HPXCacheVariables.cmake
- PR #4382[2958] - Update spellchecking whitelist and fix more typos

[2938] https://github.com/TheHPXProject/hpx/pull/4415
[2939] https://github.com/TheHPXProject/hpx/pull/4414
[2940] https://github.com/TheHPXProject/hpx/pull/4413
[2941] https://github.com/TheHPXProject/hpx/pull/4412
[2942] https://github.com/TheHPXProject/hpx/pull/4411
[2943] https://github.com/TheHPXProject/hpx/pull/4407
[2944] https://github.com/TheHPXProject/hpx/pull/4405
[2945] https://github.com/TheHPXProject/hpx/pull/4404
[2946] https://github.com/TheHPXProject/hpx/pull/4399
[2947] https://github.com/TheHPXProject/hpx/pull/4398
[2948] https://github.com/TheHPXProject/hpx/pull/4395
[2949] https://github.com/TheHPXProject/hpx/pull/4393
[2950] https://github.com/TheHPXProject/hpx/pull/4391
[2951] https://github.com/TheHPXProject/hpx/pull/4390
[2952] https://github.com/TheHPXProject/hpx/pull/4388
[2953] https://github.com/TheHPXProject/hpx/pull/4387
[2954] https://github.com/TheHPXProject/hpx/pull/4386
[2955] https://github.com/TheHPXProject/hpx/pull/4385
[2956] https://github.com/TheHPXProject/hpx/pull/4384
[2957] https://github.com/TheHPXProject/hpx/pull/4383
[2958] https://github.com/TheHPXProject/hpx/pull/4382

- PR #4380[2959] - Add a helper function to get a future from a cuda stream
- PR #4379[2960] - Add Windows and macOS CI with GitHub actions
- PR #4378[2961] - Change C++ standard handling
- PR #4377[2962] - Remove Python scripts
- PR #4374[2963] - Adding overload for *hpx::init*/*hpx::start* for use with resource partitioner
- PR #4373[2964] - Adding test that verifies for 4369 to be fixed
- PR #4372[2965] - Another attempt at fixing the integral mismatch and conversion warnings
- PR #4370[2966] - Doc updates quick start
- PR #4368[2967] - Add a whitelist of words for weird spelling suggestions
- PR #4366[2968] - Suppress or fix clang-tidy-9 warnings
- PR #4365[2969] - Removing more Boost dependencies
- PR #4363[2970] - Update clang-format config file for version 9
- PR #4362[2971] - Fix indices typo
- PR #4361[2972] - Boost cleanup
- PR #4360[2973] - Move plugins
- PR #4358[2974] - Doc updates; generating documentation. Will likely need heavy editing.
- PR #4356[2975] - Remove some minor unused and unnecessary Boost includes
- PR #4355[2976] - Fix spellcheck step in CircleCI config
- PR #4354[2977] - Lightweight utility to hold a pack as members
- PR #4352[2978] - Minor fixes to the C++ standard detection for MSVC
- PR #4351[2979] - Move generated documentation to hpx-docs repo
- PR #4347[2980] - Add cmake policy - CMP0074
- PR #4346[2981] - Remove file committed by mistake

[2959] https://github.com/TheHPXProject/hpx/pull/4380
[2960] https://github.com/TheHPXProject/hpx/pull/4379
[2961] https://github.com/TheHPXProject/hpx/pull/4378
[2962] https://github.com/TheHPXProject/hpx/pull/4377
[2963] https://github.com/TheHPXProject/hpx/pull/4374
[2964] https://github.com/TheHPXProject/hpx/pull/4373
[2965] https://github.com/TheHPXProject/hpx/pull/4372
[2966] https://github.com/TheHPXProject/hpx/pull/4370
[2967] https://github.com/TheHPXProject/hpx/pull/4368
[2968] https://github.com/TheHPXProject/hpx/pull/4366
[2969] https://github.com/TheHPXProject/hpx/pull/4365
[2970] https://github.com/TheHPXProject/hpx/pull/4363
[2971] https://github.com/TheHPXProject/hpx/pull/4362
[2972] https://github.com/TheHPXProject/hpx/pull/4361
[2973] https://github.com/TheHPXProject/hpx/pull/4360
[2974] https://github.com/TheHPXProject/hpx/pull/4358
[2975] https://github.com/TheHPXProject/hpx/pull/4356
[2976] https://github.com/TheHPXProject/hpx/pull/4355
[2977] https://github.com/TheHPXProject/hpx/pull/4354
[2978] https://github.com/TheHPXProject/hpx/pull/4352
[2979] https://github.com/TheHPXProject/hpx/pull/4351
[2980] https://github.com/TheHPXProject/hpx/pull/4347
[2981] https://github.com/TheHPXProject/hpx/pull/4346

- PR #4342[2982] - Remove HCC and SYCL options from CMakeLists.txt
- PR #4341[2983] - Fix launch process test with APEX enabled
- PR #4340[2984] - Testing Cirrus CI
- PR #4339[2985] - Post 1.4.0 updates
- PR #4338[2986] - Spelling corrections and CircleCI spell check
- PR #4333[2987] - Flatten bound callables
- PR #4332[2988] - This is a collection of mostly minor (cleanup) fixes
- PR #4331[2989] - This adds the missing tests for async_colocated and async_continue_colocated
- PR #4330[2990] - Remove HPX.Compute host default_executor
- PR #4328[2991] - Generate global header for basic_execution module
- PR #4327[2992] - Use INTERNAL_FLAGS option for all examples and components
- PR #4326[2993] - Usage of temporary allocator in assignment operator of compute::vector
- PR #4325[2994] - Use hpx::threads::get_cache_line_size in prefetching.hpp
- PR #4324[2995] - Enable compatibility headers option for execution module
- PR #4316[2996] - Add clang format indentppdirectives
- PR #4313[2997] - Introduce index_pack alias to pack of size_t
- PR #4312[2998] - Fixing compatibility header for pack.hpp
- PR #4311[2999] - Dataflow annotations for APEX
- PR #4309[3000] - Update launching_and_configuring_hpx_applications.rst
- PR #4306[3001] - Fix schedule hint not being taken from executor
- PR #4305[3002] - Implementing *hpx::functional::tag_invoke*
- PR #4304[3003] - Improve pack support utilities
- PR #4303[3004] - Remove errors module dependency on datastructures

[2982] https://github.com/TheHPXProject/hpx/pull/4342
[2983] https://github.com/TheHPXProject/hpx/pull/4341
[2984] https://github.com/TheHPXProject/hpx/pull/4340
[2985] https://github.com/TheHPXProject/hpx/pull/4339
[2986] https://github.com/TheHPXProject/hpx/pull/4338
[2987] https://github.com/TheHPXProject/hpx/pull/4333
[2988] https://github.com/TheHPXProject/hpx/pull/4332
[2989] https://github.com/TheHPXProject/hpx/pull/4331
[2990] https://github.com/TheHPXProject/hpx/pull/4330
[2991] https://github.com/TheHPXProject/hpx/pull/4328
[2992] https://github.com/TheHPXProject/hpx/pull/4327
[2993] https://github.com/TheHPXProject/hpx/pull/4326
[2994] https://github.com/TheHPXProject/hpx/pull/4325
[2995] https://github.com/TheHPXProject/hpx/pull/4324
[2996] https://github.com/TheHPXProject/hpx/pull/4316
[2997] https://github.com/TheHPXProject/hpx/pull/4313
[2998] https://github.com/TheHPXProject/hpx/pull/4312
[2999] https://github.com/TheHPXProject/hpx/pull/4311
[3000] https://github.com/TheHPXProject/hpx/pull/4309
[3001] https://github.com/TheHPXProject/hpx/pull/4306
[3002] https://github.com/TheHPXProject/hpx/pull/4305
[3003] https://github.com/TheHPXProject/hpx/pull/4304
[3004] https://github.com/TheHPXProject/hpx/pull/4303

- PR #4301[3005] - Clean up thread executors
- PR #4294[3006] - Logging revamp
- PR #4292[3007] - Remove SPDX tag from Boost License file to allow for github to recognize it
- PR #4291[3008] - Add format support for std::tm
- PR #4290[3009] - Simplify compatible tuples check
- PR #4288[3010] - A lightweight take on boost::lexical_cast
- PR #4287[3011] - Forking boost::lexical_cast as a new module
- PR #4277[3012] - MPI_futures
- PR #4270[3013] - Refactor future implementation
- PR #4265[3014] - Threading module
- PR #4259[3015] - Module naming base
- PR #4251[3016] - Local workrequesting scheduler
- PR #4250[3017] - Inline execution of scoped tasks, if possible
- PR #4247[3018] - Add execution in module headers
- PR #4246[3019] - Expose CMake targets officially
- PR #4239[3020] - Doc updates miscellaneous (partially completed during Google Season of Docs)
- PR #4233[3021] - Remove project() from modules + fix CMAKE_SOURCE_DIR issue
- PR #4231[3022] - Module local lcos
- PR #4207[3023] - Command line handling module
- PR #4206[3024] - Runtime configuration module
- PR #4141[3025] - Doc updates examples local to remote (partially completed during Google Season of Docs)
- PR #4091[3026] - Split runtime into local and distributed parts

[3005] https://github.com/TheHPXProject/hpx/pull/4301
[3006] https://github.com/TheHPXProject/hpx/pull/4294
[3007] https://github.com/TheHPXProject/hpx/pull/4292
[3008] https://github.com/TheHPXProject/hpx/pull/4291
[3009] https://github.com/TheHPXProject/hpx/pull/4290
[3010] https://github.com/TheHPXProject/hpx/pull/4288
[3011] https://github.com/TheHPXProject/hpx/pull/4287
[3012] https://github.com/TheHPXProject/hpx/pull/4277
[3013] https://github.com/TheHPXProject/hpx/pull/4270
[3014] https://github.com/TheHPXProject/hpx/pull/4265
[3015] https://github.com/TheHPXProject/hpx/pull/4259
[3016] https://github.com/TheHPXProject/hpx/pull/4251
[3017] https://github.com/TheHPXProject/hpx/pull/4250
[3018] https://github.com/TheHPXProject/hpx/pull/4247
[3019] https://github.com/TheHPXProject/hpx/pull/4246
[3020] https://github.com/TheHPXProject/hpx/pull/4239
[3021] https://github.com/TheHPXProject/hpx/pull/4233
[3022] https://github.com/TheHPXProject/hpx/pull/4231
[3023] https://github.com/TheHPXProject/hpx/pull/4207
[3024] https://github.com/TheHPXProject/hpx/pull/4206
[3025] https://github.com/TheHPXProject/hpx/pull/4141
[3026] https://github.com/TheHPXProject/hpx/pull/4091

- PR #4017[3027] - Require C++14

HPX V1.4.1 (Feb 12, 2020)

General changes

This is a bugfix release. It contains the following changes:

- Fix compilation issues on Windows, macOS, FreeBSD, and with gcc 10
- Install missing `pdb` files on Windows
- Allow running tests using an installed version of *HPX*
- Skip MPI finalization if HPX has not initialized MPI
- Give a hard error when attempting to use IO counters on Windows

Closed issues

- Issue #4320[3028] - HPX 1.4.0 does not compile with gcc 10
- Issue #4336[3029] - Building HPX 1.4.0 with IO Counters breaks (Windows)
- Issue #4334[3030] - HPX `Debug` and `RelWithDebinfo` builds on Windows not installing `.pdb` files
- Issue #4322[3031] - Undefine VT1 and VT2 after boost includes
- Issue #4314[3032] - Compile error on 1.4.0
- Issue #4307[3033] - `ld: error: duplicate symbol: freebsd_environ`

Closed pull requests

- PR #4376[3034] - Attempt to fix some test build errors on Windows
- PR #4357[3035] - Adding missing `#include`s to fix gcc V10 linker problems
- PR #4353[3036] - Skip `MPI_Finalize` if `MPI_Init` is not called from HPX
- PR #4343[3037] - Give a hard error if IO counters are enabled on non-Linux systems
- PR #4337[3038] - Installing pdb files on Windows
- PR #4335[3039] - Adding capability to buildsystem to use an installed version of HPX

[3027] https://github.com/TheHPXProject/hpx/pull/4017
[3028] https://github.com/TheHPXProject/hpx/issues/4320
[3029] https://github.com/TheHPXProject/hpx/issues/4336
[3030] https://github.com/TheHPXProject/hpx/issues/4334
[3031] https://github.com/TheHPXProject/hpx/issues/4322
[3032] https://github.com/TheHPXProject/hpx/issues/4314
[3033] https://github.com/TheHPXProject/hpx/issues/4307
[3034] https://github.com/TheHPXProject/hpx/pull/4376
[3035] https://github.com/TheHPXProject/hpx/pull/4357
[3036] https://github.com/TheHPXProject/hpx/pull/4353
[3037] https://github.com/TheHPXProject/hpx/pull/4343
[3038] https://github.com/TheHPXProject/hpx/pull/4337
[3039] https://github.com/TheHPXProject/hpx/pull/4335

- PR #4315[3040] - Forcing exported symbols from composable_guard to be linked into core library
- PR #4310[3041] - Remove environment handling from `exception.cpp`

HPX V1.4.0 (January 15, 2020)

General changes

- We have added the collectives `all_to_all` and `all_reduce`.
- We have added APIs for resiliency, which allows replication and replay for failed tasks. See the *documentation* for more details.
- Components can now be checkpointed.
- Performance improvements to schedulers and coroutines. A significant change is the addition of stackless coroutines. These are to be used for tasks that do not need to be suspended and can reduce overheads noticeably in applications with short tasks. A stackless coroutine can be created with the new stack size `thread_stacksize_nostack`.
- We have added an implementation of `unique_any`, which is a non-copyable version of `any`.
- The `shared_priority_queue_scheduler` has been improved. It now has lower overheads than the default scheduler in many situations. Unlike the default scheduler it fully supports NUMA scheduling hints. Enable it with the command line option `--hpx:queuing=shared-priority`. This scheduler should still be considered experimental, but its use is encouraged in real applications to help us make it production ready.
- We have added the performance counters `background-receive-duration` and `background-receive-overhead` for inspecting the time and overhead spent on receiving parcels in the background.
- Compilation time has been further improved when `HPX_WITH_NETWORKING=OFF`.
- We no longer require compiled Boost dependencies in certain configurations. This requires at least Boost 1.70, compiling on x86 with GCC 9, clang (libc++) 9, or VS2019 in C++17 mode. The dependency on Boost.Filesystem can explicitly be turned on with `HPX_FILESYSTEM_WITH_BOOST_FILESYSTEM_COMPATIBILITY=ON` (it is off by default if the standard library supports `std::filesystem`). Boost.ProgramOptions has been copied into the HPX repository. We have a compatibility layer for users who must explicitly use Boost.ProgramOptions instead of the ProgramOptions provided by HPX. To remove the dependency `HPX_PROGRAM_OPTIONS_WITH_BOOST_PROGRAM_OPTIONS_COMPATIBILITY` must be explicitly set to `OFF`. This option will be removed in a future release. We have also removed several other header-only dependencies on Boost.
- It is now possible to use the process affinity mask set by tools like `numactl` and various batch environments with the command line option `--hpx:use-process-mask`. Enabling this option implies `--hpx:ignore-batch-env`.
- It is now possible to create standalone thread pools without starting the runtime. See the `standalone_thread_pool_executor.cpp` test in the `execution` module for an example.

[3040] https://github.com/TheHPXProject/hpx/pull/4315
[3041] https://github.com/TheHPXProject/hpx/pull/4310

- Tasks annotated with `hpx::util::annotated_function` now have their correct name when using APEX to generate OTF2 files.
- Cloning of APEX was defective in previous releases (it required manual intervention to check out the correct tag or branch). This has been fixed.
- The option `HPX_WITH_MORE_THAN_64_THREADS` is now ignored and will be removed in a future release. The value is instead derived directly from `HPX_WITH_MAX_CPU_COUNT` option.
- We have deprecated compiling in C++11 mode. The next release will require a C++14 capable compiler.
- We have deprecated support for the Vc library. This option will be replaced with SIMD support from the standard library in a future release.
- We have significantly refactored our CMake setup. This is intended to be a non-breaking change and will allow for using HPX through CMake targets in the future.
- We have continued modularizing the HPX library. In the process we have rearranged many header files into module-specific directories. All moved headers have compatibility headers which forward from the old location to the new location, together with a deprecation warning. The compatibility headers will eventually be removed.
- We now enforce formatting with `clang-format` on the majority of our source files.
- We have added SPDX license tags to all files.
- Many bugfixes.

Breaking changes

- The `HPX_WITH_THREAD_COMPATIBILITY` option and the associated compatibility layer has been removed.
- The `HPX_WITH_INCLUSIVE_SCAN_COMPATIBILITY` option and the associated compatibility layer has been removed.
- The `HPX_WITH_UNWRAPPED_COMPATIBLITY` option and the associated compatibility layer has been removed.

Closed issues

- Issue #4282[3042] - Build Issues with Release on Windows
- Issue #4278[3043] - Build Issues with CMake 3.14.4
- Issue #4273[3044] - Clients of HPX 1.4.0-rc2 with APEX ar not linked to libhpx-apex
- Issue #4269[3045] - Building HPX 1.4.0-rc2 with support for APEX fails
- Issue #4263[3046] - Compilation fail on latest master
- Issue #4232[3047] - Configure of HPX project using CMake FetchContent fails

[3042] https://github.com/TheHPXProject/hpx/issues/4282
[3043] https://github.com/TheHPXProject/hpx/issues/4278
[3044] https://github.com/TheHPXProject/hpx/issues/4273
[3045] https://github.com/TheHPXProject/hpx/issues/4269
[3046] https://github.com/TheHPXProject/hpx/issues/4263
[3047] https://github.com/TheHPXProject/hpx/issues/4232

- Issue #4223[3048] - "Re-using the main() function as the main HPX entry point" doesn't work
- Issue #4220[3049] - HPX won't compile - error building `resource_partitioner`
- Issue #4215[3050] - HPX 1.4.0rc1 does not link on s390x
- Issue #4204[3051] - Trouble compiling HPX with Intel compiler
- Issue #4199[3052] - Refactor APEX to eliminate circular dependency
- Issue #4187[3053] - HPX can't build on OSX
- Issue #4185[3054] - Simple debug output for development
- Issue #4182[3055] - `@HPX_CONF_PREFIX@` is the empty string
- Issue #4169[3056] - HPX won't build with APEX
- Issue #4163[3057] - Add back `HPX_LIBRARIES` and `HPX_INCLUDE_DIRS`
- Issue #4161[3058] - It should be possible to call `find_package(HPX)` multiple times
- Issue #4155[3059] - `get_self_id()` for stackless threads returns `invalid_thread_id`
- Issue #4151[3060] - build error with MPI code
- Issue #4150[3061] - hpx won't build on POWER9 with clang 8
- Issue #4148[3062] - `cacheline_data` delivers poor performance with C++17 compared to C++14
- Issue #4144[3063] - target general in `HPX_LIBRARIES` does not exist
- Issue #4134[3064] - CMake Error when `-DHPX_WITH_HPXMP=ON`
- Issue #4132[3065] - parallel fill leaves elements unfilled
- Issue #4123[3066] - PAPI performance counters are inaccessible
- Issue #4118[3067] - `static_chunk_size` is not obeyed in scan algorithms
- Issue #4115[3068] - dependency chaining error with APEX
- Issue #4107[3069] - Initializing runtime without entry point function and command line arguments

[3048] https://github.com/TheHPXProject/hpx/issues/4223
[3049] https://github.com/TheHPXProject/hpx/issues/4220
[3050] https://github.com/TheHPXProject/hpx/issues/4215
[3051] https://github.com/TheHPXProject/hpx/issues/4204
[3052] https://github.com/TheHPXProject/hpx/issues/4199
[3053] https://github.com/TheHPXProject/hpx/issues/4187
[3054] https://github.com/TheHPXProject/hpx/issues/4185
[3055] https://github.com/TheHPXProject/hpx/issues/4182
[3056] https://github.com/TheHPXProject/hpx/issues/4169
[3057] https://github.com/TheHPXProject/hpx/issues/4163
[3058] https://github.com/TheHPXProject/hpx/issues/4161
[3059] https://github.com/TheHPXProject/hpx/issues/4155
[3060] https://github.com/TheHPXProject/hpx/issues/4151
[3061] https://github.com/TheHPXProject/hpx/issues/4150
[3062] https://github.com/TheHPXProject/hpx/issues/4148
[3063] https://github.com/TheHPXProject/hpx/issues/4144
[3064] https://github.com/TheHPXProject/hpx/issues/4134
[3065] https://github.com/TheHPXProject/hpx/issues/4132
[3066] https://github.com/TheHPXProject/hpx/issues/4123
[3067] https://github.com/TheHPXProject/hpx/issues/4118
[3068] https://github.com/TheHPXProject/hpx/issues/4115
[3069] https://github.com/TheHPXProject/hpx/issues/4107

- Issue #4105[3070] - Bug in `hpx:bind=numa-balanced`
- Issue #4101[3071] - Bound tasks
- Issue #4100[3072] - Add SPDX identifier to all files
- Issue #4085[3073] - `hpx_topology` library should depend on hwloc
- Issue #4067[3074] - HPX fails to build on macOS
- Issue #4056[3075] - Building without thread manager idle backoff fails
- Issue #4052[3076] - Enforce `clang-format` style for modules
- Issue #4032[3077] - Simple hello world fails to launch correctly
- Issue #4030[3078] - Allow threads to skip context switching
- Issue #4029[3079] - Add support for mimalloc
- Issue #4005[3080] - Can't link HPX when APEX enabled
- Issue #4002[3081] - Missing header for algorithm module
- Issue #3989[3082] - conversion from `long` to `unsigned int` requires a narrowing conversion on MSVC
- Issue #3958[3083] - `/statistics/average@` perf counter can't be created
- Issue #3953[3084] - CMake errors from `HPX_AddPseudoDependencies`
- Issue #3941[3085] - CMake error for APEX install target
- Issue #3940[3086] - Convert pseudo-doxygen function documentation into actual doxygen documentation
- Issue #3935[3087] - HPX compiler match too strict?
- Issue #3929[3088] - Buildbot failures on latest HPX stable
- Issue #3912[3089] - I recommend publishing a version that does not depend on the boost library
- Issue #3890[3090] - `hpx.ini` not working
- Issue #3883[3091] - cuda compilation fails because of `-faligned-new`

[3070] https://github.com/TheHPXProject/hpx/issues/4105
[3071] https://github.com/TheHPXProject/hpx/issues/4101
[3072] https://github.com/TheHPXProject/hpx/issues/4100
[3073] https://github.com/TheHPXProject/hpx/issues/4085
[3074] https://github.com/TheHPXProject/hpx/issues/4067
[3075] https://github.com/TheHPXProject/hpx/issues/4056
[3076] https://github.com/TheHPXProject/hpx/issues/4052
[3077] https://github.com/TheHPXProject/hpx/issues/4032
[3078] https://github.com/TheHPXProject/hpx/issues/4030
[3079] https://github.com/TheHPXProject/hpx/issues/4029
[3080] https://github.com/TheHPXProject/hpx/issues/4005
[3081] https://github.com/TheHPXProject/hpx/issues/4002
[3082] https://github.com/TheHPXProject/hpx/issues/3989
[3083] https://github.com/TheHPXProject/hpx/issues/3958
[3084] https://github.com/TheHPXProject/hpx/issues/3953
[3085] https://github.com/TheHPXProject/hpx/issues/3941
[3086] https://github.com/TheHPXProject/hpx/issues/3940
[3087] https://github.com/TheHPXProject/hpx/issues/3935
[3088] https://github.com/TheHPXProject/hpx/issues/3929
[3089] https://github.com/TheHPXProject/hpx/issues/3912
[3090] https://github.com/TheHPXProject/hpx/issues/3890
[3091] https://github.com/TheHPXProject/hpx/issues/3883

- Issue #3879[3092] - HPX fails to configure with `-DHPX_WITH_TESTS=OFF`
- Issue #3871[3093] - `dataflow` does not support void allocators
- Issue #3867[3094] - Latest HTML docs placed in wrong directory on GitHub pages
- Issue #3866[3095] - Make sure all tests use `HPX_TEST*` macros and not `HPX_ASSERT`
- Issue #3857[3096] - CMake all-keyword or all-plain for `target_link_libraries`
- Issue #3856[3097] - `hpx_setup_target` adds rogue flags
- Issue #3850[3098] - HPX fails to build on POWER8 with Clang7
- Issue #3848[3099] - Remove `lva` member from `thread_init_data`
- Issue #3838[3100] - `hpx::parallel::count/count_if` failing tests
- Issue #3651[3101] - `hpx::parallel::transform_reduce` with non const reference as lambda parameter
- Issue #3560[3102] - Apex integration with HPX not working properly
- Issue #3322[3103] - No warning when mixing debug/release builds

Closed pull requests

- PR #4300[3104] - Checks for `MPI_Init` being called twice
- PR #4299[3105] - Small CMake fixes
- PR #4298[3106] - Remove extra call to annotate function that messes up traces
- PR #4296[3107] - Fixing collectives locking problem
- PR #4295[3108] - Do not check `LICENSE_1_0.txt` for inspect violations
- PR #4293[3109] - Applying two small changes fixing carious MSVC/Windows problems
- PR #4285[3110] - Delete `apex.hpp`
- PR #4276[3111] - Disable doxygen generation for `hpx/debugging/print.hpp` file
- PR #4275[3112] - Make sure APEX is linked to even when not explicitly referenced

[3092] https://github.com/TheHPXProject/hpx/issues/3879
[3093] https://github.com/TheHPXProject/hpx/issues/3871
[3094] https://github.com/TheHPXProject/hpx/issues/3867
[3095] https://github.com/TheHPXProject/hpx/issues/3866
[3096] https://github.com/TheHPXProject/hpx/issues/3857
[3097] https://github.com/TheHPXProject/hpx/issues/3856
[3098] https://github.com/TheHPXProject/hpx/issues/3850
[3099] https://github.com/TheHPXProject/hpx/issues/3848
[3100] https://github.com/TheHPXProject/hpx/issues/3838
[3101] https://github.com/TheHPXProject/hpx/issues/3651
[3102] https://github.com/TheHPXProject/hpx/issues/3560
[3103] https://github.com/TheHPXProject/hpx/issues/3322
[3104] https://github.com/TheHPXProject/hpx/pull/4300
[3105] https://github.com/TheHPXProject/hpx/pull/4299
[3106] https://github.com/TheHPXProject/hpx/pull/4298
[3107] https://github.com/TheHPXProject/hpx/pull/4296
[3108] https://github.com/TheHPXProject/hpx/pull/4295
[3109] https://github.com/TheHPXProject/hpx/pull/4293
[3110] https://github.com/TheHPXProject/hpx/pull/4285
[3111] https://github.com/TheHPXProject/hpx/pull/4276
[3112] https://github.com/TheHPXProject/hpx/pull/4275

- PR #4272[3113] - Fix pushing of documentation
- PR #4271[3114] - Updating APEX tag, don't create new task_wrapper on `operator=` of hpx_thread object
- PR #4268[3115] - Testing for noexcept function specializations in C++11/14 mode
- PR #4267[3116] - Fixing MSVC warning
- PR #4266[3117] - Make sure macOS Travis CI fails if build step fails
- PR #4264[3118] - Clean up compatibility header options
- PR #4262[3119] - Cleanup modules `CMakeLists.txt`
- PR #4261[3120] - Fixing HPX/APEX linking and dependencies for external projects like Phylanx
- PR #4260[3121] - Fix docs compilation problems
- PR #4258[3122] - Couple of minor changes
- PR #4257[3123] - Fix apex annotation for async dispatch
- PR #4256[3124] - Remove lambdas from assert expressions
- PR #4255[3125] - Ignoring lock in `all_to_all` and `all_reduce`
- PR #4254[3126] - Adding action specializations for noexcept functions
- PR #4253[3127] - Move `partlit.hpp` to affinity module
- PR #4252[3128] - Make mismatching build types a hard error in CMake
- PR #4249[3129] - Scheduler improvement
- PR #4248[3130] - update hpxmp tag to v0.3.0
- PR #4245[3131] - Adding high performance channels
- PR #4244[3132] - Ignore lock in ignore_while_locked_1485 test
- PR #4243[3133] - Fix PAPI command line option documentation
- PR #4242[3134] - Ignore lock in target_distribution_policy

[3113] https://github.com/TheHPXProject/hpx/pull/4272
[3114] https://github.com/TheHPXProject/hpx/pull/4271
[3115] https://github.com/TheHPXProject/hpx/pull/4268
[3116] https://github.com/TheHPXProject/hpx/pull/4267
[3117] https://github.com/TheHPXProject/hpx/pull/4266
[3118] https://github.com/TheHPXProject/hpx/pull/4264
[3119] https://github.com/TheHPXProject/hpx/pull/4262
[3120] https://github.com/TheHPXProject/hpx/pull/4261
[3121] https://github.com/TheHPXProject/hpx/pull/4260
[3122] https://github.com/TheHPXProject/hpx/pull/4258
[3123] https://github.com/TheHPXProject/hpx/pull/4257
[3124] https://github.com/TheHPXProject/hpx/pull/4256
[3125] https://github.com/TheHPXProject/hpx/pull/4255
[3126] https://github.com/TheHPXProject/hpx/pull/4254
[3127] https://github.com/TheHPXProject/hpx/pull/4253
[3128] https://github.com/TheHPXProject/hpx/pull/4252
[3129] https://github.com/TheHPXProject/hpx/pull/4249
[3130] https://github.com/TheHPXProject/hpx/pull/4248
[3131] https://github.com/TheHPXProject/hpx/pull/4245
[3132] https://github.com/TheHPXProject/hpx/pull/4244
[3133] https://github.com/TheHPXProject/hpx/pull/4243
[3134] https://github.com/TheHPXProject/hpx/pull/4242

- PR #4241[3135] - Fix `start_stop_callbacks` test
- PR #4240[3136] - Mostly fix clang CUDA compilation
- PR #4238[3137] - Google Season of Docs updates to documentation; grammar edits.
- PR #4237[3138] - fixing annotated task to use the name, not the desc
- PR #4236[3139] - Move module print summary to modules
- PR #4235[3140] - Don't use alignas in `cache_{aligned,line}_data`
- PR #4234[3141] - Add basic overview sentence to all modules
- PR #4230[3142] - Add OS X builds to Travis CI
- PR #4229[3143] - Remove leftover queue compatibility checks
- PR #4226[3144] - Fixing APEX shutdown by explicitly shutting down throttling
- PR #4225[3145] - Allow `CMAKE_INSTALL_PREFIX` to be a relative path
- PR #4224[3146] - Deprecate verbs parcelport
- PR #4222[3147] - Update `register_{thread,work}` namespaces
- PR #4221[3148] - Changing `HPX_GCC_VERSION` check from `70000` to `70300`
- PR #4218[3149] - Google Season of Docs updates to documentation; grammar edits.
- PR #4217[3150] - Google Season of Docs updates to documentation; grammar edits.
- PR #4216[3151] - Fixing gcc warning on 32bit platforms (integer truncation)
- PR #4214[3152] - Apex callback refactoring
- PR #4213[3153] - Clean up allocator checks for dependent projects
- PR #4212[3154] - Google Season of Docs updates to documentation; grammar edits.
- PR #4211[3155] - Google Season of Docs updates to documentation; contributing to hpx
- PR #4210[3156] - Attempting to fix Intel compilation
- PR #4209[3157] - Fix CUDA 10 build

[3135] https://github.com/TheHPXProject/hpx/pull/4241
[3136] https://github.com/TheHPXProject/hpx/pull/4240
[3137] https://github.com/TheHPXProject/hpx/pull/4238
[3138] https://github.com/TheHPXProject/hpx/pull/4237
[3139] https://github.com/TheHPXProject/hpx/pull/4236
[3140] https://github.com/TheHPXProject/hpx/pull/4235
[3141] https://github.com/TheHPXProject/hpx/pull/4234
[3142] https://github.com/TheHPXProject/hpx/pull/4230
[3143] https://github.com/TheHPXProject/hpx/pull/4229
[3144] https://github.com/TheHPXProject/hpx/pull/4226
[3145] https://github.com/TheHPXProject/hpx/pull/4225
[3146] https://github.com/TheHPXProject/hpx/pull/4224
[3147] https://github.com/TheHPXProject/hpx/pull/4222
[3148] https://github.com/TheHPXProject/hpx/pull/4221
[3149] https://github.com/TheHPXProject/hpx/pull/4218
[3150] https://github.com/TheHPXProject/hpx/pull/4217
[3151] https://github.com/TheHPXProject/hpx/pull/4216
[3152] https://github.com/TheHPXProject/hpx/pull/4214
[3153] https://github.com/TheHPXProject/hpx/pull/4213
[3154] https://github.com/TheHPXProject/hpx/pull/4212
[3155] https://github.com/TheHPXProject/hpx/pull/4211
[3156] https://github.com/TheHPXProject/hpx/pull/4210
[3157] https://github.com/TheHPXProject/hpx/pull/4209

- PR #4205[3158] - Making sure that differences in CMAKE_BUILD_TYPE are not reported on multi-configuration cmake generators
- PR #4203[3159] - Deprecate Vc
- PR #4202[3160] - Fix CUDA configuration
- PR #4200[3161] - Making sure hpx_wrap is not passed on to linker on non-Linux systems
- PR #4198[3162] - Fix execution_agent.cpp compilation with GCC 5
- PR #4197[3163] - Remove deprecated options for 1.4.0 release
- PR #4196[3164] - minor fixes for building on OSX Darwin
- PR #4195[3165] - Use full clone on CircleCI for pushing stable tag
- PR #4193[3166] - Add scheduling hints to hello_world_distributed
- PR #4192[3167] - Set up CUDA in HPXConfig.cmake
- PR #4191[3168] - Export allocators root variables
- PR #4190[3169] - Don't use constexpr in thread_data with GCC <= 6
- PR #4189[3170] - Only use quick_exit if available
- PR #4188[3171] - Google Season of Docs updates to documentation; writing single node hpx applications
- PR #4186[3172] - correct vc to cuda in cuda cmake
- PR #4184[3173] - Resetting some cached variables to make sure those are re-filled
- PR #4183[3174] - Fix hpxcxx configuration
- PR #4181[3175] - Rename base libraries var
- PR #4180[3176] - Move header left behind earlier to plugin module
- PR #4179[3177] - Moving zip_iterator and transform_iterator to iterator_support module
- PR #4178[3178] - Move checkpointing support to its own module
- PR #4177[3179] - Small const fix to basic_execution module

[3158] https://github.com/TheHPXProject/hpx/pull/4205
[3159] https://github.com/TheHPXProject/hpx/pull/4203
[3160] https://github.com/TheHPXProject/hpx/pull/4202
[3161] https://github.com/TheHPXProject/hpx/pull/4200
[3162] https://github.com/TheHPXProject/hpx/pull/4198
[3163] https://github.com/TheHPXProject/hpx/pull/4197
[3164] https://github.com/TheHPXProject/hpx/pull/4196
[3165] https://github.com/TheHPXProject/hpx/pull/4195
[3166] https://github.com/TheHPXProject/hpx/pull/4193
[3167] https://github.com/TheHPXProject/hpx/pull/4192
[3168] https://github.com/TheHPXProject/hpx/pull/4191
[3169] https://github.com/TheHPXProject/hpx/pull/4190
[3170] https://github.com/TheHPXProject/hpx/pull/4189
[3171] https://github.com/TheHPXProject/hpx/pull/4188
[3172] https://github.com/TheHPXProject/hpx/pull/4186
[3173] https://github.com/TheHPXProject/hpx/pull/4184
[3174] https://github.com/TheHPXProject/hpx/pull/4183
[3175] https://github.com/TheHPXProject/hpx/pull/4181
[3176] https://github.com/TheHPXProject/hpx/pull/4180
[3177] https://github.com/TheHPXProject/hpx/pull/4179
[3178] https://github.com/TheHPXProject/hpx/pull/4178
[3179] https://github.com/TheHPXProject/hpx/pull/4177

- PR #4176[3180] - Add back `HPX_LIBRARIES` and friends to `HPXConfig.cmake`
- PR #4175[3181] - Make Vc public and add it to `HPXConfig.cmake`
- PR #4173[3182] - Wait for runtime to be running before returning from hpx::start
- PR #4172[3183] - More protection against shutdown problems in error handling scenarios.
- PR #4171[3184] - Ignore lock in `condition_variable::wait`
- PR #4170[3185] - Adding APEX dependency to MPI parcelport
- PR #4168[3186] - Adding utility include
- PR #4167[3187] - Add a condition to setup the external libraries
- PR #4166[3188] - Add an `INTERNAL_FLAGS` option to link to `hpx_internal_flags`
- PR #4165[3189] - Forward `HPX_*` cmake cache variables to external projects
- PR #4164[3190] - Affinity and batch environment modules
- PR #4162[3191] - Handle `quick exit`
- PR #4160[3192] - Using `target_link_libraries` for cmake versions >= 3.12
- PR #4159[3193] - Make sure `HPX_WITH_NATIVE_TLS` is forwarded to dependent projects
- PR #4158[3194] - Adding allocator imported target as a dependency of allocator module
- PR #4157[3195] - Add `hpx_memory` as a dependency of parcelport plugins
- PR #4156[3196] - Stackless coroutines now can refer to themselves (through get_self() and friends)
- PR #4154[3197] - Added CMake policy CMP0060 for HPX applications.
- PR #4153[3198] - add header `iomanip` to tests and tool
- PR #4152[3199] - Casting MPI tag value
- PR #4149[3200] - Add back private `m_desc` member variable in program_options module
- PR #4147[3201] - Resource partitioner and threadmanager modules
- PR #4146[3202] - Google Season of Docs updates to documentation; creating hpx projects

[3180] https://github.com/TheHPXProject/hpx/pull/4176
[3181] https://github.com/TheHPXProject/hpx/pull/4175
[3182] https://github.com/TheHPXProject/hpx/pull/4173
[3183] https://github.com/TheHPXProject/hpx/pull/4172
[3184] https://github.com/TheHPXProject/hpx/pull/4171
[3185] https://github.com/TheHPXProject/hpx/pull/4170
[3186] https://github.com/TheHPXProject/hpx/pull/4168
[3187] https://github.com/TheHPXProject/hpx/pull/4167
[3188] https://github.com/TheHPXProject/hpx/pull/4166
[3189] https://github.com/TheHPXProject/hpx/pull/4165
[3190] https://github.com/TheHPXProject/hpx/pull/4164
[3191] https://github.com/TheHPXProject/hpx/pull/4162
[3192] https://github.com/TheHPXProject/hpx/pull/4160
[3193] https://github.com/TheHPXProject/hpx/pull/4159
[3194] https://github.com/TheHPXProject/hpx/pull/4158
[3195] https://github.com/TheHPXProject/hpx/pull/4157
[3196] https://github.com/TheHPXProject/hpx/pull/4156
[3197] https://github.com/TheHPXProject/hpx/pull/4154
[3198] https://github.com/TheHPXProject/hpx/pull/4153
[3199] https://github.com/TheHPXProject/hpx/pull/4152
[3200] https://github.com/TheHPXProject/hpx/pull/4149
[3201] https://github.com/TheHPXProject/hpx/pull/4147
[3202] https://github.com/TheHPXProject/hpx/pull/4146

- PR #4145[3203] - Adding basic support for stackless threads
- PR #4143[3204] - Exclude `test_client_1950` from all target
- PR #4142[3205] - Add a new `thread_pool_executor`
- PR #4140[3206] - Google Season of Docs updates to documentation; why hpx
- PR #4139[3207] - Remove runtime includes from coroutines module
- PR #4138[3208] - Forking `boost::intrusive_ptr` and adding it as `hpx::intrusive_ptr`
- PR #4137[3209] - Fixing TSS destruction
- PR #4136[3210] - HPX.Compute modules
- PR #4133[3211] - Fix `block_executor`
- PR #4131[3212] - Applying fixes based on reports from PVS Studio
- PR #4130[3213] - Adding missing header to build system
- PR #4129[3214] - Fixing compilation if `HPX_WITH_DATAPAR_VC` is enabled
- PR #4128[3215] - Renaming `moveonly_any` to `unique_any`
- PR #4126[3216] - Attempt to fix `basic_any` constructor for gcc 7
- PR #4125[3217] - Changing `extra_archive_data` implementation
- PR #4124[3218] - Don't link to Boost.System unless required
- PR #4122[3219] - Add kernel launch helper utility (+saxpy demo) and merge in octotiger changes
- PR #4121[3220] - Fixing migration test if networking is disabled.
- PR #4120[3221] - Google Season of Docs updates to documentation; hpx build system v1
- PR #4119[3222] - Making sure `chunk_size` and `max_chunk` are actually applied to parallel algorithms if specified
- PR #4117[3223] - Make CircleCI formatting check store diff
- PR #4116[3224] - Fix automatically setting C++ standard

[3203] https://github.com/TheHPXProject/hpx/pull/4145
[3204] https://github.com/TheHPXProject/hpx/pull/4143
[3205] https://github.com/TheHPXProject/hpx/pull/4142
[3206] https://github.com/TheHPXProject/hpx/pull/4140
[3207] https://github.com/TheHPXProject/hpx/pull/4139
[3208] https://github.com/TheHPXProject/hpx/pull/4138
[3209] https://github.com/TheHPXProject/hpx/pull/4137
[3210] https://github.com/TheHPXProject/hpx/pull/4136
[3211] https://github.com/TheHPXProject/hpx/pull/4133
[3212] https://github.com/TheHPXProject/hpx/pull/4131
[3213] https://github.com/TheHPXProject/hpx/pull/4130
[3214] https://github.com/TheHPXProject/hpx/pull/4129
[3215] https://github.com/TheHPXProject/hpx/pull/4128
[3216] https://github.com/TheHPXProject/hpx/pull/4126
[3217] https://github.com/TheHPXProject/hpx/pull/4125
[3218] https://github.com/TheHPXProject/hpx/pull/4124
[3219] https://github.com/TheHPXProject/hpx/pull/4122
[3220] https://github.com/TheHPXProject/hpx/pull/4121
[3221] https://github.com/TheHPXProject/hpx/pull/4120
[3222] https://github.com/TheHPXProject/hpx/pull/4119
[3223] https://github.com/TheHPXProject/hpx/pull/4117
[3224] https://github.com/TheHPXProject/hpx/pull/4116

- PR #4114[3225] - Module serialization
- PR #4113[3226] - Module datastructures
- PR #4111[3227] - Fixing performance regression introduced earlier
- PR #4110[3228] - Adding missing SPDX tags
- PR #4109[3229] - Overload for start without entry point/argv.
- PR #4108[3230] - Making sure C++ standard is properly detected and propagated
- PR #4106[3231] - use `std::round` for guaranteed rounding without errors
- PR #4104[3232] - Extend `scheduler_mode` with new `work_stealing` and task assignment modes
- PR #4103[3233] - Add this to lambda capture list
- PR #4102[3234] - Add spdx license and check
- PR #4099[3235] - Module coroutines
- PR #4098[3236] - Fix append module path in module CMakeLists template
- PR #4097[3237] - Function tests
- PR #4096[3238] - Removing return of `thread_result_type` from functions not needing them
- PR #4095[3239] - Stop-gap measure until cmake overhaul is in place
- PR #4094[3240] - Deprecate `HPX_WITH_MORE_THAN_64_THREADS`
- PR #4093[3241] - Fix initialization of `global_num_tasks` in `parallel_executor`
- PR #4092[3242] - Add support for mi-malloc
- PR #4090[3243] - Execution context
- PR #4089[3244] - Make counters in coroutines optional
- PR #4087[3245] - Making `hpx::util::any` compatible with C++17
- PR #4084[3246] - Making sure destination array for `std::transform` is properly resized

[3225] https://github.com/TheHPXProject/hpx/pull/4114
[3226] https://github.com/TheHPXProject/hpx/pull/4113
[3227] https://github.com/TheHPXProject/hpx/pull/4111
[3228] https://github.com/TheHPXProject/hpx/pull/4110
[3229] https://github.com/TheHPXProject/hpx/pull/4109
[3230] https://github.com/TheHPXProject/hpx/pull/4108
[3231] https://github.com/TheHPXProject/hpx/pull/4106
[3232] https://github.com/TheHPXProject/hpx/pull/4104
[3233] https://github.com/TheHPXProject/hpx/pull/4103
[3234] https://github.com/TheHPXProject/hpx/pull/4102
[3235] https://github.com/TheHPXProject/hpx/pull/4099
[3236] https://github.com/TheHPXProject/hpx/pull/4098
[3237] https://github.com/TheHPXProject/hpx/pull/4097
[3238] https://github.com/TheHPXProject/hpx/pull/4096
[3239] https://github.com/TheHPXProject/hpx/pull/4095
[3240] https://github.com/TheHPXProject/hpx/pull/4094
[3241] https://github.com/TheHPXProject/hpx/pull/4093
[3242] https://github.com/TheHPXProject/hpx/pull/4092
[3243] https://github.com/TheHPXProject/hpx/pull/4090
[3244] https://github.com/TheHPXProject/hpx/pull/4089
[3245] https://github.com/TheHPXProject/hpx/pull/4087
[3246] https://github.com/TheHPXProject/hpx/pull/4084

- PR #4083[3247] - Adapting `thread_queue_mc` to behave even if no 128bit atomics are available
- PR #4082[3248] - Fix compilation on GCC 5
- PR #4081[3249] - Adding option allowing to force using Boost.FileSystem
- PR #4080[3250] - Updating module dependencies
- PR #4079[3251] - Add missing tests for iterator_support module
- PR #4078[3252] - Disable parcel-layer if networking is disabled
- PR #4077[3253] - Add missing include that causes build fails
- PR #4076[3254] - Enable compatibility headers for functional module
- PR #4075[3255] - Coroutines module
- PR #4073[3256] - Use `configure_file` for generated files in modules
- PR #4071[3257] - Fixing MPI detection for PMIx
- PR #4070[3258] - Fix macOS builds
- PR #4069[3259] - Moving more facilities to the collectives module
- PR #4068[3260] - Adding main HPX `#include` directory to modules
- PR #4066[3261] - Switching the use of `message(STATUS "...")` to hpx_info
- PR #4065[3262] - Move Boost.Filesystem handling to filesystem module
- PR #4064[3263] - Fix program_options test with older boost versions
- PR #4062[3264] - The `cpu_features` tool fails to compile on anything but x86 architectures
- PR #4061[3265] - Add `clang-format` checking step for modules
- PR #4060[3266] - Making sure `HPX_IDLE_BACKOFF_TIME_MAX` is always defined (even if its unused)
- PR #4059[3267] - Renaming module `hpx_parallel_executors` into `hpx_execution`
- PR #4058[3268] - Do not build networking tests when networking disabled

---

3247 https://github.com/TheHPXProject/hpx/pull/4083
3248 https://github.com/TheHPXProject/hpx/pull/4082
3249 https://github.com/TheHPXProject/hpx/pull/4081
3250 https://github.com/TheHPXProject/hpx/pull/4080
3251 https://github.com/TheHPXProject/hpx/pull/4079
3252 https://github.com/TheHPXProject/hpx/pull/4078
3253 https://github.com/TheHPXProject/hpx/pull/4077
3254 https://github.com/TheHPXProject/hpx/pull/4076
3255 https://github.com/TheHPXProject/hpx/pull/4075
3256 https://github.com/TheHPXProject/hpx/pull/4073
3257 https://github.com/TheHPXProject/hpx/pull/4071
3258 https://github.com/TheHPXProject/hpx/pull/4070
3259 https://github.com/TheHPXProject/hpx/pull/4069
3260 https://github.com/TheHPXProject/hpx/pull/4068
3261 https://github.com/TheHPXProject/hpx/pull/4066
3262 https://github.com/TheHPXProject/hpx/pull/4065
3263 https://github.com/TheHPXProject/hpx/pull/4064
3264 https://github.com/TheHPXProject/hpx/pull/4062
3265 https://github.com/TheHPXProject/hpx/pull/4061
3266 https://github.com/TheHPXProject/hpx/pull/4060
3267 https://github.com/TheHPXProject/hpx/pull/4059
3268 https://github.com/TheHPXProject/hpx/pull/4058

- PR #4057[3269] - Printing configuration summary for modules as well
- PR #4055[3270] - Google Season of Docs updates to documentation; hpx build systems
- PR #4054[3271] - Add troubleshooting section to manual
- PR #4051[3272] - Add more variations to `future_overhead` test
- PR #4050[3273] - Creating plugin module
- PR #4049[3274] - Move missing modules tests
- PR #4047[3275] - Add boost/filesystem headers to inspect deprecated headers
- PR #4045[3276] - Module functional
- PR #4043[3277] - Fix preconditions and error messages for suspension functions
- PR #4041[3278] - Pass HPX_STANDARD on to dependent projects via HPXConfig.cmake
- PR #4040[3279] - Program options module
- PR #4039[3280] - Moving non-serializable `any` (`any_nonser`) to datastructures module
- PR #4038[3281] - Adding MPark's variant (V1.4.0) to HPX
- PR #4037[3282] - Adding resiliency module
- PR #4036[3283] - Add C++17 filesystem compatibility header
- PR #4035[3284] - Fixing support for mpirun
- PR #4028[3285] - CMake to target based directives
- PR #4027[3286] - Remove GitLab CI configuration
- PR #4026[3287] - Threading refactoring
- PR #4025[3288] - Refactoring thread queue configuration options
- PR #4024[3289] - Fix padding calculation in `cache_aligned_data.hpp`
- PR #4023[3290] - Fixing Codacy issues
- PR #4022[3291] - Make sure process mask option is passed to `affinity_data`

[3269] https://github.com/TheHPXProject/hpx/pull/4057
[3270] https://github.com/TheHPXProject/hpx/pull/4055
[3271] https://github.com/TheHPXProject/hpx/pull/4054
[3272] https://github.com/TheHPXProject/hpx/pull/4051
[3273] https://github.com/TheHPXProject/hpx/pull/4050
[3274] https://github.com/TheHPXProject/hpx/pull/4049
[3275] https://github.com/TheHPXProject/hpx/pull/4047
[3276] https://github.com/TheHPXProject/hpx/pull/4045
[3277] https://github.com/TheHPXProject/hpx/pull/4043
[3278] https://github.com/TheHPXProject/hpx/pull/4041
[3279] https://github.com/TheHPXProject/hpx/pull/4040
[3280] https://github.com/TheHPXProject/hpx/pull/4039
[3281] https://github.com/TheHPXProject/hpx/pull/4038
[3282] https://github.com/TheHPXProject/hpx/pull/4037
[3283] https://github.com/TheHPXProject/hpx/pull/4036
[3284] https://github.com/TheHPXProject/hpx/pull/4035
[3285] https://github.com/TheHPXProject/hpx/pull/4028
[3286] https://github.com/TheHPXProject/hpx/pull/4027
[3287] https://github.com/TheHPXProject/hpx/pull/4026
[3288] https://github.com/TheHPXProject/hpx/pull/4025
[3289] https://github.com/TheHPXProject/hpx/pull/4024
[3290] https://github.com/TheHPXProject/hpx/pull/4023
[3291] https://github.com/TheHPXProject/hpx/pull/4022

- PR #4021[3292] - Warn about compiling in C++11 mode
- PR #4020[3293] - Module concurrency
- PR #4019[3294] - Module topology
- PR #4018[3295] - Update deprecated header in `thread_queue_mc.hpp`
- PR #4015[3296] - Avoid overwriting artifacts
- PR #4014[3297] - Future overheads
- PR #4013[3298] - Update URL to test output conversion script
- PR #4012[3299] - Fix CUDA compilation
- PR #4011[3300] - Fixing cyclic dependencies between modules
- PR #4010[3301] - Ignore stable tag on CircleCI
- PR #4009[3302] - Check circular dependencies in a circle ci step
- PR #4008[3303] - Extend cache aligned data to handle tuple-like data
- PR #4007[3304] - Fixing migration for components that have actions returning a client
- PR #4006[3305] - Move is_value_proxy.hpp to algorithms module
- PR #4004[3306] - Shorten CTest timeout on CircleCI
- PR #4003[3307] - Refactoring to remove (internal) dependencies
- PR #4001[3308] - Exclude tests from all target
- PR #4000[3309] - Module errors
- PR #3999[3310] - Enable support for compatibility headers for logging module
- PR #3998[3311] - Add process thread binding option
- PR #3997[3312] - Export handle_assert function
- PR #3996[3313] - Attempt to solve issue where `-latomic` does not support 128bit atomics
- PR #3993[3314] - Make sure `__LINE__` is an unsigned

3292 https://github.com/TheHPXProject/hpx/pull/4021
3293 https://github.com/TheHPXProject/hpx/pull/4020
3294 https://github.com/TheHPXProject/hpx/pull/4019
3295 https://github.com/TheHPXProject/hpx/pull/4018
3296 https://github.com/TheHPXProject/hpx/pull/4015
3297 https://github.com/TheHPXProject/hpx/pull/4014
3298 https://github.com/TheHPXProject/hpx/pull/4013
3299 https://github.com/TheHPXProject/hpx/pull/4012
3300 https://github.com/TheHPXProject/hpx/pull/4011
3301 https://github.com/TheHPXProject/hpx/pull/4010
3302 https://github.com/TheHPXProject/hpx/pull/4009
3303 https://github.com/TheHPXProject/hpx/pull/4008
3304 https://github.com/TheHPXProject/hpx/pull/4007
3305 https://github.com/TheHPXProject/hpx/pull/4006
3306 https://github.com/TheHPXProject/hpx/pull/4004
3307 https://github.com/TheHPXProject/hpx/pull/4003
3308 https://github.com/TheHPXProject/hpx/pull/4001
3309 https://github.com/TheHPXProject/hpx/pull/4000
3310 https://github.com/TheHPXProject/hpx/pull/3999
3311 https://github.com/TheHPXProject/hpx/pull/3998
3312 https://github.com/TheHPXProject/hpx/pull/3997
3313 https://github.com/TheHPXProject/hpx/pull/3996
3314 https://github.com/TheHPXProject/hpx/pull/3993

- PR #3991[3315] - Fix dependencies and flags for header tests
- PR #3990[3316] - Documentation tags fixes
- PR #3988[3317] - Adding missing solution folder for format module test
- PR #3987[3318] - Move runtime-dependent functions out of command line handling
- PR #3986[3319] - Fix CMake configuration with PAPI on
- PR #3985[3320] - Module timing
- PR #3984[3321] - Fix default behaviour of paths in `add_hpx_component`
- PR #3982[3322] - Parallel executors module
- PR #3981[3323] - Segmented algorithms module
- PR #3980[3324] - Module logging
- PR #3979[3325] - Module util
- PR #3978[3326] - Fix `clang-tidy` step on CircleCI
- PR #3977[3327] - Fixing solution folders for moved components
- PR #3976[3328] - Module format
- PR #3975[3329] - Enable deprecation warnings on CircleCI
- PR #3974[3330] - Fix typos in documentation
- PR #3973[3331] - Fix compilation with GCC 9
- PR #3972[3332] - Add condition to clone apex + use of new cmake var APEX_ROOT
- PR #3971[3333] - Add testing module
- PR #3968[3334] - Remove unneeded file in hardware module
- PR #3967[3335] - Remove leftover PIC settings from main CMakeLists.txt
- PR #3966[3336] - Add missing export option in `add_hpx_module`
- PR #3965[3337] - Change `current_function_helper` back to non-constexpr

[3315] https://github.com/TheHPXProject/hpx/pull/3991
[3316] https://github.com/TheHPXProject/hpx/pull/3990
[3317] https://github.com/TheHPXProject/hpx/pull/3988
[3318] https://github.com/TheHPXProject/hpx/pull/3987
[3319] https://github.com/TheHPXProject/hpx/pull/3986
[3320] https://github.com/TheHPXProject/hpx/pull/3985
[3321] https://github.com/TheHPXProject/hpx/pull/3984
[3322] https://github.com/TheHPXProject/hpx/pull/3982
[3323] https://github.com/TheHPXProject/hpx/pull/3981
[3324] https://github.com/TheHPXProject/hpx/pull/3980
[3325] https://github.com/TheHPXProject/hpx/pull/3979
[3326] https://github.com/TheHPXProject/hpx/pull/3978
[3327] https://github.com/TheHPXProject/hpx/pull/3977
[3328] https://github.com/TheHPXProject/hpx/pull/3976
[3329] https://github.com/TheHPXProject/hpx/pull/3975
[3330] https://github.com/TheHPXProject/hpx/pull/3974
[3331] https://github.com/TheHPXProject/hpx/pull/3973
[3332] https://github.com/TheHPXProject/hpx/pull/3972
[3333] https://github.com/TheHPXProject/hpx/pull/3971
[3334] https://github.com/TheHPXProject/hpx/pull/3968
[3335] https://github.com/TheHPXProject/hpx/pull/3967
[3336] https://github.com/TheHPXProject/hpx/pull/3966
[3337] https://github.com/TheHPXProject/hpx/pull/3965

- PR #3964[3338] - Fixing merge problems
- PR #3962[3339] - Add a trait for `std::array` for unwrapping
- PR #3961[3340] - Making `hpx::util::tuple<Ts...>` and `std::tuple<Ts...>` convertible
- PR #3960[3341] - fix compilation with CUDA 10 and GCC 6
- PR #3959[3342] - Fix C++11 incompatibility
- PR #3957[3343] - Algorithms module
- PR #3956[3344] - [`HPX_AddModule`] Fix lower name var to upper
- PR #3955[3345] - Fix CMake configuration with examples off and tests on
- PR #3954[3346] - Move components to separate subdirectory in root of repository
- PR #3952[3347] - Update `papi.cpp`
- PR #3951[3348] - Exclude modules header tests from all target
- PR #3950[3349] - Adding `all_reduce` facility to collectives module
- PR #3949[3350] - This adds a configuration file that will cause for stale issues to be automatically closed
- PR #3948[3351] - Fixing ALPS environment
- PR #3947[3352] - Add major compiler version check for building hpx as a binary package
- PR #3946[3353] - [Modules] Move the location of the generated headers
- PR #3945[3354] - Simplify tests and examples cmake
- PR #3943[3355] - Remove example module
- PR #3942[3356] - Add `NOEXPORT` option to `add_hpx_{component,library}`
- PR #3938[3357] - Use https for CDash submissions
- PR #3937[3358] - Add `HPX_WITH_BUILD_BINARY_PACKAGE` to the compiler check (refs #3935)
- PR #3936[3359] - Fixing installation of binaries on windows

[3338] https://github.com/TheHPXProject/hpx/pull/3964
[3339] https://github.com/TheHPXProject/hpx/pull/3962
[3340] https://github.com/TheHPXProject/hpx/pull/3961
[3341] https://github.com/TheHPXProject/hpx/pull/3960
[3342] https://github.com/TheHPXProject/hpx/pull/3959
[3343] https://github.com/TheHPXProject/hpx/pull/3957
[3344] https://github.com/TheHPXProject/hpx/pull/3956
[3345] https://github.com/TheHPXProject/hpx/pull/3955
[3346] https://github.com/TheHPXProject/hpx/pull/3954
[3347] https://github.com/TheHPXProject/hpx/pull/3952
[3348] https://github.com/TheHPXProject/hpx/pull/3951
[3349] https://github.com/TheHPXProject/hpx/pull/3950
[3350] https://github.com/TheHPXProject/hpx/pull/3949
[3351] https://github.com/TheHPXProject/hpx/pull/3948
[3352] https://github.com/TheHPXProject/hpx/pull/3947
[3353] https://github.com/TheHPXProject/hpx/pull/3946
[3354] https://github.com/TheHPXProject/hpx/pull/3945
[3355] https://github.com/TheHPXProject/hpx/pull/3943
[3356] https://github.com/TheHPXProject/hpx/pull/3942
[3357] https://github.com/TheHPXProject/hpx/pull/3938
[3358] https://github.com/TheHPXProject/hpx/pull/3937
[3359] https://github.com/TheHPXProject/hpx/pull/3936

- PR #3934[3360] - Add set function for `sliding_semaphore` `max_difference`
- PR #3933[3361] - Remove `cudadevrt` from compile/link flags as it breaks downstream projects
- PR #3932[3362] - Fixing 3929
- PR #3931[3363] - Adding `all_to_all`
- PR #3930[3364] - Add test demonstrating the use of broadcast with component actions
- PR #3928[3365] - fixed number of tasks and number of threads for heterogeneous slurm environments
- PR #3927[3366] - Moving Cache module's tests into separate solution folder
- PR #3926[3367] - Move unit tests to cache module
- PR #3925[3368] - Move version check to config module
- PR #3924[3369] - Add schedule hint executor parameters
- PR #3923[3370] - Allow aligning objects bigger than the cache line size
- PR #3922[3371] - Add Windows builds with Travis CI
- PR #3921[3372] - Add ccls cache directory to gitignore
- PR #3920[3373] - Fix `git_external` fetching of tags
- PR #3905[3374] - Correct rostambod url. Fix typo in doc
- PR #3904[3375] - Fix bug in context_base.hpp
- PR #3903[3376] - Adding new performance counters
- PR #3902[3377] - Add `add_hpx_module` function
- PR #3901[3378] - Factoring out container remapping into a separate trait
- PR #3900[3379] - Making sure errors during command line processing are properly reported and will not cause assertions
- PR #3899[3380] - Remove old compatibility bases from `make_action`
- PR #3898[3381] - Make parameter size be of type `size_t`

[3360] https://github.com/TheHPXProject/hpx/pull/3934
[3361] https://github.com/TheHPXProject/hpx/pull/3933
[3362] https://github.com/TheHPXProject/hpx/pull/3932
[3363] https://github.com/TheHPXProject/hpx/pull/3931
[3364] https://github.com/TheHPXProject/hpx/pull/3930
[3365] https://github.com/TheHPXProject/hpx/pull/3928
[3366] https://github.com/TheHPXProject/hpx/pull/3927
[3367] https://github.com/TheHPXProject/hpx/pull/3926
[3368] https://github.com/TheHPXProject/hpx/pull/3925
[3369] https://github.com/TheHPXProject/hpx/pull/3924
[3370] https://github.com/TheHPXProject/hpx/pull/3923
[3371] https://github.com/TheHPXProject/hpx/pull/3922
[3372] https://github.com/TheHPXProject/hpx/pull/3921
[3373] https://github.com/TheHPXProject/hpx/pull/3920
[3374] https://github.com/TheHPXProject/hpx/pull/3905
[3375] https://github.com/TheHPXProject/hpx/pull/3904
[3376] https://github.com/TheHPXProject/hpx/pull/3903
[3377] https://github.com/TheHPXProject/hpx/pull/3902
[3378] https://github.com/TheHPXProject/hpx/pull/3901
[3379] https://github.com/TheHPXProject/hpx/pull/3900
[3380] https://github.com/TheHPXProject/hpx/pull/3899
[3381] https://github.com/TheHPXProject/hpx/pull/3898

- PR #3897[3382] - Making sure all tests are disabled if `HPX_WITH_TESTS=OFF`
- PR #3895[3383] - Add documentation for annotated_function
- PR #3894[3384] - Working around VS2019 problem with `make_action`
- PR #3892[3385] - Avoid MSVC compatibility warning in internal allocator
- PR #3891[3386] - Removal of the default intel config include
- PR #3888[3387] - Fix `async_customization` dataflow example and Clarify what's being tested
- PR #3887[3388] - Add Doxygen documentation
- PR #3882[3389] - Minor docs fixes
- PR #3880[3390] - Updating APEX version tag
- PR #3878[3391] - Making sure symbols are properly exported from modules (needed for Windows/MacOS)
- PR #3877[3392] - Documentation
- PR #3876[3393] - Module hardware
- PR #3875[3394] - Converted typedefs in actions submodule to using directives
- PR #3874[3395] - Allow one to suppress target keywords in `hpx_setup_target` for backwards compatibility
- PR #3873[3396] - Add scripts to create releases and generate lists of PRs and issues
- PR #3872[3397] - Fix latest HTML docs location
- PR #3870[3398] - Module cache
- PR #3869[3399] - Post 1.3.0 version bumps
- PR #3868[3400] - Replace the macro `HPX_ASSERT` by `HPX_TEST` in tests
- PR #3845[3401] - Assertion module
- PR #3839[3402] - Make tuple serialization non-intrusive
- PR #3832[3403] - Config module

[3382] https://github.com/TheHPXProject/hpx/pull/3897
[3383] https://github.com/TheHPXProject/hpx/pull/3895
[3384] https://github.com/TheHPXProject/hpx/pull/3894
[3385] https://github.com/TheHPXProject/hpx/pull/3892
[3386] https://github.com/TheHPXProject/hpx/pull/3891
[3387] https://github.com/TheHPXProject/hpx/pull/3888
[3388] https://github.com/TheHPXProject/hpx/pull/3887
[3389] https://github.com/TheHPXProject/hpx/pull/3882
[3390] https://github.com/TheHPXProject/hpx/pull/3880
[3391] https://github.com/TheHPXProject/hpx/pull/3878
[3392] https://github.com/TheHPXProject/hpx/pull/3877
[3393] https://github.com/TheHPXProject/hpx/pull/3876
[3394] https://github.com/TheHPXProject/hpx/pull/3875
[3395] https://github.com/TheHPXProject/hpx/pull/3874
[3396] https://github.com/TheHPXProject/hpx/pull/3873
[3397] https://github.com/TheHPXProject/hpx/pull/3872
[3398] https://github.com/TheHPXProject/hpx/pull/3870
[3399] https://github.com/TheHPXProject/hpx/pull/3869
[3400] https://github.com/TheHPXProject/hpx/pull/3868
[3401] https://github.com/TheHPXProject/hpx/pull/3845
[3402] https://github.com/TheHPXProject/hpx/pull/3839
[3403] https://github.com/TheHPXProject/hpx/pull/3832

- PR #3799[3404] - Remove compat namespace and its contents
- PR #3701[3405] - MoodyCamel lockfree
- PR #3496[3406] - Disabling MPI's (deprecated) C++ interface
- PR #3192[3407] - Move type info into `hpx::debug` namespace and add print helper functions
- PR #3159[3408] - Support Checkpointing Components

HPX V1.3.0 (May 23, 2019)

General changes

- Performance improvements: the schedulers have significantly reduced overheads from removing false sharing and the parallel executor has been updated to create fewer futures.
- HPX now defaults to not turning on networking when running on one locality. This means that you can run multiple instances on the same system without adding command line options.
- Multiple issues reported by Clang sanitizers have been fixed.
- We have added (back) single-page HTML documentation and PDF documentation.
- We have started modularizing the HPX library. This is useful both for developers and users. In the long term users will be able to consume only parts of the HPX libraries if they do not require all the functionality that HPX currently provides.
- We have added an implementation of `function_ref`.
- The `barrier` and `latch` classes have gained a few additional member functions.

Breaking changes

- Executable and library targets are now created without the `_exe` and `_lib` suffix respectively. For example, the target `1d_stencil_1_exe` is now simply called `1d_stencil_1`.
- We have removed the following deprecated functionality: `queue`, `scoped_unlock`, and support for input iterators in algorithms.
- We have turned off the compatibility layer for `unwrapped` by default. The functionality will be removed in the next release. The option can still be turned on using the CMake[3409] option `HPX_WITH_UNWRAPPED_SUPPORT`. Likewise, `inclusive_scan` compatibility overloads have been turned off by default. They can still be turned on with `HPX_WITH_INCLUSIVE_SCAN_COMPATIBILITY`.
- The minimum compiler and dependency versions have been updated. We now support GCC from version 5 onwards, Clang from version 4 onwards, and Boost from version 1.61.0 onwards.

[3404] https://github.com/TheHPXProject/hpx/pull/3799
[3405] https://github.com/TheHPXProject/hpx/pull/3701
[3406] https://github.com/TheHPXProject/hpx/pull/3496
[3407] https://github.com/TheHPXProject/hpx/pull/3192
[3408] https://github.com/TheHPXProject/hpx/pull/3159
[3409] https://www.cmake.org

- The headers for preprocessor macros have moved as a result of the functionality being moved to a separate module. The old headers are deprecated and will be removed in a future version of HPX. You can turn off the warnings by setting `HPX_PREPROCESSOR_WITH_DEPRECATION_WARNINGS=OFF` or turn off the compatibility headers completely with `HPX_PREPROCESSOR_WITH_COMPATIBILITY_HEADERS=OFF`.

Closed issues

- Issue #3863[3410] - shouldn't "-faligned-new" be a usage requirement?
- Issue #3841[3411] - Build error with msvc 19 caused by SFINAE and C++17
- Issue #3836[3412] - master branch does not build with idle rate counters enabled
- Issue #3819[3413] - Add debug suffix to modules built in debug mode
- Issue #3817[3414] - `HPX_INCLUDE_DIRS` contains non-existent directory
- Issue #3810[3415] - Source groups are not created for files in modules
- Issue #3805[3416] - HPX won't compile with `-DHPX_WITH_APEX=TRUE`
- Issue #3792[3417] - Barrier Hangs When Locality Zero not included
- Issue #3778[3418] - Replace `throw()` with `noexcept`
- Issue #3763[3419] - configurable sort limit per task
- Issue #3758[3420] - dataflow doesn't convert `future<future<T>>` to `future<T>`
- Issue #3757[3421] - When compiling undefined reference to `hpx::hpx_check_version_1_2` HPX V1.2.1, Ubuntu 18.04.01 Server Edition
- Issue #3753[3422] - `--hpx:list-counters=full` crashes
- Issue #3746[3423] - Detection of MPI with pmix
- Issue #3744[3424] - Separate spinlock from same cacheline as internal data for all LCOs
- Issue #3743[3425] - hpxcxx's shebang doesn't specify the python version
- Issue #3738[3426] - Unable to debug parcelport on a single node
- Issue #3735[3427] - Latest master: Can't compile in MSVC

3410 https://github.com/TheHPXProject/hpx/issues/3863
3411 https://github.com/TheHPXProject/hpx/issues/3841
3412 https://github.com/TheHPXProject/hpx/issues/3836
3413 https://github.com/TheHPXProject/hpx/issues/3819
3414 https://github.com/TheHPXProject/hpx/issues/3817
3415 https://github.com/TheHPXProject/hpx/issues/3810
3416 https://github.com/TheHPXProject/hpx/issues/3805
3417 https://github.com/TheHPXProject/hpx/issues/3792
3418 https://github.com/TheHPXProject/hpx/issues/3778
3419 https://github.com/TheHPXProject/hpx/issues/3763
3420 https://github.com/TheHPXProject/hpx/issues/3758
3421 https://github.com/TheHPXProject/hpx/issues/3757
3422 https://github.com/TheHPXProject/hpx/issues/3753
3423 https://github.com/TheHPXProject/hpx/issues/3746
3424 https://github.com/TheHPXProject/hpx/issues/3744
3425 https://github.com/TheHPXProject/hpx/issues/3743
3426 https://github.com/TheHPXProject/hpx/issues/3738
3427 https://github.com/TheHPXProject/hpx/issues/3735

- Issue #3731[3428] - `util::bound` seems broken on Clang with older libstdc++
- Issue #3724[3429] - Allow to pre-set command line options through environment
- Issue #3723[3430] - examples/resource_partitioner build issue on master branch / ubuntu 18
- Issue #3721[3431] - faced a building error
- Issue #3720[3432] - Hello World example fails to link
- Issue #3719[3433] - pkg-config produces invalid output: `-l-pthread`
- Issue #3718[3434] - Please make the python executable configurable through cmake
- Issue #3717[3435] - interested to contribute to the organisation
- Issue #3699[3436] - Remove 'HPX runtime' executable
- Issue #3698[3437] - Ignore all locks while handling asserts
- Issue #3689[3438] - Incorrect and inconsistent website structure http://stellar.cct.lsu.edu/downloads/.
- Issue #3681[3439] - Broken links on http://stellar.cct.lsu.edu/2015/05/hpx-archives-now-on-gmane/
- Issue #3676[3440] - HPX master built from source, cmake fails to link main.cpp example in docs
- Issue #3673[3441] - HPX build fails with `std::atomic` missing error
- Issue #3670[3442] - Generate PDF again from documentation (with Sphinx)
- Issue #3643[3443] - Warnings when compiling HPX 1.2.1 with gcc 9
- Issue #3641[3444] - Trouble with using ranges-v3 and `hpx::parallel::reduce`
- Issue #3639[3445] - `util::unwrapping` does not work well with member functions
- Issue #3634[3446] - The build fails if `shared_future<>::then` is called with a thread executor
- Issue #3622[3447] - VTune Amplifier 2019 not working with `use_itt_notify=1`
- Issue #3616[3448] - HPX Fails to Build with CUDA 10

[3428] https://github.com/TheHPXProject/hpx/issues/3731
[3429] https://github.com/TheHPXProject/hpx/issues/3724
[3430] https://github.com/TheHPXProject/hpx/issues/3723
[3431] https://github.com/TheHPXProject/hpx/issues/3721
[3432] https://github.com/TheHPXProject/hpx/issues/3720
[3433] https://github.com/TheHPXProject/hpx/issues/3719
[3434] https://github.com/TheHPXProject/hpx/issues/3718
[3435] https://github.com/TheHPXProject/hpx/issues/3717
[3436] https://github.com/TheHPXProject/hpx/issues/3699
[3437] https://github.com/TheHPXProject/hpx/issues/3698
[3438] https://github.com/TheHPXProject/hpx/issues/3689
[3439] https://github.com/TheHPXProject/hpx/issues/3681
[3440] https://github.com/TheHPXProject/hpx/issues/3676
[3441] https://github.com/TheHPXProject/hpx/issues/3673
[3442] https://github.com/TheHPXProject/hpx/issues/3670
[3443] https://github.com/TheHPXProject/hpx/issues/3643
[3444] https://github.com/TheHPXProject/hpx/issues/3641
[3445] https://github.com/TheHPXProject/hpx/issues/3639
[3446] https://github.com/TheHPXProject/hpx/issues/3634
[3447] https://github.com/TheHPXProject/hpx/issues/3622
[3448] https://github.com/TheHPXProject/hpx/issues/3616

- Issue #3612[3449] - False sharing of scheduling counters
- Issue #3609[3450] - executor_parameters timeout with gcc <= 7 and Debug mode
- Issue #3601[3451] - Misleading error message on power pc for rdtsc and rdtscp
- Issue #3598[3452] - Build of some examples fails when using Vc
- Issue #3594[3453] - Error: The number of OS threads requested (20) does not match the number of threads to bind (12): HPX(bad_parameter)
- Issue #3592[3454] - Undefined Reference Error
- Issue #3589[3455] - include could not find load file: HPX_Utils.cmake
- Issue #3587[3456] - HPX won't compile on POWER8 with Clang 7
- Issue #3583[3457] - Fedora and openSUSE instructions missing on "Distribution Packages" page
- Issue #3578[3458] - Build error when configuring with `HPX_HAVE_ALGORITHM_INPUT_ITERATOR_SUPPORT=ON`
- Issue #3575[3459] - Merge openSUSE reproducible patch
- Issue #3570[3460] - Update HPX to work with the latest VC version
- Issue #3567[3461] - Build succeed and make failed for `hpx:cout`
- Issue #3565[3462] - Polymorphic simple component destructor not getting called
- Issue #3559[3463] - 1.2.0 is missing from download page
- Issue #3554[3464] - Clang 6.0 warning of hiding overloaded virtual function
- Issue #3510[3465] - Build on ppc64 fails
- Issue #3482[3466] - Improve error message when `HPX_WITH_MAX_CPU_COUNT` is too low for given system
- Issue #3453[3467] - Two HPX applications can't run at the same time.
- Issue #3452[3468] - Scaling issue on the change to 2 NUMA domains
- Issue #3442[3469] - HPX set_difference, set_intersection failure cases

[3449] https://github.com/TheHPXProject/hpx/issues/3612
[3450] https://github.com/TheHPXProject/hpx/issues/3609
[3451] https://github.com/TheHPXProject/hpx/issues/3601
[3452] https://github.com/TheHPXProject/hpx/issues/3598
[3453] https://github.com/TheHPXProject/hpx/issues/3594
[3454] https://github.com/TheHPXProject/hpx/issues/3592
[3455] https://github.com/TheHPXProject/hpx/issues/3589
[3456] https://github.com/TheHPXProject/hpx/issues/3587
[3457] https://github.com/TheHPXProject/hpx/issues/3583
[3458] https://github.com/TheHPXProject/hpx/issues/3578
[3459] https://github.com/TheHPXProject/hpx/issues/3575
[3460] https://github.com/TheHPXProject/hpx/issues/3570
[3461] https://github.com/TheHPXProject/hpx/issues/3567
[3462] https://github.com/TheHPXProject/hpx/issues/3565
[3463] https://github.com/TheHPXProject/hpx/issues/3559
[3464] https://github.com/TheHPXProject/hpx/issues/3554
[3465] https://github.com/TheHPXProject/hpx/issues/3510
[3466] https://github.com/TheHPXProject/hpx/issues/3482
[3467] https://github.com/TheHPXProject/hpx/issues/3453
[3468] https://github.com/TheHPXProject/hpx/issues/3452
[3469] https://github.com/TheHPXProject/hpx/issues/3442

- Issue #3437[3470] - Ensure parent_task pointer when child task is created and child/parent are on same locality
- Issue #3255[3471] - Suspension with lock for `--hpx:list-component-types`
- Issue #3034[3472] - Use C++17 structured bindings for serialization
- Issue #2999[3473] - Change thread scheduling use of `size_t` for thread indexing

Closed pull requests

- PR #3865[3474] - adds hpx_target_compile_option_if_available
- PR #3864[3475] - Helper functions that are useful in numa binding and testing of allocator
- PR #3862[3476] - Temporary fix to local_dataflow_boost_small_vector test
- PR #3860[3477] - Add cache line padding to intermediate results in for loop reduction
- PR #3859[3478] - Remove HPX_TLL_PUBLIC and HPX_TLL_PRIVATE from CMake files
- PR #3858[3479] - Add compile flags and definitions to modules
- PR #3851[3480] - update hpxmp release tag to v0.2.0
- PR #3849[3481] - Correct BOOST_ROOT variable name in quick start guide
- PR #3847[3482] - Fix attach_debugger configuration option
- PR #3846[3483] - Add tests for libs header tests
- PR #3844[3484] - Fixing source_groups in preprocessor module to properly handle compatibility headers
- PR #3843[3485] - This fixes the launch_process/launched_process pair of tests
- PR #3842[3486] - Fix macro call with ITTNOTIFY enabled
- PR #3840[3487] - Fixing SLURM environment parsing
- PR #3837[3488] - Fixing misplaced #endif
- PR #3835[3489] - make all latch members protected for consistency

[3470] https://github.com/TheHPXProject/hpx/issues/3437
[3471] https://github.com/TheHPXProject/hpx/issues/3255
[3472] https://github.com/TheHPXProject/hpx/issues/3034
[3473] https://github.com/TheHPXProject/hpx/issues/2999
[3474] https://github.com/TheHPXProject/hpx/pull/3865
[3475] https://github.com/TheHPXProject/hpx/pull/3864
[3476] https://github.com/TheHPXProject/hpx/pull/3862
[3477] https://github.com/TheHPXProject/hpx/pull/3860
[3478] https://github.com/TheHPXProject/hpx/pull/3859
[3479] https://github.com/TheHPXProject/hpx/pull/3858
[3480] https://github.com/TheHPXProject/hpx/pull/3851
[3481] https://github.com/TheHPXProject/hpx/pull/3849
[3482] https://github.com/TheHPXProject/hpx/pull/3847
[3483] https://github.com/TheHPXProject/hpx/pull/3846
[3484] https://github.com/TheHPXProject/hpx/pull/3844
[3485] https://github.com/TheHPXProject/hpx/pull/3843
[3486] https://github.com/TheHPXProject/hpx/pull/3842
[3487] https://github.com/TheHPXProject/hpx/pull/3840
[3488] https://github.com/TheHPXProject/hpx/pull/3837
[3489] https://github.com/TheHPXProject/hpx/pull/3835

- PR #3834[3490] - Disable transpose_block_numa example on CircleCI
- PR #3833[3491] - make latch **counter_** protected for deriving latch in hpxmp
- PR #3831[3492] - Fix CircleCI config for modules
- PR #3830[3493] - minor fix: option HPX_WITH_TEST was not working correctly
- PR #3828[3494] - Avoid for binaries that depend on HPX to directly link against internal modules
- PR #3827[3495] - Adding shortcut for `hpx::get_ptr<>(sync, id)` for a local, non-migratable objects
- PR #3826[3496] - Fix and update modules documentation
- PR #3825[3497] - Updating default APEX version to 2.1.3 with HPX
- PR #3823[3498] - Fix pkgconfig libs handling
- PR #3822[3499] - Change includes in hpx_wrap.cpp to more specific includes
- PR #3821[3500] - Disable barrier_3792 test when networking is disabled
- PR #3820[3501] - Assorted CMake fixes
- PR #3815[3502] - Removing left-over debug output
- PR #3814[3503] - Allow setting default scheduler mode via the configuration database
- PR #3813[3504] - Make the deprecation warnings issued by the old pp headers optional
- PR #3812[3505] - Windows requires to handle symlinks to directories differently from those linking files
- PR #3811[3506] - Clean up PP module and library skeleton
- PR #3806[3507] - Moving include path configuration to before APEX
- PR #3804[3508] - Fix latch
- PR #3803[3509] - Update hpxcxx to look at lib64 and use python3
- PR #3802[3510] - Numa binding allocator
- PR #3801[3511] - Remove duplicated includes

[3490] https://github.com/TheHPXProject/hpx/pull/3834
[3491] https://github.com/TheHPXProject/hpx/pull/3833
[3492] https://github.com/TheHPXProject/hpx/pull/3831
[3493] https://github.com/TheHPXProject/hpx/pull/3830
[3494] https://github.com/TheHPXProject/hpx/pull/3828
[3495] https://github.com/TheHPXProject/hpx/pull/3827
[3496] https://github.com/TheHPXProject/hpx/pull/3826
[3497] https://github.com/TheHPXProject/hpx/pull/3825
[3498] https://github.com/TheHPXProject/hpx/pull/3823
[3499] https://github.com/TheHPXProject/hpx/pull/3822
[3500] https://github.com/TheHPXProject/hpx/pull/3821
[3501] https://github.com/TheHPXProject/hpx/pull/3820
[3502] https://github.com/TheHPXProject/hpx/pull/3815
[3503] https://github.com/TheHPXProject/hpx/pull/3814
[3504] https://github.com/TheHPXProject/hpx/pull/3813
[3505] https://github.com/TheHPXProject/hpx/pull/3812
[3506] https://github.com/TheHPXProject/hpx/pull/3811
[3507] https://github.com/TheHPXProject/hpx/pull/3806
[3508] https://github.com/TheHPXProject/hpx/pull/3804
[3509] https://github.com/TheHPXProject/hpx/pull/3803
[3510] https://github.com/TheHPXProject/hpx/pull/3802
[3511] https://github.com/TheHPXProject/hpx/pull/3801

- PR #3800[3512] - Attempt to fix Posix context switching after lazy init changes
- PR #3798[3513] - count and count_if accepts different iterator types
- PR #3797[3514] - Adding a couple of `override` keywords to overloaded virtual functions
- PR #3796[3515] - Re-enable testing all schedulers in shutdown_suspended_test
- PR #3795[3516] - Change `std::terminate` to std::abort in `SIGSEGV` handler
- PR #3794[3517] - Fixing #3792
- PR #3793[3518] - Extending migrate_polymorphic_component unit test
- PR #3791[3519] - Change `throw()` to `noexcept`
- PR #3790[3520] - Remove deprecated options for 1.3.0 release
- PR #3789[3521] - Remove Boost filesystem compatibility header
- PR #3788[3522] - Disabled even more spots that should not execute if networking is disabled
- PR #3787[3523] - Bump minimal boost supported version to 1.61.0
- PR #3786[3524] - Bump minimum required versions for 1.3.0 release
- PR #3785[3525] - Explicitly set number of jobs for all ninja invocations on CircleCI
- PR #3784[3526] - Fix leak and address sanitizer problems
- PR #3783[3527] - Disabled even more spots that should not execute is networking is disabled
- PR #3782[3528] - Cherry-picked tuple and thread_init_data fixes from #3701
- PR #3781[3529] - Fix generic context coroutines after lazy stack allocation changes
- PR #3780[3530] - Rename hello world examples
- PR #3776[3531] - Sort algorithms now use the supplied chunker to determine the required minimal chunk size
- PR #3775[3532] - Disable Boost auto-linking
- PR #3774[3533] - Tag and push stable builds

[3512] https://github.com/TheHPXProject/hpx/pull/3800
[3513] https://github.com/TheHPXProject/hpx/pull/3798
[3514] https://github.com/TheHPXProject/hpx/pull/3797
[3515] https://github.com/TheHPXProject/hpx/pull/3796
[3516] https://github.com/TheHPXProject/hpx/pull/3795
[3517] https://github.com/TheHPXProject/hpx/pull/3794
[3518] https://github.com/TheHPXProject/hpx/pull/3793
[3519] https://github.com/TheHPXProject/hpx/pull/3791
[3520] https://github.com/TheHPXProject/hpx/pull/3790
[3521] https://github.com/TheHPXProject/hpx/pull/3789
[3522] https://github.com/TheHPXProject/hpx/pull/3788
[3523] https://github.com/TheHPXProject/hpx/pull/3787
[3524] https://github.com/TheHPXProject/hpx/pull/3786
[3525] https://github.com/TheHPXProject/hpx/pull/3785
[3526] https://github.com/TheHPXProject/hpx/pull/3784
[3527] https://github.com/TheHPXProject/hpx/pull/3783
[3528] https://github.com/TheHPXProject/hpx/pull/3782
[3529] https://github.com/TheHPXProject/hpx/pull/3781
[3530] https://github.com/TheHPXProject/hpx/pull/3780
[3531] https://github.com/TheHPXProject/hpx/pull/3776
[3532] https://github.com/TheHPXProject/hpx/pull/3775
[3533] https://github.com/TheHPXProject/hpx/pull/3774

- PR #3773[3534] - Enable migration of polymorphic components
- PR #3771[3535] - Fix link to stackoverflow in documentation
- PR #3770[3536] - Replacing constexpr if in brace-serialization code
- PR #3769[3537] - Fix SIGSEGV handler
- PR #3768[3538] - Adding flags to scheduler allowing to control thread stealing and idle back-off
- PR #3767[3539] - Fix help formatting in hpxrun.py
- PR #3765[3540] - Fix a couple of bugs in the thread test
- PR #3764[3541] - Workaround for SFINAE regression in msvc14.2
- PR #3762[3542] - Prevent MSVC from prematurely instantiating things
- PR #3761[3543] - Update python scripts to work with python 3
- PR #3760[3544] - Fix callable vtable for GCC4.9
- PR #3759[3545] - Rename `PAGE_SIZE` to `PAGE_SIZE_` because AppleClang
- PR #3755[3546] - Making sure locks are not held during suspension
- PR #3754[3547] - Disable more code if networking is not available/not enabled
- PR #3752[3548] - Move `util::format` implementation to source file
- PR #3751[3549] - Fixing problems with `lcos::barrier` and iostreams
- PR #3750[3550] - Change error message to take into account `use_guard_page` setting
- PR #3749[3551] - Fix lifetime problem in `run_as_hpx_thread`
- PR #3748[3552] - Fixed unusable behavior of the clang code analyzer.
- PR #3747[3553] - Added `PMIX_RANK` to the defaults of `HPX_WITH_PARCELPORT_MPI_ENV`.
- PR #3745[3554] - Introduced `cache_aligned_data` and `cache_line_data` helper structure
- PR #3742[3555] - Remove more unused functionality from util/logging

[3534] https://github.com/TheHPXProject/hpx/pull/3773
[3535] https://github.com/TheHPXProject/hpx/pull/3771
[3536] https://github.com/TheHPXProject/hpx/pull/3770
[3537] https://github.com/TheHPXProject/hpx/pull/3769
[3538] https://github.com/TheHPXProject/hpx/pull/3768
[3539] https://github.com/TheHPXProject/hpx/pull/3767
[3540] https://github.com/TheHPXProject/hpx/pull/3765
[3541] https://github.com/TheHPXProject/hpx/pull/3764
[3542] https://github.com/TheHPXProject/hpx/pull/3762
[3543] https://github.com/TheHPXProject/hpx/pull/3761
[3544] https://github.com/TheHPXProject/hpx/pull/3760
[3545] https://github.com/TheHPXProject/hpx/pull/3759
[3546] https://github.com/TheHPXProject/hpx/pull/3755
[3547] https://github.com/TheHPXProject/hpx/pull/3754
[3548] https://github.com/TheHPXProject/hpx/pull/3752
[3549] https://github.com/TheHPXProject/hpx/pull/3751
[3550] https://github.com/TheHPXProject/hpx/pull/3750
[3551] https://github.com/TheHPXProject/hpx/pull/3749
[3552] https://github.com/TheHPXProject/hpx/pull/3748
[3553] https://github.com/TheHPXProject/hpx/pull/3747
[3554] https://github.com/TheHPXProject/hpx/pull/3745
[3555] https://github.com/TheHPXProject/hpx/pull/3742

- PR #3740[3556] - Fix includes in partitioned vector tests
- PR #3739[3557] - More fixes to make sure that `std::flush` really flushes all output
- PR #3737[3558] - Fix potential shutdown problems
- PR #3736[3559] - Fix `guided_pool_executor` after dataflow changes caused compilation fail
- PR #3734[3560] - Limiting executor
- PR #3732[3561] - More constrained bound constructors
- PR #3730[3562] - Attempt to fix deadlocks during component loading
- PR #3729[3563] - Add latch member function `count_up` and reset, requested by hpxMP
- PR #3728[3564] - Send even empty buffers on `hpx::endl` and `hpx::flush`
- PR #3727[3565] - Adding example demonstrating how to customize the memory management for a component
- PR #3726[3566] - Adding support for passing command line options through the `HPX_COMMANDLINE_OPTIONS` environment variable
- PR #3722[3567] - Document known broken OpenMPI builds
- PR #3716[3568] - Add barrier reset function, requested by hpxMP for reusing barrier
- PR #3715[3569] - More work on functions and vtables
- PR #3714[3570] - Generate single-page HTML, PDF, manpage from documentation
- PR #3713[3571] - Updating default APEX version to 2.1.2
- PR #3712[3572] - Update release procedure
- PR #3710[3573] - Fix the C++11 build, after #3704
- PR #3709[3574] - Move some component_registry functionality to source file
- PR #3708[3575] - Ignore all locks while handling assertions
- PR #3707[3576] - Remove obsolete hpx runtime executable
- PR #3705[3577] - Fix and simplify `make_ready_future` overload sets

[3556] https://github.com/TheHPXProject/hpx/pull/3740
[3557] https://github.com/TheHPXProject/hpx/pull/3739
[3558] https://github.com/TheHPXProject/hpx/pull/3737
[3559] https://github.com/TheHPXProject/hpx/pull/3736
[3560] https://github.com/TheHPXProject/hpx/pull/3734
[3561] https://github.com/TheHPXProject/hpx/pull/3732
[3562] https://github.com/TheHPXProject/hpx/pull/3730
[3563] https://github.com/TheHPXProject/hpx/pull/3729
[3564] https://github.com/TheHPXProject/hpx/pull/3728
[3565] https://github.com/TheHPXProject/hpx/pull/3727
[3566] https://github.com/TheHPXProject/hpx/pull/3726
[3567] https://github.com/TheHPXProject/hpx/pull/3722
[3568] https://github.com/TheHPXProject/hpx/pull/3716
[3569] https://github.com/TheHPXProject/hpx/pull/3715
[3570] https://github.com/TheHPXProject/hpx/pull/3714
[3571] https://github.com/TheHPXProject/hpx/pull/3713
[3572] https://github.com/TheHPXProject/hpx/pull/3712
[3573] https://github.com/TheHPXProject/hpx/pull/3710
[3574] https://github.com/TheHPXProject/hpx/pull/3709
[3575] https://github.com/TheHPXProject/hpx/pull/3708
[3576] https://github.com/TheHPXProject/hpx/pull/3707
[3577] https://github.com/TheHPXProject/hpx/pull/3705

- PR #3704[3578] - Reduce use of binders
- PR #3703[3579] - Ini
- PR #3702[3580] - Fixing CUDA compiler errors
- PR #3700[3581] - Added `barrier::increment` function to increase total number of thread
- PR #3697[3582] - One more attempt to fix migration…
- PR #3694[3583] - Fixing component migration
- PR #3693[3584] - Print thread state when getting disallowed value in set_thread_state
- PR #3692[3585] - Only disable `constexpr` with clang-cuda, not nvcc+gcc
- PR #3691[3586] - Link with libsupc++ if needed for thread_local
- PR #3690[3587] - Remove thousands separators in set_operations_3442 to comply with C++11
- PR #3688[3588] - Decouple serialization from function vtables
- PR #3687[3589] - Fix a couple of test failures
- PR #3686[3590] - Make sure tests.unit.build are run after install on CircleCI
- PR #3685[3591] - Revise quickstart CMakeLists.txt explanation
- PR #3684[3592] - Provide concept emulation for Ranges-TS concepts
- PR #3683[3593] - Ignore uninitialized chunks
- PR #3682[3594] - Ignore uninitialized chunks. Check proper indices.
- PR #3680[3595] - Ignore uninitialized chunks. Check proper range indices
- PR #3679[3596] - Simplify basic action implementations
- PR #3678[3597] - Making sure `HPX_HAVE_LIBATOMIC` is unset before checking
- PR #3677[3598] - Fix generated full version number to be usable in expressions
- PR #3674[3599] - Reduce functional utilities call depth

[3578] https://github.com/TheHPXProject/hpx/pull/3704
[3579] https://github.com/TheHPXProject/hpx/pull/3703
[3580] https://github.com/TheHPXProject/hpx/pull/3702
[3581] https://github.com/TheHPXProject/hpx/pull/3700
[3582] https://github.com/TheHPXProject/hpx/pull/3697
[3583] https://github.com/TheHPXProject/hpx/pull/3694
[3584] https://github.com/TheHPXProject/hpx/pull/3693
[3585] https://github.com/TheHPXProject/hpx/pull/3692
[3586] https://github.com/TheHPXProject/hpx/pull/3691
[3587] https://github.com/TheHPXProject/hpx/pull/3690
[3588] https://github.com/TheHPXProject/hpx/pull/3688
[3589] https://github.com/TheHPXProject/hpx/pull/3687
[3590] https://github.com/TheHPXProject/hpx/pull/3686
[3591] https://github.com/TheHPXProject/hpx/pull/3685
[3592] https://github.com/TheHPXProject/hpx/pull/3684
[3593] https://github.com/TheHPXProject/hpx/pull/3683
[3594] https://github.com/TheHPXProject/hpx/pull/3682
[3595] https://github.com/TheHPXProject/hpx/pull/3680
[3596] https://github.com/TheHPXProject/hpx/pull/3679
[3597] https://github.com/TheHPXProject/hpx/pull/3678
[3598] https://github.com/TheHPXProject/hpx/pull/3677
[3599] https://github.com/TheHPXProject/hpx/pull/3674

- PR #3672[3600] - Change new build system to use existing macros related to pseudo dependencies
- PR #3669[3601] - Remove indirection in `function_ref` when thread description is disabled
- PR #3668[3602] - Unbreaking `async_*cb*` tests
- PR #3667[3603] - Generate version.hpp
- PR #3665[3604] - Enabling MPI parcelport for gitlab runners
- PR #3664[3605] - making clang-tidy work properly again
- PR #3662[3606] - Attempt to fix exception handling
- PR #3661[3607] - Move `lcos::latch` to source file
- PR #3660[3608] - Fix accidentally explicit gid_type default constructor
- PR #3659[3609] - Parallel executor latch
- PR #3658[3610] - Fixing execution_parameters
- PR #3657[3611] - Avoid dangling references in wait_all
- PR #3656[3612] - Avoiding lifetime problems with sync_put_parcel
- PR #3655[3613] - Fixing nullptr dereference inside of function
- PR #3652[3614] - Attempt to fix `thread_map_type` definition with C++11
- PR #3650[3615] - Allowing for end iterator being different from begin iterator
- PR #3649[3616] - Added architecture identification to cmake to be able to detect timestamp support
- PR #3645[3617] - Enabling sanitizers on gitlab runner
- PR #3644[3618] - Attempt to tackle timeouts during startup
- PR #3642[3619] - Cleanup parallel partitioners
- PR #3640[3620] - Dataflow now works with functions that return a reference
- PR #3637[3621] - Merging the executor-enabled overloads of `shared_future<>::then`

[3600] https://github.com/TheHPXProject/hpx/pull/3672
[3601] https://github.com/TheHPXProject/hpx/pull/3669
[3602] https://github.com/TheHPXProject/hpx/pull/3668
[3603] https://github.com/TheHPXProject/hpx/pull/3667
[3604] https://github.com/TheHPXProject/hpx/pull/3665
[3605] https://github.com/TheHPXProject/hpx/pull/3664
[3606] https://github.com/TheHPXProject/hpx/pull/3662
[3607] https://github.com/TheHPXProject/hpx/pull/3661
[3608] https://github.com/TheHPXProject/hpx/pull/3660
[3609] https://github.com/TheHPXProject/hpx/pull/3659
[3610] https://github.com/TheHPXProject/hpx/pull/3658
[3611] https://github.com/TheHPXProject/hpx/pull/3657
[3612] https://github.com/TheHPXProject/hpx/pull/3656
[3613] https://github.com/TheHPXProject/hpx/pull/3655
[3614] https://github.com/TheHPXProject/hpx/pull/3652
[3615] https://github.com/TheHPXProject/hpx/pull/3650
[3616] https://github.com/TheHPXProject/hpx/pull/3649
[3617] https://github.com/TheHPXProject/hpx/pull/3645
[3618] https://github.com/TheHPXProject/hpx/pull/3644
[3619] https://github.com/TheHPXProject/hpx/pull/3642
[3620] https://github.com/TheHPXProject/hpx/pull/3640
[3621] https://github.com/TheHPXProject/hpx/pull/3637

- PR #3633[3622] - Replace deprecated boost endian macros
- PR #3632[3623] - Add instructions on getting HPX to documentation
- PR #3631[3624] - Simplify parcel creation
- PR #3630[3625] - Small additions and fixes to release procedure
- PR #3629[3626] - Modular pp
- PR #3627[3627] - Implement `util::function_ref`
- PR #3626[3628] - Fix cancelable_action_client example
- PR #3625[3629] - Added automatic serialization for simple structs (see #3034)
- PR #3624[3630] - Updating the default order of priority for `thread_description`
- PR #3621[3631] - Update copyright year and other small formatting fixes
- PR #3620[3632] - Adding support for gitlab runner
- PR #3619[3633] - Store debug logs and core dumps on CircleCI
- PR #3618[3634] - Various optimizations
- PR #3617[3635] - Fix link to the gpg key (#2)
- PR #3615[3636] - Fix unused variable warnings with networking off
- PR #3614[3637] - Restructuring counter data in scheduler to reduce false sharing
- PR #3613[3638] - Adding support for gitlab runners
- PR #3610[3639] - Don't wait for `stop_condition` in main thread
- PR #3608[3640] - Add inline keyword to `invalid_thread_id` definition for nvcc
- PR #3607[3641] - Adding configuration key that allows one to explicitly add a directory to the component search path
- PR #3606[3642] - Add nvcc to exclude constexpress since is it not supported by nvcc
- PR #3605[3643] - Add `inline` to definition of checkpoint stream operators to fix link error
- PR #3604[3644] - Use format for string formatting

[3622] https://github.com/TheHPXProject/hpx/pull/3633
[3623] https://github.com/TheHPXProject/hpx/pull/3632
[3624] https://github.com/TheHPXProject/hpx/pull/3631
[3625] https://github.com/TheHPXProject/hpx/pull/3630
[3626] https://github.com/TheHPXProject/hpx/pull/3629
[3627] https://github.com/TheHPXProject/hpx/pull/3627
[3628] https://github.com/TheHPXProject/hpx/pull/3626
[3629] https://github.com/TheHPXProject/hpx/pull/3625
[3630] https://github.com/TheHPXProject/hpx/pull/3624
[3631] https://github.com/TheHPXProject/hpx/pull/3621
[3632] https://github.com/TheHPXProject/hpx/pull/3620
[3633] https://github.com/TheHPXProject/hpx/pull/3619
[3634] https://github.com/TheHPXProject/hpx/pull/3618
[3635] https://github.com/TheHPXProject/hpx/pull/3617
[3636] https://github.com/TheHPXProject/hpx/pull/3615
[3637] https://github.com/TheHPXProject/hpx/pull/3614
[3638] https://github.com/TheHPXProject/hpx/pull/3613
[3639] https://github.com/TheHPXProject/hpx/pull/3610
[3640] https://github.com/TheHPXProject/hpx/pull/3608
[3641] https://github.com/TheHPXProject/hpx/pull/3607
[3642] https://github.com/TheHPXProject/hpx/pull/3606
[3643] https://github.com/TheHPXProject/hpx/pull/3605
[3644] https://github.com/TheHPXProject/hpx/pull/3604

- PR #3603[3645] - Improve the error message for using to less `MAX_CPU_COUNT`
- PR #3602[3646] - Improve the error message for to small values of `MAX_CPU_COUNT`
- PR #3600[3647] - Parallel executor aggregated
- PR #3599[3648] - Making sure networking is disabled for default one-locality-runs
- PR #3596[3649] - Store thread exit functions in `forward_list` instead of `deque` to avoid allocations
- PR #3590[3650] - Fix typo/mistake in thread queue `cleanup_terminated`
- PR #3588[3651] - Fix formatting errors in launching_and_configuring_hpx_applications.rst
- PR #3586[3652] - Make bind propagate value category
- PR #3585[3653] - Extend Cmake for building hpx as distribution packages (refs #3575)
- PR #3584[3654] - Untangle function storage from object pointer
- PR #3582[3655] - Towards Modularized HPX
- PR #3580[3656] - Remove extra `||` in merge.hpp
- PR #3577[3657] - Partially revert "Remove vtable empty flag"
- PR #3576[3658] - Make sure empty startup/shutdown functions are not being used
- PR #3574[3659] - Make sure `DATAPAR` settings are conveyed to depending projects
- PR #3573[3660] - Make sure HPX is usable with latest released version of Vc (V1.4.1)
- PR #3572[3661] - Adding test ensuring ticket 3565 is fixed
- PR #3571[3662] - Make empty `[unique_]function` vtable non-dependent
- PR #3566[3663] - Fix compilation with dynamic bitset for CPU masks
- PR #3563[3664] - Drop `util::[unique_]function` target_type
- PR #3562[3665] - Removing the target suffixes
- PR #3561[3666] - Replace executor traits return type deduction (keep non-SFINAE)

[3645] https://github.com/TheHPXProject/hpx/pull/3603
[3646] https://github.com/TheHPXProject/hpx/pull/3602
[3647] https://github.com/TheHPXProject/hpx/pull/3600
[3648] https://github.com/TheHPXProject/hpx/pull/3599
[3649] https://github.com/TheHPXProject/hpx/pull/3596
[3650] https://github.com/TheHPXProject/hpx/pull/3590
[3651] https://github.com/TheHPXProject/hpx/pull/3588
[3652] https://github.com/TheHPXProject/hpx/pull/3586
[3653] https://github.com/TheHPXProject/hpx/pull/3585
[3654] https://github.com/TheHPXProject/hpx/pull/3584
[3655] https://github.com/TheHPXProject/hpx/pull/3582
[3656] https://github.com/TheHPXProject/hpx/pull/3580
[3657] https://github.com/TheHPXProject/hpx/pull/3577
[3658] https://github.com/TheHPXProject/hpx/pull/3576
[3659] https://github.com/TheHPXProject/hpx/pull/3574
[3660] https://github.com/TheHPXProject/hpx/pull/3573
[3661] https://github.com/TheHPXProject/hpx/pull/3572
[3662] https://github.com/TheHPXProject/hpx/pull/3571
[3663] https://github.com/TheHPXProject/hpx/pull/3566
[3664] https://github.com/TheHPXProject/hpx/pull/3563
[3665] https://github.com/TheHPXProject/hpx/pull/3562
[3666] https://github.com/TheHPXProject/hpx/pull/3561

- PR #3557[3667] - Replace the last usages of boost::atomic
- PR #3556[3668] - Replace `boost::scoped_array` with `std::unique_ptr`
- PR #3552[3669] - (Re)move APEX readme
- PR #3548[3670] - Replace `boost::scoped_ptr` with `std::unique_ptr`
- PR #3547[3671] - Remove last use of Boost.Signals2
- PR #3544[3672] - Post 1.2.0 version bumps
- PR #3543[3673] - added Ubuntu dependency list to readme
- PR #3531[3674] - Warnings, warnings…
- PR #3527[3675] - Add CircleCI filter for building all tags
- PR #3525[3676] - Segmented algorithms
- PR #3517[3677] - Replace `boost::regex` with C++11 `<regex>`
- PR #3514[3678] - Cleaning up the build system
- PR #3505[3679] - Fixing type attribute warning for `transfer_action`
- PR #3504[3680] - Add support for rpm packaging
- PR #3499[3681] - Improving spinlock pools
- PR #3498[3682] - Remove thread specific ptr
- PR #3486[3683] - Fix comparison for expect_connecting_localities config entry
- PR #3469[3684] - Enable (existing) code for extracting stack pointer on Power platform

HPX V1.2.1 (Feb 19, 2019)

General changes

This is a bugfix release. It contains the following changes:

- Fix compilation on ARM, s390x and 32-bit architectures.
- Fix a critical bug in the `future` implementation.
- Fix several problems in the CMake configuration which affects external projects.

3667 https://github.com/TheHPXProject/hpx/pull/3557
3668 https://github.com/TheHPXProject/hpx/pull/3556
3669 https://github.com/TheHPXProject/hpx/pull/3552
3670 https://github.com/TheHPXProject/hpx/pull/3548
3671 https://github.com/TheHPXProject/hpx/pull/3547
3672 https://github.com/TheHPXProject/hpx/pull/3544
3673 https://github.com/TheHPXProject/hpx/pull/3543
3674 https://github.com/TheHPXProject/hpx/pull/3531
3675 https://github.com/TheHPXProject/hpx/pull/3527
3676 https://github.com/TheHPXProject/hpx/pull/3525
3677 https://github.com/TheHPXProject/hpx/pull/3517
3678 https://github.com/TheHPXProject/hpx/pull/3514
3679 https://github.com/TheHPXProject/hpx/pull/3505
3680 https://github.com/TheHPXProject/hpx/pull/3504
3681 https://github.com/TheHPXProject/hpx/pull/3499
3682 https://github.com/TheHPXProject/hpx/pull/3498
3683 https://github.com/TheHPXProject/hpx/pull/3486
3684 https://github.com/TheHPXProject/hpx/pull/3469

- Add support for Boost 1.69.0.

Closed issues

- Issue #3638[3685] - Build HPX 1.2 with boost 1.69
- Issue #3635[3686] - Non-deterministic crashing on Stampede2
- Issue #3550[3687] - 1>e:000workhpxsrcthrow_exception.cpp(54): error C2440: '<function-style-cast>': cannot convert from 'boost::system::error_code' to 'hpx::exception'
- Issue #3549[3688] - HPX 1.2.0 does not build on i686, but release candidate did
- Issue #3511[3689] - Build on s390x fails
- Issue #3509[3690] - Build on armv7l fails

Closed pull requests

- PR #3695[3691] - Don't install CMake templates and packaging files
- PR #3666[3692] - Fixing yet another race in future_data
- PR #3663[3693] - Fixing race between setting and getting the value inside future_data
- PR #3648[3694] - Adding timestamp option for S390x platform
- PR #3647[3695] - Blind attempt to fix warnings issued by gcc V9
- PR #3611[3696] - Include GNUInstallDirs earlier to have it available for subdirectories
- PR #3595[3697] - Use GNUInstallDirs lib path in pkgconfig config file
- PR #3593[3698] - Add include(GNUInstallDirs) to HPXMacros.cmake
- PR #3591[3699] - Fix compilation error on arm7 architecture. Compiles and runs on Fedora 29 on Pi 3.
- PR #3558[3700] - Adding constructor *exception(boost::system::error_code const&)*
- PR #3555[3701] - cmake: make install locations configurable
- PR #3551[3702] - Fix uint64_t causing compilation fail on i686

[3685] https://github.com/TheHPXProject/hpx/issues/3638
[3686] https://github.com/TheHPXProject/hpx/issues/3635
[3687] https://github.com/TheHPXProject/hpx/issues/3550
[3688] https://github.com/TheHPXProject/hpx/issues/3549
[3689] https://github.com/TheHPXProject/hpx/issues/3511
[3690] https://github.com/TheHPXProject/hpx/issues/3509
[3691] https://github.com/TheHPXProject/hpx/pull/3695
[3692] https://github.com/TheHPXProject/hpx/pull/3666
[3693] https://github.com/TheHPXProject/hpx/pull/3663
[3694] https://github.com/TheHPXProject/hpx/pull/3648
[3695] https://github.com/TheHPXProject/hpx/pull/3647
[3696] https://github.com/TheHPXProject/hpx/pull/3611
[3697] https://github.com/TheHPXProject/hpx/pull/3595
[3698] https://github.com/TheHPXProject/hpx/pull/3593
[3699] https://github.com/TheHPXProject/hpx/pull/3591
[3700] https://github.com/TheHPXProject/hpx/pull/3558
[3701] https://github.com/TheHPXProject/hpx/pull/3555
[3702] https://github.com/TheHPXProject/hpx/pull/3551

HPX V1.2.0 (Nov 12, 2018)

General changes

Here are some of the main highlights and changes for this release:

- Thanks to the work of our Google Summer of Code student, Nikunj Gupta, we now have a new implementation of `hpx_main.hpp` on supported platforms (Linux, BSD and MacOS). This is intended to be a less fragile drop-in replacement for the old implementation relying on preprocessor macros. The new implementation does not require changes if you are using the CMake[3703] or pkg-config. The old behaviour can be restored by setting `HPX_WITH_DYNAMIC_HPX_MAIN=OFF` during CMake[3704] configuration. The implementation on Windows is unchanged.
- We have added functionality to allow passing scheduling hints to our schedulers. These will allow us to create executors that for example target a specific NUMA domain or allow for *HPX* threads to be pinned to a particular worker thread.
- We have significantly improved the performance of our futures implementation by making the shared state atomic.
- We have replaced Boostbook by Sphinx for our documentation. This means the documentation is easier to navigate with built-in search and table of contents. We have also added a quick start section and restructured the documentation to be easier to follow for new users.
- We have added a new option to the `--hpx:threads` command line option. It is now possible to use `cores` to tell *HPX* to only use one worker thread per core, unlike the existing option `all` which uses one worker thread per processing unit (processing unit can be a hyperthread if hyperthreads are available). The default value of `--hpx:threads` has also been changed to `cores` as this leads to better performance in most cases.
- All command line options can now be passed alongside configuration options when initializing *HPX*. This means that some options that were previously only available on the command line can now be set as configuration options.
- HPXMP is a portable, scalable, and flexible application programming interface using the OpenMP specification that supports multi-platform shared memory multiprocessing programming in C and C++. HPXMP can be enabled within *HPX* by setting `DHPX_WITH_HPXMP=ON` during CMake[3705] configuration.
- Two new performance counters were added for measuring the time spent doing background work. `/threads/time/background-work-duration` returns the time spent doing background on a given thread or locality, while `/threads/time/background-overhead` returns the fraction of time spent doing background work with respect to the overall time spent running the scheduler. The new performance counters are disabled by default and can be turned on by setting `HPX_WITH_BACKGROUND_THREAD_COUNTERS=ON` during CMake[3706] configuration.
- The idling behaviour of *HPX* has been tweaked to allow for faster idling. This is useful in interactive applications where the *HPX* worker threads may not have work all the time. This behaviour can be tweaked and turned off as before with `HPX_WITH_THREAD_MANAGER_IDLE_BACKOFF=OFF` during CMake[3707] configuration.

[3703] https://www.cmake.org
[3704] https://www.cmake.org
[3705] https://www.cmake.org
[3706] https://www.cmake.org
[3707] https://www.cmake.org

- It is now possible to register callback functions for *HPX* worker thread events. Callbacks can be registered for starting and stopping worker threads, and for when errors occur.

Breaking changes

- The implementation of `hpx_main.hpp` has changed. If you are using custom Makefiles you will need to make changes. Please see the documentation on *using Makefiles* for more details.
- The default value of `--hpx:threads` has changed from `all` to `cores`. The new option `cores` only starts one worker thread per core.
- We have dropped support for Boost 1.56 and 1.57. The minimal version of Boost we now test is 1.58.
- Our `boost::format`-based formatting implementation has been revised and replaced with a custom implementation. This changes the formatting syntax and requires changes if you are relying on `hpx::util::format` or `hpx::util::format_to`. The pull request for this change contains more information: PR #3266[3708].
- The following deprecated options have now been completely removed: `HPX_WITH_ASYNC_FUNCTION_COMPATIBILITY`, `HPX_WITH_LOCAL_DATAFLOW`, `HPX_WITH_GENERIC_EXECUTION_POLICY`, `HPX_WITH_BOOST_CHRONO_COMPATIBILITY`, `HPX_WITH_EXECUTOR_COMPATIBILITY`, `HPX_WITH_EXECUTION_POLICY_COMPATIBILITY`, and `HPX_WITH_TRANSFORM_REDUCE_COMPATIBILITY`.

Closed issues

- Issue #3538[3709] - numa handling incorrect for hwloc 2
- Issue #3533[3710] - Cmake version 3.5.1does not work (git ff26b35 2018-11-06)
- Issue #3526[3711] - Failed building hpx-1.2.0-rc1 on Ubuntu16.04 x86-64 Virtualbox VM
- Issue #3512[3712] - Build on aarch64 fails
- Issue #3475[3713] - HPX fails to link if the MPI parcelport is enabled
- Issue #3462[3714] - CMake configuration shows a minor and inconsequential failure to create a symlink
- Issue #3461[3715] - Compilation Problems with the most recent Clang
- Issue #3460[3716] - Deadlock when create_partitioner fails (assertion fails) in debug mode
- Issue #3455[3717] - HPX build failing with HWLOC errors on POWER8 with hwloc 1.8
- Issue #3438[3718] - HPX no longer builds on IBM POWER8

[3708] https://github.com/TheHPXProject/hpx/pull/3266
[3709] https://github.com/TheHPXProject/hpx/issues/3538
[3710] https://github.com/TheHPXProject/hpx/issues/3533
[3711] https://github.com/TheHPXProject/hpx/issues/3526
[3712] https://github.com/TheHPXProject/hpx/issues/3512
[3713] https://github.com/TheHPXProject/hpx/issues/3475
[3714] https://github.com/TheHPXProject/hpx/issues/3462
[3715] https://github.com/TheHPXProject/hpx/issues/3461
[3716] https://github.com/TheHPXProject/hpx/issues/3460
[3717] https://github.com/TheHPXProject/hpx/issues/3455
[3718] https://github.com/TheHPXProject/hpx/issues/3438

- Issue #3426[3719] - hpx build failed on MacOS
- Issue #3424[3720] - CircleCI builds broken for forked repositories
- Issue #3422[3721] - Benchmarks in tests.performance.local are not run nightly
- Issue #3408[3722] - CMake Targets for HPX
- Issue #3399[3723] - processing unit out of bounds
- Issue #3395[3724] - Floating point bug in hpx/runtime/threads/policies/scheduler_base.hpp
- Issue #3378[3725] - compile error with lcos::communicator
- Issue #3376[3726] - Failed to build HPX with APEX using clang
- Issue #3366[3727] - Adapted Safe_Object example fails for –hpx:threads > 1
- Issue #3360[3728] - Segmentation fault when passing component id as parameter
- Issue #3358[3729] - HPX runtime hangs after multiple (~thousands) start-stop sequences
- Issue #3352[3730] - Support TCP provider in libfabric ParcelPort
- Issue #3342[3731] - undefined reference to __atomic_load_16
- Issue #3339[3732] - setting command line options/flags from init cfg is not obvious
- Issue #3325[3733] - AGAS migrates components prematurely
- Issue #3321[3734] - hpx bad_parameter handling is awful
- Issue #3318[3735] - Benchmarks fail to build with C++11
- Issue #3304[3736] - hpx::threads::run_as_hpx_thread does not properly handle exceptions
- Issue #3300[3737] - Setting pu step or offset results in no threads in default pool
- Issue #3297[3738] - Crash with APEX when running Phylanx lra_csv with > 1 thread
- Issue #3296[3739] - Building HPX with APEX configuration gives compiler warnings
- Issue #3290[3740] - make tests failing at hello_world_component
- Issue #3285[3741] - possible compilation error when "using namespace std;" is defined before including "hpx" headers files

[3719] https://github.com/TheHPXProject/hpx/issues/3426
[3720] https://github.com/TheHPXProject/hpx/issues/3424
[3721] https://github.com/TheHPXProject/hpx/issues/3422
[3722] https://github.com/TheHPXProject/hpx/issues/3408
[3723] https://github.com/TheHPXProject/hpx/issues/3399
[3724] https://github.com/TheHPXProject/hpx/issues/3395
[3725] https://github.com/TheHPXProject/hpx/issues/3378
[3726] https://github.com/TheHPXProject/hpx/issues/3376
[3727] https://github.com/TheHPXProject/hpx/issues/3366
[3728] https://github.com/TheHPXProject/hpx/issues/3360
[3729] https://github.com/TheHPXProject/hpx/issues/3358
[3730] https://github.com/TheHPXProject/hpx/issues/3352
[3731] https://github.com/TheHPXProject/hpx/issues/3342
[3732] https://github.com/TheHPXProject/hpx/issues/3339
[3733] https://github.com/TheHPXProject/hpx/issues/3325
[3734] https://github.com/TheHPXProject/hpx/issues/3321
[3735] https://github.com/TheHPXProject/hpx/issues/3318
[3736] https://github.com/TheHPXProject/hpx/issues/3304
[3737] https://github.com/TheHPXProject/hpx/issues/3300
[3738] https://github.com/TheHPXProject/hpx/issues/3297
[3739] https://github.com/TheHPXProject/hpx/issues/3296
[3740] https://github.com/TheHPXProject/hpx/issues/3290
[3741] https://github.com/TheHPXProject/hpx/issues/3285

- Issue #3280[3742] - HPX fails on OSX
- Issue #3272[3743] - CircleCI does not upload generated docker image any more
- Issue #3270[3744] - Error when compiling CUDA examples
- Issue #3267[3745] - `tests.unit.host_.block_allocator` fails occasionally
- Issue #3264[3746] - Possible move to Sphinx for documentation
- Issue #3263[3747] - Documentation improvements
- Issue #3259[3748] - `set_parcel_write_handler` test fails occasionally
- Issue #3258[3749] - Links to source code in documentation are broken
- Issue #3247[3750] - Rare `tests.unit.host_.block_allocator` test failure on 1.1.0-rc1
- Issue #3244[3751] - Slowing down and speeding up an interval_timer
- Issue #3215[3752] - Cannot build both tests and examples on MSVC with pseudo-dependencies enabled
- Issue #3195[3753] - Unnecessary customization point route causing performance penalty
- Issue #3088[3754] - A strange thing in parallel::sort.
- Issue #2650[3755] - libfabric support for passive endpoints
- Issue #1205[3756] - TSS is broken

Closed pull requests

- PR #3542[3757] - Fix numa lookup from pu when using hwloc 2.x
- PR #3541[3758] - Fixing the build system of the MPI parcelport
- PR #3540[3759] - Updating HPX people section
- PR #3539[3760] - Splitting test to avoid OOM on CircleCI
- PR #3537[3761] - Fix guided exec
- PR #3536[3762] - Updating grants which support the LSU team

[3742] https://github.com/TheHPXProject/hpx/issues/3280
[3743] https://github.com/TheHPXProject/hpx/issues/3272
[3744] https://github.com/TheHPXProject/hpx/issues/3270
[3745] https://github.com/TheHPXProject/hpx/issues/3267
[3746] https://github.com/TheHPXProject/hpx/issues/3264
[3747] https://github.com/TheHPXProject/hpx/issues/3263
[3748] https://github.com/TheHPXProject/hpx/issues/3259
[3749] https://github.com/TheHPXProject/hpx/issues/3258
[3750] https://github.com/TheHPXProject/hpx/issues/3247
[3751] https://github.com/TheHPXProject/hpx/issues/3244
[3752] https://github.com/TheHPXProject/hpx/issues/3215
[3753] https://github.com/TheHPXProject/hpx/issues/3195
[3754] https://github.com/TheHPXProject/hpx/issues/3088
[3755] https://github.com/TheHPXProject/hpx/issues/2650
[3756] https://github.com/TheHPXProject/hpx/issues/1205
[3757] https://github.com/TheHPXProject/hpx/pull/3542
[3758] https://github.com/TheHPXProject/hpx/pull/3541
[3759] https://github.com/TheHPXProject/hpx/pull/3540
[3760] https://github.com/TheHPXProject/hpx/pull/3539
[3761] https://github.com/TheHPXProject/hpx/pull/3537
[3762] https://github.com/TheHPXProject/hpx/pull/3536

- PR #3535[3763] - Fix hiding of docker credentials
- PR #3534[3764] - Fixing #3533
- PR #3532[3765] - fixing minor doc typo –hpx:print-counter-at arg
- PR #3530[3766] - Changing APEX default tag to v2.1.0
- PR #3529[3767] - Remove leftover security options and documentation
- PR #3528[3768] - Fix hwloc version check
- PR #3524[3769] - Do not build guided pool examples with older GCC compilers
- PR #3523[3770] - Fix logging regression
- PR #3522[3771] - Fix more warnings
- PR #3521[3772] - Fixing argument handling in induction and reduction clauses for parallel::for_loop
- PR #3520[3773] - Remove docs symlink and versioned docs folders
- PR #3519[3774] - hpxMP release
- PR #3518[3775] - Change all steps to use new docker image on CircleCI
- PR #3516[3776] - Drop usage of deprecated facilities removed in C++17
- PR #3515[3777] - Remove remaining uses of Boost.TypeTraits
- PR #3513[3778] - Fixing a CMake problem when trying to use libfabric
- PR #3508[3779] - Remove memory_block component
- PR #3507[3780] - Propagating the MPI compile definitions to all relevant targets
- PR #3503[3781] - Update documentation colors and logo
- PR #3502[3782] - Fix bogus `throws` bindings in scheduled_thread_pool_impl
- PR #3501[3783] - Split parallel::remove_if tests to avoid OOM on CircleCI
- PR #3500[3784] - Support NONAMEPREFIX in add_hpx_library()
- PR #3497[3785] - Note that cuda support requires cmake 3.9

[3763] https://github.com/TheHPXProject/hpx/pull/3535
[3764] https://github.com/TheHPXProject/hpx/pull/3534
[3765] https://github.com/TheHPXProject/hpx/pull/3532
[3766] https://github.com/TheHPXProject/hpx/pull/3530
[3767] https://github.com/TheHPXProject/hpx/pull/3529
[3768] https://github.com/TheHPXProject/hpx/pull/3528
[3769] https://github.com/TheHPXProject/hpx/pull/3524
[3770] https://github.com/TheHPXProject/hpx/pull/3523
[3771] https://github.com/TheHPXProject/hpx/pull/3522
[3772] https://github.com/TheHPXProject/hpx/pull/3521
[3773] https://github.com/TheHPXProject/hpx/pull/3520
[3774] https://github.com/TheHPXProject/hpx/pull/3519
[3775] https://github.com/TheHPXProject/hpx/pull/3518
[3776] https://github.com/TheHPXProject/hpx/pull/3516
[3777] https://github.com/TheHPXProject/hpx/pull/3515
[3778] https://github.com/TheHPXProject/hpx/pull/3513
[3779] https://github.com/TheHPXProject/hpx/pull/3508
[3780] https://github.com/TheHPXProject/hpx/pull/3507
[3781] https://github.com/TheHPXProject/hpx/pull/3503
[3782] https://github.com/TheHPXProject/hpx/pull/3502
[3783] https://github.com/TheHPXProject/hpx/pull/3501
[3784] https://github.com/TheHPXProject/hpx/pull/3500
[3785] https://github.com/TheHPXProject/hpx/pull/3497

- PR #3495[3786] - Fixing dataflow
- PR #3493[3787] - Remove deprecated options for 1.2.0 part 2
- PR #3492[3788] - Add CUDA_LINK_LIBRARIES_KEYWORD to allow PRIVATE keyword in linkage t…
- PR #3491[3789] - Changing Base docker image
- PR #3490[3790] - Don't create tasks immediately with hpx::apply
- PR #3489[3791] - Remove deprecated options for 1.2.0
- PR #3488[3792] - Revert "Use BUILD_INTERFACE generator expression to fix cmake flag exports"
- PR #3487[3793] - Revert "Fixing type attribute warning for transfer_action"
- PR #3485[3794] - Use BUILD_INTERFACE generator expression to fix cmake flag exports
- PR #3483[3795] - Fixing type attribute warning for transfer_action
- PR #3481[3796] - Remove unused variables
- PR #3480[3797] - Towards a more lightweight transfer action
- PR #3479[3798] - Fix FLAGS - Use correct version of target_compile_options
- PR #3478[3799] - Making sure the application's exit code is properly propagated back to the OS
- PR #3476[3800] - Don't print docker credentials as part of the environment.
- PR #3473[3801] - Fixing invalid cmake code if no jemalloc prefix was given
- PR #3472[3802] - Attempting to work around recent clang test compilation failures
- PR #3471[3803] - Enable jemalloc on windows
- PR #3470[3804] - Updates readme
- PR #3468[3805] - Avoid hang if there is an exception thrown during startup
- PR #3467[3806] - Add compiler specific fallthrough attributes if C++17 attribute is not available

3786 https://github.com/TheHPXProject/hpx/pull/3495
3787 https://github.com/TheHPXProject/hpx/pull/3493
3788 https://github.com/TheHPXProject/hpx/pull/3492
3789 https://github.com/TheHPXProject/hpx/pull/3491
3790 https://github.com/TheHPXProject/hpx/pull/3490
3791 https://github.com/TheHPXProject/hpx/pull/3489
3792 https://github.com/TheHPXProject/hpx/pull/3488
3793 https://github.com/TheHPXProject/hpx/pull/3487
3794 https://github.com/TheHPXProject/hpx/pull/3485
3795 https://github.com/TheHPXProject/hpx/pull/3483
3796 https://github.com/TheHPXProject/hpx/pull/3481
3797 https://github.com/TheHPXProject/hpx/pull/3480
3798 https://github.com/TheHPXProject/hpx/pull/3479
3799 https://github.com/TheHPXProject/hpx/pull/3478
3800 https://github.com/TheHPXProject/hpx/pull/3476
3801 https://github.com/TheHPXProject/hpx/pull/3473
3802 https://github.com/TheHPXProject/hpx/pull/3472
3803 https://github.com/TheHPXProject/hpx/pull/3471
3804 https://github.com/TheHPXProject/hpx/pull/3470
3805 https://github.com/TheHPXProject/hpx/pull/3468
3806 https://github.com/TheHPXProject/hpx/pull/3467

- PR #3466[3807] - - bugfix : fix compilation with llvm-7.0
- PR #3465[3808] - This patch adds various optimizations extracted from the thread_local_allocator work
- PR #3464[3809] - Check for forked repos in CircleCI docker push step
- PR #3463[3810] - - cmake : create the parent directory before symlinking
- PR #3459[3811] - Remove unused/incomplete functionality from util/logging
- PR #3458[3812] - Fix a problem with scope of CMAKE_CXX_FLAGS and hpx_add_compile_flag
- PR #3457[3813] - Fixing more size_t -> int16_t (and similar) warnings
- PR #3456[3814] - Add #ifdefs to topology.cpp to support old hwloc versions again
- PR #3454[3815] - Fixing warnings related to silent conversion of size_t –> int16_t
- PR #3451[3816] - Add examples as unit tests
- PR #3450[3817] - Constexpr-fying bind and other functional facilities
- PR #3446[3818] - Fix some thread suspension timeouts
- PR #3445[3819] - Fix various warnings
- PR #3443[3820] - Only enable service pool config options if pools are enabled
- PR #3441[3821] - Fix missing closing brackets in documentation
- PR #3439[3822] - Use correct MPI CXX libraries for MPI parcelport
- PR #3436[3823] - Add projection function to find_* (and fix very bad bug)
- PR #3435[3824] - Fixing 1205
- PR #3434[3825] - Fix threads cores
- PR #3433[3826] - Add Heise Online to release announcement list
- PR #3432[3827] - Don't track task dependencies for distributed runs
- PR #3431[3828] - Circle CI setting changes for hpxMP

[3807] https://github.com/TheHPXProject/hpx/pull/3466
[3808] https://github.com/TheHPXProject/hpx/pull/3465
[3809] https://github.com/TheHPXProject/hpx/pull/3464
[3810] https://github.com/TheHPXProject/hpx/pull/3463
[3811] https://github.com/TheHPXProject/hpx/pull/3459
[3812] https://github.com/TheHPXProject/hpx/pull/3458
[3813] https://github.com/TheHPXProject/hpx/pull/3457
[3814] https://github.com/TheHPXProject/hpx/pull/3456
[3815] https://github.com/TheHPXProject/hpx/pull/3454
[3816] https://github.com/TheHPXProject/hpx/pull/3451
[3817] https://github.com/TheHPXProject/hpx/pull/3450
[3818] https://github.com/TheHPXProject/hpx/pull/3446
[3819] https://github.com/TheHPXProject/hpx/pull/3445
[3820] https://github.com/TheHPXProject/hpx/pull/3443
[3821] https://github.com/TheHPXProject/hpx/pull/3441
[3822] https://github.com/TheHPXProject/hpx/pull/3439
[3823] https://github.com/TheHPXProject/hpx/pull/3436
[3824] https://github.com/TheHPXProject/hpx/pull/3435
[3825] https://github.com/TheHPXProject/hpx/pull/3434
[3826] https://github.com/TheHPXProject/hpx/pull/3433
[3827] https://github.com/TheHPXProject/hpx/pull/3432
[3828] https://github.com/TheHPXProject/hpx/pull/3431

- PR #3430[3829] - Fix unused params warning
- PR #3429[3830] - One thread per core
- PR #3428[3831] - This suppresses a deprecation warning that is being issued by MSVC 19.15.26726
- PR #3427[3832] - Fixes #3426
- PR #3425[3833] - Use source cache and workspace between job steps on CircleCI
- PR #3421[3834] - Add CDash timing output to future overhead test (for graphs)
- PR #3420[3835] - Add guided_pool_executor
- PR #3419[3836] - Fix typo in CircleCI config
- PR #3418[3837] - Add sphinx documentation
- PR #3415[3838] - Scheduler NUMA hint and shared priority scheduler
- PR #3414[3839] - Adding step to synchronize the APEX release
- PR #3413[3840] - Fixing multiple defines of APEX_HAVE_HPX
- PR #3412[3841] - Fixes linking with libhpx_wrap error with BSD and Windows based systems
- PR #3410[3842] - Fix typo in CMakeLists.txt
- PR #3409[3843] - Fix brackets and indentation in existing_performance_counters.qbk
- PR #3407[3844] - Fix unused param and extra ; warnings emitted by gcc 8.x
- PR #3406[3845] - Adding thread local allocator and use it for future shared states
- PR #3405[3846] - Adding DHPX_HAVE_THREAD_LOCAL_STORAGE=ON to builds
- PR #3404[3847] - fixing multiple definition of main() in linux
- PR #3402[3848] - Allow debug option to be enabled only for Linux systems with dynamic main on
- PR #3401[3849] - Fix cuda_future_helper.h when compiling with C++11
- PR #3400[3850] - Fix floating point exception scheduler_base idle backoff

[3829] https://github.com/TheHPXProject/hpx/pull/3430
[3830] https://github.com/TheHPXProject/hpx/pull/3429
[3831] https://github.com/TheHPXProject/hpx/pull/3428
[3832] https://github.com/TheHPXProject/hpx/pull/3427
[3833] https://github.com/TheHPXProject/hpx/pull/3425
[3834] https://github.com/TheHPXProject/hpx/pull/3421
[3835] https://github.com/TheHPXProject/hpx/pull/3420
[3836] https://github.com/TheHPXProject/hpx/pull/3419
[3837] https://github.com/TheHPXProject/hpx/pull/3418
[3838] https://github.com/TheHPXProject/hpx/pull/3415
[3839] https://github.com/TheHPXProject/hpx/pull/3414
[3840] https://github.com/TheHPXProject/hpx/pull/3413
[3841] https://github.com/TheHPXProject/hpx/pull/3412
[3842] https://github.com/TheHPXProject/hpx/pull/3410
[3843] https://github.com/TheHPXProject/hpx/pull/3409
[3844] https://github.com/TheHPXProject/hpx/pull/3407
[3845] https://github.com/TheHPXProject/hpx/pull/3406
[3846] https://github.com/TheHPXProject/hpx/pull/3405
[3847] https://github.com/TheHPXProject/hpx/pull/3404
[3848] https://github.com/TheHPXProject/hpx/pull/3402
[3849] https://github.com/TheHPXProject/hpx/pull/3401
[3850] https://github.com/TheHPXProject/hpx/pull/3400

- PR #3398[3851] - Atomic future state
- PR #3397[3852] - Fixing code for older gcc versions
- PR #3396[3853] - Allowing to register thread event functions (start/stop/error)
- PR #3394[3854] - Fix small mistake in primary_namespace_server.cpp
- PR #3393[3855] - Explicitly instantiate configured schedulers
- PR #3392[3856] - Add performance counters background overhead and background work duration
- PR #3391[3857] - Adapt integration of HPXMP to latest build system changes
- PR #3390[3858] - Make AGAS measurements optional
- PR #3389[3859] - Fix deadlock during shutdown
- PR #3388[3860] - Add several functionalities allowing to optimize synchronous action invocation
- PR #3387[3861] - Add cmake option to opt out of fail-compile tests
- PR #3386[3862] - Adding support for boost::container::small_vector to dataflow
- PR #3385[3863] - Adds Debug option for hpx initializing from main
- PR #3384[3864] - This hopefully fixes two tests that occasionally fail
- PR #3383[3865] - Making sure thread local storage is enable for hpxMP
- PR #3382[3866] - Fix usage of HPX_CAPTURE together with default value capture [=]
- PR #3381[3867] - Replace undefined instantiations of uniform_int_distribution
- PR #3380[3868] - Add missing semicolons to uses of HPX_COMPILER_FENCE
- PR #3379[3869] - Fixing #3378
- PR #3377[3870] - Adding build system support to integrate hpxmp into hpx at the user's machine
- PR #3375[3871] - Replacing wrapper for __libc_start_main with main

[3851] https://github.com/TheHPXProject/hpx/pull/3398
[3852] https://github.com/TheHPXProject/hpx/pull/3397
[3853] https://github.com/TheHPXProject/hpx/pull/3396
[3854] https://github.com/TheHPXProject/hpx/pull/3394
[3855] https://github.com/TheHPXProject/hpx/pull/3393
[3856] https://github.com/TheHPXProject/hpx/pull/3392
[3857] https://github.com/TheHPXProject/hpx/pull/3391
[3858] https://github.com/TheHPXProject/hpx/pull/3390
[3859] https://github.com/TheHPXProject/hpx/pull/3389
[3860] https://github.com/TheHPXProject/hpx/pull/3388
[3861] https://github.com/TheHPXProject/hpx/pull/3387
[3862] https://github.com/TheHPXProject/hpx/pull/3386
[3863] https://github.com/TheHPXProject/hpx/pull/3385
[3864] https://github.com/TheHPXProject/hpx/pull/3384
[3865] https://github.com/TheHPXProject/hpx/pull/3383
[3866] https://github.com/TheHPXProject/hpx/pull/3382
[3867] https://github.com/TheHPXProject/hpx/pull/3381
[3868] https://github.com/TheHPXProject/hpx/pull/3380
[3869] https://github.com/TheHPXProject/hpx/pull/3379
[3870] https://github.com/TheHPXProject/hpx/pull/3377
[3871] https://github.com/TheHPXProject/hpx/pull/3375

- PR #3374[3872] - Adds hpx_wrap to HPX_LINK_LIBRARIES which links only when specified.
- PR #3373[3873] - Forcing cache settings in HPXConfig.cmake to guarantee updated values
- PR #3372[3874] - Fix some more c++11 build problems
- PR #3371[3875] - Adds HPX_LINKER_FLAGS to HPX applications without editing their source codes
- PR #3370[3876] - util::format: add type_specifier<> specializations for %!s(MISSING) and %!l(MISSING)s
- PR #3369[3877] - Adding configuration option to allow explicit disable of the new hpx_main feature on Linux
- PR #3368[3878] - Updates doc with recent hpx_wrap implementation
- PR #3367[3879] - Adds Mac OS implementation to hpx_main.hpp
- PR #3365[3880] - Fix order of hpx libs in HPX_CONF_LIBRARIES.
- PR #3363[3881] - Apex fixing null wrapper
- PR #3361[3882] - Making sure all parcels get destroyed on an HPX thread (TCP pp)
- PR #3359[3883] - Feature/improveerrorforcompiler
- PR #3357[3884] - Static/dynamic executable implementation
- PR #3355[3885] - Reverting changes introduced by #3283 as those make applications hang
- PR #3354[3886] - Add external dependencies to HPX_LIBRARY_DIR
- PR #3353[3887] - Fix libfabric tcp
- PR #3351[3888] - Move obsolete header to tests directory.
- PR #3350[3889] - Renaming two functions to avoid problem described in #3285
- PR #3349[3890] - Make idle backoff exponential with maximum sleep time
- PR #3347[3891] - Replace *simple_component** with *component** in the Documentation
- PR #3346[3892] - Fix CMakeLists.txt example in quick start

[3872] https://github.com/TheHPXProject/hpx/pull/3374
[3873] https://github.com/TheHPXProject/hpx/pull/3373
[3874] https://github.com/TheHPXProject/hpx/pull/3372
[3875] https://github.com/TheHPXProject/hpx/pull/3371
[3876] https://github.com/TheHPXProject/hpx/pull/3370
[3877] https://github.com/TheHPXProject/hpx/pull/3369
[3878] https://github.com/TheHPXProject/hpx/pull/3368
[3879] https://github.com/TheHPXProject/hpx/pull/3367
[3880] https://github.com/TheHPXProject/hpx/pull/3365
[3881] https://github.com/TheHPXProject/hpx/pull/3363
[3882] https://github.com/TheHPXProject/hpx/pull/3361
[3883] https://github.com/TheHPXProject/hpx/pull/3359
[3884] https://github.com/TheHPXProject/hpx/pull/3357
[3885] https://github.com/TheHPXProject/hpx/pull/3355
[3886] https://github.com/TheHPXProject/hpx/pull/3354
[3887] https://github.com/TheHPXProject/hpx/pull/3353
[3888] https://github.com/TheHPXProject/hpx/pull/3351
[3889] https://github.com/TheHPXProject/hpx/pull/3350
[3890] https://github.com/TheHPXProject/hpx/pull/3349
[3891] https://github.com/TheHPXProject/hpx/pull/3347
[3892] https://github.com/TheHPXProject/hpx/pull/3346

- PR #3345[3893] - Fix automatic setting of HPX_MORE_THAN_64_THREADS
- PR #3344[3894] - Reduce amount of information printed for unknown command line options
- PR #3343[3895] - Safeguard HPX against destruction in global contexts
- PR #3341[3896] - Allowing for all command line options to be used as configuration settings
- PR #3340[3897] - Always convert inspect results to JUnit XML
- PR #3336[3898] - Only run docker push on master on CircleCI
- PR #3335[3899] - Update description of hpx.os_threads config parameter.
- PR #3334[3900] - Making sure early logging settings don't get mixed with others
- PR #3333[3901] - Update CMake links and versions in documentation
- PR #3332[3902] - Add notes on target suffixes to CMake documentation
- PR #3331[3903] - Add quickstart section to documentation
- PR #3330[3904] - Rename resource_partitioner test to avoid conflicts with pseudodependencies
- PR #3328[3905] - Making sure object is pinned while executing actions, even if action returns a future
- PR #3327[3906] - Add missing std::forward to tuple.hpp
- PR #3326[3907] - Make sure logging is up and running while modules are being discovered.
- PR #3324[3908] - Replace C++14 overload of std::equal with C++11 code.
- PR #3323[3909] - Fix a missing apex thread data (wrapper) initialization
- PR #3320[3910] - Adding support for -std=c++2a (define *HPX_WITH_CXX2A=On*)
- PR #3319[3911] - Replacing C++14 feature with equivalent C++11 code
- PR #3317[3912] - Fix compilation with VS 15.7.1 and /std:c++latest
- PR #3316[3913] - Fix includes for 1d_stencil_*_omp examples

[3893] https://github.com/TheHPXProject/hpx/pull/3345
[3894] https://github.com/TheHPXProject/hpx/pull/3344
[3895] https://github.com/TheHPXProject/hpx/pull/3343
[3896] https://github.com/TheHPXProject/hpx/pull/3341
[3897] https://github.com/TheHPXProject/hpx/pull/3340
[3898] https://github.com/TheHPXProject/hpx/pull/3336
[3899] https://github.com/TheHPXProject/hpx/pull/3335
[3900] https://github.com/TheHPXProject/hpx/pull/3334
[3901] https://github.com/TheHPXProject/hpx/pull/3333
[3902] https://github.com/TheHPXProject/hpx/pull/3332
[3903] https://github.com/TheHPXProject/hpx/pull/3331
[3904] https://github.com/TheHPXProject/hpx/pull/3330
[3905] https://github.com/TheHPXProject/hpx/pull/3328
[3906] https://github.com/TheHPXProject/hpx/pull/3327
[3907] https://github.com/TheHPXProject/hpx/pull/3326
[3908] https://github.com/TheHPXProject/hpx/pull/3324
[3909] https://github.com/TheHPXProject/hpx/pull/3323
[3910] https://github.com/TheHPXProject/hpx/pull/3320
[3911] https://github.com/TheHPXProject/hpx/pull/3319
[3912] https://github.com/TheHPXProject/hpx/pull/3317
[3913] https://github.com/TheHPXProject/hpx/pull/3316

- PR #3314[3914] - Remove some unused parameter warnings
- PR #3313[3915] - Fix pu-step and pu-offset command line options
- PR #3312[3916] - Add conversion of inspect reports to JUnit XML
- PR #3311[3917] - Fix escaping of closing braces in format specification syntax
- PR #3310[3918] - Don't overwrite user settings with defaults in registration database
- PR #3309[3919] - Fixing potential stack overflow for dataflow
- PR #3308[3920] - This updates the .clang-format configuration file to utilize newer features
- PR #3306[3921] - Marking migratable objects in their gid to allow not handling migration in AGAS
- PR #3305[3922] - Add proper exception handling to run_as_hpx_thread
- PR #3303[3923] - Changed std::rand to a better inbuilt PRNG Generator
- PR #3302[3924] - All non-migratable (simple) components now encode their lva and component type in their gid
- PR #3301[3925] - Add nullptr_t overloads to resource partitioner
- PR #3298[3926] - Apex task wrapper memory bug
- PR #3295[3927] - Fix mistakes after merge of CircleCI config
- PR #3294[3928] - Fix partitioned vector include in partitioned_vector_find tests
- PR #3293[3929] - Adding emplace support to promise and make_ready_future
- PR #3292[3930] - Add new cuda kernel synchronization with hpx::future demo
- PR #3291[3931] - Fixes #3290
- PR #3289[3932] - Fixing Docker image creation
- PR #3288[3933] - Avoid allocating shared state for wait_all
- PR #3287[3934] - Fixing /scheduler/utilization/instantaneous performance counter
- PR #3286[3935] - dataflow() and future::then() use sync policy where possible

3914 https://github.com/TheHPXProject/hpx/pull/3314
3915 https://github.com/TheHPXProject/hpx/pull/3313
3916 https://github.com/TheHPXProject/hpx/pull/3312
3917 https://github.com/TheHPXProject/hpx/pull/3311
3918 https://github.com/TheHPXProject/hpx/pull/3310
3919 https://github.com/TheHPXProject/hpx/pull/3309
3920 https://github.com/TheHPXProject/hpx/pull/3308
3921 https://github.com/TheHPXProject/hpx/pull/3306
3922 https://github.com/TheHPXProject/hpx/pull/3305
3923 https://github.com/TheHPXProject/hpx/pull/3303
3924 https://github.com/TheHPXProject/hpx/pull/3302
3925 https://github.com/TheHPXProject/hpx/pull/3301
3926 https://github.com/TheHPXProject/hpx/pull/3298
3927 https://github.com/TheHPXProject/hpx/pull/3295
3928 https://github.com/TheHPXProject/hpx/pull/3294
3929 https://github.com/TheHPXProject/hpx/pull/3293
3930 https://github.com/TheHPXProject/hpx/pull/3292
3931 https://github.com/TheHPXProject/hpx/pull/3291
3932 https://github.com/TheHPXProject/hpx/pull/3289
3933 https://github.com/TheHPXProject/hpx/pull/3288
3934 https://github.com/TheHPXProject/hpx/pull/3287
3935 https://github.com/TheHPXProject/hpx/pull/3286

- PR #3284[3936] - Background thread can use relaxed atomics to manipulate thread state
- PR #3283[3937] - Do not unwrap ready future
- PR #3282[3938] - Fix virtual method override warnings in static schedulers
- PR #3281[3939] - Disable set_area_membind_nodeset for OSX
- PR #3279[3940] - Add two variations to the future_overhead benchmark
- PR #3278[3941] - Fix circleci workspace
- PR #3277[3942] - Support external plugins
- PR #3276[3943] - Fix missing parenthesis in hello_compute.cu.
- PR #3274[3944] - Reinit counters synchronously in reinit_counters test
- PR #3273[3945] - Splitting tests to avoid compiler OOM
- PR #3271[3946] - Remove leftover code from context_generic_context.hpp
- PR #3269[3947] - Fix bulk_construct with count = 0
- PR #3268[3948] - Replace constexpr with HPX_CXX14_CONSTEXPR and HPX_CONSTEXPR
- PR #3266[3949] - Replace boost::format with custom sprintf-based implementation
- PR #3265[3950] - Split parallel tests on CircleCI
- PR #3262[3951] - Making sure documentation correctly links to source files
- PR #3261[3952] - Apex refactoring fix rebind
- PR #3260[3953] - Isolate performance counter parser into a separate TU
- PR #3256[3954] - Post 1.1.0 version bumps
- PR #3254[3955] - Adding trait for actions allowing to make runtime decision on whether to execute it directly
- PR #3253[3956] - Bump minimal supported Boost to 1.58.0
- PR #3251[3957] - Adds new feature: changing interval used in interval_timer (issue 3244)

[3936] https://github.com/TheHPXProject/hpx/pull/3284
[3937] https://github.com/TheHPXProject/hpx/pull/3283
[3938] https://github.com/TheHPXProject/hpx/pull/3282
[3939] https://github.com/TheHPXProject/hpx/pull/3281
[3940] https://github.com/TheHPXProject/hpx/pull/3279
[3941] https://github.com/TheHPXProject/hpx/pull/3278
[3942] https://github.com/TheHPXProject/hpx/pull/3277
[3943] https://github.com/TheHPXProject/hpx/pull/3276
[3944] https://github.com/TheHPXProject/hpx/pull/3274
[3945] https://github.com/TheHPXProject/hpx/pull/3273
[3946] https://github.com/TheHPXProject/hpx/pull/3271
[3947] https://github.com/TheHPXProject/hpx/pull/3269
[3948] https://github.com/TheHPXProject/hpx/pull/3268
[3949] https://github.com/TheHPXProject/hpx/pull/3266
[3950] https://github.com/TheHPXProject/hpx/pull/3265
[3951] https://github.com/TheHPXProject/hpx/pull/3262
[3952] https://github.com/TheHPXProject/hpx/pull/3261
[3953] https://github.com/TheHPXProject/hpx/pull/3260
[3954] https://github.com/TheHPXProject/hpx/pull/3256
[3955] https://github.com/TheHPXProject/hpx/pull/3254
[3956] https://github.com/TheHPXProject/hpx/pull/3253
[3957] https://github.com/TheHPXProject/hpx/pull/3251

- PR #3239[3958] - Changing std::rand() to a better inbuilt PRNG generator.
- PR #3234[3959] - Disable background thread when networking is off
- PR #3232[3960] - Clean up suspension tests
- PR #3230[3961] - Add optional scheduler mode parameter to create_thread_pool function
- PR #3228[3962] - Allow suspension also on static schedulers
- PR #3163[3963] - libfabric parcelport w/o HPX_PARCELPORT_LIBFABRIC_ENDPOINT_RDM
- PR #3036[3964] - Switching to CircleCI 2.0

HPX V1.1.0 (Mar 24, 2018)

General changes

Here are some of the main highlights and changes for this release (in no particular order):

- We have changed the way *HPX* manages the processing units on a node. We do not longer implicitly bind all available cores to a single thread pool. The user has now full control over what processing units are bound to what thread pool, each with a separate scheduler. It is now also possible to create your own scheduler implementation and control what processing units this scheduler should use. We added the `hpx::resource::partitioner` that manages all available processing units and assigns resources to the used thread pools. Thread pools can be now be suspended/resumed independently. This functionality helps in running *HPX* concurrently to code that is directly relying on OpenMP[3965] and/or MPI[3966].
- We have continued to implement various parallel algorithms. *HPX* now almost completely implements all of the parallel algorithms as specified by the C++17 Standard[3967]. We have also continued to implement these algorithms for the distributed use case (for segmented data structures, such as `hpx::partitioned_vector`).
- Added a compatibility layer for `std::thread`, `std::mutex`, and `std::condition_variable` allowing for the code to use those facilities where available and to fall back to the corresponding Boost facilities otherwise. The CMake[3968] configuration option `-DHPX_WITH_THREAD_COMPATIBILITY=On` can be used to force using the Boost equivalents.
- The parameter sequence for the `hpx::parallel::transform_inclusive_scan` overload taking one iterator range has changed (again) to match the changes this algorithm has undergone while being moved to C++17. The old overloads can be still enabled at configure time by passing `-DHPX_WITH_TRANSFORM_REDUCE_COMPATIBILITY=On` to CMake[3969].

[3958] https://github.com/TheHPXProject/hpx/pull/3239
[3959] https://github.com/TheHPXProject/hpx/pull/3234
[3960] https://github.com/TheHPXProject/hpx/pull/3232
[3961] https://github.com/TheHPXProject/hpx/pull/3230
[3962] https://github.com/TheHPXProject/hpx/pull/3228
[3963] https://github.com/TheHPXProject/hpx/pull/3163
[3964] https://github.com/TheHPXProject/hpx/pull/3036
[3965] https://openmp.org/wp/
[3966] https://en.wikipedia.org/wiki/Message_Passing_Interface
[3967] http://www.open-std.org/jtc1/sc22/wg21
[3968] https://www.cmake.org
[3969] https://www.cmake.org

- The parameter sequence for the `hpx::parallel::inclusive_scan` overload taking one iterator range has changed to match the changes this algorithm has undergone while being moved to C++17. The old overloads can be still enabled at configure time by passing `-DHPX_WITH_INCLUSIVE_SCAN_COMPATIBILITY=On` to CMake.
- Added a helper facility `hpx::local_new` which is equivalent to `hpx::new_` except that it creates components locally only. As a consequence, the used component constructor may accept non-serializable argument types and/or non-const references or pointers.
- Removed the (broken) component type `hpx::lcos::queue`<T>. The old type is still available at configure time by passing `-DHPX_WITH_QUEUE_COMPATIBILITY=On` to CMake.
- The parallel algorithms adopted for C++17 restrict the iterator categories usable with those to at least forward iterators. Our implementation of the parallel algorithms was supporting input iterators (and output iterators) as well by simply falling back to sequential execution. We have now made our implementations conforming by requiring at least forward iterators. In order to enable the old behavior use the compatibility option `-DHPX_WITH_ALGORITHM_INPUT_ITERATOR_SUPPORT=On` on the CMake[3970] command line.
- We have added the functionalities allowing for LCOs being implemented using (simple) components. Before LCOs had to always be implemented using managed components.
- User defined components don't have to be default-constructible anymore. Return types from actions don't have to be default-constructible anymore either. Our serialization layer now in general supports non-default-constructible types.
- We have added a new launch policy `hpx::launch::lazy` that allows oneto defer the decision on what launch policy to use to the point of execution. This policy is initialized with a function (object) that – when invoked – is expected to produce the desired launch policy.

Breaking changes

- We have dropped support for the gcc compiler version V4.8. The minimal gcc version we now test on is gcc V4.9. The minimally required version of CMake[3971] is now V3.3.2.
- We have dropped support for the Visual Studio 2013 compiler version. The minimal Visual Studio version we now test on is Visual Studio 2015.5.
- We have dropped support for the Boost V1.51-V1.54. The minimal version of Boost we now test is Boost V1.55.
- We have dropped support for the `hpx::util::unwrapped` API. `hpx::util::unwrapped` will stay functional to some degree, until it finally gets removed in a later version of HPX. The functional usage of `hpx::util::unwrapped` should be changed to the new `hpx::util::unwrapping` function whereas the immediate usage should be replaced to `hpx::util::unwrap`.
- The performance counter names referring to properties as exposed by the threading subsystem have changes as those now additionally have to specify the thread-pool. See the corresponding documentation for more details.
- The overloads of `hpx::async` that invoke an action do not perform implicit unwrapping of the returned future anymore in case the invoked function does return a future in the

[3970] https://www.cmake.org
[3971] https://www.cmake.org

first place. In this case `hpx::async` now returns a `hpx::future<future<T>>` making its behavior conforming to its local counterpart.

- We have replaced the use of `boost::exception_ptr` in our APIs with the equivalent `std::exception_ptr`. Please change your codes accordingly. No compatibility settings are provided.
- We have removed the compatibility settings for `HPX_WITH_COLOCATED_BACKWARDS_COMPATIBILITY` and `HPX_WITH_COMPONENT_GET_GID_COMPATIBILITY` as their life-cycle has reached its end.
- We have removed the experimental thread schedulers hierarchy_scheduler, periodic_priority_scheduler and throttling_scheduler in an effort to clean up and consolidate our thread schedulers.

Bug fixes (closed tickets)

Here is a list of the important tickets we closed for this release.

- PR #3250[3972] - Apex refactoring with guids
- PR #3249[3973] - Updating People.qbk
- PR #3246[3974] - Assorted fixes for CUDA
- PR #3245[3975] - Apex refactoring with guids
- PR #3242[3976] - Modify task counting in thread_queue.hpp
- PR #3240[3977] - Fixed typos
- PR #3238[3978] - Readding accidentally removed std::abort
- PR #3237[3979] - Adding Pipeline example
- PR #3236[3980] - Fixing memory_block
- PR #3233[3981] - Make schedule_thread take suspended threads into account
- Issue #3226[3982] - memory_block is breaking, signaling SIGSEGV on a thread on creation and freeing
- PR #3225[3983] - Applying quick fix for hwloc-2.0
- Issue #3224[3984] - HPX counters crashing the application
- PR #3223[3985] - Fix returns when setting config entries

[3972] https://github.com/TheHPXProject/hpx/pull/3250
[3973] https://github.com/TheHPXProject/hpx/pull/3249
[3974] https://github.com/TheHPXProject/hpx/pull/3246
[3975] https://github.com/TheHPXProject/hpx/pull/3245
[3976] https://github.com/TheHPXProject/hpx/pull/3242
[3977] https://github.com/TheHPXProject/hpx/pull/3240
[3978] https://github.com/TheHPXProject/hpx/pull/3238
[3979] https://github.com/TheHPXProject/hpx/pull/3237
[3980] https://github.com/TheHPXProject/hpx/pull/3236
[3981] https://github.com/TheHPXProject/hpx/pull/3233
[3982] https://github.com/TheHPXProject/hpx/issues/3226
[3983] https://github.com/TheHPXProject/hpx/pull/3225
[3984] https://github.com/TheHPXProject/hpx/issues/3224
[3985] https://github.com/TheHPXProject/hpx/pull/3223

- Issue #3222[3986] - Errors linking libhpx.so
- Issue #3221[3987] - HPX on Mac OS X with HWLoc 2.0.0 fails to run
- PR #3216[3988] - Reorder a variadic array to satisfy VS 2017 15.6
- PR #3214[3989] - Changed prerequisites.qbk to avoid confusion while building boost
- PR #3213[3990] - Relax locks for thread suspension to avoid holding locks when yielding
- PR #3212[3991] - Fix check in sequenced_executor test
- PR #3211[3992] - Use preinit_array to set argc/argv in init_globally example
- PR #3210[3993] - Adapted parallel::{search | search_n} for Ranges TS (see #1668)
- PR #3209[3994] - Fix locking problems during shutdown
- Issue #3208[3995] - init_globally throwing a run-time error
- PR #3206[3996] - Addition of new arithmetic performance counter "Count"
- PR #3205[3997] - Fixing return type calculation for bulk_then_execute
- PR #3204[3998] - Changing std::rand() to a better inbuilt PRNG generator
- PR #3203[3999] - Resolving problems during shutdown for VS2015
- PR #3202[4000] - Making sure resource partitioner is not accessed if its not valid
- PR #3201[4001] - Fixing optional::swap
- Issue #3200[4002] - hpx::util::optional fails
- PR #3199[4003] - Fix sliding_semaphore test
- PR #3198[4004] - Set pre_main status before launching run_helper
- PR #3197[4005] - Update README.rst
- PR #3194[4006] - parallel::{fill|fill_n} updated for Ranges TS
- PR #3193[4007] - Updating Runtime.cpp by adding correct description of Performance counters during register
- PR #3191[4008] - Fix sliding_semaphore_2338 test

[3986] https://github.com/TheHPXProject/hpx/issues/3222
[3987] https://github.com/TheHPXProject/hpx/issues/3221
[3988] https://github.com/TheHPXProject/hpx/pull/3216
[3989] https://github.com/TheHPXProject/hpx/pull/3214
[3990] https://github.com/TheHPXProject/hpx/pull/3213
[3991] https://github.com/TheHPXProject/hpx/pull/3212
[3992] https://github.com/TheHPXProject/hpx/pull/3211
[3993] https://github.com/TheHPXProject/hpx/pull/3210
[3994] https://github.com/TheHPXProject/hpx/pull/3209
[3995] https://github.com/TheHPXProject/hpx/issues/3208
[3996] https://github.com/TheHPXProject/hpx/pull/3206
[3997] https://github.com/TheHPXProject/hpx/pull/3205
[3998] https://github.com/TheHPXProject/hpx/pull/3204
[3999] https://github.com/TheHPXProject/hpx/pull/3203
[4000] https://github.com/TheHPXProject/hpx/pull/3202
[4001] https://github.com/TheHPXProject/hpx/pull/3201
[4002] https://github.com/TheHPXProject/hpx/issues/3200
[4003] https://github.com/TheHPXProject/hpx/pull/3199
[4004] https://github.com/TheHPXProject/hpx/pull/3198
[4005] https://github.com/TheHPXProject/hpx/pull/3197
[4006] https://github.com/TheHPXProject/hpx/pull/3194
[4007] https://github.com/TheHPXProject/hpx/pull/3193
[4008] https://github.com/TheHPXProject/hpx/pull/3191

- PR #3190[4009] - Topology improvements
- PR #3189[4010] - Deleting one include of median from BOOST library to arithmetics_counter file
- PR #3188[4011] - Optionally disable printing of diagnostics during terminate
- PR #3187[4012] - Suppressing cmake warning issued by cmake > V3.11
- PR #3185[4013] - Remove unused scoped_unlock, unlock_guard_try
- PR #3184[4014] - Fix nqueen example
- PR #3183[4015] - Add runtime start/stop, resume/suspend and OpenMP benchmarks
- Issue #3182[4016] - bulk_then_execute has unexpected return type/does not compile
- Issue #3181[4017] - hwloc 2.0 breaks topo class and cannot be used
- Issue #3180[4018] - Schedulers that don't support suspend/resume are unusable
- PR #3179[4019] - Various minor changes to support FLeCSI
- PR #3178[4020] - Fix #3124
- PR #3177[4021] - Removed allgather
- PR #3176[4022] - Fixed Documentation for "using_hpx_pkgconfig"
- PR #3174[4023] - Add hpx::iostream::ostream overload to format_to
- PR #3172[4024] - Fix lifo queue backend
- PR #3171[4025] - adding the missing unset() function to cpu_mask() for case of more than 64 threads
- PR #3170[4026] - Add cmake flag -DHPX_WITH_FAULT_TOLERANCE=ON (OFF by default)
- PR #3169[4027] - Adapted parallel::{count|count_if} for Ranges TS (see #1668)
- PR #3168[4028] - Changing used namespace for seq execution policy
- Issue #3167[4029] - Update GSoC projects

4009 https://github.com/TheHPXProject/hpx/pull/3190
4010 https://github.com/TheHPXProject/hpx/pull/3189
4011 https://github.com/TheHPXProject/hpx/pull/3188
4012 https://github.com/TheHPXProject/hpx/pull/3187
4013 https://github.com/TheHPXProject/hpx/pull/3185
4014 https://github.com/TheHPXProject/hpx/pull/3184
4015 https://github.com/TheHPXProject/hpx/pull/3183
4016 https://github.com/TheHPXProject/hpx/issues/3182
4017 https://github.com/TheHPXProject/hpx/issues/3181
4018 https://github.com/TheHPXProject/hpx/issues/3180
4019 https://github.com/TheHPXProject/hpx/pull/3179
4020 https://github.com/TheHPXProject/hpx/pull/3178
4021 https://github.com/TheHPXProject/hpx/pull/3177
4022 https://github.com/TheHPXProject/hpx/pull/3176
4023 https://github.com/TheHPXProject/hpx/pull/3174
4024 https://github.com/TheHPXProject/hpx/pull/3172
4025 https://github.com/TheHPXProject/hpx/pull/3171
4026 https://github.com/TheHPXProject/hpx/pull/3170
4027 https://github.com/TheHPXProject/hpx/pull/3169
4028 https://github.com/TheHPXProject/hpx/pull/3168
4029 https://github.com/TheHPXProject/hpx/issues/3167

- Issue #3166[4030] - Application (Octotiger) gets stuck on hpx::finalize when only using one thread
- Issue #3165[4031] - Compilation of parallel algorithms with HPX_WITH_DATAPAR is broken
- PR #3164[4032] - Fixing component migration
- PR #3162[4033] - regex_from_pattern: escape regex special characters to avoid misinterpretation
- Issue #3161[4034] - Building HPX with hwloc 2.0.0 fails
- PR #3160[4035] - Fixing the handling of quoted command line arguments.
- PR #3158[4036] - Fixing a race with timed suspension (second attempt)
- PR #3157[4037] - Revert "Fixing a race with timed suspension"
- PR #3156[4038] - Fixing serialization of classes with incompatible serialize signature
- PR #3154[4039] - More refactorings based on clang-tidy reports
- PR #3153[4040] - Fixing a race with timed suspension
- PR #3152[4041] - Documentation for runtime suspension
- PR #3151[4042] - Use small_vector only from boost version 1.59 onwards
- PR #3150[4043] - Avoiding more stack overflows
- PR #3148[4044] - Refactoring component_base and base_action/transfer_base_action
- PR #3147[4045] - Move yield_while out of detail namespace and into own file
- PR #3145[4046] - Remove a leftover of the cxx11 std array cleanup
- PR #3144[4047] - Minor changes to how actions are executed
- PR #3143[4048] - Fix stack overhead
- PR #3142[4049] - Fix typo in config.hpp
- PR #3141[4050] - Fixing small_vector compatibility with older boost version
- PR #3140[4051] - is_heap_text fix

[4030] https://github.com/TheHPXProject/hpx/issues/3166
[4031] https://github.com/TheHPXProject/hpx/issues/3165
[4032] https://github.com/TheHPXProject/hpx/pull/3164
[4033] https://github.com/TheHPXProject/hpx/pull/3162
[4034] https://github.com/TheHPXProject/hpx/issues/3161
[4035] https://github.com/TheHPXProject/hpx/pull/3160
[4036] https://github.com/TheHPXProject/hpx/pull/3158
[4037] https://github.com/TheHPXProject/hpx/pull/3157
[4038] https://github.com/TheHPXProject/hpx/pull/3156
[4039] https://github.com/TheHPXProject/hpx/pull/3154
[4040] https://github.com/TheHPXProject/hpx/pull/3153
[4041] https://github.com/TheHPXProject/hpx/pull/3152
[4042] https://github.com/TheHPXProject/hpx/pull/3151
[4043] https://github.com/TheHPXProject/hpx/pull/3150
[4044] https://github.com/TheHPXProject/hpx/pull/3148
[4045] https://github.com/TheHPXProject/hpx/pull/3147
[4046] https://github.com/TheHPXProject/hpx/pull/3145
[4047] https://github.com/TheHPXProject/hpx/pull/3144
[4048] https://github.com/TheHPXProject/hpx/pull/3143
[4049] https://github.com/TheHPXProject/hpx/pull/3142
[4050] https://github.com/TheHPXProject/hpx/pull/3141
[4051] https://github.com/TheHPXProject/hpx/pull/3140

- Issue #3139[4052] - Error in is_heap_tests.hpp
- PR #3138[4053] - Partially reverting #3126
- PR #3137[4054] - Suspend speedup
- PR #3136[4055] - Revert "Fixing #2325"
- PR #3135[4056] - Improving destruction of threads
- Issue #3134[4057] - HPX_SERIALIZATION_SPLIT_FREE does not stop compiler from looking for serialize() method
- PR #3133[4058] - Make hwloc compulsory
- PR #3132[4059] - Update CXX14 constexpr feature test
- PR #3131[4060] - Fixing #2325
- PR #3130[4061] - Avoid completion handler allocation
- PR #3129[4062] - Suspend runtime
- PR #3128[4063] - Make docbook dtd and xsl path names consistent
- PR #3127[4064] - Add hpx::start nullptr overloads
- PR #3126[4065] - Cleaning up coroutine implementation
- PR #3125[4066] - Replacing nullptr with hpx::threads::invalid_thread_id
- Issue #3124[4067] - Add hello_world_component to CI builds
- PR #3123[4068] - Add new constructor.
- PR #3122[4069] - Fixing #3121
- Issue #3121[4070] - HPX_SMT_PAUSE is broken on non-x86 platforms when __GNUC__ is defined
- PR #3120[4071] - Don't use boost::intrusive_ptr for thread_id_type
- PR #3119[4072] - Disable default executor compatibility with V1 executors
- PR #3118[4073] - Adding performance_counter::reinit to allow for dynamically changing counter sets

4052 https://github.com/TheHPXProject/hpx/issues/3139
4053 https://github.com/TheHPXProject/hpx/pull/3138
4054 https://github.com/TheHPXProject/hpx/pull/3137
4055 https://github.com/TheHPXProject/hpx/pull/3136
4056 https://github.com/TheHPXProject/hpx/pull/3135
4057 https://github.com/TheHPXProject/hpx/issues/3134
4058 https://github.com/TheHPXProject/hpx/pull/3133
4059 https://github.com/TheHPXProject/hpx/pull/3132
4060 https://github.com/TheHPXProject/hpx/pull/3131
4061 https://github.com/TheHPXProject/hpx/pull/3130
4062 https://github.com/TheHPXProject/hpx/pull/3129
4063 https://github.com/TheHPXProject/hpx/pull/3128
4064 https://github.com/TheHPXProject/hpx/pull/3127
4065 https://github.com/TheHPXProject/hpx/pull/3126
4066 https://github.com/TheHPXProject/hpx/pull/3125
4067 https://github.com/TheHPXProject/hpx/issues/3124
4068 https://github.com/TheHPXProject/hpx/pull/3123
4069 https://github.com/TheHPXProject/hpx/pull/3122
4070 https://github.com/TheHPXProject/hpx/issues/3121
4071 https://github.com/TheHPXProject/hpx/pull/3120
4072 https://github.com/TheHPXProject/hpx/pull/3119
4073 https://github.com/TheHPXProject/hpx/pull/3118

- PR #3117[4074] - Replace uses of boost/experimental::optional with util::optional
- PR #3116[4075] - Moving background thread APEX timer #2980
- PR #3115[4076] - Fixing race condition in channel test
- PR #3114[4077] - Avoid using util::function for thread function wrappers
- PR #3113[4078] - cmake V3.10.2 has changed the variable names used for MPI
- PR #3112[4079] - Minor fixes to exclusive_scan algorithm
- PR #3111[4080] - Revert "fix detection of cxx11_std_atomic"
- PR #3110[4081] - Suspend thread pool
- PR #3109[4082] - Fixing thread scheduling when yielding a thread id
- PR #3108[4083] - Revert "Suspend thread pool"
- PR #3107[4084] - Remove UB from thread::id relational operators
- PR #3106[4085] - Add cmake test for std::decay_t to fix cuda build
- PR #3105[4086] - Fixing refcount for async traversal frame
- PR #3104[4087] - Local execution of direct actions is now actually performed directly
- PR #3103[4088] - Adding support for generic counter_raw_values performance counter type
- Issue #3102[4089] - Introduce generic performance counter type returning an array of values
- PR #3101[4090] - Revert "Adapting stack overhead limit for gcc 4.9"
- PR #3100[4091] - Fix #3068 (condition_variable deadlock)
- PR #3099[4092] - Fixing lock held during suspension in papi counter component
- PR #3098[4093] - Unbreak broadcast_wait_for_2822 test
- PR #3097[4094] - Adapting stack overhead limit for gcc 4.9
- PR #3096[4095] - fix detection of cxx11_std_atomic

[4074] https://github.com/TheHPXProject/hpx/pull/3117
[4075] https://github.com/TheHPXProject/hpx/pull/3116
[4076] https://github.com/TheHPXProject/hpx/pull/3115
[4077] https://github.com/TheHPXProject/hpx/pull/3114
[4078] https://github.com/TheHPXProject/hpx/pull/3113
[4079] https://github.com/TheHPXProject/hpx/pull/3112
[4080] https://github.com/TheHPXProject/hpx/pull/3111
[4081] https://github.com/TheHPXProject/hpx/pull/3110
[4082] https://github.com/TheHPXProject/hpx/pull/3109
[4083] https://github.com/TheHPXProject/hpx/pull/3108
[4084] https://github.com/TheHPXProject/hpx/pull/3107
[4085] https://github.com/TheHPXProject/hpx/pull/3106
[4086] https://github.com/TheHPXProject/hpx/pull/3105
[4087] https://github.com/TheHPXProject/hpx/pull/3104
[4088] https://github.com/TheHPXProject/hpx/pull/3103
[4089] https://github.com/TheHPXProject/hpx/issues/3102
[4090] https://github.com/TheHPXProject/hpx/pull/3101
[4091] https://github.com/TheHPXProject/hpx/pull/3100
[4092] https://github.com/TheHPXProject/hpx/pull/3099
[4093] https://github.com/TheHPXProject/hpx/pull/3098
[4094] https://github.com/TheHPXProject/hpx/pull/3097
[4095] https://github.com/TheHPXProject/hpx/pull/3096

- PR #3095[4096] - Add ciso646 header to get _LIBCPP_VERSION for testing inplace merge
- PR #3094[4097] - Relax atomic operations on performance counter values
- PR #3093[4098] - Short-circuit all_of/any_of/none_of instantiations
- PR #3092[4099] - Take advantage of C++14 lambda capture initialization syntax, where possible
- PR #3091[4100] - Remove more references to Boost from logging code
- PR #3090[4101] - Unify use of yield/yield_k
- PR #3089[4102] - Fix a strange thing in parallel::detail::handle_exception. (Fix #2834.)
- Issue #3088[4103] - A strange thing in parallel::sort.
- PR #3087[4104] - Fixing assertion in default_distribution_policy
- PR #3086[4105] - Implement parallel::remove and parallel::remove_if
- PR #3085[4106] - Addressing breaking changes in Boost V1.66
- PR #3084[4107] - Ignore build warnings round 2
- PR #3083[4108] - Fix typo HPX_WITH_MM_PREFECTH
- PR #3081[4109] - Pre-decay template arguments early
- PR #3080[4110] - Suspend thread pool
- PR #3079[4111] - Ignore build warnings
- PR #3078[4112] - Don't test inplace_merge with libc++
- PR #3076[4113] - Fixing 3075: Part 1
- PR #3074[4114] - Fix more build warnings
- PR #3073[4115] - Suspend thread cleanup
- PR #3072[4116] - Change existing symbol_namespace::iterate to return all data instead of invoking a callback
- PR #3071[4117] - Fixing pack_traversal_async test

[4096] https://github.com/TheHPXProject/hpx/pull/3095
[4097] https://github.com/TheHPXProject/hpx/pull/3094
[4098] https://github.com/TheHPXProject/hpx/pull/3093
[4099] https://github.com/TheHPXProject/hpx/pull/3092
[4100] https://github.com/TheHPXProject/hpx/pull/3091
[4101] https://github.com/TheHPXProject/hpx/pull/3090
[4102] https://github.com/TheHPXProject/hpx/pull/3089
[4103] https://github.com/TheHPXProject/hpx/issues/3088
[4104] https://github.com/TheHPXProject/hpx/pull/3087
[4105] https://github.com/TheHPXProject/hpx/pull/3086
[4106] https://github.com/TheHPXProject/hpx/pull/3085
[4107] https://github.com/TheHPXProject/hpx/pull/3084
[4108] https://github.com/TheHPXProject/hpx/pull/3083
[4109] https://github.com/TheHPXProject/hpx/pull/3081
[4110] https://github.com/TheHPXProject/hpx/pull/3080
[4111] https://github.com/TheHPXProject/hpx/pull/3079
[4112] https://github.com/TheHPXProject/hpx/pull/3078
[4113] https://github.com/TheHPXProject/hpx/pull/3076
[4114] https://github.com/TheHPXProject/hpx/pull/3074
[4115] https://github.com/TheHPXProject/hpx/pull/3073
[4116] https://github.com/TheHPXProject/hpx/pull/3072
[4117] https://github.com/TheHPXProject/hpx/pull/3071

- PR #3070[4118] - Fix dynamic_counters_loaded_1508 test by adding dependency to memory_component
- PR #3069[4119] - Fix scheduling loop exit
- Issue #3068[4120] - hpx::lcos::condition_variable could be suspect to deadlocks
- PR #3067[4121] - #ifdef out random_shuffle deprecated in later c++
- PR #3066[4122] - Make coalescing test depend on coalescing library to ensure it gets built
- PR #3065[4123] - Workaround for minimal_timed_async_executor_test compilation failures, attempts to copy a deferred call (in unevaluated context)
- PR #3064[4124] - Fixing wrong condition in wrapper_heap
- PR #3062[4125] - Fix exception handling for execution::seq
- PR #3061[4126] - Adapt MSVC C++ mode handling to VS15.5
- PR #3060[4127] - Fix compiler problem in MSVC release mode
- PR #3059[4128] - Fixing #2931
- Issue #3058[4129] - minimal_timed_async_executor_test_exe fails to compile on master (d6f505c)
- PR #3057[4130] - Fix stable_merge_2964 compilation problems
- PR #3056[4131] - Fix some build warnings caused by unused variables/unnecessary tests
- PR #3055[4132] - Update documentation for running tests
- Issue #3054[4133] - Assertion failure when using bulk hpx::**new_** in asynchronous mode
- PR #3052[4134] - Do not bind test running to cmake test build rule
- PR #3051[4135] - Fix HPX-Qt interaction in Qt example.
- Issue #3048[4136] - nqueen example fails occasionally
- PR #3047[4137] - Fixing #3044
- PR #3046[4138] - Add OS thread suspension
- PR #3042[4139] - PyCicle - first attempt at a build toold for checking PR's

[4118] https://github.com/TheHPXProject/hpx/pull/3070
[4119] https://github.com/TheHPXProject/hpx/pull/3069
[4120] https://github.com/TheHPXProject/hpx/issues/3068
[4121] https://github.com/TheHPXProject/hpx/pull/3067
[4122] https://github.com/TheHPXProject/hpx/pull/3066
[4123] https://github.com/TheHPXProject/hpx/pull/3065
[4124] https://github.com/TheHPXProject/hpx/pull/3064
[4125] https://github.com/TheHPXProject/hpx/pull/3062
[4126] https://github.com/TheHPXProject/hpx/pull/3061
[4127] https://github.com/TheHPXProject/hpx/pull/3060
[4128] https://github.com/TheHPXProject/hpx/pull/3059
[4129] https://github.com/TheHPXProject/hpx/issues/3058
[4130] https://github.com/TheHPXProject/hpx/pull/3057
[4131] https://github.com/TheHPXProject/hpx/pull/3056
[4132] https://github.com/TheHPXProject/hpx/pull/3055
[4133] https://github.com/TheHPXProject/hpx/issues/3054
[4134] https://github.com/TheHPXProject/hpx/pull/3052
[4135] https://github.com/TheHPXProject/hpx/pull/3051
[4136] https://github.com/TheHPXProject/hpx/issues/3048
[4137] https://github.com/TheHPXProject/hpx/pull/3047
[4138] https://github.com/TheHPXProject/hpx/pull/3046
[4139] https://github.com/TheHPXProject/hpx/pull/3042

- PR #3041[4140] - Fix a problem about asynchronous execution of parallel::merge and parallel::partition.
- PR #3040[4141] - Fix a mistake about exception handling in asynchronous execution of scan_partitioner.
- PR #3039[4142] - Consistently use executors to schedule work
- PR #3038[4143] - Fixing local direct function execution and lambda actions perfect forwarding
- PR #3035[4144] - Make parallel unit test names match build target/folder names
- PR #3033[4145] - Fix setting of default build type
- Issue #3032[4146] - Fix partitioner arg copy found in #2982
- Issue #3031[4147] - Errors linking libhpx.so due to missing references (master branch, commit 6679a8882)
- PR #3030[4148] - Revert "implement executor then interface with && forwarding reference"
- PR #3029[4149] - Run CI inspect checks before building
- PR #3028[4150] - Added range version of parallel::move
- Issue #3027[4151] - Implement all scheduling APIs in terms of executors
- PR #3026[4152] - implement executor then interface with && forwarding reference
- PR #3025[4153] - Fix typo uninitialized to uninitialized
- PR #3024[4154] - Inspect fixes
- PR #3023[4155] - P0356 Simplified partial function application
- PR #3022[4156] - Master fixes
- PR #3021[4157] - Segfault fix
- PR #3020[4158] - Disable command-line aliasing for applications that use user_main
- PR #3019[4159] - Adding enable_elasticity option to pool configuration
- PR #3018[4160] - Fix stack overflow detection configuration in header files

[4140] https://github.com/TheHPXProject/hpx/pull/3041
[4141] https://github.com/TheHPXProject/hpx/pull/3040
[4142] https://github.com/TheHPXProject/hpx/pull/3039
[4143] https://github.com/TheHPXProject/hpx/pull/3038
[4144] https://github.com/TheHPXProject/hpx/pull/3035
[4145] https://github.com/TheHPXProject/hpx/pull/3033
[4146] https://github.com/TheHPXProject/hpx/issues/3032
[4147] https://github.com/TheHPXProject/hpx/issues/3031
[4148] https://github.com/TheHPXProject/hpx/pull/3030
[4149] https://github.com/TheHPXProject/hpx/pull/3029
[4150] https://github.com/TheHPXProject/hpx/pull/3028
[4151] https://github.com/TheHPXProject/hpx/issues/3027
[4152] https://github.com/TheHPXProject/hpx/pull/3026
[4153] https://github.com/TheHPXProject/hpx/pull/3025
[4154] https://github.com/TheHPXProject/hpx/pull/3024
[4155] https://github.com/TheHPXProject/hpx/pull/3023
[4156] https://github.com/TheHPXProject/hpx/pull/3022
[4157] https://github.com/TheHPXProject/hpx/pull/3021
[4158] https://github.com/TheHPXProject/hpx/pull/3020
[4159] https://github.com/TheHPXProject/hpx/pull/3019
[4160] https://github.com/TheHPXProject/hpx/pull/3018

- PR #3017[4161] - Speed up local action execution
- PR #3016[4162] - Unify stack-overflow detection options, remove reference to libsigsegv
- PR #3015[4163] - Speeding up accessing the resource partitioner and the topology info
- Issue #3014[4164] - HPX does not compile on POWER8 with gcc 5.4
- Issue #3013[4165] - hello_world occasionally prints multiple lines from a single OS-thread
- PR #3012[4166] - Silence warning about casting away qualifiers in itt_notify.hpp
- PR #3011[4167] - Fix cpuset leak in hwloc_topology_info.cpp
- PR #3010[4168] - Remove useless decay_copy
- PR #3009[4169] - Fixing 2996
- PR #3008[4170] - Remove unused internal function
- PR #3007[4171] - Fixing wrapper_heap alignment problems
- Issue #3006[4172] - hwloc memory leak
- PR #3004[4173] - Silence C4251 (needs to have dll-interface) for future_data_void
- Issue #3003[4174] - Suspension of runtime
- PR #3001[4175] - Attempting to avoid data races in async_traversal while evaluating dataflow()
- PR #3000[4176] - Adding hpx::util::optional as a first step to replace experimental::optional
- PR #2998[4177] - Cleanup up and Fixing component creation and deletion
- Issue #2996[4178] - Build fails with HPX_WITH_HWLOC=OFF
- PR #2995[4179] - Push more future_data functionality to source file
- PR #2994[4180] - WIP: Fix throttle test
- PR #2993[4181] - Making sure –hpx:help does not throw for required (but missing) arguments
- PR #2992[4182] - Adding non-blocking (on destruction) service executors

4161 https://github.com/TheHPXProject/hpx/pull/3017
4162 https://github.com/TheHPXProject/hpx/pull/3016
4163 https://github.com/TheHPXProject/hpx/pull/3015
4164 https://github.com/TheHPXProject/hpx/issues/3014
4165 https://github.com/TheHPXProject/hpx/issues/3013
4166 https://github.com/TheHPXProject/hpx/pull/3012
4167 https://github.com/TheHPXProject/hpx/pull/3011
4168 https://github.com/TheHPXProject/hpx/pull/3010
4169 https://github.com/TheHPXProject/hpx/pull/3009
4170 https://github.com/TheHPXProject/hpx/pull/3008
4171 https://github.com/TheHPXProject/hpx/pull/3007
4172 https://github.com/TheHPXProject/hpx/issues/3006
4173 https://github.com/TheHPXProject/hpx/pull/3004
4174 https://github.com/TheHPXProject/hpx/issues/3003
4175 https://github.com/TheHPXProject/hpx/pull/3001
4176 https://github.com/TheHPXProject/hpx/pull/3000
4177 https://github.com/TheHPXProject/hpx/pull/2998
4178 https://github.com/TheHPXProject/hpx/issues/2996
4179 https://github.com/TheHPXProject/hpx/pull/2995
4180 https://github.com/TheHPXProject/hpx/pull/2994
4181 https://github.com/TheHPXProject/hpx/pull/2993
4182 https://github.com/TheHPXProject/hpx/pull/2992

- Issue #2991[4183] - run_as_os_thread locks up
- Issue #2990[4184] - –help will not work until all required options are provided
- PR #2989[4185] - Improve error messages caused by misuse of dataflow
- PR #2988[4186] - Improve error messages caused by misuse of .then
- Issue #2987[4187] - stack overflow detection producing false positives
- PR #2986[4188] - Deduplicate non-dependent thread_info logging types
- PR #2985[4189] - Adapted parallel::{all_of|any_of|none_of} for Ranges TS (see #1668)
- PR #2984[4190] - Refactor one_size_heap code to simplify code
- PR #2983[4191] - Fixing local_new_component
- PR #2982[4192] - Clang tidy
- PR #2981[4193] - Simplify allocator rebinding in pack traversal
- PR #2979[4194] - Fixing integer overflows
- PR #2978[4195] - Implement parallel::inplace_merge
- Issue #2977[4196] - Make hwloc compulsory instead of optional
- PR #2976[4197] - Making sure client_base instance that registered the component does not unregister it when being destructed
- PR #2975[4198] - Change version of pulled APEX to master
- PR #2974[4199] - Fix domain not being freed at the end of scheduling loop
- PR #2973[4200] - Fix small typos
- PR #2972[4201] - Adding uintstd.h header
- PR #2971[4202] - Fall back to creating local components using local_new
- PR #2970[4203] - Improve is_tuple_like trait
- PR #2969[4204] - Fix HPX_WITH_MORE_THAN_64_THREADS default value
- PR #2968[4205] - Cleaning up dataflow overload set

4183 https://github.com/TheHPXProject/hpx/issues/2991
4184 https://github.com/TheHPXProject/hpx/issues/2990
4185 https://github.com/TheHPXProject/hpx/pull/2989
4186 https://github.com/TheHPXProject/hpx/pull/2988
4187 https://github.com/TheHPXProject/hpx/issues/2987
4188 https://github.com/TheHPXProject/hpx/pull/2986
4189 https://github.com/TheHPXProject/hpx/pull/2985
4190 https://github.com/TheHPXProject/hpx/pull/2984
4191 https://github.com/TheHPXProject/hpx/pull/2983
4192 https://github.com/TheHPXProject/hpx/pull/2982
4193 https://github.com/TheHPXProject/hpx/pull/2981
4194 https://github.com/TheHPXProject/hpx/pull/2979
4195 https://github.com/TheHPXProject/hpx/pull/2978
4196 https://github.com/TheHPXProject/hpx/issues/2977
4197 https://github.com/TheHPXProject/hpx/pull/2976
4198 https://github.com/TheHPXProject/hpx/pull/2975
4199 https://github.com/TheHPXProject/hpx/pull/2974
4200 https://github.com/TheHPXProject/hpx/pull/2973
4201 https://github.com/TheHPXProject/hpx/pull/2972
4202 https://github.com/TheHPXProject/hpx/pull/2971
4203 https://github.com/TheHPXProject/hpx/pull/2970
4204 https://github.com/TheHPXProject/hpx/pull/2969
4205 https://github.com/TheHPXProject/hpx/pull/2968

- PR #2967[4206] - Make parallel::merge is stable. (Fix #2964.)
- PR #2966[4207] - Fixing a couple of held locks during exception handling
- PR #2965[4208] - Adding missing #include
- Issue #2964[4209] - parallel merge is not stable
- PR #2963[4210] - Making sure any function object passed to dataflow is released after being invoked
- PR #2962[4211] - Partially reverting #2891
- PR #2961[4212] - Attempt to fix the gcc 4.9 problem with the async pack traversal
- Issue #2959[4213] - Program terminates during error handling
- Issue #2958[4214] - HPX_PLAIN_ACTION breaks due to missing include
- PR #2957[4215] - Fixing errors generated by mixing different attribute syntaxes
- Issue #2956[4216] - Mixing attribute syntaxes leads to compiler errors
- Issue #2955[4217] - Fix OS-Thread throttling
- PR #2953[4218] - Making sure any hpx.os_threads=N supplied through a –hpx::config file is taken into account
- PR #2952[4219] - Removing wrong call to cleanup_terminated_locked
- PR #2951[4220] - Revert "Make sure the function vtables are initialized before use"
- PR #2950[4221] - Fix a namespace compilation error when some schedulers are disabled
- Issue #2949[4222] - master branch giving lockups on shutdown
- Issue #2947[4223] - hpx.ini is not used correctly at initialization
- PR #2946[4224] - Adding explicit feature test for thread_local
- PR #2945[4225] - Make sure the function vtables are initialized before use
- PR #2944[4226] - Attempting to solve affinity problems on CircleCI
- PR #2943[4227] - Changing channel actions to be direct

[4206] https://github.com/TheHPXProject/hpx/pull/2967
[4207] https://github.com/TheHPXProject/hpx/pull/2966
[4208] https://github.com/TheHPXProject/hpx/pull/2965
[4209] https://github.com/TheHPXProject/hpx/issues/2964
[4210] https://github.com/TheHPXProject/hpx/pull/2963
[4211] https://github.com/TheHPXProject/hpx/pull/2962
[4212] https://github.com/TheHPXProject/hpx/pull/2961
[4213] https://github.com/TheHPXProject/hpx/issues/2959
[4214] https://github.com/TheHPXProject/hpx/issues/2958
[4215] https://github.com/TheHPXProject/hpx/pull/2957
[4216] https://github.com/TheHPXProject/hpx/issues/2956
[4217] https://github.com/TheHPXProject/hpx/issues/2955
[4218] https://github.com/TheHPXProject/hpx/pull/2953
[4219] https://github.com/TheHPXProject/hpx/pull/2952
[4220] https://github.com/TheHPXProject/hpx/pull/2951
[4221] https://github.com/TheHPXProject/hpx/pull/2950
[4222] https://github.com/TheHPXProject/hpx/issues/2949
[4223] https://github.com/TheHPXProject/hpx/issues/2947
[4224] https://github.com/TheHPXProject/hpx/pull/2946
[4225] https://github.com/TheHPXProject/hpx/pull/2945
[4226] https://github.com/TheHPXProject/hpx/pull/2944
[4227] https://github.com/TheHPXProject/hpx/pull/2943

- PR #2942[4228] - Adding split_future for std::vector
- PR #2941[4229] - Add a feature test to test for CXX11 override
- Issue #2940[4230] - Add split_future for future<vector<T>>
- PR #2939[4231] - Making error reporting during problems with setting affinity masks more verbose
- PR #2938[4232] - Fix this various executors
- PR #2937[4233] - Fix some typos in documentation
- PR #2934[4234] - Remove the need for "complete" SFINAE checks
- PR #2933[4235] - Making sure parallel::for_loop is executed in parallel if requested
- PR #2932[4236] - Classify chunk_size_iterator to input iterator tag. (Fix #2866)
- Issue #2931[4237] - –hpx:help triggers unusual error with clang build
- PR #2930[4238] - Add #include files needed to set _POSIX_VERSION for debug check
- PR #2929[4239] - Fix a couple of deprecated c++ features
- PR #2928[4240] - Fixing execution parameters
- Issue #2927[4241] - CMake warning: … cycle in constraint graph
- PR #2926[4242] - Default pool rename
- Issue #2925[4243] - Default pool cannot be renamed
- Issue #2924[4244] - hpx:attach-debugger=startup does not work any more
- PR #2923[4245] - Alloc membind
- PR #2922[4246] - This fixes CircleCI errors when running with –hpx:bind=none
- PR #2921[4247] - Custom pool executor was missing priority and stacksize options
- PR #2920[4248] - Adding test to trigger problem reported in #2916
- PR #2919[4249] - Make sure the resource_partitioner is properly destructed on hpx::finalize
- Issue #2918[4250] - hpx::init calls wrong (first) callback when called multiple times

[4228] https://github.com/TheHPXProject/hpx/pull/2942
[4229] https://github.com/TheHPXProject/hpx/pull/2941
[4230] https://github.com/TheHPXProject/hpx/issues/2940
[4231] https://github.com/TheHPXProject/hpx/pull/2939
[4232] https://github.com/TheHPXProject/hpx/pull/2938
[4233] https://github.com/TheHPXProject/hpx/pull/2937
[4234] https://github.com/TheHPXProject/hpx/pull/2934
[4235] https://github.com/TheHPXProject/hpx/pull/2933
[4236] https://github.com/TheHPXProject/hpx/pull/2932
[4237] https://github.com/TheHPXProject/hpx/issues/2931
[4238] https://github.com/TheHPXProject/hpx/pull/2930
[4239] https://github.com/TheHPXProject/hpx/pull/2929
[4240] https://github.com/TheHPXProject/hpx/pull/2928
[4241] https://github.com/TheHPXProject/hpx/issues/2927
[4242] https://github.com/TheHPXProject/hpx/pull/2926
[4243] https://github.com/TheHPXProject/hpx/issues/2925
[4244] https://github.com/TheHPXProject/hpx/issues/2924
[4245] https://github.com/TheHPXProject/hpx/pull/2923
[4246] https://github.com/TheHPXProject/hpx/pull/2922
[4247] https://github.com/TheHPXProject/hpx/pull/2921
[4248] https://github.com/TheHPXProject/hpx/pull/2920
[4249] https://github.com/TheHPXProject/hpx/pull/2919
[4250] https://github.com/TheHPXProject/hpx/issues/2918

- PR #2917[4251] - Adding util::checkpoint
- Issue #2916[4252] - Weird runtime failures when using a channel and chained continuations
- PR #2915[4253] - Introduce executor parameters customization points
- Issue #2914[4254] - Task assignment to current Pool has unintended consequences
- PR #2913[4255] - Fix rp hang
- PR #2912[4256] - Update contributors
- PR #2911[4257] - Fixing CUDA problems
- PR #2910[4258] - Improve error reporting for process component on POSIX systems
- PR #2909[4259] - Fix typo in include path
- PR #2908[4260] - Use proper container according to iterator tag in benchmarks of parallel algorithms
- PR #2907[4261] - Optionally force-delete remaining channel items on close
- PR #2906[4262] - Making sure generated performance counter names are correct
- Issue #2905[4263] - collecting idle-rate performance counters on multiple localities produces an error
- Issue #2904[4264] - build broken for Intel 17 compilers
- PR #2903[4265] - Documentation Updates– Adding New People
- PR #2902[4266] - Fixing service_executor
- PR #2901[4267] - Fixing partitioned_vector creation
- PR #2900[4268] - Add numa-balanced mode to hpx::bind, spread cores over numa domains
- Issue #2899[4269] - hpx::bind does not have a mode that balances cores over numa domains
- PR #2898[4270] - Adding missing #include and missing guard for optional code section
- PR #2897[4271] - Removing dependency on Boost.ICL
- Issue #2896[4272] - Debug build fails without -fpermissive with GCC 7.1 and Boost 1.65

[4251] https://github.com/TheHPXProject/hpx/pull/2917
[4252] https://github.com/TheHPXProject/hpx/issues/2916
[4253] https://github.com/TheHPXProject/hpx/pull/2915
[4254] https://github.com/TheHPXProject/hpx/issues/2914
[4255] https://github.com/TheHPXProject/hpx/pull/2913
[4256] https://github.com/TheHPXProject/hpx/pull/2912
[4257] https://github.com/TheHPXProject/hpx/pull/2911
[4258] https://github.com/TheHPXProject/hpx/pull/2910
[4259] https://github.com/TheHPXProject/hpx/pull/2909
[4260] https://github.com/TheHPXProject/hpx/pull/2908
[4261] https://github.com/TheHPXProject/hpx/pull/2907
[4262] https://github.com/TheHPXProject/hpx/pull/2906
[4263] https://github.com/TheHPXProject/hpx/issues/2905
[4264] https://github.com/TheHPXProject/hpx/issues/2904
[4265] https://github.com/TheHPXProject/hpx/pull/2903
[4266] https://github.com/TheHPXProject/hpx/pull/2902
[4267] https://github.com/TheHPXProject/hpx/pull/2901
[4268] https://github.com/TheHPXProject/hpx/pull/2900
[4269] https://github.com/TheHPXProject/hpx/issues/2899
[4270] https://github.com/TheHPXProject/hpx/pull/2898
[4271] https://github.com/TheHPXProject/hpx/pull/2897
[4272] https://github.com/TheHPXProject/hpx/issues/2896

- PR #2895[4273] - Fixing SLURM environment parsing
- PR #2894[4274] - Fix incorrect handling of compile definition with value 0
- Issue #2893[4275] - Disabling schedulers causes build errors
- PR #2892[4276] - added list serializer
- PR #2891[4277] - Resource Partitioner Fixes
- Issue #2890[4278] - Destroying a non-empty channel causes an assertion failure
- PR #2889[4279] - Add check for libatomic
- PR #2888[4280] - Fix compilation problems if HPX_WITH_ITT_NOTIFY=ON
- PR #2887[4281] - Adapt broadcast() to non-unwrapping async<Action>
- PR #2886[4282] - Replace Boost.Random with C++11 <random>
- Issue #2885[4283] - regression in broadcast?
- Issue #2884[4284] - linking `-latomic` is not portable
- PR #2883[4285] - Explicitly set -pthread flag if available
- PR #2882[4286] - Wrap boost::format uses
- Issue #2881[4287] - hpx not compiling with `HPX_WITH_ITTNOTIFY=On`
- Issue #2880[4288] - hpx::bind scatter/balanced give wrong pu masks
- PR #2878[4289] - Fix incorrect pool usage masks setup in RP/thread manager
- PR #2877[4290] - Require `std::array` by default
- PR #2875[4291] - Deprecate use of BOOST_ASSERT
- PR #2874[4292] - Changed serialization of boost.variant to use variadic templates
- Issue #2873[4293] - building with parcelport_mpi fails on cori
- PR #2871[4294] - Adding missing support for throttling scheduler
- PR #2870[4295] - Disambiguate use of base_lco_with_value macros with channel

4273 https://github.com/TheHPXProject/hpx/pull/2895
4274 https://github.com/TheHPXProject/hpx/pull/2894
4275 https://github.com/TheHPXProject/hpx/issues/2893
4276 https://github.com/TheHPXProject/hpx/pull/2892
4277 https://github.com/TheHPXProject/hpx/pull/2891
4278 https://github.com/TheHPXProject/hpx/issues/2890
4279 https://github.com/TheHPXProject/hpx/pull/2889
4280 https://github.com/TheHPXProject/hpx/pull/2888
4281 https://github.com/TheHPXProject/hpx/pull/2887
4282 https://github.com/TheHPXProject/hpx/pull/2886
4283 https://github.com/TheHPXProject/hpx/issues/2885
4284 https://github.com/TheHPXProject/hpx/issues/2884
4285 https://github.com/TheHPXProject/hpx/pull/2883
4286 https://github.com/TheHPXProject/hpx/pull/2882
4287 https://github.com/TheHPXProject/hpx/issues/2881
4288 https://github.com/TheHPXProject/hpx/issues/2880
4289 https://github.com/TheHPXProject/hpx/pull/2878
4290 https://github.com/TheHPXProject/hpx/pull/2877
4291 https://github.com/TheHPXProject/hpx/pull/2875
4292 https://github.com/TheHPXProject/hpx/pull/2874
4293 https://github.com/TheHPXProject/hpx/issues/2873
4294 https://github.com/TheHPXProject/hpx/pull/2871
4295 https://github.com/TheHPXProject/hpx/pull/2870

- Issue #2869[4296] - Difficulty compiling `HPX_REGISTER_CHANNEL_DECLARATION(double)`
- PR #2868[4297] - Removing unneeded assert
- PR #2867[4298] - Implement parallel::unique
- Issue #2866[4299] - The chunk_size_iterator violates multipass guarantee
- PR #2865[4300] - Only use sched_getcpu on linux machines
- PR #2864[4301] - Create redistribution archive for successful builds
- PR #2863[4302] - Replace casts/assignments with hard-coded memcpy operations
- Issue #2862[4303] - sched_getcpu not available on MacOS
- PR #2861[4304] - Fixing unmatched header defines and recursive inclusion of threadmanager
- Issue #2860[4305] - Master program fails with assertion 'type == data_type_address' failed: HPX(assertion_failure)
- Issue #2852[4306] - Support for ARM64
- PR #2858[4307] - Fix misplaced #if #endif's that cause build failure without THREAD_CUMULATIVE_COUNTS
- PR #2857[4308] - Fix some listing in documentation
- PR #2856[4309] - Fixing component handling for lcos
- PR #2855[4310] - Add documentation for coarrays
- PR #2854[4311] - Support ARM64 in timestamps
- PR #2853[4312] - Update Table 17. Non-modifying Parallel Algorithms in Documentation
- PR #2851[4313] - Allowing for non-default-constructible component types
- PR #2850[4314] - Enable returning future<R> from actions where R is not default-constructible
- PR #2849[4315] - Unify serialization of non-default-constructable types
- Issue #2848[4316] - Components have to be default constructible

[4296] https://github.com/TheHPXProject/hpx/issues/2869
[4297] https://github.com/TheHPXProject/hpx/pull/2868
[4298] https://github.com/TheHPXProject/hpx/pull/2867
[4299] https://github.com/TheHPXProject/hpx/issues/2866
[4300] https://github.com/TheHPXProject/hpx/pull/2865
[4301] https://github.com/TheHPXProject/hpx/pull/2864
[4302] https://github.com/TheHPXProject/hpx/pull/2863
[4303] https://github.com/TheHPXProject/hpx/issues/2862
[4304] https://github.com/TheHPXProject/hpx/pull/2861
[4305] https://github.com/TheHPXProject/hpx/issues/2860
[4306] https://github.com/TheHPXProject/hpx/issues/2852
[4307] https://github.com/TheHPXProject/hpx/pull/2858
[4308] https://github.com/TheHPXProject/hpx/pull/2857
[4309] https://github.com/TheHPXProject/hpx/pull/2856
[4310] https://github.com/TheHPXProject/hpx/pull/2855
[4311] https://github.com/TheHPXProject/hpx/pull/2854
[4312] https://github.com/TheHPXProject/hpx/pull/2853
[4313] https://github.com/TheHPXProject/hpx/pull/2851
[4314] https://github.com/TheHPXProject/hpx/pull/2850
[4315] https://github.com/TheHPXProject/hpx/pull/2849
[4316] https://github.com/TheHPXProject/hpx/issues/2848

- Issue #2847[4317] - Returning a future<R> where R is not default-constructable broken
- Issue #2846[4318] - Unify serialization of non-default-constructible types
- PR #2845[4319] - Add Visual Studio 2015 to the tested toolchains in Appveyor
- Issue #2844[4320] - Change the appveyor build to use the minimal required MSVC version
- Issue #2843[4321] - multi node hello_world hangs
- PR #2842[4322] - Correcting Spelling mistake in docs
- PR #2841[4323] - Fix usage of std::aligned_storage
- PR #2840[4324] - Remove constexpr from a void function
- Issue #2839[4325] - memcpy buffer overflow: load_construct_data() and std::complex members
- Issue #2835[4326] - `constexpr` functions with `void` return type break compilation with CUDA 8.0
- Issue #2834[4327] - One suspicion in parallel::detail::handle_exception
- PR #2833[4328] - Implement parallel::merge
- PR #2832[4329] - Fix a strange thing in parallel::util::detail::handle_local_exceptions. (Fix #2818)
- PR #2830[4330] - Break the debugger when a test failed
- Issue #2831[4331] - `parallel/executors/execution_fwd.hpp` causes compilation failure in C++11 mode.
- PR #2829[4332] - Implement an API for asynchronous pack traversal
- PR #2828[4333] - Split unit test builds on CircleCI to avoid timeouts
- Issue #2827[4334] - failure to compile hello_world example with -Werror
- PR #2824[4335] - Making sure promises are marked as started when used as continuations
- PR #2823[4336] - Add documentation for partitioned_vector_view
- Issue #2822[4337] - Yet another issue with wait_for similar to #2796

[4317] https://github.com/TheHPXProject/hpx/issues/2847
[4318] https://github.com/TheHPXProject/hpx/issues/2846
[4319] https://github.com/TheHPXProject/hpx/pull/2845
[4320] https://github.com/TheHPXProject/hpx/issues/2844
[4321] https://github.com/TheHPXProject/hpx/issues/2843
[4322] https://github.com/TheHPXProject/hpx/pull/2842
[4323] https://github.com/TheHPXProject/hpx/pull/2841
[4324] https://github.com/TheHPXProject/hpx/pull/2840
[4325] https://github.com/TheHPXProject/hpx/issues/2839
[4326] https://github.com/TheHPXProject/hpx/issues/2835
[4327] https://github.com/TheHPXProject/hpx/issues/2834
[4328] https://github.com/TheHPXProject/hpx/pull/2833
[4329] https://github.com/TheHPXProject/hpx/pull/2832
[4330] https://github.com/TheHPXProject/hpx/pull/2830
[4331] https://github.com/TheHPXProject/hpx/issues/2831
[4332] https://github.com/TheHPXProject/hpx/pull/2829
[4333] https://github.com/TheHPXProject/hpx/pull/2828
[4334] https://github.com/TheHPXProject/hpx/issues/2827
[4335] https://github.com/TheHPXProject/hpx/pull/2824
[4336] https://github.com/TheHPXProject/hpx/pull/2823
[4337] https://github.com/TheHPXProject/hpx/issues/2822

- PR #2821[4338] - Fix bugs and improve that about HPX_HAVE_CXX11_AUTO_RETURN_VALUE of CMake
- PR #2820[4339] - Support C++11 in benchmark codes of parallel::partition and parallel::partition_copy
- PR #2819[4340] - Fix compile errors in unit test of container version of parallel::partition
- Issue #2818[4341] - A strange thing in parallel::util::detail::handle_local_exceptions
- Issue #2815[4342] - HPX fails to compile with HPX_WITH_CUDA=ON and the new CUDA 9.0 RC
- Issue #2814[4343] - Using 'gmakeN' after 'cmake' produces error in src/CMakeFiles/hpx.dir/runtime/agas/addressing_service.cpp.o
- PR #2813[4344] - Properly support [[noreturn]] attribute if available
- Issue #2812[4345] - Compilation fails with gcc 7.1.1
- PR #2811[4346] - Adding hpx::launch::lazy and support for async, dataflow, and future::then
- PR #2810[4347] - Add option allowing to disable deprecation warning
- PR #2809[4348] - Disable throttling scheduler if HWLOC is not found/used
- PR #2808[4349] - Fix compile errors on some environments of parallel::partition
- Issue #2807[4350] - Difficulty building with `HPX_WITH_HWLOC=Off`
- PR #2806[4351] - Partitioned vector
- PR #2805[4352] - Serializing collections with non-default constructible data
- PR #2802[4353] - Fix FreeBSD 11
- Issue #2801[4354] - Rate limiting techniques in io_service
- Issue #2800[4355] - New Launch Policy: async_if
- PR #2799[4356] - Fix a unit test failure on GCC in tuple_cat
- PR #2798[4357] - bump minimum required cmake to 3.0 in test
- PR #2797[4358] - Making sure future::wait_for et.al. work properly for action results

4338 https://github.com/TheHPXProject/hpx/pull/2821
4339 https://github.com/TheHPXProject/hpx/pull/2820
4340 https://github.com/TheHPXProject/hpx/pull/2819
4341 https://github.com/TheHPXProject/hpx/issues/2818
4342 https://github.com/TheHPXProject/hpx/issues/2815
4343 https://github.com/TheHPXProject/hpx/issues/2814
4344 https://github.com/TheHPXProject/hpx/pull/2813
4345 https://github.com/TheHPXProject/hpx/issues/2812
4346 https://github.com/TheHPXProject/hpx/pull/2811
4347 https://github.com/TheHPXProject/hpx/pull/2810
4348 https://github.com/TheHPXProject/hpx/pull/2809
4349 https://github.com/TheHPXProject/hpx/pull/2808
4350 https://github.com/TheHPXProject/hpx/issues/2807
4351 https://github.com/TheHPXProject/hpx/pull/2806
4352 https://github.com/TheHPXProject/hpx/pull/2805
4353 https://github.com/TheHPXProject/hpx/pull/2802
4354 https://github.com/TheHPXProject/hpx/issues/2801
4355 https://github.com/TheHPXProject/hpx/issues/2800
4356 https://github.com/TheHPXProject/hpx/pull/2799
4357 https://github.com/TheHPXProject/hpx/pull/2798
4358 https://github.com/TheHPXProject/hpx/pull/2797

- Issue #2796[4359] - wait_for does always in "deferred" state for calls on remote localities
- Issue #2795[4360] - Serialization of types without default constructor
- PR #2794[4361] - Fixing test for partitioned_vector iteration
- PR #2792[4362] - Implemented segmented find and its variations for partitioned vector
- PR #2791[4363] - Circumvent scary warning about placement new
- PR #2790[4364] - Fix OSX build
- PR #2789[4365] - Resource partitioner
- PR #2788[4366] - Adapt parallel::is_heap and parallel::is_heap_until to Ranges TS
- PR #2787[4367] - Unwrap hotfixes
- PR #2786[4368] - Update CMake Minimum Version to 3.3.2 (refs #2565)
- Issue #2785[4369] - Issues with masks and cpuset
- PR #2784[4370] - Error with reduce and transform reduce fixed
- PR #2783[4371] - StackOverflow integration with libsigsegv
- PR #2782[4372] - Replace boost::atomic with std::atomic (where possible)
- PR #2781[4373] - Check for and optionally use [[deprecated]] attribute
- PR #2780[4374] - Adding empty (but non-trivial) destructor to circumvent warnings
- PR #2779[4375] - Exception info tweaks
- PR #2778[4376] - Implement parallel::partition
- PR #2777[4377] - Improve error handling in gather_here/gather_there
- PR #2776[4378] - Fix a bug in compiler version check
- PR #2775[4379] - Fix compilation when HPX_WITH_LOGGING is OFF
- PR #2774[4380] - Removing dependency on Boost.Date_Time
- PR #2773[4381] - Add sync_images() method to spmd_block class

[4359] https://github.com/TheHPXProject/hpx/issues/2796
[4360] https://github.com/TheHPXProject/hpx/issues/2795
[4361] https://github.com/TheHPXProject/hpx/pull/2794
[4362] https://github.com/TheHPXProject/hpx/pull/2792
[4363] https://github.com/TheHPXProject/hpx/pull/2791
[4364] https://github.com/TheHPXProject/hpx/pull/2790
[4365] https://github.com/TheHPXProject/hpx/pull/2789
[4366] https://github.com/TheHPXProject/hpx/pull/2788
[4367] https://github.com/TheHPXProject/hpx/pull/2787
[4368] https://github.com/TheHPXProject/hpx/pull/2786
[4369] https://github.com/TheHPXProject/hpx/issues/2785
[4370] https://github.com/TheHPXProject/hpx/pull/2784
[4371] https://github.com/TheHPXProject/hpx/pull/2783
[4372] https://github.com/TheHPXProject/hpx/pull/2782
[4373] https://github.com/TheHPXProject/hpx/pull/2781
[4374] https://github.com/TheHPXProject/hpx/pull/2780
[4375] https://github.com/TheHPXProject/hpx/pull/2779
[4376] https://github.com/TheHPXProject/hpx/pull/2778
[4377] https://github.com/TheHPXProject/hpx/pull/2777
[4378] https://github.com/TheHPXProject/hpx/pull/2776
[4379] https://github.com/TheHPXProject/hpx/pull/2775
[4380] https://github.com/TheHPXProject/hpx/pull/2774
[4381] https://github.com/TheHPXProject/hpx/pull/2773

- PR #2772[4382] - Adding documentation for PAPI counters
- PR #2771[4383] - Removing boost preprocessor dependency
- PR #2770[4384] - Adding test, fixing deadlock in config registry
- PR #2769[4385] - Remove some other warnings and errors detected by clang 5.0
- Issue #2768[4386] - Is there iterator tag for HPX?
- PR #2767[4387] - Improvements to continuation annotation
- PR #2765[4388] - gcc split stack support for HPX threads #620
- PR #2764[4389] - Fix some uses of begin/end, remove unnecessary includes
- PR #2763[4390] - Bump minimal Boost version to 1.55.0
- PR #2762[4391] - hpx::partitioned_vector serializer
- PR #2761[4392] - Adding configuration summary to cmake output and –hpx:info
- PR #2760[4393] - Removing 1d_hydro example as it is broken
- PR #2758[4394] - Remove various warnings detected by clang 5.0
- Issue #2757[4395] - In case of a “raw thread” is needed per core for implementing parallel algorithm, what is good practice in HPX?
- PR #2756[4396] - Allowing for LCOs to be simple components
- PR #2755[4397] - Removing make_index_pack_unrolled
- PR #2754[4398] - Implement parallel::unique_copy
- PR #2753[4399] - Fixing detection of [[fallthrough]] attribute
- PR #2752[4400] - New thread priority names
- PR #2751[4401] - Replace boost::exception with proposed exception_info
- PR #2750[4402] - Replace boost::iterator_range
- PR #2749[4403] - Fixing hdf5 examples
- Issue #2748[4404] - HPX fails to build with enabled hdf5 examples

4382 https://github.com/TheHPXProject/hpx/pull/2772
4383 https://github.com/TheHPXProject/hpx/pull/2771
4384 https://github.com/TheHPXProject/hpx/pull/2770
4385 https://github.com/TheHPXProject/hpx/pull/2769
4386 https://github.com/TheHPXProject/hpx/issues/2768
4387 https://github.com/TheHPXProject/hpx/pull/2767
4388 https://github.com/TheHPXProject/hpx/pull/2765
4389 https://github.com/TheHPXProject/hpx/pull/2764
4390 https://github.com/TheHPXProject/hpx/pull/2763
4391 https://github.com/TheHPXProject/hpx/pull/2762
4392 https://github.com/TheHPXProject/hpx/pull/2761
4393 https://github.com/TheHPXProject/hpx/pull/2760
4394 https://github.com/TheHPXProject/hpx/pull/2758
4395 https://github.com/TheHPXProject/hpx/issues/2757
4396 https://github.com/TheHPXProject/hpx/pull/2756
4397 https://github.com/TheHPXProject/hpx/pull/2755
4398 https://github.com/TheHPXProject/hpx/pull/2754
4399 https://github.com/TheHPXProject/hpx/pull/2753
4400 https://github.com/TheHPXProject/hpx/pull/2752
4401 https://github.com/TheHPXProject/hpx/pull/2751
4402 https://github.com/TheHPXProject/hpx/pull/2750
4403 https://github.com/TheHPXProject/hpx/pull/2749
4404 https://github.com/TheHPXProject/hpx/issues/2748

- Issue #2747[4405] - Inherited task priorities break certain DAG optimizations
- Issue #2746[4406] - HPX segfaulting with valgrind
- PR #2745[4407] - Adding extended arithmetic performance counters
- PR #2744[4408] - Adding ability to statistics counters to reset base counter
- Issue #2743[4409] - Statistics counter does not support resetting
- PR #2742[4410] - Making sure Vc V2 builds without additional HPX configuration flags
- PR #2741[4411] - Deprecate unwrapped and implement unwrap and unwrapping
- PR #2740[4412] - Coroutine stackoverflow detection for linux/posix; Issue #2408
- PR #2739[4413] - Add files via upload
- PR #2738[4414] - Appveyor support
- PR #2737[4415] - Fixing 2735
- Issue #2736[4416] - 1d_hydro example doesn't work
- Issue #2735[4417] - partitioned_vector_subview test failing
- PR #2734[4418] - Add C++11 range utilities
- PR #2733[4419] - Adapting iterator requirements for parallel algorithms
- PR #2732[4420] - Integrate C++ Co-arrays
- PR #2731[4421] - Adding on_migrated event handler to migratable component instances
- Issue #2729[4422] - Add on_migrated() event handler to migratable components
- Issue #2728[4423] - Why Projection is needed in parallel algorithms?
- PR #2727[4424] - Cmake files for StackOverflow Detection
- PR #2726[4425] - CMake for Stack Overflow Detection
- PR #2725[4426] - Implemented segmented algorithms for partitioned vector
- PR #2724[4427] - Fix examples in Action documentation

4405 https://github.com/TheHPXProject/hpx/issues/2747
4406 https://github.com/TheHPXProject/hpx/issues/2746
4407 https://github.com/TheHPXProject/hpx/pull/2745
4408 https://github.com/TheHPXProject/hpx/pull/2744
4409 https://github.com/TheHPXProject/hpx/issues/2743
4410 https://github.com/TheHPXProject/hpx/pull/2742
4411 https://github.com/TheHPXProject/hpx/pull/2741
4412 https://github.com/TheHPXProject/hpx/pull/2740
4413 https://github.com/TheHPXProject/hpx/pull/2739
4414 https://github.com/TheHPXProject/hpx/pull/2738
4415 https://github.com/TheHPXProject/hpx/pull/2737
4416 https://github.com/TheHPXProject/hpx/issues/2736
4417 https://github.com/TheHPXProject/hpx/issues/2735
4418 https://github.com/TheHPXProject/hpx/pull/2734
4419 https://github.com/TheHPXProject/hpx/pull/2733
4420 https://github.com/TheHPXProject/hpx/pull/2732
4421 https://github.com/TheHPXProject/hpx/pull/2731
4422 https://github.com/TheHPXProject/hpx/issues/2729
4423 https://github.com/TheHPXProject/hpx/issues/2728
4424 https://github.com/TheHPXProject/hpx/pull/2727
4425 https://github.com/TheHPXProject/hpx/pull/2726
4426 https://github.com/TheHPXProject/hpx/pull/2725
4427 https://github.com/TheHPXProject/hpx/pull/2724

- PR #2723[4428] - Enable lcos::channel<T>::register_as
- Issue #2722[4429] - channel register_as() failing on compilation
- PR #2721[4430] - Mind map
- PR #2720[4431] - reorder forward declarations to get rid of C++14-only auto return types
- PR #2719[4432] - Add documentation for partitioned_vector and add features in pack.hpp
- Issue #2718[4433] - Some forward declarations in execution_fwd.hpp aren't C++11-compatible
- PR #2717[4434] - Config support for fallthrough attribute
- PR #2716[4435] - Implement parallel::partition_copy
- PR #2715[4436] - initial import of icu string serializer
- PR #2714[4437] - initial import of valarray serializer
- PR #2713[4438] - Remove slashes before CMAKE_FILES_DIRECTORY variables
- PR #2712[4439] - Fixing wait for 1751
- PR #2711[4440] - Adjust code for minimal supported GCC having being bumped to 4.9
- PR #2710[4441] - Adding code of conduct
- PR #2709[4442] - Fixing UB in destroy tests
- PR #2708[4443] - Add inline to prevent multiple definition issue
- Issue #2707[4444] - Multiple defined symbols for task_block.hpp in VS2015
- PR #2706[4445] - Adding .clang-format file
- PR #2704[4446] - Add a synchronous mapping API
- Issue #2703[4447] - Request: Add the .clang-format file to the repository
- Issue #2702[4448] - STEllAR-GROUP/Vc slower than VCv1 possibly due to wrong instructions generated
- Issue #2701[4449] - Datapar with STEllAR-GROUP/Vc requires obscure flag

[4428] https://github.com/TheHPXProject/hpx/pull/2723
[4429] https://github.com/TheHPXProject/hpx/issues/2722
[4430] https://github.com/TheHPXProject/hpx/pull/2721
[4431] https://github.com/TheHPXProject/hpx/pull/2720
[4432] https://github.com/TheHPXProject/hpx/pull/2719
[4433] https://github.com/TheHPXProject/hpx/issues/2718
[4434] https://github.com/TheHPXProject/hpx/pull/2717
[4435] https://github.com/TheHPXProject/hpx/pull/2716
[4436] https://github.com/TheHPXProject/hpx/pull/2715
[4437] https://github.com/TheHPXProject/hpx/pull/2714
[4438] https://github.com/TheHPXProject/hpx/pull/2713
[4439] https://github.com/TheHPXProject/hpx/pull/2712
[4440] https://github.com/TheHPXProject/hpx/pull/2711
[4441] https://github.com/TheHPXProject/hpx/pull/2710
[4442] https://github.com/TheHPXProject/hpx/pull/2709
[4443] https://github.com/TheHPXProject/hpx/pull/2708
[4444] https://github.com/TheHPXProject/hpx/issues/2707
[4445] https://github.com/TheHPXProject/hpx/pull/2706
[4446] https://github.com/TheHPXProject/hpx/pull/2704
[4447] https://github.com/TheHPXProject/hpx/issues/2703
[4448] https://github.com/TheHPXProject/hpx/issues/2702
[4449] https://github.com/TheHPXProject/hpx/issues/2701

- Issue #2700[4450] - Naming inconsistency in parallel algorithms
- Issue #2699[4451] - Iterator requirements are different from standard in parallel copy_if.
- PR #2698[4452] - Properly releasing parcelport write handlers
- Issue #2697[4453] - Compile error in addressing_service.cpp
- Issue #2696[4454] - Building and using HPX statically: undefined references from runtime_support_server.cpp
- Issue #2695[4455] - Executor changes cause compilation failures
- PR #2694[4456] - Refining C++ language mode detection for MSVC
- PR #2693[4457] - P0443 r2
- PR #2692[4458] - Partially reverting changes to parcel_await
- Issue #2689[4459] - HPX build fails when HPX_WITH_CUDA is enabled
- PR #2688[4460] - Make Cuda Clang builds pass
- PR #2687[4461] - Add an is_tuple_like trait for sequenceable type detection
- PR #2686[4462] - Allowing throttling scheduler to be used without idle backoff
- PR #2685[4463] - Add support of std::array to hpx::util::tuple_size and tuple_element
- PR #2684[4464] - Adding new statistics performance counters
- PR #2683[4465] - Replace boost::exception_ptr with std::exception_ptr
- Issue #2682[4466] - HPX does not compile with HPX_WITH_THREAD_MANAGER_IDLE_BACKOFF=OFF
- PR #2681[4467] - Attempt to fix problem in managed_component_base
- PR #2680[4468] - Fix bad size during archive creation
- Issue #2679[4469] - Mismatch between size of archive and container
- Issue #2678[4470] - In parallel algorithm, other tasks are executed to the end even if an exception occurs in any task.
- PR #2677[4471] - Adding include check for std::addressof

[4450] https://github.com/TheHPXProject/hpx/issues/2700
[4451] https://github.com/TheHPXProject/hpx/issues/2699
[4452] https://github.com/TheHPXProject/hpx/pull/2698
[4453] https://github.com/TheHPXProject/hpx/issues/2697
[4454] https://github.com/TheHPXProject/hpx/issues/2696
[4455] https://github.com/TheHPXProject/hpx/issues/2695
[4456] https://github.com/TheHPXProject/hpx/pull/2694
[4457] https://github.com/TheHPXProject/hpx/pull/2693
[4458] https://github.com/TheHPXProject/hpx/pull/2692
[4459] https://github.com/TheHPXProject/hpx/issues/2689
[4460] https://github.com/TheHPXProject/hpx/pull/2688
[4461] https://github.com/TheHPXProject/hpx/pull/2687
[4462] https://github.com/TheHPXProject/hpx/pull/2686
[4463] https://github.com/TheHPXProject/hpx/pull/2685
[4464] https://github.com/TheHPXProject/hpx/pull/2684
[4465] https://github.com/TheHPXProject/hpx/pull/2683
[4466] https://github.com/TheHPXProject/hpx/issues/2682
[4467] https://github.com/TheHPXProject/hpx/pull/2681
[4468] https://github.com/TheHPXProject/hpx/pull/2680
[4469] https://github.com/TheHPXProject/hpx/issues/2679
[4470] https://github.com/TheHPXProject/hpx/issues/2678
[4471] https://github.com/TheHPXProject/hpx/pull/2677

- PR #2676[4472] - Adding parallel::destroy and destroy_n
- PR #2675[4473] - Making sure statistics counters work as expected
- PR #2674[4474] - Turning assertions into exceptions
- PR #2673[4475] - Inhibit direct conversion from future<future<T>> –> future<void>
- PR #2672[4476] - C++17 invoke forms
- PR #2671[4477] - Adding uninitialized_value_construct and uninitialized_value_construct_n
- PR #2670[4478] - Integrate spmd multidimensional views for partitioned_vectors
- PR #2669[4479] - Adding uninitialized_default_construct and uninitialized_default_construct_n
- PR #2668[4480] - Fixing documentation index
- Issue #2667[4481] - Ambiguity of nested hpx::future<void>'s
- Issue #2666[4482] - Statistics Performance counter is not working
- PR #2664[4483] - Adding uninitialized_move and uninitialized_move_n
- Issue #2663[4484] - Seg fault in managed_component::get_base_gid, possibly cause by util::reinitializable_static
- Issue #2662[4485] - Crash in managed_component::get_base_gid due to problem with util::reinitializable_static
- PR #2665[4486] - Hide the `detail` namespace in doxygen per default
- PR #2660[4487] - Add documentation to hpx::util::unwrapped and hpx::util::unwrapped2
- PR #2659[4488] - Improve integration with vcpkg
- PR #2658[4489] - Unify access_data trait for use in both, serialization and de-serialization
- PR #2657[4490] - Removing hpx::lcos::queue<T>
- PR #2656[4491] - Reduce MAX_TERMINATED_THREADS default, improve memory use on manycore cpus
- PR #2655[4492] - Mainteinance for emulate-deleted macros

[4472] https://github.com/TheHPXProject/hpx/pull/2676
[4473] https://github.com/TheHPXProject/hpx/pull/2675
[4474] https://github.com/TheHPXProject/hpx/pull/2674
[4475] https://github.com/TheHPXProject/hpx/pull/2673
[4476] https://github.com/TheHPXProject/hpx/pull/2672
[4477] https://github.com/TheHPXProject/hpx/pull/2671
[4478] https://github.com/TheHPXProject/hpx/pull/2670
[4479] https://github.com/TheHPXProject/hpx/pull/2669
[4480] https://github.com/TheHPXProject/hpx/pull/2668
[4481] https://github.com/TheHPXProject/hpx/issues/2667
[4482] https://github.com/TheHPXProject/hpx/issues/2666
[4483] https://github.com/TheHPXProject/hpx/pull/2664
[4484] https://github.com/TheHPXProject/hpx/issues/2663
[4485] https://github.com/TheHPXProject/hpx/issues/2662
[4486] https://github.com/TheHPXProject/hpx/pull/2665
[4487] https://github.com/TheHPXProject/hpx/pull/2660
[4488] https://github.com/TheHPXProject/hpx/pull/2659
[4489] https://github.com/TheHPXProject/hpx/pull/2658
[4490] https://github.com/TheHPXProject/hpx/pull/2657
[4491] https://github.com/TheHPXProject/hpx/pull/2656
[4492] https://github.com/TheHPXProject/hpx/pull/2655

- PR #2654[4493] - Implement parallel is_heap and is_heap_until
- PR #2653[4494] - Drop support for VS2013
- PR #2652[4495] - This patch makes sure that all parcels in a batch are properly handled
- PR #2649[4496] - Update docs (Table 18) - move transform to end
- Issue #2647[4497] - hpx::parcelset::detail::parcel_data::**has_continuation_** is uninitialized
- Issue #2644[4498] - Some .vcxproj in the HPX.sln fail to build
- Issue #2641[4499] - `hpx::lcos::queue` should be deprecated
- PR #2640[4500] - A new throttling policy with public APIs to suspend/resume
- PR #2639[4501] - Fix a tiny typo in tutorial.
- Issue #2638[4502] - Invalid return type 'void' of constexpr function
- PR #2636[4503] - Add and use HPX_MSVC_WARNING_PRAGMA for #pragma warning
- PR #2633[4504] - Distributed define_spmd_block
- PR #2632[4505] - Making sure container serialization uses size-compatible types
- PR #2631[4506] - Add lcos::local::one_element_channel
- PR #2629[4507] - Move unordered_map out of parcelport into hpx/concurrent
- PR #2628[4508] - Making sure that shutdown does not hang
- PR #2627[4509] - Fix serialization
- PR #2626[4510] - Generate `cmake_variables.qbk` and `cmake_toolchains.qbk` outside of the source tree
- PR #2625[4511] - Supporting -std=c++17 flag
- PR #2624[4512] - Fixing a small cmake typo
- PR #2622[4513] - Update CMake minimum required version to 3.0.2 (closes #2621)
- Issue #2621[4514] - Compiling hpx master fails with /usr/bin/ld: final link failed: Bad value
- PR #2620[4515] - Remove warnings due to some captured variables

[4493] https://github.com/TheHPXProject/hpx/pull/2654
[4494] https://github.com/TheHPXProject/hpx/pull/2653
[4495] https://github.com/TheHPXProject/hpx/pull/2652
[4496] https://github.com/TheHPXProject/hpx/pull/2649
[4497] https://github.com/TheHPXProject/hpx/issues/2647
[4498] https://github.com/TheHPXProject/hpx/issues/2644
[4499] https://github.com/TheHPXProject/hpx/issues/2641
[4500] https://github.com/TheHPXProject/hpx/pull/2640
[4501] https://github.com/TheHPXProject/hpx/pull/2639
[4502] https://github.com/TheHPXProject/hpx/issues/2638
[4503] https://github.com/TheHPXProject/hpx/pull/2636
[4504] https://github.com/TheHPXProject/hpx/pull/2633
[4505] https://github.com/TheHPXProject/hpx/pull/2632
[4506] https://github.com/TheHPXProject/hpx/pull/2631
[4507] https://github.com/TheHPXProject/hpx/pull/2629
[4508] https://github.com/TheHPXProject/hpx/pull/2628
[4509] https://github.com/TheHPXProject/hpx/pull/2627
[4510] https://github.com/TheHPXProject/hpx/pull/2626
[4511] https://github.com/TheHPXProject/hpx/pull/2625
[4512] https://github.com/TheHPXProject/hpx/pull/2624
[4513] https://github.com/TheHPXProject/hpx/pull/2622
[4514] https://github.com/TheHPXProject/hpx/issues/2621
[4515] https://github.com/TheHPXProject/hpx/pull/2620

- PR #2619[4516] - LF multiple parcels
- PR #2618[4517] - Some fixes to libfabric that didn't get caught before the merge
- PR #2617[4518] - Adding `hpx::local_new`
- PR #2616[4519] - Documentation: Extract all entities in order to autolink functions correctly
- Issue #2615[4520] - Documentation: Linking functions is broken
- PR #2614[4521] - Adding serialization for std::deque
- PR #2613[4522] - We need to link with boost.thread and boost.chrono if we use boost.context
- PR #2612[4523] - Making sure for_loop_n(par, …) is actually executed in parallel
- PR #2611[4524] - Add documentation to invoke_fused and friends NFC
- PR #2610[4525] - Added reduction templates using an identity value
- PR #2608[4526] - Fixing some unused vars in inspect
- PR #2607[4527] - Fixed build for mingw
- PR #2606[4528] - Supporting generic context for boost >= 1.61
- PR #2605[4529] - Parcelport libfabric3
- PR #2604[4530] - Adding allocator support to promise and friends
- PR #2603[4531] - Barrier hang
- PR #2602[4532] - Changes to scheduler to steal from one high-priority queue
- Issue #2601[4533] - High priority tasks are not executed first
- PR #2600[4534] - Compat fixes
- PR #2599[4535] - Compatibility layer for threading support
- PR #2598[4536] - V1.1
- PR #2597[4537] - Release V1.0

[4516] https://github.com/TheHPXProject/hpx/pull/2619
[4517] https://github.com/TheHPXProject/hpx/pull/2618
[4518] https://github.com/TheHPXProject/hpx/pull/2617
[4519] https://github.com/TheHPXProject/hpx/pull/2616
[4520] https://github.com/TheHPXProject/hpx/issues/2615
[4521] https://github.com/TheHPXProject/hpx/pull/2614
[4522] https://github.com/TheHPXProject/hpx/pull/2613
[4523] https://github.com/TheHPXProject/hpx/pull/2612
[4524] https://github.com/TheHPXProject/hpx/pull/2611
[4525] https://github.com/TheHPXProject/hpx/pull/2610
[4526] https://github.com/TheHPXProject/hpx/pull/2608
[4527] https://github.com/TheHPXProject/hpx/pull/2607
[4528] https://github.com/TheHPXProject/hpx/pull/2606
[4529] https://github.com/TheHPXProject/hpx/pull/2605
[4530] https://github.com/TheHPXProject/hpx/pull/2604
[4531] https://github.com/TheHPXProject/hpx/pull/2603
[4532] https://github.com/TheHPXProject/hpx/pull/2602
[4533] https://github.com/TheHPXProject/hpx/issues/2601
[4534] https://github.com/TheHPXProject/hpx/pull/2600
[4535] https://github.com/TheHPXProject/hpx/pull/2599
[4536] https://github.com/TheHPXProject/hpx/pull/2598
[4537] https://github.com/TheHPXProject/hpx/pull/2597

- PR #2592[4538] - First attempt to introduce spmd_block in hpx
- PR #2586[4539] - local_segment in segmented_iterator_traits
- Issue #2584[4540] - Add allocator support to promise, packaged_task and friends
- PR #2576[4541] - Add missing dependencies of cuda based tests
- PR #2575[4542] - Remove warnings due to some captured variables
- Issue #2574[4543] - MSVC 2015 Compiler crash when building HPX
- Issue #2568[4544] - Remove throttle_scheduler as it has been abandoned
- Issue #2566[4545] - Add an inline versioning namespace before 1.0 release
- Issue #2565[4546] - Raise minimal cmake version requirement
- PR #2556[4547] - Fixing scan partitioner
- PR #2546[4548] - Broadcast async
- Issue #2543[4549] - make install fails due to a non-existing .so file
- PR #2495[4550] - wait_or_add_new returning thread_id_type
- Issue #2480[4551] - Unable to register new performance counter
- Issue #2471[4552] - no type named 'fcontext_t' in namespace
- Issue #2456[4553] - Re-implement hpx::util::unwrapped
- Issue #2455[4554] - Add more arithmetic performance counters
- PR #2454[4555] - Fix a couple of warnings and compiler errors
- PR #2453[4556] - Timed executor support
- PR #2447[4557] - Implementing new executor API (P0443)
- Issue #2439[4558] - Implement executor proposal
- Issue #2408[4559] - Stackoverflow detection for linux, e.g. based on libsigsegv
- PR #2377[4560] - Add a customization point for put_parcel so we can override actions

[4538] https://github.com/TheHPXProject/hpx/pull/2592
[4539] https://github.com/TheHPXProject/hpx/pull/2586
[4540] https://github.com/TheHPXProject/hpx/issues/2584
[4541] https://github.com/TheHPXProject/hpx/pull/2576
[4542] https://github.com/TheHPXProject/hpx/pull/2575
[4543] https://github.com/TheHPXProject/hpx/issues/2574
[4544] https://github.com/TheHPXProject/hpx/issues/2568
[4545] https://github.com/TheHPXProject/hpx/issues/2566
[4546] https://github.com/TheHPXProject/hpx/issues/2565
[4547] https://github.com/TheHPXProject/hpx/pull/2556
[4548] https://github.com/TheHPXProject/hpx/pull/2546
[4549] https://github.com/TheHPXProject/hpx/issues/2543
[4550] https://github.com/TheHPXProject/hpx/pull/2495
[4551] https://github.com/TheHPXProject/hpx/issues/2480
[4552] https://github.com/TheHPXProject/hpx/issues/2471
[4553] https://github.com/TheHPXProject/hpx/issues/2456
[4554] https://github.com/TheHPXProject/hpx/issues/2455
[4555] https://github.com/TheHPXProject/hpx/pull/2454
[4556] https://github.com/TheHPXProject/hpx/pull/2453
[4557] https://github.com/TheHPXProject/hpx/pull/2447
[4558] https://github.com/TheHPXProject/hpx/issues/2439
[4559] https://github.com/TheHPXProject/hpx/issues/2408
[4560] https://github.com/TheHPXProject/hpx/pull/2377

- Issue #2368[4561] - HPX_ASSERT problem
- Issue #2324[4562] - Change default number of threads used to the maximum of the system
- Issue #2266[4563] - hpx_0.9.99 make tests fail
- PR #2195[4564] - Support for code completion in VIM
- Issue #2137[4565] - Hpx does not compile over osx
- Issue #2092[4566] - make tests should just build the tests
- Issue #2026[4567] - Build HPX with Apple's clang
- Issue #1932[4568] - hpx with PBS fails on multiple localities
- PR #1914[4569] - Parallel heap algorithm implementations WIP
- Issue #1598[4570] - Disconnecting a locality results in segfault using heartbeat example
- Issue #1404[4571] - unwrapped doesn't work with movable only types
- Issue #1400[4572] - hpx::util::unwrapped doesn't work with non-future types
- Issue #1205[4573] - TSS is broken
- Issue #1126[4574] - vector<future<T> > does not work gracefully with dataflow, when_all and unwrapped
- Issue #1056[4575] - Thread manager cleanup
- Issue #863[4576] - Futures should not require a default constructor
- Issue #856[4577] - Allow runtimemode_connect to be used with security enabled
- Issue #726[4578] - Valgrind
- Issue #701[4579] - Add RCR performance counter component
- Issue #528[4580] - Add support for known failures and warning count/comparisons to hpx_run_tests.py

[4561] https://github.com/TheHPXProject/hpx/issues/2368
[4562] https://github.com/TheHPXProject/hpx/issues/2324
[4563] https://github.com/TheHPXProject/hpx/issues/2266
[4564] https://github.com/TheHPXProject/hpx/pull/2195
[4565] https://github.com/TheHPXProject/hpx/issues/2137
[4566] https://github.com/TheHPXProject/hpx/issues/2092
[4567] https://github.com/TheHPXProject/hpx/issues/2026
[4568] https://github.com/TheHPXProject/hpx/issues/1932
[4569] https://github.com/TheHPXProject/hpx/pull/1914
[4570] https://github.com/TheHPXProject/hpx/issues/1598
[4571] https://github.com/TheHPXProject/hpx/issues/1404
[4572] https://github.com/TheHPXProject/hpx/issues/1400
[4573] https://github.com/TheHPXProject/hpx/issues/1205
[4574] https://github.com/TheHPXProject/hpx/issues/1126
[4575] https://github.com/TheHPXProject/hpx/issues/1056
[4576] https://github.com/TheHPXProject/hpx/issues/863
[4577] https://github.com/TheHPXProject/hpx/issues/856
[4578] https://github.com/TheHPXProject/hpx/issues/726
[4579] https://github.com/TheHPXProject/hpx/issues/701
[4580] https://github.com/TheHPXProject/hpx/issues/528

HPX V1.0.0 (Apr 24, 2017)

General changes

Here are some of the main highlights and changes for this release (in no particular order):

- Added the facility `hpx::split_future` which allows one to convert a `future<tuple<Ts...>>` into a `tuple<future<Ts>...>`. This functionality is not available when compiling *HPX* with VS2012.
- Added a new type of performance counter which allows one to return a list of values for each invocation. We also added a first counter of this type which collects a histogram of the times between parcels being created.
- Added new LCOs: `hpx::lcos::channel` and `hpx::lcos::local::channel` which are very similar to the well known channel constructs used in the Go language.
- Added new performance counters reporting the amount of data handled by the networking layer on a action-by-action basis (please see PR #2289[4581] for more details).
- Added a new facility `hpx::lcos::barrier`, replacing the equally named older one. The new facility has a slightly changed API and is much more efficient. Most notable, the new facility exposes a (global) function `hpx::lcos::barrier::synchronize()` which represents a global barrier across all localities.
- We have started to add support for vectorization to our parallel algorithm implementations. This support depends on using an external library, currently either Vc Library or **|boost_simd|_**. Please see Issue #2333[4582] for a list of currently supported algorithms. This is an experimental feature and its implementation and/or API might change in the future. Please see this blog-post[4583] for more information.
- The parameter sequence for the `hpx::parallel::transform_reduce` overload taking one iterator range has changed to match the changes this algorithm has undergone while being moved to C++17. The old overload can be still enabled at configure time by specifying `-DHPX_WITH_TRANSFORM_REDUCE_COMPATIBILITY=On` to CMake.
- The algorithm `hpx::parallel::inner_product` has been renamed to `hpx::parallel::transform_reduce` to match the changes this algorithm has undergone while being moved to C++17. The old inner_product names can be still enabled at configure time by specifying `-DHPX_WITH_TRANSFORM_REDUCE_COMPATIBILITY=On` to CMake.
- Added versions of `hpx::get_ptr` taking client side representations for component instances as their parameter (instead of a global id).
- Added the helper utility `hpx::performance_counters::performance_counter_set` helping to encapsulate a set of performance counters to be managed concurrently.
- All execution policies and related classes have been renamed to be consistent with the naming changes applied for C++17. All policies now live in the namespace `hpx::parallel::execution`. The ols names can be still enabled at configure time by specifying `-DHPX_WITH_EXECUTION_POLICY_COMPATIBILITY=On` to CMake.
- The thread scheduling subsystem has undergone a major refactoring which results in significant performance improvements. We have also imroved the performance of creating `hpx::future` and of various facilities handling those.

[4581] https://github.com/TheHPXProject/hpx/pull/2289
[4582] https://github.com/TheHPXProject/hpx/issues/2333
[4583] http://stellar-group.org/2016/09/vectorized-cpp-parallel-algorithms-with-hpx/

- We have consolidated all of the code in HPX.Compute related to the integration of CUDA. `hpx::partitioned_vector` has been enabled to be usable with `hpx::compute::vector` which allows one to place the partitions on one or more GPU devices.
- Added new performance counters exposing various internals of the thread scheduling subsystem, such as the current idle- and busy-loop counters and instantaneous scheduler utilization.
- Extended and improved the use of the ITTNotify hooks allowing to collect performance counter data and function annotation information from within the Intel Amplifier tool.

Breaking changes

- We have dropped support for the gcc compiler versions V4.6 and 4.7. The minimal gcc version we now test on is gcc V4.8.
- We have removed (default) support for `boost::chrono` in interfaces, uses of it have been replaced with `std::chrono`. This facility can be still enabled at configure time by specifying `-DHPX_WITH_BOOST_CHRONO_COMPATIBILITY=On` to CMake.
- The parameter sequence for the `hpx::parallel::transform_reduce` overload taking one iterator range has changed to match the changes this algorithm has undergone while being moved to C++17.
- The algorithm `hpx::parallel::inner_product` has been renamed to `hpx::parallel::transform_reduce` to match the changes this algorithm has undergone while being moved to C++17.
- the build options `HPX_WITH_COLOCATED_BACKWARDS_COMPATIBILITY` and `HPX_WITH_COMPONENT_GET_GID_COMPATIBILITY` are now disabled by default. Please change your code still depending on the deprecated interfaces.

Bug fixes (closed tickets)

Here is a list of the important tickets we closed for this release.

- PR #2596[4584] - Adding apex data
- PR #2595[4585] - Remove obsolete file
- Issue #2594[4586] - FindOpenCL.cmake mismatch with the official cmake module
- PR #2592[4587] - First attempt to introduce spmd_block in hpx
- Issue #2591[4588] - Feature request: continuation (then) which does not require the callable object to take a future<R> as parameter
- PR #2588[4589] - Daint fixes
- PR #2587[4590] - Fixing transfer_(continuation)_action::schedule

[4584] https://github.com/TheHPXProject/hpx/pull/2596
[4585] https://github.com/TheHPXProject/hpx/pull/2595
[4586] https://github.com/TheHPXProject/hpx/issues/2594
[4587] https://github.com/TheHPXProject/hpx/pull/2592
[4588] https://github.com/TheHPXProject/hpx/issues/2591
[4589] https://github.com/TheHPXProject/hpx/pull/2588
[4590] https://github.com/TheHPXProject/hpx/pull/2587

- PR #2585[4591] - Work around MSVC having an ICE when compiling with -Ob2
- PR #2583[4592] - changing 7zip command to 7za in roll_release.sh
- PR #2582[4593] - First attempt to introduce spmd_block in hpx
- PR #2581[4594] - Enable annotated function for parallel algorithms
- PR #2580[4595] - First attempt to introduce spmd_block in hpx
- PR #2579[4596] - Make thread NICE level setting an option
- PR #2578[4597] - Implementing enqueue instead of busy wait when no sender is available
- PR #2577[4598] - Retrieve -std=c++11 consistent nvcc flag
- PR #2576[4599] - Add missing dependencies of cuda based tests
- PR #2575[4600] - Remove warnings due to some captured variables
- PR #2573[4601] - Attempt to resolve resolve_locality
- PR #2572[4602] - Adding APEX hooks to background thread
- PR #2571[4603] - Pick up hpx.ignore_batch_env from config map
- PR #2570[4604] - Add commandline options –hpx:print-counters-locally
- PR #2569[4605] - Fix computeapi unit tests
- PR #2567[4606] - This adds another barrier::synchronize before registering performance counters
- PR #2564[4607] - Cray static toolchain support
- PR #2563[4608] - Fixed unhandled exception during startup
- PR #2562[4609] - Remove partitioned_vector.cu from build tree when nvcc is used
- Issue #2561[4610] - octo-tiger crash with commit 6e921495ff6c26f125d62629cbaad0525f14f7ab
- PR #2560[4611] - Prevent -Wundef warnings on Vc version checks
- PR #2559[4612] - Allowing CUDA callback to set the future directly from an OS thread

[4591] https://github.com/TheHPXProject/hpx/pull/2585
[4592] https://github.com/TheHPXProject/hpx/pull/2583
[4593] https://github.com/TheHPXProject/hpx/pull/2582
[4594] https://github.com/TheHPXProject/hpx/pull/2581
[4595] https://github.com/TheHPXProject/hpx/pull/2580
[4596] https://github.com/TheHPXProject/hpx/pull/2579
[4597] https://github.com/TheHPXProject/hpx/pull/2578
[4598] https://github.com/TheHPXProject/hpx/pull/2577
[4599] https://github.com/TheHPXProject/hpx/pull/2576
[4600] https://github.com/TheHPXProject/hpx/pull/2575
[4601] https://github.com/TheHPXProject/hpx/pull/2573
[4602] https://github.com/TheHPXProject/hpx/pull/2572
[4603] https://github.com/TheHPXProject/hpx/pull/2571
[4604] https://github.com/TheHPXProject/hpx/pull/2570
[4605] https://github.com/TheHPXProject/hpx/pull/2569
[4606] https://github.com/TheHPXProject/hpx/pull/2567
[4607] https://github.com/TheHPXProject/hpx/pull/2564
[4608] https://github.com/TheHPXProject/hpx/pull/2563
[4609] https://github.com/TheHPXProject/hpx/pull/2562
[4610] https://github.com/TheHPXProject/hpx/issues/2561
[4611] https://github.com/TheHPXProject/hpx/pull/2560
[4612] https://github.com/TheHPXProject/hpx/pull/2559

- PR #2558[4613] - Remove warnings due to float precisions
- PR #2557[4614] - Removing bogus handling of compile flags for CUDA
- PR #2556[4615] - Fixing scan partitioner
- PR #2554[4616] - Add more diagnostics to error thrown from find_appropriate_destination
- Issue #2555[4617] - No valid parcelport configured
- PR #2553[4618] - Add cmake cuda_arch option
- PR #2552[4619] - Remove incomplete datapar bindings to libflatarray
- PR #2551[4620] - Rename hwloc_topology to hwloc_topology_info
- PR #2550[4621] - Apex api updates
- PR #2549[4622] - Pre-include defines.hpp to get the macro HPX_HAVE_CUDA value
- PR #2548[4623] - Fixing issue with disconnect
- PR #2546[4624] - Some fixes around cuda clang partitioned_vector example
- PR #2545[4625] - Fix uses of the Vc2 datapar flags; the value, not the type, should be passed to functions
- PR #2542[4626] - Make HPX_WITH_MALLOC easier to use
- PR #2541[4627] - avoid recompiles when enabling/disabling examples
- PR #2540[4628] - Fixing usage of target_link_libraries()
- PR #2539[4629] - fix RPATH behaviour
- Issue #2538[4630] - HPX_WITH_CUDA corrupts compilation flags
- PR #2537[4631] - Add output of a Bazel Skylark extension for paths and compile options
- PR #2536[4632] - Add counter exposing total available memory to Windows as well
- PR #2535[4633] - Remove obsolete support for security
- Issue #2534[4634] - Remove command line option `--hpx:run-agas-server`
- PR #2533[4635] - Pre-cache locality endpoints during bootstrap

[4613] https://github.com/TheHPXProject/hpx/pull/2558
[4614] https://github.com/TheHPXProject/hpx/pull/2557
[4615] https://github.com/TheHPXProject/hpx/pull/2556
[4616] https://github.com/TheHPXProject/hpx/pull/2554
[4617] https://github.com/TheHPXProject/hpx/issues/2555
[4618] https://github.com/TheHPXProject/hpx/pull/2553
[4619] https://github.com/TheHPXProject/hpx/pull/2552
[4620] https://github.com/TheHPXProject/hpx/pull/2551
[4621] https://github.com/TheHPXProject/hpx/pull/2550
[4622] https://github.com/TheHPXProject/hpx/pull/2549
[4623] https://github.com/TheHPXProject/hpx/pull/2548
[4624] https://github.com/TheHPXProject/hpx/pull/2546
[4625] https://github.com/TheHPXProject/hpx/pull/2545
[4626] https://github.com/TheHPXProject/hpx/pull/2542
[4627] https://github.com/TheHPXProject/hpx/pull/2541
[4628] https://github.com/TheHPXProject/hpx/pull/2540
[4629] https://github.com/TheHPXProject/hpx/pull/2539
[4630] https://github.com/TheHPXProject/hpx/issues/2538
[4631] https://github.com/TheHPXProject/hpx/pull/2537
[4632] https://github.com/TheHPXProject/hpx/pull/2536
[4633] https://github.com/TheHPXProject/hpx/pull/2535
[4634] https://github.com/TheHPXProject/hpx/issues/2534
[4635] https://github.com/TheHPXProject/hpx/pull/2533

- PR #2532[4636] - Fixing handling of GIDs during serialization preprocessing
- PR #2531[4637] - Amend uses of the term "functor"
- PR #2529[4638] - added counter for reading available memory
- PR #2527[4639] - Facilities to create actions from lambdas
- PR #2526[4640] - Updated docs: HPX_WITH_EXAMPLES
- PR #2525[4641] - Remove warnings related to unused captured variables
- Issue #2524[4642] - CMAKE failed because it is missing: TCMALLOC_LIBRARY TCMALLOC_INCLUDE_DIR
- PR #2523[4643] - Fixing compose_cb stack overflow
- PR #2522[4644] - Instead of unlocking, ignore the lock while creating the message handler
- PR #2521[4645] - Create `LPROGRESS_` logging macro to simplify progress tracking and timings
- PR #2520[4646] - Intel 17 support
- PR #2519[4647] - Fix components example
- PR #2518[4648] - Fixing parcel scheduling
- Issue #2517[4649] - Race condition during Parcel Coalescing Handler creation
- Issue #2516[4650] - HPX locks up when using at least 256 localities
- Issue #2515[4651] - error: Install cannot find "/lib/hpx/libparcel_coalescing.so.0.9.99" but I can see that file
- PR #2514[4652] - Making sure that all continuations of a shared_future are invoked in order
- PR #2513[4653] - Fixing locks held during suspension
- PR #2512[4654] - MPI Parcelport improvements and fixes related to the background work changes
- PR #2511[4655] - Fixing bit-wise (zero-copy) serialization
- Issue #2509[4656] - Linking errors in hwloc_topology

4636 https://github.com/TheHPXProject/hpx/pull/2532
4637 https://github.com/TheHPXProject/hpx/pull/2531
4638 https://github.com/TheHPXProject/hpx/pull/2529
4639 https://github.com/TheHPXProject/hpx/pull/2527
4640 https://github.com/TheHPXProject/hpx/pull/2526
4641 https://github.com/TheHPXProject/hpx/pull/2525
4642 https://github.com/TheHPXProject/hpx/issues/2524
4643 https://github.com/TheHPXProject/hpx/pull/2523
4644 https://github.com/TheHPXProject/hpx/pull/2522
4645 https://github.com/TheHPXProject/hpx/pull/2521
4646 https://github.com/TheHPXProject/hpx/pull/2520
4647 https://github.com/TheHPXProject/hpx/pull/2519
4648 https://github.com/TheHPXProject/hpx/pull/2518
4649 https://github.com/TheHPXProject/hpx/issues/2517
4650 https://github.com/TheHPXProject/hpx/issues/2516
4651 https://github.com/TheHPXProject/hpx/issues/2515
4652 https://github.com/TheHPXProject/hpx/pull/2514
4653 https://github.com/TheHPXProject/hpx/pull/2513
4654 https://github.com/TheHPXProject/hpx/pull/2512
4655 https://github.com/TheHPXProject/hpx/pull/2511
4656 https://github.com/TheHPXProject/hpx/issues/2509

- PR #2508[4657] - Added documentation for debugging with core files
- PR #2506[4658] - Fixing background work invocations
- PR #2505[4659] - Fix tuple serialization
- Issue #2504[4660] - Ensure continuations are called in the order they have been attached
- PR #2503[4661] - Adding serialization support for Vc v2 (datapar)
- PR #2502[4662] - Resolve various, minor compiler warnings
- PR #2501[4663] - Some other fixes around cuda examples
- Issue #2500[4664] - nvcc / cuda clang issue due to a missing -DHPX_WITH_CUDA flag
- PR #2499[4665] - Adding support for std::array to wait_all and friends
- PR #2498[4666] - Execute background work as HPX thread
- PR #2497[4667] - Fixing configuration options for spinlock-deadlock detection
- PR #2496[4668] - Accounting for different compilers in CrayKNL toolchain file
- PR #2494[4669] - Adding component base class which ties a component instance to a given executor
- PR #2493[4670] - Enable controlling amount of pending threads which must be available to allow thread stealing
- PR #2492[4671] - Adding new command line option –hpx:print-counter-reset
- PR #2491[4672] - Resolve ambiguities when compiling with APEX
- PR #2490[4673] - Resuming threads waiting on future with higher priority
- Issue #2489[4674] - nvcc issue because -std=c++11 appears twice
- PR #2488[4675] - Adding performance counters exposing the internal idle and busy-loop counters
- PR #2487[4676] - Allowing for plain suspend to reschedule thread right away
- PR #2486[4677] - Only flag HPX code for CUDA if HPX_WITH_CUDA is set
- PR #2485[4678] - Making thread-queue parameters runtime-configurable

[4657] https://github.com/TheHPXProject/hpx/pull/2508
[4658] https://github.com/TheHPXProject/hpx/pull/2506
[4659] https://github.com/TheHPXProject/hpx/pull/2505
[4660] https://github.com/TheHPXProject/hpx/issues/2504
[4661] https://github.com/TheHPXProject/hpx/pull/2503
[4662] https://github.com/TheHPXProject/hpx/pull/2502
[4663] https://github.com/TheHPXProject/hpx/pull/2501
[4664] https://github.com/TheHPXProject/hpx/issues/2500
[4665] https://github.com/TheHPXProject/hpx/pull/2499
[4666] https://github.com/TheHPXProject/hpx/pull/2498
[4667] https://github.com/TheHPXProject/hpx/pull/2497
[4668] https://github.com/TheHPXProject/hpx/pull/2496
[4669] https://github.com/TheHPXProject/hpx/pull/2494
[4670] https://github.com/TheHPXProject/hpx/pull/2493
[4671] https://github.com/TheHPXProject/hpx/pull/2492
[4672] https://github.com/TheHPXProject/hpx/pull/2491
[4673] https://github.com/TheHPXProject/hpx/pull/2490
[4674] https://github.com/TheHPXProject/hpx/issues/2489
[4675] https://github.com/TheHPXProject/hpx/pull/2488
[4676] https://github.com/TheHPXProject/hpx/pull/2487
[4677] https://github.com/TheHPXProject/hpx/pull/2486
[4678] https://github.com/TheHPXProject/hpx/pull/2485

- PR #2484[4679] - Added atomic counter for parcel-destinations
- PR #2483[4680] - Added priority-queue lifo scheduler
- PR #2482[4681] - Changing scheduler to steal only if more than a minimal number of tasks are available
- PR #2481[4682] - Extending command line option –hpx:print-counter-destination to support value 'none'
- PR #2479[4683] - Added option to disable signal handler
- PR #2478[4684] - Making sure the sine performance counter module gets loaded only for the corresponding example
- Issue #2477[4685] - Breaking at a throw statement
- PR #2476[4686] - Annotated function
- PR #2475[4687] - Ensure that using %osthread% during logging will not throw for non-hpx threads
- PR #2474[4688] - Remove now superficial non_direct actions from base_lco and friends
- PR #2473[4689] - Refining support for ITTNotify
- PR #2472[4690] - Some fixes around hpx compute
- Issue #2470[4691] - redefinition of boost::detail::spinlock
- Issue #2469[4692] - Dataflow performance issue
- PR #2468[4693] - Perf docs update
- PR #2466[4694] - Guarantee to execute remote direct actions on HPX-thread
- PR #2465[4695] - Improve demo : Async copy and fixed device handling
- PR #2464[4696] - Adding performance counter exposing instantaneous scheduler utilization
- PR #2463[4697] - Downcast to future<void>
- PR #2462[4698] - Fixed usage of ITT-Notify API with Intel Amplifier
- PR #2461[4699] - Cublas demo

[4679] https://github.com/TheHPXProject/hpx/pull/2484
[4680] https://github.com/TheHPXProject/hpx/pull/2483
[4681] https://github.com/TheHPXProject/hpx/pull/2482
[4682] https://github.com/TheHPXProject/hpx/pull/2481
[4683] https://github.com/TheHPXProject/hpx/pull/2479
[4684] https://github.com/TheHPXProject/hpx/pull/2478
[4685] https://github.com/TheHPXProject/hpx/issues/2477
[4686] https://github.com/TheHPXProject/hpx/pull/2476
[4687] https://github.com/TheHPXProject/hpx/pull/2475
[4688] https://github.com/TheHPXProject/hpx/pull/2474
[4689] https://github.com/TheHPXProject/hpx/pull/2473
[4690] https://github.com/TheHPXProject/hpx/pull/2472
[4691] https://github.com/TheHPXProject/hpx/issues/2470
[4692] https://github.com/TheHPXProject/hpx/issues/2469
[4693] https://github.com/TheHPXProject/hpx/pull/2468
[4694] https://github.com/TheHPXProject/hpx/pull/2466
[4695] https://github.com/TheHPXProject/hpx/pull/2465
[4696] https://github.com/TheHPXProject/hpx/pull/2464
[4697] https://github.com/TheHPXProject/hpx/pull/2463
[4698] https://github.com/TheHPXProject/hpx/pull/2462
[4699] https://github.com/TheHPXProject/hpx/pull/2461

- PR #2460[4700] - Fixing thread bindings
- PR #2459[4701] - Make -std=c++11 nvcc flag consistent for in-build and installed versions
- Issue #2457[4702] - Segmentation fault when registering a partitioned vector
- PR #2452[4703] - Properly releasing global barrier for unhandled exceptions
- PR #2451[4704] - Fixing long shutdown times
- PR #2450[4705] - Attempting to fix initialization errors on newer platforms (Boost V1.63)
- PR #2449[4706] - Replace BOOST_COMPILER_FENCE with an HPX version
- PR #2448[4707] - This fixes a possible race in the migration code

•**PR #2445**[?] **- Fixing dataflow et.al. for futures or future-ranges wrapped**
into ref()

- PR #2444[4709] - Fix segfaults
- PR #2443[4710] - Issue 2442
- Issue #2442[4711] - Mismatch between #if/#endif and namespace scope brackets in this_thread_executers.hpp
- Issue #2441[4712] - undeclared identifier BOOST_COMPILER_FENCE
- PR #2440[4713] - Knl build
- PR #2438[4714] - Datapar backend
- PR #2437[4715] - Adapt algorithm parameter sequence changes from C++17
- PR #2436[4716] - Adapt execution policy name changes from C++17
- Issue #2435[4717] - Trunk broken, undefined reference to hpx::thread::interrupt(hpx::thread::id, bool)
- PR #2434[4718] - More fixes to resource manager
- PR #2433[4719] - Added versions of `hpx::get_ptr` taking client side representations
- PR #2432[4720] - Warning fixes
- PR #2431[4721] - Adding facility representing set of performance counters

4700 https://github.com/TheHPXProject/hpx/pull/2460
4701 https://github.com/TheHPXProject/hpx/pull/2459
4702 https://github.com/TheHPXProject/hpx/issues/2457
4703 https://github.com/TheHPXProject/hpx/pull/2452
4704 https://github.com/TheHPXProject/hpx/pull/2451
4705 https://github.com/TheHPXProject/hpx/pull/2450
4706 https://github.com/TheHPXProject/hpx/pull/2449
4707 https://github.com/TheHPXProject/hpx/pull/2448
4708 https://github.com/TheHPXProject/hpx/pull/2445
4709 https://github.com/TheHPXProject/hpx/pull/2444
4710 https://github.com/TheHPXProject/hpx/pull/2443
4711 https://github.com/TheHPXProject/hpx/issues/2442
4712 https://github.com/TheHPXProject/hpx/issues/2441
4713 https://github.com/TheHPXProject/hpx/pull/2440
4714 https://github.com/TheHPXProject/hpx/pull/2438
4715 https://github.com/TheHPXProject/hpx/pull/2437
4716 https://github.com/TheHPXProject/hpx/pull/2436
4717 https://github.com/TheHPXProject/hpx/issues/2435
4718 https://github.com/TheHPXProject/hpx/pull/2434
4719 https://github.com/TheHPXProject/hpx/pull/2433
4720 https://github.com/TheHPXProject/hpx/pull/2432
4721 https://github.com/TheHPXProject/hpx/pull/2431

- PR #2430[4722] - Fix parallel_executor thread spawning
- PR #2429[4723] - Fix attribute warning for gcc
- Issue #2427[4724] - Seg fault running octo-tiger with latest HPX commit
- Issue #2426[4725] - Bug in 9592f5c0bc29806fce0dbe73f35b6ca7e027edcb causes immediate crash in Octo-tiger
- PR #2425[4726] - Fix nvcc errors due to constexpr specifier
- Issue #2424[4727] - Async action on component present on hpx::find_here is executing synchronously
- PR #2423[4728] - Fix nvcc errors due to constexpr specifier
- PR #2422[4729] - Implementing hpx::this_thread thread data functions
- PR #2421[4730] - Adding benchmark for wait_all
- Issue #2420[4731] - Returning object of a component client from another component action fails
- PR #2419[4732] - Infiniband parcelport
- Issue #2418[4733] - gcc + nvcc fails to compile code that uses partitioned_vector
- PR #2417[4734] - Fixing context switching
- PR #2416[4735] - Adding fixes and workarounds to allow compilation with nvcc/msvc (VS2015up3)
- PR #2415[4736] - Fix errors coming from hpx compute examples
- PR #2414[4737] - Fixing msvc12
- PR #2413[4738] - Enable cuda/nvcc or cuda/clang when using add_hpx_executable()
- PR #2412[4739] - Fix issue in HPX_SetupTarget.cmake when cuda is used
- PR #2411[4740] - This fixes the core compilation issues with MSVC12
- Issue #2410[4741] - `undefined reference to opal_hwloc191_hwloc_.....`
- PR #2409[4742] - Fixing locking for channel and receive_buffer

[4722] https://github.com/TheHPXProject/hpx/pull/2430
[4723] https://github.com/TheHPXProject/hpx/pull/2429
[4724] https://github.com/TheHPXProject/hpx/issues/2427
[4725] https://github.com/TheHPXProject/hpx/issues/2426
[4726] https://github.com/TheHPXProject/hpx/pull/2425
[4727] https://github.com/TheHPXProject/hpx/issues/2424
[4728] https://github.com/TheHPXProject/hpx/pull/2423
[4729] https://github.com/TheHPXProject/hpx/pull/2422
[4730] https://github.com/TheHPXProject/hpx/pull/2421
[4731] https://github.com/TheHPXProject/hpx/issues/2420
[4732] https://github.com/TheHPXProject/hpx/pull/2419
[4733] https://github.com/TheHPXProject/hpx/issues/2418
[4734] https://github.com/TheHPXProject/hpx/pull/2417
[4735] https://github.com/TheHPXProject/hpx/pull/2416
[4736] https://github.com/TheHPXProject/hpx/pull/2415
[4737] https://github.com/TheHPXProject/hpx/pull/2414
[4738] https://github.com/TheHPXProject/hpx/pull/2413
[4739] https://github.com/TheHPXProject/hpx/pull/2412
[4740] https://github.com/TheHPXProject/hpx/pull/2411
[4741] https://github.com/TheHPXProject/hpx/issues/2410
[4742] https://github.com/TheHPXProject/hpx/pull/2409

- PR #2407[4743] - Solving #2402 and #2403
- PR #2406[4744] - Improve guards
- PR #2405[4745] - Enable parallel::for_each for iterators returning proxy types
- PR #2404[4746] - Forward the explicitly given result_type in the hpx invoke
- Issue #2403[4747] - datapar_execution + zip iterator: lambda arguments aren't references
- Issue #2402[4748] - datapar algorithm instantiated with wrong type #2402
- PR #2401[4749] - Added support for imported libraries to HPX_Libraries.cmake
- PR #2400[4750] - Use CMake policy CMP0060
- Issue #2399[4751] - Error trying to push back vector of futures to vector
- PR #2398[4752] - Allow config #defines to be written out to custom config/defines.hpp
- Issue #2397[4753] - CMake generated config defines can cause tedious rebuilds category
- Issue #2396[4754] - BOOST_ROOT paths are not used at link time
- PR #2395[4755] - Fix target_link_libraries() issue when HPX Cuda is enabled
- Issue #2394[4756] - Template compilation error using HPX_WITH_DATAPAR_LIBFLATARRAY
- PR #2393[4757] - Fixing lock registration for recursive mutex
- PR #2392[4758] - Add keywords in target_link_libraries in hpx_setup_target
- PR #2391[4759] - Clang goroutines
- Issue #2390[4760] - Adapt execution policy name changes from C++17
- PR #2389[4761] - Chunk allocator and pool are not used and are obsolete
- PR #2388[4762] - Adding functionalities to datapar needed by octotiger
- PR #2387[4763] - Fixing race condition for early parcels
- Issue #2386[4764] - Lock registration broken for recursive_mutex
- PR #2385[4765] - Datapar zip iterator

4743 https://github.com/TheHPXProject/hpx/pull/2407
4744 https://github.com/TheHPXProject/hpx/pull/2406
4745 https://github.com/TheHPXProject/hpx/pull/2405
4746 https://github.com/TheHPXProject/hpx/pull/2404
4747 https://github.com/TheHPXProject/hpx/issues/2403
4748 https://github.com/TheHPXProject/hpx/issues/2402
4749 https://github.com/TheHPXProject/hpx/pull/2401
4750 https://github.com/TheHPXProject/hpx/pull/2400
4751 https://github.com/TheHPXProject/hpx/issues/2399
4752 https://github.com/TheHPXProject/hpx/pull/2398
4753 https://github.com/TheHPXProject/hpx/issues/2397
4754 https://github.com/TheHPXProject/hpx/issues/2396
4755 https://github.com/TheHPXProject/hpx/pull/2395
4756 https://github.com/TheHPXProject/hpx/issues/2394
4757 https://github.com/TheHPXProject/hpx/pull/2393
4758 https://github.com/TheHPXProject/hpx/pull/2392
4759 https://github.com/TheHPXProject/hpx/pull/2391
4760 https://github.com/TheHPXProject/hpx/issues/2390
4761 https://github.com/TheHPXProject/hpx/pull/2389
4762 https://github.com/TheHPXProject/hpx/pull/2388
4763 https://github.com/TheHPXProject/hpx/pull/2387
4764 https://github.com/TheHPXProject/hpx/issues/2386
4765 https://github.com/TheHPXProject/hpx/pull/2385

- PR #2384[4766] - Fixing race condition in for_loop_reduction
- PR #2383[4767] - Continuations
- PR #2382[4768] - add LibFlatArray-based backend for datapar
- PR #2381[4769] - remove unused typedef to get rid of compiler warnings
- PR #2380[4770] - Tau cleanup
- PR #2379[4771] - Can send immediate
- PR #2378[4772] - Renaming copy_helper/copy_n_helper/move_helper/move_n_helper
- Issue #2376[4773] - Boost trunk's spinlock initializer fails to compile
- PR #2375[4774] - Add support for minimal thread local data
- PR #2374[4775] - Adding API functions set_config_entry_callback
- PR #2373[4776] - Add a simple utility for debugging that gives suspended task backtraces
- PR #2372[4777] - Barrier Fixes
- Issue #2370[4778] - Can't wait on a wrapped future
- PR #2369[4779] - Fixing stable_partition
- PR #2367[4780] - Fixing find_prefixes for Windows platforms
- PR #2366[4781] - Testing for experimental/optional only in C++14 mode
- PR #2364[4782] - Adding set_config_entry
- PR #2363[4783] - Fix papi
- PR #2362[4784] - Adding missing macros for new non-direct actions
- PR #2361[4785] - Improve cmake output to help debug compiler incompatibility check
- PR #2360[4786] - Fixing race condition in condition_variable
- PR #2359[4787] - Fixing shutdown when parcels are still in flight
- Issue #2357[4788] - failed to insert console_print_action into typename_to_id_t registry

[4766] https://github.com/TheHPXProject/hpx/pull/2384
[4767] https://github.com/TheHPXProject/hpx/pull/2383
[4768] https://github.com/TheHPXProject/hpx/pull/2382
[4769] https://github.com/TheHPXProject/hpx/pull/2381
[4770] https://github.com/TheHPXProject/hpx/pull/2380
[4771] https://github.com/TheHPXProject/hpx/pull/2379
[4772] https://github.com/TheHPXProject/hpx/pull/2378
[4773] https://github.com/TheHPXProject/hpx/issues/2376
[4774] https://github.com/TheHPXProject/hpx/pull/2375
[4775] https://github.com/TheHPXProject/hpx/pull/2374
[4776] https://github.com/TheHPXProject/hpx/pull/2373
[4777] https://github.com/TheHPXProject/hpx/pull/2372
[4778] https://github.com/TheHPXProject/hpx/issues/2370
[4779] https://github.com/TheHPXProject/hpx/pull/2369
[4780] https://github.com/TheHPXProject/hpx/pull/2367
[4781] https://github.com/TheHPXProject/hpx/pull/2366
[4782] https://github.com/TheHPXProject/hpx/pull/2364
[4783] https://github.com/TheHPXProject/hpx/pull/2363
[4784] https://github.com/TheHPXProject/hpx/pull/2362
[4785] https://github.com/TheHPXProject/hpx/pull/2361
[4786] https://github.com/TheHPXProject/hpx/pull/2360
[4787] https://github.com/TheHPXProject/hpx/pull/2359
[4788] https://github.com/TheHPXProject/hpx/issues/2357

- PR #2356[4789] - Fixing return type of get_iterator_tuple
- PR #2355[4790] - Fixing compilation against Boost 1 62
- PR #2354[4791] - Adding serialization for mask_type if CPU_COUNT > 64
- PR #2353[4792] - Adding hooks to tie in APEX into the parcel layer
- Issue #2352[4793] - Compile errors when using intel 17 beta (for KNL) on edison
- PR #2351[4794] - Fix function vtable get_function_address implementation
- Issue #2350[4795] - Build failure - master branch (4de09f5) with Intel Compiler v17
- PR #2349[4796] - Enabling zero-copy serialization support for std::vector<>
- PR #2348[4797] - Adding test to verify #2334 is fixed
- PR #2347[4798] - Bug fixes for hpx.compute and hpx::lcos::channel
- PR #2346[4799] - Removing cmake "find" files that are in the APEX cmake Modules
- PR #2345[4800] - Implemented parallel::stable_partition
- PR #2344[4801] - Making hpx::lcos::channel usable with basename registration
- PR #2343[4802] - Fix a couple of examples that failed to compile after recent api changes
- Issue #2342[4803] - Enabling APEX causes link errors
- PR #2341[4804] - Removing cmake "find" files that are in the APEX cmake Modules
- PR #2340[4805] - Implemented all existing datapar algorithms using Boost.SIMD
- PR #2339[4806] - Fixing 2338
- PR #2338[4807] - Possible race in sliding semaphore
- PR #2337[4808] - Adjust osu_latency test to measure window_size parcels in flight at once
- PR #2336[4809] - Allowing remote direct actions to be executed without spawning a task
- PR #2335[4810] - Making sure multiple components are properly initialized from arguments
- Issue #2334[4811] - Cannot construct component with large vector on a remote locality

[4789] https://github.com/TheHPXProject/hpx/pull/2356
[4790] https://github.com/TheHPXProject/hpx/pull/2355
[4791] https://github.com/TheHPXProject/hpx/pull/2354
[4792] https://github.com/TheHPXProject/hpx/pull/2353
[4793] https://github.com/TheHPXProject/hpx/issues/2352
[4794] https://github.com/TheHPXProject/hpx/pull/2351
[4795] https://github.com/TheHPXProject/hpx/issues/2350
[4796] https://github.com/TheHPXProject/hpx/pull/2349
[4797] https://github.com/TheHPXProject/hpx/pull/2348
[4798] https://github.com/TheHPXProject/hpx/pull/2347
[4799] https://github.com/TheHPXProject/hpx/pull/2346
[4800] https://github.com/TheHPXProject/hpx/pull/2345
[4801] https://github.com/TheHPXProject/hpx/pull/2344
[4802] https://github.com/TheHPXProject/hpx/pull/2343
[4803] https://github.com/TheHPXProject/hpx/issues/2342
[4804] https://github.com/TheHPXProject/hpx/pull/2341
[4805] https://github.com/TheHPXProject/hpx/pull/2340
[4806] https://github.com/TheHPXProject/hpx/pull/2339
[4807] https://github.com/TheHPXProject/hpx/pull/2338
[4808] https://github.com/TheHPXProject/hpx/pull/2337
[4809] https://github.com/TheHPXProject/hpx/pull/2336
[4810] https://github.com/TheHPXProject/hpx/pull/2335
[4811] https://github.com/TheHPXProject/hpx/issues/2334

- PR #2332[4812] - Fixing hpx::lcos::local::barrier
- PR #2331[4813] - Updating APEX support to include OTF2
- PR #2330[4814] - Support for data-parallelism for parallel algorithms
- Issue #2329[4815] - Coordinate settings in cmake
- PR #2328[4816] - fix LibGeoDecomp builds with HPX + GCC 5.3.0 + CUDA 8RC
- PR #2326[4817] - Making scan_partitioner work (for now)
- Issue #2323[4818] - Constructing a vector of components only correctly initializes the first component
- PR #2322[4819] - Fix problems that bubbled up after merging #2278
- PR #2321[4820] - Scalable barrier
- PR #2320[4821] - Std flag fixes
- Issue #2319[4822] - -std=c++14 and -std=c++1y with Intel can't build recent Boost builds due to insufficient C++14 support; don't enable these flags by default for Intel
- PR #2318[4823] - Improve handling of –hpx:bind=<bind-spec>
- PR #2317[4824] - Making sure command line warnings are printed once only
- PR #2316[4825] - Fixing command line handling for default bind mode
- PR #2315[4826] - Set id_retrieved if set_id is present
- Issue #2314[4827] - Warning for requested/allocated thread discrepancy is printed twice
- Issue #2313[4828] - –hpx:print-bind doesn't work with –hpx:pu-step
- Issue #2312[4829] - –hpx:bind range specifier restrictions are overly restrictive
- Issue #2311[4830] - hpx_0.9.99 out of project build fails
- PR #2310[4831] - Simplify function registration
- PR #2309[4832] - Spelling and grammar revisions in documentation (and some code)
- PR #2306[4833] - Correct minor typo in the documentation

[4812] https://github.com/TheHPXProject/hpx/pull/2332
[4813] https://github.com/TheHPXProject/hpx/pull/2331
[4814] https://github.com/TheHPXProject/hpx/pull/2330
[4815] https://github.com/TheHPXProject/hpx/issues/2329
[4816] https://github.com/TheHPXProject/hpx/pull/2328
[4817] https://github.com/TheHPXProject/hpx/pull/2326
[4818] https://github.com/TheHPXProject/hpx/issues/2323
[4819] https://github.com/TheHPXProject/hpx/pull/2322
[4820] https://github.com/TheHPXProject/hpx/pull/2321
[4821] https://github.com/TheHPXProject/hpx/pull/2320
[4822] https://github.com/TheHPXProject/hpx/issues/2319
[4823] https://github.com/TheHPXProject/hpx/pull/2318
[4824] https://github.com/TheHPXProject/hpx/pull/2317
[4825] https://github.com/TheHPXProject/hpx/pull/2316
[4826] https://github.com/TheHPXProject/hpx/pull/2315
[4827] https://github.com/TheHPXProject/hpx/issues/2314
[4828] https://github.com/TheHPXProject/hpx/issues/2313
[4829] https://github.com/TheHPXProject/hpx/issues/2312
[4830] https://github.com/TheHPXProject/hpx/issues/2311
[4831] https://github.com/TheHPXProject/hpx/pull/2310
[4832] https://github.com/TheHPXProject/hpx/pull/2309
[4833] https://github.com/TheHPXProject/hpx/pull/2306

- PR #2305[4834] - Cleaning up and fixing parcel coalescing
- PR #2304[4835] - Inspect checks for stream related includes
- PR #2303[4836] - Add functionality allowing to enumerate threads of given state
- PR #2301[4837] - Algorithm overloads fix for VS2013
- PR #2300[4838] - Use <cstdint>, add inspect checks
- PR #2299[4839] - Replace boost::[c]ref with std::[c]ref, add inspect checks
- PR #2297[4840] - Fixing compilation with no hw_loc
- PR #2296[4841] - Hpx compute
- PR #2295[4842] - Making sure for_loop(execution::par, 0, N, …) is actually executed in parallel
- PR #2294[4843] - Throwing exceptions if the runtime is not up and running
- PR #2293[4844] - Removing unused parcel port code
- PR #2292[4845] - Refactor function vtables
- PR #2291[4846] - Fixing 2286
- PR #2290[4847] - Simplify algorithm overloads
- PR #2289[4848] - Adding performance counters reporting parcel related data on a per-action basis
- Issue #2288[4849] - Remove dormant parcelports
- Issue #2286[4850] - adjustments to parcel handling to support parcelports that do not need a connection cache
- PR #2285[4851] - add CMake option to disable package export
- PR #2283[4852] - Add more inspect checks for use of deprecated components
- Issue #2282[4853] - Arithmetic exception in executor static chunker
- Issue #2281[4854] - For loop doesn't parallelize
- PR #2280[4855] - Fixing 2277: build failure with PAPI

[4834] https://github.com/TheHPXProject/hpx/pull/2305
[4835] https://github.com/TheHPXProject/hpx/pull/2304
[4836] https://github.com/TheHPXProject/hpx/pull/2303
[4837] https://github.com/TheHPXProject/hpx/pull/2301
[4838] https://github.com/TheHPXProject/hpx/pull/2300
[4839] https://github.com/TheHPXProject/hpx/pull/2299
[4840] https://github.com/TheHPXProject/hpx/pull/2297
[4841] https://github.com/TheHPXProject/hpx/pull/2296
[4842] https://github.com/TheHPXProject/hpx/pull/2295
[4843] https://github.com/TheHPXProject/hpx/pull/2294
[4844] https://github.com/TheHPXProject/hpx/pull/2293
[4845] https://github.com/TheHPXProject/hpx/pull/2292
[4846] https://github.com/TheHPXProject/hpx/pull/2291
[4847] https://github.com/TheHPXProject/hpx/pull/2290
[4848] https://github.com/TheHPXProject/hpx/pull/2289
[4849] https://github.com/TheHPXProject/hpx/issues/2288
[4850] https://github.com/TheHPXProject/hpx/issues/2286
[4851] https://github.com/TheHPXProject/hpx/pull/2285
[4852] https://github.com/TheHPXProject/hpx/pull/2283
[4853] https://github.com/TheHPXProject/hpx/issues/2282
[4854] https://github.com/TheHPXProject/hpx/issues/2281
[4855] https://github.com/TheHPXProject/hpx/pull/2280

- PR #2279[4856] - Child vs parent stealing
- Issue #2277[4857] - master branch build failure (53c5b4f) with papi
- PR #2276[4858] - Compile time launch policies
- PR #2275[4859] - Replace boost::chrono with std::chrono in interfaces
- PR #2274[4860] - Replace most uses of Boost.Assign with initializer list
- PR #2273[4861] - Fixed typos
- PR #2272[4862] - Inspect checks
- PR #2270[4863] - Adding test verifying -Ihpx.os_threads=all
- PR #2269[4864] - Added inspect check for now obsolete boost type traits
- PR #2268[4865] - Moving more code into source files
- Issue #2267[4866] - Add inspect support to deprecate Boost.TypeTraits
- PR #2265[4867] - Adding channel LCO
- PR #2264[4868] - Make support for std::ref mandatory
- PR #2263[4869] - Constrain tuple_member forwarding constructor
- Issue #2262[4870] - Test hpx.os_threads=all
- Issue #2261[4871] - OS X: Error: no matching constructor for initialization of 'hpx::lcos::local::condition_variable_any'
- Issue #2260[4872] - Make support for std::ref mandatory
- PR #2259[4873] - Remove most of Boost.MPL, Boost.EnableIf and Boost.TypeTraits
- PR #2258[4874] - Fixing #2256
- PR #2257[4875] - Fixing launch process
- Issue #2256[4876] - Actions are not registered if not invoked
- PR #2255[4877] - Coalescing histogram
- PR #2254[4878] - Silence explicit initialization in copy-constructor warnings

4856 https://github.com/TheHPXProject/hpx/pull/2279
4857 https://github.com/TheHPXProject/hpx/issues/2277
4858 https://github.com/TheHPXProject/hpx/pull/2276
4859 https://github.com/TheHPXProject/hpx/pull/2275
4860 https://github.com/TheHPXProject/hpx/pull/2274
4861 https://github.com/TheHPXProject/hpx/pull/2273
4862 https://github.com/TheHPXProject/hpx/pull/2272
4863 https://github.com/TheHPXProject/hpx/pull/2270
4864 https://github.com/TheHPXProject/hpx/pull/2269
4865 https://github.com/TheHPXProject/hpx/pull/2268
4866 https://github.com/TheHPXProject/hpx/issues/2267
4867 https://github.com/TheHPXProject/hpx/pull/2265
4868 https://github.com/TheHPXProject/hpx/pull/2264
4869 https://github.com/TheHPXProject/hpx/pull/2263
4870 https://github.com/TheHPXProject/hpx/issues/2262
4871 https://github.com/TheHPXProject/hpx/issues/2261
4872 https://github.com/TheHPXProject/hpx/issues/2260
4873 https://github.com/TheHPXProject/hpx/pull/2259
4874 https://github.com/TheHPXProject/hpx/pull/2258
4875 https://github.com/TheHPXProject/hpx/pull/2257
4876 https://github.com/TheHPXProject/hpx/issues/2256
4877 https://github.com/TheHPXProject/hpx/pull/2255
4878 https://github.com/TheHPXProject/hpx/pull/2254

- PR #2253[4879] - Drop support for GCC 4.6 and 4.7
- PR #2252[4880] - Prepare V1.0
- PR #2251[4881] - Convert to 0.9.99
- PR #2249[4882] - Adding iterator_facade and iterator_adaptor
- Issue #2248[4883] - Need a feature to yield to a new task immediately
- PR #2246[4884] - Adding split_future
- PR #2245[4885] - Add an example for handing over a component instance to a dynamically launched locality
- Issue #2243[4886] - Add example demonstrating AGAS symbolic name registration
- Issue #2242[4887] - pkgconfig test broken on CentOS 7 / Boost 1.61
- Issue #2241[4888] - Compilation error for partitioned vector in hpx_compute branch
- PR #2240[4889] - Fixing termination detection on one locality
- Issue #2239[4890] - Create a new facility lcos::split_all
- Issue #2236[4891] - hpx::cout vs. std::cout
- PR #2232[4892] - Implement local-only primary namespace service
- Issue #2147[4893] - would like to know how much data is being routed by particular actions
- Issue #2109[4894] - Warning while compiling hpx
- Issue #1973[4895] - Setting INTERFACE_COMPILE_OPTIONS for hpx_init in CMake taints Fortran_FLAGS
- Issue #1864[4896] - run_guarded using bound function ignores reference
- Issue #1754[4897] - Running with TCP parcelport causes immediate crash or freeze
- Issue #1655[4898] - Enable zip_iterator to be used with Boost traversal iterator categories
- Issue #1591[4899] - Optimize AGAS for shared memory only operation
- Issue #1401[4900] - Need an efficient infiniband parcelport

[4879] https://github.com/TheHPXProject/hpx/pull/2253
[4880] https://github.com/TheHPXProject/hpx/pull/2252
[4881] https://github.com/TheHPXProject/hpx/pull/2251
[4882] https://github.com/TheHPXProject/hpx/pull/2249
[4883] https://github.com/TheHPXProject/hpx/issues/2248
[4884] https://github.com/TheHPXProject/hpx/pull/2246
[4885] https://github.com/TheHPXProject/hpx/pull/2245
[4886] https://github.com/TheHPXProject/hpx/issues/2243
[4887] https://github.com/TheHPXProject/hpx/issues/2242
[4888] https://github.com/TheHPXProject/hpx/issues/2241
[4889] https://github.com/TheHPXProject/hpx/pull/2240
[4890] https://github.com/TheHPXProject/hpx/issues/2239
[4891] https://github.com/TheHPXProject/hpx/issues/2236
[4892] https://github.com/TheHPXProject/hpx/pull/2232
[4893] https://github.com/TheHPXProject/hpx/issues/2147
[4894] https://github.com/TheHPXProject/hpx/issues/2109
[4895] https://github.com/TheHPXProject/hpx/issues/1973
[4896] https://github.com/TheHPXProject/hpx/issues/1864
[4897] https://github.com/TheHPXProject/hpx/issues/1754
[4898] https://github.com/TheHPXProject/hpx/issues/1655
[4899] https://github.com/TheHPXProject/hpx/issues/1591
[4900] https://github.com/TheHPXProject/hpx/issues/1401

- Issue #1125[4901] - Fix the IPC parcelport
- Issue #839[4902] - Refactor ibverbs and shmem parcelport
- Issue #702[4903] - Add instrumentation of parcel layer
- Issue #668[4904] - Implement ispc task interface
- Issue #533[4905] - Thread queue/deque internal parameters should be runtime configurable
- Issue #475[4906] - Create a means of combining performance counters into querysets

HPX V0.9.99 (Jul 15, 2016)

General changes

As the version number of this release hints, we consider this release to be a preview for the upcoming *HPX* V1.0. All of the functionalities we set out to implement for V1.0 are in place; all of the features we wanted to have exposed are ready. We are very happy with the stability and performance of *HPX* and we would like to present this release to the community in order for us to gather broad feedback before releasing V1.0. We still expect for some minor details to change, but on the whole this release represents what we would like to have in a V1.0.

Overall, since the last release we have had almost 1600 commits while closing almost 400 tickets. These numbers reflect the incredible development activity we have seen over the last couple of months. We would like to express a big 'Thank you!' to all contributors and those who helped to make this release happen.

The most notable addition in terms of new functionality available with this release is the full implementation of object migration (i.e. the ability to transparently move *HPX* components to a different compute node). Additionally, this release of *HPX* cleans up many minor issues and some API inconsistencies.

Here are some of the main highlights and changes for this release (in no particular order):

- We have fixed a couple of issues in AGAS and the parcel layer which have caused hangs, segmentation faults at exit, and a slowdown of applications over time. Fixing those has significantly increased the overall stability and performance of distributed runs.
- We have started to add parallel algorithm overloads based on the C++ Extensions for Ranges (N4560[4907]) proposal. This also includes the addition of projections to the existing algorithms. Please see Issue #1668[4908] for a list of algorithms which have been adapted to N4560[4909].
- We have implemented index-based parallel for-loops based on a corresponding standardization proposal (P0075R1[4910]). Please see Issue #2016[4911] for a list of available algorithms.

[4901] https://github.com/TheHPXProject/hpx/issues/1125
[4902] https://github.com/TheHPXProject/hpx/issues/839
[4903] https://github.com/TheHPXProject/hpx/issues/702
[4904] https://github.com/TheHPXProject/hpx/issues/668
[4905] https://github.com/TheHPXProject/hpx/issues/533
[4906] https://github.com/TheHPXProject/hpx/issues/475
[4907] http://wg21.link/n4560
[4908] https://github.com/TheHPXProject/hpx/issues/1668
[4909] http://wg21.link/n4560
[4910] http://wg21.link/p0075r1
[4911] https://github.com/TheHPXProject/hpx/issues/2016

- We have added implementations for more parallel algorithms as proposed for the upcoming C++ 17 Standard. See Issue #1141[4912] for an overview of which algorithms are available by now.
- We have started to implement a new prototypical functionality with *HPX.Compute* which uniformly exposes some of the higher level APIs to heterogeneous architectures (currently CUDA). This functionality is an early preview and should not be considered stable. It may change considerably in the future.
- We have pervasively added (optional) executor arguments to all API functions which schedule new work. Executors are now used throughout the code base as the main means of executing tasks.
- Added `hpx::make_future<R>(future<T> &&)` allowing to convert a future of any type `T` into a future of any other type `R`, either based on default conversion rules of the embedded types or using a given explicit conversion function.
- We finally finished the implementation of transparent migration of components to another locality. It is now possible to trigger a migration operation without 'stopping the world' for the object to migrate. *HPX* will make sure that no work is being performed on an object before it is migrated and that all subsequently scheduled work for the migrated object will be transparently forwarded to the new locality. Please note that the global id of the migrated object does not change, thus the application will not have to be changed in any way to support this new functionality. Please note that this feature is currently considered experimental. See Issue #559[4913] and PR #1966[4914] for more details.
- The `hpx::dataflow` facility is now usable with actions. Similarly to `hpx::async`, actions can be specified as an explicit template argument (`hpx::dataflow<Action>(target, ...)`) or as the first argument (`hpx::dataflow(Action(), target, ...)`). We have also enabled the use of distribution policies as the target for dataflow invocations. Please see Issue #1265[4915] and PR #1912[4916] for more information.
- Adding overloads of `gather_here` and `gather_there` to accept the plain values of the data to gather (in addition to the existing overloads expecting futures).
- We have cleaned up and refactored large parts of the code base. This helped reducing compile and link times of *HPX* itself and also of applications depending on it. We have further decreased the dependency of *HPX* on the Boost libraries by replacing part of those with facilities available from the standard libraries.
- Wherever possible we have removed dependencies of our API on Boost by replacing those with the equivalent facility from the C++11 standard library.
- We have added new performance counters for parcel coalescing, file-IO, the AGAS cache, and overall scheduler time. Resetting performance counters has been overhauled and fixed.
- We have introduced a generic client type `hpx::components::client<>` and added support for using it with `hpx::async`. This removes the necessity to implement specific client types for every component type without losing type safety. This deemphasizes the need for using the low level `hpx::id_type` for referencing (possibly remote) component instances. The plan is to deprecate the direct use of `hpx::id_type` in user code in the future.

[4912] https://github.com/TheHPXProject/hpx/issues/1141
[4913] https://github.com/TheHPXProject/hpx/issues/559
[4914] https://github.com/TheHPXProject/hpx/pull/1966
[4915] https://github.com/TheHPXProject/hpx/issues/1265
[4916] https://github.com/TheHPXProject/hpx/pull/1912

- We have added a special iterator which supports automatic prefetching of one or more arrays for speeding up loop-like code (see `hpx::parallel::util::make_prefetcher_context()`).
- We have extended the interfaces exposed from executors (as proposed by N4406[4917]) to accept an arbitrary number of arguments.

Breaking changes

- In order to move the dataflow facility to `namespace hpx` we added a definition of `hpx::dataflow` which might create ambiguities in existing codes. The previous definition of this facility (`hpx::lcos::local::dataflow`) has been deprecated and is available only if the constant `-DHPX_WITH_LOCAL_DATAFLOW_COMPATIBILITY=On` to CMake[4918] is defined at configuration time. Please explicitly qualify all uses of the dataflow facility if you enable this compatibility setting and encounter ambiguities.
- The adaptation of the C++ Extensions for Ranges (N4560[4919]) proposal imposes some breaking changes related to the return types of some of the parallel algorithms. Please see Issue #1668[4920] for a list of algorithms which have already been adapted.
- The facility `hpx::lcos::make_future_void()` has been replaced by `hpx::make_future<void>()`.
- We have removed support for Intel V13 and gcc 4.4.x.
- We have removed (default) support for the generic `hpx::parallel::execution_poliy` because it was removed from the Parallelism TS (__cpp11_n4104__) while it was being added to the upcoming C++17 Standard. This facility can be still enabled at configure time by specifying `-DHPX_WITH_GENERIC_EXECUTION_POLICY=On` to CMake.
- Uses of `boost::shared_ptr` and related facilities have been replaced with `std::shared_ptr` and friends. Uses of `boost::unique_lock`, `boost::lock_guard` etc. have also been replaced by the equivalent (and equally named) tools available from the C++11 standard library.
- Facilities that used to expect an explicit `boost::unique_lock` now take an `std::unique_lock`. Additionally, `condition_variable` no longer aliases `condition_variable_any`; its interface now only works with `std::unique_lock<local::mutex>`.
- Uses of `boost::function`, `boost::bind`, `boost::tuple` have been replaced by the corresponding facilities in *HPX* (`hpx::util::function`, `hpx::util::bind`, and `hpx::util::tuple`, respectively).

[4917] http://wg21.link/n4406
[4918] https://www.cmake.org
[4919] http://wg21.link/n4560
[4920] https://github.com/TheHPXProject/hpx/issues/1668

Bug fixes (closed tickets)

Here is a list of the important tickets we closed for this release.

- PR #2250[4921] - change default chunker of parallel executor to static one
- PR #2247[4922] - HPX on ppc64le
- PR #2244[4923] - Fixing MSVC problems
- PR #2238[4924] - Fixing small typos
- PR #2237[4925] - Fixing small typos
- PR #2234[4926] - Fix broken add test macro when extra args are passed in
- PR #2231[4927] - Fixing possible race during future awaiting in serialization
- PR #2230[4928] - Fix stream nvcc
- PR #2229[4929] - Fixed run_as_hpx_thread
- PR #2228[4930] - On prefetching_test branch : adding prefetching_iterator and related tests used for prefetching containers within lambda functions
- PR #2227[4931] - Support for HPXCL's opencl::event
- PR #2226[4932] - Preparing for release of V0.9.99
- PR #2225[4933] - fix issue when compiling components with hpxcxx
- PR #2224[4934] - Compute alloc fix
- PR #2223[4935] - Simplify promise
- PR #2222[4936] - Replace last uses of boost::function by util::function_nonser
- PR #2221[4937] - Fix config tests
- PR #2220[4938] - Fixing gcc 4.6 compilation issues
- PR #2219[4939] - nullptr support for `[unique_]function`
- PR #2218[4940] - Introducing clang tidy
- PR #2216[4941] - Replace NULL with nullptr

[4921] https://github.com/TheHPXProject/hpx/pull/2250
[4922] https://github.com/TheHPXProject/hpx/pull/2247
[4923] https://github.com/TheHPXProject/hpx/pull/2244
[4924] https://github.com/TheHPXProject/hpx/pull/2238
[4925] https://github.com/TheHPXProject/hpx/pull/2237
[4926] https://github.com/TheHPXProject/hpx/pull/2234
[4927] https://github.com/TheHPXProject/hpx/pull/2231
[4928] https://github.com/TheHPXProject/hpx/pull/2230
[4929] https://github.com/TheHPXProject/hpx/pull/2229
[4930] https://github.com/TheHPXProject/hpx/pull/2228
[4931] https://github.com/TheHPXProject/hpx/pull/2227
[4932] https://github.com/TheHPXProject/hpx/pull/2226
[4933] https://github.com/TheHPXProject/hpx/pull/2225
[4934] https://github.com/TheHPXProject/hpx/pull/2224
[4935] https://github.com/TheHPXProject/hpx/pull/2223
[4936] https://github.com/TheHPXProject/hpx/pull/2222
[4937] https://github.com/TheHPXProject/hpx/pull/2221
[4938] https://github.com/TheHPXProject/hpx/pull/2220
[4939] https://github.com/TheHPXProject/hpx/pull/2219
[4940] https://github.com/TheHPXProject/hpx/pull/2218
[4941] https://github.com/TheHPXProject/hpx/pull/2216

- Issue #2214[4942] - Let inspect flag use of NULL, suggest nullptr instead
- PR #2213[4943] - Require support for nullptr
- PR #2212[4944] - Properly find jemalloc through pkg-config
- PR #2211[4945] - Disable a couple of warnings reported by Intel on Windows
- PR #2210[4946] - Fixed host::block_allocator::bulk_construct
- PR #2209[4947] - Started to clean up new sort algorithms, made things compile for sort_by_key
- PR #2208[4948] - A couple of fixes that were exposed by a new sort algorithm
- PR #2207[4949] - Adding missing includes in /hpx/include/serialization.hpp
- PR #2206[4950] - Call package_action::get_future before package_action::apply
- PR #2205[4951] - The indirect_packaged_task::operator() needs to be run on a HPX thread
- PR #2204[4952] - Variadic executor parameters
- PR #2203[4953] - Delay-initialize members of partitioned iterator
- PR #2202[4954] - Added segmented fill for hpx::vector
- Issue #2201[4955] - Null Thread id encountered on partitioned_vector
- PR #2200[4956] - Fix hangs
- PR #2199[4957] - Deprecating hpx/traits.hpp
- PR #2198[4958] - Making explicit inclusion of external libraries into build
- PR #2197[4959] - Fix typo in QT CMakeLists
- PR #2196[4960] - Fixing a gcc warning about attributes being ignored
- PR #2194[4961] - Fixing partitioned_vector_spmd_foreach example
- Issue #2193[4962] - partitioned_vector_spmd_foreach seg faults
- PR #2192[4963] - Support Boost.Thread v4
- PR #2191[4964] - HPX.Compute prototype

[4942] https://github.com/TheHPXProject/hpx/issues/2214
[4943] https://github.com/TheHPXProject/hpx/pull/2213
[4944] https://github.com/TheHPXProject/hpx/pull/2212
[4945] https://github.com/TheHPXProject/hpx/pull/2211
[4946] https://github.com/TheHPXProject/hpx/pull/2210
[4947] https://github.com/TheHPXProject/hpx/pull/2209
[4948] https://github.com/TheHPXProject/hpx/pull/2208
[4949] https://github.com/TheHPXProject/hpx/pull/2207
[4950] https://github.com/TheHPXProject/hpx/pull/2206
[4951] https://github.com/TheHPXProject/hpx/pull/2205
[4952] https://github.com/TheHPXProject/hpx/pull/2204
[4953] https://github.com/TheHPXProject/hpx/pull/2203
[4954] https://github.com/TheHPXProject/hpx/pull/2202
[4955] https://github.com/TheHPXProject/hpx/issues/2201
[4956] https://github.com/TheHPXProject/hpx/pull/2200
[4957] https://github.com/TheHPXProject/hpx/pull/2199
[4958] https://github.com/TheHPXProject/hpx/pull/2198
[4959] https://github.com/TheHPXProject/hpx/pull/2197
[4960] https://github.com/TheHPXProject/hpx/pull/2196
[4961] https://github.com/TheHPXProject/hpx/pull/2194
[4962] https://github.com/TheHPXProject/hpx/issues/2193
[4963] https://github.com/TheHPXProject/hpx/pull/2192
[4964] https://github.com/TheHPXProject/hpx/pull/2191

- PR #2190[4965] - Spawning operation on new thread if remaining stack space becomes too small
- PR #2189[4966] - Adding callback taking index and future to when_each
- PR #2188[4967] - Adding new example demonstrating receive_buffer
- PR #2187[4968] - Mask 128-bit ints if CUDA is being used
- PR #2186[4969] - Make startup & shutdown functions unique_function
- PR #2185[4970] - Fixing logging output not to cause hang on shutdown
- PR #2184[4971] - Allowing component clients as action return types
- Issue #2183[4972] - Enabling logging output causes hang on shutdown
- Issue #2182[4973] - 1d_stencil seg fault
- Issue #2181[4974] - Setting small stack size does not change default
- PR #2180[4975] - Changing default bind mode to balanced
- PR #2179[4976] - adding prefetching_iterator and related tests used for prefetching containers within lambda functions
- PR #2177[4977] - Fixing 2176
- Issue #2176[4978] - Launch process test fails on OSX
- PR #2175[4979] - Fix unbalanced config/warnings includes, add some new ones
- PR #2174[4980] - Fix test categorization : regression not unit
- Issue #2172[4981] - Different performance results
- Issue #2171[4982] - "negative entry in reference count table" running octotiger on 32 nodes on queenbee
- Issue #2170[4983] - Error while compiling on Mac + boost 1.60
- PR #2168[4984] - Fixing problems with is_bitwise_serializable
- Issue #2167[4985] - startup & shutdown function should accept unique_function
- Issue #2166[4986] - Simple receive_buffer example

[4965] https://github.com/TheHPXProject/hpx/pull/2190
[4966] https://github.com/TheHPXProject/hpx/pull/2189
[4967] https://github.com/TheHPXProject/hpx/pull/2188
[4968] https://github.com/TheHPXProject/hpx/pull/2187
[4969] https://github.com/TheHPXProject/hpx/pull/2186
[4970] https://github.com/TheHPXProject/hpx/pull/2185
[4971] https://github.com/TheHPXProject/hpx/pull/2184
[4972] https://github.com/TheHPXProject/hpx/issues/2183
[4973] https://github.com/TheHPXProject/hpx/issues/2182
[4974] https://github.com/TheHPXProject/hpx/issues/2181
[4975] https://github.com/TheHPXProject/hpx/pull/2180
[4976] https://github.com/TheHPXProject/hpx/pull/2179
[4977] https://github.com/TheHPXProject/hpx/pull/2177
[4978] https://github.com/TheHPXProject/hpx/issues/2176
[4979] https://github.com/TheHPXProject/hpx/pull/2175
[4980] https://github.com/TheHPXProject/hpx/pull/2174
[4981] https://github.com/TheHPXProject/hpx/issues/2172
[4982] https://github.com/TheHPXProject/hpx/issues/2171
[4983] https://github.com/TheHPXProject/hpx/issues/2170
[4984] https://github.com/TheHPXProject/hpx/pull/2168
[4985] https://github.com/TheHPXProject/hpx/issues/2167
[4986] https://github.com/TheHPXProject/hpx/issues/2166

- PR #2165[4987] - Fix wait all
- PR #2164[4988] - Fix wait all
- PR #2163[4989] - Fix some typos in config tests
- PR #2162[4990] - Improve #includes
- PR #2160[4991] - Add inspect check for missing #include <list>
- PR #2159[4992] - Add missing finalize call to stop test hanging
- PR #2158[4993] - Algo fixes
- PR #2157[4994] - Stack check
- Issue #2156[4995] - OSX reports stack space incorrectly (generic context coroutines)
- Issue #2155[4996] - Race condition suspected in runtime
- PR #2154[4997] - Replace boost::detail::atomic_count with the new util::atomic_count
- PR #2153[4998] - Fix stack overflow on OSX
- PR #2152[4999] - Define is_bitwise_serializable as is_trivially_copyable when available
- PR #2151[5000] - Adding missing <cstring> for std::mem* functions
- Issue #2150[5001] - Unable to use component clients as action return types
- PR #2149[5002] - std::memmove copies bytes, use bytes*sizeof(type) when copying larger types
- PR #2146[5003] - Adding customization point for parallel copy/move
- PR #2145[5004] - Applying changes to address warnings issued by latest version of PVS Studio
- Issue #2148[5005] - hpx::parallel::copy is broken after trivially copyable changes
- PR #2144[5006] - Some minor tweaks to compute prototype
- PR #2143[5007] - Added Boost version support information over OSX platform
- PR #2142[5008] - Fixing memory leak in example

4987 https://github.com/TheHPXProject/hpx/pull/2165
4988 https://github.com/TheHPXProject/hpx/pull/2164
4989 https://github.com/TheHPXProject/hpx/pull/2163
4990 https://github.com/TheHPXProject/hpx/pull/2162
4991 https://github.com/TheHPXProject/hpx/pull/2160
4992 https://github.com/TheHPXProject/hpx/pull/2159
4993 https://github.com/TheHPXProject/hpx/pull/2158
4994 https://github.com/TheHPXProject/hpx/pull/2157
4995 https://github.com/TheHPXProject/hpx/issues/2156
4996 https://github.com/TheHPXProject/hpx/issues/2155
4997 https://github.com/TheHPXProject/hpx/pull/2154
4998 https://github.com/TheHPXProject/hpx/pull/2153
4999 https://github.com/TheHPXProject/hpx/pull/2152
5000 https://github.com/TheHPXProject/hpx/pull/2151
5001 https://github.com/TheHPXProject/hpx/issues/2150
5002 https://github.com/TheHPXProject/hpx/pull/2149
5003 https://github.com/TheHPXProject/hpx/pull/2146
5004 https://github.com/TheHPXProject/hpx/pull/2145
5005 https://github.com/TheHPXProject/hpx/issues/2148
5006 https://github.com/TheHPXProject/hpx/pull/2144
5007 https://github.com/TheHPXProject/hpx/pull/2143
5008 https://github.com/TheHPXProject/hpx/pull/2142

- PR #2141[5009] - Add missing specializations in execution policies
- PR #2139[5010] - This PR fixes a few problems reported by Clang's Undefined Behavior sanitizer
- PR #2138[5011] - Revert "Adding fedora docs"
- PR #2136[5012] - Removed double semicolon
- PR #2135[5013] - Add deprecated #include check for hpx_fwd.hpp
- PR #2134[5014] - Resolved memory leak in stencil_8
- PR #2133[5015] - Replace uses of boost pointer containers
- PR #2132[5016] - Removing unused typedef
- PR #2131[5017] - Add several include checks for std facilities
- PR #2130[5018] - Fixing parcel compression, adding test
- PR #2129[5019] - Fix invalid attribute warnings
- Issue #2128[5020] - hpx::init seems to segfault
- PR #2127[5021] - Making executor_traits N-nary
- PR #2126[5022] - GCC 4.6 fails to deduce the correct type in lambda
- PR #2125[5023] - Making parcel coalescing test actually test something
- Issue #2124[5024] - Make a testcase for parcel compression
- Issue #2123[5025] - hpx/hpx/runtime/applier_fwd.hpp - Multiple defined types
- Issue #2122[5026] - Exception in primary_namespace::resolve_free_list
- Issue #2121[5027] - Possible memory leak in 1d_stencil_8
- PR #2120[5028] - Fixing 2119
- Issue #2119[5029] - reduce_by_key compilation problems
- Issue #2118[5030] - Premature unwrapping of boost::ref'ed arguments
- PR #2117[5031] - Added missing initializer on last constructor for thread_description

[5009] https://github.com/TheHPXProject/hpx/pull/2141
[5010] https://github.com/TheHPXProject/hpx/pull/2139
[5011] https://github.com/TheHPXProject/hpx/pull/2138
[5012] https://github.com/TheHPXProject/hpx/pull/2136
[5013] https://github.com/TheHPXProject/hpx/pull/2135
[5014] https://github.com/TheHPXProject/hpx/pull/2134
[5015] https://github.com/TheHPXProject/hpx/pull/2133
[5016] https://github.com/TheHPXProject/hpx/pull/2132
[5017] https://github.com/TheHPXProject/hpx/pull/2131
[5018] https://github.com/TheHPXProject/hpx/pull/2130
[5019] https://github.com/TheHPXProject/hpx/pull/2129
[5020] https://github.com/TheHPXProject/hpx/issues/2128
[5021] https://github.com/TheHPXProject/hpx/pull/2127
[5022] https://github.com/TheHPXProject/hpx/pull/2126
[5023] https://github.com/TheHPXProject/hpx/pull/2125
[5024] https://github.com/TheHPXProject/hpx/issues/2124
[5025] https://github.com/TheHPXProject/hpx/issues/2123
[5026] https://github.com/TheHPXProject/hpx/issues/2122
[5027] https://github.com/TheHPXProject/hpx/issues/2121
[5028] https://github.com/TheHPXProject/hpx/pull/2120
[5029] https://github.com/TheHPXProject/hpx/issues/2119
[5030] https://github.com/TheHPXProject/hpx/issues/2118
[5031] https://github.com/TheHPXProject/hpx/pull/2117

- PR #2116[5032] - Use a lightweight bind implementation when no placeholders are given
- PR #2115[5033] - Replace boost::shared_ptr with std::shared_ptr
- PR #2114[5034] - Adding hook functions for executor_parameter_traits supporting timers
- Issue #2113[5035] - Compilation error with gcc version 4.9.3 (MacPorts gcc49 4.9.3_0)
- PR #2112[5036] - Replace uses of safe_bool with explicit operator bool
- Issue #2111[5037] - Compilation error on QT example
- Issue #2110[5038] - Compilation error when passing non-future argument to unwrapped continuation in dataflow
- Issue #2109[5039] - Warning while compiling hpx
- Issue #2109[5040] - Stack trace of last bug causing issues with octotiger
- Issue #2108[5041] - Stack trace of last bug causing issues with octotiger
- PR #2107[5042] - Making sure that a missing parcel_coalescing module does not cause startup exceptions
- PR #2106[5043] - Stop using hpx_fwd.hpp
- Issue #2105[5044] - coalescing plugin handler is not optional any more
- Issue #2104[5045] - Make executor_traits N-nary
- Issue #2103[5046] - Build error with octotiger and hpx commit e657426d
- PR #2102[5047] - Combining thread data storage
- PR #2101[5048] - Added repartition version of 1d stencil that uses any performance counter
- PR #2100[5049] - Drop obsolete TR1 result_of protocol
- PR #2099[5050] - Replace uses of boost::bind with util::bind
- PR #2098[5051] - Deprecated inspect checks
- PR #2097[5052] - Reduce by key, extends #1141
- PR #2096[5053] - Moving local cache from external to hpx/util

[5032] https://github.com/TheHPXProject/hpx/pull/2116
[5033] https://github.com/TheHPXProject/hpx/pull/2115
[5034] https://github.com/TheHPXProject/hpx/pull/2114
[5035] https://github.com/TheHPXProject/hpx/issues/2113
[5036] https://github.com/TheHPXProject/hpx/pull/2112
[5037] https://github.com/TheHPXProject/hpx/issues/2111
[5038] https://github.com/TheHPXProject/hpx/issues/2110
[5039] https://github.com/TheHPXProject/hpx/issues/2109
[5040] https://github.com/TheHPXProject/hpx/issues/2109
[5041] https://github.com/TheHPXProject/hpx/issues/2108
[5042] https://github.com/TheHPXProject/hpx/pull/2107
[5043] https://github.com/TheHPXProject/hpx/pull/2106
[5044] https://github.com/TheHPXProject/hpx/issues/2105
[5045] https://github.com/TheHPXProject/hpx/issues/2104
[5046] https://github.com/TheHPXProject/hpx/issues/2103
[5047] https://github.com/TheHPXProject/hpx/pull/2102
[5048] https://github.com/TheHPXProject/hpx/pull/2101
[5049] https://github.com/TheHPXProject/hpx/pull/2100
[5050] https://github.com/TheHPXProject/hpx/pull/2099
[5051] https://github.com/TheHPXProject/hpx/pull/2098
[5052] https://github.com/TheHPXProject/hpx/pull/2097
[5053] https://github.com/TheHPXProject/hpx/pull/2096

- PR #2095[5054] - Bump minimum required Boost to 1.50.0
- PR #2094[5055] - Add include checks for several Boost utilities
- Issue #2093[5056] - /…/local_cache.hpp(89): error #303: explicit type is missing ("int" assumed)
- PR #2091[5057] - Fix for Raspberry pi build
- PR #2090[5058] - Fix storage size for util::function<>
- PR #2089[5059] - Fix #2088
- Issue #2088[5060] - More verbose output from cmake configuration
- PR #2087[5061] - Making sure init_globally always executes hpx_main
- Issue #2086[5062] - Race condition with recent HPX
- PR #2085[5063] - Adding #include checker
- PR #2084[5064] - Replace boost lock types with standard library ones
- PR #2083[5065] - Simplify packaged task
- PR #2082[5066] - Updating APEX version for testing
- PR #2081[5067] - Cleanup exception headers
- PR #2080[5068] - Make call_once variadic
- Issue #2079[5069] - With GNU C++, line 85 of hpx/config/version.hpp causes link failure when linking application
- Issue #2078[5070] - Simple test fails with _GLIBCXX_DEBUG defined
- PR #2077[5071] - Instantiate board in nqueen client
- PR #2076[5072] - Moving coalescing registration to TUs
- PR #2075[5073] - Fixed some documentation typos
- PR #2074[5074] - Adding flush-mode to message handler flush
- PR #2073[5075] - Fixing performance regression introduced lately

5054 https://github.com/TheHPXProject/hpx/pull/2095
5055 https://github.com/TheHPXProject/hpx/pull/2094
5056 https://github.com/TheHPXProject/hpx/issues/2093
5057 https://github.com/TheHPXProject/hpx/pull/2091
5058 https://github.com/TheHPXProject/hpx/pull/2090
5059 https://github.com/TheHPXProject/hpx/pull/2089
5060 https://github.com/TheHPXProject/hpx/issues/2088
5061 https://github.com/TheHPXProject/hpx/pull/2087
5062 https://github.com/TheHPXProject/hpx/issues/2086
5063 https://github.com/TheHPXProject/hpx/pull/2085
5064 https://github.com/TheHPXProject/hpx/pull/2084
5065 https://github.com/TheHPXProject/hpx/pull/2083
5066 https://github.com/TheHPXProject/hpx/pull/2082
5067 https://github.com/TheHPXProject/hpx/pull/2081
5068 https://github.com/TheHPXProject/hpx/pull/2080
5069 https://github.com/TheHPXProject/hpx/issues/2079
5070 https://github.com/TheHPXProject/hpx/issues/2078
5071 https://github.com/TheHPXProject/hpx/pull/2077
5072 https://github.com/TheHPXProject/hpx/pull/2076
5073 https://github.com/TheHPXProject/hpx/pull/2075
5074 https://github.com/TheHPXProject/hpx/pull/2074
5075 https://github.com/TheHPXProject/hpx/pull/2073

- PR #2072[5076] - Refactor local::condition_variable
- PR #2071[5077] - Timer based on boost::asio::deadline_timer
- PR #2070[5078] - Refactor tuple based functionality
- PR #2069[5079] - Fixed typos
- Issue #2068[5080] - Seg fault with octotiger
- PR #2067[5081] - Algorithm cleanup
- PR #2066[5082] - Split credit fixes
- PR #2065[5083] - Rename HPX_MOVABLE_BUT_NOT_COPYABLE to HPX_MOVABLE_ONLY
- PR #2064[5084] - Fixed some typos in docs
- PR #2063[5085] - Adding example demonstrating template components
- Issue #2062[5086] - Support component templates
- PR #2061[5087] - Replace some uses of lexical_cast<string> with C++11 std::to_string
- PR #2060[5088] - Replace uses of boost::noncopyable with HPX_NON_COPYABLE
- PR #2059[5089] - Adding missing for_loop algorithms
- PR #2058[5090] - Move several definitions to more appropriate headers
- PR #2057[5091] - Simplify assert_owns_lock and ignore_while_checking
- PR #2056[5092] - Replacing std::result_of with util::result_of
- PR #2055[5093] - Fix process launching/connecting back
- PR #2054[5094] - Add a forwarding coroutine header
- PR #2053[5095] - Replace uses of boost::unordered_map with std::unordered_map
- PR #2052[5096] - Rewrite tuple unwrap
- PR #2050[5097] - Replace uses of BOOST_SCOPED_ENUM with C++11 scoped enums
- PR #2049[5098] - Attempt to narrow down split_credit problem

[5076] https://github.com/TheHPXProject/hpx/pull/2072
[5077] https://github.com/TheHPXProject/hpx/pull/2071
[5078] https://github.com/TheHPXProject/hpx/pull/2070
[5079] https://github.com/TheHPXProject/hpx/pull/2069
[5080] https://github.com/TheHPXProject/hpx/issues/2068
[5081] https://github.com/TheHPXProject/hpx/pull/2067
[5082] https://github.com/TheHPXProject/hpx/pull/2066
[5083] https://github.com/TheHPXProject/hpx/pull/2065
[5084] https://github.com/TheHPXProject/hpx/pull/2064
[5085] https://github.com/TheHPXProject/hpx/pull/2063
[5086] https://github.com/TheHPXProject/hpx/issues/2062
[5087] https://github.com/TheHPXProject/hpx/pull/2061
[5088] https://github.com/TheHPXProject/hpx/pull/2060
[5089] https://github.com/TheHPXProject/hpx/pull/2059
[5090] https://github.com/TheHPXProject/hpx/pull/2058
[5091] https://github.com/TheHPXProject/hpx/pull/2057
[5092] https://github.com/TheHPXProject/hpx/pull/2056
[5093] https://github.com/TheHPXProject/hpx/pull/2055
[5094] https://github.com/TheHPXProject/hpx/pull/2054
[5095] https://github.com/TheHPXProject/hpx/pull/2053
[5096] https://github.com/TheHPXProject/hpx/pull/2052
[5097] https://github.com/TheHPXProject/hpx/pull/2050
[5098] https://github.com/TheHPXProject/hpx/pull/2049

- PR #2048[5099] - Fixing gcc startup hangs
- PR #2047[5100] - Fixing when_xxx and wait_xxx for MSVC12
- PR #2046[5101] - adding persistent_auto_chunk_size and related tests for for_each
- PR #2045[5102] - Fixing HPX_HAVE_THREAD_BACKTRACE_DEPTH build time configuration
- PR #2044[5103] - Adding missing service executor types
- PR #2043[5104] - Removing ambiguous definitions for is_future_range and future_range_traits
- PR #2042[5105] - Clarify that HPX builds can use (much) more than 2GB per process
- PR #2041[5106] - Changing future_iterator_traits to support pointers
- Issue #2040[5107] - Improve documentation memory usage warning?
- PR #2039[5108] - Coroutine cleanup
- PR #2038[5109] - Fix cmake policy CMP0042 warning MACOSX_RPATH
- PR #2037[5110] - Avoid redundant specialization of [**unique_**]function_nonser
- PR #2036[5111] - nvcc dies with an internal error upon pushing/popping warnings inside templates
- Issue #2035[5112] - Use a less restrictive iterator definition in hpx::lcos::detail::future_iterator_traits
- PR #2034[5113] - Fixing compilation error with thread queue wait time performance counter
- Issue #2033[5114] - Compilation error when compiling with thread queue waittime performance counter
- Issue #2032[5115] - Ambiguous template instantiation for is_future_range and future_range_traits.
- PR #2031[5116] - Don't restart timer on every incoming parcel
- PR #2030[5117] - Unify handling of execution policies in parallel algorithms
- PR #2029[5118] - Make pkg-config .pc files use .dylib on OSX

[5099] https://github.com/TheHPXProject/hpx/pull/2048
[5100] https://github.com/TheHPXProject/hpx/pull/2047
[5101] https://github.com/TheHPXProject/hpx/pull/2046
[5102] https://github.com/TheHPXProject/hpx/pull/2045
[5103] https://github.com/TheHPXProject/hpx/pull/2044
[5104] https://github.com/TheHPXProject/hpx/pull/2043
[5105] https://github.com/TheHPXProject/hpx/pull/2042
[5106] https://github.com/TheHPXProject/hpx/pull/2041
[5107] https://github.com/TheHPXProject/hpx/issues/2040
[5108] https://github.com/TheHPXProject/hpx/pull/2039
[5109] https://github.com/TheHPXProject/hpx/pull/2038
[5110] https://github.com/TheHPXProject/hpx/pull/2037
[5111] https://github.com/TheHPXProject/hpx/pull/2036
[5112] https://github.com/TheHPXProject/hpx/issues/2035
[5113] https://github.com/TheHPXProject/hpx/pull/2034
[5114] https://github.com/TheHPXProject/hpx/issues/2033
[5115] https://github.com/TheHPXProject/hpx/issues/2032
[5116] https://github.com/TheHPXProject/hpx/pull/2031
[5117] https://github.com/TheHPXProject/hpx/pull/2030
[5118] https://github.com/TheHPXProject/hpx/pull/2029

- PR #2028[5119] - Adding process component
- PR #2027[5120] - Making check for compiler compatibility independent on compiler path
- PR #2025[5121] - Fixing inspect tool
- PR #2024[5122] - Intel13 removal
- PR #2023[5123] - Fix errors related to older boost versions and parameter pack expansions in lambdas
- Issue #2022[5124] - gmake fail: "No rule to make target /usr/lib46/libboost_context-mt.so"
- PR #2021[5125] - Added Sudoku example
- Issue #2020[5126] - Make errors related to init_globally.cpp example while building HPX out of the box
- PR #2019[5127] - Fixed some compilation and cmake errors encountered in nqueen example
- PR #2018[5128] - For loop algorithms
- PR #2017[5129] - Non-recursive at_index implementation
- Issue #2016[5130] - Add index-based for-loops
- Issue #2015[5131] - Change default bind-mode to balanced
- PR #2014[5132] - Fixed dataflow if invoked action returns a future
- PR #2013[5133] - Fixing compilation issues with external example
- PR #2012[5134] - Added Sierpinski Triangle example
- Issue #2011[5135] - Compilation error while running sample hello_world_component code
- PR #2010[5136] - Segmented move implemented for hpx::vector
- Issue #2009[5137] - pkg-config order incorrect on 14.04 / GCC 4.8
- Issue #2008[5138] - Compilation error in dataflow of action returning a future
- PR #2007[5139] - Adding new performance counter exposing overall scheduler time
- PR #2006[5140] - Function includes

[5119] https://github.com/TheHPXProject/hpx/pull/2028
[5120] https://github.com/TheHPXProject/hpx/pull/2027
[5121] https://github.com/TheHPXProject/hpx/pull/2025
[5122] https://github.com/TheHPXProject/hpx/pull/2024
[5123] https://github.com/TheHPXProject/hpx/pull/2023
[5124] https://github.com/TheHPXProject/hpx/issues/2022
[5125] https://github.com/TheHPXProject/hpx/pull/2021
[5126] https://github.com/TheHPXProject/hpx/issues/2020
[5127] https://github.com/TheHPXProject/hpx/pull/2019
[5128] https://github.com/TheHPXProject/hpx/pull/2018
[5129] https://github.com/TheHPXProject/hpx/pull/2017
[5130] https://github.com/TheHPXProject/hpx/issues/2016
[5131] https://github.com/TheHPXProject/hpx/issues/2015
[5132] https://github.com/TheHPXProject/hpx/pull/2014
[5133] https://github.com/TheHPXProject/hpx/pull/2013
[5134] https://github.com/TheHPXProject/hpx/pull/2012
[5135] https://github.com/TheHPXProject/hpx/issues/2011
[5136] https://github.com/TheHPXProject/hpx/pull/2010
[5137] https://github.com/TheHPXProject/hpx/issues/2009
[5138] https://github.com/TheHPXProject/hpx/issues/2008
[5139] https://github.com/TheHPXProject/hpx/pull/2007
[5140] https://github.com/TheHPXProject/hpx/pull/2006

- PR #2005[5141] - Adding an example demonstrating how to initialize HPX from a global object
- PR #2004[5142] - Fixing 2000
- PR #2003[5143] - Adding generation parameter to gather to enable using it more than once
- PR #2002[5144] - Turn on position independent code to solve link problem with hpx_init
- Issue #2001[5145] - Gathering more than once segfaults
- Issue #2000[5146] - Undefined reference to hpx::assertion_failed
- Issue #1999[5147] - Seg fault in hpx::lcos::base_lco_with_value<*>::set_value_nonvirt() when running octo-tiger
- PR #1998[5148] - Detect unknown command line options
- PR #1997[5149] - Extending thread description
- PR #1996[5150] - Adding natvis files to solution (MSVC only)
- Issue #1995[5151] - Command line handling does not produce error
- PR #1994[5152] - Possible missing include in test_utils.hpp
- PR #1993[5153] - Add missing LANGUAGES tag to a hpx_add_compile_flag_if_available() call in CMakeLists.txt
- PR #1992[5154] - Fixing shared_executor_test
- PR #1991[5155] - Making sure the winsock library is properly initialized
- PR #1990[5156] - Fixing bind_test placeholder ambiguity coming from boost-1.60
- PR #1989[5157] - Performance tuning
- PR #1987[5158] - Make configurable size of internal storage in util::function
- PR #1986[5159] - AGAS Refactoring+1753 Cache mods
- PR #1985[5160] - Adding missing task_block::run() overload taking an executor
- PR #1984[5161] - Adding an optimized LRU Cache implementation (for AGAS)
- PR #1983[5162] - Avoid invoking migration table look up for all objects

5141 https://github.com/TheHPXProject/hpx/pull/2005
5142 https://github.com/TheHPXProject/hpx/pull/2004
5143 https://github.com/TheHPXProject/hpx/pull/2003
5144 https://github.com/TheHPXProject/hpx/pull/2002
5145 https://github.com/TheHPXProject/hpx/issues/2001
5146 https://github.com/TheHPXProject/hpx/issues/2000
5147 https://github.com/TheHPXProject/hpx/issues/1999
5148 https://github.com/TheHPXProject/hpx/pull/1998
5149 https://github.com/TheHPXProject/hpx/pull/1997
5150 https://github.com/TheHPXProject/hpx/pull/1996
5151 https://github.com/TheHPXProject/hpx/issues/1995
5152 https://github.com/TheHPXProject/hpx/pull/1994
5153 https://github.com/TheHPXProject/hpx/pull/1993
5154 https://github.com/TheHPXProject/hpx/pull/1992
5155 https://github.com/TheHPXProject/hpx/pull/1991
5156 https://github.com/TheHPXProject/hpx/pull/1990
5157 https://github.com/TheHPXProject/hpx/pull/1989
5158 https://github.com/TheHPXProject/hpx/pull/1987
5159 https://github.com/TheHPXProject/hpx/pull/1986
5160 https://github.com/TheHPXProject/hpx/pull/1985
5161 https://github.com/TheHPXProject/hpx/pull/1984
5162 https://github.com/TheHPXProject/hpx/pull/1983

- PR #1981[5163] - Replacing uintptr_t (which is not defined everywhere) with std::size_t
- PR #1980[5164] - Optimizing LCO continuations
- PR #1979[5165] - Fixing Cori
- PR #1978[5166] - Fix test check that got broken in hasty fix to memory overflow
- PR #1977[5167] - Refactor action traits
- PR #1976[5168] - Fixes typo in README.rst
- PR #1975[5169] - Reduce size of benchmark timing arrays to fix test failures
- PR #1974[5170] - Add action to update data owned by the partitioned_vector component
- PR #1972[5171] - Adding partitioned_vector SPMD example
- PR #1971[5172] - Fixing 1965
- PR #1970[5173] - Papi fixes
- PR #1969[5174] - Fixing continuation recursions to not depend on fixed amount of recursions
- PR #1968[5175] - More segmented algorithms
- Issue #1967[5176] - Simplify component implementations
- PR #1966[5177] - Migrate components
- Issue #1964[5178] - fatal error: 'boost/lockfree/detail/branch_hints.hpp' file not found
- Issue #1962[5179] - parallel:copy_if has race condition when used on in place arrays
- PR #1963[5180] - Fixing Static Parcelport initialization
- PR #1961[5181] - Fix function target
- Issue #1960[5182] - Papi counters don't reset
- PR #1959[5183] - Fixing 1958
- Issue #1958[5184] - inclusive_scan gives incorrect results with non-commutative operator
- PR #1957[5185] - Fixing #1950

[5163] https://github.com/TheHPXProject/hpx/pull/1981
[5164] https://github.com/TheHPXProject/hpx/pull/1980
[5165] https://github.com/TheHPXProject/hpx/pull/1979
[5166] https://github.com/TheHPXProject/hpx/pull/1978
[5167] https://github.com/TheHPXProject/hpx/pull/1977
[5168] https://github.com/TheHPXProject/hpx/pull/1976
[5169] https://github.com/TheHPXProject/hpx/pull/1975
[5170] https://github.com/TheHPXProject/hpx/pull/1974
[5171] https://github.com/TheHPXProject/hpx/pull/1972
[5172] https://github.com/TheHPXProject/hpx/pull/1971
[5173] https://github.com/TheHPXProject/hpx/pull/1970
[5174] https://github.com/TheHPXProject/hpx/pull/1969
[5175] https://github.com/TheHPXProject/hpx/pull/1968
[5176] https://github.com/TheHPXProject/hpx/issues/1967
[5177] https://github.com/TheHPXProject/hpx/pull/1966
[5178] https://github.com/TheHPXProject/hpx/issues/1964
[5179] https://github.com/TheHPXProject/hpx/issues/1962
[5180] https://github.com/TheHPXProject/hpx/pull/1963
[5181] https://github.com/TheHPXProject/hpx/pull/1961
[5182] https://github.com/TheHPXProject/hpx/issues/1960
[5183] https://github.com/TheHPXProject/hpx/pull/1959
[5184] https://github.com/TheHPXProject/hpx/issues/1958
[5185] https://github.com/TheHPXProject/hpx/pull/1957

- PR #1956[5186] - Sort by key example
- PR #1955[5187] - Adding regression test for #1946: Hang in wait_all() in distributed run
- Issue #1954[5188] - HPX releases should not use -Werror
- PR #1953[5189] - Adding performance analysis for AGAS cache
- PR #1952[5190] - Adapting test for explicit variadics to fail for gcc 4.6
- PR #1951[5191] - Fixing memory leak
- Issue #1950[5192] - Simplify external builds
- PR #1949[5193] - Fixing yet another lock that is being held during suspension
- PR #1948[5194] - Fixed container algorithms for Intel
- PR #1947[5195] - Adding workaround for tagged_tuple
- Issue #1946[5196] - Hang in wait_all() in distributed run
- PR #1945[5197] - Fixed container algorithm tests
- Issue #1944[5198] - assertion 'p.destination_locality() == hpx::get_locality()' failed
- PR #1943[5199] - Fix a couple of compile errors with clang
- PR #1942[5200] - Making parcel coalescing functional
- Issue #1941[5201] - Re-enable parcel coalescing
- PR #1940[5202] - Touching up make_future
- PR #1939[5203] - Fixing problems in over-subscription management in the resource manager
- PR #1938[5204] - Removing use of unified Boost.Thread header
- PR #1937[5205] - Cleaning up the use of Boost.Accumulator headers
- PR #1936[5206] - Making sure interval timer is started for aggregating performance counters
- PR #1935[5207] - Tagged results

[5186] https://github.com/TheHPXProject/hpx/pull/1956
[5187] https://github.com/TheHPXProject/hpx/pull/1955
[5188] https://github.com/TheHPXProject/hpx/issues/1954
[5189] https://github.com/TheHPXProject/hpx/pull/1953
[5190] https://github.com/TheHPXProject/hpx/pull/1952
[5191] https://github.com/TheHPXProject/hpx/pull/1951
[5192] https://github.com/TheHPXProject/hpx/issues/1950
[5193] https://github.com/TheHPXProject/hpx/pull/1949
[5194] https://github.com/TheHPXProject/hpx/pull/1948
[5195] https://github.com/TheHPXProject/hpx/pull/1947
[5196] https://github.com/TheHPXProject/hpx/issues/1946
[5197] https://github.com/TheHPXProject/hpx/pull/1945
[5198] https://github.com/TheHPXProject/hpx/issues/1944
[5199] https://github.com/TheHPXProject/hpx/pull/1943
[5200] https://github.com/TheHPXProject/hpx/pull/1942
[5201] https://github.com/TheHPXProject/hpx/issues/1941
[5202] https://github.com/TheHPXProject/hpx/pull/1940
[5203] https://github.com/TheHPXProject/hpx/pull/1939
[5204] https://github.com/TheHPXProject/hpx/pull/1938
[5205] https://github.com/TheHPXProject/hpx/pull/1937
[5206] https://github.com/TheHPXProject/hpx/pull/1936
[5207] https://github.com/TheHPXProject/hpx/pull/1935

- PR #1934[5208] - Fix remote async with deferred launch policy
- Issue #1933[5209] - Floating point exception in `statistics_counter<boost::accumulators::tag::mean>::get_counter_value`
- PR #1932[5210] - Removing superfluous includes of boost/lockfree/detail/branch_hints.hpp
- PR #1931[5211] - fix compilation with clang 3.8.0
- Issue #1930[5212] - Missing online documentation for HPX 0.9.11
- PR #1929[5213] - LWG2485: get() should be overloaded for const tuple&&
- PR #1928[5214] - Revert "Using ninja for circle-ci builds"
- PR #1927[5215] - Using ninja for circle-ci builds
- PR #1926[5216] - Fixing serialization of std::array
- Issue #1925[5217] - Issues with static HPX libraries
- Issue #1924[5218] - Performance degrading over time
- Issue #1923[5219] - serialization of std::array appears broken in latest commit
- PR #1922[5220] - Container algorithms
- PR #1921[5221] - Tons of smaller quality improvements
- Issue #1920[5222] - Seg fault in hpx::serialization::output_archive::add_gid when running octotiger
- Issue #1919[5223] - Intel 15 compiler bug preventing HPX build
- PR #1918[5224] - Address sanitizer fixes
- PR #1917[5225] - Fixing compilation problems of parallel::sort with Intel compilers
- PR #1916[5226] - Making sure code compiles if HPX_WITH_HWLOC=Off
- Issue #1915[5227] - max_cores undefined if HPX_WITH_HWLOC=Off
- PR #1913[5228] - Add utility member functions for partitioned_vector
- PR #1912[5229] - Adding support for invoking actions to dataflow

5208 https://github.com/TheHPXProject/hpx/pull/1934
5209 https://github.com/TheHPXProject/hpx/issues/1933
5210 https://github.com/TheHPXProject/hpx/pull/1932
5211 https://github.com/TheHPXProject/hpx/pull/1931
5212 https://github.com/TheHPXProject/hpx/issues/1930
5213 https://github.com/TheHPXProject/hpx/pull/1929
5214 https://github.com/TheHPXProject/hpx/pull/1928
5215 https://github.com/TheHPXProject/hpx/pull/1927
5216 https://github.com/TheHPXProject/hpx/pull/1926
5217 https://github.com/TheHPXProject/hpx/issues/1925
5218 https://github.com/TheHPXProject/hpx/issues/1924
5219 https://github.com/TheHPXProject/hpx/issues/1923
5220 https://github.com/TheHPXProject/hpx/pull/1922
5221 https://github.com/TheHPXProject/hpx/pull/1921
5222 https://github.com/TheHPXProject/hpx/issues/1920
5223 https://github.com/TheHPXProject/hpx/issues/1919
5224 https://github.com/TheHPXProject/hpx/pull/1918
5225 https://github.com/TheHPXProject/hpx/pull/1917
5226 https://github.com/TheHPXProject/hpx/pull/1916
5227 https://github.com/TheHPXProject/hpx/issues/1915
5228 https://github.com/TheHPXProject/hpx/pull/1913
5229 https://github.com/TheHPXProject/hpx/pull/1912

- PR #1911[5230] - Adding first batch of container algorithms
- PR #1910[5231] - Keep cmake_module_path
- PR #1909[5232] - Fix mpirun with pbs
- PR #1908[5233] - Changing parallel::sort to return the last iterator as proposed by N4560
- PR #1907[5234] - Adding a minimum version for Open MPI
- PR #1906[5235] - Updates to the Release Procedure
- PR #1905[5236] - Fixing #1903
- PR #1904[5237] - Making sure std containers are cleared before serialization loads data
- Issue #1903[5238] - When running octotiger, I get: assertion `'(*new_gids_)[gid].size() == 1' failed: HPX(assertion_failure)`
- Issue #1902[5239] - Immediate crash when running hpx/octotiger with _GLIBCXX_DEBUG defined.
- PR #1901[5240] - Making non-serializable classes non-serializable
- Issue #1900[5241] - Two possible issues with std::list serialization
- PR #1899[5242] - Fixing a problem with credit splitting as revealed by #1898
- Issue #1898[5243] - Accessing component from locality where it was not created segfaults
- PR #1897[5244] - Changing parallel::sort to return the last iterator as proposed by N4560
- Issue #1896[5245] - version 1.0?
- Issue #1895[5246] - Warning comment on numa_allocator is not very clear
- PR #1894[5247] - Add support for compilers that have thread_local
- PR #1893[5248] - Fixing 1890
- PR #1892[5249] - Adds typed future_type for executor_traits
- PR #1891[5250] - Fix wording in certain parallel algorithm docs
- Issue #1890[5251] - Invoking papi counters give segfault

[5230] https://github.com/TheHPXProject/hpx/pull/1911
[5231] https://github.com/TheHPXProject/hpx/pull/1910
[5232] https://github.com/TheHPXProject/hpx/pull/1909
[5233] https://github.com/TheHPXProject/hpx/pull/1908
[5234] https://github.com/TheHPXProject/hpx/pull/1907
[5235] https://github.com/TheHPXProject/hpx/pull/1906
[5236] https://github.com/TheHPXProject/hpx/pull/1905
[5237] https://github.com/TheHPXProject/hpx/pull/1904
[5238] https://github.com/TheHPXProject/hpx/issues/1903
[5239] https://github.com/TheHPXProject/hpx/issues/1902
[5240] https://github.com/TheHPXProject/hpx/pull/1901
[5241] https://github.com/TheHPXProject/hpx/issues/1900
[5242] https://github.com/TheHPXProject/hpx/pull/1899
[5243] https://github.com/TheHPXProject/hpx/issues/1898
[5244] https://github.com/TheHPXProject/hpx/pull/1897
[5245] https://github.com/TheHPXProject/hpx/issues/1896
[5246] https://github.com/TheHPXProject/hpx/issues/1895
[5247] https://github.com/TheHPXProject/hpx/pull/1894
[5248] https://github.com/TheHPXProject/hpx/pull/1893
[5249] https://github.com/TheHPXProject/hpx/pull/1892
[5250] https://github.com/TheHPXProject/hpx/pull/1891
[5251] https://github.com/TheHPXProject/hpx/issues/1890

- PR #1889[5252] - Fixing problems as reported by clang-check
- PR #1888[5253] - WIP parallel is_heap
- PR #1887[5254] - Fixed resetting performance counters related to idle-rate, etc
- Issue #1886[5255] - Run hpx with qsub does not work
- PR #1885[5256] - Warning cleaning pass
- PR #1884[5257] - Add missing parallel algorithm header
- PR #1883[5258] - Add feature test for thread_local on Clang for TLS
- PR #1882[5259] - Fix some redundant qualifiers
- Issue #1881[5260] - Unable to compile Octotiger using HPX and Intel MPI on SuperMIC
- Issue #1880[5261] - clang with libc++ on Linux needs TLS case
- PR #1879[5262] - Doc fixes for #1868
- PR #1878[5263] - Simplify functions
- PR #1877[5264] - Removing most usage of Boost.Config
- PR #1876[5265] - Add missing parallel algorithms to algorithm.hpp
- PR #1875[5266] - Simplify callables
- PR #1874[5267] - Address long standing FIXME on using `std::unique_ptr` with incomplete types
- PR #1873[5268] - Fixing 1871
- PR #1872[5269] - Making sure PBS environment uses specified node list even if no PBS_NODEFILE env is available
- Issue #1871[5270] - Fortran checks should be optional
- PR #1870[5271] - Touch local::mutex
- PR #1869[5272] - Documentation refactoring based off #1868
- PR #1867[5273] - Embrace static_assert

[5252] https://github.com/TheHPXProject/hpx/pull/1889
[5253] https://github.com/TheHPXProject/hpx/pull/1888
[5254] https://github.com/TheHPXProject/hpx/pull/1887
[5255] https://github.com/TheHPXProject/hpx/issues/1886
[5256] https://github.com/TheHPXProject/hpx/pull/1885
[5257] https://github.com/TheHPXProject/hpx/pull/1884
[5258] https://github.com/TheHPXProject/hpx/pull/1883
[5259] https://github.com/TheHPXProject/hpx/pull/1882
[5260] https://github.com/TheHPXProject/hpx/issues/1881
[5261] https://github.com/TheHPXProject/hpx/issues/1880
[5262] https://github.com/TheHPXProject/hpx/pull/1879
[5263] https://github.com/TheHPXProject/hpx/pull/1878
[5264] https://github.com/TheHPXProject/hpx/pull/1877
[5265] https://github.com/TheHPXProject/hpx/pull/1876
[5266] https://github.com/TheHPXProject/hpx/pull/1875
[5267] https://github.com/TheHPXProject/hpx/pull/1874
[5268] https://github.com/TheHPXProject/hpx/pull/1873
[5269] https://github.com/TheHPXProject/hpx/pull/1872
[5270] https://github.com/TheHPXProject/hpx/issues/1871
[5271] https://github.com/TheHPXProject/hpx/pull/1870
[5272] https://github.com/TheHPXProject/hpx/pull/1869
[5273] https://github.com/TheHPXProject/hpx/pull/1867

- PR #1866[5274] - Fix #1803 with documentation refactoring
- PR #1865[5275] - Setting OUTPUT_NAME as target properties
- PR #1863[5276] - Use SYSTEM for boost includes
- PR #1862[5277] - Minor cleanups
- PR #1861[5278] - Minor Corrections for Release
- PR #1860[5279] - Fixing hpx gdb script
- Issue #1859[5280] - reset_active_counters resets times and thread counts before some of the counters are evaluated
- PR #1858[5281] - Release V0.9.11
- PR #1857[5282] - removing diskperf example from 9.11 release
- PR #1856[5283] - fix return in packaged_task_base::reset()
- Issue #1842[5284] - Install error: file INSTALL cannot find libhpx_parcel_coalescing.so.0.9.11
- PR #1839[5285] - Adding fedora docs
- PR #1824[5286] - Changing version on master to V0.9.12
- PR #1818[5287] - Fixing #1748
- Issue #1815[5288] - seg fault in AGAS
- Issue #1803[5289] - wait_all documentation
- Issue #1796[5290] - Outdated documentation to be revised
- Issue #1759[5291] - glibc munmap_chunk or free(): invalid pointer on SuperMIC
- Issue #1753[5292] - HPX performance degrades with time since execution begins
- Issue #1748[5293] - All public HPX headers need to be self contained
- PR #1719[5294] - How to build HPX with Visual Studio
- Issue #1684[5295] - Race condition when using –hpx:connect?

[5274] https://github.com/TheHPXProject/hpx/pull/1866
[5275] https://github.com/TheHPXProject/hpx/pull/1865
[5276] https://github.com/TheHPXProject/hpx/pull/1863
[5277] https://github.com/TheHPXProject/hpx/pull/1862
[5278] https://github.com/TheHPXProject/hpx/pull/1861
[5279] https://github.com/TheHPXProject/hpx/pull/1860
[5280] https://github.com/TheHPXProject/hpx/issues/1859
[5281] https://github.com/TheHPXProject/hpx/pull/1858
[5282] https://github.com/TheHPXProject/hpx/pull/1857
[5283] https://github.com/TheHPXProject/hpx/pull/1856
[5284] https://github.com/TheHPXProject/hpx/issues/1842
[5285] https://github.com/TheHPXProject/hpx/pull/1839
[5286] https://github.com/TheHPXProject/hpx/pull/1824
[5287] https://github.com/TheHPXProject/hpx/pull/1818
[5288] https://github.com/TheHPXProject/hpx/issues/1815
[5289] https://github.com/TheHPXProject/hpx/issues/1803
[5290] https://github.com/TheHPXProject/hpx/issues/1796
[5291] https://github.com/TheHPXProject/hpx/issues/1759
[5292] https://github.com/TheHPXProject/hpx/issues/1753
[5293] https://github.com/TheHPXProject/hpx/issues/1748
[5294] https://github.com/TheHPXProject/hpx/pull/1719
[5295] https://github.com/TheHPXProject/hpx/issues/1684

- PR #1658[5296] - Add serialization for std::set (as there is for std::vector and std::map)
- PR #1641[5297] - Generic client
- Issue #1632[5298] - heartbeat example fails on separate nodes
- PR #1603[5299] - Adds preferred namespace check to inspect tool
- Issue #1559[5300] - Extend inspect tool
- Issue #1523[5301] - Remote async with deferred launch policy never executes
- Issue #1472[5302] - Serialization issues
- Issue #1457[5303] - Implement N4392: C++ Latches and Barriers
- PR #1444[5304] - Enabling usage of moveonly types for component construction
- Issue #1407[5305] - The Intel 13 compiler has failing unit tests
- Issue #1405[5306] - Allow component constructors to take movable only types
- Issue #1265[5307] - Enable dataflow() to be usable with actions
- Issue #1236[5308] - NUMA aware allocators
- Issue #802[5309] - Fix Broken Examples
- Issue #559[5310] - Add hpx::migrate facility
- Issue #449[5311] - Make actions with template arguments usable and add documentation
- Issue #279[5312] - Refactor addressing_service into a base class and two derived classes
- Issue #224[5313] - Changing thread state metadata is not thread safe
- Issue #55[5314] - Uniform syntax for enums should be implemented

[5296] https://github.com/TheHPXProject/hpx/pull/1658
[5297] https://github.com/TheHPXProject/hpx/pull/1641
[5298] https://github.com/TheHPXProject/hpx/issues/1632
[5299] https://github.com/TheHPXProject/hpx/pull/1603
[5300] https://github.com/TheHPXProject/hpx/issues/1559
[5301] https://github.com/TheHPXProject/hpx/issues/1523
[5302] https://github.com/TheHPXProject/hpx/issues/1472
[5303] https://github.com/TheHPXProject/hpx/issues/1457
[5304] https://github.com/TheHPXProject/hpx/pull/1444
[5305] https://github.com/TheHPXProject/hpx/issues/1407
[5306] https://github.com/TheHPXProject/hpx/issues/1405
[5307] https://github.com/TheHPXProject/hpx/issues/1265
[5308] https://github.com/TheHPXProject/hpx/issues/1236
[5309] https://github.com/TheHPXProject/hpx/issues/802
[5310] https://github.com/TheHPXProject/hpx/issues/559
[5311] https://github.com/TheHPXProject/hpx/issues/449
[5312] https://github.com/TheHPXProject/hpx/issues/279
[5313] https://github.com/TheHPXProject/hpx/issues/224
[5314] https://github.com/TheHPXProject/hpx/issues/55

HPX V0.9.11 (Nov 11, 2015)

Our main focus for this release was the design and development of a coherent set of higher-level APIs exposing various types of parallelism to the application programmer. We introduced the concepts of an `executor`, which can be used to customize the `where` and `when` of execution of tasks in the context of parallelizing codes. We extended all APIs related to managing parallel tasks to support executors which gives the user the choce of either using one of the predefined executor types or to provide its own, possibly application specific, executor. We paid very close attention to align all of these changes with the existing C++ Standards documents or with the ongoing proposals for standardization.

This release is the first after our change to a new development policy. We switched all development to be strictly performed on branches only, all direct commits to our main branch (`master`) are prohibited. Any change has to go through a peer review before it will be merged to `master`. As a result the overall stability of our code base has significantly increased, the development process itself has been simplified. This change manifests itself in a large number of pull-requests which have been merged (please see below for a full list of closed issues and pull-requests). All in all for this release, we closed almost 100 issues and merged over 290 pull-requests. There have been over 1600 commits to the master branch since the last release.

General changes

- We are moving into the direction of unifying managed and simple components. As such, the classes *hpx::components::component* and *hpx::components::component_base* have been added which currently just forward to the currently existing simple component facilities. The examples have been converted to only use those two classes.
- Added integration with the CircleCI[5315] hosted continuous integration service. This gives us constant and immediate feedback on the health of our master branch.
- The compiler configuration subsystem in the build system has been reimplemented. Instead of using Boost.Config we now use our own lightweight set of cmake scripts to determine the available language and library features supported by the used compiler.
- The API for creating instances of components has been consolidated. All component instances should be created using the *hpx::new_* only. It allows one to instantiate both, single component instances and multiple component instances. The placement of the created components can be controlled by special distribution policies. Please see the corresponding documentation outlining the use of *hpx::new_*.
- Introduced four new distribution policies which can be used with many API functions which traditionally expected to be used with a locality id. The new distribution policies are:
  - *hpx::components::default_distribution_policy* which tries to place multiple component instances as evenly as possible.
  - *hpx::components::colocating_distribution_policy* which will refer to the locality where a given component instance is currently placed.
  - *hpx::components::binpacking_distribution_policy* which will place multiple component instances as evenly as possible based on any performance counter.

[5315] https://circleci.com/gh/TheHPXProject/hpx

- *hpx::components::target_distribution_policy* which allows one to represent a given locality in the context of a distrwibution policy.

- The new distribution policies can now be also used with `hpx::async`. This change also deprecates `hpx::async_colocated(id, ...)` which now is replaced by a distribution policy: `hpx::async(hpx::colocated(id), ...)`.
- The `hpx::vector` and `hpx::unordered_map` data structures can now be used with the new distribution policies as well.
- The parallel facility `hpx::parallel::task_region` has been renamed to `hpx::parallel::task_block` based on the changes in the corresponding standardization proposal N4411[5316].
- Added extensions to the parallel facility `hpx::parallel::task_block` allowing to combine a task_block with an execution policy. This implies a minor breaking change as the `hpx::parallel::task_block` is now a template.
- Added new LCOs: `hpx::lcos::latch` and `hpx::lcos::local::latch` which semantically conform to the proposed `std::latch` (see N4399[5317]).
- Added performance counters exposing data related to data transferred by input/output (filesystem) operations (thanks to Maciej Brodowicz).
- Added performance counters allowing to track the number of action invocations (local and remote invocations).
- Added new command line options –hpx:print-counter-at and –hpx:reset-counters.
- The `hpx::vector` component has been renamed to `hpx::partitioned_vector` to make it explicit that the underlying memory is not contiguous.
- Introduced a completely new and uniform higher-level parallelism API which is based on executors. All existing parallelism APIs have been adapted to this. We have added a large number of different executor types, such as a numa-aware executor, a this-thread executor, etc.
- Added support for the MingW toolchain on Windows (thanks to Eric Lemanissier).
- HPX now includes support for APEX, (Autonomic Performance Environment for eXascale). APEX is an instrumentation and software adaptation library that provides an interface to TAU profiling / tracing as well as runtime adaptation of HPX applications through policy definitions. For more information and documentation, please see https://github.com/UO-OACISS/xpress-apex. To enable APEX at configuration time, specify `-DHPX_WITH_APEX=On`. To also include support for TAU profiling, specify `-DHPX_WITH_TAU=On` and specify the `-DTAU_ROOT`, `-DTAU_ARCH` and `-DTAU_OPTIONS` cmake parameters.
- We have implemented many more of the *Using parallel algorithms*. Please see Issue #1141[5318] for the list of all available parallel algorithms (thanks to Daniel Bourgeois and John Biddiscombe for contributing their work).

[5316] http://wg21.link/n4411
[5317] http://wg21.link/n4399
[5318] https://github.com/TheHPXProject/hpx/issues/1141

Breaking changes

- We are moving into the direction of unifying managed and simple components. In order to stop exposing the old facilities, all examples have been converted to use the new classes. The breaking change in this release is that performance counters are now a *hpx::components::component_base* instead of *hpx::components::managed_component_base*.
- We removed the support for stackless threads. It turned out that there was no performance benefit when using stackless threads. As such, we decided to clean up our codebase. This feature was not documented.
- The CMake project name has changed from 'hpx' to 'HPX' for consistency and compatibility with naming conventions and other CMake projects. Generated config files go into <prefix>/lib/cmake/HPX and not <prefix>/lib/cmake/hpx.
- The macro `HPX_REGISTER_MINIMAL_COMPONENT_FACTORY` has been deprecated. Please use *HPX_REGISTER_COMPONENT*. instead. The old macro will be removed in the next release.
- The obsolete distributing_factory and binpacking_factory components have been removed. The corresponding functionality is now provided by the *hpx::new_* API function in conjunction with the `hpx::default_layout` and `hpx::binpacking` distribution policies (*hpx::components::default_distribution_policy* and *hpx::components::binpacking_distribution_policy*)
- The API function `hpx::new_colocated` has been deprecated. Please use the consolidated API *hpx::new_* in conjunction with the new `hpx::colocated` distribution policy (*hpx::components::colocating_distribution_policy*) instead. The old API function will still be available for at least one release of *HPX* if the configuration variable `HPX_WITH_COLOCATED_BACKWARDS_COMPATIBILITY` is enabled.
- The API function `hpx::async_colocated` has been deprecated. Please use the consolidated API `hpx::async` in conjunction with the new `hpx::colocated` distribution policy (*hpx::components::colocating_distribution_policy*) instead. The old API function will still be available for at least one release of *HPX* if the configuration variable `HPX_WITH_COLOCATED_BACKWARDS_COMPATIBILITY` is enabled.
- The obsolete remote_object component has been removed.
- Replaced the use of Boost.Serialization with our own solution. While the new version is mostly compatible with Boost.Serialization, this change requires some minor code modifications in user code. For more information, please see the corresponding announcement[5319] on the hpx-users@stellar.cct.lsu.edu mailing list.
- The names used by cmake to influence various configuration options have been unified. The new naming scheme relies on all configuration constants to start with `HPX_WITH_...`, while the preprocessor constant which is used at build time starts with `HPX_HAVE_...`. For instance, the former cmake command line `-DHPX_MALLOC=...` now has to be specified a `-DHPX_WITH_MALLOC=...` and will cause the preprocessor constant `HPX_HAVE_MALLOC` to be defined. The actual name of the constant (i.e. `MALLOC`) has not changed. Please see the corresponding documentation for more details (*CMake options*).
- The `get_gid()` functions exposed by the component base classes `hpx::components::server::simple_component_base`, `hpx::components::server::managed_component_base`, and `hpx::components::server::fixed_component_base` have been replaced by

[5319] http://thread.gmane.org/gmane.comp.lib.hpx.devel/196 two new functions: `get_unmanaged_id()` and `get_id()`. To enable the old function name for backwards compatibility, use the cmake configuration option `HPX_WITH_COMPONENT_GET_GID_COMPATIBILITY=On`.

- All functions which were named `get_gid()` but were returning `hpx::id_type` have been renamed to `get_id()`. To enable the old function names for backwards compatibility, use the cmake configuration option `HPX_WITH_COMPONENT_GET_GID_COMPATIBILITY=On`.

Bug fixes (closed tickets)

Here is a list of the important tickets we closed for this release.

- PR #1855[5320] - Completely removing external/endian
- PR #1854[5321] - Don't pollute CMAKE_CXX_FLAGS through find_package()
- PR #1853[5322] - Updating CMake configuration to get correct version of TAU library
- PR #1852[5323] - Fixing Performance Problems with MPI Parcelport
- PR #1851[5324] - Fixing hpx_add_link_flag() and hpx_remove_link_flag()
- PR #1850[5325] - Fixing 1836, adding parallel::sort
- PR #1849[5326] - Fixing configuration for use of more than 64 cores
- PR #1848[5327] - Change default APEX version for release
- PR #1847[5328] - Fix client_base::then on release
- PR #1846[5329] - Removing broken lcos::local::channel from release
- PR #1845[5330] - Adding example demonstrating a possible safe-object implementation to release
- PR #1844[5331] - Removing stubs from accumulator examples
- PR #1843[5332] - Don't pollute CMAKE_CXX_FLAGS through find_package()
- PR #1841[5333] - Fixing client_base<>::then
- PR #1840[5334] - Adding example demonstrating a possible safe-object implementation
- PR #1838[5335] - Update version rc1
- PR #1837[5336] - Removing broken lcos::local::channel

5320 https://github.com/TheHPXProject/hpx/pull/1855
5321 https://github.com/TheHPXProject/hpx/pull/1854
5322 https://github.com/TheHPXProject/hpx/pull/1853
5323 https://github.com/TheHPXProject/hpx/pull/1852
5324 https://github.com/TheHPXProject/hpx/pull/1851
5325 https://github.com/TheHPXProject/hpx/pull/1850
5326 https://github.com/TheHPXProject/hpx/pull/1849
5327 https://github.com/TheHPXProject/hpx/pull/1848
5328 https://github.com/TheHPXProject/hpx/pull/1847
5329 https://github.com/TheHPXProject/hpx/pull/1846
5330 https://github.com/TheHPXProject/hpx/pull/1845
5331 https://github.com/TheHPXProject/hpx/pull/1844
5332 https://github.com/TheHPXProject/hpx/pull/1843
5333 https://github.com/TheHPXProject/hpx/pull/1841
5334 https://github.com/TheHPXProject/hpx/pull/1840
5335 https://github.com/TheHPXProject/hpx/pull/1838
5336 https://github.com/TheHPXProject/hpx/pull/1837

- PR #1835[5337] - Adding explicit move constructor and assignment operator to hpx::lcos::promise
- PR #1834[5338] - Making hpx::lcos::promise move-only
- PR #1833[5339] - Adding fedora docs
- Issue #1832[5340] - hpx::lcos::promise<> must be move-only
- PR #1831[5341] - Fixing resource manager gcc5.2
- PR #1830[5342] - Fix intel13
- PR #1829[5343] - Unbreaking thread test
- PR #1828[5344] - Fixing #1620
- PR #1827[5345] - Fixing a memory management issue for the Parquet application
- Issue #1826[5346] - Memory management issue in hpx::lcos::promise
- PR #1825[5347] - Adding hpx::components::component and hpx::components::component_base
- PR #1823[5348] - Adding git commit id to circleci build
- PR #1822[5349] - applying fixes suggested by clang 3.7
- PR #1821[5350] - Hyperlink fixes
- PR #1820[5351] - added parallel multi-locality sanity test
- PR #1819[5352] - Fixing #1667
- Issue #1817[5353] - Hyperlinks generated by inspect tool are wrong
- PR #1816[5354] - Support hpxrx
- PR #1814[5355] - Fix async to dispatch to the correct locality in all cases
- Issue #1813[5356] - async(launch::…, action(), …) always invokes locally
- PR #1812[5357] - fixed syntax error in CMakeLists.txt
- PR #1811[5358] - Agas optimizations

[5337] https://github.com/TheHPXProject/hpx/pull/1835
[5338] https://github.com/TheHPXProject/hpx/pull/1834
[5339] https://github.com/TheHPXProject/hpx/pull/1833
[5340] https://github.com/TheHPXProject/hpx/issues/1832
[5341] https://github.com/TheHPXProject/hpx/pull/1831
[5342] https://github.com/TheHPXProject/hpx/pull/1830
[5343] https://github.com/TheHPXProject/hpx/pull/1829
[5344] https://github.com/TheHPXProject/hpx/pull/1828
[5345] https://github.com/TheHPXProject/hpx/pull/1827
[5346] https://github.com/TheHPXProject/hpx/issues/1826
[5347] https://github.com/TheHPXProject/hpx/pull/1825
[5348] https://github.com/TheHPXProject/hpx/pull/1823
[5349] https://github.com/TheHPXProject/hpx/pull/1822
[5350] https://github.com/TheHPXProject/hpx/pull/1821
[5351] https://github.com/TheHPXProject/hpx/pull/1820
[5352] https://github.com/TheHPXProject/hpx/pull/1819
[5353] https://github.com/TheHPXProject/hpx/issues/1817
[5354] https://github.com/TheHPXProject/hpx/pull/1816
[5355] https://github.com/TheHPXProject/hpx/pull/1814
[5356] https://github.com/TheHPXProject/hpx/issues/1813
[5357] https://github.com/TheHPXProject/hpx/pull/1812
[5358] https://github.com/TheHPXProject/hpx/pull/1811

- PR #1810[5359] - drop superfluous typedefs
- PR #1809[5360] - Allow HPX to be used as an optional package in 3rd party code
- PR #1808[5361] - Fixing #1723
- PR #1807[5362] - Making sure resolve_localities does not hang during normal operation
- Issue #1806[5363] - Spinlock no longer movable and deletes operator '=', breaks MiniGhost
- Issue #1804[5364] - register_with_basename causes hangs
- PR #1801[5365] - Enhanced the inspect tool to take user directly to the problem with hyperlinks
- Issue #1800[5366] - Problems compiling application on smic
- PR #1799[5367] - Fixing cv exceptions
- PR #1798[5368] - Documentation refactoring & updating
- PR #1797[5369] - Updating the activeharmony CMake module
- PR #1795[5370] - Fixing cv
- PR #1794[5371] - Fix connect with hpx::runtime_mode_connect
- PR #1793[5372] - fix a wrong use of HPX_MAX_CPU_COUNT instead of HPX_HAVE_MAX_CPU_COUNT
- PR #1792[5373] - Allow for default constructed parcel instances to be moved
- PR #1791[5374] - Fix connect with hpx::runtime_mode_connect
- Issue #1790[5375] - assertion `action_.get()` failed: HPX(assertion_failure) when running Octotiger with pull request 1786
- PR #1789[5376] - Fixing discover_counter_types API function
- Issue #1788[5377] - connect with hpx::runtime_mode_connect
- Issue #1787[5378] - discover_counter_types not working
- PR #1786[5379] - Changing addressing_service to use std::unordered_map instead of std::map

[5359] https://github.com/TheHPXProject/hpx/pull/1810
[5360] https://github.com/TheHPXProject/hpx/pull/1809
[5361] https://github.com/TheHPXProject/hpx/pull/1808
[5362] https://github.com/TheHPXProject/hpx/pull/1807
[5363] https://github.com/TheHPXProject/hpx/issues/1806
[5364] https://github.com/TheHPXProject/hpx/issues/1804
[5365] https://github.com/TheHPXProject/hpx/pull/1801
[5366] https://github.com/TheHPXProject/hpx/issues/1800
[5367] https://github.com/TheHPXProject/hpx/pull/1799
[5368] https://github.com/TheHPXProject/hpx/pull/1798
[5369] https://github.com/TheHPXProject/hpx/pull/1797
[5370] https://github.com/TheHPXProject/hpx/pull/1795
[5371] https://github.com/TheHPXProject/hpx/pull/1794
[5372] https://github.com/TheHPXProject/hpx/pull/1793
[5373] https://github.com/TheHPXProject/hpx/pull/1792
[5374] https://github.com/TheHPXProject/hpx/pull/1791
[5375] https://github.com/TheHPXProject/hpx/issues/1790
[5376] https://github.com/TheHPXProject/hpx/pull/1789
[5377] https://github.com/TheHPXProject/hpx/issues/1788
[5378] https://github.com/TheHPXProject/hpx/issues/1787
[5379] https://github.com/TheHPXProject/hpx/pull/1786

- PR #1785[5380] - Fix is_iterator for container algorithms
- PR #1784[5381] - Adding new command line options:
- PR #1783[5382] - Minor changes for APEX support
- PR #1782[5383] - Drop legacy forwarding action traits
- PR #1781[5384] - Attempt to resolve the race between cv::wait_xxx and cv::notify_all
- PR #1780[5385] - Removing serialize_sequence
- PR #1779[5386] - Fixed #1501: hwloc configuration options are wrong for MIC
- PR #1778[5387] - Removing ability to enable/disable parcel handling
- PR #1777[5388] - Completely removing stackless threads
- PR #1776[5389] - Cleaning up util/plugin
- PR #1775[5390] - Agas fixes
- PR #1774[5391] - Action invocation count
- PR #1773[5392] - replaced MSVC variable with WIN32
- PR #1772[5393] - Fixing Problems in MPI parcelport and future serialization.
- PR #1771[5394] - Fixing intel 13 compiler errors related to variadic template template parameters for `lcos::when_` tests
- PR #1770[5395] - Forwarding decay to `std::`
- PR #1769[5396] - Add more characters with special regex meaning to the existing patch
- PR #1768[5397] - Adding test for receive_buffer
- PR #1767[5398] - Making sure that uptime counter throws exception on any attempt to be reset
- PR #1766[5399] - Cleaning up code related to throttling scheduler
- PR #1765[5400] - Restricting thread_data to creating only with intrusive_pointers
- PR #1764[5401] - Fixing 1763

[5380] https://github.com/TheHPXProject/hpx/pull/1785
[5381] https://github.com/TheHPXProject/hpx/pull/1784
[5382] https://github.com/TheHPXProject/hpx/pull/1783
[5383] https://github.com/TheHPXProject/hpx/pull/1782
[5384] https://github.com/TheHPXProject/hpx/pull/1781
[5385] https://github.com/TheHPXProject/hpx/pull/1780
[5386] https://github.com/TheHPXProject/hpx/pull/1779
[5387] https://github.com/TheHPXProject/hpx/pull/1778
[5388] https://github.com/TheHPXProject/hpx/pull/1777
[5389] https://github.com/TheHPXProject/hpx/pull/1776
[5390] https://github.com/TheHPXProject/hpx/pull/1775
[5391] https://github.com/TheHPXProject/hpx/pull/1774
[5392] https://github.com/TheHPXProject/hpx/pull/1773
[5393] https://github.com/TheHPXProject/hpx/pull/1772
[5394] https://github.com/TheHPXProject/hpx/pull/1771
[5395] https://github.com/TheHPXProject/hpx/pull/1770
[5396] https://github.com/TheHPXProject/hpx/pull/1769
[5397] https://github.com/TheHPXProject/hpx/pull/1768
[5398] https://github.com/TheHPXProject/hpx/pull/1767
[5399] https://github.com/TheHPXProject/hpx/pull/1766
[5400] https://github.com/TheHPXProject/hpx/pull/1765
[5401] https://github.com/TheHPXProject/hpx/pull/1764

- Issue #1763[5402] - UB in thread_data::operator delete
- PR #1762[5403] - Making sure all serialization registries/factories are unique
- PR #1761[5404] - Fixed #1751: hpx::future::wait_for fails a simple test
- PR #1758[5405] - Fixing #1757
- Issue #1757[5406] - pinning not correct using –hpx:bind
- Issue #1756[5407] - compilation error with MinGW
- PR #1755[5408] - Making output serialization const-correct
- Issue #1753[5409] - HPX performance degrades with time since execution begins
- Issue #1752[5410] - Error in AGAS
- Issue #1751[5411] - hpx::future::wait_for fails a simple test
- PR #1750[5412] - Removing hpx_fwd.hpp includes
- PR #1749[5413] - Simplify result_of and friends
- PR #1747[5414] - Removed superfluous code from message_buffer.hpp
- PR #1746[5415] - Tuple dependencies
- Issue #1745[5416] - Broken when_some which takes iterators
- PR #1744[5417] - Refining archive interface
- PR #1743[5418] - Fixing when_all when only a single future is passed
- PR #1742[5419] - Config includes
- PR #1741[5420] - Os executors
- Issue #1740[5421] - hpx::promise has some problems
- PR #1739[5422] - Parallel composition with generic containers
- Issue #1738[5423] - After building program and successfully linking to a version of hpx DHPX_DIR seems to be ignored
- Issue #1737[5424] - Uptime problems

[5402] https://github.com/TheHPXProject/hpx/issues/1763
[5403] https://github.com/TheHPXProject/hpx/pull/1762
[5404] https://github.com/TheHPXProject/hpx/pull/1761
[5405] https://github.com/TheHPXProject/hpx/pull/1758
[5406] https://github.com/TheHPXProject/hpx/issues/1757
[5407] https://github.com/TheHPXProject/hpx/issues/1756
[5408] https://github.com/TheHPXProject/hpx/pull/1755
[5409] https://github.com/TheHPXProject/hpx/issues/1753
[5410] https://github.com/TheHPXProject/hpx/issues/1752
[5411] https://github.com/TheHPXProject/hpx/issues/1751
[5412] https://github.com/TheHPXProject/hpx/pull/1750
[5413] https://github.com/TheHPXProject/hpx/pull/1749
[5414] https://github.com/TheHPXProject/hpx/pull/1747
[5415] https://github.com/TheHPXProject/hpx/pull/1746
[5416] https://github.com/TheHPXProject/hpx/issues/1745
[5417] https://github.com/TheHPXProject/hpx/pull/1744
[5418] https://github.com/TheHPXProject/hpx/pull/1743
[5419] https://github.com/TheHPXProject/hpx/pull/1742
[5420] https://github.com/TheHPXProject/hpx/pull/1741
[5421] https://github.com/TheHPXProject/hpx/issues/1740
[5422] https://github.com/TheHPXProject/hpx/pull/1739
[5423] https://github.com/TheHPXProject/hpx/issues/1738
[5424] https://github.com/TheHPXProject/hpx/issues/1737

- PR #1736[5425] - added convenience c-tor and begin()/end() to serialize_buffer
- PR #1735[5426] - Config includes
- PR #1734[5427] - Fixed #1688: Add timer counters for tfunc_total and exec_total
- Issue #1733[5428] - Add unit test for hpx/lcos/local/receive_buffer.hpp
- PR #1732[5429] - Renaming get_os_thread_count
- PR #1731[5430] - Basename registration
- Issue #1730[5431] - Use after move of thread_init_data
- PR #1729[5432] - Rewriting channel based on new gate component
- PR #1728[5433] - Fixing #1722
- PR #1727[5434] - Fixing compile problems with apply_colocated
- PR #1726[5435] - Apex integration
- PR #1725[5436] - fixed test timeouts
- PR #1724[5437] - Renaming vector
- Issue #1723[5438] - Drop support for intel compilers and gcc 4.4. based standard libs
- Issue #1722[5439] - Add support for detecting non-ready futures before serialization
- PR #1721[5440] - Unifying parallel executors, initializing from launch policy
- PR #1720[5441] - dropped superfluous typedef
- Issue #1718[5442] - Windows 10 x64, VS 2015 - Unknown CMake command "add_hpx_pseudo_target".
- PR #1717[5443] - Timed executor traits for thread-executors
- PR #1716[5444] - serialization of arrays didn't work with non-pod types. fixed
- PR #1715[5445] - List serialization
- PR #1714[5446] - changing misspellings
- PR #1713[5447] - Fixed distribution policy executors

5425 https://github.com/TheHPXProject/hpx/pull/1736
5426 https://github.com/TheHPXProject/hpx/pull/1735
5427 https://github.com/TheHPXProject/hpx/pull/1734
5428 https://github.com/TheHPXProject/hpx/issues/1733
5429 https://github.com/TheHPXProject/hpx/pull/1732
5430 https://github.com/TheHPXProject/hpx/pull/1731
5431 https://github.com/TheHPXProject/hpx/issues/1730
5432 https://github.com/TheHPXProject/hpx/pull/1729
5433 https://github.com/TheHPXProject/hpx/pull/1728
5434 https://github.com/TheHPXProject/hpx/pull/1727
5435 https://github.com/TheHPXProject/hpx/pull/1726
5436 https://github.com/TheHPXProject/hpx/pull/1725
5437 https://github.com/TheHPXProject/hpx/pull/1724
5438 https://github.com/TheHPXProject/hpx/issues/1723
5439 https://github.com/TheHPXProject/hpx/issues/1722
5440 https://github.com/TheHPXProject/hpx/pull/1721
5441 https://github.com/TheHPXProject/hpx/pull/1720
5442 https://github.com/TheHPXProject/hpx/issues/1718
5443 https://github.com/TheHPXProject/hpx/pull/1717
5444 https://github.com/TheHPXProject/hpx/pull/1716
5445 https://github.com/TheHPXProject/hpx/pull/1715
5446 https://github.com/TheHPXProject/hpx/pull/1714
5447 https://github.com/TheHPXProject/hpx/pull/1713

- PR #1712[5448] - Moving library detection to be executed after feature tests
- PR #1711[5449] - Simplify parcel
- PR #1710[5450] - Compile only tests
- PR #1709[5451] - Implemented timed executors
- PR #1708[5452] - Implement parallel::executor_traits for thread-executors
- PR #1707[5453] - Various fixes to threads::executors to make custom schedulers work
- PR #1706[5454] - Command line option –hpx:cores does not work as expected
- Issue #1705[5455] - command line option –hpx:cores does not work as expected
- PR #1704[5456] - vector deserialization is speeded up a little
- PR #1703[5457] - Fixing shared_mutes
- Issue #1702[5458] - Shared_mutex does not compile with no_mutex cond_var
- PR #1701[5459] - Add distribution_policy_executor
- PR #1700[5460] - Executor parameters
- PR #1699[5461] - Readers writer lock
- PR #1698[5462] - Remove leftovers
- PR #1697[5463] - Fixing held locks
- PR #1696[5464] - Modified Scan Partitioner for Algorithms
- PR #1695[5465] - This thread executors
- PR #1694[5466] - Fixed #1688: Add timer counters for tfunc_total and exec_total
- PR #1693[5467] - Fix #1691: is_executor template specification fails for inherited executors
- PR #1692[5468] - Fixed #1662: Possible exception source in coalescing_message_handler
- Issue #1691[5469] - is_executor template specification fails for inherited executors
- PR #1690[5470] - added macro for non-intrusive serialization of classes without a default c-tor

[5448] https://github.com/TheHPXProject/hpx/pull/1712
[5449] https://github.com/TheHPXProject/hpx/pull/1711
[5450] https://github.com/TheHPXProject/hpx/pull/1710
[5451] https://github.com/TheHPXProject/hpx/pull/1709
[5452] https://github.com/TheHPXProject/hpx/pull/1708
[5453] https://github.com/TheHPXProject/hpx/pull/1707
[5454] https://github.com/TheHPXProject/hpx/pull/1706
[5455] https://github.com/TheHPXProject/hpx/issues/1705
[5456] https://github.com/TheHPXProject/hpx/pull/1704
[5457] https://github.com/TheHPXProject/hpx/pull/1703
[5458] https://github.com/TheHPXProject/hpx/issues/1702
[5459] https://github.com/TheHPXProject/hpx/pull/1701
[5460] https://github.com/TheHPXProject/hpx/pull/1700
[5461] https://github.com/TheHPXProject/hpx/pull/1699
[5462] https://github.com/TheHPXProject/hpx/pull/1698
[5463] https://github.com/TheHPXProject/hpx/pull/1697
[5464] https://github.com/TheHPXProject/hpx/pull/1696
[5465] https://github.com/TheHPXProject/hpx/pull/1695
[5466] https://github.com/TheHPXProject/hpx/pull/1694
[5467] https://github.com/TheHPXProject/hpx/pull/1693
[5468] https://github.com/TheHPXProject/hpx/pull/1692
[5469] https://github.com/TheHPXProject/hpx/issues/1691
[5470] https://github.com/TheHPXProject/hpx/pull/1690

- PR #1689[5471] - Replace value_or_error with custom storage, unify future_data state
- Issue #1688[5472] - Add timer counters for tfunc_total and exec_total
- PR #1687[5473] - Fixed interval timer
- PR #1686[5474] - Fixing cmake warnings about not existing pseudo target dependencies
- PR #1685[5475] - Converting partitioners to use bulk async execute
- PR #1683[5476] - Adds a tool for inspect that checks for character limits
- PR #1682[5477] - Change project name to (uppercase) HPX
- PR #1681[5478] - Counter shortnames
- PR #1680[5479] - Extended Non-intrusive Serialization to Ease Usage for Library Developers
- PR #1679[5480] - Working on 1544: More executor changes
- PR #1678[5481] - Transpose fixes
- PR #1677[5482] - Improve Boost compatibility check
- PR #1676[5483] - 1d stencil fix
- Issue #1675[5484] - hpx project name is not HPX
- PR #1674[5485] - Fixing the MPI parcelport
- PR #1673[5486] - added move semantics to map/vector deserialization
- PR #1672[5487] - Vs2015 await
- PR #1671[5488] - Adapt transform for #1668
- PR #1670[5489] - Started to work on #1668
- PR #1669[5490] - Add this_thread_executors
- Issue #1667[5491] - Apple build instructions in docs are out of date
- PR #1666[5492] - Apex integration
- PR #1665[5493] - Fixes an error with the whitespace check that showed the incorrect loca-

[5471] https://github.com/TheHPXProject/hpx/pull/1689
[5472] https://github.com/TheHPXProject/hpx/issues/1688
[5473] https://github.com/TheHPXProject/hpx/pull/1687
[5474] https://github.com/TheHPXProject/hpx/pull/1686
[5475] https://github.com/TheHPXProject/hpx/pull/1685
[5476] https://github.com/TheHPXProject/hpx/pull/1683
[5477] https://github.com/TheHPXProject/hpx/pull/1682
[5478] https://github.com/TheHPXProject/hpx/pull/1681
[5479] https://github.com/TheHPXProject/hpx/pull/1680
[5480] https://github.com/TheHPXProject/hpx/pull/1679
[5481] https://github.com/TheHPXProject/hpx/pull/1678
[5482] https://github.com/TheHPXProject/hpx/pull/1677
[5483] https://github.com/TheHPXProject/hpx/pull/1676
[5484] https://github.com/TheHPXProject/hpx/issues/1675
[5485] https://github.com/TheHPXProject/hpx/pull/1674
[5486] https://github.com/TheHPXProject/hpx/pull/1673
[5487] https://github.com/TheHPXProject/hpx/pull/1672
[5488] https://github.com/TheHPXProject/hpx/pull/1671
[5489] https://github.com/TheHPXProject/hpx/pull/1670
[5490] https://github.com/TheHPXProject/hpx/pull/1669
[5491] https://github.com/TheHPXProject/hpx/issues/1667
[5492] https://github.com/TheHPXProject/hpx/pull/1666
[5493] https://github.com/TheHPXProject/hpx/pull/1665

tion of the error

- Issue #1664[5494] - Inspect tool found incorrect endline whitespace
- PR #1663[5495] - Improve use of locks
- Issue #1662[5496] - Possible exception source in coalescing_message_handler
- PR #1661[5497] - Added support for 128bit number serialization
- PR #1660[5498] - Serialization 128bits
- PR #1659[5499] - Implemented inner_product and adjacent_diff algos
- PR #1658[5500] - Add serialization for std::set (as there is for std::vector and std::map)
- PR #1657[5501] - Use of shared_ptr in io_service_pool changed to unique_ptr
- Issue #1656[5502] - 1d_stencil codes all have wrong factor
- PR #1654[5503] - When using runtime_mode_connect, find the correct localhost public ip address
- PR #1653[5504] - Fixing 1617
- PR #1652[5505] - Remove traits::action_may_require_id_splitting
- PR #1651[5506] - Fixed performance counters related to AGAS cache timings
- PR #1650[5507] - Remove leftovers of traits::type_size
- PR #1649[5508] - Shorten target names on Windows to shorten used path names
- PR #1648[5509] - Fixing problems introduced by merging #1623 for older compilers
- PR #1647[5510] - Simplify running automatic builds on Windows
- Issue #1646[5511] - Cache insert and update performance counters are broken
- Issue #1644[5512] - Remove leftovers of traits::type_size
- Issue #1643[5513] - Remove traits::action_may_require_id_splitting
- PR #1642[5514] - Adds spell checker to the inspect tool for qbk and doxygen comments
- PR #1640[5515] - First step towards fixing 688

[5494] https://github.com/TheHPXProject/hpx/issues/1664
[5495] https://github.com/TheHPXProject/hpx/pull/1663
[5496] https://github.com/TheHPXProject/hpx/issues/1662
[5497] https://github.com/TheHPXProject/hpx/pull/1661
[5498] https://github.com/TheHPXProject/hpx/pull/1660
[5499] https://github.com/TheHPXProject/hpx/pull/1659
[5500] https://github.com/TheHPXProject/hpx/pull/1658
[5501] https://github.com/TheHPXProject/hpx/pull/1657
[5502] https://github.com/TheHPXProject/hpx/issues/1656
[5503] https://github.com/TheHPXProject/hpx/pull/1654
[5504] https://github.com/TheHPXProject/hpx/pull/1653
[5505] https://github.com/TheHPXProject/hpx/pull/1652
[5506] https://github.com/TheHPXProject/hpx/pull/1651
[5507] https://github.com/TheHPXProject/hpx/pull/1650
[5508] https://github.com/TheHPXProject/hpx/pull/1649
[5509] https://github.com/TheHPXProject/hpx/pull/1648
[5510] https://github.com/TheHPXProject/hpx/pull/1647
[5511] https://github.com/TheHPXProject/hpx/issues/1646
[5512] https://github.com/TheHPXProject/hpx/issues/1644
[5513] https://github.com/TheHPXProject/hpx/issues/1643
[5514] https://github.com/TheHPXProject/hpx/pull/1642
[5515] https://github.com/TheHPXProject/hpx/pull/1640

- PR #1639[5516] - Re-apply remaining changes from limit_dataflow_recursion branch
- PR #1638[5517] - This fixes possible deadlock in the test ignore_while_locked_1485
- PR #1637[5518] - Fixing hpx::wait_all() invoked with two vector<future<T>>
- PR #1636[5519] - Partially re-apply changes from limit_dataflow_recursion branch
- PR #1635[5520] - Adding missing test for #1572
- PR #1634[5521] - Revert "Limit recursion-depth in dataflow to a configurable constant"
- PR #1633[5522] - Add command line option to ignore batch environment
- PR #1631[5523] - hpx::lcos::queue exhibits strange behavior
- PR #1630[5524] - Fixed endline_whitespace_check.cpp to detect lines with only whitespace
- Issue #1629[5525] - Inspect trailing whitespace checker problem
- PR #1628[5526] - Removed meaningless const qualifiers. Minor icpc fix.
- PR #1627[5527] - Fixing the queue LCO and add example demonstrating its use
- PR #1626[5528] - Deprecating get_gid(), add get_id() and get_unmanaged_id()
- PR #1625[5529] - Allowing to specify whether to send credits along with message
- Issue #1624[5530] - Lifetime issue
- Issue #1623[5531] - hpx::wait_all() invoked with two vector<future<T>> fails
- PR #1622[5532] - Executor partitioners
- PR #1621[5533] - Clean up coroutines implementation
- Issue #1620[5534] - Revert #1535
- PR #1619[5535] - Fix result type calculation for hpx::make_continuation
- PR #1618[5536] - Fixing RDTSC on Xeon/Phi
- Issue #1617[5537] - hpx cmake not working when run as a subproject
- Issue #1616[5538] - cmake problem resulting in RDTSC not working correctly for Xeon

[5516] https://github.com/TheHPXProject/hpx/pull/1639
[5517] https://github.com/TheHPXProject/hpx/pull/1638
[5518] https://github.com/TheHPXProject/hpx/pull/1637
[5519] https://github.com/TheHPXProject/hpx/pull/1636
[5520] https://github.com/TheHPXProject/hpx/pull/1635
[5521] https://github.com/TheHPXProject/hpx/pull/1634
[5522] https://github.com/TheHPXProject/hpx/pull/1633
[5523] https://github.com/TheHPXProject/hpx/pull/1631
[5524] https://github.com/TheHPXProject/hpx/pull/1630
[5525] https://github.com/TheHPXProject/hpx/issues/1629
[5526] https://github.com/TheHPXProject/hpx/pull/1628
[5527] https://github.com/TheHPXProject/hpx/pull/1627
[5528] https://github.com/TheHPXProject/hpx/pull/1626
[5529] https://github.com/TheHPXProject/hpx/pull/1625
[5530] https://github.com/TheHPXProject/hpx/issues/1624
[5531] https://github.com/TheHPXProject/hpx/issues/1623
[5532] https://github.com/TheHPXProject/hpx/pull/1622
[5533] https://github.com/TheHPXProject/hpx/pull/1621
[5534] https://github.com/TheHPXProject/hpx/issues/1620
[5535] https://github.com/TheHPXProject/hpx/pull/1619
[5536] https://github.com/TheHPXProject/hpx/pull/1618
[5537] https://github.com/TheHPXProject/hpx/issues/1617
[5538] https://github.com/TheHPXProject/hpx/issues/1616

Phi creates very strange results for duration counters

- Issue #1615[5539] - hpx::make_continuation requires input and output to be the same
- PR #1614[5540] - Fixed remove copy test
- Issue #1613[5541] - Dataflow causes stack overflow
- PR #1612[5542] - Modified foreach partitioner to use bulk execute
- PR #1611[5543] - Limit recursion-depth in dataflow to a configurable constant
- PR #1610[5544] - Increase timeout for CircleCI
- PR #1609[5545] - Refactoring thread manager, mainly extracting thread pool
- PR #1608[5546] - Fixed running multiple localities without localities parameter
- PR #1607[5547] - More algorithm fixes to adjacentfind
- Issue #1606[5548] - Running without localities parameter binds to bogus port range
- Issue #1605[5549] - Too many serializations
- PR #1604[5550] - Changes the HPX image into a hyperlink
- PR #1601[5551] - Fixing problems with remove_copy algorithm tests
- PR #1600[5552] - Actions with ids cleanup
- PR #1599[5553] - Duplicate binding of global ids should fail
- PR #1598[5554] - Fixing array access
- PR #1597[5555] - Improved the reliability of connecting/disconnecting localities
- Issue #1596[5556] - Duplicate id binding should fail
- PR #1595[5557] - Fixing more cmake config constants
- PR #1594[5558] - Fixing preprocessor constant used to enable C++11 chrono
- PR #1593[5559] - Adding operator|() for hpx::launch
- Issue #1592[5560] - Error (typo) in the docs

5539 https://github.com/TheHPXProject/hpx/issues/1615
5540 https://github.com/TheHPXProject/hpx/pull/1614
5541 https://github.com/TheHPXProject/hpx/issues/1613
5542 https://github.com/TheHPXProject/hpx/pull/1612
5543 https://github.com/TheHPXProject/hpx/pull/1611
5544 https://github.com/TheHPXProject/hpx/pull/1610
5545 https://github.com/TheHPXProject/hpx/pull/1609
5546 https://github.com/TheHPXProject/hpx/pull/1608
5547 https://github.com/TheHPXProject/hpx/pull/1607
5548 https://github.com/TheHPXProject/hpx/issues/1606
5549 https://github.com/TheHPXProject/hpx/issues/1605
5550 https://github.com/TheHPXProject/hpx/pull/1604
5551 https://github.com/TheHPXProject/hpx/pull/1601
5552 https://github.com/TheHPXProject/hpx/pull/1600
5553 https://github.com/TheHPXProject/hpx/pull/1599
5554 https://github.com/TheHPXProject/hpx/pull/1598
5555 https://github.com/TheHPXProject/hpx/pull/1597
5556 https://github.com/TheHPXProject/hpx/issues/1596
5557 https://github.com/TheHPXProject/hpx/pull/1595
5558 https://github.com/TheHPXProject/hpx/pull/1594
5559 https://github.com/TheHPXProject/hpx/pull/1593
5560 https://github.com/TheHPXProject/hpx/issues/1592

- Issue #1590[5561] - CMake fails when CMAKE_BINARY_DIR contains '+'.
- Issue #1589[5562] - Disconnecting a locality results in segfault using heartbeat example
- PR #1588[5563] - Fix doc string for config option HPX_WITH_EXAMPLES
- PR #1586[5564] - Fixing 1493
- PR #1585[5565] - Additional Check for Inspect Tool to detect Endline Whitespace
- Issue #1584[5566] - Clean up coroutines implementation
- PR #1583[5567] - Adding a check for end line whitespace
- PR #1582[5568] - Attempt to fix assert firing after scheduling loop was exited
- PR #1581[5569] - Fixed adjacentfind_binary test
- PR #1580[5570] - Prevent some of the internal cmake lists from growing indefinitely
- PR #1579[5571] - Removing type_size trait, replacing it with special archive type
- Issue #1578[5572] - Remove demangle_helper
- PR #1577[5573] - Get ptr problems
- Issue #1576[5574] - Refactor async, dataflow, and future::then
- PR #1575[5575] - Fixing tests for parallel rotate
- PR #1574[5576] - Cleaning up schedulers
- PR #1573[5577] - Fixing thread pool executor
- PR #1572[5578] - Fixing number of configured localities
- PR #1571[5579] - Reimplement decay
- PR #1570[5580] - Refactoring async, apply, and dataflow APIs
- PR #1569[5581] - Changed range for mach-o library lookup
- PR #1568[5582] - Mark decltype support as required
- PR #1567[5583] - Removed const from algorithms

5561 https://github.com/TheHPXProject/hpx/issues/1590
5562 https://github.com/TheHPXProject/hpx/issues/1589
5563 https://github.com/TheHPXProject/hpx/pull/1588
5564 https://github.com/TheHPXProject/hpx/pull/1586
5565 https://github.com/TheHPXProject/hpx/pull/1585
5566 https://github.com/TheHPXProject/hpx/issues/1584
5567 https://github.com/TheHPXProject/hpx/pull/1583
5568 https://github.com/TheHPXProject/hpx/pull/1582
5569 https://github.com/TheHPXProject/hpx/pull/1581
5570 https://github.com/TheHPXProject/hpx/pull/1580
5571 https://github.com/TheHPXProject/hpx/pull/1579
5572 https://github.com/TheHPXProject/hpx/issues/1578
5573 https://github.com/TheHPXProject/hpx/pull/1577
5574 https://github.com/TheHPXProject/hpx/issues/1576
5575 https://github.com/TheHPXProject/hpx/pull/1575
5576 https://github.com/TheHPXProject/hpx/pull/1574
5577 https://github.com/TheHPXProject/hpx/pull/1573
5578 https://github.com/TheHPXProject/hpx/pull/1572
5579 https://github.com/TheHPXProject/hpx/pull/1571
5580 https://github.com/TheHPXProject/hpx/pull/1570
5581 https://github.com/TheHPXProject/hpx/pull/1569
5582 https://github.com/TheHPXProject/hpx/pull/1568
5583 https://github.com/TheHPXProject/hpx/pull/1567

- Issue #1566[5584] - CMAKE Configuration Test Failures for clang 3.5 on debian
- PR #1565[5585] - Dylib support
- PR #1564[5586] - Converted partitioners and some algorithms to use executors
- PR #1563[5587] - Fix several #includes for Boost.Preprocessor
- PR #1562[5588] - Adding configuration option disabling/enabling all message handlers
- PR #1561[5589] - Removed all occurrences of boost::move replacing it with std::move
- Issue #1560[5590] - Leftover HPX_REGISTER_ACTION_DECLARATION_2
- PR #1558[5591] - Revisit async/apply SFINAE conditions
- PR #1557[5592] - Removing type_size trait, replacing it with special archive type
- PR #1556[5593] - Executor algorithms
- PR #1555[5594] - Remove the necessity to specify archive flags on the receiving end
- PR #1554[5595] - Removing obsolete Boost.Serialization macros
- PR #1553[5596] - Properly fix HPX_DEFINE_*_ACTION macros
- PR #1552[5597] - Fixed algorithms relying on copy_if implementation
- PR #1551[5598] - Pxfs - Modifying FindOrangeFS.cmake based on OrangeFS 2.9.X
- Issue #1550[5599] - Passing plain identifier inside HPX_DEFINE_PLAIN_ACTION_1
- PR #1549[5600] - Fixing intel14/libstdc++4.4
- PR #1548[5601] - Moving raw_ptr to detail namespace
- PR #1547[5602] - Adding support for executors to future.then
- PR #1546[5603] - Executor traits result types
- PR #1545[5604] - Integrate executors with dataflow
- PR #1543[5605] - Fix potential zero-copy for primarynamespace::bulk_service_async et.al.
- PR #1542[5606] - Merging HPX0.9.10 into pxfs branch

[5584] https://github.com/TheHPXProject/hpx/issues/1566
[5585] https://github.com/TheHPXProject/hpx/pull/1565
[5586] https://github.com/TheHPXProject/hpx/pull/1564
[5587] https://github.com/TheHPXProject/hpx/pull/1563
[5588] https://github.com/TheHPXProject/hpx/pull/1562
[5589] https://github.com/TheHPXProject/hpx/pull/1561
[5590] https://github.com/TheHPXProject/hpx/issues/1560
[5591] https://github.com/TheHPXProject/hpx/pull/1558
[5592] https://github.com/TheHPXProject/hpx/pull/1557
[5593] https://github.com/TheHPXProject/hpx/pull/1556
[5594] https://github.com/TheHPXProject/hpx/pull/1555
[5595] https://github.com/TheHPXProject/hpx/pull/1554
[5596] https://github.com/TheHPXProject/hpx/pull/1553
[5597] https://github.com/TheHPXProject/hpx/pull/1552
[5598] https://github.com/TheHPXProject/hpx/pull/1551
[5599] https://github.com/TheHPXProject/hpx/issues/1550
[5600] https://github.com/TheHPXProject/hpx/pull/1549
[5601] https://github.com/TheHPXProject/hpx/pull/1548
[5602] https://github.com/TheHPXProject/hpx/pull/1547
[5603] https://github.com/TheHPXProject/hpx/pull/1546
[5604] https://github.com/TheHPXProject/hpx/pull/1545
[5605] https://github.com/TheHPXProject/hpx/pull/1543
[5606] https://github.com/TheHPXProject/hpx/pull/1542

- PR #1541[5607] - Removed stale cmake tests, unused since the great cmake refactoring
- PR #1540[5608] - Fix idle-rate on platforms without TSC
- PR #1539[5609] - Reporting situation if zero-copy-serialization was performed by a parcel generated from a plain apply/async
- PR #1538[5610] - Changed return type of bulk executors and added test
- Issue #1537[5611] - Incorrect cpuid config tests
- PR #1536[5612] - Changed return type of bulk executors and added test
- PR #1535[5613] - Make sure promise::get_gid() can be called more than once
- PR #1534[5614] - Fixed async_callback with bound callback
- PR #1533[5615] - Updated the link in the documentation to a publically- accessible URL
- PR #1532[5616] - Make sure sync primitives are not copyable nor movable
- PR #1531[5617] - Fix unwrapped issue with future ranges of void type
- PR #1530[5618] - Serialization complex
- Issue #1528[5619] - Unwrapped issue with future<void>
- Issue #1527[5620] - HPX does not build with Boost 1.58.0
- PR #1526[5621] - Added support for boost.multi_array serialization
- PR #1525[5622] - Properly handle deferred futures, fixes #1506
- PR #1524[5623] - Making sure invalid action argument types generate clear error message
- Issue #1522[5624] - Need serialization support for boost multi array
- Issue #1521[5625] - Remote async and zero-copy serialization optimizations don't play well together
- PR #1520[5626] - Fixing UB whil registering polymorphic classes for serialization
- PR #1519[5627] - Making detail::condition_variable safe to use
- PR #1518[5628] - Fix when_some bug missing indices in its result

[5607] https://github.com/TheHPXProject/hpx/pull/1541
[5608] https://github.com/TheHPXProject/hpx/pull/1540
[5609] https://github.com/TheHPXProject/hpx/pull/1539
[5610] https://github.com/TheHPXProject/hpx/pull/1538
[5611] https://github.com/TheHPXProject/hpx/issues/1537
[5612] https://github.com/TheHPXProject/hpx/pull/1536
[5613] https://github.com/TheHPXProject/hpx/pull/1535
[5614] https://github.com/TheHPXProject/hpx/pull/1534
[5615] https://github.com/TheHPXProject/hpx/pull/1533
[5616] https://github.com/TheHPXProject/hpx/pull/1532
[5617] https://github.com/TheHPXProject/hpx/pull/1531
[5618] https://github.com/TheHPXProject/hpx/pull/1530
[5619] https://github.com/TheHPXProject/hpx/issues/1528
[5620] https://github.com/TheHPXProject/hpx/issues/1527
[5621] https://github.com/TheHPXProject/hpx/pull/1526
[5622] https://github.com/TheHPXProject/hpx/pull/1525
[5623] https://github.com/TheHPXProject/hpx/pull/1524
[5624] https://github.com/TheHPXProject/hpx/issues/1522
[5625] https://github.com/TheHPXProject/hpx/issues/1521
[5626] https://github.com/TheHPXProject/hpx/pull/1520
[5627] https://github.com/TheHPXProject/hpx/pull/1519
[5628] https://github.com/TheHPXProject/hpx/pull/1518

- Issue #1517[5629] - Typo may affect CMake build system tests
- PR #1516[5630] - Fixing Posix context
- PR #1515[5631] - Fixing Posix context
- PR #1514[5632] - Correct problems with loading dynamic components
- PR #1513[5633] - Fixing intel glibc4 4
- Issue #1508[5634] - memory and papi counters do not work
- Issue #1507[5635] - Unrecognized Command Line Option Error causing exit status 0
- Issue #1506[5636] - Properly handle deferred futures
- PR #1505[5637] - Adding #include - would not compile without this
- Issue #1502[5638] - `boost::filesystem::exists` throws unexpected exception
- Issue #1501[5639] - hwloc configuration options are wrong for MIC
- PR #1504[5640] - Making sure boost::filesystem::exists() does not throw
- PR #1500[5641] - Exit application on `--hpx:version`/`-v` and `--hpx:info`
- PR #1498[5642] - Extended task block
- PR #1497[5643] - Unique ptr serialization
- PR #1496[5644] - Unique ptr serialization (closed)
- PR #1495[5645] - Switching circleci build type to debug
- Issue #1494[5646] - `--hpx:version`/`-v` does not exit after printing version information
- Issue #1493[5647] - add an `hpx_` prefix to libraries and components to avoid name conflicts
- Issue #1492[5648] - Define and ensure limitations for arguments to async/apply
- PR #1489[5649] - Enable idle rate counter on demand
- PR #1488[5650] - Made sure `detail::condition_variable` can be safely destroyed
- PR #1487[5651] - Introduced default (main) template implementation for `ignore_while_checking`

[5629] https://github.com/TheHPXProject/hpx/issues/1517
[5630] https://github.com/TheHPXProject/hpx/pull/1516
[5631] https://github.com/TheHPXProject/hpx/pull/1515
[5632] https://github.com/TheHPXProject/hpx/pull/1514
[5633] https://github.com/TheHPXProject/hpx/pull/1513
[5634] https://github.com/TheHPXProject/hpx/issues/1508
[5635] https://github.com/TheHPXProject/hpx/issues/1507
[5636] https://github.com/TheHPXProject/hpx/issues/1506
[5637] https://github.com/TheHPXProject/hpx/pull/1505
[5638] https://github.com/TheHPXProject/hpx/issues/1502
[5639] https://github.com/TheHPXProject/hpx/issues/1501
[5640] https://github.com/TheHPXProject/hpx/pull/1504
[5641] https://github.com/TheHPXProject/hpx/pull/1500
[5642] https://github.com/TheHPXProject/hpx/pull/1498
[5643] https://github.com/TheHPXProject/hpx/pull/1497
[5644] https://github.com/TheHPXProject/hpx/pull/1496
[5645] https://github.com/TheHPXProject/hpx/pull/1495
[5646] https://github.com/TheHPXProject/hpx/issues/1494
[5647] https://github.com/TheHPXProject/hpx/issues/1493
[5648] https://github.com/TheHPXProject/hpx/issues/1492
[5649] https://github.com/TheHPXProject/hpx/pull/1489
[5650] https://github.com/TheHPXProject/hpx/pull/1488
[5651] https://github.com/TheHPXProject/hpx/pull/1487

- PR #1486[5652] - Add HPX inspect tool
- Issue #1485[5653] - `ignore_while_locked` doesn't support all Lockable types
- PR #1484[5654] - Docker image generation
- PR #1483[5655] - Move external endian library into HPX
- PR #1482[5656] - Actions with integer type ids
- Issue #1481[5657] - Sync primitives safe destruction
- Issue #1480[5658] - Move external/boost/endian into hpx/util
- Issue #1478[5659] - Boost inspect violations
- PR #1479[5660] - Adds serialization for arrays; some further/minor fixes
- PR #1477[5661] - Fixing problems with the Intel compiler using a GCC 4.4 std library
- PR #1476[5662] - Adding `hpx::lcos::latch` and `hpx::lcos::local::latch`
- Issue #1475[5663] - Boost inspect violations
- PR #1473[5664] - Fixing action move tests
- Issue #1471[5665] - Sync primitives should not be movable
- PR #1470[5666] - Removing `hpx::util::polymorphic_factory`
- PR #1468[5667] - Fixed container creation
- Issue #1467[5668] - HPX application fail during finalization
- Issue #1466[5669] - HPX doesn't pick up Torque's nodefile on SuperMIC
- Issue #1464[5670] - HPX option for pre and post bootstrap performance counters
- PR #1463[5671] - Replacing `async_colocated(id, ...)` with `async(colocated(id), ...)`
- PR #1462[5672] - Consolidated task_region with N4411
- PR #1461[5673] - Consolidate inconsistent CMake option names
- Issue #1460[5674] - Which malloc is actually used? or at least which one is HPX built with

[5652] https://github.com/TheHPXProject/hpx/pull/1486
[5653] https://github.com/TheHPXProject/hpx/issues/1485
[5654] https://github.com/TheHPXProject/hpx/pull/1484
[5655] https://github.com/TheHPXProject/hpx/pull/1483
[5656] https://github.com/TheHPXProject/hpx/pull/1482
[5657] https://github.com/TheHPXProject/hpx/issues/1481
[5658] https://github.com/TheHPXProject/hpx/issues/1480
[5659] https://github.com/TheHPXProject/hpx/issues/1478
[5660] https://github.com/TheHPXProject/hpx/pull/1479
[5661] https://github.com/TheHPXProject/hpx/pull/1477
[5662] https://github.com/TheHPXProject/hpx/pull/1476
[5663] https://github.com/TheHPXProject/hpx/issues/1475
[5664] https://github.com/TheHPXProject/hpx/pull/1473
[5665] https://github.com/TheHPXProject/hpx/issues/1471
[5666] https://github.com/TheHPXProject/hpx/pull/1470
[5667] https://github.com/TheHPXProject/hpx/pull/1468
[5668] https://github.com/TheHPXProject/hpx/issues/1467
[5669] https://github.com/TheHPXProject/hpx/issues/1466
[5670] https://github.com/TheHPXProject/hpx/issues/1464
[5671] https://github.com/TheHPXProject/hpx/pull/1463
[5672] https://github.com/TheHPXProject/hpx/pull/1462
[5673] https://github.com/TheHPXProject/hpx/pull/1461
[5674] https://github.com/TheHPXProject/hpx/issues/1460

- Issue #1459[5675] - Make cmake configure step fail explicitly if compiler version is not supported
- Issue #1458[5676] - Update `parallel::task_region` with N4411
- PR #1456[5677] - Consolidating `new_<>()`
- Issue #1455[5678] - Replace `async_colocated(id, ...)` with `async(colocated(id), ...)`
- PR #1454[5679] - Removed harmful std::moves from return statements
- PR #1453[5680] - Use range-based for-loop instead of Boost.Foreach
- PR #1452[5681] - C++ feature tests
- PR #1451[5682] - When serializing, pass archive flags to traits::get_type_size
- Issue #1450[5683] - traits:get_type_size needs archive flags to enable zero_copy optimizations
- Issue #1449[5684] - "couldn't create performance counter" - AGAS
- Issue #1448[5685] - Replace distributing factories with `new_<T[]>(...)`
- PR #1447[5686] - Removing obsolete remote_object component
- PR #1446[5687] - Hpx serialization
- PR #1445[5688] - Replacing travis with circleci
- PR #1443[5689] - Always stripping HPX command line arguments before executing start function
- PR #1442[5690] - Adding –hpx:bind=none to disable thread affinities
- Issue #1439[5691] - Libraries get linked in multiple times, RPATH is not properly set
- PR #1438[5692] - Removed superfluous typedefs
- Issue #1437[5693] - `hpx::init()` should strip HPX-related flags from argv
- Issue #1436[5694] - Add strong scaling option to htts
- PR #1435[5695] - Adding `async_cb`, `async_continue_cb`, and `async_colocated_cb`

[5675] https://github.com/TheHPXProject/hpx/issues/1459
[5676] https://github.com/TheHPXProject/hpx/issues/1458
[5677] https://github.com/TheHPXProject/hpx/pull/1456
[5678] https://github.com/TheHPXProject/hpx/issues/1455
[5679] https://github.com/TheHPXProject/hpx/pull/1454
[5680] https://github.com/TheHPXProject/hpx/pull/1453
[5681] https://github.com/TheHPXProject/hpx/pull/1452
[5682] https://github.com/TheHPXProject/hpx/pull/1451
[5683] https://github.com/TheHPXProject/hpx/issues/1450
[5684] https://github.com/TheHPXProject/hpx/issues/1449
[5685] https://github.com/TheHPXProject/hpx/issues/1448
[5686] https://github.com/TheHPXProject/hpx/pull/1447
[5687] https://github.com/TheHPXProject/hpx/pull/1446
[5688] https://github.com/TheHPXProject/hpx/pull/1445
[5689] https://github.com/TheHPXProject/hpx/pull/1443
[5690] https://github.com/TheHPXProject/hpx/pull/1442
[5691] https://github.com/TheHPXProject/hpx/issues/1439
[5692] https://github.com/TheHPXProject/hpx/pull/1438
[5693] https://github.com/TheHPXProject/hpx/issues/1437
[5694] https://github.com/TheHPXProject/hpx/issues/1436
[5695] https://github.com/TheHPXProject/hpx/pull/1435

- PR #1434[5696] - Added missing install rule, removed some dead CMake code
- PR #1433[5697] - Add GitExternal and SubProject cmake scripts from eyescale/cmake repo
- Issue #1432[5698] - Add command line flag to disable thread pinning
- PR #1431[5699] - Fix #1423
- Issue #1430[5700] - Inconsistent CMake option names
- Issue #1429[5701] - Configure setting `HPX_HAVE_PARCELPORT_MPI` is ignored
- PR #1428[5702] - Fixes #1419 (closed)
- PR #1427[5703] - Adding stencil_iterator and transform_iterator
- PR #1426[5704] - Fixes #1419
- PR #1425[5705] - During serialization memory allocation should honour allocator chunk size
- Issue #1424[5706] - chunk allocation during serialization does not use memory pool/allocator chunk size
- Issue #1423[5707] - Remove `HPX_STD_UNIQUE_PTR`
- Issue #1422[5708] - hpx:threads=all allocates too many os threads
- PR #1420[5709] - added .travis.yml
- Issue #1419[5710] - Unify enums: `hpx::runtime::state` and `hpx::state`
- PR #1416[5711] - Adding travis builder
- Issue #1414[5712] - Correct directory for dispatch_gcc46.hpp iteration
- Issue #1410[5713] - Set operation algorithms
- Issue #1389[5714] - Parallel algorithms relying on scan partitioner break for small number of elements
- Issue #1325[5715] - Exceptions thrown during parcel handling are not handled correctly
- Issue #1315[5716] - Errors while running performance tests
- Issue #1309[5717] - `hpx::vector` partitions are not easily extendable by applications

[5696] https://github.com/TheHPXProject/hpx/pull/1434
[5697] https://github.com/TheHPXProject/hpx/pull/1433
[5698] https://github.com/TheHPXProject/hpx/issues/1432
[5699] https://github.com/TheHPXProject/hpx/pull/1431
[5700] https://github.com/TheHPXProject/hpx/issues/1430
[5701] https://github.com/TheHPXProject/hpx/issues/1429
[5702] https://github.com/TheHPXProject/hpx/pull/1428
[5703] https://github.com/TheHPXProject/hpx/pull/1427
[5704] https://github.com/TheHPXProject/hpx/pull/1426
[5705] https://github.com/TheHPXProject/hpx/pull/1425
[5706] https://github.com/TheHPXProject/hpx/issues/1424
[5707] https://github.com/TheHPXProject/hpx/issues/1423
[5708] https://github.com/TheHPXProject/hpx/issues/1422
[5709] https://github.com/TheHPXProject/hpx/pull/1420
[5710] https://github.com/TheHPXProject/hpx/issues/1419
[5711] https://github.com/TheHPXProject/hpx/pull/1416
[5712] https://github.com/TheHPXProject/hpx/issues/1414
[5713] https://github.com/TheHPXProject/hpx/issues/1410
[5714] https://github.com/TheHPXProject/hpx/issues/1389
[5715] https://github.com/TheHPXProject/hpx/issues/1325
[5716] https://github.com/TheHPXProject/hpx/issues/1315
[5717] https://github.com/TheHPXProject/hpx/issues/1309

- PR #1300[5718] - Added serialization/de-serialization to examples.tuplespace
- Issue #1251[5719] - hpx::threads::get_thread_count doesn't consider pending threads
- Issue #1008[5720] - Decrease in application performance overtime; occasional spikes of major slowdown
- Issue #1001[5721] - Zero copy serialization raises assert
- Issue #721[5722] - Make HPX usable for Xeon Phi
- Issue #524[5723] - Extend scheduler to support threads which can't be stolen

HPX V0.9.10 (Mar 24, 2015)

General changes

This is the 12th official release of *HPX*. It coincides with the 7th anniversary of the first commit to our source code repository. Since then, we have seen over 12300 commits amounting to more than 220000 lines of C++ code.

The major focus of this release was to improve the reliability of large scale runs. We believe to have achieved this goal as we now can reliably run *HPX* applications on up to ~24k cores. We have also shown that HPX can be used with success for symmetric runs (applications using both, host cores and Intel Xeon/Phi coprocessors). This is a huge step forward in terms of the usability of *HPX*. The main focus of this work involved isolating the causes of the segmentation faults at start up and shut down. Many of these issues were discovered to be the result of the suspension of threads which hold locks.

A very important improvement introduced with this release is the refactoring of the code representing our parcel-port implementation. Parcel- ports can now be implemented by 3rd parties as independent plugins which are dynamically loaded at runtime (static linking of parcel-ports is also supported). This refactoring also includes a massive improvement of the performance of our existing parcel-ports. We were able to significantly reduce the networking latencies and to improve the available networking bandwidth. Please note that in this release we disabled the ibverbs and ipc parcel ports as those have not been ported to the new plugin system yet (see Issue #839[5724]).

Another corner stone of this release is our work towards a complete implementation of __cpp11_n4104__ (Working Draft, Technical Specification for C++ Extensions for Parallelism). This document defines a set of parallel algorithms to be added to the C++ standard library. We now have implemented about 75% of all specified parallel algorithms (see [link hpx.manual.parallel.parallel_algorithms Parallel Algorithms] for more details). We also implemented some extensions to __cpp11_n4104__ allowing to invoke all of the algorithms asynchronously.

This release adds a first implementation of `hpx::vector` which is a distributed data structure closely aligned to the functionality of `std::vector`. The difference is that `hpx::vector` stores the data in partitions where the partitions can be distributed over different localities. We started to work on allowing to use the parallel algorithms with `hpx::vector`. At this point we have implemented only a few of the parallel algorithms

[5718] https://github.com/TheHPXProject/hpx/pull/1300
[5719] https://github.com/TheHPXProject/hpx/issues/1251
[5720] https://github.com/TheHPXProject/hpx/issues/1008
[5721] https://github.com/TheHPXProject/hpx/issues/1001
[5722] https://github.com/TheHPXProject/hpx/issues/721
[5723] https://github.com/TheHPXProject/hpx/issues/524
[5724] https://github.com/TheHPXProject/hpx/issues/839

to support distributed data structures (like `hpx::vector`) for testing purposes (see Issue #1338[5725] for a documentation of our progress).

Breaking changes

With this release we put a lot of effort into changing the code base to be more compatible to C++11. These changes have caused the following issues for backward compatibility:

- Move to Variadics- All of the API now uses variadic templates. However, this change required to modify the argument sequence for some of the exiting API functions (*`hpx::async_continue`*, `hpx::apply_continue`, *hpx::when_each*, *hpx::wait_each*, synchronous invocation of actions).
- Changes to Macros- We also removed the macros `HPX_STD_FUNCTION` and `HPX_STD_TUPLE`. This shouldn't affect any user code as we replaced `HPX_STD_FUNCTION` with `hpx::util::function_nonser` which was the default expansion used for this macro. All *HPX* API functions which expect a `hpx::util::function_nonser` (or a `hpx::util::unique_function_nonser`) can now be transparently called with a compatible `std::function` instead. Similarly, `HPX_STD_TUPLE` was replaced by its default expansion as well: `hpx::util::tuple`.
- Changes to `hpx::unique_future`- `hpx::unique_future`, which was deprecated in the previous release for *hpx::future* is now completely removed from *HPX*. This completes the transition to a completely standards conforming implementation of `hpx::future`.
- Changes to Supported Compilers. Finally, in order to utilize more C++11 semantics, we have officially dropped support for GCC 4.4 and MSVC 2012. Please see our *Prerequisites* page for more details.

Bug fixes (closed tickets)

Here is a list of the important tickets we closed for this release.

- Issue #1402[5726] - Internal shared_future serialization copies
- Issue #1399[5727] - Build takes unusually long time…
- Issue #1398[5728] - Tests using the scan partitioner are broken on at least gcc 4.7 and intel compiler
- Issue #1397[5729] - Completely remove hpx::unique_future
- Issue #1396[5730] - Parallel scan algorithms with different initial values
- Issue #1395[5731] - Race Condition - 1d_stencil_8 - SuperMIC
- Issue #1394[5732] - "suspending thread while at least one lock is being held" - 1d_stencil_8 - SuperMIC

[5725] https://github.com/TheHPXProject/hpx/issues/1338
[5726] https://github.com/TheHPXProject/hpx/issues/1402
[5727] https://github.com/TheHPXProject/hpx/issues/1399
[5728] https://github.com/TheHPXProject/hpx/issues/1398
[5729] https://github.com/TheHPXProject/hpx/issues/1397
[5730] https://github.com/TheHPXProject/hpx/issues/1396
[5731] https://github.com/TheHPXProject/hpx/issues/1395
[5732] https://github.com/TheHPXProject/hpx/issues/1394

- Issue #1393[5733] - SEGFAULT in 1d_stencil_8 on SuperMIC
- Issue #1392[5734] - Fixing #1168
- Issue #1391[5735] - Parallel Algorithms for scan partitioner for small number of elements
- Issue #1387[5736] - Failure with more than 4 localities
- Issue #1386[5737] - Dispatching unhandled exceptions to outer user code
- Issue #1385[5738] - Adding Copy algorithms, fixing `parallel::copy_if`
- Issue #1384[5739] - Fixing 1325
- Issue #1383[5740] - Fixed #504: Refactor Dataflow LCO to work with futures, this removes the dataflow component as it is obsolete
- Issue #1382[5741] - `is_sorted`, `is_sorted_until` and `is_partitioned` algorithms
- Issue #1381[5742] - fix for CMake versions prior to 3.1
- Issue #1380[5743] - resolved warning in CMake 3.1 and newer
- Issue #1379[5744] - Compilation error with papi
- Issue #1378[5745] - Towards safer migration
- Issue #1377[5746] - HPXConfig.cmake should include `TCMALLOC_LIBRARY` and `TCMALLOC_INCLUDE_DIR`
- Issue #1376[5747] - Warning on uninitialized member
- Issue #1375[5748] - Fixing 1163
- Issue #1374[5749] - Fixing the MSVC 12 release builder
- Issue #1373[5750] - Modifying parallel search algorithm for zero length searches
- Issue #1372[5751] - Modifying parallel search algorithm for zero length searches
- Issue #1371[5752] - Avoid holding a lock during agas::incref while doing a credit split
- Issue #1370[5753] - `--hpx:bind` throws unexpected error
- Issue #1369[5754] - Getting rid of (void) in loops

5733 https://github.com/TheHPXProject/hpx/issues/1393
5734 https://github.com/TheHPXProject/hpx/issues/1392
5735 https://github.com/TheHPXProject/hpx/issues/1391
5736 https://github.com/TheHPXProject/hpx/issues/1387
5737 https://github.com/TheHPXProject/hpx/issues/1386
5738 https://github.com/TheHPXProject/hpx/issues/1385
5739 https://github.com/TheHPXProject/hpx/issues/1384
5740 https://github.com/TheHPXProject/hpx/issues/1383
5741 https://github.com/TheHPXProject/hpx/issues/1382
5742 https://github.com/TheHPXProject/hpx/issues/1381
5743 https://github.com/TheHPXProject/hpx/issues/1380
5744 https://github.com/TheHPXProject/hpx/issues/1379
5745 https://github.com/TheHPXProject/hpx/issues/1378
5746 https://github.com/TheHPXProject/hpx/issues/1377
5747 https://github.com/TheHPXProject/hpx/issues/1376
5748 https://github.com/TheHPXProject/hpx/issues/1375
5749 https://github.com/TheHPXProject/hpx/issues/1374
5750 https://github.com/TheHPXProject/hpx/issues/1373
5751 https://github.com/TheHPXProject/hpx/issues/1372
5752 https://github.com/TheHPXProject/hpx/issues/1371
5753 https://github.com/TheHPXProject/hpx/issues/1370
5754 https://github.com/TheHPXProject/hpx/issues/1369

- Issue #1368[5755] - Variadic templates support for tuple
- Issue #1367[5756] - One last batch of variadic templates support
- Issue #1366[5757] - Fixing symbolic namespace hang
- Issue #1365[5758] - More held locks
- Issue #1364[5759] - Add counters 1363
- Issue #1363[5760] - Add thread overhead counters
- Issue #1362[5761] - Std config removal
- Issue #1361[5762] - Parcelport plugins
- Issue #1360[5763] - Detuplify transfer_action
- Issue #1359[5764] - Removed obsolete checks
- Issue #1358[5765] - Fixing 1352
- Issue #1357[5766] - Variadic templates support for runtime_support and components
- Issue #1356[5767] - fixed coordinate test for intel13
- Issue #1355[5768] - fixed coordinate.hpp
- Issue #1354[5769] - Lexicographical Compare completed
- Issue #1353[5770] - HPX should set `Boost_ADDITIONAL_VERSIONS` flags
- Issue #1352[5771] - Error: Cannot find action '' in type registry: HPX(bad_action_code)
- Issue #1351[5772] - Variadic templates support for appliers
- Issue #1350[5773] - Actions simplification
- Issue #1349[5774] - Variadic when and wait functions
- Issue #1348[5775] - Added hpx_init header to test files
- Issue #1347[5776] - Another batch of variadic templates support
- Issue #1346[5777] - Segmented copy

5755 https://github.com/TheHPXProject/hpx/issues/1368
5756 https://github.com/TheHPXProject/hpx/issues/1367
5757 https://github.com/TheHPXProject/hpx/issues/1366
5758 https://github.com/TheHPXProject/hpx/issues/1365
5759 https://github.com/TheHPXProject/hpx/issues/1364
5760 https://github.com/TheHPXProject/hpx/issues/1363
5761 https://github.com/TheHPXProject/hpx/issues/1362
5762 https://github.com/TheHPXProject/hpx/issues/1361
5763 https://github.com/TheHPXProject/hpx/issues/1360
5764 https://github.com/TheHPXProject/hpx/issues/1359
5765 https://github.com/TheHPXProject/hpx/issues/1358
5766 https://github.com/TheHPXProject/hpx/issues/1357
5767 https://github.com/TheHPXProject/hpx/issues/1356
5768 https://github.com/TheHPXProject/hpx/issues/1355
5769 https://github.com/TheHPXProject/hpx/issues/1354
5770 https://github.com/TheHPXProject/hpx/issues/1353
5771 https://github.com/TheHPXProject/hpx/issues/1352
5772 https://github.com/TheHPXProject/hpx/issues/1351
5773 https://github.com/TheHPXProject/hpx/issues/1350
5774 https://github.com/TheHPXProject/hpx/issues/1349
5775 https://github.com/TheHPXProject/hpx/issues/1348
5776 https://github.com/TheHPXProject/hpx/issues/1347
5777 https://github.com/TheHPXProject/hpx/issues/1346

- Issue #1345[5778] - Attempting to fix hangs during shutdown
- Issue #1344[5779] - Std config removal
- Issue #1343[5780] - Removing various distribution policies for hpx::vector
- Issue #1342[5781] - Inclusive scan
- Issue #1341[5782] - Exclusive scan
- Issue #1340[5783] - Adding `parallel::count` for distributed data structures, adding tests
- Issue #1339[5784] - Update argument order for transform_reduce
- Issue #1337[5785] - Fix dataflow to handle properly ranges of futures
- Issue #1336[5786] - dataflow needs to hold onto futures passed to it
- Issue #1335[5787] - Fails to compile with msvc14
- Issue #1334[5788] - Examples build problem
- Issue #1333[5789] - Distributed transform reduce
- Issue #1332[5790] - Variadic templates support for actions
- Issue #1331[5791] - Some ambiguous calls of map::erase have been prevented by adding additional check in locality constructor.
- Issue #1330[5792] - Defining Plain Actions does not work as described in the documentation
- Issue #1329[5793] - Distributed vector cleanup
- Issue #1328[5794] - Sync docs and comments with code in hello_world example
- Issue #1327[5795] - Typos in docs
- Issue #1326[5796] - Documentation and code diverged in Fibonacci tutorial
- Issue #1325[5797] - Exceptions thrown during parcel handling are not handled correctly
- Issue #1324[5798] - fixed bandwidth calculation
- Issue #1323[5799] - mmap() failed to allocate thread stack due to insufficient resources

[5778] https://github.com/TheHPXProject/hpx/issues/1345
[5779] https://github.com/TheHPXProject/hpx/issues/1344
[5780] https://github.com/TheHPXProject/hpx/issues/1343
[5781] https://github.com/TheHPXProject/hpx/issues/1342
[5782] https://github.com/TheHPXProject/hpx/issues/1341
[5783] https://github.com/TheHPXProject/hpx/issues/1340
[5784] https://github.com/TheHPXProject/hpx/issues/1339
[5785] https://github.com/TheHPXProject/hpx/issues/1337
[5786] https://github.com/TheHPXProject/hpx/issues/1336
[5787] https://github.com/TheHPXProject/hpx/issues/1335
[5788] https://github.com/TheHPXProject/hpx/issues/1334
[5789] https://github.com/TheHPXProject/hpx/issues/1333
[5790] https://github.com/TheHPXProject/hpx/issues/1332
[5791] https://github.com/TheHPXProject/hpx/issues/1331
[5792] https://github.com/TheHPXProject/hpx/issues/1330
[5793] https://github.com/TheHPXProject/hpx/issues/1329
[5794] https://github.com/TheHPXProject/hpx/issues/1328
[5795] https://github.com/TheHPXProject/hpx/issues/1327
[5796] https://github.com/TheHPXProject/hpx/issues/1326
[5797] https://github.com/TheHPXProject/hpx/issues/1325
[5798] https://github.com/TheHPXProject/hpx/issues/1324
[5799] https://github.com/TheHPXProject/hpx/issues/1323

- Issue #1322[5800] - HPX fails to build aa182cf
- Issue #1321[5801] - Limiting size of outgoing messages while coalescing parcels
- Issue #1320[5802] - passing a future with launch::deferred in remote function call causes hang
- Issue #1319[5803] - An exception when tries to specify number high priority threads with abp-priority
- Issue #1318[5804] - Unable to run program with abp-priority and numa-sensitivity enabled
- Issue #1317[5805] - N4071 Search/Search_n finished, minor changes
- Issue #1316[5806] - Add config option to make -Ihpx.run_hpx_main!=1 the default
- Issue #1314[5807] - Variadic support for async and apply
- Issue #1313[5808] - Adjust when_any/some to the latest proposed interfaces
- Issue #1312[5809] - Fixing #857: hpx::naming::locality leaks parcelport specific information into the public interface
- Issue #1311[5810] - Distributed get'er/set'er_values for distributed vector
- Issue #1310[5811] - Crashing in hpx::parcelset::policies::mpi::connection_handler::handle_messages() on SuperMIC
- Issue #1308[5812] - Unable to execute an application with –hpx:threads
- Issue #1307[5813] - merge_graph linking issue
- Issue #1306[5814] - First batch of variadic templates support
- Issue #1305[5815] - Create a compiler wrapper
- Issue #1304[5816] - Provide a compiler wrapper for hpx
- Issue #1303[5817] - Drop support for GCC44
- Issue #1302[5818] - Fixing #1297
- Issue #1301[5819] - Compilation error when tried to use boost range iterators with wait_all
- Issue #1298[5820] - Distributed vector

[5800] https://github.com/TheHPXProject/hpx/issues/1322
[5801] https://github.com/TheHPXProject/hpx/issues/1321
[5802] https://github.com/TheHPXProject/hpx/issues/1320
[5803] https://github.com/TheHPXProject/hpx/issues/1319
[5804] https://github.com/TheHPXProject/hpx/issues/1318
[5805] https://github.com/TheHPXProject/hpx/issues/1317
[5806] https://github.com/TheHPXProject/hpx/issues/1316
[5807] https://github.com/TheHPXProject/hpx/issues/1314
[5808] https://github.com/TheHPXProject/hpx/issues/1313
[5809] https://github.com/TheHPXProject/hpx/issues/1312
[5810] https://github.com/TheHPXProject/hpx/issues/1311
[5811] https://github.com/TheHPXProject/hpx/issues/1310
[5812] https://github.com/TheHPXProject/hpx/issues/1308
[5813] https://github.com/TheHPXProject/hpx/issues/1307
[5814] https://github.com/TheHPXProject/hpx/issues/1306
[5815] https://github.com/TheHPXProject/hpx/issues/1305
[5816] https://github.com/TheHPXProject/hpx/issues/1304
[5817] https://github.com/TheHPXProject/hpx/issues/1303
[5818] https://github.com/TheHPXProject/hpx/issues/1302
[5819] https://github.com/TheHPXProject/hpx/issues/1301
[5820] https://github.com/TheHPXProject/hpx/issues/1298

- Issue #1297[5821] - Unable to invoke component actions recursively
- Issue #1294[5822] - HDF5 build error
- Issue #1275[5823] - The parcelport implementation is non-optimal
- Issue #1267[5824] - Added classes and unit tests for local_file, orangefs_file and pxfs_file
- Issue #1264[5825] - Error "assertion '!m_fun' failed" randomly occurs when using TCP
- Issue #1254[5826] - thread binding seems to not work properly
- Issue #1220[5827] - parallel::copy_if is broken
- Issue #1217[5828] - Find a better way of fixing the issue patched by #1216
- Issue #1168[5829] - Starting HPX on Cray machines using aprun isn't working correctly
- Issue #1085[5830] - Replace startup and shutdown barriers with broadcasts
- Issue #981[5831] - With SLURM, –hpx:threads=8 should not be necessary
- Issue #857[5832] - hpx::naming::locality leaks parcelport specific information into the public interface
- Issue #850[5833] - "flush" not documented
- Issue #763[5834] - Create buildbot instance that uses std::bind as HPX_STD_BIND
- Issue #680[5835] - Convert parcel ports into a plugin system
- Issue #582[5836] - Make exception thrown from HPX threads available from `hpx::init`
- Issue #504[5837] - Refactor Dataflow LCO to work with futures
- Issue #196[5838] - Don't store copies of the locality network metadata in the gva table

[5821] https://github.com/TheHPXProject/hpx/issues/1297
[5822] https://github.com/TheHPXProject/hpx/issues/1294
[5823] https://github.com/TheHPXProject/hpx/issues/1275
[5824] https://github.com/TheHPXProject/hpx/issues/1267
[5825] https://github.com/TheHPXProject/hpx/issues/1264
[5826] https://github.com/TheHPXProject/hpx/issues/1254
[5827] https://github.com/TheHPXProject/hpx/issues/1220
[5828] https://github.com/TheHPXProject/hpx/issues/1217
[5829] https://github.com/TheHPXProject/hpx/issues/1168
[5830] https://github.com/TheHPXProject/hpx/issues/1085
[5831] https://github.com/TheHPXProject/hpx/issues/981
[5832] https://github.com/TheHPXProject/hpx/issues/857
[5833] https://github.com/TheHPXProject/hpx/issues/850
[5834] https://github.com/TheHPXProject/hpx/issues/763
[5835] https://github.com/TheHPXProject/hpx/issues/680
[5836] https://github.com/TheHPXProject/hpx/issues/582
[5837] https://github.com/TheHPXProject/hpx/issues/504
[5838] https://github.com/TheHPXProject/hpx/issues/196

HPX V0.9.9 (Oct 31, 2014, codename Spooky)

General changes

We have had over 1500 commits since the last release and we have closed over 200 tickets (bugs, feature requests, pull requests, etc.). These are by far the largest numbers of commits and resolved issues for any of the *HPX* releases so far. We are especially happy about the large number of people who contributed for the first time to *HPX*.

- We completed the transition from the older (non-conforming) implementation of `hpx::future` to the new and fully conforming version by removing the old code and by renaming the type `hpx::unique_future` to `hpx::future`. In order to maintain backwards compatibility with existing code which uses the type `hpx::unique_future` we support the configuration variable `HPX_UNIQUE_FUTURE_ALIAS`. If this variable is set to `ON` while running cmake it will additionally define a template alias for this type.
- We rewrote and significantly changed our build system. Please have a look at the new (now generated) documentation here: *Building HPX*. Please revisit your build scripts to adapt to the changes. The most notable changes are:
- `HPX_NO_INSTALL` is no longer necessary.
- For external builds, you need to set `HPX_DIR` instead of `HPX_ROOT` as described here: *Using HPX with CMake-based projects*.
- IDEs that support multiple configurations (Visual Studio and XCode) can now be used as intended. that means no build dir.
- Building HPX statically (without dynamic libraries) is now supported (`-DHPX_STATIC_LINKING=On`).
- Please note that many variables used to configure the build process have been renamed to unify the naming conventions (see the section *CMake options* for more information).
- This also fixes a long list of issues, for more information see Issue #1204[5839].
- We started to implement various proposals to the C++ Standardization committee related to parallelism and concurrency, most notably N4409[5840] (Working Draft, Technical Specification for C++ Extensions for Parallelism), N4411[5841] (Task Region Rev. 3), and N4313[5842] (Working Draft, Technical Specification for C++ Extensions for Concurrency).
- We completely remodeled our automatic build system to run builds and unit tests on various systems and compilers. This allows us to find most bugs right as they were introduced and helps to maintain a high level of quality and compatibility. The newest build logs can be found at *HPX* Buildbot Website[5843].

5839 https://github.com/TheHPXProject/hpx/issues/1204
5840 http://wg21.link/n4409
5841 http://wg21.link/n4411
5842 http://wg21.link/n4313
5843 http://rostam.cct.lsu.edu/

Bug fixes (closed tickets)

Here is a list of the important tickets we closed for this release.

- Issue #1296[5844] - Rename make_error_future to make_exceptional_future, adjust to N4123
- Issue #1295[5845] - building issue
- Issue #1293[5846] - Transpose example
- Issue #1292[5847] - Wrong abs() function used in example
- Issue #1291[5848] - non-synchronized shift operators have been removed
- Issue #1290[5849] - RDTSCP is defined as true for Xeon Phi build
- Issue #1289[5850] - Fixing 1288
- Issue #1288[5851] - Add new performance counters
- Issue #1287[5852] - Hierarchy scheduler broken performance counters
- Issue #1286[5853] - Algorithm cleanup
- Issue #1285[5854] - Broken Links in Documentation
- Issue #1284[5855] - Uninitialized copy
- Issue #1283[5856] - missing boost::scoped_ptr includes
- Issue #1282[5857] - Update documentation of build options for schedulers
- Issue #1281[5858] - reset idle rate counter
- Issue #1280[5859] - Bug when executing on Intel MIC
- Issue #1279[5860] - Add improved when_all/wait_all
- Issue #1278[5861] - Implement improved when_all/wait_all
- Issue #1277[5862] - feature request: get access to argc argv and variables_map
- Issue #1276[5863] - Remove merging map
- Issue #1274[5864] - Weird (wrong) string code in papi.cpp

[5844] https://github.com/TheHPXProject/hpx/issues/1296
[5845] https://github.com/TheHPXProject/hpx/issues/1295
[5846] https://github.com/TheHPXProject/hpx/issues/1293
[5847] https://github.com/TheHPXProject/hpx/issues/1292
[5848] https://github.com/TheHPXProject/hpx/issues/1291
[5849] https://github.com/TheHPXProject/hpx/issues/1290
[5850] https://github.com/TheHPXProject/hpx/issues/1289
[5851] https://github.com/TheHPXProject/hpx/issues/1288
[5852] https://github.com/TheHPXProject/hpx/issues/1287
[5853] https://github.com/TheHPXProject/hpx/issues/1286
[5854] https://github.com/TheHPXProject/hpx/issues/1285
[5855] https://github.com/TheHPXProject/hpx/issues/1284
[5856] https://github.com/TheHPXProject/hpx/issues/1283
[5857] https://github.com/TheHPXProject/hpx/issues/1282
[5858] https://github.com/TheHPXProject/hpx/issues/1281
[5859] https://github.com/TheHPXProject/hpx/issues/1280
[5860] https://github.com/TheHPXProject/hpx/issues/1279
[5861] https://github.com/TheHPXProject/hpx/issues/1278
[5862] https://github.com/TheHPXProject/hpx/issues/1277
[5863] https://github.com/TheHPXProject/hpx/issues/1276
[5864] https://github.com/TheHPXProject/hpx/issues/1274

- Issue #1273[5865] - Sequential task execution policy
- Issue #1272[5866] - Avoid CMake name clash for Boost.Thread library
- Issue #1271[5867] - Updates on HPX Test Units
- Issue #1270[5868] - hpx/util/safe_lexical_cast.hpp is added
- Issue #1269[5869] - Added default value for "LIB" cmake variable
- Issue #1268[5870] - Memory Counters not working
- Issue #1266[5871] - FindHPX.cmake is not installed
- Issue #1263[5872] - apply_remote test takes too long
- Issue #1262[5873] - Chrono cleanup
- Issue #1261[5874] - Need make install for papi counters and this builds all the examples
- Issue #1260[5875] - Documentation of Stencil example claims
- Issue #1259[5876] - Avoid double-linking Boost on Windows
- Issue #1257[5877] - Adding additional parameter to create_thread
- Issue #1256[5878] - added buildbot changes to release notes
- Issue #1255[5879] - Cannot build MiniGhost
- Issue #1253[5880] - hpx::thread defects
- Issue #1252[5881] - HPX_PREFIX is too fragile
- Issue #1250[5882] - switch_to_fiber_emulation does not work properly
- Issue #1249[5883] - Documentation is generated under Release folder
- Issue #1248[5884] - Fix usage of hpx_generic_coroutine_context and get tests passing on powerpc
- Issue #1247[5885] - Dynamic linking error
- Issue #1246[5886] - Make cpuid.cpp C++11 compliant
- Issue #1245[5887] - HPX fails on startup (setting thread affinity mask)

[5865] https://github.com/TheHPXProject/hpx/issues/1273
[5866] https://github.com/TheHPXProject/hpx/issues/1272
[5867] https://github.com/TheHPXProject/hpx/issues/1271
[5868] https://github.com/TheHPXProject/hpx/issues/1270
[5869] https://github.com/TheHPXProject/hpx/issues/1269
[5870] https://github.com/TheHPXProject/hpx/issues/1268
[5871] https://github.com/TheHPXProject/hpx/issues/1266
[5872] https://github.com/TheHPXProject/hpx/issues/1263
[5873] https://github.com/TheHPXProject/hpx/issues/1262
[5874] https://github.com/TheHPXProject/hpx/issues/1261
[5875] https://github.com/TheHPXProject/hpx/issues/1260
[5876] https://github.com/TheHPXProject/hpx/issues/1259
[5877] https://github.com/TheHPXProject/hpx/issues/1257
[5878] https://github.com/TheHPXProject/hpx/issues/1256
[5879] https://github.com/TheHPXProject/hpx/issues/1255
[5880] https://github.com/TheHPXProject/hpx/issues/1253
[5881] https://github.com/TheHPXProject/hpx/issues/1252
[5882] https://github.com/TheHPXProject/hpx/issues/1250
[5883] https://github.com/TheHPXProject/hpx/issues/1249
[5884] https://github.com/TheHPXProject/hpx/issues/1248
[5885] https://github.com/TheHPXProject/hpx/issues/1247
[5886] https://github.com/TheHPXProject/hpx/issues/1246
[5887] https://github.com/TheHPXProject/hpx/issues/1245

- Issue #1244[5888] - HPX_WITH_RDTSC configure test fails, but should succeed
- Issue #1243[5889] - CTest dashboard info for CSCS CDash drop location
- Issue #1242[5890] - Mac fixes
- Issue #1241[5891] - Failure in Distributed with Boost 1.56
- Issue #1240[5892] - fix a race condition in examples.diskperf
- Issue #1239[5893] - fix wait_each in examples.diskperf
- Issue #1238[5894] - Fixed #1237: hpx::util::portable_binary_iarchive failed
- Issue #1237[5895] - hpx::util::portable_binary_iarchive faileds
- Issue #1235[5896] - Fixing clang warnings and errors
- Issue #1234[5897] - TCP runs fail: Transport endpoint is not connected
- Issue #1233[5898] - Making sure the correct number of threads is registered with AGAS
- Issue #1232[5899] - Fixing race in wait_xxx
- Issue #1231[5900] - Parallel minmax
- Issue #1230[5901] - Distributed run of 1d_stencil_8 uses less threads than spec. & sometimes gives errors
- Issue #1229[5902] - Unstable number of threads
- Issue #1228[5903] - HPX link error (cmake / MPI)
- Issue #1226[5904] - Warning about struct/class thread_counters
- Issue #1225[5905] - Adding parallel::replace etc
- Issue #1224[5906] - Extending dataflow to pass through non-future arguments
- Issue #1223[5907] - Remaining find algorithms implemented, N4071
- Issue #1222[5908] - Merging all the changes
- Issue #1221[5909] - No error output when using mpirun with hpx
- Issue #1219[5910] - Adding new AGAS cache performance counters

5888 https://github.com/TheHPXProject/hpx/issues/1244
5889 https://github.com/TheHPXProject/hpx/issues/1243
5890 https://github.com/TheHPXProject/hpx/issues/1242
5891 https://github.com/TheHPXProject/hpx/issues/1241
5892 https://github.com/TheHPXProject/hpx/issues/1240
5893 https://github.com/TheHPXProject/hpx/issues/1239
5894 https://github.com/TheHPXProject/hpx/issues/1238
5895 https://github.com/TheHPXProject/hpx/issues/1237
5896 https://github.com/TheHPXProject/hpx/issues/1235
5897 https://github.com/TheHPXProject/hpx/issues/1234
5898 https://github.com/TheHPXProject/hpx/issues/1233
5899 https://github.com/TheHPXProject/hpx/issues/1232
5900 https://github.com/TheHPXProject/hpx/issues/1231
5901 https://github.com/TheHPXProject/hpx/issues/1230
5902 https://github.com/TheHPXProject/hpx/issues/1229
5903 https://github.com/TheHPXProject/hpx/issues/1228
5904 https://github.com/TheHPXProject/hpx/issues/1226
5905 https://github.com/TheHPXProject/hpx/issues/1225
5906 https://github.com/TheHPXProject/hpx/issues/1224
5907 https://github.com/TheHPXProject/hpx/issues/1223
5908 https://github.com/TheHPXProject/hpx/issues/1222
5909 https://github.com/TheHPXProject/hpx/issues/1221
5910 https://github.com/TheHPXProject/hpx/issues/1219

- Issue #1216[5911] - Fixing using futures (clients) as arguments to actions
- Issue #1215[5912] - Error compiling simple component
- Issue #1214[5913] - Stencil docs
- Issue #1213[5914] - Using more than a few dozen MPI processes on SuperMike results in a seg fault before getting to hpx_main
- Issue #1212[5915] - Parallel rotate
- Issue #1211[5916] - Direct actions cause the future's shared_state to be leaked
- Issue #1210[5917] - Refactored local::promise to be standard conformant
- Issue #1209[5918] - Improve command line handling
- Issue #1208[5919] - Adding parallel::reverse and parallel::reverse_copy
- Issue #1207[5920] - Add copy_backward and move_backward
- Issue #1206[5921] - N4071 additional algorithms implemented
- Issue #1204[5922] - Cmake simplification and various other minor changes
- Issue #1203[5923] - Implementing new launch policy for (local) async: `hpx::launch::fork`.
- Issue #1202[5924] - Failed assertion in connection_cache.hpp
- Issue #1201[5925] - pkg-config doesn't add mpi link directories
- Issue #1200[5926] - Error when querying time performance counters
- Issue #1199[5927] - library path is now configurable (again)
- Issue #1198[5928] - Error when querying performance counters
- Issue #1197[5929] - tests fail with intel compiler
- Issue #1196[5930] - Silence several warnings
- Issue #1195[5931] - Rephrase initializers to work with VC++ 2012
- Issue #1194[5932] - Simplify parallel algorithms

5911 https://github.com/TheHPXProject/hpx/issues/1216
5912 https://github.com/TheHPXProject/hpx/issues/1215
5913 https://github.com/TheHPXProject/hpx/issues/1214
5914 https://github.com/TheHPXProject/hpx/issues/1213
5915 https://github.com/TheHPXProject/hpx/issues/1212
5916 https://github.com/TheHPXProject/hpx/issues/1211
5917 https://github.com/TheHPXProject/hpx/issues/1210
5918 https://github.com/TheHPXProject/hpx/issues/1209
5919 https://github.com/TheHPXProject/hpx/issues/1208
5920 https://github.com/TheHPXProject/hpx/issues/1207
5921 https://github.com/TheHPXProject/hpx/issues/1206
5922 https://github.com/TheHPXProject/hpx/issues/1204
5923 https://github.com/TheHPXProject/hpx/issues/1203
5924 https://github.com/TheHPXProject/hpx/issues/1202
5925 https://github.com/TheHPXProject/hpx/issues/1201
5926 https://github.com/TheHPXProject/hpx/issues/1200
5927 https://github.com/TheHPXProject/hpx/issues/1199
5928 https://github.com/TheHPXProject/hpx/issues/1198
5929 https://github.com/TheHPXProject/hpx/issues/1197
5930 https://github.com/TheHPXProject/hpx/issues/1196
5931 https://github.com/TheHPXProject/hpx/issues/1195
5932 https://github.com/TheHPXProject/hpx/issues/1194

- Issue #1193[5933] - Adding `parallel::equal`
- Issue #1192[5934] - HPX(out_of_memory) on including <hpx/hpx.hpp>
- Issue #1191[5935] - Fixing #1189
- Issue #1190[5936] - Chrono cleanup
- Issue #1189[5937] - Deadlock .. somewhere? (probably serialization)
- Issue #1188[5938] - Removed `future::get_status()`
- Issue #1186[5939] - Fixed FindOpenCL to find current AMD APP SDK
- Issue #1184[5940] - Tweaking future unwrapping
- Issue #1183[5941] - Extended `parallel::reduce`
- Issue #1182[5942] - `future::unwrap` hangs for `launch::deferred`
- Issue #1181[5943] - Adding `all_of`, `any_of`, and `none_of` and corresponding documentation
- Issue #1180[5944] - `hpx::cout` defect
- Issue #1179[5945] - `hpx::async` does not work for member function pointers when called on types with self-defined unary `operator*`
- Issue #1178[5946] - Implemented variadic `hpx::util::zip_iterator`
- Issue #1177[5947] - MPI parcelport defect
- Issue #1176[5948] - `HPX_DEFINE_COMPONENT_CONST_ACTION_TPL` does not have a 2-argument version
- Issue #1175[5949] - Create util::zip_iterator working with util::tuple<>
- Issue #1174[5950] - Error Building HPX on linux, root_certificate_authority.cpp
- Issue #1173[5951] - hpx::cout output lost
- Issue #1172[5952] - HPX build error with Clang 3.4.2
- Issue #1171[5953] - `CMAKE_INSTALL_PREFIX` ignored
- Issue #1170[5954] - Close hpx_benchmarks repository on Github

5933 https://github.com/TheHPXProject/hpx/issues/1193
5934 https://github.com/TheHPXProject/hpx/issues/1192
5935 https://github.com/TheHPXProject/hpx/issues/1191
5936 https://github.com/TheHPXProject/hpx/issues/1190
5937 https://github.com/TheHPXProject/hpx/issues/1189
5938 https://github.com/TheHPXProject/hpx/issues/1188
5939 https://github.com/TheHPXProject/hpx/issues/1186
5940 https://github.com/TheHPXProject/hpx/issues/1184
5941 https://github.com/TheHPXProject/hpx/issues/1183
5942 https://github.com/TheHPXProject/hpx/issues/1182
5943 https://github.com/TheHPXProject/hpx/issues/1181
5944 https://github.com/TheHPXProject/hpx/issues/1180
5945 https://github.com/TheHPXProject/hpx/issues/1179
5946 https://github.com/TheHPXProject/hpx/issues/1178
5947 https://github.com/TheHPXProject/hpx/issues/1177
5948 https://github.com/TheHPXProject/hpx/issues/1176
5949 https://github.com/TheHPXProject/hpx/issues/1175
5950 https://github.com/TheHPXProject/hpx/issues/1174
5951 https://github.com/TheHPXProject/hpx/issues/1173
5952 https://github.com/TheHPXProject/hpx/issues/1172
5953 https://github.com/TheHPXProject/hpx/issues/1171
5954 https://github.com/TheHPXProject/hpx/issues/1170

- Issue #1169[5955] - Buildbot emails have syntax error in url
- Issue #1167[5956] - Merge partial implementation of standards proposal N3960
- Issue #1166[5957] - Fixed several compiler warnings
- Issue #1165[5958] - cmake warns: "tests.regressions.actions" does not exist
- Issue #1164[5959] - Want my own serialization of hpx::future
- Issue #1162[5960] - Segfault in hello_world example
- Issue #1161[5961] - Use `HPX_ASSERT` to aid the compiler
- Issue #1160[5962] - Do not put -DNDEBUG into hpx_application.pc
- Issue #1159[5963] - Support Clang 3.4.2
- Issue #1158[5964] - Fixed #1157: Rename when_n/wait_n, add when_xxx_n/wait_xxx_n
- Issue #1157[5965] - Rename when_n/wait_n, add when_xxx_n/wait_xxx_n
- Issue #1156[5966] - Force inlining fails
- Issue #1155[5967] - changed header of printout to be compatible with python csv module
- Issue #1154[5968] - Fixing iostreams
- Issue #1153[5969] - Standard manipulators (like std::endl) do not work with hpx::ostream
- Issue #1152[5970] - Functions revamp
- Issue #1151[5971] - Suppressing cmake 3.0 policy warning for CMP0026
- Issue #1150[5972] - Client Serialization error
- Issue #1149[5973] - Segfault on Stampede
- Issue #1148[5974] - Refactoring mini-ghost
- Issue #1147[5975] - N3960 copy_if and copy_n implemented and tested
- Issue #1146[5976] - Stencil print
- Issue #1145[5977] - N3960 hpx::parallel::copy implemented and tested

[5955] https://github.com/TheHPXProject/hpx/issues/1169
[5956] https://github.com/TheHPXProject/hpx/issues/1167
[5957] https://github.com/TheHPXProject/hpx/issues/1166
[5958] https://github.com/TheHPXProject/hpx/issues/1165
[5959] https://github.com/TheHPXProject/hpx/issues/1164
[5960] https://github.com/TheHPXProject/hpx/issues/1162
[5961] https://github.com/TheHPXProject/hpx/issues/1161
[5962] https://github.com/TheHPXProject/hpx/issues/1160
[5963] https://github.com/TheHPXProject/hpx/issues/1159
[5964] https://github.com/TheHPXProject/hpx/issues/1158
[5965] https://github.com/TheHPXProject/hpx/issues/1157
[5966] https://github.com/TheHPXProject/hpx/issues/1156
[5967] https://github.com/TheHPXProject/hpx/issues/1155
[5968] https://github.com/TheHPXProject/hpx/issues/1154
[5969] https://github.com/TheHPXProject/hpx/issues/1153
[5970] https://github.com/TheHPXProject/hpx/issues/1152
[5971] https://github.com/TheHPXProject/hpx/issues/1151
[5972] https://github.com/TheHPXProject/hpx/issues/1150
[5973] https://github.com/TheHPXProject/hpx/issues/1149
[5974] https://github.com/TheHPXProject/hpx/issues/1148
[5975] https://github.com/TheHPXProject/hpx/issues/1147
[5976] https://github.com/TheHPXProject/hpx/issues/1146
[5977] https://github.com/TheHPXProject/hpx/issues/1145

- Issue #1144[5978] - OpenMP examples 1d_stencil do not build
- Issue #1143[5979] - 1d_stencil OpenMP examples do not build
- Issue #1142[5980] - Cannot build HPX with gcc 4.6 on OS X
- Issue #1140[5981] - Fix OpenMP lookup, enable usage of config tests in external CMake projects.
- Issue #1139[5982] - hpx/hpx/config/compiler_specific.hpp
- Issue #1138[5983] - clean up pkg-config files
- Issue #1137[5984] - Improvements to create binary packages
- Issue #1136[5985] - HPX_GCC_VERSION not defined on all compilers
- Issue #1135[5986] - Avoiding collision between winsock2.h and windows.h
- Issue #1134[5987] - Making sure, that hpx::finalize can be called from any locality
- Issue #1133[5988] - 1d stencil examples
- Issue #1131[5989] - Refactor unique_function implementation
- Issue #1130[5990] - Unique function
- Issue #1129[5991] - Some fixes to the Build system on OS X
- Issue #1128[5992] - Action future args
- Issue #1127[5993] - Executor causes segmentation fault
- Issue #1124[5994] - Adding new API functions: `register_id_with_basename`, `unregister_id_with_basename`, `find_ids_from_basename`; adding test
- Issue #1123[5995] - Reduce nesting of try-catch construct in `encode_parcels`?
- Issue #1122[5996] - Client base fixes
- Issue #1121[5997] - Update `hpxrun.py.in`
- Issue #1120[5998] - HTTS2 tests compile errors on v110 (VS2012)
- Issue #1119[5999] - Remove references to boost::atomic in accumulator example

5978 https://github.com/TheHPXProject/hpx/issues/1144
5979 https://github.com/TheHPXProject/hpx/issues/1143
5980 https://github.com/TheHPXProject/hpx/issues/1142
5981 https://github.com/TheHPXProject/hpx/issues/1140
5982 https://github.com/TheHPXProject/hpx/issues/1139
5983 https://github.com/TheHPXProject/hpx/issues/1138
5984 https://github.com/TheHPXProject/hpx/issues/1137
5985 https://github.com/TheHPXProject/hpx/issues/1136
5986 https://github.com/TheHPXProject/hpx/issues/1135
5987 https://github.com/TheHPXProject/hpx/issues/1134
5988 https://github.com/TheHPXProject/hpx/issues/1133
5989 https://github.com/TheHPXProject/hpx/issues/1131
5990 https://github.com/TheHPXProject/hpx/issues/1130
5991 https://github.com/TheHPXProject/hpx/issues/1129
5992 https://github.com/TheHPXProject/hpx/issues/1128
5993 https://github.com/TheHPXProject/hpx/issues/1127
5994 https://github.com/TheHPXProject/hpx/issues/1124
5995 https://github.com/TheHPXProject/hpx/issues/1123
5996 https://github.com/TheHPXProject/hpx/issues/1122
5997 https://github.com/TheHPXProject/hpx/issues/1121
5998 https://github.com/TheHPXProject/hpx/issues/1120
5999 https://github.com/TheHPXProject/hpx/issues/1119

- Issue #1118[6000] - Only build test thread_pool_executor_1114_test if HPX_SCHEDULER is set
- Issue #1117[6001] - local_queue_executor linker error on vc110
- Issue #1116[6002] - Disabled performance counter should give runtime errors, not invalid data
- Issue #1115[6003] - Compile error with Intel C++ 13.1
- Issue #1114[6004] - Default constructed executor is not usable
- Issue #1113[6005] - Fast compilation of logging causes ABI incompatibilities between different NDEBUG values
- Issue #1112[6006] - Using thread_pool_executors causes segfault
- Issue #1111[6007] - hpx::threads::get_thread_data always returns zero
- Issue #1110[6008] - Remove unnecessary null pointer checks
- Issue #1109[6009] - More tests adjustments
- Issue #1108[6010] - Clarify build rules for "libboost_atomic-mt.so"?
- Issue #1107[6011] - Remove unnecessary null pointer checks
- Issue #1106[6012] - network_storage benchmark improvements, adding legends to plots and tidying layout
- Issue #1105[6013] - Add more plot outputs and improve instructions doc
- Issue #1104[6014] - Complete quoting for parameters of some CMake commands
- Issue #1103[6015] - Work on test/scripts
- Issue #1102[6016] - Changed minimum requirement of window install to 2012
- Issue #1101[6017] - Changed minimum requirement of window install to 2012
- Issue #1100[6018] - Changed readme to no longer specify using MSVC 2010 compiler
- Issue #1099[6019] - Error returning futures from component actions
- Issue #1098[6020] - Improve storage test

[6000] https://github.com/TheHPXProject/hpx/issues/1118
[6001] https://github.com/TheHPXProject/hpx/issues/1117
[6002] https://github.com/TheHPXProject/hpx/issues/1116
[6003] https://github.com/TheHPXProject/hpx/issues/1115
[6004] https://github.com/TheHPXProject/hpx/issues/1114
[6005] https://github.com/TheHPXProject/hpx/issues/1113
[6006] https://github.com/TheHPXProject/hpx/issues/1112
[6007] https://github.com/TheHPXProject/hpx/issues/1111
[6008] https://github.com/TheHPXProject/hpx/issues/1110
[6009] https://github.com/TheHPXProject/hpx/issues/1109
[6010] https://github.com/TheHPXProject/hpx/issues/1108
[6011] https://github.com/TheHPXProject/hpx/issues/1107
[6012] https://github.com/TheHPXProject/hpx/issues/1106
[6013] https://github.com/TheHPXProject/hpx/issues/1105
[6014] https://github.com/TheHPXProject/hpx/issues/1104
[6015] https://github.com/TheHPXProject/hpx/issues/1103
[6016] https://github.com/TheHPXProject/hpx/issues/1102
[6017] https://github.com/TheHPXProject/hpx/issues/1101
[6018] https://github.com/TheHPXProject/hpx/issues/1100
[6019] https://github.com/TheHPXProject/hpx/issues/1099
[6020] https://github.com/TheHPXProject/hpx/issues/1098

- Issue #1097[6021] - data_actions quickstart example calls missing function decorate_action of data_get_action
- Issue #1096[6022] - MPI parcelport broken with new zero copy optimization
- Issue #1095[6023] - Warning C4005: _WIN32_WINNT: Macro redefinition
- Issue #1094[6024] - Syntax error for -DHPX_UNIQUE_FUTURE_ALIAS in master
- Issue #1093[6025] - Syntax error for -DHPX_UNIQUE_FUTURE_ALIAS
- Issue #1092[6026] - Rename unique_future<> back to future<>
- Issue #1091[6027] - Inconsistent error message
- Issue #1090[6028] - On windows 8.1 the examples crashed if using more than one os thread
- Issue #1089[6029] - Components should be allowed to have their own executor
- Issue #1088[6030] - Add possibility to select a network interface for the ibverbs parcelport
- Issue #1087[6031] - ibverbs and ipc parcelport uses zero copy optimization
- Issue #1083[6032] - Make shell examples copyable in docs
- Issue #1082[6033] - Implement proper termination detection during shutdown
- Issue #1081[6034] - Implement thread_specific_ptr for hpx::threads
- Issue #1072[6035] - make install not working properly
- Issue #1070[6036] - Complete quoting for parameters of some CMake commands
- Issue #1059[6037] - Fix more unused variable warnings
- Issue #1051[6038] - Implement when_each
- Issue #973[6039] - Would like option to report hwloc bindings
- Issue #970[6040] - Bad flags for Fortran compiler
- Issue #941[6041] - Create a proper user level context switching class for BG/Q
- Issue #935[6042] - Build error with gcc 4.6 and Boost 1.54.0 on hpx trunk and 0.9.6
- Issue #934[6043] - Want to build HPX without dynamic libraries

6021 https://github.com/TheHPXProject/hpx/issues/1097
6022 https://github.com/TheHPXProject/hpx/issues/1096
6023 https://github.com/TheHPXProject/hpx/issues/1095
6024 https://github.com/TheHPXProject/hpx/issues/1094
6025 https://github.com/TheHPXProject/hpx/issues/1093
6026 https://github.com/TheHPXProject/hpx/issues/1092
6027 https://github.com/TheHPXProject/hpx/issues/1091
6028 https://github.com/TheHPXProject/hpx/issues/1090
6029 https://github.com/TheHPXProject/hpx/issues/1089
6030 https://github.com/TheHPXProject/hpx/issues/1088
6031 https://github.com/TheHPXProject/hpx/issues/1087
6032 https://github.com/TheHPXProject/hpx/issues/1083
6033 https://github.com/TheHPXProject/hpx/issues/1082
6034 https://github.com/TheHPXProject/hpx/issues/1081
6035 https://github.com/TheHPXProject/hpx/issues/1072
6036 https://github.com/TheHPXProject/hpx/issues/1070
6037 https://github.com/TheHPXProject/hpx/issues/1059
6038 https://github.com/TheHPXProject/hpx/issues/1051
6039 https://github.com/TheHPXProject/hpx/issues/973
6040 https://github.com/TheHPXProject/hpx/issues/970
6041 https://github.com/TheHPXProject/hpx/issues/941
6042 https://github.com/TheHPXProject/hpx/issues/935
6043 https://github.com/TheHPXProject/hpx/issues/934

- Issue #927[6044] - Make hpx/lcos/reduce.hpp accept futures of id_type
- Issue #926[6045] - All unit tests that are run with more than one thread with CTest/hpx_run_test should configure hpx.os_threads
- Issue #925[6046] - regression_dataflow_791 needs to be brought in line with HPX standards
- Issue #899[6047] - Fix race conditions in regression tests
- Issue #879[6048] - Hung test leads to cascading test failure; make tests should support the MPI parcelport
- Issue #865[6049] - future<T> and friends shall work for movable only Ts
- Issue #847[6050] - Dynamic libraries are not installed on OS X
- Issue #816[6051] - First Program tutorial pull request
- Issue #799[6052] - Wrap lexical_cast to avoid exceptions
- Issue #720[6053] - broken configuration when using ccmake on Ubuntu
- Issue #622[6054] - `--hpx:hpx` and `--hpx:debug-hpx-log` is nonsensical
- Issue #525[6055] - Extend barrier LCO test to run in distributed
- Issue #515[6056] - Multi-destination version of hpx::apply is broken
- Issue #509[6057] - Push Boost.Atomic changes upstream
- Issue #503[6058] - Running HPX applications on Windows should not require setting %PATH%
- Issue #461[6059] - Add a compilation sanity test
- Issue #456[6060] - hpx_run_tests.py should log output from tests that timeout
- Issue #454[6061] - Investigate threadmanager performance
- Issue #345[6062] - Add more versatile environmental/cmake variable support to hpx_find_* CMake macros
- Issue #209[6063] - Support multiple configurations in generated build files
- Issue #190[6064] - hpx::cout should be a std::ostream

6044 https://github.com/TheHPXProject/hpx/issues/927
6045 https://github.com/TheHPXProject/hpx/issues/926
6046 https://github.com/TheHPXProject/hpx/issues/925
6047 https://github.com/TheHPXProject/hpx/issues/899
6048 https://github.com/TheHPXProject/hpx/issues/879
6049 https://github.com/TheHPXProject/hpx/issues/865
6050 https://github.com/TheHPXProject/hpx/issues/847
6051 https://github.com/TheHPXProject/hpx/issues/816
6052 https://github.com/TheHPXProject/hpx/issues/799
6053 https://github.com/TheHPXProject/hpx/issues/720
6054 https://github.com/TheHPXProject/hpx/issues/622
6055 https://github.com/TheHPXProject/hpx/issues/525
6056 https://github.com/TheHPXProject/hpx/issues/515
6057 https://github.com/TheHPXProject/hpx/issues/509
6058 https://github.com/TheHPXProject/hpx/issues/503
6059 https://github.com/TheHPXProject/hpx/issues/461
6060 https://github.com/TheHPXProject/hpx/issues/456
6061 https://github.com/TheHPXProject/hpx/issues/454
6062 https://github.com/TheHPXProject/hpx/issues/345
6063 https://github.com/TheHPXProject/hpx/issues/209
6064 https://github.com/TheHPXProject/hpx/issues/190

- Issue #189[6065] - iostreams component should use startup/shutdown functions
- Issue #183[6066] - Use Boost.ICL for correctness in AGAS
- Issue #44[6067] - Implement real futures

HPX V0.9.8 (Mar 24, 2014)

We have had over 800 commits since the last release and we have closed over 65 tickets (bugs, feature requests, etc.).

With the changes below, *HPX* is once again leading the charge of a whole new era of computation. By intrinsically breaking down and synchronizing the work to be done, *HPX* insures that application developers will no longer have to fret about where a segment of code executes. That allows coders to focus their time and energy to understanding the data dependencies of their algorithms and thereby the core obstacles to an efficient code. Here are some of the advantages of using *HPX*:

- *HPX* is solidly rooted in a sophisticated theoretical execution model – ParalleX
- *HPX* exposes an API fully conforming to the C++11 and the draft C++14 standards, extended and applied to distributed computing. Everything programmers know about the concurrency primitives of the standard C++ library is still valid in the context of *HPX*.
- It provides a competitive, high performance implementation of modern, future-proof ideas which gives an smooth migration path from today's mainstream techniques
- There is no need for the programmer to worry about lower level parallelization paradigms like threads or message passing; no need to understand pthreads, MPI, OpenMP, or Windows threads, etc.
- There is no need to think about different types of parallelism such as tasks, pipelines, or fork-join, task or data parallelism.
- The same source of your program compiles and runs on Linux, BlueGene/Q, Mac OS X, Windows, and Android.
- The same code runs on shared memory multi-core systems and supercomputers, on handheld devices and Intel® Xeon Phi™ accelerators, or a heterogeneous mix of those.

General changes

- A major API breaking change for this release was introduced by implementing `hpx::future` and `hpx::shared_future` fully in conformance with the C++11 Standard[6068]. While `hpx::shared_future` is new and will not create any compatibility problems, we revised the interface and implementation of the existing `hpx::future`. For more details please see the mailing list archive[6069]. To avoid any incompatibilities for existing code we named the type which implements the `std::future` interface as `hpx::unique_future`. For the next release this will be renamed to `hpx::future`, making it full conforming to C++11 Standard[6070].

[6065] https://github.com/TheHPXProject/hpx/issues/189
[6066] https://github.com/TheHPXProject/hpx/issues/183
[6067] https://github.com/TheHPXProject/hpx/issues/44
[6068] http://www.open-std.org/jtc1/sc22/wg21
[6069] http://mail.cct.lsu.edu/pipermail/hpx-users/2014-January/000141.html
[6070] http://www.open-std.org/jtc1/sc22/wg21

- A large part of the code base of *HPX* has been refactored and partially re-implemented. The main changes were related to
  - The threading subsystem: these changes significantly reduce the amount of overheads caused by the schedulers, improve the modularity of the code base, and extend the variety of available scheduling algorithms.
  - The parcel subsystem: these changes improve the performance of the *HPX* networking layer, modularize the structure of the parcelports, and simplify the creation of new parcelports for other underlying networking libraries.
  - The API subsystem: these changes improved the conformance of the API to C++11 Standard, extend and unify the available API functionality, and decrease the overheads created by various elements of the API.
  - The robustness of the component loading subsystem has been improved significantly, allowing to more portably and more reliably register the components needed by an application as startup. This additionally speeds up general application initialization.
- We added new API functionality like `hpx::migrate` and `hpx::copy_component` which are the basic building blocks necessary for implementing higher level abstractions for system-wide load balancing, runtime-adaptive resource management, and object-oriented checkpointing and state-management.
- We removed the use of C++11 move emulation (using Boost.Move), replacing it with C++11 rvalue references. This is the first step towards using more and more native C++11 facilities which we plan to introduce in the future.
- We improved the reference counting scheme used by *HPX* which helps managing distributed objects and memory. This improves the overall stability of *HPX* and further simplifies writing real world applications.
- The minimal Boost version required to use *HPX* is now V1.49.0.
- This release coincides with the first release of HPXPI (V0.1.0), the first implementation of the XPI specification[6071].

Bug fixes (closed tickets)

Here is a list of the important tickets we closed for this release.

- Issue #1086[6072] - Expose internal boost::shared_array to allow user management of array lifetime
- Issue #1083[6073] - Make shell examples copyable in docs
- Issue #1080[6074] - /threads{locality#*/total}/count/cumulative broken
- Issue #1079[6075] - Build problems on OS X
- Issue #1078[6076] - Improve robustness of component loading
- Issue #1077[6077] - Fix a missing enum definition for 'take' mode

[6071] https://github.com/STEllAR-GROUP/hpxpi/blob/master/spec.pdf?raw=true
[6072] https://github.com/TheHPXProject/hpx/issues/1086
[6073] https://github.com/TheHPXProject/hpx/issues/1083
[6074] https://github.com/TheHPXProject/hpx/issues/1080
[6075] https://github.com/TheHPXProject/hpx/issues/1079
[6076] https://github.com/TheHPXProject/hpx/issues/1078
[6077] https://github.com/TheHPXProject/hpx/issues/1077

- Issue #1076[6078] - Merge Jb master
- Issue #1075[6079] - Unknown CMake command "add_hpx_pseudo_target"
- Issue #1074[6080] - Implement `apply_continue_callback` and `apply_colocated_callback`
- Issue #1073[6081] - The new `apply_colocated` and `async_colocated` functions lead to automatic registered functions
- Issue #1071[6082] - Remove deferred_packaged_task
- Issue #1069[6083] - serialize_buffer with allocator fails at destruction
- Issue #1068[6084] - Coroutine include and forward declarations missing
- Issue #1067[6085] - Add allocator support to util::serialize_buffer
- Issue #1066[6086] - Allow for MPI_Init being called before HPX launches
- Issue #1065[6087] - AGAS cache isn't used/populated on worker localities
- Issue #1064[6088] - Reorder includes to ensure ws2 includes early
- Issue #1063[6089] - Add `hpx::runtime::suspend` and `hpx::runtime::resume`
- Issue #1062[6090] - Fix `async_continue` to properly handle return types
- Issue #1061[6091] - Implement `async_colocated` and `apply_colocated`
- Issue #1060[6092] - Implement minimal component migration
- Issue #1058[6093] - Remove `HPX_UTIL_TUPLE` from code base
- Issue #1057[6094] - Add performance counters for threading subsystem
- Issue #1055[6095] - Thread allocation uses two memory pools
- Issue #1053[6096] - Work stealing flawed
- Issue #1052[6097] - Fix a number of warnings
- Issue #1049[6098] - Fixes for TLS on OSX and more reliable test running
- Issue #1048[6099] - Fixing after 588 hang

6078 https://github.com/TheHPXProject/hpx/issues/1076
6079 https://github.com/TheHPXProject/hpx/issues/1075
6080 https://github.com/TheHPXProject/hpx/issues/1074
6081 https://github.com/TheHPXProject/hpx/issues/1073
6082 https://github.com/TheHPXProject/hpx/issues/1071
6083 https://github.com/TheHPXProject/hpx/issues/1069
6084 https://github.com/TheHPXProject/hpx/issues/1068
6085 https://github.com/TheHPXProject/hpx/issues/1067
6086 https://github.com/TheHPXProject/hpx/issues/1066
6087 https://github.com/TheHPXProject/hpx/issues/1065
6088 https://github.com/TheHPXProject/hpx/issues/1064
6089 https://github.com/TheHPXProject/hpx/issues/1063
6090 https://github.com/TheHPXProject/hpx/issues/1062
6091 https://github.com/TheHPXProject/hpx/issues/1061
6092 https://github.com/TheHPXProject/hpx/issues/1060
6093 https://github.com/TheHPXProject/hpx/issues/1058
6094 https://github.com/TheHPXProject/hpx/issues/1057
6095 https://github.com/TheHPXProject/hpx/issues/1055
6096 https://github.com/TheHPXProject/hpx/issues/1053
6097 https://github.com/TheHPXProject/hpx/issues/1052
6098 https://github.com/TheHPXProject/hpx/issues/1049
6099 https://github.com/TheHPXProject/hpx/issues/1048

- Issue #1047[6100] - Use port '0' for networking when using one locality
- Issue #1046[6101] - `composable_guard` test is broken when having more than one thread
- Issue #1045[6102] - Security missing headers
- Issue #1044[6103] - Native TLS on FreeBSD via __thread
- Issue #1043[6104] - `async` et.al. compute the wrong result type
- Issue #1042[6105] - `async` et.al. implicitly unwrap reference_wrappers
- Issue #1041[6106] - Remove redundant costly Kleene stars from regex searches
- Issue #1040[6107] - CMake script regex match patterns has unnecessary kleenes
- Issue #1039[6108] - Remove use of Boost.Move and replace with std::move and real rvalue refs
- Issue #1038[6109] - Bump minimal required Boost to 1.49.0
- Issue #1037[6110] - Implicit unwrapping of futures in async broken
- Issue #1036[6111] - Scheduler hangs when user code attempts to "block" OS-threads
- Issue #1035[6112] - Idle-rate counter always reports 100% idle rate
- Issue #1034[6113] - Symbolic name registration causes application hangs
- Issue #1033[6114] - Application options read in from an options file generate an error message
- Issue #1032[6115] - `hpx::id_type` local reference counting is wrong
- Issue #1031[6116] - Negative entry in reference count table
- Issue #1030[6117] - Implement condition_variable
- Issue #1029[6118] - Deadlock in thread scheduling subsystem
- Issue #1028[6119] - HPX-thread cumulative count performance counters report incorrect value
- Issue #1027[6120] - Expose `hpx::thread_interrupted` error code as a separate exception type

[6100] https://github.com/TheHPXProject/hpx/issues/1047
[6101] https://github.com/TheHPXProject/hpx/issues/1046
[6102] https://github.com/TheHPXProject/hpx/issues/1045
[6103] https://github.com/TheHPXProject/hpx/issues/1044
[6104] https://github.com/TheHPXProject/hpx/issues/1043
[6105] https://github.com/TheHPXProject/hpx/issues/1042
[6106] https://github.com/TheHPXProject/hpx/issues/1041
[6107] https://github.com/TheHPXProject/hpx/issues/1040
[6108] https://github.com/TheHPXProject/hpx/issues/1039
[6109] https://github.com/TheHPXProject/hpx/issues/1038
[6110] https://github.com/TheHPXProject/hpx/issues/1037
[6111] https://github.com/TheHPXProject/hpx/issues/1036
[6112] https://github.com/TheHPXProject/hpx/issues/1035
[6113] https://github.com/TheHPXProject/hpx/issues/1034
[6114] https://github.com/TheHPXProject/hpx/issues/1033
[6115] https://github.com/TheHPXProject/hpx/issues/1032
[6116] https://github.com/TheHPXProject/hpx/issues/1031
[6117] https://github.com/TheHPXProject/hpx/issues/1030
[6118] https://github.com/TheHPXProject/hpx/issues/1029
[6119] https://github.com/TheHPXProject/hpx/issues/1028
[6120] https://github.com/TheHPXProject/hpx/issues/1027

- Issue #1026[6121] - Exceptions thrown in asynchronous calls can be lost if the value of the future is never queried
- Issue #1025[6122] - `future::wait_for/wait_until` do not remove callback
- Issue #1024[6123] - Remove dependence to boost assert and create hpx assert
- Issue #1023[6124] - Segfaults with tcmalloc
- Issue #1022[6125] - prerequisites link in readme is broken
- Issue #1020[6126] - HPX Deadlock on external synchronization
- Issue #1019[6127] - Convert using `BOOST_ASSERT` to `HPX_ASSERT`
- Issue #1018[6128] - compiling bug with gcc 4.8.1
- Issue #1017[6129] - Possible crash in io_pool executor
- Issue #1016[6130] - Crash at startup
- Issue #1014[6131] - Implement Increment/Decrement Merging
- Issue #1013[6132] - Add more logging channels to enable greater control over logging granularity
- Issue #1012[6133] - `--hpx:debug-hpx-log` and `--hpx:debug-agas-log` lead to non-thread safe writes
- Issue #1011[6134] - After installation, running applications from the build/staging directory no longer works
- Issue #1010[6135] - Mergeable decrement requests are not being merged
- Issue #1009[6136] - `--hpx:list-symbolic-names` crashes
- Issue #1007[6137] - Components are not properly destroyed
- Issue #1006[6138] - Segfault/hang in set_data
- Issue #1003[6139] - Performance counter naming issue
- Issue #982[6140] - Race condition during startup
- Issue #912[6141] - OS X: component type not found in map

[6121] https://github.com/TheHPXProject/hpx/issues/1026
[6122] https://github.com/TheHPXProject/hpx/issues/1025
[6123] https://github.com/TheHPXProject/hpx/issues/1024
[6124] https://github.com/TheHPXProject/hpx/issues/1023
[6125] https://github.com/TheHPXProject/hpx/issues/1022
[6126] https://github.com/TheHPXProject/hpx/issues/1020
[6127] https://github.com/TheHPXProject/hpx/issues/1019
[6128] https://github.com/TheHPXProject/hpx/issues/1018
[6129] https://github.com/TheHPXProject/hpx/issues/1017
[6130] https://github.com/TheHPXProject/hpx/issues/1016
[6131] https://github.com/TheHPXProject/hpx/issues/1014
[6132] https://github.com/TheHPXProject/hpx/issues/1013
[6133] https://github.com/TheHPXProject/hpx/issues/1012
[6134] https://github.com/TheHPXProject/hpx/issues/1011
[6135] https://github.com/TheHPXProject/hpx/issues/1010
[6136] https://github.com/TheHPXProject/hpx/issues/1009
[6137] https://github.com/TheHPXProject/hpx/issues/1007
[6138] https://github.com/TheHPXProject/hpx/issues/1006
[6139] https://github.com/TheHPXProject/hpx/issues/1003
[6140] https://github.com/TheHPXProject/hpx/issues/982
[6141] https://github.com/TheHPXProject/hpx/issues/912

- Issue #663[6142] - Create a buildbot slave based on Clang 3.2/OSX
- Issue #636[6143] - Expose `this_locality::apply<act>(p1, p2);` for local execution
- Issue #197[6144] - Add `--console=address` option for PBS runs
- Issue #175[6145] - Asynchronous AGAS API

HPX V0.9.7 (Nov 13, 2013)

We have had over 1000 commits since the last release and we have closed over 180 tickets (bugs, feature requests, etc.).

General changes

- Ported HPX to BlueGene/Q
- Improved HPX support for Xeon/Phi accelerators
- Reimplemented `hpx::bind`, `hpx::tuple`, and `hpx::function` for better performance and better compliance with the C++11 Standard. Added `hpx::mem_fn`.
- Reworked `hpx::when_all` and `hpx::when_any` for better compliance with the ongoing C++ standardization effort, added heterogeneous version for those functions. Added `hpx::when_any_swapped`.
- Added `hpx::copy` as a precursor for a migrate functionality
- Added `hpx::get_ptr` allowing to directly access the memory underlying a given component
- Added the `hpx::lcos::broadcast`, `hpx::lcos::reduce`, and `hpx::lcos::fold` collective operations
- Added `hpx::get_locality_name` allowing to retrieve the name of any of the localities for the application.
- Added support for more flexible thread affinity control from the HPX command line, such as new modes for `--hpx:bind` (`balanced`, `scattered`, `compact`), improved default settings when running multiple localities on the same node.
- Added experimental executors for simpler thread pooling and scheduling. This API may change in the future as it will stay aligned with the ongoing C++ standardization efforts.
- Massively improved the performance of the HPX serialization code. Added partial support for zero copy serialization of array and bitwise-copyable types.
- General performance improvements of the code related to threads and futures.

[6142] https://github.com/TheHPXProject/hpx/issues/663
[6143] https://github.com/TheHPXProject/hpx/issues/636
[6144] https://github.com/TheHPXProject/hpx/issues/197
[6145] https://github.com/TheHPXProject/hpx/issues/175

Bug fixes (closed tickets)

Here is a list of the important tickets we closed for this release.

- Issue #1005[6146] - Allow one to disable array optimizations and zero copy optimizations for each parcelport
- Issue #1004[6147] - Generate new HPX logo image for the docs
- Issue #1002[6148] - If MPI parcelport is not available, running HPX under mpirun should fail
- Issue #1001[6149] - Zero copy serialization raises assert
- Issue #1000[6150] - Can't connect to a HPX application running with the MPI parcelport from a non MPI parcelport locality
- Issue #999[6151] - Optimize `hpx::when_n`
- Issue #998[6152] - Fixed const-correctness
- Issue #997[6153] - Making serialize_buffer::data() type save
- Issue #996[6154] - Memory leak in hpx::lcos::promise
- Issue #995[6155] - Race while registering pre-shutdown functions
- Issue #994[6156] - thread_rescheduling regression test does not compile
- Issue #992[6157] - Correct comments and messages
- Issue #991[6158] - setcap cap_sys_rawio=ep for power profiling causes an HPX application to abort
- Issue #989[6159] - Jacobi hangs during execution
- Issue #988[6160] - multiple_init test is failing
- Issue #986[6161] - Can't call a function called "init" from "main" when using `<hpx/hpx_main.hpp>`
- Issue #984[6162] - Reference counting tests are failing
- Issue #983[6163] - thread_suspension_executor test fails
- Issue #980[6164] - Terminating HPX threads don't leave stack in virgin state

[6146] https://github.com/TheHPXProject/hpx/issues/1005
[6147] https://github.com/TheHPXProject/hpx/issues/1004
[6148] https://github.com/TheHPXProject/hpx/issues/1002
[6149] https://github.com/TheHPXProject/hpx/issues/1001
[6150] https://github.com/TheHPXProject/hpx/issues/1000
[6151] https://github.com/TheHPXProject/hpx/issues/999
[6152] https://github.com/TheHPXProject/hpx/issues/998
[6153] https://github.com/TheHPXProject/hpx/issues/997
[6154] https://github.com/TheHPXProject/hpx/issues/996
[6155] https://github.com/TheHPXProject/hpx/issues/995
[6156] https://github.com/TheHPXProject/hpx/issues/994
[6157] https://github.com/TheHPXProject/hpx/issues/992
[6158] https://github.com/TheHPXProject/hpx/issues/991
[6159] https://github.com/TheHPXProject/hpx/issues/989
[6160] https://github.com/TheHPXProject/hpx/issues/988
[6161] https://github.com/TheHPXProject/hpx/issues/986
[6162] https://github.com/TheHPXProject/hpx/issues/984
[6163] https://github.com/TheHPXProject/hpx/issues/983
[6164] https://github.com/TheHPXProject/hpx/issues/980

- Issue #979[6165] - Static scheduler not in documents
- Issue #978[6166] - Preprocessing limits are broken
- Issue #977[6167] - Make tests.regressions.lcos.future_hang_on_get shorter
- Issue #976[6168] - Wrong library order in pkgconfig
- Issue #975[6169] - Please reopen #963
- Issue #974[6170] - Option pu-offset ignored in fixing_588 branch
- Issue #972[6171] - Cannot use MKL with HPX
- Issue #969[6172] - Non-existent INI files requested on the command line via `--hpx:config` do not cause warnings or errors.
- Issue #968[6173] - Cannot build examples in fixing_588 branch
- Issue #967[6174] - Command line description of `--hpx:queuing` seems wrong
- Issue #966[6175] - `--hpx:print-bind` physical core numbers are wrong
- Issue #965[6176] - Deadlock when building in Release mode
- Issue #963[6177] - Not all worker threads are working
- Issue #962[6178] - Problem with SLURM integration
- Issue #961[6179] - `--hpx:print-bind` outputs incorrect information
- Issue #960[6180] - Fix cut and paste error in documentation of get_thread_priority
- Issue #959[6181] - Change link to boost.atomic in documentation to point to boost.org
- Issue #958[6182] - Undefined reference to intrusive_ptr_release
- Issue #957[6183] - Make tuple standard compliant
- Issue #956[6184] - Segfault with a3382fb
- Issue #955[6185] - `--hpx:nodes` and `--hpx:nodefiles` do not work with foreign nodes
- Issue #954[6186] - Make order of arguments for hpx::async and hpx::broadcast consistent
- Issue #953[6187] - Cannot use MKL with HPX

6165 https://github.com/TheHPXProject/hpx/issues/979
6166 https://github.com/TheHPXProject/hpx/issues/978
6167 https://github.com/TheHPXProject/hpx/issues/977
6168 https://github.com/TheHPXProject/hpx/issues/976
6169 https://github.com/TheHPXProject/hpx/issues/975
6170 https://github.com/TheHPXProject/hpx/issues/974
6171 https://github.com/TheHPXProject/hpx/issues/972
6172 https://github.com/TheHPXProject/hpx/issues/969
6173 https://github.com/TheHPXProject/hpx/issues/968
6174 https://github.com/TheHPXProject/hpx/issues/967
6175 https://github.com/TheHPXProject/hpx/issues/966
6176 https://github.com/TheHPXProject/hpx/issues/965
6177 https://github.com/TheHPXProject/hpx/issues/963
6178 https://github.com/TheHPXProject/hpx/issues/962
6179 https://github.com/TheHPXProject/hpx/issues/961
6180 https://github.com/TheHPXProject/hpx/issues/960
6181 https://github.com/TheHPXProject/hpx/issues/959
6182 https://github.com/TheHPXProject/hpx/issues/958
6183 https://github.com/TheHPXProject/hpx/issues/957
6184 https://github.com/TheHPXProject/hpx/issues/956
6185 https://github.com/TheHPXProject/hpx/issues/955
6186 https://github.com/TheHPXProject/hpx/issues/954
6187 https://github.com/TheHPXProject/hpx/issues/953

- Issue #952[6188] - `register_[pre_]shutdown_function` never throw
- Issue #951[6189] - Assert when number of threads is greater than hardware concurrency
- Issue #948[6190] - `HPX_HAVE_GENERIC_CONTEXT_COROUTINES` conflicts with `HPX_HAVE_FIBER_BASED_COROUTINES`
- Issue #947[6191] - Need MPI_THREAD_MULTIPLE for backward compatibility
- Issue #946[6192] - HPX does not call `MPI_Finalize`
- Issue #945[6193] - Segfault with `hpx::lcos::broadcast`
- Issue #944[6194] - OS X: assertion `pu_offset_ < hardware_concurrency` failed
- Issue #943[6195] - #include <hpx/hpx_main.hpp> does not work
- Issue #942[6196] - Make the BG/Q work with -O3
- Issue #940[6197] - Use separator when concatenating locality name
- Issue #939[6198] - Refactor MPI parcelport to use `MPI_Wait` instead of multiple `MPI_Test` calls
- Issue #938[6199] - Want to officially access `client_base::gid_`
- Issue #937[6200] - `client_base::gid_` should be private``
- Issue #936[6201] - Want doxygen-like source code index
- Issue #935[6202] - Build error with gcc 4.6 and Boost 1.54.0 on hpx trunk and 0.9.6
- Issue #933[6203] - Cannot build HPX with Boost 1.54.0
- Issue #932[6204] - Components are destructed too early
- Issue #931[6205] - Make HPX work on BG/Q
- Issue #930[6206] - make git-docs is broken
- Issue #929[6207] - Generating index in docs broken
- Issue #928[6208] - Optimize `hpx::util::static_` for C++11 compilers supporting magic statics
- Issue #924[6209] - Make kill_process_tree (in process.py) more robust on Mac OSX

[6188] https://github.com/TheHPXProject/hpx/issues/952
[6189] https://github.com/TheHPXProject/hpx/issues/951
[6190] https://github.com/TheHPXProject/hpx/issues/948
[6191] https://github.com/TheHPXProject/hpx/issues/947
[6192] https://github.com/TheHPXProject/hpx/issues/946
[6193] https://github.com/TheHPXProject/hpx/issues/945
[6194] https://github.com/TheHPXProject/hpx/issues/944
[6195] https://github.com/TheHPXProject/hpx/issues/943
[6196] https://github.com/TheHPXProject/hpx/issues/942
[6197] https://github.com/TheHPXProject/hpx/issues/940
[6198] https://github.com/TheHPXProject/hpx/issues/939
[6199] https://github.com/TheHPXProject/hpx/issues/938
[6200] https://github.com/TheHPXProject/hpx/issues/937
[6201] https://github.com/TheHPXProject/hpx/issues/936
[6202] https://github.com/TheHPXProject/hpx/issues/935
[6203] https://github.com/TheHPXProject/hpx/issues/933
[6204] https://github.com/TheHPXProject/hpx/issues/932
[6205] https://github.com/TheHPXProject/hpx/issues/931
[6206] https://github.com/TheHPXProject/hpx/issues/930
[6207] https://github.com/TheHPXProject/hpx/issues/929
[6208] https://github.com/TheHPXProject/hpx/issues/928
[6209] https://github.com/TheHPXProject/hpx/issues/924

- Issue #923[6210] - Correct BLAS and RNPL cmake tests
- Issue #922[6211] - Cannot link against BLAS
- Issue #921[6212] - Implement `hpx::mem_fn`
- Issue #920[6213] - Output locality with `--hpx:print-bind`
- Issue #919[6214] - Correct grammar; simplify boolean expressions
- Issue #918[6215] - Link to hello_world.cpp is broken
- Issue #917[6216] - adapt cmake file to new boostbook version
- Issue #916[6217] - fix problem building documentation with xsltproc >= 1.1.27
- Issue #915[6218] - Add another TBBMalloc library search path
- Issue #914[6219] - Build problem with Intel compiler on Stampede (TACC)
- Issue #913[6220] - fix error messages in fibonacci examples
- Issue #911[6221] - Update OS X build instructions
- Issue #910[6222] - Want like to specify MPI_ROOT instead of compiler wrapper script
- Issue #909[6223] - Warning about void* arithmetic
- Issue #908[6224] - Buildbot for MIC is broken
- Issue #906[6225] - Can't use `--hpx:bind=balanced` with multiple MPI processes
- Issue #905[6226] - `--hpx:bind` documentation should describe full grammar
- Issue #904[6227] - Add hpx::lcos::fold and hpx::lcos::inverse_fold collective operation
- Issue #903[6228] - Add `hpx::when_any_swapped()`
- Issue #902[6229] - Add `hpx::lcos::reduce` collective operation
- Issue #901[6230] - Web documentation is not searchable
- Issue #900[6231] - Web documentation for trunk has no index
- Issue #898[6232] - Some tests fail with GCC 4.8.1 and MPI parcel port

[6210] https://github.com/TheHPXProject/hpx/issues/923
[6211] https://github.com/TheHPXProject/hpx/issues/922
[6212] https://github.com/TheHPXProject/hpx/issues/921
[6213] https://github.com/TheHPXProject/hpx/issues/920
[6214] https://github.com/TheHPXProject/hpx/issues/919
[6215] https://github.com/TheHPXProject/hpx/issues/918
[6216] https://github.com/TheHPXProject/hpx/issues/917
[6217] https://github.com/TheHPXProject/hpx/issues/916
[6218] https://github.com/TheHPXProject/hpx/issues/915
[6219] https://github.com/TheHPXProject/hpx/issues/914
[6220] https://github.com/TheHPXProject/hpx/issues/913
[6221] https://github.com/TheHPXProject/hpx/issues/911
[6222] https://github.com/TheHPXProject/hpx/issues/910
[6223] https://github.com/TheHPXProject/hpx/issues/909
[6224] https://github.com/TheHPXProject/hpx/issues/908
[6225] https://github.com/TheHPXProject/hpx/issues/906
[6226] https://github.com/TheHPXProject/hpx/issues/905
[6227] https://github.com/TheHPXProject/hpx/issues/904
[6228] https://github.com/TheHPXProject/hpx/issues/903
[6229] https://github.com/TheHPXProject/hpx/issues/902
[6230] https://github.com/TheHPXProject/hpx/issues/901
[6231] https://github.com/TheHPXProject/hpx/issues/900
[6232] https://github.com/TheHPXProject/hpx/issues/898

- Issue #897[6233] - HWLOC causes failures on Mac
- Issue #896[6234] - pu-offset leads to startup error
- Issue #895[6235] - `hpx::get_locality_name` not defined
- Issue #894[6236] - Race condition at shutdown
- Issue #893[6237] - `--hpx:print-bind` switches std::cout to hexadecimal mode
- Issue #892[6238] - `hwloc_topology_load` can be expensive – don't call multiple times
- Issue #891[6239] - The documentation for `get_locality_name` is wrong
- Issue #890[6240] - `--hpx:print-bind` should not exit
- Issue #889[6241] - `--hpx:debug-hpx-log=FILE` does not work
- Issue #888[6242] - MPI parcelport does not exit cleanly for –hpx:print-bind
- Issue #887[6243] - Choose thread affinities more cleverly
- Issue #886[6244] - Logging documentation is confusing
- Issue #885[6245] - Two threads are slower than one
- Issue #884[6246] - is_callable failing with member pointers in C++11
- Issue #883[6247] - Need help with is_callable_test
- Issue #882[6248] - tests.regressions.lcos.future_hang_on_get does not terminate
- Issue #881[6249] - tests/regressions/block_matrix/matrix.hh won't compile with GCC 4.8.1
- Issue #880[6250] - HPX does not work on OS X
- Issue #878[6251] - `future::unwrap` triggers assertion
- Issue #877[6252] - "make tests" has build errors on Ubuntu 12.10
- Issue #876[6253] - tcmalloc is used by default, even if it is not present
- Issue #875[6254] - global_fixture is defined in a header file
- Issue #874[6255] - Some tests take very long

[6233] https://github.com/TheHPXProject/hpx/issues/897
[6234] https://github.com/TheHPXProject/hpx/issues/896
[6235] https://github.com/TheHPXProject/hpx/issues/895
[6236] https://github.com/TheHPXProject/hpx/issues/894
[6237] https://github.com/TheHPXProject/hpx/issues/893
[6238] https://github.com/TheHPXProject/hpx/issues/892
[6239] https://github.com/TheHPXProject/hpx/issues/891
[6240] https://github.com/TheHPXProject/hpx/issues/890
[6241] https://github.com/TheHPXProject/hpx/issues/889
[6242] https://github.com/TheHPXProject/hpx/issues/888
[6243] https://github.com/TheHPXProject/hpx/issues/887
[6244] https://github.com/TheHPXProject/hpx/issues/886
[6245] https://github.com/TheHPXProject/hpx/issues/885
[6246] https://github.com/TheHPXProject/hpx/issues/884
[6247] https://github.com/TheHPXProject/hpx/issues/883
[6248] https://github.com/TheHPXProject/hpx/issues/882
[6249] https://github.com/TheHPXProject/hpx/issues/881
[6250] https://github.com/TheHPXProject/hpx/issues/880
[6251] https://github.com/TheHPXProject/hpx/issues/878
[6252] https://github.com/TheHPXProject/hpx/issues/877
[6253] https://github.com/TheHPXProject/hpx/issues/876
[6254] https://github.com/TheHPXProject/hpx/issues/875
[6255] https://github.com/TheHPXProject/hpx/issues/874

- Issue #873[6256] - Add block-matrix code as regression test
- Issue #872[6257] - HPX documentation does not say how to run tests with detailed output
- Issue #871[6258] - All tests fail with "make test"
- Issue #870[6259] - Please explicitly disable serialization in classes that don't support it
- Issue #868[6260] - boost_any test failing
- Issue #867[6261] - Reduce the number of copies of `hpx::function` arguments
- Issue #863[6262] - Futures should not require a default constructor
- Issue #862[6263] - value_or_error shall not default construct its result
- Issue #861[6264] - `HPX_UNUSED` macro
- Issue #860[6265] - Add functionality to copy construct a component
- Issue #859[6266] - `hpx::endl` should flush
- Issue #858[6267] - Create `hpx::get_ptr`<> allowing to access component implementation
- Issue #855[6268] - Implement `hpx::INVOKE`
- Issue #854[6269] - `hpx/hpx.hpp` does not include `hpx/include/iostreams.hpp`
- Issue #853[6270] - Feature request: null future
- Issue #852[6271] - Feature request: Locality names
- Issue #851[6272] - `hpx::cout` output does not appear on screen
- Issue #849[6273] - All tests fail on OS X after installing
- Issue #848[6274] - Update OS X build instructions
- Issue #846[6275] - Update hpx_external_example
- Issue #845[6276] - Issues with having both debug and release modules in the same directory
- Issue #844[6277] - Create configuration header
- Issue #843[6278] - Tests should use CTest

6256 https://github.com/TheHPXProject/hpx/issues/873
6257 https://github.com/TheHPXProject/hpx/issues/872
6258 https://github.com/TheHPXProject/hpx/issues/871
6259 https://github.com/TheHPXProject/hpx/issues/870
6260 https://github.com/TheHPXProject/hpx/issues/868
6261 https://github.com/TheHPXProject/hpx/issues/867
6262 https://github.com/TheHPXProject/hpx/issues/863
6263 https://github.com/TheHPXProject/hpx/issues/862
6264 https://github.com/TheHPXProject/hpx/issues/861
6265 https://github.com/TheHPXProject/hpx/issues/860
6266 https://github.com/TheHPXProject/hpx/issues/859
6267 https://github.com/TheHPXProject/hpx/issues/858
6268 https://github.com/TheHPXProject/hpx/issues/855
6269 https://github.com/TheHPXProject/hpx/issues/854
6270 https://github.com/TheHPXProject/hpx/issues/853
6271 https://github.com/TheHPXProject/hpx/issues/852
6272 https://github.com/TheHPXProject/hpx/issues/851
6273 https://github.com/TheHPXProject/hpx/issues/849
6274 https://github.com/TheHPXProject/hpx/issues/848
6275 https://github.com/TheHPXProject/hpx/issues/846
6276 https://github.com/TheHPXProject/hpx/issues/845
6277 https://github.com/TheHPXProject/hpx/issues/844
6278 https://github.com/TheHPXProject/hpx/issues/843

- Issue #842[6279] - Remove buffer_pool from MPI parcelport
- Issue #841[6280] - Add possibility to broadcast an index with hpx::lcos::broadcast
- Issue #838[6281] - Simplify `util::tuple`
- Issue #837[6282] - Adopt boost::tuple tests for `util::tuple`
- Issue #836[6283] - Adopt boost::function tests for `util::function`
- Issue #835[6284] - Tuple interface missing pieces
- Issue #833[6285] - Partially preprocessing files not working
- Issue #832[6286] - Native papi counters do not work with wild cards
- Issue #831[6287] - Arithmetics counter fails if only one parameter is given
- Issue #830[6288] - Convert hpx::util::function to use new scheme for serializing its base pointer
- Issue #829[6289] - Consistently use `decay<T>` instead of `remove_const< remove_reference<T>>`
- Issue #828[6290] - Update future implementation to N3721 and N3722
- Issue #827[6291] - Enable MPI parcelport for bootstrapping whenever application was started using mpirun
- Issue #826[6292] - Support command line option `--hpx:print-bind` even if `--hpx::bind` was not used
- Issue #825[6293] - Memory counters give segfault when attempting to use thread wild cards or numbers only total works
- Issue #824[6294] - Enable lambda functions to be used with hpx::async/hpx::apply
- Issue #823[6295] - Using a hashing filter
- Issue #822[6296] - Silence unused variable warning
- Issue #821[6297] - Detect if a function object is callable with given arguments
- Issue #820[6298] - Allow wildcards to be used for performance counter names
- Issue #819[6299] - Make the AGAS symbolic name registry distributed

6279 https://github.com/TheHPXProject/hpx/issues/842
6280 https://github.com/TheHPXProject/hpx/issues/841
6281 https://github.com/TheHPXProject/hpx/issues/838
6282 https://github.com/TheHPXProject/hpx/issues/837
6283 https://github.com/TheHPXProject/hpx/issues/836
6284 https://github.com/TheHPXProject/hpx/issues/835
6285 https://github.com/TheHPXProject/hpx/issues/833
6286 https://github.com/TheHPXProject/hpx/issues/832
6287 https://github.com/TheHPXProject/hpx/issues/831
6288 https://github.com/TheHPXProject/hpx/issues/830
6289 https://github.com/TheHPXProject/hpx/issues/829
6290 https://github.com/TheHPXProject/hpx/issues/828
6291 https://github.com/TheHPXProject/hpx/issues/827
6292 https://github.com/TheHPXProject/hpx/issues/826
6293 https://github.com/TheHPXProject/hpx/issues/825
6294 https://github.com/TheHPXProject/hpx/issues/824
6295 https://github.com/TheHPXProject/hpx/issues/823
6296 https://github.com/TheHPXProject/hpx/issues/822
6297 https://github.com/TheHPXProject/hpx/issues/821
6298 https://github.com/TheHPXProject/hpx/issues/820
6299 https://github.com/TheHPXProject/hpx/issues/819

- Issue #818[6300] - Add future::then() overload taking an executor
- Issue #817[6301] - Fixed typo
- Issue #815[6302] - Create an lco that is performing an efficient broadcast of actions
- Issue #814[6303] - Papi counters cannot specify thread#* to get the counts for all threads
- Issue #813[6304] - Scoped unlock
- Issue #811[6305] - simple_central_tuplespace_client run error
- Issue #810[6306] - ostream error when << any objects
- Issue #809[6307] - Optimize parcel serialization
- Issue #808[6308] - HPX applications throw exception when executed from the build directory
- Issue #807[6309] - Create performance counters exposing overall AGAS statistics
- Issue #795[6310] - Create timed make_ready_future
- Issue #794[6311] - Create heterogeneous `when_all`/`when_any`/etc.
- Issue #721[6312] - Make HPX usable for Xeon Phi
- Issue #694[6313] - CMake should complain if you attempt to build an example without its dependencies
- Issue #692[6314] - SLURM support broken
- Issue #683[6315] - python/hpx/process.py imports epoll on all platforms
- Issue #619[6316] - Automate the doc building process
- Issue #600[6317] - GTC performance broken
- Issue #577[6318] - Allow for zero copy serialization/networking
- Issue #551[6319] - Change executable names to have debug postfix in Debug builds
- Issue #544[6320] - Write a custom .lib file on Windows pulling in hpx_init and hpx.dll, phase out hpx_init

6300 https://github.com/TheHPXProject/hpx/issues/818
6301 https://github.com/TheHPXProject/hpx/issues/817
6302 https://github.com/TheHPXProject/hpx/issues/815
6303 https://github.com/TheHPXProject/hpx/issues/814
6304 https://github.com/TheHPXProject/hpx/issues/813
6305 https://github.com/TheHPXProject/hpx/issues/811
6306 https://github.com/TheHPXProject/hpx/issues/810
6307 https://github.com/TheHPXProject/hpx/issues/809
6308 https://github.com/TheHPXProject/hpx/issues/808
6309 https://github.com/TheHPXProject/hpx/issues/807
6310 https://github.com/TheHPXProject/hpx/issues/795
6311 https://github.com/TheHPXProject/hpx/issues/794
6312 https://github.com/TheHPXProject/hpx/issues/721
6313 https://github.com/TheHPXProject/hpx/issues/694
6314 https://github.com/TheHPXProject/hpx/issues/692
6315 https://github.com/TheHPXProject/hpx/issues/683
6316 https://github.com/TheHPXProject/hpx/issues/619
6317 https://github.com/TheHPXProject/hpx/issues/600
6318 https://github.com/TheHPXProject/hpx/issues/577
6319 https://github.com/TheHPXProject/hpx/issues/551
6320 https://github.com/TheHPXProject/hpx/issues/544

- Issue #534[6321] - `hpx::init` should take functions by `std::function` and should accept all forms of hpx_main
- Issue #508[6322] - FindPackage fails to set FOO_LIBRARY_DIR
- Issue #506[6323] - Add cmake support to generate ini files for external applications
- Issue #470[6324] - Changing build-type after configure does not update boost library names
- Issue #453[6325] - Document `hpx_run_tests.py`
- Issue #445[6326] - Significant performance mismatch between MPI and HPX in SMP for allgather example
- Issue #443[6327] - Make docs viewable from build directory
- Issue #421[6328] - Support multiple HPX instances per node in a batch environment like PBS or SLURM
- Issue #316[6329] - Add message size limitation
- Issue #249[6330] - Clean up locking code in big boot barrier
- Issue #136[6331] - Persistent CMake variables need to be marked as cache variables

HPX V0.9.6 (Jul 30, 2013)

We have had over 1200 commits since the last release and we have closed roughly 140 tickets (bugs, feature requests, etc.).

General changes

The major new features in this release are:

- We further consolidated the API exposed by *HPX*. We aligned our APIs as much as possible with the existing C++11 Standard[6332] and related proposals to the C++ standardization committee (such as N3632[6333] and N3857[6334]).
- We implemented a first version of a distributed AGAS service which essentially eliminates all explicit AGAS network traffic.
- We created a native ibverbs parcelport allowing to take advantage of the superior latency and bandwidth characteristics of Infiniband networks.
- We successfully ported *HPX* to the Xeon Phi platform.
- Support for the SLURM scheduling system was implemented.

6321 https://github.com/TheHPXProject/hpx/issues/534
6322 https://github.com/TheHPXProject/hpx/issues/508
6323 https://github.com/TheHPXProject/hpx/issues/506
6324 https://github.com/TheHPXProject/hpx/issues/470
6325 https://github.com/TheHPXProject/hpx/issues/453
6326 https://github.com/TheHPXProject/hpx/issues/445
6327 https://github.com/TheHPXProject/hpx/issues/443
6328 https://github.com/TheHPXProject/hpx/issues/421
6329 https://github.com/TheHPXProject/hpx/issues/316
6330 https://github.com/TheHPXProject/hpx/issues/249
6331 https://github.com/TheHPXProject/hpx/issues/136
6332 http://www.open-std.org/jtc1/sc22/wg21
6333 http://wg21.link/n3632
6334 http://wg21.link/n3857

- Major efforts have been dedicated to improving the performance counter framework, numerous new counters were implemented and new APIs were added.
- We added a modular parcel compression system allowing to improve bandwidth utilization (by reducing the overall size of the transferred data).
- We added a modular parcel coalescing system allowing to combine several parcels into larger messages. This reduces latencies introduced by the communication layer.
- Added an experimental executors API allowing to use different scheduling policies for different parts of the code. This API has been modelled after the Standards proposal N3562[6335]. This API is bound to change in the future, though.
- Added minimal security support for localities which is enforced on the parcelport level. This support is preliminary and experimental and might change in the future.
- We created a parcelport using low level MPI functions. This is in support of legacy applications which are to be gradually ported and to support platforms where MPI is the only available portable networking layer.
- We added a preliminary and experimental implementation of a tuple-space object which exposes an interface similar to such systems described in the literature (see for instance The Linda Coordination Language[6336]).

Bug fixes (closed tickets)

Here is a list of the important tickets we closed for this release. This is again a very long list of newly implemented features and fixed issues.

- Issue #806[6337] - make (all) in examples folder does nothing
- Issue #805[6338] - Adding the introduction and fixing DOCBOOK dependencies for Windows use
- Issue #804[6339] - Add stackless (non-suspendable) thread type
- Issue #803[6340] - Create proper serialization support functions for util::tuple
- Issue #800[6341] - Add possibility to disable array optimizations during serialization
- Issue #798[6342] - HPX_LIMIT does not work for local dataflow
- Issue #797[6343] - Create a parcelport which uses MPI
- Issue #796[6344] - Problem with Large Numbers of Threads
- Issue #793[6345] - Changing dataflow test case to hang consistently
- Issue #792[6346] - CMake Error

6335 http://wg21.link/n3562
6336 https://en.wikipedia.org/wiki/Linda_(coordination_language)
6337 https://github.com/TheHPXProject/hpx/issues/806
6338 https://github.com/TheHPXProject/hpx/issues/805
6339 https://github.com/TheHPXProject/hpx/issues/804
6340 https://github.com/TheHPXProject/hpx/issues/803
6341 https://github.com/TheHPXProject/hpx/issues/800
6342 https://github.com/TheHPXProject/hpx/issues/798
6343 https://github.com/TheHPXProject/hpx/issues/797
6344 https://github.com/TheHPXProject/hpx/issues/796
6345 https://github.com/TheHPXProject/hpx/issues/793
6346 https://github.com/TheHPXProject/hpx/issues/792

- Issue #791[6347] - Problems with local::dataflow
- Issue #790[6348] - wait_for() doesn't compile
- Issue #789[6349] - HPX with Intel compiler segfaults
- Issue #788[6350] - Intel compiler support
- Issue #787[6351] - Fixed SFINAEd specializations
- Issue #786[6352] - Memory issues during benchmarking.
- Issue #785[6353] - Create an API allowing to register external threads with HPX
- Issue #784[6354] - util::plugin is throwing an error when a symbol is not found
- Issue #783[6355] - How does hpx:bind work?
- Issue #782[6356] - Added quotes around STRING REPLACE potentially empty arguments
- Issue #781[6357] - Make sure no exceptions propagate into the thread manager
- Issue #780[6358] - Allow arithmetics performance counters to expand its parameters
- Issue #779[6359] - Test case for 778
- Issue #778[6360] - Swapping futures segfaults
- Issue #777[6361] - hpx::lcos::details::when_xxx don't restore completion handlers
- Issue #776[6362] - Compiler chokes on dataflow overload with launch policy
- Issue #775[6363] - Runtime error with local dataflow (copying futures?)
- Issue #774[6364] - Using local dataflow without explicit namespace
- Issue #773[6365] - Local dataflow with unwrap: functor operators need to be const
- Issue #772[6366] - Allow (remote) actions to return a future
- Issue #771[6367] - Setting HPX_LIMIT gives huge boost MPL errors
- Issue #770[6368] - Add launch policy to (local) dataflow
- Issue #769[6369] - Make compile time configuration information available

[6347] https://github.com/TheHPXProject/hpx/issues/791
[6348] https://github.com/TheHPXProject/hpx/issues/790
[6349] https://github.com/TheHPXProject/hpx/issues/789
[6350] https://github.com/TheHPXProject/hpx/issues/788
[6351] https://github.com/TheHPXProject/hpx/issues/787
[6352] https://github.com/TheHPXProject/hpx/issues/786
[6353] https://github.com/TheHPXProject/hpx/issues/785
[6354] https://github.com/TheHPXProject/hpx/issues/784
[6355] https://github.com/TheHPXProject/hpx/issues/783
[6356] https://github.com/TheHPXProject/hpx/issues/782
[6357] https://github.com/TheHPXProject/hpx/issues/781
[6358] https://github.com/TheHPXProject/hpx/issues/780
[6359] https://github.com/TheHPXProject/hpx/issues/779
[6360] https://github.com/TheHPXProject/hpx/issues/778
[6361] https://github.com/TheHPXProject/hpx/issues/777
[6362] https://github.com/TheHPXProject/hpx/issues/776
[6363] https://github.com/TheHPXProject/hpx/issues/775
[6364] https://github.com/TheHPXProject/hpx/issues/774
[6365] https://github.com/TheHPXProject/hpx/issues/773
[6366] https://github.com/TheHPXProject/hpx/issues/772
[6367] https://github.com/TheHPXProject/hpx/issues/771
[6368] https://github.com/TheHPXProject/hpx/issues/770
[6369] https://github.com/TheHPXProject/hpx/issues/769

- Issue #768[6370] - Const correctness problem in local dataflow
- Issue #767[6371] - Add launch policies to async
- Issue #766[6372] - Mark data structures for optimized (array based) serialization
- Issue #765[6373] - Align hpx::any with N3508: Any Library Proposal (Revision 2)
- Issue #764[6374] - Align hpx::future with newest N3558: A Standardized Representation of Asynchronous Operations
- Issue #762[6375] - added a human readable output for the ping pong example
- Issue #761[6376] - Ambiguous typename when constructing derived component
- Issue #760[6377] - Simple components can not be derived
- Issue #759[6378] - make install doesn't give a complete install
- Issue #758[6379] - Stack overflow when using locking_hook<>
- Issue #757[6380] - copy paste error; unsupported function overloading
- Issue #756[6381] - GTCX runtime issue in Gordon
- Issue #755[6382] - Papi counters don't work with reset and evaluate API's
- Issue #753[6383] - cmake bugfix and improved component action docs
- Issue #752[6384] - hpx simple component docs
- Issue #750[6385] - Add hpx::util::any
- Issue #749[6386] - Thread phase counter is not reset
- Issue #748[6387] - Memory performance counter are not registered
- Issue #747[6388] - Create performance counters exposing arithmetic operations
- Issue #745[6389] - apply_callback needs to invoke callback when applied locally
- Issue #744[6390] - CMake fixes
- Issue #743[6391] - Problem Building github version of HPX
- Issue #742[6392] - Remove HPX_STD_BIND

[6370] https://github.com/TheHPXProject/hpx/issues/768
[6371] https://github.com/TheHPXProject/hpx/issues/767
[6372] https://github.com/TheHPXProject/hpx/issues/766
[6373] https://github.com/TheHPXProject/hpx/issues/765
[6374] https://github.com/TheHPXProject/hpx/issues/764
[6375] https://github.com/TheHPXProject/hpx/issues/762
[6376] https://github.com/TheHPXProject/hpx/issues/761
[6377] https://github.com/TheHPXProject/hpx/issues/760
[6378] https://github.com/TheHPXProject/hpx/issues/759
[6379] https://github.com/TheHPXProject/hpx/issues/758
[6380] https://github.com/TheHPXProject/hpx/issues/757
[6381] https://github.com/TheHPXProject/hpx/issues/756
[6382] https://github.com/TheHPXProject/hpx/issues/755
[6383] https://github.com/TheHPXProject/hpx/issues/753
[6384] https://github.com/TheHPXProject/hpx/issues/752
[6385] https://github.com/TheHPXProject/hpx/issues/750
[6386] https://github.com/TheHPXProject/hpx/issues/749
[6387] https://github.com/TheHPXProject/hpx/issues/748
[6388] https://github.com/TheHPXProject/hpx/issues/747
[6389] https://github.com/TheHPXProject/hpx/issues/745
[6390] https://github.com/TheHPXProject/hpx/issues/744
[6391] https://github.com/TheHPXProject/hpx/issues/743
[6392] https://github.com/TheHPXProject/hpx/issues/742

- Issue #741[6393] - assertion 'px != 0' failed: HPX(assertion_failure) for low numbers of OS threads
- Issue #739[6394] - Performance counters do not count to the end of the program or evaluation
- Issue #738[6395] - Dedicated AGAS server runs don't work; console ignores -a option.
- Issue #737[6396] - Missing bind overloads
- Issue #736[6397] - Performance counter wildcards do not always work
- Issue #735[6398] - Create native ibverbs parcelport based on rdma operations
- Issue #734[6399] - Threads stolen performance counter total is incorrect
- Issue #733[6400] - Test benchmarks need to be checked and fixed
- Issue #732[6401] - Build fails with Mac, using mac ports clang-3.3 on latest git branch
- Issue #731[6402] - Add global start/stop API for performance counters
- Issue #730[6403] - Performance counter values are apparently incorrect
- Issue #729[6404] - Unhandled switch
- Issue #728[6405] - Serialization of hpx::util::function between two localities causes seg faults
- Issue #727[6406] - Memory counters on Mac OS X
- Issue #725[6407] - Restore original thread priority on resume
- Issue #724[6408] - Performance benchmarks do not depend on main HPX libraries
- Issue #723[6409] - [teletype]–hpx:nodes=``cat $PBS_NODEFILE`` works; –hpx:nodefile=$PBS_NODEFILE does not.[c++]
- Issue #722[6410] - Fix binding const member functions as actions
- Issue #719[6411] - Create performance counter exposing compression ratio
- Issue #718[6412] - Add possibility to compress parcel data
- Issue #717[6413] - strip_credit_from_gid has misleading semantics

6393 https://github.com/TheHPXProject/hpx/issues/741
6394 https://github.com/TheHPXProject/hpx/issues/739
6395 https://github.com/TheHPXProject/hpx/issues/738
6396 https://github.com/TheHPXProject/hpx/issues/737
6397 https://github.com/TheHPXProject/hpx/issues/736
6398 https://github.com/TheHPXProject/hpx/issues/735
6399 https://github.com/TheHPXProject/hpx/issues/734
6400 https://github.com/TheHPXProject/hpx/issues/733
6401 https://github.com/TheHPXProject/hpx/issues/732
6402 https://github.com/TheHPXProject/hpx/issues/731
6403 https://github.com/TheHPXProject/hpx/issues/730
6404 https://github.com/TheHPXProject/hpx/issues/729
6405 https://github.com/TheHPXProject/hpx/issues/728
6406 https://github.com/TheHPXProject/hpx/issues/727
6407 https://github.com/TheHPXProject/hpx/issues/725
6408 https://github.com/TheHPXProject/hpx/issues/724
6409 https://github.com/TheHPXProject/hpx/issues/723
6410 https://github.com/TheHPXProject/hpx/issues/722
6411 https://github.com/TheHPXProject/hpx/issues/719
6412 https://github.com/TheHPXProject/hpx/issues/718
6413 https://github.com/TheHPXProject/hpx/issues/717

- Issue #716[6414] - Non-option arguments to programs run using `pbsdsh` must be before `--hpx:nodes`, contrary to directions
- Issue #715[6415] - Re-thrown exceptions should retain the original call site
- Issue #714[6416] - failed assertion in debug mode
- Issue #713[6417] - Add performance counters monitoring connection caches
- Issue #712[6418] - Adjust parcel related performance counters to be connection type specific
- Issue #711[6419] - configuration failure
- Issue #710[6420] - Error "timed out while trying to find room in the connection cache" when trying to start multiple localities on a single computer
- Issue #709[6421] - Add new thread state 'staged' referring to task descriptions
- Issue #708[6422] - Detect/mitigate bad non-system installs of GCC on Redhat systems
- Issue #707[6423] - Many examples do not link with Git HEAD version
- Issue #706[6424] - `hpx::init` removes portions of non-option command line arguments before last = sign
- Issue #705[6425] - Create rolling average and median aggregating performance counters
- Issue #704[6426] - Create performance counter to expose thread queue waiting time
- Issue #703[6427] - Add support to HPX build system to find librcrtool.a and related headers
- Issue #699[6428] - Generalize instrumentation support
- Issue #698[6429] - compilation failure with hwloc absent
- Issue #697[6430] - Performance counter counts should be zero indexed
- Issue #696[6431] - Distributed problem
- Issue #695[6432] - Bad perf counter time printed
- Issue #693[6433] - `--help` doesn't print component specific command line options
- Issue #692[6434] - SLURM support broken

6414 https://github.com/TheHPXProject/hpx/issues/716
6415 https://github.com/TheHPXProject/hpx/issues/715
6416 https://github.com/TheHPXProject/hpx/issues/714
6417 https://github.com/TheHPXProject/hpx/issues/713
6418 https://github.com/TheHPXProject/hpx/issues/712
6419 https://github.com/TheHPXProject/hpx/issues/711
6420 https://github.com/TheHPXProject/hpx/issues/710
6421 https://github.com/TheHPXProject/hpx/issues/709
6422 https://github.com/TheHPXProject/hpx/issues/708
6423 https://github.com/TheHPXProject/hpx/issues/707
6424 https://github.com/TheHPXProject/hpx/issues/706
6425 https://github.com/TheHPXProject/hpx/issues/705
6426 https://github.com/TheHPXProject/hpx/issues/704
6427 https://github.com/TheHPXProject/hpx/issues/703
6428 https://github.com/TheHPXProject/hpx/issues/699
6429 https://github.com/TheHPXProject/hpx/issues/698
6430 https://github.com/TheHPXProject/hpx/issues/697
6431 https://github.com/TheHPXProject/hpx/issues/696
6432 https://github.com/TheHPXProject/hpx/issues/695
6433 https://github.com/TheHPXProject/hpx/issues/693
6434 https://github.com/TheHPXProject/hpx/issues/692

- Issue #691[6435] - exception while executing any application linked with hwloc
- Issue #690[6436] - thread_id_test and thread_launcher_test failing
- Issue #689[6437] - Make the buildbots use hwloc
- Issue #687[6438] - compilation error fix (hwloc_topology)
- Issue #686[6439] - Linker Error for Applications
- Issue #684[6440] - Pinning of service thread fails when number of worker threads equals the number of cores
- Issue #682[6441] - Add performance counters exposing number of stolen threads
- Issue #681[6442] - Add apply_continue for asynchronous chaining of actions
- Issue #679[6443] - Remove obsolete async_callback API functions
- Issue #678[6444] - Add new API for setting/triggering LCOs
- Issue #677[6445] - Add async_continue for true continuation style actions
- Issue #676[6446] - Buildbot for gcc 4.4 broken
- Issue #675[6447] - Partial preprocessing broken
- Issue #674[6448] - HPX segfaults when built with gcc 4.7
- Issue #673[6449] - `use_guard_pages` has inconsistent preprocessor guards
- Issue #672[6450] - External build breaks if library path has spaces
- Issue #671[6451] - release tarballs are tarbombs
- Issue #670[6452] - CMake won't find Boost headers in layout=versioned install
- Issue #669[6453] - Links in docs to source files broken if not installed
- Issue #667[6454] - Not reading ini file properly
- Issue #664[6455] - Adapt new meanings of 'const' and 'mutable'
- Issue #661[6456] - Implement BTL Parcel port
- Issue #655[6457] - Make HPX work with the "decltype" result_of

6435 https://github.com/TheHPXProject/hpx/issues/691
6436 https://github.com/TheHPXProject/hpx/issues/690
6437 https://github.com/TheHPXProject/hpx/issues/689
6438 https://github.com/TheHPXProject/hpx/issues/687
6439 https://github.com/TheHPXProject/hpx/issues/686
6440 https://github.com/TheHPXProject/hpx/issues/684
6441 https://github.com/TheHPXProject/hpx/issues/682
6442 https://github.com/TheHPXProject/hpx/issues/681
6443 https://github.com/TheHPXProject/hpx/issues/679
6444 https://github.com/TheHPXProject/hpx/issues/678
6445 https://github.com/TheHPXProject/hpx/issues/677
6446 https://github.com/TheHPXProject/hpx/issues/676
6447 https://github.com/TheHPXProject/hpx/issues/675
6448 https://github.com/TheHPXProject/hpx/issues/674
6449 https://github.com/TheHPXProject/hpx/issues/673
6450 https://github.com/TheHPXProject/hpx/issues/672
6451 https://github.com/TheHPXProject/hpx/issues/671
6452 https://github.com/TheHPXProject/hpx/issues/670
6453 https://github.com/TheHPXProject/hpx/issues/669
6454 https://github.com/TheHPXProject/hpx/issues/667
6455 https://github.com/TheHPXProject/hpx/issues/664
6456 https://github.com/TheHPXProject/hpx/issues/661
6457 https://github.com/TheHPXProject/hpx/issues/655

- Issue #647[6458] - documentation for specifying the number of high priority threads `--hpx:high-priority-threads`
- Issue #643[6459] - Error parsing host file
- Issue #642[6460] - HWLoc issue with TAU
- Issue #639[6461] - Logging potentially suspends a running thread
- Issue #634[6462] - Improve error reporting from parcel layer
- Issue #627[6463] - Add tests for async and apply overloads that accept regular C++ functions
- Issue #626[6464] - hpx/future.hpp header
- Issue #601[6465] - Intel support
- Issue #557[6466] - Remove action codes
- Issue #531[6467] - AGAS request and response classes should use switch statements
- Issue #529[6468] - Investigate the state of hwloc support
- Issue #526[6469] - Make HPX aware of hyper-threading
- Issue #518[6470] - Create facilities allowing to use plain arrays as action arguments
- Issue #473[6471] - hwloc thread binding is broken on CPUs with hyperthreading
- Issue #383[6472] - Change result type detection for hpx::util::bind to use result_of protocol
- Issue #341[6473] - Consolidate route code
- Issue #219[6474] - Only copy arguments into actions once
- Issue #177[6475] - Implement distributed AGAS
- Issue #43[6476] - Support for Darwin (Xcode + Clang)

6458 https://github.com/TheHPXProject/hpx/issues/647
6459 https://github.com/TheHPXProject/hpx/issues/643
6460 https://github.com/TheHPXProject/hpx/issues/642
6461 https://github.com/TheHPXProject/hpx/issues/639
6462 https://github.com/TheHPXProject/hpx/issues/634
6463 https://github.com/TheHPXProject/hpx/issues/627
6464 https://github.com/TheHPXProject/hpx/issues/626
6465 https://github.com/TheHPXProject/hpx/issues/601
6466 https://github.com/TheHPXProject/hpx/issues/557
6467 https://github.com/TheHPXProject/hpx/issues/531
6468 https://github.com/TheHPXProject/hpx/issues/529
6469 https://github.com/TheHPXProject/hpx/issues/526
6470 https://github.com/TheHPXProject/hpx/issues/518
6471 https://github.com/TheHPXProject/hpx/issues/473
6472 https://github.com/TheHPXProject/hpx/issues/383
6473 https://github.com/TheHPXProject/hpx/issues/341
6474 https://github.com/TheHPXProject/hpx/issues/219
6475 https://github.com/TheHPXProject/hpx/issues/177
6476 https://github.com/TheHPXProject/hpx/issues/43

HPX V0.9.5 (Jan 16, 2013)

We have had over 1000 commits since the last release and we have closed roughly 150 tickets (bugs, feature requests, etc.).

General changes

This release is continuing along the lines of code and API consolidation, and overall usability inprovements. We dedicated much attention to performance and we were able to significantly improve the threading and networking subsystems.

We successfully ported *HPX* to the Android platform. *HPX* applications now not only can run on mobile devices, but we support heterogeneous applications running across architecture boundaries. At the Supercomputing Conference 2012 we demonstrated connecting Android tablets to simulations running on a Linux cluster. The Android tablet was used to query performance counters from the Linux simulation and to steer its parameters.

We successfully ported *HPX* to Mac OSX (using the Clang compiler). Thanks to Pyry Jahkola for contributing the corresponding patches. Please see the section macos_installation for more details.

We made a special effort to make HPX usable in highly concurrent use cases. Many of the HPX API functions which possibly take longer than 100 microseconds to execute now can be invoked asynchronously. We added uniform support for composing futures which simplifies to write asynchronous code. HPX actions (function objects encapsulating possibly concurrent remote function invocations) are now well integrated with all other API facilities such like `hpx::bind`.

All of the API has been aligned as much as possible with established paradigms. HPX now mirrors many of the facilities as defined in the C++11 Standard, such as `hpx::thread`, `hpx::function`, `hpx::future`, etc.

A lot of work has been put into improving the documentation. Many of the API functions are documented now, concepts are explained in detail, and examples are better described than before. The new documentation index enables finding information with lesser effort.

This is the first release of HPX we perform after the move to Github[6477] This step has enabled a wider participation from the community and further encourages us in our decision to release HPX as a true open source library (HPX is licensed under the very liberal Boost Software License[6478]).

Bug fixes (closed tickets)

Here is a list of the important tickets we closed for this release. This is by far the longest list of newly implemented features and fixed issues for any of HPX' releases so far.

- Issue #666[6479] - Segfault on calling hpx::finalize twice
- Issue #665[6480] - Adding declaration num_of_cores

[6477] https://github.com/TheHPXProject/hpx/
[6478] https://www.boost.org/LICENSE_1_0.txt
[6479] https://github.com/TheHPXProject/hpx/issues/666
[6480] https://github.com/TheHPXProject/hpx/issues/665

- Issue #662[6481] - pkgconfig is building wrong
- Issue #660[6482] - Need uninterrupt function
- Issue #659[6483] - Move our logging library into a different namespace
- Issue #658[6484] - Dynamic performance counter types are broken
- Issue #657[6485] - HPX v0.9.5 (RC1) hello_world example segfaulting
- Issue #656[6486] - Define the affinity of parcel-pool, io-pool, and timer-pool threads
- Issue #654[6487] - Integrate the Boost auto_index tool with documentation
- Issue #653[6488] - Make HPX build on OS X + Clang + libc++
- Issue #651[6489] - Add fine-grained control for thread pinning
- Issue #650[6490] - Command line no error message when using -hpx:(anything)
- Issue #645[6491] - Command line aliases don't work in [teletype]``@file``[c++]
- Issue #644[6492] - Terminated threads are not always properly cleaned up
- Issue #640[6493] - `future_data<T>::set_on_completed_` used without locks
- Issue #638[6494] - hpx build with intel compilers fails on linux
- Issue #637[6495] - –copy-dt-needed-entries breaks with gold
- Issue #635[6496] - Boost V1.53 will add Boost.Lockfree and Boost.Atomic
- Issue #633[6497] - Re-add examples to final 0.9.5 release
- Issue #632[6498] - Example `thread_aware_timer` is broken
- Issue #631[6499] - FFT application throws error in parcellayer
- Issue #630[6500] - Event synchronization example is broken
- Issue #629[6501] - Waiting on futures hangs
- Issue #628[6502] - Add an `HPX_ALWAYS_ASSERT` macro
- Issue #625[6503] - Port coroutines context switch benchmark

6481 https://github.com/TheHPXProject/hpx/issues/662
6482 https://github.com/TheHPXProject/hpx/issues/660
6483 https://github.com/TheHPXProject/hpx/issues/659
6484 https://github.com/TheHPXProject/hpx/issues/658
6485 https://github.com/TheHPXProject/hpx/issues/657
6486 https://github.com/TheHPXProject/hpx/issues/656
6487 https://github.com/TheHPXProject/hpx/issues/654
6488 https://github.com/TheHPXProject/hpx/issues/653
6489 https://github.com/TheHPXProject/hpx/issues/651
6490 https://github.com/TheHPXProject/hpx/issues/650
6491 https://github.com/TheHPXProject/hpx/issues/645
6492 https://github.com/TheHPXProject/hpx/issues/644
6493 https://github.com/TheHPXProject/hpx/issues/640
6494 https://github.com/TheHPXProject/hpx/issues/638
6495 https://github.com/TheHPXProject/hpx/issues/637
6496 https://github.com/TheHPXProject/hpx/issues/635
6497 https://github.com/TheHPXProject/hpx/issues/633
6498 https://github.com/TheHPXProject/hpx/issues/632
6499 https://github.com/TheHPXProject/hpx/issues/631
6500 https://github.com/TheHPXProject/hpx/issues/630
6501 https://github.com/TheHPXProject/hpx/issues/629
6502 https://github.com/TheHPXProject/hpx/issues/628
6503 https://github.com/TheHPXProject/hpx/issues/625

- Issue #621[6504] - New INI section for stack sizes
- Issue #618[6505] - pkg_config support does not work with a HPX debug build
- Issue #617[6506] - hpx/external/logging/boost/logging/detail/cache_before_init.hpp:139:67: error: 'get_thread_id' was not declared in this scope
- Issue #616[6507] - Change wait_xxx not to use locking
- Issue #615[6508] - Revert visibility 'fix' (fb0b6b8245dad1127b0c25ebafd9386b3945cca9)
- Issue #614[6509] - Fix Dataflow linker error
- Issue #613[6510] - find_here should throw an exception on failure
- Issue #612[6511] - Thread phase doesn't show up in debug mode
- Issue #611[6512] - Make stack guard pages configurable at runtime (initialization time)
- Issue #610[6513] - Co-Locate Components
- Issue #609[6514] - future_overhead
- Issue #608[6515] - `--hpx:list-counter-infos` problem
- Issue #607[6516] - Update Boost.Context based backend for coroutines
- Issue #606[6517] - 1d_wave_equation is not working
- Issue #605[6518] - Any C++ function that has serializable arguments and a serializable return type should be remotable
- Issue #604[6519] - Connecting localities isn't working anymore
- Issue #603[6520] - Do not verify any ini entries read from a file
- Issue #602[6521] - Rename argument_size to type_size/ added implementation to get parcel size
- Issue #599[6522] - Enable locality specific command line options
- Issue #598[6523] - Need an API that accesses the performance counter reporting the system uptime
- Issue #597[6524] - compiling on ranger

[6504] https://github.com/TheHPXProject/hpx/issues/621
[6505] https://github.com/TheHPXProject/hpx/issues/618
[6506] https://github.com/TheHPXProject/hpx/issues/617
[6507] https://github.com/TheHPXProject/hpx/issues/616
[6508] https://github.com/TheHPXProject/hpx/issues/615
[6509] https://github.com/TheHPXProject/hpx/issues/614
[6510] https://github.com/TheHPXProject/hpx/issues/613
[6511] https://github.com/TheHPXProject/hpx/issues/612
[6512] https://github.com/TheHPXProject/hpx/issues/611
[6513] https://github.com/TheHPXProject/hpx/issues/610
[6514] https://github.com/TheHPXProject/hpx/issues/609
[6515] https://github.com/TheHPXProject/hpx/issues/608
[6516] https://github.com/TheHPXProject/hpx/issues/607
[6517] https://github.com/TheHPXProject/hpx/issues/606
[6518] https://github.com/TheHPXProject/hpx/issues/605
[6519] https://github.com/TheHPXProject/hpx/issues/604
[6520] https://github.com/TheHPXProject/hpx/issues/603
[6521] https://github.com/TheHPXProject/hpx/issues/602
[6522] https://github.com/TheHPXProject/hpx/issues/599
[6523] https://github.com/TheHPXProject/hpx/issues/598
[6524] https://github.com/TheHPXProject/hpx/issues/597

- Issue #595[6525] - I need a place to store data in a thread self pointer
- Issue #594[6526] - 32/64 interoperability
- Issue #593[6527] - Warn if logging is disabled at compile time but requested at runtime
- Issue #592[6528] - Add optional argument value to `--hpx:list-counters` and `--hpx:list-counter-infos`
- Issue #591[6529] - Allow for wildcards in performance counter names specified with `--hpx:print-counter`
- Issue #590[6530] - Local promise semantic differences
- Issue #589[6531] - Create API to query performance counter names
- Issue #587[6532] - Add get_num_localities and get_num_threads to AGAS API
- Issue #586[6533] - Adjust local AGAS cache size based on number of localities
- Issue #585[6534] - Error while using counters in HPX
- Issue #584[6535] - counting argument size of actions, initial pass.
- Issue #581[6536] - Remove `RemoteResult` template parameter for `future<>`
- Issue #580[6537] - Add possibility to hook into actions
- Issue #578[6538] - Use angle brackets in HPX error dumps
- Issue #576[6539] - Exception incorrectly thrown when `--help` is used
- Issue #575[6540] - HPX(bad_component_type) with gcc 4.7.2 and boost 1.51
- Issue #574[6541] - `--hpx:connect` command line parameter not working correctly
- Issue #571[6542] - `hpx::wait()` (callback version) should pass the future to the callback function
- Issue #570[6543] - `hpx::wait` should operate on `boost::arrays` and `std::lists`
- Issue #569[6544] - Add a logging sink for Android
- Issue #568[6545] - 2-argument version of `HPX_DEFINE_COMPONENT_ACTION`
- Issue #567[6546] - Connecting to a running HPX application works only once

[6525] https://github.com/TheHPXProject/hpx/issues/595
[6526] https://github.com/TheHPXProject/hpx/issues/594
[6527] https://github.com/TheHPXProject/hpx/issues/593
[6528] https://github.com/TheHPXProject/hpx/issues/592
[6529] https://github.com/TheHPXProject/hpx/issues/591
[6530] https://github.com/TheHPXProject/hpx/issues/590
[6531] https://github.com/TheHPXProject/hpx/issues/589
[6532] https://github.com/TheHPXProject/hpx/issues/587
[6533] https://github.com/TheHPXProject/hpx/issues/586
[6534] https://github.com/TheHPXProject/hpx/issues/585
[6535] https://github.com/TheHPXProject/hpx/issues/584
[6536] https://github.com/TheHPXProject/hpx/issues/581
[6537] https://github.com/TheHPXProject/hpx/issues/580
[6538] https://github.com/TheHPXProject/hpx/issues/578
[6539] https://github.com/TheHPXProject/hpx/issues/576
[6540] https://github.com/TheHPXProject/hpx/issues/575
[6541] https://github.com/TheHPXProject/hpx/issues/574
[6542] https://github.com/TheHPXProject/hpx/issues/571
[6543] https://github.com/TheHPXProject/hpx/issues/570
[6544] https://github.com/TheHPXProject/hpx/issues/569
[6545] https://github.com/TheHPXProject/hpx/issues/568
[6546] https://github.com/TheHPXProject/hpx/issues/567

- Issue #565[6547] - HPX doesn't shutdown properly
- Issue #564[6548] - Partial preprocessing of new component creation interface
- Issue #563[6549] - Add `hpx::start/hpx::stop` to avoid blocking main thread
- Issue #562[6550] - All command line arguments swallowed by hpx
- Issue #561[6551] - Boost.Tuple is not move aware
- Issue #558[6552] - `boost::shared_ptr<>` style semantics/syntax for client classes
- Issue #556[6553] - Creation of partially preprocessed headers should be enabled for Boost newer than V1.50
- Issue #555[6554] - `BOOST_FORCEINLINE` does not name a type
- Issue #554[6555] - Possible race condition in thread `get_id()`
- Issue #552[6556] - Move enable client_base
- Issue #550[6557] - Add stack size category 'huge'
- Issue #549[6558] - ShenEOS run seg-faults on single or distributed runs
- Issue #545[6559] - `AUTOGLOB` broken for add_hpx_component
- Issue #542[6560] - FindHPX_HDF5 still searches multiple times
- Issue #541[6561] - Quotes around application name in hpx::init
- Issue #539[6562] - Race conditition occurring with new lightweight threads
- Issue #535[6563] - hpx_run_tests.py exits with no error code when tests are missing
- Issue #530[6564] - Thread description(<unknown>) in logs
- Issue #523[6565] - Make thread objects more lightweight
- Issue #521[6566] - `hpx::error_code` is not usable for lightweight error handling
- Issue #520[6567] - Add full user environment to HPX logs
- Issue #519[6568] - Build succeeds, running fails
- Issue #517[6569] - Add a guard page to linux coroutine stacks

[6547] https://github.com/TheHPXProject/hpx/issues/565
[6548] https://github.com/TheHPXProject/hpx/issues/564
[6549] https://github.com/TheHPXProject/hpx/issues/563
[6550] https://github.com/TheHPXProject/hpx/issues/562
[6551] https://github.com/TheHPXProject/hpx/issues/561
[6552] https://github.com/TheHPXProject/hpx/issues/558
[6553] https://github.com/TheHPXProject/hpx/issues/556
[6554] https://github.com/TheHPXProject/hpx/issues/555
[6555] https://github.com/TheHPXProject/hpx/issues/554
[6556] https://github.com/TheHPXProject/hpx/issues/552
[6557] https://github.com/TheHPXProject/hpx/issues/550
[6558] https://github.com/TheHPXProject/hpx/issues/549
[6559] https://github.com/TheHPXProject/hpx/issues/545
[6560] https://github.com/TheHPXProject/hpx/issues/542
[6561] https://github.com/TheHPXProject/hpx/issues/541
[6562] https://github.com/TheHPXProject/hpx/issues/539
[6563] https://github.com/TheHPXProject/hpx/issues/535
[6564] https://github.com/TheHPXProject/hpx/issues/530
[6565] https://github.com/TheHPXProject/hpx/issues/523
[6566] https://github.com/TheHPXProject/hpx/issues/521
[6567] https://github.com/TheHPXProject/hpx/issues/520
[6568] https://github.com/TheHPXProject/hpx/issues/519
[6569] https://github.com/TheHPXProject/hpx/issues/517

- Issue #516[6570] - hpx::thread::detach suspends while holding locks, leads to hang in debug
- Issue #514[6571] - Preprocessed headers for <hpx/apply.hpp> don't compile
- Issue #513[6572] - Buildbot configuration problem
- Issue #512[6573] - Implement action based stack size customization
- Issue #511[6574] - Move action priority into a separate type trait
- Issue #510[6575] - trunk broken
- Issue #507[6576] - no matching function for call to `boost::scoped_ptr<hpx::threads::topology>::scoped_ptr(hpx::threads::linux_topology*)`
- Issue #505[6577] - undefined_symbol regression test currently failing
- Issue #502[6578] - Adding OpenCL and OCLM support to HPX for Windows and Linux
- Issue #501[6579] - find_package(HPX) sets cmake output variables
- Issue #500[6580] - wait_any/wait_all are badly named
- Issue #499[6581] - Add support for disabling pbs support in pbs runs
- Issue #498[6582] - Error during no-cache runs
- Issue #496[6583] - Add partial preprocessing support to cmake
- Issue #495[6584] - Support HPX modules exporting startup/shutdown functions only
- Issue #494[6585] - Allow modules to specify when to run startup/shutdown functions
- Issue #493[6586] - Avoid constructing a string in make_success_code
- Issue #492[6587] - Performance counter creation is no longer synchronized at startup
- Issue #491[6588] - Performance counter creation is no longer synchronized at startup
- Issue #490[6589] - Sheneos on_completed_bulk seg fault in distributed
- Issue #489[6590] - compiling issue with g++44
- Issue #488[6591] - Adding OpenCL and OCLM support to HPX for the MSVC platform
- Issue #487[6592] - FindHPX.cmake problems

[6570] https://github.com/TheHPXProject/hpx/issues/516
[6571] https://github.com/TheHPXProject/hpx/issues/514
[6572] https://github.com/TheHPXProject/hpx/issues/513
[6573] https://github.com/TheHPXProject/hpx/issues/512
[6574] https://github.com/TheHPXProject/hpx/issues/511
[6575] https://github.com/TheHPXProject/hpx/issues/510
[6576] https://github.com/TheHPXProject/hpx/issues/507
[6577] https://github.com/TheHPXProject/hpx/issues/505
[6578] https://github.com/TheHPXProject/hpx/issues/502
[6579] https://github.com/TheHPXProject/hpx/issues/501
[6580] https://github.com/TheHPXProject/hpx/issues/500
[6581] https://github.com/TheHPXProject/hpx/issues/499
[6582] https://github.com/TheHPXProject/hpx/issues/498
[6583] https://github.com/TheHPXProject/hpx/issues/496
[6584] https://github.com/TheHPXProject/hpx/issues/495
[6585] https://github.com/TheHPXProject/hpx/issues/494
[6586] https://github.com/TheHPXProject/hpx/issues/493
[6587] https://github.com/TheHPXProject/hpx/issues/492
[6588] https://github.com/TheHPXProject/hpx/issues/491
[6589] https://github.com/TheHPXProject/hpx/issues/490
[6590] https://github.com/TheHPXProject/hpx/issues/489
[6591] https://github.com/TheHPXProject/hpx/issues/488
[6592] https://github.com/TheHPXProject/hpx/issues/487

- Issue #485[6593] - Change distributing_factory and binpacking_factory to use bulk creation
- Issue #484[6594] - Change `HPX_DONT_USE_PREPROCESSED_FILES` to `HPX_USE_PREPROCESSED_FILES`
- Issue #483[6595] - Memory counter for Windows
- Issue #479[6596] - strange errors appear when requesting performance counters on multiple nodes
- Issue #477[6597] - Create (global) timer for multi-threaded measurements
- Issue #472[6598] - Add partial preprocessing using Wave
- Issue #471[6599] - Segfault stack traces don't show up in release
- Issue #468[6600] - External projects need to link with internal components
- Issue #462[6601] - Startup/shutdown functions are called more than once
- Issue #458[6602] - Consolidate hpx::util::high_resolution_timer and `hpx::util::high_resolution_clock`
- Issue #457[6603] - index out of bounds in `allgather_and_gate` on 4 cores or more
- Issue #448[6604] - Make HPX compile with clang
- Issue #447[6605] - 'make tests' should execute tests on local installation
- Issue #446[6606] - Remove SVN-related code from the codebase
- Issue #444[6607] - race condition in smp
- Issue #441[6608] - Patched Boost.Serialization headers should only be installed if needed
- Issue #439[6609] - Components using `HPX_REGISTER_STARTUP_MODULE` fail to compile with MSVC
- Issue #436[6610] - Verify that no locks are being held while threads are suspended
- Issue #435[6611] - Installing HPX should not clobber existing Boost installation
- Issue #434[6612] - Logging external component failed (Boost 1.50)
- Issue #433[6613] - Runtime crash when building all examples

6593 https://github.com/TheHPXProject/hpx/issues/485
6594 https://github.com/TheHPXProject/hpx/issues/484
6595 https://github.com/TheHPXProject/hpx/issues/483
6596 https://github.com/TheHPXProject/hpx/issues/479
6597 https://github.com/TheHPXProject/hpx/issues/477
6598 https://github.com/TheHPXProject/hpx/issues/472
6599 https://github.com/TheHPXProject/hpx/issues/471
6600 https://github.com/TheHPXProject/hpx/issues/468
6601 https://github.com/TheHPXProject/hpx/issues/462
6602 https://github.com/TheHPXProject/hpx/issues/458
6603 https://github.com/TheHPXProject/hpx/issues/457
6604 https://github.com/TheHPXProject/hpx/issues/448
6605 https://github.com/TheHPXProject/hpx/issues/447
6606 https://github.com/TheHPXProject/hpx/issues/446
6607 https://github.com/TheHPXProject/hpx/issues/444
6608 https://github.com/TheHPXProject/hpx/issues/441
6609 https://github.com/TheHPXProject/hpx/issues/439
6610 https://github.com/TheHPXProject/hpx/issues/436
6611 https://github.com/TheHPXProject/hpx/issues/435
6612 https://github.com/TheHPXProject/hpx/issues/434
6613 https://github.com/TheHPXProject/hpx/issues/433

- Issue #432[6614] - Dataflow hangs on 512 cores/64 nodes
- Issue #430[6615] - Problem with distributing factory
- Issue #424[6616] - File paths referring to XSL-files need to be properly escaped
- Issue #417[6617] - Make dataflow LCOs work out of the box by using partial preprocessing
- Issue #413[6618] - hpx_svnversion.py fails on Windows
- Issue #412[6619] - Make hpx::error_code equivalent to hpx::exception
- Issue #398[6620] - HPX clobbers out-of-tree application specific CMake variables (specifically `CMAKE_BUILD_TYPE`)
- Issue #394[6621] - Remove code generating random port numbers for network
- Issue #378[6622] - ShenEOS scaling issues
- Issue #354[6623] - Create a coroutines wrapper for Boost.Context
- Issue #349[6624] - Commandline option `--localities=N/-lN` should be necessary only on AGAS locality
- Issue #334[6625] - Add auto_index support to cmake based documentation toolchain
- Issue #318[6626] - Network benchmarks
- Issue #317[6627] - Implement network performance counters
- Issue #310[6628] - Duplicate logging entries
- Issue #230[6629] - Add compile time option to disable thread debugging info
- Issue #171[6630] - Add an INI option to turn off deadlock detection independently of logging
- Issue #170[6631] - OSHL internal counters are incorrect
- Issue #103[6632] - Better diagnostics for multiple component/action registerations under the same name
- Issue #48[6633] - Support for Darwin (Xcode + Clang)
- Issue #21[6634] - Build fails with GCC 4.6

[6614] https://github.com/TheHPXProject/hpx/issues/432
[6615] https://github.com/TheHPXProject/hpx/issues/430
[6616] https://github.com/TheHPXProject/hpx/issues/424
[6617] https://github.com/TheHPXProject/hpx/issues/417
[6618] https://github.com/TheHPXProject/hpx/issues/413
[6619] https://github.com/TheHPXProject/hpx/issues/412
[6620] https://github.com/TheHPXProject/hpx/issues/398
[6621] https://github.com/TheHPXProject/hpx/issues/394
[6622] https://github.com/TheHPXProject/hpx/issues/378
[6623] https://github.com/TheHPXProject/hpx/issues/354
[6624] https://github.com/TheHPXProject/hpx/issues/349
[6625] https://github.com/TheHPXProject/hpx/issues/334
[6626] https://github.com/TheHPXProject/hpx/issues/318
[6627] https://github.com/TheHPXProject/hpx/issues/317
[6628] https://github.com/TheHPXProject/hpx/issues/310
[6629] https://github.com/TheHPXProject/hpx/issues/230
[6630] https://github.com/TheHPXProject/hpx/issues/171
[6631] https://github.com/TheHPXProject/hpx/issues/170
[6632] https://github.com/TheHPXProject/hpx/issues/103
[6633] https://github.com/TheHPXProject/hpx/issues/48
[6634] https://github.com/TheHPXProject/hpx/issues/21

HPX V0.9.0 (Jul 5, 2012)

We have had roughly 800 commits since the last release and we have closed approximately 80 tickets (bugs, feature requests, etc.).

General changes

- Significant improvements made to the usability of *HPX* in large-scale, distributed environments.
- Renamed `hpx::lcos::packaged_task` to `hpx::lcos::packaged_action` to reflect the semantic differences to a packaged_task as defined by the C++11 Standard[6635].
- *HPX* now exposes *hpx::thread* which is compliant to the C++11 std::thread type except that it (purely locally) represents an *HPX* thread. This new type does not expose any of the remote capabilities of the underlying *HPX*-thread implementation.
- The type `hpx::lcos::future` is now compliant to the C++11 std::future<> type. This type can be used to synchronize both, local and remote operations. In both cases the control flow will 'return' to the future in order to trigger any continuation.
- The types `hpx::lcos::local::promise` and `hpx::lcos::local::packaged_task` are now compliant to the C++11 `std::promise<>` and `std::packaged_task<>` types. These can be used to create a future representing local work only. Use the types `hpx::lcos::promise` and `hpx::lcos::packaged_action` to wrap any (possibly remote) action into a future.
- *hpx::thread* and `hpx::lcos::future` are now cancelable.
- Added support for sequential and logic composition of `hpx::lcos::futures`. The member function `hpx::lcos::future::when` permits futures to be sequentially composed. The helper functions *hpx::wait_all*, *hpx::wait_any*, and `hpx::wait_n` can be used to wait for more than one future at a time.
- *HPX* now exposes *`hpx::apply`* and *hpx::async* as the preferred way of creating (or invoking) any deferred work. These functions are usable with various types of functions, function objects, and actions and provide a uniform way to spawn deferred tasks.
- *HPX* now utilizes `hpx::util::bind` to (partially) bind local functions and function objects, and also actions. Remote bound actions can have placeholders as well.
- *HPX* continuations are now fully polymorphic. The class `hpx::actions::forwarding_continuation` is an example of how the user can write is own types of continuations. It can be used to execute any function as an continuation of a particular action.
- Reworked the action invocation API to be fully conformant to normal functions. Actions can now be invoked using *`hpx::apply`*, *hpx::async*, or using the `operator()` implemented on actions. Actions themselves can now be cheaply instantiated as they do not have any members anymore.
- Reworked the lazy action invocation API. Actions can now be directly bound using `hpx::util::bind` by passing an action instance as the first argument.
- A minimal *HPX* program now looks like this:

[6635] http://www.open-std.org/jtc1/sc22/wg21

```
#include <hpx/hpx_init.hpp>

int hpx_main()
{
    return hpx::finalize();
}

int main()
{
    return hpx::init();
}
```

This removes the immediate dependency on the Boost.Program_options[6636] library.

> **Note**
>
> This minimal version of an *HPX* program does not support any of the default command line arguments (such as –help, or command line options related to PBS). It is suggested to always pass `argc` and `argv` to *HPX* as shown in the example below.

- In order to support those, but still not to depend on Boost.Program_options[6637], the minimal program can be written as:

```
#include <hpx/hpx_init.hpp>

// The arguments
↪for hpx_main can be left off, which very similar to the
// behavior of ``main()`` as defined by C++.
int hpx_main(int argc, char* argv[])
{
    return hpx::finalize();
}

int main(int argc, char* argv[])
{
    return hpx::init(argc, argv);
}
```

- Added performance counters exposing the number of component instances which are alive on a given locality.
- Added performance counters exposing then number of messages sent and received, the number of parcels sent and received, the number of bytes sent and received, the overall time required to send and receive data, and the overall time required to serialize and deserialize the data.
- Added a new component: `hpx::components::binpacking_factory` which is equivalent to the existing `hpx::components::distributing_factory` component, except that it equalizes the overall population of the components to create. It exposes two factory methods, one based on the number of existing instances of the component type to

[6636] https://www.boost.org/doc/html/program_options.html
[6637] https://www.boost.org/doc/html/program_options.html

create, and one based on an arbitrary performance counter which will be queried for all relevant localities.

* Added API functions allowing to access elements of the diagnostic information embedded in the given exception: *hpx::get_locality_id*, `hpx::get_host_name`, `hpx::get_process_id`, `hpx::get_function_name`, `hpx::get_file_name`, `hpx::get_line_number`, `hpx::get_os_thread`, `hpx::get_thread_id`, and `hpx::get_thread_description`.

Bug fixes (closed tickets)

Here is a list of the important tickets we closed for this release:

* Issue #71[6638] - GIDs that are not serialized via `handle_gid<>` should raise an exception
* Issue #105[6639] - Allow for `hpx::util::functions` to be registered in the AGAS symbolic namespace
* Issue #107[6640] - Nasty threadmanger race condition (reproducible in sheneos_test)
* Issue #108[6641] - Add millisecond resolution to *HPX* logs on Linux
* Issue #110[6642] - Shutdown hang in distributed with release build
* Issue #116[6643] - Don't use TSS for the applier and runtime pointers
* Issue #162[6644] - Move local synchronous execution shortcut from hpx::function to the applier
* Issue #172[6645] - Cache sources in CMake and check if they change manually
* Issue #178[6646] - Add an INI option to turn off ranged-based AGAS caching
* Issue #187[6647] - Support for disabling performance counter deployment
* Issue #202[6648] - Support for sending performance counter data to a specific file
* Issue #218[6649] - boost.coroutines allows different stack sizes, but stack pool is unaware of this
* Issue #231[6650] - Implement movable `boost::bind`
* Issue #232[6651] - Implement movable `boost::function`
* Issue #236[6652] - Allow binding `hpx::util::function` to actions
* Issue #239[6653] - Replace `hpx::function` with `hpx::util::function`

6638 https://github.com/TheHPXProject/hpx/issues/71
6639 https://github.com/TheHPXProject/hpx/issues/105
6640 https://github.com/TheHPXProject/hpx/issues/107
6641 https://github.com/TheHPXProject/hpx/issues/108
6642 https://github.com/TheHPXProject/hpx/issues/110
6643 https://github.com/TheHPXProject/hpx/issues/116
6644 https://github.com/TheHPXProject/hpx/issues/162
6645 https://github.com/TheHPXProject/hpx/issues/172
6646 https://github.com/TheHPXProject/hpx/issues/178
6647 https://github.com/TheHPXProject/hpx/issues/187
6648 https://github.com/TheHPXProject/hpx/issues/202
6649 https://github.com/TheHPXProject/hpx/issues/218
6650 https://github.com/TheHPXProject/hpx/issues/231
6651 https://github.com/TheHPXProject/hpx/issues/232
6652 https://github.com/TheHPXProject/hpx/issues/236
6653 https://github.com/TheHPXProject/hpx/issues/239

- Issue #240[6654] - Can't specify RemoteResult with lcos::async
- Issue #242[6655] - REGISTER_TEMPLATE support for plain actions
- Issue #243[6656] - `handle_gid<>` support for `hpx::util::function`
- Issue #245[6657] - `*_c_cache code` throws an exception if the queried GID is not in the local cache
- Issue #246[6658] - Undefined references in dataflow/adaptive1d example
- Issue #252[6659] - Problems configuring sheneos with CMake
- Issue #254[6660] - Lifetime of components doesn't end when client goes out of scope
- Issue #259[6661] - CMake does not detect that MSVC10 has lambdas
- Issue #260[6662] - io_service_pool segfault
- Issue #261[6663] - Late parcel executed outside of pxthread
- Issue #263[6664] - Cannot select allocator with CMake
- Issue #264[6665] - Fix allocator select
- Issue #267[6666] - Runtime error for hello_world
- Issue #269[6667] - pthread_affinity_np test fails to compile
- Issue #270[6668] - Compiler noise due to -Wcast-qual
- Issue #275[6669] - Problem with configuration tests/include paths on Gentoo
- Issue #325[6670] - Sheneos is 200-400 times slower than the fortran equivalent
- Issue #331[6671] - *hpx::init* and `hpx_main()` should not depend on program_options
- Issue #333[6672] - Add doxygen support to CMake for doc toolchain
- Issue #340[6673] - Performance counters for parcels
- Issue #346[6674] - Component loading error when running hello_world in distributed on MSVC2010
- Issue #362[6675] - Missing initializer error

[6654] https://github.com/TheHPXProject/hpx/issues/240
[6655] https://github.com/TheHPXProject/hpx/issues/242
[6656] https://github.com/TheHPXProject/hpx/issues/243
[6657] https://github.com/TheHPXProject/hpx/issues/245
[6658] https://github.com/TheHPXProject/hpx/issues/246
[6659] https://github.com/TheHPXProject/hpx/issues/252
[6660] https://github.com/TheHPXProject/hpx/issues/254
[6661] https://github.com/TheHPXProject/hpx/issues/259
[6662] https://github.com/TheHPXProject/hpx/issues/260
[6663] https://github.com/TheHPXProject/hpx/issues/261
[6664] https://github.com/TheHPXProject/hpx/issues/263
[6665] https://github.com/TheHPXProject/hpx/issues/264
[6666] https://github.com/TheHPXProject/hpx/issues/267
[6667] https://github.com/TheHPXProject/hpx/issues/269
[6668] https://github.com/TheHPXProject/hpx/issues/270
[6669] https://github.com/TheHPXProject/hpx/issues/275
[6670] https://github.com/TheHPXProject/hpx/issues/325
[6671] https://github.com/TheHPXProject/hpx/issues/331
[6672] https://github.com/TheHPXProject/hpx/issues/333
[6673] https://github.com/TheHPXProject/hpx/issues/340
[6674] https://github.com/TheHPXProject/hpx/issues/346
[6675] https://github.com/TheHPXProject/hpx/issues/362

- Issue #363[6676] - Parcel port serialization error
- Issue #366[6677] - Parcel buffering leads to types incompatible exception
- Issue #368[6678] - Scalable alternative to rand() needed for *HPX*
- Issue #369[6679] - IB over IP is substantially slower than just using standard TCP/IP
- Issue #374[6680] - `hpx::lcos::wait` should work with dataflows and arbitrary classes meeting the future interface
- Issue #375[6681] - Conflicting/ambiguous overloads of `hpx::lcos::wait`
- Issue #376[6682] - Find_HPX.cmake should set CMake variable HPX_FOUND for out of tree builds
- Issue #377[6683] - ShenEOS interpolate bulk and interpolate_one_bulk are broken
- Issue #379[6684] - Add support for distributed runs under SLURM
- Issue #382[6685] - _Unwind_Word not declared in boost.backtrace
- Issue #387[6686] - Doxygen should look only at list of specified files
- Issue #388[6687] - Running `make install` on an out-of-tree application is broken
- Issue #391[6688] - Out-of-tree application segfaults when running in qsub
- Issue #392[6689] - Remove HPX_NO_INSTALL option from cmake build system
- Issue #396[6690] - Pragma related warnings when compiling with older gcc versions
- Issue #399[6691] - Out of tree component build problems
- Issue #400[6692] - Out of source builds on Windows: linker should not receive compiler flags
- Issue #401[6693] - Out of source builds on Windows: components need to be linked with hpx_serialization
- Issue #404[6694] - gfortran fails to link automatically when fortran files are present
- Issue #405[6695] - Inability to specify linking order for external libraries
- Issue #406[6696] - Adapt action limits such that dataflow applications work without additional defines

[6676] https://github.com/TheHPXProject/hpx/issues/363
[6677] https://github.com/TheHPXProject/hpx/issues/366
[6678] https://github.com/TheHPXProject/hpx/issues/368
[6679] https://github.com/TheHPXProject/hpx/issues/369
[6680] https://github.com/TheHPXProject/hpx/issues/374
[6681] https://github.com/TheHPXProject/hpx/issues/375
[6682] https://github.com/TheHPXProject/hpx/issues/376
[6683] https://github.com/TheHPXProject/hpx/issues/377
[6684] https://github.com/TheHPXProject/hpx/issues/379
[6685] https://github.com/TheHPXProject/hpx/issues/382
[6686] https://github.com/TheHPXProject/hpx/issues/387
[6687] https://github.com/TheHPXProject/hpx/issues/388
[6688] https://github.com/TheHPXProject/hpx/issues/391
[6689] https://github.com/TheHPXProject/hpx/issues/392
[6690] https://github.com/TheHPXProject/hpx/issues/396
[6691] https://github.com/TheHPXProject/hpx/issues/399
[6692] https://github.com/TheHPXProject/hpx/issues/400
[6693] https://github.com/TheHPXProject/hpx/issues/401
[6694] https://github.com/TheHPXProject/hpx/issues/404
[6695] https://github.com/TheHPXProject/hpx/issues/405
[6696] https://github.com/TheHPXProject/hpx/issues/406

- Issue #415[6697] - `locality_results` is not a member of `hpx::components::server`
- Issue #425[6698] - Breaking changes to `traits::*result` wrt `std::vector<id_type>`
- Issue #426[6699] - AUTOGLOB needs to be updated to support fortran

HPX V0.8.1 (Apr 21, 2012)

This is a point release including important bug fixes for *HPX V0.8.0 (Mar 23, 2012)*.

General changes

- *HPX* does not need to be installed anymore to be functional.

Bug fixes (closed tickets)

Here is a list of the important tickets we closed for this point release:

- Issue #295[6700] - Don't require install path to be known at compile time.
- Issue #371[6701] - Add hpx iostreams to standard build.
- Issue #384[6702] - Fix compilation with GCC 4.7.
- Issue #390[6703] - Remove keep_factory_alive startup call from ShenEOS; add shutdown call to H5close.
- Issue #393[6704] - Thread affinity control is broken.

Bug fixes (commits)

Here is a list of the important commits included in this point release:

- r7642 - External: Fix backtrace memory violation.
- **r7775 - Components: Fix symbol visibility bug with component startup** providers. This prevents one components providers from overriding another components.
- r7778 - Components: Fix startup/shutdown provider shadowing issues.

[6697] https://github.com/TheHPXProject/hpx/issues/415
[6698] https://github.com/TheHPXProject/hpx/issues/425
[6699] https://github.com/TheHPXProject/hpx/issues/426
[6700] https://github.com/TheHPXProject/hpx/issues/295
[6701] https://github.com/TheHPXProject/hpx/issues/371
[6702] https://github.com/TheHPXProject/hpx/issues/384
[6703] https://github.com/TheHPXProject/hpx/issues/390
[6704] https://github.com/TheHPXProject/hpx/issues/393

HPX V0.8.0 (Mar 23, 2012)

We have had roughly 1000 commits since the last release and we have closed approximately 70 tickets (bugs, feature requests, etc.).

General changes

- Improved PBS support, allowing for arbitrary naming schemes of node-hostnames.
- Finished verification of the reference counting framework.
- Implemented decrement merging logic to optimize the distributed reference counting system.
- Restructured the LCO framework. Renamed `hpx::lcos::eager_future<>` and `hpx::lcos::lazy_future<>` into `hpx::lcos::packaged_task` and `hpx::lcos::deferred_packaged_task`. Split `hpx::lcos::promise` into `hpx::lcos::packaged_task` and `hpx::lcos::future`. Added 'local' futures (in namespace `hpx::lcos::local`).
- Improved the general performance of local and remote action invocations. This (under certain circumstances) drastically reduces the number of copies created for each of the parameters and return values.
- Reworked the performance counter framework. Performance counters are now created only when needed, which reduces the overall resource requirements. The new framework allows for much more flexible creation and management of performance counters. The new sine example application demonstrates some of the capabilities of the new infrastructure.
- Added a buildbot-based continuous build system which gives instant, automated feedback on each commit to SVN.
- Added more automated tests to verify proper functioning of *HPX*.
- Started to create documentation for *HPX* and its API.
- Added documentation toolchain to the build system.
- Added dataflow LCO.
- Changed default *HPX* command line options to have `hpx:` prefix. For instance, the former option `--threads` is now `--hpx:threads`. This has been done to make ambiguities with possible application specific command line options as unlikely as possible. See the section *HPX Command Line Options* for a full list of available options.
- Added the possibility to define command line aliases. The former short (one-letter) command line options have been predefined as aliases for backwards compatibility. See the section *HPX Command Line Options* for a detailed description of command line option aliasing.
- Network connections are now cached based on the connected host. The number of simultaneous connections to a particular host is now limited. Parcels are buffered and bundled if all connections are in use.
- Added more refined thread affinity control. This is based on the external library Portable Hardware Locality (HWLOC).
- Improved support for Windows builds with CMake.
- Added support for components to register their own command line options.

- Added the possibility to register custom startup/shutdown functions for any component. These functions are guaranteed to be executed by an *HPX* thread.
- Added two new experimental thread schedulers: hierarchy_scheduler and periodic_priority_scheduler. These can be activated by using the command line options `--hpx:queuing=hierarchy` or `--hpx:queuing=periodic`.

Example applications

- Graph500 performance benchmark[6705] (thanks to Matthew Anderson for contributing this application).
- GTC (Gyrokinetic Toroidal Code)[6706]: a skeleton for particle in cell type codes.
- Random Memory Access: an example demonstrating random memory accesses in a large array
- ShenEOS example[6707], demonstrating partitioning of large read-only data structures and exposing an interpolation API.
- Sine performance counter demo.
- Accumulator examples demonstrating how to write and use *HPX* components.
- Quickstart examples (like hello_world, fibonacci, quicksort, factorial, etc.) demonstrating simple *HPX* concepts which introduce some of the concepts in *HPX*.
- Load balancing and work stealing demos.

API changes

- Moved all local LCOs into a separate namespace `hpx::lcos::local` (for instance, `hpx::lcos::local_mutex` is now `hpx::lcos::local::mutex`).
- Replaced `hpx::actions::function` with `hpx::util::function`. Cleaned up related code.
- Removed `hpx::traits::handle_gid` and moved handling of global reference counts into the corresponding serialization code.
- Changed terminology: `prefix` is now called `locality_id`, renamed the corresponding API functions (such as `hpx::get_prefix`, which is now called `hpx::get_locality_id`).
- Adding *hpx::find_remote_localities*, and *hpx::get_num_localities*.
- Changed performance counter naming scheme to make it more bash friendly. The new performance counter naming scheme is now

```
/object{parentname#parentindex/instance#index}/counter#parameters
```

- Added `hpx::get_worker_thread_num` replacing `hpx::threadmanager_base::get_thread_num`.
- Renamed `hpx::get_num_os_threads` to `hpx::get_os_threads_count`.
- Added `hpx::threads::get_thread_count`.

[6705] http://www.graph500.org/
[6706] http://www.nersc.gov/research-and-development/benchmarking-and-workload-characterization/nersc-6-benchmarks/gtc/
[6707] http://stellarcollapse.org/equationofstate

- Restructured the Futures sub-system, renaming types in accordance with the terminology used by the C++11 ISO standard.

Bug fixes (closed tickets)

Here is a list of the important tickets we closed for this release:

- Issue #31[6708] - Specialize handle_gid<> for examples and tests
- Issue #72[6709] - Fix AGAS reference counting
- Issue #104[6710] - heartbeat throws an exception when decrefing the performance counter it's watching
- Issue #111[6711] - throttle causes an exception on the target application
- Issue #142[6712] - One failed component loading causes an unrelated component to fail
- Issue #165[6713] - Remote exception propagation bug in AGAS reference counting test
- Issue #186[6714] - Test credit exhaustion/splitting (e.g. prepare_gid and symbol NS)
- Issue #188[6715] - Implement remaining AGAS reference counting test cases
- Issue #258[6716] - No type checking of GIDs in stubs classes
- Issue #271[6717] - Seg fault/shared pointer assertion in distributed code
- Issue #281[6718] - CMake options need descriptive text
- Issue #283[6719] - AGAS caching broken (gva_cache needs to be rewritten with ICL)
- Issue #285[6720] - HPX_INSTALL root directory not the same as CMAKE_INSTALL_PREFIX
- Issue #286[6721] - New segfault in dataflow applications
- Issue #289[6722] - Exceptions should only be logged if not handled
- Issue #290[6723] - c++11 tests failure
- Issue #293[6724] - Build target for component libraries
- Issue #296[6725] - Compilation error with Boost V1.49rc1
- Issue #298[6726] - Illegal instructions on termination

[6708] https://github.com/TheHPXProject/hpx/issues/31
[6709] https://github.com/TheHPXProject/hpx/issues/72
[6710] https://github.com/TheHPXProject/hpx/issues/104
[6711] https://github.com/TheHPXProject/hpx/issues/111
[6712] https://github.com/TheHPXProject/hpx/issues/142
[6713] https://github.com/TheHPXProject/hpx/issues/165
[6714] https://github.com/TheHPXProject/hpx/issues/186
[6715] https://github.com/TheHPXProject/hpx/issues/188
[6716] https://github.com/TheHPXProject/hpx/issues/258
[6717] https://github.com/TheHPXProject/hpx/issues/271
[6718] https://github.com/TheHPXProject/hpx/issues/281
[6719] https://github.com/TheHPXProject/hpx/issues/283
[6720] https://github.com/TheHPXProject/hpx/issues/285
[6721] https://github.com/TheHPXProject/hpx/issues/286
[6722] https://github.com/TheHPXProject/hpx/issues/289
[6723] https://github.com/TheHPXProject/hpx/issues/290
[6724] https://github.com/TheHPXProject/hpx/issues/293
[6725] https://github.com/TheHPXProject/hpx/issues/296
[6726] https://github.com/TheHPXProject/hpx/issues/298

- Issue #299[6727] - gravity aborts with multiple threads
- Issue #301[6728] - Build error with Boost trunk
- Issue #303[6729] - Logging assertion failure in distributed runs
- Issue #304[6730] - Exception 'what' strings are lost when exceptions from decode_parcel are reported
- Issue #306[6731] - Performance counter user interface issues
- Issue #307[6732] - Logging exception in distributed runs
- Issue #308[6733] - Logging deadlocks in distributed
- Issue #309[6734] - Reference counting test failures and exceptions
- Issue #311[6735] - Merge AGAS remote_interface with the runtime_support object
- Issue #314[6736] - Object tracking for id_types
- Issue #315[6737] - Remove handle_gid and handle credit splitting in id_type serialization
- Issue #320[6738] - applier::get_locality_id() should return an error value (or throw an exception)
- Issue #321[6739] - Optimization for id_types which are never split should be restored
- Issue #322[6740] - Command line processing ignored with Boost 1.47.0
- Issue #323[6741] - Credit exhaustion causes object to stay alive
- Issue #324[6742] - Duplicate exception messages
- Issue #326[6743] - Integrate Quickbook with CMake
- Issue #329[6744] - –help and –version should still work
- Issue #330[6745] - Create pkg-config files
- Issue #337[6746] - Improve usability of performance counter timestamps
- Issue #338[6747] - Non-std exceptions deriving from std::exceptions in tfunc may be sliced
- Issue #339[6748] - Decrease the number of send_pending_parcels threads

[6727] https://github.com/TheHPXProject/hpx/issues/299
[6728] https://github.com/TheHPXProject/hpx/issues/301
[6729] https://github.com/TheHPXProject/hpx/issues/303
[6730] https://github.com/TheHPXProject/hpx/issues/304
[6731] https://github.com/TheHPXProject/hpx/issues/306
[6732] https://github.com/TheHPXProject/hpx/issues/307
[6733] https://github.com/TheHPXProject/hpx/issues/308
[6734] https://github.com/TheHPXProject/hpx/issues/309
[6735] https://github.com/TheHPXProject/hpx/issues/311
[6736] https://github.com/TheHPXProject/hpx/issues/314
[6737] https://github.com/TheHPXProject/hpx/issues/315
[6738] https://github.com/TheHPXProject/hpx/issues/320
[6739] https://github.com/TheHPXProject/hpx/issues/321
[6740] https://github.com/TheHPXProject/hpx/issues/322
[6741] https://github.com/TheHPXProject/hpx/issues/323
[6742] https://github.com/TheHPXProject/hpx/issues/324
[6743] https://github.com/TheHPXProject/hpx/issues/326
[6744] https://github.com/TheHPXProject/hpx/issues/329
[6745] https://github.com/TheHPXProject/hpx/issues/330
[6746] https://github.com/TheHPXProject/hpx/issues/337
[6747] https://github.com/TheHPXProject/hpx/issues/338
[6748] https://github.com/TheHPXProject/hpx/issues/339

- Issue #343[6749] - Dynamically setting the stack size doesn't work
- Issue #351[6750] - 'make install' does not update documents
- Issue #353[6751] - Disable FIXMEs in the docs by default; add a doc developer CMake option to enable FIXMEs
- Issue #355[6752] - 'make' doesn't do anything after correct configuration
- Issue #356[6753] - Don't use `hpx::util::static_` in topology code
- Issue #359[6754] - Infinite recursion in hpx::tuple serialization
- Issue #361[6755] - Add compile time option to disable logging completely
- Issue #364[6756] - Installation seriously broken in r7443

HPX V0.7.0 (Dec 12, 2011)

We have had roughly 1000 commits since the last release and we have closed approximately 120 tickets (bugs, feature requests, etc.).

General changes

- Completely removed code related to deprecated AGAS V1, started to work on AGAS V2.1.
- Started to clean up and streamline the exposed APIs (see 'API changes' below for more details).
- Revamped and unified performance counter framework, added a lot of new performance counter instances for monitoring of a diverse set of internal *HPX* parameters (queue lengths, access statistics, etc.).
- Improved general error handling and logging support.
- Fixed several race conditions, improved overall stability, decreased memory footprint, improved overall performance (major optimizations include native TLS support and ranged-based AGAS caching).
- Added support for running *HPX* applications with PBS.
- Many updates to the build system, added support for gcc 4.5.x and 4.6.x, added C++11 support.
- Many updates to default command line options.
- Added many tests, set up buildbot for continuous integration testing.
- Better shutdown handling of distributed applications.

[6749] https://github.com/TheHPXProject/hpx/issues/343
[6750] https://github.com/TheHPXProject/hpx/issues/351
[6751] https://github.com/TheHPXProject/hpx/issues/353
[6752] https://github.com/TheHPXProject/hpx/issues/355
[6753] https://github.com/TheHPXProject/hpx/issues/356
[6754] https://github.com/TheHPXProject/hpx/issues/359
[6755] https://github.com/TheHPXProject/hpx/issues/361
[6756] https://github.com/TheHPXProject/hpx/issues/364

Example applications

- quickstart/factorial and quickstart/fibonacci, future-recursive parallel algorithms.
- quickstart/hello_world, distributed hello world example.
- quickstart/rma, simple remote memory access example
- quickstart/quicksort, parallel quicksort implementation.
- gtc, gyrokinetic torodial code.
- bfs, breadth-first-search, example code for a graph application.
- sheneos, partitioning of large data sets.
- accumulator, simple component example.
- balancing/os_thread_num, balancing/px_thread_phase, examples demonstrating load balancing and work stealing.

API changes

- Added `hpx::find_all_localities`.
- Added `hpx::terminate` for non-graceful termination of applications.
- Added `hpx::lcos::async` functions for simpler asynchronous programming.
- Added new AGAS interface for handling of symbolic namespace (`hpx::agas::*`).
- Renamed `hpx::components::wait` to `hpx::lcos::wait`.
- Renamed `hpx::lcos::future_value` to `hpx::lcos::promise`.
- Renamed `hpx::lcos::recursive_mutex` to `hpx::lcos::local_recursive_mutex`, `hpx::lcos::mutex` to `hpx::lcos::local_mutex`
- Removed support for Boost versions older than V1.38, recommended Boost version is now V1.47 and newer.
- Removed `hpx::process` (this will be replaced by a real process implementation in the future).
- Removed non-functional LCO code (`hpx::lcos::dataflow`, `hpx::lcos::thunk`, `hpx::lcos::dataflow_variable`).
- Removed deprecated `hpx::naming::full_address`.

Bug fixes (closed tickets)

Here is a list of the important tickets we closed for this release:

- Issue #28[6757] - Integrate Windows/Linux CMake code for *HPX* core
- Issue #32[6758] - hpx::cout() should be hpx::cout
- Issue #33[6759] - AGAS V2 legacy client does not properly handle error_code

[6757] https://github.com/TheHPXProject/hpx/issues/28
[6758] https://github.com/TheHPXProject/hpx/issues/32
[6759] https://github.com/TheHPXProject/hpx/issues/33

- Issue #60[6760] - AGAS: allow for registerid to optionally take ownership of the gid
- Issue #62[6761] - adaptive1d compilation failure in Fusion
- Issue #64[6762] - Parcel subsystem doesn't resolve domain names
- Issue #83[6763] - No error handling if no console is available
- Issue #84[6764] - No error handling if a hosted locality is treated as the bootstrap server
- Issue #90[6765] - Add general commandline option -N
- Issue #91[6766] - Add possibility to read command line arguments from file
- Issue #92[6767] - Always log exceptions/errors to the log file
- Issue #93[6768] - Log the command line/program name
- Issue #95[6769] - Support for distributed launches
- Issue #97[6770] - Attempt to create a bad component type in AMR examples
- Issue #100[6771] - factorial and factorial_get examples trigger AGAS component type assertions
- Issue #101[6772] - Segfault when hpx::process::here() is called in fibonacci2
- Issue #102[6773] - unknown_component_address in int_object_semaphore_client
- Issue #114[6774] - marduk raises assertion with default parameters
- Issue #115[6775] - Logging messages for SMP runs (on the console) shouldn't be buffered
- Issue #119[6776] - marduk linking strategy breaks other applications
- Issue #121[6777] - pbsdsh problem
- Issue #123[6778] - marduk, dataflow and adaptive1d fail to build
- Issue #124[6779] - Lower default preprocessing arity
- Issue #125[6780] - Move hpx::detail::diagnostic_information out of the detail namespace
- Issue #126[6781] - Test definitions for AGAS reference counting
- Issue #128[6782] - Add averaging performance counter

6760 https://github.com/TheHPXProject/hpx/issues/60
6761 https://github.com/TheHPXProject/hpx/issues/62
6762 https://github.com/TheHPXProject/hpx/issues/64
6763 https://github.com/TheHPXProject/hpx/issues/83
6764 https://github.com/TheHPXProject/hpx/issues/84
6765 https://github.com/TheHPXProject/hpx/issues/90
6766 https://github.com/TheHPXProject/hpx/issues/91
6767 https://github.com/TheHPXProject/hpx/issues/92
6768 https://github.com/TheHPXProject/hpx/issues/93
6769 https://github.com/TheHPXProject/hpx/issues/95
6770 https://github.com/TheHPXProject/hpx/issues/97
6771 https://github.com/TheHPXProject/hpx/issues/100
6772 https://github.com/TheHPXProject/hpx/issues/101
6773 https://github.com/TheHPXProject/hpx/issues/102
6774 https://github.com/TheHPXProject/hpx/issues/114
6775 https://github.com/TheHPXProject/hpx/issues/115
6776 https://github.com/TheHPXProject/hpx/issues/119
6777 https://github.com/TheHPXProject/hpx/issues/121
6778 https://github.com/TheHPXProject/hpx/issues/123
6779 https://github.com/TheHPXProject/hpx/issues/124
6780 https://github.com/TheHPXProject/hpx/issues/125
6781 https://github.com/TheHPXProject/hpx/issues/126
6782 https://github.com/TheHPXProject/hpx/issues/128

- Issue #129[6783] - Error with endian.hpp while building adaptive1d
- Issue #130[6784] - Bad initialization of performance counters
- Issue #131[6785] - Add global startup/shutdown functions to component modules
- Issue #132[6786] - Avoid using auto_ptr
- Issue #133[6787] - On Windows hpx.dll doesn't get installed
- Issue #134[6788] - HPX_LIBRARY does not reflect real library name (on Windows)
- Issue #135[6789] - Add detection of unique_ptr to build system
- Issue #137[6790] - Add command line option allowing to repeatedly evaluate performance counters
- Issue #139[6791] - Logging is broken
- Issue #140[6792] - CMake problem on windows
- Issue #141[6793] - Move all non-component libraries into $PREFIX/lib/hpx
- Issue #143[6794] - adaptive1d throws an exception with the default command line options
- Issue #146[6795] - Early exception handling is broken
- Issue #147[6796] - Sheneos doesn't link on Linux
- Issue #149[6797] - sheneos_test hangs
- Issue #154[6798] - Compilation fails for r5661
- Issue #155[6799] - Sine performance counters example chokes on chrono headers
- Issue #156[6800] - Add build type to –version
- Issue #157[6801] - Extend AGAS caching to store gid ranges
- Issue #158[6802] - r5691 doesn't compile
- Issue #160[6803] - Re-add AGAS function for resolving a locality to its prefix
- Issue #168[6804] - Managed components should be able to access their own GID
- Issue #169[6805] - Rewrite AGAS future pool

6783 https://github.com/TheHPXProject/hpx/issues/129
6784 https://github.com/TheHPXProject/hpx/issues/130
6785 https://github.com/TheHPXProject/hpx/issues/131
6786 https://github.com/TheHPXProject/hpx/issues/132
6787 https://github.com/TheHPXProject/hpx/issues/133
6788 https://github.com/TheHPXProject/hpx/issues/134
6789 https://github.com/TheHPXProject/hpx/issues/135
6790 https://github.com/TheHPXProject/hpx/issues/137
6791 https://github.com/TheHPXProject/hpx/issues/139
6792 https://github.com/TheHPXProject/hpx/issues/140
6793 https://github.com/TheHPXProject/hpx/issues/141
6794 https://github.com/TheHPXProject/hpx/issues/143
6795 https://github.com/TheHPXProject/hpx/issues/146
6796 https://github.com/TheHPXProject/hpx/issues/147
6797 https://github.com/TheHPXProject/hpx/issues/149
6798 https://github.com/TheHPXProject/hpx/issues/154
6799 https://github.com/TheHPXProject/hpx/issues/155
6800 https://github.com/TheHPXProject/hpx/issues/156
6801 https://github.com/TheHPXProject/hpx/issues/157
6802 https://github.com/TheHPXProject/hpx/issues/158
6803 https://github.com/TheHPXProject/hpx/issues/160
6804 https://github.com/TheHPXProject/hpx/issues/168
6805 https://github.com/TheHPXProject/hpx/issues/169

- Issue #179[6806] - Complete switch to request class for AGAS server interface
- Issue #182[6807] - Sine performance counter is loaded by other examples
- Issue #185[6808] - Write tests for symbol namespace reference counting
- Issue #191[6809] - Assignment of read-only variable in point_geometry
- Issue #200[6810] - Seg faults when querying performance counters
- Issue #204[6811] - –ifnames and suffix stripping needs to be more generic
- Issue #205[6812] - –list-* and –print-counter-* options do not work together and produce no warning
- Issue #207[6813] - Implement decrement entry merging
- Issue #208[6814] - Replace the spinlocks in AGAS with hpx::lcos::local_mutexes
- Issue #210[6815] - Add an –ifprefix option
- Issue #214[6816] - Performance test for PX-thread creation
- Issue #216[6817] - VS2010 compilation
- Issue #222[6818] - r6045 context_linux_x86.hpp
- Issue #223[6819] - fibonacci hangs when changing the state of an active thread
- Issue #225[6820] - Active threads end up in the FEB wait queue
- Issue #226[6821] - VS Build Error for Accumulator Client
- Issue #228[6822] - Move all traits into namespace hpx::traits
- Issue #229[6823] - Invalid initialization of reference in thread_init_data
- Issue #235[6824] - Invalid GID in iostreams
- Issue #238[6825] - Demangle type names for the default implementation of get_action_name
- Issue #241[6826] - C++11 support breaks GCC 4.5
- Issue #247[6827] - Reference to temporary with GCC 4.4

[6806] https://github.com/TheHPXProject/hpx/issues/179
[6807] https://github.com/TheHPXProject/hpx/issues/182
[6808] https://github.com/TheHPXProject/hpx/issues/185
[6809] https://github.com/TheHPXProject/hpx/issues/191
[6810] https://github.com/TheHPXProject/hpx/issues/200
[6811] https://github.com/TheHPXProject/hpx/issues/204
[6812] https://github.com/TheHPXProject/hpx/issues/205
[6813] https://github.com/TheHPXProject/hpx/issues/207
[6814] https://github.com/TheHPXProject/hpx/issues/208
[6815] https://github.com/TheHPXProject/hpx/issues/210
[6816] https://github.com/TheHPXProject/hpx/issues/214
[6817] https://github.com/TheHPXProject/hpx/issues/216
[6818] https://github.com/TheHPXProject/hpx/issues/222
[6819] https://github.com/TheHPXProject/hpx/issues/223
[6820] https://github.com/TheHPXProject/hpx/issues/225
[6821] https://github.com/TheHPXProject/hpx/issues/226
[6822] https://github.com/TheHPXProject/hpx/issues/228
[6823] https://github.com/TheHPXProject/hpx/issues/229
[6824] https://github.com/TheHPXProject/hpx/issues/235
[6825] https://github.com/TheHPXProject/hpx/issues/238
[6826] https://github.com/TheHPXProject/hpx/issues/241
[6827] https://github.com/TheHPXProject/hpx/issues/247

- Issue #248[6828] - Seg fault at shutdown with GCC 4.4
- Issue #253[6829] - Default component action registration kills compiler
- Issue #272[6830] - G++ unrecognized command line option
- Issue #273[6831] - quicksort example doesn't compile
- Issue #277[6832] - Invalid CMake logic for Windows

2.11.3 Namespace changes

HPX V1.9.0 Namespace changes

The latest release includes amongst others changes in the namespaces so that *HPX* facilities correspond to the C++ Standard Library. The old namespaces are deprecated. Below is a comprehensive list of the namespace changes.

Table 2.194: Namespace changes in V1.9.0

Old namespace	New namespace
`hpx::util::mem_fn`	*hpx::mem_fn*
`hpx::util::invoke`	*hpx::invoke*
`hpx::util::invoke_r`	`hpx::invoke_r`
`hpx::util::invoke_fused`	*hpx::invoke_fused*
`hpx::util::invoke_fused_r`	*hpx::invoke_fused_r*
`hpx::util::unlock_guard`	*hpx::unlock_guard*
`hpx::parallel::v1::reduce_by_key`	`hpx::experimental::reduce_by_key`
`hpx::parallel::v1::sort_by_key`	*hpx::experimental::sort_by_key*
`hpx::parallel::task_canceled_exception`	*hpx::experimental::task_canceled_exception*
`hpx::parallel::task_block`	*hpx::experimental::task_block*
`hpx::parallel::define_task_block`	*hpx::experimental::define_task_block* |
`hpx::parallel::define_task_block_restore_threac`	*hpx::experimental::define_task_block_restore_thread*
`hpx::execution::experimental::task_group`	*hpx::experimental::task_group*

2.12 Citing *HPX*

Please cite *HPX* whenever you use it for publications. Use our paper in The Journal of Open Source Software as the main citation for *HPX*: [6833]. Use the Zenodo entry for referring to the latest version of *HPX*: [6834]. Entries for citing specific versions of *HPX* can also be found at [6835].

[6828] https://github.com/TheHPXProject/hpx/issues/248
[6829] https://github.com/TheHPXProject/hpx/issues/253
[6830] https://github.com/TheHPXProject/hpx/issues/272
[6831] https://github.com/TheHPXProject/hpx/issues/273
[6832] https://github.com/TheHPXProject/hpx/issues/277
[6833] https://joss.theoj.org/papers/022e5917b95517dff20cd3742ab95eca
[6834] https://doi.org/10.5281/zenodo.598202
[6835] https://doi.org/10.5281/zenodo.598202

2.13 *HPX* users

A list of institutions and projects using *HPX* can be found on the *HPX* Users[6836] page.

2.14 About *HPX*

2.14.1 History

The development of High Performance ParalleX (*HPX*) began in 2007. At that time, Hartmut Kaiser became interested in the work done by the ParalleX group at the Center for Computation and Technology (CCT)[6837], a multi-disciplinary research institute at Louisiana State University (LSU)[6838]. The ParalleX group was working to develop a new and experimental execution model for future high performance computing architectures. This model was christened ParalleX. The first implementations of ParalleX were crude, and many of those designs had to be discarded entirely. However, over time the team learned quite a bit about how to design a parallel, distributed runtime system which implements the concepts of ParalleX.

From the very beginning, this endeavour has been a group effort. In addition to a handful of interested researchers, there have always been graduate and undergraduate students participating in the discussions, design, and implementation of *HPX*. In 2011 we decided to formalize our collective research efforts by creating the STE||AR[6839] group (**S**ystems **T**echnology, **E**mergent Para**ll**elism, and **A**lgorithm **R**esearch). Over time, the team grew to include researchers around the country and the world. In 2014, the STE||AR[6840] Group was reorganized to become the international community it is today. This consortium of researchers aims to develop stable, sustainable, and scalable tools which will enable application developers to exploit the parallelism latent in the machines of today and tomorrow. Our goal of the *HPX* project is to create a high quality, freely available, open source implementation of ParalleX concepts for conventional and future systems by building a modular and standards conforming runtime system for SMP and distributed application environments. The API exposed by *HPX* is conformant to the interfaces defined by the C++ ISO Standard and adheres to the programming guidelines used by the Boost[6841] collection of C++ libraries. We steer the development of *HPX* with real world applications and aim to provide a smooth migration path for domain scientists.

To learn more about STE||AR[6842] and ParalleX, see *People* and *Why HPX?*.

[6836] https://hpx.dev/hpx-users/
[6837] https://www.cct.lsu.edu
[6838] https://www.lsu.edu
[6839] https://stellar-group.org
[6840] https://stellar-group.org
[6841] https://www.boost.org/
[6842] https://stellar-group.org

2.14.2 People

The STE||AR[6843] Group (pronounced as stellar) stands for "**S**ystems **T**echnology, **E**mergent Para**ll**elism, and **A**lgorithm **R**esearch". We are an international group of faculty, researchers, and students working at various institutions around the world. The goal of the STE||AR[6844] Group is to promote the development of scalable parallel applications by providing a community for ideas, a framework for collaboration, and a platform for communicating these concepts to the broader community.

Our work is focused on building technologies for scalable parallel applications. *HPX*, our general purpose C++ runtime system for parallel and distributed applications, is no exception. We use *HPX* for a broad range of scientific applications, helping scientists and developers to write code which scales better and shows better performance compared to more conventional programming models such as MPI.

HPX is based on *ParalleX* which is a new (and still experimental) parallel execution model aiming to overcome the limitations imposed by the current hardware and the techniques we use to write applications today. Our group focuses on two types of applications - those requiring excellent strong scaling, allowing for a dramatic reduction of execution time for fixed workloads and those needing highest level of sustained performance through massive parallelism. These applications are presently unable (through conventional practices) to effectively exploit a relatively small number of cores in a multi-core system. By extension, these application will not be able to exploit high-end exascale computing systems which are likely to employ hundreds of millions of such cores by the end of this decade.

Critical bottlenecks to the effective use of new generation high performance computing (HPC) systems include:

- *Starvation*: due to lack of usable application parallelism and means of managing it,
- *Overhead*: reduction to permit strong scalability, improve efficiency, and enable dynamic resource management,
- *Latency*: from remote access across system or to local memories,
- *Contention*: due to multicore chip I/O pins, memory banks, and system interconnects.

The ParalleX model has been devised to address these challenges by enabling a new computing dynamic through the application of message-driven computation in a global address space context with lightweight synchronization. The work on *HPX* is centered around implementing the concepts as defined by the ParalleX model. *HPX* is currently targeted at conventional machines, such as classical Linux based Beowulf clusters and SMP nodes.

We fully understand that the success of *HPX* (and ParalleX) is very much the result of the work of many people. To see a list of who is contributing see our tables below.

[6843] https://stellar-group.org
[6844] https://stellar-group.org

HPX contributors

Table 2.195: Contributors

Name	Institution	Email
Hartmut Kaiser	Center for Computation and Technology (CCT)[6845], Louisiana State University (LSU)[6846]	hkaiser@cct.lsu.edu
Thomas Heller	Department of Computer Science 3 - Computer Architecture[6847], Friedrich-Alexander University Erlangen-Nuremberg (FAU)[6848]	thom.heller@gmail.com
Agustin Berge		agustinberge@gmail.com
Mikael Simberg	Swiss National Supercomputing Centre[6849]	simbergm@cscs.ch
John Biddiscombe	Swiss National Supercomputing Centre[6850]	biddisco@cscs.ch
Anton Bikineev	Center for Computation and Technology (CCT)[6851], Louisiana State University (LSU)[6852]	ant.bikineev@gmail.com
Martin Stumpf	Department of Computer Science 3 - Computer Architecture[6853], Friedrich-Alexander University Erlangen-Nuremberg (FAU)[6854]	martin.h.stumpf@gmail.com
Bryce Adelstein Lelbach		brycelelbach@gmail.com
Shuangyang Yang	Center for Computation and Technology (CCT)[6855], Louisiana State University (LSU)[6856]	syang16@cct.lsu.edu
Jeroen Habraken		vexocide@gmail.com
Steven Brandt	Center for Computation and Technology (CCT)[6857], Louisiana State University (LSU)[6858]	sbrandt@cct.lsu.edu
Antoine Tran Tan	Paris-Saclay University[6859],	antoine.trantan@universite-paris-saclay.fr
Adrian S. Lemoine	AMD[6860]	Adrian.Lemoine@amd.com
Maciej Brodowicz		maciekab@gmail.com
Giannis Gonidelis	Center for Computation and Technology (CCT)[6861], Louisiana State University (LSU)[6862]	gonidelis@hotmail.com

Contributors to this document

Table 2.196: Documentation authors

Name	Institution	Email
Hartmut Kaiser	Center for Computation and Technology (CCT)[6863], Louisiana State University (LSU)[6864]	hkaiser@cct.lsu.edu
Thomas Heller	Department of Computer Science 3 - Computer Architecture[6865], Friedrich-Alexander University Erlangen-Nuremberg (FAU)[6866]	thom.heller@gmail.com
Bryce Adelstein Lelbach		brycelelbach@gmail.com
Vinay C Amatya	Center for Computation and Technology (CCT)[6867], Louisiana State University (LSU)[6868]	vamatya@cct.lsu.edu
Steven Brandt	Center for Computation and Technology (CCT)[6869], Louisiana State University (LSU)[6870]	sbrandt@cct.lsu.edu
Maciej Brodowicz		maciekab@gmail.com
Adrian S. Lemoine	AMD[6871]	Adrian.Lemoine@amd.com
Rebecca Stobaugh		rstobaugh1@gmail.com
Dimitra Karatza	Faculty of Electrical Engineering, Mathematics & Computer Science[6872], Delft University of Technology[6873]	dimitra.karatza11@gmail.com
Bhumit Attarde		bhumitattarde2@gmail.com

[6845] https://www.cct.lsu.edu
[6846] https://www.lsu.edu
[6847] https://www3.cs.fau.de
[6848] https://www.fau.de
[6849] https://www.cscs.ch
[6850] https://www.cscs.ch
[6851] https://www.cct.lsu.edu
[6852] https://www.lsu.edu
[6853] https://www3.cs.fau.de
[6854] https://www.fau.de
[6855] https://www.cct.lsu.edu
[6856] https://www.lsu.edu
[6857] https://www.cct.lsu.edu
[6858] https://www.lsu.edu
[6859] https://www.universite-paris-saclay.fr/en
[6860] https://www.amd.com/en
[6861] https://www.cct.lsu.edu
[6862] https://www.lsu.edu

Acknowledgements

Thanks also to the following people who contributed directly or indirectly to the project through discussions, pull requests, documentation patches, etc.

- Panos Syskakis for benchmarking and optimizing our parallel algorithms.
- Shreyas Atre, for contributing fixes to our implementation of senders/receivers and extending our coroutines integration with senders/receivers.
- Alexander Neumann, for contributing fixes to the cmake build system.
- Dimitra Karatza, for her work on refactoring the documentation and providing a new user-friendly environment during and after Google Season of Docs 2021.
- Srinivas Yadav, for his work on SIMD support in algorithms before and during Google Summer of Code 2021.
- Akhil Nair, for his work on adapting algorithms to C++20 before and during Google Summer of Code 2021.
- Alexander Toktarev, for updating the parallel algorithm customization points to use `tag_fallback_invoke` for the default implementations.
- Brice Goglin, for reporting and helping fix issues related to the integration of hwloc in *HPX*.
- Giannis Gonidelis, for his work on the ranges adaptation during the Google Summer of Code 2020.
- Auriane Reverdell (Swiss National Supercomputing Centre[6874]), for her tireless work on refactoring our CMake setup and modularizing *HPX*.
- Christopher Hinz, for his work on refactoring our CMake setup.
- Weile Wei, for fixing *HPX* builds with CUDA on Summit.
- Severin Strobl, for fixing our CMake setup related to linking and adding new entry points to the *HPX* runtime.
- Rebecca Stobaugh, for her major documentation review and contributions during and after the 2019 Google Season of Documentation.
- Jan Melech, for adding automatic serialization of simple structs.
- Austin McCartney, for adding concept emulation of the Ranges TS bidirectional and random access iterator concepts.
- Marco Diers, reporting and fixing issues related PMIx.
- Maximilian Bremer, for reporting multiple issues and extending the component migration tests.

[6863] https://www.cct.lsu.edu
[6864] https://www.lsu.edu
[6865] https://www3.cs.fau.de
[6866] https://www.fau.de
[6867] https://www.cct.lsu.edu
[6868] https://www.lsu.edu
[6869] https://www.cct.lsu.edu
[6870] https://www.lsu.edu
[6871] https://www.amd.com/en
[6872] https://www.tudelft.nl/en/eemcs
[6873] https://www.tudelft.nl/en/
[6874] https://www.cscs.ch

- Piotr Mikolajczyk, for his improvements and fixes to the set and count algorithms.
- Grant Rostig, for reporting several deficiencies on our web pages.
- Jakub Golinowski, for implementing an *HPX* backend for OpenCV and in the process improving documentation and reporting issues.
- Mikael Simberg (Swiss National Supercomputing Centre[6875]), for his tireless help cleaning up and maintaining *HPX*.
- Tianyi Zhang, for his work on HPXMP.
- Shahrzad Shirzad, for her contributions related to Phylanx.
- Christopher Ogle, for his contributions to the parallel algorithms.
- Surya Priy, for his work with statistic performance counters.
- Anushi Maheshwari, for her work on random number generation.
- Bruno Pitrus, for his work with parallel algorithms.
- Nikunj Gupta, for rewriting the implementation of `hpx_main.hpp` and for his fixes for tests.
- Christopher Taylor, for his interest in *HPX* and the fixes he provided. Chris also contributed support for RISC-V architectures.
- Shoshana Jakobovits, for her work on the resource partitioner.
- Denis Blank, who re-wrote our unwrapped function to accept plain values arbitrary containers, and properly deal with nested futures.
- Ajai V. George, who implemented several of the parallel algorithms.
- Taeguk Kwon, who worked on implementing parallel algorithms as well as adapting the parallel algorithms to the Ranges TS.
- Zach Byerly (Louisiana State University (LSU)[6876]), who in his work developing applications on top of *HPX* opened tickets and contributed to the *HPX* examples.
- Daniel Estermann, for his work porting *HPX* to the Raspberry Pi.
- Alireza Kheirkhahan (Louisiana State University (LSU)[6877]), who built and administered our local cluster as well as his work in distributed IO.
- Abhimanyu Rawat, who worked on stack overflow detection.
- David Pfander, who improved signal handling in *HPX*, provided his optimization expertise, and worked on incorporating the Vc vectorization into *HPX*.
- Denis Demidov, who contributed his insights with VexCL.
- Khalid Hasanov, who contributed changes which allowed to run *HPX* on 64Bit power-pc architectures.
- Zahra Khatami (Louisiana State University (LSU)[6878]), who contributed the prefetching iterators and the persistent auto chunking executor parameters implementation.
- Marcin Copik, who worked on implementing GPU support for C++AMP and HCC. He also worked on implementing a HCC backend for *HPX.Compute*.

[6875] https://www.cscs.ch
[6876] https://www.lsu.edu
[6877] https://www.lsu.edu
[6878] https://www.lsu.edu

- Minh-Khanh Do, who contributed the implementation of several segmented algorithms.
- Bibek Wagle (Louisiana State University (LSU)[6879]), who worked on fixing and analyzing the performance of the *parcel* coalescing plugin in *HPX*.
- Lukas Troska, who reported several problems and contributed various test cases allowing to reproduce the corresponding issues.
- Andreas Schaefer, who worked on integrating his library (LibGeoDecomp[6880]) with *HPX*. He reported various problems and submitted several patches to fix issues allowing for a better integration with LibGeoDecomp[6881].
- Satyaki Upadhyay, who contributed several examples to *HPX*.
- Brandon Cordes, who contributed several improvements to the inspect tool.
- Harris Brakmic, who contributed an extensive build system description for building *HPX* with Visual Studio.
- Parsa Amini (Louisiana State University (LSU)[6882]), who refactored and simplified the implementation of *AGAS* in *HPX* and who works on its implementation and optimization.
- Luis Martinez de Bartolome who implemented a build system extension for *HPX* integrating it with the Conan[6883] C/C++ package manager.
- Vinay C Amatya (Louisiana State University (LSU)[6884]), who contributed to the documentation and provided some of the *HPX* examples.
- Kevin Huck and Nick Chaimov (University of Oregon[6885]), who contributed the integration of APEX (Autonomic Performance Environment for eXascale) with *HPX*.
- Francisco Jose Tapia, who helped with implementing the parallel sort algorithm for *HPX*.
- Patrick Diehl, who worked on implementing CUDA support for our companion library targeting GPGPUs (HPXCL[6886]).
- Eric Lemanissier contributed fixes to allow compilation using the MingW toolchain.
- Nidhi Makhijani who helped cleaning up some enum consistencies in *HPX* and contributed to the resource manager used in the thread scheduling subsystem. She also worked on *HPX* in the context of the Google Summer of Code 2015.
- Larry Xiao, Devang Bacharwar, Marcin Copik, and Konstantin Kronfeldner who worked on *HPX* in the context of the Google Summer of Code program 2015.
- Daniel Bourgeois (Center for Computation and Technology (CCT)[6887]) who contributed to *HPX* the implementation of several parallel algorithms (as proposed by N4313[6888]).
- Anuj Sharma and Christopher Bross (Department of Computer Science 3 - Computer Architecture[6889]), who worked on *HPX* in the context of the Google Summer of Code[6890] program 2014.

[6879] https://www.lsu.edu
[6880] https://www.libgeodecomp.org/
[6881] https://www.libgeodecomp.org/
[6882] https://www.lsu.edu
[6883] https://www.conan.io/
[6884] https://www.lsu.edu
[6885] https://uoregon.edu/
[6886] https://github.com/STEllAR-GROUP/hpxcl/
[6887] https://www.cct.lsu.edu
[6888] http://wg21.link/n4313
[6889] https://www3.cs.fau.de
[6890] https://developers.google.com/open-source/soc/

- Martin Stumpf (Department of Computer Science 3 - Computer Architecture[6891]), who rebuilt our contiguous testing infrastructure (see the *HPX* Buildbot Website[6892]). Martin is also working on HPXCL[6893] (mainly all work related to OpenCL[6894]) and implementing an *HPX* backend for POCL[6895], a portable computing language solution based on OpenCL[6896].
- Grant Mercer (University of Nevada, Las Vegas[6897]), who helped creating many of the parallel algorithms (as proposed by N4313[6898]).
- Damond Howard (Louisiana State University (LSU)[6899]), who works on HPXCL[6900] (mainly all work related to CUDA[6901]).
- Christoph Junghans (Los Alamos National Lab), who helped making our buildsystem more portable.
- Antoine Tran Tan (Laboratoire de Recherche en Informatique, Paris), who worked on integrating *HPX* as a backend for NT2[6902]. He also contributed an implementation of an API similar to Fortran co-arrays on top of *HPX*.
- John Biddiscombe (Swiss National Supercomputing Centre[6903]), who helped with the BlueGene/Q port of *HPX*, implemented the parallel sort algorithm, and made several other contributions.
- Erik Schnetter (Perimeter Institute for Theoretical Physics), who greatly helped to make *HPX* more robust by submitting a large amount of problem reports, feature requests, and made several direct contributions.
- Mathias Gaunard (Metascale), who contributed several patches to reduce compile time warnings generated while compiling *HPX*.
- Andreas Buhr, who helped with improving our documentation, especially by suggesting some fixes for inconsistencies.
- Patricia Grubel (New Mexico State University[6904]), who contributed the description of the different *HPX* thread scheduler policies and is working on the performance analysis of our thread scheduling subsystem.
- Lars Viklund, whose wit, passion for testing, and love of odd architectures has been an amazing contribution to our team. He has also contributed platform specific patches for FreeBSD and MSVC12.
- Agustin Berge, who contributed patches fixing some very nasty hidden template metaprogramming issues. He rewrote large parts of the API elements ensuring strict conformance with the C++ ISO Standard.
- Anton Bikineev for contributing changes to make using `boost::lexical_cast` safer, he also contributed a thread safety fix to the iostreams module. He also contributed a

[6891] https://www3.cs.fau.de
[6892] http://rostam.cct.lsu.edu/
[6893] https://github.com/STEllAR-GROUP/hpxcl/
[6894] https://www.khronos.org/opencl/
[6895] https://portablecl.org/
[6896] https://www.khronos.org/opencl/
[6897] https://www.unlv.edu
[6898] http://wg21.link/n4313
[6899] https://www.lsu.edu
[6900] https://github.com/STEllAR-GROUP/hpxcl/
[6901] https://www.nvidia.com/object/cuda_home_new.html
[6902] https://www.numscale.com/nt2/
[6903] https://www.cscs.ch
[6904] https://www.nmsu.edu

complete rewrite of the serialization infrastructure replacing Boost.Serialization inside *HPX*.

- Pyry Jahkola, who contributed the Mac OS build system and build documentation on how to build *HPX* using Clang and libc++.
- Mario Mulansky, who created an *HPX* backend for his Boost.Odeint library, and who submitted several test cases allowing us to reproduce and fix problems in *HPX*.
- Rekha Raj, who contributed changes to the description of the Windows build instructions.
- Jeremy Kemp how worked on an *HPX* OpenMP backend and added regression tests.
- Alex Nagelberg for his work on implementing a C wrapper API for *HPX*.
- Chen Guo, helvihartmann, Nicholas Pezolano, and John West who added and improved examples in *HPX*.
- Joseph Kleinhenz, Markus Elfring, Kirill Kropivyansky, Alexander Neundorf, Bryant Lam, and Alex Hirsch who improved our CMake.
- Tapasweni Pathak, Praveen Velliengiri, Jean-Loup Tastet, Michael Levine, Aalekh Nigam, HadrienG2, Prayag Verma, lslada, Alex Myczko, and Avyav Kumar who improved the documentation.
- Jayesh Badwaik, J. F. Bastien, Christoph Garth, Christopher Hinz, Brandon Kohn, Mario Lang, Maikel Nadolski, pierrele, hendrx, Dekken, woodmeister123, xaguilar, Andrew Kemp, Dylan Stark, Matthew Anderson, Jeremy Wilke, Jiazheng Yuan, CyberDrudge, david8dixon, Maxwell Reeser, Raffaele Solca, Marco Ippolito, Jules Penuchot, Weile Wei, Severin Strobl, Kor de Jong, albestro, Jeff Trull, Yuri Victorovich, and Gregor Daiss who contributed to the general improvement of *HPX*.

HPX Funding Acknowledgements[6905] lists current and past funding sources for *HPX*. Special thanks to Google Summer of Code[6906] and Google Season of Docs[6907] for the continuous support they provide which helps us enhance both our code and our documentation.

[6905] https://hpx.dev/funding-acknowledgements/
[6906] https://developers.google.com/open-source/soc/
[6907] https://developers.google.com/season-of-docs

CHAPTER

THREE

INDEX

- genindex